

TF-M Reference Manual

Generated by Doxygen 1.8.17

1 TF-M Reference Manual	1
2 Deprecated List	3
3 Module Index	5
3.1 Modules	5
4 Data Structure Index	7
4.1 Data Structures	7
5 File Index	11
5.1 File List	11
6 Module Documentation	21
6.1 Implementation-specific definitions	21
6.2 API version	22
6.2.1 Detailed Description	22
6.2.2 Macro Definition Documentation	22
6.2.2.1 PSA_CRYPTO_API_VERSION_MAJOR	22
6.2.2.2 PSA_CRYPTO_API_VERSION_MINOR	22
6.3 Library initialization	23
6.3.1 Detailed Description	23
6.3.2 Function Documentation	23
6.3.2.1 psa_crypto_init()	23
6.4 Key management	24
6.4.1 Detailed Description	24
6.4.2 Function Documentation	24
6.4.2.1 psa_copy_key()	24
6.4.2.2 psa_destroy_key()	25
6.4.2.3 psa_purge_key()	26
6.5 Key import and export	28
6.5.1 Detailed Description	28
6.5.2 Function Documentation	28
6.5.2.1 psa_export_key()	28
6.5.2.2 psa_export_public_key()	29
6.5.2.3 psa_import_key()	31
6.6 Message digests	33
6.6.1 Detailed Description	33
6.6.2 Macro Definition Documentation	33
6.6.2.1 PSA_HASH_OPERATION_INIT	33
6.6.3 Typedef Documentation	33
6.6.3.1 psa_hash_operation_t	34
6.6.4 Function Documentation	34
6.6.4.1 psa_hash_abort()	34

6.6.4.2	psa_hash_clone()	35
6.6.4.3	psa_hash_compare()	36
6.6.4.4	psa_hash_compute()	36
6.6.4.5	psa_hash_finish()	37
6.6.4.6	psa_hash_setup()	38
6.6.4.7	psa_hash_update()	39
6.6.4.8	psa_hash_verify()	40
6.7	Extendable-operation functions (XOF)	42
6.7.1	Detailed Description	42
6.7.2	Macro Definition Documentation	42
6.7.2.1	PSA_XOF_OPERATION_INIT	42
6.7.3	Typedef Documentation	42
6.7.3.1	psa_xof_operation_t	43
6.7.4	Function Documentation	43
6.7.4.1	psa_xof_abort()	43
6.7.4.2	psa_xof_output()	44
6.7.4.3	psa_xof_set_context()	44
6.7.4.4	psa_xof_setup()	45
6.7.4.5	psa_xof_update()	46
6.8	Message authentication codes	47
6.8.1	Detailed Description	47
6.8.2	Macro Definition Documentation	47
6.8.2.1	PSA_MAC_OPERATION_INIT	47
6.8.3	Typedef Documentation	47
6.8.3.1	psa_mac_operation_t	48
6.8.4	Function Documentation	48
6.8.4.1	psa_mac_abort()	48
6.8.4.2	psa_mac_compute()	49
6.8.4.3	psa_mac_sign_finish()	50
6.8.4.4	psa_mac_sign_setup()	51
6.8.4.5	psa_mac_update()	52
6.8.4.6	psa_mac_verify()	53
6.8.4.7	psa_mac_verify_finish()	54
6.8.4.8	psa_mac_verify_setup()	55
6.9	Symmetric ciphers	57
6.9.1	Detailed Description	57
6.9.2	Macro Definition Documentation	57
6.9.2.1	PSA_CIPHER_OPERATION_INIT	57
6.9.3	Typedef Documentation	57
6.9.3.1	psa_cipher_operation_t	58
6.9.4	Function Documentation	58
6.9.4.1	psa_cipher_abort()	58

6.9.4.2	psa_cipher_decrypt()	59
6.9.4.3	psa_cipher_decrypt_setup()	60
6.9.4.4	psa_cipher_encrypt()	61
6.9.4.5	psa_cipher_encrypt_setup()	62
6.9.4.6	psa_cipher_finish()	63
6.9.4.7	psa_cipher_generate_iv()	64
6.9.4.8	psa_cipher_set_iv()	65
6.9.4.9	psa_cipher_update()	66
6.10	Authenticated encryption with associated data (AEAD)	68
6.10.1	Detailed Description	68
6.10.2	Macro Definition Documentation	68
6.10.2.1	PSA_AEAD_OPERATION_INIT	68
6.10.3	Typedef Documentation	69
6.10.3.1	psa_aead_operation_t	69
6.10.4	Function Documentation	69
6.10.4.1	psa_aead_abort()	69
6.10.4.2	psa_aead_decrypt()	70
6.10.4.3	psa_aead_decrypt_setup()	72
6.10.4.4	psa_aead_encrypt()	73
6.10.4.5	psa_aead_encrypt_setup()	75
6.10.4.6	psa_aead_finish()	76
6.10.4.7	psa_aead_generate_nonce()	78
6.10.4.8	psa_aead_set_lengths()	79
6.10.4.9	psa_aead_set_nonce()	80
6.10.4.10	psa_aead_update()	81
6.10.4.11	psa_aead_update_ad()	83
6.10.4.12	psa_aead_verify()	84
6.11	Asymmetric cryptography	87
6.11.1	Detailed Description	87
6.11.2	Function Documentation	87
6.11.2.1	psa_asymmetric_decrypt()	87
6.11.2.2	psa_asymmetric_encrypt()	89
6.11.2.3	psa_sign_hash()	90
6.11.2.4	psa_sign_message()	91
6.11.2.5	psa_unwrap_key()	93
6.11.2.6	psa_verify_hash()	95
6.11.2.7	psa_verify_message()	96
6.12	Key derivation and pseudorandom generation	98
6.12.1	Detailed Description	98
6.12.2	Macro Definition Documentation	98
6.12.2.1	PSA_KEY_DERIVATION_OPERATION_INIT	99
6.12.2.2	PSA_KEY_DERIVATION_UNLIMITED_CAPACITY	99

6.12.3 Typedef Documentation	99
6.12.3.1 <code>psa_key_derivation_operation_t</code>	99
6.12.4 Function Documentation	100
6.12.4.1 <code>psa_key_agreement()</code>	100
6.12.4.2 <code>psa_key_derivation_abort()</code>	101
6.12.4.3 <code>psa_key_derivation_get_capacity()</code>	102
6.12.4.4 <code>psa_key_derivation_input_bytes()</code>	103
6.12.4.5 <code>psa_key_derivation_input_integer()</code>	104
6.12.4.6 <code>psa_key_derivation_input_key()</code>	105
6.12.4.7 <code>psa_key_derivation_key_agreement()</code>	106
6.12.4.8 <code>psa_key_derivation_output_bytes()</code>	107
6.12.4.9 <code>psa_key_derivation_output_key()</code>	108
6.12.4.10 <code>psa_key_derivation_output_key_custom()</code>	110
6.12.4.11 <code>psa_key_derivation_set_capacity()</code>	111
6.12.4.12 <code>psa_key_derivation_setup()</code>	112
6.12.4.13 <code>psa_key_derivation_verify_bytes()</code>	113
6.12.4.14 <code>psa_key_derivation_verify_key()</code>	114
6.12.4.15 <code>psa_raw_key_agreement()</code>	115
6.13 Random generation	117
6.13.1 Detailed Description	117
6.13.2 Function Documentation	117
6.13.2.1 <code>psa_generate_key()</code>	117
6.13.2.2 <code>psa_generate_key_custom()</code>	118
6.13.2.3 <code>psa_generate_random()</code>	119
6.14 Interruptible sign/verify hash	121
6.14.1 Detailed Description	121
6.14.2 Typedef Documentation	121
6.14.2.1 <code>psa_sign_hash_interruptible_operation_t</code>	122
6.14.2.2 <code>psa_verify_hash_interruptible_operation_t</code>	122
6.14.3 Function Documentation	123
6.14.3.1 <code>psa_interruptible_get_max_ops()</code>	123
6.14.3.2 <code>psa_interruptible_set_max_ops()</code>	123
6.14.3.3 <code>psa_sign_hash_abort()</code>	124
6.14.3.4 <code>psa_sign_hash_complete()</code>	125
6.14.3.5 <code>psa_sign_hash_get_num_ops()</code>	126
6.14.3.6 <code>psa_sign_hash_start()</code>	127
6.14.3.7 <code>psa_verify_hash_abort()</code>	128
6.14.3.8 <code>psa_verify_hash_complete()</code>	129
6.14.3.9 <code>psa_verify_hash_get_num_ops()</code>	130
6.14.3.10 <code>psa_verify_hash_start()</code>	131
6.15 Interruptible Key Agreement	133
6.15.1 Detailed Description	133

6.15.2 Typedef Documentation	133
6.15.2.1 psa_key_agreement_iop_t	133
6.15.3 Function Documentation	134
6.15.3.1 psa_key_agreement_iop_abort()	134
6.15.3.2 psa_key_agreement_iop_complete()	134
6.15.3.3 psa_key_agreement_iop_get_num_ops()	136
6.15.3.4 psa_key_agreement_iop_setup()	136
6.16 Interruptible Key Generation	140
6.16.1 Detailed Description	140
6.16.2 Typedef Documentation	140
6.16.2.1 psa_generate_key_iop_t	140
6.16.3 Function Documentation	141
6.16.3.1 psa_generate_key_iop_abort()	141
6.16.3.2 psa_generate_key_iop_complete()	141
6.16.3.3 psa_generate_key_iop_get_num_ops()	143
6.16.3.4 psa_generate_key_iop_setup()	143
6.17 Interruptible public-key export	146
6.17.1 Detailed Description	146
6.17.2 Typedef Documentation	146
6.17.2.1 psa_export_public_key_iop_t	146
6.17.3 Function Documentation	147
6.17.3.1 psa_export_public_key_iop_abort()	147
6.17.3.2 psa_export_public_key_iop_complete()	147
6.17.3.3 psa_export_public_key_iop_get_num_ops()	149
6.17.3.4 psa_export_public_key_iop_setup()	149
6.18 Random and entropy drivers	151
6.18.1 Detailed Description	151
6.18.2 Macro Definition Documentation	151
6.18.2.1 PSA_DRIVER_GET_ENTROPY_FLAGS_NONE	151
6.18.3 Typedef Documentation	151
6.18.3.1 psa_driver_get_entropy_flags_t	151
6.19 Random generator	152
6.19.1 Detailed Description	152
6.19.2 Function Documentation	152
6.19.2.1 psa_random_deplete()	152
6.19.2.2 psa_random_reseed()	152
6.19.2.3 psa_random_set_prediction_resistance()	154
6.20 Built-in keys	155
6.20.1 Detailed Description	155
6.20.2 Macro Definition Documentation	155
6.20.2.1 MBEDTLS_PSA_KEY_ID_BUILTIN_MAX	155
6.20.2.2 MBEDTLS_PSA_KEY_ID_BUILTIN_MIN	155

6.20.3 Typedef Documentation	155
6.20.3.1 psa_drv_slot_number_t	155
6.21 Functions defined by a client provider	156
6.22 Password-authenticated key exchange (PAKE)	157
6.22.1 Detailed Description	158
6.22.2 Macro Definition Documentation	158
6.22.2.1 PSA_PAKE_PRIMITIVE	158
6.22.2.2 PSA_PAKE_PRIMITIVE_TYPE_DH	159
6.22.2.3 PSA_PAKE_PRIMITIVE_TYPE_ECC	159
6.22.2.4 PSA_PAKE_ROLE_CLIENT	160
6.22.2.5 PSA_PAKE_ROLE_FIRST	160
6.22.2.6 PSA_PAKE_ROLE_NONE	160
6.22.2.7 PSA_PAKE_ROLE_SECOND	160
6.22.2.8 PSA_PAKE_ROLE_SERVER	161
6.22.2.9 PSA_PAKE_STEP_CONFIRM	161
6.22.2.10 PSA_PAKE_STEP_KEY_SHARE	161
6.22.2.11 PSA_PAKE_STEP_ZK_PROOF	162
6.22.2.12 PSA_PAKE_STEP_ZK_PUBLIC	162
6.22.3 Typedef Documentation	162
6.22.3.1 psa_crypto_driver_pake_inputs_t	163
6.22.3.2 psa_jpake_computation_stage_t	163
6.22.3.3 psa_pake_cipher_suite_t	163
6.22.3.4 psa_pake_family_t	163
6.22.3.5 psa_pake_operation_t	164
6.22.3.6 psa_pake_primitive_t	164
6.22.3.7 psa_pake_primitive_type_t	164
6.22.3.8 psa_pake_role_t	165
6.22.3.9 psa_pake_step_t	165
6.22.4 Function Documentation	165
6.22.4.1 psa_crypto_driver_pake_get_cipher_suite()	165
6.22.4.2 psa_crypto_driver_pake_get_password()	166
6.22.4.3 psa_crypto_driver_pake_get_password_len()	166
6.22.4.4 psa_crypto_driver_pake_get_peer()	166
6.22.4.5 psa_crypto_driver_pake_get_peer_len()	167
6.22.4.6 psa_crypto_driver_pake_get_user()	167
6.22.4.7 psa_crypto_driver_pake_get_user_len()	168
6.22.4.8 psa_pake_abort()	168
6.22.4.9 psa_pake_get_shared_key()	169
6.22.4.10 psa_pake_input()	171
6.22.4.11 psa_pake_output()	172
6.22.4.12 psa_pake_set_context()	173
6.22.4.13 psa_pake_set_peer()	174

6.22.4.14	psa_pake_set_role()	175
6.22.4.15	psa_pake_set_user()	176
6.22.4.16	psa_pake_setup()	176
6.23	Error codes	179
6.23.1	Detailed Description	179
6.23.2	Macro Definition Documentation	179
6.23.2.1	PSA_ERROR_ALREADY_EXISTS	179
6.23.2.2	PSA_ERROR_BAD_STATE	180
6.23.2.3	PSA_ERROR_BUFFER_TOO_SMALL	180
6.23.2.4	PSA_ERROR_COMMUNICATION_FAILURE	180
6.23.2.5	PSA_ERROR_CORRUPTION_DETECTED	181
6.23.2.6	PSA_ERROR_DATA_CORRUPT	181
6.23.2.7	PSA_ERROR_DATA_INVALID	182
6.23.2.8	PSA_ERROR_DOES_NOT_EXIST	182
6.23.2.9	PSA_ERROR_GENERIC_ERROR	182
6.23.2.10	PSA_ERROR_HARDWARE_FAILURE	182
6.23.2.11	PSA_ERROR_INSUFFICIENT_DATA	183
6.23.2.12	PSA_ERROR_INSUFFICIENT_ENTROPY	183
6.23.2.13	PSA_ERROR_INSUFFICIENT_MEMORY	183
6.23.2.14	PSA_ERROR_INSUFFICIENT_STORAGE	183
6.23.2.15	PSA_ERROR_INVALID_ARGUMENT	184
6.23.2.16	PSA_ERROR_INVALID_HANDLE	184
6.23.2.17	PSA_ERROR_INVALID_PADDING	184
6.23.2.18	PSA_ERROR_INVALID_SIGNATURE	185
6.23.2.19	PSA_ERROR_NOT_PERMITTED	185
6.23.2.20	PSA_ERROR_NOT_SUPPORTED	185
6.23.2.21	PSA_ERROR_SERVICE_FAILURE	185
6.23.2.22	PSA_ERROR_STORAGE_FAILURE	186
6.23.2.23	PSA_OPERATION_INCOMPLETE	186
6.23.2.24	PSA_SUCCESS	186
6.23.3	Typedef Documentation	186
6.23.3.1	psa_status_t	186
6.24	Key and algorithm types	187
6.24.1	Detailed Description	192
6.24.2	Macro Definition Documentation	192
6.24.2.1	PSA_AEAD_TAG_LENGTH_OFFSET	192
6.24.2.2	PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG	193
6.24.2.3	PSA_ALG_AEAD_FROM_BLOCK_FLAG	193
6.24.2.4	PSA_ALG_AEAD_GET_TAG_LENGTH	193
6.24.2.5	PSA_ALG_AEAD_TAG_LENGTH_MASK	193
6.24.2.6	PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG	194
6.24.2.7	PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG	194

6.24.2.8 PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG_CASE	195
6.24.2.9 PSA_ALG_AEAD_WITH_SHORTENED_TAG	195
6.24.2.10 PSA_ALG_AES_KW	196
6.24.2.11 PSA_ALG_AES_KWP	196
6.24.2.12 PSA_ALG_ANY_HASH	197
6.24.2.13 PSA_ALG_AT_LEAST_THIS_LENGTH_MAC	197
6.24.2.14 PSA_ALG_CATEGORY_AEAD	198
6.24.2.15 PSA_ALG_CATEGORY_ASYMMETRIC_ENCRYPTION	198
6.24.2.16 PSA_ALG_CATEGORY_CIPHER	198
6.24.2.17 PSA_ALG_CATEGORY_HASH	198
6.24.2.18 PSA_ALG_CATEGORY_KEY_AGREEMENT	199
6.24.2.19 PSA_ALG_CATEGORY_KEY_DERIVATION	199
6.24.2.20 PSA_ALG_CATEGORY_KEY_WRAP	199
6.24.2.21 PSA_ALG_CATEGORY_MAC	199
6.24.2.22 PSA_ALG_CATEGORY_MASK	199
6.24.2.23 PSA_ALG_CATEGORY_PAKE	199
6.24.2.24 PSA_ALG_CATEGORY_SIGN	200
6.24.2.25 PSA_ALG_CATEGORY_XOF	200
6.24.2.26 PSA_ALG_CBC_MAC	200
6.24.2.27 PSA_ALG_CBC_NO_PADDING	200
6.24.2.28 PSA_ALG_CBC_PKCS7	201
6.24.2.29 PSA_ALG_CCM	201
6.24.2.30 PSA_ALG_CCM_STAR_NO_TAG	201
6.24.2.31 PSA_ALG_CFB	201
6.24.2.32 PSA_ALG_CHACHA20_POLY1305	202
6.24.2.33 PSA_ALG_CIPHER_FROM_BLOCK_FLAG	202
6.24.2.34 PSA_ALG_CIPHER_MAC_BASE	202
6.24.2.35 PSA_ALG_CIPHER_STREAM_FLAG	202
6.24.2.36 PSA_ALG_CMAC	202
6.24.2.37 PSA_ALG_CTR	203
6.24.2.38 PSA_ALG_DETERMINISTIC_DSA	203
6.24.2.39 PSA_ALG_DETERMINISTIC_DSA_BASE	203
6.24.2.40 PSA_ALG_DETERMINISTIC_ECDSA	204
6.24.2.41 PSA_ALG_DETERMINISTIC_ECDSA_BASE	205
6.24.2.42 PSA_ALG_DSA	205
6.24.2.43 PSA_ALG_DSA_BASE	205
6.24.2.44 PSA_ALG_DSA_DETERMINISTIC_FLAG	206
6.24.2.45 PSA_ALG_DSA_IS_DETERMINISTIC	206
6.24.2.46 PSA_ALG_ECB_NO_PADDING	206
6.24.2.47 PSA_ALG_ECDH	207
6.24.2.48 PSA_ALG_ECDSA	207
6.24.2.49 PSA_ALG_ECDSA_ANY	208

6.24.2.50 PSA_ALG_ECDSA_BASE	208
6.24.2.51 PSA_ALG_ECDSA_DETERMINISTIC_FLAG	208
6.24.2.52 PSA_ALG_ECDSA_IS_DETERMINISTIC	208
6.24.2.53 PSA_ALG_ED25519PH	208
6.24.2.54 PSA_ALG_ED448PH	209
6.24.2.55 PSA_ALG_FFDH	209
6.24.2.56 PSA_ALG_FULL_LENGTH_MAC	209
6.24.2.57 PSA_ALG_GCM	210
6.24.2.58 PSA_ALG_GET_HASH	210
6.24.2.59 PSA_ALG_HASH_EDDSA_BASE	210
6.24.2.60 PSA_ALG_HASH_MASK	211
6.24.2.61 PSA_ALG_HKDF	211
6.24.2.62 PSA_ALG_HKDF_BASE	211
6.24.2.63 PSA_ALG_HKDF_EXPAND	212
6.24.2.64 PSA_ALG_HKDF_EXPAND_BASE	212
6.24.2.65 PSA_ALG_HKDF_EXTRACT	213
6.24.2.66 PSA_ALG_HKDF_EXTRACT_BASE	213
6.24.2.67 PSA_ALG_HKDF_GET_HASH	214
6.24.2.68 PSA_ALG_HMAC	214
6.24.2.69 PSA_ALG_HMAC_BASE	214
6.24.2.70 PSA_ALG_HMAC_GET_HASH	214
6.24.2.71 PSA_ALG_IS_AEAD	214
6.24.2.72 PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER	215
6.24.2.73 PSA_ALG_IS_ANY_HKDF	215
6.24.2.74 PSA_ALG_IS_ASYMMETRIC_ENCRYPTION	216
6.24.2.75 PSA_ALG_IS_BLOCK_CIPHER_MAC	216
6.24.2.76 PSA_ALG_IS_CIPHER	217
6.24.2.77 PSA_ALG_IS_DETERMINISTIC_DSA	217
6.24.2.78 PSA_ALG_IS_DETERMINISTIC_ECDSA	217
6.24.2.79 PSA_ALG_IS_DSA	217
6.24.2.80 PSA_ALG_IS_ECDH	217
6.24.2.81 PSA_ALG_IS_ECDSA	218
6.24.2.82 PSA_ALG_IS_FFDH	218
6.24.2.83 PSA_ALG_IS_HASH	219
6.24.2.84 PSA_ALG_IS_HASH_AND_SIGN	219
6.24.2.85 PSA_ALG_IS_HASH_EDDSA	220
6.24.2.86 PSA_ALG_IS_HKDF	220
6.24.2.87 PSA_ALG_IS_HKDF_EXPAND	220
6.24.2.88 PSA_ALG_IS_HKDF_EXTRACT	221
6.24.2.89 PSA_ALG_IS_HMAC	221
6.24.2.90 PSA_ALG_IS_JPAKE	222
6.24.2.91 PSA_ALG_IS_KEY_AGREEMENT	222

6.24.2.92 PSA_ALG_IS_KEY_DERIVATION	223
6.24.2.93 PSA_ALG_IS_KEY_DERIVATION_OR_AGREEMENT	223
6.24.2.94 PSA_ALG_IS_KEY_DERIVATION_STRETCHING	223
6.24.2.95 PSA_ALG_IS_KEY_WRAP	224
6.24.2.96 PSA_ALG_IS_MAC	224
6.24.2.97 PSA_ALG_IS_PAKE	225
6.24.2.98 PSA_ALG_IS_PBKDF2	225
6.24.2.99 PSA_ALG_IS_PBKDF2_HMAC	225
6.24.2.100 PSA_ALG_IS_RANDOMIZED_DSA	226
6.24.2.101 PSA_ALG_IS_RANDOMIZED_ECDSA	226
6.24.2.102 PSA_ALG_IS_RAW_KEY_AGREEMENT	226
6.24.2.103 PSA_ALG_IS_RSA_OAEP	226
6.24.2.104 PSA_ALG_IS_RSA_PKCS1V15_SIGN	227
6.24.2.105 PSA_ALG_IS_RSA_PSS	227
6.24.2.106 PSA_ALG_IS_RSA_PSS_ANY_SALT	227
6.24.2.107 PSA_ALG_IS_RSA_PSS_STANDARD_SALT	228
6.24.2.108 PSA_ALG_IS_SIGN	228
6.24.2.109 PSA_ALG_IS_SIGN_HASH	229
6.24.2.110 PSA_ALG_IS_SIGN_MESSAGE	229
6.24.2.111 PSA_ALG_IS_SP800_108_COUNTER_CMAC	230
6.24.2.112 PSA_ALG_IS_SPAKE2P	230
6.24.2.113 PSA_ALG_IS_SPAKE2P_CMAC	231
6.24.2.114 PSA_ALG_IS_SPAKE2P_HMAC	231
6.24.2.115 PSA_ALG_IS_STREAM_CIPHER	231
6.24.2.116 PSA_ALG_IS_TLS12_PRF	231
6.24.2.117 PSA_ALG_IS_TLS12_PSK_TO_MS	232
6.24.2.118 PSA_ALG_IS_VENDOR_DEFINED	232
6.24.2.119 PSA_ALG_IS_VENDOR_HASH_AND_SIGN [1/2]	233
6.24.2.120 PSA_ALG_IS_VENDOR_HASH_AND_SIGN [2/2]	233
6.24.2.121 PSA_ALG_IS_WILDCARD	233
6.24.2.122 PSA_ALG_IS_XOF	234
6.24.2.123 PSA_ALG_JPAKE	235
6.24.2.124 PSA_ALG_JPAKE_BASE	237
6.24.2.125 PSA_ALG_KEY_AGREEMENT	237
6.24.2.126 PSA_ALG_KEY_AGREEMENT_GET_BASE	237
6.24.2.127 PSA_ALG_KEY_AGREEMENT_GET_KDF	237
6.24.2.128 PSA_ALG_KEY_AGREEMENT_MASK	238
6.24.2.129 PSA_ALG_KEY_DERIVATION_MASK	238
6.24.2.130 PSA_ALG_KEY_DERIVATION_STRETCHING_FLAG	238
6.24.2.131 PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG	238
6.24.2.132 PSA_ALG_MAC_SUBCATEGORY_MASK	238
6.24.2.133 PSA_ALG_MAC_TRUNCATION_MASK	238

6.24.2.134 PSA_ALG_MD5	239
6.24.2.135 PSA_ALG_NONE	239
6.24.2.136 PSA_ALG_OFB	239
6.24.2.137 PSA_ALG_PBKDF2_AES_CMAC_PRF_128	239
6.24.2.138 PSA_ALG_PBKDF2_HMAC	239
6.24.2.139 PSA_ALG_PBKDF2_HMAC_BASE	240
6.24.2.140 PSA_ALG_PBKDF2_HMAC_GET_HASH	240
6.24.2.141 PSA_ALG_PURE_EDDSA	240
6.24.2.142 PSA_ALG_RIPEMD160	241
6.24.2.143 PSA_ALG_RSA_OAEP	241
6.24.2.144 PSA_ALG_RSA_OAEP_BASE	241
6.24.2.145 PSA_ALG_RSA_OAEP_GET_HASH	241
6.24.2.146 PSA_ALG_RSA_PKCS1V15_CRYPT	242
6.24.2.147 PSA_ALG_RSA_PKCS1V15_SIGN	242
6.24.2.148 PSA_ALG_RSA_PKCS1V15_SIGN_BASE	242
6.24.2.149 PSA_ALG_RSA_PKCS1V15_SIGN_RAW	243
6.24.2.150 PSA_ALG_RSA_PSS	243
6.24.2.151 PSA_ALG_RSA_PSS_ANY_SALT	243
6.24.2.152 PSA_ALG_RSA_PSS_ANY_SALT_BASE	244
6.24.2.153 PSA_ALG_RSA_PSS_BASE	244
6.24.2.154 PSA_ALG_SHA3_224	244
6.24.2.155 PSA_ALG_SHA3_256	244
6.24.2.156 PSA_ALG_SHA3_384	245
6.24.2.157 PSA_ALG_SHA3_512	245
6.24.2.158 PSA_ALG_SHA_1	245
6.24.2.159 PSA_ALG_SHA_224	245
6.24.2.160 PSA_ALG_SHA_256	245
6.24.2.161 PSA_ALG_SHA_384	246
6.24.2.162 PSA_ALG_SHA_512	246
6.24.2.163 PSA_ALG_SHA_512_224	246
6.24.2.164 PSA_ALG_SHA_512_256	246
6.24.2.165 PSA_ALG_SHAKE128	246
6.24.2.166 PSA_ALG_SHAKE256	247
6.24.2.167 PSA_ALG_SHAKE256_512	247
6.24.2.168 PSA_ALG_SIGN_GET_HASH	247
6.24.2.169 PSA_ALG_SP800_108_COUNTER_CMAC	248
6.24.2.170 PSA_ALG_SPAKE2P_CMAC	248
6.24.2.171 PSA_ALG_SPAKE2P_CMAC_BASE	248
6.24.2.172 PSA_ALG_SPAKE2P_HMAC	248
6.24.2.173 PSA_ALG_SPAKE2P_HMAC_BASE	248
6.24.2.174 PSA_ALG_SPAKE2P_MATTER	249
6.24.2.175 PSA_ALG_STREAM_CIPHER	249

6.24.2.176 PSA_ALG_TLS12_ECJPAKE_TO_PMS	249
6.24.2.177 PSA_ALG_TLS12_PRF	249
6.24.2.178 PSA_ALG_TLS12_PRF_BASE	250
6.24.2.179 PSA_ALG_TLS12_PRF_GET_HASH	250
6.24.2.180 PSA_ALG_TLS12_PSK_TO_MS	250
6.24.2.181 PSA_ALG_TLS12_PSK_TO_MS_BASE	251
6.24.2.182 PSA_ALG_TLS12_PSK_TO_MS_GET_HASH	251
6.24.2.183 PSA_ALG_TRUNCATED_MAC	252
6.24.2.184 PSA_ALG_VENDOR_FLAG	252
6.24.2.185 PSA_ALG_XOF_CONTEXT_FLAG	253
6.24.2.186 PSA_ALG_XOF_HAS_CONTEXT	253
6.24.2.187 PSA_ALG_XTS	253
6.24.2.188 PSA_BLOCK_CIPHER_BLOCK_LENGTH	253
6.24.2.189 PSA_DH_FAMILY_RFC7919	254
6.24.2.190 PSA_ECC_FAMILY_BRAINPOOL_P_R1	254
6.24.2.191 PSA_ECC_FAMILY_IS_WEIERSTRASS	255
6.24.2.192 PSA_ECC_FAMILY_MONTGOMERY	255
6.24.2.193 PSA_ECC_FAMILY_SECP_K1	255
6.24.2.194 PSA_ECC_FAMILY_SECP_R1	255
6.24.2.195 PSA_ECC_FAMILY_SECP_R2	256
6.24.2.196 PSA_ECC_FAMILY_SECT_K1	256
6.24.2.197 PSA_ECC_FAMILY_SECT_R1	256
6.24.2.198 PSA_ECC_FAMILY_SECT_R2	257
6.24.2.199 PSA_ECC_FAMILY_TWISTED_EDWARDS	257
6.24.2.200 PSA_GET_KEY_TYPE_BLOCK_SIZE_EXPONENT	257
6.24.2.201 PSA_KEY_TYPE_AES	258
6.24.2.202 PSA_KEY_TYPE_ARIA	258
6.24.2.203 PSA_KEY_TYPE_CAMELLIA	258
6.24.2.204 PSA_KEY_TYPE_CATEGORY_FLAG_PAIR	258
6.24.2.205 PSA_KEY_TYPE_CATEGORY_KEY_PAIR	258
6.24.2.206 PSA_KEY_TYPE_CATEGORY_MASK	259
6.24.2.207 PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY	259
6.24.2.208 PSA_KEY_TYPE_CATEGORY_RAW	259
6.24.2.209 PSA_KEY_TYPE_CATEGORY_SYMMETRIC	259
6.24.2.210 PSA_KEY_TYPE_CHACHA20	259
6.24.2.211 PSA_KEY_TYPE_DERIVE	260
6.24.2.212 PSA_KEY_TYPE_DH_GET_FAMILY	260
6.24.2.213 PSA_KEY_TYPE_DH_GROUP_MASK	260
6.24.2.214 PSA_KEY_TYPE_DH_KEY_PAIR	260
6.24.2.215 PSA_KEY_TYPE_DH_KEY_PAIR_BASE	261
6.24.2.216 PSA_KEY_TYPE_DH_PUBLIC_KEY	261
6.24.2.217 PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE	261

6.24.2.218 PSA_KEY_TYPE_DSA_KEY_PAIR	261
6.24.2.219 PSA_KEY_TYPE_DSA_PUBLIC_KEY	262
6.24.2.220 PSA_KEY_TYPE_ECC_CURVE_MASK	262
6.24.2.221 PSA_KEY_TYPE_ECC_GET_FAMILY	262
6.24.2.222 PSA_KEY_TYPE_ECC_KEY_PAIR	262
6.24.2.223 PSA_KEY_TYPE_ECC_KEY_PAIR_BASE	263
6.24.2.224 PSA_KEY_TYPE_ECC_PUBLIC_KEY	263
6.24.2.225 PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE	263
6.24.2.226 PSA_KEY_TYPE_HAS_ECC_FAMILY	263
6.24.2.227 PSA_KEY_TYPE_HMAC	264
6.24.2.228 PSA_KEY_TYPE_IS_ASYMMETRIC	264
6.24.2.229 PSA_KEY_TYPE_IS_DH	264
6.24.2.230 PSA_KEY_TYPE_IS_DH_KEY_PAIR	264
6.24.2.231 PSA_KEY_TYPE_IS_DH_PUBLIC_KEY	265
6.24.2.232 PSA_KEY_TYPE_IS_DSA	265
6.24.2.233 PSA_KEY_TYPE_IS_ECC	265
6.24.2.234 PSA_KEY_TYPE_IS_ECC_KEY_PAIR	265
6.24.2.235 PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY	266
6.24.2.236 PSA_KEY_TYPE_IS_KEY_PAIR	266
6.24.2.237 PSA_KEY_TYPE_IS_PUBLIC_KEY	266
6.24.2.238 PSA_KEY_TYPE_IS_RSA	266
6.24.2.239 PSA_KEY_TYPE_IS_SPAKE2P	267
6.24.2.240 PSA_KEY_TYPE_IS_SPAKE2P_KEY_PAIR	267
6.24.2.241 PSA_KEY_TYPE_IS_SPAKE2P_PUBLIC_KEY	267
6.24.2.242 PSA_KEY_TYPE_IS_UNSTRUCTURED	267
6.24.2.243 PSA_KEY_TYPE_IS_VENDOR_DEFINED	268
6.24.2.244 PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY	268
6.24.2.245 PSA_KEY_TYPE_NONE	268
6.24.2.246 PSA_KEY_TYPE_PASSWORD	269
6.24.2.247 PSA_KEY_TYPE_PASSWORD_HASH	269
6.24.2.248 PSA_KEY_TYPE_PEPPER	269
6.24.2.249 PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR	269
6.24.2.250 PSA_KEY_TYPE_RAW_DATA	270
6.24.2.251 PSA_KEY_TYPE_RSA_KEY_PAIR	270
6.24.2.252 PSA_KEY_TYPE_RSA_PUBLIC_KEY	270
6.24.2.253 PSA_KEY_TYPE_SPAKE2P_KEY_PAIR	271
6.24.2.254 PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE	271
6.24.2.255 PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY	271
6.24.2.256 PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE	271
6.24.2.257 PSA_KEY_TYPE_VENDOR_FLAG	271
6.24.2.258 PSA_MAC_TRUNCATED_LENGTH	271
6.24.2.259 PSA_MAC_TRUNCATION_OFFSET	272

6.24.3 Typedef Documentation	272
6.24.3.1 psa_algorithm_t	272
6.24.3.2 psa_dh_family_t	273
6.24.3.3 psa_ecc_family_t	273
6.24.3.4 psa_key_type_t	273
6.25 Key lifetimes	274
6.25.1 Detailed Description	274
6.25.2 Macro Definition Documentation	274
6.25.2.1 MBEDTLS_SVC_KEY_ID_GET_KEY_ID	274
6.25.2.2 MBEDTLS_SVC_KEY_ID_GET_OWNER_ID	275
6.25.2.3 MBEDTLS_SVC_KEY_ID_INIT	275
6.25.2.4 PSA_KEY_ID_NULL	275
6.25.2.5 PSA_KEY_ID_USER_MAX	275
6.25.2.6 PSA_KEY_ID_USER_MIN	275
6.25.2.7 PSA_KEY_ID_VENDOR_MAX	276
6.25.2.8 PSA_KEY_ID_VENDOR_MIN	276
6.25.2.9 PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION	276
6.25.2.10 PSA_KEY_LIFETIME_GET_LOCATION	276
6.25.2.11 PSA_KEY_LIFETIME_GET_PERSISTENCE	277
6.25.2.12 PSA_KEY_LIFETIME_IS_READ_ONLY	277
6.25.2.13 PSA_KEY_LIFETIME_IS_VOLATILE	277
6.25.2.14 PSA_KEY_LIFETIME_PERSISTENT	278
6.25.2.15 PSA_KEY_LIFETIME_VOLATILE	278
6.25.2.16 PSA_KEY_LOCATION_LOCAL_STORAGE	279
6.25.2.17 PSA_KEY_LOCATION_VENDOR_FLAG	279
6.25.2.18 PSA_KEY_PERSISTENCE_DEFAULT	279
6.25.2.19 PSA_KEY_PERSISTENCE_READ_ONLY	279
6.25.2.20 PSA_KEY_PERSISTENCE_VOLATILE	280
6.25.3 Typedef Documentation	280
6.25.3.1 mbedtls_svc_key_id_t	280
6.25.3.2 psa_key_id_t	280
6.25.3.3 psa_key_lifetime_t	281
6.25.3.4 psa_key_location_t	281
6.25.3.5 psa_key_persistence_t	282
6.26 Key policies	283
6.26.1 Detailed Description	283
6.26.2 Macro Definition Documentation	283
6.26.2.1 PSA_KEY_USAGE_COPY	283
6.26.2.2 PSA_KEY_USAGE_DECRYPT	284
6.26.2.3 PSA_KEY_USAGE_DERIVE	284
6.26.2.4 PSA_KEY_USAGE_DERIVE_PUBLIC	284
6.26.2.5 PSA_KEY_USAGE_ENCRYPT	285

6.26.2.6	PSA_KEY_USAGE_EXPORT	285
6.26.2.7	PSA_KEY_USAGE_SIGN_HASH	285
6.26.2.8	PSA_KEY_USAGE_SIGN_MESSAGE	285
6.26.2.9	PSA_KEY_USAGE_UNWRAP	286
6.26.2.10	PSA_KEY_USAGE_VERIFY_DERIVATION	286
6.26.2.11	PSA_KEY_USAGE_VERIFY_HASH	286
6.26.2.12	PSA_KEY_USAGE_VERIFY_MESSAGE	287
6.26.2.13	PSA_KEY_USAGE_WRAP	287
6.26.3	Typedef Documentation	287
6.26.3.1	psa_key_usage_t	287
6.27	Key attributes	288
6.27.1	Detailed Description	288
6.27.2	Macro Definition Documentation	288
6.27.2.1	PSA_KEY_ATTRIBUTES_INIT	288
6.27.2.2	PSA_PAKE_OPERATION_STAGE_COLLECT_INPUTS	288
6.27.2.3	PSA_PAKE_OPERATION_STAGE_COMPUTATION	289
6.27.2.4	PSA_PAKE_OPERATION_STAGE_SETUP	289
6.27.3	Typedef Documentation	289
6.27.3.1	psa_key_attributes_t	289
6.27.4	Function Documentation	291
6.27.4.1	psa_get_key_attributes()	291
6.27.4.2	psa_reset_key_attributes()	292
6.28	Key derivation	293
6.28.1	Detailed Description	293
6.28.2	Macro Definition Documentation	293
6.28.2.1	PSA_KEY_DERIVATION_INPUT_CONTEXT	293
6.28.2.2	PSA_KEY_DERIVATION_INPUT_COST	293
6.28.2.3	PSA_KEY_DERIVATION_INPUT_INFO	294
6.28.2.4	PSA_KEY_DERIVATION_INPUT_LABEL	294
6.28.2.5	PSA_KEY_DERIVATION_INPUT_OTHER_SECRET	294
6.28.2.6	PSA_KEY_DERIVATION_INPUT_PASSWORD	294
6.28.2.7	PSA_KEY_DERIVATION_INPUT_SALT	295
6.28.2.8	PSA_KEY_DERIVATION_INPUT_SECRET	295
6.28.2.9	PSA_KEY_DERIVATION_INPUT_SEED	295
6.28.3	Typedef Documentation	295
6.28.3.1	psa_custom_key_parameters_t	296
6.28.3.2	psa_key_derivation_step_t	296
6.29	Helper macros	297
6.29.1	Detailed Description	297
6.29.2	Macro Definition Documentation	297
6.29.2.1	MBEDTLS_PSA_ALG_AEAD_EQUAL	297
6.30	Interruptible operations	298

6.30.1 Detailed Description	298
6.30.2 Macro Definition Documentation	298
6.30.2.1 PSA_INTERRUPTIBLE_MAX_OPS_UNLIMITED	298
6.31 Set of functions implementing a thin shim	299
6.31.1 Detailed Description	299
6.31.2 Function Documentation	299
6.31.2.1 tfm_crypto_aead_interface()	299
6.31.2.2 tfm_crypto_asymmetric_encrypt_interface()	300
6.31.2.3 tfm_crypto_asymmetric_sign_interface()	300
6.31.2.4 tfm_crypto_cipher_interface()	301
6.31.2.5 tfm_crypto_hash_interface()	301
6.31.2.6 tfm_crypto_key_derivation_interface()	302
6.31.2.7 tfm_crypto_key_management_interface()	302
6.31.2.8 tfm_crypto_mac_interface()	302
6.31.2.9 tfm_crypto_random_interface()	303
6.32 Function that implement allocation and deallocation of	304
6.32.1 Detailed Description	304
6.32.2 Function Documentation	304
6.32.2.1 tfm_crypto_init_alloc()	304
6.32.2.2 tfm_crypto_operation_alloc()	305
6.32.2.3 tfm_crypto_operation_lookup()	305
6.32.2.4 tfm_crypto_operation_release()	305
6.33 This group implements the partition initialisation	307
6.33.1 Detailed Description	307
6.33.2 Function Documentation	307
6.33.2.1 tfm_crypto_init()	307
6.33.2.2 tfm_crypto_sfn()	307
6.34 Set of functions implementing the abstractions of the underlying cryptographic	308
6.34.1 Detailed Description	308
6.34.2 Function Documentation	308
6.34.2.1 mbedtls_psa_platform_get_builtin_key()	308
6.34.2.2 tfm_crypto_core_library_init()	309
6.34.2.3 tfm_crypto_library_get_info()	310
6.34.2.4 tfm_crypto_library_get_library_key_id_set_owner()	310
6.34.2.5 tfm_crypto_library_key_id_init()	310
6.35 Tfm_builtin_key_loader	312
6.35.1 Detailed Description	312
6.35.2 Function Documentation	312
6.35.2.1 tfm_builtin_key_loader_get_builtin_key()	312
6.35.2.2 tfm_builtin_key_loader_get_key_buffer_size()	313
6.35.2.3 tfm_builtin_key_loader_init()	313
6.36 Asn1_module	314

6.36.1 Detailed Description	315
6.36.2 Macro Definition Documentation	315
6.36.2.1 MBEDTLS_ASN1_BIT_STRING	316
6.36.2.2 MBEDTLS_ASN1_BMP_STRING	316
6.36.2.3 MBEDTLS_ASN1_BOOLEAN	316
6.36.2.4 MBEDTLS_ASN1_CONSTRUCTED	316
6.36.2.5 MBEDTLS_ASN1_CONTEXT_SPECIFIC	316
6.36.2.6 MBEDTLS_ASN1_ENUMERATED	316
6.36.2.7 MBEDTLS_ASN1_GENERALIZED_TIME	317
6.36.2.8 MBEDTLS_ASN1_IA5_STRING	317
6.36.2.9 MBEDTLS_ASN1_INTEGER	317
6.36.2.10 MBEDTLS_ASN1_IS_STRING_TAG	317
6.36.2.11 MBEDTLS_ASN1_NULL	317
6.36.2.12 MBEDTLS_ASN1_OCTET_STRING	318
6.36.2.13 MBEDTLS_ASN1_OID	318
6.36.2.14 MBEDTLS_ASN1_PRIMITIVE	318
6.36.2.15 MBEDTLS_ASN1_PRINTABLE_STRING	318
6.36.2.16 MBEDTLS_ASN1_SEQUENCE	318
6.36.2.17 MBEDTLS_ASN1_SET	318
6.36.2.18 MBEDTLS_ASN1_T61_STRING	319
6.36.2.19 MBEDTLS_ASN1_TAG_CLASS_MASK	319
6.36.2.20 MBEDTLS_ASN1_TAG_PC_MASK	319
6.36.2.21 MBEDTLS_ASN1_TAG_VALUE_MASK	319
6.36.2.22 MBEDTLS_ASN1_UNIVERSAL_STRING	319
6.36.2.23 MBEDTLS_ASN1_UTC_TIME	319
6.36.2.24 MBEDTLS_ASN1_UTF8_STRING	320
6.36.2.25 MBEDTLS_ERR_ASN1_INVALID_DATA	320
6.36.2.26 MBEDTLS_ERR_ASN1_INVALID_LENGTH	320
6.36.2.27 MBEDTLS_ERR_ASN1_LENGTH_MISMATCH	320
6.36.2.28 MBEDTLS_ERR_ASN1_OUT_OF_DATA	320
6.36.2.29 MBEDTLS_ERR_ASN1_UNEXPECTED_TAG	321
6.36.2.30 MBEDTLS_OID_CMP	321
6.36.2.31 MBEDTLS_OID_CMP_RAW	321
6.36.2.32 MBEDTLS_OID_SIZE	321
6.36.3 Typedef Documentation	322
6.36.3.1 mbedtls_asn1_bitstring	322
6.36.3.2 mbedtls_asn1_buf	322
6.36.3.3 mbedtls_asn1_named_data	322
6.36.3.4 mbedtls_asn1_sequence	322
6.36.4 Function Documentation	322
6.36.4.1 MBEDTLS_PRIVATE()	322
6.36.5 Variable Documentation	322

6.36.5.1 buf	323
6.36.5.2 len [1/2]	323
6.36.5.3 len [2/2]	323
6.36.5.4 next [1/2]	323
6.36.5.5 next [2/2]	323
6.36.5.6 oid	324
6.36.5.7 p [1/2]	324
6.36.5.8 p [2/2]	324
6.36.5.9 tag	324
6.36.5.10 unused_bits	324
6.36.5.11 val	324
7 Data Structure Documentation	325
7.1 _MTB_SRF_DATA_ALIGN Struct Reference	325
7.1.1 Detailed Description	326
7.1.2 Field Documentation	326
7.1.2.1 call_type	326
7.1.2.2 params	326
7.1.2.3 reply	326
7.1.2.4 semaphore_idx	326
7.2 asset_desc_t Struct Reference	327
7.2.1 Detailed Description	327
7.2.2 Field Documentation	327
7.2.2.1 "@15	327
7.2.2.2 attr	327
7.2.2.3 dev	328
7.2.2.4 dev_ref	328
7.2.2.5 limit	328
7.2.2.6 mem	328
7.2.2.7 reserved	328
7.2.2.8 start	328
7.3 attest_boot_data Struct Reference	329
7.3.1 Detailed Description	329
7.3.2 Field Documentation	329
7.3.2.1 data	329
7.3.2.2 header	330
7.4 attest_token_encode_ctx Struct Reference	330
7.4.1 Detailed Description	330
7.4.2 Field Documentation	330
7.4.2.1 cbor_enc_ctx	330
7.4.2.2 key_select	331
7.4.2.3 signer_ctx	331

7.5 bi_list_node_t Struct Reference	331
7.5.1 Detailed Description	331
7.5.2 Field Documentation	331
7.5.2.1 bnext	332
7.5.2.2 bprev	332
7.6 boot_data_access_policy Struct Reference	332
7.6.1 Detailed Description	332
7.6.2 Field Documentation	332
7.6.2.1 major_type	332
7.6.2.2 partition_id	333
7.7 client_id_region_t Struct Reference	333
7.7.1 Detailed Description	333
7.7.2 Field Documentation	333
7.7.2.1 base	333
7.7.2.2 limit	333
7.8 client_params_t Struct Reference	334
7.8.1 Detailed Description	334
7.8.2 Field Documentation	334
7.8.2.1 ns_client_id_stateless	334
7.8.2.2 p_invecs	334
7.8.2.3 p_outvecs	335
7.9 composite_addr_t Union Reference	335
7.9.1 Detailed Description	335
7.9.2 Field Documentation	335
7.9.2.1 p_byte	335
7.9.2.2 p_word	335
7.9.2.3 uint_addr	336
7.10 connection_t Struct Reference	336
7.10.1 Detailed Description	337
7.10.2 Field Documentation	337
7.10.2.1 caller_outvec	337
7.10.2.2 invec_accessed	337
7.10.2.3 invec_base	337
7.10.2.4 msg	337
7.10.2.5 outvec_base	337
7.10.2.6 outvec_written	338
7.10.2.7 p_client	338
7.10.2.8 service	338
7.10.2.9 status	338
7.11 context_ctrl_t Struct Reference	338
7.11.1 Detailed Description	339
7.11.2 Field Documentation	339

7.11.2.1 exc_ret	339
7.11.2.2 sp	339
7.11.2.3 sp_base	339
7.11.2.4 sp_limit	339
7.12 context_flih_ret_t Struct Reference	340
7.12.1 Detailed Description	340
7.12.2 Field Documentation	340
7.12.2.1 addi_ctx	340
7.12.2.2 dummy	341
7.12.2.3 exc_return	341
7.12.2.4 psp	341
7.12.2.5 psplim	341
7.12.2.6 stack_seal	341
7.12.2.7 state_ctx	341
7.13 critical_section_t Struct Reference	342
7.13.1 Detailed Description	342
7.13.2 Field Documentation	342
7.13.2.1 state	342
7.14 full_context_t Struct Reference	342
7.14.1 Detailed Description	343
7.14.2 Field Documentation	343
7.14.2.1 addi_ctx	343
7.14.2.2 stat_ctx	343
7.15 fwu_image_info_data_s Struct Reference	343
7.15.1 Detailed Description	344
7.15.2 Field Documentation	344
7.15.2.1 data	344
7.15.2.2 header	344
7.16 ifx_driver_flash_obj_t Struct Reference	344
7.16.1 Detailed Description	345
7.16.2 Field Documentation	345
7.16.2.1 flash_info	345
7.16.2.2 start_address	345
7.17 ifx_driver_rram_obj_t Struct Reference	345
7.17.1 Detailed Description	346
7.17.2 Field Documentation	346
7.17.2.1 flash_info	346
7.17.2.2 rram_base	346
7.17.2.3 start_address	347
7.18 ifx_driver_smif_mem_t Struct Reference	347
7.18.1 Detailed Description	347
7.18.2 Field Documentation	347

7.18.2.1 block_config	347
7.18.2.2 mem_config	348
7.18.2.3 mem_config_dual_quad_pair	348
7.18.2.4 smif_base	348
7.18.2.5 smif_context	348
7.19 ifx_driver_smif_obj_t Struct Reference	349
7.19.1 Detailed Description	349
7.19.2 Field Documentation	349
7.19.2.1 flash_info	350
7.19.2.2 memory	350
7.19.2.3 offset	350
7.19.2.4 size	350
7.20 ifx_flash_driver_instance_t Struct Reference	351
7.20.1 Detailed Description	351
7.20.2 Field Documentation	351
7.20.2.1 driver	351
7.20.2.2 handler	352
7.21 ifx_flash_driver_t Struct Reference	352
7.21.1 Detailed Description	352
7.21.2 Field Documentation	352
7.21.2.1 EraseChip	353
7.21.2.2 EraseSector	353
7.21.2.3 GetCapabilities	353
7.21.2.4 GetInfo	353
7.21.2.5 GetStatus	353
7.21.2.6 GetVersion	353
7.21.2.7 Initialize	354
7.21.2.8 PowerControl	354
7.21.2.9 ProgramData	354
7.21.2.10 ReadData	354
7.21.2.11 Uninitialize	354
7.22 ifx_mpc_cfg_t Struct Reference	354
7.22.1 Detailed Description	355
7.22.2 Field Documentation	355
7.22.2.1 attr	355
7.22.2.2 mem_offset	355
7.22.2.3 mem_size	355
7.23 ifx_mpc_ext_cache_cfg_info_t Struct Reference	355
7.23.1 Detailed Description	355
7.23.2 Field Documentation	356
7.23.2.1 attr	356
7.23.2.2 first_block_idx	356

7.23.2.3 last_block_idx	356
7.23.2.4 pc_cache_mask	356
7.24 ifx_mpc_numbered_mmio_config_t Struct Reference	357
7.24.1 Detailed Description	357
7.24.2 Field Documentation	357
7.24.2.1 ns_mask	357
7.24.2.2 pc_mask	357
7.25 ifx_mpc_policy_t Struct Reference	358
7.25.1 Detailed Description	358
7.25.2 Field Documentation	358
7.25.2.1 mpc_struct	358
7.25.2.2 n_flash_mpc	358
7.25.2.3 n_ram_mpc	359
7.25.2.4 unused	359
7.26 ifx_mpc_raw_region_config_t Struct Reference	359
7.26.1 Detailed Description	359
7.26.2 Field Documentation	359
7.26.2.1 mpc_base	360
7.26.2.2 mpc_block_size	360
7.26.2.3 ns_mask	360
7.26.2.4 offset	360
7.26.2.5 pc_apply_mask	360
7.26.2.6 r_mask	361
7.26.2.7 size	361
7.26.2.8 w_mask	361
7.27 ifx_mpc_region_config_t Struct Reference	361
7.27.1 Detailed Description	361
7.27.2 Field Documentation	362
7.27.2.1 address	362
7.27.2.2 ns_mask	362
7.27.2.3 r_mask	362
7.27.2.4 size	362
7.27.2.5 w_mask	362
7.28 ifx_mpc_sert_mem_t Struct Reference	363
7.28.1 Detailed Description	363
7.28.2 Field Documentation	363
7.28.2.1 attr_cache	363
7.28.2.2 block_count	363
7.28.2.3 mem	364
7.29 ifx_mtb_mailbox_reply_t Struct Reference	364
7.29.1 Detailed Description	364
7.29.2 Field Documentation	364

7.29.2.1 out_vec_len	364
7.29.2.2 return_val	364
7.30 ifx_nv_counters Struct Reference	365
7.30.1 Detailed Description	365
7.30.2 Field Documentation	365
7.30.2.1 bytes	365
7.30.2.2 checksum	365
7.30.2.3 counters	366
7.30.2.4 init_value	366
7.30.2.5 nv_cnt	366
7.31 ifx_nv_init_done Union Reference	366
7.31.1 Detailed Description	366
7.31.2 Field Documentation	366
7.31.2.1 block	366
7.31.2.2 flag	367
7.32 ifx_original_iovec_t Struct Reference	367
7.32.1 Detailed Description	367
7.32.2 Field Documentation	367
7.32.2.1 invec	368
7.32.2.2 invec_count	368
7.32.2.3 outvec	368
7.32.2.4 outvec_count	368
7.33 ifx_partition_info_t Struct Reference	368
7.33.1 Detailed Description	369
7.33.2 Field Documentation	369
7.33.2.1 asset_cnt	369
7.33.2.2 ifx_domain	369
7.33.2.3 ifx_ldinfo	369
7.33.2.4 p_assets	370
7.33.2.5 p_ldinfo	370
7.33.2.6 p_peri_regions	370
7.33.2.7 peri_region_count	370
7.34 ifx_partition_load_info_t Struct Reference	371
7.34.1 Detailed Description	371
7.34.2 Field Documentation	371
7.34.2.1 privileged	371
7.34.2.2 protect_cfg_enabled	372
7.35 ifx_ppc_named_mmio_config_t Struct Reference	372
7.35.1 Detailed Description	372
7.35.2 Field Documentation	372
7.35.2.1 allow_unpriv	372
7.35.2.2 pc_mask	373

7.35.2.3 sec_attr	373
7.36 ifx_ppc_regions_config_t Struct Reference	373
7.36.1 Detailed Description	373
7.36.2 Field Documentation	373
7.36.2.1 region_count	374
7.36.2.2 regions	374
7.37 ifx_ppcx_config_t Struct Reference	374
7.37.1 Detailed Description	374
7.37.2 Field Documentation	374
7.37.2.1 allow_unpriv	375
7.37.2.2 pc_mask	375
7.37.2.3 region_count	375
7.37.2.4 regions	375
7.37.2.5 sec_attr	375
7.38 ifx_protection_region_config_t Struct Reference	376
7.38.1 Detailed Description	376
7.38.2 Field Documentation	376
7.38.2.1 mpu_regions_enable	376
7.39 ifx_psa_client_params_t Struct Reference	376
7.39.1 Detailed Description	377
7.39.2 Field Documentation	377
7.39.2.1 "@19	377
7.39.2.2 handle	377
7.39.2.3 in_len	377
7.39.2.4 in_vec	378
7.39.2.5 out_len	378
7.39.2.6 out_vec	378
7.39.2.7 psa_call_params	378
7.39.2.8 psa_close_params	378
7.39.2.9 psa_connect_params	378
7.39.2.10 psa_version_params	378
7.39.2.11 sid	379
7.39.2.12 type	379
7.39.2.13 version	379
7.40 ifx_rram_counter_t Struct Reference	379
7.40.1 Detailed Description	379
7.40.2 Field Documentation	379
7.40.2.1 bytes	380
7.40.2.2 checksum	380
7.40.2.3 data	380
7.40.2.4 value	380
7.41 ifx_sau_config_t Struct Reference	380

7.41.1 Detailed Description	380
7.41.2 Field Documentation	381
7.41.2.1 base_addr	381
7.41.2.2 nsc	381
7.41.2.3 size	381
7.42 ifx_timer_irq_test_dev_config Struct Reference	381
7.42.1 Detailed Description	381
7.42.2 Field Documentation	382
7.42.2.1 tcpwm_base	382
7.42.2.2 tcpwm_config	382
7.42.2.3 tcpwm_counter_num	382
7.43 irq_load_info_t Struct Reference	382
7.43.1 Detailed Description	383
7.43.2 Field Documentation	383
7.43.2.1 client_id_base	383
7.43.2.2 client_id_limit	383
7.43.2.3 flih_func	383
7.43.2.4 init	383
7.43.2.5 mbox_flags	383
7.43.2.6 pid	384
7.43.2.7 signal	384
7.43.2.8 source	384
7.44 irq_t Struct Reference	384
7.44.1 Detailed Description	385
7.44.2 Field Documentation	385
7.44.2.1 p_ildi	385
7.44.2.2 p_pt	385
7.45 its_block_meta_t Struct Reference	385
7.45.1 Detailed Description	386
7.45.2 Field Documentation	386
7.45.2.1 data_start	386
7.45.2.2 free_size	386
7.45.2.3 phy_id	386
7.46 its_file_meta_t Struct Reference	387
7.46.1 Detailed Description	387
7.46.2 Field Documentation	387
7.46.2.1 cur_size	387
7.46.2.2 data_idx	387
7.46.2.3 flags	388
7.46.2.4 id	388
7.46.2.5 lblock	388
7.46.2.6 max_size	388

7.47 its_flash_fs_config_t Struct Reference	388
7.47.1 Detailed Description	389
7.47.2 Field Documentation	389
7.47.2.1 block_size	389
7.47.2.2 erase_val	389
7.47.2.3 flash_area_addr	389
7.47.2.4 flash_dev	389
7.47.2.5 max_file_size	390
7.47.2.6 max_num_files	390
7.47.2.7 num_blocks	390
7.47.2.8 program_unit	390
7.47.2.9 sector_size	390
7.48 its_flash_fs_ctx_t Struct Reference	391
7.48.1 Detailed Description	391
7.48.2 Field Documentation	391
7.48.2.1 active_metablock	391
7.48.2.2 cfg	392
7.48.2.3 meta_block_header	392
7.48.2.4 ops	392
7.48.2.5 scratch_metablock	392
7.49 its_flash_fs_file_info_t Struct Reference	392
7.49.1 Detailed Description	393
7.49.2 Field Documentation	393
7.49.2.1 flags	393
7.49.2.2 size_current	393
7.49.2.3 size_max	393
7.50 its_flash_fs_ops_t Struct Reference	394
7.50.1 Detailed Description	394
7.50.2 Field Documentation	394
7.50.2.1 erase	394
7.50.2.2 flush	395
7.50.2.3 init	395
7.50.2.4 read	396
7.50.2.5 write	396
7.51 its_flash_nand_dev_t Struct Reference	397
7.51.1 Detailed Description	397
7.51.2 Field Documentation	397
7.51.2.1 buf_block_id_0	397
7.51.2.2 buf_block_id_1	398
7.51.2.3 buf_size	398
7.51.2.4 driver	398
7.51.2.5 write_buf_0	398

7.51.2.6 write_buf_1	398
7.52 its_metadata_block_header_comp_t Struct Reference	398
7.52.1 Detailed Description	399
7.52.2 Field Documentation	399
7.52.2.1 active_swap_count	399
7.52.2.2 fs_version	399
7.52.2.3 scratch_dblock	400
7.53 its_metadata_block_header_t Struct Reference	400
7.53.1 Detailed Description	400
7.53.2 Field Documentation	400
7.53.2.1 active_swap_count	401
7.53.2.2 fs_version	401
7.53.2.3 metadata_xor	401
7.53.2.4 scratch_dblock	401
7.54 mailbox_init_t Struct Reference	402
7.54.1 Detailed Description	402
7.54.2 Member Function Documentation	402
7.54.2.1 __ALIGNED()	402
7.54.3 Field Documentation	402
7.54.3.1 slot_count	403
7.54.3.2 slots	403
7.55 mailbox_msg_t Struct Reference	403
7.55.1 Detailed Description	404
7.55.2 Field Documentation	404
7.55.2.1 call_type	404
7.55.2.2 client_id	404
7.55.2.3 params	404
7.56 mailbox_reply_t Struct Reference	404
7.56.1 Detailed Description	405
7.56.2 Field Documentation	405
7.56.2.1 out_vec_len	405
7.56.2.2 return_val	405
7.57 mailbox_slot_t Struct Reference	405
7.57.1 Detailed Description	405
7.57.2 Member Function Documentation	405
7.57.2.1 __ALIGNED() [1/2]	406
7.57.2.2 __ALIGNED() [2/2]	406
7.58 mailbox_status_t Struct Reference	406
7.58.1 Detailed Description	406
7.58.2 Field Documentation	406
7.58.2.1 pend_slots	406
7.58.2.2 replied_slots	407

7.59 mbedtls_asn1_bitstring Struct Reference	407
7.59.1 Detailed Description	407
7.60 mbedtls_asn1_buf Struct Reference	407
7.60.1 Detailed Description	408
7.61 mbedtls_asn1_named_data Struct Reference	408
7.61.1 Detailed Description	408
7.62 mbedtls_asn1_sequence Struct Reference	409
7.62.1 Detailed Description	409
7.63 mbedtls_lmots_parameters_t Struct Reference	409
7.63.1 Detailed Description	410
7.63.2 Member Function Documentation	410
7.63.2.1 MBEDTLS_PRIVATE() [1/3]	410
7.63.2.2 MBEDTLS_PRIVATE() [2/3]	410
7.63.2.3 MBEDTLS_PRIVATE() [3/3]	410
7.64 mbedtls_lmots_public_t Struct Reference	410
7.64.1 Detailed Description	411
7.64.2 Member Function Documentation	411
7.64.2.1 MBEDTLS_PRIVATE() [1/3]	411
7.64.2.2 MBEDTLS_PRIVATE() [2/3]	411
7.64.2.3 MBEDTLS_PRIVATE() [3/3]	412
7.65 mbedtls_lms_parameters_t Struct Reference	412
7.65.1 Detailed Description	412
7.65.2 Member Function Documentation	412
7.65.2.1 MBEDTLS_PRIVATE() [1/3]	412
7.65.2.2 MBEDTLS_PRIVATE() [2/3]	412
7.65.2.3 MBEDTLS_PRIVATE() [3/3]	413
7.66 mbedtls_lms_public_t Struct Reference	413
7.66.1 Detailed Description	413
7.66.2 Member Function Documentation	414
7.66.2.1 MBEDTLS_PRIVATE() [1/3]	414
7.66.2.2 MBEDTLS_PRIVATE() [2/3]	414
7.66.2.3 MBEDTLS_PRIVATE() [3/3]	414
7.67 mbedtls_md_context_t Struct Reference	414
7.67.1 Detailed Description	414
7.67.2 Member Function Documentation	415
7.67.2.1 MBEDTLS_PRIVATE() [1/2]	415
7.67.2.2 MBEDTLS_PRIVATE() [2/2]	415
7.68 mbedtls_pk_context Struct Reference	415
7.68.1 Detailed Description	415
7.68.2 Member Function Documentation	415
7.68.2.1 MBEDTLS_PRIVATE() [1/6]	416
7.68.2.2 MBEDTLS_PRIVATE() [2/6]	416

7.68.2.3 MBEDTLS_PRIVATE() [3/6]	416
7.68.2.4 MBEDTLS_PRIVATE() [4/6]	416
7.68.2.5 MBEDTLS_PRIVATE() [5/6]	416
7.68.2.6 MBEDTLS_PRIVATE() [6/6]	416
7.69 mbedtls_platform_context Struct Reference	417
7.69.1 Detailed Description	417
7.69.2 Member Function Documentation	417
7.69.2.1 MBEDTLS_PRIVATE() [1/2]	417
7.69.2.2 MBEDTLS_PRIVATE() [2/2]	417
7.70 mbedtls_psa_stats_s Struct Reference	418
7.70.1 Detailed Description	418
7.70.2 Member Function Documentation	418
7.70.2.1 MBEDTLS_PRIVATE() [1/9]	418
7.70.2.2 MBEDTLS_PRIVATE() [2/9]	418
7.70.2.3 MBEDTLS_PRIVATE() [3/9]	419
7.70.2.4 MBEDTLS_PRIVATE() [4/9]	419
7.70.2.5 MBEDTLS_PRIVATE() [5/9]	419
7.70.2.6 MBEDTLS_PRIVATE() [6/9]	419
7.70.2.7 MBEDTLS_PRIVATE() [7/9]	419
7.70.2.8 MBEDTLS_PRIVATE() [8/9]	419
7.70.2.9 MBEDTLS_PRIVATE() [9/9]	419
7.71 mpu_armv8m_region_cfg_t Struct Reference	420
7.71.1 Detailed Description	420
7.71.2 Field Documentation	420
7.71.2.1 attr_access	420
7.71.2.2 attr_exec	420
7.71.2.3 attr_sh	420
7.71.2.4 region_attridx	421
7.71.2.5 region_base	421
7.71.2.6 region_limit	421
7.72 ns_mailbox_queue_t Struct Reference	421
7.72.1 Detailed Description	422
7.72.2 Member Function Documentation	422
7.72.2.1 __ALIGNED() [1/2]	422
7.72.2.2 __ALIGNED() [2/2]	422
7.72.3 Field Documentation	422
7.72.3.1 empty_slots	422
7.72.3.2 is_full	423
7.72.3.3 slots	423
7.73 ns_mailbox_req_t Struct Reference	423
7.73.1 Detailed Description	424
7.73.2 Field Documentation	424

7.73.2.1 call_type	424
7.73.2.2 client_id	424
7.73.2.3 owner	424
7.73.2.4 params_ptr	424
7.73.2.5 reply	424
7.73.2.6 woken_flag	425
7.74 ns_mailbox_slot_t Struct Reference	425
7.74.1 Detailed Description	425
7.74.2 Field Documentation	425
7.74.2.1 is_woken	426
7.74.2.2 owner	426
7.74.2.3 reply	426
7.75 ns_mailbox_stats_res_t Struct Reference	426
7.75.1 Detailed Description	426
7.75.2 Field Documentation	426
7.75.2.1 avg_nr_slots	427
7.75.2.2 avg_nr_slots_tenths	427
7.76 partition_head_t Struct Reference	427
7.76.1 Detailed Description	427
7.76.2 Field Documentation	428
7.76.2.1 next	428
7.76.2.2 reserved	428
7.77 partition_load_info_t Struct Reference	428
7.77.1 Detailed Description	428
7.77.2 Field Documentation	429
7.77.2.1 client_id_base	429
7.77.2.2 client_id_limit	429
7.77.2.3 entry	429
7.77.2.4 flags	429
7.77.2.5 heap_size	429
7.77.2.6 load_order	430
7.77.2.7 nassets	430
7.77.2.8 ndeps	430
7.77.2.9 nirqs	430
7.77.2.10 nservices	430
7.77.2.11 pid	430
7.77.2.12 psa_ff_ver	431
7.77.2.13 stack_size	431
7.78 partition_t Struct Reference	431
7.78.1 Detailed Description	432
7.78.2 Field Documentation	432
7.78.2.1 boundary	432

7.78.2.2 next	432
7.78.2.3 p_ldinf	432
7.78.2.4 p_reqs	432
7.78.2.5 signals_allowed	432
7.78.2.6 signals_asserted	433
7.78.2.7 signals_waiting	433
7.78.2.8 state	433
7.79 partition_tfm_sp_idle_load_info_t Struct Reference	433
7.79.1 Detailed Description	434
7.79.2 Field Documentation	434
7.79.2.1 heap_addr	434
7.79.2.2 load_info	434
7.79.2.3 stack_addr	434
7.80 partition_tfm_sp_ns_agent_tz_load_info_t Struct Reference	434
7.80.1 Detailed Description	435
7.80.2 Field Documentation	435
7.80.2.1 heap_addr	435
7.80.2.2 load_info	435
7.80.2.3 stack_addr	435
7.81 ps_crypto_t Union Reference	435
7.81.1 Detailed Description	436
7.81.2 Field Documentation	436
7.81.2.1 client_id	436
7.81.2.2 iv	436
7.81.2.3 ref	436
7.81.2.4 tag	437
7.81.2.5 uid	437
7.82 ps_obj_header_t Struct Reference	437
7.82.1 Detailed Description	438
7.82.2 Field Documentation	438
7.82.2.1 fid	438
7.82.2.2 info	438
7.82.2.3 version	438
7.83 ps_obj_table_ctx_t Struct Reference	439
7.83.1 Detailed Description	439
7.83.2 Field Documentation	439
7.83.2.1 active_table	439
7.83.2.2 obj_table	440
7.83.2.3 scratch_table	440
7.84 ps_obj_table_entry_t Struct Reference	440
7.84.1 Detailed Description	440
7.84.2 Field Documentation	440

7.84.2.1 client_id	440
7.84.2.2 uid	441
7.84.2.3 version	441
7.85 ps_obj_table_info_t Struct Reference	441
7.85.1 Detailed Description	441
7.85.2 Field Documentation	441
7.85.2.1 fid	442
7.85.2.2 version	442
7.86 ps_obj_table_init_ctx_t Struct Reference	442
7.86.1 Detailed Description	443
7.86.2 Field Documentation	443
7.86.2.1 p_table	443
7.86.2.2 table_state	443
7.87 ps_obj_table_t Struct Reference	443
7.87.1 Detailed Description	444
7.87.2 Field Documentation	444
7.87.2.1 obj_db	444
7.87.2.2 swap_count	444
7.87.2.3 version	444
7.88 ps_object_info_t Struct Reference	445
7.88.1 Detailed Description	445
7.88.2 Field Documentation	445
7.88.2.1 create_flags	445
7.88.2.2 current_size	445
7.88.2.3 max_size	446
7.89 ps_object_t Struct Reference	446
7.89.1 Detailed Description	446
7.89.2 Field Documentation	447
7.89.2.1 data	447
7.89.2.2 header	447
7.90 psa_aead_operation_s Struct Reference	447
7.90.1 Detailed Description	447
7.90.2 Member Function Documentation	448
7.90.2.1 MBEDTLS_PRIVATE() [1/6]	448
7.90.2.2 MBEDTLS_PRIVATE() [2/6]	448
7.90.2.3 MBEDTLS_PRIVATE() [3/6]	448
7.90.2.4 MBEDTLS_PRIVATE() [4/6]	448
7.90.2.5 MBEDTLS_PRIVATE() [5/6]	448
7.90.2.6 MBEDTLS_PRIVATE() [6/6]	449
7.91 psa_cipher_operation_s Struct Reference	449
7.91.1 Detailed Description	449
7.91.2 Member Function Documentation	449

7.91.2.1 MBEDTLS_PRIVATE() [1/3]	449
7.91.2.2 MBEDTLS_PRIVATE() [2/3]	449
7.91.2.3 MBEDTLS_PRIVATE() [3/3]	450
7.92 psa_client_params_t Struct Reference	450
7.92.1 Detailed Description	451
7.92.2 Field Documentation	451
7.92.2.1 "@1	451
7.92.2.2 handle	451
7.92.2.3 in_len	451
7.92.2.4 in_vec	451
7.92.2.5 out_len	451
7.92.2.6 out_vec	452
7.92.2.7 psa_call_params	452
7.92.2.8 psa_close_params	452
7.92.2.9 psa_connect_params	452
7.92.2.10 psa_version_params	452
7.92.2.11 sid	452
7.92.2.12 type	452
7.92.2.13 version	453
7.93 psa_crypto_driver_pake_inputs_s Struct Reference	453
7.93.1 Detailed Description	453
7.93.2 Member Function Documentation	453
7.93.2.1 MBEDTLS_PRIVATE() [1/8]	453
7.93.2.2 MBEDTLS_PRIVATE() [2/8]	453
7.93.2.3 MBEDTLS_PRIVATE() [3/8]	454
7.93.2.4 MBEDTLS_PRIVATE() [4/8]	454
7.93.2.5 MBEDTLS_PRIVATE() [5/8]	454
7.93.2.6 MBEDTLS_PRIVATE() [6/8]	454
7.93.2.7 MBEDTLS_PRIVATE() [7/8]	454
7.93.2.8 MBEDTLS_PRIVATE() [8/8]	454
7.94 psa_custom_key_parameters_s Struct Reference	454
7.94.1 Detailed Description	455
7.94.2 Field Documentation	455
7.94.2.1 flags	455
7.95 psa_driver_aead_context_t Union Reference	455
7.95.1 Detailed Description	455
7.95.2 Field Documentation	455
7.95.2.1 dummy	456
7.95.2.2 mbedtls_ctx	456
7.96 psa_driver_cipher_context_t Union Reference	456
7.96.1 Detailed Description	456
7.96.2 Field Documentation	456

7.96.2.1 dummy	456
7.96.2.2 mbedtls_ctx	457
7.97 psa_driver_hash_context_t Union Reference	457
7.97.1 Detailed Description	457
7.97.2 Field Documentation	457
7.97.2.1 dummy	457
7.97.2.2 mbedtls_ctx	457
7.98 psa_driver_key_derivation_context_t Union Reference	458
7.98.1 Detailed Description	458
7.98.2 Field Documentation	458
7.98.2.1 dummy	458
7.99 psa_driver_mac_context_t Union Reference	458
7.99.1 Detailed Description	458
7.99.2 Field Documentation	458
7.99.2.1 dummy	459
7.99.2.2 mbedtls_ctx	459
7.100 psa_driver_pake_context_t Union Reference	459
7.100.1 Detailed Description	459
7.100.2 Field Documentation	459
7.100.2.1 dummy	459
7.100.2.2 mbedtls_ctx	460
7.101 psa_driver_sign_hash_interruptible_context_t Union Reference	460
7.101.1 Detailed Description	460
7.101.2 Field Documentation	460
7.101.2.1 dummy	460
7.101.2.2 mbedtls_ctx	460
7.102 psa_driver_verify_hash_interruptible_context_t Union Reference	461
7.102.1 Detailed Description	461
7.102.2 Field Documentation	461
7.102.2.1 dummy	461
7.102.2.2 mbedtls_ctx	461
7.103 psa_driver_xof_context_t Union Reference	461
7.103.1 Detailed Description	462
7.103.2 Field Documentation	462
7.103.2.1 dummy	462
7.103.2.2 mbedtls_ctx	462
7.104 psa_export_public_key_iop_s Struct Reference	462
7.104.1 Detailed Description	462
7.104.2 Member Function Documentation	463
7.104.2.1 MBEDTLS_PRIVATE() [1/3]	463
7.104.2.2 MBEDTLS_PRIVATE() [2/3]	463
7.104.2.3 MBEDTLS_PRIVATE() [3/3]	463

7.105 psa_fwu_component_info_t Struct Reference	463
7.105.1 Detailed Description	464
7.105.2 Field Documentation	464
7.105.2.1 error	464
7.105.2.2 flags	464
7.105.2.3 impl	464
7.105.2.4 location	465
7.105.2.5 max_size	465
7.105.2.6 state	465
7.105.2.7 version	465
7.106 psa_fwu_image_version_t Struct Reference	465
7.106.1 Detailed Description	466
7.106.2 Field Documentation	466
7.106.2.1 build	466
7.106.2.2 major	466
7.106.2.3 minor	466
7.106.2.4 patch	467
7.107 psa_fwu_impl_info_t Struct Reference	467
7.107.1 Detailed Description	467
7.107.2 Field Documentation	467
7.107.2.1 candidate_digest	467
7.108 psa_generate_key_iop_s Struct Reference	468
7.108.1 Detailed Description	468
7.108.2 Member Function Documentation	468
7.108.2.1 MBEDTLS_PRIVATE() [1/4]	468
7.108.2.2 MBEDTLS_PRIVATE() [2/4]	468
7.108.2.3 MBEDTLS_PRIVATE() [3/4]	468
7.108.2.4 MBEDTLS_PRIVATE() [4/4]	469
7.109 psa_hash_operation_s Struct Reference	469
7.109.1 Detailed Description	469
7.109.2 Member Function Documentation	469
7.109.2.1 MBEDTLS_PRIVATE() [1/2]	469
7.109.2.2 MBEDTLS_PRIVATE() [2/2]	469
7.110 psa_invec Struct Reference	470
7.110.1 Detailed Description	470
7.110.2 Field Documentation	470
7.110.2.1 base	470
7.110.2.2 len	470
7.111 psa_jpake_computation_stage_s Struct Reference	470
7.111.1 Detailed Description	471
7.111.2 Member Function Documentation	471
7.111.2.1 MBEDTLS_PRIVATE() [1/5]	471

7.111.2.2 MBEDTLS_PRIVATE() [2/5]	471
7.111.2.3 MBEDTLS_PRIVATE() [3/5]	471
7.111.2.4 MBEDTLS_PRIVATE() [4/5]	471
7.111.2.5 MBEDTLS_PRIVATE() [5/5]	472
7.112 psa_key_agreement_iop_s Struct Reference	472
7.112.1 Detailed Description	472
7.112.2 Member Function Documentation	472
7.112.2.1 MBEDTLS_PRIVATE() [1/4]	472
7.112.2.2 MBEDTLS_PRIVATE() [2/4]	472
7.112.2.3 MBEDTLS_PRIVATE() [3/4]	473
7.112.2.4 MBEDTLS_PRIVATE() [4/4]	473
7.113 psa_key_attributes_s Struct Reference	473
7.113.1 Detailed Description	473
7.113.2 Member Function Documentation	473
7.113.2.1 MBEDTLS_PRIVATE() [1/5]	473
7.113.2.2 MBEDTLS_PRIVATE() [2/5]	474
7.113.2.3 MBEDTLS_PRIVATE() [3/5]	474
7.113.2.4 MBEDTLS_PRIVATE() [4/5]	474
7.113.2.5 MBEDTLS_PRIVATE() [5/5]	474
7.114 psa_key_derivation_s Struct Reference	474
7.114.1 Detailed Description	474
7.114.2 Member Function Documentation	475
7.114.2.1 MBEDTLS_PRIVATE() [1/3]	475
7.114.2.2 MBEDTLS_PRIVATE() [2/3]	475
7.114.2.3 MBEDTLS_PRIVATE() [3/3]	475
7.115 psa_key_policy_s Struct Reference	475
7.115.1 Detailed Description	475
7.115.2 Member Function Documentation	475
7.115.2.1 MBEDTLS_PRIVATE() [1/3]	476
7.115.2.2 MBEDTLS_PRIVATE() [2/3]	476
7.115.2.3 MBEDTLS_PRIVATE() [3/3]	476
7.116 psa_mac_operation_s Struct Reference	476
7.116.1 Detailed Description	476
7.116.2 Member Function Documentation	476
7.116.2.1 MBEDTLS_PRIVATE() [1/3]	477
7.116.2.2 MBEDTLS_PRIVATE() [2/3]	477
7.116.2.3 MBEDTLS_PRIVATE() [3/3]	477
7.117 psa_msg_t Struct Reference	477
7.117.1 Detailed Description	477
7.117.2 Field Documentation	478
7.117.2.1 client_id	478
7.117.2.2 handle	478

7.117.2.3 in_size	478
7.117.2.4 out_size	478
7.117.2.5 rhandle	478
7.117.2.6 type	479
7.118 psa_outvec Struct Reference	479
7.118.1 Detailed Description	479
7.118.2 Field Documentation	479
7.118.2.1 base	479
7.118.2.2 len	479
7.119 psa_pake_cipher_suite_s Struct Reference	480
7.119.1 Detailed Description	480
7.119.2 Field Documentation	480
7.119.2.1 algorithm	480
7.119.2.2 bits	480
7.119.2.3 family	480
7.119.2.4 key_confirmation	481
7.119.2.5 type	481
7.120 psa_pake_operation_s Struct Reference	481
7.120.1 Detailed Description	481
7.120.2 Member Function Documentation	481
7.120.2.1 MBEDTLS_PRIVATE() [1/6]	482
7.120.2.2 MBEDTLS_PRIVATE() [2/6]	482
7.120.2.3 MBEDTLS_PRIVATE() [3/6]	482
7.120.2.4 MBEDTLS_PRIVATE() [4/6]	482
7.120.2.5 MBEDTLS_PRIVATE() [5/6]	482
7.120.2.6 MBEDTLS_PRIVATE() [6/6]	482
7.121 psa_sign_hash_interruptible_operation_s Struct Reference	483
7.121.1 Detailed Description	483
7.121.2 Member Function Documentation	483
7.121.2.1 MBEDTLS_PRIVATE() [1/3]	483
7.121.2.2 MBEDTLS_PRIVATE() [2/3]	483
7.121.2.3 MBEDTLS_PRIVATE() [3/3]	483
7.122 psa_storage_info_t Struct Reference	484
7.122.1 Detailed Description	484
7.122.2 Field Documentation	484
7.122.2.1 capacity	484
7.122.2.2 flags	484
7.122.2.3 size	484
7.123 psa_verify_hash_interruptible_operation_s Struct Reference	485
7.123.1 Detailed Description	485
7.123.2 Member Function Documentation	485
7.123.2.1 MBEDTLS_PRIVATE() [1/3]	485

7.123.2.2 MBEDTLS_PRIVATE() [2/3]	485
7.123.2.3 MBEDTLS_PRIVATE() [3/3]	485
7.124 psa_xof_operation_s Struct Reference	486
7.124.1 Detailed Description	486
7.124.2 Member Function Documentation	486
7.124.2.1 MBEDTLS_PRIVATE() [1/2]	486
7.124.2.2 MBEDTLS_PRIVATE() [2/2]	486
7.124.3 Field Documentation	486
7.124.3.1 active	487
7.124.3.2 allows_context	487
7.124.3.3 has_context	487
7.124.3.4 has_input	487
7.124.3.5 has_output	487
7.124.3.6 requires_context	487
7.125 rot_psa_its_storage_info_t Struct Reference	488
7.125.1 Detailed Description	488
7.125.2 Field Documentation	488
7.125.2.1 capacity	488
7.125.2.2 flags	488
7.125.2.3 size	488
7.126 rot_psa_ps_storage_info_t Struct Reference	489
7.126.1 Detailed Description	489
7.126.2 Field Documentation	489
7.126.2.1 capacity	489
7.126.2.2 flags	489
7.126.2.3 size	489
7.127 service_head_t Struct Reference	490
7.127.1 Detailed Description	490
7.127.2 Field Documentation	490
7.127.2.1 next	490
7.127.2.2 reserved	491
7.128 service_load_info_t Struct Reference	491
7.128.1 Detailed Description	491
7.128.2 Field Documentation	491
7.128.2.1 flags	491
7.128.2.2 name_strid	491
7.128.2.3 sfn	492
7.128.2.4 sid	492
7.128.2.5 version	492
7.129 service_t Struct Reference	492
7.129.1 Detailed Description	493
7.129.2 Field Documentation	493

7.129.2.1 next	493
7.129.2.2 p_ldinf	493
7.129.2.3 partition	493
7.130 shared_data_tlv_entry Struct Reference	493
7.130.1 Detailed Description	494
7.130.2 Field Documentation	494
7.130.2.1 tlv_len	494
7.130.2.2 tlv_type	494
7.131 shared_data_tlv_header Struct Reference	494
7.131.1 Detailed Description	494
7.131.2 Field Documentation	494
7.131.2.1 tlv_magic	494
7.131.2.2 tlv_tot_len	495
7.132 tfm_additional_context_t Struct Reference	495
7.132.1 Detailed Description	495
7.132.2 Field Documentation	495
7.132.2.1 callee	495
7.132.2.2 integ_sign	495
7.132.2.3 reserved	495
7.133 tfm_boot_data Struct Reference	495
7.133.1 Detailed Description	496
7.133.2 Field Documentation	496
7.133.2.1 data	496
7.133.2.2 header	496
7.134 tfm_built_in_key_t Struct Reference	496
7.134.1 Detailed Description	497
7.134.2 Field Documentation	497
7.134.2.1 attr	497
7.134.2.2 is_loaded	497
7.134.2.3 key	497
7.134.2.4 key_len	498
7.135 tfm_crypto_aead_pack_input Struct Reference	498
7.135.1 Detailed Description	498
7.135.2 Field Documentation	498
7.135.2.1 nonce	498
7.135.2.2 nonce_length	498
7.136 tfm_crypto_key_id_s Struct Reference	498
7.136.1 Detailed Description	499
7.136.2 Field Documentation	499
7.136.2.1 key_id	499
7.136.2.2 owner	499
7.137 tfm_crypto_operation_s Struct Reference	499

7.137.1 Detailed Description	500
7.137.2 Field Documentation	500
7.137.2.1 aead	500
7.137.2.2 cipher	500
7.137.2.3 hash	500
7.137.2.4 in_use	500
7.137.2.5 key_deriv	500
7.137.2.6 mac	500
7.137.2.7 operation	501
7.137.2.8 owner	501
7.137.2.9 type	501
7.138 tfm_crypto_pack_iovec Struct Reference	501
7.138.1 Detailed Description	502
7.138.2 Field Documentation	502
7.138.2.1 "@9	502
7.138.2.2 ad_length	502
7.138.2.3 aead_in	502
7.138.2.4 alg	502
7.138.2.5 capacity	502
7.138.2.6 function_id	502
7.138.2.7 key_id	502
7.138.2.8 op_handle	503
7.138.2.9 plaintext_length	503
7.138.2.10 step	503
7.138.2.11 value	503
7.139 tfm_fwu_ctx_s Struct Reference	503
7.139.1 Detailed Description	503
7.139.2 Field Documentation	503
7.139.2.1 component_state	503
7.139.2.2 error	503
7.139.2.3 in_use	504
7.140 tfm_fwu_mcuboot_ctx_s Struct Reference	504
7.140.1 Detailed Description	504
7.140.2 Field Documentation	504
7.140.2.1 fap	504
7.140.2.2 loaded_size	504
7.141 tfm_ns_ctx_t Struct Reference	504
7.141.1 Detailed Description	504
7.141.2 Field Documentation	504
7.141.2.1 gid	505
7.141.2.2 nsid	505
7.141.2.3 ref_cnt	505

7.141.2.4 tid	505
7.142 tfm_pool_chunk_t Struct Reference	505
7.142.1 Detailed Description	505
7.142.2 Field Documentation	505
7.142.2.1 data	506
7.142.2.2 magic	506
7.142.2.3 next	506
7.143 tfm_pool_instance_t Struct Reference	506
7.143.1 Detailed Description	506
7.143.2 Field Documentation	506
7.143.2.1 chunks	507
7.143.2.2 chunksz	507
7.143.2.3 next	507
7.143.2.4 pool_sz	507
7.144 tfm_state_context_t Struct Reference	507
7.144.1 Detailed Description	507
7.144.2 Field Documentation	507
7.144.2.1 lr	507
7.144.2.2 r0	507
7.144.2.3 r1	508
7.144.2.4 r12	508
7.144.2.5 r2	508
7.144.2.6 r3	508
7.144.2.7 ra	508
7.144.2.8 xpsr	508
7.145 thread_t Struct Reference	508
7.145.1 Detailed Description	509
7.145.2 Field Documentation	509
7.145.2.1 flags	509
7.145.2.2 next	509
7.145.2.3 p_context_ctrl	509
7.145.2.4 priority	509
7.145.2.5 state	509
7.146 u64_in_u32_regs_t Union Reference	509
7.146.1 Detailed Description	510
7.146.2 Field Documentation	510
7.146.2.1 r0	510
7.146.2.2 r1	510
7.146.2.3 u32_regs	510
7.146.2.4 u64_val	510
7.147 vectors Struct Reference	510
7.147.1 Detailed Description	511

7.147.2 Field Documentation	511
7.147.2.1 in_use	511
7.147.2.2 in_vec	511
7.147.2.3 out_len	511
7.147.2.4 out_vec	511
8 File Documentation	513
8.1 docs/doxygen/mainpage.dox File Reference	513
8.2 interface/dir_interface.dox File Reference	513
8.3 interface/include/crypto_keys/tfm_builtin_key_ids.h File Reference	513
8.3.1 Enumeration Type Documentation	513
8.3.1.1 tfm_builtin_key_id_t	513
8.4 platform/ext/target/infineon/p32m6/spe/services/crypto/tfm_builtin_key_ids.h File Reference	514
8.4.1 Macro Definition Documentation	514
8.4.1.1 MBEDTLS_PSA_KEY_ID_BUILTIN_MIN	514
8.4.2 Enumeration Type Documentation	515
8.4.2.1 tfm_builtin_key_id_t	515
8.5 interface/include/hybrid_platform/api_broker_defs.h File Reference	515
8.5.1 Function Documentation	516
8.5.1.1 psa_call_multicore()	516
8.5.1.2 psa_call_tz()	517
8.5.1.3 psa_close_multicore()	517
8.5.1.4 psa_close_tz()	517
8.5.1.5 psa_connect_multicore()	517
8.5.1.6 psa_connect_tz()	517
8.5.1.7 psa_framework_version_multicore()	517
8.5.1.8 psa_framework_version_tz()	517
8.5.1.9 psa_version_multicore()	517
8.5.1.10 psa_version_tz()	517
8.6 interface/include/mbdtdls/asn1.h File Reference	518
8.6.1 Detailed Description	519
8.7 interface/include/mbdtdls/asn1write.h File Reference	520
8.7.1 Detailed Description	520
8.7.2 Macro Definition Documentation	520
8.7.2.1 MBEDTLS_ASN1_CHK_ADD	521
8.7.2.2 MBEDTLS_ASN1_CHK_CLEANUP_ADD	521
8.8 interface/include/mbdtdls/base64.h File Reference	521
8.8.1 Detailed Description	522
8.8.2 Macro Definition Documentation	522
8.8.2.1 MBEDTLS_ERR_BASE64_INVALID_CHARACTER	522
8.8.3 Function Documentation	522
8.8.3.1 mbdtdls_base64_decode()	522

8.8.3.2 mbedtls_base64_encode()	522
8.9 interface/include/mbedtls/compat-3-crypto.h File Reference	523
8.9.1 Detailed Description	524
8.9.2 Macro Definition Documentation	524
8.9.2.1 MBEDTLS_ERR_ASN1_ALLOC_FAILED	524
8.9.2.2 MBEDTLS_ERR_ASN1_BUF_TOO_SMALL	524
8.9.2.3 MBEDTLS_ERR_BASE64_BUFFER_TOO_SMALL	524
8.9.2.4 MBEDTLS_ERR_LMS_ALLOC_FAILED	524
8.9.2.5 MBEDTLS_ERR_LMS_BAD_INPUT_DATA	524
8.9.2.6 MBEDTLS_ERR_LMS_BUFFER_TOO_SMALL	525
8.9.2.7 MBEDTLS_ERR_PEM_ALLOC_FAILED	525
8.9.2.8 MBEDTLS_ERR_PEM_BAD_INPUT_DATA	525
8.9.2.9 MBEDTLS_ERR_PK_ALLOC_FAILED	525
8.9.2.10 MBEDTLS_ERR_PK_BAD_INPUT_DATA	525
8.9.2.11 MBEDTLS_ERR_PK_BUFFER_TOO_SMALL	525
8.10 interface/include/mbedtls/constant_time.h File Reference	525
8.10.1 Function Documentation	526
8.10.1.1 mbedtls_ct_memcmp()	526
8.11 interface/include/mbedtls/lms.h File Reference	526
8.11.1 Detailed Description	528
8.11.2 Macro Definition Documentation	528
8.11.2.1 MBEDTLS_ERR_LMS_OUT_OF_PRIVATE_KEYS	528
8.11.2.2 MBEDTLS_ERR_LMS_VERIFY_FAILED	528
8.11.2.3 MBEDTLS_LMOTS_C_RANDOM_VALUE_LEN	528
8.11.2.4 MBEDTLS_LMOTS_I_KEY_ID_LEN	528
8.11.2.5 MBEDTLS_LMOTS_N_HASH_LEN	528
8.11.2.6 MBEDTLS_LMOTS_N_HASH_LEN_MAX	529
8.11.2.7 MBEDTLS_LMOTS_P_SIG_DIGIT_COUNT	529
8.11.2.8 MBEDTLS_LMOTS_P_SIG_DIGIT_COUNT_MAX	529
8.11.2.9 MBEDTLS_LMOTS_Q_LEAF_ID_LEN	529
8.11.2.10 MBEDTLS_LMOTS_SIG_LEN	529
8.11.2.11 MBEDTLS_LMOTS_TYPE_LEN	529
8.11.2.12 MBEDTLS_LMS_H_TREE_HEIGHT	529
8.11.2.13 MBEDTLS_LMS_M_NODE_BYTES	529
8.11.2.14 MBEDTLS_LMS_M_NODE_BYTES_MAX	529
8.11.2.15 MBEDTLS_LMS_PUBLIC_KEY_LEN	530
8.11.2.16 MBEDTLS_LMS_SIG_LEN	530
8.11.2.17 MBEDTLS_LMS_TYPE_LEN	530
8.11.3 Enumeration Type Documentation	530
8.11.3.1 mbedtls_lmots_algorithm_type_t	530
8.11.3.2 mbedtls_lms_algorithm_type_t	530
8.11.4 Function Documentation	531

8.11.4.1 mbedtls_lms_export_public_key()	531
8.11.4.2 mbedtls_lms_import_public_key()	531
8.11.4.3 mbedtls_lms_public_free()	532
8.11.4.4 mbedtls_lms_public_init()	532
8.11.4.5 mbedtls_lms_verify()	532
8.12 interface/include/mbedtls/md.h File Reference	532
8.12.1 Detailed Description	534
8.12.2 Macro Definition Documentation	534
8.12.2.1 MBEDTLS_ERR_MD_ALLOC_FAILED	535
8.12.2.2 MBEDTLS_ERR_MD_BAD_INPUT_DATA	535
8.12.2.3 MBEDTLS_ERR_MD_FEATURE_UNAVAILABLE	535
8.12.2.4 MBEDTLS_MD_MAX_SIZE	535
8.12.3 Typedef Documentation	535
8.12.3.1 mbedtls_md_context_t	535
8.12.3.2 mbedtls_md_info_t	535
8.12.4 Enumeration Type Documentation	535
8.12.4.1 mbedtls_md_engine_t	535
8.12.4.2 mbedtls_md_type_t	536
8.12.5 Function Documentation	536
8.12.5.1 mbedtls_md()	536
8.12.5.2 mbedtls_md_clone()	537
8.12.5.3 mbedtls_md_finish()	537
8.12.5.4 mbedtls_md_free()	537
8.12.5.5 mbedtls_md_get_size()	538
8.12.5.6 mbedtls_md_get_type()	538
8.12.5.7 mbedtls_md_info_from_type()	538
8.12.5.8 mbedtls_md_init()	538
8.12.5.9 mbedtls_md_setup()	539
8.12.5.10 mbedtls_md_starts()	539
8.12.5.11 mbedtls_md_update()	539
8.13 interface/include/mbedtls/memory_buffer_alloc.h File Reference	540
8.13.1 Detailed Description	541
8.13.2 Macro Definition Documentation	541
8.13.2.1 MBEDTLS_MEMORY_ALIGN_MULTIPLE	541
8.13.2.2 MBEDTLS_MEMORY_VERIFY_ALLOC	541
8.13.2.3 MBEDTLS_MEMORY_VERIFY_ALWAYS	541
8.13.2.4 MBEDTLS_MEMORY_VERIFY_FREE	541
8.13.2.5 MBEDTLS_MEMORY_VERIFY_NONE	542
8.13.3 Function Documentation	542
8.13.3.1 mbedtls_memory_buffer_alloc_free()	542
8.13.3.2 mbedtls_memory_buffer_alloc_init()	542
8.13.3.3 mbedtls_memory_buffer_alloc_verify()	542

8.13.3.4 mbedtls_memory_buffer_set_verify()	543
8.14 interface/include/mbedtls/nist_kw.h File Reference	543
8.14.1 Detailed Description	544
8.14.2 Enumeration Type Documentation	544
8.14.2.1 mbedtls_nist_kw_mode_t	544
8.14.3 Function Documentation	544
8.14.3.1 mbedtls_nist_kw_unwrap()	544
8.14.3.2 mbedtls_nist_kw_wrap()	545
8.15 interface/include/mbedtls/pem.h File Reference	546
8.15.1 Detailed Description	546
8.15.2 Macro Definition Documentation	547
8.15.2.1 MBEDTLS_ERR_PEM_FEATURE_UNAVAILABLE	547
8.15.2.2 MBEDTLS_ERR_PEM_INVALID_DATA	547
8.15.2.3 MBEDTLS_ERR_PEM_INVALID_ENC_IV	547
8.15.2.4 MBEDTLS_ERR_PEM_NO_HEADER_FOOTER_PRESENT	547
8.15.2.5 MBEDTLS_ERR_PEM_PASSWORD_MISMATCH	547
8.15.2.6 MBEDTLS_ERR_PEM_PASSWORD_REQUIRED	547
8.15.2.7 MBEDTLS_ERR_PEM_UNKNOWN_ENC_ALG	547
8.16 interface/include/mbedtls/pk.h File Reference	547
8.16.1 Detailed Description	550
8.16.2 Macro Definition Documentation	550
8.16.2.1 MBEDTLS_ERR_PK_FEATURE_UNAVAILABLE	550
8.16.2.2 MBEDTLS_ERR_PK_FILE_IO_ERROR	550
8.16.2.3 MBEDTLS_ERR_PK_INVALID_ALG	550
8.16.2.4 MBEDTLS_ERR_PK_INVALID_PUBKEY	550
8.16.2.5 MBEDTLS_ERR_PK_KEY_INVALID_FORMAT	550
8.16.2.6 MBEDTLS_ERR_PK_KEY_INVALID_VERSION	551
8.16.2.7 MBEDTLS_ERR_PK_PASSWORD_MISMATCH	551
8.16.2.8 MBEDTLS_ERR_PK_PASSWORD_REQUIRED	551
8.16.2.9 MBEDTLS_ERR_PK_TYPE_MISMATCH	551
8.16.2.10 MBEDTLS_ERR_PK_UNKNOWN_NAMED_CURVE	551
8.16.2.11 MBEDTLS_ERR_PK_UNKNOWN_PK_ALG	551
8.16.2.12 MBEDTLS_PK_ALG_ECDSA	551
8.16.2.13 MBEDTLS_PK_HAVE_PRIVATE_HEADER	551
8.16.2.14 MBEDTLS_PK_MAX_EC_PUBKEY_RAW_LEN	552
8.16.2.15 MBEDTLS_PK_MAX_PUBKEY_RAW_LEN	552
8.16.2.16 MBEDTLS_PK_MAX_RSA_PUBKEY_RAW_LEN	552
8.16.2.17 MBEDTLS_PK_SIGNATURE_MAX_SIZE ^[1/2]	552
8.16.2.18 MBEDTLS_PK_SIGNATURE_MAX_SIZE ^[2/2]	552
8.16.2.19 MBEDTLS_PK_USE_PSA_EC_DATA	552
8.16.2.20 MBEDTLS_PK_USE_PSA_RSA_DATA	552
8.16.3 Typedef Documentation	552

8.16.3.1 mbedtls_pk_context	552
8.16.3.2 mbedtls_pk_info_t	552
8.16.3.3 mbedtls_pk_restart_ctx	553
8.16.4 Enumeration Type Documentation	553
8.16.4.1 mbedtls_pk_sigalg_t	553
8.16.5 Function Documentation	553
8.16.5.1 mbedtls_pk_can_do_psa()	553
8.16.5.2 mbedtls_pk_check_pair()	554
8.16.5.3 mbedtls_pk_copy_from_psa()	554
8.16.5.4 mbedtls_pk_copy_public_from_psa()	555
8.16.5.5 mbedtls_pk_free()	555
8.16.5.6 mbedtls_pk_get_bitlen()	556
8.16.5.7 mbedtls_pk_get_key_type()	556
8.16.5.8 mbedtls_pk_get_psa_attributes()	556
8.16.5.9 mbedtls_pk_import_into_psa()	558
8.16.5.10 mbedtls_pk_init()	559
8.16.5.11 mbedtls_pk_sign()	559
8.16.5.12 mbedtls_pk_sign_ext()	560
8.16.5.13 mbedtls_pk_sign_restartable()	560
8.16.5.14 mbedtls_pk_verify()	561
8.16.5.15 mbedtls_pk_verify_ext()	562
8.16.5.16 mbedtls_pk_verify_restartable()	562
8.16.5.17 mbedtls_pk_wrap_psa()	563
8.17 interface/include/mbedtls/platform.h File Reference	564
8.17.1 Detailed Description	565
8.17.2 Macro Definition Documentation	566
8.17.2.1 mbedtls_calloc	566
8.17.2.2 mbedtls_exit	566
8.17.2.3 MBEDTLS_EXIT_FAILURE	566
8.17.2.4 MBEDTLS_EXIT_SUCCESS	566
8.17.2.5 mbedtls_fprintf	566
8.17.2.6 mbedtls_free	566
8.17.2.7 MBEDTLS_PLATFORM_DEV_RANDOM	566
8.17.2.8 MBEDTLS_PLATFORM_STD_CALLOC	566
8.17.2.9 MBEDTLS_PLATFORM_STD_EXIT	566
8.17.2.10 MBEDTLS_PLATFORM_STD_EXIT_FAILURE	567
8.17.2.11 MBEDTLS_PLATFORM_STD_EXIT_SUCCESS	567
8.17.2.12 MBEDTLS_PLATFORM_STD_FPRINTF	567
8.17.2.13 MBEDTLS_PLATFORM_STD_FREE	567
8.17.2.14 MBEDTLS_PLATFORM_STD_PRINTF	567
8.17.2.15 MBEDTLS_PLATFORM_STD_SETBUF	567
8.17.2.16 MBEDTLS_PLATFORM_STD_SNPRINTF	567

8.17.2.17 MBEDTLS_PLATFORM_STD_TIME	567
8.17.2.18 MBEDTLS_PLATFORM_STD_VSNPRINTF	568
8.17.2.19 mbedtls_printf	568
8.17.2.20 mbedtls_setbuf	568
8.17.2.21 mbedtls_snprintf	568
8.17.2.22 mbedtls_vsnprintf	568
8.17.3 Typedef Documentation	568
8.17.3.1 mbedtls_platform_context	568
8.17.4 Function Documentation	568
8.17.4.1 mbedtls_platform_get_entropy()	568
8.17.4.2 mbedtls_platform_setup()	569
8.17.4.3 mbedtls_platform_teardown()	569
8.18 interface/include/mbedtls/platform_time.h File Reference	570
8.18.1 Detailed Description	570
8.18.2 Macro Definition Documentation	570
8.18.2.1 mbedtls_time	571
8.18.3 Typedef Documentation	571
8.18.3.1 mbedtls_ms_time_t	571
8.18.3.2 mbedtls_time_t	571
8.18.4 Function Documentation	571
8.18.4.1 mbedtls_ms_time()	571
8.19 interface/include/mbedtls/platform_util.h File Reference	571
8.19.1 Detailed Description	572
8.19.2 Macro Definition Documentation	572
8.19.2.1 MBEDTLS_CHECK_RETURN	572
8.19.2.2 MBEDTLS_CHECK_RETURN_CRITICAL	572
8.19.2.3 MBEDTLS_CHECK_RETURN_OPTIONAL	573
8.19.2.4 MBEDTLS_CHECK_RETURN_TYPICAL	573
8.19.2.5 MBEDTLS_DEPRECATED	573
8.19.2.6 MBEDTLS_DEPRECATED_NUMERIC_CONSTANT	573
8.19.2.7 MBEDTLS_DEPRECATED_STRING_CONSTANT	573
8.19.2.8 MBEDTLS_IGNORE_RETURN	574
8.19.3 Function Documentation	574
8.19.3.1 mbedtls_platform_zeroize()	574
8.20 interface/include/mbedtls/private/pk_private.h File Reference	574
8.20.1 Detailed Description	575
8.21 interface/include/mbedtls/private_access.h File Reference	575
8.21.1 Detailed Description	575
8.21.2 Macro Definition Documentation	575
8.21.2.1 MBEDTLS_PRIVATE	575
8.22 interface/include/mbedtls/psa_util.h File Reference	576
8.22.1 Detailed Description	576

8.23 interface/include/mbedtls/threading.h File Reference	576
8.23.1 Detailed Description	577
8.23.2 Macro Definition Documentation	577
8.23.2.1 MBEDTLS_ERR_THREADING_MUTEX_ERROR	577
8.23.2.2 MBEDTLS_ERR_THREADING_USAGE_ERROR	577
8.24 interface/include/multi_core/tfm_mailbox.h File Reference	577
8.24.1 Macro Definition Documentation	578
8.24.1.1 MAILBOX_ALIGN	578
8.24.1.2 MAILBOX_CACHE_LINE_SIZE	578
8.24.1.3 MAILBOX_CALLBACK_REG_ERROR	579
8.24.1.4 MAILBOX_CHAN_BUSY	579
8.24.1.5 MAILBOX_GENERIC_ERROR	579
8.24.1.6 MAILBOX_INIT_ERROR	579
8.24.1.7 MAILBOX_INVALID_PARAMS	579
8.24.1.8 MAILBOX_NO_PEND_EVENT	579
8.24.1.9 MAILBOX_NO_PERMS	579
8.24.1.10 MAILBOX_PSA_CALL	579
8.24.1.11 MAILBOX_PSA_CLOSE	579
8.24.1.12 MAILBOX_PSA_CONNECT	579
8.24.1.13 MAILBOX_PSA_FRAMEWORK_VERSION	580
8.24.1.14 MAILBOX_PSA_VERSION	580
8.24.1.15 MAILBOX_QUEUE_FULL	580
8.24.1.16 MAILBOX_SUCCESS	580
8.24.2 Typedef Documentation	580
8.24.2.1 mailbox_queue_status_t	580
8.24.3 Function Documentation	580
8.24.3.1 __ALIGNED()	580
8.24.4 Variable Documentation	580
8.24.4.1 __ALIGNED	580
8.24.4.2 pend_slots	580
8.24.4.3 replied_slots	580
8.25 interface/include/multi_core/tfm_multi_core_api.h File Reference	581
8.25.1 Function Documentation	581
8.25.1.1 tfm_ns_wait_for_s_cpu_ready()	581
8.25.1.2 tfm_platform_ns_wait_for_s_cpu_ready()	582
8.26 interface/include/multi_core/tfm_ns_mailbox.h File Reference	582
8.26.1 Macro Definition Documentation	584
8.26.1.1 DCACHE_IS_ENABLED	584
8.26.1.2 MAILBOX_CLEAN_CACHE	584
8.26.1.3 MAILBOX_DISABLE_CACHE	584
8.26.1.4 MAILBOX_ENABLE_CACHE	584
8.26.1.5 MAILBOX_INVALIDATE_CACHE	584

8.26.1.6 tfm_ns_mailbox_os_spin_lock	584
8.26.1.7 tfm_ns_mailbox_os_spin_unlock	585
8.26.1.8 tfm_ns_mailbox_os_wait_reply	585
8.26.1.9 tfm_ns_mailbox_os_wake_task_isr	585
8.26.1.10 tfm_ns_mailbox_thread_runner	585
8.26.2 Function Documentation	585
8.26.2.1 mailbox_set_queue_ptr()	585
8.26.2.2 tfm_ns_mailbox_client_call()	585
8.26.2.3 tfm_ns_mailbox_hal_enter_critical()	586
8.26.2.4 tfm_ns_mailbox_hal_enter_critical_isr()	587
8.26.2.5 tfm_ns_mailbox_hal_exit_critical()	587
8.26.2.6 tfm_ns_mailbox_hal_exit_critical_isr()	588
8.26.2.7 tfm_ns_mailbox_hal_init()	588
8.26.2.8 tfm_ns_mailbox_hal_notify_peer()	589
8.26.2.9 tfm_ns_mailbox_init()	589
8.26.2.10 tfm_ns_multi_core_boot()	590
8.27 interface/include/multi_core/tfm_ns_mailbox_test.h File Reference	591
8.27.1 Function Documentation	591
8.27.1.1 tfm_ns_mailbox_stats_avg_slot()	592
8.27.1.2 tfm_ns_mailbox_tx_stats_init()	592
8.27.1.3 tfm_ns_mailbox_tx_stats_reinit()	592
8.27.1.4 tfm_ns_mailbox_tx_stats_update()	592
8.28 interface/include/os_wrapper/common.h File Reference	593
8.28.1 Macro Definition Documentation	593
8.28.1.1 OS_WRAPPER_DEFAULT_STACK_SIZE	593
8.28.1.2 OS_WRAPPER_ERROR	593
8.28.1.3 OS_WRAPPER_SUCCESS	593
8.28.1.4 OS_WRAPPER_WAIT_FOREVER	594
8.29 interface/include/os_wrapper/kernel.h File Reference	594
8.29.1 Function Documentation	595
8.29.1.1 os_wrapper_is_kernel_started()	595
8.30 interface/include/os_wrapper/mutex.h File Reference	595
8.30.1 Function Documentation	596
8.30.1.1 os_wrapper_mutex_acquire()	596
8.30.1.2 os_wrapper_mutex_create()	597
8.30.1.3 os_wrapper_mutex_delete()	597
8.30.1.4 os_wrapper_mutex_release()	597
8.31 interface/include/psa/api_broker.h File Reference	598
8.31.1 Macro Definition Documentation	599
8.31.1.1 PSA_CALL_MULTICORE	599
8.31.1.2 PSA_CALL_TZ	599
8.31.1.3 PSA_CLOSE_MULTICORE	599

8.31.1.4	PSA_CLOSE_TZ	599
8.31.1.5	PSA_CONNECT_MULTICORE	599
8.31.1.6	PSA_CONNECT_TZ	599
8.31.1.7	PSA_FRAMEWORK_VERSION_MULTICORE	599
8.31.1.8	PSA_FRAMEWORK_VERSION_TZ	599
8.31.1.9	PSA_VERSION_MULTICORE	599
8.31.1.10	PSA_VERSION_TZ	600
8.32	interface/include/psa/client.h File Reference	600
8.32.1	Macro Definition Documentation	601
8.32.1.1	IOVEC_LEN	601
8.32.1.2	PSA_CALL_TYPE_MAX	601
8.32.1.3	PSA_CALL_TYPE_MIN	601
8.32.1.4	PSA_FRAMEWORK_VERSION	601
8.32.1.5	PSA_HANDLE_IS_VALID	602
8.32.1.6	PSA_HANDLE_TO_ERROR	602
8.32.1.7	PSA_IPC_CALL	602
8.32.1.8	PSA_MAX_IOVEC	602
8.32.1.9	PSA_NULL_HANDLE	602
8.32.1.10	PSA_VERSION_NONE	602
8.32.1.11	ROT_SIZE_MAX	602
8.32.2	Typedef Documentation	602
8.32.2.1	psa_handle_t	602
8.32.2.2	psa_invec	603
8.32.2.3	psa_outvec	603
8.32.2.4	rot_size_t	603
8.32.3	Function Documentation	603
8.32.3.1	psa_call()	603
8.32.3.2	psa_close()	605
8.32.3.3	psa_connect()	606
8.32.3.4	psa_framework_version()	607
8.32.3.5	psa_version()	607
8.33	interface/include/psa/crypto.h File Reference	608
8.33.1	Detailed Description	613
8.34	interface/include/psa/crypto_compat.h File Reference	614
8.34.1	Detailed Description	614
8.34.2	Macro Definition Documentation	614
8.34.2.1	PSA_ALG_JPAKE_BETA	614
8.34.2.2	PSA_EXPORT_KEY_PAIR_OR_PUBLIC_MAX_SIZE	614
8.34.2.3	PSA_KEY_TYPE_DES	615
8.34.3	Function Documentation	615
8.34.3.1	psa_can_do_cipher()	615
8.34.3.2	psa_can_do_hash()	615

8.35 interface/include/psa/crypto_driver_common.h File Reference	615
8.35.1 Detailed Description	616
8.35.2 Enumeration Type Documentation	616
8.35.2.1 psa_encrypt_or_decrypt_t	616
8.36 interface/include/psa/crypto_driver_contexts_composites.h File Reference	616
8.36.1 Detailed Description	617
8.37 interface/include/psa/crypto_driver_contexts_key_derivation.h File Reference	617
8.37.1 Detailed Description	618
8.38 interface/include/psa/crypto_driver_contexts_primitives.h File Reference	618
8.38.1 Detailed Description	619
8.39 interface/include/psa/crypto_driver_random.h File Reference	620
8.39.1 Detailed Description	621
8.40 interface/include/psa/crypto_extra.h File Reference	621
8.40.1 Detailed Description	624
8.40.2 Macro Definition Documentation	624
8.40.2.1 MBEDTLS_PSA_KEY_SLOT_COUNT	624
8.40.2.2 MBEDTLS_PSA_STATIC_KEY_SLOT_BUFFER_SIZE	624
8.40.2.3 PSA_CRYPTOTO_RANDOM_SEED_UID	625
8.40.2.4 PSA_JPAKE_EXPECTED_INPUTS	625
8.40.2.5 PSA_JPAKE_EXPECTED_OUTPUTS	625
8.40.2.6 PSA_PAKE_CIPHER_SUITE_INIT	625
8.40.2.7 PSA_PAKE_CONFIRMED_KEY	625
8.40.2.8 PSA_PAKE_INPUT_MAX_SIZE	625
8.40.2.9 PSA_PAKE_INPUT_SIZE	625
8.40.2.10 PSA_PAKE_OPERATION_INIT	626
8.40.2.11 PSA_PAKE_OUTPUT_MAX_SIZE	626
8.40.2.12 PSA_PAKE_OUTPUT_SIZE	626
8.40.2.13 PSA_PAKE_UNCONFIRMED_KEY	627
8.40.3 Typedef Documentation	627
8.40.3.1 mbedtls_psa_stats_t	627
8.40.3.2 psa_crypto_driver_pake_step_t	627
8.40.3.3 psa_jpake_io_mode_t	627
8.40.3.4 psa_jpake_round_t	628
8.40.4 Enumeration Type Documentation	628
8.40.4.1 psa_crypto_driver_pake_step	628
8.40.4.2 psa_jpake_io_mode	628
8.40.4.3 psa_jpake_round	628
8.40.5 Function Documentation	629
8.40.5.1 MBEDTLS_PRIVATE() [1/3]	629
8.40.5.2 MBEDTLS_PRIVATE() [2/3]	629
8.40.5.3 MBEDTLS_PRIVATE() [3/3]	629
8.40.5.4 mbedtls_psa_crypto_free()	629

8.40.5.5 mbedtls_psa_get_stats()	629
8.41 interface/include/psa/crypto_platform.h File Reference	629
8.41.1 Detailed Description	630
8.42 interface/include/psa/crypto_sizes.h File Reference	630
8.42.1 Detailed Description	632
8.42.2 Macro Definition Documentation	632
8.42.2.1 PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE	632
8.42.2.2 PSA_AEAD_DECRYPT_OUTPUT_SIZE	633
8.42.2.3 PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE	633
8.42.2.4 PSA_AEAD_ENCRYPT_OUTPUT_SIZE	634
8.42.2.5 PSA_AEAD_FINISH_OUTPUT_MAX_SIZE	634
8.42.2.6 PSA_AEAD_FINISH_OUTPUT_SIZE	634
8.42.2.7 PSA_AEAD_NONCE_LENGTH	635
8.42.2.8 PSA_AEAD_NONCE_MAX_SIZE	635
8.42.2.9 PSA_AEAD_TAG_LENGTH	636
8.42.2.10 PSA_AEAD_TAG_MAX_SIZE	636
8.42.2.11 PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE	636
8.42.2.12 PSA_AEAD_UPDATE_OUTPUT_SIZE	636
8.42.2.13 PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE	637
8.42.2.14 PSA_AEAD_VERIFY_OUTPUT_SIZE	637
8.42.2.15 PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE	638
8.42.2.16 PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE	638
8.42.2.17 PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE	638
8.42.2.18 PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE	639
8.42.2.19 PSA_BITS_TO_BYTES	639
8.42.2.20 PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE	639
8.42.2.21 PSA_BYTES_TO_BITS	639
8.42.2.22 PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE	639
8.42.2.23 PSA_CIPHER_DECRYPT_OUTPUT_SIZE	640
8.42.2.24 PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE	640
8.42.2.25 PSA_CIPHER_ENCRYPT_OUTPUT_SIZE	641
8.42.2.26 PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE	641
8.42.2.27 PSA_CIPHER_FINISH_OUTPUT_SIZE	641
8.42.2.28 PSA_CIPHER_IV_LENGTH	642
8.42.2.29 PSA_CIPHER_IV_MAX_SIZE	642
8.42.2.30 PSA_CIPHER_MAX_KEY_LENGTH	643
8.42.2.31 PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE	643
8.42.2.32 PSA_CIPHER_UPDATE_OUTPUT_SIZE	643
8.42.2.33 PSA_ECDSA_SIGNATURE_SIZE	644
8.42.2.34 PSA_EXPORT_ASYMMETRIC_KEY_MAX_SIZE	644
8.42.2.35 PSA_EXPORT_KEY_OUTPUT_SIZE	644
8.42.2.36 PSA_EXPORT_KEY_PAIR_MAX_SIZE	645

8.42.2.37	PSA_EXPORT_PUBLIC_KEY_MAX_SIZE	645
8.42.2.38	PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE	645
8.42.2.39	PSA_HASH_BLOCK_LENGTH	646
8.42.2.40	PSA_HASH_LENGTH	647
8.42.2.41	PSA_HASH_MAX_SIZE	647
8.42.2.42	PSA_HMAC_MAX_HASH_BLOCK_SIZE	647
8.42.2.43	PSA_KEY_EXPORT_ASN1_INTEGER_MAX_SIZE	647
8.42.2.44	PSA_KEY_EXPORT_DSA_KEY_PAIR_MAX_SIZE	647
8.42.2.45	PSA_KEY_EXPORT_DSA_PUBLIC_KEY_MAX_SIZE	648
8.42.2.46	PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE	648
8.42.2.47	PSA_KEY_EXPORT_ECC_PUBLIC_KEY_MAX_SIZE	648
8.42.2.48	PSA_KEY_EXPORT_FFDH_KEY_PAIR_MAX_SIZE	648
8.42.2.49	PSA_KEY_EXPORT_FFDH_PUBLIC_KEY_MAX_SIZE	648
8.42.2.50	PSA_KEY_EXPORT_RSA_KEY_PAIR_MAX_SIZE	648
8.42.2.51	PSA_KEY_EXPORT_RSA_PUBLIC_KEY_MAX_SIZE	648
8.42.2.52	PSA_MAC_LENGTH	648
8.42.2.53	PSA_MAC_MAX_SIZE	649
8.42.2.54	PSA_MAX_OF_THREE	649
8.42.2.55	PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE	649
8.42.2.56	PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE	649
8.42.2.57	PSA_ROUND_UP_TO_MULTIPLE	650
8.42.2.58	PSA_RSA_MINIMUM_PADDING_SIZE	650
8.42.2.59	PSA_SIGN_OUTPUT_SIZE	650
8.42.2.60	PSA_SIGNATURE_MAX_SIZE	651
8.42.2.61	PSA_TLS12_ECJPAKE_TO_PMS_DATA_SIZE	651
8.42.2.62	PSA_TLS12_ECJPAKE_TO_PMS_INPUT_SIZE	651
8.42.2.63	PSA_TLS12_PSK_TO_MS_PSK_MAX_SIZE	651
8.42.2.64	PSA_VENDOR_ECC_MAX_CURVE_BITS	651
8.42.2.65	PSA_VENDOR_ECDSA_SIGNATURE_MAX_SIZE	652
8.42.2.66	PSA_VENDOR_FFDH_MAX_KEY_BITS	652
8.42.2.67	PSA_VENDOR_PBKDF2_MAX_ITERATIONS	652
8.42.2.68	PSA_VENDOR_RSA_GENERATE_MIN_KEY_BITS	652
8.42.2.69	PSA_VENDOR_RSA_MAX_KEY_BITS	652
8.43	interface/include/psa/crypto_struct.h File Reference	652
8.43.1	Detailed Description	653
8.43.2	Macro Definition Documentation	654
8.43.2.1	PSA_CUSTOM_KEY_PARAMETERS_INIT	654
8.43.2.2	PSA_EXPORT_PUBLIC_KEY_IOP_INIT	654
8.43.2.3	PSA_GENERATE_KEY_IOP_INIT	654
8.43.2.4	PSA_KEY_AGREEMENT_IOP_INIT	654
8.43.2.5	PSA_KEY_BITS_TOO_LARGE	654
8.43.2.6	PSA_KEY_POLICY_INIT	655

8.43.2.7	PSA_MAX_KEY_BITS	655
8.43.2.8	PSA_SIGN_HASH_INTERRUPTIBLE_OPERATION_INIT	655
8.43.2.9	PSA_VERIFY_HASH_INTERRUPTIBLE_OPERATION_INIT	655
8.43.3	Typedef Documentation	655
8.43.3.1	psa_key_bits_t	655
8.43.3.2	psa_key_policy_t	655
8.43.4	Variable Documentation	655
8.43.4.1	MBEDTLS_PRIVATE	655
8.44	interface/include/psa/crypto_types.h File Reference	655
8.44.1	Detailed Description	656
8.45	interface/include/psa/crypto_values.h File Reference	657
8.45.1	Detailed Description	663
8.46	interface/include/psa/crypto_values_hms.h File Reference	663
8.46.1	Macro Definition Documentation	663
8.46.1.1	PSA_ALG_HSS	664
8.46.1.2	PSA_ALG_HSS_BASE	664
8.46.1.3	PSA_ALG_IS_HSS	664
8.46.1.4	PSA_ALG_IS_LMS	664
8.46.1.5	PSA_ALG_LMS	664
8.46.1.6	PSA_ALG_LMS_BASE	664
8.47	interface/include/psa/error.h File Reference	664
8.47.1	Detailed Description	666
8.47.2	Macro Definition Documentation	666
8.47.2.1	PSA_ERROR_ALREADY_EXISTS	666
8.47.2.2	PSA_ERROR_BAD_STATE	666
8.47.2.3	PSA_ERROR_BUFFER_TOO_SMALL	666
8.47.2.4	PSA_ERROR_COMMUNICATION_FAILURE	666
8.47.2.5	PSA_ERROR_CONNECTION_BUSY	667
8.47.2.6	PSA_ERROR_CONNECTION_REFUSED	667
8.47.2.7	PSA_ERROR_CORRUPTION_DETECTED	667
8.47.2.8	PSA_ERROR_DATA_CORRUPT	667
8.47.2.9	PSA_ERROR_DATA_INVALID	668
8.47.2.10	PSA_ERROR_DOES_NOT_EXIST	668
8.47.2.11	PSA_ERROR_GENERIC_ERROR	668
8.47.2.12	PSA_ERROR_HARDWARE_FAILURE	668
8.47.2.13	PSA_ERROR_INSUFFICIENT_DATA	668
8.47.2.14	PSA_ERROR_INSUFFICIENT_MEMORY	668
8.47.2.15	PSA_ERROR_INSUFFICIENT_STORAGE	669
8.47.2.16	PSA_ERROR_INVALID_ARGUMENT	669
8.47.2.17	PSA_ERROR_INVALID_HANDLE	669
8.47.2.18	PSA_ERROR_INVALID_SIGNATURE	669
8.47.2.19	PSA_ERROR_NOT_PERMITTED	669

8.47.2.20 PSA_ERROR_NOT_SUPPORTED	669
8.47.2.21 PSA_ERROR_PROGRAMMER_ERROR	670
8.47.2.22 PSA_ERROR_SERVICE_FAILURE	670
8.47.2.23 PSA_ERROR_STORAGE_FAILURE	670
8.47.2.24 PSA_OPERATION_INCOMPLETE	670
8.47.2.25 PSA_SUCCESS	670
8.47.3 Typedef Documentation	670
8.47.3.1 psa_status_t	670
8.48 interface/include/psa/internal_trusted_storage.h File Reference	671
8.48.1 Macro Definition Documentation	672
8.48.1.1 PSA_ITS_API_VERSION_MAJOR	672
8.48.1.2 PSA_ITS_API_VERSION_MINOR	672
8.48.2 Function Documentation	672
8.48.2.1 psa_its_get()	672
8.48.2.2 psa_its_get_info()	673
8.48.2.3 psa_its_remove()	674
8.48.2.4 psa_its_set()	675
8.49 interface/include/psa/lifecycle.h File Reference	676
8.49.1 Macro Definition Documentation	677
8.49.1.1 PSA_LIFECYCLE_ASSEMBLY_AND_TEST	677
8.49.1.2 PSA_LIFECYCLE_DECOMMISSIONED	677
8.49.1.3 PSA_LIFECYCLE_IMP_STATE_MASK	677
8.49.1.4 PSA_LIFECYCLE_NON_PSA_ROT_DEBUG	677
8.49.1.5 PSA_LIFECYCLE_PSA_ROT_PROVISIONING	677
8.49.1.6 PSA_LIFECYCLE_PSA_STATE_MASK	677
8.49.1.7 PSA_LIFECYCLE_RECOVERABLE_PSA_ROT_DEBUG	677
8.49.1.8 PSA_LIFECYCLE_SECURED	677
8.49.1.9 PSA_LIFECYCLE_UNKNOWN	678
8.49.2 Function Documentation	678
8.49.2.1 psa_rot_lifecycle_state()	678
8.50 interface/include/psa/protected_storage.h File Reference	678
8.50.1 Macro Definition Documentation	679
8.50.1.1 PSA_PS_API_VERSION_MAJOR	679
8.50.1.2 PSA_PS_API_VERSION_MINOR	679
8.50.2 Function Documentation	679
8.50.2.1 psa_ps_create()	679
8.50.2.2 psa_ps_get()	680
8.50.2.3 psa_ps_get_info()	681
8.50.2.4 psa_ps_get_support()	682
8.50.2.5 psa_ps_remove()	682
8.50.2.6 psa_ps_set()	683
8.50.2.7 psa_ps_set_extended()	684

8.51 interface/include/psa/service.h File Reference	685
8.51.1 Macro Definition Documentation	687
8.51.1.1 PSA_BLOCK	687
8.51.1.2 PSA_DOORBELL	687
8.51.1.3 PSA_FLIH_NO_SIGNAL	687
8.51.1.4 PSA_FLIH_SIGNAL	687
8.51.1.5 PSA_IPC_CONNECT	688
8.51.1.6 PSA_IPC_DISCONNECT	688
8.51.1.7 PSA_POLL	688
8.51.1.8 PSA_WAIT_ANY	688
8.51.2 Typedef Documentation	688
8.51.2.1 psa_flih_result_t	688
8.51.2.2 psa_irq_status_t	688
8.51.2.3 psa_msg_t	688
8.51.2.4 psa_signal_t	688
8.51.3 Function Documentation	688
8.51.3.1 psa_clear()	688
8.51.3.2 psa_eoi()	689
8.51.3.3 psa_get()	689
8.51.3.4 psa_irq_disable()	690
8.51.3.5 psa_irq_enable()	690
8.51.3.6 psa_notify()	691
8.51.3.7 psa_panic()	691
8.51.3.8 psa_read()	692
8.51.3.9 psa_reply()	693
8.51.3.10 psa_reset_signal()	694
8.51.3.11 psa_set_rhandle()	694
8.51.3.12 psa_skip()	694
8.51.3.13 psa_wait()	695
8.51.3.14 psa_write()	696
8.52 interface/include/psa/storage_common.h File Reference	697
8.52.1 Macro Definition Documentation	698
8.52.1.1 PSA_ERROR_DATA_CORRUPT	698
8.52.1.2 PSA_ERROR_INVALID_SIGNATURE	698
8.52.1.3 PSA_STORAGE_FLAG_NO_CONFIDENTIALITY	698
8.52.1.4 PSA_STORAGE_FLAG_NO_REPLAY_PROTECTION	698
8.52.1.5 PSA_STORAGE_FLAG_NONE	699
8.52.1.6 PSA_STORAGE_FLAG_WRITE_ONCE	699
8.52.1.7 PSA_STORAGE_SUPPORT_SET_EXTENDED	699
8.52.2 Typedef Documentation	699
8.52.2.1 psa_storage_create_flags_t	699
8.52.2.2 psa_storage_uid_t	699

8.53 interface/include/psa/update.h File Reference	699
8.53.1 Macro Definition Documentation	701
8.53.1.1 PSA_ERROR_DEPENDENCY_NEEDED	701
8.53.1.2 PSA_ERROR_FLASH_ABUSE	702
8.53.1.3 PSA_ERROR_INSUFFICIENT_POWER	702
8.53.1.4 PSA_FWU_API_VERSION_MAJOR	702
8.53.1.5 PSA_FWU_API_VERSION_MINOR	702
8.53.1.6 PSA_FWU_CANDIDATE	702
8.53.1.7 PSA_FWU_FAILED	702
8.53.1.8 PSA_FWU_FLAG_ENCRYPTION	702
8.53.1.9 PSA_FWU_FLAG_VOLATILE_STAGING	702
8.53.1.10 PSA_FWU_MAX_WRITE_SIZE	702
8.53.1.11 PSA_FWU_READY	703
8.53.1.12 PSA_FWU_REJECTED	703
8.53.1.13 PSA_FWU_STAGED	703
8.53.1.14 PSA_FWU_TRIAL	703
8.53.1.15 PSA_FWU_UPDATED	703
8.53.1.16 PSA_FWU_WRITING	703
8.53.1.17 PSA_SUCCESS_REBOOT	703
8.53.1.18 PSA_SUCCESS_RESTART	703
8.53.2 Typedef Documentation	704
8.53.2.1 psa_fwu_component_info_t	704
8.53.2.2 psa_fwu_component_t	704
8.53.2.3 psa_fwu_image_version_t	704
8.53.3 Function Documentation	704
8.53.3.1 psa_fwu_accept()	704
8.53.3.2 psa_fwu_cancel()	704
8.53.3.3 psa_fwu_clean()	705
8.53.3.4 psa_fwu_finish()	705
8.53.3.5 psa_fwu_install()	706
8.53.3.6 psa_fwu_query()	706
8.53.3.7 psa_fwu_reject()	707
8.53.3.8 psa_fwu_request_reboot()	707
8.53.3.9 psa_fwu_start()	708
8.53.3.10 psa_fwu_write()	708
8.54 interface/include/tf-psa-crypto/build_info.h File Reference	709
8.54.1 Detailed Description	710
8.54.2 Macro Definition Documentation	710
8.54.2.1 TF_PSA_CRYPTO_CONFIG_FILES_READ	710
8.54.2.2 TF_PSA_CRYPTO_CONFIG_IS_FINALIZED	710
8.54.2.3 TF_PSA_CRYPTO_VERSION_MAJOR	710
8.54.2.4 TF_PSA_CRYPTO_VERSION_MINOR	710

8.54.2.5 TF_PSA_CRYPTO_VERSION_NUMBER	710
8.54.2.6 TF_PSA_CRYPTO_VERSION_PATCH	711
8.54.2.7 TF_PSA_CRYPTO_VERSION_STRING	711
8.54.2.8 TF_PSA_CRYPTO_VERSION_STRING_FULL	711
8.54.3 Typedef Documentation	711
8.54.3.1 mbedtls_iso_c_forbids_empty_translation_units	711
8.55 interface/include/tf-psa-crypto/private/crypto_adjust_config_auto_enabled.h File Reference	711
8.55.1 Detailed Description	711
8.55.2 Macro Definition Documentation	712
8.55.2.1 PSA_WANT_KEY_TYPE_DERIVE	712
8.55.2.2 PSA_WANT_KEY_TYPE_PASSWORD	712
8.55.2.3 PSA_WANT_KEY_TYPE_PASSWORD_HASH	712
8.55.2.4 PSA_WANT_KEY_TYPE_RAW_DATA	712
8.56 interface/include/tf-psa-crypto/private/crypto_adjust_config_dependencies.h File Reference	712
8.56.1 Detailed Description	712
8.57 interface/include/tf-psa-crypto/private/crypto_adjust_config_derived.h File Reference	712
8.57.1 Detailed Description	713
8.57.2 Macro Definition Documentation	713
8.57.2.1 MBEDTLS_ENTROPY_TRUE_SOURCES	713
8.57.2.2 MBEDTLS_PSA_BUILTIN_GET_ENTROPY_DEFINED	713
8.57.2.3 MBEDTLS_PSA_CRYPTO_RNG_STRENGTH	713
8.57.2.4 MBEDTLS_PSA_DRIVER_GET_ENTROPY_DEFINED	713
8.58 interface/include/tf-psa-crypto/private/crypto_adjust_config_key_pair_types.h File Reference	714
8.58.1 Detailed Description	714
8.59 interface/include/tf-psa-crypto/private/crypto_adjust_config_support.h File Reference	714
8.59.1 Detailed Description	714
8.60 interface/include/tf-psa-crypto/private/crypto_adjust_config_synonyms.h File Reference	715
8.60.1 Detailed Description	715
8.61 interface/include/tf-psa-crypto/version.h File Reference	715
8.61.1 Detailed Description	716
8.62 interface/include/tfm_attest_defs.h File Reference	716
8.62.1 Macro Definition Documentation	716
8.62.1.1 TFM_ATTEST_GET_TOKEN	716
8.62.1.2 TFM_ATTEST_GET_TOKEN_SIZE	716
8.63 interface/include/tfm_attest_iat_defs.h File Reference	716
8.63.1 Macro Definition Documentation	717
8.63.1.1 IAT_SW_COMPONENT_MEASUREMENT_DESC	717
8.63.1.2 IAT_SW_COMPONENT_MEASUREMENT_TYPE	718
8.63.1.3 IAT_SW_COMPONENT_MEASUREMENT_VALUE	718
8.63.1.4 IAT_SW_COMPONENT_SIGNER_ID	718
8.63.1.5 IAT_SW_COMPONENT_VERSION	718
8.64 interface/include/tfm_crypto_defs.h File Reference	718

8.64.1 Macro Definition Documentation	720
8.64.1.1 AEAD_FUNCS	720
8.64.1.2 ASYM_ENCRYPT_FUNCS	720
8.64.1.3 ASYM_SIGN_FUNCS	721
8.64.1.4 BASE__VALUE	721
8.64.1.5 CIPHER_FUNCS	721
8.64.1.6 HASH_FUNCS	721
8.64.1.7 KEY_DERIVATION_FUNCS	721
8.64.1.8 KEY_MANAGEMENT_FUNCS	722
8.64.1.9 MAC_FUNCS	722
8.64.1.10 RANDOM_FUNCS	722
8.64.1.11 TFM_CRYPT_GET_GROUP_ID	722
8.64.1.12 TFM_CRYPT_MAX_NONCE_LENGTH	722
8.64.1.13 X	722
8.64.2 Enumeration Type Documentation	722
8.64.2.1 tfm_crypto_func_sid_t	723
8.64.2.2 tfm_crypto_group_id_t	724
8.65 interface/include/tfm_fwu_defs.h File Reference	725
8.65.1 Macro Definition Documentation	725
8.65.1.1 TFM_FWU_ACCEPT	725
8.65.1.2 TFM_FWU_CANCEL	726
8.65.1.3 TFM_FWU_CLEAN	726
8.65.1.4 TFM_FWU_FINISH	726
8.65.1.5 TFM_FWU_INSTALL	726
8.65.1.6 TFM_FWU_QUERY	726
8.65.1.7 TFM_FWU_REJECT	726
8.65.1.8 TFM_FWU_REQUEST_REBOOT	726
8.65.1.9 TFM_FWU_START	726
8.65.1.10 TFM_FWU_WRITE	726
8.66 interface/include/tfm_fwu_impl_info.h File Reference	726
8.67 interface/include/tfm_its_defs.h File Reference	727
8.67.1 Macro Definition Documentation	728
8.67.1.1 TFM_ITS_GET	728
8.67.1.2 TFM_ITS_GET_INFO	728
8.67.1.3 TFM_ITS_INVALID_UID	728
8.67.1.4 TFM_ITS_REMOVE	728
8.67.1.5 TFM_ITS_SET	728
8.68 interface/include/tfm_ns_client_ext.h File Reference	728
8.68.1 Macro Definition Documentation	730
8.68.1.1 TFM_NS_CLIENT_ERR_INVALID_ACCESS	730
8.68.1.2 TFM_NS_CLIENT_ERR_INVALID_NSID	730
8.68.1.3 TFM_NS_CLIENT_ERR_INVALID_TOKEN	730

8.68.1.4 TFM_NS_CLIENT_ERR_SUCCESS	730
8.68.1.5 TFM_NS_CLIENT_INVALID_TOKEN	730
8.68.2 Function Documentation	730
8.68.2.1 tfm_nsce_acquire_ctx()	730
8.68.2.2 tfm_nsce_init()	731
8.68.2.3 tfm_nsce_load_ctx()	731
8.68.2.4 tfm_nsce_release_ctx()	732
8.68.2.5 tfm_nsce_save_ctx()	733
8.69 interface/include/tfm_ns_interface.h File Reference	733
8.69.1 Typedef Documentation	735
8.69.1.1 veneer_fn	735
8.69.2 Enumeration Type Documentation	735
8.69.2.1 tfm_reenter_check_err_t	735
8.69.3 Function Documentation	735
8.69.3.1 tfm_ns_check_exit()	735
8.69.3.2 tfm_ns_check_safe_entry()	735
8.69.3.3 tfm_ns_interface_dispatch()	736
8.69.3.4 tfm_ns_interface_init()	737
8.70 interface/include/tfm_platform_api.h File Reference	738
8.70.1 Macro Definition Documentation	739
8.70.1.1 TFM_PLATFORM_API_ID_IOCTL	739
8.70.1.2 TFM_PLATFORM_API_ID_NV_INCREMENT	739
8.70.1.3 TFM_PLATFORM_API_ID_NV_READ	739
8.70.1.4 TFM_PLATFORM_API_ID_SYSTEM_RESET	740
8.70.1.5 TFM_PLATFORM_API_VERSION_MAJOR	740
8.70.1.6 TFM_PLATFORM_API_VERSION_MINOR	740
8.70.2 Typedef Documentation	740
8.70.2.1 tfm_platform_ioctl_req_t	740
8.70.3 Enumeration Type Documentation	740
8.70.3.1 tfm_platform_err_t	740
8.70.4 Function Documentation	740
8.70.4.1 tfm_platform_ioctl()	740
8.70.4.2 tfm_platform_nv_counter_increment()	741
8.70.4.3 tfm_platform_nv_counter_read()	742
8.70.4.4 tfm_platform_system_reset()	743
8.71 interface/include/tfm_ps_defs.h File Reference	744
8.71.1 Macro Definition Documentation	744
8.71.1.1 TFM_PS_GET	744
8.71.1.2 TFM_PS_GET_INFO	744
8.71.1.3 TFM_PS_GET_SUPPORT	744
8.71.1.4 TFM_PS_INVALID_UID	744
8.71.1.5 TFM_PS_REMOVE	744

8.71.1.6 TFM_PS_SET	745
8.72 interface/include/tfm_psa_call_pack.h File Reference	745
8.72.1 Macro Definition Documentation	746
8.72.1.1 IN_LEN_MASK	746
8.72.1.2 IN_LEN_OFFSET	746
8.72.1.3 NS_INVEC_BIT	746
8.72.1.4 NS_INVEC_OFFSET	746
8.72.1.5 NS_OUTVEC_BIT	746
8.72.1.6 NS_OUTVEC_OFFSET	746
8.72.1.7 NS_VEC_DESC_BIT	746
8.72.1.8 OUT_LEN_MASK	747
8.72.1.9 OUT_LEN_OFFSET	747
8.72.1.10 PARAM_HAS_IOVEC	747
8.72.1.11 PARAM_IS_NS_INVEC	747
8.72.1.12 PARAM_IS_NS_OUTVEC	747
8.72.1.13 PARAM_IS_NS_VEC	747
8.72.1.14 PARAM_PACK	747
8.72.1.15 PARAM_SET_NS_INVEC	747
8.72.1.16 PARAM_SET_NS_OUTVEC	748
8.72.1.17 PARAM_SET_NS_VEC	748
8.72.1.18 PARAM_UNPACK_IN_LEN	748
8.72.1.19 PARAM_UNPACK_OUT_LEN	748
8.72.1.20 PARAM_UNPACK_TYPE	748
8.72.1.21 TYPE_MASK	748
8.72.2 Function Documentation	748
8.72.2.1 tfm_psa_call_pack()	748
8.73 interface/include/tfm_veneers.h File Reference	748
8.73.1 Function Documentation	750
8.73.1.1 tfm_ns_check_exit_veneer()	750
8.73.1.2 tfm_ns_check_safe_entry_veneer()	750
8.73.1.3 tfm_psa_call_veneer()	750
8.73.1.4 tfm_psa_close_veneer()	751
8.73.1.5 tfm_psa_connect_veneer()	751
8.73.1.6 tfm_psa_framework_version_veneer()	751
8.73.1.7 tfm_psa_version_veneer()	752
8.74 interface/src/hybrid_platform/api_broker.c File Reference	753
8.74.1 Function Documentation	754
8.74.1.1 psa_call()	754
8.74.1.2 psa_close()	755
8.74.1.3 psa_connect()	756
8.74.1.4 psa_framework_version()	756
8.74.1.5 psa_version()	756

8.74.1.6 tfm_hybrid_plat_api_broker_set_exec_target()	757
8.75 interface/src/multi_core/tfm_multi_core_ns_api.c File Reference	757
8.75.1 Function Documentation	758
8.75.1.1 tfm_ns_wait_for_s_cpu_ready()	758
8.76 interface/src/multi_core/tfm_multi_core_psa_ns_api.c File Reference	758
8.76.1 Macro Definition Documentation	759
8.76.1.1 NON_SECURE_CLIENT_ID	759
8.76.1.2 PSA_INTER_CORE_COMM_ERR	759
8.76.2 Function Documentation	759
8.76.2.1 PSA_CALL_MULTICORE()	759
8.76.2.2 PSA_CLOSE_MULTICORE()	759
8.76.2.3 PSA_CONNECT_MULTICORE()	759
8.76.2.4 PSA_FRAMEWORK_VERSION_MULTICORE()	760
8.76.2.5 PSA_VERSION_MULTICORE()	760
8.77 interface/src/multi_core/tfm_ns_mailbox.c File Reference	760
8.77.1 Function Documentation	761
8.77.1.1 mailbox_set_queue_ptr()	761
8.77.1.2 tfm_ns_mailbox_client_call()	761
8.78 interface/src/multi_core/tfm_ns_mailbox_common.c File Reference	762
8.78.1 Function Documentation	762
8.78.1.1 tfm_ns_mailbox_init()	762
8.78.2 Variable Documentation	763
8.78.2.1 cache_enabled	763
8.79 interface/src/multi_core/tfm_ns_mailbox_thread.c File Reference	763
8.79.1 Macro Definition Documentation	764
8.79.1.1 NOT_WOKEN	764
8.79.1.2 WOKEN_UP	764
8.79.2 Function Documentation	765
8.79.2.1 mailbox_set_queue_ptr()	765
8.79.2.2 tfm_ns_mailbox_client_call()	765
8.79.2.3 tfm_ns_mailbox_thread_runner()	766
8.79.2.4 tfm_ns_mailbox_wake_reply_owner_isr()	766
8.80 interface/src/os_wrapper/tfm_ns_interface_bare_metal.c File Reference	766
8.80.1 Function Documentation	767
8.80.1.1 tfm_ns_interface_dispatch()	767
8.80.1.2 tfm_ns_interface_init()	768
8.81 interface/src/os_wrapper/tfm_ns_interface_rtos.c File Reference	768
8.81.1 Function Documentation	769
8.81.1.1 tfm_ns_interface_dispatch()	769
8.81.1.2 tfm_ns_interface_init()	770
8.82 interface/src/tfm_attest_api.c File Reference	771
8.82.1 Function Documentation	771

8.82.1.1	psa_initial_attest_get_token()	771
8.82.1.2	psa_initial_attest_get_token_size()	772
8.83	interface/src/tfm_crypto_api.c File Reference	772
8.83.1	Macro Definition Documentation	775
8.83.1.1	API_DISPATCH	776
8.83.1.2	API_DISPATCH_NO_OUTVEC	776
8.83.1.3	TFM_CRYPTAPI	776
8.83.2	Function Documentation	776
8.83.2.1	psa_can_do_cipher()	776
8.83.2.2	psa_can_do_hash()	776
8.83.2.3	psa_cipher_generate_iv()	777
8.83.2.4	psa_cipher_set_iv()	777
8.83.2.5	psa_cipher_update()	777
8.84	interface/src/tfm_fwu_api.c File Reference	777
8.84.1	Function Documentation	778
8.84.1.1	psa_fwu_accept()	778
8.84.1.2	psa_fwu_cancel()	778
8.84.1.3	psa_fwu_clean()	779
8.84.1.4	psa_fwu_finish()	779
8.84.1.5	psa_fwu_install()	780
8.84.1.6	psa_fwu_query()	780
8.84.1.7	psa_fwu_reject()	781
8.84.1.8	psa_fwu_request_reboot()	781
8.84.1.9	psa_fwu_start()	782
8.84.1.10	psa_fwu_write()	782
8.85	interface/src/tfm_its_api.c File Reference	783
8.85.1	Function Documentation	784
8.85.1.1	psa_its_get()	784
8.85.1.2	psa_its_get_info()	785
8.85.1.3	psa_its_remove()	786
8.85.1.4	psa_its_set()	786
8.86	interface/src/tfm_platform_api.c File Reference	787
8.86.1	Function Documentation	788
8.86.1.1	tfm_platform_ioctl()	788
8.86.1.2	tfm_platform_nv_counter_increment()	789
8.86.1.3	tfm_platform_nv_counter_read()	790
8.86.1.4	tfm_platform_system_reset()	791
8.87	interface/src/tfm_ps_api.c File Reference	791
8.87.1	Function Documentation	792
8.87.1.1	psa_ps_create()	792
8.87.1.2	psa_ps_get()	793
8.87.1.3	psa_ps_get_info()	794

8.87.1.4	psa_ps_get_support()	795
8.87.1.5	psa_ps_remove()	795
8.87.1.6	psa_ps_set()	796
8.87.1.7	psa_ps_set_extended()	797
8.88	interface/src/tfm_psa_call.c File Reference	798
8.88.1	Function Documentation	799
8.88.1.1	psa_call()	799
8.89	interface/src/tfm_tz_psa_ns_api.c File Reference	800
8.89.1	Function Documentation	800
8.89.1.1	PSA_CALL_TZ()	800
8.89.1.2	PSA_CLOSE_TZ()	801
8.89.1.3	PSA_CONNECT_TZ()	801
8.89.1.4	PSA_FRAMEWORK_VERSION_TZ()	801
8.89.1.5	PSA_VERSION_TZ()	801
8.89.1.6	tfm_ns_check_exit()	802
8.89.1.7	tfm_ns_check_safe_entry()	802
8.90	platform/ext/target/infineon/common/device/include/cmsis.h File Reference	803
8.91	platform/ext/target/infineon/common/device/include/device_cfg.h File Reference	804
8.91.1	Macro Definition Documentation	804
8.91.1.1	IFX_IRQ_TEST_TIMER_S	804
8.92	platform/ext/target/infineon/common/device/include/device_definition.h File Reference	804
8.92.1	Typedef Documentation	805
8.92.1.1	ifx_timer_irq_test_dev_t	805
8.93	platform/ext/target/infineon/common/device/include/ifx_platform_startup.h File Reference	805
8.94	platform/ext/target/infineon/common/device/include/ifx_spm_init.h File Reference	807
8.94.1	Function Documentation	807
8.94.1.1	ifx_init_spm_peripherals()	807
8.95	platform/ext/target/infineon/common/device/include/ifx_startup.h File Reference	807
8.95.1	Macro Definition Documentation	809
8.95.1.1	DEFAULT_IRQ_HANDLER	809
8.95.1.2	IFX_VECTOR_TABLE_ATTRIBUTE	809
8.95.1.3	IS_POWER_OF_TWO	809
8.95.2	Typedef Documentation	809
8.95.2.1	VECTOR_TABLE_Type	809
8.95.3	Variable Documentation	809
8.95.3.1	__vector_table	809
8.96	platform/ext/target/infineon/common/device/src/device_definition.c File Reference	809
8.96.1	Detailed Description	810
8.97	platform/ext/target/infineon/common/device/src/ifx_spm_init.c File Reference	810
8.97.1	Function Documentation	811
8.97.1.1	ifx_init_spm_peripherals()	811
8.98	platform/ext/target/infineon/common/device/src/ifx_startup.c File Reference	811

8.98.1 Function Documentation	812
8.98.1.1 __PROGRAM_START()	812
8.98.1.2 Default_Handler()	812
8.98.1.3 Reset_Handler()	812
8.98.2 Variable Documentation	813
8.98.2.1 __INITIAL_SP	813
8.98.2.2 __STACK_LIMIT	813
8.98.2.3 IFX_VECTOR_TABLE_ATTRIBUTE	813
8.99 platform/ext/target/infineon/common/drivers/assets/ifx_assets_rram.c File Reference	813
8.99.1 Function Documentation	814
8.99.1.1 ifx_assets_rram_read_asset()	814
8.99.1.2 ifx_assets_rram_read_block()	815
8.100 platform/ext/target/infineon/common/drivers/assets/ifx_assets_rram.h File Reference	815
8.100.1 Macro Definition Documentation	816
8.100.1.1 IFX_ASSETS_RRAM_CHECKSUM_SIZE	816
8.100.2 Function Documentation	816
8.100.2.1 ifx_assets_rram_read_asset()	816
8.100.2.2 ifx_assets_rram_read_block()	817
8.101 platform/ext/target/infineon/common/drivers/flash/flash/ifx_driver_flash.c File Reference	817
8.101.1 Macro Definition Documentation	818
8.101.1.1 IFX_DRIVER_FLASH_EVENT	818
8.101.1.2 IFX_DRIVER_FLASH_FUNCTION_ARGUMENTS	818
8.101.1.3 IFX_DRIVER_FLASH_FUNCTION_PARAMETERS	818
8.101.1.4 IFX_DRIVER_FLASH_INSTANCE	819
8.101.1.5 IFX_DRIVER_FLASH_VERSION	819
8.101.1.6 IFX_FDF_ARGS	819
8.101.1.7 IFX_FDF_PARAMS	819
8.101.1.8 IFX_UNUSED_FLASH_DRIVER_INSTANCE	819
8.101.2 Variable Documentation	819
8.101.2.1 ifx_driver_flash	819
8.102 platform/ext/target/infineon/common/drivers/flash/flash/ifx_driver_flash.h File Reference	819
8.102.1 Macro Definition Documentation	821
8.102.1.1 IFX_DRIVER_FLASH_CREATE_INSTANCE	821
8.102.1.2 IFX_DRIVER_FLASH_ERASE_VALUE	821
8.102.1.3 IFX_DRIVER_FLASH_PROGRAM_UNIT	821
8.102.1.4 IFX_DRIVER_FLASH_SECTOR_SIZE	821
8.102.2 Variable Documentation	821
8.102.2.1 ifx_driver_flash	821
8.103 platform/ext/target/infineon/common/drivers/flash/flash/ifx_driver_private.c File Reference	821
8.103.1 Function Documentation	822
8.103.1.1 ifx_flash_driver_get_region_size()	822
8.103.1.2 ifx_flash_driver_initialize()	823

8.103.1.3 ifx_flash_driver_validate_region()	823
8.104 platform/ext/target/infineon/common/drivers/flash/ifx_driver_private.h File Reference	824
8.104.1 Function Documentation	825
8.104.1.1 ifx_flash_driver_get_region_size()	825
8.104.1.2 ifx_flash_driver_initialize()	826
8.104.1.3 ifx_flash_driver_validate_region()	826
8.105 platform/ext/target/infineon/common/drivers/flash/ifx_flash_driver_api.h File Reference	827
8.105.1 Macro Definition Documentation	828
8.105.1.1 IFX_CREATE_CMSIS_FLASH_DRIVER	828
8.105.2 Typedef Documentation	828
8.105.2.1 ifx_flash_driver_event_t	828
8.105.2.2 ifx_flash_driver_instance_t	828
8.105.2.3 ifx_flash_driver_t	829
8.105.3 Variable Documentation	829
8.105.3.1 ifx_flash_driver_handler_t	829
8.106 platform/ext/target/infineon/common/drivers/flash/rram/ifx_driver_rram.c File Reference	829
8.106.1 Macro Definition Documentation	830
8.106.1.1 IFX_DRIVER_RRAM_VERSION	830
8.106.2 Function Documentation	830
8.106.2.1 TFM_COVERITY_DEVIATE_LINE()	830
8.106.3 Variable Documentation	831
8.106.3.1 ifx_driver_rram	831
8.107 platform/ext/target/infineon/common/drivers/flash/rram/ifx_driver_rram.h File Reference	832
8.107.1 Macro Definition Documentation	833
8.107.1.1 IFX_DRIVER_RRAM_CREATE_INSTANCE	833
8.107.1.2 IFX_DRIVER_RRAM_ERASE_VALUE	833
8.107.1.3 IFX_DRIVER_RRAM_PC_LOCK_RETRIES	833
8.107.1.4 IFX_DRIVER_RRAM_PROGRAM_UNIT	833
8.107.2 Typedef Documentation	833
8.107.2.1 ifx_driver_rram_obj_t	833
8.107.3 Variable Documentation	834
8.107.3.1 ifx_driver_rram	834
8.108 platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif.c File Reference	834
8.108.1 Macro Definition Documentation	835
8.108.1.1 IFX_DRIVER_SMIF_VERSION	835
8.108.1.2 IFX_SMIF_SHORT_POLLING_RETRIES	835
8.108.2 Function Documentation	835
8.108.2.1 ifx_driver_smif_erase_chip()	835
8.108.2.2 ifx_driver_smif_erase_sector()	836
8.108.2.3 ifx_driver_smif_get_capabilities()	836
8.108.2.4 ifx_driver_smif_get_info()	837
8.108.2.5 ifx_driver_smif_get_status()	837

8.108.2.6 ifx_driver_smif_get_version()	837
8.108.2.7 ifx_driver_smif_initialize()	837
8.108.2.8 ifx_driver_smif_poll_block_busy()	838
8.108.2.9 ifx_driver_smif_power_control()	838
8.108.2.10 ifx_driver_smif_set_mode()	838
8.108.2.11 ifx_driver_smif_uninitialize()	839
8.108.2.12 ifx_driver_smif_validate_region()	839
8.109 platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif.h File Reference	840
8.109.1 Macro Definition Documentation	841
8.109.1.1 IFX_DRIVER_SMIF_MMIO_INSTANCE	841
8.109.1.2 IFX_DRIVER_SMIF_XIP_INSTANCE	842
8.109.2 Typedef Documentation	842
8.109.2.1 ifx_driver_smif_mem_t	842
8.109.2.2 ifx_driver_smif_obj_t	842
8.109.3 Variable Documentation	842
8.109.3.1 ifx_driver_smif_mmio	842
8.109.3.2 ifx_driver_smif_xip	842
8.110 platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif_mmio.c File Reference	842
8.110.1 Variable Documentation	843
8.110.1.1 ifx_driver_smif_mmio	843
8.111 platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif_private.h File Reference	843
8.111.1 Macro Definition Documentation	845
8.111.1.1 IFX_DRIVER_SMIF_DATA_WIDTH	845
8.111.2 Function Documentation	845
8.111.2.1 ifx_driver_smif_erase_chip()	845
8.111.2.2 ifx_driver_smif_erase_sector()	845
8.111.2.3 ifx_driver_smif_get_capabilities()	846
8.111.2.4 ifx_driver_smif_get_info()	846
8.111.2.5 ifx_driver_smif_get_status()	847
8.111.2.6 ifx_driver_smif_get_version()	847
8.111.2.7 ifx_driver_smif_initialize()	847
8.111.2.8 ifx_driver_smif_power_control()	848
8.111.2.9 ifx_driver_smif_set_mode()	848
8.111.2.10 ifx_driver_smif_uninitialize()	848
8.111.2.11 ifx_driver_smif_validate_region()	849
8.112 platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif_xip.c File Reference	849
8.112.1 Variable Documentation	850
8.112.1.1 ifx_driver_smif_xip	850
8.113 platform/ext/target/infineon/common/drivers/protection/mpu_armv8m_drv.h File Reference	850
8.113.1 Macro Definition Documentation	852
8.113.1.1 HARDFAULT_NMI_ENABLE	852
8.113.1.2 MPU_ARMV8M_MAIR_ATTR_CODE_IDX	852

8.113.1.3 MPU_ARMV8M_MAIR_ATTR_CODE_VAL	852
8.113.1.4 MPU_ARMV8M_MAIR_ATTR_DATA_IDX	852
8.113.1.5 MPU_ARMV8M_MAIR_ATTR_DATA_VAL	852
8.113.1.6 MPU_ARMV8M_MAIR_ATTR_DEVICE_IDX	852
8.113.1.7 MPU_ARMV8M_MAIR_ATTR_DEVICE_VAL	852
8.113.1.8 PRIVILEGED_DEFAULT_ENABLE	853
8.113.2 Enumeration Type Documentation	853
8.113.2.1 mpu_armv8m_attr_access_t	853
8.113.2.2 mpu_armv8m_attr_exec_t	853
8.113.2.3 mpu_armv8m_attr_shared_t	853
8.114 platform/ext/target/infineon/common/drivers/stdio/uart_pdl_stdout.c File Reference	853
8.114.1 Macro Definition Documentation	854
8.114.1.1 IFX_UART_USE_SPM_LOG_MSG	854
8.114.2 Function Documentation	854
8.114.2.1 stdio_init()	854
8.114.2.2 stdio_is_initialized()	855
8.114.2.3 stdio_is_initialized_reset()	855
8.114.2.4 stdio_output_string()	855
8.114.2.5 stdio_uninit()	856
8.115 platform/ext/target/infineon/common/drivers/stdio/uart_pdl_stdout.h File Reference	856
8.115.1 Function Documentation	857
8.115.1.1 stdio_output_string_raw()	857
8.116 platform/ext/target/infineon/common/interface/include/ifx_mtb_mailbox/ifx_mtb_mailbox.h File Reference	857
8.116.1 Macro Definition Documentation	859
8.116.1.1 IFX_MTB_MAILBOX_GENERIC_ERROR	859
8.116.1.2 IFX_MTB_MAILBOX_INVALID_PARAMS	859
8.116.1.3 IFX_MTB_MAILBOX_PSA_CALL	859
8.116.1.4 IFX_MTB_MAILBOX_PSA_CLOSE	859
8.116.1.5 IFX_MTB_MAILBOX_PSA_CONNECT	859
8.116.1.6 IFX_MTB_MAILBOX_PSA_FRAMEWORK_VERSION	859
8.116.1.7 IFX_MTB_MAILBOX_PSA_VERSION	859
8.116.1.8 IFX_MTB_MAILBOX_SUCCESS	859
8.116.2 Function Documentation	859
8.116.2.1 ifx_mtb_mailbox_client_call()	860
8.117 platform/ext/target/infineon/common/interface/include/ifx_platform_api.h File Reference	860
8.117.1 Macro Definition Documentation	861
8.117.1.1 IFX_PLATFORM_IOCTL_APP_MAX	862
8.117.1.2 IFX_PLATFORM_IOCTL_APP_MIN	862
8.117.2 Function Documentation	862
8.117.2.1 ifx_platform_log_msg()	862
8.118 platform/ext/target/infineon/common/interface/include/mtb_srf_ipc_custom_packet.h File Reference	863

8.118.1 Typedef Documentation	863
8.118.1.1 mtb_srf_ipc_packet_t	863
8.119 platform/ext/target/infineon/common/interface/src/ifx_mtb_mailbox/ifx_mtb_mailbox.c File Reference	864
8.119.1 Macro Definition Documentation	864
8.119.1.1 IFX_MTB_MAILBOX_CACHE_PRESENT	864
8.119.2 Function Documentation	864
8.119.2.1 ifx_mtb_mailbox_client_call()	865
8.120 platform/ext/target/infineon/common/interface/src/ifx_mtb_mailbox/ifx_mtb_mailbox_psa_ns_api.c File Reference	865
8.120.1 Function Documentation	866
8.120.1.1 psa_call()	866
8.120.1.2 psa_close()	868
8.120.1.3 psa_connect()	869
8.120.1.4 psa_framework_version()	870
8.120.1.5 psa_version()	870
8.121 platform/ext/target/infineon/common/interface/src/ifx_mtb_srf.c File Reference	871
8.121.1 Function Documentation	871
8.121.1.1 mtb_srf_request_submit()	872
8.122 platform/ext/target/infineon/common/interface/src/ifx_mtb_srf_relay.c File Reference	872
8.123 platform/ext/target/infineon/common/interface/src/ifx_platform_api.c File Reference	872
8.123.1 Function Documentation	873
8.123.1.1 ifx_platform_log_msg()	873
8.124 platform/ext/target/infineon/common/interface/src/ifx_platform_private.h File Reference	874
8.124.1 Macro Definition Documentation	875
8.124.1.1 IFX_CUSTOM_PLATFORM_HAL_IOCTL	875
8.124.1.2 IFX_PLATFORM_IOCTL_LOG_MSG_MAX	875
8.124.1.3 IFX_PLATFORM_IOCTL_LOG_MSG_MIN	875
8.125 platform/ext/target/infineon/common/libs/ifx_se_rt_services_utils/spe/ifx_se_tfm_utils.h File Reference	876
8.125.1 Detailed Description	876
8.125.2 Macro Definition Documentation	876
8.125.2.1 IFX_SE_TFM_SYSCALL_CONTEXT	877
8.126 platform/ext/target/infineon/common/libs/tf-psa-crypto/ifx_crypto_platform.h File Reference	877
8.126.1 Macro Definition Documentation	877
8.126.1.1 PSA_ALG_KDF_IFX_SE_AES_CMAC	878
8.126.1.2 PSA_CRYPTO_IFX_SE_ATTEST_PRIV_KEY_ID	878
8.126.1.3 PSA_CRYPTO_IFX_SE_ATTEST_PRIV_SLOT_NUMBER	878
8.126.1.4 PSA_CRYPTO_IFX_SE_ATTEST_PUB_KEY_ID	878
8.126.1.5 PSA_CRYPTO_IFX_SE_ATTEST_PUB_SLOT_NUMBER	878
8.126.1.6 PSA_CRYPTO_IFX_SE_DEVICE_KEY_ID	878
8.126.1.7 PSA_CRYPTO_IFX_SE_DEVICE_PRIV_SLOT_NUMBER	878
8.126.1.8 PSA_CRYPTO_IFX_SE_HUK_KEY_ID	878
8.126.1.9 PSA_CRYPTO_IFX_SE_HUK_SLOT_NUMBER	879

8.126.1.10	PSA_CRYPT0_IFX_SE_IFX_ROT_KEY_ID	879
8.126.1.11	PSA_CRYPT0_IFX_SE_IFX_ROT_SLOT_NUMBER	879
8.126.1.12	PSA_CRYPT0_IFX_SE_OEM_ROT_KEY_ID	879
8.126.1.13	PSA_CRYPT0_IFX_SE_OEM_ROT_SLOT_NUMBER	879
8.126.1.14	PSA_CRYPT0_IFX_SE_SERVICES_UPD_KEY_ID	879
8.126.1.15	PSA_CRYPT0_IFX_SE_SERVICES_UPD_SLOT_NUMBER	879
8.126.1.16	PSA_IFX_SE_KEY_DERIVATION_MAX_CAPACITY	879
8.126.1.17	PSA_KEY_LOCATION_IFX_SE	879
8.127	platform/ext/target/infineon/common/libs/tf-psa-crypto/afx_mbedtls_builitn_keys.c File Reference	880
8.127.1	Detailed Description	880
8.128	platform/ext/target/infineon/common/libs/tf-psa-crypto/platform_alt.h File Reference	880
8.128.1	Detailed Description	881
8.128.2	Macro Definition Documentation	882
8.128.2.1	MBEDTLS_PSA_KEY_ID_BUILTIN_MIN	882
8.128.3	Typedef Documentation	882
8.128.3.1	mbedtls_platform_context	882
8.129	platform/ext/target/infineon/common/nspe/mailbox/platform_ns_mailbox.c File Reference	882
8.129.1	Function Documentation	883
8.129.1.1	IFX_IPC_TFM_TO_NS_IPC_INTR()	883
8.129.1.2	tfm_ns_mailbox_hal_enter_critical()	883
8.129.1.3	tfm_ns_mailbox_hal_enter_critical_isr()	884
8.129.1.4	tfm_ns_mailbox_hal_exit_critical()	884
8.129.1.5	tfm_ns_mailbox_hal_exit_critical_isr()	885
8.129.1.6	tfm_ns_mailbox_hal_init()	885
8.129.1.7	tfm_ns_mailbox_hal_notify_peer()	886
8.129.1.8	tfm_ns_multi_core_boot()	886
8.129.1.9	tfm_platform_ns_wait_for_s_cpu_ready()	886
8.130	platform/ext/target/infineon/common/nspe/mailbox/tfm_ns_mailbox_rtos_api.c File Reference	887
8.130.1	Macro Definition Documentation	888
8.130.1.1	MAILBOX_THREAD_FLAG	888
8.130.1.2	MAX_SEMAPHORE_COUNT	888
8.130.2	Function Documentation	888
8.130.2.1	tfm_ns_mailbox_os_get_task_handle()	888
8.130.2.2	tfm_ns_mailbox_os_lock_acquire()	888
8.130.2.3	tfm_ns_mailbox_os_lock_init()	889
8.130.2.4	tfm_ns_mailbox_os_lock_release()	889
8.130.2.5	tfm_ns_mailbox_os_spin_lock()	889
8.130.2.6	tfm_ns_mailbox_os_spin_unlock()	889
8.130.2.7	tfm_ns_mailbox_os_wait_reply()	890
8.130.2.8	tfm_ns_mailbox_os_wake_task_isr()	890
8.131	platform/ext/target/infineon/common/nspe/os_wrapper/os_wrapper_cyabs_rtos.c File Reference	891
8.132	platform/ext/target/infineon/common/nspe/os_wrapper/semaphore.h File Reference	891

8.132.1 Function Documentation	892
8.132.1.1 os_wrapper_semaphore_acquire()	892
8.132.1.2 os_wrapper_semaphore_create()	892
8.132.1.3 os_wrapper_semaphore_delete()	893
8.132.1.4 os_wrapper_semaphore_release()	893
8.133 platform/ext/target/infineon/common/shared/ifx_stdio_core.h File Reference	893
8.133.1 Detailed Description	894
8.133.2 Macro Definition Documentation	894
8.133.2.1 IFX_STDIO_CORE_S_ID	894
8.133.3 Enumeration Type Documentation	894
8.133.3.1 ifx_stdio_core_id_t	894
8.134 platform/ext/target/infineon/common/shared/mailbox/ifx_platform_mailbox.h File Reference	895
8.134.1 Macro Definition Documentation	896
8.134.1.1 tfm_hal_remap_ns_cpu_address	896
8.135 platform/ext/target/infineon/common/shared/mailbox/platform_multicore.c File Reference	896
8.135.1 Macro Definition Documentation	897
8.135.1.1 IPC_RX_CHAN	897
8.135.1.2 IPC_RX_INT_MASK	897
8.135.1.3 IPC_RX_INTR_STRUCT	897
8.135.1.4 IPC_TX_CHAN	897
8.135.1.5 IPC_TX_NOTIFY_MASK	897
8.135.2 Function Documentation	897
8.135.2.1 ifx_mailbox_fetch_msg_data()	897
8.135.2.2 ifx_mailbox_fetch_msg_ptr()	898
8.135.2.3 ifx_mailbox_send_msg_data()	898
8.135.2.4 ifx_mailbox_send_msg_ptr()	899
8.135.2.5 ifx_mailbox_wait_for_notify()	899
8.136 platform/ext/target/infineon/common/shared/mailbox/platform_multicore.h File Reference	899
8.136.1 Macro Definition Documentation	901
8.136.1.1 IFX_IPC_SYNC_MAGIC	901
8.136.1.2 IFX_NS_MAILBOX_INIT_ENABLE	901
8.136.1.3 IFX_PLATFORM_MAILBOX_INIT_ERROR	901
8.136.1.4 IFX_PLATFORM_MAILBOX_INVALID_PARAMS	901
8.136.1.5 IFX_PLATFORM_MAILBOX_RX_ERROR	901
8.136.1.6 IFX_PLATFORM_MAILBOX_SUCCESS	901
8.136.1.7 IFX_PLATFORM_MAILBOX_TX_ERROR	901
8.136.1.8 IFX_PSA_CLIENT_CALL_REPLY_MAGIC	901
8.136.1.9 IFX_PSA_CLIENT_CALL_REQ_MAGIC	901
8.136.1.10 IFX_S_MAILBOX_READY	902
8.136.2 Function Documentation	902
8.136.2.1 ifx_mailbox_fetch_msg_data()	902
8.136.2.2 ifx_mailbox_fetch_msg_ptr()	902

8.136.2.3 ifx_mailbox_send_msg_data()	903
8.136.2.4 ifx_mailbox_send_msg_ptr()	903
8.136.2.5 ifx_mailbox_wait_for_notify()	903
8.137 platform/ext/target/infineon/common/shared/partition/ifx_common_flash_layout.h File Reference	904
8.137.1 Macro Definition Documentation	904
8.137.1.1 IFX_COMMON_FLASH_LAYOUT_H	904
8.138 platform/ext/target/infineon/common/shared/platform_nv_counters_ids.h File Reference	905
8.138.1 Macro Definition Documentation	905
8.138.1.1 PLAT_NV_COUNTER_NS_MAX	905
8.138.2 Enumeration Type Documentation	905
8.138.2.1 tfm_nv_counter_t	905
8.139 platform/ext/target/infineon/common/shared/tfm_peripherals_def_common.h File Reference	906
8.139.1 Macro Definition Documentation	906
8.139.1.1 DEFAULT_IRQ_PRIORITY	907
8.140 platform/ext/target/infineon/common/spe/faults/faults.c File Reference	907
8.140.1 Macro Definition Documentation	907
8.140.1.1 SCB_SHCSR_ENABLED_FAULTS	907
8.140.2 Function Documentation	908
8.140.2.1 FIH_RET_TYPE()	908
8.140.2.2 ifx_fault_irq_handler()	910
8.140.2.3 ifx_msc_fault_irq_handler()	911
8.140.2.4 IFX_TFM_FAULT_IRQ()	911
8.140.2.5 IFX_TFM_MSC_IRQ()	911
8.140.2.6 tfm_hal_platform_exception_info()	912
8.141 platform/ext/target/infineon/common/spe/faults/faults.h File Reference	912
8.141.1 Function Documentation	913
8.141.1.1 FIH_RET_TYPE()	913
8.142 platform/ext/target/infineon/common/spe/faults/faults_cat1b.c File Reference	917
8.143 platform/ext/target/infineon/common/spe/faults/faults_cat1d.c File Reference	917
8.144 platform/ext/target/infineon/common/spe/faults/faults_cat1e.c File Reference	918
8.145 platform/ext/target/infineon/common/spe/faults/faults_dump.h File Reference	918
8.145.1 Function Documentation	919
8.145.1.1 ifx_faults_dump_default_fault()	919
8.145.1.2 ifx_faults_dump_mpc_fault()	921
8.145.1.3 ifx_faults_dump_peri_gpx_ahb_vio_fault()	921
8.145.1.4 ifx_faults_dump_peri_gpx_timeout_vio_fault()	921
8.145.1.5 ifx_faults_dump_peri_msx_ppc_vio_fault()	921
8.145.1.6 ifx_faults_dump_peri_ppc_pc_mask_vio_fault()	922
8.146 platform/ext/target/infineon/common/spe/fih/tfm_fih_prng.c File Reference	922
8.146.1 Macro Definition Documentation	923
8.146.1.1 PRNG_A	923
8.146.1.2 PRNG_B	923

8.146.1.3 PRNG_C	923
8.146.2 Function Documentation	923
8.146.2.1 fih_delay_init()	923
8.146.2.2 fih_delay_random()	923
8.146.2.3 tfm_fih_random_generate()	924
8.147 platform/ext/target/infineon/common/spe/fih/tfm_fih_trng.h File Reference	924
8.147.1 Function Documentation	925
8.147.1.1 ifx_trng()	925
8.148 platform/ext/target/infineon/common/spe/fih/tfm_fih_trng_cryptolite.c File Reference	925
8.148.1 Function Documentation	925
8.148.1.1 ifx_trng()	925
8.149 platform/ext/target/infineon/common/spe/fih/tfm_fih_trng_mxcrypto.c File Reference	925
8.149.1 Function Documentation	926
8.149.1.1 ifx_trng()	926
8.150 platform/ext/target/infineon/common/spe/fih/tfm_fih_trng_mxtrng.c File Reference	926
8.150.1 Macro Definition Documentation	927
8.150.1.1 IFX_MXTRNG_STUB_RANDOM_VALUE	927
8.150.2 Function Documentation	927
8.150.2.1 ifx_trng()	927
8.151 platform/ext/target/infineon/common/spe/otp/otp_flash.c File Reference	927
8.151.1 Function Documentation	928
8.151.1.1 tfm_plat_otp_get_size()	928
8.151.1.2 tfm_plat_otp_init()	928
8.151.1.3 tfm_plat_otp_read()	928
8.151.1.4 tfm_plat_otp_write()	929
8.152 platform/ext/target/infineon/common/spe/platform_otp_ids.h File Reference	929
8.152.1 Enumeration Type Documentation	930
8.152.1.1 tfm_otp_element_id_t	930
8.153 platform/ext/target/infineon/common/spe/protection/partition_assets.h File Reference	930
8.154 platform/ext/target/infineon/common/spe/protection/partition_info.h File Reference	931
8.154.1 Function Documentation	932
8.154.1.1 ifx_protection_get_partition_info()	932
8.155 platform/ext/target/infineon/common/spe/protection/protection_data_common.c File Reference	932
8.155.1 Variable Documentation	933
8.155.1.1 ifx_secure_interrupts_config	933
8.155.1.2 ifx_secure_interrupts_config_count	933
8.156 platform/ext/target/infineon/common/spe/protection/protection_data_common.h File Reference	933
8.156.1 Variable Documentation	934
8.156.1.1 ifx_secure_interrupts_config	934
8.156.1.2 ifx_secure_interrupts_config_count	934
8.156.1.3 ifx_tfm_fault_sources	934
8.156.1.4 ifx_tfm_fault_sources_count	935

8.157 platform/ext/target/infineon/common/spe/protection/protection_mpc_api.h File Reference	935
8.157.1 Detailed Description	936
8.157.2 Function Documentation	936
8.157.2.1 FIH_RET_TYPE()	936
8.157.3 Variable Documentation	936
8.157.3.1 access_type	936
8.157.3.2 base	937
8.157.3.3 size	937
8.158 platform/ext/target/infineon/common/spe/protection/protection_mpc_hw_mpc.c File Reference . . .	937
8.158.1 Macro Definition Documentation	938
8.158.1.1 IFX_MPC_BLK_CFG_BLOCK_SIZE_Msk	938
8.158.1.2 IFX_MPC_BLK_CFG_BLOCK_SIZE_Pos	938
8.158.1.3 IFX_MPC_BLOCK_SIZE_TO_BYTES	938
8.158.2 Function Documentation	938
8.158.2.1 FIH_CALL()	938
8.158.2.2 FIH_RET() [1/2]	939
8.158.2.3 FIH_RET() [2/2]	940
8.158.2.4 FIH_RET_TYPE()	940
8.158.2.5 for()	943
8.158.2.6 if()	944
8.158.2.7 ifx_mpc_apply_configuration_with_mpc()	945
8.158.3 Variable Documentation	945
8.158.3.1 access_type	945
8.158.3.2 address	945
8.158.3.3 asset	945
8.158.3.4 base	946
8.158.3.5 i	946
8.158.3.6 ifx_fixed_mpc_static_config	946
8.158.3.7 ifx_fixed_mpc_static_config_count	946
8.158.3.8 mpc_reg_cfg	946
8.158.3.9 ns_mask	946
8.158.3.10 r_mask	946
8.158.3.11 res	946
8.158.3.12 size	946
8.158.3.13 split_count	947
8.158.3.14 w_mask	947
8.159 platform/ext/target/infineon/common/spe/protection/protection_mpc_hw_mpc.h File Reference . . .	947
8.159.1 Detailed Description	948
8.159.2 Macro Definition Documentation	948
8.159.2.1 IFX_MAX_FIXED_CONFIGS	948
8.159.2.2 IFX_MPC_APPLY_ALL_PCS	948
8.159.2.3 IFX_MPC_NAMED_MMIO_APP_ROT_CFG	948

8.159.2.4 IFX_MPC_NAMED_MMIO_PSA_ROT_CFG	948
8.159.2.5 IFX_MPC_NAMED_MMIO_SPM_ROT_CFG	949
8.159.3 Function Documentation	949
8.159.3.1 FIH_RET_TYPE()	949
8.159.3.2 ifx_mpc_apply_configuration_with_mpc()	966
8.159.4 Variable Documentation	966
8.159.4.1 ifx_fixed_mpc_static_config	966
8.159.4.2 ifx_fixed_mpc_static_config_count	967
8.160 platform/ext/target/infineon/common/spe/protection/protection_mpc_sert.c File Reference	967
8.160.1 Macro Definition Documentation	968
8.160.1.1 IFX_MPC_EXT_CACHE_ATTR_DEFAULT	968
8.160.1.2 IFX_MPC_SERT_DEFAULT_ATTR	968
8.160.2 Function Documentation	968
8.160.2.1 FIH_RET()	968
8.160.2.2 FIH_RET_TYPE()	969
8.160.2.3 for()	969
8.160.2.4 if() [1/5]	969
8.160.2.5 if() [2/5]	969
8.160.2.6 if() [3/5]	970
8.160.2.7 if() [4/5]	970
8.160.2.8 if() [5/5]	970
8.160.2.9 ifx_mpc_sert_apply_configuration()	971
8.160.3 Variable Documentation	971
8.160.3.1 access_type	971
8.160.3.2 attr	971
8.160.3.3 base	971
8.160.3.4 block_count	971
8.160.3.5 block_size	971
8.160.3.6 first_block_idx	972
8.160.3.7 is_secure	972
8.160.3.8 last_block_idx	972
8.160.3.9 mask	972
8.160.3.10 memory_config	972
8.160.3.11 offset	972
8.160.3.12 pc	972
8.160.3.13 size	972
8.161 platform/ext/target/infineon/common/spe/protection/protection_mpc_sert.h File Reference	972
8.161.1 Detailed Description	974
8.161.2 Macro Definition Documentation	974
8.161.2.1 IFX_MPC_EXT_BLOCK_SIZE_TO_BYTES	974
8.161.2.2 IFX_MPC_EXT_CACHE_NS_BIT	974
8.161.2.3 IFX_MPC_EXT_CACHE_PC_BIT	974

8.161.2.4 IFX_MPC_EXT_CACHE_PC_FIELD_BITS	974
8.161.2.5 IFX_MPC_EXT_CACHE_PC_FIELD_SHIFT	974
8.161.2.6 IFX_MPC_EXT_CACHE_READ_BIT	975
8.161.2.7 IFX_MPC_EXT_CACHE_WRITE_BIT	975
8.161.2.8 IFX_MPC_SERT_CACHE_SIZE	975
8.161.3 Function Documentation	975
8.161.3.1 FIH_RET_TYPE()	975
8.161.3.2 ifx_mpc_sert_apply_configuration()	976
8.161.4 Variable Documentation	976
8.161.4.1 access_type	976
8.161.4.2 base	976
8.161.4.3 ifx_mpc_sert_memories	976
8.161.4.4 ifx_mpc_sert_memories_count	976
8.161.4.5 memory_config	976
8.161.4.6 size	976
8.162 platform/ext/target/infineon/common/spe/protection/protection_mpc_sw_policy.c File Reference	977
8.162.1 Macro Definition Documentation	978
8.162.1.1 CY_FLASH_BASE_S	978
8.162.1.2 CY_SRAM0_BASE_S	978
8.162.1.3 IFX_MPC_BLOCK_SIZE	978
8.162.1.4 IFX_MPC_GET_PC_ATTR	978
8.162.1.5 IFX_MPC_PC_ATTR_MSK	978
8.162.1.6 IFX_MPC_PC_ATTR_NS_MSK	978
8.162.1.7 IFX_MPC_PC_ATTR_R_MSK	978
8.162.1.8 IFX_MPC_PC_ATTR_SIZE_IN_BITS	979
8.162.1.9 IFX_MPC_PC_ATTR_W_MSK	979
8.162.2 Function Documentation	979
8.162.2.1 FIH_RET()	979
8.162.2.2 FIH_RET_TYPE()	979
8.162.2.3 for()	982
8.162.2.4 if() [1/2]	983
8.162.2.5 if() [2/2]	983
8.162.2.6 TFM_COVERITY_DEVIATE_LINE()	983
8.162.3 Variable Documentation	983
8.162.3.1 access_type	983
8.162.3.2 base	983
8.162.3.3 base_s	984
8.162.3.4 else	984
8.162.3.5 mpc_base_addr	984
8.162.3.6 mpc_end_idx	984
8.162.3.7 mpc_policy	984
8.162.3.8 size	984

8.163 platform/ext/target/infineon/common/spe/protection/protection_mpc_sw_policy.h File Reference	984
8.163.1 Detailed Description	984
8.163.2 Macro Definition Documentation	985
8.163.2.1 IFX_MPC_NAMED_MMIO_APP_ROT_CFG	985
8.163.2.2 IFX_MPC_NAMED_MMIO_PSA_ROT_CFG	985
8.163.2.3 IFX_MPC_NAMED_MMIO_SPM_ROT_CFG	985
8.164 platform/ext/target/infineon/common/spe/protection/protection_mpu.c File Reference	985
8.164.1 Function Documentation	986
8.164.1.1 FIH_RET()	986
8.164.1.2 FIH_RET_TYPE()	986
8.164.1.3 if()	989
8.164.1.4 ifx_mpu_config_enable_region()	989
8.164.2 Variable Documentation	989
8.164.2.1 attr_access	989
8.164.2.2 attr_exec	989
8.164.2.3 attr_sh	989
8.164.2.4 else	990
8.164.2.5 p_asset	990
8.164.2.6 region_base	990
8.164.2.7 region_limit	990
8.165 platform/ext/target/infineon/common/spe/protection/protection_mpu.h File Reference	990
8.165.1 Function Documentation	991
8.165.1.1 FIH_RET_TYPE()	991
8.165.2 Variable Documentation	992
8.165.2.1 p_asset	992
8.166 platform/ext/target/infineon/common/spe/protection/protection_msc.c File Reference	992
8.166.1 Function Documentation	992
8.166.1.1 FIH_RET_TYPE()	992
8.167 platform/ext/target/infineon/common/spe/protection/protection_msc.h File Reference	996
8.167.1 Function Documentation	997
8.167.1.1 FIH_RET_TYPE()	997
8.168 platform/ext/target/infineon/common/spe/protection/protection_pc.c File Reference	1014
8.168.1 Macro Definition Documentation	1015
8.168.1.1 IFX_RETURN_ON_EXCEPTION_BIT_MASK	1015
8.168.1.2 IFX_VTOR_ADDRESS	1015
8.168.2 Function Documentation	1015
8.168.2.1 FIH_RET_TYPE()	1015
8.168.2.2 ifx_arch_switch_protection_context()	1019
8.168.2.3 ifx_arch_switch_protection_context_from_irq()	1020
8.168.2.4 ifx_arch_switch_protection_context_thread()	1020
8.168.2.5 ifx_pc2_exception_handler()	1020
8.168.2.6 TFM_COVERITY_DEVIATE_LINE()	1021

8.168.3 Variable Documentation	1021
8.168.3.1 ifx_arch_active_protection_context	1021
8.168.3.2 ifx_pc2_vector_table	1021
8.169 platform/ext/target/infineon/common/spe/protection/protection_pc.h File Reference	1021
8.169.1 Function Documentation	1022
8.169.1.1 FIH_RET_TYPE()	1022
8.169.1.2 ifx_arch_switch_protection_context_thread()	1039
8.169.2 Variable Documentation	1040
8.169.2.1 ifx_arch_active_protection_context	1040
8.170 platform/ext/target/infineon/common/spe/protection/protection_pc_mask.h File Reference	1040
8.170.1 Macro Definition Documentation	1041
8.170.1.1 IFX_GET_PC	1041
8.170.2 Typedef Documentation	1041
8.170.2.1 ifx_pc_mask_t	1041
8.171 platform/ext/target/infineon/common/spe/protection/protection_ppc_api.h File Reference	1041
8.171.1 Detailed Description	1042
8.171.2 Function Documentation	1042
8.171.2.1 FIH_RET_TYPE()	1042
8.171.3 Variable Documentation	1043
8.171.3.1 asset	1043
8.172 platform/ext/target/infineon/common/spe/protection/protection_ppc_v1.c File Reference	1043
8.172.1 Function Documentation	1044
8.172.1.1 fih_delay()	1044
8.172.1.2 FIH_RET()	1045
8.172.1.3 if() [1/3]	1046
8.172.1.4 if() [2/3]	1046
8.172.1.5 if() [3/3]	1046
8.172.1.6 ifx_check_config_validity()	1047
8.172.2 Variable Documentation	1047
8.172.2.1 asset	1047
8.172.2.2 ppc_attribute	1047
8.172.2.3 ppc_pcmask	1047
8.173 platform/ext/target/infineon/common/spe/protection/protection_ppc_v1.h File Reference	1047
8.173.1 Detailed Description	1048
8.173.2 Macro Definition Documentation	1048
8.173.2.1 IFX_PPC_NAMED_MMIO_APP_ROT_CFG	1048
8.173.2.2 IFX_PPC_NAMED_MMIO_PSA_ROT_CFG	1048
8.173.2.3 IFX_PPC_NAMED_MMIO_SPM_ROT_CFG	1049
8.174 platform/ext/target/infineon/common/spe/protection/protection_ppc_v2.c File Reference	1049
8.174.1 Function Documentation	1049
8.174.1.1 fih_delay()	1050
8.174.1.2 FIH_RET()	1050

8.174.1.3 if() [1/3]	1050
8.174.1.4 if() [2/3]	1050
8.174.1.5 if() [3/3]	1051
8.174.1.6 ifx_check_config_validity()	1051
8.174.2 Variable Documentation	1051
8.174.2.1 asset	1051
8.174.2.2 ppc_attribute	1051
8.174.2.3 ppc_pcmask	1051
8.175 platform/ext/target/infineon/common/spe/protection/protection_ppc_v2.h File Reference	1052
8.175.1 Detailed Description	1052
8.175.2 Macro Definition Documentation	1052
8.175.2.1 IFX_PPC_NAMED_MMIO_APP_ROT_CFG	1052
8.175.2.2 IFX_PPC_NAMED_MMIO_PSA_ROT_CFG	1053
8.175.2.3 IFX_PPC_NAMED_MMIO_SPM_ROT_CFG	1053
8.176 platform/ext/target/infineon/common/spe/protection/protection_sau.c File Reference	1053
8.176.1 Macro Definition Documentation	1053
8.176.1.1 IFX_SAU_RESERVED_REGION_COUNT	1054
8.176.1.2 IFX_SAU_VENEER_REGION_COUNT	1054
8.176.2 Function Documentation	1054
8.176.2.1 FIH_RET_TYPE()	1054
8.177 platform/ext/target/infineon/common/spe/protection/protection_sau.h File Reference	1058
8.177.1 Function Documentation	1059
8.177.1.1 FIH_RET_TYPE()	1059
8.178 platform/ext/target/infineon/common/spe/protection/protection_shared_data.c File Reference	1076
8.178.1 Variable Documentation	1077
8.178.1.1 ifx_spm_state	1077
8.178.1.2 ifx_spm_state_inv	1077
8.179 platform/ext/target/infineon/common/spe/protection/protection_shared_data.h File Reference	1078
8.179.1 Macro Definition Documentation	1079
8.179.1.1 IFX_IS_SPM_INITIALIZING	1079
8.179.1.2 IFX_IS_SPM_RUNNING	1079
8.179.1.3 IFX_SET_SPM_STATE	1079
8.179.1.4 IFX_SPM_STATE_INITIALIZING	1079
8.179.1.5 IFX_SPM_STATE_RUNNING	1079
8.179.2 Variable Documentation	1079
8.179.2.1 ifx_spm_state	1079
8.179.2.2 ifx_spm_state_inv	1080
8.180 platform/ext/target/infineon/common/spe/protection/protection_types.h File Reference	1080
8.180.1 Macro Definition Documentation	1081
8.180.1.1 IFX_ASSET_ATTR_COMBINE	1081
8.180.1.2 IFX_ASSET_ATTR_IS_SET	1081
8.180.1.3 IFX_GET_BOUNDARY_DOMAIN	1081

8.180.1.4 IFX_GET_PARTITION_PC	1082
8.180.1.5 IFX_IS_PARTITION_PRIVILEGED	1082
8.180.1.6 IFX_PROTECT_APP_ROT_DOMAIN_CFG	1082
8.180.1.7 IFX_PROTECT_APP_ROT_SINGLE_DOMAIN	1082
8.180.1.8 IFX_PROTECT_IS_DOMAIN_EQUAL	1082
8.180.1.9 IFX_PROTECT_IS_PRIVILEGED_DOMAIN	1082
8.180.1.10 IFX_PROTECT_IS_PRIVILEGED_DOMAIN_PRIVATE	1082
8.180.1.11 IFX_PROTECT_PSA_ROT_DOMAIN_CFG	1083
8.180.1.12 IFX_PROTECT_PSA_ROT_SINGLE_DOMAIN	1083
8.180.1.13 IFX_PROTECT_SPM_DOMAIN	1083
8.180.1.14 IFX_PROTECT_SPM_DOMAIN_CFG	1083
8.180.1.15 IFX_SPM_BOUNDARY	1083
8.180.2 Typedef Documentation	1083
8.180.2.1 ifx_partition_info_t	1083
8.180.2.2 ifx_ppc_regions_config_t	1083
8.181 platform/ext/target/infineon/common/spe/protection/protection_tz.c File Reference	1083
8.181.1 Function Documentation	1084
8.181.1.1 fih_delay()	1085
8.181.1.2 FIH_RET() [1/2]	1085
8.181.1.3 FIH_RET() [2/2]	1085
8.181.1.4 if() [1/8]	1085
8.181.1.5 if() [2/8]	1085
8.181.1.6 if() [3/8]	1085
8.181.1.7 if() [4/8]	1086
8.181.1.8 if() [5/8]	1086
8.181.1.9 if() [6/8]	1086
8.181.1.10 if() [7/8]	1087
8.181.1.11 if() [8/8]	1087
8.181.1.12 switch() [1/2]	1087
8.181.1.13 switch() [2/2]	1088
8.181.2 Variable Documentation	1088
8.181.2.1 access_type	1088
8.181.2.2 base	1088
8.181.2.3 else	1088
8.181.2.4 flags	1088
8.181.2.5 is_non_secure	1088
8.181.2.6 known	1089
8.181.2.7 known_secure_level	1089
8.181.2.8 pb	1089
8.181.2.9 pe	1089
8.181.2.10 perme	1089
8.181.2.11 singleCheck	1089

8.181.2.12 size	1089
8.182 platform/ext/target/infineon/common/spe/protection/protection_tz.h File Reference	1089
8.182.1 Function Documentation	1090
8.182.1.1 FIH_RET_TYPE()	1090
8.182.2 Variable Documentation	1091
8.182.2.1 access_type	1091
8.182.2.2 base	1091
8.182.2.3 size	1091
8.183 platform/ext/target/infineon/common/spe/protection/protection_utils.c File Reference	1091
8.184 platform/ext/target/infineon/common/spe/protection/protection_utils.h File Reference	1092
8.185 platform/ext/target/infineon/common/spe/protection/tfm_hal_isolation.c File Reference	1092
8.185.1 Function Documentation	1094
8.185.1.1 __DMB()	1094
8.185.1.2 __DSB()	1094
8.185.1.3 __ISB()	1095
8.185.1.4 __set_CONTROL()	1095
8.185.1.5 FIH_CALL() [1/3]	1095
8.185.1.6 FIH_CALL() [2/3]	1095
8.185.1.7 FIH_CALL() [3/3]	1096
8.185.1.8 FIH_RET() [1/3]	1096
8.185.1.9 FIH_RET() [2/3]	1096
8.185.1.10 FIH_RET() [3/3]	1096
8.185.1.11 FIH_RET_TYPE()	1097
8.185.1.12 for() [1/2]	1097
8.185.1.13 for() [2/2]	1098
8.185.1.14 if() [1/9]	1098
8.185.1.15 if() [2/9]	1099
8.185.1.16 if() [3/9]	1099
8.185.1.17 if() [4/9]	1099
8.185.1.18 if() [5/9]	1100
8.185.1.19 if() [6/9]	1100
8.185.1.20 if() [7/9]	1100
8.185.1.21 if() [8/9]	1101
8.185.1.22 if() [9/9]	1101
8.185.1.23 TFM_COVERITY_DEVIATE_BLOCK()	1102
8.185.1.24 TFM_COVERITY_DEVIATE_LINE()	1103
8.185.2 Variable Documentation	1103
8.185.2.1 access_type	1103
8.185.2.2 asset	1103
8.185.2.3 asset_cnt	1103
8.185.2.4 base	1103
8.185.2.5 boundary	1104

8.185.2.6	cfg	1104
8.185.2.7	ctrl	1104
8.185.2.8	else	1104
8.185.2.9	p_assets	1104
8.185.2.10	p_boundary	1104
8.185.2.11	p_info	1104
8.185.2.12	p_ldinf	1104
8.185.2.13	size	1105
8.186	platform/ext/target/infineon/common/spe/protection/tfm_platform_arch_hooks.c File Reference	1105
8.186.1	Function Documentation	1105
8.186.1.1	ifx_arch_get_context_protection_context()	1105
8.186.1.2	ifx_arch_get_current_component_protection_context()	1105
8.187	platform/ext/target/infineon/common/spe/protection/tfm_platform_arch_hooks.h File Reference	1106
8.187.1	Macro Definition Documentation	1107
8.187.1.1	IFX_PLATFORM_EXIT_FROM_INTERRUPT_WITH_PC_RESTORE_HOOK	1107
8.187.1.2	PLATFORM_HARD_FAULT_HANDLER_HOOK	1107
8.187.1.3	PLATFORM_HARD_FAULT_HANDLER_HOOK_IAR_REQUIRED	1107
8.187.1.4	PLATFORM_HARD_FAULT_HANDLER_HOOK_VALIDATE_PC	1107
8.187.1.5	PLATFORM_INIT_SPM_FUNC_CONTEXT_HOOK	1108
8.187.1.6	PLATFORM_PENDSV_HANDLER_EXIT_HOOK	1108
8.187.1.7	PLATFORM_PENDSV_HANDLER_HOOK_IAR_REQUIRED	1108
8.187.1.8	PLATFORM_SVC_HANDLER_EXIT_HOOK	1108
8.187.1.9	PLATFORM_SVC_HANDLER_FROM_FLIH_FUNC_ENTER_HOOK	1108
8.187.1.10	PLATFORM_SVC_HANDLER_FROM_FLIH_FUNC_EXIT_HOOK	1109
8.187.1.11	PLATFORM_SVC_HANDLER_GET_MSP_HOOK	1109
8.187.1.12	PLATFORM_SVC_HANDLER_HOOK_IAR_REQUIRED	1109
8.187.1.13	PLATFORM_SVC_HANDLER_TO_FLIH_FUNC_ENTER_HOOK	1109
8.187.1.14	PLATFORM_SVC_HANDLER_TO_FLIH_FUNC_EXIT_HOOK	1109
8.187.1.15	PLATFORM_THREAD_MODE_SPM_RETURN_HOOK	1109
8.187.2	Function Documentation	1110
8.187.2.1	ifx_arch_get_context_protection_context()	1110
8.187.2.2	ifx_arch_get_current_component_protection_context()	1110
8.187.2.3	ifx_arch_switch_protection_context()	1110
8.187.2.4	ifx_arch_switch_protection_context_from_irq()	1110
8.188	platform/ext/target/infineon/common/spe/provisioning/ifx_dap.c File Reference	1111
8.189	platform/ext/target/infineon/common/spe/provisioning/provisioning.c File Reference	1111
8.189.1	Function Documentation	1112
8.189.1.1	ifx_get_security_lifecycle()	1112
8.190	platform/ext/target/infineon/common/spe/provisioning/provisioning.h File Reference	1112
8.190.1	Enumeration Type Documentation	1113
8.190.1.1	ifx_dap_state_t	1113
8.190.2	Function Documentation	1114

8.190.2.1 ifx_get_dap_state()	1114
8.190.2.2 ifx_get_security_lifecycle()	1114
8.191 platform/ext/target/infineon/common/spe/services/attestation/ifx_attest_hal.c File Reference	1115
8.191.1 Macro Definition Documentation	1116
8.191.1.1 BOOL_SEED_INITIALIZED	1116
8.191.1.2 BOOL_SEED_UNINITIALIZED	1116
8.191.1.3 IFX_ATTESTATION_KEY_ID	1116
8.191.1.4 IFX_ATTESTATION_PROFILE_DEFINITION	1116
8.191.1.5 IFX_ATTESTATION_VERIFICATION_SERVICE	1116
8.191.2 Function Documentation	1116
8.191.2.1 tfm_attest_hal_get_profile_definition()	1116
8.191.2.2 tfm_attest_hal_get_security_lifecycle()	1117
8.191.2.3 tfm_attest_hal_get_verification_service()	1117
8.191.2.4 tfm_plat_get_boot_seed()	1117
8.192 platform/ext/target/infineon/common/spe/services/attestation/ifx_attest_hal_se_rt.c File Reference	1118
8.192.1 Function Documentation	1118
8.192.1.1 tfm_attest_hal_get_token()	1118
8.192.1.2 tfm_attest_hal_get_token_size()	1119
8.193 platform/ext/target/infineon/common/spe/services/attestation/ifx_plat_device_id.c File Reference	1119
8.193.1 Macro Definition Documentation	1119
8.193.1.1 IFX_ADD_TO_IMPLEMENTATION_ID	1119
8.193.1.2 IFX_ATTESTATION_HW_VERSION	1120
8.193.2 Function Documentation	1120
8.193.2.1 tfm_plat_get_cert_ref()	1120
8.193.2.2 tfm_plat_get_implementation_id()	1120
8.194 platform/ext/target/infineon/common/spe/services/crypto/crypto_keys_flash.c File Reference	1120
8.194.1 Function Documentation	1121
8.194.1.1 tfm_plat_builtin_key_get_desc_table_ptr()	1121
8.194.1.2 tfm_plat_builtin_key_get_policy_table_ptr()	1122
8.195 platform/ext/target/infineon/common/spe/services/crypto/crypto_keys_rram.c File Reference	1122
8.195.1 Macro Definition Documentation	1122
8.195.1.1 MAPPED_MAILBOX_NS_AGENT_DEFAULT_CLIENT_ID	1123
8.195.1.2 MAPPED_TZ_NS_AGENT_DEFAULT_CLIENT_ID	1123
8.195.1.3 NUMBER_OF_ELEMENTS_OF	1123
8.195.1.4 TFM_MBOX_NS_PARTITION_ID	1123
8.195.1.5 TFM_TZ_NS_PARTITION_ID	1123
8.195.2 Function Documentation	1123
8.195.2.1 tfm_plat_builtin_key_get_desc_table_ptr()	1123
8.195.2.2 tfm_plat_builtin_key_get_policy_table_ptr()	1123
8.196 platform/ext/target/infineon/common/spe/services/crypto/crypto_keys_se_rt.c File Reference	1123
8.196.1 Macro Definition Documentation	1124
8.196.1.1 NUMBER_OF_ELEMENTS_OF	1124

8.196.2 Function Documentation	1124
8.196.2.1 tfm_plat_builtin_key_get_desc_table_ptr()	1124
8.197 platform/ext/target/infineon/common/spe/services/crypto/crypto_nv_seed.c File Reference	1124
8.197.1 Function Documentation	1125
8.197.1.1 tfm_plat_crypto_nv_seed_write()	1125
8.197.1.2 tfm_plat_crypto_provision_entropy_seed()	1125
8.198 platform/ext/target/infineon/common/spe/services/crypto/crypto_nv_seed_cryptolite.c File Reference	1125
8.198.1 Function Documentation	1126
8.198.1.1 tfm_plat_crypto_nv_seed_read()	1126
8.199 platform/ext/target/infineon/common/spe/services/crypto/crypto_nv_seed_mxcrypto.c File Reference	1126
8.199.1 Function Documentation	1127
8.199.1.1 tfm_plat_crypto_nv_seed_read()	1127
8.200 platform/ext/target/infineon/common/spe/services/crypto/crypto_nv_seed_mxtrng.c File Reference	1127
8.200.1 Function Documentation	1128
8.200.1.1 tfm_plat_crypto_nv_seed_read()	1128
8.201 platform/ext/target/infineon/common/spe/services/crypto/crypto_rnd_se_rt.c File Reference	1128
8.202 platform/ext/target/infineon/common/spe/services/crypto/mbedtls_accel_configs/crypto_hw_cryptolite_config.h File Reference	1129
8.203 platform/ext/target/infineon/common/spe/services/crypto/mbedtls_accel_configs/crypto_hw_mxcrypto_config.h File Reference	1129
8.204 platform/ext/target/infineon/common/spe/services/its/drivers/its_flash_driver.c File Reference	1129
8.205 platform/ext/target/infineon/common/spe/services/its/drivers/its_rram_driver.c File Reference	1129
8.206 platform/ext/target/infineon/common/spe/services/its/its_hal.c File Reference	1130
8.206.1 Function Documentation	1130
8.206.1.1 tfm_hal_its_fs_info()	1130
8.207 platform/ext/target/infineon/common/spe/services/mailbox/platform_hal_multi_core_cm55.c File Reference	1131
8.207.1 Macro Definition Documentation	1131
8.207.1.1 IFX_IS_REGION_IN_DTCM_MEMORY	1131
8.207.1.2 IFX_IS_REGION_IN_DTCM_MEMORY_REMAPPED	1131
8.207.1.3 IFX_IS_REGION_IN_ITCM_MEMORY	1132
8.207.1.4 IFX_IS_REGION_IN_ITCM_MEMORY_REMAPPED	1132
8.208 platform/ext/target/infineon/common/spe/services/mailbox/platform_spe_mailbox.c File Reference	1132
8.208.1 Macro Definition Documentation	1133
8.208.1.1 MAILBOX_CLEAN_CACHE	1133
8.208.1.2 MAILBOX_INVALIDATE_CACHE	1133
8.208.2 Function Documentation	1133
8.208.2.1 tfm_mailbox_hal_deinit()	1133
8.208.2.2 tfm_mailbox_hal_enter_critical()	1133
8.208.2.3 tfm_mailbox_hal_exit_critical()	1133
8.208.2.4 tfm_mailbox_hal_init()	1134
8.208.2.5 tfm_mailbox_hal_notify_peer()	1134
8.209 platform/ext/target/infineon/common/spe/services/mailbox/tfm_hal_multi_core.c File Reference	1134

8.209.1 Function Documentation	1135
8.209.1.1 tfm_hal_wait_for_ns_cpu_ready()	1135
8.210 platform/ext/target/infineon/common/spe/services/mailbox/tfm_interrupts.c File Reference	1135
8.210.1 Function Documentation	1136
8.210.1.1 IFX_IPC_NS_TO_TFM_IPC_INTR()	1136
8.210.1.2 mailbox_irq_init()	1136
8.211 platform/ext/target/infineon/common/spe/services/platform/drivers/nv_counters_flash_driver.c File Reference	1136
8.211.1 Macro Definition Documentation	1136
8.211.1.1 IFX_TFM_NV_COUNTERS_ERASE_VALUE	1137
8.212 platform/ext/target/infineon/common/spe/services/platform/drivers/nv_counters_flash_driver.h File Reference	1137
8.212.1 Macro Definition Documentation	1138
8.212.1.1 IFX_NV_COUNTERS_CMSIS_FLASH_INSTANCE	1138
8.212.1.2 IFX_TFM_NV_COUNTERS_PROGRAM_UNIT	1138
8.212.1.3 IFX_TFM_NV_COUNTERS_SECTOR_SIZE	1138
8.212.2 Variable Documentation	1138
8.212.2.1 ifx_nv_counters_cmsis_flash_instance	1138
8.213 platform/ext/target/infineon/common/spe/services/platform/drivers/nv_counters_rram_driver.c File Reference	1138
8.213.1 Macro Definition Documentation	1139
8.213.1.1 IFX_TFM_NV_COUNTERS_ERASE_VALUE	1139
8.214 platform/ext/target/infineon/common/spe/services/platform/drivers/nv_counters_rram_driver.h File Reference	1139
8.214.1 Macro Definition Documentation	1140
8.214.1.1 IFX_NV_COUNTERS_CMSIS_FLASH_INSTANCE	1140
8.214.1.2 IFX_TFM_NV_COUNTERS_PROGRAM_UNIT	1140
8.214.1.3 IFX_TFM_NV_COUNTERS_SECTOR_SIZE	1140
8.214.2 Variable Documentation	1140
8.214.2.1 ifx_nv_counters_cmsis_flash_instance	1140
8.215 platform/ext/target/infineon/common/spe/services/platform/nv_counters_common.c File Reference	1141
8.215.1 Function Documentation	1141
8.215.1.1 tfm_plat_ns_counter_idx_to_nv_counter()	1141
8.215.1.2 tfm_plat_nv_counter_permissions_check()	1141
8.216 platform/ext/target/infineon/common/spe/services/platform/nv_counters_flash.c File Reference	1141
8.216.1 Macro Definition Documentation	1143
8.216.1.1 IFX_NUM_NV_COUNTERS	1143
8.216.1.2 IFX_NV_COUNTER_SIZE	1143
8.216.1.3 IFX_NV_COUNTERS_BLOCK_SIZE	1143
8.216.1.4 IFX_NV_COUNTERS_CONTENT_SIZE	1143
8.216.1.5 IFX_NV_COUNTERS_CRC_INIT	1143
8.216.1.6 IFX_NV_COUNTERS_INITIALIZED	1143
8.216.1.7 IFX_NV_INIT_DONE_BLOCK_SIZE	1143

8.216.1.8 IFX_NVM_INIT_DONE_FLAG	1143
8.216.1.9 IFX_TFM_NV_COUNTERS_AREA_OFFSET	1144
8.216.1.10 IFX_TFM_NV_COUNTERS_BACKUP_OFFSET	1144
8.216.1.11 IFX_TFM_NV_COUNTERS_INIT_DONE_OFFSET	1144
8.216.2 Typedef Documentation	1144
8.216.2.1 ifx_nv_counters_t	1144
8.216.2.2 ifx_nv_init_done_t	1144
8.216.3 Function Documentation	1144
8.216.3.1 tfm_plat_increment_nv_counter()	1144
8.216.3.2 tfm_plat_init_nv_counter()	1145
8.216.3.3 tfm_plat_read_nv_counter()	1145
8.216.3.4 tfm_plat_set_nv_counter()	1146
8.217 platform/ext/target/infineon/common/spe/services/platform/nv_counters_rram.c File Reference	1146
8.217.1 Macro Definition Documentation	1147
8.217.1.1 IFX_NVM_INIT_DONE_FLAG	1147
8.217.1.2 IFX_TFM_NV_COUNTERS_BACKUP_AREA_OFFSET	1147
8.217.1.3 IFX_TFM_NV_COUNTERS_BACKUP_AREA_SIZE	1147
8.217.1.4 IFX_TFM_NV_COUNTERS_BASE	1147
8.217.1.5 IFX_TFM_NV_COUNTERS_COUNTER_AREA_OFFSET	1148
8.217.1.6 IFX_TFM_NV_COUNTERS_EXPECTED_AREA_SIZE	1148
8.217.1.7 IFX_TFM_NV_COUNTERS_NVM_INIT_DONE_FLAG_OFFSET	1148
8.217.2 Typedef Documentation	1148
8.217.2.1 ifx_rram_counter_value_t	1148
8.217.3 Function Documentation	1148
8.217.3.1 ifx_rram_clear_counters_and_backups()	1148
8.217.3.2 tfm_plat_increment_nv_counter()	1148
8.217.3.3 tfm_plat_init_nv_counter()	1149
8.217.3.4 tfm_plat_read_nv_counter()	1149
8.217.3.5 tfm_plat_set_nv_counter()	1150
8.218 platform/ext/target/infineon/common/spe/services/platform/nv_counters_se_rt.c File Reference	1150
8.218.1 Macro Definition Documentation	1151
8.218.1.1 IFX_PLAT_NV_COUNTER_NS_0	1151
8.218.1.2 IFX_PLAT_NV_COUNTER_PS_0	1151
8.218.1.3 IFX_PLAT_RRAM_COUNTER_NUMBER	1151
8.218.2 Function Documentation	1151
8.218.2.1 tfm_plat_increment_nv_counter()	1151
8.218.2.2 tfm_plat_init_nv_counter()	1152
8.218.2.3 tfm_plat_read_nv_counter()	1152
8.219 platform/ext/target/infineon/common/spe/services/platform/tfm_platform_system.c File Reference	1152
8.219.1 Function Documentation	1152
8.219.1.1 tfm_platform_hal_ioctl()	1152
8.219.1.2 tfm_platform_hal_system_reset()	1153

8.220 platform/ext/target/infineon/common/spe/services/ps/drivers/ps_flash_driver.c File Reference . . .	1153
8.221 platform/ext/target/infineon/common/spe/services/ps/drivers/ps_rram_driver.c File Reference . . .	1154
8.222 platform/ext/target/infineon/common/spe/services/ps/drivers/ps_smif_driver.c File Reference . . .	1154
8.223 platform/ext/target/infineon/common/spe/services/ps/ps_hal.c File Reference	1155
8.223.1 Function Documentation	1155
8.223.1.1 tfm_hal_ps_fs_info()	1156
8.224 platform/ext/target/infineon/common/spe/services/ps/ps_keys_se_rt.c File Reference	1156
8.224.1 Function Documentation	1156
8.224.1.1 tfm_platform_ps_set_key()	1156
8.225 platform/ext/target/infineon/common/spe/svc/platform_svc_api.c File Reference	1157
8.225.1 Function Documentation	1157
8.225.1.1 ifx_call_platform_original_iovec()	1158
8.225.1.2 ifx_call_platform_system_reset()	1158
8.225.1.3 ifx_call_platform_uart_log()	1158
8.226 platform/ext/target/infineon/common/spe/svc/platform_svc_api.h File Reference	1159
8.226.1 Typedef Documentation	1160
8.226.1.1 ifx_original_iovec_t	1160
8.226.2 Function Documentation	1160
8.226.2.1 ifx_call_platform_enable_systick()	1160
8.226.2.2 ifx_call_platform_original_iovec()	1161
8.226.2.3 ifx_call_platform_system_reset()	1161
8.226.2.4 ifx_call_platform_uart_log()	1161
8.227 platform/ext/target/infineon/common/spe/svc/platform_svc_handler.c File Reference	1162
8.227.1 Function Documentation	1162
8.227.1.1 platform_svc_handlers()	1162
8.228 platform/ext/target/infineon/common/spe/svc/platform_svc_private.h File Reference	1163
8.228.1 Macro Definition Documentation	1163
8.228.1.1 IFX_SVC_PLATFORM_ENABLE_SYSTICK	1163
8.228.1.2 IFX_SVC_PLATFORM_ORIGINAL_IOVEC	1164
8.228.1.3 IFX_SVC_PLATFORM_SYSTEM_RESET	1164
8.228.1.4 IFX_SVC_PLATFORM_UART_LOG	1164
8.229 platform/ext/target/infineon/common/spe/v80m/target_cfg.c File Reference	1164
8.229.1 Macro Definition Documentation	1164
8.229.1.1 SCB_AIRCR_WRITE_MASK	1164
8.229.2 Function Documentation	1165
8.229.2.1 FIH_RET_TYPE()	1165
8.229.2.2 ifx_init_debug()	1169
8.229.2.3 ifx_init_system_control_block()	1169
8.230 platform/ext/target/infineon/common/spe/v80m/target_cfg.h File Reference	1170
8.230.1 Function Documentation	1171
8.230.1.1 FIH_RET_TYPE()	1171
8.230.1.2 ifx_init_debug()	1190

8.230.1.3 ifx_init_system_control_block()	1190
8.231 platform/ext/target/infineon/common/spe/v80m/tfm_hal_platform.c File Reference	1191
8.231.1 Function Documentation	1191
8.231.1.1 FIH_RET_TYPE()	1191
8.232 platform/ext/target/infineon/common/utis/ifx_boot_shared_data.c File Reference	1196
8.233 platform/ext/target/infineon/common/utis/ifx_boot_shared_data.h File Reference	1197
8.234 platform/ext/target/infineon/common/utis/ifx_fih.h File Reference	1197
8.234.1 Macro Definition Documentation	1198
8.234.1.1 FIH_CLEAR_REG	1199
8.234.1.2 FIH_INVALID_VALUE	1199
8.234.1.3 FIH_SET_REG	1199
8.234.1.4 FIH_WRITE_REG	1199
8.234.1.5 IFX_FIH_BOOL	1199
8.234.1.6 IFX_FIH_DECLARE	1199
8.234.1.7 IFX_FIH_EQ	1199
8.234.1.8 IFX_FIH_FALSE	1199
8.234.1.9 ifx_fih_to_aapcs_fih	1200
8.234.1.10 IFX_FIH_TRUE	1200
8.234.2 Typedef Documentation	1200
8.234.2.1 ifx_aapcs_fih_int	1200
8.235 platform/ext/target/infineon/common/utis/ifx_interrupt_defs.h File Reference	1200
8.235.1 Macro Definition Documentation	1200
8.235.1.1 IFX_CONCAT	1200
8.235.1.2 IFX_IRQ_NAME_TO_HANDLER	1201
8.236 platform/ext/target/infineon/common/utis/ifx_regions.c File Reference	1201
8.236.1 Function Documentation	1201
8.236.1.1 ifx_is_region_inside_other()	1201
8.236.1.2 ifx_is_region_overlap_other()	1202
8.237 platform/ext/target/infineon/common/utis/ifx_regions.h File Reference	1202
8.237.1 Function Documentation	1203
8.237.1.1 ifx_is_region_inside_other()	1203
8.237.1.2 ifx_is_region_overlap_other()	1204
8.237.1.3 REGION_DECLARE() [1/9]	1204
8.237.1.4 REGION_DECLARE() [2/9]	1205
8.237.1.5 REGION_DECLARE() [3/9]	1205
8.237.1.6 REGION_DECLARE() [4/9]	1205
8.237.1.7 REGION_DECLARE() [5/9]	1205
8.237.1.8 REGION_DECLARE() [6/9]	1205
8.237.1.9 REGION_DECLARE() [7/9]	1205
8.237.1.10 REGION_DECLARE() [8/9]	1205
8.237.1.11 REGION_DECLARE() [9/9]	1205
8.238 platform/ext/target/infineon/common/utis/ifx_utils.h File Reference	1206

8.238.1 Macro Definition Documentation	1206
8.238.1.1 IFX_ALIGN_UP_TO	1206
8.238.1.2 IFX_ASSERT	1206
8.238.1.3 IFX_ROUND_DOWN_TO_MULTIPLE	1206
8.238.1.4 IFX_ROUND_UP_TO_MULTIPLE	1206
8.238.1.5 IFX_UNSIGNED	1207
8.238.1.6 IFX_UNSIGNED_TO_VALUE	1207
8.238.1.7 IFX_UNUSED	1207
8.239 platform/ext/target/infineon/common/utis/mxs22.h File Reference	1207
8.239.1 Macro Definition Documentation	1208
8.239.1.1 IFX_IDAU_SECURE_ADDRESS_MASK	1208
8.239.1.2 IFX_NS_ADDRESS_ALIAS	1208
8.239.1.3 IFX_NS_ADDRESS_ALIAS_T	1208
8.239.1.4 IFX_S_ADDRESS_ALIAS	1208
8.239.1.5 IFX_S_ADDRESS_ALIAS_T	1208
8.240 platform/ext/target/infineon/common/utis/se/ifx_se_crc32.c File Reference	1208
8.240.1 Detailed Description	1209
8.240.2 Function Documentation	1209
8.240.2.1 ifx_se_crc32d6_close()	1209
8.240.2.2 ifx_se_crc32d6_open()	1210
8.240.2.3 ifx_se_crc32d6a()	1210
8.240.2.4 ifx_se_crc32d6a_update()	1211
8.240.2.5 ifx_se_crc32d6b()	1212
8.240.2.6 ifx_se_crc32d6b_update()	1212
8.241 platform/ext/target/infineon/common/utis/se/ifx_se_crc32.h File Reference	1213
8.241.1 Detailed Description	1214
8.241.2 Macro Definition Documentation	1214
8.241.2.1 IFX_CRC32_BYTE_WIDTH	1214
8.241.2.2 IFX_CRC32_CALC	1214
8.241.2.3 IFX_CRC32_CALC_APPEND	1214
8.241.2.4 IFX_CRC32_CRC_SIZE	1215
8.241.2.5 IFX_CRC32_CRC_WIDTH	1215
8.241.2.6 IFX_CRC32_INIT	1215
8.241.2.7 IFX_CRC32_WORD_WIDTH	1215
8.241.3 Function Documentation	1215
8.241.3.1 ifx_se_crc32d6_close()	1215
8.241.3.2 ifx_se_crc32d6_open()	1215
8.241.3.3 ifx_se_crc32d6a()	1216
8.241.3.4 ifx_se_crc32d6a_update()	1217
8.241.3.5 ifx_se_crc32d6b()	1217
8.241.3.6 ifx_se_crc32d6b_update()	1218
8.242 platform/ext/target/infineon/psc3m6/board/shared/device/include/ifx_core_interrupts.h File Reference	1219

8.242.1 Macro Definition Documentation	1219
8.242.1.1 IFX_CORE_DEFINE_INTERRUPTS_LIST	1219
8.242.1.2 IFX_CORE_INTERRUPTS_LIST	1219
8.243 platform/ext/target/infineon/psc3m6/board/shared/device/include/project_memory_layout.h File Reference	1219
8.243.1 Macro Definition Documentation	1221
8.243.1.1 IFX_CM33_NS_DATA_ADDR	1221
8.243.1.2 IFX_CM33_NS_DATA_LOCATION	1221
8.243.1.3 IFX_CM33_NS_DATA_OFFSET	1222
8.243.1.4 IFX_CM33_NS_DATA_SIZE	1222
8.243.1.5 IFX_CM33_NS_IMAGE_EXECUTE_ADDR	1222
8.243.1.6 IFX_CM33_NS_IMAGE_EXECUTE_LOCATION	1222
8.243.1.7 IFX_CM33_NS_IMAGE_EXECUTE_OFFSET	1222
8.243.1.8 IFX_CM33_NS_IMAGE_EXECUTE_SIZE	1222
8.243.1.9 IFX_FF_TEST_DRIVER_PARTITION_MMIO_ADDR	1222
8.243.1.10 IFX_FF_TEST_DRIVER_PARTITION_MMIO_LOCATION	1222
8.243.1.11 IFX_FF_TEST_DRIVER_PARTITION_MMIO_OFFSET	1223
8.243.1.12 IFX_FF_TEST_DRIVER_PARTITION_MMIO_SIZE	1223
8.243.1.13 IFX_FF_TEST_SERVER_PARTITION_MMIO_ADDR	1223
8.243.1.14 IFX_FF_TEST_SERVER_PARTITION_MMIO_LOCATION	1223
8.243.1.15 IFX_FF_TEST_SERVER_PARTITION_MMIO_OFFSET	1223
8.243.1.16 IFX_FF_TEST_SERVER_PARTITION_MMIO_SIZE	1223
8.243.1.17 IFX_FLASH_CBUS_BASE	1223
8.243.1.18 IFX_FLASH_SBUS_BASE	1223
8.243.1.19 IFX_FLASH_SIZE	1223
8.243.1.20 IFX_SRAM0_CBUS_BASE	1224
8.243.1.21 IFX_SRAM0_SBUS_BASE	1224
8.243.1.22 IFX_SRAM0_SIZE	1224
8.243.1.23 IFX_SRAM1_CBUS_BASE	1224
8.243.1.24 IFX_SRAM1_SBUS_BASE	1224
8.243.1.25 IFX_SRAM1_SIZE	1224
8.243.1.26 IFX_TEST_AROT_PARTITION_MEMORY_ADDR	1224
8.243.1.27 IFX_TEST_AROT_PARTITION_MEMORY_LOCATION	1224
8.243.1.28 IFX_TEST_AROT_PARTITION_MEMORY_OFFSET	1224
8.243.1.29 IFX_TEST_AROT_PARTITION_MEMORY_SIZE	1224
8.243.1.30 IFX_TEST_PROT_PARTITION_MEMORY_ADDR	1225
8.243.1.31 IFX_TEST_PROT_PARTITION_MEMORY_LOCATION	1225
8.243.1.32 IFX_TEST_PROT_PARTITION_MEMORY_OFFSET	1225
8.243.1.33 IFX_TEST_PROT_PARTITION_MEMORY_SIZE	1225
8.243.1.34 IFX_TFM_BOOT_SHARED_DATA_ADDR	1225
8.243.1.35 IFX_TFM_BOOT_SHARED_DATA_LOCATION	1225
8.243.1.36 IFX_TFM_BOOT_SHARED_DATA_OFFSET	1225

8.243.1.37 IFX_TFM_BOOT_SHARED_DATA_SIZE	1225
8.243.1.38 IFX_TFM_DATA_ADDR	1225
8.243.1.39 IFX_TFM_DATA_LOCATION	1225
8.243.1.40 IFX_TFM_DATA_OFFSET	1226
8.243.1.41 IFX_TFM_DATA_SIZE	1226
8.243.1.42 IFX_TFM_ITS_ADDR	1226
8.243.1.43 IFX_TFM_ITS_LOCATION	1226
8.243.1.44 IFX_TFM_ITS_OFFSET	1226
8.243.1.45 IFX_TFM_ITS_SIZE	1226
8.243.1.46 IFX_TFM_NV_COUNTERS_AREA_ADDR	1226
8.243.1.47 IFX_TFM_NV_COUNTERS_AREA_LOCATION	1226
8.243.1.48 IFX_TFM_NV_COUNTERS_AREA_OFFSET	1226
8.243.1.49 IFX_TFM_NV_COUNTERS_AREA_SIZE	1226
8.243.1.50 IFX_TFM_PS_ADDR	1227
8.243.1.51 IFX_TFM_PS_LOCATION	1227
8.243.1.52 IFX_TFM_PS_OFFSET	1227
8.243.1.53 IFX_TFM_PS_SIZE	1227
8.244 platform/ext/target/infineon/psc3m6/board/shared/ifx_peripherals_def.h File Reference	1227
8.244.1 Macro Definition Documentation	1228
8.244.1.1 IFX_IRQ_TEST_FLIH_INTERRUPT	1228
8.244.1.2 IFX_IRQ_TEST_FLIH_INTERRUPT_GPIO_PIN	1228
8.244.1.3 IFX_IRQ_TEST_FLIH_INTERRUPT_GPIO_PORT	1228
8.244.1.4 IFX_IRQ_TEST_NS_INTERRUPT	1228
8.244.1.5 IFX_IRQ_TEST_NS_INTERRUPT_GPIO_PIN	1228
8.244.1.6 IFX_IRQ_TEST_NS_INTERRUPT_GPIO_PORT	1228
8.244.1.7 TFM_FPU_NS_TEST_IRQ	1229
8.245 platform/ext/target/infineon/psc3m6/board/spe/include/ifx_s_peripherals_def.h File Reference	1229
8.245.1 Macro Definition Documentation	1230
8.245.1.1 IFX_LCS_BASE	1230
8.245.1.2 IFX_SFLASH	1230
8.245.1.3 IFX_TFM_FAULT_IRQ	1230
8.245.1.4 IFX_TFM_FAULT_STRUCT	1230
8.245.1.5 IFX_TFM_MSC_IRQ	1230
8.245.1.6 TFM_FPU_S_TEST_IRQ	1230
8.246 platform/ext/target/infineon/psc3m6/board/spe/include/protection_regions_cfg.h File Reference	1230
8.246.1 Macro Definition Documentation	1231
8.246.1.1 CY_SAU_REGION_CNT	1231
8.247 platform/ext/target/infineon/psc3m6/board/spe/source/protection_regions_cfg.c File Reference	1231
8.247.1 Variable Documentation	1232
8.247.1.1 ifx_ppcx_static_config	1232
8.247.1.2 ifx_ppcx_static_config_count	1232
8.247.1.3 ifx_tfm_fault_sources	1232

8.247.1.4 ifx_tfm_fault_sources_count	1232
8.248 platform/ext/target/infineon/psc3m6/config/ifx_platform_spe_config.h File Reference	1232
8.248.1 Detailed Description	1233
8.248.2 Macro Definition Documentation	1233
8.248.2.1 IFX_APP_ROT_PPC_DYNAMIC_ISOLATION	1233
8.248.2.2 IFX_APP_ROT_PRIVILEGED	1233
8.248.2.3 IFX_MAX_SPLIT_REGIONS_COUNT	1234
8.248.2.4 IFX_MPC_CM55_MPC	1234
8.248.2.5 IFX_MPC_CONFIGURED_BY_TFM	1234
8.248.2.6 IFX_MPC_DRIVER_HW_MPC_WITH_ROT	1234
8.248.2.7 IFX_MPC_DRIVER_HW_MPC_WITHOUT_ROT	1234
8.248.2.8 IFX_MPC_DRIVER_SW_POLICY	1234
8.248.2.9 IFX_MSC_ACG_RESP_CONFIG	1234
8.248.2.10 IFX_MSC_ACG_RESP_CONFIG_V1	1234
8.248.2.11 IFX_MSC_TFM_CORE_BUS_MASTER_ID	1234
8.248.2.12 IFX_MULTIPLE_IAK_KEY_TYPES	1234
8.248.2.13 IFX_NOT_ROT_CONFIGURATION	1235
8.248.2.14 IFX_NS_AGENT_TZ_PRIVILEGED	1235
8.248.2.15 IFX_PC_CM33_NSPE	1235
8.248.2.16 IFX_PC_CM33_NSPE_ID	1235
8.248.2.17 IFX_PC_DEBUGGER	1235
8.248.2.18 IFX_PC_DEBUGGER_ID	1235
8.248.2.19 IFX_PC_DEFAULT	1235
8.248.2.20 IFX_PC_NONE	1235
8.248.2.21 IFX_PC_TFM_AROT	1235
8.248.2.22 IFX_PC_TFM_AROT_ID	1235
8.248.2.23 IFX_PC_TFM_PROT	1236
8.248.2.24 IFX_PC_TFM_PROT_ID	1236
8.248.2.25 IFX_PC_TFM_SPM	1236
8.248.2.26 IFX_PC_TFM_SPM_ID	1236
8.248.2.27 IFX_PC_TZ_NSPE	1236
8.248.2.28 IFX_PC_TZ_NSPE_ID	1236
8.248.2.29 IFX_PLATFORM_MPC_PRESENT	1236
8.248.2.30 IFX_PLATFORM_PPC_PRESENT	1236
8.248.2.31 IFX_PSA_ROT_PRIVILEGED	1236
8.248.2.32 IFX_REGION_MAX_MPC_COUNT	1236
8.249 platform/ext/target/infineon/psc3m6/config/ifx_platform_spe_types.h File Reference	1237
8.249.1 Detailed Description	1237
8.249.2 Macro Definition Documentation	1237
8.249.2.1 IFX_MPC_IS_EXTERNAL	1237
8.249.2.2 IFX_PROTECTION_MPU_PERIPHERAL_REGION_LIMIT	1238
8.249.2.3 IFX_PROTECTION_MPU_PERIPHERAL_REGION_START	1238

8.250 platform/ext/target/infineon/psc3m6/config_tfm_target.h File Reference	1238
8.250.1 Macro Definition Documentation	1238
8.250.1.1 ATTEST_TOKEN_PROFILE_PSA_2_0_0	1238
8.250.1.2 CONFIG_TFM_NS_AGENT_TZ_STACK_SIZE	1238
8.250.1.3 IFX_MEMORY_CONFIGURATOR_MPC_CONFIG	1238
8.250.1.4 IFX_MEMORY_CONFIGURATOR_SAU_CONFIG	1238
8.250.1.5 ITS_BUF_SIZE	1239
8.250.1.6 LOG_PRIV_BUFFER_SIZE	1239
8.250.1.7 LOG_UNPRIV_BUFFER_SIZE	1239
8.250.1.8 PLATFORM_SP_STACK_SIZE	1239
8.250.1.9 SECURE_TEST_PARTITION_STACK_SIZE	1239
8.251 platform/ext/target/infineon/psc3m6/shared/partition/flash_layout.h File Reference	1239
8.252 platform/ext/target/infineon/psc3m6/shared/partition/region_defs.h File Reference	1240
8.252.1 Macro Definition Documentation	1240
8.252.1.1 BOOT_TFM_SHARED_DATA_BASE	1241
8.252.1.2 BOOT_TFM_SHARED_DATA_LIMIT	1241
8.252.1.3 BOOT_TFM_SHARED_DATA_SIZE	1241
8.252.1.4 IFX_CM33_NS_CODE_LIMIT	1241
8.252.1.5 IFX_CM33_NS_CODE_SIZE	1241
8.252.1.6 IFX_CM33_NS_CODE_START	1241
8.252.1.7 IFX_CM33_NS_DATA_LIMIT	1241
8.252.1.8 S_CODE_LIMIT	1241
8.252.1.9 S_CODE_SIZE	1241
8.252.1.10 S_CODE_START	1242
8.252.1.11 S_CODE_VECTOR_TABLE_SIZE	1242
8.252.1.12 S_DATA_LIMIT	1242
8.252.1.13 S_DATA_SIZE	1242
8.252.1.14 S_DATA_START	1242
8.252.1.15 S_MSP_STACK_SIZE	1242
8.252.1.16 SHARED_BOOT_MEASUREMENT_BASE	1242
8.252.1.17 SHARED_BOOT_MEASUREMENT_LIMIT	1242
8.252.1.18 SHARED_BOOT_MEASUREMENT_SIZE	1242
8.253 platform/ext/target/infineon/psc3m6/shared/tfm_peripherals_def.h File Reference	1243
8.253.1 Macro Definition Documentation	1243
8.253.1.1 IFX_TEST_PERIPHERAL_1	1244
8.253.1.2 IFX_TEST_PERIPHERAL_1_BASE	1244
8.253.1.3 IFX_TEST_PERIPHERAL_1_SIZE	1244
8.253.1.4 IFX_TEST_PERIPHERAL_2	1244
8.253.1.5 IFX_TEST_PERIPHERAL_2_BASE	1244
8.253.1.6 IFX_TEST_PERIPHERAL_2_SIZE	1244
8.254 platform/ext/target/infineon/psc3m6/spe/protection/platform_partition_assets.h File Reference	1244
8.255 platform/ext/target/infineon/psc3m6/spe/protection/protection_data.c File Reference	1245

8.255.1 Function Documentation	1246
8.255.1.1 FIH_RET()	1246
8.255.1.2 FIH_RET_TYPE()	1246
8.255.1.3 if()	1246
8.255.1.4 ifx_platform_mpc_is_rot()	1246
8.255.2 Variable Documentation	1246
8.255.2.1 attr	1246
8.255.2.2 else	1246
8.255.2.3 ifx_memory_cm33_config	1247
8.255.2.4 ifx_memory_cm33_config_count	1247
8.255.2.5 limit	1247
8.255.2.6 p_asset	1247
8.255.2.7 start	1247
8.255.2.8 valid	1247
8.256 platform/ext/target/infineon/psc3m6/spe/services/crypto/crypto_keys_flash.h File Reference	1247
8.256.1 Macro Definition Documentation	1249
8.256.1.1 IFX_CRYPTOKEYS_HUK_ALGORITHM	1249
8.256.1.2 IFX_CRYPTOKEYS_HUK_KEY_BITS	1249
8.256.1.3 IFX_CRYPTOKEYS_HUK_KEY_LEN	1249
8.256.1.4 IFX_CRYPTOKEYS_HUK_TYPE	1249
8.256.1.5 IFX_CRYPTOKEYS_IAK_ALGORITHM_BY_TYPE	1250
8.256.1.6 IFX_CRYPTOKEYS_IAK_IS_VALID	1250
8.256.1.7 IFX_CRYPTOKEYS_IAK_KEY_BITS_BY_TYPE	1250
8.256.1.8 IFX_CRYPTOKEYS_IAK_KEY_LEN_BY_TYPE	1250
8.256.1.9 IFX_CRYPTOKEYS_IAK_SECP256R1_ALGORITHM	1250
8.256.1.10 IFX_CRYPTOKEYS_IAK_SECP256R1_KEY_BITS	1250
8.256.1.11 IFX_CRYPTOKEYS_IAK_SECP256R1_KEY_LEN	1251
8.256.1.12 IFX_CRYPTOKEYS_IAK_SECP256R1_TYPE	1251
8.256.1.13 IFX_CRYPTOKEYS_IAK_SECP384R1_ALGORITHM	1251
8.256.1.14 IFX_CRYPTOKEYS_IAK_SECP384R1_KEY_BITS	1251
8.256.1.15 IFX_CRYPTOKEYS_IAK_SECP384R1_KEY_LEN	1251
8.256.1.16 IFX_CRYPTOKEYS_IAK_SECP384R1_TYPE	1251
8.256.1.17 IFX_CRYPTOKEYS_IAK_SECP521R1_ALGORITHM	1251
8.256.1.18 IFX_CRYPTOKEYS_IAK_SECP521R1_KEY_BITS	1251
8.256.1.19 IFX_CRYPTOKEYS_IAK_SECP521R1_KEY_LEN	1251
8.256.1.20 IFX_CRYPTOKEYS_IAK_SECP521R1_TYPE	1251
8.256.1.21 IFX_CRYPTOKEYS_IAK_TYPE_BY_TYPE	1252
8.256.1.22 IFX_CRYPTOKEYS_PTR	1252
8.256.1.23 IFX_HUK_CRYPTOKEYS_MAX_LEN	1252
8.256.1.24 IFX_IAK_CRYPTOKEYS_MAX_LEN	1252
8.256.1.25 IFX_IAK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP256R1	1252
8.256.1.26 IFX_IAK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP384R1	1252

8.256.1.27 IFX_IAK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP521R1	1252
8.256.1.28 IFX_IAK_CRYPTOKEYS_TYPE_SYMMETRIC	1252
8.256.2 Variable Documentation	1252
8.256.2.1 __PACKED_STRUCT	1253
8.256.2.2 iak	1253
8.256.2.3 iak_key_type	1253
8.256.2.4 ifx_plat_keys_t	1253
8.257 platform/ext/target/infineon/psoc3m6/spe/services/crypto/ifx_pdl_cryptolite_config.h File Reference	1253
8.257.1 Macro Definition Documentation	1254
8.257.1.1 CY_CRYPTOLITE_CFG_AES_C	1254
8.257.1.2 CY_CRYPTOLITE_CFG_CBC_MAC_C	1254
8.257.1.3 CY_CRYPTOLITE_CFG_CIPHER_MODE_CBC	1254
8.257.1.4 CY_CRYPTOLITE_CFG_CIPHER_MODE_CCM	1254
8.257.1.5 CY_CRYPTOLITE_CFG_CIPHER_MODE_CFB	1254
8.257.1.6 CY_CRYPTOLITE_CFG_CIPHER_MODE_CTR	1255
8.257.1.7 CY_CRYPTOLITE_CFG_CMAC_C	1255
8.257.1.8 CY_CRYPTOLITE_CFG_ECDSA_C	1255
8.257.1.9 CY_CRYPTOLITE_CFG_ECDSA_SIGN_C	1255
8.257.1.10 CY_CRYPTOLITE_CFG_ECDSA_VERIFY_C	1255
8.257.1.11 CY_CRYPTOLITE_CFG_ECP_C	1255
8.257.1.12 CY_CRYPTOLITE_CFG_ECP_DP_SECP256K1_ENABLED	1255
8.257.1.13 CY_CRYPTOLITE_CFG_ECP_DP_SECP256R1_ENABLED	1255
8.257.1.14 CY_CRYPTOLITE_CFG_ECP_DP_SECP384R1_ENABLED	1255
8.257.1.15 CY_CRYPTOLITE_CFG_HKDF_C	1255
8.257.1.16 CY_CRYPTOLITE_CFG_HMAC_C	1256
8.257.1.17 CY_CRYPTOLITE_CFG_SHA2_256_ENABLED	1256
8.257.1.18 CY_CRYPTOLITE_CFG_SHA2_512_ENABLED	1256
8.257.1.19 CY_CRYPTOLITE_CFG_SHA_C	1256
8.257.1.20 CY_CRYPTOLITE_CFG_TRNG_C	1256
8.257.1.21 IFX_PSA_CRYPTOLITE_AEAD	1256
8.257.1.22 IFX_PSA_CRYPTOLITE_CBC_NO_PADDING	1256
8.257.1.23 IFX_PSA_CRYPTOLITE_CCM	1256
8.257.1.24 IFX_PSA_CRYPTOLITE_CFB	1256
8.257.1.25 IFX_PSA_CRYPTOLITE_CMAC	1256
8.257.1.26 IFX_PSA_CRYPTOLITE_CTR	1257
8.257.1.27 IFX_PSA_CRYPTOLITE_ECC_PUBLIC_KEY_EXPORT	1257
8.257.1.28 IFX_PSA_CRYPTOLITE_ECC_SECP_R1_256	1257
8.257.1.29 IFX_PSA_CRYPTOLITE_ECDH	1257
8.257.1.30 IFX_PSA_CRYPTOLITE_ECDSA_SIGN	1257
8.257.1.31 IFX_PSA_CRYPTOLITE_ECDSA_VERIFY	1257
8.257.1.32 IFX_PSA_CRYPTOLITE_ECDSA_VERIFY_USE_PK	1257
8.257.1.33 IFX_PSA_CRYPTOLITE_HKDF	1257

8.257.1.34 IFX_PSA_CRYPTOLITE_HMAC	1257
8.257.1.35 IFX_PSA_CRYPTOLITE_KEY_DERIVATION	1257
8.257.1.36 IFX_PSA_CRYPTOLITE_KEY_GENERATION	1258
8.257.1.37 IFX_PSA_CRYPTOLITE_MAC	1258
8.257.1.38 IFX_PSA_CRYPTOLITE_PBKDF2_HMAC	1258
8.257.1.39 IFX_PSA_CRYPTOLITE_PUBLIC_KEY_EXPORT	1258
8.257.1.40 IFX_PSA_CRYPTOLITE_SHA	1258
8.257.1.41 IFX_PSA_CRYPTOLITE_SHA_224	1258
8.257.1.42 IFX_PSA_CRYPTOLITE_SHA_256	1258
8.257.1.43 IFX_PSA_CRYPTOLITE_SHA_384	1258
8.257.1.44 IFX_PSA_CRYPTOLITE_SHA_512	1258
8.257.1.45 IFX_PSA_CRYPTOLITE_TLS12_PRF	1258
8.257.1.46 IFX_PSA_CRYPTOLITE_TLS12_PSK_TO_MS	1259
8.257.1.47 IFX_PSA_CRYPTOLITE_USE_STACK_MEM	1259
8.258 platform/ext/target/infineon/psc3m6/spe/services/crypto/ifx_pdl_cryptosuite_config.h File Reference	1259
8.258.1 Macro Definition Documentation	1259
8.258.1.1 IFX_MXCRYPTOSUITE_USE_STATIC_MEM	1259
8.259 platform/ext/target/infineon/psc3m6/spe/services/crypto/ifx_pdl_psa_cryptolite_config.h File Reference	1259
8.259.1 Macro Definition Documentation	1260
8.259.1.1 IFX_PSA_CRYPTOLITE_ECC_PUBLIC_KEY_EXPORT	1260
8.259.1.2 IFX_PSA_CRYPTOLITE_ECC_SECP_R1_256	1260
8.259.1.3 IFX_PSA_CRYPTOLITE_ECDH	1260
8.259.1.4 IFX_PSA_CRYPTOLITE_ECDSA_SIGN	1260
8.259.1.5 IFX_PSA_CRYPTOLITE_ECDSA_VERIFY	1260
8.259.1.6 IFX_PSA_CRYPTOLITE_ECDSA_VERIFY_USE_PK	1260
8.259.1.7 IFX_PSA_CRYPTOLITE_HMAC	1260
8.259.1.8 IFX_PSA_CRYPTOLITE_KEY_DERIVATION	1260
8.259.1.9 IFX_PSA_CRYPTOLITE_KEY_GENERATION	1260
8.259.1.10 IFX_PSA_CRYPTOLITE_PUBLIC_KEY_EXPORT	1261
8.259.1.11 IFX_PSA_CRYPTOLITE_SHA	1261
8.259.1.12 IFX_PSA_CRYPTOLITE_SHA_256	1261
8.260 platform/ext/target/infineon/psc3m6/spe/services/crypto/mbdctl_target_config.h File Reference . .	1261
8.260.1 Macro Definition Documentation	1261
8.260.1.1 MBEDTLS_PSA_CRYPTO_BUILTIN_KEYS	1261
8.261 platform/ext/target/infineon/psc3m6/spe/services/crypto/platform_builtin_key_loader_ids.h File Reference	1261
8.261.1 Macro Definition Documentation	1262
8.261.1.1 TFM_BUILTIN_MAX_KEY_LEN	1262
8.261.2 Enumeration Type Documentation	1262
8.261.2.1 psa_drv_slot_number_t	1262
8.262 secure_fw/include/async.h File Reference	1262
8.262.1 Macro Definition Documentation	1263

8.262.1.1 ASYNC_MSG_REPLY	1263
8.263 secure_fw/include/build_config_check.h File Reference	1263
8.264 secure_fw/include/compiler_ext_defs.h File Reference	1264
8.264.1 Macro Definition Documentation	1264
8.264.1.1 SYNTAX_UNIFIED	1264
8.265 secure_fw/include/config_tfm.h File Reference	1264
8.266 secure_fw/include/crt_impl_private.h File Reference	1265
8.266.1 Macro Definition Documentation	1266
8.266.1.1 ADDR_WORD_UNALIGNED	1266
8.267 secure_fw/include/security_defs.h File Reference	1266
8.267.1 Macro Definition Documentation	1266
8.267.1.1 STACK_SEAL_PATTERN	1266
8.268 secure_fw/partitions/crypto/config_crypto_check.h File Reference	1266
8.269 secure_fw/partitions/crypto/config_engine_buf.h File Reference	1267
8.270 secure_fw/partitions/crypto/crypto_aead.c File Reference	1267
8.271 secure_fw/partitions/crypto/crypto_alloc.c File Reference	1268
8.271.1 Macro Definition Documentation	1269
8.271.1.1 TFM_CRYPTO_INVALID_HANDLE	1269
8.271.2 Enumeration Type Documentation	1269
8.271.2.1 anonymous enum	1269
8.272 secure_fw/partitions/crypto/crypto_asymmetric.c File Reference	1269
8.273 secure_fw/partitions/crypto/crypto_check_config.h File Reference	1270
8.274 secure_fw/partitions/crypto/crypto_cipher.c File Reference	1271
8.275 secure_fw/partitions/crypto/crypto_hash.c File Reference	1272
8.276 secure_fw/partitions/crypto/crypto_init.c File Reference	1273
8.276.1 Macro Definition Documentation	1273
8.276.1.1 ALIGN	1274
8.276.1.2 TFM_CRYPTO_IOVEC_ALIGNMENT	1274
8.276.2 Function Documentation	1274
8.276.2.1 tfm_crypto_get_caller_id()	1274
8.277 secure_fw/partitions/crypto/crypto_key_derivation.c File Reference	1274
8.278 secure_fw/partitions/crypto/crypto_key_management.c File Reference	1275
8.279 secure_fw/partitions/crypto/crypto_library.c File Reference	1276
8.280 secure_fw/partitions/crypto/crypto_library.h File Reference	1277
8.280.1 Detailed Description	1278
8.280.2 Macro Definition Documentation	1278
8.280.2.1 CRYPTO_LIBRARY_GET_KEY_ID	1278
8.280.2.2 CRYPTO_LIBRARY_GET_OWNER	1278
8.280.2.3 TFM_CRYPTO_KEY_ATTR_OFFSET_CLIENT_SERVER	1278
8.280.3 Typedef Documentation	1279
8.280.3.1 tfm_crypto_library_key_id_t	1279
8.281 secure_fw/partitions/crypto/crypto_mac.c File Reference	1279

8.282 secure_fw/partitions/crypto/crypto_rng.c File Reference	1279
8.283 secure_fw/partitions/crypto/crypto_spe.h File Reference	1280
8.283.1 Detailed Description	1282
8.283.2 Macro Definition Documentation	1282
8.283.2.1 psa_aead_abort	1282
8.283.2.2 psa_aead_decrypt	1282
8.283.2.3 psa_aead_decrypt_setup	1282
8.283.2.4 psa_aead_encrypt	1282
8.283.2.5 psa_aead_encrypt_setup	1282
8.283.2.6 psa_aead_finish	1282
8.283.2.7 psa_aead_generate_nonce	1282
8.283.2.8 psa_aead_set_lengths	1282
8.283.2.9 psa_aead_set_nonce	1283
8.283.2.10 psa_aead_update	1283
8.283.2.11 psa_aead_update_ad	1283
8.283.2.12 psa_aead_verify	1283
8.283.2.13 psa_asymmetric_decrypt	1283
8.283.2.14 psa_asymmetric_encrypt	1283
8.283.2.15 psa_can_do_cipher	1283
8.283.2.16 psa_can_do_hash	1283
8.283.2.17 psa_cipher_abort	1283
8.283.2.18 psa_cipher_decrypt	1283
8.283.2.19 psa_cipher_decrypt_setup	1284
8.283.2.20 psa_cipher_encrypt	1284
8.283.2.21 psa_cipher_encrypt_setup	1284
8.283.2.22 psa_cipher_finish	1284
8.283.2.23 psa_cipher_generate_iv	1284
8.283.2.24 psa_cipher_operation_init	1284
8.283.2.25 psa_cipher_set_iv	1284
8.283.2.26 psa_cipher_update	1284
8.283.2.27 psa_copy_key	1284
8.283.2.28 psa_crypto_init	1284
8.283.2.29 psa_destroy_key	1285
8.283.2.30 psa_export_key	1285
8.283.2.31 psa_export_public_key	1285
8.283.2.32 PSA_FUNCTION_NAME	1285
8.283.2.33 psa_generate_key	1285
8.283.2.34 psa_generate_random	1285
8.283.2.35 psa_get_key_attributes	1285
8.283.2.36 psa_hash_abort	1285
8.283.2.37 psa_hash_clone	1285
8.283.2.38 psa_hash_compare	1285

8.283.2.39	psa_hash_compute	1286
8.283.2.40	psa_hash_finish	1286
8.283.2.41	psa_hash_operation_init	1286
8.283.2.42	psa_hash_setup	1286
8.283.2.43	psa_hash_update	1286
8.283.2.44	psa_hash_verify	1286
8.283.2.45	psa_import_key	1286
8.283.2.46	psa_key_derivation_abort	1286
8.283.2.47	psa_key_derivation_get_capacity	1286
8.283.2.48	psa_key_derivation_input_bytes	1286
8.283.2.49	psa_key_derivation_input_integer	1287
8.283.2.50	psa_key_derivation_input_key	1287
8.283.2.51	psa_key_derivation_key_agreement	1287
8.283.2.52	psa_key_derivation_output_bytes	1287
8.283.2.53	psa_key_derivation_output_key	1287
8.283.2.54	psa_key_derivation_set_capacity	1287
8.283.2.55	psa_key_derivation_setup	1287
8.283.2.56	psa_key_derivation_verify_bytes	1287
8.283.2.57	psa_key_derivation_verify_key	1287
8.283.2.58	psa_mac_abort	1287
8.283.2.59	psa_mac_compute	1288
8.283.2.60	psa_mac_operation_init	1288
8.283.2.61	psa_mac_sign_finish	1288
8.283.2.62	psa_mac_sign_setup	1288
8.283.2.63	psa_mac_update	1288
8.283.2.64	psa_mac_verify	1288
8.283.2.65	psa_mac_verify_finish	1288
8.283.2.66	psa_mac_verify_setup	1288
8.283.2.67	psa_purge_key	1288
8.283.2.68	psa_raw_key_agreement	1288
8.283.2.69	psa_reset_key_attributes	1289
8.283.2.70	psa_sign_hash	1289
8.283.2.71	psa_sign_message	1289
8.283.2.72	psa_verify_hash	1289
8.283.2.73	psa_verify_message	1289
8.284	secure_fw/partitions/crypto/dir_crypto.dox File Reference	1289
8.285	secure_fw/partitions/crypto/psa_driver_api/tfm_built_in_key_loader.c File Reference	1289
8.285.1	Macro Definition Documentation	1290
8.285.1.1	NUMBER_OF_ELEMENTS_OF	1290
8.285.1.2	TFM_BUILTIN_MAX_KEY_LEN	1291
8.285.1.3	TFM_BUILTIN_MAX_KEYS	1291
8.286	secure_fw/partitions/crypto/psa_driver_api/tfm_built_in_key_loader.h File Reference	1291

8.286.1 Macro Definition Documentation	1292
8.286.1.1 PSA_CRYPTODRIVER_TFM_BUILTIN_KEY_LOADER	1292
8.286.1.2 TFM_BUILTIN_KEY_LOADER_KEY_LOCATION	1292
8.286.1.3 TFM_BUILTIN_KEY_LOADER_LIFETIME	1292
8.287 secure_fw/partitions/crypto/tfm_crypto_api.h File Reference	1292
8.287.1 Enumeration Type Documentation	1294
8.287.1.1 tfm_crypto_operation_type	1294
8.287.2 Function Documentation	1294
8.287.2.1 tfm_crypto_get_caller_id()	1294
8.288 secure_fw/partitions/crypto/tfm_crypto_key.h File Reference	1295
8.288.1 Macro Definition Documentation	1295
8.288.1.1 TFM_CRYPTODRIVER_KEY_ID_S_INIT	1295
8.289 secure_fw/partitions/crypto/tfm_mbedcrypto_alt.c File Reference	1296
8.289.1 Detailed Description	1296
8.290 secure_fw/partitions/crypto/tfm_mbedcrypto_include.h File Reference	1296
8.290.1 Macro Definition Documentation	1297
8.290.1.1 PSA_CRYPTODRIVER_SECURE	1297
8.291 secure_fw/partitions/dir_services.dox File Reference	1297
8.292 secure_fw/partitions/firmware_update/bootloader/mcuboot/tfm_mcuboot_fw.c File Reference	1297
8.292.1 Macro Definition Documentation	1298
8.292.1.1 MAX_IMAGE_INFO_LENGTH	1298
8.292.2 Typedef Documentation	1298
8.292.2.1 fwu_image_info_data_t	1298
8.292.2.2 tfm_fwu_mcuboot_ctx_t	1298
8.292.3 Function Documentation	1299
8.292.3.1 fwu_bootloader_abort()	1299
8.292.3.2 fwu_bootloader_clean_component()	1299
8.292.3.3 fwu_bootloader_get_image_info()	1299
8.292.3.4 fwu_bootloader_init()	1299
8.292.3.5 fwu_bootloader_install_image()	1300
8.292.3.6 fwu_bootloader_load_image()	1301
8.292.3.7 fwu_bootloader_mark_image_accepted()	1301
8.292.3.8 fwu_bootloader_reject_staged_image()	1301
8.292.3.9 fwu_bootloader_reject_trial_image()	1303
8.292.3.10 fwu_bootloader_staging_area_init()	1303
8.293 secure_fw/partitions/firmware_update/bootloader/tfm_bootloader_fw_abstraction.h File Reference	1304
8.293.1 Function Documentation	1305
8.293.1.1 fwu_bootloader_clean_component()	1305
8.293.1.2 fwu_bootloader_get_image_info()	1305
8.293.1.3 fwu_bootloader_init()	1306
8.293.1.4 fwu_bootloader_install_image()	1306
8.293.1.5 fwu_bootloader_load_image()	1307

8.293.1.6 fwu_bootloader_mark_image_accepted()	1307
8.293.1.7 fwu_bootloader_reject_staged_image()	1308
8.293.1.8 fwu_bootloader_reject_trial_image()	1308
8.293.1.9 fwu_bootloader_staging_area_init()	1308
8.294 secure_fw/partitions/firmware_update/tfm_fwu_req_mgr.c File Reference	1309
8.294.1 Macro Definition Documentation	1310
8.294.1.1 COMPONENTS_ITER	1310
8.294.2 Typedef Documentation	1310
8.294.2.1 tfm_fwu_ctx_t	1310
8.294.3 Function Documentation	1310
8.294.3.1 tfm_firmware_update_service_sfn()	1310
8.294.3.2 tfm_fwu_entry()	1310
8.294.3.3 tfm_fwu_reject()	1310
8.295 secure_fw/partitions/idle_partition/idle_partition.c File Reference	1311
8.295.1 Function Documentation	1311
8.295.1.1 tfm_idle_thread()	1311
8.296 secure_fw/partitions/idle_partition/load_info_idle_sp.c File Reference	1312
8.296.1 Macro Definition Documentation	1313
8.296.1.1 IDLE_SP_STACK_SIZE	1313
8.296.2 Function Documentation	1313
8.296.2.1 tfm_idle_thread()	1313
8.296.3 Variable Documentation	1313
8.296.3.1 idle_sp_stack	1313
8.296.3.2 tfm_sp_idle_load	1313
8.297 secure_fw/partitions/initial_attestation/attest.h File Reference	1313
8.297.1 Enumeration Type Documentation	1315
8.297.1.1 psa_attest_err_t	1315
8.297.2 Function Documentation	1315
8.297.2.1 attest_get_boot_data()	1315
8.297.2.2 attest_get_caller_client_id()	1316
8.297.2.3 attest_init()	1316
8.297.2.4 initial_attest_get_token()	1317
8.297.2.5 initial_attest_get_token_size()	1317
8.298 secure_fw/partitions/initial_attestation/attest_asymmetric_key.c File Reference	1318
8.298.1 Macro Definition Documentation	1318
8.298.1.1 ECC_COORD_SIZE	1318
8.298.1.2 INSTANCE_ID_SIZE	1319
8.298.1.3 MAX_ENCODED_COSE_KEY_SIZE	1319
8.298.2 Function Documentation	1319
8.298.2.1 attest_get_instance_id()	1319
8.299 secure_fw/partitions/initial_attestation/attest_boot_data.c File Reference	1319
8.299.1 Macro Definition Documentation	1320

8.299.1.1 ATTEST_BOOT_STATUS_MAX_SIZE	1320
8.299.2 Function Documentation	1320
8.299.2.1 attest_boot_data_init()	1320
8.299.2.2 attest_encode_sw_components_array()	1321
8.300 secure_fw/partitions/initial_attestation/attest_boot_data.h File Reference	1322
8.300.1 Function Documentation	1323
8.300.1.1 attest_boot_data_init()	1323
8.300.1.2 attest_encode_sw_components_array()	1323
8.301 secure_fw/partitions/initial_attestation/attest_core.c File Reference	1324
8.301.1 Function Documentation	1325
8.301.1.1 attest_init()	1325
8.301.1.2 initial_attest_get_token()	1325
8.301.1.3 initial_attest_get_token_size()	1326
8.302 secure_fw/partitions/initial_attestation/attest_key.h File Reference	1326
8.302.1 Function Documentation	1327
8.302.1.1 attest_get_instance_id()	1327
8.303 secure_fw/partitions/initial_attestation/attest_symmetric_key.c File Reference	1327
8.303.1 Macro Definition Documentation	1328
8.303.1.1 INSTANCE_ID_HASH_ALG	1328
8.303.1.2 KID_BUF_LEN	1328
8.303.1.3 SYMMETRIC_IAK_MAX_SIZE	1328
8.303.2 Function Documentation	1328
8.303.2.1 attest_get_instance_id()	1328
8.304 secure_fw/partitions/initial_attestation/attest_token.h File Reference	1329
8.304.1 Detailed Description	1331
8.304.2 Enumeration Type Documentation	1331
8.304.2.1 attest_token_err_t	1331
8.304.3 Function Documentation	1332
8.304.3.1 attest_token_encode_add_bstr()	1332
8.304.3.2 attest_token_encode_add_cbor()	1332
8.304.3.3 attest_token_encode_add_integer()	1332
8.304.3.4 attest_token_encode_add_tstr()	1333
8.304.3.5 attest_token_encode_borrow_cbor_cntxt()	1333
8.304.3.6 attest_token_encode_finish()	1333
8.304.3.7 attest_token_encode_start()	1334
8.305 secure_fw/partitions/initial_attestation/attest_token_encode.c File Reference	1334
8.305.1 Detailed Description	1335
8.305.2 Function Documentation	1335
8.305.2.1 attest_token_encode_add_bstr()	1335
8.305.2.2 attest_token_encode_add_cbor()	1336
8.305.2.3 attest_token_encode_add_integer()	1336
8.305.2.4 attest_token_encode_add_tstr()	1336

8.305.2.5 attest_token_encode_borrow_cbor_cntxt()	1337
8.305.2.6 attest_token_encode_finish()	1337
8.305.2.7 attest_token_encode_start()	1337
8.306 secure_fw/partitions/initial_attestation/dir_initial_attestation.dox File Reference	1338
8.307 secure_fw/partitions/initial_attestation/tfm_attest.c File Reference	1338
8.307.1 Function Documentation	1339
8.307.1.1 attest_get_boot_data()	1339
8.307.1.2 attest_get_caller_client_id()	1339
8.307.2 Variable Documentation	1340
8.307.2.1 g_attest_caller_id	1340
8.308 secure_fw/partitions/initial_attestation/tfm_attest_req_mngr.c File Reference	1340
8.308.1 Macro Definition Documentation	1341
8.308.1.1 ECC_P256_PUBLIC_KEY_SIZE	1341
8.308.2 Typedef Documentation	1341
8.308.2.1 attest_func_t	1341
8.308.3 Function Documentation	1341
8.308.3.1 attest_partition_init()	1341
8.308.3.2 tfm_attestation_service_sfn()	1341
8.308.4 Variable Documentation	1342
8.308.4.1 g_attest_caller_id	1342
8.309 secure_fw/partitions/internal_trusted_storage/flash/its_flash.c File Reference	1342
8.310 secure_fw/partitions/internal_trusted_storage/flash/its_flash.h File Reference	1342
8.310.1 Macro Definition Documentation	1343
8.310.1.1 ITS_FLASH_ALIGNMENT	1343
8.310.1.2 ITS_FLASH_DEV	1343
8.310.1.3 ITS_FLASH_MAX_ALIGNMENT	1344
8.310.1.4 ITS_FLASH_OPS	1344
8.310.1.5 PS_FLASH_ALIGNMENT	1344
8.311 secure_fw/partitions/internal_trusted_storage/flash/its_flash_nand.c File Reference	1344
8.311.1 Variable Documentation	1344
8.311.1.1 its_flash_fs_ops_nand	1345
8.312 secure_fw/partitions/internal_trusted_storage/flash/its_flash_nand.h File Reference	1345
8.312.1 Detailed Description	1346
8.312.2 Variable Documentation	1346
8.312.2.1 its_flash_fs_ops_nand	1346
8.313 secure_fw/partitions/internal_trusted_storage/flash/its_flash_nor.c File Reference	1346
8.313.1 Variable Documentation	1346
8.313.1.1 its_flash_fs_ops_nor	1347
8.314 secure_fw/partitions/internal_trusted_storage/flash/its_flash_nor.h File Reference	1347
8.314.1 Detailed Description	1347
8.314.2 Variable Documentation	1347
8.314.2.1 its_flash_fs_ops_nor	1347

8.315 secure_fw/partitions/internal_trusted_storage/flash/its_flash_ram.c File Reference	1347
8.315.1 Variable Documentation	1348
8.315.1.1 its_flash_fs_ops_ram	1348
8.316 secure_fw/partitions/internal_trusted_storage/flash/its_flash_ram.h File Reference	1348
8.316.1 Detailed Description	1349
8.316.2 Variable Documentation	1349
8.316.2.1 its_flash_fs_ops_ram	1349
8.317 secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs.c File Reference	1349
8.317.1 Macro Definition Documentation	1350
8.317.1.1 ITS_FLASH_FS_FLAG_DELETE	1350
8.317.1.2 ITS_FLASH_FS_INTERNAL_FLAGS_MASK	1351
8.317.2 Function Documentation	1351
8.317.2.1 its_flash_fs_file_delete()	1351
8.317.2.2 its_flash_fs_file_get_info()	1351
8.317.2.3 its_flash_fs_file_read()	1352
8.317.2.4 its_flash_fs_file_write()	1353
8.317.2.5 its_flash_fs_init_ctx()	1353
8.317.2.6 its_flash_fs_prepare()	1354
8.317.2.7 its_flash_fs_wipe_all()	1355
8.318 secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs.h File Reference	1355
8.318.1 Detailed Description	1357
8.318.2 Macro Definition Documentation	1357
8.318.2.1 ITS_BLOCK_INVALID_ID	1357
8.318.2.2 ITS_FLASH_FS_FLAG_CREATE	1357
8.318.2.3 ITS_FLASH_FS_FLAG_TRUNCATE	1357
8.318.2.4 ITS_FLASH_FS_USER_FLAGS_MASK	1357
8.318.2.5 ITS_FLASH_FS_WRITE_FLAGS_MASK	1357
8.318.3 Typedef Documentation	1358
8.318.3.1 its_flash_fs_ctx_t	1358
8.318.4 Function Documentation	1358
8.318.4.1 its_flash_fs_file_delete()	1358
8.318.4.2 its_flash_fs_file_get_info()	1358
8.318.4.3 its_flash_fs_file_read()	1359
8.318.4.4 its_flash_fs_file_write()	1360
8.318.4.5 its_flash_fs_init_ctx()	1360
8.318.4.6 its_flash_fs_prepare()	1361
8.318.4.7 its_flash_fs_wipe_all()	1362
8.319 secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_dblock.c File Reference	1362
8.319.1 Function Documentation	1363
8.319.1.1 its_flash_fs_dblock_compact_block()	1363
8.319.1.2 its_flash_fs_dblock_read_file()	1364
8.319.1.3 its_flash_fs_dblock_write_file()	1365

8.320 secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_dblock.h File Reference	1366
8.320.1 Function Documentation	1367
8.320.1.1 its_flash_fs_dblock_compact_block()	1367
8.320.1.2 its_flash_fs_dblock_read_file()	1368
8.320.1.3 its_flash_fs_dblock_write_file()	1369
8.321 secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_mblock.c File Reference	1370
8.321.1 Macro Definition Documentation	1371
8.321.1.1 ITS_BLOCK_META_HEADER_SIZE	1371
8.321.1.2 ITS_BLOCK_METADATA_SIZE	1372
8.321.1.3 ITS_FILE_METADATA_SIZE	1372
8.321.1.4 ITS_MAX_BLOCK_DATA_COPY	1372
8.321.1.5 ITS_METADATA_BLOCK0	1372
8.321.1.6 ITS_METADATA_BLOCK1	1372
8.321.1.7 ITS_OTHER_META_BLOCK	1372
8.321.2 Function Documentation	1372
8.321.2.1 its_flash_fs_block_to_block_move()	1372
8.321.2.2 its_flash_fs_mblock_cp_file_meta()	1373
8.321.2.3 its_flash_fs_mblock_cur_data_scratch_id()	1374
8.321.2.4 its_flash_fs_mblock_get_file_idx_flag()	1374
8.321.2.5 its_flash_fs_mblock_get_file_idx_meta()	1375
8.321.2.6 its_flash_fs_mblock_init()	1376
8.321.2.7 its_flash_fs_mblock_meta_update_finalize()	1377
8.321.2.8 its_flash_fs_mblock_migrate_lb0_data_to_scratch()	1377
8.321.2.9 its_flash_fs_mblock_read_block_metadata()	1378
8.321.2.10 its_flash_fs_mblock_read_block_metadata_comp()	1379
8.321.2.11 its_flash_fs_mblock_read_file_meta()	1379
8.321.2.12 its_flash_fs_mblock_reserve_file()	1380
8.321.2.13 its_flash_fs_mblock_reset_metablock()	1381
8.321.2.14 its_flash_fs_mblock_set_data_scratch()	1381
8.321.2.15 its_flash_fs_mblock_update_scratch_block_meta()	1382
8.321.2.16 its_flash_fs_mblock_update_scratch_file_meta()	1382
8.322 secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_mblock.h File Reference	1383
8.322.1 Macro Definition Documentation	1385
8.322.1.1 _T1	1385
8.322.1.2 _T1_COMP	1385
8.322.1.3 _T2	1386
8.322.1.4 _T3	1386
8.322.1.5 ITS_BACKWARD_SUPPORTED_VERSION	1386
8.322.1.6 ITS_LOGICAL_DBLOCK0	1386
8.322.1.7 ITS_METADATA_INVALID_INDEX	1386
8.322.1.8 ITS_SUPPORTED_VERSION	1386
8.322.2 Function Documentation	1386

8.322.2.1	its_flash_fs_block_to_block_move()	1386
8.322.2.2	its_flash_fs_mblock_cp_file_meta()	1387
8.322.2.3	its_flash_fs_mblock_cur_data_scratch_id()	1388
8.322.2.4	its_flash_fs_mblock_get_file_idx_flag()	1388
8.322.2.5	its_flash_fs_mblock_get_file_idx_meta()	1389
8.322.2.6	its_flash_fs_mblock_init()	1390
8.322.2.7	its_flash_fs_mblock_meta_update_finalize()	1391
8.322.2.8	its_flash_fs_mblock_migrate_lb0_data_to_scratch()	1391
8.322.2.9	its_flash_fs_mblock_read_block_metadata()	1392
8.322.2.10	its_flash_fs_mblock_read_block_metadata_comp()	1393
8.322.2.11	its_flash_fs_mblock_read_file_meta()	1393
8.322.2.12	its_flash_fs_mblock_reserve_file()	1394
8.322.2.13	its_flash_fs_mblock_reset_metablock()	1395
8.322.2.14	its_flash_fs_mblock_set_data_scratch()	1395
8.322.2.15	its_flash_fs_mblock_update_scratch_block_meta()	1396
8.322.2.16	its_flash_fs_mblock_update_scratch_file_meta()	1396
8.323	secure_fw/partitions/internal_trusted_storage/its_crypto_interface.c File Reference	1397
8.323.1	Function Documentation	1398
8.323.1.1	tfm_its_crypt_file()	1398
8.324	secure_fw/partitions/internal_trusted_storage/its_crypto_interface.h File Reference	1398
8.324.1	Function Documentation	1399
8.324.1.1	tfm_its_crypt_file()	1399
8.325	secure_fw/partitions/internal_trusted_storage/its_utils.c File Reference	1400
8.325.1	Function Documentation	1400
8.325.1.1	its_utils_check_contained_in()	1400
8.325.1.2	its_utils_validate_fid()	1401
8.326	secure_fw/partitions/internal_trusted_storage/its_utils.h File Reference	1401
8.326.1	Macro Definition Documentation	1403
8.326.1.1	ITS_DATA_SIZE_FIELD_SIZE	1403
8.326.1.2	ITS_DEFAULT_EMPTY_BUFF_VAL	1403
8.326.1.3	ITS_FILE_ID_SIZE	1403
8.326.1.4	ITS_FLAG_SIZE	1403
8.326.1.5	ITS_UTILS_ALIGN	1403
8.326.1.6	ITS_UTILS_BOUND_CHECK	1403
8.326.1.7	ITS_UTILS_IS_ALIGNED	1404
8.326.1.8	ITS_UTILS_MAX	1404
8.326.1.9	ITS_UTILS_MIN	1404
8.326.2	Function Documentation	1404
8.326.2.1	its_utils_check_contained_in()	1405
8.326.2.2	its_utils_validate_fid()	1405
8.327	secure_fw/partitions/internal_trusted_storage/tfm_internal_trusted_storage.c File Reference	1406
8.327.1	Function Documentation	1407

8.327.1.1 tfm_its_get()	1407
8.327.1.2 tfm_its_get_info()	1408
8.327.1.3 tfm_its_init()	1409
8.327.1.4 tfm_its_remove()	1410
8.327.1.5 tfm_its_set()	1411
8.327.2 Variable Documentation	1412
8.327.2.1 p_psa_dest_data	1412
8.327.2.2 p_psa_src_data	1412
8.328 secure_fw/partitions/internal_trusted_storage/tfm_internal_trusted_storage.h File Reference	1412
8.328.1 Function Documentation	1413
8.328.1.1 tfm_its_get()	1413
8.328.1.2 tfm_its_get_info()	1414
8.328.1.3 tfm_its_init()	1415
8.328.1.4 tfm_its_remove()	1416
8.328.1.5 tfm_its_set()	1417
8.329 secure_fw/partitions/internal_trusted_storage/tfm_its_req_mgr.c File Reference	1418
8.329.1 Function Documentation	1418
8.329.1.1 its_req_mgr_read()	1418
8.329.1.2 its_req_mgr_write()	1419
8.329.1.3 tfm_internal_trusted_storage_service_sfn()	1419
8.329.1.4 tfm_its_entry()	1419
8.330 secure_fw/partitions/internal_trusted_storage/tfm_its_req_mgr.h File Reference	1420
8.330.1 Function Documentation	1420
8.330.1.1 its_req_mgr_read()	1421
8.330.1.2 its_req_mgr_write()	1421
8.331 secure_fw/partitions/lib/runtime/assert.c File Reference	1421
8.331.1 Function Documentation	1422
8.331.1.1 __assert_func()	1422
8.332 secure_fw/partitions/lib/runtime/crt_exit.c File Reference	1422
8.332.1 Function Documentation	1423
8.332.1.1 _exit()	1423
8.332.1.2 exit()	1424
8.333 secure_fw/partitions/lib/runtime/crt_memcmp.c File Reference	1424
8.333.1 Function Documentation	1425
8.333.1.1 memcmp()	1425
8.334 secure_fw/partitions/lib/runtime/crt_memmove.c File Reference	1425
8.334.1 Macro Definition Documentation	1426
8.334.1.1 memcpy_f	1426
8.334.2 Function Documentation	1426
8.334.2.1 memmove()	1426
8.335 secure_fw/partitions/lib/runtime/crt_start.c File Reference	1426
8.335.1 Function Documentation	1426

8.335.1.1 _start()	1427
8.335.1.2 main()	1427
8.336 secure_fw/partitions/lib/runtime/crt_strlen.c File Reference	1427
8.336.1 Function Documentation	1428
8.336.1.1 strlen()	1428
8.337 secure_fw/partitions/lib/runtime/crt_strnlen.c File Reference	1428
8.337.1 Function Documentation	1428
8.337.1.1 strnlen()	1428
8.338 secure_fw/partitions/lib/runtime/crt_vprintf.c File Reference	1429
8.338.1 Function Documentation	1429
8.338.1.1 vprintf()	1429
8.339 secure_fw/partitions/lib/runtime/include/service_api.h File Reference	1429
8.339.1 Function Documentation	1430
8.339.1.1 tfm_core_get_boot_data()	1430
8.340 secure_fw/partitions/lib/runtime/include/tfm_string.h File Reference	1431
8.340.1 Function Documentation	1431
8.340.1.1 strnlen()	1431
8.341 secure_fw/partitions/lib/runtime/psa_api_ipc.c File Reference	1432
8.341.1 Function Documentation	1433
8.341.1.1 psa_framework_version()	1433
8.341.1.2 psa_get()	1433
8.341.1.3 psa_panic()	1434
8.341.1.4 psa_read()	1434
8.341.1.5 psa_reply()	1435
8.341.1.6 psa_rot_lifecycle_state()	1435
8.341.1.7 psa_skip()	1435
8.341.1.8 psa_version()	1436
8.341.1.9 psa_wait()	1436
8.341.1.10 psa_write()	1437
8.341.1.11 tfm_psa_call_pack()	1437
8.342 secure_fw/partitions/lib/runtime/service_api.c File Reference	1438
8.342.1 Function Documentation	1438
8.342.1.1 tfm_core_get_boot_data()	1438
8.342.1.2 tfm_flih_func_return()	1439
8.343 secure_fw/partitions/lib/runtime/sfn_common_thread.c File Reference	1439
8.343.1 Function Documentation	1439
8.343.1.1 common_sfn_thread()	1439
8.344 secure_fw/partitions/lib/runtime/sprt_partition_metadata_indicator.c File Reference	1440
8.344.1 Variable Documentation	1440
8.344.1.1 p_partition_metadata	1440
8.345 secure_fw/partitions/lib/runtime/sprt_partition_metadata_indicator.h File Reference	1440
8.345.1 Macro Definition Documentation	1441

8.345.1.1 PART_METADATA	1441
8.345.2 Variable Documentation	1441
8.345.2.1 p_partition_metadata	1441
8.346 secure_fw/partitions/ns_agent_mailbox/ns_agent_mailbox.c File Reference	1442
8.346.1 Enumeration Type Documentation	1442
8.346.1.1 mailbox_state	1442
8.346.2 Function Documentation	1442
8.346.2.1 ns_agent_mailbox_entry()	1443
8.347 secure_fw/partitions/ns_agent_mailbox/ns_agent_mailbox_rpc.c File Reference	1443
8.347.1 Function Documentation	1443
8.347.1.1 tfm_rpc_client_call_handler()	1443
8.347.1.2 tfm_rpc_client_process_new_msg()	1443
8.347.1.3 tfm_rpc_register_ops()	1443
8.347.1.4 tfm_rpc_register_ops_multi_srcs()	1444
8.347.1.5 tfm_rpc_unregister_ops()	1444
8.348 secure_fw/partitions/ns_agent_mailbox/tfm_multi_core_client_id.c File Reference	1444
8.348.1 Function Documentation	1444
8.348.1.1 tfm_multi_core_hal_client_id_translate()	1445
8.348.1.2 tfm_multi_core_register_client_id_range()	1445
8.349 secure_fw/partitions/ns_agent_mailbox/tfm_multi_core_mbox.c File Reference	1445
8.349.1 Function Documentation	1446
8.349.1.1 tfm_multi_core_clear_mbox_irq()	1446
8.349.1.2 tfm_multi_core_set_mbox_irq()	1446
8.350 secure_fw/partitions/ns_agent_mailbox/tfm_spe_mailbox.c File Reference	1446
8.350.1 Macro Definition Documentation	1448
8.350.1.1 MAILBOX_CLEAN_CACHE	1448
8.350.1.2 MAILBOX_INVALIDATE_CACHE	1448
8.350.2 Function Documentation	1448
8.350.2.1 clear_nspe_queue_pend_status()	1448
8.350.2.2 clear_spe_queue_empty_status()	1448
8.350.2.3 get_nspe_queue_pend_status()	1448
8.350.2.4 get_nspe_reply_addr()	1448
8.350.2.5 get_spe_mailbox_msg_handle()	1449
8.350.2.6 get_spe_mailbox_msg_idx()	1449
8.350.2.7 get_spe_queue_empty_status()	1449
8.350.2.8 set_nspe_queue_replied_status()	1449
8.350.2.9 set_spe_queue_empty_status()	1449
8.350.2.10 tfm_inter_core_comm_deinit()	1449
8.350.2.11 tfm_inter_core_comm_disable()	1450
8.350.2.12 tfm_inter_core_comm_enable()	1450
8.350.2.13 tfm_inter_core_comm_init()	1450
8.350.2.14 tfm_mailbox_handle_msg()	1451

8.350.2.15 tfm_mailbox_reply_msg()	1451
8.351 secure_fw/partitions/ns_agent_mailbox/tfm_spe_mailbox.h File Reference	1451
8.351.1 Function Documentation	1452
8.351.1.1 tfm_mailbox_handle_msg()	1452
8.351.1.2 tfm_mailbox_reply_msg()	1453
8.352 secure_fw/partitions/ns_agent_tz/load_info_ns_agent_tz.c File Reference	1453
8.352.1 Macro Definition Documentation	1454
8.352.1.1 TFM_SP_NS_AGENT_NDEPS	1454
8.352.1.2 TFM_SP_NS_AGENT_NSERVS	1454
8.352.2 Function Documentation	1454
8.352.2.1 __aligned()	1454
8.352.2.2 ns_agent_tz_main()	1454
8.352.3 Variable Documentation	1454
8.352.3.1 tfm_sp_ns_agent_tz_load	1454
8.353 secure_fw/partitions/ns_agent_tz/ns_agent_tz.c File Reference	1455
8.353.1 Function Documentation	1455
8.353.1.1 ns_agent_tz_main()	1455
8.354 secure_fw/partitions/ns_agent_tz/ns_agent_tz_v80m.c File Reference	1455
8.354.1 Function Documentation	1456
8.354.1.1 ns_agent_tz_main()	1456
8.355 secure_fw/partitions/ns_agent_tz/psa_api_veneers.c File Reference	1456
8.355.1 Function Documentation	1456
8.355.1.1 tfm_psa_call_veneer()	1457
8.355.1.2 tfm_psa_close_veneer()	1457
8.355.1.3 tfm_psa_connect_veneer()	1457
8.355.1.4 tfm_psa_framework_version_veneer()	1457
8.355.1.5 tfm_psa_version_veneer()	1458
8.356 secure_fw/partitions/ns_agent_tz/psa_api_veneers_common.c File Reference	1458
8.356.1 Macro Definition Documentation	1459
8.356.1.1 S_STACK_INTEGRITY_SIGN	1459
8.356.2 Function Documentation	1459
8.356.2.1 tfm_ns_check_exit_veneer()	1459
8.356.2.2 tfm_ns_check_safe_entry_veneer()	1460
8.356.3 Variable Documentation	1460
8.356.3.1 ns_entry_flag_valid	1460
8.357 secure_fw/partitions/ns_agent_tz/psa_api_veneers_common.h File Reference	1460
8.358 secure_fw/partitions/ns_agent_tz/psa_api_veneers_v80m.c File Reference	1461
8.358.1 Function Documentation	1461
8.358.1.1 tfm_psa_call_veneer()	1461
8.358.1.2 tfm_psa_close_veneer()	1463
8.358.1.3 tfm_psa_connect_veneer()	1463
8.358.1.4 tfm_psa_framework_version_veneer()	1464

8.358.1.5 tfm_psa_version_veneer()	1464
8.358.2 Variable Documentation	1465
8.358.2.1 ret_err	1465
8.359 secure_fw/partitions/platform/dir_platform.dox File Reference	1465
8.360 secure_fw/partitions/platform/platform_sp.c File Reference	1465
8.360.1 Macro Definition Documentation	1466
8.360.1.1 NV_COUNTER_ID_SIZE	1466
8.360.1.2 NV_COUNTER_SIZE	1466
8.360.2 Typedef Documentation	1466
8.360.2.1 plat_func_t	1466
8.360.3 Function Documentation	1466
8.360.3.1 platform_sp_init()	1467
8.360.3.2 platform_sp_system_reset()	1467
8.360.3.3 tfm_platform_service_sfn()	1467
8.361 secure_fw/partitions/platform/platform_sp.h File Reference	1467
8.361.1 Function Documentation	1468
8.361.1.1 platform_sp_init()	1469
8.361.1.2 platform_sp_system_reset()	1469
8.362 secure_fw/partitions/protected_storage/config_ps_check.h File Reference	1469
8.363 secure_fw/partitions/protected_storage/crypto/ps_crypto_interface.c File Reference	1470
8.363.1 Macro Definition Documentation	1472
8.363.1.1 LABEL_LEN	1472
8.363.1.2 PS_CRYPT0_AEAD_ALG	1472
8.363.1.3 PS_CRYPT0_ALG	1472
8.363.1.4 ps_crypto_setkey	1472
8.363.1.5 PS_KEY_TYPE	1472
8.363.1.6 PS_KEY_USAGE	1472
8.363.2 Typedef Documentation	1472
8.363.2.1 PS_ERROR_NOT_AEAD_ALG	1472
8.363.3 Function Documentation	1472
8.363.3.1 ps_crypto_auth_and_decrypt()	1472
8.363.3.2 ps_crypto_authenticate()	1473
8.363.3.3 ps_crypto_encrypt_and_tag()	1473
8.363.3.4 ps_crypto_generate_auth_tag()	1474
8.363.3.5 ps_crypto_get_iv()	1474
8.363.3.6 ps_crypto_init()	1475
8.363.3.7 ps_crypto_set_iv()	1475
8.363.3.8 ps_crypto_to_blocks()	1475
8.364 secure_fw/partitions/protected_storage/crypto/ps_crypto_interface.h File Reference	1476
8.364.1 Function Documentation	1477
8.364.1.1 ps_crypto_auth_and_decrypt()	1477
8.364.1.2 ps_crypto_authenticate()	1479

8.364.1.3 ps_crypto_encrypt_and_tag()	1479
8.364.1.4 ps_crypto_generate_auth_tag()	1480
8.364.1.5 ps_crypto_get_iv()	1480
8.364.1.6 ps_crypto_init()	1481
8.364.1.7 ps_crypto_set_iv()	1481
8.364.1.8 ps_crypto_to_blocks()	1481
8.365 secure_fw/partitions/protected_storage/dir_protected_storage.dox File Reference	1482
8.366 secure_fw/partitions/protected_storage/nv_counters/ps_nv_counters.c File Reference	1482
8.366.1 Function Documentation	1482
8.366.1.1 ps_increment_nv_counter()	1483
8.366.1.2 ps_read_nv_counter()	1483
8.367 secure_fw/partitions/protected_storage/nv_counters/ps_nv_counters.h File Reference	1484
8.367.1 Macro Definition Documentation	1485
8.367.1.1 PS_NV_COUNTER_SIZE	1485
8.367.1.2 TFM_PS_NV_COUNTER_1	1485
8.367.1.3 TFM_PS_NV_COUNTER_2	1485
8.367.1.4 TFM_PS_NV_COUNTER_3	1485
8.367.2 Function Documentation	1485
8.367.2.1 ps_increment_nv_counter()	1485
8.367.2.2 ps_read_nv_counter()	1486
8.368 secure_fw/partitions/protected_storage/ps_encrypted_object.c File Reference	1486
8.368.1 Macro Definition Documentation	1487
8.368.1.1 PS_CRYPTOFBUF_LEN	1487
8.368.1.2 PS_ENCRYPT_SIZE	1487
8.368.1.3 PS_MAX_ENCRYPTED_OBJ_SIZE	1488
8.368.1.4 PS_OBJECT_START_POSITION	1488
8.368.1.5 STORED_HEADER_DATA_SIZE	1488
8.368.2 Function Documentation	1488
8.368.2.1 ps_encrypted_object_blocks()	1488
8.368.2.2 ps_encrypted_object_read()	1488
8.368.2.3 ps_encrypted_object_write()	1490
8.368.3 Variable Documentation	1490
8.368.3.1 auth_data_t	1490
8.369 secure_fw/partitions/protected_storage/ps_encrypted_object.h File Reference	1491
8.369.1 Function Documentation	1492
8.369.1.1 ps_encrypted_object_blocks()	1492
8.369.1.2 ps_encrypted_object_read()	1492
8.369.1.3 ps_encrypted_object_write()	1493
8.370 secure_fw/partitions/protected_storage/ps_filesystem_interface.c File Reference	1493
8.370.1 Function Documentation	1494
8.370.1.1 psa_its_get()	1494
8.370.1.2 psa_its_get_info()	1495

8.370.1.3	psa_its_remove()	1496
8.370.1.4	psa_its_set()	1497
8.370.2	Variable Documentation	1498
8.370.2.1	p_psa_dest_data	1498
8.370.2.2	p_psa_src_data	1498
8.371	secure_fw/partitions/protected_storage/ps_object_defs.h File Reference	1498
8.371.1	Macro Definition Documentation	1499
8.371.1.1	PS_MAX_NUM_OBJECTS	1499
8.371.1.2	PS_MAX_OBJECT_DATA_SIZE	1500
8.371.1.3	PS_MAX_OBJECT_SIZE	1500
8.371.1.4	PS_OBJECT_BUF_SIZE	1500
8.371.1.5	PS_OBJECT_HEADER_SIZE	1500
8.372	secure_fw/partitions/protected_storage/ps_object_system.c File Reference	1500
8.372.1	Macro Definition Documentation	1501
8.372.1.1	PS_OBJECT_SIZE	1501
8.372.1.2	PS_OBJECT_START_POSITION	1501
8.372.2	Enumeration Type Documentation	1501
8.372.2.1	read_type_t	1501
8.372.3	Function Documentation	1502
8.372.3.1	ps_init_empty_object()	1502
8.372.3.2	ps_object_create()	1502
8.372.3.3	ps_object_delete()	1503
8.372.3.4	ps_object_get_info()	1504
8.372.3.5	ps_object_read()	1505
8.372.3.6	ps_object_write()	1506
8.372.3.7	ps_system_prepare()	1507
8.372.3.8	ps_system_wipe_all()	1507
8.373	secure_fw/partitions/protected_storage/ps_object_system.h File Reference	1508
8.373.1	Function Documentation	1509
8.373.1.1	ps_object_create()	1509
8.373.1.2	ps_object_delete()	1510
8.373.1.3	ps_object_get_info()	1511
8.373.1.4	ps_object_read()	1512
8.373.1.5	ps_object_write()	1513
8.373.1.6	ps_system_prepare()	1514
8.373.1.7	ps_system_wipe_all()	1514
8.374	secure_fw/partitions/protected_storage/ps_object_table.c File Reference	1515
8.374.1	Macro Definition Documentation	1517
8.374.1.1	PS_CRYPTTO_ASSOCIATED_DATA	1517
8.374.1.2	PS_CRYPTTO_ASSOCIATED_DATA_LEN	1517
8.374.1.3	PS_FLASH_DEFAULT_VAL	1517
8.374.1.4	PS_INVALID_NVC_VALUE	1517

8.374.1.5 PS_NON_AUTH_OBJ_TABLE_SIZE	1517
8.374.1.6 PS_NUM_OBJ_TABLES	1518
8.374.1.7 PS_OBJ_TABLE_ENTRIES	1518
8.374.1.8 PS_OBJ_TABLE_IDX_0	1518
8.374.1.9 PS_OBJ_TABLE_IDX_1	1518
8.374.1.10 PS_OBJ_TABLE_SIZE	1518
8.374.1.11 PS_OBJECT_FS_ID	1518
8.374.1.12 PS_OBJECT_FS_ID_TO_IDX	1518
8.374.1.13 PS_OBJECT_SYSTEM_VERSION	1519
8.374.1.14 PS_OBJECT_TABLE_OBJECT_OFFSET	1519
8.374.1.15 PS_OBJECTS_TABLE_ENTRY_SIZE	1519
8.374.1.16 PS_TABLE_FS_ID	1519
8.374.2 Typedef Documentation	1519
8.374.2.1 OBJ_TABLE_NOT_FIT_IN_STATIC_OBJ_DATA_BUF	1519
8.374.3 Enumeration Type Documentation	1519
8.374.3.1 ps_obj_table_state	1519
8.374.4 Function Documentation	1520
8.374.4.1 ps_object_table_create()	1520
8.374.4.2 ps_object_table_delete_object()	1520
8.374.4.3 ps_object_table_delete_old_table()	1520
8.374.4.4 ps_object_table_fs_read_table()	1521
8.374.4.5 ps_object_table_fs_write_table()	1521
8.374.4.6 ps_object_table_get_free_fid()	1521
8.374.4.7 ps_object_table_get_obj_tbl_info()	1522
8.374.4.8 ps_object_table_init()	1523
8.374.4.9 ps_object_table_obj_exist()	1523
8.374.4.10 ps_object_table_set_obj_tbl_info()	1524
8.374.4.11 ps_object_table_validate_version()	1524
8.374.4.12 ps_table_free_idx()	1525
8.375 secure_fw/partitions/protected_storage/ps_object_table.h File Reference	1525
8.375.1 Function Documentation	1527
8.375.1.1 ps_object_table_create()	1527
8.375.1.2 ps_object_table_delete_object()	1527
8.375.1.3 ps_object_table_delete_old_table()	1528
8.375.1.4 ps_object_table_get_free_fid()	1528
8.375.1.5 ps_object_table_get_obj_tbl_info()	1528
8.375.1.6 ps_object_table_init()	1530
8.375.1.7 ps_object_table_obj_exist()	1531
8.375.1.8 ps_object_table_set_obj_tbl_info()	1531
8.376 secure_fw/partitions/protected_storage/ps_utils.c File Reference	1531
8.376.1 Function Documentation	1532
8.376.1.1 ps_utils_check_contained_in()	1532

8.377 secure_fw/partitions/protected_storage/ps_utils.h File Reference	1533
8.377.1 Macro Definition Documentation	1534
8.377.1.1 PS_DEFAULT_EMPTY_BUFF_VAL	1534
8.377.1.2 PS_INVALID_FID	1534
8.377.1.3 PS_UTILS_BOUND_CHECK	1534
8.377.1.4 PS_UTILS_MIN	1534
8.377.2 Function Documentation	1535
8.377.2.1 ps_utils_check_contained_in()	1535
8.378 secure_fw/partitions/protected_storage/tfm_protected_storage.c File Reference	1535
8.378.1 Function Documentation	1536
8.378.1.1 tfm_ps_get()	1536
8.378.1.2 tfm_ps_get_info()	1537
8.378.1.3 tfm_ps_get_support()	1538
8.378.1.4 tfm_ps_init()	1538
8.378.1.5 tfm_ps_remove()	1539
8.378.1.6 tfm_ps_set()	1540
8.379 secure_fw/partitions/protected_storage/tfm_protected_storage.h File Reference	1540
8.379.1 Function Documentation	1542
8.379.1.1 tfm_ps_get()	1542
8.379.1.2 tfm_ps_get_info()	1543
8.379.1.3 tfm_ps_get_support()	1544
8.379.1.4 tfm_ps_init()	1544
8.379.1.5 tfm_ps_remove()	1545
8.379.1.6 tfm_ps_set()	1546
8.380 secure_fw/partitions/protected_storage/tfm_ps_req_mgr.c File Reference	1546
8.380.1 Function Documentation	1547
8.380.1.1 ps_req_mgr_read_asset_data()	1547
8.380.1.2 ps_req_mgr_write_asset_data()	1548
8.380.1.3 tfm_protected_storage_service_sfn()	1548
8.380.1.4 tfm_ps_entry()	1548
8.381 secure_fw/partitions/protected_storage/tfm_ps_req_mgr.h File Reference	1549
8.381.1 Function Documentation	1550
8.381.1.1 ps_req_mgr_read_asset_data()	1550
8.381.1.2 ps_req_mgr_write_asset_data()	1550
8.382 secure_fw/shared/crt_memcpy.c File Reference	1551
8.382.1 Function Documentation	1551
8.382.1.1 memcpy()	1551
8.383 secure_fw/shared/crt_memset.c File Reference	1553
8.383.1 Function Documentation	1553
8.383.1.1 memset()	1553
8.384 secure_fw/shared/crt_strcmp.c File Reference	1554
8.384.1 Function Documentation	1554

8.384.1.1 strcmp()	1554
8.385 secure_fw/shared/crt_strncmp.c File Reference	1554
8.385.1 Function Documentation	1555
8.385.1.1 strncmp()	1555
8.386 secure_fw/spm/core/arch/tfm_arch.c File Reference	1555
8.386.1 Function Documentation	1556
8.386.1.1 tfm_arch_free_msp_and_exc_ret()	1556
8.386.1.2 tfm_arch_init_context()	1556
8.386.1.3 tfm_arch_refresh_hardware_context()	1557
8.387 secure_fw/spm/core/arch/tfm_arch_v6m_v7m.c File Reference	1557
8.387.1 Function Documentation	1558
8.387.1.1 SVC_Handler()	1558
8.387.1.2 tfm_arch_config_extensions()	1558
8.387.1.3 tfm_arch_set_secure_exception_priorities()	1558
8.387.2 Variable Documentation	1558
8.387.2.1 psp_limit	1558
8.387.2.2 scheduler_lock	1559
8.388 secure_fw/spm/core/arch/tfm_arch_v6m_v7m.h File Reference	1559
8.388.1 Macro Definition Documentation	1560
8.388.1.1 EXC_RETURN_MODE	1560
8.388.1.2 EXC_RETURN_SPSEL	1560
8.388.1.3 SCB_ICSR_ADDR	1560
8.388.1.4 SCB_ICSR_PENDSVSET_BIT	1560
8.388.2 Function Documentation	1560
8.388.2.1 arch_seal_thread_stack()	1560
8.388.2.2 arch_update_process_sp()	1561
8.388.2.3 is_default_stacking_rules_apply()	1561
8.388.2.4 is_return_secure_stack()	1562
8.388.2.5 tfm_arch_check_msp_sealing()	1562
8.388.2.6 tfm_arch_config_branch_protection()	1563
8.388.2.7 tfm_arch_get_psplim()	1563
8.388.2.8 tfm_arch_set_msplim()	1563
8.388.2.9 tfm_arch_set_psplim()	1564
8.388.3 Variable Documentation	1564
8.388.3.1 psp_limit	1564
8.389 secure_fw/spm/core/arch/tfm_arch_v8m_base.c File Reference	1564
8.389.1 Function Documentation	1565
8.389.1.1 SVC_Handler()	1565
8.389.1.2 tfm_arch_config_extensions()	1565
8.389.1.3 tfm_arch_set_secure_exception_priorities()	1565
8.389.2 Variable Documentation	1565
8.389.2.1 scheduler_lock	1565

8.390 secure_fw/spm/core/arch/tfm_arch_v8m_main.c File Reference	1565
8.390.1 Function Documentation	1566
8.390.1.1 SVC_Handler()	1566
8.390.1.2 tfm_arch_config_extensions()	1566
8.390.1.3 tfm_arch_set_secure_exception_priorities()	1567
8.390.2 Variable Documentation	1567
8.390.2.1 scheduler_lock	1567
8.391 secure_fw/spm/core/backend_ipc.c File Reference	1567
8.391.1 Typedef Documentation	1568
8.391.1.1 comp_init_fn_t	1568
8.391.2 Function Documentation	1568
8.391.2.1 ARCH_CLAIM_CTXCTRL_INSTANCE()	1568
8.391.2.2 backend_abi_entering_spm()	1568
8.391.2.3 backend_abi_leaving_spm()	1569
8.391.2.4 backend_assert_signal()	1569
8.391.2.5 backend_init_comp_assuredly()	1569
8.391.2.6 backend_messaging()	1570
8.391.2.7 backend_replying()	1570
8.391.2.8 backend_system_run()	1570
8.391.2.9 backend_wait_signals()	1570
8.391.2.10 common_sfn_thread()	1571
8.391.2.11 ipc_schedule()	1571
8.391.3 Variable Documentation	1571
8.391.3.1 p_spm_thread_context	1572
8.391.3.2 partition_listhead	1572
8.391.3.3 psa_api_svc	1572
8.391.3.4 psa_api_thread_fn_call	1572
8.392 secure_fw/spm/core/backend_sfn.c File Reference	1572
8.392.1 Macro Definition Documentation	1573
8.392.1.1 SFN_PARTITION_STATE_INITED	1573
8.392.1.2 SFN_PARTITION_STATE_NOT_INITED	1573
8.392.2 Function Documentation	1573
8.392.2.1 backend_assert_signal()	1573
8.392.2.2 backend_init_comp_assuredly()	1574
8.392.2.3 backend_messaging()	1574
8.392.2.4 backend_replying()	1574
8.392.2.5 backend_system_run()	1574
8.392.2.6 backend_wait_signals()	1575
8.392.3 Variable Documentation	1575
8.392.3.1 p_current_partition	1575
8.392.3.2 partition_listhead	1575
8.393 secure_fw/spm/core/internal_status_code.h File Reference	1575

8.393.1 Macro Definition Documentation	1576
8.393.1.1 SPM_ERROR_BAD_PARAMETERS	1576
8.393.1.2 SPM_ERROR_GENERIC	1576
8.393.1.3 SPM_ERROR_MEMORY_CHECK	1577
8.393.1.4 SPM_ERROR_SHORT_BUFFER	1577
8.393.1.5 SPM_ERROR_VERSION	1577
8.393.1.6 SPM_SUCCESS	1577
8.393.1.7 STATUS_NEED_SCHEDULE	1577
8.394 secure_fw/spm/core/interrupt.c File Reference	1577
8.394.1 Function Documentation	1578
8.394.1.1 get_irq_info_for_signal()	1578
8.394.1.2 spm_handle_interrupt()	1579
8.394.1.3 tfm_flih_func_return()	1579
8.394.1.4 tfm_flih_prepare_depriv_flih()	1579
8.394.1.5 tfm_flih_return_to_isr()	1580
8.395 secure_fw/spm/core/interrupt.h File Reference	1580
8.395.1 Function Documentation	1581
8.395.1.1 get_irq_info_for_signal()	1581
8.395.1.2 spm_handle_interrupt()	1582
8.395.1.3 tfm_flih_prepare_depriv_flih()	1583
8.395.1.4 tfm_flih_return_to_isr()	1583
8.395.1.5 tfm_get_isr_cookie()	1584
8.396 secure_fw/spm/core/mailbox_agent_api.c File Reference	1584
8.396.1 Function Documentation	1585
8.396.1.1 tfm_spm_agent_psa_call()	1585
8.397 secure_fw/spm/core/main.c File Reference	1585
8.397.1 Function Documentation	1586
8.397.1.1 get_spm_boundary()	1586
8.397.1.2 main()	1586
8.398 secure_fw/spm/core/memory_symbols.h File Reference	1587
8.398.1 Macro Definition Documentation	1587
8.398.1.1 PART_INFOLIST_END	1588
8.398.1.2 PART_INFOLIST_START	1588
8.398.1.3 PART_INFORAM_END	1588
8.398.1.4 PART_INFORAM_START	1588
8.398.1.5 SERV_INFORAM_END	1588
8.398.1.6 SERV_INFORAM_START	1588
8.398.1.7 SPM_BOOT_STACK_BOTTOM	1588
8.398.1.8 SPM_BOOT_STACK_TOP	1588
8.398.2 Function Documentation	1588
8.398.2.1 REGION_DECLARE() [1/4]	1588
8.398.2.2 REGION_DECLARE() [2/4]	1589

8.398.2.3 REGION_DECLARE() [3/4]	1589
8.398.2.4 REGION_DECLARE() [4/4]	1589
8.399 secure_fw/spm/core/ns_agent_mailbox_interface.c File Reference	1589
8.399.1 Function Documentation	1590
8.399.1.1 ns_agent_boot_ns_core()	1590
8.399.1.2 ns_agent_mailbox_disable()	1590
8.399.1.3 ns_agent_mailbox_enable()	1590
8.399.1.4 ns_agent_ns_core_shutdown()	1591
8.400 secure_fw/spm/core/psa_api.c File Reference	1591
8.400.1 Function Documentation	1592
8.400.1.1 spm_handle_programmer_errors()	1592
8.400.1.2 tfm_spm_get_lifecycle_state()	1593
8.400.1.3 tfm_spm_partition_psa_panic()	1594
8.400.1.4 tfm_spm_partition_psa_reply()	1594
8.401 secure_fw/spm/core/psa_call_api.c File Reference	1595
8.401.1 Function Documentation	1595
8.401.1.1 spm_associate_call_params()	1596
8.401.1.2 tfm_spm_client_psa_call()	1596
8.402 secure_fw/spm/core/psa_connection_api.c File Reference	1597
8.402.1 Function Documentation	1597
8.402.1.1 spm_psa_close_client_id_associated()	1597
8.402.1.2 spm_psa_connect_client_id_associated()	1597
8.402.1.3 tfm_spm_client_psa_close()	1598
8.402.1.4 tfm_spm_client_psa_connect()	1598
8.402.1.5 tfm_spm_partition_psa_set_rhandle()	1599
8.403 secure_fw/spm/core/psa_interface_sfn.c File Reference	1599
8.403.1 Function Documentation	1600
8.403.1.1 psa_framework_version()	1600
8.403.1.2 psa_panic()	1600
8.403.1.3 psa_read()	1601
8.403.1.4 psa_skip()	1602
8.403.1.5 psa_version()	1603
8.403.1.6 psa_write()	1604
8.403.1.7 tfm_psa_call_pack()	1605
8.404 secure_fw/spm/core/psa_interface_svc.c File Reference	1605
8.404.1 Function Documentation	1606
8.404.1.1 psa_framework_version_svc()	1606
8.404.1.2 psa_get_svc()	1606
8.404.1.3 psa_panic_svc()	1606
8.404.1.4 psa_read_svc()	1607
8.404.1.5 psa_reply_svc()	1607
8.404.1.6 psa_rot_lifecycle_state_svc()	1607

8.404.1.7	psa_skip_svc()	1607
8.404.1.8	psa_version_svc()	1607
8.404.1.9	psa_wait_svc()	1607
8.404.1.10	psa_write_svc()	1607
8.404.1.11	tfm_psa_call_pack_svc()	1608
8.404.2	Variable Documentation	1608
8.404.2.1	psa_api_svc	1608
8.405	secure_fw/spm/core/psa_interface_thread_fn_call.c File Reference	1608
8.405.1	Macro Definition Documentation	1609
8.405.1.1	TFM_THREAD_FN_CALL_ENTRY	1609
8.405.2	Function Documentation	1609
8.405.2.1	psa_framework_version_thread_fn_call()	1609
8.405.2.2	psa_get_thread_fn_call()	1609
8.405.2.3	psa_panic_thread_fn_call()	1610
8.405.2.4	psa_read_thread_fn_call()	1610
8.405.2.5	psa_reply_thread_fn_call()	1610
8.405.2.6	psa_rot_lifecycle_state_thread_fn_call()	1611
8.405.2.7	psa_skip_thread_fn_call()	1611
8.405.2.8	psa_version_thread_fn_call()	1611
8.405.2.9	psa_wait_thread_fn_call()	1612
8.405.2.10	psa_write_thread_fn_call()	1612
8.405.2.11	tfm_psa_call_pack_thread_fn_call()	1612
8.405.3	Variable Documentation	1612
8.405.3.1	psa_api_thread_fn_call	1613
8.406	secure_fw/spm/core/psa_irq_api.c File Reference	1613
8.406.1	Function Documentation	1613
8.406.1.1	tfm_spm_partition_psa_irq_disable()	1613
8.406.1.2	tfm_spm_partition_psa_irq_enable()	1614
8.407	secure_fw/spm/core/psa_mmiovec_api.c File Reference	1614
8.407.1	Function Documentation	1614
8.407.1.1	tfm_spm_partition_psa_map_invec()	1614
8.407.1.2	tfm_spm_partition_psa_map_outvec()	1615
8.407.1.3	tfm_spm_partition_psa_unmap_invec()	1615
8.407.1.4	tfm_spm_partition_psa_unmap_outvec()	1615
8.408	secure_fw/spm/core/psa_read_write_skip_api.c File Reference	1615
8.408.1	Function Documentation	1615
8.408.1.1	tfm_spm_partition_psa_read()	1616
8.408.1.2	tfm_spm_partition_psa_skip()	1616
8.408.1.3	tfm_spm_partition_psa_write()	1617
8.409	secure_fw/spm/core/psa_version_api.c File Reference	1618
8.409.1	Function Documentation	1618
8.409.1.1	tfm_spm_client_psa_framework_version()	1619

8.409.1.2 tfm_spm_client_psa_version()	1619
8.410 secure_fw/spm/core/rom_loader.c File Reference	1620
8.410.1 Function Documentation	1621
8.410.1.1 load_a_partition_assuredly()	1621
8.410.1.2 load_irqs_assuredly()	1621
8.410.1.3 load_services_assuredly()	1622
8.411 secure_fw/spm/core/spm.h File Reference	1622
8.411.1 Macro Definition Documentation	1624
8.411.1.1 CLIENT_HANDLE_VALUE_MIN	1624
8.411.1.2 GET_CTX_OWNER	1624
8.411.1.3 GET_INDEX_FROM_STATIC_HANDLE	1624
8.411.1.4 GET_THRD_OWNER	1624
8.411.1.5 GET_VERSION_FROM_STATIC_HANDLE	1624
8.411.1.6 IS_STATIC_HANDLE	1624
8.411.1.7 PSA_TIMEOUT_MASK	1624
8.411.1.8 SPM_INVALID_PARTITION_IDX	1624
8.411.1.9 STATIC_HANDLE_IDX_BIT_WIDTH	1625
8.411.1.10 STATIC_HANDLE_IDX_MASK	1625
8.411.1.11 STATIC_HANDLE_INDICATOR_OFFSET	1625
8.411.1.12 STATIC_HANDLE_NUM_LIMIT	1625
8.411.1.13 STATIC_HANDLE_VER_BIT_WIDTH	1625
8.411.1.14 STATIC_HANDLE_VER_MASK	1625
8.411.1.15 STATIC_HANDLE_VER_OFFSET	1625
8.411.1.16 TFM_CLIENT_ID_IS_NS	1625
8.411.1.17 tfm_spm_is_rpc_msg	1625
8.411.2 Enumeration Type Documentation	1625
8.411.2.1 connection_status	1626
8.411.3 Function Documentation	1626
8.411.3.1 connection_to_handle()	1626
8.411.3.2 handle_to_connection()	1626
8.411.3.3 ipc_schedule()	1626
8.411.3.4 spm_allocate_connection()	1627
8.411.3.5 spm_associate_call_params()	1628
8.411.3.6 spm_free_connection()	1628
8.411.3.7 spm_get_idle_connection()	1628
8.411.3.8 spm_init_connection_space()	1629
8.411.3.9 spm_init_idle_connection()	1629
8.411.3.10 spm_msg_handle_to_connection()	1630
8.411.3.11 spm_validate_connection()	1631
8.411.3.12 tfm_spm_check_authorization()	1631
8.411.3.13 tfm_spm_check_client_version()	1632
8.411.3.14 tfm_spm_get_client_id()	1633

8.411.3.15 tfm_spm_get_service_by_sid()	1634
8.411.3.16 tfm_spm_init()	1634
8.411.3.17 tfm_spm_is_ns_caller()	1635
8.411.3.18 tfm_spm_partition_get_running_partition_id()	1636
8.412 secure_fw/spm/core/spm_connection_pool.c File Reference	1636
8.412.1 Macro Definition Documentation	1637
8.412.1.1 CONVERSION_FACTOR_BITOFFSET	1637
8.412.1.2 CONVERSION_FACTOR_VALUE	1637
8.412.1.3 CONVERSION_FACTOR_VALUE_MAX	1637
8.412.2 Function Documentation	1637
8.412.2.1 connection_to_handle()	1637
8.412.2.2 handle_to_connection()	1637
8.412.2.3 spm_allocate_connection()	1637
8.412.2.4 spm_free_connection()	1638
8.412.2.5 spm_init_connection_space()	1638
8.412.2.6 spm_validate_connection()	1638
8.413 secure_fw/spm/core/spm_ipc.c File Reference	1639
8.413.1 Function Documentation	1640
8.413.1.1 spm_get_idle_connection()	1640
8.413.1.2 spm_init_idle_connection()	1641
8.413.1.3 spm_msg_handle_to_connection()	1641
8.413.1.4 tfm_spm_check_authorization()	1642
8.413.1.5 tfm_spm_check_client_version()	1643
8.413.1.6 tfm_spm_get_client_id()	1643
8.413.1.7 tfm_spm_get_service_by_sid()	1644
8.413.1.8 tfm_spm_init()	1645
8.413.1.9 tfm_spm_is_ns_caller()	1645
8.413.1.10 tfm_spm_partition_get_running_partition_id()	1646
8.413.2 Variable Documentation	1646
8.413.2.1 stateless_services_ref_tbl	1646
8.414 secure_fw/spm/core/spm_local_connection.c File Reference	1647
8.414.1 Macro Definition Documentation	1647
8.414.1.1 CONNECTION_SIZE	1647
8.414.1.2 FIXED_STATIC_HANDLE	1647
8.414.2 Function Documentation	1647
8.414.2.1 connection_to_handle()	1648
8.414.2.2 handle_to_connection()	1648
8.414.2.3 spm_allocate_connection()	1648
8.414.2.4 spm_free_connection()	1648
8.414.2.5 spm_init_connection_space()	1649
8.414.2.6 spm_validate_connection()	1649
8.415 secure_fw/spm/core/stack_watermark.c File Reference	1649

8.415.1 Macro Definition Documentation	1650
8.415.1.1 SPMLOG	1650
8.415.1.2 SPMLOG_VAL	1650
8.415.1.3 STACK_WATERMARK_VAL	1650
8.415.2 Function Documentation	1650
8.415.2.1 for()	1650
8.415.2.2 spm_log_msgval() [1/2]	1651
8.415.2.3 spm_log_msgval() [2/2]	1651
8.415.2.4 tfm_hal_output_spm_log() [1/2]	1651
8.415.2.5 tfm_hal_output_spm_log() [2/2]	1651
8.415.2.6 UNI_LIST_FOREACH()	1651
8.415.2.7 while()	1651
8.415.3 Variable Documentation	1652
8.415.3.1 void	1652
8.416 secure_fw/spm/core/stack_watermark.h File Reference	1652
8.416.1 Macro Definition Documentation	1652
8.416.1.1 dump_used_stacks	1652
8.416.1.2 watermark_spm_stack	1653
8.416.1.3 watermark_stack	1653
8.417 secure_fw/spm/core/tfm_boot_data.c File Reference	1653
8.417.1 Macro Definition Documentation	1654
8.417.1.1 BOOT_DATA_INVALID	1654
8.417.1.2 BOOT_DATA_VALID	1654
8.417.2 Function Documentation	1654
8.417.2.1 tfm_core_get_boot_data_handler()	1654
8.417.2.2 tfm_core_validate_boot_data()	1654
8.418 secure_fw/spm/core/tfm_boot_data.h File Reference	1654
8.418.1 Function Documentation	1655
8.418.1.1 tfm_core_get_boot_data_handler()	1655
8.418.1.2 tfm_core_validate_boot_data()	1656
8.419 secure_fw/spm/core/tfm_multi_core.h File Reference	1656
8.419.1 Macro Definition Documentation	1657
8.419.1.1 CLIENT_ID_OWNER_MAGIC	1657
8.419.2 Function Documentation	1657
8.419.2.1 tfm_inter_core_comm_deinit()	1657
8.419.2.2 tfm_inter_core_comm_disable()	1657
8.419.2.3 tfm_inter_core_comm_enable()	1658
8.419.2.4 tfm_inter_core_comm_init()	1658
8.419.2.5 tfm_multi_core_clear_mbox_irq()	1658
8.419.2.6 tfm_multi_core_hal_client_id_translate()	1658
8.419.2.7 tfm_multi_core_register_client_id_range()	1659
8.419.2.8 tfm_multi_core_set_mbox_irq()	1659

8.420 secure_fw/spm/core/tfm_pools.c File Reference	1659
8.420.1 Macro Definition Documentation	1660
8.420.1.1 POOL_MAGIC_ALLOCATED	1660
8.420.2 Function Documentation	1660
8.420.2.1 is_valid_chunk_data_in_pool()	1660
8.420.2.2 tfm_pool_alloc()	1661
8.420.2.3 tfm_pool_free()	1661
8.420.2.4 tfm_pool_init()	1662
8.421 secure_fw/spm/core/tfm_pools.h File Reference	1663
8.421.1 Macro Definition Documentation	1664
8.421.1.1 POOL_BUFFER_SIZE	1664
8.421.1.2 POOL_HEAD_SIZE	1664
8.421.1.3 TFM_POOL_DECLARE	1665
8.421.2 Function Documentation	1665
8.421.2.1 is_valid_chunk_data_in_pool()	1665
8.421.2.2 tfm_pool_alloc()	1665
8.421.2.3 tfm_pool_free()	1666
8.421.2.4 tfm_pool_init()	1667
8.422 secure_fw/spm/core/tfm_rpc.c File Reference	1668
8.422.1 Function Documentation	1668
8.422.1.1 tfm_rpc_psa_call()	1668
8.422.1.2 tfm_rpc_psa_framework_version()	1669
8.422.1.3 tfm_rpc_psa_version()	1669
8.423 secure_fw/spm/core/tfm_rpc.h File Reference	1669
8.424 secure_fw/spm/core/tfm_svcalls.c File Reference	1669
8.424.1 Macro Definition Documentation	1670
8.424.1.1 INVALID_PSP_VALUE	1670
8.424.2 Function Documentation	1670
8.424.2.1 spm_svc_handler()	1671
8.424.2.2 tfm_svc_thread_mode_spm_return()	1671
8.425 secure_fw/spm/core/tfm_svcalls.h File Reference	1671
8.425.1 Function Documentation	1672
8.425.1.1 spm_svc_handler()	1672
8.425.1.2 tfm_svc_thread_mode_spm_return()	1673
8.426 secure_fw/spm/core/thread.c File Reference	1673
8.426.1 Macro Definition Documentation	1674
8.426.1.1 LIST_HEAD	1674
8.426.1.2 RNBL_HEAD	1674
8.426.2 Function Documentation	1674
8.426.2.1 thrd_next()	1674
8.426.2.2 thrd_set_query_callback()	1675
8.426.2.3 thrd_set_state()	1675

8.426.2.4	thrd_start()	1675
8.426.2.5	thrd_start_scheduler()	1675
8.426.3	Variable Documentation	1675
8.426.3.1	p_curr_thrd	1676
8.427	secure_fw/spm/core/thread.h File Reference	1676
8.427.1	Macro Definition Documentation	1677
8.427.1.1	CURRENT_THREAD	1677
8.427.1.2	THRD_ERR_GENERIC	1677
8.427.1.3	THRD_GENERAL_EXIT	1677
8.427.1.4	THRD_INIT	1677
8.427.1.5	THRD_PRIOR_HIGH	1677
8.427.1.6	THRD_PRIOR_HIGHEST	1678
8.427.1.7	THRD_PRIOR_LOW	1678
8.427.1.8	THRD_PRIOR_LOWEST	1678
8.427.1.9	THRD_PRIOR_MEDIUM	1678
8.427.1.10	THRD_SET_PRIORITY	1678
8.427.1.11	THRD_STATE_BLOCK	1678
8.427.1.12	THRD_STATE_CREATING	1678
8.427.1.13	THRD_STATE_DETACH	1678
8.427.1.14	THRD_STATE_INVALID	1678
8.427.1.15	THRD_STATE_RET_VAL_AVAIL	1678
8.427.1.16	THRD_STATE_RUNNABLE	1679
8.427.1.17	THRD_SUCCESS	1679
8.427.1.18	THRD_UPDATE_CUR_CTXCTRL	1679
8.427.2	Typedef Documentation	1679
8.427.2.1	thrd_fn_t	1679
8.427.2.2	thrd_query_state_t	1679
8.427.3	Function Documentation	1679
8.427.3.1	thrd_next()	1679
8.427.3.2	thrd_set_query_callback()	1680
8.427.3.3	thrd_set_state()	1680
8.427.3.4	thrd_start()	1680
8.427.3.5	thrd_start_scheduler()	1680
8.427.4	Variable Documentation	1680
8.427.4.1	p_curr_thrd	1681
8.428	platform/ext/target/infineon/common/nspe/os_wrapper/thread.h File Reference	1681
8.428.1	Typedef Documentation	1682
8.428.1.1	os_wrapper_thread_func	1682
8.428.2	Function Documentation	1682
8.428.2.1	os_wrapper_thread_exit()	1682
8.428.2.2	os_wrapper_thread_get_handle()	1682
8.428.2.3	os_wrapper_thread_get_priority()	1682

8.428.2.4	os_wrapper_thread_new()	1683
8.428.2.5	os_wrapper_thread_set_flag()	1683
8.428.2.6	os_wrapper_thread_set_flag_isr()	1683
8.428.2.7	os_wrapper_thread_wait_flag()	1685
8.429	secure_fw/spm/core/utilities.c File Reference	1685
8.429.1	Function Documentation	1686
8.429.1.1	tfm_core_panic()	1686
8.430	secure_fw/spm/include/aapcs_local.h File Reference	1687
8.430.1	Macro Definition Documentation	1688
8.430.1.1	AAPCS_DUAL_U32_AS_U64	1688
8.430.1.2	AAPCS_DUAL_U32_SET	1688
8.430.1.3	AAPCS_DUAL_U32_SET_A0	1689
8.430.1.4	AAPCS_DUAL_U32_SET_A1	1689
8.430.1.5	AAPCS_DUAL_U32_T	1689
8.431	secure_fw/spm/include/bitops.h File Reference	1689
8.431.1	Macro Definition Documentation	1690
8.431.1.1	IS_ONLY_ONE_BIT_IN_UINT32	1690
8.432	secure_fw/spm/include/boot/tfm_boot_status.h File Reference	1690
8.432.1	Macro Definition Documentation	1692
8.432.1.1	CLAIM_MASK	1692
8.432.1.2	CLAIM_POS	1692
8.432.1.3	GET_FWU_CLAIM	1692
8.432.1.4	GET_FWU_MODULE	1692
8.432.1.5	GET_IAS_CLAIM	1692
8.432.1.6	GET_IAS_MEASUREMENT_CLAIM	1692
8.432.1.7	GET_IAS_MODULE	1693
8.432.1.8	GET_MAJOR	1693
8.432.1.9	GET_MBS_CLAIM	1693
8.432.1.10	GET_MBS_SLOT	1693
8.432.1.11	GET_MINOR	1693
8.432.1.12	MAJOR_MASK	1693
8.432.1.13	MAJOR_POS	1693
8.432.1.14	MASK_LEFT_SHIFT	1693
8.432.1.15	MASK_RIGHT_SHIFT	1693
8.432.1.16	MEASUREMENT_CLAIM_POS	1694
8.432.1.17	MINOR_MASK	1694
8.432.1.18	MINOR_POS	1694
8.432.1.19	MODULE_MASK	1694
8.432.1.20	MODULE_POS	1694
8.432.1.21	SET_FWU_MINOR	1694
8.432.1.22	SET_IAS_MINOR	1694
8.432.1.23	SET_MBS_MINOR	1695

8.432.1.24 SET_TLV_TYPE	1695
8.432.1.25 SHARED_DATA_ENTRY_HEADER_SIZE	1695
8.432.1.26 SHARED_DATA_ENTRY_SIZE	1695
8.432.1.27 SHARED_DATA_HEADER_SIZE	1695
8.432.1.28 SHARED_DATA_TLV_INFO_MAGIC	1695
8.432.1.29 SLOT_ID_MASK	1695
8.432.1.30 SLOT_ID_POS	1695
8.432.1.31 SW_AROT	1696
8.432.1.32 SW_BL2	1696
8.432.1.33 SW_BOOT_RECORD	1696
8.432.1.34 SW_GENERAL	1696
8.432.1.35 SW_MAX	1696
8.432.1.36 SW_MEASURE_METADATA	1696
8.432.1.37 SW_MEASURE_TYPE	1697
8.432.1.38 SW_MEASURE_VALUE	1697
8.432.1.39 SW_MEASURE_VALUE_NON_EXTENDABLE	1697
8.432.1.40 SW_NSPE	1697
8.432.1.41 SW_PROT	1697
8.432.1.42 SW_S_NS	1697
8.432.1.43 SW_SIGNER_ID	1697
8.432.1.44 SW_SPE	1697
8.432.1.45 SW_TYPE	1697
8.432.1.46 SW_VERSION	1697
8.432.1.47 TLV_MAJOR_CORE	1698
8.432.1.48 TLV_MAJOR_FWU	1698
8.432.1.49 TLV_MAJOR_IAS	1698
8.432.1.50 TLV_MAJOR_INVALID	1698
8.432.1.51 TLV_MAJOR_MBS	1698
8.433 secure_fw/spm/include/config_spm.h File Reference	1698
8.434 secure_fw/spm/include/critical_section.h File Reference	1699
8.434.1 Macro Definition Documentation	1699
8.434.1.1 CRITICAL_SECTION_ENTER	1699
8.434.1.2 CRITICAL_SECTION_INIT	1700
8.434.1.3 CRITICAL_SECTION_LEAVE	1700
8.434.1.4 CRITICAL_SECTION_STATIC_INIT	1700
8.435 secure_fw/spm/include/current.h File Reference	1700
8.435.1 Macro Definition Documentation	1700
8.435.1.1 GET_CURRENT_COMPONENT	1700
8.435.1.2 SET_CURRENT_COMPONENT	1701
8.436 secure_fw/spm/include/ffm/backend.h File Reference	1701
8.436.1 Macro Definition Documentation	1701
8.436.1.1 PARTITION_LIST_ADDR	1702

8.436.1.2 SPM_THREAD_CONTEXT	1702
8.436.2 Function Documentation	1702
8.436.2.1 backend_assert_signal()	1702
8.436.2.2 backend_init_comp_assuredly()	1702
8.436.2.3 backend_messaging()	1703
8.436.2.4 backend_replying()	1703
8.436.2.5 backend_system_run()	1704
8.436.2.6 backend_wait_signals()	1704
8.436.3 Variable Documentation	1705
8.436.3.1 p_spm_thread_context	1705
8.436.3.2 partition_listhead	1705
8.437 secure_fw/spm/include/ffm/backend_ipc.h File Reference	1705
8.437.1 Macro Definition Documentation	1705
8.437.1.1 BACKEND_SERVICE_SET	1705
8.437.1.2 BACKEND_SPM_INIT	1706
8.437.2 Function Documentation	1706
8.437.2.1 backend_abi_entering_spm()	1706
8.437.2.2 backend_abi_leaving_spm()	1706
8.438 secure_fw/spm/include/ffm/backend_sfn.h File Reference	1706
8.438.1 Macro Definition Documentation	1707
8.438.1.1 BACKEND_SERVICE_SET	1707
8.438.1.2 BACKEND_SPM_INIT	1707
8.439 secure_fw/spm/include/ffm/mailbox_agent_api.h File Reference	1707
8.439.1 Macro Definition Documentation	1708
8.439.1.1 agent_psa_close	1708
8.439.1.2 agent_psa_connect	1708
8.439.2 Function Documentation	1708
8.439.2.1 agent_psa_call()	1708
8.440 secure_fw/spm/include/ffm/psa_api.h File Reference	1709
8.440.1 Macro Definition Documentation	1710
8.440.1.1 tfm_spm_agent_psa_call	1710
8.440.1.2 tfm_spm_agent_psa_close	1710
8.440.1.3 tfm_spm_agent_psa_connect	1710
8.440.1.4 tfm_spm_client_psa_close	1710
8.440.1.5 tfm_spm_client_psa_connect	1710
8.440.1.6 tfm_spm_partition_psa_clear	1710
8.440.1.7 tfm_spm_partition_psa_eoi	1711
8.440.1.8 tfm_spm_partition_psa_irq_disable	1711
8.440.1.9 tfm_spm_partition_psa_irq_enable	1711
8.440.1.10 tfm_spm_partition_psa_notify	1711
8.440.1.11 tfm_spm_partition_psa_reset_signal	1711
8.440.1.12 tfm_spm_partition_psa_set_rhandle	1711

8.440.2 Function Documentation	1711
8.440.2.1 spm_handle_programmer_errors()	1711
8.440.2.2 tfm_spm_client_psa_call()	1712
8.440.2.3 tfm_spm_client_psa_framework_version()	1713
8.440.2.4 tfm_spm_client_psa_version()	1713
8.440.2.5 tfm_spm_get_lifecycle_state()	1714
8.440.2.6 tfm_spm_partition_psa_panic()	1715
8.440.2.7 tfm_spm_partition_psa_read()	1715
8.440.2.8 tfm_spm_partition_psa_reply()	1716
8.440.2.9 tfm_spm_partition_psa_skip()	1717
8.440.2.10 tfm_spm_partition_psa_write()	1718
8.441 secure_fw/spm/include/interface/runtime_defs.h File Reference	1718
8.441.1 Typedef Documentation	1719
8.441.1.1 service_fn_t	1719
8.441.1.2 sfn_init_fn_t	1719
8.442 secure_fw/spm/include/interface/svc_num.h File Reference	1720
8.442.1 Macro Definition Documentation	1721
8.442.1.1 TFM_SVC_AGENT_PSA_CALL	1721
8.442.1.2 TFM_SVC_AGENT_PSA_CLOSE	1721
8.442.1.3 TFM_SVC_AGENT_PSA_CONNECT	1721
8.442.1.4 TFM_SVC_FLIH_FUNC_RETURN	1721
8.442.1.5 TFM_SVC_GET_BOOT_DATA	1721
8.442.1.6 TFM_SVC_IS_HANDLER_MODE	1721
8.442.1.7 TFM_SVC_IS_PLATFORM	1721
8.442.1.8 TFM_SVC_IS_PSA_API	1722
8.442.1.9 TFM_SVC_IS_SPM	1722
8.442.1.10 TFM_SVC_NUM_HANDLER_MODE_MSK	1722
8.442.1.11 TFM_SVC_NUM_HANDLER_MODE_POS	1722
8.442.1.12 TFM_SVC_NUM_INDEX_MSK	1722
8.442.1.13 TFM_SVC_NUM_PLATFORM_HANDLER	1722
8.442.1.14 TFM_SVC_NUM_PLATFORM_MSK	1722
8.442.1.15 TFM_SVC_NUM_PLATFORM_POS	1722
8.442.1.16 TFM_SVC_NUM_PLATFORM_THREAD	1723
8.442.1.17 TFM_SVC_NUM_PSA_API_MSK	1723
8.442.1.18 TFM_SVC_NUM_PSA_API_POS	1723
8.442.1.19 TFM_SVC_NUM_PSA_API_THREAD	1723
8.442.1.20 TFM_SVC_NUM_SPM_HANDLER	1723
8.442.1.21 TFM_SVC_NUM_SPM_THREAD	1723
8.442.1.22 TFM_SVC_OUTPUT_UNPRIV_STRING	1723
8.442.1.23 TFM_SVC_PREPARE_DEPRIV_FLIH	1723
8.442.1.24 TFM_SVC_PSA_CALL	1724
8.442.1.25 TFM_SVC_PSA_CLEAR	1724

8.442.1.26 TFM_SVC_PSA_CLOSE	1724
8.442.1.27 TFM_SVC_PSA_CONNECT	1724
8.442.1.28 TFM_SVC_PSA_EOI	1724
8.442.1.29 TFM_SVC_PSA_FRAMEWORK_VERSION	1724
8.442.1.30 TFM_SVC_PSA_GET	1724
8.442.1.31 TFM_SVC_PSA_IRQ_DISABLE	1724
8.442.1.32 TFM_SVC_PSA_IRQ_ENABLE	1724
8.442.1.33 TFM_SVC_PSA_LIFECYCLE	1724
8.442.1.34 TFM_SVC_PSA_MAP_INVEC	1725
8.442.1.35 TFM_SVC_PSA_MAP_OUTVEC	1725
8.442.1.36 TFM_SVC_PSA_NOTIFY	1725
8.442.1.37 TFM_SVC_PSA_PANIC	1725
8.442.1.38 TFM_SVC_PSA_READ	1725
8.442.1.39 TFM_SVC_PSA_REPLY	1725
8.442.1.40 TFM_SVC_PSA_RESET_SIGNAL	1725
8.442.1.41 TFM_SVC_PSA_SET_RHANDLE	1725
8.442.1.42 TFM_SVC_PSA_SKIP	1725
8.442.1.43 TFM_SVC_PSA_UNMAP_INVEC	1725
8.442.1.44 TFM_SVC_PSA_UNMAP_OUTVEC	1726
8.442.1.45 TFM_SVC_PSA_VERSION	1726
8.442.1.46 TFM_SVC_PSA_WAIT	1726
8.442.1.47 TFM_SVC_PSA_WRITE	1726
8.442.1.48 TFM_SVC_SPM_INIT	1726
8.442.1.49 TFM_SVC_THREAD_MODE_SPM_RETURN	1726
8.443 secure_fw/spm/include/lists.h File Reference	1726
8.443.1 Macro Definition Documentation	1727
8.443.1.1 BI_LIST_FOR_EACH	1727
8.443.1.2 BI_LIST_INIT_NODE	1727
8.443.1.3 BI_LIST_INSERT_AFTER	1727
8.443.1.4 BI_LIST_INSERT_BEFORE	1727
8.443.1.5 BI_LIST_IS_EMPTY	1728
8.443.1.6 BI_LIST_NEXT_NODE	1728
8.443.1.7 BI_LIST_REMOVE_NODE	1728
8.443.1.8 UNI_LIST_FOREACH	1728
8.443.1.9 UNI_LIST_FOREACH_NODE_PNODE	1728
8.443.1.10 UNI_LIST_FOREACH_NODE_PREV	1729
8.443.1.11 UNI_LIST_INIT_NODE	1729
8.443.1.12 UNI_LIST_INSERT_AFTER	1729
8.443.1.13 UNI_LIST_IS_EMPTY	1729
8.443.1.14 UNI_LIST_MOVE_AFTER	1729
8.443.1.15 UNI_LIST_NEXT_NODE	1730
8.443.1.16 UNI_LIST_REMOVE_NODE	1730

8.443.1.17 UNI_LIST_REMOVE_NODE_BY_PNODE	1730
8.444 secure_fw/spm/include/load/asset_defs.h File Reference	1730
8.444.1 Macro Definition Documentation	1731
8.444.1.1 ASSET_ATTR_MMIO	1731
8.444.2 Enumeration Type Documentation	1731
8.444.2.1 assets_attribute_t	1732
8.445 secure_fw/spm/include/load/interrupt_defs.h File Reference	1732
8.445.1 Enumeration Type Documentation	1733
8.445.1.1 mbox_irq_flags_t	1733
8.446 secure_fw/spm/include/load/ns_client_id_tz.h File Reference	1733
8.446.1 Macro Definition Documentation	1733
8.446.1.1 TZ_NS_CLIENT_ID_BASE	1733
8.446.1.2 TZ_NS_CLIENT_ID_LIMIT	1733
8.447 secure_fw/spm/include/load/partition_defs.h File Reference	1734
8.447.1 Macro Definition Documentation	1735
8.447.1.1 ENTRY_TO_POSITION	1735
8.447.1.2 INVALID_PARTITION_ID	1735
8.447.1.3 IS_IPC_MODEL	1735
8.447.1.4 IS_NS_AGENT	1735
8.447.1.5 IS_NS_AGENT_MAILBOX	1735
8.447.1.6 IS_NS_AGENT_TZ	1735
8.447.1.7 IS_PSA_ROT	1736
8.447.1.8 LOAD_ORDER_BY_PRIORITY	1736
8.447.1.9 PARTITION_INFO_MAGIC	1736
8.447.1.10 PARTITION_INFO_MAGIC_MASK	1736
8.447.1.11 PARTITION_INFO_VERSION_MASK	1736
8.447.1.12 PARTITION_MODEL_IPC	1736
8.447.1.13 PARTITION_MODEL_PSA_ROT	1736
8.447.1.14 PARTITION_NS_AGENT_MB	1736
8.447.1.15 PARTITION_NS_AGENT_TZ	1736
8.447.1.16 PARTITION_PRI_HIGH	1737
8.447.1.17 PARTITION_PRI_HIGHEST	1737
8.447.1.18 PARTITION_PRI_LOW	1737
8.447.1.19 PARTITION_PRI_LOWEST	1737
8.447.1.20 PARTITION_PRI_MASK	1737
8.447.1.21 PARTITION_PRI_NORMAL	1737
8.447.1.22 PARTITION_TYPE_TO_INDEX	1737
8.447.1.23 POSITION_TO_ENTRY	1737
8.447.1.24 PTR_TO_REFERENCE	1737
8.447.1.25 REFERENCE_TO_PTR	1738
8.447.1.26 TFM_SP_IDLE	1738
8.447.1.27 TFM_SP_TZ_AGENT	1738

8.447.1.28 TO_THREAD_PRIORITY	1738
8.448 secure_fw/spm/include/load/service_defs.h File Reference	1738
8.448.1 Macro Definition Documentation	1739
8.448.1.1 SERVICE_ENABLED_MM_IOVEC	1739
8.448.1.2 SERVICE_FLAG_MM_IOVEC	1739
8.448.1.3 SERVICE_FLAG_NS_ACCESSIBLE	1739
8.448.1.4 SERVICE_FLAG_STATELESS	1739
8.448.1.5 SERVICE_FLAG_STATELESS_HINDEX_MASK	1740
8.448.1.6 SERVICE_FLAG_VERSION_POLICY_BIT	1740
8.448.1.7 SERVICE_GET_STATELESS_HINDEX	1740
8.448.1.8 SERVICE_GET_VERSION_POLICY	1740
8.448.1.9 SERVICE_IS_NS_ACCESSIBLE	1740
8.448.1.10 SERVICE_IS_STATELESS	1740
8.448.1.11 SERVICE_VERSION_POLICY_RELAXED	1740
8.448.1.12 SERVICE_VERSION_POLICY_STRICT	1740
8.448.1.13 STRID_TO_STRING_PTR	1740
8.448.1.14 STRING_PTR_TO_STRID	1741
8.449 secure_fw/spm/include/load/spm_load_api.h File Reference	1741
8.449.1 Macro Definition Documentation	1742
8.449.1.1 LOAD_ALLOCED_STACK_ADDR	1742
8.449.1.2 LOAD_INFO_ASSET	1742
8.449.1.3 LOAD_INFO_DEPS	1742
8.449.1.4 LOAD_INFO_EXT_LENGTH	1742
8.449.1.5 LOAD_INFO_IRQ	1742
8.449.1.6 LOAD_INFO_SERVICE	1742
8.449.1.7 LOAD_INF SZ_BYTES	1742
8.449.1.8 NO_MORE_PARTITION	1743
8.449.2 Function Documentation	1743
8.449.2.1 load_a_partition_assuredly()	1743
8.449.2.2 load_irqs_assuredly()	1743
8.449.2.3 load_services_assuredly()	1743
8.450 secure_fw/spm/include/ns_agent_mailbox_defs.h File Reference	1744
8.450.1 Macro Definition Documentation	1744
8.450.1.1 BOOT_NS_CORE	1744
8.450.1.2 DISABLE_MAILBOX	1744
8.450.1.3 ENABLE_MAILBOX	1744
8.450.1.4 NS_CORE_SHUTDOWN	1744
8.451 secure_fw/spm/include/ns_agent_mailbox_interface.h File Reference	1745
8.451.1 Function Documentation	1746
8.451.1.1 ns_agent_boot_ns_core()	1746
8.451.1.2 ns_agent_mailbox_disable()	1746
8.451.1.3 ns_agent_mailbox_enable()	1746

8.451.1.4 ns_agent_ns_core_shutdown()	1747
8.452 secure_fw/spm/include/tfm_arch.h File Reference	1747
8.452.1 Macro Definition Documentation	1749
8.452.1.1 ARCH_CLAIM_CTXCTRL_INSTANCE	1749
8.452.1.2 ARCH_CTXCTRL_ALLOCATE_STACK	1749
8.452.1.3 ARCH_CTXCTRL_ALLOCATED_PTR	1749
8.452.1.4 ARCH_CTXCTRL_EXCRET_PATTERN	1749
8.452.1.5 ARCH_CTXCTRL_INIT	1750
8.452.1.6 ARCH_FLUSH_FP_CONTEXT	1750
8.452.1.7 ARCH_STATE_CTX_SET_R0	1750
8.452.1.8 PENDSV_PRIO_FOR_SCHED	1750
8.452.1.9 SCHEDULER_ATTEMPTED	1750
8.452.1.10 SCHEDULER_LOCKED	1750
8.452.1.11 SCHEDULER_UNLOCKED	1750
8.452.1.12 TFM_FPU_CONTEXT_SIZE	1751
8.452.1.13 XPSR_T32	1751
8.452.2 Function Documentation	1751
8.452.2.1 __restore_irq()	1751
8.452.2.2 __save_disable_irq()	1751
8.452.2.3 __set_CONTROL_nPRIV()	1751
8.452.2.4 arch_acquire_sched_lock()	1752
8.452.2.5 arch_attempt_schedule()	1752
8.452.2.6 arch_clean_stack_and_launch()	1752
8.452.2.7 arch_release_sched_lock()	1753
8.452.2.8 tfm_arch_config_extensions()	1753
8.452.2.9 tfm_arch_free_msp_and_exc_ret()	1753
8.452.2.10 tfm_arch_init_context()	1754
8.452.2.11 tfm_arch_is_priv()	1754
8.452.2.12 tfm_arch_refresh_hardware_context()	1755
8.452.2.13 tfm_arch_set_context_ret_code()	1755
8.452.2.14 tfm_arch_set_secure_exception_priorities()	1755
8.452.2.15 tfm_arch_thread_fn_call()	1756
8.453 secure_fw/spm/include/tfm_arch_v8m.h File Reference	1756
8.453.1 Macro Definition Documentation	1757
8.453.1.1 EXC_RETURN_HANDLER	1757
8.453.1.2 EXC_RETURN_RES1	1757
8.453.1.3 EXC_RETURN_THREAD_MSP	1757
8.453.1.4 EXC_RETURN_THREAD_PSP	1757
8.453.1.5 SCB_ICSR_ADDR	1757
8.453.1.6 SCB_ICSR_PENDSVSET_BIT	1758
8.453.1.7 TFM_NS_EXC_DISABLE	1758
8.453.1.8 TFM_NS_EXC_ENABLE	1758

8.453.2 Function Documentation	1758
8.453.2.1 arch_seal_thread_stack()	1758
8.453.2.2 arch_update_process_sp()	1758
8.453.2.3 is_default_stacking_rules_apply()	1759
8.453.2.4 is_return_secure_stack()	1759
8.453.2.5 is_stack_alloc_fp_space()	1759
8.453.2.6 tfm_arch_check_msp_sealing()	1760
8.453.2.7 tfm_arch_config_branch_protection()	1760
8.453.2.8 tfm_arch_get_psplim()	1760
8.453.2.9 tfm_arch_set_msplim()	1760
8.453.2.10 tfm_arch_set_psplim()	1761
8.453.3 Variable Documentation	1761
8.453.3.1 __STACK_SEAL	1761
8.454 secure_fw/spm/include/tfm_core_trustzone.h File Reference	1761
8.454.1 Macro Definition Documentation	1762
8.454.1.1 TFM_ADDITIONAL_FP_CONTEXT_WORDS	1762
8.454.1.2 TFM_BASIC_FP_CONTEXT_WORDS	1762
8.454.1.3 TFM_STACK_SEAL_VALUE	1762
8.454.1.4 TFM_STACK_SEAL_VALUE_64	1762
8.454.1.5 TFM_STACK_SEALED_SIZE	1762
8.454.1.6 TFM_VENEER_LR_BIT0_MASK	1762
8.455 secure_fw/spm/include/tfm_exc_num.h File Reference	1762
8.455.1 Macro Definition Documentation	1763
8.455.1.1 EXC_NUM_PENDSV	1763
8.455.1.2 EXC_NUM_SVCALL	1763
8.455.1.3 EXC_NUM_THREAD_MODE	1763
8.455.2 Function Documentation	1763
8.455.2.1 __get_active_exc_num()	1764
8.456 secure_fw/spm/include/tfm_hybrid_platform.h File Reference	1764
8.456.1 Macro Definition Documentation	1764
8.456.1.1 TFM_HYBRID_PLAT_SCHED_BALANCED	1764
8.456.1.2 TFM_HYBRID_PLAT_SCHED_NSPE	1764
8.456.1.3 TFM_HYBRID_PLAT_SCHED_OFF	1765
8.456.1.4 TFM_HYBRID_PLAT_SCHED_SPE	1765
8.457 secure_fw/spm/include/tfm_nspm.h File Reference	1765
8.457.1 Macro Definition Documentation	1765
8.457.1.1 TFM_NS_CLIENT_INVALID_ID	1766
8.457.2 Function Documentation	1766
8.457.2.1 tfm_nspm_ctx_init()	1766
8.457.2.2 tfm_nspm_get_current_client_id()	1766
8.457.2.3 tz_ns_agent_register_client_id_range()	1767
8.458 secure_fw/spm/include/utilities.h File Reference	1767

8.458.1 Macro Definition Documentation	1768
8.458.1.1 ERROR_MSG	1768
8.458.1.2 M2S	1768
8.458.1.3 spm_memcpy	1768
8.458.1.4 spm_memset	1769
8.458.1.5 STRINGIFY_EXPAND	1769
8.458.1.6 TO_CONTAINER	1769
8.458.2 Function Documentation	1769
8.458.2.1 get_spm_boundary()	1769
8.458.2.2 tfm_core_panic()	1769
8.459 secure_fw/spm/ns_client_ext/tfm_ns_client_ext.c File Reference	1771
8.459.1 Macro Definition Documentation	1772
8.459.1.1 IS_INVALID_TOKEN	1772
8.459.1.2 MAKE_NS_CLIENT_TOKEN	1772
8.459.1.3 NS_CLIENT_TOKEN_TO_CTX_IDX	1773
8.459.1.4 NS_CLIENT_TOKEN_TO_GID	1773
8.459.1.5 NS_CLIENT_TOKEN_TO_TID	1773
8.459.2 Function Documentation	1773
8.459.2.1 tfm_nsce_acquire_ctx()	1773
8.459.2.2 tfm_nsce_init()	1774
8.459.2.3 tfm_nsce_load_ctx()	1774
8.459.2.4 tfm_nsce_release_ctx()	1775
8.459.2.5 tfm_nsce_save_ctx()	1776
8.460 secure_fw/spm/ns_client_ext/tfm_ns_ctx.c File Reference	1776
8.460.1 Function Documentation	1777
8.460.1.1 acquire_ns_ctx()	1777
8.460.1.2 get_nsid_from_active_ns_ctx()	1777
8.460.1.3 init_ns_ctx()	1777
8.460.1.4 load_ns_ctx()	1778
8.460.1.5 release_ns_ctx()	1778
8.460.1.6 save_ns_ctx()	1779
8.461 secure_fw/spm/ns_client_ext/tfm_ns_ctx.h File Reference	1779
8.461.1 Macro Definition Documentation	1780
8.461.1.1 TFM_NS_CONTEXT_MAX	1780
8.461.1.2 TFM_NS_CONTEXT_MAX_TID	1780
8.461.2 Function Documentation	1780
8.461.2.1 acquire_ns_ctx()	1780
8.461.2.2 get_nsid_from_active_ns_ctx()	1780
8.461.2.3 init_ns_ctx()	1781
8.461.2.4 load_ns_ctx()	1781
8.461.2.5 release_ns_ctx()	1781
8.461.2.6 save_ns_ctx()	1782

8.462 secure_fw/spm/ns_client_ext/tfm_spm_ns_ctx.c File Reference	1782
8.462.1 Macro Definition Documentation	1783
8.462.1.1 DEFAULT_NS_CLIENT_ID	1783
8.462.2 Function Documentation	1783
8.462.2.1 tfm_nspm_ctx_init()	1783
8.462.2.2 tfm_nspm_get_current_client_id()	1783
8.462.2.3 tz_ns_agent_register_client_id_range()	1784

Chapter 1

TF-M Reference Manual

This document gives a reference for the TF-M core and services. This guide does not cover the non-secure application example, the boot-loader and the target platform implementations.

Chapter 2

Deprecated List

Global [PSA_EXPORT_KEY_PAIR_OR_PUBLIC_MAX_SIZE](#)

Please use [PSA_EXPORT_ASYMMETRIC_KEY_MAX_SIZE](#) instead.

Chapter 3

Module Index

3.1 Modules

Here is a list of all modules:

Implementation-specific definitions	21
API version	22
Library initialization	23
Key management	24
Key import and export	28
Message digests	33
Extendable-operation functions (XOF)	42
Message authentication codes	47
Symmetric ciphers	57
Authenticated encryption with associated data (AEAD)	68
Asymmetric cryptography	87
Key derivation and pseudorandom generation	98
Random generation	117
Interruptible sign/verify hash	121
Interruptible Key Agreement	133
Interruptible Key Generation	140
Interruptible public-key export	146
Random and entropy drivers	151
Random generator	152
Built-in keys	155
Functions defined by a client provider	156
Password-authenticated key exchange (PAKE)	157
Error codes	179
Key and algorithm types	187
Key lifetimes	274
Key policies	283
Key attributes	288
Key derivation	293
Helper macros	297
Interruptible operations	298
Set of functions implementing a thin shim	299
Function that implement allocation and deallocation of	304
This group implements the partition initialisation	307
Set of functions implementing the abstractions of the underlying cryptographic	308
Tfm_builtin_key_loader	312
Asn1_module	314

Chapter 4

Data Structure Index

4.1 Data Structures

Here are the data structures with brief descriptions:

_MTB_SRF_DATA_ALIGN	
MTB SRF IPC request structure suitable for TFM needs. This structure overrides default mtb↔	
_srf_ipc_packet_t SRF structure	325
asset_desc_t	
Asset descriptor	327
attest_boot_data	
Contains the received boot status information from bootloader	329
attest_token_encode_ctx	330
bi_list_node_t	331
boot_data_access_policy	
Defines the access policy of secure partitions to data items in shared data area (between boot-loader and runtime firmware)	332
client_id_region_t	333
client_params_t	334
composite_addr_t	335
connection_t	336
context_ctrl_t	338
context_flih_ret_t	340
critical_section_t	342
full_context_t	342
fwu_image_info_data_s	343
ifx_driver_flash_obj_t	
Structure containing the FLASH driver instance parameters	344
ifx_driver_rram_obj_t	
Structure containing the RRAM driver instance parameters	345
ifx_driver_smif_mem_t	
Structure containing the SMIF driver memory configuration	347
ifx_driver_smif_obj_t	
Structure containing the SMIF driver instance data	349
ifx_flash_driver_instance_t	
Flash driver instance	351
ifx_flash_driver_t	
Access structure of the Flash Driver	352
ifx_mpc_cfg_t	354
ifx_mpc_ext_cache_cfg_info_t	355

ifx_mpc_numbered_mmio_config_t	357
ifx_mpc_policy_t	358
ifx_mpc_raw_region_config_t	
Configuration structure for MPC raw region	359
ifx_mpc_region_config_t	361
ifx_mpc_sert_mem_t	
Descriptor of an external memory whose MPC is handled by SE RT Services	363
ifx_mtb_mailbox_reply_t	364
ifx_nv_counters	
Struct representing the NV counter data in flash	365
ifx_nv_init_done	366
ifx_original_iovec_t	367
ifx_partition_info_t	
Structure describing partition info	368
ifx_partition_load_info_t	
Structure used to store platform specific partition properties	371
ifx_ppc_named_mmio_config_t	372
ifx_ppc_regions_config_t	373
ifx_ppcx_config_t	374
ifx_protection_region_config_t	
Structure used to store platform specific protection properties	376
ifx_psa_client_params_t	376
ifx_rram_counter_t	379
ifx_sau_config_t	380
ifx_timer_irq_test_dev_config	381
irq_load_info_t	382
irq_t	384
its_block_meta_t	
Structure to store information about each physical flash memory block	385
its_file_meta_t	
Structure to store file metadata	387
its_flash_fs_config_t	
Structure containing the flash filesystem configuration parameters	388
its_flash_fs_ctx_t	
Structure to store the ITS flash file system context	391
its_flash_fs_file_info_t	
Structure containing file information	392
its_flash_fs_ops_t	
Structure containing the filesystem flash operation parameters	394
its_flash_nand_dev_t	397
its_metadata_block_header_comp_t	
The structure of metadata block header in ITS_BACKWARD_SUPPORTED_VERSION	398
its_metadata_block_header_t	
Structure to store the metadata block header	400
mailbox_init_t	402
mailbox_msg_t	403
mailbox_reply_t	404
mailbox_slot_t	405
mailbox_status_t	406
mbedtls_asn1_bitstring	407
mbedtls_asn1_buf	407
mbedtls_asn1_named_data	408
mbedtls_asn1_sequence	409
mbedtls_lmots_parameters_t	409
mbedtls_lmots_public_t	410
mbedtls_lms_parameters_t	412
mbedtls_lms_public_t	413
mbedtls_md_context_t	414

mbedtls_pk_context	
Public key container	415
mbedtls_platform_context	
The platform context structure	417
mbedtls_psa_stats_s	
Statistics about resource consumption related to the PSA keystore	418
mpu_armv8m_region_cfg_t	420
ns_mailbox_queue_t	421
ns_mailbox_req_t	423
ns_mailbox_slot_t	425
ns_mailbox_stats_res_t	
The structure to hold the statistics result of NSPE mailbox	426
partition_head_t	427
partition_load_info_t	428
partition_t	431
partition_tfm_sp_idle_load_info_t	433
partition_tfm_sp_ns_agent_tz_load_info_t	434
ps_crypto_t	435
ps_obj_header_t	
Metadata attached as a header to object data before storage	437
ps_obj_table_ctx_t	
Object table context structure	439
ps_obj_table_entry_t	440
ps_obj_table_info_t	
Object table information structure	441
ps_obj_table_init_ctx_t	442
ps_obj_table_t	
Object table structure	443
ps_object_info_t	
Object information	445
ps_object_t	
The object to be written to the file system below. Made up of the object header and the object data	446
psa_aead_operation_s	447
psa_cipher_operation_s	449
psa_client_params_t	450
psa_crypto_driver_pake_inputs_s	453
psa_custom_key_parameters_s	454
psa_driver_aead_context_t	455
psa_driver_cipher_context_t	456
psa_driver_hash_context_t	457
psa_driver_key_derivation_context_t	458
psa_driver_mac_context_t	458
psa_driver_pake_context_t	459
psa_driver_sign_hash_interruptible_context_t	460
psa_driver_verify_hash_interruptible_context_t	461
psa_driver_xof_context_t	461
psa_export_public_key_iop_s	
The context for PSA interruptible export public-key	462
psa_fwu_component_info_t	
Information about the firmware store for a firmware component	463
psa_fwu_image_version_t	
Version information about a firmware image	465
psa_fwu_impl_info_t	
The implementation-specific data in the component information structure	467
psa_generate_key_iop_s	
The context for PSA interruptible key generation	468
psa_hash_operation_s	469
psa_invec	470

psa_jpake_computation_stage_s	470
psa_key_agreement_iop_s	
The context for PSA interruptible key agreement	472
psa_key_attributes_s	473
psa_key_derivation_s	474
psa_key_policy_s	475
psa_mac_operation_s	476
psa_msg_t	477
psa_outvec	479
psa_pake_cipher_suite_s	480
psa_pake_operation_s	481
psa_sign_hash_interruptible_operation_s	
The context for PSA interruptible hash signing	483
psa_storage_info_t	484
psa_verify_hash_interruptible_operation_s	
The context for PSA interruptible hash verification	485
psa_xof_operation_s	486
rot_psa_its_storage_info_t	488
rot_psa_ps_storage_info_t	489
service_head_t	490
service_load_info_t	491
service_t	492
shared_data_tlv_entry	493
shared_data_tlv_header	494
tfm_additional_context_t	495
tfm_boot_data	
Store the data for the runtime SW	495
tfm_builtin_key_t	
A structure which describes a builtin key slot	496
tfm_crypto_aead_pack_input	
This type is used to overcome a limitation in the number of maximum IOVEC's that can be used especially in <code>psa_aead_encrypt</code> and <code>psa_aead_decrypt</code> . By using this type we pack the nonce and the actual nonce_length at part of the same structure	498
tfm_crypto_key_id_s	
The type which describes a key identifier to the Crypto service. The key identifiers must clearly provide a dedicated indication of the entity owner which owns the key	498
tfm_crypto_operation_s	
A type describing the context stored in Secure memory by the TF-M Crypto service to support multipart calls on secure side	499
tfm_crypto_pack_iovec	
Structure used to pack non-pointer types in a call to PSA Crypto APIs	501
tfm_fwu_ctx_s	503
tfm_fwu_mcuboot_ctx_s	504
tfm_ns_ctx_t	504
tfm_pool_chunk_t	505
tfm_pool_instance_t	506
tfm_state_context_t	507
thread_t	508
u64_in_u32_regs_t	509
vectors	510

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

interface/include/tfm_attest_defs.h	716
interface/include/tfm_attest_iat_defs.h	716
interface/include/tfm_crypto_defs.h	718
interface/include/tfm_fwu_defs.h	725
interface/include/tfm_fwu_impl_info.h	726
interface/include/tfm_its_defs.h	727
interface/include/tfm_ns_client_ext.h	728
interface/include/tfm_ns_interface.h	733
interface/include/tfm_platform_api.h	738
interface/include/tfm_ps_defs.h	744
interface/include/tfm_psa_call_pack.h	745
interface/include/tfm_veneers.h	748
interface/include/crypto_keys/tfm_builitn_key_ids.h	513
interface/include/hybrid_platform/api_broker_defs.h	515
interface/include/mbedtls/asn1.h	
Generic ASN.1 parsing	518
interface/include/mbedtls/asn1write.h	
ASN.1 buffer writing functionality	520
interface/include/mbedtls/base64.h	
RFC 1521 base64 encoding/decoding	521
interface/include/mbedtls/compat-3-crypto.h	
Compatibility definitions for MbedTLS 3.x code built with MbedTLS 4.x or TF-PSA-Crypto 1.x	523
interface/include/mbedtls/constant_time.h	525
interface/include/mbedtls/lms.h	
This file provides an API for the LMS post-quantum-safe stateful-hash public-key signature scheme as defined in RFC8554 and NIST.SP.200-208. This implementation currently only supports a single parameter set MBEDTLS_LMS_SHA256_M32_H10 in order to reduce complexity. This is one of the signature schemes recommended by the IETF draft SUIT standard for IOT firmware upgrades (RFC9019)	526
interface/include/mbedtls/md.h	
This file contains the generic functions for message-digest (hashing) and HMAC	532
interface/include/mbedtls/memory_buffer_alloc.h	
Buffer-based memory allocator	540
interface/include/mbedtls/nist_kw.h	
This file provides an API for key wrapping (KW) and key wrapping with padding (KWP) as defined in NIST SP 800-38F. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-38F.pdf	543

interface/include/mbedtls/pem.h	
Privacy Enhanced Mail (PEM) decoding	546
interface/include/mbedtls/pk.h	
Public Key abstraction layer	547
interface/include/mbedtls/platform.h	
This file contains the definitions and functions of the Mbed TLS platform abstraction layer	564
interface/include/mbedtls/platform_time.h	
Mbed TLS Platform time abstraction	570
interface/include/mbedtls/platform_util.h	
Common and shared functions used by multiple modules in the Mbed TLS library	571
interface/include/mbedtls/private_access.h	
Optionally activate declarations of private identifiers in public headers	575
interface/include/mbedtls/psa_util.h	
Utility functions for the use of the PSA Crypto library	576
interface/include/mbedtls/threading.h	
Threading abstraction layer	576
interface/include/mbedtls/private/pk_private.h	
Private Public Key abstraction layer	574
interface/include/multi_core/tfm_mailbox.h	577
interface/include/multi_core/tfm_multi_core_api.h	581
interface/include/multi_core/tfm_ns_mailbox.h	582
interface/include/multi_core/tfm_ns_mailbox_test.h	591
interface/include/os_wrapper/common.h	593
interface/include/os_wrapper/kernel.h	594
interface/include/os_wrapper/mutex.h	595
interface/include/psa/api_broker.h	598
interface/include/psa/client.h	600
interface/include/psa/crypto.h	
Platform Security Architecture cryptography module	608
interface/include/psa/crypto_compat.h	
PSA cryptography module: Backward compatibility aliases	614
interface/include/psa/crypto_driver_common.h	
Definitions for all PSA crypto drivers	615
interface/include/psa/crypto_driver_contexts_composites.h	
Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for 'composite' operations, i.e. those operations which need to make use of other operations from the primitives (crypto_driver_contexts_primitives.h)	616
interface/include/psa/crypto_driver_contexts_key_derivation.h	
Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for key derivation operations	617
interface/include/psa/crypto_driver_contexts_primitives.h	
Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for 'primitive' operations, i.e. those operations which do not rely on other contexts	618
interface/include/psa/crypto_driver_random.h	
Definitions for PSA random and entropy drivers	620
interface/include/psa/crypto_extra.h	
PSA cryptography module: Mbed TLS vendor extensions	621
interface/include/psa/crypto_platform.h	
PSA cryptography module: Mbed TLS platform definitions	629
interface/include/psa/crypto_sizes.h	
PSA cryptography module: Mbed TLS buffer size macros	630
interface/include/psa/crypto_struct.h	
PSA cryptography module: Mbed TLS structured type implementations	652
interface/include/psa/crypto_types.h	
PSA cryptography module: type aliases	655
interface/include/psa/crypto_values.h	
PSA cryptography module: macros to build and analyze integer values	657

interface/include/psa/crypto_values_lms.h	663
interface/include/psa/error.h	
Standard error codes for the SPM and RoT Services	664
interface/include/psa/internal_trusted_storage.h	671
interface/include/psa/lifecycle.h	676
interface/include/psa/protected_storage.h	678
interface/include/psa/service.h	685
interface/include/psa/storage_common.h	697
interface/include/psa/update.h	699
interface/include/tf-psa-crypto/build_info.h	
Build-time configuration info	709
interface/include/tf-psa-crypto/version.h	
Run-time version information	715
interface/include/tf-psa-crypto/private/crypto_adjust_config_auto_enabled.h	
Adjust PSA configuration: enable always-on features	711
interface/include/tf-psa-crypto/private/crypto_adjust_config_dependencies.h	
Adjust PSA configuration by resolving some dependencies	712
interface/include/tf-psa-crypto/private/crypto_adjust_config_derived.h	
Adjust PSA configuration by defining internal symbols	712
interface/include/tf-psa-crypto/private/crypto_adjust_config_key_pair_types.h	
Adjust PSA configuration for key pair types	714
interface/include/tf-psa-crypto/private/crypto_adjust_config_support.h	
Adjust TF-PSA-Crypto configuration: support modules	714
interface/include/tf-psa-crypto/private/crypto_adjust_config_synonyms.h	
Adjust PSA configuration: enable quasi-synonyms	715
interface/src/tfm_attest_api.c	771
interface/src/tfm_crypto_api.c	772
interface/src/tfm_fwu_api.c	777
interface/src/tfm_its_api.c	783
interface/src/tfm_platform_api.c	787
interface/src/tfm_ps_api.c	791
interface/src/tfm_psa_call.c	798
interface/src/tfm_tz_psa_ns_api.c	800
interface/src/hybrid_platform/api_broker.c	753
interface/src/multi_core/tfm_multi_core_ns_api.c	757
interface/src/multi_core/tfm_multi_core_psa_ns_api.c	758
interface/src/multi_core/tfm_ns_mailbox.c	760
interface/src/multi_core/tfm_ns_mailbox_common.c	762
interface/src/multi_core/tfm_ns_mailbox_thread.c	763
interface/src/os_wrapper/tfm_ns_interface_bare_metal.c	766
interface/src/os_wrapper/tfm_ns_interface_rtos.c	768
platform/ext/target/infineon/common/device/include/cmsis.h	803
platform/ext/target/infineon/common/device/include/device_cfg.h	804
platform/ext/target/infineon/common/device/include/device_definition.h	804
platform/ext/target/infineon/common/device/include/ifx_platform_startup.h	805
platform/ext/target/infineon/common/device/include/ifx_spm_init.h	807
platform/ext/target/infineon/common/device/include/ifx_startup.h	807
platform/ext/target/infineon/common/device/src/device_definition.c	
This file defines exports the structures based on the peripheral definitions from device_cfg.h .	
This retarget file is meant to be used as a helper for baremetal applications and/or as an example of how to configure the generic driver structures	809
platform/ext/target/infineon/common/device/src/ifx_spm_init.c	810
platform/ext/target/infineon/common/device/src/ifx_startup.c	811
platform/ext/target/infineon/common/drivers/assets/ifx_assets_rram.c	813
platform/ext/target/infineon/common/drivers/assets/ifx_assets_rram.h	815
platform/ext/target/infineon/common/drivers/flash/ifx_driver_private.c	821
platform/ext/target/infineon/common/drivers/flash/ifx_driver_private.h	824
platform/ext/target/infineon/common/drivers/flash/ifx_flash_driver_api.h	827

platform/ext/target/infineon/common/drivers/flash/flash/ifx_driver_flash.c	817
platform/ext/target/infineon/common/drivers/flash/flash/ifx_driver_flash.h	819
platform/ext/target/infineon/common/drivers/flash/rram/ifx_driver_rram.c	829
platform/ext/target/infineon/common/drivers/flash/rram/ifx_driver_rram.h	832
platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif.c	834
platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif.h	840
platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif_mmio.c	842
platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif_private.h	843
platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif_xip.c	849
platform/ext/target/infineon/common/drivers/protection/mpu_armv8m_drv.h	850
platform/ext/target/infineon/common/drivers/stdio/uart_pdl_stdout.c	853
platform/ext/target/infineon/common/drivers/stdio/uart_pdl_stdout.h	856
platform/ext/target/infineon/common/interface/include/ifx_platform_api.h	860
platform/ext/target/infineon/common/interface/include/mtb_srf_ipc_custom_packet.h	863
platform/ext/target/infineon/common/interface/include/ifx_mtb_mailbox/ifx_mtb_mailbox.h	857
platform/ext/target/infineon/common/interface/src/ifx_mtb_srf.c	871
platform/ext/target/infineon/common/interface/src/ifx_mtb_srf_relay.c	872
platform/ext/target/infineon/common/interface/src/ifx_platform_api.c	872
platform/ext/target/infineon/common/interface/src/ifx_platform_private.h	874
platform/ext/target/infineon/common/interface/src/ifx_mtb_mailbox/ifx_mtb_mailbox.c	864
platform/ext/target/infineon/common/interface/src/ifx_mtb_mailbox/ifx_mtb_mailbox_psa_ns_api.c	865
platform/ext/target/infineon/common/libs/ifx_se_rt_services_utils/spe/ifx_se_tfm_utils.h	
This file is a wrapper for ifx-se-rt-services-utils library for TF-M. It adds additional stuff needed to use this library from within secure partitions	876
platform/ext/target/infineon/common/libs/tf-psa-crypto/ifx_crypto_platform.h	877
platform/ext/target/infineon/common/libs/tf-psa-crypto/ifx_mbedtls_built_in_keys.c	
Mbedtls IFX builtin keys implementation	880
platform/ext/target/infineon/common/libs/tf-psa-crypto/platform_alt.h	
This file contains the definitions and functions of the IFX Mbed TLS platform abstraction layer	880
platform/ext/target/infineon/common/nspe/mailbox/platform_ns_mailbox.c	882
platform/ext/target/infineon/common/nspe/mailbox/tfm_ns_mailbox_rtos_api.c	887
platform/ext/target/infineon/common/nspe/os_wrapper/os_wrapper_cyabs_rtos.c	891
platform/ext/target/infineon/common/nspe/os_wrapper/semaphore.h	891
platform/ext/target/infineon/common/nspe/os_wrapper/thread.h	1681
platform/ext/target/infineon/common/shared/ifx_stdio_core.h	
STDIO core prefix configuration for all Infineon platforms	893
platform/ext/target/infineon/common/shared/platform_nv_counters_ids.h	905
platform/ext/target/infineon/common/shared/tfm_peripherals_def_common.h	906
platform/ext/target/infineon/common/shared/mailbox/ifx_platform_mailbox.h	895
platform/ext/target/infineon/common/shared/mailbox/platform_multicore.c	896
platform/ext/target/infineon/common/shared/mailbox/platform_multicore.h	899
platform/ext/target/infineon/common/shared/partition/ifx_common_flash_layout.h	904
platform/ext/target/infineon/common/spe/platform_otp_ids.h	929
platform/ext/target/infineon/common/spe/faults/faults.c	907
platform/ext/target/infineon/common/spe/faults/faults.h	912
platform/ext/target/infineon/common/spe/faults/faults_cat1b.c	917
platform/ext/target/infineon/common/spe/faults/faults_cat1d.c	917
platform/ext/target/infineon/common/spe/faults/faults_cat1e.c	918
platform/ext/target/infineon/common/spe/faults/faults_dump.h	918
platform/ext/target/infineon/common/spe/fih/tfm_fih_prng.c	922
platform/ext/target/infineon/common/spe/fih/tfm_fih_trng.h	924
platform/ext/target/infineon/common/spe/fih/tfm_fih_trng_cryptolite.c	925
platform/ext/target/infineon/common/spe/fih/tfm_fih_trng_mxcrypto.c	925
platform/ext/target/infineon/common/spe/fih/tfm_fih_trng_mxtrng.c	926
platform/ext/target/infineon/common/spe/otp/otp_flash.c	927
platform/ext/target/infineon/common/spe/protection/partition_assets.h	930
platform/ext/target/infineon/common/spe/protection/partition_info.h	931
platform/ext/target/infineon/common/spe/protection/protection_data_common.c	932

platform/ext/target/infineon/common/spe/protection/protection_data_common.h	933
platform/ext/target/infineon/common/spe/protection/protection_mpc_api.h	
This file contains MPC driver API declaration	935
platform/ext/target/infineon/common/spe/protection/protection_mpc_hw_mpc.c	937
platform/ext/target/infineon/common/spe/protection/protection_mpc_hw_mpc.h	
This file contains types/API used by HW MPC driver, see cy_mpc.h in PDL	947
platform/ext/target/infineon/common/spe/protection/protection_mpc_sert.c	967
platform/ext/target/infineon/common/spe/protection/protection_mpc_sert.h	
This file contains the API used by HW MPC driver, to protect memory via SE RT Basic/Full	972
platform/ext/target/infineon/common/spe/protection/protection_mpc_sw_policy.c	977
platform/ext/target/infineon/common/spe/protection/protection_mpc_sw_policy.h	
This file contains types/API used by MPC SW Policy driver	984
platform/ext/target/infineon/common/spe/protection/protection_mpu.c	985
platform/ext/target/infineon/common/spe/protection/protection_mpu.h	990
platform/ext/target/infineon/common/spe/protection/protection_msc.c	992
platform/ext/target/infineon/common/spe/protection/protection_msc.h	996
platform/ext/target/infineon/common/spe/protection/protection_pc.c	1014
platform/ext/target/infineon/common/spe/protection/protection_pc.h	1021
platform/ext/target/infineon/common/spe/protection/protection_pc_mask.h	1040
platform/ext/target/infineon/common/spe/protection/protection_ppc_api.h	
This file contains PPC driver API declaration	1041
platform/ext/target/infineon/common/spe/protection/protection_ppc_v1.c	1043
platform/ext/target/infineon/common/spe/protection/protection_ppc_v1.h	
This file contains types/API used by PPC driver	1047
platform/ext/target/infineon/common/spe/protection/protection_ppc_v2.c	1049
platform/ext/target/infineon/common/spe/protection/protection_ppc_v2.h	
This file contains types/API used by PPC driver	1052
platform/ext/target/infineon/common/spe/protection/protection_sau.c	1053
platform/ext/target/infineon/common/spe/protection/protection_sau.h	1058
platform/ext/target/infineon/common/spe/protection/protection_shared_data.c	1076
platform/ext/target/infineon/common/spe/protection/protection_shared_data.h	1078
platform/ext/target/infineon/common/spe/protection/protection_types.h	1080
platform/ext/target/infineon/common/spe/protection/protection_tz.c	1083
platform/ext/target/infineon/common/spe/protection/protection_tz.h	1089
platform/ext/target/infineon/common/spe/protection/protection_utils.c	1091
platform/ext/target/infineon/common/spe/protection/protection_utils.h	1092
platform/ext/target/infineon/common/spe/protection/tfm_hal_isolation.c	1092
platform/ext/target/infineon/common/spe/protection/tfm_platform_arch_hooks.c	1105
platform/ext/target/infineon/common/spe/protection/tfm_platform_arch_hooks.h	1106
platform/ext/target/infineon/common/spe/provisioning/ifx_dap.c	1111
platform/ext/target/infineon/common/spe/provisioning/provisioning.c	1111
platform/ext/target/infineon/common/spe/provisioning/provisioning.h	1112
platform/ext/target/infineon/common/spe/services/attestation/ifx_attest_hal.c	1115
platform/ext/target/infineon/common/spe/services/attestation/ifx_attest_hal_se_rt.c	1118
platform/ext/target/infineon/common/spe/services/attestation/ifx_plat_device_id.c	1119
platform/ext/target/infineon/common/spe/services/crypto/crypto_keys_flash.c	1120
platform/ext/target/infineon/common/spe/services/crypto/crypto_keys_rram.c	1122
platform/ext/target/infineon/common/spe/services/crypto/crypto_keys_se_rt.c	1123
platform/ext/target/infineon/common/spe/services/crypto/crypto_nv_seed.c	1124
platform/ext/target/infineon/common/spe/services/crypto/crypto_nv_seed_cryptolite.c	1125
platform/ext/target/infineon/common/spe/services/crypto/crypto_nv_seed_mxcrypto.c	1126
platform/ext/target/infineon/common/spe/services/crypto/crypto_nv_seed_mxrng.c	1127
platform/ext/target/infineon/common/spe/services/crypto/crypto_rnd_se_rt.c	1128
platform/ext/target/infineon/common/spe/services/crypto/mbedtls_accel_configs/crypto_hw_cryptolite_config.h	1129
platform/ext/target/infineon/common/spe/services/crypto/mbedtls_accel_configs/crypto_hw_mxcrypto_config.h	1129
platform/ext/target/infineon/common/spe/services/its/its_hal.c	1130

platform/ext/target/infineon/common/spe/services/its/drivers/its_flash_driver.c	1129
platform/ext/target/infineon/common/spe/services/its/drivers/its_rram_driver.c	1129
platform/ext/target/infineon/common/spe/services/mailbox/platform_hal_multi_core_cm55.c	1131
platform/ext/target/infineon/common/spe/services/mailbox/platform_spe_mailbox.c	1132
platform/ext/target/infineon/common/spe/services/mailbox/tfm_hal_multi_core.c	1134
platform/ext/target/infineon/common/spe/services/mailbox/tfm_interrupts.c	1135
platform/ext/target/infineon/common/spe/services/platform/nv_counters_common.c	1141
platform/ext/target/infineon/common/spe/services/platform/nv_counters_flash.c	1141
platform/ext/target/infineon/common/spe/services/platform/nv_counters_rram.c	1146
platform/ext/target/infineon/common/spe/services/platform/nv_counters_se_rt.c	1150
platform/ext/target/infineon/common/spe/services/platform/tfm_platform_system.c	1152
platform/ext/target/infineon/common/spe/services/platform/drivers/nv_counters_flash_driver.c	1136
platform/ext/target/infineon/common/spe/services/platform/drivers/nv_counters_flash_driver.h	1137
platform/ext/target/infineon/common/spe/services/platform/drivers/nv_counters_rram_driver.c	1138
platform/ext/target/infineon/common/spe/services/platform/drivers/nv_counters_rram_driver.h	1139
platform/ext/target/infineon/common/spe/services/ps/ps_hal.c	1155
platform/ext/target/infineon/common/spe/services/ps/ps_keys_se_rt.c	1156
platform/ext/target/infineon/common/spe/services/ps/drivers/ps_flash_driver.c	1153
platform/ext/target/infineon/common/spe/services/ps/drivers/ps_rram_driver.c	1154
platform/ext/target/infineon/common/spe/services/ps/drivers/ps_smif_driver.c	1154
platform/ext/target/infineon/common/spe/svc/platform_svc_api.c	1157
platform/ext/target/infineon/common/spe/svc/platform_svc_api.h	1159
platform/ext/target/infineon/common/spe/svc/platform_svc_handler.c	1162
platform/ext/target/infineon/common/spe/svc/platform_svc_private.h	1163
platform/ext/target/infineon/common/spe/v80m/target_cfg.c	1164
platform/ext/target/infineon/common/spe/v80m/target_cfg.h	1170
platform/ext/target/infineon/common/spe/v80m/tfm_hal_platform.c	1191
platform/ext/target/infineon/common/utils/ifx_boot_shared_data.c	1196
platform/ext/target/infineon/common/utils/ifx_boot_shared_data.h	1197
platform/ext/target/infineon/common/utils/ifx_fih.h	1197
platform/ext/target/infineon/common/utils/ifx_interrupt_defs.h	1200
platform/ext/target/infineon/common/utils/ifx_regions.c	1201
platform/ext/target/infineon/common/utils/ifx_regions.h	1202
platform/ext/target/infineon/common/utils/ifx_utils.h	1206
platform/ext/target/infineon/common/utils/mxs22.h	1207
platform/ext/target/infineon/common/utils/se/ifx_se_crc32.c	
Fast high-distance 32-bit CRC for data integrity protection	1208
platform/ext/target/infineon/common/utils/se/ifx_se_crc32.h	
Fast high-distance 32-bit CRC for data integrity protection	1213
platform/ext/target/infineon/psc3m6/config_tfm_target.h	1238
platform/ext/target/infineon/psc3m6/board/shared/ifx_peripherals_def.h	1227
platform/ext/target/infineon/psc3m6/board/shared/device/include/ifx_core_interrupts.h	1219
platform/ext/target/infineon/psc3m6/board/shared/device/include/project_memory_layout.h	1219
platform/ext/target/infineon/psc3m6/board/spe/include/ifx_s_peripherals_def.h	1229
platform/ext/target/infineon/psc3m6/board/spe/include/protection_regions_cfg.h	1230
platform/ext/target/infineon/psc3m6/board/spe/source/protection_regions_cfg.c	1231
platform/ext/target/infineon/psc3m6/config/ifx_platform_spe_config.h	
This file contains platform specific configuration used to build secure image	1232
platform/ext/target/infineon/psc3m6/config/ifx_platform_spe_types.h	
Platform-specific types and data declaration used to build the secure image	1237
platform/ext/target/infineon/psc3m6/shared/tfm_peripherals_def.h	1243
platform/ext/target/infineon/psc3m6/shared/partition/flash_layout.h	1239
platform/ext/target/infineon/psc3m6/shared/partition/region_defs.h	1240
platform/ext/target/infineon/psc3m6/spe/protection/platform_partition_assets.h	1244
platform/ext/target/infineon/psc3m6/spe/protection/protection_data.c	1245
platform/ext/target/infineon/psc3m6/spe/services/crypto/crypto_keys_flash.h	1247
platform/ext/target/infineon/psc3m6/spe/services/crypto/ifx_pdl_cryptolite_config.h	1253
platform/ext/target/infineon/psc3m6/spe/services/crypto/ifx_pdl_cryptosuite_config.h	1259

platform/ext/target/infineon/psc3m6/spe/services/crypto/ifx_pdl_psa_cryptolite_config.h	1259
platform/ext/target/infineon/psc3m6/spe/services/crypto/mbedtls_target_config.h	1261
platform/ext/target/infineon/psc3m6/spe/services/crypto/platform_builtin_key_loader_ids.h	1261
platform/ext/target/infineon/psc3m6/spe/services/crypto/tfm_builtin_key_ids.h	514
secure_fw/include/async.h	1262
secure_fw/include/build_config_check.h	1263
secure_fw/include/compiler_ext_defs.h	1264
secure_fw/include/config_tfm.h	1264
secure_fw/include/crt_impl_private.h	1265
secure_fw/include/security_defs.h	1266
secure_fw/partitions/crypto/config_crypto_check.h	1266
secure_fw/partitions/crypto/config_engine_buf.h	1267
secure_fw/partitions/crypto/crypto_aead.c	1267
secure_fw/partitions/crypto/crypto_alloc.c	1268
secure_fw/partitions/crypto/crypto_asymmetric.c	1269
secure_fw/partitions/crypto/crypto_check_config.h	1270
secure_fw/partitions/crypto/crypto_cipher.c	1271
secure_fw/partitions/crypto/crypto_hash.c	1272
secure_fw/partitions/crypto/crypto_init.c	1273
secure_fw/partitions/crypto/crypto_key_derivation.c	1274
secure_fw/partitions/crypto/crypto_key_management.c	1275
secure_fw/partitions/crypto/crypto_library.c	1276
secure_fw/partitions/crypto/crypto_library.h	
This file contains some abstractions required to interface the TF-M Crypto service to an underlying cryptographic library that implements the PSA Crypto API. The TF-M Crypto service uses this library to provide a PSA Crypto core layer implementation and a software or hardware based implementation of crypto algorithms	1277
secure_fw/partitions/crypto/crypto_mac.c	1279
secure_fw/partitions/crypto/crypto_rng.c	1279
secure_fw/partitions/crypto/crypto_spe.h	
When TF-PSA-Crypto is built with the MBEDTLS_PSA_CRYPTOP_SPM option enabled, this header is included by all .c files in TF-PSA-Crypto that use PSA Crypto function names. This avoids duplication of symbols between TF-M and TF-PSA-Crypto	1280
secure_fw/partitions/crypto/tfm_crypto_api.h	1292
secure_fw/partitions/crypto/tfm_crypto_key.h	1295
secure_fw/partitions/crypto/tfm_mbedcrypto_alt.c	1296
secure_fw/partitions/crypto/tfm_mbedcrypto_include.h	1296
secure_fw/partitions/crypto/psa_driver_api/tfm_builtin_key_loader.c	1289
secure_fw/partitions/crypto/psa_driver_api/tfm_builtin_key_loader.h	1291
secure_fw/partitions/firmware_update/tfm_fwu_req_mgr.c	1309
secure_fw/partitions/firmware_update/bootloader/tfm_bootloader_fwu_abstraction.h	1304
secure_fw/partitions/firmware_update/bootloader/mcuboot/tfm_mcuboot_fwu.c	1297
secure_fw/partitions/idle_partition/idle_partition.c	1311
secure_fw/partitions/idle_partition/load_info_idle_sp.c	1312
secure_fw/partitions/initial_attestation/attest.h	1313
secure_fw/partitions/initial_attestation/attest_asymmetric_key.c	1318
secure_fw/partitions/initial_attestation/attest_boot_data.c	1319
secure_fw/partitions/initial_attestation/attest_boot_data.h	1322
secure_fw/partitions/initial_attestation/attest_core.c	1324
secure_fw/partitions/initial_attestation/attest_key.h	1326
secure_fw/partitions/initial_attestation/attest_symmetric_key.c	1327
secure_fw/partitions/initial_attestation/attest_token.h	
Attestation Token Creation Interface	1329
secure_fw/partitions/initial_attestation/attest_token_encode.c	
Attestation token creation implementation	1334
secure_fw/partitions/initial_attestation/tfm_attest.c	1338
secure_fw/partitions/initial_attestation/tfm_attest_req_mgr.c	1340
secure_fw/partitions/internal_trusted_storage/its_crypto_interface.c	1397

secure_fw/partitions/internal_trusted_storage/its_crypto_interface.h	1398
secure_fw/partitions/internal_trusted_storage/its_utils.c	1400
secure_fw/partitions/internal_trusted_storage/its_utils.h	1401
secure_fw/partitions/internal_trusted_storage/tfm_internal_trusted_storage.c	1406
secure_fw/partitions/internal_trusted_storage/tfm_internal_trusted_storage.h	1412
secure_fw/partitions/internal_trusted_storage/tfm_its_req_mgr.c	1418
secure_fw/partitions/internal_trusted_storage/tfm_its_req_mgr.h	1420
secure_fw/partitions/internal_trusted_storage/flash/its_flash.c	1342
secure_fw/partitions/internal_trusted_storage/flash/its_flash.h	1342
secure_fw/partitions/internal_trusted_storage/flash/its_flash_nand.c	1344
secure_fw/partitions/internal_trusted_storage/flash/its_flash_nand.h	
Implementations of the flash interface functions for a NAND flash device. See its_flash_fs_ops_t	
for full documentation of functions	1345
secure_fw/partitions/internal_trusted_storage/flash/its_flash_nor.c	1346
secure_fw/partitions/internal_trusted_storage/flash/its_flash_nor.h	
Implementations of the flash interface functions for a NOR flash device. See its_flash_fs_ops_t	
for full documentation of functions	1347
secure_fw/partitions/internal_trusted_storage/flash/its_flash_ram.c	1347
secure_fw/partitions/internal_trusted_storage/flash/its_flash_ram.h	
Implementations of the flash interface functions for an emulated flash device using RAM. See	
its_flash_fs_ops_t for full documentation of functions	1348
secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs.c	1349
secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs.h	
Internal Trusted Storage service filesystem abstraction APIs. The purpose of this abstraction is	
to have the ability to plug-in other filesystems or filesystem proxies (supplicant)	1355
secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_dblock.c	1362
secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_dblock.h	1366
secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_mblock.c	1370
secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_mblock.h	1383
secure_fw/partitions/lib/runtime/assert.c	1421
secure_fw/partitions/lib/runtime/crt_exit.c	1422
secure_fw/partitions/lib/runtime/crt_memcmp.c	1424
secure_fw/partitions/lib/runtime/crt_memmove.c	1425
secure_fw/partitions/lib/runtime/crt_start.c	1426
secure_fw/partitions/lib/runtime/crt_strlen.c	1427
secure_fw/partitions/lib/runtime/crt_strnlen.c	1428
secure_fw/partitions/lib/runtime/crt_vprintf.c	1429
secure_fw/partitions/lib/runtime/psa_api_ipc.c	1432
secure_fw/partitions/lib/runtime/service_api.c	1438
secure_fw/partitions/lib/runtime/sfn_common_thread.c	1439
secure_fw/partitions/lib/runtime/sprt_partition_metadata_indicator.c	1440
secure_fw/partitions/lib/runtime/sprt_partition_metadata_indicator.h	1440
secure_fw/partitions/lib/runtime/include/service_api.h	1429
secure_fw/partitions/lib/runtime/include/tfm_string.h	1431
secure_fw/partitions/ns_agent_mailbox/ns_agent_mailbox.c	1442
secure_fw/partitions/ns_agent_mailbox/ns_agent_mailbox_rpc.c	1443
secure_fw/partitions/ns_agent_mailbox/tfm_multi_core_client_id.c	1444
secure_fw/partitions/ns_agent_mailbox/tfm_multi_core_mbox.c	1445
secure_fw/partitions/ns_agent_mailbox/tfm_spe_mailbox.c	1446
secure_fw/partitions/ns_agent_mailbox/tfm_spe_mailbox.h	1451
secure_fw/partitions/ns_agent_tz/load_info_ns_agent_tz.c	1453
secure_fw/partitions/ns_agent_tz/ns_agent_tz.c	1455
secure_fw/partitions/ns_agent_tz/ns_agent_tz_v80m.c	1455
secure_fw/partitions/ns_agent_tz/psa_api_veneers.c	1456
secure_fw/partitions/ns_agent_tz/psa_api_veneers_common.c	1458
secure_fw/partitions/ns_agent_tz/psa_api_veneers_common.h	1460
secure_fw/partitions/ns_agent_tz/psa_api_veneers_v80m.c	1461
secure_fw/partitions/platform/platform_sp.c	1465

secure_fw/partitions/platform/platform_sp.h	1467
secure_fw/partitions/protected_storage/config_ps_check.h	1469
secure_fw/partitions/protected_storage/ps_encrypted_object.c	1486
secure_fw/partitions/protected_storage/ps_encrypted_object.h	1491
secure_fw/partitions/protected_storage/ps_filesystem_interface.c	1493
secure_fw/partitions/protected_storage/ps_object_defs.h	1498
secure_fw/partitions/protected_storage/ps_object_system.c	1500
secure_fw/partitions/protected_storage/ps_object_system.h	1508
secure_fw/partitions/protected_storage/ps_object_table.c	1515
secure_fw/partitions/protected_storage/ps_object_table.h	1525
secure_fw/partitions/protected_storage/ps_utils.c	1531
secure_fw/partitions/protected_storage/ps_utils.h	1533
secure_fw/partitions/protected_storage/tfm_protected_storage.c	1535
secure_fw/partitions/protected_storage/tfm_protected_storage.h	1540
secure_fw/partitions/protected_storage/tfm_ps_req_mgr.c	1546
secure_fw/partitions/protected_storage/tfm_ps_req_mgr.h	1549
secure_fw/partitions/protected_storage/crypto/ps_crypto_interface.c	1470
secure_fw/partitions/protected_storage/crypto/ps_crypto_interface.h	1476
secure_fw/partitions/protected_storage/nv_counters/ps_nv_counters.c	1482
secure_fw/partitions/protected_storage/nv_counters/ps_nv_counters.h	1484
secure_fw/shared/crt_memcpy.c	1551
secure_fw/shared/crt_memset.c	1553
secure_fw/shared/crt_strcmp.c	1554
secure_fw/shared/crt_strncmp.c	1554
secure_fw/spm/core/backend_ipc.c	1567
secure_fw/spm/core/backend_sfn.c	1572
secure_fw/spm/core/internal_status_code.h	1575
secure_fw/spm/core/interrupt.c	1577
secure_fw/spm/core/interrupt.h	1580
secure_fw/spm/core/mailbox_agent_api.c	1584
secure_fw/spm/core/main.c	1585
secure_fw/spm/core/memory_symbols.h	1587
secure_fw/spm/core/ns_agent_mailbox_interface.c	1589
secure_fw/spm/core/psa_api.c	1591
secure_fw/spm/core/psa_call_api.c	1595
secure_fw/spm/core/psa_connection_api.c	1597
secure_fw/spm/core/psa_interface_sfn.c	1599
secure_fw/spm/core/psa_interface_svc.c	1605
secure_fw/spm/core/psa_interface_thread_fn_call.c	1608
secure_fw/spm/core/psa_irq_api.c	1613
secure_fw/spm/core/psa_mmiovec_api.c	1614
secure_fw/spm/core/psa_read_write_skip_api.c	1615
secure_fw/spm/core/psa_version_api.c	1618
secure_fw/spm/core/rom_loader.c	1620
secure_fw/spm/core/spm.h	1622
secure_fw/spm/core/spm_connection_pool.c	1636
secure_fw/spm/core/spm_ipc.c	1639
secure_fw/spm/core/spm_local_connection.c	1647
secure_fw/spm/core/stack_watermark.c	1649
secure_fw/spm/core/stack_watermark.h	1652
secure_fw/spm/core/tfm_boot_data.c	1653
secure_fw/spm/core/tfm_boot_data.h	1654
secure_fw/spm/core/tfm_multi_core.h	1656
secure_fw/spm/core/tfm_pools.c	1659
secure_fw/spm/core/tfm_pools.h	1663
secure_fw/spm/core/tfm_rpc.c	1668
secure_fw/spm/core/tfm_rpc.h	1669
secure_fw/spm/core/tfm_svcalls.c	1669

secure_fw/spm/core/tfm_svcalls.h	1671
secure_fw/spm/core/thread.c	1673
secure_fw/spm/core/thread.h	1676
secure_fw/spm/core/utilities.c	1685
secure_fw/spm/core/arch/tfm_arch.c	1555
secure_fw/spm/core/arch/tfm_arch_v6m_v7m.c	1557
secure_fw/spm/core/arch/tfm_arch_v6m_v7m.h	1559
secure_fw/spm/core/arch/tfm_arch_v8m_base.c	1564
secure_fw/spm/core/arch/tfm_arch_v8m_main.c	1565
secure_fw/spm/include/aapcs_local.h	1687
secure_fw/spm/include/bitops.h	1689
secure_fw/spm/include/config_spm.h	1698
secure_fw/spm/include/critical_section.h	1699
secure_fw/spm/include/current.h	1700
secure_fw/spm/include/lists.h	1726
secure_fw/spm/include/ns_agent_mailbox_defs.h	1744
secure_fw/spm/include/ns_agent_mailbox_interface.h	1745
secure_fw/spm/include/tfm_arch.h	1747
secure_fw/spm/include/tfm_arch_v8m.h	1756
secure_fw/spm/include/tfm_core_trustzone.h	1761
secure_fw/spm/include/tfm_exc_num.h	1762
secure_fw/spm/include/tfm_hybrid_platform.h	1764
secure_fw/spm/include/tfm_nspm.h	1765
secure_fw/spm/include/utilities.h	1767
secure_fw/spm/include/boot/tfm_boot_status.h	1690
secure_fw/spm/include/ffm/backend.h	1701
secure_fw/spm/include/ffm/backend_ipc.h	1705
secure_fw/spm/include/ffm/backend_sfn.h	1706
secure_fw/spm/include/ffm/mailbox_agent_api.h	1707
secure_fw/spm/include/ffm/psa_api.h	1709
secure_fw/spm/include/interface/runtime_defs.h	1718
secure_fw/spm/include/interface/svc_num.h	1720
secure_fw/spm/include/load/asset_defs.h	1730
secure_fw/spm/include/load/interrupt_defs.h	1732
secure_fw/spm/include/load/ns_client_id_tz.h	1733
secure_fw/spm/include/load/partition_defs.h	1734
secure_fw/spm/include/load/service_defs.h	1738
secure_fw/spm/include/load/spm_load_api.h	1741
secure_fw/spm/ns_client_ext/tfm_ns_client_ext.c	1771
secure_fw/spm/ns_client_ext/tfm_ns_ctx.c	1776
secure_fw/spm/ns_client_ext/tfm_ns_ctx.h	1779
secure_fw/spm/ns_client_ext/tfm_spm_ns_ctx.c	1782

Chapter 6

Module Documentation

6.1 Implementation-specific definitions

6.2 API version

Macros

- `#define PSA_CRYPTO_API_VERSION_MAJOR 1`
- `#define PSA_CRYPTO_API_VERSION_MINOR 2`

6.2.1 Detailed Description

6.2.2 Macro Definition Documentation

6.2.2.1 PSA_CRYPTO_API_VERSION_MAJOR

```
#define PSA_CRYPTO_API_VERSION_MAJOR 1
```

The major version of this implementation of the PSA Crypto API

Definition at line 49 of file `crypto.h`.

6.2.2.2 PSA_CRYPTO_API_VERSION_MINOR

```
#define PSA_CRYPTO_API_VERSION_MINOR 2
```

The minor version of this implementation of the PSA Crypto API

Definition at line 54 of file `crypto.h`.

6.3 Library initialization

Functions

- [psa_status_t psa_crypto_init \(void\)](#)

Library initialization.

6.3.1 Detailed Description

6.3.2 Function Documentation

6.3.2.1 [psa_crypto_init\(\)](#)

```
psa_status_t psa_crypto_init (  
    void )
```

Library initialization.

Applications must call this function before calling any other function in this module.

Applications may call this function more than once. Once a call succeeds, subsequent calls are guaranteed to succeed.

If the application calls other functions before calling [psa_crypto_init\(\)](#), the behavior is undefined. Implementations are encouraged to either perform the operation as if the library had been initialized or to return [PSA_ERROR_BAD_STATE](#) or some other applicable error. In particular, implementations should not return a success status if the lack of initialization may have security implications, for example due to improper seeding of the random number generator.

Return values

PSA_SUCCESS	
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_INSUFFICIENT_STORAGE	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_INSUFFICIENT_ENTROPY	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_DATA_INVALID	
PSA_ERROR_DATA_CORRUPT	

Definition at line 59 of file `tfm_crypto_api.c`.

6.4 Key management

Functions

- `psa_status_t psa_purge_key (mbedtls_svc_key_id_t key)`
- `psa_status_t psa_copy_key (mbedtls_svc_key_id_t source_key, const psa_key_attributes_t *attributes, mbedtls_svc_key_id_t *target_key)`
- `psa_status_t psa_destroy_key (mbedtls_svc_key_id_t key)`

Destroy a key.

6.4.1 Detailed Description

6.4.2 Function Documentation

6.4.2.1 `psa_copy_key()`

```
psa_status_t psa_copy_key (
    mbedtls_svc_key_id_t source_key,
    const psa_key_attributes_t * attributes,
    mbedtls_svc_key_id_t * target_key )
```

Make a copy of a key.

Copy key material from one location to another.

This function is primarily useful to copy a key from one location to another, since it populates a key using the material from another key which may have a different lifetime.

This function may be used to share a key with a different party, subject to implementation-defined restrictions on key sharing.

The policy on the source key must have the usage flag `PSA_KEY_USAGE_COPY` set. This flag is sufficient to permit the copy if the key has the lifetime `PSA_KEY_LIFETIME_VOLATILE` or `PSA_KEY_LIFETIME_PERSISTENT`. Some secure elements do not provide a way to copy a key without making it extractable from the secure element. If a key is located in such a secure element, then the key must have both usage flags `PSA_KEY_USAGE_COPY` and `PSA_KEY_USAGE_EXPORT` in order to make a copy of the key outside the secure element.

The resulting key may only be used in a way that conforms to both the policy of the original key and the policy specified in the `attributes` parameter:

- The usage flags on the resulting key are the bitwise-and of the usage flags on the source policy and the usage flags in `attributes`.
- If both allow the same algorithm or wildcard-based algorithm policy, the resulting key has the same algorithm policy.
- If either of the policies allows an algorithm and the other policy allows a wildcard-based algorithm policy that includes this algorithm, the resulting key allows the same algorithm.
- If the policies do not allow any algorithm in common, this function fails with the status `PSA_ERROR_INVALID_ARGUMENT`.

The effect of this function on implementation-defined attributes is implementation-defined.

Parameters

	<i>source_key</i>	The key to copy. It must allow the usage PSA_KEY_USAGE_COPY . If a private or secret key is being copied outside of a secure element it must also allow PSA_KEY_USAGE_EXPORT .
in	<i>attributes</i>	The attributes for the new key. They are used as follows: - The key type and size may be 0. If either is nonzero, it must match the corresponding attribute of the source key. - The key location (the lifetime and, for persistent keys, the key identifier) is used directly. - The policy constraints (usage flags and algorithm policy) are combined from the source key and <i>attributes</i> so that both sets of restrictions apply, as described in the documentation of this function.
out	<i>target_key</i>	On success, an identifier for the newly created key. For persistent keys, this is the key identifier defined in <i>attributes</i> . 0 on failure.

Return values

PSA_SUCCESS	
PSA_ERROR_INVALID_HANDLE	<i>source_key</i> is invalid.
PSA_ERROR_ALREADY_EXISTS	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
PSA_ERROR_INVALID_ARGUMENT	The lifetime or identifier in <i>attributes</i> are invalid, or the policy constraints on the source and specified in <i>attributes</i> are incompatible, or <i>attributes</i> specifies a key type or key size which does not match the attributes of the source key.
PSA_ERROR_NOT_PERMITTED	The source key does not have the PSA_KEY_USAGE_COPY usage flag, or the source key is not exportable and its lifetime does not allow copying it to the target's lifetime.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_INSUFFICIENT_STORAGE	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_DATA_INVALID	
PSA_ERROR_DATA_CORRUPT	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 220 of file `tfm_crypto_api.c`.

6.4.2.2 `psa_destroy_key()`

```
psa_status_t psa_destroy_key (
    mbedtls_svc_key_id_t key )
```

Destroy a key.

This function destroys a key from both volatile memory and, if applicable, non-volatile storage. Implementations shall make a best effort to ensure that the key material cannot be recovered.

This function also erases any metadata such as policies and frees resources associated with the key.

If a key is currently in use in a multipart operation, then destroying the key will cause the multipart operation to fail.

Warning

We can only guarantee that the the key material will eventually be wiped from memory. With threading enabled and during concurrent execution, copies of the key material may still exist until all threads have finished using the key.

Parameters

<i>key</i>	Identifier of the key to erase. If this is 0, do nothing and return PSA_SUCCESS .
------------	---

Return values

PSA_SUCCESS	<i>key</i> was a valid identifier and the key material that it referred to has been erased. Alternatively, <i>key</i> is 0.
PSA_ERROR_NOT_PERMITTED	The key cannot be erased because it is read-only, either due to a policy or due to physical restrictions.
PSA_ERROR_INVALID_HANDLE	<i>key</i> is not a valid identifier nor 0.
PSA_ERROR_COMMUNICATION_FAILURE	There was a failure in communication with the cryptoprocessor. The key material may still be present in the cryptoprocessor.
PSA_ERROR_DATA_INVALID	This error is typically a result of either storage corruption on a cleartext storage backend, or an attempt to read data that was written by an incompatible version of the library.
PSA_ERROR_STORAGE_FAILURE	The storage is corrupted. Implementations shall make a best effort to erase key material even in this stage, however applications should be aware that it may be impossible to guarantee that the key material is not recoverable in such cases.
PSA_ERROR_CORRUPTION_DETECTED	An unexpected condition which is not a storage corruption or a communication failure occurred. The cryptoprocessor may have been compromised.
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 128 of file `tfm_crypto_api.c`.

6.4.2.3 `psa_purge_key()`

```
psa_status_t psa_purge_key (
    mbedtls_svc_key_id_t key )
```


Remove non-essential copies of key material from memory.

If the key identifier designates a volatile key, this functions does not do anything and returns successfully.

If the key identifier designates a persistent key, then this function will free all resources associated with the key in volatile memory. The key data in persistent storage is not affected and the key can still be used.

Parameters

<i>key</i>	Identifier of the key to purge.
------------	---------------------------------

Return values

<i>PSA_SUCCESS</i>	The key material will have been removed from memory if it is not currently required.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<i>key</i> is not a valid key identifier.
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 207 of file `tfm_crypto_api.c`.

6.5 Key import and export

Functions

- [psa_status_t psa_import_key](#) (const [psa_key_attributes_t](#) *attributes, const uint8_t *data, size_t data_length, [mbedtls_svc_key_id_t](#) *key)
Import a key in binary format.
- [psa_status_t psa_export_key](#) ([mbedtls_svc_key_id_t](#) key, uint8_t *data, size_t data_size, size_t *data_length)
Export a key in binary format.
- [psa_status_t psa_export_public_key](#) ([mbedtls_svc_key_id_t](#) key, uint8_t *data, size_t data_size, size_t *data_length)
Export a public key or the public part of a key pair in binary format.

6.5.1 Detailed Description

6.5.2 Function Documentation

6.5.2.1 [psa_export_key\(\)](#)

```
psa_status_t psa_export_key (
    mbedtls_svc_key_id_t key,
    uint8_t * data,
    size_t data_size,
    size_t * data_length )
```

Export a key in binary format.

The output of this function can be passed to [psa_import_key\(\)](#) to create an equivalent object.

If the implementation of [psa_import_key\(\)](#) supports other formats beyond the format specified here, the output from [psa_export_key\(\)](#) must use the representation specified here, not the original representation.

For standard key types, the output format is as follows:

- For symmetric keys (including MAC keys), the format is the raw bytes of the key.
- For RSA key pairs ([PSA_KEY_TYPE_RSA_KEY_PAIR](#)), the format is the non-encrypted DER encoding of the representation defined by PKCS#1 (RFC 8017) as RSAPrivateKey, version 0.

```
RSAPrivateKey ::= SEQUENCE {
    version          INTEGER, -- must be 0
    modulus          INTEGER, -- n
    publicExponent   INTEGER, -- e
    privateExponent  INTEGER, -- d
    prime1           INTEGER, -- p
    prime2           INTEGER, -- q
    exponent1        INTEGER, -- d mod (p-1)
    exponent2        INTEGER, -- d mod (q-1)
    coefficient       INTEGER, -- (inverse of q) mod p
}
```

- For elliptic curve key pairs (key types for which `PSA_KEY_TYPE_IS_ECC_KEY_PAIR` is true), the format is a representation of the private value as a `ceiling(m/8)`-byte string where `m` is the bit size associated with the curve, i.e. the bit size of the order of the curve's coordinate field. This byte string is in little-endian order for Montgomery curves (curve types `PSA_ECC_FAMILY_CURVEXXX`), and in big-endian order for Weierstrass curves (curve types `PSA_ECC_FAMILY_SECTXXX`, `PSA_ECC_FAMILY_SECPXXX` and `PSA_ECC_FAMILY_BRAINPOOL_PXXX`). For Weierstrass curves, this is the content of the `privateKey` field of the `ECPrivateKey` format defined by RFC 5915. For Montgomery curves, the format is defined by RFC 7748, and output is masked according to §5. For twisted Edwards curves, the private key is as defined by RFC 8032 (a 32-byte string for Edwards25519, a 57-byte string for Edwards448).
- For Diffie-Hellman key exchange key pairs (key types for which `PSA_KEY_TYPE_IS_DH_KEY_PAIR` is true), the format is the representation of the private key x as a big-endian byte string. The length of the byte string is the private key size in bytes (leading zeroes are not stripped).
- For public keys (key types for which `PSA_KEY_TYPE_IS_PUBLIC_KEY` is true), the format is the same as for `psa_export_public_key()`.

The policy on the key must have the usage flag `PSA_KEY_USAGE_EXPORT` set.

Parameters

	<i>key</i>	Identifier of the key to export. It must allow the usage <code>PSA_KEY_USAGE_EXPORT</code> , unless it is a public key.
out	<i>data</i>	Buffer where the key data is to be written.
	<i>data_size</i>	Size of the <code>data</code> buffer in bytes.
out	<i>data_length</i>	On success, the number of bytes that make up the key data.

Return values

<code>PSA_SUCCESS</code>	
<code>PSA_ERROR_INVALID_HANDLE</code>	
<code>PSA_ERROR_NOT_PERMITTED</code>	The key does not have the <code>PSA_KEY_USAGE_EXPORT</code> flag.
<code>PSA_ERROR_NOT_SUPPORTED</code>	
<code>PSA_ERROR_BUFFER_TOO_SMALL</code>	The size of the <code>data</code> buffer is too small. You can determine a sufficient buffer size by calling <code>PSA_EXPORT_KEY_OUTPUT_SIZE(type, bits)</code> where <code>type</code> is the key type and <code>bits</code> is the key size in bits.
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 158 of file `tfm_crypto_api.c`.

6.5.2.2 `psa_export_public_key()`

```
psa_status_t psa_export_public_key (
    mbedtls_svc_key_id_t key,
```

```
uint8_t * data,
size_t data_size,
size_t * data_length )
```

Export a public key or the public part of a key pair in binary format.

The output of this function can be passed to [psa_import_key\(\)](#) to create an object that is equivalent to the public key.

This specification supports a single format for each key type. Implementations may support other formats as long as the standard format is supported. Implementations that support other formats should ensure that the formats are clearly unambiguous so as to minimize the risk that an invalid input is accidentally interpreted according to a different format.

For standard key types, the output format is as follows:

- For RSA public keys ([PSA_KEY_TYPE_RSA_PUBLIC_KEY](#)), the DER encoding of the representation defined by RFC 3279 §2.3.1 as RSAPublicKey.

```
RSAPublicKey ::= SEQUENCE {
    modulus          INTEGER,      -- n
    publicExponent   INTEGER }    -- e
```
- For elliptic curve keys on a twisted Edwards curve (key types for which [PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY](#) is true and [PSA_KEY_TYPE_ECC_GET_FAMILY](#) returns [PSA_ECC_FAMILY_TWISTED_EDWARDS](#)), the public key is as defined by RFC 8032 (a 32-byte string for Edwards25519, a 57-byte string for Edwards448).
- For other elliptic curve public keys (key types for which [PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY](#) is true), the format is the uncompressed representation defined by SEC1 §2.3.3 as the content of an ECPoint. Let m be the bit size associated with the curve, i.e. the bit size of q for a curve over F_q . The representation consists of:
 - The byte 0x04;
 - x_P as a $\text{ceiling}(m/8)$ -byte string, big-endian;
 - y_P as a $\text{ceiling}(m/8)$ -byte string, big-endian.
- For Diffie-Hellman key exchange public keys (key types for which [PSA_KEY_TYPE_IS_DH_PUBLIC_KEY](#) is true), the format is the representation of the public key $y = g^x \bmod p$ as a big-endian byte string. The length of the byte string is the length of the base prime p in bytes.

Exporting a public key object or the public part of a key pair is always permitted, regardless of the key's usage flags.

Parameters

	<i>key</i>	Identifier of the key to export.
out	<i>data</i>	Buffer where the key data is to be written.
	<i>data_size</i>	Size of the <i>data</i> buffer in bytes.
out	<i>data_length</i>	On success, the number of bytes that make up the key data.

Return values

PSA_SUCCESS	
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_INVALID_ARGUMENT	The key is neither a public key nor a key pair.
PSA_ERROR_NOT_SUPPORTED	

Return values

PSA_ERROR_BUFFER_TOO_SMALL	The size of the <code>data</code> buffer is too small. You can determine a sufficient buffer size by calling PSA_EXPORT_KEY_OUTPUT_SIZE(PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_BITS) where <code>type</code> is the key type and <code>bits</code> is the key size in bits.
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 182 of file `tfm_crypto_api.c`.

6.5.2.3 `psa_import_key()`

```
psa_status_t psa_import_key (
    const psa_key_attributes_t * attributes,
    const uint8_t * data,
    size_t data_length,
    mbedtls_svc_key_id_t * key )
```

Import a key in binary format.

This function supports any output from [psa_export_key\(\)](#). Refer to the documentation of [psa_export_public_key\(\)](#) for the format of public keys and to the documentation of [psa_export_key\(\)](#) for the format for other key types.

The key data determines the key size. The attributes may optionally specify a key size; in this case it must match the size determined from the key data. A key size of 0 in `attributes` indicates that the key size is solely determined by the key data.

Implementations must reject an attempt to import a key of size 0.

This specification supports a single format for each key type. Implementations may support other formats as long as the standard format is supported. Implementations that support other formats should ensure that the formats are clearly unambiguous so as to minimize the risk that an invalid input is accidentally interpreted according to a different format.

Parameters

in	<i>attributes</i>	The attributes for the new key. The key size is always determined from the <code>data</code> buffer. If the key size in <code>attributes</code> is nonzero, it must be equal to the size from <code>data</code> .
out	<i>key</i>	On success, an identifier to the newly created key. For persistent keys, this is the key identifier defined in <code>attributes</code> . 0 on failure.
in	<i>data</i>	Buffer containing the key data. The content of this buffer is interpreted according to the type declared in <code>attributes</code> . All implementations must support at least the format described in the documentation of psa_export_key() or psa_export_public_key() for the chosen type. Implementations may allow other formats, but should be conservative: implementations should err on the side of rejecting content if it may be erroneous (e.g. wrong type or truncated data).
Generated by Doxygen	<i>data_length</i>	Size of the <code>data</code> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
<i>PSA_ERROR_ALREADY_EXISTS</i>	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The key type or key size is not supported, either by the implementation in general or in this particular persistent location.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The key attributes, as a whole, are invalid, or the key data is not correctly formatted, or the size in <code>attributes</code> is nonzero and does not match the size of the key data.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_INSUFFICIENT_STORAGE</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 108 of file `tfm_crypto_api.c`.

6.6 Message digests

Macros

- `#define PSA_HASH_OPERATION_INIT { 0, { 0 } }`

Typedefs

- `typedef struct psa_hash_operation_s psa_hash_operation_t`

Functions

- `psa_status_t psa_hash_compute` (`psa_algorithm_t` alg, `const uint8_t *input`, `size_t input_length`, `uint8_t *hash`, `size_t hash_size`, `size_t *hash_length`)
- `psa_status_t psa_hash_compare` (`psa_algorithm_t` alg, `const uint8_t *input`, `size_t input_length`, `const uint8_t *hash`, `size_t hash_length`)
- `psa_status_t psa_hash_setup` (`psa_hash_operation_t *operation`, `psa_algorithm_t` alg)
- `psa_status_t psa_hash_update` (`psa_hash_operation_t *operation`, `const uint8_t *input`, `size_t input_length`)
- `psa_status_t psa_hash_finish` (`psa_hash_operation_t *operation`, `uint8_t *hash`, `size_t hash_size`, `size_t *hash_length`)
- `psa_status_t psa_hash_verify` (`psa_hash_operation_t *operation`, `const uint8_t *hash`, `size_t hash_length`)
- `psa_status_t psa_hash_abort` (`psa_hash_operation_t *operation`)
- `psa_status_t psa_hash_clone` (`const psa_hash_operation_t *source_operation`, `psa_hash_operation_t *target_operation`)

6.6.1 Detailed Description

6.6.2 Macro Definition Documentation

6.6.2.1 PSA_HASH_OPERATION_INIT

```
#define PSA_HASH_OPERATION_INIT { 0, { 0 } }
```

This macro returns a suitable initializer for a hash operation object of type `psa_hash_operation_t`.

Definition at line 82 of file `crypto_struct.h`.

6.6.3 Typedef Documentation

6.6.3.1 `psa_hash_operation_t`

```
typedef struct psa_hash_operation_s psa_hash_operation_t
```

The type of the state data structure for multipart hash operations.

Before calling any function on a hash operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_hash_operation_t operation;
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_hash_operation_t operation = {0};
```
- Initialize the structure to the initializer `PSA_HASH_OPERATION_INIT`, for example:

```
psa_hash_operation_t operation = PSA_HASH_OPERATION_INIT;
```
- Assign the result of the function `psa_hash_operation_init()` to the structure, for example:

```
psa_hash_operation_t operation;
operation = psa_hash_operation_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 940 of file `crypto.h`.

6.6.4 Function Documentation

6.6.4.1 `psa_hash_abort()`

```
psa_status_t psa_hash_abort (
    psa_hash_operation_t * operation )
```

Abort a hash operation.

Aborting an operation frees all associated resources except for the `operation` structure itself. Once aborted, the operation object can be reused for another operation by calling `psa_hash_setup()` again.

You may call this function any time after the operation object has been initialized by one of the methods described in `psa_hash_operation_t`.

In particular, calling `psa_hash_abort()` after the operation has been terminated by a call to `psa_hash_abort()`, `psa_hash_finish()` or `psa_hash_verify()` is safe and has no effect.

Parameters

in, out	<i>operation</i>	Initialized hash operation.
---------	------------------	-----------------------------

Return values

<i>PSA_SUCCESS</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 478 of file `tfm_crypto_api.c`.

6.6.4.2 `psa_hash_clone()`

```
psa_status_t psa_hash_clone (
    const psa_hash_operation_t * source_operation,
    psa_hash_operation_t * target_operation )
```

Clone a hash operation.

This function copies the state of an ongoing hash operation to a new operation object. In other words, this function is equivalent to calling [psa_hash_setup\(\)](#) on `target_operation` with the same algorithm that `source_operation` was set up for, then [psa_hash_update\(\)](#) on `target_operation` with the same input that that was passed to `source_operation`. After this function returns, the two objects are independent, i.e. subsequent calls involving one of the objects do not affect the other object.

Parameters

in	<i>source_operation</i>	The active hash operation to clone.
in, out	<i>target_operation</i>	The operation object to set up. It must be initialized but not active.

Return values

<i>PSA_SUCCESS</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_BAD_STATE</i>	The <code>source_operation</code> state is not valid (it must be active), or the <code>target_operation</code> state is not valid (it must be inactive), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 495 of file `tfm_crypto_api.c`.

6.6.4.3 `psa_hash_compare()`

```
psa_status_t psa_hash_compare (
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    const uint8_t * hash,
    size_t hash_length )
```

Calculate the hash (digest) of a message and compare it with a reference value.

Parameters

	<i>alg</i>	The hash algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (alg) is true).
in	<i>input</i>	Buffer containing the message to hash.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
in	<i>hash</i>	Buffer containing the expected hash value.
	<i>hash_length</i>	Size of the <i>hash</i> buffer in bytes.

Return values

PSA_SUCCESS	The expected hash is identical to the actual hash of the input.
PSA_ERROR_INVALID_SIGNATURE	The hash of the message was calculated successfully, but it differs from the expected hash.
PSA_ERROR_NOT_SUPPORTED	<i>alg</i> is not supported or is not a hash algorithm.
PSA_ERROR_INVALID_ARGUMENT	<i>input_length</i> or <i>hash_length</i> do not match the hash size for <i>alg</i>
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 549 of file `tfm_crypto_api.c`.

6.6.4.4 `psa_hash_compute()`

```
psa_status_t psa_hash_compute (
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    uint8_t * hash,
    size_t hash_size,
    size_t * hash_length )
```

Calculate the hash (digest) of a message.

Note

To verify the hash of a message against an expected value, use [psa_hash_compare\(\)](#) instead.

Parameters

	<i>alg</i>	The hash algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (alg) is true).
in	<i>input</i>	Buffer containing the message to hash.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
out	<i>hash</i>	Buffer where the hash is to be written.
	<i>hash_size</i>	Size of the <i>hash</i> buffer in bytes.
out	<i>hash_length</i>	On success, the number of bytes that make up the hash value. This is always PSA_HASH_LENGTH (alg).

Return values

PSA_SUCCESS	Success.
PSA_ERROR_NOT_SUPPORTED	alg is not supported or is not a hash algorithm.
PSA_ERROR_INVALID_ARGUMENT	
PSA_ERROR_BUFFER_TOO_SMALL	hash_size is too small
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 520 of file tfm_crypto_api.c.

6.6.4.5 [psa_hash_finish\(\)](#)

```
psa_status_t psa_hash_finish (
    psa_hash_operation_t * operation,
    uint8_t * hash,
    size_t hash_size,
    size_t * hash_length )
```

Finish the calculation of the hash of a message.

The application must call [psa_hash_setup\(\)](#) before calling this function. This function calculates the hash of the message formed by concatenating the inputs passed to preceding calls to [psa_hash_update\(\)](#).

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_hash_abort\(\)](#).

Warning

Applications should not call this function if they expect a specific value for the hash. Call [psa_hash_verify\(\)](#) instead. Beware that comparing integrity or authenticity data such as hash values with a function such as `memcmp` is risky because the time taken by the comparison may leak information about the hashed data which could allow an attacker to guess a valid hash and thereby bypass security controls.

Parameters

in, out	<i>operation</i>	Active hash operation.
out	<i>hash</i>	Buffer where the hash is to be written.
	<i>hash_size</i>	Size of the <code>hash</code> buffer in bytes.
out	<i>hash_length</i>	On success, the number of bytes that make up the hash value. This is always <code>PSA_HASH_LENGTH(alg)</code> where <code>alg</code> is the hash algorithm that is calculated.

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_BUFFER_TOO_SMALL</code>	The size of the <code>hash</code> buffer is too small. You can determine a sufficient buffer size by calling <code>PSA_HASH_LENGTH(alg)</code> where <code>alg</code> is the hash algorithm that is calculated.
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid (it must be active), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 432 of file `tfm_crypto_api.c`.

6.6.4.6 `psa_hash_setup()`

```
psa_status_t psa_hash_setup (
    psa_hash_operation_t * operation,
    psa_algorithm_t alg )
```

Set up a multipart hash operation.

The sequence of operations to calculate a hash (message digest) is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `psa_hash_operation_t`, e.g. `PSA_HASH_OPERATION_INIT`.
3. Call `psa_hash_setup()` to specify the algorithm.
4. Call `psa_hash_update()` zero, one or more times, passing a fragment of the message each time. The hash that is calculated is the hash of the concatenation of these messages in order.
5. To calculate the hash, call `psa_hash_finish()`. To compare the hash with an expected value, call `psa_hash_verify()`.

If an error occurs at any step after a call to `psa_hash_setup()`, the operation will need to be reset by a call to `psa_hash_abort()`. The application may call `psa_hash_abort()` at any time after the operation has been initialized.

After a successful call to `psa_hash_setup()`, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to `psa_hash_finish()` or `psa_hash_verify()`.
- A call to `psa_hash_abort()`.

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for psa_hash_operation_t and not yet in use.
	<i>alg</i>	The hash algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (<i>alg</i>) is true).

Return values

PSA_SUCCESS	Success.
PSA_ERROR_NOT_SUPPORTED	<i>alg</i> is not a supported hash algorithm.
PSA_ERROR_INVALID_ARGUMENT	<i>alg</i> is not a hash algorithm.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be inactive), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 396 of file `tfm_crypto_api.c`.

6.6.4.7 `psa_hash_update()`

```
psa_status_t psa_hash_update (
    psa_hash_operation_t * operation,
    const uint8_t * input,
    size_t input_length )
```

Add a message fragment to a multipart hash operation.

The application must call [psa_hash_setup\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_hash_abort\(\)](#).

Parameters

<i>in, out</i>	<i>operation</i>	Active hash operation.
<i>in</i>	<i>input</i>	Buffer containing the message fragment to hash.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	

Return values

<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 415 of file `tfm_crypto_api.c`.

6.6.4.8 `psa_hash_verify()`

```
psa_status_t psa_hash_verify (
    psa_hash_operation_t * operation,
    const uint8_t * hash,
    size_t hash_length )
```

Finish the calculation of the hash of a message and compare it with an expected value.

The application must call [psa_hash_setup\(\)](#) before calling this function. This function calculates the hash of the message formed by concatenating the inputs passed to preceding calls to [psa_hash_update\(\)](#). It then compares the calculated hash with the expected hash passed as a parameter to this function.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_hash_abort\(\)](#).

Note

Implementations shall make the best effort to ensure that the comparison between the actual hash and the expected hash is performed in constant time.

Parameters

in, out	<i>operation</i>	Active hash operation.
in	<i>hash</i>	Buffer containing the expected hash value.
	<i>hash_length</i>	Size of the <code>hash</code> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	The expected hash is identical to the actual hash of the message.
<i>PSA_ERROR_INVALID_SIGNATURE</i>	The hash of the message was calculated successfully, but it differs from the expected hash.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	

Return values

<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.
--	---

Definition at line 458 of file `tfm_crypto_api.c`.

6.7 Extendable-operation functions (XOF)

Macros

- `#define PSA_XOF_OPERATION_INIT { 0, 0, 0, 0, 0, 0, 0, { 0 } }`

Typedefs

- `typedef struct psa_xof_operation_s psa_xof_operation_t`

Functions

- `psa_status_t psa_xof_setup (psa_xof_operation_t *operation, psa_algorithm_t alg)`
- `psa_status_t psa_xof_set_context (psa_xof_operation_t *operation, const uint8_t *context, size_t context_length)`
- `psa_status_t psa_xof_update (psa_xof_operation_t *operation, const uint8_t *input, size_t input_length)`
- `psa_status_t psa_xof_output (psa_xof_operation_t *operation, uint8_t *output, size_t output_length)`
- `psa_status_t psa_xof_abort (psa_xof_operation_t *operation)`

6.7.1 Detailed Description

6.7.2 Macro Definition Documentation

6.7.2.1 PSA_XOF_OPERATION_INIT

```
#define PSA_XOF_OPERATION_INIT { 0, 0, 0, 0, 0, 0, 0, { 0 } }
```

This macro returns a suitable initializer for a XOF operation object of type `psa_xof_operation_t`.

Definition at line 118 of file `crypto_struct.h`.

6.7.3 Typedef Documentation

6.7.3.1 `psa_xof_operation_t`

```
typedef struct psa_xof_operation_s psa_xof_operation_t
```

The type of the state data structure for multipart XOF operations.

Before calling any function on a XOF operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_xof_operation_t operation;
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_xof_operation_t operation = {0};
```
- Initialize the structure to the initializer `PSA_XOF_OPERATION_INIT`, for example:

```
psa_xof_operation_t operation = PSA_XOF_OPERATION_INIT;
```
- Assign the result of the function `psa_xof_operation_init()` to the structure, for example:

```
psa_xof_operation_t operation;
operation = psa_xof_operation_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 1208 of file `crypto.h`.

6.7.4 Function Documentation

6.7.4.1 `psa_xof_abort()`

```
psa_status_t psa_xof_abort (
    psa_xof_operation_t * operation )
```

Abort a multipart XOF (extendable-operation function) operation.

Parameters

in, out	<i>operation</i>	The operation object to abort. It must have been initialized as per the documentation for <code>psa_xof_operation_t</code> and not yet in use.
---------	------------------	--

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is corrupted.

6.7.4.2 `psa_xof_output()`

```
psa_status_t psa_xof_output (
    psa_xof_operation_t * operation,
    uint8_t * output,
    size_t output_length )
```

Extract output from a multipart XOF (extendable-operation function) operation.

This function switches the operation to output mode, even when `output_length` is 0.

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to use. It must have been set up with psa_xof_setup() . It must have a context set with psa_xof_set_context() if the algorithm requires it. It must not yet have been aborted with psa_xof_abort() .
<i>out</i>	<i>output</i>	On success, the output fragment.
	<i>output_length</i>	The number of bytes to write to <i>output</i> .

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be active, and it must have a context set if the algorithm requires it).

6.7.4.3 `psa_xof_set_context()`

```
psa_status_t psa_xof_set_context (
    psa_xof_operation_t * operation,
    const uint8_t * context,
    size_t context_length )
```

Pass a context to a multipart XOF (extendable-operation function) operation.

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to use. It must have been set up with psa_xof_setup() , and must not yet have been received a context with psa_xof_set_context() , received input with psa_xof_update() , switched to output mode with psa_xof_output() , or aborted with psa_xof_abort() .
<i>in</i>	<i>context</i>	The context to use.
	<i>context_length</i>	Size of the <code>context</code> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The algorithm used by <code>operation</code> does not allow a context, or the context value is invalid for this algorithm.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active, it must not already have a context set, it must not already have input, and it must not have already been switched to output mode).

6.7.4.4 `psa_xof_setup()`

```

psa_status_t psa_xof_setup (
    psa_xof_operation_t * operation,
    psa_algorithm_t alg )

```

Set up a multipart XOF (extendable-operation function) operation.

The sequence of operations to calculate a XOF is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [`psa\_xof\_operation\_t`](#), e.g. [`PSA\_XOF\_OPERATION\_INIT`](#).
3. Call [`psa\_xof\_setup\(\)`](#) to specify the algorithm.
4. If the XOF uses a context, call [`psa\_xof\_set\_context\(\)`](#).
5. Call [`psa\_xof\_update\(\)`](#) zero, one or more times, passing successive fragments of the input.
6. Call [`psa\_xof\_output\(\)`](#) zero, one or more times to obtain successive fragments of the output.
7. Call [`psa\_xof\_abort\(\)`](#) to free the resources associated with the operation (other than the operation object itself).

If an error occurs at any step after a call to [`psa\_xof\_setup\(\)`](#), the operation will need to be reset by a call to [`psa\_xof\_abort\(\)`](#). The application may call [`psa\_xof\_abort\(\)`](#) at any time after the operation has been initialized.

After a successful call to [`psa\_xof\_setup\(\)`](#), the application must eventually terminate the operation by calling [`psa\_xof\_abort\(\)`](#).

Parameters

<code>in, out</code>	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for <code>psa_xof_operation_t</code> and not yet in use.
	<i>alg</i>	The XOF algorithm to compute (PSA_ALG_XXX value such that <code>PSA_ALG_IS_XOF(alg)</code> is true).

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_NOT_SUPPORTED</i>	alg is not supported.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be inactive).

6.7.4.5 `psa_xof_update()`

```

psa_status_t psa_xof_update (
    psa_xof_operation_t * operation,
    const uint8_t * input,
    size_t input_length )

```

Pass input to a multipart XOF (extendable-operation function) operation.

This function switches the operation to input mode, even when `input_length` is 0.

Parameters

<code>in, out</code>	<i>operation</i>	The operation object to use. It must have been set up with <code>psa_xof_setup()</code> . It must have a context set with <code>psa_xof_set_context()</code> if the algorithm requires it. It must not yet have been switched to output mode with <code>psa_xof_output()</code> or aborted with <code>psa_xof_abort()</code> .
<code>in</code>	<i>input</i>	The input fragment.
	<i>input_length</i>	Size of the <code>input</code> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active, it must have a context set if the algorithm requires it, and it must not yet have been switched to output mode).

6.8 Message authentication codes

Macros

- `#define PSA_MAC_OPERATION_INIT { 0, 0, 0, { 0 } }`

Typedefs

- typedef struct `psa_mac_operation_s` `psa_mac_operation_t`

Functions

- `psa_status_t psa_mac_compute` (`MBEDTLS_SVC_KEY_ID_T` key, `psa_algorithm_t` alg, const `uint8_t` *input, `size_t` input_length, `uint8_t` *mac, `size_t` mac_size, `size_t` *mac_length)
- `psa_status_t psa_mac_verify` (`MBEDTLS_SVC_KEY_ID_T` key, `psa_algorithm_t` alg, const `uint8_t` *input, `size_t` input_length, const `uint8_t` *mac, `size_t` mac_length)
- `psa_status_t psa_mac_sign_setup` (`psa_mac_operation_t` *operation, `MBEDTLS_SVC_KEY_ID_T` key, `psa_algorithm_t` alg)
- `psa_status_t psa_mac_verify_setup` (`psa_mac_operation_t` *operation, `MBEDTLS_SVC_KEY_ID_T` key, `psa_algorithm_t` alg)
- `psa_status_t psa_mac_update` (`psa_mac_operation_t` *operation, const `uint8_t` *input, `size_t` input_length)
- `psa_status_t psa_mac_sign_finish` (`psa_mac_operation_t` *operation, `uint8_t` *mac, `size_t` mac_size, `size_t` *mac_length)
- `psa_status_t psa_mac_verify_finish` (`psa_mac_operation_t` *operation, const `uint8_t` *mac, `size_t` mac_length)
- `psa_status_t psa_mac_abort` (`psa_mac_operation_t` *operation)

6.8.1 Detailed Description

6.8.2 Macro Definition Documentation

6.8.2.1 PSA_MAC_OPERATION_INIT

```
#define PSA_MAC_OPERATION_INIT { 0, 0, 0, { 0 } }
```

This macro returns a suitable initializer for a MAC operation object of type `psa_mac_operation_t`.

Definition at line 192 of file `crypto_struct.h`.

6.8.3 Typedef Documentation

6.8.3.1 `psa_mac_operation_t`

```
typedef struct psa_mac_operation_s psa_mac_operation_t
```

The type of the state data structure for multipart MAC operations.

Before calling any function on a MAC operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_mac_operation_t operation;
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_mac_operation_t operation = {0};
```
- Initialize the structure to the initializer `PSA_MAC_OPERATION_INIT`, for example:

```
psa_mac_operation_t operation = PSA_MAC_OPERATION_INIT;
```
- Assign the result of the function `psa_mac_operation_init()` to the structure, for example:

```
psa_mac_operation_t operation;
operation = psa_mac_operation_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 1488 of file `crypto.h`.

6.8.4 Function Documentation

6.8.4.1 `psa_mac_abort()`

```
psa_status_t psa_mac_abort (
    psa_mac_operation_t * operation )
```

Abort a MAC operation.

Aborting an operation frees all associated resources except for the `operation` structure itself. Once aborted, the operation object can be reused for another operation by calling `psa_mac_sign_setup()` or `psa_mac_verify_setup()` again.

You may call this function any time after the operation object has been initialized by one of the methods described in `psa_mac_operation_t`.

In particular, calling `psa_mac_abort()` after the operation has been terminated by a call to `psa_mac_abort()`, `psa_mac_sign_finish()` or `psa_mac_verify_finish()` is safe and has no effect.

Parameters

in, out	<i>operation</i>	Initialized MAC operation.
---------	------------------	----------------------------

Return values

<i>PSA_SUCCESS</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 674 of file `tfm_crypto_api.c`.

6.8.4.2 `psa_mac_compute()`

```
psa_status_t psa_mac_compute (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    uint8_t * mac,
    size_t mac_size,
    size_t * mac_length )
```

Calculate the MAC (message authentication code) of a message.

Note

To verify the MAC of a message against an expected value, use [`psa\_mac\_verify\(\)`](#) instead. Beware that comparing integrity or authenticity data such as MAC values with a function such as `memcmp` is risky because the time taken by the comparison may leak information about the MAC value which could allow an attacker to guess a valid MAC and thereby bypass security controls.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must allow the usage <code>PSA_KEY_USAGE_SIGN_MESSAGE</code> .
	<i>alg</i>	The MAC algorithm to compute (PSA_ALG_XXX value such that <code>PSA_ALG_IS_MAC(alg)</code> is true).
in	<i>input</i>	Buffer containing the input message.
	<i>input_length</i>	Size of the <code>input</code> buffer in bytes.
out	<i>mac</i>	Buffer where the MAC value is to be written.
	<i>mac_size</i>	Size of the <code>mac</code> buffer in bytes.
out	<i>mac_length</i>	On success, the number of bytes that make up the MAC value.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	

Return values

<i>PSA_ERROR_INVALID_ARGUMENT</i>	key is not compatible with alg.
<i>PSA_ERROR_NOT_SUPPORTED</i>	alg is not supported or is not a MAC algorithm.
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	mac_size is too small
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	The key could not be retrieved from storage.
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1441 of file tfm_crypto_api.c.

6.8.4.3 `psa_mac_sign_finish()`

```
psa_status_t psa_mac_sign_finish (
    psa_mac_operation_t * operation,
    uint8_t * mac,
    size_t mac_size,
    size_t * mac_length )
```

Finish the calculation of the MAC of a message.

The application must call [`psa\_mac\_sign\_setup\(\)`](#) before calling this function. This function calculates the MAC of the message formed by concatenating the inputs passed to preceding calls to [`psa\_mac\_update\(\)`](#).

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_mac\_abort\(\)`](#).

Warning

Applications should not call this function if they expect a specific value for the MAC. Call [`psa\_mac\_verify\_finish\(\)`](#) instead. Beware that comparing integrity or authenticity data such as MAC values with a function such as `memcmp` is risky because the time taken by the comparison may leak information about the MAC value which could allow an attacker to guess a valid MAC and thereby bypass security controls.

Parameters

in, out	<i>operation</i>	Active MAC operation.
out	<i>mac</i>	Buffer where the MAC value is to be written.
	<i>mac_size</i>	Size of the mac buffer in bytes.
out	<i>mac_length</i>	On success, the number of bytes that make up the MAC value. This is always <code>PSA_MAC_LENGTH(key_type, key_bits, alg)</code> where <code>key_type</code> and <code>key_bits</code> are the type and bit-size respectively of the key and <code>alg</code> is the MAC algorithm that is calculated.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	The size of the <code>mac</code> buffer is too small. You can determine a sufficient buffer size by calling <i>PSA_MAC_LENGTH()</i> .
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be an active mac sign operation), or the library has not been previously initialized by <i>psa_crypto_init()</i> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 628 of file `tfm_crypto_api.c`.

6.8.4.4 `psa_mac_sign_setup()`

```
psa_status_t psa_mac_sign_setup (
    psa_mac_operation_t * operation,
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg )
```

Set up a multipart MAC calculation operation.

This function sets up the calculation of the MAC (message authentication code) of a byte string. To verify the MAC of a message against an expected value, use [`psa\_mac\_verify\_setup\(\)`](#) instead.

The sequence of operations to calculate a MAC is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [`psa\_mac\_operation\_t`](#), e.g. [`PSA\_MAC\_OPERATION\_INIT`](#).
3. Call [`psa\_mac\_sign\_setup\(\)`](#) to specify the algorithm and key.
4. Call [`psa\_mac\_update\(\)`](#) zero, one or more times, passing a fragment of the message each time. The MAC that is calculated is the MAC of the concatenation of these messages in order.
5. At the end of the message, call [`psa\_mac\_sign\_finish\(\)`](#) to finish calculating the MAC value and retrieve it.

If an error occurs at any step after a call to [`psa\_mac\_sign\_setup\(\)`](#), the operation will need to be reset by a call to [`psa\_mac\_abort\(\)`](#). The application may call [`psa\_mac\_abort\(\)`](#) at any time after the operation has been initialized.

After a successful call to [`psa\_mac\_sign\_setup\(\)`](#), the application must eventually terminate the operation through one of the following methods:

- A successful call to [`psa\_mac\_sign\_finish\(\)`](#).
- A call to [`psa\_mac\_abort\(\)`](#).

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for psa_mac_operation_t and not yet in use.
	<i>key</i>	Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage <code>PSA_KEY_USAGE_SIGN_MESSAGE</code> .
	<i>alg</i>	The MAC algorithm to compute (<code>PSA_ALG_XXX</code> value such that PSA_ALG_IS_MAC (<i>alg</i>) is true).

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	
PSA_ERROR_INVALID_ARGUMENT	<i>key</i> is not compatible with <i>alg</i> .
PSA_ERROR_NOT_SUPPORTED	<i>alg</i> is not supported or is not a MAC algorithm.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	The key could not be retrieved from storage.
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be inactive), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 569 of file `tfm_crypto_api.c`.

6.8.4.5 `psa_mac_update()`

```
psa_status_t psa_mac_update (
    psa_mac_operation_t * operation,
    const uint8_t * input,
    size_t input_length )
```

Add a message fragment to a multipart MAC operation.

The application must call [psa_mac_sign_setup\(\)](#) or [psa_mac_verify_setup\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_mac_abort\(\)](#).

Parameters

<i>in, out</i>	<i>operation</i>	Active MAC operation.
<i>in</i>	<i>input</i>	Buffer containing the message fragment to add to the MAC calculation.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 611 of file `tfm_crypto_api.c`.

6.8.4.6 `psa_mac_verify()`

```
psa_status_t psa_mac_verify (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    const uint8_t * mac,
    size_t mac_length )
```

Calculate the MAC of a message and compare it with a reference value.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must allow the usage <code>PSA_KEY_USAGE_VERIFY_MESSAGE</code> .
	<i>alg</i>	The MAC algorithm to compute (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_MAC(alg)</code> is true).
in	<i>input</i>	Buffer containing the input message.
	<i>input_length</i>	Size of the <code>input</code> buffer in bytes.
in	<i>mac</i>	Buffer containing the expected MAC value.
	<i>mac_length</i>	Size of the <code>mac</code> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	The expected MAC is identical to the actual MAC of the input.
<i>PSA_ERROR_INVALID_SIGNATURE</i>	The MAC of the message was calculated successfully, but it differs from the expected value.
<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<code>key</code> is not compatible with <code>alg</code> .
<i>PSA_ERROR_NOT_SUPPORTED</i>	<code>alg</code> is not supported or is not a MAC algorithm.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	

Return values

<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	The key could not be retrieved from storage.
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1470 of file `tfm_crypto_api.c`.

6.8.4.7 `psa_mac_verify_finish()`

```
psa_status_t psa_mac_verify_finish (
    psa_mac_operation_t * operation,
    const uint8_t * mac,
    size_t mac_length )
```

Finish the calculation of the MAC of a message and compare it with an expected value.

The application must call [`psa\_mac\_verify\_setup\(\)`](#) before calling this function. This function calculates the MAC of the message formed by concatenating the inputs passed to preceding calls to [`psa\_mac\_update\(\)`](#). It then compares the calculated MAC with the expected MAC passed as a parameter to this function.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_mac\_abort\(\)`](#).

Note

Implementations shall make the best effort to ensure that the comparison between the actual MAC and the expected MAC is performed in constant time.

Parameters

<i>in, out</i>	<i>operation</i>	Active MAC operation.
<i>in</i>	<i>mac</i>	Buffer containing the expected MAC value.
	<i>mac_length</i>	Size of the <code>mac</code> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	The expected MAC is identical to the actual MAC of the message.
<i>PSA_ERROR_INVALID_SIGNATURE</i>	The MAC of the message was calculated successfully, but it differs from the expected MAC.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	

Return values

PSA_ERROR_BAD_STATE	The operation state is not valid (it must be an active mac verify operation), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.
-------------------------------------	--

Definition at line 654 of file `tfm_crypto_api.c`.

6.8.4.8 `psa_mac_verify_setup()`

```
psa_status_t psa_mac_verify_setup (
    psa_mac_operation_t * operation,
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg )
```

Set up a multipart MAC verification operation.

This function sets up the verification of the MAC (message authentication code) of a byte string against an expected value.

The sequence of operations to verify a MAC is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_mac_operation_t](#), e.g. [PSA_MAC_OPERATION_INIT](#).
3. Call [psa_mac_verify_setup\(\)](#) to specify the algorithm and key.
4. Call [psa_mac_update\(\)](#) zero, one or more times, passing a fragment of the message each time. The MAC that is calculated is the MAC of the concatenation of these messages in order.
5. At the end of the message, call [psa_mac_verify_finish\(\)](#) to finish calculating the actual MAC of the message and verify it against the expected value.

If an error occurs at any step after a call to [psa_mac_verify_setup\(\)](#), the operation will need to be reset by a call to [psa_mac_abort\(\)](#). The application may call [psa_mac_abort\(\)](#) at any time after the operation has been initialized.

After a successful call to [psa_mac_verify_setup\(\)](#), the application must eventually terminate the operation through one of the following methods:

- A successful call to [psa_mac_verify_finish\(\)](#).
- A call to [psa_mac_abort\(\)](#).

Parameters

in, out	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for psa_mac_operation_t and not yet in use.
	<i>key</i>	Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage <code>PSA_KEY_USAGE_VERIFY_MESSAGE</code> .
	<i>alg</i>	The MAC algorithm to compute (<code>PSA_ALG_XXX</code> value such that PSA_ALG_IS_MAC (<i>alg</i>) is true).
Generated by Doxygen		

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	key is not compatible with alg.
<i>PSA_ERROR_NOT_SUPPORTED</i>	alg is not supported or is not a MAC algorithm.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	The key could not be retrieved from storage.
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be inactive), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 590 of file tfm_crypto_api.c.

6.9 Symmetric ciphers

Macros

- `#define PSA_CIPHER_OPERATION_INIT { 0, 0, 0, 0, { 0 } }`

Typedefs

- `typedef struct psa_cipher_operation_s psa_cipher_operation_t`

Functions

- `psa_status_t psa_cipher_encrypt (mbedtls_svc_key_id_t key, psa_algorithm_t alg, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)`
- `psa_status_t psa_cipher_decrypt (mbedtls_svc_key_id_t key, psa_algorithm_t alg, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)`
- `psa_status_t psa_cipher_encrypt_setup (psa_cipher_operation_t *operation, mbedtls_svc_key_id_t key, psa_algorithm_t alg)`
- `psa_status_t psa_cipher_decrypt_setup (psa_cipher_operation_t *operation, mbedtls_svc_key_id_t key, psa_algorithm_t alg)`
- `psa_status_t psa_cipher_generate_iv (psa_cipher_operation_t *operation, uint8_t *iv, size_t iv_size, size_t *iv_length)`
- `psa_status_t psa_cipher_set_iv (psa_cipher_operation_t *operation, const uint8_t *iv, size_t iv_length)`
- `psa_status_t psa_cipher_update (psa_cipher_operation_t *operation, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)`
- `psa_status_t psa_cipher_finish (psa_cipher_operation_t *operation, uint8_t *output, size_t output_size, size_t *output_length)`
- `psa_status_t psa_cipher_abort (psa_cipher_operation_t *operation)`

6.9.1 Detailed Description

6.9.2 Macro Definition Documentation

6.9.2.1 PSA_CIPHER_OPERATION_INIT

```
#define PSA_CIPHER_OPERATION_INIT { 0, 0, 0, 0, { 0 } }
```

This macro returns a suitable initializer for a cipher operation object of type `psa_cipher_operation_t`.

Definition at line 150 of file `crypto_struct.h`.

6.9.3 Typedef Documentation

6.9.3.1 `psa_cipher_operation_t`

```
typedef struct psa_cipher_operation_s psa_cipher_operation_t
```

The type of the state data structure for multipart cipher operations.

Before calling any function on a cipher operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_cipher_operation_t operation;
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_cipher_operation_t operation = {0};
```
- Initialize the structure to the initializer `PSA_CIPHER_OPERATION_INIT`, for example:

```
psa_cipher_operation_t operation = PSA_CIPHER_OPERATION_INIT;
```
- Assign the result of the function `psa_cipher_operation_init()` to the structure, for example:

```
psa_cipher_operation_t operation;
operation = psa_cipher_operation_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 1901 of file `crypto.h`.

6.9.4 Function Documentation

6.9.4.1 `psa_cipher_abort()`

```
psa_status_t psa_cipher_abort (
    psa_cipher_operation_t * operation )
```

Abort a cipher operation.

Aborting an operation frees all associated resources except for the `operation` structure itself. Once aborted, the operation object can be reused for another operation by calling `psa_cipher_encrypt_setup()` or `psa_cipher_decrypt_setup()` again.

You may call this function any time after the operation object has been initialized as described in `psa_cipher_operation_t`.

In particular, calling `psa_cipher_abort()` after the operation has been terminated by a call to `psa_cipher_abort()` or `psa_cipher_finish()` is safe and has no effect.

Parameters

in, out	<i>operation</i>	Initialized cipher operation.
---------	------------------	-------------------------------

Return values

PSA_SUCCESS	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 353 of file `tfm_crypto_api.c`.

6.9.4.2 `psa_cipher_decrypt()`

```
psa_status_t psa_cipher_decrypt (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    uint8_t * output,
    size_t output_size,
    size_t * output_length )
```

Decrypt a message using a symmetric cipher.

This function decrypts a message encrypted with a symmetric cipher.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage PSA_KEY_USAGE_DECRYPT .
	<i>alg</i>	The cipher algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_CIPHER(alg) is true).
in	<i>input</i>	Buffer containing the message to decrypt. This consists of the IV followed by the ciphertext proper.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
out	<i>output</i>	Buffer where the plaintext is to be written.
	<i>output_size</i>	Size of the <i>output</i> buffer in bytes.
out	<i>output_length</i>	On success, the number of bytes that make up the output.

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	
PSA_ERROR_INVALID_ARGUMENT	<i>key</i> is not compatible with <i>alg</i> .
PSA_ERROR_NOT_SUPPORTED	<i>alg</i> is not supported or is not a cipher algorithm.
PSA_ERROR_BUFFER_TOO_SMALL	
PSA_ERROR_INSUFFICIENT_MEMORY	

Return values

PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1521 of file `tfm_crypto_api.c`.

6.9.4.3 `psa_cipher_decrypt_setup()`

```
psa_status_t psa_cipher_decrypt_setup (
    psa_cipher_operation_t * operation,
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg )
```

Set the key for a multipart symmetric decryption operation.

The sequence of operations to decrypt a message with a symmetric cipher is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_cipher_operation_t](#), e.g. [PSA_CIPHER_OPERATION_INIT](#).
3. Call [psa_cipher_decrypt_setup\(\)](#) to specify the algorithm and key.
4. Call [psa_cipher_set_iv\(\)](#) with the IV (initialization vector) for the decryption. If the IV is prepended to the ciphertext, you can call [psa_cipher_update\(\)](#) on a buffer containing the IV followed by the beginning of the message.
5. Call [psa_cipher_update\(\)](#) zero, one or more times, passing a fragment of the message each time.
6. Call [psa_cipher_finish\(\)](#).

If an error occurs at any step after a call to [psa_cipher_decrypt_setup\(\)](#), the operation will need to be reset by a call to [psa_cipher_abort\(\)](#). The application may call [psa_cipher_abort\(\)](#) at any time after the operation has been initialized.

After a successful call to [psa_cipher_decrypt_setup\(\)](#), the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to [psa_cipher_finish\(\)](#).
- A call to [psa_cipher_abort\(\)](#).

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for psa_cipher_operation_t and not yet in use.
	<i>key</i>	Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage PSA_KEY_USAGE_DECRYPT .
	<i>alg</i>	The cipher algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_CIPHER (<i>alg</i>) is true).

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	
PSA_ERROR_INVALID_ARGUMENT	<i>key</i> is not compatible with <i>alg</i> .
PSA_ERROR_NOT_SUPPORTED	<i>alg</i> is not supported or is not a cipher algorithm.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be inactive), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 304 of file `tfm_crypto_api.c`.

6.9.4.4 `psa_cipher_encrypt()`

```
psa_status_t psa_cipher_encrypt (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    uint8_t * output,
    size_t output_size,
    size_t * output_length )
```

Encrypt a message using a symmetric cipher.

This function encrypts a message with a random IV (initialization vector). Use the multipart operation interface with a [psa_cipher_operation_t](#) object to provide other forms of IV.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must allow the usage PSA_KEY_USAGE_ENCRYPT .
	<i>alg</i>	The cipher algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_CIPHER (<i>alg</i>) is true).

Parameters

in	<i>input</i>	Buffer containing the message to encrypt.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
out	<i>output</i>	Buffer where the output is to be written. The output contains the IV followed by the ciphertext proper.
	<i>output_size</i>	Size of the <i>output</i> buffer in bytes.
out	<i>output_length</i>	On success, the number of bytes that make up the output.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	key is not compatible with alg.
<i>PSA_ERROR_NOT_SUPPORTED</i>	alg is not supported or is not a cipher algorithm.
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1492 of file `tfm_crypto_api.c`.

6.9.4.5 `psa_cipher_encrypt_setup()`

```
psa_status_t psa_cipher_encrypt_setup (
    psa_cipher_operation_t * operation,
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg )
```

Set the key for a multipart symmetric encryption operation.

The sequence of operations to encrypt a message with a symmetric cipher is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [`psa\_cipher\_operation\_t`](#), e.g. [`PSA\_CIPHER\_OPERATION\_INIT`](#).
3. Call [`psa\_cipher\_encrypt\_setup\(\)`](#) to specify the algorithm and key.
4. Call either [`psa\_cipher\_generate\_iv\(\)`](#) or [`psa\_cipher\_set\_iv\(\)`](#) to generate or set the IV (initialization vector). You should use [`psa\_cipher\_generate\_iv\(\)`](#) unless the protocol you are implementing requires a specific IV value.

5. Call `psa_cipher_update()` zero, one or more times, passing a fragment of the message each time.
6. Call `psa_cipher_finish()`.

If an error occurs at any step after a call to `psa_cipher_encrypt_setup()`, the operation will need to be reset by a call to `psa_cipher_abort()`. The application may call `psa_cipher_abort()` at any time after the operation has been initialized.

After a successful call to `psa_cipher_encrypt_setup()`, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to `psa_cipher_finish()`.
- A call to `psa_cipher_abort()`.

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for <code>psa_cipher_operation_t</code> and not yet in use.
	<i>key</i>	Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage <code>PSA_KEY_USAGE_ENCRYPT</code> .
	<i>alg</i>	The cipher algorithm to compute (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_CIPHER(alg)</code> is true).

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_INVALID_HANDLE</code>	
<code>PSA_ERROR_NOT_PERMITTED</code>	
<code>PSA_ERROR_INVALID_ARGUMENT</code>	<code>key</code> is not compatible with <code>alg</code> .
<code>PSA_ERROR_NOT_SUPPORTED</code>	<code>alg</code> is not supported or is not a cipher algorithm.
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid (it must be inactive), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 283 of file `tfm_crypto_api.c`.

6.9.4.6 `psa_cipher_finish()`

```
psa_status_t psa_cipher_finish (
    psa_cipher_operation_t * operation,
    uint8_t * output,
```

```
size_t output_size,
size_t * output_length )
```

Finish encrypting or decrypting a message in a cipher operation.

The application must call [psa_cipher_encrypt_setup\(\)](#) or [psa_cipher_decrypt_setup\(\)](#) before calling this function. The choice of setup function determines whether this function encrypts or decrypts its input.

This function finishes the encryption or decryption of the message formed by concatenating the inputs passed to preceding calls to [psa_cipher_update\(\)](#).

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_cipher_abort\(\)](#).

Parameters

in, out	<i>operation</i>	Active cipher operation.
out	<i>output</i>	Buffer where the output is to be written.
	<i>output_size</i>	Size of the <code>output</code> buffer in bytes.
out	<i>output_length</i>	On success, the number of bytes that make up the returned output.

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INVALID_ARGUMENT	The total input size passed to this operation is not valid for this particular algorithm. For example, the algorithm is a based on block cipher and requires a whole number of blocks, but the total input size is not a multiple of the block size.
PSA_ERROR_INVALID_PADDING	This is a decryption operation for an algorithm that includes padding, and the ciphertext does not contain valid padding.
PSA_ERROR_BUFFER_TOO_SMALL	The size of the <code>output</code> buffer is too small.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be active, with an IV set if required for the algorithm), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 370 of file `tfm_crypto_api.c`.

6.9.4.7 `psa_cipher_generate_iv()`

```
psa_status_t psa_cipher_generate_iv (
    psa_cipher_operation_t * operation,
    uint8_t * iv,
```

```
size_t iv_size,
size_t * iv_length )
```

Generate an IV for a symmetric encryption operation.

This function generates a random IV (initialization vector), nonce or initial counter value for the encryption operation as appropriate for the chosen algorithm, key type and key size.

The application must call [psa_cipher_encrypt_setup\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_cipher_abort\(\)](#).

Parameters

in, out	<i>operation</i>	Active cipher operation.
out	<i>iv</i>	Buffer where the generated IV is to be written.
	<i>iv_size</i>	Size of the <i>iv</i> buffer in bytes.
out	<i>iv_length</i>	On success, the number of bytes of the generated IV.

Return values

PSA_SUCCESS	Success.
PSA_ERROR_BUFFER_TOO_SMALL	The size of the <i>iv</i> buffer is too small.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be active, with no IV set), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

6.9.4.8 psa_cipher_set_iv()

```
psa_status_t psa_cipher_set_iv (
    psa_cipher_operation_t * operation,
    const uint8_t * iv,
    size_t iv_length )
```

Set the IV for a symmetric encryption or decryption operation.

This function sets the IV (initialization vector), nonce or initial counter value for the encryption or decryption operation.

The application must call [psa_cipher_encrypt_setup\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_cipher_abort\(\)](#).

Note

When encrypting, applications should use [psa_cipher_generate_iv\(\)](#) instead of this function, unless implementing a protocol that requires a non-random IV.

Parameters

<i>in, out</i>	<i>operation</i>	Active cipher operation.
<i>in</i>	<i>iv</i>	Buffer containing the IV to use.
	<i>iv_length</i>	Size of the IV in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The size of <i>iv</i> is not acceptable for the chosen algorithm, or the chosen algorithm does not use an IV.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be an active cipher encrypt operation, with no IV set), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.9.4.9 `psa_cipher_update()`

```
psa_status_t psa_cipher_update (
    psa_cipher_operation_t * operation,
    const uint8_t * input,
    size_t input_length,
    uint8_t * output,
    size_t output_size,
    size_t * output_length )
```

Encrypt or decrypt a message fragment in an active cipher operation.

Before calling this function, you must:

1. Call either [`psa\_cipher\_encrypt\_setup\(\)`](#) or [`psa\_cipher\_decrypt\_setup\(\)`](#). The choice of setup function determines whether this function encrypts or decrypts its input.
2. If the algorithm requires an IV, call [`psa\_cipher\_generate\_iv\(\)`](#) (recommended when encrypting) or [`psa\_cipher\_set\_iv\(\)`](#).

If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_cipher\_abort\(\)`](#).

Parameters

<i>in, out</i>	<i>operation</i>	Active cipher operation.
<i>in</i>	<i>input</i>	Buffer containing the message fragment to encrypt or decrypt.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
<i>out</i>	<i>output</i>	Buffer where the output is to be written.
	<i>output_size</i>	Size of the <i>output</i> buffer in bytes.
<i>out</i>	<i>output_length</i>	On success, the number of bytes that make up the returned output.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	The size of the <code>output</code> buffer is too small.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active, with an IV set if required for the algorithm), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.10 Authenticated encryption with associated data (AEAD)

Macros

- `#define PSA_AEAD_OPERATION_INIT { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, { 0 } }`

Typedefs

- `typedef struct psa_aead_operation_s psa_aead_operation_t`

Functions

- `psa_status_t psa_aead_encrypt (mbedtls_svc_key_id_t key, psa_algorithm_t alg, const uint8_t *nonce, size_t nonce_length, const uint8_t *additional_data, size_t additional_data_length, const uint8_t *plaintext, size_t plaintext_length, uint8_t *ciphertext, size_t ciphertext_size, size_t *ciphertext_length)`
- `psa_status_t psa_aead_decrypt (mbedtls_svc_key_id_t key, psa_algorithm_t alg, const uint8_t *nonce, size_t nonce_length, const uint8_t *additional_data, size_t additional_data_length, const uint8_t *ciphertext, size_t ciphertext_length, uint8_t *plaintext, size_t plaintext_size, size_t *plaintext_length)`
- `psa_status_t psa_aead_encrypt_setup (psa_aead_operation_t *operation, mbedtls_svc_key_id_t key, psa_algorithm_t alg)`
- `psa_status_t psa_aead_decrypt_setup (psa_aead_operation_t *operation, mbedtls_svc_key_id_t key, psa_algorithm_t alg)`
- `psa_status_t psa_aead_generate_nonce (psa_aead_operation_t *operation, uint8_t *nonce, size_t nonce_size, size_t *nonce_length)`
- `psa_status_t psa_aead_set_nonce (psa_aead_operation_t *operation, const uint8_t *nonce, size_t nonce_size, size_t *nonce_length)`
- `psa_status_t psa_aead_set_lengths (psa_aead_operation_t *operation, size_t ad_length, size_t plaintext_size, size_t *plaintext_length)`
- `psa_status_t psa_aead_update_ad (psa_aead_operation_t *operation, const uint8_t *input, size_t input_size, size_t *input_length)`
- `psa_status_t psa_aead_update (psa_aead_operation_t *operation, const uint8_t *input, size_t input_size, size_t *input_length, uint8_t *output, size_t output_size, size_t *output_length)`
- `psa_status_t psa_aead_finish (psa_aead_operation_t *operation, uint8_t *ciphertext, size_t ciphertext_size, size_t *ciphertext_length, uint8_t *tag, size_t tag_size, size_t *tag_length)`
- `psa_status_t psa_aead_verify (psa_aead_operation_t *operation, uint8_t *plaintext, size_t plaintext_size, size_t *plaintext_length, const uint8_t *tag, size_t tag_length)`
- `psa_status_t psa_aead_abort (psa_aead_operation_t *operation)`

6.10.1 Detailed Description

6.10.2 Macro Definition Documentation

6.10.2.1 PSA_AEAD_OPERATION_INIT

```
#define PSA_AEAD_OPERATION_INIT { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, { 0 } }
```

This macro returns a suitable initializer for an AEAD operation object of type `psa_aead_operation_t`.

Definition at line 231 of file `crypto_struct.h`.

6.10.3 Typedef Documentation

6.10.3.1 `psa_aead_operation_t`

```
typedef struct psa_aead_operation_s psa_aead_operation_t
```

The type of the state data structure for multipart AEAD operations.

Before calling any function on an AEAD operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_aead_operation_t operation;  
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_aead_operation_t operation = {0};
```
- Initialize the structure to the initializer `PSA_AEAD_OPERATION_INIT`, for example:

```
psa_aead_operation_t operation = PSA_AEAD_OPERATION_INIT;
```
- Assign the result of the function `psa_aead_operation_init()` to the structure, for example:

```
psa_aead_operation_t operation;  
operation = psa_aead_operation_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 2419 of file `crypto.h`.

6.10.4 Function Documentation

6.10.4.1 `psa_aead_abort()`

```
psa_status_t psa_aead_abort (  
    psa_aead_operation_t * operation )
```

Abort an AEAD operation.

Aborting an operation frees all associated resources except for the `operation` structure itself. Once aborted, the operation object can be reused for another operation by calling `psa_aead_encrypt_setup()` or `psa_aead_decrypt_setup()` again.

You may call this function any time after the operation object has been initialized as described in `psa_aead_operation_t`.

In particular, calling `psa_aead_abort()` after the operation has been terminated by a call to `psa_aead_abort()`, `psa_aead_finish()` or `psa_aead_verify()` is safe and has no effect.

Parameters

in, out	<i>operation</i>	Initialized AEAD operation.
---------	------------------	-----------------------------

Return values

<i>PSA_SUCCESS</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1090 of file `tfm_crypto_api.c`.

6.10.4.2 `psa_aead_decrypt()`

```
psa_status_t psa_aead_decrypt (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * nonce,
    size_t nonce_length,
    const uint8_t * additional_data,
    size_t additional_data_length,
    const uint8_t * ciphertext,
    size_t ciphertext_length,
    uint8_t * plaintext,
    size_t plaintext_size,
    size_t * plaintext_length )
```

Process an authenticated decryption operation.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must allow the usage <code>PSA_KEY_USAGE_DECRYPT</code> .
	<i>alg</i>	The AEAD algorithm to compute (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_AEAD(alg)</code> is true).
in	<i>nonce</i>	Nonce or IV to use.
	<i>nonce_length</i>	Size of the <code>nonce</code> buffer in bytes.
in	<i>additional_data</i>	Additional data that has been authenticated but not encrypted.
	<i>additional_data_length</i>	Size of <code>additional_data</code> in bytes.
in	<i>ciphertext</i>	Data that has been authenticated and encrypted. For algorithms where the encrypted data and the authentication tag are defined as separate inputs, the buffer must contain the encrypted data followed by the authentication tag.
	<i>ciphertext_length</i>	Size of <code>ciphertext</code> in bytes.
out	<i>plaintext</i>	Output buffer for the decrypted data.

Parameters

	<i>plaintext_size</i>	Size of the <code>plaintext</code> buffer in bytes. This must be appropriate for the selected algorithm and key: - A sufficient output size is <code>PSA_AEAD_DECRYPT_OUTPUT_SIZE(key_type, alg, ciphertext_length)</code> where <code>key_type</code> is the type of key. <code>PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE(ciphertext_length)</code> evaluates to the maximum plaintext size of any supported AEAD decryption.
out	<i>plaintext_length</i>	On success, the size of the output in the <code>plaintext</code> buffer.

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_INVALID_HANDLE</code>	
<code>PSA_ERROR_INVALID_SIGNATURE</code>	The ciphertext is not authentic.
<code>PSA_ERROR_NOT_PERMITTED</code>	
<code>PSA_ERROR_INVALID_ARGUMENT</code>	key is not compatible with <code>alg</code> .
<code>PSA_ERROR_NOT_SUPPORTED</code>	<code>alg</code> is not supported or is not an AEAD algorithm.
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_BUFFER_TOO_SMALL</code>	<code>plaintext_size</code> is too small. <code>PSA_AEAD_DECRYPT_OUTPUT_SIZE(key_type, alg, ciphertext_length)</code> or <code>PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE(ciphertext_length)</code> can be used to determine the required buffer size.
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 752 of file `tfm_crypto_api.c`.

Here is the call graph for this function:



6.10.4.3 `psa_aead_decrypt_setup()`

```
psa_status_t psa_aead_decrypt_setup (
    psa_aead_operation_t * operation,
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg )
```

Set the key for a multipart authenticated decryption operation.

The sequence of operations to decrypt a message with authentication is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `psa_aead_operation_t`, e.g. `PSA_AEAD_OPERATION_INIT`.
3. Call `psa_aead_decrypt_setup()` to specify the algorithm and key.
4. If needed, call `psa_aead_set_lengths()` to specify the length of the inputs to the subsequent calls to `psa_aead_update_ad()` and `psa_aead_update()`. See the documentation of `psa_aead_set_lengths()` for details.
5. Call `psa_aead_set_nonce()` with the nonce for the decryption.
6. Call `psa_aead_update_ad()` zero, one or more times, passing a fragment of the non-encrypted additional authenticated data each time.
7. Call `psa_aead_update()` zero, one or more times, passing a fragment of the ciphertext to decrypt each time.
8. Call `psa_aead_verify()`.

If an error occurs at any step after a call to `psa_aead_decrypt_setup()`, the operation will need to be reset by a call to `psa_aead_abort()`. The application may call `psa_aead_abort()` at any time after the operation has been initialized.

After a successful call to `psa_aead_decrypt_setup()`, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to `psa_aead_verify()`.
- A call to `psa_aead_abort()`.

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for <code>psa_aead_operation_t</code> and not yet in use.
	<i>key</i>	Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage <code>PSA_KEY_USAGE_DECRYPT</code> .
	<i>alg</i>	The AEAD algorithm to compute (PSA_ALG_XXX value such that <code>PSA_ALG_IS_AEAD(alg)</code> is true).

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_INVALID_HANDLE</code>	
<code>PSA_ERROR_NOT_PERMITTED</code>	
<code>PSA_ERROR_INVALID_ARGUMENT</code>	key is not compatible with alg.

Return values

PSA_ERROR_NOT_SUPPORTED	<code>alg</code> is not supported or is not an AEAD algorithm.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be inactive), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 836 of file `tfm_crypto_api.c`.

6.10.4.4 `psa_aead_encrypt()`

```
psa_status_t psa_aead_encrypt (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * nonce,
    size_t nonce_length,
    const uint8_t * additional_data,
    size_t additional_data_length,
    const uint8_t * plaintext,
    size_t plaintext_length,
    uint8_t * ciphertext,
    size_t ciphertext_size,
    size_t * ciphertext_length )
```

Process an authenticated encryption operation.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must allow the usage PSA_KEY_USAGE_ENCRYPT .
	<i>alg</i>	The AEAD algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_AEAD(alg) is true).
in	<i>nonce</i>	Nonce or IV to use.
	<i>nonce_length</i>	Size of the <i>nonce</i> buffer in bytes.
in	<i>additional_data</i>	Additional data that will be authenticated but not encrypted.
	<i>additional_data_length</i>	Size of <i>additional_data</i> in bytes.
in	<i>plaintext</i>	Data that will be authenticated and encrypted.
	<i>plaintext_length</i>	Size of <i>plaintext</i> in bytes.
out	<i>ciphertext</i>	Output buffer for the authenticated and encrypted data. The additional data is not part of this output. For algorithms where the encrypted data and the authentication tag are defined as separate outputs, the authentication tag is appended to the encrypted data.

Parameters

	<i>ciphertext_size</i>	Size of the <code>ciphertext</code> buffer in bytes. This must be appropriate for the selected algorithm and key: - A sufficient output size is <code>PSA_AEAD_ENCRYPT_OUTPUT_SIZE(key_type, alg, plaintext_length)</code> where <code>key_type</code> is the type of key. <code>PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE(plaintext_length)</code> evaluates to the maximum ciphertext size of any supported AEAD encryption.
out	<i>ciphertext_length</i>	On success, the size of the output in the <code>ciphertext</code> buffer.

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_INVALID_HANDLE</code>	
<code>PSA_ERROR_NOT_PERMITTED</code>	
<code>PSA_ERROR_INVALID_ARGUMENT</code>	key is not compatible with <code>alg</code> .
<code>PSA_ERROR_NOT_SUPPORTED</code>	<code>alg</code> is not supported or is not an AEAD algorithm.
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_BUFFER_TOO_SMALL</code>	<code>ciphertext_size</code> is too small. <code>PSA_AEAD_ENCRYPT_OUTPUT_SIZE(key_type, alg, plaintext_length)</code> or <code>PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE(plaintext_length)</code> can be used to determine the required buffer size.
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 691 of file `tfm_crypto_api.c`.

Here is the call graph for this function:



6.10.4.5 `psa_aead_encrypt_setup()`

```
psa_status_t psa_aead_encrypt_setup (
    psa_aead_operation_t * operation,
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg )
```

Set the key for a multipart authenticated encryption operation.

The sequence of operations to encrypt a message with authentication is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `psa_aead_operation_t`, e.g. `PSA_AEAD_OPERATION_INIT`.
3. Call `psa_aead_encrypt_setup()` to specify the algorithm and key.
4. If needed, call `psa_aead_set_lengths()` to specify the length of the inputs to the subsequent calls to `psa_aead_update_ad()` and `psa_aead_update()`. See the documentation of `psa_aead_set_lengths()` for details.
5. Call either `psa_aead_generate_nonce()` or `psa_aead_set_nonce()` to generate or set the nonce. You should use `psa_aead_generate_nonce()` unless the protocol you are implementing requires a specific nonce value.
6. Call `psa_aead_update_ad()` zero, one or more times, passing a fragment of the non-encrypted additional authenticated data each time.
7. Call `psa_aead_update()` zero, one or more times, passing a fragment of the message to encrypt each time.
8. Call `psa_aead_finish()`.

If an error occurs at any step after a call to `psa_aead_encrypt_setup()`, the operation will need to be reset by a call to `psa_aead_abort()`. The application may call `psa_aead_abort()` at any time after the operation has been initialized.

After a successful call to `psa_aead_encrypt_setup()`, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to `psa_aead_finish()`.
- A call to `psa_aead_abort()`.

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for <code>psa_aead_operation_t</code> and not yet in use.
	<i>key</i>	Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage <code>PSA_KEY_USAGE_ENCRYPT</code> .
	<i>alg</i>	The AEAD algorithm to compute (PSA_ALG_XXX value such that <code>PSA_ALG_IS_AEAD(alg)</code> is true).

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid (it must be inactive), or the library has not been previously initialized by <code>psa_crypto_init()</code> .

Return values

<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	key is not compatible with alg.
<i>PSA_ERROR_NOT_SUPPORTED</i>	alg is not supported or is not an AEAD algorithm.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 813 of file `tfm_crypto_api.c`.

6.10.4.6 `psa_aead_finish()`

```
psa_status_t psa_aead_finish (
    psa_aead_operation_t * operation,
    uint8_t * ciphertext,
    size_t ciphertext_size,
    size_t * ciphertext_length,
    uint8_t * tag,
    size_t tag_size,
    size_t * tag_length )
```

Finish encrypting a message in an AEAD operation.

The operation must have been set up with [`psa\_aead\_encrypt\_setup\(\)`](#).

This function finishes the authentication of the additional data formed by concatenating the inputs passed to preceding calls to [`psa\_aead\_update\_ad\(\)`](#) with the plaintext formed by concatenating the inputs passed to preceding calls to [`psa\_aead\_update\(\)`](#).

This function has two output buffers:

- `ciphertext` contains trailing ciphertext that was buffered from preceding calls to [`psa\_aead\_update\(\)`](#).
- `tag` contains the authentication tag.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_aead\_abort\(\)`](#).

Parameters

in, out	<i>operation</i>	Active AEAD operation.
out	<i>ciphertext</i>	Buffer where the last part of the ciphertext is to be written.

Parameters

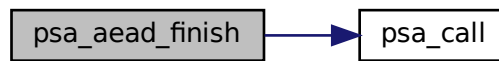
	<i>ciphertext_size</i>	Size of the <code>ciphertext</code> buffer in bytes. This must be appropriate for the selected algorithm and key: - A sufficient output size is <code>PSA_AEAD_FINISH_OUTPUT_SIZE(key_type, alg)</code> where <code>key_type</code> is the type of key and <code>alg</code> is the algorithm that were used to set up the operation. <code>PSA_AEAD_FINISH_OUTPUT_MAX_SIZE</code> evaluates to the maximum output size of any supported AEAD algorithm.
out	<i>ciphertext_length</i>	On success, the number of bytes of returned ciphertext.
out	<i>tag</i>	Buffer where the authentication tag is to be written.
	<i>tag_size</i>	Size of the <code>tag</code> buffer in bytes. This must be appropriate for the selected algorithm and key: - The exact tag size is <code>PSA_AEAD_TAG_LENGTH(key_type, key_bits, alg)</code> where <code>key_type</code> and <code>key_bits</code> are the type and bit-size of the key, and <code>alg</code> is the algorithm that were used in the call to <code>psa_aead_encrypt_setup()</code>. <code>PSA_AEAD_TAG_MAX_SIZE</code> evaluates to the maximum tag size of any supported AEAD algorithm.
out	<i>tag_length</i>	On success, the number of bytes that make up the returned tag.

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_BUFFER_TOO_SMALL</code>	The size of the <code>ciphertext</code> or <code>tag</code> buffer is too small. <code>PSA_AEAD_FINISH_OUTPUT_SIZE(key_type, alg)</code> or <code>PSA_AEAD_FINISH_OUTPUT_MAX_SIZE</code> can be used to determine the required <code>ciphertext</code> buffer size. <code>PSA_AEAD_TAG_LENGTH(key_type, key_bits, alg)</code> or <code>PSA_AEAD_TAG_MAX_SIZE</code> can be used to determine the required <code>tag</code> buffer size.
<code>PSA_ERROR_INVALID_ARGUMENT</code>	The total length of input to <code>psa_aead_update_ad()</code> so far is less than the additional data length that was previously specified with <code>psa_aead_set_lengths()</code> , or the total length of input to <code>psa_aead_update()</code> so far is less than the plaintext length that was previously specified with <code>psa_aead_set_lengths()</code> .
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid (it must be an active encryption operation with a nonce set), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 990 of file `tfm_crypto_api.c`.

Here is the call graph for this function:



6.10.4.7 `psa_aead_generate_nonce()`

```

psa_status_t psa_aead_generate_nonce (
    psa_aead_operation_t * operation,
    uint8_t * nonce,
    size_t nonce_size,
    size_t * nonce_length )
  
```

Generate a random nonce for an authenticated encryption operation.

This function generates a random nonce for the authenticated encryption operation with an appropriate size for the chosen algorithm, key type and key size.

The application must call [psa_aead_encrypt_setup\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Parameters

in, out	<i>operation</i>	Active AEAD operation.
out	<i>nonce</i>	Buffer where the generated nonce is to be written.
	<i>nonce_size</i>	Size of the <code>nonce</code> buffer in bytes.
out	<i>nonce_length</i>	On success, the number of bytes of the generated nonce.

Return values

PSA_SUCCESS	Success.
PSA_ERROR_BUFFER_TOO_SMALL	The size of the <code>nonce</code> buffer is too small.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	

Return values

<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be an active aead encrypt operation, with no nonce set), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.
--	--

Definition at line 859 of file `tfm_crypto_api.c`.

6.10.4.8 `psa_aead_set_lengths()`

```
psa_status_t psa_aead_set_lengths (
    psa_aead_operation_t * operation,
    size_t ad_length,
    size_t plaintext_length )
```

Declare the lengths of the message and additional data for AEAD.

The application must call this function before calling [`psa\_aead\_update\_ad\(\)`](#) or [`psa\_aead\_update\(\)`](#) if the algorithm for the operation requires it. If the algorithm does not require it, calling this function is optional, but if this function is called then the implementation must enforce the lengths.

You may call this function before or after setting the nonce with [`psa\_aead\_set\_nonce\(\)`](#) or [`psa\_aead\_generate\_nonce\(\)`](#).

- For [`PSA\_ALG\_CCM`](#), calling this function is required.
- For the other AEAD algorithms defined in this specification, calling this function is not required.
- For vendor-defined algorithm, refer to the vendor documentation.

If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_aead\_abort\(\)`](#).

Parameters

<i>in, out</i>	<i>operation</i>	Active AEAD operation.
	<i>ad_length</i>	Size of the non-encrypted additional authenticated data in bytes.
	<i>plaintext_length</i>	Size of the plaintext to encrypt in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	At least one of the lengths is not acceptable for the chosen algorithm.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	

Return values

<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active, and <i>psa_aead_update_ad()</i> and <i>psa_aead_update()</i> must not have been called yet), or the library has not been previously initialized by <i>psa_crypto_init()</i> . It is implementation-dependent whether a failure to initialize results in this error code.
--	---

Definition at line 902 of file `tfm_crypto_api.c`.

6.10.4.9 `psa_aead_set_nonce()`

```
psa_status_t psa_aead_set_nonce (
    psa_aead_operation_t * operation,
    const uint8_t * nonce,
    size_t nonce_length )
```

Set the nonce for an authenticated encryption or decryption operation.

This function sets the nonce for the authenticated encryption or decryption operation.

The application must call [*psa_aead_encrypt_setup\(\)*](#) or [*psa_aead_decrypt_setup\(\)*](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [*psa_aead_abort\(\)*](#).

Note

When encrypting, applications should use [*psa_aead_generate_nonce\(\)*](#) instead of this function, unless implementing a protocol that requires a non-random IV.

Parameters

in, out	<i>operation</i>	Active AEAD operation.
in	<i>nonce</i>	Buffer containing the nonce to use.
	<i>nonce_length</i>	Size of the nonce in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The size of <code>nonce</code> is not acceptable for the chosen algorithm.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active, with no nonce set), or the library has not been previously initialized by <i>psa_crypto_init()</i> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 883 of file tfm_crypto_api.c.

6.10.4.10 `psa_aead_update()`

```
psa_status_t psa_aead_update (
    psa_aead_operation_t * operation,
    const uint8_t * input,
    size_t input_length,
    uint8_t * output,
    size_t output_size,
    size_t * output_length )
```

Encrypt or decrypt a message fragment in an active AEAD operation.

Before calling this function, you must:

1. Call either `psa_aead_encrypt_setup()` or `psa_aead_decrypt_setup()`. The choice of setup function determines whether this function encrypts or decrypts its input.
2. Set the nonce with `psa_aead_generate_nonce()` or `psa_aead_set_nonce()`.
3. Call `psa_aead_update_ad()` to pass all the additional data.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_aead_abort()`.

Warning

When decrypting, until `psa_aead_verify()` has returned `PSA_SUCCESS`, there is no guarantee that the input is valid. Therefore, until you have called `psa_aead_verify()` and it has returned `PSA_SUCCESS`:

- Do not use the output in any way other than storing it in a confidential location. If you take any action that depends on the tentative decrypted data, this action will need to be undone if the input turns out not to be valid. Furthermore, if an adversary can observe that this action took place (for example through timing), they may be able to use this fact as an oracle to decrypt any message encrypted with the same key.
- In particular, do not copy the output anywhere but to a memory or storage space that you have exclusive access to.

This function does not require the input to be aligned to any particular block boundary. If the implementation can only process a whole block at a time, it must consume all the input provided, but it may delay the end of the corresponding output until a subsequent call to `psa_aead_update()`, `psa_aead_finish()` or `psa_aead_verify()` provides sufficient input. The amount of data that can be delayed in this way is bounded by `PSA_AEAD_UPDATE_OUTPUT_SIZE`.

Parameters

<code>in, out</code>	<i>operation</i>	Active AEAD operation.
<code>in</code>	<i>input</i>	Buffer containing the message fragment to encrypt or decrypt.
	<i>input_length</i>	Size of the <code>input</code> buffer in bytes.
<code>out</code>	<i>output</i>	Buffer where the output is to be written.

Parameters

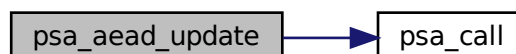
	<i>output_size</i>	Size of the <code>output</code> buffer in bytes. This must be appropriate for the selected algorithm and key: - A sufficient output size is <code>PSA_AEAD_UPDATE_OUTPUT_SIZE(key_type, alg, input_length)</code> where <code>key_type</code> is the type of key and <code>alg</code> is the algorithm that were used to set up the operation. <code>PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE(input_length)</code> evaluates to the maximum output size of any supported AEAD algorithm.
<code>out</code>	<i>output_length</i>	On success, the number of bytes that make up the returned output.

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_BUFFER_TOO_SMALL</code>	The size of the <code>output</code> buffer is too small. <code>PSA_AEAD_UPDATE_OUTPUT_SIZE(key_type, alg, input_length)</code> or <code>PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE(input_length)</code> can be used to determine the required buffer size.
<code>PSA_ERROR_INVALID_ARGUMENT</code>	The total length of input to <code>psa_aead_update_ad()</code> so far is less than the additional data length that was previously specified with <code>psa_aead_set_lengths()</code> , or the total input length overflows the plaintext length that was previously specified with <code>psa_aead_set_lengths()</code> .
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid (it must be active, have a nonce set, and have lengths set if required by the algorithm), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 952 of file `tfm_crypto_api.c`.

Here is the call graph for this function:



6.10.4.11 `psa_aead_update_ad()`

```
psa_status_t psa_aead_update_ad (
    psa_aead_operation_t * operation,
    const uint8_t * input,
    size_t input_length )
```

Pass additional data to an active AEAD operation.

Additional data is authenticated, but not encrypted.

You may call this function multiple times to pass successive fragments of the additional data. You may not call this function after passing data to encrypt or decrypt with `psa_aead_update()`.

Before calling this function, you must:

1. Call either `psa_aead_encrypt_setup()` or `psa_aead_decrypt_setup()`.
2. Set the nonce with `psa_aead_generate_nonce()` or `psa_aead_set_nonce()`.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_aead_abort()`.

Warning

When decrypting, until `psa_aead_verify()` has returned `PSA_SUCCESS`, there is no guarantee that the input is valid. Therefore, until you have called `psa_aead_verify()` and it has returned `PSA_SUCCESS`, treat the input as untrusted and prepare to undo any action that depends on the input if `psa_aead_verify()` returns an error status.

Parameters

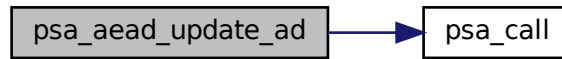
<code>in, out</code>	<code>operation</code>	Active AEAD operation.
<code>in</code>	<code>input</code>	Buffer containing the fragment of additional data.
	<code>input_length</code>	Size of the <code>input</code> buffer in bytes.

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_INVALID_ARGUMENT</code>	The total input length overflows the additional data length that was previously specified with <code>psa_aead_set_lengths()</code> .
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid (it must be active, have a nonce set, have lengths set if required by the algorithm, and <code>psa_aead_update()</code> must not have been called yet), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 922 of file `tfm_crypto_api.c`.

Here is the call graph for this function:



6.10.4.12 `psa_aead_verify()`

```
psa_status_t psa_aead_verify (
    psa_aead_operation_t * operation,
    uint8_t * plaintext,
    size_t plaintext_size,
    size_t * plaintext_length,
    const uint8_t * tag,
    size_t tag_length )
```

Finish authenticating and decrypting a message in an AEAD operation.

The operation must have been set up with [psa_aead_decrypt_setup\(\)](#).

This function finishes the authenticated decryption of the message components:

- The additional data consisting of the concatenation of the inputs passed to preceding calls to [psa_aead_update_ad\(\)](#).
- The ciphertext consisting of the concatenation of the inputs passed to preceding calls to [psa_aead_update\(\)](#).
- The tag passed to this function call.

If the authentication tag is correct, this function outputs any remaining plaintext and reports success. If the authentication tag is not correct, this function returns [PSA_ERROR_INVALID_SIGNATURE](#).

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Parameters

Note

Implementations shall make the best effort to ensure that the comparison between the actual tag and the expected tag is performed in constant time.

Parameters

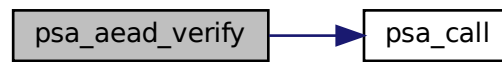
in, out	<i>operation</i>	Active AEAD operation.
out	<i>plaintext</i>	Buffer where the last part of the plaintext is to be written. This is the remaining data from previous calls to <code>psa_aead_update()</code> that could not be processed until the end of the input.
	<i>plaintext_size</i>	Size of the <code>plaintext</code> buffer in bytes. This must be appropriate for the selected algorithm and key: - A sufficient output size is <code>PSA_AEAD_VERIFY_OUTPUT_SIZE(key_type, alg)</code> where <code>key_type</code> is the type of key and <code>alg</code> is the algorithm that were used to set up the operation. <code>PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE</code> evaluates to the maximum output size of any supported AEAD algorithm.
out	<i>plaintext_length</i>	On success, the number of bytes of returned plaintext.
in	<i>tag</i>	Buffer containing the authentication tag.
	<i>tag_length</i>	Size of the <code>tag</code> buffer in bytes.

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_INVALID_SIGNATURE</code>	The calculations were successful, but the authentication tag is not correct.
<code>PSA_ERROR_BUFFER_TOO_SMALL</code>	The size of the <code>plaintext</code> buffer is too small. <code>PSA_AEAD_VERIFY_OUTPUT_SIZE(key_type, alg)</code> or <code>PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE</code> can be used to determine the required buffer size.
<code>PSA_ERROR_INVALID_ARGUMENT</code>	The total length of input to <code>psa_aead_update_ad()</code> so far is less than the additional data length that was previously specified with <code>psa_aead_set_lengths()</code> , or the total length of input to <code>psa_aead_update()</code> so far is less than the plaintext length that was previously specified with <code>psa_aead_set_lengths()</code> .
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid (it must be an active decryption operation with a nonce set), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1042 of file tfm_crypto_api.c.

Here is the call graph for this function:



6.11 Asymmetric cryptography

Functions

- `psa_status_t psa_sign_message` (`mbedtls_svc_key_id_t` key, `psa_algorithm_t` alg, `const uint8_t *input`, `size_t input_length`, `uint8_t *signature`, `size_t signature_size`, `size_t *signature_length`)
Sign a message with a private key. For hash-and-sign algorithms, this includes the hashing step.
- `psa_status_t psa_verify_message` (`mbedtls_svc_key_id_t` key, `psa_algorithm_t` alg, `const uint8_t *input`, `size_t input_length`, `const uint8_t *signature`, `size_t signature_length`)
Verify the signature of a message with a public key, using a hash-and-sign verification algorithm.
- `psa_status_t psa_unwrap_key` (`const psa_key_attributes_t *attributes`, `psa_key_id_t` wrapping_key, `psa_algorithm_t` alg, `const uint8_t *data`, `size_t data_length`, `psa_key_id_t *key`)
Unwrap and import a key using a specified wrapping key.
- `psa_status_t psa_sign_hash` (`mbedtls_svc_key_id_t` key, `psa_algorithm_t` alg, `const uint8_t *hash`, `size_t hash_length`, `uint8_t *signature`, `size_t signature_size`, `size_t *signature_length`)
Sign a hash or short message with a private key.
- `psa_status_t psa_verify_hash` (`mbedtls_svc_key_id_t` key, `psa_algorithm_t` alg, `const uint8_t *hash`, `size_t hash_length`, `const uint8_t *signature`, `size_t signature_length`)
Verify the signature of a hash or short message using a public key.
- `psa_status_t psa_asymmetric_encrypt` (`mbedtls_svc_key_id_t` key, `psa_algorithm_t` alg, `const uint8_t *input`, `size_t input_length`, `const uint8_t *salt`, `size_t salt_length`, `uint8_t *output`, `size_t output_size`, `size_t *output_length`)
Encrypt a short message with a public key.
- `psa_status_t psa_asymmetric_decrypt` (`mbedtls_svc_key_id_t` key, `psa_algorithm_t` alg, `const uint8_t *input`, `size_t input_length`, `const uint8_t *salt`, `size_t salt_length`, `uint8_t *output`, `size_t output_size`, `size_t *output_length`)
Decrypt a short message with a private key.

6.11.1 Detailed Description

6.11.2 Function Documentation

6.11.2.1 `psa_asymmetric_decrypt()`

```
psa_status_t psa_asymmetric_decrypt (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    const uint8_t * salt,
    size_t salt_length,
    uint8_t * output,
    size_t output_size,
    size_t * output_length )
```

Decrypt a short message with a private key.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must be an asymmetric key pair. It must allow the usage PSA_KEY_USAGE_DECRYPT .
	<i>alg</i>	An asymmetric encryption algorithm that is compatible with the type of <i>key</i> .
in	<i>input</i>	The message to decrypt.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
in	<i>salt</i>	A salt or label, if supported by the encryption algorithm. If the algorithm does not support a salt, pass <code>NULL</code> . If the algorithm supports an optional salt and you do not want to pass a salt, pass <code>NULL</code> .

- For [PSA_ALG_RSA_PKCS1V15_CRYPT](#), no salt is supported.

Parameters

	<i>salt_length</i>	Size of the <i>salt</i> buffer in bytes. If <i>salt</i> is <code>NULL</code> , pass 0.
out	<i>output</i>	Buffer where the decrypted message is to be written.
	<i>output_size</i>	Size of the <i>output</i> buffer in bytes.
out	<i>output_length</i>	On success, the number of bytes that make up the returned output.

Return values

PSA_SUCCESS	
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	
PSA_ERROR_BUFFER_TOO_SMALL	The size of the <i>output</i> buffer is too small. You can determine a sufficient buffer size by calling PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE (<i>key</i> ↔ <i>_type</i> , <i>key_bits</i> , <i>alg</i>) where <i>key_type</i> and <i>key_bits</i> are the type and bit-size respectively of <i>key</i> .
PSA_ERROR_NOT_SUPPORTED	
PSA_ERROR_INVALID_ARGUMENT	
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_INSUFFICIENT_ENTROPY	
PSA_ERROR_INVALID_PADDING	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1255 of file `tfm_crypto_api.c`.

Here is the call graph for this function:



6.11.2.2 psa_asymmetric_encrypt()

```

psa_status_t psa_asymmetric_encrypt (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    const uint8_t * salt,
    size_t salt_length,
    uint8_t * output,
    size_t output_size,
    size_t * output_length )
  
```

Encrypt a short message with a public key.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. It must allow the usage PSA_KEY_USAGE_ENCRYPT .
	<i>alg</i>	An asymmetric encryption algorithm that is compatible with the type of <i>key</i> .
in	<i>input</i>	The message to encrypt.
	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
in	<i>salt</i>	A salt or label, if supported by the encryption algorithm. If the algorithm does not support a salt, pass <code>NULL</code> . If the algorithm supports an optional salt and you do not want to pass a salt, pass <code>NULL</code> .

- For [PSA_ALG_RSA_PKCS1V15_CRYPT](#), no salt is supported.

Parameters

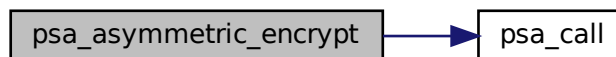
	<i>salt_length</i>	Size of the <i>salt</i> buffer in bytes. If <i>salt</i> is <code>NULL</code> , pass 0.
out	<i>output</i>	Buffer where the encrypted message is to be written.
	<i>output_size</i>	Size of the <i>output</i> buffer in bytes.
out	<i>output_length</i>	On success, the number of bytes that make up the returned output.

Return values

<i>PSA_SUCCESS</i>	
<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	The size of the output buffer is too small. You can determine a sufficient buffer size by calling <code>PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE</code> (key_type, key_bits, alg) where key_type and key_bits are the type and bit-size respectively of key.
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1210 of file tfm_crypto_api.c.

Here is the call graph for this function:



6.11.2.3 `psa_sign_hash()`

```

psa_status_t psa_sign_hash (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * hash,
    size_t hash_length,
    uint8_t * signature,
    size_t signature_size,
    size_t * signature_length )
  
```

Sign a hash or short message with a private key.

Note that to perform a hash-and-sign signature algorithm, you must first calculate the hash by calling [`psa\_hash\_setup\(\)`](#), [`psa\_hash\_update\(\)`](#) and [`psa\_hash\_finish\(\)`](#), or alternatively by calling [`psa\_hash\_compute\(\)`](#). Then pass the resulting hash as the `hash` parameter to this function. You can use [`PSA\_ALG\_SIGN\_GET\_HASH\(alg\)`](#) to determine the hash algorithm to use.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must be an asymmetric key pair. The key must allow the usage PSA_KEY_USAGE_SIGN_HASH .
	<i>alg</i>	A signature algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_SIGN_HASH (<i>alg</i>) is true), that is compatible with the type of <i>key</i> .
in	<i>hash</i>	The hash or message to sign.
	<i>hash_length</i>	Size of the <i>hash</i> buffer in bytes.
out	<i>signature</i>	Buffer where the signature is to be written.
	<i>signature_size</i>	Size of the <i>signature</i> buffer in bytes.
out	<i>signature_length</i>	On success, the number of bytes that make up the returned signature value.

Return values

PSA_SUCCESS	
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	
PSA_ERROR_BUFFER_TOO_SMALL	The size of the <i>signature</i> buffer is too small. You can determine a sufficient buffer size by calling PSA_SIGN_OUTPUT_SIZE (<i>key_type</i> , <i>key_bits</i> , <i>alg</i>) where <i>key_type</i> and <i>key_bits</i> are the type and bit-size respectively of <i>key</i> .
PSA_ERROR_NOT_SUPPORTED	
PSA_ERROR_INVALID_ARGUMENT	
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_INSUFFICIENT_ENTROPY	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1158 of file `tfm_crypto_api.c`.

6.11.2.4 `psa_sign_message()`

```
psa_status_t psa_sign_message (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    uint8_t * signature,
    size_t signature_size,
    size_t * signature_length )
```

Sign a message with a private key. For hash-and-sign algorithms, this includes the hashing step.

Note

To perform a multi-part hash-and-sign signature algorithm, first use a multi-part hash operation and then pass the resulting hash to [psa_sign_hash\(\)](#). [PSA_ALG_GET_HASH\(alg\)](#) can be used to determine the hash algorithm to use.

Parameters

in	<i>key</i>	Identifier of the key to use for the operation. It must be an asymmetric key pair. The key must allow the usage PSA_KEY_USAGE_SIGN_MESSAGE .
in	<i>alg</i>	An asymmetric signature algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_SIGN_MESSAGE(alg) is true), that is compatible with the type of <i>key</i> .
in	<i>input</i>	The input message to sign.
in	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
out	<i>signature</i>	Buffer where the signature is to be written.
in	<i>signature_size</i>	Size of the <i>signature</i> buffer in bytes. This must be appropriate for the selected algorithm and key: - The required signature size is PSA_SIGN_OUTPUT_SIZE(key_type, key_bits, alg) where <i>key_type</i> and <i>key_bits</i> are the type and bit-size respectively of key. PSA_SIGNATURE_MAX_SIZE evaluates to the maximum signature size of any supported signature algorithm.
out	<i>signature_length</i>	On success, the number of bytes that make up the returned signature value.

Return values

PSA_SUCCESS	
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	The key does not have the PSA_KEY_USAGE_SIGN_MESSAGE flag, or it does not permit the requested algorithm.
PSA_ERROR_BUFFER_TOO_SMALL	The size of the <i>signature</i> buffer is too small. You can determine a sufficient buffer size by calling PSA_SIGN_OUTPUT_SIZE(key_type, key_bits, alg) where <i>key_type</i> and <i>key_bits</i> are the type and bit-size respectively of <i>key</i> .
PSA_ERROR_NOT_SUPPORTED	
PSA_ERROR_INVALID_ARGUMENT	
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_DATA_CORRUPT	
PSA_ERROR_DATA_INVALID	
PSA_ERROR_INSUFFICIENT_ENTROPY	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1107 of file tfm_crypto_api.c.

6.11.2.5 `psa_unwrap_key()`

```
psa_status_t psa_unwrap_key (
    const psa_key_attributes_t * attributes,
    psa_key_id_t wrapping_key,
    psa_algorithm_t alg,
    const uint8_t * data,
    size_t data_length,
    psa_key_id_t * key )
```

Unwrap and import a key using a specified wrapping key.

\function psa_unwrap_key

The key is unwrapped and extracted from the `data` buffer. Its location, policy, and type are taken from `attributes`. The wrapped key determines the size. `psa_get_key_bits(attributes)` must either match the size or be 0. Zero-sized keys must be rejected by the implementation.

Note

The function first decrypts `data` using `alg` and `wrapping_key`, then processes the result and `attributes` as if calling `psa_import_key()`.

The key material remains within the cryptoprocessor, providing an extra layer of protection.

The API does not support asymmetric private key objects outside of a key pair. If the public key is missing, it will be reconstructed from the private key.

Implementation recommendation: support unwrapping keys created via `psa_wrap_key()` using the same algorithm and compatible attributes. Reject clearly malformed or truncated data.

Parameters

<i>attributes</i>	The attributes for the new key. The following attributes are required for all keys: • The key type determines how the decrypted <code>data</code> buffer is interpreted. The following attributes must be set for keys used in cryptographic operations: • The key permitted-algorithm policy, see permitted-algorithms. • The key usage flags, see key-usage-flags. The following attributes must be set for keys that do not use the default volatile lifetime: • The key lifetime, see key-lifetimes. • The key identifier is required for a key with a persistent lifetime, see key-identifiers. The following attributes are optional: • If the key size is nonzero, it must be equal to the key size determined from <code>data</code>.
-------------------	---

Note

This is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

Parameters

	<i>wrapping_key</i>	Identifier of the key to use for the unwrapping operation. It must permit the usage <code>PSA_KEY_USAGE_UNWRAP</code> .
	<i>alg</i>	The key-wrapping algorithm: a value of type <code>psa_algorithm_t</code> such that <code>PSA_ALG_IS_KEY_WRAP(alg)</code> is true.
in	<i>data</i>	Buffer containing the wrapped key data. The content of this buffer is unwrapped using the algorithm <code>alg</code> , and then interpreted according to the type declared in <code>attributes</code> .
	<i>data_length</i>	Size of the <code>data</code> buffer in bytes.
	<i>key</i>	On success, an identifier for the newly created key. <code>PSA_KEY_ID_NULL</code> on failure.

Return values

<i>PSA_SUCCESS</i>	Success. If the key is persistent, the key material and metadata have been saved to persistent storage.
<i>PSA_ERROR_ALREADY_EXISTS</i>	Attempt to create a persistent key, but a persistent key with the same identifier already exists.
<i>PSA_ERROR_INVALID_SIGNATURE</i>	The wrapped key data could not be authenticated.
<i>PSA_ERROR_INVALID_HANDLE</i>	<code>wrapping_key</code> is not a valid key identifier.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The following conditions can result in this error: <code>alg</code> is not supported or is not a key-wrapping algorithm. <code>wrapping_key</code> is not supported for use with <code>alg</code>. - The key attributes are not supported, either generally or in the specified storage location.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The following conditions can result in this error: <code>alg</code> is not a key-wrapping algorithm. <code>wrapping_key</code> is not compatible with <code>alg</code>. - The key type is invalid. - The key size is nonzero and is incompatible with the data. - The key lifetime is invalid. - The key identifier is invalid for the key lifetime. - Invalid key usage flags. - Invalid permitted algorithm. - Invalid key attributes overall. - Invalid key format or data layout.
<i>PSA_ERROR_NOT_PERMITTED</i>	The following conditions can result in this error: <code>wrapping_key</code> lacks the <code>PSA_KEY_USAGE_UNWRAP</code> flag or does not permit the requested algorithm. - Creating a key with the specified attributes is not allowed by implementation policy.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	

Return values

PSA_ERROR_INSUFFICIENT_STORAGE	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_DATA_CORRUPT	
PSA_ERROR_DATA_INVALID	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_BAD_STATE	The library requires initializing by a call to psa_crypto_init() .

6.11.2.6 `psa_verify_hash()`

```

psa_status_t psa_verify_hash (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * hash,
    size_t hash_length,
    const uint8_t * signature,
    size_t signature_length )

```

Verify the signature of a hash or short message using a public key.

Note that to perform a hash-and-sign signature algorithm, you must first calculate the hash by calling [psa_hash_setup\(\)](#), [psa_hash_update\(\)](#) and [psa_hash_finish\(\)](#), or alternatively by calling [psa_hash_compute\(\)](#). Then pass the resulting hash as the `hash` parameter to this function. You can use [PSA_ALG_SIGN_GET_HASH\(alg\)](#) to determine the hash algorithm to use.

Parameters

	<i>key</i>	Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. The key must allow the usage PSA_KEY_USAGE_VERIFY_HASH .
	<i>alg</i>	A signature algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_SIGN_HASH(alg) is true), that is compatible with the type of <i>key</i> .
in	<i>hash</i>	The hash or message whose signature is to be verified.
	<i>hash_length</i>	Size of the <i>hash</i> buffer in bytes.
in	<i>signature</i>	Buffer containing the signature to verify.
	<i>signature_length</i>	Size of the <i>signature</i> buffer in bytes.

Return values

PSA_SUCCESS	The signature is valid.
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	
PSA_ERROR_INVALID_SIGNATURE	The calculation was performed successfully, but the passed signature is not a valid signature.
PSA_ERROR_NOT_SUPPORTED	
PSA_ERROR_INVALID_ARGUMENT	
PSA_ERROR_INSUFFICIENT_MEMORY	

Return values

PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1188 of file `tfm_crypto_api.c`.

6.11.2.7 `psa_verify_message()`

```
psa_status_t psa_verify_message (
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * input,
    size_t input_length,
    const uint8_t * signature,
    size_t signature_length )
```

Verify the signature of a message with a public key, using a hash-and-sign verification algorithm.

Note

To perform a multi-part hash-and-sign signature verification algorithm, first use a multi-part hash operation to hash the message and then pass the resulting hash to [psa_verify_hash\(\)](#). `PSA_ALG_GET_HASH(alg)` can be used to determine the hash algorithm to use.

Parameters

in	<i>key</i>	Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. The key must allow the usage PSA_KEY_USAGE_VERIFY_MESSAGE .
in	<i>alg</i>	An asymmetric signature algorithm (<code>PSA_ALG_XXX</code> value such that PSA_ALG_IS_SIGN_MESSAGE(alg) is true), that is compatible with the type of <i>key</i> .
in	<i>input</i>	The message whose signature is to be verified.
in	<i>input_length</i>	Size of the <i>input</i> buffer in bytes.
in	<i>signature</i>	Buffer containing the signature to verify.
in	<i>signature_length</i>	Size of the <i>signature</i> buffer in bytes.

Return values

PSA_SUCCESS	
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	The key does not have the PSA_KEY_USAGE_SIGN_MESSAGE flag, or it does not permit the requested algorithm.

Return values

<i>PSA_ERROR_INVALID_SIGNATURE</i>	The calculation was performed successfully, but the passed signature is not a valid signature.
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1136 of file `tfm_crypto_api.c`.

6.12 Key derivation and pseudorandom generation

Macros

- `#define PSA_KEY_DERIVATION_OPERATION_INIT { 0, 0, 0, { 0 } }`
- `#define PSA_KEY_DERIVATION_UNLIMITED_CAPACITY ((size_t) (-1))`

Typedefs

- `typedef struct psa_key_derivation_s psa_key_derivation_operation_t`

Functions

- `psa_status_t psa_key_derivation_setup (psa_key_derivation_operation_t *operation, psa_algorithm_t alg)`
- `psa_status_t psa_key_derivation_get_capacity (const psa_key_derivation_operation_t *operation, size_t *capacity)`
- `psa_status_t psa_key_derivation_set_capacity (psa_key_derivation_operation_t *operation, size_t capacity)`
- `psa_status_t psa_key_derivation_input_bytes (psa_key_derivation_operation_t *operation, psa_key_derivation_step_t step, const uint8_t *data, size_t data_length)`
- `psa_status_t psa_key_derivation_input_integer (psa_key_derivation_operation_t *operation, psa_key_derivation_step_t step, uint64_t value)`
- `psa_status_t psa_key_derivation_input_key (psa_key_derivation_operation_t *operation, psa_key_derivation_step_t step, mbedtls_svc_key_id_t key)`
- `psa_status_t psa_key_derivation_key_agreement (psa_key_derivation_operation_t *operation, psa_key_derivation_step_t step, mbedtls_svc_key_id_t private_key, const uint8_t *peer_key, size_t peer_key_length)`
- `psa_status_t psa_key_derivation_output_bytes (psa_key_derivation_operation_t *operation, uint8_t *output, size_t output_length)`
- `psa_status_t psa_key_derivation_output_key (const psa_key_attributes_t *attributes, psa_key_derivation_operation_t *operation, mbedtls_svc_key_id_t *key)`
- `psa_status_t psa_key_derivation_output_key_custom (const psa_key_attributes_t *attributes, psa_key_derivation_operation_t *operation, const psa_custom_key_parameters_t *custom, const uint8_t *custom_data, size_t custom_data_length, mbedtls_svc_key_id_t *key)`
- `psa_status_t psa_key_derivation_verify_bytes (psa_key_derivation_operation_t *operation, const uint8_t *expected, size_t expected_length)`
- `psa_status_t psa_key_derivation_verify_key (psa_key_derivation_operation_t *operation, psa_key_id_t expected)`
- `psa_status_t psa_key_derivation_abort (psa_key_derivation_operation_t *operation)`
- `psa_status_t psa_raw_key_agreement (psa_algorithm_t alg, mbedtls_svc_key_id_t private_key, const uint8_t *peer_key, size_t peer_key_length, uint8_t *output, size_t output_size, size_t *output_length)`
- `psa_status_t psa_key_agreement (mbedtls_svc_key_id_t private_key, const uint8_t *peer_key, size_t peer_key_length, psa_algorithm_t alg, const psa_key_attributes_t *attributes, mbedtls_svc_key_id_t *key)`

6.12.1 Detailed Description

6.12.2 Macro Definition Documentation

6.12.2.1 PSA_KEY_DERIVATION_OPERATION_INIT

```
#define PSA_KEY_DERIVATION_OPERATION_INIT { 0, 0, 0, { 0 } }
```

This macro returns a suitable initializer for a key derivation operation object of type [psa_key_derivation_operation_t](#).

Definition at line 260 of file `crypto_struct.h`.

6.12.2.2 PSA_KEY_DERIVATION_UNLIMITED_CAPACITY

```
#define PSA_KEY_DERIVATION_UNLIMITED_CAPACITY ((size_t) (-1))
```

Use the maximum possible capacity for a key derivation operation.

Use this value as the capacity argument when setting up a key derivation to indicate that the operation should have the maximum possible capacity. The value of the maximum possible capacity depends on the key derivation algorithm.

Definition at line 3665 of file `crypto.h`.

6.12.3 Typedef Documentation

6.12.3.1 psa_key_derivation_operation_t

```
typedef struct psa\_key\_derivation\_s psa\_key\_derivation\_operation\_t
```

The type of the state data structure for key derivation operations.

Before calling any function on a key derivation operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa\_key\_derivation\_operation\_t operation;  
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa\_key\_derivation\_operation\_t operation = {0};
```
- Initialize the structure to the initializer [PSA_KEY_DERIVATION_OPERATION_INIT](#), for example:

```
psa\_key\_derivation\_operation\_t operation = PSA\_KEY\_DERIVATION\_OPERATION\_INIT;
```
- Assign the result of the function `psa_key_derivation_operation_init()` to the structure, for example:

```
psa\_key\_derivation\_operation\_t operation;  
operation = psa\_key\_derivation\_operation\_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 3532 of file `crypto.h`.

6.12.4 Function Documentation

6.12.4.1 `psa_key_agreement()`

```
psa_status_t psa_key_agreement (
    mbedtls_svc_key_id_t private_key,
    const uint8_t * peer_key,
    size_t peer_key_length,
    psa_algorithm_t alg,
    const psa_key_attributes_t * attributes,
    mbedtls_svc_key_id_t * key )
```

Perform a key agreement and return the shared secret as a derivation key.

Parameters

	<i>private_key</i>	Identifier of the private key to use. It must allow the usage PSA_KEY_USAGE_DERIVE .
in	<i>peer_key</i>	Public key of the peer. It must be in the same format that psa_import_key() accepts. The standard formats for public keys are documented in the documentation of psa_export_public_key() .
	<i>peer_key_length</i>	Size of <i>peer_key</i> in bytes.
	<i>alg</i>	The key agreement algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_RAW_KEY_AGREEMENT (alg) is true).
in	<i>attributes</i>	The attributes for the new key. This function uses the attributes as follows: - The key type must be one of PSA_KEY_TYPE_DERIVE, PSA_KEY_TYPE_RAW_DATA, PSA_KEY_TYPE_HMAC, or PSA_KEY_TYPE_PASSWORD. - The size of the returned key is always the bit-size of the shared secret, rounded up to a whole number of bytes. The key size in attributes can be zero; if it is nonzero, it must be equal to the output size of the key agreement, in bits. The output size, in bits, of the key agreement is 8 * PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE(type, bits), where type and bits are the type and bit-size of <i>private_key</i>. - The key permitted-algorithm policy is required for keys that will be used for a cryptographic operation. - The key usage flags define what operations are permitted with the key. - The key lifetime and identifier are required for a persistent key.
out	<i>key</i>	On success, an identifier for the newly created key. PSA_KEY_ID_NULL on failure.

Return values

PSA_SUCCESS	Success.
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() .
PSA_ERROR_INVALID_HANDLE	<i>private_key</i> is not a valid key identifier.

Return values

<i>PSA_ERROR_NOT_PERMITTED</i>	<code>private_key</code> does not have the <code>PSA_KEY_USAGE_DERIVE</code> flag, or it does not permit the requested algorithm.
<i>PSA_ERROR_ALREADY_EXISTS</i>	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	• <code>alg</code> is not a key agreement algorithm. • <code>private_key</code> is not compatible with <code>alg</code>. • <code>peer_key</code> is not valid for <code>alg</code> or not compatible with <code>private_key</code>. • The output key attributes in <code>attributes</code> are not valid: – The key type is not valid for key agreement output. – The key size is nonzero, and is not the size of the shared secret. – The key lifetime is invalid. – The key identifier is not valid for the key lifetime. – The key usage flags include invalid values. – The key's permitted-usage algorithm is invalid. – The key attributes, as a whole, are invalid.
<i>PSA_ERROR_NOT_SUPPORTED</i>	• <code>alg</code> is not a supported key agreement algorithm. • <code>private_key</code> is not supported for use with <code>alg</code>. • The output key attributes, as a whole, are not supported, either by the implementation in general or in the specified storage location.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_INSUFFICIENT_STORAGE</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	

6.12.4.2 `psa_key_derivation_abort()`

```
psa_status_t psa_key_derivation_abort (
    psa_key_derivation_operation_t * operation )
```

Abort a key derivation operation.

Aborting an operation frees all associated resources except for the `operation` structure itself. Once aborted, the operation object can be reused for another operation by calling `psa_key_derivation_setup()` again.

This function may be called at any time after the operation object has been initialized as described in `psa_key_derivation_operation_t`.

In particular, it is valid to call `psa_key_derivation_abort()` twice, or to call `psa_key_derivation_abort()` on an operation that has not been set up.

Parameters

<code>in, out</code>	<code>operation</code>	The operation to abort.
----------------------	------------------------	-------------------------

Return values

<code>PSA_SUCCESS</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1360 of file `tfm_crypto_api.c`.

6.12.4.3 `psa_key_derivation_get_capacity()`

```
psa_status_t psa_key_derivation_get_capacity (
    const psa_key_derivation_operation_t * operation,
    size_t * capacity )
```

Retrieve the current capacity of a key derivation operation.

The capacity of a key derivation is the maximum number of bytes that it can return. When you get N bytes of output from a key derivation operation, this reduces its capacity by N .

Parameters

<code>in</code>	<code>operation</code>	The operation to query.
<code>out</code>	<code>capacity</code>	On success, the capacity of the operation.

Return values

<code>PSA_SUCCESS</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	

Return values

PSA_ERROR_BAD_STATE	The operation state is not valid (it must be active), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.
-------------------------------------	--

Definition at line 1300 of file `tfm_crypto_api.c`.

6.12.4.4 `psa_key_derivation_input_bytes()`

```
psa_status_t psa_key_derivation_input_bytes (
    psa_key_derivation_operation_t * operation,
    psa_key_derivation_step_t step,
    const uint8_t * data,
    size_t data_length )
```

Provide an input for key derivation or key agreement.

Which inputs are required and in what order depends on the algorithm. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function passes direct inputs, which is usually correct for non-secret inputs. To pass a secret input, which should be in a key object, call [psa_key_derivation_input_key\(\)](#) instead of this function. Refer to the documentation of individual step types (PSA_KEY_DERIVATION_INPUT_XXX values of type [psa_key_derivation_step_t](#)) for more information.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Parameters

<i>in, out</i>	<i>operation</i>	The key derivation operation object to use. It must have been set up with psa_key_derivation_setup() and must not have produced any output yet.
	<i>step</i>	Which step the input data is for.
<i>in</i>	<i>data</i>	Input data to use.
	<i>data_length</i>	Size of the <code>data</code> buffer in bytes.

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INVALID_ARGUMENT	<code>step</code> is not compatible with the operation's algorithm, or <code>step</code> does not allow direct inputs.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	

Return values

<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid for this input <code>step</code> , or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.
--	---

Definition at line 1617 of file `tfm_crypto_api.c`.

6.12.4.5 `psa_key_derivation_input_integer()`

```
psa_status_t psa_key_derivation_input_integer (
    psa_key_derivation_operation_t * operation,
    psa_key_derivation_step_t step,
    uint64_t value )
```

Provide a numeric input for key derivation or key agreement.

Which inputs are required and in what order depends on the algorithm. However, when an algorithm requires a particular order, numeric inputs usually come first as they tend to be configuration parameters. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function is used for inputs which are fixed-size non-negative integers.

If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_key\_derivation\_abort\(\)`](#).

Parameters

<code>in, out</code>	<i>operation</i>	The key derivation operation object to use. It must have been set up with <code>psa_key_derivation_setup()</code> and must not have produced any output yet.
	<i>step</i>	Which step the input data is for.
<code>in</code>	<i>value</i>	The value of the numeric input.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<code>step</code> is not compatible with the operation's algorithm, or <code>step</code> does not allow numeric inputs.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid for this input <code>step</code> , or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1659 of file `tfm_crypto_api.c`.

6.12.4.6 `psa_key_derivation_input_key()`

```
psa_status_t psa_key_derivation_input_key (
    psa_key_derivation_operation_t * operation,
    psa_key_derivation_step_t step,
    mbedtls_svc_key_id_t key )
```

Provide an input for key derivation in the form of a key.

Which inputs are required and in what order depends on the algorithm. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function obtains input from a key object, which is usually correct for secret inputs or for non-secret personalization strings kept in the key store. To pass a non-secret parameter which is not in the key store, call [psa_key_derivation_input_bytes\(\)](#) instead of this function. Refer to the documentation of individual step types ($P \leftrightarrow SA_KEY_DERIVATION_INPUT_xxx$ values of type [psa_key_derivation_step_t](#)) for more information.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Parameters

<i>in, out</i>	<i>operation</i>	The key derivation operation object to use. It must have been set up with psa_key_derivation_setup() and must not have produced any output yet.
	<i>step</i>	Which step the input data is for.
	<i>key</i>	Identifier of the key. It must have an appropriate type for step and must allow the usage PSA_KEY_USAGE_DERIVE or PSA_KEY_USAGE_VERIFY_DERIVATION (see note) and the algorithm used by the operation.

Note

Once all inputs steps are completed, the operations will allow:

- [psa_key_derivation_output_bytes\(\)](#) if each input was either a direct input or a key with [PSA_KEY_USAGE_DERIVE](#) set;
- [psa_key_derivation_output_key\(\)](#) or [psa_key_derivation_output_key_custom\(\)](#) if the input for step [PSA_KEY_DERIVATION_INPUT_SECRET](#) or [PSA_KEY_DERIVATION_INPUT_PASSWORD](#) was from a key slot with [PSA_KEY_USAGE_DERIVE](#) and each other input was either a direct input or a key with [PSA_KEY_USAGE_DERIVE](#) set;
- [psa_key_derivation_verify_bytes\(\)](#) if each input was either a direct input or a key with [PSA_KEY_USAGE_VERIFY_DERIVATION](#) set;
- [psa_key_derivation_verify_key\(\)](#) under the same conditions as [psa_key_derivation_verify_bytes\(\)](#).

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	The key allows neither PSA_KEY_USAGE_DERIVE nor PSA_KEY_USAGE_VERIFY_DERIVATION , or it doesn't allow this algorithm.

Return values

<i>PSA_ERROR_INVALID_ARGUMENT</i>	<code>step</code> is not compatible with the operation's algorithm, or <code>step</code> does not allow key inputs of the given type or does not allow key inputs at all.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid for this input <code>step</code> , or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1341 of file `tfm_crypto_api.c`.

6.12.4.7 `psa_key_derivation_key_agreement()`

```
psa_status_t psa_key_derivation_key_agreement (
    psa_key_derivation_operation_t * operation,
    psa_key_derivation_step_t step,
    mbedtls_svc_key_id_t private_key,
    const uint8_t * peer_key,
    size_t peer_key_length )
```

Perform a key agreement and use the shared secret as input to a key derivation.

A key agreement algorithm takes two inputs: a private key `private_key` a public key `peer_key`. The result of this function is passed as input to a key derivation. The output of this key derivation can be extracted by reading from the resulting operation to produce keys and other cryptographic material.

If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_key\_derivation\_abort\(\)`](#).

Parameters

in, out	<i>operation</i>	The key derivation operation object to use. It must have been set up with <code>psa_key_derivation_setup()</code> with a key agreement and derivation algorithm <code>alg</code> (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_KEY_AGREEMENT(alg)</code> is true and <code>PSA_ALG_IS_RAW_KEY_AGREEMENT(alg)</code> is false). The operation must be ready for an input of the type given by <code>step</code> .
	<i>step</i>	Which step the input data is for.
	<i>private_key</i>	Identifier of the private key to use. It must allow the usage <code>PSA_KEY_USAGE_DERIVE</code> .
in	<i>peer_key</i>	Public key of the peer. The peer key must be in the same format that <code>psa_import_key()</code> accepts for the public key type corresponding to the type of <code>private_key</code> . That is, this function performs the equivalent of <code>psa_import_key(..., peer_key, peer_key_length)</code> where with key attributes indicating the public key type corresponding to the type of <code>private_key</code> . For example, for EC keys, this means that <code>peer_key</code> is interpreted as a point on the curve that the private key is on. The standard formats for public keys are documented in the documentation of <code>psa_export_public_key()</code> .
	<i>peer_key_length</i>	Size of <code>peer_key</code> in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<code>private_key</code> is not compatible with <code>alg</code> , or <code>peer_key</code> is not valid for <code>alg</code> or not compatible with <code>private_key</code> , or <code>step</code> does not allow an input resulting from a key agreement.
<i>PSA_ERROR_NOT_SUPPORTED</i>	<code>alg</code> is not supported or is not a key derivation algorithm.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid for this key agreement <code>step</code> , or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1378 of file `tfm_crypto_api.c`.

6.12.4.8 `psa_key_derivation_output_bytes()`

```
psa_status_t psa_key_derivation_output_bytes (
    psa_key_derivation_operation_t * operation,
    uint8_t * output,
    size_t output_length )
```

Read some data from a key derivation operation.

This function calculates output bytes from a key derivation algorithm and return those bytes. If you view the key derivation's output as a stream of bytes, this function destructively reads the requested number of bytes from the stream. The operation's capacity decreases by the number of bytes read.

If this function returns an error status other than [`PSA\_ERROR\_INSUFFICIENT\_DATA`](#), the operation enters an error state and must be aborted by calling [`psa\_key\_derivation\_abort\(\)`](#).

Parameters

<code>in, out</code>	<i>operation</i>	The key derivation operation object to read from.
<code>out</code>	<i>output</i>	Buffer where the output will be written.
	<i>output_length</i>	Number of bytes to output.

Return values

<i>PSA_SUCCESS</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	One of the inputs was a key whose policy didn't allow <code>PSA_KEY_USAGE_DERIVE</code> .

Return values

<i>PSA_ERROR_INSUFFICIENT_DATA</i>	The operation's capacity was less than <code>output_length</code> bytes. Note that in this case, no output is written to the output buffer. The operation's capacity is set to 0, thus subsequent calls to this function will not succeed, even with a smaller output buffer.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active and completed all required input steps), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1320 of file `tfm_crypto_api.c`.

6.12.4.9 `psa_key_derivation_output_key()`

```
psa_status_t psa_key_derivation_output_key (
    const psa_key_attributes_t * attributes,
    psa_key_derivation_operation_t * operation,
    mbedtls_svc_key_id_t * key )
```

Derive a key from an ongoing key derivation operation.

This function calculates output bytes from a key derivation algorithm and uses those bytes to generate a key deterministically. The key's location, usage policy, type and size are taken from `attributes`.

If you view the key derivation's output as a stream of bytes, this function destructively reads as many bytes as required from the stream. The operation's capacity decreases by the number of bytes read.

If this function returns an error status other than [`PSA\_ERROR\_INSUFFICIENT\_DATA`](#), the operation enters an error state and must be aborted by calling [`psa\_key\_derivation\_abort\(\)`](#).

How much output is produced and consumed from the operation, and how the key is derived, depends on the key type and on the key size (denoted `bits` below):

- For key types for which the key is an arbitrary sequence of bytes of a given size, this function is functionally equivalent to calling [`psa\_key\_derivation\_output\_bytes`](#) and passing the resulting output to [`psa\_import\_key`](#). However, this function has a security benefit: if the implementation provides an isolation boundary then the key material is not exposed outside the isolation boundary. As a consequence, for these key types, this function always consumes exactly $(bits / 8)$ bytes from the operation. The following key types defined in this specification follow this scheme:
 - [`PSA\_KEY\_TYPE\_AES`](#);
 - [`PSA\_KEY\_TYPE\_ARIA`](#);
 - [`PSA\_KEY\_TYPE\_CAMELLIA`](#);

- [PSA_KEY_TYPE_DERIVE](#);
 - [PSA_KEY_TYPE_HMAC](#);
 - [PSA_KEY_TYPE_PASSWORD_HASH](#).
- For ECC keys on a Montgomery elliptic curve ([PSA_KEY_TYPE_ECC_KEY_PAIR](#)(*curve*) where *curve* designates a Montgomery curve), this function always draws a byte string whose length is determined by the curve, and sets the mandatory bits accordingly. That is:
 - Curve25519 ([PSA_ECC_FAMILY_MONTGOMERY](#), 255 bits): draw a 32-byte string and process it as specified in RFC 7748 §5.
 - Curve448 ([PSA_ECC_FAMILY_MONTGOMERY](#), 448 bits): draw a 56-byte string and process it as specified in RFC 7748 §5.
 - For key types for which the key is represented by a single sequence of *bits* bits with constraints as to which bit sequences are acceptable, this function draws a byte string of length (*bits* / 8) bytes rounded up to the nearest whole number of bytes. If the resulting byte string is acceptable, it becomes the key, otherwise the drawn bytes are discarded. This process is repeated until an acceptable byte string is drawn. The byte string drawn from the operation is interpreted as specified for the output produced by [psa_export_key\(\)](#). The following key types defined in this specification follow this scheme:
 - Finite-field Diffie-Hellman keys ([PSA_KEY_TYPE_DH_KEY_PAIR](#)(*group*) where *group* designates any Diffie-Hellman group) and ECC keys on a Weierstrass elliptic curve ([PSA_KEY_TYPE_ECC_KEY_PAIR](#)(*curve*) where *curve* designates a Weierstrass curve). For these key types, interpret the byte string as integer in big-endian order. Discard it if it is not in the range $[0, N - 2]$ where *N* is the boundary of the private key domain (the prime *p* for Diffie-Hellman, the subprime *q* for DSA, or the order of the curve's base point for ECC). Add 1 to the resulting integer and use this as the private key *x*. This method allows compliance to NIST standards, specifically the methods titled "key-pair generation by testing candidates" in NIST SP 800-56A §5.6.1.1.4 for Diffie-Hellman, in FIPS 186-4 §B.1.2 for DSA, and in NIST SP 800-56A §5.6.1.2.2 or FIPS 186-4 §B.4.2 for elliptic curve keys.
 - For other key types, including [PSA_KEY_TYPE_RSA_KEY_PAIR](#), the way in which the operation output is consumed is implementation-defined.

In all cases, the data that is read is discarded from the operation. The operation's capacity is decreased by the number of bytes read.

For algorithms that take an input step [PSA_KEY_DERIVATION_INPUT_SECRET](#), the input to that step must be provided with [psa_key_derivation_input_key\(\)](#). Future versions of this specification may include additional restrictions on the derived key based on the attributes and strength of the secret key.

Note

This function is equivalent to calling [psa_key_derivation_output_key_custom\(\)](#) with the custom production parameters [PSA_CUSTOM_KEY_PARAMETERS_INIT](#) and `custom_data_length == 0` (i.e. `custom_data` is empty).

Parameters

in	<i>attributes</i>	The attributes for the new key. If the key type to be created is PSA_KEY_TYPE_PASSWORD_HASH then the algorithm in the policy must be the same as in the current operation.
in, out	<i>operation</i>	The key derivation operation object to read from.
out	<i>key</i>	On success, an identifier for the newly created key. For persistent keys, this is the key identifier defined in <i>attributes</i> . 0 on failure.

Return values

<i>PSA_SUCCESS</i>	Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
<i>PSA_ERROR_ALREADY_EXISTS</i>	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
<i>PSA_ERROR_INSUFFICIENT_DATA</i>	There was not enough data to create the desired key. Note that in this case, no output is written to the output buffer. The operation's capacity is set to 0, thus subsequent calls to this function will not succeed, even with a smaller output buffer.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The key type or key size is not supported, either by the implementation in general or in this particular location.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The provided key attributes are not valid for the operation.
<i>PSA_ERROR_NOT_PERMITTED</i>	The <i>PSA_KEY_DERIVATION_INPUT_SECRET</i> or <i>PSA_KEY_DERIVATION_INPUT_PASSWORD</i> input was not provided through a key; or one of the inputs was a key whose policy didn't allow <i>PSA_KEY_USAGE_DERIVE</i> .
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_INSUFFICIENT_STORAGE</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active and completed all required input steps), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1637 of file `tfm_crypto_api.c`.

6.12.4.10 `psa_key_derivation_output_key_custom()`

```
psa_status_t psa_key_derivation_output_key_custom (
    const psa_key_attributes_t * attributes,
    psa_key_derivation_operation_t * operation,
    const psa_custom_key_parameters_t * custom,
    const uint8_t * custom_data,
    size_t custom_data_length,
    mbedtls_svc_key_id_t * key )
```

Derive a key from an ongoing key derivation operation with custom production parameters.

See the description of `psa_key_derivation_out_key()` for the operation of this function with the default production parameters. Mbed TLS currently does not currently support any non-default production parameters.

Note

This function is experimental and may change in future minor versions of Mbed TLS.

Parameters

in	<i>attributes</i>	The attributes for the new key. If the key type to be created is PSA_KEY_TYPE_PASSWORD_HASH then the algorithm in the policy must be the same as in the current operation.
in, out	<i>operation</i>	The key derivation operation object to read from.
in	<i>custom</i>	Customization parameters for the key generation. When this is PSA_CUSTOM_KEY_PARAMETERS_INIT with <code>custom_data_length = 0</code> , this function is equivalent to psa_key_derivation_output_key() .
in	<i>custom_data</i>	Variable-length data associated with <i>custom</i> .
	<i>custom_data_length</i>	Length of <i>custom_data</i> in bytes.
out	<i>key</i>	On success, an identifier for the newly created key. For persistent keys, this is the key identifier defined in <i>attributes</i> . 0 on failure.

Return values

PSA_SUCCESS	Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
PSA_ERROR_ALREADY_EXISTS	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
PSA_ERROR_INSUFFICIENT_DATA	There was not enough data to create the desired key. Note that in this case, no output is written to the output buffer. The operation's capacity is set to 0, thus subsequent calls to this function will not succeed, even with a smaller output buffer.
PSA_ERROR_NOT_SUPPORTED	The key type or key size is not supported, either by the implementation in general or in this particular location.
PSA_ERROR_INVALID_ARGUMENT	The provided key attributes are not valid for the operation.
PSA_ERROR_NOT_PERMITTED	The PSA_KEY_DERIVATION_INPUT_SECRET or PSA_KEY_DERIVATION_INPUT_PASSWORD input was not provided through a key; or one of the inputs was a key whose policy didn't allow PSA_KEY_USAGE_DERIVE .
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_INSUFFICIENT_STORAGE	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_DATA_INVALID	
PSA_ERROR_DATA_CORRUPT	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be active and completed all required input steps), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

6.12.4.11 `psa_key_derivation_set_capacity()`

```
psa_status_t psa_key_derivation_set_capacity (
    psa_key_derivation_operation_t * operation,
    size_t capacity )
```

Set the maximum capacity of a key derivation operation.

The capacity of a key derivation operation is the maximum number of bytes that the key derivation operation can return from this point onwards.

Parameters

<i>in, out</i>	<i>operation</i>	The key derivation operation object to modify.
	<i>capacity</i>	The new capacity of the operation. It must be less or equal to the operation's current capacity.

Return values

<i>PSA_SUCCESS</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<i>capacity</i> is larger than the operation's current capacity. In this case, the operation object remains valid and its capacity remains unchanged.
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1600 of file `tfm_crypto_api.c`.

6.12.4.12 `psa_key_derivation_setup()`

```
psa_status_t psa_key_derivation_setup (
    psa_key_derivation_operation_t * operation,
    psa_algorithm_t alg )
```

Set up a key derivation operation.

A key derivation algorithm takes some inputs and uses them to generate a byte stream in a deterministic way. This byte stream can be used to produce keys and other cryptographic material.

To derive a key:

1. Start with an initialized object of type [`psa\_key\_derivation\_operation\_t`](#).
2. Call [`psa\_key\_derivation\_setup\(\)`](#) to select the algorithm.
3. Provide the inputs for the key derivation by calling [`psa\_key\_derivation\_input\_bytes\(\)`](#) or [`psa\_key\_derivation\_input\_key\(\)`](#) as appropriate. Which inputs are needed, in what order, and whether they may be keys and if so of what type depends on the algorithm.
4. Optionally set the operation's maximum capacity with [`psa\_key\_derivation\_set\_capacity\(\)`](#). You may do this before, in the middle of or after providing inputs. For some algorithms, this step is mandatory because the output depends on the maximum capacity.

5. To derive a key, call [psa_key_derivation_output_key\(\)](#) or [psa_key_derivation_output_key_custom\(\)](#). To derive a byte string for a different purpose, call [psa_key_derivation_output_bytes\(\)](#). Successive calls to these functions use successive output bytes calculated by the key derivation algorithm.
6. Clean up the key derivation operation object with [psa_key_derivation_abort\(\)](#).

If this function returns an error, the key derivation operation object is not changed.

If an error occurs at any step after a call to [psa_key_derivation_setup\(\)](#), the operation will need to be reset by a call to [psa_key_derivation_abort\(\)](#).

Implementations must reject an attempt to derive a key of size 0.

Parameters

<i>in, out</i>	<i>operation</i>	The key derivation operation object to set up. It must have been initialized but not set up yet.
	<i>alg</i>	The key derivation algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_KEY_DERIVATION (<i>alg</i>) is true).

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INVALID_ARGUMENT	<i>alg</i> is not a key derivation algorithm.
PSA_ERROR_NOT_SUPPORTED	<i>alg</i> is not supported or is not a key derivation algorithm.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be inactive), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1581 of file `tfm_crypto_api.c`.

6.12.4.13 [psa_key_derivation_verify_bytes\(\)](#)

```
psa_status_t psa_key_derivation_verify_bytes (
    psa_key_derivation_operation_t * operation,
    const uint8_t * expected,
    size_t expected_length )
```

Compare output data from a key derivation operation to an expected value.

This function calculates output bytes from a key derivation algorithm and compares those bytes to an expected value in constant time. If you view the key derivation's output as a stream of bytes, this function destructively reads the expected number of bytes from the stream before comparing them. The operation's capacity decreases by the number of bytes read.

This is functionally equivalent to the following code:

```
psa_key_derivation_output_bytes(operation, tmp, output_length);
if (memcmp(output, tmp, output_length) != 0)
    return PSA_ERROR_INVALID_SIGNATURE;
```

except (1) it works even if the key's policy does not allow outputting the bytes, and (2) the comparison will be done in constant time.

If this function returns an error status other than [PSA_ERROR_INSUFFICIENT_DATA](#) or [PSA_ERROR_INVALID_SIGNATURE](#), the operation enters an error state and must be aborted by calling [psa_key_derivation_abort\(\)](#).

Parameters

<i>in, out</i>	<i>operation</i>	The key derivation operation object to read from.
<i>in</i>	<i>expected</i>	Buffer containing the expected derivation output.
	<i>expected_length</i>	Length of the expected output; this is also the number of bytes that will be read.

Return values

PSA_SUCCESS	
PSA_ERROR_INVALID_SIGNATURE	The output was read successfully, but it differs from the expected output.
PSA_ERROR_NOT_PERMITTED	One of the inputs was a key whose policy didn't allow PSA_KEY_USAGE_VERIFY_DERIVATION .
PSA_ERROR_INSUFFICIENT_DATA	The operation's capacity was less than <code>output_length</code> bytes. Note that in this case, the operation's capacity is set to 0, thus subsequent calls to this function will not succeed, even with a smaller expected output.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The operation state is not valid (it must be active and completed all required input steps), or the library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1678 of file `tfm_crypto_api.c`.

6.12.4.14 `psa_key_derivation_verify_key()`

```
psa_status_t psa_key_derivation_verify_key (
    psa_key_derivation_operation_t * operation,
    psa_key_id_t expected )
```

Compare output data from a key derivation operation to an expected value stored in a key object.

This function calculates output bytes from a key derivation algorithm and compares those bytes to an expected value, provided as key of type [PSA_KEY_TYPE_PASSWORD_HASH](#). If you view the key derivation's output as a

stream of bytes, this function destructively reads the number of bytes corresponding to the length of the expected value from the stream before comparing them. The operation's capacity decreases by the number of bytes read.

This is functionally equivalent to exporting the key and calling `psa_key_derivation_verify_bytes()` on the result, except that it works even if the key cannot be exported.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA` or `PSA_ERROR_INVALID_SIGNATURE`, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

Parameters

<code>in, out</code>	<code>operation</code>	The key derivation operation object to read from.
<code>in</code>	<code>expected</code>	A key of type <code>PSA_KEY_TYPE_PASSWORD_HASH</code> containing the expected output. Its policy must include the <code>PSA_KEY_USAGE_VERIFY_DERIVATION</code> flag and the permitted algorithm must match the operation. The value of this key was likely computed by a previous call to <code>psa_key_derivation_output_key()</code> or <code>psa_key_derivation_output_key_custom()</code> .

Return values

<code>PSA_SUCCESS</code>	
<code>PSA_ERROR_INVALID_SIGNATURE</code>	The output was read successfully, but it differs from the expected output.
<code>PSA_ERROR_INVALID_HANDLE</code>	The key passed as the expected value does not exist.
<code>PSA_ERROR_INVALID_ARGUMENT</code>	The key passed as the expected value has an invalid type.
<code>PSA_ERROR_NOT_PERMITTED</code>	The key passed as the expected value does not allow this usage or this algorithm; or one of the inputs was a key whose policy didn't allow <code>PSA_KEY_USAGE_VERIFY_DERIVATION</code> .
<code>PSA_ERROR_INSUFFICIENT_DATA</code>	The operation's capacity was less than the length of the expected value. In this case, the operation's capacity is set to 0, thus subsequent calls to this function will not succeed, even with a smaller expected output.
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_HARDWARE_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid (it must be active and completed all required input steps), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1690 of file `tfm_crypto_api.c`.

6.12.4.15 `psa_raw_key_agreement()`

```
psa_status_t psa_raw_key_agreement (
    psa_algorithm_t alg,
```

```

mbedtls_svc_key_id_t private_key,
const uint8_t * peer_key,
size_t peer_key_length,
uint8_t * output,
size_t output_size,
size_t * output_length )

```

Perform a key agreement and return the raw shared secret.

Warning

The raw result of a key agreement algorithm such as finite-field Diffie-Hellman or elliptic curve Diffie-Hellman has biases and should not be used directly as key material. It should instead be passed as input to a key derivation algorithm. To chain a key agreement with a key derivation, use [psa_key_derivation_key_agreement\(\)](#) and other functions from the key derivation interface.

Parameters

	<i>alg</i>	The key agreement algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_RAW_KEY_AGREEMENT (alg) is true).
	<i>private_key</i>	Identifier of the private key to use. It must allow the usage PSA_KEY_USAGE_DERIVE .
in	<i>peer_key</i>	Public key of the peer. It must be in the same format that psa_import_key() accepts. The standard formats for public keys are documented in the documentation of psa_export_public_key() .
	<i>peer_key_length</i>	Size of <i>peer_key</i> in bytes.
out	<i>output</i>	Buffer where the decrypted message is to be written.
	<i>output_size</i>	Size of the <i>output</i> buffer in bytes.
out	<i>output_length</i>	On success, the number of bytes that make up the returned output.

Return values

PSA_SUCCESS	Success.
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_NOT_PERMITTED	
PSA_ERROR_INVALID_ARGUMENT	alg is not a key agreement algorithm, or private_key is not compatible with alg, or peer_key is not valid for alg or not compatible with private_key.
PSA_ERROR_BUFFER_TOO_SMALL	output_size is too small
PSA_ERROR_NOT_SUPPORTED	alg is not a supported key agreement algorithm.
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1550 of file tfm_crypto_api.c.

6.13 Random generation

Functions

- [psa_status_t psa_generate_random](#) (uint8_t *output, size_t output_size)
Generate random bytes.
- [psa_status_t psa_generate_key](#) (const [psa_key_attributes_t](#) *attributes, [mbedtls_svc_key_id_t](#) *key)
Generate a key or key pair.
- [psa_status_t psa_generate_key_custom](#) (const [psa_key_attributes_t](#) *attributes, const [psa_custom_key_parameters_t](#) *custom, const uint8_t *custom_data, size_t custom_data_length, [mbedtls_svc_key_id_t](#) *key)
Generate a key or key pair using custom production parameters.

6.13.1 Detailed Description

6.13.2 Function Documentation

6.13.2.1 psa_generate_key()

```
psa_status_t psa_generate_key (
    const psa_key_attributes_t * attributes,
    mbedtls_svc_key_id_t * key )
```

Generate a key or key pair.

The key is generated randomly. Its location, usage policy, type and size are taken from `attributes`.

Implementations must reject an attempt to generate a key of size 0.

The following type-specific considerations apply:

- For RSA keys ([PSA_KEY_TYPE_RSA_KEY_PAIR](#)), the public exponent is 65537. The modulus is a product of two probabilistic primes between 2^{n-1} and 2^n where n is the bit size specified in the attributes.

Note

This function is equivalent to calling [psa_generate_key_custom\(\)](#) with the custom production parameters [PSA_CUSTOM_KEY_PARAMETERS_INIT](#) and `custom_data_length == 0` (i.e. `custom_data` is empty).

Parameters

in	<i>attributes</i>	The attributes for the new key.
out	<i>key</i>	On success, an identifier for the newly created key. For persistent keys, this is the key identifier defined in <code>attributes</code> . 0 on failure.

Return values

<i>PSA_SUCCESS</i>	Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
<i>PSA_ERROR_ALREADY_EXISTS</i>	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_INSUFFICIENT_STORAGE</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1422 of file `tfm_crypto_api.c`.

6.13.2.2 `psa_generate_key_custom()`

```
psa_status_t psa_generate_key_custom (
    const psa_key_attributes_t * attributes,
    const psa_custom_key_parameters_t * custom,
    const uint8_t * custom_data,
    size_t custom_data_length,
    mbedtls_svc_key_id_t * key )
```

Generate a key or key pair using custom production parameters.

See the description of [`psa\_generate\_key\(\)`](#) for the operation of this function with the default production parameters. In addition, this function supports the following production customizations, described in more detail in the documentation of [`psa\_custom\_key\_parameters\_t`](#):

- RSA keys: generation with a custom public exponent.

Note

This function is experimental and may change in future minor versions of Mbed TLS.

Parameters

in	<i>attributes</i>	The attributes for the new key.
in	<i>custom</i>	Customization parameters for the key generation. When this is <code>PSA_CUSTOM_KEY_PARAMETERS_INIT</code> with <code>custom_data_length = 0</code> , this function is equivalent to <code>psa_generate_key()</code> .
in	<i>custom_data</i>	Variable-length data associated with <i>custom</i> . Generated by Doxygen
	<i>custom_data_length</i>	Length of <i>custom_data</i> in bytes.
out	<i>key</i>	On success, an identifier for the newly created key. For persistent keys, this is the key identifier defined in <i>attributes</i> . 0 on failure.

Return values

<i>PSA_SUCCESS</i>	Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
<i>PSA_ERROR_ALREADY_EXISTS</i>	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_INSUFFICIENT_STORAGE</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.13.2.3 `psa_generate_random()`

```
psa_status_t psa_generate_random (
    uint8_t * output,
    size_t output_size )
```

Generate random bytes.

Warning

This function **can** fail! Callers **MUST** check the return status and **MUST NOT** use the content of the output buffer if the return status is not [`PSA\_SUCCESS`](#).

Note

To generate a key, use [`psa\_generate\_key\(\)`](#) instead.

Parameters

out	<i>output</i>	Output buffer for the generated data.
	<i>output_size</i>	Number of bytes to generate and output.

Return values

<i>PSA_SUCCESS</i>	
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	

Return values

<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

Definition at line 1400 of file `tfm_crypto_api.c`.

6.14 Interruptible sign/verify hash

Typedefs

- typedef struct [psa_sign_hash_interruptible_operation_s](#) [psa_sign_hash_interruptible_operation_t](#)
- typedef struct [psa_verify_hash_interruptible_operation_s](#) [psa_verify_hash_interruptible_operation_t](#)

Functions

- [void](#) [psa_interruptible_set_max_ops](#) ([uint32_t](#) max_ops)
Set the maximum number of ops allowed to be executed by an interruptible function in a single call.
- [uint32_t](#) [psa_interruptible_get_max_ops](#) ([void](#))
Get the maximum number of ops allowed to be executed by an interruptible function in a single call. This will return the last value set by [psa_interruptible_set_max_ops\(\)](#) or [PSA_INTERRUPTIBLE_MAX_OPS_UNLIMITED](#) if that function has never been called.
- [uint32_t](#) [psa_sign_hash_get_num_ops](#) ([const](#) [psa_sign_hash_interruptible_operation_t](#) *operation)
Get the number of ops that a hash signing operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa_sign_hash_interruptible_abort\(\)](#) on the operation, a value of 0 will be returned.
- [uint32_t](#) [psa_verify_hash_get_num_ops](#) ([const](#) [psa_verify_hash_interruptible_operation_t](#) *operation)
Get the number of ops that a hash verification operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa_verify_hash_interruptible_abort\(\)](#) on the operation, a value of 0 will be returned.
- [psa_status_t](#) [psa_sign_hash_start](#) ([psa_sign_hash_interruptible_operation_t](#) *operation, [mbedtls_svc_key_id_t](#) key, [psa_algorithm_t](#) alg, [const](#) [uint8_t](#) *hash, [size_t](#) hash_length)
Start signing a hash or short message with a private key, in an interruptible manner.
- [psa_status_t](#) [psa_sign_hash_complete](#) ([psa_sign_hash_interruptible_operation_t](#) *operation, [uint8_t](#) *signature, [size_t](#) signature_size, [size_t](#) *signature_length)
Continue and eventually complete the action of signing a hash or short message with a private key, in an interruptible manner.
- [psa_status_t](#) [psa_sign_hash_abort](#) ([psa_sign_hash_interruptible_operation_t](#) *operation)
Abort a sign hash operation.
- [psa_status_t](#) [psa_verify_hash_start](#) ([psa_verify_hash_interruptible_operation_t](#) *operation, [mbedtls_svc_key_id_t](#) key, [psa_algorithm_t](#) alg, [const](#) [uint8_t](#) *hash, [size_t](#) hash_length, [const](#) [uint8_t](#) *signature, [size_t](#) signature_length)
Start reading and verifying a hash or short message, in an interruptible manner.
- [psa_status_t](#) [psa_verify_hash_complete](#) ([psa_verify_hash_interruptible_operation_t](#) *operation)
Continue and eventually complete the action of reading and verifying a hash or short message signed with a private key, in an interruptible manner.
- [psa_status_t](#) [psa_verify_hash_abort](#) ([psa_verify_hash_interruptible_operation_t](#) *operation)
Abort a verify hash operation.

6.14.1 Detailed Description

6.14.2 Typedef Documentation

6.14.2.1 `psa_sign_hash_interruptible_operation_t`

```
typedef struct psa_sign_hash_interruptible_operation_s psa_sign_hash_interruptible_operation_t
```

The type of the state data structure for interruptible hash signing operations.

Before calling any function on a sign hash operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_sign_hash_interruptible_operation_t operation;
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_sign_hash_interruptible_operation_t operation = {0};
```
- Initialize the structure to the initializer `PSA_SIGN_HASH_INTERRUPTIBLE_OPERATION_INIT`, for example:

```
psa_sign_hash_interruptible_operation_t operation =
PSA_SIGN_HASH_INTERRUPTIBLE_OPERATION_INIT;
```
- Assign the result of the function `psa_sign_hash_interruptible_operation_init()` to the structure, for example:

```
psa_sign_hash_interruptible_operation_t operation;
operation = psa_sign_hash_interruptible_operation_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 4626 of file `crypto.h`.

6.14.2.2 `psa_verify_hash_interruptible_operation_t`

```
typedef struct psa_verify_hash_interruptible_operation_s psa_verify_hash_interruptible_operation_t
```

The type of the state data structure for interruptible hash verification operations.

Before calling any function on a sign hash operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_verify_hash_interruptible_operation_t operation;
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_verify_hash_interruptible_operation_t operation = {0};
```
- Initialize the structure to the initializer `PSA_VERIFY_HASH_INTERRUPTIBLE_OPERATION_INIT`, for example:

```
psa_verify_hash_interruptible_operation_t operation =
PSA_VERIFY_HASH_INTERRUPTIBLE_OPERATION_INIT;
```
- Assign the result of the function `psa_verify_hash_interruptible_operation_init()` to the structure, for example:

```
psa_verify_hash_interruptible_operation_t operation;
operation = psa_verify_hash_interruptible_operation_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 4659 of file `crypto.h`.

6.14.3 Function Documentation

6.14.3.1 `psa_interruptible_get_max_ops()`

```
uint32_t psa_interruptible_get_max_ops (
    void )
```

Get the maximum number of ops allowed to be executed by an interruptible function in a single call. This will return the last value set by `psa_interruptible_set_max_ops()` or `PSA_INTERRUPTIBLE_MAX_OPS_UNLIMITED` if that function has never been called.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Returns

Maximum number of ops allowed to be executed by an interruptible function in a single call.

6.14.3.2 `psa_interruptible_set_max_ops()`

```
void psa_interruptible_set_max_ops (
    uint32_t max_ops )
```

Set the maximum number of ops allowed to be executed by an interruptible function in a single call.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

The time taken to execute a single op is implementation specific and depends on software, hardware, the algorithm, key type and curve chosen. Even within a single operation, successive ops can take differing amounts of time. The only guarantee is that lower values for `max_ops` means functions will block for a lesser maximum amount of time. The functions `psa_sign_interruptible_get_num_ops()`, `psa_verify_interruptible_get_num_ops()` and `psa_generate_key_iop_get_num_ops()` are provided to help with tuning this value.

This value defaults to `PSA_INTERRUPTIBLE_MAX_OPS_UNLIMITED`, which means the whole operation will be done in one go, regardless of the number of ops required.

If more ops are needed to complete a computation, `PSA_OPERATION_INCOMPLETE` will be returned by the function performing the computation. It is then the caller's responsibility to either call again with the same operation context until it returns 0 or an error code; or to call the relevant abort function if the answer is no longer required.

The interpretation of `max_ops` is also implementation defined. On a hard real time system, this can indicate a hard deadline, as a real-time system needs a guarantee of not spending more than X time, however care must be taken in such an implementation to avoid the situation whereby calls just return, not being able to do any actual work within the allotted time. On a non-real-time system, the implementation can be more relaxed, but again whether this number should be interpreted as as hard or soft limit or even whether a less than or equals as regards to ops executed in a single call is implementation defined.

For keys in local storage when no accelerator driver applies, please see also the documentation for `psa_interruptible_set_max_ops()`, which is the internal implementation in these cases.

Warning

With implementations that interpret this number as a hard limit, setting this number too small may result in an infinite loop, whereby each call results in immediate return with no ops done (as there is not enough time to execute any), and thus no result will ever be achieved.

Note

This only applies to functions whose documentation mentions they may return [PSA_OPERATION_INCOMPLETE](#).

Parameters

<i>max_ops</i>	The maximum number of ops to be executed in a single call. This can be a number from 0 to PSA_INTERRUPTIBLE_MAX_OPS_UNLIMITED , where 0 is the least amount of work done per call.
----------------	--

6.14.3.3 psa_sign_hash_abort()

```
psa_status_t psa_sign_hash_abort (
    psa_sign_hash_interruptible_operation_t * operation )
```

Abort a sign hash operation.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function is the only function that clears the number of ops completed as part of the operation. Please ensure you copy this value via [psa_sign_hash_get_num_ops\(\)](#) if required before calling.

Aborting an operation frees all associated resources except for the `operation` structure itself. Once aborted, the operation object can be reused for another operation by calling [psa_sign_hash_start\(\)](#) again.

You may call this function any time after the operation object has been initialized. In particular, calling [psa_sign_hash_abort\(\)](#) after the operation has already been terminated by a call to [psa_sign_hash_abort\(\)](#) or [psa_sign_hash_complete\(\)](#) is safe.

Parameters

<i>in, out</i>	<i>operation</i>	Initialized sign hash operation.
----------------	------------------	----------------------------------

Return values

PSA_SUCCESS	The operation was aborted successfully.
PSA_ERROR_NOT_SUPPORTED	

Return values

<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.
--	--

6.14.3.4 `psa_sign_hash_complete()`

```
psa_status_t psa_sign_hash_complete (
    psa_sign_hash_interruptible_operation_t * operation,
    uint8_t * signature,
    size_t signature_size,
    size_t * signature_length )
```

Continue and eventually complete the action of signing a hash or short message with a private key, in an interruptible manner.

See also

[`psa\_sign\_hash\_start\(\)`](#)

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with [`psa\_sign\_hash\_start\(\)`](#) is equivalent to [`psa\_sign\_hash\(\)`](#) but this function can return early and resume according to the limit set with [`psa\_interruptible\_set\_max\_ops\(\)`](#) to reduce the maximum time spent in a function call.

Users should call this function on the same operation object repeatedly until it either returns 0 or an error. This function will return [`PSA\_OPERATION\_INCOMPLETE`](#) if there is more work to do. Alternatively users can call [`psa\_sign\_hash\_abort\(\)`](#) at any point if they no longer want the result.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_sign\_hash\_abort\(\)`](#).

Parameters

in, out	<i>operation</i>	The <code>psa_sign_hash_interruptible_operation_t</code> to use. This must be initialized first, and have had <code>psa_sign_hash_start()</code> called with it first.
out	<i>signature</i>	Buffer where the signature is to be written.
	<i>signature_size</i>	Size of the <i>signature</i> buffer in bytes. This must be appropriate for the selected algorithm and key: - The required signature size is <code>PSA_SIGN_OUTPUT_SIZE(key_type, key_bits, alg)</code> where <i>key_type</i> and <i>key_bits</i> are the type and bit-size respectively of key. <code>PSA_SIGNATURE_MAX_SIZE</code> evaluates to the maximum signature size of any supported signature algorithm.
Generated by Doxygen		
out	<i>signature_length</i>	On success, the number of bytes that make up the returned signature value.

Return values

<i>PSA_SUCCESS</i>	Operation completed successfully
<i>PSA_OPERATION_INCOMPLETE</i>	Operation was interrupted due to the setting of <code>psa_interruptible_set_max_ops()</code> . There is still work to be done. Call this function again with the same operation object.
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	The size of the <code>signature</code> buffer is too small. You can determine a sufficient buffer size by calling <code>PSA_SIGN_OUTPUT_SIZE(key_type, key_bits, alg)</code> where <code>key_type</code> and <code>key_bits</code> are the type and bit-size respectively of key.
<i>PSA_ERROR_BAD_STATE</i>	An operation was not previously started on this context via <code>psa_sign_hash_start()</code> .
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has either not been previously initialized by <code>psa_crypto_init()</code> or you did not previously call <code>psa_sign_hash_start()</code> with this operation object. It is implementation-dependent whether a failure to initialize results in this error code.

6.14.3.5 `psa_sign_hash_get_num_ops()`

```
uint32_t psa_sign_hash_get_num_ops (
    const psa\_sign\_hash\_interruptible\_operation\_t * operation )
```

Get the number of ops that a hash signing operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [`psa\_sign\_hash\_interruptible\_abort\(\)`](#) on the operation, a value of 0 will be returned.

Note

This interface is guaranteed re-entrant and thus may be called from driver code.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

This is a helper provided to help you tune the value passed to [`psa\_interruptible\_set\_max\_ops\(\)`](#).

Parameters

<i>operation</i>	The <code>psa_sign_hash_interruptible_operation_t</code> to use. This must be initialized first.
------------------	--

Returns

Number of ops that the operation has taken so far.

6.14.3.6 `psa_sign_hash_start()`

```
psa_status_t psa_sign_hash_start (
    psa_sign_hash_interruptible_operation_t * operation,
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * hash,
    size_t hash_length )
```

Start signing a hash or short message with a private key, in an interruptible manner.

See also

`psa_sign_hash_complete()`

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with `psa_sign_hash_complete()` is equivalent to `psa_sign_hash()` but `psa_sign_hash_complete()` can return early and resume according to the limit set with `psa_interruptible_set_max_ops()` to reduce the maximum time spent in a function call.

Users should call `psa_sign_hash_complete()` repeatedly on the same context after a successful call to this function until `psa_sign_hash_complete()` either returns 0 or an error. `psa_sign_hash_complete()` will return `PSA_OPERATION_INCOMPLETE` if there is more work to do. Alternatively users can call `psa_sign_hash_abort()` at any point if they no longer want the result.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_sign_hash_abort()`.

Parameters

<i>in, out</i>	<i>operation</i>	The <code>psa_sign_hash_interruptible_operation_t</code> to use. This must be initialized first.
	<i>key</i>	Identifier of the key to use for the operation. It must be an asymmetric key pair. The key must allow the usage <code>PSA_KEY_USAGE_SIGN_HASH</code> .
	<i>alg</i>	A signature algorithm (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_SIGN_HASH(alg)</code> is true), that is compatible with the type of <i>key</i> .
<i>in</i>	<i>hash</i>	The hash or message to sign.
	<i>hash_length</i>	Size of the <i>hash</i> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	The operation started successfully - call <code>psa_sign_hash_complete()</code> with the same context to complete the operation
<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_NOT_PERMITTED</i>	The key does not have the <code>PSA_KEY_USAGE_SIGN_HASH</code> flag, or it does not permit the requested algorithm.
<i>PSA_ERROR_BAD_STATE</i>	An operation has previously been started on this context, and is still in progress.
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.14.3.7 `psa_verify_hash_abort()`

```
psa_status_t psa_verify_hash_abort (
    psa_verify_hash_interruptible_operation_t * operation )
```

Abort a verify hash operation.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function is the only function that clears the number of ops completed as part of the operation. Please ensure you copy this value via [`psa\_verify\_hash\_get\_num\_ops\(\)`](#) if required before calling.

Aborting an operation frees all associated resources except for the operation structure itself. Once aborted, the operation object can be reused for another operation by calling [`psa\_verify\_hash\_start\(\)`](#) again.

You may call this function any time after the operation object has been initialized. In particular, calling [`psa\_verify\_hash\_abort\(\)`](#) after the operation has already been terminated by a call to [`psa\_verify\_hash\_abort\(\)`](#) or [`psa\_verify\_hash\_complete\(\)`](#) is safe.

Parameters

<code>in, out</code>	<code>operation</code>	Initialized verify hash operation.
----------------------	------------------------	------------------------------------

Return values

<code>PSA_SUCCESS</code>	The operation was aborted successfully.
<code>PSA_ERROR_NOT_SUPPORTED</code>	
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.14.3.8 `psa_verify_hash_complete()`

```
psa_status_t psa_verify_hash_complete (
    psa_verify_hash_interruptible_operation_t * operation )
```

Continue and eventually complete the action of reading and verifying a hash or short message signed with a private key, in an interruptible manner.

See also

[`psa\_verify\_hash\_start\(\)`](#)

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with [`psa\_verify\_hash\_start\(\)`](#) is equivalent to [`psa\_verify\_hash\(\)`](#) but this function can return early and resume according to the limit set with [`psa\_interruptible\_set\_max\_ops\(\)`](#) to reduce the maximum time spent in a function call.

Users should call this function on the same operation object repeatedly until it either returns 0 or an error. This function will return [`PSA\_OPERATION\_INCOMPLETE`](#) if there is more work to do. Alternatively users can call [`psa\_verify\_hash\_abort\(\)`](#) at any point if they no longer want the result.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_verify\_hash\_abort\(\)`](#).

Parameters

<code>in, out</code>	<code>operation</code>	The <code>psa_verify_hash_interruptible_operation_t</code> to use. This must be initialized first, and have had <code>psa_verify_hash_start()</code> called with it first.
----------------------	------------------------	--

Return values

<i>PSA_SUCCESS</i>	Operation completed successfully, and the passed signature is valid.
<i>PSA_OPERATION_INCOMPLETE</i>	Operation was interrupted due to the setting of <code>psa_interruptible_set_max_ops()</code> . There is still work to be done. Call this function again with the same operation object.
<i>PSA_ERROR_INVALID_HANDLE</i>	
<i>PSA_ERROR_INVALID_SIGNATURE</i>	The calculation was performed successfully, but the passed signature is not a valid signature.
<i>PSA_ERROR_BAD_STATE</i>	An operation was not previously started on this context via <code>psa_verify_hash_start()</code> .
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has either not been previously initialized by <code>psa_crypto_init()</code> or you did not previously call <code>psa_verify_hash_start()</code> on this object. It is implementation-dependent whether a failure to initialize results in this error code.

6.14.3.9 `psa_verify_hash_get_num_ops()`

```
uint32_t psa_verify_hash_get_num_ops (
    const psa\_verify\_hash\_interruptible\_operation\_t * operation )
```

Get the number of ops that a hash verification operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [`psa\_verify\_hash\_interruptible\_abort\(\)`](#) on the operation, a value of 0 will be returned.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

This is a helper provided to help you tune the value passed to [`psa\_interruptible\_set\_max\_ops\(\)`](#).

Parameters

<i>operation</i>	The <code>psa_verify_hash_interruptible_operation_t</code> to use. This must be initialized first.
------------------	--

Returns

Number of ops that the operation has taken so far.

6.14.3.10 `psa_verify_hash_start()`

```
psa_status_t psa_verify_hash_start (
    psa_verify_hash_interruptible_operation_t * operation,
    mbedtls_svc_key_id_t key,
    psa_algorithm_t alg,
    const uint8_t * hash,
    size_t hash_length,
    const uint8_t * signature,
    size_t signature_length )
```

Start reading and verifying a hash or short message, in an interruptible manner.

See also

`psa_verify_hash_complete()`

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with `psa_verify_hash_complete()` is equivalent to `psa_verify_hash()` but `psa_verify_hash_complete()` can return early and resume according to the limit set with `psa_interruptible_set_max_ops()` to reduce the maximum time spent in a function.

Users should call `psa_verify_hash_complete()` repeatedly on the same operation object after a successful call to this function until `psa_verify_hash_complete()` either returns 0 or an error. `psa_verify_hash_complete()` will return `PSA_OPERATION_INCOMPLETE` if there is more work to do. Alternatively users can call `psa_verify_hash_abort()` at any point if they no longer want the result.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_verify_hash_abort()`.

Parameters

in, out	<i>operation</i>	The <code>psa_verify_hash_interruptible_operation_t</code> to use. This must be initialized first.
	<i>key</i>	Identifier of the key to use for the operation. The key must allow the usage <code>PSA_KEY_USAGE_VERIFY_HASH</code> .
	<i>alg</i>	A signature algorithm (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_SIGN_HASH(alg)</code> is true), that is compatible with the type of key.
in	<i>hash</i>	The hash whose signature is to be verified.
	<i>hash_length</i>	Size of the <code>hash</code> buffer in bytes.
in	<i>signature</i>	Buffer containing the signature to verify.
	<i>signature_length</i>	Size of the <code>signature</code> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	The operation started successfully - please call <code>psa_verify_hash_complete()</code> with the same context to complete the operation.
<i>PSA_ERROR_BAD_STATE</i>	Another operation has already been started on this context, and is still in progress.
<i>PSA_ERROR_NOT_PERMITTED</i>	The key does not have the <code>PSA_KEY_USAGE_VERIFY_HASH</code> flag, or it does not permit the requested algorithm.
<i>PSA_ERROR_NOT_SUPPORTED</i>	
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_BAD_STATE</i>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.15 Interruptible Key Agreement

Typedefs

- typedef struct [psa_key_agreement_iop_s](#) [psa_key_agreement_iop_t](#)

Functions

- [uint32_t](#) [psa_key_agreement_iop_get_num_ops](#) ([psa_key_agreement_iop_t](#) *operation)
Get the number of ops that a key agreement operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa_key_agreement_iop_abort\(\)](#) on the operation, a value of 0 will be returned.
- [psa_status_t](#) [psa_key_agreement_iop_setup](#) ([psa_key_agreement_iop_t](#) *operation, [mbedtls_svc_key_id_t](#) private_key, const [uint8_t](#) *peer_key, [size_t](#) peer_key_length, [psa_algorithm_t](#) alg, const [psa_key_attributes_t](#) *attributes)
Start a key agreement operation, in an interruptible manner.
- [psa_status_t](#) [psa_key_agreement_iop_complete](#) ([psa_key_agreement_iop_t](#) *operation, [mbedtls_svc_key_id_t](#) *key)
Continue and eventually complete the action of key agreement, in an interruptible manner.
- [psa_status_t](#) [psa_key_agreement_iop_abort](#) ([psa_key_agreement_iop_t](#) *operation)
Abort a key agreement operation.

6.15.1 Detailed Description

6.15.2 Typedef Documentation

6.15.2.1 [psa_key_agreement_iop_t](#)

```
typedef struct psa\_key\_agreement\_iop\_s psa\_key\_agreement\_iop\_t
```

The type of the state data structure for interruptible key agreement operations.

Before calling any function on an interruptible key agreement object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa\_key\_agreement\_iop\_t operation;  
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa\_key\_agreement\_iop\_t operation = {0};
```
- Initialize the structure to the initializer [PSA_KEY_AGREEMENT_IOP_INIT](#), for example:
- [psa_key_agreement_iop_t](#) operation = [PSA_KEY_AGREEMENT_IOP_INIT](#);
- Assign the result of the function [psa_key_agreement_iop_init\(\)](#) to the structure, for example:

```
psa\_key\_agreement\_iop\_t operation;  
operation = psa\_key\_agreement\_iop\_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 5255 of file `crypto.h`.

6.15.3 Function Documentation

6.15.3.1 `psa_key_agreement_iop_abort()`

```
psa_status_t psa_key_agreement_iop_abort (
    psa_key_agreement_iop_t * operation )
```

Abort a key agreement operation.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function clears the number of ops completed as part of the operation. Please ensure you copy this value via `psa_key_agreement_iop_get_num_ops()` if required before calling.

Aborting an operation frees all associated resources except for the operation structure itself. Once aborted, the operation object can be reused for another operation by calling `psa_key_agreement_iop_setup()` again.

You may call this function any time after the operation object has been initialized. In particular, calling `psa_key_agreement_iop_abort()` after the operation has already been terminated by a call to `psa_key_agreement_iop_abort()` or `psa_key_agreement_iop_complete()` is safe.

Parameters

<code>in, out</code>	<code>operation</code>	The <code>psa_key_agreement_iop_t</code> to use
----------------------	------------------------	---

Return values

<code>PSA_SUCCESS</code>	The operation was aborted successfully.
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> .

6.15.3.2 `psa_key_agreement_iop_complete()`

```
psa_status_t psa_key_agreement_iop_complete (
    psa_key_agreement_iop_t * operation,
    mbedtls_svc_key_id_t * key )
```

Continue and eventually complete the action of key agreement, in an interruptible manner.

See also

[psa_key_agreement_iop_setup\(\)](#)

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with [psa_key_agreement_iop_setup\(\)](#) is equivalent to [psa_raw_key_agreement\(\)](#) but this function can return early and resume according to the limit set with [psa_interruptible_set_max_ops\(\)](#) to reduce the maximum time spent in a function call.

Users should call this function on the same operation object repeatedly while it returns [PSA_OPERATION_INCOMPLETE](#), stopping when it returns either [PSA_SUCCESS](#) or an error. Alternatively users can call [psa_key_agreement_iop_abort\(\)](#) at any point if they no longer want the result.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_key_agreement_iop_abort\(\)](#).

Parameters

in, out	<i>operation</i>	The psa_key_agreement_iop_t to use. This must be initialized first, and have had psa_key_agreement_iop_start() called with it first.
out	<i>key</i>	On success, an identifier for the newly created key. On failure this will be set to PSA_KEY_ID_NULL .

Return values

PSA_SUCCESS	The operation is complete and <i>key</i> contains the shared secret. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
PSA_OPERATION_INCOMPLETE	Operation was interrupted due to the setting of psa_interruptible_set_max_ops() . There is still work to be done. Call this function again with the same operation object.
PSA_ERROR_ALREADY_EXISTS	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
PSA_ERROR_INVALID_SIGNATURE	The calculation was performed successfully, but the passed signature is not a valid signature.
PSA_ERROR_BAD_STATE	An operation was not previously started on this context via psa_key_agreement_iop_start() .
PSA_ERROR_NOT_SUPPORTED	
PSA_ERROR_INVALID_ARGUMENT	
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_DATA_CORRUPT	
PSA_ERROR_DATA_INVALID	
PSA_ERROR_INSUFFICIENT_ENTROPY	

Return values

<code>PSA_ERROR_BAD_STATE</code>	The following conditions can result in this error: • The library has not been previously initialized by <code>psa_crypto_init()</code>. • The operation state is not valid: it must be inactive.
--	--

6.15.3.3 `psa_key_agreement_iop_get_num_ops()`

```
uint32_t psa_key_agreement_iop_get_num_ops (
    psa_key_agreement_iop_t * operation )
```

Get the number of ops that a key agreement operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [`psa\_key\_agreement\_iop\_abort\(\)`](#) on the operation, a value of 0 will be returned.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

This is a helper provided to help you tune the value passed to [`psa\_interruptible\_set\_max\_ops\(\)`](#).

Parameters

<i>operation</i>	The <code>psa_key_agreement_iop_t</code> to use. This must be initialized first.
------------------	--

Returns

Number of ops that the operation has taken so far.

6.15.3.4 `psa_key_agreement_iop_setup()`

```
psa_status_t psa_key_agreement_iop_setup (
    psa_key_agreement_iop_t * operation,
    mbedtls_svc_key_id_t private_key,
    const uint8_t * peer_key,
    size_t peer_key_length,
    psa_algorithm_t alg,
    const psa_key_attributes_t * attributes )
```

Start a key agreement operation, in an interruptible manner.

See also

[psa_key_agreement_iop_complete\(\)](#)

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

The raw result of a key agreement algorithm such elliptic curve Diffie-Hellman has biases and should not be used directly as key material. It should instead be passed as input to a key derivation algorithm.

Note

This function combined with [psa_key_agreement_iop_complete\(\)](#) is equivalent to [psa_raw_key_agreement\(\)](#) but [psa_key_agreement_iop_complete\(\)](#) can return early and resume according to the limit set with [psa_interruptible_set_max_ops\(\)](#) to reduce the maximum time spent in a function.

Users should call [psa_key_agreement_iop_complete\(\)](#) repeatedly on the same operation object after a successful call to this function until [psa_key_agreement_iop_complete\(\)](#) either returns [PSA_SUCCESS](#) or an error. [psa_key_agreement_iop_complete\(\)](#) will return [PSA_OPERATION_INCOMPLETE](#) if there is more work to do. Alternatively users can call [psa_key_agreement_iop_abort\(\)](#) at any point if they no longer want the result.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_key_agreement_iop_abort\(\)](#).

Parameters

in, out	<i>operation</i>	The psa_key_agreement_iop_t to use. This must be initialized as per the documentation for psa_key_agreement_iop_t , and be inactive.
	<i>private_key</i>	Identifier of the private key to use. It must allow the usage PSA_KEY_USAGE_DERIVE .
in	<i>peer_key</i>	Public key of the peer. It must be in the same format that psa_import_key() accepts. The standard formats for public keys are documented in the documentation of psa_export_public_key() . The peer key data is parsed with the type PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) where <i>type</i> is the type of <i>private_key</i> , and with the same bit-size as <i>private_key</i> .
	<i>peer_key_length</i>	Size of <i>peer_key</i> in bytes.
	<i>alg</i>	The key agreement algorithm to compute (a PSA_ALG_XXX value such that PSA_ALG_IS_KEY_AGREEMENT(alg) is true).

Parameters

in	<i>attributes</i>	The attributes for the new key. The following attributes are required for all keys: - The key type, which must be one of PSA_KEY_TYPE_DERIVE, PSA_KEY_TYPE_RAW_DATA, PSA_KEY_TYPE_HMAC or PSA_KEY_TYPE_PASSWORD. The following attributes must be set for keys used in cryptographic operations: - The key permitted-algorithm policy - The key usage flags The following attributes must be set for keys that do not use the default volatile lifetime: - The key lifetime - The key identifier is required for a key with a persistent lifetime The following attributes are optional: - If the key size is nonzero, it must be equal to the output size of the key agreement, in bits. The output size, in bits, of the key agreement is $8 * \text{PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE}(\text{type}, \text{bits})$, where <i>type</i> and <i>bits</i> are the type and bit-size of <i>private_key</i>.
----	-------------------	--

Note

attributes is an input parameter, it is not updated with the final key attributes. The final attributes of the new key can be queried by calling [psa_get_key_attributes\(\)](#) with the key's identifier.

This function clears the number of ops completed as part of the operation. Please ensure you copy this value via [psa_key_agreement_iop_get_num_ops\(\)](#) if required before calling.

Return values

PSA_SUCCESS	The operation started successfully. Call psa_key_agreement_iop_complete() with the same context to complete the operation.
PSA_ERROR_BAD_STATE	Another operation has already been started on this context, and is still in progress.
PSA_ERROR_NOT_PERMITTED	The following conditions can result in this error: Either the <i>private_key</i> does not have the PSA_KEY_USAGE_DERIVE flag, or it does not permit the requested algorithm.
PSA_ERROR_INVALID_HANDLE	<i>private_key</i> is not a valid key identifier.
PSA_ERROR_ALREADY_EXISTS	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

Return values

<i>PSA_ERROR_INVALID_ARGUMENT</i>	The following conditions can result in this error: • <code>alg</code> is not a key agreement algorithm. • <code>private_key</code> is not compatible with <code>alg</code>. • <code>peer_key</code> is not a valid public key corresponding to <code>private_key</code>. • The output key attributes in <code>attributes</code> are not valid: – The key type is not valid for key agreement output. – The key size is nonzero, and is not the size of the shared secret. – The key lifetime is invalid. – The key identifier is not valid for the key lifetime. – The key usage flags include invalid values. – The key's permitted-usage algorithm is invalid. – The key attributes, as a whole, are invalid.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The following conditions can result in this error: • <code>alg</code> is not supported. • <code>private_key</code> is not supported for use with <code>alg</code>. • Only elliptic curve Diffie-Hellman with ECC keys is supported, not finite field Diffie-Hellman with DH keys.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_INSUFFICIENT_STORAGE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The following conditions can result in this error: • The library has not been previously initialized by <code>psa_crypto_init()</code>. • The operation state is not valid: it must be inactive.

6.16 Interruptible Key Generation

Typedefs

- typedef struct [psa_generate_key_iop_s](#) [psa_generate_key_iop_t](#)

Functions

- uint32_t [psa_generate_key_iop_get_num_ops](#) ([psa_generate_key_iop_t](#) *operation)
Get the number of ops that a key generation operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa_generate_key_iop_abort\(\)](#) on the operation, a value of 0 will be returned.
- [psa_status_t](#) [psa_generate_key_iop_setup](#) ([psa_generate_key_iop_t](#) *operation, const [psa_key_attributes_t](#) *attributes)
Start a key generation operation, in an interruptible manner.
- [psa_status_t](#) [psa_generate_key_iop_complete](#) ([psa_generate_key_iop_t](#) *operation, [mbedtls_svc_key_id_t](#) *key)
Continue and eventually complete the action of key generation, in an interruptible manner.
- [psa_status_t](#) [psa_generate_key_iop_abort](#) ([psa_generate_key_iop_t](#) *operation)
Abort a key generation operation.

6.16.1 Detailed Description

6.16.2 Typedef Documentation

6.16.2.1 [psa_generate_key_iop_t](#)

```
typedef struct psa\_generate\_key\_iop\_s psa\_generate\_key\_iop\_t
```

The type of the state data structure for interruptible key generation operations.

Before calling any function on an interruptible key generation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa\_generate\_key\_iop\_t operation;  
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa\_generate\_key\_iop\_t operation = {0};
```
- Initialize the structure to the initializer [PSA_GENERATE_KEY_IOP_INIT](#), for example:

```
psa\_generate\_key\_iop\_t operation = PSA\_GENERATE\_KEY\_IOP\_INIT;
```
- Assign the result of the function [psa_generate_key_iop_init\(\)](#) to the structure, for example:

```
psa\_generate\_key\_iop\_t operation;  
operation = psa\_generate\_key\_iop\_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 5604 of file `crypto.h`.

6.16.3 Function Documentation

6.16.3.1 `psa_generate_key_iop_abort()`

```
psa_status_t psa_generate_key_iop_abort (
    psa_generate_key_iop_t * operation )
```

Abort a key generation operation.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function clears the number of ops completed as part of the operation. Please ensure you copy this value via `psa_generate_key_iop_get_num_ops()` if required before calling.

Aborting an operation frees all associated resources except for the operation structure itself. Once aborted, the operation object can be reused for another operation by calling `psa_generate_key_iop_setup()` again.

You may call this function any time after the operation object has been initialized. In particular, calling `psa_generate_key_iop_abort()` after the operation has already been terminated by a call to `psa_generate_key_iop_abort()` or `psa_generate_key_iop_complete()` is safe.

Parameters

<i>in, out</i>	<i>operation</i>	The <code>psa_key_agreement_iop_t</code> to use
----------------	------------------	---

Return values

<code>PSA_SUCCESS</code>	The operation was aborted successfully.
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> .

6.16.3.2 `psa_generate_key_iop_complete()`

```
psa_status_t psa_generate_key_iop_complete (
    psa_generate_key_iop_t * operation,
    mbedtls_svc_key_id_t * key )
```

Continue and eventually complete the action of key generation, in an interruptible manner.

See also

[psa_generate_key_iop_setup\(\)](#)

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with [psa_generate_key_iop_setup\(\)](#) is equivalent to [psa_generate_key\(\)](#) but this function can return early and resume according to the limit set with [psa_interruptible_set_max_ops\(\)](#) to reduce the maximum time spent in a function call.

Users should call this function on the same operation object repeatedly whilst it returns [PSA_OPERATION_INCOMPLETE](#), stopping when it returns either [PSA_SUCCESS](#) or an error. Alternatively users can call [psa_generate_key_iop_abort\(\)](#) at any point if they no longer want the result.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_generate_key_iop_abort\(\)](#).

Parameters

in, out	<i>operation</i>	The psa_generate_key_iop_t to use. This must be initialized first, and have had psa_generate_key_iop_setup() called with it first.
out	<i>key</i>	On success, an identifier for the newly created key, on failure this will be set to PSA_KEY_ID_NULL .

Return values

PSA_SUCCESS	The operation is complete and <i>key</i> contains the new key. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
PSA_OPERATION_INCOMPLETE	Operation was interrupted due to the setting of psa_interruptible_set_max_ops() . There is still work to be done. Call this function again with the same operation object.
PSA_ERROR_ALREADY_EXISTS	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
PSA_ERROR_NOT_SUPPORTED	
PSA_ERROR_INVALID_ARGUMENT	
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_HARDWARE_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_DATA_CORRUPT	
PSA_ERROR_DATA_INVALID	
PSA_ERROR_INSUFFICIENT_ENTROPY	

Return values

<code>PSA_ERROR_BAD_STATE</code>	The following conditions can result in this error: • The library has not been previously initialized by <code>psa_crypto_init()</code>. • The operation state is not valid: it must be active.
--	--

6.16.3.3 `psa_generate_key_iop_get_num_ops()`

```
uint32_t psa_generate_key_iop_get_num_ops (
    psa_generate_key_iop_t * operation )
```

Get the number of ops that a key generation operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [`psa\_generate\_key\_iop\_abort\(\)`](#) on the operation, a value of 0 will be returned.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises. This is a helper provided to help you tune the value passed to [`psa\_interruptible\_set\_max\_ops\(\)`](#).

Parameters

<i>operation</i>	The <code>psa_generate_key_iop_t</code> to use. This must be initialized first.
------------------	---

Returns

Number of ops that the operation has taken so far.

6.16.3.4 `psa_generate_key_iop_setup()`

```
psa_status_t psa_generate_key_iop_setup (
    psa_generate_key_iop_t * operation,
    const psa_key_attributes_t * attributes )
```

Start a key generation operation, in an interruptible manner.

See also

[`psa\_generate\_key\_iop\_complete\(\)`](#)

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with `psa_generate_key_iop_complete()` is equivalent to `psa_generate_key()` but `psa_generate_key_iop_complete()` can return early and resume according to the limit set with `psa_interruptible_set_max_ops()` to reduce the maximum time spent in a function.

Users should call `psa_generate_key_iop_complete()` repeatedly on the same operation object after a successful call to this function until `psa_generate_key_iop_complete()` either returns `PSA_SUCCESS` or an error. `psa_generate_key_iop_complete()` will return `PSA_OPERATION_INCOMPLETE` if there is more work to do. Alternatively users can call `psa_generate_key_iop_abort()` at any point if they no longer want the result.

This function clears the number of ops completed as part of the operation. Please ensure you copy this value via `psa_generate_key_iop_get_num_ops()` if required before calling.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_generate_key_iop_abort()`.

Only asymmetric key pairs are supported. (See `attributes`.)

Parameters

in, out	<i>operation</i>	The <code>psa_generate_key_iop_t</code> to use. This must be initialized as per the documentation for <code>psa_generate_key_iop_t</code> , and be inactive.
in	<i>attributes</i>	The attributes for the new key. The following attributes are required for all keys: • The key type. It must be an asymmetric key-pair. • The key size. It must be a valid size for the key type. The following attributes must be set for keys used in cryptographic operations: • The key permitted-algorithm policy. • The key usage flags. The following attributes must be set for keys that do not use the default volatile lifetime: • The key lifetime. • The key identifier is required for a key with a persistent lifetime,

Note

`attributes` is an input parameter, it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

Return values

<code>PSA_SUCCESS</code>	The operation started successfully. Call <code>psa_generate_key_iop_complete()</code> with the same context to complete the operation.
<code>PSA_ERROR_ALREADY_EXISTS</code>	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier
<code>PSA_ERROR_NOT_SUPPORTED</code>	The key attributes, as a whole, are not supported, either in general or in the specified storage location.

Return values

<i>PSA_ERROR_INVALID_ARGUMENT</i>	The following conditions can result in this error: • The key type is invalid, or is an asymmetric public key type. • The key size is not valid for the key type. • The key lifetime is invalid. • The key identifier is not valid for the key lifetime. • The key usage flags include invalid values. • The key's permitted-usage algorithm is invalid. • The key attributes, as a whole, are invalid.
<i>PSA_ERROR_NOT_PERMITTED</i>	Creating a key with the specified attributes is not permitted.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_HARDWARE_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_INSUFFICIENT_STORAGE</i>	
<i>PSA_ERROR_BAD_STATE</i>	The following conditions can result in this error: • The library has not been previously initialized by <code>psa_crypto_init()</code>. • The operation state is not valid: it must be inactive.

6.17 Interruptible public-key export

Typedefs

- typedef struct `psa_export_public_key_iop_s` `psa_export_public_key_iop_t`

Functions

- uint32_t `psa_export_public_key_iop_get_num_ops` (`psa_export_public_key_iop_t` *operation)
Get the number of ops that an export public-key operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling `psa_export_public_key_iop_abort()` on the operation, a value of 0 will be returned.
- `psa_status_t` `psa_export_public_key_iop_setup` (`psa_export_public_key_iop_t` *operation, `mbdts_svc_key_id_t` key)
Start an interruptible operation to export a public key or the public part of a key pair in binary format.
- `psa_status_t` `psa_export_public_key_iop_complete` (`psa_export_public_key_iop_t` *operation, uint8_t *data, size_t data_size, size_t *data_length)
Continue and eventually complete the action of exporting a public key, in an interruptible manner.
- `psa_status_t` `psa_export_public_key_iop_abort` (`psa_export_public_key_iop_t` *operation)
Abort an interruptible public-key export operation.

6.17.1 Detailed Description

6.17.2 Typedef Documentation

6.17.2.1 `psa_export_public_key_iop_t`

```
typedef struct psa_export_public_key_iop_s psa_export_public_key_iop_t
```

The type of the state data structure for interruptible public-key export operations.

Before calling any function on an interruptible export public-key object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_export_public_key_iop_t operation;
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_export_public_key_iop_t operation = {0};
```
- Initialize the structure to the initializer `PSA_EXPORT_PUBLIC_KEY_IOP_INIT`, for example:

```
psa_export_public_key_iop_t operation = PSA_EXPORT_PUBLIC_KEY_IOP_INIT;
```
- Assign the result of the function `psa_export_public_key_iop_init()` to the structure, for example:

```
psa_export_public_key_iop_t operation;
operation = psa_export_public_key_iop_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 5894 of file `crypto.h`.

6.17.3 Function Documentation

6.17.3.1 `psa_export_public_key_iop_abort()`

```
psa_status_t psa_export_public_key_iop_abort (
    psa_export_public_key_iop_t * operation )
```

Abort an interruptible public-key export operation.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function clears the number of ops completed as part of the operation. Please ensure you copy this value via `psa_export_public_key_iop_get_num_ops()` if required before calling.

Aborting an operation frees all associated resources except for the operation structure itself. Once aborted, the operation object can be reused for another operation by calling `psa_export_public_key_iop_setup()` again.

You may call this function any time after the operation object has been initialized. In particular, calling `psa_export_public_key_iop_abort()` after the operation has already been terminated by a call to `psa_export_public_key_iop_abort()` or `psa_export_public_key_iop_complete()` is safe.

Parameters

<code>in, out</code>	<code>operation</code>	The <code>psa_export_public_key_iop_t</code> to use
----------------------	------------------------	---

Return values

<code>PSA_SUCCESS</code>	The operation was aborted successfully.
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> .
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	

6.17.3.2 `psa_export_public_key_iop_complete()`

```
psa_status_t psa_export_public_key_iop_complete (
    psa_export_public_key_iop_t * operation,
    uint8_t * data,
    size_t data_size,
    size_t * data_length )
```

Continue and eventually complete the action of exporting a public key, in an interruptible manner.

See also

[psa_export_public_key_iop_setup\(\)](#)

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with [psa_export_public_key_iop_setup\(\)](#) is equivalent to [psa_export_public_key\(\)](#) but this function can return early and resume according to the limit set with [psa_interruptible_set_max_ops\(\)](#) to reduce the maximum time spent in a function call.

Users should call this function on the same operation object repeatedly whilst it returns [PSA_OPERATION_INCOMPLETE](#), stopping when it returns either [PSA_SUCCESS](#) or an error. Alternatively users can call [psa_export_public_key_iop_abort\(\)](#) at any point if they no longer want the result.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_export_public_key_iop_abort\(\)](#).

Parameters

in, out	<i>operation</i>	The psa_export_public_key_iop_t to use. This must be initialized first, and have had psa_export_public_key_iop_setup() called with it first.
out	<i>data</i>	Buffer where the key data is to be written.
in	<i>data_size</i>	Size of the <i>data</i> buffer in bytes. This must be appropriate for the key: - The required output size is PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE(type, bits) where <i>type</i> is the key type and <i>bits</i> is the key size in bits. PSA_EXPORT_PUBLIC_KEY_MAX_SIZE evaluates to the maximum output size of any supported public key or public part of a key pair.
out	<i>data_length</i>	On success, the number of bytes that make up the key data.

Return values

PSA_SUCCESS	Success. The first (<i>*data_length</i>) bytes of data contain the exported public key.
PSA_ERROR_BAD_STATE	The following conditions can result in this error: - The library has not been previously initialized by psa_crypto_init(). - The operation state is not valid: it must be active.
PSA_ERROR_BUFFER_TOO_SMALL	The size of the data buffer is too small. PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE() , PSA_EXPORT_PUBLIC_KEY_MAX_SIZE .
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_DATA_CORRUPT	

Return values

<code>PSA_ERROR_DATA_INVALID</code>	
<code>PSA_OPERATION_INCOMPLETE</code>	Operation was interrupted due to the setting of <code>psa_interruptible_set_max_ops()</code> . There is still work to be done. Call this function again with the same operation object.

6.17.3.3 `psa_export_public_key_iop_get_num_ops()`

```
uint32_t psa_export_public_key_iop_get_num_ops (
    psa_export_public_key_iop_t * operation )
```

Get the number of ops that an export public-key operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [`psa\_export\_public\_key\_iop\_abort\(\)`](#) on the operation, a value of 0 will be returned.

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises. This is a helper provided to help you tune the value passed to [`psa\_interruptible\_set\_max\_ops\(\)`](#).

Parameters

<i>operation</i>	The <code>psa_export_public_key_iop_t</code> to use. This must be initialized first.
------------------	--

Returns

Number of ops that the operation has taken so far.

6.17.3.4 `psa_export_public_key_iop_setup()`

```
psa_status_t psa_export_public_key_iop_setup (
    psa_export_public_key_iop_t * operation,
    mbedtls_svc_key_id_t key )
```

Start an interruptible operation to export a public key or the public part of a key pair in binary format.

See also

[`psa\_export\_public\_key\_iop\_complete\(\)`](#)

Warning

This is a beta API, and thus subject to change at any point. It is not bound by the usual interface stability promises.

Note

This function combined with `psa_export_public_key_iop_complete()` is equivalent to `psa_export_public_key()` but `psa_export_public_key_iop_complete()` can return early and resume according to the limit set with `psa_interruptible_set_max_ops()` to reduce the maximum time spent in a function.

Users should call `psa_export_public_key_iop_complete()` repeatedly on the same operation object after a successful call to this function until `psa_export_public_key_iop_complete()` either returns `PSA_SUCCESS` or an error. `psa_export_public_key_iop_complete()` will return `PSA_OPERATION_INCOMPLETE` if there is more work to do. Alternatively users can call `psa_export_public_key_iop_abort()` at any point if they no longer want the result.

This function clears the number of ops completed as part of the operation. Please ensure you copy this value via `psa_export_public_key_iop_get_num_ops()` if required before calling.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_export_public_key_iop_abort()`.

Parameters

<code>in, out</code>	<code>operation</code>	The <code>psa_export_public_key_iop_t</code> to use. This must be initialized as per the documentation for <code>psa_export_public_key_iop_t</code> , and be inactive.
<code>in</code>	<code>key</code>	Identifier of the key to export.

Return values

<code>PSA_SUCCESS</code>	The operation started successfully. Call <code>psa_export_public_key_iop_complete()</code> with the same context to complete the operation.
<code>PSA_ERROR_INVALID_HANDLE</code>	<code>key</code> is not a valid key identifier.
<code>PSA_ERROR_INVALID_ARGUMENT</code>	The key is neither a public key nor a key pair.
<code>PSA_ERROR_NOT_SUPPORTED</code>	The following conditions can result in this error: - The key's storage location does not support export of the key. - The implementation does not support export of keys with this key type.
<code>PSA_ERROR_BAD_STATE</code>	The following conditions can result in this error: - The library has not been previously initialized by <code>psa_crypto_init()</code>. - The operation state is not valid: it must be inactive.
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_STORAGE_FAILURE</code>	
<code>PSA_ERROR_DATA_CORRUPT</code>	
<code>PSA_ERROR_DATA_INVALID</code>	
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	

6.18 Random and entropy drivers

Macros

- `#define PSA_DRIVER_GET_ENTROPY_FLAGS_NONE ((psa_driver_get_entropy_flags_t) 0)`

Typedefs

- `typedef uint32_t psa_driver_get_entropy_flags_t`

6.18.1 Detailed Description

6.18.2 Macro Definition Documentation

6.18.2.1 PSA_DRIVER_GET_ENTROPY_FLAGS_NONE

```
#define PSA_DRIVER_GET_ENTROPY_FLAGS_NONE ((psa_driver_get_entropy_flags_t) 0)
```

Flags requesting the default behavior for a "get_entropy" driver entry point. This is equivalent to 0.

See also

[psa_driver_get_entropy_flags_t](#)

Definition at line 41 of file `crypto_driver_random.h`.

6.18.3 Typedef Documentation

6.18.3.1 psa_driver_get_entropy_flags_t

```
typedef uint32_t psa_driver_get_entropy_flags_t
```

The type of the `flags` argument to "get_entropy" driver entry points.

This implementation does not support any flags yet.

Definition at line 34 of file `crypto_driver_random.h`.

6.19 Random generator

Functions

- [psa_status_t psa_random_reseed](#) (const uint8_t *perso, size_t perso_size)
- [psa_status_t psa_random_deplete](#) (void)
- [psa_status_t psa_random_set_prediction_resistance](#) (unsigned enabled)

6.19.1 Detailed Description

6.19.2 Function Documentation

6.19.2.1 `psa_random_deplete()`

```
psa_status_t psa_random_deplete (
    void )
```

Force a reseed of the PSA random generator the next time it is used.

The entropy source(s) are the ones configured at compile time.

The random generator is always seeded automatically before use, and it is reseeded as needed based on the configured policy, so most applications do not need to call this function.

This function has a similar purpose as [psa_random_reseed\(\)](#), but the reseed will happen the next time the random generator is used. The advantage of this function is that it does not fail unless the system is in an unintended state, so it can be used in contexts where propagating errors is difficult.

Note

This function has no effect when `#MBEDTLS_PSA_CRYPTO_EXTERNAL_RNG` is enabled.

If prediction resistance is enabled (either explicitly, or because the reseed interval is set to 1), calling this function is unnecessary since the random generator will always reseed anyway.

Return values

PSA_SUCCESS	The reseed succeeded.
PSA_ERROR_BAD_STATE	The PSA random generator is not active.
PSA_ERROR_NOT_SUPPORTED	PSA uses an external random generator because the compilation option <code>#MBEDTLS_PSA_CRYPTO_EXTERNAL_RNG</code> is enabled. This configuration does not support explicit reseeding.

6.19.2.2 `psa_random_reseed()`

```
psa_status_t psa_random_reseed (
```

```
const uint8_t * perso,
size_t perso_size )
```

Force an immediate reseed of the PSA random generator.

The entropy source(s) are the ones configured at compile time.

The random generator is always seeded automatically before use, and it is reseeded as needed based on the configured policy, so most applications do not need to call this function.

The main reason to call this function is in scenarios where the process state is cloned (i.e. duplicated) while the random generator is active. In such scenarios, you must call this function in every clone of the original process before performing any cryptographic operation that uses randomness. (Note that any operation that uses a private or secret key may use randomness internally even if the result is not randomized, but hashing and signature verification are ok.) For example:

- If the process is part of a live virtual machine that is cloned, call this function after cloning so that the new instance has a distinct random generator state.
- If the process is part of a hibernated image that may be resumed multiple times, call this function after resuming so that each resumed instance has a distinct random generator state.
- If the process is cloned through the `fork()` system call, the child process should call this function before using the random generator.

An additional consideration applies in configurations where there is no actual entropy source, only a nonvolatile seed (i.e. `#MBEDTLS_ENTROPY_NV_SEED` and `#MBEDTLS_ENTROPY_NO_SOURCES_OK` are enabled, and `#MBEDTLS_PSA_BUILTIN_GET_ENTROPY` and `#MBEDTLS_PSA_DRIVER_GET_ENTROPY` are disabled). In such configurations, simply calling `psa_random_reseed()` in multiple cloned processes would result in the same random generator state in all the clones. To avoid this, in such configurations, you must pass a unique `perso` string in every clone.

Note

This function has no effect when the compilation option `#MBEDTLS_PSA_CRYPTO_EXTERNAL_RNG` is enabled.

In client-server builds, this function may not be available from clients, since the decision to reseed is generally based on the server state.

If the entropy source fails, the random generator remains usable: subsequent calls to generate random data will succeed until the random generator itself decides to reseed. If you want to force a reseed, either treat the failure as a fatal error, or call `psa_random_deplete()` instead of this function (or in addition).

Parameters

in	<i>perso</i>	A personalization string, i.e. a byte string to inject into the random generator state in addition to entropy obtained from the normal source(s). In most cases, it is fine for <i>perso</i> to be empty. The main use case for a personalization string is when the random generator state is cloned, as described above, and there is no actual entropy source.
	<i>perso_size</i>	Length of <i>perso</i> in bytes.

Return values

<code>PSA_SUCCESS</code>	The reseed succeeded.
<code>PSA_ERROR_BAD_STATE</code>	The PSA random generator is not active.

Return values

<i>PSA_ERROR_NOT_SUPPORTED</i>	PSA uses an external random generator because the compilation option <code>#MBEDTLS_PSA_CRYPTO_EXTERNAL_RNG</code> is enabled. This configuration does not support explicit reseeding.
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	The entropy source failed.

6.19.2.3 `psa_random_set_prediction_resistance()`

```
psa_status_t psa_random_set_prediction_resistance (
    unsigned enabled )
```

Enable or disable prediction resistance in the PSA random generator.

When prediction resistance is enabled, the random generator injects extra entropy before each request regardless of its size. As a consequence, a temporary compromise of the random generator state does not, by itself, compromise future steps. Furthermore, duplicating the random generator state (because the running application instance is cloned) is safe since it will not lead to identical random generator outputs in the clones.

When prediction resistance is disabled, the random generator injects extra entropy periodically only as determined by `#MBEDTLS_PSA_RNG_RESEED_INTERVAL`.

Prediction resistance is disabled by default, although setting `#MBEDTLS_PSA_RNG_RESEED_INTERVAL` to 1 satisfies the prediction resistance property even when the specific setting for prediction resistance is disabled.

Note

This function has no effect when `#MBEDTLS_PSA_CRYPTO_EXTERNAL_RNG` is enabled.

Prediction resistance cannot be enabled when the only entropy source is a nonvolatile seed, since prediction resistance is effectively impossible to achieve without actual entropy.

Parameters

<i>enabled</i>	1 to enable prediction resistance. 0 to disable prediction resistance.
----------------	--

Return values

<i>PSA_SUCCESS</i>	The PSA random generator is active, and prediction resistance has been changed to the desired option.
<i>PSA_ERROR_BAD_STATE</i>	The PSA random generator is not active.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<code>enabled</code> is not valid.
<i>PSA_ERROR_NOT_SUPPORTED</i>	PSA uses an external random generator because the compilation option <code>#MBEDTLS_PSA_CRYPTO_EXTERNAL_RNG</code> is enabled. Or, the random generator only has a nonvolatile seed but no entropy source, and prediction resistance has been requested.

6.20 Built-in keys

Macros

- `#define MBEDTLS_PSA_KEY_ID_BUILTIN_MIN ((psa_key_id_t) 0x7fff0000)`
- `#define MBEDTLS_PSA_KEY_ID_BUILTIN_MAX ((psa_key_id_t) 0x7ffffeff)`

Typedefs

- `typedef uint64_t psa_drv_slot_number_t`

6.20.1 Detailed Description

6.20.2 Macro Definition Documentation

6.20.2.1 MBEDTLS_PSA_KEY_ID_BUILTIN_MAX

```
#define MBEDTLS_PSA_KEY_ID_BUILTIN_MAX ((psa_key_id_t) 0x7ffffeff)
```

The maximum value for a key identifier that is built into the implementation.

See [MBEDTLS_PSA_KEY_ID_BUILTIN_MIN](#) for more information.

Definition at line 501 of file `crypto_extra.h`.

6.20.2.2 MBEDTLS_PSA_KEY_ID_BUILTIN_MIN

```
#define MBEDTLS_PSA_KEY_ID_BUILTIN_MIN ((psa_key_id_t) 0x7fff0000)
```

The minimum value for a key identifier that is built into the implementation.

The range of key identifiers from [MBEDTLS_PSA_KEY_ID_BUILTIN_MIN](#) to [MBEDTLS_PSA_KEY_ID_BUILTIN_MAX](#) within the range from [PSA_KEY_ID_VENDOR_MIN](#) and [PSA_KEY_ID_VENDOR_MAX](#) and must not intersect with any other set of implementation-chosen key identifiers.

This value is part of the library's API since changing it would invalidate the values of built-in key identifiers in applications.

Definition at line 494 of file `crypto_extra.h`.

6.20.3 Typedef Documentation

6.20.3.1 psa_drv_slot_number_t

```
typedef uint64_t psa_drv_slot_number_t
```

A slot number identifying a key in a driver.

Values of this type are used to identify built-in keys.

Definition at line 507 of file `crypto_extra.h`.

6.21 Functions defined by a client provider

The functions in this group are meant to be implemented by providers of the PSA Crypto client interface. They are provided by the library when `#MBEDTLS_PSA_CRYPTOC_C` is enabled.

The functions in this group are meant to be implemented by providers of the PSA Crypto client interface. They are provided by the library when `#MBEDTLS_PSA_CRYPTOC_C` is enabled.

Note

All functions in this group are experimental, as using alternative client interface providers is experimental.

6.22 Password-authenticated key exchange (PAKE)

This is a proposed PAKE interface for the PSA Crypto API. It is not part of the official PSA Crypto API yet.

Macros

- `#define PSA_PAKE_ROLE_NONE ((psa_pake_role_t) 0x00)`
- `#define PSA_PAKE_ROLE_FIRST ((psa_pake_role_t) 0x01)`
- `#define PSA_PAKE_ROLE_SECOND ((psa_pake_role_t) 0x02)`
- `#define PSA_PAKE_ROLE_CLIENT ((psa_pake_role_t) 0x11)`
- `#define PSA_PAKE_ROLE_SERVER ((psa_pake_role_t) 0x12)`
- `#define PSA_PAKE_PRIMITIVE_TYPE_ECC ((psa_pake_primitive_type_t) 0x01)`
- `#define PSA_PAKE_PRIMITIVE_TYPE_DH ((psa_pake_primitive_type_t) 0x02)`
- `#define PSA_PAKE_PRIMITIVE(pake_type, pake_family, pake_bits)`
- `#define PSA_PAKE_STEP_KEY_SHARE ((psa_pake_step_t) 0x01)`
- `#define PSA_PAKE_STEP_ZK_PUBLIC ((psa_pake_step_t) 0x02)`
- `#define PSA_PAKE_STEP_ZK_PROOF ((psa_pake_step_t) 0x03)`
- `#define PSA_PAKE_STEP_CONFIRM ((psa_pake_step_t) 0x04)`

Typedefs

- `typedef uint8_t psa_pake_role_t`
Encoding of the application role of PAKE.
- `typedef uint8_t psa_pake_step_t`
- `typedef uint8_t psa_pake_primitive_type_t`
- `typedef uint8_t psa_pake_family_t`
Encoding of the family of the primitive associated with the PAKE.
- `typedef uint32_t psa_pake_primitive_t`
Encoding of the primitive associated with the PAKE.
- `typedef struct psa_pake_cipher_suite_s psa_pake_cipher_suite_t`
- `typedef struct psa_pake_operation_s psa_pake_operation_t`
- `typedef struct psa_crypto_driver_pake_inputs_s psa_crypto_driver_pake_inputs_t`
- `typedef struct psa_jpake_computation_stage_s psa_jpake_computation_stage_t`

Functions

- `psa_status_t psa_crypto_driver_pake_get_password_len (const psa_crypto_driver_pake_inputs_t *inputs, size_t *password_len)`
- `psa_status_t psa_crypto_driver_pake_get_password (const psa_crypto_driver_pake_inputs_t *inputs, uint8_t *buffer, size_t buffer_size, size_t *buffer_length)`
- `psa_status_t psa_crypto_driver_pake_get_user_len (const psa_crypto_driver_pake_inputs_t *inputs, size_t *user_len)`
- `psa_status_t psa_crypto_driver_pake_get_peer_len (const psa_crypto_driver_pake_inputs_t *inputs, size_t *peer_len)`
- `psa_status_t psa_crypto_driver_pake_get_user (const psa_crypto_driver_pake_inputs_t *inputs, uint8_t *user_id, size_t user_id_size, size_t *user_id_len)`
- `psa_status_t psa_crypto_driver_pake_get_peer (const psa_crypto_driver_pake_inputs_t *inputs, uint8_t *peer_id, size_t peer_id_size, size_t *peer_id_length)`
- `psa_status_t psa_crypto_driver_pake_get_cipher_suite (const psa_crypto_driver_pake_inputs_t *inputs, psa_pake_cipher_suite_t *cipher_suite)`

- [psa_status_t](#) [psa_pake_setup](#) ([psa_pake_operation_t](#) *operation, [mbedtls_svc_key_id_t](#) password_key, const [psa_pake_cipher_suite_t](#) *cipher_suite)
- [psa_status_t](#) [psa_pake_set_user](#) ([psa_pake_operation_t](#) *operation, const [uint8_t](#) *user_id, [size_t](#) user_id_len)
- [psa_status_t](#) [psa_pake_set_peer](#) ([psa_pake_operation_t](#) *operation, const [uint8_t](#) *peer_id, [size_t](#) peer_id_len)
- [psa_status_t](#) [psa_pake_set_role](#) ([psa_pake_operation_t](#) *operation, [psa_pake_role_t](#) role)
- [psa_status_t](#) [psa_pake_set_context](#) ([psa_pake_operation_t](#) *operation, const [uint8_t](#) *context, [size_t](#) context_len)
- [psa_status_t](#) [psa_pake_output](#) ([psa_pake_operation_t](#) *operation, [psa_pake_step_t](#) step, [uint8_t](#) *output, [size_t](#) output_size, [size_t](#) *output_length)
- [psa_status_t](#) [psa_pake_input](#) ([psa_pake_operation_t](#) *operation, [psa_pake_step_t](#) step, const [uint8_t](#) *input, [size_t](#) input_length)
- [psa_status_t](#) [psa_pake_get_shared_key](#) ([psa_pake_operation_t](#) *operation, const [psa_key_attributes_t](#) *attributes, [mbedtls_svc_key_id_t](#) *key)
- [psa_status_t](#) [psa_pake_abort](#) ([psa_pake_operation_t](#) *operation)

6.22.1 Detailed Description

This is a proposed PAKE interface for the PSA Crypto API. It is not part of the official PSA Crypto API yet.

Note

The content of this section is not part of the stable API and ABI of Mbed TLS and may change arbitrarily from version to version. Same holds for the corresponding macros [PSA_ALG_CATEGORY_PAKE](#) and [PSA_ALG_JPAKE](#).

6.22.2 Macro Definition Documentation

6.22.2.1 PSA_PAKE_PRIMITIVE

```
#define PSA_PAKE_PRIMITIVE(  
    pake_type,  
    pake_family,  
    pake_bits )
```

Value:

```
((pake_bits & 0xFFFF) != pake_bits) ? 0 :  
( (psa_pake_primitive_t) (((pake_type) << 24 |  
                           (pake_family) << 16) | (pake_bits))) )
```

Construct a PAKE primitive from type, family and bit-size.

Parameters

<i>pake_type</i>	The type of the primitive (value of type psa_pake_primitive_type_t).
<i>pake_family</i>	The family of the primitive (the type and interpretation of this parameter depends on <i>pake_type</i> , for more information consult the documentation of individual psa_pake_primitive_type_t constants).
<i>pake_bits</i>	The bit-size of the primitive (Value of type <i>size_t</i> . The interpretation of this parameter depends on <i>pake_family</i> , for more information consult the documentation of individual psa_pake_primitive_type_t constants).

Returns

The constructed primitive value of type [psa_pake_primitive_t](#). Return 0 if the requested primitive can't be encoded as [psa_pake_primitive_t](#).

Definition at line 973 of file `crypto_extra.h`.

6.22.2.2 PSA_PAKE_PRIMITIVE_TYPE_DH

```
#define PSA_PAKE_PRIMITIVE_TYPE_DH ((psa_pake_primitive_type_t) 0x02)
```

The PAKE primitive type indicating the use of Diffie-Hellman groups.

The values of the `family` and `bits` fields of the cipher suite identify a specific Diffie-Hellman group, using the same mapping that is used for Diffie-Hellman ([psa_dh_family_t](#)) keys.

(Here `family` means the value returned by `psa_pake_cs_get_family()` and `bits` means the value returned by `psa_pake_cs_get_bits()`.)

Input and output during the operation can involve group elements and scalar values:

1. The format for group elements is the same as for public keys on the specific group would be. For more information, consult the documentation of [psa_export_public_key\(\)](#).
2. The format for scalars is the same as for private keys on the specific group would be. For more information, consult the documentation of [psa_export_key\(\)](#).

Definition at line 952 of file `crypto_extra.h`.

6.22.2.3 PSA_PAKE_PRIMITIVE_TYPE_ECC

```
#define PSA_PAKE_PRIMITIVE_TYPE_ECC ((psa_pake_primitive_type_t) 0x01)
```

The PAKE primitive type indicating the use of elliptic curves.

The values of the `family` and `bits` fields of the cipher suite identify a specific elliptic curve, using the same mapping that is used for ECC ([psa_ecc_family_t](#)) keys.

(Here `family` means the value returned by `psa_pake_cs_get_family()` and `bits` means the value returned by `psa_pake_cs_get_bits()`.)

Input and output during the operation can involve group elements and scalar values:

1. The format for group elements is the same as for public keys on the specific curve would be. For more information, consult the documentation of [psa_export_public_key\(\)](#).
2. The format for scalars is the same as for private keys on the specific curve would be. For more information, consult the documentation of [psa_export_key\(\)](#).

Definition at line 932 of file `crypto_extra.h`.

6.22.2.4 PSA_PAKE_ROLE_CLIENT

```
#define PSA_PAKE_ROLE_CLIENT ((psa_pake_role_t) 0x11)
```

The client in an augmented PAKE.

Augmented PAKE algorithms need to differentiate between client and server.

Definition at line 906 of file `crypto_extra.h`.

6.22.2.5 PSA_PAKE_ROLE_FIRST

```
#define PSA_PAKE_ROLE_FIRST ((psa_pake_role_t) 0x01)
```

The first peer in a balanced PAKE.

Although balanced PAKE algorithms are symmetric, some of them needs an ordering of peers for the transcript calculations. If the algorithm does not need this, both [PSA_PAKE_ROLE_FIRST](#) and [PSA_PAKE_ROLE_SECOND](#) are accepted.

Definition at line 891 of file `crypto_extra.h`.

6.22.2.6 PSA_PAKE_ROLE_NONE

```
#define PSA_PAKE_ROLE_NONE ((psa_pake_role_t) 0x00)
```

A value to indicate no role in a PAKE algorithm. This value can be used in a call to [psa_pake_set_role\(\)](#) for symmetric PAKE algorithms which do not assign roles.

Definition at line 882 of file `crypto_extra.h`.

6.22.2.7 PSA_PAKE_ROLE_SECOND

```
#define PSA_PAKE_ROLE_SECOND ((psa_pake_role_t) 0x02)
```

The second peer in a balanced PAKE.

Although balanced PAKE algorithms are symmetric, some of them needs an ordering of peers for the transcript calculations. If the algorithm does not need this, either [PSA_PAKE_ROLE_FIRST](#) or [PSA_PAKE_ROLE_SECOND](#) are accepted.

Definition at line 900 of file `crypto_extra.h`.

6.22.2.8 PSA_PAKE_ROLE_SERVER

```
#define PSA_PAKE_ROLE_SERVER ((psa_pake_role_t) 0x12)
```

The server in an augmented PAKE.

Augmented PAKE algorithms need to differentiate between client and server.

Definition at line 912 of file `crypto_extra.h`.

6.22.2.9 PSA_PAKE_STEP_CONFIRM

```
#define PSA_PAKE_STEP_CONFIRM ((psa_pake_step_t) 0x04)
```

The key confirmation value.

This is only used with PAKE algorithms with an explicit key confirmation phase.

Refer to the documentation of the PAKE algorithm for information about the input format.

Definition at line 1038 of file `crypto_extra.h`.

6.22.2.10 PSA_PAKE_STEP_KEY_SHARE

```
#define PSA_PAKE_STEP_KEY_SHARE ((psa_pake_step_t) 0x01)
```

The key share being sent to or received from the peer.

The format for both input and output at this step is the same as for public keys on the group determined by the primitive ([psa_pake_primitive_t](#)) would be.

For more information on the format, consult the documentation of [psa_export_public_key\(\)](#).

For information regarding how the group is determined, consult the documentation [PSA_PAKE_PRIMITIVE](#).

Definition at line 990 of file `crypto_extra.h`.

6.22.2.11 PSA_PAKE_STEP_ZK_PROOF

```
#define PSA_PAKE_STEP_ZK_PROOF ((psa_pake_step_t) 0x03)
```

A Schnorr NIZKP proof.

This is the proof in the Schnorr Non-Interactive Zero-Knowledge Proof (the value denoted by the letter 'r' in RFC 8235).

Both for input and output, the value at this step is an integer less than the order of the group selected in the cipher suite. The format depends on the group as well:

- For Montgomery curves, the encoding is little endian.
- For everything else the encoding is big endian (see Section 2.3.8 of *SEC 1: Elliptic Curve Cryptography* at <https://www.secg.org/sec1-v2.pdf>).

In both cases leading zeroes are allowed as long as the length in bytes does not exceed the byte length of the group order.

For information regarding how the group is determined, consult the documentation [PSA_PAKE_PRIMITIVE](#).

Definition at line 1028 of file `crypto_extra.h`.

6.22.2.12 PSA_PAKE_STEP_ZK_PUBLIC

```
#define PSA_PAKE_STEP_ZK_PUBLIC ((psa_pake_step_t) 0x02)
```

A Schnorr NIZKP public key.

This is the ephemeral public key in the Schnorr Non-Interactive Zero-Knowledge Proof (the value denoted by the letter 'V' in RFC 8235).

The format for both input and output at this step is the same as for public keys on the group determined by the primitive ([psa_pake_primitive_t](#)) would be.

For more information on the format, consult the documentation of [psa_export_public_key\(\)](#).

For information regarding how the group is determined, consult the documentation [PSA_PAKE_PRIMITIVE](#).

Definition at line 1007 of file `crypto_extra.h`.

6.22.3 Typedef Documentation

6.22.3.1 `psa_crypto_driver_pake_inputs_t`

```
typedef struct psa_crypto_driver_pake_inputs_s psa_crypto_driver_pake_inputs_t
```

The type of input values for PAKE operations.

Definition at line 1402 of file `crypto_extra.h`.

6.22.3.2 `psa_jpake_computation_stage_t`

```
typedef struct psa_jpake_computation_stage_s psa_jpake_computation_stage_t
```

The type of computation stage for J-PAKE operations.

Definition at line 1405 of file `crypto_extra.h`.

6.22.3.3 `psa_pake_cipher_suite_t`

```
typedef struct psa_pake_cipher_suite_s psa_pake_cipher_suite_t
```

The type of the data structure for PAKE cipher suites.

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 1273 of file `crypto_extra.h`.

6.22.3.4 `psa_pake_family_t`

```
typedef uint8_t psa_pake_family_t
```

Encoding of the family of the primitive associated with the PAKE.

For more information see the documentation of individual `PSA_PAKE_PRIMITIVE_TYPE_XXX` constants.

Definition at line 870 of file `crypto_extra.h`.

6.22.3.5 `psa_pake_operation_t`

```
typedef struct psa_pake_operation_s psa_pake_operation_t
```

The type of the state data structure for PAKE operations.

Before calling any function on a PAKE operation object, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_pake_operation_t operation;  
memset(&operation, 0, sizeof(operation));
```
- Initialize the structure to logical zero values, for example:

```
psa_pake_operation_t operation = {0};
```
- Initialize the structure to the initializer `PSA_PAKE_OPERATION_INIT`, for example:

```
psa_pake_operation_t operation = PSA_PAKE_OPERATION_INIT;
```
- Assign the result of the function `psa_pake_operation_init()` to the structure, for example:

```
psa_pake_operation_t operation;  
operation = psa_pake_operation_init();
```

This is an implementation-defined `struct`. Applications should not make any assumptions about the content of this structure. Implementation details can change in future versions without notice.

Definition at line 1399 of file `crypto_extra.h`.

6.22.3.6 `psa_pake_primitive_t`

```
typedef uint32_t psa_pake_primitive_t
```

Encoding of the primitive associated with the PAKE.

For more information see the documentation of the `PSA_PAKE_PRIMITIVE` macro.

Definition at line 876 of file `crypto_extra.h`.

6.22.3.7 `psa_pake_primitive_type_t`

```
typedef uint8_t psa_pake_primitive_type_t
```

Encoding of the type of the PAKE's primitive.

Values defined by this standard will never be in the range 0x80-0xff. Vendors who define additional types must use an encoding in this range.

For more information see the documentation of individual `PSA_PAKE_PRIMITIVE_TYPE_XXX` constants.

Definition at line 863 of file `crypto_extra.h`.

6.22.3.8 `psa_pake_role_t`

```
typedef uint8_t psa_pake_role_t
```

Encoding of the application role of PAKE.

Encodes the application's role in the algorithm is being executed. For more information see the documentation of individual `PSA_PAKE_ROLE_XXX` constants.

Definition at line 845 of file `crypto_extra.h`.

6.22.3.9 `psa_pake_step_t`

```
typedef uint8_t psa_pake_step_t
```

Encoding of input and output indicators for PAKE.

Some PAKE algorithms need to exchange more data than just a single key share. This type is for encoding additional input and output data for such algorithms.

Definition at line 853 of file `crypto_extra.h`.

6.22.4 Function Documentation

6.22.4.1 `psa_crypto_driver_pake_get_cipher_suite()`

```
psa_status_t psa_crypto_driver_pake_get_cipher_suite (
    const psa_crypto_driver_pake_inputs_t * inputs,
    psa_pake_cipher_suite_t * cipher_suite )
```

Get the cipher suite from given inputs.

Parameters

in	<i>inputs</i>	Operation inputs.
out	<i>cipher_suite</i>	Return buffer for role.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BAD_STATE</i>	Cipher_suite hasn't been set yet.

6.22.4.2 `psa_crypto_driver_pake_get_password()`

```
psa_status_t psa_crypto_driver_pake_get_password (
    const psa_crypto_driver_pake_inputs_t * inputs,
    uint8_t * buffer,
    size_t buffer_size,
    size_t * buffer_length )
```

Get the password from given inputs.

Parameters

in	<i>inputs</i>	Operation inputs.
out	<i>buffer</i>	Return buffer for password.
	<i>buffer_size</i>	Size of the return buffer in bytes.
out	<i>buffer_length</i>	Actual size of the password in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BAD_STATE</i>	Password hasn't been set yet.

6.22.4.3 `psa_crypto_driver_pake_get_password_len()`

```
psa_status_t psa_crypto_driver_pake_get_password_len (
    const psa_crypto_driver_pake_inputs_t * inputs,
    size_t * password_len )
```

Get the length of the password in bytes from given inputs.

Parameters

in	<i>inputs</i>	Operation inputs.
out	<i>password_len</i>	Password length.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BAD_STATE</i>	Password hasn't been set yet.

6.22.4.4 `psa_crypto_driver_pake_get_peer()`

```
psa_status_t psa_crypto_driver_pake_get_peer (
    const psa_crypto_driver_pake_inputs_t * inputs,
```

```
uint8_t * peer_id,
size_t peer_id_size,
size_t * peer_id_length )
```

Get the peer id from given inputs.

Parameters

in	<i>inputs</i>	Operation inputs.
out	<i>peer_id</i>	Peer id.
	<i>peer_id_size</i>	Size of <i>peer_id</i> in bytes.
out	<i>peer_id_length</i>	Size of the peer id in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BAD_STATE</i>	Peer id hasn't been set yet.
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	The size of the <i>peer_id</i> is too small.

6.22.4.5 `psa_crypto_driver_pake_get_peer_len()`

```
psa_status_t psa_crypto_driver_pake_get_peer_len (
    const psa_crypto_driver_pake_inputs_t * inputs,
    size_t * peer_len )
```

Get the length of the peer id in bytes from given inputs.

Parameters

in	<i>inputs</i>	Operation inputs.
out	<i>peer_len</i>	Peer id length.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BAD_STATE</i>	Peer id hasn't been set yet.

6.22.4.6 `psa_crypto_driver_pake_get_user()`

```
psa_status_t psa_crypto_driver_pake_get_user (
    const psa_crypto_driver_pake_inputs_t * inputs,
    uint8_t * user_id,
    size_t user_id_size,
    size_t * user_id_len )
```

Get the user id from given inputs.

Parameters

in	<i>inputs</i>	Operation inputs.
out	<i>user_id</i>	User id.
	<i>user_id_size</i>	Size of <i>user_id</i> in bytes.
out	<i>user_id_len</i>	Size of the user id in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BAD_STATE</i>	User id hasn't been set yet.
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	The size of the <i>user_id</i> is too small.

6.22.4.7 `psa_crypto_driver_pake_get_user_len()`

```
psa_status_t psa_crypto_driver_pake_get_user_len (
    const psa_crypto_driver_pake_inputs_t * inputs,
    size_t * user_len )
```

Get the length of the user id in bytes from given inputs.

Parameters

in	<i>inputs</i>	Operation inputs.
out	<i>user_len</i>	User id length.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BAD_STATE</i>	User id hasn't been set yet.

6.22.4.8 `psa_pake_abort()`

```
psa_status_t psa_pake_abort (
    psa_pake_operation_t * operation )
```

Abort a PAKE operation.

Aborting an operation frees all associated resources except for the `operation` structure itself. Once aborted, the operation object can be reused for another operation by calling [`psa\_pake\_setup\(\)`](#) again.

This function may be called at any time after the operation object has been initialized as described in [`psa\_pake\_operation\_t`](#).

In particular, calling [`psa\_pake\_abort\(\)`](#) after the operation has been terminated by a call to [`psa\_pake\_abort\(\)`](#) or [`psa\_pake\_get\_shared\_key\(\)`](#) is safe and has no effect.

Parameters

<code>in, out</code>	<code>operation</code>	The operation to abort.
----------------------	------------------------	-------------------------

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_BAD_STATE</code>	The library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.22.4.9 `psa_pake_get_shared_key()`

```
psa_status_t psa_pake_get_shared_key (
    psa_pake_operation_t * operation,
    const psa_key_attributes_t * attributes,
    mbedtls_svc_key_id_t * key )
```

Extract the shared secret from the PAKE as a key.

This is the final call in a PAKE operation, which retrieves the shared secret as a key. It is recommended that this key is used as an input to a key derivation operation to produce additional cryptographic keys. For some PAKE algorithms, the shared secret is also suitable for use as a key in cryptographic operations such as encryption. Refer to the documentation of individual PAKE algorithms for more information, see PAKE algorithms.

Depending on the key confirmation requested in the cipher suite, [`psa\_pake\_get\_shared\_key\(\)`](#) must be called either before or after the key-confirmation output and input steps for the PAKE algorithm. The key confirmation affects the guarantees that can be made about the shared key:

Unconfirmed key:

If the cipher suite used to set up the operation requested an unconfirmed key, the application must call [`psa\_pake\_get\_shared\_key\(\)`](#) after the key-exchange output and input steps are completed. The PAKE algorithm provides a cryptographic guarantee that only a peer who used the same password and identity inputs is able to compute the same key. However, there is no guarantee that the peer is the participant it claims to be and was able to compute the same key.

Since the peer is not authenticated, no action should be taken that assumes that the peer is who it claims to be. For example, do not access restricted resources on the peer's behalf until an explicit authentication has succeeded.

Note

Some PAKE algorithms do not enable the output of the shared secret until it has been confirmed.

Confirmed key:

If the cipher suite used to set up the operation requested a confirmed key, the application must call [`psa\_pake\_get\_shared\_key\(\)`](#) after the key-exchange and key-confirmation output and input steps are completed.

Following key confirmation, the PAKE algorithm provides a cryptographic guarantee that the peer used the same password and identity inputs, and has computed the identical shared secret key.

Since the peer is not authenticated, no action should be taken that assumes that the peer is who it claims to be. For example, do not access restricted resources on the peer's behalf until an explicit authentication has succeeded.

Note

Some PAKE algorithms do not include any key-confirmation steps.

The exact sequence of calls to perform a password-authenticated key exchange depends on the algorithm in use. Refer to the documentation of individual PAKE algorithms for more information. See PAKE algorithms.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_pake_abort\(\)](#).

Parameters

<i>in, out</i>	<i>operation</i>	Active PAKE operation.
<i>in</i>	<i>attributes</i>	The attributes for the new key. This function uses the attributes as follows: The key type is required. All PAKE algorithms can output a key of type PSA_KEY_TYPE_DERIVE or PSA_KEY_TYPE_HMAC . PAKE algorithms that produce a pseudo-random shared secret, can also output block-cipher key types, for example PSA_KEY_TYPE_AES . Refer to the documentation of individual PAKE algorithms for more information. See PAKE algorithms.

The key size in attributes must be zero. The returned key size is always determined from the PAKE shared secret.

The key permitted-algorithm policy is required for keys that will be used for a cryptographic operation.

The key usage flags define what operations are permitted with the key.

The key lifetime and identifier are required for a persistent key.

Note

This is an input parameter: It is not updated with the final key attributes. The final attributes of the new key can be queried by calling [psa_get_key_attributes\(\)](#) with the key's identifier.

Parameters

<i>out</i>	<i>key</i>	On success, an identifier for the newly created key. PSA_KEY_ID_NULL on failure.
------------	------------	--

Return values

PSA_SUCCESS	Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
PSA_ERROR_BAD_STATE	The following conditions can result in this error: The state of PAKE operation <i>operation</i> is not valid: It must be ready to return the shared secret. For an unconfirmed key, this will be when the key-exchange output and input steps are complete, but prior to any key-confirmation output and input steps. For a confirmed key, this will be when all key-exchange and key-confirmation output and input steps are complete. The library requires initializing by a call to psa_crypto_init() .
PSA_ERROR_NOT_PERMITTED	The implementation does not permit creating a key with the specified attributes due to some implementation-specific policy.
PSA_ERROR_ALREADY_EXISTS	This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.

Return values

<i>PSA_ERROR_INVALID_ARGUMENT</i>	The following conditions can result in this error: The <code>key</code> type is not valid for output from this <code>operation</code> 's algorithm. The <code>key</code> size is nonzero. The <code>key</code> lifetime is invalid. The <code>key</code> identifier is not valid for the <code>key</code> lifetime. The <code>key</code> usage flags include invalid values. The <code>key</code> 's permitted-usage algorithm is invalid. The <code>key</code> attributes, as a whole, are invalid.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The <code>key</code> attributes, as a whole, are not supported for creation from a PAKE secret, either by the implementation in general or in the specified storage location.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	

6.22.4.10 `psa_pake_input()`

```
psa_status_t psa_pake_input (
    psa_pake_operation_t * operation,
    psa_pake_step_t step,
    const uint8_t * input,
    size_t input_length )
```

Provide input for a step of a password-authenticated key exchange.

Depending on the algorithm being executed, you might need to call this function several times or you might not need to call this at all.

The exact sequence of calls to perform a password-authenticated key exchange depends on the algorithm in use. Refer to the documentation of individual PAKE algorithm types (`PSA_ALG_XXX` values of type [psa_algorithm_t](#) such that `PSA_ALG_IS_PAKE(alg)` is true) for more information.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_pake_abort\(\)](#).

Parameters

<code>in, out</code>	<i>operation</i>	Active PAKE operation.
	<i>step</i>	The step for which the input is provided.
<code>in</code>	<i>input</i>	Buffer containing the input in the format appropriate for this <code>step</code> . Refer to the documentation of the individual <code>PSA_PAKE_STEP_XXX</code> constants for more information.
	<i>input_length</i>	Size of the <code>input</code> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
------------------------------------	----------

Return values

<i>PSA_ERROR_INVALID_SIGNATURE</i>	The verification fails for a <i>PSA_PAKE_STEP_ZK_PROOF</i> input step.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<code>input_length</code> is not compatible with the operation's algorithm, or the input is not valid for the operation's algorithm, cipher suite or step.
<i>PSA_ERROR_NOT_SUPPORTED</i>	step <code>p</code> is not supported with the operation's algorithm, or the input is not supported for the operation's algorithm, cipher suite or step.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active, and fully set up, and this call must conform to the algorithm's requirements for ordering of input and output steps), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.22.4.11 `psa_pake_output()`

```

psa_status_t psa_pake_output (
    psa_pake_operation_t * operation,
    psa_pake_step_t step,
    uint8_t * output,
    size_t output_size,
    size_t * output_length )

```

Get output for a step of a password-authenticated key exchange.

Depending on the algorithm being executed, you might need to call this function several times or you might not need to call this at all.

The exact sequence of calls to perform a password-authenticated key exchange depends on the algorithm in use. Refer to the documentation of individual PAKE algorithm types (`PSA_ALG_XXX` values of type [`psa\_algorithm\_t`](#) such that `PSA_ALG_IS_PAKE(alg)` is true) for more information.

If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_pake\_abort\(\)`](#).

Parameters

<code>in, out</code>	<i>operation</i>	Active PAKE operation.
	<i>step</i>	The step of the algorithm for which the output is requested.
<code>out</code>	<i>output</i>	Buffer where the output is to be written in the format appropriate for this step. Refer to the documentation of the individual <code>PSA_PAKE_STEP_XXX</code> constants for more information.
	<i>output_size</i>	Size of the <code>output</code> buffer in bytes. This must be at least <code>PSA_PAKE_OUTPUT_SIZE(alg, primitive, output_step)</code> where <code>alg</code> and <code>primitive</code> are the PAKE algorithm and primitive in the operation's cipher suite, and <code>step</code> is the output step.
<code>out</code>	<i>output_length</i>	On success, the number of bytes of the returned output.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_BUFFER_TOO_SMALL</i>	The size of the <code>output</code> buffer is too small.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<code>step</code> is not compatible with the operation's algorithm.
<i>PSA_ERROR_NOT_SUPPORTED</i>	<code>step</code> is not supported with the operation's algorithm.
<i>PSA_ERROR_INSUFFICIENT_ENTROPY</i>	
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid (it must be active, and fully set up, and this call must conform to the algorithm's requirements for ordering of input and output steps), or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.22.4.12 `psa_pake_set_context()`

```

psa_status_t psa_pake_set_context (
    psa_pake_operation_t * operation,
    const uint8_t * context,
    size_t context_len )

```

Set the context data for a password-authenticated key exchange.

Not all PAKE algorithms use context data. Only call this function for algorithms that need it.

Parameters

<code>in, out</code>	<i>operation</i>	The operation object to specify the application's role for. It must have been set up by <code>psa_pake_setup()</code> and not yet in use (neither <code>psa_pake_output()</code> nor <code>psa_pake_input()</code> has been called yet). It must be an operation for which the context hasn't been specified (<code>psa_pake_set_context()</code> hasn't been called yet).
<code>in</code>	<i>context</i>	The context to set.
	<i>context_len</i>	The length of <code>context</code> in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The algorithm in <code>operation</code> does not use a context.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The library configuration does not support PAKE algorithms with a context, or this specific context value is not supported for the given <code>operation</code> .
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	

Return values

<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid, or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.
--	---

6.22.4.13 `psa_pake_set_peer()`

```
psa_status_t psa_pake_set_peer (
    psa_pake_operation_t * operation,
    const uint8_t * peer_id,
    size_t peer_id_len )
```

Set the peer ID for a password-authenticated key exchange.

Call this function in addition to [`psa\_pake\_set\_user\(\)`](#) for PAKE algorithms that associate a user identifier with each side of the session. For PAKE algorithms that associate a single user identifier with the session, call [`psa\_pake\_set\_user\(\)`](#) only.

Refer to the documentation of individual PAKE algorithm types (PSA_ALG_XXX values of type [`psa\_algorithm\_t`](#) such that [`PSA\_ALG\_IS\_PAKE\(alg\)`](#) is true) for more information.

Note

When using the built-in implementation of [`PSA\_ALG\_JPAKE`](#), the peer ID must be "client" (6-byte string) or "server" (6-byte string). Third-party drivers may or may not have this limitation.

Parameters

in, out	<i>operation</i>	The operation object to set the peer ID for. It must have been set up by <code>psa_pake_setup()</code> and not yet in use (neither <code>psa_pake_output()</code> nor <code>psa_pake_input()</code> has been called yet). It must be on operation for which the peer ID hasn't been set (<code>psa_pake_set_peer()</code> hasn't been called yet).
in	<i>peer_id</i>	The peer's ID to authenticate.
	<i>peer_id_len</i>	Size of the <i>peer_id</i> buffer in bytes.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	<i>peer_id</i> is not valid for the <i>operation</i> 's algorithm and cipher suite.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The algorithm doesn't associate a second identity with the session.
<i>PSA_ERROR_INSUFFICIENT_MEMORY</i>	
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	

Return values

<i>PSA_ERROR_BAD_STATE</i>	Calling <code>psa_pake_set_peer()</code> is invalid with the <code>operation</code> 's algorithm, the operation state is not valid, or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.
--	---

6.22.4.14 `psa_pake_set_role()`

```
psa_status_t psa_pake_set_role (
    psa_pake_operation_t * operation,
    psa_pake_role_t role )
```

Set the application role for a password-authenticated key exchange.

Not all PAKE algorithms need to differentiate the communicating entities. It is optional to call this function for PAKEs that don't require a role to be specified. For such PAKEs the application role parameter is ignored, or [`PSA\_PAKE\_ROLE\_NONE`](#) can be passed as `role`.

Refer to the documentation of individual PAKE algorithm types (`PSA_ALG_XXX` values of type [`psa\_algorithm\_t`](#) such that [`PSA\_ALG\_IS\_PAKE`](#)(`alg`) is true) for more information.

Parameters

<i>in, out</i>	<i>operation</i>	The operation object to specify the application's role for. It must have been set up by <code>psa_pake_setup()</code> and not yet in use (neither <code>psa_pake_output()</code> nor <code>psa_pake_input()</code> has been called yet). It must be on operation for which the application's role hasn't been specified (<code>psa_pake_set_role()</code> hasn't been called yet).
	<i>role</i>	A value of type <code>psa_pake_role_t</code> indicating the application's role in the PAKE the algorithm that is being set up. For more information see the documentation of <code>PSA_PAKE_ROLE_XXX</code> constants.

Return values

<i>PSA_SUCCESS</i>	Success.
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The <code>role</code> is not a valid PAKE role in the <code>operation</code> 's algorithm.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The <code>role</code> for this algorithm is not supported or is not valid.
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_BAD_STATE</i>	The operation state is not valid, or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.22.4.15 `psa_pake_set_user()`

```
psa_status_t psa_pake_set_user (
    psa_pake_operation_t * operation,
    const uint8_t * user_id,
    size_t user_id_len )
```

Set the user ID for a password-authenticated key exchange.

Call this function to set the user ID. For PAKE algorithms that associate a user identifier with each side of the session you need to call `psa_pake_set_peer()` as well. For PAKE algorithms that associate a single user identifier with the session, call `psa_pake_set_user()` only.

Refer to the documentation of individual PAKE algorithm types (PSA_ALG_XXX values of type `psa_algorithm_t` such that `PSA_ALG_IS_PAKE(alg)` is true) for more information.

Note

When using the built-in implementation of `PSA_ALG_JPAKE`, the user ID must be "client" (6-byte string) or "server" (6-byte string). Third-party drivers may or may not have this limitation.

Parameters

in, out	<i>operation</i>	The operation object to set the user ID for. It must have been set up by <code>psa_pake_setup()</code> and not yet in use (neither <code>psa_pake_output()</code> nor <code>psa_pake_input()</code> has been called yet). It must be on operation for which the user ID hasn't been set (<code>psa_pake_set_user()</code> hasn't been called yet).
in	<i>user_id</i>	The user ID to authenticate with.
	<i>user_id_len</i>	Size of the <i>user_id</i> buffer in bytes.

Return values

<code>PSA_SUCCESS</code>	Success.
<code>PSA_ERROR_INVALID_ARGUMENT</code>	<i>user_id</i> is not valid for the operation's algorithm and cipher suite.
<code>PSA_ERROR_NOT_SUPPORTED</code>	The value of <i>user_id</i> is not supported by the implementation.
<code>PSA_ERROR_INSUFFICIENT_MEMORY</code>	
<code>PSA_ERROR_COMMUNICATION_FAILURE</code>	
<code>PSA_ERROR_CORRUPTION_DETECTED</code>	
<code>PSA_ERROR_BAD_STATE</code>	The operation state is not valid, or the library has not been previously initialized by <code>psa_crypto_init()</code> . It is implementation-dependent whether a failure to initialize results in this error code.

6.22.4.16 `psa_pake_setup()`

```
psa_status_t psa_pake_setup (
    psa_pake_operation_t * operation,
```

```

mbedtls_svc_key_id_t password_key,
const psa_pake_cipher_suite_t * cipher_suite )

```

Setup a password-authenticated key exchange.

The sequence of operations to set up a password-authenticated key exchange operation is as follows:

1. Allocate a PAKE operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for [psa_pake_operation_t](#). For example, using [PSA_PAKE_OPERATION_INIT](#).
3. Call [psa_pake_setup\(\)](#) to specify the cipher suite.
4. Call [psa_pake_set_xxx\(\)](#) functions on the operation to complete the setup. The exact sequence of [psa_pake_set_xxx\(\)](#) functions that needs to be called depends on the algorithm in use.

A typical sequence of calls to perform a password-authenticated key exchange:

1. Call [psa_pake_output\(\)](#)(operation, [PSA_PAKE_STEP_KEY_SHARE](#), ...) to get the key share that needs to be sent to the peer.
2. Call [psa_pake_input\(\)](#)(operation, [PSA_PAKE_STEP_KEY_SHARE](#), ...) to provide the key share that was received from the peer.
3. Depending on the algorithm additional calls to [psa_pake_output\(\)](#) and [psa_pake_input\(\)](#) might be necessary.
4. Call [psa_pake_get_shared_key\(\)](#) to access the shared secret.

Refer to the documentation of individual PAKE algorithms for details on the required set up and operation for each algorithm, and for constraints on the format and content of valid passwords. See PAKE algorithms.

After a successful call to [psa_pake_setup\(\)](#), the operation is active, and the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to [psa_pake_get_shared_key\(\)](#).
- A call to [psa_pake_abort\(\)](#).

If [psa_pake_setup\(\)](#) returns an error, the operation object is unchanged. If a subsequent function call with an active operation returns an error, the operation enters an error state.

To abandon an active operation, or reset an operation in an error state, call [psa_pake_abort\(\)](#).

Parameters

in, out	<i>operation</i>	The operation object to set up. It must have been initialized as per the documentation for psa_pake_operation_t and not yet in use.
in	<i>password_key</i>	Identifier of the key holding the password or a value derived from the password. It must remain valid until the operation terminates.

The valid key types depend on the PAKE algorithm, and participant role. Refer to the documentation of individual PAKE algorithms for more information, see PAKE algorithms.

The key must permit the usage [PSA_KEY_USAGE_DERIVE](#).

Parameters

in	<i>cipher_suite</i>	The cipher suite to use. A PAKE cipher suite fully characterizes a PAKE algorithm, including the PAKE algorithm.
----	---------------------	--

The cipher suite must be compatible with the key type of `password_key`.

Return values

<i>PSA_SUCCESS</i>	Success. The operation is now active.
<i>PSA_ERROR_BAD_STATE</i>	The following conditions can result in this error: • The operation state is not valid: it must be inactive. • The library requires initializing by a call to <i>psa_crypto_init()</i>.
<i>PSA_ERROR_INVALID_HANDLE</i>	<code>password_key</code> is not a valid key identifier.
<i>PSA_ERROR_NOT_PERMITTED</i>	<code>password_key</code> does not have the <i>PSA_KEY_USAGE_DERIVE</i> flag, or it does not permit the algorithm in <code>cipher_suite</code> .
<i>PSA_ERROR_INVALID_ARGUMENT</i>	The following conditions can result in this error: • The algorithm in <code>cipher_suite</code> is not a PAKE algorithm, or encodes an invalid hash algorithm. • The PAKE primitive in <code>cipher_suite</code> is not compatible with the PAKE algorithm. • The key confirmation value in <code>cipher_suite</code> is not compatible with the PAKE algorithm and primitive. • The key type or key size of <code>password_key</code> is not compatible with <code>cipher_suite</code>.
<i>PSA_ERROR_NOT_SUPPORTED</i>	The following conditions can result in this error: • The algorithm in <code>cipher_suite</code> is not a supported PAKE algorithm, or encodes an unsupported hash algorithm. • The PAKE primitive in <code>cipher_suite</code> is not supported or not compatible with the PAKE algorithm. • The key confirmation value in <code>cipher_suite</code> is not supported, or not compatible, with the PAKE algorithm and primitive. • The key type or key size of <code>password_key</code> is not supported with <code>cipher_suite</code>.
<i>PSA_ERROR_COMMUNICATION_FAILURE</i>	
<i>PSA_ERROR_CORRUPTION_DETECTED</i>	
<i>PSA_ERROR_STORAGE_FAILURE</i>	
<i>PSA_ERROR_DATA_CORRUPT</i>	
<i>PSA_ERROR_DATA_INVALID</i>	

6.23 Error codes

Macros

- `#define PSA_SUCCESS ((psa_status_t)0)`
- `#define PSA_ERROR_GENERIC_ERROR ((psa_status_t)-132)`
- `#define PSA_ERROR_NOT_SUPPORTED ((psa_status_t)-134)`
- `#define PSA_ERROR_NOT_PERMITTED ((psa_status_t)-133)`
- `#define PSA_ERROR_BUFFER_TOO_SMALL ((psa_status_t)-138)`
- `#define PSA_ERROR_ALREADY_EXISTS ((psa_status_t)-139)`
- `#define PSA_ERROR_DOES_NOT_EXIST ((psa_status_t)-140)`
- `#define PSA_ERROR_BAD_STATE ((psa_status_t)-137)`
- `#define PSA_ERROR_INVALID_ARGUMENT ((psa_status_t)-135)`
- `#define PSA_ERROR_INSUFFICIENT_MEMORY ((psa_status_t)-141)`
- `#define PSA_ERROR_INSUFFICIENT_STORAGE ((psa_status_t)-142)`
- `#define PSA_ERROR_COMMUNICATION_FAILURE ((psa_status_t)-145)`
- `#define PSA_ERROR_STORAGE_FAILURE ((psa_status_t)-146)`
- `#define PSA_ERROR_HARDWARE_FAILURE ((psa_status_t)-147)`
- `#define PSA_ERROR_CORRUPTION_DETECTED ((psa_status_t)-151)`
- `#define PSA_ERROR_INSUFFICIENT_ENTROPY ((psa_status_t)-148)`
- `#define PSA_ERROR_INVALID_SIGNATURE ((psa_status_t)-149)`
- `#define PSA_ERROR_INVALID_PADDING ((psa_status_t)-150)`
- `#define PSA_ERROR_INSUFFICIENT_DATA ((psa_status_t)-143)`
- `#define PSA_ERROR_SERVICE_FAILURE ((psa_status_t)-144)`
- `#define PSA_ERROR_INVALID_HANDLE ((psa_status_t)-136)`
- `#define PSA_ERROR_DATA_CORRUPT ((psa_status_t)-152)`
- `#define PSA_ERROR_DATA_INVALID ((psa_status_t)-153)`
- `#define PSA_OPERATION_INCOMPLETE ((psa_status_t)-248)`

Typedefs

- `typedef int32_t psa_status_t`
Function return status.

6.23.1 Detailed Description

6.23.2 Macro Definition Documentation

6.23.2.1 PSA_ERROR_ALREADY_EXISTS

```
#define PSA_ERROR_ALREADY_EXISTS ((psa_status_t)-139)
```

Asking for an item that already exists

Implementations should return this error, when attempting to write an item (like a key) that already exists.

Definition at line 105 of file `crypto_values.h`.

6.23.2.2 PSA_ERROR_BAD_STATE

```
#define PSA_ERROR_BAD_STATE ((psa_status_t)-137)
```

The requested action cannot be performed in the current state.

Multipart operations return this error when one of the functions is called out of sequence. Refer to the function descriptions for permitted sequencing of functions.

Implementations shall not return this error code to indicate that a key either exists or not, but shall instead return [PSA_ERROR_ALREADY_EXISTS](#) or [PSA_ERROR_DOES_NOT_EXIST](#) as applicable.

Implementations shall not return this error code to indicate that a key identifier is invalid, but shall return [PSA_ERROR_INVALID_HANDLE](#) instead.

Definition at line 127 of file `crypto_values.h`.

6.23.2.3 PSA_ERROR_BUFFER_TOO_SMALL

```
#define PSA_ERROR_BUFFER_TOO_SMALL ((psa_status_t)-138)
```

An output buffer is too small.

Applications can call the `PSA_XXX_SIZE` macro listed in the function description to determine a sufficient buffer size.

Implementations should preferably return this error code only in cases when performing the operation with a larger output buffer would succeed. However implementations may return this error if a function has invalid or unsupported parameters in addition to the parameters that determine the necessary output buffer size.

Definition at line 99 of file `crypto_values.h`.

6.23.2.4 PSA_ERROR_COMMUNICATION_FAILURE

```
#define PSA_ERROR_COMMUNICATION_FAILURE ((psa_status_t)-145)
```

There was a communication failure inside the implementation.

This can indicate a communication failure between the application and an external cryptoprocessor or between the cryptoprocessor and an external volatile or persistent memory. A communication failure may be transient or permanent depending on the cause.

Warning

If a function returns this error, it is undetermined whether the requested action has completed or not. Implementations should return [PSA_SUCCESS](#) on successful completion whenever possible, however functions may return [PSA_ERROR_COMMUNICATION_FAILURE](#) if the requested action was completed successfully in an external cryptoprocessor but there was a breakdown of communication before the cryptoprocessor could report the status to the application.

Definition at line 170 of file `crypto_values.h`.

6.23.2.5 PSA_ERROR_CORRUPTION_DETECTED

```
#define PSA_ERROR_CORRUPTION_DETECTED ((psa_status_t)-151)
```

A tampering attempt was detected.

If an application receives this error code, there is no guarantee that previously accessed or computed data was correct and remains confidential. Applications should not perform any security function and should enter a safe failure state.

Implementations may return this error code if they detect an invalid state that cannot happen during normal operation and that indicates that the implementation's security guarantees no longer hold. Depending on the implementation architecture and on its security and safety goals, the implementation may forcibly terminate the application.

This error code is intended as a last resort when a security breach is detected and it is unsure whether the keystore data is still protected. Implementations shall only return this error code to report an alarm from a tampering detector, to indicate that the confidentiality of stored data can no longer be guaranteed, or to indicate that the integrity of previously returned data is now considered compromised. Implementations shall not use this error code to indicate a hardware failure that merely makes it impossible to perform the requested operation (use [PSA_ERROR_COMMUNICATION_FAILURE](#), [PSA_ERROR_STORAGE_FAILURE](#), [PSA_ERROR_HARDWARE_FAILURE](#), [PSA_ERROR_INSUFFICIENT_ENTROPY](#) or other applicable error code instead).

This error indicates an attack against the application. Implementations shall not return this error code as a consequence of the behavior of the application itself.

Definition at line 232 of file `crypto_values.h`.

6.23.2.6 PSA_ERROR_DATA_CORRUPT

```
#define PSA_ERROR_DATA_CORRUPT ((psa_status_t)-152)
```

Stored data has been corrupted.

This error indicates that some persistent storage has suffered corruption. It does not indicate the following situations, which have specific error codes:

- A corruption of volatile memory - use [PSA_ERROR_CORRUPTION_DETECTED](#).
- A communication error between the cryptoprocessor and its external storage - use [PSA_ERROR_COMMUNICATION_FAILURE](#).
- When the storage is in a valid state but is full - use [PSA_ERROR_INSUFFICIENT_STORAGE](#).
- When the storage fails for other reasons - use [PSA_ERROR_STORAGE_FAILURE](#).
- When the stored data is not valid - use [PSA_ERROR_DATA_INVALID](#).

Note

A storage corruption does not indicate that any data that was previously read is invalid. However this previously read data might no longer be readable from storage.

When a storage failure occurs, it is no longer possible to ensure the global integrity of the keystore.

Definition at line 314 of file `crypto_values.h`.

6.23.2.7 PSA_ERROR_DATA_INVALID

```
#define PSA_ERROR_DATA_INVALID ((psa_status_t)-153)
```

Data read from storage is not valid for the implementation.

This error indicates that some data read from storage does not have a valid format. It does not indicate the following situations, which have specific error codes:

- When the storage or stored data is corrupted - use [PSA_ERROR_DATA_CORRUPT](#)
- When the storage fails for other reasons - use [PSA_ERROR_STORAGE_FAILURE](#)
- An invalid argument to the API - use [PSA_ERROR_INVALID_ARGUMENT](#)

This error is typically a result of either storage corruption on a cleartext storage backend, or an attempt to read data that was written by an incompatible version of the library.

Definition at line 330 of file `crypto_values.h`.

6.23.2.8 PSA_ERROR_DOES_NOT_EXIST

```
#define PSA_ERROR_DOES_NOT_EXIST ((psa_status_t)-140)
```

Asking for an item that doesn't exist

Implementations should return this error, if a requested item (like a key) does not exist.

Definition at line 111 of file `crypto_values.h`.

6.23.2.9 PSA_ERROR_GENERIC_ERROR

```
#define PSA_ERROR_GENERIC_ERROR ((psa_status_t)-132)
```

An error occurred that does not correspond to any defined failure cause.

Implementations may use this error code if none of the other standard error codes are applicable.

Definition at line 65 of file `crypto_values.h`.

6.23.2.10 PSA_ERROR_HARDWARE_FAILURE

```
#define PSA_ERROR_HARDWARE_FAILURE ((psa_status_t)-147)
```

A hardware failure was detected.

A hardware failure may be transient or permanent depending on the cause.

Definition at line 201 of file `crypto_values.h`.

6.23.2.11 PSA_ERROR_INSUFFICIENT_DATA

```
#define PSA_ERROR_INSUFFICIENT_DATA ((psa_status_t)-143)
```

Return this error when there's insufficient data when attempting to read from a resource.

Definition at line 281 of file `crypto_values.h`.

6.23.2.12 PSA_ERROR_INSUFFICIENT_ENTROPY

```
#define PSA_ERROR_INSUFFICIENT_ENTROPY ((psa_status_t)-148)
```

There is not enough entropy to generate random data needed for the requested action.

This error indicates a failure of a hardware random generator. Application writers should note that this error can be returned not only by functions whose purpose is to generate random data, such as key, IV or nonce generation, but also by functions that execute an algorithm with a randomized result, as well as functions that use randomization of intermediate computations as a countermeasure to certain attacks.

Implementations should avoid returning this error after `psa_crypto_init()` has succeeded. Implementations should generate sufficient entropy during initialization and subsequently use a cryptographically secure pseudorandom generator (PRNG). However implementations may return this error at any time if a policy requires the PRNG to be reseeded during normal operation.

Definition at line 251 of file `crypto_values.h`.

6.23.2.13 PSA_ERROR_INSUFFICIENT_MEMORY

```
#define PSA_ERROR_INSUFFICIENT_MEMORY ((psa_status_t)-141)
```

There is not enough runtime memory.

If the action is carried out across multiple security realms, this error can refer to available memory in any of the security realms.

Definition at line 144 of file `crypto_values.h`.

6.23.2.14 PSA_ERROR_INSUFFICIENT_STORAGE

```
#define PSA_ERROR_INSUFFICIENT_STORAGE ((psa_status_t)-142)
```

There is not enough persistent storage.

Functions that modify the key storage return this error code if there is insufficient storage space on the host media. In addition, many functions that do not otherwise access storage may return this error code if the implementation requires a mandatory log entry for the requested action and the log storage space is full.

Definition at line 153 of file `crypto_values.h`.

6.23.2.15 PSA_ERROR_INVALID_ARGUMENT

```
#define PSA_ERROR_INVALID_ARGUMENT ((psa_status_t)-135)
```

The parameters passed to the function are invalid.

Implementations may return this error any time a parameter or combination of parameters are recognized as invalid.

Implementations shall not return this error code to indicate that a key identifier is invalid, but shall return [PSA_ERROR_INVALID_HANDLE](#) instead.

Definition at line 138 of file `crypto_values.h`.

6.23.2.16 PSA_ERROR_INVALID_HANDLE

```
#define PSA_ERROR_INVALID_HANDLE ((psa_status_t)-136)
```

The key identifier is not valid. See also [:ref:`key-handles`](#).

Definition at line 290 of file `crypto_values.h`.

6.23.2.17 PSA_ERROR_INVALID_PADDING

```
#define PSA_ERROR_INVALID_PADDING ((psa_status_t)-150)
```

The decrypted padding is incorrect.

Warning

In some protocols, when decrypting data, it is essential that the behavior of the application does not depend on whether the padding is correct, down to precise timing. Applications should prefer protocols that use authenticated encryption rather than plain encryption. If the application must perform a decryption of unauthenticated data, the application writer should take care not to reveal whether the padding is invalid.

Implementations should strive to make valid and invalid padding as close as possible to indistinguishable to an external observer. In particular, the timing of a decryption operation should not depend on the validity of the padding.

Definition at line 277 of file `crypto_values.h`.

6.23.2.18 PSA_ERROR_INVALID_SIGNATURE

```
#define PSA_ERROR_INVALID_SIGNATURE ((psa_status_t)-149)
```

The signature, MAC or hash is incorrect.

Verification functions return this error if the verification calculations completed successfully, and the value to be verified was determined to be incorrect.

If the value to verify has an invalid size, implementations may return either [PSA_ERROR_INVALID_ARGUMENT](#) or [PSA_ERROR_INVALID_SIGNATURE](#).

Definition at line 261 of file `crypto_values.h`.

6.23.2.19 PSA_ERROR_NOT_PERMITTED

```
#define PSA_ERROR_NOT_PERMITTED ((psa_status_t)-133)
```

The requested action is denied by a policy.

Implementations should return this error code when the parameters are recognized as valid and supported, and a policy explicitly denies the requested operation.

If a subset of the parameters of a function call identify a forbidden operation, and another subset of the parameters are not valid or not supported, it is unspecified whether the function returns [PSA_ERROR_NOT_PERMITTED](#), [PSA_ERROR_NOT_SUPPORTED](#) or [PSA_ERROR_INVALID_ARGUMENT](#).

Definition at line 87 of file `crypto_values.h`.

6.23.2.20 PSA_ERROR_NOT_SUPPORTED

```
#define PSA_ERROR_NOT_SUPPORTED ((psa_status_t)-134)
```

The requested operation or a parameter is not supported by this implementation.

Implementations should return this error code when an enumeration parameter such as a key type, algorithm, etc. is not recognized. If a combination of parameters is recognized and identified as not valid, return [PSA_ERROR_INVALID_ARGUMENT](#) instead.

Definition at line 74 of file `crypto_values.h`.

6.23.2.21 PSA_ERROR_SERVICE_FAILURE

```
#define PSA_ERROR_SERVICE_FAILURE ((psa_status_t)-144)
```

This can be returned if a function can no longer operate correctly. For example, if an essential initialization operation failed or a mutex operation failed.

Definition at line 286 of file `crypto_values.h`.

6.23.2.22 PSA_ERROR_STORAGE_FAILURE

```
#define PSA_ERROR_STORAGE_FAILURE ((psa_status_t)-146)
```

There was a storage failure that may have led to data loss.

This error indicates that some persistent storage is corrupted. It should not be used for a corruption of volatile memory (use [PSA_ERROR_CORRUPTION_DETECTED](#)), for a communication error between the cryptoprocessor and its external storage (use [PSA_ERROR_COMMUNICATION_FAILURE](#)), or when the storage is in a valid state but is full (use [PSA_ERROR_INSUFFICIENT_STORAGE](#)).

Note that a storage failure does not indicate that any data that was previously read is invalid. However this previously read data may no longer be readable from storage.

When a storage failure occurs, it is no longer possible to ensure the global integrity of the keystore. Depending on the global integrity guarantees offered by the implementation, access to other data may or may not fail even if the data is still readable but its integrity cannot be guaranteed.

Implementations should only use this error code to report a permanent storage corruption. However application writers should keep in mind that transient errors while reading the storage may be reported using this error code.

Definition at line 195 of file `crypto_values.h`.

6.23.2.23 PSA_OPERATION_INCOMPLETE

```
#define PSA_OPERATION_INCOMPLETE ((psa_status_t)-248)
```

The function that returns this status is defined as interruptible and still has work to do, thus the user should call the function again with the same operation context until it either returns [PSA_SUCCESS](#) or any other error. This is not an error per se, more a notification of status.

Definition at line 337 of file `crypto_values.h`.

6.23.2.24 PSA_SUCCESS

```
#define PSA_SUCCESS ((psa_status_t)0)
```

The action was completed successfully.

Definition at line 58 of file `crypto_values.h`.

6.23.3 Typedef Documentation

6.23.3.1 psa_status_t

```
typedef int32_t psa_status_t
```

Function return status.

This is either [PSA_SUCCESS](#) (which is zero), indicating success, or a small negative value indicating that an error occurred. Errors are encoded as one of the `PSA_ERROR_XXX` values defined here.

Definition at line 52 of file `crypto_types.h`.

6.24 Key and algorithm types

Macros

- `#define PSA_KEY_TYPE_DSA_PUBLIC_KEY ((psa_key_type_t) 0x4002)`
- `#define PSA_KEY_TYPE_DSA_KEY_PAIR ((psa_key_type_t) 0x7002)`
- `#define PSA_KEY_TYPE_IS_DSA(type) (PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) == PSA_KEY_TYPE_DSA_PUBLIC_KEY)`
- `#define PSA_ALG_DSA_BASE ((psa_algorithm_t) 0x06000400)`
- `#define PSA_ALG_DSA(hash_alg) (PSA_ALG_DSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_DETERMINISTIC_DSA_BASE ((psa_algorithm_t) 0x06000500)`
- `#define PSA_ALG_DSA_DETERMINISTIC_FLAG PSA_ALG_ECDSA_DETERMINISTIC_FLAG`
- `#define PSA_ALG_DETERMINISTIC_DSA(hash_alg) (PSA_ALG_DETERMINISTIC_DSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_DSA(alg)`
- `#define PSA_ALG_DSA_IS_DETERMINISTIC(alg) (((alg) & PSA_ALG_DSA_DETERMINISTIC_FLAG) != 0)`
- `#define PSA_ALG_IS_DETERMINISTIC_DSA(alg) (PSA_ALG_IS_DSA(alg) && PSA_ALG_DSA_IS_DETERMINISTIC(alg))`
- `#define PSA_ALG_IS_RANDOMIZED_DSA(alg) (PSA_ALG_IS_DSA(alg) && !PSA_ALG_DSA_IS_DETERMINISTIC(alg))`
- `#define PSA_ALG_IS_VENDOR_HASH_AND_SIGN(alg) PSA_ALG_IS_DSA(alg)`
- `#define PSA_ALG_CATEGORY_PAKE ((psa_algorithm_t) 0x0a000000)`
- `#define PSA_ALG_IS_PAKE(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_PAKE)`
- `#define PSA_ALG_JPAKE_BASE ((psa_algorithm_t) 0x0a000100)`
- `#define PSA_ALG_JPAKE(hash_alg) (PSA_ALG_JPAKE_BASE | ((hash_alg) & (PSA_ALG_HASH_MASK)))`
- `#define PSA_ALG_IS_JPAKE(alg) (((alg) & (~PSA_ALG_HASH_MASK)) == PSA_ALG_JPAKE_BASE)`
- `#define PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4400)`
- `#define PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE ((psa_key_type_t) 0x7400)`
- `#define PSA_KEY_TYPE_SPAKE2P_KEY_PAIR(curve) (PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE | (curve))`
- `#define PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY(curve) (PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE | (curve))`
- `#define PSA_KEY_TYPE_IS_SPAKE2P_KEY_PAIR(type)`
- `#define PSA_KEY_TYPE_IS_SPAKE2P_PUBLIC_KEY(type)`
- `#define PSA_KEY_TYPE_IS_SPAKE2P(type)`
- `#define PSA_ALG_SPAKE2P_HMAC_BASE ((psa_algorithm_t) 0x0a000400)`
- `#define PSA_ALG_SPAKE2P_HMAC(hash_alg) (PSA_ALG_SPAKE2P_HMAC_BASE | ((hash_alg) & (PSA_ALG_HASH_MASK)))`
- `#define PSA_ALG_IS_SPAKE2P_HMAC(alg) (((alg) & (~PSA_ALG_HASH_MASK)) == PSA_ALG_SPAKE2P_HMAC_BASE)`
- `#define PSA_ALG_SPAKE2P_CMAC_BASE ((psa_algorithm_t) 0x0a000500)`
- `#define PSA_ALG_SPAKE2P_CMAC(hash_alg) (PSA_ALG_SPAKE2P_CMAC_BASE | ((hash_alg) & (PSA_ALG_HASH_MASK)))`
- `#define PSA_ALG_IS_SPAKE2P_CMAC(alg) (((alg) & (~PSA_ALG_HASH_MASK)) == PSA_ALG_SPAKE2P_CMAC_BASE)`
- `#define PSA_ALG_SPAKE2P_MATTER ((psa_algorithm_t) 0x0a000609)`
- `#define PSA_ALG_IS_SPAKE2P(alg)`
- `#define PSA_KEY_TYPE_NONE ((psa_key_type_t) 0x0000)`
- `#define PSA_KEY_TYPE_VENDOR_FLAG ((psa_key_type_t) 0x8000)`
- `#define PSA_KEY_TYPE_CATEGORY_MASK ((psa_key_type_t) 0x7000)`
- `#define PSA_KEY_TYPE_CATEGORY_RAW ((psa_key_type_t) 0x1000)`
- `#define PSA_KEY_TYPE_CATEGORY_SYMMETRIC ((psa_key_type_t) 0x2000)`
- `#define PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY ((psa_key_type_t) 0x4000)`
- `#define PSA_KEY_TYPE_CATEGORY_KEY_PAIR ((psa_key_type_t) 0x7000)`
- `#define PSA_KEY_TYPE_CATEGORY_FLAG_PAIR ((psa_key_type_t) 0x3000)`
- `#define PSA_KEY_TYPE_IS_VENDOR_DEFINED(type) (((type) & PSA_KEY_TYPE_VENDOR_FLAG) != 0)`
- `#define PSA_KEY_TYPE_IS_UNSTRUCTURED(type)`
- `#define PSA_KEY_TYPE_IS_ASYMMETRIC(type)`

```

• #define PSA_KEY_TYPE_IS_PUBLIC_KEY(type) (((type) & PSA_KEY_TYPE_CATEGORY_MASK) ==
  PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY)
• #define PSA_KEY_TYPE_IS_KEY_PAIR(type) (((type) & PSA_KEY_TYPE_CATEGORY_MASK) ==
  PSA_KEY_TYPE_CATEGORY_KEY_PAIR)
• #define PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY(type) ((type) | PSA_KEY_TYPE_CATEGORY_FLAG_PAIR)
• #define PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) ((type) & ~PSA_KEY_TYPE_CATEGORY_FLAG_PAIR)
• #define PSA_KEY_TYPE_RAW_DATA ((psa_key_type_t) 0x1001)
• #define PSA_KEY_TYPE_HMAC ((psa_key_type_t) 0x1100)
• #define PSA_KEY_TYPE_DERIVE ((psa_key_type_t) 0x1200)
• #define PSA_KEY_TYPE_PASSWORD ((psa_key_type_t) 0x1203)
• #define PSA_KEY_TYPE_PASSWORD_HASH ((psa_key_type_t) 0x1205)
• #define PSA_KEY_TYPE_PEPPER ((psa_key_type_t) 0x1206)
• #define PSA_KEY_TYPE_AES ((psa_key_type_t) 0x2400)
• #define PSA_KEY_TYPE_ARIA ((psa_key_type_t) 0x2406)
• #define PSA_KEY_TYPE_CAMELLIA ((psa_key_type_t) 0x2403)
• #define PSA_KEY_TYPE_CHACHA20 ((psa_key_type_t) 0x2004)
• #define PSA_KEY_TYPE_RSA_PUBLIC_KEY ((psa_key_type_t) 0x4001)
• #define PSA_KEY_TYPE_RSA_KEY_PAIR ((psa_key_type_t) 0x7001)
• #define PSA_KEY_TYPE_IS_RSA(type) (PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) ==
  PSA_KEY_TYPE_RSA_PUBLIC_KEY)
• #define PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4100)
• #define PSA_KEY_TYPE_ECC_KEY_PAIR_BASE ((psa_key_type_t) 0x7100)
• #define PSA_KEY_TYPE_ECC_CURVE_MASK ((psa_key_type_t) 0x00ff)
• #define PSA_KEY_TYPE_ECC_KEY_PAIR(curve) (PSA_KEY_TYPE_ECC_KEY_PAIR_BASE | (curve))
• #define PSA_KEY_TYPE_ECC_PUBLIC_KEY(curve) (PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE |
  (curve))
• #define PSA_KEY_TYPE_IS_ECC(type)
• #define PSA_KEY_TYPE_IS_ECC_KEY_PAIR(type)
• #define PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY(type)
• #define PSA_KEY_TYPE_HAS_ECC_FAMILY(type) (PSA_KEY_TYPE_IS_ECC(type) || PSA_KEY_TYPE_IS_SPAKE2P(type))
• #define PSA_KEY_TYPE_ECC_GET_FAMILY(type)
• #define PSA_ECC_FAMILY_IS_WEIERSTRASS(family) ((family & 0xc0) == 0)
• #define PSA_ECC_FAMILY_SECP_K1 ((psa_ecc_family_t) 0x17)
• #define PSA_ECC_FAMILY_SECP_R1 ((psa_ecc_family_t) 0x12)
• #define PSA_ECC_FAMILY_SECP_R2 ((psa_ecc_family_t) 0x1b)
• #define PSA_ECC_FAMILY_SECT_K1 ((psa_ecc_family_t) 0x27)
• #define PSA_ECC_FAMILY_SECT_R1 ((psa_ecc_family_t) 0x22)
• #define PSA_ECC_FAMILY_SECT_R2 ((psa_ecc_family_t) 0x2b)
• #define PSA_ECC_FAMILY_BRAINPOOL_P_R1 ((psa_ecc_family_t) 0x30)
• #define PSA_ECC_FAMILY_MONTGOMERY ((psa_ecc_family_t) 0x41)
• #define PSA_ECC_FAMILY_TWISTED_EDWARDS ((psa_ecc_family_t) 0x42)
• #define PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4200)
• #define PSA_KEY_TYPE_DH_KEY_PAIR_BASE ((psa_key_type_t) 0x7200)
• #define PSA_KEY_TYPE_DH_GROUP_MASK ((psa_key_type_t) 0x00ff)
• #define PSA_KEY_TYPE_DH_KEY_PAIR(group) (PSA_KEY_TYPE_DH_KEY_PAIR_BASE | (group))
• #define PSA_KEY_TYPE_DH_PUBLIC_KEY(group) (PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE | (group))
• #define PSA_KEY_TYPE_IS_DH(type)
• #define PSA_KEY_TYPE_IS_DH_KEY_PAIR(type)
• #define PSA_KEY_TYPE_IS_DH_PUBLIC_KEY(type)
• #define PSA_KEY_TYPE_DH_GET_FAMILY(type)
• #define PSA_DH_FAMILY_RFC7919 ((psa_dh_family_t) 0x03)
• #define PSA_GET_KEY_TYPE_BLOCK_SIZE_EXPONENT(type) (((type) >> 8) & 7)
• #define PSA_BLOCK_CIPHER_BLOCK_LENGTH(type)
• #define PSA_ALG_VENDOR_FLAG ((psa_algorithm_t) 0x80000000)
• #define PSA_ALG_CATEGORY_MASK ((psa_algorithm_t) 0x7f000000)

```

```

• #define PSA_ALG_CATEGORY_HASH ((psa_algorithm_t) 0x02000000)
• #define PSA_ALG_CATEGORY_MAC ((psa_algorithm_t) 0x03000000)
• #define PSA_ALG_CATEGORY_CIPHER ((psa_algorithm_t) 0x04000000)
• #define PSA_ALG_CATEGORY_AEAD ((psa_algorithm_t) 0x05000000)
• #define PSA_ALG_CATEGORY_SIGN ((psa_algorithm_t) 0x06000000)
• #define PSA_ALG_CATEGORY_ASYMMETRIC_ENCRYPTION ((psa_algorithm_t) 0x07000000)
• #define PSA_ALG_CATEGORY_KEY_DERIVATION ((psa_algorithm_t) 0x08000000)
• #define PSA_ALG_CATEGORY_KEY_AGREEMENT ((psa_algorithm_t) 0x09000000)
• #define PSA_ALG_CATEGORY_XOF ((psa_algorithm_t) 0x0d000000)
• #define PSA_ALG_CATEGORY_KEY_WRAP ((psa_algorithm_t) 0x0B000000)
• #define PSA_ALG_IS_VENDOR_DEFINED(alg) (((alg) & PSA_ALG_VENDOR_FLAG) != 0)
• #define PSA_ALG_IS_HASH(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_HASH)
• #define PSA_ALG_IS_MAC(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_MAC)
• #define PSA_ALG_IS_CIPHER(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_CIPHER)
• #define PSA_ALG_IS_AEAD(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_AEAD)
• #define PSA_ALG_IS_SIGN(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_SIGN)
• #define PSA_ALG_IS_ASYMMETRIC_ENCRYPTION(alg) (((alg) & PSA_ALG_CATEGORY_MASK) ==
  PSA_ALG_CATEGORY_ASYMMETRIC_ENCRYPTION)
• #define PSA_ALG_IS_KEY_AGREEMENT(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_KEY_AGREEMENT)
• #define PSA_ALG_IS_KEY_DERIVATION(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_KEY_DERIVATION)
• #define PSA_ALG_IS_KEY_DERIVATION_STRETCHING(alg)
• #define PSA_ALG_IS_XOF(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_XOF)
• #define PSA_ALG_NONE ((psa_algorithm_t)0)
• #define PSA_ALG_HASH_MASK ((psa_algorithm_t) 0x000000ff)
• #define PSA_ALG_MD5 ((psa_algorithm_t) 0x02000003)
• #define PSA_ALG_RIPEMD160 ((psa_algorithm_t) 0x02000004)
• #define PSA_ALG_SHA_1 ((psa_algorithm_t) 0x02000005)
• #define PSA_ALG_SHA_224 ((psa_algorithm_t) 0x02000008)
• #define PSA_ALG_SHA_256 ((psa_algorithm_t) 0x02000009)
• #define PSA_ALG_SHA_384 ((psa_algorithm_t) 0x0200000a)
• #define PSA_ALG_SHA_512 ((psa_algorithm_t) 0x0200000b)
• #define PSA_ALG_SHA_512_224 ((psa_algorithm_t) 0x0200000c)
• #define PSA_ALG_SHA_512_256 ((psa_algorithm_t) 0x0200000d)
• #define PSA_ALG_SHA3_224 ((psa_algorithm_t) 0x02000010)
• #define PSA_ALG_SHA3_256 ((psa_algorithm_t) 0x02000011)
• #define PSA_ALG_SHA3_384 ((psa_algorithm_t) 0x02000012)
• #define PSA_ALG_SHA3_512 ((psa_algorithm_t) 0x02000013)
• #define PSA_ALG_SHAKE256_512 ((psa_algorithm_t) 0x02000015)
• #define PSA_ALG_ANY_HASH ((psa_algorithm_t) 0x020000ff)
• #define PSA_ALG_SHAKE128 ((psa_algorithm_t) 0x0d000100)
• #define PSA_ALG_SHAKE256 ((psa_algorithm_t) 0x0d000200)
• #define PSA_ALG_XOF_CONTEXT_FLAG ((psa_algorithm_t) 0x00008000)
• #define PSA_ALG_XOF_HAS_CONTEXT(xof_alg) (((xof_alg) & PSA_ALG_XOF_CONTEXT_FLAG) != 0)
• #define PSA_ALG_MAC_SUBCATEGORY_MASK ((psa_algorithm_t) 0x00c00000)
• #define PSA_ALG_HMAC_BASE ((psa_algorithm_t) 0x03800000)
• #define PSA_ALG_HMAC(hash_alg) (PSA_ALG_HMAC_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_HMAC_GET_HASH(hmac_alg) (PSA_ALG_CATEGORY_HASH | ((hmac_alg) &
  PSA_ALG_HASH_MASK))
• #define PSA_ALG_IS_HMAC(alg)
• #define PSA_ALG_MAC_TRUNCATION_MASK ((psa_algorithm_t) 0x003f0000)
• #define PSA_MAC_TRUNCATION_OFFSET 16
• #define PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG ((psa_algorithm_t) 0x00008000)
• #define PSA_ALG_TRUNCATED_MAC(mac_alg, mac_length)
• #define PSA_ALG_FULL_LENGTH_MAC(mac_alg)

```

```

• #define PSA_MAC_TRUNCATED_LENGTH(mac_alg) (((mac_alg) & PSA_ALG_MAC_TRUNCATION_MASK)
  >> PSA_MAC_TRUNCATION_OFFSET)
• #define PSA_ALG_AT_LEAST_THIS_LENGTH_MAC(mac_alg, min_mac_length)
• #define PSA_ALG_CIPHER_MAC_BASE ((psa_algorithm_t) 0x03c00000)
• #define PSA_ALG_CBC_MAC ((psa_algorithm_t) 0x03c00100)
• #define PSA_ALG_CMAC ((psa_algorithm_t) 0x03c00200)
• #define PSA_ALG_IS_BLOCK_CIPHER_MAC(alg)
• #define PSA_ALG_CIPHER_STREAM_FLAG ((psa_algorithm_t) 0x00800000)
• #define PSA_ALG_CIPHER_FROM_BLOCK_FLAG ((psa_algorithm_t) 0x00400000)
• #define PSA_ALG_IS_STREAM_CIPHER(alg)
• #define PSA_ALG_STREAM_CIPHER ((psa_algorithm_t) 0x04800100)
• #define PSA_ALG_CTR ((psa_algorithm_t) 0x04c01000)
• #define PSA_ALG_CFB ((psa_algorithm_t) 0x04c01100)
• #define PSA_ALG_OFB ((psa_algorithm_t) 0x04c01200)
• #define PSA_ALG_XTS ((psa_algorithm_t) 0x0440ff00)
• #define PSA_ALG_ECB_NO_PADDING ((psa_algorithm_t) 0x04404400)
• #define PSA_ALG_CBC_NO_PADDING ((psa_algorithm_t) 0x04404000)
• #define PSA_ALG_CBC_PKCS7 ((psa_algorithm_t) 0x04404100)
• #define PSA_ALG_AEAD_FROM_BLOCK_FLAG ((psa_algorithm_t) 0x00400000)
• #define PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER(alg)
• #define PSA_ALG_CCM ((psa_algorithm_t) 0x05500100)
• #define PSA_ALG_CCM_STAR_NO_TAG ((psa_algorithm_t) 0x04c01300)
• #define PSA_ALG_GCM ((psa_algorithm_t) 0x05500200)
• #define PSA_ALG_CHACHA20_POLY1305 ((psa_algorithm_t) 0x05100500)
• #define PSA_ALG_AEAD_TAG_LENGTH_MASK ((psa_algorithm_t) 0x003f0000)
• #define PSA_AEAD_TAG_LENGTH_OFFSET 16
• #define PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG ((psa_algorithm_t) 0x00008000)
• #define PSA_ALG_AEAD_WITH_SHORTENED_TAG(aead_alg, tag_length)
• #define PSA_ALG_AEAD_GET_TAG_LENGTH(aead_alg)
• #define PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG(aead_alg)
• #define PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG_CASE(aead_alg, ref)
• #define PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG(aead_alg, min_tag_length)
• #define PSA_ALG_RSA_PKCS1V15_SIGN_BASE ((psa_algorithm_t) 0x06000200)
• #define PSA_ALG_RSA_PKCS1V15_SIGN(hash_alg) (PSA_ALG_RSA_PKCS1V15_SIGN_BASE |
  ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_RSA_PKCS1V15_SIGN_RAW PSA_ALG_RSA_PKCS1V15_SIGN_BASE
• #define PSA_ALG_IS_RSA_PKCS1V15_SIGN(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_PKCS1V15_SIGN_RAW)
• #define PSA_ALG_RSA_PSS_BASE ((psa_algorithm_t) 0x06000300)
• #define PSA_ALG_RSA_PSS_ANY_SALT_BASE ((psa_algorithm_t) 0x06001300)
• #define PSA_ALG_RSA_PSS(hash_alg) (PSA_ALG_RSA_PSS_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_RSA_PSS_ANY_SALT(hash_alg) (PSA_ALG_RSA_PSS_ANY_SALT_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_IS_RSA_PSS_STANDARD_SALT(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_PSS_BASE)
• #define PSA_ALG_IS_RSA_PSS_ANY_SALT(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_PSS_ANY_SALT_ANY_SALT)
• #define PSA_ALG_IS_RSA_PSS(alg)
• #define PSA_ALG_ECDSA_BASE ((psa_algorithm_t) 0x06000600)
• #define PSA_ALG_ECDSA(hash_alg) (PSA_ALG_ECDSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_ECDSA_ANY PSA_ALG_ECDSA_BASE
• #define PSA_ALG_DETERMINISTIC_ECDSA_BASE ((psa_algorithm_t) 0x06000700)
• #define PSA_ALG_DETERMINISTIC_ECDSA(hash_alg) (PSA_ALG_DETERMINISTIC_ECDSA_BASE |
  ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_ECDSA_DETERMINISTIC_FLAG ((psa_algorithm_t) 0x00000100)
• #define PSA_ALG_IS_ECDSA(alg)

```

- `#define PSA_ALG_ECDSA_IS_DETERMINISTIC(alg) (((alg) & PSA_ALG_ECDSA_DETERMINISTIC_FLAG) != 0)`
- `#define PSA_ALG_IS_DETERMINISTIC_ECDSA(alg) (PSA_ALG_IS_ECDSA(alg) && PSA_ALG_ECDSA_IS_DETERMINISTIC(alg))`
- `#define PSA_ALG_IS_RANDOMIZED_ECDSA(alg) (PSA_ALG_IS_ECDSA(alg) && !PSA_ALG_ECDSA_IS_DETERMINISTIC(alg))`
- `#define PSA_ALG_PURE_EDDSA ((psa_algorithm_t) 0x06000800)`
- `#define PSA_ALG_HASH_EDDSA_BASE ((psa_algorithm_t) 0x06000900)`
- `#define PSA_ALG_IS_HASH_EDDSA(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HASH_EDDSA_BASE)`
- `#define PSA_ALG_ED25519PH (PSA_ALG_HASH_EDDSA_BASE | (PSA_ALG_SHA_512 & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_ED448PH (PSA_ALG_HASH_EDDSA_BASE | (PSA_ALG_SHAKE256_512 & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_VENDOR_HASH_AND_SIGN(alg) 0`
- `#define PSA_ALG_IS_SIGN_HASH(alg)`
- `#define PSA_ALG_IS_SIGN_MESSAGE(alg) (PSA_ALG_IS_SIGN_HASH(alg) || (alg) == PSA_ALG_PURE_EDDSA)`
- `#define PSA_ALG_IS_HASH_AND_SIGN(alg)`
- `#define PSA_ALG_SIGN_GET_HASH(alg)`
- `#define PSA_ALG_RSA_PKCS1V15_CRYPT ((psa_algorithm_t) 0x07000200)`
- `#define PSA_ALG_RSA_OAEP_BASE ((psa_algorithm_t) 0x07000300)`
- `#define PSA_ALG_RSA_OAEP(hash_alg) (PSA_ALG_RSA_OAEP_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_RSA_OAEP(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_OAEP_BASE)`
- `#define PSA_ALG_RSA_OAEP_GET_HASH(alg)`
- `#define PSA_ALG_SP800_108_COUNTER_CMAC ((psa_algorithm_t) 0x08000800)`
- `#define PSA_ALG_IS_SP800_108_COUNTER_CMAC(alg) ((alg) == PSA_ALG_SP800_108_COUNTER_CMAC)`
- `#define PSA_ALG_HKDF_BASE ((psa_algorithm_t) 0x08000100)`
- `#define PSA_ALG_HKDF(hash_alg) (PSA_ALG_HKDF_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_HKDF(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_BASE)`
- `#define PSA_ALG_HKDF_GET_HASH(hkdf_alg) (PSA_ALG_CATEGORY_HASH | ((hkdf_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_HKDF_EXTRACT_BASE ((psa_algorithm_t) 0x08000400)`
- `#define PSA_ALG_HKDF_EXTRACT(hash_alg) (PSA_ALG_HKDF_EXTRACT_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_HKDF_EXTRACT(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_EXTRACT_BASE)`
- `#define PSA_ALG_HKDF_EXPAND_BASE ((psa_algorithm_t) 0x08000500)`
- `#define PSA_ALG_HKDF_EXPAND(hash_alg) (PSA_ALG_HKDF_EXPAND_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_HKDF_EXPAND(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_EXPAND_BASE)`
- `#define PSA_ALG_IS_ANY_HKDF(alg)`
- `#define PSA_ALG_TLS12_PRF_BASE ((psa_algorithm_t) 0x08000200)`
- `#define PSA_ALG_TLS12_PRF(hash_alg) (PSA_ALG_TLS12_PRF_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_TLS12_PRF(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_TLS12_PRF_BASE)`
- `#define PSA_ALG_TLS12_PRF_GET_HASH(hkdf_alg) (PSA_ALG_CATEGORY_HASH | ((hkdf_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_TLS12_PSK_TO_MS_BASE ((psa_algorithm_t) 0x08000300)`
- `#define PSA_ALG_TLS12_PSK_TO_MS(hash_alg) (PSA_ALG_TLS12_PSK_TO_MS_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_TLS12_PSK_TO_MS(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_TLS12_PSK_TO_MS_BASE)`
- `#define PSA_ALG_TLS12_PSK_TO_MS_GET_HASH(hkdf_alg) (PSA_ALG_CATEGORY_HASH | ((hkdf_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_TLS12_ECJPAKE_TO_PMS ((psa_algorithm_t) 0x08000609)`
- `#define PSA_ALG_KEY_DERIVATION_STRETCHING_FLAG ((psa_algorithm_t) 0x00800000)`
- `#define PSA_ALG_PBKDF2_HMAC_BASE ((psa_algorithm_t) 0x08800100)`
- `#define PSA_ALG_PBKDF2_HMAC(hash_alg) (PSA_ALG_PBKDF2_HMAC_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_PBKDF2_HMAC(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_PBKDF2_HMAC_BASE)`
- `#define PSA_ALG_PBKDF2_HMAC_GET_HASH(pbkdf2_alg) (PSA_ALG_CATEGORY_HASH | ((pbkdf2_alg) & PSA_ALG_HASH_MASK))`

- `#define PSA_ALG_PBKDF2_AES_CMAC_PRF_128 ((psa_algorithm_t) 0x08800200)`
- `#define PSA_ALG_IS_PBKDF2(kdf_alg)`
- `#define PSA_ALG_KEY_DERIVATION_MASK ((psa_algorithm_t) 0xfe00ffff)`
- `#define PSA_ALG_KEY_AGREEMENT_MASK ((psa_algorithm_t) 0xffff0000)`
- `#define PSA_ALG_KEY_AGREEMENT(ka_alg, kdf_alg) (((ka_alg) | (kdf_alg))`
- `#define PSA_ALG_KEY_AGREEMENT_GET_KDF(alg) (((alg) & PSA_ALG_KEY_DERIVATION_MASK) |`
`PSA_ALG_CATEGORY_KEY_DERIVATION)`
- `#define PSA_ALG_KEY_AGREEMENT_GET_BASE(alg) (((alg) & PSA_ALG_KEY_AGREEMENT_MASK) |`
`PSA_ALG_CATEGORY_KEY_AGREEMENT)`
- `#define PSA_ALG_IS_RAW_KEY_AGREEMENT(alg)`
- `#define PSA_ALG_IS_KEY_DERIVATION_OR_AGREEMENT(alg) ((PSA_ALG_IS_KEY_DERIVATION(alg)`
`|| PSA_ALG_IS_KEY_AGREEMENT(alg)))`
- `#define PSA_ALG_FFDH ((psa_algorithm_t) 0x09010000)`
- `#define PSA_ALG_IS_FFDH(alg) (PSA_ALG_KEY_AGREEMENT_GET_BASE(alg) == PSA_ALG_FFDH)`
- `#define PSA_ALG_ECDH ((psa_algorithm_t) 0x09020000)`
- `#define PSA_ALG_IS_ECDH(alg) (PSA_ALG_KEY_AGREEMENT_GET_BASE(alg) == PSA_ALG_ECDH)`
- `#define PSA_ALG_IS_WILDCARD(alg)`
- `#define PSA_ALG_GET_HASH(alg) (((alg) & 0x000000ff) == 0 ? ((psa_algorithm_t) 0) : 0x02000000 | ((alg)`
`& 0x000000ff))`
- `#define PSA_ALG_AES_KW ((psa_algorithm_t) 0x0B400100)`
- `#define PSA_ALG_AES_KWP ((psa_algorithm_t) 0x0BC00200)`
- `#define PSA_ALG_IS_KEY_WRAP(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEG←`
`ORY_AES_KEY_WRAP)`

Check whether the specified algorithm is a key-wrapping algorithm.

Typedefs

- `typedef uint16_t psa_key_type_t`
Encoding of a key type.
- `typedef uint8_t psa_ecc_family_t`
- `typedef uint8_t psa_dh_family_t`
- `typedef uint32_t psa_algorithm_t`
Encoding of a cryptographic algorithm.

6.24.1 Detailed Description

6.24.2 Macro Definition Documentation

6.24.2.1 PSA_AEAD_TAG_LENGTH_OFFSET

```
#define PSA_AEAD_TAG_LENGTH_OFFSET 16
```

Definition at line 1332 of file `crypto_values.h`.

6.24.2.2 PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG

```
#define PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG ((psa_algorithm_t) 0x00008000)
```

Definition at line 1340 of file `crypto_values.h`.

6.24.2.3 PSA_ALG_AEAD_FROM_BLOCK_FLAG

```
#define PSA_ALG_AEAD_FROM_BLOCK_FLAG ((psa_algorithm_t) 0x00400000)
```

Definition at line 1278 of file `crypto_values.h`.

6.24.2.4 PSA_ALG_AEAD_GET_TAG_LENGTH

```
#define PSA_ALG_AEAD_GET_TAG_LENGTH(  
    aead_alg )
```

Value:

```
((aead_alg) & PSA_ALG_AEAD_TAG_LENGTH_MASK) »  
PSA_AEAD_TAG_LENGTH_OFFSET) \
```

Retrieve the tag length of a specified AEAD algorithm

Parameters

<i>aead_alg</i>	An AEAD algorithm identifier (value of type <code>psa_algorithm_t</code> such that <code>PSA_ALG_IS_AEAD(aead_alg)</code> is true).
-----------------	---

Returns

The tag length specified by the input algorithm.

Unspecified if `aead_alg` is not a supported AEAD algorithm.

Definition at line 1376 of file `crypto_values.h`.

6.24.2.5 PSA_ALG_AEAD_TAG_LENGTH_MASK

```
#define PSA_ALG_AEAD_TAG_LENGTH_MASK ((psa_algorithm_t) 0x003f0000)
```

Definition at line 1331 of file `crypto_values.h`.

6.24.2.6 PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG

```
#define PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG(
    aead_alg,
    min_tag_length )
```

Value:

```
(PSA_ALG_AEAD_WITH_SHORTENED_TAG(aead_alg, min_tag_length) |
    PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG)
```

Macro to build an AEAD minimum-tag-length wildcard algorithm.

A minimum-tag-length AEAD wildcard algorithm permits all AEAD algorithms sharing the same base algorithm, and where the tag length of the specific algorithm is equal to or larger then the minimum tag length specified by the wildcard algorithm.

Note

When setting the minimum required tag length to less than the smallest tag length allowed by the base algorithm, this effectively becomes an 'any-tag-length-allowed' policy for that base algorithm.

Parameters

<i>aead_alg</i>	An AEAD algorithm identifier (value of type psa_algorithm_t such that PSA_ALG_IS_AEAD (aead_alg) is true).
<i>min_tag_length</i>	Desired minimum length of the authentication tag in bytes. This must be at least 1 and at most the largest allowed tag length of the algorithm.

Returns

The corresponding AEAD wildcard algorithm with the specified minimum length.

Unspecified if *aead_alg* is not a supported AEAD algorithm or if *min_tag_length* is less than 1 or too large for the specified AEAD algorithm.

Definition at line 1423 of file `crypto_values.h`.

6.24.2.7 PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG

```
#define PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG(
    aead_alg )
```

Value:

```
(
    PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG_CASE(aead_alg, PSA_ALG_CCM) \
    PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG_CASE(aead_alg, PSA_ALG_GCM) \
    PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG_CASE(aead_alg, PSA_ALG_CHACHA20_POLY1305) \
    0)
```

Calculate the corresponding AEAD algorithm with the default tag length.

Parameters

<i>aead_alg</i>	An AEAD algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_AEAD (<i>aead_alg</i>) is true).
-----------------	--

Returns

The corresponding AEAD algorithm with the default tag length for that algorithm.

Definition at line 1388 of file `crypto_values.h`.

6.24.2.8 PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG_CASE

```
#define PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG_CASE(  
    aead_alg,  
    ref )
```

Value:

```
PSA_ALG_AEAD_WITH_SHORTENED_TAG(aead_alg, 0) ==  
PSA_ALG_AEAD_WITH_SHORTENED_TAG(ref, 0) ?  
ref :
```

Definition at line 1394 of file `crypto_values.h`.

6.24.2.9 PSA_ALG_AEAD_WITH_SHORTENED_TAG

```
#define PSA_ALG_AEAD_WITH_SHORTENED_TAG(  
    aead_alg,  
    tag_length )
```

Value:

```
((aead_alg) & ~(PSA_ALG_AEAD_TAG_LENGTH_MASK |  
                PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG)) |  
((tag_length) << PSA_ALG_AEAD_TAG_LENGTH_OFFSET &  
 PSA_ALG_AEAD_TAG_LENGTH_MASK)
```

Macro to build a shortened AEAD algorithm.

A shortened AEAD algorithm is similar to the corresponding AEAD algorithm, but has an authentication tag that consists of fewer bytes. Depending on the algorithm, the tag length may affect the calculation of the ciphertext.

Parameters

<i>aead_alg</i>	An AEAD algorithm identifier (value of type psa_algorithm_t such that PSA_ALG_IS_AEAD (<i>aead_alg</i>) is true).
<i>tag_length</i>	Desired length of the authentication tag in bytes.

Returns

The corresponding AEAD algorithm with the specified length.

Unspecified if `aead_alg` is not a supported AEAD algorithm or if `tag_length` is not valid for the specified AEAD algorithm.

Definition at line 1360 of file `crypto_values.h`.

6.24.2.10 PSA_ALG_AES_KW

```
#define PSA_ALG_AES_KW ((psa_algorithm_t)0x0B400100)
```

The AES-KW key-wrapping algorithm.

This is the NIST Key Wrap algorithm, using an AES key-encryption key, as defined in [NIST SP 800-38F](#). The algorithm is also specified in [RFC 3394](#).

Keys to be wrapped must have a length that is a multiple of the AES 'semi-block' size — that is, a multiple of 8 bytes.

To wrap keys whose lengths are not a multiple of the AES semi-block size, use `PSA_ALG_AES_KWP`.

Definition at line 2339 of file `crypto_values.h`.

6.24.2.11 PSA_ALG_AES_KWP

```
#define PSA_ALG_AES_KWP ((psa_algorithm_t)0x0BC00200)
```

The AES-KWP key-wrapping algorithm with padding.

This is the NIST Key Wrap with Padding algorithm, using an AES key-encryption key, as defined in [NIST SP 800-38F](#). The algorithm is also specified in [RFC 5649](#).

This algorithm can wrap a key of any length.

Definition at line 2351 of file `crypto_values.h`.

6.24.2.12 PSA_ALG_ANY_HASH

```
#define PSA_ALG_ANY_HASH ((psa_algorithm_t) 0x020000ff)
```

In a hash-and-sign algorithm policy, allow any hash algorithm.

This value may be used to form the algorithm usage field of a policy for a signature algorithm that is parametrized by a hash. The key may then be used to perform operations using the same signature algorithm parametrized with any supported hash.

That is, suppose that `PSA_XXX_SIGNATURE` is one of the following macros:

- `PSA_ALG_RSA_PKCS1V15_SIGN`, `PSA_ALG_RSA_PSS`, `PSA_ALG_RSA_PSS_ANY_SALT`,
- `PSA_ALG_ECDSA`, `PSA_ALG_DETERMINISTIC_ECDSA`. Then you may create and use a key as follows:
- Set the key usage field using `PSA_ALG_ANY_HASH`, for example:

```
psa_set_key_usage_flags(&attributes, PSA_KEY_USAGE_SIGN_HASH); // or VERIFY
psa_set_key_algorithm(&attributes, PSA_XXX_SIGNATURE(PSA_ALG_ANY_HASH));
```
- Import or generate key material.
- Call `psa_sign_hash()` or `psa_verify_hash()`, passing an algorithm built from `PSA_XXX_SIGNATURE` and a specific hash. Each call to sign or verify a message may use a different hash.

```
psa_sign_hash(key, PSA_XXX_SIGNATURE(PSA_ALG_SHA_256), ...);
psa_sign_hash(key, PSA_XXX_SIGNATURE(PSA_ALG_SHA_512), ...);
psa_sign_hash(key, PSA_XXX_SIGNATURE(PSA_ALG_SHA3_256), ...);
```

This value may not be used to build other algorithms that are parametrized over a hash. For any valid use of this macro to build an algorithm `alg`, `PSA_ALG_IS_HASH_AND_SIGN(alg)` is true.

This value may not be used to build an algorithm specification to perform an operation. It is only valid to build policies.

Definition at line 985 of file `crypto_values.h`.

6.24.2.13 PSA_ALG_AT_LEAST_THIS_LENGTH_MAC

```
#define PSA_ALG_AT_LEAST_THIS_LENGTH_MAC(
    mac_alg,
    min_mac_length )
```

Value:

```
(PSA_ALG_TRUNCATED_MAC(mac_alg, min_mac_length) |
 PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG)
```

Macro to build a MAC minimum-MAC-length wildcard algorithm.

A minimum-MAC-length MAC wildcard algorithm permits all MAC algorithms sharing the same base algorithm, and where the (potentially truncated) MAC length of the specific algorithm is equal to or larger than the wildcard algorithm's minimum MAC length.

Note

When setting the minimum required MAC length to less than the smallest MAC length allowed by the base algorithm, this effectively becomes an 'any-MAC-length-allowed' policy for that base algorithm.

Parameters

<i>mac_alg</i>	A MAC algorithm identifier (value of type psa_algorithm_t such that PSA_ALG_IS_MAC (<i>mac_alg</i>) is true).
<i>min_mac_length</i>	Desired minimum length of the message authentication code in bytes. This must be at most the untruncated length of the MAC and must be at least 1.

Returns

The corresponding MAC wildcard algorithm with the specified minimum length.

Unspecified if *mac_alg* is not a supported MAC algorithm or if *min_mac_length* is less than 1 or too large for the specified MAC algorithm.

Definition at line 1160 of file `crypto_values.h`.

6.24.2.14 PSA_ALG_CATEGORY_AEAD

```
#define PSA_ALG_CATEGORY_AEAD ((psa_algorithm_t) 0x05000000)
```

Definition at line 776 of file `crypto_values.h`.

6.24.2.15 PSA_ALG_CATEGORY_ASYMMETRIC_ENCRYPTION

```
#define PSA_ALG_CATEGORY_ASYMMETRIC_ENCRYPTION ((psa_algorithm_t) 0x07000000)
```

Definition at line 778 of file `crypto_values.h`.

6.24.2.16 PSA_ALG_CATEGORY_CIPHER

```
#define PSA_ALG_CATEGORY_CIPHER ((psa_algorithm_t) 0x04000000)
```

Definition at line 775 of file `crypto_values.h`.

6.24.2.17 PSA_ALG_CATEGORY_HASH

```
#define PSA_ALG_CATEGORY_HASH ((psa_algorithm_t) 0x02000000)
```

Definition at line 773 of file `crypto_values.h`.

6.24.2.18 PSA_ALG_CATEGORY_KEY_AGREEMENT

```
#define PSA_ALG_CATEGORY_KEY_AGREEMENT ((psa_algorithm_t) 0x09000000)
```

Definition at line 780 of file crypto_values.h.

6.24.2.19 PSA_ALG_CATEGORY_KEY_DERIVATION

```
#define PSA_ALG_CATEGORY_KEY_DERIVATION ((psa_algorithm_t) 0x08000000)
```

Definition at line 779 of file crypto_values.h.

6.24.2.20 PSA_ALG_CATEGORY_KEY_WRAP

```
#define PSA_ALG_CATEGORY_KEY_WRAP ((psa_algorithm_t) 0x0B000000)
```

Definition at line 782 of file crypto_values.h.

6.24.2.21 PSA_ALG_CATEGORY_MAC

```
#define PSA_ALG_CATEGORY_MAC ((psa_algorithm_t) 0x03000000)
```

Definition at line 774 of file crypto_values.h.

6.24.2.22 PSA_ALG_CATEGORY_MASK

```
#define PSA_ALG_CATEGORY_MASK ((psa_algorithm_t) 0x7f000000)
```

Definition at line 772 of file crypto_values.h.

6.24.2.23 PSA_ALG_CATEGORY_PAKE

```
#define PSA_ALG_CATEGORY_PAKE ((psa_algorithm_t) 0x0a000000)
```

Definition at line 595 of file crypto_extra.h.

6.24.2.24 PSA_ALG_CATEGORY_SIGN

```
#define PSA_ALG_CATEGORY_SIGN ((psa_algorithm_t) 0x06000000)
```

Definition at line 777 of file `crypto_values.h`.

6.24.2.25 PSA_ALG_CATEGORY_XOF

```
#define PSA_ALG_CATEGORY_XOF ((psa_algorithm_t) 0x0d000000)
```

Definition at line 781 of file `crypto_values.h`.

6.24.2.26 PSA_ALG_CBC_MAC

```
#define PSA_ALG_CBC_MAC ((psa_algorithm_t) 0x03c00100)
```

The CBC-MAC construction over a block cipher

Warning

CBC-MAC is insecure in many cases. A more secure mode, such as [PSA_ALG_CMAC](#), is recommended.

Definition at line 1170 of file `crypto_values.h`.

6.24.2.27 PSA_ALG_CBC_NO_PADDING

```
#define PSA_ALG_CBC_NO_PADDING ((psa_algorithm_t) 0x04404000)
```

The CBC block cipher chaining mode, with no padding.

The underlying block cipher is determined by the key type.

This symmetric cipher mode can only be used with messages whose lengths are whole number of blocks for the chosen block cipher.

Definition at line 1268 of file `crypto_values.h`.

6.24.2.28 PSA_ALG_CBC_PKCS7

```
#define PSA_ALG_CBC_PKCS7 ((psa_algorithm_t) 0x04404100)
```

The CBC block cipher chaining mode with PKCS#7 padding.

The underlying block cipher is determined by the key type.

This is the padding method defined by PKCS#7 (RFC 2315) §10.3.

Definition at line 1276 of file `crypto_values.h`.

6.24.2.29 PSA_ALG_CCM

```
#define PSA_ALG_CCM ((psa_algorithm_t) 0x05500100)
```

The CCM authenticated encryption algorithm.

The underlying block cipher is determined by the key type.

Definition at line 1297 of file `crypto_values.h`.

6.24.2.30 PSA_ALG_CCM_STAR_NO_TAG

```
#define PSA_ALG_CCM_STAR_NO_TAG ((psa_algorithm_t) 0x04c01300)
```

The CCM* cipher mode without authentication.

This is CCM* as specified in IEEE 802.15.4 §7, with a tag length of 0. For CCM* with a nonzero tag length, use the AEAD algorithm [PSA_ALG_CCM](#).

The underlying block cipher is determined by the key type.

Currently only 13-byte long IV's are supported.

Definition at line 1308 of file `crypto_values.h`.

6.24.2.31 PSA_ALG_CFB

```
#define PSA_ALG_CFB ((psa_algorithm_t) 0x04c01100)
```

The CFB stream cipher mode.

The underlying block cipher is determined by the key type.

Definition at line 1225 of file `crypto_values.h`.

6.24.2.32 PSA_ALG_CHACHA20_POLY1305

```
#define PSA_ALG_CHACHA20_POLY1305 ((psa_algorithm_t) 0x05100500)
```

The Chacha20-Poly1305 AEAD algorithm.

The ChaCha20_Poly1305 construction is defined in RFC 7539.

Implementations must support 12-byte nonces, may support 8-byte nonces, and should reject other sizes.

Implementations must support 16-byte tags and should reject other sizes.

Definition at line 1325 of file crypto_values.h.

6.24.2.33 PSA_ALG_CIPHER_FROM_BLOCK_FLAG

```
#define PSA_ALG_CIPHER_FROM_BLOCK_FLAG ((psa_algorithm_t) 0x00400000)
```

Definition at line 1187 of file crypto_values.h.

6.24.2.34 PSA_ALG_CIPHER_MAC_BASE

```
#define PSA_ALG_CIPHER_MAC_BASE ((psa_algorithm_t) 0x03c00000)
```

Definition at line 1164 of file crypto_values.h.

6.24.2.35 PSA_ALG_CIPHER_STREAM_FLAG

```
#define PSA_ALG_CIPHER_STREAM_FLAG ((psa_algorithm_t) 0x00800000)
```

Definition at line 1186 of file crypto_values.h.

6.24.2.36 PSA_ALG_CMAC

```
#define PSA_ALG_CMAC ((psa_algorithm_t) 0x03c00200)
```

The CMAC construction over a block cipher

Definition at line 1172 of file crypto_values.h.

6.24.2.37 PSA_ALG_CTR

```
#define PSA_ALG_CTR ((psa_algorithm_t) 0x04c01000)
```

The CTR stream cipher mode.

CTR is a stream cipher which is built from a block cipher. The underlying block cipher is determined by the key type. For example, to use AES-128-CTR, use this algorithm with a key of type [PSA_KEY_TYPE_AES](#) and a length of 128 bits (16 bytes).

Definition at line 1219 of file `crypto_values.h`.

6.24.2.38 PSA_ALG_DETERMINISTIC_DSA

```
#define PSA_ALG_DETERMINISTIC_DSA(  
    hash_alg ) (PSA_ALG_DETERMINISTIC_DSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

Deterministic DSA signature with hashing.

This is the deterministic variant defined by RFC 6979 of the signature scheme defined by FIPS 186-4.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (<i>hash_alg</i>) is true). This includes PSA_ALG_ANY_HASH when specifying the algorithm in a usage policy.
-----------------	---

Returns

The corresponding DSA signature algorithm.

Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 228 of file `crypto_extra.h`.

6.24.2.39 PSA_ALG_DETERMINISTIC_DSA_BASE

```
#define PSA_ALG_DETERMINISTIC_DSA_BASE ((psa_algorithm_t) 0x06000500)
```

Definition at line 212 of file `crypto_extra.h`.

6.24.2.40 PSA_ALG_DETERMINISTIC_ECDSA

```
#define PSA_ALG_DETERMINISTIC_ECDSA(  
    hash_alg ) (PSA_ALG_DETERMINISTIC_ECDSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

Deterministic ECDSA signature with hashing.

This is the deterministic ECDSA signature scheme defined by RFC 6979.

The representation of a signature is the same as with [PSA_ALG_ECDSA\(\)](#).

Note that when this algorithm is used for verification, signatures made with randomized ECDSA ([PSA_ALG_ECDSA](#)(hash_alg)) with the same private key are accepted. In other words, [PSA_ALG_DETERMINISTIC_ECDSA](#)(hash_alg) differs from [PSA_ALG_ECDSA](#)(hash_alg) only for signature, not for verification.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (<i>hash_alg</i>) is true). This includes PSA_ALG_ANY_HASH when specifying the algorithm in a usage policy.
-----------------	---

Returns

The corresponding deterministic ECDSA signature algorithm.

Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 1601 of file `crypto_values.h`.

6.24.2.41 PSA_ALG_DETERMINISTIC_ECDSA_BASE

```
#define PSA_ALG_DETERMINISTIC_ECDSA_BASE ((psa_algorithm_t) 0x06000700)
```

Definition at line 1578 of file `crypto_values.h`.

6.24.2.42 PSA_ALG_DSA

```
#define PSA_ALG_DSA(  
    hash_alg ) (PSA_ALG_DSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

DSA signature with hashing.

This is the signature scheme defined by FIPS 186-4, with a random per-message secret number (*k*).

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (<i>hash_alg</i>) is true). This includes PSA_ALG_ANY_HASH when specifying the algorithm in a usage policy.
-----------------	---

Returns

The corresponding DSA signature algorithm.

Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 210 of file `crypto_extra.h`.

6.24.2.43 PSA_ALG_DSA_BASE

```
#define PSA_ALG_DSA_BASE ((psa_algorithm_t) 0x06000400)
```

Definition at line 195 of file `crypto_extra.h`.

6.24.2.44 PSA_ALG_DSA_DETERMINISTIC_FLAG

```
#define PSA_ALG_DSA_DETERMINISTIC_FLAG PSA_ALG_ECDSA_DETERMINISTIC_FLAG
```

Definition at line 213 of file `crypto_extra.h`.

6.24.2.45 PSA_ALG_DSA_IS_DETERMINISTIC

```
#define PSA_ALG_DSA_IS_DETERMINISTIC(  
    alg ) (((alg) & PSA_ALG_DSA_DETERMINISTIC_FLAG) != 0)
```

Definition at line 233 of file `crypto_extra.h`.

6.24.2.46 PSA_ALG_ECB_NO_PADDING

```
#define PSA_ALG_ECB_NO_PADDING ((psa_algorithm_t) 0x04404400)
```

The Electronic Code Book (ECB) mode of a block cipher, with no padding.

Warning

ECB mode does not protect the confidentiality of the encrypted data except in extremely narrow circumstances. It is recommended that applications only use ECB if they need to construct an operating mode that the implementation does not provide. Implementations are encouraged to provide the modes that applications need in preference to supporting direct access to ECB.

The underlying block cipher is determined by the key type.

This symmetric cipher mode can only be used with messages whose lengths are a multiple of the block size of the chosen block cipher.

ECB mode does not accept an initialization vector (IV). When using a multi-part cipher operation with this algorithm, `psa_cipher_generate_iv()` and `psa_cipher_set_iv()` must not be called.

Definition at line 1259 of file `crypto_values.h`.

6.24.2.47 PSA_ALG_ECDH

```
#define PSA_ALG_ECDH ((psa_algorithm_t) 0x09020000)
```

The elliptic curve Diffie-Hellman (ECDH) key agreement algorithm.

The shared secret produced by key agreement is the x-coordinate of the shared secret point. It is always $\text{ceiling}(m / 8)$ bytes long where m is the bit size associated with the curve, i.e. the bit size of the order of the curve's coordinate field. When m is not a multiple of 8, the byte containing the most significant bit of the shared secret is padded with zero bits. The byte order is either little-endian or big-endian depending on the curve type.

- For Montgomery curves (curve types `PSA_ECC_FAMILY_CURVEXXX`), the shared secret is the x-coordinate of $d_A Q_B = d_B Q_A$ in little-endian byte order. The bit size is 448 for Curve448 and 255 for Curve25519.
- For Weierstrass curves over prime fields (curve types `PSA_ECC_FAMILY_SECPXXX` and `PSA_ECC_FAMILY_BRAINPOOL_PXXX`), the shared secret is the x-coordinate of $d_A Q_B = d_B Q_A$ in big-endian byte order. The bit size is $m = \text{ceiling}(\log_2(p))$ for the field F_p .
- For Weierstrass curves over binary fields (curve types `PSA_ECC_FAMILY_SECTXXX`), the shared secret is the x-coordinate of $d_A Q_B = d_B Q_A$ in big-endian byte order. The bit size is m for the field F_{2^m} .

Definition at line 2272 of file `crypto_values.h`.

6.24.2.48 PSA_ALG_ECDSA

```
#define PSA_ALG_ECDSA(  
    hash_alg ) (PSA_ALG_ECDSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

ECDSA signature with hashing.

This is the ECDSA signature scheme defined by ANSI X9.62, with a random per-message secret number (k).

The representation of the signature as a byte string consists of the concatenation of the signature values r and s . Each of r and s is encoded as an N -octet string, where N is the length of the base point of the curve in octets. Each value is represented in big-endian order (most significant octet first).

Parameters

<i>hash_alg</i>	A hash algorithm (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_HASH(hash_alg)</code> is true). This includes <code>PSA_ALG_ANY_HASH</code> when specifying the algorithm in a usage policy.
-----------------	---

Returns

The corresponding ECDSA signature algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

Definition at line 1566 of file `crypto_values.h`.

6.24.2.49 PSA_ALG_ECDSA_ANY

```
#define PSA_ALG_ECDSA_ANY PSA_ALG_ECDSA_BASE
```

ECDSA signature without hashing.

This is the same signature scheme as [PSA_ALG_ECDSA\(\)](#), but without specifying a hash algorithm. This algorithm may only be used to sign or verify a sequence of bytes that should be an already-calculated hash. Note that the input is padded with zeros on the left or truncated on the left as required to fit the curve size.

Definition at line 1577 of file `crypto_values.h`.

6.24.2.50 PSA_ALG_ECDSA_BASE

```
#define PSA_ALG_ECDSA_BASE ((psa_algorithm_t) 0x06000600)
```

Definition at line 1545 of file `crypto_values.h`.

6.24.2.51 PSA_ALG_ECDSA_DETERMINISTIC_FLAG

```
#define PSA_ALG_ECDSA_DETERMINISTIC_FLAG ((psa_algorithm_t) 0x00000100)
```

Definition at line 1603 of file `crypto_values.h`.

6.24.2.52 PSA_ALG_ECDSA_IS_DETERMINISTIC

```
#define PSA_ALG_ECDSA_IS_DETERMINISTIC(  
    alg) (((alg) & PSA_ALG_ECDSA_DETERMINISTIC_FLAG) != 0)
```

Definition at line 1607 of file `crypto_values.h`.

6.24.2.53 PSA_ALG_ED25519PH

```
#define PSA_ALG_ED25519PH (PSA_ALG_HASH_EDDSA_BASE | (PSA_ALG_SHA_512 & PSA_ALG_HASH_MASK))
```

Edwards-curve digital signature algorithm with prehashing (HashEdDSA), using SHA-512 and the Edwards25519 curve.

See [PSA_ALG_PURE_EDDSA](#) regarding context support and the signature format.

This algorithm is Ed25519 as specified in RFC 8032. The curve is Edwards25519. The prehash is SHA-512. The hash function used internally is SHA-512.

This is a hash-and-sign algorithm: to calculate a signature, you can either:

- call [psa_sign_message\(\)](#) on the message;
- or calculate the SHA-512 hash of the message with [psa_hash_compute\(\)](#) or with a multi-part hash operation started with [psa_hash_setup\(\)](#), using the hash algorithm [PSA_ALG_SHA_512](#), then sign the calculated hash with [psa_sign_hash\(\)](#). Verifying a signature is similar, using [psa_verify_message\(\)](#) or [psa_verify_hash\(\)](#) instead of the signature function.

Definition at line 1669 of file `crypto_values.h`.

6.24.2.54 PSA_ALG_ED448PH

```
#define PSA_ALG_ED448PH (PSA_ALG_HASH_EDDSA_BASE | (PSA_ALG_SHAKE256_512 & PSA_ALG_HASH_MASK))
```

Edwards-curve digital signature algorithm with prehashing (HashEdDSA), using SHAKE256 and the Edwards448 curve.

See [PSA_ALG_PURE_EDDSA](#) regarding context support and the signature format.

This algorithm is Ed448 as specified in RFC 8032. The curve is Edwards448. The prehash is the first 64 bytes of the SHAKE256 output. The hash function used internally is the first 114 bytes of the SHAKE256 output.

This is a hash-and-sign algorithm: to calculate a signature, you can either:

- call [psa_sign_message\(\)](#) on the message;
- or calculate the first 64 bytes of the SHAKE256 output of the message with [psa_hash_compute\(\)](#) or with a multi-part hash operation started with [psa_hash_setup\(\)](#), using the hash algorithm [PSA_ALG_SHAKE256_512](#), then sign the calculated hash with [psa_sign_hash\(\)](#). Verifying a signature is similar, using [psa_verify_message\(\)](#) or [psa_verify_hash\(\)](#) instead of the signature function.

Definition at line 1694 of file `crypto_values.h`.

6.24.2.55 PSA_ALG_FFDH

```
#define PSA_ALG_FFDH ((psa_algorithm_t) 0x09010000)
```

The finite-field Diffie-Hellman (DH) key agreement algorithm.

The shared secret produced by key agreement is g^{ab} in big-endian format. It is $\text{ceiling}(m / 8)$ bytes long where m is the size of the prime p in bits.

Definition at line 2230 of file `crypto_values.h`.

6.24.2.56 PSA_ALG_FULL_LENGTH_MAC

```
#define PSA_ALG_FULL_LENGTH_MAC(  
    mac_alg )
```

Value:

```
((mac_alg) & ~(PSA_ALG_MAC_TRUNCATION_MASK |  
               PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG)) \
```

Macro to build the base MAC algorithm corresponding to a truncated MAC algorithm.

Parameters

<i>mac_alg</i>	A MAC algorithm identifier (value of type psa_algorithm_t such that PSA_ALG_IS_MAC (<i>mac_alg</i>) is true). This may be a truncated or untruncated MAC algorithm.
----------------	---

Returns

The corresponding base MAC algorithm.

Unspecified if `mac_alg` is not a supported MAC algorithm.

Definition at line 1118 of file `crypto_values.h`.

6.24.2.57 PSA_ALG_GCM

```
#define PSA_ALG_GCM ((psa_algorithm_t) 0x05500200)
```

The GCM authenticated encryption algorithm.

The underlying block cipher is determined by the key type.

Definition at line 1314 of file `crypto_values.h`.

6.24.2.58 PSA_ALG_GET_HASH

```
#define PSA_ALG_GET_HASH(  
    alg ) (((alg) & 0x000000ff) == 0 ? ((psa_algorithm_t) 0) : 0x02000000 | ((alg)  
& 0x000000ff))
```

Get the hash used by a composite algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

The underlying hash algorithm if `alg` is a composite algorithm that uses a hash algorithm.

0 if `alg` is not a composite algorithm that uses a hash.

Definition at line 2322 of file `crypto_values.h`.

6.24.2.59 PSA_ALG_HASH_EDDSA_BASE

```
#define PSA_ALG_HASH_EDDSA_BASE ((psa_algorithm_t) 0x06000900)
```

Definition at line 1644 of file `crypto_values.h`.

6.24.2.60 PSA_ALG_HASH_MASK

```
#define PSA_ALG_HASH_MASK ((psa_algorithm_t) 0x000000ff)
```

Definition at line 917 of file `crypto_values.h`.

6.24.2.61 PSA_ALG_HKDF

```
#define PSA_ALG_HKDF(  
    hash_alg ) (PSA_ALG_HKDF_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

Macro to build an HKDF algorithm.

For example, `PSA_ALG_HKDF(PSA_ALG_SHA_256)` is HKDF using HMAC-SHA-256.

This key derivation algorithm uses the following inputs:

- `PSA_KEY_DERIVATION_INPUT_SALT` is the salt used in the "extract" step. It is optional; if omitted, the derivation uses an empty salt.
- `PSA_KEY_DERIVATION_INPUT_SECRET` is the secret key used in the "extract" step.
- `PSA_KEY_DERIVATION_INPUT_INFO` is the info string used in the "expand" step. You must pass `PSA_KEY_DERIVATION_INPUT_SALT` before `PSA_KEY_DERIVATION_INPUT_SECRET`. You may pass `PSA_KEY_DERIVATION_INPUT_INFO` at any time after `steup` and before starting to generate output.

Warning

HKDF processes the salt as follows: first hash it with `hash_alg` if the salt is longer than the block size of the hash algorithm; then pad with null bytes up to the block size. As a result, it is possible for distinct salt inputs to result in the same outputs. To ensure unique outputs, it is recommended to use a fixed length for salt values.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that <code>PSA_ALG_IS_HASH(hash_alg)</code> is true).
-----------------	--

Returns

The corresponding HKDF algorithm.
Unspecified if `hash_alg` is not a supported hash algorithm.

Definition at line 1866 of file `crypto_values.h`.

6.24.2.62 PSA_ALG_HKDF_BASE

```
#define PSA_ALG_HKDF_BASE ((psa_algorithm_t) 0x08000100)
```

Definition at line 1839 of file `crypto_values.h`.

6.24.2.63 PSA_ALG_HKDF_EXPAND

```
#define PSA_ALG_HKDF_EXPAND(  
    hash_alg ) (PSA_ALG_HKDF_EXPAND_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

Macro to build an HKDF-Expand algorithm.

For example, `PSA_ALG_HKDF_EXPAND(PSA_ALG_SHA_256)` is HKDF-Expand using HMAC-SHA-256.

This key derivation algorithm uses the following inputs:

- `PSA_KEY_DERIVATION_INPUT_SECRET` is the pseudorandom key (PRK).
- `PSA_KEY_DERIVATION_INPUT_INFO` is the info string.

The inputs are mandatory and must be passed in the order above. Each input may only be passed once.

Warning

HKDF-Expand is not meant to be used on its own. `PSA_ALG_HKDF` should be used instead if possible. `PSA_ALG_HKDF_EXPAND` is provided as a separate algorithm for the sake of protocols that use it as a building block. It may also be a slight performance optimization in applications that use HKDF with the same salt and key but many different info strings.

Parameters

<i>hash_alg</i>	A hash algorithm (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_HASH(hash_alg)</code> is true).
-----------------	--

Returns

The corresponding HKDF-Expand algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

Definition at line 1959 of file `crypto_values.h`.

6.24.2.64 PSA_ALG_HKDF_EXPAND_BASE

```
#define PSA_ALG_HKDF_EXPAND_BASE ((psa_algorithm_t) 0x08000500)
```

Definition at line 1933 of file `crypto_values.h`.

6.24.2.65 PSA_ALG_HKDF_EXTRACT

```
#define PSA_ALG_HKDF_EXTRACT(  
    hash_alg ) (PSA_ALG_HKDF_EXTRACT_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

Macro to build an HKDF-Extract algorithm.

For example, `PSA_ALG_HKDF_EXTRACT(PSA_ALG_SHA_256)` is HKDF-Extract using HMAC-SHA-256.

This key derivation algorithm uses the following inputs:

- `PSA_KEY_DERIVATION_INPUT_SALT` is the salt.
- `PSA_KEY_DERIVATION_INPUT_SECRET` is the input keying material used in the "extract" step. The inputs are mandatory and must be passed in the order above. Each input may only be passed once.

Warning

HKDF-Extract is not meant to be used on its own. `PSA_ALG_HKDF` should be used instead if possible. `PSA_ALG_HKDF_EXTRACT` is provided as a separate algorithm for the sake of protocols that use it as a building block. It may also be a slight performance optimization in applications that use HKDF with the same salt and key but many different info strings.

HKDF processes the salt as follows: first hash it with `hash_alg` if the salt is longer than the block size of the hash algorithm; then pad with null bytes up to the block size. As a result, it is possible for distinct salt inputs to result in the same outputs. To ensure unique outputs, it is recommended to use a fixed length for salt values.

Parameters

<i>hash_alg</i>	A hash algorithm (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_HASH(hash_alg)</code> is true).
-----------------	--

Returns

The corresponding HKDF-Extract algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

Definition at line 1917 of file `crypto_values.h`.

6.24.2.66 PSA_ALG_HKDF_EXTRACT_BASE

```
#define PSA_ALG_HKDF_EXTRACT_BASE ( (psa_algorithm_t) 0x08000400)
```

Definition at line 1884 of file `crypto_values.h`.

6.24.2.67 PSA_ALG_HKDF_GET_HASH

```
#define PSA_ALG_HKDF_GET_HASH(  
    hkdf_alg ) (PSA_ALG_CATEGORY_HASH | ((hkdf_alg) & PSA_ALG_HASH_MASK))
```

Definition at line 1881 of file `crypto_values.h`.

6.24.2.68 PSA_ALG_HMAC

```
#define PSA_ALG_HMAC(  
    hash_alg ) (PSA_ALG_HMAC_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

Macro to build an HMAC algorithm.

For example, `PSA_ALG_HMAC(PSA_ALG_SHA_256)` is HMAC-SHA-256.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that <code>PSA_ALG_IS_HASH(hash_alg)</code> is true).
-----------------	--

Returns

The corresponding HMAC algorithm.

Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 1030 of file `crypto_values.h`.

6.24.2.69 PSA_ALG_HMAC_BASE

```
#define PSA_ALG_HMAC_BASE ((psa_algorithm_t) 0x03800000)
```

Definition at line 1018 of file `crypto_values.h`.

6.24.2.70 PSA_ALG_HMAC_GET_HASH

```
#define PSA_ALG_HMAC_GET_HASH(  
    hmac_alg ) (PSA_ALG_CATEGORY_HASH | ((hmac_alg) & PSA_ALG_HASH_MASK))
```

Definition at line 1033 of file `crypto_values.h`.

6.24.2.71 PSA_ALG_IS_AEAD

```
#define PSA_ALG_IS_AEAD(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_AEAD)
```

Whether the specified algorithm is an authenticated encryption with associated data (AEAD) algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is an AEAD algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 833 of file `crypto_values.h`.

6.24.2.72 PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER

```
#define PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER(  
    alg )
```

Value:

```
((alg) & (PSA_ALG_CATEGORY_MASK | PSA_ALG_AEAD_FROM_BLOCK_FLAG)) == \  
(PSA_ALG_CATEGORY_AEAD | PSA_ALG_AEAD_FROM_BLOCK_FLAG)
```

Whether the specified algorithm is an AEAD mode on a block cipher.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is an AEAD algorithm which is an AEAD mode based on a block cipher, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 1289 of file `crypto_values.h`.

6.24.2.73 PSA_ALG_IS_ANY_HKDF

```
#define PSA_ALG_IS_ANY_HKDF(  
    alg )
```

Value:

```
((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_BASE ||  
(alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_EXTRACT_BASE ||  
(alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_EXPAND_BASE)
```

Whether the specified algorithm is an HKDF or HKDF-Extract or HKDF-Expand algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if `alg` is any HKDF type algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

Definition at line 1985 of file `crypto_values.h`.

6.24.2.74 PSA_ALG_IS_ASYMMETRIC_ENCRYPTION

```
#define PSA_ALG_IS_ASYMMETRIC_ENCRYPTION(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_ASYMMETRIC_ENCRYPTION)
```

Whether the specified algorithm is an asymmetric encryption algorithm, also known as public-key encryption algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is an asymmetric encryption algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 857 of file `crypto_values.h`.

6.24.2.75 PSA_ALG_IS_BLOCK_CIPHER_MAC

```
#define PSA_ALG_IS_BLOCK_CIPHER_MAC(  
    alg )
```

Value:

```
((alg) & (PSA_ALG_CATEGORY_MASK | PSA_ALG_MAC_SUBCATEGORY_MASK)) == \  
PSA_ALG_CIPHER_MAC_BASE)
```

Whether the specified algorithm is a MAC algorithm based on a block cipher.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is a MAC algorithm based on a block cipher, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 1182 of file `crypto_values.h`.

6.24.2.76 PSA_ALG_IS_CIPHER

```
#define PSA_ALG_IS_CIPHER(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_CIPHER)
```

Whether the specified algorithm is a symmetric cipher algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a symmetric cipher algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 821 of file `crypto_values.h`.

6.24.2.77 PSA_ALG_IS_DETERMINISTIC_DSA

```
#define PSA_ALG_IS_DETERMINISTIC_DSA(  
    alg ) (PSA_ALG_IS_DSA(alg) && PSA_ALG_DSA_IS_DETERMINISTIC(alg))
```

Definition at line 235 of file `crypto_extra.h`.

6.24.2.78 PSA_ALG_IS_DETERMINISTIC_ECDSA

```
#define PSA_ALG_IS_DETERMINISTIC_ECDSA(  
    alg ) (PSA_ALG_IS_ECDSA(alg) && PSA_ALG_ECDSA_IS_DETERMINISTIC(alg))
```

Definition at line 1609 of file `crypto_values.h`.

6.24.2.79 PSA_ALG_IS_DSA

```
#define PSA_ALG_IS_DSA(  
    alg )
```

Value:

```
((alg) & ~PSA_ALG_HASH_MASK & ~PSA_ALG_DSA_DETERMINISTIC_FLAG) == \  
PSA_ALG_DSA_BASE)
```

Definition at line 230 of file `crypto_extra.h`.

6.24.2.80 PSA_ALG_IS_ECDH

```
#define PSA_ALG_IS_ECDH(  
    alg ) (PSA_ALG_KEY_AGREEMENT_GET_BASE(alg) == PSA_ALG_ECDH)
```

Whether the specified algorithm is an elliptic curve Diffie-Hellman algorithm.

This includes the raw elliptic curve Diffie-Hellman algorithm as well as elliptic curve Diffie-Hellman followed by any supporter key derivation algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is an elliptic curve Diffie-Hellman algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported key agreement algorithm identifier.

Definition at line 2288 of file `crypto_values.h`.

6.24.2.81 PSA_ALG_IS_ECDSA

```
#define PSA_ALG_IS_ECDSA(  
    alg )
```

Value:

```
((alg) & ~PSA_ALG_HASH_MASK & ~PSA_ALG_ECDSA_DETERMINISTIC_FLAG) == \  
PSA_ALG_ECDSA_BASE)
```

Definition at line 1604 of file `crypto_values.h`.

6.24.2.82 PSA_ALG_IS_FFDH

```
#define PSA_ALG_IS_FFDH(  
    alg ) (PSA_ALG_KEY_AGREEMENT_GET_BASE(alg) == PSA_ALG_FFDH)
```

Whether the specified algorithm is a finite field Diffie-Hellman algorithm.

This includes the raw finite field Diffie-Hellman algorithm as well as finite-field Diffie-Hellman followed by any supporter key derivation algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a finite field Diffie-Hellman algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported key agreement algorithm identifier.

Definition at line 2244 of file `crypto_values.h`.

6.24.2.83 PSA_ALG_IS_HASH

```
#define PSA_ALG_IS_HASH(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_HASH)
```

Whether the specified algorithm is a hash algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a hash algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 799 of file `crypto_values.h`.

6.24.2.84 PSA_ALG_IS_HASH_AND_SIGN

```
#define PSA_ALG_IS_HASH_AND_SIGN(  
    alg )
```

Value:

```
(PSA_ALG_IS_SIGN_HASH(alg) &&  
 (alg) & PSA_ALG_HASH_MASK) != 0) \
```

Whether the specified algorithm is a hash-and-sign algorithm.

Hash-and-sign algorithms are asymmetric (public-key) signature algorithms structured in two parts: first the calculation of a hash in a way that does not depend on the key, then the calculation of a signature from the hash value and the key. Hash-and-sign algorithms encode the hash used for the hashing step, and you can call [PSA_ALG_SIGN_GET_HASH](#) to extract this algorithm.

Thus, for a hash-and-sign algorithm, [psa_sign_message\(key, alg, input, ...\)](#) is equivalent to [psa_hash_compute\(PSA_ALG_SIGN_GET_HASH\(alg\), input, ..., hash, ...\)](#); [psa_sign_hash\(key, alg, hash, ..., signature, ...\)](#);

Most usefully, separating the hash from the signature allows the hash to be calculated in multiple steps with [psa_hash_setup\(\)](#), [psa_hash_update\(\)](#) and [psa_hash_finish\(\)](#). Likewise [psa_verify_message\(\)](#) is equivalent to calculating the hash and then calling [psa_verify_hash\(\)](#).

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a hash-and-sign algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 1764 of file `crypto_values.h`.

6.24.2.85 PSA_ALG_IS_HASH_EDDSA

```
#define PSA_ALG_IS_HASH_EDDSA(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HASH_EDDSA_BASE)
```

Definition at line 1645 of file `crypto_values.h`.

6.24.2.86 PSA_ALG_IS_HKDF

```
#define PSA_ALG_IS_HKDF(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_BASE)
```

Whether the specified algorithm is an HKDF algorithm.

HKDF is a family of key derivation algorithms that are based on a hash function and the HMAC construction.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if *alg* is an HKDF algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported key derivation algorithm identifier.

Definition at line 1879 of file `crypto_values.h`.

6.24.2.87 PSA_ALG_IS_HKDF_EXPAND

```
#define PSA_ALG_IS_HKDF_EXPAND(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_EXPAND_BASE)
```

Whether the specified algorithm is an HKDF-Expand algorithm.

HKDF-Expand is a family of key derivation algorithms that are based on a hash function and the HMAC construction.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is an HKDF-Expand algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

Definition at line 1972 of file `crypto_values.h`.

6.24.2.88 PSA_ALG_IS_HKDF_EXTRACT

```
#define PSA_ALG_IS_HKDF_EXTRACT(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_EXTRACT_BASE)
```

Whether the specified algorithm is an HKDF-Extract algorithm.

HKDF-Extract is a family of key derivation algorithms that are based on a hash function and the HMAC construction.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is an HKDF-Extract algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

Definition at line 1930 of file `crypto_values.h`.

6.24.2.89 PSA_ALG_IS_HMAC

```
#define PSA_ALG_IS_HMAC(  
    alg )
```

Value:

```
((alg) & (PSA_ALG_CATEGORY_MASK | PSA_ALG_MAC_SUBCATEGORY_MASK)) == \  
PSA_ALG_HMAC_BASE)
```

Whether the specified algorithm is an HMAC algorithm.

HMAC is a family of MAC algorithms that are based on a hash function.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is an HMAC algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 1046 of file `crypto_values.h`.

6.24.2.90 PSA_ALG_IS_JPAKE

```
#define PSA_ALG_IS_JPAKE(  
    alg ) (((alg) & (~PSA_ALG_HASH_MASK)) == PSA_ALG_JPAKE_BASE)
```

Whether the specified algorithm is a JPAKE algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is of the form `PSA_ALG_JPAKE(hash_alg)` for some hash algorithm `hash_alg`, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 748 of file `crypto_extra.h`.

6.24.2.91 PSA_ALG_IS_KEY_AGREEMENT

```
#define PSA_ALG_IS_KEY_AGREEMENT(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_KEY_AGREEMENT)
```

Whether the specified algorithm is a key agreement algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is a key agreement algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 868 of file `crypto_values.h`.

6.24.2.92 PSA_ALG_IS_KEY_DERIVATION

```
#define PSA_ALG_IS_KEY_DERIVATION(  
    alg ) ((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_KEY_DERIVATION)
```

Whether the specified algorithm is a key derivation algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a key derivation algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 879 of file `crypto_values.h`.

6.24.2.93 PSA_ALG_IS_KEY_DERIVATION_OR_AGREEMENT

```
#define PSA_ALG_IS_KEY_DERIVATION_OR_AGREEMENT(  
    alg ) ((PSA_ALG_IS_KEY_DERIVATION(alg) || PSA_ALG_IS_KEY_AGREEMENT(alg)))
```

Definition at line 2220 of file `crypto_values.h`.

6.24.2.94 PSA_ALG_IS_KEY_DERIVATION_STRETCHING

```
#define PSA_ALG_IS_KEY_DERIVATION_STRETCHING(  
    alg )
```

Value:

```
(PSA_ALG_IS_KEY_DERIVATION(alg) &&  
 (alg) & PSA_ALG_KEY_DERIVATION_STRETCHING_FLAG) \
```

Whether the specified algorithm is a key stretching / password hashing algorithm.

A key stretching / password hashing algorithm is a key derivation algorithm that is suitable for use with a low-entropy secret such as a password. Equivalently, it's a key derivation algorithm that uses a [PSA_KEY_DERIVATION_INPUT_PASSWORD](#) input step.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if `alg` is a key stretching / password hashing algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 896 of file `crypto_values.h`.

6.24.2.95 PSA_ALG_IS_KEY_WRAP

```
#define PSA_ALG_IS_KEY_WRAP(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_AES_KEY_WRAP)
```

Check whether the specified algorithm is a key-wrapping algorithm.

Parameters

<i>alg</i>	An algorithm identifier; a value of type <code>psa_algorithm_t</code> .
------------	---

Returns

1 if `alg` is a key-wrapping algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 2364 of file `crypto_values.h`.

6.24.2.96 PSA_ALG_IS_MAC

```
#define PSA_ALG_IS_MAC(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_MAC)
```

Whether the specified algorithm is a MAC algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is a MAC algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 810 of file `crypto_values.h`.

6.24.2.97 PSA_ALG_IS_PAKE

```
#define PSA_ALG_IS_PAKE(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_PAKE)
```

Whether the specified algorithm is a password-authenticated key exchange.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a password-authenticated key exchange (PAKE) algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 606 of file `crypto_extra.h`.

6.24.2.98 PSA_ALG_IS_PBKDF2

```
#define PSA_ALG_IS_PBKDF2(  
    kdf_alg )
```

Value:

```
(PSA_ALG_IS_PBKDF2_HMAC(kdf_alg) || \  
 (kdf_alg) == PSA_ALG_PBKDF2_AES_CMACEPRF_128))
```

Definition at line 2172 of file `crypto_values.h`.

6.24.2.99 PSA_ALG_IS_PBKDF2_HMAC

```
#define PSA_ALG_IS_PBKDF2_HMAC(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_PBKDF2_HMAC_BASE)
```

Whether the specified algorithm is a PBKDF2-HMAC algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a PBKDF2-HMAC algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported key derivation algorithm identifier.

Definition at line 2157 of file `crypto_values.h`.

6.24.2.100 PSA_ALG_IS_RANDOMIZED_DSA

```
#define PSA_ALG_IS_RANDOMIZED_DSA(  
    alg ) (PSA_ALG_IS_DSA(alg) && !PSA_ALG_DSA_IS_DETERMINISTIC(alg))
```

Definition at line 237 of file `crypto_extra.h`.

6.24.2.101 PSA_ALG_IS_RANDOMIZED_ECDSA

```
#define PSA_ALG_IS_RANDOMIZED_ECDSA(  
    alg ) (PSA_ALG_IS_ECDSA(alg) && !PSA_ALG_ECDSA_IS_DETERMINISTIC(alg))
```

Definition at line 1611 of file `crypto_values.h`.

6.24.2.102 PSA_ALG_IS_RAW_KEY_AGREEMENT

```
#define PSA_ALG_IS_RAW_KEY_AGREEMENT(  
    alg )
```

Value:

```
(PSA_ALG_IS_KEY_AGREEMENT(alg) &&  
    PSA_ALG_KEY_AGREEMENT_GET_KDF(alg) == PSA_ALG_CATEGORY_KEY_DERIVATION)
```

Whether the specified algorithm is a raw key agreement algorithm.

A raw key agreement algorithm is one that does not specify a key derivation function. Usually, raw key agreement algorithms are constructed directly with a `PSA_ALG_XXX` macro while non-raw key agreement algorithms are constructed with `PSA_ALG_KEY_AGREEMENT()`.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if *alg* is a raw key agreement algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 2216 of file `crypto_values.h`.

6.24.2.103 PSA_ALG_IS_RSA_OAEP

```
#define PSA_ALG_IS_RSA_OAEP(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_OAEP_BASE)
```

Definition at line 1819 of file `crypto_values.h`.

6.24.2.104 PSA_ALG_IS_RSA_PKCS1V15_SIGN

```
#define PSA_ALG_IS_RSA_PKCS1V15_SIGN(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_PKCS1V15_SIGN_BASE)
```

Definition at line 1452 of file `crypto_values.h`.

6.24.2.105 PSA_ALG_IS_RSA_PSS

```
#define PSA_ALG_IS_RSA_PSS(  
    alg )
```

Value:

```
(PSA_ALG_IS_RSA_PSS_STANDARD_SALT(alg) ||  
 PSA_ALG_IS_RSA_PSS_ANY_SALT(alg)) \
```

Whether the specified algorithm is RSA PSS.

This includes any of the RSA PSS algorithm variants, regardless of the constraints on salt length.

Parameters

<i>alg</i>	An algorithm value or an algorithm policy wildcard.
------------	---

Returns

1 if *alg* is of the form `PSA_ALG_RSA_PSS(hash_alg)` or `PSA_ALG_RSA_PSS_ANY_SALT_BASE(hash_↵_alg)`, where *hash_alg* is a hash algorithm or `PSA_ALG_ANY_HASH`. 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier or policy.

Definition at line 1541 of file `crypto_values.h`.

6.24.2.106 PSA_ALG_IS_RSA_PSS_ANY_SALT

```
#define PSA_ALG_IS_RSA_PSS_ANY_SALT(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_PSS_ANY_SALT_BASE)
```

Whether the specified algorithm is RSA PSS with any salt.

Parameters

<i>alg</i>	An algorithm value or an algorithm policy wildcard.
------------	---

Returns

1 if `alg` is of the form `PSA_ALG_RSA_PSS_ANY_SALT_BASE(hash_alg)`, where `hash_alg` is a hash algorithm or `PSA_ALG_ANY_HASH`. 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier or policy.

Definition at line 1523 of file `crypto_values.h`.

6.24.2.107 PSA_ALG_IS_RSA_PSS_STANDARD_SALT

```
#define PSA_ALG_IS_RSA_PSS_STANDARD_SALT(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_PSS_BASE)
```

Whether the specified algorithm is RSA PSS with standard salt.

Parameters

<i>alg</i>	An algorithm value or an algorithm policy wildcard.
------------	---

Returns

1 if `alg` is of the form `PSA_ALG_RSA_PSS(hash_alg)`, where `hash_alg` is a hash algorithm or `PSA_ALG_ANY_HASH`. 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier or policy.

Definition at line 1509 of file `crypto_values.h`.

6.24.2.108 PSA_ALG_IS_SIGN

```
#define PSA_ALG_IS_SIGN(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_SIGN)
```

Whether the specified algorithm is an asymmetric signature algorithm, also known as public-key signature algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is an asymmetric signature algorithm, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 845 of file `crypto_values.h`.

6.24.2.109 PSA_ALG_IS_SIGN_HASH

```
#define PSA_ALG_IS_SIGN_HASH(  
    alg )
```

Value:

```
(PSA_ALG_IS_RSA_PSS(alg) || PSA_ALG_IS_RSA_PKCS1V15_SIGN(alg) ||  
 PSA_ALG_IS_ECDSA(alg) || PSA_ALG_IS_HASH_EDDSA(alg) ||  
 PSA_ALG_IS_VENDOR_HASH_AND_SIGN(alg))
```

Whether the specified algorithm is a signature algorithm that can be used with [psa_sign_hash\(\)](#) and [psa_verify_hash\(\)](#).

This encompasses all strict hash-and-sign algorithms categorized by [PSA_ALG_IS_HASH_AND_SIGN\(\)](#), as well as algorithms that follow the paradigm more loosely:

- [PSA_ALG_RSA_PKCS1V15_SIGN_RAW](#) (expects its input to be an encoded hash)
- [PSA_ALG_ECDSA_ANY](#) (doesn't specify what kind of hash the input is)

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a signature algorithm that can be used to sign a hash. 0 if *alg* is a signature algorithm that can only be used to sign a message. 0 if *alg* is not a signature algorithm. This macro can return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 1719 of file `crypto_values.h`.

6.24.2.110 PSA_ALG_IS_SIGN_MESSAGE

```
#define PSA_ALG_IS_SIGN_MESSAGE(  
    alg ) (PSA_ALG_IS_SIGN_HASH(alg) || (alg) == PSA_ALG_PURE_EDDSA)
```

Whether the specified algorithm is a signature algorithm that can be used with [psa_sign_message\(\)](#) and [psa_verify_message\(\)](#).

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a signature algorithm that can be used to sign a message. 0 if *alg* is a signature algorithm that can only be used to sign an already-calculated hash. 0 if *alg* is not a signature algorithm. This macro can return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 1735 of file `crypto_values.h`.

6.24.2.111 `PSA_ALG_IS_SP800_108_COUNTER_CMAC`

```
#define PSA_ALG_IS_SP800_108_COUNTER_CMAC(  
    alg ) ((alg) == PSA_ALG_SP800_108_COUNTER_CMAC)
```

Whether the specified algorithm is NIST SP800-108 Counter CMAC.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is NIST SP800 -108 Counter CMAC. This macro may return either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

Definition at line 1836 of file `crypto_values.h`.

6.24.2.112 `PSA_ALG_IS_SPAKE2P`

```
#define PSA_ALG_IS_SPAKE2P(  
    alg )
```

Value:

```
(PSA_ALG_IS_SPAKE2P_HMAC(alg) || \  
 PSA_ALG_IS_SPAKE2P_CMACE(alg) || \  
 (alg) == PSA_ALG_SPAKE2P_MATTER)
```

Whether the specified algorithm is any SPAKE2+ algorithm variant.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if `alg` is of the form `PSA_ALG_SPAKE2P_CMACE(hash_alg)`, `PSA_ALG_SPAKE2P_HMAC(hash_↵alg)` or `PSA_ALG_SPAKE2P_MATTER` for some hash algorithm `hash_alg`, 0 otherwise. This macro may return either 0 or 1 if `alg` is not a supported algorithm identifier.

Definition at line 820 of file `crypto_extra.h`.

6.24.2.113 PSA_ALG_IS_SPAKE2P_CMAC

```
#define PSA_ALG_IS_SPAKE2P_CMAC(  
    alg ) (((alg) & (~(PSA\_ALG\_HASH\_MASK))) == PSA\_ALG\_SPAKE2P\_CMAC\_BASE)
```

Definition at line 801 of file `crypto_extra.h`.

6.24.2.114 PSA_ALG_IS_SPAKE2P_HMAC

```
#define PSA_ALG_IS_SPAKE2P_HMAC(  
    alg ) (((alg) & (~(PSA\_ALG\_HASH\_MASK))) == PSA\_ALG\_SPAKE2P\_HMAC\_BASE)
```

Definition at line 791 of file `crypto_extra.h`.

6.24.2.115 PSA_ALG_IS_STREAM_CIPHER

```
#define PSA_ALG_IS_STREAM_CIPHER(  
    alg )
```

Value:

```
((alg) & (PSA\_ALG\_CATEGORY\_MASK | PSA\_ALG\_CIPHER\_STREAM\_FLAG)) == \  
(PSA\_ALG\_CATEGORY\_CIPHER | PSA\_ALG\_CIPHER\_STREAM\_FLAG)
```

Whether the specified algorithm is a stream cipher.

A stream cipher is a symmetric cipher that encrypts or decrypts messages by applying a bitwise-xor with a stream of bytes that is generated from a key.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a stream cipher algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier or if it is not a symmetric cipher algorithm.

Definition at line 1201 of file `crypto_values.h`.

6.24.2.116 PSA_ALG_IS_TLS12_PRF

```
#define PSA_ALG_IS_TLS12_PRF(  
    alg ) (((alg) & ~PSA\_ALG\_HASH\_MASK) == PSA\_ALG\_TLS12\_PRF\_BASE)
```

Whether the specified algorithm is a TLS-1.2 PRF algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a TLS-1.2 PRF algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported key derivation algorithm identifier.

Definition at line 2028 of file `crypto_values.h`.

6.24.2.117 PSA_ALG_IS_TLS12_PSK_TO_MS

```
#define PSA_ALG_IS_TLS12_PSK_TO_MS(  
    alg ) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_TLS12_PSK_TO_MS_BASE)
```

Whether the specified algorithm is a TLS-1.2 PSK to MS algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type psa_algorithm_t).
------------	---

Returns

1 if *alg* is a TLS-1.2 PSK to MS algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported key derivation algorithm identifier.

Definition at line 2091 of file `crypto_values.h`.

6.24.2.118 PSA_ALG_IS_VENDOR_DEFINED

```
#define PSA_ALG_IS_VENDOR_DEFINED(  
    alg ) (((alg) & PSA_ALG_VENDOR_FLAG) != 0)
```

Whether an algorithm is vendor-defined.

See also [PSA_ALG_VENDOR_FLAG](#).

Definition at line 788 of file `crypto_values.h`.

6.24.2.119 PSA_ALG_IS_VENDOR_HASH_AND_SIGN [1/2]

```
#define PSA_ALG_IS_VENDOR_HASH_AND_SIGN(
    alg ) PSA_ALG_IS_DSA(alg)
```

Definition at line 261 of file `crypto_extra.h`.

6.24.2.120 PSA_ALG_IS_VENDOR_HASH_AND_SIGN [2/2]

```
#define PSA_ALG_IS_VENDOR_HASH_AND_SIGN(
    alg ) 0
```

Definition at line 1700 of file `crypto_values.h`.

6.24.2.121 PSA_ALG_IS_WILDCARD

```
#define PSA_ALG_IS_WILDCARD(
    alg )
```

Value:

```
(PSA_ALG_IS_HASH_AND_SIGN(alg) ?
 PSA_ALG_SIGN_GET_HASH(alg) == PSA_ALG_ANY_HASH :
 PSA_ALG_IS_MAC(alg) ?
 (alg & PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG) != 0 :
 PSA_ALG_IS_AEAD(alg) ?
 (alg & PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG) != 0 :
 (alg) == PSA_ALG_ANY_HASH)
```

Whether the specified algorithm encoding is a wildcard.

Wildcard values may only be used to set the usage algorithm field in a policy, not to perform an operation.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if *alg* is a wildcard algorithm encoding.

0 if *alg* is a non-wildcard algorithm encoding (suitable for an operation).

This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 2304 of file `crypto_values.h`.

6.24.2.122 PSA_ALG_IS_XOF

```
#define PSA_ALG_IS_XOF(  
    alg ) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_XOF)
```

Whether the specified algorithm is a XOF (extendable-output function) algorithm.

Parameters

<i>alg</i>	An algorithm identifier (value of type <code>psa_algorithm_t</code>).
------------	--

Returns

1 if *alg* is a XOF algorithm, 0 otherwise. This macro may return either 0 or 1 if *alg* is not a supported algorithm identifier.

Definition at line 909 of file `crypto_values.h`.

6.24.2.123 PSA_ALG_JPAKE

```
#define PSA_ALG_JPAKE(  
    hash_alg ) (PSA_ALG_JPAKE_BASE | ((hash_alg) & (PSA_ALG_HASH_MASK)))
```

The Password-authenticated key exchange by juggling (J-PAKE) algorithm.

This is J-PAKE as defined by RFC 8236, instantiated with the following parameters:

- The group can be either an elliptic curve or defined over a finite field.
- Schnorr NIZK proof as defined by RFC 8235 and using the same group as the J-PAKE algorithm.
- A cryptographic hash function.

To select these parameters and set up the cipher suite, call these functions in any order:

```
psa_pake_cs_set_algorithm(cipher_suite, PSA_ALG_JPAKE);  
psa_pake_cs_set_primitive(cipher_suite,  
    PSA_PAKE_PRIMITIVE(type, family, bits));
```

For more information on how to set a specific curve or field, refer to the documentation of the individual `PSA_PAKE_PRIMITIVE_TYPE_XXX` constants.

After initializing a J-PAKE operation, call

```
psa_pake_setup(operation, cipher_suite);  
psa_pake_set_user(operation, ...);  
psa_pake_set_peer(operation, ...);
```

The password is provided as a key. This can be the password text itself, in an agreed character encoding, or some value derived from the password as required by a higher level protocol.

(The implementation converts the key material to a number as described in Section 2.3.8 of *SEC 1: Elliptic Curve Cryptography* (<https://www.secg.org/sec1-v2.pdf>), before reducing it modulo q . Here q is order of the group defined by the primitive set in the cipher suite. The `psa_pake_setup()` function returns an error if the result of the reduction is 0.)

The key exchange flow for J-PAKE is as follows:

1. To get the first round data that needs to be sent to the peer, call

```
// Get g1
psa_pake_output(operation, #PSA_PAKE_STEP_KEY_SHARE, ...);
// Get the ZKP public key for x1
psa_pake_output(operation, #PSA_PAKE_STEP_ZK_PUBLIC, ...);
// Get the ZKP proof for x1
psa_pake_output(operation, #PSA_PAKE_STEP_ZK_PROOF, ...);
// Get g2
psa_pake_output(operation, #PSA_PAKE_STEP_KEY_SHARE, ...);
// Get the ZKP public key for x2
psa_pake_output(operation, #PSA_PAKE_STEP_ZK_PUBLIC, ...);
// Get the ZKP proof for x2
psa_pake_output(operation, #PSA_PAKE_STEP_ZK_PROOF, ...);
```

2. To provide the first round data received from the peer to the operation, call

```
// Set g3
psa_pake_input(operation, #PSA_PAKE_STEP_KEY_SHARE, ...);
// Set the ZKP public key for x3
psa_pake_input(operation, #PSA_PAKE_STEP_ZK_PUBLIC, ...);
// Set the ZKP proof for x3
psa_pake_input(operation, #PSA_PAKE_STEP_ZK_PROOF, ...);
// Set g4
psa_pake_input(operation, #PSA_PAKE_STEP_KEY_SHARE, ...);
// Set the ZKP public key for x4
psa_pake_input(operation, #PSA_PAKE_STEP_ZK_PUBLIC, ...);
// Set the ZKP proof for x4
psa_pake_input(operation, #PSA_PAKE_STEP_ZK_PROOF, ...);
```

3. To get the second round data that needs to be sent to the peer, call

```
// Get A
psa_pake_output(operation, #PSA_PAKE_STEP_KEY_SHARE, ...);
// Get ZKP public key for x2*s
psa_pake_output(operation, #PSA_PAKE_STEP_ZK_PUBLIC, ...);
// Get ZKP proof for x2*s
psa_pake_output(operation, #PSA_PAKE_STEP_ZK_PROOF, ...);
```

4. To provide the second round data received from the peer to the operation, call

```
// Set B
psa_pake_input(operation, #PSA_PAKE_STEP_KEY_SHARE, ...);
// Set ZKP public key for x4*s
psa_pake_input(operation, #PSA_PAKE_STEP_ZK_PUBLIC, ...);
// Set ZKP proof for x4*s
psa_pake_input(operation, #PSA_PAKE_STEP_ZK_PROOF, ...);
```

5. To access the shared secret call

```
// Get Ka=Kb=K
psa_pake_get_shared_key()
```

For more information consult the documentation of the individual `PSA_PAKE_STEP_XXX` constants.

At this point there is a cryptographic guarantee that only the authenticated party who used the same password is able to compute the key. But there is no guarantee that the peer is the party it claims to be and was able to do so.

That is, the authentication is only implicit (the peer is not authenticated at this point, and no action should be taken that assume that they are - like for example accessing restricted files).

To make the authentication explicit there are various methods, see Section 5 of RFC 8236 for two examples.

Note

As of TF-PSA-Crypto 1.0.0, the JPAKE implementation has the following limitations:

- The only supported primitive is ECC on the curve `secp256r1`, i.e. `PSA_PAKE_PRIMITIVE(PSA_PAKE_PRIMITIVE_TYPE_ECC, PSA_ECC_FAMILY_SECP_R1, 256)`.
- The only supported hash algorithm is SHA-256, i.e. `PSA_ALG_SHA_256`.
- When using the built-in implementation, the user ID and the peer ID must be `"client"` (6-byte string) and `"server"` (6-byte string), or the other way round. Third-party drivers may or may not have this limitation.

Definition at line 736 of file `crypto_extra.h`.

6.24.2.124 PSA_ALG_JPAKE_BASE

```
#define PSA_ALG_JPAKE_BASE ((psa_algorithm_t) 0x0a000100)
```

Definition at line 609 of file `crypto_extra.h`.

6.24.2.125 PSA_ALG_KEY_AGREEMENT

```
#define PSA_ALG_KEY_AGREEMENT(  
    ka_alg,  
    kdf_alg ) ((ka_alg) | (kdf_alg))
```

Macro to build a combined algorithm that chains a key agreement with a key derivation.

Parameters

<i>ka_alg</i>	A key agreement algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_KEY_AGREEMENT (ka_alg) is true).
<i>kdf_alg</i>	A key derivation algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_KEY_DERIVATION (kdf_alg) is true).

Returns

The corresponding key agreement and derivation algorithm.

Unspecified if *ka_alg* is not a supported key agreement algorithm or *kdf_alg* is not a supported key derivation algorithm.

Definition at line 2193 of file `crypto_values.h`.

6.24.2.126 PSA_ALG_KEY_AGREEMENT_GET_BASE

```
#define PSA_ALG_KEY_AGREEMENT_GET_BASE(  
    alg ) (((alg) & PSA_ALG_KEY_AGREEMENT_MASK) | PSA_ALG_CATEGORY_KEY_AGREEMENT)
```

Definition at line 2199 of file `crypto_values.h`.

6.24.2.127 PSA_ALG_KEY_AGREEMENT_GET_KDF

```
#define PSA_ALG_KEY_AGREEMENT_GET_KDF(  
    alg ) (((alg) & PSA_ALG_KEY_DERIVATION_MASK) | PSA_ALG_CATEGORY_KEY_DERIVATION)
```

Definition at line 2196 of file `crypto_values.h`.

6.24.2.128 PSA_ALG_KEY_AGREEMENT_MASK

```
#define PSA_ALG_KEY_AGREEMENT_MASK ((psa_algorithm_t) 0xffff0000)
```

Definition at line 2177 of file `crypto_values.h`.

6.24.2.129 PSA_ALG_KEY_DERIVATION_MASK

```
#define PSA_ALG_KEY_DERIVATION_MASK ((psa_algorithm_t) 0xfe00ffff)
```

Definition at line 2176 of file `crypto_values.h`.

6.24.2.130 PSA_ALG_KEY_DERIVATION_STRETCHING_FLAG

```
#define PSA_ALG_KEY_DERIVATION_STRETCHING_FLAG ((psa_algorithm_t) 0x00800000)
```

Definition at line 2117 of file `crypto_values.h`.

6.24.2.131 PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG

```
#define PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG ((psa_algorithm_t) 0x00008000)
```

Definition at line 1066 of file `crypto_values.h`.

6.24.2.132 PSA_ALG_MAC_SUBCATEGORY_MASK

```
#define PSA_ALG_MAC_SUBCATEGORY_MASK ((psa_algorithm_t) 0x00c00000)
```

Definition at line 1017 of file `crypto_values.h`.

6.24.2.133 PSA_ALG_MAC_TRUNCATION_MASK

```
#define PSA_ALG_MAC_TRUNCATION_MASK ((psa_algorithm_t) 0x003f0000)
```

Definition at line 1057 of file `crypto_values.h`.

6.24.2.134 PSA_ALG_MD5

```
#define PSA_ALG_MD5 ((psa_algorithm_t) 0x02000003)
```

MD5

Definition at line 919 of file `crypto_values.h`.

6.24.2.135 PSA_ALG_NONE

```
#define PSA_ALG_NONE ((psa_algorithm_t) 0)
```

An invalid algorithm identifier value.

Definition at line 914 of file `crypto_values.h`.

6.24.2.136 PSA_ALG_OFB

```
#define PSA_ALG_OFB ((psa_algorithm_t) 0x04c01200)
```

The OFB stream cipher mode.

The underlying block cipher is determined by the key type.

Definition at line 1231 of file `crypto_values.h`.

6.24.2.137 PSA_ALG_PBKDF2_AES_CMAC_PRF_128

```
#define PSA_ALG_PBKDF2_AES_CMAC_PRF_128 ((psa_algorithm_t) 0x08800200)
```

The PBKDF2-AES-CMAC-PRF-128 password hashing / key stretching algorithm.

PBKDF2 is defined by PKCS#5, republished as RFC 8018 (section 5.2). This macro specifies the PBKDF2 algorithm constructed using the AES-CMAC-PRF-128 PRF specified by RFC 4615.

This key derivation algorithm uses the same inputs as [PSA_ALG_PBKDF2_HMAC\(\)](#) with the same constraints.

Definition at line 2170 of file `crypto_values.h`.

6.24.2.138 PSA_ALG_PBKDF2_HMAC

```
#define PSA_ALG_PBKDF2_HMAC(  
    hash_alg ) (PSA_ALG_PBKDF2_HMAC_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

Macro to build a PBKDF2-HMAC password hashing / key stretching algorithm.

PBKDF2 is defined by PKCS#5, republished as RFC 8018 (section 5.2). This macro specifies the PBKDF2 algorithm constructed using a PRF based on HMAC with the specified hash. For example, [PSA_ALG_PBKDF2_HMAC\(PSA_ALG_SHA_256\)](#) specifies PBKDF2 using the PRF HMAC-SHA-256.

This key derivation algorithm uses the following inputs, which must be provided in the following order:

- [PSA_KEY_DERIVATION_INPUT_COST](#) is the iteration count. This input step must be used exactly once.
- [PSA_KEY_DERIVATION_INPUT_SALT](#) is the salt. This input step must be used one or more times; if used several times, the inputs will be concatenated. This can be used to build the final salt from multiple sources, both public and secret (also known as pepper).
- [PSA_KEY_DERIVATION_INPUT_PASSWORD](#) is the password to be hashed. This input step must be used exactly once.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (<i>hash_alg</i>) is true).
-----------------	---

Returns

The corresponding PBKDF2-HMAC-XXX algorithm.

Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 2146 of file `crypto_values.h`.

6.24.2.139 PSA_ALG_PBKDF2_HMAC_BASE

```
#define PSA_ALG_PBKDF2_HMAC_BASE ((psa_algorithm_t) 0x08800100)
```

Definition at line 2119 of file `crypto_values.h`.

6.24.2.140 PSA_ALG_PBKDF2_HMAC_GET_HASH

```
#define PSA_ALG_PBKDF2_HMAC_GET_HASH(  
    pbkdf2_alg ) (PSA_ALG_CATEGORY_HASH | ((pbkdf2_alg) & PSA_ALG_HASH_MASK))
```

Definition at line 2159 of file `crypto_values.h`.

6.24.2.141 PSA_ALG_PURE_EDDSA

```
#define PSA_ALG_PURE_EDDSA ((psa_algorithm_t) 0x06000800)
```

Edwards-curve digital signature algorithm without prehashing (PureEdDSA), using standard parameters.

Contexts are not supported in the current version of this specification because there is no suitable signature interface that can take the context as a parameter. A future version of this specification may add suitable functions and extend this algorithm to support contexts.

PureEdDSA requires an elliptic curve key on a twisted Edwards curve. In this specification, the following curves are supported:

- [PSA_ECC_FAMILY_TWISTED_EDWARDS](#), 255-bit: Ed25519 as specified in RFC 8032. The curve is Edwards25519. The hash function used internally is SHA-512.
- [PSA_ECC_FAMILY_TWISTED_EDWARDS](#), 448-bit: Ed448 as specified in RFC 8032. The curve is Edwards448. The hash function used internally is the first 114 bytes of the SHAKE256 output.

This algorithm can be used with [psa_sign_message\(\)](#) and [psa_verify_message\(\)](#). Since there is no prehashing, it cannot be used with [psa_sign_hash\(\)](#) or [psa_verify_hash\(\)](#).

The signature format is the concatenation of R and S as defined by RFC 8032 §5.1.6 and §5.2.6 (a 64-byte string for Ed25519, a 114-byte string for Ed448).

Definition at line 1642 of file `crypto_values.h`.

6.24.2.142 PSA_ALG_RIPEMD160

```
#define PSA_ALG_RIPEMD160 ((psa_algorithm_t) 0x02000004)
```

PSA_ALG_RIPEMD160

Definition at line 921 of file crypto_values.h.

6.24.2.143 PSA_ALG_RSA_OAEP

```
#define PSA_ALG_RSA_OAEP(  
    hash_alg ) (PSA_ALG_RSA_OAEP_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

RSA OAEP encryption.

This is the encryption scheme defined by RFC 8017 (PKCS#1: RSA Cryptography Specifications) under the name RSAES-OAEP, with the message generation function MGF1.

Parameters

<i>hash_alg</i>	The hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH(hash_alg) is true) to use for MGF1.
-----------------	---

Returns

The corresponding RSA OAEP encryption algorithm.

Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 1817 of file crypto_values.h.

6.24.2.144 PSA_ALG_RSA_OAEP_BASE

```
#define PSA_ALG_RSA_OAEP_BASE ((psa_algorithm_t) 0x07000300)
```

Definition at line 1802 of file crypto_values.h.

6.24.2.145 PSA_ALG_RSA_OAEP_GET_HASH

```
#define PSA_ALG_RSA_OAEP_GET_HASH(  
    alg )
```

Value:

```
(PSA_ALG_IS_RSA_OAEP(alg) ?  
    ((alg) & PSA_ALG_HASH_MASK) | PSA_ALG_CATEGORY_HASH :  
    0)
```

Definition at line 1821 of file crypto_values.h.

6.24.2.146 PSA_ALG_RSA_PKCS1V15_CRYPT

```
#define PSA_ALG_RSA_PKCS1V15_CRYPT ((psa_algorithm_t) 0x07000200)
```

RSA PKCS#1 v1.5 encryption.

Warning

Calling [psa_asymmetric_decrypt\(\)](#) with this algorithm as a parameter is considered an inherently dangerous function (CWE-242). Unless it is used in a side channel free and safe way (eg. implementing the TLS protocol as per 7.4.7.1 of RFC 5246), the calling code is vulnerable.

Definition at line 1800 of file `crypto_values.h`.

6.24.2.147 PSA_ALG_RSA_PKCS1V15_SIGN

```
#define PSA_ALG_RSA_PKCS1V15_SIGN(  
    hash_alg ) (PSA_ALG_RSA_PKCS1V15_SIGN_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

RSA PKCS#1 v1.5 signature with hashing.

This is the signature scheme defined by RFC 8017 (PKCS#1: RSA Cryptography Specifications) under the name RSASSA-PKCS1-v1_5.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (hash_alg) is true). This includes PSA_ALG_ANY_HASH when specifying the algorithm in a usage policy.
-----------------	--

Returns

The corresponding RSA PKCS#1 v1.5 signature algorithm.
Unspecified if `hash_alg` is not a supported hash algorithm.

Definition at line 1443 of file `crypto_values.h`.

6.24.2.148 PSA_ALG_RSA_PKCS1V15_SIGN_BASE

```
#define PSA_ALG_RSA_PKCS1V15_SIGN_BASE ((psa_algorithm_t) 0x06000200)
```

Definition at line 1427 of file `crypto_values.h`.

6.24.2.149 PSA_ALG_RSA_PKCS1V15_SIGN_RAW

```
#define PSA_ALG_RSA_PKCS1V15_SIGN_RAW PSA_ALG_RSA_PKCS1V15_SIGN_BASE
```

Raw PKCS#1 v1.5 signature.

The input to this algorithm is the DigestInfo structure used by RFC 8017 (PKCS#1: RSA Cryptography Specifications), §9.2 steps 3–6.

Definition at line 1451 of file crypto_values.h.

6.24.2.150 PSA_ALG_RSA_PSS

```
#define PSA_ALG_RSA_PSS(  
    hash_alg ) (PSA_ALG_RSA_PSS_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

RSA PSS signature with hashing.

This is the signature scheme defined by RFC 8017 (PKCS#1: RSA Cryptography Specifications) under the name RSASSA-PSS, with the message generation function MGF1, and with a salt length equal to the length of the hash, or the largest possible salt length for the algorithm and key size if that is smaller than the hash length. The specified hash algorithm is used to hash the input message, to create the salted hash, and for the mask generation.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (hash_alg) is true). This includes PSA_ALG_ANY_HASH when specifying the algorithm in a usage policy.
-----------------	--

Returns

The corresponding RSA PSS signature algorithm.

Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 1477 of file crypto_values.h.

6.24.2.151 PSA_ALG_RSA_PSS_ANY_SALT

```
#define PSA_ALG_RSA_PSS_ANY_SALT(  
    hash_alg ) (PSA_ALG_RSA_PSS_ANY_SALT_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

RSA PSS signature with hashing with relaxed verification.

This algorithm has the same behavior as [PSA_ALG_RSA_PSS](#) when signing, but allows an arbitrary salt length (including 0) when verifying a signature.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (<i>hash_alg</i>) is true). This includes PSA_ALG_ANY_HASH when specifying the algorithm in a usage policy.
-----------------	---

Returns

The corresponding RSA PSS signature algorithm.
 Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 1495 of file `crypto_values.h`.

6.24.2.152 PSA_ALG_RSA_PSS_ANY_SALT_BASE

```
#define PSA_ALG_RSA_PSS_ANY_SALT_BASE ((psa_algorithm_t) 0x06001300)
```

Definition at line 1456 of file `crypto_values.h`.

6.24.2.153 PSA_ALG_RSA_PSS_BASE

```
#define PSA_ALG_RSA_PSS_BASE ((psa_algorithm_t) 0x06000300)
```

Definition at line 1455 of file `crypto_values.h`.

6.24.2.154 PSA_ALG_SHA3_224

```
#define PSA_ALG_SHA3_224 ((psa_algorithm_t) 0x02000010)
```

SHA3-224

Definition at line 937 of file `crypto_values.h`.

6.24.2.155 PSA_ALG_SHA3_256

```
#define PSA_ALG_SHA3_256 ((psa_algorithm_t) 0x02000011)
```

SHA3-256

Definition at line 939 of file `crypto_values.h`.

6.24.2.156 PSA_ALG_SHA3_384

```
#define PSA_ALG_SHA3_384 ((psa_algorithm_t) 0x02000012)
```

SHA3-384

Definition at line 941 of file crypto_values.h.

6.24.2.157 PSA_ALG_SHA3_512

```
#define PSA_ALG_SHA3_512 ((psa_algorithm_t) 0x02000013)
```

SHA3-512

Definition at line 943 of file crypto_values.h.

6.24.2.158 PSA_ALG_SHA_1

```
#define PSA_ALG_SHA_1 ((psa_algorithm_t) 0x02000005)
```

SHA1

Definition at line 923 of file crypto_values.h.

6.24.2.159 PSA_ALG_SHA_224

```
#define PSA_ALG_SHA_224 ((psa_algorithm_t) 0x02000008)
```

SHA2-224

Definition at line 925 of file crypto_values.h.

6.24.2.160 PSA_ALG_SHA_256

```
#define PSA_ALG_SHA_256 ((psa_algorithm_t) 0x02000009)
```

SHA2-256

Definition at line 927 of file crypto_values.h.

6.24.2.161 PSA_ALG_SHA_384

```
#define PSA_ALG_SHA_384 ((psa_algorithm_t) 0x0200000a)
```

SHA2-384

Definition at line 929 of file crypto_values.h.

6.24.2.162 PSA_ALG_SHA_512

```
#define PSA_ALG_SHA_512 ((psa_algorithm_t) 0x0200000b)
```

SHA2-512

Definition at line 931 of file crypto_values.h.

6.24.2.163 PSA_ALG_SHA_512_224

```
#define PSA_ALG_SHA_512_224 ((psa_algorithm_t) 0x0200000c)
```

SHA2-512/224

Definition at line 933 of file crypto_values.h.

6.24.2.164 PSA_ALG_SHA_512_256

```
#define PSA_ALG_SHA_512_256 ((psa_algorithm_t) 0x0200000d)
```

SHA2-512/256

Definition at line 935 of file crypto_values.h.

6.24.2.165 PSA_ALG_SHAKE128

```
#define PSA_ALG_SHAKE128 ((psa_algorithm_t) 0x0d000100)
```

The SHAKE128 XOF (extendable-output function) algorithm.

This is the SHAKE128 extendable-output function defined in FIPS 202, based on the Keccak sponge construction.

Definition at line 992 of file crypto_values.h.

6.24.2.166 PSA_ALG_SHAKE256

```
#define PSA_ALG_SHAKE256 ((psa_algorithm_t) 0x0d000200)
```

The SHAKE256 XOF (extendable-output function) algorithm.

This is the SHAKE256 extendable-output function defined in FIPS 202, based on the Keccak sponge construction.

Definition at line 999 of file `crypto_values.h`.

6.24.2.167 PSA_ALG_SHAKE256_512

```
#define PSA_ALG_SHAKE256_512 ((psa_algorithm_t) 0x02000015)
```

The first 512 bits (64 bytes) of the SHAKE256 output.

This is the prehashing for Ed448ph (see [PSA_ALG_ED448PH](#)). For other scenarios where a hash function based on SHA3/SHAKE is desired, SHA3-512 has the same output size and a (theoretically) higher security strength.

Definition at line 950 of file `crypto_values.h`.

6.24.2.168 PSA_ALG_SIGN_GET_HASH

```
#define PSA_ALG_SIGN_GET_HASH(  
    alg )
```

Value:

```
(PSA_ALG_IS_HASH_AND_SIGN(alg) ?  
    ((alg) & PSA_ALG_HASH_MASK) | PSA_ALG_CATEGORY_HASH :  
    0)
```

Get the hash used by a hash-and-sign signature algorithm.

A hash-and-sign algorithm is a signature algorithm which is composed of two phases: first a hashing phase which does not use the key and produces a hash of the input message, then a signing phase which only uses the hash and the key and not the message itself.

Parameters

<i>alg</i>	A signature algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_SIGN (<i>alg</i>) is true).
------------	---

Returns

The underlying hash algorithm if *alg* is a hash-and-sign algorithm.

0 if *alg* is a signature algorithm that does not follow the hash-and-sign structure.

Unspecified if *alg* is not a signature algorithm or if it is not supported by the implementation.

Definition at line 1786 of file `crypto_values.h`.

6.24.2.169 PSA_ALG_SP800_108_COUNTER_CMAC

```
#define PSA_ALG_SP800_108_COUNTER_CMAC ((psa_algorithm_t) 0x08000800)
```

Definition at line 1826 of file `crypto_values.h`.

6.24.2.170 PSA_ALG_SPAKE2P_CMAC

```
#define PSA_ALG_SPAKE2P_CMAC(  
    hash_alg ) (PSA_ALG_SPAKE2P_CMAC_BASE | ((hash_alg) & (PSA_ALG_HASH_MASK)))
```

Definition at line 799 of file `crypto_extra.h`.

6.24.2.171 PSA_ALG_SPAKE2P_CMAC_BASE

```
#define PSA_ALG_SPAKE2P_CMAC_BASE ((psa_algorithm_t) 0x0a000500)
```

SPAKE2+ algorithm using CMAC for key confirmation.

Not implemented yet.

Definition at line 798 of file `crypto_extra.h`.

6.24.2.172 PSA_ALG_SPAKE2P_HMAC

```
#define PSA_ALG_SPAKE2P_HMAC(  
    hash_alg ) (PSA_ALG_SPAKE2P_HMAC_BASE | ((hash_alg) & (PSA_ALG_HASH_MASK)))
```

SPAKE2+ algorithm using HMAC for key confirmation.

Not implemented yet.

Definition at line 789 of file `crypto_extra.h`.

6.24.2.173 PSA_ALG_SPAKE2P_HMAC_BASE

```
#define PSA_ALG_SPAKE2P_HMAC_BASE ((psa_algorithm_t) 0x0a000400)
```

Definition at line 783 of file `crypto_extra.h`.

6.24.2.174 PSA_ALG_SPAKE2P_MATTER

```
#define PSA_ALG_SPAKE2P_MATTER ((psa_algorithm_t) 0x0a000609)
```

SPAKE2+ algorithm variant used by the Matter specification version 1.2.

Not implemented yet.

Definition at line 808 of file `crypto_extra.h`.

6.24.2.175 PSA_ALG_STREAM_CIPHER

```
#define PSA_ALG_STREAM_CIPHER ((psa_algorithm_t) 0x04800100)
```

The stream cipher mode of a stream cipher algorithm.

The underlying stream cipher is determined by the key type.

- To use ChaCha20, use a key type of [PSA_KEY_TYPE_CHACHA20](#).

Definition at line 1210 of file `crypto_values.h`.

6.24.2.176 PSA_ALG_TLS12_ECJPAKE_TO_PMS

```
#define PSA_ALG_TLS12_ECJPAKE_TO_PMS ((psa_algorithm_t) 0x08000609)
```

Definition at line 2108 of file `crypto_values.h`.

6.24.2.177 PSA_ALG_TLS12_PRF

```
#define PSA_ALG_TLS12_PRF(  
    hash_alg ) (PSA_ALG_TLS12_PRF_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
```

Macro to build a TLS-1.2 PRF algorithm.

TLS 1.2 uses a custom pseudorandom function (PRF) for key schedule, specified in Section 5 of RFC 5246. It is based on HMAC and can be used with either SHA-256 or SHA-384.

This key derivation algorithm uses the following inputs, which must be passed in the order given here:

- [PSA_KEY_DERIVATION_INPUT_SEED](#) is the seed.
- [PSA_KEY_DERIVATION_INPUT_SECRET](#) is the secret key.
- [PSA_KEY_DERIVATION_INPUT_LABEL](#) is the label.

For the application to TLS-1.2 key expansion, the seed is the concatenation of `ServerHello.Random` + `ClientHello.Random`, and the label is "key expansion".

For example, `PSA_ALG_TLS12_PRF(PSA_ALG_SHA_256)` represents the TLS 1.2 PRF using HMAC-SHA-256.

Parameters

<i>hash_alg</i>	A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH (<i>hash_alg</i>) is true).
-----------------	---

Returns

The corresponding TLS-1.2 PRF algorithm.

Unspecified if *hash_alg* is not a supported hash algorithm.

Definition at line 2017 of file `crypto_values.h`.

6.24.2.178 PSA_ALG_TLS12_PRF_BASE

```
#define PSA_ALG_TLS12_PRF_BASE ((psa\_algorithm\_t) 0x08000200)
```

Definition at line 1990 of file `crypto_values.h`.

6.24.2.179 PSA_ALG_TLS12_PRF_GET_HASH

```
#define PSA_ALG_TLS12_PRF_GET_HASH(  
    hkdf_alg) (PSA\_ALG\_CATEGORY\_HASH | ((hkdf_alg) & PSA\_ALG\_HASH\_MASK))
```

Definition at line 2030 of file `crypto_values.h`.

6.24.2.180 PSA_ALG_TLS12_PSK_TO_MS

```
#define PSA_ALG_TLS12_PSK_TO_MS(  
    hash_alg) (PSA\_ALG\_TLS12\_PSK\_TO\_MS\_BASE | ((hash_alg) & PSA\_ALG\_HASH\_MASK))
```

Macro to build a TLS-1.2 PSK-to-MasterSecret algorithm.

In a pure-PSK handshake in TLS 1.2, the master secret is derived from the PreSharedKey (PSK) through the application of padding (RFC 4279, Section 2) and the TLS-1.2 PRF (RFC 5246, Section 5). The latter is based on HMAC and can be used with either SHA-256 or SHA-384.

This key derivation algorithm uses the following inputs, which must be passed in the order given here:

- [PSA_KEY_DERIVATION_INPUT_SEED](#) is the seed.
- [PSA_KEY_DERIVATION_INPUT_OTHER_SECRET](#) is the other secret for the computation of the premaster secret. This input is optional; if omitted, it defaults to a string of null bytes with the same length as the secret (PSK) input.
- [PSA_KEY_DERIVATION_INPUT_SECRET](#) is the secret key.
- [PSA_KEY_DERIVATION_INPUT_LABEL](#) is the label.

For the application to TLS-1.2, the seed (which is forwarded to the TLS-1.2 PRF) is the concatenation of the ClientHello.Random + ServerHello.Random, the label is "master secret" or "extended master secret" and the other secret depends on the key exchange specified in the cipher suite:

- for a plain PSK cipher suite (RFC 4279, Section 2), omit `PSA_KEY_DERIVATION_INPUT_OTHER_SECRET`
- for a ECDHE-PSK cipher suite (RFC 5489, Section 2), the other secret should be the output of the `PSA_ALG_FFDH` or `PSA_ALG_ECDH` key agreement performed with the peer. The recommended way to pass this input is to use a key derivation algorithm constructed as `PSA_ALG_KEY_AGREEMENT(ka_alg, PSA_ALG_TLS12_PSK_TO_MS(PSA_ALG_SHA_256))` and to call `psa_key_derivation_key_agreement()`. Alternatively, this input may be an output of `psa_raw_key_agreement()` passed with `psa_key_derivation_input_bytes()`, or an equivalent input passed with `psa_key_derivation_input_bytes()` or `psa_key_derivation_input_key()`.

For example, `PSA_ALG_TLS12_PSK_TO_MS(PSA_ALG_SHA_256)` represents the TLS-1.2 PSK to Master Secret derivation PRF using HMAC-SHA-256.

Parameters

<i>hash_alg</i>	A hash algorithm (<code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_HASH(hash_alg)</code> is true).
-----------------	--

Returns

The corresponding TLS-1.2 PSK to MS algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

Definition at line 2080 of file `crypto_values.h`.

6.24.2.181 PSA_ALG_TLS12_PSK_TO_MS_BASE

```
#define PSA_ALG_TLS12_PSK_TO_MS_BASE ((psa_algorithm_t) 0x08000300)
```

Definition at line 2033 of file `crypto_values.h`.

6.24.2.182 PSA_ALG_TLS12_PSK_TO_MS_GET_HASH

```
#define PSA_ALG_TLS12_PSK_TO_MS_GET_HASH(  
    hkdf_alg ) (PSA_ALG_CATEGORY_HASH | ((hkdf_alg) & PSA_ALG_HASH_MASK))
```

Definition at line 2093 of file `crypto_values.h`.

6.24.2.183 PSA_ALG_TRUNCATED_MAC

```
#define PSA_ALG_TRUNCATED_MAC(  
    mac_alg,  
    mac_length )
```

Value:

```
((mac_alg) & ~(PSA_ALG_MAC_TRUNCATION_MASK |  
               PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG)) |  
(mac_length) << (PSA_MAC_TRUNCATION_OFFSET & PSA_ALG_MAC_TRUNCATION_MASK)
```

Macro to build a truncated MAC algorithm.

A truncated MAC algorithm is identical to the corresponding MAC algorithm except that the MAC value for the truncated algorithm consists of only the first `mac_length` bytes of the MAC value for the untruncated algorithm.

Note

This macro may allow constructing algorithm identifiers that are not valid, either because the specified length is larger than the untruncated MAC or because the specified length is smaller than permitted by the implementation.

It is implementation-defined whether a truncated MAC that is truncated to the same length as the MAC of the untruncated algorithm is considered identical to the untruncated algorithm for policy comparison purposes.

Parameters

<i>mac_alg</i>	A MAC algorithm identifier (value of type psa_algorithm_t such that PSA_ALG_IS_MAC (<code>mac_alg</code>) is true). This may be a truncated or untruncated MAC algorithm.
<i>mac_length</i>	Desired length of the truncated MAC in bytes. This must be at most the full length of the MAC and must be at least an implementation-specified minimum. The implementation-specified minimum shall not be zero.

Returns

The corresponding MAC algorithm with the specified length.

Unspecified if `mac_alg` is not a supported MAC algorithm or if `mac_length` is too small or too large for the specified MAC algorithm.

Definition at line 1101 of file `crypto_values.h`.

6.24.2.184 PSA_ALG_VENDOR_FLAG

```
#define PSA_ALG_VENDOR_FLAG ((psa_algorithm_t) 0x80000000)
```

Vendor-defined algorithm flag.

Algorithms defined by this standard will never have the [PSA_ALG_VENDOR_FLAG](#) bit set. Vendors who define additional algorithms must use an encoding with the [PSA_ALG_VENDOR_FLAG](#) bit set and should respect the bitwise structure used by standard encodings whenever practical.

Definition at line 770 of file `crypto_values.h`.

6.24.2.185 PSA_ALG_XOF_CONTEXT_FLAG

```
#define PSA_ALG_XOF_CONTEXT_FLAG ((psa_algorithm_t) 0x00008000)
```

Definition at line 1001 of file `crypto_values.h`.

6.24.2.186 PSA_ALG_XOF_HAS_CONTEXT

```
#define PSA_ALG_XOF_HAS_CONTEXT(  
    xof_alg ) ((xof_alg) & PSA_ALG_XOF_CONTEXT_FLAG) != 0)
```

Whether the specified XOF algorithm supports a context.

Parameters

<i>xof_alg</i>	A XOF algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_XOF (<i>xof_alg</i>) is true).
----------------	--

Returns

1 if *xof_alg* supports a context parameter passed with [psa_xof_set_context\(\)](#). This includes XOF algorithms with an optional context. 0 if *xof_alg* does not allow a context parameter. Unspecified if *xof_alg* is not a supported XOF algorithm.

Definition at line 1014 of file `crypto_values.h`.

6.24.2.187 PSA_ALG_XTS

```
#define PSA_ALG_XTS ((psa_algorithm_t) 0x0440ff00)
```

The XTS cipher mode.

XTS is a cipher mode which is built from a block cipher. It requires at least one full block of input, but beyond this minimum the input does not need to be a whole number of blocks.

Definition at line 1239 of file `crypto_values.h`.

6.24.2.188 PSA_BLOCK_CIPHER_BLOCK_LENGTH

```
#define PSA_BLOCK_CIPHER_BLOCK_LENGTH(  
    type )
```

Value:

```
((type) & PSA_KEY_TYPE_CATEGORY_MASK) == PSA_KEY_TYPE_CATEGORY_SYMMETRIC ? \  
1u << PSA_GET_KEY_TYPE_BLOCK_SIZE_EXPONENT(type) : \  
0u)
```

The block size of a block cipher.

Parameters

<i>type</i>	A cipher key type (value of type psa_key_type_t).
-------------	--

Returns

The block size for a block cipher, or 1 for a stream cipher. The return value is undefined if `type` is not a supported cipher key type.

Note

It is possible to build stream cipher algorithms on top of a block cipher, for example CTR mode ([PSA_ALG_CTR](#)). This macro only takes the key type into account, so it cannot be used to determine the size of the data that [psa_cipher_update\(\)](#) might buffer for future processing in general.

This macro returns a compile-time constant if its argument is one.

Warning

This macro may evaluate its argument multiple times.

Definition at line 753 of file `crypto_values.h`.

6.24.2.189 **PSA_DH_FAMILY_RFC7919**

```
#define PSA_DH_FAMILY_RFC7919 ((psa\_dh\_family\_t) 0x03)
```

Diffie-Hellman groups defined in RFC 7919 Appendix A.

This family includes groups with the following key sizes (in bits): 2048, 3072, 4096, 6144, 8192. A given implementation may support all of these sizes or only a subset.

Definition at line 731 of file `crypto_values.h`.

6.24.2.190 **PSA_ECC_FAMILY_BRAINPOOL_P_R1**

```
#define PSA_ECC_FAMILY_BRAINPOOL_P_R1 ((psa\_ecc\_family\_t) 0x30)
```

Brainpool P random curves.

This family comprises the following curves: brainpoolP160r1, brainpoolP192r1, brainpoolP224r1, brainpoolP256r1, brainpoolP320r1, brainpoolP384r1, brainpoolP512r1. It is defined in RFC 5639.

Note

Mbed TLS only supports the 256-bit, 384-bit and 512-bit curves in this family.

Definition at line 656 of file `crypto_values.h`.

6.24.2.191 PSA_ECC_FAMILY_IS_WEIERSTRASS

```
#define PSA_ECC_FAMILY_IS_WEIERSTRASS(  
    family ) ((family & 0xc0) == 0)
```

Check if the curve of given family is Weierstrass elliptic curve.

Definition at line 586 of file `crypto_values.h`.

6.24.2.192 PSA_ECC_FAMILY_MONTGOMERY

```
#define PSA_ECC_FAMILY_MONTGOMERY ((psa_ecc_family_t) 0x41)
```

Curve25519 and Curve448.

This family comprises the following Montgomery curves:

- 255-bit: Bernstein et al., *Curve25519: new Diffie-Hellman speed records*, LNCS 3958, 2006. The algorithm [PSA_ALG_ECDH](#) performs X25519 when used with this curve.
- 448-bit: Hamburg, *Ed448-Goldilocks, a new elliptic curve*, NIST ECC Workshop, 2015. The algorithm [PSA_ALG_ECDH](#) performs X448 when used with this curve.

Definition at line 668 of file `crypto_values.h`.

6.24.2.193 PSA_ECC_FAMILY_SECP_K1

```
#define PSA_ECC_FAMILY_SECP_K1 ((psa_ecc_family_t) 0x17)
```

SEC Koblitz curves over prime fields.

This family comprises the following curves: secp256k1. They are defined in *Standards for Efficient Cryptography, SEC 2: Recommended Elliptic Curve Domain Parameters*. <https://www.secg.org/sec2-v2.pdf>

Definition at line 596 of file `crypto_values.h`.

6.24.2.194 PSA_ECC_FAMILY_SECP_R1

```
#define PSA_ECC_FAMILY_SECP_R1 ((psa_ecc_family_t) 0x12)
```

SEC random curves over prime fields.

This family comprises the following curves: secp256r1, secp384r1, secp521r1. They are defined in *Standards for Efficient Cryptography, SEC 2: Recommended Elliptic Curve Domain Parameters*. <https://www.secg.org/sec2-v2.pdf>

Definition at line 606 of file `crypto_values.h`.

6.24.2.195 PSA_ECC_FAMILY_SECP_R2

```
#define PSA_ECC_FAMILY_SECP_R2 ((psa_ecc_family_t) 0x1b)
```

Definition at line 608 of file `crypto_values.h`.

6.24.2.196 PSA_ECC_FAMILY_SECT_K1

```
#define PSA_ECC_FAMILY_SECT_K1 ((psa_ecc_family_t) 0x27)
```

SEC Koblitz curves over binary fields.

This family comprises the following curves: `sect163k1`, `sect233k1`, `sect239k1`, `sect283k1`, `sect409k1`, `sect571k1`. They are defined in *Standards for Efficient Cryptography, SEC 2: Recommended Elliptic Curve Domain Parameters*.
<https://www.secg.org/sec2-v2.pdf>

Note

Mbed TLS does not support any curve in this family.

Definition at line 620 of file `crypto_values.h`.

6.24.2.197 PSA_ECC_FAMILY_SECT_R1

```
#define PSA_ECC_FAMILY_SECT_R1 ((psa_ecc_family_t) 0x22)
```

SEC random curves over binary fields.

This family comprises the following curves: `sect163r1`, `sect233r1`, `sect283r1`, `sect409r1`, `sect571r1`. They are defined in *Standards for Efficient Cryptography, SEC 2: Recommended Elliptic Curve Domain Parameters*.
<https://www.secg.org/sec2-v2.pdf>

Note

Mbed TLS does not support any curve in this family.

Definition at line 632 of file `crypto_values.h`.

6.24.2.198 PSA_ECC_FAMILY_SECT_R2

```
#define PSA_ECC_FAMILY_SECT_R2 ((psa_ecc_family_t) 0x2b)
```

SEC additional random curves over binary fields.

This family comprises the following curve: sect163r2. It is defined in *Standards for Efficient Cryptography, SEC 2: Recommended Elliptic Curve Domain Parameters*. <https://www.secg.org/sec2-v2.pdf>

Note

Mbed TLS does not support any curve in this family.

Definition at line 644 of file `crypto_values.h`.

6.24.2.199 PSA_ECC_FAMILY_TWISTED_EDWARDS

```
#define PSA_ECC_FAMILY_TWISTED_EDWARDS ((psa_ecc_family_t) 0x42)
```

The twisted Edwards curves Ed25519 and Ed448.

These curves are suitable for EdDSA ([PSA_ALG_PURE_EDDSA](#) for both curves, [PSA_ALG_ED25519PH](#) for the 255-bit curve, [PSA_ALG_ED448PH](#) for the 448-bit curve).

This family comprises the following twisted Edwards curves:

- 255-bit: Edwards25519, the twisted Edwards curve birationally equivalent to Curve25519. Bernstein et al., *Twisted Edwards curves*, Africacrypt 2008.
- 448-bit: Edwards448, the twisted Edwards curve birationally equivalent to Curve448. Hamburg, *Ed448-Goldilocks, a new elliptic curve*, NIST ECC Workshop, 2015.

Note

Mbed TLS does not support Edwards curves yet.

Definition at line 686 of file `crypto_values.h`.

6.24.2.200 PSA_GET_KEY_TYPE_BLOCK_SIZE_EXPONENT

```
#define PSA_GET_KEY_TYPE_BLOCK_SIZE_EXPONENT(  
    type) (((type) >> 8) & 7)
```

Definition at line 733 of file `crypto_values.h`.

6.24.2.201 PSA_KEY_TYPE_AES

```
#define PSA_KEY_TYPE_AES ((psa_key_type_t) 0x2400)
```

Key for a cipher, AEAD or MAC algorithm based on the AES block cipher.

The size of the key can be 16 bytes (AES-128), 24 bytes (AES-192) or 32 bytes (AES-256).

Definition at line 500 of file `crypto_values.h`.

6.24.2.202 PSA_KEY_TYPE_ARIA

```
#define PSA_KEY_TYPE_ARIA ((psa_key_type_t) 0x2406)
```

Key for a cipher, AEAD or MAC algorithm based on the ARIA block cipher.

Definition at line 504 of file `crypto_values.h`.

6.24.2.203 PSA_KEY_TYPE_CAMELLIA

```
#define PSA_KEY_TYPE_CAMELLIA ((psa_key_type_t) 0x2403)
```

Key for a cipher, AEAD or MAC algorithm based on the Camellia block cipher.

Definition at line 508 of file `crypto_values.h`.

6.24.2.204 PSA_KEY_TYPE_CATEGORY_FLAG_PAIR

```
#define PSA_KEY_TYPE_CATEGORY_FLAG_PAIR ((psa_key_type_t) 0x3000)
```

Definition at line 374 of file `crypto_values.h`.

6.24.2.205 PSA_KEY_TYPE_CATEGORY_KEY_PAIR

```
#define PSA_KEY_TYPE_CATEGORY_KEY_PAIR ((psa_key_type_t) 0x7000)
```

Definition at line 372 of file `crypto_values.h`.

6.24.2.206 PSA_KEY_TYPE_CATEGORY_MASK

```
#define PSA_KEY_TYPE_CATEGORY_MASK ((psa_key_type_t) 0x7000)
```

Definition at line 368 of file crypto_values.h.

6.24.2.207 PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY

```
#define PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY ((psa_key_type_t) 0x4000)
```

Definition at line 371 of file crypto_values.h.

6.24.2.208 PSA_KEY_TYPE_CATEGORY_RAW

```
#define PSA_KEY_TYPE_CATEGORY_RAW ((psa_key_type_t) 0x1000)
```

Definition at line 369 of file crypto_values.h.

6.24.2.209 PSA_KEY_TYPE_CATEGORY_SYMMETRIC

```
#define PSA_KEY_TYPE_CATEGORY_SYMMETRIC ((psa_key_type_t) 0x2000)
```

Definition at line 370 of file crypto_values.h.

6.24.2.210 PSA_KEY_TYPE_CHACHA20

```
#define PSA_KEY_TYPE_CHACHA20 ((psa_key_type_t) 0x2004)
```

Key for the ChaCha20 stream cipher or the ChaCha20-Poly1305 AEAD algorithm.

ChaCha20 and the ChaCha20_Poly1305 construction are defined in RFC 7539.

Note

For ChaCha20 and ChaCha20_Poly1305, Mbed TLS only supports 12-byte nonces.

For ChaCha20, the initial counter value is 0. To encrypt or decrypt with the initial counter value 1, you can process and discard a 64-byte block before the real data.

Definition at line 521 of file crypto_values.h.

6.24.2.211 PSA_KEY_TYPE_DERIVE

```
#define PSA_KEY_TYPE_DERIVE ((psa_key_type_t) 0x1200)
```

A secret for key derivation.

This key type is for high-entropy secrets only. For low-entropy secrets, [PSA_KEY_TYPE_PASSWORD](#) should be used instead.

These keys can be used as the [PSA_KEY_DERIVATION_INPUT_SECRET](#) or [PSA_KEY_DERIVATION_INPUT_PASSWORD](#) input of key derivation algorithms.

The key policy determines which key derivation algorithm the key can be used for.

Definition at line 455 of file `crypto_values.h`.

6.24.2.212 PSA_KEY_TYPE_DH_GET_FAMILY

```
#define PSA_KEY_TYPE_DH_GET_FAMILY(  
    type )
```

Value:

```
((psa_dh_family_t) (PSA_KEY_TYPE_IS_DH(type) ?  
    ((type) & PSA_KEY_TYPE_DH_GROUP_MASK) :  
    0))
```

Extract the group from a Diffie-Hellman key type.

Definition at line 720 of file `crypto_values.h`.

6.24.2.213 PSA_KEY_TYPE_DH_GROUP_MASK

```
#define PSA_KEY_TYPE_DH_GROUP_MASK ((psa_key_type_t) 0x00ff)
```

Definition at line 690 of file `crypto_values.h`.

6.24.2.214 PSA_KEY_TYPE_DH_KEY_PAIR

```
#define PSA_KEY_TYPE_DH_KEY_PAIR(  
    group ) (PSA_KEY_TYPE_DH_KEY_PAIR_BASE | (group))
```

Diffie-Hellman key pair.

Parameters

<i>group</i>	A value of type psa_dh_family_t that identifies the Diffie-Hellman group to be used.
--------------	--

Definition at line 696 of file crypto_values.h.

6.24.2.215 PSA_KEY_TYPE_DH_KEY_PAIR_BASE

```
#define PSA_KEY_TYPE_DH_KEY_PAIR_BASE ((psa_key_type_t) 0x7200)
```

Definition at line 689 of file crypto_values.h.

6.24.2.216 PSA_KEY_TYPE_DH_PUBLIC_KEY

```
#define PSA_KEY_TYPE_DH_PUBLIC_KEY(  
    group ) (PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE | (group))
```

Diffie-Hellman public key.

Parameters

<i>group</i>	A value of type <code>psa_dh_family_t</code> that identifies the Diffie-Hellman group to be used.
--------------	---

Definition at line 703 of file crypto_values.h.

6.24.2.217 PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE

```
#define PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4200)
```

Definition at line 688 of file crypto_values.h.

6.24.2.218 PSA_KEY_TYPE_DSA_KEY_PAIR

```
#define PSA_KEY_TYPE_DSA_KEY_PAIR ((psa_key_type_t) 0x7002)
```

DSA key pair (private and public key).

The import and export format is the representation of the private key x as a big-endian byte string. The length of the byte string is the private key size in bytes (leading zeroes are not stripped).

Deterministic DSA key derivation with `psa_generate_derived_key` follows FIPS 186-4 §B.1.2: interpret the byte string as integer in big-endian order. Discard it if it is not in the range $[0, N - 2]$ where N is the boundary of the private key domain (the prime p for Diffie-Hellman, the subprime q for DSA, or the order of the curve's base point for ECC). Add 1 to the resulting integer and use this as the private key x .

Definition at line 189 of file crypto_extra.h.

6.24.2.219 PSA_KEY_TYPE_DSA_PUBLIC_KEY

```
#define PSA_KEY_TYPE_DSA_PUBLIC_KEY ((psa_key_type_t) 0x4002)
```

DSA public key.

The import and export format is the representation of the public key $y = g^x \bmod p$ as a big-endian byte string. The length of the byte string is the length of the base prime p in bytes.

Definition at line 171 of file `crypto_extra.h`.

6.24.2.220 PSA_KEY_TYPE_ECC_CURVE_MASK

```
#define PSA_KEY_TYPE_ECC_CURVE_MASK ((psa_key_type_t) 0x00ff)
```

Definition at line 539 of file `crypto_values.h`.

6.24.2.221 PSA_KEY_TYPE_ECC_GET_FAMILY

```
#define PSA_KEY_TYPE_ECC_GET_FAMILY(  
    type )
```

Value:

```
((psa_ecc_family_t) (PSA_KEY_TYPE_HAS_ECC_FAMILY(type) ?  
    ((type) & PSA_KEY_TYPE_ECC_CURVE_MASK) : \  
    0))
```

Extract the curve from an elliptic curve key type.

Definition at line 580 of file `crypto_values.h`.

6.24.2.222 PSA_KEY_TYPE_ECC_KEY_PAIR

```
#define PSA_KEY_TYPE_ECC_KEY_PAIR(  
    curve ) (PSA_KEY_TYPE_ECC_KEY_PAIR_BASE | (curve))
```

Elliptic curve key pair.

The size of an elliptic curve key is the bit size associated with the curve, i.e. the bit size of q for a curve over a field F_q . See the documentation of `PSA_ECC_FAMILY_XXX` curve families for details.

Parameters

<i>curve</i>	A value of type <code>psa_ecc_family_t</code> that identifies the ECC curve to be used.
--------------	---

Definition at line 549 of file `crypto_values.h`.

6.24.2.223 PSA_KEY_TYPE_ECC_KEY_PAIR_BASE

```
#define PSA_KEY_TYPE_ECC_KEY_PAIR_BASE ((psa_key_type_t) 0x7100)
```

Definition at line 538 of file `crypto_values.h`.

6.24.2.224 PSA_KEY_TYPE_ECC_PUBLIC_KEY

```
#define PSA_KEY_TYPE_ECC_PUBLIC_KEY(  
    curve ) (PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE | (curve))
```

Elliptic curve public key.

The size of an elliptic curve public key is the same as the corresponding private key (see [PSA_KEY_TYPE_ECC_KEY_PAIR](#) and the documentation of `PSA_ECC_FAMILY_XXX` curve families).

Parameters

<i>curve</i>	A value of type psa_ecc_family_t that identifies the ECC curve to be used.
--------------	--

Definition at line 560 of file `crypto_values.h`.

6.24.2.225 PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE

```
#define PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4100)
```

Definition at line 537 of file `crypto_values.h`.

6.24.2.226 PSA_KEY_TYPE_HAS_ECC_FAMILY

```
#define PSA_KEY_TYPE_HAS_ECC_FAMILY(  
    type ) (PSA_KEY_TYPE_IS_ECC(type) || PSA_KEY_TYPE_IS_SPAKE2P(type))
```

Definition at line 576 of file `crypto_values.h`.

6.24.2.227 PSA_KEY_TYPE_HMAC

```
#define PSA_KEY_TYPE_HMAC ((psa_key_type_t) 0x1100)
```

HMAC key.

The key policy determines which underlying hash algorithm the key can be used for.

HMAC keys should generally have the same size as the underlying hash. This size can be calculated with `PSA_HASH_LENGTH(alg)` where `alg` is the HMAC algorithm or the underlying hash algorithm.

Definition at line 442 of file `crypto_values.h`.

6.24.2.228 PSA_KEY_TYPE_IS_ASYMMETRIC

```
#define PSA_KEY_TYPE_IS_ASYMMETRIC(  
    type )
```

Value:

```
((type) & PSA_KEY_TYPE_CATEGORY_MASK  
& ~PSA_KEY_TYPE_CATEGORY_FLAG_PAIR) ==  
PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY)
```

\

Whether a key type is asymmetric: either a key pair or a public key.

Definition at line 392 of file `crypto_values.h`.

6.24.2.229 PSA_KEY_TYPE_IS_DH

```
#define PSA_KEY_TYPE_IS_DH(  
    type )
```

Value:

```
((PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) &  
~PSA_KEY_TYPE_DH_GROUP_MASK) == PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE)
```

\

Whether a key type is a Diffie-Hellman key (pair or public-only).

Definition at line 707 of file `crypto_values.h`.

6.24.2.230 PSA_KEY_TYPE_IS_DH_KEY_PAIR

```
#define PSA_KEY_TYPE_IS_DH_KEY_PAIR(  
    type )
```

Value:

```
((type) & ~PSA_KEY_TYPE_DH_GROUP_MASK) ==  
PSA_KEY_TYPE_DH_KEY_PAIR_BASE)
```

\

Whether a key type is a Diffie-Hellman key pair.

Definition at line 711 of file `crypto_values.h`.

6.24.2.231 PSA_KEY_TYPE_IS_DH_PUBLIC_KEY

```
#define PSA_KEY_TYPE_IS_DH_PUBLIC_KEY(  
    type )
```

Value:

```
((type) & ~PSA_KEY_TYPE_DH_GROUP_MASK) ==  
PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE) \
```

Whether a key type is a Diffie-Hellman public key.

Definition at line 715 of file `crypto_values.h`.

6.24.2.232 PSA_KEY_TYPE_IS_DSA

```
#define PSA_KEY_TYPE_IS_DSA(  
    type ) (PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) == PSA_KEY_TYPE_DSA_PUBLIC_KEY)
```

Whether a key type is a DSA key (pair or public-only).

Definition at line 192 of file `crypto_extra.h`.

6.24.2.233 PSA_KEY_TYPE_IS_ECC

```
#define PSA_KEY_TYPE_IS_ECC(  
    type )
```

Value:

```
((PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) &  
~PSA_KEY_TYPE_ECC_CURVE_MASK) == PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE) \
```

Whether a key type is an elliptic curve key (pair or public-only).

Definition at line 564 of file `crypto_values.h`.

6.24.2.234 PSA_KEY_TYPE_IS_ECC_KEY_PAIR

```
#define PSA_KEY_TYPE_IS_ECC_KEY_PAIR(  
    type )
```

Value:

```
((type) & ~PSA_KEY_TYPE_ECC_CURVE_MASK) ==  
PSA_KEY_TYPE_ECC_KEY_PAIR_BASE) \
```

Whether a key type is an elliptic curve key pair.

Definition at line 568 of file `crypto_values.h`.

6.24.2.235 PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY

```
#define PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY(  
    type )
```

Value:

```
((type) & ~PSA_KEY_TYPE_ECC_CURVE_MASK) ==  
PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE) \
```

Whether a key type is an elliptic curve public key.

Definition at line 572 of file `crypto_values.h`.

6.24.2.236 PSA_KEY_TYPE_IS_KEY_PAIR

```
#define PSA_KEY_TYPE_IS_KEY_PAIR(  
    type ) ((type) & PSA_KEY_TYPE_CATEGORY_MASK) == PSA_KEY_TYPE_CATEGORY_KEY_PAIR)
```

Whether a key type is a key pair containing a private part and a public part.

Definition at line 401 of file `crypto_values.h`.

6.24.2.237 PSA_KEY_TYPE_IS_PUBLIC_KEY

```
#define PSA_KEY_TYPE_IS_PUBLIC_KEY(  
    type ) ((type) & PSA_KEY_TYPE_CATEGORY_MASK) == PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY)
```

Whether a key type is the public part of a key pair.

Definition at line 397 of file `crypto_values.h`.

6.24.2.238 PSA_KEY_TYPE_IS_RSA

```
#define PSA_KEY_TYPE_IS_RSA(  
    type ) (PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) == PSA_KEY_TYPE_RSA_PUBLIC_KEY)
```

Whether a key type is an RSA key (pair or public-only).

Definition at line 534 of file `crypto_values.h`.

6.24.2.239 PSA_KEY_TYPE_IS_SPAKE2P

```
#define PSA_KEY_TYPE_IS_SPAKE2P(  
    type )
```

Value:

```
((PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) & \  
  ~PSA_KEY_TYPE_ECC_CURVE_MASK) == PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE)
```

Whether a key type is a SPAKE2+ key pair or public key type.

Definition at line 779 of file `crypto_extra.h`.

6.24.2.240 PSA_KEY_TYPE_IS_SPAKE2P_KEY_PAIR

```
#define PSA_KEY_TYPE_IS_SPAKE2P_KEY_PAIR(  
    type )
```

Value:

```
((type) & ~PSA_KEY_TYPE_ECC_CURVE_MASK) == \  
  PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE)
```

Whether a key type is a SPAKE2+ key pair type.

Definition at line 769 of file `crypto_extra.h`.

6.24.2.241 PSA_KEY_TYPE_IS_SPAKE2P_PUBLIC_KEY

```
#define PSA_KEY_TYPE_IS_SPAKE2P_PUBLIC_KEY(  
    type )
```

Value:

```
((type) & ~PSA_KEY_TYPE_ECC_CURVE_MASK) == \  
  PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE)
```

Whether a key type is a SPAKE2+ public key type.

Definition at line 774 of file `crypto_extra.h`.

6.24.2.242 PSA_KEY_TYPE_IS_UNSTRUCTURED

```
#define PSA_KEY_TYPE_IS_UNSTRUCTURED(  
    type )
```

Value:

```
((type) & PSA_KEY_TYPE_CATEGORY_MASK) == PSA_KEY_TYPE_CATEGORY_RAW || \  
  ((type) & PSA_KEY_TYPE_CATEGORY_MASK) == PSA_KEY_TYPE_CATEGORY_SYMMETRIC)
```

Whether a key type is an unstructured array of bytes.

This encompasses both symmetric keys and non-key data.

Definition at line 387 of file `crypto_values.h`.

6.24.2.243 PSA_KEY_TYPE_IS_VENDOR_DEFINED

```
#define PSA_KEY_TYPE_IS_VENDOR_DEFINED(  
    type ) (((type) & PSA_KEY_TYPE_VENDOR_FLAG) != 0)
```

Whether a key type is vendor-defined.

See also [PSA_KEY_TYPE_VENDOR_FLAG](#).

Definition at line 380 of file `crypto_values.h`.

6.24.2.244 PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY

```
#define PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY(  
    type ) ((type) | PSA_KEY_TYPE_CATEGORY_FLAG_PAIR)
```

The key pair type corresponding to a public key type.

You may also pass a key pair type as `type`, it will be left unchanged.

Parameters

<i>type</i>	A public key type or key pair type.
-------------	-------------------------------------

Returns

The corresponding key pair type. If `type` is not a public key or a key pair, the return value is undefined.

Definition at line 413 of file `crypto_values.h`.

6.24.2.245 PSA_KEY_TYPE_NONE

```
#define PSA_KEY_TYPE_NONE ((psa_key_type_t) 0x0000)
```

An invalid key type value.

Zero is not the encoding of any key type.

Definition at line 357 of file `crypto_values.h`.

6.24.2.246 PSA_KEY_TYPE_PASSWORD

```
#define PSA_KEY_TYPE_PASSWORD ((psa_key_type_t) 0x1203)
```

A low-entropy secret for password hashing or key derivation.

This key type is suitable for passwords and passphrases which are typically intended to be memorizable by humans, and have a low entropy relative to their size. It can be used for randomly generated or derived keys with maximum or near-maximum entropy, but [PSA_KEY_TYPE_DERIVE](#) is more suitable for such keys. It is not suitable for passwords with extremely low entropy, such as numerical PINs.

These keys can be used as the [PSA_KEY_DERIVATION_INPUT_PASSWORD](#) input of key derivation algorithms. Algorithms that accept such an input were designed to accept low-entropy secret and are known as password hashing or key stretching algorithms.

These keys cannot be used as the [PSA_KEY_DERIVATION_INPUT_SECRET](#) input of key derivation algorithms, as the algorithms that take such an input expect it to be high-entropy.

The key policy determines which key derivation algorithm the key can be used for, among the permissible subset defined above.

Definition at line 478 of file `crypto_values.h`.

6.24.2.247 PSA_KEY_TYPE_PASSWORD_HASH

```
#define PSA_KEY_TYPE_PASSWORD_HASH ((psa_key_type_t) 0x1205)
```

A secret value that can be used to verify a password hash.

The key policy determines which key derivation algorithm the key can be used for, among the same permissible subset as for [PSA_KEY_TYPE_PASSWORD](#).

Definition at line 486 of file `crypto_values.h`.

6.24.2.248 PSA_KEY_TYPE_PEPPER

```
#define PSA_KEY_TYPE_PEPPER ((psa_key_type_t) 0x1206)
```

A secret value that can be used in when computing a password hash.

The key policy determines which key derivation algorithm the key can be used for, among the subset of algorithms that can use pepper.

Definition at line 493 of file `crypto_values.h`.

6.24.2.249 PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR

```
#define PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(  
    type) ((type) & ~PSA_KEY_TYPE_CATEGORY_FLAG_PAIR)
```

The public key type corresponding to a key pair type.

You may also pass a public key type as `type`, it will be left unchanged.

Parameters

<i>type</i>	A public key type or key pair type.
-------------	-------------------------------------

Returns

The corresponding public key type. If `type` is not a public key or a key pair, the return value is undefined.

Definition at line 425 of file `crypto_values.h`.

6.24.2.250 PSA_KEY_TYPE_RAW_DATA

```
#define PSA_KEY_TYPE_RAW_DATA ((psa_key_type_t) 0x1001)
```

Raw data.

A "key" of this type cannot be used for any cryptographic operation. Applications may use this type to store arbitrary data in the keystore.

Definition at line 432 of file `crypto_values.h`.

6.24.2.251 PSA_KEY_TYPE_RSA_KEY_PAIR

```
#define PSA_KEY_TYPE_RSA_KEY_PAIR ((psa_key_type_t) 0x7001)
```

RSA key pair (private and public key).

The size of an RSA key is the bit size of the modulus.

Definition at line 532 of file `crypto_values.h`.

6.24.2.252 PSA_KEY_TYPE_RSA_PUBLIC_KEY

```
#define PSA_KEY_TYPE_RSA_PUBLIC_KEY ((psa_key_type_t) 0x4001)
```

RSA public key.

The size of an RSA key is the bit size of the modulus.

Definition at line 527 of file `crypto_values.h`.

6.24.2.253 PSA_KEY_TYPE_SPAKE2P_KEY_PAIR

```
#define PSA_KEY_TYPE_SPAKE2P_KEY_PAIR(  
    curve ) (PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE | (curve))
```

SPAKE2+ key pair.

Not implemented yet.

Definition at line 758 of file `crypto_extra.h`.

6.24.2.254 PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE

```
#define PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE ((psa_key_type_t) 0x7400)
```

Definition at line 752 of file `crypto_extra.h`.

6.24.2.255 PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY

```
#define PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY(  
    curve ) (PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE | (curve))
```

SPAKE2+ public key.

Not implemented yet.

Definition at line 765 of file `crypto_extra.h`.

6.24.2.256 PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE

```
#define PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4400)
```

Definition at line 751 of file `crypto_extra.h`.

6.24.2.257 PSA_KEY_TYPE_VENDOR_FLAG

```
#define PSA_KEY_TYPE_VENDOR_FLAG ((psa_key_type_t) 0x8000)
```

Vendor-defined key type flag.

Key types defined by this standard will never have the [PSA_KEY_TYPE_VENDOR_FLAG](#) bit set. Vendors who define additional key types must use an encoding with the [PSA_KEY_TYPE_VENDOR_FLAG](#) bit set and should respect the bitwise structure used by standard encodings whenever practical.

Definition at line 366 of file `crypto_values.h`.

6.24.2.258 PSA_MAC_TRUNCATED_LENGTH

```
#define PSA_MAC_TRUNCATED_LENGTH(  
    mac_alg ) (((mac_alg) & PSA_ALG_MAC_TRUNCATION_MASK) >> PSA_MAC_TRUNCATION_OFFSET)
```

Length to which a MAC algorithm is truncated.

Parameters

<i>mac_alg</i>	A MAC algorithm identifier (value of type psa_algorithm_t such that PSA_ALG_IS_MAC (<i>mac_alg</i>) is true).
----------------	---

Returns

- Length of the truncated MAC in bytes.
- 0 if *mac_alg* is a non-truncated MAC algorithm.
- Unspecified if *mac_alg* is not a supported MAC algorithm.

Definition at line 1133 of file `crypto_values.h`.

6.24.2.259 PSA_MAC_TRUNCATION_OFFSET

```
#define PSA_MAC_TRUNCATION_OFFSET 16
```

Definition at line 1058 of file `crypto_values.h`.

6.24.3 Typedef Documentation**6.24.3.1 psa_algorithm_t**

```
typedef uint32_t psa_algorithm_t
```

Encoding of a cryptographic algorithm.

Values of this type are generally constructed by macros called `PSA_ALG_XXX`.

For algorithms that can be applied to multiple key types, this type does not encode the key type. For example, for symmetric ciphers based on a block cipher, [psa_algorithm_t](#) encodes the block cipher mode and the padding mode while the block cipher itself is encoded via [psa_key_type_t](#).

Note

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 127 of file `crypto_types.h`.

6.24.3.2 `psa_dh_family_t`

```
typedef uint8_t psa_dh_family_t
```

The type of PSA Diffie-Hellman group family identifiers.

Values of this type are generally constructed by macros called `PSA_DH_FAMILY_XXX`.

The group identifier is required to create a Diffie-Hellman key using the `PSA_KEY_TYPE_DH_KEY_PAIR()` or `PSA_KEY_TYPE_DH_PUBLIC_KEY()` macros.

Values defined by this standard will never be in the range 0x80-0xff. Vendors who define additional families must use an encoding in this range.

Note

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 109 of file `crypto_types.h`.

6.24.3.3 `psa_ecc_family_t`

```
typedef uint8_t psa_ecc_family_t
```

The type of PSA elliptic curve family identifiers.

Values of this type are generally constructed by macros called `PSA_ECC_FAMILY_XXX`.

The curve identifier is required to create an ECC key using the `PSA_KEY_TYPE_ECC_KEY_PAIR()` or `PSA_KEY_TYPE_ECC_PUBLIC_KEY()` macros.

Values defined by this standard will never be in the range 0x80-0xff. Vendors who define additional families must use an encoding in this range.

Note

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 90 of file `crypto_types.h`.

6.24.3.4 `psa_key_type_t`

```
typedef uint16_t psa_key_type_t
```

Encoding of a key type.

Values of this type are generally constructed by macros called `PSA_KEY_TYPE_XXX`.

Note

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 71 of file `crypto_types.h`.

6.25 Key lifetimes

Macros

- `#define PSA_KEY_LIFETIME_VOLATILE ((psa_key_lifetime_t) 0x00000000)`
- `#define PSA_KEY_LIFETIME_PERSISTENT ((psa_key_lifetime_t) 0x00000001)`
- `#define PSA_KEY_PERSISTENCE_VOLATILE ((psa_key_persistence_t) 0x00)`
- `#define PSA_KEY_PERSISTENCE_DEFAULT ((psa_key_persistence_t) 0x01)`
- `#define PSA_KEY_PERSISTENCE_READ_ONLY ((psa_key_persistence_t) 0xff)`
- `#define PSA_KEY_LIFETIME_GET_PERSISTENCE(lifetime) ((psa_key_persistence_t) ((lifetime) & 0x000000ff))`
- `#define PSA_KEY_LIFETIME_GET_LOCATION(lifetime) ((psa_key_location_t) ((lifetime) >> 8))`
- `#define PSA_KEY_LIFETIME_IS_VOLATILE(lifetime)`
- `#define PSA_KEY_LIFETIME_IS_READ_ONLY(lifetime)`
- `#define PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION(persistence, location) ((location) << 8 | (persistence))`
- `#define PSA_KEY_LOCATION_LOCAL_STORAGE ((psa_key_location_t) 0x00000000)`
- `#define PSA_KEY_LOCATION_VENDOR_FLAG ((psa_key_location_t) 0x80000000)`
- `#define PSA_KEY_ID_NULL ((psa_key_id_t) 0)`
- `#define PSA_KEY_ID_USER_MIN ((psa_key_id_t) 0x00000001)`
- `#define PSA_KEY_ID_USER_MAX ((psa_key_id_t) 0x3fffffff)`
- `#define PSA_KEY_ID_VENDOR_MIN ((psa_key_id_t) 0x40000000)`
- `#define PSA_KEY_ID_VENDOR_MAX ((psa_key_id_t) 0x7fffffff)`
- `#define MBEDTLS_SVC_KEY_ID_INIT ((psa_key_id_t) 0)`
- `#define MBEDTLS_SVC_KEY_ID_GET_KEY_ID(id) (id)`
- `#define MBEDTLS_SVC_KEY_ID_GET_OWNER_ID(id) (0)`

Typedefs

- `typedef uint32_t psa_key_lifetime_t`
- `typedef uint8_t psa_key_persistence_t`
- `typedef uint32_t psa_key_location_t`
- `typedef uint32_t psa_key_id_t`
- `typedef psa_key_id_t mbedtls_svc_key_id_t`

6.25.1 Detailed Description

6.25.2 Macro Definition Documentation

6.25.2.1 MBEDTLS_SVC_KEY_ID_GET_KEY_ID

```
#define MBEDTLS_SVC_KEY_ID_GET_KEY_ID(  
    id ) (id)
```

Definition at line 2520 of file `crypto_values.h`.

6.25.2.2 MBEDTLS_SVC_KEY_ID_GET_OWNER_ID

```
#define MBEDTLS_SVC_KEY_ID_GET_OWNER_ID(  
    id ) (0)
```

Definition at line 2521 of file `crypto_values.h`.

6.25.2.3 MBEDTLS_SVC_KEY_ID_INIT

```
#define MBEDTLS_SVC_KEY_ID_INIT ( (psa_key_id_t) 0 )
```

Definition at line 2519 of file `crypto_values.h`.

6.25.2.4 PSA_KEY_ID_NULL

```
#define PSA_KEY_ID_NULL ( (psa_key_id_t) 0 )
```

The null key identifier.

Definition at line 2501 of file `crypto_values.h`.

6.25.2.5 PSA_KEY_ID_USER_MAX

```
#define PSA_KEY_ID_USER_MAX ( (psa_key_id_t) 0x3fffffff )
```

The maximum value for a key identifier chosen by the application.

Definition at line 2508 of file `crypto_values.h`.

6.25.2.6 PSA_KEY_ID_USER_MIN

```
#define PSA_KEY_ID_USER_MIN ( (psa_key_id_t) 0x00000001 )
```

The minimum value for a key identifier chosen by the application.

Definition at line 2505 of file `crypto_values.h`.

6.25.2.7 PSA_KEY_ID_VENDOR_MAX

```
#define PSA_KEY_ID_VENDOR_MAX ((psa_key_id_t) 0x7fffffff)
```

The maximum value for a key identifier chosen by the implementation.

Definition at line 2514 of file `crypto_values.h`.

6.25.2.8 PSA_KEY_ID_VENDOR_MIN

```
#define PSA_KEY_ID_VENDOR_MIN ((psa_key_id_t) 0x40000000)
```

The minimum value for a key identifier chosen by the implementation.

Definition at line 2511 of file `crypto_values.h`.

6.25.2.9 PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION

```
#define PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION(  
    persistence,  
    location ) ((location) << 8 | (persistence))
```

Construct a lifetime from a persistence level and a location.

Parameters

<i>persistence</i>	The persistence level (value of type psa_key_persistence_t).
<i>location</i>	The location indicator (value of type psa_key_location_t).

Returns

The constructed lifetime value.

Definition at line 2479 of file `crypto_values.h`.

6.25.2.10 PSA_KEY_LIFETIME_GET_LOCATION

```
#define PSA_KEY_LIFETIME_GET_LOCATION(  
    lifetime ) ((psa_key_location_t) ((lifetime) >> 8))
```

Definition at line 2426 of file `crypto_values.h`.

6.25.2.11 PSA_KEY_LIFETIME_GET_PERSISTENCE

```
#define PSA_KEY_LIFETIME_GET_PERSISTENCE(  
    lifetime ) ((psa_key_persistence_t) ((lifetime) & 0x000000ff))
```

Definition at line 2423 of file `crypto_values.h`.

6.25.2.12 PSA_KEY_LIFETIME_IS_READ_ONLY

```
#define PSA_KEY_LIFETIME_IS_READ_ONLY(  
    lifetime )
```

Value:

```
(PSA_KEY_LIFETIME_GET_PERSISTENCE(lifetime) == \  
    PSA_KEY_PERSISTENCE_READ_ONLY)
```

Whether a key lifetime indicates that the key is read-only.

Read-only keys cannot be created or destroyed through the PSA Crypto API. They must be created through platform-specific means that bypass the API.

Some platforms may offer ways to destroy read-only keys. For example, consider a platform with multiple levels of privilege, where a low-privilege application can use a key but is not allowed to destroy it, and the platform exposes the key to the application with a read-only lifetime. High-privilege code can destroy the key even though the application sees the key as read-only.

Parameters

<i>lifetime</i>	The lifetime value to query (value of type <code>psa_key_lifetime_t</code>).
-----------------	---

Returns

1 if the key is read-only, otherwise 0.

Definition at line 2466 of file `crypto_values.h`.

6.25.2.13 PSA_KEY_LIFETIME_IS_VOLATILE

```
#define PSA_KEY_LIFETIME_IS_VOLATILE(  
    lifetime )
```

Value:

```
(PSA_KEY_LIFETIME_GET_PERSISTENCE(lifetime) == \  
    PSA_KEY_PERSISTENCE_VOLATILE)
```

Whether a key lifetime indicates that the key is volatile.

A volatile key is automatically destroyed by the implementation when the application instance terminates. In particular, a volatile key is automatically destroyed on a power reset of the device.

A key that is not volatile is persistent. Persistent keys are preserved until the application explicitly destroys them or until an implementation-specific device management event occurs (for example, a factory reset).

Parameters

<i>lifetime</i>	The lifetime value to query (value of type psa_key_lifetime_t).
-----------------	--

Returns

1 if the key is volatile, otherwise 0.

Definition at line 2445 of file `crypto_values.h`.

6.25.2.14 PSA_KEY_LIFETIME_PERSISTENT

```
#define PSA_KEY_LIFETIME_PERSISTENT ((psa_key_lifetime_t) 0x00000001)
```

The default lifetime for persistent keys.

A persistent key remains in storage until it is explicitly destroyed or until the corresponding storage area is wiped. This specification does not define any mechanism to wipe a storage area, but integrations may provide their own mechanism (for example to perform a factory reset, to prepare for device refurbishment, or to uninstall an application).

This lifetime value is the default storage area for the calling application. Integrations of Mbed TLS may support other persistent lifetimes. See [psa_key_lifetime_t](#) for more information.

Definition at line 2403 of file `crypto_values.h`.

6.25.2.15 PSA_KEY_LIFETIME_VOLATILE

```
#define PSA_KEY_LIFETIME_VOLATILE ((psa_key_lifetime_t) 0x00000000)
```

The default lifetime for volatile keys.

A volatile key only exists as long as the identifier to it is not destroyed. The key material is guaranteed to be erased on a power reset.

A key with this lifetime is typically stored in the RAM area of the PSA Crypto subsystem. However this is an implementation choice. If an implementation stores data about the key in a non-volatile memory, it must release all the resources associated with the key and erase the key material if the calling application terminates.

Definition at line 2389 of file `crypto_values.h`.

6.25.2.16 PSA_KEY_LOCATION_LOCAL_STORAGE

```
#define PSA_KEY_LOCATION_LOCAL_STORAGE ((psa_key_location_t) 0x000000)
```

The local storage area for persistent keys.

This storage area is available on all systems that can store persistent keys without delegating the storage to a third-party cryptoprocessor.

See [psa_key_location_t](#) for more information.

Definition at line 2489 of file `crypto_values.h`.

6.25.2.17 PSA_KEY_LOCATION_VENDOR_FLAG

```
#define PSA_KEY_LOCATION_VENDOR_FLAG ((psa_key_location_t) 0x800000)
```

Definition at line 2491 of file `crypto_values.h`.

6.25.2.18 PSA_KEY_PERSISTENCE_DEFAULT

```
#define PSA_KEY_PERSISTENCE_DEFAULT ((psa_key_persistence_t) 0x01)
```

The default persistence level for persistent keys.

See [psa_key_persistence_t](#) for more information.

Definition at line 2415 of file `crypto_values.h`.

6.25.2.19 PSA_KEY_PERSISTENCE_READ_ONLY

```
#define PSA_KEY_PERSISTENCE_READ_ONLY ((psa_key_persistence_t) 0xff)
```

A persistence level indicating that a key is never destroyed.

See [psa_key_persistence_t](#) for more information.

Definition at line 2421 of file `crypto_values.h`.

6.25.2.20 PSA_KEY_PERSISTENCE_VOLATILE

```
#define PSA_KEY_PERSISTENCE_VOLATILE ((psa\_key\_persistence\_t) 0x00)
```

The persistence level of volatile keys.

See [psa_key_persistence_t](#) for more information.

Definition at line 2409 of file `crypto_values.h`.

6.25.3 Typedef Documentation

6.25.3.1 mbedtls_svc_key_id_t

```
typedef psa\_key\_id\_t mbedtls\_svc\_key\_id\_t
```

Encoding of key identifiers as seen inside the PSA Crypto implementation.

When PSA Crypto is built as a library inside an application, this type is identical to [psa_key_id_t](#). When PSA Crypto is built as a service that can store keys on behalf of multiple clients, this type encodes the [psa_key_id_t](#) value seen by each client application as well as extra information that identifies the client that owns the key.

Note

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 285 of file `crypto_types.h`.

6.25.3.2 psa_key_id_t

```
typedef uint32_t psa\_key\_id\_t
```

Encoding of identifiers of persistent keys.

- Applications may freely choose key identifiers in the range [PSA_KEY_ID_USER_MIN](#) to [PSA_KEY_ID_USER_MAX](#).
- The implementation may define additional key identifiers in the range [PSA_KEY_ID_VENDOR_MIN](#) to [PSA_KEY_ID_VENDOR_MAX](#).
- 0 is reserved as an invalid key identifier.
- Key identifiers outside these ranges are reserved for future use.

Note

Values of this type are encoded in the persistent key store. Any changes to how values are allocated must require careful consideration to allow backward compatibility.

Definition at line 268 of file `crypto_types.h`.

6.25.3.3 `psa_key_lifetime_t`

```
typedef uint32_t psa_key_lifetime_t
```

Encoding of key lifetimes.

The lifetime of a key indicates where it is stored and what system actions may create and destroy it.

Lifetime values have the following structure:

- Bits 0-7 (`PSA_KEY_LIFETIME_GET_PERSISTENCE(lifetime)`): persistence level. This value indicates what device management actions can cause it to be destroyed. In particular, it indicates whether the key is *volatile* or *persistent*. See [psa_key_persistence_t](#) for more information.
- Bits 8-31 (`PSA_KEY_LIFETIME_GET_LOCATION(lifetime)`): location indicator. This value indicates which part of the system has access to the key material and can perform operations using the key. See [psa_key_location_t](#) for more information.

Volatile keys are automatically destroyed when the application instance terminates or on a power reset of the device. Persistent keys are preserved until the application explicitly destroys them or until an integration-specific device management event occurs (for example, a factory reset).

Persistent keys have a key identifier of type [mbedtls_svc_key_id_t](#). This identifier remains valid throughout the lifetime of the key, even if the application instance that created the key terminates. The application can call `psa_open_key()` to open a persistent key that it created previously.

The default lifetime of a key is `PSA_KEY_LIFETIME_VOLATILE`. The lifetime `PSA_KEY_LIFETIME_PERSISTENT` is supported if persistent storage is available. Other lifetime values may be supported depending on the library configuration.

Values of this type are generally constructed by macros called `PSA_KEY_LIFETIME_XXX`.

Note

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 176 of file `crypto_types.h`.

6.25.3.4 `psa_key_location_t`

```
typedef uint32_t psa_key_location_t
```

Encoding of key location indicators.

If an integration of Mbed TLS can make calls to external cryptoprocessors such as secure elements, the location of a key indicates which secure element performs the operations on the key. Depending on the design of the secure element, the key material may be stored either in the secure element, or in wrapped (encrypted) form alongside the key metadata in the primary local storage.

The PSA Cryptography API specification defines the following values of location indicators:

- 0: primary local storage. This location is always available. The primary local storage is typically the same storage area that contains the key metadata.
- 1: primary secure element. Integrations of Mbed TLS should support this value if there is a secure element attached to the operating environment. As a guideline, secure elements may provide higher resistance against side channel and physical attacks than the primary local storage, but may have restrictions on supported key types, sizes, policies and operations and may have different performance characteristics.
- 2–0x7fffff: other locations defined by a PSA specification. The PSA Cryptography API does not currently assign any meaning to these locations, but future versions of that specification or other PSA specifications may do so.
- 0x800000–0xffffffff: vendor-defined locations. No PSA specification will assign a meaning to locations in this range.

Note

Key location indicators are 24-bit values. Key management interfaces operate on lifetimes (type [psa_key_lifetime_t](#)) which encode the location as the upper 24 bits of a 32-bit value.

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 253 of file `crypto_types.h`.

6.25.3.5 `psa_key_persistence_t`

```
typedef uint8_t psa_key_persistence_t
```

Encoding of key persistence levels.

What distinguishes different persistence levels is what device management events may cause keys to be destroyed. *Volatile* keys are destroyed by a power reset. Persistent keys may be destroyed by events such as a transfer of ownership or a factory reset. What management events actually affect persistent keys at different levels is outside the scope of the PSA Cryptography specification.

The PSA Cryptography specification defines the following values of persistence levels:

- 0 = `PSA_KEY_PERSISTENCE_VOLATILE`: volatile key. A volatile key is automatically destroyed by the implementation when the application instance terminates. In particular, a volatile key is automatically destroyed on a power reset of the device.
- 1 = `PSA_KEY_PERSISTENCE_DEFAULT`: persistent key with a default lifetime.
- 2–254: currently not supported by Mbed TLS.
- 255 = `PSA_KEY_PERSISTENCE_READ_ONLY`: read-only or write-once key. A key with this persistence level cannot be destroyed. Mbed TLS does not currently offer a way to create such keys, but integrations of Mbed TLS can use it for built-in keys that the application cannot modify (for example, a hardware unique key (HUK)).

Note

Key persistence levels are 8-bit values. Key management interfaces operate on lifetimes (type [psa_key_lifetime_t](#)) which encode the persistence as the lower 8 bits of a 32-bit value.

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 212 of file `crypto_types.h`.

6.26 Key policies

Macros

- `#define PSA_KEY_USAGE_EXPORT ((psa_key_usage_t) 0x00000001)`
- `#define PSA_KEY_USAGE_COPY ((psa_key_usage_t) 0x00000002)`
- `#define PSA_KEY_USAGE_DERIVE_PUBLIC ((psa_key_usage_t) 0x00000080)`
- `#define PSA_KEY_USAGE_ENCRYPT ((psa_key_usage_t) 0x00000100)`
- `#define PSA_KEY_USAGE_DECRYPT ((psa_key_usage_t) 0x00000200)`
- `#define PSA_KEY_USAGE_SIGN_MESSAGE ((psa_key_usage_t) 0x00000400)`
- `#define PSA_KEY_USAGE_VERIFY_MESSAGE ((psa_key_usage_t) 0x00000800)`
- `#define PSA_KEY_USAGE_SIGN_HASH ((psa_key_usage_t) 0x00001000)`
- `#define PSA_KEY_USAGE_VERIFY_HASH ((psa_key_usage_t) 0x00002000)`
- `#define PSA_KEY_USAGE_DERIVE ((psa_key_usage_t) 0x00004000)`
- `#define PSA_KEY_USAGE_VERIFY_DERIVATION ((psa_key_usage_t) 0x00008000)`
- `#define PSA_KEY_USAGE_WRAP ((psa_key_usage_t) 0x00010000)`
- `#define PSA_KEY_USAGE_UNWRAP ((psa_key_usage_t) 0x00020000)`

Permission to unwrap another key with the key.

Typedefs

- `typedef uint32_t psa_key_usage_t`
Encoding of permitted usage on a key.

6.26.1 Detailed Description

6.26.2 Macro Definition Documentation

6.26.2.1 PSA_KEY_USAGE_COPY

```
#define PSA_KEY_USAGE_COPY ((psa_key_usage_t) 0x00000002)
```

Whether the key may be copied.

This flag allows the use of `psa_copy_key()` to make a copy of the key with the same policy or a more restrictive policy.

For lifetimes for which the key is located in a secure element which enforce the non-exportability of keys, copying a key outside the secure element also requires the usage flag `PSA_KEY_USAGE_EXPORT`. Copying the key inside the secure element is permitted with just `PSA_KEY_USAGE_COPY` if the secure element supports it. For keys with the lifetime `PSA_KEY_LIFETIME_VOLATILE` or `PSA_KEY_LIFETIME_PERSISTENT`, the usage flag `PSA_KEY_USAGE_COPY` is sufficient to permit the copy.

Definition at line 2643 of file `crypto_values.h`.

6.26.2.2 PSA_KEY_USAGE_DECRYPT

```
#define PSA_KEY_USAGE_DECRYPT ((psa_key_usage_t) 0x00000200)
```

Whether the key may be used to decrypt a message.

This flag allows the key to be used for a symmetric decryption operation, for an AEAD decryption-and-verification operation, or for an asymmetric decryption operation, if otherwise permitted by the key's type and policy.

For a key pair, this concerns the private key.

Definition at line 2684 of file `crypto_values.h`.

6.26.2.3 PSA_KEY_USAGE_DERIVE

```
#define PSA_KEY_USAGE_DERIVE ((psa_key_usage_t) 0x00004000)
```

Whether the key may be used to derive other keys or produce a password hash.

This flag allows the key to be used for a key derivation operation or for a key agreement operation, if otherwise permitted by the key's type and policy.

If this flag is present on all keys used in calls to [psa_key_derivation_input_key\(\)](#) for a key derivation operation, then it permits calling [psa_key_derivation_output_bytes\(\)](#) or [psa_key_derivation_output_key\(\)](#) at the end of the operation.

Definition at line 2738 of file `crypto_values.h`.

6.26.2.4 PSA_KEY_USAGE_DERIVE_PUBLIC

```
#define PSA_KEY_USAGE_DERIVE_PUBLIC ((psa_key_usage_t) 0x00000080)
```

Whether the key may be used the public side of a key agreement or PAKE.

This macro can be used when checking a key's capabilities, for example with [mbedtls_pk_can_do_psa\(\)](#).

Note

Currently, no API function requires this flag. Key agreement functions ([psa_raw_key_agreement\(\)](#), [psa_key_agreement\(\)](#), [psa_key_derivation_key_agreement\(\)](#)) and [psa_pake_input\(\)](#) take the public key in exported form, not as a key object, so no usage flag is involved. For PAKE algorithms with a verifier role such as SPAKE2+, [psa_pake_setup\(\)](#) requires [PSA_KEY_USAGE_DERIVE](#) even when passing a public key in the verifier role.

The value of this macro is determined by a draft version of the PSA Cryptography API, and may change before this draft is finalized.

Definition at line 2662 of file `crypto_values.h`.

6.26.2.5 PSA_KEY_USAGE_ENCRYPT

```
#define PSA_KEY_USAGE_ENCRYPT ((psa_key_usage_t) 0x00000100)
```

Whether the key may be used to encrypt a message.

This flag allows the key to be used for a symmetric encryption operation, for an AEAD encryption-and-authentication operation, or for an asymmetric encryption operation, if otherwise permitted by the key's type and policy.

For a key pair, this concerns the public key.

Definition at line 2673 of file `crypto_values.h`.

6.26.2.6 PSA_KEY_USAGE_EXPORT

```
#define PSA_KEY_USAGE_EXPORT ((psa_key_usage_t) 0x00000001)
```

Whether the key may be exported.

A public key or the public part of a key pair may always be exported regardless of the value of this permission flag.

If a key does not have export permission, implementations shall not allow the key to be exported in plain form from the cryptoprocessor, whether through `psa_export_key()` or through a proprietary interface. The key may however be exportable in a wrapped form, i.e. in a form where it is encrypted by another key.

Definition at line 2627 of file `crypto_values.h`.

6.26.2.7 PSA_KEY_USAGE_SIGN_HASH

```
#define PSA_KEY_USAGE_SIGN_HASH ((psa_key_usage_t) 0x00001000)
```

Whether the key may be used to sign a message.

This flag allows the key to be used for a MAC calculation operation or for an asymmetric signature operation, if otherwise permitted by the key's type and policy.

For a key pair, this concerns the private key.

Definition at line 2714 of file `crypto_values.h`.

6.26.2.8 PSA_KEY_USAGE_SIGN_MESSAGE

```
#define PSA_KEY_USAGE_SIGN_MESSAGE ((psa_key_usage_t) 0x00000400)
```

Whether the key may be used to sign a message.

This flag allows the key to be used for a MAC calculation operation or for an asymmetric message signature operation, if otherwise permitted by the key's type and policy.

For a key pair, this concerns the private key.

Definition at line 2694 of file `crypto_values.h`.

6.26.2.9 PSA_KEY_USAGE_UNWRAP

```
#define PSA_KEY_USAGE_UNWRAP ((psa_key_usage_t) 0x00020000)
```

Permission to unwrap another key with the key.

This flag is required to use the key in a key-unwrapping operation.

The flag must be present on keys used with the following APIs:

- [psa_unwrap_key\(\)](#)

Definition at line 2774 of file `crypto_values.h`.

6.26.2.10 PSA_KEY_USAGE_VERIFY_DERIVATION

```
#define PSA_KEY_USAGE_VERIFY_DERIVATION ((psa_key_usage_t) 0x00008000)
```

Whether the key may be used to verify the result of a key derivation, including password hashing.

This flag allows the key to be used:

This flag allows the key to be used in a key derivation operation, if otherwise permitted by the key's type and policy.

If this flag is present on all keys used in calls to [psa_key_derivation_input_key\(\)](#) for a key derivation operation, then it permits calling [psa_key_derivation_verify_bytes\(\)](#) or [psa_key_derivation_verify_key\(\)](#) at the end of the operation.

Definition at line 2753 of file `crypto_values.h`.

6.26.2.11 PSA_KEY_USAGE_VERIFY_HASH

```
#define PSA_KEY_USAGE_VERIFY_HASH ((psa_key_usage_t) 0x00002000)
```

Whether the key may be used to verify a message signature.

This flag allows the key to be used for a MAC verification operation or for an asymmetric signature verification operation, if otherwise permitted by the key's type and policy.

For a key pair, this concerns the public key.

Definition at line 2724 of file `crypto_values.h`.

6.26.2.12 PSA_KEY_USAGE_VERIFY_MESSAGE

```
#define PSA_KEY_USAGE_VERIFY_MESSAGE ((psa_key_usage_t) 0x00000800)
```

Whether the key may be used to verify a message.

This flag allows the key to be used for a MAC verification operation or for an asymmetric message signature verification operation, if otherwise permitted by the key's type and policy.

For a key pair, this concerns the public key.

Definition at line 2704 of file `crypto_values.h`.

6.26.2.13 PSA_KEY_USAGE_WRAP

```
#define PSA_KEY_USAGE_WRAP ((psa_key_usage_t) 0x00010000)
```

Permission to wrap another key with the key.

This flag is required to use the key in a key-wrapping operation.

The flag must be present on keys used with the following APIs:

- `psa_wrap_key()`

Definition at line 2764 of file `crypto_values.h`.

6.26.3 Typedef Documentation

6.26.3.1 `psa_key_usage_t`

```
typedef uint32_t psa_key_usage_t
```

Encoding of permitted usage on a key.

Values of this type are generally constructed as bitwise-ors of macros called `PSA_KEY_USAGE_XXX`.

Note

Values of this type are encoded in the persistent key store. Any changes to existing values will require bumping the storage format version and providing a translation when reading the old format.

Definition at line 316 of file `crypto_types.h`.

6.27 Key attributes

Macros

- `#define PSA_KEY_ATTRIBUTES_INIT`
- `#define PSA_PAKE_OPERATION_STAGE_SETUP 0`
- `#define PSA_PAKE_OPERATION_STAGE_COLLECT_INPUTS 1`
- `#define PSA_PAKE_OPERATION_STAGE_COMPUTATION 2`

Typedefs

- `typedef struct psa_key_attributes_s psa_key_attributes_t`

Functions

- `psa_status_t psa_get_key_attributes(mbedtls_svc_key_id_t key, psa_key_attributes_t *attributes)`
- `void psa_reset_key_attributes(psa_key_attributes_t *attributes)`

6.27.1 Detailed Description

6.27.2 Macro Definition Documentation

6.27.2.1 PSA_KEY_ATTRIBUTES_INIT

```
#define PSA_KEY_ATTRIBUTES_INIT
```

Value:

```
{ PSA_KEY_TYPE_NONE, 0,
  PSA_KEY_LIFETIME_VOLATILE,
  PSA_KEY_POLICY_INIT,
  MBEDTLS_SVC_KEY_ID_INIT }
```

This macro returns a suitable initializer for a key attribute structure of type `psa_key_attributes_t`.

Definition at line 329 of file `crypto_struct.h`.

6.27.2.2 PSA_PAKE_OPERATION_STAGE_COLLECT_INPUTS

```
#define PSA_PAKE_OPERATION_STAGE_COLLECT_INPUTS 1
```

Definition at line 273 of file `crypto_extra.h`.

6.27.2.3 PSA_PAKE_OPERATION_STAGE_COMPUTATION

```
#define PSA_PAKE_OPERATION_STAGE_COMPUTATION 2
```

Definition at line 274 of file `crypto_extra.h`.

6.27.2.4 PSA_PAKE_OPERATION_STAGE_SETUP

```
#define PSA_PAKE_OPERATION_STAGE_SETUP 0
```

PAKE operation stages.

Definition at line 272 of file `crypto_extra.h`.

6.27.3 Typedef Documentation

6.27.3.1 `psa_key_attributes_t`

```
typedef struct psa\_key\_attributes\_s psa\_key\_attributes\_t
```

The type of a structure containing key attributes.

This is an opaque structure that can represent the metadata of a key object. Metadata that can be stored in attributes includes:

- The location of the key in storage, indicated by its key identifier and its lifetime.
- The key's policy, comprising usage flags and a specification of the permitted algorithm(s).
- Information about the key itself: the key type and its size.
- Additional implementation-defined attributes.

The actual key material is not considered an attribute of a key. Key attributes do not contain information that is generally considered highly confidential.

An attribute structure works like a simple data structure where each function `psa_set_key_xxx` sets a field and the corresponding function `psa_get_key_xxx` retrieves the value of the corresponding field. However, a future version of the library may report values that are equivalent to the original one, but have a different encoding. Invalid values may be mapped to different, also invalid values.

An attribute structure may contain references to auxiliary resources, for example pointers to allocated memory or indirect references to pre-calculated values. In order to free such resources, the application must call [psa_reset_key_attributes\(\)](#). As an exception, calling [psa_reset_key_attributes\(\)](#) on an attribute structure is optional if the structure has only been modified by the following functions since it was initialized or last reset with [psa_reset_key_attributes\(\)](#):

- [psa_set_key_id\(\)](#)

- `psa_set_key_lifetime()`
- `psa_set_key_type()`
- `psa_set_key_bits()`
- `psa_set_key_usage_flags()`
- `psa_set_key_algorithm()`

Before calling any function on a key attribute structure, the application must initialize it by any of the following means:

- Set the structure to all-bits-zero, for example:

```
psa_key_attributes_t attributes;
memset(&attributes, 0, sizeof(attributes));
```
- Initialize the structure to logical zero values, for example:

```
psa_key_attributes_t attributes = {0};
```
- Initialize the structure to the initializer `PSA_KEY_ATTRIBUTES_INIT`, for example:

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
```
- Assign the result of the function `psa_key_attributes_init()` to the structure, for example:

```
psa_key_attributes_t attributes;
attributes = psa_key_attributes_init();
```

A freshly initialized attribute structure contains the following values:

- lifetime: `PSA_KEY_LIFETIME_VOLATILE`.
- key identifier: 0 (which is not a valid key identifier).
- type: 0 (meaning that the type is unspecified).
- key size: 0 (meaning that the size is unspecified).
- usage flags: 0 (which allows no usage except exporting a public key).
- algorithm: 0 (which allows no cryptographic usage, but allows exporting).

A typical sequence to create a key is as follows:

1. Create and initialize an attribute structure.
2. If the key is persistent, call `psa_set_key_id()`. Also call `psa_set_key_lifetime()` to place the key in a non-default location.
3. Set the key policy with `psa_set_key_usage_flags()` and `psa_set_key_algorithm()`.
4. Set the key type with `psa_set_key_type()`. Skip this step if copying an existing key with `psa_copy_key()`.
5. When generating a random key with `psa_generate_key()` or deriving a key with `psa_key_derivation_output_key()`, set the desired key size with `psa_set_key_bits()`.
6. Call a key creation function: `psa_import_key()`, `psa_generate_key()`, `psa_key_derivation_output_key()` or `psa_copy_key()`. This function reads the attribute structure, creates a key with these attributes, and outputs a key identifier to the newly created key.
7. The attribute structure is now no longer necessary. You may call `psa_reset_key_attributes()`, although this is optional with the workflow presented here because the attributes currently defined in this specification do not require any additional resources beyond the structure itself.

A typical sequence to query a key's attributes is as follows:

1. Call [psa_get_key_attributes\(\)](#).
2. Call `psa_get_key_xxx` functions to retrieve the attribute(s) that you are interested in.
3. Call [psa_reset_key_attributes\(\)](#) to free any resources that may be used by the attribute structure.

Once a key has been created, it is impossible to change its attributes.

Definition at line 425 of file `crypto_types.h`.

6.27.4 Function Documentation

6.27.4.1 `psa_get_key_attributes()`

```
psa_status_t psa_get_key_attributes (
    mbedtls_svc_key_id_t key,
    psa_key_attributes_t * attributes )
```

Retrieve the attributes of a key.

This function first resets the attribute structure as with [psa_reset_key_attributes\(\)](#). It then copies the attributes of the given key into the given attribute structure.

Note

This function may allocate memory or other resources. Once you have called this function on an attribute structure, you must call [psa_reset_key_attributes\(\)](#) to free these resources.

Parameters

in	<i>key</i>	Identifier of the key to query.
in, out	<i>attributes</i>	On success, the attributes of the key. On failure, equivalent to a freshly-initialized structure.

Return values

PSA_SUCCESS	
PSA_ERROR_INVALID_HANDLE	
PSA_ERROR_INSUFFICIENT_MEMORY	
PSA_ERROR_COMMUNICATION_FAILURE	
PSA_ERROR_CORRUPTION_DETECTED	
PSA_ERROR_STORAGE_FAILURE	
PSA_ERROR_DATA_CORRUPT	
PSA_ERROR_DATA_INVALID	

Return values

PSA_ERROR_BAD_STATE	The library has not been previously initialized by psa_crypto_init() . It is implementation-dependent whether a failure to initialize results in this error code.
-------------------------------------	---

Definition at line 141 of file `tfm_crypto_api.c`.

6.27.4.2 `psa_reset_key_attributes()`

```
void psa_reset_key_attributes (
    psa_key_attributes_t * attributes )
```

Reset a key attribute structure to a freshly initialized state.

You must initialize the attribute structure as described in the documentation of the type [psa_key_attributes_t](#) before calling this function. Once the structure has been initialized, you may call this function at any time.

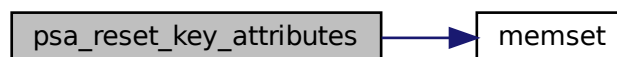
This function frees any auxiliary resources that the structure may contain.

Parameters

<code>in, out</code>	<code>attributes</code>	The attribute structure to reset.
----------------------	-------------------------	-----------------------------------

Definition at line 1705 of file `tfm_crypto_api.c`.

Here is the call graph for this function:



6.28 Key derivation

Macros

- `#define PSA_KEY_DERIVATION_INPUT_SECRET ((psa_key_derivation_step_t) 0x0101)`
- `#define PSA_KEY_DERIVATION_INPUT_PASSWORD ((psa_key_derivation_step_t) 0x0102)`
- `#define PSA_KEY_DERIVATION_INPUT_OTHER_SECRET ((psa_key_derivation_step_t) 0x0103)`
- `#define PSA_KEY_DERIVATION_INPUT_LABEL ((psa_key_derivation_step_t) 0x0201)`
- `#define PSA_KEY_DERIVATION_INPUT_SALT ((psa_key_derivation_step_t) 0x0202)`
- `#define PSA_KEY_DERIVATION_INPUT_INFO ((psa_key_derivation_step_t) 0x0203)`
- `#define PSA_KEY_DERIVATION_INPUT_SEED ((psa_key_derivation_step_t) 0x0204)`
- `#define PSA_KEY_DERIVATION_INPUT_COST ((psa_key_derivation_step_t) 0x0205)`
- `#define PSA_KEY_DERIVATION_INPUT_CONTEXT ((psa_key_derivation_step_t) 0x0206)`

Typedefs

- `typedef uint16_t psa_key_derivation_step_t`
Encoding of the step of a key derivation.
- `typedef struct psa_custom_key_parameters_s psa_custom_key_parameters_t`
Custom parameters for key generation or key derivation.

6.28.1 Detailed Description

6.28.2 Macro Definition Documentation

6.28.2.1 PSA_KEY_DERIVATION_INPUT_CONTEXT

```
#define PSA_KEY_DERIVATION_INPUT_CONTEXT ((psa_key_derivation_step_t) 0x0206)
```

A Context for key derivation.

This should be a direct input. It can also be a key of type [PSA_KEY_TYPE_RAW_DATA](#).

Definition at line 2871 of file `crypto_values.h`.

6.28.2.2 PSA_KEY_DERIVATION_INPUT_COST

```
#define PSA_KEY_DERIVATION_INPUT_COST ((psa_key_derivation_step_t) 0x0205)
```

A cost parameter for password hashing / key stretching.

This must be a direct input, passed to [psa_key_derivation_input_integer\(\)](#).

Definition at line 2864 of file `crypto_values.h`.

6.28.2.3 PSA_KEY_DERIVATION_INPUT_INFO

```
#define PSA_KEY_DERIVATION_INPUT_INFO ((psa_key_derivation_step_t) 0x0203)
```

An information string for key derivation.

This should be a direct input. It can also be a key of type [PSA_KEY_TYPE_RAW_DATA](#).

Definition at line 2851 of file `crypto_values.h`.

6.28.2.4 PSA_KEY_DERIVATION_INPUT_LABEL

```
#define PSA_KEY_DERIVATION_INPUT_LABEL ((psa_key_derivation_step_t) 0x0201)
```

A label for key derivation.

This should be a direct input. It can also be a key of type [PSA_KEY_TYPE_RAW_DATA](#).

Definition at line 2836 of file `crypto_values.h`.

6.28.2.5 PSA_KEY_DERIVATION_INPUT_OTHER_SECRET

```
#define PSA_KEY_DERIVATION_INPUT_OTHER_SECRET ((psa_key_derivation_step_t) 0x0103)
```

A high-entropy additional secret input for key derivation.

This is typically the shared secret resulting from a key agreement obtained via [psa_key_derivation_key_agreement\(\)](#). It may alternatively be a key of type `PSA_KEY_TYPE_DERIVE` passed to [psa_key_derivation_input_key\(\)](#), or a direct input passed to [psa_key_derivation_input_bytes\(\)](#).

Definition at line 2828 of file `crypto_values.h`.

6.28.2.6 PSA_KEY_DERIVATION_INPUT_PASSWORD

```
#define PSA_KEY_DERIVATION_INPUT_PASSWORD ((psa_key_derivation_step_t) 0x0102)
```

A low-entropy secret input for password hashing / key stretching.

This is usually a key of type [PSA_KEY_TYPE_PASSWORD](#) (passed to [psa_key_derivation_input_key\(\)](#)) or a direct input (passed to [psa_key_derivation_input_bytes\(\)](#)) that is a password or passphrase. It can also be high-entropy secret such as a key of type [PSA_KEY_TYPE_DERIVE](#) or the shared secret resulting from a key agreement.

The secret can also be a direct input (passed to [key_derivation_input_bytes\(\)](#)). In this case, the derivation operation may not be used to derive keys: the operation will only allow [psa_key_derivation_output_bytes\(\)](#), [psa_key_derivation_verify_bytes\(\)](#), or [psa_key_derivation_verify_key\(\)](#), but not [psa_key_derivation_output_key\(\)](#).

Definition at line 2819 of file `crypto_values.h`.

6.28.2.7 PSA_KEY_DERIVATION_INPUT_SALT

```
#define PSA_KEY_DERIVATION_INPUT_SALT ((psa_key_derivation_step_t) 0x0202)
```

A salt for key derivation.

This should be a direct input. It can also be a key of type [PSA_KEY_TYPE_RAW_DATA](#) or [PSA_KEY_TYPE_PEPPER](#).

Definition at line 2844 of file `crypto_values.h`.

6.28.2.8 PSA_KEY_DERIVATION_INPUT_SECRET

```
#define PSA_KEY_DERIVATION_INPUT_SECRET ((psa_key_derivation_step_t) 0x0101)
```

A secret input for key derivation.

This should be a key of type [PSA_KEY_TYPE_DERIVE](#) (passed to [psa_key_derivation_input_key\(\)](#)) or the shared secret resulting from a key agreement (obtained via [psa_key_derivation_key_agreement\(\)](#)).

The secret can also be a direct input (passed to [key_derivation_input_bytes\(\)](#)). In this case, the derivation operation may not be used to derive keys: the operation will only allow [psa_key_derivation_output_bytes\(\)](#), [psa_key_derivation_verify_bytes\(\)](#), or [psa_key_derivation_verify_key\(\)](#), but not [psa_key_derivation_output_key\(\)](#).

Definition at line 2801 of file `crypto_values.h`.

6.28.2.9 PSA_KEY_DERIVATION_INPUT_SEED

```
#define PSA_KEY_DERIVATION_INPUT_SEED ((psa_key_derivation_step_t) 0x0204)
```

A seed for key derivation.

This should be a direct input. It can also be a key of type [PSA_KEY_TYPE_RAW_DATA](#).

Definition at line 2858 of file `crypto_values.h`.

6.28.3 Typedef Documentation

6.28.3.1 `psa_custom_key_parameters_t`

```
typedef struct psa_custom_key_parameters_s psa_custom_key_parameters_t
```

Custom parameters for key generation or key derivation.

This is a structure type with at least the following field:

- `flags`: an unsigned integer type. 0 for the default production parameters.

Functions that take such a structure as input also take an associated input buffer `custom_data` of length `custom_data_length`.

The interpretation of this structure and the associated `custom_data` parameter depend on the type of the created key.

- `PSA_KEY_TYPE_RSA_KEY_PAIR`:
 - `flags`: must be 0.
 - `custom_data`: the public exponent, in little-endian order. This must be an odd integer and must not be 1. Implementations must support 65537, should support 3 and may support other values. When not using a driver, Mbed TLS supports values up to `INT_MAX`. If this is empty, the default value 65537 is used.
- Other key types: reserved for future use. `flags` must be 0.

Definition at line 462 of file `crypto_types.h`.

6.28.3.2 `psa_key_derivation_step_t`

```
typedef uint16_t psa_key_derivation_step_t
```

Encoding of the step of a key derivation.

Values of this type are generally constructed by macros called `PSA_KEY_DERIVATION_INPUT_XXX`.

Definition at line 438 of file `crypto_types.h`.

6.29 Helper macros

Macros

- #define `MBEDTLS_PSA_ALG_AEAD_EQUAL`(`aead_alg_1`, `aead_alg_2`)

6.29.1 Detailed Description

6.29.2 Macro Definition Documentation

6.29.2.1 MBEDTLS_PSA_ALG_AEAD_EQUAL

```
#define MBEDTLS_PSA_ALG_AEAD_EQUAL(  
    aead_alg_1,  
    aead_alg_2 )
```

Value:

```
((!(((aead_alg_1) ^ (aead_alg_2)) & \  
~(PSA_ALG_AEAD_TAG_LENGTH_MASK | PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG)))
```

Check if two AEAD algorithm identifiers refer to the same AEAD algorithm regardless of the tag length they encode.

Parameters

<i>aead_alg_1</i>	An AEAD algorithm identifier.
<i>aead_alg_2</i>	An AEAD algorithm identifier.

Returns

1 if both identifiers refer to the same AEAD algorithm, 0 otherwise. Unspecified if neither `aead_alg_1` nor `aead_alg_2` are a supported AEAD algorithm.

Definition at line 2893 of file `crypto_values.h`.

6.30 Interruptible operations

Macros

- `#define` [PSA_INTERRUPTIBLE_MAX_OPS_UNLIMITED](#) `UINT32_MAX`

6.30.1 Detailed Description

6.30.2 Macro Definition Documentation

6.30.2.1 `PSA_INTERRUPTIBLE_MAX_OPS_UNLIMITED`

```
#define PSA_INTERRUPTIBLE_MAX_OPS_UNLIMITED UINT32_MAX
```

Maximum value for use with [psa_interruptible_set_max_ops\(\)](#) to determine the maximum number of ops allowed to be executed by an interruptible function in a single call.

Definition at line 2909 of file `crypto_values.h`.

6.31 Set of functions implementing a thin shim

layer between the TF-M Crypto service frontend and the underlying library which implements the PSA Crypto APIs (i.e. mbed TLS)

Functions

- `psa_status_t tfm_crypto_aead_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`
This function acts as interface for the AEAD module.
- `psa_status_t tfm_crypto_asymmetric_sign_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`
This function acts as interface for the Asymmetric signing module.
- `psa_status_t tfm_crypto_asymmetric_encrypt_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`
This function acts as interface for the Asymmetric encryption module.
- `psa_status_t tfm_crypto_cipher_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`
This function acts as interface for the Cipher module.
- `psa_status_t tfm_crypto_hash_interface (psa_invec in_vec[], psa_outvec out_vec[])`
This function acts as interface for the Hash module.
- `psa_status_t tfm_crypto_key_derivation_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`
This function acts as interface for the Key derivation module.
- `psa_status_t tfm_crypto_key_management_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`
This function acts as interface for the Key management module.
- `psa_status_t tfm_crypto_mac_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`
This function acts as interface for the MAC module.
- `psa_status_t tfm_crypto_random_interface (psa_invec in_vec[], psa_outvec out_vec[])`
This function acts as interface for the Random module.

6.31.1 Detailed Description

layer between the TF-M Crypto service frontend and the underlying library which implements the PSA Crypto APIs (i.e. mbed TLS)

6.31.2 Function Documentation

6.31.2.1 tfm_crypto_aead_interface()

```
psa_status_t tfm_crypto_aead_interface (
    psa_invec in_vec[],
    psa_outvec out_vec[],
    struct tfm_crypto_key_id_s * encoded_key )
```

This function acts as interface for the AEAD module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters
in	<i>encoded_key</i>	Key encoded with partition_id and key_id

Returns

Return values as described in [psa_status_t](#)

Definition at line 259 of file crypto_aead.c.

6.31.2.2 tfm_crypto_asymmetric_encrypt_interface()

```
psa_status_t tfm_crypto_asymmetric_encrypt_interface (
    psa_invec in_vec[],
    psa_outvec out_vec[],
    struct tfm_crypto_key_id_s * encoded_key )
```

This function acts as interface for the Asymmetric encryption module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters
in	<i>encoded_key</i>	Key encoded with partition_id and key_id

Returns

Return values as described in [psa_status_t](#)

Definition at line 158 of file crypto_asymmetric.c.

6.31.2.3 tfm_crypto_asymmetric_sign_interface()

```
psa_status_t tfm_crypto_asymmetric_sign_interface (
    psa_invec in_vec[],
    psa_outvec out_vec[],
    struct tfm_crypto_key_id_s * encoded_key )
```

This function acts as interface for the Asymmetric signing module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters
in	<i>encoded_key</i>	Key encoded with partition_id and key_id

Returns

Return values as described in [psa_status_t](#)

Definition at line 92 of file `crypto_asymmetric.c`.

6.31.2.4 tfm_crypto_cipher_interface()

```
psa_status_t tfm_crypto_cipher_interface (
    psa_invec in_vec[],
    psa_outvec out_vec[],
    struct tfm_crypto_key_id_s * encoded_key )
```

This function acts as interface for the Cipher module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters
in	<i>encoded_key</i>	Key encoded with partition_id and key_id

Returns

Return values as described in [psa_status_t](#)

Definition at line 315 of file `crypto_cipher.c`.

6.31.2.5 tfm_crypto_hash_interface()

```
psa_status_t tfm_crypto_hash_interface (
    psa_invec in_vec[],
    psa_outvec out_vec[] )
```

This function acts as interface for the Hash module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters

Returns

Return values as described in [psa_status_t](#)

Definition at line 265 of file `crypto_hash.c`.

6.31.2.6 tfm_crypto_key_derivation_interface()

```
psa_status_t tfm_crypto_key_derivation_interface (
    psa_invec in_vec[],
    psa_outvec out_vec[],
    struct tfm_crypto_key_id_s * encoded_key )
```

This function acts as interface for the Key derivation module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters
in	<i>encoded_key</i>	Key encoded with partition_id and key_id

Returns

Return values as described in [psa_status_t](#)

Definition at line 196 of file crypto_key_derivation.c.

6.31.2.7 tfm_crypto_key_management_interface()

```
psa_status_t tfm_crypto_key_management_interface (
    psa_invec in_vec[],
    psa_outvec out_vec[],
    struct tfm_crypto_key_id_s * encoded_key )
```

This function acts as interface for the Key management module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters
in	<i>encoded_key</i>	Key encoded with partition_id and key_id

Returns

Return values as described in [psa_status_t](#)

Definition at line 173 of file crypto_key_management.c.

6.31.2.8 tfm_crypto_mac_interface()

```
psa_status_t tfm_crypto_mac_interface (
    psa_invec in_vec[],
```

```

    psa_outvec out_vec[],
    struct tfm_crypto_key_id_s * encoded_key )

```

This function acts as interface for the MAC module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters
in	<i>encoded_key</i>	Key encoded with partition_id and key_id

Returns

Return values as described in [psa_status_t](#)

Definition at line 185 of file crypto_mac.c.

6.31.2.9 tfm_crypto_random_interface()

```

psa_status_t tfm_crypto_random_interface (
    psa_invec in_vec[],
    psa_outvec out_vec[] )

```

This function acts as interface for the Random module.

Parameters

in	<i>in_vec</i>	Array of invec parameters
out	<i>out_vec</i>	Array of outvec parameters

Returns

Return values as described in [psa_status_t](#)

Definition at line 24 of file crypto_rng.c.

6.32 Function that implement allocation and deallocation of

contexts to be stored in the secure world for multipart operations

Functions

- [psa_status_t tfm_crypto_init_alloc](#) (void)
Initialise the Alloc module.
- [psa_status_t tfm_crypto_operation_alloc](#) (enum [tfm_crypto_operation_type](#) type, uint32_t *handle, void **ctx)
Allocate an operation context in the backend.
- [psa_status_t tfm_crypto_operation_release](#) (uint32_t *handle)
Release an operation context in the backend.
- [psa_status_t tfm_crypto_operation_lookup](#) (enum [tfm_crypto_operation_type](#) type, uint32_t handle, void **ctx)
Look up an operation context in the backend for the corresponding frontend operation.

6.32.1 Detailed Description

contexts to be stored in the secure world for multipart operations

6.32.2 Function Documentation

6.32.2.1 tfm_crypto_init_alloc()

```
psa_status_t tfm_crypto_init_alloc (
    void )
```

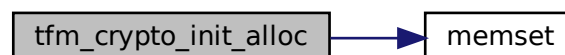
Initialise the Alloc module.

Returns

Return values as described in [psa_status_t](#)

Definition at line 74 of file crypto_alloc.c.

Here is the call graph for this function:



6.32.2.2 tfm_crypto_operation_alloc()

```
psa_status_t tfm_crypto_operation_alloc (
    enum tfm_crypto_operation_type type,
    uint32_t * handle,
    void ** ctx )
```

Allocate an operation context in the backend.

Parameters

in	<i>type</i>	Type of the operation context to allocate
out	<i>handle</i>	Pointer to hold the allocated handle
	<i>[out]</i>	ctx Double pointer to the corresponding context

Returns

Return values as described in [psa_status_t](#)

Definition at line 81 of file crypto_alloc.c.

6.32.2.3 tfm_crypto_operation_lookup()

```
psa_status_t tfm_crypto_operation_lookup (
    enum tfm_crypto_operation_type type,
    uint32_t handle,
    void ** ctx )
```

Look up an operation context in the backend for the corresponding frontend operation.

Parameters

in	<i>type</i>	Type of the operation context to look up
in	<i>handle</i>	Handle of the context to lookup
out	<i>ctx</i>	Double pointer to the corresponding context

Returns

Return values as described in [psa_status_t](#)

Definition at line 152 of file crypto_alloc.c.

6.32.2.4 tfm_crypto_operation_release()

```
psa_status_t tfm_crypto_operation_release (
    uint32_t * handle )
```

Release an operation context in the backend.

Parameters

<i>[in/out]</i>	handle Pointer to the handle of the context to release
-----------------	--

Returns

Return values as described in [psa_status_t](#)

Definition at line 119 of file crypto_alloc.c.

6.33 This group implements the partition initialisation

function to be called during TF-M boot, and the SFN servicing call to be called when a SFN request needs to be handled by the SPM. When built in IPC mode, the SPM directly provides a scheduling handler that deals with messages and calls the SFN entry point below just when needed, i.e. no need for a dedicated message handler needs to be implemented here

Functions

- [psa_status_t tfm_crypto_init](#) (void)
Initialise the service.
- [psa_status_t tfm_crypto_sfn](#) (const [psa_msg_t](#) *msg)

6.33.1 Detailed Description

function to be called during TF-M boot, and the SFN servicing call to be called when a SFN request needs to be handled by the SPM. When built in IPC mode, the SPM directly provides a scheduling handler that deals with messages and calls the SFN entry point below just when needed, i.e. no need for a dedicated message handler needs to be implemented here

6.33.2 Function Documentation

6.33.2.1 tfm_crypto_init()

```
psa_status_t tfm_crypto_init (  
    void )
```

Initialise the service.

Returns

Return values as described in [psa_status_t](#)

Definition at line 381 of file `crypto_init.c`.

6.33.2.2 tfm_crypto_sfn()

```
psa_status_t tfm_crypto_sfn (  
    const psa\_msg\_t * msg )
```

Definition at line 400 of file `crypto_init.c`.

6.34 Set of functions implementing the abstractions of the underlying cryptographic

library that implements the PSA Crypto APIs to provide the PSA Crypto core functionality to the TF-M Crypto service. Currently it supports only an TF-PSA-Crypto based abstraction.

Functions

- `tfm_crypto_library_key_id_t tfm_crypto_library_key_id_init (int32_t owner, psa_key_id_t key_id)`
Function used to initialise an object of `tfm_crypto_library_key_id_t` to a (owner, key_id) pair.
- `char * tfm_crypto_library_get_info (void)`
This function is used to retrieve a string describing the library used in the backend to provide information to the crypto service and the user.
- `psa_status_t tfm_crypto_core_library_init (void)`
This function is used to perform the necessary steps to initialise the underlying library that provides the implementation of the PSA Crypto core to the TF-M Crypto service.
- `void tfm_crypto_library_get_library_key_id_set_owner (int32_t owner, psa_key_attributes_t *attr)`
Allows to set the owner of a library key embedded into the key attributes structure.
- `psa_status_t mbedtls_psa_platform_get_built_in_key (mbedtls_svc_key_id_t key_id, psa_key_lifetime_t *lifetime, psa_drv_slot_number_t *slot_number)`
This function is required by TF-PSA-Crypto to enable support for platform builtin keys in the PSA Crypto core layer implemented by TF-PSA-Crypto. This function is not standardized by the API hence this layer directly provides the symbol required by the library.

6.34.1 Detailed Description

library that implements the PSA Crypto APIs to provide the PSA Crypto core functionality to the TF-M Crypto service. Currently it supports only an TF-PSA-Crypto based abstraction.

6.34.2 Function Documentation

6.34.2.1 mbedtls_psa_platform_get_built_in_key()

```
psa_status_t mbedtls_psa_platform_get_built_in_key (
    mbedtls_svc_key_id_t key_id,
    psa_key_lifetime_t * lifetime,
    psa_drv_slot_number_t * slot_number )
```

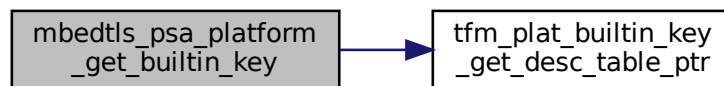
This function is required by TF-PSA-Crypto to enable support for platform builtin keys in the PSA Crypto core layer implemented by TF-PSA-Crypto. This function is not standardized by the API hence this layer directly provides the symbol required by the library.

Note

It maps builtin key IDs to cryptographic drivers and slots. The actual data is deferred to a platform function, as different platforms may have different key storage capabilities.

Definition at line 125 of file `crypto_library.c`.

Here is the call graph for this function:

**6.34.2.2 tfm_crypto_core_library_init()**

```
psa_status_t tfm_crypto_core_library_init (  
    void )
```

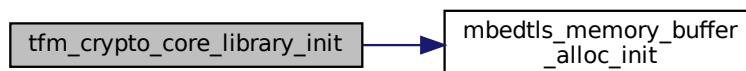
This function is used to perform the necessary steps to initialise the underlying library that provides the implementation of the PSA Crypto core to the TF-M Crypto service.

Returns

PSA_SUCCESS on successful initialisation

Definition at line 89 of file `crypto_library.c`.

Here is the call graph for this function:



6.34.2.3 tfm_crypto_library_get_info()

```
char* tfm_crypto_library_get_info (
    void )
```

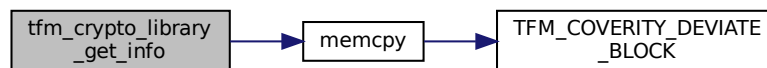
This function is used to retrieve a string describing the library used in the backend to provide information to the crypto service and the user.

Returns

A NULL terminated string describing the backend library

Definition at line 83 of file `crypto_library.c`.

Here is the call graph for this function:



6.34.2.4 tfm_crypto_library_get_library_key_id_set_owner()

```
void tfm_crypto_library_get_library_key_id_set_owner (
    int32_t owner,
    psa_key_attributes_t * attr )
```

Allows to set the owner of a library key embedded into the key attributes structure.

Parameters

in	<i>owner</i>	The owner value to be written into the key attributes structure
out	<i>attr</i>	Pointer to the key attributes into which we want to e

Definition at line 110 of file `crypto_library.c`.

6.34.2.5 tfm_crypto_library_key_id_init()

```
tfm_crypto_library_key_id_t tfm_crypto_library_key_id_init (
    int32_t owner,
    psa_key_id_t key_id )
```

Function used to initialise an object of `tfm_crypto_library_key_id_t` to a (owner, key_id) pair.

Parameters

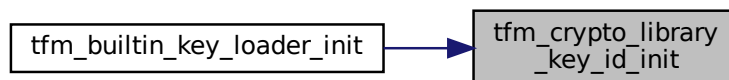
in	<i>owner</i>	Owner of the key
in	<i>key↔ _id</i>	key ID associated to the key of type psa_key_id_t

Returns

An object of type [tfm_crypto_library_key_id_t](#)

Definition at line 78 of file `crypto_library.c`.

Here is the caller graph for this function:



6.35 Tfm_builtin_key_loader

Functions

- [psa_status_t tfm_builtin_key_loader_init \(void\)](#)
This is the initialisation function for the tfm_builtin_key_loader driver, to be called from the PSA core initialisation subsystem. It discovers the keys available in the underlying hardware platform and loads them in memory visible to the PSA Crypto subsystem to be used to the normal APIs.
- [psa_status_t tfm_builtin_key_loader_get_key_buffer_size \(tfm_crypto_library_key_id_t key_id, size_t *len\)](#)
Returns the length of a key from the builtin driver.
- [psa_status_t tfm_builtin_key_loader_get_builtin_key \(psa_drv_slot_number_t slot_number, psa_key_attributes_t *attributes, uint8_t *key_buffer, size_t key_buffer_size, size_t *key_buffer_length\)](#)
Returns the attributes and key material of a key from the builtin driver to be used by the PSA Crypto core.

6.35.1 Detailed Description

6.35.2 Function Documentation

6.35.2.1 tfm_builtin_key_loader_get_builtin_key()

```
psa_status_t tfm_builtin_key_loader_get_builtin_key (
    psa_drv_slot_number_t slot_number,
    psa_key_attributes_t * attributes,
    uint8_t * key_buffer,
    size_t key_buffer_size,
    size_t * key_buffer_length )
```

Returns the attributes and key material of a key from the builtin driver to be used by the PSA Crypto core.

Note

This function is called by the psa crypto driver wrapper.

Parameters

in	<i>slot_number</i>	The slot of the key
out	<i>attributes</i>	The attributes of the key.
out	<i>key_buffer</i>	The buffer to output the key material into.
in	<i>key_buffer_size</i>	The size of the key material buffer.
out	<i>key_buffer_length</i>	The length of the key material returned.

Returns

Returns error code specified in [psa_status_t](#)

Definition at line 279 of file tfm_builtin_key_loader.c.

6.35.2.2 tfm_builtin_key_loader_get_key_buffer_size()

```
psa_status_t tfm_builtin_key_loader_get_key_buffer_size (
    mbedtls_svc_key_id_t key_id,
    size_t * len )
```

Returns the length of a key from the builtin driver.

Note

This function is called by the psa crypto driver wrapper.

Parameters

in	<i>key_id</i>	The ID of the key to return the length of. The type of this must match the expected type of the underlying library that provides the key management for the PSA Crypto core, and must support encoding the owner in addition to the key_id.
out	<i>len</i>	The length of the key.

Returns

Returns error code specified in [psa_status_t](#)

Definition at line 254 of file tfm_builtin_key_loader.c.

6.35.2.3 tfm_builtin_key_loader_init()

```
psa_status_t tfm_builtin_key_loader_init (
    void )
```

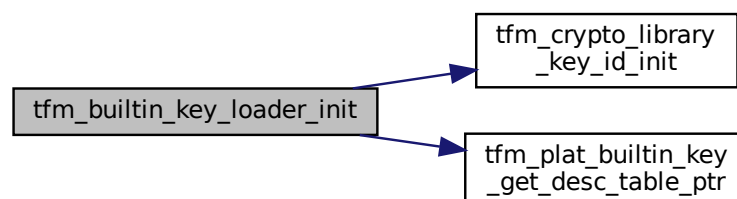
This is the initialisation function for the tfm_builtin_key_loader driver, to be called from the PSA core initialisation subsystem. It discovers the keys available in the underlying hardware platform and loads them in memory visible to the PSA Crypto subsystem to be used to the normal APIs.

Returns

Returns error code specified in [psa_status_t](#)

Definition at line 200 of file tfm_builtin_key_loader.c.

Here is the call graph for this function:



6.36 Asn1_module

Data Structures

- struct `MBEDTLS_ASN1_BUF`
- struct `MBEDTLS_ASN1_BITSTRING`
- struct `MBEDTLS_ASN1_SEQUENCE`
- struct `MBEDTLS_ASN1_NAMED_DATA`

Macros

- `#define MBEDTLS_OID_SIZE(x) (sizeof(x) - 1)`
- `#define MBEDTLS_OID_CMP(oid_str, oid_buf)`
- `#define MBEDTLS_OID_CMP_RAW(oid_str, oid_buf, oid_buf_len)`

Functions

- unsigned char `MBEDTLS_PRIVATE(next_merged)`

Variables

- int `tag`
- size_t `len`
- unsigned char * `p`
- size_t `len`
- unsigned char `unused_bits`
- unsigned char * `p`
- `MBEDTLS_ASN1_BUF` `buf`
- struct `MBEDTLS_ASN1_SEQUENCE` * `next`
- `MBEDTLS_ASN1_BUF` `oid`
- `MBEDTLS_ASN1_BUF` `val`
- struct `MBEDTLS_ASN1_NAMED_DATA` * `next`

ASN1 Error codes

These error codes are combined with other error codes for higher error granularity. e.g. X.509 and PKCS #7 error codes ASN1 is a standard to specify data structures.

- `#define MBEDTLS_ERR_ASN1_OUT_OF_DATA -0x0060`
- `#define MBEDTLS_ERR_ASN1_UNEXPECTED_TAG -0x0062`
- `#define MBEDTLS_ERR_ASN1_INVALID_LENGTH -0x0064`
- `#define MBEDTLS_ERR_ASN1_LENGTH_MISMATCH -0x0066`
- `#define MBEDTLS_ERR_ASN1_INVALID_DATA -0x0068`

DER constants

These constants comply with the DER encoded ASN.1 type tags. DER encoding uses hexadecimal representation. An example DER sequence is:

- 0x02 – tag indicating INTEGER
 - 0x01 – length in octets
 - 0x05 – value
-
- `#define MBEDTLS_ASN1_BOOLEAN 0x01`
 - `#define MBEDTLS_ASN1_INTEGER 0x02`
 - `#define MBEDTLS_ASN1_BIT_STRING 0x03`
 - `#define MBEDTLS_ASN1_OCTET_STRING 0x04`
 - `#define MBEDTLS_ASN1_NULL 0x05`
 - `#define MBEDTLS_ASN1_OID 0x06`
 - `#define MBEDTLS_ASN1_ENUMERATED 0x0A`
 - `#define MBEDTLS_ASN1_UTF8_STRING 0x0C`
 - `#define MBEDTLS_ASN1_SEQUENCE 0x10`
 - `#define MBEDTLS_ASN1_SET 0x11`
 - `#define MBEDTLS_ASN1_PRINTABLE_STRING 0x13`
 - `#define MBEDTLS_ASN1_T61_STRING 0x14`
 - `#define MBEDTLS_ASN1_IA5_STRING 0x16`
 - `#define MBEDTLS_ASN1_UTC_TIME 0x17`
 - `#define MBEDTLS_ASN1_GENERALIZED_TIME 0x18`
 - `#define MBEDTLS_ASN1_UNIVERSAL_STRING 0x1C`
 - `#define MBEDTLS_ASN1_BMP_STRING 0x1E`
 - `#define MBEDTLS_ASN1_PRIMITIVE 0x00`
 - `#define MBEDTLS_ASN1_CONSTRUCTED 0x20`
 - `#define MBEDTLS_ASN1_CONTEXT_SPECIFIC 0x80`
 - `#define MBEDTLS_ASN1_IS_STRING_TAG(tag)`
 - `#define MBEDTLS_ASN1_TAG_CLASS_MASK 0xC0`
 - `#define MBEDTLS_ASN1_TAG_PC_MASK 0x20`
 - `#define MBEDTLS_ASN1_TAG_VALUE_MASK 0x1F`

Functions to parse ASN.1 data structures

- `typedef struct mbedtls_asn1_buf mbedtls_asn1_buf`
- `typedef struct mbedtls_asn1_bitstring mbedtls_asn1_bitstring`
- `typedef struct mbedtls_asn1_sequence mbedtls_asn1_sequence`
- `typedef struct mbedtls_asn1_named_data mbedtls_asn1_named_data`

6.36.1 Detailed Description

6.36.2 Macro Definition Documentation

6.36.2.1 MBEDTLS_ASN1_BIT_STRING

```
#define MBEDTLS_ASN1_BIT_STRING 0x03
```

Definition at line 62 of file asn1.h.

6.36.2.2 MBEDTLS_ASN1_BMP_STRING

```
#define MBEDTLS_ASN1_BMP_STRING 0x1E
```

Definition at line 76 of file asn1.h.

6.36.2.3 MBEDTLS_ASN1_BOOLEAN

```
#define MBEDTLS_ASN1_BOOLEAN 0x01
```

Definition at line 60 of file asn1.h.

6.36.2.4 MBEDTLS_ASN1_CONSTRUCTED

```
#define MBEDTLS_ASN1_CONSTRUCTED 0x20
```

Definition at line 78 of file asn1.h.

6.36.2.5 MBEDTLS_ASN1_CONTEXT_SPECIFIC

```
#define MBEDTLS_ASN1_CONTEXT_SPECIFIC 0x80
```

Definition at line 79 of file asn1.h.

6.36.2.6 MBEDTLS_ASN1_ENUMERATED

```
#define MBEDTLS_ASN1_ENUMERATED 0x0A
```

Definition at line 66 of file asn1.h.

6.36.2.7 MBEDTLS_ASN1_GENERALIZED_TIME

```
#define MBEDTLS_ASN1_GENERALIZED_TIME 0x18
```

Definition at line 74 of file asn1.h.

6.36.2.8 MBEDTLS_ASN1_IA5_STRING

```
#define MBEDTLS_ASN1_IA5_STRING 0x16
```

Definition at line 72 of file asn1.h.

6.36.2.9 MBEDTLS_ASN1_INTEGER

```
#define MBEDTLS_ASN1_INTEGER 0x02
```

Definition at line 61 of file asn1.h.

6.36.2.10 MBEDTLS_ASN1_IS_STRING_TAG

```
#define MBEDTLS_ASN1_IS_STRING_TAG(  
    tag )
```

Value:

```
((unsigned int) (tag) < 32u && (  
    ((1u < (tag)) & ((1u < MBEDTLS_ASN1_BMP_STRING) |  
        (1u < MBEDTLS_ASN1_UTF8_STRING) |  
        (1u < MBEDTLS_ASN1_T61_STRING) |  
        (1u < MBEDTLS_ASN1_IA5_STRING) |  
        (1u < MBEDTLS_ASN1_UNIVERSAL_STRING) |  
        (1u < MBEDTLS_ASN1_PRINTABLE_STRING))) != 0))
```

Definition at line 83 of file asn1.h.

6.36.2.11 MBEDTLS_ASN1_NULL

```
#define MBEDTLS_ASN1_NULL 0x05
```

Definition at line 64 of file asn1.h.

6.36.2.12 MBEDTLS_ASN1_OCTET_STRING

```
#define MBEDTLS_ASN1_OCTET_STRING 0x04
```

Definition at line 63 of file asn1.h.

6.36.2.13 MBEDTLS_ASN1_OID

```
#define MBEDTLS_ASN1_OID 0x06
```

Definition at line 65 of file asn1.h.

6.36.2.14 MBEDTLS_ASN1_PRIMITIVE

```
#define MBEDTLS_ASN1_PRIMITIVE 0x00
```

Definition at line 77 of file asn1.h.

6.36.2.15 MBEDTLS_ASN1_PRINTABLE_STRING

```
#define MBEDTLS_ASN1_PRINTABLE_STRING 0x13
```

Definition at line 70 of file asn1.h.

6.36.2.16 MBEDTLS_ASN1_SEQUENCE

```
#define MBEDTLS_ASN1_SEQUENCE 0x10
```

Definition at line 68 of file asn1.h.

6.36.2.17 MBEDTLS_ASN1_SET

```
#define MBEDTLS_ASN1_SET 0x11
```

Definition at line 69 of file asn1.h.

6.36.2.18 MBEDTLS_ASN1_T61_STRING

```
#define MBEDTLS_ASN1_T61_STRING 0x14
```

Definition at line 71 of file asn1.h.

6.36.2.19 MBEDTLS_ASN1_TAG_CLASS_MASK

```
#define MBEDTLS_ASN1_TAG_CLASS_MASK 0xC0
```

Definition at line 102 of file asn1.h.

6.36.2.20 MBEDTLS_ASN1_TAG_PC_MASK

```
#define MBEDTLS_ASN1_TAG_PC_MASK 0x20
```

Definition at line 103 of file asn1.h.

6.36.2.21 MBEDTLS_ASN1_TAG_VALUE_MASK

```
#define MBEDTLS_ASN1_TAG_VALUE_MASK 0x1F
```

Definition at line 104 of file asn1.h.

6.36.2.22 MBEDTLS_ASN1_UNIVERSAL_STRING

```
#define MBEDTLS_ASN1_UNIVERSAL_STRING 0x1C
```

Definition at line 75 of file asn1.h.

6.36.2.23 MBEDTLS_ASN1_UTC_TIME

```
#define MBEDTLS_ASN1_UTC_TIME 0x17
```

Definition at line 73 of file asn1.h.

6.36.2.24 MBEDTLS_ASN1_UTF8_STRING

```
#define MBEDTLS_ASN1_UTF8_STRING 0x0C
```

Definition at line 67 of file asn1.h.

6.36.2.25 MBEDTLS_ERR_ASN1_INVALID_DATA

```
#define MBEDTLS_ERR_ASN1_INVALID_DATA -0x0068
```

Data is invalid.

Definition at line 46 of file asn1.h.

6.36.2.26 MBEDTLS_ERR_ASN1_INVALID_LENGTH

```
#define MBEDTLS_ERR_ASN1_INVALID_LENGTH -0x0064
```

Error when trying to determine the length or invalid length.

Definition at line 42 of file asn1.h.

6.36.2.27 MBEDTLS_ERR_ASN1_LENGTH_MISMATCH

```
#define MBEDTLS_ERR_ASN1_LENGTH_MISMATCH -0x0066
```

Actual length differs from expected length.

Definition at line 44 of file asn1.h.

6.36.2.28 MBEDTLS_ERR_ASN1_OUT_OF_DATA

```
#define MBEDTLS_ERR_ASN1_OUT_OF_DATA -0x0060
```

Out of data when parsing an ASN1 data structure.

Definition at line 38 of file asn1.h.

6.36.2.29 MBEDTLS_ERR_ASN1_UNEXPECTED_TAG

```
#define MBEDTLS_ERR_ASN1_UNEXPECTED_TAG -0x0062
```

ASN1 tag was of an unexpected value.

Definition at line 40 of file asn1.h.

6.36.2.30 MBEDTLS_OID_CMP

```
#define MBEDTLS_OID_CMP(  
    oid_str,  
    oid_buf )
```

Value:

```
((MBEDTLS_OID_SIZE(oid_str) != (oid_buf)->len) ||  
  memcmp((oid_str), (oid_buf)->p, (oid_buf)->len) != 0) \
```

Compares an [mbedtls_asn1_buf](#) structure to a reference OID.

Only works for 'defined' oid_str values (MBEDTLS_OID_HMAC_SHA1), you cannot use a 'unsigned char *oid' here!

Definition at line 117 of file asn1.h.

6.36.2.31 MBEDTLS_OID_CMP_RAW

```
#define MBEDTLS_OID_CMP_RAW(  
    oid_str,  
    oid_buf,  
    oid_buf_len )
```

Value:

```
((MBEDTLS_OID_SIZE(oid_str) != (oid_buf_len)) ||  
  memcmp((oid_str), (oid_buf), (oid_buf_len)) != 0) \
```

Definition at line 121 of file asn1.h.

6.36.2.32 MBEDTLS_OID_SIZE

```
#define MBEDTLS_OID_SIZE(  
    x ) (sizeof(x) - 1)
```

Returns the size of the binary string, without the trailing \0

Definition at line 109 of file asn1.h.

6.36.3 Typedef Documentation

6.36.3.1 mbedtls_asn1_bitstring

```
typedef struct mbedtls_asn1_bitstring mbedtls_asn1_bitstring
```

Container for ASN1 bit strings.

6.36.3.2 mbedtls_asn1_buf

```
typedef struct mbedtls_asn1_buf mbedtls_asn1_buf
```

Type-length-value structure that allows for ASN1 using DER.

6.36.3.3 mbedtls_asn1_named_data

```
typedef struct mbedtls_asn1_named_data mbedtls_asn1_named_data
```

Container for a sequence or list of 'named' ASN.1 data items

6.36.3.4 mbedtls_asn1_sequence

```
typedef struct mbedtls_asn1_sequence mbedtls_asn1_sequence
```

Container for a sequence of ASN.1 items

6.36.4 Function Documentation

6.36.4.1 MBEDTLS_PRIVATE()

```
unsigned char MBEDTLS_PRIVATE (
    next_merged )
```

Merge next item into the current one?

This field exists for the sake of Mbed TLS's X.509 certificate parsing code and may change in future versions of the library.

6.36.5 Variable Documentation

6.36.5.1 buf

```
MBEDTLS_ASN1_BUF buf
```

Buffer containing the given ASN.1 item.

Definition at line 158 of file asn1.h.

6.36.5.2 len [1/2]

```
size_t len
```

ASN1 length, in octets.

Definition at line 139 of file asn1.h.

6.36.5.3 len [2/2]

```
size_t len
```

ASN1 length, in octets.

Definition at line 148 of file asn1.h.

6.36.5.4 next [1/2]

```
struct mbedtls_asn1_sequence* next
```

The next entry in the sequence.

The details of memory management for sequences are not documented and may change in future versions. Set this field to `NULL` when initializing a structure, and do not modify it except via Mbed TLS library functions.

Definition at line 167 of file asn1.h.

6.36.5.5 next [2/2]

```
struct mbedtls_asn1_named_data* next
```

The next entry in the sequence.

The details of memory management for named data sequences are not documented and may change in future versions. Set this field to `NULL` when initializing a structure, and do not modify it except via Mbed TLS library functions.

Definition at line 185 of file asn1.h.

6.36.5.6 oid

`MBEDTLS_ASN1_BUF oid`

The object identifier.

Definition at line 175 of file `asn1.h`.

6.36.5.7 p [1/2]

`unsigned char* p`

ASN1 data, e.g. in ASCII.

Definition at line 140 of file `asn1.h`.

6.36.5.8 p [2/2]

`unsigned char* p`

Raw ASN1 data for the bit string

Definition at line 150 of file `asn1.h`.

6.36.5.9 tag

`int tag`

ASN1 type, e.g. `MBEDTLS_ASN1_UTF8_STRING`.

Definition at line 138 of file `asn1.h`.

6.36.5.10 unused_bits

`unsigned char unused_bits`

Number of unused bits at the end of the string

Definition at line 149 of file `asn1.h`.

6.36.5.11 val

`MBEDTLS_ASN1_BUF val`

The named value.

Definition at line 176 of file `asn1.h`.

Chapter 7

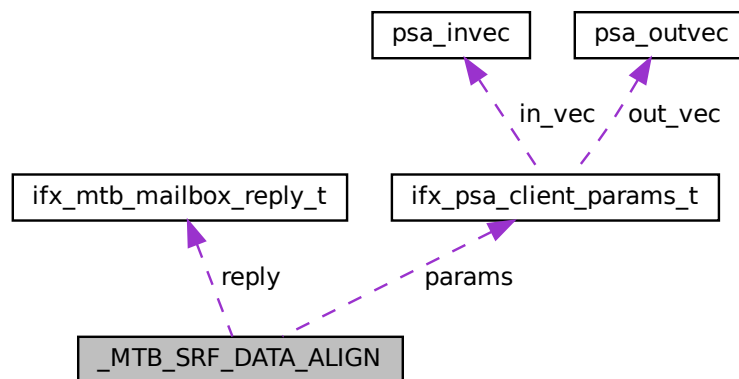
Data Structure Documentation

7.1 _MTB_SRF_DATA_ALIGN Struct Reference

MTB SRF IPC request structure suitable for TFM needs. This structure overrides default `mtb_srf_ipc_packet_t` SRF structure.

```
#include <platform/ext/target/infineon/common/interface/include/mtb_srf_↔  
ipc_custom_packet.h>
```

Collaboration diagram for `_MTB_SRF_DATA_ALIGN`:



Data Fields

- `uint32_t` [call_type](#)
- `ifx_psa_client_params_t` [params](#)
- `ifx_mtb_mailbox_reply_t` [reply](#)
- `uint16_t` [semaphore_idx](#)

7.1.1 Detailed Description

MTB SRF IPC request structure suitable for TFM needs. This structure overrides default `mtb_srf_ipc_packet_t` SRF structure.

This structure is designed to be allocated in shared memory for inter-process communication (IPC). All members are stored as values (not pointers) to ensure the entire request is self-contained and can be safely copied into and out of shared memory. Parameters required for the PSA client call are copied directly into this structure, and the structure itself is allocated in the shared memory region. This design ensures that all data needed for the request is accessible to both communicating parties without relying on process-specific pointers/memory.

Definition at line 32 of file `mtb_srf_ipc_custom_packet.h`.

7.1.2 Field Documentation

7.1.2.1 `call_type`

```
uint32_t call_type
```

Definition at line 34 of file `mtb_srf_ipc_custom_packet.h`.

7.1.2.2 `params`

```
ifx_psa_client_params_t params
```

Definition at line 35 of file `mtb_srf_ipc_custom_packet.h`.

7.1.2.3 `reply`

```
ifx_mtb_mailbox_reply_t reply
```

Definition at line 37 of file `mtb_srf_ipc_custom_packet.h`.

7.1.2.4 `semaphore_idx`

```
uint16_t semaphore_idx
```

Definition at line 40 of file `mtb_srf_ipc_custom_packet.h`.

The documentation for this struct was generated from the following file:

- `platform/ext/target/infineon/common/interface/include/mtb_srf_ipc_custom_packet.h`

7.2 `asset_desc_t` Struct Reference

Asset descriptor.

```
#include <secure_fw/spm/include/load/asset_defs.h>
```

Data Fields

- union {
 - struct {
 - `uintptr_t` `start`
 - `uintptr_t` `limit`
 - `mem`
 - struct {
 - `uintptr_t` `dev_ref`
 - `uintptr_t` `reserved`
 - `dev`
- enum `assets_attribute_t` `attr`

7.2.1 Detailed Description

Asset descriptor.

Definition at line 30 of file `asset_defs.h`.

7.2.2 Field Documentation

7.2.2.1 `"@15`

```
union { ... }
```

7.2.2.2 `attr`

```
enum assets_attribute_t attr
```

Assets attributes

Definition at line 41 of file `asset_defs.h`.

7.2.2.3 dev

```
struct { ... } dev
```

7.2.2.4 dev_ref

```
uintptr_t dev_ref
```

< Device asset type

Definition at line 37 of file `asset_defs.h`.

7.2.2.5 limit

```
uintptr_t limit
```

Definition at line 34 of file `asset_defs.h`.

7.2.2.6 mem

```
struct { ... } mem
```

7.2.2.7 reserved

```
uintptr_t reserved
```

Definition at line 38 of file `asset_defs.h`.

7.2.2.8 start

```
uintptr_t start
```

< Memory-based asset type

Definition at line 33 of file `asset_defs.h`.

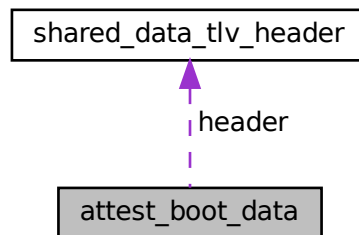
The documentation for this struct was generated from the following file:

- `secure_fw/spm/include/load/asset\_defs.h`

7.3 attest_boot_data Struct Reference

Contains the received boot status information from bootloader.

Collaboration diagram for attest_boot_data:



Data Fields

- struct [shared_data_tlv_header](#) header
- uint8_t [data](#) [512]

7.3.1 Detailed Description

Contains the received boot status information from bootloader.

This is a redefinition of [tfm_boot_data](#) to allocate the appropriate, service dependent size of boot_data.

Definition at line 35 of file attest_boot_data.c.

7.3.2 Field Documentation

7.3.2.1 data

```
uint8_t data[512]
```

Definition at line 37 of file attest_boot_data.c.

7.3.2.2 header

```
struct shared_data_tlv_header header
```

Definition at line 36 of file attest_boot_data.c.

The documentation for this struct was generated from the following file:

- [secure_fw/partitions/initial_attestation/attest_boot_data.c](#)

7.4 attest_token_encode_ctx Struct Reference

```
#include <secure_fw/partitions/initial_attestation/attest_token.h>
```

Data Fields

- QCBOREncodeContext [cbor_enc_ctx](#)
- int32_t [key_select](#)
- struct t_cose_sign1_sign_ctx [signer_ctx](#)

7.4.1 Detailed Description

The context for creating an attestation token. The caller of `attest_token_encode` must create one of these and pass it to the functions here. It is small enough that it can go on the stack. It is most of the memory needed to create a token except the output buffer and any memory requirements for the cryptographic operations.

The structure is opaque for the caller.

This is roughly $148 + 4 + 32 = 184$ bytes

Definition at line 109 of file `attest_token.h`.

7.4.2 Field Documentation

7.4.2.1 cbor_enc_ctx

```
QCBOREncodeContext cbor_enc_ctx
```

Definition at line 111 of file `attest_token.h`.

7.4.2.2 key_select

```
int32_t key_select
```

Definition at line 112 of file attest_token.h.

7.4.2.3 signer_ctx

```
struct t_cose_sign1_sign_ctx signer_ctx
```

Definition at line 116 of file attest_token.h.

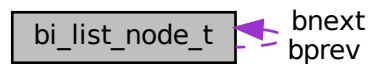
The documentation for this struct was generated from the following file:

- secure_fw/partitions/initial_attestation/[attest_token.h](#)

7.5 bi_list_node_t Struct Reference

```
#include <secure_fw/spm/include/lists.h>
```

Collaboration diagram for bi_list_node_t:



Data Fields

- struct [bi_list_node_t](#) * `bprev`
- struct [bi_list_node_t](#) * `bnext`

7.5.1 Detailed Description

Definition at line 12 of file lists.h.

7.5.2 Field Documentation

7.5.2.1 bnext

```
struct bi_list_node_t* bnext
```

Definition at line 14 of file lists.h.

7.5.2.2 bprev

```
struct bi_list_node_t* bprev
```

Definition at line 13 of file lists.h.

The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/lists.h](#)

7.6 boot_data_access_policy Struct Reference

Defines the access policy of secure partitions to data items in shared data area (between bootloader and runtime firmware).

Data Fields

- `int32_t` [partition_id](#)
- `uint32_t` [major_type](#)

7.6.1 Detailed Description

Defines the access policy of secure partitions to data items in shared data area (between bootloader and runtime firmware).

Definition at line 56 of file tfm_boot_data.c.

7.6.2 Field Documentation

7.6.2.1 major_type

```
uint32_t major_type
```

Definition at line 58 of file tfm_boot_data.c.

7.6.2.2 partition_id

```
int32_t partition_id
```

Definition at line 57 of file tfm_boot_data.c.

The documentation for this struct was generated from the following file:

- [secure_fw/spm/core/tfm_boot_data.c](#)

7.7 client_id_region_t Struct Reference

Data Fields

- int32_t [base](#)
- int32_t [limit](#)

7.7.1 Detailed Description

Definition at line 19 of file tfm_spm_ns_ctx.c.

7.7.2 Field Documentation

7.7.2.1 base

```
int32_t base
```

Definition at line 20 of file tfm_spm_ns_ctx.c.

7.7.2.2 limit

```
int32_t limit
```

Definition at line 21 of file tfm_spm_ns_ctx.c.

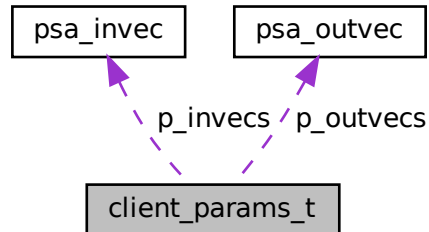
The documentation for this struct was generated from the following file:

- [secure_fw/spm/ns_client_ext/tfm_spm_ns_ctx.c](#)

7.8 client_params_t Struct Reference

```
#include <secure_fw/spm/include/ffm/mailbox_agent_api.h>
```

Collaboration diagram for client_params_t:



Data Fields

- `int32_t ns_client_id_stateless`
- `psa_invec * p_invecs`
- `psa_outvec * p_outvecs`

7.8.1 Detailed Description

Definition at line 18 of file mailbox_agent_api.h.

7.8.2 Field Documentation

7.8.2.1 ns_client_id_stateless

```
int32_t ns_client_id_stateless
```

Definition at line 19 of file mailbox_agent_api.h.

7.8.2.2 p_invecs

```
psa_invec* p_invecs
```

Definition at line 20 of file mailbox_agent_api.h.

7.8.2.3 p_outvecs

```
psa_outvec* p_outvecs
```

Definition at line 21 of file mailbox_agent_api.h.

The documentation for this struct was generated from the following file:

- secure_fw/spm/include/ffm/[mailbox_agent_api.h](#)

7.9 composite_addr_t Union Reference

```
#include <secure_fw/include/crt_impl_private.h>
```

Data Fields

- uintptr_t [uint_addr](#)
- uint8_t * [p_byte](#)
- uint32_t * [p_word](#)

7.9.1 Detailed Description

Definition at line 16 of file crt_impl_private.h.

7.9.2 Field Documentation

7.9.2.1 p_byte

```
uint8_t* p_byte
```

Definition at line 18 of file crt_impl_private.h.

7.9.2.2 p_word

```
uint32_t* p_word
```

Definition at line 19 of file crt_impl_private.h.

7.9.2.3 uint_addr

```
uintptr_t uint_addr
```

Definition at line 17 of file `crt_impl_private.h`.

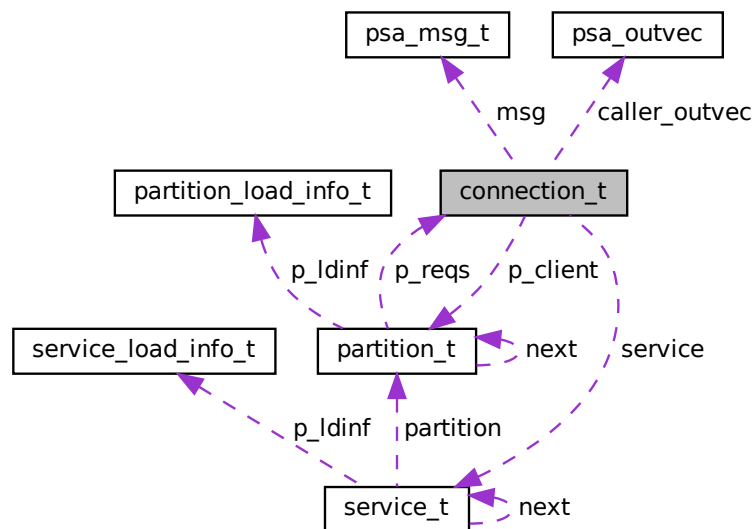
The documentation for this union was generated from the following file:

- `secure_fw/include/crt_impl_private.h`

7.10 connection_t Struct Reference

```
#include <secure_fw/spm/core/spm.h>
```

Collaboration diagram for `connection_t`:



Data Fields

- enum `connection_status` `status`
- struct `partition_t` * `p_client`
- const struct `service_t` * `service`
- `psa_msg_t` `msg`
- const `void` * `invec_base` [`PSA_MAX_IOVEC`]
- `size_t` `invec_accessed` [`PSA_MAX_IOVEC`]
- `void` * `outvec_base` [`PSA_MAX_IOVEC`]
- `size_t` `outvec_written` [`PSA_MAX_IOVEC`]
- `psa_outvec` * `caller_outvec`

7.10.1 Detailed Description

Definition at line 76 of file spm.h.

7.10.2 Field Documentation

7.10.2.1 caller_outvec

```
psa_outvec* caller_outvec
```

Definition at line 85 of file spm.h.

7.10.2.2 invec_accessed

```
size_t invec_accessed[PSA_MAX_IOVEC]
```

Definition at line 82 of file spm.h.

7.10.2.3 invec_base

```
const void* invec_base[PSA_MAX_IOVEC]
```

Definition at line 81 of file spm.h.

7.10.2.4 msg

```
psa_msg_t msg
```

Definition at line 80 of file spm.h.

7.10.2.5 outvec_base

```
void* outvec_base[PSA_MAX_IOVEC]
```

Definition at line 83 of file spm.h.

7.10.2.6 outvec_written

```
size_t outvec_written[PSA_MAX_IOVEC]
```

Definition at line 84 of file spm.h.

7.10.2.7 p_client

```
struct partition_t* p_client
```

Definition at line 78 of file spm.h.

7.10.2.8 service

```
const struct service_t* service
```

Definition at line 79 of file spm.h.

7.10.2.9 status

```
enum connection_status status
```

Definition at line 77 of file spm.h.

The documentation for this struct was generated from the following file:

- [secure_fw/spm/core/spm.h](#)

7.11 context_ctrl_t Struct Reference

```
#include <secure_fw/spm/include/tfm_arch.h>
```

Data Fields

- uint32_t [sp](#)
- uint32_t [exc_ret](#)
- uint32_t [sp_limit](#)
- uint32_t [sp_base](#)

7.11.1 Detailed Description

Definition at line 136 of file tfm_arch.h.

7.11.2 Field Documentation

7.11.2.1 exc_ret

```
uint32_t exc_ret
```

Definition at line 141 of file tfm_arch.h.

7.11.2.2 sp

```
uint32_t sp
```

Definition at line 137 of file tfm_arch.h.

7.11.2.3 sp_base

```
uint32_t sp_base
```

Definition at line 146 of file tfm_arch.h.

7.11.2.4 sp_limit

```
uint32_t sp_limit
```

Definition at line 145 of file tfm_arch.h.

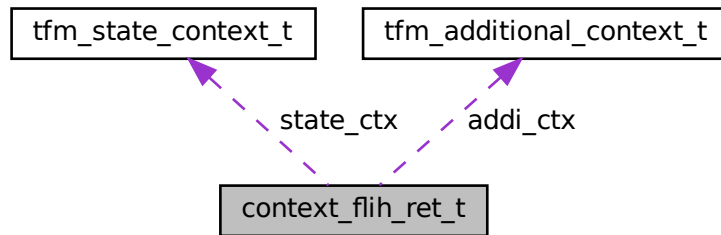
The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/tfm_arch.h](#)

7.12 context_flih_ret_t Struct Reference

```
#include <secure_fw/spm/include/tfm_arch.h>
```

Collaboration diagram for context_flih_ret_t:



Data Fields

- uint64_t [stack_seal](#)
- struct [tfm_additional_context_t](#) [addi_ctx](#)
- uint32_t [exc_return](#)
- uint32_t [dummy](#)
- uint32_t [psp](#)
- uint32_t [psplim](#)
- struct [tfm_state_context_t](#) [state_ctx](#)

7.12.1 Detailed Description

Definition at line 153 of file `tfm_arch.h`.

7.12.2 Field Documentation

7.12.2.1 addi_ctx

```
struct tfm\_additional\_context\_t addi\_ctx
```

Definition at line 155 of file `tfm_arch.h`.

7.12.2.2 dummy

```
uint32_t dummy
```

Definition at line 160 of file tfm_arch.h.

7.12.2.3 exc_return

```
uint32_t exc_return
```

Definition at line 159 of file tfm_arch.h.

7.12.2.4 psp

```
uint32_t psp
```

Definition at line 161 of file tfm_arch.h.

7.12.2.5 psplim

```
uint32_t psplim
```

Definition at line 162 of file tfm_arch.h.

7.12.2.6 stack_seal

```
uint64_t stack_seal
```

Definition at line 154 of file tfm_arch.h.

7.12.2.7 state_ctx

```
struct tfm\_state\_context\_t state_ctx
```

Definition at line 163 of file tfm_arch.h.

The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/tfm_arch.h](#)

7.13 critical_section_t Struct Reference

```
#include <secure_fw/spm/include/critical_section.h>
```

Data Fields

- uint32_t [state](#)

7.13.1 Detailed Description

Definition at line 14 of file critical_section.h.

7.13.2 Field Documentation

7.13.2.1 state

```
uint32_t state
```

Definition at line 15 of file critical_section.h.

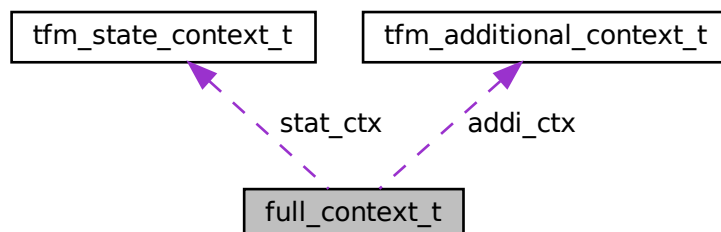
The documentation for this struct was generated from the following file:

- secure_fw/spm/include/[critical_section.h](#)

7.14 full_context_t Struct Reference

```
#include <secure_fw/spm/include/tfm_arch.h>
```

Collaboration diagram for full_context_t:



Data Fields

- struct [tfm_additional_context_t](#) addi_ctx
- struct [tfm_state_context_t](#) stat_ctx

7.14.1 Detailed Description

Definition at line 123 of file tfm_arch.h.

7.14.2 Field Documentation

7.14.2.1 addi_ctx

```
struct tfm\_additional\_context\_t addi_ctx
```

Definition at line 124 of file tfm_arch.h.

7.14.2.2 stat_ctx

```
struct tfm\_state\_context\_t stat_ctx
```

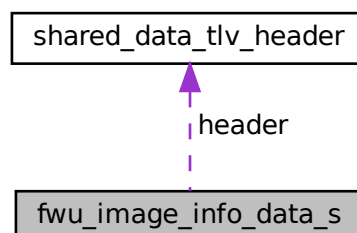
Definition at line 128 of file tfm_arch.h.

The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/tfm_arch.h](#)

7.15 fwu_image_info_data_s Struct Reference

Collaboration diagram for fwu_image_info_data_s:



Data Fields

- struct [shared_data_tlv_header](#) header
- uint8_t [data](#)[(MCUBOOT_IMAGE_NUMBER*(sizeof(struct image_version)+[SHARED_DATA_ENTRY_HEADER_SIZE](#)))]

7.15.1 Detailed Description

Definition at line 36 of file `tfm_mcuboot_fw.c`.

7.15.2 Field Documentation

7.15.2.1 data

```
uint8_t data[(MCUBOOT_IMAGE_NUMBER*(sizeof(struct image_version)+ SHARED\_DATA\_ENTRY\_HEADER\_SIZE))]
```

Definition at line 38 of file `tfm_mcuboot_fw.c`.

7.15.2.2 header

```
struct shared\_data\_tlv\_header header
```

Definition at line 37 of file `tfm_mcuboot_fw.c`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/firmware_update/bootloader/mcuboot/tfm\_mcuboot\_fw.c`

7.16 ifx_driver_flash_obj_t Struct Reference

Structure containing the FLASH driver instance parameters.

```
#include <platform/ext/target/infineon/common/drivers/flash/flash/ifx_↵  
driver_flash.h>
```

Data Fields

- uint32_t [start_address](#)
- ARM_FLASH_INFO [flash_info](#)

7.16.1 Detailed Description

Structure containing the FLASH driver instance parameters.

FLASH driver uses [start_address](#), `flash_info.sector_size` and `flash_info.sector_count` to define a working region that is handled by active instance. Any access outside of this region is invalid and FLASH driver returns `ARM_DRIVE↵R_ERROR_PARAMETER` in such cases.

[ifx_driver_flash](#) interface expects address to be relative to [start_address](#).

Note

This structure can be a constant.

Definition at line 38 of file `ifx_driver_flash.h`.

7.16.2 Field Documentation

7.16.2.1 flash_info

```
ARM_FLASH_INFO flash_info
```

`flash_info.sector_size` - sector size for region. It should be multiple of FLASH row size `CY_FLASH_SIZEOF_ROW`.

Definition at line 45 of file `ifx_driver_flash.h`.

7.16.2.2 start_address

```
uint32_t start_address
```

Region start address

Definition at line 39 of file `ifx_driver_flash.h`.

The documentation for this struct was generated from the following file:

- `platform/ext/target/infineon/common/drivers/flash/flash/ifx\_driver\_flash.h`

7.17 ifx_driver_rram_obj_t Struct Reference

Structure containing the RRAM driver instance parameters.

```
#include <platform/ext/target/infineon/common/drivers/flash/rram/ifx_driver↵_rram.h>
```

Data Fields

- RRAMC_Type * [rram_base](#)
- uint32_t [start_address](#)
- ARM_FLASH_INFO [flash_info](#)

7.17.1 Detailed Description

Structure containing the RRAM driver instance parameters.

RRAM flash driver uses [start_address](#), `flash_info.sector_size` and `flash_info.sector_count` to define a working region that is handled by active instance. Any access outside of this region is invalid and RRAM flash driver returns `ARM_DRIVER_ERROR_PARAMETER` in such cases.

[ifx_driver_rram](#) interface expects address to be relative to [start_address](#).

Note

This structure can be a constant.

Definition at line 46 of file `ifx_driver_rram.h`.

7.17.2 Field Documentation

7.17.2.1 flash_info

```
ARM_FLASH_INFO flash_info
```

`flash_info.sector_size` - sector size for region. It should be multiple of RRAM block size `CY_RRAM_BLOCK_SIZE_BYTES`.

Definition at line 54 of file `ifx_driver_rram.h`.

7.17.2.2 rram_base

```
RRAMC_Type* rram_base
```

The pointer to the RRAMC instance.

Definition at line 47 of file `ifx_driver_rram.h`.

7.17.2.3 start_address

```
uint32_t start_address
```

Region start address

Definition at line 48 of file ifx_driver_rram.h.

The documentation for this struct was generated from the following file:

- platform/ext/target/infineon/common/drivers/flash/rram/[ifx_driver_rram.h](#)

7.18 ifx_driver_smif_mem_t Struct Reference

Structure containing the SMIF driver memory configuration.

```
#include <platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_↵_smif.h>
```

Data Fields

- SMIF_Type * [smif_base](#)
- const cy_stc_smif_block_config_t * [block_config](#)
- const cy_stc_smif_mem_config_t * [mem_config](#)
- const cy_stc_smif_mem_config_t * [mem_config_dual_quad_pair](#)
- cy_stc_smif_context_t * [smif_context](#)

7.18.1 Detailed Description

Structure containing the SMIF driver memory configuration.

SMIF driver instance represent memory region with own boundaries. Such instance can share common SMIF memory configuration and context.

Definition at line 27 of file ifx_driver_smif.h.

7.18.2 Field Documentation

7.18.2.1 block_config

```
const cy_stc_smif_block_config_t* block_config
```

SMIF block config for Cy_SMIF_MemInit

Definition at line 29 of file ifx_driver_smif.h.

7.18.2.2 mem_config

```
const cy_stc_smif_mem_config_t* mem_config
```

SMIF memory configuration

Definition at line 30 of file ifx_driver_smif.h.

7.18.2.3 mem_config_dual_quad_pair

```
const cy_stc_smif_mem_config_t* mem_config_dual_quad_pair
```

SMIF dual-quad pair memory config

Definition at line 31 of file ifx_driver_smif.h.

7.18.2.4 smif_base

```
SMIF_Type* smif_base
```

Holds the base address of the SMIF block registers

Definition at line 28 of file ifx_driver_smif.h.

7.18.2.5 smif_context

```
cy_stc_smif_context_t* smif_context
```

SMIF driver context

Definition at line 32 of file ifx_driver_smif.h.

The documentation for this struct was generated from the following file:

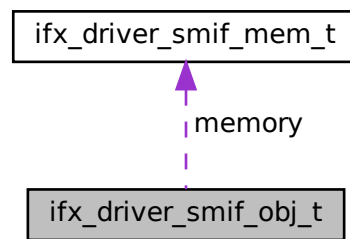
- [platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_smif.h](#)

7.19 ifx_driver_smif_obj_t Struct Reference

Structure containing the SMIF driver instance data.

```
#include <platform/ext/target/infineon/common/drivers/flash/smif/ifx_driver_↵
_smif.h>
```

Collaboration diagram for ifx_driver_smif_obj_t:



Data Fields

- const [ifx_driver_smif_mem_t](#) * [memory](#)
- uint32_t [offset](#)
- uint32_t [size](#)
- struct _ARM_FLASH_INFO * [flash_info](#)

7.19.1 Detailed Description

Structure containing the SMIF driver instance data.

SMIF flash driver uses [offset](#), [size](#) to define a working region that is handled by active instance. Any access outside of this region is invalid and SMIF flash driver returns ARM_DRIVER_ERROR_PARAMETER in such cases.

ifx_driver_smif interface expects address to be relative to [offset](#).

Note

This structure is modified by ifx_driver_smif.Initialize, so it should not be a constant.

Definition at line 47 of file ifx_driver_smif.h.

7.19.2 Field Documentation

7.19.2.1 flash_info

```
struct _ARM_FLASH_INFO* flash_info
```

Definition at line 55 of file ifx_driver_smif.h.

7.19.2.2 memory

```
const ifx_driver_smif_mem_t* memory
```

Memory configuration must be provided by BSP

Definition at line 48 of file ifx_driver_smif.h.

7.19.2.3 offset

```
uint32_t offset
```

Region offset relative to SMIF memory

Definition at line 50 of file ifx_driver_smif.h.

7.19.2.4 size

```
uint32_t size
```

Region size

Definition at line 51 of file ifx_driver_smif.h.

The documentation for this struct was generated from the following file:

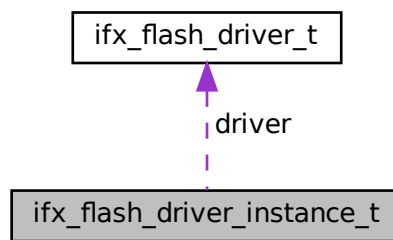
- platform/ext/target/infineon/common/drivers/flash/smif/[ifx_driver_smif.h](#)

7.20 ifx_flash_driver_instance_t Struct Reference

Flash driver instance.

```
#include <platform/ext/target/infineon/common/drivers/flash/ifx_flash_driver_api.h>
```

Collaboration diagram for ifx_flash_driver_instance_t:



Data Fields

- [ifx_flash_driver_handler_t](#) handler
- [ifx_flash_driver_t](#) * driver

7.20.1 Detailed Description

Flash driver instance.

You can create a separate driver instance for same flash chip. But each instance can handle different areas of flash memory. This allows to provide isolation of data on driver level too.

Definition at line 76 of file ifx_flash_driver_api.h.

7.20.2 Field Documentation

7.20.2.1 driver

```
ifx\_flash\_driver\_t* driver
```

Definition at line 78 of file ifx_flash_driver_api.h.

7.20.2.2 handler

`ifx_flash_driver_handler_t` handler

Definition at line 77 of file `ifx_flash_driver_api.h`.

The documentation for this struct was generated from the following file:

- `platform/ext/target/infineon/common/drivers/flash/ifx_flash_driver_api.h`

7.21 ifx_flash_driver_t Struct Reference

Access structure of the Flash Driver.

```
#include <platform/ext/target/infineon/common/drivers/flash/ifx_flash_driver_api.h>
```

Data Fields

- `ARM_DRIVER_VERSION`(* [GetVersion](#))(ifx_flash_driver_handler_t handler)
- `ARM_FLASH_CAPABILITIES`(* [GetCapabilities](#))(ifx_flash_driver_handler_t handler)
- `int32_t`(* [Initialize](#))(ifx_flash_driver_handler_t handler, ifx_flash_driver_event_t cb_event)
- `int32_t`(* [Uninitialize](#))(ifx_flash_driver_handler_t handler)
- `int32_t`(* [PowerControl](#))(ifx_flash_driver_handler_t handler, ARM_POWER_STATE state)
- `int32_t`(* [ReadData](#))(ifx_flash_driver_handler_t handler, uint32_t addr, void *data, uint32_t cnt)
- `int32_t`(* [ProgramData](#))(ifx_flash_driver_handler_t handler, uint32_t addr, const void *data, uint32_t cnt)
- `int32_t`(* [EraseSector](#))(ifx_flash_driver_handler_t handler, uint32_t addr)
- `int32_t`(* [EraseChip](#))(ifx_flash_driver_handler_t handler)
- `struct_ARM_FLASH_STATUS`(* [GetStatus](#))(ifx_flash_driver_handler_t handler)
- `ARM_FLASH_INFO` *([GetInfo](#))(ifx_flash_driver_handler_t handler)

7.21.1 Detailed Description

Access structure of the Flash Driver.

Interface is similar to CMSIS flash driver interface (`ARM_DRIVER_FLASH`) except that there is an additional driver instance handler (see [ifx_flash_driver_instance_t::handler](#)) which is specific to driver implementation.

Definition at line 39 of file `ifx_flash_driver_api.h`.

7.21.2 Field Documentation

7.21.2.1 EraseChip

```
int32_t (* EraseChip(ifx_flash_driver_handler_t handler)
```

Definition at line 62 of file ifx_flash_driver_api.h.

7.21.2.2 EraseSector

```
int32_t (* EraseSector(ifx_flash_driver_handler_t handler, uint32_t addr)
```

Definition at line 59 of file ifx_flash_driver_api.h.

7.21.2.3 GetCapabilities

```
ARM_FLASH_CAPABILITIES (* GetCapabilities(ifx_flash_driver_handler_t handler)
```

Definition at line 43 of file ifx_flash_driver_api.h.

7.21.2.4 GetInfo

```
ARM_FLASH_INFO* (* GetInfo(ifx_flash_driver_handler_t handler)
```

Definition at line 66 of file ifx_flash_driver_api.h.

7.21.2.5 GetStatus

```
struct _ARM_FLASH_STATUS (* GetStatus(ifx_flash_driver_handler_t handler)
```

Definition at line 64 of file ifx_flash_driver_api.h.

7.21.2.6 GetVersion

```
ARM_DRIVER_VERSION (* GetVersion(ifx_flash_driver_handler_t handler)
```

Definition at line 41 of file ifx_flash_driver_api.h.

7.21.2.7 Initialize

```
int32_t(* Initialize(ifx\_flash\_driver\_handler\_t handler, ifx\_flash\_driver\_event\_t cb_event)
```

Definition at line 45 of file [ifx_flash_driver_api.h](#).

7.21.2.8 PowerControl

```
int32_t(* PowerControl(ifx\_flash\_driver\_handler\_t handler, ARM_POWER_STATE state)
```

Definition at line 50 of file [ifx_flash_driver_api.h](#).

7.21.2.9 ProgramData

```
int32_t(* ProgramData(ifx\_flash\_driver\_handler\_t handler, uint32_t addr, const void *data,  
uint32_t cnt)
```

Definition at line 56 of file [ifx_flash_driver_api.h](#).

7.21.2.10 ReadData

```
int32_t(* ReadData(ifx\_flash\_driver\_handler\_t handler, uint32_t addr, void *data, uint32_t cnt)
```

Definition at line 53 of file [ifx_flash_driver_api.h](#).

7.21.2.11 Uninitialize

```
int32_t(* Uninitialize(ifx\_flash\_driver\_handler\_t handler)
```

Definition at line 48 of file [ifx_flash_driver_api.h](#).

The documentation for this struct was generated from the following file:

- [platform/ext/target/infineon/common/drivers/flash/ifx_flash_driver_api.h](#)

7.22 ifx_mpc_cfg_t Struct Reference

Data Fields

- uint16_t [mem_offset](#)
- uint16_t [mem_size](#)
- uint32_t [attr](#)

7.22.1 Detailed Description

Definition at line 41 of file protection_mpc_sw_policy.c.

7.22.2 Field Documentation

7.22.2.1 attr

uint32_t attr

Definition at line 44 of file protection_mpc_sw_policy.c.

7.22.2.2 mem_offset

uint16_t mem_offset

Definition at line 42 of file protection_mpc_sw_policy.c.

7.22.2.3 mem_size

uint16_t mem_size

Definition at line 43 of file protection_mpc_sw_policy.c.

The documentation for this struct was generated from the following file:

- [platform/ext/target/infineon/common/spe/protection/protection_mpc_sw_policy.c](#)

7.23 ifx_mpc_ext_cache_cfg_info_t Struct Reference

Data Fields

- uint32_t [attr](#)
- uint32_t [pc_cache_mask](#)
- uint32_t [first_block_idx](#)
- uint32_t [last_block_idx](#)

7.23.1 Detailed Description

Definition at line 28 of file protection_mpc_sert.c.

7.23.2 Field Documentation

7.23.2.1 attr

```
uint32_t attr
```

Bit field with MPC cached attributes

Definition at line 30 of file protection_mpc_sert.c.

7.23.2.2 first_block_idx

```
uint32_t first_block_idx
```

First block index in cached attributes

Definition at line 34 of file protection_mpc_sert.c.

7.23.2.3 last_block_idx

```
uint32_t last_block_idx
```

Last block index in cached attributes

Definition at line 36 of file protection_mpc_sert.c.

7.23.2.4 pc_cache_mask

```
uint32_t pc_cache_mask
```

Cache-bit mask derived from pc_apply_mask for the selected PCs.

Definition at line 32 of file protection_mpc_sert.c.

The documentation for this struct was generated from the following file:

- platform/ext/target/infineon/common/spe/protection/[protection_mpc_sert.c](#)

7.24 ifx_mpc_numbered_mmio_config_t Struct Reference

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵  
mpc_hw_mpc.h>
```

Data Fields

- [ifx_pc_mask_t pc_mask](#)
- [ifx_pc_mask_t ns_mask](#)

7.24.1 Detailed Description

Definition at line 29 of file protection_mpc_hw_mpc.h.

7.24.2 Field Documentation

7.24.2.1 ns_mask

```
ifx_pc_mask_t ns_mask
```

NS bit mask for each PC

Definition at line 33 of file protection_mpc_hw_mpc.h.

7.24.2.2 pc_mask

```
ifx_pc_mask_t pc_mask
```

Mask that define what PCs must be modified. 1 - must be applied new value, 0 - must be left old value no matter what it was

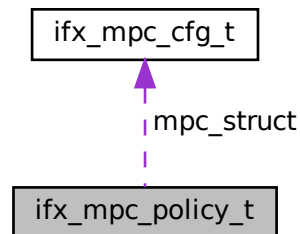
Definition at line 30 of file protection_mpc_hw_mpc.h.

The documentation for this struct was generated from the following file:

- platform/ext/target/infineon/common/spe/protection/[protection_mpc_hw_mpc.h](#)

7.25 ifx_mpc_policy_t Struct Reference

Collaboration diagram for ifx_mpc_policy_t:



Data Fields

- `uint8_t n_ram_mpc`
- `uint8_t n_flash_mpc`
- `uint8_t unused` [2]
- `ifx_mpc_cfg_t mpc_struct` [32]

7.25.1 Detailed Description

Definition at line 50 of file `protection_mpc_sw_policy.c`.

7.25.2 Field Documentation

7.25.2.1 mpc_struct

```
ifx_mpc_cfg_t mpc_struct[32]
```

Definition at line 54 of file `protection_mpc_sw_policy.c`.

7.25.2.2 n_flash_mpc

```
uint8_t n_flash_mpc
```

Definition at line 52 of file `protection_mpc_sw_policy.c`.

7.25.2.3 n_ram_mpc

```
uint8_t n_ram_mpc
```

Definition at line 51 of file protection_mpc_sw_policy.c.

7.25.2.4 unused

```
uint8_t unused[2]
```

Definition at line 53 of file protection_mpc_sw_policy.c.

The documentation for this struct was generated from the following file:

- platform/ext/target/infineon/common/spe/protection/[protection_mpc_sw_policy.c](#)

7.26 ifx_mpc_raw_region_config_t Struct Reference

Configuration structure for MPC raw region.

```
#include <platform/ext/target/infineon/common/spe/protection/protection_  
mpc_hw_mpc.h>
```

Data Fields

- MPC_Type * [mpc_base](#)
- cy_en_mpc_size_t [mpc_block_size](#)
- uint32_t [offset](#)
- uint32_t [size](#)
- ifx_pc_mask_t [pc_apply_mask](#)
- ifx_pc_mask_t [ns_mask](#)
- ifx_pc_mask_t [r_mask](#)
- ifx_pc_mask_t [w_mask](#)

7.26.1 Detailed Description

Configuration structure for MPC raw region.

Definition at line 47 of file protection_mpc_hw_mpc.h.

7.26.2 Field Documentation

7.26.2.1 mpc_base

`MPC_Type* mpc_base`

Base address of the MPC controller

Definition at line 48 of file `protection_mpc_hw_mpc.h`.

7.26.2.2 mpc_block_size

`cy_en_mpc_size_t mpc_block_size`

Size of the MPC block

Definition at line 49 of file `protection_mpc_hw_mpc.h`.

7.26.2.3 ns_mask

`ifx_pc_mask_t ns_mask`

NS bit mask for each PC

Definition at line 53 of file `protection_mpc_hw_mpc.h`.

7.26.2.4 offset

`uint32_t offset`

Definition at line 50 of file `protection_mpc_hw_mpc.h`.

7.26.2.5 pc_apply_mask

`ifx_pc_mask_t pc_apply_mask`

PCs to update/verify.

Definition at line 52 of file `protection_mpc_hw_mpc.h`.

7.26.2.6 r_mask

`ifx_pc_mask_t r_mask`

R bit mask for each PC

Definition at line 54 of file `protection_mpc_hw_mpc.h`.

7.26.2.7 size

`uint32_t size`

Definition at line 51 of file `protection_mpc_hw_mpc.h`.

7.26.2.8 w_mask

`ifx_pc_mask_t w_mask`

W bit mask for each PC

Definition at line 55 of file `protection_mpc_hw_mpc.h`.

The documentation for this struct was generated from the following file:

- [platform/ext/target/infineon/common/spe/protection/protection_mpc_hw_mpc.h](#)

7.27 ifx_mpc_region_config_t Struct Reference

```
#include <platform/ext/target/infineon/common/spe/protection/protection_  
mpc_hw_mpc.h>
```

Data Fields

- `uint32_t` [address](#)
- `uint32_t` [size](#)
- `ifx_pc_mask_t` [ns_mask](#)
- `ifx_pc_mask_t` [r_mask](#)
- `ifx_pc_mask_t` [w_mask](#)

7.27.1 Detailed Description

Definition at line 36 of file `protection_mpc_hw_mpc.h`.

7.27.2 Field Documentation

7.27.2.1 address

```
uint32_t address
```

Definition at line 37 of file `protection_mpc_hw_mpc.h`.

7.27.2.2 ns_mask

```
ifx_pc_mask_t ns_mask
```

NS bit mask for each PC

Definition at line 39 of file `protection_mpc_hw_mpc.h`.

7.27.2.3 r_mask

```
ifx_pc_mask_t r_mask
```

R bit mask for each PC

Definition at line 40 of file `protection_mpc_hw_mpc.h`.

7.27.2.4 size

```
uint32_t size
```

Definition at line 38 of file `protection_mpc_hw_mpc.h`.

7.27.2.5 w_mask

```
ifx_pc_mask_t w_mask
```

W bit mask for each PC

Definition at line 41 of file `protection_mpc_hw_mpc.h`.

The documentation for this struct was generated from the following file:

- `platform/ext/target/infineon/common/spe/protection/protection_mpc_hw_mpc.h`

7.28 ifx_mpc_sert_mem_t Struct Reference

Descriptor of an external memory whose MPC is handled by SE RT Services.

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵  
mpc_sert.h>
```

Data Fields

- const ifx_memory_config_t * [mem](#)
- uint32_t * [attr_cache](#)
- uint32_t [block_count](#)

7.28.1 Detailed Description

Descriptor of an external memory whose MPC is handled by SE RT Services.

Platforms provide one entry per external MPC in [ifx_mpc_sert_memories](#).

Definition at line 74 of file protection_mpc_sert.h.

7.28.2 Field Documentation

7.28.2.1 attr_cache

```
uint32_t* attr_cache
```

Cache storage, [block_count](#) words

Definition at line 76 of file protection_mpc_sert.h.

7.28.2.2 block_count

```
uint32_t block_count
```

Number of MPC blocks (equal to [attr_cache](#) words)

Definition at line 77 of file protection_mpc_sert.h.

7.28.2.3 mem

```
const ifx_memory_config_t* mem
```

MPC identity, block size, base and size

Definition at line 75 of file protection_mpc_sert.h.

The documentation for this struct was generated from the following file:

- [platform/ext/target/infineon/common/spe/protection/protection_mpc_sert.h](#)

7.29 ifx_mtb_mailbox_reply_t Struct Reference

```
#include <platform/ext/target/infineon/common/interface/include/ifx_mtb_mailbox/ifx_mtb_mailbox.h>
```

Data Fields

- `int32_t` [return_val](#)
- `size_t` [out_vec_len](#) [[PSA_MAX_IOVEC](#)]

7.29.1 Detailed Description

Definition at line 73 of file ifx_mtb_mailbox.h.

7.29.2 Field Documentation

7.29.2.1 out_vec_len

```
size_t out_vec_len[PSA\_MAX\_IOVEC]
```

Definition at line 75 of file ifx_mtb_mailbox.h.

7.29.2.2 return_val

```
int32_t return_val
```

Definition at line 74 of file ifx_mtb_mailbox.h.

The documentation for this struct was generated from the following file:

- [platform/ext/target/infineon/common/interface/include/ifx_mtb_mailbox/ifx_mtb_mailbox.h](#)

7.30 ifx_nv_counters Struct Reference

Struct representing the NV counter data in flash.

Data Fields

- uint32_t [checksum](#)
- uint32_t [init_value](#)
- union {
 - uint32_t [counters](#) [(((uint32_t) [PLAT_NV_COUNTER_MAX](#))]
 - uint8_t [bytes](#) [[IFX_ROUND_UP_TO_MULTIPLE](#)((2U * [sizeof](#)(uint32_t)) + (((uint32_t) [PLAT_NV_COUNTER_MAX](#)) * [sizeof](#)(uint32_t))), [IFX_TFM_NV_COUNTERS_PROGRAM_UNIT](#) - (2U * [sizeof](#)(uint32_t))]

 } [nv_cnt](#)

7.30.1 Detailed Description

Struct representing the NV counter data in flash.

Definition at line 63 of file `nv_counters_flash.c`.

7.30.2 Field Documentation

7.30.2.1 bytes

```
uint8_t bytes[IFX_ROUND_UP_TO_MULTIPLE((2U * sizeof(uint32_t)) + ((uint32_t) PLAT_NV_COUNTER_MAX)
* sizeof(uint32_t)), IFX_TFM_NV_COUNTERS_PROGRAM_UNIT) - (2U * sizeof(uint32_t))]
```

Definition at line 70 of file `nv_counters_flash.c`.

7.30.2.2 checksum

```
uint32_t checksum
```

Definition at line 64 of file `nv_counters_flash.c`.

7.30.2.3 counters

```
uint32_t counters[(uint32_t) PLAT_NV_COUNTER_MAX]
```

Array of NV counters

Definition at line 67 of file `nv_counters_flash.c`.

7.30.2.4 init_value

```
uint32_t init_value
```

Definition at line 65 of file `nv_counters_flash.c`.

7.30.2.5 nv_cnt

```
union { ... } nv_cnt
```

The documentation for this struct was generated from the following file:

- `platform/ext/target/infineon/common/spe/services/platform/nv_counters_flash.c`

7.31 ifx_nv_init_done Union Reference

Data Fields

- `uint32_t flag`
- `uint8_t block[IFX_ROUND_UP_TO_MULTIPLE(sizeof(uint32_t), IFX_TFM_NV_COUNTERS_PROGRAM_UNIT)]`

7.31.1 Detailed Description

Definition at line 78 of file `nv_counters_flash.c`.

7.31.2 Field Documentation

7.31.2.1 block

```
uint8_t block[IFX_ROUND_UP_TO_MULTIPLE(sizeof(uint32_t), IFX_TFM_NV_COUNTERS_PROGRAM_UNIT)]
```

Definition at line 80 of file `nv_counters_flash.c`.

7.31.2.2 flag

```
uint32_t flag
```

Definition at line 79 of file `nv_counters_flash.c`.

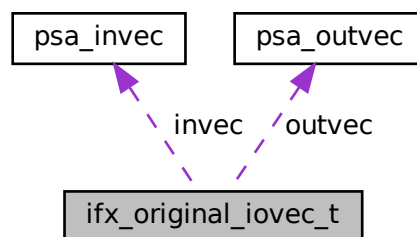
The documentation for this union was generated from the following file:

- [platform/ext/target/infineon/common/spe/services/platform/nv_counters_flash.c](#)

7.32 ifx_original_iovec_t Struct Reference

```
#include <platform/ext/target/infineon/common/spe/svc/platform_svc_api.h>
```

Collaboration diagram for `ifx_original_iovec_t`:



Data Fields

- [psa_invec invec](#) [[PSA_MAX_IOVEC](#)]
- [psa_outvec outvec](#) [[PSA_MAX_IOVEC](#)]
- [size_t invec_count](#)
- [size_t outvec_count](#)

7.32.1 Detailed Description

Structure to hold original input and output vectors and their counts

Definition at line 18 of file `platform_svc_api.h`.

7.32.2 Field Documentation

7.32.2.1 invec

```
psa_invec invec[PSA_MAX_IOVEC]
```

Definition at line 19 of file platform_svc_api.h.

7.32.2.2 invec_count

```
size_t invec_count
```

Definition at line 21 of file platform_svc_api.h.

7.32.2.3 outvec

```
psa_outvec outvec[PSA_MAX_IOVEC]
```

Definition at line 20 of file platform_svc_api.h.

7.32.2.4 outvec_count

```
size_t outvec_count
```

Definition at line 22 of file platform_svc_api.h.

The documentation for this struct was generated from the following file:

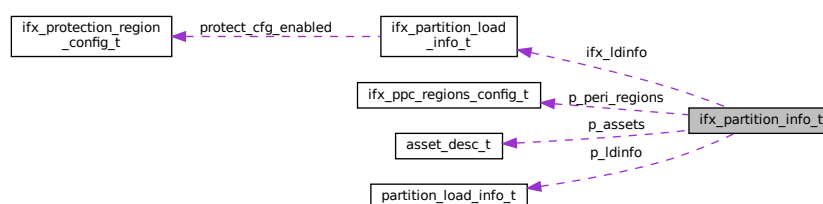
- platform/ext/target/infineon/common/spe/svc/platform_svc_api.h

7.33 ifx_partition_info_t Struct Reference

Structure describing partition info.

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵  
types.h>
```

Collaboration diagram for ifx_partition_info_t:



Data Fields

- const struct [partition_load_info_t](#) * [p_ldinfo](#)
- const [ifx_partition_load_info_t](#) * [ifx_ldinfo](#)
- uintptr_t [ifx_domain](#)
- const struct [asset_desc_t](#) * [p_assets](#)
Platform specific assets which can't be added to manifest.
- uint32_t [asset_cnt](#)
Number of items in [p_assets](#).
- const [ifx_ppc_regions_config_t](#) * [p_peri_regions](#)
Partition peripheral assets added by platform or Edge Protect Configurator.
- uint32_t [peri_region_count](#)
Number of items in [p_peri_regions](#).

7.33.1 Detailed Description

Structure describing partition info.

Pointer to this structure is used to provide boundary for isolation HAL.

Definition at line 252 of file `protection_types.h`.

7.33.2 Field Documentation

7.33.2.1 `asset_cnt`

```
uint32_t asset_cnt
```

Number of items in [p_assets](#).

Definition at line 262 of file `protection_types.h`.

7.33.2.2 `ifx_domain`

```
uintptr_t ifx_domain
```

Definition at line 258 of file `protection_types.h`.

7.33.2.3 `ifx_ldinfo`

```
const ifx_partition_load_info_t* ifx_ldinfo
```

Definition at line 256 of file `protection_types.h`.

7.33.2.4 p_assets

```
const struct asset\_desc\_t* p_assets
```

Platform specific assets which can't be added to manifest.

Definition at line 260 of file [protection_types.h](#).

7.33.2.5 p_ldinfo

```
const struct partition\_load\_info\_t* p_ldinfo
```

Definition at line 254 of file [protection_types.h](#).

7.33.2.6 p_peri_regions

```
const ifx\_ppc\_regions\_config\_t* p_peri_regions
```

Partition peripheral assets added by platform or Edge Protect Configurator.

Definition at line 268 of file [protection_types.h](#).

7.33.2.7 peri_region_count

```
uint32_t peri_region_count
```

Number of items in [p_peri_regions](#).

Definition at line 270 of file [protection_types.h](#).

The documentation for this struct was generated from the following file:

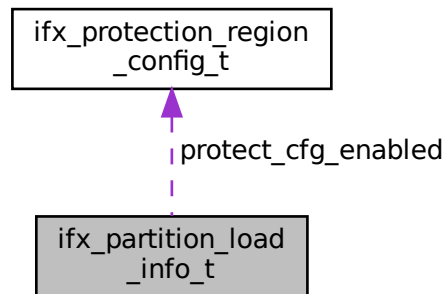
- [platform/ext/target/infineon/common/spe/protection/protection_types.h](#)

7.34 ifx_partition_load_info_t Struct Reference

Structure used to store platform specific partition properties.

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵
types.h>
```

Collaboration diagram for ifx_partition_load_info_t:



Data Fields

- [IFX_FIH_BOOL privileged](#)
Is partition privileged.
- const [ifx_protection_region_config_t](#) * [protect_cfg_enabled](#)
Partition configuration applied by tfm_hal_bind_boundary or by tfm_hal_activate_boundary when partition is enabled.

7.34.1 Detailed Description

Structure used to store platform specific partition properties.

Definition at line 226 of file protection_types.h.

7.34.2 Field Documentation

7.34.2.1 privileged

[IFX_FIH_BOOL](#) privileged

Is partition privileged.

Definition at line 228 of file protection_types.h.

7.34.2.2 protect_cfg_enabled

```
const ifx_protection_region_config_t* protect_cfg_enabled
```

Partition configuration applied by tfm_hal_bind_boundary or by tfm_hal_activate_boundary when partition is enabled.

Definition at line 239 of file protection_types.h.

The documentation for this struct was generated from the following file:

- platform/ext/target/infineon/common/spe/protection/protection_types.h

7.35 ifx_ppc_named_mmio_config_t Struct Reference

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵  
ppc_v1.h>
```

Data Fields

- cy_en_ppc_sec_attribute_t [sec_attr](#)
- bool [allow_unpriv](#)
- ifx_pc_mask_t [pc_mask](#)

7.35.1 Detailed Description

Definition at line 30 of file protection_ppc_v1.h.

7.35.2 Field Documentation

7.35.2.1 allow_unpriv

```
bool allow_unpriv
```

Whether unprivileged access is permitted

Definition at line 32 of file protection_ppc_v1.h.

7.35.2.2 pc_mask

`ifx_pc_mask_t pc_mask`

PC mask

Definition at line 33 of file `protection_ppc_v1.h`.

7.35.2.3 sec_attr

`cy_en_ppc_sec_attribute_t sec_attr`

Security attribute

Definition at line 31 of file `protection_ppc_v1.h`.

The documentation for this struct was generated from the following files:

- `platform/ext/target/infineon/common/spe/protection/protection_ppc_v1.h`
- `platform/ext/target/infineon/common/spe/protection/protection_ppc_v2.h`

7.36 ifx_ppc_regions_config_t Struct Reference

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵  
types.h>
```

Data Fields

- `const cy_en_prot_region_t * regions`
Pointer to the array of regions for the domain.
- `uint32_t region_count`
Number of items in [regions](#).

7.36.1 Detailed Description

Definition at line 177 of file `protection_types.h`.

7.36.2 Field Documentation

7.36.2.1 region_count

```
uint32_t region_count
```

Number of items in [regions](#).

Definition at line 181 of file `protection_types.h`.

7.36.2.2 regions

```
const cy_en_prot_region_t* regions
```

Pointer to the array of regions for the domain.

Definition at line 179 of file `protection_types.h`.

The documentation for this struct was generated from the following file:

- `platform/ext/target/infineon/common/spe/protection/protection_types.h`

7.37 ifx_ppcx_config_t Struct Reference

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵  
ppc_v1.h>
```

Data Fields

- `cy_en_ppc_sec_attribute_t` [sec_attr](#)
- `bool` [allow_unpriv](#)
- `ifx_pc_mask_t` [pc_mask](#)
- `const cy_en_prot_region_t *` [regions](#)
- `size_t` [region_count](#)

7.37.1 Detailed Description

Definition at line 36 of file `protection_ppc_v1.h`.

7.37.2 Field Documentation

7.37.2.1 allow_unpriv

```
bool allow_unpriv
```

Whether unprivileged access is permitted

Definition at line 38 of file protection_ppc_v1.h.

7.37.2.2 pc_mask

```
ifx_pc_mask_t pc_mask
```

PC mask

Definition at line 39 of file protection_ppc_v1.h.

7.37.2.3 region_count

```
size_t region_count
```

Number of items in /ref regions

Definition at line 41 of file protection_ppc_v1.h.

7.37.2.4 regions

```
const cy_en_prot_region_t * regions
```

Array of PPC regions

Definition at line 40 of file protection_ppc_v1.h.

7.37.2.5 sec_attr

```
cy_en_ppc_sec_attribute_t sec_attr
```

Security attribute

Definition at line 37 of file protection_ppc_v1.h.

The documentation for this struct was generated from the following files:

- platform/ext/target/infineon/common/spe/protection/[protection_ppc_v1.h](#)
- platform/ext/target/infineon/common/spe/protection/[protection_ppc_v2.h](#)

7.38 ifx_protection_region_config_t Struct Reference

Structure used to store platform specific protection properties.

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵
types.h>
```

Data Fields

- [IFX_FIH_BOOL mpu_regions_enable](#)

7.38.1 Detailed Description

Structure used to store platform specific protection properties.

Definition at line 203 of file protection_types.h.

7.38.2 Field Documentation

7.38.2.1 mpu_regions_enable

[IFX_FIH_BOOL](#) mpu_regions_enable

Definition at line 210 of file protection_types.h.

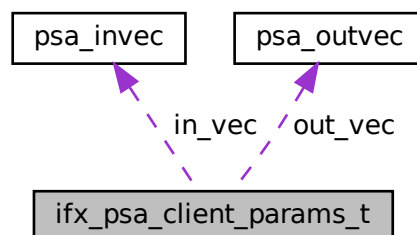
The documentation for this struct was generated from the following file:

- platform/ext/target/infineon/common/spe/protection/[protection_types.h](#)

7.39 ifx_psa_client_params_t Struct Reference

```
#include <platform/ext/target/infineon/common/interface/include/ifx_mtb_↵
mailbox/ifx_mtb_mailbox.h>
```

Collaboration diagram for ifx_psa_client_params_t:



Data Fields

- union {
 - struct {
 - uint32_t sid } psa_version_params
 - struct {
 - uint32_t sid
 - uint32_t version } psa_connect_params
 - struct {
 - psa_handle_t handle
 - int32_t type
 - psa_invec in_vec [PSA_MAX_IOVEC]
 - size_t in_len
 - psa_outvec out_vec [PSA_MAX_IOVEC]
 - size_t out_len } psa_call_params
 - struct {
 - psa_handle_t handle } psa_close_params };

7.39.1 Detailed Description

Definition at line 43 of file ifx_mtb_mailbox.h.

7.39.2 Field Documentation

7.39.2.1 "@19

```
union { ... }
```

7.39.2.2 handle

```
psa_handle_t handle
```

Definition at line 55 of file ifx_mtb_mailbox.h.

7.39.2.3 in_len

```
size_t in_len
```

Definition at line 58 of file ifx_mtb_mailbox.h.

7.39.2.4 in_vec

```
psa_invec in_vec[PSA_MAX_IOVEC]
```

Definition at line 57 of file ifx_mtb_mailbox.h.

7.39.2.5 out_len

```
size_t out_len
```

Definition at line 60 of file ifx_mtb_mailbox.h.

7.39.2.6 out_vec

```
psa_outvec out_vec[PSA_MAX_IOVEC]
```

Definition at line 59 of file ifx_mtb_mailbox.h.

7.39.2.7 psa_call_params

```
struct { ... } psa_call_params
```

7.39.2.8 psa_close_params

```
struct { ... } psa_close_params
```

7.39.2.9 psa_connect_params

```
struct { ... } psa_connect_params
```

7.39.2.10 psa_version_params

```
struct { ... } psa_version_params
```


7.39.2.11 sid

`uint32_t sid`

Definition at line 46 of file `ifx_mtb_mailbox.h`.

7.39.2.12 type

`int32_t type`

Definition at line 56 of file `ifx_mtb_mailbox.h`.

7.39.2.13 version

`uint32_t version`

Definition at line 51 of file `ifx_mtb_mailbox.h`.

The documentation for this struct was generated from the following file:

- `platform/ext/target/infineon/common/interface/include/ifx_mtb_mailbox/ifx_mtb_mailbox.h`

7.40 ifx_rram_counter_t Struct Reference

Data Fields

- union {
 - `ifx_rram_counter_value_t value`
 - `uint8_t bytes [sizeof(ifx_rram_counter_value_t)]`
- `uint32_t checksum`

7.40.1 Detailed Description

Definition at line 23 of file `nv_counters_rram.c`.

7.40.2 Field Documentation

7.40.2.1 bytes

```
uint8_t bytes[sizeof(ifx_rram_counter_value_t)]
```

Definition at line 28 of file `nv_counters_rram.c`.

7.40.2.2 checksum

```
uint32_t checksum
```

Definition at line 30 of file `nv_counters_rram.c`.

7.40.2.3 data

```
union { ... } data
```

7.40.2.4 value

```
ifx_rram_counter_value_t value
```

Definition at line 27 of file `nv_counters_rram.c`.

The documentation for this struct was generated from the following file:

- `platform/ext/target/infineon/common/spe/services/platform/nv_counters_rram.c`

7.41 ifx_sau_config_t Struct Reference

```
#include <platform/ext/target/infineon/common/spe/protection/protection_↵  
types.h>
```

Data Fields

- uint32_t `base_addr`
- uint32_t `size`
- bool `nsc`

7.41.1 Detailed Description

Definition at line 301 of file `protection_types.h`.

7.41.2 Field Documentation

7.41.2.1 base_addr

uint32_t base_addr

Definition at line 302 of file protection_types.h.

7.41.2.2 nsc

bool nsc

Definition at line 304 of file protection_types.h.

7.41.2.3 size

uint32_t size

Definition at line 303 of file protection_types.h.

The documentation for this struct was generated from the following file:

- platform/ext/target/infineon/common/spe/protection/[protection_types.h](#)

7.42 ifx_timer_irq_test_dev_config Struct Reference

```
#include <platform/ext/target/infineon/common/device/include/device_definition.h>
```

Data Fields

- TCPWM_Type * [tcpwm_base](#)
- uint32_t [tcpwm_counter_num](#)
- const cy_stc_tcpwm_counter_config_t * [tcpwm_config](#)

7.42.1 Detailed Description

Definition at line 31 of file device_definition.h.

7.42.2 Field Documentation

7.42.2.1 tcpwm_base

```
TCPWM_Type* tcpwm_base
```

Definition at line 32 of file `device_definition.h`.

7.42.2.2 tcpwm_config

```
const cy_stc_tcpwm_counter_config_t* tcpwm_config
```

Definition at line 34 of file `device_definition.h`.

7.42.2.3 tcpwm_counter_num

```
uint32_t tcpwm_counter_num
```

Definition at line 33 of file `device_definition.h`.

The documentation for this struct was generated from the following file:

- [platform/ext/target/infineon/common/device/include/device_definition.h](#)

7.43 irq_load_info_t Struct Reference

```
#include <secure_fw/spm/include/load/interrupt_defs.h>
```

Data Fields

- enum `tfm_hal_status_t`(* [init](#))(void *pt, const struct [irq_load_info_t](#) *pildi)
- [psa_flih_result_t](#)(* [flih_func](#))(void)
- int32_t [pid](#)
- uint32_t [source](#)
- [psa_signal_t](#) [signal](#)
- int32_t [client_id_base](#)
- int32_t [client_id_limit](#)
- uint32_t [mbox_flags](#)

7.43.1 Detailed Description

Definition at line 16 of file interrupt_defs.h.

7.43.2 Field Documentation

7.43.2.1 client_id_base

```
int32_t client_id_base
```

Definition at line 27 of file interrupt_defs.h.

7.43.2.2 client_id_limit

```
int32_t client_id_limit
```

Definition at line 28 of file interrupt_defs.h.

7.43.2.3 flih_func

```
psa_flih_result_t(* flih_func(void)
```

Definition at line 22 of file interrupt_defs.h.

7.43.2.4 init

```
enum tfm_hal_status_t(* init(void *pt, const struct irq_load_info_t *pildi)
```

Definition at line 21 of file interrupt_defs.h.

7.43.2.5 mbox_flags

```
uint32_t mbox_flags
```

Definition at line 29 of file interrupt_defs.h.

7.43.2.6 pid

```
int32_t pid
```

Definition at line 23 of file interrupt_defs.h.

7.43.2.7 signal

```
psa_signal_t signal
```

Definition at line 25 of file interrupt_defs.h.

7.43.2.8 source

```
uint32_t source
```

Definition at line 24 of file interrupt_defs.h.

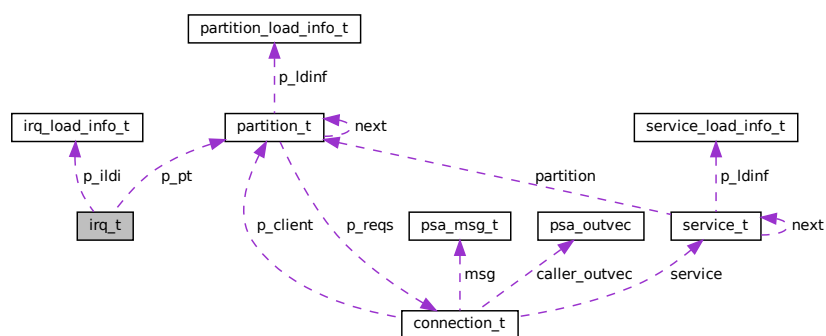
The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/load/interrupt_defs.h](#)

7.44 irq_t Struct Reference

```
#include <secure_fw/spm/include/load/interrupt_defs.h>
```

Collaboration diagram for irq_t:



Data Fields

- struct [partition_t](#) * [p_pt](#)
- const struct [irq_load_info_t](#) * [p_ildi](#)

7.44.1 Detailed Description

Definition at line 33 of file `interrupt_defs.h`.

7.44.2 Field Documentation

7.44.2.1 [p_ildi](#)

```
const struct irq\_load\_info\_t* p\_ildi
```

Definition at line 35 of file `interrupt_defs.h`.

7.44.2.2 [p_pt](#)

```
struct partition\_t* p\_pt
```

Definition at line 34 of file `interrupt_defs.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/spm/include/load/interrupt\_defs.h`

7.45 its_block_meta_t Struct Reference

Structure to store information about each physical flash memory block.

```
#include <secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash↵_fs_mblock.h>
```

Data Fields

- uint32_t [phy_id](#)
- size_t [data_start](#)
- size_t [free_size](#)

7.45.1 Detailed Description

Structure to store information about each physical flash memory block.

Note

This structure is programmed to flash, so its size must be padded to a multiple of the maximum required flash program unit.

Definition at line 128 of file `its_flash_fs_mblock.h`.

7.45.2 Field Documentation

7.45.2.1 `data_start`

```
size_t data_start
```

Offset from the beginning of the block to the * location where the data starts

Definition at line 129 of file `its_flash_fs_mblock.h`.

7.45.2.2 `free_size`

```
size_t free_size
```

Number of bytes free at end of block (set during * block compaction for gap reuse)

Definition at line 129 of file `its_flash_fs_mblock.h`.

7.45.2.3 `phy_id`

```
uint32_t phy_id
```

Physical ID of this logical block

Definition at line 129 of file `its_flash_fs_mblock.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/internal_trusted_storage/flash_fs/its\_flash\_fs\_mblock.h`

7.46 its_file_meta_t Struct Reference

Structure to store file metadata.

```
#include <secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_↵_fs_mblock.h>
```

Data Fields

- uint32_t lblock
- size_t data_idx
- size_t cur_size
- size_t max_size
- uint32_t flags
- uint8_t id [ITS_FILE_ID_SIZE]

7.46.1 Detailed Description

Structure to store file metadata.

Note

This structure is programmed to flash, so its size must be padded to a multiple of the maximum required flash program unit.

Definition at line 165 of file its_flash_fs_mblock.h.

7.46.2 Field Documentation

7.46.2.1 cur_size

```
size_t cur_size
```

Definition at line 166 of file its_flash_fs_mblock.h.

7.46.2.2 data_idx

```
size_t data_idx
```

Definition at line 166 of file its_flash_fs_mblock.h.

7.46.2.3 flags

```
uint32_t flags
```

Definition at line 166 of file `its_flash_fs_mblock.h`.

7.46.2.4 id

```
uint8_t id[ITS_FILE_ID_SIZE]
```

Definition at line 166 of file `its_flash_fs_mblock.h`.

7.46.2.5 lblock

```
uint32_t lblock
```

Definition at line 166 of file `its_flash_fs_mblock.h`.

7.46.2.6 max_size

```
size_t max_size
```

Definition at line 166 of file `its_flash_fs_mblock.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_mblock.h`

7.47 its_flash_fs_config_t Struct Reference

Structure containing the flash filesystem configuration parameters.

```
#include <secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_↵_fs.h>
```

Data Fields

- const `void *` `flash_dev`
- `uint32_t` `flash_area_addr`
- `uint32_t` `sector_size`
- `uint32_t` `block_size`
- `uint16_t` `num_blocks`
- `uint16_t` `program_unit`
- `uint16_t` `max_file_size`
- `uint16_t` `max_num_files`
- `uint8_t` `erase_val`

7.47.1 Detailed Description

Structure containing the flash filesystem configuration parameters.

Definition at line 51 of file its_flash_fs.h.

7.47.2 Field Documentation

7.47.2.1 block_size

```
uint32_t block_size
```

Size of a logical filesystem erase unit, a multiple of sector_size.

Definition at line 57 of file its_flash_fs.h.

7.47.2.2 erase_val

```
uint8_t erase_val
```

Value of a byte after erase (usually 0xFF)

Definition at line 64 of file its_flash_fs.h.

7.47.2.3 flash_area_addr

```
uint32_t flash_area_addr
```

Base address of the flash region

Definition at line 53 of file its_flash_fs.h.

7.47.2.4 flash_dev

```
const void* flash_dev
```

Pointer to the flash device

Definition at line 52 of file its_flash_fs.h.

7.47.2.5 max_file_size

```
uint16_t max_file_size
```

Maximum file size

Definition at line 62 of file `its_flash_fs.h`.

7.47.2.6 max_num_files

```
uint16_t max_num_files
```

Maximum number of files

Definition at line 63 of file `its_flash_fs.h`.

7.47.2.7 num_blocks

```
uint16_t num_blocks
```

Number of logical erase blocks

Definition at line 60 of file `its_flash_fs.h`.

7.47.2.8 program_unit

```
uint16_t program_unit
```

Minimum size of a program operation

Definition at line 61 of file `its_flash_fs.h`.

7.47.2.9 sector_size

```
uint32_t sector_size
```

Size of the flash device's physical erase unit

Definition at line 54 of file `its_flash_fs.h`.

The documentation for this struct was generated from the following file:

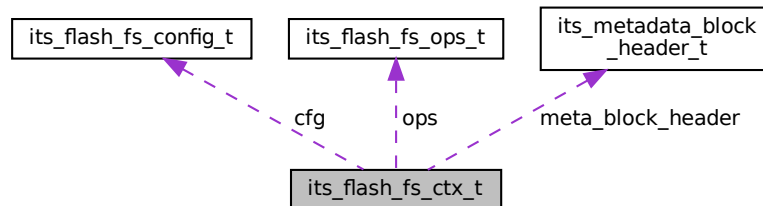
- `secure_fw/partitions/internal_trusted_storage/flash_fs/its\_flash\_fs.h`

7.48 its_flash_fs_ctx_t Struct Reference

Structure to store the ITS flash file system context.

```
#include <secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_
_fs_mblock.h>
```

Collaboration diagram for its_flash_fs_ctx_t:



Data Fields

- const struct [its_flash_fs_config_t](#) * `cfg`
- const struct [its_flash_fs_ops_t](#) * `ops`
- struct [its_metadata_block_header_t](#) `meta_block_header`
- uint32_t `active_metablock`
- uint32_t `scratch_metablock`

7.48.1 Detailed Description

Structure to store the ITS flash file system context.

Definition at line 179 of file `its_flash_fs_mblock.h`.

7.48.2 Field Documentation

7.48.2.1 active_metablock

```
uint32_t active_metablock
```

Active metadata block

Definition at line 185 of file `its_flash_fs_mblock.h`.

7.48.2.2 cfg

```
const struct its_flash_fs_config_t* cfg
```

Filesystem configuration

Definition at line 180 of file its_flash_fs_mblock.h.

7.48.2.3 meta_block_header

```
struct its_metadata_block_header_t meta_block_header
```

Metadata block header

Definition at line 182 of file its_flash_fs_mblock.h.

7.48.2.4 ops

```
const struct its_flash_fs_ops_t* ops
```

Filesystem flash operations

Definition at line 181 of file its_flash_fs_mblock.h.

7.48.2.5 scratch_metablock

```
uint32_t scratch_metablock
```

Scratch metadata block

Definition at line 186 of file its_flash_fs_mblock.h.

The documentation for this struct was generated from the following file:

- [secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_mblock.h](#)

7.49 its_flash_fs_file_info_t Struct Reference

Structure containing file information.

```
#include <secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_↵  
_fs.h>
```

Data Fields

- `size_t` [size_current](#)
- `size_t` [size_max](#)
- `uint32_t` [flags](#)

7.49.1 Detailed Description

Structure containing file information.

This structure is not written to the filesystem, it is used by the file system functions to simplify accessing the containing information.

Definition at line 179 of file `its_flash_fs.h`.

7.49.2 Field Documentation

7.49.2.1 flags

```
uint32_t flags
```

Flags set when the file was created

Definition at line 182 of file `its_flash_fs.h`.

7.49.2.2 size_current

```
size_t size_current
```

The current size of the file in bytes

Definition at line 180 of file `its_flash_fs.h`.

7.49.2.3 size_max

```
size_t size_max
```

The maximum size of the file in bytes.

Definition at line 181 of file `its_flash_fs.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/internal_trusted_storage/flash_fs/`[its_flash_fs.h](#)

7.50 its_flash_fs_ops_t Struct Reference

Structure containing the filesystem flash operation parameters.

```
#include <secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash↵_fs.h>
```

Data Fields

- [psa_status_t](#)(* [init](#))(const struct [its_flash_fs_config_t](#) *cfg)
Initializes the flash device.
- [psa_status_t](#)(* [read](#))(const struct [its_flash_fs_config_t](#) *cfg, uint32_t block_id, uint8_t *buf, size_t [offset](#), size_t [size](#))
Reads block data from the position specified by block ID and offset.
- [psa_status_t](#)(* [write](#))(const struct [its_flash_fs_config_t](#) *cfg, uint32_t block_id, const uint8_t *buf, size_t [offset](#), size_t [size](#))
Writes block data to the position specified by block ID and offset.
- [psa_status_t](#)(* [flush](#))(const struct [its_flash_fs_config_t](#) *cfg, uint32_t block_id)
Flushes modifications to a block to flash. Must be called after a sequence of calls to [write\(\)](#) (including via [its_flash↵_block_to_block_move\(\)](#)) for one block ID, before any call to the same functions for a different block ID.
- [psa_status_t](#)(* [erase](#))(const struct [its_flash_fs_config_t](#) *cfg, uint32_t block_id)
Erases block ID data.

7.50.1 Detailed Description

Structure containing the filesystem flash operation parameters.

Definition at line 72 of file [its_flash_fs.h](#).

7.50.2 Field Documentation

7.50.2.1 erase

```
psa\_status\_t(* erase(const struct its\_flash\_fs\_config\_t *cfg, uint32_t block_id)
```

Erases block ID data.

Parameters

in	<i>cfg</i>	Filesystem configuration
in	<i>block↵_id</i>	Block ID

Note

This function assumes the input value is valid.

Returns

Returns `PSA_SUCCESS` if the function is executed correctly. Otherwise, it returns `PSA_ERROR_STORAGE_FAILURE`.

Definition at line 155 of file `its_flash_fs.h`.

7.50.2.2 flush

```
psa_status_t (* flush)(const struct its_flash_fs_config_t *cfg, uint32_t block_id)
```

Flushes modifications to a block to flash. Must be called after a sequence of calls to `write()` (including via `its_flash_block_to_block_move()`) for one block ID, before any call to the same functions for a different block ID.

Parameters

in	<i>cfg</i>	Filesystem configuration
in	<i>block_id</i>	Block ID

Note

It is permitted for `write()` to commit block updates immediately, in which case this function is a no-op.

Returns

Returns `PSA_SUCCESS` if the function is executed correctly. Otherwise, it returns `PSA_ERROR_STORAGE_FAILURE`.

Definition at line 141 of file `its_flash_fs.h`.

7.50.2.3 init

```
psa_status_t (* init)(const struct its_flash_fs_config_t *cfg)
```

Initializes the flash device.

Parameters

in	<i>cfg</i>	Filesystem configuration
----	------------	--------------------------

Returns

Returns `PSA_SUCCESS` if the function is executed correctly. Otherwise, it returns `PSA_ERROR_STORAGE_FAILURE`.

Definition at line 81 of file `its_flash_fs.h`.

7.50.2.4 read

```
psa_status_t (* read(const struct its_flash_fs_config_t *cfg, uint32_t block_id, uint8_t *buf,
size_t offset, size_t size)
```

Reads block data from the position specified by block ID and offset.

Parameters

in	<i>cfg</i>	Filesystem configuration
in	<i>block_id</i>	Block ID
out	<i>buf</i>	Buffer pointer to store the data read
in	<i>offset</i>	Offset position from the init of the block
in	<i>size</i>	Number of bytes to read

Note

This function assumes all input values are valid. That is, the address range, based on `block_id`, `offset` and `size`, is a valid range in flash.

Returns

Returns `PSA_SUCCESS` if the function is executed correctly. Otherwise, it returns `PSA_ERROR_STORAGE_FAILURE`.

Definition at line 100 of file `its_flash_fs.h`.

7.50.2.5 write

```
psa_status_t (* write(const struct its_flash_fs_config_t *cfg, uint32_t block_id, const uint8_t
*buf, size_t offset, size_t size)
```

Writes block data to the position specified by block ID and offset.

Parameters

in	<i>cfg</i>	Filesystem configuration
in	<i>block_id</i>	Block ID
in	<i>buf</i>	Buffer pointer to the write data
in	<i>offset</i>	Offset position from the init of the block
in	<i>size</i>	Number of bytes to write

Note

This function assumes all input values are valid. That is, the address range, based on block_id, offset and size, is a valid range in flash.

Returns

Returns PSA_SUCCESS if the function is executed correctly. Otherwise, it returns PSA_ERROR_STORAGE_FAILURE.

Definition at line 121 of file its_flash_fs.h.

The documentation for this struct was generated from the following file:

- secure_fw/partitions/internal_trusted_storage/flash_fs/[its_flash_fs.h](#)

7.51 its_flash_nand_dev_t Struct Reference

```
#include <secure_fw/partitions/internal_trusted_storage/flash/its_flash_nand.h>
```

Data Fields

- ARM_DRIVER_FLASH * [driver](#)
- uint32_t [buf_block_id_0](#)
- uint32_t [buf_block_id_1](#)
- uint8_t * [write_buf_0](#)
- uint8_t * [write_buf_1](#)
- size_t [buf_size](#)

7.51.1 Detailed Description

Definition at line 27 of file its_flash_nand.h.

7.51.2 Field Documentation

7.51.2.1 buf_block_id_0

```
uint32_t buf_block_id_0
```

Definition at line 32 of file its_flash_nand.h.

7.51.2.2 buf_block_id_1

```
uint32_t buf_block_id_1
```

Definition at line 33 of file `its_flash_nand.h`.

7.51.2.3 buf_size

```
size_t buf_size
```

Definition at line 36 of file `its_flash_nand.h`.

7.51.2.4 driver

```
ARM_DRIVER_FLASH* driver
```

Definition at line 28 of file `its_flash_nand.h`.

7.51.2.5 write_buf_0

```
uint8_t* write_buf_0
```

Definition at line 34 of file `its_flash_nand.h`.

7.51.2.6 write_buf_1

```
uint8_t* write_buf_1
```

Definition at line 35 of file `its_flash_nand.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/internal_trusted_storage/flash/its_flash_nand.h`

7.52 its_metadata_block_header_comp_t Struct Reference

The structure of metadata block header in ITS_BACKWARD_SUPPORTED_VERSION.

```
#include <secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash↵_fs_mblock.h>
```

Data Fields

- uint32_t [scratch_dblock](#)
- uint8_t [fs_version](#)
- uint8_t [active_swap_count](#)

7.52.1 Detailed Description

The structure of metadata block header in ITS_BACKWARD_SUPPORTED_VERSION.

Note

The `active_swap_count` must be the last member to allow it to be programmed last.

This structure is programmed to flash, so its size must be padded to a multiple of the maximum required flash program unit.

Definition at line 104 of file `its_flash_fs_mblock.h`.

7.52.2 Field Documentation

7.52.2.1 `active_swap_count`

```
uint8_t active_swap_count
```

Number of times the metadata blocks have * been swapped

Definition at line 105 of file `its_flash_fs_mblock.h`.

7.52.2.2 `fs_version`

```
uint8_t fs_version
```

Filesystem version

Definition at line 105 of file `its_flash_fs_mblock.h`.

7.52.2.3 scratch_dblock

```
uint32_t scratch_dblock
```

Physical block ID of the data * section's scratch block

Definition at line 105 of file `its_flash_fs_mblock.h`.

The documentation for this struct was generated from the following file:

- [secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_fs_mblock.h](#)

7.53 its_metadata_block_header_t Struct Reference

Structure to store the metadata block header.

```
#include <secure_fw/partitions/internal_trusted_storage/flash_fs/its_flash_↵_fs_mblock.h>
```

Data Fields

- `uint32_t` [scratch_dblock](#)
- `uint8_t` [fs_version](#)
- `uint8_t` [metadata_xor](#)
- `uint8_t` [active_swap_count](#)

7.53.1 Detailed Description

Structure to store the metadata block header.

Note

The `active_swap_count` must be the last member to allow it to be programmed last. The `fs_version` must be at the same position as it is in [its_metadata_block_header_comp_t](#).

This structure is programmed to flash, so its size must be padded to a multiple of the maximum required flash program unit.

Definition at line 73 of file `its_flash_fs_mblock.h`.

7.53.2 Field Documentation

7.53.2.1 active_swap_count

```
uint8_t active_swap_count
```

Number of times the metadata blocks have * been swapped

Definition at line 74 of file its_flash_fs_mblock.h.

7.53.2.2 fs_version

```
uint8_t fs_version
```

Filesystem version

Definition at line 74 of file its_flash_fs_mblock.h.

7.53.2.3 metadata_xor

```
uint8_t metadata_xor
```

XOR value based on the whole metadata(not * including the metadata block header)

Definition at line 74 of file its_flash_fs_mblock.h.

7.53.2.4 scratch_dblock

```
uint32_t scratch_dblock
```

Physical block ID of the data * section's scratch block

Definition at line 74 of file its_flash_fs_mblock.h.

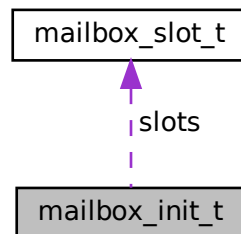
The documentation for this struct was generated from the following file:

- secure_fw/partitions/internal_trusted_storage/flash_fs/[its_flash_fs_mblock.h](#)

7.54 mailbox_init_t Struct Reference

```
#include <interface/include/multi_core/tfm_mailbox.h>
```

Collaboration diagram for mailbox_init_t:



Public Member Functions

- struct `mailbox_status_t` *status `__ALIGNED` (32)

Data Fields

- uint32_t `slot_count`
- struct `mailbox_slot_t` * `slots`

7.54.1 Detailed Description

Definition at line 157 of file `tfm_mailbox.h`.

7.54.2 Member Function Documentation

7.54.2.1 `__ALIGNED()`

```
struct mailbox_status_t* status __ALIGNED (
    32 )
```

7.54.3 Field Documentation

7.54.3.1 slot_count

```
uint32_t slot_count
```

Definition at line 162 of file tfm_mailbox.h.

7.54.3.2 slots

```
struct mailbox_slot_t* slots
```

Definition at line 165 of file tfm_mailbox.h.

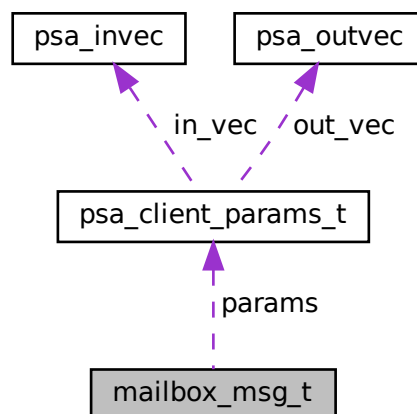
The documentation for this struct was generated from the following file:

- [interface/include/multi_core/tfm_mailbox.h](#)

7.55 mailbox_msg_t Struct Reference

```
#include <interface/include/multi_core/tfm_mailbox.h>
```

Collaboration diagram for mailbox_msg_t:



Data Fields

- `uint32_t` [call_type](#)
- `struct` [psa_client_params_t](#) `params`
- `int32_t` [client_id](#)

7.55.1 Detailed Description

Definition at line 97 of file tfm_mailbox.h.

7.55.2 Field Documentation

7.55.2.1 call_type

```
uint32_t call_type
```

Definition at line 98 of file tfm_mailbox.h.

7.55.2.2 client_id

```
int32_t client_id
```

Definition at line 103 of file tfm_mailbox.h.

7.55.2.3 params

```
struct psa_client_params_t params
```

Definition at line 99 of file tfm_mailbox.h.

The documentation for this struct was generated from the following file:

- [interface/include/multi_core/tfm_mailbox.h](#)

7.56 mailbox_reply_t Struct Reference

```
#include <interface/include/multi_core/tfm_mailbox.h>
```

Data Fields

- int32_t [return_val](#)
- size_t [out_vec_len](#) [[PSA_MAX_IOVEC](#)]

7.56.1 Detailed Description

Definition at line 115 of file tfm_mailbox.h.

7.56.2 Field Documentation

7.56.2.1 out_vec_len

```
size_t out_vec_len[PSA_MAX_IOVEC]
```

Definition at line 117 of file tfm_mailbox.h.

7.56.2.2 return_val

```
int32_t return_val
```

Definition at line 116 of file tfm_mailbox.h.

The documentation for this struct was generated from the following file:

- [interface/include/multi_core/tfm_mailbox.h](#)

7.57 mailbox_slot_t Struct Reference

```
#include <interface/include/multi_core/tfm_mailbox.h>
```

Public Member Functions

- struct [mailbox_msg_t](#) msg [__ALIGNED](#) (32)
- struct [mailbox_reply_t](#) reply [__ALIGNED](#) (32)

7.57.1 Detailed Description

Definition at line 126 of file tfm_mailbox.h.

7.57.2 Member Function Documentation

7.57.2.1 `__ALIGNED()` [1/2]

```
struct mailbox_msg_t msg __ALIGNED (  
    32 )
```

7.57.2.2 `__ALIGNED()` [2/2]

```
struct mailbox_reply_t reply __ALIGNED (  
    32 )
```

The documentation for this struct was generated from the following file:

- [interface/include/multi_core/tfm_mailbox.h](#)

7.58 mailbox_status_t Struct Reference

```
#include <interface/include/multi_core/tfm_mailbox.h>
```

Data Fields

- [mailbox_queue_status_t pend_slots](#)
- [mailbox_queue_status_t replied_slots](#)

7.58.1 Detailed Description

Definition at line 142 of file `tfm_mailbox.h`.

7.58.2 Field Documentation

7.58.2.1 `pend_slots`

```
mailbox_queue_status_t pend_slots
```

Definition at line 143 of file `tfm_mailbox.h`.

7.58.2.2 replied_slots

`mailbox_queue_status_t` replied_slots

Definition at line 146 of file `tfm_mailbox.h`.

The documentation for this struct was generated from the following file:

- `interface/include/multi_core/tfm_mailbox.h`

7.59 mbedtls_asn1_bitstring Struct Reference

```
#include <interface/include/mbedtls/asn1.h>
```

Data Fields

- `size_t` `len`
- unsigned char `unused_bits`
- unsigned char * `p`

7.59.1 Detailed Description

Container for ASN1 bit strings.

Definition at line 147 of file `asn1.h`.

The documentation for this struct was generated from the following file:

- `interface/include/mbedtls/asn1.h`

7.60 mbedtls_asn1_buf Struct Reference

```
#include <interface/include/mbedtls/asn1.h>
```

Data Fields

- int `tag`
- `size_t` `len`
- unsigned char * `p`

7.60.1 Detailed Description

Type-length-value structure that allows for ASN1 using DER.

Definition at line 137 of file `asn1.h`.

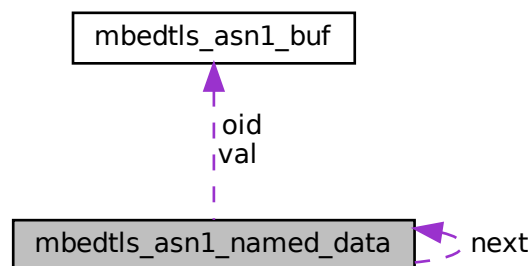
The documentation for this struct was generated from the following file:

- `interface/include/mbedtls/asn1.h`

7.61 mbedtls_asn1_named_data Struct Reference

```
#include <interface/include/mbedtls/asn1.h>
```

Collaboration diagram for `mbedtls_asn1_named_data`:



Public Member Functions

- unsigned char `MBEDTLS_PRIVATE` (next_merged)

Data Fields

- `mbedtls_asn1_buf` oid
- `mbedtls_asn1_buf` val
- struct `mbedtls_asn1_named_data` * next

7.61.1 Detailed Description

Container for a sequence or list of 'named' ASN.1 data items

Definition at line 174 of file `asn1.h`.

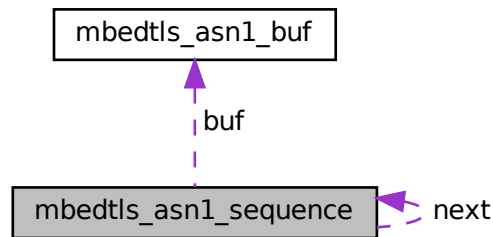
The documentation for this struct was generated from the following file:

- `interface/include/mbedtls/asn1.h`

7.62 mbedtls_asn1_sequence Struct Reference

```
#include <interface/include/mbedtls/asn1.h>
```

Collaboration diagram for mbedtls_asn1_sequence:



Data Fields

- [mbedtls_asn1_buf](#) buf
- struct [mbedtls_asn1_sequence](#) * next

7.62.1 Detailed Description

Container for a sequence of ASN.1 items

Definition at line 157 of file [asn1.h](#).

The documentation for this struct was generated from the following file:

- [interface/include/mbedtls/asn1.h](#)

7.63 mbedtls_lmots_parameters_t Struct Reference

```
#include <interface/include/mbedtls/lms.h>
```

Public Member Functions

- unsigned char [MBEDTLS_PRIVATE](#) (l_key_identifier[(16u)])
- unsigned char [MBEDTLS_PRIVATE](#) (q_leaf_identifier[(4u)])
- [mbedtls_lmots_algorithm_type_t](#) [MBEDTLS_PRIVATE](#) (type)

7.63.1 Detailed Description

LMOTS parameters structure.

This contains the metadata associated with an LMOTS key, detailing the algorithm type, the key ID, and the leaf identifier should be key be part of a LMS key.

Definition at line 91 of file lms.h.

7.63.2 Member Function Documentation

7.63.2.1 MBEDTLS_PRIVATE() [1/3]

```
unsigned char MBEDTLS_PRIVATE (
    I_key_identifier [(16u)] )
```

The key identifier.

7.63.2.2 MBEDTLS_PRIVATE() [2/3]

```
unsigned char MBEDTLS_PRIVATE (
    q_leaf_identifier [(4u)] )
```

Which leaf of the LMS key this is. 0 if the key is not part of an LMS key.

7.63.2.3 MBEDTLS_PRIVATE() [3/3]

```
mbdts_lmots_algorithm_type_t MBEDTLS_PRIVATE (
    type )
```

The LM-OTS key type identifier as per IANA. Only SHA256_N32_W8 is currently supported.

The documentation for this struct was generated from the following file:

- [interface/include/mbedtls/lms.h](#)

7.64 mbedtls_lmots_public_t Struct Reference

```
#include <interface/include/mbedtls/lms.h>
```

Public Member Functions

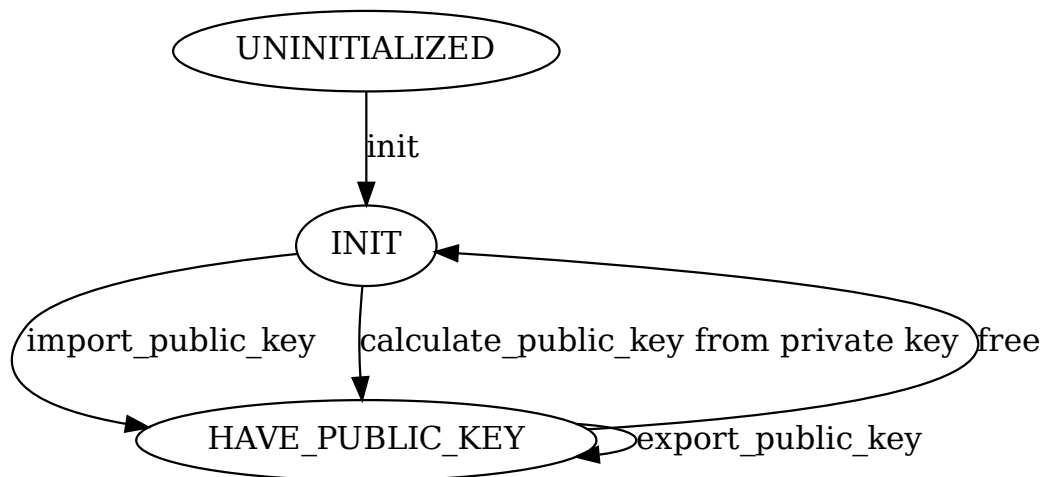
- [mbedtls_lmots_parameters_t MBEDTLS_PRIVATE](#) (params)
- unsigned char [MBEDTLS_PRIVATE](#) (public_key)[(32u)]
- unsigned char [MBEDTLS_PRIVATE](#) (have_public_key)

7.64.1 Detailed Description

LMOTS public context structure.

A LMOTS public key is a hash output, and the applicable parameter set.

The context must be initialized before it is used. A public key must either be imported or generated from a private context.



Definition at line 119 of file lms.h.

7.64.2 Member Function Documentation

7.64.2.1 MBEDTLS_PRIVATE() [1/3]

```
unsigned char MBEDTLS_PRIVATE (
    have_public_key )
```

Whether the context contains a public key. Boolean values only.

7.64.2.2 MBEDTLS_PRIVATE() [2/3]

```
mbedtls_lmots_parameters_t MBEDTLS_PRIVATE (
    params )
```

7.64.2.3 MBEDTLS_PRIVATE() [3/3]

```
unsigned char MBEDTLS_PRIVATE (
    public_key )
```

The documentation for this struct was generated from the following file:

- [interface/include/mbedtls/lms.h](#)

7.65 mbedtls_lms_parameters_t Struct Reference

```
#include <interface/include/mbedtls/lms.h>
```

Public Member Functions

- unsigned char [MBEDTLS_PRIVATE](#) (I_key_identifier[(16u)])
- [mbedtls_lmots_algorithm_type_t](#) [MBEDTLS_PRIVATE](#) (otstype)
- [mbedtls_lms_algorithm_type_t](#) [MBEDTLS_PRIVATE](#) (type)

7.65.1 Detailed Description

LMS parameters structure.

This contains the metadata associated with an LMS key, detailing the algorithm type, the type of the underlying OTS algorithm, and the key ID.

Definition at line 159 of file lms.h.

7.65.2 Member Function Documentation

7.65.2.1 MBEDTLS_PRIVATE() [1/3]

```
unsigned char MBEDTLS_PRIVATE (
    I_key_identifier [(16u)] )
```

The key identifier.

7.65.2.2 MBEDTLS_PRIVATE() [2/3]

```
mbedtls\_lmots\_algorithm\_type\_t MBEDTLS_PRIVATE (
    otstype )
```

The LM-OTS key type identifier as per IANA. Only SHA256_N32_W8 is currently supported.

7.65.2.3 MBEDTLS_PRIVATE() [3/3]

```
mbedtls_lms_algorithm_type_t MBEDTLS_PRIVATE (
    type )
```

The LMS key type identifier as per IANA. Only SHA256_M32_H10 is currently supported.

The documentation for this struct was generated from the following file:

- [interface/include/mbedtls/lms.h](#)

7.66 mbedtls_lms_public_t Struct Reference

```
#include <interface/include/mbedtls/lms.h>
```

Public Member Functions

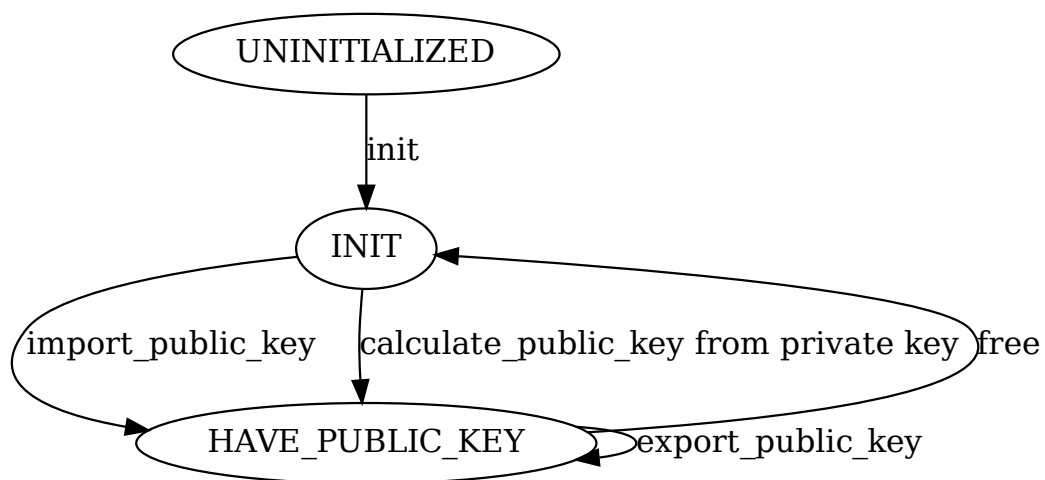
- [mbedtls_lms_parameters_t MBEDTLS_PRIVATE](#) (params)
- unsigned char [MBEDTLS_PRIVATE](#) (T_1_pub_key)[32]
- unsigned char [MBEDTLS_PRIVATE](#) (have_public_key)

7.66.1 Detailed Description

LMS public context structure.

A LMS public key is the hash output that is the root of the Merkle tree, and the applicable parameter set

The context must be initialized before it is used. A public key must either be imported or generated from a private context.



Definition at line 188 of file `lms.h`.

7.66.2 Member Function Documentation

7.66.2.1 MBEDTLS_PRIVATE() [1/3]

```
unsigned char MBEDTLS_PRIVATE (
    have_public_key )
```

Whether the context contains a public key. Boolean values only.

7.66.2.2 MBEDTLS_PRIVATE() [2/3]

```
MBEDTLS_LMS_PARAMETERS_T MBEDTLS_PRIVATE (
    params )
```

7.66.2.3 MBEDTLS_PRIVATE() [3/3]

```
unsigned char MBEDTLS_PRIVATE (
    T_1_pub_key )
```

The public key, in the form of the Merkle tree root node.

The documentation for this struct was generated from the following file:

- [interface/include/mbedtls/lms.h](#)

7.67 mbedtls_md_context_t Struct Reference

```
#include <interface/include/mbedtls/md.h>
```

Public Member Functions

- `const mbedtls_md_info_t * MBEDTLS_PRIVATE (md_info)`
- `void * MBEDTLS_PRIVATE (md_ctx)`

7.67.1 Detailed Description

The generic message-digest context.

Definition at line 109 of file md.h.

7.67.2 Member Function Documentation

7.67.2.1 MBEDTLS_PRIVATE() [1/2]

```
void* MBEDTLS_PRIVATE (
    md_ctx )
```

The digest-specific context (legacy) or the PSA operation.

7.67.2.2 MBEDTLS_PRIVATE() [2/2]

```
const mbedtls_md_info_t* MBEDTLS_PRIVATE (
    md_info )
```

Information about the associated message digest.

The documentation for this struct was generated from the following file:

- [interface/include/mbedtls/md.h](#)

7.68 mbedtls_pk_context Struct Reference

Public key container.

```
#include <interface/include/mbedtls/pk.h>
```

Public Member Functions

- const [mbedtls_pk_info_t](#) * [MBEDTLS_PRIVATE](#) (pk_info)
- [psa_key_type_t](#) [MBEDTLS_PRIVATE](#) (psa_type)
- [mbedtls_svc_key_id_t](#) [MBEDTLS_PRIVATE](#) (priv_id)
- [uint8_t](#) [MBEDTLS_PRIVATE](#) (pub_raw)[0]
- [size_t](#) [MBEDTLS_PRIVATE](#) (pub_raw_len)
- [size_t](#) [MBEDTLS_PRIVATE](#) (bits)

7.68.1 Detailed Description

Public key container.

Definition at line 106 of file pk.h.

7.68.2 Member Function Documentation

7.68.2.1 MBEDTLS_PRIVATE() [1/6]

```
size_t MBEDTLS_PRIVATE (
    bits )
```

7.68.2.2 MBEDTLS_PRIVATE() [2/6]

```
const mbedtls_pk_info_t* MBEDTLS_PRIVATE (
    pk_info )
```

7.68.2.3 MBEDTLS_PRIVATE() [3/6]

```
mbedtls_svc_key_id_t MBEDTLS_PRIVATE (
    priv_id )
```

7.68.2.4 MBEDTLS_PRIVATE() [4/6]

```
psa_key_type_t MBEDTLS_PRIVATE (
    psa_type )
```

7.68.2.5 MBEDTLS_PRIVATE() [5/6]

```
uint8_t MBEDTLS_PRIVATE (
    pub_raw )
```

7.68.2.6 MBEDTLS_PRIVATE() [6/6]

```
size_t MBEDTLS_PRIVATE (
    pub_raw_len )
```

The documentation for this struct was generated from the following file:

- [interface/include/mbedtls/pk.h](#)

7.69 mbedtls_platform_context Struct Reference

The platform context structure.

```
#include <interface/include/mbedtls/platform.h>
```

Public Member Functions

- char [MBEDTLS_PRIVATE](#) (dummy)
- char [MBEDTLS_PRIVATE](#) (dummy)

7.69.1 Detailed Description

The platform context structure.

Note

This structure may be used to assist platform-specific setup or teardown operations.

Definition at line 429 of file platform.h.

7.69.2 Member Function Documentation

7.69.2.1 MBEDTLS_PRIVATE() [1/2]

```
char MBEDTLS_PRIVATE (
    dummy )
```

A placeholder member, as empty structs are not portable.

7.69.2.2 MBEDTLS_PRIVATE() [2/2]

```
char MBEDTLS_PRIVATE (
    dummy )
```

A placeholder member, as empty structs are not portable.

The documentation for this struct was generated from the following files:

- interface/include/mbedtls/[platform.h](#)
- platform/ext/target/infineon/common/libs/tf-psa-crypto/[platform_alt.h](#)

7.70 mbedtls_psa_stats_s Struct Reference

Statistics about resource consumption related to the PSA keystore.

```
#include <interface/include/psa/crypto_extra.h>
```

Public Member Functions

- `size_t MBEDTLS_PRIVATE (volatile_slots)`
- `size_t MBEDTLS_PRIVATE (persistent_slots)`
- `size_t MBEDTLS_PRIVATE (external_slots)`
- `size_t MBEDTLS_PRIVATE (half_filled_slots)`
- `size_t MBEDTLS_PRIVATE (cache_slots)`
- `size_t MBEDTLS_PRIVATE (empty_slots)`
- `size_t MBEDTLS_PRIVATE (locked_slots)`
- `psa_key_id_t MBEDTLS_PRIVATE (max_open_internal_key_id)`
- `psa_key_id_t MBEDTLS_PRIVATE (max_open_external_key_id)`

7.70.1 Detailed Description

Statistics about resource consumption related to the PSA keystore.

Note

The content of this structure is not part of the stable API and ABI of Mbed TLS and may change arbitrarily from version to version.

Definition at line 127 of file `crypto_extra.h`.

7.70.2 Member Function Documentation

7.70.2.1 MBEDTLS_PRIVATE() [1/9]

```
size_t MBEDTLS_PRIVATE (
    cache_slots )
```

Number of slots that contain cache data.

7.70.2.2 MBEDTLS_PRIVATE() [2/9]

```
size_t MBEDTLS_PRIVATE (
    empty_slots )
```

Number of slots that are not used for anything.

7.70.2.3 MBEDTLS_PRIVATE() [3/9]

```
size_t MBEDTLS_PRIVATE (
    external_slots )
```

Number of slots containing a reference to a key in a secure element.

7.70.2.4 MBEDTLS_PRIVATE() [4/9]

```
size_t MBEDTLS_PRIVATE (
    half_filled_slots )
```

Number of slots which are occupied, but do not contain key material yet.

7.70.2.5 MBEDTLS_PRIVATE() [5/9]

```
size_t MBEDTLS_PRIVATE (
    locked_slots )
```

Number of slots that are locked.

7.70.2.6 MBEDTLS_PRIVATE() [6/9]

```
psa_key_id_t MBEDTLS_PRIVATE (
    max_open_external_key_id )
```

Largest key id value among open keys in secure elements.

7.70.2.7 MBEDTLS_PRIVATE() [7/9]

```
psa_key_id_t MBEDTLS_PRIVATE (
    max_open_internal_key_id )
```

Largest key id value among open keys in internal persistent storage.

7.70.2.8 MBEDTLS_PRIVATE() [8/9]

```
size_t MBEDTLS_PRIVATE (
    persistent_slots )
```

Number of slots containing key material for a key which is in internal persistent storage.

7.70.2.9 MBEDTLS_PRIVATE() [9/9]

```
size_t MBEDTLS_PRIVATE (
    volatile_slots )
```

Number of slots containing key material for a volatile key.

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_extra.h](#)

7.71 mpu_armv8m_region_cfg_t Struct Reference

```
#include <platform/ext/target/infineon/common/drivers/protection/mpu_armv8m_↵_drv.h>
```

Data Fields

- uint32_t [region_base](#)
- uint32_t [region_limit](#)
- uint32_t [region_attridx](#)
- enum [mpu_armv8m_attr_exec_t](#) [attr_exec](#)
- enum [mpu_armv8m_attr_access_t](#) [attr_access](#)
- enum [mpu_armv8m_attr_shared_t](#) [attr_sh](#)

7.71.1 Detailed Description

Definition at line 49 of file mpu_armv8m_drv.h.

7.71.2 Field Documentation

7.71.2.1 attr_access

```
enum mpu\_armv8m\_attr\_access\_t attr_access
```

Definition at line 54 of file mpu_armv8m_drv.h.

7.71.2.2 attr_exec

```
enum mpu\_armv8m\_attr\_exec\_t attr_exec
```

Definition at line 53 of file mpu_armv8m_drv.h.

7.71.2.3 attr_sh

```
enum mpu\_armv8m\_attr\_shared\_t attr_sh
```

Definition at line 55 of file mpu_armv8m_drv.h.

7.71.2.4 region_attridx

```
uint32_t region_attridx
```

Definition at line 52 of file mpu_armv8m_drv.h.

7.71.2.5 region_base

```
uint32_t region_base
```

Definition at line 50 of file mpu_armv8m_drv.h.

7.71.2.6 region_limit

```
uint32_t region_limit
```

Definition at line 51 of file mpu_armv8m_drv.h.

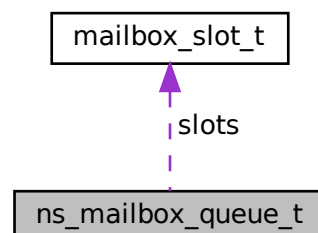
The documentation for this struct was generated from the following file:

- [platform/ext/target/infineon/common/drivers/protection/mpu_armv8m_drv.h](#)

7.72 ns_mailbox_queue_t Struct Reference

```
#include <interface/include/multi_core/tfm_ns_mailbox.h>
```

Collaboration diagram for ns_mailbox_queue_t:



Public Member Functions

- struct [mailbox_status_t](#) status [__ALIGNED](#) (32)
- struct [ns_mailbox_slot_t](#) slots_ns[[NUM_MAILBOX_QUEUE_SLOT](#)] [__ALIGNED](#) (32)

Data Fields

- struct [mailbox_slot_t](#) slots [[NUM_MAILBOX_QUEUE_SLOT](#)]
- [mailbox_queue_status_t](#) empty_slots
- bool [is_full](#)

7.72.1 Detailed Description

Definition at line 68 of file [tfm_ns_mailbox.h](#).

7.72.2 Member Function Documentation

7.72.2.1 [__ALIGNED\(\)](#) [1/2]

```
struct mailbox\_status\_t status \_\_ALIGNED (  
    32 )
```

7.72.2.2 [__ALIGNED\(\)](#) [2/2]

```
struct ns\_mailbox\_slot\_t slots_ns [NUM\_MAILBOX\_QUEUE\_SLOT] \_\_ALIGNED (  
    32 )
```

7.72.3 Field Documentation

7.72.3.1 [empty_slots](#)

[mailbox_queue_status_t](#) [empty_slots](#)

Definition at line 75 of file [tfm_ns_mailbox.h](#).

7.72.3.2 is_full

```
bool is_full
```

Definition at line 90 of file tfm_ns_mailbox.h.

7.72.3.3 slots

```
struct mailbox_slot_t slots[NUM_MAILBOX_QUEUE_SLOT]
```

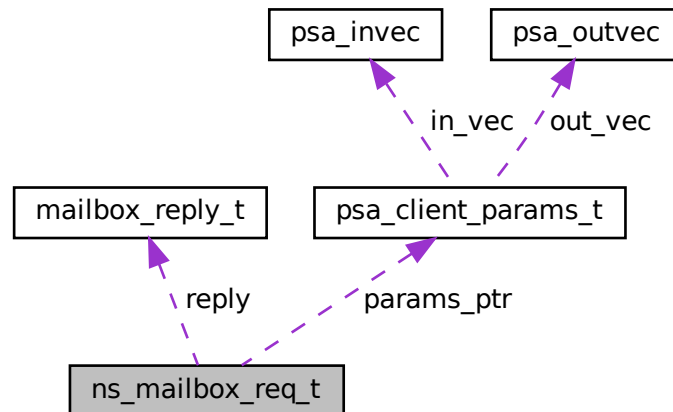
Definition at line 70 of file tfm_ns_mailbox.h.

The documentation for this struct was generated from the following file:

- [interface/include/multi_core/tfm_ns_mailbox.h](#)

7.73 ns_mailbox_req_t Struct Reference

Collaboration diagram for ns_mailbox_req_t:



Data Fields

- `uint32_t` [call_type](#)
- `const struct psa\_client\_params\_t *` [params_ptr](#)
- `int32_t` [client_id](#)
- `const void *` [owner](#)
- `struct mailbox\_reply\_t *` [reply](#)
- `uint8_t *` [woken_flag](#)

7.73.1 Detailed Description

Definition at line 20 of file tfm_ns_mailbox_thread.c.

7.73.2 Field Documentation

7.73.2.1 call_type

```
uint32_t call_type
```

Definition at line 21 of file tfm_ns_mailbox_thread.c.

7.73.2.2 client_id

```
int32_t client_id
```

Definition at line 25 of file tfm_ns_mailbox_thread.c.

7.73.2.3 owner

```
const void* owner
```

Definition at line 32 of file tfm_ns_mailbox_thread.c.

7.73.2.4 params_ptr

```
const struct psa_client_params_t* params_ptr
```

Definition at line 22 of file tfm_ns_mailbox_thread.c.

7.73.2.5 reply

```
struct mailbox_reply_t* reply
```

Definition at line 33 of file tfm_ns_mailbox_thread.c.

7.73.2.6 woken_flag

```
uint8_t* woken_flag
```

Definition at line 37 of file `tfm_ns_mailbox_thread.c`.

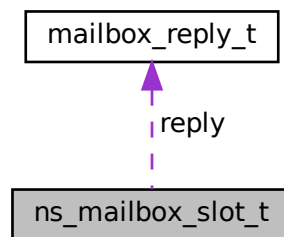
The documentation for this struct was generated from the following file:

- `interface/src/multi_core/tfm_ns_mailbox_thread.c`

7.74 ns_mailbox_slot_t Struct Reference

```
#include <interface/include/multi_core/tfm_ns_mailbox.h>
```

Collaboration diagram for `ns_mailbox_slot_t`:



Data Fields

- const `void` * `owner`
- struct `mailbox_reply_t` * `reply`
- bool `is_woken`

7.74.1 Detailed Description

Definition at line 49 of file `tfm_ns_mailbox.h`.

7.74.2 Field Documentation

7.74.2.1 is_woken

```
bool is_woken
```

Definition at line 60 of file tfm_ns_mailbox.h.

7.74.2.2 owner

```
const void* owner
```

Definition at line 50 of file tfm_ns_mailbox.h.

7.74.2.3 reply

```
struct mailbox_reply_t* reply
```

Definition at line 51 of file tfm_ns_mailbox.h.

The documentation for this struct was generated from the following file:

- [interface/include/multi_core/tfm_ns_mailbox.h](#)

7.75 ns_mailbox_stats_res_t Struct Reference

The structure to hold the statistics result of NSPE mailbox.

```
#include <interface/include/multi_core/tfm_ns_mailbox_test.h>
```

Data Fields

- uint8_t [avg_nr_slots](#)
- uint8_t [avg_nr_slots_tenths](#)

7.75.1 Detailed Description

The structure to hold the statistics result of NSPE mailbox.

Definition at line 24 of file tfm_ns_mailbox_test.h.

7.75.2 Field Documentation

7.75.2.1 avg_nr_slots

```
uint8_t avg_nr_slots
```

Definition at line 25 of file `tfm_ns_mailbox_test.h`.

7.75.2.2 avg_nr_slots_tenths

```
uint8_t avg_nr_slots_tenths
```

Definition at line 29 of file `tfm_ns_mailbox_test.h`.

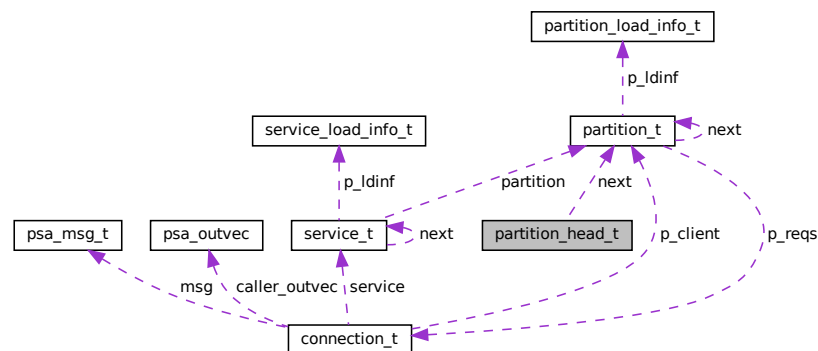
The documentation for this struct was generated from the following file:

- `interface/include/multi_core/tfm_ns_mailbox_test.h`

7.76 partition_head_t Struct Reference

```
#include <secure_fw/spm/include/load/spm_load_api.h>
```

Collaboration diagram for `partition_head_t`:



Data Fields

- `uint32_t reserved`
- `struct partition_t * next`

7.76.1 Detailed Description

Definition at line 49 of file `spm_load_api.h`.

7.76.2 Field Documentation

7.76.2.1 next

```
struct partition\_t* next
```

Definition at line 51 of file `spm_load_api.h`.

7.76.2.2 reserved

```
uint32_t reserved
```

Definition at line 50 of file `spm_load_api.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/spm/include/load/spm\_load\_api.h`

7.77 `partition_load_info_t` Struct Reference

```
#include <secure_fw/spm/include/load/partition_defs.h>
```

Data Fields

- uint32_t [psa_ff_ver](#)
- int32_t [pid](#)
- uint32_t [flags](#)
- uintptr_t [entry](#)
- size_t [stack_size](#)
- size_t [heap_size](#)
- uint32_t [ndeps](#)
- uint32_t [nservices](#)
- uint32_t [nassets](#)
- uint32_t [nirqs](#)
- int32_t [client_id_base](#)
- int32_t [client_id_limit](#)
- uint16_t [load_order](#)

7.77.1 Detailed Description

Definition at line 99 of file `partition_defs.h`.

7.77.2 Field Documentation

7.77.2.1 client_id_base

```
int32_t client_id_base
```

Definition at line 112 of file partition_defs.h.

7.77.2.2 client_id_limit

```
int32_t client_id_limit
```

Definition at line 113 of file partition_defs.h.

7.77.2.3 entry

```
uintptr_t entry
```

Definition at line 105 of file partition_defs.h.

7.77.2.4 flags

```
uint32_t flags
```

Definition at line 102 of file partition_defs.h.

7.77.2.5 heap_size

```
size_t heap_size
```

Definition at line 107 of file partition_defs.h.

7.77.2.6 load_order

```
uint16_t load_order
```

Definition at line 114 of file partition_defs.h.

7.77.2.7 nassets

```
uint32_t nassets
```

Definition at line 110 of file partition_defs.h.

7.77.2.8 ndeps

```
uint32_t ndeps
```

Definition at line 108 of file partition_defs.h.

7.77.2.9 nirqs

```
uint32_t nirqs
```

Definition at line 111 of file partition_defs.h.

7.77.2.10 nservices

```
uint32_t nservices
```

Definition at line 109 of file partition_defs.h.

7.77.2.11 pid

```
int32_t pid
```

Definition at line 101 of file partition_defs.h.

7.77.2.12 psa_ff_ver

```
uint32_t psa_ff_ver
```

Definition at line 100 of file partition_defs.h.

7.77.2.13 stack_size

```
size_t stack_size
```

Definition at line 106 of file partition_defs.h.

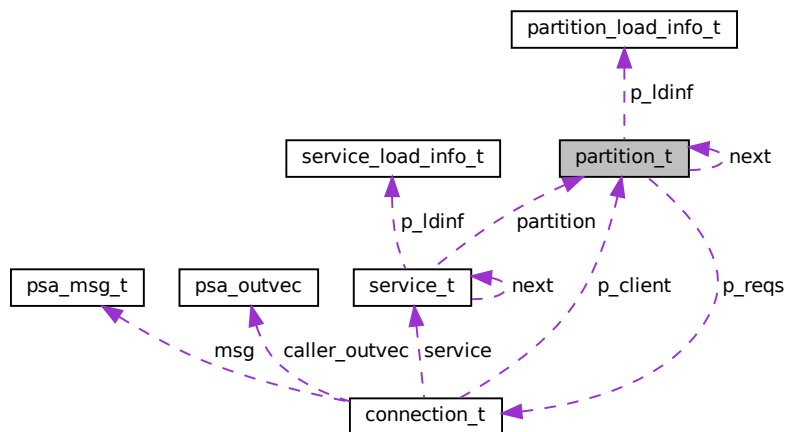
The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/load/partition_defs.h](#)

7.78 partition_t Struct Reference

```
#include <secure_fw/spm/core/spm.h>
```

Collaboration diagram for partition_t:



Data Fields

- const struct [partition_load_info_t](#) * `p_ldinf`
- `uintptr_t` `boundary`
- `uint32_t` `signals_allowed`
- `uint32_t` `signals_waiting`
- volatile `uint32_t` `signals_asserted`
- `uint32_t` `state`
- struct [connection_t](#) * `p_reqs`
- struct [partition_t](#) * `next`

7.78.1 Detailed Description

Definition at line 104 of file spm.h.

7.78.2 Field Documentation

7.78.2.1 boundary

```
uintptr_t boundary
```

Definition at line 106 of file spm.h.

7.78.2.2 next

```
struct partition\_t* next
```

Definition at line 119 of file spm.h.

7.78.2.3 p_ldinf

```
const struct partition\_load\_info\_t* p_ldinf
```

Definition at line 105 of file spm.h.

7.78.2.4 p_reqs

```
struct connection\_t* p_reqs
```

Definition at line 118 of file spm.h.

7.78.2.5 signals_allowed

```
uint32_t signals_allowed
```

Definition at line 107 of file spm.h.

7.78.2.6 signals_asserted

```
volatile uint32_t signals_asserted
```

Definition at line 109 of file `spm.h`.

7.78.2.7 signals_waiting

```
uint32_t signals_waiting
```

Definition at line 108 of file `spm.h`.

7.78.2.8 state

```
uint32_t state
```

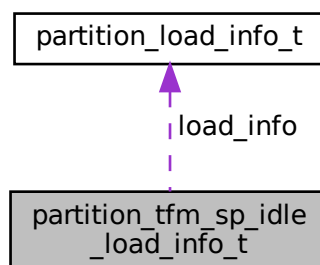
Definition at line 116 of file `spm.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/spm/core/spm.h`

7.79 partition_tfm_sp_idle_load_info_t Struct Reference

Collaboration diagram for `partition_tfm_sp_idle_load_info_t`:



Data Fields

- struct `partition_load_info_t` `load_info`
- `uintptr_t` `stack_addr`
- `uintptr_t` `heap_addr`

7.79.1 Detailed Description

Definition at line 28 of file `load_info_idle_sp.c`.

7.79.2 Field Documentation

7.79.2.1 heap_addr

```
uintptr_t heap_addr
```

Definition at line 33 of file `load_info_idle_sp.c`.

7.79.2.2 load_info

```
struct partition_load_info_t load_info
```

Definition at line 30 of file `load_info_idle_sp.c`.

7.79.2.3 stack_addr

```
uintptr_t stack_addr
```

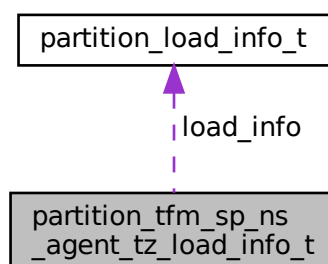
Definition at line 32 of file `load_info_idle_sp.c`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/idle_partition/load_info_idle_sp.c`

7.80 partition_tfm_sp_ns_agent_tz_load_info_t Struct Reference

Collaboration diagram for `partition_tfm_sp_ns_agent_tz_load_info_t`:



Data Fields

- struct [partition_load_info_t](#) [load_info](#)
- uintptr_t [stack_addr](#)
- uintptr_t [heap_addr](#)

7.80.1 Detailed Description

Definition at line 40 of file `load_info_ns_agent_tz.c`.

7.80.2 Field Documentation

7.80.2.1 heap_addr

```
uintptr_t heap_addr
```

Definition at line 45 of file `load_info_ns_agent_tz.c`.

7.80.2.2 load_info

```
struct partition\_load\_info\_t load_info
```

Definition at line 42 of file `load_info_ns_agent_tz.c`.

7.80.2.3 stack_addr

```
uintptr_t stack_addr
```

Definition at line 44 of file `load_info_ns_agent_tz.c`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/ns_agent_tz/load_info_ns_agent_tz.c`

7.81 ps_crypto_t Union Reference

```
#include <secure_fw/partitions/protected_storage/crypto/ps_crypto_interface.h>
```

Data Fields

- struct {
 uint8_t [tag](#) [PS_TAG_LEN_BYTES]
 [psa_storage_uid_t](#) [uid](#)
 int32_t [client_id](#)
 uint8_t [iv](#) [PS_IV_LEN_BYTES]
} [ref](#)

7.81.1 Detailed Description

Definition at line 27 of file `ps_crypto_interface.h`.

7.81.2 Field Documentation

7.81.2.1 `client_id`

```
int32_t client_id
```

Owner client ID for key label

Definition at line 31 of file `ps_crypto_interface.h`.

7.81.2.2 `iv`

```
uint8_t iv[PS_IV_LEN_BYTES]
```

IV value of AEAD object

Definition at line 32 of file `ps_crypto_interface.h`.

7.81.2.3 `ref`

```
struct { ... } ref
```

7.81.2.4 tag

```
uint8_t tag[PS_TAG_LEN_BYTES]
```

MAC value of AEAD object

Definition at line 29 of file ps_crypto_interface.h.

7.81.2.5 uid

```
psa_storage_uid_t uid
```

UID for key label

Definition at line 30 of file ps_crypto_interface.h.

The documentation for this union was generated from the following file:

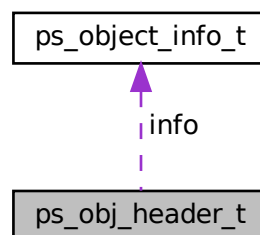
- secure_fw/partitions/protected_storage/crypto/[ps_crypto_interface.h](#)

7.82 ps_obj_header_t Struct Reference

Metadata attached as a header to object data before storage.

```
#include <secure_fw/partitions/protected_storage/ps_object_defs.h>
```

Collaboration diagram for ps_obj_header_t:



Data Fields

- uint32_t [version](#)
- uint32_t [fid](#)
- struct [ps_object_info_t](#) [info](#)

7.82.1 Detailed Description

Metadata attached as a header to object data before storage.

Definition at line 38 of file `ps_object_defs.h`.

7.82.2 Field Documentation

7.82.2.1 `fid`

```
uint32_t fid
```

File ID

Definition at line 43 of file `ps_object_defs.h`.

7.82.2.2 `info`

```
struct ps_object_info_t info
```

Object information

Definition at line 45 of file `ps_object_defs.h`.

7.82.2.3 `version`

```
uint32_t version
```

Object version

Definition at line 42 of file `ps_object_defs.h`.

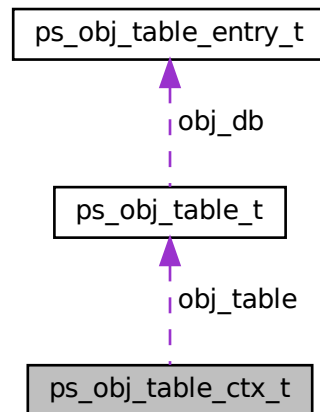
The documentation for this struct was generated from the following file:

- `secure_fw/partitions/protected_storage/ps_object_defs.h`

7.83 ps_obj_table_ctx_t Struct Reference

Object table context structure.

Collaboration diagram for ps_obj_table_ctx_t:



Data Fields

- struct [ps_obj_table_t](#) `obj_table`
- `uint8_t` `active_table`
- `uint8_t` `scratch_table`

7.83.1 Detailed Description

Object table context structure.

Object table init context structure.

Definition at line 147 of file `ps_object_table.c`.

7.83.2 Field Documentation

7.83.2.1 active_table

`uint8_t` `active_table`

Active object table

Definition at line 149 of file `ps_object_table.c`.

7.83.2.2 obj_table

```
struct ps_obj_table_t obj_table
```

Object tables

Definition at line 148 of file ps_object_table.c.

7.83.2.3 scratch_table

```
uint8_t scratch_table
```

Scratch object table

Definition at line 150 of file ps_object_table.c.

The documentation for this struct was generated from the following file:

- [secure_fw/partitions/protected_storage/ps_object_table.c](#)

7.84 ps_obj_table_entry_t Struct Reference

Data Fields

- uint32_t [version](#)
- [psa_storage_uid_t](#) uid
- int32_t [client_id](#)

7.84.1 Detailed Description

Definition at line 40 of file ps_object_table.c.

7.84.2 Field Documentation

7.84.2.1 client_id

```
int32_t client_id
```

Client ID

Definition at line 50 of file ps_object_table.c.

7.84.2.2 uid

`psa_storage_uid_t uid`

Object UID

Definition at line 49 of file `ps_object_table.c`.

7.84.2.3 version

`uint32_t version`

File version

Definition at line 47 of file `ps_object_table.c`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/protected_storage/ps_object_table.c`

7.85 ps_obj_table_info_t Struct Reference

Object table information structure.

```
#include <secure_fw/partitions/protected_storage/ps_object_table.h>
```

Data Fields

- `uint32_t fid`
- `uint32_t version`

7.85.1 Detailed Description

Object table information structure.

Definition at line 26 of file `ps_object_table.h`.

7.85.2 Field Documentation

7.85.2.1 fid

```
uint32_t fid
```

File ID in the file system

Definition at line 27 of file `ps_object_table.h`.

7.85.2.2 version

```
uint32_t version
```

Object version

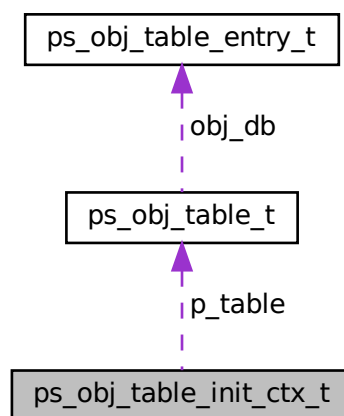
Definition at line 34 of file `ps_object_table.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/protected_storage/ps_object_table.h`

7.86 ps_obj_table_init_ctx_t Struct Reference

Collaboration diagram for `ps_obj_table_init_ctx_t`:



Data Fields

- struct `ps_obj_table_t` * `p_table` [2]
- enum `ps_obj_table_state` `table_state` [2]

7.86.1 Detailed Description

Definition at line 221 of file ps_object_table.c.

7.86.2 Field Documentation

7.86.2.1 p_table

```
struct ps_obj_table_t* p_table[2]
```

Pointers to object tables

Definition at line 222 of file ps_object_table.c.

7.86.2.2 table_state

```
enum ps_obj_table_state table_state[2]
```

Array to indicate if the object table X is valid

Definition at line 225 of file ps_object_table.c.

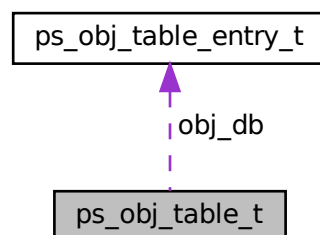
The documentation for this struct was generated from the following file:

- [secure_fw/partitions/protected_storage/ps_object_table.c](#)

7.87 ps_obj_table_t Struct Reference

Object table structure.

Collaboration diagram for ps_obj_table_t:



Data Fields

- `uint8_t version`
- `uint8_t swap_count`
- `struct ps_obj_table_entry_t obj_db [(PS_NUM_ASSETS+1)]`

7.87.1 Detailed Description

Object table structure.

Definition at line 64 of file `ps_object_table.c`.

7.87.2 Field Documentation

7.87.2.1 `obj_db`

```
struct ps_obj_table_entry_t obj_db[ (PS_NUM_ASSETS+1) ]
```

Table's entries

Definition at line 77 of file `ps_object_table.c`.

7.87.2.2 `swap_count`

```
uint8_t swap_count
```

Swap counter to distinguish 2 different object tables.

Definition at line 72 of file `ps_object_table.c`.

7.87.2.3 `version`

```
uint8_t version
```

PS object system version.

Definition at line 69 of file `ps_object_table.c`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/protected_storage/ps_object_table.c`

7.88 ps_object_info_t Struct Reference

Object information.

```
#include <secure_fw/partitions/protected_storage/ps_object_defs.h>
```

Data Fields

- uint32_t [current_size](#)
- uint32_t [max_size](#)
- [psa_storage_create_flags_t](#) [create_flags](#)

7.88.1 Detailed Description

Object information.

Definition at line 27 of file ps_object_defs.h.

7.88.2 Field Documentation

7.88.2.1 create_flags

```
psa_storage_create_flags_t create_flags
```

Object creation flags

Definition at line 30 of file ps_object_defs.h.

7.88.2.2 current_size

```
uint32_t current_size
```

Current size of the object content in bytes

Definition at line 28 of file ps_object_defs.h.

7.88.2.3 max_size

```
uint32_t max_size
```

Maximum size of the object content in bytes

Definition at line 29 of file `ps_object_defs.h`.

The documentation for this struct was generated from the following file:

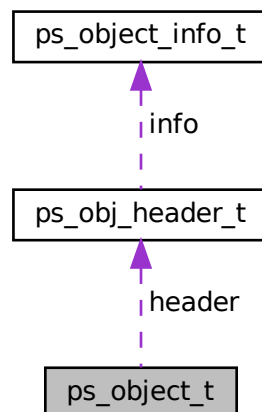
- `secure_fw/partitions/protected_storage/ps_object_defs.h`

7.89 ps_object_t Struct Reference

The object to be written to the file system below. Made up of the object header and the object data.

```
#include <secure_fw/partitions/protected_storage/ps_object_defs.h>
```

Collaboration diagram for `ps_object_t`:



Data Fields

- struct `ps_obj_header_t` `header`
- `uint8_t` `data` [`PS_MAX_ASSET_SIZE`]

7.89.1 Detailed Description

The object to be written to the file system below. Made up of the object header and the object data.

Definition at line 64 of file `ps_object_defs.h`.

7.89.2 Field Documentation

7.89.2.1 data

```
uint8_t data[ PS_MAX_ASSET_SIZE]
```

Object data (and tag if encrypted)

Definition at line 66 of file `ps_object_defs.h`.

7.89.2.2 header

```
struct ps_obj_header_t header
```

Object header

Definition at line 65 of file `ps_object_defs.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/protected_storage/ps_object_defs.h`

7.90 psa_aead_operation_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int `MBEDTLS_PRIVATE` (id)
- `psa_algorithm_t` `MBEDTLS_PRIVATE` (alg)
- `psa_key_type_t` `MBEDTLS_PRIVATE` (key_type)
- `size_t` `MBEDTLS_PRIVATE` (ad_remaining)
- `size_t` `MBEDTLS_PRIVATE` (body_remaining)
- unsigned int `MBEDTLS_PRIVATE`(nonce_set) unsigned int `MBEDTLS_PRIVATE`(lengths_set) unsigned int `MBEDTLS_PRIVATE`(ad_started) unsigned int `MBEDTLS_PRIVATE`(body_started) unsigned int `MBEDTLS_PRIVATE`(is_encrypt) `psa_driver_aead_context_t` `MBEDTLS_PRIVATE` (ctx)

7.90.1 Detailed Description

Definition at line 200 of file `crypto_struct.h`.

7.90.2 Member Function Documentation

7.90.2.1 MBEDTLS_PRIVATE() [1/6]

```
size_t MBEDTLS_PRIVATE (
    ad_remaining )
```

7.90.2.2 MBEDTLS_PRIVATE() [2/6]

```
psa_algorithm_t MBEDTLS_PRIVATE (
    alg )
```

7.90.2.3 MBEDTLS_PRIVATE() [3/6]

```
size_t MBEDTLS_PRIVATE (
    body_remaining )
```

7.90.2.4 MBEDTLS_PRIVATE() [4/6]

```
unsigned int MBEDTLS_PRIVATE (nonce_set) unsigned int MBEDTLS_PRIVATE (lengths_set) unsigned
int MBEDTLS_PRIVATE (ad_started) unsigned int MBEDTLS_PRIVATE (body_started) unsigned int MB
EDTLS_PRIVATE (is_encrypt) psa_driver_aead_context_t MBEDTLS_PRIVATE (
    ctx )
```

7.90.2.5 MBEDTLS_PRIVATE() [5/6]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_crypto_driver_↵wrappers.h` ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

7.90.2.6 MBEDTLS_PRIVATE() [6/6]

```
psa_key_type_t MBEDTLS_PRIVATE (
    key_type )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.91 psa_cipher_operation_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- unsigned int [MBEDTLS_PRIVATE](#)(iv_required) unsigned int [MBEDTLS_PRIVATE](#)(iv_set) uint8_t [MBEDTLS_PRIVATE](#) (default_iv_length)
- [psa_driver_cipher_context_t](#) [MBEDTLS_PRIVATE](#) (ctx)

7.91.1 Detailed Description

Definition at line 126 of file `crypto_struct.h`.

7.91.2 Member Function Documentation**7.91.2.1 MBEDTLS_PRIVATE()** [1/3]

```
psa_driver_cipher_context_t MBEDTLS_PRIVATE (
    ctx )
```

7.91.2.2 MBEDTLS_PRIVATE() [2/3]

```
unsigned int MBEDTLS_PRIVATE (iv_required) unsigned int MBEDTLS_PRIVATE (iv_set) uint8_t MBE←
DTLS_PRIVATE (
    default_iv_length )
```

7.91.2.3 MBEDTLS_PRIVATE() [3/3]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_crypto_driver_` wrappers.h ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

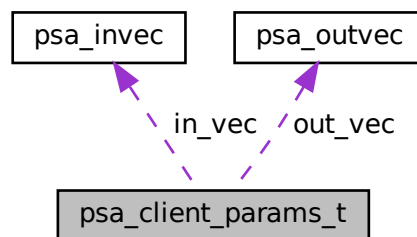
The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.92 psa_client_params_t Struct Reference

```
#include <interface/include/multi_core/tfm_mailbox.h>
```

Collaboration diagram for `psa_client_params_t`:



Data Fields

```
• union {
    struct {
        uint32_t sid
    } psa_version_params
    struct {
        uint32_t sid
        uint32_t version
    } psa_connect_params
    struct {
        psa_handle_t handle
        int32_t type
        psa_invec in_vec [PSA_MAX_IOVEC]
        size_t in_len
        psa_outvec out_vec [PSA_MAX_IOVEC]
        size_t out_len
    } psa_call_params
    struct {
        psa_handle_t handle
    } psa_close_params
};
```


7.92.1 Detailed Description

Definition at line 70 of file tfm_mailbox.h.

7.92.2 Field Documentation

7.92.2.1 "@1

```
union { ... }
```

7.92.2.2 handle

```
psa_handle_t handle
```

Definition at line 82 of file tfm_mailbox.h.

7.92.2.3 in_len

```
size_t in_len
```

Definition at line 85 of file tfm_mailbox.h.

7.92.2.4 in_vec

```
psa_invec in_vec[PSA_MAX_IOVEC]
```

Definition at line 84 of file tfm_mailbox.h.

7.92.2.5 out_len

```
size_t out_len
```

Definition at line 87 of file tfm_mailbox.h.

7.92.2.6 out_vec

```
psa_outvec out_vec[PSA_MAX_IOVEC]
```

Definition at line 86 of file tfm_mailbox.h.

7.92.2.7 psa_call_params

```
struct { ... } psa_call_params
```

7.92.2.8 psa_close_params

```
struct { ... } psa_close_params
```

7.92.2.9 psa_connect_params

```
struct { ... } psa_connect_params
```

7.92.2.10 psa_version_params

```
struct { ... } psa_version_params
```

7.92.2.11 sid

```
uint32_t sid
```

Definition at line 73 of file tfm_mailbox.h.

7.92.2.12 type

```
int32_t type
```

Definition at line 83 of file tfm_mailbox.h.

7.92.2.13 version

uint32_t version

Definition at line 78 of file tfm_mailbox.h.

The documentation for this struct was generated from the following file:

- [interface/include/multi_core/tfm_mailbox.h](#)

7.93 psa_crypto_driver_pake_inputs_s Struct Reference

```
#include <interface/include/psa/crypto_extra.h>
```

Public Member Functions

- uint8_t * [MBEDTLS_PRIVATE](#) (password)
- size_t [MBEDTLS_PRIVATE](#) (password_len)
- uint8_t * [MBEDTLS_PRIVATE](#) (user)
- size_t [MBEDTLS_PRIVATE](#) (user_len)
- uint8_t * [MBEDTLS_PRIVATE](#) (peer)
- size_t [MBEDTLS_PRIVATE](#) (peer_len)
- [psa_key_attributes_t](#) [MBEDTLS_PRIVATE](#) (attributes)
- struct [psa_pake_cipher_suite_s](#) [MBEDTLS_PRIVATE](#) (cipher_suite)

7.93.1 Detailed Description

Definition at line 1175 of file crypto_extra.h.

7.93.2 Member Function Documentation

7.93.2.1 MBEDTLS_PRIVATE() [1/8]

```
psa_key_attributes_t MBEDTLS_PRIVATE (
    attributes )
```

7.93.2.2 MBEDTLS_PRIVATE() [2/8]

```
struct psa_pake_cipher_suite_s MBEDTLS_PRIVATE (
    cipher_suite )
```

7.93.2.3 MBEDTLS_PRIVATE() [3/8]

```
uint8_t* MBEDTLS_PRIVATE (
    password )
```

7.93.2.4 MBEDTLS_PRIVATE() [4/8]

```
size_t MBEDTLS_PRIVATE (
    password_len )
```

7.93.2.5 MBEDTLS_PRIVATE() [5/8]

```
uint8_t* MBEDTLS_PRIVATE (
    peer )
```

7.93.2.6 MBEDTLS_PRIVATE() [6/8]

```
size_t MBEDTLS_PRIVATE (
    peer_len )
```

7.93.2.7 MBEDTLS_PRIVATE() [7/8]

```
uint8_t* MBEDTLS_PRIVATE (
    user )
```

7.93.2.8 MBEDTLS_PRIVATE() [8/8]

```
size_t MBEDTLS_PRIVATE (
    user_len )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_extra.h](#)

7.94 psa_custom_key_parameters_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Data Fields

- uint32_t [flags](#)

7.94.1 Detailed Description

Definition at line 269 of file `crypto_struct.h`.

7.94.2 Field Documentation

7.94.2.1 flags

```
uint32_t flags
```

Definition at line 271 of file `crypto_struct.h`.

The documentation for this struct was generated from the following file:

- `interface/include/psa/crypto_struct.h`

7.95 psa_driver_aead_context_t Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_composites.h>
```

Data Fields

- unsigned [dummy](#)
- mbedtls_psa_aead_operation_t [mbedtls_ctx](#)

7.95.1 Detailed Description

Definition at line 136 of file `crypto_driver_contexts_composites.h`.

7.95.2 Field Documentation

7.95.2.1 dummy

unsigned dummy

Definition at line 137 of file crypto_driver_contexts_composites.h.

7.95.2.2 mbedtls_ctx

mbedtls_psa_aead_operation_t mbedtls_ctx

Definition at line 138 of file crypto_driver_contexts_composites.h.

The documentation for this union was generated from the following file:

- [interface/include/psa/crypto_driver_contexts_composites.h](#)

7.96 psa_driver_cipher_context_t Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_primitives.h>
```

Data Fields

- unsigned [dummy](#)
- mbedtls_psa_cipher_operation_t [mbedtls_ctx](#)

7.96.1 Detailed Description

Definition at line 132 of file crypto_driver_contexts_primitives.h.

7.96.2 Field Documentation

7.96.2.1 dummy

unsigned dummy

Definition at line 133 of file crypto_driver_contexts_primitives.h.

7.96.2.2 mbedtls_ctx

mbedtls_psa_cipher_operation_t mbedtls_ctx

Definition at line 134 of file crypto_driver_contexts_primitives.h.

The documentation for this union was generated from the following file:

- [interface/include/psa/crypto_driver_contexts_primitives.h](#)

7.97 psa_driver_hash_context_t Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_primitives.h>
```

Data Fields

- unsigned [dummy](#)
- mbedtls_psa_hash_operation_t [mbedtls_ctx](#)

7.97.1 Detailed Description

Definition at line 113 of file crypto_driver_contexts_primitives.h.

7.97.2 Field Documentation

7.97.2.1 dummy

unsigned dummy

Definition at line 114 of file crypto_driver_contexts_primitives.h.

7.97.2.2 mbedtls_ctx

mbedtls_psa_hash_operation_t mbedtls_ctx

Definition at line 115 of file crypto_driver_contexts_primitives.h.

The documentation for this union was generated from the following file:

- [interface/include/psa/crypto_driver_contexts_primitives.h](#)

7.98 `psa_driver_key_derivation_context_t` Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_key_derivation.h>
```

Data Fields

- unsigned [dummy](#)

7.98.1 Detailed Description

Definition at line 34 of file `crypto_driver_contexts_key_derivation.h`.

7.98.2 Field Documentation

7.98.2.1 `dummy`

`unsigned dummy`

Definition at line 35 of file `crypto_driver_contexts_key_derivation.h`.

The documentation for this union was generated from the following file:

- `interface/include/psa/c`[rypto_driver_contexts_key_derivation.h](#)

7.99 `psa_driver_mac_context_t` Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_composites.h>
```

Data Fields

- unsigned [dummy](#)
- `MBEDTLS_PSA_MAC_OPERATION_T` [mbedtls_ctx](#)

7.99.1 Detailed Description

Definition at line 124 of file `crypto_driver_contexts_composites.h`.

7.99.2 Field Documentation

7.99.2.1 dummy

unsigned dummy

Definition at line 125 of file crypto_driver_contexts_composites.h.

7.99.2.2 mbedtls_ctx

mbedtls_psa_mac_operation_t mbedtls_ctx

Definition at line 126 of file crypto_driver_contexts_composites.h.

The documentation for this union was generated from the following file:

- [interface/include/psa/crypto_driver_contexts_composites.h](#)

7.100 psa_driver_pake_context_t Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_composites.h>
```

Data Fields

- unsigned [dummy](#)
- mbedtls_psa_pake_operation_t [mbedtls_ctx](#)

7.100.1 Detailed Description

Definition at line 157 of file crypto_driver_contexts_composites.h.

7.100.2 Field Documentation

7.100.2.1 dummy

unsigned dummy

Definition at line 158 of file crypto_driver_contexts_composites.h.

7.100.2.2 mbedtls_ctx

mbedtls_psa_pake_operation_t mbedtls_ctx

Definition at line 159 of file crypto_driver_contexts_composites.h.

The documentation for this union was generated from the following file:

- [interface/include/psa/crypto_driver_contexts_composites.h](#)

7.101 psa_driver_sign_hash_interruptible_context_t Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_composites.h>
```

Data Fields

- unsigned [dummy](#)
- mbedtls_psa_sign_hash_interruptible_operation_t [mbedtls_ctx](#)

7.101.1 Detailed Description

Definition at line 147 of file crypto_driver_contexts_composites.h.

7.101.2 Field Documentation

7.101.2.1 dummy

unsigned dummy

Definition at line 148 of file crypto_driver_contexts_composites.h.

7.101.2.2 mbedtls_ctx

mbedtls_psa_sign_hash_interruptible_operation_t mbedtls_ctx

Definition at line 149 of file crypto_driver_contexts_composites.h.

The documentation for this union was generated from the following file:

- [interface/include/psa/crypto_driver_contexts_composites.h](#)

7.102 `psa_driver_verify_hash_interruptible_context_t` Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_composites.h>
```

Data Fields

- unsigned [dummy](#)
- `mbdttls_psa_verify_hash_interruptible_operation_t` [mbdttls_ctx](#)

7.102.1 Detailed Description

Definition at line 152 of file `crypto_driver_contexts_composites.h`.

7.102.2 Field Documentation

7.102.2.1 `dummy`

```
unsigned dummy
```

Definition at line 153 of file `crypto_driver_contexts_composites.h`.

7.102.2.2 `mbdttls_ctx`

```
mbdttls_psa_verify_hash_interruptible_operation_t mbdttls_ctx
```

Definition at line 154 of file `crypto_driver_contexts_composites.h`.

The documentation for this union was generated from the following file:

- `interface/include/psa/crypto\_driver\_contexts\_composites.h`

7.103 `psa_driver_xof_context_t` Union Reference

```
#include <interface/include/psa/crypto_driver_contexts_primitives.h>
```

Data Fields

- unsigned [dummy](#)
- `mbdttls_psa_xof_operation_t` [mbdttls_ctx](#)

7.103.1 Detailed Description

Definition at line 124 of file `crypto_driver_contexts_primitives.h`.

7.103.2 Field Documentation

7.103.2.1 dummy

```
unsigned dummy
```

Definition at line 125 of file `crypto_driver_contexts_primitives.h`.

7.103.2.2 mbedtls_ctx

```
mbedtls_psa_xof_operation_t mbedtls_ctx
```

Definition at line 126 of file `crypto_driver_contexts_primitives.h`.

The documentation for this union was generated from the following file:

- [interface/include/psa/crypto_driver_contexts_primitives.h](#)

7.104 psa_export_public_key_iop_s Struct Reference

The context for PSA interruptible export public-key.

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- `mbedtls_psa_export_public_key_iop_t` [MBEDTLS_PRIVATE](#) (ctx)
- unsigned int `MBEDTLS_PRIVATE`(error_occurred) uint32_t [MBEDTLS_PRIVATE](#) (num_ops)

7.104.1 Detailed Description

The context for PSA interruptible export public-key.

Definition at line 607 of file `crypto_struct.h`.

7.104.2 Member Function Documentation

7.104.2.1 MBEDTLS_PRIVATE() [1/3]

```
MBEDTLS_PSA_EXPORT_PUBLIC_KEY_IOP_T MBEDTLS_PRIVATE (
    ctx )
```

7.104.2.2 MBEDTLS_PRIVATE() [2/3]

```
UNSIGNED_INT MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_crypto_driver_↔wrappers.h`. ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

7.104.2.3 MBEDTLS_PRIVATE() [3/3]

```
UNSIGNED_INT MBEDTLS_PRIVATE (error_occurred) UINT32_T MBEDTLS_PRIVATE (
    num_ops )
```

The documentation for this struct was generated from the following file:

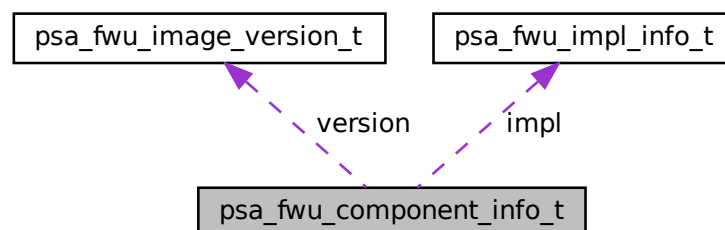
- [interface/include/psa/crypto_struct.h](#)

7.105 psa_fwu_component_info_t Struct Reference

Information about the firmware store for a firmware component.

```
#include <interface/include/psa/update.h>
```

Collaboration diagram for `psa_fwu_component_info_t`:



Data Fields

- [uint8_t state](#)
- [psa_status_t error](#)
- [psa_fwu_image_version_t version](#)
- [uint32_t max_size](#)
- [uint32_t flags](#)
- [uint32_t location](#)
- [psa_fwu_impl_info_t impl](#)

7.105.1 Detailed Description

Information about the firmware store for a firmware component.

Definition at line 147 of file update.h.

7.105.2 Field Documentation

7.105.2.1 error

[psa_status_t error](#)

Error for second image when store state is REJECTED or FAILED.

Definition at line 149 of file update.h.

7.105.2.2 flags

[uint32_t flags](#)

Flags that describe extra information about the firmware component.

Definition at line 152 of file update.h.

7.105.2.3 impl

[psa_fwu_impl_info_t impl](#)

Reserved for implementation-specific usage.

Definition at line 154 of file update.h.

7.105.2.4 location

`uint32_t location`

Implementation-defined image location.

Definition at line 153 of file `update.h`.

7.105.2.5 max_size

`uint32_t max_size`

Maximum image size in bytes.

Definition at line 151 of file `update.h`.

7.105.2.6 state

`uint8_t state`

State of the component.

Definition at line 148 of file `update.h`.

7.105.2.7 version

`psa_fwu_image_version_t version`

Version of active image.

Definition at line 150 of file `update.h`.

The documentation for this struct was generated from the following file:

- `interface/include/psa/update.h`

7.106 psa_fwu_image_version_t Struct Reference

Version information about a firmware image.

```
#include <interface/include/psa/update.h>
```

Data Fields

- `uint8_t` [major](#)
- `uint8_t` [minor](#)
- `uint16_t` [patch](#)
- `uint32_t` [build](#)

7.106.1 Detailed Description

Version information about a firmware image.

Definition at line 81 of file `update.h`.

7.106.2 Field Documentation

7.106.2.1 `build`

```
uint32_t build
```

The build number of an image.

Definition at line 85 of file `update.h`.

7.106.2.2 `major`

```
uint8_t major
```

The major version of an image.

Definition at line 82 of file `update.h`.

7.106.2.3 `minor`

```
uint8_t minor
```

The minor version of an image.

Definition at line 83 of file `update.h`.

7.106.2.4 patch

uint16_t patch

The revision or patch version of an image.

Definition at line 84 of file update.h.

The documentation for this struct was generated from the following file:

- [interface/include/psa/update.h](#)

7.107 psa_fwu_impl_info_t Struct Reference

The implementation-specific data in the component information structure.

```
#include <interface/include/tfm_fwu_impl_info.h>
```

Data Fields

- uint8_t [candidate_digest](#) [TFM_FWU_MAX_DIGEST_SIZE]

7.107.1 Detailed Description

The implementation-specific data in the component information structure.

Definition at line 25 of file tfm_fwu_impl_info.h.

7.107.2 Field Documentation

7.107.2.1 candidate_digest

uint8_t candidate_digest [TFM_FWU_MAX_DIGEST_SIZE]

Definition at line 27 of file tfm_fwu_impl_info.h.

The documentation for this struct was generated from the following file:

- [interface/include/tfm_fwu_impl_info.h](#)

7.108 psa_generate_key_iop_s Struct Reference

The context for PSA interruptible key generation.

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- mbedtls_psa_generate_key_iop_t [MBEDTLS_PRIVATE](#) (ctx)
- [psa_key_attributes_t](#) [MBEDTLS_PRIVATE](#) (attributes)
- unsigned int [MBEDTLS_PRIVATE](#)(error_occurred) uint32_t [MBEDTLS_PRIVATE](#) (num_ops)

7.108.1 Detailed Description

The context for PSA interruptible key generation.

Definition at line 569 of file crypto_struct.h.

7.108.2 Member Function Documentation

7.108.2.1 MBEDTLS_PRIVATE() [1/4]

```
psa_key_attributes_t MBEDTLS_PRIVATE (
    attributes )
```

7.108.2.2 MBEDTLS_PRIVATE() [2/4]

```
mbedtls_psa_generate_key_iop_t MBEDTLS_PRIVATE (
    ctx )
```

7.108.2.3 MBEDTLS_PRIVATE() [3/4]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_crypto_driver_↵ wrappers.h` ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

7.108.2.4 MBEDTLS_PRIVATE() [4/4]

```
unsigned int MBEDTLS_PRIVATE (error_occurred) uint32_t MBEDTLS_PRIVATE (
    num_ops )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.109 psa_hash_operation_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- [psa_driver_hash_context_t](#) [MBEDTLS_PRIVATE](#) (ctx)

7.109.1 Detailed Description

Definition at line 65 of file `crypto_struct.h`.

7.109.2 Member Function Documentation**7.109.2.1 MBEDTLS_PRIVATE()** [1/2]

```
psa_driver_hash_context_t MBEDTLS_PRIVATE (
    ctx )
```

7.109.2.2 MBEDTLS_PRIVATE() [2/2]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_driver_wrappers.h`. ID value zero means the context is not valid or not assigned to any driver (i.e. the driver context is not active, in use).

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.110 psa_invec Struct Reference

```
#include <interface/include/psa/client.h>
```

Data Fields

- const `void *` `base`
- `size_t` `len`

7.110.1 Detailed Description

A read-only input memory region provided to an RoT Service.

Definition at line 88 of file `client.h`.

7.110.2 Field Documentation

7.110.2.1 `base`

```
const void* base
```

the start address of the memory buffer

Definition at line 89 of file `client.h`.

7.110.2.2 `len`

```
size_t len
```

the size in bytes

Definition at line 90 of file `client.h`.

The documentation for this struct was generated from the following file:

- `interface/include/psa/client.h`

7.111 psa_jpake_computation_stage_s Struct Reference

```
#include <interface/include/psa/crypto_extra.h>
```

Public Member Functions

- [psa_jpake_round_t MBEDTLS_PRIVATE](#) (round)
- [psa_jpake_io_mode_t MBEDTLS_PRIVATE](#) (io_mode)
- [uint8_t MBEDTLS_PRIVATE](#) (inputs)
- [uint8_t MBEDTLS_PRIVATE](#) (outputs)
- [psa_pake_step_t MBEDTLS_PRIVATE](#) (step)

7.111.1 Detailed Description

Definition at line 1213 of file crypto_extra.h.

7.111.2 Member Function Documentation

7.111.2.1 MBEDTLS_PRIVATE() [1/5]

```
uint8_t MBEDTLS_PRIVATE (  
    inputs )
```

7.111.2.2 MBEDTLS_PRIVATE() [2/5]

```
psa_jpake_io_mode_t MBEDTLS_PRIVATE (  
    io_mode )
```

7.111.2.3 MBEDTLS_PRIVATE() [3/5]

```
uint8_t MBEDTLS_PRIVATE (  
    outputs )
```

7.111.2.4 MBEDTLS_PRIVATE() [4/5]

```
psa_jpake_round_t MBEDTLS_PRIVATE (  
    round )
```

7.111.2.5 MBEDTLS_PRIVATE() [5/5]

```
psa_pake_step_t MBEDTLS_PRIVATE (
    step )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_extra.h](#)

7.112 psa_key_agreement_iop_s Struct Reference

The context for PSA interruptible key agreement.

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- mbedtls_psa_key_agreement_interruptible_operation_t [MBEDTLS_PRIVATE](#) (mbedtls_ctx)
- uint32_t [MBEDTLS_PRIVATE](#) (num_ops)
- [psa_key_attributes_t](#) [MBEDTLS_PRIVATE](#) (attributes)

7.112.1 Detailed Description

The context for PSA interruptible key agreement.

Definition at line 531 of file `crypto_struct.h`.

7.112.2 Member Function Documentation

7.112.2.1 MBEDTLS_PRIVATE() [1/4]

```
psa_key_attributes_t MBEDTLS_PRIVATE (
    attributes )
```

7.112.2.2 MBEDTLS_PRIVATE() [2/4]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_crypto_driver_↔wrappers.h`. ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

7.112.2.3 MBEDTLS_PRIVATE() [3/4]

```
MBEDTLS_PRIVATE(mbedtls_psa_key_agreement_interruptible_operation_t) MBEDTLS_PRIVATE (
    mbedtls_ctx )
```

7.112.2.4 MBEDTLS_PRIVATE() [4/4]

```
uint32_t MBEDTLS_PRIVATE (
    num_ops )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.113 psa_key_attributes_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- [psa_key_type_t MBEDTLS_PRIVATE](#) (type)
- [psa_key_bits_t MBEDTLS_PRIVATE](#) (bits)
- [psa_key_lifetime_t MBEDTLS_PRIVATE](#) (lifetime)
- [psa_key_policy_t MBEDTLS_PRIVATE](#) (policy)
- [mbedtls_svc_key_id_t MBEDTLS_PRIVATE](#) (id)

7.113.1 Detailed Description

Definition at line 310 of file `crypto_struct.h`.

7.113.2 Member Function Documentation

7.113.2.1 MBEDTLS_PRIVATE() [1/5]

```
psa_key_bits_t MBEDTLS_PRIVATE (
    bits )
```

7.113.2.2 MBEDTLS_PRIVATE() [2/5]

```
mbedtls_svc_key_id_t MBEDTLS_PRIVATE (
    id )
```

7.113.2.3 MBEDTLS_PRIVATE() [3/5]

```
psa_key_lifetime_t MBEDTLS_PRIVATE (
    lifetime )
```

7.113.2.4 MBEDTLS_PRIVATE() [4/5]

```
psa_key_policy_t MBEDTLS_PRIVATE (
    policy )
```

7.113.2.5 MBEDTLS_PRIVATE() [5/5]

```
psa_key_type_t MBEDTLS_PRIVATE (
    type )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.114 psa_key_derivation_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- [psa_algorithm_t MBEDTLS_PRIVATE](#) (alg)
- unsigned int MBEDTLS_PRIVATE(can_output_key) size_t [MBEDTLS_PRIVATE](#) (capacity)
- [psa_driver_key_derivation_context_t MBEDTLS_PRIVATE](#) (ctx)

7.114.1 Detailed Description

Definition at line 245 of file `crypto_struct.h`.

7.114.2 Member Function Documentation

7.114.2.1 MBEDTLS_PRIVATE() [1/3]

```
psa_algorithm_t MBEDTLS_PRIVATE (
    alg )
```

7.114.2.2 MBEDTLS_PRIVATE() [2/3]

```
unsigned int MBEDTLS_PRIVATE (can_output_key) size_t MBEDTLS_PRIVATE (
    capacity )
```

7.114.2.3 MBEDTLS_PRIVATE() [3/3]

```
psa_driver_key_derivation_context_t MBEDTLS_PRIVATE (
    ctx )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.115 psa_key_policy_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- [psa_key_usage_t MBEDTLS_PRIVATE \(usage\)](#)
- [psa_algorithm_t MBEDTLS_PRIVATE \(alg\)](#)
- [psa_algorithm_t MBEDTLS_PRIVATE \(alg2\)](#)

7.115.1 Detailed Description

Definition at line 283 of file [crypto_struct.h](#).

7.115.2 Member Function Documentation

7.115.2.1 MBEDTLS_PRIVATE() [1/3]

```
psa_algorithm_t MBEDTLS_PRIVATE (
    alg )
```

7.115.2.2 MBEDTLS_PRIVATE() [2/3]

```
psa_algorithm_t MBEDTLS_PRIVATE (
    alg2 )
```

7.115.2.3 MBEDTLS_PRIVATE() [3/3]

```
psa_key_usage_t MBEDTLS_PRIVATE (
    usage )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.116 psa_mac_operation_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- uint8_t [MBEDTLS_PRIVATE](#) (mac_size)
- unsigned int [MBEDTLS_PRIVATE](#)(is_sign) [psa_driver_mac_context_t](#) [MBEDTLS_PRIVATE](#) (ctx)

7.116.1 Detailed Description

Definition at line 172 of file `crypto_struct.h`.

7.116.2 Member Function Documentation

7.116.2.1 `MBEDTLS_PRIVATE()` [1/3]

```
unsigned int MBEDTLS_PRIVATE (is_sign) psa\_driver\_mac\_context\_t MBEDTLS_PRIVATE (
    ctx )
```

7.116.2.2 `MBEDTLS_PRIVATE()` [2/3]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_driver_wrappers.h`. ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

7.116.2.3 `MBEDTLS_PRIVATE()` [3/3]

```
uint8_t MBEDTLS_PRIVATE (
    mac_size )
```

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.117 `psa_msg_t` Struct Reference

```
#include <interface/include/psa/service.h>
```

Data Fields

- `int32_t` [type](#)
- [psa_handle_t](#) [handle](#)
- `int32_t` [client_id](#)
- `void *` [rhandle](#)
- `size_t` [in_size](#) [[PSA_MAX_IOVEC](#)]
- `size_t` [out_size](#) [[PSA_MAX_IOVEC](#)]

7.117.1 Detailed Description

Describe a message received by an RoT Service after calling [psa_get\(\)](#).

Definition at line 68 of file `service.h`.

7.117.2 Field Documentation

7.117.2.1 client_id

```
int32_t client_id
```

Definition at line 77 of file service.h.

7.117.2.2 handle

```
psa_handle_t handle
```

Definition at line 74 of file service.h.

7.117.2.3 in_size

```
size_t in_size[PSA_MAX_IOVEC]
```

Definition at line 88 of file service.h.

7.117.2.4 out_size

```
size_t out_size[PSA_MAX_IOVEC]
```

Definition at line 91 of file service.h.

7.117.2.5 rhandle

```
void* rhandle
```

Definition at line 83 of file service.h.

7.117.2.6 type

`int32_t type`

Definition at line 69 of file `service.h`.

The documentation for this struct was generated from the following file:

- `interface/include/psa/service.h`

7.118 psa_outvec Struct Reference

```
#include <interface/include/psa/client.h>
```

Data Fields

- `void * base`
- `size_t len`

7.118.1 Detailed Description

A writable output memory region provided to an RoT Service.

Definition at line 96 of file `client.h`.

7.118.2 Field Documentation

7.118.2.1 base

`void* base`

the start address of the memory buffer

Definition at line 97 of file `client.h`.

7.118.2.2 len

`size_t len`

the size in bytes

Definition at line 98 of file `client.h`.

The documentation for this struct was generated from the following file:

- `interface/include/psa/client.h`

7.119 psa_pake_cipher_suite_s Struct Reference

```
#include <interface/include/psa/crypto_extra.h>
```

Data Fields

- [psa_algorithm_t](#) algorithm
- [psa_pake_primitive_type_t](#) type
- [psa_pake_family_t](#) family
- [uint16_t](#) bits
- [uint32_t](#) key_confirmation

7.119.1 Detailed Description

Definition at line 1167 of file `crypto_extra.h`.

7.119.2 Field Documentation

7.119.2.1 algorithm

[psa_algorithm_t](#) algorithm

Definition at line 1168 of file `crypto_extra.h`.

7.119.2.2 bits

[uint16_t](#) bits

Definition at line 1171 of file `crypto_extra.h`.

7.119.2.3 family

[psa_pake_family_t](#) family

Definition at line 1170 of file `crypto_extra.h`.

7.119.2.4 key_confirmation

uint32_t key_confirmation

Definition at line 1172 of file crypto_extra.h.

7.119.2.5 type

psa_pake_primitive_type_t type

Definition at line 1169 of file crypto_extra.h.

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_extra.h](#)

7.120 psa_pake_operation_s Struct Reference

```
#include <interface/include/psa/crypto_extra.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- [psa_algorithm_t](#) [MBEDTLS_PRIVATE](#) (alg)
- [psa_pake_primitive_t](#) [MBEDTLS_PRIVATE](#) (primitive)
- uint8_t [MBEDTLS_PRIVATE](#) (stage)
- union {
 } [MBEDTLS_PRIVATE](#) (computation_stage)
- union {
 } [MBEDTLS_PRIVATE](#) (data)

7.120.1 Detailed Description

Definition at line 1231 of file crypto_extra.h.

7.120.2 Member Function Documentation

7.120.2.1 MBEDTLS_PRIVATE() [1/6]

```
psa_algorithm_t MBEDTLS_PRIVATE (
    alg )
```

7.120.2.2 MBEDTLS_PRIVATE() [2/6]

```
union psa_pake_operation_s::@6 MBEDTLS_PRIVATE (
    computation_stage )
```

7.120.2.3 MBEDTLS_PRIVATE() [3/6]

```
union psa_pake_operation_s::@7 MBEDTLS_PRIVATE (
    data )
```

7.120.2.4 MBEDTLS_PRIVATE() [4/6]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_crypto_driver_↔wrappers.h` ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

7.120.2.5 MBEDTLS_PRIVATE() [5/6]

```
psa_pake_primitive_t MBEDTLS_PRIVATE (
    primitive )
```

7.120.2.6 MBEDTLS_PRIVATE() [6/6]

```
uint8_t MBEDTLS_PRIVATE (
    stage )
```

The documentation for this struct was generated from the following file:

- `interface/include/psa/crypto_extra.h`

7.121 psa_sign_hash_interruptible_operation_s Struct Reference

The context for PSA interruptible hash signing.

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- [psa_driver_sign_hash_interruptible_context_t](#) [MBEDTLS_PRIVATE](#) (ctx)
- unsigned int [MBEDTLS_PRIVATE](#)(error_occurred) uint32_t [MBEDTLS_PRIVATE](#) (num_ops)

7.121.1 Detailed Description

The context for PSA interruptible hash signing.

Definition at line 455 of file `crypto_struct.h`.

7.121.2 Member Function Documentation

7.121.2.1 MBEDTLS_PRIVATE() [1/3]

```
psa_driver_sign_hash_interruptible_context_t MBEDTLS_PRIVATE (
    ctx )
```

7.121.2.2 MBEDTLS_PRIVATE() [2/3]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_crypto_driver_↔wrappers.h` ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

7.121.2.3 MBEDTLS_PRIVATE() [3/3]

```
unsigned int MBEDTLS_PRIVATE (error_occurred) uint32_t MBEDTLS_PRIVATE (
    num_ops )
```

The documentation for this struct was generated from the following file:

- `interface/include/psa/crypto_struct.h`

7.122 `psa_storage_info_t` Struct Reference

```
#include <interface/include/psa/storage_common.h>
```

Data Fields

- `size_t` [capacity](#)
- `size_t` [size](#)
- `psa_storage_create_flags_t` [flags](#)

7.122.1 Detailed Description

Definition at line 34 of file `storage_common.h`.

7.122.2 Field Documentation

7.122.2.1 `capacity`

```
size_t capacity
```

Definition at line 35 of file `storage_common.h`.

7.122.2.2 `flags`

```
psa_storage_create_flags_t flags
```

Definition at line 37 of file `storage_common.h`.

7.122.2.3 `size`

```
size_t size
```

Definition at line 36 of file `storage_common.h`.

The documentation for this struct was generated from the following file:

- `interface/include/psa/storage_common.h`

7.123 psa_verify_hash_interruptible_operation_s Struct Reference

The context for PSA interruptible hash verification.

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- [psa_driver_verify_hash_interruptible_context_t](#) [MBEDTLS_PRIVATE](#) (ctx)
- unsigned int [MBEDTLS_PRIVATE](#)(error_occurred) uint32_t [MBEDTLS_PRIVATE](#) (num_ops)

7.123.1 Detailed Description

The context for PSA interruptible hash verification.

Definition at line 493 of file `crypto_struct.h`.

7.123.2 Member Function Documentation

7.123.2.1 MBEDTLS_PRIVATE() [1/3]

```
psa_driver_verify_hash_interruptible_context_t MBEDTLS_PRIVATE (
    ctx )
```

7.123.2.2 MBEDTLS_PRIVATE() [2/3]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_crypto_driver_↔wrappers.h` ID value zero means the context is not valid or not assigned to any driver (i.e. none of the driver contexts are active).

7.123.2.3 MBEDTLS_PRIVATE() [3/3]

```
unsigned int MBEDTLS_PRIVATE (error_occurred) uint32_t MBEDTLS_PRIVATE (
    num_ops )
```

The documentation for this struct was generated from the following file:

- `interface/include/psa/crypto_struct.h`

7.124 psa_xof_operation_s Struct Reference

```
#include <interface/include/psa/crypto_struct.h>
```

Public Member Functions

- unsigned int [MBEDTLS_PRIVATE](#) (id)
- [psa_driver_xof_context_t](#) [MBEDTLS_PRIVATE](#) (ctx)

Data Fields

- unsigned [requires_context](#): 1
- unsigned [allows_context](#): 1
- unsigned [active](#): 1
- unsigned [has_context](#): 1
- unsigned [has_input](#): 1
- unsigned [has_output](#): 1

7.124.1 Detailed Description

Definition at line 90 of file `crypto_struct.h`.

7.124.2 Member Function Documentation

7.124.2.1 MBEDTLS_PRIVATE() [1/2]

```
psa_driver_xof_context_t MBEDTLS_PRIVATE (
    ctx )
```

7.124.2.2 MBEDTLS_PRIVATE() [2/2]

```
unsigned int MBEDTLS_PRIVATE (
    id )
```

Unique ID indicating which driver got assigned to do the operation. Since driver contexts are driver-specific, swapping drivers halfway through the operation is not supported. ID values are auto-generated in `psa_driver_wrappers.h`. ID value zero means the context is not valid or not assigned to any driver (i.e. the driver context is not active, in use).

7.124.3 Field Documentation

7.124.3.1 active

`unsigned active`

Definition at line 107 of file `crypto_struct.h`.

7.124.3.2 allows_context

`unsigned allows_context`

Definition at line 104 of file `crypto_struct.h`.

7.124.3.3 has_context

`unsigned has_context`

Definition at line 108 of file `crypto_struct.h`.

7.124.3.4 has_input

`unsigned has_input`

Definition at line 109 of file `crypto_struct.h`.

7.124.3.5 has_output

`unsigned has_output`

Definition at line 110 of file `crypto_struct.h`.

7.124.3.6 requires_context

`unsigned requires_context`

Definition at line 103 of file `crypto_struct.h`.

The documentation for this struct was generated from the following file:

- [interface/include/psa/crypto_struct.h](#)

7.125 rot_psa_its_storage_info_t Struct Reference

Data Fields

- [rot_size_t](#) capacity
- [rot_size_t](#) size
- [psa_storage_create_flags_t](#) flags

7.125.1 Detailed Description

Definition at line 13 of file `tfm_its_api.c`.

7.125.2 Field Documentation

7.125.2.1 capacity

`rot_size_t` capacity

Definition at line 14 of file `tfm_its_api.c`.

7.125.2.2 flags

`psa_storage_create_flags_t` flags

Definition at line 16 of file `tfm_its_api.c`.

7.125.2.3 size

`rot_size_t` size

Definition at line 15 of file `tfm_its_api.c`.

The documentation for this struct was generated from the following file:

- `interface/src/tfm_its_api.c`

7.126 rot_psa_ps_storage_info_t Struct Reference

Data Fields

- [rot_size_t](#) capacity
- [rot_size_t](#) size
- [psa_storage_create_flags_t](#) flags

7.126.1 Detailed Description

Definition at line 13 of file tfm_ps_api.c.

7.126.2 Field Documentation

7.126.2.1 capacity

[rot_size_t](#) capacity

Definition at line 14 of file tfm_ps_api.c.

7.126.2.2 flags

[psa_storage_create_flags_t](#) flags

Definition at line 16 of file tfm_ps_api.c.

7.126.2.3 size

[rot_size_t](#) size

Definition at line 15 of file tfm_ps_api.c.

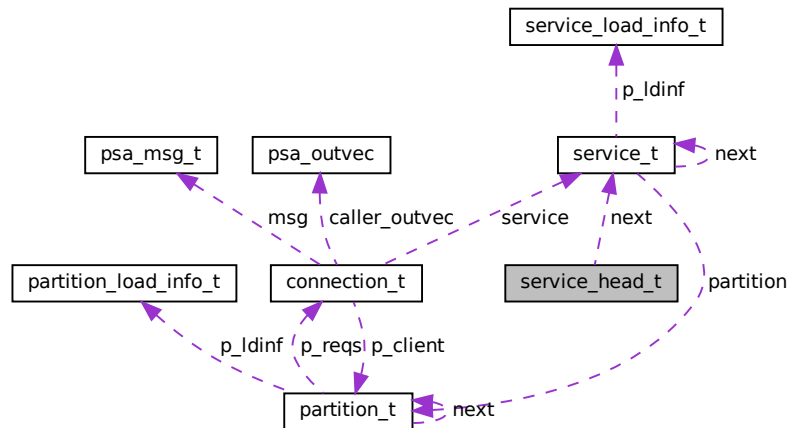
The documentation for this struct was generated from the following file:

- [interface/src/tfm_ps_api.c](#)

7.127 service_head_t Struct Reference

```
#include <secure_fw/spm/include/load/spm_load_api.h>
```

Collaboration diagram for service_head_t:



Data Fields

- `uint32_t reserved`
- `struct service_t * next`

7.127.1 Detailed Description

Definition at line 55 of file `spm_load_api.h`.

7.127.2 Field Documentation

7.127.2.1 next

```
struct service_t* next
```

Definition at line 57 of file `spm_load_api.h`.

7.127.2.2 reserved

```
uint32_t reserved
```

Definition at line 56 of file `spm_load_api.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/spm/include/load/spm_load_api.h`

7.128 service_load_info_t Struct Reference

```
#include <secure_fw/spm/include/load/service_defs.h>
```

Data Fields

- `uintptr_t` [name_strid](#)
- `uint32_t` [sid](#)
- `uint32_t` [flags](#)
- `uint32_t` [version](#)
- `uintptr_t` [sfn](#)

7.128.1 Detailed Description

Definition at line 47 of file `service_defs.h`.

7.128.2 Field Documentation

7.128.2.1 flags

```
uint32_t flags
```

Definition at line 50 of file `service_defs.h`.

7.128.2.2 name_strid

```
uintptr_t name_strid
```

Definition at line 48 of file `service_defs.h`.

7.128.2.3 sfn

```
uintptr_t sfn
```

Definition at line 52 of file service_defs.h.

7.128.2.4 sid

```
uint32_t sid
```

Definition at line 49 of file service_defs.h.

7.128.2.5 version

```
uint32_t version
```

Definition at line 51 of file service_defs.h.

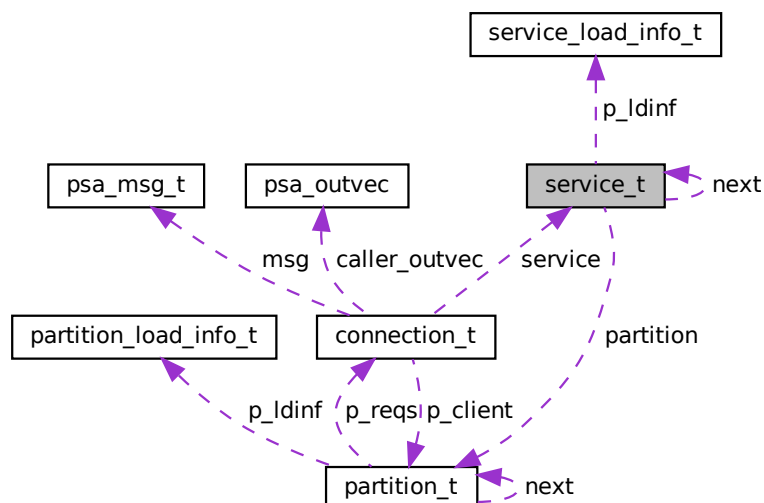
The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/load/service_defs.h](#)

7.129 service_t Struct Reference

```
#include <secure_fw/spm/core/spm.h>
```

Collaboration diagram for service_t:



Data Fields

- const struct [service_load_info_t](#) * p_ldinf
- struct [partition_t](#) * partition
- struct [service_t](#) * next

7.129.1 Detailed Description

Definition at line 123 of file spm.h.

7.129.2 Field Documentation

7.129.2.1 next

```
struct service\_t* next
```

Definition at line 126 of file spm.h.

7.129.2.2 p_ldinf

```
const struct service\_load\_info\_t* p_ldinf
```

Definition at line 124 of file spm.h.

7.129.2.3 partition

```
struct partition\_t* partition
```

Definition at line 125 of file spm.h.

The documentation for this struct was generated from the following file:

- [secure_fw/spm/core/spm.h](#)

7.130 shared_data_tlv_entry Struct Reference

```
#include <secure_fw/spm/include/boot/tfm_boot_status.h>
```

Data Fields

- uint16_t [tlv_type](#)
- uint16_t [tlv_len](#)

7.130.1 Detailed Description

Shared data TLV entry header format. All fields in little endian.

| **tlv_type(16)** | **tlv_len(16)** |

| **Raw data** |

Definition at line 179 of file tfm_boot_status.h.

7.130.2 Field Documentation

7.130.2.1 tlv_len

uint16_t tlv_len

Definition at line 181 of file tfm_boot_status.h.

7.130.2.2 tlv_type

uint16_t tlv_type

Definition at line 180 of file tfm_boot_status.h.

The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/boot/tfm_boot_status.h](#)

7.131 shared_data_tlv_header Struct Reference

```
#include <secure_fw/spm/include/boot/tfm_boot_status.h>
```

Data Fields

- uint16_t [tlv_magic](#)
- uint16_t [tlv_tot_len](#)

7.131.1 Detailed Description

Shared data TLV header. All fields in little endian.

| **tlv_magic(16)** | **tlv_tot_len(16)** |

Definition at line 163 of file tfm_boot_status.h.

7.131.2 Field Documentation

7.131.2.1 tlv_magic

uint16_t tlv_magic

Definition at line 164 of file tfm_boot_status.h.

7.131.2.2 tlv_tot_len

uint16_t tlv_tot_len

Definition at line 165 of file tfm_boot_status.h.

The documentation for this struct was generated from the following file:

- [secure_fw/spm/include/boot/tfm_boot_status.h](#)

7.132 tfm_additional_context_t Struct Reference

```
#include <secure_fw/spm/include/tfm_arch.h>
```

Data Fields

- uint32_t [integ_sign](#)
- uint32_t [reserved](#)
- uint32_t [callee](#) [8]

7.132.1 Detailed Description

Definition at line 101 of file tfm_arch.h.

7.132.2 Field Documentation

7.132.2.1 callee

uint32_t callee[8]

Definition at line 104 of file tfm_arch.h.

7.132.2.2 integ_sign

uint32_t integ_sign

Definition at line 102 of file tfm_arch.h.

7.132.2.3 reserved

uint32_t reserved

Definition at line 103 of file tfm_arch.h.

The documentation for this struct was generated from the following file:

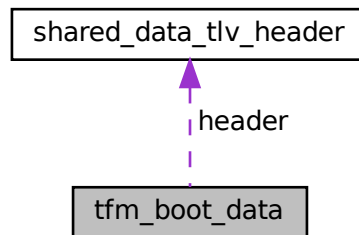
- [secure_fw/spm/include/tfm_arch.h](#)

7.133 tfm_boot_data Struct Reference

Store the data for the runtime SW.

```
#include <secure_fw/spm/include/boot/tfm_boot_status.h>
```

Collaboration diagram for `tfm_boot_data`:



Data Fields

- struct [shared_data_tlv_header](#) `header`
- `uint8_t` [data](#) []

7.133.1 Detailed Description

Store the data for the runtime SW.

Definition at line 189 of file `tfm_boot_status.h`.

7.133.2 Field Documentation

7.133.2.1 data

```
uint8_t data[]
```

Definition at line 191 of file `tfm_boot_status.h`.

7.133.2.2 header

```
struct shared_data_tlv_header header
```

Definition at line 190 of file `tfm_boot_status.h`.

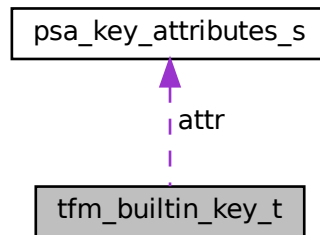
The documentation for this struct was generated from the following file:

- `secure_fw/spm/include/boot/tfm_boot_status.h`

7.134 tfm_builtin_key_t Struct Reference

A structure which describes a builtin key slot.

Collaboration diagram for tfm_builtin_key_t:



Data Fields

- `uint8_t key [(48)]`
- `size_t key_len`
- `psa_key_attributes_t attr`
- `uint32_t is_loaded`

7.134.1 Detailed Description

A structure which describes a builtin key slot.
Definition at line 31 of file `tfm_builtin_key_loader.c`.

7.134.2 Field Documentation

7.134.2.1 attr

`psa_key_attributes_t attr`
Key attributes associated to the key
Definition at line 34 of file `tfm_builtin_key_loader.c`.

7.134.2.2 is_loaded

`uint32_t is_loaded`
Boolean indicating whether the slot is being used
Definition at line 35 of file `tfm_builtin_key_loader.c`.

7.134.2.3 key

`uint8_t key [(48)]`
Raw key material, 4-byte aligned
Definition at line 32 of file `tfm_builtin_key_loader.c`.

7.134.2.4 key_len

size_t key_len

Size of the key material held in the key buffer

Definition at line 33 of file tfm_builtin_key_loader.c.

The documentation for this struct was generated from the following file:

- [secure_fw/partitions/crypto/psa_driver_api/tfm_builtin_key_loader.c](#)

7.135 tfm_crypto_aead_pack_input Struct Reference

This type is used to overcome a limitation in the number of maximum IOVECs that can be used especially in `psa_aead_encrypt` and `psa_aead_decrypt`. By using this type we pack the nonce and the actual nonce_length at part of the same structure.

`#include <interface/include/tfm_crypto_defs.h>`

Data Fields

- `uint8_t nonce [(16u)]`
- `uint32_t nonce_length`

7.135.1 Detailed Description

This type is used to overcome a limitation in the number of maximum IOVECs that can be used especially in `psa_aead_encrypt` and `psa_aead_decrypt`. By using this type we pack the nonce and the actual nonce_length at part of the same structure.

Definition at line 34 of file `tfm_crypto_defs.h`.

7.135.2 Field Documentation

7.135.2.1 nonce

`uint8_t nonce [(16u)]`

Definition at line 35 of file `tfm_crypto_defs.h`.

7.135.2.2 nonce_length

`uint32_t nonce_length`

Definition at line 36 of file `tfm_crypto_defs.h`.

The documentation for this struct was generated from the following file:

- [interface/include/tfm_crypto_defs.h](#)

7.136 tfm_crypto_key_id_s Struct Reference

The type which describes a key identifier to the Crypto service. The key identifiers must clearly provide a dedicated indication of the entity owner which owns the key.

`#include <secure_fw/partitions/crypto/tfm_crypto_key.h>`

Data Fields

- `psa_key_id_t key_id`
- `int32_t owner`

7.136.1 Detailed Description

The type which describes a key identifier to the Crypto service. The key identifiers must clearly provide a dedicated indication of the entity owner which owns the key.

A macro to perform static initialisation of a structure

Definition at line 23 of file `tfm_crypto_key.h`.

7.136.2 Field Documentation

7.136.2.1 key_id

`psa_key_id_t` `key_id`

Key ID for the key itself

Definition at line 24 of file `tfm_crypto_key.h`.

7.136.2.2 owner

`int32_t` `owner`

ID of the entity owning the key

Definition at line 25 of file `tfm_crypto_key.h`.

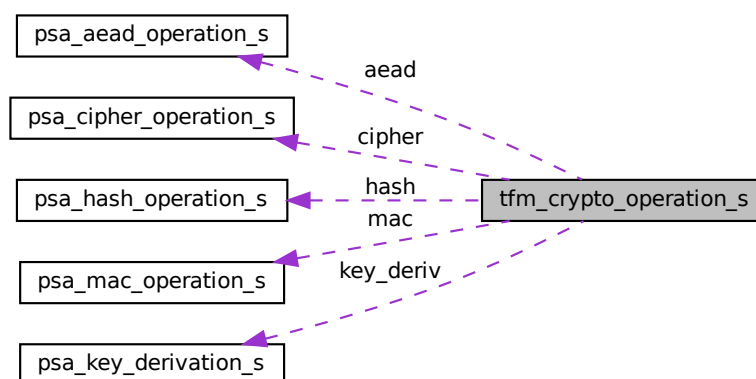
The documentation for this struct was generated from the following file:

- `secure_fw/partitions/crypto/tfm_crypto_key.h`

7.137 tfm_crypto_operation_s Struct Reference

A type describing the context stored in Secure memory by the TF-M Crypto service to support multipart calls on secure side.

Collaboration diagram for `tfm_crypto_operation_s`:



Data Fields

- `uint32_t` `in_use`
- `int32_t` `owner`
- `enum` `tfm_crypto_operation_type` `type`

- union {
 - psa_cipher_operation_t cipher
 - psa_mac_operation_t mac
 - psa_hash_operation_t hash
 - psa_key_derivation_operation_t key_deriv
 - psa_aead_operation_t aead
- } operation

7.137.1 Detailed Description

A type describing the context stored in Secure memory by the TF-M Crypto service to support multipart calls on secure side.

Definition at line 35 of file crypto_alloc.c.

7.137.2 Field Documentation

7.137.2.1 aead

psa_aead_operation_t aead

AEAD operation context

Definition at line 46 of file crypto_alloc.c.

7.137.2.2 cipher

psa_cipher_operation_t cipher

Cipher operation context

Definition at line 42 of file crypto_alloc.c.

7.137.2.3 hash

psa_hash_operation_t hash

Hash operation context

Definition at line 44 of file crypto_alloc.c.

7.137.2.4 in_use

uint32_t in_use

Indicates if the operation is in use

Definition at line 36 of file crypto_alloc.c.

7.137.2.5 key_deriv

psa_key_derivation_operation_t key_deriv

Key derivation operation context

Definition at line 45 of file crypto_alloc.c.

7.137.2.6 mac

psa_mac_operation_t mac

MAC operation context

Definition at line 43 of file crypto_alloc.c.

7.137.2.7 operation

```
union { ... } operation
```

7.137.2.8 owner

```
int32_t owner
```

Indicates an ID of the owner of the context

Definition at line 37 of file `crypto_alloc.c`.

7.137.2.9 type

```
enum tfm_crypto_operation_type type
```

Type of the operation

Definition at line 40 of file `crypto_alloc.c`.

The documentation for this struct was generated from the following file:

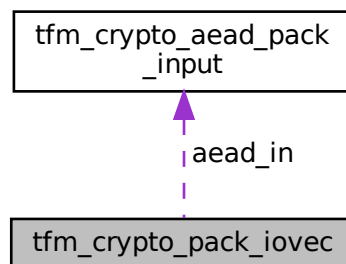
- `secure_fw/partitions/crypto/crypto_alloc.c`

7.138 tfm_crypto_pack_iovec Struct Reference

Structure used to pack non-pointer types in a call to PSA Crypto APIs.

```
#include <interface/include/tfm_crypto_defs.h>
```

Collaboration diagram for `tfm_crypto_pack_iovec`:



Data Fields

- `psa_key_id_t` `key_id`
 - `psa_algorithm_t` `alg`
 - `uint32_t` `op_handle`
 - `uint32_t` `ad_length`
 - `uint32_t` `plaintext_length`
 - `struct tfm_crypto_aead_pack_input` `aead_in`
 - `uint16_t` `function_id`
 - `uint16_t` `step`
 - `union` {
 - `uint32_t` `capacity`
 - `uint64_t` `value`
- ```
};
```

### 7.138.1 Detailed Description

Structure used to pack non-pointer types in a call to PSA Crypto APIs.  
Definition at line 43 of file `tfm_crypto_defs.h`.

### 7.138.2 Field Documentation

#### 7.138.2.1 "@9

```
union { ... }
```

#### 7.138.2.2 `ad_length`

```
uint32_t ad_length
```

Additional Data length for multipart AEAD  
Definition at line 49 of file `tfm_crypto_defs.h`.

#### 7.138.2.3 `aead_in`

```
struct tfm_crypto_aead_pack_input aead_in
```

Packs AEAD-related inputs  
Definition at line 52 of file `tfm_crypto_defs.h`.

#### 7.138.2.4 `alg`

```
psa_algorithm_t alg
```

Algorithm  
Definition at line 45 of file `tfm_crypto_defs.h`.

#### 7.138.2.5 `capacity`

```
uint32_t capacity
```

Key derivation capacity  
Definition at line 60 of file `tfm_crypto_defs.h`.

#### 7.138.2.6 `function_id`

```
uint16_t function_id
```

Used to identify the function in the API dispatcher to the service backend See `tfm_crypto_func_sid` for detail  
Definition at line 54 of file `tfm_crypto_defs.h`.

#### 7.138.2.7 `key_id`

```
psa_key_id_t key_id
```

Key id  
Definition at line 44 of file `tfm_crypto_defs.h`.

### 7.138.2.8 op\_handle

uint32\_t op\_handle

Client context handle associated to a multipart operation

Definition at line 46 of file tfm\_crypto\_defs.h.

### 7.138.2.9 plaintext\_length

uint32\_t plaintext\_length

Plaintext length for multipart AEAD

Definition at line 50 of file tfm\_crypto\_defs.h.

### 7.138.2.10 step

uint16\_t step

Key derivation step

Definition at line 58 of file tfm\_crypto\_defs.h.

### 7.138.2.11 value

uint64\_t value

Key derivation integer for update

Definition at line 61 of file tfm\_crypto\_defs.h.

The documentation for this struct was generated from the following file:

- interface/include/[tfm\\_crypto\\_defs.h](#)

## 7.139 tfm\_fwu\_ctx\_s Struct Reference

### Data Fields

- [psa\\_status\\_t](#) error
- [uint8\\_t](#) component\_state
- [bool](#) in\_use

### 7.139.1 Detailed Description

Definition at line 24 of file tfm\_fwu\_req\_mngr.c.

### 7.139.2 Field Documentation

#### 7.139.2.1 component\_state

uint8\_t component\_state

Definition at line 26 of file tfm\_fwu\_req\_mngr.c.

#### 7.139.2.2 error

[psa\\_status\\_t](#) error

Definition at line 25 of file tfm\_fwu\_req\_mngr.c.

### 7.139.2.3 in\_use

`bool in_use`

Definition at line 27 of file `tfm_fwu_req_mngr.c`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/firmware_update/tfm_fwu_req_mngr.c`

## 7.140 tfm\_fwu\_mcuboot\_ctx\_s Struct Reference

### Data Fields

- `const struct flash_area * fap`
- `size_t loaded_size`

### 7.140.1 Detailed Description

Definition at line 41 of file `tfm_mcuboot_fw.c`.

### 7.140.2 Field Documentation

#### 7.140.2.1 fap

`const struct flash_area* fap`

Definition at line 43 of file `tfm_mcuboot_fw.c`.

#### 7.140.2.2 loaded\_size

`size_t loaded_size`

Definition at line 46 of file `tfm_mcuboot_fw.c`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/firmware_update/bootloader/mcuboot/tfm_mcuboot_fw.c`

## 7.141 tfm\_ns\_ctx\_t Struct Reference

```
#include <secure_fw/spm/ns_client_ext/tfm_ns_ctx.h>
```

### Data Fields

- `int32_t nsid`
- `uint8_t gid`
- `uint8_t tid`
- `uint8_t ref_cnt`

### 7.141.1 Detailed Description

Definition at line 19 of file `tfm_ns_ctx.h`.

### 7.141.2 Field Documentation

### 7.141.2.1 gid

uint8\_t gid

Definition at line 21 of file tfm\_ns\_ctx.h.

### 7.141.2.2 nsid

int32\_t nsid

Definition at line 20 of file tfm\_ns\_ctx.h.

### 7.141.2.3 ref\_cnt

uint8\_t ref\_cnt

Definition at line 23 of file tfm\_ns\_ctx.h.

### 7.141.2.4 tid

uint8\_t tid

Definition at line 22 of file tfm\_ns\_ctx.h.

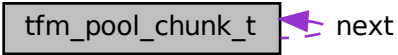
The documentation for this struct was generated from the following file:

- secure\_fw/spm/ns\_client\_ext/[tfm\\_ns\\_ctx.h](#)

## 7.142 tfm\_pool\_chunk\_t Struct Reference

```
#include <secure_fw/spm/core/tfm_pools.h>
```

Collaboration diagram for tfm\_pool\_chunk\_t:



```
graph LR; A[tfm_pool_chunk_t] --> A; A --> B[next];
```

### Data Fields

- struct [tfm\\_pool\\_chunk\\_t](#) \* [next](#)
- uint32\_t [magic](#)
- uint8\_t [data](#) []

### 7.142.1 Detailed Description

Definition at line 20 of file tfm\_pools.h.

### 7.142.2 Field Documentation

### 7.142.2.1 data

```
uint8_t data[]
```

Definition at line 23 of file tfm\_pools.h.

### 7.142.2.2 magic

```
uint32_t magic
```

Definition at line 22 of file tfm\_pools.h.

### 7.142.2.3 next

```
struct tfm_pool_chunk_t* next
```

Definition at line 21 of file tfm\_pools.h.

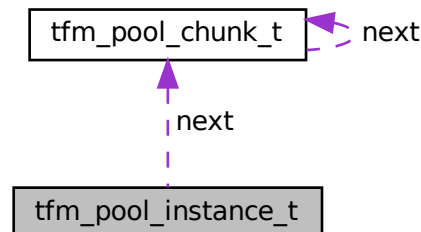
The documentation for this struct was generated from the following file:

- [secure\\_fw/spm/core/tfm\\_pools.h](#)

## 7.143 tfm\_pool\_instance\_t Struct Reference

```
#include <secure_fw/spm/core/tfm_pools.h>
```

Collaboration diagram for tfm\_pool\_instance\_t:



### Data Fields

- struct [tfm\\_pool\\_chunk\\_t](#) \* [next](#)
- size\_t [chunksz](#)
- size\_t [pool\\_sz](#)
- uint8\_t [chunks](#) []

### 7.143.1 Detailed Description

Definition at line 26 of file tfm\_pools.h.

### 7.143.2 Field Documentation



### 7.143.2.1 chunks

```
uint8_t chunks[]
```

Definition at line 30 of file `tfm_pools.h`.

### 7.143.2.2 chunksz

```
size_t chunksz
```

Definition at line 28 of file `tfm_pools.h`.

### 7.143.2.3 next

```
struct tfm_pool_chunk_t* next
```

Definition at line 27 of file `tfm_pools.h`.

### 7.143.2.4 pool\_sz

```
size_t pool_sz
```

Definition at line 29 of file `tfm_pools.h`.

The documentation for this struct was generated from the following file:

- `secure_fw/spm/core/tfm_pools.h`

## 7.144 tfm\_state\_context\_t Struct Reference

```
#include <secure_fw/spm/include/tfm_arch.h>
```

### Data Fields

- `uint32_t r0`
- `uint32_t r1`
- `uint32_t r2`
- `uint32_t r3`
- `uint32_t r12`
- `uint32_t lr`
- `uint32_t ra`
- `uint32_t xpsr`

### 7.144.1 Detailed Description

Definition at line 89 of file `tfm_arch.h`.

### 7.144.2 Field Documentation

#### 7.144.2.1 lr

```
uint32_t lr
```

Definition at line 95 of file `tfm_arch.h`.

#### 7.144.2.2 r0

```
uint32_t r0
```

Definition at line 90 of file `tfm_arch.h`.

**7.144.2.3 r1**

```
uint32_t r1
```

Definition at line 91 of file tfm\_arch.h.

**7.144.2.4 r12**

```
uint32_t r12
```

Definition at line 94 of file tfm\_arch.h.

**7.144.2.5 r2**

```
uint32_t r2
```

Definition at line 92 of file tfm\_arch.h.

**7.144.2.6 r3**

```
uint32_t r3
```

Definition at line 93 of file tfm\_arch.h.

**7.144.2.7 ra**

```
uint32_t ra
```

Definition at line 96 of file tfm\_arch.h.

**7.144.2.8 xpsr**

```
uint32_t xpsr
```

Definition at line 97 of file tfm\_arch.h.

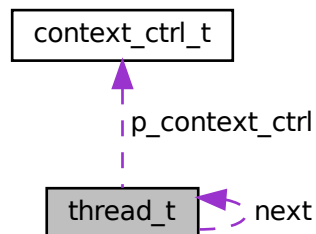
The documentation for this struct was generated from the following file:

- [secure\\_fw/spm/include/tfm\\_arch.h](#)

**7.145 thread\_t Struct Reference**

```
#include <secure_fw/spm/core/thread.h>
```

Collaboration diagram for thread\_t:



## Data Fields

- uint16\_t [priority](#)
- uint8\_t [state](#)
- uint16\_t [flags](#)
- struct [context\\_ctrl\\_t](#) \* [p\\_context\\_ctrl](#)
- struct [thread\\_t](#) \* [next](#)

### 7.145.1 Detailed Description

Definition at line 44 of file thread.h.

### 7.145.2 Field Documentation

#### 7.145.2.1 flags

uint16\_t flags

Definition at line 47 of file thread.h.

#### 7.145.2.2 next

struct [thread\\_t](#)\* next

Definition at line 49 of file thread.h.

#### 7.145.2.3 p\_context\_ctrl

struct [context\\_ctrl\\_t](#)\* p\_context\_ctrl

Definition at line 48 of file thread.h.

#### 7.145.2.4 priority

uint16\_t priority

Definition at line 45 of file thread.h.

#### 7.145.2.5 state

uint8\_t state

Definition at line 46 of file thread.h.

The documentation for this struct was generated from the following file:

- [secure\\_fw/spm/core/thread.h](#)

## 7.146 u64\_in\_u32\_regs\_t Union Reference

```
#include <secure_fw/spm/include/aapcs_local.h>
```

## Data Fields

- struct {
  - uint32\_t [r0](#)
  - uint32\_t [r1](#) } [u32\\_regs](#)
- uint64\_t [u64\\_val](#)

### 7.146.1 Detailed Description

Definition at line 37 of file `aapcs_local.h`.

### 7.146.2 Field Documentation

#### 7.146.2.1 `r0`

```
uint32_t r0
```

Definition at line 39 of file `aapcs_local.h`.

#### 7.146.2.2 `r1`

```
uint32_t r1
```

Definition at line 40 of file `aapcs_local.h`.

#### 7.146.2.3 `u32_regs`

```
struct { ... } u32_regs
```

#### 7.146.2.4 `u64_val`

```
uint64_t u64_val
```

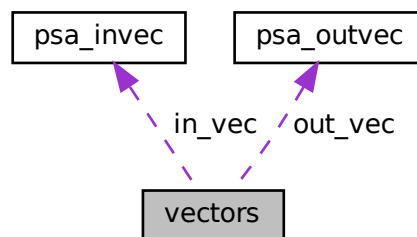
Definition at line 43 of file `aapcs_local.h`.

The documentation for this union was generated from the following file:

- `secure_fw/spm/include/aapcs_local.h`

## 7.147 `vectors` Struct Reference

Collaboration diagram for `vectors`:



### Data Fields

- `psa_invec in_vec` [`PSA_MAX_IOVEC`]
- `psa_outvec out_vec` [`PSA_MAX_IOVEC`]
- `size_t out_len`
- `bool in_use`

### 7.147.1 Detailed Description

Definition at line 47 of file `tfm_spe_mailbox.c`.

### 7.147.2 Field Documentation

#### 7.147.2.1 `in_use`

`bool in_use`

Definition at line 51 of file `tfm_spe_mailbox.c`.

#### 7.147.2.2 `in_vec`

`psa_invec in_vec[PSA_MAX_IOVEC]`

Definition at line 48 of file `tfm_spe_mailbox.c`.

#### 7.147.2.3 `out_len`

`size_t out_len`

Definition at line 50 of file `tfm_spe_mailbox.c`.

#### 7.147.2.4 `out_vec`

`psa_outvec out_vec[PSA_MAX_IOVEC]`

Definition at line 49 of file `tfm_spe_mailbox.c`.

The documentation for this struct was generated from the following file:

- `secure_fw/partitions/ns_agent_mailbox/tfm_spe_mailbox.c`



## Chapter 8

# File Documentation

### 8.1 docs/doxygen/mainpage.dox File Reference

### 8.2 interface/dir\_interface.dox File Reference

### 8.3 interface/include/crypto\_keys/tfm\_builtin\_key\_ids.h File Reference

#### Enumerations

- enum `tfm_builtin_key_id_t` {  
    `TFM_BUILTIN_KEY_ID_MIN` = 0x7FFF815Bu, `TFM_BUILTIN_KEY_ID_HUK`, `TFM_BUILTIN_KEY_ID_IAK`,  
    `TFM_BUILTIN_KEY_ID_PLAT_SPECIFIC_MIN` = 0x7FFF816Bu,  
    `TFM_BUILTIN_KEY_ID_MAX` = 0x7FFF817Bu, `TFM_BUILTIN_KEY_ID_MIN` = ((psa\_key\_id\_t)0x7fff0000),  
    `TFM_BUILTIN_KEY_ID_HUK` = ((psa\_key\_id\_t)0x7fff0000), `TFM_BUILTIN_KEY_ID_IAK`,  
    `TFM_BUILTIN_KEY_ID_MAX` }

*The persistent key identifiers for TF-M builtin keys.*

#### 8.3.1 Enumeration Type Documentation

##### 8.3.1.1 `tfm_builtin_key_id_t`

enum `tfm_builtin_key_id_t`

The persistent key identifiers for TF-M builtin keys.

#### Note

The value of `TFM_BUILTIN_KEY_ID_MIN` (and therefore of the whole range) is completely arbitrary except for being inside the PSA builtin keys range. The range is specified by the limits defined through MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN and MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MAX

#### Enumerator

|                                                   |  |
|---------------------------------------------------|--|
| <code>TFM_BUILTIN_KEY_ID_MIN</code>               |  |
| <code>TFM_BUILTIN_KEY_ID_HUK</code>               |  |
| <code>TFM_BUILTIN_KEY_ID_IAK</code>               |  |
| <code>TFM_BUILTIN_KEY_ID_PLAT_SPECIFIC_MIN</code> |  |
| <code>TFM_BUILTIN_KEY_ID_MAX</code>               |  |
| <code>TFM_BUILTIN_KEY_ID_MIN</code>               |  |
| <code>TFM_BUILTIN_KEY_ID_HUK</code>               |  |
| <code>TFM_BUILTIN_KEY_ID_IAK</code>               |  |
| <code>TFM_BUILTIN_KEY_ID_MAX</code>               |  |





The persistent key identifiers for TF-M builtin keys.

The value of TFM\_BUILTIN\_KEY\_ID\_MIN (and therefore of the whole range) is completely arbitrary except for being inside the PSA builtin keys range.

Definition at line 29 of file tfm\_builtin\_key\_ids.h.

## 8.4.2 Enumeration Type Documentation

### 8.4.2.1 tfm\_builtin\_key\_id\_t

```
enum tfm_builtin_key_id_t
```

Enumerator

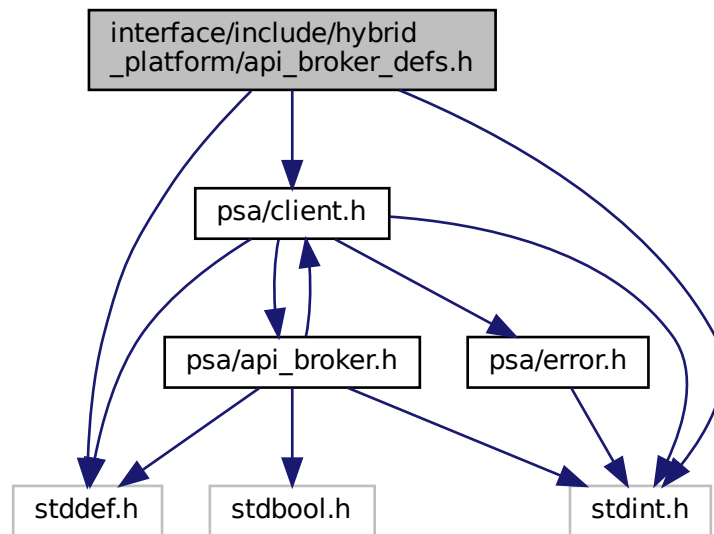
|                                      |  |
|--------------------------------------|--|
| TFM_BUILTIN_KEY_ID_MIN               |  |
| TFM_BUILTIN_KEY_ID_HUK               |  |
| TFM_BUILTIN_KEY_ID_IAK               |  |
| TFM_BUILTIN_KEY_ID_PLAT_SPECIFIC_MIN |  |
| TFM_BUILTIN_KEY_ID_MAX               |  |
| TFM_BUILTIN_KEY_ID_MIN               |  |
| TFM_BUILTIN_KEY_ID_HUK               |  |
| TFM_BUILTIN_KEY_ID_IAK               |  |
| TFM_BUILTIN_KEY_ID_MAX               |  |

Definition at line 32 of file tfm\_builtin\_key\_ids.h.

## 8.5 interface/include/hybrid\_platform/api\_broker\_defs.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "psa/client.h"
```

Include dependency graph for `apiBroker_defs.h`:



## Functions

- `uint32_t psa_framework_version_tz (void)`
- `uint32_t psa_version_tz (uint32_t sid)`
- `psa_status_t psa_call_tz (psa_handle_t handle, int32_t type, const psa_invec *in_vec, size_t in_len, psa_outvec *out_vec, size_t out_len)`
- `psa_handle_t psa_connect_tz (uint32_t sid, uint32_t version)`
- `void psa_close_tz (psa_handle_t handle)`
- `uint32_t psa_framework_version_multicore (void)`
- `uint32_t psa_version_multicore (uint32_t sid)`
- `psa_status_t psa_call_multicore (psa_handle_t handle, int32_t type, const psa_invec *in_vec, size_t in_len, psa_outvec *out_vec, size_t out_len)`
- `void psa_close_multicore (psa_handle_t handle)`
- `psa_handle_t psa_connect_multicore (uint32_t sid, uint32_t version)`

## 8.5.1 Function Documentation

### 8.5.1.1 `psa_call_multicore()`

```

psa_status_t psa_call_multicore (
 psa_handle_t handle,
 int32_t type,
 const psa_invec * in_vec,
 size_t in_len,
 psa_outvec * out_vec,
 size_t out_len)

```

#### 8.5.1.2 psa\_call\_tz()

```
psa_status_t psa_call_tz (
 psa_handle_t handle,
 int32_t type,
 const psa_invec * in_vec,
 size_t in_len,
 psa_outvec * out_vec,
 size_t out_len)
```

#### 8.5.1.3 psa\_close\_multicore()

```
void psa_close_multicore (
 psa_handle_t handle)
```

#### 8.5.1.4 psa\_close\_tz()

```
void psa_close_tz (
 psa_handle_t handle)
```

#### 8.5.1.5 psa\_connect\_multicore()

```
psa_handle_t psa_connect_multicore (
 uint32_t sid,
 uint32_t version)
```

#### 8.5.1.6 psa\_connect\_tz()

```
psa_handle_t psa_connect_tz (
 uint32_t sid,
 uint32_t version)
```

#### 8.5.1.7 psa\_framework\_version\_multicore()

```
uint32_t psa_framework_version_multicore (
 void)
```

#### 8.5.1.8 psa\_framework\_version\_tz()

```
uint32_t psa_framework_version_tz (
 void)
```

#### 8.5.1.9 psa\_version\_multicore()

```
uint32_t psa_version_multicore (
 uint32_t sid)
```

#### 8.5.1.10 psa\_version\_tz()

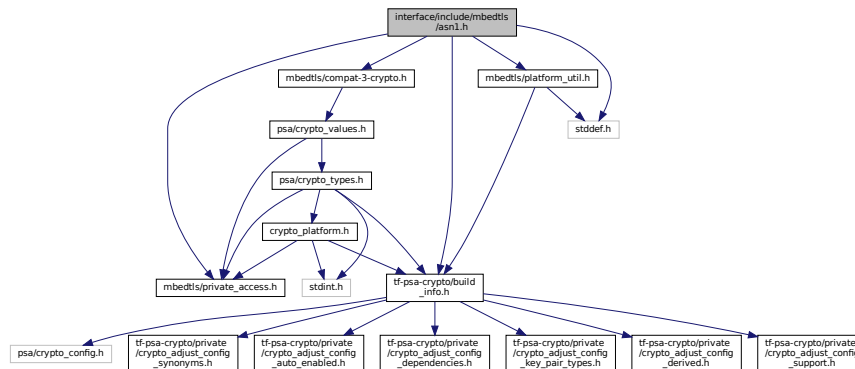
```
uint32_t psa_version_tz (
 uint32_t sid)
```

## 8.6 interface/include/mbedtls/asn1.h File Reference

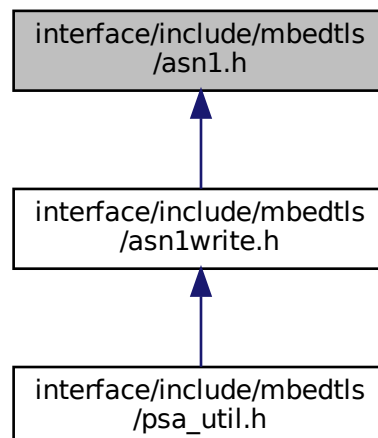
Generic ASN.1 parsing.

```
#include "mbedtls/private_access.h"
#include "tf-psa-crypto/build_info.h"
#include "mbedtls/platform_util.h"
#include "mbedtls/compat-3-crypto.h"
#include <stddef.h>
```

Include dependency graph for `asn1.h`:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [mbedtls\\_asn1\\_buf](#)
- struct [mbedtls\\_asn1\\_bitstring](#)
- struct [mbedtls\\_asn1\\_sequence](#)
- struct [mbedtls\\_asn1\\_named\\_data](#)

## Macros

- #define `MBEDTLS_OID_SIZE(x)` (`sizeof(x) - 1`)
- #define `MBEDTLS_OID_CMP(oid_str, oid_buf)`
- #define `MBEDTLS_OID_CMP_RAW(oid_str, oid_buf, oid_buf_len)`

### ASN1 Error codes

*These error codes are combined with other error codes for higher error granularity. e.g. X.509 and PKCS #7 error codes ASN1 is a standard to specify data structures.*

- #define `MBEDTLS_ERR_ASN1_OUT_OF_DATA` -0x0060
- #define `MBEDTLS_ERR_ASN1_UNEXPECTED_TAG` -0x0062
- #define `MBEDTLS_ERR_ASN1_INVALID_LENGTH` -0x0064
- #define `MBEDTLS_ERR_ASN1_LENGTH_MISMATCH` -0x0066
- #define `MBEDTLS_ERR_ASN1_INVALID_DATA` -0x0068

### DER constants

*These constants comply with the DER encoded ASN.1 type tags. DER encoding uses hexadecimal representation. An example DER sequence is:*

- *0x02 – tag indicating INTEGER*
- *0x01 – length in octets*
- *0x05 – value*
- #define `MBEDTLS_ASN1_BOOLEAN` 0x01
- #define `MBEDTLS_ASN1_INTEGER` 0x02
- #define `MBEDTLS_ASN1_BIT_STRING` 0x03
- #define `MBEDTLS_ASN1_OCTET_STRING` 0x04
- #define `MBEDTLS_ASN1_NULL` 0x05
- #define `MBEDTLS_ASN1_OID` 0x06
- #define `MBEDTLS_ASN1_ENUMERATED` 0x0A
- #define `MBEDTLS_ASN1_UTF8_STRING` 0x0C
- #define `MBEDTLS_ASN1_SEQUENCE` 0x10
- #define `MBEDTLS_ASN1_SET` 0x11
- #define `MBEDTLS_ASN1_PRINTABLE_STRING` 0x13
- #define `MBEDTLS_ASN1_T61_STRING` 0x14
- #define `MBEDTLS_ASN1_IA5_STRING` 0x16
- #define `MBEDTLS_ASN1_UTCTIME` 0x17
- #define `MBEDTLS_ASN1_GENERALIZED_TIME` 0x18
- #define `MBEDTLS_ASN1_UNIVERSAL_STRING` 0x1C
- #define `MBEDTLS_ASN1_BMP_STRING` 0x1E
- #define `MBEDTLS_ASN1_PRIMITIVE` 0x00
- #define `MBEDTLS_ASN1_CONSTRUCTED` 0x20
- #define `MBEDTLS_ASN1_CONTEXT_SPECIFIC` 0x80
- #define `MBEDTLS_ASN1_IS_STRING_TAG(tag)`
- #define `MBEDTLS_ASN1_TAG_CLASS_MASK` 0xC0
- #define `MBEDTLS_ASN1_TAG_PC_MASK` 0x20
- #define `MBEDTLS_ASN1_TAG_VALUE_MASK` 0x1F

## Typedefs

### Functions to parse ASN.1 data structures

- typedef struct `mbedtls_asn1_buf` `mbedtls_asn1_buf`
- typedef struct `mbedtls_asn1_bitstring` `mbedtls_asn1_bitstring`
- typedef struct `mbedtls_asn1_sequence` `mbedtls_asn1_sequence`
- typedef struct `mbedtls_asn1_named_data` `mbedtls_asn1_named_data`

## 8.6.1 Detailed Description

Generic ASN.1 parsing.

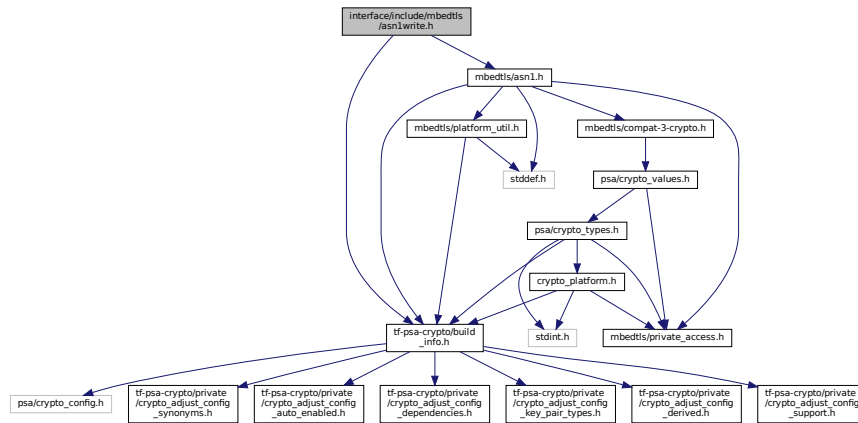
## 8.7 interface/include/mbedtls/asn1write.h File Reference

ASN.1 buffer writing functionality.

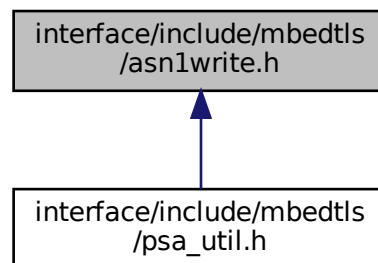
```
#include "tf-psa-crypto/build_info.h"
```

```
#include "mbedtls/asn1.h"
```

Include dependency graph for `asn1write.h`:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define MBEDTLS_ASN1_CHK_ADD(g, f)`
- `#define MBEDTLS_ASN1_CHK_CLEANUP_ADD(g, f)`

### 8.7.1 Detailed Description

ASN.1 buffer writing functionality.

### 8.7.2 Macro Definition Documentation

### 8.7.2.1 MBEDTLS\_ASN1\_CHK\_ADD

```
#define MBEDTLS_ASN1_CHK_ADD(
 g,
 f)
```

**Value:**

```
do
{
 if ((ret = (f)) < 0)
 return ret;
 else
 (g) += ret;
} while (0)
```

Definition at line 17 of file asn1write.h.

### 8.7.2.2 MBEDTLS\_ASN1\_CHK\_CLEANUP\_ADD

```
#define MBEDTLS_ASN1_CHK_CLEANUP_ADD(
 g,
 f)
```

**Value:**

```
do
{
 if ((ret = (f)) < 0)
 goto cleanup;
 else
 (g) += ret;
} while (0)
```

Definition at line 26 of file asn1write.h.

## 8.8 interface/include/mbedtls/base64.h File Reference

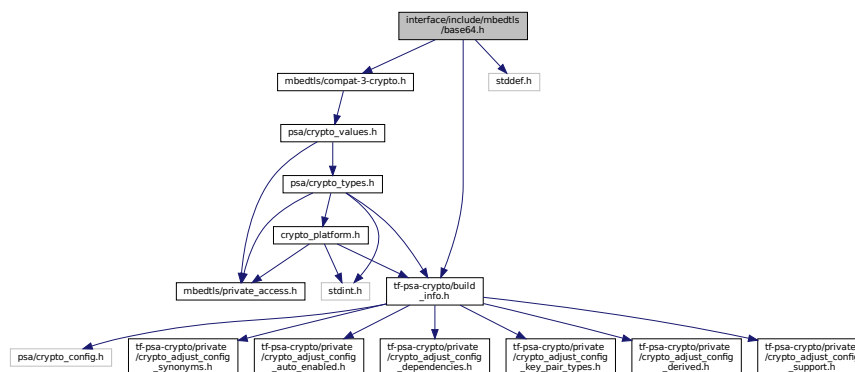
RFC 1521 base64 encoding/decoding.

```
#include "tf-psa-crypto/build_info.h"
```

```
#include "mbedtls/compat-3-crypto.h"
```

```
#include <stddef.h>
```

Include dependency graph for base64.h:



## Macros

- #define [MBEDTLS\\_ERR\\_BASE64\\_INVALID\\_CHARACTER](#) -0x002C

## Functions

- int [mbedtls\\_base64\\_encode](#) (unsigned char \*dst, size\_t dlen, size\_t \*olen, const unsigned char \*src, size\_t slen)

*Encode a buffer into base64 format.*

- int `MBEDTLS_base64_decode` (unsigned char \*dst, size\_t dlen, size\_t \*olen, const unsigned char \*src, size\_t slen)

*Decode a base64-formatted buffer.*

## 8.8.1 Detailed Description

RFC 1521 base64 encoding/decoding.

## 8.8.2 Macro Definition Documentation

### 8.8.2.1 MBEDTLS\_ERR\_BASE64\_INVALID\_CHARACTER

```
#define MBEDTLS_ERR_BASE64_INVALID_CHARACTER -0x002C
```

Invalid character in input.

Definition at line 19 of file base64.h.

## 8.8.3 Function Documentation

### 8.8.3.1 mbedtls\_base64\_decode()

```
int mbedtls_base64_decode (
 unsigned char * dst,
 size_t dlen,
 size_t * olen,
 const unsigned char * src,
 size_t slen)
```

Decode a base64-formatted buffer.

#### Parameters

|             |                                                    |
|-------------|----------------------------------------------------|
| <i>dst</i>  | destination buffer (can be NULL for checking size) |
| <i>dlen</i> | size of the destination buffer                     |
| <i>olen</i> | number of bytes written                            |
| <i>src</i>  | source buffer                                      |
| <i>slen</i> | amount of data to be decoded                       |

#### Returns

0 if successful, [PSA\\_ERROR\\_BUFFER\\_TOO\\_SMALL](#), or `MBEDTLS_ERR_BASE64_INVALID_CHARACTER` if the input data is not correct. \*olen is always updated to reflect the amount of data that has (or would have) been written.

#### Note

Call this function with \*dst = NULL or dlen = 0 to obtain the required buffer size in \*olen

### 8.8.3.2 mbedtls\_base64\_encode()

```
int mbedtls_base64_encode (
 unsigned char * dst,
 size_t dlen,
```



```

size_t * olen,
const unsigned char * src,
size_t slen)

```

Encode a buffer into base64 format.

#### Parameters

|             |                                |
|-------------|--------------------------------|
| <i>dst</i>  | destination buffer             |
| <i>dlen</i> | size of the destination buffer |
| <i>olen</i> | number of bytes written        |
| <i>src</i>  | source buffer                  |
| <i>slen</i> | amount of data to be encoded   |

#### Returns

0 if successful, or [PSA\\_ERROR\\_BUFFER\\_TOO\\_SMALL](#). \*olen is always updated to reflect the amount of data that has (or would have) been written. If that length cannot be represented, then no data is written to the buffer and \*olen is set to the maximum length representable as a size\_t.

#### Note

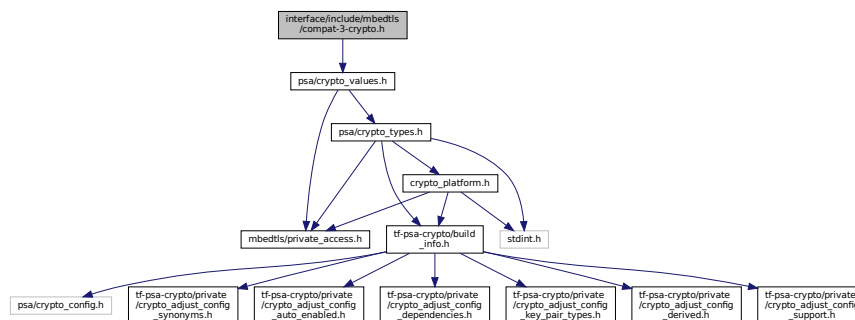
Call this function with dlen = 0 to obtain the required buffer size in \*olen

## 8.9 interface/include/mbedtls/compat-3-crypto.h File Reference

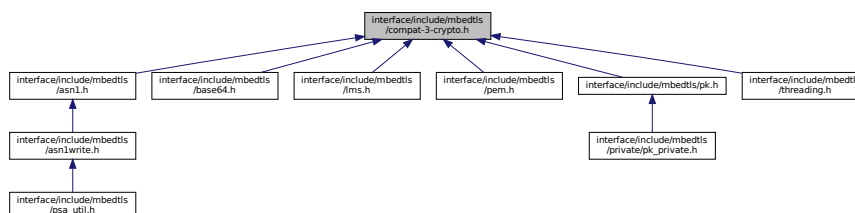
Compatibility definitions for MbedTLS 3.x code built with MbedTLS 4.x or TF-PSA-Crypto 1.x.

```
#include "psa/crypto_values.h"
```

Include dependency graph for compat-3-crypto.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define MBEDTLS_ERR_BASE64_BUFFER_TOO_SMALL PSA_ERROR_BUFFER_TOO_SMALL`
- `#define MBEDTLS_ERR_ASN1_BUF_TOO_SMALL PSA_ERROR_BUFFER_TOO_SMALL`
- `#define MBEDTLS_ERR_LMS_BUFFER_TOO_SMALL PSA_ERROR_BUFFER_TOO_SMALL`
- `#define MBEDTLS_ERR_PK_BUFFER_TOO_SMALL PSA_ERROR_BUFFER_TOO_SMALL`
- `#define MBEDTLS_ERR_PK_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY`
- `#define MBEDTLS_ERR_PEM_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY`
- `#define MBEDTLS_ERR_ASN1_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY`
- `#define MBEDTLS_ERR_LMS_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY`
- `#define MBEDTLS_ERR_PK_BAD_INPUT_DATA PSA_ERROR_INVALID_ARGUMENT`
- `#define MBEDTLS_ERR_PEM_BAD_INPUT_DATA PSA_ERROR_INVALID_ARGUMENT`
- `#define MBEDTLS_ERR_LMS_BAD_INPUT_DATA PSA_ERROR_INVALID_ARGUMENT`

### 8.9.1 Detailed Description

Compatibility definitions for MbedTLS 3.x code built with MbedTLS 4.x or TF-PSA-Crypto 1.x.

### 8.9.2 Macro Definition Documentation

#### 8.9.2.1 MBEDTLS\_ERR\_ASN1\_ALLOC\_FAILED

```
#define MBEDTLS_ERR_ASN1_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY
```

Memory allocation failed

Definition at line 32 of file compat-3-crypto.h.

#### 8.9.2.2 MBEDTLS\_ERR\_ASN1\_BUF\_TOO\_SMALL

```
#define MBEDTLS_ERR_ASN1_BUF_TOO_SMALL PSA_ERROR_BUFFER_TOO_SMALL
```

Buffer too small when writing ASN.1 data structure.

Definition at line 21 of file compat-3-crypto.h.

#### 8.9.2.3 MBEDTLS\_ERR\_BASE64\_BUFFER\_TOO\_SMALL

```
#define MBEDTLS_ERR_BASE64_BUFFER_TOO_SMALL PSA_ERROR_BUFFER_TOO_SMALL
```

Output buffer too small.

Definition at line 19 of file compat-3-crypto.h.

#### 8.9.2.4 MBEDTLS\_ERR\_LMS\_ALLOC\_FAILED

```
#define MBEDTLS_ERR_LMS_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY
```

LMS failed to allocate space for a private key

Definition at line 34 of file compat-3-crypto.h.

#### 8.9.2.5 MBEDTLS\_ERR\_LMS\_BAD\_INPUT\_DATA

```
#define MBEDTLS_ERR_LMS_BAD_INPUT_DATA PSA_ERROR_INVALID_ARGUMENT
```

Bad data has been input to an LMS function

Definition at line 41 of file compat-3-crypto.h.

#### 8.9.2.6 MBEDTLS\_ERR\_LMS\_BUFFER\_TOO\_SMALL

`#define MBEDTLS_ERR_LMS_BUFFER_TOO_SMALL PSA_ERROR_BUFFER_TOO_SMALL`  
Input/output buffer is too small to contain requited data  
Definition at line 23 of file compat-3-crypto.h.

#### 8.9.2.7 MBEDTLS\_ERR\_PEM\_ALLOC\_FAILED

`#define MBEDTLS_ERR_PEM_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY`  
Failed to allocate memory.  
Definition at line 30 of file compat-3-crypto.h.

#### 8.9.2.8 MBEDTLS\_ERR\_PEM\_BAD\_INPUT\_DATA

`#define MBEDTLS_ERR_PEM_BAD_INPUT_DATA PSA_ERROR_INVALID_ARGUMENT`  
Bad input parameters to function.  
Definition at line 39 of file compat-3-crypto.h.

#### 8.9.2.9 MBEDTLS\_ERR\_PK\_ALLOC\_FAILED

`#define MBEDTLS_ERR_PK_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY`  
Memory allocation failed.  
Definition at line 28 of file compat-3-crypto.h.

#### 8.9.2.10 MBEDTLS\_ERR\_PK\_BAD\_INPUT\_DATA

`#define MBEDTLS_ERR_PK_BAD_INPUT_DATA PSA_ERROR_INVALID_ARGUMENT`  
Bad input parameters to function.  
Definition at line 37 of file compat-3-crypto.h.

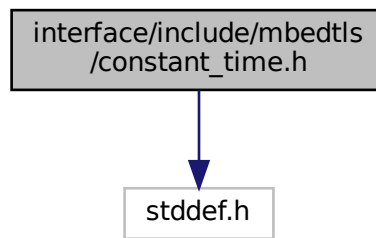
#### 8.9.2.11 MBEDTLS\_ERR\_PK\_BUFFER\_TOO\_SMALL

`#define MBEDTLS_ERR_PK_BUFFER_TOO_SMALL PSA_ERROR_BUFFER_TOO_SMALL`  
The output buffer is too small.  
Definition at line 25 of file compat-3-crypto.h.

## 8.10 interface/include/mbedtls/constant\_time.h File Reference

`#include <stddef.h>`

Include dependency graph for `constant_time.h`:



## Functions

- `int mbedtls_ct_memcmp` (`const void *a`, `const void *b`, `size_t n`)

### 8.10.1 Function Documentation

#### 8.10.1.1 `mbedtls_ct_memcmp()`

```
int mbedtls_ct_memcmp (
 const void * a,
 const void * b,
 size_t n)
```

Constant-time functions Constant-time buffer comparison without branches.

This is equivalent to the standard `memcmp` function, but is likely to be compiled to code using bitwise operations rather than a branch, such that the time taken is constant w.r.t. the data pointed to by `a` and `b`, and w.r.t. whether `a` and `b` are equal or not. It is not constant-time w.r.t. `n`.

This function can be used to write constant-time code by replacing branches with bit operations using masks.

#### Parameters

|          |                                                                                    |
|----------|------------------------------------------------------------------------------------|
| <i>a</i> | Pointer to the first buffer, containing at least <i>n</i> bytes. May not be NULL.  |
| <i>b</i> | Pointer to the second buffer, containing at least <i>n</i> bytes. May not be NULL. |
| <i>n</i> | The number of bytes to compare.                                                    |

#### Returns

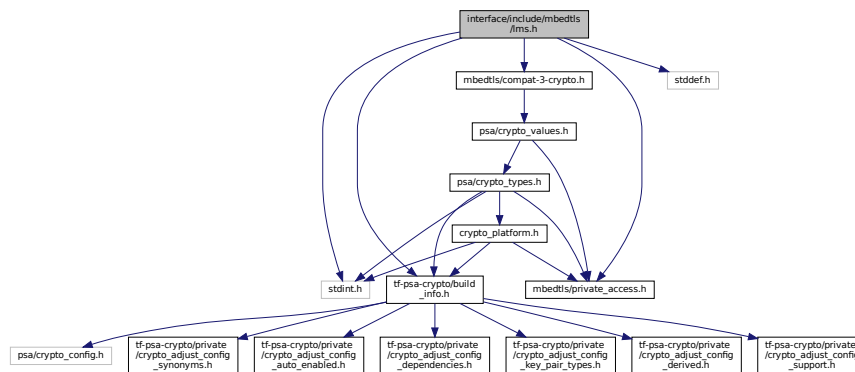
Zero if the contents of the two buffers are the same, otherwise non-zero.

## 8.11 `interface/include/mbedtls/lms.h` File Reference

This file provides an API for the LMS post-quantum-safe stateful-hash public-key signature scheme as defined in RFC8554 and NIST.SP.200-208. This implementation currently only supports a single parameter set `MBEDTLS_LMS_SHA256_M32_H10` in order to reduce complexity. This is one of the signature schemes recommended by the IETF draft SUIF standard for IOT firmware upgrades (RFC9019).

```
#include <stdint.h>
#include <stddef.h>
#include "mbedtls/private_access.h"
```

```
#include "tf-psa-crypto/build_info.h"
#include "mbedtls/compat-3-crypto.h"
Include dependency graph for lms.h:
```



## Data Structures

- struct [mbedtls\\_lmots\\_parameters\\_t](#)
- struct [mbedtls\\_lmots\\_public\\_t](#)
- struct [mbedtls\\_lms\\_parameters\\_t](#)
- struct [mbedtls\\_lms\\_public\\_t](#)

## Macros

- `#define MBEDTLS_ERR_LMS_OUT_OF_PRIVATE_KEYS -0x0013`
- `#define MBEDTLS_ERR_LMS_VERIFY_FAILED -0x0015`
- `#define MBEDTLS_LMOTS_N_HASH_LEN_MAX (32u)`
- `#define MBEDTLS_LMOTS_P_SIG_DIGIT_COUNT_MAX (34u)`
- `#define MBEDTLS_LMOTS_N_HASH_LEN(type) ((type) == MBEDTLS_LMOTS_SHA256_N32_W8 ? 32u : 0)`
- `#define MBEDTLS_LMOTS_I_KEY_ID_LEN (16u)`
- `#define MBEDTLS_LMOTS_Q_LEAF_ID_LEN (4u)`
- `#define MBEDTLS_LMOTS_TYPE_LEN (4u)`
- `#define MBEDTLS_LMOTS_P_SIG_DIGIT_COUNT(type) ((type) == MBEDTLS_LMOTS_SHA256_N32_W8 ? 34u : 0)`
- `#define MBEDTLS_LMOTS_C_RANDOM_VALUE_LEN(type) (MBEDTLS_LMOTS_N_HASH_LEN(type))`
- `#define MBEDTLS_LMOTS_SIG_LEN(type)`
- `#define MBEDTLS_LMS_TYPE_LEN (4)`
- `#define MBEDTLS_LMS_H_TREE_HEIGHT(type) ((type) == MBEDTLS_LMS_SHA256_M32_H10 ? 10u : 0)`
- `#define MBEDTLS_LMS_M_NODE_BYTES(type) ((type) == MBEDTLS_LMS_SHA256_M32_H10 ? 32 : 0)`
- `#define MBEDTLS_LMS_M_NODE_BYTES_MAX 32`
- `#define MBEDTLS_LMS_SIG_LEN(type, otstype)`
- `#define MBEDTLS_LMS_PUBLIC_KEY_LEN(type)`

## Enumerations

- enum [mbedtls\\_lms\\_algorithm\\_type\\_t](#) { `MBEDTLS_LMS_SHA256_M32_H10` = 0x6 }
- enum [mbedtls\\_lmots\\_algorithm\\_type\\_t](#) { `MBEDTLS_LMOTS_SHA256_N32_W8` = 4 }

## Functions

- `void mbedtls_lms_public_init (mbedtls_lms_public_t *ctx)`  
*This function initializes an LMS public context.*
- `void mbedtls_lms_public_free (mbedtls_lms_public_t *ctx)`  
*This function uninitializes an LMS public context.*
- `int mbedtls_lms_import_public_key (mbedtls_lms_public_t *ctx, const unsigned char *key, size_t key_size)`  
*This function imports an LMS public key into a public LMS context.*
- `int mbedtls_lms_export_public_key (const mbedtls_lms_public_t *ctx, unsigned char *key, size_t key_size, size_t *key_len)`  
*This function exports an LMS public key from a LMS public context that already contains a public key.*
- `int mbedtls_lms_verify (const mbedtls_lms_public_t *ctx, const unsigned char *msg, size_t msg_size, const unsigned char *sig, size_t sig_size)`  
*This function verifies a LMS signature, using a LMS context that contains a public key.*

### 8.11.1 Detailed Description

This file provides an API for the LMS post-quantum-safe stateful-hash public-key signature scheme as defined in RFC8554 and NIST.SP.200-208. This implementation currently only supports a single parameter set MBEDTLS\_LMS\_SHA256\_M32\_H10 in order to reduce complexity. This is one of the signature schemes recommended by the IETF draft SUIT standard for IOT firmware upgrades (RFC9019).

### 8.11.2 Macro Definition Documentation

#### 8.11.2.1 MBEDTLS\_ERR\_LMS\_OUT\_OF\_PRIVATE\_KEYS

```
#define MBEDTLS_ERR_LMS_OUT_OF_PRIVATE_KEYS -0x0013
```

Specified LMS key has utilised all of its private keys

Definition at line 25 of file lms.h.

#### 8.11.2.2 MBEDTLS\_ERR\_LMS\_VERIFY\_FAILED

```
#define MBEDTLS_ERR_LMS_VERIFY_FAILED -0x0015
```

LMS signature verification failed

Definition at line 26 of file lms.h.

#### 8.11.2.3 MBEDTLS\_LMOTS\_C\_RANDOM\_VALUE\_LEN

```
#define MBEDTLS_LMOTS_C_RANDOM_VALUE_LEN(
 type) (MBEDTLS_LMOTS_N_HASH_LEN(type))
```

Definition at line 36 of file lms.h.

#### 8.11.2.4 MBEDTLS\_LMOTS\_I\_KEY\_ID\_LEN

```
#define MBEDTLS_LMOTS_I_KEY_ID_LEN (16u)
```

Definition at line 32 of file lms.h.

#### 8.11.2.5 MBEDTLS\_LMOTS\_N\_HASH\_LEN

```
#define MBEDTLS_LMOTS_N_HASH_LEN(
 type) ((type) == MBEDTLS_LMOTS_SHA256_N32_W8 ? 32u : 0)
```

Definition at line 31 of file lms.h.

**8.11.2.6 MBEDTLS\_LMOTS\_N\_HASH\_LEN\_MAX**

```
#define MBEDTLS_LMOTS_N_HASH_LEN_MAX (32u)
```

Definition at line 29 of file lms.h.

**8.11.2.7 MBEDTLS\_LMOTS\_P\_SIG\_DIGIT\_COUNT**

```
#define MBEDTLS_LMOTS_P_SIG_DIGIT_COUNT(
 type) ((type) == MBEDTLS_LMOTS_SHA256_N32_W8 ? 34u : 0)
```

Definition at line 35 of file lms.h.

**8.11.2.8 MBEDTLS\_LMOTS\_P\_SIG\_DIGIT\_COUNT\_MAX**

```
#define MBEDTLS_LMOTS_P_SIG_DIGIT_COUNT_MAX (34u)
```

Definition at line 30 of file lms.h.

**8.11.2.9 MBEDTLS\_LMOTS\_Q\_LEAF\_ID\_LEN**

```
#define MBEDTLS_LMOTS_Q_LEAF_ID_LEN (4u)
```

Definition at line 33 of file lms.h.

**8.11.2.10 MBEDTLS\_LMOTS\_SIG\_LEN**

```
#define MBEDTLS_LMOTS_SIG_LEN(
 type)
```

**Value:**

```
(MBEDTLS_LMOTS_TYPE_LEN + \
MBEDTLS_LMOTS_C_RANDOM_VALUE_LEN(type) + \
(MBEDTLS_LMOTS_P_SIG_DIGIT_COUNT(type) * \
MBEDTLS_LMOTS_N_HASH_LEN(type))
```

Definition at line 38 of file lms.h.

**8.11.2.11 MBEDTLS\_LMOTS\_TYPE\_LEN**

```
#define MBEDTLS_LMOTS_TYPE_LEN (4u)
```

Definition at line 34 of file lms.h.

**8.11.2.12 MBEDTLS\_LMS\_H\_TREE\_HEIGHT**

```
#define MBEDTLS_LMS_H_TREE_HEIGHT(
 type) ((type) == MBEDTLS_LMS_SHA256_M32_H10 ? 10u : 0)
```

Definition at line 45 of file lms.h.

**8.11.2.13 MBEDTLS\_LMS\_M\_NODE\_BYTES**

```
#define MBEDTLS_LMS_M_NODE_BYTES(
 type) ((type) == MBEDTLS_LMS_SHA256_M32_H10 ? 32 : 0)
```

Definition at line 50 of file lms.h.

**8.11.2.14 MBEDTLS\_LMS\_M\_NODE\_BYTES\_MAX**

```
#define MBEDTLS_LMS_M_NODE_BYTES_MAX 32
```

Definition at line 51 of file lms.h.

### 8.11.2.15 MBEDTLS\_LMS\_PUBLIC\_KEY\_LEN

```
#define MBEDTLS_LMS_PUBLIC_KEY_LEN(
 type)
```

**Value:**

```
(MBEDTLS_LMS_TYPE_LEN + \
MBEDTLS_LMOTS_TYPE_LEN + \
MBEDTLS_LMOTS_I_KEY_ID_LEN + \
MBEDTLS_LMS_M_NODE_BYTES(type))
```

Definition at line 59 of file lms.h.

### 8.11.2.16 MBEDTLS\_LMS\_SIG\_LEN

```
#define MBEDTLS_LMS_SIG_LEN(
 type,
 otstype)
```

**Value:**

```
(MBEDTLS_LMOTS_Q_LEAF_ID_LEN + \
MBEDTLS_LMOTS_SIG_LEN(otstype) + \
MBEDTLS_LMS_TYPE_LEN + \
(MBEDTLS_LMS_H_TREE_HEIGHT(type) * \
MBEDTLS_LMS_M_NODE_BYTES(type)))
```

Definition at line 53 of file lms.h.

### 8.11.2.17 MBEDTLS\_LMS\_TYPE\_LEN

```
#define MBEDTLS_LMS_TYPE_LEN (4)
```

Definition at line 44 of file lms.h.

## 8.11.3 Enumeration Type Documentation

### 8.11.3.1 mbedtls\_lmots\_algorithm\_type\_t

```
enum mbedtls_lmots_algorithm_type_t
```

The Identifier of the LMOTS parameter set, as per <https://www.iana.org/assignments/leighton-micali-signature-parameters-for-the-lmots-digital-signature-algorithm>. We are only implementing a subset of the types, particularly N32\_W8, for the sake of simplicity.

**Enumerator**

|                             |  |
|-----------------------------|--|
| MBEDTLS_LMOTS_SHA256_N32_W8 |  |
|-----------------------------|--|

Definition at line 81 of file lms.h.

### 8.11.3.2 mbedtls\_lms\_algorithm\_type\_t

```
enum mbedtls_lms_algorithm_type_t
```

The Identifier of the LMS parameter set, as per <https://www.iana.org/assignments/leighton-micali-signature-parameters-for-the-lms-digital-signature-algorithm>. We are only implementing a subset of the types, particularly H10, for the sake of simplicity.

**Enumerator**

|                            |  |
|----------------------------|--|
| MBEDTLS_LMS_SHA256_M32_H10 |  |
|----------------------------|--|

Definition at line 73 of file lms.h.



## 8.11.4 Function Documentation

### 8.11.4.1 mbedtls\_lms\_export\_public\_key()

```
int mbedtls_lms_export_public_key (
 const mbedtls_lms_public_t * ctx,
 unsigned char * key,
 size_t key_size,
 size_t * key_len)
```

This function exports an LMS public key from a LMS public context that already contains a public key.

#### Note

Before this function is called, the context must have been initialized and the context must contain a public key.  
See IETF RFC8554 for details of the encoding of this public key.

#### Parameters

|                 |                                                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| <i>ctx</i>      | The initialized LMS public context that contains the public key.                                                   |
| <i>key</i>      | The buffer into which the key will be output. Must be at least <a href="#">MBEDTLS_LMS_PUBLIC_KEY_LEN</a> in size. |
| <i>key_size</i> | The size of the key buffer.                                                                                        |
| <i>key_len</i>  | If not NULL, will be written with the size of the key.                                                             |

#### Returns

0 on success.  
A non-zero error code on failure.

### 8.11.4.2 mbedtls\_lms\_import\_public\_key()

```
int mbedtls_lms_import_public_key (
 mbedtls_lms_public_t * ctx,
 const unsigned char * key,
 size_t key_size)
```

This function imports an LMS public key into a public LMS context.

#### Note

Before this function is called, the context must have been initialized.  
See IETF RFC8554 for details of the encoding of this public key.

#### Parameters

|                 |                                                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------------------------------|
| <i>ctx</i>      | The initialized LMS context store the key in.                                                                        |
| <i>key</i>      | The buffer from which the key will be read. <a href="#">MBEDTLS_LMS_PUBLIC_KEY_LEN</a> bytes will be read from this. |
| <i>key_size</i> | The size of the key being imported.                                                                                  |

#### Returns

0 on success.  
A non-zero error code on failure.

#### 8.11.4.3 mbedtls\_lms\_public\_free()

```
void mbedtls_lms_public_free (
 mbedtls_lms_public_t * ctx)
```

This function uninitializes an LMS public context.

##### Parameters

|            |                                                              |
|------------|--------------------------------------------------------------|
| <i>ctx</i> | The initialized LMS context that will then be uninitialized. |
|------------|--------------------------------------------------------------|

#### 8.11.4.4 mbedtls\_lms\_public\_init()

```
void mbedtls_lms_public_init (
 mbedtls_lms_public_t * ctx)
```

This function initializes an LMS public context.

##### Parameters

|            |                                                              |
|------------|--------------------------------------------------------------|
| <i>ctx</i> | The uninitialized LMS context that will then be initialized. |
|------------|--------------------------------------------------------------|

#### 8.11.4.5 mbedtls\_lms\_verify()

```
int mbedtls_lms_verify (
 const mbedtls_lms_public_t * ctx,
 const unsigned char * msg,
 size_t msg_size,
 const unsigned char * sig,
 size_t sig_size)
```

This function verifies a LMS signature, using a LMS context that contains a public key.

##### Note

Before this function is called, the context must have been initialized and must contain a public key (either by import or generation).

##### Parameters

|                 |                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| <i>ctx</i>      | The initialized LMS public context from which the public key will be read.                                       |
| <i>msg</i>      | The buffer from which the message will be read.                                                                  |
| <i>msg_size</i> | The size of the message that will be read.                                                                       |
| <i>sig</i>      | The buf from which the signature will be read. <a href="#">MBEDTLS_LMS_SIG_LEN</a> bytes will be read from this. |
| <i>sig_size</i> | The size of the signature to be verified.                                                                        |

##### Returns

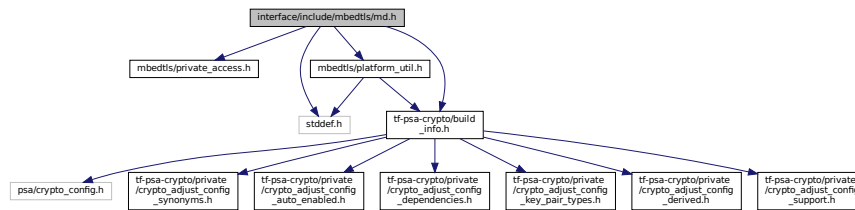
- 0 on successful verification.
- A non-zero error code on failure.

## 8.12 interface/include/mbedtls/md.h File Reference

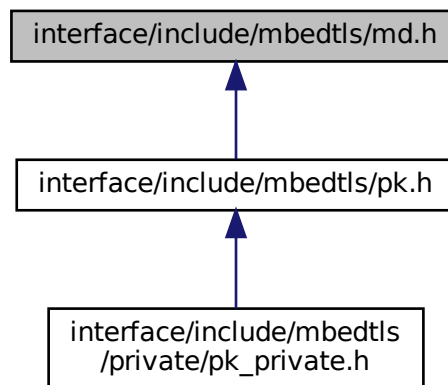
This file contains the generic functions for message-digest (hashing) and HMAC.

```
#include "mbedtls/private_access.h"
#include <stddef.h>
#include "tf-psa-crypto/build_info.h"
#include "mbedtls/platform_util.h"
```

Include dependency graph for md.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [mbedtls\\_md\\_context\\_t](#)

## Macros

- #define [MBEDTLS\\_ERR\\_MD\\_FEATURE\\_UNAVAILABLE](#) -0x5080
- #define [MBEDTLS\\_ERR\\_MD\\_BAD\\_INPUT\\_DATA](#) PSA\_ERROR\_INVALID\_ARGUMENT
- #define [MBEDTLS\\_ERR\\_MD\\_ALLOC\\_FAILED](#) PSA\_ERROR\_INSUFFICIENT\_MEMORY
- #define [MBEDTLS\\_MD\\_MAX\\_SIZE](#)

## Typedefs

- typedef struct [mbedtls\\_md\\_info\\_t](#) [mbedtls\\_md\\_info\\_t](#)
- typedef struct [mbedtls\\_md\\_context\\_t](#) [mbedtls\\_md\\_context\\_t](#)

## Enumerations

- enum `MBEDTLS_MD_TYPE` {  
`MBEDTLS_MD_NONE` = 0, `MBEDTLS_MD_MD5` = 0x03, `MBEDTLS_MD_RIPEMD160` = 0x04, `MBEDTLS_MD_SHA1` = 0x05,  
`MBEDTLS_MD_SHA224` = 0x08, `MBEDTLS_MD_SHA256` = 0x09, `MBEDTLS_MD_SHA384` = 0x0a,  
`MBEDTLS_MD_SHA512` = 0x0b,  
`MBEDTLS_MD_SHA3_224` = 0x10, `MBEDTLS_MD_SHA3_256` = 0x11, `MBEDTLS_MD_SHA3_384` = 0x12,  
`MBEDTLS_MD_SHA3_512` = 0x13 }  
*Supported message digests.*
- enum `MBEDTLS_MD_ENGINE` { `MBEDTLS_MD_ENGINE_LEGACY` = 0, `MBEDTLS_MD_ENGINE_PSA` }

## Functions

- const `MBEDTLS_MD_INFO` \* `MBEDTLS_MD_INFO_FROM_TYPE` (`MBEDTLS_MD_TYPE` md\_type)  
*This function returns the message-digest information associated with the given digest type.*
- void `MBEDTLS_MD_INIT` (`MBEDTLS_MD_CONTEXT` \*ctx)  
*This function initializes a message-digest context without binding it to a particular message-digest algorithm.*
- void `MBEDTLS_MD_FREE` (`MBEDTLS_MD_CONTEXT` \*ctx)  
*This function clears the internal structure of ctx and frees any embedded internal structure, but does not free ctx itself.*
- `MBEDTLS_CHECK_RETURN_TYPICAL` int `MBEDTLS_MD_SETUP` (`MBEDTLS_MD_CONTEXT` \*ctx, const `MBEDTLS_MD_INFO` \*md\_info, int hmac)  
*This function selects the message digest algorithm to use, and allocates internal structures.*
- `MBEDTLS_CHECK_RETURN_TYPICAL` int `MBEDTLS_MD_CLONE` (`MBEDTLS_MD_CONTEXT` \*dst, const `MBEDTLS_MD_CONTEXT` \*src)  
*This function clones the state of a message-digest context.*
- unsigned char `MBEDTLS_MD_GET_SIZE` (const `MBEDTLS_MD_INFO` \*md\_info)  
*This function extracts the message-digest size from the message-digest information structure.*
- `MBEDTLS_MD_TYPE` `MBEDTLS_MD_GET_TYPE` (const `MBEDTLS_MD_INFO` \*md\_info)  
*This function extracts the message-digest type from the message-digest information structure.*
- `MBEDTLS_CHECK_RETURN_TYPICAL` int `MBEDTLS_MD_STARTS` (`MBEDTLS_MD_CONTEXT` \*ctx)  
*This function starts a message-digest computation.*
- `MBEDTLS_CHECK_RETURN_TYPICAL` int `MBEDTLS_MD_UPDATE` (`MBEDTLS_MD_CONTEXT` \*ctx, const unsigned char \*input, size\_t ilen)  
*This function feeds an input buffer into an ongoing message-digest computation.*
- `MBEDTLS_CHECK_RETURN_TYPICAL` int `MBEDTLS_MD_FINISH` (`MBEDTLS_MD_CONTEXT` \*ctx, unsigned char \*output)  
*This function finishes the digest operation, and writes the result to the output buffer.*
- `MBEDTLS_CHECK_RETURN_TYPICAL` int `MBEDTLS_MD` (const `MBEDTLS_MD_INFO` \*md\_info, const unsigned char \*input, size\_t ilen, unsigned char \*output)  
*This function calculates the message-digest of a buffer, with respect to a configurable message-digest algorithm in a single call.*

### 8.12.1 Detailed Description

This file contains the generic functions for message-digest (hashing) and HMAC.

Author

Adriaan de Jong [dejong@fox-it.com](mailto:dejong@fox-it.com)

### 8.12.2 Macro Definition Documentation

### 8.12.2.1 MBEDTLS\_ERR\_MD\_ALLOC\_FAILED

```
#define MBEDTLS_ERR_MD_ALLOC_FAILED PSA_ERROR_INSUFFICIENT_MEMORY
```

Failed to allocate memory.

Definition at line 28 of file md.h.

### 8.12.2.2 MBEDTLS\_ERR\_MD\_BAD\_INPUT\_DATA

```
#define MBEDTLS_ERR_MD_BAD_INPUT_DATA PSA_ERROR_INVALID_ARGUMENT
```

Bad input parameters to function.

Definition at line 26 of file md.h.

### 8.12.2.3 MBEDTLS\_ERR\_MD\_FEATURE\_UNAVAILABLE

```
#define MBEDTLS_ERR_MD_FEATURE_UNAVAILABLE -0x5080
```

The selected feature is not available.

Definition at line 24 of file md.h.

### 8.12.2.4 MBEDTLS\_MD\_MAX\_SIZE

```
#define MBEDTLS_MD_MAX_SIZE
```

**Value:**

```
20 /* longest known is SHA1 or RIPE MD-160
or smaller (MD5 and earlier) */
```

Definition at line 82 of file md.h.

## 8.12.3 Typedef Documentation

### 8.12.3.1 mbedtls\_md\_context\_t

```
typedef struct mbedtls_md_context_t mbedtls_md_context_t
```

The generic message-digest context.

### 8.12.3.2 mbedtls\_md\_info\_t

```
typedef struct mbedtls_md_info_t mbedtls_md_info_t
```

Opaque struct.

Constructed using [mbedtls\\_md\\_info\\_from\\_type](#).

Fields can be accessed with [mbedtls\\_md\\_get\\_size](#) and [mbedtls\\_md\\_get\\_type](#).

Definition at line 94 of file md.h.

## 8.12.4 Enumeration Type Documentation

### 8.12.4.1 mbedtls\_md\_engine\_t

```
enum mbedtls_md_engine_t
```

**Enumerator**

|                          |  |
|--------------------------|--|
| MBEDTLS_MD_ENGINE_LEGACY |  |
| MBEDTLS_MD_ENGINE_PSA    |  |

Definition at line 101 of file md.h.

#### 8.12.4.2 mbedtls\_md\_type\_t

enum `mbedtls_md_type_t`

Supported message digests.

##### Warning

MD5 and SHA-1 are considered weak message digests and their use constitutes a security risk. We recommend considering stronger message digests instead.

##### Enumerator

|                                   |                                |
|-----------------------------------|--------------------------------|
| <code>MBEDTLS_MD_NONE</code>      | None.                          |
| <code>MBEDTLS_MD_MD5</code>       | The MD5 message digest.        |
| <code>MBEDTLS_MD_RIPEMD160</code> | The RIPEMD-160 message digest. |
| <code>MBEDTLS_MD_SHA1</code>      | The SHA-1 message digest.      |
| <code>MBEDTLS_MD_SHA224</code>    | The SHA-224 message digest.    |
| <code>MBEDTLS_MD_SHA256</code>    | The SHA-256 message digest.    |
| <code>MBEDTLS_MD_SHA384</code>    | The SHA-384 message digest.    |
| <code>MBEDTLS_MD_SHA512</code>    | The SHA-512 message digest.    |
| <code>MBEDTLS_MD_SHA3_224</code>  | The SHA3-224 message digest.   |
| <code>MBEDTLS_MD_SHA3_256</code>  | The SHA3-256 message digest.   |
| <code>MBEDTLS_MD_SHA3_384</code>  | The SHA3-384 message digest.   |
| <code>MBEDTLS_MD_SHA3_512</code>  | The SHA3-512 message digest.   |

Definition at line 50 of file `md.h`.

### 8.12.5 Function Documentation

#### 8.12.5.1 mbedtls\_md()

```
MBEDTLS_CHECK_RETURN_TYPICAL int mbedtls_md (
 const mbedtls_md_info_t * md_info,
 const unsigned char * input,
 size_t ilen,
 unsigned char * output)
```

This function calculates the message-digest of a buffer, with respect to a configurable message-digest algorithm in a single call.

The result is calculated as `Output = message_digest(input buffer)`.

##### Parameters

|                |                                                                   |
|----------------|-------------------------------------------------------------------|
| <i>md_info</i> | The information structure of the message-digest algorithm to use. |
| <i>input</i>   | The buffer holding the data.                                      |
| <i>ilen</i>    | The length of the input data.                                     |
| <i>output</i>  | The generic message-digest checksum result.                       |

##### Returns

0 on success.

`MBEDTLS_ERR_MD_BAD_INPUT_DATA` on parameter-verification failure.

### 8.12.5.2 mbedtls\_md\_clone()

```
MBEDTLS_CHECK_RETURN_TYPICAL int mbedtls_md_clone (
 mbedtls_md_context_t * dst,
 const mbedtls_md_context_t * src)
```

This function clones the state of a message-digest context.

#### Note

You must call [mbedtls\\_md\\_setup\(\)](#) on `dst` before calling this function.

The two contexts must have the same type, for example, both are SHA-256.

#### Warning

This function clones the message-digest state, not the HMAC state.

#### Parameters

|            |                           |
|------------|---------------------------|
| <i>dst</i> | The destination context.  |
| <i>src</i> | The context to be cloned. |

#### Returns

0 on success.

[MBEDTLS\\_ERR\\_MD\\_BAD\\_INPUT\\_DATA](#) on parameter-verification failure.

[MBEDTLS\\_ERR\\_MD\\_FEATURE\\_UNAVAILABLE](#) if both contexts are not using the same engine. This can be avoided by moving the call to [psa\\_crypto\\_init\(\)](#) before the first call to [mbedtls\\_md\\_setup\(\)](#).

### 8.12.5.3 mbedtls\_md\_finish()

```
MBEDTLS_CHECK_RETURN_TYPICAL int mbedtls_md_finish (
 mbedtls_md_context_t * ctx,
 unsigned char * output)
```

This function finishes the digest operation, and writes the result to the output buffer.

Call this function after a call to [mbedtls\\_md\\_starts\(\)](#), followed by any number of calls to [mbedtls\\_md\\_update\(\)](#). Afterwards, you may either clear the context with [mbedtls\\_md\\_free\(\)](#), or call [mbedtls\\_md\\_starts\(\)](#) to reuse the context for another digest operation with the same algorithm.

#### Parameters

|               |                                                            |
|---------------|------------------------------------------------------------|
| <i>ctx</i>    | The generic message-digest context.                        |
| <i>output</i> | The buffer for the generic message-digest checksum result. |

#### Returns

0 on success.

[MBEDTLS\\_ERR\\_MD\\_BAD\\_INPUT\\_DATA](#) on parameter-verification failure.

### 8.12.5.4 mbedtls\_md\_free()

```
void mbedtls_md_free (
 mbedtls_md_context_t * ctx)
```

This function clears the internal structure of `ctx` and frees any embedded internal structure, but does not free `ctx` itself.

If you have called `MBEDTLS_MD_Setup()` on `ctx`, you must call `MBEDTLS_MD_Free()` when you are no longer using the context. Calling this function if you have previously called `MBEDTLS_MD_Init()` and nothing else is optional. You must not call this function if you have not called `MBEDTLS_MD_Init()`.

#### 8.12.5.5 `MBEDTLS_MD_Get_Size()`

```
unsigned char mbedtls_md_get_size (
 const mbedtls_md_info_t * md_info)
```

This function extracts the message-digest size from the message-digest information structure.

##### Parameters

|                      |                                                                   |
|----------------------|-------------------------------------------------------------------|
| <code>md_info</code> | The information structure of the message-digest algorithm to use. |
|----------------------|-------------------------------------------------------------------|

##### Returns

The size of the message-digest output in Bytes.

#### 8.12.5.6 `MBEDTLS_MD_Get_Type()`

```
mbedtls_md_type_t mbedtls_md_get_type (
 const mbedtls_md_info_t * md_info)
```

This function extracts the message-digest type from the message-digest information structure.

##### Parameters

|                      |                                                                   |
|----------------------|-------------------------------------------------------------------|
| <code>md_info</code> | The information structure of the message-digest algorithm to use. |
|----------------------|-------------------------------------------------------------------|

##### Returns

The type of the message digest.

#### 8.12.5.7 `MBEDTLS_MD_Info_From_Type()`

```
const mbedtls_md_info_t* mbedtls_md_info_from_type (
 mbedtls_md_type_t md_type)
```

This function returns the message-digest information associated with the given digest type.

##### Parameters

|                      |                                   |
|----------------------|-----------------------------------|
| <code>md_type</code> | The type of digest to search for. |
|----------------------|-----------------------------------|

##### Returns

The message-digest information associated with `md_type`.

NULL if the associated message-digest information is not found.

#### 8.12.5.8 `MBEDTLS_MD_Init()`

```
void mbedtls_md_init (
 mbedtls_md_context_t * ctx)
```



This function initializes a message-digest context without binding it to a particular message-digest algorithm. This function should always be called first. It prepares the context for [mbedtls\\_md\\_setup\(\)](#) for binding it to a message-digest algorithm.

#### 8.12.5.9 mbedtls\_md\_setup()

```
MBEDTLS_CHECK_RETURN_TYPICAL int mbedtls_md_setup (
 mbedtls_md_context_t * ctx,
 const mbedtls_md_info_t * md_info,
 int hmac)
```

This function selects the message digest algorithm to use, and allocates internal structures. It should be called after [mbedtls\\_md\\_init\(\)](#) or [mbedtls\\_md\\_free\(\)](#). Makes it necessary to call [mbedtls\\_md\\_free\(\)](#) later.

##### Parameters

|                |                                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------------------|
| <i>ctx</i>     | The context to set up.                                                                                         |
| <i>md_info</i> | The information structure of the message-digest algorithm to use.                                              |
| <i>hmac</i>    | Defines if HMAC is used. 0: HMAC is not used (saves some memory), or non-zero: HMAC is used with this context. |

##### Note

From TF-PSA-Crypto 1.0 and Mbed TLS 4.0 onwards, `hmac` MUST be set to 0. HMAC operations are no longer supported via MD and may only be performed via the `psa_mac` API.

##### Returns

0 on success.

[MBEDTLS\\_ERR\\_MD\\_BAD\\_INPUT\\_DATA](#) on parameter-verification failure.

[MBEDTLS\\_ERR\\_MD\\_ALLOC\\_FAILED](#) on memory-allocation failure.

#### 8.12.5.10 mbedtls\_md\_starts()

```
MBEDTLS_CHECK_RETURN_TYPICAL int mbedtls_md_starts (
 mbedtls_md_context_t * ctx)
```

This function starts a message-digest computation.

You must call this function after setting up the context with [mbedtls\\_md\\_setup\(\)](#), and before passing data with [mbedtls\\_md\\_update\(\)](#).

##### Parameters

|            |                                     |
|------------|-------------------------------------|
| <i>ctx</i> | The generic message-digest context. |
|------------|-------------------------------------|

##### Returns

0 on success.

[MBEDTLS\\_ERR\\_MD\\_BAD\\_INPUT\\_DATA](#) on parameter-verification failure.

#### 8.12.5.11 mbedtls\_md\_update()

```
MBEDTLS_CHECK_RETURN_TYPICAL int mbedtls_md_update (
 mbedtls_md_context_t * ctx,
```

```
const unsigned char * input,
size_t ilen)
```

This function feeds an input buffer into an ongoing message-digest computation.

You must call `MBEDTLS_MD_STARTS()` before calling this function. You may call this function multiple times. Afterwards, call `MBEDTLS_MD_FINISH()`.

#### Parameters

|                    |                                     |
|--------------------|-------------------------------------|
| <code>ctx</code>   | The generic message-digest context. |
| <code>input</code> | The buffer holding the input data.  |
| <code>ilen</code>  | The length of the input data.       |

#### Returns

0 on success.

`MBEDTLS_ERR_MD_BAD_INPUT_DATA` on parameter-verification failure.

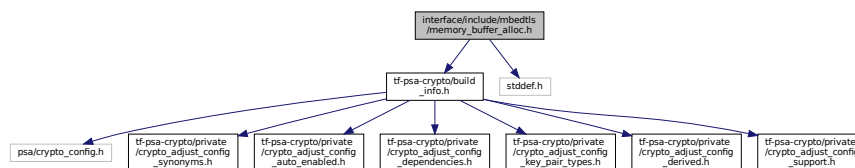
## 8.13 interface/include/mbedtls/memory\_buffer\_alloc.h File Reference

Buffer-based memory allocator.

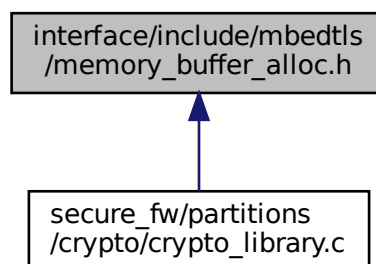
```
#include "tf-psa-crypto/build_info.h"
```

```
#include <stddef.h>
```

Include dependency graph for `memory_buffer_alloc.h`:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define MBEDTLS_MEMORY_VERIFY_NONE 0`
- `#define MBEDTLS_MEMORY_VERIFY_ALLOC (1 << 0)`

- `#define MBEDTLS_MEMORY_VERIFY_FREE` (1 << 1)
- `#define MBEDTLS_MEMORY_VERIFY_ALWAYS`

### SECTION: Module settings

The configuration options you can set for this module are in this section. Either change them in `mbedtls_config.h` or define them on the compiler command line.

- `#define MBEDTLS_MEMORY_ALIGN_MULTIPLE` 4

## Functions

- `void mbedtls_memory_buffer_alloc_init` (unsigned char \*buf, size\_t len)  
Initialize use of stack-based memory allocator. The stack-based allocator does memory management inside the presented buffer and does not call `calloc()` and `free()`. It sets the global `mbedtls_calloc()` and `mbedtls_free()` pointers to its own functions. (Provided `mbedtls_calloc()` and `mbedtls_free()` are thread-safe if `MBEDTLS_THREADING_C` is defined)
- `void mbedtls_memory_buffer_alloc_free` (void)  
Free the mutex for thread-safety and clear remaining memory.
- `void mbedtls_memory_buffer_set_verify` (int verify)  
Determine when the allocator should automatically verify the state of the entire chain of headers / meta-data. (Default: `MBEDTLS_MEMORY_VERIFY_NONE`)
- `int mbedtls_memory_buffer_alloc_verify` (void)  
Verifies that all headers in the memory buffer are correct and contain sane values. Helps debug buffer-overflow errors.

### 8.13.1 Detailed Description

Buffer-based memory allocator.

### 8.13.2 Macro Definition Documentation

#### 8.13.2.1 MBEDTLS\_MEMORY\_ALIGN\_MULTIPLE

```
#define MBEDTLS_MEMORY_ALIGN_MULTIPLE 4
```

Align on multiples of this value  
Definition at line 26 of file `memory_buffer_alloc.h`.

#### 8.13.2.2 MBEDTLS\_MEMORY\_VERIFY\_ALLOC

```
#define MBEDTLS_MEMORY_VERIFY_ALLOC (1 << 0)
```

Definition at line 32 of file `memory_buffer_alloc.h`.

#### 8.13.2.3 MBEDTLS\_MEMORY\_VERIFY\_ALWAYS

```
#define MBEDTLS_MEMORY_VERIFY_ALWAYS
```

**Value:**

```
(MBEDTLS_MEMORY_VERIFY_ALLOC | \
 MBEDTLS_MEMORY_VERIFY_FREE)
```

Definition at line 34 of file `memory_buffer_alloc.h`.

#### 8.13.2.4 MBEDTLS\_MEMORY\_VERIFY\_FREE

```
#define MBEDTLS_MEMORY_VERIFY_FREE (1 << 1)
```

Definition at line 33 of file `memory_buffer_alloc.h`.

### 8.13.2.5 MBEDTLS\_MEMORY\_VERIFY\_NONE

```
#define MBEDTLS_MEMORY_VERIFY_NONE 0
```

Definition at line 31 of file `memory_buffer_alloc.h`.

## 8.13.3 Function Documentation

### 8.13.3.1 mbedtls\_memory\_buffer\_alloc\_free()

```
void mbedtls_memory_buffer_alloc_free (
 void)
```

Free the mutex for thread-safety and clear remaining memory.

### 8.13.3.2 mbedtls\_memory\_buffer\_alloc\_init()

```
void mbedtls_memory_buffer_alloc_init (
 unsigned char * buf,
 size_t len)
```

Initialize use of stack-based memory allocator. The stack-based allocator does memory management inside the presented buffer and does not call `calloc()` and `free()`. It sets the global `mbedtls_calloc()` and `mbedtls_free()` pointers to its own functions. (Provided `mbedtls_calloc()` and `mbedtls_free()` are thread-safe if `MBEDTLS_THREADING_C` is defined)

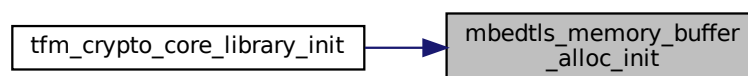
#### Note

This code is not optimized and provides a straight-forward implementation of a stack-based memory allocator.

#### Parameters

|            |                       |
|------------|-----------------------|
| <i>buf</i> | buffer to use as heap |
| <i>len</i> | size of the buffer    |

Here is the caller graph for this function:



### 8.13.3.3 mbedtls\_memory\_buffer\_alloc\_verify()

```
int mbedtls_memory_buffer_alloc_verify (
 void)
```

Verifies that all headers in the memory buffer are correct and contain sane values. Helps debug buffer-overflow errors.

Prints out first failure if `MBEDTLS_MEMORY_DEBUG` is defined. Prints out full header information if `MBEDTLS_MEMORY_DEBUG` is defined. (Includes stack trace information for each block if `MBEDTLS_MEMORY_BACKTRACE` is defined as well).

## Returns

0 if verified, 1 otherwise

## 8.13.3.4 mbedtls\_memory\_buffer\_set\_verify()

```
void mbedtls_memory_buffer_set_verify (
 int verify)
```

Determine when the allocator should automatically verify the state of the entire chain of headers / meta-data. (Default: MBEDTLS\_MEMORY\_VERIFY\_NONE)

## Parameters

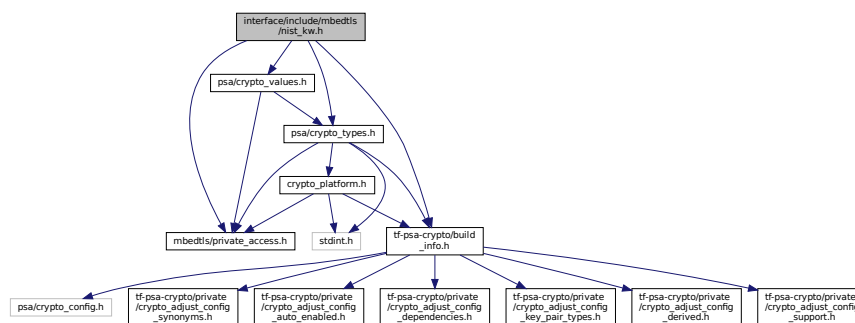
|               |                                                                                                                            |
|---------------|----------------------------------------------------------------------------------------------------------------------------|
| <i>verify</i> | One of MBEDTLS_MEMORY_VERIFY_NONE, MBEDTLS_MEMORY_VERIFY_ALLOC, MBEDTLS_MEMORY_VERIFY_FREE or MBEDTLS_MEMORY_VERIFY_ALWAYS |
|---------------|----------------------------------------------------------------------------------------------------------------------------|

## 8.14 interface/include/mbedtls/nist\_kw.h File Reference

This file provides an API for key wrapping (KW) and key wrapping with padding (KWP) as defined in NIST SP 800-38F. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-38F.pdf>.

```
#include "mbedtls/private_access.h"
#include "tf-psa-crypto/build_info.h"
#include "psa/crypto_types.h"
#include "psa/crypto_values.h"
```

Include dependency graph for nist\_kw.h:



## Enumerations

- enum `mbedtls_nist_kw_mode_t` { `MBEDTLS_KW_MODE_KW` = 0, `MBEDTLS_KW_MODE_KWP` = 1 }

## Functions

- `psa_status_t` `mbedtls_nist_kw_wrap` (`mbedtls_svc_key_id_t` key, `mbedtls_nist_kw_mode_t` mode, const unsigned char \*input, `size_t` input\_length, unsigned char \*output, `size_t` output\_size, `size_t` \*output\_length)

*This function encrypts a buffer using key wrapping.*

- `psa_status_t` `mbedtls_nist_kw_unwrap` (`mbedtls_svc_key_id_t` key, `mbedtls_nist_kw_mode_t` mode, const unsigned char \*input, `size_t` input\_length, unsigned char \*output, `size_t` output\_size, `size_t` \*output\_length)

*This function decrypts a buffer using key wrapping.*

### 8.14.1 Detailed Description

This file provides an API for key wrapping (KW) and key wrapping with padding (KWP) as defined in NIST SP 800-38F. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-38F.pdf>.

Key wrapping specifies a deterministic authenticated-encryption mode of operation, according to *NIST SP 800-38F: Recommendation for Block Cipher Modes of Operation: Methods for Key Wrapping*. Its purpose is to protect cryptographic keys.

Its equivalent is RFC 3394 for KW, and RFC 5649 for KWP. <https://tools.ietf.org/html/rfc3394>  
<https://tools.ietf.org/html/rfc5649>

### 8.14.2 Enumeration Type Documentation

#### 8.14.2.1 mbedtls\_nist\_kw\_mode\_t

```
enum mbedtls_nist_kw_mode_t
```

Enumerator

|                     |  |
|---------------------|--|
| MBEDTLS_KW_MODE_KW  |  |
| MBEDTLS_KW_MODE_KWP |  |

Definition at line 35 of file nist\_kw.h.

### 8.14.3 Function Documentation

#### 8.14.3.1 mbedtls\_nist\_kw\_unwrap()

```
psa_status_t mbedtls_nist_kw_unwrap (
 mbedtls_svc_key_id_t key,
 mbedtls_nist_kw_mode_t mode,
 const unsigned char * input,
 size_t input_length,
 unsigned char * output,
 size_t output_size,
 size_t * output_length)
```

This function decrypts a buffer using key wrapping.

Parameters

|              |                                                                                                                                                                                                                                                                                                                                                                     |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>key</i>   | The key wrapping PSA key ID to use for encryption. The key should have the following attributes: <ul style="list-style-type: none"> <li>• type: <a href="#">PSA_KEY_TYPE_AES</a></li> <li>• algorithm: <a href="#">PSA_ALG_ECB_NO_PADDING</a></li> <li>• usage flag: <a href="#">PSA_KEY_USAGE_DECRYPT</a> + other flags if required by the application.</li> </ul> |
| <i>mode</i>  | The key wrapping mode to use ( <a href="#">MBEDTLS_KW_MODE_KW</a> or <a href="#">MBEDTLS_KW_MODE_KWP</a> )                                                                                                                                                                                                                                                          |
| <i>input</i> | The buffer holding the input data.                                                                                                                                                                                                                                                                                                                                  |

## Parameters

|     |                      |                                                                                                                                                                                                                                                                                                                                                                         |
|-----|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>input_length</i>  | The length of the input data in Bytes. The input uses units of 8 Bytes called semiblocks. The input must be a multiple of semiblocks. <ul style="list-style-type: none"> <li>• For KW mode: a multiple of 8 bytes between 24 and <math>2^{57}</math> inclusive.</li> <li>• For KWP mode: a multiple of 8 bytes between 16 and <math>2^{32}</math> inclusive.</li> </ul> |
| out | <i>output</i>        | The buffer holding the output data. The output buffer's minimal length is 8 bytes shorter than <i>in_len</i> .                                                                                                                                                                                                                                                          |
|     | <i>output_size</i>   | The capacity of the output buffer.                                                                                                                                                                                                                                                                                                                                      |
| out | <i>output_length</i> | The number of bytes written to the output buffer. 0 on failure. For KWP mode, the length could be up to 15 bytes shorter than <i>in_len</i> , depending on how much padding was added to the data.                                                                                                                                                                      |

## Returns

0 on success.

[PSA\\_ERROR\\_INVALID\\_ARGUMENT](#) for invalid input length.

[PSA\\_ERROR\\_INVALID\\_SIGNATURE](#) for invalid ciphertext.

Another error code on failure of the underlying cipher.

## 8.14.3.2 mbedtls\_nist\_kw\_wrap()

```
psa_status_t mbedtls_nist_kw_wrap (
 mbedtls_svc_key_id_t key,
 mbedtls_nist_kw_mode_t mode,
 const unsigned char * input,
 size_t input_length,
 unsigned char * output,
 size_t output_size,
 size_t * output_length)
```

This function encrypts a buffer using key wrapping.

## Parameters

|  |                     |                                                                                                                                                                                                                                                                                                                                                                     |
|--|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <i>key</i>          | The key wrapping PSA key ID to use for encryption. The key should have the following attributes: <ul style="list-style-type: none"> <li>• type: <a href="#">PSA_KEY_TYPE_AES</a></li> <li>• algorithm: <a href="#">PSA_ALG_ECB_NO_PADDING</a></li> <li>• usage flag: <a href="#">PSA_KEY_USAGE_ENCRYPT</a> + other flags if required by the application.</li> </ul> |
|  | <i>mode</i>         | The key wrapping mode to use ( <a href="#">MBEDTLS_KW_MODE_KW</a> or <a href="#">MBEDTLS_KW_MODE_KWP</a> )                                                                                                                                                                                                                                                          |
|  | <i>input</i>        | The buffer holding the input data.                                                                                                                                                                                                                                                                                                                                  |
|  | <i>input_length</i> | The length of the input data in Bytes. The input uses units of 8 Bytes called semiblocks. <ul style="list-style-type: none"> <li>• For KW mode: a multiple of 8 bytes between 16 and <math>2^{57-8}</math> inclusive.</li> <li>• For KWP mode: any length between 1 and <math>2^{32-1}</math> inclusive.</li> </ul>                                                 |

## Parameters

|     |                      |                                                                                                                                                                                                                                                                               |
|-----|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out | <i>output</i>        | The buffer holding the output data. <ul style="list-style-type: none"> <li>For KW mode: Must be at least 8 bytes larger than <i>in_len</i>.</li> <li>For KWP mode: Must be at least 8 bytes larger rounded up to a multiple of 8 bytes for KWP (15 bytes at most).</li> </ul> |
|     | <i>output_size</i>   | The capacity of the output buffer. 0 on failure.                                                                                                                                                                                                                              |
| out | <i>output_length</i> | On success, the number of bytes written to the output buffer. 0 on failure.                                                                                                                                                                                                   |

## Returns

0 on success.

[PSA\\_ERROR\\_INVALID\\_ARGUMENT](#) for invalid input length.

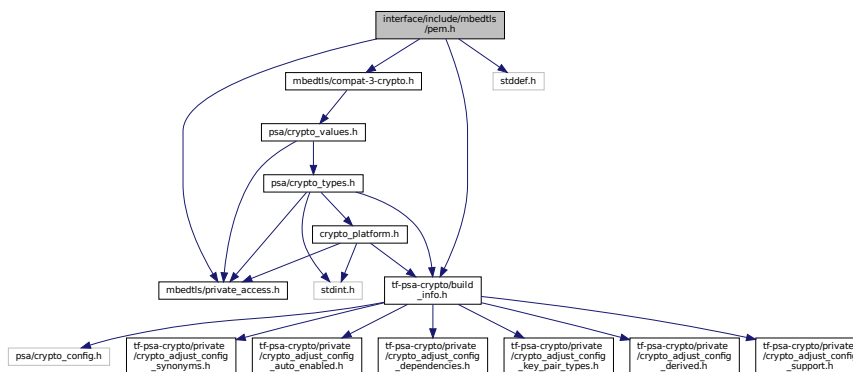
Another error code on failure of the underlying cipher.

## 8.15 interface/include/mbedtls/pem.h File Reference

Privacy Enhanced Mail (PEM) decoding.

```
#include "mbedtls/private_access.h"
#include "tf-psa-crypto/build_info.h"
#include "mbedtls/compat-3-crypto.h"
#include <stddef.h>
```

Include dependency graph for pem.h:



## Macros

### PEM Error codes

These error codes are returned in case of errors reading the PEM data.

- #define [MBEDTLS\\_ERR\\_PEM\\_NO\\_HEADER\\_FOOTER\\_PRESENT](#) -0x1080
- #define [MBEDTLS\\_ERR\\_PEM\\_INVALID\\_DATA](#) -0x1100
- #define [MBEDTLS\\_ERR\\_PEM\\_INVALID\\_ENC\\_IV](#) -0x1200
- #define [MBEDTLS\\_ERR\\_PEM\\_UNKNOWN\\_ENC\\_ALG](#) -0x1280
- #define [MBEDTLS\\_ERR\\_PEM\\_PASSWORD\\_REQUIRED](#) -0x1300
- #define [MBEDTLS\\_ERR\\_PEM\\_PASSWORD\\_MISMATCH](#) -0x1380
- #define [MBEDTLS\\_ERR\\_PEM\\_FEATURE\\_UNAVAILABLE](#) -0x1400

### 8.15.1 Detailed Description

Privacy Enhanced Mail (PEM) decoding.



## 8.15.2 Macro Definition Documentation

### 8.15.2.1 MBEDTLS\_ERR\_PEM\_FEATURE\_UNAVAILABLE

#define MBEDTLS\_ERR\_PEM\_FEATURE\_UNAVAILABLE -0x1400  
Unavailable feature, e.g. hashing/encryption combination.  
Definition at line 38 of file pem.h.

### 8.15.2.2 MBEDTLS\_ERR\_PEM\_INVALID\_DATA

#define MBEDTLS\_ERR\_PEM\_INVALID\_DATA -0x1100  
PEM string is not as expected.  
Definition at line 28 of file pem.h.

### 8.15.2.3 MBEDTLS\_ERR\_PEM\_INVALID\_ENC\_IV

#define MBEDTLS\_ERR\_PEM\_INVALID\_ENC\_IV -0x1200  
RSA IV is not in hex-format.  
Definition at line 30 of file pem.h.

### 8.15.2.4 MBEDTLS\_ERR\_PEM\_NO\_HEADER\_FOOTER\_PRESENT

#define MBEDTLS\_ERR\_PEM\_NO\_HEADER\_FOOTER\_PRESENT -0x1080  
No PEM header or footer found.  
Definition at line 26 of file pem.h.

### 8.15.2.5 MBEDTLS\_ERR\_PEM\_PASSWORD\_MISMATCH

#define MBEDTLS\_ERR\_PEM\_PASSWORD\_MISMATCH -0x1380  
Given private key password does not allow for correct decryption.  
Definition at line 36 of file pem.h.

### 8.15.2.6 MBEDTLS\_ERR\_PEM\_PASSWORD\_REQUIRED

#define MBEDTLS\_ERR\_PEM\_PASSWORD\_REQUIRED -0x1300  
Private key password can't be empty.  
Definition at line 34 of file pem.h.

### 8.15.2.7 MBEDTLS\_ERR\_PEM\_UNKNOWN\_ENC\_ALG

#define MBEDTLS\_ERR\_PEM\_UNKNOWN\_ENC\_ALG -0x1280  
Unsupported key encryption algorithm.  
Definition at line 32 of file pem.h.

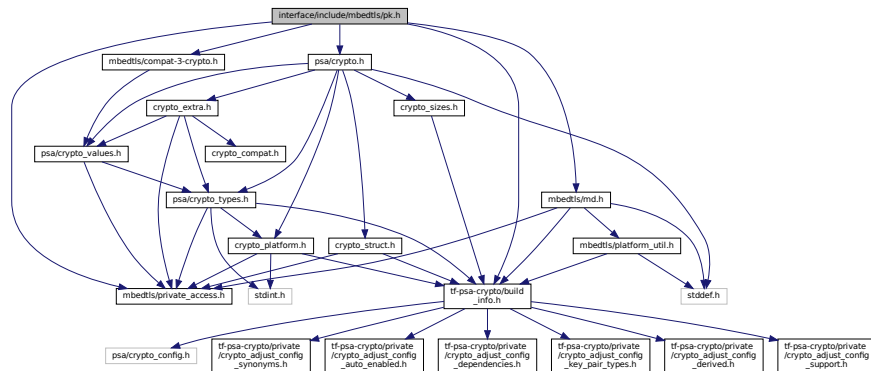
## 8.16 interface/include/mbedtls/pk.h File Reference

Public Key abstraction layer.

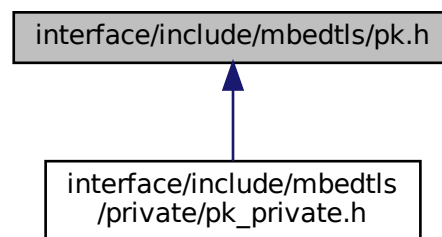
```
#include "mbedtls/private_access.h"
#include "tf-psa-crypto/build_info.h"
#include "mbedtls/compat-3-crypto.h"
#include "mbedtls/md.h"
```

```
#include "psa/crypto.h"
```

Include dependency graph for pk.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `mbedtls_pk_context`  
*Public key container.*

## Macros

- `#define MBEDTLS_PK_HAVE_PRIVATE_HEADER`
- `#define MBEDTLS_ERR_PK_TYPE_MISMATCH -0x3F00`
- `#define MBEDTLS_ERR_PK_FILE_IO_ERROR -0x3E00`
- `#define MBEDTLS_ERR_PK_KEY_INVALID_VERSION -0x3D80`
- `#define MBEDTLS_ERR_PK_KEY_INVALID_FORMAT -0x3D00`
- `#define MBEDTLS_ERR_PK_UNKNOWN_PK_ALG -0x3C80`
- `#define MBEDTLS_ERR_PK_PASSWORD_REQUIRED -0x3C00`
- `#define MBEDTLS_ERR_PK_PASSWORD_MISMATCH -0x3B80`
- `#define MBEDTLS_ERR_PK_INVALID_PUBKEY -0x3B00`
- `#define MBEDTLS_ERR_PK_INVALID_ALG -0x3A80`
- `#define MBEDTLS_ERR_PK_UNKNOWN_NAMED_CURVE -0x3A00`
- `#define MBEDTLS_ERR_PK_FEATURE_UNAVAILABLE -0x3980`
- `#define MBEDTLS_PK_SIGNATURE_MAX_SIZE PSA_SIGNATURE_MAX_SIZE`

*Maximum size of a signature made by `mbedtls_pk_sign()` and other signature functions.*

- `#define MBEDTLS_PK_SIGNATURE_MAX_SIZE (PSA_VENDOR_ECDSA_SIGNATURE_MAX_SIZE + 11)`  
*Maximum size of a signature made by `mbedtls_pk_sign()` and other signature functions.*
- `#define MBEDTLS_PK_USE_PSA_EC_DATA`
- `#define MBEDTLS_PK_USE_PSA_RSA_DATA`
- `#define MBEDTLS_PK_MAX_EC_PUBKEY_RAW_LEN PSA_KEY_EXPORT_ECC_PUBLIC_KEY_MAX_SIZE(PSA_VENDOR_ECDSA_MAX_KEY_SIZE)`
- `#define MBEDTLS_PK_MAX_RSA_PUBKEY_RAW_LEN PSA_KEY_EXPORT_RSA_PUBLIC_KEY_MAX_SIZE(PSA_VENDOR_RSA_MAX_KEY_SIZE)`
- `#define MBEDTLS_PK_MAX_PUBKEY_RAW_LEN 0`
- `#define MBEDTLS_PK_ALG_ECDSA(hash_alg) PSA_ALG_ECDSA(hash_alg)`

## Typedefs

- `typedef struct mbedtls_pk_info_t mbedtls_pk_info_t`
- `typedef struct mbedtls_pk_context mbedtls_pk_context`  
*Public key container.*
- `typedef void mbedtls_pk_restart_ctx`

## Enumerations

- `enum mbedtls_pk_sigalg_t { MBEDTLS_PK_SIGALG_NONE = 0, MBEDTLS_PK_SIGALG_RSA_PKCS1V15, MBEDTLS_PK_SIGALG_RSA_PSS, MBEDTLS_PK_SIGALG_ECDSA }`

## Functions

- `void mbedtls_pk_init (mbedtls_pk_context *ctx)`  
*Initialize a `mbedtls_pk_context` (as empty).*
- `void mbedtls_pk_free (mbedtls_pk_context *ctx)`  
*Empty a `mbedtls_pk_context`. After this, the context can be re-used as if it had been freshly initialized.*
- `int mbedtls_pk_wrap_psa (mbedtls_pk_context *ctx, const mbedtls_svc_key_id_t key)`  
*Populate a PK context by wrapping a PSA key pair.*
- `size_t mbedtls_pk_get_bitlen (const mbedtls_pk_context *ctx)`  
*Get the size in bits of the underlying key.*
- `int mbedtls_pk_can_do_psa (const mbedtls_pk_context *pk, psa_algorithm_t alg, psa_key_usage_t usage)`  
*Tell if the key wrapped in the PK context is able to perform the `usage` operation using the `alg` algorithm.*
- `int mbedtls_pk_get_psa_attributes (const mbedtls_pk_context *pk, psa_key_usage_t usage, psa_key_attributes_t *attributes)`  
*Determine valid PSA attributes that can be used to import a key into PSA.*
- `psa_key_type_t mbedtls_pk_get_key_type (const mbedtls_pk_context *pk)`  
*Get the PSA key type corresponding to the key represented by the given PK context.*
- `int mbedtls_pk_import_into_psa (const mbedtls_pk_context *pk, const psa_key_attributes_t *attributes, mbedtls_svc_key_id_t *key_id)`  
*Import a key into the PSA key store.*
- `int mbedtls_pk_copy_from_psa (mbedtls_svc_key_id_t key_id, mbedtls_pk_context *pk)`  
*Populate a PK context with the key material from a PSA key.*
- `int mbedtls_pk_copy_public_from_psa (mbedtls_svc_key_id_t key_id, mbedtls_pk_context *pk)`  
*Populate a PK context with the public key material of a PSA key.*
- `int mbedtls_pk_verify (mbedtls_pk_context *ctx, mbedtls_md_type_t md_alg, const unsigned char *hash, size_t hash_len, const unsigned char *sig, size_t sig_len)`  
*Verify signature.*
- `int mbedtls_pk_verify_restartable (mbedtls_pk_context *ctx, mbedtls_md_type_t md_alg, const unsigned char *hash, size_t hash_len, const unsigned char *sig, size_t sig_len, mbedtls_pk_restart_ctx *rs_ctx)`  
*Restartable version of `mbedtls_pk_verify()`*
- `int mbedtls_pk_verify_ext (mbedtls_pk_sigalg_t type, mbedtls_pk_context *ctx, mbedtls_md_type_t md_alg, const unsigned char *hash, size_t hash_len, const unsigned char *sig, size_t sig_len)`

*Verify signature, selecting a specific algorithm.*

- int `MBEDTLS_PK_SIGN` (`MBEDTLS_PK_CONTEXT` \*ctx, `MBEDTLS_MD_TYPE_T` md\_alg, const unsigned char \*hash, size\_t hash\_len, unsigned char \*sig, size\_t sig\_size, size\_t \*sig\_len)

*Make signature.*

- int `MBEDTLS_PK_SIGN_RESTARTABLE` (`MBEDTLS_PK_CONTEXT` \*ctx, `MBEDTLS_MD_TYPE_T` md\_alg, const unsigned char \*hash, size\_t hash\_len, unsigned char \*sig, size\_t sig\_size, size\_t \*sig\_len, `MBEDTLS_PK_RESTART_CTX` \*rs\_ctx)

*Restartable version of `MBEDTLS_PK_SIGN` ()*

- int `MBEDTLS_PK_SIGN_EXT` (`MBEDTLS_PK_SIGALG_T` sig\_type, `MBEDTLS_PK_CONTEXT` \*ctx, `MBEDTLS_MD_TYPE_T` md\_alg, const unsigned char \*hash, size\_t hash\_len, unsigned char \*sig, size\_t sig\_size, size\_t \*sig\_len)

*Generate a signature, selecting a specific algorithm.*

- int `MBEDTLS_PK_CHECK_PAIR` (const `MBEDTLS_PK_CONTEXT` \*pub, const `MBEDTLS_PK_CONTEXT` \*prv)

*Check if a public-private pair of keys matches.*

## 8.16.1 Detailed Description

Public Key abstraction layer.

## 8.16.2 Macro Definition Documentation

### 8.16.2.1 MBEDTLS\_ERR\_PK\_FEATURE\_UNAVAILABLE

```
#define MBEDTLS_ERR_PK_FEATURE_UNAVAILABLE -0x3980
```

Unavailable feature, e.g. RSA disabled for RSA key.

Definition at line 43 of file pk.h.

### 8.16.2.2 MBEDTLS\_ERR\_PK\_FILE\_IO\_ERROR

```
#define MBEDTLS_ERR_PK_FILE_IO_ERROR -0x3E00
```

Read/write of file failed.

Definition at line 25 of file pk.h.

### 8.16.2.3 MBEDTLS\_ERR\_PK\_INVALID\_ALG

```
#define MBEDTLS_ERR_PK_INVALID_ALG -0x3A80
```

The algorithm tag or value is invalid.

Definition at line 39 of file pk.h.

### 8.16.2.4 MBEDTLS\_ERR\_PK\_INVALID\_PUBKEY

```
#define MBEDTLS_ERR_PK_INVALID_PUBKEY -0x3B00
```

The pubkey tag or value is invalid (only RSA and EC are supported).

Definition at line 37 of file pk.h.

### 8.16.2.5 MBEDTLS\_ERR\_PK\_KEY\_INVALID\_FORMAT

```
#define MBEDTLS_ERR_PK_KEY_INVALID_FORMAT -0x3D00
```

Invalid key tag or value.

Definition at line 29 of file pk.h.

#### 8.16.2.6 MBEDTLS\_ERR\_PK\_KEY\_INVALID\_VERSION

```
#define MBEDTLS_ERR_PK_KEY_INVALID_VERSION -0x3D80
```

Unsupported key version

Definition at line 27 of file pk.h.

#### 8.16.2.7 MBEDTLS\_ERR\_PK\_PASSWORD\_MISMATCH

```
#define MBEDTLS_ERR_PK_PASSWORD_MISMATCH -0x3B80
```

Given private key password does not allow for correct decryption.

Definition at line 35 of file pk.h.

#### 8.16.2.8 MBEDTLS\_ERR\_PK\_PASSWORD\_REQUIRED

```
#define MBEDTLS_ERR_PK_PASSWORD_REQUIRED -0x3C00
```

Private key password can't be empty.

Definition at line 33 of file pk.h.

#### 8.16.2.9 MBEDTLS\_ERR\_PK\_TYPE\_MISMATCH

```
#define MBEDTLS_ERR_PK_TYPE_MISMATCH -0x3F00
```

Type mismatch, eg attempt to do ECDSA with an RSA key

Definition at line 23 of file pk.h.

#### 8.16.2.10 MBEDTLS\_ERR\_PK\_UNKNOWN\_NAMED\_CURVE

```
#define MBEDTLS_ERR_PK_UNKNOWN_NAMED_CURVE -0x3A00
```

Elliptic curve is unsupported (only NIST curves are supported).

Definition at line 41 of file pk.h.

#### 8.16.2.11 MBEDTLS\_ERR\_PK\_UNKNOWN\_PK\_ALG

```
#define MBEDTLS_ERR_PK_UNKNOWN_PK_ALG -0x3C80
```

Key algorithm is unsupported (only RSA and EC are supported).

Definition at line 31 of file pk.h.

#### 8.16.2.12 MBEDTLS\_PK\_ALG\_ECDSA

```
#define MBEDTLS_PK_ALG_ECDSA(
 hash_alg) PSA_ALG_ECDSA(hash_alg)
```

This helper exposes which ECDSA variant the PK module uses by default: this is deterministic ECDSA if available, or randomized otherwise.

##### Warning

This default algorithm selection might change in the future.

Definition at line 161 of file pk.h.

#### 8.16.2.13 MBEDTLS\_PK\_HAVE\_PRIVATE\_HEADER

```
#define MBEDTLS_PK_HAVE_PRIVATE_HEADER
```

Definition at line 13 of file pk.h.

#### 8.16.2.14 MBEDTLS\_PK\_MAX\_EC\_PUBKEY\_RAW\_LEN

```
#define MBEDTLS_PK_MAX_EC_PUBKEY_RAW_LEN PSA_KEY_EXPORT_ECC_PUBLIC_KEY_MAX_SIZE(PSA_VENDOR_ECC_MAX_CURVE_BITS)
```

Definition at line 85 of file pk.h.

#### 8.16.2.15 MBEDTLS\_PK\_MAX\_PUBKEY\_RAW\_LEN

```
#define MBEDTLS_PK_MAX_PUBKEY_RAW_LEN 0
```

Definition at line 91 of file pk.h.

#### 8.16.2.16 MBEDTLS\_PK\_MAX\_RSA\_PUBKEY\_RAW\_LEN

```
#define MBEDTLS_PK_MAX_RSA_PUBKEY_RAW_LEN PSA_KEY_EXPORT_RSA_PUBLIC_KEY_MAX_SIZE(PSA_VENDOR_RSA_MAX_KEY_BITS)
```

Definition at line 88 of file pk.h.

#### 8.16.2.17 MBEDTLS\_PK\_SIGNATURE\_MAX\_SIZE [1/2]

```
#define MBEDTLS_PK_SIGNATURE_MAX_SIZE PSA_SIGNATURE_MAX_SIZE
```

Maximum size of a signature made by `mbedtls_pk_sign()` and other signature functions.  
Definition at line 73 of file pk.h.

#### 8.16.2.18 MBEDTLS\_PK\_SIGNATURE\_MAX\_SIZE [2/2]

```
#define MBEDTLS_PK_SIGNATURE_MAX_SIZE (PSA_VENDOR_ECDSA_SIGNATURE_MAX_SIZE + 11)
```

Maximum size of a signature made by `mbedtls_pk_sign()` and other signature functions.  
Definition at line 73 of file pk.h.

#### 8.16.2.19 MBEDTLS\_PK\_USE\_PSA\_EC\_DATA

```
#define MBEDTLS_PK_USE_PSA_EC_DATA
```

Definition at line 79 of file pk.h.

#### 8.16.2.20 MBEDTLS\_PK\_USE\_PSA\_RSA\_DATA

```
#define MBEDTLS_PK_USE_PSA_RSA_DATA
```

Definition at line 80 of file pk.h.

### 8.16.3 Typedef Documentation

#### 8.16.3.1 mbedtls\_pk\_context

```
typedef struct mbedtls_pk_context mbedtls_pk_context
```

Public key container.

#### 8.16.3.2 mbedtls\_pk\_info\_t

```
typedef struct mbedtls_pk_info_t mbedtls_pk_info_t
```

Definition at line 83 of file pk.h.

### 8.16.3.3 mbedtls\_pk\_restart\_ctx

typedef void mbedtls\_pk\_restart\_ctx  
Definition at line 149 of file pk.h.

## 8.16.4 Enumeration Type Documentation

### 8.16.4.1 mbedtls\_pk\_sigalg\_t

enum mbedtls\_pk\_sigalg\_t

Enumerator

|                                |  |
|--------------------------------|--|
| MBEDTLS_PK_SIGALG_NONE         |  |
| MBEDTLS_PK_SIGALG_RSA_PKCS1V15 |  |
| MBEDTLS_PK_SIGALG_RSA_PSS      |  |
| MBEDTLS_PK_SIGALG_ECDSA        |  |

Definition at line 49 of file pk.h.

## 8.16.5 Function Documentation

### 8.16.5.1 mbedtls\_pk\_can\_do\_psa()

```
int mbedtls_pk_can_do_psa (
 const mbedtls_pk_context * pk,
 psa_algorithm_t alg,
 psa_key_usage_t usage)
```

Tell if the key wrapped in the PK context is able to perform the `usage` operation using the `alg` algorithm. The operation may be a PK function, a PSA operation on the underlying PSA key if the PK object wraps a PSA key, or a PSA operation on a key obtained with [mbedtls\\_pk\\_import\\_into\\_psa\(\)](#).

Note

As of TF-PSA-Crypto 1.0.0, this function returns 0 if the key type and policy are suitable for the requested algorithm and usage, even if the key would not work for some other reason, for example an RSA key that is too small for OAEP with the specified hash. This behavior may change without notice in future versions of the library.

Parameters

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pk</i>  | The context to query. It must have been populated.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>alg</i> | PSA algorithm to check against. Allowed values are: <ul style="list-style-type: none"> <li>• <a href="#">PSA_ALG_RSA_PKCS1V15_SIGN(hash)</a>,</li> <li>• <a href="#">PSA_ALG_RSA_PSS(hash)</a>,</li> <li>• <a href="#">PSA_ALG_RSA_PSS_ANY_SALT(hash)</a>,</li> <li>• <a href="#">PSA_ALG_RSA_PKCS1V15_CRYPT</a>,</li> <li>• <a href="#">PSA_ALG_RSA_OAEP(hash)</a>,</li> <li>• <a href="#">PSA_ALG_ECDSA(hash)</a>,</li> <li>• <a href="#">MBEDTLS_PK_ALG_ECDSA(hash)</a>, where hash is a specified algorithm.</li> </ul> |

## Parameters

|              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>usage</i> | <p>PSA usage flag that the key must be verified against. A single flag from the following list must be specified:</p> <ul style="list-style-type: none"> <li>• <a href="#">PSA_KEY_USAGE_SIGN_HASH</a>,</li> <li>• <a href="#">PSA_KEY_USAGE_VERIFY_HASH</a>,</li> <li>• <a href="#">PSA_KEY_USAGE_DECRYPT</a>,</li> <li>• <a href="#">PSA_KEY_USAGE_ENCRYPT</a>,</li> <li>• <a href="#">PSA_KEY_USAGE_DERIVE</a>,</li> <li>• <a href="#">PSA_KEY_USAGE_DERIVE_PUBLIC</a>.</li> </ul> |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

- 1 if the key can do operation on the given type.
- 0 if the key cannot do the operations, or the context has not been populated.

8.16.5.2 `mbedtls_pk_check_pair()`

```
int mbedtls_pk_check_pair (
 const mbedtls_pk_context * pub,
 const mbedtls_pk_context * prv)
```

Check if a public-private pair of keys matches.

## Parameters

|            |                                             |
|------------|---------------------------------------------|
| <i>pub</i> | Context holding a public key.               |
| <i>prv</i> | Context holding a private (and public) key. |

## Returns

- 0 on success (keys were checked and match each other).
- [PSA\\_ERROR\\_INVALID\\_ARGUMENT](#) if a context is invalid.
- Another non-zero value if the keys do not match.

8.16.5.3 `mbedtls_pk_copy_from_psa()`

```
int mbedtls_pk_copy_from_psa (
 mbedtls_svc_key_id_t key_id,
 mbedtls_pk_context * pk)
```

Populate a PK context with the key material from a PSA key.

This key:

- must be exportable and
- must be an RSA or EC key pair or public key (FFDH is not supported in PK).

Once this function returns the PK object will be completely independent from the original PSA key that it was generated from.



**Note**

This function only copies the key material but discards policy information entirely. See [mbedtls\\_pk\\_get\\_psa\\_attribute](#) for details on which algorithm is going to be used by PK for contexts populated with this function.

If you want to retain the PSA policy, see [mbedtls\\_pk\\_wrap\\_psa\(\)](#) - but then the PSA key needs to live at least as long as the PK context.

**Parameters**

|               |                                               |
|---------------|-----------------------------------------------|
| <i>key_id</i> | The key identifier of the key stored in PSA.  |
| <i>pk</i>     | The PK context to populate. It must be empty. |

**Returns**

0 on success.

[PSA\\_ERROR\\_INVALID\\_ARGUMENT](#) in case the provided input parameters are not correct.

**8.16.5.4 mbedtls\_pk\_copy\_public\_from\_psa()**

```
int mbedtls_pk_copy_public_from_psa (
 mbedtls_svc_key_id_t key_id,
 mbedtls_pk_context * pk)
```

Populate a PK context with the public key material of a PSA key.

The key must be an RSA or ECC key. It can be either a public key or a key pair, and only the public key is copied.

Once this function returns the PK object will be completely independent from the original PSA key that it was generated from.

**Note**

This function only copies the key material but discards policy information entirely. See [mbedtls\\_pk\\_get\\_psa\\_attribute](#) for details on which algorithm is going to be used by PK for contexts populated with this function.

If you want to retain the PSA policy, see [mbedtls\\_pk\\_wrap\\_psa\(\)](#) - but then the PSA key needs to live at least as long as the PK context.

**Parameters**

|               |                                               |
|---------------|-----------------------------------------------|
| <i>key_id</i> | The key identifier of the key stored in PSA.  |
| <i>pk</i>     | The PK context to populate. It must be empty. |

**Returns**

0 on success.

[PSA\\_ERROR\\_INVALID\\_ARGUMENT](#) in case the provided input parameters are not correct.

**8.16.5.5 mbedtls\_pk\_free()**

```
void mbedtls_pk_free (
 mbedtls_pk_context * ctx)
```

Empty a [mbedtls\\_pk\\_context](#). After this, the context can be re-used as if it had been freshly initialized.

## Parameters

|                  |                                                                                                                 |
|------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>ctx</code> | The context to clear. It must have been initialized. If this is <code>NULL</code> , this function does nothing. |
|------------------|-----------------------------------------------------------------------------------------------------------------|

## Note

For contexts that have been populated with [mbedtls\\_pk\\_wrap\\_psa\(\)](#), this does not free the underlying PSA key and you still need to call [psa\\_destroy\\_key\(\)](#) independently if you want to destroy that key.

8.16.5.6 `mbedtls_pk_get_bitlen()`

```
size_t mbedtls_pk_get_bitlen (
 const mbedtls_pk_context * ctx)
```

Get the size in bits of the underlying key.

## Parameters

|                  |                                                    |
|------------------|----------------------------------------------------|
| <code>ctx</code> | The context to query. It must have been populated. |
|------------------|----------------------------------------------------|

## Returns

Key size in bits, or 0 on error

8.16.5.7 `mbedtls_pk_get_key_type()`

```
psa_key_type_t mbedtls_pk_get_key_type (
 const mbedtls_pk_context * pk)
```

Get the PSA key type corresponding to the key represented by the given PK context.

## Parameters

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <code>pk</code> | The context to query. It must already be initialized. |
|-----------------|-------------------------------------------------------|

## Returns

A PSA key type. Specifically, one of:

- `PSA_KEY_TYPE_RSA_KEY_PAIR`
- `PSA_KEY_TYPE_RSA_PUBLIC_KEY`
- [PSA\\_KEY\\_TYPE\\_ECC\\_KEY\\_PAIR\(curve\)](#)
- [PSA\\_KEY\\_TYPE\\_ECC\\_PUBLIC\\_KEY\(curve\)](#)

`PSA_KEY_TYPE_NONE`, if the context has not been populated.

8.16.5.8 `mbedtls_pk_get_psa_attributes()`

```
int mbedtls_pk_get_psa_attributes (
 const mbedtls_pk_context * pk,
 psa_key_usage_t usage,
 psa_key_attributes_t * attributes)
```

Determine valid PSA attributes that can be used to import a key into PSA.

The attributes determined by this function are suitable for calling [mbedtls\\_pk\\_import\\_into\\_psa\(\)](#) to create a PSA key with the same key material.

The typical flow of operations involving this function is

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
int ret = mbedtls_pk_get_psa_attributes(pk, &attributes);
if (ret != 0) ...; // error handling omitted
// Tweak attributes if desired
psa_key_id_t key_id = 0;
ret = mbedtls_pk_import_into_psa(pk, &attributes, &key_id);
if (ret != 0) ...; // error handling omitted
```

#### Parameters

|    |              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>pk</i>    | The PK context to use. It must have been populated. It can either contain a key pair or just a public key.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|    | <i>usage</i> | <p>A single <code>PSA_KEY_USAGE_XXX</code> flag among the following:</p> <ul style="list-style-type: none"> <li><b>PSA_KEY_USAGE_DECRYPT</b>: <code>pk</code> must contain a key pair. The output <code>attributes</code> will contain a key pair type, and the usage policy will allow <b>PSA_KEY_USAGE_ENCRYPT</b> as well as <b>PSA_KEY_USAGE_DECRYPT</b>.</li> <li><b>PSA_KEY_USAGE_DERIVE</b>: <code>pk</code> must contain a key pair. The output <code>attributes</code> will contain a key pair type.</li> <li><b>PSA_KEY_USAGE_ENCRYPT</b>: The output <code>attributes</code> will contain a public key type.</li> <li><b>PSA_KEY_USAGE_SIGN_HASH</b>: <code>pk</code> must contain a key pair. The output <code>attributes</code> will contain a key pair type, and the usage policy will allow <b>PSA_KEY_USAGE_VERIFY_HASH</b> as well as <b>PSA_KEY_USAGE_SIGN_HASH</b>.</li> <li><b>PSA_KEY_USAGE_SIGN_MESSAGE</b>: <code>pk</code> must contain a key pair. The output <code>attributes</code> will contain a key pair type, and the usage policy will allow <b>PSA_KEY_USAGE_VERIFY_MESSAGE</b> as well as <b>PSA_KEY_USAGE_SIGN_MESSAGE</b>.</li> <li><b>PSA_KEY_USAGE_VERIFY_HASH</b>: The output <code>attributes</code> will contain a public key type.</li> <li><b>PSA_KEY_USAGE_VERIFY_MESSAGE</b>: The output <code>attributes</code> will contain a public key type.</li> </ul> |

## Parameters

|     |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out | <i>attributes</i> | <p>On success, valid attributes to import the key into PSA.</p> <ul style="list-style-type: none"> <li>• The lifetime and key identifier are unchanged. If the attribute structure was initialized or reset before calling this function, this will result in a volatile key. Call <code>psa_set_key_identifier()</code> before or after this function if you wish to create a persistent key. Call <code>psa_set_key_lifetime()</code> before or after this function if you wish to import the key in a secure element.</li> <li>• The key type and bit-size are determined by the contents of the PK context. If the PK context contains a key pair, the key type can be either a key pair type or the corresponding public key type, depending on <code>usage</code>. If the PK context contains a public key, the key type is a public key type.</li> <li>• The key's policy is determined by the key type and the <code>usage</code> parameter. The <code>usage</code> always allows <code>usage</code>, exporting and copying the key, and possibly other permissions as documented for the <code>usage</code> parameter. The enrolment algorithm (if available in this build) is left unchanged. For keys created with <code>MBEDTLS_PK_WRAP_PSA()</code>, the primary algorithm is the same as the original PSA key. Otherwise, it is determined as follows: <ul style="list-style-type: none"> <li>– For RSA keys: <code>PSA_ALG_RSA_PKCS1V15_SIGN(PSA_ALG_ANY_HASH)</code> if <code>usage</code> is SIGN/VERIFY, and <code>PSA_ALG_RSA_PKCS1V15_CRYPT</code> if <code>usage</code> is ENCRYPT/DECRYPT.</li> <li>– For ECC keys: <code>MBEDTLS_PK_ALG_ECDSA(PSA_ALG_ANY_HASH)</code> if <code>usage</code> is SIGN/VERIFY, and <code>PSA_ALG_ECDH</code> if <code>usage</code> is DERIVE.</li> </ul> </li> </ul> |
|-----|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

0 on success. `MBEDTLS_ERR_PK_TYPE_MISMATCH` if `pk` does not contain a key compatible with the desired `usage`. Another error code on other failures.

8.16.5.9 `MBEDTLS_PK_IMPORT_INTO_PSA()`

```
int mbedtls_pk_import_into_psa (
 const mbedtls_pk_context * pk,
 const psa_key_attributes_t * attributes,
 mbedtls_svc_key_id_t * key_id)
```

Import a key into the PSA key store.

This function is equivalent to calling `psa_import_key()` with the key material from `pk`.

The typical way to use this function is:

1. Call `mbedtls_pk_get_psa_attributes()` to obtain attributes for the given key.
2. If desired, modify the attributes, for example:
  - To create a persistent key, call `psa_set_key_identifier()` and optionally `psa_set_key_lifetime()`.
  - To import only the public part of a key pair:

```
psa_set_key_type(&attributes,
 PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(
 psa_get_key_type(&attributes)));
```
  - Restrict the key usage if desired.
3. Call `mbedtls_pk_import_into_psa()`.

## Parameters

|     |                   |                                                                                                                                                                                                                                                                                                                              |
|-----|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pk</i>         | The PK context to use. It must have been populated. It can either contain a key pair or just a public key.                                                                                                                                                                                                                   |
| in  | <i>attributes</i> | The attributes to use for the new key. They must be compatible with <i>pk</i> . In particular, the key type must match the content of <i>pk</i> . If <i>pk</i> contains a key pair, the key type in <i>attributes</i> can be either the key pair type or the corresponding public key type (to import only the public part). |
| out | <i>key_id</i>     | On success, the identifier of the newly created key. On error, this is <a href="#">MBEDTLS_SVC_KEY_ID_INIT</a> .                                                                                                                                                                                                             |

## Returns

0 on success. [MBEDTLS\\_ERR\\_PK\\_TYPE\\_MISMATCH](#) if *pk* does not contain a key of the type identified in *attributes*. Another error code on other failures.

## 8.16.5.10 mbedtls\_pk\_init()

```
void mbedtls_pk_init (
 mbedtls_pk_context * ctx)
```

Initialize a [mbedtls\\_pk\\_context](#) (as empty).

After this, you want to populate the context using one of the following functions:

- `\c mbedtls_pk_wrap_psa()`
- `\c mbedtls_pk_copy_from_psa()`
- `\c mbedtls_pk_copy_public_from_psa()`
- `\c mbedtls_pk_parse_key()`
- `\c mbedtls_pk_parse_public_key()`
- `\c mbedtls_pk_parse_keyfile()`
- `\c mbedtls_pk_parse_public_keyfile()`

## Parameters

|            |                                                                 |
|------------|-----------------------------------------------------------------|
| <i>ctx</i> | The context to initialize. This must not be <code>NULL</code> . |
|------------|-----------------------------------------------------------------|

## 8.16.5.11 mbedtls\_pk\_sign()

```
int mbedtls_pk_sign (
 mbedtls_pk_context * ctx,
 mbedtls_md_type_t md_alg,
 const unsigned char * hash,
 size_t hash_len,
 unsigned char * sig,
 size_t sig_size,
 size_t * sig_len)
```

Make signature.

## Note

The signature algorithm used will be the one that would be selected by [mbedtls\\_pk\\_get\\_psa\\_attributes\(\)](#) called with a usage of [PSA\\_KEY\\_USAGE\\_SIGN\\_HASH](#) - see that function's documentation for details. If you want to select a specific signature algorithm, see [mbedtls\\_pk\\_sign\\_ext\(\)](#).

## Parameters

|                 |                                                                                                                                                                                                                     |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>ctx</i>      | The PK context to use. It must have been populated with a private key.                                                                                                                                              |
| <i>md_alg</i>   | Hash algorithm used                                                                                                                                                                                                 |
| <i>hash</i>     | Hash of the message to sign                                                                                                                                                                                         |
| <i>hash_len</i> | Hash length                                                                                                                                                                                                         |
| <i>sig</i>      | Place to write the signature. It must have enough room for the signature.<br><a href="#">MBEDTLS_PK_SIGNATURE_MAX_SIZE</a> is always enough. You may use a smaller buffer if it is large enough given the key type. |
| <i>sig_size</i> | The size of the <i>sig</i> buffer in bytes.                                                                                                                                                                         |
| <i>sig_len</i>  | On successful return, the number of bytes written to <i>sig</i> .                                                                                                                                                   |

## Returns

0 on success, or a specific error code.

8.16.5.12 `mbedtls_pk_sign_ext()`

```
int mbedtls_pk_sign_ext (
 mbedtls_pk_sigalg_t sig_type,
 mbedtls_pk_context * ctx,
 mbedtls_md_type_t md_alg,
 const unsigned char * hash,
 size_t hash_len,
 unsigned char * sig,
 size_t sig_size,
 size_t * sig_len)
```

Generate a signature, selecting a specific algorithm.

## Parameters

|                 |                                                                                                                                                                                                                     |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>sig_type</i> | Signature type to generate.                                                                                                                                                                                         |
| <i>ctx</i>      | The PK context to use. It must have been populated with a private key.                                                                                                                                              |
| <i>md_alg</i>   | Hash algorithm used                                                                                                                                                                                                 |
| <i>hash</i>     | Hash of the message to sign                                                                                                                                                                                         |
| <i>hash_len</i> | Hash length                                                                                                                                                                                                         |
| <i>sig</i>      | Place to write the signature. It must have enough room for the signature.<br><a href="#">MBEDTLS_PK_SIGNATURE_MAX_SIZE</a> is always enough. You may use a smaller buffer if it is large enough given the key type. |
| <i>sig_size</i> | The size of the <i>sig</i> buffer in bytes.                                                                                                                                                                         |
| <i>sig_len</i>  | On successful return, the number of bytes written to <i>sig</i> .                                                                                                                                                   |

## Returns

0 on success, [MBEDTLS\\_ERR\\_PK\\_TYPE\\_MISMATCH](#) if the PK context can't be used for this type of signature, or a specific error code.

8.16.5.13 `mbedtls_pk_sign_restartable()`

```
int mbedtls_pk_sign_restartable (
 mbedtls_pk_context * ctx,
 mbedtls_md_type_t md_alg,
```

```

 const unsigned char * hash,
 size_t hash_len,
 unsigned char * sig,
 size_t sig_size,
 size_t * sig_len,
 mbedtls_pk_restart_ctx * rs_ctx)

```

Restartable version of `mbedtls_pk_sign()`

#### Note

Performs the same job as `mbedtls_pk_sign()`, but can return early and restart according to the limit set with `psa_interruptible_set_max_ops()` to reduce blocking for ECC operations. For RSA, same as `mbedtls_pk_sign()`.

For ECC keys, always uses `MBEDTLS_PK_ALG_ECDSA(hash)`, where hash is the PSA alg identifier corresponding to hash.

This function currently does not work on ECC keys created with `mbedtls_pk_wrap_psa()`.

#### Parameters

|                 |                                                                                                                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>ctx</i>      | The PK context to use. It must have been populated with a private key.                                                                                                                                        |
| <i>md_alg</i>   | Hash algorithm used.                                                                                                                                                                                          |
| <i>hash</i>     | Hash of the message to sign                                                                                                                                                                                   |
| <i>hash_len</i> | Hash length                                                                                                                                                                                                   |
| <i>sig</i>      | Place to write the signature. It must have enough room for the signature. <code>MBEDTLS_PK_SIGNATURE_MAX_SIZE</code> is always enough. You may use a smaller buffer if it is large enough given the key type. |
| <i>sig_size</i> | The size of the <i>sig</i> buffer in bytes.                                                                                                                                                                   |
| <i>sig_len</i>  | On successful return, the number of bytes written to <i>sig</i> .                                                                                                                                             |
| <i>rs_ctx</i>   | Restart context (NULL to disable restart)                                                                                                                                                                     |

#### Returns

See `mbedtls_pk_sign()`.

`PSA_OPERATION_INCOMPLETE` if the maximum number of operations was reached: see `psa_interruptible_set_max_ops()`.

#### 8.16.5.14 mbedtls\_pk\_verify()

```

int mbedtls_pk_verify (
 mbedtls_pk_context * ctx,
 mbedtls_md_type_t md_alg,
 const unsigned char * hash,
 size_t hash_len,
 const unsigned char * sig,
 size_t sig_len)

```

Verify signature.

#### Note

The signature algorithm used will be the one that would be selected by `mbedtls_pk_get_psa_attributes()` called with a usage of `PSA_KEY_USAGE_VERIFY_HASH` - see that function's documentation for details. If you want to select a specific signature algorithm, see `mbedtls_pk_verify_ext()`.

This function currently does not work on RSA keys created with `mbedtls_pk_wrap_psa()`.

## Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>ctx</i>      | The PK context to use. It must have been populated. |
| <i>md_alg</i>   | Hash algorithm used.                                |
| <i>hash</i>     | Hash of the message to sign                         |
| <i>hash_len</i> | Hash length                                         |
| <i>sig</i>      | Signature to verify                                 |
| <i>sig_len</i>  | Signature length                                    |

## Returns

0 on success (signature is valid), [PSA\\_ERROR\\_INVALID\\_SIGNATURE](#) if the signature is invalid, or another specific error code.

8.16.5.15 `MBEDTLS_PK_VERIFY_EXT()`

```
int mbedtls_pk_verify_ext (
 mbedtls_pk_sigalg_t type,
 mbedtls_pk_context * ctx,
 mbedtls_md_type_t md_alg,
 const unsigned char * hash,
 size_t hash_len,
 const unsigned char * sig,
 size_t sig_len)
```

Verify signature, selecting a specific algorithm.

## Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>type</i>     | Signature type to verify                            |
| <i>ctx</i>      | The PK context to use. It must have been populated. |
| <i>md_alg</i>   | Hash algorithm used.                                |
| <i>hash</i>     | Hash of the message to sign                         |
| <i>hash_len</i> | Hash length                                         |
| <i>sig</i>      | Signature to verify                                 |
| <i>sig_len</i>  | Signature length                                    |

## Note

If *type* is [MBEDTLS\\_PK\\_SIGALG\\_RSA\\_PSS](#), then any salt length is accepted: [PSA\\_ALG\\_RSA\\_PSS\\_ANY\\_SALT](#) is used.

## Returns

0 on success (signature is valid), [MBEDTLS\\_ERR\\_PK\\_TYPE\\_MISMATCH](#) if the PK context can't be used for this type of signature, [PSA\\_ERROR\\_INVALID\\_SIGNATURE](#) if the signature is invalid, or a specific error code.

8.16.5.16 `MBEDTLS_PK_VERIFY_RESTARTABLE()`

```
int mbedtls_pk_verify_restartable (
 mbedtls_pk_context * ctx,
 mbedtls_md_type_t md_alg,
 const unsigned char * hash,
```



```

size_t hash_len,
const unsigned char * sig,
size_t sig_len,
mbedtls_pk_restart_ctx * rs_ctx)

```

Restartable version of `mbedtls_pk_verify()`

#### Note

Performs the same job as `mbedtls_pk_verify()`, but can return early and restart according to the limit set with `psa_interruptible_set_max_ops()` to reduce blocking for ECC operations. For RSA, same as `mbedtls_pk_verify()`.

#### Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>ctx</i>      | The PK context to use. It must have been populated. |
| <i>md_alg</i>   | Hash algorithm used                                 |
| <i>hash</i>     | Hash of the message to sign                         |
| <i>hash_len</i> | Hash length                                         |
| <i>sig</i>      | Signature to verify                                 |
| <i>sig_len</i>  | Signature length                                    |
| <i>rs_ctx</i>   | Restart context (NULL to disable restart)           |

#### Returns

See `mbedtls_pk_verify()`, or

`PSA_OPERATION_INCOMPLETE` if maximum number of operations was reached: see `psa_interruptible_set_max_`

#### 8.16.5.17 `mbedtls_pk_wrap_psa()`

```

int mbedtls_pk_wrap_psa (
 mbedtls_pk_context * ctx,
 const mbedtls_svc_key_id_t key)

```

Populate a PK context by wrapping a PSA key pair.

The PSA key must be an EC or RSA key pair (FFDH is not supported in PK).

The resulting context can only perform operations that are allowed by the key's policy. Additionally, it currently has the following limitations:

- restartable operations can't be used;
- for RSA keys, signature verification is not supported.

#### Warning

The PSA wrapped key must remain valid as long as the wrapping PK context is in use, that is at least between the point this function is called and the point `mbedtls_pk_free()` is called on this context.

#### Parameters

|            |                                                              |
|------------|--------------------------------------------------------------|
| <i>ctx</i> | The context to populate. It must be empty.                   |
| <i>key</i> | The PSA key to wrap, which must hold an ECC or RSA key pair. |

## Returns

0 on success.

[PSA\\_ERROR\\_INVALID\\_ARGUMENT](#) on invalid input (context already used, invalid key identifier).

[MBEDTLS\\_ERR\\_PK\\_FEATURE\\_UNAVAILABLE](#) if the key is not an ECC or RSA key pair.

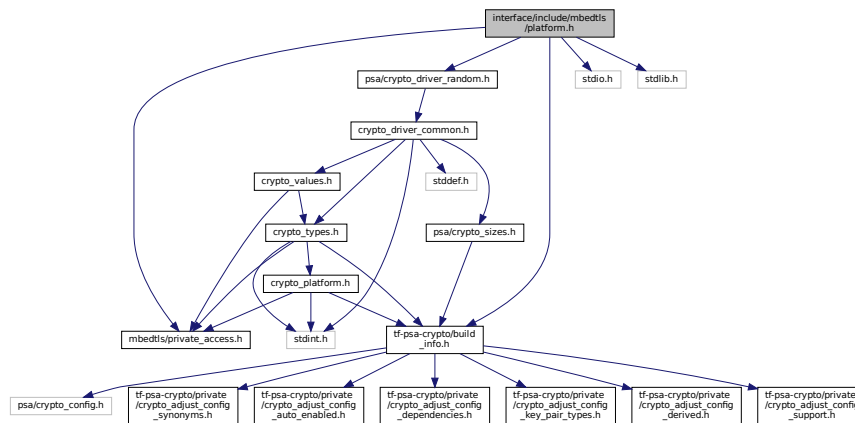
[PSA\\_ERROR\\_INSUFFICIENT\\_MEMORY](#) on allocation failure.

## 8.17 interface/include/mbedtls/platform.h File Reference

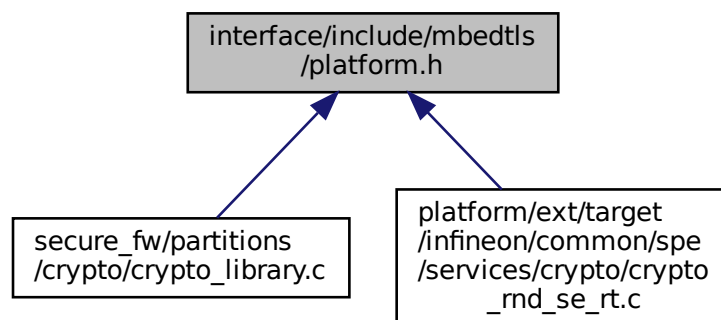
This file contains the definitions and functions of the Mbed TLS platform abstraction layer.

```
#include "mbedtls/private_access.h"
#include "tf-psa-crypto/build_info.h"
#include <psa/crypto_driver_random.h>
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for platform.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [mbedtls\\_platform\\_context](#)

*The platform context structure.*

## Macros

- #define `mbedtls_free` `free`
- #define `mbedtls_calloc` `calloc`
- #define `mbedtls_fprintf` `fprintf`
- #define `mbedtls_printf` `printf`
- #define `mbedtls_snprintf` `MBEDTLS_PLATFORM_STD_SNPRINTF`
- #define `mbedtls_vsnprintf` `vsnprintf`
- #define `mbedtls_setbuf` `setbuf`
- #define `mbedtls_exit` `exit`
- #define `MBEDTLS_EXIT_SUCCESS` `MBEDTLS_PLATFORM_STD_EXIT_SUCCESS`
- #define `MBEDTLS_EXIT_FAILURE` `MBEDTLS_PLATFORM_STD_EXIT_FAILURE`
- #define `MBEDTLS_PLATFORM_DEV_RANDOM` `"/dev/random"`

## SECTION: Module settings

*The configuration options you can set for this module are in this section. Either change them in `mbedtls_config.h` or define them on the compiler command line.*

- #define `MBEDTLS_PLATFORM_STD_SNPRINTF` `snprintf`
- #define `MBEDTLS_PLATFORM_STD_VSNPRINTF` `vsnprintf`
- #define `MBEDTLS_PLATFORM_STD_PRINTF` `printf`
- #define `MBEDTLS_PLATFORM_STD_FPRINTF` `fprintf`
- #define `MBEDTLS_PLATFORM_STD_CALLOC` `calloc`
- #define `MBEDTLS_PLATFORM_STD_FREE` `free`
- #define `MBEDTLS_PLATFORM_STD_SETBUF` `setbuf`
- #define `MBEDTLS_PLATFORM_STD_EXIT` `exit`
- #define `MBEDTLS_PLATFORM_STD_TIME` `time`
- #define `MBEDTLS_PLATFORM_STD_EXIT_SUCCESS` `EXIT_SUCCESS`
- #define `MBEDTLS_PLATFORM_STD_EXIT_FAILURE` `EXIT_FAILURE`

## Typedefs

- typedef struct `mbedtls_platform_context` `mbedtls_platform_context`  
*The platform context structure.*

## Functions

- int `mbedtls_platform_get_entropy` (`psa_driver_get_entropy_flags_t` flags, size\_t \*estimate\_bits, unsigned char \*output, size\_t output\_size)  
*User defined callback function that is used from the entropy module to gather entropy data from some hardware device.*
- int `mbedtls_platform_setup` (`mbedtls_platform_context` \*ctx)  
*This function performs any platform-specific initialization operations.*
- void `mbedtls_platform_teardown` (`mbedtls_platform_context` \*ctx)  
*This function performs any platform teardown operations.*

### 8.17.1 Detailed Description

This file contains the definitions and functions of the Mbed TLS platform abstraction layer.

The platform abstraction layer removes the need for the library to directly link to standard C library functions or operating system services, making the library easier to port and embed. Application developers and users of the library can provide their own implementations of these functions, or implementations specific to their platform, which can be statically linked to the library or dynamically configured at runtime.

When all compilation options related to platform abstraction are disabled, this header just defines `mbedtls_XXX` function names as aliases to the standard `XXX` function.

Most modules in the library and example programs are expected to include this header.

## 8.17.2 Macro Definition Documentation

### 8.17.2.1 mbedtls\_calloc

```
#define mbedtls_calloc calloc
```

Definition at line 152 of file platform.h.

### 8.17.2.2 mbedtls\_exit

```
#define mbedtls_exit exit
```

Definition at line 336 of file platform.h.

### 8.17.2.3 MBEDTLS\_EXIT\_FAILURE

```
#define MBEDTLS_EXIT_FAILURE MBEDTLS_PLATFORM_STD_EXIT_FAILURE
```

Definition at line 349 of file platform.h.

### 8.17.2.4 MBEDTLS\_EXIT\_SUCCESS

```
#define MBEDTLS_EXIT_SUCCESS MBEDTLS_PLATFORM_STD_EXIT_SUCCESS
```

Definition at line 344 of file platform.h.

### 8.17.2.5 mbedtls\_fprintf

```
#define mbedtls_fprintf fprintf
```

Definition at line 179 of file platform.h.

### 8.17.2.6 mbedtls\_free

```
#define mbedtls_free free
```

Definition at line 151 of file platform.h.

### 8.17.2.7 MBEDTLS\_PLATFORM\_DEV\_RANDOM

```
#define MBEDTLS_PLATFORM_DEV_RANDOM "/dev/random"
```

Definition at line 363 of file platform.h.

### 8.17.2.8 MBEDTLS\_PLATFORM\_STD\_CALLOC

```
#define MBEDTLS_PLATFORM_STD_CALLOC calloc
```

The default `calloc` function to use.  
Definition at line 69 of file platform.h.

### 8.17.2.9 MBEDTLS\_PLATFORM\_STD\_EXIT

```
#define MBEDTLS_PLATFORM_STD_EXIT exit
```

The default `exit` function to use.  
Definition at line 78 of file platform.h.

#### 8.17.2.10 MBEDTLS\_PLATFORM\_STD\_EXIT\_FAILURE

```
#define MBEDTLS_PLATFORM_STD_EXIT_FAILURE EXIT_FAILURE
```

The default exit value to use.

Definition at line 87 of file platform.h.

#### 8.17.2.11 MBEDTLS\_PLATFORM\_STD\_EXIT\_SUCCESS

```
#define MBEDTLS_PLATFORM_STD_EXIT_SUCCESS EXIT_SUCCESS
```

The default exit value to use.

Definition at line 84 of file platform.h.

#### 8.17.2.12 MBEDTLS\_PLATFORM\_STD\_FPRINTF

```
#define MBEDTLS_PLATFORM_STD_FPRINTF fprintf
```

The default `fprintf` function to use.

Definition at line 66 of file platform.h.

#### 8.17.2.13 MBEDTLS\_PLATFORM\_STD\_FREE

```
#define MBEDTLS_PLATFORM_STD_FREE free
```

The default `free` function to use.

Definition at line 72 of file platform.h.

#### 8.17.2.14 MBEDTLS\_PLATFORM\_STD\_PRINTF

```
#define MBEDTLS_PLATFORM_STD_PRINTF printf
```

The default `printf` function to use.

Definition at line 63 of file platform.h.

#### 8.17.2.15 MBEDTLS\_PLATFORM\_STD\_SETBUF

```
#define MBEDTLS_PLATFORM_STD_SETBUF setbuf
```

The default `setbuf` function to use.

Definition at line 75 of file platform.h.

#### 8.17.2.16 MBEDTLS\_PLATFORM\_STD\_SNPRINTF

```
#define MBEDTLS_PLATFORM_STD_SNPRINTF snprintf
```

The default `snprintf` function to use.

Definition at line 57 of file platform.h.

#### 8.17.2.17 MBEDTLS\_PLATFORM\_STD\_TIME

```
#define MBEDTLS_PLATFORM_STD_TIME time
```

The default `time` function to use.

Definition at line 81 of file platform.h.

#### 8.17.2.18 MBEDTLS\_PLATFORM\_STD\_VSNPRINTF

```
#define MBEDTLS_PLATFORM_STD_VSNPRINTF vsnprintf
```

The default `vsnprintf` function to use.

Definition at line 60 of file `platform.h`.

#### 8.17.2.19 mbedtls\_printf

```
#define mbedtls_printf printf
```

Definition at line 214 of file `platform.h`.

#### 8.17.2.20 mbedtls\_setbuf

```
#define mbedtls_setbuf setbuf
```

Definition at line 311 of file `platform.h`.

#### 8.17.2.21 mbedtls\_snprintf

```
#define mbedtls_snprintf MBEDTLS_PLATFORM_STD_SNPRINTF
```

Definition at line 236 of file `platform.h`.

#### 8.17.2.22 mbedtls\_vsnprintf

```
#define mbedtls_vsnprintf vsnprintf
```

Definition at line 258 of file `platform.h`.

### 8.17.3 Typedef Documentation

#### 8.17.3.1 mbedtls\_platform\_context

```
typedef struct mbedtls_platform_context mbedtls_platform_context
```

The platform context structure.

##### Note

This structure may be used to assist platform-specific setup or teardown operations.

### 8.17.4 Function Documentation

#### 8.17.4.1 mbedtls\_platform\_get\_entropy()

```
int mbedtls_platform_get_entropy (
 psa_driver_get_entropy_flags_t flags,
 size_t * estimate_bits,
 unsigned char * output,
 size_t output_size)
```

User defined callback function that is used from the entropy module to gather entropy data from some hardware device.

## Parameters

|     |                      |                                                                                                     |
|-----|----------------------|-----------------------------------------------------------------------------------------------------|
|     | <i>flags</i>         | A mask of <code>PSA_DRIVER_GET_ENTROPY_XXX</code> flags. As of TF-PSA-Crypto 1.0, this is always 0. |
| out | <i>estimate_bits</i> | Measure of the entropy content (in bits) of the data written in the <code>output</code> buffer.     |
| out | <i>output</i>        | Output buffer where the entropy data will be stored.                                                |
|     | <i>output_size</i>   | Size of the <code>output</code> buffer in bytes.                                                    |

## Return values

|  |                                                |                                                   |
|--|------------------------------------------------|---------------------------------------------------|
|  | 0                                              | Success.                                          |
|  | <a href="#">PSA_ERROR_INSUFFICIENT_ENTROPY</a> | The entropy source failed.                        |
|  | <a href="#">PSA_ERROR_NOT_SUPPORTED</a>        | The value of <code>flags</code> is not supported. |

## Warning

For the time being TF-PSA-Crypto only supports implementations that return a maximum entropy output on each call, i.e. `estimate_bits = 8 * output_size`. Returning a smaller entropy content is the same as returning [PSA\\_ERROR\\_INSUFFICIENT\\_ENTROPY](#) so the hardware polling will fail. In the future TF-PSA-Crypto will be smarter and capable to cope with entropy sources with lower entropy content (i.e.  $0 < \text{estimate\_bits} < 8 * \text{output\_size}$ ) by calling the callback function in loop.

## Note

This function is not meant to be called by application code, and it is not guaranteed that this function will exist or will behave in the same way in future versions of the library. Applications should call [psa\\_generate\\_random\(\)](#) to obtain random data.

## 8.17.4.2 mbedtls\_platform\_setup()

```
int mbedtls_platform_setup (
 mbedtls_platform_context * ctx)
```

This function performs any platform-specific initialization operations.

## Note

This function should be called before any other library functions.

Its implementation is platform-specific, and unless platform-specific code is provided, it does nothing.

The usage and necessity of this function is dependent on the platform.

## Parameters

|            |                       |
|------------|-----------------------|
| <i>ctx</i> | The platform context. |
|------------|-----------------------|

## Returns

0 on success.

## 8.17.4.3 mbedtls\_platform\_teardown()

```
void mbedtls_platform_teardown (
```

```
mbedtls_platform_context * ctx)
```

This function performs any platform teardown operations.

#### Note

This function should be called after every other Mbed TLS module has been correctly freed using the appropriate free function.

Its implementation is platform-specific, and unless platform-specific code is provided, it does nothing.

#### Note

The usage and necessity of this function is dependent on the platform.

#### Parameters

|                  |                       |
|------------------|-----------------------|
| <code>ctx</code> | The platform context. |
|------------------|-----------------------|

## 8.18 interface/include/mbedtls/platform\_time.h File Reference

Mbed TLS Platform time abstraction.

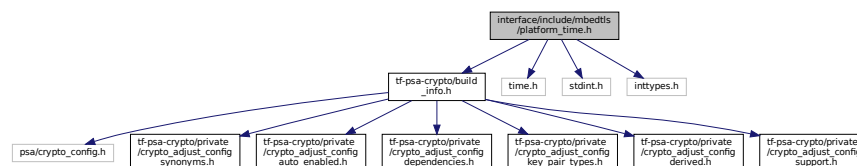
```
#include "tf-psa-crypto/build_info.h"
```

```
#include <time.h>
```

```
#include <stdint.h>
```

```
#include <inttypes.h>
```

Include dependency graph for platform\_time.h:



## Macros

- `#define` `mbedtls_time` `time`

## Typedefs

- `typedef` `time_t` `mbedtls_time_t`
- `typedef` `int64_t` `mbedtls_ms_time_t`

## Functions

- `mbedtls_ms_time_t` `mbedtls_ms_time` (`void`)

*Get time in milliseconds.*

### 8.18.1 Detailed Description

Mbed TLS Platform time abstraction.

### 8.18.2 Macro Definition Documentation



### 8.18.2.1 mbedtls\_time

```
#define mbedtls_time time
```

Definition at line 71 of file platform\_time.h.

## 8.18.3 Typedef Documentation

### 8.18.3.1 mbedtls\_ms\_time\_t

```
typedef int64_t mbedtls_ms_time_t
```

Definition at line 35 of file platform\_time.h.

### 8.18.3.2 mbedtls\_time\_t

```
typedef time_t mbedtls_time_t
```

Definition at line 27 of file platform\_time.h.

## 8.18.4 Function Documentation

### 8.18.4.1 mbedtls\_ms\_time()

```
mbedtls_ms_time_t mbedtls_ms_time (
 void)
```

Get time in milliseconds.

#### Returns

Monotonically-increasing current time in milliseconds.

#### Note

Define MBEDTLS\_PLATFORM\_MS\_TIME\_ALT to be able to provide an alternative implementation

#### Warning

This function returns a monotonically-increasing time value from a start time that will differ from platform to platform, and possibly from run to run of the process.

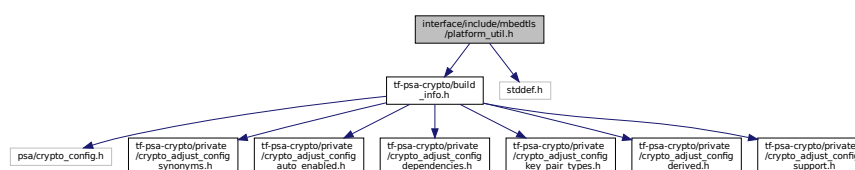
## 8.19 interface/include/mbedtls/platform\_util.h File Reference

Common and shared functions used by multiple modules in the Mbed TLS library.

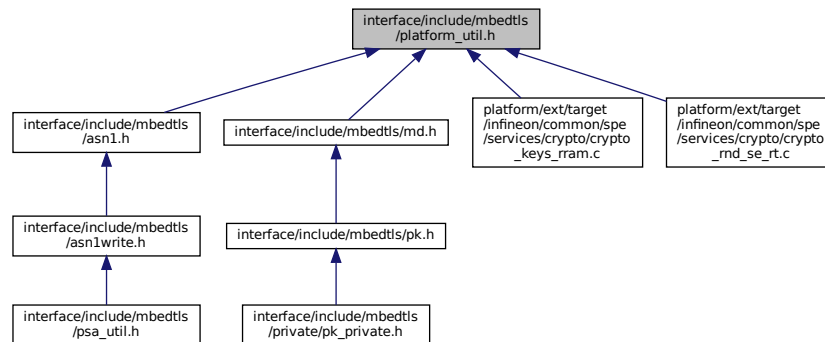
```
#include "tf-psa-crypto/build_info.h"
```

```
#include <stddef.h>
```

Include dependency graph for platform\_util.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define MBEDTLS_DEPRECATED`
- `#define MBEDTLS_DEPRECATED_STRING_CONSTANT(VAL) VAL`
- `#define MBEDTLS_DEPRECATED_NUMERIC_CONSTANT(VAL) VAL`
- `#define MBEDTLS_CHECK_RETURN`
- `#define MBEDTLS_CHECK_RETURN_CRITICAL MBEDTLS_CHECK_RETURN`
- `#define MBEDTLS_CHECK_RETURN_TYPICAL`
- `#define MBEDTLS_CHECK_RETURN_OPTIONAL`
- `#define MBEDTLS_IGNORE_RETURN(result) ((void) !(result))`

## Functions

- `void mbedtls_platform_zeroize(void *buf, size_t len)`  
Securely zeroize a buffer.

### 8.19.1 Detailed Description

Common and shared functions used by multiple modules in the Mbed TLS library.

### 8.19.2 Macro Definition Documentation

#### 8.19.2.1 MBEDTLS\_CHECK\_RETURN

```
#define MBEDTLS_CHECK_RETURN
```

Definition at line 57 of file platform\_util.h.

#### 8.19.2.2 MBEDTLS\_CHECK\_RETURN\_CRITICAL

```
#define MBEDTLS_CHECK_RETURN_CRITICAL MBEDTLS_CHECK_RETURN
```

Critical-failure function

This macro appearing at the beginning of the declaration of a function indicates that its return value should be checked in all applications. Omitting the check is very likely to indicate a bug in the application and will result in a compile-time warning if `MBEDTLS_CHECK_RETURN` is implemented for the compiler in use.

**Note**

The use of this macro is a work in progress. This macro may be added to more functions in the future. Such an extension is not considered an API break, provided that there are near-unavoidable circumstances under which the function can fail. For example, signature/MAC/AEAD verification functions, and functions that require a random generator, are considered return-check-critical.

Definition at line 77 of file platform\_util.h.

**8.19.2.3 MBEDTLS\_CHECK\_RETURN\_OPTIONAL**

```
#define MBEDTLS_CHECK_RETURN_OPTIONAL
```

Benign-failure function

This macro appearing at the beginning of the declaration of a function indicates that it is rarely useful to check its return value.

This macro has an empty expansion. It exists for documentation purposes: a [MBEDTLS\\_CHECK\\_RETURN\\_OPTIONAL](#) annotation indicates that the function has been analyzed for return-check usefulness, whereas the lack of an annotation indicates that the function has not been analyzed and its return-check usefulness is unknown.

Definition at line 113 of file platform\_util.h.

**8.19.2.4 MBEDTLS\_CHECK\_RETURN\_TYPICAL**

```
#define MBEDTLS_CHECK_RETURN_TYPICAL
```

Ordinary-failure function

This macro appearing at the beginning of the declaration of a function indicates that its return value should be generally be checked in portable applications. Omitting the check will result in a compile-time warning if [MBEDTLS\\_CHECK\\_RETURN](#) is implemented for the compiler in use and `#MBEDTLS_CHECK_RETURN_WARNING` is enabled in the compile-time configuration.

You can use [MBEDTLS\\_IGNORE\\_RETURN](#) to explicitly ignore the return value of a function that is annotated with [MBEDTLS\\_CHECK\\_RETURN](#).

**Note**

The use of this macro is a work in progress. This macro will be added to more functions in the future. Eventually this should appear before most functions returning an error code (as `int` in the `mbedtls_xxx` API or as `psa_status_t` in the `psa_xxx` API).

Definition at line 99 of file platform\_util.h.

**8.19.2.5 MBEDTLS\_DEPRECATED**

```
#define MBEDTLS_DEPRECATED
```

Definition at line 37 of file platform\_util.h.

**8.19.2.6 MBEDTLS\_DEPRECATED\_NUMERIC\_CONSTANT**

```
#define MBEDTLS_DEPRECATED_NUMERIC_CONSTANT(
 VAL) VAL
```

Definition at line 39 of file platform\_util.h.

**8.19.2.7 MBEDTLS\_DEPRECATED\_STRING\_CONSTANT**

```
#define MBEDTLS_DEPRECATED_STRING_CONSTANT(
 VAL) VAL
```

Definition at line 38 of file platform\_util.h.

### 8.19.2.8 MBEDTLS\_IGNORE\_RETURN

```
#define MBEDTLS_IGNORE_RETURN(
 result) ((void) !(result))
```

Call this macro with one argument, a function call, to suppress a warning from [MBEDTLS\\_CHECK\\_RETURN](#) due to that function call.

Definition at line 129 of file platform\_util.h.

## 8.19.3 Function Documentation

### 8.19.3.1 mbedtls\_platform\_zeroize()

```
void mbedtls_platform_zeroize (
 void * buf,
 size_t len)
```

Securely zeroize a buffer.

The function is meant to wipe the data contained in a buffer so that it can no longer be recovered even if the program memory is later compromised. Call this function on sensitive data stored on the stack before returning from a function, and on sensitive data stored on the heap before freeing the heap object.

It is extremely difficult to guarantee that calls to `mbedtls_platform_zeroize()` are not removed by aggressive compiler optimizations in a portable way. For this reason, Mbed TLS provides the configuration option `MBEDTLS_PLATFORM_ZEROIZE_ALT`, which allows users to configure `mbedtls_platform_zeroize()` to use a suitable implementation for their platform and needs

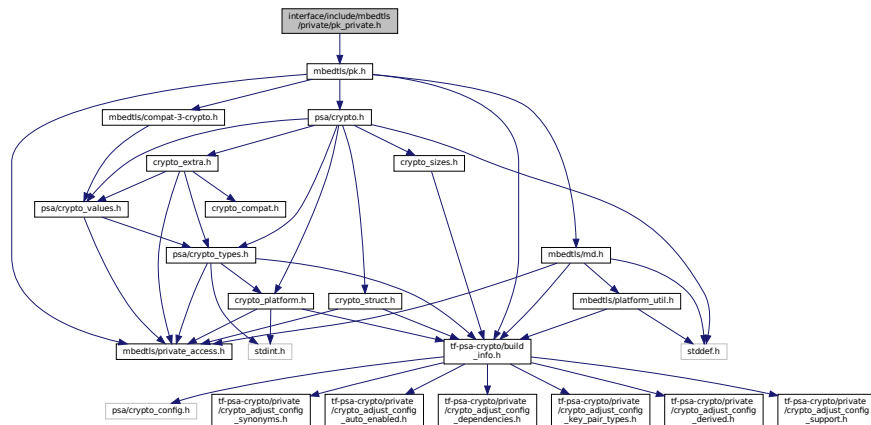
#### Parameters

|            |                               |
|------------|-------------------------------|
| <i>buf</i> | Buffer to be zeroized         |
| <i>len</i> | Length of the buffer in bytes |

## 8.20 interface/include/mbedtls/private/pk\_private.h File Reference

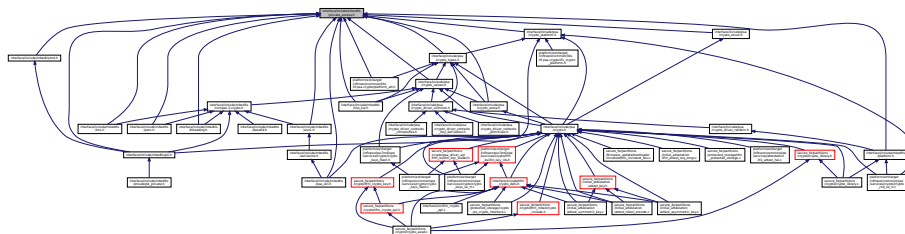
Private Public Key abstraction layer.

Include dependency graph for pk\_private.h:



Private Public Key abstraction layer.

Optionally activate declarations of private identifiers in public headers.  
This graph shows which files directly or indirectly include this file:



- `#define MBEDTLS_PRIVATE(member) private ##member`

Optionally activate declarations of private identifiers in public headers.  
This header is reserved for internal use in TF-PSA-Crypto and Mbed TLS.

```
#define MBEDTLS_PRIVATE(
 member) private_##member
```

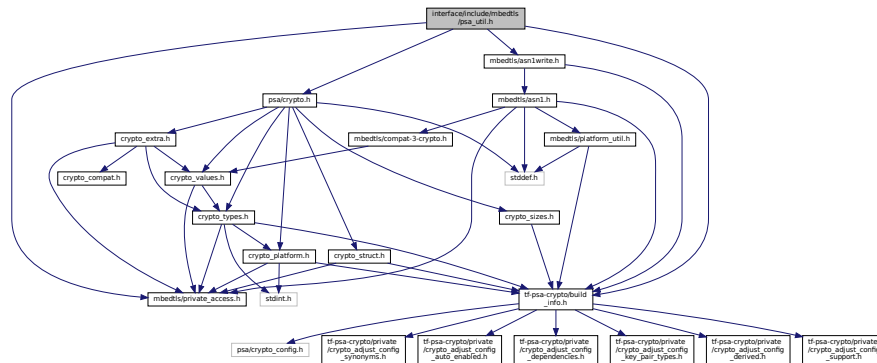
Definition at line 32 of file private access.h.

## 8.22 interface/include/mbedtls/psa\_util.h File Reference

## Utility functions for the use of the PSA Crypto library.

```
#include "mbedtls/private_access.h"
#include "tf-psa-crypto/build_info.h"
#include "psa/crypto.h"
#include <mbedtls/asn1write.h>
```

Include dependency graph for psa\_util.h:



### 8.22.1 Detailed Description

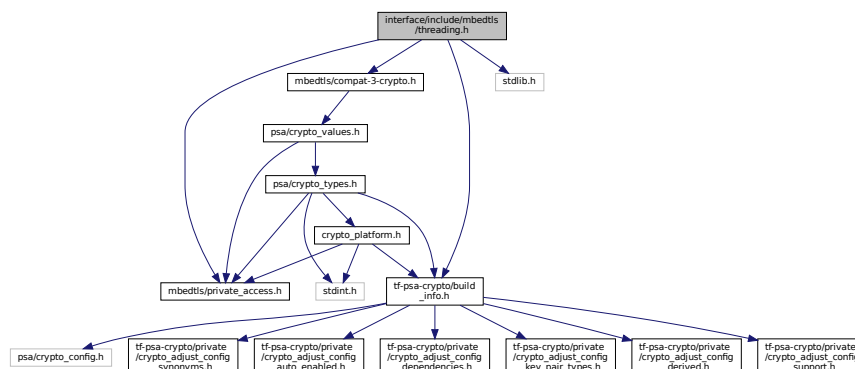
# Utility functions for the use of the PSA Crypto library.

## 8.23 interface/include/mbedtls/threading.h File Reference

Threading abstraction layer.

```
#include "mbedtls/private_access.h"
#include "tf-psa-crypto/build_info.h"
#include "mbedtls/compat-3-crypto.h"
#include <stdlib.h>
```

Include dependency graph for `threading.h`:



## Macros

- `#define MBEDTLS_ERR_THREADING_USAGE_ERROR -0x001E`
- `#define MBEDTLS_ERR_THREADING_MUTEX_ERROR MBEDTLS_ERR_THREADING_USAGE_ERROR`

### 8.23.1 Detailed Description

Threading abstraction layer.

### 8.23.2 Macro Definition Documentation

#### 8.23.2.1 MBEDTLS\_ERR\_THREADING\_MUTEX\_ERROR

```
#define MBEDTLS_ERR_THREADING_MUTEX_ERROR MBEDTLS_ERR_THREADING_USAGE_ERROR
```

A historical alias for [MBEDTLS\\_ERR\\_THREADING\\_USAGE\\_ERROR](#).

Definition at line 33 of file `threading.h`.

#### 8.23.2.2 MBEDTLS\_ERR\_THREADING\_USAGE\_ERROR

```
#define MBEDTLS_ERR_THREADING_USAGE_ERROR -0x001E
```

Detected error in mutex or condition variable usage.

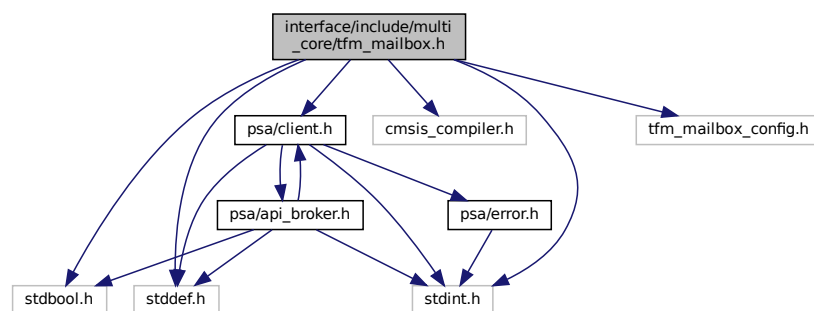
Note that depending on the platform, many usage errors of synchronization primitives have undefined behavior. But where it is practical to detect usage errors at runtime, mutex and condition primitives can return this error code.

Definition at line 30 of file `threading.h`.

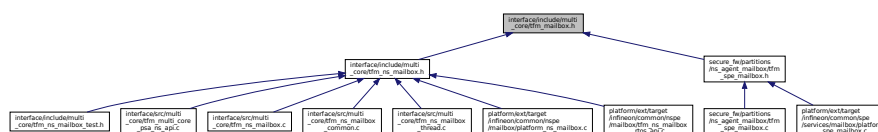
## 8.24 interface/include/multi\_core/tfm\_mailbox.h File Reference

```
#include <stdbool.h>
#include <stdint.h>
#include <stddef.h>
#include "cmsis_compiler.h"
#include "psa/client.h"
#include "tfm_mailbox_config.h"
```

Include dependency graph for `tfm_mailbox.h`:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [psa\\_client\\_params\\_t](#)
- struct [mailbox\\_msg\\_t](#)
- struct [mailbox\\_reply\\_t](#)
- struct [mailbox\\_slot\\_t](#)
- struct [mailbox\\_status\\_t](#)
- struct [mailbox\\_init\\_t](#)

## Macros

- `#define MAILBOX_CACHE_LINE_SIZE 32`
- `#define MAILBOX_ALIGN __ALIGNED(MAILBOX_CACHE_LINE_SIZE)`
- `#define MAILBOX_PSA_FRAMEWORK_VERSION (0x1)`
- `#define MAILBOX_PSA_VERSION (0x2)`
- `#define MAILBOX_PSA_CONNECT (0x3)`
- `#define MAILBOX_PSA_CALL (0x4)`
- `#define MAILBOX_PSA_CLOSE (0x5)`
- `#define MAILBOX_SUCCESS (0)`
- `#define MAILBOX_QUEUE_FULL (INT32_MIN + 1)`
- `#define MAILBOX_INVALID_PARAMS (INT32_MIN + 2)`
- `#define MAILBOX_NO_PERMS (INT32_MIN + 3)`
- `#define MAILBOX_NO_PENDING_EVENT (INT32_MIN + 4)`
- `#define MAILBOX_CHANNEL_BUSY (INT32_MIN + 5)`
- `#define MAILBOX_CALLBACK_REG_ERROR (INT32_MIN + 6)`
- `#define MAILBOX_INIT_ERROR (INT32_MIN + 7)`
- `#define MAILBOX_GENERIC_ERROR (INT32_MIN + 8)`

## Typedefs

- typedef uint32\_t [mailbox\\_queue\\_status\\_t](#)

## Functions

- struct [mailbox\\_slot\\_t](#) `__ALIGNED (32)`

## Variables

- [mailbox\\_queue\\_status\\_t](#) `pend_slots`
- [mailbox\\_queue\\_status\\_t](#) `replied_slots`
- struct [mailbox\\_init\\_t](#) `__ALIGNED`

### 8.24.1 Macro Definition Documentation

#### 8.24.1.1 MAILBOX\_ALIGN

```
#define MAILBOX_ALIGN __ALIGNED(MAILBOX_CACHE_LINE_SIZE)
```

Definition at line 45 of file `tfm_mailbox.h`.

#### 8.24.1.2 MAILBOX\_CACHE\_LINE\_SIZE

```
#define MAILBOX_CACHE_LINE_SIZE 32
```

Definition at line 43 of file `tfm_mailbox.h`.



#### 8.24.1.3 MAILBOX\_CALLBACK\_REG\_ERROR

```
#define MAILBOX_CALLBACK_REG_ERROR (INT32_MIN + 6)
```

Definition at line 63 of file tfm\_mailbox.h.

#### 8.24.1.4 MAILBOX\_CHAN\_BUSY

```
#define MAILBOX_CHAN_BUSY (INT32_MIN + 5)
```

Definition at line 62 of file tfm\_mailbox.h.

#### 8.24.1.5 MAILBOX\_GENERIC\_ERROR

```
#define MAILBOX_GENERIC_ERROR (INT32_MIN + 8)
```

Definition at line 65 of file tfm\_mailbox.h.

#### 8.24.1.6 MAILBOX\_INIT\_ERROR

```
#define MAILBOX_INIT_ERROR (INT32_MIN + 7)
```

Definition at line 64 of file tfm\_mailbox.h.

#### 8.24.1.7 MAILBOX\_INVALID\_PARAMS

```
#define MAILBOX_INVALID_PARAMS (INT32_MIN + 2)
```

Definition at line 59 of file tfm\_mailbox.h.

#### 8.24.1.8 MAILBOX\_NO\_PEND\_EVENT

```
#define MAILBOX_NO_PEND_EVENT (INT32_MIN + 4)
```

Definition at line 61 of file tfm\_mailbox.h.

#### 8.24.1.9 MAILBOX\_NO\_PERMS

```
#define MAILBOX_NO_PERMS (INT32_MIN + 3)
```

Definition at line 60 of file tfm\_mailbox.h.

#### 8.24.1.10 MAILBOX\_PSA\_CALL

```
#define MAILBOX_PSA_CALL (0x4)
```

Definition at line 53 of file tfm\_mailbox.h.

#### 8.24.1.11 MAILBOX\_PSA\_CLOSE

```
#define MAILBOX_PSA_CLOSE (0x5)
```

Definition at line 54 of file tfm\_mailbox.h.

#### 8.24.1.12 MAILBOX\_PSA\_CONNECT

```
#define MAILBOX_PSA_CONNECT (0x3)
```

Definition at line 52 of file tfm\_mailbox.h.

#### 8.24.1.13 MAILBOX\_PSA\_FRAMEWORK\_VERSION

```
#define MAILBOX_PSA_FRAMEWORK_VERSION (0x1)
```

Definition at line 50 of file tfm\_mailbox.h.

#### 8.24.1.14 MAILBOX\_PSA\_VERSION

```
#define MAILBOX_PSA_VERSION (0x2)
```

Definition at line 51 of file tfm\_mailbox.h.

#### 8.24.1.15 MAILBOX\_QUEUE\_FULL

```
#define MAILBOX_QUEUE_FULL (INT32_MIN + 1)
```

Definition at line 58 of file tfm\_mailbox.h.

#### 8.24.1.16 MAILBOX\_SUCCESS

```
#define MAILBOX_SUCCESS (0)
```

Definition at line 57 of file tfm\_mailbox.h.

### 8.24.2 Typedef Documentation

#### 8.24.2.1 mailbox\_queue\_status\_t

```
typedef uint32_t mailbox_queue_status_t
```

Definition at line 133 of file tfm\_mailbox.h.

### 8.24.3 Function Documentation

#### 8.24.3.1 \_\_ALIGNED()

```
struct mailbox_slot_t __ALIGNED (
 32)
```

### 8.24.4 Variable Documentation

#### 8.24.4.1 \_\_ALIGNED

```
struct mailbox_status_t __ALIGNED
```

#### 8.24.4.2 pend\_slots

```
mailbox_queue_status_t pend_slots
```

Definition at line 134 of file tfm\_mailbox.h.

#### 8.24.4.3 replied\_slots

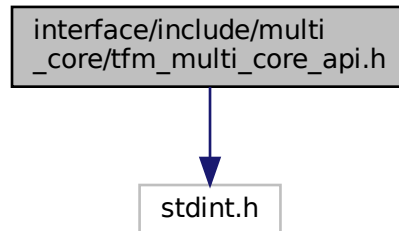
```
mailbox_queue_status_t replied_slots
```

Definition at line 137 of file tfm\_mailbox.h.

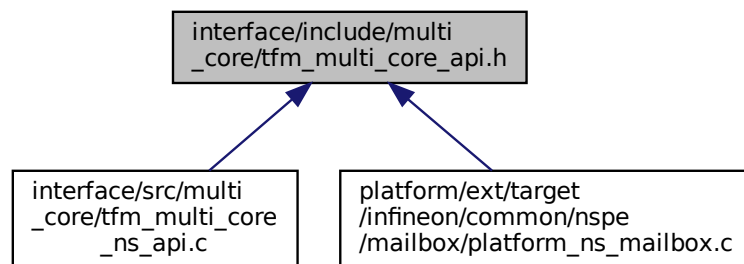
## 8.25 interface/include/multi\_core/tfm\_multi\_core\_api.h File Reference

```
#include <stdint.h>
```

Include dependency graph for tfm\_multi\_core\_api.h:



This graph shows which files directly or indirectly include this file:



### Functions

- `int32_t tfm_ns_wait_for_s_cpu_ready (void)`  
*Called on the non-secure CPU. Flags that the non-secure side has completed its initialization. Waits, if necessary, for the secure CPU to flag that it has completed its initialization.*
- `int32_t tfm_platform_ns_wait_for_s_cpu_ready (void)`  
*Synchronisation with secure CPU, platform-specific implementation. Flags that the non-secure side has completed its initialization. Waits, if necessary, for the secure CPU to flag that it has completed its initialization.*

### 8.25.1 Function Documentation

#### 8.25.1.1 tfm\_ns\_wait\_for\_s\_cpu\_ready()

```
int32_t tfm_ns_wait_for_s_cpu_ready (
 void)
```

Called on the non-secure CPU. Flags that the non-secure side has completed its initialization. Waits, if necessary, for the secure CPU to flag that it has completed its initialization.

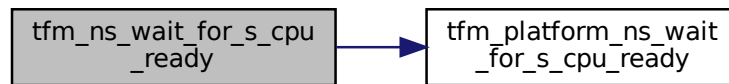
**Returns**

Return 0 if succeeds.

Otherwise, return specific error code.

Definition at line 10 of file `tfm_multi_core_ns_api.c`.

Here is the call graph for this function:

**8.25.1.2 tfm\_platform\_ns\_wait\_for\_s\_cpu\_ready()**

```
int32_t tfm_platform_ns_wait_for_s_cpu_ready (
 void)
```

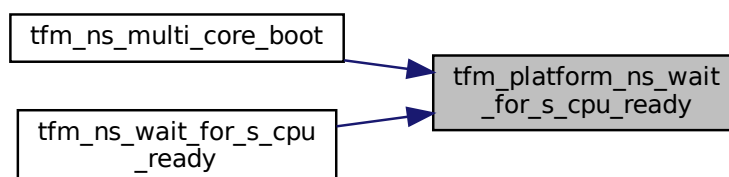
Synchronisation with secure CPU, platform-specific implementation. Flags that the non-secure side has completed its initialization. Waits, if necessary, for the secure CPU to flag that it has completed its initialization.

**Return values**

|                          |                      |
|--------------------------|----------------------|
| <i>Return</i>            | 0 if succeeds.       |
| <i>Otherwise, return</i> | specific error code. |

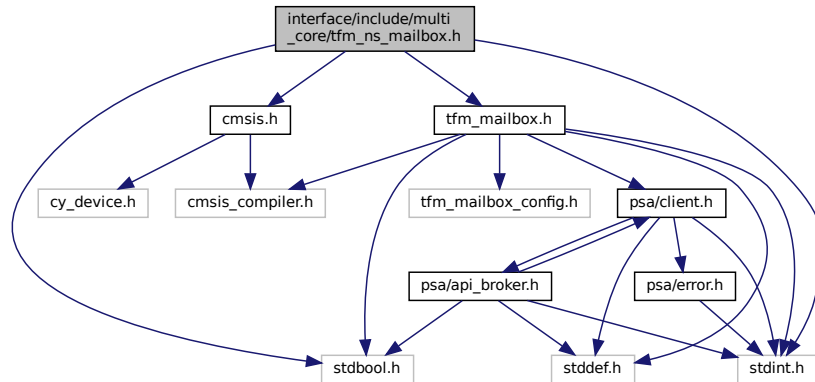
Definition at line 82 of file `platform_ns_mailbox.c`.

Here is the caller graph for this function:

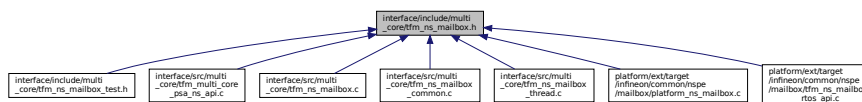
**8.26 interface/include/multi\_core/tfm\_ns\_mailbox.h File Reference**

```
#include <stdbool.h>
#include <stdint.h>
#include "cmsis.h"
#include "tfm_mailbox.h"
```

Include dependency graph for tfm\_ns\_mailbox.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ns\\_mailbox\\_slot\\_t](#)
- struct [ns\\_mailbox\\_queue\\_t](#)

## Macros

- #define [MAILBOX\\_CLEAN\\_CACHE\(addr, size\) \\_\\_DSB\(\)](#)
- #define [MAILBOX\\_INVALIDATE\\_CACHE\(addr, size\) do {} while \(0\)](#)
- #define [MAILBOX\\_DISABLE\\_CACHE\(\) do {} while \(0\)](#)
- #define [MAILBOX\\_ENABLE\\_CACHE\(\) do {} while \(0\)](#)
- #define [DCACHE\\_IS\\_ENABLED](#) false
- #define [tfm\\_ns\\_mailbox\\_thread\\_runner\(args\) do {} while \(0\)](#)
- #define [tfm\\_ns\\_mailbox\\_os\\_wait\\_reply\(\) do {} while \(0\)](#)
- #define [tfm\\_ns\\_mailbox\\_os\\_wake\\_task\\_isr\(task\) do {} while \(0\)](#)
- #define [tfm\\_ns\\_mailbox\\_os\\_spin\\_lock\(\) do {} while \(0\)](#)
- #define [tfm\\_ns\\_mailbox\\_os\\_spin\\_unlock\(\) do {} while \(0\)](#)

## Functions

- [int32\\_t tfm\\_ns\\_multi\\_core\\_boot](#) (struct [ns\\_mailbox\\_queue\\_t](#) \*queue)  
*Performs multicore boot sequence in NSPE.*
- [int32\\_t tfm\\_ns\\_mailbox\\_init](#) (struct [ns\\_mailbox\\_queue\\_t](#) \*queue)  
*NSPE mailbox initialization.*
- [int32\\_t tfm\\_ns\\_mailbox\\_client\\_call](#) (uint32\_t call\_type, const struct [psa\\_client\\_params\\_t](#) \*params, int32\_t client\_id, struct [mailbox\\_reply\\_t](#) \*reply)  
*Send PSA client call to SPE via mailbox. Wait and fetch PSA client call result.*
- [int32\\_t tfm\\_ns\\_mailbox\\_hal\\_init](#) (struct [ns\\_mailbox\\_queue\\_t](#) \*queue)

- Platform specific NSPE mailbox initialization. Invoked by `tfm_ns_mailbox_init()`.
- `int32_t tfm_ns_mailbox_hal_notify_peer (void)`  
Notify SPE to deal with the PSA client call sent via mailbox.
- `uint32_t tfm_ns_mailbox_hal_enter_critical (void)`  
Enter critical section of NSPE mailbox.
- `void tfm_ns_mailbox_hal_exit_critical (uint32_t state)`  
Exit critical section of NSPE mailbox.
- `uint32_t tfm_ns_mailbox_hal_enter_critical_isr (void)`  
Enter critical section of NSPE mailbox in IRQ handler.
- `void tfm_ns_mailbox_hal_exit_critical_isr (uint32_t state)`  
Enter critical section of NSPE mailbox in IRQ handler.
- `void mailbox_set_queue_ptr (struct ns_mailbox_queue_t *queue)`  
Set the NS mailbox queue in the respective mailbox implementation.

## 8.26.1 Macro Definition Documentation

### 8.26.1.1 DCACHE\_IS\_ENABLED

```
#define DCACHE_IS_ENABLED false
```

Definition at line 35 of file `tfm_ns_mailbox.h`.

### 8.26.1.2 MAILBOX\_CLEAN\_CACHE

```
#define MAILBOX_CLEAN_CACHE(
 addr,
 size) __DSB()
```

Definition at line 31 of file `tfm_ns_mailbox.h`.

### 8.26.1.3 MAILBOX\_DISABLE\_CACHE

```
#define MAILBOX_DISABLE_CACHE() do {} while (0)
```

Definition at line 33 of file `tfm_ns_mailbox.h`.

### 8.26.1.4 MAILBOX\_ENABLE\_CACHE

```
#define MAILBOX_ENABLE_CACHE() do {} while (0)
```

Definition at line 34 of file `tfm_ns_mailbox.h`.

### 8.26.1.5 MAILBOX\_INVALIDATE\_CACHE

```
#define MAILBOX_INVALIDATE_CACHE(
 addr,
 size) do {} while (0)
```

Definition at line 32 of file `tfm_ns_mailbox.h`.

### 8.26.1.6 tfm\_ns\_mailbox\_os\_spin\_lock

```
#define tfm_ns_mailbox_os_spin_lock() do {} while (0)
```

Definition at line 380 of file `tfm_ns_mailbox.h`.

### 8.26.1.7 tfm\_ns\_mailbox\_os\_spin\_unlock

```
#define tfm_ns_mailbox_os_spin_unlock() do {} while (0)
```

Definition at line 382 of file tfm\_ns\_mailbox.h.

### 8.26.1.8 tfm\_ns\_mailbox\_os\_wait\_reply

```
#define tfm_ns_mailbox_os_wait_reply() do {} while (0)
```

Definition at line 347 of file tfm\_ns\_mailbox.h.

### 8.26.1.9 tfm\_ns\_mailbox\_os\_wake\_task\_isr

```
#define tfm_ns_mailbox_os_wake_task_isr(
 task) do {} while (0)
```

Definition at line 349 of file tfm\_ns\_mailbox.h.

### 8.26.1.10 tfm\_ns\_mailbox\_thread\_runner

```
#define tfm_ns_mailbox_thread_runner(
 args) do {} while (0)
```

Definition at line 145 of file tfm\_ns\_mailbox.h.

## 8.26.2 Function Documentation

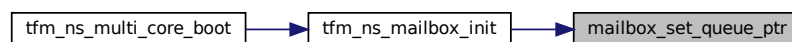
### 8.26.2.1 mailbox\_set\_queue\_ptr()

```
void mailbox_set_queue_ptr (
 struct ns_mailbox_queue_t * queue)
```

Set the NS mailbox queue in the respective mailbox implementation.

Definition at line 331 of file tfm\_ns\_mailbox.c.

Here is the caller graph for this function:



### 8.26.2.2 tfm\_ns\_mailbox\_client\_call()

```
int32_t tfm_ns_mailbox_client_call (
 uint32_t call_type,
 const struct psa_client_params_t * params,
 int32_t client_id,
 struct mailbox_reply_t * reply)
```

Send PSA client call to SPE via mailbox. Wait and fetch PSA client call result.

#### Parameters

|    |                  |                                     |
|----|------------------|-------------------------------------|
| in | <i>call_type</i> | PSA client call type                |
| in | <i>params</i>    | Parameters used for PSA client call |

## Parameters

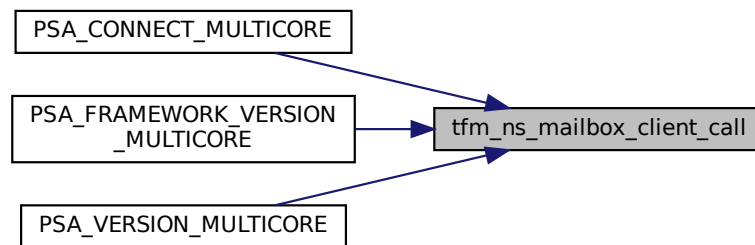
|     |                  |                                                                                                                                            |
|-----|------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>client_id</i> | Optional client ID of non-secure caller. It is required to identify the non-secure caller when NSPE OS enforces non-secure task isolation. |
| out | <i>reply</i>     | The buffer written with PSA client call result.                                                                                            |

## Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | The PSA client call is completed successfully.   |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 186 of file `tfm_ns_mailbox.c`.

Here is the caller graph for this function:



### 8.26.2.3 tfm\_ns\_mailbox\_hal\_enter\_critical()

```
uint32_t tfm_ns_mailbox_hal_enter_critical (
 void)
```

Enter critical section of NSPE mailbox.

Is used to lock data shared between cores running secure and non-secure side. Should not be used to lock non-secure data that are not shared with secure core.

#### Note

The implementation depends on platform specific hardware and use case.

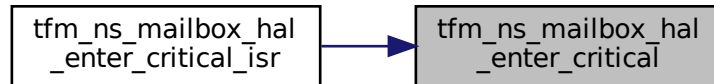


**Returns**

Platform specific critical section state. Use it to exit from critical section by passing result to [tfm\\_ns\\_mailbox\\_hal\\_exit\\_critical](#).

Definition at line 187 of file platform\_ns\_mailbox.c.

Here is the caller graph for this function:

**8.26.2.4 tfm\_ns\_mailbox\_hal\_enter\_critical\_isr()**

```
uint32_t tfm_ns_mailbox_hal_enter_critical_isr (
 void)
```

Enter critical section of NSPE mailbox in IRQ handler.

Is used to lock data shared between cores running secure and non-secure side. Should not be used to lock non-secure data that are not shared with secure core.

**Note**

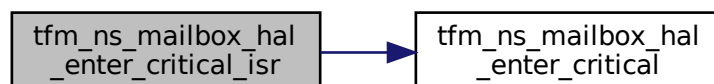
The implementation depends on platform specific hardware and use case.

**Returns**

Platform specific critical section state. Use it to exit from critical section by passing result to [tfm\\_ns\\_mailbox\\_hal\\_exit\\_critical\\_isr](#).

Definition at line 216 of file platform\_ns\_mailbox.c.

Here is the call graph for this function:

**8.26.2.5 tfm\_ns\_mailbox\_hal\_exit\_critical()**

```
void tfm_ns_mailbox_hal_exit_critical (
 uint32_t state)
```

Exit critical section of NSPE mailbox.

**Note**

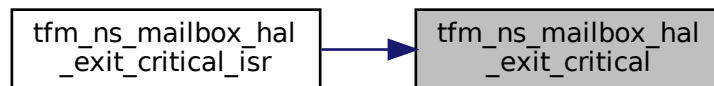
The implementation depends on platform specific hardware and use case.

## Parameters

|                 |                    |                                                                                        |
|-----------------|--------------------|----------------------------------------------------------------------------------------|
| <code>in</code> | <code>state</code> | Critical section state returned by <a href="#">tfm_ns_mailbox_hal_enter_critical</a> . |
|-----------------|--------------------|----------------------------------------------------------------------------------------|

Definition at line 206 of file `platform_ns_mailbox.c`.

Here is the caller graph for this function:



### 8.26.2.6 tfm\_ns\_mailbox\_hal\_exit\_critical\_isr()

```
void tfm_ns_mailbox_hal_exit_critical_isr (
 uint32_t state)
```

Enter critical section of NSPE mailbox in IRQ handler.

## Note

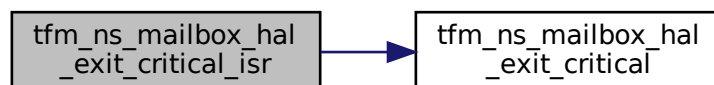
The implementation depends on platform specific hardware and use case.

## Parameters

|                 |                    |                                                                                            |
|-----------------|--------------------|--------------------------------------------------------------------------------------------|
| <code>in</code> | <code>state</code> | Critical section state returned by <a href="#">tfm_ns_mailbox_hal_enter_critical_isr</a> . |
|-----------------|--------------------|--------------------------------------------------------------------------------------------|

Definition at line 225 of file `platform_ns_mailbox.c`.

Here is the call graph for this function:



### 8.26.2.7 tfm\_ns\_mailbox\_hal\_init()

```
int32_t tfm_ns_mailbox_hal_init (
 struct ns_mailbox_queue_t * queue)
```

Platform specific NSPE mailbox initialization. Invoked by [tfm\\_ns\\_mailbox\\_init](#)().

## Parameters

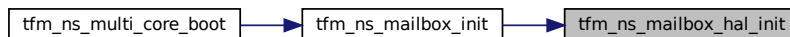
|                 |                    |                                                           |
|-----------------|--------------------|-----------------------------------------------------------|
| <code>in</code> | <code>queue</code> | The base address of NSPE mailbox queue to be initialized. |
|-----------------|--------------------|-----------------------------------------------------------|

## Return values

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| <code>MAILBOX_SUCCESS</code> | Operation succeeded.                             |
| <i>Other</i>                 | return code Operation failed with an error code. |

Definition at line 133 of file platform\_ns\_mailbox.c.

Here is the caller graph for this function:



## 8.26.2.8 tfm\_ns\_mailbox\_hal\_notify\_peer()

```
int32_t tfm_ns_mailbox_hal_notify_peer (
 void)
```

Notify SPE to deal with the PSA client call sent via mailbox.

## Note

The implementation depends on platform specific hardware and use case.

## Return values

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| <code>MAILBOX_SUCCESS</code> | Operation succeeded.                             |
| <i>Other</i>                 | return code Operation failed with an error code. |

Definition at line 118 of file platform\_ns\_mailbox.c.

## 8.26.2.9 tfm\_ns\_mailbox\_init()

```
int32_t tfm_ns_mailbox_init (
 struct ns_mailbox_queue_t * queue)
```

NSPE mailbox initialization.

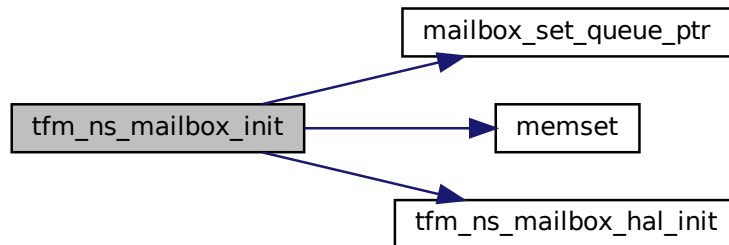
## Parameters

|                 |                    |                                                           |
|-----------------|--------------------|-----------------------------------------------------------|
| <code>in</code> | <code>queue</code> | The base address of NSPE mailbox queue to be initialized. |
|-----------------|--------------------|-----------------------------------------------------------|

## Return values

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| <code>MAILBOX_SUCCESS</code> | Operation succeeded.                             |
| <i>Other</i>                 | return code Operation failed with an error code. |

Definition at line 19 of file tfm\_ns\_mailbox\_common.c.  
Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.26.2.10 tfm\_ns\_multi\_core\_boot()

```
int32_t tfm_ns_multi_core_boot (
 struct ns_mailbox_queue_t * queue)
```

Performs multicore boot sequence in NSPE.

##### Parameters

|    |              |                                                           |
|----|--------------|-----------------------------------------------------------|
| in | <i>queue</i> | The base address of NSPE mailbox queue to be initialized. |
|----|--------------|-----------------------------------------------------------|

##### Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | Operation succeeded.                             |
| <i>Other</i>           | return code Operation failed with an error code. |

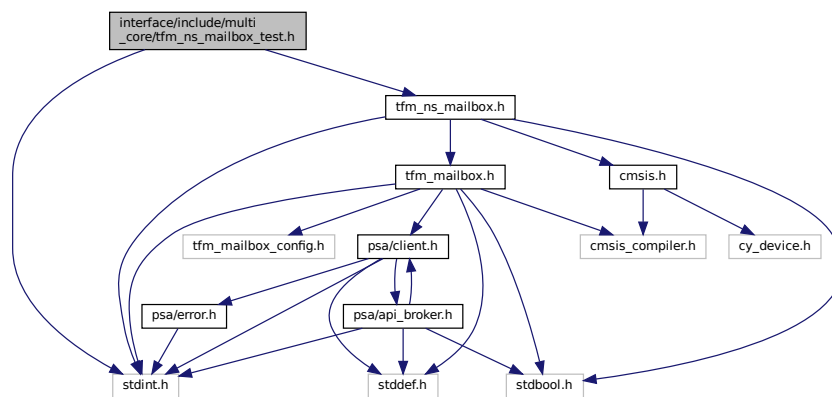
Definition at line 26 of file platform\_ns\_mailbox.c.

```

graph LR
 A[tfm_ns_multi_core_boot] --> B[tfm_ns_mailbox_init]
 A --> C[tfm_platform_ns_wait_for_s_cpu_ready]
 B --> D[mailbox_set_queue_ptr]
 B --> E[memset]
 B --> F[tfm_ns_mailbox_hal_init]

```

```
#include <stdint.h>
#include "tfm_ns_mailbox.h"
Include dependency graph for tfm_ns_mailbox_test.h:
```



- struct `ns_mailbox_stats_res_t`  
*The structure to hold the statistics result of NSPE mailbox.*

- `void tfm_ns_mailbox_tx_stats_init` (struct `ns_mailbox_queue_t` \*ns\_queue)  
*Initialize the statistics module in TF-M NSPE mailbox.*
- `int32_t tfm_ns_mailbox_tx_stats_reinit` (void)  
*Re-initialize the statistics module in TF-M NSPE mailbox. Clean up statistics data.*
- `void tfm_ns_mailbox_tx_stats_update` (void)  
*Update the statistics result of NSPE mailbox message transmission.*
- `void tfm_ns_mailbox_stats_avg_slot` (struct `ns_mailbox_stats_res_t` \*stats\_res)  
*Calculate the average number of used NS mailbox queue slots each time NS task requires a queue slot to submit mailbox message, which is recorded in NS mailbox statistics module.*

Generated by Doxygen

### 8.27.1.1 tfm\_ns\_mailbox\_stats\_avg\_slot()

```
void tfm_ns_mailbox_stats_avg_slot (
 struct ns_mailbox_stats_res_t * stats_res)
```

Calculate the average number of used NS mailbox queue slots each time NS task requires a queue slot to submit mailbox message, which is recorded in NS mailbox statistics module.

#### Note

This function is only available when multi-core tests are enabled.

#### Parameters

|    |           |                                                                        |
|----|-----------|------------------------------------------------------------------------|
| in | stats_res | The buffer to be written with <a href="#">ns_mailbox_stats_res_t</a> . |
|----|-----------|------------------------------------------------------------------------|

### 8.27.1.2 tfm\_ns\_mailbox\_tx\_stats\_init()

```
void tfm_ns_mailbox_tx_stats_init (
 struct ns_mailbox_queue_t * ns_queue)
```

Initialize the statistics module in TF-M NSPE mailbox.

#### Note

This function is only available when multi-core tests are enabled.

#### Parameters

|    |          |                                       |
|----|----------|---------------------------------------|
| in | ns_queue | The NSPE mailbox queue to be tracked. |
|----|----------|---------------------------------------|

### 8.27.1.3 tfm\_ns\_mailbox\_tx\_stats\_reinit()

```
int32_t tfm_ns_mailbox_tx_stats_reinit (
 void)
```

Re-initialize the statistics module in TF-M NSPE mailbox. Clean up statistics data.

#### Note

This function is only available when multi-core tests are enabled.

#### Returns

[MAILBOX\\_SUCCESS](#) if the operation succeeded, or other return code in case of error

### 8.27.1.4 tfm\_ns\_mailbox\_tx\_stats\_update()

```
void tfm_ns_mailbox_tx_stats_update (
 void)
```

Update the statistics result of NSPE mailbox message transmission.

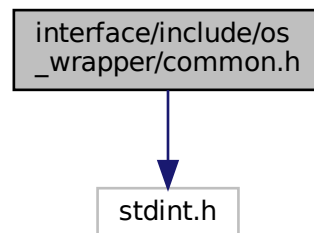
#### Note

This function is only available when multi-core tests are enabled.

## 8.28 interface/include/os\_wrapper/common.h File Reference

```
#include <stdint.h>
```

Include dependency graph for common.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define OS_WRAPPER_SUCCESS (0x0)`
- `#define OS_WRAPPER_ERROR (0xFFFFFFFFFU)`
- `#define OS_WRAPPER_WAIT_FOREVER (0xFFFFFFFFFU)`
- `#define OS_WRAPPER_DEFAULT_STACK_SIZE (-1)`

### 8.28.1 Macro Definition Documentation

#### 8.28.1.1 OS\_WRAPPER\_DEFAULT\_STACK\_SIZE

```
#define OS_WRAPPER_DEFAULT_STACK_SIZE (-1)
```

Definition at line 20 of file common.h.

#### 8.28.1.2 OS\_WRAPPER\_ERROR

```
#define OS_WRAPPER_ERROR (0xFFFFFFFFFU)
```

Definition at line 18 of file common.h.

#### 8.28.1.3 OS\_WRAPPER\_SUCCESS

```
#define OS_WRAPPER_SUCCESS (0x0)
```

Definition at line 17 of file common.h.

### 8.28.1.4 OS\_WRAPPER\_WAIT\_FOREVER

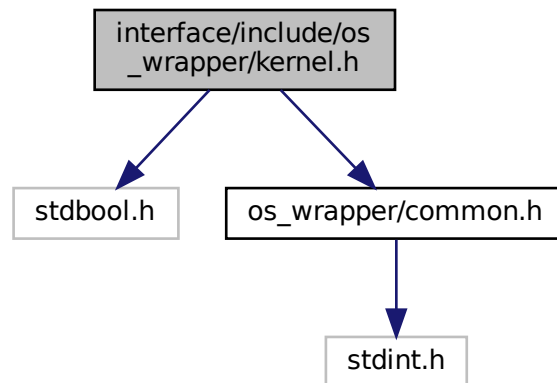
```
#define OS_WRAPPER_WAIT_FOREVER (0xFFFFFFFFU)
```

Definition at line 19 of file common.h.

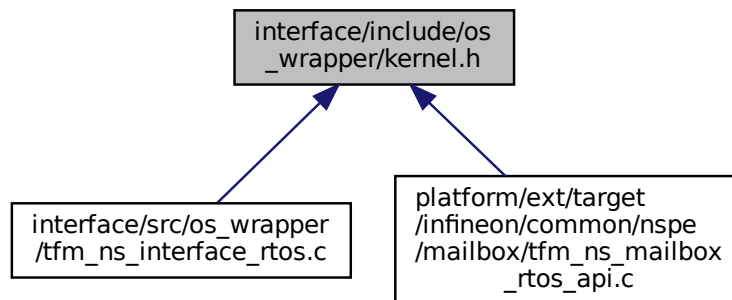
## 8.29 interface/include/os\_wrapper/kernel.h File Reference

```
#include <stdbool.h>
#include "os_wrapper/common.h"
```

Include dependency graph for kernel.h:



This graph shows which files directly or indirectly include this file:



## Functions

- bool [os\\_wrapper\\_is\\_kernel\\_started](#) (void)

*Returns value that indicates whether RTOS kernel has started running.*



## 8.29.1 Function Documentation

### 8.29.1.1 os\_wrapper\_is\_kernel\_started()

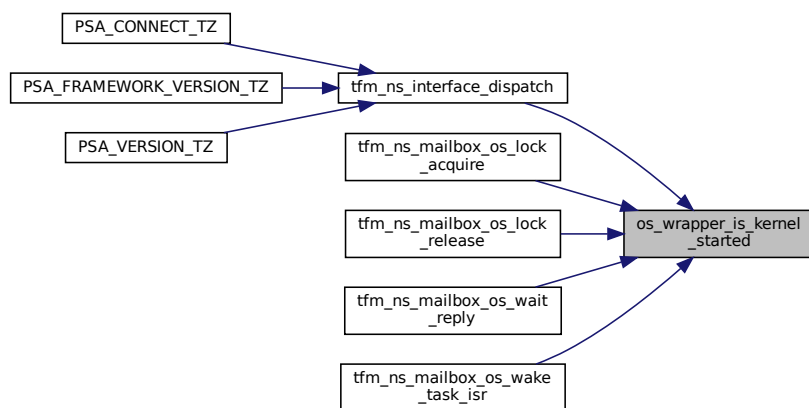
```
bool os_wrapper_is_kernel_started (
 void)
```

Returns value that indicates whether RTOS kernel has started running.

#### Returns

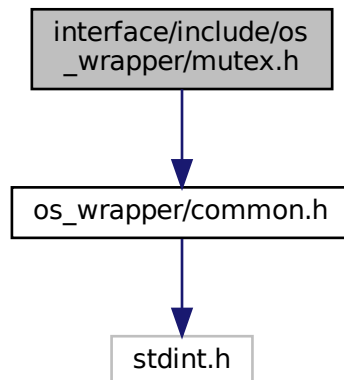
true if RTOS kernel has been started. false if RTOS kernel has not been started.

Here is the caller graph for this function:

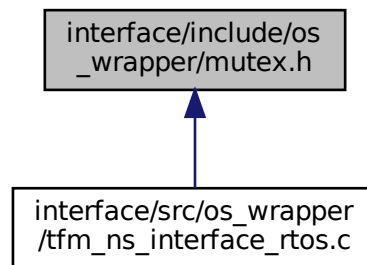


## 8.30 interface/include/os\_wrapper/mutex.h File Reference

```
#include "os_wrapper/common.h"
Include dependency graph for mutex.h:
```



This graph shows which files directly or indirectly include this file:



## Functions

- `void * os_wrapper_mutex_create (void)`  
*Creates a mutex for mutual exclusion of resources.*
- `uint32_t os_wrapper_mutex_acquire (void *handle, uint32_t timeout)`  
*Acquires a mutex that is created by `os_wrapper_mutex_create()`*
- `uint32_t os_wrapper_mutex_release (void *handle)`  
*Releases the mutex acquired previously.*
- `uint32_t os_wrapper_mutex_delete (void *handle)`  
*Deletes a mutex that is created by `os_wrapper_mutex_create()`*

## 8.30.1 Function Documentation

### 8.30.1.1 os\_wrapper\_mutex\_acquire()

```
uint32_t os_wrapper_mutex_acquire (
 void * handle,
 uint32_t timeout)
```

Acquires a mutex that is created by `os_wrapper_mutex_create()`

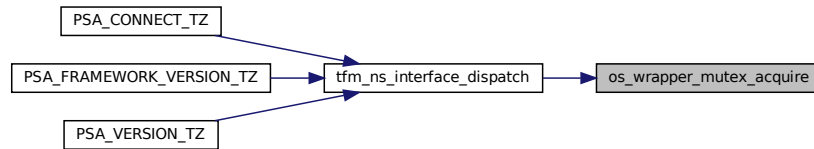
#### Parameters

|    |                |                                                                                                                                                                                                                                                                |
|----|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>handle</i>  | The handle of the mutex to acquire. Should be one of the handles returned by <code>os_wrapper_mutex_create()</code>                                                                                                                                            |
| in | <i>timeout</i> | The maximum amount of time(in tick periods) for the thread to wait for the mutex to be available. If timeout is zero, the function will return immediately. Setting timeout to <code>OS_WRAPPER_WAIT_FOREVER</code> will cause the thread to wait indefinitely |

## Returns

[OS\\_WRAPPER\\_SUCCESS](#) on success or [OS\\_WRAPPER\\_ERROR](#) on error or timeout

Here is the caller graph for this function:

8.30.1.2 `os_wrapper_mutex_create()`

```
void* os_wrapper_mutex_create (
 void)
```

Creates a mutex for mutual exclusion of resources.

## Returns

The handle of the created mutex on success or NULL on error

8.30.1.3 `os_wrapper_mutex_delete()`

```
uint32_t os_wrapper_mutex_delete (
 void * handle)
```

Deletes a mutex that is created by [os\\_wrapper\\_mutex\\_create\(\)](#)

## Parameters

|    |               |                                       |
|----|---------------|---------------------------------------|
| in | <i>handle</i> | The handle of the mutex to be deleted |
|----|---------------|---------------------------------------|

## Returns

[OS\\_WRAPPER\\_SUCCESS](#) on success or [OS\\_WRAPPER\\_ERROR](#) on error

8.30.1.4 `os_wrapper_mutex_release()`

```
uint32_t os_wrapper_mutex_release (
 void * handle)
```

Releases the mutex acquired previously.

## Parameters

|    |               |                                                |
|----|---------------|------------------------------------------------|
| in | <i>handle</i> | The handle of the mutex that has been acquired |
|----|---------------|------------------------------------------------|

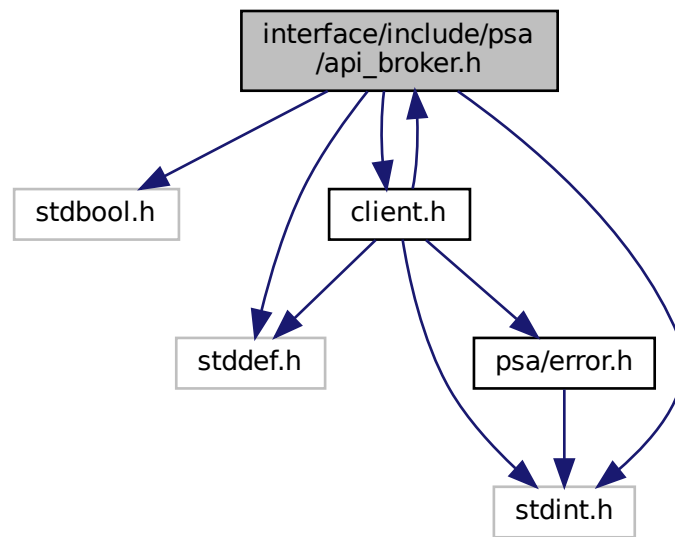
## Returns

[OS\\_WRAPPER\\_SUCCESS](#) on success or [OS\\_WRAPPER\\_ERROR](#) on error

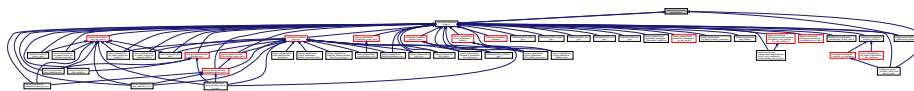
### 8.31 interface/include/psa/apiBroker.h File Reference

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "client.h"
```

Include dependency graph for apiBroker.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define PSA_CLOSE_TZ psa_close`
- `#define PSA_CONNECT_TZ psa_connect`
- `#define PSA_CALL_TZ psa_call`
- `#define PSA_FRAMEWORK_VERSION_TZ psa_framework_version`
- `#define PSA_VERSION_TZ psa_version`
- `#define PSA_CLOSE_MULTICORE psa_close`
- `#define PSA_CONNECT_MULTICORE psa_connect`
- `#define PSA_CALL_MULTICORE psa_call`
- `#define PSA_FRAMEWORK_VERSION_MULTICORE psa_framework_version`
- `#define PSA_VERSION_MULTICORE psa_version`

## 8.31.1 Macro Definition Documentation

### 8.31.1.1 PSA\_CALL\_MULTICORE

```
#define PSA_CALL_MULTICORE psa_call
```

Definition at line 73 of file api\_broker.h.

### 8.31.1.2 PSA\_CALL\_TZ

```
#define PSA_CALL_TZ psa_call
```

Definition at line 67 of file api\_broker.h.

### 8.31.1.3 PSA\_CLOSE\_MULTICORE

```
#define PSA_CLOSE_MULTICORE psa_close
```

Definition at line 71 of file api\_broker.h.

### 8.31.1.4 PSA\_CLOSE\_TZ

```
#define PSA_CLOSE_TZ psa_close
```

Definition at line 65 of file api\_broker.h.

### 8.31.1.5 PSA\_CONNECT\_MULTICORE

```
#define PSA_CONNECT_MULTICORE psa_connect
```

Definition at line 72 of file api\_broker.h.

### 8.31.1.6 PSA\_CONNECT\_TZ

```
#define PSA_CONNECT_TZ psa_connect
```

Definition at line 66 of file api\_broker.h.

### 8.31.1.7 PSA\_FRAMEWORK\_VERSION\_MULTICORE

```
#define PSA_FRAMEWORK_VERSION_MULTICORE psa_framework_version
```

Definition at line 74 of file api\_broker.h.

### 8.31.1.8 PSA\_FRAMEWORK\_VERSION\_TZ

```
#define PSA_FRAMEWORK_VERSION_TZ psa_framework_version
```

Definition at line 68 of file api\_broker.h.

### 8.31.1.9 PSA\_VERSION\_MULTICORE

```
#define PSA_VERSION_MULTICORE psa_version
```

Definition at line 75 of file api\_broker.h.

### 8.31.1.10 PSA\_VERSION\_TZ

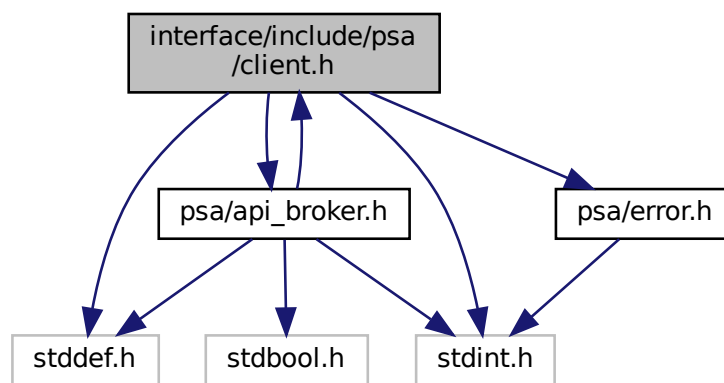
```
#define PSA_VERSION_TZ psa_version
```

Definition at line 69 of file `api_broker.h`.

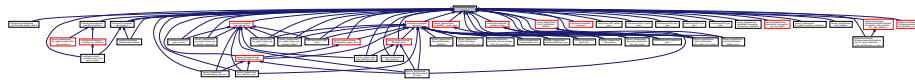
## 8.32 interface/include/psa/client.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "psa/api_broker.h"
#include "psa/error.h"
```

Include dependency graph for `client.h`:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [psa\\_invec](#)
- struct [psa\\_outvec](#)

## Macros

- #define [IOVEC\\_LEN](#)(arr) ((uint32\_t)(sizeof(arr)/sizeof(arr[0])))
- #define [ROT\\_SIZE\\_MAX](#) UINT32\_MAX
- #define [PSA\\_FRAMEWORK\\_VERSION](#) (0x0101u)
- #define [PSA\\_VERSION\\_NONE](#) (0u)
- #define [PSA\\_NULL\\_HANDLE](#) (([psa\\_handle\\_t](#))0)
- #define [PSA\\_HANDLE\\_IS\\_VALID](#)(handle) (([psa\\_handle\\_t](#))(handle) > 0)
- #define [PSA\\_HANDLE\\_TO\\_ERROR](#)(handle) (([psa\\_status\\_t](#))(handle))
- #define [PSA\\_MAX\\_IOVEC](#) (4u)
- #define [PSA\\_CALL\\_TYPE\\_MIN](#) (0)
- #define [PSA\\_CALL\\_TYPE\\_MAX](#) (INT16\_MAX)
- #define [PSA\\_IPC\\_CALL](#) (0)

## Typedefs

- typedef uint32\_t [rot\\_size\\_t](#)
- typedef int32\_t [psa\\_handle\\_t](#)
- typedef struct [psa\\_invec](#) [psa\\_invec](#)
- typedef struct [psa\\_outvec](#) [psa\\_outvec](#)

## Functions

- uint32\_t [psa\\_framework\\_version](#) (void)  
*Retrieve the version of the PSA Framework API that is implemented.*
- uint32\_t [psa\\_version](#) (uint32\_t sid)  
*Retrieve the version of an RoT Service or indicate that it is not present on this system.*
- [psa\\_handle\\_t](#) [psa\\_connect](#) (uint32\_t sid, uint32\_t version)  
*Connect to an RoT Service by its SID.*
- [psa\\_status\\_t](#) [psa\\_call](#) ([psa\\_handle\\_t](#) handle, int32\_t type, const [psa\\_invec](#) \*in\_vec, size\_t in\_len, [psa\\_outvec](#) \*out\_vec, size\_t out\_len)  
*Call an RoT Service on an established connection.*
- void [psa\\_close](#) ([psa\\_handle\\_t](#) handle)  
*Close a connection to an RoT Service.*

### 8.32.1 Macro Definition Documentation

#### 8.32.1.1 IOVEC\_LEN

```
#define IOVEC_LEN(
 arr) ((uint32_t) (sizeof(arr) / sizeof(arr[0])))
```

Definition at line 22 of file client.h.

#### 8.32.1.2 PSA\_CALL\_TYPE\_MAX

```
#define PSA_CALL_TYPE_MAX (INT16_MAX)
```

Definition at line 76 of file client.h.

#### 8.32.1.3 PSA\_CALL\_TYPE\_MIN

```
#define PSA_CALL_TYPE_MIN (0)
```

The minimum and maximum value in THIS implementation that can be passed as the type parameter in a call to [psa\\_call\(\)](#).

Definition at line 75 of file client.h.

#### 8.32.1.4 PSA\_FRAMEWORK\_VERSION

```
#define PSA_FRAMEWORK_VERSION (0x0101u)
```

The version of the PSA Framework API that is being used to build the calling firmware. Only part of features of FF-M v1.1 have been implemented. FF-M v1.1 is compatible with v1.0.

Definition at line 39 of file client.h.

#### 8.32.1.5 PSA\_HANDLE\_IS\_VALID

```
#define PSA_HANDLE_IS_VALID(
 handle) ((psa_handle_t)(handle) > 0)
```

Tests whether a handle value returned by [psa\\_connect\(\)](#) is valid.  
Definition at line 56 of file client.h.

#### 8.32.1.6 PSA\_HANDLE\_TO\_ERROR

```
#define PSA_HANDLE_TO_ERROR(
 handle) ((psa_status_t)(handle))
```

Converts the handle value returned from a failed call [psa\\_connect\(\)](#) into an error code.  
Definition at line 62 of file client.h.

#### 8.32.1.7 PSA\_IPC\_CALL

```
#define PSA_IPC_CALL (0)
```

An IPC message type that indicates a generic client request.  
Definition at line 81 of file client.h.

#### 8.32.1.8 PSA\_MAX\_IOVEC

```
#define PSA_MAX_IOVEC (4u)
```

Maximum number of input and output vectors for a request to [psa\\_call\(\)](#).  
Definition at line 67 of file client.h.

#### 8.32.1.9 PSA\_NULL\_HANDLE

```
#define PSA_NULL_HANDLE ((psa_handle_t)0)
```

The zero-value null handle can be assigned to variables used in clients and RoT Services, indicating that there is no current connection or message.  
Definition at line 51 of file client.h.

#### 8.32.1.10 PSA\_VERSION\_NONE

```
#define PSA_VERSION_NONE (0u)
```

Return value from [psa\\_version\(\)](#) if the requested RoT Service is not present in the system.  
Definition at line 45 of file client.h.

#### 8.32.1.11 ROT\_SIZE\_MAX

```
#define ROT_SIZE_MAX UINT32_MAX
```

Definition at line 30 of file client.h.

### 8.32.2 Typedef Documentation

#### 8.32.2.1 psa\_handle\_t

```
typedef int32_t psa_handle_t
```

Definition at line 83 of file client.h.



### 8.32.2.2 `psa_invec`

```
typedef struct psa_invec psa_invec
```

A read-only input memory region provided to an RoT Service.

### 8.32.2.3 `psa_outvec`

```
typedef struct psa_outvec psa_outvec
```

A writable output memory region provided to an RoT Service.

### 8.32.2.4 `rot_size_t`

```
typedef uint32_t rot_size_t
```

Type definitions equivalent to `size_t` as defined in the RoT Service environment.

Definition at line 29 of file `client.h`.

## 8.32.3 Function Documentation

### 8.32.3.1 `psa_call()`

```
psa_status_t psa_call (
 psa_handle_t handle,
 int32_t type,
 const psa_invec * in_vec,
 size_t in_len,
 psa_outvec * out_vec,
 size_t out_len)
```

Call an RoT Service on an established connection.

#### Note

FF-M 1.0 proposes 6 parameters for `psa_call` but the secure gateway ABI support at most 4 parameters. T↔F-M chooses to encode 'in\_len', 'out\_len', and 'type' into a 32-bit integer to improve efficiency. Compared with struct-based encoding, this method saves extra memory check and memory copy operation. The disadvantage is that the 'type' range has to be reduced into a 16-bit integer. So with this encoding, the valid range for 'type' is 0-32767.

#### Parameters

|         |                |                                                                             |
|---------|----------------|-----------------------------------------------------------------------------|
| in      | <i>handle</i>  | A handle to an established connection.                                      |
| in      | <i>type</i>    | The request type. Must be zero( <a href="#">PSA_IPC_CALL</a> ) or positive. |
| in      | <i>in_vec</i>  | Array of input <a href="#">psa_invec</a> structures.                        |
| in      | <i>in_len</i>  | Number of input <a href="#">psa_invec</a> structures.                       |
| in, out | <i>out_vec</i> | Array of output <a href="#">psa_outvec</a> structures.                      |
| in      | <i>out_len</i> | Number of output <a href="#">psa_outvec</a> structures.                     |

#### Return values

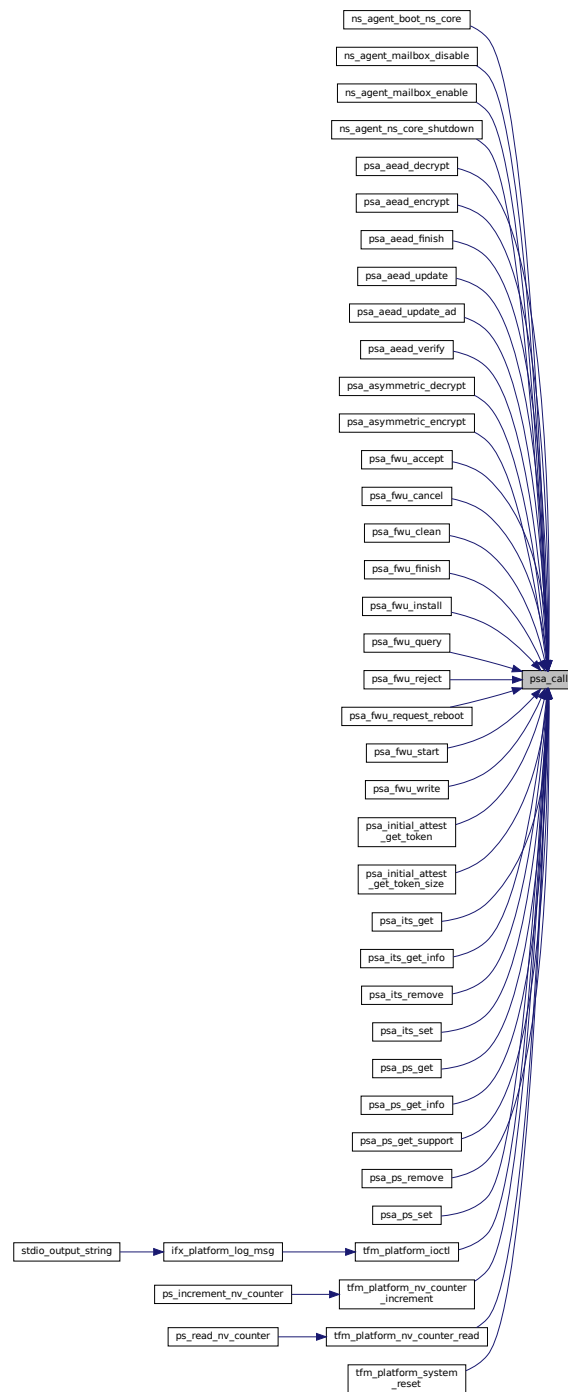
|          |                                    |
|----------|------------------------------------|
| $\geq 0$ | RoT Service-specific status value. |
| $< 0$    | RoT Service-specific error code.   |

## Return values

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_ERROR_PROGRAMMER_ERROR</i> | <p>The connection has been terminated by the RoT Service. The call is a PROGRAMMER ERROR if one or more of the following are true:</p> <ul style="list-style-type: none"><li>• An invalid handle was passed.</li><li>• The connection is already handling a request.</li><li>• <code>type &lt; 0</code>.</li><li>• An invalid memory reference was provided.</li><li>• <code>in_len + out_len &gt; PSA_MAX_IOVEC</code>.</li><li>• The message is unrecognized by the RoT Service or incorrectly formatted.</li></ul> |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Definition at line 88 of file `api_broker.c`.

Here is the caller graph for this function:



### 8.32.3.2 `psa_close()`

```
void psa_close (
 psa_handle_t handle)
```

Close a connection to an RoT Service.

## Parameters

|    |               |                                                            |
|----|---------------|------------------------------------------------------------|
| in | <i>handle</i> | A handle to an established connection, or the null handle. |
|----|---------------|------------------------------------------------------------|

## Note

The call is a PROGRAMMER ERROR if one or more of the following occurs:

- An invalid handle was provided that is not the null handle.
- The connection is currently handling a request.

Definition at line 106 of file api\_broker.c.

8.32.3.3 `psa_connect()`

```
psa_handle_t psa_connect (
 uint32_t sid,
 uint32_t version)
```

Connect to an RoT Service by its SID.

## Parameters

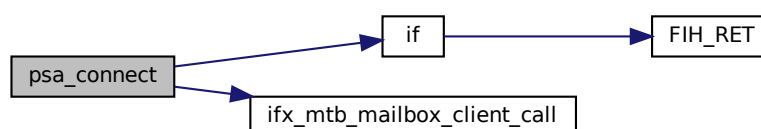
|    |                |                                       |
|----|----------------|---------------------------------------|
| in | <i>sid</i>     | ID of the RoT Service to connect to.  |
| in | <i>version</i> | Requested version of the RoT Service. |

## Return values

|                                     |                                                                                                                                                                                                                                                                                             |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| >                                   | 0 A handle for the connection.                                                                                                                                                                                                                                                              |
| <i>PSA_ERROR_CONNECTION_REFUSED</i> | The SPM or RoT Service has refused the connection.                                                                                                                                                                                                                                          |
| <i>PSA_ERROR_CONNECTION_BUSY</i>    | The SPM or RoT Service cannot make the connection at the moment.                                                                                                                                                                                                                            |
| <i>PROGRAMMER ERROR</i>             | <p>The call is a PROGRAMMER ERROR if one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• The RoT Service ID is not present.</li> <li>• The RoT Service version is not supported.</li> <li>• The caller is not allowed to access the RoT service.</li> </ul> |

Definition at line 101 of file api\_broker.c.

Here is the call graph for this function:



**8.32.3.4 psa\_framework\_version()**

```
uint32_t psa_framework_version (
 void)
```

Retrieve the version of the PSA Framework API that is implemented.

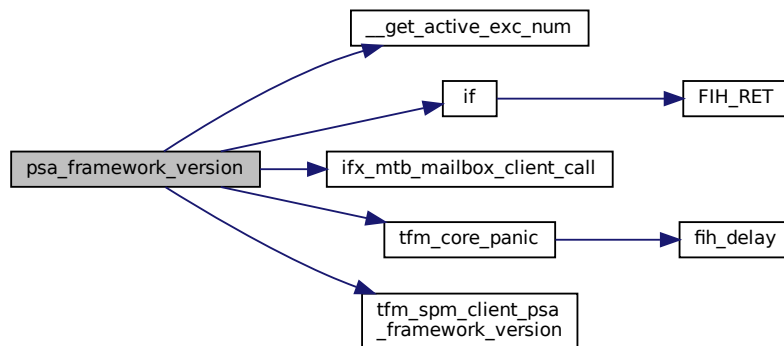
**Returns**

version The version of the PSA Framework implementation that is providing the runtime services to the caller. The major and minor version are encoded as follows:

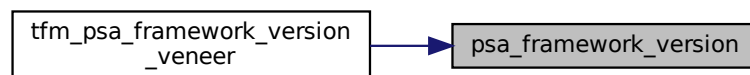
- version[15:8] – major version number.
- version[7:0] – minor version number.

Definition at line 78 of file api\_broker.c.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.32.3.5 psa\_version()**

```
uint32_t psa_version (
 uint32_t sid)
```

Retrieve the version of an RoT Service or indicate that it is not present on this system.

**Parameters**

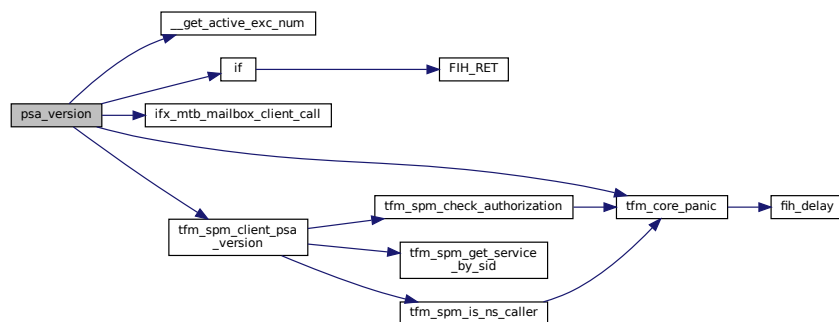
|    |     |                                 |
|----|-----|---------------------------------|
| in | sid | ID of the RoT Service to query. |
|----|-----|---------------------------------|

## Return values

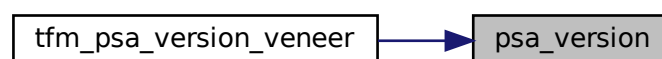
|                               |                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------|
| <code>PSA_VERSION_NONE</code> | The RoT Service is not implemented, or the caller is not permitted to access the service. |
| <code>&gt;</code>             | 0 The version of the implemented RoT Service.                                             |

Definition at line 83 of file `api_broker.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.33 interface/include/psa/crypto.h File Reference

Platform Security Architecture cryptography module.

```

#include "crypto_platform.h"
#include <stddef.h>
#include "crypto_types.h"
#include "crypto_values.h"
#include "crypto_sizes.h"
#include "crypto_struct.h"
#include "crypto_extra.h"

```

```

graph TD
 Root["interface/include/psa/crypto.h"]
 StdDef["stddef.h"]
 CryptoExtra["crypto_extra.h"]
 CryptoValues["crypto_values.h"]
 CryptoTypes["crypto_types.h"]
 CryptoSizes["crypto_sizes.h"]
 CryptoStruct["crypto_struct.h"]
 CryptoPlatform["crypto_platform.h"]
 StdInt["stdint.h"]
 MbedtlsAccess["mbedtls/private_access.h"]
 TfPsaBuildInfo["tf-psa-crypto/build/info.h"]
 PsaConfig["psa/crypto_config.h"]
 TfPsaSynonyms["tf-psa-crypto/private/crypto_adjust_config_synonyms.h"]
 TfPsaAutoEnabled["tf-psa-crypto/private/crypto_adjust_config_auto_enabled.h"]
 TfPsaDependencies["tf-psa-crypto/private/crypto_adjust_config_dependencies.h"]
 TfPsaKeyPairTypes["tf-psa-crypto/private/crypto_adjust_config_key_pair_types.h"]
 TfPsaDerived["tf-psa-crypto/private/crypto_adjust_config_derived.h"]
 TfPsaSupport["tf-psa-crypto/private/crypto_adjust_config_support.h"]

 Root --> StdDef
 Root --> CryptoExtra
 Root --> CryptoValues
 Root --> CryptoTypes
 Root --> CryptoSizes
 Root --> CryptoStruct
 Root --> CryptoPlatform
 Root --> StdInt
 Root --> MbedtlsAccess

 CryptoExtra --> CryptoCompat["crypto_compat.h"]

 CryptoValues --> CryptoPlatform
 CryptoTypes --> CryptoPlatform

 CryptoStruct --> CryptoPlatform
 CryptoStruct --> MbedtlsAccess

 StdInt --> TfPsaBuildInfo
 MbedtlsAccess --> TfPsaBuildInfo

 TfPsaBuildInfo --> PsaConfig
 TfPsaBuildInfo --> TfPsaSynonyms
 TfPsaBuildInfo --> TfPsaAutoEnabled
 TfPsaBuildInfo --> TfPsaDependencies
 TfPsaBuildInfo --> TfPsaKeyPairTypes
 TfPsaBuildInfo --> TfPsaDerived
 TfPsaBuildInfo --> TfPsaSupport

```

[illegible]

- #define PSA\_CRYPTO\_API\_VERSION\_MAJOR 1
- #define PSA\_CRYPTO\_API\_VERSION\_MINOR 2
- #define PSA\_KEY\_DERIVATION\_UNLIMITED\_CAPACITY ((size\_t) (-1))

- typedef struct [psa\\_hash\\_operation\\_s](#) [psa\\_hash\\_operation\\_t](#)
- typedef struct [psa\\_xof\\_operation\\_s](#) [psa\\_xof\\_operation\\_t](#)
- typedef struct [psa\\_mac\\_operation\\_s](#) [psa\\_mac\\_operation\\_t](#)
- typedef struct [psa\\_cipher\\_operation\\_s](#) [psa\\_cipher\\_operation\\_t](#)
- typedef struct [psa\\_aead\\_operation\\_s](#) [psa\\_aead\\_operation\\_t](#)
- typedef struct [psa\\_key\\_derivation\\_s](#) [psa\\_key\\_derivation\\_operation\\_t](#)
- typedef struct [psa\\_sign\\_hash\\_interruptible\\_operation\\_s](#) [psa\\_sign\\_hash\\_interruptible\\_operation\\_t](#)
- typedef struct [psa\\_verify\\_hash\\_interruptible\\_operation\\_s](#) [psa\\_verify\\_hash\\_interruptible\\_operation\\_t](#)
- typedef struct [psa\\_key\\_agreement\\_iop\\_s](#) [psa\\_key\\_agreement\\_iop\\_t](#)
- typedef struct [psa\\_generate\\_key\\_iop\\_s](#) [psa\\_generate\\_key\\_iop\\_t](#)
- typedef struct [psa\\_export\\_public\\_key\\_iop\\_s](#) [psa\\_export\\_public\\_key\\_iop\\_t](#)

- `psa_status_t psa_crypto_init (void)`  
*Library initialization.*
- `psa_status_t psa_get_key_attributes (mbedtls_svc_key_id_t key, psa_key_attributes_t *attributes)`
- `void psa_reset_key_attributes (psa_key_attributes_t *attributes)`
- `psa_status_t psa_purge_key (mbedtls_svc_key_id_t key)`
- `psa_status_t psa_copy_key (mbedtls_svc_key_id_t source_key, const psa_key_attributes_t *attributes, mbedtls_svc_key_id_t *target_key)`
- `psa_status_t psa_destroy_key (mbedtls_svc_key_id_t key)`

*Destroy a key.*

- [psa\\_status\\_t psa\\_import\\_key](#) (const [psa\\_key\\_attributes\\_t](#) \*attributes, const uint8\_t \*data, size\_t data\_length, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)

*Import a key in binary format.*

- [psa\\_status\\_t psa\\_export\\_key](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, uint8\_t \*data, size\_t data\_size, size\_t \*data\_length)

*Export a key in binary format.*

- [psa\\_status\\_t psa\\_export\\_public\\_key](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, uint8\_t \*data, size\_t data\_size, size\_t \*data\_length)

*Export a public key or the public part of a key pair in binary format.*

- [psa\\_status\\_t psa\\_hash\\_compute](#) ([psa\\_algorithm\\_t](#) alg, const uint8\_t \*input, size\_t input\_length, uint8\_t \*hash, size\_t hash\_size, size\_t \*hash\_length)
- [psa\\_status\\_t psa\\_hash\\_compare](#) ([psa\\_algorithm\\_t](#) alg, const uint8\_t \*input, size\_t input\_length, const uint8\_t \*hash, size\_t hash\_length)
- [psa\\_status\\_t psa\\_hash\\_setup](#) ([psa\\_hash\\_operation\\_t](#) \*operation, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_hash\\_update](#) ([psa\\_hash\\_operation\\_t](#) \*operation, const uint8\_t \*input, size\_t input\_length)
- [psa\\_status\\_t psa\\_hash\\_finish](#) ([psa\\_hash\\_operation\\_t](#) \*operation, uint8\_t \*hash, size\_t hash\_size, size\_t \*hash\_length)
- [psa\\_status\\_t psa\\_hash\\_verify](#) ([psa\\_hash\\_operation\\_t](#) \*operation, const uint8\_t \*hash, size\_t hash\_length)
- [psa\\_status\\_t psa\\_hash\\_abort](#) ([psa\\_hash\\_operation\\_t](#) \*operation)
- [psa\\_status\\_t psa\\_hash\\_clone](#) (const [psa\\_hash\\_operation\\_t](#) \*source\_operation, [psa\\_hash\\_operation\\_t](#) \*target\_operation)
- [psa\\_status\\_t psa\\_xof\\_setup](#) ([psa\\_xof\\_operation\\_t](#) \*operation, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_xof\\_set\\_context](#) ([psa\\_xof\\_operation\\_t](#) \*operation, const uint8\_t \*context, size\_t context\_length)
- [psa\\_status\\_t psa\\_xof\\_update](#) ([psa\\_xof\\_operation\\_t](#) \*operation, const uint8\_t \*input, size\_t input\_length)
- [psa\\_status\\_t psa\\_xof\\_output](#) ([psa\\_xof\\_operation\\_t](#) \*operation, uint8\_t \*output, size\_t output\_length)
- [psa\\_status\\_t psa\\_xof\\_abort](#) ([psa\\_xof\\_operation\\_t](#) \*operation)
- [psa\\_status\\_t psa\\_mac\\_compute](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const uint8\_t \*input, size\_t input\_length, uint8\_t \*mac, size\_t mac\_size, size\_t \*mac\_length)
- [psa\\_status\\_t psa\\_mac\\_verify](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const uint8\_t \*input, size\_t input\_length, const uint8\_t \*mac, size\_t mac\_length)
- [psa\\_status\\_t psa\\_mac\\_sign\\_setup](#) ([psa\\_mac\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_mac\\_verify\\_setup](#) ([psa\\_mac\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_mac\\_update](#) ([psa\\_mac\\_operation\\_t](#) \*operation, const uint8\_t \*input, size\_t input\_length)
- [psa\\_status\\_t psa\\_mac\\_sign\\_finish](#) ([psa\\_mac\\_operation\\_t](#) \*operation, uint8\_t \*mac, size\_t mac\_size, size\_t \*mac\_length)
- [psa\\_status\\_t psa\\_mac\\_verify\\_finish](#) ([psa\\_mac\\_operation\\_t](#) \*operation, const uint8\_t \*mac, size\_t mac\_length)
- [psa\\_status\\_t psa\\_mac\\_abort](#) ([psa\\_mac\\_operation\\_t](#) \*operation)
- [psa\\_status\\_t psa\\_cipher\\_encrypt](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const uint8\_t \*input, size\_t input\_length, uint8\_t \*output, size\_t output\_size, size\_t \*output\_length)
- [psa\\_status\\_t psa\\_cipher\\_decrypt](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const uint8\_t \*input, size\_t input\_length, uint8\_t \*output, size\_t output\_size, size\_t \*output\_length)
- [psa\\_status\\_t psa\\_cipher\\_encrypt\\_setup](#) ([psa\\_cipher\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_cipher\\_decrypt\\_setup](#) ([psa\\_cipher\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_cipher\\_generate\\_iv](#) ([psa\\_cipher\\_operation\\_t](#) \*operation, uint8\_t \*iv, size\_t iv\_size, size\_t \*iv\_length)
- [psa\\_status\\_t psa\\_cipher\\_set\\_iv](#) ([psa\\_cipher\\_operation\\_t](#) \*operation, const uint8\_t \*iv, size\_t iv\_length)
- [psa\\_status\\_t psa\\_cipher\\_update](#) ([psa\\_cipher\\_operation\\_t](#) \*operation, const uint8\_t \*input, size\_t input\_length, uint8\_t \*output, size\_t output\_size, size\_t \*output\_length)



- [psa\\_status\\_t psa\\_cipher\\_finish](#) ([psa\\_cipher\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
- [psa\\_status\\_t psa\\_cipher\\_abort](#) ([psa\\_cipher\\_operation\\_t](#) \*operation)
- [psa\\_status\\_t psa\\_aead\\_encrypt](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*nonce, [size\\_t](#) nonce\_length, const [uint8\\_t](#) \*additional\_data, [size\\_t](#) additional\_data\_length, const [uint8\\_t](#) \*plaintext, [size\\_t](#) plaintext\_length, [uint8\\_t](#) \*ciphertext, [size\\_t](#) ciphertext\_size, [size\\_t](#) \*ciphertext\_length)
- [psa\\_status\\_t psa\\_aead\\_decrypt](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*nonce, [size\\_t](#) nonce\_length, const [uint8\\_t](#) \*additional\_data, [size\\_t](#) additional\_data\_length, const [uint8\\_t](#) \*ciphertext, [size\\_t](#) ciphertext\_length, [uint8\\_t](#) \*plaintext, [size\\_t](#) plaintext\_size, [size\\_t](#) \*plaintext\_length)
- [psa\\_status\\_t psa\\_aead\\_encrypt\\_setup](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_aead\\_decrypt\\_setup](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_aead\\_generate\\_nonce](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*nonce, [size\\_t](#) nonce\_size, [size\\_t](#) \*nonce\_length)
- [psa\\_status\\_t psa\\_aead\\_set\\_nonce](#) ([psa\\_aead\\_operation\\_t](#) \*operation, const [uint8\\_t](#) \*nonce, [size\\_t](#) nonce\_size, [size\\_t](#) \*nonce\_length)
- [psa\\_status\\_t psa\\_aead\\_set\\_lengths](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [size\\_t](#) ad\_length, [size\\_t](#) plaintext\_size, [size\\_t](#) \*plaintext\_length)
- [psa\\_status\\_t psa\\_aead\\_update\\_ad](#) ([psa\\_aead\\_operation\\_t](#) \*operation, const [uint8\\_t](#) \*input, [size\\_t](#) input\_size, [size\\_t](#) \*input\_length)
- [psa\\_status\\_t psa\\_aead\\_update](#) ([psa\\_aead\\_operation\\_t](#) \*operation, const [uint8\\_t](#) \*input, [size\\_t](#) input\_size, [size\\_t](#) input\_length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
- [psa\\_status\\_t psa\\_aead\\_finish](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*ciphertext, [size\\_t](#) ciphertext\_size, [size\\_t](#) \*ciphertext\_length, [uint8\\_t](#) \*tag, [size\\_t](#) tag\_size, [size\\_t](#) \*tag\_length)
- [psa\\_status\\_t psa\\_aead\\_verify](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*plaintext, [size\\_t](#) plaintext\_size, [size\\_t](#) \*plaintext\_length, const [uint8\\_t](#) \*tag, [size\\_t](#) tag\_size, [size\\_t](#) \*tag\_length)
- [psa\\_status\\_t psa\\_aead\\_abort](#) ([psa\\_aead\\_operation\\_t](#) \*operation)
- [psa\\_status\\_t psa\\_sign\\_message](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_size, [size\\_t](#) \*input\_length, [uint8\\_t](#) \*signature, [size\\_t](#) signature\_size, [size\\_t](#) \*signature\_length)  
*Sign a message with a private key. For hash-and-sign algorithms, this includes the hashing step.*
- [psa\\_status\\_t psa\\_verify\\_message](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) \*input\_size, [size\\_t](#) \*input\_length, const [uint8\\_t](#) \*signature, [size\\_t](#) signature\_size, [size\\_t](#) \*signature\_length)  
*Verify the signature of a message with a public key, using a hash-and-sign verification algorithm.*
- [psa\\_status\\_t psa\\_unwrap\\_key](#) (const [psa\\_key\\_attributes\\_t](#) \*attributes, [psa\\_key\\_id\\_t](#) wrapping\_key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*data, [size\\_t](#) data\_size, [size\\_t](#) \*data\_length, [psa\\_key\\_id\\_t](#) \*key)  
*Unwrap and import a key using a specified wrapping key.*
- [psa\\_status\\_t psa\\_sign\\_hash](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*hash, [size\\_t](#) hash\_size, [size\\_t](#) \*hash\_length, [uint8\\_t](#) \*signature, [size\\_t](#) signature\_size, [size\\_t](#) \*signature\_length)  
*Sign a hash or short message with a private key.*
- [psa\\_status\\_t psa\\_verify\\_hash](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*hash, [size\\_t](#) \*hash\_size, [size\\_t](#) \*hash\_length, const [uint8\\_t](#) \*signature, [size\\_t](#) signature\_size, [size\\_t](#) \*signature\_length)  
*Verify the signature of a hash or short message using a public key.*
- [psa\\_status\\_t psa\\_asymmetric\\_encrypt](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_size, [size\\_t](#) \*input\_length, const [uint8\\_t](#) \*salt, [size\\_t](#) salt\_size, [size\\_t](#) \*salt\_length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)  
*Encrypt a short message with a public key.*
- [psa\\_status\\_t psa\\_asymmetric\\_decrypt](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_size, [size\\_t](#) \*input\_length, const [uint8\\_t](#) \*salt, [size\\_t](#) salt\_size, [size\\_t](#) \*salt\_length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)  
*Decrypt a short message with a private key.*
- [psa\\_status\\_t psa\\_key\\_derivation\\_setup](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_key\\_derivation\\_get\\_capacity](#) (const [psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [size\\_t](#) \*capacity)
- [psa\\_status\\_t psa\\_key\\_derivation\\_set\\_capacity](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [size\\_t](#) capacity)

- [psa\\_status\\_t psa\\_key\\_derivation\\_input\\_bytes](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_derivation\\_step\\_t](#) step, const [uint8\\_t](#) \*data, [size\\_t](#) data\_length)
- [psa\\_status\\_t psa\\_key\\_derivation\\_input\\_integer](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_derivation\\_step\\_t](#) step, [uint64\\_t](#) value)
- [psa\\_status\\_t psa\\_key\\_derivation\\_input\\_key](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_derivation\\_step\\_t](#) step, [mbedtls\\_svc\\_key\\_id\\_t](#) key)
- [psa\\_status\\_t psa\\_key\\_derivation\\_key\\_agreement](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_derivation\\_step\\_t](#) step, [mbedtls\\_svc\\_key\\_id\\_t](#) private\_key, const [uint8\\_t](#) \*peer\_key, [size\\_t](#) peer\_key\_length)
- [psa\\_status\\_t psa\\_key\\_derivation\\_output\\_bytes](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*output, [size\\_t](#) output\_length)
- [psa\\_status\\_t psa\\_key\\_derivation\\_output\\_key](#) (const [psa\\_key\\_attributes\\_t](#) \*attributes, [psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)
- [psa\\_status\\_t psa\\_key\\_derivation\\_output\\_key\\_custom](#) (const [psa\\_key\\_attributes\\_t](#) \*attributes, [psa\\_key\\_derivation\\_operation\\_t](#) \*operation, const [psa\\_custom\\_key\\_parameters\\_t](#) \*custom, const [uint8\\_t](#) \*custom\_data, [size\\_t](#) custom\_data\_length, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)
- [psa\\_status\\_t psa\\_key\\_derivation\\_verify\\_bytes](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, const [uint8\\_t](#) \*expected, [size\\_t](#) expected\_length)
- [psa\\_status\\_t psa\\_key\\_derivation\\_verify\\_key](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_id\\_t](#) expected)
- [psa\\_status\\_t psa\\_key\\_derivation\\_abort](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation)
- [psa\\_status\\_t psa\\_raw\\_key\\_agreement](#) ([psa\\_algorithm\\_t](#) alg, [mbedtls\\_svc\\_key\\_id\\_t](#) private\_key, const [uint8\\_t](#) \*peer\_key, [size\\_t](#) peer\_key\_length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
- [psa\\_status\\_t psa\\_key\\_agreement](#) ([mbedtls\\_svc\\_key\\_id\\_t](#) private\_key, const [uint8\\_t](#) \*peer\_key, [size\\_t](#) peer\_key\_length, [psa\\_algorithm\\_t](#) alg, const [psa\\_key\\_attributes\\_t](#) \*attributes, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)
- [psa\\_status\\_t psa\\_generate\\_random](#) ([uint8\\_t](#) \*output, [size\\_t](#) output\_size)  
Generate random bytes.
- [psa\\_status\\_t psa\\_generate\\_key](#) (const [psa\\_key\\_attributes\\_t](#) \*attributes, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)  
Generate a key or key pair.
- [psa\\_status\\_t psa\\_generate\\_key\\_custom](#) (const [psa\\_key\\_attributes\\_t](#) \*attributes, const [psa\\_custom\\_key\\_parameters\\_t](#) \*custom, const [uint8\\_t](#) \*custom\_data, [size\\_t](#) custom\_data\_length, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)  
Generate a key or key pair using custom production parameters.
- [void psa\\_interruptible\\_set\\_max\\_ops](#) ([uint32\\_t](#) max\_ops)  
Set the maximum number of ops allowed to be executed by an interruptible function in a single call.
- [uint32\\_t psa\\_interruptible\\_get\\_max\\_ops](#) ([void](#))  
Get the maximum number of ops allowed to be executed by an interruptible function in a single call. This will return the last value set by [psa\\_interruptible\\_set\\_max\\_ops\(\)](#) or [PSA\\_INTERRUPTIBLE\\_MAX\\_OPS\\_UNLIMITED](#) if that function has never been called.
- [uint32\\_t psa\\_sign\\_hash\\_get\\_num\\_ops](#) (const [psa\\_sign\\_hash\\_interruptible\\_operation\\_t](#) \*operation)  
Get the number of ops that a hash signing operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa\\_sign\\_hash\\_interruptible\\_abort\(\)](#) on the operation, a value of 0 will be returned.
- [uint32\\_t psa\\_verify\\_hash\\_get\\_num\\_ops](#) (const [psa\\_verify\\_hash\\_interruptible\\_operation\\_t](#) \*operation)  
Get the number of ops that a hash verification operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa\\_verify\\_hash\\_interruptible\\_abort\(\)](#) on the operation, a value of 0 will be returned.
- [psa\\_status\\_t psa\\_sign\\_hash\\_start](#) ([psa\\_sign\\_hash\\_interruptible\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*hash, [size\\_t](#) hash\_length)  
Start signing a hash or short message with a private key, in an interruptible manner.
- [psa\\_status\\_t psa\\_sign\\_hash\\_complete](#) ([psa\\_sign\\_hash\\_interruptible\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*signature, [size\\_t](#) signature\_size, [size\\_t](#) \*signature\_length)  
Continue and eventually complete the action of signing a hash or short message with a private key, in an interruptible manner.
- [psa\\_status\\_t psa\\_sign\\_hash\\_abort](#) ([psa\\_sign\\_hash\\_interruptible\\_operation\\_t](#) \*operation)  
Abort a sign hash operation.

- [psa\\_status\\_t psa\\_verify\\_hash\\_start](#) ([psa\\_verify\\_hash\\_interruptible\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, [const uint8\\_t](#) \*hash, [size\\_t](#) hash\_length, [const uint8\\_t](#) \*signature, [size\\_t](#) signature\_length)
 

*Start reading and verifying a hash or short message, in an interruptible manner.*
- [psa\\_status\\_t psa\\_verify\\_hash\\_complete](#) ([psa\\_verify\\_hash\\_interruptible\\_operation\\_t](#) \*operation)
 

*Continue and eventually complete the action of reading and verifying a hash or short message signed with a private key, in an interruptible manner.*
- [psa\\_status\\_t psa\\_verify\\_hash\\_abort](#) ([psa\\_verify\\_hash\\_interruptible\\_operation\\_t](#) \*operation)
 

*Abort a verify hash operation.*
- [uint32\\_t psa\\_key\\_agreement\\_iop\\_get\\_num\\_ops](#) ([psa\\_key\\_agreement\\_iop\\_t](#) \*operation)
 

*Get the number of ops that a key agreement operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa\\_key\\_agreement\\_iop\\_abort\(\)](#) on the operation, a value of 0 will be returned.*
- [psa\\_status\\_t psa\\_key\\_agreement\\_iop\\_setup](#) ([psa\\_key\\_agreement\\_iop\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) private\_key, [const uint8\\_t](#) \*peer\_key, [size\\_t](#) peer\_key\_length, [psa\\_algorithm\\_t](#) alg, [const psa\\_key\\_attributes\\_t](#) \*attributes)
 

*Start a key agreement operation, in an interruptible manner.*
- [psa\\_status\\_t psa\\_key\\_agreement\\_iop\\_complete](#) ([psa\\_key\\_agreement\\_iop\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)
 

*Continue and eventually complete the action of key agreement, in an interruptible manner.*
- [psa\\_status\\_t psa\\_key\\_agreement\\_iop\\_abort](#) ([psa\\_key\\_agreement\\_iop\\_t](#) \*operation)
 

*Abort a key agreement operation.*
- [uint32\\_t psa\\_generate\\_key\\_iop\\_get\\_num\\_ops](#) ([psa\\_generate\\_key\\_iop\\_t](#) \*operation)
 

*Get the number of ops that a key generation operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa\\_generate\\_key\\_iop\\_abort\(\)](#) on the operation, a value of 0 will be returned.*
- [psa\\_status\\_t psa\\_generate\\_key\\_iop\\_setup](#) ([psa\\_generate\\_key\\_iop\\_t](#) \*operation, [const psa\\_key\\_attributes\\_t](#) \*attributes)
 

*Start a key generation operation, in an interruptible manner.*
- [psa\\_status\\_t psa\\_generate\\_key\\_iop\\_complete](#) ([psa\\_generate\\_key\\_iop\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)
 

*Continue and eventually complete the action of key generation, in an interruptible manner.*
- [psa\\_status\\_t psa\\_generate\\_key\\_iop\\_abort](#) ([psa\\_generate\\_key\\_iop\\_t](#) \*operation)
 

*Abort a key generation operation.*
- [uint32\\_t psa\\_export\\_public\\_key\\_iop\\_get\\_num\\_ops](#) ([psa\\_export\\_public\\_key\\_iop\\_t](#) \*operation)
 

*Get the number of ops that an export public-key operation has taken so far. If the operation has completed, then this will represent the number of ops required for the entire operation. After initialization or calling [psa\\_export\\_public\\_key\\_iop\\_abort\(\)](#) on the operation, a value of 0 will be returned.*
- [psa\\_status\\_t psa\\_export\\_public\\_key\\_iop\\_setup](#) ([psa\\_export\\_public\\_key\\_iop\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) key)
 

*Start an interruptible operation to export a public key or the public part of a key pair in binary format.*
- [psa\\_status\\_t psa\\_export\\_public\\_key\\_iop\\_complete](#) ([psa\\_export\\_public\\_key\\_iop\\_t](#) \*operation, [uint8\\_t](#) \*data, [size\\_t](#) data\_size, [size\\_t](#) \*data\_length)
 

*Continue and eventually complete the action of exporting a public key, in an interruptible manner.*
- [psa\\_status\\_t psa\\_export\\_public\\_key\\_iop\\_abort](#) ([psa\\_export\\_public\\_key\\_iop\\_t](#) \*operation)
 

*Abort an interruptible public-key export operation.*

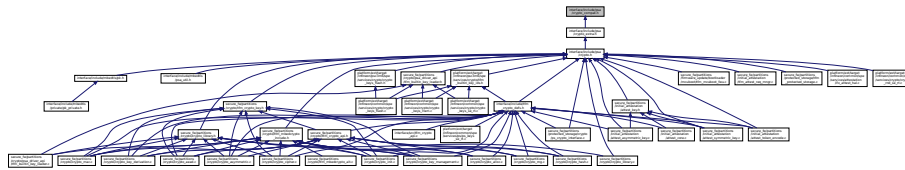
### 8.33.1 Detailed Description

Platform Security Architecture cryptography module.

## 8.34 interface/include/psa/crypto\_compat.h File Reference

PSA cryptography module: Backward compatibility aliases.

This graph shows which files directly or indirectly include this file:



### Macros

- `#define PSA_KEY_TYPE_DES ((psa_key_type_t) 0x2301)`
- `#define PSA_ALG_JPAKE_BETA PSA_ALG_JPAKE_BASE`
- `#define PSA_EXPORT_KEY_PAIR_OR_PUBLIC_MAX_SIZE ((size_t) MBEDTLS_DEPRECATED_NUMERIC_CONSTANT(P`

### Functions

- `int psa_can_do_hash (psa_algorithm_t hash_alg)`
- `int psa_can_do_cipher (psa_key_type_t key_type, psa_algorithm_t cipher_alg)`

#### 8.34.1 Detailed Description

PSA cryptography module: Backward compatibility aliases.

This header declares alternative names for macro and functions. New application code should not use these names. These names may be removed in a future version of Mbed TLS.

#### Note

This file may not be included directly. Applications must include [psa/crypto.h](#).

#### 8.34.2 Macro Definition Documentation

##### 8.34.2.1 PSA\_ALG\_JPAKE\_BETA

```
#define PSA_ALG_JPAKE_BETA PSA_ALG_JPAKE_BASE
```

The beta encoding of JPAKE algorithms, with no hash.

This came from the beta version of the PSA Crypto PAKE 1.2 extension, which is what Mbed TLS 3.x implemented. Since TF-PSA-Crypto 1.0.0, we no longer support the beta version of specification, so this algorithm encoding is no longer supported in JPAKE cipher suites. Use [PSA\\_ALG\\_JPAKE](#) instead.

#### Note

It is unspecified whether a key with [PSA\\_ALG\\_JPAKE\\_BETA](#) in its policy may be used to perform a JPAKE operation.

Definition at line 71 of file `crypto_compat.h`.

##### 8.34.2.2 PSA\_EXPORT\_KEY\_PAIR\_OR\_PUBLIC\_MAX\_SIZE

```
#define PSA_EXPORT_KEY_PAIR_OR_PUBLIC_MAX_SIZE ((size_t) MBEDTLS_DEPRECATED_NUMERIC_CONSTANT (PSA_EXPORT_ASYMMETRIC_KEY_MAX_SIZE))
```

Old non-standard name for [PSA\\_EXPORT\\_ASYMMETRIC\\_KEY\\_MAX\\_SIZE](#).

**Deprecated** Please use [PSA\\_EXPORT\\_ASYMMETRIC\\_KEY\\_MAX\\_SIZE](#) instead.

Definition at line 77 of file `crypto_compat.h`.

### 8.34.2.3 PSA\_KEY\_TYPE\_DES

```
#define PSA_KEY_TYPE_DES ((psa_key_type_t) 0x2301)
```

Definition at line 54 of file crypto\_compat.h.

## 8.34.3 Function Documentation

### 8.34.3.1 psa\_can\_do\_cipher()

```
int psa_can_do_cipher (
 psa_key_type_t key_type,
 psa_algorithm_t cipher_alg)
```

Definition at line 87 of file tfm\_crypto\_api.c.

### 8.34.3.2 psa\_can\_do\_hash()

```
int psa_can_do_hash (
 psa_algorithm_t hash_alg)
```

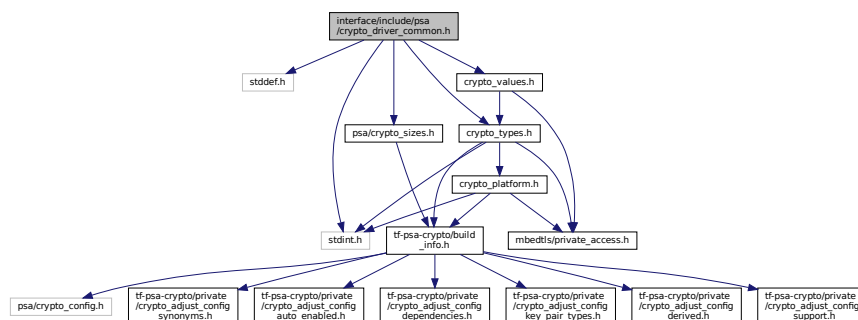
Definition at line 67 of file tfm\_crypto\_api.c.

## 8.35 interface/include/psa/crypto\_driver\_common.h File Reference

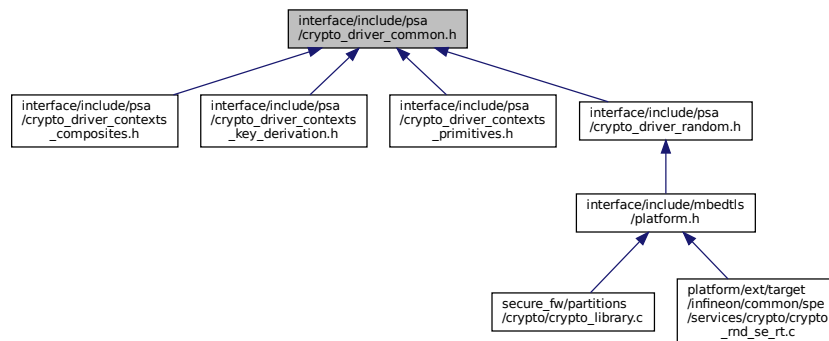
Definitions for all PSA crypto drivers.

```
#include <stddef.h>
#include <stdint.h>
#include "crypto_types.h"
#include "crypto_values.h"
#include <psa/crypto_sizes.h>
```

Include dependency graph for crypto\_driver\_common.h:



This graph shows which files directly or indirectly include this file:



## Enumerations

- enum [psa\\_encrypt\\_or\\_decrypt\\_t](#) { [PSA\\_CRYPTO\\_DRIVER\\_DECRYPT](#), [PSA\\_CRYPTO\\_DRIVER\\_ENCRYPT](#) }

### 8.35.1 Detailed Description

Definitions for all PSA crypto drivers.

This file contains common definitions shared by all PSA crypto drivers. Do not include it directly: instead, include the header file(s) for the type(s) of driver that you are implementing.

This file is part of the PSA Crypto Driver Model, containing functions for driver developers to implement to enable hardware to be called in a standardized way by a PSA Cryptographic API implementation. The functions comprising the driver model, which driver authors implement, are not intended to be called by application developers.

### 8.35.2 Enumeration Type Documentation

#### 8.35.2.1 [psa\\_encrypt\\_or\\_decrypt\\_t](#)

enum [psa\\_encrypt\\_or\\_decrypt\\_t](#)

For encrypt-decrypt functions, whether the operation is an encryption or a decryption.

Enumerator

|                                           |  |
|-------------------------------------------|--|
| <a href="#">PSA_CRYPTO_DRIVER_DECRYPT</a> |  |
| <a href="#">PSA_CRYPTO_DRIVER_ENCRYPT</a> |  |

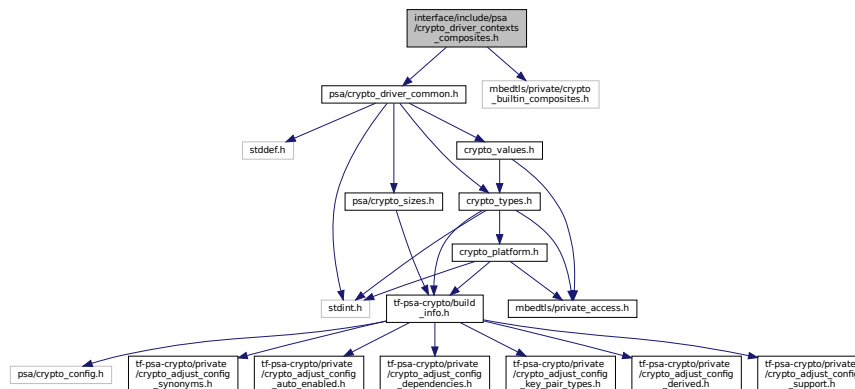
Definition at line 37 of file [crypto\\_driver\\_common.h](#).

## 8.36 [interface/include/psa/crypto\\_driver\\_contexts\\_composites.h](#) File Reference

Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for 'composite' operations, i.e. those operations which need to make use of other operations from the primitives ([crypto\\_driver\\_contexts\\_primitives.h](#))

```
#include "psa/crypto_driver_common.h"
#include "mbedtls/private/crypto_builtin_composites.h"
```

Include dependency graph for crypto\_driver\_contexts\_composites.h:



## Data Structures

- union [psa\\_driver\\_mac\\_context\\_t](#)
- union [psa\\_driver\\_aead\\_context\\_t](#)
- union [psa\\_driver\\_sign\\_hash\\_interruptible\\_context\\_t](#)
- union [psa\\_driver\\_verify\\_hash\\_interruptible\\_context\\_t](#)
- union [psa\\_driver\\_pake\\_context\\_t](#)

### 8.36.1 Detailed Description

Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for 'composite' operations, i.e. those operations which need to make use of other operations from the primitives ([crypto\\_driver\\_contexts\\_primitives.h](#))

#### Warning

This file will be auto-generated in the future.

#### Note

This file may not be included directly. Applications must include [psa/crypto.h](#).

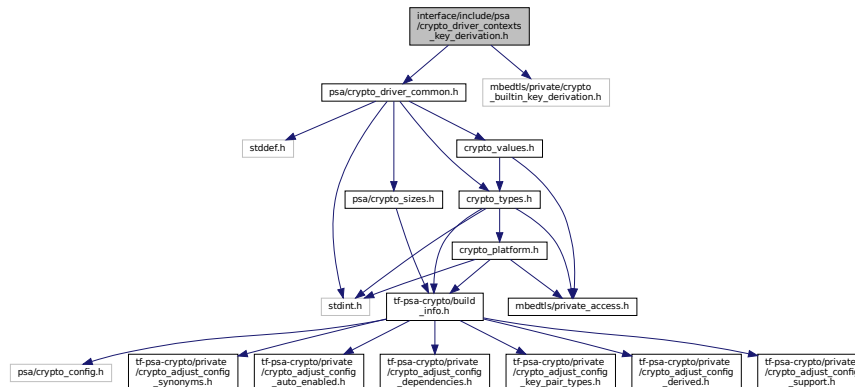
This header and its content are not part of the Mbed TLS API and applications must not depend on it. Its main purpose is to define the multi-part state objects of the PSA drivers included in the cryptographic library. The definitions of these objects are then used by [crypto\\_struct.h](#) to define the implementation-defined types of PSA multi-part state objects.

## 8.37 interface/include/psa/crypto\_driver\_contexts\_key\_derivation.h File Reference

Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for key derivation operations.

```
#include "psa/crypto_driver_common.h"
#include "mbedtls/private/crypto_builtin_key_derivation.h"
```

Include dependency graph for `crypto_driver_contexts_key_derivation.h`:



## Data Structures

- union [psa\\_driver\\_key\\_derivation\\_context\\_t](#)

### 8.37.1 Detailed Description

Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for key derivation operations.

#### Warning

This file will be auto-generated in the future.

#### Note

This file may not be included directly. Applications must include [psa/crypto.h](#).

This header and its content are not part of the Mbed TLS API and applications must not depend on it. Its main purpose is to define the multi-part state objects of the PSA drivers included in the cryptographic library. The definitions of these objects are then used by [crypto\\_struct.h](#) to define the implementation-defined types of PSA multi-part state objects.

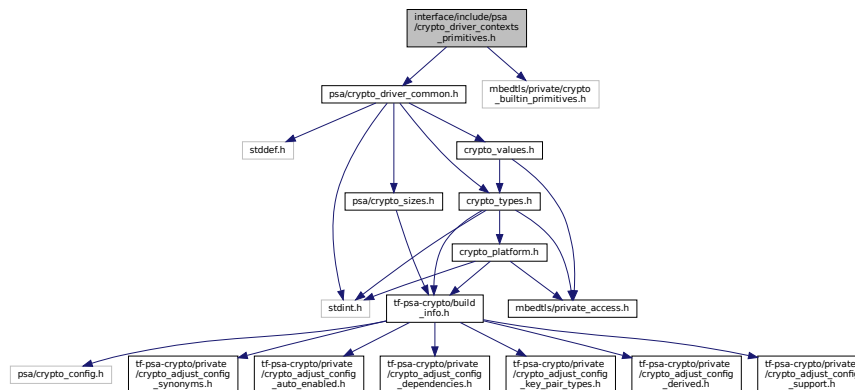
## 8.38 interface/include/psa/crypto\_driver\_contexts\_primitives.h File Reference

Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for 'primitive' operations, i.e. those operations which do not rely on other contexts.

```
#include "psa/crypto_driver_common.h"
#include "mbedtls/private/crypto_builtin_primitives.h"
```



Include dependency graph for crypto\_driver\_contexts\_primitives.h:



## Data Structures

- union [psa\\_driver\\_hash\\_context\\_t](#)
- union [psa\\_driver\\_xof\\_context\\_t](#)
- union [psa\\_driver\\_cipher\\_context\\_t](#)

### 8.38.1 Detailed Description

Declaration of context structures for use with the PSA driver wrapper interface. This file contains the context structures for 'primitive' operations, i.e. those operations which do not rely on other contexts.

#### Warning

This file will be auto-generated in the future.

**Note**

This file may not be included directly. Applications must include [psa/crypto.h](#).

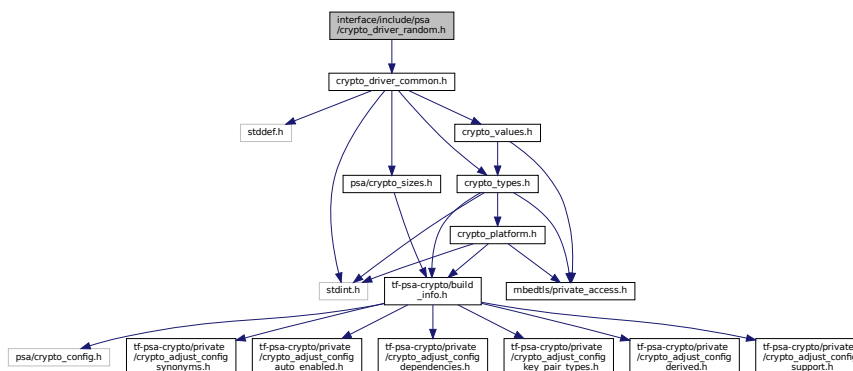
This header and its content are not part of the Mbed TLS API and applications must not depend on it. Its main purpose is to define the multi-part state objects of the PSA drivers included in the cryptographic library. The definitions of these objects are then used by [crypto\\_struct.h](#) to define the implementation-defined types of PSA multi-part state objects.

## 8.39 interface/include/psa/crypto\_driver\_random.h File Reference

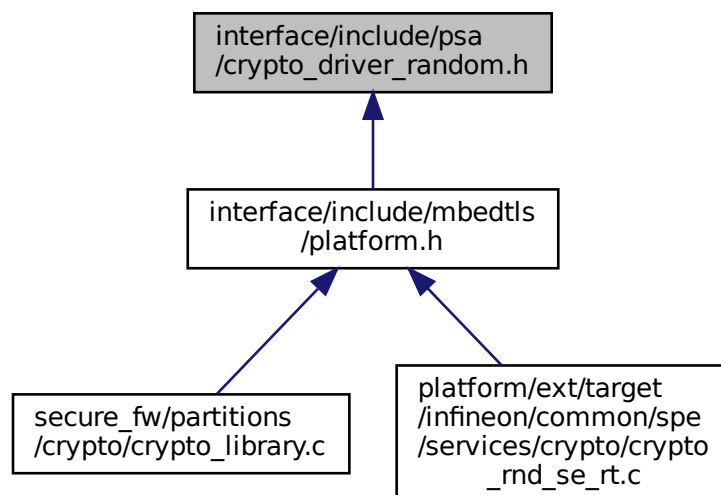
Definitions for PSA random and entropy drivers.

```
#include "crypto_driver_common.h"
```

Include dependency graph for `crypto_driver_random.h`:



This graph shows which files directly or indirectly include this file:

**Macros**

- `#define PSA_DRIVER_GET_ENTROPY_FLAGS_NONE ((psa_driver_get_entropy_flags_t) 0)`

## Typedefs

- typedef uint32\_t [psa\\_driver\\_get\\_entropy\\_flags\\_t](#)

### 8.39.1 Detailed Description

Definitions for PSA random and entropy drivers.

This file is part of the PSA Crypto Driver Model, containing functions for driver developers to implement to enable hardware to be called in a standardized way by a PSA Cryptographic API implementation. The functions comprising the driver model, which driver authors implement, are not intended to be called by application developers.

## 8.40 interface/include/psa/crypto\_extra.h File Reference

PSA cryptography module: Mbed TLS vendor extensions.

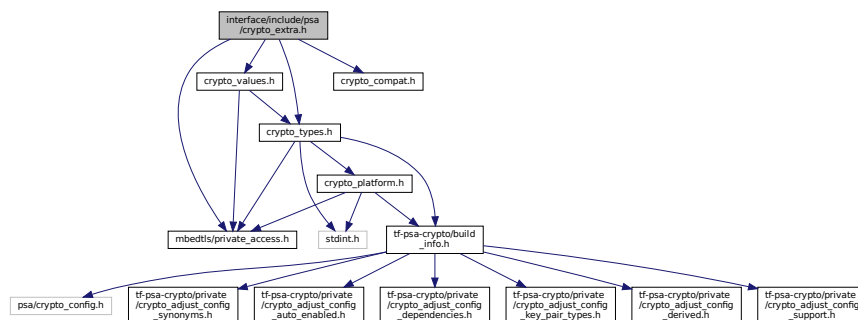
```
#include "mbedtls/private_access.h"
```

```
#include "crypto_types.h"
```

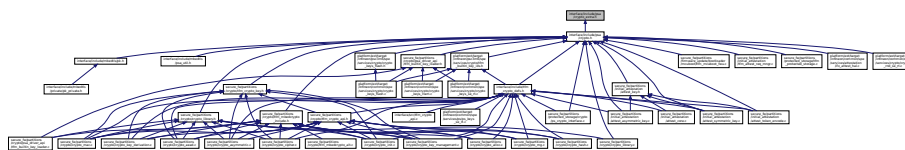
```
#include "crypto_compat.h"
```

```
#include "crypto_values.h"
```

Include dependency graph for crypto\_extra.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [mbedtls\\_psa\\_stats\\_s](#)  
Statistics about resource consumption related to the PSA keystore.
- struct [psa\\_pake\\_cipher\\_suite\\_s](#)
- struct [psa\\_crypto\\_driver\\_pake\\_inputs\\_s](#)
- struct [psa\\_jpake\\_computation\\_stage\\_s](#)
- struct [psa\\_pake\\_operation\\_s](#)

## Macros

- #define [PSA\\_CRYPTO\\_ITS\\_RANDOM\\_SEED\\_UID](#) 0xFFFFFFFF52
- #define [MBEDTLS\\_PSA\\_KEY\\_SLOT\\_COUNT](#) 32
- #define [MBEDTLS\\_PSA\\_STATIC\\_KEY\\_SLOT\\_BUFFER\\_SIZE](#) 1

```

• #define PSA_KEY_TYPE_DSA_PUBLIC_KEY ((psa_key_type_t) 0x4002)
• #define PSA_KEY_TYPE_DSA_KEY_PAIR ((psa_key_type_t) 0x7002)
• #define PSA_KEY_TYPE_IS_DSA(type) (PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) ==
 PSA_KEY_TYPE_DSA_PUBLIC_KEY)
• #define PSA_ALG_DSA_BASE ((psa_algorithm_t) 0x06000400)
• #define PSA_ALG_DSA(hash_alg) (PSA_ALG_DSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_DETERMINISTIC_DSA_BASE ((psa_algorithm_t) 0x06000500)
• #define PSA_ALG_DSA_DETERMINISTIC_FLAG PSA_ALG_ECDSA_DETERMINISTIC_FLAG
• #define PSA_ALG_DETERMINISTIC_DSA(hash_alg) (PSA_ALG_DETERMINISTIC_DSA_BASE | ((hash_↵
 _alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_IS_DSA(alg)
• #define PSA_ALG_DSA_IS_DETERMINISTIC(alg) (((alg) & PSA_ALG_DSA_DETERMINISTIC_FLAG) != 0)
• #define PSA_ALG_IS_DETERMINISTIC_DSA(alg) (PSA_ALG_IS_DSA(alg) && PSA_ALG_DSA_IS_DETERMINISTIC(alg))
• #define PSA_ALG_IS_RANDOMIZED_DSA(alg) (PSA_ALG_IS_DSA(alg) && !PSA_ALG_DSA_IS_DETERMINISTIC(alg))
• #define PSA_ALG_IS_VENDOR_HASH_AND_SIGN(alg) PSA_ALG_IS_DSA(alg)
• #define PSA_PAKE_OPERATION_STAGE_SETUP 0
• #define PSA_PAKE_OPERATION_STAGE_COLLECT_INPUTS 1
• #define PSA_PAKE_OPERATION_STAGE_COMPUTATION 2
• #define MBEDTLS_PSA_KEY_ID_BUILTIN_MIN ((psa_key_id_t) 0x7fff0000)
• #define MBEDTLS_PSA_KEY_ID_BUILTIN_MAX ((psa_key_id_t) 0x7ffffeff)
• #define PSA_ALG_CATEGORY_PAKE ((psa_algorithm_t) 0x0a000000)
• #define PSA_ALG_IS_PAKE(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_PAKE)
• #define PSA_ALG_JPAKE_BASE ((psa_algorithm_t) 0x0a000100)
• #define PSA_ALG_JPAKE(hash_alg) (PSA_ALG_JPAKE_BASE | ((hash_alg) & (PSA_ALG_HASH_MASK)))
• #define PSA_ALG_IS_JPAKE(alg) (((alg) & (~PSA_ALG_HASH_MASK))) == PSA_ALG_JPAKE_BASE)
• #define PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4400)
• #define PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE ((psa_key_type_t) 0x7400)
• #define PSA_KEY_TYPE_SPAKE2P_KEY_PAIR(curve) (PSA_KEY_TYPE_SPAKE2P_KEY_PAIR_BASE |
 (curve))
• #define PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY(curve) (PSA_KEY_TYPE_SPAKE2P_PUBLIC_KEY_BASE
 | (curve))
• #define PSA_KEY_TYPE_IS_SPAKE2P_KEY_PAIR(type)
• #define PSA_KEY_TYPE_IS_SPAKE2P_PUBLIC_KEY(type)
• #define PSA_KEY_TYPE_IS_SPAKE2P(type)
• #define PSA_ALG_SPAKE2P_HMAC_BASE ((psa_algorithm_t) 0x0a000400)
• #define PSA_ALG_SPAKE2P_HMAC(hash_alg) (PSA_ALG_SPAKE2P_HMAC_BASE | ((hash_alg) &
 (PSA_ALG_HASH_MASK)))
• #define PSA_ALG_IS_SPAKE2P_HMAC(alg) (((alg) & (~PSA_ALG_HASH_MASK))) == PSA_ALG_SPAKE2P_HMAC_BASE)
• #define PSA_ALG_SPAKE2P_CMAC_BASE ((psa_algorithm_t) 0x0a000500)
• #define PSA_ALG_SPAKE2P_CMAC(hash_alg) (PSA_ALG_SPAKE2P_CMAC_BASE | ((hash_alg) &
 (PSA_ALG_HASH_MASK)))
• #define PSA_ALG_IS_SPAKE2P_CMAC(alg) (((alg) & (~PSA_ALG_HASH_MASK))) == PSA_ALG_SPAKE2P_CMAC_BASE)
• #define PSA_ALG_SPAKE2P_MATTER ((psa_algorithm_t) 0x0a000609)
• #define PSA_ALG_IS_SPAKE2P(alg)
• #define PSA_PAKE_ROLE_NONE ((psa_pake_role_t) 0x00)
• #define PSA_PAKE_ROLE_FIRST ((psa_pake_role_t) 0x01)
• #define PSA_PAKE_ROLE_SECOND ((psa_pake_role_t) 0x02)
• #define PSA_PAKE_ROLE_CLIENT ((psa_pake_role_t) 0x11)
• #define PSA_PAKE_ROLE_SERVER ((psa_pake_role_t) 0x12)
• #define PSA_PAKE_PRIMITIVE_TYPE_ECC ((psa_pake_primitive_type_t) 0x01)
• #define PSA_PAKE_PRIMITIVE_TYPE_DH ((psa_pake_primitive_type_t) 0x02)
• #define PSA_PAKE_PRIMITIVE(pake_type, pake_family, pake_bits)
• #define PSA_PAKE_STEP_KEY_SHARE ((psa_pake_step_t) 0x01)
• #define PSA_PAKE_STEP_ZK_PUBLIC ((psa_pake_step_t) 0x02)
• #define PSA_PAKE_STEP_ZK_PROOF ((psa_pake_step_t) 0x03)

```

- `#define PSA_PAKE_STEP_CONFIRM ((psa_pake_step_t) 0x04)`
- `#define PSA_PAKE_OUTPUT_SIZE(alg, primitive, output_step)`
- `#define PSA_PAKE_INPUT_SIZE(alg, primitive, input_step)`
- `#define PSA_PAKE_OUTPUT_MAX_SIZE 65`
- `#define PSA_PAKE_INPUT_MAX_SIZE 65`
- `#define PSA_PAKE_CIPHER_SUITE_INIT { PSA_ALG_NONE, 0, 0, 0, 0 }`
- `#define PSA_PAKE_OPERATION_INIT`
- `#define PSA_PAKE_CONFIRMED_KEY 0`
- `#define PSA_PAKE_UNCONFIRMED_KEY 1`
- `#define PSA_JPAKE_EXPECTED_INPUTS(round)`
- `#define PSA_JPAKE_EXPECTED_OUTPUTS(round)`

## Typedefs

- typedef struct `mbedtls_psa_stats_s` `mbedtls_psa_stats_t`  
*Statistics about resource consumption related to the PSA keystore.*
- typedef uint64\_t `psa_drv_slot_number_t`
- typedef uint8\_t `psa_pake_role_t`  
*Encoding of the application role of PAKE.*
- typedef uint8\_t `psa_pake_step_t`
- typedef uint8\_t `psa_pake_primitive_type_t`
- typedef uint8\_t `psa_pake_family_t`  
*Encoding of the family of the primitive associated with the PAKE.*
- typedef uint32\_t `psa_pake_primitive_t`  
*Encoding of the primitive associated with the PAKE.*
- typedef enum `psa_crypto_driver_pake_step` `psa_crypto_driver_pake_step_t`
- typedef enum `psa_jpake_round` `psa_jpake_round_t`
- typedef enum `psa_jpake_io_mode` `psa_jpake_io_mode_t`
- typedef struct `psa_pake_cipher_suite_s` `psa_pake_cipher_suite_t`
- typedef struct `psa_pake_operation_s` `psa_pake_operation_t`
- typedef struct `psa_crypto_driver_pake_inputs_s` `psa_crypto_driver_pake_inputs_t`
- typedef struct `psa_jpake_computation_stage_s` `psa_jpake_computation_stage_t`

## Enumerations

- enum `psa_crypto_driver_pake_step` {  
`PSA_JPAKE_STEP_INVALID` = 0, `PSA_JPAKE_X1_STEP_KEY_SHARE` = 1, `PSA_JPAKE_X1_STEP_ZK_PUBLIC` = 2, `PSA_JPAKE_X1_STEP_ZK_PROOF` = 3,  
`PSA_JPAKE_X2_STEP_KEY_SHARE` = 4, `PSA_JPAKE_X2_STEP_ZK_PUBLIC` = 5, `PSA_JPAKE_X2_STEP_ZK_PROOF` = 6, `PSA_JPAKE_X2S_STEP_KEY_SHARE` = 7,  
`PSA_JPAKE_X2S_STEP_ZK_PUBLIC` = 8, `PSA_JPAKE_X2S_STEP_ZK_PROOF` = 9, `PSA_JPAKE_X4S_STEP_KEY_SHARE` = 10, `PSA_JPAKE_X4S_STEP_ZK_PUBLIC` = 11,  
`PSA_JPAKE_X4S_STEP_ZK_PROOF` = 12 }
- enum `psa_jpake_round` { `PSA_JPAKE_FIRST` = 0, `PSA_JPAKE_SECOND` = 1, `PSA_JPAKE_FINISHED` = 2 }
- enum `psa_jpake_io_mode` { `PSA_JPAKE_INPUT` = 0, `PSA_JPAKE_OUTPUT` = 1 }

## Functions

- void `mbedtls_psa_crypto_free` (void)  
*Library deinitialization.*
- void `mbedtls_psa_get_stats` (`mbedtls_psa_stats_t` \*stats)  
*Get statistics about resource consumption related to the PSA keystore.*
- `psa_status_t` `psa_random_reseed` (const uint8\_t \*perso, size\_t perso\_size)
- `psa_status_t` `psa_random_deplete` (void)

- [psa\\_status\\_t psa\\_random\\_set\\_prediction\\_resistance](#) (unsigned enabled)
- [uint8\\_t MBEDTLS\\_PRIVATE](#) (dummy)
- [psa\\_driver\\_pake\\_context\\_t MBEDTLS\\_PRIVATE](#) (ctx)
- [struct psa\\_crypto\\_driver\\_pake\\_inputs\\_s MBEDTLS\\_PRIVATE](#) (inputs)
- [psa\\_status\\_t psa\\_crypto\\_driver\\_pake\\_get\\_password\\_len](#) (const [psa\\_crypto\\_driver\\_pake\\_inputs\\_t](#) \*inputs, [size\\_t](#) \*password\_len)
- [psa\\_status\\_t psa\\_crypto\\_driver\\_pake\\_get\\_password](#) (const [psa\\_crypto\\_driver\\_pake\\_inputs\\_t](#) \*inputs, [uint8\\_t](#) \*buffer, [size\\_t](#) buffer\_size, [size\\_t](#) \*buffer\_length)
- [psa\\_status\\_t psa\\_crypto\\_driver\\_pake\\_get\\_user\\_len](#) (const [psa\\_crypto\\_driver\\_pake\\_inputs\\_t](#) \*inputs, [size\\_t](#) \*user\_len)
- [psa\\_status\\_t psa\\_crypto\\_driver\\_pake\\_get\\_peer\\_len](#) (const [psa\\_crypto\\_driver\\_pake\\_inputs\\_t](#) \*inputs, [size\\_t](#) \*peer\_len)
- [psa\\_status\\_t psa\\_crypto\\_driver\\_pake\\_get\\_user](#) (const [psa\\_crypto\\_driver\\_pake\\_inputs\\_t](#) \*inputs, [uint8\\_t](#) \*user\_id, [size\\_t](#) user\_id\_size, [size\\_t](#) \*user\_id\_len)
- [psa\\_status\\_t psa\\_crypto\\_driver\\_pake\\_get\\_peer](#) (const [psa\\_crypto\\_driver\\_pake\\_inputs\\_t](#) \*inputs, [uint8\\_t](#) \*peer\_id, [size\\_t](#) peer\_id\_size, [size\\_t](#) \*peer\_id\_length)
- [psa\\_status\\_t psa\\_crypto\\_driver\\_pake\\_get\\_cipher\\_suite](#) (const [psa\\_crypto\\_driver\\_pake\\_inputs\\_t](#) \*inputs, [psa\\_pake\\_cipher\\_suite\\_t](#) \*cipher\_suite)
- [psa\\_status\\_t psa\\_pake\\_setup](#) ([psa\\_pake\\_operation\\_t](#) \*operation, [mbedtls\\_svc\\_key\\_id\\_t](#) password\_key, const [psa\\_pake\\_cipher\\_suite\\_t](#) \*cipher\_suite)
- [psa\\_status\\_t psa\\_pake\\_set\\_user](#) ([psa\\_pake\\_operation\\_t](#) \*operation, const [uint8\\_t](#) \*user\_id, [size\\_t](#) user\_id\_len)
- [psa\\_status\\_t psa\\_pake\\_set\\_peer](#) ([psa\\_pake\\_operation\\_t](#) \*operation, const [uint8\\_t](#) \*peer\_id, [size\\_t](#) peer\_id\_len)
- [psa\\_status\\_t psa\\_pake\\_set\\_role](#) ([psa\\_pake\\_operation\\_t](#) \*operation, [psa\\_pake\\_role\\_t](#) role)
- [psa\\_status\\_t psa\\_pake\\_set\\_context](#) ([psa\\_pake\\_operation\\_t](#) \*operation, const [uint8\\_t](#) \*context, [size\\_t](#) context\_len)
- [psa\\_status\\_t psa\\_pake\\_output](#) ([psa\\_pake\\_operation\\_t](#) \*operation, [psa\\_pake\\_step\\_t](#) step, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
- [psa\\_status\\_t psa\\_pake\\_input](#) ([psa\\_pake\\_operation\\_t](#) \*operation, [psa\\_pake\\_step\\_t](#) step, const [uint8\\_t](#) \*input, [size\\_t](#) input\_length)
- [psa\\_status\\_t psa\\_pake\\_get\\_shared\\_key](#) ([psa\\_pake\\_operation\\_t](#) \*operation, const [psa\\_key\\_attributes\\_t](#) \*attributes, [mbedtls\\_svc\\_key\\_id\\_t](#) \*key)
- [psa\\_status\\_t psa\\_pake\\_abort](#) ([psa\\_pake\\_operation\\_t](#) \*operation)

### 8.40.1 Detailed Description

PSA cryptography module: Mbed TLS vendor extensions.

Note

This file may not be included directly. Applications must include [psa/crypto.h](#).

This file is reserved for vendor-specific definitions.

### 8.40.2 Macro Definition Documentation

#### 8.40.2.1 MBEDTLS\_PSA\_KEY\_SLOT\_COUNT

```
#define MBEDTLS_PSA_KEY_SLOT_COUNT 32
```

Definition at line 33 of file [crypto\\_extra.h](#).

#### 8.40.2.2 MBEDTLS\_PSA\_STATIC\_KEY\_SLOT\_BUFFER\_SIZE

```
#define MBEDTLS_PSA_STATIC_KEY_SLOT_BUFFER_SIZE 1
```

Definition at line 41 of file [crypto\\_extra.h](#).

#### 8.40.2.3 PSA\_CRYPTO\_ITS\_RANDOM\_SEED\_UID

```
#define PSA_CRYPTO_ITS_RANDOM_SEED_UID 0xFFFFFFFF52
```

Definition at line 29 of file crypto\_extra.h.

#### 8.40.2.4 PSA\_JPAKE\_EXPECTED\_INPUTS

```
#define PSA_JPAKE_EXPECTED_INPUTS(
 round)
```

**Value:**

```
(round) == PSA_JPAKE_FINISHED ? 0 : \
(round) == PSA_JPAKE_FIRST ? 2 : 1))
```

Definition at line 1226 of file crypto\_extra.h.

#### 8.40.2.5 PSA\_JPAKE\_EXPECTED\_OUTPUTS

```
#define PSA_JPAKE_EXPECTED_OUTPUTS(
 round)
```

**Value:**

```
(round) == PSA_JPAKE_FINISHED ? 0 : \
(round) == PSA_JPAKE_FIRST ? 2 : 1))
```

Definition at line 1228 of file crypto\_extra.h.

#### 8.40.2.6 PSA\_PAKE\_CIPHER\_SUITE\_INIT

```
#define PSA_PAKE_CIPHER_SUITE_INIT { PSA_ALG_NONE, 0, 0, 0, 0 }
```

Returns a suitable initializer for a PAKE cipher suite object of type `psa_pake_cipher_suite_t`.

Definition at line 1130 of file crypto\_extra.h.

#### 8.40.2.7 PSA\_PAKE\_CONFIRMED\_KEY

```
#define PSA_PAKE_CONFIRMED_KEY 0
```

A key confirmation value that indicates an confirmed key in a PAKE cipher suite.

This key confirmation value will result in the PAKE algorithm exchanging data to verify that the shared key is identical for both parties. This is the default key confirmation value in an initialized PAKE cipher suite object.

Some algorithms do not include confirmation of the shared key.

Definition at line 1151 of file crypto extra.h.

#### 8.40.2.8 PSA PAKE INPUT MAX SIZE

```
#define PSA_PAKE_INPUT_MAX_SIZE 65
```

Input buffer size for `psa_pake_input()` for any of the supported PAKE algorithm and primitive suites and input step.

This macro must expand to a compile-time constant integer.

The value of this macro must be at least as large as the largest value returned by `PSA_PAKE_INPUT_SIZE()`

See also [PSA\\_PAKE\\_INPUT\\_SIZE](#)(alg, primitive, output\_step).

Definition at line 1125 of file crypto\_extra.h.

#### 8.40.2.9 PSA PAKE INPUT SIZE

```
#define PSA_PAKE_INPUT_SIZE(
 alg,
 primitive,
 input_step)
```

**Value:**

```
(PSA_ALG_IS_JPAKE(alg) &&
primitive == PSA_PAKE_PRIMITIVE(PSA_PAKE_PRIMITIVE_TYPE_ECC,
```

```

 PSA_ECC_FAMILY_SECP_R1, 256) ? \
 (
 input_step == PSA_PAKE_STEP_KEY_SHARE ? 65 : \
 input_step == PSA_PAKE_STEP_ZK_PUBLIC ? 65 : \
 32
) : \
 0)

```

A sufficient input buffer size for [psa\\_pake\\_input\(\)](#).

The value returned by this macro is guaranteed to be large enough for any valid input to [psa\\_pake\\_input\(\)](#) in an operation with the specified parameters.

See also [PSA\\_PAKE\\_INPUT\\_MAX\\_SIZE](#)

#### Parameters

|                   |                                                                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------|
| <i>alg</i>        | A PAKE algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_PAKE(alg)</a> is true).            |
| <i>primitive</i>  | A primitive of type <a href="#">psa_pake_primitive_t</a> that is compatible with algorithm <i>alg</i> . |
| <i>input_step</i> | A value of type <a href="#">psa_pake_step_t</a> that is valid for the algorithm <i>alg</i> .            |

#### Returns

A sufficient input buffer size for the specified input, cipher suite and algorithm. If the cipher suite, the input type or PAKE algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 1092 of file `crypto_extra.h`.

#### 8.40.2.10 PSA\_PAKE\_OPERATION\_INIT

```
#define PSA_PAKE_OPERATION_INIT
```

##### Value:

```

 { 0, PSA_ALG_NONE, 0, PSA_PAKE_OPERATION_STAGE_SETUP, \
 { 0 }, { { 0 } } }

```

Returns a suitable initializer for a PAKE operation object of type [psa\\_pake\\_operation\\_t](#).

Definition at line 1138 of file `crypto_extra.h`.

#### 8.40.2.11 PSA\_PAKE\_OUTPUT\_MAX\_SIZE

```
#define PSA_PAKE_OUTPUT_MAX_SIZE 65
```

Output buffer size for [psa\\_pake\\_output\(\)](#) for any of the supported PAKE algorithm and primitive suites and output step.

This macro must expand to a compile-time constant integer.

The value of this macro must be at least as large as the largest value returned by [PSA\\_PAKE\\_OUTPUT\\_SIZE\(\)](#)

See also [PSA\\_PAKE\\_OUTPUT\\_SIZE\(alg, primitive, output\\_step\)](#).

Definition at line 1113 of file `crypto_extra.h`.

#### 8.40.2.12 PSA\_PAKE\_OUTPUT\_SIZE

```

#define PSA_PAKE_OUTPUT_SIZE(
 alg,
 primitive,
 output_step)

```

##### Value:

```

 (PSA_ALG_IS_JPAKE(alg) &&
 primitive == PSA_PAKE_PRIMITIVE(PSA_PAKE_PRIMITIVE_TYPE_ECC,
 PSA_ECC_FAMILY_SECP_R1, 256) ? \
 (
 output_step == PSA_PAKE_STEP_KEY_SHARE ? 65 : \
 output_step == PSA_PAKE_STEP_ZK_PUBLIC ? 65 : \
 32
) : \
 0)

```



A sufficient output buffer size for `psa_pake_output()`.

If the size of the output buffer is at least this large, it is guaranteed that `psa_pake_output()` will not fail due to an insufficient output buffer size. The actual size of the output might be smaller in any given call.

See also `PSA_PAKE_OUTPUT_MAX_SIZE`

#### Parameters

|                    |                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------|
| <i>alg</i>         | A PAKE algorithm ( <code>PSA_ALG_XXX</code> value such that <code>PSA_ALG_IS_PAKE(alg)</code> is true).    |
| <i>primitive</i>   | A primitive of type <code>psa_pake_primitive_t</code> that is compatible with algorithm <code>alg</code> . |
| <i>output_step</i> | A value of type <code>psa_pake_step_t</code> that is valid for the algorithm <code>alg</code> .            |

#### Returns

A sufficient output buffer size for the specified PAKE algorithm, primitive, and output step. If the PAKE algorithm, primitive, or output step is not recognized, or the parameters are incompatible, return 0.

Definition at line 1062 of file `crypto_extra.h`.

### 8.40.2.13 PSA\_PAKE\_UNCONFIRMED\_KEY

```
#define PSA_PAKE_UNCONFIRMED_KEY 1
```

A key confirmation value that indicates an unconfirmed key in a PAKE cipher suite.

This key confirmation value will result in the PAKE algorithm terminating prior to confirming that the resulting shared key is identical for both parties.

Some algorithms do not support returning an unconfirmed shared key.

#### Warning

When the shared key is not confirmed as part of the PAKE operation, the application is responsible for mitigating risks that arise from the possible mismatch in the output keys.

Definition at line 1165 of file `crypto_extra.h`.

## 8.40.3 Typedef Documentation

### 8.40.3.1 mbedtls\_psa\_stats\_t

```
typedef struct mbedtls_psa_stats_s mbedtls_psa_stats_t
```

Statistics about resource consumption related to the PSA keystore.

#### Note

The content of this structure is not part of the stable API and ABI of Mbed TLS and may change arbitrarily from version to version.

### 8.40.3.2 psa\_crypto\_driver\_pake\_step\_t

```
typedef enum psa_crypto_driver_pake_step psa_crypto_driver_pake_step_t
```

### 8.40.3.3 psa\_jpake\_io\_mode\_t

```
typedef enum psa_jpake_io_mode psa_jpake_io_mode_t
```

#### 8.40.3.4 psa\_jpake\_round\_t

```
typedef enum psa_jpake_round psa_jpake_round_t
```

### 8.40.4 Enumeration Type Documentation

#### 8.40.4.1 psa\_crypto\_driver\_pake\_step

```
enum psa_crypto_driver_pake_step
```

##### Enumerator

|                              |  |
|------------------------------|--|
| PSA_JPAKE_STEP_INVALID       |  |
| PSA_JPAKE_X1_STEP_KEY_SHARE  |  |
| PSA_JPAKE_X1_STEP_ZK_PUBLIC  |  |
| PSA_JPAKE_X1_STEP_ZK_PROOF   |  |
| PSA_JPAKE_X2_STEP_KEY_SHARE  |  |
| PSA_JPAKE_X2_STEP_ZK_PUBLIC  |  |
| PSA_JPAKE_X2_STEP_ZK_PROOF   |  |
| PSA_JPAKE_X2S_STEP_KEY_SHARE |  |
| PSA_JPAKE_X2S_STEP_ZK_PUBLIC |  |
| PSA_JPAKE_X2S_STEP_ZK_PROOF  |  |
| PSA_JPAKE_X4S_STEP_KEY_SHARE |  |
| PSA_JPAKE_X4S_STEP_ZK_PUBLIC |  |
| PSA_JPAKE_X4S_STEP_ZK_PROOF  |  |

Definition at line 1186 of file crypto\_extra.h.

#### 8.40.4.2 psa\_jpake\_io\_mode

```
enum psa_jpake_io_mode
```

##### Enumerator

|                  |  |
|------------------|--|
| PSA_JPAKE_INPUT  |  |
| PSA_JPAKE_OUTPUT |  |

Definition at line 1208 of file crypto\_extra.h.

#### 8.40.4.3 psa\_jpake\_round

```
enum psa_jpake_round
```

##### Enumerator

|                    |  |
|--------------------|--|
| PSA_JPAKE_FIRST    |  |
| PSA_JPAKE_SECOND   |  |
| PSA_JPAKE_FINISHED |  |

Definition at line 1202 of file crypto\_extra.h.

## 8.40.5 Function Documentation

### 8.40.5.1 MBEDTLS\_PRIVATE() [1/3]

```
psa_driver_pake_context_t MBEDTLS_PRIVATE::MBEDTLS_PRIVATE (
 ctx)
```

### 8.40.5.2 MBEDTLS\_PRIVATE() [2/3]

```
uint8_t MBEDTLS_PRIVATE::MBEDTLS_PRIVATE (
 dummy)
```

### 8.40.5.3 MBEDTLS\_PRIVATE() [3/3]

```
struct psa_crypto_driver_pake_inputs_s MBEDTLS_PRIVATE::MBEDTLS_PRIVATE (
 inputs)
```

### 8.40.5.4 mbedtls\_psa\_crypto\_free()

```
void mbedtls_psa_crypto_free (
 void)
```

Library deinitialization.

This function clears all data associated with the PSA layer, including the whole key store. This function is not thread safe, it wipes every key slot regardless of state and reader count. It should only be called when no slot is in use.

This is an Mbed TLS extension.

### 8.40.5.5 mbedtls\_psa\_get\_stats()

```
void mbedtls_psa_get_stats (
 mbedtls_psa_stats_t * stats)
```

Get statistics about resource consumption related to the PSA keystore.

#### Note

When Mbed TLS is built as part of a service, with isolation between the application and the keystore, the service may or may not expose this function.

## 8.41 interface/include/psa/crypto\_platform.h File Reference

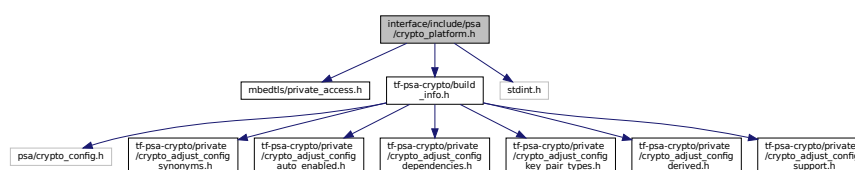
PSA cryptography module: Mbed TLS platform definitions.

```
#include "mbedtls/private_access.h"
```

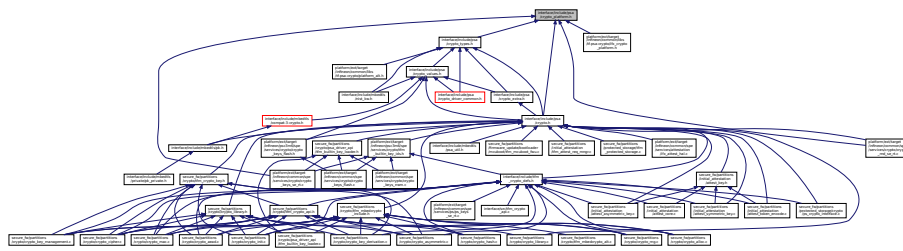
```
#include "tf-psa-crypto/build_info.h"
```

```
#include <stdint.h>
```

Include dependency graph for crypto\_platform.h:



This graph shows which files directly or indirectly include this file:



### 8.41.1 Detailed Description

PSA cryptography module: Mbed TLS platform definitions.

PSA cryptography module: TF-M platform definitions.

#### Note

This file may not be included directly. Applications must include [psa/crypto.h](#).

This file contains platform-dependent type definitions.

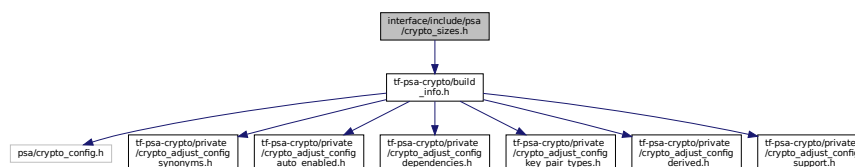
In implementations with isolation between the application and the cryptography module, implementers should take care to ensure that the definitions that are exposed to applications match what the module implements.

## 8.42 interface/include/psa/crypto\_sizes.h File Reference

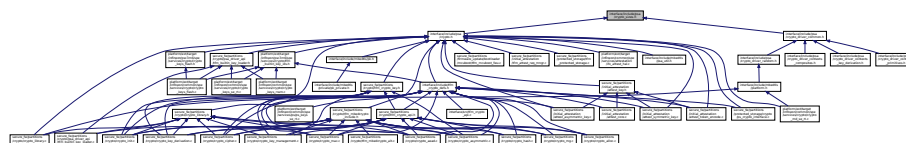
PSA cryptography module: Mbed TLS buffer size macros.

```
#include "tf-psa-crypto/build_info.h"
```

Include dependency graph for crypto\_sizes.h:



This graph shows which files directly or indirectly include this file:



### Macros

- #define [PSA\\_BITS\\_TO\\_BYTES](#)(bits) (((bits) + 7u) / 8u)
- #define [PSA\\_BYTES\\_TO\\_BITS](#)(bytes) ((bytes) \* 8u)
- #define [PSA\\_MAX\\_OF\\_THREE](#)(a, b, c)
- #define [PSA\\_ROUND\\_UP\\_TO\\_MULTIPLE](#)(block\_size, length) (((length) + (block\_size) - 1) / (block\_size) \* (block\_size))
- #define [PSA\\_HASH\\_LENGTH](#)(alg)
- #define [PSA\\_HASH\\_BLOCK\\_LENGTH](#)(alg)

- #define [PSA\\_HMAC\\_MAX\\_HASH\\_BLOCK\\_SIZE](#) 64u
- #define [PSA\\_HASH\\_MAX\\_SIZE](#) 20u
- #define [PSA\\_MAC\\_MAX\\_SIZE](#) [PSA\\_HASH\\_MAX\\_SIZE](#)
- #define [PSA\\_AEAD\\_TAG\\_LENGTH](#)(key\_type, key\_bits, alg)
- #define [PSA\\_AEAD\\_TAG\\_MAX\\_SIZE](#) 16u
- #define [PSA\\_VENDOR\\_RSA\\_MAX\\_KEY\\_BITS](#) 4096u
- #define [PSA\\_VENDOR\\_RSA\\_GENERATE\\_MIN\\_KEY\\_BITS](#) 1024
- #define [PSA\\_VENDOR\\_FFDH\\_MAX\\_KEY\\_BITS](#) 0u
- #define [PSA\\_VENDOR\\_ECC\\_MAX\\_CURVE\\_BITS](#) 0u
- #define [PSA\\_TLS12\\_PSK\\_TO\\_MS\\_PSK\\_MAX\\_SIZE](#) 128u
- #define [PSA\\_TLS12\\_ECJPAKE\\_TO\\_PMS\\_INPUT\\_SIZE](#) 65u
- #define [PSA\\_TLS12\\_ECJPAKE\\_TO\\_PMS\\_DATA\\_SIZE](#) 32u
- #define [PSA\\_VENDOR\\_PBKDF2\\_MAX\\_ITERATIONS](#) 0xffffffffU
- #define [PSA\\_BLOCK\\_CIPHER\\_BLOCK\\_MAX\\_SIZE](#) 16u
- #define [PSA\\_MAC\\_LENGTH](#)(key\_type, key\_bits, alg)
- #define [PSA\\_AEAD\\_ENCRYPT\\_OUTPUT\\_SIZE](#)(key\_type, alg, plaintext\_length)
- #define [PSA\\_AEAD\\_ENCRYPT\\_OUTPUT\\_MAX\\_SIZE](#)(plaintext\_length) ((plaintext\_length) + [PSA\\_AEAD\\_TAG\\_MAX\\_SIZE](#))
- #define [PSA\\_AEAD\\_DECRYPT\\_OUTPUT\\_SIZE](#)(key\_type, alg, ciphertext\_length)
- #define [PSA\\_AEAD\\_DECRYPT\\_OUTPUT\\_MAX\\_SIZE](#)(ciphertext\_length) (ciphertext\_length)
- #define [PSA\\_AEAD\\_NONCE\\_LENGTH](#)(key\_type, alg)
- #define [PSA\\_AEAD\\_NONCE\\_MAX\\_SIZE](#) 13u
- #define [PSA\\_AEAD\\_UPDATE\\_OUTPUT\\_SIZE](#)(key\_type, alg, input\_length)
- #define [PSA\\_AEAD\\_UPDATE\\_OUTPUT\\_MAX\\_SIZE](#)(input\_length) ([PSA\\_ROUND\\_UP\\_TO\\_MULTIPLE](#)([PSA\\_BLOCK\\_CIPHER\\_BLOCK\\_MAX\\_SIZE](#), (input\_length)))
- #define [PSA\\_AEAD\\_FINISH\\_OUTPUT\\_SIZE](#)(key\_type, alg)
- #define [PSA\\_AEAD\\_FINISH\\_OUTPUT\\_MAX\\_SIZE](#) ([PSA\\_BLOCK\\_CIPHER\\_BLOCK\\_MAX\\_SIZE](#))
- #define [PSA\\_AEAD\\_VERIFY\\_OUTPUT\\_SIZE](#)(key\_type, alg)
- #define [PSA\\_AEAD\\_VERIFY\\_OUTPUT\\_MAX\\_SIZE](#) ([PSA\\_BLOCK\\_CIPHER\\_BLOCK\\_MAX\\_SIZE](#))
- #define [PSA\\_RSA\\_MINIMUM\\_PADDING\\_SIZE](#)(alg)
- #define [PSA\\_ECDSA\\_SIGNATURE\\_SIZE](#)(curve\_bits) ([PSA\\_BITS\\_TO\\_BYTES](#)(curve\_bits) \* 2u)
- ECDSA signature size for a given curve bit size.*
- #define [PSA\\_SIGN\\_OUTPUT\\_SIZE](#)(key\_type, key\_bits, alg)
- #define [PSA\\_VENDOR\\_ECDSA\\_SIGNATURE\\_MAX\\_SIZE](#) [PSA\\_ECDSA\\_SIGNATURE\\_SIZE](#)([PSA\\_VENDOR\\_ECC\\_MAX\\_CURVE\\_BITS](#))
- #define [PSA\\_SIGNATURE\\_MAX\\_SIZE](#) 1
- #define [PSA\\_ASYMMETRIC\\_ENCRYPT\\_OUTPUT\\_SIZE](#)(key\_type, key\_bits, alg)
- #define [PSA\\_ASYMMETRIC\\_ENCRYPT\\_OUTPUT\\_MAX\\_SIZE](#) ([PSA\\_BITS\\_TO\\_BYTES](#)([PSA\\_VENDOR\\_RSA\\_MAX\\_KEY\\_BITS](#)))
- #define [PSA\\_ASYMMETRIC\\_DECRYPT\\_OUTPUT\\_SIZE](#)(key\_type, key\_bits, alg)
- #define [PSA\\_ASYMMETRIC\\_DECRYPT\\_OUTPUT\\_MAX\\_SIZE](#) ([PSA\\_BITS\\_TO\\_BYTES](#)([PSA\\_VENDOR\\_RSA\\_MAX\\_KEY\\_BITS](#)))
- #define [PSA\\_KEY\\_EXPORT\\_ASN1\\_INTEGER\\_MAX\\_SIZE](#)(bits) ((bits) / 8u + 5u)
- #define [PSA\\_KEY\\_EXPORT\\_RSA\\_PUBLIC\\_KEY\\_MAX\\_SIZE](#)(key\_bits) ([PSA\\_KEY\\_EXPORT\\_ASN1\\_INTEGER\\_MAX\\_SIZE](#)(key\_bits) + 11u)
- #define [PSA\\_KEY\\_EXPORT\\_RSA\\_KEY\\_PAIR\\_MAX\\_SIZE](#)(key\_bits) (9u \* [PSA\\_KEY\\_EXPORT\\_ASN1\\_INTEGER\\_MAX\\_SIZE](#)(key\_bits) / 2u + 1u) + 14u)
- #define [PSA\\_KEY\\_EXPORT\\_DSA\\_PUBLIC\\_KEY\\_MAX\\_SIZE](#)(key\_bits) ([PSA\\_KEY\\_EXPORT\\_ASN1\\_INTEGER\\_MAX\\_SIZE](#)(key\_bits) \* 3u + 59u)
- #define [PSA\\_KEY\\_EXPORT\\_DSA\\_KEY\\_PAIR\\_MAX\\_SIZE](#)(key\_bits) ([PSA\\_KEY\\_EXPORT\\_ASN1\\_INTEGER\\_MAX\\_SIZE](#)(key\_bits) \* 3u + 75u)
- #define [PSA\\_KEY\\_EXPORT\\_ECC\\_PUBLIC\\_KEY\\_MAX\\_SIZE](#)(key\_bits) (2u \* [PSA\\_BITS\\_TO\\_BYTES](#)(key\_bits) + 1u)
- #define [PSA\\_KEY\\_EXPORT\\_ECC\\_KEY\\_PAIR\\_MAX\\_SIZE](#)(key\_bits) ([PSA\\_BITS\\_TO\\_BYTES](#)(key\_bits))
- #define [PSA\\_KEY\\_EXPORT\\_FFDH\\_KEY\\_PAIR\\_MAX\\_SIZE](#)(key\_bits) ([PSA\\_BITS\\_TO\\_BYTES](#)(key\_bits))
- #define [PSA\\_KEY\\_EXPORT\\_FFDH\\_PUBLIC\\_KEY\\_MAX\\_SIZE](#)(key\_bits) ([PSA\\_BITS\\_TO\\_BYTES](#)(key\_bits))
- #define [PSA\\_EXPORT\\_KEY\\_OUTPUT\\_SIZE](#)(key\_type, key\_bits)
- #define [PSA\\_EXPORT\\_PUBLIC\\_KEY\\_OUTPUT\\_SIZE](#)(key\_type, key\_bits)

- `#define PSA_EXPORT_KEY_PAIR_MAX_SIZE 1`
- `#define PSA_EXPORT_PUBLIC_KEY_MAX_SIZE 1`
- `#define PSA_EXPORTASYMMETRIC_KEY_MAX_SIZE`
- `#define PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE(key_type, key_bits)`
- `#define PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE 1`
- `#define PSA_CIPHER_MAX_KEY_LENGTH 0u`
- `#define PSA_CIPHER_IV_LENGTH(key_type, alg)`
- `#define PSA_CIPHER_IV_MAX_SIZE 16u`
- `#define PSA_CIPHER_ENCRYPT_OUTPUT_SIZE(key_type, alg, input_length)`
- `#define PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE(input_length)`
- `#define PSA_CIPHER_DECRYPT_OUTPUT_SIZE(key_type, alg, input_length)`
- `#define PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE(input_length) (input_length)`
- `#define PSA_CIPHER_UPDATE_OUTPUT_SIZE(key_type, alg, input_length)`
- `#define PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE(input_length) (PSA_ROUND_UP_TO_MULTIPLE(PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE, input_length))`
- `#define PSA_CIPHER_FINISH_OUTPUT_SIZE(key_type, alg)`
- `#define PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE (PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE)`

### 8.42.1 Detailed Description

PSA cryptography module: Mbed TLS buffer size macros.

#### Note

This file may not be included directly. Applications must include [psa/crypto.h](#).

This file contains the definitions of macros that are useful to compute buffer sizes. The signatures and semantics of these macros are standardized, but the definitions are not, because they depend on the available algorithms and, in some cases, on permitted tolerances on buffer sizes.

In implementations with isolation between the application and the cryptography module, implementers should take care to ensure that the definitions that are exposed to applications match what the module implements.

Macros that compute sizes whose values do not depend on the implementation are in [crypto.h](#).

### 8.42.2 Macro Definition Documentation

#### 8.42.2.1 PSA\_AEAD\_DECRYPT\_OUTPUT\_MAX\_SIZE

```
#define PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE(
 ciphertext_length) (ciphertext_length)
```

A sufficient output buffer size for [psa\\_aead\\_decrypt\(\)](#), for any of the supported key types and AEAD algorithms.

If the size of the plaintext buffer is at least this large, it is guaranteed that [psa\\_aead\\_decrypt\(\)](#) will not fail due to an insufficient buffer size.

#### Note

This macro returns a compile-time constant if its arguments are compile-time constants.

See also [PSA\\_AEAD\\_DECRYPT\\_OUTPUT\\_SIZE\(key\\_type, alg, ciphertext\\_length\)](#).

#### Parameters

|                          |                                  |
|--------------------------|----------------------------------|
| <i>ciphertext_length</i> | Size of the ciphertext in bytes. |
|--------------------------|----------------------------------|

#### Returns

A sufficient output buffer size for any of the supported key types and AEAD algorithms.

Definition at line 422 of file [crypto\\_sizes.h](#).

### 8.42.2.2 PSA\_AEAD\_DECRYPT\_OUTPUT\_SIZE

```
#define PSA_AEAD_DECRYPT_OUTPUT_SIZE(
 key_type,
 alg,
 ciphertext_length)
```

#### Value:

```
(PSA_AEAD_NONCE_LENGTH(key_type, alg) != 0 &&
(ciphertext_length) > PSA_ALG_AEAD_GET_TAG_LENGTH(alg) ?
(ciphertext_length) - PSA_ALG_AEAD_GET_TAG_LENGTH(alg) :
0u)
```

The maximum size of the output of [psa\\_aead\\_decrypt\(\)](#), in bytes.

If the size of the plaintext buffer is at least this large, it is guaranteed that [psa\\_aead\\_decrypt\(\)](#) will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the plaintext may be smaller.

See also [PSA\\_AEAD\\_DECRYPT\\_OUTPUT\\_MAX\\_SIZE](#)(ciphertext\_length).

#### Warning

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

#### Parameters

|                          |                                                                                                         |
|--------------------------|---------------------------------------------------------------------------------------------------------|
| <i>key_type</i>          | A symmetric key type that is compatible with algorithm <i>alg</i> .                                     |
| <i>alg</i>               | An AEAD algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_AEAD</a> ( <i>alg</i> ) is true). |
| <i>ciphertext_length</i> | Size of the plaintext in bytes.                                                                         |

#### Returns

The AEAD ciphertext size for the specified algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 398 of file `crypto_sizes.h`.

### 8.42.2.3 PSA\_AEAD\_ENCRYPT\_OUTPUT\_MAX\_SIZE

```
#define PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE(
 plaintext_length) ((plaintext_length) + PSA_AEAD_TAG_MAX_SIZE)
```

A sufficient output buffer size for [psa\\_aead\\_encrypt\(\)](#), for any of the supported key types and AEAD algorithms.

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa\\_aead\\_encrypt\(\)](#) will not fail due to an insufficient buffer size.

#### Note

This macro returns a compile-time constant if its arguments are compile-time constants.

See also [PSA\\_AEAD\\_ENCRYPT\\_OUTPUT\\_SIZE](#)(key\_type, alg, plaintext\_length).

#### Parameters

|                         |                                 |
|-------------------------|---------------------------------|
| <i>plaintext_length</i> | Size of the plaintext in bytes. |
|-------------------------|---------------------------------|

#### Returns

A sufficient output buffer size for any of the supported key types and AEAD algorithms.

Definition at line 368 of file `crypto_sizes.h`.

#### 8.42.2.4 PSA\_AEAD\_ENCRYPT\_OUTPUT\_SIZE

```
#define PSA_AEAD_ENCRYPT_OUTPUT_SIZE(
 key_type,
 alg,
 plaintext_length)
```

##### Value:

```
(PSA_AEAD_NONCE_LENGTH(key_type, alg) != 0 ?
 (plaintext_length) + PSA_ALG_AEAD_GET_TAG_LENGTH(alg) :
 0u)
```

The maximum size of the output of [psa\\_aead\\_encrypt\(\)](#), in bytes.

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa\\_aead\\_encrypt\(\)](#) will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the ciphertext may be smaller.

See also [PSA\\_AEAD\\_ENCRYPT\\_OUTPUT\\_MAX\\_SIZE](#)(plaintext\_length).

##### Warning

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

##### Parameters

|                         |                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------|
| <i>key_type</i>         | A symmetric key type that is compatible with algorithm <i>alg</i> .                                     |
| <i>alg</i>              | An AEAD algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_AEAD</a> ( <i>alg</i> ) is true). |
| <i>plaintext_length</i> | Size of the plaintext in bytes.                                                                         |

##### Returns

The AEAD ciphertext size for the specified algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 345 of file `crypto_sizes.h`.

#### 8.42.2.5 PSA\_AEAD\_FINISH\_OUTPUT\_MAX\_SIZE

```
#define PSA_AEAD_FINISH_OUTPUT_MAX_SIZE (PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE)
```

A sufficient ciphertext buffer size for [psa\\_aead\\_finish\(\)](#), for any of the supported key types and AEAD algorithms.

See also [PSA\\_AEAD\\_FINISH\\_OUTPUT\\_SIZE](#)(key\_type, alg).

Definition at line 554 of file `crypto_sizes.h`.

#### 8.42.2.6 PSA\_AEAD\_FINISH\_OUTPUT\_SIZE

```
#define PSA_AEAD_FINISH_OUTPUT_SIZE(
 key_type,
 alg)
```

##### Value:

```
(PSA_AEAD_NONCE_LENGTH(key_type, alg) != 0 && \
 PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER(alg) ? \
 PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) : \
 0u)
```

A sufficient ciphertext buffer size for [psa\\_aead\\_finish\(\)](#).

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa\\_aead\\_finish\(\)](#) will not fail due to an insufficient ciphertext buffer size. The actual size of the output may be smaller in any given call.

See also [PSA\\_AEAD\\_FINISH\\_OUTPUT\\_MAX\\_SIZE](#).

##### Parameters

|                 |                                                                                                         |
|-----------------|---------------------------------------------------------------------------------------------------------|
| <i>key_type</i> | A symmetric key type that is compatible with algorithm <i>alg</i> .                                     |
| <i>alg</i>      | An AEAD algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_AEAD</a> ( <i>alg</i> ) is true). |



### Returns

A sufficient ciphertext buffer size for the specified algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 543 of file `crypto_sizes.h`.

#### 8.42.2.7 PSA\_AEAD\_NONCE\_LENGTH

```
#define PSA_AEAD_NONCE_LENGTH(
 key_type,
 alg)
```

### Value:

```
(PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) == 16 ? \
MBEDTLS_PSA_ALG_AEAD_EQUAL(alg, PSA_ALG_CCM) ? 13u : \
MBEDTLS_PSA_ALG_AEAD_EQUAL(alg, PSA_ALG_GCM) ? 12u : \
0u : \
(key_type) == PSA_KEY_TYPE_CHACHA20 && \
MBEDTLS_PSA_ALG_AEAD_EQUAL(alg, PSA_ALG_CHACHA20_POLY1305) ? 12u : \
0u)
```

The default nonce size for an AEAD algorithm, in bytes.

This macro can be used to allocate a buffer of sufficient size to store the nonce output from [psa\\_aead\\_generate\\_nonce\(\)](#).

See also [PSA\\_AEAD\\_NONCE\\_MAX\\_SIZE](#).

### Note

This is not the maximum size of nonce supported as input to [psa\\_aead\\_set\\_nonce\(\)](#), [psa\\_aead\\_encrypt\(\)](#) or [psa\\_aead\\_decrypt\(\)](#), just the default size that is generated by [psa\\_aead\\_generate\\_nonce\(\)](#).

### Warning

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

### Parameters

|                 |                                                                                                             |
|-----------------|-------------------------------------------------------------------------------------------------------------|
| <i>key_type</i> | A symmetric key type that is compatible with algorithm <i>alg</i> .                                         |
| <i>alg</i>      | An AEAD algorithm ( <code>PSA_ALG_XXX</code> value such that <a href="#">PSA_ALG_IS_AEAD(alg)</a> is true). |

### Returns

The default nonce size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 450 of file `crypto_sizes.h`.

#### 8.42.2.8 PSA\_AEAD\_NONCE\_MAX\_SIZE

```
#define PSA_AEAD_NONCE_MAX_SIZE 13u
```

The maximum default nonce size among all supported pairs of key types and AEAD algorithms, in bytes.

This is equal to or greater than any value that [PSA\\_AEAD\\_NONCE\\_LENGTH\(\)](#) may return.

### Note

This is not the maximum size of nonce supported as input to [psa\\_aead\\_set\\_nonce\(\)](#), [psa\\_aead\\_encrypt\(\)](#) or [psa\\_aead\\_decrypt\(\)](#), just the largest size that may be generated by [psa\\_aead\\_generate\\_nonce\(\)](#).

Definition at line 470 of file `crypto_sizes.h`.

### 8.42.2.9 PSA\_AEAD\_TAG\_LENGTH

```
#define PSA_AEAD_TAG_LENGTH(
 key_type,
 key_bits,
 alg)
```

#### Value:

```
(PSA_AEAD_NONCE_LENGTH(key_type, alg) != 0 ?
 PSA_ALG_AEAD_GET_TAG_LENGTH(alg) :
 ((void) (key_bits), 0u))
```

The length of a tag for an AEAD algorithm, in bytes.

This macro can be used to allocate a buffer of sufficient size to store the tag output from [psa\\_aead\\_finish\(\)](#).

See also [PSA\\_AEAD\\_TAG\\_MAX\\_SIZE](#).

#### Parameters

|                 |                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------|
| <i>key_type</i> | The type of the AEAD key.                                                                      |
| <i>key_bits</i> | The size of the AEAD key in bits.                                                              |
| <i>alg</i>      | An AEAD algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_AEAD</a> (alg) is true). |

#### Returns

The tag length for the specified algorithm and key. If the AEAD algorithm does not have an identified tag that can be distinguished from the rest of the ciphertext, return 0. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 181 of file `crypto_sizes.h`.

### 8.42.2.10 PSA\_AEAD\_TAG\_MAX\_SIZE

```
#define PSA_AEAD_TAG_MAX_SIZE 16u
```

The maximum tag size for all supported AEAD algorithms, in bytes.

See also [PSA\\_AEAD\\_TAG\\_LENGTH](#)(key\_type, key\_bits, alg).

Definition at line 190 of file `crypto_sizes.h`.

### 8.42.2.11 PSA\_AEAD\_UPDATE\_OUTPUT\_MAX\_SIZE

```
#define PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE(
 input_length) (PSA_ROUND_UP_TO_MULTIPLE(PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE, (input↔
_length)))
```

A sufficient output buffer size for [psa\\_aead\\_update\(\)](#), for any of the supported key types and AEAD algorithms.

If the size of the output buffer is at least this large, it is guaranteed that [psa\\_aead\\_update\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA\\_AEAD\\_UPDATE\\_OUTPUT\\_SIZE](#)(key\_type, alg, input\_length).

#### Parameters

|                     |                             |
|---------------------|-----------------------------|
| <i>input_length</i> | Size of the input in bytes. |
|---------------------|-----------------------------|

Definition at line 519 of file `crypto_sizes.h`.

### 8.42.2.12 PSA\_AEAD\_UPDATE\_OUTPUT\_SIZE

```
#define PSA_AEAD_UPDATE_OUTPUT_SIZE(
 key_type,
 alg,
```

```
input_length)
```

**Value:**

```
(PSA_AEAD_NONCE_LENGTH(key_type, alg) != 0 ?
 \
 PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER(alg) ?
 \
 PSA_ROUND_UP_TO_MULTIPLE(PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type), (input_length)) : \
 (input_length) : \
 0u)
```

A sufficient output buffer size for [psa\\_aead\\_update\(\)](#).

If the size of the output buffer is at least this large, it is guaranteed that [psa\\_aead\\_update\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output may be smaller in any given call.

See also [PSA\\_AEAD\\_UPDATE\\_OUTPUT\\_MAX\\_SIZE\(input\\_length\)](#).

**Warning**

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

**Parameters**

|                     |                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------|
| <i>key_type</i>     | A symmetric key type that is compatible with algorithm <i>alg</i> .                           |
| <i>alg</i>          | An AEAD algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_AEAD(alg)</a> is true). |
| <i>input_length</i> | Size of the input in bytes.                                                                   |

**Returns**

A sufficient output buffer size for the specified algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 502 of file `crypto_sizes.h`.

**8.42.2.13 PSA\_AEAD\_VERIFY\_OUTPUT\_MAX\_SIZE**

```
#define PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE (PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE)
```

A sufficient plaintext buffer size for [psa\\_aead\\_verify\(\)](#), for any of the supported key types and AEAD algorithms.

See also [PSA\\_AEAD\\_VERIFY\\_OUTPUT\\_SIZE\(key\\_type, alg\)](#).

Definition at line 588 of file `crypto_sizes.h`.

**8.42.2.14 PSA\_AEAD\_VERIFY\_OUTPUT\_SIZE**

```
#define PSA_AEAD_VERIFY_OUTPUT_SIZE(
 key_type,
 alg)
```

**Value:**

```
(PSA_AEAD_NONCE_LENGTH(key_type, alg) != 0 && \
 PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER(alg) ? \
 PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) : \
 0u)
```

A sufficient plaintext buffer size for [psa\\_aead\\_verify\(\)](#).

If the size of the plaintext buffer is at least this large, it is guaranteed that [psa\\_aead\\_verify\(\)](#) will not fail due to an insufficient plaintext buffer size. The actual size of the output may be smaller in any given call.

See also [PSA\\_AEAD\\_VERIFY\\_OUTPUT\\_MAX\\_SIZE](#).

**Parameters**

|                 |                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------|
| <i>key_type</i> | A symmetric key type that is compatible with algorithm <i>alg</i> .                           |
| <i>alg</i>      | An AEAD algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_AEAD(alg)</a> is true). |

**Returns**

A sufficient plaintext buffer size for the specified algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 577 of file `crypto_sizes.h`.

**8.42.2.15 PSA\_ASYMMETRIC\_DECRYPT\_OUTPUT\_MAX\_SIZE**

```
#define PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE (PSA_BITS_TO_BYTES(PSA_VENDOR_RSA_MAX_KEY_BITS))
```

A sufficient output buffer size for `psa_asymmetric_decrypt()`, for any supported asymmetric decryption.

This macro assumes that RSA is the only supported asymmetric encryption.

See also `PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE(key_type, key_bits, alg)`.

Definition at line 735 of file `crypto_sizes.h`.

**8.42.2.16 PSA\_ASYMMETRIC\_DECRYPT\_OUTPUT\_SIZE**

```
#define PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE(
 key_type,
 key_bits,
 alg)
```

**Value:**

```
(PSA_KEY_TYPE_IS_RSA(key_type) ?
 \
 PSA_BITS_TO_BYTES(key_bits) - PSA_RSA_MINIMUM_PADDING_SIZE(alg) :
 0u)
```

Sufficient output buffer size for `psa_asymmetric_decrypt()`.

This macro returns a sufficient buffer size for a plaintext produced using a key of the specified type and size, with the specified algorithm. Note that the actual size of the plaintext may be smaller, depending on the algorithm.

**Warning**

This function may call its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

**Parameters**

|                 |                                                                                          |
|-----------------|------------------------------------------------------------------------------------------|
| <i>key_type</i> | An asymmetric key type (this may indifferently be a key pair type or a public key type). |
| <i>key_bits</i> | The size of the key in bits.                                                             |
| <i>alg</i>      | The asymmetric encryption algorithm.                                                     |

**Returns**

If the parameters are valid and supported, return a buffer size in bytes that guarantees that `psa_asymmetric_decrypt()` will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported, return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

Definition at line 723 of file `crypto_sizes.h`.

**8.42.2.17 PSA\_ASYMMETRIC\_ENCRYPT\_OUTPUT\_MAX\_SIZE**

```
#define PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE (PSA_BITS_TO_BYTES(PSA_VENDOR_RSA_MAX_KEY_BITS))
```

A sufficient output buffer size for `psa_asymmetric_encrypt()`, for any supported asymmetric encryption.

See also `PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE(key_type, key_bits, alg)`.

Definition at line 695 of file `crypto_sizes.h`.

**8.42.2.18 PSA\_ASYMMETRIC\_ENCRYPT\_OUTPUT\_SIZE**

```
#define PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE(
 key_type,
 key_bits,
 alg)
```

**Value:**

```
(PSA_KEY_TYPE_IS_RSA(key_type) ?
 \
 ((void) alg, PSA_BITS_TO_BYTES(key_bits)) :
 0u) \
```

Sufficient output buffer size for [psa\\_asymmetric\\_encrypt\(\)](#).

This macro returns a sufficient buffer size for a ciphertext produced using a key of the specified type and size, with the specified algorithm. Note that the actual size of the ciphertext may be smaller, depending on the algorithm.

**Warning**

This function may call its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

**Parameters**

|                 |                                                                                          |
|-----------------|------------------------------------------------------------------------------------------|
| <i>key_type</i> | An asymmetric key type (this may indifferently be a key pair type or a public key type). |
| <i>key_bits</i> | The size of the key in bits.                                                             |
| <i>alg</i>      | The asymmetric encryption algorithm.                                                     |

**Returns**

If the parameters are valid and supported, return a buffer size in bytes that guarantees that [psa\\_asymmetric\\_encrypt\(\)](#) will not fail with [PSA\\_ERROR\\_BUFFER\\_TOO\\_SMALL](#). If the parameters are a valid combination that is not supported, return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

Definition at line 684 of file `crypto_sizes.h`.

**8.42.2.19 PSA\_BITS\_TO\_BYTES**

```
#define PSA_BITS_TO_BYTES(
 bits) (((bits) + 7u) / 8u)
```

Definition at line 33 of file `crypto_sizes.h`.

**8.42.2.20 PSA\_BLOCK\_CIPHER\_BLOCK\_MAX\_SIZE**

```
#define PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE 16u
```

The maximum size of a block cipher.

Definition at line 290 of file `crypto_sizes.h`.

**8.42.2.21 PSA\_BYTES\_TO\_BITS**

```
#define PSA_BYTES_TO_BITS(
 bytes) ((bytes) * 8u)
```

Definition at line 34 of file `crypto_sizes.h`.

**8.42.2.22 PSA\_CIPHER\_DECRYPT\_OUTPUT\_MAX\_SIZE**

```
#define PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE(
 input_length) (input_length)
```

A sufficient output buffer size for [psa\\_cipher\\_decrypt\(\)](#), for any of the supported key types and cipher algorithms. If the size of the output buffer is at least this large, it is guaranteed that [psa\\_cipher\\_decrypt\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA\\_CIPHER\\_DECRYPT\\_OUTPUT\\_SIZE](#)(key\_type, alg, input\_length).

#### Parameters

|                     |                             |
|---------------------|-----------------------------|
| <i>input_length</i> | Size of the input in bytes. |
|---------------------|-----------------------------|

Definition at line 1224 of file crypto\_sizes.h.

#### 8.42.2.23 PSA\_CIPHER\_DECRYPT\_OUTPUT\_SIZE

```
#define PSA_CIPHER_DECRYPT_OUTPUT_SIZE(
 key_type,
 alg,
 input_length)
```

#### Value:

```
(PSA_ALG_IS_CIPHER(alg) &&
 ((key_type) & PSA_KEY_TYPE_CATEGORY_MASK) == PSA_KEY_TYPE_CATEGORY_SYMMETRIC ? \
 (input_length) : \
 0u)
```

The maximum size of the output of [psa\\_cipher\\_decrypt\(\)](#), in bytes.

If the size of the output buffer is at least this large, it is guaranteed that [psa\\_cipher\\_decrypt\(\)](#) will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the output might be smaller.

See also [PSA\\_CIPHER\\_DECRYPT\\_OUTPUT\\_MAX\\_SIZE](#)(input\_length).

#### Parameters

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <i>key_type</i>     | A symmetric key type that is compatible with algorithm alg.                                       |
| <i>alg</i>          | A cipher algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_CIPHER</a> (alg) is true). |
| <i>input_length</i> | Size of the input in bytes.                                                                       |

#### Returns

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 1208 of file crypto\_sizes.h.

#### 8.42.2.24 PSA\_CIPHER\_ENCRYPT\_OUTPUT\_MAX\_SIZE

```
#define PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE(
 input_length)
```

#### Value:

```
(PSA_ROUND_UP_TO_MULTIPLE(PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE,
 (input_length) + 1u) +
 PSA_CIPHER_IV_MAX_SIZE)
```

A sufficient output buffer size for [psa\\_cipher\\_encrypt\(\)](#), for any of the supported key types and cipher algorithms.

If the size of the output buffer is at least this large, it is guaranteed that [psa\\_cipher\\_encrypt\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA\\_CIPHER\\_ENCRYPT\\_OUTPUT\\_SIZE](#)(key\_type, alg, input\_length).

#### Parameters

|                     |                             |
|---------------------|-----------------------------|
| <i>input_length</i> | Size of the input in bytes. |
|---------------------|-----------------------------|

Definition at line 1184 of file crypto\_sizes.h.

### 8.42.2.25 PSA\_CIPHER\_ENCRYPT\_OUTPUT\_SIZE

```
#define PSA_CIPHER_ENCRYPT_OUTPUT_SIZE(
 key_type,
 alg,
 input_length)
```

**Value:**

```
(alg == PSA_ALG_CBC_PKCS7 ?
 \
 (PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) != 0 ?
 PSA_ROUND_UP_TO_MULTIPLE(PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type), \
 (input_length) + 1u) +
 PSA_CIPHER_IV_LENGTH((key_type), (alg)) : 0u) :
 (PSA_ALG_IS_CIPHER(alg) ?
 \
 (input_length) + PSA_CIPHER_IV_LENGTH((key_type), (alg)) :
 0u))
```

The maximum size of the output of [psa\\_cipher\\_encrypt\(\)](#), in bytes.

If the size of the output buffer is at least this large, it is guaranteed that [psa\\_cipher\\_encrypt\(\)](#) will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the output might be smaller.

See also [PSA\\_CIPHER\\_ENCRYPT\\_OUTPUT\\_MAX\\_SIZE](#)(input\_length).

#### Warning

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

#### Parameters

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <i>key_type</i>     | A symmetric key type that is compatible with algorithm alg.                                       |
| <i>alg</i>          | A cipher algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_CIPHER</a> (alg) is true). |
| <i>input_length</i> | Size of the input in bytes.                                                                       |

#### Returns

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 1163 of file crypto\_sizes.h.

### 8.42.2.26 PSA\_CIPHER\_FINISH\_OUTPUT\_MAX\_SIZE

```
#define PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE (PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE)
```

A sufficient ciphertext buffer size for [psa\\_cipher\\_finish\(\)](#), for any of the supported key types and cipher algorithms.

See also [PSA\\_CIPHER\\_FINISH\\_OUTPUT\\_SIZE](#)(key\_type, alg).

Definition at line 1298 of file crypto\_sizes.h.

### 8.42.2.27 PSA\_CIPHER\_FINISH\_OUTPUT\_SIZE

```
#define PSA_CIPHER_FINISH_OUTPUT_SIZE(
 key_type,
 alg)
```

**Value:**

```
(PSA_ALG_IS_CIPHER(alg) ?
 (alg == PSA_ALG_CBC_PKCS7 ?
 PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) :
 0u) :
 0u)
```

A sufficient ciphertext buffer size for [psa\\_cipher\\_finish\(\)](#).

If the size of the ciphertext buffer is at least this large, it is guaranteed that [psa\\_cipher\\_finish\(\)](#) will not fail due to an insufficient ciphertext buffer size. The actual size of the output might be smaller in any given call. See also [PSA\\_CIPHER\\_FINISH\\_OUTPUT\\_MAX\\_SIZE\(\)](#).

#### Parameters

|                 |                                                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------|
| <i>key_type</i> | A symmetric key type that is compatible with algorithm <i>alg</i> .                              |
| <i>alg</i>      | A cipher algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_CIPHER(alg)</a> is true). |

#### Returns

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 1286 of file `crypto_sizes.h`.

### 8.42.2.28 PSA\_CIPHER\_IV\_LENGTH

```
#define PSA_CIPHER_IV_LENGTH(
 key_type,
 alg)
```

#### Value:

```
(PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) > 1 && \
(alg) == PSA_ALG_CTR || \
(alg) == PSA_ALG_CFB || \
(alg) == PSA_ALG_OFB || \
(alg) == PSA_ALG_XTS || \
(alg) == PSA_ALG_CBC_NO_PADDING || \
(alg) == PSA_ALG_CBC_PKCS7) ? PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) : \
(key_type) == PSA_KEY_TYPE_CHACHA20 && \
(alg) == PSA_ALG_STREAM_CIPHER ? 12u : \
(alg) == PSA_ALG_CCM_STAR_NO_TAG ? 13u : \
0u)
```

The default IV size for a cipher algorithm, in bytes.

The IV that is generated as part of a call to [psa\\_cipher\\_encrypt\(\)](#) is always the default IV length for the algorithm.

This macro can be used to allocate a buffer of sufficient size to store the IV output from [psa\\_cipher\\_generate\\_iv\(\)](#) when using a multi-part cipher operation.

See also [PSA\\_CIPHER\\_IV\\_MAX\\_SIZE](#).

#### Warning

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

#### Parameters

|                 |                                                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------|
| <i>key_type</i> | A symmetric key type that is compatible with algorithm <i>alg</i> .                              |
| <i>alg</i>      | A cipher algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_CIPHER(alg)</a> is true). |

#### Returns

The default IV size for the specified key type and algorithm. If the algorithm does not use an IV, return 0. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 1121 of file `crypto_sizes.h`.

### 8.42.2.29 PSA\_CIPHER\_IV\_MAX\_SIZE

```
#define PSA_CIPHER_IV_MAX_SIZE 16u
```

The maximum IV size for all supported cipher algorithms, in bytes.



See also [PSA\\_CIPHER\\_IV\\_LENGTH\(\)](#).

Definition at line 1138 of file `crypto_sizes.h`.

### 8.42.2.30 PSA\_CIPHER\_MAX\_KEY\_LENGTH

```
#define PSA_CIPHER_MAX_KEY_LENGTH 0u
```

Maximum key length for ciphers.

Since there is no additional `PSA_WANT_xxx` symbol to specify the size of the key once a cipher is enabled (as it happens for asymmetric keys for example), the maximum key length is taken into account for each cipher. The resulting value will be the maximum cipher's key length given depending on which ciphers are enabled.

Note: max value for AES used below would be doubled if XTS were enabled, but this mode is currently not supported in Mbed TLS implementation of PSA APIs.

Definition at line 1094 of file `crypto_sizes.h`.

### 8.42.2.31 PSA\_CIPHER\_UPDATE\_OUTPUT\_MAX\_SIZE

```
#define PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE(
 input_length) (PSA_ROUND_UP_TO_MULTIPLE(PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE, input_↵
 _length))
```

A sufficient output buffer size for [psa\\_cipher\\_update\(\)](#), for any of the supported key types and cipher algorithms.

If the size of the output buffer is at least this large, it is guaranteed that [psa\\_cipher\\_update\(\)](#) will not fail due to an insufficient buffer size.

See also [PSA\\_CIPHER\\_UPDATE\\_OUTPUT\\_SIZE](#)(key\_type, alg, input\_length).

#### Parameters

|                     |                             |
|---------------------|-----------------------------|
| <i>input_length</i> | Size of the input in bytes. |
|---------------------|-----------------------------|

Definition at line 1266 of file `crypto_sizes.h`.

### 8.42.2.32 PSA\_CIPHER\_UPDATE\_OUTPUT\_SIZE

```
#define PSA_CIPHER_UPDATE_OUTPUT_SIZE(
 key_type,
 alg,
 input_length)
```

#### Value:

```
(PSA_ALG_IS_CIPHER(alg) ?
 \
 (PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) != 0 ?
 \
 ((alg) == PSA_ALG_CBC_PKCS7 ||
 \
 (alg) == PSA_ALG_CBC_NO_PADDING ||
 \
 (alg) == PSA_ALG_ECB_NO_PADDING) ?
 \
 PSA_ROUND_UP_TO_MULTIPLE(PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type), \
 \
 input_length) :
 \
 (input_length)) : 0u) :
```

A sufficient output buffer size for [psa\\_cipher\\_update\(\)](#).

If the size of the output buffer is at least this large, it is guaranteed that [psa\\_cipher\\_update\(\)](#) will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also [PSA\\_CIPHER\\_UPDATE\\_OUTPUT\\_MAX\\_SIZE](#)(input\_length).

#### Parameters

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <i>key_type</i>     | A symmetric key type that is compatible with algorithm alg.                                       |
| <i>alg</i>          | A cipher algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_CIPHER</a> (alg) is true). |
| <i>input_length</i> | Size of the input in bytes.                                                                       |

**Returns**

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0.

Definition at line 1245 of file `crypto_sizes.h`.

**8.42.2.33 PSA\_ECDSA\_SIGNATURE\_SIZE**

```
#define PSA_ECDSA_SIGNATURE_SIZE(
 curve_bits) (PSA_BITS_TO_BYTES(curve_bits) * 2u)
```

ECDSA signature size for a given curve bit size.

**Parameters**

|                   |                     |
|-------------------|---------------------|
| <i>curve_bits</i> | Curve size in bits. |
|-------------------|---------------------|

**Returns**

Signature size in bytes.

**Note**

This macro returns a compile-time constant if its argument is one.

Definition at line 603 of file `crypto_sizes.h`.

**8.42.2.34 PSA\_EXPORT\_ASYMMETRIC\_KEY\_MAX\_SIZE**

```
#define PSA_EXPORT_ASYMMETRIC_KEY_MAX_SIZE
```

**Value:**

```
((PSA_EXPORT_KEY_PAIR_MAX_SIZE > PSA_EXPORT_PUBLIC_KEY_MAX_SIZE) ? \
 PSA_EXPORT_KEY_PAIR_MAX_SIZE : PSA_EXPORT_PUBLIC_KEY_MAX_SIZE)
```

Definition at line 1027 of file `crypto_sizes.h`.

**8.42.2.35 PSA\_EXPORT\_KEY\_OUTPUT\_SIZE**

```
#define PSA_EXPORT_KEY_OUTPUT_SIZE(
 key_type,
 key_bits)
```

**Value:**

```
((key_type) == PSA_KEY_TYPE_RSA_KEY_PAIR ? PSA_KEY_EXPORT_RSA_KEY_PAIR_MAX_SIZE(key_bits) : \
 (key_type) == PSA_KEY_TYPE_RSA_PUBLIC_KEY ? PSA_KEY_EXPORT_RSA_PUBLIC_KEY_MAX_SIZE(key_bits) : \
 PSA_KEY_TYPE_IS_ECC_KEY_PAIR(key_type) ? PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE(key_bits) : \
 PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY(key_type) ? PSA_KEY_EXPORT_ECC_PUBLIC_KEY_MAX_SIZE(key_bits) : \
 PSA_BITS_TO_BYTES(key_bits)) /*unstructured; FFDH public or private*/
```

Sufficient output buffer size for `psa_export_key()` or `psa_export_public_key()`.

This macro returns a compile-time constant if its arguments are compile-time constants.

**Warning**

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

The following code illustrates how to allocate enough memory to export a key by querying the key type and size at runtime.

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
psa_status_t status;
status = psa_get_key_attributes(key, &attributes);
if (status != PSA_SUCCESS) handle_error(...);
psa_key_type_t key_type = psa_get_key_type(&attributes);
size_t key_bits = psa_get_key_bits(&attributes);
size_t buffer_size = PSA_EXPORT_KEY_OUTPUT_SIZE(key_type, key_bits);
```

```

psa_reset_key_attributes(&attributes);
uint8_t *buffer = malloc(buffer_size);
if (buffer == NULL) handle_error(...);
size_t buffer_length;
status = psa_export_key(key, buffer, buffer_size, &buffer_length);
if (status != PSA_SUCCESS) handle_error(...);

```

#### Parameters

|                 |                              |
|-----------------|------------------------------|
| <i>key_type</i> | A supported key type.        |
| <i>key_bits</i> | The size of the key in bits. |

#### Returns

If the parameters are valid and supported, return a buffer size in bytes that guarantees that [psa\\_export\\_key\(\)](#) or [psa\\_export\\_public\\_key\(\)](#) will not fail with [PSA\\_ERROR\\_BUFFER\\_TOO\\_SMALL](#). If the parameters are a valid combination that is not supported, return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

Definition at line 903 of file `crypto_sizes.h`.

#### 8.42.2.36 PSA\_EXPORT\_KEY\_PAIR\_MAX\_SIZE

```
#define PSA_EXPORT_KEY_PAIR_MAX_SIZE 1
```

Sufficient buffer size for exporting any asymmetric key pair.

This macro expands to a compile-time constant integer. This value is a sufficient buffer size when calling [psa\\_export\\_key\(\)](#) to export any asymmetric key pair, regardless of the exact key type and key size.

See also [PSA\\_EXPORT\\_KEY\\_OUTPUT\\_SIZE\(key\\_type, key\\_bits\)](#).

Definition at line 969 of file `crypto_sizes.h`.

#### 8.42.2.37 PSA\_EXPORT\_PUBLIC\_KEY\_MAX\_SIZE

```
#define PSA_EXPORT_PUBLIC_KEY_MAX_SIZE 1
```

Sufficient buffer size for exporting any asymmetric public key.

This macro expands to a compile-time constant integer. This value is a sufficient buffer size when calling [psa\\_export\\_key\(\)](#) or [psa\\_export\\_public\\_key\(\)](#) to export any asymmetric public key, regardless of the exact key type and key size.

See also [PSA\\_EXPORT\\_PUBLIC\\_KEY\\_OUTPUT\\_SIZE\(key\\_type, key\\_bits\)](#).

Definition at line 1002 of file `crypto_sizes.h`.

#### 8.42.2.38 PSA\_EXPORT\_PUBLIC\_KEY\_OUTPUT\_SIZE

```

#define PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE(
 key_type,
 key_bits)

```

##### Value:

```

(PSA_KEY_TYPE_IS_RSA(key_type) ? PSA_KEY_EXPORT_RSA_PUBLIC_KEY_MAX_SIZE(key_bits) : \
 PSA_KEY_TYPE_IS_ECC(key_type) ? PSA_KEY_EXPORT_ECC_PUBLIC_KEY_MAX_SIZE(key_bits) : \
 PSA_KEY_TYPE_IS_DH(key_type) ? PSA_BITS_TO_BYTES(key_bits) : \
 0u)

```

Sufficient output buffer size for [psa\\_export\\_public\\_key\(\)](#).

This macro returns a compile-time constant if its arguments are compile-time constants.

#### Warning

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

The following code illustrates how to allocate enough memory to export a public key by querying the key type and size at runtime.

```

psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
psa_status_t status;
status = psa_get_key_attributes(key, &attributes);
if (status != PSA_SUCCESS) handle_error(...);
psa_key_type_t key_type = psa_get_key_type(&attributes);
size_t key_bits = psa_get_key_bits(&attributes);
size_t buffer_size = PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE(key_type, key_bits);
psa_reset_key_attributes(&attributes);
uint8_t *buffer = malloc(buffer_size);
if (buffer == NULL) handle_error(...);
size_t buffer_length;
status = psa_export_public_key(key, buffer, buffer_size, &buffer_length);
if (status != PSA_SUCCESS) handle_error(...);

```

#### Parameters

|                 |                                    |
|-----------------|------------------------------------|
| <i>key_type</i> | A public key or key pair key type. |
| <i>key_bits</i> | The size of the key in bits.       |

#### Returns

If the parameters are valid and supported, return a buffer size in bytes that guarantees that `psa_export_public_key()` will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported, return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

If the parameters are valid and supported, return the same result as `PSA_EXPORT_KEY_OUTPUT_SIZE(PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(key_type), key_bits)`.

Definition at line 955 of file `crypto_sizes.h`.

### 8.42.2.39 PSA\_HASH\_BLOCK\_LENGTH

```

#define PSA_HASH_BLOCK_LENGTH(
 alg)

```

#### Value:

```

(
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_MD5 ? 64u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_RIPEMD160 ? 64u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_1 ? 64u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_224 ? 64u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_256 ? 64u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_384 ? 128u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_512 ? 128u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_512_224 ? 128u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_512_256 ? 128u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA3_224 ? 144u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA3_256 ? 136u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA3_384 ? 104u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA3_512 ? 72u :
 0u)

```

The input block size of a hash algorithm, in bytes.

Hash algorithms process their input data in blocks. Hash operations will retain any partial blocks until they have enough input to fill the block or until the operation is finished. This affects the output from `psa_hash_suspend()`.

#### Parameters

|            |                                                                                           |
|------------|-------------------------------------------------------------------------------------------|
| <i>alg</i> | A hash algorithm (PSA_ALG_XXX value such that <code>PSA_ALG_IS_HASH(alg)</code> is true). |
|------------|-------------------------------------------------------------------------------------------|

#### Returns

The block size in bytes for the specified hash algorithm. If the hash algorithm is not recognized, return 0. An implementation can return either 0 or the correct size for a hash algorithm that it recognizes, but does not support.

Definition at line 85 of file `crypto_sizes.h`.

### 8.42.2.40 PSA\_HASH\_LENGTH

```
#define PSA_HASH_LENGTH(
 alg)
```

Value:

```
(
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_MD5 ? 16u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_RIPEMD160 ? 20u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_1 ? 20u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_224 ? 28u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_256 ? 32u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_384 ? 48u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_512 ? 64u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_512_224 ? 28u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA_512_256 ? 32u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA3_224 ? 28u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA3_256 ? 32u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA3_384 ? 48u :
 PSA_ALG_HMAC_GET_HASH(alg) == PSA_ALG_SHA3_512 ? 64u :
 0u)
```

The size of the output of [psa\\_hash\\_finish\(\)](#), in bytes.

This is also the hash size that [psa\\_hash\\_verify\(\)](#) expects.

#### Parameters

|            |                                                                                                                                                                                                   |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>alg</i> | A hash algorithm (PSA_ALG_XXX value such that <a href="#">PSA_ALG_IS_HASH</a> (alg) is true), or an HMAC algorithm ( <a href="#">PSA_ALG_HMAC</a> (hash_alg) where hash_alg is a hash algorithm). |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Returns

The hash size for the specified hash algorithm. If the hash algorithm is not recognized, return 0.

Definition at line 53 of file `crypto_sizes.h`.

### 8.42.2.41 PSA\_HASH\_MAX\_SIZE

```
#define PSA_HASH_MAX_SIZE 20u
```

Maximum size of a hash.

This macro expands to a compile-time constant integer. This value is the maximum size of a hash in bytes.

Definition at line 143 of file `crypto_sizes.h`.

### 8.42.2.42 PSA\_HMAC\_MAX\_HASH\_BLOCK\_SIZE

```
#define PSA_HMAC_MAX_HASH_BLOCK_SIZE 64u
```

Definition at line 131 of file `crypto_sizes.h`.

### 8.42.2.43 PSA\_KEY\_EXPORT\_ASN1\_INTEGER\_MAX\_SIZE

```
#define PSA_KEY_EXPORT_ASN1_INTEGER_MAX_SIZE(
 bits) ((bits) / 8u + 5u)
```

Definition at line 752 of file `crypto_sizes.h`.

### 8.42.2.44 PSA\_KEY\_EXPORT\_DSA\_KEY\_PAIR\_MAX\_SIZE

```
#define PSA_KEY_EXPORT_DSA_KEY_PAIR_MAX_SIZE(
 key_bits) (PSA_KEY_EXPORT_ASN1_INTEGER_MAX_SIZE(key_bits) * 3u + 75u)
```

Definition at line 829 of file `crypto_sizes.h`.

**8.42.2.45 PSA\_KEY\_EXPORT\_DSA\_PUBLIC\_KEY\_MAX\_SIZE**

```
#define PSA_KEY_EXPORT_DSA_PUBLIC_KEY_MAX_SIZE(
 key_bits) (PSA_KEY_EXPORT_ASN1_INTEGER_MAX_SIZE(key_bits) * 3u + 59u)
```

Definition at line 810 of file `crypto_sizes.h`.

**8.42.2.46 PSA\_KEY\_EXPORT\_ECC\_KEY\_PAIR\_MAX\_SIZE**

```
#define PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE(
 key_bits) (PSA_BITS_TO_BYTES(key_bits))
```

Definition at line 849 of file `crypto_sizes.h`.

**8.42.2.47 PSA\_KEY\_EXPORT\_ECC\_PUBLIC\_KEY\_MAX\_SIZE**

```
#define PSA_KEY_EXPORT_ECC_PUBLIC_KEY_MAX_SIZE(
 key_bits) (2u * PSA_BITS_TO_BYTES(key_bits) + 1u)
```

Definition at line 842 of file `crypto_sizes.h`.

**8.42.2.48 PSA\_KEY\_EXPORT\_FFDH\_KEY\_PAIR\_MAX\_SIZE**

```
#define PSA_KEY_EXPORT_FFDH_KEY_PAIR_MAX_SIZE(
 key_bits) (PSA_BITS_TO_BYTES(key_bits))
```

Definition at line 856 of file `crypto_sizes.h`.

**8.42.2.49 PSA\_KEY\_EXPORT\_FFDH\_PUBLIC\_KEY\_MAX\_SIZE**

```
#define PSA_KEY_EXPORT_FFDH_PUBLIC_KEY_MAX_SIZE(
 key_bits) (PSA_BITS_TO_BYTES(key_bits))
```

Definition at line 861 of file `crypto_sizes.h`.

**8.42.2.50 PSA\_KEY\_EXPORT\_RSA\_KEY\_PAIR\_MAX\_SIZE**

```
#define PSA_KEY_EXPORT_RSA_KEY_PAIR_MAX_SIZE(
 key_bits) (9u * PSA_KEY_EXPORT_ASN1_INTEGER_MAX_SIZE((key_bits) / 2u + 1u) +
 14u)
```

Definition at line 791 of file `crypto_sizes.h`.

**8.42.2.51 PSA\_KEY\_EXPORT\_RSA\_PUBLIC\_KEY\_MAX\_SIZE**

```
#define PSA_KEY_EXPORT_RSA_PUBLIC_KEY_MAX_SIZE(
 key_bits) (PSA_KEY_EXPORT_ASN1_INTEGER_MAX_SIZE(key_bits) + 11u)
```

Definition at line 766 of file `crypto_sizes.h`.

**8.42.2.52 PSA\_MAC\_LENGTH**

```
#define PSA_MAC_LENGTH(
 key_type,
 key_bits,
 alg)
```

**Value:**

```
((alg) & PSA_ALG_MAC_TRUNCATION_MASK) ? PSA_MAC_TRUNCATED_LENGTH(alg) :
PSA_ALG_IS_HMAC(alg) ? PSA_HASH_LENGTH(PSA_ALG_HMAC_GET_HASH(alg)) :
PSA_ALG_IS_BLOCK_CIPHER_MAC(alg) ? PSA_BLOCK_CIPHER_BLOCK_LENGTH(key_type) :
((void) (key_type), (void) (key_bits), 0u)
```

The size of the output of [psa\\_mac\\_sign\\_finish\(\)](#), in bytes.  
This is also the MAC size that [psa\\_mac\\_verify\\_finish\(\)](#) expects.

#### Warning

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

#### Parameters

|                 |                                                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| <i>key_type</i> | The type of the MAC key.                                                                                           |
| <i>key_bits</i> | The size of the MAC key in bits.                                                                                   |
| <i>alg</i>      | A MAC algorithm ( <code>PSA_ALG_XXX</code> value such that <a href="#">PSA_ALG_IS_MAC</a> ( <i>alg</i> ) is true). |

#### Returns

The MAC size for the specified algorithm with the specified key parameters.

0 if the MAC algorithm is not recognized.

Either 0 or the correct size for a MAC algorithm that the implementation recognizes, but does not support.

Unspecified if the key parameters are not consistent with the algorithm.

Definition at line 313 of file `crypto_sizes.h`.

#### 8.42.2.53 PSA\_MAC\_MAX\_SIZE

```
#define PSA_MAC_MAX_SIZE PSA_HASH_MAX_SIZE
```

Maximum size of a MAC.

This macro expands to a compile-time constant integer. This value is the maximum size of a MAC in bytes.

Definition at line 158 of file `crypto_sizes.h`.

#### 8.42.2.54 PSA\_MAX\_OF\_THREE

```
#define PSA_MAX_OF_THREE(
 a,
 b,
 c)
```

**Value:**

```
((a) <= (b) ? (b) <= (c) ? \
(c) : (b) : (a) <= (c) ? (c) : (a))
```

Definition at line 35 of file `crypto_sizes.h`.

#### 8.42.2.55 PSA\_RAW\_KEY\_AGREEMENT\_OUTPUT\_MAX\_SIZE

```
#define PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE 1
```

Maximum size of the output from [psa\\_raw\\_key\\_agreement\(\)](#).

This macro expands to a compile-time constant integer. This value is the maximum size of the output any raw key agreement algorithm, in bytes.

See also [PSA\\_RAW\\_KEY\\_AGREEMENT\\_OUTPUT\\_SIZE](#)(*key\_type*, *key\_bits*).

Definition at line 1065 of file `crypto_sizes.h`.

#### 8.42.2.56 PSA\_RAW\_KEY\_AGREEMENT\_OUTPUT\_SIZE

```
#define PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE(
 key_type,
 key_bits)
```

**Value:**

```
((PSA_KEY_TYPE_IS_ECC_KEY_PAIR(key_type) || \
 PSA_KEY_TYPE_IS_DH_KEY_PAIR(key_type)) ? PSA_BITS_TO_BYTES(key_bits) : 0u)
```

Sufficient output buffer size for [psa\\_raw\\_key\\_agreement\(\)](#).

This macro returns a compile-time constant if its arguments are compile-time constants.

**Warning**

This macro may evaluate its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

See also [PSA\\_RAW\\_KEY\\_AGREEMENT\\_OUTPUT\\_MAX\\_SIZE](#).

**Parameters**

|                 |                              |
|-----------------|------------------------------|
| <i>key_type</i> | A supported key type.        |
| <i>key_bits</i> | The size of the key in bits. |

**Returns**

If the parameters are valid and supported, return a buffer size in bytes that guarantees that [psa\\_raw\\_key\\_agreement\(\)](#) will not fail with [PSA\\_ERROR\\_BUFFER\\_TOO\\_SMALL](#). If the parameters are a valid combination that is not supported, return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

Definition at line 1054 of file `crypto_sizes.h`.

**8.42.2.57 PSA\_ROUND\_UP\_TO\_MULTIPLE**

```
#define PSA_ROUND_UP_TO_MULTIPLE(
 block_size,
 length) (((length) + (block_size) - 1) / (block_size) * (block_size))
```

Definition at line 38 of file `crypto_sizes.h`.

**8.42.2.58 PSA\_RSA\_MINIMUM\_PADDING\_SIZE**

```
#define PSA_RSA_MINIMUM_PADDING_SIZE(
 alg)
```

**Value:**

```
(PSA_ALG_IS_RSA_OAEP(alg) ?
 2u * PSA_HASH_LENGTH(PSA_ALG_RSA_OAEP_GET_HASH(alg)) + 1u :
 11u /*PKCS#1v1.5*/) \
```

Definition at line 590 of file `crypto_sizes.h`.

**8.42.2.59 PSA\_SIGN\_OUTPUT\_SIZE**

```
#define PSA_SIGN_OUTPUT_SIZE(
 key_type,
 key_bits,
 alg)
```

**Value:**

```
(PSA_KEY_TYPE_IS_RSA(key_type) ? ((void) alg, PSA_BITS_TO_BYTES(key_bits)) : \
 PSA_KEY_TYPE_IS_ECC(key_type) ? PSA_ECDSA_SIGNATURE_SIZE(key_bits) : \
 ((void) alg, 0u))
```

Sufficient signature buffer size for [psa\\_sign\\_hash\(\)](#).

This macro returns a sufficient buffer size for a signature using a key of the specified type and size, with the specified algorithm. Note that the actual size of the signature may be smaller (some algorithms produce a variable-size signature).



### Warning

This function may call its arguments multiple times or zero times, so you should not pass arguments that contain side effects.

### Parameters

|                 |                                                                                          |
|-----------------|------------------------------------------------------------------------------------------|
| <i>key_type</i> | An asymmetric key type (this may indifferently be a key pair type or a public key type). |
| <i>key_bits</i> | The size of the key in bits.                                                             |
| <i>alg</i>      | The signature algorithm.                                                                 |

### Returns

If the parameters are valid and supported, return a buffer size in bytes that guarantees that [psa\\_sign\\_hash\(\)](#) will not fail with [PSA\\_ERROR\\_BUFFER\\_TOO\\_SMALL](#). If the parameters are a valid combination that is not supported, return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

Definition at line 631 of file `crypto_sizes.h`.

#### 8.42.2.60 PSA\_SIGNATURE\_MAX\_SIZE

```
#define PSA_SIGNATURE_MAX_SIZE 1
```

Maximum size of an asymmetric signature.

This macro expands to a compile-time constant integer. This value is the maximum size of a signature in bytes.

Definition at line 646 of file `crypto_sizes.h`.

#### 8.42.2.61 PSA\_TLS12\_ECJPAKE\_TO\_PMS\_DATA\_SIZE

```
#define PSA_TLS12_ECJPAKE_TO_PMS_DATA_SIZE 32u
```

Definition at line 283 of file `crypto_sizes.h`.

#### 8.42.2.62 PSA\_TLS12\_ECJPAKE\_TO\_PMS\_INPUT\_SIZE

```
#define PSA_TLS12_ECJPAKE_TO_PMS_INPUT_SIZE 65u
```

Definition at line 278 of file `crypto_sizes.h`.

#### 8.42.2.63 PSA\_TLS12\_PSK\_TO\_MS\_PSK\_MAX\_SIZE

```
#define PSA_TLS12_PSK_TO_MS_PSK_MAX_SIZE 128u
```

This macro returns the maximum supported length of the PSK for the TLS-1.2 PSK-to-MS key derivation ([PSA\\_ALG\\_TLS12\\_PSK\\_TO\\_MS](#)(`hash_alg`)).

The maximum supported length does not depend on the chosen hash algorithm.

Quoting RFC 4279, Sect 5.3: TLS implementations supporting these ciphersuites MUST support arbitrary PSK identities up to 128 octets in length, and arbitrary PSKs up to 64 octets in length. Supporting longer identities and keys is RECOMMENDED.

Therefore, no implementation should define a value smaller than 64 for [PSA\\_TLS12\\_PSK\\_TO\\_MS\\_PSK\\_MAX\\_SIZE](#).  
Definition at line 274 of file `crypto_sizes.h`.

#### 8.42.2.64 PSA\_VENDOR\_ECC\_MAX\_CURVE\_BITS

```
#define PSA_VENDOR_ECC_MAX_CURVE_BITS 0u
```

Definition at line 256 of file `crypto_sizes.h`.



- struct [psa\\_cipher\\_operation\\_s](#)
- struct [psa\\_mac\\_operation\\_s](#)
- struct [psa\\_aead\\_operation\\_s](#)
- struct [psa\\_key\\_derivation\\_s](#)
- struct [psa\\_custom\\_key\\_parameters\\_s](#)
- struct [psa\\_key\\_policy\\_s](#)
- struct [psa\\_key\\_attributes\\_s](#)
- struct [psa\\_sign\\_hash\\_interruptible\\_operation\\_s](#)  
*The context for PSA interruptible hash signing.*
- struct [psa\\_verify\\_hash\\_interruptible\\_operation\\_s](#)  
*The context for PSA interruptible hash verification.*
- struct [psa\\_key\\_agreement\\_iop\\_s](#)  
*The context for PSA interruptible key agreement.*
- struct [psa\\_generate\\_key\\_iop\\_s](#)  
*The context for PSA interruptible key generation.*
- struct [psa\\_export\\_public\\_key\\_iop\\_s](#)  
*The context for PSA interruptible export public-key.*

## Macros

- `#define PSA_HASH_OPERATION_INIT { 0, { 0 } }`
- `#define PSA_XOF_OPERATION_INIT { 0, 0, 0, 0, 0, 0, { 0 } }`
- `#define PSA_CIPHER_OPERATION_INIT { 0, 0, 0, 0, { 0 } }`
- `#define PSA_MAC_OPERATION_INIT { 0, 0, 0, { 0 } }`
- `#define PSA_AEAD_OPERATION_INIT { 0, 0, 0, 0, 0, 0, 0, 0, { 0 } }`
- `#define PSA_KEY_DERIVATION_OPERATION_INIT { 0, 0, 0, { 0 } }`
- `#define PSA_CUSTOM_KEY_PARAMETERS_INIT { 0 }`
- `#define PSA_KEY_POLICY_INIT { 0, 0, 0 }`
- `#define PSA_KEY_BITS_TOO_LARGE ((psa_key_bits_t) -1)`
- `#define PSA_MAX_KEY_BITS 0xfff8`
- `#define PSA_KEY_ATTRIBUTES_INIT`
- `#define PSA_SIGN_HASH_INTERRUPTIBLE_OPERATION_INIT { 0, { 0 }, 0, 0 }`
- `#define PSA_VERIFY_HASH_INTERRUPTIBLE_OPERATION_INIT { 0, { 0 }, 0, 0 }`
- `#define PSA_KEY_AGREEMENT_IOP_INIT`
- `#define PSA_GENERATE_KEY_IOP_INIT`
- `#define PSA_EXPORT_PUBLIC_KEY_IOP_INIT { 0, MBEDTLS_PSA_EXPORT_PUBLIC_KEY_IOP_INIT, 0, 0 }`

## Typedefs

- typedef struct [psa\\_key\\_policy\\_s](#) [psa\\_key\\_policy\\_t](#)
- typedef uint16\_t [psa\\_key\\_bits\\_t](#)

## Variables

- struct [psa\\_hash\\_operation\\_s](#) [MBEDTLS\\_PRIVATE](#)

### 8.43.1 Detailed Description

PSA cryptography module: Mbed TLS structured type implementations.

#### Note

This file may not be included directly. Applications must include [psa/crypto.h](#).

This file contains the definitions of some data structures with implementation-specific definitions.

In implementations with isolation between the application and the cryptography module, it is expected that the front-end and the back-end would have different versions of this file.

**Design notes about multipart operation structures** For multipart operations without driver delegation support, each multipart operation structure contains a `psa_algorithm_t alg` field which indicates which specific algorithm the structure is for. When the structure is not in use, `alg` is 0. Most of the structure consists of a union which is discriminated by `alg`.

For multipart operations with driver delegation support, each multipart operation structure contains an `unsigned int id` field indicating which driver got assigned to do the operation. When the structure is not in use, 'id' is 0. The structure contains also a driver context which is the union of the contexts of all drivers able to handle the type of multipart operation.

Note that when `alg` or `id` is 0, the content of other fields is undefined. In particular, it is not guaranteed that a freshly-initialized structure is all-zero: we initialize structures to something like `{0, 0}`, which is only guaranteed to initialize the first member of the union; GCC and Clang initialize the whole structure to 0 (at the time of writing), but MSVC and CompCert don't.

In Mbed TLS, multipart operation structures live independently from the key. This allows Mbed TLS to free the key objects when destroying a key slot. If a multipart operation needs to remember the key after the setup function returns, the operation structure needs to contain a copy of the key.

## 8.43.2 Macro Definition Documentation

### 8.43.2.1 PSA\_CUSTOM\_KEY\_PARAMETERS\_INIT

```
#define PSA_CUSTOM_KEY_PARAMETERS_INIT { 0 }
```

The default production parameters for key generation or key derivation.

Calling `psa_generate_key_custom()` or `psa_key_derivation_output_key_custom()` with `custom=PSA_CUSTOM_KEY_PARAMETERS_INIT` and `custom_data_length=0` is equivalent to calling `psa_generate_key()` or `psa_key_derivation_output_key()` respectively.

Definition at line 281 of file `crypto_struct.h`.

### 8.43.2.2 PSA\_EXPORT\_PUBLIC\_KEY\_IOP\_INIT

```
#define PSA_EXPORT_PUBLIC_KEY_IOP_INIT { 0, MBEDTLS_PSA_EXPORT_PUBLIC_KEY_IOP_INIT, 0, 0 }
```

Definition at line 629 of file `crypto_struct.h`.

### 8.43.2.3 PSA\_GENERATE\_KEY\_IOP\_INIT

```
#define PSA_GENERATE_KEY_IOP_INIT
```

**Value:**

```
{ 0, MBEDTLS_PSA_GENERATE_KEY_IOP_INIT, PSA_KEY_ATTRIBUTES_INIT, \
 0, 0 }
```

Definition at line 592 of file `crypto_struct.h`.

### 8.43.2.4 PSA\_KEY\_AGREEMENT\_IOP\_INIT

```
#define PSA_KEY_AGREEMENT_IOP_INIT
```

**Value:**

```
{ 0, MBEDTLS_PSA_KEY_AGREEMENT_IOP_INIT, 0, \
 PSA_KEY_ATTRIBUTES_INIT, 0 }
```

Definition at line 554 of file `crypto_struct.h`.

### 8.43.2.5 PSA\_KEY\_BITS\_TOO\_LARGE

```
#define PSA_KEY_BITS_TOO_LARGE ((psa_key_bits_t) -1)
```

Definition at line 302 of file `crypto_struct.h`.

#### 8.43.2.6 PSA\_KEY\_POLICY\_INIT

```
#define PSA_KEY_POLICY_INIT { 0, 0, 0 }
```

Definition at line 290 of file `crypto_struct.h`.

#### 8.43.2.7 PSA\_MAX\_KEY\_BITS

```
#define PSA_MAX_KEY_BITS 0xffff8
```

Definition at line 308 of file `crypto_struct.h`.

#### 8.43.2.8 PSA\_SIGN\_HASH\_INTERRUPTIBLE\_OPERATION\_INIT

```
#define PSA_SIGN_HASH_INTERRUPTIBLE_OPERATION_INIT { 0, { 0 }, 0, 0 }
```

Definition at line 478 of file `crypto_struct.h`.

#### 8.43.2.9 PSA\_VERIFY\_HASH\_INTERRUPTIBLE\_OPERATION\_INIT

```
#define PSA_VERIFY_HASH_INTERRUPTIBLE_OPERATION_INIT { 0, { 0 }, 0, 0 }
```

Definition at line 516 of file `crypto_struct.h`.

### 8.43.3 Typedef Documentation

#### 8.43.3.1 psa\_key\_bits\_t

```
typedef uint16_t psa_key_bits_t
```

Definition at line 299 of file `crypto_struct.h`.

#### 8.43.3.2 psa\_key\_policy\_t

```
typedef struct psa_key_policy_s psa_key_policy_t
```

Definition at line 288 of file `crypto_struct.h`.

### 8.43.4 Variable Documentation

#### 8.43.4.1 MBEDTLS\_PRIVATE

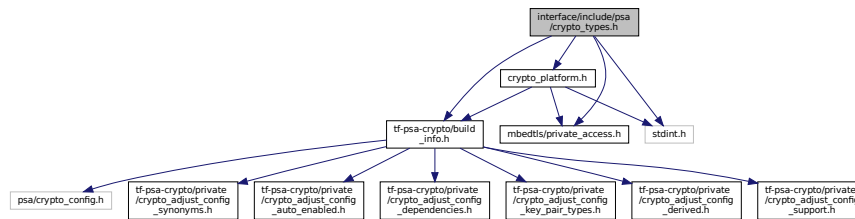
```
struct psa_hash_operation_s MBEDTLS_PRIVATE
```

## 8.44 interface/include/psa/crypto\_types.h File Reference

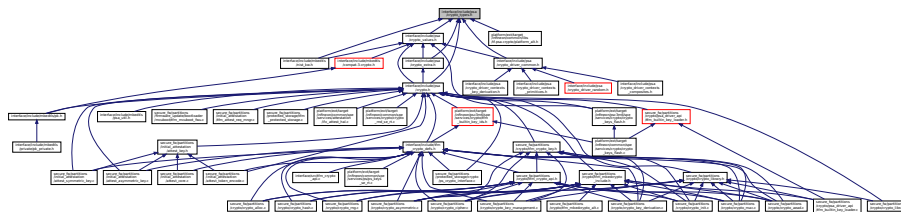
PSA cryptography module: type aliases.

```
#include "tf-psa-crypto/build_info.h"
#include "mbedtls/private_access.h"
#include "crypto_platform.h"
#include <stdint.h>
```

Include dependency graph for `crypto_types.h`:



This graph shows which files directly or indirectly include this file:



## Typedefs

- typedef int32\_t [psa\\_status\\_t](#)  
*Function return status.*
- typedef uint16\_t [psa\\_key\\_type\\_t](#)  
*Encoding of a key type.*
- typedef uint8\_t [psa\\_ecc\\_family\\_t](#)
- typedef uint8\_t [psa\\_dh\\_family\\_t](#)
- typedef uint32\_t [psa\\_algorithm\\_t](#)  
*Encoding of a cryptographic algorithm.*
- typedef uint32\_t [psa\\_key\\_lifetime\\_t](#)
- typedef uint8\_t [psa\\_key\\_persistence\\_t](#)
- typedef uint32\_t [psa\\_key\\_location\\_t](#)
- typedef uint32\_t [psa\\_key\\_id\\_t](#)
- typedef [psa\\_key\\_id\\_t](#) mbedtls\_svc\_key\_id\_t
- typedef uint32\_t [psa\\_key\\_usage\\_t](#)  
*Encoding of permitted usage on a key.*
- typedef struct [psa\\_key\\_attributes\\_s](#) [psa\\_key\\_attributes\\_t](#)
- typedef uint16\_t [psa\\_key\\_derivation\\_step\\_t](#)  
*Encoding of the step of a key derivation.*
- typedef struct [psa\\_custom\\_key\\_parameters\\_s](#) [psa\\_custom\\_key\\_parameters\\_t](#)  
*Custom parameters for key generation or key derivation.*

### 8.44.1 Detailed Description

PSA cryptography module: type aliases.

#### Note

This file may not be included directly. Applications must include [psa/crypto.h](#). Drivers must include the appropriate driver header file.

This file contains portable definitions of integral types for properties of cryptographic keys, designations of cryptographic algorithms, and error codes returned by the library.  
This header file does not declare any function.

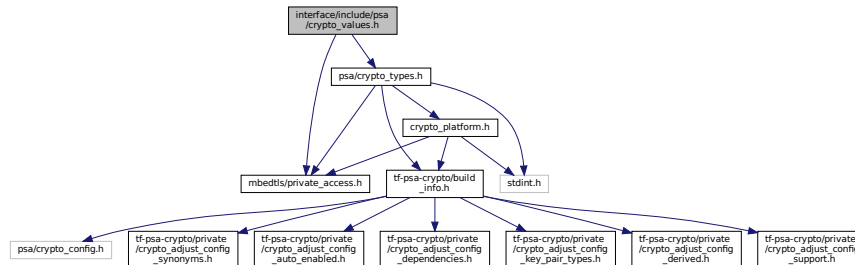
## 8.45 interface/include/psa/crypto\_values.h File Reference

PSA cryptography module: macros to build and analyze integer values.

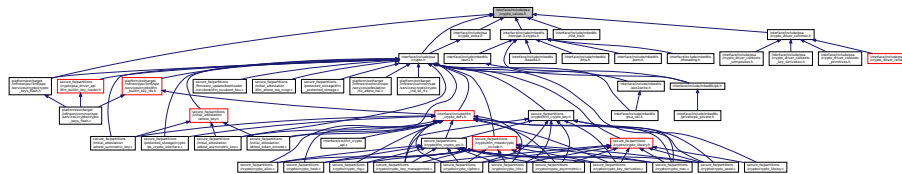
```
#include "mbedtls/private_access.h"
```

```
#include <psa/crypto_types.h>
```

Include dependency graph for crypto\_values.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define PSA_SUCCESS ((psa_status_t)0)`
- `#define PSA_ERROR_GENERIC_ERROR ((psa_status_t)-132)`
- `#define PSA_ERROR_NOT_SUPPORTED ((psa_status_t)-134)`
- `#define PSA_ERROR_NOT_PERMITTED ((psa_status_t)-133)`
- `#define PSA_ERROR_BUFFER_TOO_SMALL ((psa_status_t)-138)`
- `#define PSA_ERROR_ALREADY_EXISTS ((psa_status_t)-139)`
- `#define PSA_ERROR_DOES_NOT_EXIST ((psa_status_t)-140)`
- `#define PSA_ERROR_BAD_STATE ((psa_status_t)-137)`
- `#define PSA_ERROR_INVALID_ARGUMENT ((psa_status_t)-135)`
- `#define PSA_ERROR_INSUFFICIENT_MEMORY ((psa_status_t)-141)`
- `#define PSA_ERROR_INSUFFICIENT_STORAGE ((psa_status_t)-142)`
- `#define PSA_ERROR_COMMUNICATION_FAILURE ((psa_status_t)-145)`
- `#define PSA_ERROR_STORAGE_FAILURE ((psa_status_t)-146)`
- `#define PSA_ERROR_HARDWARE_FAILURE ((psa_status_t)-147)`
- `#define PSA_ERROR_CORRUPTION_DETECTED ((psa_status_t)-151)`
- `#define PSA_ERROR_INSUFFICIENT_ENTROPY ((psa_status_t)-148)`
- `#define PSA_ERROR_INVALID_SIGNATURE ((psa_status_t)-149)`
- `#define PSA_ERROR_INVALID_PADDING ((psa_status_t)-150)`
- `#define PSA_ERROR_INSUFFICIENT_DATA ((psa_status_t)-143)`
- `#define PSA_ERROR_SERVICE_FAILURE ((psa_status_t)-144)`
- `#define PSA_ERROR_INVALID_HANDLE ((psa_status_t)-136)`
- `#define PSA_ERROR_DATA_CORRUPT ((psa_status_t)-152)`
- `#define PSA_ERROR_DATA_INVALID ((psa_status_t)-153)`
- `#define PSA_OPERATION_INCOMPLETE ((psa_status_t)-248)`
- `#define PSA_KEY_TYPE_NONE ((psa_key_type_t) 0x0000)`

```

• #define PSA_KEY_TYPE_VENDOR_FLAG ((psa_key_type_t) 0x8000)
• #define PSA_KEY_TYPE_CATEGORY_MASK ((psa_key_type_t) 0x7000)
• #define PSA_KEY_TYPE_CATEGORY_RAW ((psa_key_type_t) 0x1000)
• #define PSA_KEY_TYPE_CATEGORY_SYMMETRIC ((psa_key_type_t) 0x2000)
• #define PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY ((psa_key_type_t) 0x4000)
• #define PSA_KEY_TYPE_CATEGORY_KEY_PAIR ((psa_key_type_t) 0x7000)
• #define PSA_KEY_TYPE_CATEGORY_FLAG_PAIR ((psa_key_type_t) 0x3000)
• #define PSA_KEY_TYPE_IS_VENDOR_DEFINED(type) (((type) & PSA_KEY_TYPE_VENDOR_FLAG) != 0)
• #define PSA_KEY_TYPE_IS_UNSTRUCTURED(type)
• #define PSA_KEY_TYPE_IS_ASYMMETRIC(type)
• #define PSA_KEY_TYPE_IS_PUBLIC_KEY(type) (((type) & PSA_KEY_TYPE_CATEGORY_MASK) ==
PSA_KEY_TYPE_CATEGORY_PUBLIC_KEY)
• #define PSA_KEY_TYPE_IS_KEY_PAIR(type) (((type) & PSA_KEY_TYPE_CATEGORY_MASK) ==
PSA_KEY_TYPE_CATEGORY_KEY_PAIR)
• #define PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY(type) ((type) | PSA_KEY_TYPE_CATEGORY_FLAG_PAIR)
• #define PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) ((type) & ~PSA_KEY_TYPE_CATEGORY_FLAG_PAIR)
• #define PSA_KEY_TYPE_RAW_DATA ((psa_key_type_t) 0x1001)
• #define PSA_KEY_TYPE_HMAC ((psa_key_type_t) 0x1100)
• #define PSA_KEY_TYPE_DERIVE ((psa_key_type_t) 0x1200)
• #define PSA_KEY_TYPE_PASSWORD ((psa_key_type_t) 0x1203)
• #define PSA_KEY_TYPE_PASSWORD_HASH ((psa_key_type_t) 0x1205)
• #define PSA_KEY_TYPE_PEPPER ((psa_key_type_t) 0x1206)
• #define PSA_KEY_TYPE_AES ((psa_key_type_t) 0x2400)
• #define PSA_KEY_TYPE_ARIA ((psa_key_type_t) 0x2406)
• #define PSA_KEY_TYPE_CAMELLIA ((psa_key_type_t) 0x2403)
• #define PSA_KEY_TYPE_CHACHA20 ((psa_key_type_t) 0x2004)
• #define PSA_KEY_TYPE_RSA_PUBLIC_KEY ((psa_key_type_t) 0x4001)
• #define PSA_KEY_TYPE_RSA_KEY_PAIR ((psa_key_type_t) 0x7001)
• #define PSA_KEY_TYPE_IS_RSA(type) (PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type) ==
PSA_KEY_TYPE_RSA_PUBLIC_KEY)
• #define PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4100)
• #define PSA_KEY_TYPE_ECC_KEY_PAIR_BASE ((psa_key_type_t) 0x7100)
• #define PSA_KEY_TYPE_ECC_CURVE_MASK ((psa_key_type_t) 0x00ff)
• #define PSA_KEY_TYPE_ECC_KEY_PAIR(curve) (PSA_KEY_TYPE_ECC_KEY_PAIR_BASE | (curve))
• #define PSA_KEY_TYPE_ECC_PUBLIC_KEY(curve) (PSA_KEY_TYPE_ECC_PUBLIC_KEY_BASE |
(curve))
• #define PSA_KEY_TYPE_IS_ECC(type)
• #define PSA_KEY_TYPE_IS_ECC_KEY_PAIR(type)
• #define PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY(type)
• #define PSA_KEY_TYPE_HAS_ECC_FAMILY(type) (PSA_KEY_TYPE_IS_ECC(type) || PSA_KEY_TYPE_IS_SPAKE2P(type)
• #define PSA_KEY_TYPE_ECC_GET_FAMILY(type)
• #define PSA_ECC_FAMILY_IS_WEIERSTRASS(family) ((family & 0xc0) == 0)
• #define PSA_ECC_FAMILY_SECP_K1 ((psa_ecc_family_t) 0x17)
• #define PSA_ECC_FAMILY_SECP_R1 ((psa_ecc_family_t) 0x12)
• #define PSA_ECC_FAMILY_SECP_R2 ((psa_ecc_family_t) 0x1b)
• #define PSA_ECC_FAMILY_SECT_K1 ((psa_ecc_family_t) 0x27)
• #define PSA_ECC_FAMILY_SECT_R1 ((psa_ecc_family_t) 0x22)
• #define PSA_ECC_FAMILY_SECT_R2 ((psa_ecc_family_t) 0x2b)
• #define PSA_ECC_FAMILY_BRAINPOOL_P_R1 ((psa_ecc_family_t) 0x30)
• #define PSA_ECC_FAMILY_MONTGOMERY ((psa_ecc_family_t) 0x41)
• #define PSA_ECC_FAMILY_TWISTED_EDWARDS ((psa_ecc_family_t) 0x42)
• #define PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE ((psa_key_type_t) 0x4200)
• #define PSA_KEY_TYPE_DH_KEY_PAIR_BASE ((psa_key_type_t) 0x7200)
• #define PSA_KEY_TYPE_DH_GROUP_MASK ((psa_key_type_t) 0x00ff)
• #define PSA_KEY_TYPE_DH_KEY_PAIR(group) (PSA_KEY_TYPE_DH_KEY_PAIR_BASE | (group))

```



- `#define PSA_KEY_TYPE_DH_PUBLIC_KEY(group) (PSA_KEY_TYPE_DH_PUBLIC_KEY_BASE | (group))`
- `#define PSA_KEY_TYPE_IS_DH(type)`
- `#define PSA_KEY_TYPE_IS_DH_KEY_PAIR(type)`
- `#define PSA_KEY_TYPE_IS_DH_PUBLIC_KEY(type)`
- `#define PSA_KEY_TYPE_DH_GET_FAMILY(type)`
- `#define PSA_DH_FAMILY_RFC7919 ((psa_dh_family_t) 0x03)`
- `#define PSA_GET_KEY_TYPE_BLOCK_SIZE_EXPONENT(type) (((type) >> 8) & 7)`
- `#define PSA_BLOCK_CIPHER_BLOCK_LENGTH(type)`
- `#define PSA_ALG_VENDOR_FLAG ((psa_algorithm_t) 0x80000000)`
- `#define PSA_ALG_CATEGORY_MASK ((psa_algorithm_t) 0x7f000000)`
- `#define PSA_ALG_CATEGORY_HASH ((psa_algorithm_t) 0x02000000)`
- `#define PSA_ALG_CATEGORY_MAC ((psa_algorithm_t) 0x03000000)`
- `#define PSA_ALG_CATEGORY_CIPHER ((psa_algorithm_t) 0x04000000)`
- `#define PSA_ALG_CATEGORY_AEAD ((psa_algorithm_t) 0x05000000)`
- `#define PSA_ALG_CATEGORY_SIGN ((psa_algorithm_t) 0x06000000)`
- `#define PSA_ALG_CATEGORY_ASYMMETRIC_ENCRYPTION ((psa_algorithm_t) 0x07000000)`
- `#define PSA_ALG_CATEGORY_KEY_DERIVATION ((psa_algorithm_t) 0x08000000)`
- `#define PSA_ALG_CATEGORY_KEY_AGREEMENT ((psa_algorithm_t) 0x09000000)`
- `#define PSA_ALG_CATEGORY_XOF ((psa_algorithm_t) 0x0d000000)`
- `#define PSA_ALG_CATEGORY_KEY_WRAP ((psa_algorithm_t) 0x0B000000)`
- `#define PSA_ALG_IS_VENDOR_DEFINED(alg) (((alg) & PSA_ALG_VENDOR_FLAG) != 0)`
- `#define PSA_ALG_IS_HASH(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_HASH)`
- `#define PSA_ALG_IS_MAC(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_MAC)`
- `#define PSA_ALG_IS_CIPHER(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_CIPHER)`
- `#define PSA_ALG_IS_AEAD(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_AEAD)`
- `#define PSA_ALG_IS_SIGN(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_SIGN)`
- `#define PSA_ALG_IS_ASYMMETRIC_ENCRYPTION(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_ASYMMETRIC_ENCRYPTION)`
- `#define PSA_ALG_IS_KEY_AGREEMENT(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_KEY_AGREEMENT)`
- `#define PSA_ALG_IS_KEY_DERIVATION(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_KEY_DERIVATION)`
- `#define PSA_ALG_IS_KEY_DERIVATION_STRETCHING(alg)`
- `#define PSA_ALG_IS_XOF(alg) (((alg) & PSA_ALG_CATEGORY_MASK) == PSA_ALG_CATEGORY_XOF)`
- `#define PSA_ALG_NONE ((psa_algorithm_t) 0)`
- `#define PSA_ALG_HASH_MASK ((psa_algorithm_t) 0x000000ff)`
- `#define PSA_ALG_MD5 ((psa_algorithm_t) 0x02000003)`
- `#define PSA_ALG_RIPEMD160 ((psa_algorithm_t) 0x02000004)`
- `#define PSA_ALG_SHA_1 ((psa_algorithm_t) 0x02000005)`
- `#define PSA_ALG_SHA_224 ((psa_algorithm_t) 0x02000008)`
- `#define PSA_ALG_SHA_256 ((psa_algorithm_t) 0x02000009)`
- `#define PSA_ALG_SHA_384 ((psa_algorithm_t) 0x0200000a)`
- `#define PSA_ALG_SHA_512 ((psa_algorithm_t) 0x0200000b)`
- `#define PSA_ALG_SHA_512_224 ((psa_algorithm_t) 0x0200000c)`
- `#define PSA_ALG_SHA_512_256 ((psa_algorithm_t) 0x0200000d)`
- `#define PSA_ALG_SHA3_224 ((psa_algorithm_t) 0x02000010)`
- `#define PSA_ALG_SHA3_256 ((psa_algorithm_t) 0x02000011)`
- `#define PSA_ALG_SHA3_384 ((psa_algorithm_t) 0x02000012)`
- `#define PSA_ALG_SHA3_512 ((psa_algorithm_t) 0x02000013)`
- `#define PSA_ALG_SHAKE256_512 ((psa_algorithm_t) 0x02000015)`
- `#define PSA_ALG_ANY_HASH ((psa_algorithm_t) 0x020000ff)`
- `#define PSA_ALG_SHAKE128 ((psa_algorithm_t) 0x0d000100)`
- `#define PSA_ALG_SHAKE256 ((psa_algorithm_t) 0x0d000200)`
- `#define PSA_ALG_XOF_CONTEXT_FLAG ((psa_algorithm_t) 0x00008000)`
- `#define PSA_ALG_XOF_HAS_CONTEXT(xof_alg) (((xof_alg) & PSA_ALG_XOF_CONTEXT_FLAG) != 0)`
- `#define PSA_ALG_MAC_SUBCATEGORY_MASK ((psa_algorithm_t) 0x00c00000)`
- `#define PSA_ALG_HMAC_BASE ((psa_algorithm_t) 0x03800000)`

```

• #define PSA_ALG_HMAC(hash_alg) (PSA_ALG_HMAC_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_HMAC_GET_HASH(hmac_alg) (PSA_ALG_CATEGORY_HASH | ((hmac_alg) &
 PSA_ALG_HASH_MASK))
• #define PSA_ALG_IS_HMAC(alg)
• #define PSA_ALG_MAC_TRUNCATION_MASK ((psa_algorithm_t) 0x003f0000)
• #define PSA_MAC_TRUNCATION_OFFSET 16
• #define PSA_ALG_MAC_AT_LEAST_THIS_LENGTH_FLAG ((psa_algorithm_t) 0x00008000)
• #define PSA_ALG_TRUNCATED_MAC(mac_alg, mac_length)
• #define PSA_ALG_FULL_LENGTH_MAC(mac_alg)
• #define PSA_MAC_TRUNCATED_LENGTH(mac_alg) (((mac_alg) & PSA_ALG_MAC_TRUNCATION_MASK)
 >> PSA_MAC_TRUNCATION_OFFSET)
• #define PSA_ALG_AT_LEAST_THIS_LENGTH_MAC(mac_alg, min_mac_length)
• #define PSA_ALG_CIPHER_MAC_BASE ((psa_algorithm_t) 0x03c00000)
• #define PSA_ALG_CBC_MAC ((psa_algorithm_t) 0x03c00100)
• #define PSA_ALG_CMAC ((psa_algorithm_t) 0x03c00200)
• #define PSA_ALG_IS_BLOCK_CIPHER_MAC(alg)
• #define PSA_ALG_CIPHER_STREAM_FLAG ((psa_algorithm_t) 0x00800000)
• #define PSA_ALG_CIPHER_FROM_BLOCK_FLAG ((psa_algorithm_t) 0x00400000)
• #define PSA_ALG_IS_STREAM_CIPHER(alg)
• #define PSA_ALG_STREAM_CIPHER ((psa_algorithm_t) 0x04800100)
• #define PSA_ALG_CTR ((psa_algorithm_t) 0x04c01000)
• #define PSA_ALG_CFB ((psa_algorithm_t) 0x04c01100)
• #define PSA_ALG_OFB ((psa_algorithm_t) 0x04c01200)
• #define PSA_ALG_XTS ((psa_algorithm_t) 0x0440ff00)
• #define PSA_ALG_ECB_NO_PADDING ((psa_algorithm_t) 0x04404400)
• #define PSA_ALG_CBC_NO_PADDING ((psa_algorithm_t) 0x04404000)
• #define PSA_ALG_CBC_PKCS7 ((psa_algorithm_t) 0x04404100)
• #define PSA_ALG_AEAD_FROM_BLOCK_FLAG ((psa_algorithm_t) 0x00400000)
• #define PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER(alg)
• #define PSA_ALG_CCM ((psa_algorithm_t) 0x05500100)
• #define PSA_ALG_CCM_STAR_NO_TAG ((psa_algorithm_t) 0x04c01300)
• #define PSA_ALG_GCM ((psa_algorithm_t) 0x05500200)
• #define PSA_ALG_CHACHA20_POLY1305 ((psa_algorithm_t) 0x05100500)
• #define PSA_ALG_AEAD_TAG_LENGTH_MASK ((psa_algorithm_t) 0x003f0000)
• #define PSA_AEAD_TAG_LENGTH_OFFSET 16
• #define PSA_ALG_AEAD_AT_LEAST_THIS_LENGTH_FLAG ((psa_algorithm_t) 0x00008000)
• #define PSA_ALG_AEAD_WITH_SHORTENED_TAG(aead_alg, tag_length)
• #define PSA_ALG_AEAD_GET_TAG_LENGTH(aead_alg)
• #define PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG(aead_alg)
• #define PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG_CASE(aead_alg, ref)
• #define PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG(aead_alg, min_tag_length)
• #define PSA_ALG_RSA_PKCS1V15_SIGN_BASE ((psa_algorithm_t) 0x06000200)
• #define PSA_ALG_RSA_PKCS1V15_SIGN(hash_alg) (PSA_ALG_RSA_PKCS1V15_SIGN_BASE |
 ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_RSA_PKCS1V15_SIGN_RAW PSA_ALG_RSA_PKCS1V15_SIGN_BASE
• #define PSA_ALG_IS_RSA_PKCS1V15_SIGN(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_PKCS1V15_SIGN)
• #define PSA_ALG_RSA_PSS_BASE ((psa_algorithm_t) 0x06000300)
• #define PSA_ALG_RSA_PSS_ANY_SALT_BASE ((psa_algorithm_t) 0x06001300)
• #define PSA_ALG_RSA_PSS(hash_alg) (PSA_ALG_RSA_PSS_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_RSA_PSS_ANY_SALT(hash_alg) (PSA_ALG_RSA_PSS_ANY_SALT_BASE | ((hash_↵
 _alg) & PSA_ALG_HASH_MASK))
• #define PSA_ALG_IS_RSA_PSS_STANDARD_SALT(alg) (((alg) & ~PSA_ALG_HASH_MASK) ==
 PSA_ALG_RSA_PSS_BASE)
• #define PSA_ALG_IS_RSA_PSS_ANY_SALT(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_PSS_ANY_SALT)
• #define PSA_ALG_IS_RSA_PSS(alg)

```

- `#define PSA_ALG_ECDSA_BASE ((psa_algorithm_t) 0x06000600)`
- `#define PSA_ALG_ECDSA(hash_alg) (PSA_ALG_ECDSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_ECDSA_ANY PSA_ALG_ECDSA_BASE`
- `#define PSA_ALG_DETERMINISTIC_ECDSA_BASE ((psa_algorithm_t) 0x06000700)`
- `#define PSA_ALG_DETERMINISTIC_ECDSA(hash_alg) (PSA_ALG_DETERMINISTIC_ECDSA_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_ECDSA_DETERMINISTIC_FLAG ((psa_algorithm_t) 0x00000100)`
- `#define PSA_ALG_IS_ECDSA(alg)`
- `#define PSA_ALG_ECDSA_IS_DETERMINISTIC(alg) (((alg) & PSA_ALG_ECDSA_DETERMINISTIC_FLAG) != 0)`
- `#define PSA_ALG_IS_DETERMINISTIC_ECDSA(alg) (PSA_ALG_IS_ECDSA(alg) && PSA_ALG_ECDSA_IS_DETERMINISTIC(alg))`
- `#define PSA_ALG_IS_RANDOMIZED_ECDSA(alg) (PSA_ALG_IS_ECDSA(alg) && !PSA_ALG_ECDSA_IS_DETERMINISTIC(alg))`
- `#define PSA_ALG_PURE_EDDSA ((psa_algorithm_t) 0x06000800)`
- `#define PSA_ALG_HASH_EDDSA_BASE ((psa_algorithm_t) 0x06000900)`
- `#define PSA_ALG_IS_HASH_EDDSA(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HASH_EDDSA_BASE)`
- `#define PSA_ALG_ED25519PH (PSA_ALG_HASH_EDDSA_BASE | (PSA_ALG_SHA_512 & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_ED448PH (PSA_ALG_HASH_EDDSA_BASE | (PSA_ALG_SHAKE256_512 & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_VENDOR_HASH_AND_SIGN(alg) 0`
- `#define PSA_ALG_IS_SIGN_HASH(alg)`
- `#define PSA_ALG_IS_SIGN_MESSAGE(alg) (PSA_ALG_IS_SIGN_HASH(alg) || (alg) == PSA_ALG_PURE_EDDSA)`
- `#define PSA_ALG_IS_HASH_AND_SIGN(alg)`
- `#define PSA_ALG_SIGN_GET_HASH(alg)`
- `#define PSA_ALG_RSA_PKCS1V15_CRYPT ((psa_algorithm_t) 0x07000200)`
- `#define PSA_ALG_RSA_OAEP_BASE ((psa_algorithm_t) 0x07000300)`
- `#define PSA_ALG_RSA_OAEP(hash_alg) (PSA_ALG_RSA_OAEP_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_RSA_OAEP(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_RSA_OAEP_BASE)`
- `#define PSA_ALG_RSA_OAEP_GET_HASH(alg)`
- `#define PSA_ALG_SP800_108_COUNTER_CMAC ((psa_algorithm_t) 0x08000800)`
- `#define PSA_ALG_IS_SP800_108_COUNTER_CMAC(alg) ((alg) == PSA_ALG_SP800_108_COUNTER_CMAC)`
- `#define PSA_ALG_HKDF_BASE ((psa_algorithm_t) 0x08000100)`
- `#define PSA_ALG_HKDF(hash_alg) (PSA_ALG_HKDF_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_HKDF(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_BASE)`
- `#define PSA_ALG_HKDF_GET_HASH(hkdf_alg) (PSA_ALG_CATEGORY_HASH | ((hkdf_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_HKDF_EXTRACT_BASE ((psa_algorithm_t) 0x08000400)`
- `#define PSA_ALG_HKDF_EXTRACT(hash_alg) (PSA_ALG_HKDF_EXTRACT_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_HKDF_EXTRACT(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_EXTRACT_BASE)`
- `#define PSA_ALG_HKDF_EXPAND_BASE ((psa_algorithm_t) 0x08000500)`
- `#define PSA_ALG_HKDF_EXPAND(hash_alg) (PSA_ALG_HKDF_EXPAND_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_HKDF_EXPAND(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HKDF_EXPAND_BASE)`
- `#define PSA_ALG_IS_ANY_HKDF(alg)`
- `#define PSA_ALG_TLS12_PRF_BASE ((psa_algorithm_t) 0x08000200)`
- `#define PSA_ALG_TLS12_PRF(hash_alg) (PSA_ALG_TLS12_PRF_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_TLS12_PRF(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_TLS12_PRF_BASE)`
- `#define PSA_ALG_TLS12_PRF_GET_HASH(hkdf_alg) (PSA_ALG_CATEGORY_HASH | ((hkdf_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_TLS12_PSK_TO_MS_BASE ((psa_algorithm_t) 0x08000300)`
- `#define PSA_ALG_TLS12_PSK_TO_MS(hash_alg) (PSA_ALG_TLS12_PSK_TO_MS_BASE | ((hash_alg) & PSA_ALG_HASH_MASK))`
- `#define PSA_ALG_IS_TLS12_PSK_TO_MS(alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_TLS12_PSK_TO_MS_BASE)`
- `#define PSA_ALG_TLS12_PSK_TO_MS_GET_HASH(hkdf_alg) (PSA_ALG_CATEGORY_HASH | ((hkdf_alg) & PSA_ALG_HASH_MASK))`

- #define PSA\_ALG\_TLS12\_ECJPAKE\_TO\_PMS ((psa\_algorithm\_t) 0x08000609)
- #define PSA\_ALG\_KEY\_DERIVATION\_STRETCHING\_FLAG ((psa\_algorithm\_t) 0x00800000)
- #define PSA\_ALG\_PBKDF2\_HMAC\_BASE ((psa\_algorithm\_t) 0x08800100)
- #define PSA\_ALG\_PBKDF2\_HMAC(hash\_alg) (PSA\_ALG\_PBKDF2\_HMAC\_BASE | ((hash\_alg) & PSA\_ALG\_HASH\_MASK))
- #define PSA\_ALG\_IS\_PBKDF2\_HMAC(alg) (((alg) & ~PSA\_ALG\_HASH\_MASK) == PSA\_ALG\_PBKDF2\_HMAC\_BASE)
- #define PSA\_ALG\_PBKDF2\_HMAC\_GET\_HASH(pbkdf2\_alg) (PSA\_ALG\_CATEGORY\_HASH | ((pbkdf2\_alg) & PSA\_ALG\_HASH\_MASK))
- #define PSA\_ALG\_PBKDF2\_AES\_CMAC\_PRF\_128 ((psa\_algorithm\_t) 0x08800200)
- #define PSA\_ALG\_IS\_PBKDF2(kdf\_alg)
- #define PSA\_ALG\_KEY\_DERIVATION\_MASK ((psa\_algorithm\_t) 0xfe00ffff)
- #define PSA\_ALG\_KEY\_AGREEMENT\_MASK ((psa\_algorithm\_t) 0xffff0000)
- #define PSA\_ALG\_KEY\_AGREEMENT(ka\_alg, kdf\_alg) ((ka\_alg) | (kdf\_alg))
- #define PSA\_ALG\_KEY\_AGREEMENT\_GET\_KDF(alg) (((alg) & PSA\_ALG\_KEY\_DERIVATION\_MASK) | PSA\_ALG\_CATEGORY\_KEY\_DERIVATION)
- #define PSA\_ALG\_KEY\_AGREEMENT\_GET\_BASE(alg) (((alg) & PSA\_ALG\_KEY\_AGREEMENT\_MASK) | PSA\_ALG\_CATEGORY\_KEY\_AGREEMENT)
- #define PSA\_ALG\_IS\_RAW\_KEY\_AGREEMENT(alg)
- #define PSA\_ALG\_IS\_KEY\_DERIVATION\_OR\_AGREEMENT(alg) ((PSA\_ALG\_IS\_KEY\_DERIVATION(alg) || PSA\_ALG\_IS\_KEY\_AGREEMENT(alg)))
- #define PSA\_ALG\_FFDH ((psa\_algorithm\_t) 0x09010000)
- #define PSA\_ALG\_IS\_FFDH(alg) (PSA\_ALG\_KEY\_AGREEMENT\_GET\_BASE(alg) == PSA\_ALG\_FFDH)
- #define PSA\_ALG\_ECDH ((psa\_algorithm\_t) 0x09020000)
- #define PSA\_ALG\_IS\_ECDH(alg) (PSA\_ALG\_KEY\_AGREEMENT\_GET\_BASE(alg) == PSA\_ALG\_ECDH)
- #define PSA\_ALG\_IS\_WILDCARD(alg)
- #define PSA\_ALG\_GET\_HASH(alg) (((alg) & 0x000000ff) == 0 ? ((psa\_algorithm\_t) 0) : 0x02000000 | ((alg) & 0x000000ff))
- #define PSA\_ALG\_AES\_KW ((psa\_algorithm\_t) 0x0B400100)
- #define PSA\_ALG\_AES\_KWP ((psa\_algorithm\_t) 0x0BC00200)
- #define PSA\_ALG\_IS\_KEY\_WRAP(alg) (((alg) & PSA\_ALG\_CATEGORY\_MASK) == PSA\_ALG\_CATEGORY\_AES\_KEY\_WRAP)

*Check whether the specified algorithm is a key-wrapping algorithm.*

- #define PSA\_KEY\_LIFETIME\_VOLATILE ((psa\_key\_lifetime\_t) 0x00000000)
- #define PSA\_KEY\_LIFETIME\_PERSISTENT ((psa\_key\_lifetime\_t) 0x00000001)
- #define PSA\_KEY\_PERSISTENCE\_VOLATILE ((psa\_key\_persistence\_t) 0x00)
- #define PSA\_KEY\_PERSISTENCE\_DEFAULT ((psa\_key\_persistence\_t) 0x01)
- #define PSA\_KEY\_PERSISTENCE\_READ\_ONLY ((psa\_key\_persistence\_t) 0xff)
- #define PSA\_KEY\_LIFETIME\_GET\_PERSISTENCE(lifetime) ((psa\_key\_persistence\_t) ((lifetime) & 0x000000ff))
- #define PSA\_KEY\_LIFETIME\_GET\_LOCATION(lifetime) ((psa\_key\_location\_t) ((lifetime) >> 8))
- #define PSA\_KEY\_LIFETIME\_IS\_VOLATILE(lifetime)
- #define PSA\_KEY\_LIFETIME\_IS\_READ\_ONLY(lifetime)
- #define PSA\_KEY\_LIFETIME\_FROM\_PERSISTENCE\_AND\_LOCATION(persistence, location) ((location) << 8 | (persistence))
- #define PSA\_KEY\_LOCATION\_LOCAL\_STORAGE ((psa\_key\_location\_t) 0x0000000)
- #define PSA\_KEY\_LOCATION\_VENDOR\_FLAG ((psa\_key\_location\_t) 0x8000000)
- #define PSA\_KEY\_ID\_NULL ((psa\_key\_id\_t) 0)
- #define PSA\_KEY\_ID\_USER\_MIN ((psa\_key\_id\_t) 0x00000001)
- #define PSA\_KEY\_ID\_USER\_MAX ((psa\_key\_id\_t) 0x3fffffff)
- #define PSA\_KEY\_ID\_VENDOR\_MIN ((psa\_key\_id\_t) 0x40000000)
- #define PSA\_KEY\_ID\_VENDOR\_MAX ((psa\_key\_id\_t) 0x7fffffff)
- #define MBEDTLS\_SVC\_KEY\_ID\_INIT ((psa\_key\_id\_t) 0)
- #define MBEDTLS\_SVC\_KEY\_ID\_GET\_KEY\_ID(id) (id)
- #define MBEDTLS\_SVC\_KEY\_ID\_GET\_OWNER\_ID(id) (0)
- #define PSA\_KEY\_USAGE\_EXPORT ((psa\_key\_usage\_t) 0x00000001)

- #define [PSA\\_KEY\\_USAGE\\_COPY](#) (([psa\\_key\\_usage\\_t](#)) 0x00000002)
  - #define [PSA\\_KEY\\_USAGE\\_DERIVE\\_PUBLIC](#) (([psa\\_key\\_usage\\_t](#)) 0x00000080)
  - #define [PSA\\_KEY\\_USAGE\\_ENCRYPT](#) (([psa\\_key\\_usage\\_t](#)) 0x00000100)
  - #define [PSA\\_KEY\\_USAGE\\_DECRYPT](#) (([psa\\_key\\_usage\\_t](#)) 0x00000200)
  - #define [PSA\\_KEY\\_USAGE\\_SIGN\\_MESSAGE](#) (([psa\\_key\\_usage\\_t](#)) 0x00000400)
  - #define [PSA\\_KEY\\_USAGE\\_VERIFY\\_MESSAGE](#) (([psa\\_key\\_usage\\_t](#)) 0x00000800)
  - #define [PSA\\_KEY\\_USAGE\\_SIGN\\_HASH](#) (([psa\\_key\\_usage\\_t](#)) 0x00001000)
  - #define [PSA\\_KEY\\_USAGE\\_VERIFY\\_HASH](#) (([psa\\_key\\_usage\\_t](#)) 0x00002000)
  - #define [PSA\\_KEY\\_USAGE\\_DERIVE](#) (([psa\\_key\\_usage\\_t](#)) 0x00004000)
  - #define [PSA\\_KEY\\_USAGE\\_VERIFY\\_DERIVATION](#) (([psa\\_key\\_usage\\_t](#)) 0x00008000)
  - #define [PSA\\_KEY\\_USAGE\\_WRAP](#) (([psa\\_key\\_usage\\_t](#)) 0x00010000)
  - #define [PSA\\_KEY\\_USAGE\\_UNWRAP](#) (([psa\\_key\\_usage\\_t](#)) 0x00020000)
- Permission to unwrap another key with the key.*
- #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_SECRET](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0101)
  - #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_PASSWORD](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0102)
  - #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_OTHER\\_SECRET](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0103)
  - #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_LABEL](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0201)
  - #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_SALT](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0202)
  - #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_INFO](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0203)
  - #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_SEED](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0204)
  - #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_COST](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0205)
  - #define [PSA\\_KEY\\_DERIVATION\\_INPUT\\_CONTEXT](#) (([psa\\_key\\_derivation\\_step\\_t](#)) 0x0206)
  - #define [MBEDTLS\\_PSA\\_ALG\\_AEAD\\_EQUAL](#)([aead\\_alg\\_1](#), [aead\\_alg\\_2](#))
  - #define [PSA\\_INTERRUPTIBLE\\_MAX\\_OPS\\_UNLIMITED](#) [UINT32\\_MAX](#)

### 8.45.1 Detailed Description

PSA cryptography module: macros to build and analyze integer values.

#### Note

This file may not be included directly. Applications must include [psa/crypto.h](#). Drivers must include the appropriate driver header file.

This file contains portable definitions of macros to build and analyze values of integral types that encode properties of cryptographic keys, designations of cryptographic algorithms, and error codes returned by the library.

Note that many of the constants defined in this file are embedded in the persistent key store, as part of key meta-data (including usage policies). As a consequence, they must not be changed (unless the storage format version changes).

This header file only defines preprocessor macros.

## 8.46 interface/include/psa/crypto\_values\_lms.h File Reference

### Macros

- #define [PSA\\_ALG\\_LMS\\_BASE](#) 0x00100000
- #define [PSA\\_ALG\\_IS\\_LMS](#)(alg) (((alg) & ~[PSA\\_ALG\\_HASH\\_MASK](#)) == [PSA\\_ALG\\_LMS\\_BASE](#))
- #define [PSA\\_ALG\\_LMS](#)(hash)
- #define [PSA\\_ALG\\_HSS\\_BASE](#) 0x00200000
- #define [PSA\\_ALG\\_IS\\_HSS](#)(alg) (((alg) & ~[PSA\\_ALG\\_HASH\\_MASK](#)) == [PSA\\_ALG\\_HSS\\_BASE](#))
- #define [PSA\\_ALG\\_HSS](#)(hash)

### 8.46.1 Macro Definition Documentation

### 8.46.1.1 PSA\_ALG\_HSS

```
#define PSA_ALG_HSS(
 hash)
```

**Value:**

```
(\
 PSA_ALG_VENDOR_FLAG | \
 PSA_ALG_CATEGORY_SIGN | \
 PSA_ALG_HSS_BASE | \
 ((hash) & PSA_ALG_HASH_MASK) \
)
```

Definition at line 26 of file `crypto_values_lms.h`.

### 8.46.1.2 PSA\_ALG\_HSS\_BASE

```
#define PSA_ALG_HSS_BASE 0x00200000
```

Definition at line 22 of file `crypto_values_lms.h`.

### 8.46.1.3 PSA\_ALG\_IS\_HSS

```
#define PSA_ALG_IS_HSS(
 alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_HSS_BASE)
```

Definition at line 24 of file `crypto_values_lms.h`.

### 8.46.1.4 PSA\_ALG\_IS\_LMS

```
#define PSA_ALG_IS_LMS(
 alg) (((alg) & ~PSA_ALG_HASH_MASK) == PSA_ALG_LMS_BASE)
```

Definition at line 13 of file `crypto_values_lms.h`.

### 8.46.1.5 PSA\_ALG\_LMS

```
#define PSA_ALG_LMS(
 hash)
```

**Value:**

```
(\
 PSA_ALG_VENDOR_FLAG | \
 PSA_ALG_CATEGORY_SIGN | \
 PSA_ALG_LMS_BASE | \
 ((hash) & PSA_ALG_HASH_MASK) \
)
```

Definition at line 15 of file `crypto_values_lms.h`.

### 8.46.1.6 PSA\_ALG\_LMS\_BASE

```
#define PSA_ALG_LMS_BASE 0x00100000
```

Definition at line 11 of file `crypto_values_lms.h`.

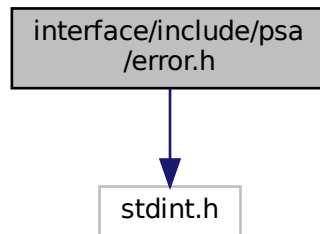
## 8.47 interface/include/psa/error.h File Reference

Standard error codes for the SPM and RoT Services.



```
#include <stdint.h>
```

Include dependency graph for error.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define PSA_SUCCESS ((psa_status_t)0)`
- `#define PSA_ERROR_PROGRAMMER_ERROR ((psa_status_t)-129)`
- `#define PSA_ERROR_CONNECTION_REFUSED ((psa_status_t)-130)`
- `#define PSA_ERROR_CONNECTION_BUSY ((psa_status_t)-131)`
- `#define PSA_ERROR_GENERIC_ERROR ((psa_status_t)-132)`
- `#define PSA_ERROR_NOT_PERMITTED ((psa_status_t)-133)`
- `#define PSA_ERROR_NOT_SUPPORTED ((psa_status_t)-134)`
- `#define PSA_ERROR_INVALID_ARGUMENT ((psa_status_t)-135)`
- `#define PSA_ERROR_INVALID_HANDLE ((psa_status_t)-136)`
- `#define PSA_ERROR_BAD_STATE ((psa_status_t)-137)`
- `#define PSA_ERROR_BUFFER_TOO_SMALL ((psa_status_t)-138)`
- `#define PSA_ERROR_ALREADY_EXISTS ((psa_status_t)-139)`
- `#define PSA_ERROR_DOES_NOT_EXIST ((psa_status_t)-140)`
- `#define PSA_ERROR_INSUFFICIENT_MEMORY ((psa_status_t)-141)`
- `#define PSA_ERROR_INSUFFICIENT_STORAGE ((psa_status_t)-142)`
- `#define PSA_ERROR_INSUFFICIENT_DATA ((psa_status_t)-143)`
- `#define PSA_ERROR_SERVICE_FAILURE ((psa_status_t)-144)`
- `#define PSA_ERROR_COMMUNICATION_FAILURE ((psa_status_t)-145)`
- `#define PSA_ERROR_STORAGE_FAILURE ((psa_status_t)-146)`
- `#define PSA_ERROR_HARDWARE_FAILURE ((psa_status_t)-147)`
- `#define PSA_ERROR_INVALID_SIGNATURE ((psa_status_t)-149)`
- `#define PSA_ERROR_CORRUPTION_DETECTED ((psa_status_t)-151)`
- `#define PSA_ERROR_DATA_CORRUPT ((psa_status_t)-152)`
- `#define PSA_ERROR_DATA_INVALID ((psa_status_t)-153)`
- `#define PSA_OPERATION_INCOMPLETE ((psa_status_t)-248)`

## Typedefs

- `typedef int32_t psa_status_t`

### 8.47.1 Detailed Description

Standard error codes for the SPM and RoT Services.

### 8.47.2 Macro Definition Documentation

#### 8.47.2.1 PSA\_ERROR\_ALREADY\_EXISTS

```
#define PSA_ERROR_ALREADY_EXISTS ((psa_status_t)-139)
```

Asking for an item that already exists

Implementations should return this error, when attempting to write an item (like a key) that already exists.

Definition at line 129 of file error.h.

#### 8.47.2.2 PSA\_ERROR\_BAD\_STATE

```
#define PSA_ERROR_BAD_STATE ((psa_status_t)-137)
```

The requested action cannot be performed in the current state.

Multipart operations return this error when one of the functions is called out of sequence. Refer to the function descriptions for permitted sequencing of functions.

Implementations shall not return this error code to indicate that a key either exists or not, but shall instead return [PSA\\_ERROR\\_ALREADY\\_EXISTS](#) or [PSA\\_ERROR\\_DOES\\_NOT\\_EXIST](#) as applicable.

Implementations shall not return this error code to indicate that a key identifier is invalid, but shall return [PSA\\_ERROR\\_INVALID\\_HANDLE](#) instead.

Definition at line 111 of file error.h.

#### 8.47.2.3 PSA\_ERROR\_BUFFER\_TOO\_SMALL

```
#define PSA_ERROR_BUFFER_TOO_SMALL ((psa_status_t)-138)
```

An output buffer is too small.

Applications can call the `PSA_XXX_SIZE` macro listed in the function description to determine a sufficient buffer size.

Implementations should preferably return this error code only in cases when performing the operation with a larger output buffer would succeed. However implementations may return this error if a function has invalid or unsupported parameters in addition to the parameters that determine the necessary output buffer size.

Definition at line 123 of file error.h.

#### 8.47.2.4 PSA\_ERROR\_COMMUNICATION\_FAILURE

```
#define PSA_ERROR_COMMUNICATION_FAILURE ((psa_status_t)-145)
```

There was a communication failure inside the implementation.

This can indicate a communication failure between the application and an external cryptoprocessor or between the cryptoprocessor and an external volatile or persistent memory. A communication failure may be transient or permanent depending on the cause.

##### Warning

If a function returns this error, it is undetermined whether the requested action has completed or not. Implementations should return [PSA\\_SUCCESS](#) on successful completion whenever possible, however functions may return [PSA\\_ERROR\\_COMMUNICATION\\_FAILURE](#) if the requested action was completed successfully in an external cryptoprocessor but there was a breakdown of communication before the cryptoprocessor could report the status to the application.

Definition at line 176 of file error.h.



#### 8.47.2.5 PSA\_ERROR\_CONNECTION\_BUSY

```
#define PSA_ERROR_CONNECTION_BUSY ((psa_status_t)-131)
```

A status code that indicates that the caller cannot connect to a service.

Definition at line 51 of file error.h.

#### 8.47.2.6 PSA\_ERROR\_CONNECTION\_REFUSED

```
#define PSA_ERROR_CONNECTION_REFUSED ((psa_status_t)-130)
```

A status code that indicates that the caller is not permitted to connect to a Service.

Definition at line 47 of file error.h.

#### 8.47.2.7 PSA\_ERROR\_CORRUPTION\_DETECTED

```
#define PSA_ERROR_CORRUPTION_DETECTED ((psa_status_t)-151)
```

A tampering attempt was detected.

If an application receives this error code, there is no guarantee that previously accessed or computed data was correct and remains confidential. Applications should not perform any security function and should enter a safe failure state.

Implementations may return this error code if they detect an invalid state that cannot happen during normal operation and that indicates that the implementation's security guarantees no longer hold. Depending on the implementation architecture and on its security and safety goals, the implementation may forcibly terminate the application.

This error code is intended as a last resort when a security breach is detected and it is unsure whether the keystore data is still protected. Implementations shall only return this error code to report an alarm from a tampering detector, to indicate that the confidentiality of stored data can no longer be guaranteed, or to indicate that the integrity of previously returned data is now considered compromised. Implementations shall not use this error code to indicate a hardware failure that merely makes it impossible to perform the requested operation (use [PSA\\_ERROR\\_COMMUNICATION\\_FAILURE](#), [PSA\\_ERROR\\_STORAGE\\_FAILURE](#), [PSA\\_ERROR\\_HARDWARE\\_FAILURE](#), [PSA\\_ERROR\\_INSUFFICIENT\\_ENTROPY](#) or other applicable error code instead).

This error indicates an attack against the application. Implementations shall not return this error code as a consequence of the behavior of the application itself.

Definition at line 252 of file error.h.

#### 8.47.2.8 PSA\_ERROR\_DATA\_CORRUPT

```
#define PSA_ERROR_DATA_CORRUPT ((psa_status_t)-152)
```

Stored data has been corrupted.

This error indicates that some persistent storage has suffered corruption. It does not indicate the following situations, which have specific error codes:

- A corruption of volatile memory - use [PSA\\_ERROR\\_CORRUPTION\\_DETECTED](#).
- A communication error between the cryptoprocessor and its external storage - use [PSA\\_ERROR\\_COMMUNICATION\\_FAILURE](#).
- When the storage is in a valid state but is full - use [PSA\\_ERROR\\_INSUFFICIENT\\_STORAGE](#).
- When the storage fails for other reasons - use [PSA\\_ERROR\\_STORAGE\\_FAILURE](#).
- When the stored data is not valid - use [PSA\\_ERROR\\_DATA\\_INVALID](#).

##### Note

A storage corruption does not indicate that any data that was previously read is invalid. However this previously read data might no longer be readable from storage.

When a storage failure occurs, it is no longer possible to ensure the global integrity of the keystore.

Definition at line 276 of file error.h.

#### 8.47.2.9 PSA\_ERROR\_DATA\_INVALID

```
#define PSA_ERROR_DATA_INVALID ((psa_status_t)-153)
```

Data read from storage is not valid for the implementation.

This error indicates that some data read from storage does not have a valid format. It does not indicate the following situations, which have specific error codes:

- When the storage or stored data is corrupted - use [PSA\\_ERROR\\_DATA\\_CORRUPT](#)
- When the storage fails for other reasons - use [PSA\\_ERROR\\_STORAGE\\_FAILURE](#)
- An invalid argument to the API - use [PSA\\_ERROR\\_INVALID\\_ARGUMENT](#)

This error is typically a result of either storage corruption on a cleartext storage backend, or an attempt to read data that was written by an incompatible version of the library.

Definition at line 292 of file error.h.

#### 8.47.2.10 PSA\_ERROR\_DOES\_NOT\_EXIST

```
#define PSA_ERROR_DOES_NOT_EXIST ((psa_status_t)-140)
```

Asking for an item that doesn't exist

Implementations should return this error, if a requested item (like a key) does not exist.

Definition at line 135 of file error.h.

#### 8.47.2.11 PSA\_ERROR\_GENERIC\_ERROR

```
#define PSA_ERROR_GENERIC_ERROR ((psa_status_t)-132)
```

An error occurred that does not correspond to any defined failure cause.

Implementations may use this error code if none of the other standard error codes are applicable.

Definition at line 58 of file error.h.

#### 8.47.2.12 PSA\_ERROR\_HARDWARE\_FAILURE

```
#define PSA_ERROR_HARDWARE_FAILURE ((psa_status_t)-147)
```

A hardware failure was detected.

A hardware failure may be transient or permanent depending on the cause.

Definition at line 207 of file error.h.

#### 8.47.2.13 PSA\_ERROR\_INSUFFICIENT\_DATA

```
#define PSA_ERROR_INSUFFICIENT_DATA ((psa_status_t)-143)
```

Return this error when there's insufficient data when attempting to read from a resource.

Definition at line 154 of file error.h.

#### 8.47.2.14 PSA\_ERROR\_INSUFFICIENT\_MEMORY

```
#define PSA_ERROR_INSUFFICIENT_MEMORY ((psa_status_t)-141)
```

There is not enough runtime memory.

If the action is carried out across multiple security realms, this error can refer to available memory in any of the security realms.

Definition at line 141 of file error.h.

#### 8.47.2.15 PSA\_ERROR\_INSUFFICIENT\_STORAGE

```
#define PSA_ERROR_INSUFFICIENT_STORAGE ((psa_status_t)-142)
```

There is not enough persistent storage.

Functions that modify the key storage return this error code if there is insufficient storage space on the host media. In addition, many functions that do not otherwise access storage may return this error code if the implementation requires a mandatory log entry for the requested action and the log storage space is full.

Definition at line 150 of file error.h.

#### 8.47.2.16 PSA\_ERROR\_INVALID\_ARGUMENT

```
#define PSA_ERROR_INVALID_ARGUMENT ((psa_status_t)-135)
```

The parameters passed to the function are invalid.

Implementations may return this error any time a parameter or combination of parameters are recognized as invalid. Implementations shall not return this error code to indicate that a key identifier is invalid, but shall return [PSA\\_ERROR\\_INVALID\\_HANDLE](#) instead.

Definition at line 91 of file error.h.

#### 8.47.2.17 PSA\_ERROR\_INVALID\_HANDLE

```
#define PSA_ERROR_INVALID_HANDLE ((psa_status_t)-136)
```

The key identifier is not valid. See also :ref:`key-handles`.

Definition at line 95 of file error.h.

#### 8.47.2.18 PSA\_ERROR\_INVALID\_SIGNATURE

```
#define PSA_ERROR_INVALID_SIGNATURE ((psa_status_t)-149)
```

The signature, MAC or hash is incorrect.

Verification functions return this error if the verification calculations completed successfully, and the value to be verified was determined to be incorrect.

If the value to verify has an invalid size, implementations may return either [PSA\\_ERROR\\_INVALID\\_ARGUMENT](#) or [PSA\\_ERROR\\_INVALID\\_SIGNATURE](#).

Definition at line 219 of file error.h.

#### 8.47.2.19 PSA\_ERROR\_NOT\_PERMITTED

```
#define PSA_ERROR_NOT_PERMITTED ((psa_status_t)-133)
```

The requested action is denied by a policy.

Implementations should return this error code when the parameters are recognized as valid and supported, and a policy explicitly denies the requested operation.

If a subset of the parameters of a function call identify a forbidden operation, and another subset of the parameters are not valid or not supported, it is unspecified whether the function returns [PSA\\_ERROR\\_NOT\\_PERMITTED](#), [PSA\\_ERROR\\_NOT\\_SUPPORTED](#) or [PSA\\_ERROR\\_INVALID\\_ARGUMENT](#).

Definition at line 71 of file error.h.

#### 8.47.2.20 PSA\_ERROR\_NOT\_SUPPORTED

```
#define PSA_ERROR_NOT_SUPPORTED ((psa_status_t)-134)
```

The requested operation or a parameter is not supported by this implementation.

Implementations should return this error code when an enumeration parameter such as a key type, algorithm, etc. is not recognized. If a combination of parameters is recognized and identified as not valid, return [PSA\\_ERROR\\_INVALID\\_ARGUMENT](#) instead.

Definition at line 80 of file error.h.

#### 8.47.2.21 PSA\_ERROR\_PROGRAMMER\_ERROR

```
#define PSA_ERROR_PROGRAMMER_ERROR ((psa_status_t)-129)
```

A status code that indicates a PROGRAMMER ERROR in the client

This error indicates that the function has detected an abnormal call, which typically indicates a programming error in the caller, or an abuse of the API.

Definition at line 43 of file error.h.

#### 8.47.2.22 PSA\_ERROR\_SERVICE\_FAILURE

```
#define PSA_ERROR_SERVICE_FAILURE ((psa_status_t)-144)
```

This can be returned if a function can no longer operate correctly. For example, if an essential initialization operation failed or a mutex operation failed.

Definition at line 159 of file error.h.

#### 8.47.2.23 PSA\_ERROR\_STORAGE\_FAILURE

```
#define PSA_ERROR_STORAGE_FAILURE ((psa_status_t)-146)
```

There was a storage failure that may have led to data loss.

This error indicates that some persistent storage is corrupted. It should not be used for a corruption of volatile memory (use [PSA\\_ERROR\\_CORRUPTION\\_DETECTED](#)), for a communication error between the cryptoprocessor and its external storage (use [PSA\\_ERROR\\_COMMUNICATION\\_FAILURE](#)), or when the storage is in a valid state but is full (use [PSA\\_ERROR\\_INSUFFICIENT\\_STORAGE](#)).

Note that a storage failure does not indicate that any data that was previously read is invalid. However this previously read data may no longer be readable from storage.

When a storage failure occurs, it is no longer possible to ensure the global integrity of the keystore. Depending on the global integrity guarantees offered by the implementation, access to other data may or may not fail even if the data is still readable but its integrity cannot be guaranteed.

Implementations should only use this error code to report a permanent storage corruption. However application writers should keep in mind that transient errors while reading the storage may be reported using this error code.

Definition at line 201 of file error.h.

#### 8.47.2.24 PSA\_OPERATION\_INCOMPLETE

```
#define PSA_OPERATION_INCOMPLETE ((psa_status_t)-248)
```

The function that returns this status is defined as interruptible and still has work to do, thus the user should call the function again with the same operation context until it either returns [PSA\\_SUCCESS](#) or any other error. This is not an error per se, more a notification of status.

Definition at line 301 of file error.h.

#### 8.47.2.25 PSA\_SUCCESS

```
#define PSA_SUCCESS ((psa_status_t)0)
```

The action was completed successfully.

Definition at line 34 of file error.h.

### 8.47.3 Typedef Documentation

#### 8.47.3.1 psa\_status\_t

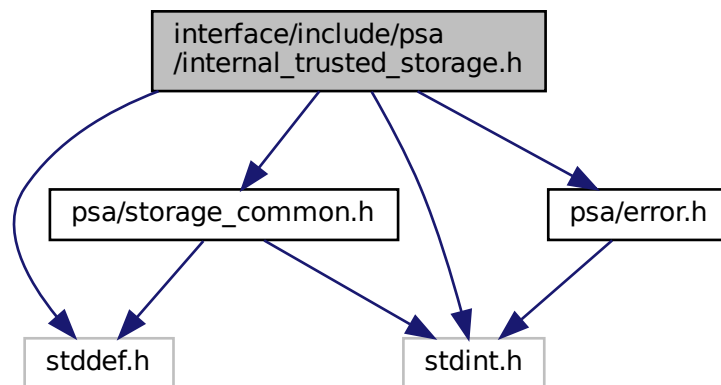
```
typedef int32_t psa_status_t
```

Definition at line 27 of file error.h.

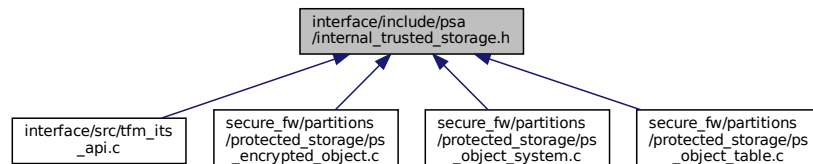
## 8.48 interface/include/psa/internal\_trusted\_storage.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "psa/error.h"
#include "psa/storage_common.h"
```

Include dependency graph for internal\_trusted\_storage.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define PSA_ITS_API_VERSION_MAJOR 1`
- `#define PSA_ITS_API_VERSION_MINOR 0`

### Functions

- `psa_status_t psa_its_set(psa_storage_uid_t uid, size_t data_length, const void *p_data, psa_storage_create_flags_t create_flags)`  
Create a new, or modify an existing, uid/value pair.
- `psa_status_t psa_its_get(psa_storage_uid_t uid, size_t data_offset, size_t data_size, void *p_data, size_t *p_data_length)`  
Retrieve data associated with a provided UID.
- `psa_status_t psa_its_get_info(psa_storage_uid_t uid, struct psa_storage_info_t *p_info)`  
Retrieve the metadata about the provided uid.
- `psa_status_t psa_its_remove(psa_storage_uid_t uid)`

*Remove the provided uid and its associated data from the storage.*

## 8.48.1 Macro Definition Documentation

### 8.48.1.1 PSA\_ITS\_API\_VERSION\_MAJOR

```
#define PSA_ITS_API_VERSION_MAJOR 1
```

This file describes the PSA Internal Trusted Storage API The major version number of the PSA ITS API Definition at line 23 of file internal\_trusted\_storage.h.

### 8.48.1.2 PSA\_ITS\_API\_VERSION\_MINOR

```
#define PSA_ITS_API_VERSION_MINOR 0
```

The minor version number of the PSA ITS API Definition at line 26 of file internal\_trusted\_storage.h.

## 8.48.2 Function Documentation

### 8.48.2.1 psa\_its\_get()

```
psa_status_t psa_its_get (
 psa_storage_uid_t uid,
 size_t data_offset,
 size_t data_size,
 void * p_data,
 size_t * p_data_length)
```

Retrieve data associated with a provided UID.

Retrieves up to `data_size` bytes of the data associated with `uid`, starting at `data_offset` bytes from the beginning of the data. Upon successful completion, the data will be placed in the `p_data` buffer, which must be at least `data_size` bytes in size. The length of the data returned will be in `p_data_length`. If `data_size` is 0, the contents of `p_data_length` will be set to zero.

#### Parameters

|     |                      |                                                                              |
|-----|----------------------|------------------------------------------------------------------------------|
| in  | <i>uid</i>           | The uid value                                                                |
| in  | <i>data_offset</i>   | The starting offset of the data requested                                    |
| in  | <i>data_size</i>     | The amount of data requested                                                 |
| out | <i>p_data</i>        | On success, the buffer where the data will be placed                         |
| out | <i>p_data_length</i> | On success, this will contain size of the data placed in <code>p_data</code> |

#### Returns

A status indicating the success/failure of the operation

#### Return values

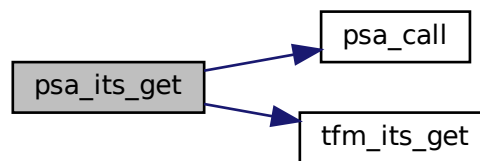
|                                  |                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>               | The operation completed successfully                                                          |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>  | The operation failed because the provided <code>uid</code> value was not found in the storage |
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the physical storage has failed (Fatal error)                    |

## Return values

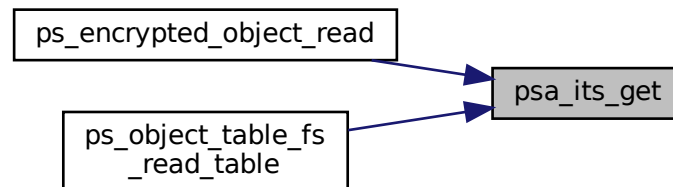
|                                         |                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>PSA_ERROR_INVALID_ARGUMENT</code> | The operation failed because one of the provided arguments ( <code>p_data</code> , <code>p_data_length</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access. In addition, this can also happen if <code>data_offset</code> is larger than the size of the data associated with <code>uid</code> |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Definition at line 38 of file `tfm_its_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.48.2.2 psa\_its\_get\_info()

```

psa_status_t psa_its_get_info (
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Retrieve the metadata about the provided uid.

Retrieves the metadata stored for a given `uid` as a `psa_storage_info_t` structure.

## Parameters

|     |                     |                                                                                                  |
|-----|---------------------|--------------------------------------------------------------------------------------------------|
| in  | <code>uid</code>    | The <code>uid</code> value                                                                       |
| out | <code>p_info</code> | A pointer to the <code>psa_storage_info_t</code> struct that will be populated with the metadata |

**Returns**

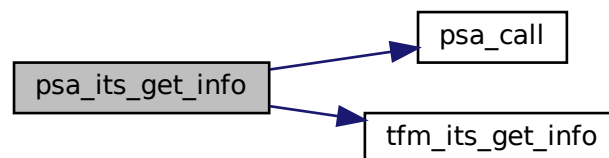
A status indicating the success/failure of the operation

**Return values**

|                                   |                                                                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                  |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                                                                      |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                                                                            |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided pointers( <i>p_info</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

Definition at line 75 of file `tfm_its_api.c`.

Here is the call graph for this function:

**8.48.2.3 psa\_its\_remove()**

```

psa_status_t psa_its_remove (
 psa_storage_uid_t uid)

```

Remove the provided uid and its associated data from the storage.

Deletes the data from internal storage.

**Parameters**

|                 |            |               |
|-----------------|------------|---------------|
| <code>in</code> | <i>uid</i> | The uid value |
|-----------------|------------|---------------|

**Returns**

A status indicating the success/failure of the operation

**Return values**

|                                   |                                                                                                                    |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                               |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                   |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code>      |

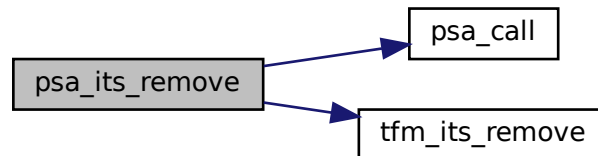


## Return values

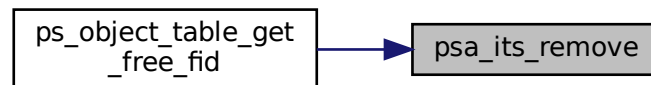
|                                  |                                                                            |
|----------------------------------|----------------------------------------------------------------------------|
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the physical storage has failed (Fatal error) |
|----------------------------------|----------------------------------------------------------------------------|

Definition at line 104 of file `tfm_its_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

8.48.2.4 `psa_its_set()`

```

psa_status_t psa_its_set (
 psa_storage_uid_t uid,
 size_t data_length,
 const void * p_data,
 psa_storage_create_flags_t create_flags)

```

Create a new, or modify an existing, uid/value pair.

Stores data in the internal storage.

## Parameters

|    |                     |                                                |
|----|---------------------|------------------------------------------------|
| in | <i>uid</i>          | The identifier for the data                    |
| in | <i>data_length</i>  | The size in bytes of the data in <i>p_data</i> |
| in | <i>p_data</i>       | A buffer containing the data                   |
| in | <i>create_flags</i> | The flags that the data will be stored with    |

**Returns**

A status indicating the success/failure of the operation

**Return values**

|                                       |                                                                                                                                                                             |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                                                                                        |
| <i>PSA_ERROR_NOT_PERMITTED</i>        | The operation failed because the provided <code>uid</code> value was already created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code>                                          |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because one or more of the flags provided in <code>create_flags</code> is not supported or is not valid                                                |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because there was insufficient space on the storage medium                                                                                             |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (Fatal error)                                                                                                  |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because one of the provided pointers( <code>p_data</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

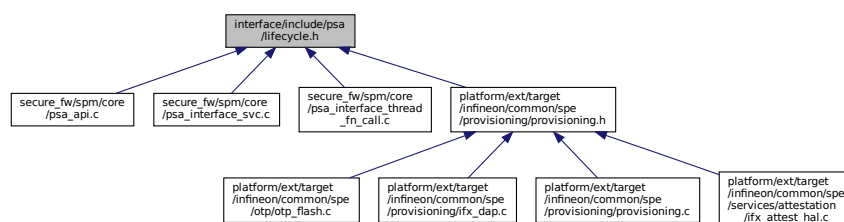
Definition at line 19 of file `tfm_its_api.c`.

Here is the call graph for this function:



## 8.49 interface/include/psa/lifecycle.h File Reference

This graph shows which files directly or indirectly include this file:

**Macros**

- `#define PSA_LIFECYCLE_PSA_STATE_MASK` (0xff00u)
- `#define PSA_LIFECYCLE_IMP_STATE_MASK` (0x00ffu)
- `#define PSA_LIFECYCLE_UNKNOWN` (0x0000u)
- `#define PSA_LIFECYCLE_ASSEMBLY_AND_TEST` (0x1000u)
- `#define PSA_LIFECYCLE_PSA_ROT_PROVISIONING` (0x2000u)
- `#define PSA_LIFECYCLE_SECURED` (0x3000u)

- `#define PSA_LIFECYCLE_NON_PSA_ROT_DEBUG (0x4000u)`
- `#define PSA_LIFECYCLE_RECOVERABLE_PSA_ROT_DEBUG (0x5000u)`
- `#define PSA_LIFECYCLE_DECOMMISSIONED (0x6000u)`

## Functions

- `uint32_t psa_rot_lifecycle_state (void)`

### 8.49.1 Macro Definition Documentation

#### 8.49.1.1 PSA\_LIFECYCLE\_ASSEMBLY\_AND\_TEST

```
#define PSA_LIFECYCLE_ASSEMBLY_AND_TEST (0x1000u)
```

Definition at line 18 of file lifecycle.h.

#### 8.49.1.2 PSA\_LIFECYCLE\_DECOMMISSIONED

```
#define PSA_LIFECYCLE_DECOMMISSIONED (0x6000u)
```

Definition at line 23 of file lifecycle.h.

#### 8.49.1.3 PSA\_LIFECYCLE\_IMP\_STATE\_MASK

```
#define PSA_LIFECYCLE_IMP_STATE_MASK (0x00ffu)
```

Definition at line 16 of file lifecycle.h.

#### 8.49.1.4 PSA\_LIFECYCLE\_NON\_PSA\_ROT\_DEBUG

```
#define PSA_LIFECYCLE_NON_PSA_ROT_DEBUG (0x4000u)
```

Definition at line 21 of file lifecycle.h.

#### 8.49.1.5 PSA\_LIFECYCLE\_PSA\_ROT\_PROVISIONING

```
#define PSA_LIFECYCLE_PSA_ROT_PROVISIONING (0x2000u)
```

Definition at line 19 of file lifecycle.h.

#### 8.49.1.6 PSA\_LIFECYCLE\_PSA\_STATE\_MASK

```
#define PSA_LIFECYCLE_PSA_STATE_MASK (0xff00u)
```

Definition at line 15 of file lifecycle.h.

#### 8.49.1.7 PSA\_LIFECYCLE\_RECOVERABLE\_PSA\_ROT\_DEBUG

```
#define PSA_LIFECYCLE_RECOVERABLE_PSA_ROT_DEBUG (0x5000u)
```

Definition at line 22 of file lifecycle.h.

#### 8.49.1.8 PSA\_LIFECYCLE\_SECURED

```
#define PSA_LIFECYCLE_SECURED (0x3000u)
```

Definition at line 20 of file lifecycle.h.

### 8.49.1.9 PSA\_LIFECYCLE\_UNKNOWN

```
#define PSA_LIFECYCLE_UNKNOWN (0x0000u)
```

Definition at line 17 of file lifecycle.h.

## 8.49.2 Function Documentation

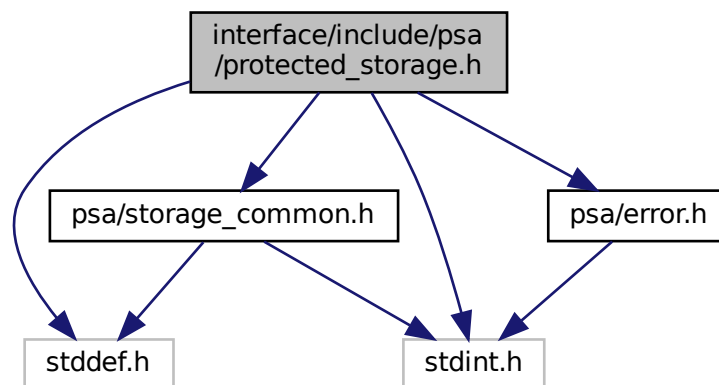
### 8.49.2.1 psa\_rot\_lifecycle\_state()

```
uint32_t psa_rot_lifecycle_state (
 void)
```

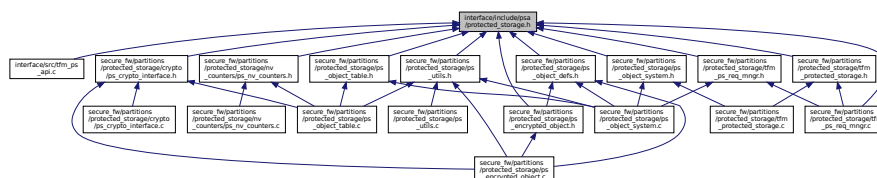
Definition at line 79 of file psa\_api\_ipc.c.

## 8.50 interface/include/psa/protected\_storage.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "psa/error.h"
#include "psa/storage_common.h"
Include dependency graph for protected_storage.h:
```



This graph shows which files directly or indirectly include this file:



## Macros

- `#define` `PSA_PS_API_VERSION_MAJOR` 1

*PSA\_PS\_API\_VERSION version.*

- `#define PSA_PS_API_VERSION_MINOR 0`

## Functions

- `psa_status_t psa_ps_set (psa_storage_uid_t uid, size_t data_length, const void *p_data, psa_storage_create_flags_t create_flags)`  
*Create a new, or modify an existing, uid/value pair.*
- `psa_status_t psa_ps_get (psa_storage_uid_t uid, size_t data_offset, size_t data_size, void *p_data, size_t *p_data_length)`  
*Retrieve data associated with a provided uid.*
- `psa_status_t psa_ps_get_info (psa_storage_uid_t uid, struct psa_storage_info_t *p_info)`  
*Retrieve the metadata about the provided uid.*
- `psa_status_t psa_ps_remove (psa_storage_uid_t uid)`  
*Remove the provided uid and its associated data from the storage.*
- `psa_status_t psa_ps_create (psa_storage_uid_t uid, size_t capacity, psa_storage_create_flags_t create_flags)`  
*Reserves storage for the specified uid.*
- `psa_status_t psa_ps_set_extended (psa_storage_uid_t uid, size_t data_offset, size_t data_length, const void *p_data)`  
*Sets partial data into an asset.*
- `uint32_t psa_ps_get_support (void)`  
*Lists optional features.*

## 8.50.1 Macro Definition Documentation

### 8.50.1.1 PSA\_PS\_API\_VERSION\_MAJOR

```
#define PSA_PS_API_VERSION_MAJOR 1
```

PSA\_PS\_API\_VERSION version.  
Major and minor PSA\_PS\_API\_VERSION numbers  
Definition at line 28 of file protected\_storage.h.

### 8.50.1.2 PSA\_PS\_API\_VERSION\_MINOR

```
#define PSA_PS_API_VERSION_MINOR 0
```

Definition at line 29 of file protected\_storage.h.

## 8.50.2 Function Documentation

### 8.50.2.1 psa\_ps\_create()

```
psa_status_t psa_ps_create (
 psa_storage_uid_t uid,
 size_t capacity,
 psa_storage_create_flags_t create_flags)
```

Reserves storage for the specified uid.

Upon success, the capacity of the storage will be capacity, and the size will be 0. It is only necessary to call this function for assets that will be written with the `psa_ps_set_extended` function. If only the `psa_ps_set` function is needed, calls to this function are redundant.

**Parameters**

|    |                     |                                        |
|----|---------------------|----------------------------------------|
| in | <i>uid</i>          | The uid value                          |
| in | <i>capacity</i>     | The capacity to be allocated in bytes  |
| in | <i>create_flags</i> | Flags indicating properties of storage |

**Returns**

A status indicating the success/failure of the operation

**Return values**

|                                       |                                                                                                             |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                        |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (Fatal error)                                  |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because the capacity is bigger than the current available space                        |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because the function is not implemented or one or more create_flags are not supported. |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because uid was 0 or create_flags specified flags that are not defined in the API.     |
| <i>PSA_ERROR_GENERIC_ERROR</i>        | The operation failed due to an unspecified error                                                            |
| <i>PSA_ERROR_ALREADY_EXISTS</i>       | Storage for the specified uid already exists                                                                |

Definition at line 117 of file tfm\_ps\_api.c.

**8.50.2.2 psa\_ps\_get()**

```
psa_status_t psa_ps_get (
 psa_storage_uid_t uid,
 size_t data_offset,
 size_t data_size,
 void * p_data,
 size_t * p_data_length)
```

Retrieve data associated with a provided uid.

Retrieves up to data\_size bytes of the data associated with uid, starting at data\_offset bytes from the beginning of the data. Upon successful completion, the data will be placed in the p\_data buffer, which must be at least data\_size bytes in size. The length of the data returned will be in p\_data\_length. If data\_size is 0, the contents of p\_data\_length will be set to zero.

**Parameters**

|     |                      |                                                                 |
|-----|----------------------|-----------------------------------------------------------------|
| in  | <i>uid</i>           | The uid value                                                   |
| in  | <i>data_offset</i>   | The starting offset of the data requested                       |
| in  | <i>data_size</i>     | The amount of data requested                                    |
| out | <i>p_data</i>        | On success, the buffer where the data will be placed            |
| out | <i>p_data_length</i> | On success, this will contain size of the data placed in p_data |

**Returns**

A status indicating the success/failure of the operation

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                 | The operation completed successfully                                                                                                                                                                                                                                                                                   |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>  | The operation failed because one of the provided arguments ( <i>p_data</i> , <i>p_data_length</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access. In addition, this can also happen if <i>data_offset</i> is larger than the size of the data associated with <i>uid</i> |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>    | The operation failed because the provided <i>uid</i> value was not found in the storage                                                                                                                                                                                                                                |
| <i>PSA_ERROR_STORAGE_FAILURE</i>   | The operation failed because the physical storage has failed (Fatal error)                                                                                                                                                                                                                                             |
| <i>PSA_ERROR_GENERIC_ERROR</i>     | The operation failed because of an unspecified internal failure                                                                                                                                                                                                                                                        |
| <i>PSA_ERROR_DATA_CORRUPT</i>      | The operation failed because the data associated with the UID was corrupt                                                                                                                                                                                                                                              |
| <i>PSA_ERROR_INVALID_SIGNATURE</i> | The operation failed because the data associated with the UID failed authentication                                                                                                                                                                                                                                    |

Definition at line 38 of file `tfm_ps_api.c`.

Here is the call graph for this function:

8.50.2.3 `psa_ps_get_info()`

```

psa_status_t psa_ps_get_info (
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Retrieve the metadata about the provided *uid*.

Retrieves the metadata stored for a given *uid*

## Parameters

|     |               |                                                                                                  |
|-----|---------------|--------------------------------------------------------------------------------------------------|
| in  | <i>uid</i>    | The <i>uid</i> value                                                                             |
| out | <i>p_info</i> | A pointer to the <code>psa_storage_info_t</code> struct that will be populated with the metadata |

## Returns

A status indicating the success/failure of the operation

## Return values

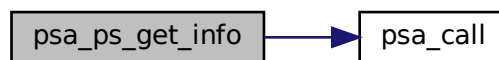
|                                   |                                                                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                  |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided pointers( <i>p_info</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

## Return values

|                                  |                                                                                  |
|----------------------------------|----------------------------------------------------------------------------------|
| <i>PSA_ERROR_DOES_NOT_EXIST</i>  | The operation failed because the provided uid value was not found in the storage |
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the physical storage has failed (Fatal error)       |
| <i>PSA_ERROR_GENERIC_ERROR</i>   | The operation failed because of an unspecified internal failure                  |
| <i>PSA_ERROR_DATA_CORRUPT</i>    | The operation failed because the data associated with the UID was corrupt        |

Definition at line 75 of file tfm\_ps\_api.c.

Here is the call graph for this function:



#### 8.50.2.4 psa\_ps\_get\_support()

```
uint32_t psa_ps_get_support (
 void)
```

Lists optional features.

## Returns

A bitmask with flags set for all of the optional features supported by the implementation. Currently defined flags are limited to `PSA_STORAGE_SUPPORT_SET_EXTENDED`

Definition at line 138 of file tfm\_ps\_api.c.

Here is the call graph for this function:



#### 8.50.2.5 psa\_ps\_remove()

```
psa_status_t psa_ps_remove (
 psa_storage_uid_t uid)
```

Remove the provided uid and its associated data from the storage.

Removes previously stored data and any associated metadata, including rollback protection data.



## Parameters

|    |            |               |
|----|------------|---------------|
| in | <i>uid</i> | The uid value |
|----|------------|---------------|

## Returns

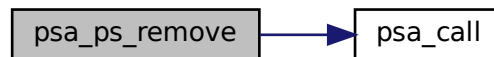
A status indicating the success/failure of the operation

## Return values

|                                   |                                                                                                                    |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                               |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                   |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was created with <i>PSA_STORAGE_FLAG_WRITE_ONCE</i>            |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                         |
| <i>PSA_ERROR_GENERIC_ERROR</i>    | The operation failed because of an unspecified internal failure                                                    |

Definition at line 103 of file `tfm_ps_api.c`.

Here is the call graph for this function:

8.50.2.6 `psa_ps_set()`

```

psa_status_t psa_ps_set (
 psa_storage_uid_t uid,
 size_t data_length,
 const void * p_data,
 psa_storage_create_flags_t create_flags)

```

Create a new, or modify an existing, uid/value pair.  
Stores data in the protected storage.

## Parameters

|    |                     |                                                |
|----|---------------------|------------------------------------------------|
| in | <i>uid</i>          | The identifier for the data                    |
| in | <i>data_length</i>  | The size in bytes of the data in <i>p_data</i> |
| in | <i>p_data</i>       | A buffer containing the data                   |
| in | <i>create_flags</i> | The flags that the data will be stored with    |

**Returns**

A status indicating the success/failure of the operation

**Return values**

|                                       |                                                                                                                                                                             |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                                                                                        |
| <i>PSA_ERROR_NOT_PERMITTED</i>        | The operation failed because the provided <code>uid</code> value was already created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code>                                          |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because one of the provided pointers( <code>p_data</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because one or more of the flags provided in <code>create_flags</code> is not supported or is not valid                                                |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because there was insufficient space on the storage medium                                                                                             |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (Fatal error)                                                                                                  |
| <i>PSA_ERROR_GENERIC_ERROR</i>        | The operation failed because of an unspecified internal failure                                                                                                             |

Definition at line 19 of file `tfm_ps_api.c`.

Here is the call graph for this function:

**8.50.2.7 psa\_ps\_set\_extended()**

```

psa_status_t psa_ps_set_extended (
 psa_storage_uid_t uid,
 size_t data_offset,
 size_t data_length,
 const void * p_data)

```

Sets partial data into an asset.

Before calling this function, the storage must have been reserved with a call to `psa_ps_create`. It can also be used to overwrite data in an asset that was created with a call to `psa_ps_set`. Calling this function with `data_length = 0` is permitted, which will make no change to the stored data. This function can overwrite existing data and/or extend it up to the capacity for the `uid` specified in `psa_ps_create`, but cannot create gaps.

That is, it has preconditions:

- `data_offset <= size`
- `data_offset + data_length <= capacity` and postconditions:
- `size = max(size, data_offset + data_length)`
- capacity unchanged.

## Parameters

|    |                    |                                                               |
|----|--------------------|---------------------------------------------------------------|
| in | <i>uid</i>         | The <code>uid</code> value                                    |
| in | <i>data_offset</i> | Offset within the asset to start the write                    |
| in | <i>data_length</i> | The size in bytes of the data in <code>p_data</code> to write |
| in | <i>p_data</i>      | Pointer to a buffer which contains the data to write          |

## Returns

A status indicating the success/failure of the operation

## Return values

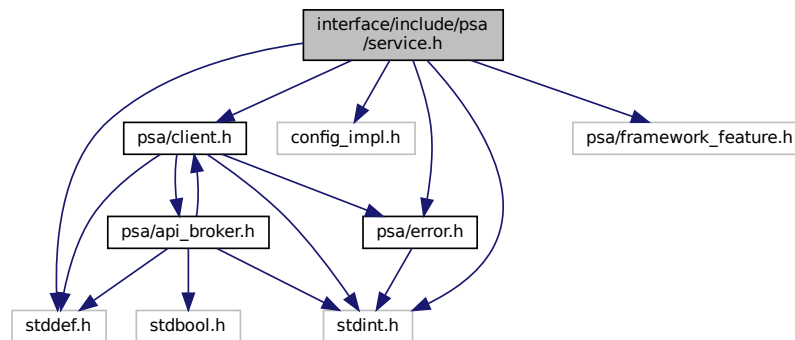
|                                    |                                                                                                                                                                 |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                 | The asset exists, the input parameters are correct and the data is correctly written in the physical storage.                                                   |
| <i>PSA_ERROR_STORAGE_FAILURE</i>   | The data was not written correctly in the physical storage                                                                                                      |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>  | The operation failed because one or more of the preconditions listed above regarding <code>data_offset</code> , size, or <code>data_length</code> was violated. |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>    | The specified <code>uid</code> was not found                                                                                                                    |
| <i>PSA_ERROR_NOT_SUPPORTED</i>     | The implementation of the API does not support this function                                                                                                    |
| <i>PSA_ERROR_GENERIC_ERROR</i>     | The operation failed due to an unspecified error                                                                                                                |
| <i>PSA_ERROR_DATA_CORRUPT</i>      | The operation failed because the existing data has been corrupted.                                                                                              |
| <i>PSA_ERROR_INVALID_SIGNATURE</i> | The operation failed because the existing data failed authentication (MAC check failed).                                                                        |
| <i>PSA_ERROR_NOT_PERMITTED</i>     | The operation failed because it was attempted on an asset which was written with the flag <code>PSA_STORAGE_FLAG_WRITE_ONCE</code>                              |

Definition at line 127 of file `tfm_ps_api.c`.

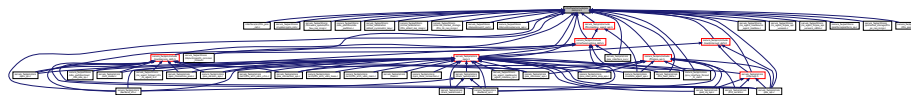
## 8.51 interface/include/psa/service.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "config_impl.h"
#include "psa/client.h"
#include "psa/error.h"
#include "psa/framework_feature.h"
```

Include dependency graph for service.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [psa\\_msg\\_t](#)

## Macros

- #define [PSA\\_POLL](#) (0x00000000u)
- #define [PSA\\_BLOCK](#) (0x80000000u)
- #define [PSA\\_WAIT\\_ANY](#) (0xFFFFFFFFu)
- #define [PSA\\_DOORBELL](#) (0x00000008u)
- #define [PSA\\_IPC\\_CONNECT](#) (-1)
- #define [PSA\\_IPC\\_DISCONNECT](#) (-2)
- #define [PSA\\_FLIH\\_NO\\_SIGNAL](#) (([psa\\_flih\\_result\\_t](#)) 0)
- #define [PSA\\_FLIH\\_SIGNAL](#) (([psa\\_flih\\_result\\_t](#)) 1)

## Typedefs

- typedef uint32\_t [psa\\_signal\\_t](#)
- typedef uint32\_t [psa\\_irq\\_status\\_t](#)
- typedef uint32\_t [psa\\_flih\\_result\\_t](#)
- typedef struct [psa\\_msg\\_t](#) [psa\\_msg\\_t](#)

## Functions

- [psa\\_signal\\_t](#) [psa\\_wait](#) ([psa\\_signal\\_t](#) signal\_mask, uint32\_t timeout)  
*Return the Secure Partition interrupt signals that have been asserted from a subset of signals provided by the caller.*
- [psa\\_status\\_t](#) [psa\\_get](#) ([psa\\_signal\\_t](#) signal, [psa\\_msg\\_t](#) \*msg)  
*Retrieve the message which corresponds to a given RoT Service signal and remove the message from the RoT Service queue.*
- void [psa\\_set\\_rhandle](#) ([psa\\_handle\\_t](#) msg\_handle, void \*rhandle)  
*Associate some RoT Service private data with a client connection.*

- `size_t psa_read (psa_handle_t msg_handle, uint32_t invec_idx, void *buffer, size_t num_bytes)`  
*Read a message parameter or part of a message parameter from a client input vector.*
- `size_t psa_skip (psa_handle_t msg_handle, uint32_t invec_idx, size_t num_bytes)`  
*Skip over part of a client input vector.*
- `void psa_write (psa_handle_t msg_handle, uint32_t outvec_idx, const void *buffer, size_t num_bytes)`  
*Write a message response to a client output vector.*
- `void psa_reply (psa_handle_t msg_handle, psa_status_t status)`  
*Complete handling of a specific message and unblock the client.*
- `void psa_notify (int32_t partition_id)`  
*Send a PSA\_DOORBELL signal to a specific Secure Partition.*
- `void psa_clear (void)`  
*Clear the PSA\_DOORBELL signal.*
- `void psa_eoi (psa_signal_t irq_signal)`  
*Inform the SPM that an interrupt has been handled (end of interrupt).*
- `void psa_panic (void)`  
*Terminate execution within the calling Secure Partition and will not return.*
- `void psa_irq_enable (psa_signal_t irq_signal)`  
*Enable an interrupt.*
- `psa_irq_status_t psa_irq_disable (psa_signal_t irq_signal)`  
*Disable an interrupt and return the status of the interrupt prior to being disabled by this call.*
- `void psa_reset_signal (psa_signal_t irq_signal)`  
*Reset the signal for an interrupt that is using FLIH handling.*

## 8.51.1 Macro Definition Documentation

### 8.51.1.1 PSA\_BLOCK

```
#define PSA_BLOCK (0x80000000u)
```

A timeout value that requests a blocking wait operation.

Definition at line 34 of file service.h.

### 8.51.1.2 PSA\_DOORBELL

```
#define PSA_DOORBELL (0x00000008u)
```

The signal number for the Secure Partition doorbell.

Definition at line 44 of file service.h.

### 8.51.1.3 PSA\_FLIH\_NO\_SIGNAL

```
#define PSA_FLIH_NO_SIGNAL ((psa_flih_result_t) 0)
```

Definition at line 53 of file service.h.

### 8.51.1.4 PSA\_FLIH\_SIGNAL

```
#define PSA_FLIH_SIGNAL ((psa_flih_result_t) 1)
```

Definition at line 54 of file service.h.

#### 8.51.1.5 PSA\_IPC\_CONNECT

```
#define PSA_IPC_CONNECT (-1)
```

Definition at line 48 of file service.h.

#### 8.51.1.6 PSA\_IPC\_DISCONNECT

```
#define PSA_IPC_DISCONNECT (-2)
```

Definition at line 50 of file service.h.

#### 8.51.1.7 PSA\_POLL

```
#define PSA_POLL (0x00000000u)
```

A timeout value that requests a polling wait operation.  
Definition at line 29 of file service.h.

#### 8.51.1.8 PSA\_WAIT\_ANY

```
#define PSA_WAIT_ANY (0xFFFFFFFFu)
```

A mask value that includes all Secure Partition signals.  
Definition at line 39 of file service.h.

### 8.51.2 Typedef Documentation

#### 8.51.2.1 psa\_flih\_result\_t

```
typedef uint32_t psa_flih_result_t
```

Definition at line 63 of file service.h.

#### 8.51.2.2 psa\_irq\_status\_t

```
typedef uint32_t psa_irq_status_t
```

Definition at line 60 of file service.h.

#### 8.51.2.3 psa\_msg\_t

```
typedef struct psa_msg_t psa_msg_t
```

Describe a message received by an RoT Service after calling [psa\\_get\(\)](#).

#### 8.51.2.4 psa\_signal\_t

```
typedef uint32_t psa_signal_t
```

Definition at line 57 of file service.h.

### 8.51.3 Function Documentation

#### 8.51.3.1 psa\_clear()

```
void psa_clear (
```

[void](#) )

Clear the PSA\_DOORBELL signal.

**Note**

The call is a "PROGRAMMER ERROR" if the Secure Partition's doorbell signal is not currently asserted.

**8.51.3.2 psa\_eoi()**

```
void psa_eoi (
 psa_signal_t irq_signal)
```

Inform the SPM that an interrupt has been handled (end of interrupt).

**Parameters**

|    |                   |                                               |
|----|-------------------|-----------------------------------------------|
| in | <i>irq_signal</i> | The interrupt signal that has been processed. |
|----|-------------------|-----------------------------------------------|

**Note**

The call is a "PROGRAMMER ERROR" if one or more of the following occur:

- *irq\_signal* is not an interrupt signal.
- *irq\_signal* indicates more than one signal.
- *irq\_signal* is not currently asserted.
- The interrupt is not using SLIH.

**8.51.3.3 psa\_get()**

```
psa_status_t psa_get (
 psa_signal_t signal,
 psa_msg_t * msg)
```

Retrieve the message which corresponds to a given RoT Service signal and remove the message from the RoT Service queue.

**Parameters**

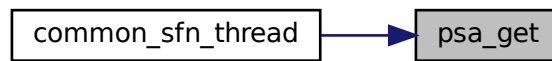
|     |               |                                                                        |
|-----|---------------|------------------------------------------------------------------------|
| in  | <i>signal</i> | The signal value for an asserted RoT Service.                          |
| out | <i>msg</i>    | Pointer to <a href="#">psa_msg_t</a> object for receiving the message. |

**Return values**

|                                 |                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>              | Success, *msg will contain the delivered message.                                                                                                                                                                                                                                                                                                               |
| <i>PSA_ERROR_DOES_NOT_EXIST</i> | Message could not be delivered.                                                                                                                                                                                                                                                                                                                                 |
| <i>PROGRAMMER ERROR</i>         | <p>The call is invalid because one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• signal has more than a single bit set.</li> <li>• signal does not correspond to an RoT Service.</li> <li>• The RoT Service signal is not currently asserted.</li> <li>• The msg pointer provided is not a valid memory reference.</li> </ul> |

Definition at line 44 of file `psa_api_ipc.c`.

Here is the caller graph for this function:



#### 8.51.3.4 psa\_irq\_disable()

```

psa_irq_status_t psa_irq_disable (
 psa_signal_t irq_signal)

```

Disable an interrupt and return the status of the interrupt prior to being disabled by this call.

##### Parameters

|    |                   |                                                                                                                                                                |
|----|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>irq_signal</i> | The signal for the interrupt to be disabled. This must have a single bit set, which must be the signal value for an interrupt in the calling Secure Partition. |
|----|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|

##### Return values

|                  |                                                                                                                                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0                | The interrupt was disabled prior to this call. 1 The interrupt was enabled prior to this call.                                                                                                           |
| PROGRAMMER ERROR | If one or more of the following are true: <ul style="list-style-type: none"> <li>• <i>irq_signal</i> is not an interrupt signal.</li> <li>• <i>irq_signal</i> indicates more than one signal.</li> </ul> |

##### Note

The current implementation always return 1. Do not use the return.

#### 8.51.3.5 psa\_irq\_enable()

```

void psa_irq_enable (
 psa_signal_t irq_signal)

```

Enable an interrupt.

##### Parameters

|    |                   |                                                                                                                                                               |
|----|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>irq_signal</i> | The signal for the interrupt to be enabled. This must have a single bit set, which must be the signal value for an interrupt in the calling Secure Partition. |
|----|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|

##### Note

The call is a "PROGRAMMER ERROR" if one or more of the following occur:

- *irq\_signal* is not an interrupt signal.



- *irq\_signal* indicates more than one signal.

### 8.51.3.6 `psa_notify()`

```
void psa_notify (
 int32_t partition_id)
```

Send a PSA\_DOORBELL signal to a specific Secure Partition.

#### Parameters

|    |                     |                                              |
|----|---------------------|----------------------------------------------|
| in | <i>partition_id</i> | Secure Partition ID of the target partition. |
|----|---------------------|----------------------------------------------|

#### Note

The call is a "PROGRAMMER ERROR" if *partition\_id* does not correspond to a Secure Partition.

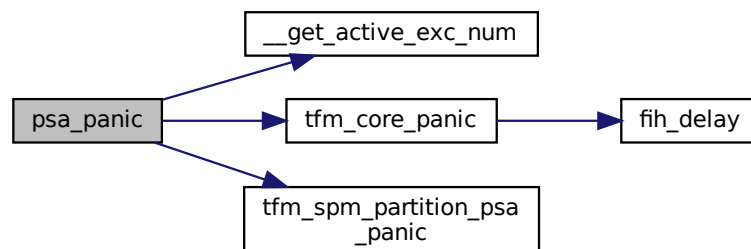
### 8.51.3.7 `psa_panic()`

```
void psa_panic (
 void)
```

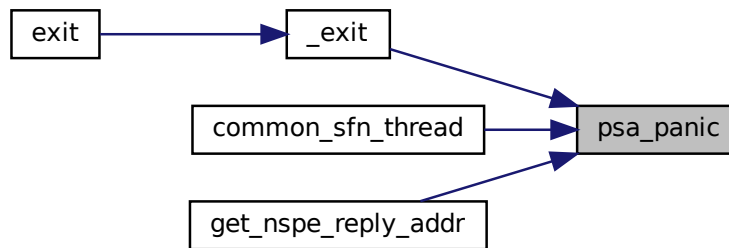
Terminate execution within the calling Secure Partition and will not return.

Definition at line 71 of file `psa_api_ipc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.51.3.8 psa\_read()

```

size_t psa_read (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 void * buffer,
 size_t num_bytes)

```

Read a message parameter or part of a message parameter from a client input vector.

#### Parameters

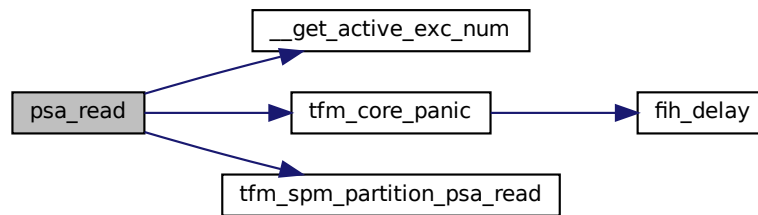
|     |                   |                                                                                           |
|-----|-------------------|-------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                          |
| in  | <i>invec_idx</i>  | Index of the input vector to read from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| out | <i>buffer</i>     | Buffer in the Secure Partition to copy the requested data to.                             |
| in  | <i>num_bytes</i>  | Maximum number of bytes to be read from the client input vector.                          |

#### Return values

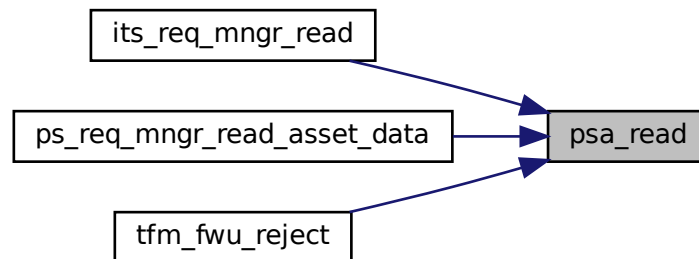
|                         |                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>&gt;0</i>            | Number of bytes copied.                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>0</i>                | There was no remaining data in this input vector.                                                                                                                                                                                                                                                                                                                                                                |
| <i>PROGRAMMER ERROR</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• <i>msg_handle</i> is invalid.</li> <li>• <i>msg_handle</i> does not refer to a <a href="#">PSA_IPC_CALL</a> message.</li> <li>• <i>invec_idx</i> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> <li>• the memory reference for <i>buffer</i> is invalid or not writable.</li> </ul> |

Definition at line 49 of file `psa_api_ipc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.51.3.9 psa\_reply()

```

void psa_reply (
 psa_handle_t msg_handle,
 psa_status_t status)

```

Complete handling of a specific message and unblock the client.

#### Parameters

|    |                   |                                                    |
|----|-------------------|----------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                   |
| in | <i>status</i>     | Message result value to be reported to the client. |

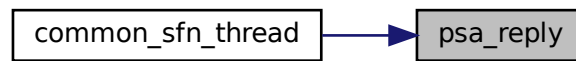
#### Note

The call is a "PROGRAMMER ERROR" if one or more of the following occur:

- `msg_handle` is invalid.
- An invalid status code is specified for the type of message.

Definition at line 66 of file `psa_api_ipc.c`.

Here is the caller graph for this function:



#### 8.51.3.10 psa\_reset\_signal()

```
void psa_reset_signal (
 psa_signal_t irq_signal)
```

Reset the signal for an interrupt that is using FLIH handling.

##### Parameters

|    |                   |                                                                                                                                                                        |
|----|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>irq_signal</i> | The interrupt signal to be reset. This must have a single bit set, corresponding to a currently asserted signal for an interrupt that is defined to use FLIH handling. |
|----|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

##### Note

The call is a "PROGRAMMER ERROR" if one or more of the following occur:

- *irq\_signal* is not a signal for an interrupt that is specified with FLIH handling in the Secure Partition manifest.
- *irq\_signal* indicates more than one signal.
- *irq\_signal* is not currently asserted.

#### 8.51.3.11 psa\_set\_rhandle()

```
void psa_set_rhandle (
 psa_handle_t msg_handle,
 void * rhandle)
```

Associate some RoT Service private data with a client connection.

##### Parameters

|    |                   |                                              |
|----|-------------------|----------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.             |
| in | <i>rhandle</i>    | Reverse handle allocated by the RoT Service. |

##### Note

When successful, *rhandle* will be provided with all subsequent messages delivered on this connection.

The call is a "PROGRAMMER ERROR" if *msg\_handle* is invalid.

#### 8.51.3.12 psa\_skip()

```
size_t psa_skip (
```

```

psa_handle_t msg_handle,
uint32_t invec_idx,
size_t num_bytes)

```

Skip over part of a client input vector.

#### Parameters

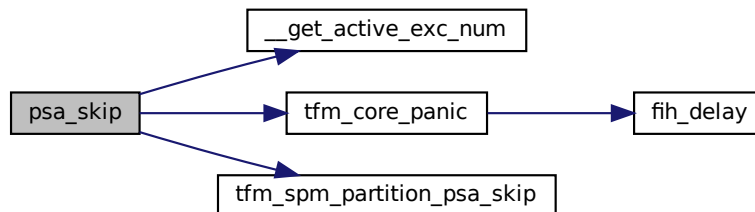
|    |                   |                                                                                       |
|----|-------------------|---------------------------------------------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                                                      |
| in | <i>invec_idx</i>  | Index of input vector to skip from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in | <i>num_bytes</i>  | Maximum number of bytes to skip in the client input vector.                           |

#### Return values

|                         |                                                                                                                                                                                                                                                                                                        |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $>0$                    | Number of bytes skipped.                                                                                                                                                                                                                                                                               |
| $0$                     | There was no remaining data in this input vector.                                                                                                                                                                                                                                                      |
| <i>PROGRAMMER ERROR</i> | The call is invalid, one or more of the following are true: <ul style="list-style-type: none"> <li>• <i>msg_handle</i> is invalid.</li> <li>• <i>msg_handle</i> does not refer to a request message.</li> <li>• <i>invec_idx</i> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> </ul> |

Definition at line 55 of file `psa_api_ipc.c`.

Here is the call graph for this function:



#### 8.51.3.13 psa\_wait()

```

psa_signal_t psa_wait (
 psa_signal_t signal_mask,
 uint32_t timeout)

```

Return the Secure Partition interrupt signals that have been asserted from a subset of signals provided by the caller.

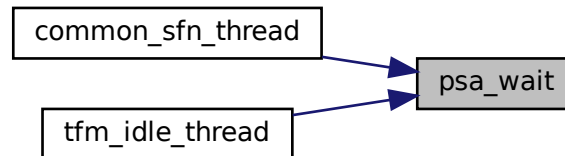
#### Parameters

|    |                    |                                                                                                  |
|----|--------------------|--------------------------------------------------------------------------------------------------|
| in | <i>signal_mask</i> | A set of signals to query. Signals that are not in this set will be ignored.                     |
| in | <i>timeout</i>     | Specify either blocking <a href="#">PSA_BLOCK</a> or polling <a href="#">PSA_POLL</a> operation. |

## Return values

|                    |                                                                            |
|--------------------|----------------------------------------------------------------------------|
| <code>&gt;0</code> | At least one signal is asserted.                                           |
| <code>0</code>     | No signals are asserted. This is only seen when a polling timeout is used. |

Definition at line 39 of file `psa_api_ipc.c`.  
Here is the caller graph for this function:

8.51.3.14 `psa_write()`

```

void psa_write (
 psa_handle_t msg_handle,
 uint32_t outvec_idx,
 const void * buffer,
 size_t num_bytes)

```

Write a message response to a client output vector.

## Parameters

|     |                         |                                                                                                  |
|-----|-------------------------|--------------------------------------------------------------------------------------------------|
| in  | <code>msg_handle</code> | Handle for the client's message.                                                                 |
| out | <code>outvec_idx</code> | Index of output vector in message to write to. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in  | <code>buffer</code>     | Buffer with the data to write.                                                                   |
| in  | <code>num_bytes</code>  | Number of bytes to write to the client output vector.                                            |

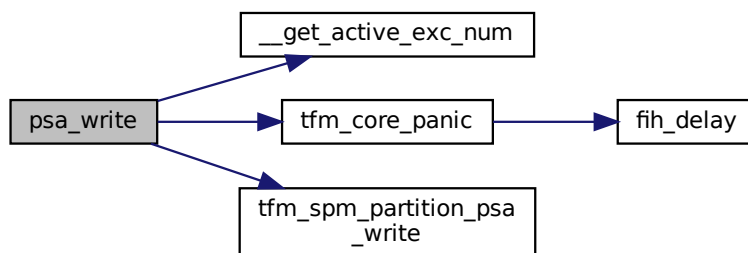
## Note

The call is a "PROGRAMMER ERROR" if one or more of the following occur:

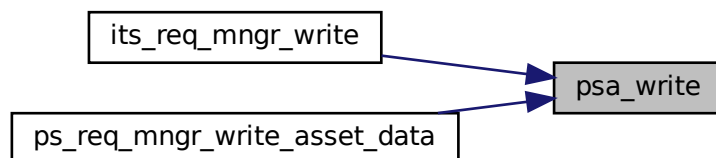
- `msg_handle` is invalid.
- `msg_handle` does not refer to a request message.
- `outvec_idx` is equal to or greater than [PSA\\_MAX\\_IOVEC](#).
- the memory reference for `buffer` is invalid.
- the call attempts to write data past the end of the client output vector.

Definition at line 60 of file `psa_api_ipc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

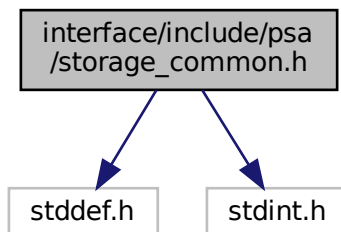


## 8.52 interface/include/psa/storage\_common.h File Reference

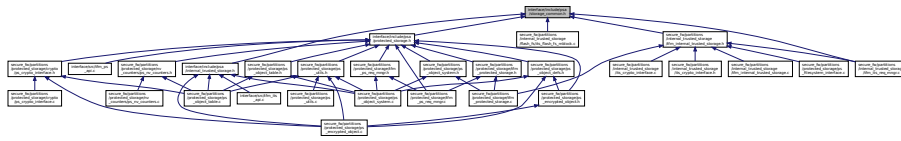
```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for `storage_common.h`:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [psa\\_storage\\_info\\_t](#)

## Macros

- `#define PSA_STORAGE_FLAG_NONE 0u`
- `#define PSA_STORAGE_FLAG_WRITE_ONCE (1u << 0)`
- `#define PSA_STORAGE_FLAG_NO_CONFIDENTIALITY (1u << 1)`
- `#define PSA_STORAGE_FLAG_NO_REPLAY_PROTECTION (1u << 2)`
- `#define PSA_STORAGE_SUPPORT_SET_EXTENDED (1u << 0)`
- `#define PSA_ERROR_INVALID_SIGNATURE ((psa_status_t)-149)`
- `#define PSA_ERROR_DATA_CORRUPT ((psa_status_t)-152)`

## Typedefs

- `typedef uint32_t psa_storage_create_flags_t`
- `typedef uint64_t psa_storage_uid_t`

## 8.52.1 Macro Definition Documentation

### 8.52.1.1 PSA\_ERROR\_DATA\_CORRUPT

```
#define PSA_ERROR_DATA_CORRUPT ((psa_status_t)-152)
```

Definition at line 43 of file `storage_common.h`.

### 8.52.1.2 PSA\_ERROR\_INVALID\_SIGNATURE

```
#define PSA_ERROR_INVALID_SIGNATURE ((psa_status_t)-149)
```

Definition at line 42 of file `storage_common.h`.

### 8.52.1.3 PSA\_STORAGE\_FLAG\_NO\_CONFIDENTIALITY

```
#define PSA_STORAGE_FLAG_NO_CONFIDENTIALITY (1u << 1)
```

Definition at line 29 of file `storage_common.h`.

### 8.52.1.4 PSA\_STORAGE\_FLAG\_NO\_REPLAY\_PROTECTION

```
#define PSA_STORAGE_FLAG_NO_REPLAY_PROTECTION (1u << 2)
```

Definition at line 30 of file `storage_common.h`.



### 8.52.1.5 PSA\_STORAGE\_FLAG\_NONE

```
#define PSA_STORAGE_FLAG_NONE 0u
```

Definition at line 27 of file storage\_common.h.

### 8.52.1.6 PSA\_STORAGE\_FLAG\_WRITE\_ONCE

```
#define PSA_STORAGE_FLAG_WRITE_ONCE (1u << 0)
```

Definition at line 28 of file storage\_common.h.

### 8.52.1.7 PSA\_STORAGE\_SUPPORT\_SET\_EXTENDED

```
#define PSA_STORAGE_SUPPORT_SET_EXTENDED (1u << 0)
```

Definition at line 40 of file storage\_common.h.

## 8.52.2 Typedef Documentation

### 8.52.2.1 psa\_storage\_create\_flags\_t

```
typedef uint32_t psa_storage_create_flags_t
```

Definition at line 21 of file storage\_common.h.

### 8.52.2.2 psa\_storage\_uid\_t

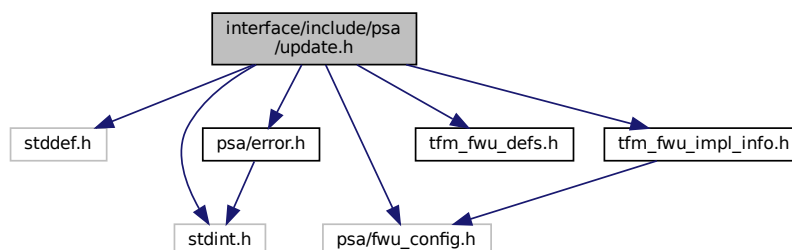
```
typedef uint64_t psa_storage_uid_t
```

Definition at line 23 of file storage\_common.h.

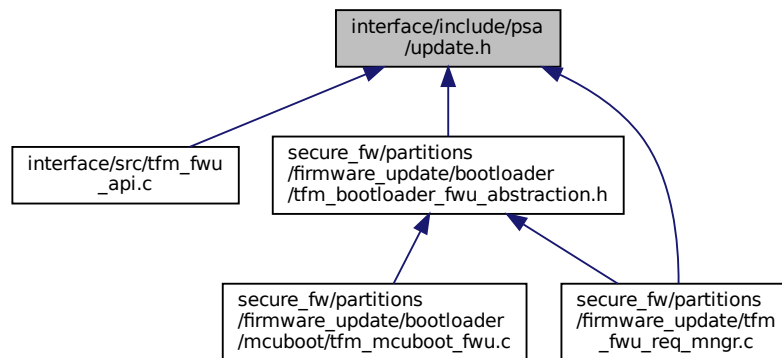
## 8.53 interface/include/psa/update.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "psa/error.h"
#include "psa/fwu_config.h"
#include "tfm_fwu_defs.h"
#include "tfm_fwu_impl_info.h"
```

Include dependency graph for update.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [psa\\_fwu\\_image\\_version\\_t](#)  
Version information about a firmware image.
- struct [psa\\_fwu\\_component\\_info\\_t](#)  
Information about the firmware store for a firmware component.

## Macros

- #define [PSA\\_FWU\\_API\\_VERSION\\_MAJOR](#) 1  
The major version of this implementation of the Firmware Update API.
- #define [PSA\\_FWU\\_API\\_VERSION\\_MINOR](#) 0  
The minor version of this implementation of the Firmware Update API.
- #define [PSA\\_ERROR\\_DEPENDENCY\\_NEEDED](#) (([psa\\_status\\_t](#))-156)  
A status code that indicates that the firmware of another component requires updating.
- #define [PSA\\_ERROR\\_FLASH\\_ABUSE](#) (([psa\\_status\\_t](#))-160)  
A status code that indicates that the system is limiting i/o operations to avoid rapid flash exhaustion.
- #define [PSA\\_ERROR\\_INSUFFICIENT\\_POWER](#) (([psa\\_status\\_t](#))-161)  
A status code that indicates that the system does not have enough power to carry out the request.
- #define [PSA\\_SUCCESS\\_REBOOT](#) (([psa\\_status\\_t](#))+1)  
The action was completed successfully and requires a system reboot to complete installation.
- #define [PSA\\_SUCCESS\\_RESTART](#) (([psa\\_status\\_t](#))+2)  
The action was completed successfully and requires a restart of the component to complete installation.
- #define [PSA\\_FWU\\_READY](#) 0u  
The READY state: the component is ready to start another update.
- #define [PSA\\_FWU\\_WRITING](#) 1u  
The WRITING state: a new firmware image is being written to the firmware store.
- #define [PSA\\_FWU\\_CANDIDATE](#) 2u  
The CANDIDATE state: a new firmware image is ready for installation.
- #define [PSA\\_FWU\\_STAGED](#) 3u  
The STAGED state: a new firmware image is queued for installation.
- #define [PSA\\_FWU\\_FAILED](#) 4u  
The FAILED state: a firmware update has been cancelled or has failed.
- #define [PSA\\_FWU\\_TRIAL](#) 5u

- The *TRIAL* state: a new firmware image requires testing prior to acceptance of the update.
- `#define PSA_FWU_REJECTED 6u`  
The *REJECTED* state: a new firmware image has been rejected after testing.
- `#define PSA_FWU_UPDATED 7u`  
The *UPDATED* state: a firmware update has been successful, and the new image is now active.
- `#define PSA_FWU_FLAG_VOLATILE_STAGING 0x00000001u`  
Flag to indicate whether the image data in the component staging area is discarded at system reset.
- `#define PSA_FWU_FLAG_ENCRYPTION 0x00000002u`  
Flag to indicate whether a firmware component expects encrypted images during an update.
- `#define PSA_FWU_MAX_WRITE_SIZE TFM_CONFIG_FWU_MAX_WRITE_SIZE`  
The maximum permitted size for block in `psa_fwu_write()`, in bytes.

## Typedefs

- `typedef uint8_t psa_fwu_component_t`  
Firmware component type identifier.
- `typedef struct psa_fwu_image_version_t psa_fwu_image_version_t`  
Version information about a firmware image.
- `typedef struct psa_fwu_component_info_t psa_fwu_component_info_t`  
Information about the firmware store for a firmware component.

## Functions

- `psa_status_t psa_fwu_query (psa_fwu_component_t component, psa_fwu_component_info_t *info)`  
Retrieve the firmware store information for a specific firmware component.
- `psa_status_t psa_fwu_start (psa_fwu_component_t component, const void *manifest, size_t manifest_size)`  
Begin a firmware update operation for a specific firmware component.
- `psa_status_t psa_fwu_write (psa_fwu_component_t component, size_t image_offset, const void *block, size_t block_size)`  
Write a firmware image, or part of a firmware image, to its staging area.
- `psa_status_t psa_fwu_finish (psa_fwu_component_t component)`  
Mark a firmware image in the staging area as ready for installation.
- `psa_status_t psa_fwu_cancel (psa_fwu_component_t component)`  
Abandon an update that is in *WRITING* or *CANDIDATE* state.
- `psa_status_t psa_fwu_clean (psa_fwu_component_t component)`  
Prepare the component for another update.
- `psa_status_t psa_fwu_install (void)`  
Start the installation of all firmware images that have been prepared for update.
- `psa_status_t psa_fwu_request_reboot (void)`  
Requests the platform to reboot.
- `psa_status_t psa_fwu_reject (psa_status_t error)`  
Abandon an installation that is in *STAGED* or *TRIAL* state.
- `psa_status_t psa_fwu_accept (void)`  
Accept a firmware update that is currently in *TRIAL* state.

### 8.53.1 Macro Definition Documentation

#### 8.53.1.1 PSA\_ERROR\_DEPENDENCY\_NEEDED

```
#define PSA_ERROR_DEPENDENCY_NEEDED ((psa_status_t)-156)
```

A status code that indicates that the firmware of another component requires updating.

Definition at line 47 of file `update.h`.

#### 8.53.1.2 PSA\_ERROR\_FLASH\_ABUSE

```
#define PSA_ERROR_FLASH_ABUSE ((psa_status_t)-160)
```

A status code that indicates that the system is limiting i/o operations to avoid rapid flash exhaustion.

Definition at line 53 of file update.h.

#### 8.53.1.3 PSA\_ERROR\_INSUFFICIENT\_POWER

```
#define PSA_ERROR_INSUFFICIENT_POWER ((psa_status_t)-161)
```

A status code that indicates that the system does not have enough power to carry out the request.

Definition at line 59 of file update.h.

#### 8.53.1.4 PSA\_FWU\_API\_VERSION\_MAJOR

```
#define PSA_FWU_API_VERSION_MAJOR 1
```

The major version of this implementation of the Firmware Update API.

Definition at line 36 of file update.h.

#### 8.53.1.5 PSA\_FWU\_API\_VERSION\_MINOR

```
#define PSA_FWU_API_VERSION_MINOR 0
```

The minor version of this implementation of the Firmware Update API.

Definition at line 41 of file update.h.

#### 8.53.1.6 PSA\_FWU\_CANDIDATE

```
#define PSA_FWU_CANDIDATE 2u
```

The CANDIDATE state: a new firmware image is ready for installation.

Definition at line 102 of file update.h.

#### 8.53.1.7 PSA\_FWU\_FAILED

```
#define PSA_FWU_FAILED 4u
```

The FAILED state: a firmware update has been cancelled or has failed.

Definition at line 112 of file update.h.

#### 8.53.1.8 PSA\_FWU\_FLAG\_ENCRYPTION

```
#define PSA_FWU_FLAG_ENCRYPTION 0x00000002u
```

Flag to indicate whether a firmware component expects encrypted images during an update.

Definition at line 142 of file update.h.

#### 8.53.1.9 PSA\_FWU\_FLAG\_VOLATILE\_STAGING

```
#define PSA_FWU_FLAG_VOLATILE_STAGING 0x00000001u
```

Flag to indicate whether the image data in the component staging area is discarded at system reset.

Definition at line 136 of file update.h.

#### 8.53.1.10 PSA\_FWU\_MAX\_WRITE\_SIZE

```
#define PSA_FWU_MAX_WRITE_SIZE TFM_CONFIG_FWU_MAX_WRITE_SIZE
```

The maximum permitted size for block in [psa\\_fwu\\_write\(\)](#), in bytes.

Definition at line 186 of file update.h.

#### 8.53.1.11 PSA\_FWU\_READY

```
#define PSA_FWU_READY 0u
```

The READY state: the component is ready to start another update.

Definition at line 91 of file update.h.

#### 8.53.1.12 PSA\_FWU\_REJECTED

```
#define PSA_FWU_REJECTED 6u
```

The REJECTED state: a new firmware image has been rejected after testing.

Definition at line 124 of file update.h.

#### 8.53.1.13 PSA\_FWU\_STAGED

```
#define PSA_FWU_STAGED 3u
```

The STAGED state: a new firmware image is queued for installation.

Definition at line 107 of file update.h.

#### 8.53.1.14 PSA\_FWU\_TRIAL

```
#define PSA_FWU_TRIAL 5u
```

The TRIAL state: a new firmware image requires testing prior to acceptance of the update.

Definition at line 118 of file update.h.

#### 8.53.1.15 PSA\_FWU\_UPDATED

```
#define PSA_FWU_UPDATED 7u
```

The UPDATED state: a firmware update has been successful, and the new image is now active.

Definition at line 130 of file update.h.

#### 8.53.1.16 PSA\_FWU\_WRITING

```
#define PSA_FWU_WRITING 1u
```

The WRITING state: a new firmware image is being written to the firmware store.

Definition at line 97 of file update.h.

#### 8.53.1.17 PSA\_SUCCESS\_REBOOT

```
#define PSA_SUCCESS_REBOOT ((psa_status_t)+1)
```

The action was completed successfully and requires a system reboot to complete installation.

Definition at line 65 of file update.h.

#### 8.53.1.18 PSA\_SUCCESS\_RESTART

```
#define PSA_SUCCESS_RESTART ((psa_status_t)+2)
```

The action was completed successfully and requires a restart of the component to complete installation.

Definition at line 71 of file update.h.

## 8.53.2 Typedef Documentation

### 8.53.2.1 `psa_fwu_component_info_t`

typedef struct `psa_fwu_component_info_t` `psa_fwu_component_info_t`  
Information about the firmware store for a firmware component.

### 8.53.2.2 `psa_fwu_component_t`

typedef uint8\_t `psa_fwu_component_t`  
Firmware component type identifier.  
Definition at line 76 of file `update.h`.

### 8.53.2.3 `psa_fwu_image_version_t`

typedef struct `psa_fwu_image_version_t` `psa_fwu_image_version_t`  
Version information about a firmware image.

## 8.53.3 Function Documentation

### 8.53.3.1 `psa_fwu_accept()`

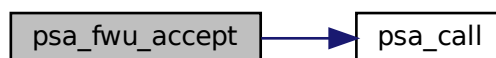
```
psa_status_t psa_fwu_accept (
 void)
```

Accept a firmware update that is currently in TRIAL state.

#### Returns

Result status.

Definition at line 96 of file `tfm_fwu_api.c`.  
Here is the call graph for this function:



### 8.53.3.2 `psa_fwu_cancel()`

```
psa_status_t psa_fwu_cancel (
 psa_fwu_component_t component)
```

Abandon an update that is in WRITING or CANDIDATE state.

#### Parameters

|                  |                                                       |
|------------------|-------------------------------------------------------|
| <i>component</i> | Identifier of the firmware component to be cancelled. |
|------------------|-------------------------------------------------------|

**Returns**

Result status.

Definition at line 56 of file tfm\_fwu\_api.c.

Here is the call graph for this function:

**8.53.3.3 psa\_fwu\_clean()**

```
psa_status_t psa_fwu_clean (
 psa_fwu_component_t component)
```

Prepare the component for another update.

**Parameters**

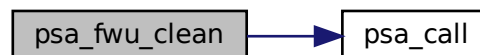
|                  |                                                  |
|------------------|--------------------------------------------------|
| <i>component</i> | Identifier of the firmware component to tidy up. |
|------------------|--------------------------------------------------|

**Returns**

Result status.

Definition at line 66 of file tfm\_fwu\_api.c.

Here is the call graph for this function:

**8.53.3.4 psa\_fwu\_finish()**

```
psa_status_t psa_fwu_finish (
 psa_fwu_component_t component)
```

Mark a firmware image in the staging area as ready for installation.

**Parameters**

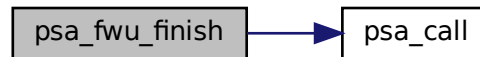
|                  |                                                  |
|------------------|--------------------------------------------------|
| <i>component</i> | Identifier of the firmware component to install. |
|------------------|--------------------------------------------------|

**Returns**

Result status.

Definition at line 40 of file `tfm_fwu_api.c`.

Here is the call graph for this function:

**8.53.3.5 psa\_fwu\_install()**

```
psa_status_t psa_fwu_install (
 void)
```

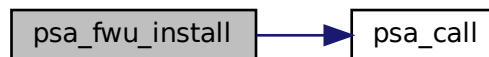
Start the installation of all firmware images that have been prepared for update.

**Returns**

Result status.

Definition at line 50 of file `tfm_fwu_api.c`.

Here is the call graph for this function:

**8.53.3.6 psa\_fwu\_query()**

```
psa_status_t psa_fwu_query (
 psa_fwu_component_t component,
 psa_fwu_component_info_t * info)
```

Retrieve the firmware store information for a specific firmware component.

**Parameters**

|                  |                                                        |
|------------------|--------------------------------------------------------|
| <i>component</i> | Firmware component for which information is requested. |
| <i>info</i>      | Output parameter for component information.            |

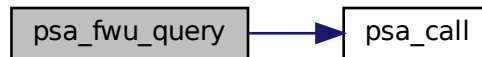


**Returns**

Result status.

Definition at line 76 of file tfm\_fwu\_api.c.

Here is the call graph for this function:

**8.53.3.7 psa\_fwu\_reject()**

```
psa_status_t psa_fwu_reject (
 psa_status_t error)
```

Abandon an installation that is in STAGED or TRIAL state.

**Parameters**

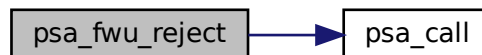
|              |                                                               |
|--------------|---------------------------------------------------------------|
| <i>error</i> | An application-specific error code chosen by the application. |
|--------------|---------------------------------------------------------------|

**Returns**

Result status.

Definition at line 102 of file tfm\_fwu\_api.c.

Here is the call graph for this function:

**8.53.3.8 psa\_fwu\_request\_reboot()**

```
psa_status_t psa_fwu_request_reboot (
 void)
```

Requests the platform to reboot.

**Returns**

Result status.

Definition at line 90 of file tfm\_fwu\_api.c.

Here is the call graph for this function:



### 8.53.3.9 psa\_fwu\_start()

```

psa_status_t psa_fwu_start (
 psa_fwu_component_t component,
 const void * manifest,
 size_t manifest_size)

```

Begin a firmware update operation for a specific firmware component.

#### Parameters

|                      |                                                                      |
|----------------------|----------------------------------------------------------------------|
| <i>component</i>     | Identifier of the firmware component to be updated.                  |
| <i>manifest</i>      | A pointer to a buffer containing a detached manifest for the update. |
| <i>manifest_size</i> | The size of the detached manifest.                                   |

#### Returns

Result status.

Definition at line 12 of file tfm\_fwu\_api.c.

Here is the call graph for this function:



### 8.53.3.10 psa\_fwu\_write()

```

psa_status_t psa_fwu_write (
 psa_fwu_component_t component,
 size_t image_offset,
 const void * block,
 size_t block_size)

```

Write a firmware image, or part of a firmware image, to its staging area.

## Parameters

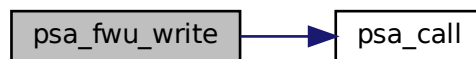
|                     |                                                     |
|---------------------|-----------------------------------------------------|
| <i>component</i>    | Identifier of the firmware component being updated. |
| <i>image_offset</i> | The offset of the data block in the whole image.    |
| <i>block</i>        | A buffer containing a block of image data.          |
| <i>block_size</i>   | Size of block, in bytes.                            |

## Returns

Result status.

Definition at line 25 of file tfm\_fwu\_api.c.

Here is the call graph for this function:



## 8.54 interface/include/tf-psa-crypto/build\_info.h File Reference

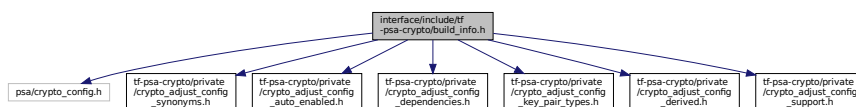
Build-time configuration info.

```

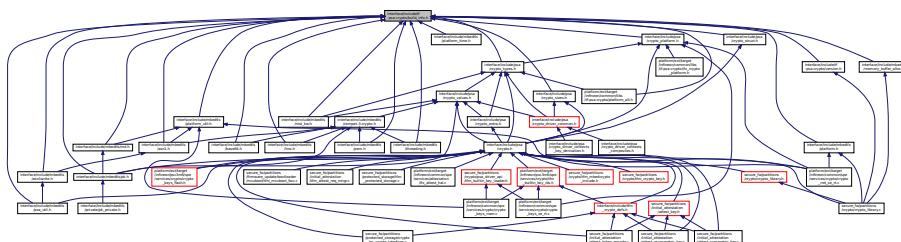
#include "psa/crypto_config.h"
#include "tf-psa-crypto/private/crypto_adjust_config_synonyms.h"
#include "tf-psa-crypto/private/crypto_adjust_config_auto_enabled.h"
#include "tf-psa-crypto/private/crypto_adjust_config_dependencies.h"
#include "tf-psa-crypto/private/crypto_adjust_config_key_pair_types.h"
#include "tf-psa-crypto/private/crypto_adjust_config_derived.h"
#include "tf-psa-crypto/private/crypto_adjust_config_support.h"

```

Include dependency graph for build\_info.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define TF_PSA_CRYPTO_VERSION_MAJOR 1`
- `#define TF_PSA_CRYPTO_VERSION_MINOR 1`
- `#define TF_PSA_CRYPTO_VERSION_PATCH 0`
- `#define TF_PSA_CRYPTO_VERSION_NUMBER 0x01010000`
- `#define TF_PSA_CRYPTO_VERSION_STRING "1.1.0"`
- `#define TF_PSA_CRYPTO_VERSION_STRING_FULL "TF-PSA-Crypto 1.1.0"`
- `#define TF_PSA_CRYPTO_CONFIG_FILES_READ`
- `#define TF_PSA_CRYPTO_CONFIG_IS_FINALIZED`

## Typedefs

- `typedef int mbedtls_iso_c_forbids_empty_translation_units`

### 8.54.1 Detailed Description

Build-time configuration info.

Include this file if you need to depend on the configuration options defined in `crypto_config.h` or `TF_PSA_CRYPT↵  
TO_CONFIG_FILE`.

### 8.54.2 Macro Definition Documentation

#### 8.54.2.1 TF\_PSA\_CRYPTO\_CONFIG\_FILES\_READ

```
#define TF_PSA_CRYPTO_CONFIG_FILES_READ
```

Definition at line 131 of file `build_info.h`.

#### 8.54.2.2 TF\_PSA\_CRYPTO\_CONFIG\_IS\_FINALIZED

```
#define TF_PSA_CRYPTO_CONFIG_IS_FINALIZED
```

Definition at line 186 of file `build_info.h`.

#### 8.54.2.3 TF\_PSA\_CRYPTO\_VERSION\_MAJOR

```
#define TF_PSA_CRYPTO_VERSION_MAJOR 1
```

The version number x.y.z is split into three parts. Major, Minor, Patchlevel  
Definition at line 27 of file `build_info.h`.

#### 8.54.2.4 TF\_PSA\_CRYPTO\_VERSION\_MINOR

```
#define TF_PSA_CRYPTO_VERSION_MINOR 1
```

Definition at line 28 of file `build_info.h`.

#### 8.54.2.5 TF\_PSA\_CRYPTO\_VERSION\_NUMBER

```
#define TF_PSA_CRYPTO_VERSION_NUMBER 0x01010000
```

The single version number has the following structure: MMNNPP00 Major version | Minor version | Patch version  
Definition at line 36 of file `build_info.h`.

### 8.54.2.6 TF\_PSA\_CRYPTO\_VERSION\_PATCH

#define TF\_PSA\_CRYPTO\_VERSION\_PATCH 0  
Definition at line 29 of file build\_info.h.

### 8.54.2.7 TF\_PSA\_CRYPTO\_VERSION\_STRING

#define TF\_PSA\_CRYPTO\_VERSION\_STRING "1.1.0"  
Definition at line 37 of file build\_info.h.

### 8.54.2.8 TF\_PSA\_CRYPTO\_VERSION\_STRING\_FULL

#define TF\_PSA\_CRYPTO\_VERSION\_STRING\_FULL "TF-PSA-Crypto 1.1.0"  
Definition at line 38 of file build\_info.h.

## 8.54.3 Typedef Documentation

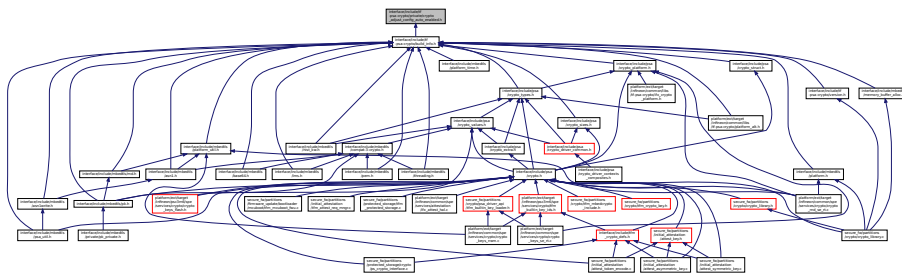
### 8.54.3.1 mbedtls\_iso\_c\_forbids\_empty\_translation\_units

typedef int mbedtls\_iso\_c\_forbids\_empty\_translation\_units  
Definition at line 193 of file build\_info.h.

## 8.55 interface/include/tf-psa-crypto/private/crypto\_adjust\_config\_auto\_enabled.h File Reference

Adjust PSA configuration: enable always-on features.

This graph shows which files directly or indirectly include this file:



## Macros

- #define PSA\_WANT\_KEY\_TYPE\_DERIVE 1
- #define PSA\_WANT\_KEY\_TYPE\_PASSWORD 1
- #define PSA\_WANT\_KEY\_TYPE\_PASSWORD\_HASH 1
- #define PSA\_WANT\_KEY\_TYPE\_RAW\_DATA 1

### 8.55.1 Detailed Description

Adjust PSA configuration: enable always-on features.

This is an internal header. Do not include it directly.

Always enable certain features which require a negligible amount of code to implement, to avoid some edge cases in the configuration combinatorics.

## 8.55.2 Macro Definition Documentation

### 8.55.2.1 PSA\_WANT\_KEY\_TYPE\_DERIVE

```
#define PSA_WANT_KEY_TYPE_DERIVE 1
```

Definition at line 18 of file `crypto_adjust_config_auto_enabled.h`.

### 8.55.2.2 PSA\_WANT\_KEY\_TYPE\_PASSWORD

```
#define PSA_WANT_KEY_TYPE_PASSWORD 1
```

Definition at line 19 of file `crypto_adjust_config_auto_enabled.h`.

### 8.55.2.3 PSA\_WANT\_KEY\_TYPE\_PASSWORD\_HASH

```
#define PSA_WANT_KEY_TYPE_PASSWORD_HASH 1
```

Definition at line 20 of file `crypto_adjust_config_auto_enabled.h`.

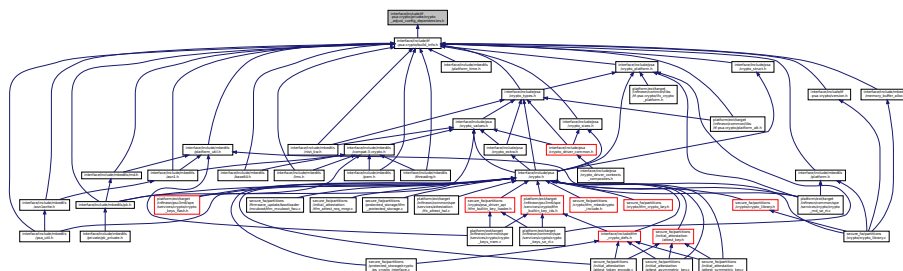
### 8.55.2.4 PSA\_WANT\_KEY\_TYPE\_RAW\_DATA

```
#define PSA_WANT_KEY_TYPE_RAW_DATA 1
```

Definition at line 21 of file `crypto_adjust_config_auto_enabled.h`.

## 8.56 interface/include/tf-psa-crypto/private/crypto\_adjust\_config\_↵ dependencies.h File Reference

Adjust PSA configuration by resolving some dependencies.  
This graph shows which files directly or indirectly include this file:



### 8.56.1 Detailed Description

Adjust PSA configuration by resolving some dependencies.  
This is an internal header. Do not include it directly.  
See docs/proposed/psa-conditional-inclusion-c.md. If the Mbed TLS implementation of a cryptographic mechanism A depends on a cryptographic mechanism B then if the cryptographic mechanism A is enabled and not accelerated enable B. Note that if A is enabled and accelerated, it is not necessary to enable B for A support.

## 8.57 interface/include/tf-psa-crypto/private/crypto\_adjust\_config\_↵ derived.h File Reference

Adjust PSA configuration by defining internal symbols.

[illegible]

```

• #define MBEDTLS_PSA_BUILTIN_GET_ENTROPY_DEFINED 0
• #define MBEDTLS_PSA_DRIVER_GET_ENTROPY_DEFINED 0
• #define MBEDTLS_ENTROPY_TRUE_SOURCES
• #define MBEDTLS_PSA_CRYPTO_RNG_STRENGTH 256

```

Adjust PSA configuration by defining internal symbols.  
This is an internal header. Do not include it directly.

#### 8.57.2.1 MBEDTLS\_ENTROPY\_TRUE\_SOURCES

```
(\
MBEDTLS_PSA_BUILTIN_GET_ENTROPY_DEFINED + \
MBEDTLS_PSA_DRIVER_GET_ENTROPY_DEFINED + \
0)
```

### 8.57.2.2 MBEDTLS\_PSA\_BUILTIN\_GET\_ENTROPY\_DEFINED

Definition at line 23 of file crypto\_adjust\_config\_derived.h.

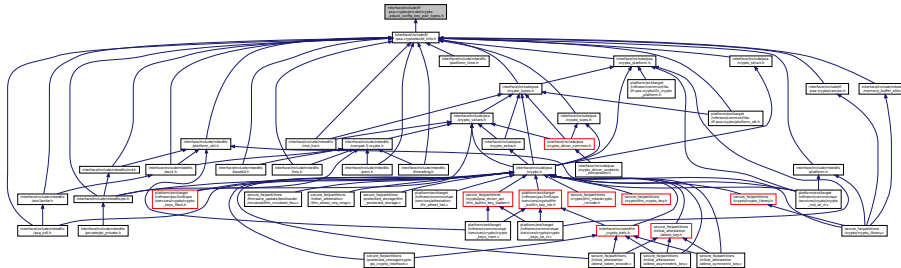
Definition at line 109 of file `crypto_adjust_config_derived.h`.

Definition at line 28 of file crypto\_adjust\_config\_derived.h.

## 8.58 interface/include/tf-psa-crypto/private/crypto\_adjust\_config\_key\_pair\_types.h File Reference

Adjust PSA configuration for key pair types.

This graph shows which files directly or indirectly include this file:



### 8.58.1 Detailed Description

Adjust PSA configuration for key pair types.

This is an internal header. Do not include it directly.

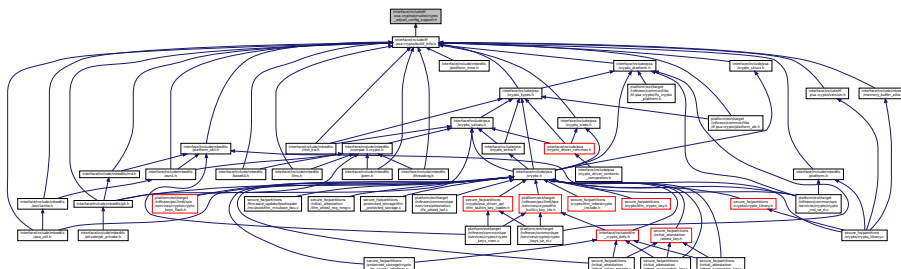
See docs/proposed/psa-conditional-inclusion-c.md.

- Support non-basic operations in a keypair type implicitly enables basic support for that keypair type.
- Support for a keypair type implicitly enables the corresponding public key type.
- Basic support for a keypair type implicitly enables import/export support for that keypair type. Warning: this is implementation-specific (mainly for the benefit of testing) and may change in the future!

## 8.59 interface/include/tf-psa-crypto/private/crypto\_adjust\_config\_support.h File Reference

Adjust TF-PSA-Crypto configuration: support modules.

This graph shows which files directly or indirectly include this file:



### 8.59.1 Detailed Description

Adjust TF-PSA-Crypto configuration: support modules.

This is an internal header. Do not include it directly.

Activate parts of support modules, based on the user configuration as well as requirements of generic code and requirements of driver-specific code.



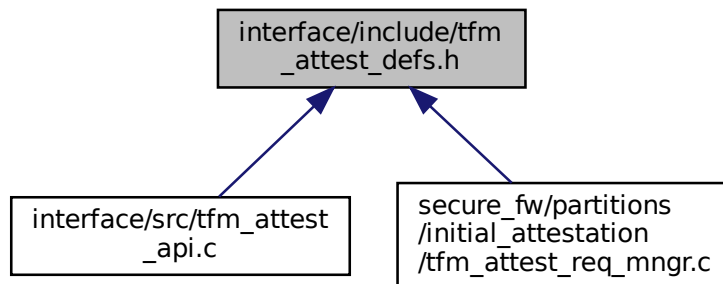


### 8.61.1 Detailed Description

Run-time version information.

## 8.62 interface/include/tfm\_attest\_defs.h File Reference

This graph shows which files directly or indirectly include this file:



### Macros

- `#define TFM_ATTEST_GET_TOKEN 1001`
- `#define TFM_ATTEST_GET_TOKEN_SIZE 1002`

### 8.62.1 Macro Definition Documentation

#### 8.62.1.1 TFM\_ATTEST\_GET\_TOKEN

```
#define TFM_ATTEST_GET_TOKEN 1001
```

Definition at line 16 of file `tfm_attest_defs.h`.

#### 8.62.1.2 TFM\_ATTEST\_GET\_TOKEN\_SIZE

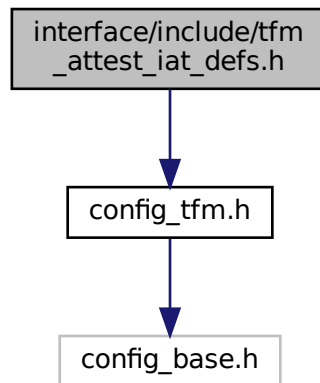
```
#define TFM_ATTEST_GET_TOKEN_SIZE 1002
```

Definition at line 17 of file `tfm_attest_defs.h`.

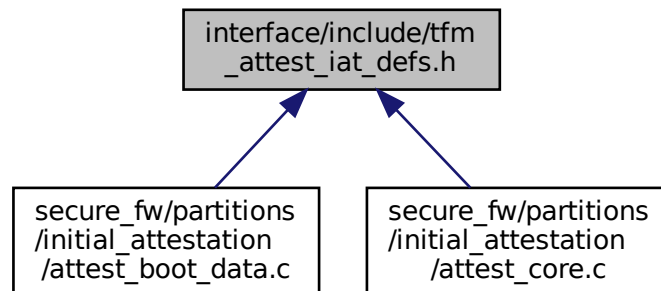
## 8.63 interface/include/tfm\_attest\_iat\_defs.h File Reference

```
#include "config_tfm.h"
```

Include dependency graph for tfm\_attest\_iat\_defs.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IAT_SW_COMPONENT_MEASUREMENT_TYPE` (1)
- `#define IAT_SW_COMPONENT_MEASUREMENT_VALUE` (2)
- `#define IAT_SW_COMPONENT_VERSION` (4)
- `#define IAT_SW_COMPONENT_SIGNER_ID` (5)
- `#define IAT_SW_COMPONENT_MEASUREMENT_DESC` (6)

### 8.63.1 Macro Definition Documentation

#### 8.63.1.1 IAT\_SW\_COMPONENT\_MEASUREMENT\_DESC

```
#define IAT_SW_COMPONENT_MEASUREMENT_DESC (6)
```

Definition at line 83 of file tfm\_attest\_iat\_defs.h.

### 8.63.1.2 IAT\_SW\_COMPONENT\_MEASUREMENT\_TYPE

```
#define IAT_SW_COMPONENT_MEASUREMENT_TYPE (1)
```

Definition at line 78 of file `tfm_attest_iat_defs.h`.

### 8.63.1.3 IAT\_SW\_COMPONENT\_MEASUREMENT\_VALUE

```
#define IAT_SW_COMPONENT_MEASUREMENT_VALUE (2)
```

Definition at line 79 of file `tfm_attest_iat_defs.h`.

### 8.63.1.4 IAT\_SW\_COMPONENT\_SIGNER\_ID

```
#define IAT_SW_COMPONENT_SIGNER_ID (5)
```

Definition at line 82 of file `tfm_attest_iat_defs.h`.

### 8.63.1.5 IAT\_SW\_COMPONENT\_VERSION

```
#define IAT_SW_COMPONENT_VERSION (4)
```

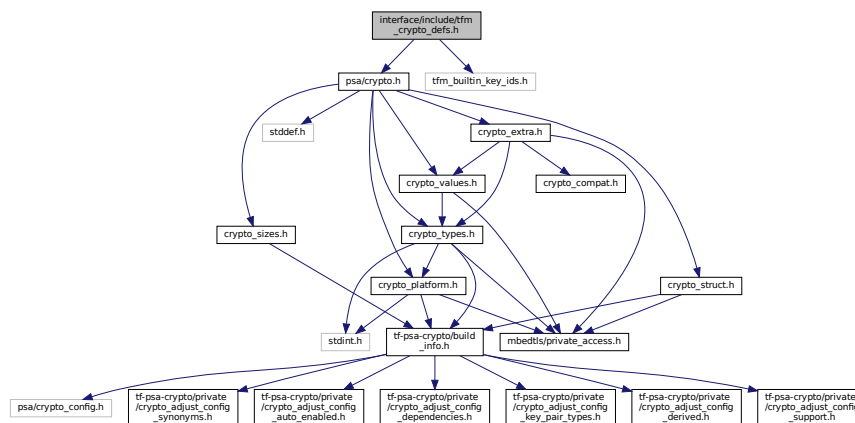
Definition at line 81 of file `tfm_attest_iat_defs.h`.

## 8.64 interface/include/tfm\_crypto\_defs.h File Reference

```
#include "psa/crypto.h"
```

```
#include "tfm_built_in_key_ids.h"
```

Include dependency graph for `tfm_crypto_defs.h`:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `tfm_crypto_aead_pack_input`

This type is used to overcome a limitation in the number of maximum IOVECs that can be used especially in `psa_aead_encrypt` and `psa_aead_decrypt`. By using this type we pack the nonce and the actual nonce\_length at part of the same structure.

- struct `tfm_crypto_pack_iovec`

Structure used to pack non-pointer types in a call to PSA Crypto APIs.

## Macros

- #define `TFM_CRYPTO_MAX_NONCE_LENGTH` (16u)  
The maximum supported length of a nonce through the TF-M interfaces.
- #define `RANDOM_FUNCS` X(TFM\_CRYPTO\_GENERATE\_RANDOM)
- #define `KEY_MANAGEMENT_FUNCS`
- #define `HASH_FUNCS`
- #define `MAC_FUNCS`
- #define `CIPHER_FUNCS`
- #define `AEAD_FUNCS`
- #define `ASYM_SIGN_FUNCS`
- #define `ASYM_ENCRYPT_FUNCS`
- #define `KEY_DERIVATION_FUNCS`
- #define `BASE__VALUE`(x) (((uint16\_t) (((uint16\_t)(x)) << 8) & 0xFF00))
- #define `X`(FUNCTION\_NAME) FUNCTION\_NAME ## \_SID,
- #define `TFM_CRYPTO_GET_GROUP_ID`(\_function\_id) ((enum `tfm_crypto_group_id_t`) (((uint16\_t) (\_function\_id) >> 8) & 0xFF))

This macro is used to extract the group\_id from an encoded function id by accessing the upper 8 bits. A \_function\_id is uint16\_t type.

## Enumerations

- enum `tfm_crypto_group_id_t` {  
`TFM_CRYPTO_GROUP_ID_RANDOM` = `UINT8_C(1)`, `TFM_CRYPTO_GROUP_ID_KEY_MANAGEMENT` = `UINT8_C(2)`, `TFM_CRYPTO_GROUP_ID_HASH` = `UINT8_C(3)`, `TFM_CRYPTO_GROUP_ID_MAC` = `UINT8_C(4)`,  
`TFM_CRYPTO_GROUP_ID_CIPHER` = `UINT8_C(5)`, `TFM_CRYPTO_GROUP_ID_AEAD` = `UINT8_C(6)`, `TFM_CRYPTO_GROUP_ID_ASYM_SIGN` = `UINT8_C(7)`, `TFM_CRYPTO_GROUP_ID_ASYM_ENCRYPT` = `UINT8_C(8)`,  
`TFM_CRYPTO_GROUP_ID_KEY_DERIVATION` = `UINT8_C(9)` }
- Type associated to the group of a function encoding. There can be nine groups (Random, Key management, Hash, MAC, Cipher, AEAD, Asym sign, Asym encrypt, Key derivation).
- enum `tfm_crypto_func_sid_t` {  
`BASE__RANDOM` = (((uint16\_t) (((uint16\_t) (TFM\_CRYPTO\_GROUP\_ID\_RANDOM)) << 8) & 0xFF00)) - 1, `TFM_CRYPTO_GENERATE_RANDOM_SID`, `BASE__KEY_MANAGEMENT` = (((uint16\_t) (((uint16\_t) (TFM\_CRYPTO\_GROUP\_ID\_KEY\_MANAGEMENT)) << 8) & 0xFF00)) - 1, `TFM_CRYPTO_GET_KEY_ATTRIBUTES_SID`, `TFM_CRYPTO_ABANDONED_OPEN_KEY_SID`, `TFM_CRYPTO_ABANDONED_CLOSE_KEY_SID`, `TFM_CRYPTO_IMPORT_KEY_SID`, `TFM_CRYPTO_DESTROY_KEY_SID`, `TFM_CRYPTO_EXPORT_KEY_SID`, `TFM_CRYPTO_EXPORT_PUBLIC_KEY_SID`, `TFM_CRYPTO_PURGE_KEY_SID`, `TFM_CRYPTO_COPY_KEY_SID`, `TFM_CRYPTO_GENERATE_KEY_SID`, `BASE__HASH` = (((uint16\_t) (((uint16\_t) (TFM\_CRYPTO\_GROUP\_ID\_HASH)) << 8) & 0xFF00)) - 1, `TFM_CRYPTO_HASH_COMPUTE_SID`, `TFM_CRYPTO_HASH_COMPARE_SID`, `TFM_CRYPTO_HASH_SETUP_SID`, `TFM_CRYPTO_HASH_UPDATE_SID`, `TFM_CRYPTO_HASH_CLONE_SID`, `TFM_CRYPTO_HASH_FINISH_SID`, `TFM_CRYPTO_HASH_VERIFY_SID`, `TFM_CRYPTO_HASH_ABORT_SID`, `TFM_CRYPTO_CAN_DO_HASH_SID`, `BASE__MAC` = (((uint16\_t) (((uint16\_t) (TFM\_CRYPTO\_GROUP\_ID\_MAC)) << 8) & 0xFF00)) - 1, `TFM_CRYPTO_MAC_COMPUTE_SID`, `TFM_CRYPTO_MAC_VERIFY_SID`, `TFM_CRYPTO_MAC_SIGN_SETUP_SID`, `TFM_CRYPTO_MAC_VERIFY_SETUP_SID`, `TFM_CRYPTO_MAC_UPDATE_SID`, `TFM_CRYPTO_MAC_SIGN_FINISH_SID`, `TFM_CRYPTO_MAC_VERIFY_FINISH_SID`, `TFM_CRYPTO_MAC_ABORT_SID`,

```

BASE__CIPHER = (((uint16_t)(((uint16_t)(TFM_CRYPTO_GROUP_ID_CIPHER)) << 8) & 0xFF00)) - 1, TFM_CRYPTO_CIPHER_ENCRYPT_SID, TFM_CRYPTO_CIPHER_DECRYPT_SID,
TFM_CRYPTO_CIPHER_ENCRYPT_SETUP_SID,
TFM_CRYPTO_CIPHER_DECRYPT_SETUP_SID, TFM_CRYPTO_CIPHER_GENERATE_IV_SID,
TFM_CRYPTO_CIPHER_SET_IV_SID, TFM_CRYPTO_CIPHER_UPDATE_SID,
TFM_CRYPTO_CIPHER_FINISH_SID, TFM_CRYPTO_CIPHER_ABORT_SID, TFM_CRYPTO_CAN_DO_CIPHER_SID,
BASE__AEAD = (((uint16_t)(((uint16_t)(TFM_CRYPTO_GROUP_ID_AEAD)) << 8) & 0xFF00)) - 1,
TFM_CRYPTO_AEAD_ENCRYPT_SID, TFM_CRYPTO_AEAD_DECRYPT_SID, TFM_CRYPTO_AEAD_ENCRYPT_SETUP_SID,
TFM_CRYPTO_AEAD_DECRYPT_SETUP_SID,
TFM_CRYPTO_AEAD_GENERATE_NONCE_SID, TFM_CRYPTO_AEAD_SET_NONCE_SID, TFM_CRYPTO_AEAD_SET_LENGTHS_SID,
TFM_CRYPTO_AEAD_UPDATE_AD_SID,
TFM_CRYPTO_AEAD_UPDATE_SID, TFM_CRYPTO_AEAD_FINISH_SID, TFM_CRYPTO_AEAD_VERIFY_SID,
TFM_CRYPTO_AEAD_ABORT_SID,
BASE__ASYM_SIGN = (((uint16_t)(((uint16_t)(TFM_CRYPTO_GROUP_ID_ASYM_SIGN)) << 8) & 0xFF00)) - 1, TFM_CRYPTO_ASYMMETRIC_SIGN_MESSAGE_SID, TFM_CRYPTO_ASYMMETRIC_VERIFY_MESSAGE_SID,
TFM_CRYPTO_ASYMMETRIC_SIGN_HASH_SID,
TFM_CRYPTO_ASYMMETRIC_VERIFY_HASH_SID, BASE__ASYM_ENCRYPT = (((uint16_t)(((uint16_t)(
TFM_CRYPTO_GROUP_ID_ASYM_ENCRYPT)) << 8) & 0xFF00)) - 1, TFM_CRYPTO_ASYMMETRIC_ENCRYPT_SID,
TFM_CRYPTO_ASYMMETRIC_DECRYPT_SID,
BASE__KEY_DERIVATION = (((uint16_t)(((uint16_t)(TFM_CRYPTO_GROUP_ID_KEY_DERIVATION)) << 8) & 0xFF00)) - 1, TFM_CRYPTO_RAW_KEY_AGREEMENT_SID, TFM_CRYPTO_KEY_DERIVATION_SETUP_SID,
TFM_CRYPTO_KEY_DERIVATION_GET_CAPACITY_SID,
TFM_CRYPTO_KEY_DERIVATION_SET_CAPACITY_SID, TFM_CRYPTO_KEY_DERIVATION_INPUT_BYTES_SID,
TFM_CRYPTO_KEY_DERIVATION_INPUT_KEY_SID, TFM_CRYPTO_KEY_DERIVATION_INPUT_INTEGER_SID,
TFM_CRYPTO_KEY_DERIVATION_KEY_AGREEMENT_SID, TFM_CRYPTO_KEY_DERIVATION_OUTPUT_BYTES_SID,
TFM_CRYPTO_KEY_DERIVATION_OUTPUT_KEY_SID, TFM_CRYPTO_KEY_DERIVATION_ABORT_SID
}

```

*This type defines numerical progressive values identifying a function API exposed through the interfaces (S or NS). It's used to dispatch the requests from S/NS to the corresponding API implementation in the Crypto service backend.*

## 8.64.1 Macro Definition Documentation

### 8.64.1.1 AEAD\_FUNCS

```
#define AEAD_FUNCS
```

**Value:**

```

X(TFM_CRYPTO_AEAD_ENCRYPT) \
X(TFM_CRYPTO_AEAD_DECRYPT) \
X(TFM_CRYPTO_AEAD_ENCRYPT_SETUP) \
X(TFM_CRYPTO_AEAD_DECRYPT_SETUP) \
X(TFM_CRYPTO_AEAD_GENERATE_NONCE) \
X(TFM_CRYPTO_AEAD_SET_NONCE) \
X(TFM_CRYPTO_AEAD_SET_LENGTHS) \
X(TFM_CRYPTO_AEAD_UPDATE_AD) \
X(TFM_CRYPTO_AEAD_UPDATE) \
X(TFM_CRYPTO_AEAD_FINISH) \
X(TFM_CRYPTO_AEAD_VERIFY) \
X(TFM_CRYPTO_AEAD_ABORT) \

```

Definition at line 131 of file tfm\_crypto\_defs.h.

### 8.64.1.2 ASYM\_ENCRYPT\_FUNCS

```
#define ASYM_ENCRYPT_FUNCS
```

**Value:**

```

X(TFM_CRYPTO_ASYMMETRIC_ENCRYPT) \
X(TFM_CRYPTO_ASYMMETRIC_DECRYPT) \

```

Definition at line 151 of file tfm\_crypto\_defs.h.

### 8.64.1.3 ASYM\_SIGN\_FUNCS

```
#define ASYM_SIGN_FUNCS
```

**Value:**

```
X(TFM_CRYPTO_ASYMMETRIC_SIGN_MESSAGE) \
X(TFM_CRYPTO_ASYMMETRIC_VERIFY_MESSAGE) \
X(TFM_CRYPTO_ASYMMETRIC_SIGN_HASH) \
X(TFM_CRYPTO_ASYMMETRIC_VERIFY_HASH)
```

Definition at line 145 of file tfm\_crypto\_defs.h.

### 8.64.1.4 BASE\_\_VALUE

```
#define BASE__VALUE(
```

```
 x) ((uint16_t)((uint16_t)(x) << 8) & 0xFF00))
```

Definition at line 168 of file tfm\_crypto\_defs.h.

### 8.64.1.5 CIPHER\_FUNCS

```
#define CIPHER_FUNCS
```

**Value:**

```
X(TFM_CRYPTO_CIPHER_ENCRYPT) \
X(TFM_CRYPTO_CIPHER_DECRYPT) \
X(TFM_CRYPTO_CIPHER_ENCRYPT_SETUP) \
X(TFM_CRYPTO_CIPHER_DECRYPT_SETUP) \
X(TFM_CRYPTO_CIPHER_GENERATE_IV) \
X(TFM_CRYPTO_CIPHER_SET_IV) \
X(TFM_CRYPTO_CIPHER_UPDATE) \
X(TFM_CRYPTO_CIPHER_FINISH) \
X(TFM_CRYPTO_CIPHER_ABORT) \
X(TFM_CRYPTO_CAN_DO_CIPHER)
```

Definition at line 119 of file tfm\_crypto\_defs.h.

### 8.64.1.6 HASH\_FUNCS

```
#define HASH_FUNCS
```

**Value:**

```
X(TFM_CRYPTO_HASH_COMPUTE) \
X(TFM_CRYPTO_HASH_COMPARE) \
X(TFM_CRYPTO_HASH_SETUP) \
X(TFM_CRYPTO_HASH_UPDATE) \
X(TFM_CRYPTO_HASH_CLONE) \
X(TFM_CRYPTO_HASH_FINISH) \
X(TFM_CRYPTO_HASH_VERIFY) \
X(TFM_CRYPTO_HASH_ABORT) \
X(TFM_CRYPTO_CAN_DO_HASH)
```

Definition at line 98 of file tfm\_crypto\_defs.h.

### 8.64.1.7 KEY\_DERIVATION\_FUNCS

```
#define KEY_DERIVATION_FUNCS
```

**Value:**

```
X(TFM_CRYPTO_RAW_KEY_AGREEMENT) \
X(TFM_CRYPTO_KEY_DERIVATION_SETUP) \
X(TFM_CRYPTO_KEY_DERIVATION_GET_CAPACITY) \
X(TFM_CRYPTO_KEY_DERIVATION_SET_CAPACITY) \
X(TFM_CRYPTO_KEY_DERIVATION_INPUT_BYTES) \
X(TFM_CRYPTO_KEY_DERIVATION_INPUT_KEY) \
X(TFM_CRYPTO_KEY_DERIVATION_INPUT_INTEGER) \
X(TFM_CRYPTO_KEY_DERIVATION_KEY_AGREEMENT) \
X(TFM_CRYPTO_KEY_DERIVATION_OUTPUT_BYTES) \
X(TFM_CRYPTO_KEY_DERIVATION_OUTPUT_KEY) \
X(TFM_CRYPTO_KEY_DERIVATION_ABORT)
```

Definition at line 155 of file tfm\_crypto\_defs.h.

### 8.64.1.8 KEY\_MANAGEMENT\_FUNCS

```
#define KEY_MANAGEMENT_FUNCS
```

**Value:**

```
X(TFM_CRYPTO_GET_KEY_ATTRIBUTES) \
X(TFM_CRYPTO_ABANDONED_OPEN_KEY) \
X(TFM_CRYPTO_ABANDONED_CLOSE_KEY) \
X(TFM_CRYPTO_IMPORT_KEY) \
X(TFM_CRYPTO_DESTROY_KEY) \
X(TFM_CRYPTO_EXPORT_KEY) \
X(TFM_CRYPTO_EXPORT_PUBLIC_KEY) \
X(TFM_CRYPTO_PURGE_KEY) \
X(TFM_CRYPTO_COPY_KEY) \
X(TFM_CRYPTO_GENERATE_KEY)
```

Definition at line 86 of file tfm\_crypto\_defs.h.

### 8.64.1.9 MAC\_FUNCS

```
#define MAC_FUNCS
```

**Value:**

```
X(TFM_CRYPTO_MAC_COMPUTE) \
X(TFM_CRYPTO_MAC_VERIFY) \
X(TFM_CRYPTO_MAC_SIGN_SETUP) \
X(TFM_CRYPTO_MAC_VERIFY_SETUP) \
X(TFM_CRYPTO_MAC_UPDATE) \
X(TFM_CRYPTO_MAC_SIGN_FINISH) \
X(TFM_CRYPTO_MAC_VERIFY_FINISH) \
X(TFM_CRYPTO_MAC_ABORT)
```

Definition at line 109 of file tfm\_crypto\_defs.h.

### 8.64.1.10 RANDOM\_FUNCS

```
#define RANDOM_FUNCS X(TFM_CRYPTO_GENERATE_RANDOM)
```

Definition at line 83 of file tfm\_crypto\_defs.h.

### 8.64.1.11 TFM\_CRYPTO\_GET\_GROUP\_ID

```
#define TFM_CRYPTO_GET_GROUP_ID(
 _function_id) ((enum tfm_crypto_group_id_t)((uint16_t)(_function_id) >> 8) &
 0xFF)
```

This macro is used to extract the group\_id from an encoded function id by accessing the upper 8 bits. A *\_function\_id* is uint16\_t type.

Definition at line 209 of file tfm\_crypto\_defs.h.

### 8.64.1.12 TFM\_CRYPTO\_MAX\_NONCE\_LENGTH

```
#define TFM_CRYPTO_MAX_NONCE_LENGTH (16u)
```

The maximum supported length of a nonce through the TF-M interfaces.

Definition at line 26 of file tfm\_crypto\_defs.h.

### 8.64.1.13 X

```
#define X(
 FUNCTION_NAME) FUNCTION_NAME ## _SID,
```

Definition at line 183 of file tfm\_crypto\_defs.h.

## 8.64.2 Enumeration Type Documentation



## 8.64.2.1 tfm\_crypto\_func\_sid\_t

enum `tfm_crypto_func_sid_t`

This type defines numerical progressive values identifying a function API exposed through the interfaces (S or NS). It's used to dispatch the requests from S/NS to the corresponding API implementation in the Crypto service backend.

## Note

Each function SID is encoded as uint16\_t. +-----+-----+ | Group ID | Func ID | +-----+-----+  
(MSB)15 8 7 0(LSB)

## Enumerator

|                                     |  |
|-------------------------------------|--|
| BASE__RANDOM                        |  |
| TFM_CRYPTO_GENERATE_RANDOM_SID      |  |
| BASE__KEY_MANAGEMENT                |  |
| TFM_CRYPTO_GET_KEY_ATTRIBUTES_SID   |  |
| TFM_CRYPTO_ABANDONED_OPEN_KEY_SID   |  |
| TFM_CRYPTO_ABANDONED_CLOSE_KEY_SID  |  |
| TFM_CRYPTO_IMPORT_KEY_SID           |  |
| TFM_CRYPTO_DESTROY_KEY_SID          |  |
| TFM_CRYPTO_EXPORT_KEY_SID           |  |
| TFM_CRYPTO_EXPORT_PUBLIC_KEY_SID    |  |
| TFM_CRYPTO_PURGE_KEY_SID            |  |
| TFM_CRYPTO_COPY_KEY_SID             |  |
| TFM_CRYPTO_GENERATE_KEY_SID         |  |
| BASE__HASH                          |  |
| TFM_CRYPTO_HASH_COMPUTE_SID         |  |
| TFM_CRYPTO_HASH_COMPARE_SID         |  |
| TFM_CRYPTO_HASH_SETUP_SID           |  |
| TFM_CRYPTO_HASH_UPDATE_SID          |  |
| TFM_CRYPTO_HASH_CLONE_SID           |  |
| TFM_CRYPTO_HASH_FINISH_SID          |  |
| TFM_CRYPTO_HASH_VERIFY_SID          |  |
| TFM_CRYPTO_HASH_ABORT_SID           |  |
| TFM_CRYPTO_CAN_DO_HASH_SID          |  |
| BASE__MAC                           |  |
| TFM_CRYPTO_MAC_COMPUTE_SID          |  |
| TFM_CRYPTO_MAC_VERIFY_SID           |  |
| TFM_CRYPTO_MAC_SIGN_SETUP_SID       |  |
| TFM_CRYPTO_MAC_VERIFY_SETUP_SID     |  |
| TFM_CRYPTO_MAC_UPDATE_SID           |  |
| TFM_CRYPTO_MAC_SIGN_FINISH_SID      |  |
| TFM_CRYPTO_MAC_VERIFY_FINISH_SID    |  |
| TFM_CRYPTO_MAC_ABORT_SID            |  |
| BASE__CIPHER                        |  |
| TFM_CRYPTO_CIPHER_ENCRYPT_SID       |  |
| TFM_CRYPTO_CIPHER_DECRYPT_SID       |  |
| TFM_CRYPTO_CIPHER_ENCRYPT_SETUP_SID |  |
| TFM_CRYPTO_CIPHER_DECRYPT_SETUP_SID |  |
| TFM_CRYPTO_CIPHER_GENERATE_IV_SID   |  |
| TFM_CRYPTO_CIPHER_SET_IV_SID        |  |
| TFM_CRYPTO_CIPHER_UPDATE_SID        |  |
| TFM_CRYPTO_CIPHER_FINISH_SID        |  |

## Enumerator

|                                             |  |
|---------------------------------------------|--|
| TFM_CRYPTO_CIPHER_ABORT_SID                 |  |
| TFM_CRYPTO_CAN_DO_CIPHER_SID                |  |
| BASE__AEAD                                  |  |
| TFM_CRYPTO_AEAD_ENCRYPT_SID                 |  |
| TFM_CRYPTO_AEAD_DECRYPT_SID                 |  |
| TFM_CRYPTO_AEAD_ENCRYPT_SETUP_SID           |  |
| TFM_CRYPTO_AEAD_DECRYPT_SETUP_SID           |  |
| TFM_CRYPTO_AEAD_GENERATE_NONCE_SID          |  |
| TFM_CRYPTO_AEAD_SET_NONCE_SID               |  |
| TFM_CRYPTO_AEAD_SET_LENGTHS_SID             |  |
| TFM_CRYPTO_AEAD_UPDATE_AD_SID               |  |
| TFM_CRYPTO_AEAD_UPDATE_SID                  |  |
| TFM_CRYPTO_AEAD_FINISH_SID                  |  |
| TFM_CRYPTO_AEAD_VERIFY_SID                  |  |
| TFM_CRYPTO_AEAD_ABORT_SID                   |  |
| BASE__ASYM_SIGN                             |  |
| TFM_CRYPTO_ASYMMETRIC_SIGN_MESSAGE_SID      |  |
| TFM_CRYPTO_ASYMMETRIC_VERIFY_MESSAGE_SID    |  |
| TFM_CRYPTO_ASYMMETRIC_SIGN_HASH_SID         |  |
| TFM_CRYPTO_ASYMMETRIC_VERIFY_HASH_SID       |  |
| BASE__ASYM_ENCRYPT                          |  |
| TFM_CRYPTO_ASYMMETRIC_ENCRYPT_SID           |  |
| TFM_CRYPTO_ASYMMETRIC_DECRYPT_SID           |  |
| BASE__KEY_DERIVATION                        |  |
| TFM_CRYPTO_RAW_KEY_AGREEMENT_SID            |  |
| TFM_CRYPTO_KEY_DERIVATION_SETUP_SID         |  |
| TFM_CRYPTO_KEY_DERIVATION_GET_CAPACITY_SID  |  |
| TFM_CRYPTO_KEY_DERIVATION_SET_CAPACITY_SID  |  |
| TFM_CRYPTO_KEY_DERIVATION_INPUT_BYTES_SID   |  |
| TFM_CRYPTO_KEY_DERIVATION_INPUT_KEY_SID     |  |
| TFM_CRYPTO_KEY_DERIVATION_INPUT_INTEGER_SID |  |
| TFM_CRYPTO_KEY_DERIVATION_KEY_AGREEMENT_SID |  |
| TFM_CRYPTO_KEY_DERIVATION_OUTPUT_BYTES_SID  |  |
| TFM_CRYPTO_KEY_DERIVATION_OUTPUT_KEY_SID    |  |
| TFM_CRYPTO_KEY_DERIVATION_ABORT_SID         |  |

Definition at line 182 of file tfm\_crypto\_defs.h.

### 8.64.2.2 tfm\_crypto\_group\_id\_t

enum [tfm\\_crypto\\_group\\_id\\_t](#)

Type associated to the group of a function encoding. There can be nine groups (Random, Key management, Hash, MAC, Cipher, AEAD, Asym sign, Asym encrypt, Key derivation).

## Enumerator

|                                    |  |
|------------------------------------|--|
| TFM_CRYPTO_GROUP_ID_RANDOM         |  |
| TFM_CRYPTO_GROUP_ID_KEY_MANAGEMENT |  |
| TFM_CRYPTO_GROUP_ID_HASH           |  |
| TFM_CRYPTO_GROUP_ID_MAC            |  |

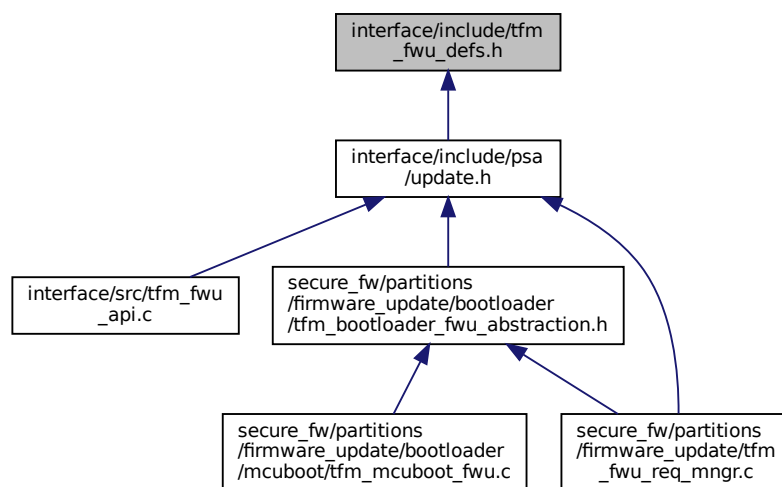
## Enumerator

|                                    |  |
|------------------------------------|--|
| TFM_CRYPTO_GROUP_ID_CIPHER         |  |
| TFM_CRYPTO_GROUP_ID_AEAD           |  |
| TFM_CRYPTO_GROUP_ID_ASYNC_SIGN     |  |
| TFM_CRYPTO_GROUP_ID_ASYNC_ENCRYPT  |  |
| TFM_CRYPTO_GROUP_ID_KEY_DERIVATION |  |

Definition at line 70 of file tfm\_crypto\_defs.h.

## 8.65 interface/include/tfm\_fwu\_defs.h File Reference

This graph shows which files directly or indirectly include this file:



### Macros

- `#define TFM_FWU_START 1001`
- `#define TFM_FWU_WRITE 1002`
- `#define TFM_FWU_FINISH 1003`
- `#define TFM_FWU_CANCEL 1004`
- `#define TFM_FWU_INSTALL 1005`
- `#define TFM_FWU_CLEAN 1006`
- `#define TFM_FWU_REJECT 1007`
- `#define TFM_FWU_REQUEST_REBOOT 1008`
- `#define TFM_FWU_ACCEPT 1009`
- `#define TFM_FWU_QUERY 1010`

### 8.65.1 Macro Definition Documentation

#### 8.65.1.1 TFM\_FWU\_ACCEPT

```
#define TFM_FWU_ACCEPT 1009
```

Definition at line 24 of file `tfm_fwu_defs.h`.

#### 8.65.1.2 TFM\_FWU\_CANCEL

```
#define TFM_FWU_CANCEL 1004
```

Definition at line 19 of file tfm\_fwu\_defs.h.

#### 8.65.1.3 TFM\_FWU\_CLEAN

```
#define TFM_FWU_CLEAN 1006
```

Definition at line 21 of file tfm\_fwu\_defs.h.

#### 8.65.1.4 TFM\_FWU\_FINISH

```
#define TFM_FWU_FINISH 1003
```

Definition at line 18 of file tfm\_fwu\_defs.h.

#### 8.65.1.5 TFM\_FWU\_INSTALL

```
#define TFM_FWU_INSTALL 1005
```

Definition at line 20 of file tfm\_fwu\_defs.h.

#### 8.65.1.6 TFM\_FWU\_QUERY

```
#define TFM_FWU_QUERY 1010
```

Definition at line 25 of file tfm\_fwu\_defs.h.

#### 8.65.1.7 TFM\_FWU\_REJECT

```
#define TFM_FWU_REJECT 1007
```

Definition at line 22 of file tfm\_fwu\_defs.h.

#### 8.65.1.8 TFM\_FWU\_REQUEST\_REBOOT

```
#define TFM_FWU_REQUEST_REBOOT 1008
```

Definition at line 23 of file tfm\_fwu\_defs.h.

#### 8.65.1.9 TFM\_FWU\_START

```
#define TFM_FWU_START 1001
```

Definition at line 16 of file tfm\_fwu\_defs.h.

#### 8.65.1.10 TFM\_FWU\_WRITE

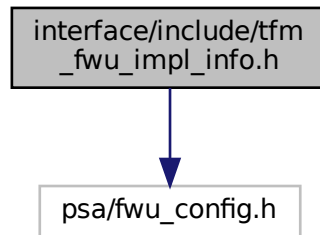
```
#define TFM_FWU_WRITE 1002
```

Definition at line 17 of file tfm\_fwu\_defs.h.

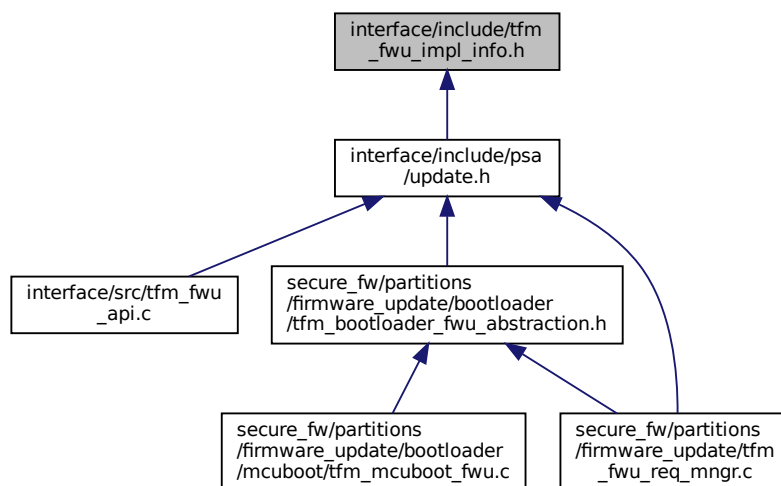
## 8.66 interface/include/tfm\_fwu\_impl\_info.h File Reference

```
#include "psa/fwu_config.h"
```

Include dependency graph for tfm\_fwu\_impl\_info.h:



This graph shows which files directly or indirectly include this file:



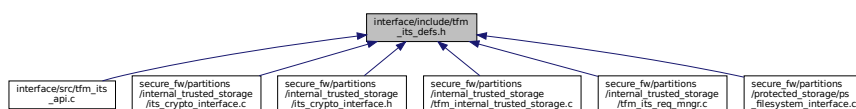
## Data Structures

- struct [psa\\_fwu\\_impl\\_info\\_t](#)

*The implementation-specific data in the component information structure.*

## 8.67 interface/include/tfm\_its\_defs.h File Reference

This graph shows which files directly or indirectly include this file:



## Macros

- `#define TFM_ITS_INVALID_UID 0`
- `#define TFM_ITS_SET 1001`
- `#define TFM_ITS_GET 1002`
- `#define TFM_ITS_GET_INFO 1003`
- `#define TFM_ITS_REMOVE 1004`

### 8.67.1 Macro Definition Documentation

#### 8.67.1.1 TFM\_ITS\_GET

```
#define TFM_ITS_GET 1002
```

Definition at line 20 of file `tfm_its_defs.h`.

#### 8.67.1.2 TFM\_ITS\_GET\_INFO

```
#define TFM_ITS_GET_INFO 1003
```

Definition at line 21 of file `tfm_its_defs.h`.

#### 8.67.1.3 TFM\_ITS\_INVALID\_UID

```
#define TFM_ITS_INVALID_UID 0
```

Definition at line 16 of file `tfm_its_defs.h`.

#### 8.67.1.4 TFM\_ITS\_REMOVE

```
#define TFM_ITS_REMOVE 1004
```

Definition at line 22 of file `tfm_its_defs.h`.

#### 8.67.1.5 TFM\_ITS\_SET

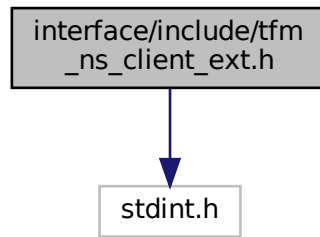
```
#define TFM_ITS_SET 1001
```

Definition at line 19 of file `tfm_its_defs.h`.

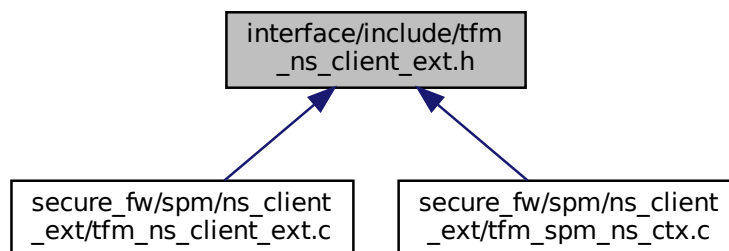
## 8.68 interface/include/tfm\_ns\_client\_ext.h File Reference

```
#include <stdint.h>
```

Include dependency graph for tfm\_ns\_client\_ext.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define TFM_NS_CLIENT_INVALID_TOKEN 0xFFFFFFFF`
- `#define TFM_NS_CLIENT_ERR_SUCCESS 0x0`
- `#define TFM_NS_CLIENT_ERR_INVALID_TOKEN 0x1`
- `#define TFM_NS_CLIENT_ERR_INVALID_NSID 0x2`
- `#define TFM_NS_CLIENT_ERR_INVALID_ACCESS 0x3`

## Functions

- `uint32_t tfm_nsce_init (uint32_t ctx_requested)`  
*Initialize the non-secure client extension.*
- `uint32_t tfm_nsce_acquire_ctx (uint8_t group_id, uint8_t thread_id)`  
*Acquire the context for a non-secure client.*
- `uint32_t tfm_nsce_release_ctx (uint32_t token)`  
*Release the context for the non-secure client.*
- `uint32_t tfm_nsce_load_ctx (uint32_t token, int32_t nsid)`  
*Load the context for the non-secure client.*
- `uint32_t tfm_nsce_save_ctx (uint32_t token)`  
*Save the context for the non-secure client.*

## 8.68.1 Macro Definition Documentation

### 8.68.1.1 TFM\_NS\_CLIENT\_ERR\_INVALID\_ACCESS

```
#define TFM_NS_CLIENT_ERR_INVALID_ACCESS 0x3
```

Definition at line 23 of file `tfm_ns_client_ext.h`.

### 8.68.1.2 TFM\_NS\_CLIENT\_ERR\_INVALID\_NSID

```
#define TFM_NS_CLIENT_ERR_INVALID_NSID 0x2
```

Definition at line 22 of file `tfm_ns_client_ext.h`.

### 8.68.1.3 TFM\_NS\_CLIENT\_ERR\_INVALID\_TOKEN

```
#define TFM_NS_CLIENT_ERR_INVALID_TOKEN 0x1
```

Definition at line 21 of file `tfm_ns_client_ext.h`.

### 8.68.1.4 TFM\_NS\_CLIENT\_ERR\_SUCCESS

```
#define TFM_NS_CLIENT_ERR_SUCCESS 0x0
```

Definition at line 20 of file `tfm_ns_client_ext.h`.

### 8.68.1.5 TFM\_NS\_CLIENT\_INVALID\_TOKEN

```
#define TFM_NS_CLIENT_INVALID_TOKEN 0xFFFFFFFF
```

Definition at line 17 of file `tfm_ns_client_ext.h`.

## 8.68.2 Function Documentation

### 8.68.2.1 tfm\_nsce\_acquire\_ctx()

```
uint32_t tfm_nsce_acquire_ctx (
 uint8_t group_id,
 uint8_t thread_id)
```

Acquire the context for a non-secure client.

This function should be called before a non-secure client calling the PSA API into TF-M. It is to request the allocation of the context for the upcoming service call from that non-secure client. The non-secure clients in one group share the same context. The thread ID is used to identify the different non-secure clients.

#### Parameters

|    |                  |                                        |
|----|------------------|----------------------------------------|
| in | <i>group_id</i>  | The group ID of the non-secure client  |
| in | <i>thread_id</i> | The thread ID of the non-secure client |

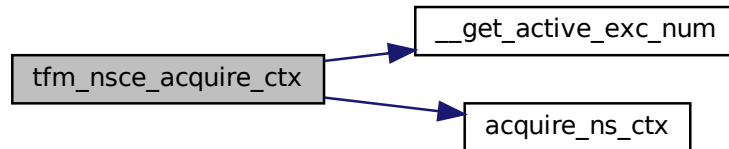


**Returns**

Returns the token of the allocated context. 0xFFFFFFFF means the allocation failed and the token is invalid.

Definition at line 48 of file `tfm_ns_client_ext.c`.

Here is the call graph for this function:

**8.68.2.2 tfm\_nsce\_init()**

```
uint32_t tfm_nsce_init (
 uint32_t ctx_requested)
```

Initialize the non-secure client extension.

This function should be called before any other non-secure client APIs. It gives NSPE the opportunity to initialize the non-secure client extension in TF-M. Also, NSPE can get the number of allocated non-secure client context slots in the return value. That is useful if NSPE wants to decide the group (context) assignment at runtime.

**Parameters**

|    |                      |                                                                                                                        |
|----|----------------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ctx_requested</i> | The number of non-secure context requested from the NS entity. If request maximum available context, then set it to 0. |
|----|----------------------|------------------------------------------------------------------------------------------------------------------------|

**Returns**

Returns the number of non-secure context allocated to the NS entity. The allocated context number  $\leq$  maximum supported context number. If the initialization is failed, then 0 is returned.

Definition at line 30 of file `tfm_ns_client_ext.c`.

Here is the call graph for this function:

**8.68.2.3 tfm\_nsce\_load\_ctx()**

```
uint32_t tfm_nsce_load_ctx (
```

```
uint32_t token,
int32_t nsid)
```

Load the context for the non-secure client.

This function should be called when a non-secure client is going to be scheduled in at the non-secure side. The caller is usually the scheduler of the RTOS. The non-secure client ID is managed by the non-secure world and passed to TF-M as the input parameter of TF-M.

#### Parameters

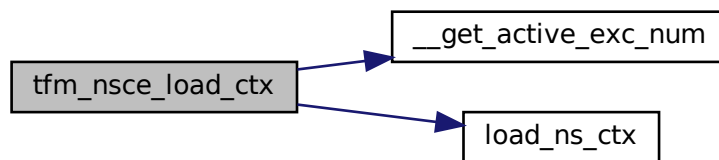
|    |              |                                                         |
|----|--------------|---------------------------------------------------------|
| in | <i>token</i> | The token returned by <code>tfm_nsce_acquire_ctx</code> |
| in | <i>nsid</i>  | The non-secure client ID for this client                |

#### Returns

Returns the error code.

Definition at line 96 of file `tfm_ns_client_ext.c`.

Here is the call graph for this function:



#### 8.68.2.4 tfm\_nsce\_release\_ctx()

```
uint32_t tfm_nsce_release_ctx (
 uint32_t token)
```

Release the context for the non-secure client.

This function should be called when a non-secure client is going to be terminated or will not call TF-M secure services in the future. It is to release the context allocated for the calling non-secure client. If the calling non-secure client is the only thread in the group, then the context will be deallocated. Otherwise, the context will still be taken for the other threads in the group.

#### Parameters

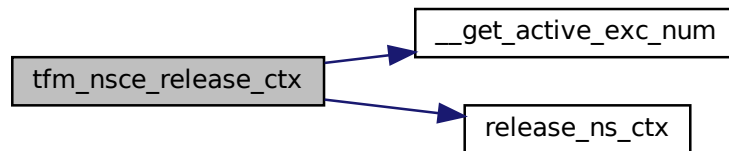
|    |              |                                                         |
|----|--------------|---------------------------------------------------------|
| in | <i>token</i> | The token returned by <code>tfm_nsce_acquire_ctx</code> |
|----|--------------|---------------------------------------------------------|

**Returns**

Returns the error code.

Definition at line 67 of file `tfm_ns_client_ext.c`.

Here is the call graph for this function:

**8.68.2.5 tfm\_nsce\_save\_ctx()**

```
uint32_t tfm_nsce_save_ctx (
 uint32_t token)
```

Save the context for the non-secure client.

This function should be called when a non-secure client is going to be scheduled out at the non-secure side. The caller is usually the scheduler of the RTOS.

**Parameters**

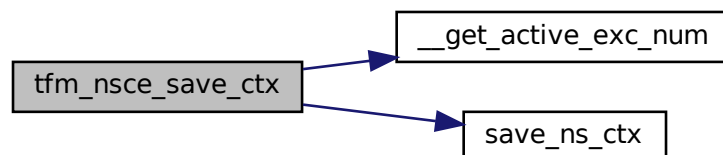
|    |              |                                                         |
|----|--------------|---------------------------------------------------------|
| in | <i>token</i> | The token returned by <code>tfm_nsce_acquire_ctx</code> |
|----|--------------|---------------------------------------------------------|

**Returns**

Returns the error code.

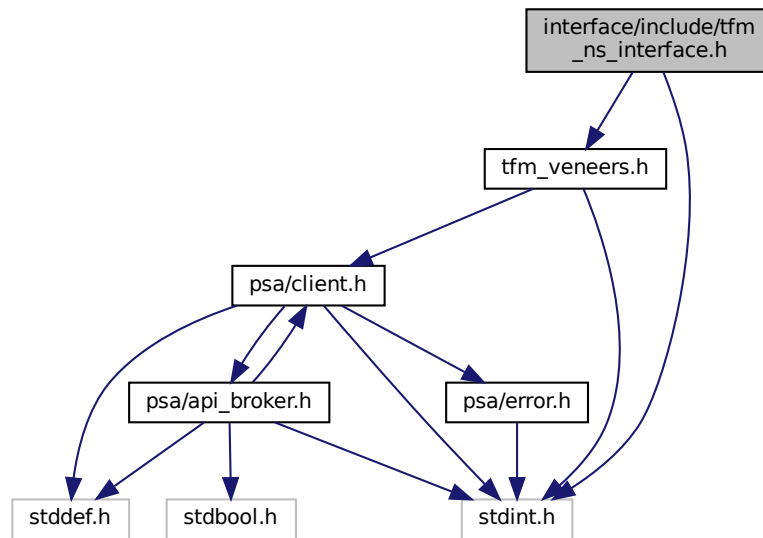
Definition at line 129 of file `tfm_ns_client_ext.c`.

Here is the call graph for this function:

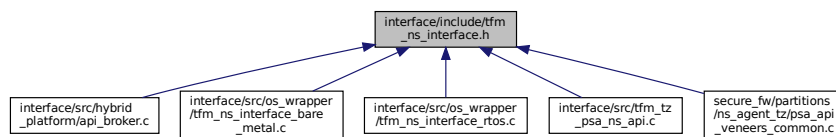
**8.69 interface/include/tfm\_ns\_interface.h File Reference**

```
#include <stdint.h>
#include "tfm_veneers.h"
```

Include dependency graph for `tfm_ns_interface.h`:



This graph shows which files directly or indirectly include this file:



## Typedefs

- typedef `int32_t(* veneer_fn)` (`uint32_t arg0`, `uint32_t arg1`, `uint32_t arg2`, `uint32_t arg3`)

## Enumerations

- enum `tfm_reenter_check_err_t` { `TFM_REENTRANCY_CHECK_ERR_INVALID_ACCESS` = -2, `TFM_REENTRANCY_CHECK_ERR_INVALID_ACCESS` = -1, `TFM_REENTRANCY_CHECK_SUCCESS` = 0, `TFM_REENTRANCY_CHECK_ERR_MAX` = `INT32_MAX` }

*Reentrancy check error returned type.*

## Functions

- `int32_t tfm_ns_interface_dispatch` (`veneer_fn fn`, `uint32_t arg0`, `uint32_t arg1`, `uint32_t arg2`, `uint32_t arg3`)  
*NS interface, veneer function dispatcher.*
- `uint32_t tfm_ns_interface_init` (`void`)  
*NS interface initialization function.*
- enum `tfm_reenter_check_err_t tfm_ns_check_safe_entry` (`void`)  
*Perform a reentrancy check before safely proceeding with a PSA Function Call.*
- enum `tfm_reenter_check_err_t tfm_ns_check_exit` (`void`)  
*Release the reentrancy flag.*

## 8.69.1 Typedef Documentation

### 8.69.1.1 veneer\_fn

typedef int32\_t(\* veneer\_fn) (uint32\_t arg0, uint32\_t arg1, uint32\_t arg2, uint32\_t arg3)  
Definition at line 19 of file tfm\_ns\_interface.h.

## 8.69.2 Enumeration Type Documentation

### 8.69.2.1 tfm\_reenter\_check\_err\_t

enum [tfm\\_reenter\\_check\\_err\\_t](#)  
Reentrancy check error returned type.

Enumerator

|                                         |  |
|-----------------------------------------|--|
| TFM_REENTRANCY_CHECK_ERR_INVALID_ACCESS |  |
| TFM_REENTRANCY_CHECK_ERR_FLAG_STATUS    |  |
| TFM_REENTRANCY_CHECK_SUCCESS            |  |
| _TFM_REENTRANCY_CHECK_ERR_MAX           |  |

Definition at line 70 of file tfm\_ns\_interface.h.

## 8.69.3 Function Documentation

### 8.69.3.1 tfm\_ns\_check\_exit()

enum [tfm\\_reenter\\_check\\_err\\_t](#) tfm\_ns\_check\_exit (  
void )

Release the reentrancy flag.

Note

Requires TFM\_TZ\_REENTRANCY\_CHECK option to be enabled

Returns

Returns [tfm\\_reenter\\_check\\_err\\_t](#) status code.

[TFM\\_REENTRANCY\\_CHECK\\_SUCCESS](#) The caller successfully cleared the previously set flag

[TFM\\_REENTRANCY\\_CHECK\\_ERR\\_FLAG\\_STATUS](#) Either: There was an attempt to clear twice the check flag Or: TFM\_TZ\_REENTRANCY\_CHECK is not enabled

Definition at line 39 of file tfm\_tz\_psa\_ns\_api.c.

### 8.69.3.2 tfm\_ns\_check\_safe\_entry()

enum [tfm\\_reenter\\_check\\_err\\_t](#) tfm\_ns\_check\_safe\_entry (  
void )

Perform a reentrancy check before safely proceeding with a PSA Function Call.

Note

Requires TFM\_TZ\_REENTRANCY\_CHECK option to be enabled

**Returns**

Returns `tfm_reenter_check_err_t` status code.

`TFM_REENTRANCY_CHECK_SUCCESS` The caller can then perform the `psa_call` successfully

`TFM_REENTRANCY_CHECK_ERR_FLAG_STATUS` Either: The check flag has been set already, thus either another call is in progress or it needs to be cleared. Or: `TFM_TZ_REENTRANCY_CHECK` is not enabled

`TFM_REENTRANCY_CHECK_ERR_INVALID_ACCESS` The caller cannot safely proceed with a `psa_call` because the secure state was interrupted and requires completion. Try later.

Definition at line 34 of file `tfm_tz_psa_ns_api.c`.

**8.69.3.3 tfm\_ns\_interface\_dispatch()**

```
int32_t tfm_ns_interface_dispatch (
 veneer_fn fn,
 uint32_t arg0,
 uint32_t arg1,
 uint32_t arg2,
 uint32_t arg3)
```

NS interface, veneer function dispatcher.

This function implements the dispatching mechanism for the desired veneer function, to be called with the parameters described from `arg0` to `arg3`.

**Note**

NSPE can use default implementation of this function or implement this function according to NS specific implementation and actual usage scenario.

**Parameters**

|    |             |                                                 |
|----|-------------|-------------------------------------------------|
| in | <i>fn</i>   | Function pointer to the veneer function desired |
| in | <i>arg0</i> | Argument 0 of fn                                |
| in | <i>arg1</i> | Argument 1 of fn                                |
| in | <i>arg2</i> | Argument 2 of fn                                |
| in | <i>arg3</i> | Argument 3 of fn                                |

**Returns**

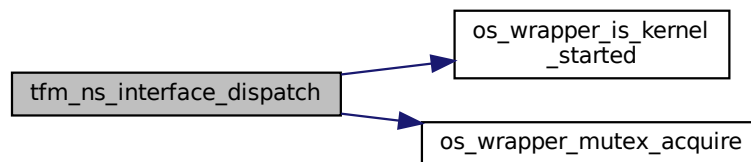
Returns the same return value of the requested veneer function

**Note**

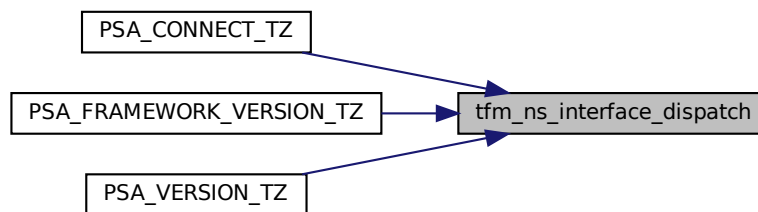
This API must ensure the return value is from the veneer function. Other unrecoverable errors must be considered as fatal error and should not return.

Definition at line 23 of file tfm\_ns\_interface\_bare\_metal.c.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.69.3.4 tfm\_ns\_interface\_init()**

```
uint32_t tfm_ns_interface_init (
 void)
```

NS interface initialization function.

This function initializes TF-M NS interface.

**Note**

NSPE can use default implementation of this function or implement this function according to NS specific implementation and actual usage scenario.

## Returns

[OS\\_WRAPPER\\_SUCCESS](#) on success or [OS\\_WRAPPER\\_ERROR](#) on error

Definition at line 30 of file `tfm_ns_interface_bare_metal.c`.

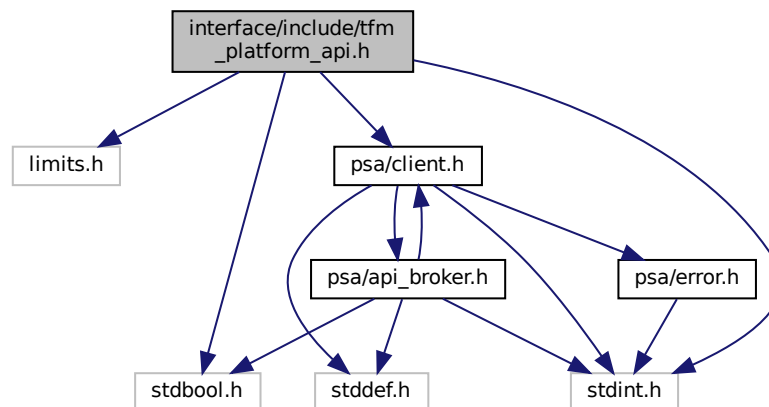
Here is the call graph for this function:



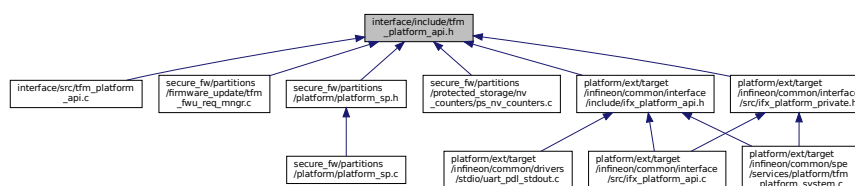
## 8.70 interface/include/tfm\_platform\_api.h File Reference

```
#include <limits.h>
#include <stdbool.h>
#include <stdint.h>
#include "psa/client.h"
```

Include dependency graph for `tfm_platform_api.h`:



This graph shows which files directly or indirectly include this file:





## Macros

- `#define TFM_PLATFORM_API_VERSION_MAJOR` (0)  
*TFM secure partition platform API version.*
- `#define TFM_PLATFORM_API_VERSION_MINOR` (3)
- `#define TFM_PLATFORM_API_ID_NV_READ` (1010)
- `#define TFM_PLATFORM_API_ID_NV_INCREMENT` (1011)
- `#define TFM_PLATFORM_API_ID_SYSTEM_RESET` (1012)
- `#define TFM_PLATFORM_API_ID_IOCTL` (1013)

## Typedefs

- `typedef int32_t tfm_platform_ioctl_req_t`

## Enumerations

- `enum tfm_platform_err_t` {  
`TFM_PLATFORM_ERR_SUCCESS` = 0, `TFM_PLATFORM_ERR_SYSTEM_ERROR`, `TFM_PLATFORM_ERR_INVALID_PARAMETER`,  
`TFM_PLATFORM_ERR_NOT_SUPPORTED`,  
`TFM_PLATFORM_ERR_FORCE_INT_SIZE` = INT\_MAX }  
*Platform service error types.*

## Functions

- `enum tfm_platform_err_t tfm_platform_system_reset` (void)  
*Resets the system.*
- `enum tfm_platform_err_t tfm_platform_ioctl` (tfm\_platform\_ioctl\_req\_t request, psa\_invec \*input, psa\_outvec \*output)  
*Performs a platform-specific service.*
- `enum tfm_platform_err_t tfm_platform_nv_counter_increment` (uint32\_t counter\_id)  
*Increments the given non-volatile (NV) counter by one.*
- `enum tfm_platform_err_t tfm_platform_nv_counter_read` (uint32\_t counter\_id, uint32\_t size, uint8\_t \*val)  
*Reads the given non-volatile (NV) counter.*

### 8.70.1 Macro Definition Documentation

#### 8.70.1.1 TFM\_PLATFORM\_API\_ID\_IOCTL

`#define TFM_PLATFORM_API_ID_IOCTL` (1013)  
 Definition at line 29 of file `tfm_platform_api.h`.

#### 8.70.1.2 TFM\_PLATFORM\_API\_ID\_NV\_INCREMENT

`#define TFM_PLATFORM_API_ID_NV_INCREMENT` (1011)  
 Definition at line 27 of file `tfm_platform_api.h`.

#### 8.70.1.3 TFM\_PLATFORM\_API\_ID\_NV\_READ

`#define TFM_PLATFORM_API_ID_NV_READ` (1010)  
 Definition at line 26 of file `tfm_platform_api.h`.

#### 8.70.1.4 TFM\_PLATFORM\_API\_ID\_SYSTEM\_RESET

```
#define TFM_PLATFORM_API_ID_SYSTEM_RESET (1012)
```

Definition at line 28 of file tfm\_platform\_api.h.

#### 8.70.1.5 TFM\_PLATFORM\_API\_VERSION\_MAJOR

```
#define TFM_PLATFORM_API_VERSION_MAJOR (0)
```

TFM secure partition platform API version.  
Definition at line 23 of file tfm\_platform\_api.h.

#### 8.70.1.6 TFM\_PLATFORM\_API\_VERSION\_MINOR

```
#define TFM_PLATFORM_API_VERSION_MINOR (3)
```

Definition at line 24 of file tfm\_platform\_api.h.

### 8.70.2 Typedef Documentation

#### 8.70.2.1 tfm\_platform\_ioctl\_req\_t

```
typedef int32_t tfm_platform_ioctl_req_t
```

Definition at line 47 of file tfm\_platform\_api.h.

### 8.70.3 Enumeration Type Documentation

#### 8.70.3.1 tfm\_platform\_err\_t

```
enum tfm_platform_err_t
```

Platform service error types.

##### Enumerator

|                                 |  |
|---------------------------------|--|
| TFM_PLATFORM_ERR_SUCCESS        |  |
| TFM_PLATFORM_ERR_SYSTEM_ERROR   |  |
| TFM_PLATFORM_ERR_INVALID_PARAM  |  |
| TFM_PLATFORM_ERR_NOT_SUPPORTED  |  |
| TFM_PLATFORM_ERR_FORCE_INT_SIZE |  |

Definition at line 37 of file tfm\_platform\_api.h.

### 8.70.4 Function Documentation

#### 8.70.4.1 tfm\_platform\_ioctl()

```
enum tfm_platform_err_t tfm_platform_ioctl (
 tfm_platform_ioctl_req_t request,
 psa_invec * input,
 psa_outvec * output)
```

Performs a platform-specific service.

## Parameters

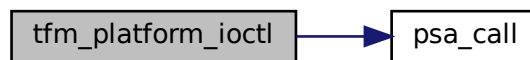
|         |                |                                                              |
|---------|----------------|--------------------------------------------------------------|
| in      | <i>request</i> | Request identifier (valid values vary based on the platform) |
| in      | <i>input</i>   | Input buffer to the requested service (or NULL)              |
| in, out | <i>output</i>  | Output buffer to the requested service (or NULL)             |

## Returns

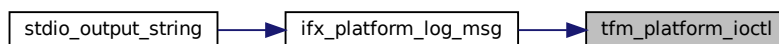
Returns values as specified by the [tfm\\_platform\\_err\\_t](#)

Definition at line 30 of file `tfm_platform_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.70.4.2 tfm\_platform\_nv\_counter\_increment()

```
enum tfm_platform_err_t tfm_platform_nv_counter_increment (
 uint32_t counter_id)
```

Increments the given non-volatile (NV) counter by one.

## Parameters

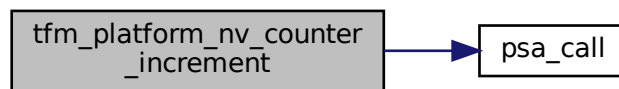
|    |                   |                |
|----|-------------------|----------------|
| in | <i>counter_id</i> | NV counter ID. |
|----|-------------------|----------------|

**Returns**

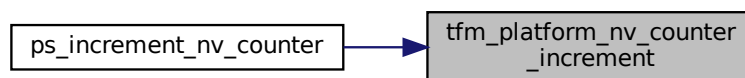
TFM\_PLATFORM\_ERR\_SUCCESS if the value is read correctly. Otherwise, it returns TFM\_PLATFORM\_ERR\_SYSTEM\_ERROR.

Definition at line 67 of file tfm\_platform\_api.c.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.70.4.3 tfm\_platform\_nv\_counter\_read()**

```

enum tfm_platform_err_t tfm_platform_nv_counter_read (
 uint32_t counter_id,
 uint32_t size,
 uint8_t * val)

```

Reads the given non-volatile (NV) counter.

**Parameters**

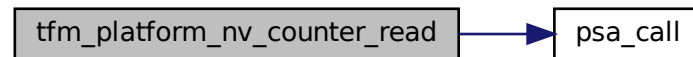
|     |                   |                                                        |
|-----|-------------------|--------------------------------------------------------|
| in  | <i>counter_id</i> | NV counter ID.                                         |
| in  | <i>size</i>       | Size of the buffer to store NV counter value in bytes. |
| out | <i>val</i>        | Pointer to store the current NV counter value.         |

#### Returns

TFM\_PLATFORM\_ERR\_SUCCESS if the value is read correctly. Otherwise, it returns TFM\_PLATFORM\_ERR\_SYSTEM\_ERROR.

Definition at line 87 of file tfm\_platform\_api.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.70.4.4 tfm\_platform\_system\_reset()

```
enum tfm_platform_err_t tfm_platform_system_reset (
 void)
```

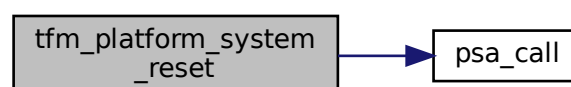
Resets the system.

#### Returns

Returns values as specified by the [tfm\\_platform\\_err\\_t](#)

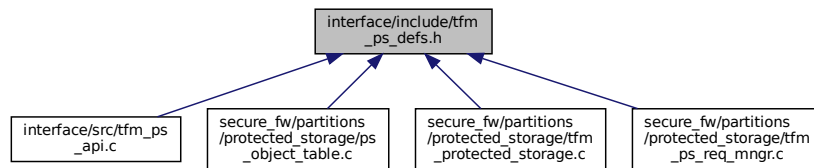
Definition at line 13 of file tfm\_platform\_api.c.

Here is the call graph for this function:



## 8.71 interface/include/tfm\_ps\_defs.h File Reference

This graph shows which files directly or indirectly include this file:



### Macros

- `#define TFM_PS_INVALID_UID 0`
- `#define TFM_PS_SET 1001`
- `#define TFM_PS_GET 1002`
- `#define TFM_PS_GET_INFO 1003`
- `#define TFM_PS_REMOVE 1004`
- `#define TFM_PS_GET_SUPPORT 1005`

### 8.71.1 Macro Definition Documentation

#### 8.71.1.1 TFM\_PS\_GET

```
#define TFM_PS_GET 1002
```

Definition at line 20 of file `tfm_ps_defs.h`.

#### 8.71.1.2 TFM\_PS\_GET\_INFO

```
#define TFM_PS_GET_INFO 1003
```

Definition at line 21 of file `tfm_ps_defs.h`.

#### 8.71.1.3 TFM\_PS\_GET\_SUPPORT

```
#define TFM_PS_GET_SUPPORT 1005
```

Definition at line 23 of file `tfm_ps_defs.h`.

#### 8.71.1.4 TFM\_PS\_INVALID\_UID

```
#define TFM_PS_INVALID_UID 0
```

Definition at line 16 of file `tfm_ps_defs.h`.

#### 8.71.1.5 TFM\_PS\_REMOVE

```
#define TFM_PS_REMOVE 1004
```

Definition at line 22 of file `tfm_ps_defs.h`.

## 8.71.1.6 TFM\_PS\_SET

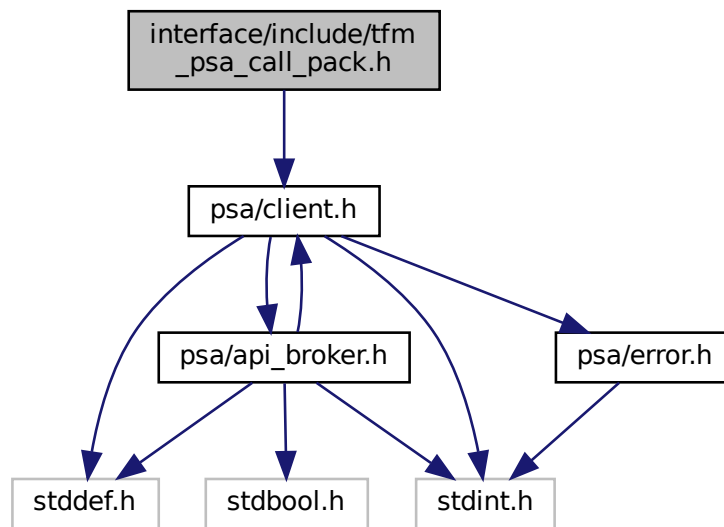
```
#define TFM_PS_SET 1001
```

Definition at line 19 of file tfm\_psa\_defs.h.

## 8.72 interface/include/tfm\_psa\_call\_pack.h File Reference

```
#include "psa/client.h"
```

Include dependency graph for tfm\_psa\_call\_pack.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define TYPE_MASK 0xFFFFUL`
- `#define IN_LEN_OFFSET 24`
- `#define IN_LEN_MASK (0x7UL << IN_LEN_OFFSET)`
- `#define OUT_LEN_OFFSET 16`
- `#define OUT_LEN_MASK (0x7UL << OUT_LEN_OFFSET)`
- `#define NS_VEC_DESC_BIT 0x80000000UL`
- `#define NS_INVEC_OFFSET 27`
- `#define NS_INVEC_BIT (1UL << NS_INVEC_OFFSET)`
- `#define NS_OUTVEC_OFFSET 19`
- `#define NS_OUTVEC_BIT (1UL << NS_OUTVEC_OFFSET)`
- `#define PARAM_PACK(type, in_len, out_len)`
- `#define PARAM_UNPACK_TYPE(ctrl_param) ((int32_t)(int16_t)((ctrl_param) & TYPE_MASK))`
- `#define PARAM_UNPACK_IN_LEN(ctrl_param) ((size_t)(((ctrl_param) & IN_LEN_MASK) >> IN_LEN_OFFSET))`

- `#define PARAM_UNPACK_OUT_LEN(ctrl_param) ((size_t)(((ctrl_param) & OUT_LEN_MASK) >> OUT_LEN_OFFSET))`
- `#define PARAM_SET_NS_VEC(ctrl_param) ((ctrl_param) | NS_VEC_DESC_BIT)`
- `#define PARAM_IS_NS_VEC(ctrl_param) ((ctrl_param) & NS_VEC_DESC_BIT)`
- `#define PARAM_SET_NS_INVEC(ctrl_param) ((ctrl_param) | NS_INVEC_BIT)`
- `#define PARAM_IS_NS_INVEC(ctrl_param) ((ctrl_param) & NS_INVEC_BIT)`
- `#define PARAM_SET_NS_OUTVEC(ctrl_param) ((ctrl_param) | NS_OUTVEC_BIT)`
- `#define PARAM_IS_NS_OUTVEC(ctrl_param) ((ctrl_param) & NS_OUTVEC_BIT)`
- `#define PARAM_HAS_IOVEC(ctrl_param) ((ctrl_param) != (uint32_t)PARAM_UNPACK_TYPE(ctrl_param))`

## Functions

- `psa_status_t tfm_psa_call_pack (psa_handle_t handle, uint32_t ctrl_param, const psa_invec *in_vec, psa_outvec *out_vec)`

## 8.72.1 Macro Definition Documentation

### 8.72.1.1 IN\_LEN\_MASK

```
#define IN_LEN_MASK (0x7UL << IN_LEN_OFFSET)
```

Definition at line 29 of file `tfm_psa_call_pack.h`.

### 8.72.1.2 IN\_LEN\_OFFSET

```
#define IN_LEN_OFFSET 24
```

Definition at line 28 of file `tfm_psa_call_pack.h`.

### 8.72.1.3 NS\_INVEC\_BIT

```
#define NS_INVEC_BIT (1UL << NS_INVEC_OFFSET)
```

Definition at line 42 of file `tfm_psa_call_pack.h`.

### 8.72.1.4 NS\_INVEC\_OFFSET

```
#define NS_INVEC_OFFSET 27
```

Definition at line 41 of file `tfm_psa_call_pack.h`.

### 8.72.1.5 NS\_OUTVEC\_BIT

```
#define NS_OUTVEC_BIT (1UL << NS_OUTVEC_OFFSET)
```

Definition at line 44 of file `tfm_psa_call_pack.h`.

### 8.72.1.6 NS\_OUTVEC\_OFFSET

```
#define NS_OUTVEC_OFFSET 19
```

Definition at line 43 of file `tfm_psa_call_pack.h`.

### 8.72.1.7 NS\_VEC\_DESC\_BIT

```
#define NS_VEC_DESC_BIT 0x80000000UL
```

Definition at line 39 of file `tfm_psa_call_pack.h`.



### 8.72.1.8 OUT\_LEN\_MASK

```
#define OUT_LEN_MASK (0x7UL << OUT_LEN_OFFSET)
```

Definition at line 32 of file tfm\_psa\_call\_pack.h.

### 8.72.1.9 OUT\_LEN\_OFFSET

```
#define OUT_LEN_OFFSET 16
```

Definition at line 31 of file tfm\_psa\_call\_pack.h.

### 8.72.1.10 PARAM\_HAS\_IOVEC

```
#define PARAM_HAS_IOVEC(
 ctrl_param) ((ctrl_param) != (uint32_t)PARAM_UNPACK_TYPE(ctrl_param))
```

Definition at line 68 of file tfm\_psa\_call\_pack.h.

### 8.72.1.11 PARAM\_IS\_NS\_INVEC

```
#define PARAM_IS_NS_INVEC(
 ctrl_param) ((ctrl_param) & NS_INVEC_BIT)
```

Definition at line 64 of file tfm\_psa\_call\_pack.h.

### 8.72.1.12 PARAM\_IS\_NS\_OUTVEC

```
#define PARAM_IS_NS_OUTVEC(
 ctrl_param) ((ctrl_param) & NS_OUTVEC_BIT)
```

Definition at line 66 of file tfm\_psa\_call\_pack.h.

### 8.72.1.13 PARAM\_IS\_NS\_VEC

```
#define PARAM_IS_NS_VEC(
 ctrl_param) ((ctrl_param) & NS_VEC_DESC_BIT)
```

Definition at line 61 of file tfm\_psa\_call\_pack.h.

### 8.72.1.14 PARAM\_PACK

```
#define PARAM_PACK(
 type,
 in_len,
 out_len)
```

**Value:**

```
((((uint32_t)(type)) & TYPE_MASK) | \
 (((uint32_t)(in_len)) << IN_LEN_OFFSET) & IN_LEN_MASK) | \
 (((uint32_t)(out_len)) << OUT_LEN_OFFSET) & OUT_LEN_MASK)
```

Definition at line 46 of file tfm\_psa\_call\_pack.h.

### 8.72.1.15 PARAM\_SET\_NS\_INVEC

```
#define PARAM_SET_NS_INVEC(
 ctrl_param) ((ctrl_param) | NS_INVEC_BIT)
```

Definition at line 63 of file tfm\_psa\_call\_pack.h.

**8.72.1.16 PARAM\_SET\_NS\_OUTVEC**

```
#define PARAM_SET_NS_OUTVEC(
 ctrl_param) ((ctrl_param) | NS_OUTVEC_BIT)
```

Definition at line 65 of file tfm\_psa\_call\_pack.h.

**8.72.1.17 PARAM\_SET\_NS\_VEC**

```
#define PARAM_SET_NS_VEC(
 ctrl_param) ((ctrl_param) | NS_VEC_DESC_BIT)
```

Definition at line 60 of file tfm\_psa\_call\_pack.h.

**8.72.1.18 PARAM\_UNPACK\_IN\_LEN**

```
#define PARAM_UNPACK_IN_LEN(
 ctrl_param) ((size_t)((ctrl_param) & IN_LEN_MASK) >> IN_LEN_OFFSET))
```

Definition at line 54 of file tfm\_psa\_call\_pack.h.

**8.72.1.19 PARAM\_UNPACK\_OUT\_LEN**

```
#define PARAM_UNPACK_OUT_LEN(
 ctrl_param) ((size_t)((ctrl_param) & OUT_LEN_MASK) >> OUT_LEN_OFFSET))
```

Definition at line 57 of file tfm\_psa\_call\_pack.h.

**8.72.1.20 PARAM\_UNPACK\_TYPE**

```
#define PARAM_UNPACK_TYPE(
 ctrl_param) ((int32_t)(int16_t)((ctrl_param) & TYPE_MASK))
```

Definition at line 51 of file tfm\_psa\_call\_pack.h.

**8.72.1.21 TYPE\_MASK**

```
#define TYPE_MASK 0xFFFFFUL
```

Definition at line 26 of file tfm\_psa\_call\_pack.h.

**8.72.2 Function Documentation****8.72.2.1 tfm\_psa\_call\_pack()**

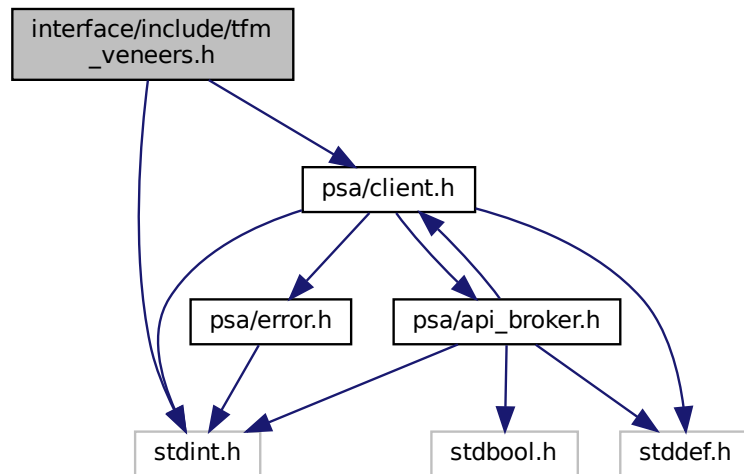
```
psa_status_t tfm_psa_call_pack (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * in_vec,
 psa_outvec * out_vec)
```

Definition at line 30 of file psa\_api\_ipc.c.

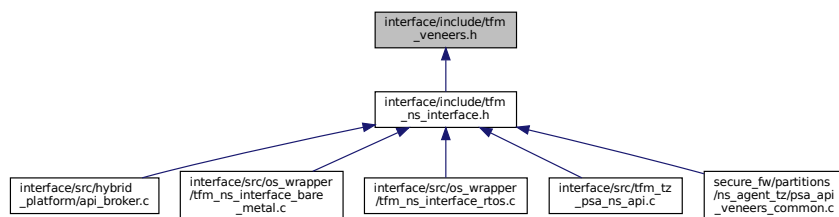
**8.73 interface/include/tfm\_veneers.h File Reference**

```
#include <stdint.h>
#include "psa/client.h"
```

Include dependency graph for tfm\_veneers.h:



This graph shows which files directly or indirectly include this file:



## Functions

- `int32_t tfm_ns_check_safe_entry_veneer (void)`  
NS-side to perform a safety check for reentrancy before proceeding with a PSA Call.
- `int32_t tfm_ns_check_exit_veneer (void)`  
NS-side to let the test flag go.
- `uint32_t tfm_psa_framework_version_veneer (void)`  
Retrieve the version of the PSA Framework API that is implemented.
- `uint32_t tfm_psa_version_veneer (uint32_t sid)`  
Return version of secure function provided by secure binary.
- `psa_handle_t tfm_psa_connect_veneer (uint32_t sid, uint32_t version)`  
Connect to secure function.
- `psa_status_t tfm_psa_call_veneer (psa_handle_t handle, uint32_t ctrl_param, const psa_invec *in_vec, psa_outvec *out_vec)`  
Call a secure function referenced by a connection handle.
- `void tfm_psa_close_veneer (psa_handle_t handle)`  
Close connection to secure function referenced by a connection handle.

## 8.73.1 Function Documentation

### 8.73.1.1 tfm\_ns\_check\_exit\_veneer()

```
int32_t tfm_ns_check_exit_veneer (
 void)
```

NS-side to let the test flag go.

#### Note

Requires TFM\_TZ\_REENTRANCY\_CHECK option to be enabled

#### Returns

0 on success.

-1 failure to clear the test flag.

Definition at line 118 of file psa\_api\_veneers\_common.c.

### 8.73.1.2 tfm\_ns\_check\_safe\_entry\_veneer()

```
int32_t tfm_ns_check_safe_entry_veneer (
 void)
```

NS-side to perform a safety check for reentrancy before proceeding with a PSA Call.

#### Note

Requires TFM\_TZ\_REENTRANCY\_CHECK option to be enabled

#### Returns

0 on success.

-1 the test flag was already set.

-2 invalid reenter attempt.

Definition at line 61 of file psa\_api\_veneers\_common.c.

### 8.73.1.3 tfm\_psa\_call\_veneer()

```
psa_status_t tfm_psa_call_veneer (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * in_vec,
 psa_outvec * out_vec)
```

Call a secure function referenced by a connection handle.

#### Parameters

|         |                   |                                                                             |
|---------|-------------------|-----------------------------------------------------------------------------|
| in      | <i>handle</i>     | Handle to connection.                                                       |
| in      | <i>ctrl_param</i> | Parameters combined in uint32_t, includes request type, in_num and out_num. |
| in      | <i>in_vec</i>     | Array of input <a href="#">psa_invec</a> structures.                        |
| in, out | <i>out_vec</i>    | Array of output <a href="#">psa_outvec</a> structures.                      |

**Returns**

Returns `psa_status_t` status code.

Definition at line 67 of file `psa_api_veneers.c`.

**8.73.1.4 tfm\_psa\_close\_veneer()**

```
void tfm_psa_close_veneer (
 psa_handle_t handle)
```

Close connection to secure function referenced by a connection handle.

**Parameters**

|    |               |                      |
|----|---------------|----------------------|
| in | <i>handle</i> | Handle to connection |
|----|---------------|----------------------|

Definition at line 138 of file `psa_api_veneers.c`.

**8.73.1.5 tfm\_psa\_connect\_veneer()**

```
psa_handle_t tfm_psa_connect_veneer (
 uint32_t sid,
 uint32_t version)
```

Connect to secure function.

**Parameters**

|    |                |                                    |
|----|----------------|------------------------------------|
| in | <i>sid</i>     | ID of secure service.              |
| in | <i>version</i> | Version of SF requested by client. |

**Returns**

Returns handle to connection.

Definition at line 129 of file `psa_api_veneers.c`.

Here is the caller graph for this function:

**8.73.1.6 tfm\_psa\_framework\_version\_veneer()**

```
uint32_t tfm_psa_framework_version_veneer (
 void)
```

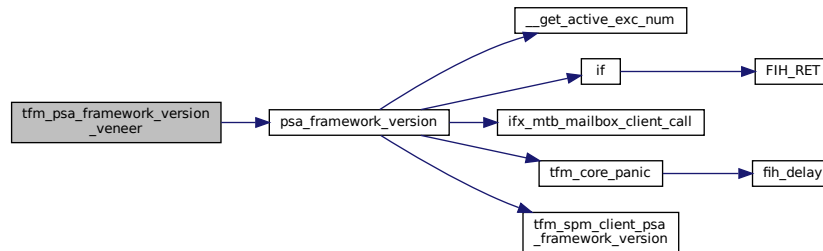
Retrieve the version of the PSA Framework API that is implemented.

**Returns**

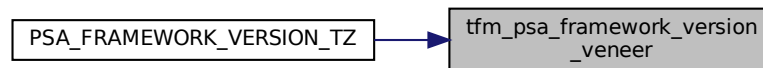
The version of the PSA Framework.

Definition at line 29 of file `psa_api_veneers.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.73.1.7 tfm\_psa\_version\_veneer()**

```
uint32_t tfm_psa_version_veneer (
 uint32_t sid)
```

Return version of secure function provided by secure binary.

**Parameters**

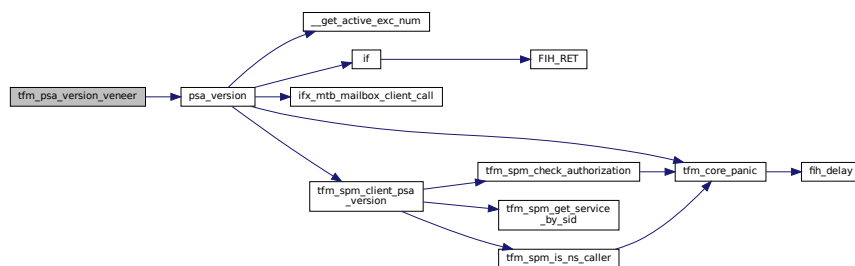
|                 |                  |                       |
|-----------------|------------------|-----------------------|
| <code>in</code> | <code>sid</code> | ID of secure service. |
|-----------------|------------------|-----------------------|

## Returns

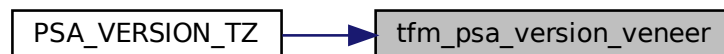
Version number of secure function.

Definition at line 48 of file psa\_api\_veneers.c.

Here is the call graph for this function:



Here is the caller graph for this function:



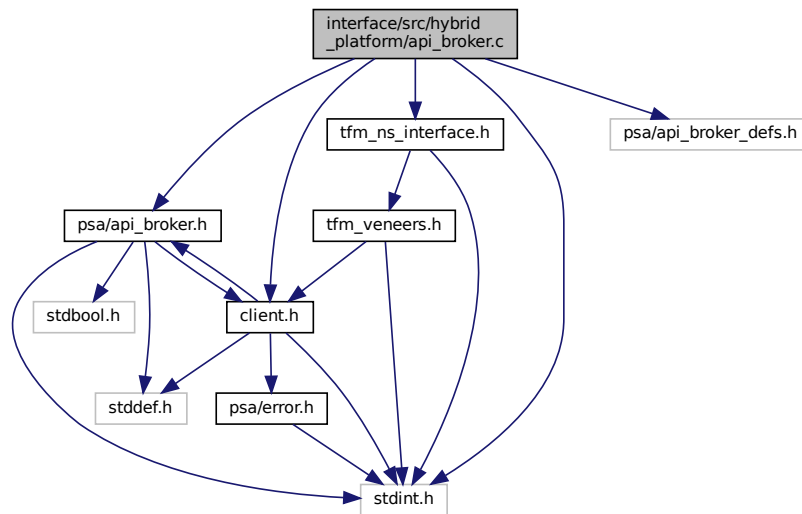
## 8.74 interface/src/hybrid\_platform/apiBroker.c File Reference

```

#include <stdint.h>
#include "psa/apiBroker.h"
#include "psa/client.h"
#include "tfm_ns_interface.h"
#include "psa/apiBroker_defs.h"

```

Include dependency graph for `api_broker.c`:



## Functions

- `int32_t tfm_hybrid_plat_api_broker_set_exec_target` (enum `tfm_hybrid_plat_api_broker_pe_exec_t` id)
- `uint32_t psa_framework_version` (void)  
Retrieve the version of the PSA Framework API that is implemented.
- `uint32_t psa_version` (uint32\_t sid)  
Retrieve the version of an RoT Service or indicate that it is not present on this system.
- `psa_status_t psa_call` (`psa_handle_t` handle, int32\_t type, const `psa_invec` \*in\_vec, size\_t in\_len, `psa_outvec` \*out\_vec, size\_t out\_len)  
Call an RoT Service on an established connection.
- `psa_handle_t psa_connect` (uint32\_t sid, uint32\_t version)  
Connect to an RoT Service by its SID.
- `void psa_close` (`psa_handle_t` handle)  
Close a connection to an RoT Service.

## 8.74.1 Function Documentation

### 8.74.1.1 `psa_call()`

```

psa_status_t psa_call (
 psa_handle_t handle,
 int32_t type,
 const psa_invec * in_vec,
 size_t in_len,
 psa_outvec * out_vec,
 size_t out_len)

```

Call an RoT Service on an established connection.



**Note**

FF-M 1.0 proposes 6 parameters for `psa_call` but the secure gateway ABI support at most 4 parameters. T↔ F-M chooses to encode 'in\_len', 'out\_len', and 'type' into a 32-bit integer to improve efficiency. Compared with struct-based encoding, this method saves extra memory check and memory copy operation. The disadvantage is that the 'type' range has to be reduced into a 16-bit integer. So with this encoding, the valid range for 'type' is 0-32767.

**Parameters**

|         |                |                                                                             |
|---------|----------------|-----------------------------------------------------------------------------|
| in      | <i>handle</i>  | A handle to an established connection.                                      |
| in      | <i>type</i>    | The request type. Must be zero( <a href="#">PSA_IPC_CALL</a> ) or positive. |
| in      | <i>in_vec</i>  | Array of input <a href="#">psa_invec</a> structures.                        |
| in      | <i>in_len</i>  | Number of input <a href="#">psa_invec</a> structures.                       |
| in, out | <i>out_vec</i> | Array of output <a href="#">psa_outvec</a> structures.                      |
| in      | <i>out_len</i> | Number of output <a href="#">psa_outvec</a> structures.                     |

**Return values**

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\geq 0$                          | RoT Service-specific status value.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| $< 0$                             | RoT Service-specific error code.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>PSA_ERROR_PROGRAMMER_ERROR</i> | <p>The connection has been terminated by the RoT Service. The call is a PROGRAMMER ERROR if one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• An invalid handle was passed.</li> <li>• The connection is already handling a request.</li> <li>• <code>type &lt; 0</code>.</li> <li>• An invalid memory reference was provided.</li> <li>• <code>in_len + out_len &gt; PSA_MAX_IOVEC</code>.</li> <li>• The message is unrecognized by the RoT Service or incorrectly formatted.</li> </ul> |

Definition at line 88 of file `api_broker.c`.

**8.74.1.2 `psa_close()`**

```
void psa_close (
 psa_handle_t handle)
```

Close a connection to an RoT Service.

**Parameters**

|    |               |                                                            |
|----|---------------|------------------------------------------------------------|
| in | <i>handle</i> | A handle to an established connection, or the null handle. |
|----|---------------|------------------------------------------------------------|

**Note**

The call is a PROGRAMMER ERROR if one or more of the following occurs:

- An invalid handle was provided that is not the null handle.
- The connection is currently handling a request.

Definition at line 106 of file api\_broker.c.

### 8.74.1.3 psa\_connect()

```
psa_handle_t psa_connect (
 uint32_t sid,
 uint32_t version)
```

Connect to an RoT Service by its SID.

#### Parameters

|    |                |                                       |
|----|----------------|---------------------------------------|
| in | <i>sid</i>     | ID of the RoT Service to connect to.  |
| in | <i>version</i> | Requested version of the RoT Service. |

#### Return values

|                                     |                                                                                                                                                                                                                                                                                      |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| >                                   | 0 A handle for the connection.                                                                                                                                                                                                                                                       |
| <i>PSA_ERROR_CONNECTION_REFUSED</i> | The SPM or RoT Service has refused the connection.                                                                                                                                                                                                                                   |
| <i>PSA_ERROR_CONNECTION_BUSY</i>    | The SPM or RoT Service cannot make the connection at the moment.                                                                                                                                                                                                                     |
| <i>PROGRAMMER ERROR</i>             | The call is a PROGRAMMER ERROR if one or more of the following are true: <ul style="list-style-type: none"> <li>• The RoT Service ID is not present.</li> <li>• The RoT Service version is not supported.</li> <li>• The caller is not allowed to access the RoT service.</li> </ul> |

Definition at line 101 of file api\_broker.c.

### 8.74.1.4 psa\_framework\_version()

```
uint32_t psa_framework_version (
 void)
```

Retrieve the version of the PSA Framework API that is implemented.

#### Returns

version The version of the PSA Framework implementation that is providing the runtime services to the caller. The major and minor version are encoded as follows:

- version[15:8] – major version number.
- version[7:0] – minor version number.

Definition at line 78 of file api\_broker.c.

### 8.74.1.5 psa\_version()

```
uint32_t psa_version (
 uint32_t sid)
```

Retrieve the version of an RoT Service or indicate that it is not present on this system.

#### Parameters

|    |            |                                 |
|----|------------|---------------------------------|
| in | <i>sid</i> | ID of the RoT Service to query. |
|----|------------|---------------------------------|

## Return values

|                               |                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------|
| <code>PSA_VERSION_NONE</code> | The RoT Service is not implemented, or the caller is not permitted to access the service. |
| <code>&gt;</code>             | 0 The version of the implemented RoT Service.                                             |

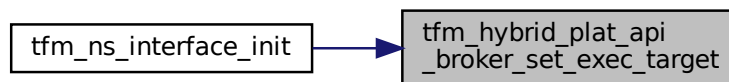
Definition at line 83 of file `api_broker.c`.

#### 8.74.1.6 `tfm_hybrid_plat_api_broker_set_exec_target()`

```
int32_t tfm_hybrid_plat_api_broker_set_exec_target (
 enum tfm_hybrid_plat_api_broker_pe_exec_t id)
```

Definition at line 60 of file `api_broker.c`.

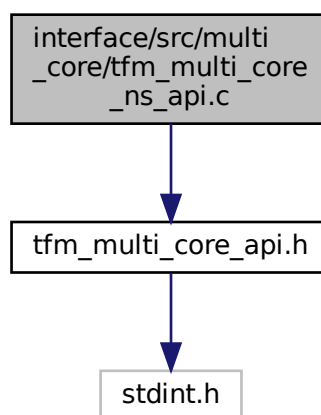
Here is the caller graph for this function:



## 8.75 interface/src/multi\_core/tfm\_multi\_core\_ns\_api.c File Reference

```
#include "tfm_multi_core_api.h"
```

Include dependency graph for `tfm_multi_core_ns_api.c`:



## Functions

- `int32_t` `tfm_ns_wait_for_s_cpu_ready` (`void`)

*Called on the non-secure CPU. Flags that the non-secure side has completed its initialization. Waits, if necessary, for the secure CPU to flag that it has completed its initialization.*

## 8.75.1 Function Documentation

### 8.75.1.1 tfm\_ns\_wait\_for\_s\_cpu\_ready()

```
int32_t tfm_ns_wait_for_s_cpu_ready (
 void)
```

Called on the non-secure CPU. Flags that the non-secure side has completed its initialization. Waits, if necessary, for the secure CPU to flag that it has completed its initialization.

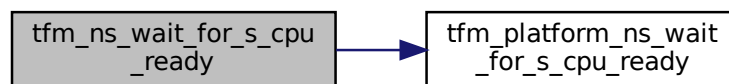
#### Returns

Return 0 if succeeds.

Otherwise, return specific error code.

Definition at line 10 of file tfm\_multi\_core\_ns\_api.c.

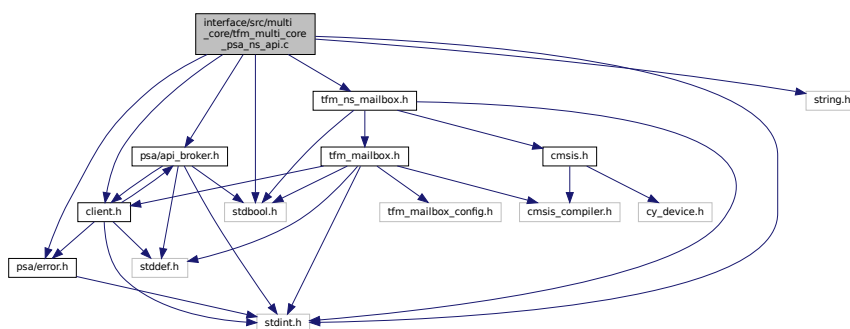
Here is the call graph for this function:



## 8.76 interface/src/multi\_core/tfm\_multi\_core\_psa\_ns\_api.c File Reference

```
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include "psa/api_broker.h"
#include "psa/client.h"
#include "psa/error.h"
#include "tfm_ns_mailbox.h"
```

Include dependency graph for tfm\_multi\_core\_psa\_ns\_api.c:



## Macros

- `#define NON_SECURE_CLIENT_ID (-1)`
- `#define PSA_INTER_CORE_COMM_ERR (INT32_MIN + 0xFF)`

## Functions

- `uint32_t PSA_FRAMEWORK_VERSION_MULTICORE (void)`
- `uint32_t PSA_VERSION_MULTICORE (uint32_t sid)`
- `psa_handle_t PSA_CONNECT_MULTICORE (uint32_t sid, uint32_t version)`
- `psa_status_t PSA_CALL_MULTICORE (psa_handle_t handle, int32_t type, const psa_invec *in_vec, size_t in_len, psa_outvec *out_vec, size_t out_len)`
- `void PSA_CLOSE_MULTICORE (psa_handle_t handle)`

## 8.76.1 Macro Definition Documentation

### 8.76.1.1 NON\_SECURE\_CLIENT\_ID

```
#define NON_SECURE_CLIENT_ID (-1)
```

Definition at line 31 of file `tfm_multi_core_psa_ns_api.c`.

### 8.76.1.2 PSA\_INTER\_CORE\_COMM\_ERR

```
#define PSA_INTER_CORE_COMM_ERR (INT32_MIN + 0xFF)
```

Definition at line 37 of file `tfm_multi_core_psa_ns_api.c`.

## 8.76.2 Function Documentation

### 8.76.2.1 PSA\_CALL\_MULTICORE()

```
psa_status_t PSA_CALL_MULTICORE (
 psa_handle_t handle,
 int32_t type,
 const psa_invec * in_vec,
 size_t in_len,
 psa_outvec * out_vec,
 size_t out_len)
```

Definition at line 103 of file `tfm_multi_core_psa_ns_api.c`.

### 8.76.2.2 PSA\_CLOSE\_MULTICORE()

```
void PSA_CLOSE_MULTICORE (
 psa_handle_t handle)
```

Definition at line 145 of file `tfm_multi_core_psa_ns_api.c`.

### 8.76.2.3 PSA\_CONNECT\_MULTICORE()

```
psa_handle_t PSA_CONNECT_MULTICORE (
 uint32_t sid,
 uint32_t version)
```

Definition at line 83 of file `tfm_multi_core_psa_ns_api.c`.

Here is the call graph for this function:



#### 8.76.2.4 PSA\_FRAMEWORK\_VERSION\_MULTICORE()

```
uint32_t PSA_FRAMEWORK_VERSION_MULTICORE (
 void)
```

Definition at line 47 of file `tfm_multi_core_psa_ns_api.c`.

Here is the call graph for this function:



#### 8.76.2.5 PSA\_VERSION\_MULTICORE()

```
uint32_t PSA_VERSION_MULTICORE (
 uint32_t sid)
```

Definition at line 64 of file `tfm_multi_core_psa_ns_api.c`.

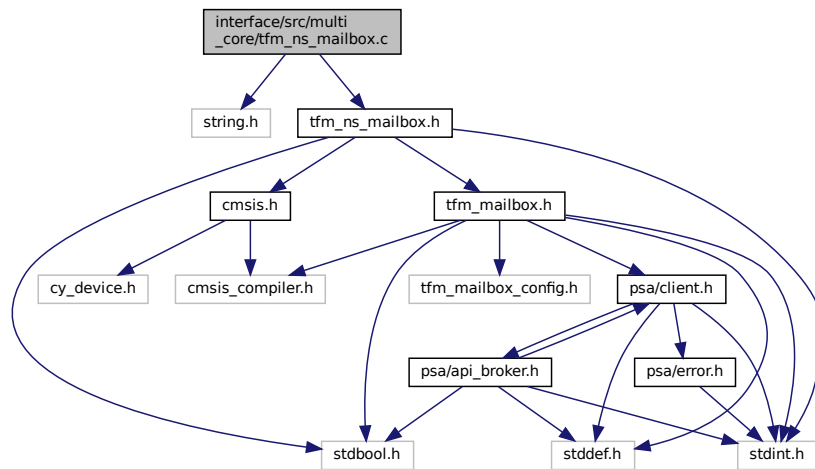
Here is the call graph for this function:



## 8.77 interface/src/multi\_core/tfm\_ns\_mailbox.c File Reference

```
#include <string.h>
#include "tfm_ns_mailbox.h"
```

Include dependency graph for tfm\_ns\_mailbox.c:



## Functions

- `int32_t tfm_ns_mailbox_client_call` (uint32\_t call\_type, const struct `psa_client_params_t` \*params, int32\_t client\_id, struct `mailbox_reply_t` \*reply)  
Send PSA client call to SPE via mailbox. Wait and fetch PSA client call result.
- `void mailbox_set_queue_ptr` (struct `ns_mailbox_queue_t` \*queue)  
Set the NS mailbox queue in the respective mailbox implementation.

### 8.77.1 Function Documentation

#### 8.77.1.1 mailbox\_set\_queue\_ptr()

```
void mailbox_set_queue_ptr (
 struct ns_mailbox_queue_t * queue)
```

Set the NS mailbox queue in the respective mailbox implementation.

Definition at line 331 of file `tfm_ns_mailbox.c`.

#### 8.77.1.2 tfm\_ns\_mailbox\_client\_call()

```
int32_t tfm_ns_mailbox_client_call (
 uint32_t call_type,
 const struct psa_client_params_t * params,
 int32_t client_id,
 struct mailbox_reply_t * reply)
```

Send PSA client call to SPE via mailbox. Wait and fetch PSA client call result.

#### Parameters

|     |                  |                                                                                                                                            |
|-----|------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>call_type</i> | PSA client call type                                                                                                                       |
| in  | <i>params</i>    | Parameters used for PSA client call                                                                                                        |
| in  | <i>client_id</i> | Optional client ID of non-secure caller. It is required to identify the non-secure caller when NSPE OS enforces non-secure task isolation. |
| out | <i>reply</i>     | The buffer written with PSA client call result.                                                                                            |

## Return values

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| <code>MAILBOX_SUCCESS</code> | The PSA client call is completed successfully.   |
| <i>Other</i>                 | return code Operation failed with an error code. |

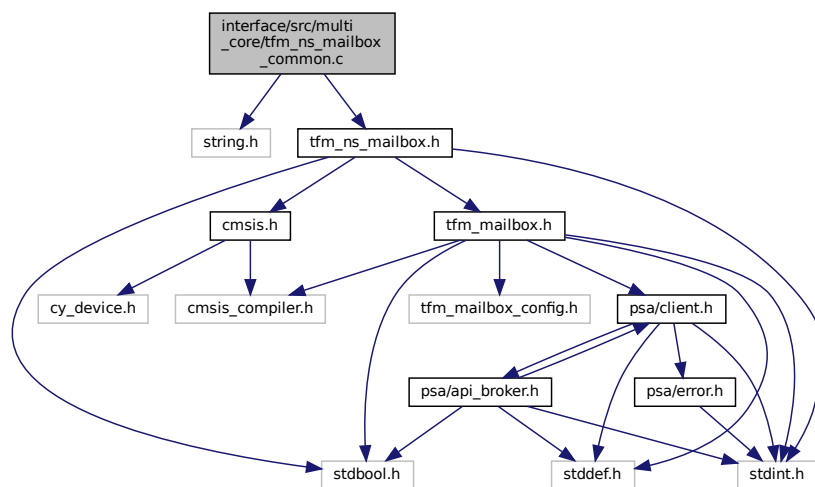
Definition at line 186 of file `tfm_ns_mailbox.c`.

## 8.78 interface/src/multi\_core/tfm\_ns\_mailbox\_common.c File Reference

```
#include <string.h>
```

```
#include "tfm_ns_mailbox.h"
```

Include dependency graph for `tfm_ns_mailbox_common.c`:



## Functions

- `int32_t tfm_ns_mailbox_init (struct ns\_mailbox\_queue\_t *queue)`  
NSPE mailbox initialization.

## Variables

- `bool cache\_enabled`

### 8.78.1 Function Documentation

#### 8.78.1.1 tfm\_ns\_mailbox\_init()

```
int32_t tfm_ns_mailbox_init (
 struct ns_mailbox_queue_t * queue)
```

NSPE mailbox initialization.

#### Parameters

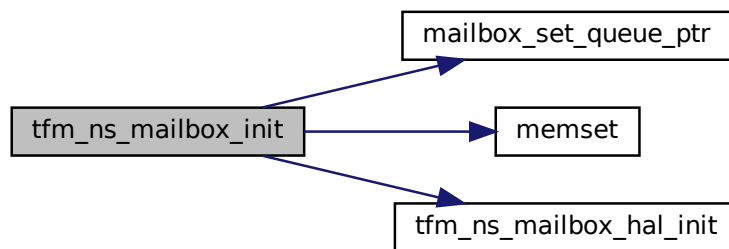
|    |              |                                                           |
|----|--------------|-----------------------------------------------------------|
| in | <i>queue</i> | The base address of NSPE mailbox queue to be initialized. |
|----|--------------|-----------------------------------------------------------|



## Return values

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| <code>MAILBOX_SUCCESS</code> | Operation succeeded.                             |
| <i>Other</i>                 | return code Operation failed with an error code. |

Definition at line 19 of file `tfm_ns_mailbox_common.c`.  
Here is the call graph for this function:



Here is the caller graph for this function:



## 8.78.2 Variable Documentation

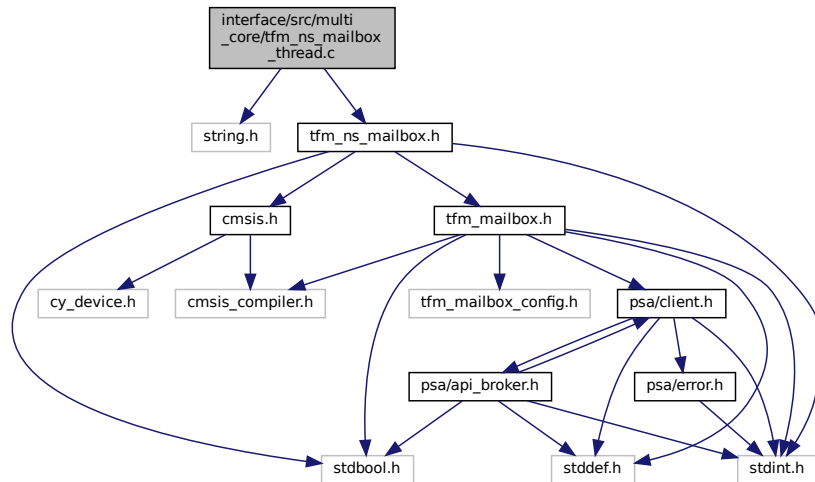
### 8.78.2.1 `cache_enabled`

```
bool cache_enabled
```

## 8.79 interface/src/multi\_core/tfm\_ns\_mailbox\_thread.c File Reference

```
#include <string.h>
#include "tfm_ns_mailbox.h"
```

Include dependency graph for `tfm_ns_mailbox_thread.c`:



## Data Structures

- struct [ns\\_mailbox\\_req\\_t](#)

## Macros

- `#define NOT_WOKEN 0x0`
- `#define WOKEN_UP 0x5C`

## Functions

- `int32_t tfm_ns_mailbox_client_call (uint32_t call_type, const struct psa\_client\_params\_t *params, int32_t client_id, struct mailbox\_reply\_t *reply)`  
*Send PSA client call to SPE via mailbox. Wait and fetch PSA client call result.*
- `void tfm_ns_mailbox_thread_runner (void *args)`
- `int32_t tfm_ns_mailbox_wake_reply_owner_isr (void)`
- `void mailbox_set_queue_ptr (struct ns\_mailbox\_queue\_t *queue)`  
*Set the NS mailbox queue in the respective mailbox implementation.*

## 8.79.1 Macro Definition Documentation

### 8.79.1.1 NOT\_WOKEN

```
#define NOT_WOKEN 0x0
```

Definition at line 13 of file `tfm_ns_mailbox_thread.c`.

### 8.79.1.2 WOKEN\_UP

```
#define WOKEN_UP 0x5C
```

Definition at line 14 of file `tfm_ns_mailbox_thread.c`.

## 8.79.2 Function Documentation

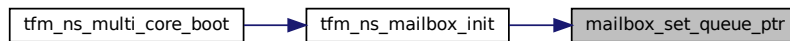
### 8.79.2.1 mailbox\_set\_queue\_ptr()

```
void mailbox_set_queue_ptr (
 struct ns_mailbox_queue_t * queue)
```

Set the NS mailbox queue in the respective mailbox implementation.

Definition at line 332 of file tfm\_ns\_mailbox\_thread.c.

Here is the caller graph for this function:



### 8.79.2.2 tfm\_ns\_mailbox\_client\_call()

```
int32_t tfm_ns_mailbox_client_call (
 uint32_t call_type,
 const struct psa_client_params_t * params,
 int32_t client_id,
 struct mailbox_reply_t * reply)
```

Send PSA client call to SPE via mailbox. Wait and fetch PSA client call result.

#### Parameters

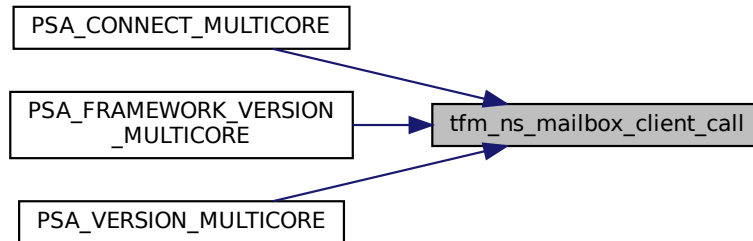
|     |                  |                                                                                                                                            |
|-----|------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>call_type</i> | PSA client call type                                                                                                                       |
| in  | <i>params</i>    | Parameters used for PSA client call                                                                                                        |
| in  | <i>client_id</i> | Optional client ID of non-secure caller. It is required to identify the non-secure caller when NSPE OS enforces non-secure task isolation. |
| out | <i>reply</i>     | The buffer written with PSA client call result.                                                                                            |

#### Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | The PSA client call is completed successfully.   |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 190 of file tfm\_ns\_mailbox\_thread.c.

Here is the caller graph for this function:



### 8.79.2.3 `tfm_ns_mailbox_thread_runner()`

```
void tfm_ns_mailbox_thread_runner (
 void * args)
```

Definition at line 224 of file `tfm_ns_mailbox_thread.c`.

### 8.79.2.4 `tfm_ns_mailbox_wake_reply_owner_isr()`

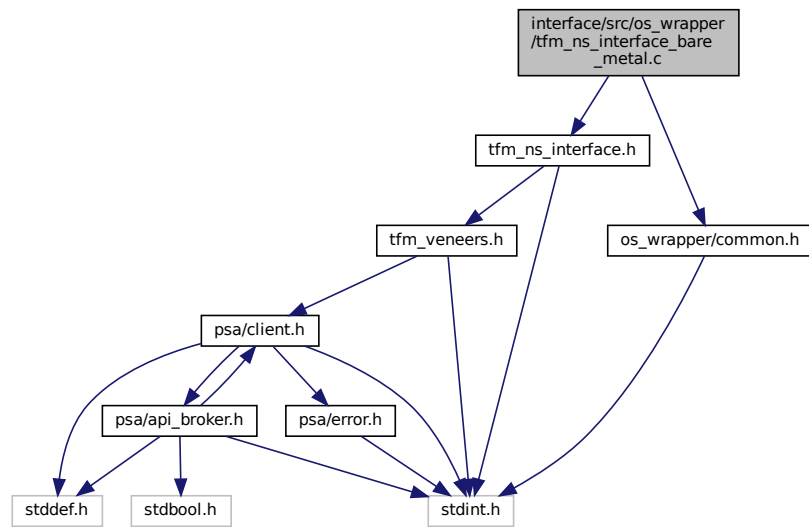
```
int32_t tfm_ns_mailbox_wake_reply_owner_isr (
 void)
```

Definition at line 252 of file `tfm_ns_mailbox_thread.c`.

## 8.80 `interface/src/os_wrapper/tfm_ns_interface_bare_metal.c` File Reference

```
#include "tfm_ns_interface.h"
#include "os_wrapper/common.h"
```

Include dependency graph for tfm\_ns\_interface\_bare\_metal.c:



## Functions

- `int32_t tfm_ns_interface_dispatch(veneer_fn fn, uint32_t arg0, uint32_t arg1, uint32_t arg2, uint32_t arg3)`  
NS interface, veneer function dispatcher.
- `uint32_t tfm_ns_interface_init(void)`  
NS interface initialization function.

## 8.80.1 Function Documentation

### 8.80.1.1 tfm\_ns\_interface\_dispatch()

```
int32_t tfm_ns_interface_dispatch (
 veneer_fn fn,
 uint32_t arg0,
 uint32_t arg1,
 uint32_t arg2,
 uint32_t arg3)
```

NS interface, veneer function dispatcher.

This function implements the dispatching mechanism for the desired veneer function, to be called with the parameters described from arg0 to arg3.

#### Note

NSPE can use default implementation of this function or implement this function according to NS specific implementation and actual usage scenario.

#### Parameters

|    |             |                                                 |
|----|-------------|-------------------------------------------------|
| in | <i>fn</i>   | Function pointer to the veneer function desired |
| in | <i>arg0</i> | Argument 0 of fn                                |
| in | <i>arg1</i> | Argument 1 of fn                                |

**Parameters**

|    |             |                  |
|----|-------------|------------------|
| in | <i>arg2</i> | Argument 2 of fn |
| in | <i>arg3</i> | Argument 3 of fn |

**Returns**

Returns the same return value of the requested veneer function

**Note**

This API must ensure the return value is from the veneer function. Other unrecoverable errors must be considered as fatal error and should not return.

Definition at line 23 of file `tfm_ns_interface_bare_metal.c`.

**8.80.1.2 tfm\_ns\_interface\_init()**

```
uint32_t tfm_ns_interface_init (
 void)
```

NS interface initialization function.

This function initializes TF-M NS interface.

**Note**

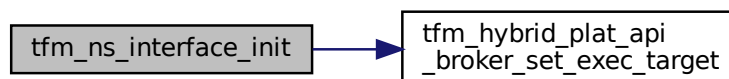
NSPE can use default implementation of this function or implement this function according to NS specific implementation and actual usage scenario.

**Returns**

[OS\\_WRAPPER\\_SUCCESS](#) on success or [OS\\_WRAPPER\\_ERROR](#) on error

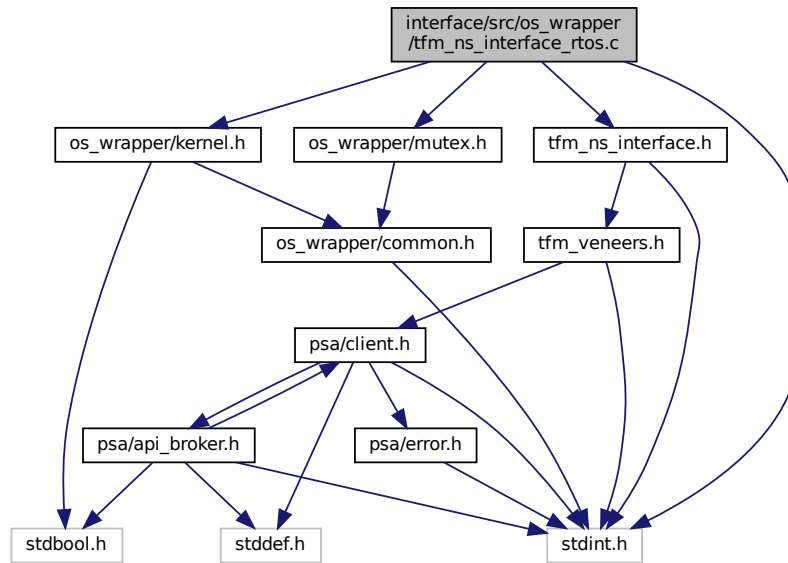
Definition at line 30 of file `tfm_ns_interface_bare_metal.c`.

Here is the call graph for this function:

**8.81 interface/src/os\_wrapper/tfm\_ns\_interface\_rtos.c File Reference**

```
#include <stdint.h>
#include "os_wrapper/kernel.h"
#include "os_wrapper/mutex.h"
#include "tfm_ns_interface.h"
```

Include dependency graph for tfm\_ns\_interface\_rtos.c:



## Functions

- `int32_t tfm_ns_interface_dispatch(veneer_fn fn, uint32_t arg0, uint32_t arg1, uint32_t arg2, uint32_t arg3)`  
NS interface, veneer function dispatcher.
- `uint32_t tfm_ns_interface_init(void)`  
NS interface initialization function.

## 8.81.1 Function Documentation

### 8.81.1.1 tfm\_ns\_interface\_dispatch()

```

int32_t tfm_ns_interface_dispatch (
 veneer_fn fn,
 uint32_t arg0,
 uint32_t arg1,
 uint32_t arg2,
 uint32_t arg3)

```

NS interface, veneer function dispatcher.

This function implements the dispatching mechanism for the desired veneer function, to be called with the parameters described from arg0 to arg3.

#### Note

NSPE can use default implementation of this function or implement this function according to NS specific implementation and actual usage scenario.

#### Parameters

|    |             |                                                 |
|----|-------------|-------------------------------------------------|
| in | <i>fn</i>   | Function pointer to the veneer function desired |
| in | <i>arg0</i> | Argument 0 of fn                                |

**Parameters**

|    |             |                  |
|----|-------------|------------------|
| in | <i>arg1</i> | Argument 1 of fn |
| in | <i>arg2</i> | Argument 2 of fn |
| in | <i>arg3</i> | Argument 3 of fn |

**Returns**

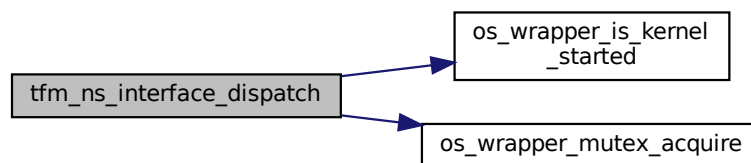
Returns the same return value of the requested veneer function

**Note**

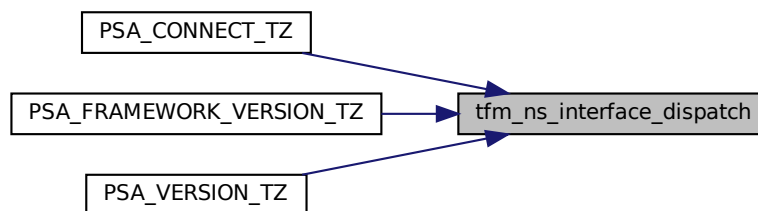
This API must ensure the return value is from the veneer function. Other unrecoverable errors must be considered as fatal error and should not return.

Definition at line 31 of file `tfm_ns_interface_rtos.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.81.1.2 tfm\_ns\_interface\_init()**

```
uint32_t tfm_ns_interface_init (
 void)
```

NS interface initialization function.

This function initializes TF-M NS interface.

**Note**

NSPE can use default implementation of this function or implement this function according to NS specific implementation and actual usage scenario.



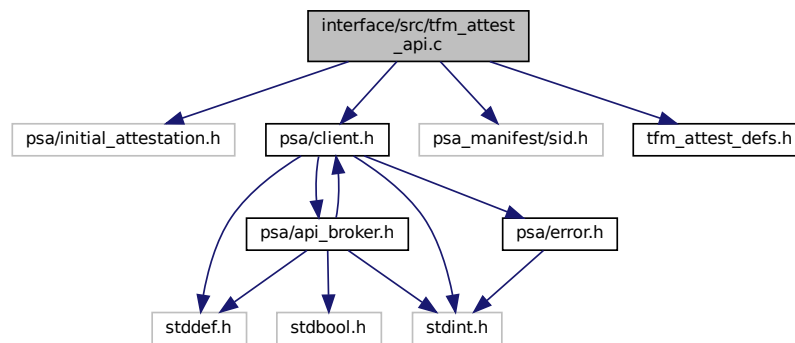
## Returns

[OS\\_WRAPPER\\_SUCCESS](#) on success or [OS\\_WRAPPER\\_ERROR](#) on error

Definition at line 59 of file `tfm_ns_interface_rtos.c`.

## 8.82 interface/src/tfm\_attest\_api.c File Reference

```
#include "psa/initial_attestation.h"
#include "psa/client.h"
#include "psa_manifest/sid.h"
#include "tfm_attest_defs.h"
Include dependency graph for tfm_attest_api.c:
```



## Functions

- [psa\\_status\\_t psa\\_initial\\_attest\\_get\\_token](#) (const uint8\_t \*auth\_challenge, size\_t challenge\_size, uint8\_t \*token\_buf, size\_t token\_buf\_size, size\_t \*token\_size)
- [psa\\_status\\_t psa\\_initial\\_attest\\_get\\_token\\_size](#) (size\_t challenge\_size, size\_t \*token\_size)

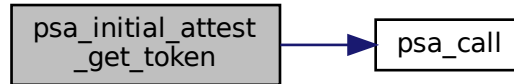
### 8.82.1 Function Documentation

#### 8.82.1.1 psa\_initial\_attest\_get\_token()

```
psa_status_t psa_initial_attest_get_token (
 const uint8_t * auth_challenge,
 size_t challenge_size,
 uint8_t * token_buf,
 size_t token_buf_size,
 size_t * token_size)
```

Definition at line 14 of file `tfm_attest_api.c`.

Here is the call graph for this function:



#### 8.82.1.2 `psa_initial_attest_get_token_size()`

```
psa_status_t psa_initial_attest_get_token_size (
 size_t challenge_size,
 size_t * token_size)
```

Definition at line 41 of file `tfm_attest_api.c`.

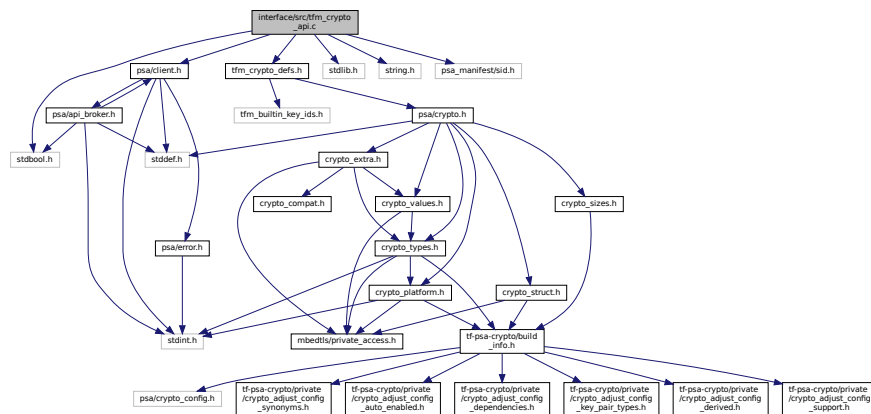
Here is the call graph for this function:



## 8.83 `interface/src/tfm_crypto_api.c` File Reference

```
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include "tfm_crypto_defs.h"
#include "psa/client.h"
#include "psa_manifest/sid.h"
```

Include dependency graph for tfm\_crypto\_api.c:



## Macros

- `#define API_DISPATCH(in_vec, out_vec)`
- `#define API_DISPATCH_NO_OUTVEC(in_vec)`
- `#define TFM_CRYPTO_API(ret, fun) ret fun`

Define the function signature of a TF-M Crypto API with return type *ret* and PSA Crypto API function name *fun*.

## Functions

- `psa_status_t psa_crypto_init (void)`  
*Library initialization.*
- `int psa_can_do_hash (psa_algorithm_t hash_alg)`
- `int psa_can_do_cipher (psa_key_type_t key_type, psa_algorithm_t cipher_alg)`
- `psa_status_t psa_import_key (const psa_key_attributes_t *attributes, const uint8_t *data, size_t data_length, psa_key_id_t *key)`  
*Import a key in binary format.*
- `psa_status_t psa_destroy_key (psa_key_id_t key)`  
*Destroy a key.*
- `psa_status_t psa_get_key_attributes (psa_key_id_t key, psa_key_attributes_t *attributes)`
- `psa_status_t psa_export_key (psa_key_id_t key, uint8_t *data, size_t data_size, size_t *data_length)`  
*Export a key in binary format.*
- `psa_status_t psa_export_public_key (psa_key_id_t key, uint8_t *data, size_t data_size, size_t *data_length)`  
*Export a public key or the public part of a key pair in binary format.*
- `psa_status_t psa_purge_key (psa_key_id_t key)`
- `psa_status_t psa_copy_key (psa_key_id_t source_key, const psa_key_attributes_t *attributes, psa_key_id_t *target_key)`
- `psa_status_t psa_cipher_generate_iv (psa_cipher_operation_t *operation, unsigned char *iv, size_t iv_size, size_t *iv_length)`
- `psa_status_t psa_cipher_set_iv (psa_cipher_operation_t *operation, const unsigned char *iv, size_t iv_length)`
- `psa_status_t psa_cipher_encrypt_setup (psa_cipher_operation_t *operation, psa_key_id_t key, psa_algorithm_t alg)`
- `psa_status_t psa_cipher_decrypt_setup (psa_cipher_operation_t *operation, psa_key_id_t key, psa_algorithm_t alg)`
- `psa_status_t psa_cipher_update (psa_cipher_operation_t *operation, const uint8_t *input, size_t input_length, unsigned char *output, size_t output_size, size_t *output_length)`
- `psa_status_t psa_cipher_abort (psa_cipher_operation_t *operation)`

- [psa\\_status\\_t psa\\_cipher\\_finish](#) ([psa\\_cipher\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
  - [psa\\_status\\_t psa\\_hash\\_setup](#) ([psa\\_hash\\_operation\\_t](#) \*operation, [psa\\_algorithm\\_t](#) alg)
  - [psa\\_status\\_t psa\\_hash\\_update](#) ([psa\\_hash\\_operation\\_t](#) \*operation, [const uint8\\_t](#) \*input, [size\\_t](#) input\_length)
  - [psa\\_status\\_t psa\\_hash\\_finish](#) ([psa\\_hash\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*hash, [size\\_t](#) hash\_size, [size\\_t](#) \*hash\_length)
  - [psa\\_status\\_t psa\\_hash\\_verify](#) ([psa\\_hash\\_operation\\_t](#) \*operation, [const uint8\\_t](#) \*hash, [size\\_t](#) hash\_length)
  - [psa\\_status\\_t psa\\_hash\\_abort](#) ([psa\\_hash\\_operation\\_t](#) \*operation)
  - [psa\\_status\\_t psa\\_hash\\_clone](#) ([const psa\\_hash\\_operation\\_t](#) \*source\_operation, [psa\\_hash\\_operation\\_t](#) \*target\_operation)
  - [psa\\_status\\_t psa\\_hash\\_compute](#) ([psa\\_algorithm\\_t](#) alg, [const uint8\\_t](#) \*input, [size\\_t](#) input\_length, [uint8\\_t](#) \*hash, [size\\_t](#) hash\_size, [size\\_t](#) \*hash\_length)
  - [psa\\_status\\_t psa\\_hash\\_compare](#) ([psa\\_algorithm\\_t](#) alg, [const uint8\\_t](#) \*input, [size\\_t](#) input\_length, [const uint8\\_t](#) \*hash, [size\\_t](#) hash\_length)
  - [psa\\_status\\_t psa\\_mac\\_sign\\_setup](#) ([psa\\_mac\\_operation\\_t](#) \*operation, [psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
  - [psa\\_status\\_t psa\\_mac\\_verify\\_setup](#) ([psa\\_mac\\_operation\\_t](#) \*operation, [psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
  - [psa\\_status\\_t psa\\_mac\\_update](#) ([psa\\_mac\\_operation\\_t](#) \*operation, [const uint8\\_t](#) \*input, [size\\_t](#) input\_length)
  - [psa\\_status\\_t psa\\_mac\\_sign\\_finish](#) ([psa\\_mac\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*mac, [size\\_t](#) mac\_size, [size\\_t](#) \*mac\_length)
  - [psa\\_status\\_t psa\\_mac\\_verify\\_finish](#) ([psa\\_mac\\_operation\\_t](#) \*operation, [const uint8\\_t](#) \*mac, [size\\_t](#) mac\_length)
  - [psa\\_status\\_t psa\\_mac\\_abort](#) ([psa\\_mac\\_operation\\_t](#) \*operation)
  - [psa\\_status\\_t psa\\_aead\\_encrypt](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, [const uint8\\_t](#) \*nonce, [size\\_t](#) nonce\_length, [const uint8\\_t](#) \*additional\_data, [size\\_t](#) additional\_data\_length, [const uint8\\_t](#) \*plaintext, [size\\_t](#) plaintext\_length, [uint8\\_t](#) \*ciphertext, [size\\_t](#) ciphertext\_size, [size\\_t](#) \*ciphertext\_length)
  - [psa\\_status\\_t psa\\_aead\\_decrypt](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, [const uint8\\_t](#) \*nonce, [size\\_t](#) nonce\_length, [const uint8\\_t](#) \*additional\_data, [size\\_t](#) additional\_data\_length, [const uint8\\_t](#) \*ciphertext, [size\\_t](#) ciphertext\_length, [uint8\\_t](#) \*plaintext, [size\\_t](#) plaintext\_size, [size\\_t](#) \*plaintext\_length)
  - [psa\\_status\\_t psa\\_aead\\_encrypt\\_setup](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
  - [psa\\_status\\_t psa\\_aead\\_decrypt\\_setup](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg)
  - [psa\\_status\\_t psa\\_aead\\_generate\\_nonce](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*nonce, [size\\_t](#) nonce\_size, [size\\_t](#) \*nonce\_length)
  - [psa\\_status\\_t psa\\_aead\\_set\\_nonce](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [const uint8\\_t](#) \*nonce, [size\\_t](#) nonce\_length)
  - [psa\\_status\\_t psa\\_aead\\_set\\_lengths](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [size\\_t](#) ad\_length, [size\\_t](#) plaintext\_length)
  - [psa\\_status\\_t psa\\_aead\\_update\\_ad](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [const uint8\\_t](#) \*input, [size\\_t](#) input\_length)
  - [psa\\_status\\_t psa\\_aead\\_update](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [const uint8\\_t](#) \*input, [size\\_t](#) input\_length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
  - [psa\\_status\\_t psa\\_aead\\_finish](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*ciphertext, [size\\_t](#) ciphertext\_size, [size\\_t](#) \*ciphertext\_length, [uint8\\_t](#) \*tag, [size\\_t](#) tag\_size, [size\\_t](#) \*tag\_length)
  - [psa\\_status\\_t psa\\_aead\\_verify](#) ([psa\\_aead\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*plaintext, [size\\_t](#) plaintext\_size, [size\\_t](#) \*plaintext\_length, [const uint8\\_t](#) \*tag, [size\\_t](#) tag\_length)
  - [psa\\_status\\_t psa\\_aead\\_abort](#) ([psa\\_aead\\_operation\\_t](#) \*operation)
  - [psa\\_status\\_t psa\\_sign\\_message](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, [const uint8\\_t](#) \*input, [size\\_t](#) input\_length, [uint8\\_t](#) \*signature, [size\\_t](#) signature\_size, [size\\_t](#) \*signature\_length)
- Sign a message with a private key. For hash-and-sign algorithms, this includes the hashing step.*
- [psa\\_status\\_t psa\\_verify\\_message](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, [const uint8\\_t](#) \*input, [size\\_t](#) input\_length, [const uint8\\_t](#) \*signature, [size\\_t](#) signature\_length)
- Verify the signature of a message with a public key, using a hash-and-sign verification algorithm.*

- [psa\\_status\\_t psa\\_sign\\_hash](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*hash, [size\\_t](#) hash\_↵length, [uint8\\_t](#) \*signature, [size\\_t](#) signature\_size, [size\\_t](#) \*signature\_length)
 

*Sign a hash or short message with a private key.*
- [psa\\_status\\_t psa\\_verify\\_hash](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*hash, [size\\_t](#) hash\_↵length, const [uint8\\_t](#) \*signature, [size\\_t](#) signature\_length)
 

*Verify the signature of a hash or short message using a public key.*
- [psa\\_status\\_t psa\\_asymmetric\\_encrypt](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_length, const [uint8\\_t](#) \*salt, [size\\_t](#) salt\_length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_↵length)
 

*Encrypt a short message with a public key.*
- [psa\\_status\\_t psa\\_asymmetric\\_decrypt](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_length, const [uint8\\_t](#) \*salt, [size\\_t](#) salt\_length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_↵length)
 

*Decrypt a short message with a private key.*
- [psa\\_status\\_t psa\\_key\\_derivation\\_get\\_capacity](#) (const [psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [size\\_t](#) ↵\*capacity)
- [psa\\_status\\_t psa\\_key\\_derivation\\_output\\_bytes](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [uint8\\_t](#) \*output, [size\\_t](#) output\_length)
- [psa\\_status\\_t psa\\_key\\_derivation\\_input\\_key](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_derivation\\_step\\_t](#) step, [psa\\_key\\_id\\_t](#) key)
- [psa\\_status\\_t psa\\_key\\_derivation\\_abort](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation)
- [psa\\_status\\_t psa\\_key\\_derivation\\_key\\_agreement](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_derivation\\_step\\_t](#) step, [psa\\_key\\_id\\_t](#) private\_key, const [uint8\\_t](#) \*peer\_key, [size\\_t](#) peer\_key\_length)
- [psa\\_status\\_t psa\\_generate\\_random](#) ([uint8\\_t](#) \*output, [size\\_t](#) output\_size)
 

*Generate random bytes.*
- [psa\\_status\\_t psa\\_generate\\_key](#) (const [psa\\_key\\_attributes\\_t](#) \*attributes, [psa\\_key\\_id\\_t](#) \*key)
 

*Generate a key or key pair.*
- [psa\\_status\\_t psa\\_mac\\_compute](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_↵length, [uint8\\_t](#) \*mac, [size\\_t](#) mac\_size, [size\\_t](#) \*mac\_length)
- [psa\\_status\\_t psa\\_mac\\_verify](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_↵length, const [uint8\\_t](#) \*mac, const [size\\_t](#) mac\_length)
- [psa\\_status\\_t psa\\_cipher\\_encrypt](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_↵length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
- [psa\\_status\\_t psa\\_cipher\\_decrypt](#) ([psa\\_key\\_id\\_t](#) key, [psa\\_algorithm\\_t](#) alg, const [uint8\\_t](#) \*input, [size\\_t](#) input\_↵length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
- [psa\\_status\\_t psa\\_raw\\_key\\_agreement](#) ([psa\\_algorithm\\_t](#) alg, [psa\\_key\\_id\\_t](#) private\_key, const [uint8\\_t](#) \*peer\_↵key, [size\\_t](#) peer\_key\_length, [uint8\\_t](#) \*output, [size\\_t](#) output\_size, [size\\_t](#) \*output\_length)
- [psa\\_status\\_t psa\\_key\\_derivation\\_setup](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_algorithm\\_t](#) alg)
- [psa\\_status\\_t psa\\_key\\_derivation\\_set\\_capacity](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [size\\_t](#) capacity)
- [psa\\_status\\_t psa\\_key\\_derivation\\_input\\_bytes](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_derivation\\_step\\_t](#) step, const [uint8\\_t](#) \*data, [size\\_t](#) data\_length)
- [psa\\_status\\_t psa\\_key\\_derivation\\_output\\_key](#) (const [psa\\_key\\_attributes\\_t](#) \*attributes, [psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_id\\_t](#) \*key)
- [psa\\_status\\_t psa\\_key\\_derivation\\_input\\_integer](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_derivation\\_step\\_t](#) step, [uint64\\_t](#) value)
- [psa\\_status\\_t psa\\_key\\_derivation\\_verify\\_bytes](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, const [uint8\\_t](#) ↵\*expected, [size\\_t](#) expected\_length)
- [psa\\_status\\_t psa\\_key\\_derivation\\_verify\\_key](#) ([psa\\_key\\_derivation\\_operation\\_t](#) \*operation, [psa\\_key\\_id\\_t](#) expected)
- [void psa\\_reset\\_key\\_attributes](#) ([psa\\_key\\_attributes\\_t](#) \*attributes)

### 8.83.1 Macro Definition Documentation

### 8.83.1.1 API\_DISPATCH

```
#define API_DISPATCH(
 in_vec,
 out_vec)
```

#### Value:

```
psa_call(TFM_CRYPTO_HANDLE, PSA_IPC_CALL, \
 in_vec, IOVEC_LEN(in_vec), \
 out_vec, IOVEC_LEN(out_vec))
```

Definition at line 17 of file tfm\_crypto\_api.c.

### 8.83.1.2 API\_DISPATCH\_NO\_OUTVEC

```
#define API_DISPATCH_NO_OUTVEC(
 in_vec)
```

#### Value:

```
psa_call(TFM_CRYPTO_HANDLE, PSA_IPC_CALL, \
 in_vec, IOVEC_LEN(in_vec), \
 (psa_outvec *)NULL, 0)
```

Definition at line 21 of file tfm\_crypto\_api.c.

### 8.83.1.3 TFM\_CRYPTO\_API

```
#define TFM_CRYPTO_API(
 ret,
 fun) ret fun
```

Define the function signature of a TF-M Crypto API with return type *ret* and PSA Crypto API function name *fun*.

#### Parameters

|            |                                                |
|------------|------------------------------------------------|
| <i>ret</i> | return type associated to the API              |
| <i>fun</i> | API name (e.g. a PSA Crypto API function name) |

#### Returns

Function signature

Definition at line 56 of file tfm\_crypto\_api.c.

## 8.83.2 Function Documentation

### 8.83.2.1 psa\_can\_do\_cipher()

```
int psa_can_do_cipher (
 psa_key_type_t key_type,
 psa_algorithm_t cipher_alg)
```

Definition at line 87 of file tfm\_crypto\_api.c.

### 8.83.2.2 psa\_can\_do\_hash()

```
int psa_can_do_hash (
 psa_algorithm_t hash_alg)
```

Definition at line 67 of file tfm\_crypto\_api.c.

8.83.2.3 `psa_cipher_generate_iv()`

```
psa_status_t psa_cipher_generate_iv (
 psa_cipher_operation_t * operation,
 unsigned char * iv,
 size_t iv_size,
 size_t * iv_length)
```

Definition at line 241 of file `tfm_crypto_api.c`.

8.83.2.4 `psa_cipher_set_iv()`

```
psa_status_t psa_cipher_set_iv (
 psa_cipher_operation_t * operation,
 const unsigned char * iv,
 size_t iv_length)
```

Definition at line 266 of file `tfm_crypto_api.c`.

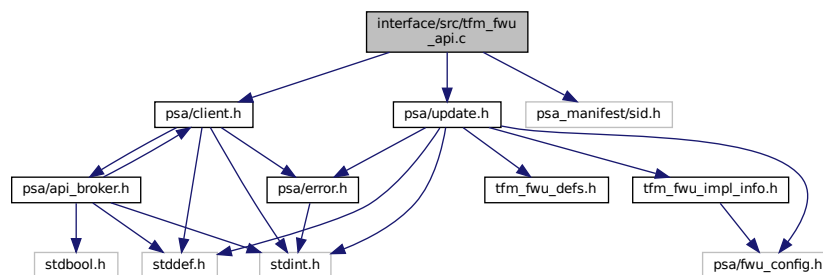
8.83.2.5 `psa_cipher_update()`

```
psa_status_t psa_cipher_update (
 psa_cipher_operation_t * operation,
 const uint8_t * input,
 size_t input_length,
 unsigned char * output,
 size_t output_size,
 size_t * output_length)
```

Definition at line 325 of file `tfm_crypto_api.c`.

## 8.84 interface/src/tfm\_fwu\_api.c File Reference

```
#include "psa/client.h"
#include "psa/update.h"
#include "psa_manifest/sid.h"
Include dependency graph for tfm_fwu_api.c:
```



## Functions

- `psa_status_t psa_fwu_start (psa_fwu_component_t component, const void *manifest, size_t manifest_size)`  
Begin a firmware update operation for a specific firmware component.
- `psa_status_t psa_fwu_write (psa_fwu_component_t component, size_t image_offset, const void *block, size_t block_size)`  
Write a firmware image, or part of a firmware image, to its staging area.

- [psa\\_status\\_t psa\\_fwu\\_finish](#) ([psa\\_fwu\\_component\\_t](#) component)  
*Mark a firmware image in the staging area as ready for installation.*
- [psa\\_status\\_t psa\\_fwu\\_install](#) ([void](#))  
*Start the installation of all firmware images that have been prepared for update.*
- [psa\\_status\\_t psa\\_fwu\\_cancel](#) ([psa\\_fwu\\_component\\_t](#) component)  
*Abandon an update that is in WRITING or CANDIDATE state.*
- [psa\\_status\\_t psa\\_fwu\\_clean](#) ([psa\\_fwu\\_component\\_t](#) component)  
*Prepare the component for another update.*
- [psa\\_status\\_t psa\\_fwu\\_query](#) ([psa\\_fwu\\_component\\_t](#) component, [psa\\_fwu\\_component\\_info\\_t](#) \*info)  
*Retrieve the firmware store information for a specific firmware component.*
- [psa\\_status\\_t psa\\_fwu\\_request\\_reboot](#) ([void](#))  
*Requests the platform to reboot.*
- [psa\\_status\\_t psa\\_fwu\\_accept](#) ([void](#))  
*Accept a firmware update that is currently in TRIAL state.*
- [psa\\_status\\_t psa\\_fwu\\_reject](#) ([psa\\_status\\_t](#) error)  
*Abandon an installation that is in STAGED or TRIAL state.*

## 8.84.1 Function Documentation

### 8.84.1.1 [psa\\_fwu\\_accept\(\)](#)

```
psa_status_t psa_fwu_accept (
 void)
```

Accept a firmware update that is currently in TRIAL state.

#### Returns

Result status.

Definition at line 96 of file `tfm_fwu_api.c`.

Here is the call graph for this function:



### 8.84.1.2 [psa\\_fwu\\_cancel\(\)](#)

```
psa_status_t psa_fwu_cancel (
 psa_fwu_component_t component)
```

Abandon an update that is in WRITING or CANDIDATE state.

#### Parameters

|                  |                                                       |
|------------------|-------------------------------------------------------|
| <i>component</i> | Identifier of the firmware component to be cancelled. |
|------------------|-------------------------------------------------------|



**Returns**

Result status.

Definition at line 56 of file tfm\_fwu\_api.c.

Here is the call graph for this function:

**8.84.1.3 psa\_fwu\_clean()**

```
psa_status_t psa_fwu_clean (
 psa_fwu_component_t component)
```

Prepare the component for another update.

**Parameters**

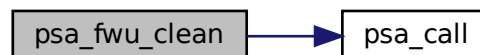
|                  |                                                  |
|------------------|--------------------------------------------------|
| <i>component</i> | Identifier of the firmware component to tidy up. |
|------------------|--------------------------------------------------|

**Returns**

Result status.

Definition at line 66 of file tfm\_fwu\_api.c.

Here is the call graph for this function:

**8.84.1.4 psa\_fwu\_finish()**

```
psa_status_t psa_fwu_finish (
 psa_fwu_component_t component)
```

Mark a firmware image in the staging area as ready for installation.

**Parameters**

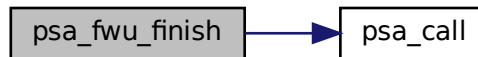
|                  |                                                  |
|------------------|--------------------------------------------------|
| <i>component</i> | Identifier of the firmware component to install. |
|------------------|--------------------------------------------------|

**Returns**

Result status.

Definition at line 40 of file `tfm_fwu_api.c`.

Here is the call graph for this function:

**8.84.1.5 psa\_fwu\_install()**

```
psa_status_t psa_fwu_install (
 void)
```

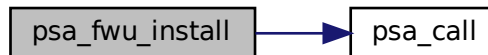
Start the installation of all firmware images that have been prepared for update.

**Returns**

Result status.

Definition at line 50 of file `tfm_fwu_api.c`.

Here is the call graph for this function:

**8.84.1.6 psa\_fwu\_query()**

```
psa_status_t psa_fwu_query (
 psa_fwu_component_t component,
 psa_fwu_component_info_t * info)
```

Retrieve the firmware store information for a specific firmware component.

**Parameters**

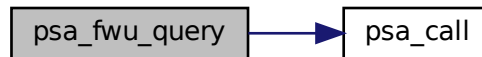
|                  |                                                        |
|------------------|--------------------------------------------------------|
| <i>component</i> | Firmware component for which information is requested. |
| <i>info</i>      | Output parameter for component information.            |

**Returns**

Result status.

Definition at line 76 of file tfm\_fwu\_api.c.

Here is the call graph for this function:

**8.84.1.7 psa\_fwu\_reject()**

```
psa_status_t psa_fwu_reject (
 psa_status_t error)
```

Abandon an installation that is in STAGED or TRIAL state.

**Parameters**

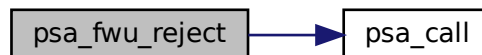
|              |                                                               |
|--------------|---------------------------------------------------------------|
| <i>error</i> | An application-specific error code chosen by the application. |
|--------------|---------------------------------------------------------------|

**Returns**

Result status.

Definition at line 102 of file tfm\_fwu\_api.c.

Here is the call graph for this function:

**8.84.1.8 psa\_fwu\_request\_reboot()**

```
psa_status_t psa_fwu_request_reboot (
 void)
```

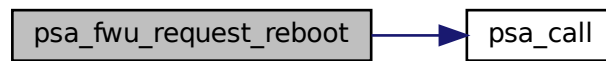
Requests the platform to reboot.

**Returns**

Result status.

Definition at line 90 of file tfm\_fwu\_api.c.

Here is the call graph for this function:



#### 8.84.1.9 psa\_fwu\_start()

```

psa_status_t psa_fwu_start (
 psa_fwu_component_t component,
 const void * manifest,
 size_t manifest_size)

```

Begin a firmware update operation for a specific firmware component.

##### Parameters

|                      |                                                                      |
|----------------------|----------------------------------------------------------------------|
| <i>component</i>     | Identifier of the firmware component to be updated.                  |
| <i>manifest</i>      | A pointer to a buffer containing a detached manifest for the update. |
| <i>manifest_size</i> | The size of the detached manifest.                                   |

##### Returns

Result status.

Definition at line 12 of file tfm\_fwu\_api.c.

Here is the call graph for this function:



#### 8.84.1.10 psa\_fwu\_write()

```

psa_status_t psa_fwu_write (
 psa_fwu_component_t component,
 size_t image_offset,
 const void * block,
 size_t block_size)

```

Write a firmware image, or part of a firmware image, to its staging area.

## Parameters

|                     |                                                     |
|---------------------|-----------------------------------------------------|
| <i>component</i>    | Identifier of the firmware component being updated. |
| <i>image_offset</i> | The offset of the data block in the whole image.    |
| <i>block</i>        | A buffer containing a block of image data.          |
| <i>block_size</i>   | Size of block, in bytes.                            |

## Returns

Result status.

Definition at line 25 of file `tfm_fwu_api.c`.

Here is the call graph for this function:



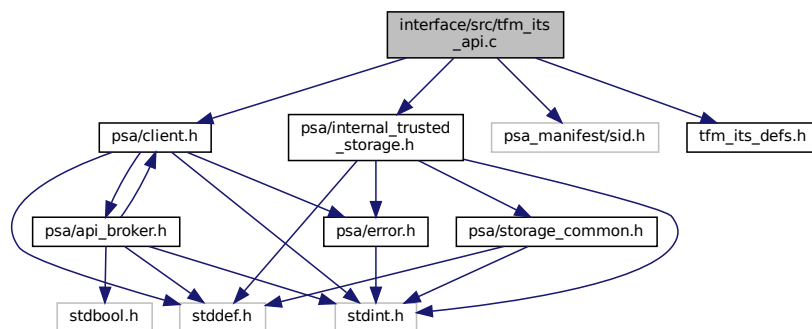
## 8.85 interface/src/tfm\_its\_api.c File Reference

```

#include "psa/client.h"
#include "psa/internal_trusted_storage.h"
#include "psa_manifest/sid.h"
#include "tfm_its_defs.h"

```

Include dependency graph for `tfm_its_api.c`:



## Data Structures

- struct [rot\\_psa\\_its\\_storage\\_info\\_t](#)

## Functions

- [psa\\_status\\_t psa\\_its\\_set](#) ([psa\\_storage\\_uid\\_t](#) uid, [size\\_t](#) data\_length, const [void](#) \*p\_data, [psa\\_storage\\_create\\_flags\\_t](#) create\_flags)

Create a new, or modify an existing, uid/value pair.

- `psa_status_t psa_its_get (psa_storage_uid_t uid, size_t data_offset, size_t data_size, void *p_data, size_t *p_data_length)`

Retrieve data associated with a provided UID.

- `psa_status_t psa_its_get_info (psa_storage_uid_t uid, struct psa_storage_info_t *p_info)`

Retrieve the metadata about the provided uid.

- `psa_status_t psa_its_remove (psa_storage_uid_t uid)`

Remove the provided uid and its associated data from the storage.

## 8.85.1 Function Documentation

### 8.85.1.1 psa\_its\_get()

```
psa_status_t psa_its_get (
 psa_storage_uid_t uid,
 size_t data_offset,
 size_t data_size,
 void * p_data,
 size_t * p_data_length)
```

Retrieve data associated with a provided UID.

Retrieves up to `data_size` bytes of the data associated with `uid`, starting at `data_offset` bytes from the beginning of the data. Upon successful completion, the data will be placed in the `p_data` buffer, which must be at least `data_size` bytes in size. The length of the data returned will be in `p_data_length`. If `data_size` is 0, the contents of `p_data_length` will be set to zero.

#### Parameters

|     |                            |                                                                              |
|-----|----------------------------|------------------------------------------------------------------------------|
| in  | <code>uid</code>           | The uid value                                                                |
| in  | <code>data_offset</code>   | The starting offset of the data requested                                    |
| in  | <code>data_size</code>     | The amount of data requested                                                 |
| out | <code>p_data</code>        | On success, the buffer where the data will be placed                         |
| out | <code>p_data_length</code> | On success, this will contain size of the data placed in <code>p_data</code> |

#### Returns

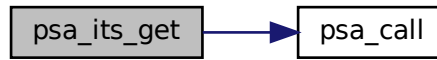
A status indicating the success/failure of the operation

#### Return values

|                                         |                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>PSA_SUCCESS</code>                | The operation completed successfully                                                                                                                                                                                                                                                                                                           |
| <code>PSA_ERROR_DOES_NOT_EXIST</code>   | The operation failed because the provided <code>uid</code> value was not found in the storage                                                                                                                                                                                                                                                  |
| <code>PSA_ERROR_STORAGE_FAILURE</code>  | The operation failed because the physical storage has failed (Fatal error)                                                                                                                                                                                                                                                                     |
| <code>PSA_ERROR_INVALID_ARGUMENT</code> | The operation failed because one of the provided arguments ( <code>p_data</code> , <code>p_data_length</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access. In addition, this can also happen if <code>data_offset</code> is larger than the size of the data associated with <code>uid</code> |

Definition at line 38 of file `tfm_its_api.c`.

Here is the call graph for this function:



### 8.85.1.2 psa\_its\_get\_info()

```

psa_status_t psa_its_get_info (
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Retrieve the metadata about the provided uid.

Retrieves the metadata stored for a given uid as a `psa_storage_info_t` structure.

#### Parameters

|     |               |                                                                                                  |
|-----|---------------|--------------------------------------------------------------------------------------------------|
| in  | <i>uid</i>    | The uid value                                                                                    |
| out | <i>p_info</i> | A pointer to the <code>psa_storage_info_t</code> struct that will be populated with the metadata |

#### Returns

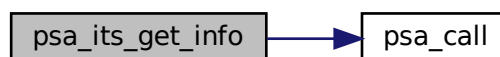
A status indicating the success/failure of the operation

#### Return values

|                                   |                                                                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                  |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                                                                      |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                                                                            |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided pointers( <i>p_info</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

Definition at line 75 of file `tfm_its_api.c`.

Here is the call graph for this function:



### 8.85.1.3 `psa_its_remove()`

```
psa_status_t psa_its_remove (
 psa_storage_uid_t uid)
```

Remove the provided uid and its associated data from the storage.  
Deletes the data from internal storage.

#### Parameters

|    |            |               |
|----|------------|---------------|
| in | <i>uid</i> | The uid value |
|----|------------|---------------|

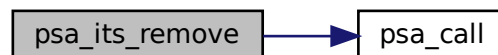
#### Returns

A status indicating the success/failure of the operation

#### Return values

|                                   |                                                                                                                    |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                               |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                   |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was created with <i>PSA_STORAGE_FLAG_WRITE_ONCE</i>            |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                         |

Definition at line 104 of file `tfm_its_api.c`.  
Here is the call graph for this function:



### 8.85.1.4 `psa_its_set()`

```
psa_status_t psa_its_set (
 psa_storage_uid_t uid,
 size_t data_length,
 const void * p_data,
 psa_storage_create_flags_t create_flags)
```

Create a new, or modify an existing, uid/value pair.  
Stores data in the internal storage.

#### Parameters

|    |                    |                                                |
|----|--------------------|------------------------------------------------|
| in | <i>uid</i>         | The identifier for the data                    |
| in | <i>data_length</i> | The size in bytes of the data in <i>p_data</i> |



## Parameters

|    |                     |                                             |
|----|---------------------|---------------------------------------------|
| in | <i>p_data</i>       | A buffer containing the data                |
| in | <i>create_flags</i> | The flags that the data will be stored with |

## Returns

A status indicating the success/failure of the operation

## Return values

|                                       |                                                                                                                                                                             |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                                                                                        |
| <i>PSA_ERROR_NOT_PERMITTED</i>        | The operation failed because the provided <code>uid</code> value was already created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code>                                          |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because one or more of the flags provided in <code>create_flags</code> is not supported or is not valid                                                |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because there was insufficient space on the storage medium                                                                                             |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (Fatal error)                                                                                                  |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because one of the provided pointers( <code>p_data</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

Definition at line 19 of file `tfm_its_api.c`.

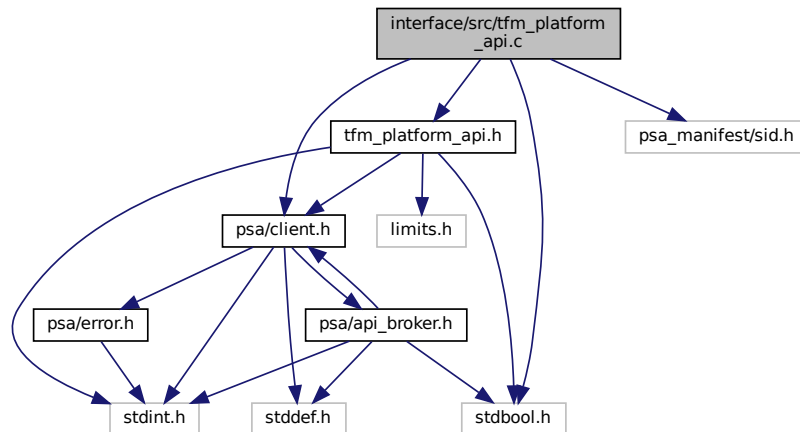
Here is the call graph for this function:



## 8.86 interface/src/tfm\_platform\_api.c File Reference

```
#include <stdbool.h>
#include "tfm_platform_api.h"
#include "psa/client.h"
#include "psa_manifest/sid.h"
```

Include dependency graph for `tfm_platform_api.c`:



## Functions

- enum `tfm_platform_err_t` `tfm_platform_system_reset` (void)  
*Resets the system.*
- enum `tfm_platform_err_t` `tfm_platform_ioctl` (`tfm_platform_ioctl_req_t` request, `psa_invec` \*input, `psa_outvec` \*output)  
*Performs a platform-specific service.*
- enum `tfm_platform_err_t` `tfm_platform_nv_counter_increment` (uint32\_t counter\_id)  
*Increments the given non-volatile (NV) counter by one.*
- enum `tfm_platform_err_t` `tfm_platform_nv_counter_read` (uint32\_t counter\_id, uint32\_t size, uint8\_t \*val)  
*Reads the given non-volatile (NV) counter.*

## 8.86.1 Function Documentation

### 8.86.1.1 `tfm_platform_ioctl()`

```
enum tfm_platform_err_t tfm_platform_ioctl (
 tfm_platform_ioctl_req_t request,
 psa_invec * input,
 psa_outvec * output)
```

Performs a platform-specific service.

#### Parameters

|         |                |                                                              |
|---------|----------------|--------------------------------------------------------------|
| in      | <i>request</i> | Request identifier (valid values vary based on the platform) |
| in      | <i>input</i>   | Input buffer to the requested service (or NULL)              |
| in, out | <i>output</i>  | Output buffer to the requested service (or NULL)             |

#### Returns

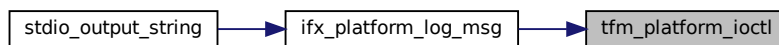
Returns values as specified by the `tfm_platform_err_t`

Definition at line 30 of file `tfm_platform_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.86.1.2 tfm\_platform\_nv\_counter\_increment()

```
enum tfm_platform_err_t tfm_platform_nv_counter_increment (
 uint32_t counter_id)
```

Increments the given non-volatile (NV) counter by one.

##### Parameters

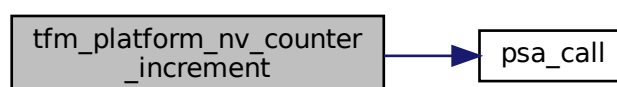
|    |                   |                |
|----|-------------------|----------------|
| in | <i>counter_id</i> | NV counter ID. |
|----|-------------------|----------------|

##### Returns

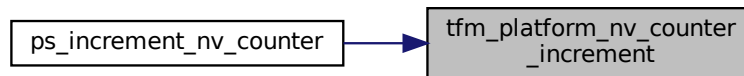
TFM\_PLATFORM\_ERR\_SUCCESS if the value is read correctly. Otherwise, it returns TFM\_PLATFORM\_↔  
ERR\_SYSTEM\_ERROR.

Definition at line 67 of file tfm\_platform\_api.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.86.1.3 tfm\_platform\_nv\_counter\_read()

```

enum tfm_platform_err_t tfm_platform_nv_counter_read (
 uint32_t counter_id,
 uint32_t size,
 uint8_t * val)

```

Reads the given non-volatile (NV) counter.

#### Parameters

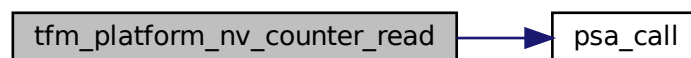
|     |                   |                                                        |
|-----|-------------------|--------------------------------------------------------|
| in  | <i>counter_id</i> | NV counter ID.                                         |
| in  | <i>size</i>       | Size of the buffer to store NV counter value in bytes. |
| out | <i>val</i>        | Pointer to store the current NV counter value.         |

#### Returns

TFM\_PLATFORM\_ERR\_SUCCESS if the value is read correctly. Otherwise, it returns TFM\_PLATFORM\_ERR\_SYSTEM\_ERROR.

Definition at line 87 of file `tfm_platform_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.86.1.4 tfm\_platform\_system\_reset()

```
enum tfm_platform_err_t tfm_platform_system_reset (
 void)
```

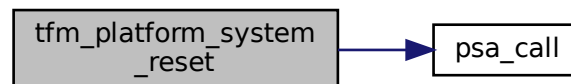
Resets the system.

#### Returns

Returns values as specified by the [tfm\\_platform\\_err\\_t](#)

Definition at line 13 of file `tfm_platform_api.c`.

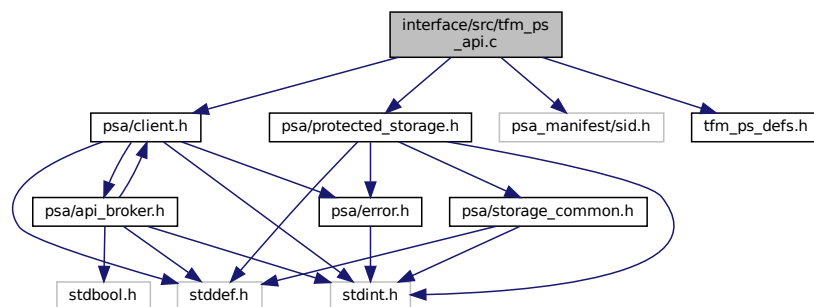
Here is the call graph for this function:



## 8.87 interface/src/tfm\_ps\_api.c File Reference

```
#include "psa/client.h"
#include "psa/protected_storage.h"
#include "psa_manifest/sid.h"
#include "tfm_ps_defs.h"
```

Include dependency graph for `tfm_ps_api.c`:



## Data Structures

- struct [rot\\_psa\\_ps\\_storage\\_info\\_t](#)

## Functions

- [psa\\_status\\_t](#) [psa\\_ps\\_set](#) ([psa\\_storage\\_uid\\_t](#) uid, [size\\_t](#) data\_length, const [void](#) \*p\_data, [psa\\_storage\\_create\\_flags\\_t](#) create\_flags)  
Create a new, or modify an existing, uid/value pair.
- [psa\\_status\\_t](#) [psa\\_ps\\_get](#) ([psa\\_storage\\_uid\\_t](#) uid, [size\\_t](#) data\_offset, [size\\_t](#) data\_size, [void](#) \*p\_data, [size\\_t](#) \*p\_data\_length)

*Retrieve data associated with a provided uid.*

- `psa_status_t psa_ps_get_info (psa_storage_uid_t uid, struct psa_storage_info_t *p_info)`

*Retrieve the metadata about the provided uid.*

- `psa_status_t psa_ps_remove (psa_storage_uid_t uid)`

*Remove the provided uid and its associated data from the storage.*

- `psa_status_t psa_ps_create (psa_storage_uid_t uid, size_t size, psa_storage_create_flags_t create_flags)`

*Reserves storage for the specified uid.*

- `psa_status_t psa_ps_set_extended (psa_storage_uid_t uid, size_t data_offset, size_t data_length, const void *p_data)`

*Sets partial data into an asset.*

- `uint32_t psa_ps_get_support (void)`

*Lists optional features.*

## 8.87.1 Function Documentation

### 8.87.1.1 `psa_ps_create()`

```
psa_status_t psa_ps_create (
 psa_storage_uid_t uid,
 size_t capacity,
 psa_storage_create_flags_t create_flags)
```

Reserves storage for the specified uid.

Upon success, the capacity of the storage will be capacity, and the size will be 0. It is only necessary to call this function for assets that will be written with the `psa_ps_set_extended` function. If only the `psa_ps_set` function is needed, calls to this function are redundant.

#### Parameters

|    |                     |                                        |
|----|---------------------|----------------------------------------|
| in | <i>uid</i>          | The uid value                          |
| in | <i>capacity</i>     | The capacity to be allocated in bytes  |
| in | <i>create_flags</i> | Flags indicating properties of storage |

#### Returns

A status indicating the success/failure of the operation

#### Return values

|                                       |                                                                                                             |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                        |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (Fatal error)                                  |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because the capacity is bigger than the current available space                        |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because the function is not implemented or one or more create_flags are not supported. |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because uid was 0 or create_flags specified flags that are not defined in the API.     |
| <i>PSA_ERROR_GENERIC_ERROR</i>        | The operation failed due to an unspecified error                                                            |
| <i>PSA_ERROR_ALREADY_EXISTS</i>       | Storage for the specified uid already exists                                                                |

Definition at line 117 of file `tfm_ps_api.c`.

### 8.87.1.2 psa\_ps\_get()

```
psa_status_t psa_ps_get (
 psa_storage_uid_t uid,
 size_t data_offset,
 size_t data_size,
 void * p_data,
 size_t * p_data_length)
```

Retrieve data associated with a provided uid.

Retrieves up to `data_size` bytes of the data associated with `uid`, starting at `data_offset` bytes from the beginning of the data. Upon successful completion, the data will be placed in the `p_data` buffer, which must be at least `data_size` bytes in size. The length of the data returned will be in `p_data_length`. If `data_size` is 0, the contents of `p_data_length` will be set to zero.

#### Parameters

|     |                      |                                                                              |
|-----|----------------------|------------------------------------------------------------------------------|
| in  | <i>uid</i>           | The uid value                                                                |
| in  | <i>data_offset</i>   | The starting offset of the data requested                                    |
| in  | <i>data_size</i>     | The amount of data requested                                                 |
| out | <i>p_data</i>        | On success, the buffer where the data will be placed                         |
| out | <i>p_data_length</i> | On success, this will contain size of the data placed in <code>p_data</code> |

#### Returns

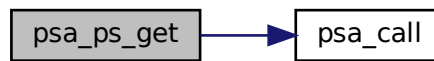
A status indicating the success/failure of the operation

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                 | The operation completed successfully                                                                                                                                                                                                                                                                                                           |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>  | The operation failed because one of the provided arguments ( <code>p_data</code> , <code>p_data_length</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access. In addition, this can also happen if <code>data_offset</code> is larger than the size of the data associated with <code>uid</code> |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>    | The operation failed because the provided <code>uid</code> value was not found in the storage                                                                                                                                                                                                                                                  |
| <i>PSA_ERROR_STORAGE_FAILURE</i>   | The operation failed because the physical storage has failed (Fatal error)                                                                                                                                                                                                                                                                     |
| <i>PSA_ERROR_GENERIC_ERROR</i>     | The operation failed because of an unspecified internal failure                                                                                                                                                                                                                                                                                |
| <i>PSA_ERROR_DATA_CORRUPT</i>      | The operation failed because the data associated with the UID was corrupt                                                                                                                                                                                                                                                                      |
| <i>PSA_ERROR_INVALID_SIGNATURE</i> | The operation failed because the data associated with the UID failed authentication                                                                                                                                                                                                                                                            |

Definition at line 38 of file `tfm_ps_api.c`.

Here is the call graph for this function:



### 8.87.1.3 psa\_ps\_get\_info()

```

psa_status_t psa_ps_get_info (
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Retrieve the metadata about the provided uid.

Retrieves the metadata stored for a given uid

#### Parameters

|     |               |                                                                                                     |
|-----|---------------|-----------------------------------------------------------------------------------------------------|
| in  | <i>uid</i>    | The uid value                                                                                       |
| out | <i>p_info</i> | A pointer to the <a href="#">psa_storage_info_t</a> struct that will be populated with the metadata |

#### Returns

A status indicating the success/failure of the operation

#### Return values

|                                   |                                                                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                  |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided pointers( <i>p_info</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                                                                      |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                                                                            |
| <i>PSA_ERROR_GENERIC_ERROR</i>    | The operation failed because of an unspecified internal failure                                                                                                       |
| <i>PSA_ERROR_DATA_CORRUPT</i>     | The operation failed because the data associated with the UID was corrupt                                                                                             |

Definition at line 75 of file `tfm_ps_api.c`.



Here is the call graph for this function:



#### 8.87.1.4 psa\_ps\_get\_support()

```
uint32_t psa_ps_get_support (
 void)
```

Lists optional features.

##### Returns

A bitmask with flags set for all of the optional features supported by the implementation. Currently defined flags are limited to `PSA_STORAGE_SUPPORT_SET_EXTENDED`

Definition at line 138 of file `tfm_ps_api.c`.

Here is the call graph for this function:



#### 8.87.1.5 psa\_ps\_remove()

```
psa_status_t psa_ps_remove (
 psa_storage_uid_t uid)
```

Remove the provided uid and its associated data from the storage.

Removes previously stored data and any associated metadata, including rollback protection data.

##### Parameters

|    |            |               |
|----|------------|---------------|
| in | <i>uid</i> | The uid value |
|----|------------|---------------|

##### Returns

A status indicating the success/failure of the operation

##### Return values

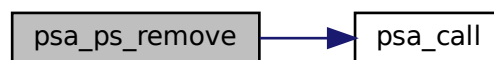
|                    |                                      |
|--------------------|--------------------------------------|
| <i>PSA_SUCCESS</i> | The operation completed successfully |
|--------------------|--------------------------------------|

## Return values

|                                   |                                                                                                                    |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                   |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was created with <i>PSA_STORAGE_FLAG_WRITE_ONCE</i>            |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                         |
| <i>PSA_ERROR_GENERIC_ERROR</i>    | The operation failed because of an unspecified internal failure                                                    |

Definition at line 103 of file `tfm_ps_api.c`.

Here is the call graph for this function:

8.87.1.6 `psa_ps_set()`

```

psa_status_t psa_ps_set (
 psa_storage_uid_t uid,
 size_t data_length,
 const void * p_data,
 psa_storage_create_flags_t create_flags)

```

Create a new, or modify an existing, uid/value pair.

Stores data in the protected storage.

## Parameters

|    |                     |                                                |
|----|---------------------|------------------------------------------------|
| in | <i>uid</i>          | The identifier for the data                    |
| in | <i>data_length</i>  | The size in bytes of the data in <i>p_data</i> |
| in | <i>p_data</i>       | A buffer containing the data                   |
| in | <i>create_flags</i> | The flags that the data will be stored with    |

## Returns

A status indicating the success/failure of the operation

## Return values

|                                   |                                                                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                  |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was already created with <i>PSA_STORAGE_FLAG_WRITE_ONCE</i>                                                       |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided pointers( <i>p_data</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

## Return values

|                                       |                                                                                                                              |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because one or more of the flags provided in <code>create_flags</code> is not supported or is not valid |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because there was insufficient space on the storage medium                                              |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (Fatal error)                                                   |
| <i>PSA_ERROR_GENERIC_ERROR</i>        | The operation failed because of an unspecified internal failure                                                              |

Definition at line 19 of file `tfm_ps_api.c`.

Here is the call graph for this function:

8.87.1.7 `psa_ps_set_extended()`

```

psa_status_t psa_ps_set_extended (
 psa_storage_uid_t uid,
 size_t data_offset,
 size_t data_length,
 const void * p_data)

```

Sets partial data into an asset.

Before calling this function, the storage must have been reserved with a call to `psa_ps_create`. It can also be used to overwrite data in an asset that was created with a call to `psa_ps_set`. Calling this function with `data_length = 0` is permitted, which will make no change to the stored data. This function can overwrite existing data and/or extend it up to the capacity for the `uid` specified in `psa_ps_create`, but cannot create gaps.

That is, it has preconditions:

- `data_offset <= size`
- `data_offset + data_length <= capacity` and postconditions:
- `size = max(size, data_offset + data_length)`
- capacity unchanged.

## Parameters

|    |                    |                                                               |
|----|--------------------|---------------------------------------------------------------|
| in | <i>uid</i>         | The <code>uid</code> value                                    |
| in | <i>data_offset</i> | Offset within the asset to start the write                    |
| in | <i>data_length</i> | The size in bytes of the data in <code>p_data</code> to write |
| in | <i>p_data</i>      | Pointer to a buffer which contains the data to write          |

**Returns**

A status indicating the success/failure of the operation

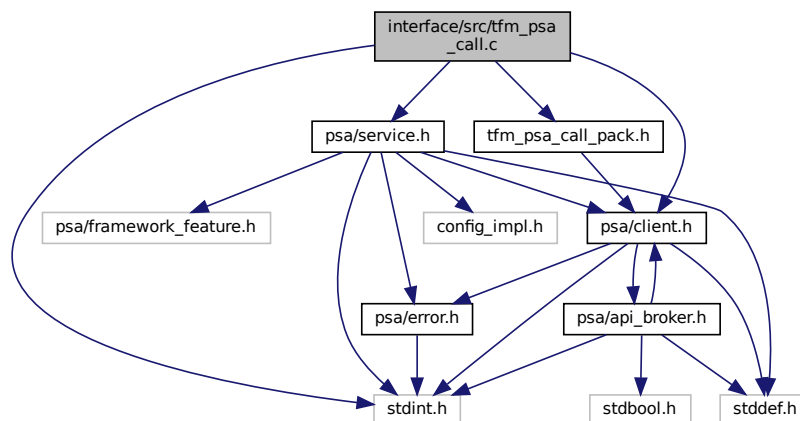
**Return values**

|                                    |                                                                                                                                      |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                 | The asset exists, the input parameters are correct and the data is correctly written in the physical storage.                        |
| <i>PSA_ERROR_STORAGE_FAILURE</i>   | The data was not written correctly in the physical storage                                                                           |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>  | The operation failed because one or more of the preconditions listed above regarding data_offset, size, or data_length was violated. |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>    | The specified uid was not found                                                                                                      |
| <i>PSA_ERROR_NOT_SUPPORTED</i>     | The implementation of the API does not support this function                                                                         |
| <i>PSA_ERROR_GENERIC_ERROR</i>     | The operation failed due to an unspecified error                                                                                     |
| <i>PSA_ERROR_DATA_CORRUPT</i>      | The operation failed because the existing data has been corrupted.                                                                   |
| <i>PSA_ERROR_INVALID_SIGNATURE</i> | The operation failed because the existing data failed authentication (MAC check failed).                                             |
| <i>PSA_ERROR_NOT_PERMITTED</i>     | The operation failed because it was attempted on an asset which was written with the flag PSA_STORAGE_FLAG_WRITE_ONCE                |

Definition at line 127 of file tfm\_psa\_api.c.

## 8.88 interface/src/tfm\_psa\_call.c File Reference

```
#include <stdint.h>
#include "psa/client.h"
#include "psa/service.h"
#include "tfm_psa_call_pack.h"
Include dependency graph for tfm_psa_call.c:
```

**Functions**

- [psa\\_status\\_t psa\\_call](#) ([psa\\_handle\\_t](#) handle, [int32\\_t](#) type, const [psa\\_invec](#) \*in\_vec, [size\\_t](#) in\_len, [psa\\_outvec](#) \*out\_vec, [size\\_t](#) out\_len)

*Call an RoT Service on an established connection.*

## 8.88.1 Function Documentation

### 8.88.1.1 `psa_call()`

```
psa_status_t psa_call (
 psa_handle_t handle,
 int32_t type,
 const psa_invec * in_vec,
 size_t in_len,
 psa_outvec * out_vec,
 size_t out_len)
```

Call an RoT Service on an established connection.

#### Note

FF-M 1.0 proposes 6 parameters for `psa_call` but the secure gateway ABI support at most 4 parameters. T↔F-M chooses to encode 'in\_len', 'out\_len', and 'type' into a 32-bit integer to improve efficiency. Compared with struct-based encoding, this method saves extra memory check and memory copy operation. The disadvantage is that the 'type' range has to be reduced into a 16-bit integer. So with this encoding, the valid range for 'type' is 0-32767.

#### Parameters

|         |                |                                                                             |
|---------|----------------|-----------------------------------------------------------------------------|
| in      | <i>handle</i>  | A handle to an established connection.                                      |
| in      | <i>type</i>    | The request type. Must be zero( <a href="#">PSA_IPC_CALL</a> ) or positive. |
| in      | <i>in_vec</i>  | Array of input <a href="#">psa_invec</a> structures.                        |
| in      | <i>in_len</i>  | Number of input <a href="#">psa_invec</a> structures.                       |
| in, out | <i>out_vec</i> | Array of output <a href="#">psa_outvec</a> structures.                      |
| in      | <i>out_len</i> | Number of output <a href="#">psa_outvec</a> structures.                     |

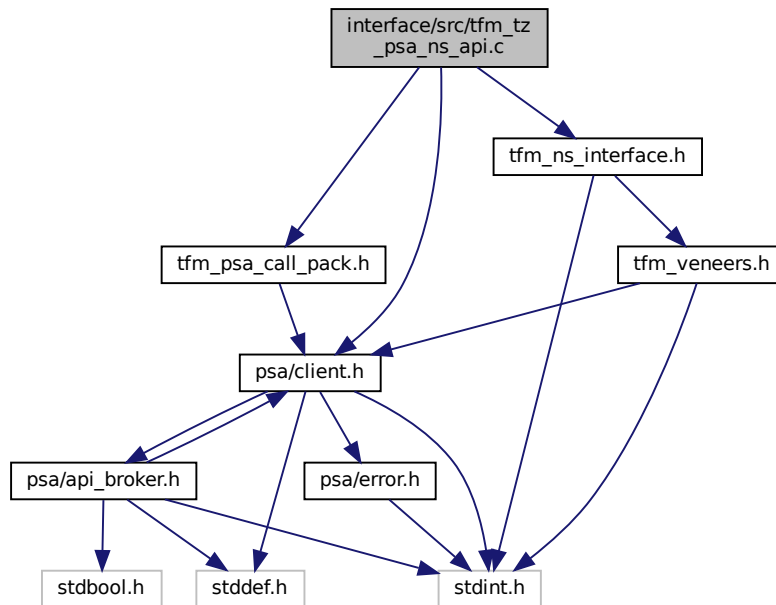
#### Return values

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\geq 0$                          | RoT Service-specific status value.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| $< 0$                             | RoT Service-specific error code.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>PSA_ERROR_PROGRAMMER_ERROR</i> | <p>The connection has been terminated by the RoT Service. The call is a PROGRAMMER ERROR if one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• An invalid handle was passed.</li> <li>• The connection is already handling a request.</li> <li>• <math>\text{type} &lt; 0</math>.</li> <li>• An invalid memory reference was provided.</li> <li>• <math>\text{in\_len} + \text{out\_len} &gt; \text{PSA\_MAX\_IOVEC}</math>.</li> <li>• The message is unrecognized by the RoT Service or incorrectly formatted.</li> </ul> |

Definition at line 13 of file `tfm_psa_call.c`.

## 8.89 interface/src/tfm\_tz\_psa\_ns\_api.c File Reference

```
#include "psa/client.h"
#include "tfm_ns_interface.h"
#include "tfm_psa_call_pack.h"
Include dependency graph for tfm_tz_psa_ns_api.c:
```



## Functions

- enum `tfm_reenter_check_err_t` `tfm_ns_check_safe_entry` (void)  
*Perform a reentrancy check before safely proceeding with a PSA Function Call.*
- enum `tfm_reenter_check_err_t` `tfm_ns_check_exit` (void)  
*Release the reentrancy flag.*
- uint32\_t `PSA_FRAMEWORK_VERSION_TZ` (void)
- uint32\_t `PSA_VERSION_TZ` (uint32\_t sid)
- `psa_status_t` `PSA_CALL_TZ` (`psa_handle_t` handle, int32\_t type, const `psa_invec` \*in\_vec, size\_t in\_len, `psa_outvec` \*out\_vec, size\_t out\_len)
- `psa_handle_t` `PSA_CONNECT_TZ` (uint32\_t sid, uint32\_t version)
- void `PSA_CLOSE_TZ` (`psa_handle_t` handle)

### 8.89.1 Function Documentation

#### 8.89.1.1 PSA\_CALL\_TZ()

```
psa_status_t PSA_CALL_TZ (
 psa_handle_t handle,
 int32_t type,
 const psa_invec * in_vec,
 size_t in_len,
```

```

 psa_outvec * out_vec,
 size_t out_len)

```

Definition at line 73 of file tfm\_tz\_psa\_ns\_api.c.

### 8.89.1.2 PSA\_CLOSE\_TZ()

```

void PSA_CLOSE_TZ (
 psa_handle_t handle)

```

Definition at line 99 of file tfm\_tz\_psa\_ns\_api.c.

### 8.89.1.3 PSA\_CONNECT\_TZ()

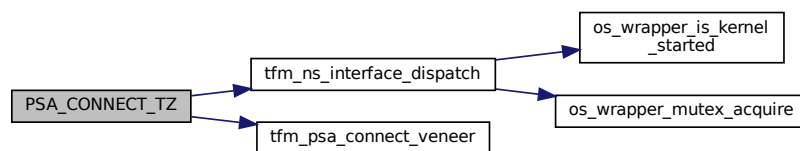
```

psa_handle_t PSA_CONNECT_TZ (
 uint32_t sid,
 uint32_t version)

```

Definition at line 94 of file tfm\_tz\_psa\_ns\_api.c.

Here is the call graph for this function:



### 8.89.1.4 PSA\_FRAMEWORK\_VERSION\_TZ()

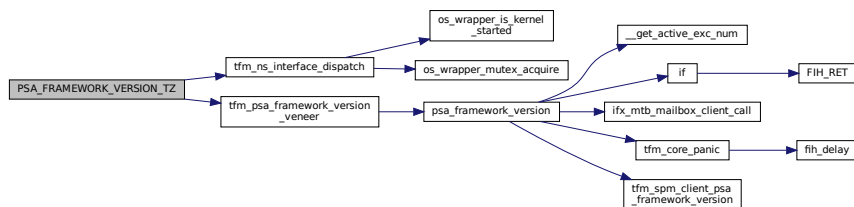
```

uint32_t PSA_FRAMEWORK_VERSION_TZ (
 void)

```

Definition at line 53 of file tfm\_tz\_psa\_ns\_api.c.

Here is the call graph for this function:



### 8.89.1.5 PSA\_VERSION\_TZ()

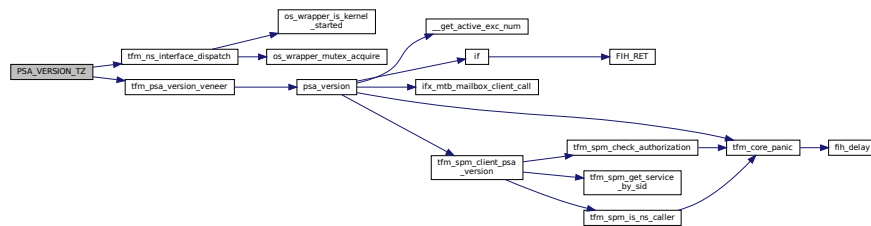
```

uint32_t PSA_VERSION_TZ (
 uint32_t sid)

```

Definition at line 63 of file tfm\_tz\_psa\_ns\_api.c.

Here is the call graph for this function:



### 8.89.1.6 tfm\_ns\_check\_exit()

```
enum tfm_reenter_check_err_t tfm_ns_check_exit (
 void)
```

Release the reentrancy flag.

#### Note

Requires TFM\_TZ\_REENTRANCY\_CHECK option to be enabled

#### Returns

Returns [tfm\\_reenter\\_check\\_err\\_t](#) status code.

[TFM\\_REENTRANCY\\_CHECK\\_SUCCESS](#) The caller successfully cleared the previously set flag

[TFM\\_REENTRANCY\\_CHECK\\_ERR\\_FLAG\\_STATUS](#) Either: There was an attempt to clear twice the check flag Or: TFM\_TZ\_REENTRANCY\_CHECK is not enabled

Definition at line 39 of file `tfm_tz_psa_ns_api.c`.

### 8.89.1.7 tfm\_ns\_check\_safe\_entry()

```
enum tfm_reenter_check_err_t tfm_ns_check_safe_entry (
 void)
```

Perform a reentrancy check before safely proceeding with a PSA Function Call.

#### Note

Requires TFM\_TZ\_REENTRANCY\_CHECK option to be enabled

#### Returns

Returns [tfm\\_reenter\\_check\\_err\\_t](#) status code.

[TFM\\_REENTRANCY\\_CHECK\\_SUCCESS](#) The caller can then perform the `psa_call` successfully

[TFM\\_REENTRANCY\\_CHECK\\_ERR\\_FLAG\\_STATUS](#) Either: The check flag has been set already, thus either another call is in progress or it needs to be cleared. Or: TFM\_TZ\_REENTRANCY\_CHECK is not enabled

[TFM\\_REENTRANCY\\_CHECK\\_ERR\\_INVALID\\_ACCESS](#) The caller cannot safely proceed with a `psa_call` because the secure state was interrupted and requires completion. Try later.

Definition at line 34 of file `tfm_tz_psa_ns_api.c`.

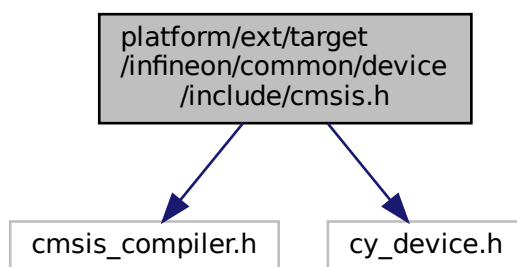


## 8.90 platform/ext/target/infineon/common/device/include/cmsis.h File Reference

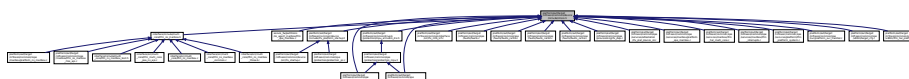
```
#include "cmsis_compiler.h"
```

```
#include "cy_device.h"
```

Include dependency graph for cmsis.h:

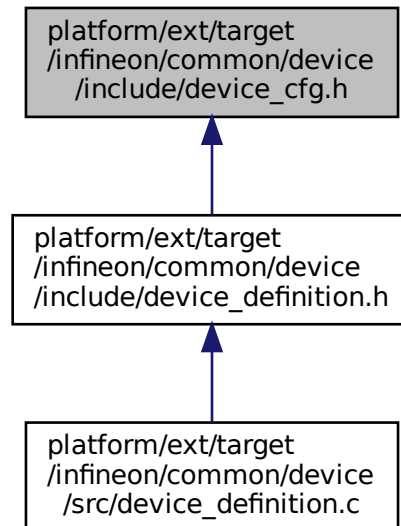


This graph shows which files directly or indirectly include this file:



## 8.91 platform/ext/target/infineon/common/device/include/device\_cfg.h File Reference

This graph shows which files directly or indirectly include this file:



### Macros

- `#define IFX_IRQ_TEST_TIMER_S`

### 8.91.1 Macro Definition Documentation

#### 8.91.1.1 IFX\_IRQ\_TEST\_TIMER\_S

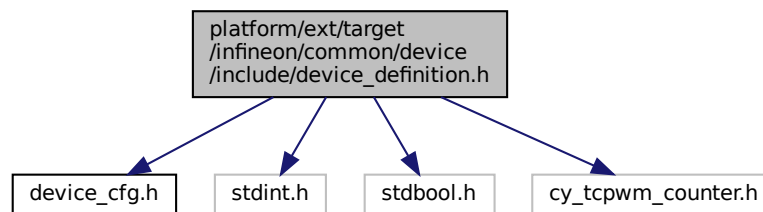
```
#define IFX_IRQ_TEST_TIMER_S
```

Definition at line 13 of file device\_cfg.h.

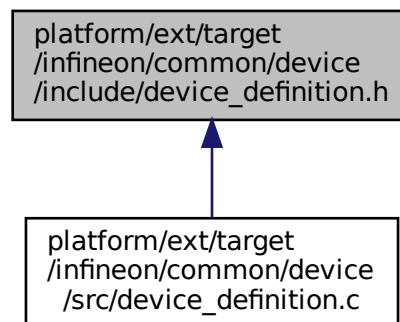
## 8.92 platform/ext/target/infineon/common/device/include/device\_definition.h File Reference

```
#include "device_cfg.h"
#include <stdint.h>
#include <stdbool.h>
#include "cy_tcpwm_counter.h"
```

Include dependency graph for device\_definition.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `ifx_timer_irq_test_dev_config`

## Typedefs

- typedef struct `ifx_timer_irq_test_dev_config` `ifx_timer_irq_test_dev_t`

### 8.92.1 Typedef Documentation

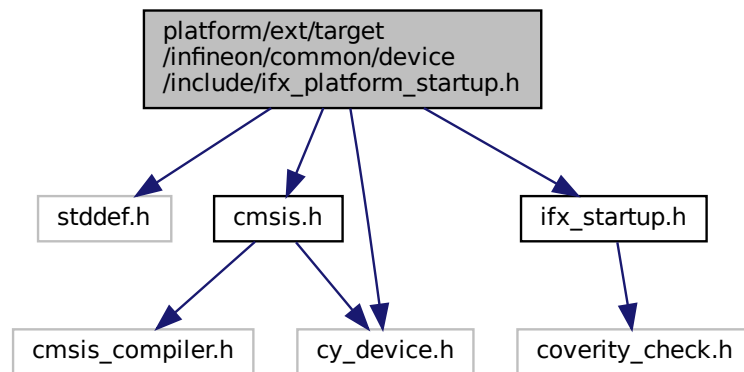
#### 8.92.1.1 ifx\_timer\_irq\_test\_dev\_t

```
typedef struct ifx_timer_irq_test_dev_config ifx_timer_irq_test_dev_t
```

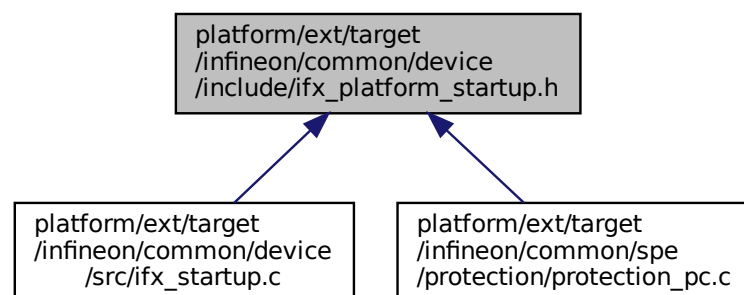
## 8.93 platform/ext/target/infineon/common/device/include/ifx\_platform\_startup.h File Reference

```
#include <stddef.h>
#include "cmsis.h"
```

```
#include "cy_device.h"
#include "ifx_startup.h"
Include dependency graph for ifx_platform_startup.h:
```

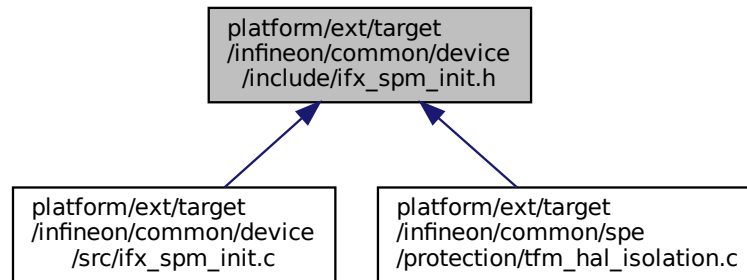


This graph shows which files directly or indirectly include this file:



## 8.94 platform/ext/target/infineon/common/device/include/ifs\_spm\_init.h File Reference

This graph shows which files directly or indirectly include this file:



### Functions

- [void ifs\\_init\\_spm\\_peripherals \(void\)](#)

#### 8.94.1 Function Documentation

##### 8.94.1.1 ifs\_init\_spm\_peripherals()

```
void ifs_init_spm_peripherals (
 void)
```

Initialize SPM peripherals.

Definition at line 17 of file `ifs_spm_init.c`.

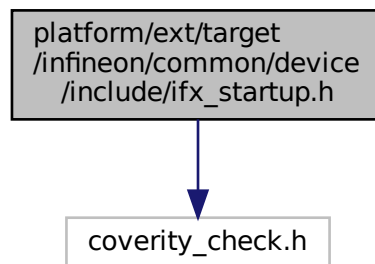
Here is the call graph for this function:



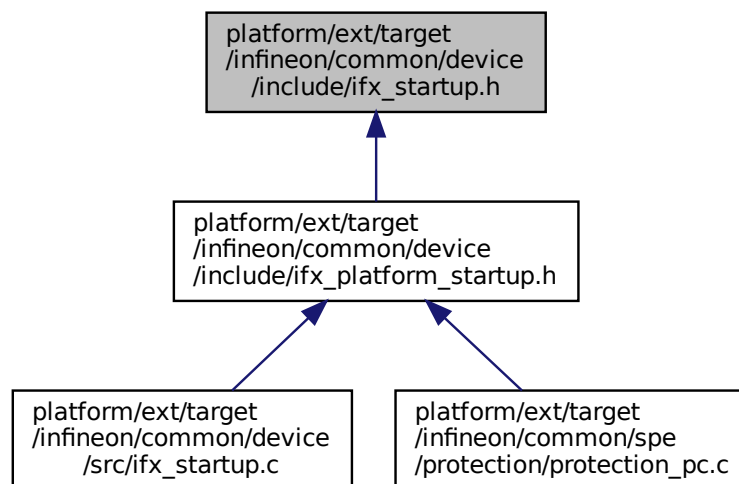
## 8.95 platform/ext/target/infineon/common/device/include/ifs\_startup.h File Reference

```
#include "coverity_check.h"
```

Include dependency graph for ifx\_startup.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IS_POWER_OF_TWO(x) ((x) && !((x) & ((x) - 1)))`
- `#define IFX_VECTOR_TABLE_ATTRIBUTE __attribute__((aligned(VECTORTABLE_ALIGN))) __VECTO↵  
R_TABLE_ATTRIBUTE`
- `#define DEFAULT_IRQ_HANDLER(handler_name) __NO_RETURN void handler_name(void) __attribute↵  
__ ((weak, alias("Default_Handler")));`

## Typedefs

- `typedef void(* VECTOR_TABLE_Type) (void)`

## Variables

- `const VECTOR_TABLE_Type __vector_table [VECTORTABLE_SIZE]`

## 8.95.1 Macro Definition Documentation

### 8.95.1.1 DEFAULT\_IRQ\_HANDLER

```
#define DEFAULT_IRQ_HANDLER(
 handler_name) __NO_RETURN void handler_name(void) __attribute__((weak, alias("Default↵
_Handler")));
```

Definition at line 61 of file ifx\_startup.h.

### 8.95.1.2 IFX\_VECTOR\_TABLE\_ATTRIBUTE

```
#define IFX_VECTOR_TABLE_ATTRIBUTE __attribute__((aligned(VECTORTABLE_ALIGN))) __VECTOR_TABLE↵
E_ATTRIBUTE
```

Definition at line 39 of file ifx\_startup.h.

### 8.95.1.3 IS\_POWER\_OF\_TWO

```
#define IS_POWER_OF_TWO(
 x) ((x) && !((x) & ((x) - 1)))
```

Definition at line 15 of file ifx\_startup.h.

## 8.95.2 Typedef Documentation

### 8.95.2.1 VECTOR\_TABLE\_Type

```
typedef void(* VECTOR_TABLE_Type) (void)
```

Definition at line 35 of file ifx\_startup.h.

## 8.95.3 Variable Documentation

### 8.95.3.1 \_\_vector\_table

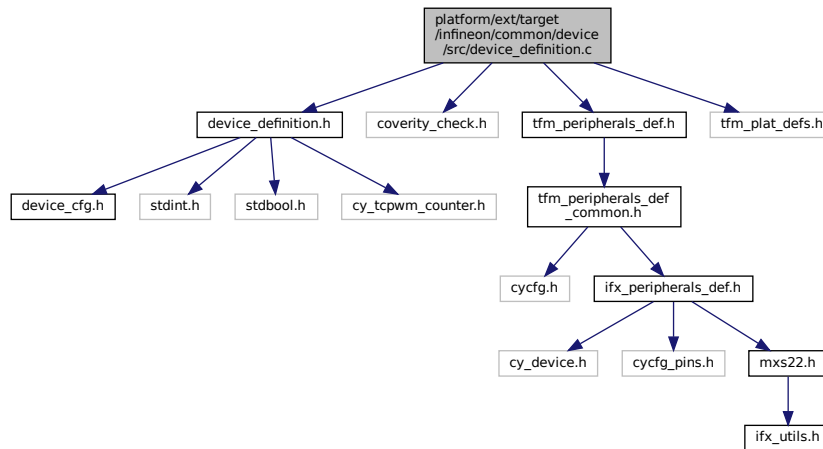
```
const VECTOR_TABLE_Type __vector_table[VECTORTABLE_SIZE]
```

## 8.96 platform/ext/target/infineon/common/device/src/device\_definition.c File Reference

This file defines exports the structures based on the peripheral definitions from [device\\_cfg.h](#). This retarget file is meant to be used as a helper for baremetal applications and/or as an example of how to configure the generic driver structures.

```
#include "device_definition.h"
#include "coverity_check.h"
#include "tfm_peripherals_def.h"
#include "tfm_plat_defs.h"
```

Include dependency graph for device\_definition.c:



### 8.96.1 Detailed Description

This file defines exports the structures based on the peripheral definitions from [device\\_cfg.h](#). This retarget file is meant to be used as a helper for baremetal applications and/or as an example of how to configure the generic driver structures.

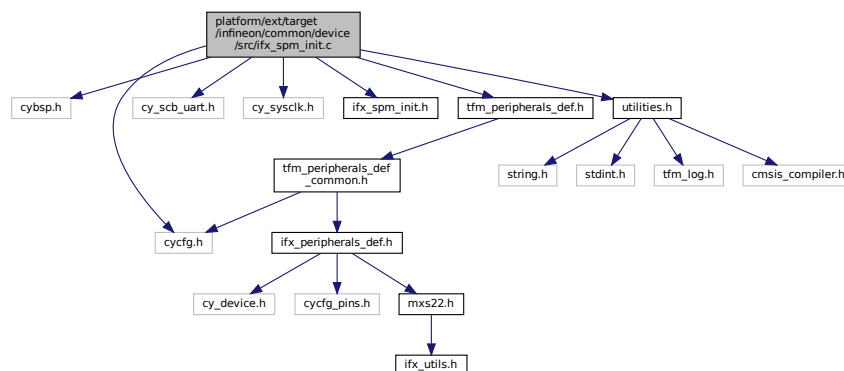
## 8.97 platform/ext/target/infineon/common/device/src/ifx\_spm\_init.c File Reference

```

#include "cybsp.h"
#include "cycfg.h"
#include "cy_scb_uart.h"
#include "cy_sysclk.h"
#include "ifx_spm_init.h"
#include "tfm_peripherals_def.h"
#include "utilities.h"

```

Include dependency graph for ifx\_spm\_init.c:





## Functions

- [void ifx\\_init\\_spm\\_peripherals \(void\)](#)

### 8.97.1 Function Documentation

#### 8.97.1.1 ifx\_init\_spm\_peripherals()

```
void ifx_init_spm_peripherals (
 void)
```

Initialize SPM peripherals.

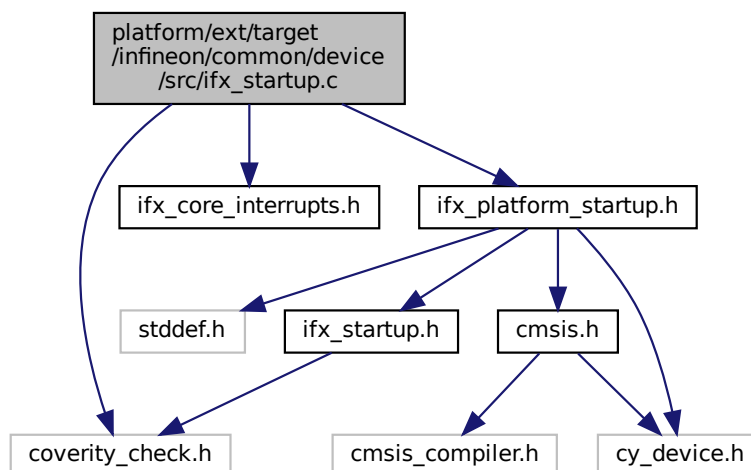
Definition at line 17 of file ifx\_spm\_init.c.

Here is the call graph for this function:



## 8.98 platform/ext/target/infineon/common/device/src/ifx\_startup.c File Reference

```
#include "coverity_check.h"
#include "ifx_core_interrupts.h"
#include "ifx_platform_startup.h"
Include dependency graph for ifx_startup.c:
```



## Functions

- `__NO_RETURN` [void \\_\\_PROGRAM\\_START \(void\)](#)

- `__NO_RETURN void Reset_Handler (void)`
- `__NO_RETURN void Default_Handler (void)`

## Variables

- `uint32_t __INITIAL_SP`
- `uint32_t __STACK_LIMIT`
- `const IFX_CORE_DEFINE_INTERRUPTS_LIST VECTOR_TABLE_Type __vector_table[VECTORTABLE_SIZE] IFX_VECTOR_TABLE_ATTRIBUTE`

## 8.98.1 Function Documentation

### 8.98.1.1 `__PROGRAM_START()`

```
__NO_RETURN void __PROGRAM_START (
 void)
```

Here is the caller graph for this function:



### 8.98.1.2 `Default_Handler()`

```
__NO_RETURN void Default_Handler (
 void)
```

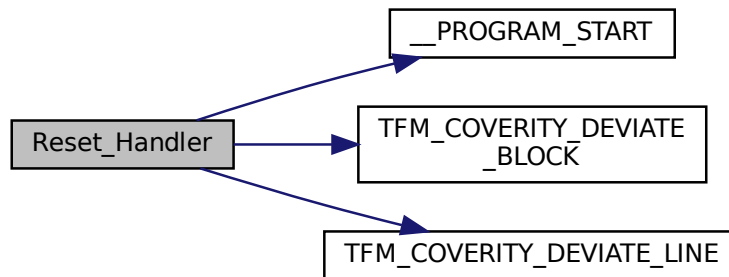
Definition at line 201 of file ifx\_startup.c.

### 8.98.1.3 `Reset_Handler()`

```
__NO_RETURN void Reset_Handler (
 void)
```

Definition at line 125 of file ifx\_startup.c.

Here is the call graph for this function:



## 8.98.2 Variable Documentation

### 8.98.2.1 \_\_INITIAL\_SP

```
uint32_t __INITIAL_SP
```

### 8.98.2.2 \_\_STACK\_LIMIT

```
uint32_t __STACK_LIMIT
```

### 8.98.2.3 IFX\_VECTOR\_TABLE\_ATTRIBUTE

```
const IFX_CORE_DEFINE_INTERRUPTS_LIST VECTOR_TABLE_Type __vector_table [VECTORTABLE_SIZE] IFX_VECTOR_TABLE_ATTRIBUTE
```

**Initial value:**

```
= {
 (VECTOR_TABLE_Type) (&__INITIAL_SP),
 Reset_Handler,
 NMI_Handler,
 HardFault_Handler,
 MemManage_Handler,
 BusFault_Handler,
 UsageFault_Handler,
 SecureFault_Handler,
 0,
 0,
 0,
 SVC_Handler,
 DebugMon_Handler,
 0,
 PendSV_Handler,
 SysTick_Handler,

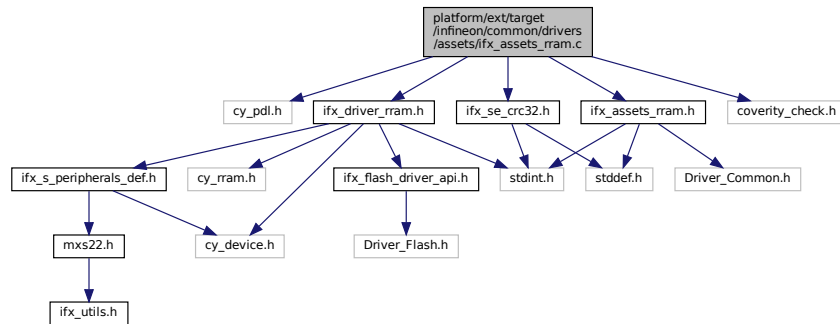
 IFX_CORE_INTERRUPTS_LIST
}
```

Definition at line 86 of file `ifx_startup.c`.

## 8.99 platform/ext/target/infineon/common/drivers/assets/ifx\_assets\_ram.c File Reference

```
#include "cy_pdl.h"
```

```
#include "ifx_assets_rram.h"
#include "ifx_driver_rram.h"
#include "ifx_se_crc32.h"
#include "coverity_check.h"
Include dependency graph for ifx_assets_rram.c:
```



## Functions

- `int32_t ifx_assets_rram_read_block` (`uint32_t address`, `void *data`, `uint32_t length`)
- `int32_t ifx_assets_rram_read_asset` (`uint32_t address`, `void *asset`, `size_t size`)

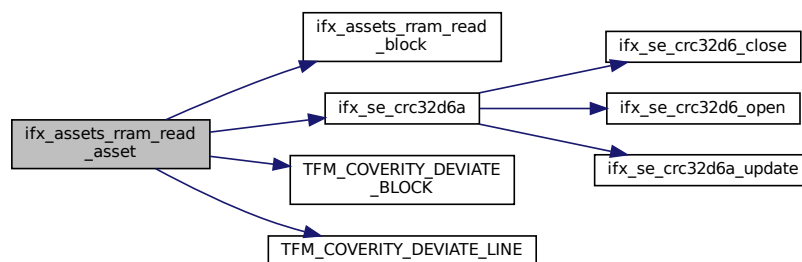
### 8.99.1 Function Documentation

#### 8.99.1.1 ifx\_assets\_rram\_read\_asset()

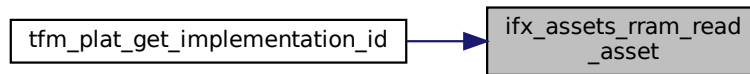
```
int32_t ifx_assets_rram_read_asset (
 uint32_t address,
 void * asset,
 size_t size)
```

Definition at line 41 of file `ifx_assets_rram.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

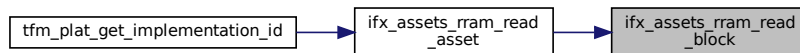


### 8.99.1.2 ifx\_assets\_rram\_read\_block()

```
int32_t ifx_assets_rram_read_block (
 uint32_t address,
 void * data,
 uint32_t length)
```

Definition at line 16 of file `ifx_assets_rram.c`.

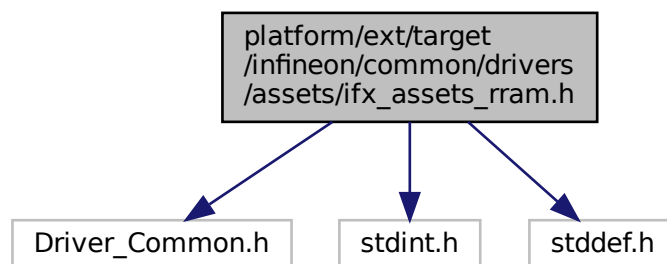
Here is the caller graph for this function:



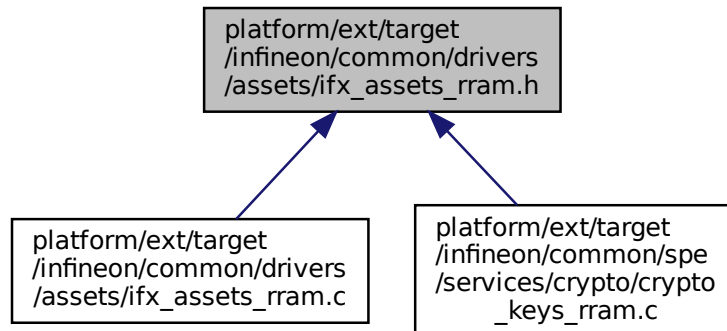
## 8.100 platform/ext/target/infineon/common/drivers/assets/ifx\_assets\_rram.h File Reference

```
#include "Driver_Common.h"
#include <stdint.h>
#include <stddef.h>
```

Include dependency graph for `ifx_assets_rram.h`:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_ASSETS_RRAM_CHECKSUM_SIZE (4)`

## Functions

- `int32_t ifx_assets_rram_read_block (uint32_t address, void *data, uint32_t length)`
- `int32_t ifx_assets_rram_read_asset (uint32_t address, void *asset, size_t size)`

### 8.100.1 Macro Definition Documentation

#### 8.100.1.1 IFX\_ASSETS\_RRAM\_CHECKSUM\_SIZE

```
#define IFX_ASSETS_RRAM_CHECKSUM_SIZE (4)
```

Definition at line 18 of file ifx\_assets\_rram.h.

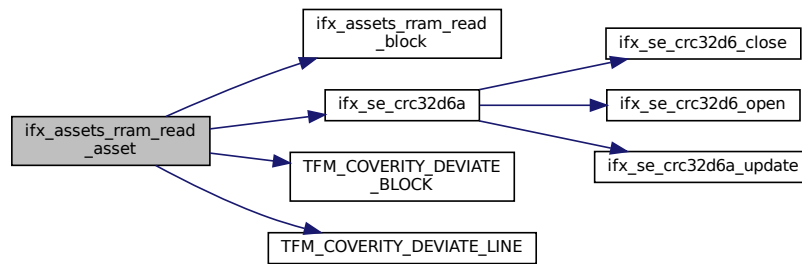
### 8.100.2 Function Documentation

#### 8.100.2.1 ifx\_assets\_rram\_read\_asset()

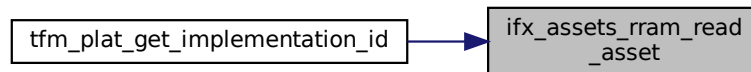
```
int32_t ifx_assets_rram_read_asset (
 uint32_t address,
 void * asset,
 size_t size)
```

Definition at line 41 of file ifx\_assets\_rram.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.100.2.2 ifx\_assets\_rram\_read\_block()

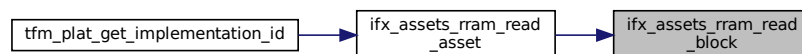
```

int32_t ifx_assets_rram_read_block (
 uint32_t address,
 void * data,
 uint32_t length)

```

Definition at line 16 of file ifx\_assets\_rram.c.

Here is the caller graph for this function:



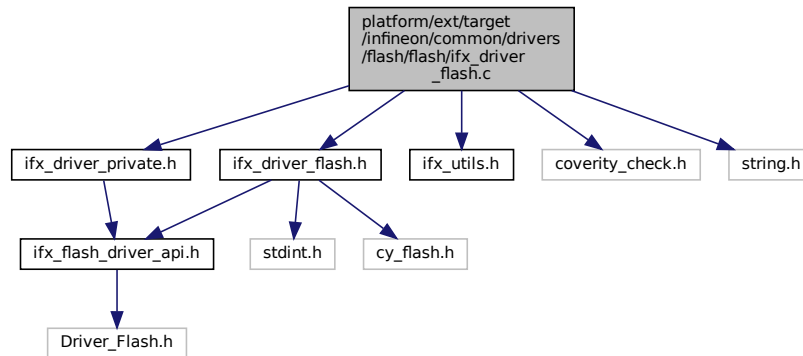
## 8.101 platform/ext/target/infineon/common/drivers/flash/flash/ifx\_driver\_flash.c File Reference

```

#include "ifx_driver_flash.h"
#include "ifx_driver_private.h"
#include "ifx_utils.h"
#include "coverity_check.h"
#include <string.h>

```

Include dependency graph for ifx\_driver\_flash.c:



## Macros

- `#define IFX_DRIVER_FLASH_VERSION ARM_DRIVER_VERSION_MAJOR_MINOR(0x1U, 0x0U)`
- `#define IFX_DRIVER_FLASH_EVENT ifx_flash_driver_event_t`
- `#define IFX_DRIVER_FLASH_FUNCTION_ARGUMENTS(...) ifx_flash_driver_handler_t handler __VA_OPT__(, ) __VA_ARGS__`
- `#define IFX_DRIVER_FLASH_FUNCTION_PARAMETERS(...) handler __VA_OPT__(, ) __VA_ARGS__`
- `#define IFX_DRIVER_FLASH_INSTANCE (handler)`
- `#define IFX_UNUSED_FLASH_DRIVER_INSTANCE IFX_UNUSED(handler)`
- `#define IFX_FDF_ARGS(...) IFX_DRIVER_FLASH_FUNCTION_ARGUMENTS(__VA_ARGS__)`
- `#define IFX_FDF_PARAMS(...) IFX_DRIVER_FLASH_FUNCTION_PARAMETERS(__VA_ARGS__)`

## Variables

- `const struct ifx_flash_driver_t ifx_driver_flash`

## 8.101.1 Macro Definition Documentation

### 8.101.1.1 IFX\_DRIVER\_FLASH\_EVENT

```
#define IFX_DRIVER_FLASH_EVENT ifx_flash_driver_event_t
```

Definition at line 36 of file ifx\_driver\_flash.c.

### 8.101.1.2 IFX\_DRIVER\_FLASH\_FUNCTION\_ARGUMENTS

```
#define IFX_DRIVER_FLASH_FUNCTION_ARGUMENTS(
 ...) ifx_flash_driver_handler_t handler __VA_OPT__(,) __VA_ARGS__
```

Definition at line 45 of file ifx\_driver\_flash.c.

### 8.101.1.3 IFX\_DRIVER\_FLASH\_FUNCTION\_PARAMETERS

```
#define IFX_DRIVER_FLASH_FUNCTION_PARAMETERS(
 ...) handler __VA_OPT__(,) __VA_ARGS__
```

Definition at line 46 of file ifx\_driver\_flash.c.



#### 8.101.1.4 IFX\_DRIVER\_FLASH\_INSTANCE

```
#define IFX_DRIVER_FLASH_INSTANCE (handler)
```

Definition at line 50 of file ifx\_driver\_flash.c.

#### 8.101.1.5 IFX\_DRIVER\_FLASH\_VERSION

```
#define IFX_DRIVER_FLASH_VERSION ARM_DRIVER_VERSION_MAJOR_MINOR(0x1U, 0x0U)
```

Definition at line 22 of file ifx\_driver\_flash.c.

#### 8.101.1.6 IFX\_FDF\_ARGS

```
#define IFX_FDF_ARGS(
 ...) IFX_DRIVER_FLASH_FUNCTION_ARGUMENTS(__VA_ARGS__)
```

Definition at line 55 of file ifx\_driver\_flash.c.

#### 8.101.1.7 IFX\_FDF\_PARAMS

```
#define IFX_FDF_PARAMS(
 ...) IFX_DRIVER_FLASH_FUNCTION_PARAMETERS(__VA_ARGS__)
```

Definition at line 57 of file ifx\_driver\_flash.c.

#### 8.101.1.8 IFX\_UNUSED\_FLASH\_DRIVER\_INSTANCE

```
#define IFX_UNUSED_FLASH_DRIVER_INSTANCE IFX_UNUSED(handler)
```

Definition at line 51 of file ifx\_driver\_flash.c.

### 8.101.2 Variable Documentation

#### 8.101.2.1 ifx\_driver\_flash

```
const struct ifx_flash_driver_t ifx_driver_flash
```

**Initial value:**

```
= {
 .GetVersion = ifx_driver_flash_get_version,
 .GetCapabilities = ifx_driver_flash_get_capabilities,
 .Initialize = ifx_driver_flash_initialize,
 .Uninitialize = ifx_driver_flash_uninitialize,
 .PowerControl = ifx_driver_flash_power_control,
 .ReadData = ifx_driver_flash_read_data,
 .ProgramData = ifx_driver_flash_program_data,
 .EraseSector = ifx_driver_flash_erase_sector,
 .EraseChip = ifx_driver_flash_erase_chip,
 .GetStatus = ifx_driver_flash_get_status,
 .GetInfo = ifx_driver_flash_get_info,
}
```

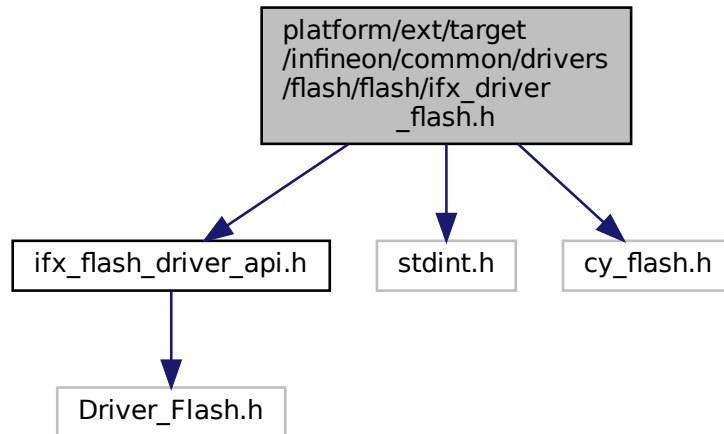
Definition at line 315 of file ifx\_driver\_flash.c.

## 8.102 platform/ext/target/infineon/common/drivers/flash/flash/ifx\_driver\_flash.h File Reference

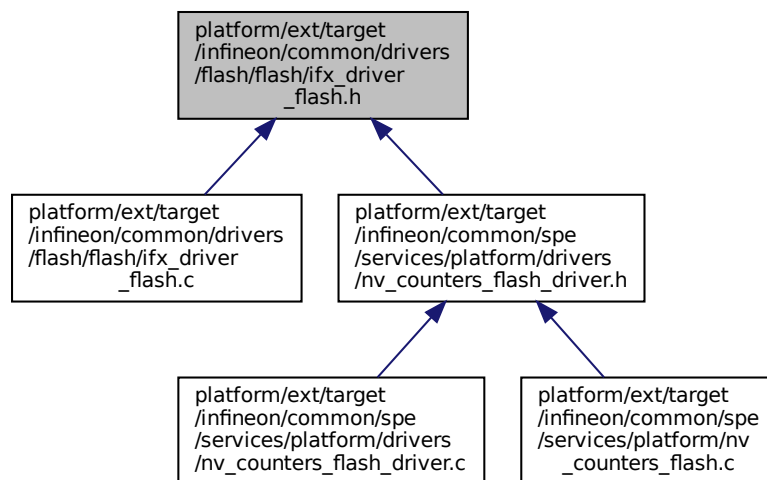
```
#include "ifx_flash_driver_api.h"
#include <stdint.h>
```

```
#include "cy_flash.h"
```

Include dependency graph for ifx\_driver\_flash.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ifx\\_driver\\_flash\\_obj\\_t](#)

*Structure containing the FLASH driver instance parameters.*

## Macros

- #define [IFX\\_DRIVER\\_FLASH\\_ERASE\\_VALUE](#) (0xFFU)
- #define [IFX\\_DRIVER\\_FLASH\\_PROGRAM\\_UNIT](#) (4U)

- `#define IFX_DRIVER_FLASH_SECTOR_SIZE CY_FLASH_SIZEOF_ROW /* 512 B */`
- `#define IFX_DRIVER_FLASH_CREATE_INSTANCE(instance_name, instance_obj)`

## Variables

- const struct `ifx_flash_driver_t` `ifx_driver_flash`

## 8.102.1 Macro Definition Documentation

### 8.102.1.1 IFX\_DRIVER\_FLASH\_CREATE\_INSTANCE

```
#define IFX_DRIVER_FLASH_CREATE_INSTANCE(
 instance_name,
 instance_obj)
```

#### Value:

```
/* Use variable to validate instance_obj type */ \
static const ifx_flash_driver_instance_t instance_name = { \
 .handler = &(instance_obj), \
 .driver = &ifx_driver_flash, \
};
```

Definition at line 59 of file `ifx_driver_flash.h`.

### 8.102.1.2 IFX\_DRIVER\_FLASH\_ERASE\_VALUE

```
#define IFX_DRIVER_FLASH_ERASE_VALUE (0xFFU)
```

Definition at line 21 of file `ifx_driver_flash.h`.

### 8.102.1.3 IFX\_DRIVER\_FLASH\_PROGRAM\_UNIT

```
#define IFX_DRIVER_FLASH_PROGRAM_UNIT (4U)
```

Definition at line 22 of file `ifx_driver_flash.h`.

### 8.102.1.4 IFX\_DRIVER\_FLASH\_SECTOR\_SIZE

```
#define IFX_DRIVER_FLASH_SECTOR_SIZE CY_FLASH_SIZEOF_ROW /* 512 B */
```

Definition at line 23 of file `ifx_driver_flash.h`.

## 8.102.2 Variable Documentation

### 8.102.2.1 ifx\_driver\_flash

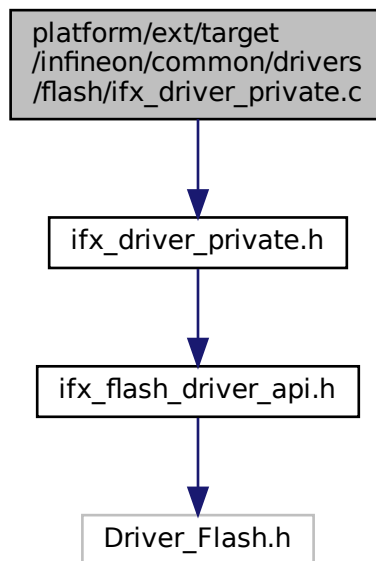
```
const struct ifx_flash_driver_t ifx_driver_flash
```

Definition at line 315 of file `ifx_driver_flash.c`.

## 8.103 platform/ext/target/infineon/common/drivers/flash/ifx\_driver\_private.c File Reference

```
#include "ifx_driver_private.h"
```

Include dependency graph for ifx\_driver\_private.c:



## Functions

- `int32_t ifx_flash_driver_get_region_size (ARM_FLASH_INFO *flash_info, uint32_t *region_size)`  
*Calculate flash driver instance region size.*
- `int32_t ifx_flash_driver_validate_region (ARM_FLASH_INFO *flash_info, uint32_t address, uint32_t size)`  
*Validate that memory block is inside of flash driver instance region.*
- `int32_t ifx_flash_driver_initialize (ARM_FLASH_INFO *flash_info, uint32_t start_address, uint32_t mem_block_size)`  
*Initialize Flash driver instance.*

## 8.103.1 Function Documentation

### 8.103.1.1 ifx\_flash\_driver\_get\_region\_size()

```
int32_t ifx_flash_driver_get_region_size (
 ARM_FLASH_INFO * flash_info,
 uint32_t * region_size)
```

Calculate flash driver instance region size.

#### Parameters

|     |                    |                                  |
|-----|--------------------|----------------------------------|
| in  | <i>flash_info</i>  | Flash driver info.               |
| out | <i>region_size</i> | Calculated region size in bytes. |

#### Return values

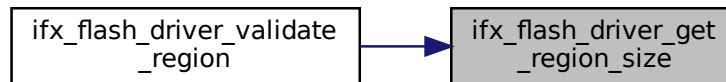
|                            |          |
|----------------------------|----------|
| <code>ARM_DRIVER_OK</code> | Success. |
|----------------------------|----------|

## Return values

|                                         |                                        |
|-----------------------------------------|----------------------------------------|
| <code>ARM_DRIVER_ERROR_PARAMETER</code> | Invalid input or region size overflow. |
|-----------------------------------------|----------------------------------------|

Definition at line 11 of file ifx\_driver\_private.c.

Here is the caller graph for this function:



## 8.103.1.2 ifx\_flash\_driver\_initialize()

```
int32_t ifx_flash_driver_initialize (
 ARM_FLASH_INFO * flash_info,
 uint32_t start_address,
 uint32_t mem_block_size)
```

Initialize Flash driver instance.

## Parameters

|    |                       |                            |
|----|-----------------------|----------------------------|
| in | <i>flash_info</i>     | Flash driver info          |
| in | <i>start_address</i>  | Flash region start address |
| in | <i>mem_block_size</i> | Size of flash memory block |

## Return values

|                            |                                        |
|----------------------------|----------------------------------------|
| <code>ARM_DRIVER_OK</code> | Success.                               |
| <i>Other</i>               | <code>ARM_DRIVER_ERROR_*</code> Failed |

Definition at line 57 of file ifx\_driver\_private.c.

## 8.103.1.3 ifx\_flash\_driver\_validate\_region()

```
int32_t ifx_flash_driver_validate_region (
 ARM_FLASH_INFO * flash_info,
 uint32_t address,
 uint32_t size)
```

Validate that memory block is inside of flash driver instance region.

## Parameters

|    |                   |                               |
|----|-------------------|-------------------------------|
| in | <i>flash_info</i> | Flash driver info             |
| in | <i>address</i>    | Start address of memory block |
| in | <i>size</i>       | Memory block size             |

## Return values

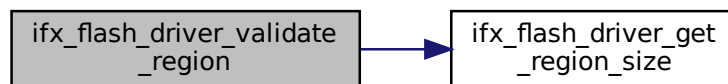
|                                         |                                                          |
|-----------------------------------------|----------------------------------------------------------|
| <code>ARM_DRIVER_OK</code>              | Memory block is inside of flash driver instance region.  |
| <code>ARM_DRIVER_ERROR_PARAMETER</code> | Memory block is outside of flash driver instance region. |

## Note

instance settings are not validated by this function!

Definition at line 28 of file `ifx_driver_private.c`.

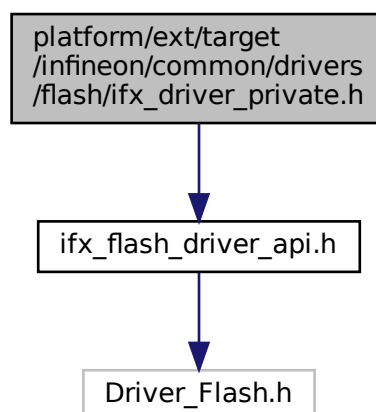
Here is the call graph for this function:



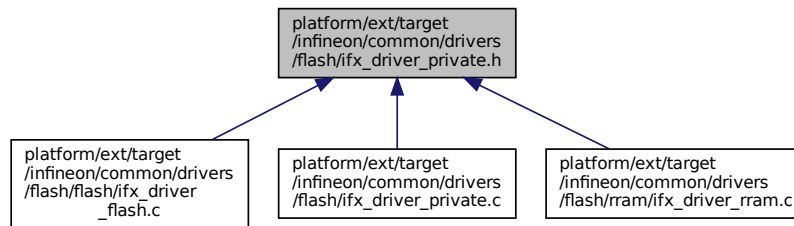
## 8.104 platform/ext/target/infineon/common/drivers/flash/ifx\_driver\_private.h File Reference

```
#include "ifx_flash_driver_api.h"
```

Include dependency graph for `ifx_driver_private.h`:



This graph shows which files directly or indirectly include this file:



## Functions

- [int32\\_t ifx\\_flash\\_driver\\_validate\\_region](#) (ARM\_FLASH\_INFO \*flash\_info, uint32\_t address, uint32\_t size)  
*Validate that memory block is inside of flash driver instance region.*
- [int32\\_t ifx\\_flash\\_driver\\_get\\_region\\_size](#) (ARM\_FLASH\_INFO \*flash\_info, uint32\_t \*region\_size)  
*Calculate flash driver instance region size.*
- [int32\\_t ifx\\_flash\\_driver\\_initialize](#) (ARM\_FLASH\_INFO \*flash\_info, uint32\_t start\_address, uint32\_t mem\_block\_size)  
*Initialize Flash driver instance.*

### 8.104.1 Function Documentation

#### 8.104.1.1 ifx\_flash\_driver\_get\_region\_size()

```
int32_t ifx_flash_driver_get_region_size (
 ARM_FLASH_INFO * flash_info,
 uint32_t * region_size)
```

Calculate flash driver instance region size.

##### Parameters

|     |                    |                                  |
|-----|--------------------|----------------------------------|
| in  | <i>flash_info</i>  | Flash driver info.               |
| out | <i>region_size</i> | Calculated region size in bytes. |

##### Return values

|                                   |                                        |
|-----------------------------------|----------------------------------------|
| <i>ARM_DRIVER_OK</i>              | Success.                               |
| <i>ARM_DRIVER_ERROR_PARAMETER</i> | Invalid input or region size overflow. |

Definition at line 11 of file ifx\_driver\_private.c.

Here is the caller graph for this function:



#### 8.104.1.2 ifx\_flash\_driver\_initialize()

```
int32_t ifx_flash_driver_initialize (
 ARM_FLASH_INFO * flash_info,
 uint32_t start_address,
 uint32_t mem_block_size)
```

Initialize Flash driver instance.

##### Parameters

|    |                       |                            |
|----|-----------------------|----------------------------|
| in | <i>flash_info</i>     | Flash driver info          |
| in | <i>start_address</i>  | Flash region start address |
| in | <i>mem_block_size</i> | Size of flash memory block |

##### Return values

|                      |                           |
|----------------------|---------------------------|
| <i>ARM_DRIVER_OK</i> | Success.                  |
| <i>Other</i>         | ARM_DRIVER_ERROR_* Failed |

Definition at line 57 of file ifx\_driver\_private.c.

#### 8.104.1.3 ifx\_flash\_driver\_validate\_region()

```
int32_t ifx_flash_driver_validate_region (
 ARM_FLASH_INFO * flash_info,
 uint32_t address,
 uint32_t size)
```

Validate that memory block is inside of flash driver instance region.

##### Parameters

|    |                   |                               |
|----|-------------------|-------------------------------|
| in | <i>flash_info</i> | Flash driver info             |
| in | <i>address</i>    | Start address of memory block |
| in | <i>size</i>       | Memory block size             |

##### Return values

|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| <i>ARM_DRIVER_OK</i>              | Memory block is inside of flash driver instance region.  |
| <i>ARM_DRIVER_ERROR_PARAMETER</i> | Memory block is outside of flash driver instance region. |

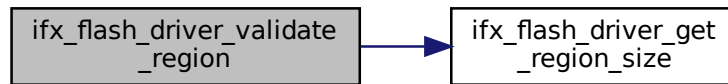


**Note**

instance settings are not validated by this function!

Definition at line 28 of file ifx\_driver\_private.c.

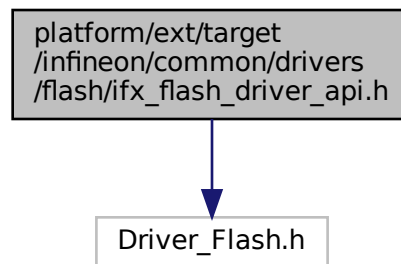
Here is the call graph for this function:



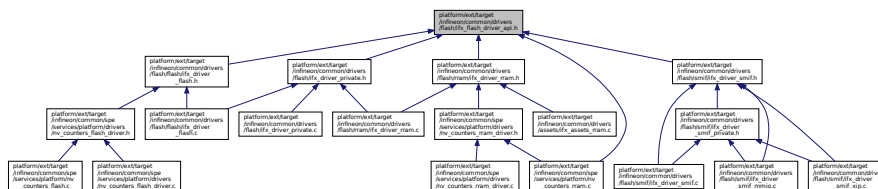
## 8.105 platform/ext/target/infineon/common/drivers/flash/ifx\_flash\_driver\_api.h File Reference

```
#include "Driver_Flash.h"
```

Include dependency graph for ifx\_flash\_driver\_api.h:



This graph shows which files directly or indirectly include this file:

**Data Structures**

- struct [ifx\\_flash\\_driver\\_t](#)  
Access structure of the Flash Driver.
- struct [ifx\\_flash\\_driver\\_instance\\_t](#)  
Flash driver instance.

## Macros

- `#define IFX_CREATE_CMSIS_FLASH_DRIVER(instance, cmsis_driver)`

## Typedefs

- typedef `void(* ifx_flash_driver_event_t) (ifx_flash_driver_handler_t handler, uint32_t event)`  
*Signal Flash event.*
- typedef struct `ifx_flash_driver_t ifx_flash_driver_t`  
*Access structure of the Flash Driver.*
- typedef struct `ifx_flash_driver_instance_t ifx_flash_driver_instance_t`  
*Flash driver instance.*

## Variables

- const typedef `void * ifx_flash_driver_handler_t`  
*Driver instance handler.*

## 8.105.1 Macro Definition Documentation

### 8.105.1.1 IFX\_CREATE\_CMSIS\_FLASH\_DRIVER

```
#define IFX_CREATE_CMSIS_FLASH_DRIVER(
 instance,
 cmsis_driver)
```

Create CMSIS flash driver wrapper for TF-M compatible implementation

instance - TF-M flash driver instance ([ifx\\_flash\\_driver\\_instance\\_t](#)) cmsis\_driver - CMSIS flash driver that is created (ARM\_DRIVER\_FLASH)

Definition at line 87 of file `ifx_flash_driver_api.h`.

## 8.105.2 Typedef Documentation

### 8.105.2.1 ifx\_flash\_driver\_event\_t

```
typedef void(* ifx_flash_driver_event_t) (ifx_flash_driver_handler_t handler, uint32_t event)
```

Signal Flash event.

#### Parameters

|    |                |                                                                   |
|----|----------------|-------------------------------------------------------------------|
| in | <i>handler</i> | Driver instance handler                                           |
| in | <i>event</i>   | Event notification mask, see <code>ARM_Flash_SignalEvent_t</code> |

Definition at line 29 of file `ifx_flash_driver_api.h`.

### 8.105.2.2 ifx\_flash\_driver\_instance\_t

```
typedef struct ifx_flash_driver_instance_t ifx_flash_driver_instance_t
```

Flash driver instance.

You can create a separate driver instance for same flash chip. But each instance can handle different areas of flash memory. This allows to provide isolation of data on driver level too.

### 8.105.2.3 ifx\_flash\_driver\_t

```
typedef struct ifx_flash_driver_t ifx_flash_driver_t
```

Access structure of the Flash Driver.

Interface is similar to CMSIS flash driver interface (ARM\_DRIVER\_FLASH) except that there is an additional driver instance handler (see [ifx\\_flash\\_driver\\_instance\\_t::handler](#)) which is specific to driver implementation.

## 8.105.3 Variable Documentation

### 8.105.3.1 ifx\_flash\_driver\_handler\_t

```
const typedef void* ifx_flash_driver_handler_t
```

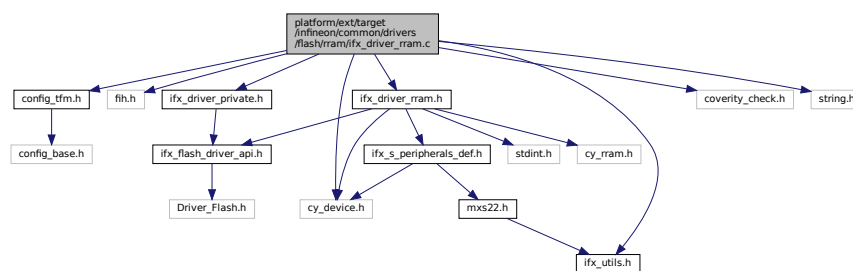
Driver instance handler.

This type is used to hide internal implementation of driver instance data.

Definition at line 22 of file [ifx\\_flash\\_driver\\_api.h](#).

## 8.106 platform/ext/target/infineon/common/drivers/flash/rram/ifx\_driver\_rram.c File Reference

```
#include "config_tfm.h"
#include "fih.h"
#include "ifx_driver_private.h"
#include "ifx_driver_rram.h"
#include "ifx_utils.h"
#include "coverity_check.h"
#include <string.h>
#include "cy_device.h"
Include dependency graph for ifx_driver_rram.c:
```



## Macros

- `#define IFX_DRIVER_RRAM_VERSION ARM_DRIVER_VERSION_MAJOR_MINOR(1UL, 0U)`

## Functions

- `TFM_COVERITY_DEVIATE_LINE` (MISRA\_C\_2023\_Rule\_2\_8, "Used in RRAM flash driver in the `IFX_DRIVER_RRAM_CREATE_INSTANCE` macro")

## Variables

- `const struct ifx_flash_driver_t ifx_driver_rram`

## 8.106.1 Macro Definition Documentation

### 8.106.1.1 IFX\_DRIVER\_RRAM\_VERSION

```
#define IFX_DRIVER_RRAM_VERSION ARM_DRIVER_VERSION_MAJOR_MINOR(1UL, 0U)
```

Definition at line 25 of file ifx\_driver\_rram.c.

## 8.106.2 Function Documentation

### 8.106.2.1 TFM\_COVERITY\_DEVIATE\_LINE()

```
TFM_COVERITY_DEVIATE_LINE (
 MISRA_C_2023_Rule_2_8 ,
 "Used in RRAM flash driver in the IFX_DRIVER_RRAM_CREATE_INSTANCE macro")
```



```

 .GetInfo = ifx_driver_rram_get_info,
}

```

Definition at line 292 of file ifx\_driver\_rram.c.

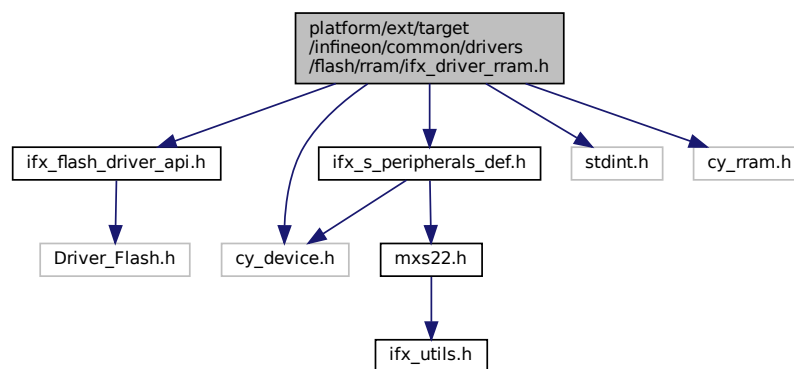
## 8.107 platform/ext/target/infineon/common/drivers/flash/rram/ifx\_driver\_rram.h File Reference

```

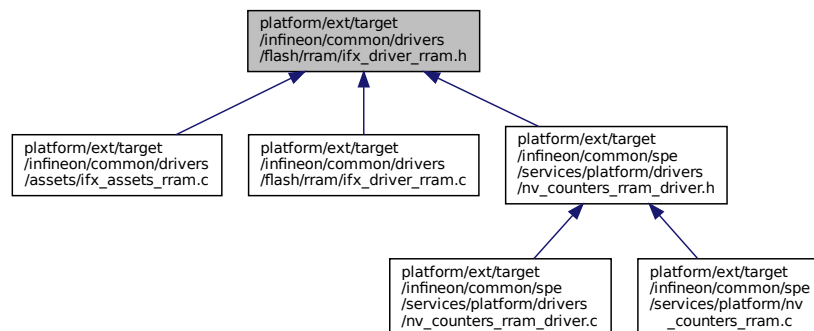
#include "ifx_flash_driver_api.h"
#include "ifx_s_peripherals_def.h"
#include <stdint.h>
#include "cy_device.h"
#include "cy_rram.h"

```

Include dependency graph for ifx\_driver\_rram.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ifx\\_driver\\_rram\\_obj\\_t](#)  
Structure containing the RRAM driver instance parameters.

## Macros

- #define [IFX\\_DRIVER\\_RRAM\\_ERASE\\_VALUE](#) (0x0UL)

- #define [IFX\\_DRIVER\\_RRAM\\_PROGRAM\\_UNIT](#) (4U)
- #define [IFX\\_DRIVER\\_RRAM\\_PC\\_LOCK\\_RETRIES](#) (100)
- #define [IFX\\_DRIVER\\_RRAM\\_CREATE\\_INSTANCE](#)(instance\_name, instance\_obj)

## Typedefs

- typedef struct [ifx\\_driver\\_rram\\_obj\\_t](#) [ifx\\_driver\\_rram\\_obj\\_t](#)  
*Structure containing the RRAM driver instance parameters.*

## Variables

- const struct [ifx\\_flash\\_driver\\_t](#) [ifx\\_driver\\_rram](#)

## 8.107.1 Macro Definition Documentation

### 8.107.1.1 IFX\_DRIVER\_RRAM\_CREATE\_INSTANCE

```
#define IFX_DRIVER_RRAM_CREATE_INSTANCE(
 instance_name,
 instance_obj)
```

#### Value:

```
/* Use variable to validate instance_obj type */ \
const ifx_flash_driver_instance_t instance_name = { \
 .handler = &(instance_obj), \
 .driver = &ifx_driver_rram, \
};
```

Definition at line 61 of file [ifx\\_driver\\_rram.h](#).

### 8.107.1.2 IFX\_DRIVER\_RRAM\_ERASE\_VALUE

```
#define IFX_DRIVER_RRAM_ERASE_VALUE (0x0UL)
```

Definition at line 27 of file [ifx\\_driver\\_rram.h](#).

### 8.107.1.3 IFX\_DRIVER\_RRAM\_PC\_LOCK\_RETRIES

```
#define IFX_DRIVER_RRAM_PC_LOCK_RETRIES (100)
```

Definition at line 31 of file [ifx\\_driver\\_rram.h](#).

### 8.107.1.4 IFX\_DRIVER\_RRAM\_PROGRAM\_UNIT

```
#define IFX_DRIVER_RRAM_PROGRAM_UNIT (4U)
```

Definition at line 28 of file [ifx\\_driver\\_rram.h](#).

## 8.107.2 Typedef Documentation

### 8.107.2.1 ifx\_driver\_rram\_obj\_t

```
typedef struct ifx_driver_rram_obj_t ifx_driver_rram_obj_t
```

Structure containing the RRAM driver instance parameters.

RRAM flash driver uses `start_address`, `flash_info.sector_size` and `flash_info.sector_count` to define a working region that is handled by active instance. Any access outside of this region is invalid and RRAM flash driver returns `ARM_DRIVER_ERROR_PARAMETER` in such cases.

[ifx\\_driver\\_rram](#) interface expects address to be relative to `start_address`.

## Note

This structure can be a constant.

### 8.107.3 Variable Documentation

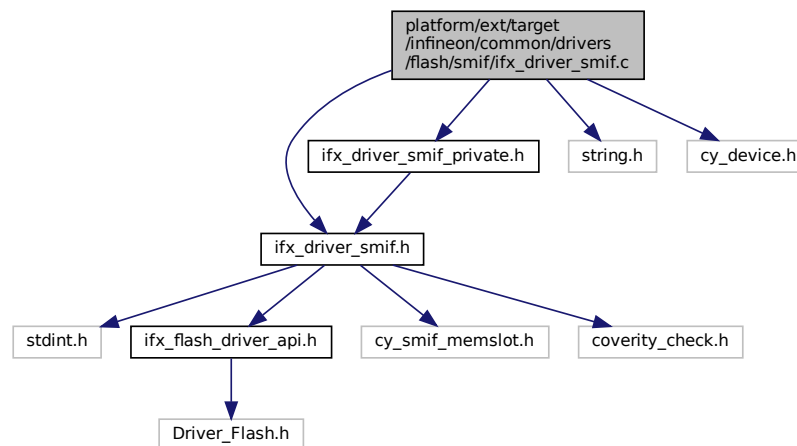
#### 8.107.3.1 ifx\_driver\_rram

const struct [ifx\\_flash\\_driver\\_t](#) ifx\_driver\_rram  
Definition at line 292 of file ifx\_driver\_rram.c.

## 8.108 platform/ext/target/infineon/common/drivers/flash/smif/ifx\_driver\_smif.c File Reference

```
#include "ifx_driver_smif.h"
#include "ifx_driver_smif_private.h"
#include <string.h>
#include "cy_device.h"
```

Include dependency graph for ifx\_driver\_smif.c:



## Macros

- #define [IFX\\_DRIVER\\_SMIF\\_VERSION](#) ARM\_DRIVER\_VERSION\_MAJOR\_MINOR(1UL, 0U)
- #define [IFX\\_SMIF\\_SHORT\\_POLLING\\_RETRIES](#) (100)

## Functions

- `int32_t ifx_driver_smif_poll_block_busy` (const [ifx\\_driver\\_smif\\_obj\\_t](#) \*obj)  
*This function checks if the status of the SMIF block is busy.*
- `int32_t ifx_driver_smif_set_mode` (const [ifx\\_driver\\_smif\\_obj\\_t](#) \*obj, [cy\\_en\\_smif\\_mode\\_t](#) mode)  
*Switch SMIF controller mode.*
- `int32_t ifx_driver_smif_validate_region` (const [ifx\\_driver\\_smif\\_obj\\_t](#) \*obj, [uint32\\_t](#) address, [uint32\\_t](#) size)  
*Validate that memory block is inside of flash driver instance region.*
- ARM\_DRIVER\_VERSION [ifx\\_driver\\_smif\\_get\\_version](#) ([ifx\\_flash\\_driver\\_handler\\_t](#) handler)
- ARM\_FLASH\_CAPABILITIES [ifx\\_driver\\_smif\\_get\\_capabilities](#) ([ifx\\_flash\\_driver\\_handler\\_t](#) handler)



- `int32_t ifx_driver_smif_initialize (ifx_flash_driver_handler_t handler, ifx_flash_driver_event_t cb_event)`  
*Initialize SMIF flash driver instance.*
- `int32_t ifx_driver_smif_uninitialize (ifx_flash_driver_handler_t handler)`
- `int32_t ifx_driver_smif_power_control (ifx_flash_driver_handler_t handler, ARM_POWER_STATE state)`
- `int32_t ifx_driver_smif_erase_sector (ifx_flash_driver_handler_t handler, uint32_t addr)`  
*Erase sector of SMIF flash driver instance.*
- `int32_t ifx_driver_smif_erase_chip (ifx_flash_driver_handler_t handler)`  
*Erase memory region allocated for SMIF flash driver instance.*
- `struct _ARM_FLASH_STATUS ifx_driver_smif_get_status (ifx_flash_driver_handler_t handler)`
- `ARM_FLASH_INFO * ifx_driver_smif_get_info (ifx_flash_driver_handler_t handler)`

## 8.108.1 Macro Definition Documentation

### 8.108.1.1 IFX\_DRIVER\_SMIF\_VERSION

```
#define IFX_DRIVER_SMIF_VERSION ARM_DRIVER_VERSION_MAJOR_MINOR(1UL, 0U)
```

Definition at line 22 of file `ifx_driver_smif.c`.

### 8.108.1.2 IFX\_SMIF\_SHORT\_POLLING\_RETRIES

```
#define IFX_SMIF_SHORT_POLLING_RETRIES (100)
```

Definition at line 26 of file `ifx_driver_smif.c`.

## 8.108.2 Function Documentation

### 8.108.2.1 ifx\_driver\_smif\_erase\_chip()

```
int32_t ifx_driver_smif_erase_chip (
 ifx_flash_driver_handler_t handler)
```

Erase memory region allocated for SMIF flash driver instance.

#### Parameters

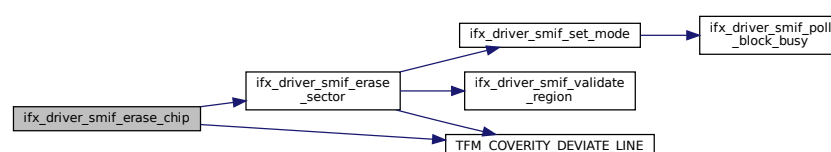
|                 |                      |                           |
|-----------------|----------------------|---------------------------|
| <code>in</code> | <code>handler</code> | Flash driver handler data |
|-----------------|----------------------|---------------------------|

#### Return values

|                            |                                        |
|----------------------------|----------------------------------------|
| <code>ARM_DRIVER_OK</code> | Success.                               |
| <i>Other</i>               | <code>ARM_DRIVER_ERROR_*</code> Failed |

Definition at line 290 of file `ifx_driver_smif.c`.

Here is the call graph for this function:



### 8.108.2.2 ifx\_driver\_smif\_erase\_sector()

```
int32_t ifx_driver_smif_erase_sector (
 ifx_flash_driver_handler_t handler,
 uint32_t addr)
```

Erase sector of SMIF flash driver instance.

#### Parameters

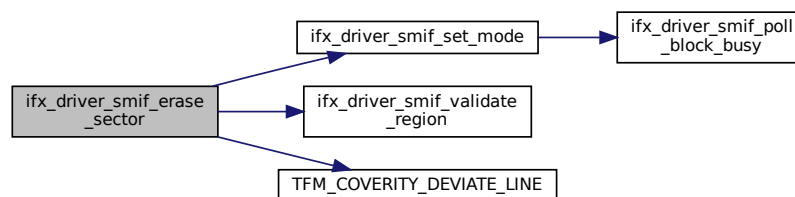
|    |                |                                            |
|----|----------------|--------------------------------------------|
| in | <i>handler</i> | Flash driver handler data                  |
| in | <i>addr</i>    | Sector address, must be aligned to sector. |

#### Return values

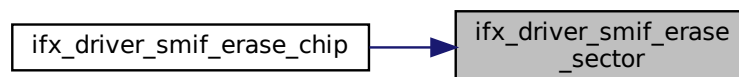
|                      |                           |
|----------------------|---------------------------|
| <i>ARM_DRIVER_OK</i> | Success.                  |
| <i>Other</i>         | ARM_DRIVER_ERROR_* Failed |

Definition at line 241 of file ifx\_driver\_smif.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.108.2.3 ifx\_driver\_smif\_get\_capabilities()

```
ARM_FLASH_CAPABILITIES ifx_driver_smif_get_capabilities (
 ifx_flash_driver_handler_t handler)
```

Definition at line 128 of file ifx\_driver\_smif.c.

**8.108.2.4 ifx\_driver\_smif\_get\_info()**

```
ARM_FLASH_INFO* ifx_driver_smif_get_info (
 ifx_flash_driver_handler_t handler)
```

Definition at line 323 of file ifx\_driver\_smif.c.

Here is the call graph for this function:

**8.108.2.5 ifx\_driver\_smif\_get\_status()**

```
struct _ARM_FLASH_STATUS ifx_driver_smif_get_status (
 ifx_flash_driver_handler_t handler)
```

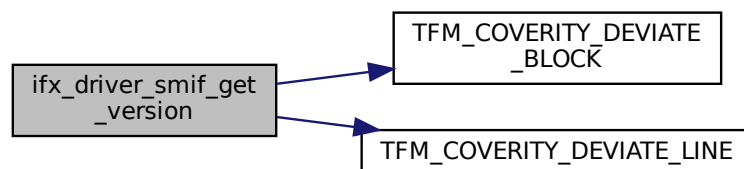
Definition at line 310 of file ifx\_driver\_smif.c.

**8.108.2.6 ifx\_driver\_smif\_get\_version()**

```
ARM_DRIVER_VERSION ifx_driver_smif_get_version (
 ifx_flash_driver_handler_t handler)
```

Definition at line 111 of file ifx\_driver\_smif.c.

Here is the call graph for this function:

**8.108.2.7 ifx\_driver\_smif\_initialize()**

```
int32_t ifx_driver_smif_initialize (
 ifx_flash_driver_handler_t handler,
 ifx_flash_driver_event_t cb_event)
```

Initialize SMIF flash driver instance.

**Parameters**

|    |                 |                                                |
|----|-----------------|------------------------------------------------|
| in | <i>handler</i>  | Flash driver handler data                      |
| in | <i>cb_event</i> | Callback event is not supported. Must be NULL. |

## Return values

|                      |                                  |
|----------------------|----------------------------------|
| <i>ARM_DRIVER_OK</i> | Success.                         |
| <i>Other</i>         | <i>ARM_DRIVER_ERROR_*</i> Failed |

Definition at line 162 of file ifx\_driver\_smif.c.

Here is the call graph for this function:



### 8.108.2.8 ifx\_driver\_smif\_poll\_block\_busy()

```
int32_t ifx_driver_smif_poll_block_busy (
 const ifx_driver_smif_obj_t * obj)
```

This function checks if the status of the SMIF block is busy.

## Parameters

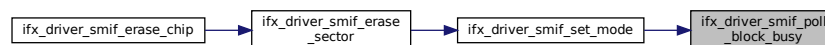
|           |            |                            |
|-----------|------------|----------------------------|
| <i>in</i> | <i>obj</i> | Flash driver instance data |
|-----------|------------|----------------------------|

## Return values

|                              |                           |
|------------------------------|---------------------------|
| <i>ARM_DRIVER_OK</i>         | Memory block is not busy. |
| <i>ARM_DRIVER_ERROR_BUSY</i> | Memory block is busy.     |

Definition at line 39 of file ifx\_driver\_smif.c.

Here is the caller graph for this function:



### 8.108.2.9 ifx\_driver\_smif\_power\_control()

```
int32_t ifx_driver_smif_power_control (
 ifx_flash_driver_handler_t handler,
 ARM_POWER_STATE state)
```

Definition at line 233 of file ifx\_driver\_smif.c.

### 8.108.2.10 ifx\_driver\_smif\_set\_mode()

```
int32_t ifx_driver_smif_set_mode (
```

```
const ifx_driver_smif_obj_t * obj,
cy_en_smif_mode_t mode)
```

Switch SMIF controller mode.

#### Parameters

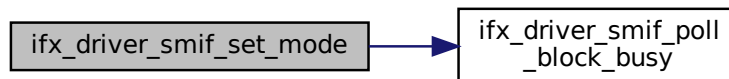
|    |                |                           |
|----|----------------|---------------------------|
| in | <i>handler</i> | Flash driver handler data |
| in | <i>mode</i>    | SMIF controller mode.     |

#### Return values

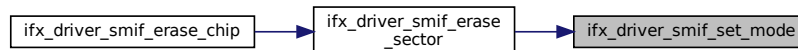
|                              |                                                          |
|------------------------------|----------------------------------------------------------|
| <i>ARM_DRIVER_OK</i>         | Memory block is inside of flash driver instance region.  |
| <i>ARM_DRIVER_ERROR_BUSY</i> | Memory block is outside of flash driver instance region. |

Definition at line 59 of file ifx\_driver\_smif.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.108.2.11 ifx\_driver\_smif\_uninitialize()

```
int32_t ifx_driver_smif_uninitialize (
 ifx_flash_driver_handler_t handler)
```

Definition at line 226 of file ifx\_driver\_smif.c.

#### 8.108.2.12 ifx\_driver\_smif\_validate\_region()

```
int32_t ifx_driver_smif_validate_region (
 const ifx_driver_smif_obj_t * obj,
 uint32_t address,
 uint32_t size)
```

Validate that memory block is inside of flash driver instance region.

#### Parameters

|    |            |                            |
|----|------------|----------------------------|
| in | <i>obj</i> | Flash driver instance data |
|----|------------|----------------------------|

## Parameters

|    |                |                               |
|----|----------------|-------------------------------|
| in | <i>address</i> | Start address of memory block |
| in | <i>size</i>    | Memory block size             |

## Return values

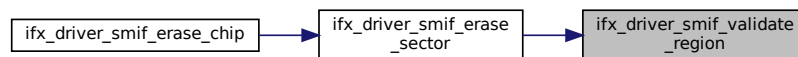
|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| <i>ARM_DRIVER_OK</i>              | Memory block is inside of flash driver instance region.  |
| <i>ARM_DRIVER_ERROR_PARAMETER</i> | Memory block is outside of flash driver instance region. |

## Note

instance settings are not validated by this function!

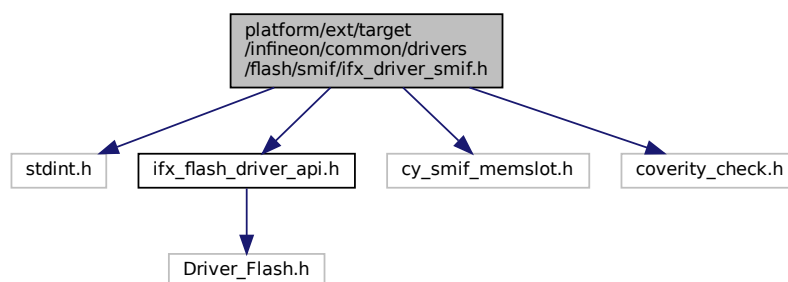
Definition at line 76 of file ifx\_driver\_smif.c.

Here is the caller graph for this function:

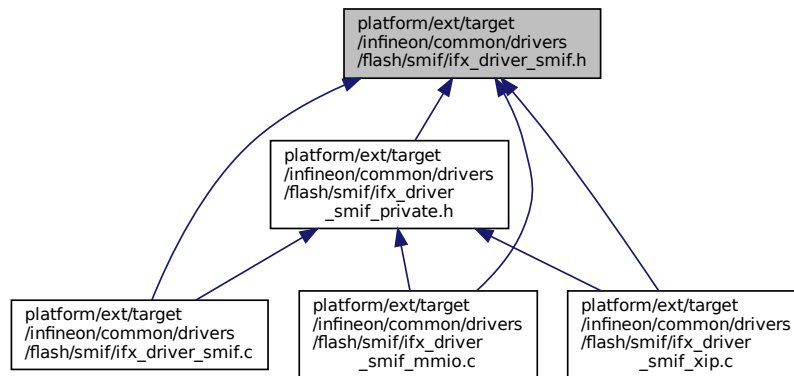


## 8.109 platform/ext/target/infineon/common/drivers/flash/smif/ifx\_driver\_smif.h File Reference

```
#include <stdint.h>
#include "ifx_flash_driver_api.h"
#include "cy_smif_memslot.h"
#include "coverity_check.h"
Include dependency graph for ifx_driver_smif.h:
```



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ifx\\_driver\\_smif\\_mem\\_t](#)  
Structure containing the SMIF driver memory configuration.
- struct [ifx\\_driver\\_smif\\_obj\\_t](#)  
Structure containing the SMIF driver instance data.

## Macros

- `#define IFX_DRIVER_SMIF_MMIO_INSTANCE(instance_name, instance_obj)`
- `#define IFX_DRIVER_SMIF_XIP_INSTANCE(instance_name, instance_obj)`

## Typedefs

- typedef struct [ifx\\_driver\\_smif\\_mem\\_t](#) [ifx\\_driver\\_smif\\_mem\\_t](#)  
Structure containing the SMIF driver memory configuration.
- typedef struct [ifx\\_driver\\_smif\\_obj\\_t](#) [ifx\\_driver\\_smif\\_obj\\_t](#)  
Structure containing the SMIF driver instance data.

## Variables

- const struct [ifx\\_flash\\_driver\\_t](#) [ifx\\_driver\\_smif\\_mmio](#)
- const struct [ifx\\_flash\\_driver\\_t](#) [ifx\\_driver\\_smif\\_xip](#)

## 8.109.1 Macro Definition Documentation

### 8.109.1.1 IFX\_DRIVER\_SMIF\_MMIO\_INSTANCE

```
#define IFX_DRIVER_SMIF_MMIO_INSTANCE(
 instance_name,
 instance_obj)
```

#### Value:

```
/* Use variable to validate instance_obj type */ \
const struct ifx_driver_smif_obj_t *instance_name##_validate_obj = &instance_obj; \
const ifx_flash_driver_instance_t instance_name = { \
 .handler = &instance_obj, \
 .driver = &ifx_driver_smif_mmio, \
```

```
};
```

Definition at line 64 of file ifx\_driver\_smif.h.

### 8.109.1.2 IFX\_DRIVER\_SMIF\_XIP\_INSTANCE

```
#define IFX_DRIVER_SMIF_XIP_INSTANCE(
 instance_name,
 instance_obj)
```

**Value:**

```
/* Use variable to validate instance_obj type */ \
const struct ifx_driver_smif_obj_t *instance_name##_validate_obj = &instance_obj; \
const ifx_flash_driver_instance_t instance_name = { \
 .handler = &instance_obj, \
 .driver = &ifx_driver_smif_xip, \
};
```

Definition at line 72 of file ifx\_driver\_smif.h.

## 8.109.2 Typedef Documentation

### 8.109.2.1 ifx\_driver\_smif\_mem\_t

```
typedef struct ifx_driver_smif_mem_t ifx_driver_smif_mem_t
```

Structure containing the SMIF driver memory configuration.

SMIF driver instance represent memory region with own boundaries. Such instance can share common SMIF memory configuration and context.

### 8.109.2.2 ifx\_driver\_smif\_obj\_t

```
typedef struct ifx_driver_smif_obj_t ifx_driver_smif_obj_t
```

Structure containing the SMIF driver instance data.

SMIF flash driver uses [offset](#), [size](#) to define a working region that is handled by active instance. Any access outside of this region is invalid and SMIF flash driver returns ARM\_DRIVER\_ERROR\_PARAMETER in such cases.

ifx\_driver\_smif interface expects address to be relative to [offset](#).

**Note**

This structure is modified by ifx\_driver\_smif.Initialize, so it should not be a constant.

## 8.109.3 Variable Documentation

### 8.109.3.1 ifx\_driver\_smif\_mmio

```
const struct ifx_flash_driver_t ifx_driver_smif_mmio
```

Definition at line 83 of file ifx\_driver\_smif\_mmio.c.

### 8.109.3.2 ifx\_driver\_smif\_xip

```
const struct ifx_flash_driver_t ifx_driver_smif_xip
```

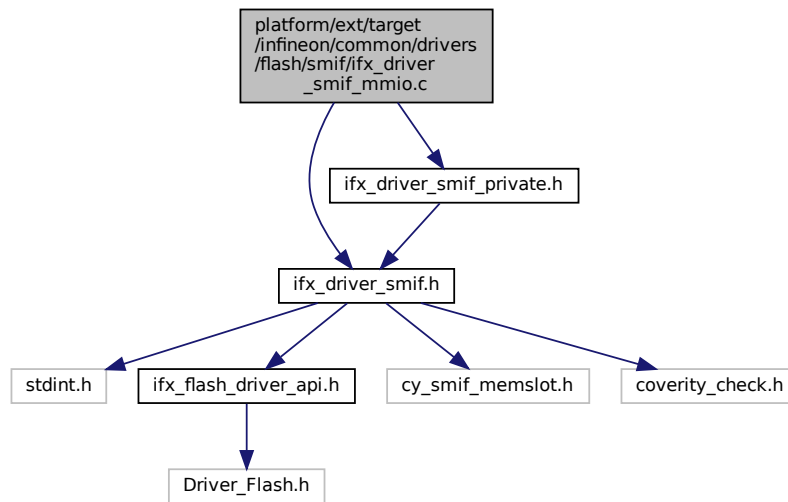
Definition at line 80 of file ifx\_driver\_smif\_xip.c.

## 8.110 platform/ext/target/infineon/common/drivers/flash/smif/ifx\_driver\_↵ \_smif\_mmio.c File Reference

```
#include "ifx_driver_smif.h"
#include "ifx_driver_smif_private.h"
```



Include dependency graph for ifx\_driver\_smif\_mmio.c:



## Variables

- const struct [ifx\\_flash\\_driver\\_t](#) `ifx_driver_smif_mmio`

### 8.110.1 Variable Documentation

#### 8.110.1.1 ifx\_driver\_smif\_mmio

```
const struct ifx_flash_driver_t ifx_driver_smif_mmio
```

**Initial value:**

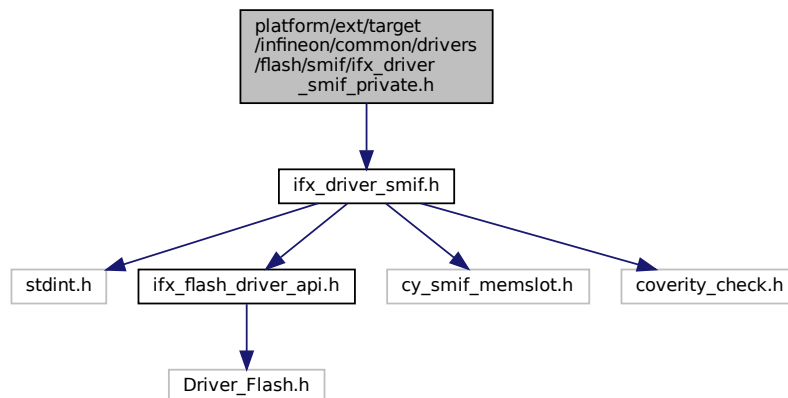
```
= {
 .GetVersion = ifx_driver_smif_get_version,
 .GetCapabilities = ifx_driver_smif_get_capabilities,
 .Initialize = ifx_driver_smif_initialize,
 .Uninitialize = ifx_driver_smif_uninitialize,
 .PowerControl = ifx_driver_smif_power_control,
 .ReadData = ifx_driver_smif_read_data,
 .ProgramData = ifx_driver_smif_program_data,
 .EraseSector = ifx_driver_smif_erase_sector,
 .EraseChip = ifx_driver_smif_erase_chip,
 .GetStatus = ifx_driver_smif_get_status,
 .GetInfo = ifx_driver_smif_get_info,
}
```

Definition at line 83 of file `ifx_driver_smif_mmio.c`.

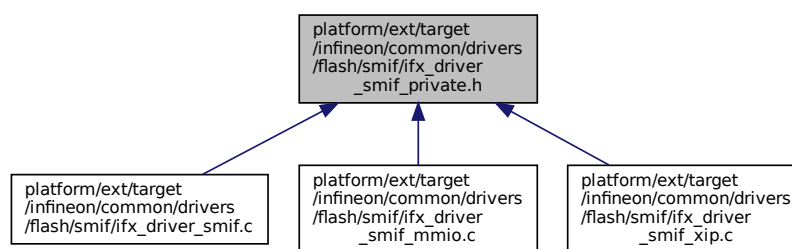
## 8.111 platform/ext/target/infineon/common/drivers/flash/smif/ifx\_driver\_smif\_private.h File Reference

```
#include "ifx_driver_smif.h"
```

Include dependency graph for ifx\_driver\_smif\_private.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_DRIVER_SMIF_DATA_WIDTH 4U`

## Functions

- `int32_t ifx_driver_smif_initialize (ifx_flash_driver_handler_t handler, ifx_flash_driver_event_t cb_event)`  
*Initialize SMIF flash driver instance.*
- `int32_t ifx_driver_smif_set_mode (const ifx_driver_smif_obj_t *obj, cy_en_smif_mode_t mode)`  
*Switch SMIF controller mode.*
- `int32_t ifx_driver_smif_validate_region (const ifx_driver_smif_obj_t *obj, uint32_t address, uint32_t size)`  
*Validate that memory block is inside of flash driver instance region.*
- `ARM_DRIVER_VERSION ifx_driver_smif_get_version (ifx_flash_driver_handler_t handler)`
- `ARM_FLASH_CAPABILITIES ifx_driver_smif_get_capabilities (ifx_flash_driver_handler_t handler)`
- `int32_t ifx_driver_smif_uninitialize (ifx_flash_driver_handler_t handler)`
- `int32_t ifx_driver_smif_power_control (ifx_flash_driver_handler_t handler, ARM_POWER_STATE state)`
- `struct _ARM_FLASH_STATUS ifx_driver_smif_get_status (ifx_flash_driver_handler_t handler)`
- `ARM_FLASH_INFO * ifx_driver_smif_get_info (ifx_flash_driver_handler_t handler)`
- `int32_t ifx_driver_smif_erase_sector (ifx_flash_driver_handler_t handler, uint32_t addr)`  
*Erase sector of SMIF flash driver instance.*

- `int32_t ifx_driver_smif_erase_chip (ifx_flash_driver_handler_t handler)`

Erase memory region allocated for SMIF flash driver instance.

## 8.111.1 Macro Definition Documentation

### 8.111.1.1 IFX\_DRIVER\_SMIF\_DATA\_WIDTH

```
#define IFX_DRIVER_SMIF_DATA_WIDTH 4U
```

Definition at line 14 of file `ifx_driver_smif_private.h`.

## 8.111.2 Function Documentation

### 8.111.2.1 ifx\_driver\_smif\_erase\_chip()

```
int32_t ifx_driver_smif_erase_chip (
 ifx_flash_driver_handler_t handler)
```

Erase memory region allocated for SMIF flash driver instance.

#### Parameters

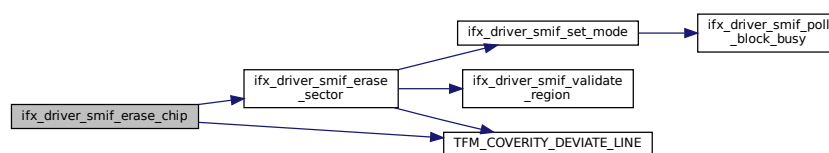
|    |                |                           |
|----|----------------|---------------------------|
| in | <i>handler</i> | Flash driver handler data |
|----|----------------|---------------------------|

#### Return values

|                      |                                  |
|----------------------|----------------------------------|
| <i>ARM_DRIVER_OK</i> | Success.                         |
| <i>Other</i>         | <i>ARM_DRIVER_ERROR_*</i> Failed |

Definition at line 290 of file `ifx_driver_smif.c`.

Here is the call graph for this function:



### 8.111.2.2 ifx\_driver\_smif\_erase\_sector()

```
int32_t ifx_driver_smif_erase_sector (
 ifx_flash_driver_handler_t handler,
 uint32_t addr)
```

Erase sector of SMIF flash driver instance.

#### Parameters

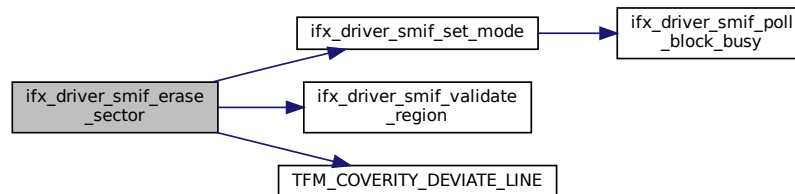
|    |                |                                            |
|----|----------------|--------------------------------------------|
| in | <i>handler</i> | Flash driver handler data                  |
| in | <i>addr</i>    | Sector address, must be aligned to sector. |

## Return values

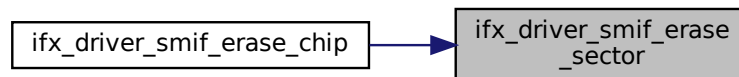
|                      |                           |
|----------------------|---------------------------|
| <i>ARM_DRIVER_OK</i> | Success.                  |
| <i>Other</i>         | ARM_DRIVER_ERROR_* Failed |

Definition at line 241 of file ifx\_driver\_smif.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.111.2.3 ifx\_driver\_smif\_get\_capabilities()

```
ARM_FLASH_CAPABILITIES ifx_driver_smif_get_capabilities (
 ifx_flash_driver_handler_t handler)
```

Definition at line 128 of file ifx\_driver\_smif.c.

### 8.111.2.4 ifx\_driver\_smif\_get\_info()

```
ARM_FLASH_INFO* ifx_driver_smif_get_info (
 ifx_flash_driver_handler_t handler)
```

Definition at line 323 of file ifx\_driver\_smif.c.

Here is the call graph for this function:



**8.111.2.5 ifx\_driver\_smif\_get\_status()**

```
struct _ARM_FLASH_STATUS ifx_driver_smif_get_status (
 ifx_flash_driver_handler_t handler)
```

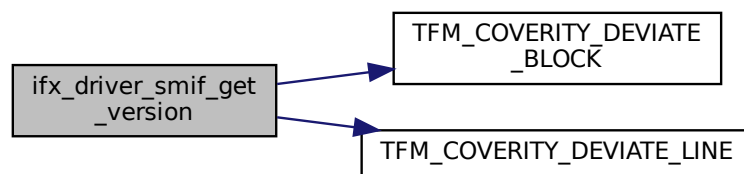
Definition at line 310 of file ifx\_driver\_smif.c.

**8.111.2.6 ifx\_driver\_smif\_get\_version()**

```
ARM_DRIVER_VERSION ifx_driver_smif_get_version (
 ifx_flash_driver_handler_t handler)
```

Definition at line 111 of file ifx\_driver\_smif.c.

Here is the call graph for this function:

**8.111.2.7 ifx\_driver\_smif\_initialize()**

```
int32_t ifx_driver_smif_initialize (
 ifx_flash_driver_handler_t handler,
 ifx_flash_driver_event_t cb_event)
```

Initialize SMIF flash driver instance.

**Parameters**

|    |                 |                                                |
|----|-----------------|------------------------------------------------|
| in | <i>handler</i>  | Flash driver handler data                      |
| in | <i>cb_event</i> | Callback event is not supported. Must be NULL. |

**Return values**

|                      |                           |
|----------------------|---------------------------|
| <i>ARM_DRIVER_OK</i> | Success.                  |
| <i>Other</i>         | ARM_DRIVER_ERROR_* Failed |

Definition at line 162 of file ifx\_driver\_smif.c.

Here is the call graph for this function:



### 8.111.2.8 ifx\_driver\_smif\_power\_control()

```
int32_t ifx_driver_smif_power_control (
 ifx_flash_driver_handler_t handler,
 ARM_POWER_STATE state)
```

Definition at line 233 of file ifx\_driver\_smif.c.

### 8.111.2.9 ifx\_driver\_smif\_set\_mode()

```
int32_t ifx_driver_smif_set_mode (
 const ifx_driver_smif_obj_t * obj,
 cy_en_smif_mode_t mode)
```

Switch SMIF controller mode.

#### Parameters

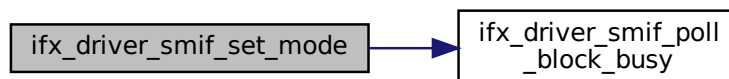
|    |                |                           |
|----|----------------|---------------------------|
| in | <i>handler</i> | Flash driver handler data |
| in | <i>mode</i>    | SMIF controller mode.     |

#### Return values

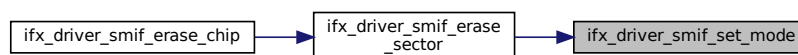
|                              |                                                          |
|------------------------------|----------------------------------------------------------|
| <i>ARM_DRIVER_OK</i>         | Memory block is inside of flash driver instance region.  |
| <i>ARM_DRIVER_ERROR_BUSY</i> | Memory block is outside of flash driver instance region. |

Definition at line 59 of file ifx\_driver\_smif.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.111.2.10 ifx\_driver\_smif\_uninitialize()

```
int32_t ifx_driver_smif_uninitialize (
 ifx_flash_driver_handler_t handler)
```

Definition at line 226 of file ifx\_driver\_smif.c.

**8.111.2.11 ifx\_driver\_smif\_validate\_region()**

```
int32_t ifx_driver_smif_validate_region (
 const ifx_driver_smif_obj_t * obj,
 uint32_t address,
 uint32_t size)
```

Validate that memory block is inside of flash driver instance region.

**Parameters**

|    |                |                               |
|----|----------------|-------------------------------|
| in | <i>obj</i>     | Flash driver instance data    |
| in | <i>address</i> | Start address of memory block |
| in | <i>size</i>    | Memory block size             |

**Return values**

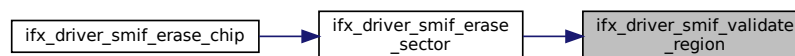
|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| <i>ARM_DRIVER_OK</i>              | Memory block is inside of flash driver instance region.  |
| <i>ARM_DRIVER_ERROR_PARAMETER</i> | Memory block is outside of flash driver instance region. |

**Note**

instance settings are not validated by this function!

Definition at line 76 of file ifx\_driver\_smif.c.

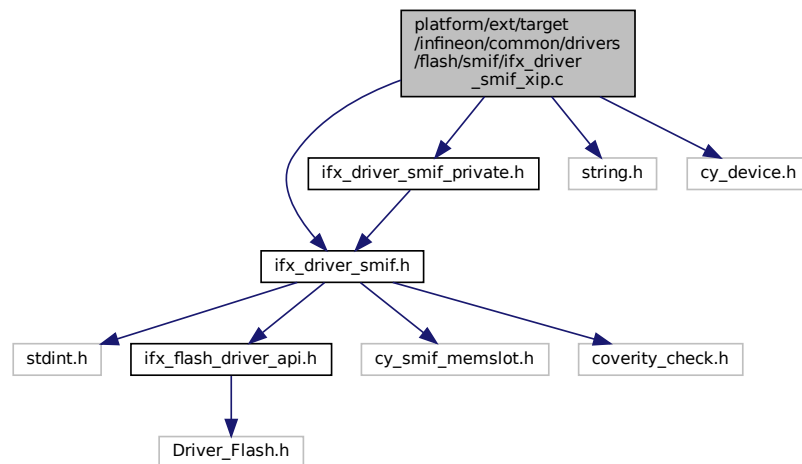
Here is the caller graph for this function:



## 8.112 platform/ext/target/infineon/common/drivers/flash/smif/ifx\_driver\_smif\_xip.c File Reference

```
#include "ifx_driver_smif.h"
#include "ifx_driver_smif_private.h"
#include <string.h>
#include "cy_device.h"
```

Include dependency graph for ifx\_driver\_smif\_xip.c:



## Variables

- const struct `ifx_flash_driver_t ifx_driver_smif_xip`

### 8.112.1 Variable Documentation

#### 8.112.1.1 ifx\_driver\_smif\_xip

```
const struct ifx_flash_driver_t ifx_driver_smif_xip
```

**Initial value:**

```
= {
 .GetVersion = ifx_driver_smif_get_version,
 .GetCapabilities = ifx_driver_smif_get_capabilities,
 .Initialize = ifx_driver_smif_initialize,
 .Uninitialize = ifx_driver_smif_uninitialize,
 .PowerControl = ifx_driver_smif_power_control,
 .ReadData = ifx_driver_smif_read_data,
 .ProgramData = ifx_driver_smif_program_data,
 .EraseSector = ifx_driver_smif_erase_sector,
 .EraseChip = ifx_driver_smif_erase_chip,
 .GetStatus = ifx_driver_smif_get_status,
 .GetInfo = ifx_driver_smif_get_info,
}
```

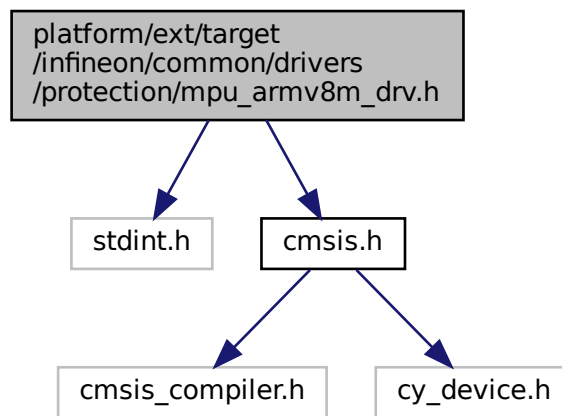
Definition at line 80 of file `ifx_driver_smif_xip.c`.

## 8.113 platform/ext/target/infineon/common/drivers/protection/mpu\_armv8m\_drv.h File Reference

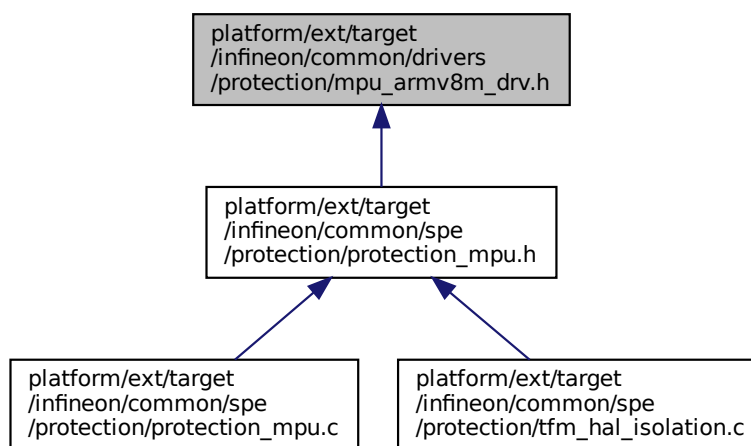
```
#include <stdint.h>
#include "cmsis.h"
```



Include dependency graph for mpu\_armv8m\_drv.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [mpu\\_armv8m\\_region\\_cfg\\_t](#)

## Macros

- #define [PRIVILEGED\\_DEFAULT\\_ENABLE](#) 1
- #define [HARDFULT\\_NMI\\_ENABLE](#) 1
- #define [MPU\\_ARMV8M\\_MAIR\\_ATTR\\_DEVICE\\_VAL](#) 0x04
- #define [MPU\\_ARMV8M\\_MAIR\\_ATTR\\_DEVICE\\_IDX](#) 0
- #define [MPU\\_ARMV8M\\_MAIR\\_ATTR\\_CODE\\_VAL](#) 0xAA

- `#define MPU_ARMV8M_MAIR_ATTR_CODE_IDX 1`
- `#define MPU_ARMV8M_MAIR_ATTR_DATA_VAL 0xFF`
- `#define MPU_ARMV8M_MAIR_ATTR_DATA_IDX 2`

## Enumerations

- enum `mpu_armv8m_attr_exec_t` { `MPU_ARMV8M_XN_EXEC_OK`, `MPU_ARMV8M_XN_EXEC_NEVER` }
- enum `mpu_armv8m_attr_access_t` { `MPU_ARMV8M_AP_RW_PRIV_ONLY`, `MPU_ARMV8M_AP_RW_PRIV_UNPRIV`, `MPU_ARMV8M_AP_RO_PRIV_ONLY`, `MPU_ARMV8M_AP_RO_PRIV_UNPRIV` }
- enum `mpu_armv8m_attr_shared_t` { `MPU_ARMV8M_SH_NONE`, `MPU_ARMV8M_SH_UNUSED`, `MPU_ARMV8M_SH_OUTER`, `MPU_ARMV8M_SH_INNER` }

## 8.113.1 Macro Definition Documentation

### 8.113.1.1 HARDFAULT\_NMI\_ENABLE

`#define HARDFAULT_NMI_ENABLE 1`

Definition at line 17 of file `mpu_armv8m_drv.h`.

### 8.113.1.2 MPU\_ARMV8M\_MAIR\_ATTR\_CODE\_IDX

`#define MPU_ARMV8M_MAIR_ATTR_CODE_IDX 1`

Definition at line 25 of file `mpu_armv8m_drv.h`.

### 8.113.1.3 MPU\_ARMV8M\_MAIR\_ATTR\_CODE\_VAL

`#define MPU_ARMV8M_MAIR_ATTR_CODE_VAL 0xAA`

Definition at line 24 of file `mpu_armv8m_drv.h`.

### 8.113.1.4 MPU\_ARMV8M\_MAIR\_ATTR\_DATA\_IDX

`#define MPU_ARMV8M_MAIR_ATTR_DATA_IDX 2`

Definition at line 28 of file `mpu_armv8m_drv.h`.

### 8.113.1.5 MPU\_ARMV8M\_MAIR\_ATTR\_DATA\_VAL

`#define MPU_ARMV8M_MAIR_ATTR_DATA_VAL 0xFF`

Definition at line 27 of file `mpu_armv8m_drv.h`.

### 8.113.1.6 MPU\_ARMV8M\_MAIR\_ATTR\_DEVICE\_IDX

`#define MPU_ARMV8M_MAIR_ATTR_DEVICE_IDX 0`

Definition at line 22 of file `mpu_armv8m_drv.h`.

### 8.113.1.7 MPU\_ARMV8M\_MAIR\_ATTR\_DEVICE\_VAL

`#define MPU_ARMV8M_MAIR_ATTR_DEVICE_VAL 0x04`

Definition at line 21 of file `mpu_armv8m_drv.h`.

**8.113.1.8 PRIVILEGED\_DEFAULT\_ENABLE**

```
#define PRIVILEGED_DEFAULT_ENABLE 1
```

Definition at line 16 of file mpu\_armv8m\_drv.h.

**8.113.2 Enumeration Type Documentation****8.113.2.1 mpu\_armv8m\_attr\_access\_t**

```
enum mpu_armv8m_attr_access_t
```

Enumerator

|                              |  |
|------------------------------|--|
| MPU_ARMV8M_AP_RW_PRIV_ONLY   |  |
| MPU_ARMV8M_AP_RW_PRIV_UNPRIV |  |
| MPU_ARMV8M_AP_RO_PRIV_ONLY   |  |
| MPU_ARMV8M_AP_RO_PRIV_UNPRIV |  |

Definition at line 35 of file mpu\_armv8m\_drv.h.

**8.113.2.2 mpu\_armv8m\_attr\_exec\_t**

```
enum mpu_armv8m_attr_exec_t
```

Enumerator

|                          |  |
|--------------------------|--|
| MPU_ARMV8M_XN_EXEC_OK    |  |
| MPU_ARMV8M_XN_EXEC_NEVER |  |

Definition at line 30 of file mpu\_armv8m\_drv.h.

**8.113.2.3 mpu\_armv8m\_attr\_shared\_t**

```
enum mpu_armv8m_attr_shared_t
```

Enumerator

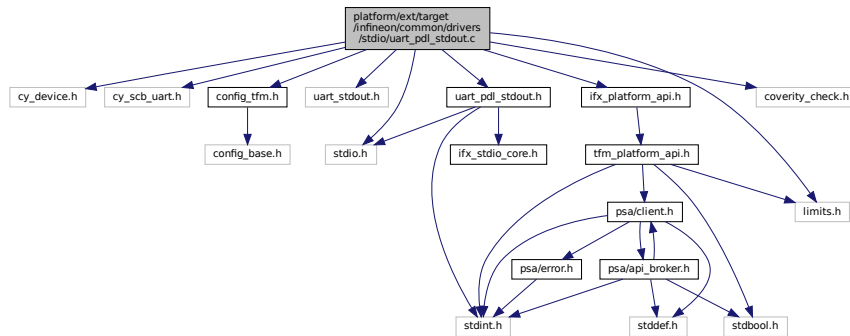
|                      |  |
|----------------------|--|
| MPU_ARMV8M_SH_NONE   |  |
| MPU_ARMV8M_SH_UNUSED |  |
| MPU_ARMV8M_SH_OUTER  |  |
| MPU_ARMV8M_SH_INNER  |  |

Definition at line 42 of file mpu\_armv8m\_drv.h.

**8.114 platform/ext/target/infineon/common/drivers/stdio/uart\_pdl\_stdout.c File Reference**

```
#include "cy_device.h"
#include "cy_scb_uart.h"
#include "config_tfm.h"
#include "uart_stdout.h"
#include "uart_pdl_stdout.h"
```

```
#include "coverity_check.h"
#include <stdio.h>
#include <limits.h>
#include "ifx_platform_api.h"
Include dependency graph for uart_pdl_stdout.c:
```



## Macros

- `#define IFX_UART_USE_SPM_LOG_MSG 1`

## Functions

- `int stdio_output_string (const char *str, uint32_t len)`
- `void stdio_init (void)`
- `void stdio_is_initialized_reset (void)`
- `bool stdio_is_initialized (void)`
- `void stdio_uninit (void)`

### 8.114.1 Macro Definition Documentation

#### 8.114.1.1 IFX\_UART\_USE\_SPM\_LOG\_MSG

```
#define IFX_UART_USE_SPM_LOG_MSG 1
Definition at line 50 of file uart_pdl_stdout.c.
```

### 8.114.2 Function Documentation

#### 8.114.2.1 stdio\_init()

```
void stdio_init (
 void)
```

Definition at line 300 of file uart\_pdl\_stdout.c.

Here is the caller graph for this function:



#### 8.114.2.2 `stdio_is_initialized()`

```
bool stdio_is_initialized (
 void)
```

Definition at line 323 of file `uart_pdl_stdout.c`.

Here is the caller graph for this function:



#### 8.114.2.3 `stdio_is_initialized_reset()`

```
void stdio_is_initialized_reset (
 void)
```

Definition at line 318 of file `uart_pdl_stdout.c`.

#### 8.114.2.4 `stdio_output_string()`

```
int stdio_output_string (
 const char * str,
 uint32_t len)
```

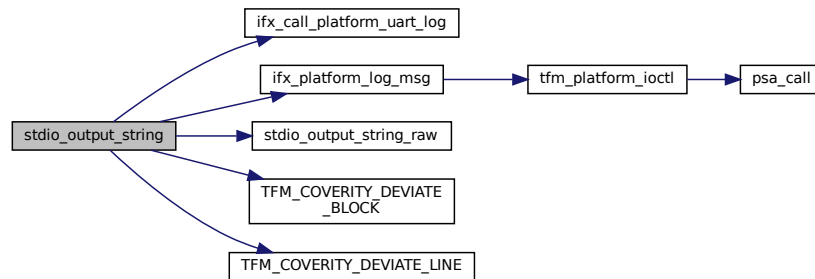
**Note**

Current implementation doesn't work if access to UART is done by multiple threads.

IFX\_PRINT\_CORE\_PREFIX optional configuration can be used to provide prefix to the messages. It's useful if system has only one serial port and there is output from multiple cores/images.

Definition at line 123 of file uart\_pdl\_stdout.c.

Here is the call graph for this function:

**8.114.2.5 stdio\_uninit()**

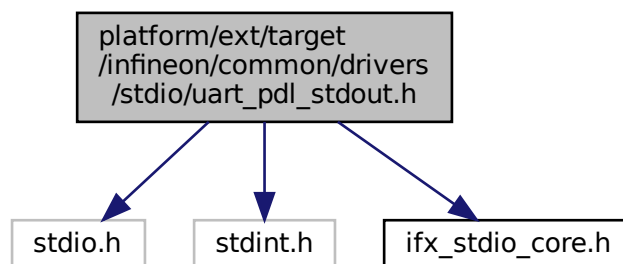
```
void stdio_uninit (
 void)
```

Definition at line 328 of file uart\_pdl\_stdout.c.

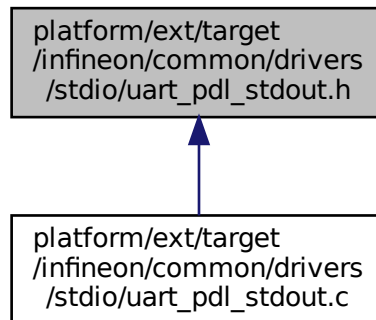
## 8.115 platform/ext/target/infineon/common/drivers/stdio/uart\_pdl\_stdout.h File Reference

```
#include <stdio.h>
#include <stdint.h>
#include "ifx_stdio_core.h"
```

Include dependency graph for uart\_pdl\_stdout.h:



This graph shows which files directly or indirectly include this file:



## Functions

- `int32_t stdio_output_string_raw` (const char \*str, uint32\_t len, [ifx\\_stdio\\_core\\_id\\_t](#) core\_id)  
Output log for SVC.

### 8.115.1 Function Documentation

#### 8.115.1.1 stdio\_output\_string\_raw()

```
int32_t stdio_output_string_raw (
 const char * str,
 uint32_t len,
 ifx_stdio_core_id_t core_id)
```

Output log for SVC.

[stdio\\_output\\_string\\_raw](#) is intended to be used by Platform service. [ifx\\_platform\\_log\\_msg](#) is a client API to print message through Platform service. core\_id is a core id Here is the caller graph for this function:

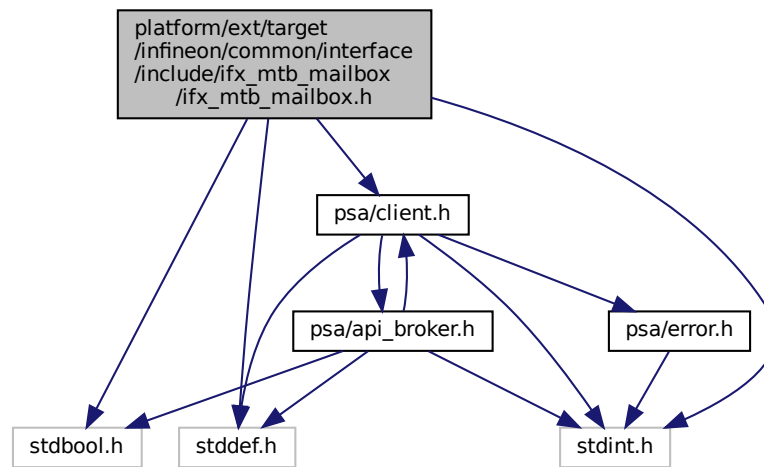


## 8.116 platform/ext/target/infineon/common/interface/include/ifx\_mtb\_mailbox/ifx\_mtb\_mailbox.h File Reference

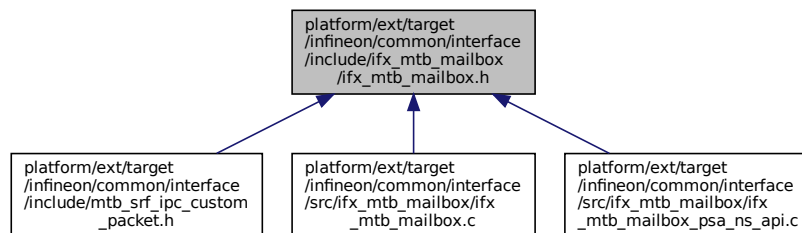
```
#include <stdbool.h>
#include <stdint.h>
#include <stddef.h>
```

```
#include "psa/client.h"
```

Include dependency graph for ifx\_mtb\_mailbox.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `ifx_psa_client_params_t`
- struct `ifx_mtb_mailbox_reply_t`

## Macros

- `#define IFX_MTB_MAILBOX_PSA_FRAMEWORK_VERSION (0x1)`
- `#define IFX_MTB_MAILBOX_PSA_VERSION (0x2)`
- `#define IFX_MTB_MAILBOX_PSA_CONNECT (0x3)`
- `#define IFX_MTB_MAILBOX_PSA_CALL (0x4)`
- `#define IFX_MTB_MAILBOX_PSA_CLOSE (0x5)`
- `#define IFX_MTB_MAILBOX_SUCCESS (0)`
- `#define IFX_MTB_MAILBOX_INVALID_PARAMS (INT32_MIN + 1)`
- `#define IFX_MTB_MAILBOX_GENERIC_ERROR (INT32_MIN + 2)`



## Functions

- `int32_t ifx_mtb_mailbox_client_call` (`uint32_t call_type`, `const ifx_psa_client_params_t *params`, `ifx_mtb_mailbox_reply_t *reply`)

*Send PSA client call to on core NSPE via MTB mailbox. Wait and fetch PSA client call result.*

## 8.116.1 Macro Definition Documentation

### 8.116.1.1 IFX\_MTB\_MAILBOX\_GENERIC\_ERROR

```
#define IFX_MTB_MAILBOX_GENERIC_ERROR (INT32_MIN + 2)
```

Definition at line 38 of file `ifx_mtb_mailbox.h`.

### 8.116.1.2 IFX\_MTB\_MAILBOX\_INVALID\_PARAMS

```
#define IFX_MTB_MAILBOX_INVALID_PARAMS (INT32_MIN + 1)
```

Definition at line 37 of file `ifx_mtb_mailbox.h`.

### 8.116.1.3 IFX\_MTB\_MAILBOX\_PSA\_CALL

```
#define IFX_MTB_MAILBOX_PSA_CALL (0x4)
```

Definition at line 32 of file `ifx_mtb_mailbox.h`.

### 8.116.1.4 IFX\_MTB\_MAILBOX\_PSA\_CLOSE

```
#define IFX_MTB_MAILBOX_PSA_CLOSE (0x5)
```

Definition at line 33 of file `ifx_mtb_mailbox.h`.

### 8.116.1.5 IFX\_MTB\_MAILBOX\_PSA\_CONNECT

```
#define IFX_MTB_MAILBOX_PSA_CONNECT (0x3)
```

Definition at line 31 of file `ifx_mtb_mailbox.h`.

### 8.116.1.6 IFX\_MTB\_MAILBOX\_PSA\_FRAMEWORK\_VERSION

```
#define IFX_MTB_MAILBOX_PSA_FRAMEWORK_VERSION (0x1)
```

Definition at line 29 of file `ifx_mtb_mailbox.h`.

### 8.116.1.7 IFX\_MTB\_MAILBOX\_PSA\_VERSION

```
#define IFX_MTB_MAILBOX_PSA_VERSION (0x2)
```

Definition at line 30 of file `ifx_mtb_mailbox.h`.

### 8.116.1.8 IFX\_MTB\_MAILBOX\_SUCCESS

```
#define IFX_MTB_MAILBOX_SUCCESS (0)
```

Definition at line 36 of file `ifx_mtb_mailbox.h`.

## 8.116.2 Function Documentation

### 8.116.2.1 ifx\_mtb\_mailbox\_client\_call()

```
int32_t ifx_mtb_mailbox_client_call (
 uint32_t call_type,
 const ifx_psa_client_params_t * params,
 ifx_mtb_mailbox_reply_t * reply)
```

Send PSA client call to on core NSPE via MTB mailbox. Wait and fetch PSA client call result.

#### Parameters

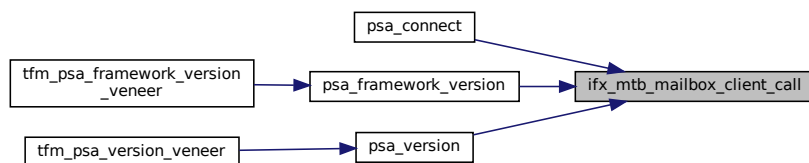
|     |                  |                                                 |
|-----|------------------|-------------------------------------------------|
| in  | <i>call_type</i> | PSA client call type                            |
| in  | <i>params</i>    | Parameters used for PSA client call             |
| out | <i>reply</i>     | The buffer written with PSA client call result. |

#### Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | The PSA client call is completed successfully.   |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 23 of file ifx\_mtb\_mailbox.c.

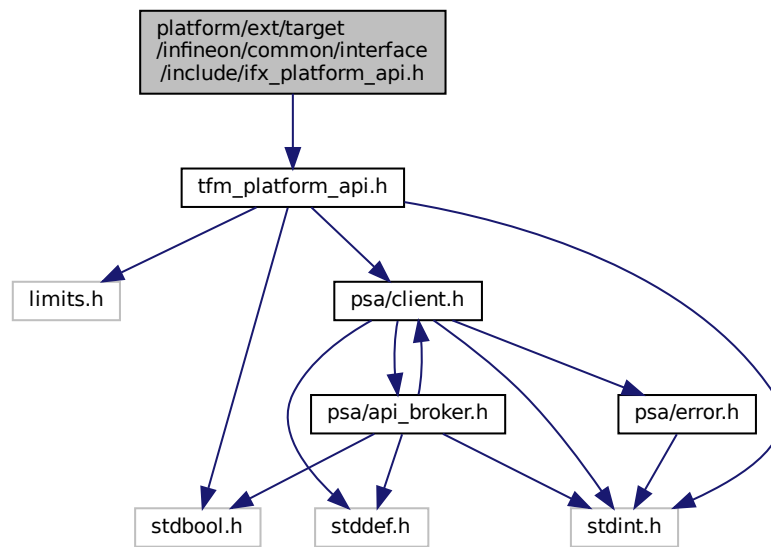
Here is the caller graph for this function:



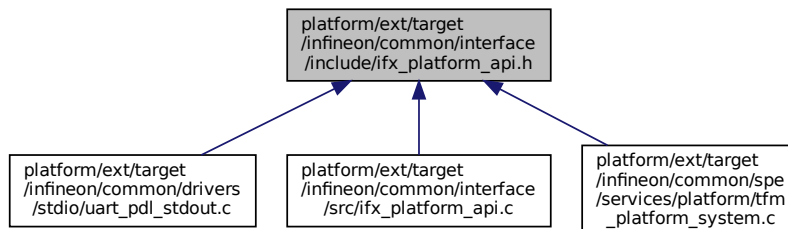
## 8.117 platform/ext/target/infineon/common/interface/include/ifx\_↵ platform\_api.h File Reference

```
#include "tfm_platform_api.h"
```

Include dependency graph for ifx\_platform\_api.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_PLATFORM_IOCTL_APP_MIN ((tfm_platform_ioctl_req_t)0x40000000)`  
*Start of application specific platform IOCTL request id.*
- `#define IFX_PLATFORM_IOCTL_APP_MAX ((tfm_platform_ioctl_req_t)0x7FFFFFFF)`  
*Start of application specific platform IOCTL request id.*

## Functions

- `uint32_t ifx_platform_log_msg (const char *msg, uint32_t msg_size)`  
*Write message via Platform service to SPM UART.*

### 8.117.1 Macro Definition Documentation

### 8.117.1.1 IFX\_PLATFORM\_IOCTL\_APP\_MAX

```
#define IFX_PLATFORM_IOCTL_APP_MAX ((tfm_platform_ioctl_req_t)0x7FFFFFFF)
```

Start of application specific platform IOCTL request id.

Definition at line 30 of file ifx\_platform\_api.h.

### 8.117.1.2 IFX\_PLATFORM\_IOCTL\_APP\_MIN

```
#define IFX_PLATFORM_IOCTL_APP_MIN ((tfm_platform_ioctl_req_t)0x40000000)
```

Start of application specific platform IOCTL request id.

Definition at line 26 of file ifx\_platform\_api.h.

## 8.117.2 Function Documentation

### 8.117.2.1 ifx\_platform\_log\_msg()

```
uint32_t ifx_platform_log_msg (
 const char * msg,
 uint32_t msg_size)
```

Write message via Platform service to SPM UART.

IFX\_PRINT\_CORE\_PREFIX optional configuration can be used to provide prefix to the messages. It's useful if system has only one serial port and there is output from multiple cores/images.

#### Parameters

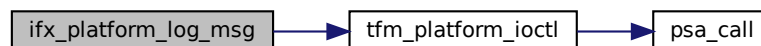
|    |                 |                          |
|----|-----------------|--------------------------|
| in | <i>msg</i>      | Message to write         |
| in | <i>msg_size</i> | Number of bytes to write |

#### Return values

|               |                   |
|---------------|-------------------|
| <i>Number</i> | of bytes written. |
|---------------|-------------------|

Definition at line 15 of file ifx\_platform\_api.c.

Here is the call graph for this function:



Here is the caller graph for this function:

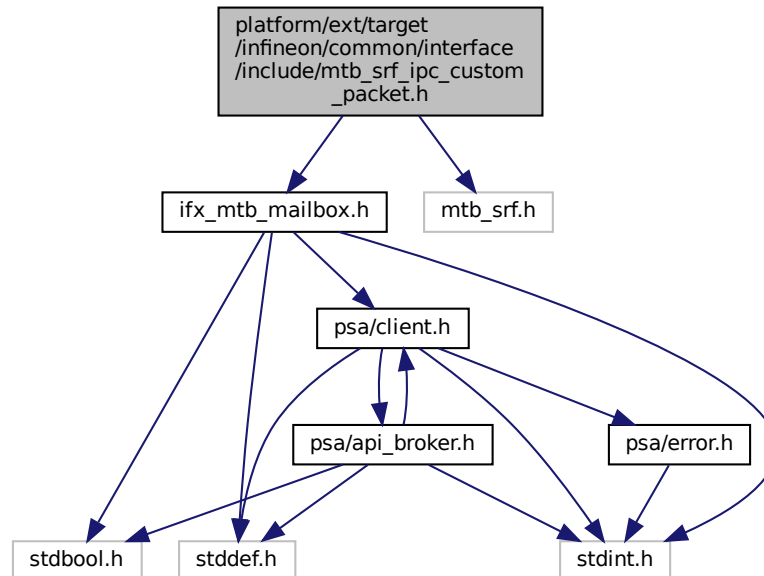


## 8.118 platform/ext/target/infineon/common/interface/include/mtb\_srf\_ipc\_custom\_packet.h File Reference

```
#include "ifx_mtb_mailbox.h"
```

```
#include "mtb_srf.h"
```

Include dependency graph for mtb\_srf\_ipc\_custom\_packet.h:



### Data Structures

- struct [\\_MTB\\_SRF\\_DATA\\_ALIGN](#)

*MTB SRF IPC request structure suitable for TFM needs. This structure overrides default `mtb_srf_ipc_packet_t` SRF structure.*

### Typedefs

- typedef struct [\\_MTB\\_SRF\\_DATA\\_ALIGN](#) `mtb_srf_ipc_packet_t`

*MTB SRF IPC request structure suitable for TFM needs. This structure overrides default `mtb_srf_ipc_packet_t` SRF structure.*

### 8.118.1 Typedef Documentation

#### 8.118.1.1 `mtb_srf_ipc_packet_t`

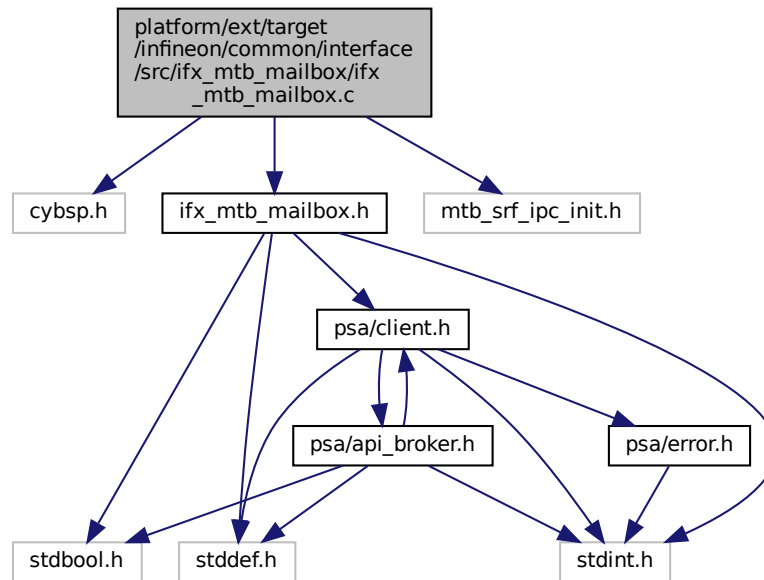
```
typedef struct _MTB_SRF_DATA_ALIGN mtb_srf_ipc_packet_t
```

MTB SRF IPC request structure suitable for TFM needs. This structure overrides default `mtb_srf_ipc_packet_t` SRF structure.

This structure is designed to be allocated in shared memory for inter-process communication (IPC). All members are stored as values (not pointers) to ensure the entire request is self-contained and can be safely copied into and out of shared memory. Parameters required for the PSA client call are copied directly into this structure, and the structure itself is allocated in the shared memory region. This design ensures that all data needed for the request is accessible to both communicating parties without relying on process-specific pointers/memory.

## 8.119 platform/ext/target/infineon/common/interface/src/ifx\_mtb\_mailbox/ifx\_mtb\_mailbox.c File Reference

```
#include "cybsp.h"
#include "ifx_mtb_mailbox.h"
#include "mtb_srf_ipc_init.h"
Include dependency graph for ifx_mtb_mailbox.c:
```



### Macros

- `#define IFX_MTB_MAILBOX_CACHE_PRESENT (0)`

### Functions

- `int32_t ifx_mtb_mailbox_client_call (uint32_t call_type, const ifx_psa_client_params_t *params, ifx_mtb_mailbox_reply_t *reply)`

*Send PSA client call to on core NSPE via MTB mailbox. Wait and fetch PSA client call result.*

### 8.119.1 Macro Definition Documentation

#### 8.119.1.1 IFX\_MTB\_MAILBOX\_CACHE\_PRESENT

```
#define IFX_MTB_MAILBOX_CACHE_PRESENT (0)
```

Definition at line 14 of file `ifx_mtb_mailbox.c`.

### 8.119.2 Function Documentation

### 8.119.2.1 ifx\_mtb\_mailbox\_client\_call()

```
int32_t ifx_mtb_mailbox_client_call (
 uint32_t call_type,
 const ifx_psa_client_params_t * params,
 ifx_mtb_mailbox_reply_t * reply)
```

Send PSA client call to on core NSPE via MTB mailbox. Wait and fetch PSA client call result.

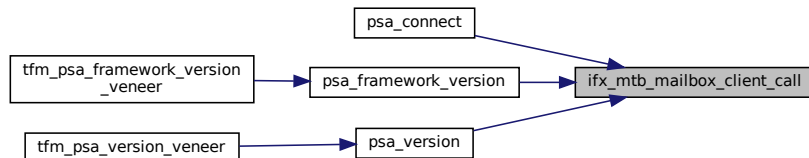
#### Parameters

|     |                  |                                                 |
|-----|------------------|-------------------------------------------------|
| in  | <i>call_type</i> | PSA client call type                            |
| in  | <i>params</i>    | Parameters used for PSA client call             |
| out | <i>reply</i>     | The buffer written with PSA client call result. |

#### Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | The PSA client call is completed successfully.   |
| <i>Other</i>           | return code Operation failed with an error code. |

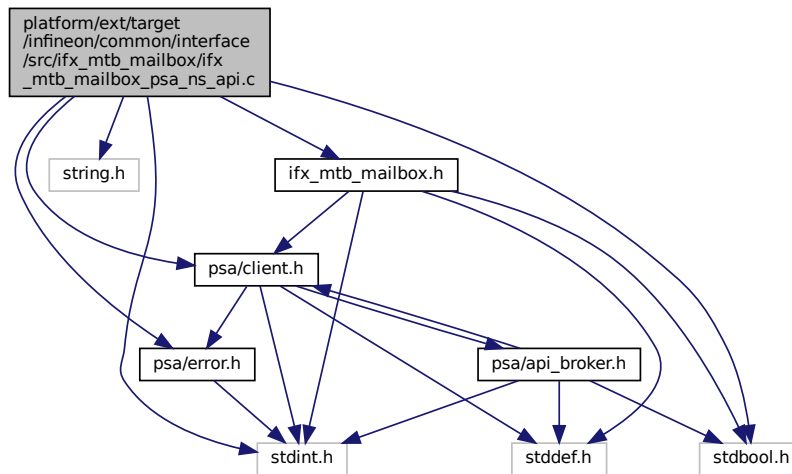
Definition at line 23 of file ifx\_mtb\_mailbox.c.  
Here is the caller graph for this function:



## 8.120 platform/ext/target/infineon/common/interface/src/ifx\_mtb\_mailbox/ifx\_mtb\_mailbox\_psa\_ns\_api.c File Reference

```
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include "ifx_mtb_mailbox.h"
#include "psa/client.h"
#include "psa/error.h"
```

Include dependency graph for ifx\_mtb\_mailbox\_psa\_ns\_api.c:



## Functions

- uint32\_t [psa\\_framework\\_version](#) (void)  
*Retrieve the version of the PSA Framework API that is implemented.*
- uint32\_t [psa\\_version](#) (uint32\_t sid)  
*Retrieve the version of an RoT Service or indicate that it is not present on this system.*
- [psa\\_handle\\_t](#) [psa\\_connect](#) (uint32\_t sid, uint32\_t version)  
*Connect to an RoT Service by its SID.*
- [psa\\_status\\_t](#) [psa\\_call](#) ([psa\\_handle\\_t](#) handle, int32\_t type, const [psa\\_invec](#) \*in\_vec, size\_t in\_len, [psa\\_outvec](#) \*out\_vec, size\_t out\_len)  
*Call an RoT Service on an established connection.*
- void [psa\\_close](#) ([psa\\_handle\\_t](#) handle)  
*Close a connection to an RoT Service.*

## 8.120.1 Function Documentation

### 8.120.1.1 [psa\\_call\(\)](#)

```

psa_status_t psa_call (
 psa_handle_t handle,
 int32_t type,
 const psa_invec * in_vec,
 size_t in_len,
 psa_outvec * out_vec,
 size_t out_len)

```

Call an RoT Service on an established connection.

#### Note

FF-M 1.0 proposes 6 parameters for [psa\\_call](#) but the secure gateway ABI support at most 4 parameters. T↔ F-M chooses to encode 'in\_len', 'out\_len', and 'type' into a 32-bit integer to improve efficiency. Compared with struct-based encoding, this method saves extra memory check and memory copy operation. The disadvantage is that the 'type' range has to be reduced into a 16-bit integer. So with this encoding, the valid range for 'type' is 0-32767.



#### Parameters

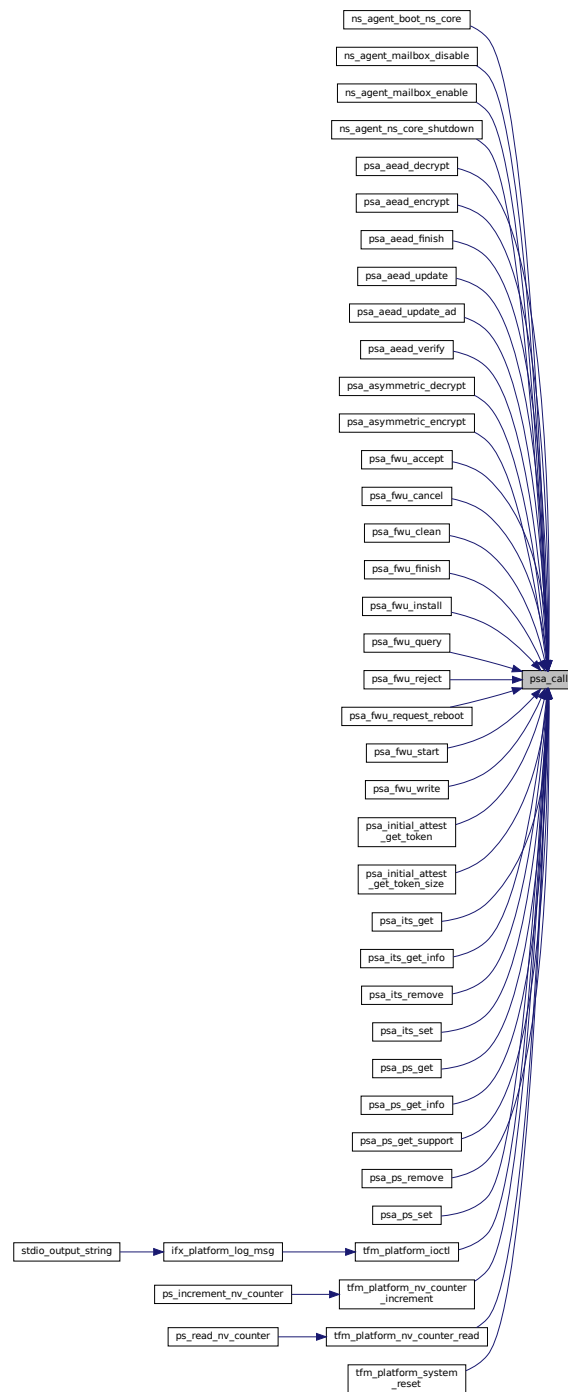
|         |                |                                                                             |
|---------|----------------|-----------------------------------------------------------------------------|
| in      | <i>handle</i>  | A handle to an established connection.                                      |
| in      | <i>type</i>    | The request type. Must be zero( <a href="#">PSA_IPC_CALL</a> ) or positive. |
| in      | <i>in_vec</i>  | Array of input <a href="#">psa_invec</a> structures.                        |
| in      | <i>in_len</i>  | Number of input <a href="#">psa_invec</a> structures.                       |
| in, out | <i>out_vec</i> | Array of output <a href="#">psa_outvec</a> structures.                      |
| in      | <i>out_len</i> | Number of output <a href="#">psa_outvec</a> structures.                     |

#### Return values

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\geq 0$                          | RoT Service-specific status value.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| $< 0$                             | RoT Service-specific error code.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>PSA_ERROR_PROGRAMMER_ERROR</i> | <p>The connection has been terminated by the RoT Service. The call is a PROGRAMMER ERROR if one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• An invalid handle was passed.</li> <li>• The connection is already handling a request.</li> <li>• <math>type &lt; 0</math>.</li> <li>• An invalid memory reference was provided.</li> <li>• <math>in\_len + out\_len &gt; PSA\_MAX\_IOVEC</math>.</li> <li>• The message is unrecognized by the RoT Service or incorrectly formatted.</li> </ul> |

Definition at line 79 of file ifx\_mtb\_mailbox\_psa\_ns\_api.c.

Here is the caller graph for this function:



### 8.120.1.2 `psa_close()`

```
void psa_close (
 psa_handle_t handle)
```

Close a connection to an RoT Service.

#### Parameters

|    |               |                                                            |
|----|---------------|------------------------------------------------------------|
| in | <i>handle</i> | A handle to an established connection, or the null handle. |
|----|---------------|------------------------------------------------------------|

#### Note

The call is a PROGRAMMER ERROR if one or more of the following occurs:

- An invalid handle was provided that is not the null handle.
- The connection is currently handling a request.

Definition at line 130 of file ifx\_mtb\_mailbox\_psa\_ns\_api.c.

#### 8.120.1.3 psa\_connect()

```
psa_handle_t psa_connect (
 uint32_t sid,
 uint32_t version)
```

Connect to an RoT Service by its SID.

#### Parameters

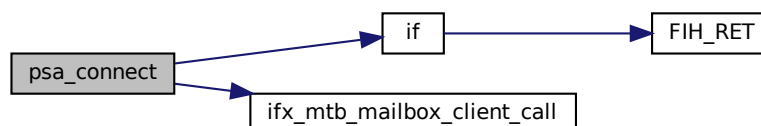
|    |                |                                       |
|----|----------------|---------------------------------------|
| in | <i>sid</i>     | ID of the RoT Service to connect to.  |
| in | <i>version</i> | Requested version of the RoT Service. |

#### Return values

|                                     |                                                                                                                                                                                                                                                                                             |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| >                                   | 0 A handle for the connection.                                                                                                                                                                                                                                                              |
| <i>PSA_ERROR_CONNECTION_REFUSED</i> | The SPM or RoT Service has refused the connection.                                                                                                                                                                                                                                          |
| <i>PSA_ERROR_CONNECTION_BUSY</i>    | The SPM or RoT Service cannot make the connection at the moment.                                                                                                                                                                                                                            |
| <i>PROGRAMMER ERROR</i>             | <p>The call is a PROGRAMMER ERROR if one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• The RoT Service ID is not present.</li> <li>• The RoT Service version is not supported.</li> <li>• The caller is not allowed to access the RoT service.</li> </ul> |

Definition at line 59 of file ifx\_mtb\_mailbox\_psa\_ns\_api.c.

Here is the call graph for this function:



#### 8.120.1.4 `psa_framework_version()`

```
uint32_t psa_framework_version (
 void)
```

Retrieve the version of the PSA Framework API that is implemented.

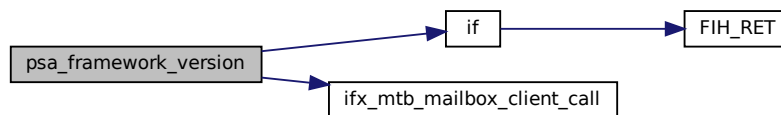
##### Returns

version The version of the PSA Framework implementation that is providing the runtime services to the caller. The major and minor version are encoded as follows:

- version[15:8] – major version number.
- version[7:0] – minor version number.

Definition at line 22 of file `ifx_mtb_mailbox_psa_ns_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.120.1.5 `psa_version()`

```
uint32_t psa_version (
 uint32_t sid)
```

Retrieve the version of an RoT Service or indicate that it is not present on this system.

##### Parameters

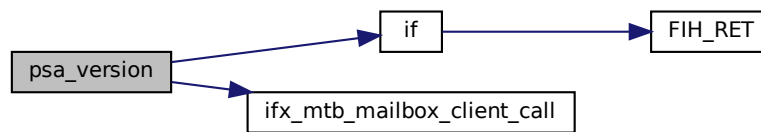
|    |            |                                 |
|----|------------|---------------------------------|
| in | <i>sid</i> | ID of the RoT Service to query. |
|----|------------|---------------------------------|

##### Return values

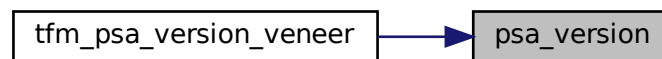
|                               |                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------|
| <code>PSA_VERSION_NONE</code> | The RoT Service is not implemented, or the caller is not permitted to access the service. |
| >                             | 0 The version of the implemented RoT Service.                                             |

Definition at line 40 of file `ifx_mtb_mailbox_psa_ns_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

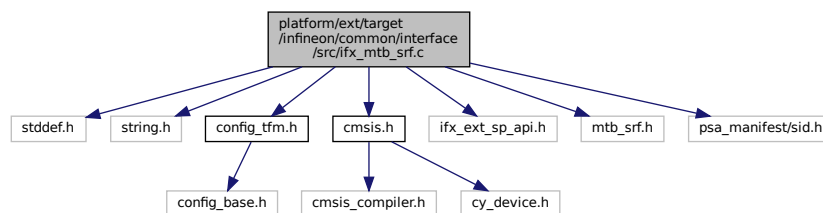


## 8.121 platform/ext/target/infineon/common/interface/src/ifx\_mtb\_srf.c File Reference

```

#include <stddef.h>
#include <string.h>
#include "config_tfm.h"
#include "cmsis.h"
#include "ifx_ext_sp_api.h"
#include "mtb_srf.h"
#include "psa_manifest/sid.h"
Include dependency graph for ifx_mtb_srf.c:

```



### Functions

- `cy_rslt_t ifx_mtb_srf_request_submit` (mtb\_srf\_invec\_ns\_t \*inVec\_ns, uint8\_t inVec\_cnt\_ns, mtb\_srf\_outvec\_ns\_t \*outVec\_ns, uint8\_t outVec\_cnt\_ns)

#### 8.121.1 Function Documentation

### 8.121.1.1 mtb\_srf\_request\_submit()

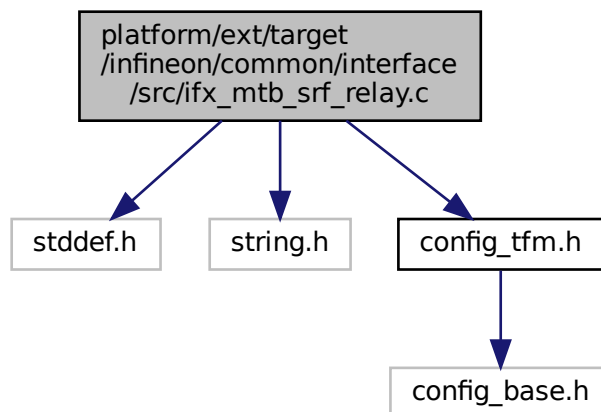
```
cy_rslt_t mtb_srf_request_submit (
 mtb_srf_invec_ns_t * inVec_ns,
 uint8_t inVec_cnt_ns,
 mtb_srf_outvec_ns_t * outVec_ns,
 uint8_t outVec_cnt_ns)
```

Definition at line 19 of file ifx\_mtb\_srf.c.

## 8.122 platform/ext/target/infineon/common/interface/src/ifx\_mtb\_srf\_relay.c File Reference

```
#include <stddef.h>
#include <string.h>
#include "config_tfm.h"
```

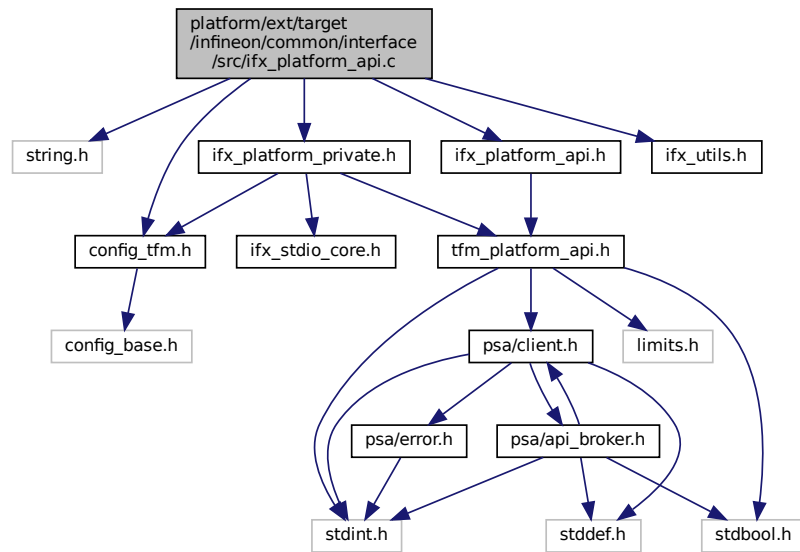
Include dependency graph for ifx\_mtb\_srf\_relay.c:



## 8.123 platform/ext/target/infineon/common/interface/src/ifx\_platform\_api.c File Reference

```
#include <string.h>
#include "config_tfm.h"
#include "ifx_platform_api.h"
#include "ifx_platform_private.h"
#include "ifx_utils.h"
```

Include dependency graph for ix\_platform\_api.c:



## Functions

- `uint32_t ix_platform_log_msg (const char *msg, uint32_t msg_size)`

*Write message via Platform service to SPM UART.*

### 8.123.1 Function Documentation

#### 8.123.1.1 ix\_platform\_log\_msg()

```
uint32_t ix_platform_log_msg (
 const char * msg,
 uint32_t msg_size)
```

Write message via Platform service to SPM UART.

IFX\_PRINT\_CORE\_PREFIX optional configuration can be used to provide prefix to the messages. It's useful if system has only one serial port and there is output from multiple cores/images.

#### Parameters

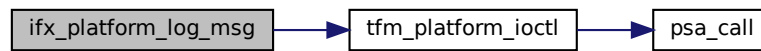
|    |                 |                          |
|----|-----------------|--------------------------|
| in | <i>msg</i>      | Message to write         |
| in | <i>msg_size</i> | Number of bytes to write |

#### Return values

|               |                   |
|---------------|-------------------|
| <i>Number</i> | of bytes written. |
|---------------|-------------------|

Definition at line 15 of file ix\_platform\_api.c.

Here is the call graph for this function:

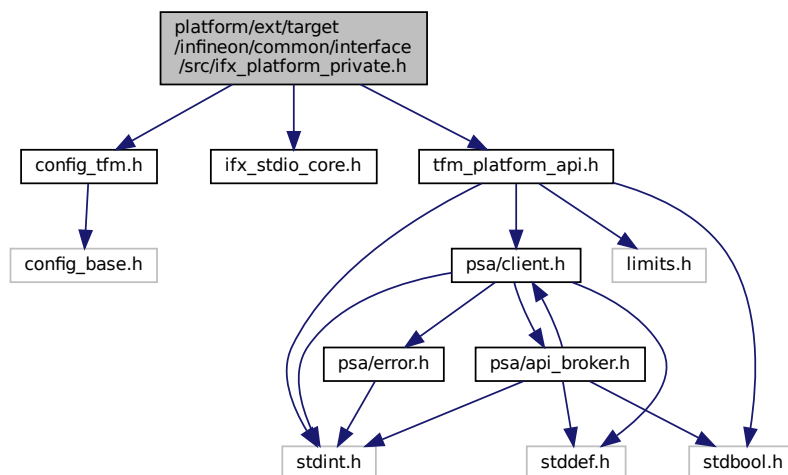


Here is the caller graph for this function:



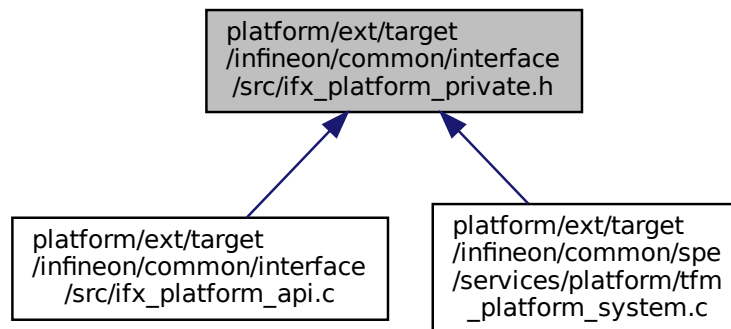
## 8.124 platform/ext/target/infineon/common/interface/src/ifx\_platform\_private.h File Reference

```
#include "config_tfm.h"
#include "ifx_stdio_core.h"
#include "tfm_platform_api.h"
Include dependency graph for ifx_platform_private.h:
```





This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_PLATFORM_IOCTL_LOG_MSG_MIN 0x00000000`  
*Log message to SPM UART for the first core id.*
- `#define IFX_PLATFORM_IOCTL_LOG_MSG_MAX (IFX_PLATFORM_IOCTL_LOG_MSG_MIN + (int32_t)IFX_STDIO_CORE_MAX_COUNT - 1)`  
*Log message to SPM UART for the last core id.*
- `#define IFX_CUSTOM_PLATFORM_HAL_IOCTL 0`

## 8.124.1 Macro Definition Documentation

### 8.124.1.1 IFX\_CUSTOM\_PLATFORM\_HAL\_IOCTL

```
#define IFX_CUSTOM_PLATFORM_HAL_IOCTL 0
```

Definition at line 40 of file `ix_platform_private.h`.

### 8.124.1.2 IFX\_PLATFORM\_IOCTL\_LOG\_MSG\_MAX

```
#define IFX_PLATFORM_IOCTL_LOG_MSG_MAX (IFX_PLATFORM_IOCTL_LOG_MSG_MIN + (int32_t)IFX_STDIO_CORE_MAX_COUNT - 1)
```

Log message to SPM UART for the last core id.  
Definition at line 37 of file `ix_platform_private.h`.

### 8.124.1.3 IFX\_PLATFORM\_IOCTL\_LOG\_MSG\_MIN

```
#define IFX_PLATFORM_IOCTL_LOG_MSG_MIN 0x00000000
```

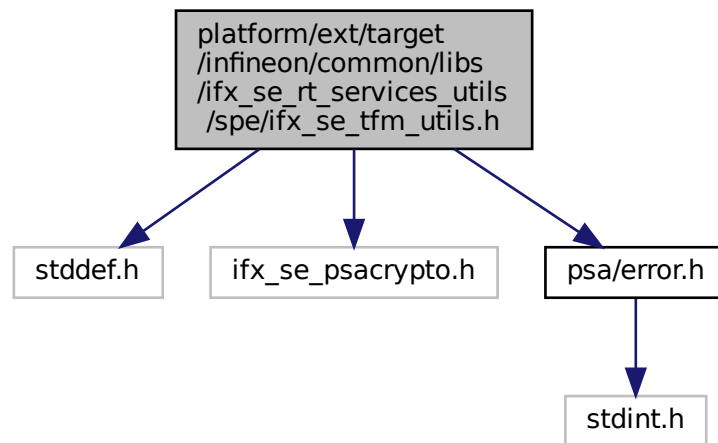
Log message to SPM UART for the first core id.  
Definition at line 32 of file `ix_platform_private.h`.

## 8.125 platform/ext/target/infineon/common/libs/ifx\_se\_rt\_services\_utils/spe/ifx\_se\_tfm\_utils.h File Reference

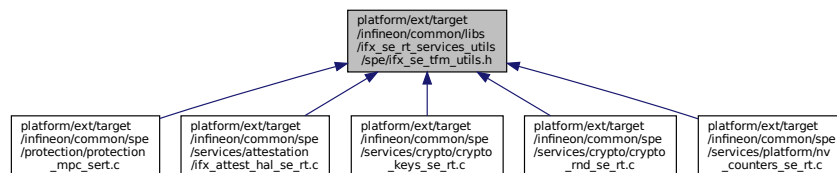
This file is a wrapper for ifx-se-rt-services-utils library for TF-M. It adds additional stuff needed to use this library from within secure partitions.

```
#include <stddef.h>
#include "ifx_se_psacrypto.h"
#include "psa/error.h"
```

Include dependency graph for ifx\_se\_tfm\_utils.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define IFX_SE_TFM_SYSCALL_CONTEXT NULL`  
Use this macro as context parameter to `ifx_se_*` functions.

### 8.125.1 Detailed Description

This file is a wrapper for ifx-se-rt-services-utils library for TF-M. It adds additional stuff needed to use this library from within secure partitions.

### 8.125.2 Macro Definition Documentation

### 8.125.2.1 IFX\_SE\_TFM\_SYSCALL\_CONTEXT

```
#define IFX_SE_TFM_SYSCALL_CONTEXT NULL
```

Use this macro as context parameter to ifx\_se\_\* functions.

Example:

```
ifx_se_generate_random(output, output_size, IFX_SE_TFM_SYSCALL_CONTEXT);
```

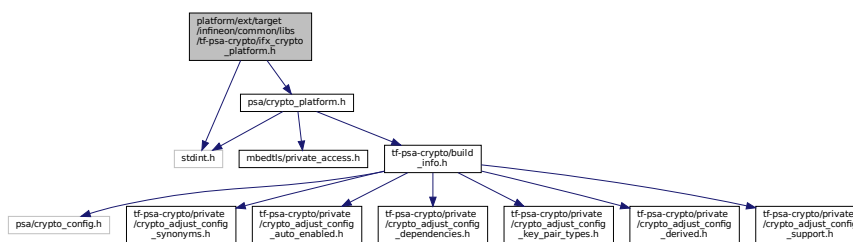
Definition at line 34 of file ifx\_se\_tfm\_utils.h.

## 8.126 platform/ext/target/infineon/common/libs/tf-psa-crypto/afx\_crypto\_platform.h File Reference

```
#include <stdint.h>
```

```
#include "psa/crypto_platform.h"
```

Include dependency graph for afx\_crypto\_platform.h:



## Macros

- #define [PSA\\_ALG\\_KDF\\_IFX\\_SE\\_AES\\_CMAC](#) ((psa\_algorithm\_t)0x08000600)
- #define [PSA\\_KEY\\_LOCATION\\_IFX\\_SE](#) ((psa\_key\_location\_t)0x800001)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_HUK\\_SLOT\\_NUMBER](#) (0u)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_OEM\\_ROT\\_SLOT\\_NUMBER](#) (1u)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_SERVICES\\_UPD\\_SLOT\\_NUMBER](#) (2u)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_IFX\\_ROT\\_SLOT\\_NUMBER](#) (3u)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_DEVICE\\_PRIV\\_SLOT\\_NUMBER](#) (4u)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_ATTEST\\_PRIV\\_SLOT\\_NUMBER](#) (5u)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_ATTEST\\_PUB\\_SLOT\\_NUMBER](#) (6u)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_HUK\\_KEY\\_ID](#) (MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN + PSA\_CRYPT0\_IFX\_SE\_HUK\_SLOT\_NUMBER)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_OEM\\_ROT\\_KEY\\_ID](#) (MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN + PSA\_CRYPT0\_IFX\_SE\_OEM\_ROT\_SLOT\_NUMBER)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_SERVICES\\_UPD\\_KEY\\_ID](#) (MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN + PSA\_CRYPT0\_IFX\_SE\_SERVICES\_UPD\_SLOT\_NUMBER)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_IFX\\_ROT\\_KEY\\_ID](#) (MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN + PSA\_CRYPT0\_IFX\_SE\_IFX\_ROT\_SLOT\_NUMBER)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_DEVICE\\_KEY\\_ID](#) (MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN + PSA\_CRYPT0\_IFX\_SE\_DEVICE\_PRIV\_SLOT\_NUMBER)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_ATTEST\\_PRIV\\_KEY\\_ID](#) (MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN + PSA\_CRYPT0\_IFX\_SE\_ATTEST\_PRIV\_SLOT\_NUMBER)
- #define [PSA\\_CRYPT0\\_IFX\\_SE\\_ATTEST\\_PUB\\_KEY\\_ID](#) (MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN + PSA\_CRYPT0\_IFX\_SE\_ATTEST\_PUB\_SLOT\_NUMBER)
- #define [PSA\\_IFX\\_SE\\_KEY\\_DERIVATION\\_MAX\\_CAPACITY](#) ((size\_t)0x0FFFFFFF)

### 8.126.1 Macro Definition Documentation

### 8.126.1.1 PSA\_ALG\_KDF\_IFX\_SE\_AES\_CMACH

```
#define PSA_ALG_KDF_IFX_SE_AES_CMACH ((psa_algorithm_t)0x08000600)
```

The CMACH KDF algorithm

Key derivation in Counter Mode using CMACH as pseudo random function (PRF) is defined in NIST SP 800-108 section 4.1.

This key derivation algorithm uses the following inputs

- [PSA\\_KEY\\_DERIVATION\\_INPUT\\_SECRET](#) is the key derivation key. It is a key that is used as an input to a key-derivation function (along with other input data) to derive keying material.
- [PSA\\_KEY\\_DERIVATION\\_INPUT\\_LABEL](#) is a string that identifies the purpose for the derived keying material, which is encoded as a bit string. The encoding method for the Label is defined in a larger context, for example, in the protocol that uses a KDF. This input is optional.
- [PSA\\_KEY\\_DERIVATION\\_INPUT\\_SEED](#) is a bit string that is used to seed the PRF. This input is optional.

Definition at line 48 of file ifx\_crypto\_platform.h.

### 8.126.1.2 PSA\_CRYPTCH\_IFX\_SE\_ATTEST\_PRIV\_KEY\_ID

```
#define PSA_CRYPTCH_IFX_SE_ATTEST_PRIV_KEY_ID (MBEDTLS_PSA_KEY_ID_BUILTIN_MIN + PSA_CRYPTCH_IFX_SE_ATTEST_PRIV_SLOT_NUMBER)
```

Definition at line 69 of file ifx\_crypto\_platform.h.

### 8.126.1.3 PSA\_CRYPTCH\_IFX\_SE\_ATTEST\_PRIV\_SLOT\_NUMBER

```
#define PSA_CRYPTCH_IFX_SE_ATTEST_PRIV_SLOT_NUMBER (5u)
```

Definition at line 60 of file ifx\_crypto\_platform.h.

### 8.126.1.4 PSA\_CRYPTCH\_IFX\_SE\_ATTEST\_PUB\_KEY\_ID

```
#define PSA_CRYPTCH_IFX_SE_ATTEST_PUB_KEY_ID (MBEDTLS_PSA_KEY_ID_BUILTIN_MIN + PSA_CRYPTCH_IFX_SE_ATTEST_PUB_SLOT_NUMBER)
```

Definition at line 70 of file ifx\_crypto\_platform.h.

### 8.126.1.5 PSA\_CRYPTCH\_IFX\_SE\_ATTEST\_PUB\_SLOT\_NUMBER

```
#define PSA_CRYPTCH_IFX_SE_ATTEST_PUB_SLOT_NUMBER (6u)
```

Definition at line 61 of file ifx\_crypto\_platform.h.

### 8.126.1.6 PSA\_CRYPTCH\_IFX\_SE\_DEVICE\_KEY\_ID

```
#define PSA_CRYPTCH_IFX_SE_DEVICE_KEY_ID (MBEDTLS_PSA_KEY_ID_BUILTIN_MIN + PSA_CRYPTCH_IFX_SE_DEVICE_PRIV_SLOT_NUMBER)
```

Definition at line 68 of file ifx\_crypto\_platform.h.

### 8.126.1.7 PSA\_CRYPTCH\_IFX\_SE\_DEVICE\_PRIV\_SLOT\_NUMBER

```
#define PSA_CRYPTCH_IFX_SE_DEVICE_PRIV_SLOT_NUMBER (4u)
```

Definition at line 59 of file ifx\_crypto\_platform.h.

### 8.126.1.8 PSA\_CRYPTCH\_IFX\_SE\_HUK\_KEY\_ID

```
#define PSA_CRYPTCH_IFX_SE_HUK_KEY_ID (MBEDTLS_PSA_KEY_ID_BUILTIN_MIN + PSA_CRYPTCH_IFX_SE_HUK_SLOT_NUMBER)
```

Definition at line 64 of file ifx\_crypto\_platform.h.

#### 8.126.1.9 PSA\_CRYPTO\_IFX\_SE\_HUK\_SLOT\_NUMBER

```
#define PSA_CRYPTO_IFX_SE_HUK_SLOT_NUMBER (0u)
```

Definition at line 55 of file ifx\_crypto\_platform.h.

#### 8.126.1.10 PSA\_CRYPTO\_IFX\_SE\_IFX\_ROT\_KEY\_ID

```
#define PSA_CRYPTO_IFX_SE_IFX_ROT_KEY_ID (MBEDTLS_PSA_KEY_ID_BUILTIN_MIN + PSA_CRYPTO_IFX_SE_IFX_ROT_SLOT_NUMBE
```

Definition at line 67 of file ifx\_crypto\_platform.h.

#### 8.126.1.11 PSA\_CRYPTO\_IFX\_SE\_IFX\_ROT\_SLOT\_NUMBER

```
#define PSA_CRYPTO_IFX_SE_IFX_ROT_SLOT_NUMBER (3u)
```

Definition at line 58 of file ifx\_crypto\_platform.h.

#### 8.126.1.12 PSA\_CRYPTO\_IFX\_SE\_OEM\_ROT\_KEY\_ID

```
#define PSA_CRYPTO_IFX_SE_OEM_ROT_KEY_ID (MBEDTLS_PSA_KEY_ID_BUILTIN_MIN + PSA_CRYPTO_IFX_SE_OEM_ROT_SLOT_NUMBE
```

Definition at line 65 of file ifx\_crypto\_platform.h.

#### 8.126.1.13 PSA\_CRYPTO\_IFX\_SE\_OEM\_ROT\_SLOT\_NUMBER

```
#define PSA_CRYPTO_IFX_SE_OEM_ROT_SLOT_NUMBER (1u)
```

Definition at line 56 of file ifx\_crypto\_platform.h.

#### 8.126.1.14 PSA\_CRYPTO\_IFX\_SE\_SERVICES\_UPD\_KEY\_ID

```
#define PSA_CRYPTO_IFX_SE_SERVICES_UPD_KEY_ID (MBEDTLS_PSA_KEY_ID_BUILTIN_MIN + PSA_CRYPTO_IFX_SE_SERVICES_UPD
```

Definition at line 66 of file ifx\_crypto\_platform.h.

#### 8.126.1.15 PSA\_CRYPTO\_IFX\_SE\_SERVICES\_UPD\_SLOT\_NUMBER

```
#define PSA_CRYPTO_IFX_SE_SERVICES_UPD_SLOT_NUMBER (2u)
```

Definition at line 57 of file ifx\_crypto\_platform.h.

#### 8.126.1.16 PSA\_IFX\_SE\_KEY\_DERIVATION\_MAX\_CAPACITY

```
#define PSA_IFX_SE_KEY_DERIVATION_MAX_CAPACITY ((size_t)0xFFFFFFFF)
```

Maximum possible number of bytes a key derivation operation can output.

This number is derived from the maximum number of bits which can be represented within IFX\_SCA\_KEY\_DERIVATION\_CAPACITY\_LENGTH bytes.

Definition at line 78 of file ifx\_crypto\_platform.h.

#### 8.126.1.17 PSA\_KEY\_LOCATION\_IFX\_SE

```
#define PSA_KEY_LOCATION_IFX_SE ((psa_key_location_t)0x800001)
```

The storage area located inside IFX SE Runtime Services

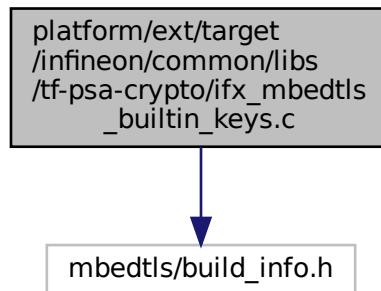
Definition at line 51 of file ifx\_crypto\_platform.h.

## 8.127 platform/ext/target/infineon/common/libs/tf-psa-crypto/ifx\_mbedtls\_builtin\_keys.c File Reference

mbedtls IFX builtin keys implementation

```
#include "mbedtls/build_info.h"
```

Include dependency graph for ifx\_mbedtls\_builtin\_keys.c:



### 8.127.1 Detailed Description

mbedtls IFX builtin keys implementation

Note

#### Copyright

(c) 2022-2026, Infineon Technologies AG, or an affiliate of Infineon Technologies AG. All rights reserved. This software, associated documentation and materials ("Software") is owned by Infineon Technologies AG or one of its affiliates ("Infineon") and is protected by and subject to worldwide patent protection, worldwide copyright laws, and international treaty provisions. Therefore, you may use this Software only as provided in the license agreement accompanying the software package from which you obtained this Software. If no license agreement applies, then any use, reproduction, modification, translation, or compilation of this Software is prohibited without the express written permission of Infineon.

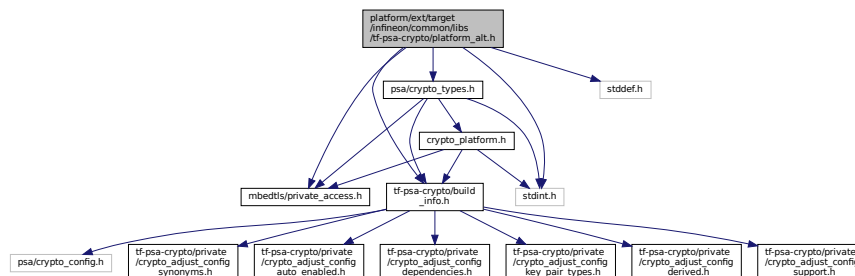
Disclaimer: UNLESS OTHERWISE EXPRESSLY AGREED WITH INFINEON, THIS SOFTWARE IS PROVIDED AS-IS, WITH NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, ALL WARRANTIES OF NON-INFRINGEMENT OF THIRD-PARTY RIGHTS AND IMPLIED WARRANTIES SUCH AS WARRANTIES OF FITNESS FOR A SPECIFIC USE/PURPOSE OR MERCHANTABILITY. Infineon reserves the right to make changes to the Software without notice. You are responsible for properly designing, programming, and testing the functionality and safety of your intended application of the Software, as well as complying with any legal requirements related to its use. Infineon does not guarantee that the Software will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches. Unless otherwise explicitly approved by Infineon, the Software may not be used in any application where a failure of the Product or any consequences of the use thereof can reasonably be expected to result in personal injury.

## 8.128 platform/ext/target/infineon/common/libs/tf-psa-crypto/platform\_alt.h File Reference

This file contains the definitions and functions of the IFX Mbed TLS platform abstraction layer.

```
#include "mbedtls/private_access.h"
#include "tf-psa-crypto/build_info.h"
#include <stddef.h>
#include <stdint.h>
#include "psa/crypto_types.h"
```

Include dependency graph for platform\_alt.h:



## Data Structures

- struct [mbedtls\\_platform\\_context](#)  
The platform context structure.

## Macros

- #define [MBEDTLS\\_PSA\\_KEY\\_ID\\_BUILTIN\\_MIN](#) ((psa\_key\_id\_t) 0x7fff0000)

## Typedefs

- typedef struct [mbedtls\\_platform\\_context](#) [mbedtls\\_platform\\_context](#)  
The platform context structure.

### 8.128.1 Detailed Description

This file contains the definitions and functions of the IFX Mbed TLS platform abstraction layer.

#### Copyright

(c) 2022-2026, Infineon Technologies AG, or an affiliate of Infineon Technologies AG. All rights reserved. This software, associated documentation and materials ("Software") is owned by Infineon Technologies AG or one of its affiliates ("Infineon") and is protected by and subject to worldwide patent protection, worldwide copyright laws, and international treaty provisions. Therefore, you may use this Software only as provided in the license agreement accompanying the software package from which you obtained this Software. If no license agreement applies, then any use, reproduction, modification, translation, or compilation of this Software is prohibited without the express written permission of Infineon.

Disclaimer: UNLESS OTHERWISE EXPRESSLY AGREED WITH INFINEON, THIS SOFTWARE IS PROVIDED AS-IS, WITH NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, ALL WARRANTIES OF NON-INFRINGEMENT OF THIRD-PARTY RIGHTS AND IMPLIED WARRANTIES SUCH AS WARRANTIES OF FITNESS FOR A SPECIFIC USE/PURPOSE OR MERCHANTABILITY. Infineon reserves the right to make changes to the Software without notice. You are responsible for properly designing, programming, and testing the functionality and safety of your intended application of the Software, as well as complying with any legal requirements related to its use. Infineon does not guarantee that the Software will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches. Unless otherwise explicitly approved by Infineon, the Software may not be used in any application where a failure of the Product or any consequences of the use thereof can reasonably be expected to result in personal injury.

## 8.128.2 Macro Definition Documentation

### 8.128.2.1 MBEDTLS\_PSA\_KEY\_ID\_BUILTIN\_MIN

```
#define MBEDTLS_PSA_KEY_ID_BUILTIN_MIN ((psa_key_id_t) 0x7fff0000)
```

Definition at line 51 of file platform\_alt.h.

## 8.128.3 Typedef Documentation

### 8.128.3.1 mbedtls\_platform\_context

```
typedef struct mbedtls_platform_context mbedtls_platform_context
```

The platform context structure.

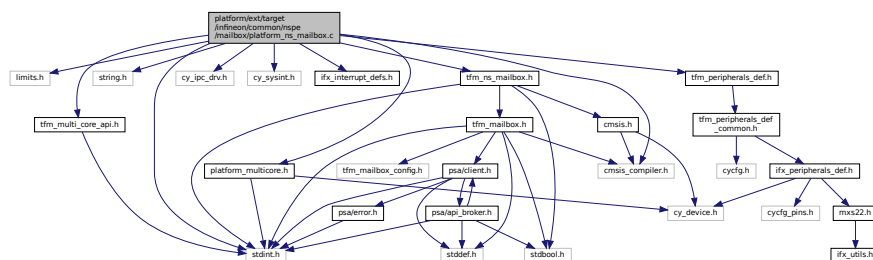
#### Note

This structure may be used to assist platform-specific setup or teardown operations.

## 8.129 platform/ext/target/infineon/common/nspe/mailbox/platform\_ns\_mailbox.c File Reference

```
#include <limits.h>
#include <stdint.h>
#include <string.h>
#include "cmsis_compiler.h"
#include "cy_ipc_drv.h"
#include "cy_sysint.h"
#include "ifx_interrupt_defs.h"
#include "tfm_multi_core_api.h"
#include "tfm_ns_mailbox.h"
#include "tfm_peripherals_def.h"
#include "platform_multicore.h"
```

Include dependency graph for platform\_ns\_mailbox.c:



## Functions

- `int32_t tfm_ns_multi_core_boot (struct ns_mailbox_queue_t *queue)`  
Performs multicore boot sequence in NSPE.
- `int32_t tfm_platform_ns_wait_for_s_cpu_ready (void)`  
Synchronisation with secure CPU, platform-specific implementation. Flags that the non-secure side has completed its initialization. Waits, if necessary, for the secure CPU to flag that it has completed its initialization.
- `void IFX_IRQ_NAME_TO_HANDLER() IFX_IPC_TFM_TO_NS_IPC_INTR (void)`



- `int32_t tfm_ns_mailbox_hal_notify_peer (void)`  
*Notify SPE to deal with the PSA client call sent via mailbox.*
- `int32_t tfm_ns_mailbox_hal_init (struct ns_mailbox_queue_t *queue)`  
*Platform specific NSPE mailbox initialization. Invoked by `tfm_ns_mailbox_init()`.*
- `uint32_t tfm_ns_mailbox_hal_enter_critical (void)`  
*Enter critical section of NSPE mailbox.*
- `void tfm_ns_mailbox_hal_exit_critical (uint32_t state)`  
*Exit critical section of NSPE mailbox.*
- `uint32_t tfm_ns_mailbox_hal_enter_critical_isr (void)`  
*Enter critical section of NSPE mailbox in IRQ handler.*
- `void tfm_ns_mailbox_hal_exit_critical_isr (uint32_t state)`  
*Enter critical section of NSPE mailbox in IRQ handler.*

## 8.129.1 Function Documentation

### 8.129.1.1 IFX\_IPC\_TFM\_TO\_NS\_IPC\_INTR()

```
void IFX_IRQ_NAME_TO_HANDLER() IFX_IPC_TFM_TO_NS_IPC_INTR (
 void)
```

Definition at line 101 of file platform\_ns\_mailbox.c.

### 8.129.1.2 tfm\_ns\_mailbox\_hal\_enter\_critical()

```
uint32_t tfm_ns_mailbox_hal_enter_critical (
 void)
```

Enter critical section of NSPE mailbox.

Is used to lock data shared between cores running secure and non-secure side. Should not be used to lock non-secure data that are not shared with secure core.

#### Note

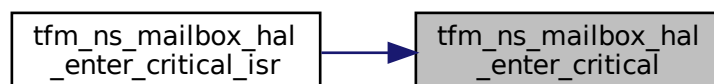
The implementation depends on platform specific hardware and use case.

#### Returns

Platform specific critical section state. Use it to exit from critical section by passing result to `tfm_ns_mailbox_hal_exit_critical`.

Definition at line 187 of file platform\_ns\_mailbox.c.

Here is the caller graph for this function:



### 8.129.1.3 tfm\_ns\_mailbox\_hal\_enter\_critical\_isr()

```
uint32_t tfm_ns_mailbox_hal_enter_critical_isr (
 void)
```

Enter critical section of NSPE mailbox in IRQ handler.

Is used to lock data shared between cores running secure and non-secure side. Should not be used to lock non-secure data that are not shared with secure core.

#### Note

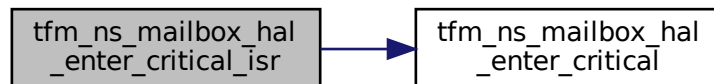
The implementation depends on platform specific hardware and use case.

#### Returns

Platform specific critical section state. Use it to exit from critical section by passing result to [tfm\\_ns\\_mailbox\\_hal\\_exit\\_critical\\_isr](#).

Definition at line 216 of file platform\_ns\_mailbox.c.

Here is the call graph for this function:



### 8.129.1.4 tfm\_ns\_mailbox\_hal\_exit\_critical()

```
void tfm_ns_mailbox_hal_exit_critical (
 uint32_t state)
```

Exit critical section of NSPE mailbox.

#### Note

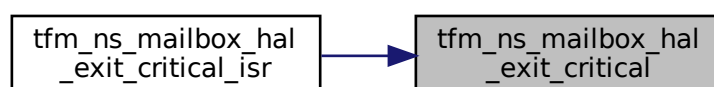
The implementation depends on platform specific hardware and use case.

#### Parameters

|    |       |                                                                                        |
|----|-------|----------------------------------------------------------------------------------------|
| in | state | Critical section state returned by <a href="#">tfm_ns_mailbox_hal_enter_critical</a> . |
|----|-------|----------------------------------------------------------------------------------------|

Definition at line 206 of file platform\_ns\_mailbox.c.

Here is the caller graph for this function:



**8.129.1.5 tfm\_ns\_mailbox\_hal\_exit\_critical\_isr()**

```
void tfm_ns_mailbox_hal_exit_critical_isr (
 uint32_t state)
```

Enter critical section of NSPE mailbox in IRQ handler.

**Note**

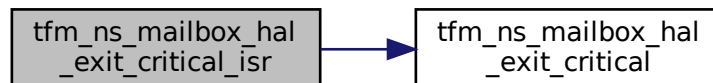
The implementation depends on platform specific hardware and use case.

**Parameters**

|           |              |                                                                                            |
|-----------|--------------|--------------------------------------------------------------------------------------------|
| <i>in</i> | <i>state</i> | Critical section state returned by <a href="#">tfm_ns_mailbox_hal_enter_critical_isr</a> . |
|-----------|--------------|--------------------------------------------------------------------------------------------|

Definition at line 225 of file platform\_ns\_mailbox.c.

Here is the call graph for this function:

**8.129.1.6 tfm\_ns\_mailbox\_hal\_init()**

```
int32_t tfm_ns_mailbox_hal_init (
 struct ns_mailbox_queue_t * queue)
```

Platform specific NSPE mailbox initialization. Invoked by [tfm\\_ns\\_mailbox\\_init](#)).

**Parameters**

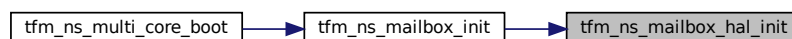
|           |              |                                                           |
|-----------|--------------|-----------------------------------------------------------|
| <i>in</i> | <i>queue</i> | The base address of NSPE mailbox queue to be initialized. |
|-----------|--------------|-----------------------------------------------------------|

**Return values**

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | Operation succeeded.                             |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 133 of file platform\_ns\_mailbox.c.

Here is the caller graph for this function:



### 8.129.1.7 tfm\_ns\_mailbox\_hal\_notify\_peer()

```
int32_t tfm_ns_mailbox_hal_notify_peer (
 void)
```

Notify SPE to deal with the PSA client call sent via mailbox.

#### Note

The implementation depends on platform specific hardware and use case.

#### Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | Operation succeeded.                             |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 118 of file platform\_ns\_mailbox.c.

### 8.129.1.8 tfm\_ns\_multi\_core\_boot()

```
int32_t tfm_ns_multi_core_boot (
 struct ns_mailbox_queue_t * queue)
```

Performs multicore boot sequence in NSPE.

#### Parameters

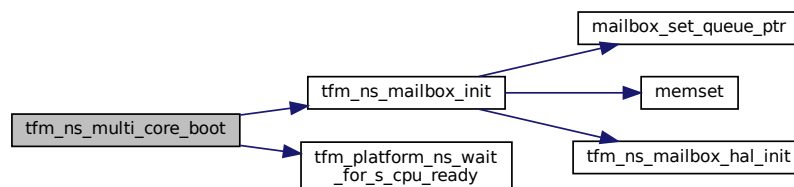
|    |              |                                                           |
|----|--------------|-----------------------------------------------------------|
| in | <i>queue</i> | The base address of NSPE mailbox queue to be initialized. |
|----|--------------|-----------------------------------------------------------|

#### Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | Operation succeeded.                             |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 26 of file platform\_ns\_mailbox.c.

Here is the call graph for this function:



### 8.129.1.9 tfm\_platform\_ns\_wait\_for\_s\_cpu\_ready()

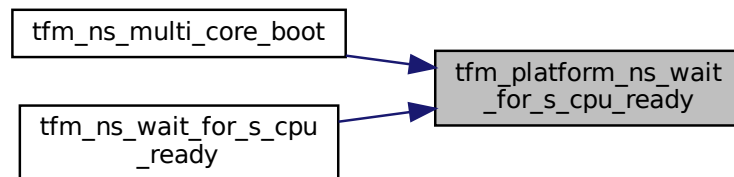
```
int32_t tfm_platform_ns_wait_for_s_cpu_ready (
 void)
```

Synchronisation with secure CPU, platform-specific implementation. Flags that the non-secure side has completed its initialization. Waits, if necessary, for the secure CPU to flag that it has completed its initialization.

#### Return values

|                          |                      |
|--------------------------|----------------------|
| <i>Return</i>            | 0 if succeeds.       |
| <i>Otherwise, return</i> | specific error code. |

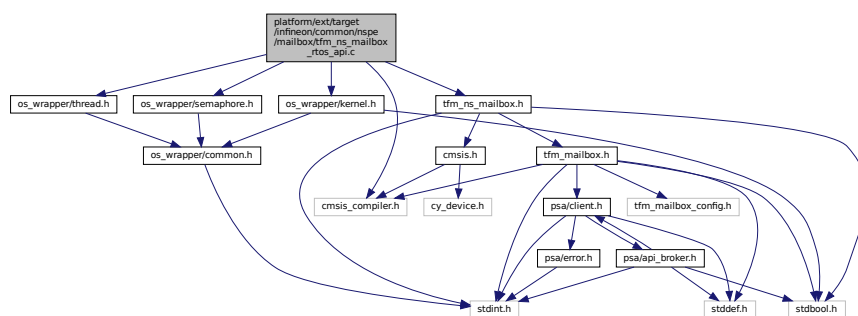
Definition at line 82 of file platform\_ns\_mailbox.c.  
Here is the caller graph for this function:



## 8.130 platform/ext/target/infineon/common/nspe/mailbox/tfm\_ns\_mailbox\_rtos\_api.c File Reference

```
#include "cmsis_compiler.h"
#include "os_wrapper/semaphore.h"
#include "os_wrapper/kernel.h"
#include "os_wrapper/thread.h"
#include "tfm_ns_mailbox.h"
```

Include dependency graph for `tfm_ns_mailbox_rtos_api.c`:



## Macros

- `#define MAILBOX_THREAD_FLAG 0x5FCA0000`
- `#define MAX_SEMAPHORE_COUNT NUM_MAILBOX_QUEUE_SLOT`

## Functions

- `const void * tfm_ns_mailbox_os_get_task_handle (void)`

- `void tfm_ns_mailbox_os_wait_reply (void)`
- `void tfm_ns_mailbox_os_wake_task_isr (const void *task_handle)`
- `void tfm_ns_mailbox_os_spin_lock (void)`
- `void tfm_ns_mailbox_os_spin_unlock (void)`
- `int32_t tfm_ns_mailbox_os_lock_init (void)`
- `int32_t tfm_ns_mailbox_os_lock_acquire (void)`
- `int32_t tfm_ns_mailbox_os_lock_release (void)`

## 8.130.1 Macro Definition Documentation

### 8.130.1.1 MAILBOX\_THREAD\_FLAG

```
#define MAILBOX_THREAD_FLAG 0x5FCA0000
```

Definition at line 34 of file `tfm_ns_mailbox_rtos_api.c`.

### 8.130.1.2 MAX\_SEMAPHORE\_COUNT

```
#define MAX_SEMAPHORE_COUNT NUM_MAILBOX_QUEUE_SLOT
```

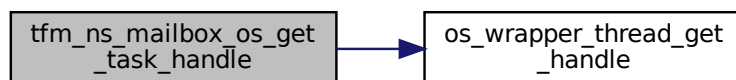
Definition at line 37 of file `tfm_ns_mailbox_rtos_api.c`.

## 8.130.2 Function Documentation

### 8.130.2.1 tfm\_ns\_mailbox\_os\_get\_task\_handle()

```
const void* tfm_ns_mailbox_os_get_task_handle (
 void)
```

Definition at line 42 of file `tfm_ns_mailbox_rtos_api.c`.  
Here is the call graph for this function:

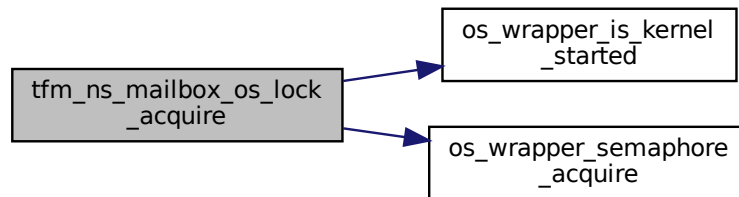


### 8.130.2.2 tfm\_ns\_mailbox\_os\_lock\_acquire()

```
int32_t tfm_ns_mailbox_os_lock_acquire (
 void)
```

Definition at line 146 of file `tfm_ns_mailbox_rtos_api.c`.

Here is the call graph for this function:



#### 8.130.2.3 `tfm_ns_mailbox_os_lock_init()`

```
int32_t tfm_ns_mailbox_os_lock_init (
 void)
```

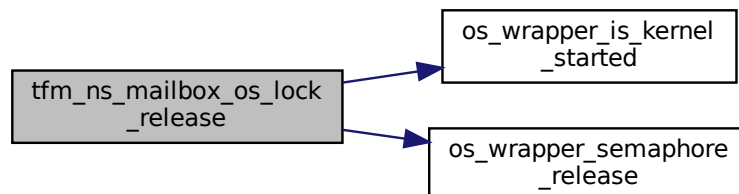
Definition at line 134 of file `tfm_ns_mailbox_rtos_api.c`.

#### 8.130.2.4 `tfm_ns_mailbox_os_lock_release()`

```
int32_t tfm_ns_mailbox_os_lock_release (
 void)
```

Definition at line 156 of file `tfm_ns_mailbox_rtos_api.c`.

Here is the call graph for this function:



#### 8.130.2.5 `tfm_ns_mailbox_os_spin_lock()`

```
void tfm_ns_mailbox_os_spin_lock (
 void)
```

Definition at line 78 of file `tfm_ns_mailbox_rtos_api.c`.

#### 8.130.2.6 `tfm_ns_mailbox_os_spin_unlock()`

```
void tfm_ns_mailbox_os_spin_unlock (
 void)
```

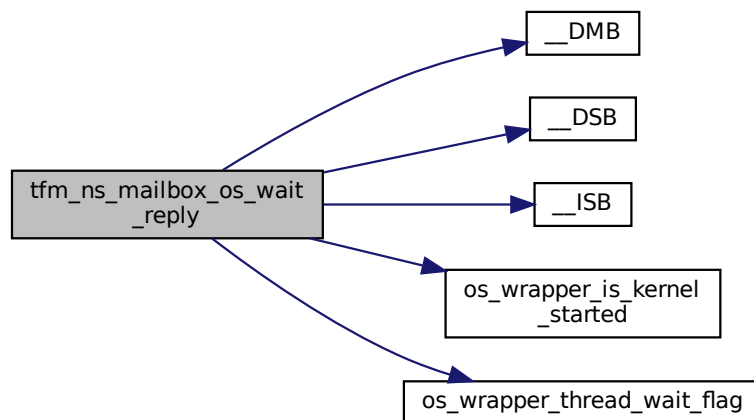
Definition at line 87 of file tfm\_ns\_mailbox\_rtos\_api.c.

### 8.130.2.7 tfm\_ns\_mailbox\_os\_wait\_reply()

```
void tfm_ns_mailbox_os_wait_reply (
 void)
```

Definition at line 47 of file tfm\_ns\_mailbox\_rtos\_api.c.

Here is the call graph for this function:

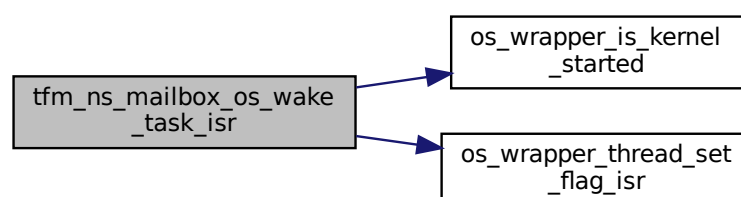


### 8.130.2.8 tfm\_ns\_mailbox\_os\_wake\_task\_isr()

```
void tfm_ns_mailbox_os_wake_task_isr (
 const void * task_handle)
```

Definition at line 65 of file tfm\_ns\_mailbox\_rtos\_api.c.

Here is the call graph for this function:



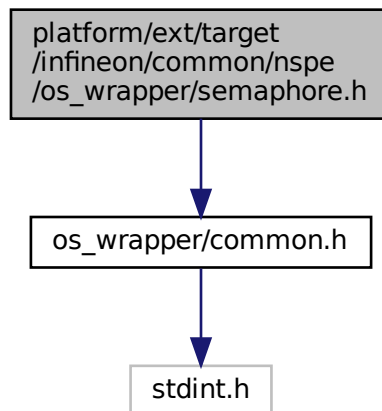


## 8.131 platform/ext/target/infineon/common/nspe/os\_wrapper/os\_wrapper\_cyabs\_rtos.c File Reference

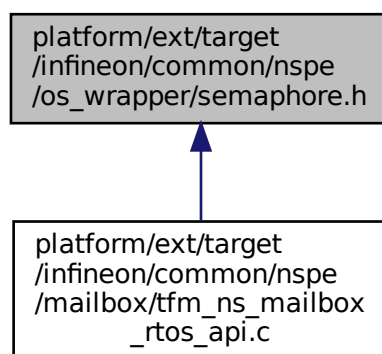
## 8.132 platform/ext/target/infineon/common/nspe/os\_wrapper/semaphore.h File Reference

```
#include "os_wrapper/common.h"
```

Include dependency graph for semaphore.h:



This graph shows which files directly or indirectly include this file:



## Functions

- `void * os_wrapper_semaphore_create` (uint32\_t max\_count, uint32\_t initial\_count, const char \*name)  
*Creates a new semaphore.*
- `uint32_t os_wrapper_semaphore_acquire` (void \*handle, uint32\_t timeout)

*Acquires the semaphore.*

- uint32\_t [os\\_wrapper\\_semaphore\\_release](#) (void \*handle)

*Releases the semaphore.*

- uint32\_t [os\\_wrapper\\_semaphore\\_delete](#) (void \*handle)

*Deletes the semaphore.*

## 8.132.1 Function Documentation

### 8.132.1.1 os\_wrapper\_semaphore\_acquire()

```
uint32_t os_wrapper_semaphore_acquire (
 void * handle,
 uint32_t timeout)
```

Acquires the semaphore.

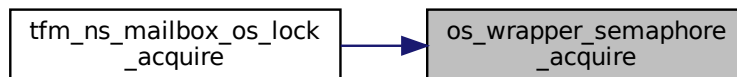
#### Parameters

|    |                |                  |
|----|----------------|------------------|
| in | <i>handle</i>  | Semaphore handle |
| in | <i>timeout</i> | Timeout value    |

#### Returns

[OS\\_WRAPPER\\_SUCCESS](#) in case of successful acquisition, or [OS\\_WRAPPER\\_ERROR](#) in case of error

Here is the caller graph for this function:



### 8.132.1.2 os\_wrapper\_semaphore\_create()

```
void* os_wrapper_semaphore_create (
 uint32_t max_count,
 uint32_t initial_count,
 const char * name)
```

Creates a new semaphore.

#### Parameters

|    |                      |                                           |
|----|----------------------|-------------------------------------------|
| in | <i>max_count</i>     | Highest count of the semaphore            |
| in | <i>initial_count</i> | Starting count of the available semaphore |
| in | <i>name</i>          | Name of the semaphore                     |

**Returns**

Returns handle of the semaphore created, or NULL in case of error

**8.132.1.3 os\_wrapper\_semaphore\_delete()**

```
uint32_t os_wrapper_semaphore_delete (
 void * handle)
```

Deletes the semaphore.

**Parameters**

|    |               |                  |
|----|---------------|------------------|
| in | <i>handle</i> | Semaphore handle |
|----|---------------|------------------|

**Returns**

[OS\\_WRAPPER\\_SUCCESS](#) in case of successful release, or [OS\\_WRAPPER\\_ERROR](#) in case of error

**8.132.1.4 os\_wrapper\_semaphore\_release()**

```
uint32_t os_wrapper_semaphore_release (
 void * handle)
```

Releases the semaphore.

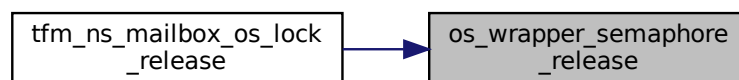
**Parameters**

|    |               |                  |
|----|---------------|------------------|
| in | <i>handle</i> | Semaphore handle |
|----|---------------|------------------|

**Returns**

[OS\\_WRAPPER\\_SUCCESS](#) in case of successful release, or [OS\\_WRAPPER\\_ERROR](#) in case of error

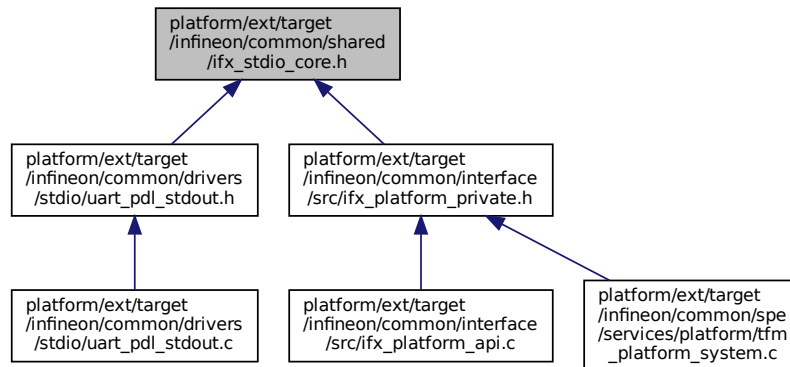
Here is the caller graph for this function:



## 8.133 platform/ext/target/infineon/common/shared/ifx\_stdio\_core.h File Reference

STDIO core prefix configuration for all Infineon platforms.

This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_STDIO_CORE_S_ID IFX_STDIO_CORE_S33`

## Enumerations

- `enum ifx_stdio_core_id_t { IFX_STDIO_CORE_S33, IFX_STDIO_CORE_N33, IFX_STDIO_CORE_N55, IFX_STDIO_CORE_MAX_COUNT }`

### 8.133.1 Detailed Description

STDIO core prefix configuration for all Infineon platforms.

This file provides STDIO core identification macros for platforms with CM33 and optionally CM55 cores. It defines:

- Core ID enumeration
- Core prefix strings for secure CM33, non-secure CM33, and non-secure CM55 output
- Macros to select core ID and prefix string at compile time

This header works for both single-core (CM33 only) and dual-core (CM33+CM55) platforms. On single-core platforms, the CM55 definitions are present but unused.

### 8.133.2 Macro Definition Documentation

#### 8.133.2.1 IFX\_STDIO\_CORE\_S\_ID

```
#define IFX_STDIO_CORE_S_ID IFX_STDIO_CORE_S33
```

Definition at line 44 of file ifx\_stdio\_core.h.

### 8.133.3 Enumeration Type Documentation

#### 8.133.3.1 ifx\_stdio\_core\_id\_t

```
enum ifx_stdio_core_id_t
```

## Enumerator

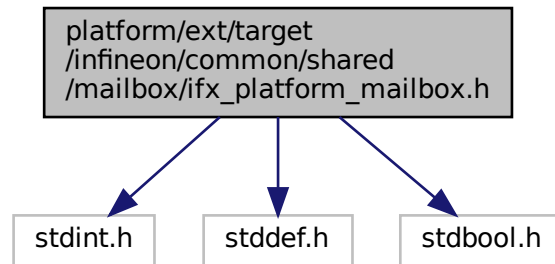
|                          |  |
|--------------------------|--|
| IFX_STDIO_CORE_S33       |  |
| IFX_STDIO_CORE_N33       |  |
| IFX_STDIO_CORE_N55       |  |
| IFX_STDIO_CORE_MAX_COUNT |  |

Definition at line 27 of file ifx\_stdio\_core.h.

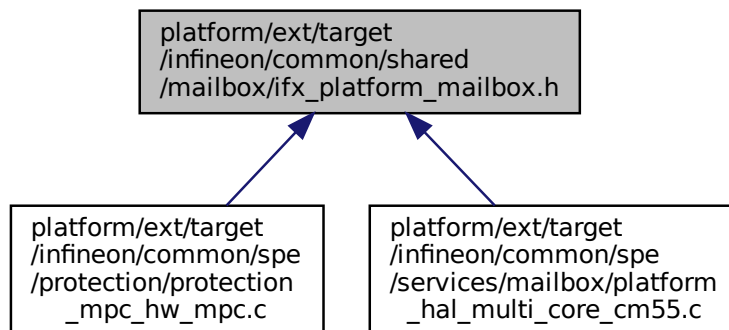
## 8.134 platform/ext/target/infineon/common/shared/mailbox/ifx\_platform\_mailbox.h File Reference

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
```

Include dependency graph for ifx\_platform\_mailbox.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define tfm_hal_remap_ns_cpu_address(addr) (addr)`

## 8.134.1 Macro Definition Documentation

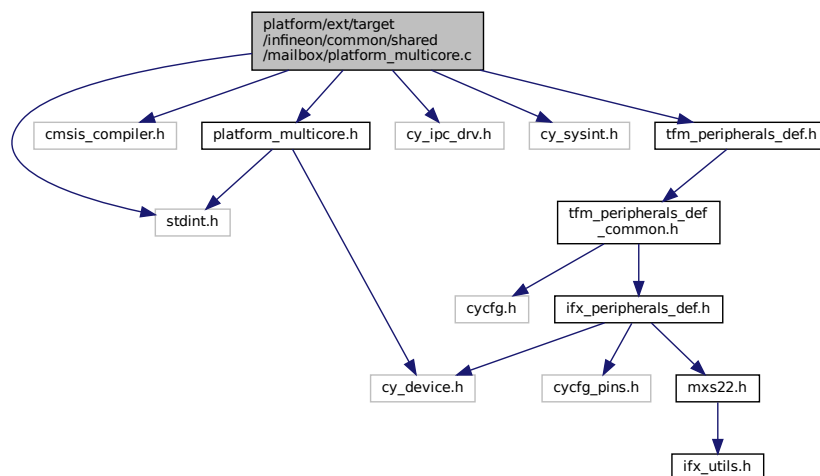
### 8.134.1.1 tfm\_hal\_remap\_ns\_cpu\_address

```
#define tfm_hal_remap_ns_cpu_address(
 addr) (addr)
```

Definition at line 47 of file ifx\_platform\_mailbox.h.

## 8.135 platform/ext/target/infineon/common/shared/mailbox/platform\_multicore.c File Reference

```
#include <stdint.h>
#include "cmsis_compiler.h"
#include "platform_multicore.h"
#include "cy_ipc_drv.h"
#include "cy_sysint.h"
#include "tfm_peripherals_def.h"
Include dependency graph for platform_multicore.c:
```



## Macros

- #define `IPC_RX_CHAN` `IFX_IPC_TFM_TO_NS_CHAN`
- #define `IPC_RX_INTR_STRUCT` `IFX_IPC_TFM_TO_NS_INTR_STRUCT`
- #define `IPC_RX_INT_MASK` `IFX_IPC_TFM_TO_NS_INTR_MASK`
- #define `IPC_TX_CHAN` `IFX_IPC_NS_TO_TFM_CHAN`
- #define `IPC_TX_NOTIFY_MASK` `IFX_IPC_NS_TO_TFM_NOTIFY_MASK`

## Functions

- `int32_t ifx_mailbox_fetch_msg_ptr(void **msg_ptr)`  
*Fetch a pointer from mailbox message.*
- `int32_t ifx_mailbox_fetch_msg_data(uint32_t *data_ptr)`  
*Fetch a data value from mailbox message.*
- `int32_t ifx_mailbox_send_msg_ptr(const void *msg_ptr)`

*Send a pointer via mailbox message.*

- `int32_t ifx_mailbox_send_msg_data (uint32_t data)`

*Send a data value via mailbox message.*

- `void ifx_mailbox_wait_for_notify (void)`

*Wait for a mailbox notify event.*

## 8.135.1 Macro Definition Documentation

### 8.135.1.1 IPC\_RX\_CHAN

```
#define IPC_RX_CHAN IFX_IPC_TFM_TO_NS_CHAN
```

Definition at line 27 of file platform\_multicore.c.

### 8.135.1.2 IPC\_RX\_INT\_MASK

```
#define IPC_RX_INT_MASK IFX_IPC_TFM_TO_NS_INTR_MASK
```

Definition at line 29 of file platform\_multicore.c.

### 8.135.1.3 IPC\_RX\_INTR\_STRUCT

```
#define IPC_RX_INTR_STRUCT IFX_IPC_TFM_TO_NS_INTR_STRUCT
```

Definition at line 28 of file platform\_multicore.c.

### 8.135.1.4 IPC\_TX\_CHAN

```
#define IPC_TX_CHAN IFX_IPC_NS_TO_TFM_CHAN
```

Definition at line 31 of file platform\_multicore.c.

### 8.135.1.5 IPC\_TX\_NOTIFY\_MASK

```
#define IPC_TX_NOTIFY_MASK IFX_IPC_NS_TO_TFM_NOTIFY_MASK
```

Definition at line 32 of file platform\_multicore.c.

## 8.135.2 Function Documentation

### 8.135.2.1 ifx\_mailbox\_fetch\_msg\_data()

```
int32_t ifx_mailbox_fetch_msg_data (
 uint32_t * data_ptr)
```

Fetch a data value from mailbox message.

#### Parameters

|     |                 |                                            |
|-----|-----------------|--------------------------------------------|
| out | <i>data_ptr</i> | The address to write the pointer value to. |
|-----|-----------------|--------------------------------------------|

#### Return values

|      |                         |
|------|-------------------------|
| 0    | The operation succeeds. |
| else | The operation fails.    |

Definition at line 53 of file platform\_multicore.c.

### 8.135.2.2 ifx\_mailbox\_fetch\_msg\_ptr()

```
int32_t ifx_mailbox_fetch_msg_ptr (
 void ** msg_ptr)
```

Fetch a pointer from mailbox message.

#### Parameters

|     |                |                                            |
|-----|----------------|--------------------------------------------|
| out | <i>msg_ptr</i> | The address to write the pointer value to. |
|-----|----------------|--------------------------------------------|

#### Return values

|             |                         |
|-------------|-------------------------|
| 0           | The operation succeeds. |
| <i>else</i> | The operation fails.    |

Definition at line 35 of file platform\_multicore.c.

Here is the caller graph for this function:



### 8.135.2.3 ifx\_mailbox\_send\_msg\_data()

```
int32_t ifx_mailbox_send_msg_data (
 uint32_t data)
```

Send a data value via mailbox message.

#### Parameters

|    |             |                           |
|----|-------------|---------------------------|
| in | <i>data</i> | The data value to be sent |
|----|-------------|---------------------------|

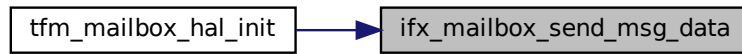
#### Return values

|             |                         |
|-------------|-------------------------|
| 0           | The operation succeeds. |
| <i>else</i> | The operation fails.    |

Definition at line 89 of file platform\_multicore.c.



Here is the caller graph for this function:



#### 8.135.2.4 ifx\_mailbox\_send\_msg\_ptr()

```
int32_t ifx_mailbox_send_msg_ptr (
 const void * msg_ptr)
```

Send a pointer via mailbox message.

##### Parameters

|    |                |                               |
|----|----------------|-------------------------------|
| in | <i>msg_ptr</i> | The pointer value to be sent. |
|----|----------------|-------------------------------|

##### Return values

|      |                         |
|------|-------------------------|
| 0    | The operation succeeds. |
| else | The operation fails.    |

Definition at line 71 of file platform\_multicore.c.

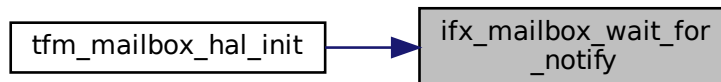
#### 8.135.2.5 ifx\_mailbox\_wait\_for\_notify()

```
void ifx_mailbox_wait_for_notify (
 void)
```

Wait for a mailbox notify event.

Definition at line 102 of file platform\_multicore.c.

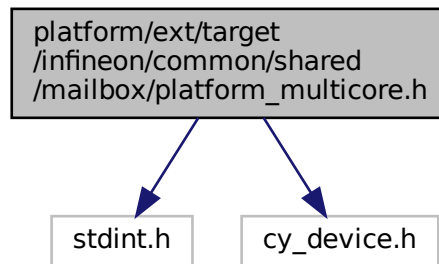
Here is the caller graph for this function:



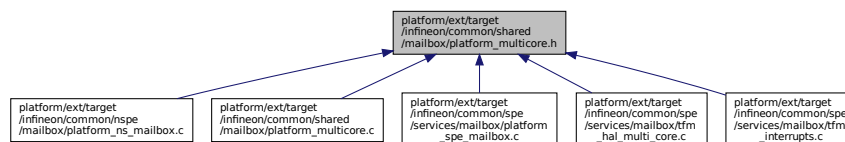
## 8.136 platform/ext/target/infineon/common/shared/mailbox/platform\_multicore.h File Reference

```
#include <stdint.h>
#include "cy_device.h"
```

Include dependency graph for platform\_multicore.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_PSA_CLIENT_CALL_REQ_MAGIC` (0xA5CF50C6U)
- `#define IFX_PSA_CLIENT_CALL_REPLY_MAGIC` (0xC605FC5AU)
- `#define IFX_NS_MAILBOX_INIT_ENABLE` (0xAEU)
- `#define IFX_S_MAILBOX_READY` (0xC3U)
- `#define IFX_PLATFORM_MAILBOX_SUCCESS` (0x0)
- `#define IFX_PLATFORM_MAILBOX_INVALID_PARAMS` (INT32\_MIN + 1)
- `#define IFX_PLATFORM_MAILBOX_TX_ERROR` (INT32\_MIN + 2)
- `#define IFX_PLATFORM_MAILBOX_RX_ERROR` (INT32\_MIN + 3)
- `#define IFX_PLATFORM_MAILBOX_INIT_ERROR` (INT32\_MIN + 4)
- `#define IFX_IPC_SYNC_MAGIC` 0x7DADE011U

## Functions

- `int32_t ifx_mailbox_fetch_msg_ptr` (void \*\*msg\_ptr)  
*Fetch a pointer from mailbox message.*
- `int32_t ifx_mailbox_fetch_msg_data` (uint32\_t \*data\_ptr)  
*Fetch a data value from mailbox message.*
- `int32_t ifx_mailbox_send_msg_ptr` (const void \*msg\_ptr)  
*Send a pointer via mailbox message.*
- `int32_t ifx_mailbox_send_msg_data` (uint32\_t data)  
*Send a data value via mailbox message.*
- `void ifx_mailbox_wait_for_notify` (void)  
*Wait for a mailbox notify event.*

## 8.136.1 Macro Definition Documentation

### 8.136.1.1 IFX\_IPC\_SYNC\_MAGIC

```
#define IFX_IPC_SYNC_MAGIC 0x7DADE011U
```

Definition at line 28 of file platform\_multicore.h.

### 8.136.1.2 IFX\_NS\_MAILBOX\_INIT\_ENABLE

```
#define IFX_NS_MAILBOX_INIT_ENABLE (0xAEU)
```

Definition at line 19 of file platform\_multicore.h.

### 8.136.1.3 IFX\_PLATFORM\_MAILBOX\_INIT\_ERROR

```
#define IFX_PLATFORM_MAILBOX_INIT_ERROR (INT32_MIN + 4)
```

Definition at line 26 of file platform\_multicore.h.

### 8.136.1.4 IFX\_PLATFORM\_MAILBOX\_INVALID\_PARAMS

```
#define IFX_PLATFORM_MAILBOX_INVALID_PARAMS (INT32_MIN + 1)
```

Definition at line 23 of file platform\_multicore.h.

### 8.136.1.5 IFX\_PLATFORM\_MAILBOX\_RX\_ERROR

```
#define IFX_PLATFORM_MAILBOX_RX_ERROR (INT32_MIN + 3)
```

Definition at line 25 of file platform\_multicore.h.

### 8.136.1.6 IFX\_PLATFORM\_MAILBOX\_SUCCESS

```
#define IFX_PLATFORM_MAILBOX_SUCCESS (0x0)
```

Definition at line 22 of file platform\_multicore.h.

### 8.136.1.7 IFX\_PLATFORM\_MAILBOX\_TX\_ERROR

```
#define IFX_PLATFORM_MAILBOX_TX_ERROR (INT32_MIN + 2)
```

Definition at line 24 of file platform\_multicore.h.

### 8.136.1.8 IFX\_PSA\_CLIENT\_CALL\_REPLY\_MAGIC

```
#define IFX_PSA_CLIENT_CALL_REPLY_MAGIC (0xC605FC5AU)
```

Definition at line 17 of file platform\_multicore.h.

### 8.136.1.9 IFX\_PSA\_CLIENT\_CALL\_REQ\_MAGIC

```
#define IFX_PSA_CLIENT_CALL_REQ_MAGIC (0xA5CF50C6U)
```

Definition at line 16 of file platform\_multicore.h.

### 8.136.1.10 IFX\_S\_MAILBOX\_READY

#define IFX\_S\_MAILBOX\_READY (0xC3U)  
Definition at line 20 of file platform\_multicore.h.

## 8.136.2 Function Documentation

### 8.136.2.1 ifx\_mailbox\_fetch\_msg\_data()

```
int32_t ifx_mailbox_fetch_msg_data (
 uint32_t * data_ptr)
```

Fetch a data value from mailbox message.

#### Parameters

|     |                 |                                            |
|-----|-----------------|--------------------------------------------|
| out | <i>data_ptr</i> | The address to write the pointer value to. |
|-----|-----------------|--------------------------------------------|

#### Return values

|             |                         |
|-------------|-------------------------|
| 0           | The operation succeeds. |
| <i>else</i> | The operation fails.    |

Definition at line 53 of file platform\_multicore.c.

### 8.136.2.2 ifx\_mailbox\_fetch\_msg\_ptr()

```
int32_t ifx_mailbox_fetch_msg_ptr (
 void ** msg_ptr)
```

Fetch a pointer from mailbox message.

#### Parameters

|     |                |                                            |
|-----|----------------|--------------------------------------------|
| out | <i>msg_ptr</i> | The address to write the pointer value to. |
|-----|----------------|--------------------------------------------|

#### Return values

|             |                         |
|-------------|-------------------------|
| 0           | The operation succeeds. |
| <i>else</i> | The operation fails.    |

Definition at line 35 of file platform\_multicore.c.

Here is the caller graph for this function:



### 8.136.2.3 ifx\_mailbox\_send\_msg\_data()

```
int32_t ifx_mailbox_send_msg_data (
 uint32_t data)
```

Send a data value via mailbox message.

#### Parameters

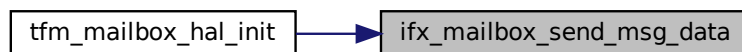
| in | <i>data</i> | The data value to be sent |
|----|-------------|---------------------------|

#### Return values

|             |                         |
|-------------|-------------------------|
| 0           | The operation succeeds. |
| <i>else</i> | The operation fails.    |

Definition at line 89 of file platform\_multicore.c.

Here is the caller graph for this function:



### 8.136.2.4 ifx\_mailbox\_send\_msg\_ptr()

```
int32_t ifx_mailbox_send_msg_ptr (
 const void * msg_ptr)
```

Send a pointer via mailbox message.

#### Parameters

| in | <i>msg_ptr</i> | The pointer value to be sent. |
|----|----------------|-------------------------------|

#### Return values

|             |                         |
|-------------|-------------------------|
| 0           | The operation succeeds. |
| <i>else</i> | The operation fails.    |

Definition at line 71 of file platform\_multicore.c.

### 8.136.2.5 ifx\_mailbox\_wait\_for\_notify()

```
void ifx_mailbox_wait_for_notify (
 void)
```

Wait for a mailbox notify event.

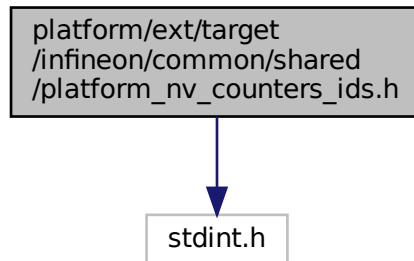
Definition at line 102 of file platform\_multicore.c.



## 8.138 platform/ext/target/infineon/common/shared/platform\_nv\_counters\_ids.h File Reference

```
#include <stdint.h>
```

Include dependency graph for platform\_nv\_counters\_ids.h:



### Macros

- `#define PLAT_NV_COUNTER_NS_MAX PLAT_NV_COUNTER_NS_2`

### Enumerations

- enum `tfm_nv_counter_t` {  
`PLAT_NV_COUNTER_PS_0 = 0, PLAT_NV_COUNTER_PS_1, PLAT_NV_COUNTER_PS_2, PLAT_NV_COUNTER_NS_0,`  
`PLAT_NV_COUNTER_NS_1, PLAT_NV_COUNTER_NS_2, PLAT_NV_COUNTER_MAX, PLAT_NV_COUNTER_BOUNDAR`  
`= UINT32_MAX }`

### 8.138.1 Macro Definition Documentation

#### 8.138.1.1 PLAT\_NV\_COUNTER\_NS\_MAX

```
#define PLAT_NV_COUNTER_NS_MAX PLAT_NV_COUNTER_NS_2
```

Definition at line 39 of file `platform_nv_counters_ids.h`.

### 8.138.2 Enumeration Type Documentation

#### 8.138.2.1 tfm\_nv\_counter\_t

```
enum tfm_nv_counter_t
```

##### Enumerator

|                      |  |
|----------------------|--|
| PLAT_NV_COUNTER_PS_0 |  |
| PLAT_NV_COUNTER_PS_1 |  |
| PLAT_NV_COUNTER_PS_2 |  |
| PLAT_NV_COUNTER_NS_0 |  |
| PLAT_NV_COUNTER_NS_1 |  |

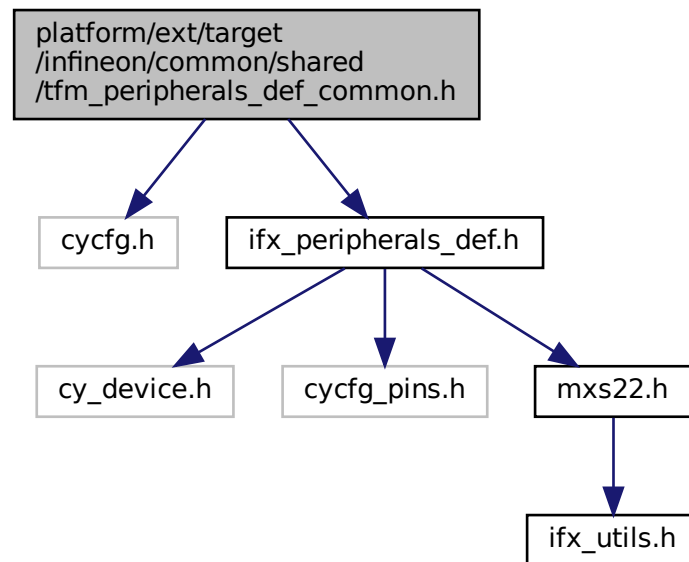
## Enumerator

|                          |  |
|--------------------------|--|
| PLAT_NV_COUNTER_NS_2     |  |
| PLAT_NV_COUNTER_MAX      |  |
| PLAT_NV_COUNTER_BOUNDARY |  |

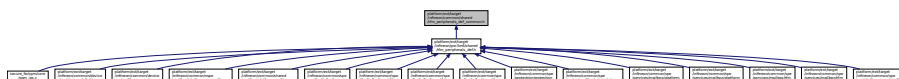
Definition at line 19 of file platform\_nv\_counters\_ids.h.

## 8.139 platform/ext/target/infineon/common/shared/tfm\_peripherals\_def\_common.h File Reference

```
#include "cycfg.h"
#include "ifx_peripherals_def.h"
Include dependency graph for tfm_peripherals_def_common.h:
```



This graph shows which files directly or indirectly include this file:



## Macros

- `#define` `DEFAULT_IRQ_PRIORITY` `(1UL << (_NVIC_PRIO_BITS - 2))`

### 8.139.1 Macro Definition Documentation





## 8.140.2 Function Documentation

### 8.140.2.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_plat_err_t)
```

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

## Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

## Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

## Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

## Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Parameters**

|    |                  |                                             |
|----|------------------|---------------------------------------------|
| in | <i>p_assets</i>  | Array of partition's assets                 |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

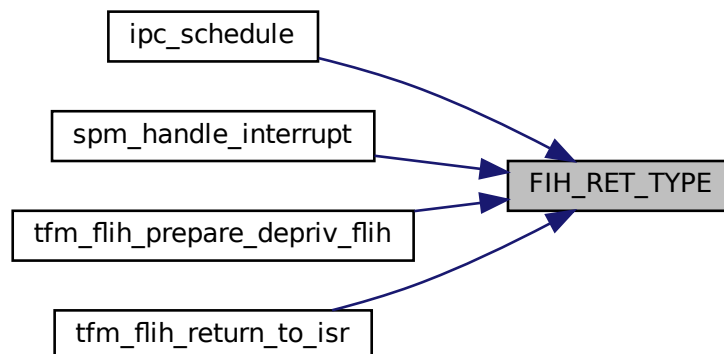
|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Definition at line 122 of file faults.c.

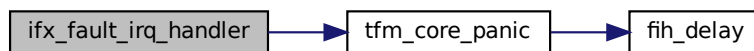
Here is the caller graph for this function:

**8.140.2.2 ifx\_fault\_irq\_handler()**

```
void ifx_fault_irq_handler (
 void)
```

Definition at line 99 of file faults.c.

Here is the call graph for this function:



### 8.140.2.3 ifx\_msc\_fault\_irq\_handler()

```
void ifx_msc_fault_irq_handler (
 void)
```

Definition at line 71 of file faults.c.

Here is the call graph for this function:

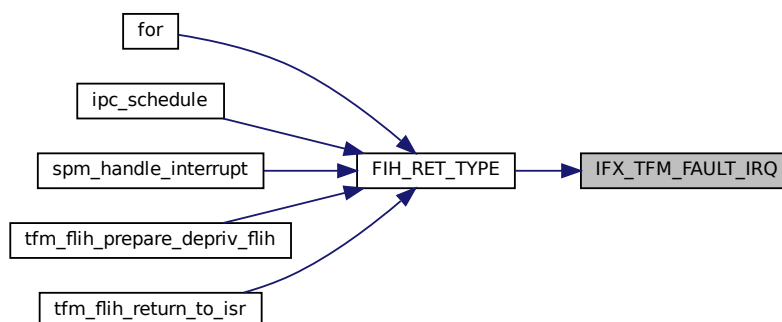


### 8.140.2.4 IFX\_TFM\_FAULT\_IRQ()

```
void IFX_IRQ_NAME_TO_HANDLER() IFX_TFM_FAULT_IRQ (
 void)
```

Definition at line 112 of file faults.c.

Here is the caller graph for this function:

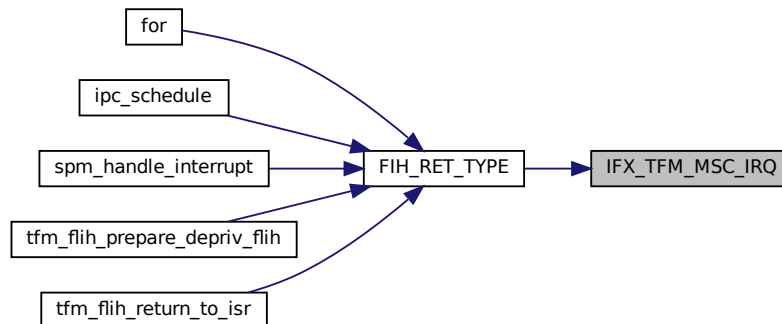


### 8.140.2.5 IFX\_TFM\_MSC\_IRQ()

```
void IFX_IRQ_NAME_TO_HANDLER() IFX_TFM_MSC_IRQ (
 void)
```

Definition at line 88 of file faults.c.

Here is the caller graph for this function:



#### 8.140.2.6 tfm\_hal\_platform\_exception\_info()

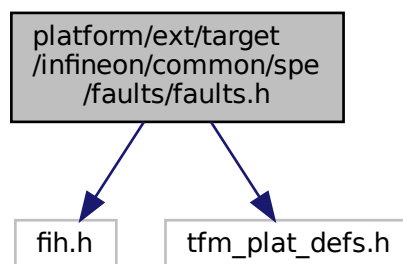
```
void tfm_hal_platform_exception_info (
 void)
```

Definition at line 295 of file faults.c.

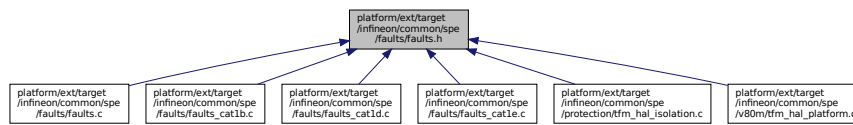
## 8.141 platform/ext/target/infineon/common/spe/faults/faults.h File Reference

```
#include "fih.h"
#include "tfm_plat_defs.h"
```

Include dependency graph for faults.h:



This graph shows which files directly or indirectly include this file:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_plat\_err\_t) ifx\_faults\_cfg(void)

*Configures fault handlers and sets priorities.*

### 8.141.1 Function Documentation

#### 8.141.1.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures fault handlers and sets priorities.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the tfm\_plat\_err\_t

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the tfm\_plat\_err\_t

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU



## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

## Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

## Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

## Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

#### Returns

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

#### Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

#### Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

#### Returns

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

#### Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

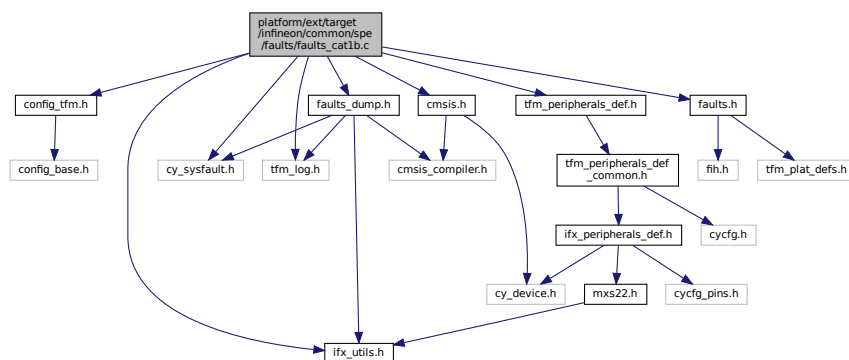
TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 122 of file faults.c.

## 8.142 platform/ext/target/infineon/common/spe/faults/faults\_cat1b.c File Reference

```
#include "config_tfm.h"
#include "cmsis.h"
#include "cy_sysfault.h"
#include "faults.h"
#include "faults_dump.h"
#include "ifx_utils.h"
#include "tfm_peripherals_def.h"
#include "tfm_log.h"
```

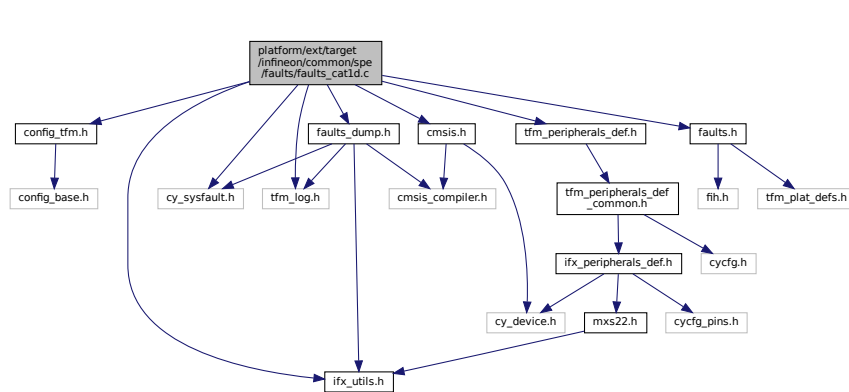
Include dependency graph for faults\_cat1b.c:



## 8.143 platform/ext/target/infineon/common/spe/faults/faults\_cat1d.c File Reference

```
#include "config_tfm.h"
#include "cmsis.h"
#include "cy_sysfault.h"
#include "faults.h"
#include "faults_dump.h"
#include "ifx_utils.h"
#include "tfm_peripherals_def.h"
#include "tfm_log.h"
```

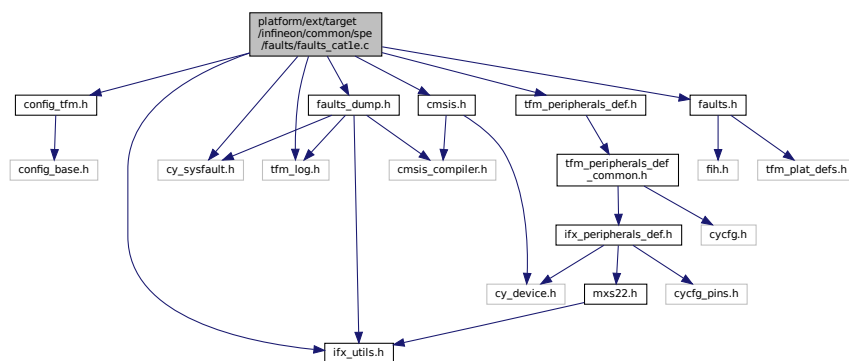
Include dependency graph for faults\_cat1d.c:



## 8.144 platform/ext/target/infineon/common/spe/faults/faults\_cat1e.c File Reference

```
#include "config_tfm.h"
#include "cmsis.h"
#include "cy_sysfault.h"
#include "faults.h"
#include "faults_dump.h"
#include "ifx_utils.h"
#include "tfm_peripherals_def.h"
#include "tfm_log.h"
```

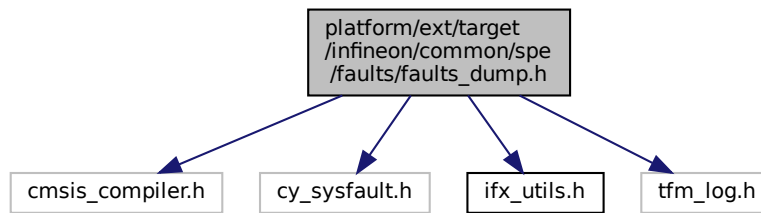
Include dependency graph for faults\_cat1e.c:



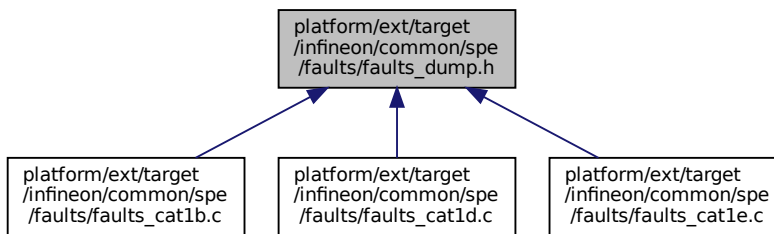
## 8.145 platform/ext/target/infineon/common/spe/faults/faults\_dump.h File Reference

```
#include "cmsis_compiler.h"
#include "cy_sysfault.h"
#include "ifx_utils.h"
#include "tfm_log.h"
```

Include dependency graph for faults\_dump.h:



This graph shows which files directly or indirectly include this file:



## Functions

- `__STATIC_FORCEINLINE void ifx_faults_dump_peri_msx_ppc_vio_fault (FAULT_STRUCT_Type *base)`  
Print PPC violation fault information captured by fault peripheral when a locked PC tries to write to PC\_MASK register.
- `__STATIC_FORCEINLINE void ifx_faults_dump_peri_ppc_pc_mask_vio_fault (FAULT_STRUCT_Type *base)`  
Print PPC MMIO access violation fault information captured by fault peripheral.
- `__STATIC_FORCEINLINE void ifx_faults_dump_peri_gpx_timeout_vio_fault (FAULT_STRUCT_Type *base)`  
Print peripheral group timeout violation fault information captured by fault peripheral.
- `__STATIC_FORCEINLINE void ifx_faults_dump_peri_gpx_ahb_vio_fault (FAULT_STRUCT_Type *base)`  
Print peripheral group AHB violation fault information captured by fault peripheral.
- `__STATIC_FORCEINLINE void ifx_faults_dump_mpc_fault (FAULT_STRUCT_Type *base)`  
Print MPC fault information captured by fault peripheral.
- `__STATIC_FORCEINLINE void ifx_faults_dump_default_fault (FAULT_STRUCT_Type *base)`  
Print fault information captured by fault peripheral.

## 8.145.1 Function Documentation

### 8.145.1.1 ifx\_faults\_dump\_default\_fault()

```

__STATIC_FORCEINLINE void ifx_faults_dump_default_fault (
 FAULT_STRUCT_Type * base)

```

Print fault information captured by fault peripheral.

## Parameters

|    |             |                                     |
|----|-------------|-------------------------------------|
| in | <i>base</i> | Fault structure with captured fault |
|----|-------------|-------------------------------------|

Definition at line 280 of file faults\_dump.h.

**8.145.1.2 ifx\_faults\_dump\_mpc\_fault()**

```
__STATIC_FORCEINLINE void ifx_faults_dump_mpc_fault (
 FAULT_STRUCT_Type * base)
```

Print MPC fault information captured by fault peripheral.

## Parameters

|    |             |                                     |
|----|-------------|-------------------------------------|
| in | <i>base</i> | Fault structure with captured fault |
|----|-------------|-------------------------------------|

Definition at line 226 of file faults\_dump.h.

**8.145.1.3 ifx\_faults\_dump\_peri\_gpx\_ahb\_vio\_fault()**

```
__STATIC_FORCEINLINE void ifx_faults_dump_peri_gpx_ahb_vio_fault (
 FAULT_STRUCT_Type * base)
```

Print peripheral group AHB violation fault information captured by fault peripheral.

## Parameters

|    |             |                                     |
|----|-------------|-------------------------------------|
| in | <i>base</i> | Fault structure with captured fault |
|----|-------------|-------------------------------------|

Definition at line 161 of file faults\_dump.h.

**8.145.1.4 ifx\_faults\_dump\_peri\_gpx\_timeout\_vio\_fault()**

```
__STATIC_FORCEINLINE void ifx_faults_dump_peri_gpx_timeout_vio_fault (
 FAULT_STRUCT_Type * base)
```

Print peripheral group timeout violation fault information captured by fault peripheral.

## Parameters

|    |             |                                     |
|----|-------------|-------------------------------------|
| in | <i>base</i> | Fault structure with captured fault |
|----|-------------|-------------------------------------|

Definition at line 140 of file faults\_dump.h.

**8.145.1.5 ifx\_faults\_dump\_peri\_msx\_ppc\_vio\_fault()**

```
__STATIC_FORCEINLINE void ifx_faults_dump_peri_msx_ppc_vio_fault (
 FAULT_STRUCT_Type * base)
```

Print PPC violation fault information captured by fault peripheral when a locked PC tries to write to PC\_MASK register.

## Parameters

|    |             |                                     |
|----|-------------|-------------------------------------|
| in | <i>base</i> | Fault structure with captured fault |
|----|-------------|-------------------------------------|

Definition at line 23 of file faults\_dump.h.

#### 8.145.1.6 ifx\_faults\_dump\_peri\_ppc\_pc\_mask\_vio\_fault()

```
__STATIC_FORCEINLINE void ifx_faults_dump_peri_ppc_pc_mask_vio_fault (
 FAULT_STRUCT_Type * base)
```

Print PPC MMIO access violation fault information captured by fault peripheral.  
It's used to report PPC violation when a locked PC tries to write to PC\_MASK register.

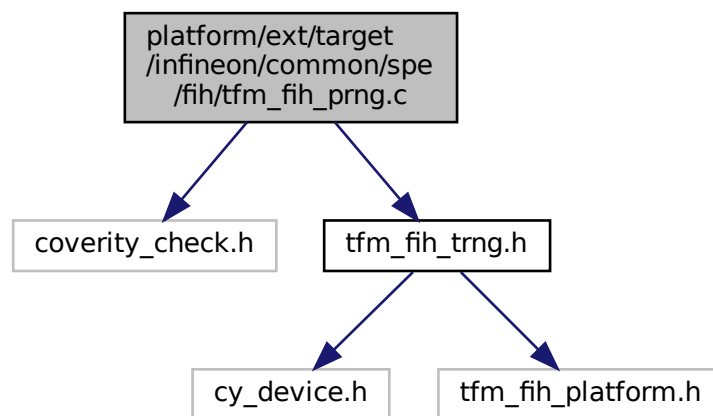
##### Parameters

|    |      |                                     |
|----|------|-------------------------------------|
| in | base | Fault structure with captured fault |
|----|------|-------------------------------------|

Definition at line 95 of file faults\_dump.h.

## 8.146 platform/ext/target/infineon/common/spe/fih/tfm\_fih\_prng.c File Reference

```
#include "coverity_check.h"
#include "tfm_fih_trng.h"
Include dependency graph for tfm_fih_prng.c:
```



## Macros

- `#define PRNG_A` 13
- `#define PRNG_B` 17
- `#define PRNG_C` 5

## Functions

- `int fih_delay_init` (void)
- `void tfm_fih_random_generate` (uint8\_t \*rand)
- `uint8_t fih_delay_random` (void)



## 8.146.1 Macro Definition Documentation

### 8.146.1.1 PRNG\_A

```
#define PRNG_A 13
```

Definition at line 12 of file tfm\_fih\_prng.c.

### 8.146.1.2 PRNG\_B

```
#define PRNG_B 17
```

Definition at line 13 of file tfm\_fih\_prng.c.

### 8.146.1.3 PRNG\_C

```
#define PRNG_C 5
```

Definition at line 14 of file tfm\_fih\_prng.c.

## 8.146.2 Function Documentation

### 8.146.2.1 fih\_delay\_init()

```
int fih_delay_init (
 void)
```

Definition at line 53 of file tfm\_fih\_prng.c.

Here is the caller graph for this function:



### 8.146.2.2 fih\_delay\_random()

```
uint8_t fih_delay_random (
 void)
```

Definition at line 70 of file tfm\_fih\_prng.c.

Here is the call graph for this function:

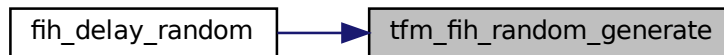


### 8.146.2.3 tfm\_fih\_random\_generate()

```
void tfm_fih_random_generate (
 uint8_t * rand)
```

Definition at line 60 of file `tfm_fih_prng.c`.

Here is the caller graph for this function:

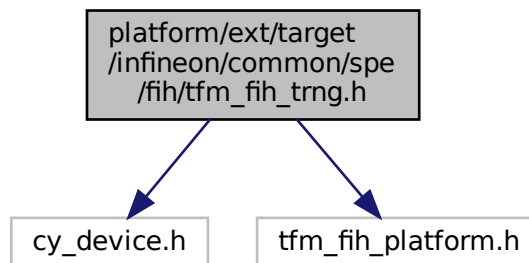


## 8.147 platform/ext/target/infineon/common/spe/fih/tfm\_fih\_trng.h File Reference

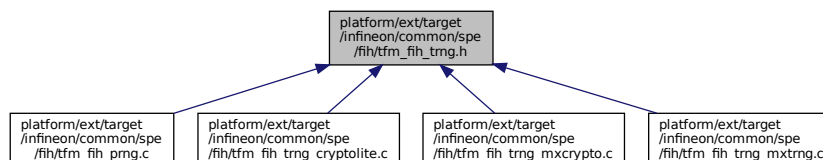
```
#include "cy_device.h"
```

```
#include "tfm_fih_platform.h"
```

Include dependency graph for `tfm_fih_trng.h`:



This graph shows which files directly or indirectly include this file:



## Functions

- `uint32_t ifx_trng (void)`

### 8.147.1 Function Documentation

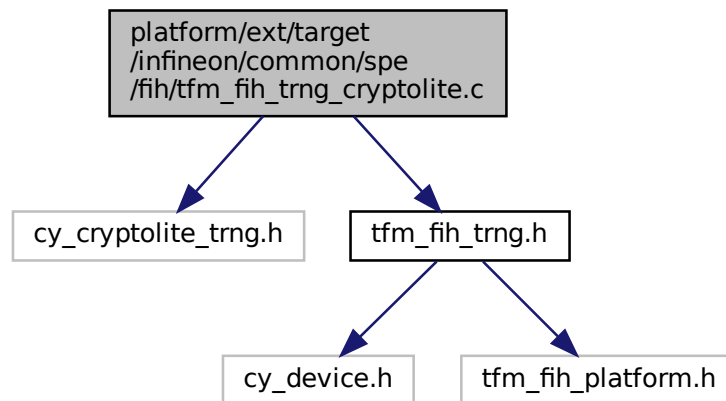
#### 8.147.1.1 ifx\_trng()

```
uint32_t ifx_trng (
 void)
```

Definition at line 16 of file tfm\_fih\_trng\_cryptolite.c.

## 8.148 platform/ext/target/infineon/common/spe/fih/tfm\_fih\_trng\_cryptolite.c File Reference

```
#include "cy_cryptolite_trng.h"
#include "tfm_fih_trng.h"
Include dependency graph for tfm_fih_trng_cryptolite.c:
```



### Functions

- uint32\_t `ifx_trng` (void)

### 8.148.1 Function Documentation

#### 8.148.1.1 ifx\_trng()

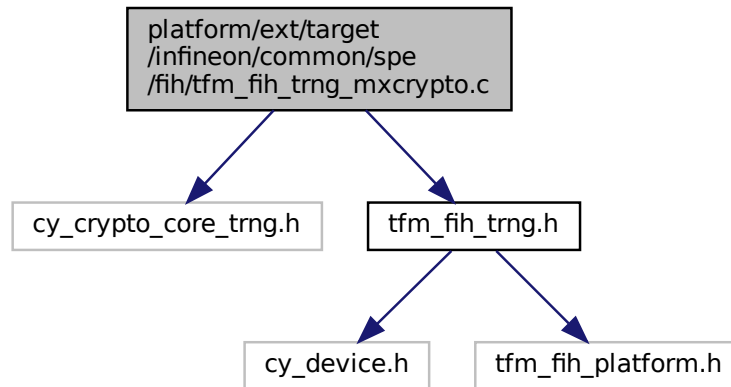
```
uint32_t ifx_trng (
 void)
```

Definition at line 16 of file tfm\_fih\_trng\_cryptolite.c.

## 8.149 platform/ext/target/infineon/common/spe/fih/tfm\_fih\_trng\_mxcrypto.c File Reference

```
#include "cy_crypto_core_trng.h"
#include "tfm_fih_trng.h"
```

Include dependency graph for tfm\_fih\_trng\_mxcrypto.c:



## Functions

- `uint32_t ifx_trng(void)`

### 8.149.1 Function Documentation

#### 8.149.1.1 ifx\_trng()

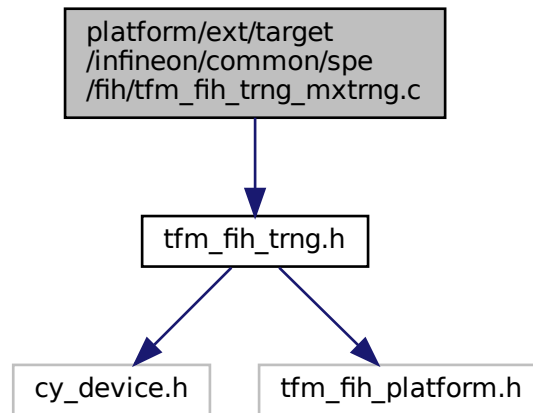
```
uint32_t ifx_trng (
 void)
```

Definition at line 16 of file `tfm_fih_trng_mxcrypto.c`.

## 8.150 platform/ext/target/infineon/common/spe/fih/tfm\_fih\_trng\_mxtrng.c File Reference

```
#include "tfm_fih_trng.h"
```

Include dependency graph for tfm\_fih\_trng\_mxtrng.c:



## Macros

- #define `IFX_MXTRNG_STUB_RANDOM_VALUE` (0xDEADBEEFuL)

## Functions

- uint32\_t `ifx_trng` (void)

### 8.150.1 Macro Definition Documentation

#### 8.150.1.1 IFX\_MXTRNG\_STUB\_RANDOM\_VALUE

```
#define IFX_MXTRNG_STUB_RANDOM_VALUE (0xDEADBEEFuL)
```

Definition at line 12 of file tfm\_fih\_trng\_mxtrng.c.

### 8.150.2 Function Documentation

#### 8.150.2.1 ifx\_trng()

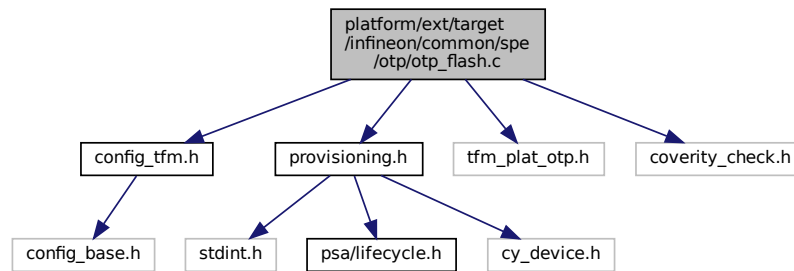
```
uint32_t ifx_trng (
 void)
```

Definition at line 18 of file tfm\_fih\_trng\_mxtrng.c.

## 8.151 platform/ext/target/infineon/common/spe/otp/otp\_flash.c File Reference

```
#include "config_tfm.h"
#include "provisioning.h"
#include "tfm_plat_otp.h"
```

```
#include "coverity_check.h"
Include dependency graph for otp_flash.c:
```



## Functions

- enum tfm\_plat\_err\_t [tfm\\_plat\\_otp\\_init](#) (void)
- enum tfm\_plat\_err\_t [tfm\\_plat\\_otp\\_read](#) (enum [tfm\\_otp\\_element\\_id\\_t](#) id, size\_t out\_len, uint8\_t \*out)
- enum tfm\_plat\_err\_t [tfm\\_plat\\_otp\\_write](#) (enum [tfm\\_otp\\_element\\_id\\_t](#) id, size\_t in\_len, const uint8\_t \*in)
- enum tfm\_plat\_err\_t [tfm\\_plat\\_otp\\_get\\_size](#) (enum [tfm\\_otp\\_element\\_id\\_t](#) id, size\_t \*size)

## 8.151.1 Function Documentation

### 8.151.1.1 tfm\_plat\_otp\_get\_size()

```
enum tfm_plat_err_t tfm_plat_otp_get_size (
 enum tfm_otp_element_id_t id,
 size_t * size)
```

Definition at line 67 of file `otp_flash.c`.

### 8.151.1.2 tfm\_plat\_otp\_init()

```
enum tfm_plat_err_t tfm_plat_otp_init (
 void)
```

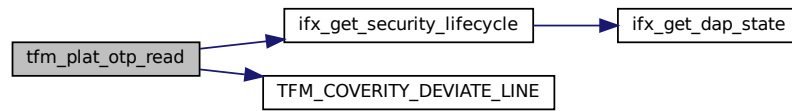
Definition at line 14 of file `otp_flash.c`.

### 8.151.1.3 tfm\_plat\_otp\_read()

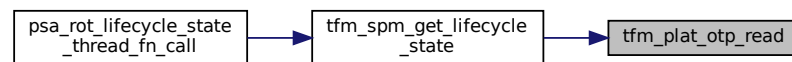
```
enum tfm_plat_err_t tfm_plat_otp_read (
 enum tfm_otp_element_id_t id,
 size_t out_len,
 uint8_t * out)
```

Definition at line 19 of file `otp_flash.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.151.1.4 tfm\_plat\_otp\_write()

```

enum tfm_plat_err_t tfm_plat_otp_write (
 enum tfm_otp_element_id_t id,
 size_t in_len,
 const uint8_t * in)

```

Definition at line 57 of file `otp_flash.c`.

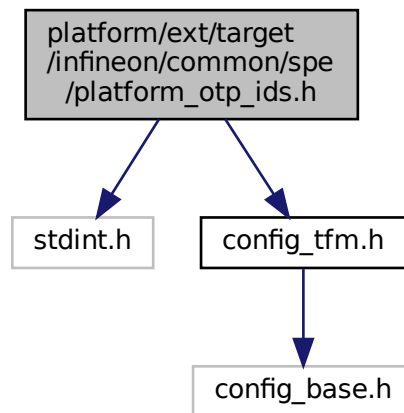
## 8.152 platform/ext/target/infineon/common/spe/platform\_otp\_ids.h File Reference

```

#include <stdint.h>
#include "config_tfm.h"

```

Include dependency graph for platform\_otp\_ids.h:



## Enumerations

- enum `tfm_otp_element_id_t` { `PLAT_OTP_ID_LCS` = 1, `PLAT_OTP_ID_MAX` = `UINT32_MAX` }

### 8.152.1 Enumeration Type Documentation

#### 8.152.1.1 tfm\_otp\_element\_id\_t

```
enum tfm_otp_element_id_t
```

Enumerator

|                              |  |
|------------------------------|--|
| <code>PLAT_OTP_ID_LCS</code> |  |
| <code>PLAT_OTP_ID_MAX</code> |  |

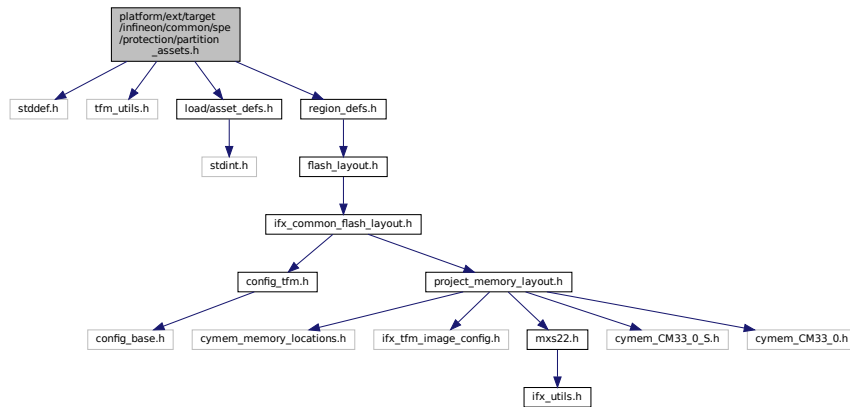
Definition at line 21 of file `platform_otp_ids.h`.

## 8.153 platform/ext/target/infineon/common/spe/protection/partition\_↵ assets.h File Reference

```
#include <stddef.h>
#include "tfm_utils.h"
#include "load/asset_defs.h"
#include "region_defs.h"
```



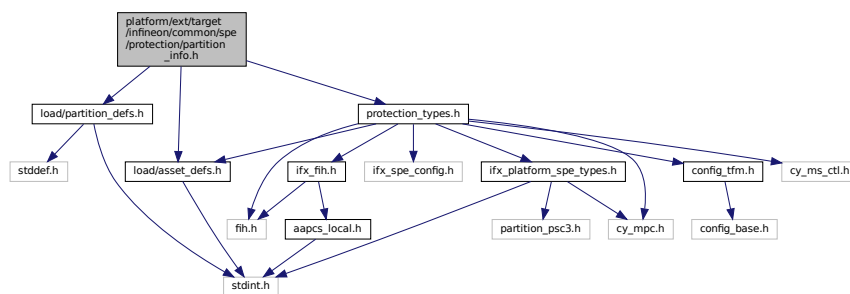
Include dependency graph for partition\_assets.h:



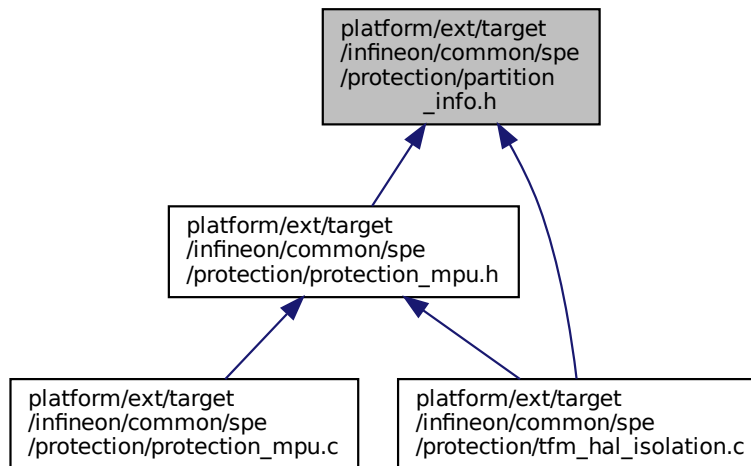
## 8.154 platform/ext/target/infineon/common/spe/protection/partition\_info.h File Reference

```
#include "load/partition_defs.h"
#include "load/asset_defs.h"
#include "protection_types.h"
```

Include dependency graph for partition\_info.h:



This graph shows which files directly or indirectly include this file:



## Functions

- `const ifx_partition_info_t * ifx_protection_get_partition_info (const struct partition_load_info_t *p_ldinfo)`  
Return platform specific partition info.

### 8.154.1 Function Documentation

#### 8.154.1.1 ifx\_protection\_get\_partition\_info()

```
const ifx_partition_info_t* ifx_protection_get_partition_info (
 const struct partition_load_info_t * p_ldinfo)
```

Return platform specific partition info.

#### Parameters

|    |                 |                                                                                          |
|----|-----------------|------------------------------------------------------------------------------------------|
| in | <i>p_ldinfo</i> | Partition load info. This function uses partition id to lookup for partition properties. |
|----|-----------------|------------------------------------------------------------------------------------------|

It's expected that it will be used only once to bind partition boundary by `tfm_hal_bind_boundary`. In all other cases boundary should be converted to constant pointer to `ifx_partition_info_t`.

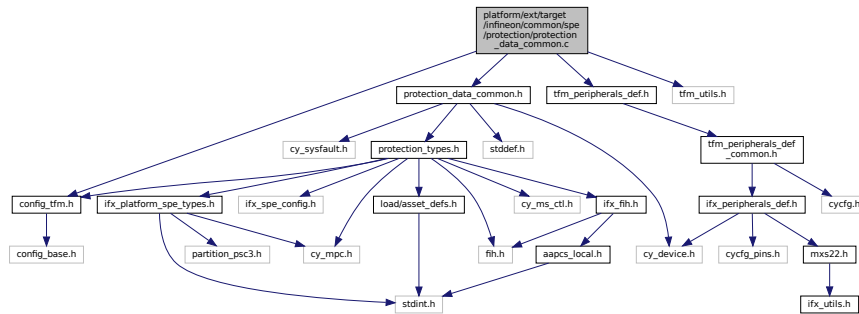
#### Returns

Pointer to platform specific partition info

## 8.155 platform/ext/target/infineon/common/spe/protection/protection\_data\_common.c File Reference

```
#include "config_tfm.h"
#include "protection_data_common.h"
#include "tfm_peripherals_def.h"
#include "tfm_utils.h"
```

Include dependency graph for protection\_data\_common.c:



## Variables

- `const IRQn_Type ifx_secure_interrupts_config []`
- `const size_t ifx_secure_interrupts_config_count = ARRAY_SIZE(ifx_secure_interrupts_config)`

### 8.155.1 Variable Documentation

#### 8.155.1.1 ifx\_secure\_interrupts\_config

```
const IRQn_Type ifx_secure_interrupts_config[]
```

**Initial value:**

```
= {
 IFX_TFM_FAULT_IRQ,
 IFX_TFM_MSC_IRQ,
}
```

Definition at line 16 of file protection\_data\_common.c.

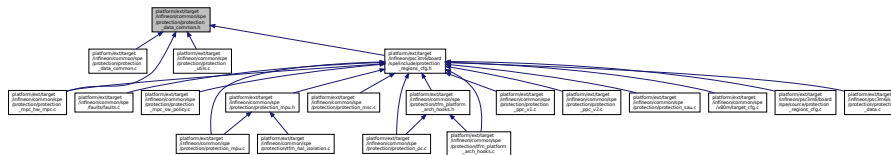
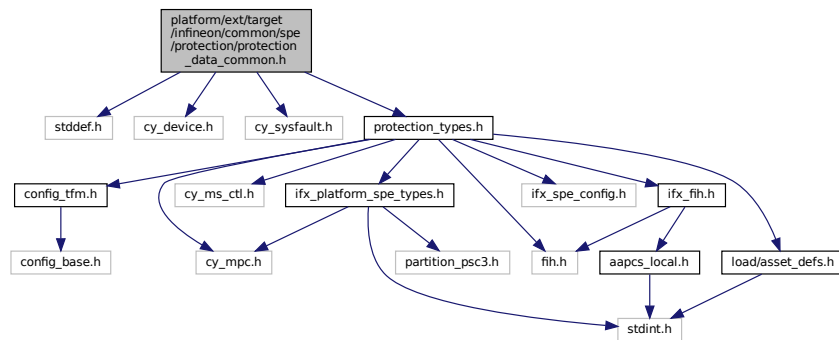
#### 8.155.1.2 ifx\_secure\_interrupts\_config\_count

```
const size_t ifx_secure_interrupts_config_count = ARRAY_SIZE(ifx_secure_interrupts_config)
```

Definition at line 25 of file protection\_data\_common.c.

## 8.156 platform/ext/target/infineon/common/spe/protection/protection\_data\_common.h File Reference ↩

```
#include <stddef.h>
#include "cy_device.h"
#include "cy_sysfault.h"
#include "protection_types.h"
```



## Variables

- const IRQn\_Type [ifx\\_secure\\_interrupts\\_config](#) []
- const size\_t [ifx\\_secure\\_interrupts\\_config\\_count](#)
- const cy\_en\_SysFault\_source\_t [ifx\\_tfm\\_fault\\_sources](#) []
- const size\_t [ifx\\_tfm\\_fault\\_sources\\_count](#)

### 8.156.1 Variable Documentation

### 8.156.1.1 ifx\_secure\_interrupts\_config

const IRQn\_Type ifx\_secure\_interrupts\_config[]  
Definition at line 16 of file protection\_data\_common.c.

### 8.156.1.2 ifx\_secure\_interrupts\_config\_count

const size\_t ifx\_secure\_interrupts\_config\_count  
Definition at line 25 of file protection\_data\_common.c.

### 8.156.1.3 ifx\_tfm\_fault\_sources

```
const cy_en_SysFault_source_t ifx_tfm_fault_sources[]
```

Definition at line 72 of file protection\_regions\_cfg.c.

## 8.156.1.4 ifx\_tfm\_fault\_sources\_count

```
const size_t ifx_tfm_fault_sources_count
```

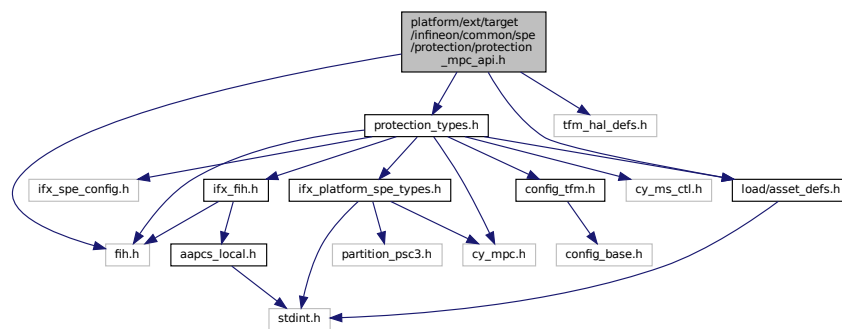
Definition at line 94 of file protection\_regions\_cfg.c.

## 8.157 platform/ext/target/infineon/common/spe/protection/protection\_mpc\_api.h File Reference

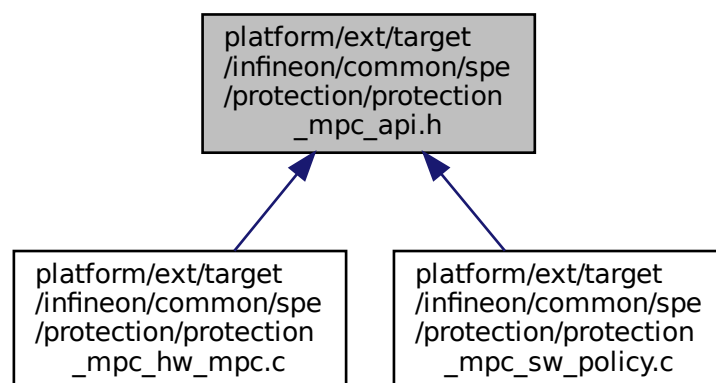
This file contains MPC driver API declaration.

```
#include "fih.h"
#include "load/asset_defs.h"
#include "protection_types.h"
#include "tfm_hal_defs.h"
```

Include dependency graph for protection\_mpc\_api.h:



This graph shows which files directly or indirectly include this file:



### Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_mpc\_init\_cfg(void)

*Configures the Memory Protection Controller.*

## Variables

- uintptr\_t [base](#)
- uintptr\_t size\_t [size](#)
- uintptr\_t size\_t uint32\_t [access\\_type](#)

### 8.157.1 Detailed Description

This file contains MPC driver API declaration.

It's expected that MPC driver implementation follows API declared in this file.

### 8.157.2 Function Documentation

#### 8.157.2.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures the Memory Protection Controller.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check access to memory region by MPC.

#### Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Definition at line 122 of file faults.c.

### 8.157.3 Variable Documentation

#### 8.157.3.1 access\_type

```
uintptr_t size_t uint32_t access_type
```

Definition at line 82 of file protection\_mpc\_api.h.

### 8.157.3.2 base

uintptr\_t base

Definition at line 80 of file protection\_mpc\_api.h.

### 8.157.3.3 size

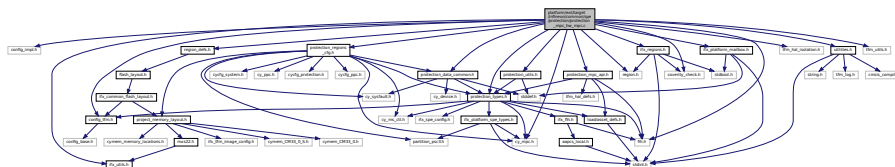
uintptr\_t size\_t size

Definition at line 81 of file protection\_mpc\_api.h.

## 8.158 platform/ext/target/infineon/common/spe/protection/protection\_mpc\_hw\_mpc.c File Reference

```
#include "config_impl.h"
#include "config_tfm.h"
#include "cy_mpc.h"
#include "fih.h"
#include "region_defs.h"
#include "protection_data_common.h"
#include "protection_types.h"
#include "protection_utils.h"
#include "protection_regions_cfg.h"
#include "protection_mpc_api.h"
#include "region.h"
#include "ifx_platform_mailbox.h"
#include "ifx_regions.h"
#include "ifx_utils.h"
#include "tfm_hal_isolation.h"
#include "utilities.h"
#include "coverity_check.h"
#include "tfm_utils.h"
```

Include dependency graph for protection\_mpc\_hw\_mpc.c:



## Macros

- `#define IFX_MPC_BLOCK_SIZE_TO_BYTES(mpc_size) (1UL << ((uint32_t)(mpc_size) + 5UL))`
- `#define IFX_MPC_BLK_CFG_BLOCK_SIZE_Pos 0UL`
- `#define IFX_MPC_BLK_CFG_BLOCK_SIZE_Msk 0xFUL`

## Functions

- `FIH_RET_TYPE` (enum tfm\_hal\_status\_t) `ifx_mpc_fill_fixed_config(void)`  
*Configures fault handlers and sets priorities.*
- `FIH_CALL` (ifx\_mpc\_apply\_configuration, fih\_rc, &mpc\_reg\_cfg)
- `FIH_RET` (fih\_rc)
- enum tfm\_hal\_status\_t `ifx_mpc_apply_configuration_with_mpc` (const ifx\_mpc\_raw\_region\_config\_t \*mpc\_reg\_cfg)  
*Apply MPC configuration for MPC controller handled by TF-M.*

- `if ((res != TFM_HAL_SUCCESS) || (split_count == 0U))`
- `for (i=0; i < split_count; i++)`
- `FIH_RET (TFM_HAL_SUCCESS)`

## Variables

- `ifx_mpc_region_config_t ifx_fixed_mpc_static_config [IFX_MAX_FIXED_CONFIGS]`
- `uint32_t ifx_fixed_mpc_static_config_count = 0`
- `const struct asset_desc_t * asset`
- `ifx_mpc_region_config_t mpc_reg_cfg`
- `mpc_reg_cfg address = asset->mem.start`
- `mpc_reg_cfg size = asset->mem.limit - asset->mem.start`
- `mpc_reg_cfg ns_mask = mpc_cfg->ns_mask`
- `mpc_reg_cfg r_mask = mpc_cfg->pc_mask`
- `mpc_reg_cfg w_mask`
- `uintptr_t base`
- `uintptr_t size_t uint32_t access_type`
- `uint32_t split_count = ARRAY_SIZE(splits)`
- `enum tfm_hal_status_t res`
- `size_t i`

## 8.158.1 Macro Definition Documentation

### 8.158.1.1 IFX\_MPC\_BLK\_CFG\_BLOCK\_SIZE\_Msk

`#define IFX_MPC_BLK_CFG_BLOCK_SIZE_Msk 0xFUL`  
 Definition at line 344 of file `protection_mpc_hw_mpc.c`.

### 8.158.1.2 IFX\_MPC\_BLK\_CFG\_BLOCK\_SIZE\_Pos

`#define IFX_MPC_BLK_CFG_BLOCK_SIZE_Pos 0UL`  
 Definition at line 343 of file `protection_mpc_hw_mpc.c`.

### 8.158.1.3 IFX\_MPC\_BLOCK\_SIZE\_TO\_BYTES

`#define IFX_MPC_BLOCK_SIZE_TO_BYTES(`  
     `mpc_size ) (1UL << ((uint32_t)(mpc_size) + 5UL))`  
 Definition at line 341 of file `protection_mpc_hw_mpc.c`.

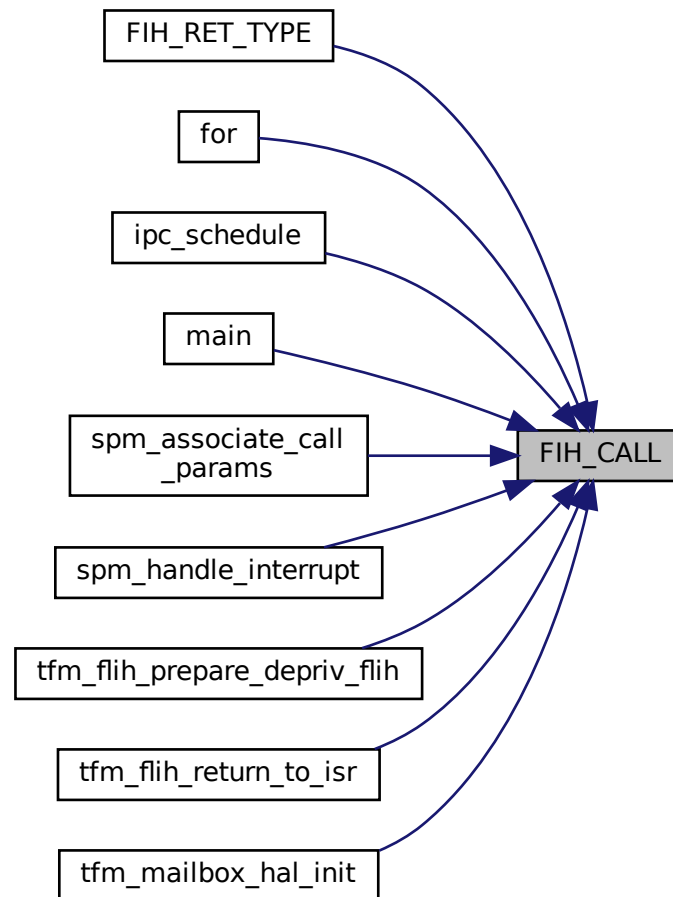
## 8.158.2 Function Documentation

### 8.158.2.1 FIH\_CALL()

```
FIH_CALL (
 ifx_mpc_apply_configuration ,
 fih_rc ,
 & mpc_reg_cfg)
```



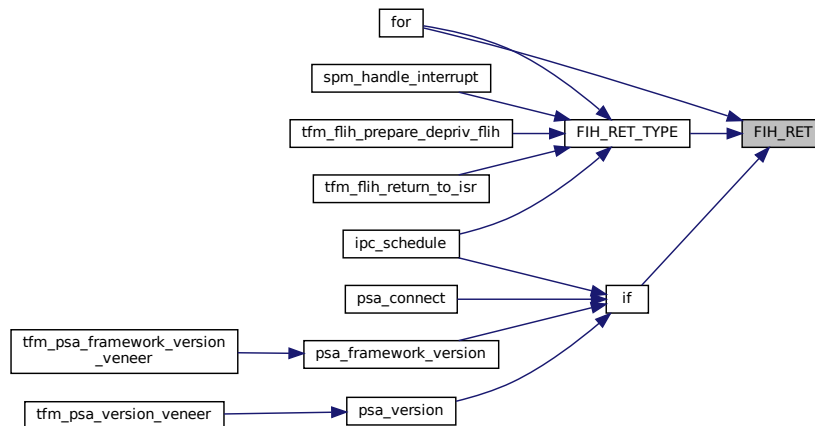
Here is the caller graph for this function:



### 8.158.2.2 FIH\_RET() [1/2]

```
FIH_RET (
 fih_rc)
```

Here is the caller graph for this function:



### 8.158.2.3 FIH\_RET() [2/2]

```

FIH_RET (
 TFM_HAL_SUCCESS
)

```

### 8.158.2.4 FIH\_RET\_TYPE()

```

FIH_RET_TYPE (
 enum tfm_hal_status_t
)

```

Configures fault handlers and sets priorities.  
 Configures the system reset request properties.  
 Configures the SAU.  
 Initialize Protection Context switching.  
 Configures MSC.  
 Populate the internal static MPC configuration array.  
 This function enables platform specific faults interrupts.  
 This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.  
 Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)  
 Apply a configuration to assets of a partition.  
 Apply a configuration to specified assets of a partition.  
 Apply PPC protection to named MMIO asset.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.  
 Check memory access permissions using MPC attribute cache.  
 Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.

Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

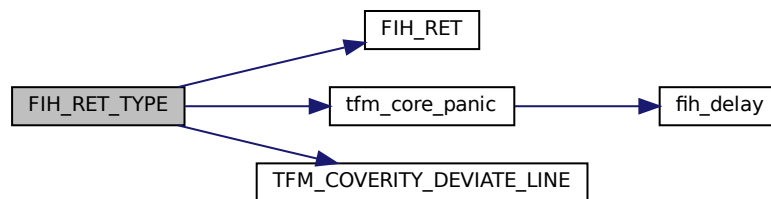
|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Definition at line 34 of file protection\_mpc\_hw\_mpc.c.

Here is the call graph for this function:



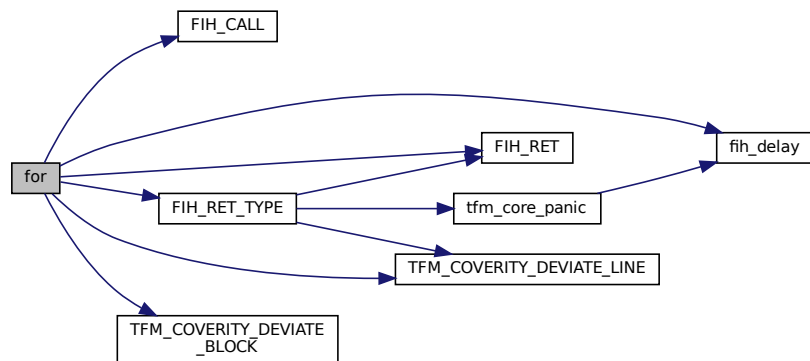
Here is the caller graph for this function:

**8.158.2.5 for()**

`for ( )`

Definition at line 961 of file protection\_mpc\_hw\_mpc.c.

Here is the call graph for this function:

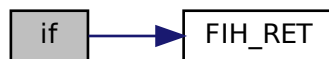


#### 8.158.2.6 if()

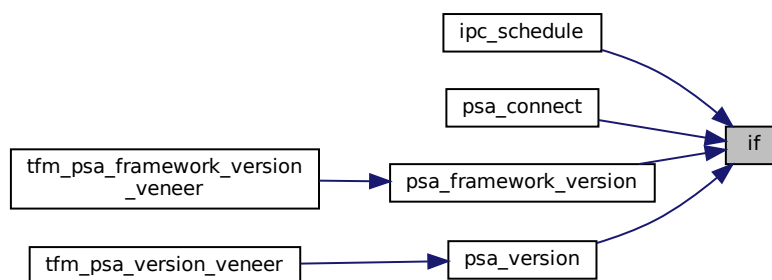
```
if (
 (res !=TFM_HAL_SUCCESS) || (split_count==0U))
```

Definition at line 932 of file protection\_mpc\_hw\_mpc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.158.2.7 ifx\_mpc\_apply\_configuration\_with\_mpc()

```
enum tfm_hal_status_t ifx_mpc_apply_configuration_with_mpc (
 const ifx_mpc_raw_region_config_t * mpc_reg_cfg)
```

Apply MPC configuration for MPC controller handled by TF-M.

#### Parameters

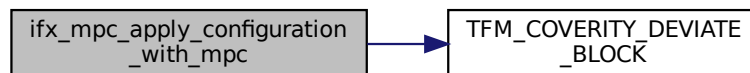
|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration.

Definition at line 455 of file protection\_mpc\_hw\_mpc.c.

Here is the call graph for this function:



## 8.158.3 Variable Documentation

### 8.158.3.1 access\_type

```
uintptr_t size_t uint32_t access_type
```

#### Initial value:

```
{
 ifx_memory_region_split_t splits[IFX_MAX_SPLIT_REGIONS_COUNT] = {0}
```

Definition at line 879 of file protection\_mpc\_hw\_mpc.c.

### 8.158.3.2 address

```
mpc_reg_cfg address = asset->mem.start
```

Definition at line 151 of file protection\_mpc\_hw\_mpc.c.

### 8.158.3.3 asset

```
const struct asset_desc_t* asset
```

#### Initial value:

```
{
 TFM_COVERITY_DEVIATE_LINE(MISRA_C_2023_Rule_10_1, "Definition of FIH_SET() treats fih_delay() returned
 int as a bool")
 IFX_FIH_DECLARE(enum tfm_hal_status_t, fih_rc, TFM_HAL_ERROR_GENERIC)
```

Definition at line 143 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.4 base**

```
uintptr_t base
```

Definition at line 876 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.5 i**

```
size_t i
```

Definition at line 960 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.6 ifx\_fixed\_mpc\_static\_config**

```
ifx_mpc_region_config_t ifx_fixed_mpc_static_config[IFX_MAX_FIXED_CONFIGS]
```

Definition at line 31 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.7 ifx\_fixed\_mpc\_static\_config\_count**

```
uint32_t ifx_fixed_mpc_static_config_count = 0
```

Definition at line 32 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.8 mpc\_reg\_cfg**

```
ifx_mpc_region_config_t mpc_reg_cfg
```

Definition at line 149 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.9 ns\_mask**

```
mpc_reg_cfg ns_mask = mpc_cfg->ns_mask
```

Definition at line 154 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.10 r\_mask**

```
mpc_reg_cfg r_mask = mpc_cfg->pc_mask
```

Definition at line 156 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.11 res**

```
enum tfm_hal_status_t res
```

**Initial value:**

```
= ifx_split_memory_region_across_mpcs(base, size,
```

```
ifx_memory_cm33_config,
ifx_memory_cm33_config_count,
splits, &split_count)
```

Definition at line 928 of file protection\_mpc\_hw\_mpc.c.

**8.158.3.12 size**

```
uintptr_t size_t size = asset->mem.limit - asset->mem.start
```

Definition at line 152 of file protection\_mpc\_hw\_mpc.c.



### 8.158.3.13 split\_count

uint32\_t split\_count = ARRAY\_SIZE(splits)  
 Definition at line 881 of file protection\_mpc\_hw\_mpc.c.

### 8.158.3.14 w\_mask

mpc\_reg\_cfg w\_mask

**Initial value:**

```
= (((uint32_t)(asset->attr)) & ((uint32_t)(ASSET_ATTR_READ_WRITE))) != 0U) ?
 mpc_cfg->pc_mask : 0U
```

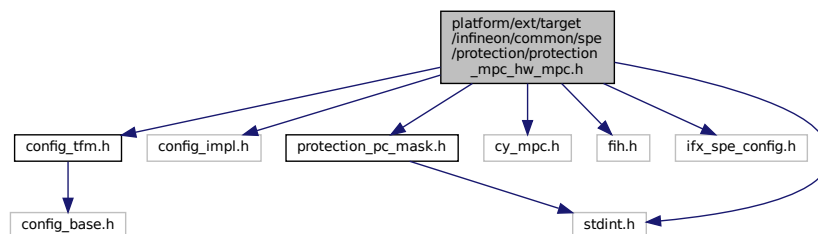
Definition at line 157 of file protection\_mpc\_hw\_mpc.c.

## 8.159 platform/ext/target/infineon/common/spe/protection/protection\_mpc\_hw\_mpc.h File Reference

This file contains types/API used by HW MPC driver, see cy\_mpc.h in PDL.

```
#include <stdint.h>
#include "config_impl.h"
#include "config_tfm.h"
#include "cy_mpc.h"
#include "fih.h"
#include "ifx_spe_config.h"
#include "protection_pc_mask.h"
```

Include dependency graph for protection\_mpc\_hw\_mpc.h:



## Data Structures

- struct [ifx\\_mpc\\_numbered\\_mmio\\_config\\_t](#)
- struct [ifx\\_mpc\\_region\\_config\\_t](#)
- struct [ifx\\_mpc\\_raw\\_region\\_config\\_t](#)

*Configuration structure for MPC raw region.*

## Macros

- #define [IFX\\_MPC\\_APPLY\\_ALL\\_PCS](#) ((ifx\_pc\_mask\_t)0xFFFFFFFFU)
- #define [IFX\\_MPC\\_NAMED\\_MMIO\\_SPM\\_ROT\\_CFG](#)
- #define [IFX\\_MPC\\_NAMED\\_MMIO\\_PSA\\_ROT\\_CFG](#)
- #define [IFX\\_MPC\\_NAMED\\_MMIO\\_APP\\_ROT\\_CFG](#)
- #define [IFX\\_MAX\\_FIXED\\_CONFIGS](#) 6U

## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_mpc\_fill\_fixed\_config(void)  
*Populate the internal static MPC configuration array.*
- enum tfm\_hal\_status\_t [ifx\\_mpc\\_apply\\_configuration\\_with\\_mpc](#) (const [ifx\\_mpc\\_raw\\_region\\_config\\_t](#) \*mpc\_reg\_cfg)  
*Apply MPC configuration for MPC controller handled by TF-M.*

## Variables

- [ifx\\_mpc\\_region\\_config\\_t](#) ifx\_fixed\_mpc\_static\_config [6U]
- uint32\_t [ifx\\_fixed\\_mpc\\_static\\_config\\_count](#)

### 8.159.1 Detailed Description

This file contains types/API used by HW MPC driver, see cy\_mpc.h in PDL.

#### Note

Don't include this file directly, include [protection\\_types.h](#) instead !!! It's expected that this file is included by [protection\\_types.h](#) if platform is configured to use HW MPC driver via IFX\_MPC\_DRIVER\_HW\_MPC option.

### 8.159.2 Macro Definition Documentation

#### 8.159.2.1 IFX\_MAX\_FIXED\_CONFIGS

```
#define IFX_MAX_FIXED_CONFIGS 6U
```

Definition at line 110 of file protection\_mpc\_hw\_mpc.h.

#### 8.159.2.2 IFX\_MPC\_APPLY\_ALL\_PCS

```
#define IFX_MPC_APPLY_ALL_PCS ((ifx_pc_mask_t)0xFFFFFFFFFU)
```

Definition at line 59 of file protection\_mpc\_hw\_mpc.h.

#### 8.159.2.3 IFX\_MPC\_NAMED\_MMIO\_APP\_ROT\_CFG

```
#define IFX_MPC_NAMED_MMIO_APP_ROT_CFG
```

##### Value:

```
.mpc_cfg = { \
 .pc_mask = IFX_PC_DEFAULT | IFX_PC_TFM_AROT, \
 .ns_mask = IFX_PC_NONE, \
},
```

Definition at line 85 of file protection\_mpc\_hw\_mpc.h.

#### 8.159.2.4 IFX\_MPC\_NAMED\_MMIO\_PSA\_ROT\_CFG

```
#define IFX_MPC_NAMED_MMIO_PSA_ROT_CFG
```

##### Value:

```
.mpc_cfg = { \
 .pc_mask = IFX_PC_DEFAULT | IFX_PC_TFM_PROT, \
 .ns_mask = IFX_PC_NONE, \
},
```

Definition at line 69 of file protection\_mpc\_hw\_mpc.h.

### 8.159.2.5 IFX\_MPC\_NAMED\_MMIO\_SPM\_ROT\_CFG

```
#define IFX_MPC_NAMED_MMIO_SPM_ROT_CFG
```

**Value:**

```
.mpc_cfg = { \
 .pc_mask = IFX_PC_DEFAULT, \
 .ns_mask = IFX_PC_NONE, \
},
```

Definition at line 62 of file protection\_mpc\_hw\_mpc.h.

## 8.159.3 Function Documentation

### 8.159.3.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Populate the internal static MPC configuration array.

Apply MPC configuration using raw interface.

Apply MPC configuration.

This function fills the `ifx_fixed_mpc_static_config` array with additional MPC region configurations that are internal to TF-M and cannot be determined by the compiler as constants. It also sets the `ifx_fixed_mpc_static_config_count` to the number of valid entries. The function may call [tfm\\_core\\_panic\(\)](#) if the number of configurations exceeds `IFX_MAX_FIXED_CONFIGS`.

#### Note

The actual regions and permissions depend on build-time configuration and linker symbols.

#### Returns

`TFM_HAL_SUCCESS` - MPC configuration applied successfully.

#### Parameters

|                 |                          |                                                                                                                  |
|-----------------|--------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>in</code> | <code>mpc_reg_cfg</code> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|-----------------|--------------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

`TFM_HAL_SUCCESS` - MPC configuration applied successfully. `TFM_HAL_ERROR_NOT_SUPPORTED` - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. `TFM_HAL_ERROR_GENERIC` - failed to apply MPC configuration. `TFM_HAL_ERROR_INVALID_INPUT` - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|                 |                          |                                                                                                                      |
|-----------------|--------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>in</code> | <code>mpc_reg_cfg</code> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|-----------------|--------------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

`TFM_HAL_SUCCESS` - MPC configuration applied successfully. `TFM_HAL_ERROR_NOT_SUPPORTED` - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. `TFM_HAL_ERROR_GENERIC` - failed to apply MPC configuration. `TFM_HAL_ERROR_INVALID_INPUT` - invalid input (e.g. provided memory region is not fully located in any known memory).

Populate the internal static MPC configuration array.  
Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

## Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

## Parameters

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Populate the internal static MPC configuration array.  
 Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Populate the internal static MPC configuration array.  
 This function enables platform specific faults interrupts.  
 This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.  
 Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)  
 Apply a configuration to assets of a partition.  
 Apply a configuration to specified assets of a partition.  
 Apply PPC protection to named MMIO asset.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.  
 Check memory access permissions using MPC attribute cache.  
 Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.



Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

## Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

## Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

## Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

#### Returns

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

#### Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

#### Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.  
 Check memory access permissions using MPC attribute cache.  
 Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                    |
|----|----------------------|------------------------------------|
| in | <i>p_info</i>        | Partition information structure.   |
| in | <i>memory_config</i> | Memory configuration structure.    |
| in | <i>base</i>          | Base address of the memory region. |

**Parameters**

|    |                    |                                                       |
|----|--------------------|-------------------------------------------------------|
| in | <i>size</i>        | Size of the memory region.                            |
| in | <i>access_type</i> | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted.  
 TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Populate the internal static MPC configuration array.  
 Configures fault handlers and sets priorities.  
 Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully.  
 TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset.  
 TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Populate the internal static MPC configuration array.  
 This function enables platform specific faults interrupts.  
 This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.  
 Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)  
 Apply a configuration to assets of a partition.  
 Apply a configuration to specified assets of a partition.  
 Apply PPC protection to named MMIO asset.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.  
 Check memory access permissions using MPC attribute cache.  
 Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |               |                                                                  |
|----|---------------|------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a> . |
|----|---------------|------------------------------------------------------------------|



## Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

## Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

## Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

## Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

## Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

## Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

## Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully.  
 TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset.  
 TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

#### Returns

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

#### Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

#### Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 122 of file faults.c.

### 8.159.3.2 ifx\_mpc\_apply\_configuration\_with\_mpc()

```
enum tfm_hal_status_t ifx_mpc_apply_configuration_with_mpc (
 const ifx_mpc_raw_region_config_t * mpc_reg_cfg)
```

Apply MPC configuration for MPC controller handled by TF-M.

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration.

Definition at line 455 of file protection\_mpc\_hw\_mpc.c.

Here is the call graph for this function:



## 8.159.4 Variable Documentation

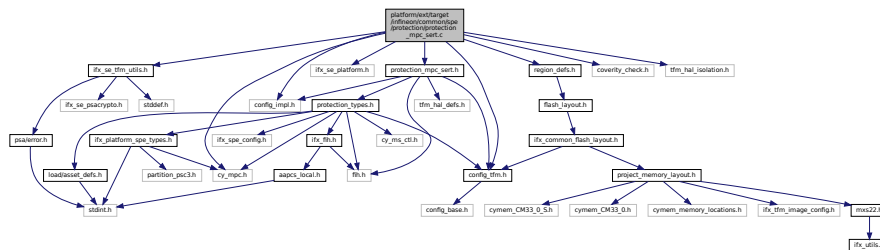
### 8.159.4.1 ifx\_fixed\_mpc\_static\_config

```
ifx_mpc_region_config_t ifx_fixed_mpc_static_config[6U]
```

Definition at line 31 of file protection\_mpc\_hw\_mpc.c.

uint32\_t ifx\_fixed\_mpc\_static\_config\_count  
Definition at line 32 of file protection\_mpc\_hw\_mpc.c.

```
#include "config_impl.h"
#include "config_tfm.h"
#include "cy_mpc.h"
#include "ifx_se_platform.h"
#include "ifx_se_tfm_utils.h"
#include "protection_mpc_sert.h"
#include "region_defs.h"
#include "coverity_check.h"
#include "tfm_hal_isolation.h"
Include dependency graph for protection_mpc_sert.c:
```



- struct `ifx_mpc_ext_cache_cfg_info_t`

- #define IFX\_MPC\_EXT\_CACHE\_ATTR\_DEFAULT (0U)
- #define IFX\_MPC\_SERT\_DEFAULT\_ATTR IFX\_MPC\_EXT\_CACHE\_PC\_BIT(IFX\_PC\_TFM\_SPM\_ID, IFX\_MPC\_EXT\_CACHE\_READ\_BIT)

- enum tfm\_hal\_status\_t ifx\_mpc\_sert\_apply\_configuration (const ifx\_mpc\_raw\_region\_config\_t \*mpc\_reg\_cfg)
  - Apply MPC configuration for MPC controller that is handled by optional SE RT Service or other non-TF-M service/hardware.
- FIH\_RET\_TYPE (enum tfm\_hal\_status\_t) ifx\_mpc\_sert\_memory\_check(const struct ifx\_partition\_info\_t \*p\_info)
  - if (sert\_mem==NULL)
  - if (memory\_config->mpc\_block\_size !=sert\_mem->mem->mpc\_block\_size)
  - if (((first\_block\_idx >=block\_count)|| (last\_block\_idx > block\_count))
  - if ((access\_type &TFM\_HAL\_ACCESS\_READABLE) !=0U)
  - if ((access\_type &TFM\_HAL\_ACCESS\_WRITABLE) !=0U)
  - for (uint32\_t block\_id=first\_block\_idx;block\_id< last\_block\_idx;block\_id++)
  - FIH\_RET (TFM\_HAL\_SUCCESS)

## Variables

- `const ifx_memory_config_t * memory_config`
- `const ifx_memory_config_t uintptr_t base`
- `const ifx_memory_config_t uintptr_t size_t size`
- `const ifx_memory_config_t uintptr_t size_t uint32_t access_type`
- `uint32_t block_size = (1UL << (((uint32_t)( memory_config->mpc_block_size ) + 5UL))`
- `uint32_t block_count = sert_mem->block_count`
- `uint32_t offset = IFX_S_ADDRESS_ALIAS(base) - memory_config->s_address`
- `uint32_t first_block_idx = offset / block_size`
- `uint32_t last_block_idx`
- `bool is_secure = (access_type & TFM_HAL_ACCESS_NS) == 0U`
- `uint32_t pc = (uint32_t)fih_int_decode(fih_int_encode(IFX_PC_TFM_SPM_ID))`
- `uint32_t mask = (1UL << (((uint32_t)( pc ) * (4U) ) + (uint32_t)( 0U) )))`
- `uint32_t attr = is_secure ? 0U : (1UL << (((uint32_t)( pc ) * (4U) ) + (uint32_t)( 0U) )))`

## 8.160.1 Macro Definition Documentation

### 8.160.1.1 IFX\_MPC\_EXT\_CACHE\_ATTR\_DEFAULT

```
#define IFX_MPC_EXT_CACHE_ATTR_DEFAULT (0U)
```

Definition at line 20 of file `protection_mpc_sert.c`.

### 8.160.1.2 IFX\_MPC\_SERT\_DEFAULT\_ATTR

```
#define IFX_MPC_SERT_DEFAULT_ATTR IFX_MPC_EXT_CACHE_PC_BIT(IFX_PC_TFM_SPM_ID, IFX_MPC_EXT_CACHE_READ_BIT)
```

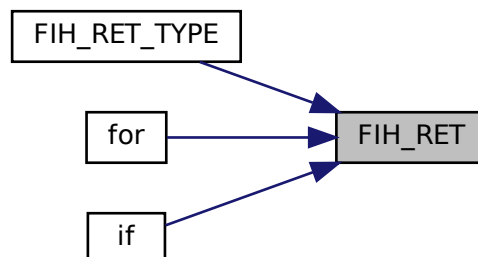
Definition at line 24 of file `protection_mpc_sert.c`.

## 8.160.2 Function Documentation

### 8.160.2.1 FIH\_RET()

```
FIH_RET (
 TFM_HAL_SUCCESS)
```

Here is the caller graph for this function:

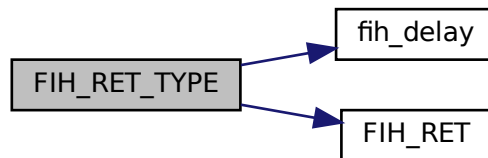


### 8.160.2.2 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t) const
```

Definition at line 374 of file protection\_mpc\_sert.c.

Here is the call graph for this function:

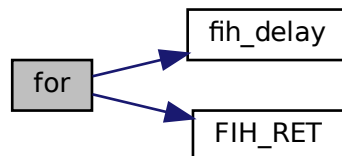


### 8.160.2.3 for()

```
for ()
```

Definition at line 346 of file protection\_mpc\_sert.c.

Here is the call graph for this function:



### 8.160.2.4 if() [1/5]

```
if (
 (access_type &TFM_HAL_ACCESS_READABLE) != 0U)
```

Definition at line 304 of file protection\_mpc\_sert.c.

### 8.160.2.5 if() [2/5]

```
if (
 (access_type &TFM_HAL_ACCESS_WRITABLE) != 0U)
```

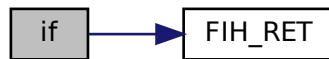
Definition at line 309 of file protection\_mpc\_sert.c.

**8.160.2.6 if()** [3/5]

```
if (
 (first_block_idx >= block_count) || (last_block_idx > block_count))
```

Definition at line 291 of file protection\_mpc\_sert.c.

Here is the call graph for this function:

**8.160.2.7 if()** [4/5]

```
if (
 memory_config->mpc_block_size != sert_mem->mem->mpc_block_size)
```

Definition at line 280 of file protection\_mpc\_sert.c.

Here is the call graph for this function:

**8.160.2.8 if()** [5/5]

```
if (
 sert_mem == NULL)
```

Definition at line 276 of file protection\_mpc\_sert.c.

Here is the call graph for this function:





### 8.160.2.9 ifx\_mpc\_sert\_apply\_configuration()

```
enum tfm_hal_status_t ifx_mpc_sert_apply_configuration (
 const ifx_mpc_raw_region_config_t * mpc_reg_cfg)
```

Apply MPC configuration for MPC controller that is handled by optional SE RT Service or other non-TF-M service/hardware.

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not supported by external service).

Definition at line 131 of file protection\_mpc\_sert.c.

## 8.160.3 Variable Documentation

### 8.160.3.1 access\_type

```
const ifx_memory_config_t uintptr_t size_t uint32_t access_type
```

#### Initial value:

```
{
 if (memory_config == NULL) {
 FIH_RET(TFM_HAL_ERROR_MEM_FAULT);
 }

 const ifx_mpc_sert_mem_t *sert_mem = ifx_mpc_sert_find_mem(memory_config->mpc)
```

Definition at line 269 of file protection\_mpc\_sert.c.

### 8.160.3.2 attr

```
uint32_t attr = is_secure ? 0U : (1UL << (((uint32_t)(pc) * (4U)) + (uint32_t)(0U)))
```

Definition at line 302 of file protection\_mpc\_sert.c.

### 8.160.3.3 base

```
const ifx_memory_config_t uintptr_t base
```

Definition at line 266 of file protection\_mpc\_sert.c.

### 8.160.3.4 block\_count

```
uint32_t block_count = sert_mem->block_count
```

Definition at line 285 of file protection\_mpc\_sert.c.

### 8.160.3.5 block\_size

```
uint32_t block_size = (1UL << ((uint32_t)(memory_config->mpc_block_size) + 5UL))
```

Definition at line 284 of file protection\_mpc\_sert.c.

### 8.160.3.6 first\_block\_idx

```
uint32_t first_block_idx = offset / block_size
```

Definition at line 287 of file protection\_mpc\_sert.c.

### 8.160.3.7 is\_secure

```
bool is_secure = (access_type & TFM_HAL_ACCESS_NS) == 0U
```

Definition at line 296 of file protection\_mpc\_sert.c.

### 8.160.3.8 last\_block\_idx

```
uint32_t last_block_idx
```

**Initial value:**

```
= IFX_ROUND_UP_TO_MULTIPLE(offset + size, block_size)
/ block_size
```

Definition at line 288 of file protection\_mpc\_sert.c.

### 8.160.3.9 mask

```
uint32_t mask = (1UL << (((uint32_t)(pc) * (4U)) + (uint32_t)(0U))))
```

Definition at line 301 of file protection\_mpc\_sert.c.

### 8.160.3.10 memory\_config

```
const ifx_memory_config_t* memory_config
```

Definition at line 265 of file protection\_mpc\_sert.c.

### 8.160.3.11 offset

```
uint32_t offset = IFX_S_ADDRESS_ALIAS(base) - memory_config->s_address
```

Definition at line 286 of file protection\_mpc\_sert.c.

### 8.160.3.12 pc

```
uint32_t pc = (uint32_t) fih_int_decode(fih_int_encode(IFX_PC_TFM_SPM_ID))
```

Definition at line 298 of file protection\_mpc\_sert.c.

### 8.160.3.13 size

```
const ifx_memory_config_t uintptr_t size_t size
```

Definition at line 267 of file protection\_mpc\_sert.c.

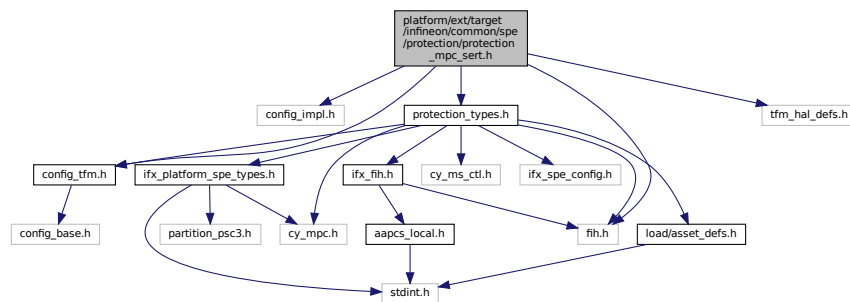
## 8.161 platform/ext/target/infineon/common/spe/protection/protection\_mpc\_sert.h File Reference

This file contains the API used by HW MPC driver, to protect memory via SE RT Basic/Full.

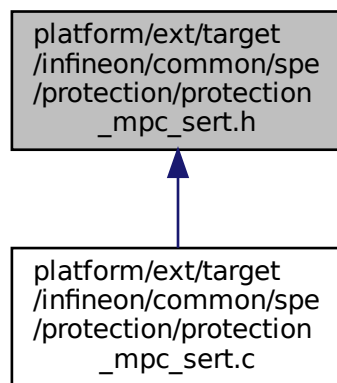
```
#include "config_impl.h"
#include "config_tfm.h"
#include "fih.h"
#include "tfm_hal_defs.h"
```

```
#include "protection_types.h"
```

Include dependency graph for protection\_mpc\_sert.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ifx\\_mpc\\_sert\\_mem\\_t](#)

*Descriptor of an external memory whose MPC is handled by SE RT Services.*

## Macros

- #define [IFX\\_MPC\\_EXT\\_BLOCK\\_SIZE\\_TO\\_BYTES](#)(mpc\_size) (1UL << ((uint32\_t)(mpc\_size) + 5UL))
- #define [IFX\\_MPC\\_EXT\\_CACHE\\_PC\\_FIELD\\_BITS](#) (4U)
- #define [IFX\\_MPC\\_EXT\\_CACHE\\_PC\\_FIELD\\_SHIFT](#)(pc) ((uint32\_t)(pc) \* IFX\_MPC\_EXT\_CACHE\_PC\_FIELD\_BITS)
- #define [IFX\\_MPC\\_EXT\\_CACHE\\_NS\\_BIT](#) (0U)
- #define [IFX\\_MPC\\_EXT\\_CACHE\\_READ\\_BIT](#) (1U)
- #define [IFX\\_MPC\\_EXT\\_CACHE\\_WRITE\\_BIT](#) (2U)
- #define [IFX\\_MPC\\_EXT\\_CACHE\\_PC\\_BIT](#)(pc, bit) (1UL << (IFX\_MPC\_EXT\_CACHE\_PC\_FIELD\_SHIFT(pc) + (uint32\_t)(bit)))
- #define [IFX\\_MPC\\_SERT\\_CACHE\\_SIZE](#)(mem\_size, block\_size) ((mem\_size) / IFX\_MPC\_EXT\_BLOCK\_SIZE\_TO\_BYTES(block\_size))

## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_mpc\_sert\_init(void)  
*Initialize the MPC attribute caches for SE RT Services.*
- enum tfm\_hal\_status\_t [ifx\\_mpc\\_sert\\_apply\\_configuration](#) (const ifx\_mpc\_raw\_region\_config\_t \*mpc\_reg\_cfg)  
*Apply MPC configuration for MPC controller that is handled by optional SE RT Service or other non-TF-M service/hardware.*

## Variables

- const [ifx\\_mpc\\_sert\\_mem\\_t](#) ifx\_mpc\_sert\_memories []
- const size\_t [ifx\\_mpc\\_sert\\_memories\\_count](#)
- const ifx\_memory\_config\_t \* [memory\\_config](#)
- const ifx\_memory\_config\_t uintptr\_t [base](#)
- const ifx\_memory\_config\_t uintptr\_t size\_t [size](#)
- const ifx\_memory\_config\_t uintptr\_t size\_t uint32\_t [access\\_type](#)

### 8.161.1 Detailed Description

This file contains the API used by HW MPC driver, to protect memory via SE RT Basic/Full.

### 8.161.2 Macro Definition Documentation

#### 8.161.2.1 IFX\_MPC\_EXT\_BLOCK\_SIZE\_TO\_BYTES

```
#define IFX_MPC_EXT_BLOCK_SIZE_TO_BYTES(
 mpc_size) (1UL << ((uint32_t)(mpc_size) + 5UL))
```

Definition at line 46 of file protection\_mpc\_sert.h.

#### 8.161.2.2 IFX\_MPC\_EXT\_CACHE\_NS\_BIT

```
#define IFX_MPC_EXT_CACHE_NS_BIT (0U)
```

Definition at line 55 of file protection\_mpc\_sert.h.

#### 8.161.2.3 IFX\_MPC\_EXT\_CACHE\_PC\_BIT

```
#define IFX_MPC_EXT_CACHE_PC_BIT(
 pc,
 bit) (1UL << (IFX_MPC_EXT_CACHE_PC_FIELD_SHIFT(pc) + (uint32_t)(bit)))
```

Definition at line 60 of file protection\_mpc\_sert.h.

#### 8.161.2.4 IFX\_MPC\_EXT\_CACHE\_PC\_FIELD\_BITS

```
#define IFX_MPC_EXT_CACHE_PC_FIELD_BITS (4U)
```

Definition at line 49 of file protection\_mpc\_sert.h.

#### 8.161.2.5 IFX\_MPC\_EXT\_CACHE\_PC\_FIELD\_SHIFT

```
#define IFX_MPC_EXT_CACHE_PC_FIELD_SHIFT(
 pc) ((uint32_t)(pc) * IFX_MPC_EXT_CACHE_PC_FIELD_BITS)
```

Definition at line 52 of file protection\_mpc\_sert.h.

### 8.161.2.6 IFX\_MPC\_EXT\_CACHE\_READ\_BIT

```
#define IFX_MPC_EXT_CACHE_READ_BIT (1U)
```

Definition at line 56 of file protection\_mpc\_sert.h.

### 8.161.2.7 IFX\_MPC\_EXT\_CACHE\_WRITE\_BIT

```
#define IFX_MPC_EXT_CACHE_WRITE_BIT (2U)
```

Definition at line 57 of file protection\_mpc\_sert.h.

### 8.161.2.8 IFX\_MPC\_SERT\_CACHE\_SIZE

```
#define IFX_MPC_SERT_CACHE_SIZE(
 mem_size,
 block_size) ((mem_size) / IFX_MPC_EXT_BLOCK_SIZE_TO_BYTES(block_size))
```

Definition at line 66 of file protection\_mpc\_sert.h.

## 8.161.3 Function Documentation

### 8.161.3.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Initialize the MPC attribute caches for SE RT Services.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Definition at line 122 of file faults.c.

### 8.161.3.2 ifx\_mpc\_sert\_apply\_configuration()

```
enum tfm_hal_status_t ifx_mpc_sert_apply_configuration (
 const ifx_mpc_raw_region_config_t * mpc_reg_cfg)
```

Apply MPC configuration for MPC controller that is handled by optional SE RT Service or other non-TF-M service/hardware.

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not supported by external service).

Definition at line 131 of file protection\_mpc\_sert.c.

## 8.161.4 Variable Documentation

### 8.161.4.1 access\_type

```
const ifx_memory_config_t uintptr_t size_t uint32_t access_type
```

Definition at line 139 of file protection\_mpc\_sert.h.

### 8.161.4.2 base

```
const ifx_memory_config_t uintptr_t base
```

Definition at line 137 of file protection\_mpc\_sert.h.

### 8.161.4.3 ifx\_mpc\_sert\_memories

```
const ifx_mpc_sert_mem_t ifx_mpc_sert_memories[]
```

Platform-provided table of external memories handled via SE RT Services.

### 8.161.4.4 ifx\_mpc\_sert\_memories\_count

```
const size_t ifx_mpc_sert_memories_count
```

Number of entries in [ifx\\_mpc\\_sert\\_memories](#).

### 8.161.4.5 memory\_config

```
const ifx_memory_config_t* memory_config
```

Definition at line 136 of file protection\_mpc\_sert.h.

### 8.161.4.6 size

```
const ifx_memory_config_t uintptr_t size_t size
```

Definition at line 138 of file protection\_mpc\_sert.h.

[illegible]

- struct ifx\_mpc\_cfg\_t
- struct ifx\_mpc\_policy\_t

- #define `IFX_MPC_BLOCK_SIZE` (0x800UL) /\* 2 KB \*/
- #define `IFX_MPC_PC_ATTR_MSK` (0xFUL) /\* Mask for MPC protection context \*/
- #define `IFX_MPC_PC_ATTR_SIZE_IN_BITS` (4UL) /\* Number of MPC attribute bits \*/
- #define `IFX_MPC_PC_ATTR_NS_MSK` (1UL << 0) /\* NS mask for MPC attributes for PC \*/
- #define `IFX_MPC_PC_ATTR_W_MSK` (1UL << 1) /\* Write mask for MPC attributes for PC \*/
- #define `IFX_MPC_PC_ATTR_R_MSK` (1UL << 2) /\* Read mask for MPC attributes for PC \*/
- #define `IFX_MPC_GET_PC_ATTR(attr, pc)`
- #define `CY_SRAM0_BASE_S` (IFX\_S\_ADDRESS\_ALIAS(CY\_SRAM0\_BASE))
- #define `CY_FLASH_BASE_S` (IFX\_S\_ADDRESS\_ALIAS(CY\_FLASH\_BASE))

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_mpc\_init\_cfg(void)  
*Configures fault handlers and sets priorities.*
- [TFM\\_COVERTITY\\_DEVIATE\\_LINE](#) (MISRA\_C\_2023\_Rule\_11\_3, "Intentional pointer type conversion, this maps MPC policy")
- if (ifx\_is\_region\_inside\_other(base\_s, base\_s+size,(IFX\_S\_ADDRESS\_ALIAS(CY\_SRAM0\_BASE)),(IFX\_S\_ADDRESS\_ALIAS(CY\_SRAM0\_BASE))+CY\_SRAM0\_SIZE))
- else if (ifx\_is\_region\_inside\_other(base\_s, base\_s+size,(IFX\_S\_ADDRESS\_ALIAS(CY\_FLASH\_BASE)),(IFX\_S\_ADDRESS\_ALIAS(CY\_FLASH\_BASE))+CY\_FLASH\_SIZE))
- for (uint32\_t mpc\_idx=mpc\_start\_idx;mpc\_idx< mpc\_end\_idx;mpc\_idx++)
- [FIH\\_RET](#) (TFM HAL ERROR MEM FAULT)

## Variables

- uintptr\_t `base`
- uintptr\_t size\_t `size`
- uintptr\_t size\_t uint32\_t `access_type`
- uint32\_t `mpc_end_idx`
- uint32\_t `mpc_base_addr`
- const ifx\_mpc\_policy\_t \*const `mpc_policy` = (ifx\_mpc\_policy\_t\*) &SFLASH->N\_RAM\_MPC
- uintptr\_t `base_s` = IFX\_S\_ADDRESS\_ALIAS(base)
- `else`

## 8.162.1 Macro Definition Documentation

### 8.162.1.1 CY\_FLASH\_BASE\_S

```
#define CY_FLASH_BASE_S (IFX_S_ADDRESS_ALIAS(CY_FLASH_BASE))
```

Definition at line 38 of file protection\_mpc\_sw\_policy.c.

### 8.162.1.2 CY\_SRAM0\_BASE\_S

```
#define CY_SRAM0_BASE_S (IFX_S_ADDRESS_ALIAS(CY_SRAM0_BASE))
```

Definition at line 36 of file protection\_mpc\_sw\_policy.c.

### 8.162.1.3 IFX\_MPC\_BLOCK\_SIZE

```
#define IFX_MPC_BLOCK_SIZE (0x800UL) /* 2 KB */
```

Definition at line 21 of file protection\_mpc\_sw\_policy.c.

### 8.162.1.4 IFX\_MPC\_GET\_PC\_ATTR

```
#define IFX_MPC_GET_PC_ATTR(
 attr,
 pc)
```

Value:

```
((attr) > ((pc) * IFX_MPC_PC_ATTR_SIZE_IN_BITS)) \
& IFX_MPC_PC_ATTR_MSK)
```

Definition at line 32 of file protection\_mpc\_sw\_policy.c.

### 8.162.1.5 IFX\_MPC\_PC\_ATTR\_MSK

```
#define IFX_MPC_PC_ATTR_MSK (0xFUL) /* Mask for MPC protection context */
```

Definition at line 25 of file protection\_mpc\_sw\_policy.c.

### 8.162.1.6 IFX\_MPC\_PC\_ATTR\_NS\_MSK

```
#define IFX_MPC_PC_ATTR_NS_MSK (1UL << 0) /* NS mask for MPC attributes for PC */
```

Definition at line 27 of file protection\_mpc\_sw\_policy.c.

### 8.162.1.7 IFX\_MPC\_PC\_ATTR\_R\_MSK

```
#define IFX_MPC_PC_ATTR_R_MSK (1UL << 2) /* Read mask for MPC attributes for PC */
```

Definition at line 29 of file protection\_mpc\_sw\_policy.c.



### 8.162.1.8 IFX\_MPC\_PC\_ATTR\_SIZE\_IN\_BITS

```
#define IFX_MPC_PC_ATTR_SIZE_IN_BITS (4UL) /* Number of MPC attribute bits */
```

Definition at line 26 of file protection\_mpc\_sw\_policy.c.

### 8.162.1.9 IFX\_MPC\_PC\_ATTR\_W\_MSK

```
#define IFX_MPC_PC_ATTR_W_MSK (1UL << 1) /* Write mask for MPC attributes for PC */
```

Definition at line 28 of file protection\_mpc\_sw\_policy.c.

## 8.162.2 Function Documentation

### 8.162.2.1 FIH\_RET()

```
FIH_RET (
 TFM_HAL_ERROR_MEM_FAULT)
```

Here is the caller graph for this function:



### 8.162.2.2 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures fault handlers and sets priorities.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the tfm\_plat\_err\_t

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.

Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

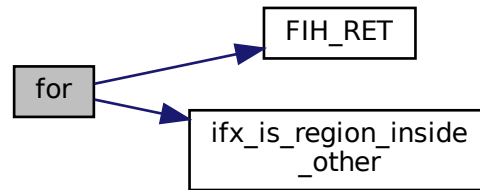
Definition at line 57 of file protection\_mpc\_sw\_policy.c.

**8.162.2.3 for()**

for ( )

Definition at line 148 of file protection\_mpc\_sw\_policy.c.

Here is the call graph for this function:



#### 8.162.2.4 if() [1/2]

```

else if (
 ifx_is_region_inside_other(base_s, base_s+size, (IFX_S_ADDRESS_ALIAS(CY_FLASH_BASE) + CY_FLASH_SIZE) - 1,
 (IFX_S_ADDRESS_ALIAS(CY_FLASH_BASE) + CY_FLASH_SIZE) - 1)
)

```

Definition at line 136 of file protection\_mpc\_sw\_policy.c.

#### 8.162.2.5 if() [2/2]

```

if (
 ifx_is_region_inside_other(base_s, base_s+size, (IFX_S_ADDRESS_ALIAS(CY_SRAM0_BASE) + CY_SRAM0_SIZE) - 1,
 (IFX_S_ADDRESS_ALIAS(CY_SRAM0_BASE) + CY_SRAM0_SIZE) - 1)
)

```

Definition at line 129 of file protection\_mpc\_sw\_policy.c.

#### 8.162.2.6 TFM\_COVERITY\_DEVIATE\_LINE()

```

TFM_COVERITY_DEVIATE_LINE (
 MISRA_C_2023_Rule_11_3 ,
 "Intentional pointer type conversion,
 this maps MPC policy")

```

### 8.162.3 Variable Documentation

#### 8.162.3.1 access\_type

```
uintptr_t size_t uint32_t access_type
```

**Initial value:**

```

{
 uint32_t mpc_start_idx

```

Definition at line 118 of file protection\_mpc\_sw\_policy.c.

#### 8.162.3.2 base

```
uintptr_t base
```

Definition at line 115 of file protection\_mpc\_sw\_policy.c.

### 8.162.3.3 base\_s

`uintptr_t base_s = IFX_S_ADDRESS_ALIAS(base)`  
 Definition at line 126 of file `protection_mpc_sw_policy.c`.

### 8.162.3.4 else

`else`  
**Initial value:**  
`{`  
     `FIH_RET(TFM_HAL_ERROR_INVALID_INPUT)`  
 Definition at line 143 of file `protection_mpc_sw_policy.c`.

### 8.162.3.5 mpc\_base\_addr

`uint32_t mpc_base_addr`  
 Definition at line 121 of file `protection_mpc_sw_policy.c`.

### 8.162.3.6 mpc\_end\_idx

`uint32_t mpc_end_idx`  
 Definition at line 120 of file `protection_mpc_sw_policy.c`.

### 8.162.3.7 mpc\_policy

`const ifx_mpc_policy_t* const mpc_policy = (ifx_mpc_policy_t*) &SFLASH->N_RAM_MPC`  
 Definition at line 124 of file `protection_mpc_sw_policy.c`.

### 8.162.3.8 size

`uintptr_t size_t size`  
 Definition at line 116 of file `protection_mpc_sw_policy.c`.

## 8.163 platform/ext/target/infineon/common/spe/protection/protection\_mpc\_sw\_policy.h File Reference

This file contains types/API used by MPC SW Policy driver.

### Macros

- `#define IFX_MPC_NAMED_MMIO_SPM_ROT_CFG`
- `#define IFX_MPC_NAMED_MMIO_PSA_ROT_CFG`
- `#define IFX_MPC_NAMED_MMIO_APP_ROT_CFG`

### 8.163.1 Detailed Description

This file contains types/API used by MPC SW Policy driver.

#### Note

Don't include this file directly, include `protection_types.h` instead !!! It's expected that this file is included by `protection_types.h` if platform is configured to use MPC SW Policy driver via `IFX_MPC_DRIVER_SW_POLICY` option.

## 8.163.2 Macro Definition Documentation

### 8.163.2.1 IFX\_MPC\_NAMED\_MMIO\_APP\_ROT\_CFG

```
#define IFX_MPC_NAMED_MMIO_APP_ROT_CFG
```

Definition at line 28 of file protection\_mpc\_sw\_policy.h.

### 8.163.2.2 IFX\_MPC\_NAMED\_MMIO\_PSA\_ROT\_CFG

```
#define IFX_MPC_NAMED_MMIO_PSA_ROT_CFG
```

Definition at line 25 of file protection\_mpc\_sw\_policy.h.

### 8.163.2.3 IFX\_MPC\_NAMED\_MMIO\_SPM\_ROT\_CFG

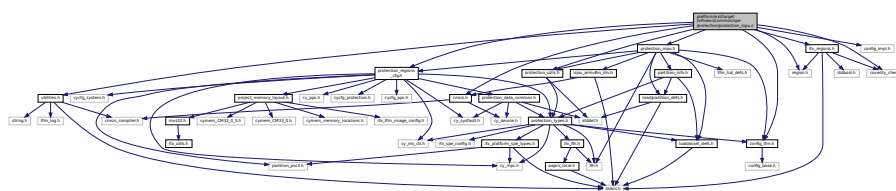
```
#define IFX_MPC_NAMED_MMIO_SPM_ROT_CFG
```

Definition at line 22 of file protection\_mpc\_sw\_policy.h.

## 8.164 platform/ext/target/infineon/common/spe/protection/protection\_mpu.c File Reference

```
#include "utilities.h"
#include "config_impl.h"
#include "config_tfm.h"
#include "cmsis.h"
#include "ifx_regions.h"
#include "protection_regions_cfg.h"
#include "protection_mpu.h"
#include "protection_utils.h"
#include "region.h"
#include "coverity_check.h"
```

Include dependency graph for protection\_mpu.c:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_mpu\_init\_cfg(void)  
*Configures fault handlers and sets priorities.*
- if (((((uint32\_t)(p\_asset->attr)) & ((uint32\_t)(ASSET\_ATTR\_READ\_WRITE))) != 0U))
- ifx\_mpu\_config\_enable\_region (ifx\_mpu\_used\_regions\_count, &mpu\_config)
- [FIH\\_RET](#) (TFM\_HAL\_SUCCESS)

## Variables

- const struct [asset\\_desc\\_t](#) \* p\_asset
- mpu\_config [region\\_base](#) = (uint32\_t)p\_asset->mem.start
- mpu\_config [region\\_limit](#) = (uint32\_t)p\_asset->mem.limit - 1U

- mpu\_config attr\_exec = MPU\_ARMV8M\_XN\_EXEC\_NEVER
- else
- mpu\_config attr\_access = MPU\_ARMV8M\_AP\_RO\_PRIV\_UNPRIV
- mpu\_config attr\_sh = MPU\_ARMV8M\_SH\_NONE

## 8.164.1 Function Documentation

### 8.164.1.1 FIH\_RET()

```
FIH_RET (
 TFM_HAL_SUCCESS)
```

### 8.164.1.2 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures fault handlers and sets priorities.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |



**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Definition at line 199 of file protection\_mpu.c.

#### 8.164.1.3 if()

```
if (
 (((uint32_t) (p_asset->attr)) & ((uint32_t) (ASSET_ATTR_READ_WRITE))) != 0U))
```

Definition at line 374 of file protection\_mpu.c.

#### 8.164.1.4 ifx\_mpu\_config\_enable\_region()

```
ifx_mpu_config_enable_region (
 ifx_mpu_used_regions_count ,
 & mpu_config)
```

### 8.164.2 Variable Documentation

#### 8.164.2.1 attr\_access

```
mpu_config attr_access = MPU_ARMV8M_AP_RO_PRIV_UNPRIV
```

Definition at line 379 of file protection\_mpu.c.

#### 8.164.2.2 attr\_exec

```
mpu_config attr_exec = MPU_ARMV8M_XN_EXEC_NEVER
```

Definition at line 370 of file protection\_mpu.c.

#### 8.164.2.3 attr\_sh

```
mpu_config attr_sh = MPU_ARMV8M_SH_NONE
```

Definition at line 382 of file protection\_mpu.c.

### 8.164.2.4 else

else

**Initial value:**

```
{
 mpu_config.region_attridx = MPU_ARMV8M_MAIR_ATTR_CODE_IDX
```

Definition at line 377 of file protection\_mpu.c.

### 8.164.2.5 p\_asset

```
const struct asset_desc_t* p_asset
```

**Initial value:**

```
{
 if ((IFX_FIH_EQ((p_info)->ifx_ldinfo->privileged, IFX_FIH_TRUE))) {
 FIH_RET(TFM_HAL_SUCCESS);
 }
 if (ifx_mpu_used_regions_count >= CPUSS_CM33_0_MPU_S_REGION_NR) {
 FIH_RET(TFM_HAL_ERROR_BAD_STATE);
 }

 struct mpu_armv8m_region_cfg_t mpu_config
```

Definition at line 340 of file protection\_mpu.c.

### 8.164.2.6 region\_base

```
mpu_config region_base = (uint32_t)p_asset->mem.start
```

Definition at line 365 of file protection\_mpu.c.

### 8.164.2.7 region\_limit

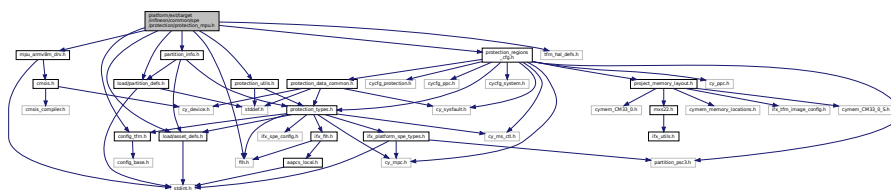
```
mpu_config region_limit = (uint32_t)p_asset->mem.limit - 1U
```

Definition at line 369 of file protection\_mpu.c.

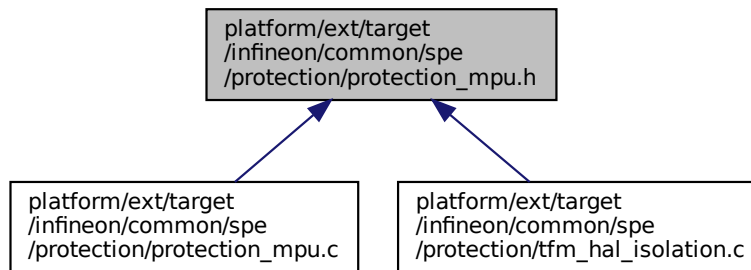
## 8.165 platform/ext/target/infineon/common/spe/protection/protection\_↵ mpu.h File Reference

```
#include "config_tfm.h"
#include "fih.h"
#include "load/asset_defs.h"
#include "mpu_armv8m_drv.h"
#include "partition_info.h"
#include "protection_regions_cfg.h"
#include "protection_utils.h"
#include "tfm_hal_defs.h"
#include "load/partition_defs.h"
```

Include dependency graph for protection\_mpu.h:



This graph shows which files directly or indirectly include this file:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum [tfm\\_hal\\_status\\_t](#)) [ifx\\_mpu\\_init\\_cfg\(void\)](#)  
*Configures the Memory Protection Unit.*

## Variables

- const struct [asset\\_desc\\_t](#) \* [p\\_asset](#)

## 8.165.1 Function Documentation

### 8.165.1.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures the Memory Protection Unit.  
 Configures the system reset request properties.  
 Configures the SAU.  
 Initialize Protection Context switching.  
 Configures MSC.  
 Populate the internal static MPC configuration array.  
 Configures fault handlers and sets priorities.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.

#### Returns

TFM\_HAL\_SUCCESS - Static MPU initialized successfully  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in [tfm\\_hal\\_status\\_t](#)

#### Parameters

|    |                |                                                                                                    |
|----|----------------|----------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <i>p_asset</i> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                |



**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |



**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

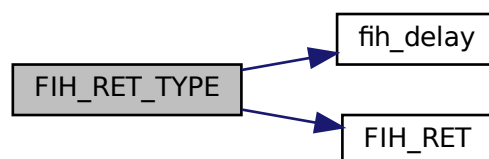
|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Definition at line 184 of file `protection_msc.c`.

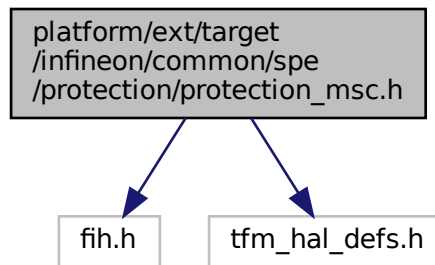
Here is the call graph for this function:



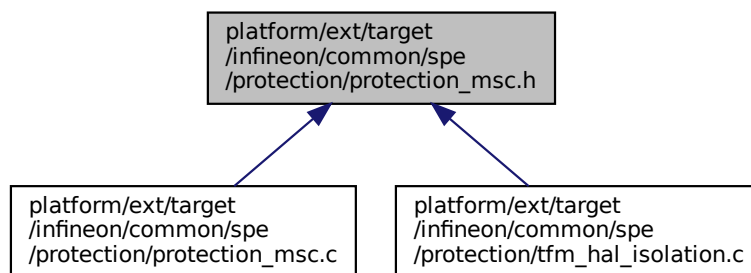
## 8.167 platform/ext/target/infineon/common/spe/protection/protection\_msc.h File Reference

```
#include "fih.h"
#include "tfm_hal_defs.h"
```

Include dependency graph for protection\_msc.h:



This graph shows which files directly or indirectly include this file:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_msc\_cfg([void](#))  
*Configures MSC.*

## 8.167.1 Function Documentation

### 8.167.1.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures MSC.

Returns

TFM\_HAL\_SUCCESS - MSC configured successfully TFM\_HAL\_ERROR\_GENERIC - failed to configure MSC

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

## Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

## Parameters

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.



Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

## Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

## Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

## Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

#### Returns

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

#### Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

#### Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.  
Apply PPC protection to named MMIO asset.  
Isolate memory region (numbered MMIO) by MPU.  
Disable MPU regions that are reserved for partition assets.  
Check memory access permissions using MPC attribute cache.  
Apply MPC configuration using raw interface.  
Apply MPC configuration.  
Check access to memory region by MPC.  
This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |               |                                  |
|----|---------------|----------------------------------|
| in | <i>p_info</i> | Partition information structure. |
|----|---------------|----------------------------------|

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted.  
 TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully.  
 TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset.  
 TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`



**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully.  
 TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset.  
 TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

#### Returns

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

#### Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

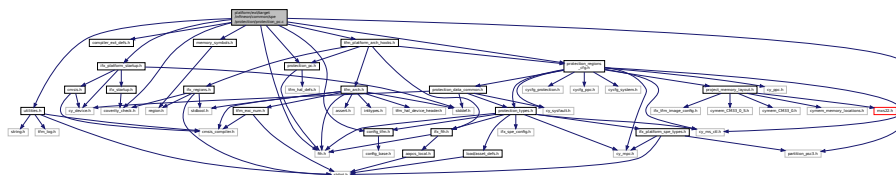
#### Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 122 of file faults.c.

## 8.168 platform/ext/target/infineon/common/spe/protection/protection\_pc.c File Reference

```
#include "config_tfm.h"
#include "compiler_ext_defs.h"
#include "cy_ms_ctl.h"
#include "utilities.h"
#include "fih.h"
#include "memory_symbols.h"
#include "protection_pc.h"
#include "protection_regions_cfg.h"
#include "ifx_platform_startup.h"
#include "coverity_check.h"
#include "tfm_platform_arch_hooks.h"
Include dependency graph for protection_pc.c:
```



## Macros

- `#define IFX_VTOR_ADDRESS __vector_table`
- `#define IFX_RETURN_ON_EXCEPTION_BIT_MASK`

## Functions

- `void ifx_pc2_exception_handler (void)`  
Default secure interrupt handler which is a hardware entry point to Protection Context 2.
- `TFM_COVERITY_DEVIATE_LINE` (MISRA\_C\_2023\_Rule\_2\_8, "Used with assembler [for](#) restoring PC after handling exception")

*Active protection context.*

- [FIH\\_RET\\_TYPE](#) (enum [tfm\\_hal\\_status\\_t](#)) [ifx\\_pc\\_init\(void\)](#)

*Configures fault handlers and sets priorities.*

- [void ifx\\_arch\\_switch\\_protection\\_context](#) (uint32\_t [pc](#))

*Perform protection context switching.*

- [void ifx\\_arch\\_switch\\_protection\\_context\\_from\\_irq](#) (uint32\_t [pc](#))

*Perform protection context switching from low priority IRQ.*

- [void ifx\\_arch\\_switch\\_protection\\_context\\_thread](#) (uint32\_t [pc](#))

*Entry point to Protection Context switching via UsageFault/HardFault handler.*

## Variables

- const [VECTOR\\_TABLE\\_Type](#) [ifx\\_pc2\\_vector\\_table](#) [VECTORTABLE\_SIZE]
- [fih\\_int](#) [ifx\\_arch\\_active\\_protection\\_context](#) = [FIH\\_INT\\_INIT\(IFX\\_PC\\_TFM\\_SPM\\_ID\)](#)

## 8.168.1 Macro Definition Documentation

### 8.168.1.1 IFX\_RETURN\_ON\_EXCEPTION\_BIT\_MASK

```
#define IFX_RETURN_ON_EXCEPTION_BIT_MASK
```

Value:

```
((1 << 2) | /* -14 NMI Handler */ \
 (0 << 3) | /* -13 Hard Fault Handler */ \
 (0 << 4) | /* -12 MPU Fault Handler */ \
 (0 << 5) | /* -11 Bus Fault Handler */ \
 (0 << 6) | /* -10 Usage Fault Handler */ \
 (0 << 7) | /* -9 Secure Fault Handler */ \
 (0 << 8) | /* -8 Reserved */ \
 (0 << 9) | /* -7 Reserved */ \
 (0 << 10) | /* -6 Reserved */ \
 (0 << 11) | /* -5 SVCall Handler */ \
 (1 << 12) | /* -4 Debug Monitor Handler */ \
 (0 << 13) | /* -3 Reserved */ \
 (0 << 14) | /* -2 PendSV Handler */ \
 (1 << 15)) /* -1 SysTick Handler */
```

Definition at line 38 of file [protection\\_pc.c](#).

### 8.168.1.2 IFX\_VTOR\_ADDRESS

```
#define IFX_VTOR_ADDRESS __vector_table
```

Definition at line 26 of file [protection\\_pc.c](#).

## 8.168.2 Function Documentation

### 8.168.2.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures fault handlers and sets priorities.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |



**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

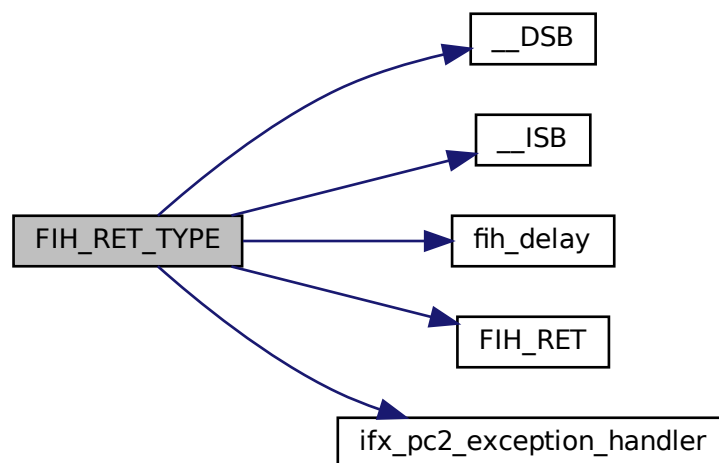
|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Definition at line 80 of file `protection_pc.c`.

Here is the call graph for this function:

**8.168.2.2 ifx\_arch\_switch\_protection\_context()**

```
void ifx_arch_switch_protection_context (
 uint32_t pc)
```

Perform protection context switching.

**Note**

It's unsafe to switch PC to other than PC2 (used by SPM) in low priority exception, because it's possible that there will be pending IRQ and MCU will try to use MSP stack pushing exception stack within PC not equal to PC2.

IRQ must be disabled if this function is called from exception/IRQ with privilege below 0.

Definition at line 155 of file protection\_pc.c.

**8.168.2.3 ifx\_arch\_switch\_protection\_context\_from\_irq()**

```
void ifx_arch_switch_protection_context_from_irq (
 uint32_t pc)
```

Perform protection context switching from low priority IRQ.

This function will complete switching to PC2 (SPM) in scope of current exception. For all other protection context it will schedule switching after exit from active IRQ in thread mode by [ifx\\_arch\\_switch\\_protection\\_context\\_thread](#).

**Note**

IRQ must be disabled when this function is called.

Definition at line 248 of file protection\_pc.c.

**8.168.2.4 ifx\_arch\_switch\_protection\_context\_thread()**

```
void ifx_arch_switch_protection_context_thread (
 uint32_t pc)
```

Entry point to Protection Context switching via UsageFault/HardFault handler.

This function schedule UsageFault as an entry point to Protection Context switching. But UsageFault escalates HardFault because masked interrupts are disabled when [ifx\\_arch\\_switch\\_protection\\_context\\_from\\_irq](#) is called.

HardFault validates that it was called because of `udf` instruction with correct set of arguments pushed via exception stack frame that are duplicate of one prepared by [ifx\\_arch\\_switch\\_protection\\_context\\_from\\_irq](#). HardFault calls Protection Context switching if all validation tests are passed. See [PLATFORM\\_HARD\\_FAULT\\_HANDLER\\_HOOK](#) for more information.

Definition at line 311 of file protection\_pc.c.

**8.168.2.5 ifx\_pc2\_exception\_handler()**

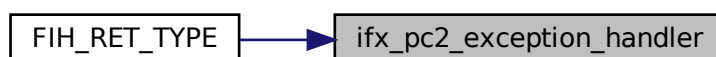
```
void ifx_pc2_exception_handler (
 void)
```

Default secure interrupt handler which is a hardware entry point to Protection Context 2.

SCB ICSR VECTACTIVE is used to detect active interrupt number and select a appropriate handler from VTOR table located at [IFX\\_VTOR\\_ADDRESS](#).

Definition at line 343 of file protection\_pc.c.

Here is the caller graph for this function:



### 8.168.2.6 TFM\_COVERITY\_DEVIATE\_LINE()

```
TFM_COVERITY_DEVIATE_LINE (
 MISRA_C_2023_Rule_2_8 ,
 "Used with assembler for restoring PC after handling exception")
```

Active protection context.

PC\_SAVED is not used because it's inconsistent if there are multiple pending IRQs. This variable is atomically updated with PC. So, this allows to properly restore protection context after handling exception.

Note: Allocate variable in scope of SPM data to avoid access by unprivileged code.

## 8.168.3 Variable Documentation

### 8.168.3.1 ifx\_arch\_active\_protection\_context

```
fih_int ifx_arch_active_protection_context = Fih_Int_Init(IFX_PC_TFM_SPM_ID)
```

Definition at line 77 of file protection\_pc.c.

### 8.168.3.2 ifx\_pc2\_vector\_table

```
const VECTOR_TABLE_Type ifx_pc2_vector_table[VECTORTABLE_SIZE]
```

Initial value:

```
= {
 [0] = (VECTOR_TABLE_Type) (&__INITIAL_SP),
 [1 ... VECTORTABLE_SIZE-1] = ifx_pc2_exception_handler,
}
```

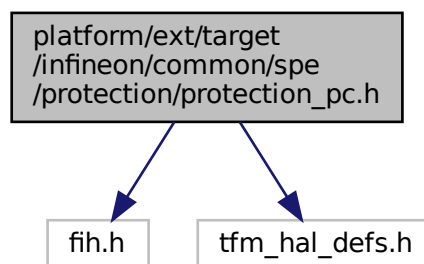
Definition at line 58 of file protection\_pc.c.

## 8.169 platform/ext/target/infineon/common/spe/protection/protection\_pc.h File Reference ↩

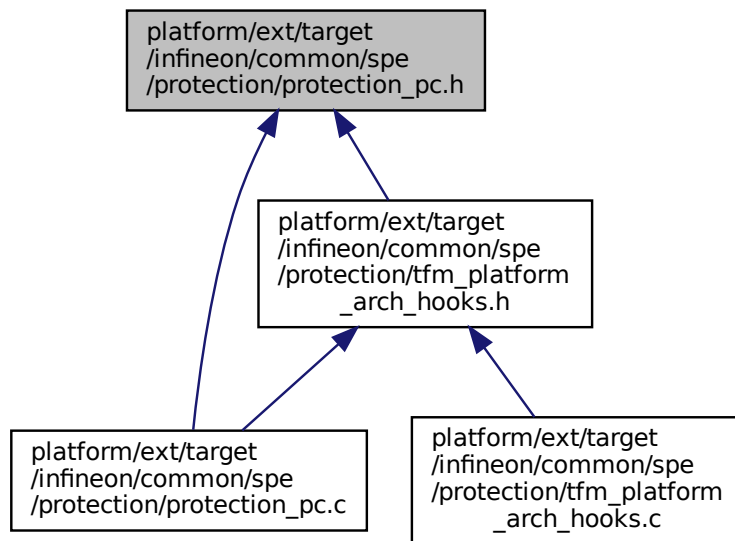
```
#include "fi.h"
```

```
#include "tfm_hal_defs.h"
```

Include dependency graph for protection\_pc.h:



This graph shows which files directly or indirectly include this file:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_pc\_init(void)  
*Initialize Protection Context switching.*
- [void ifx\\_arch\\_switch\\_protection\\_context\\_thread](#) (uint32\_t pc)  
*Entry point to Protection Context switching via UsageFault/HardFault handler.*

## Variables

- [fih\\_int ifx\\_arch\\_active\\_protection\\_context](#)

## 8.169.1 Function Documentation

### 8.169.1.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Initialize Protection Context switching.

Configures the VTOR to implement switching to Protection Context used by SPM.

#### Returns

A status code as specified in tfm\_hal\_status\_t

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.  
 Apply a configuration to specified assets of a partition.  
 Apply PPC protection to named MMIO asset.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.  
 Check memory access permissions using MPC attribute cache.  
 Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Initialize Protection Context switching.  
 Configures MSC.  
 Populate the internal static MPC configuration array.  
 Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Initialize Protection Context switching.  
 Configures MSC.  
 Populate the internal static MPC configuration array.  
 This function enables platform specific faults interrupts.  
 This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.  
 Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)  
 Apply a configuration to assets of a partition.  
 Apply a configuration to specified assets of a partition.  
 Apply PPC protection to named MMIO asset.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.  
 Check memory access permissions using MPC attribute cache.  
 Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |               |                                                                  |
|----|---------------|------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a> . |
|----|---------------|------------------------------------------------------------------|

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)



## Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully.  
 TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset.  
 TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

## Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.

Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.

TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.

TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully.

TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM.

TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration.

TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

## Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

## Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

## Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

## Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Parameters**

|    |                  |                                             |
|----|------------------|---------------------------------------------|
| in | <i>p_assets</i>  | Array of partition's assets                 |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
Check memory access permissions using MPC attribute cache.  
This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                    |
|----|----------------------|------------------------------------|
| in | <i>p_info</i>        | Partition information structure.   |
| in | <i>memory_config</i> | Memory configuration structure.    |
| in | <i>base</i>          | Base address of the memory region. |



**Parameters**

|    |                    |                                                       |
|----|--------------------|-------------------------------------------------------|
| in | <i>size</i>        | Size of the memory region.                            |
| in | <i>access_type</i> | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.  
Isolate memory region (numbered MMIO) by MPU.  
Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.  
Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 122 of file `faults.c`.

**8.169.1.2 ifx\_arch\_switch\_protection\_context\_thread()**

```
void ifx_arch_switch_protection_context_thread (
 uint32_t pc)
```

Entry point to Protection Context switching via UsageFault/HardFault handler.

This function schedule UsageFault as an entry point to Protection Context switching. But UsageFault escalates HardFault because masked interrupts are disabled when [ifx\\_arch\\_switch\\_protection\\_context\\_from\\_irq](#) is called. HardFault validates that it was called because of `udf` instruction with correct set of arguments pushed via exception stack frame that are duplicate of one prepared by [ifx\\_arch\\_switch\\_protection\\_context\\_from\\_irq](#). HardFault calls Protection Context switching if all validation tests are passed. See [PLATFORM\\_HARD\\_FAULT\\_HANDLER\\_HOOK](#) for more information.

Definition at line 311 of file `protection_pc.c`.

## 8.169.2 Variable Documentation

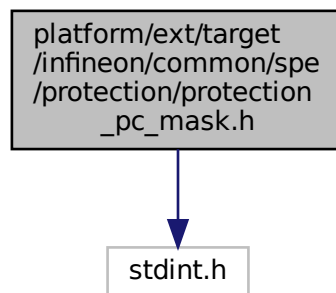
### 8.169.2.1 ifx\_arch\_active\_protection\_context

`fi_h_int ifx_arch_active_protection_context`  
 Definition at line 77 of file `protection_pc.c`.

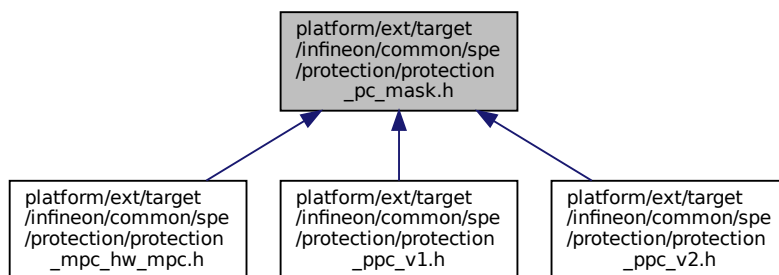
## 8.170 platform/ext/target/infineon/common/spe/protection/protection\_ pc\_mask.h File Reference

```
#include <stdint.h>
```

Include dependency graph for `protection_pc_mask.h`:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_GET_PC(mask, pc) (((mask) >> ((uint32_t)(pc))) & 1UL)`

## Typedefs

- `typedef uint32_t ifx_pc_mask_t`

## 8.170.1 Macro Definition Documentation

### 8.170.1.1 IFX\_GET\_PC

```
#define IFX_GET_PC(
 mask,
 pc) (((mask) >> ((uint32_t)(pc))) & 1UL)
```

Definition at line 15 of file protection\_pc\_mask.h.

## 8.170.2 Typedef Documentation

### 8.170.2.1 ifx\_pc\_mask\_t

```
typedef uint32_t ifx_pc_mask_t
```

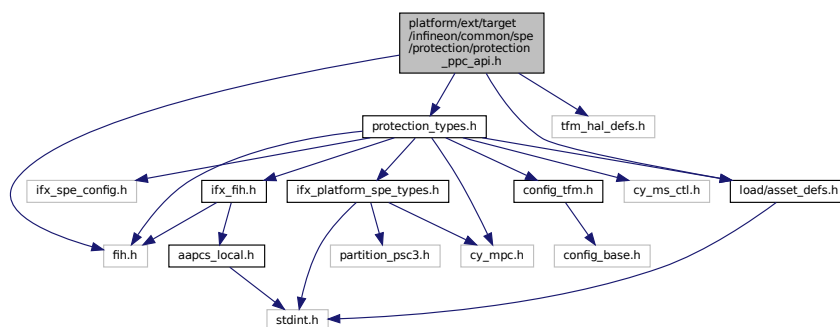
Definition at line 18 of file protection\_pc\_mask.h.

## 8.171 platform/ext/target/infineon/common/spe/protection/protection\_ppc\_api.h File Reference

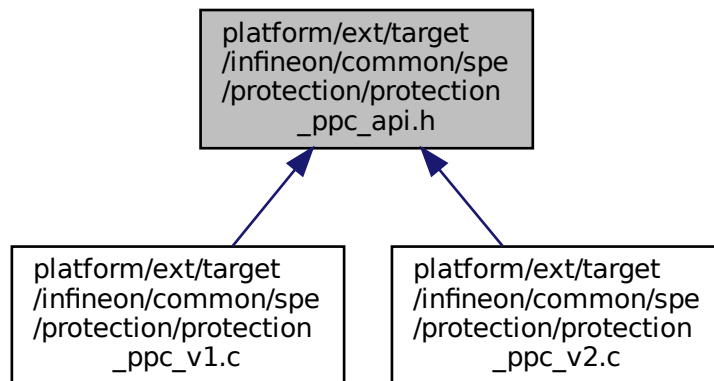
This file contains PPC driver API declaration.

```
#include "fih.h"
#include "load/asset_defs.h"
#include "protection_types.h"
#include "tfm_hal_defs.h"
```

Include dependency graph for protection\_ppc\_api.h:



This graph shows which files directly or indirectly include this file:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_ppc\_init\_cfg([void](#))  
*Configures the Peripheral Protection Controller.*

## Variables

- cy\_en\_prot\_region\_t [asset](#)

### 8.171.1 Detailed Description

This file contains PPC driver API declaration.  
It's expected that PPC driver implementation follows API declared in this file.

### 8.171.2 Function Documentation

#### 8.171.2.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures the Peripheral Protection Controller.  
Configures the system reset request properties.  
Configures the SAU.  
Initialize Protection Context switching.  
Configures MSC.  
Populate the internal static MPC configuration array.  
Configures fault handlers and sets priorities.  
Apply PPC protection to named MMIO asset.

#### Returns

TFM\_HAL\_SUCCESS - static PPC initialized successfully  
TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC





## Variables

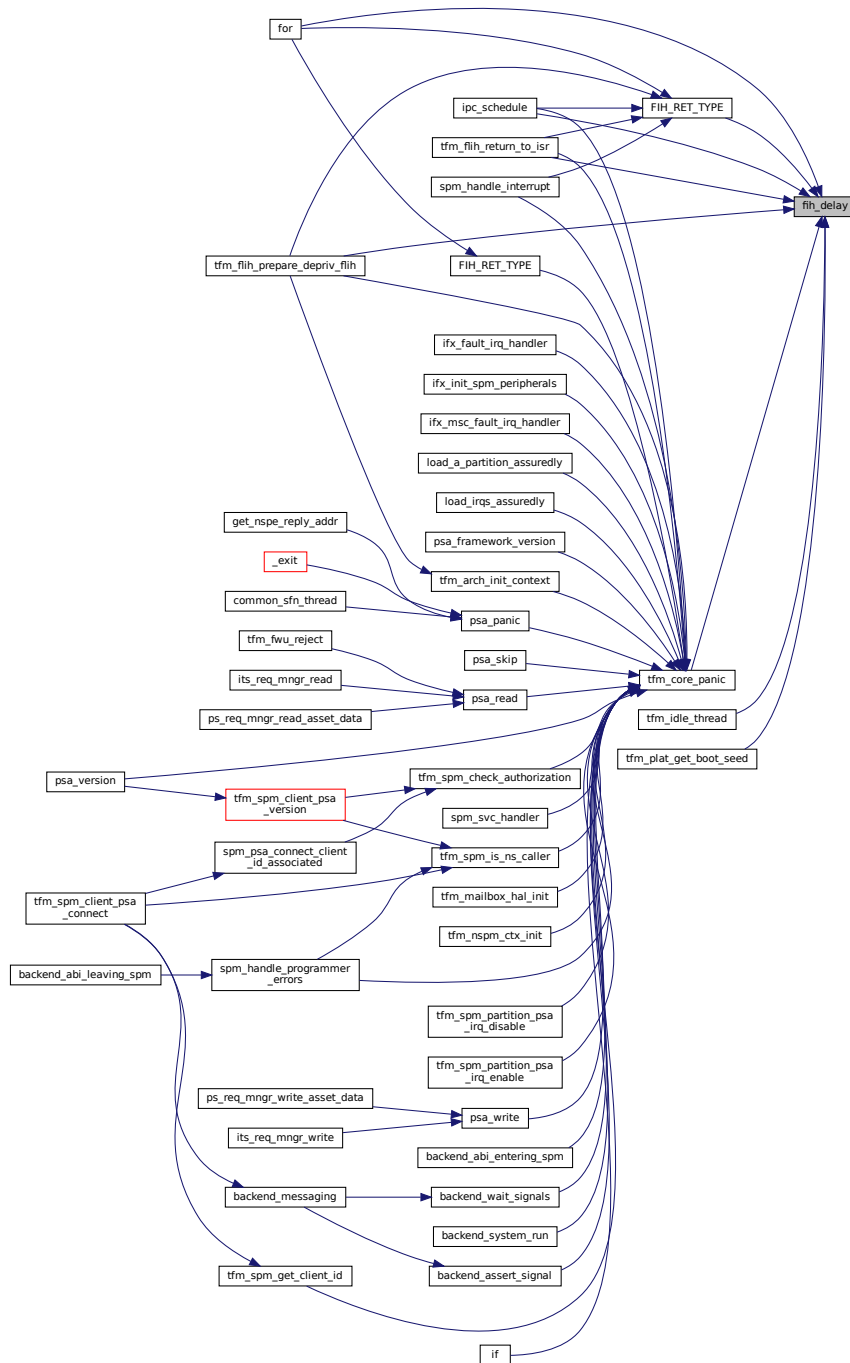
- `cy_en_prot_region_t` [asset](#)
- `cy_stc_ppc_pc_mask_t` [ppc\\_pcmask](#)
- `cy_stc_ppc_attribute_t` [ppc\\_attribute](#)

## 8.172.1 Function Documentation

### 8.172.1.1 `fih_delay()`

```
void fih_delay ()
```

Here is the caller graph for this function:



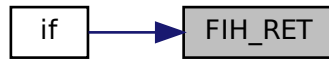
### 8.172.1.2 FIH\_RET()

```

FIH_RET (
 TFM_HAL_SUCCESS)

```

Here is the caller graph for this function:



#### 8.172.1.3 if() [1/3]

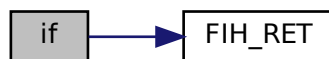
```

if (
 (ppc_cfg->pc_mask &IFX_PC_TFM_SPM) == 0U)

```

Definition at line 355 of file protection\_ppc\_v1.c.

Here is the call graph for this function:



#### 8.172.1.4 if() [2/3]

```

if (
 Cy_Ppc_ConfigAttrib(ppc_ptr, &ppc_attribute) != CY_PPC_SUCCESS)

```

Definition at line 351 of file protection\_ppc\_v1.c.

Here is the call graph for this function:



#### 8.172.1.5 if() [3/3]

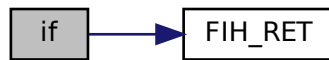
```

if (
 Cy_Ppc_SetPcMask(ppc_ptr, &ppc_pcmask) != CY_PPC_SUCCESS)

```

Definition at line 344 of file protection\_ppc\_v1.c.

Here is the call graph for this function:



#### 8.172.1.6 ifx\_check\_config\_validity()

```

ifx_check_config_validity (
 ppc_ptr ,
 & ppc_attribute,
 & ppc_pcmask)

```

### 8.172.2 Variable Documentation

#### 8.172.2.1 asset

cy\_en\_prot\_region\_t asset

**Initial value:**

```

{
 PPC_Type* ppc_ptr = Cy_Ppc_GetPointerForRegion(asset)

```

Definition at line 324 of file protection\_ppc\_v1.c.

#### 8.172.2.2 ppc\_attribute

cy\_stc\_ppc\_attribute\_t ppc\_attribute

**Initial value:**

```

= {
 .startRegion = asset,
 .endRegion = asset,
 .secAttribute = ppc_cfg->sec_attr,
 .secPrivAttribute = to_s_priv_attr(ppc_cfg->allow_unpriv, ppc_cfg->sec_attr),
 .nsPrivAttribute = to_ns_priv_attr(ppc_cfg->allow_unpriv, ppc_cfg->sec_attr),
}

```

Definition at line 334 of file protection\_ppc\_v1.c.

#### 8.172.2.3 ppc\_pcmask

cy\_stc\_ppc\_pc\_mask\_t ppc\_pcmask

**Initial value:**

```

= {
 .startRegion = asset,
 .endRegion = asset,
 .pcMask = ppc_cfg->pc_mask | IFX_PC_TFM_SPM,
}

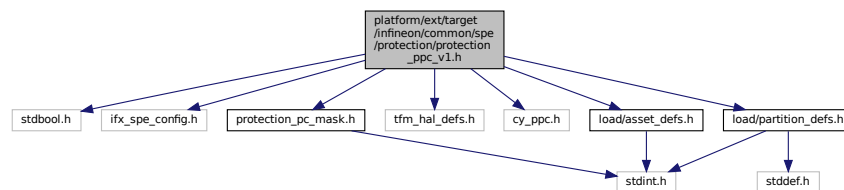
```

Definition at line 328 of file protection\_ppc\_v1.c.

## 8.173 platform/ext/target/infineon/common/spe/protection/protection\_ppc\_v1.h File Reference ↩

This file contains types/API used by PPC driver.

```
#include <stdbool.h>
#include "ifx_spe_config.h"
#include "protection_pc_mask.h"
#include "tfm_hal_defs.h"
#include "cy_ppc.h"
#include "load/partition_defs.h"
#include "load/asset_defs.h"
Include dependency graph for protection_ppc_v1.h:
```



## Data Structures

- struct [ifx\\_ppc\\_named\\_mmio\\_config\\_t](#)
- struct [ifx\\_ppcx\\_config\\_t](#)

## Macros

- #define [IFX\\_PPC\\_NAMED\\_MMIO\\_SPM\\_ROT\\_CFG](#)
- #define [IFX\\_PPC\\_NAMED\\_MMIO\\_PSA\\_ROT\\_CFG](#)
- #define [IFX\\_PPC\\_NAMED\\_MMIO\\_APP\\_ROT\\_CFG](#)

### 8.173.1 Detailed Description

This file contains types/API used by PPC driver.

#### Note

Don't include this file directly, include [protection\\_types.h](#) instead !!! It's expected that this file is included by [protection\\_types.h](#) if platform is configured to use PPC driver.

### 8.173.2 Macro Definition Documentation

#### 8.173.2.1 IFX\_PPC\_NAMED\_MMIO\_APP\_ROT\_CFG

```
#define IFX_PPC_NAMED_MMIO_APP_ROT_CFG
```

##### Value:

```
.ppc_cfg = { \
 .sec_attr = CY_PPC_SECURE, \
 .allow_unpriv = true, \
 .pc_mask = IFX_PC_DEFAULT | IFX_PC_TFM_AROT, \
},
```

Definition at line 80 of file `protection_ppc_v1.h`.

#### 8.173.2.2 IFX\_PPC\_NAMED\_MMIO\_PSA\_ROT\_CFG

```
#define IFX_PPC_NAMED_MMIO_PSA_ROT_CFG
```

##### Value:

```
.ppc_cfg = { \
 .sec_attr = CY_PPC_SECURE, \
```

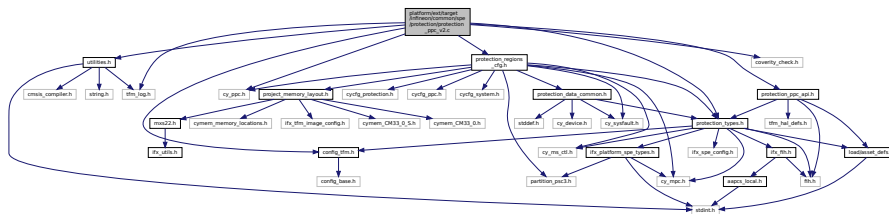
Definition at line 62 of file protection\_ppc\_v1.h.

```
#define IFX_PPC_NAMED_MMIO_SPM_ROT_CFG
```

```
.ppc_cfg = { \
 .sec_attr = CY_PPC_SECURE, \
 .allow_unpriv = false, \
 .pc_mask = IFX_PC_DEFAULT, \
},
```

**8.174 platform/ext/target/infineon/common/spe/protection/protection\_↵  
ppc\_v2.c File Reference**

Include dependency graph for protection\_ppc\_v2.c:



- `ifx_check_config_validity` (ppc\_ptr, asset, &ppc\_attribute, ppc\_pcmask)
- `if` (Cy\_Ppc\_SetPcMask(ppc\_ptr, asset, ppc\_pcmask) !=CY\_PPC\_SUCCESS)
- `void fih_delay` ()
- `if` (Cy\_Ppc\_ConfigAttrib(ppc\_ptr, asset, &ppc\_attribute) !=CY\_PPC\_SUCCESS)
- `if` ((ppc\_cfg->pc\_mask &IFX\_PC\_TFM\_SPM)==0U)
- `FIH_RET` (TFM\_HAL\_SUCCESS)

- cy\_en\_prot\_region\_t **asset**
- uint32\_t **ppc\_pcmask** = ppc\_cfg->pc\_mask | **IFX\_PC\_TFM\_SPM**
- cy\_stc\_ppc\_attribute\_t **ppc\_attribute**

Generated by Doxygen

#### 8.174.1.1 fih\_delay()

```
void fih_delay ()
```

#### 8.174.1.2 FIH\_RET()

```
FIH_RET (
 TFM_HAL_SUCCESS)
```

Here is the caller graph for this function:



#### 8.174.1.3 if() [1/3]

```
if (
 (ppc_cfg->pc_mask &IFX_PC_TFM_SPM) == 0U)
```

Definition at line 380 of file protection\_ppc\_v2.c.

Here is the call graph for this function:



#### 8.174.1.4 if() [2/3]

```
if (
 Cy_Ppc_ConfigAttrib(ppc_ptr, asset, &ppc_attribute) != CY_PPC_SUCCESS)
```

Definition at line 376 of file protection\_ppc\_v2.c.

Here is the call graph for this function:





### 8.174.1.5 if() [3/3]

```
if (
 Cy_Ppc_SetPcMask(ppc_ptr, asset, ppc_pcmask) != CY_PPC_SUCCESS)
```

Definition at line 369 of file protection\_ppc\_v2.c.

Here is the call graph for this function:



### 8.174.1.6 ifx\_check\_config\_validity()

```
ifx_check_config_validity (
 ppc_ptr ,
 asset ,
 & ppc_attribute,
 ppc_pcmask)
```

## 8.174.2 Variable Documentation

### 8.174.2.1 asset

```
cy_en_prot_region_t asset
```

**Initial value:**

```
{
 PPC_Type* ppc_ptr = Cy_Ppc_GetPointerForRegion(asset)
```

Definition at line 355 of file protection\_ppc\_v2.c.

### 8.174.2.2 ppc\_attribute

```
cy_stc_ppc_attribute_t ppc_attribute
```

**Initial value:**

```
= {
 .pcMask = ppc_pcmask,
 .secAttribute = ppc_cfg->sec_attr,
 .privAttribute = to_priv_attr(ppc_cfg->allow_unpriv),
}
```

Definition at line 361 of file protection\_ppc\_v2.c.

### 8.174.2.3 ppc\_pcmask

```
uint32_t ppc_pcmask = ppc_cfg->pc_mask | IFX_PC_TFM_SPM
```

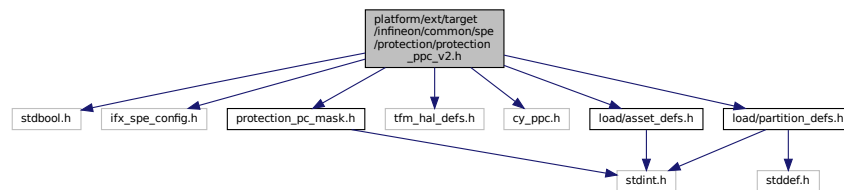
Definition at line 359 of file protection\_ppc\_v2.c.

## 8.175 platform/ext/target/infineon/common/spe/protection/protection\_↔ ppc\_v2.h File Reference

This file contains types/API used by PPC driver.

```
#include <stdbool.h>
#include "ifx_spe_config.h"
#include "protection_pc_mask.h"
#include "tfm_hal_defs.h"
#include "cy_ppc.h"
#include "load/partition_defs.h"
#include "load/asset_defs.h"
```

Include dependency graph for protection\_ppc\_v2.h:



### Data Structures

- struct [ifx\\_ppc\\_named\\_mmio\\_config\\_t](#)
- struct [ifx\\_ppcx\\_config\\_t](#)

### Macros

- #define [IFX\\_PPC\\_NAMED\\_MMIO\\_SPM\\_ROT\\_CFG](#)
- #define [IFX\\_PPC\\_NAMED\\_MMIO\\_PSA\\_ROT\\_CFG](#)
- #define [IFX\\_PPC\\_NAMED\\_MMIO\\_APP\\_ROT\\_CFG](#)

### 8.175.1 Detailed Description

This file contains types/API used by PPC driver.

#### Note

Don't include this file directly, include [protection\\_types.h](#) instead !!! It's expected that this file is included by [protection\\_types.h](#) if platform is configured to use PPC driver.

### 8.175.2 Macro Definition Documentation

#### 8.175.2.1 IFX\_PPC\_NAMED\_MMIO\_APP\_ROT\_CFG

```
#define IFX_PPC_NAMED_MMIO_APP_ROT_CFG
```

##### Value:

```
.ppc_cfg = { \
 .sec_attr = CY_PPC_SECURE, \
 .allow_unpriv = true, \
 .pc_mask = IFX_PC_DEFAULT | IFX_PC_TFM_AROT, \
},
```

Definition at line 89 of file protection\_ppc\_v2.h.

### 8.175.2.2 IFX\_PPC\_NAMED\_MMIO\_PSA\_ROT\_CFG

```
#define IFX_PPC_NAMED_MMIO_PSA_ROT_CFG
```

**Value:**

```
.ppc_cfg = { \
 .sec_attr = CY_PPC_SECURE, \
 .allow_unpriv = true, \
 .pc_mask = IFX_PC_DEFAULT | IFX_PC_TFM_PROT, \
},
```

Definition at line 71 of file protection\_ppc\_v2.h.

### 8.175.2.3 IFX\_PPC\_NAMED\_MMIO\_SPM\_ROT\_CFG

```
#define IFX_PPC_NAMED_MMIO_SPM_ROT_CFG
```

**Value:**

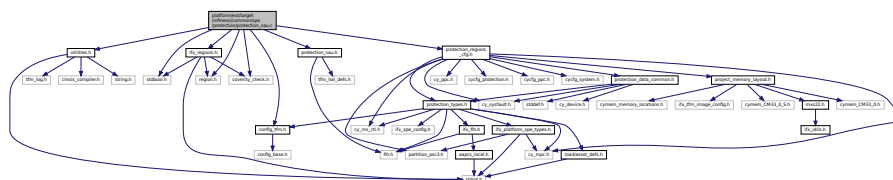
```
.ppc_cfg = { \
 .sec_attr = CY_PPC_SECURE, \
 .allow_unpriv = false, \
 .pc_mask = IFX_PC_DEFAULT, \
},
```

Definition at line 54 of file protection\_ppc\_v2.h.

## 8.176 platform/ext/target/infineon/common/spe/protection/protection\_sau.c File Reference

```
#include <stdbool.h>
#include "utilities.h"
#include "config_tfm.h"
#include "ifx_regions.h"
#include "protection_sau.h"
#include "protection_regions_cfg.h"
#include "coverity_check.h"
#include "region.h"
```

Include dependency graph for protection\_sau.c:



## Macros

- `#define IFX_SAU_VENEER_REGION_COUNT 0U`
- `#define IFX_SAU_RESERVED_REGION_COUNT IFX_SAU_VENEER_REGION_COUNT /* Veneer region */`

## Functions

- `FIH_RET_TYPE` (enum tfm\_hal\_status\_t) `ifx_sau_cfg(void)`

*Configures fault handlers and sets priorities.*

## 8.176.1 Macro Definition Documentation

### 8.176.1.1 IFX\_SAU\_RESERVED\_REGION\_COUNT

```
#define IFX_SAU_RESERVED_REGION_COUNT IFX_SAU_VENEER_REGION_COUNT /* Veneer region */
```

Definition at line 28 of file protection\_sau.c.

### 8.176.1.2 IFX\_SAU\_VENEER\_REGION\_COUNT

```
#define IFX_SAU_VENEER_REGION_COUNT 0U
```

Definition at line 24 of file protection\_sau.c.

## 8.176.2 Function Documentation

### 8.176.2.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures fault handlers and sets priorities.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

#### Returns

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

#### Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

#### Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

#### Returns

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

#### Parameters

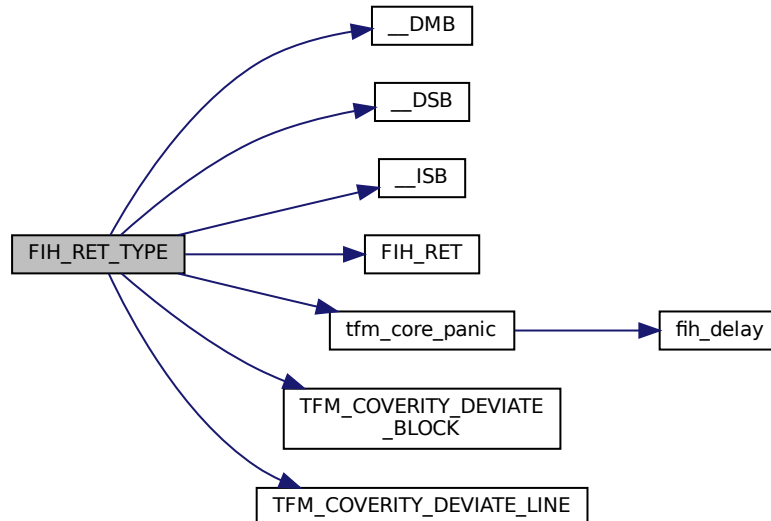
|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 34 of file protection\_sau.c.

Here is the call graph for this function:

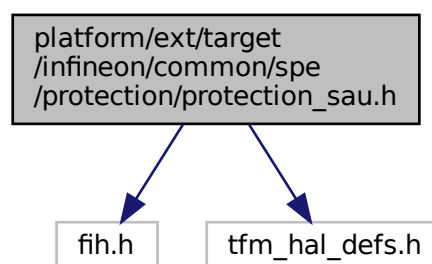


## 8.177 platform/ext/target/infineon/common/spe/protection/protection\_sau.h File Reference

```
#include "fih.h"
```

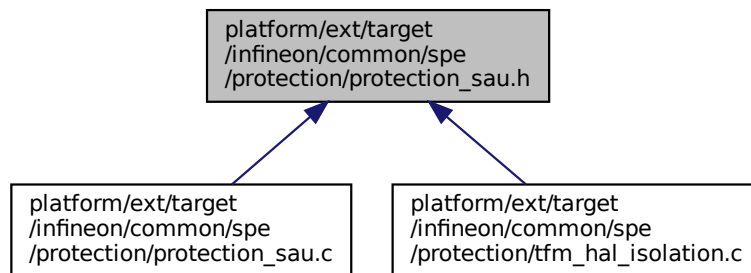
```
#include "tfm_hal_defs.h"
```

Include dependency graph for protection\_sau.h:





This graph shows which files directly or indirectly include this file:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_hal\_status\_t) ifx\_sau\_cfg([void](#))

*Configures the SAU.*

### 8.177.1 Function Documentation

#### 8.177.1.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures the SAU.

##### Returns

TFM\_HAL\_SUCCESS - SAU configured successfully TFM\_HAL\_ERROR\_GENERIC - failed to configure SAU

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

##### Returns

Returns values as specified by the tfm\_plat\_err\_t

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

## Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully.  
 TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset.  
 TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check access to memory region by MPC.

## Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

## Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

## Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

## Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

## Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

## Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

## Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU



## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

## Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

## Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

## Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

#### Returns

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

#### Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

#### Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.  
Isolate memory region (numbered MMIO) by MPU.  
Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures the SAU.  
Initialize Protection Context switching.  
Configures MSC.  
Populate the internal static MPC configuration array.  
Configures fault handlers and sets priorities.  
Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Configures the SAU.  
Initialize Protection Context switching.  
Configures MSC.  
Populate the internal static MPC configuration array.  
This function enables platform specific faults interrupts.  
This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |



**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.  
Isolate memory region (numbered MMIO) by MPU.  
Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.  
Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

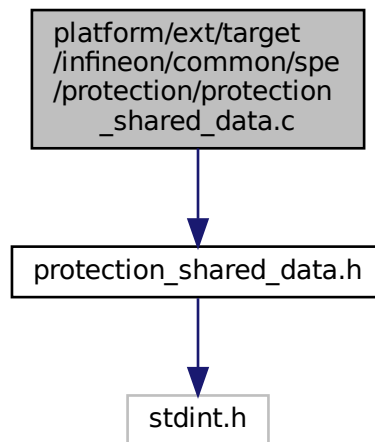
TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 122 of file faults.c.

## 8.178 platform/ext/target/infineon/common/spe/protection/protection\_shared\_data.c File Reference

```
#include "protection_shared_data.h"
```

Include dependency graph for protection\_shared\_data.c:



## Variables

- volatile uint32\_t ifx\_spm\_state = 0x3498A78Bu  
*Shared state between SPM and partitions.*
- volatile uint32\_t ifx\_spm\_state\_inv = ~(uint32\_t) 0x3498A78Bu

### 8.178.1 Variable Documentation

#### 8.178.1.1 ifx\_spm\_state

```
volatile uint32_t ifx_spm_state = 0x3498A78Bu
```

Shared state between SPM and partitions.

Partitions can access this value in read-only mode, while SPM have write access.

It's used by SE IPC Service to select a proper communication method.

Valid values are:

- **IFX\_SPM\_STATE\_INITIALIZING** - allows SPM to use SE RT Services Utils. Must be used during static initialization phase only.
- **IFX\_SPM\_STATE\_RUNNING** - SE IPC interface is used by SE IPC Service only.

Definition at line 11 of file protection\_shared\_data.c.

#### 8.178.1.2 ifx\_spm\_state\_inv

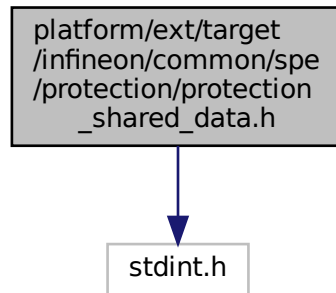
```
volatile uint32_t ifx_spm_state_inv = ~(uint32_t) 0x3498A78Bu
```

Definition at line 12 of file protection\_shared\_data.c.

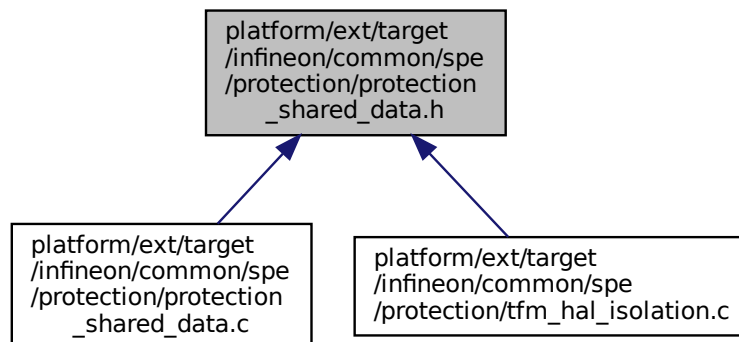
## 8.179 platform/ext/target/infineon/common/spe/protection/protection\_↵ shared\_data.h File Reference

```
#include <stdint.h>
```

Include dependency graph for protection\_shared\_data.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define IFX_SPM_STATE_INITIALIZING 0x3498A78Bu`
- `#define IFX_SPM_STATE_RUNNING 0xA78B3498u`
- `#define IFX_SET_SPM_STATE(state)`
- `#define IFX_IS_SPM_INITILIZING()`
- `#define IFX_IS_SPM_RUNNING()`

### Variables

- `volatile uint32_t ifx_spm_state`  
*Shared state between SPM and partitions.*
- `volatile uint32_t ifx_spm_state_inv`

## 8.179.1 Macro Definition Documentation

### 8.179.1.1 IFX\_IS\_SPM\_INITILIZING

```
#define IFX_IS_SPM_INITILIZING()
```

**Value:**

```
((ifx_spm_state == IFX_SPM_STATE_INITIALIZING) &&
(ifx_spm_state_inv == ~(uint32_t) IFX_SPM_STATE_INITIALIZING)) \
```

Definition at line 42 of file protection\_shared\_data.h.

### 8.179.1.2 IFX\_IS\_SPM\_RUNNING

```
#define IFX_IS_SPM_RUNNING()
```

**Value:**

```
((ifx_spm_state == IFX_SPM_STATE_RUNNING) &&
(ifx_spm_state_inv == ~(uint32_t) IFX_SPM_STATE_RUNNING)) \
```

Definition at line 45 of file protection\_shared\_data.h.

### 8.179.1.3 IFX\_SET\_SPM\_STATE

```
#define IFX_SET_SPM_STATE(
 state)
```

**Value:**

```
do {
 ifx_spm_state = (uint32_t) (state);
 ifx_spm_state_inv = ~(uint32_t) (state);
} while (0) \
```

Definition at line 36 of file protection\_shared\_data.h.

### 8.179.1.4 IFX\_SPM\_STATE\_INITIALIZING

```
#define IFX_SPM_STATE_INITIALIZING 0x3498A78Bu
```

Definition at line 15 of file protection\_shared\_data.h.

### 8.179.1.5 IFX\_SPM\_STATE\_RUNNING

```
#define IFX_SPM_STATE_RUNNING 0xA78B3498u
```

Definition at line 17 of file protection\_shared\_data.h.

## 8.179.2 Variable Documentation

### 8.179.2.1 ifx\_spm\_state

```
volatile uint32_t ifx_spm_state
```

Shared state between SPM and partitions.

Partitions can access this value in read-only mode, while SPM have write access.

It's used by SE IPC Service to select a proper communication method.

Valid values are:

- [IFX\\_SPM\\_STATE\\_INITIALIZING](#) - allows SPM to use SE RT Services Utils. Must be used during static initialization phase only.
- [IFX\\_SPM\\_STATE\\_RUNNING](#) - SE IPC interface is used by SE IPC Service only.

Definition at line 11 of file protection\_shared\_data.c.

### 8.179.2.2 ifx\_spm\_state\_inv

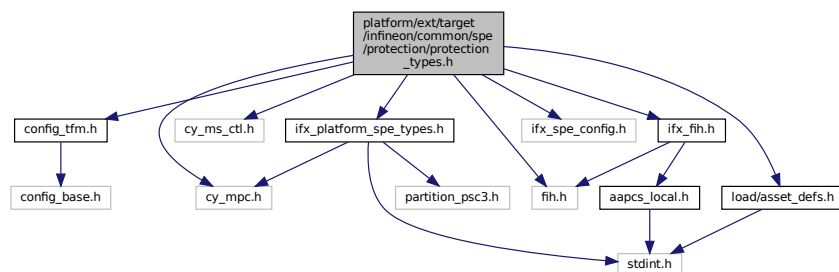
volatile uint32\_t ifx\_spm\_state\_inv

Definition at line 12 of file protection\_shared\_data.c.

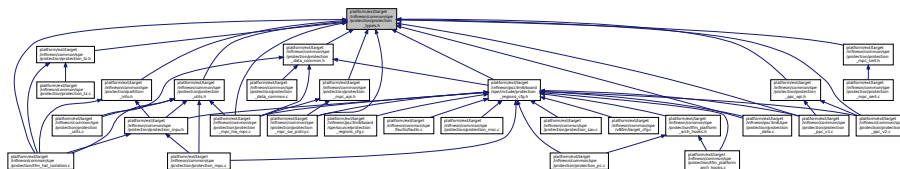
## 8.180 platform/ext/target/infineon/common/spe/protection/protection\_types.h File Reference

```
#include "config_tfm.h"
#include "cy_mpc.h"
#include "cy_ms_ctl.h"
#include "fih.h"
#include "ifx_fih.h"
#include "ifx_spe_config.h"
#include "ifx_platform_spe_types.h"
#include "load/asset_defs.h"
```

Include dependency graph for protection\_types.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ifx\\_ppc\\_regions\\_config\\_t](#)
- struct [ifx\\_protection\\_region\\_config\\_t](#)  
Structure used to store platform specific protection properties.
- struct [ifx\\_partition\\_load\\_info\\_t](#)  
Structure used to store platform specific partition properties.
- struct [ifx\\_partition\\_info\\_t](#)  
Structure describing partition info.
- struct [ifx\\_sau\\_config\\_t](#)

## Macros

- #define [IFX\\_IS\\_PARTITION\\_PRIVILEGED\(p\\_info\)](#)
- #define [IFX\\_ASSET\\_ATTR\\_IS\\_SET\(attr, mask\)](#) (((uint32\_t)(attr)) & ((uint32\_t)(mask))) != 0U

- `#define IFX_ASSET_ATTR_COMBINE(attr_0, attr_1) ((enum assets_attribute_t)((uint32_t)(attr_0) | (uint32_t)(attr_1)))`
- `#define IFX_SPM_BOUNDARY ((uintptr_t)0xFFFFFFFFU)`
- `#define IFX_PROTECT_SPM_DOMAIN ((uintptr_t)0xC273706DU)`
- `#define IFX_PROTECT_PSA_ROT_SINGLE_DOMAIN ((uintptr_t)0xA5505341U)`
- `#define IFX_PROTECT_APP_ROT_SINGLE_DOMAIN ((uintptr_t)0xB6616050U)`
- `#define IFX_PROTECT_IS_PRIVILEGED_DOMAIN_PRIVATE(privileged, ...) ((privileged) != 0)`
- `#define IFX_PROTECT_IS_PRIVILEGED_DOMAIN(domain) IFX_PROTECT_IS_PRIVILEGED_DOMAIN_PRIVATE(domain)`
- `#define IFX_PROTECT_SPM_DOMAIN_CFG (1)`
- `#define IFX_PROTECT_PSA_ROT_DOMAIN_CFG (IFX_PSA_ROT_PRIVILEGED)`
- `#define IFX_PROTECT_APP_ROT_DOMAIN_CFG (IFX_APP_ROT_PRIVILEGED)`
- `#define IFX_PROTECT_IS_DOMAIN_EQUAL(domain1, domain2) (IFX_PROTECT_IS_PRIVILEGED_DOMAIN(domain1) == IFX_PROTECT_IS_PRIVILEGED_DOMAIN(domain2))`
- `#define IFX_GET_PARTITION_PC(p_info, secure) fih_int_encode(IFX_PC_TFM_SPM_ID)`  
*Return partition's Protection Context using partition load info.*
- `#define IFX_GET_BOUNDARY_DOMAIN(boundary)`

## Typedefs

- `typedef struct ifx_ppc_regions_config_t ifx_ppc_regions_config_t`
- `typedef struct ifx_partition_info_t ifx_partition_info_t`

*Structure describing partition info.*

## 8.180.1 Macro Definition Documentation

### 8.180.1.1 IFX\_ASSET\_ATTR\_COMBINE

```
#define IFX_ASSET_ATTR_COMBINE(
 attr_0,
 attr_1) ((enum assets_attribute_t)((uint32_t)(attr_0) | (uint32_t)(attr_1)))
```

Definition at line 67 of file protection\_types.h.

### 8.180.1.2 IFX\_ASSET\_ATTR\_IS\_SET

```
#define IFX_ASSET_ATTR_IS_SET(
 attr,
 mask) (((uint32_t)(attr)) & ((uint32_t)(mask))) != 0U
```

Definition at line 64 of file protection\_types.h.

### 8.180.1.3 IFX\_GET\_BOUNDARY\_DOMAIN

```
#define IFX_GET_BOUNDARY_DOMAIN(
 boundary)
```

**Value:**

```
((boundary) == IFX_SPM_BOUNDARY) \
? IFX_PROTECT_SPM_DOMAIN \
: ((const ifx_partition_info_t *)boundary)->ifx_domain)
```

Definition at line 172 of file protection\_types.h.

**8.180.1.4 IFX\_GET\_PARTITION\_PC**

```
#define IFX_GET_PARTITION_PC(
 p_info,
 secure) fih_int_encode(IFX_PC_TFM_SPM_ID)
```

Return partition's Protection Context using partition load info.

**Parameters**

|    |               |                                                                     |
|----|---------------|---------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition load info <a href="#">ifx_partition_info_t</a> |
| in | <i>secure</i> | Boolean value, true for secure access                               |

Definition at line 167 of file `protection_types.h`.

**8.180.1.5 IFX\_IS\_PARTITION\_PRIVILEGED**

```
#define IFX_IS_PARTITION_PRIVILEGED(
 p_info)
```

**Value:**

```
(IFX_FIH_EQ((p_info)->ifx_ldinfo->privileged, \
 IFX_FIH_TRUE))
```

Definition at line 61 of file `protection_types.h`.

**8.180.1.6 IFX\_PROTECT\_APP\_ROT\_DOMAIN\_CFG**

```
#define IFX_PROTECT_APP_ROT_DOMAIN_CFG (IFX_APP_ROT_PRIVILEGED)
```

Definition at line 139 of file `protection_types.h`.

**8.180.1.7 IFX\_PROTECT\_APP\_ROT\_SINGLE\_DOMAIN**

```
#define IFX_PROTECT_APP_ROT_SINGLE_DOMAIN ((uintptr_t)0xB6616050U)
```

Definition at line 79 of file `protection_types.h`.

**8.180.1.8 IFX\_PROTECT\_IS\_DOMAIN\_EQUAL**

```
#define IFX_PROTECT_IS_DOMAIN_EQUAL(
 domain1,
 domain2) (IFX_PROTECT_IS_PRIVILEGED_DOMAIN(domain1) == IFX_PROTECT_IS_PRIVILEGED_DOMAIN(domain2))
```

Definition at line 152 of file `protection_types.h`.

**8.180.1.9 IFX\_PROTECT\_IS\_PRIVILEGED\_DOMAIN**

```
#define IFX_PROTECT_IS_PRIVILEGED_DOMAIN(
 domain) IFX_PROTECT_IS_PRIVILEGED_DOMAIN_PRIVATE domain
```

Definition at line 90 of file `protection_types.h`.

**8.180.1.10 IFX\_PROTECT\_IS\_PRIVILEGED\_DOMAIN\_PRIVATE**

```
#define IFX_PROTECT_IS_PRIVILEGED_DOMAIN_PRIVATE(
 privileged,
 ...) ((privileged) != 0)
```

Definition at line 82 of file `protection_types.h`.



**8.180.1.11 IFX\_PROTECT\_PSA\_ROT\_DOMAIN\_CFG**

```
#define IFX_PROTECT_PSA_ROT_DOMAIN_CFG (IFX_PSA_ROT_PRIVILEGED)
```

Definition at line 136 of file protection\_types.h.

**8.180.1.12 IFX\_PROTECT\_PSA\_ROT\_SINGLE\_DOMAIN**

```
#define IFX_PROTECT_PSA_ROT_SINGLE_DOMAIN ((uintptr_t)0xA5505341U)
```

Definition at line 76 of file protection\_types.h.

**8.180.1.13 IFX\_PROTECT\_SPM\_DOMAIN**

```
#define IFX_PROTECT_SPM_DOMAIN ((uintptr_t)0xC273706DU)
```

Definition at line 73 of file protection\_types.h.

**8.180.1.14 IFX\_PROTECT\_SPM\_DOMAIN\_CFG**

```
#define IFX_PROTECT_SPM_DOMAIN_CFG (1)
```

Definition at line 133 of file protection\_types.h.

**8.180.1.15 IFX\_SPM\_BOUNDARY**

```
#define IFX_SPM_BOUNDARY ((uintptr_t)0xFFFFFFFFFU)
```

Definition at line 70 of file protection\_types.h.

**8.180.2 Typedef Documentation****8.180.2.1 ifx\_partition\_info\_t**

```
typedef struct ifx_partition_info_t ifx_partition_info_t
```

Structure describing partition info.  
Pointer to this structure is used to provide boundary for isolation HAL.

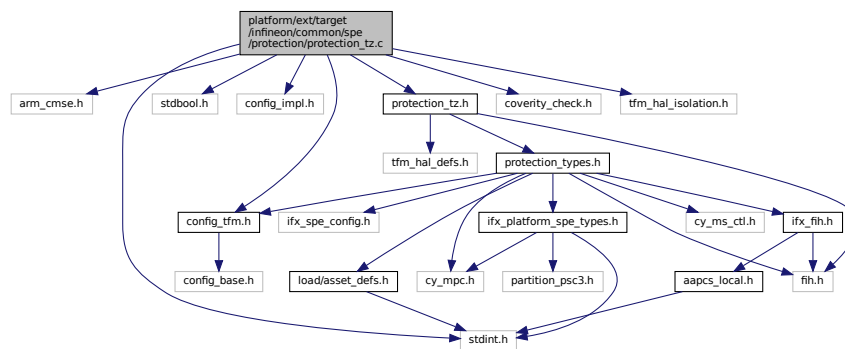
**8.180.2.2 ifx\_ppc\_regions\_config\_t**

```
typedef struct ifx_ppc_regions_config_t ifx_ppc_regions_config_t
```

**8.181 platform/ext/target/infineon/common/spe/protection/protection\_tz.c File Reference**

```
#include <arm_cmse.h>
#include <stdint.h>
#include <stdbool.h>
#include "config_impl.h"
#include "config_tfm.h"
#include "protection_tz.h"
#include "coverity_check.h"
```

```
#include "tfm_hal_isolation.h"
Include dependency graph for protection_tz.c:
```



## Functions

- `if ((UINTPTR_MAX - p) < size)`
- `if (flags & (~known))`
- `switch (flags & known_secure_level)`
- `if (permb.value != perme.value)`
- `switch (flags & (~known_secure_level))`
- `FIH_RET (TFM_HAL_ERROR_MEM_FAULT)`
- `if (IS_NS_AGENT_MAILBOX(p_info->p_ldinfo) && IFX_FIH_EQ(is_non_secure, IFX_FIH_TRUE))`
- `void fih_delay ()`
- `if ((access_type & TFM_HAL_ACCESS_WRITABLE) == TFM_HAL_ACCESS_WRITABLE)`
- `else if ((access_type & TFM_HAL_ACCESS_READABLE) != 0u)`
- `if (IS_NS_AGENT_TZ(p_info->p_ldinfo))`
- `if (FIH_NOT_EQ(fih_rc, TFM_HAL_SUCCESS))`
- `FIH_RET (fih_rc)`

## Variables

- static `size_t size`
- static `size_t uint32_t flags`
- `char * pb = (char *) p`
- `char * pe = pb + size - 1`
- `uint32_t known`
- `uint32_t known_secure_level = CMSE_MPU_UNPRIV`
- `const bool singleCheck = (((uintptr_t)pb ^ (uintptr_t)pe) < 32U)`
- `void perme`
- `uintptr_t base`
- `uintptr_t size_t uint32_t access_type`
- `IFX_FIH_BOOL is_non_secure = ((access_type & TFM_HAL_ACCESS_NS) != 0u) ? IFX_FIH_TRUE ↔ : IFX_FIH_FALSE`
- `else`

### 8.181.1 Function Documentation

**8.181.1.1 fih\_delay()**

```
void fih_delay ()
```

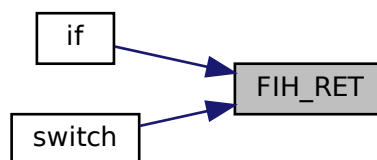
**8.181.1.2 FIH\_RET() [1/2]**

```
FIH_RET (
 fih_rc)
```

**8.181.1.3 FIH\_RET() [2/2]**

```
FIH_RET (
 TFM_HAL_ERROR_MEM_FAULT)
```

Here is the caller graph for this function:

**8.181.1.4 if() [1/8]**

```
else if (
 (access_type &TFM_HAL_ACCESS_READABLE) != 0u)
```

Definition at line 172 of file protection\_tz.c.

**8.181.1.5 if() [2/8]**

```
if (
 (access_type &TFM_HAL_ACCESS_WRITABLE) == TFM_HAL_ACCESS_WRITABLE)
```

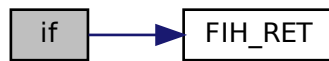
Definition at line 170 of file protection\_tz.c.

**8.181.1.6 if() [3/8]**

```
if ()
```

Definition at line 40 of file protection\_tz.c.

Here is the call graph for this function:



#### 8.181.1.7 if() [4/8]

```
if (
 FIH_NOT_EQ(fih_rc, TFM_HAL_SUCCESS))
```

Definition at line 207 of file protection\_tz.c.

Here is the call graph for this function:



#### 8.181.1.8 if() [5/8]

```
if (
 flags & ~known)
```

Definition at line 53 of file protection\_tz.c.

Here is the call graph for this function:



#### 8.181.1.9 if() [6/8]

```
if (
 IS_NS_AGENT_MAILBOX(p_info->p_ldinfo) &&IFX_FIH_EQ(is_non_secure, IFX_FIH_TRUE)
)
```

Definition at line 160 of file protection\_tz.c.

Here is the call graph for this function:



#### 8.181.1.10 if() [7/8]

```
if (
 IS_NS_AGENT_TZ(p_info->p_ldinfo))
```

Definition at line 180 of file protection\_tz.c.

#### 8.181.1.11 if() [8/8]

```
if (
 permb.value != perme.value)
```

Definition at line 89 of file protection\_tz.c.

Here is the call graph for this function:



#### 8.181.1.12 switch() [1/2]

```
switch (
 flags & ~known_secure_level)
```

Definition at line 99 of file protection\_tz.c.

Here is the call graph for this function:



### 8.181.1.13 switch() [2/2]

```
switch (
 flags & known_secure_level)
```

Definition at line 62 of file protection\_tz.c.

Here is the call graph for this function:



## 8.181.2 Variable Documentation

### 8.181.2.1 access\_type

```
uintptr_t size_t uint32_t access_type
```

**Initial value:**

```
{
 TFM_COVERITY_DEVIATE_LINE(MISRA_C_2023_Rule_10_1, "Definition of FIH_SET() treats fih_delay() returned
 int as a bool")
 IFX_FIH_DECLARE(enum tfm_hal_status_t, fih_rc, TFM_HAL_ERROR_GENERIC)
```

Definition at line 145 of file protection\_tz.c.

### 8.181.2.2 base

```
uintptr_t base
```

Definition at line 142 of file protection\_tz.c.

### 8.181.2.3 else

```
else
```

**Initial value:**

```
{
 FIH_RET(TFM_HAL_ERROR_INVALID_INPUT)
```

Definition at line 174 of file protection\_tz.c.

### 8.181.2.4 flags

```
uint32_t flags
```

**Initial value:**

```
{
 cmse_address_info_t permb, perme
```

Definition at line 35 of file protection\_tz.c.

### 8.181.2.5 is\_non\_secure

```
IFX_FIH_BOOL is_non_secure = ((access_type & TFM_HAL_ACCESS_NS) != 0u) ? IFX_FIH_TRUE ↔
: IFX_FIH_FALSE
```

Definition at line 152 of file protection\_tz.c.

**8.181.2.6 known**

```
uint32_t known
```

**Initial value:**

```
= ((uint32_t)CMSE_MPU_UNPRIV)
 | ((uint32_t)CMSE_MPU_READWRITE)
 | ((uint32_t)CMSE_MPU_READ)
```

Definition at line 45 of file protection\_tz.c.

**8.181.2.7 known\_secure\_level**

```
uint32_t known_secure_level = CMSE_MPU_UNPRIV
```

Definition at line 48 of file protection\_tz.c.

**8.181.2.8 pb**

```
char* pb = (char *) p
```

Definition at line 37 of file protection\_tz.c.

**8.181.2.9 pe**

```
pe = pb + size - 1
```

Definition at line 37 of file protection\_tz.c.

**8.181.2.10 perme**

```
void perme
```

Definition at line 94 of file protection\_tz.c.

**8.181.2.11 singleCheck**

```
const bool singleCheck = (((uintptr_t)pb ^ (uintptr_t)pe) < 32U)
```

Definition at line 59 of file protection\_tz.c.

**8.181.2.12 size**

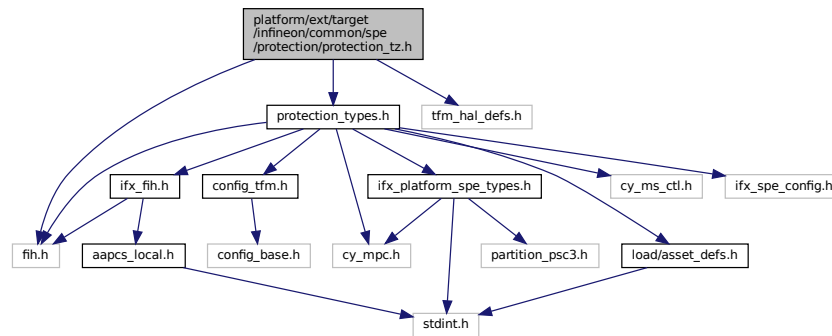
```
uintptr_t size_t size
```

Definition at line 33 of file protection\_tz.c.

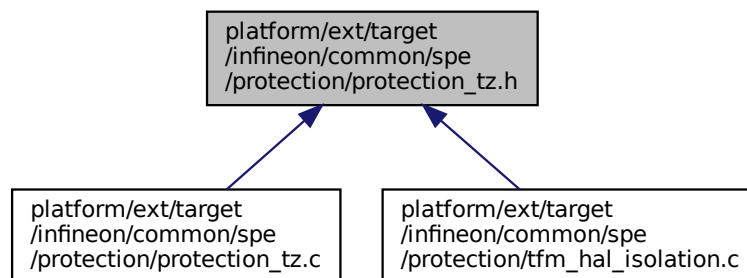
## 8.182 platform/ext/target/infineon/common/spe/protection/protection\_tz.h File Reference

```
#include "fih.h"
#include "protection_types.h"
#include "tfm_hal_defs.h"
```

Include dependency graph for `protection_tz.h`:



This graph shows which files directly or indirectly include this file:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum `tfm_hal_status_t`) `ifx_tz_memory_check(const ifx_partition_info_t *p_info`  
Check access to memory region by TrustZone (SAU, IDAU, MPU).

## Variables

- `uintptr_t` [base](#)
- `uintptr_t` `size_t` [size](#)
- `uintptr_t` `size_t` `uint32_t` [access\\_type](#)

## 8.182.1 Function Documentation

### 8.182.1.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t) const
```

Check access to memory region by TrustZone (SAU, IDAU, MPU).



## Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info                                                  |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |
| in | <i>privileged</i>  | Is partition privileged.                                                   |

## Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by TrustZone. TFM\_HAL\_ERROR\_↵  
 R\_MEM\_FAULT - The memory region has not the access permissions by TrustZone. TFM\_HAL\_ERROR\_↵  
 INVALID\_INPUT - Invalid inputs.

## 8.182.2 Variable Documentation

## 8.182.2.1 access\_type

uintptr\_t size\_t uint32\_t access\_type  
 Definition at line 34 of file protection\_tz.h.

## 8.182.2.2 base

uintptr\_t base  
 Definition at line 32 of file protection\_tz.h.

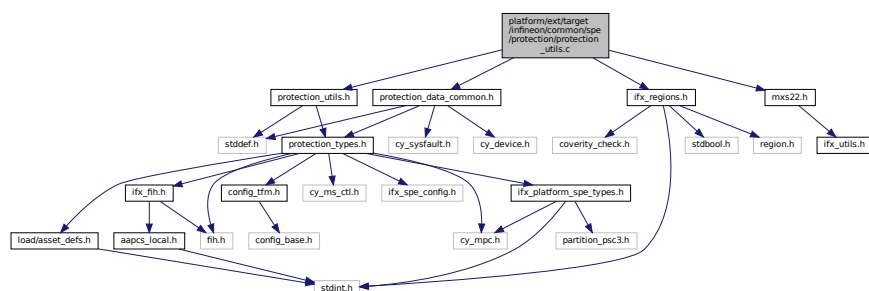
## 8.182.2.3 size

uintptr\_t size\_t size  
 Definition at line 33 of file protection\_tz.h.

8.183 platform/ext/target/infineon/common/spe/protection/protection\_↵  
utils.c File Reference

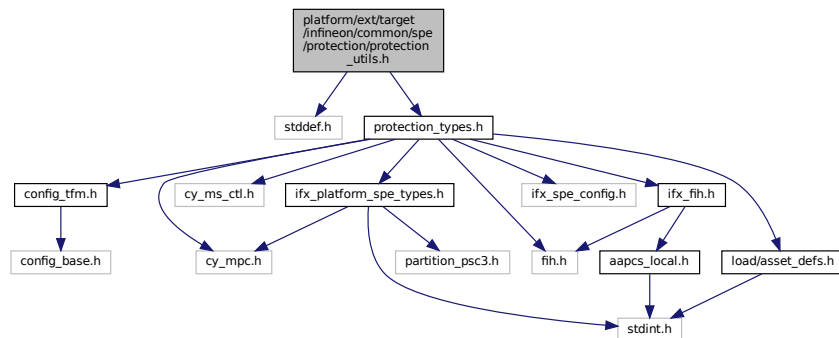
```
#include "protection_utils.h"
#include "protection_data_common.h"
#include "ifx_regions.h"
#include "mxs22.h"
```

Include dependency graph for protection\_utils.c:

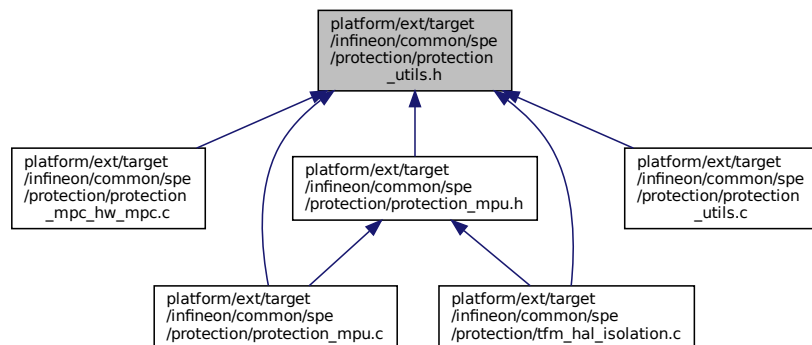


## 8.184 platform/ext/target/infineon/common/spe/protection/protection\_utils.h File Reference

```
#include <stddef.h>
#include "protection_types.h"
Include dependency graph for protection_utils.h:
```



This graph shows which files directly or indirectly include this file:

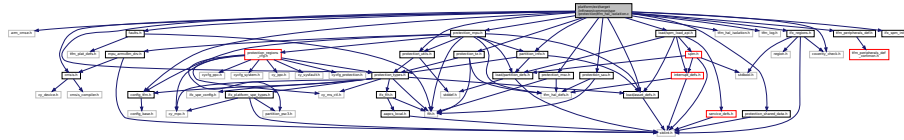


## 8.185 platform/ext/target/infineon/common/spe/protection/tfm\_hal\_isolation.c File Reference

```
#include <arm_cmse.h>
#include "cmsis.h"
#include "config_tfm.h"
#include "faults.h"
#include "protection_types.h"
#include "protection_mpu.h"
#include "protection_msc.h"
#include "protection_sau.h"
#include "protection_tz.h"
#include "partition_info.h"
#include "ifx_regions.h"
#include "tfm_hal_isolation.h"
#include "tfm_log.h"
```

```
#include "tfm_peripherals_def.h"
#include "load/spm_load_api.h"
#include "load/asset_defs.h"
#include "protection_shared_data.h"
#include "protection_utils.h"
#include "ifx_spm_init.h"
#include "coverity_check.h"
```

Include dependency graph for tfm\_hal\_isolation.c:



## Functions

- `if ((p_info==NULL)|| (cfg==NULL))`
- `for (asset_idx=0;asset_idx< asset_cnt;asset_idx++)`
- `FIH_RET (TFM_HAL_SUCCESS)`
- `if (p_ldinfo->nassets !=0U)`
- `if (p_info->asset_cnt !=0U)`
- `for (uint32_t i=0;i< p_info->peri_region_count;i++)`
- `TFM_COVERITY_DEVIATE_LINE (MISRA_C_2023_Rule_10_1, "Definition of FIH_SET() treats fih_delay() returned int as a bool")`
- `if (base==0U)`
- `if (base >(UINTPTR_MAX - size))`
- `if ((access_type & ~(TFM_HAL_ACCESS_READABLE|TFM_HAL_ACCESS_WRITABLE|TFM_HAL_ACCESS_NS)) !=0U)`
- `FIH_CALL (ifx_tz_memory_check, fih_rc, p_info, base, size, access_type)`
- `if (FIH_NOT_EQ(fih_rc, TFM_HAL_SUCCESS))`
- `FIH_CALL (ifx_mpc_memory_check, fih_rc, p_info, base, size, access_type)`
- `FIH_RET (fih_rc)`
- `if ((p_ldinfo==NULL)|| (p_boundary==NULL))`
- `FIH_CALL (ifx_protect_partition_assets, result, p_info, p_info->ifx_ldinfo->protect_cfg_enabled)`
- `if (FIH_NOT_EQ(result, TFM_HAL_SUCCESS))`
- `FIH_RET_TYPE (bool)`
- `__DMB ()`
- `TFM_COVERITY_DEVIATE_BLOCK (MISRA_C_2023_Rule_10_1, "Definition of FIH_SET() treats fih_delay() returned int as a bool")`
- `__set_CONTROL (ctrl.w)`
- `__DSB ()`
- `__ISB ()`
- `FIH_RET (result)`

## Variables

- static `cy_en_prot_region_t asset`
- static const `ifx_protection_region_config_t * cfg`
- static const `ifx_protection_region_config_t const struct asset_desc_t * p_assets`
- static const `ifx_protection_region_config_t const struct asset_desc_t uint32_t asset_cnt`
- const struct `partition_load_info_t * p_ldinfo = p_info->p_ldinfo`
- `uintptr_t base`
- `uintptr_t size_t size`
- `uintptr_t size_t uint32_t access_type`

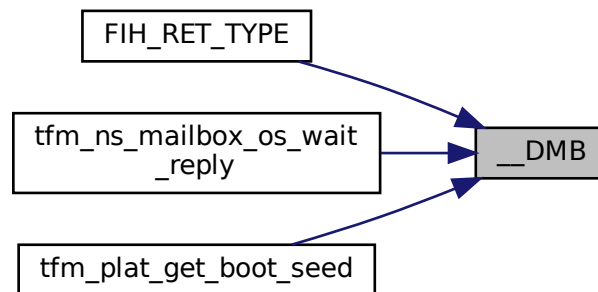
- uintptr\_t \* p\_boundary
- const ifx\_partition\_info\_t \* p\_info = ifx\_protection\_get\_partition\_info(p\_ldinf)
- uintptr\_t boundary
- CONTROL\_Type ctrl
- else

## 8.185.1 Function Documentation

### 8.185.1.1 \_\_DMB()

\_\_DMB ( )

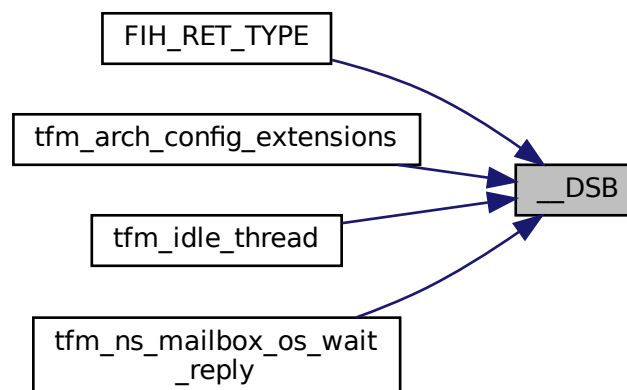
Here is the caller graph for this function:



### 8.185.1.2 \_\_DSB()

\_\_DSB ( )

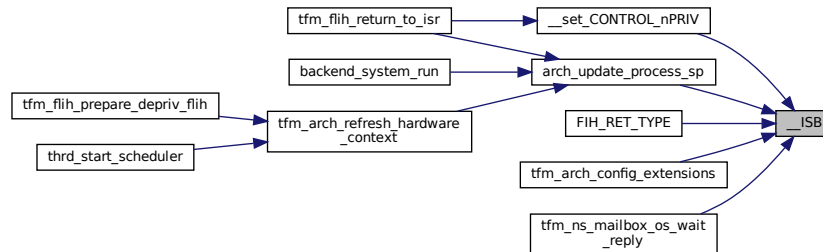
Here is the caller graph for this function:



### 8.185.1.3 \_\_ISB()

`__ISB ( )`

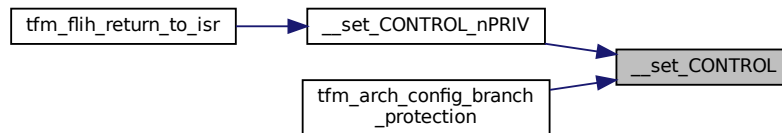
Here is the caller graph for this function:



### 8.185.1.4 \_\_set\_CONTROL()

`__set_CONTROL (`  
    `ctrl. w )`

Here is the caller graph for this function:



### 8.185.1.5 FIH\_CALL() [1/3]

```

FIH_CALL (
 ifx_mpc_memory_check ,
 fih_rc ,
 p_info ,
 base ,
 size ,
 access_type)

```

### 8.185.1.6 FIH\_CALL() [2/3]

```

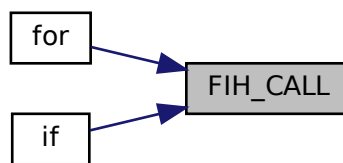
FIH_CALL (
 ifx_protect_partition_assets ,
 result ,
 p_info ,
 p_info->ifx_ldinfo-> protect_cfg_enabled)

```

**8.185.1.7 FIH\_CALL()** [3/3]

```
FIH_CALL (
 ifx_tz_memory_check ,
 fih_rc ,
 p_info ,
 base ,
 size ,
 access_type)
```

Here is the caller graph for this function:

**8.185.1.8 FIH\_RET()** [1/3]

```
FIH_RET (
 fih_rc)
```

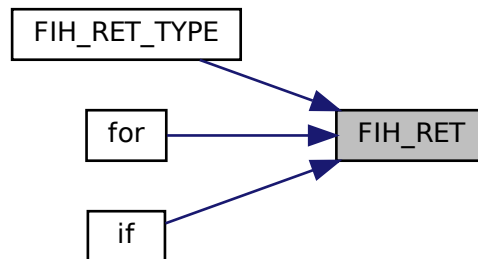
**8.185.1.9 FIH\_RET()** [2/3]

```
FIH_RET (
 result)
```

**8.185.1.10 FIH\_RET()** [3/3]

```
FIH_RET (
 TFM_HAL_SUCCESS)
```

Here is the caller graph for this function:

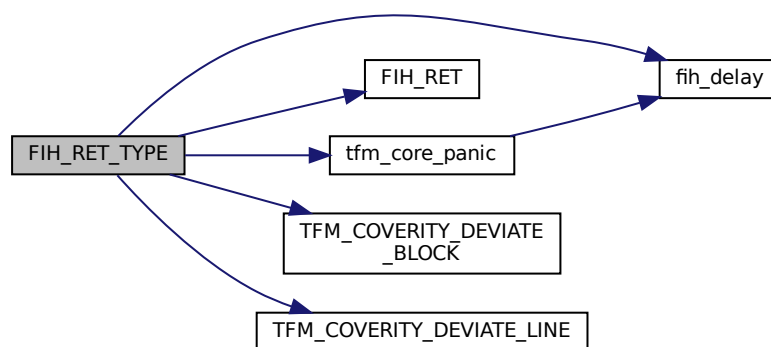


#### 8.185.1.11 `FIH_RET_TYPE()`

```
FIH_RET_TYPE (
 bool)
```

Definition at line 740 of file `tfm_hal_isolation.c`.

Here is the call graph for this function:

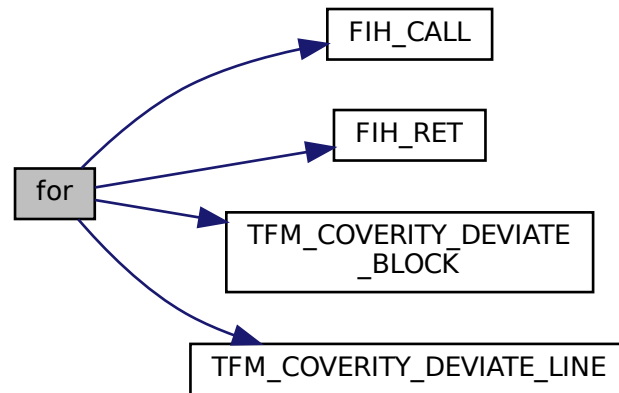


#### 8.185.1.12 `for()` [1/2]

```
for ()
```

Definition at line 144 of file `tfm_hal_isolation.c`.

Here is the call graph for this function:

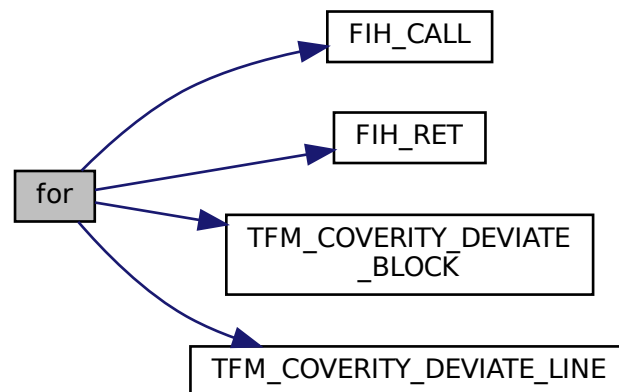


#### 8.185.1.13 for() [2/2]

```
for (
 uint32_t i = 0; i < p_info->peri_region_count; i++)
```

Definition at line 370 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:



#### 8.185.1.14 if() [1/9]

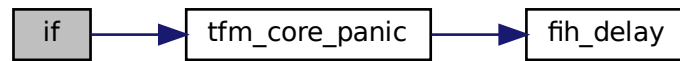
```
if (
```



```
(access_type &~(TFM_HAL_ACCESS_READABLE|TFM_HAL_ACCESS_WRITABLE|TFM_HAL_ACCESS_↵
NS)) != 0U)
```

Definition at line 581 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:

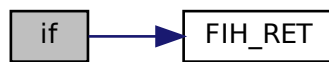


#### 8.185.1.15 if() [2/9]

```
if (
 (p_info==NULL) || (cfg==NULL))
```

Definition at line 140 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:

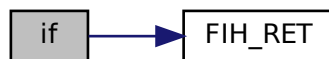


#### 8.185.1.16 if() [3/9]

```
if (
 (p_ldinf==NULL) || (p_boundary==NULL))
```

Definition at line 630 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:



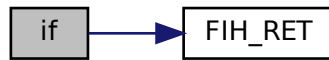
#### 8.185.1.17 if() [4/9]

```
if (
```

```
base ,
(UINTPTR_MAX - size))
```

Definition at line 576 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:



#### 8.185.1.18 if() [5/9]

```
if (
 base == 0U)
```

Definition at line 571 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:



#### 8.185.1.19 if() [6/9]

```
if (
 FIH_NOT_EQ(fih_rc, TFM_HAL_SUCCESS))
```

Definition at line 593 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:



#### 8.185.1.20 if() [7/9]

```
if (
 FIH_NOT_EQ(result, TFM_HAL_SUCCESS))
```

Definition at line 683 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:

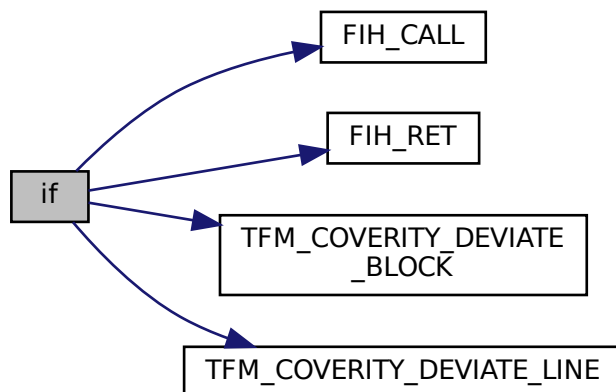


#### 8.185.1.21 `if()` [8/9]

```
if (
 p_info->asset_cnt != 0U)
```

Definition at line 276 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:

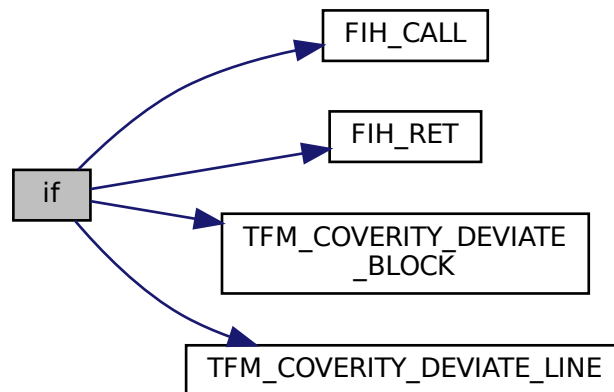


#### 8.185.1.22 `if()` [9/9]

```
if (
 p_ldinfo->nassets != 0U)
```

Definition at line 262 of file tfm\_hal\_isolation.c.

Here is the call graph for this function:

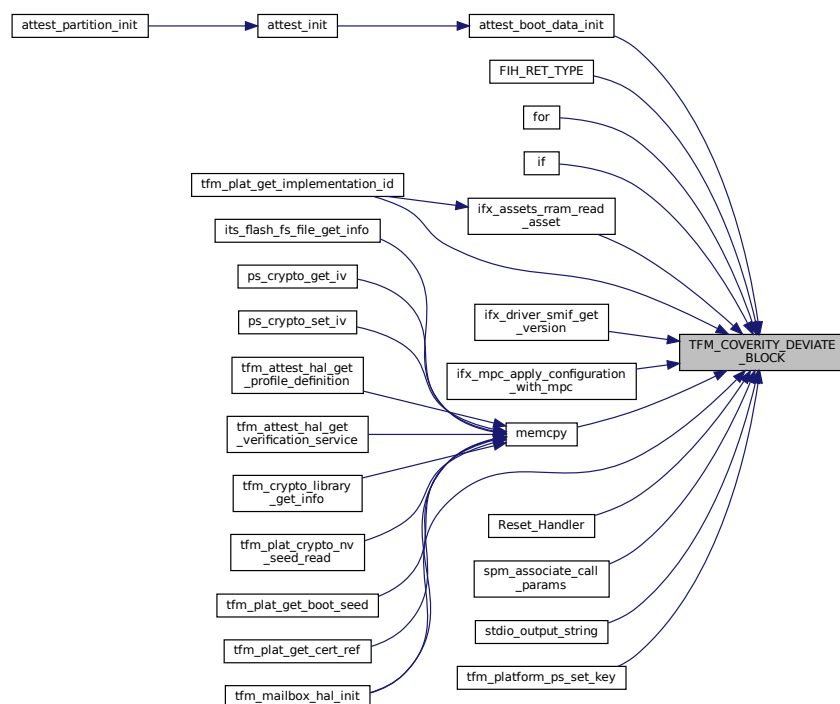


#### 8.185.1.23 TFM\_COVERITY\_DEVIATE\_BLOCK()

```
TFM_COVERITY_DEVIATE_BLOCK (
 MISRA_C_2023_Rule_10_1 ,
 "Definition of FIH_SET() treats fih_delay\(\) returned int as a bool")
```

Definition at line 813 of file tfm\_hal\_isolation.c.

Here is the caller graph for this function:

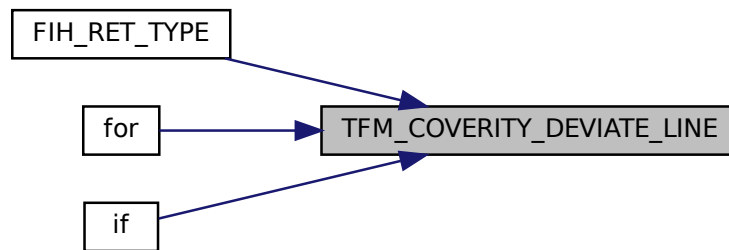


#### 8.185.1.24 TFM\_COVERITY\_DEVIATE\_LINE()

```
TFM_COVERITY_DEVIATE_LINE (
 MISRA_C_2023_Rule_10_1 ,
 "Definition of FIH_SET() treats fih_delay() returned int as a bool")
```

Definition at line 543 of file tfm\_hal\_isolation.c.

Here is the caller graph for this function:



### 8.185.2 Variable Documentation

#### 8.185.2.1 access\_type

```
uintptr_t size_t uint32_t access_type
```

**Initial value:**

```
{
 const ifx_partition_info_t *p_info = (const ifx_partition_info_t *)boundary
```

Definition at line 539 of file tfm\_hal\_isolation.c.

#### 8.185.2.2 asset

```
cy_en_prot_region_t asset
```

Definition at line 70 of file tfm\_hal\_isolation.c.

#### 8.185.2.3 asset\_cnt

```
const ifx_protection_region_config_t const struct asset_desc_t uint32_t asset_cnt
```

**Initial value:**

```
{
 size_t asset_idx
```

Definition at line 136 of file tfm\_hal\_isolation.c.

#### 8.185.2.4 base

```
uintptr_t base
```

Definition at line 537 of file tfm\_hal\_isolation.c.

### 8.185.2.5 boundary

uintptr\_t boundary

**Initial value:**

```
{
 FIH_RET_TYPE(enum tfm_hal_status_t) result
```

Definition at line 801 of file tfm\_hal\_isolation.c.

### 8.185.2.6 cfg

static const ifx\_protection\_region\_config\_t \* cfg

**Initial value:**

```
{
 FIH_RET_TYPE(enum tfm_hal_status_t) result
```

Definition at line 133 of file tfm\_hal\_isolation.c.

### 8.185.2.7 ctrl

CONTROL\_Type ctrl

Definition at line 803 of file tfm\_hal\_isolation.c.

### 8.185.2.8 else

else

**Initial value:**

```
{

 ctrl.b.nPRIV = 0
```

Definition at line 918 of file tfm\_hal\_isolation.c.

### 8.185.2.9 p\_assets

const ifx\_protection\_region\_config\_t const struct asset\_desc\_t\* p\_assets

Definition at line 134 of file tfm\_hal\_isolation.c.

### 8.185.2.10 p\_boundary

\* p\_boundary

**Initial value:**

```
{
 FIH_RET_TYPE(enum tfm_hal_status_t) result
```

Definition at line 627 of file tfm\_hal\_isolation.c.

### 8.185.2.11 p\_info

const ifx\_partition\_info\_t\* p\_info = ifx\_protection\_get\_partition\_info(p\_ldinf)

Definition at line 635 of file tfm\_hal\_isolation.c.

### 8.185.2.12 p\_ldinf

const struct partition\_load\_info\_t\* p\_ldinf = p\_info->p\_ldinfo

Definition at line 261 of file tfm\_hal\_isolation.c.

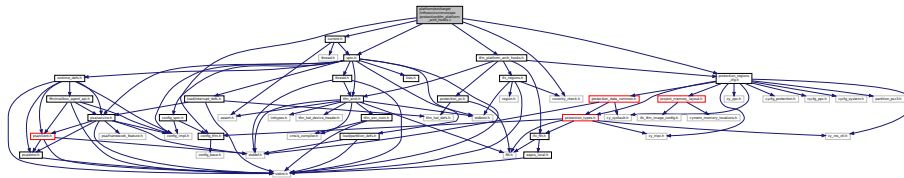
### 8.185.2.13 size

uintptr\_t size\_t size

Definition at line 538 of file tfm\_hal\_isolation.c.

## 8.186 platform/ext/target/infineon/common/spe/protection/tfm\_platform\_arch\_hooks.c File Reference

```
#include "config_tfm.h"
#include "current.h"
#include "tfm_platform_arch_hooks.h"
#include "spm.h"
#include "coverity_check.h"
#include "protection_regions_cfg.h"
Include dependency graph for tfm_platform_arch_hooks.c:
```



## Functions

- [ifx\\_aapcs\\_fih\\_int ifx\\_arch\\_get\\_context\\_protection\\_context](#) (const struct [context\\_ctrl\\_t](#) \*prev\_ctx, const struct [context\\_ctrl\\_t](#) \*cur\_ctx, uint32\_t exc\_return)
- [ifx\\_aapcs\\_fih\\_int ifx\\_arch\\_get\\_current\\_component\\_protection\\_context](#) (uint32\_t exc\_return)

### 8.186.1 Function Documentation

#### 8.186.1.1 ifx\_arch\_get\_context\_protection\_context()

```
ifx_aapcs_fih_int ifx_arch_get_context_protection_context (
 const struct context_ctrl_t * prev_ctx,
 const struct context_ctrl_t * cur_ctx,
 uint32_t exc_return)
```

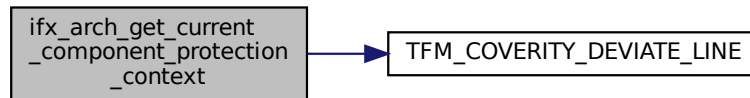
Definition at line 35 of file `tfm_platform_arch_hooks.c`.

#### 8.186.1.2 ifx\_arch\_get\_current\_component\_protection\_context()

```
ifx_aapcs_fih_int ifx_arch_get_current_component_protection_context (
 uint32_t exc_return)
```

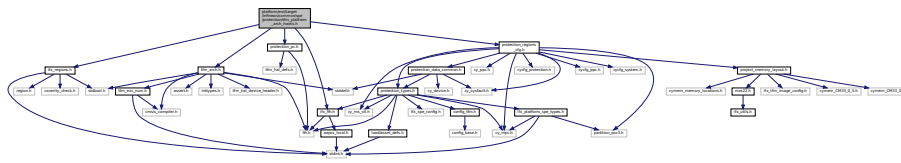
Definition at line 43 of file `tfm_platform_arch_hooks.c`.

Here is the call graph for this function:

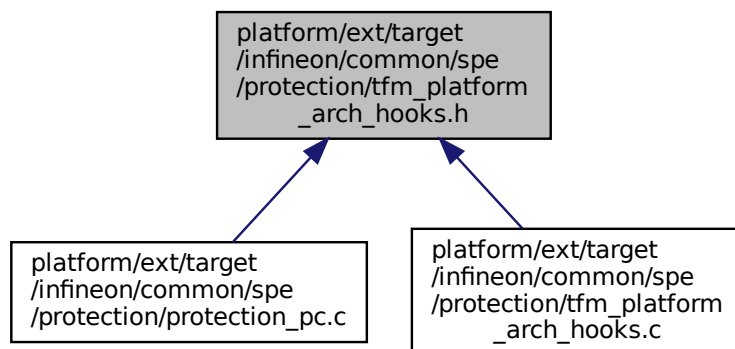


## 8.187 platform/ext/target/infineon/common/spe/protection/tfm\_platform\_arch\_hooks.h File Reference

```
#include "ifx_fih.h"
#include "ifx_regions.h"
#include "tfm_arch.h"
#include "protection_pc.h"
#include "protection_regions_cfg.h"
Include dependency graph for tfm_platform_arch_hooks.h:
```



This graph shows which files directly or indirectly include this file:



## Macros

- #define [IFX\\_PLATFORM\\_EXIT\\_FROM\\_INTERRUPT\\_WITH\\_PC\\_RESTORE\\_HOOK](#)
- #define [PLATFORM\\_SVC\\_HANDLER\\_GET\\_MSP\\_HOOK](#)(reg)
- #define [PLATFORM\\_SVC\\_HANDLER\\_EXIT\\_HOOK](#)
- #define [PLATFORM\\_INIT\\_SPM\\_FUNC\\_CONTEXT\\_HOOK](#)(msp, ctx)



*Hook to update isolation before switching to SPM thread function.*

- `#define PLATFORM_THREAD_MODE_SPM_RETURN_HOOK(exc_return, msp)`

*Hook to update isolation to handle exit from SPM thread function.*

- `#define PLATFORM_SVC_HANDLER_HOOK_IAR_REQUIRED`
- `#define PLATFORM_SVC_HANDLER_TO_FLIH_FUNC_ENTER_HOOK`
- `#define PLATFORM_SVC_HANDLER_TO_FLIH_FUNC_EXIT_HOOK`
- `#define PLATFORM_SVC_HANDLER_FROM_FLIH_FUNC_ENTER_HOOK`
- `#define PLATFORM_SVC_HANDLER_FROM_FLIH_FUNC_EXIT_HOOK`
- `#define PLATFORM_PENDSV_HANDLER_HOOK_IAR_REQUIRED`
- `#define PLATFORM_PENDSV_HANDLER_EXIT_HOOK`
- `#define PLATFORM_HARD_FAULT_HANDLER_HOOK_VALIDATE_PC()`
- `#define PLATFORM_HARD_FAULT_HANDLER_HOOK_IAR_REQUIRED _Pragma("required = ifx_arch_switch_protection_co")`
- `#define PLATFORM_HARD_FAULT_HANDLER_HOOK()`

## Functions

- `ifx_aapcs_fih_int ifx_arch_get_context_protection_context` (const struct `context_ctrl_t` \*prev\_ctx, const struct `context_ctrl_t` \*cur\_ctx, uint32\_t exc\_return)
- `ifx_aapcs_fih_int ifx_arch_get_current_component_protection_context` (uint32\_t exc\_return)
- `void ifx_arch_switch_protection_context` (uint32\_t pc)

*Perform protection context switching.*

- `void ifx_arch_switch_protection_context_from_irq` (uint32\_t pc)

*Perform protection context switching from low priority IRQ.*

## 8.187.1 Macro Definition Documentation

### 8.187.1.1 IFX\_PLATFORM\_EXIT\_FROM\_INTERRUPT\_WITH\_PC\_RESTORE\_HOOK

```
#define IFX_PLATFORM_EXIT_FROM_INTERRUPT_WITH_PC_RESTORE_HOOK
```

**Value:**

```
"cpsid i \n" \
/* Pop Protection Context from stack */ \
"pop {r0, r1} \n" \
/* Switch Protection Context */ \
"b.w ifx_arch_switch_protection_context_from_irq \n"
```

Definition at line 36 of file `tfm_platform_arch_hooks.h`.

### 8.187.1.2 PLATFORM\_HARD\_FAULT\_HANDLER\_HOOK

```
#define PLATFORM_HARD_FAULT_HANDLER_HOOK()
```

Definition at line 239 of file `tfm_platform_arch_hooks.h`.

### 8.187.1.3 PLATFORM\_HARD\_FAULT\_HANDLER\_HOOK\_IAR\_REQUIRED

```
#define PLATFORM_HARD_FAULT_HANDLER_HOOK_IAR_REQUIRED _Pragma("required = ifx_arch_switch_protection_context")
```

Definition at line 210 of file `tfm_platform_arch_hooks.h`.

### 8.187.1.4 PLATFORM\_HARD\_FAULT\_HANDLER\_HOOK\_VALIDATE\_PC

```
#define PLATFORM_HARD_FAULT_HANDLER_HOOK_VALIDATE_PC()
```

Definition at line 205 of file `tfm_platform_arch_hooks.h`.

### 8.187.1.5 PLATFORM\_INIT\_SPM\_FUNC\_CONTEXT\_HOOK

```
#define PLATFORM_INIT_SPM_FUNC_CONTEXT_HOOK(
```

```
 msp,
 ctx)
```

**Value:**

```
{ \
 /* Switch to protection context used by SPM. */ \
 /* Update saved PC stored by \ref ifx_pc2_exception_handler */ \
 ((uint32_t *)msp)[-1] = IFX_PC_TFM_SPM_ID; \
 ((uint32_t *)msp)[-2] = IFX_PC_TFM_SPM_ID; \
}
```

Hook to update isolation before switching to SPM thread function.

**Parameters**

|            |                                                                                                    |
|------------|----------------------------------------------------------------------------------------------------|
| <i>msp</i> | Address of MSP stack frame after adjusting it by <a href="#">PLATFORM_SVC_HANDLER_GET_MSP_HOOK</a> |
| <i>ctx</i> | Address of SPM PSP stack used to handle SVC request in thread mode                                 |

Definition at line 77 of file `tfm_platform_arch_hooks.h`.

### 8.187.1.6 PLATFORM\_PENDSV\_HANDLER\_EXIT\_HOOK

```
#define PLATFORM_PENDSV_HANDLER_EXIT_HOOK
```

**Value:**

```
"cpsid i \n" \
/* Pop Saved Protection Context */ \
"pop {r2, r3} \n" \
"push {lr} \n" \
"mov r2, lr \n" \
"bl ifx_arch_get_context_protection_context \n" \
"pop {lr} \n" \
"b.w ifx_arch_switch_protection_context_from_irq \n"
```

Definition at line 171 of file `tfm_platform_arch_hooks.h`.

### 8.187.1.7 PLATFORM\_PENDSV\_HANDLER\_HOOK\_IAR\_REQUIRED

```
#define PLATFORM_PENDSV_HANDLER_HOOK_IAR_REQUIRED
```

**Value:**

```
_Pragma("required = ifx_arch_get_context_protection_context") \
_Pragma("required = ifx_arch_switch_protection_context_from_irq")
```

Definition at line 162 of file `tfm_platform_arch_hooks.h`.

### 8.187.1.8 PLATFORM\_SVC\_HANDLER\_EXIT\_HOOK

```
#define PLATFORM_SVC_HANDLER_EXIT_HOOK
```

**Value:**

```
"cpsid i \n" \
/* Pop Protection Context from stack */ \
"pop {r0, r1} \n" \
/* Switch Protection Context */ \
"b.w ifx_arch_switch_protection_context \n"
```

Definition at line 54 of file `tfm_platform_arch_hooks.h`.

### 8.187.1.9 PLATFORM\_SVC\_HANDLER\_FROM\_FLIH\_FUNC\_ENTER\_HOOK

```
#define PLATFORM_SVC_HANDLER_FROM_FLIH_FUNC_ENTER_HOOK
```

**Value:**

```
/* Remove Saved PC for \ref TFM_SVC_FLIH_FUNC_RETURN SVC call from stack */ \
"pop {r0, r1} \n"
```

Definition at line 149 of file `tfm_platform_arch_hooks.h`.

**8.187.1.10 PLATFORM\_SVC\_HANDLER\_FROM\_FLIH\_FUNC\_EXIT\_HOOK**

```
#define PLATFORM_SVC_HANDLER_FROM_FLIH_FUNC_EXIT_HOOK
```

**Value:**

```
/* There is no need to switch PC, because \ref TFM_SVC_FLIH_FUNC_RETURN should
 * return to \ref spm_handle_interrupt which is running within privileged
 * PC2/SPM context */ \
 "bx lr \n"
```

Definition at line 157 of file tfm\_platform\_arch\_hooks.h.

**8.187.1.11 PLATFORM\_SVC\_HANDLER\_GET\_MSP\_HOOK**

```
#define PLATFORM_SVC_HANDLER_GET_MSP_HOOK(
 reg)
```

**Value:**

```
/* 8 bytes used to store Saved PC */ \
 "mrs reg, msp \n" \
 "add reg, #8 \n"
```

Definition at line 46 of file tfm\_platform\_arch\_hooks.h.

**8.187.1.12 PLATFORM\_SVC\_HANDLER\_HOOK\_IAR\_REQUIRED**

```
#define PLATFORM_SVC_HANDLER_HOOK_IAR_REQUIRED
```

**Value:**

```
_Pragma("required = ifx_arch_get_current_component_protection_context") \
_Pragma("required = ifx_arch_switch_protection_context")
```

Definition at line 108 of file tfm\_platform\_arch\_hooks.h.

**8.187.1.13 PLATFORM\_SVC\_HANDLER\_TO\_FLIH\_FUNC\_ENTER\_HOOK**

```
#define PLATFORM_SVC_HANDLER_TO_FLIH_FUNC_ENTER_HOOK
```

**Value:**

```
/* "Unstack" orig_exc_return, dummy, PSP, PSPLIM pushed by current handler */ \
 "pop {r0-r3} \n" \
 /* Remove Saved PC from stack */ \
 "add sp, #8 \n" \
 /* "Stack" orig_exc_return, dummy, PSP, PSPLIM pushed by current handler */ \
 "push {r0-r3} \n"
```

Definition at line 122 of file tfm\_platform\_arch\_hooks.h.

**8.187.1.14 PLATFORM\_SVC\_HANDLER\_TO\_FLIH\_FUNC\_EXIT\_HOOK**

```
#define PLATFORM_SVC_HANDLER_TO_FLIH_FUNC_EXIT_HOOK
```

**Value:**

```
"cpsid i \n" \
 "push {lr} \n" \
 "mov r0, lr \n" \
 "bl ifx_arch_get_current_component_protection_context \n" \
 "pop {lr} \n" \
 "b.w ifx_arch_switch_protection_context \n"
```

Definition at line 134 of file tfm\_platform\_arch\_hooks.h.

**8.187.1.15 PLATFORM\_THREAD\_MODE\_SPM\_RETURN\_HOOK**

```
#define PLATFORM_THREAD_MODE_SPM_RETURN_HOOK(
 exc_return,
 msp)
```

**Value:**

```
{ \
 uint32_t component_pc = ifx_arch_get_current_component_protection_context(exc_return); \
 uint32_t return_pc = component_pc; \
 ((uint32_t *)msp)[-1] = return_pc; \
 ((uint32_t *)msp)[-2] = return_pc; \
}
```

```
}
```

Hook to update isolation to handle exit from SPM thread function.  
Definition at line 98 of file `tfm_platform_arch_hooks.h`.

## 8.187.2 Function Documentation

### 8.187.2.1 ifx\_arch\_get\_context\_protection\_context()

```
ifx_aapcs_fih_int ifx_arch_get_context_protection_context (
 const struct context_ctrl_t * prev_ctx,
 const struct context_ctrl_t * cur_ctx,
 uint32_t exc_return)
```

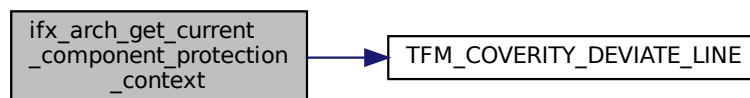
Definition at line 35 of file `tfm_platform_arch_hooks.c`.

### 8.187.2.2 ifx\_arch\_get\_current\_component\_protection\_context()

```
ifx_aapcs_fih_int ifx_arch_get_current_component_protection_context (
 uint32_t exc_return)
```

Definition at line 43 of file `tfm_platform_arch_hooks.c`.

Here is the call graph for this function:



### 8.187.2.3 ifx\_arch\_switch\_protection\_context()

```
void ifx_arch_switch_protection_context (
 uint32_t pc)
```

Perform protection context switching.

#### Note

It's unsafe to switch PC to other than PC2 (used by SPM) in low priority exception, because it's possible that there will be pending IRQ and MCU will try to use MSP stack pushing exception stack within PC not equal to PC2.

IRQ must be disabled if this function is called from exception/IRQ with privilege below 0.

Definition at line 155 of file `protection_pc.c`.

### 8.187.2.4 ifx\_arch\_switch\_protection\_context\_from\_irq()

```
void ifx_arch_switch_protection_context_from_irq (
 uint32_t pc)
```

Perform protection context switching from low priority IRQ.

This function will complete switching to PC2 (SPM) in scope of current exception. For all other protection context it will schedule switching after exit from active IRQ in thread mode by [ifx\\_arch\\_switch\\_protection\\_context\\_thread](#).

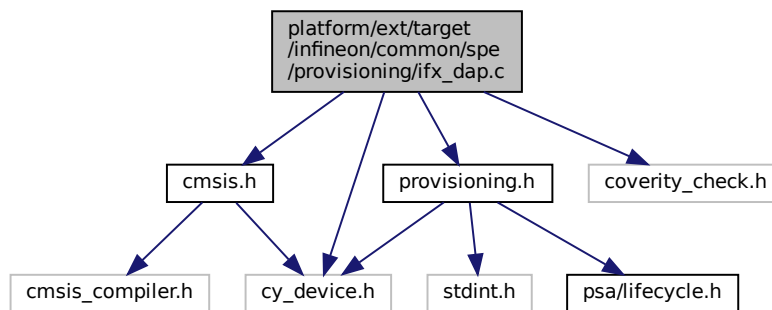
**Note**

IRQ must be disabled when this function is called.

Definition at line 248 of file protection\_pc.c.

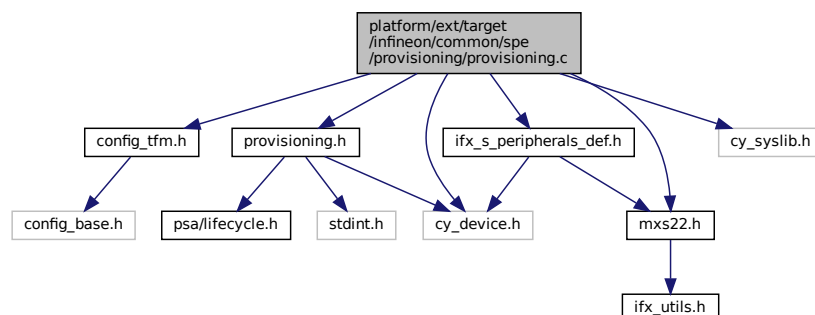
## 8.188 platform/ext/target/infineon/common/spe/provisioning/ifx\_dap.c File Reference

```
#include "cmsis.h"
#include "provisioning.h"
#include "cy_device.h"
#include "coverity_check.h"
Include dependency graph for ifx_dap.c:
```



## 8.189 platform/ext/target/infineon/common/spe/provisioning/provisioning.c File Reference

```
#include "config_tfm.h"
#include "ifx_s_peripherals_def.h"
#include "mxs22.h"
#include "provisioning.h"
#include "cy_device.h"
#include "cy_syslib.h"
Include dependency graph for provisioning.c:
```



## Functions

- uint32\_t [ifx\\_get\\_security\\_lifecycle](#) (void)

*Retrieve the security lifecycle of the device.*

### 8.189.1 Function Documentation

#### 8.189.1.1 ifx\_get\_security\_lifecycle()

```
uint32_t ifx_get_security_lifecycle (
 void)
```

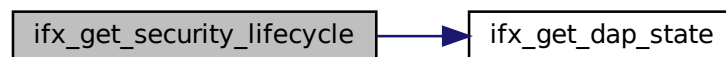
Retrieve the security lifecycle of the device.

#### Returns

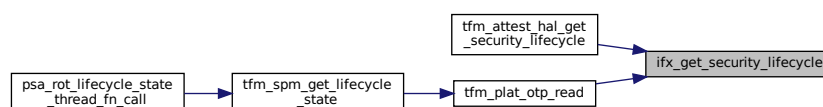
Security lifecycle state, see PSA\_LIFECYCLE\_XXX in [psa/lifecycle.h](#)

Definition at line 20 of file provisioning.c.

Here is the call graph for this function:



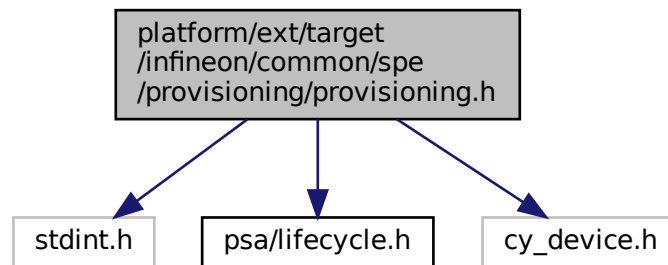
Here is the caller graph for this function:



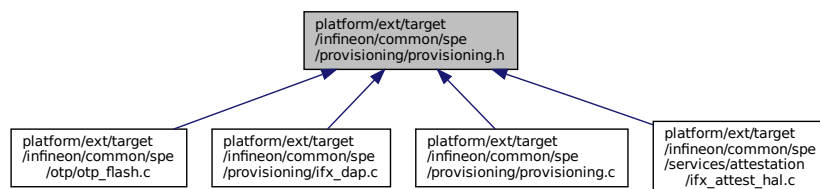
## 8.190 platform/ext/target/infineon/common/spe/provisioning/provisioning.h File Reference

```
#include <stdint.h>
#include "psa/lifecycle.h"
#include "cy_device.h"
```

Include dependency graph for provisioning.h:



This graph shows which files directly or indirectly include this file:



## Enumerations

- enum `ifx_dap_state_t` { `IFX_DAP_DISABLED`, `IFX_DAP_ENABLED_NS_ONLY`, `IFX_DAP_ENABLED_S_NS`, `IFX_DAP_UNKNOWN` }

*Enumeration used to define debug access port state.*

## Functions

- `uint32_t ifx_get_security_lifecycle (void)`  
*Retrieve the security lifecycle of the device.*
- `ifx_dap_state_t ifx_get_dap_state (void)`  
*Retrieve the state of debug access port.*

## 8.190.1 Enumeration Type Documentation

### 8.190.1.1 ifx\_dap\_state\_t

enum `ifx_dap_state_t`

Enumeration used to define debug access port state.

#### Enumerator

|                                      |  |
|--------------------------------------|--|
| <code>IFX_DAP_DISABLED</code>        |  |
| <code>IFX_DAP_ENABLED_NS_ONLY</code> |  |

## Enumerator

|                      |  |
|----------------------|--|
| IFX_DAP_ENABLED_S_NS |  |
| IFX_DAP_UNKNOWN      |  |

Definition at line 19 of file provisioning.h.

## 8.190.2 Function Documentation

### 8.190.2.1 ifx\_get\_dap\_state()

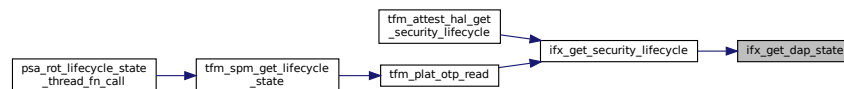
```
ifx_dap_state_t ifx_get_dap_state (
 void)
```

Retrieve the state of debug access port.

#### Returns

debug access port state, see [ifx\\_dap\\_state\\_t](#)

Here is the caller graph for this function:



### 8.190.2.2 ifx\_get\_security\_lifecycle()

```
uint32_t ifx_get_security_lifecycle (
 void)
```

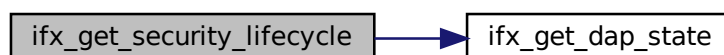
Retrieve the security lifecycle of the device.

#### Returns

Security lifecycle state, see PSA\_LIFECYCLE\_XXX in [psa/lifecycle.h](#)

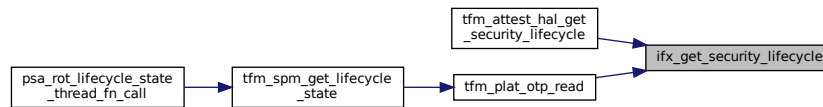
Definition at line 20 of file provisioning.c.

Here is the call graph for this function:





Here is the caller graph for this function:



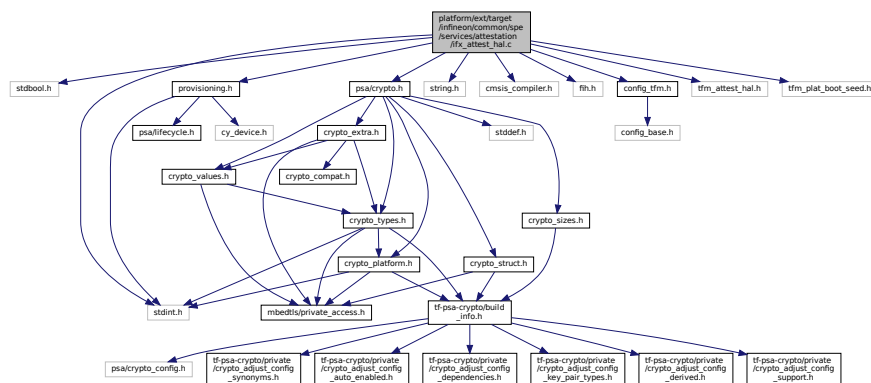
## 8.191 platform/ext/target/infineon/common/spe/services/attestation/ifx\_attest\_hal.c File Reference

```

#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include "cmsis_compiler.h"
#include "fih.h"
#include "config_tfm.h"
#include "provisioning.h"
#include "tfm_attest_hal.h"
#include "tfm_plat_boot_seed.h"
#include "psa/crypto.h"

```

Include dependency graph for ifx\_attest\_hal.c:



## Macros

- #define `IFX_ATTESTATION_KEY_ID` ((psa\_key\_id\_t)(PSA\_KEY\_ID\_VENDOR\_MIN + 1U))
- #define `IFX_ATTESTATION_PROFILE_DEFINITION` "tag:psacertified.org,2023:psa#tfm"
- #define `IFX_ATTESTATION_VERIFICATION_SERVICE` "www.trustedfirmware.org"
- #define `BOOL_SEED_INITIALIZED` 0x3Bu
- #define `BOOL_SEED_UNINITIALIZED` 0xC4u

## Functions

- enum tfm\_plat\_err\_t `tfm_plat_get_boot_seed` (uint32\_t \*size, uint8\_t \*buf)
- enum tfm\_security\_lifecycle\_t `tfm_attest_hal_get_security_lifecycle` (void)
- enum tfm\_plat\_err\_t `tfm_attest_hal_get_verification_service` (uint32\_t \*size, uint8\_t \*buf)
- enum tfm\_plat\_err\_t `tfm_attest_hal_get_profile_definition` (uint32\_t \*size, uint8\_t \*buf)

## 8.191.1 Macro Definition Documentation

### 8.191.1.1 `BOOL_SEED_INITIALIZED`

```
#define BOOL_SEED_INITIALIZED 0x3Bu
```

Definition at line 37 of file `ifx_attest_hal.c`.

### 8.191.1.2 `BOOL_SEED_UNINITIALIZED`

```
#define BOOL_SEED_UNINITIALIZED 0xC4u
```

Definition at line 38 of file `ifx_attest_hal.c`.

### 8.191.1.3 `IFX_ATTESTATION_KEY_ID`

```
#define IFX_ATTESTATION_KEY_ID ((psa_key_id_t) (PSA_KEY_ID_VENDOR_MIN + 1U))
```

Definition at line 21 of file `ifx_attest_hal.c`.

### 8.191.1.4 `IFX_ATTESTATION_PROFILE_DEFINITION`

```
#define IFX_ATTESTATION_PROFILE_DEFINITION "tag:psacertified.org,2023:psa#tfm"
```

Default value of profile definition for Initial Attestation service.  
Definition at line 27 of file `ifx_attest_hal.c`.

### 8.191.1.5 `IFX_ATTESTATION_VERIFICATION_SERVICE`

```
#define IFX_ATTESTATION_VERIFICATION_SERVICE "www.trustedfirmware.org"
```

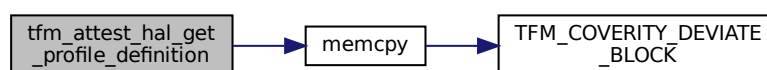
Default value of verification service for Initial Attestation.  
Definition at line 34 of file `ifx_attest_hal.c`.

## 8.191.2 Function Documentation

### 8.191.2.1 `tfm_attest_hal_get_profile_definition()`

```
enum tfm_plat_err_t tfm_attest_hal_get_profile_definition (
 uint32_t * size,
 uint8_t * buf)
```

Definition at line 137 of file `ifx_attest_hal.c`.  
Here is the call graph for this function:

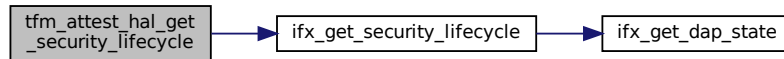


**8.191.2.2 tfm\_attest\_hal\_get\_security\_lifecycle()**

```
enum tfm_security_lifecycle_t tfm_attest_hal_get_security_lifecycle (
 void)
```

Definition at line 86 of file ifx\_attest\_hal.c.

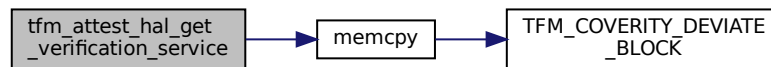
Here is the call graph for this function:

**8.191.2.3 tfm\_attest\_hal\_get\_verification\_service()**

```
enum tfm_plat_err_t tfm_attest_hal_get_verification_service (
 uint32_t * size,
 uint8_t * buf)
```

Definition at line 117 of file ifx\_attest\_hal.c.

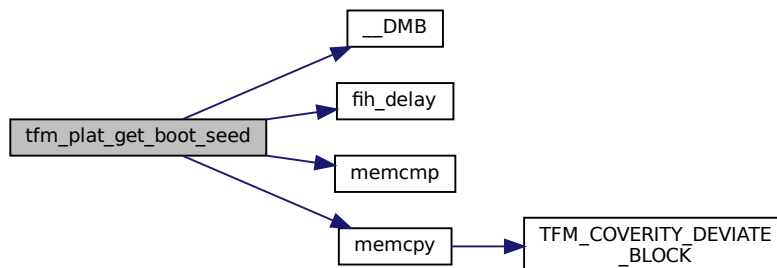
Here is the call graph for this function:

**8.191.2.4 tfm\_plat\_get\_boot\_seed()**

```
enum tfm_plat_err_t tfm_plat_get_boot_seed (
 uint32_t size,
 uint8_t * buf)
```

Definition at line 40 of file ifx\_attest\_hal.c.

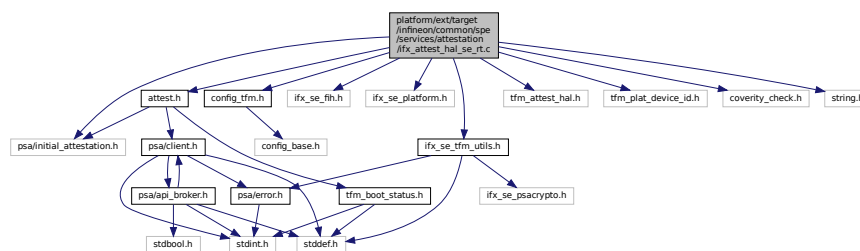
Here is the call graph for this function:



## 8.192 platform/ext/target/infineon/common/spe/services/attestation/ifx↵ \_attest\_hal\_se\_rt.c File Reference

```
#include "attest.h"
#include "config_tfm.h"
#include "ifx_se_fih.h"
#include "ifx_se_platform.h"
#include "ifx_se_tfm_utils.h"
#include "psa/initial_attestation.h"
#include "tfm_attest_hal.h"
#include "tfm_plat_device_id.h"
#include "coverity_check.h"
#include <string.h>
```

Include dependency graph for ifx\_attest\_hal\_se\_rt.c:



### Functions

- [psa\\_status\\_t tfm\\_attest\\_hal\\_get\\_token](#) (const [void](#) \*challenge\_buf, [size\\_t](#) challenge\_size, [void](#) \*token\_buf, [size\\_t](#) token\_buf\_size, [size\\_t](#) \*token\_size)
- [psa\\_status\\_t tfm\\_attest\\_hal\\_get\\_token\\_size](#) ([size\\_t](#) challenge\_size, [size\\_t](#) \*token\_size)

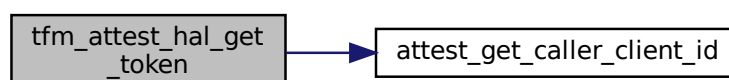
### 8.192.1 Function Documentation

#### 8.192.1.1 tfm\_attest\_hal\_get\_token()

```
psa_status_t tfm_attest_hal_get_token (
 const void * challenge_buf,
 size_t challenge_size,
 void * token_buf,
 size_t token_buf_size,
 size_t * token_size)
```

Definition at line 48 of file ifx\_attest\_hal\_se\_rt.c.

Here is the call graph for this function:



## 8.192.1.2 tfm\_attest\_hal\_get\_token\_size()

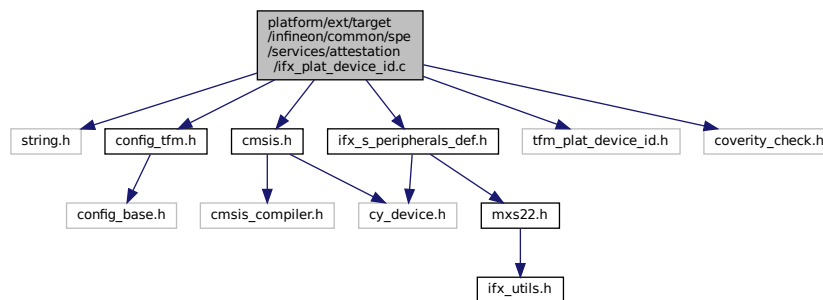
```
psa_status_t tfm_attest_hal_get_token_size (
 size_t challenge_size,
 size_t * token_size)
```

Definition at line 97 of file ifx\_attest\_hal\_se\_rt.c.

## 8.193 platform/ext/target/infineon/common/spe/services/attestation/ifx\_plat\_device\_id.c File Reference

```
#include <string.h>
#include "config_tfm.h"
#include "cmsis.h"
#include "ifx_s_peripherals_def.h"
#include "tfm_plat_device_id.h"
#include "coverity_check.h"
```

Include dependency graph for ifx\_plat\_device\_id.c:



### Macros

- #define `IFX_ADD_TO_IMPLEMENTATION_ID`(buffer, val)
- #define `IFX_ATTESTATION_HW_VERSION` "0123456789012-12345"

### Functions

- enum tfm\_plat\_err\_t `tfm_plat_get_implementation_id` (uint32\_t \*size, uint8\_t \*buf)
- enum tfm\_plat\_err\_t `tfm_plat_get_cert_ref` (uint32\_t \*size, uint8\_t \*buf)

## 8.193.1 Macro Definition Documentation

### 8.193.1.1 IFX\_ADD\_TO\_IMPLEMENTATION\_ID

```
#define IFX_ADD_TO_IMPLEMENTATION_ID(
 buffer,
 val)
```

Value:

```
(void)memcpy((buffer), \
 (const void*)(val)), \
 sizeof((val))); \
(buffer) += sizeof((val));
```

Definition at line 19 of file ifx\_plat\_device\_id.c.

### 8.193.1.2 IFX\_ATTESTATION\_HW\_VERSION

```
#define IFX_ATTESTATION_HW_VERSION "0123456789012-12345"
```

Default value of H/W version for Initial Attestation service.

Definition at line 28 of file ifx\_plat\_device\_id.c.

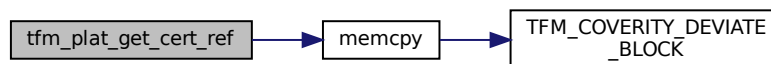
## 8.193.2 Function Documentation

### 8.193.2.1 tfm\_plat\_get\_cert\_ref()

```
enum tfm_plat_err_t tfm_plat_get_cert_ref (
 uint32_t * size,
 uint8_t * buf)
```

Definition at line 125 of file ifx\_plat\_device\_id.c.

Here is the call graph for this function:

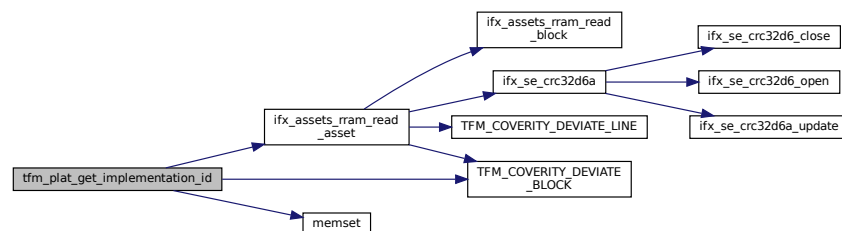


### 8.193.2.2 tfm\_plat\_get\_implementation\_id()

```
enum tfm_plat_err_t tfm_plat_get_implementation_id (
 uint32_t * size,
 uint8_t * buf)
```

Definition at line 31 of file ifx\_plat\_device\_id.c.

Here is the call graph for this function:

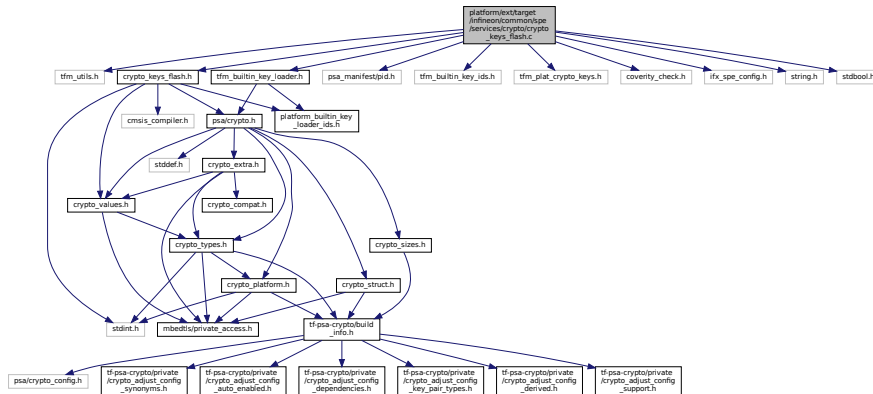


## 8.194 platform/ext/target/infineon/common/spe/services/crypto/crypto\_keys\_flash.c File Reference

```
#include "tfm_utils.h"
#include "crypto_keys_flash.h"
#include "psa_manifest/pid.h"
```

```
#include "tfm_builtin_key_ids.h"
#include "tfm_builtin_key_loader.h"
#include "tfm_plat_crypto_keys.h"
#include "coverity_check.h"
#include "ifx_spe_config.h"
#include <string.h>
#include <stdbool.h>
```

Include dependency graph for crypto\_keys\_flash.c:



## Functions

- `size_t tfm_plat_builtin_key_get_policy_table_ptr` (const tfm\_plat\_builtin\_key\_policy\_t \*desc\_ptr[])
- `size_t tfm_plat_builtin_key_get_desc_table_ptr` (const tfm\_plat\_builtin\_key\_descriptor\_t \*desc\_ptr[])

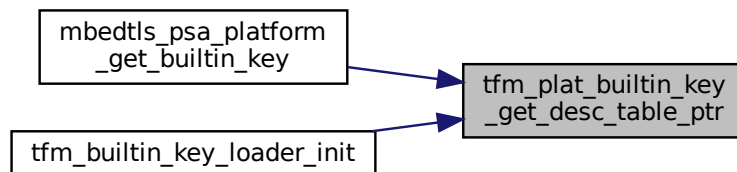
### 8.194.1 Function Documentation

#### 8.194.1.1 tfm\_plat\_builtin\_key\_get\_desc\_table\_ptr()

```
size_t tfm_plat_builtin_key_get_desc_table_ptr (
 const tfm_plat_builtin_key_descriptor_t * desc_ptr[])
```

Definition at line 183 of file crypto\_keys\_flash.c.

Here is the caller graph for this function:



### 8.194.1.2 tfm\_plat\_builtin\_key\_get\_policy\_table\_ptr()

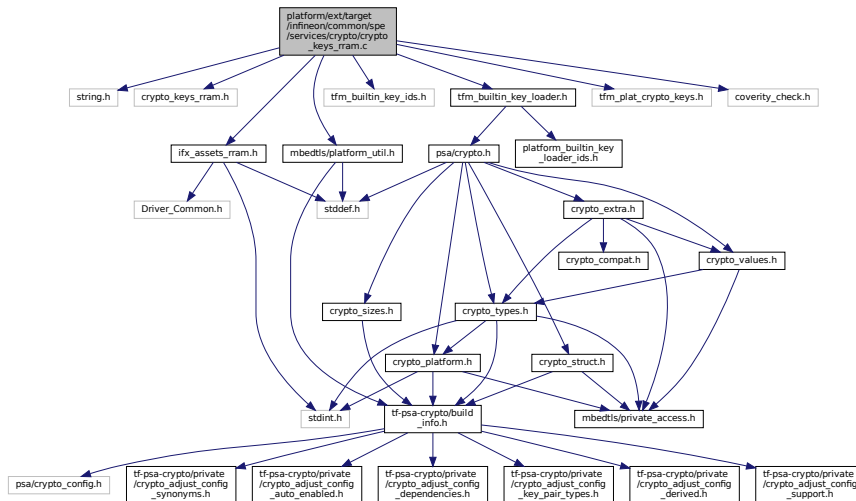
```
size_t tfm_plat_builtin_key_get_policy_table_ptr (
 const tfm_plat_builtin_key_policy_t * desc_ptr[])
```

Definition at line 177 of file crypto\_keys\_flash.c.

## 8.195 platform/ext/target/infineon/common/spe/services/crypto/crypto\_keys\_rram.c File Reference

```
#include <string.h>
#include "crypto_keys_rram.h"
#include "ifx_assets_rram.h"
#include "mbedtls/platform_util.h"
#include "tfm_builtin_key_ids.h"
#include "tfm_builtin_key_loader.h"
#include "tfm_plat_crypto_keys.h"
#include "coverity_check.h"
```

Include dependency graph for crypto\_keys\_rram.c:



## Macros

- #define [NUMBER\\_OF\\_ELEMENTS\\_OF\(x\)](#) sizeof(x)/sizeof(\*x)
- #define [MAPPED\\_TZ\\_NS\\_AGENT\\_DEFAULT\\_CLIENT\\_ID](#) 0xaaaa7fffU
- #define [MAPPED\\_MAILBOX\\_NS\\_AGENT\\_DEFAULT\\_CLIENT\\_ID](#) 0xaaaaffffU
- #define [TFM\\_TZ\\_NS\\_PARTITION\\_ID](#) MAPPED\_TZ\_NS\_AGENT\_DEFAULT\_CLIENT\_ID
- #define [TFM\\_MBOX\\_NS\\_PARTITION\\_ID](#) MAPPED\_MAILBOX\_NS\_AGENT\_DEFAULT\_CLIENT\_ID

## Functions

- size\_t [tfm\\_plat\\_builtin\\_key\\_get\\_policy\\_table\\_ptr](#) (const tfm\_plat\_builtin\_key\_policy\_t \*desc\_ptr[])
- size\_t [tfm\\_plat\\_builtin\\_key\\_get\\_desc\\_table\\_ptr](#) (const tfm\_plat\_builtin\_key\_descriptor\_t \*desc\_ptr[])

### 8.195.1 Macro Definition Documentation



**8.195.1.1 MAPPED\_MAILBOX\_NS\_AGENT\_DEFAULT\_CLIENT\_ID**

```
#define MAPPED_MAILBOX_NS_AGENT_DEFAULT_CLIENT_ID 0xaaaaffffU
```

Definition at line 37 of file crypto\_keys\_rram.c.

**8.195.1.2 MAPPED\_TZ\_NS\_AGENT\_DEFAULT\_CLIENT\_ID**

```
#define MAPPED_TZ_NS_AGENT_DEFAULT_CLIENT_ID 0xaaa7fffU
```

Definition at line 36 of file crypto\_keys\_rram.c.

**8.195.1.3 NUMBER\_OF\_ELEMENTS\_OF**

```
#define NUMBER_OF_ELEMENTS_OF(
 x) sizeof(x)/sizeof(*x)
```

Definition at line 34 of file crypto\_keys\_rram.c.

**8.195.1.4 TFM\_MBOX\_NS\_PARTITION\_ID**

```
#define TFM_MBOX_NS_PARTITION_ID MAPPED_MAILBOX_NS_AGENT_DEFAULT_CLIENT_ID
```

Definition at line 39 of file crypto\_keys\_rram.c.

**8.195.1.5 TFM\_TZ\_NS\_PARTITION\_ID**

```
#define TFM_TZ_NS_PARTITION_ID MAPPED_TZ_NS_AGENT_DEFAULT_CLIENT_ID
```

Definition at line 38 of file crypto\_keys\_rram.c.

**8.195.2 Function Documentation****8.195.2.1 tfm\_plat\_builtin\_key\_get\_desc\_table\_ptr()**

```
size_t tfm_plat_builtin_key_get_desc_table_ptr (
 const tfm_plat_builtin_key_descriptor_t * desc_ptr[])
```

Definition at line 215 of file crypto\_keys\_rram.c.

**8.195.2.2 tfm\_plat\_builtin\_key\_get\_policy\_table\_ptr()**

```
size_t tfm_plat_builtin_key_get_policy_table_ptr (
 const tfm_plat_builtin_key_policy_t * desc_ptr[])
```

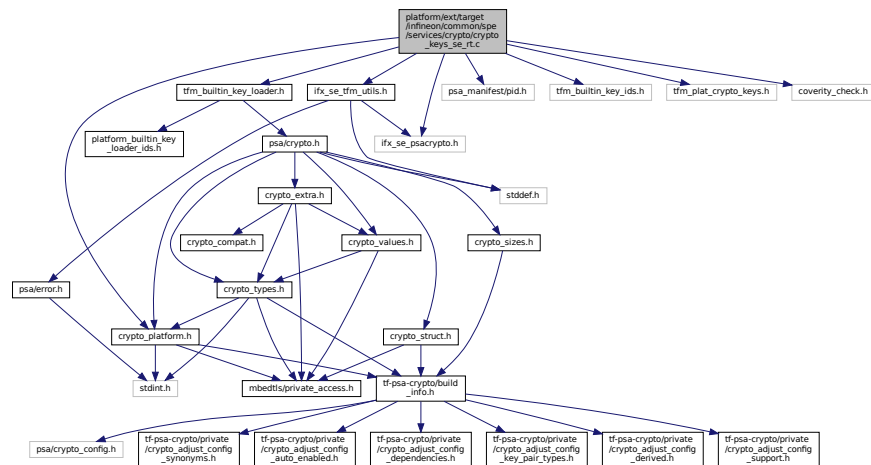
Definition at line 209 of file crypto\_keys\_rram.c.

**8.196 platform/ext/target/infineon/common/spe/services/crypto/crypto\_keys\_se\_rt.c File Reference ↩**

```
#include "crypto_platform.h"
#include "ifx_se_psacrypto.h"
#include "ifx_se_tfm_utils.h"
#include "psa_manifest/pid.h"
#include "tfm_builtin_key_ids.h"
#include "tfm_builtin_key_loader.h"
#include "tfm_plat_crypto_keys.h"
```

```
#include "coverity_check.h"
```

Include dependency graph for `crypto_keys_se_rt.c`:



## Macros

- `#define` [NUMBER\\_OF\\_ELEMENTS\\_OF\(x\)](#) `sizeof(x)/sizeof(*x)`

## Functions

- `size_t` [tfm\\_plat\\_built\\_in\\_key\\_get\\_desc\\_table\\_ptr](#) (`const tfm_plat_built_in_key_descriptor_t *desc_ptr[]`)

## 8.196.1 Macro Definition Documentation

### 8.196.1.1 NUMBER\_OF\_ELEMENTS\_OF

```
#define NUMBER_OF_ELEMENTS_OF(
 x) sizeof(x)/sizeof(*x)
```

Definition at line 22 of file `crypto_keys_se_rt.c`.

## 8.196.2 Function Documentation

### 8.196.2.1 tfm\_plat\_built\_in\_key\_get\_desc\_table\_ptr()

```
size_t tfm_plat_built_in_key_get_desc_table_ptr (
 const tfm_plat_built_in_key_descriptor_t * desc_ptr[])
```

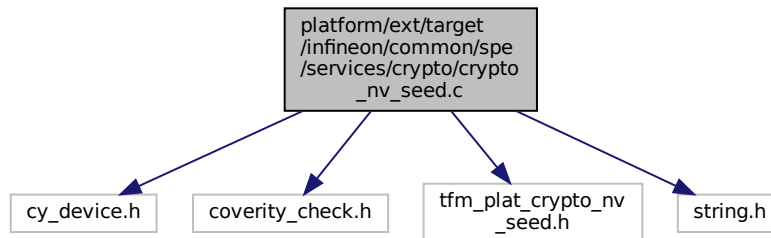
Definition at line 317 of file `crypto_keys_se_rt.c`.

## 8.197 platform/ext/target/infineon/common/spe/services/crypto/crypto\_nv\_seed.c File Reference

```
#include "cy_device.h"
#include "coverity_check.h"
#include "tfm_plat_crypto_nv_seed.h"
```

```
#include <string.h>
```

Include dependency graph for crypto\_nv\_seed.c:



## Functions

- int `tfm_plat_crypto_provision_entropy_seed` (void)
- int `tfm_plat_crypto_nv_seed_write` (const unsigned char \*buf, size\_t buf\_len)

### 8.197.1 Function Documentation

#### 8.197.1.1 tfm\_plat\_crypto\_nv\_seed\_write()

```
int tfm_plat_crypto_nv_seed_write (
 const unsigned char * buf,
 size_t buf_len)
```

Definition at line 21 of file `crypto_nv_seed.c`.

#### 8.197.1.2 tfm\_plat\_crypto\_provision\_entropy\_seed()

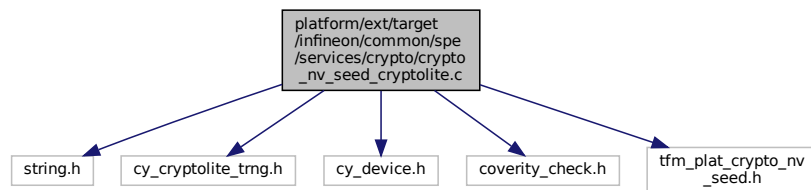
```
int tfm_plat_crypto_provision_entropy_seed (
 void)
```

Definition at line 15 of file `crypto_nv_seed.c`.

## 8.198 platform/ext/target/infineon/common/spe/services/crypto/crypto\_nv\_seed\_cryptolite.c File Reference

```
#include <string.h>
#include "cy_cryptolite_trng.h"
#include "cy_device.h"
#include "coverity_check.h"
#include "tfm_plat_crypto_nv_seed.h"
```

Include dependency graph for `crypto_nv_seed_cryptolite.c`:



## Functions

- int `tfm_plat_crypto_nv_seed_read` (unsigned char \*buf, size\_t buf\_len)

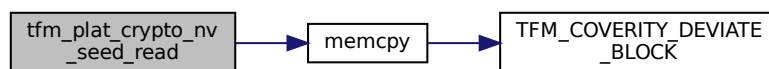
### 8.198.1 Function Documentation

#### 8.198.1.1 `tfm_plat_crypto_nv_seed_read()`

```
int tfm_plat_crypto_nv_seed_read (
 unsigned char * buf,
 size_t buf_len)
```

Definition at line 17 of file `crypto_nv_seed_cryptolite.c`.

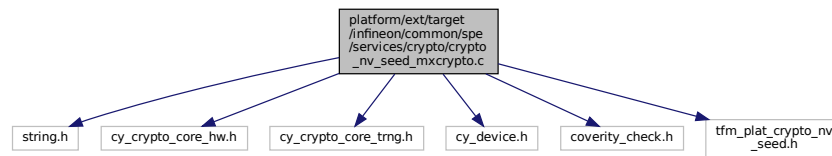
Here is the call graph for this function:



## 8.199 platform/ext/target/infineon/common/spe/services/crypto/crypto\_nv\_seed\_mxcrypto.c File Reference

```
#include <string.h>
#include "cy_crypto_core_hw.h"
#include "cy_crypto_core_trng.h"
#include "cy_device.h"
#include "coverity_check.h"
#include "tfm_plat_crypto_nv_seed.h"
```

Include dependency graph for crypto\_nv\_seed\_mxcrypto.c:



## Functions

- int [tfm\\_plat\\_crypto\\_nv\\_seed\\_read](#) (unsigned char \*buf, size\_t buf\_len)

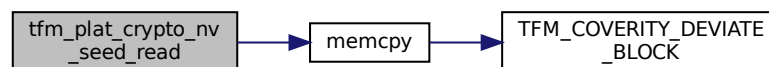
### 8.199.1 Function Documentation

#### 8.199.1.1 tfm\_plat\_crypto\_nv\_seed\_read()

```
int tfm_plat_crypto_nv_seed_read (
 unsigned char * buf,
 size_t buf_len)
```

Definition at line 18 of file crypto\_nv\_seed\_mxcrypto.c.

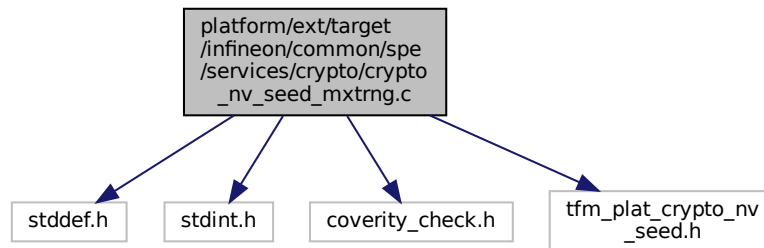
Here is the call graph for this function:



## 8.200 platform/ext/target/infineon/common/spe/services/crypto/crypto\_nv\_seed\_mxtrng.c File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "coverity_check.h"
#include "tfm_plat_crypto_nv_seed.h"
```

Include dependency graph for `crypto_nv_seed_mxtrng.c`:



## Functions

- int [tfm\\_plat\\_crypto\\_nv\\_seed\\_read](#) (unsigned char \*buf, size\_t buf\_len)

### 8.200.1 Function Documentation

#### 8.200.1.1 tfm\_plat\_crypto\_nv\_seed\_read()

```
int tfm_plat_crypto_nv_seed_read (
 unsigned char * buf,
 size_t buf_len)
```

Definition at line 17 of file `crypto_nv_seed_mxtrng.c`.

### 8.201 platform/ext/target/infineon/common/spe/services/crypto/crypto\_ rnd\_se\_rt.c File Reference

```
#include "config_tfm.h"
#include "crypto_platform.h"
#include "ifx_se_psacrypto.h"
#include "ifx_se_tfm_utils.h"
#include "ifx_utils.h"
#include "psa/crypto.h"
#include "mbedtls/platform.h"
#include "mbedtls/platform_util.h"
#include <string.h>
```

[illegible]

## 8.204 platform/ext/target/infineon/common/spe/services/its/drivers/its\_← flash\_driver.c File Reference

```

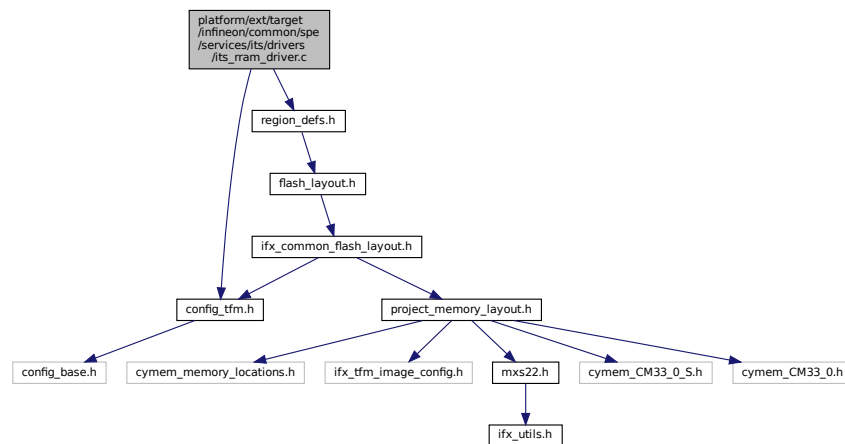
graph TD
 Root["platform/ext/target
/infineon/common/spe
/services/its/drivers
/its_flash_driver.c"]
 region_defs_h["region_defs.h"]
 flash_layout_h["flash_layout.h"]
 ifx_common_flash_layout_h["ifx_common_flash_layout.h"]
 config_tfm_h["config_tfm.h"]
 project_memory_layout_h["project_memory_layout.h"]
 config_base_h["config_base.h"]
 cymem_memory_locations_h["cymem_memory_locations.h"]
 ifx_tfm_image_config_h["ifx_tfm_image_config.h"]
 mxs22_h["mxs22.h"]
 cymem_CM33_0_S_h["cymem_CM33_0_S.h"]
 cymem_CM33_0_h["cymem_CM33_0.h"]
 ifx_utils_h["ifx_utils.h"]

 Root --> region_defs_h
 Root --> flash_layout_h
 Root --> ifx_common_flash_layout_h
 Root --> config_tfm_h
 region_defs_h --> flash_layout_h
 flash_layout_h --> ifx_common_flash_layout_h
 ifx_common_flash_layout_h --> project_memory_layout_h
 config_tfm_h --> project_memory_layout_h
 project_memory_layout_h --> config_base_h
 project_memory_layout_h --> cymem_memory_locations_h
 project_memory_layout_h --> ifx_tfm_image_config_h
 project_memory_layout_h --> mxs22_h
 project_memory_layout_h --> cymem_CM33_0_S_h
 project_memory_layout_h --> cymem_CM33_0_h
 mxs22_h --> ifx_utils_h

```

```
#include "config_tfm.h"
#include "region_defs.h"
```

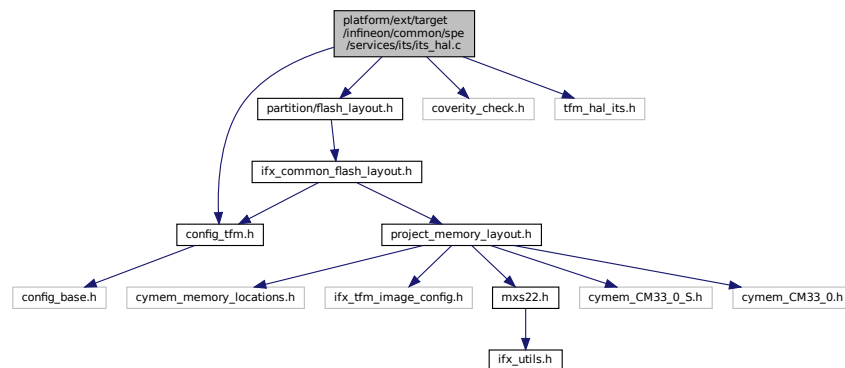
Include dependency graph for `its_ram_driver.c`:



## 8.206 platform/ext/target/infineon/common/spe/services/its/its\_hal.c File Reference

```
#include "config_tfm.h"
#include "partition/flash_layout.h"
#include "coverity_check.h"
#include "tfm_hal_its.h"
```

Include dependency graph for `its_hal.c`:



## Functions

- enum `tfm_hal_status_t` [tfm\\_hal\\_its\\_fs\\_info](#) (struct `tfm_hal_its_fs_info_t` \*`fs_info`)

### 8.206.1 Function Documentation

#### 8.206.1.1 tfm\_hal\_its\_fs\_info()

```
enum tfm_hal_status_t tfm_hal_its_fs_info (
 struct tfm_hal_its_fs_info_t * fs_info)
```

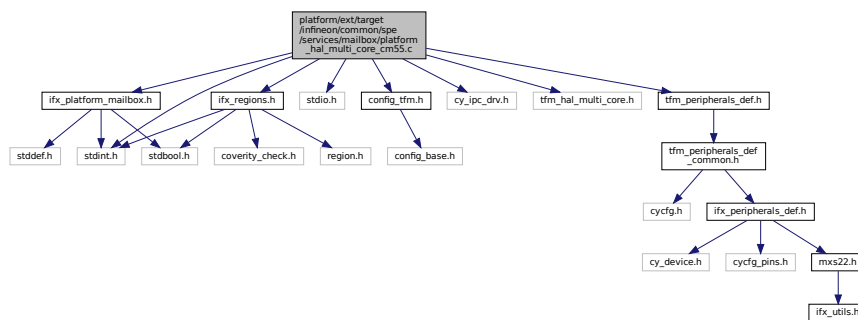


Definition at line 15 of file its\_hal.c.

## 8.207 platform/ext/target/infineon/common/spe/services/mailbox/platform\_hal\_multi\_core\_cm55.c File Reference

```
#include <stdint.h>
#include <stdio.h>
#include "config_tfm.h"
#include "cy_ipc_drv.h"
#include "ifx_platform_mailbox.h"
#include "ifx_regions.h"
#include "tfm_hal_multi_core.h"
#include "tfm_peripherals_def.h"
```

Include dependency graph for platform\_hal\_multi\_core\_cm55.c:



### Macros

- `#define IFX_IS_REGION_IN_ITCM_MEMORY(base, size)`
- `#define IFX_IS_REGION_IN_DTCM_MEMORY(base, size)`
- `#define IFX_IS_REGION_IN_ITCM_MEMORY_REMAPPED(base, size)`
- `#define IFX_IS_REGION_IN_DTCM_MEMORY_REMAPPED(base, size)`

### 8.207.1 Macro Definition Documentation

#### 8.207.1.1 IFX\_IS\_REGION\_IN\_DTCM\_MEMORY

```
#define IFX_IS_REGION_IN_DTCM_MEMORY(
 base,
 size)
```

Value:

```
(ifx_is_region_inside_other(base, base + size, \
 CY_CM55_DTCM_INTERNAL_NS_SBUS_BASE, \
 (CY_CM55_DTCM_INTERNAL_NS_SBUS_BASE \
 + CY_CM55_DTCM_INTERNAL_SIZE)) == true)
```

Definition at line 31 of file platform\_hal\_multi\_core\_cm55.c.

#### 8.207.1.2 IFX\_IS\_REGION\_IN\_DTCM\_MEMORY\_REMAPPED

```
#define IFX_IS_REGION_IN_DTCM_MEMORY_REMAPPED(
 base,
 size)
```

**Value:**

```
(ifx_is_region_inside_other(base, base + size, \
 CY_CM55_DTCM_NS_SBUS_BASE, \
 (CY_CM55_DTCM_NS_SBUS_BASE \
 + CY_CM55_DTCM_INTERNAL_SIZE)) == true)
```

Definition at line 43 of file platform\_hal\_multi\_core\_cm55.c.

### 8.207.1.3 IFX\_IS\_REGION\_IN\_ITCM\_MEMORY

```
#define IFX_IS_REGION_IN_ITCM_MEMORY(
 base,
 size)
```

**Value:**

```
(ifx_is_region_inside_other(base, base + size, \
 CY_CM55_ITCM_INTERNAL_NS_SBUS_BASE, \
 (CY_CM55_ITCM_INTERNAL_NS_SBUS_BASE \
 + CY_CM55_ITCM_INTERNAL_SIZE)) == true)
```

Definition at line 25 of file platform\_hal\_multi\_core\_cm55.c.

### 8.207.1.4 IFX\_IS\_REGION\_IN\_ITCM\_MEMORY\_REMAPPED

```
#define IFX_IS_REGION_IN_ITCM_MEMORY_REMAPPED(
 base,
 size)
```

**Value:**

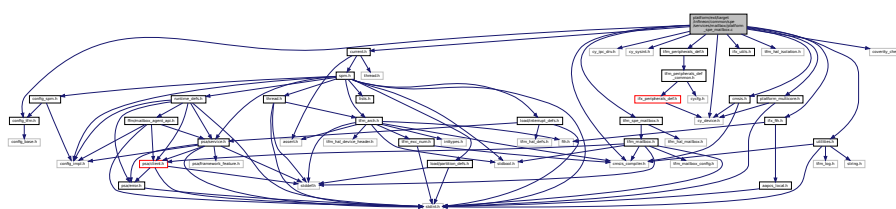
```
(ifx_is_region_inside_other(base, base + size, \
 CY_CM55_ITCM_NS_SBUS_BASE, \
 (CY_CM55_ITCM_NS_SBUS_BASE \
 + CY_CM55_ITCM_INTERNAL_SIZE)) == true)
```

Definition at line 37 of file platform\_hal\_multi\_core\_cm55.c.

## 8.208 platform/ext/target/infineon/common/spe/services/mailbox/platform\_spe\_mailbox.c File Reference

```
#include "config_tfm.h"
#include "cmsis.h"
#include "cmsis_compiler.h"
#include "cy_device.h"
#include "cy_ipc_drv.h"
#include "cy_sysint.h"
#include "current.h"
#include "ifx_fih.h"
#include "ifx_utils.h"
#include "tfm_hal_isolation.h"
#include "tfm_peripherals_def.h"
#include "tfm_spe_mailbox.h"
#include "platform_multicore.h"
#include "utilities.h"
#include "coverity_check.h"
```

Include dependency graph for platform\_spe\_mailbox.c:



## Macros

- #define `MAILBOX_CLEAN_CACHE(addr, size)` { `__DSB()`; }
- #define `MAILBOX_INVALIDATE_CACHE(addr, size)` do {} `while` (0)

## Functions

- `int32_t tfm_mailbox_hal_notify_peer` (void)
- `int32_t tfm_mailbox_hal_init` (struct secure\_mailbox\_queue\_t \*s\_queue)
- `int32_t tfm_mailbox_hal_deinit` (struct secure\_mailbox\_queue\_t \*s\_queue)
- `uint32_t tfm_mailbox_hal_enter_critical` (void)
- `void tfm_mailbox_hal_exit_critical` (uint32\_t state)

## 8.208.1 Macro Definition Documentation

### 8.208.1.1 MAILBOX\_CLEAN\_CACHE

```
#define MAILBOX_CLEAN_CACHE (
 addr,
 size) { __DSB(); }
```

Definition at line 31 of file platform\_spe\_mailbox.c.

### 8.208.1.2 MAILBOX\_INVALIDATE\_CACHE

```
#define MAILBOX_INVALIDATE_CACHE (
 addr,
 size) do {} while (0)
```

Definition at line 32 of file platform\_spe\_mailbox.c.

## 8.208.2 Function Documentation

### 8.208.2.1 tfm\_mailbox\_hal\_deinit()

```
int32_t tfm_mailbox_hal_deinit (
 struct secure_mailbox_queue_t * s_queue)
```

Definition at line 139 of file platform\_spe\_mailbox.c.

### 8.208.2.2 tfm\_mailbox\_hal\_enter\_critical()

```
uint32_t tfm_mailbox_hal_enter_critical (
 void)
```

Definition at line 150 of file platform\_spe\_mailbox.c.

### 8.208.2.3 tfm\_mailbox\_hal\_exit\_critical()

```
void tfm_mailbox_hal_exit_critical (
 uint32_t state)
```

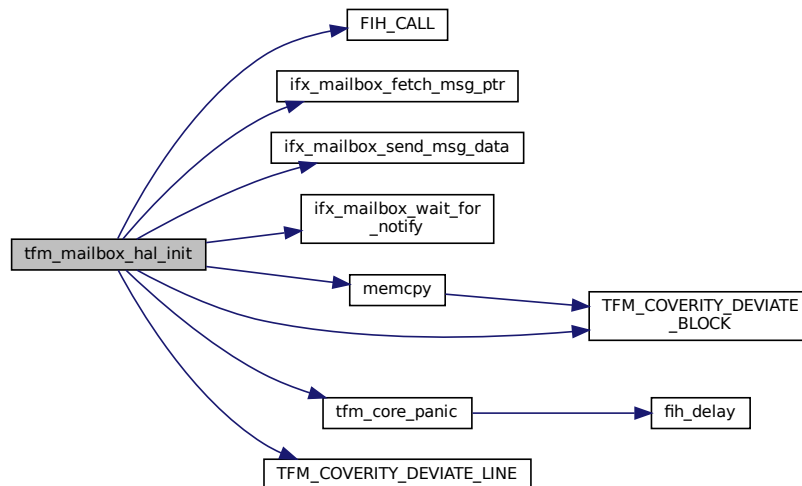
Definition at line 175 of file platform\_spe\_mailbox.c.

### 8.208.2.4 tfm\_mailbox\_hal\_init()

```
int32_t tfm_mailbox_hal_init (
 struct secure_mailbox_queue_t * s_queue)
```

Definition at line 66 of file platform\_spe\_mailbox.c.

Here is the call graph for this function:



### 8.208.2.5 tfm\_mailbox\_hal\_notify\_peer()

```
int32_t tfm_mailbox_hal_notify_peer (
 void)
```

Definition at line 51 of file platform\_spe\_mailbox.c.

## 8.209 platform/ext/target/infineon/common/spe/services/mailbox/tfm\_hal\_multi\_core.c File Reference

```
#include "config_tfm.h"
#include "cmsis.h"
#include "cy_ipc_drv.h"
#include "ifx_regions.h"
#include "platform_multicore.h"
#include "tfm_hal_multi_core.h"
#include "tfm_peripherals_def.h"
#include "tfm_log_unpriv.h"
```

```

graph TD
 Root["platform/xtarget/infineon/common/spe/services/mailbox/tfm_hal_multi_core.c"]
 Root --> config_tfm_h["config_tfm.h"]
 Root --> cmsis_h["cmsis.h"]
 Root --> cy_ipc_drv_h["cy_ipc_drv.h"]
 Root --> ifx_regions_h["ifx_regions.h"]
 Root --> platform_multicore_h["platform_multicore.h"]
 Root --> tfm_hal_multi_core_h["tfm_hal_multi_core.h"]
 Root --> tfm_peripherals_def_h["tfm_peripherals_def.h"]
 Root --> tfm_log_unpriv_h["tfm_log_unpriv.h"]

 config_tfm_h --> config_base_h["config_base.h"]

 cmsis_h --> cmsis_compiler_h["cmsis_compiler.h"]

 ifx_regions_h --> stdbool_h["stdbool.h"]
 ifx_regions_h --> region_h["region.h"]
 ifx_regions_h --> coventry_check_h["coventry_check.h"]
 ifx_regions_h --> stdint_h["stdint.h"]

 platform_multicore_h --> stdint_h

 tfm_peripherals_def_h --> tfm_peripherals_def_common_h["tfm_peripherals_def_common.h"]

 tfm_peripherals_def_common_h --> ifx_peripherals_def_h["ifx_peripherals_def.h"]

 ifx_peripherals_def_h --> cycfg_h["cycfg.h"]
 ifx_peripherals_def_h --> cycfg_pins_h["cycfg_pins.h"]
 ifx_peripherals_def_h --> mxs22_h["mxs22.h"]

 mxs22_h --> ifx_utils_h["ifx_utils.h"]

 cy_device_h["cy_device.h"] --> cy_device_h
 cy_device_h --> ifx_peripherals_def_h

```

- `void tfm_hal_wait_for_ns_cpu_ready (void)`

### 8.209.1.1 tfm\_hal\_wait\_for\_ns\_cpu\_ready()

Definition at line 19 of file tfm\_hal\_multi\_core.c.

```
#include <stdint.h>
#include "config_tfm.h"
#include "cmsis.h"
#include "cy_ipc_drv.h"
#include "ifx_interrupt_defs.h"
#include "interrupt.h"
#include "platform_multicore.h"
#include "spm.h"
#include "tfm_multi_core.h"
#include "tfm_peripherals_def.h"
#include "load/interrupt_defs.h"
#include "coverity_check.h"
```

The diagram is a dense network of nodes and directed edges. Nodes are represented by rectangular boxes containing text. Many nodes include a name followed by a date and location, such as "LAWSON, JAMES L. 07-1980 NEW YORK". Other nodes contain names alone, like "MARTIN, JOHN W.". Some nodes have additional labels or codes, such as "SMITH, ROBERT E. 10-1980 NEW YORK" and "SMITH, ROBERT E. 10-1980 NEW YORK". The edges are thin black lines with arrowheads indicating direction. There are numerous bidirectional connections between many pairs of nodes, creating a highly interconnected web. The layout is somewhat circular, with nodes arranged around a central area where many connections converge.

## Functions

- `void IFX_IRQ_NAME_TO_HANDLER() IFX_IPC_NS_TO_TFM_IPC_INTR (void)`
- `enum tfm_hal_status_t mailbox_irq_init (void *p_pt, const struct irq_load_info_t *p_ildi)`

### 8.210.1 Function Documentation

#### 8.210.1.1 IFX\_IPC\_NS\_TO\_TFM\_IPC\_INTR()

```
void IFX_IRQ_NAME_TO_HANDLER() IFX_IPC_NS_TO_TFM_IPC_INTR (
 void)
```

Definition at line 41 of file `tfm_interrupts.c`.

#### 8.210.1.2 mailbox\_irq\_init()

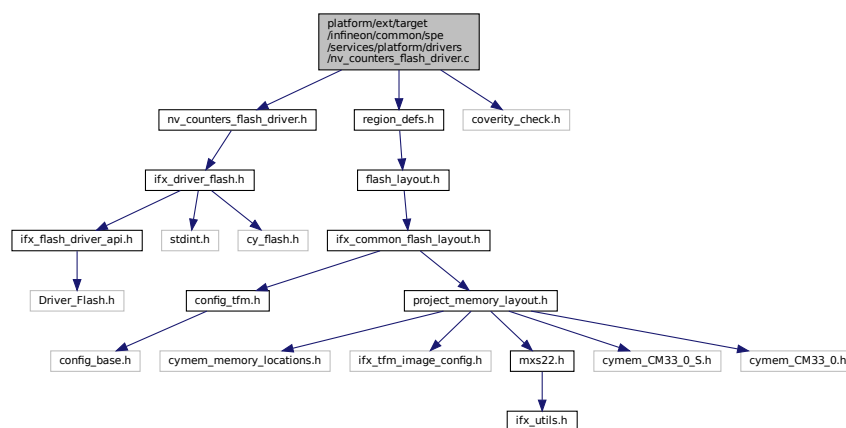
```
enum tfm_hal_status_t mailbox_irq_init (
 void * p_pt,
 const struct irq_load_info_t * p_ildi)
```

Definition at line 54 of file `tfm_interrupts.c`.

## 8.211 platform/ext/target/infineon/common/spe/services/platform/drivers/nv\_counters\_flash\_driver.c File Reference

```
#include "nv_counters_flash_driver.h"
#include "region_defs.h"
#include "coverity_check.h"
```

Include dependency graph for `nv_counters_flash_driver.c`:



## Macros

- `#define IFX_TFM_NV_COUNTERS_ERASE_VALUE IFX_DRIVER_FLASH_ERASE_VALUE`

### 8.211.1 Macro Definition Documentation

### 8.211.1.1 IFX\_TFM\_NV\_COUNTERS\_ERASE\_VALUE

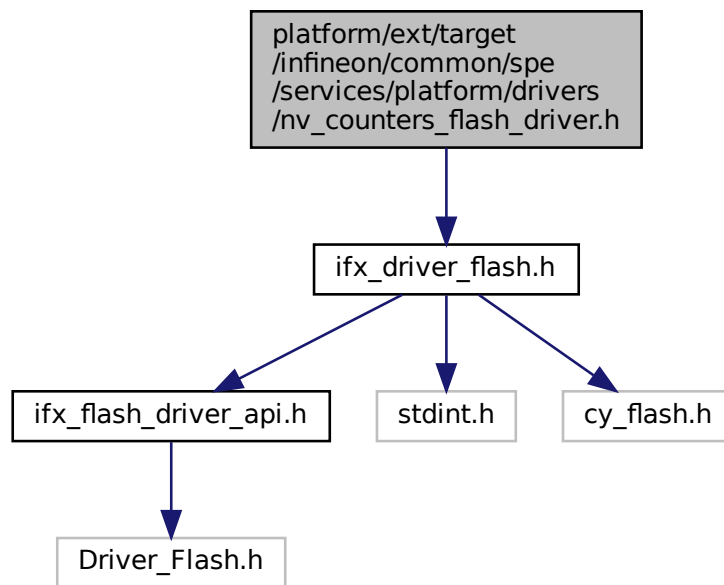
```
#define IFX_TFM_NV_COUNTERS_ERASE_VALUE IFX_DRIVER_FLASH_ERASE_VALUE
```

Definition at line 13 of file nv\_counters\_flash\_driver.c.

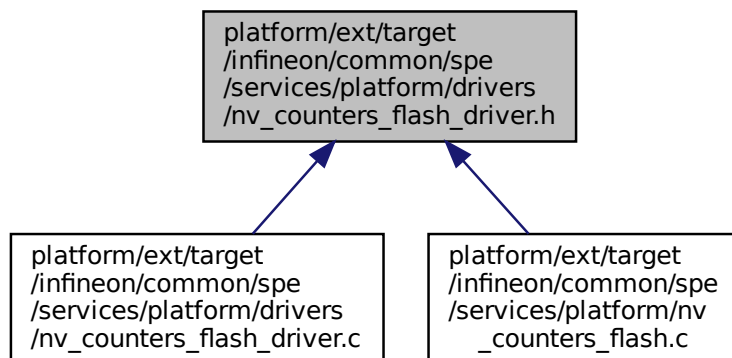
## 8.212 platform/ext/target/infineon/common/spe/services/platform/drivers/nv\_counters\_flash\_driver.h File Reference

```
#include "ifx_driver_flash.h"
```

Include dependency graph for nv\_counters\_flash\_driver.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_TFM_NV_COUNTERS_PROGRAM_UNIT IFX_DRIVER_FLASH_PROGRAM_UNIT`
- `#define IFX_TFM_NV_COUNTERS_SECTOR_SIZE IFX_DRIVER_FLASH_SECTOR_SIZE`
- `#define IFX_NV_COUNTERS_CMSIS_FLASH_INSTANCE (ifx_nv_counters_cmsis_flash_instance)`

## Variables

- `ARM_DRIVER_FLASH ifx_nv_counters_cmsis_flash_instance`

## 8.212.1 Macro Definition Documentation

### 8.212.1.1 IFX\_NV\_COUNTERS\_CMSIS\_FLASH\_INSTANCE

`#define IFX_NV_COUNTERS_CMSIS_FLASH_INSTANCE (ifx_nv_counters_cmsis_flash_instance)`  
 Definition at line 23 of file `nv_counters_flash_driver.h`.

### 8.212.1.2 IFX\_TFM\_NV\_COUNTERS\_PROGRAM\_UNIT

`#define IFX_TFM_NV_COUNTERS_PROGRAM_UNIT IFX_DRIVER_FLASH_PROGRAM_UNIT`  
 Definition at line 14 of file `nv_counters_flash_driver.h`.

### 8.212.1.3 IFX\_TFM\_NV\_COUNTERS\_SECTOR\_SIZE

`#define IFX_TFM_NV_COUNTERS_SECTOR_SIZE IFX_DRIVER_FLASH_SECTOR_SIZE`  
 Definition at line 16 of file `nv_counters_flash_driver.h`.

## 8.212.2 Variable Documentation

### 8.212.2.1 ifx\_nv\_counters\_cmsis\_flash\_instance

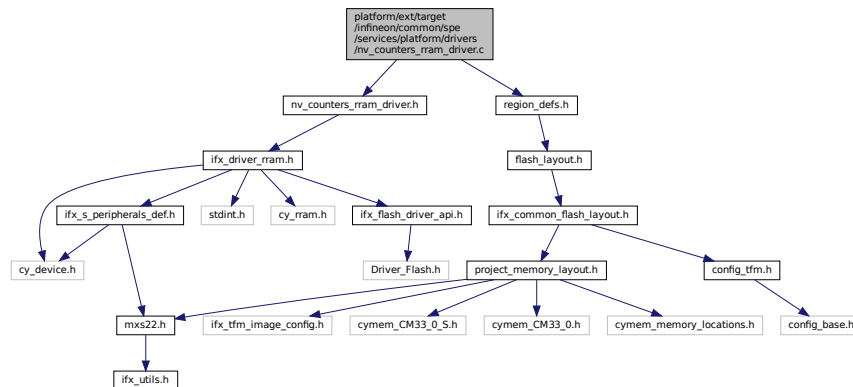
`ARM_DRIVER_FLASH ifx_nv_counters_cmsis_flash_instance`

## 8.213 platform/ext/target/infineon/common/spe/services/platform/drivers/nv\_counters\_rram\_driver.c File Reference

```
#include "nv_counters_rram_driver.h"
#include "region_defs.h"
```



Include dependency graph for nv\_counters\_rram\_driver.c:



## Macros

- `#define IFX_TFM_NV_COUNTERS_ERASE_VALUE IFX_DRIVER_RRAM_ERASE_VALUE`

### 8.213.1 Macro Definition Documentation

#### 8.213.1.1 IFX\_TFM\_NV\_COUNTERS\_ERASE\_VALUE

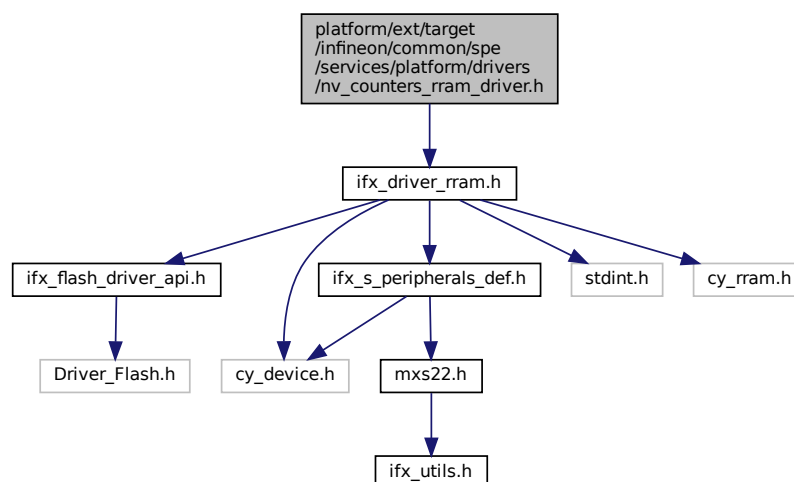
`#define IFX_TFM_NV_COUNTERS_ERASE_VALUE IFX_DRIVER_RRAM_ERASE_VALUE`

Definition at line 12 of file nv\_counters\_rram\_driver.c.

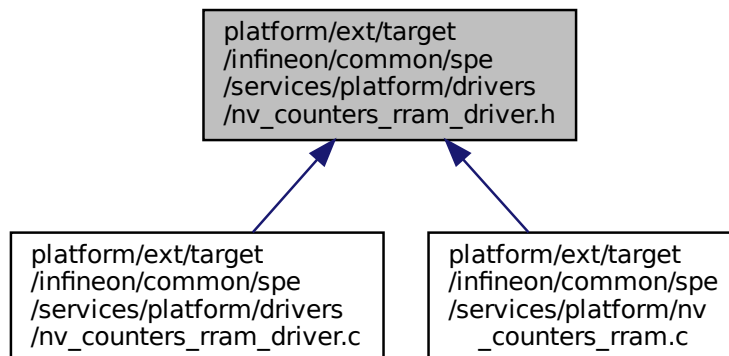
## 8.214 platform/ext/target/infineon/common/spe/services/platform/drivers/nv\_counters\_rram\_driver.h File Reference

`#include "ifx_driver_rram.h"`

Include dependency graph for nv\_counters\_rram\_driver.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_TFM_NV_COUNTERS_PROGRAM_UNIT IFX_DRIVER_RRAM_PROGRAM_UNIT`
- `#define IFX_TFM_NV_COUNTERS_SECTOR_SIZE CY_RRAM_BLOCK_SIZE_BYTES`
- `#define IFX_NV_COUNTERS_CMSIS_FLASH_INSTANCE (ifx_nv_counters_cmsis_flash_instance)`

## Variables

- `ARM_DRIVER_FLASH ifx_nv_counters_cmsis_flash_instance`

## 8.214.1 Macro Definition Documentation

### 8.214.1.1 IFX\_NV\_COUNTERS\_CMSIS\_FLASH\_INSTANCE

```
#define IFX_NV_COUNTERS_CMSIS_FLASH_INSTANCE (ifx_nv_counters_cmsis_flash_instance)
```

Definition at line 18 of file `nv_counters_rram_driver.h`.

### 8.214.1.2 IFX\_TFM\_NV\_COUNTERS\_PROGRAM\_UNIT

```
#define IFX_TFM_NV_COUNTERS_PROGRAM_UNIT IFX_DRIVER_RRAM_PROGRAM_UNIT
```

Definition at line 14 of file `nv_counters_rram_driver.h`.

### 8.214.1.3 IFX\_TFM\_NV\_COUNTERS\_SECTOR\_SIZE

```
#define IFX_TFM_NV_COUNTERS_SECTOR_SIZE CY_RRAM_BLOCK_SIZE_BYTES
```

Definition at line 15 of file `nv_counters_rram_driver.h`.

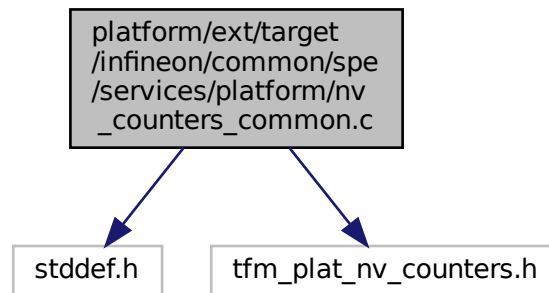
## 8.214.2 Variable Documentation

### 8.214.2.1 ifx\_nv\_counters\_cmsis\_flash\_instance

```
ARM_DRIVER_FLASH ifx_nv_counters_cmsis_flash_instance
```

## 8.215 platform/ext/target/infineon/common/spe/services/platform/nv\_counters\_common.c File Reference

```
#include <stddef.h>
#include "tfm_plat_nv_counters.h"
Include dependency graph for nv_counters_common.c:
```



### Functions

- enum tfm\_plat\_err\_t [tfm\\_plat\\_nv\\_counter\\_permissions\\_check](#) (int32\_t client\_id, enum [tfm\\_nv\\_counter\\_t](#) nv\_counter\_no, bool is\_read)
- enum tfm\_plat\_err\_t [tfm\\_plat\\_ns\\_counter\\_idx\\_to\\_nv\\_counter](#) (uint32\_t ns\_counter\_idx, enum [tfm\\_nv\\_counter\\_t](#) \*counter\_id)

### 8.215.1 Function Documentation

#### 8.215.1.1 tfm\_plat\_ns\_counter\_idx\_to\_nv\_counter()

```
enum tfm_plat_err_t tfm_plat_ns_counter_idx_to_nv_counter (
 uint32_t ns_counter_idx,
 enum tfm_nv_counter_t * counter_id)
```

Definition at line 46 of file [nv\\_counters\\_common.c](#).

#### 8.215.1.2 tfm\_plat\_nv\_counter\_permissions\_check()

```
enum tfm_plat_err_t tfm_plat_nv_counter_permissions_check (
 int32_t client_id,
 enum tfm_nv_counter_t nv_counter_no,
 bool is_read)
```

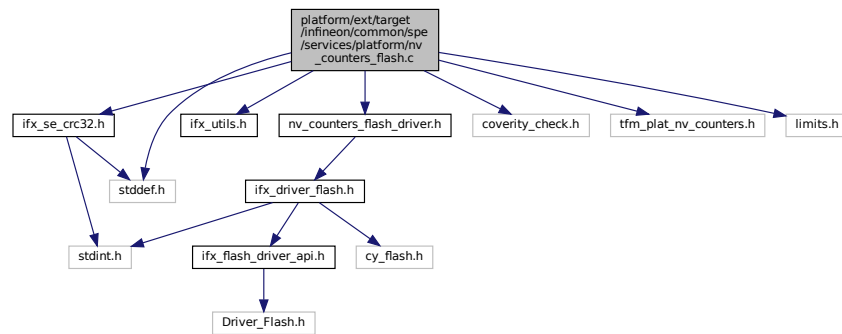
Definition at line 16 of file [nv\\_counters\\_common.c](#).

## 8.216 platform/ext/target/infineon/common/spe/services/platform/nv\_counters\_flash.c File Reference

```
#include "ifx_se_crc32.h"
#include "ifx_utils.h"
```

```
#include "nv_counters_flash_driver.h"
#include "coverity_check.h"
#include "tfm_plat_nv_counters.h"
#include <limits.h>
#include <stddef.h>
```

Include dependency graph for nv\_counters\_flash.c:



## Data Structures

- struct [ifx\\_nv\\_counters](#)  
*Struct representing the NV counter data in flash.*
- union [ifx\\_nv\\_init\\_done](#)

## Macros

- #define [IFX\\_NV\\_COUNTER\\_SIZE](#) sizeof(uint32\_t)
- #define [IFX\\_NUM\\_NV\\_COUNTERS](#) ((uint32\_t)PLAT\_NV\_COUNTER\_MAX)
- #define [IFX\\_NV\\_COUNTERS\\_CONTENT\\_SIZE](#)
- #define [IFX\\_NV\\_COUNTERS\\_BLOCK\\_SIZE](#)
- #define [IFX\\_NV\\_COUNTERS\\_INITIALIZED](#) 0xC0DE0042U
- #define [IFX\\_NVM\\_INIT\\_DONE\\_FLAG](#) 0xAA5533CCU
- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_AREA\\_OFFSET](#) (0U)
- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_BACKUP\\_OFFSET](#)
- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_INIT\\_DONE\\_OFFSET](#)
- #define [IFX\\_NV\\_COUNTERS\\_CRC\\_INIT](#) (0)
- #define [IFX\\_NV\\_INIT\\_DONE\\_BLOCK\\_SIZE](#)

## Typedefs

- typedef struct [ifx\\_nv\\_counters](#) [ifx\\_nv\\_counters\\_t](#)  
*Struct representing the NV counter data in flash.*
- typedef union [ifx\\_nv\\_init\\_done](#) [ifx\\_nv\\_init\\_done\\_t](#)

## Functions

- enum [tfm\\_plat\\_err\\_t](#) [tfm\\_plat\\_init\\_nv\\_counter](#) (void)
- enum [tfm\\_plat\\_err\\_t](#) [tfm\\_plat\\_read\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id, uint32\_t size, uint8\_t \*val)
- enum [tfm\\_plat\\_err\\_t](#) [tfm\\_plat\\_set\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id, uint32\_t value)
- enum [tfm\\_plat\\_err\\_t](#) [tfm\\_plat\\_increment\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id)

## 8.216.1 Macro Definition Documentation

### 8.216.1.1 IFX\_NUM\_NV\_COUNTERS

```
#define IFX_NUM_NV_COUNTERS ((uint32_t) PLAT_NV_COUNTER_MAX)
```

Definition at line 34 of file nv\_counters\_flash.c.

### 8.216.1.2 IFX\_NV\_COUNTER\_SIZE

```
#define IFX_NV_COUNTER_SIZE sizeof(uint32_t)
```

Definition at line 28 of file nv\_counters\_flash.c.

### 8.216.1.3 IFX\_NV\_COUNTERS\_BLOCK\_SIZE

```
#define IFX_NV_COUNTERS_BLOCK_SIZE
```

**Value:**

```
\
IFX_ROUND_UP_TO_MULTIPLE(IFX_NV_COUNTERS_CONTENT_SIZE,
IFX_TFM_NV_COUNTERS_PROGRAM_UNIT)
```

Definition at line 40 of file nv\_counters\_flash.c.

### 8.216.1.4 IFX\_NV\_COUNTERS\_CONTENT\_SIZE

```
#define IFX_NV_COUNTERS_CONTENT_SIZE
```

**Value:**

```
((2U * sizeof(uint32_t)) + \
(IFX_NUM_NV_COUNTERS * IFX_NV_COUNTER_SIZE))
```

Definition at line 37 of file nv\_counters\_flash.c.

### 8.216.1.5 IFX\_NV\_COUNTERS\_CRC\_INIT

```
#define IFX_NV_COUNTERS_CRC_INIT (0)
```

Definition at line 58 of file nv\_counters\_flash.c.

### 8.216.1.6 IFX\_NV\_COUNTERS\_INITIALIZED

```
#define IFX_NV_COUNTERS_INITIALIZED 0xC0DE0042U
```

Definition at line 43 of file nv\_counters\_flash.c.

### 8.216.1.7 IFX\_NV\_INIT\_DONE\_BLOCK\_SIZE

```
#define IFX_NV_INIT_DONE_BLOCK_SIZE
```

**Value:**

```
IFX_ROUND_UP_TO_MULTIPLE(sizeof(uint32_t), \
IFX_TFM_NV_COUNTERS_PROGRAM_UNIT)
```

Definition at line 76 of file nv\_counters\_flash.c.

### 8.216.1.8 IFX\_NVM\_INIT\_DONE\_FLAG

```
#define IFX_NVM_INIT_DONE_FLAG 0xAA5533CCU
```

Definition at line 50 of file nv\_counters\_flash.c.

### 8.216.1.9 IFX\_TFM\_NV\_COUNTERS\_AREA\_OFFSET

```
#define IFX_TFM_NV_COUNTERS_AREA_OFFSET (0U)
```

Definition at line 52 of file nv\_counters\_flash.c.

### 8.216.1.10 IFX\_TFM\_NV\_COUNTERS\_BACKUP\_OFFSET

```
#define IFX_TFM_NV_COUNTERS_BACKUP_OFFSET
```

**Value:**

```
(IFX_TFM_NV_COUNTERS_AREA_OFFSET \
+ IFX_TFM_NV_COUNTERS_SECTOR_SIZE)
```

Definition at line 53 of file nv\_counters\_flash.c.

### 8.216.1.11 IFX\_TFM\_NV\_COUNTERS\_INIT\_DONE\_OFFSET

```
#define IFX_TFM_NV_COUNTERS_INIT_DONE_OFFSET
```

**Value:**

```
(IFX_TFM_NV_COUNTERS_BACKUP_OFFSET \
+ IFX_TFM_NV_COUNTERS_SECTOR_SIZE)
```

Definition at line 55 of file nv\_counters\_flash.c.

## 8.216.2 Typedef Documentation

### 8.216.2.1 ifx\_nv\_counters\_t

```
typedef struct ifx_nv_counters ifx_nv_counters_t
```

Struct representing the NV counter data in flash.

### 8.216.2.2 ifx\_nv\_init\_done\_t

```
typedef union ifx_nv_init_done ifx_nv_init_done_t
```

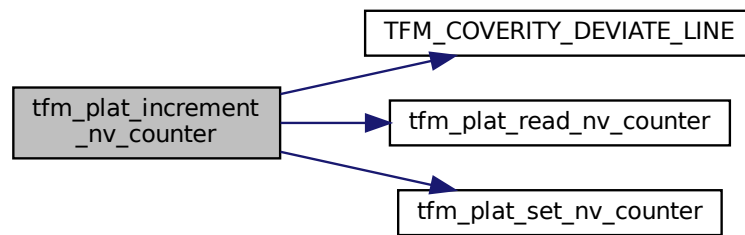
## 8.216.3 Function Documentation

### 8.216.3.1 tfm\_plat\_increment\_nv\_counter()

```
enum tfm_plat_err_t tfm_plat_increment_nv_counter (
 enum tfm_nv_counter_t counter_id)
```

Definition at line 363 of file nv\_counters\_flash.c.

Here is the call graph for this function:



### 8.216.3.2 tfm\_plat\_init\_nv\_counter()

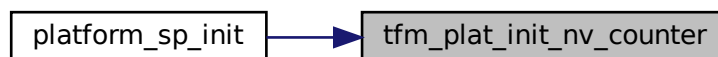
```
enum tfm_plat_err_t tfm_plat_init_nv_counter (
 void)
```

Definition at line 156 of file `nv_counters_flash.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

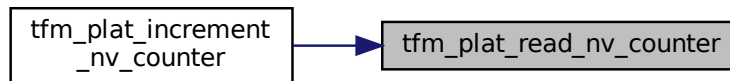


### 8.216.3.3 tfm\_plat\_read\_nv\_counter()

```
enum tfm_plat_err_t tfm_plat_read_nv_counter (
 enum tfm_nv_counter_t counter_id,
 uint32_t size,
 uint8_t * val)
```

Definition at line 266 of file `nv_counters_flash.c`.

Here is the caller graph for this function:



#### 8.216.3.4 tfm\_plat\_set\_nv\_counter()

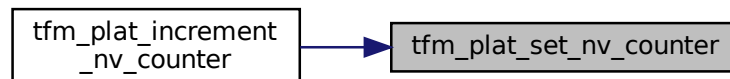
```

enum tfm_plat_err_t tfm_plat_set_nv_counter (
 enum tfm_nv_counter_t counter_id,
 uint32_t value)

```

Definition at line 300 of file nv\_counters\_flash.c.

Here is the caller graph for this function:



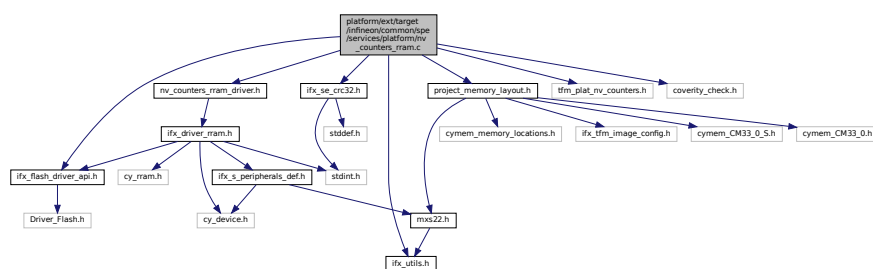
## 8.217 platform/ext/target/infineon/common/spe/services/platform/nv\_counters\_ram.c File Reference

```

#include "ifx_flash_driver_api.h"
#include "ifx_se_crc32.h"
#include "ifx_utils.h"
#include "nv_counters_rram_driver.h"
#include "project_memory_layout.h"
#include "tfm_plat_nv_counters.h"
#include "coverity_check.h"

```

Include dependency graph for nv\_counters\_rram.c:





## Data Structures

- struct [ifx\\_rram\\_counter\\_t](#)

## Macros

- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_BASE](#) (0UL)
- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_COUNTER\\_AREA\\_OFFSET](#) (0UL)
- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_BACKUP\\_AREA\\_OFFSET](#)
- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_BACKUP\\_AREA\\_SIZE](#)
- #define [IFX\\_NVM\\_INIT\\_DONE\\_FLAG](#) (0xAA5533CCUL)
- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_NVM\\_INIT\\_DONE\\_FLAG\\_OFFSET](#)
- #define [IFX\\_TFM\\_NV\\_COUNTERS\\_EXPECTED\\_AREA\\_SIZE](#)

## Typedefs

- typedef uint32\_t [ifx\\_rram\\_counter\\_value\\_t](#)

## Functions

- enum tfm\_plat\_err\_t [ifx\\_rram\\_clear\\_counters\\_and\\_backups](#) (void)
- enum tfm\_plat\_err\_t [tfm\\_plat\\_init\\_nv\\_counter](#) (void)
- enum tfm\_plat\_err\_t [tfm\\_plat\\_read\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id, uint32\_t [size](#), uint8\_t \*val)
- enum tfm\_plat\_err\_t [tfm\\_plat\\_set\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id, uint32\_t value)
- enum tfm\_plat\_err\_t [tfm\\_plat\\_increment\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id)

## 8.217.1 Macro Definition Documentation

### 8.217.1.1 IFX\_NVM\_INIT\_DONE\_FLAG

```
#define IFX_NVM_INIT_DONE_FLAG (0xAA5533CCUL)
```

Definition at line 44 of file `nv_counters_rram.c`.

### 8.217.1.2 IFX\_TFM\_NV\_COUNTERS\_BACKUP\_AREA\_OFFSET

```
#define IFX_TFM_NV_COUNTERS_BACKUP_AREA_OFFSET
```

**Value:**

```
(IFX_TFM_NV_COUNTERS_COUNTER_AREA_OFFSET \
+ ((PLAT_NV_COUNTER_MAX) \
* sizeof(ifx_rram_counter_t)))
```

Definition at line 38 of file `nv_counters_rram.c`.

### 8.217.1.3 IFX\_TFM\_NV\_COUNTERS\_BACKUP\_AREA\_SIZE

```
#define IFX_TFM_NV_COUNTERS_BACKUP_AREA_SIZE
```

**Value:**

```
(PLAT_NV_COUNTER_MAX \
* sizeof(ifx_rram_counter_t))
```

Definition at line 41 of file `nv_counters_rram.c`.

### 8.217.1.4 IFX\_TFM\_NV\_COUNTERS\_BASE

```
#define IFX_TFM_NV_COUNTERS_BASE (0UL)
```

Definition at line 34 of file `nv_counters_rram.c`.

### 8.217.1.5 IFX\_TFM\_NV\_COUNTERS\_COUNTER\_AREA\_OFFSET

```
#define IFX_TFM_NV_COUNTERS_COUNTER_AREA_OFFSET (0UL)
```

Definition at line 36 of file nv\_counters\_rram.c.

### 8.217.1.6 IFX\_TFM\_NV\_COUNTERS\_EXPECTED\_AREA\_SIZE

```
#define IFX_TFM_NV_COUNTERS_EXPECTED_AREA_SIZE
```

**Value:**

```
((uint32_t)PLAT_NV_COUNTER_MAX)), \
 (IFX_ALIGN_UP_TO((sizeof(ifx_rram_counter_t)
 * (2U *
 IFX_TFM_NV_COUNTERS_SECTOR_SIZE) \
 + IFX_TFM_NV_COUNTERS_SECTOR_SIZE)
```

Definition at line 59 of file nv\_counters\_rram.c.

### 8.217.1.7 IFX\_TFM\_NV\_COUNTERS\_NVM\_INIT\_DONE\_FLAG\_OFFSET

```
#define IFX_TFM_NV_COUNTERS_NVM_INIT_DONE_FLAG_OFFSET
```

**Value:**

```
+ \
 (IFX_ALIGN_UP_TO((IFX_TFM_NV_COUNTERS_BACKUP_AREA_OFFSET
 IFX_TFM_NV_COUNTERS_BACKUP_AREA_SIZE), \
 IFX_TFM_NV_COUNTERS_SECTOR_SIZE))
```

Definition at line 48 of file nv\_counters\_rram.c.

## 8.217.2 Typedef Documentation

### 8.217.2.1 ifx\_rram\_counter\_value\_t

```
typedef uint32_t ifx_rram_counter_value_t
```

Definition at line 21 of file nv\_counters\_rram.c.

## 8.217.3 Function Documentation

### 8.217.3.1 ifx\_rram\_clear\_counters\_and\_backups()

```
enum tfm_plat_err_t ifx_rram_clear_counters_and_backups (
 void)
```

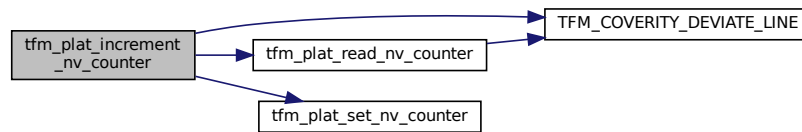
Definition at line 331 of file nv\_counters\_rram.c.

### 8.217.3.2 tfm\_plat\_increment\_nv\_counter()

```
enum tfm_plat_err_t tfm_plat_increment_nv_counter (
 enum tfm_nv_counter_t counter_id)
```

Definition at line 496 of file nv\_counters\_rram.c.

Here is the call graph for this function:



### 8.217.3.3 tfm\_plat\_init\_nv\_counter()

```
enum tfm_plat_err_t tfm_plat_init_nv_counter (
 void)
```

Definition at line 360 of file `nv_counters_rram.c`.

Here is the call graph for this function:



### 8.217.3.4 tfm\_plat\_read\_nv\_counter()

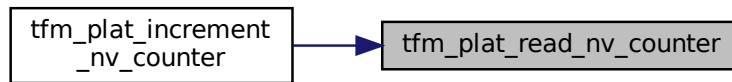
```
enum tfm_plat_err_t tfm_plat_read_nv_counter (
 enum tfm_nv_counter_t counter_id,
 uint32_t size,
 uint8_t * val)
```

Definition at line 440 of file `nv_counters_rram.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.217.3.5 tfm\_plat\_set\_nv\_counter()

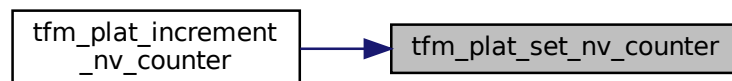
```

enum tfm_plat_err_t tfm_plat_set_nv_counter (
 enum tfm_nv_counter_t counter_id,
 uint32_t value)

```

Definition at line 453 of file nv\_counters\_ram.c.

Here is the caller graph for this function:



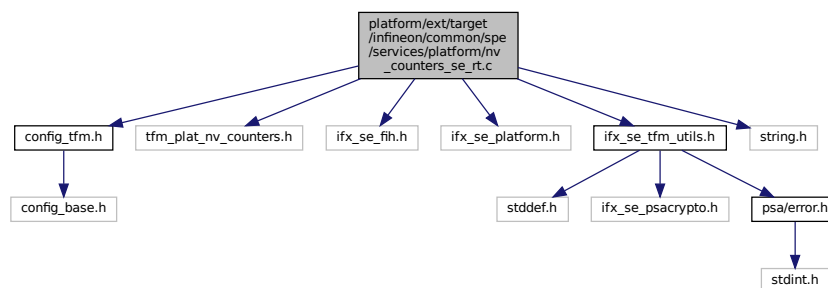
## 8.218 platform/ext/target/infineon/common/spe/services/platform/nv\_counters\_se\_rt.c File Reference

```

#include "config_tfm.h"
#include "tfm_plat_nv_counters.h"
#include "ifx_se_fih.h"
#include "ifx_se_platform.h"
#include "ifx_se_tfm_utils.h"
#include <string.h>

```

Include dependency graph for nv\_counters\_se\_rt.c:



## Macros

- #define `IFX_PLAT_NV_COUNTER_PS_0` `IFX_SE_RRAM_ROLLBACK_COUNTER_NUM_MIN`  
*RRAM counters 0..2 are used for Protected Storage.*
- #define `IFX_PLAT_NV_COUNTER_NS_0`  
*RRAM counters 3..5 are used for NS.*
- #define `IFX_PLAT_RRAM_COUNTER_NUMBER`  
*Number of RRAM counters use by Platform Service.*

## Functions

- enum `tfm_plat_err_t` `tfm_plat_init_nv_counter` (`void`)
- enum `tfm_plat_err_t` `tfm_plat_read_nv_counter` (`enum tfm_nv_counter_t` `counter_id`, `uint32_t` `size`, `uint8_t` `*val`)
- enum `tfm_plat_err_t` `tfm_plat_increment_nv_counter` (`enum tfm_nv_counter_t` `counter_id`)

## 8.218.1 Macro Definition Documentation

### 8.218.1.1 IFX\_PLAT\_NV\_COUNTER\_NS\_0

```
#define IFX_PLAT_NV_COUNTER_NS_0
```

**Value:**

```
(IFX_PLAT_NV_COUNTER_PS_0 \
+ ((uint32_t)PLAT_NV_COUNTER_PS_2 \
- (uint32_t)PLAT_NV_COUNTER_PS_0 + 1U))
```

RRAM counters 3..5 are used for NS.

Definition at line 21 of file `nv_counters_se_rt.c`.

### 8.218.1.2 IFX\_PLAT\_NV\_COUNTER\_PS\_0

```
#define IFX_PLAT_NV_COUNTER_PS_0 IFX_SE_RRAM_ROLLBACK_COUNTER_NUM_MIN
```

RRAM counters 0..2 are used for Protected Storage.

Definition at line 18 of file `nv_counters_se_rt.c`.

### 8.218.1.3 IFX\_PLAT\_RRAM\_COUNTER\_NUMBER

```
#define IFX_PLAT_RRAM_COUNTER_NUMBER
```

**Value:**

```
((PLAT_NV_COUNTER_PS_2 - PLAT_NV_COUNTER_PS_0 + 1U) \
+ (PLAT_NV_COUNTER_NS_2 - PLAT_NV_COUNTER_NS_0 + 1U))
```

Number of RRAM counters use by Platform Service.

Definition at line 26 of file `nv_counters_se_rt.c`.

## 8.218.2 Function Documentation

### 8.218.2.1 tfm\_plat\_increment\_nv\_counter()

```
enum tfm_plat_err_t tfm_plat_increment_nv_counter (
 enum tfm_nv_counter_t counter_id)
```

Definition at line 104 of file `nv_counters_se_rt.c`.

### 8.218.2.2 tfm\_plat\_init\_nv\_counter()

```
enum tfm_plat_err_t tfm_plat_init_nv_counter (
 void)
```

Definition at line 67 of file nv\_counters\_se\_rt.c.

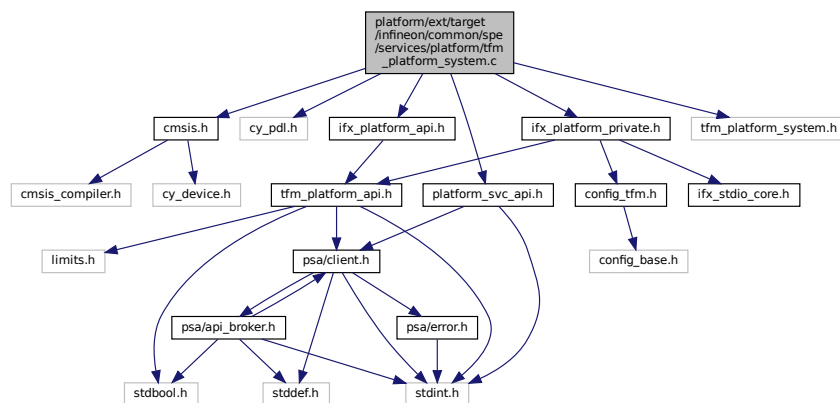
### 8.218.2.3 tfm\_plat\_read\_nv\_counter()

```
enum tfm_plat_err_t tfm_plat_read_nv_counter (
 enum tfm_nv_counter_t counter_id,
 uint32_t size,
 uint8_t * val)
```

Definition at line 72 of file nv\_counters\_se\_rt.c.

## 8.219 platform/ext/target/infineon/common/spe/services/platform/tfm\_platform\_system.c File Reference

```
#include "cmsis.h"
#include "cy_pdl.h"
#include "ifx_platform_api.h"
#include "ifx_platform_private.h"
#include "platform_svc_api.h"
#include "tfm_platform_system.h"
Include dependency graph for tfm_platform_system.c:
```



## Functions

- `void tfm_platform_hal_system_reset(void)`
- `enum tfm_platform_err_t tfm_platform_hal_ioctl (tfm_platform_ioctl_req_t request, psa_invec *in_vec, psa_outvec *out_vec)`

## 8.219.1 Function Documentation

### 8.219.1.1 tfm\_platform\_hal\_ioctl()

```
enum tfm_platform_err_t tfm_platform_hal_ioctl (
 tfm_platform_ioctl_req_t request,
```

```

 psa_invec * in_vec,
 psa_outvec * out_vec)

```

Definition at line 23 of file tfm\_platform\_system.c.

Here is the call graph for this function:



#### 8.219.1.2 tfm\_platform\_hal\_system\_reset()

```

void tfm_platform_hal_system_reset (
 void)

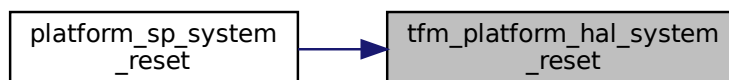
```

Definition at line 16 of file tfm\_platform\_system.c.

Here is the call graph for this function:



Here is the caller graph for this function:



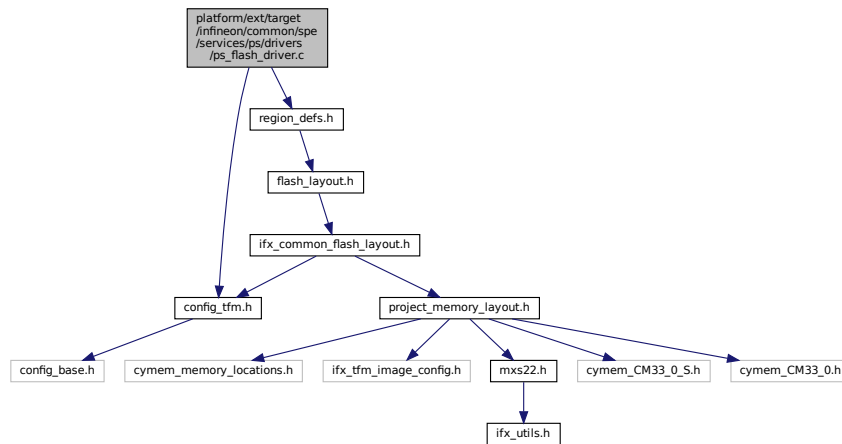
## 8.220 platform/ext/target/infineon/common/spe/services/ps/drivers/ps\_flash\_driver.c File Reference ↩

```

#include "config_tfm.h"
#include "region_defs.h"

```

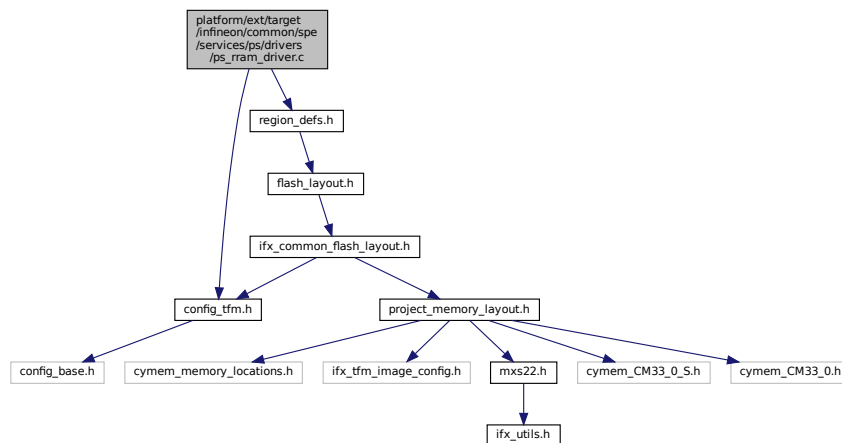
Include dependency graph for `ps_flash_driver.c`:



## 8.221 platform/ext/target/infineon/common/spe/services/ps/drivers/ps\_↩ rram\_driver.c File Reference

```
#include "config_tfm.h"
#include "region_defs.h"
```

Include dependency graph for `ps_rram_driver.c`:

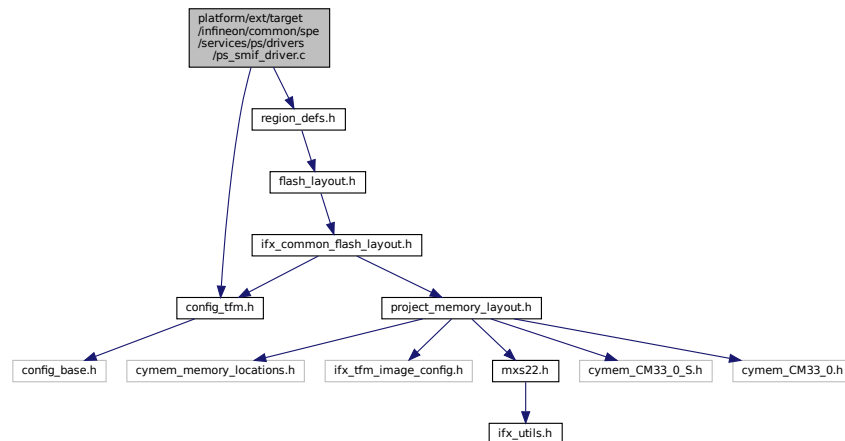


## 8.222 platform/ext/target/infineon/common/spe/services/ps/drivers/ps\_↩ smif\_driver.c File Reference

```
#include "config_tfm.h"
#include "region_defs.h"
```



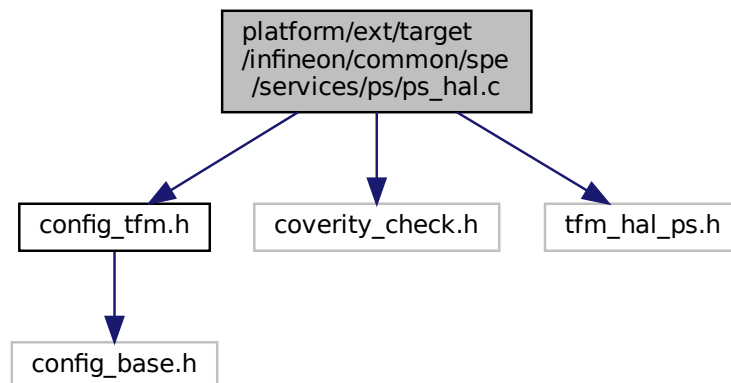
Include dependency graph for ps\_smif\_driver.c:



## 8.223 platform/ext/target/infineon/common/spe/services/ps/ps\_hal.c File Reference

```
#include "config_tfm.h"
#include "coverity_check.h"
#include "tfm_hal_ps.h"
```

Include dependency graph for ps\_hal.c:



### Functions

- enum tfm\_hal\_status\_t [tfm\\_hal\\_ps\\_fs\\_info](#) (struct tfm\_hal\_ps\_fs\_info\_t \*fs\_info)

#### 8.223.1 Function Documentation

### 8.223.1.1 tfm\_hal\_ps\_fs\_info()

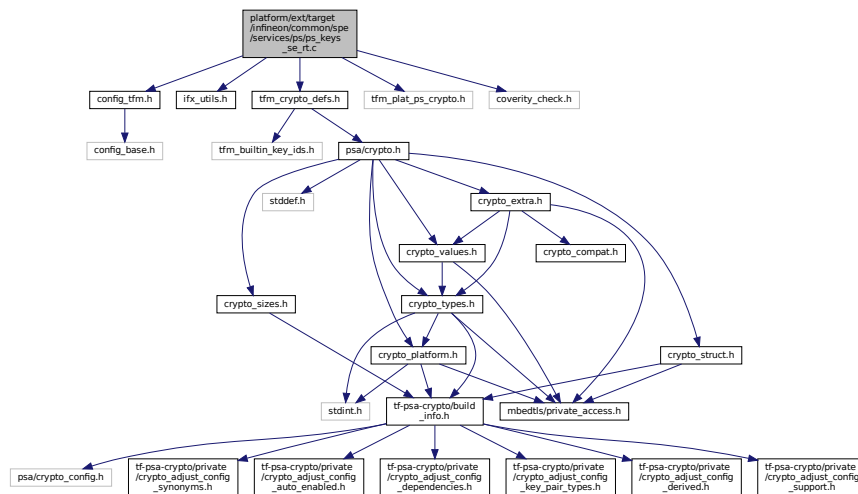
```
enum tfm_hal_status_t tfm_hal_ps_fs_info (
 struct tfm_hal_ps_fs_info_t * fs_info)
```

Definition at line 14 of file ps\_hal.c.

## 8.224 platform/ext/target/infineon/common/spe/services/ps/ps\_keys\_se\_rt.c File Reference

```
#include "config_tfm.h"
#include "ifx_utils.h"
#include "tfm_crypto_defs.h"
#include "tfm_plat_ps_crypto.h"
#include "coverity_check.h"
```

Include dependency graph for ps\_keys\_se\_rt.c:



## Functions

- [psa\\_status\\_t tfm\\_platform\\_ps\\_set\\_key \(psa\\_key\\_id\\_t \\*ps\\_key, const uint8\\_t \\*key\\_label, size\\_t key\\_label\\_len\)](#)

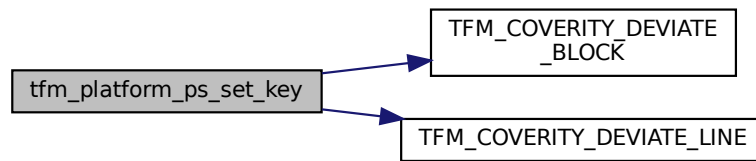
### 8.224.1 Function Documentation

#### 8.224.1.1 tfm\_platform\_ps\_set\_key()

```
psa_status_t tfm_platform_ps_set_key (
 psa_key_id_t * ps_key,
 const uint8_t * key_label,
 size_t key_label_len)
```

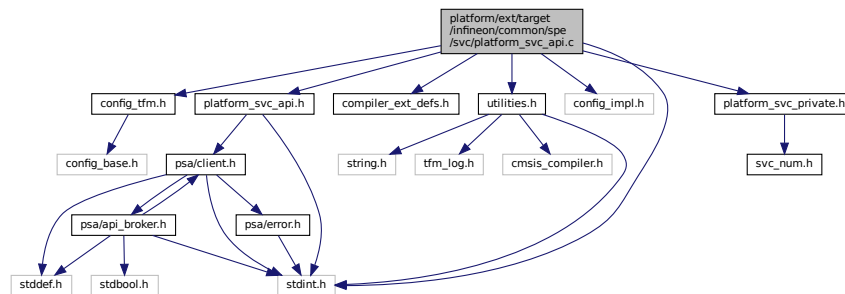
Definition at line 17 of file ps\_keys\_se\_rt.c.

Here is the call graph for this function:



## 8.225 platform/ext/target/infineon/common/spe/svc/platform\_svc\_api.c File Reference

```
#include <stdint.h>
#include "config_tfm.h"
#include "compiler_ext_defs.h"
#include "utilities.h"
#include "config_impl.h"
#include "platform_svc_api.h"
#include "platform_svc_private.h"
Include dependency graph for platform_svc_api.c:
```



### Functions

- `__naked int32_t ifx_call_platform_uart_log` (const char \*str, uint32\_t len, uint32\_t core\_id)  
*Write string to SPM UART log.*
- `__naked void ifx_call_platform_system_reset` (void)  
*Resets the system.*
- `__naked psa_status_t ifx_call_platform_original_iovec` (psa\_handle\_t msg\_handle, ifx\_original\_iovec\_t \*iovec)  
*Retrieve the original IO vectors.*

### 8.225.1 Function Documentation

### 8.225.1.1 ifx\_call\_platform\_original\_iovec()

```
__naked psa_status_t ifx_call_platform_original_iovec (
 psa_handle_t msg_handle,
 ifx_original_iovec_t * io_vec)
```

Retrieve the original IO vectors.

Definition at line 43 of file platform\_svc\_api.c.

### 8.225.1.2 ifx\_call\_platform\_system\_reset()

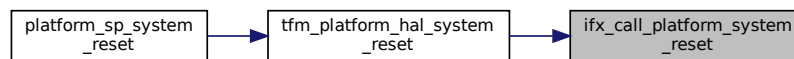
```
__naked void ifx_call_platform_system_reset (
 void)
```

Resets the system.

Requests a system reset to reset the MCU.

Definition at line 37 of file platform\_svc\_api.c.

Here is the caller graph for this function:



### 8.225.1.3 ifx\_call\_platform\_uart\_log()

```
__naked int32_t ifx_call_platform_uart_log (
 const char * str,
 uint32_t len,
 uint32_t core_id)
```

Write string to SPM UART log.

#### Parameters

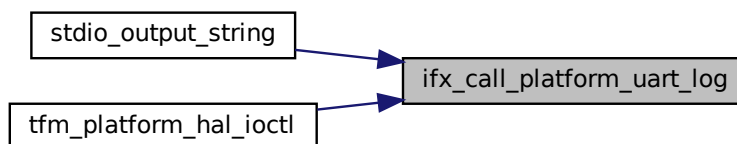
|    |                |                                                         |
|----|----------------|---------------------------------------------------------|
| in | <i>str</i>     | String to output                                        |
| in | <i>len</i>     | String size                                             |
| in | <i>core_id</i> | Core prefix id, see <a href="#">ifx_stdio_core_id_t</a> |

#### Return values

|                 |                                                    |
|-----------------|----------------------------------------------------|
| <i>Platform</i> | error code, see <a href="#">tfm_platform_err_t</a> |
|-----------------|----------------------------------------------------|

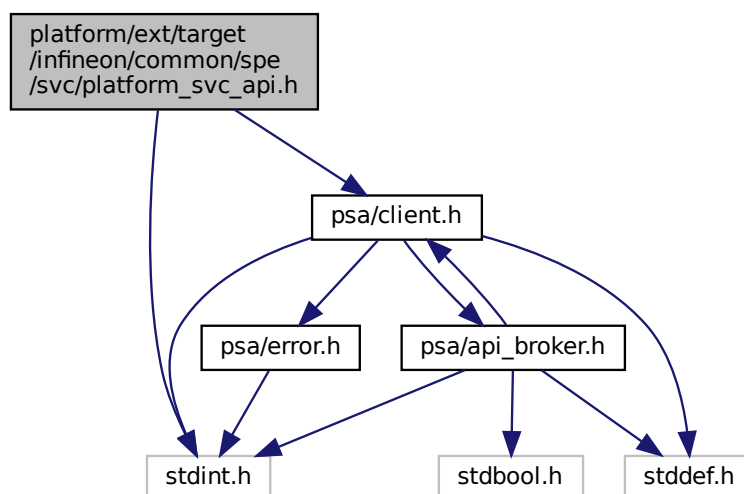
Definition at line 18 of file platform\_svc\_api.c.

Here is the caller graph for this function:

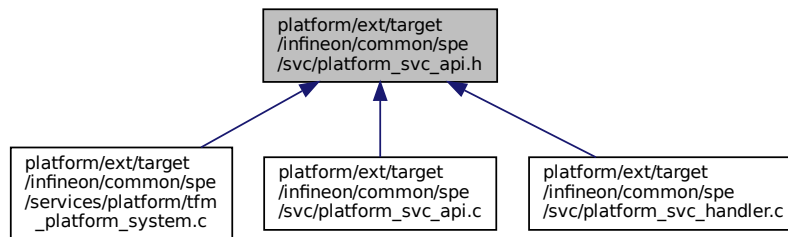


## 8.226 platform/ext/target/infineon/common/spe/svc/platform\_svc\_api.h File Reference

```
#include <stdint.h>
#include "psa/client.h"
Include dependency graph for platform_svc_api.h:
```



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ifx\\_original\\_iovec\\_t](#)

## Typedefs

- typedef struct [ifx\\_original\\_iovec\\_t](#) [ifx\\_original\\_iovec\\_t](#)

## Functions

- `int32_t ifx_call_platform_enable_systick (uint32_t enable)`  
*Enable/disable SysTick for FLIH/SLIH tf-m-tests.*
- `int32_t ifx_call_platform_uart_log (const char *str, uint32_t len, uint32_t core_id)`  
*Write string to SPM UART log.*
- `void ifx_call_platform_system_reset (void)`  
*Resets the system.*
- `psa_status_t ifx_call_platform_original_iovec (psa_handle_t msg_handle, ifx_original_iovec_t *io_vec)`  
*Retrieve the original IO vectors.*

## 8.226.1 Typedef Documentation

### 8.226.1.1 ifx\_original\_iovec\_t

```
typedef struct ifx_original_iovec_t ifx_original_iovec_t
```

Structure to hold original input and output vectors and their counts

## 8.226.2 Function Documentation

### 8.226.2.1 ifx\_call\_platform\_enable\_systick()

```
int32_t ifx_call_platform_enable_systick (
 uint32_t enable)
```

Enable/disable SysTick for FLIH/SLIH tf-m-tests.

#### Parameters

|    |        |                                            |
|----|--------|--------------------------------------------|
| in | enable | 0 - disable SysTick IRQ, != 0 - enable IRQ |
|----|--------|--------------------------------------------|

## Return values

|                 |                                                    |
|-----------------|----------------------------------------------------|
| <i>Platform</i> | error code, see <a href="#">tfm_platform_err_t</a> |
|-----------------|----------------------------------------------------|

**8.226.2.2 ifx\_call\_platform\_original\_iovec()**

```
psa_status_t ifx_call_platform_original_iovec (
 psa_handle_t msg_handle,
 ifx_original_iovec_t * io_vec)
```

Retrieve the original IO vectors.

Definition at line 43 of file platform\_svc\_api.c.

**8.226.2.3 ifx\_call\_platform\_system\_reset()**

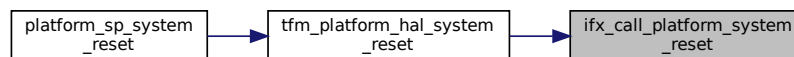
```
void ifx_call_platform_system_reset (
 void)
```

Resets the system.

Requests a system reset to reset the MCU.

Definition at line 37 of file platform\_svc\_api.c.

Here is the caller graph for this function:

**8.226.2.4 ifx\_call\_platform\_uart\_log()**

```
int32_t ifx_call_platform_uart_log (
 const char * str,
 uint32_t len,
 uint32_t core_id)
```

Write string to SPM UART log.

## Parameters

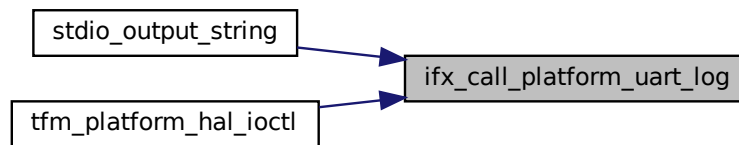
|    |                |                                                         |
|----|----------------|---------------------------------------------------------|
| in | <i>str</i>     | String to output                                        |
| in | <i>len</i>     | String size                                             |
| in | <i>core_id</i> | Core prefix id, see <a href="#">ifx_stdio_core_id_t</a> |

## Return values

|                 |                                                    |
|-----------------|----------------------------------------------------|
| <i>Platform</i> | error code, see <a href="#">tfm_platform_err_t</a> |
|-----------------|----------------------------------------------------|

Definition at line 18 of file platform\_svc\_api.c.

Here is the caller graph for this function:



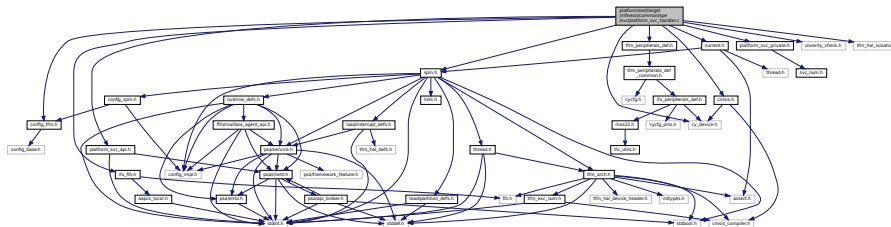
## 8.227 platform/ext/target/infineon/common/spe/svc/platform\_svc\_handler.c File Reference

```

#include "config_tfm.h"
#include "cmsis.h"
#include "current.h"
#include "cy_device.h"
#include "ifx_fih.h"
#include "platform_svc_api.h"
#include "platform_svc_private.h"
#include "spm.h"
#include "coverity_check.h"
#include "tfm_hal_isolation.h"
#include "tfm_peripherals_def.h"

```

Include dependency graph for platform\_svc\_handler.c:



### Functions

- [int32\\_t platform\\_svc\\_handlers](#) (uint8\_t svc\_num, uint32\_t \*svc\_args, uint32\_t lr)

### 8.227.1 Function Documentation

#### 8.227.1.1 platform\_svc\_handlers()

```

int32_t platform_svc_handlers (
 uint8_t svc_num,
 uint32_t * svc_args,
 uint32_t lr)

```

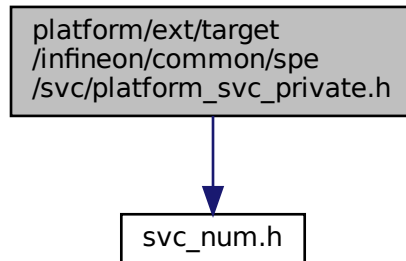
Definition at line 180 of file platform\_svc\_handler.c.



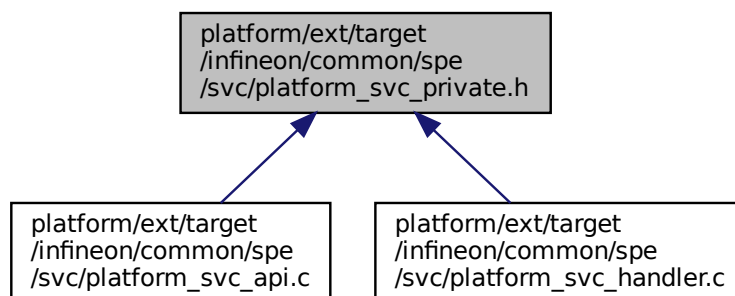
## 8.228 platform/ext/target/infineon/common/spe/svc/platform\_svc\_private.h File Reference

```
#include "svc_num.h"
```

Include dependency graph for platform\_svc\_private.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define IFX_SVC_PLATFORM_UART_LOG TFM_SVC_NUM_PLATFORM_THREAD(0x0U)`
- `#define IFX_SVC_PLATFORM_ENABLE_SYSTICK TFM_SVC_NUM_PLATFORM_THREAD(0x1U)`
- `#define IFX_SVC_PLATFORM_SYSTEM_RESET TFM_SVC_NUM_PLATFORM_THREAD(0x2U)`
- `#define IFX_SVC_PLATFORM_ORIGINAL_IOVEC TFM_SVC_NUM_PLATFORM_THREAD(0x3U)`

### 8.228.1 Macro Definition Documentation

#### 8.228.1.1 IFX\_SVC\_PLATFORM\_ENABLE\_SYSTICK

```
#define IFX_SVC_PLATFORM_ENABLE_SYSTICK TFM_SVC_NUM_PLATFORM_THREAD(0x1U)
```

Definition at line 18 of file platform\_svc\_private.h.



## 8.229.2 Function Documentation

### 8.229.2.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_plat_err_t)
```

Configures fault handlers and sets priorities.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
Check memory access permissions using MPC attribute cache.  
This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

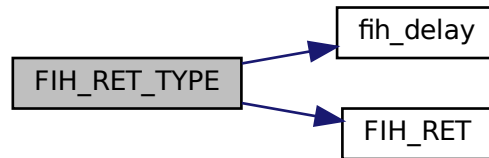
**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 19 of file target\_cfg.c.  
Here is the call graph for this function:



### 8.229.2.2 ifx\_init\_debug()

```
enum tfm_plat_err_t ifx_init_debug (
 void)
```

Configures the system debug properties.

#### Returns

Returns values as specified by the tfm\_plat\_err\_t

Definition at line 43 of file target\_cfg.c.  
Here is the caller graph for this function:



### 8.229.2.3 ifx\_init\_system\_control\_block()

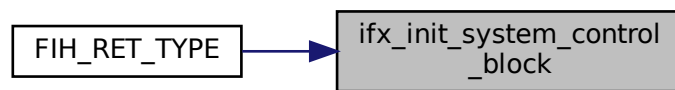
```
enum tfm_plat_err_t ifx_init_system_control_block (
 void)
```

Configures Sleep and Exception Handling (System Control Block).

### Returns

Returns values as specified by the `tfm_plat_err_t`

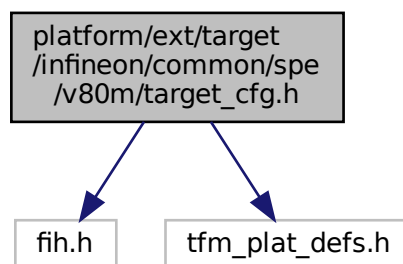
Definition at line 98 of file `target_cfg.c`.  
Here is the caller graph for this function:



## 8.230 platform/ext/target/infineon/common/spe/v80m/target\_cfg.h File Reference

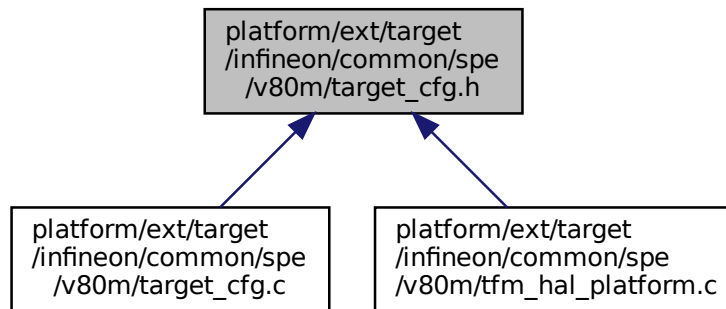
```
#include "fih.h"
#include "tfm_plat_defs.h"
```

Include dependency graph for `target_cfg.h`:





This graph shows which files directly or indirectly include this file:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum tfm\_plat\_err\_t) ifx\_system\_reset\_cfg([void](#))  
*Configures the system reset request properties.*
- enum tfm\_plat\_err\_t [ifx\\_init\\_debug](#) ([void](#))  
*Configures the system debug properties.*
- enum tfm\_plat\_err\_t [ifx\\_init\\_system\\_control\\_block](#) ([void](#))  
*Configures Sleep and Exception Handling (System Control Block).*

## 8.230.1 Function Documentation

### 8.230.1.1 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t)
```

Configures the system reset request properties.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

#### Returns

Returns values as specified by the tfm\_plat\_err\_t

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.  
 Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

#### Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures the system reset request properties.  
 Configures the SAU.  
 Initialize Protection Context switching.  
 Configures MSC.  
 Populate the internal static MPC configuration array.  
 This function enables platform specific faults interrupts.  
 This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.  
 Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)  
 Apply a configuration to assets of a partition.  
 Apply a configuration to specified assets of a partition.  
 Apply PPC protection to named MMIO asset.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.  
 Check memory access permissions using MPC attribute cache.  
 Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

#### Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

#### Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

#### Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

**Parameters**

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)



## Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully.  
 TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset.  
 TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

## Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEM\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.

Check memory access permissions using MPC attribute cache.

This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in [tfm\\_hal\\_status\\_t](#)

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU  
 TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

**Returns**

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

**Returns**

Returns values as specified by the `tfm_plat_err_t`

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

**Parameters**

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

## Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

## Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

## Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

## Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Parameters

|    |                  |                                             |
|----|------------------|---------------------------------------------|
| in | <i>p_assets</i>  | Array of partition's assets                 |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

## Returns

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

## Returns

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

## Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

## Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

## Returns

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                    |
|----|----------------------|------------------------------------|
| in | <i>p_info</i>        | Partition information structure.   |
| in | <i>memory_config</i> | Memory configuration structure.    |
| in | <i>base</i>          | Base address of the memory region. |

**Parameters**

|    |                    |                                                       |
|----|--------------------|-------------------------------------------------------|
| in | <i>size</i>        | Size of the memory region.                            |
| in | <i>access_type</i> | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Configures the system reset request properties.

Configures the SAU.

Initialize Protection Context switching.

Configures MSC.

Populate the internal static MPC configuration array.

This function enables platform specific faults interrupts.

This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.

Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)

Apply a configuration to assets of a partition.

Apply a configuration to specified assets of a partition.

Apply PPC protection to named MMIO asset.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

Check memory access permissions using MPC attribute cache.

Apply MPC configuration using raw interface.

Apply MPC configuration.

Check access to memory region by MPC.

This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

#### Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

**Returns**

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                                          |
|----|----------------|----------------------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the <code>p_asset</code> |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                                      |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

**Parameters**

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

**Parameters**

|    |                  |                                                                         |
|----|------------------|-------------------------------------------------------------------------|
| in | <i>p_info</i>    | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>       | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |
| in | <i>p_assets</i>  | Array of partition's assets                                             |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a>                             |



**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated  
 TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully  
 TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC.  
 TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC.  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.  
Isolate memory region (numbered MMIO) by MPU.  
Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.  
Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

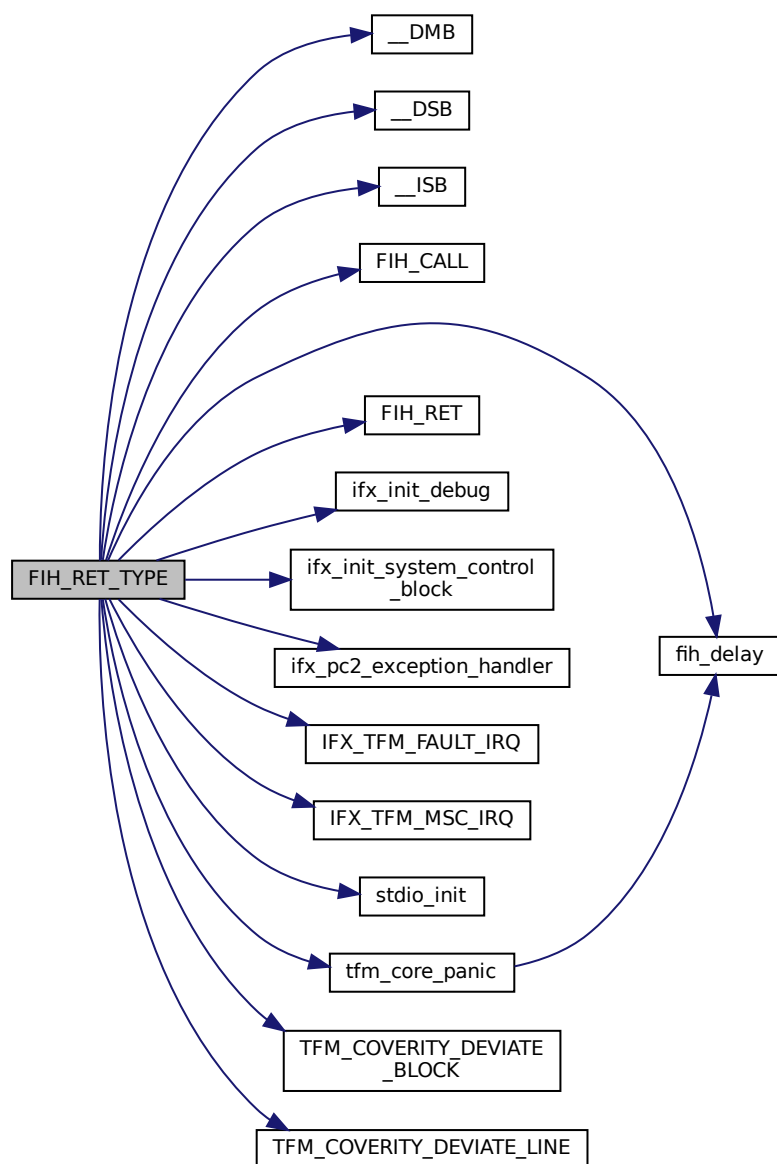
|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

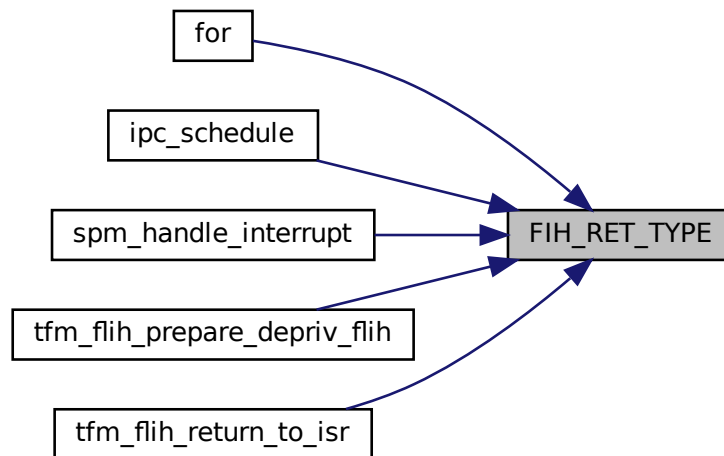
TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 122 of file faults.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.230.1.2 ifx\_init\_debug()

```
enum tfm_plat_err_t ifx_init_debug (
 void)
```

Configures the system debug properties.

##### Returns

Returns values as specified by the `tfm_plat_err_t`

Definition at line 43 of file `target_cfg.c`.

Here is the caller graph for this function:



#### 8.230.1.3 ifx\_init\_system\_control\_block()

```
enum tfm_plat_err_t ifx_init_system_control_block (
 void)
```

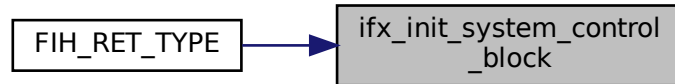
Configures Sleep and Exception Handling (System Control Block).

## Returns

Returns values as specified by the `tfm_plat_err_t`

Definition at line 98 of file `target_cfg.c`.

Here is the caller graph for this function:



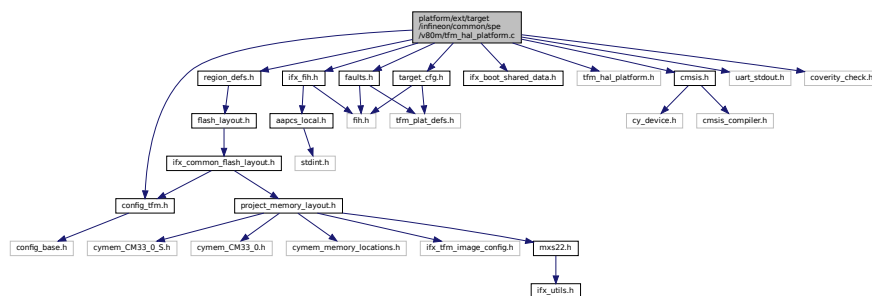
## 8.231 platform/ext/target/infineon/common/spe/v80m/tfm\_hal\_platform.c File Reference

```

#include "config_tfm.h"
#include "faults.h"
#include "ifx_fih.h"
#include "ifx_boot_shared_data.h"
#include "tfm_hal_platform.h"
#include "region_defs.h"
#include "cmsis.h"
#include "target_cfg.h"
#include "uart_stdout.h"
#include "coverity_check.h"

```

Include dependency graph for `tfm_hal_platform.c`:



## Functions

- [FIH\\_RET\\_TYPE](#) (enum `tfm_hal_status_t`) `tfm_hal_platform_init(void)`  
*Configures fault handlers and sets priorities.*

### 8.231.1 Function Documentation

#### 8.231.1.1 FIH\_RET\_TYPE()

```

FIH_RET_TYPE (
 enum tfm_hal_status_t)

```

Configures fault handlers and sets priorities.  
 Configures the system reset request properties.  
 Configures the SAU.  
 Initialize Protection Context switching.  
 Configures MSC.  
 Populate the internal static MPC configuration array.  
 This function enables platform specific faults interrupts.  
 This function enables the faults interrupts (plus the isolation boundary violation interrupts)

#### Returns

Returns values as specified by the `tfm_plat_err_t`

Configures fault handlers and sets priorities.  
 Configures all external interrupts to target the NS state, apart for the ones associated to secure peripherals (plus MPC and PPC)  
 Apply a configuration to assets of a partition.  
 Apply a configuration to specified assets of a partition.  
 Apply PPC protection to named MMIO asset.  
 Isolate memory region (numbered MMIO) by MPU.  
 Disable MPU regions that are reserved for partition assets.  
 Check memory access permissions using MPC attribute cache.  
 Apply MPC configuration using raw interface.  
 Apply MPC configuration.  
 Check access to memory region by MPC.  
 This function enables platform specific faults interrupts.

#### Returns

Returns values as specified by the `tfm_plat_err_t`

#### Parameters

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

#### Returns

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

#### Parameters

|    |                    |                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_region_config_t</a> for more details. |
|----|--------------------|------------------------------------------------------------------------------------------------------------------|

#### Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

## Parameters

|    |                    |                                                                                                                      |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>mpc_reg_cfg</i> | MPC configuration structure which will be applied. See <a href="#">ifx_mpc_raw_region_config_t</a> for more details. |
|----|--------------------|----------------------------------------------------------------------------------------------------------------------|

## Returns

TFM\_HAL\_SUCCESS - MPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - configuration can not be applied. Can occur when MPC that should protect the memory is not controlled by TFM. TFM\_HAL\_ERROR\_GENERIC - failed to apply MPC configuration. TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid input (e.g. provided memory region is not fully located in any known memory).

This function validates access permissions for a memory region using the MPC cache.

## Parameters

|    |                      |                                                       |
|----|----------------------|-------------------------------------------------------|
| in | <i>p_info</i>        | Partition information structure.                      |
| in | <i>memory_config</i> | Memory configuration structure.                       |
| in | <i>base</i>          | Base address of the memory region.                    |
| in | <i>size</i>          | Size of the memory region.                            |
| in | <i>access_type</i>   | Requested access type (read/write/secure/non-secure). |

## Returns

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

A status code as specified in [tfm\\_hal\\_status\\_t](#)

## Parameters

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

## Returns

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

## Parameters

|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

## Returns

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

## Parameters

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Parameters**

|    |                  |                                             |
|----|------------------|---------------------------------------------|
| in | <i>p_assets</i>  | Array of partition's assets                 |
| in | <i>asset_cnt</i> | Number of items in <a href="#">p_assets</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

**Parameters**

|    |               |                                                                         |
|----|---------------|-------------------------------------------------------------------------|
| in | <i>p_info</i> | Pointer to partition info <a href="#">ifx_partition_info_t</a>          |
| in | <i>cfg</i>    | Pointer to configuration <a href="#">ifx_protection_region_config_t</a> |

**Returns**

TFM\_HAL\_SUCCESS - config successfully updated TFM\_HAL\_ERROR\_GENERIC - failed to apply config  
 TFM\_HAL\_ERROR\_INVALID\_INPUT - invalid parameter

Configures fault handlers and sets priorities.  
 Check access to memory region by MPC.

**Returns**

TFM\_HAL\_SUCCESS - static MPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static MPC

**Parameters**

|    |                    |                                                                            |
|----|--------------------|----------------------------------------------------------------------------|
| in | <i>p_info</i>      | Pointer to partition info <a href="#">ifx_partition_info_t</a> .           |
| in | <i>base</i>        | The base address of the region.                                            |
| in | <i>size</i>        | The size of the region.                                                    |
| in | <i>access_type</i> | The memory access types to be checked between given memory and boundaries. |

**Returns**

TFM\_HAL\_SUCCESS - The memory region has the access permissions by MPC. TFM\_HAL\_ERROR\_MEMORY\_FAULT - The memory region has not the access permissions by MPC. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid inputs.

Configures fault handlers and sets priorities.  
 Check memory access permissions using MPC attribute cache.  
 This function sets up the attribute cache of every external memory listed in [ifx\\_mpc\\_sert\\_memories](#) to its reset defaults. The caches are used for memory protection checks.

**Returns**

TFM\_HAL\_SUCCESS on success, error code otherwise.

This function validates access permissions for a memory region using the MPC cache.

**Parameters**

|    |                      |                                    |
|----|----------------------|------------------------------------|
| in | <i>p_info</i>        | Partition information structure.   |
| in | <i>memory_config</i> | Memory configuration structure.    |
| in | <i>base</i>          | Base address of the memory region. |



**Parameters**

|    |                    |                                                       |
|----|--------------------|-------------------------------------------------------|
| in | <i>size</i>        | Size of the memory region.                            |
| in | <i>access_type</i> | Requested access type (read/write/secure/non-secure). |

**Returns**

TFM\_HAL\_SUCCESS - Access is permitted. TFM\_HAL\_ERROR\_MEM\_FAULT - Access is not permitted or invalid input.

Configures fault handlers and sets priorities.

Isolate memory region (numbered MMIO) by MPU.

Disable MPU regions that are reserved for partition assets.

**Returns**

TFM\_HAL\_SUCCESS - Static MPU initialized successfully TFM\_HAL\_ERROR\_INVALID\_INPUT - Failed to init static MPU

A status code as specified in `tfm_hal_status_t`

**Parameters**

|    |                |                                                                                             |
|----|----------------|---------------------------------------------------------------------------------------------|
| in | <i>p_info</i>  | Pointer to partition info <a href="#">ifx_partition_info_t</a> that is owner of the p_asset |
| in | <i>p_asset</i> | Asset that must be protected by MPU                                                         |

**Returns**

TFM\_HAL\_SUCCESS - Numbered MMIO was isolated successfully by MPU TFM\_HAL\_ERROR\_BAD\_STATE - Failed to isolate numbered MMIO by MPU

Configures fault handlers and sets priorities.

Apply PPC protection to named MMIO asset.

**Returns**

TFM\_HAL\_SUCCESS - static PPC initialized successfully TFM\_HAL\_ERROR\_GENERIC - failed to init static PPC

**Parameters**

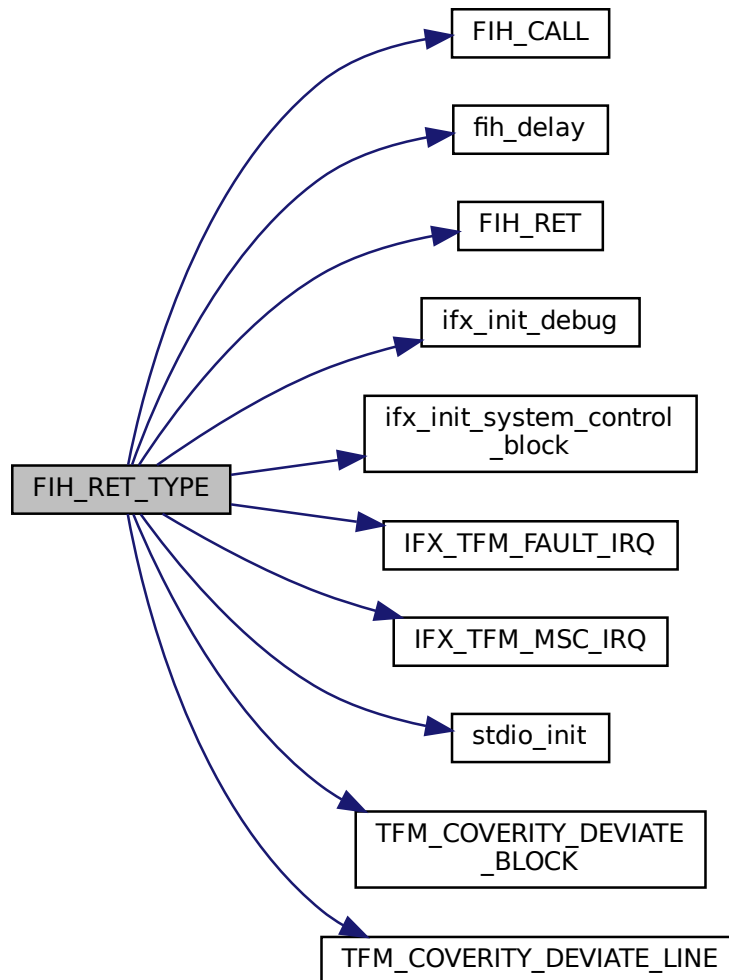
|    |                |                                                                                                                        |
|----|----------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ppc_cfg</i> | PPC configuration settings that should be used. See <a href="#">ifx_ppc_named_mmio_config_t</a> structure for details. |
| in | <i>asset</i>   | Named MMIO asset that should be protected.                                                                             |

**Returns**

TFM\_HAL\_SUCCESS - PPC configuration applied successfully. TFM\_HAL\_ERROR\_NOT\_SUPPORTED - Asset attributes are not supported. TFM\_HAL\_ERROR\_INVALID\_INPUT - Invalid named asset. TFM\_HAL\_ERROR\_GENERIC - Failed to apply PPC configuration.

Definition at line 23 of file tfm\_hal\_platform.c.

Here is the call graph for this function:



## 8.232 platform/ext/target/infineon/common/utlis/ifx\_boot\_shared\_data.c

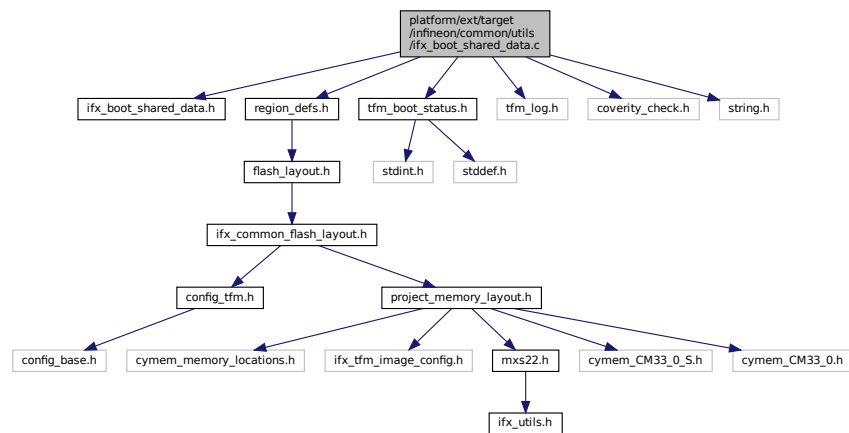
### File Reference

```

#include "ifx_boot_shared_data.h"
#include "region_defs.h"
#include "tfm_boot_status.h"
#include "tfm_log.h"
#include "coverity_check.h"
#include <string.h>

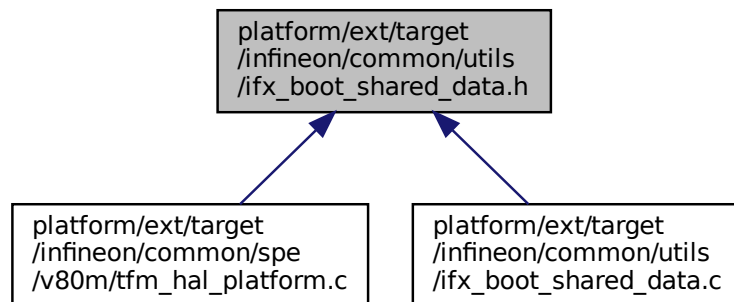
```

Include dependency graph for ifx\_boot\_shared\_data.c:



## 8.233 platform/ext/target/infineon/common/utlis/ifx\_boot\_shared\_data.h File Reference

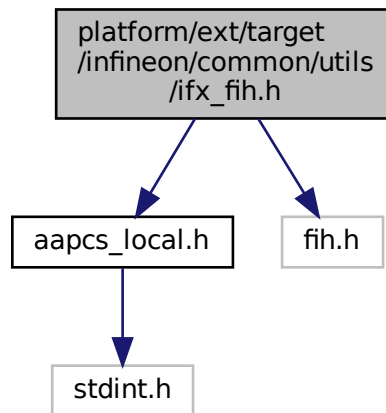
This graph shows which files directly or indirectly include this file:



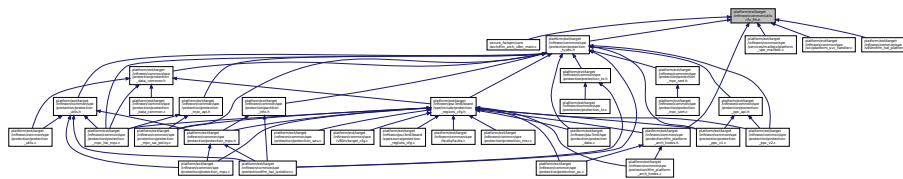
## 8.234 platform/ext/target/infineon/common/utlis/ifx\_fih.h File Reference

```
#include "aapcs_local.h"
#include "fih.h"
```

Include dependency graph for ifx\_fih.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_FIH_TRUE true`
- `#define IFX_FIH_FALSE false`
- `#define IFX_FIH_BOOL bool`
- `#define FIH_INVALID_VALUE (-1)`
- `#define ifx_fih_to_aapcs_fih(x) (x)`
- `#define IFX_FIH_EQ(x, y) ((x) == (y))`
- `#define IFX_FIH_DECLARE(type, var, val) FIH_RET_TYPE(type) FIH_SET(var, (FIH_RET_TYPE(type))(val))`
- `#define FIH_WRITE_REG(reg, set_mask, clr_mask) reg = ((reg) & ~((set_mask) | (clr_mask))) | (set_mask)`
- `#define FIH_SET_REG(reg, mask) FIH_WRITE_REG(reg, mask, 0)`
- `#define FIH_CLEAR_REG(reg, mask) FIH_WRITE_REG(reg, 0, mask)`

## Typedefs

- `typedef uint32_t ifx_aapcs_fih_int`

### 8.234.1 Macro Definition Documentation

#### 8.234.1.1 FIH\_CLEAR\_REG

```
#define FIH_CLEAR_REG(
 reg,
 mask) FIH_WRITE_REG(reg, 0, mask)
```

Definition at line 112 of file ifx\_fih.h.

#### 8.234.1.2 FIH\_INVALID\_VALUE

```
#define FIH_INVALID_VALUE (-1)
```

Definition at line 52 of file ifx\_fih.h.

#### 8.234.1.3 FIH\_SET\_REG

```
#define FIH_SET_REG(
 reg,
 mask) FIH_WRITE_REG(reg, mask, 0)
```

Definition at line 104 of file ifx\_fih.h.

#### 8.234.1.4 FIH\_WRITE\_REG

```
#define FIH_WRITE_REG(
 reg,
 set_mask,
 clr_mask) reg = ((reg) & ~((set_mask) | (clr_mask))) | (set_mask)
```

Definition at line 93 of file ifx\_fih.h.

#### 8.234.1.5 IFX\_FIH\_BOOL

```
#define IFX_FIH_BOOL bool
```

Definition at line 51 of file ifx\_fih.h.

#### 8.234.1.6 IFX\_FIH\_DECLARE

```
#define IFX_FIH_DECLARE(
 type,
 var,
 val) FIH_RET_TYPE(type) FIH_SET(var, (FIH_RET_TYPE(type))(val))
```

Definition at line 64 of file ifx\_fih.h.

#### 8.234.1.7 IFX\_FIH\_EQ

```
#define IFX_FIH_EQ(
 x,
 y) ((x) == (y))
```

Definition at line 60 of file ifx\_fih.h.

#### 8.234.1.8 IFX\_FIH\_FALSE

```
#define IFX_FIH_FALSE false
```

Definition at line 50 of file ifx\_fih.h.

### 8.234.1.9 ifx\_fih\_to\_aapcs\_fih

```
#define ifx_fih_to_aapcs_fih(
 x) (x)
```

Definition at line 58 of file ifx\_fih.h.

### 8.234.1.10 IFX\_FIH\_TRUE

```
#define IFX_FIH_TRUE true
```

Definition at line 49 of file ifx\_fih.h.

## 8.234.2 Typedef Documentation

### 8.234.2.1 ifx\_aapcs\_fih\_int

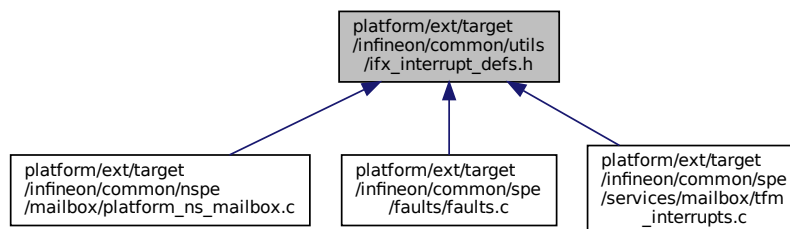
```
typedef uint32_t ifx_aapcs_fih_int
```

Type used to return uint32\_t to assembler code via registers for AAPCS ABI

Definition at line 55 of file ifx\_fih.h.

## 8.235 platform/ext/target/infineon/common/utis/ifx\_interrupt\_defs.h File Reference

This graph shows which files directly or indirectly include this file:



## Macros

- #define [IFX\\_CONCAT](#)(x, y) x##y
- #define [IFX\\_IRQ\\_NAME\\_TO\\_HANDLER](#)(irq) [IFX\\_CONCAT](#)(irq, \_Handler)

## 8.235.1 Macro Definition Documentation

### 8.235.1.1 IFX\_CONCAT

```
#define IFX_CONCAT(
 x,
 y) x##y
```

Definition at line 16 of file ifx\_interrupt\_defs.h.

### 8.235.1.2 IFX\_IRQ\_NAME\_TO\_HANDLER

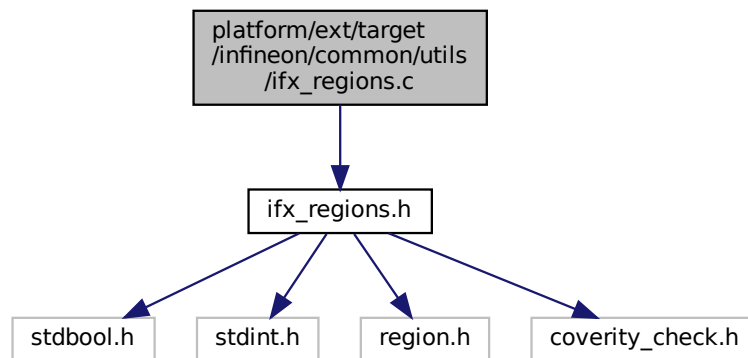
```
#define IFX_IRQ_NAME_TO_HANDLER(
 irq) IFX_CONCAT(irq, _Handler)
```

Definition at line 17 of file ifx\_interrupt\_defs.h.

## 8.236 platform/ext/target/infineon/common/utis/ifx\_regions.c File Reference

```
#include "ifx_regions.h"
```

Include dependency graph for ifx\_regions.c:



## Functions

- bool [ifx\\_is\\_region\\_inside\\_other](#) (uintptr\_t first\_region\_start, uintptr\_t first\_region\_limit, uintptr\_t second\_↵  
region\_start, uintptr\_t second\_region\_limit)  
*Check whether a first memory region is inside second memory region.*
- bool [ifx\\_is\\_region\\_overlap\\_other](#) (uintptr\_t first\_region\_start, uintptr\_t first\_region\_limit, uintptr\_t second\_↵  
region\_start, uintptr\_t second\_region\_limit)  
*Check whether a memory region overlaps other memory region.*

### 8.236.1 Function Documentation

#### 8.236.1.1 ifx\_is\_region\_inside\_other()

```
bool ifx_is_region_inside_other (
 uintptr_t first_region_start,
 uintptr_t first_region_limit,
 uintptr_t second_region_start,
 uintptr_t second_region_limit)
```

Check whether a first memory region is inside second memory region.

#### Parameters

|    |                           |                                       |
|----|---------------------------|---------------------------------------|
| in | <i>first_region_start</i> | The start address of the first region |
| in | <i>first_region_limit</i> | The end address of the first region   |

**Parameters**

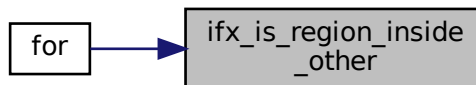
|    |                            |                                                                               |
|----|----------------------------|-------------------------------------------------------------------------------|
| in | <i>second_region_start</i> | The start address of the second region, which should contain the first region |
| in | <i>second_region_limit</i> | The end address of the second region, which should contain the first region   |

**Returns**

true if the second region contains the first region, false otherwise.

Definition at line 11 of file ifx\_regions.c.

Here is the caller graph for this function:

**8.236.1.2 ifx\_is\_region\_overlap\_other()**

```

bool ifx_is_region_overlap_other (
 uintptr_t first_region_start,
 uintptr_t first_region_limit,
 uintptr_t second_region_start,
 uintptr_t second_region_limit)

```

Check whether a memory region overlaps other memory region.

**Parameters**

|    |                            |                                                                               |
|----|----------------------------|-------------------------------------------------------------------------------|
| in | <i>first_region_start</i>  | The start address of the first region                                         |
| in | <i>first_region_limit</i>  | The end address of the first region                                           |
| in | <i>second_region_start</i> | The start address of the second region, which should overlap the first region |
| in | <i>second_region_limit</i> | The end address of the second region, which should overlap the first region   |

**Returns**

true if the second region overlaps the first region, false otherwise.

Definition at line 31 of file ifx\_regions.c.

## 8.237 platform/ext/target/infineon/common/utils/ifx\_regions.h File Reference

```

#include <stdbool.h>
#include <stdint.h>
#include "region.h"
#include "coverity_check.h"

```





**Parameters**

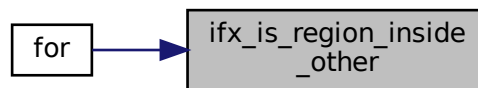
|    |                            |                                                                               |
|----|----------------------------|-------------------------------------------------------------------------------|
| in | <i>first_region_start</i>  | The start address of the first region                                         |
| in | <i>first_region_limit</i>  | The end address of the first region                                           |
| in | <i>second_region_start</i> | The start address of the second region, which should contain the first region |
| in | <i>second_region_limit</i> | The end address of the second region, which should contain the first region   |

**Returns**

true if the second region contains the first region, false otherwise.

Definition at line 11 of file ifx\_regions.c.

Here is the caller graph for this function:

**8.237.1.2 ifx\_is\_region\_overlap\_other()**

```

bool ifx_is_region_overlap_other (
 uintptr_t first_region_start,
 uintptr_t first_region_limit,
 uintptr_t second_region_start,
 uintptr_t second_region_limit)

```

Check whether a memory region overlaps other memory region.

**Parameters**

|    |                            |                                                                               |
|----|----------------------------|-------------------------------------------------------------------------------|
| in | <i>first_region_start</i>  | The start address of the first region                                         |
| in | <i>first_region_limit</i>  | The end address of the first region                                           |
| in | <i>second_region_start</i> | The start address of the second region, which should overlap the first region |
| in | <i>second_region_limit</i> | The end address of the second region, which should overlap the first region   |

**Returns**

true if the second region overlaps the first region, false otherwise.

Definition at line 31 of file ifx\_regions.c.

**8.237.1.3 REGION\_DECLARE() [1/9]**

```

REGION_DECLARE (
 Image$$,
 ARM_LIB_STACK ,
 $)

```

**8.237.1.4 REGION\_DECLARE() [2/9]**

```
REGION_DECLARE (
 Image$$,
 PT_RO_END ,
 $)
```

**8.237.1.5 REGION\_DECLARE() [3/9]**

```
REGION_DECLARE (
 Image$$,
 PT_RO_START ,
 $)
```

**8.237.1.6 REGION\_DECLARE() [4/9]**

```
REGION_DECLARE (
 Image$$,
 TFM_NS_AGENT_TZ_CODE_END ,
 $ RO)
```

**8.237.1.7 REGION\_DECLARE() [5/9]**

```
REGION_DECLARE (
 Image$$,
 TFM_NS_AGENT_TZ_CODE_START ,
 $ RO)
```

**8.237.1.8 REGION\_DECLARE() [6/9]**

```
REGION_DECLARE (
 Image$$,
 TFM_UNPRIV_BASE_CODE_END ,
 $ RO)
```

**8.237.1.9 REGION\_DECLARE() [7/9]**

```
REGION_DECLARE (
 Image$$,
 TFM_UNPRIV_BASE_CODE_START ,
 $ RO)
```

**8.237.1.10 REGION\_DECLARE() [8/9]**

```
REGION_DECLARE (
 Image$$,
 TFM_UNPRIV_CODE_END ,
 $ RO)
```

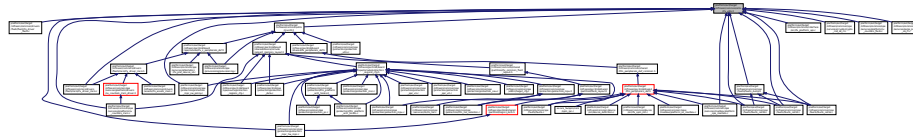
**8.237.1.11 REGION\_DECLARE() [9/9]**

```
REGION_DECLARE (
 Image$$,
```

```
TFM_UNPRIV_CODE_START ,
$ RO)
```

## 8.238 platform/ext/target/infineon/common/utils/ifx\_utils.h File Reference

This graph shows which files directly or indirectly include this file:



### Macros

- `#define IFX_ALIGN_UP_TO(x, align) (((x) % (align)) == 0u) ? (x) : (((x) / (align)) + 1u) * (align))`
- `#define IFX_ASSERT(condition) sizeof(char[!(condition) ? 1 : -1])`
- `#define IFX_UNSIGNED(x) (x ## U)`
- `#define IFX_UNSIGNED_TO_VALUE(x) IFX_UNSIGNED(x)`
- `#define IFX_ROUND_UP_TO_MULTIPLE(number, multiple)`
- `#define IFX_ROUND_DOWN_TO_MULTIPLE(number, multiple) (((number) / (multiple)) * (multiple))`
- `#define IFX_UNUSED(x) ((void)x)`

### 8.238.1 Macro Definition Documentation

#### 8.238.1.1 IFX\_ALIGN\_UP\_TO

```
#define IFX_ALIGN_UP_TO(
 x,
 align) (((x) % (align)) == 0u) ? (x) : (((x) / (align)) + 1u) * (align))
```

Definition at line 15 of file ifx\_utils.h.

#### 8.238.1.2 IFX\_ASSERT

```
#define IFX_ASSERT(
 condition) sizeof(char[!(condition) ? 1 : -1])
```

Definition at line 25 of file ifx\_utils.h.

#### 8.238.1.3 IFX\_ROUND\_DOWN\_TO\_MULTIPLE

```
#define IFX_ROUND_DOWN_TO_MULTIPLE(
 number,
 multiple) (((number) / (multiple)) * (multiple))
```

Definition at line 43 of file ifx\_utils.h.

#### 8.238.1.4 IFX\_ROUND\_UP\_TO\_MULTIPLE

```
#define IFX_ROUND_UP_TO_MULTIPLE(
 number,
 multiple)
```

**Value:**

```
(((number) + (multiple) - 1U) \
 / (multiple)) * (multiple))
```

Definition at line 40 of file ifx\_utils.h.

**8.238.1.5 IFX\_UNSIGNED**

```
#define IFX_UNSIGNED (
 x) (x ## U)
```

Definition at line 33 of file ifx\_utils.h.

**8.238.1.6 IFX\_UNSIGNED\_TO\_VALUE**

```
#define IFX_UNSIGNED_TO_VALUE (
 x) IFX_UNSIGNED(x)
```

Definition at line 37 of file ifx\_utils.h.

**8.238.1.7 IFX\_UNUSED**

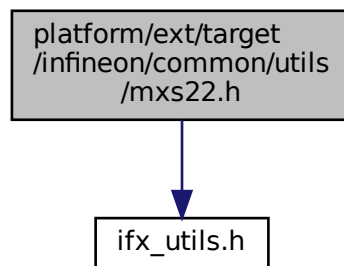
```
#define IFX_UNUSED (
 x) ((void)x)
```

Definition at line 45 of file ifx\_utils.h.

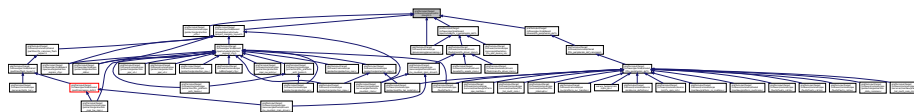
**8.239 platform/ext/target/infineon/common/utls/mxs22.h File Reference**

```
#include "ifx_utils.h"
```

Include dependency graph for mxs22.h:



This graph shows which files directly or indirectly include this file:

**Macros**

- #define IFX\_IDAU\_SECURE\_ADDRESS\_MASK (IFX\_UNSIGNED(0x10000000))

- `#define IFX_S_ADDRESS_ALIAS(x) ((x) | IFX_IDAU_SECURE_ADDRESS_MASK)`
- `#define IFX_NS_ADDRESS_ALIAS(x) ((x) & ~IFX_IDAU_SECURE_ADDRESS_MASK)`
- `#define IFX_S_ADDRESS_ALIAS_T(t, x) ((t)(IFX_S_ADDRESS_ALIAS((uint32_t)(x))))`
- `#define IFX_NS_ADDRESS_ALIAS_T(t, x) ((t)(IFX_NS_ADDRESS_ALIAS((uint32_t)(x))))`

## 8.239.1 Macro Definition Documentation

### 8.239.1.1 IFX\_IDAU\_SECURE\_ADDRESS\_MASK

```
#define IFX_IDAU_SECURE_ADDRESS_MASK (IFX_UNSIGNED(0x10000000))
```

Definition at line 16 of file mxs22.h.

### 8.239.1.2 IFX\_NS\_ADDRESS\_ALIAS

```
#define IFX_NS_ADDRESS_ALIAS(
 x) ((x) & ~IFX_IDAU_SECURE_ADDRESS_MASK)
```

Definition at line 30 of file mxs22.h.

### 8.239.1.3 IFX\_NS\_ADDRESS\_ALIAS\_T

```
#define IFX_NS_ADDRESS_ALIAS_T(
 t,
 x) ((t)(IFX_NS_ADDRESS_ALIAS((uint32_t)(x))))
```

Definition at line 37 of file mxs22.h.

### 8.239.1.4 IFX\_S\_ADDRESS\_ALIAS

```
#define IFX_S_ADDRESS_ALIAS(
 x) ((x) | IFX_IDAU_SECURE_ADDRESS_MASK)
```

Definition at line 27 of file mxs22.h.

### 8.239.1.5 IFX\_S\_ADDRESS\_ALIAS\_T

```
#define IFX_S_ADDRESS_ALIAS_T(
 t,
 x) ((t)(IFX_S_ADDRESS_ALIAS((uint32_t)(x))))
```

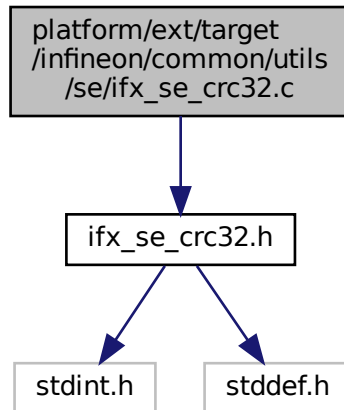
Definition at line 34 of file mxs22.h.

## 8.240 platform/ext/target/infineon/common/utils/se/ifx\_se\_crc32.c File Reference

Fast high-distance 32-bit CRC for data integrity protection.

```
#include "ifx_se_crc32.h"
```

Include dependency graph for ifx\_se\_crc32.c:



## Functions

- [void ifx\\_se\\_crc32d6\\_open](#) (uint32\_t \*P, uint32\_t init)
- [uint32\\_t ifx\\_se\\_crc32d6\\_close](#) (uint32\_t \*P)
- [void ifx\\_se\\_crc32d6a\\_update](#) (uint32\_t \*P, uint8\_t Q)
- [uint32\\_t ifx\\_se\\_crc32d6a](#) (size\_t n, uint8\_t const \*Q, uint32\_t init)
- [void ifx\\_se\\_crc32d6b\\_update](#) (uint32\_t \*P, uint16\_t Q)
- [uint32\\_t ifx\\_se\\_crc32d6b](#) (size\_t n, uint16\_t const \*Q, uint32\_t init)

### 8.240.1 Detailed Description

Fast high-distance 32-bit CRC for data integrity protection.

Version

1.0

### 8.240.2 Function Documentation

#### 8.240.2.1 ifx\_se\_crc32d6\_close()

```
uint32_t ifx_se_crc32d6_close (
 uint32_t * P)
```

Used to finalize CRC state P and return 32 bit digest.

Parameters

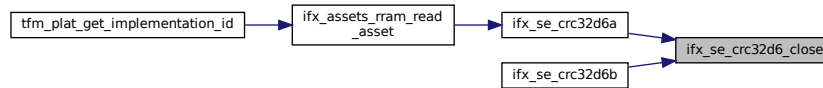
|    |   |                                          |
|----|---|------------------------------------------|
| in | P | The pointer to the CRC32 state variable. |
|----|---|------------------------------------------|

**Returns**

32 bit digest

Definition at line 24 of file ifx\_se\_crc32.c.

Here is the caller graph for this function:

**8.240.2.2 ifx\_se\_crc32d6\_open()**

```

void ifx_se_crc32d6_open (
 uint32_t * P,
 uint32_t init)

```

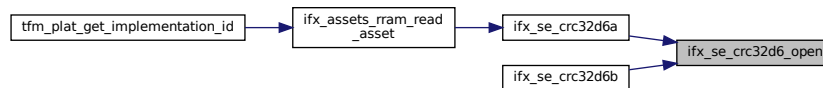
Used to initialize CRC state P.

**Parameters**

|     |             |                                          |
|-----|-------------|------------------------------------------|
| out | <i>P</i>    | The pointer to the CRC32 state variable. |
| in  | <i>init</i> | Initial value for CRC32 calculation      |

Definition at line 19 of file ifx\_se\_crc32.c.

Here is the caller graph for this function:

**8.240.2.3 ifx\_se\_crc32d6a()**

```

uint32_t ifx_se_crc32d6a (
 size_t n,
 uint8_t const * Q,
 uint32_t init)

```

Used to calculate 32 bit digest from the message.

**Parameters**

|    |             |                                      |
|----|-------------|--------------------------------------|
| in | <i>n</i>    | The length of the message.           |
| in | <i>Q</i>    | The pointer to the message.          |
| in | <i>init</i> | Initial value for CRC32 calculation. |

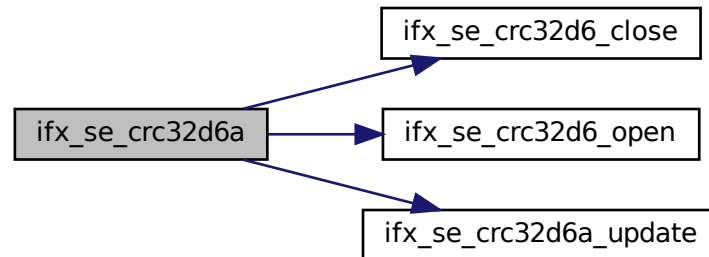


## Returns

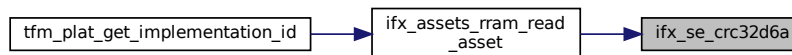
32 bit digest

Definition at line 37 of file ifx\_se\_crc32.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.240.2.4 ifx\_se\_crc32d6a\_update()

```
void ifx_se_crc32d6a_update (
 uint32_t * P,
 uint8_t Q)
```

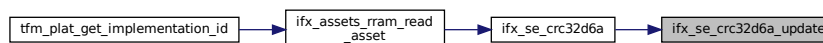
Used to update CRC state P with next message symbol Q.

## Parameters

|         |   |                                          |
|---------|---|------------------------------------------|
| in, out | P | The pointer to the CRC32 state variable. |
| in      | Q | message symbol Q                         |

Definition at line 29 of file ifx\_se\_crc32.c.

Here is the caller graph for this function:



### 8.240.2.5 ifx\_se\_crc32d6b()

```
uint32_t ifx_se_crc32d6b (
 size_t n,
 uint16_t const * Q,
 uint32_t init)
```

Used to calculate 32 bit digest from the 16-bit symbol message.

#### Parameters

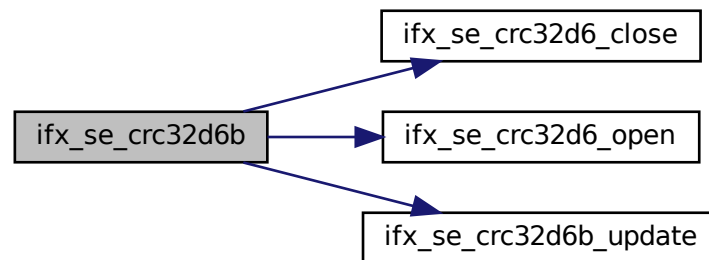
|    |             |                                      |
|----|-------------|--------------------------------------|
| in | <i>n</i>    | The length of the message.           |
| in | <i>Q</i>    | The pointer to the message.          |
| in | <i>init</i> | Initial value for CRC32 calculation. |

#### Returns

32 bit digest

Definition at line 62 of file ifx\_se\_crc32.c.

Here is the call graph for this function:



### 8.240.2.6 ifx\_se\_crc32d6b\_update()

```
void ifx_se_crc32d6b_update (
 uint32_t * P,
 uint16_t Q)
```

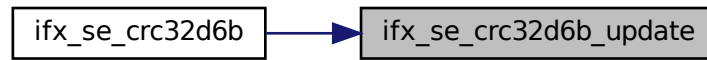
Used to update CRC state P with next 16-bit message symbol Q.

#### Parameters

|         |          |                                          |
|---------|----------|------------------------------------------|
| in, out | <i>P</i> | The pointer to the CRC32 state variable. |
| in      | <i>Q</i> | message symbol Q (16-bit)                |

Definition at line 50 of file ifx\_se\_crc32.c.

Here is the caller graph for this function:



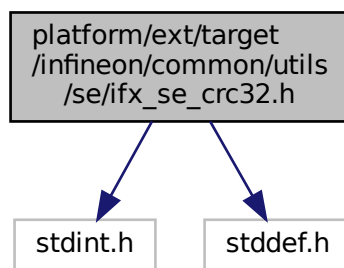
## 8.241 platform/ext/target/infineon/common/utis/se/ifs\_se\_crc32.h File Reference

Fast high-distance 32-bit CRC for data integrity protection.

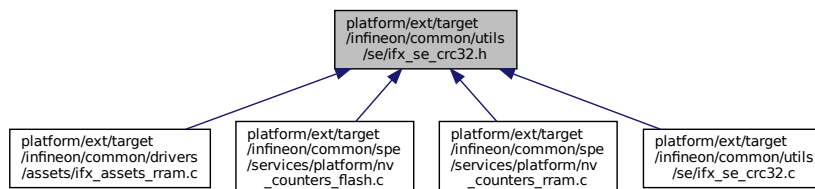
```
#include <stdint.h>
```

```
#include <stddef.h>
```

Include dependency graph for ifx\_se\_crc32.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_CRC32_CRC_WIDTH 32u`
- `#define IFX_CRC32_BYTE_WIDTH (sizeof(uint8_t) * 8u)`
- `#define IFX_CRC32_WORD_WIDTH (sizeof(uint16_t) * 8u)`
- `#define IFX_CRC32_CRC_SIZE (IFX_CRC32_CRC_WIDTH / 8u)`

- `#define IFX_CRC32_INIT` (0xFFFFFFFFuL)
- `#define IFX_CRC32_CALC`(Q, n) `ifx_se_crc32d6a`(n, Q, `IFX_CRC32_INIT`)
- `#define IFX_CRC32_CALC_APPEND`(data, data\_size)

## Functions

- `void ifx_se_crc32d6_open` (uint32\_t \*P, uint32\_t init)
- `uint32_t ifx_se_crc32d6_close` (uint32\_t \*P)
- `void ifx_se_crc32d6a_update` (uint32\_t \*P, uint8\_t Q)
- `uint32_t ifx_se_crc32d6a` (size\_t n, uint8\_t const \*Q, uint32\_t init)
- `void ifx_se_crc32d6b_update` (uint32\_t \*P, uint16\_t Q)
- `uint32_t ifx_se_crc32d6b` (size\_t n, uint16\_t const \*Q, uint32\_t init)

### 8.241.1 Detailed Description

Fast high-distance 32-bit CRC for data integrity protection.

Version

1.0

### 8.241.2 Macro Definition Documentation

#### 8.241.2.1 IFX\_CRC32\_BYTE\_WIDTH

```
#define IFX_CRC32_BYTE_WIDTH (sizeof(uint8_t) * 8u)
```

Definition at line 40 of file `ifx_se_crc32.h`.

#### 8.241.2.2 IFX\_CRC32\_CALC

```
#define IFX_CRC32_CALC(
 Q,
 n) ifx_se_crc32d6a(n, Q, IFX_CRC32_INIT)
```

Definition at line 47 of file `ifx_se_crc32.h`.

#### 8.241.2.3 IFX\_CRC32\_CALC\_APPEND

```
#define IFX_CRC32_CALC_APPEND(
 data,
 data_size)
```

Value:

```
do {
 uint32_t data_crc = IFX_CRC32_CALC((const uint8_t*)(data), (data_size));
 memcpy((uint8_t *) (data) + (data_size), &data_crc, sizeof(data_crc));
} while(0)
```

Calculates and appends CRC32 value to the and of the data block

Parameters

|    |                  |                                            |
|----|------------------|--------------------------------------------|
| in | <i>data</i>      | The pointer to the input data block        |
| in | <i>data_size</i> | The size of actual data to calculate CRC32 |

Note

The data buffer MUST be larger than `data_size` by at least `IFX_CRC32_CRC_SIZE` (4 bytes) to save the CRC32 value at the end of the data block

Definition at line 59 of file ifx\_se\_crc32.h.

#### 8.241.2.4 IFX\_CRC32\_CRC\_SIZE

```
#define IFX_CRC32_CRC_SIZE (IFX_CRC32_CRC_WIDTH / 8u)
```

Definition at line 43 of file ifx\_se\_crc32.h.

#### 8.241.2.5 IFX\_CRC32\_CRC\_WIDTH

```
#define IFX_CRC32_CRC_WIDTH 32u
```

Definition at line 39 of file ifx\_se\_crc32.h.

#### 8.241.2.6 IFX\_CRC32\_INIT

```
#define IFX_CRC32_INIT (0xFFFFFFFFuL)
```

Definition at line 45 of file ifx\_se\_crc32.h.

#### 8.241.2.7 IFX\_CRC32\_WORD\_WIDTH

```
#define IFX_CRC32_WORD_WIDTH (sizeof(uint16_t) * 8u)
```

Definition at line 41 of file ifx\_se\_crc32.h.

### 8.241.3 Function Documentation

#### 8.241.3.1 ifx\_se\_crc32d6\_close()

```
uint32_t ifx_se_crc32d6_close (
 uint32_t * P)
```

Used to finalize CRC state P and return 32 bit digest.

##### Parameters

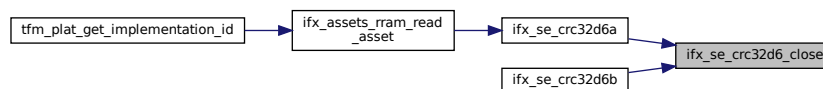
|    |          |                                          |
|----|----------|------------------------------------------|
| in | <i>P</i> | The pointer to the CRC32 state variable. |
|----|----------|------------------------------------------|

##### Returns

32 bit digest

Definition at line 24 of file ifx\_se\_crc32.c.

Here is the caller graph for this function:



#### 8.241.3.2 ifx\_se\_crc32d6\_open()

```
void ifx_se_crc32d6_open (
```

```
uint32_t * P,
uint32_t init)
```

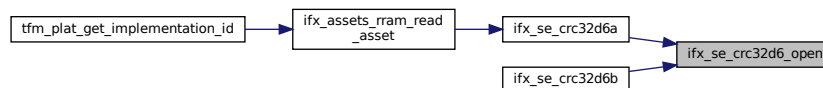
Used to initialize CRC state P.

#### Parameters

|     |             |                                          |
|-----|-------------|------------------------------------------|
| out | <i>P</i>    | The pointer to the CRC32 state variable. |
| in  | <i>init</i> | Initial value for CRC32 calculation      |

Definition at line 19 of file ifx\_se\_crc32.c.

Here is the caller graph for this function:



#### 8.241.3.3 ifx\_se\_crc32d6a()

```
uint32_t ifx_se_crc32d6a (
 size_t n,
 uint8_t const * Q,
 uint32_t init)
```

Used to calculate 32 bit digest from the message.

#### Parameters

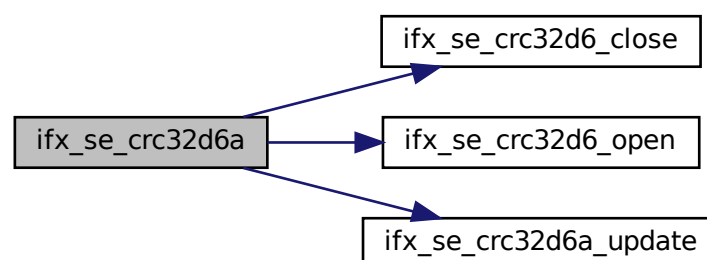
|    |             |                                      |
|----|-------------|--------------------------------------|
| in | <i>n</i>    | The length of the message.           |
| in | <i>Q</i>    | The pointer to the message.          |
| in | <i>init</i> | Initial value for CRC32 calculation. |

#### Returns

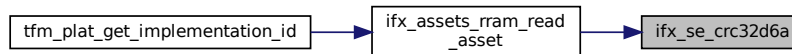
32 bit digest

Definition at line 37 of file ifx\_se\_crc32.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.241.3.4 ifx\_se\_crc32d6a\_update()

```
void ifx_se_crc32d6a_update (
 uint32_t * P,
 uint8_t Q)
```

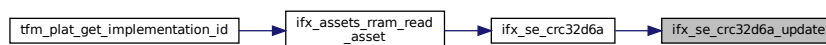
Used to update CRC state P with next message symbol Q.

##### Parameters

|         |          |                                          |
|---------|----------|------------------------------------------|
| in, out | <i>P</i> | The pointer to the CRC32 state variable. |
| in      | <i>Q</i> | message symbol Q                         |

Definition at line 29 of file ifx\_se\_crc32.c.

Here is the caller graph for this function:



#### 8.241.3.5 ifx\_se\_crc32d6b()

```
uint32_t ifx_se_crc32d6b (
 size_t n,
 uint16_t const * Q,
 uint32_t init)
```

Used to calculate 32 bit digest from the 16-bit symbol message.

##### Parameters

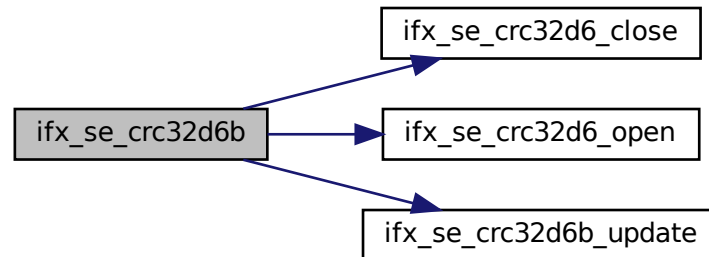
|    |             |                                      |
|----|-------------|--------------------------------------|
| in | <i>n</i>    | The length of the message.           |
| in | <i>Q</i>    | The pointer to the message.          |
| in | <i>init</i> | Initial value for CRC32 calculation. |

**Returns**

32 bit digest

Definition at line 62 of file ifx\_se\_crc32.c.

Here is the call graph for this function:

**8.241.3.6 ifx\_se\_crc32d6b\_update()**

```
void ifx_se_crc32d6b_update (
 uint32_t * P,
 uint16_t Q)
```

Used to update CRC state P with next 16-bit message symbol Q.

**Parameters**

|         |          |                                          |
|---------|----------|------------------------------------------|
| in, out | <i>P</i> | The pointer to the CRC32 state variable. |
| in      | <i>Q</i> | message symbol Q (16-bit)                |

Definition at line 50 of file ifx\_se\_crc32.c.

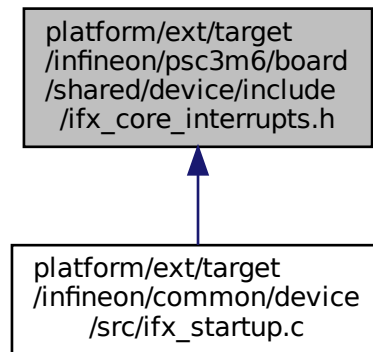
Here is the caller graph for this function:





## 8.242 platform/ext/target/infineon/psc3m6/board/shared/device/include/ifx\_core\_interrupts.h File Reference

This graph shows which files directly or indirectly include this file:



### Macros

- `#define IFX_CORE_DEFINE_INTERRUPTS_LIST`
- `#define IFX_CORE_INTERRUPTS_LIST`

### 8.242.1 Macro Definition Documentation

#### 8.242.1.1 IFX\_CORE\_DEFINE\_INTERRUPTS\_LIST

```
#define IFX_CORE_DEFINE_INTERRUPTS_LIST
```

Definition at line 17 of file ifx\_core\_interrupts.h.

#### 8.242.1.2 IFX\_CORE\_INTERRUPTS\_LIST

```
#define IFX_CORE_INTERRUPTS_LIST
```

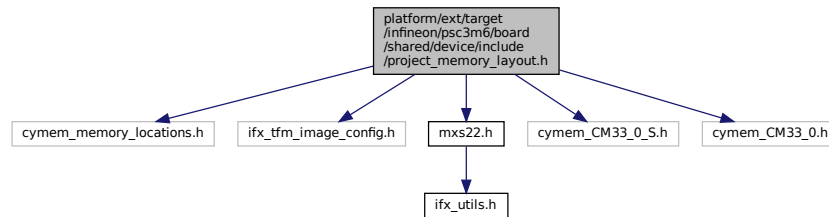
166 [Active] USBFS LOW

Definition at line 186 of file ifx\_core\_interrupts.h.

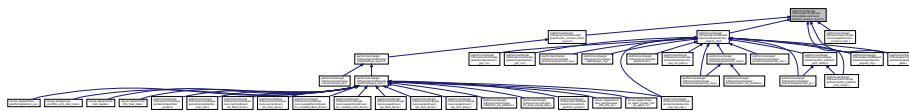
## 8.243 platform/ext/target/infineon/psc3m6/board/shared/device/include/project\_memory\_layout.h File Reference

```
#include "cymem_memory_locations.h"
#include "ifx_tfm_image_config.h"
#include "mxs22.h"
#include "cymem_CM33_0_S.h"
#include "cymem_CM33_0.h"
```

Include dependency graph for project\_memory\_layout.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_FLASH_SBUS_BASE IFX_UNSIGNED(0x22000000)`
- `#define IFX_FLASH_CBUS_BASE IFX_UNSIGNED(0x02000000)`
- `#define IFX_FLASH_SIZE IFX_UNSIGNED(0x00080000)`
- `#define IFX_SRAM0_SBUS_BASE IFX_UNSIGNED(0x24000000)`
- `#define IFX_SRAM0_CBUS_BASE IFX_UNSIGNED(0x04000000)`
- `#define IFX_SRAM0_SIZE IFX_UNSIGNED(0x00010000)`
- `#define IFX_SRAM1_SBUS_BASE IFX_UNSIGNED(0x24010000)`
- `#define IFX_SRAM1_CBUS_BASE IFX_UNSIGNED(0x04010000)`
- `#define IFX_SRAM1_SIZE IFX_UNSIGNED(0x00010000)`
- `#define IFX_TFM_ITS_OFFSET CYMEM_CM33_0_S_TFM_ITS_OFFSET`
- `#define IFX_TFM_ITS_ADDR CYMEM_CM33_0_S_TFM_ITS_S_START`
- `#define IFX_TFM_ITS_SIZE CYMEM_CM33_0_S_TFM_ITS_SIZE`
- `#define IFX_TFM_ITS_LOCATION CYMEM_CM33_0_S_TFM_ITS_LOCATION`
- `#define IFX_TFM_PS_OFFSET CYMEM_CM33_0_S_TFM_PS_OFFSET`
- `#define IFX_TFM_PS_ADDR CYMEM_CM33_0_S_TFM_PS_S_START`
- `#define IFX_TFM_PS_SIZE CYMEM_CM33_0_S_TFM_PS_SIZE`
- `#define IFX_TFM_PS_LOCATION CYMEM_CM33_0_S_TFM_PS_LOCATION`
- `#define IFX_TFM_NV_COUNTERS_AREA_OFFSET CYMEM_CM33_0_S_TFM_NV_COUNTERS_OFFSET`
- `#define IFX_TFM_NV_COUNTERS_AREA_ADDR CYMEM_CM33_0_S_TFM_NV_COUNTERS_S_START`
- `#define IFX_TFM_NV_COUNTERS_AREA_SIZE CYMEM_CM33_0_S_TFM_NV_COUNTERS_SIZE`
- `#define IFX_TFM_NV_COUNTERS_AREA_LOCATION CYMEM_CM33_0_S_TFM_NV_COUNTERS_LOCATION`
- `#define IFX_CM33_NS_IMAGE_EXECUTE_OFFSET CYMEM_CM33_0_m33ns_nvm_OFFSET`
- `#define IFX_CM33_NS_IMAGE_EXECUTE_ADDR CYMEM_CM33_0_m33ns_nvm_C_START`
- `#define IFX_CM33_NS_IMAGE_EXECUTE_SIZE CYMEM_CM33_0_m33ns_nvm_SIZE`
- `#define IFX_CM33_NS_IMAGE_EXECUTE_LOCATION CYMEM_CM33_0_m33ns_nvm_LOCATION`
- `#define IFX_TFM_DATA_OFFSET CYMEM_CM33_0_S_m33_data_OFFSET`
- `#define IFX_TFM_DATA_ADDR CYMEM_CM33_0_S_m33_data_S_START`
- `#define IFX_TFM_DATA_SIZE CYMEM_CM33_0_S_m33_data_SIZE`
- `#define IFX_TFM_DATA_LOCATION CYMEM_CM33_0_S_m33_data_LOCATION`
- `#define IFX_CM33_NS_DATA_OFFSET CYMEM_CM33_0_m33ns_data_OFFSET`

- `#define IFX_CM33_NS_DATA_ADDR CYMEM_CM33_0_m33ns_data_START`
- `#define IFX_CM33_NS_DATA_SIZE CYMEM_CM33_0_m33ns_data_SIZE`
- `#define IFX_CM33_NS_DATA_LOCATION CYMEM_CM33_0_m33ns_data_LOCATION`
- `#define IFX_TFM_BOOT_SHARED_DATA_OFFSET CYMEM_CM33_0_S_m33_shared_OFFSET`
- `#define IFX_TFM_BOOT_SHARED_DATA_ADDR CYMEM_CM33_0_S_m33_shared_S_START`
- `#define IFX_TFM_BOOT_SHARED_DATA_SIZE CYMEM_CM33_0_S_m33_shared_SIZE`
- `#define IFX_TFM_BOOT_SHARED_DATA_LOCATION CYMEM_CM33_0_S_m33_shared_LOCATION`
- `#define IFX_TEST_PROT_PARTITION_MEMORY_OFFSET CYMEM_CM33_0_S_TEST_PROT_PARTIT←  
TION_MEMORY_OFFSET`
- `#define IFX_TEST_PROT_PARTITION_MEMORY_ADDR CYMEM_CM33_0_S_TEST_PROT_PARTITI←  
ON_MEMORY_S_START`
- `#define IFX_TEST_PROT_PARTITION_MEMORY_SIZE CYMEM_CM33_0_S_TEST_PROT_PARTITIO←  
N_MEMORY_SIZE`
- `#define IFX_TEST_PROT_PARTITION_MEMORY_LOCATION CYMEM_CM33_0_S_TEST_PROT_PAR←  
TITION_MEMORY_LOCATION`
- `#define IFX_TEST_AROT_PARTITION_MEMORY_OFFSET CYMEM_CM33_0_S_TEST_AROT_PARTI←  
TION_MEMORY_OFFSET`
- `#define IFX_TEST_AROT_PARTITION_MEMORY_ADDR CYMEM_CM33_0_S_TEST_AROT_PARTITI←  
ON_MEMORY_S_START`
- `#define IFX_TEST_AROT_PARTITION_MEMORY_SIZE CYMEM_CM33_0_S_TEST_AROT_PARTITIO←  
N_MEMORY_SIZE`
- `#define IFX_TEST_AROT_PARTITION_MEMORY_LOCATION CYMEM_CM33_0_S_TEST_AROT_PAR←  
TITION_MEMORY_LOCATION`
- `#define IFX_FF_TEST_SERVER_PARTITION_MMIO_OFFSET CYMEM_CM33_0_S_TEST_PROT_PA←  
RTITION_MEMORY_OFFSET`
- `#define IFX_FF_TEST_SERVER_PARTITION_MMIO_ADDR CYMEM_CM33_0_S_TEST_PROT_PARTI←  
TION_MEMORY_S_START`
- `#define IFX_FF_TEST_SERVER_PARTITION_MMIO_SIZE CYMEM_CM33_0_S_TEST_PROT_PARTIT←  
ION_MEMORY_SIZE`
- `#define IFX_FF_TEST_SERVER_PARTITION_MMIO_LOCATION CYMEM_CM33_0_S_TEST_PROT_P←  
ARTITION_MEMORY_LOCATION`
- `#define IFX_FF_TEST_DRIVER_PARTITION_MMIO_OFFSET CYMEM_CM33_0_S_TEST_AROT_PAR←  
TITION_MEMORY_OFFSET`
- `#define IFX_FF_TEST_DRIVER_PARTITION_MMIO_ADDR CYMEM_CM33_0_S_TEST_AROT_PARTI←  
TION_MEMORY_S_START`
- `#define IFX_FF_TEST_DRIVER_PARTITION_MMIO_SIZE CYMEM_CM33_0_S_TEST_AROT_PARTITI←  
ON_MEMORY_SIZE`
- `#define IFX_FF_TEST_DRIVER_PARTITION_MMIO_LOCATION CYMEM_CM33_0_S_TEST_AROT_P←  
ARTITION_MEMORY_LOCATION`

## 8.243.1 Macro Definition Documentation

### 8.243.1.1 IFX\_CM33\_NS\_DATA\_ADDR

`#define IFX_CM33_NS_DATA_ADDR CYMEM_CM33_0_m33ns_data_START`  
Definition at line 145 of file `project_memory_layout.h`.

### 8.243.1.2 IFX\_CM33\_NS\_DATA\_LOCATION

`#define IFX_CM33_NS_DATA_LOCATION CYMEM_CM33_0_m33ns_data_LOCATION`  
Definition at line 147 of file `project_memory_layout.h`.

#### 8.243.1.3 IFX\_CM33\_NS\_DATA\_OFFSET

```
#define IFX_CM33_NS_DATA_OFFSET CYMEM_CM33_0_m33ns_data_OFFSET
```

Definition at line 144 of file project\_memory\_layout.h.

#### 8.243.1.4 IFX\_CM33\_NS\_DATA\_SIZE

```
#define IFX_CM33_NS_DATA_SIZE CYMEM_CM33_0_m33ns_data_SIZE
```

Definition at line 146 of file project\_memory\_layout.h.

#### 8.243.1.5 IFX\_CM33\_NS\_IMAGE\_EXECUTE\_ADDR

```
#define IFX_CM33_NS_IMAGE_EXECUTE_ADDR CYMEM_CM33_0_m33ns_nvm_C_START
```

Definition at line 126 of file project\_memory\_layout.h.

#### 8.243.1.6 IFX\_CM33\_NS\_IMAGE\_EXECUTE\_LOCATION

```
#define IFX_CM33_NS_IMAGE_EXECUTE_LOCATION CYMEM_CM33_0_m33ns_nvm_LOCATION
```

Definition at line 128 of file project\_memory\_layout.h.

#### 8.243.1.7 IFX\_CM33\_NS\_IMAGE\_EXECUTE\_OFFSET

```
#define IFX_CM33_NS_IMAGE_EXECUTE_OFFSET CYMEM_CM33_0_m33ns_nvm_OFFSET
```

Following macro are generated by Edge Protect solution personality:

- IFX\_TFM\_IMAGE\_EXECUTE\_REGION\_NAME
- IFX\_TFM\_IMAGE\_EXECUTE\_OFFSET
- IFX\_TFM\_IMAGE\_EXECUTE\_ADDR
- IFX\_TFM\_IMAGE\_EXECUTE\_SIZE
- IFX\_TFM\_IMAGE\_EXECUTE\_LOCATION

Definition at line 125 of file project\_memory\_layout.h.

#### 8.243.1.8 IFX\_CM33\_NS\_IMAGE\_EXECUTE\_SIZE

```
#define IFX_CM33_NS_IMAGE_EXECUTE_SIZE CYMEM_CM33_0_m33ns_nvm_SIZE
```

Definition at line 127 of file project\_memory\_layout.h.

#### 8.243.1.9 IFX\_FF\_TEST\_DRIVER\_PARTITION\_MMIO\_ADDR

```
#define IFX_FF_TEST_DRIVER_PARTITION_MMIO_ADDR CYMEM_CM33_0_S_TEST_AROT_PARTITION_MEMORY_S_S←
TART
```

Definition at line 192 of file project\_memory\_layout.h.

#### 8.243.1.10 IFX\_FF\_TEST\_DRIVER\_PARTITION\_MMIO\_LOCATION

```
#define IFX_FF_TEST_DRIVER_PARTITION_MMIO_LOCATION CYMEM_CM33_0_S_TEST_AROT_PARTITION_MEMORY←
_LOCATION
```

Definition at line 194 of file project\_memory\_layout.h.

#### 8.243.1.11 IFX\_FF\_TEST\_DRIVER\_PARTITION\_MMIO\_OFFSET

```
#define IFX_FF_TEST_DRIVER_PARTITION_MMIO_OFFSET CYMEM_CM33_0_S_TEST_AROT_PARTITION_MEMORY_OFFSET
```

Definition at line 191 of file project\_memory\_layout.h.

#### 8.243.1.12 IFX\_FF\_TEST\_DRIVER\_PARTITION\_MMIO\_SIZE

```
#define IFX_FF_TEST_DRIVER_PARTITION_MMIO_SIZE CYMEM_CM33_0_S_TEST_AROT_PARTITION_MEMORY_SIZE
```

Definition at line 193 of file project\_memory\_layout.h.

#### 8.243.1.13 IFX\_FF\_TEST\_SERVER\_PARTITION\_MMIO\_ADDR

```
#define IFX_FF_TEST_SERVER_PARTITION_MMIO_ADDR CYMEM_CM33_0_S_TEST_PROT_PARTITION_MEMORY_START
```

Definition at line 184 of file project\_memory\_layout.h.

#### 8.243.1.14 IFX\_FF\_TEST\_SERVER\_PARTITION\_MMIO\_LOCATION

```
#define IFX_FF_TEST_SERVER_PARTITION_MMIO_LOCATION CYMEM_CM33_0_S_TEST_PROT_PARTITION_MEMORY_LOCATION
```

Definition at line 186 of file project\_memory\_layout.h.

#### 8.243.1.15 IFX\_FF\_TEST\_SERVER\_PARTITION\_MMIO\_OFFSET

```
#define IFX_FF_TEST_SERVER_PARTITION_MMIO_OFFSET CYMEM_CM33_0_S_TEST_PROT_PARTITION_MEMORY_OFFSET
```

Definition at line 183 of file project\_memory\_layout.h.

#### 8.243.1.16 IFX\_FF\_TEST\_SERVER\_PARTITION\_MMIO\_SIZE

```
#define IFX_FF_TEST_SERVER_PARTITION_MMIO_SIZE CYMEM_CM33_0_S_TEST_PROT_PARTITION_MEMORY_SIZE
```

Definition at line 185 of file project\_memory\_layout.h.

#### 8.243.1.17 IFX\_FLASH\_CBUS\_BASE

```
#define IFX_FLASH_CBUS_BASE IFX_UNSIGNED(0x02000000)
```

Definition at line 58 of file project\_memory\_layout.h.

#### 8.243.1.18 IFX\_FLASH\_SBUS\_BASE

```
#define IFX_FLASH_SBUS_BASE IFX_UNSIGNED(0x22000000)
```

Definition at line 57 of file project\_memory\_layout.h.

#### 8.243.1.19 IFX\_FLASH\_SIZE

```
#define IFX_FLASH_SIZE IFX_UNSIGNED(0x00080000)
```

Definition at line 59 of file project\_memory\_layout.h.

#### 8.243.1.20 IFX\_SRAM0\_CBUS\_BASE

```
#define IFX_SRAM0_CBUS_BASE IFX_UNSIGNED(0x04000000)
```

Definition at line 62 of file project\_memory\_layout.h.

#### 8.243.1.21 IFX\_SRAM0\_SBUS\_BASE

```
#define IFX_SRAM0_SBUS_BASE IFX_UNSIGNED(0x24000000)
```

Definition at line 61 of file project\_memory\_layout.h.

#### 8.243.1.22 IFX\_SRAM0\_SIZE

```
#define IFX_SRAM0_SIZE IFX_UNSIGNED(0x00010000)
```

Definition at line 63 of file project\_memory\_layout.h.

#### 8.243.1.23 IFX\_SRAM1\_CBUS\_BASE

```
#define IFX_SRAM1_CBUS_BASE IFX_UNSIGNED(0x04010000)
```

Definition at line 66 of file project\_memory\_layout.h.

#### 8.243.1.24 IFX\_SRAM1\_SBUS\_BASE

```
#define IFX_SRAM1_SBUS_BASE IFX_UNSIGNED(0x24010000)
```

Definition at line 65 of file project\_memory\_layout.h.

#### 8.243.1.25 IFX\_SRAM1\_SIZE

```
#define IFX_SRAM1_SIZE IFX_UNSIGNED(0x00010000)
```

Definition at line 67 of file project\_memory\_layout.h.

#### 8.243.1.26 IFX\_TEST\_AROT\_PARTITION\_MEMORY\_ADDR

```
#define IFX_TEST_AROT_PARTITION_MEMORY_ADDR CYMEM_CM33_0_S_TEST_AROT_PARTITION_MEMORY_S_START
```

Definition at line 176 of file project\_memory\_layout.h.

#### 8.243.1.27 IFX\_TEST\_AROT\_PARTITION\_MEMORY\_LOCATION

```
#define IFX_TEST_AROT_PARTITION_MEMORY_LOCATION CYMEM_CM33_0_S_TEST_AROT_PARTITION_MEMORY_LOCATION
```

Definition at line 178 of file project\_memory\_layout.h.

#### 8.243.1.28 IFX\_TEST\_AROT\_PARTITION\_MEMORY\_OFFSET

```
#define IFX_TEST_AROT_PARTITION_MEMORY_OFFSET CYMEM_CM33_0_S_TEST_AROT_PARTITION_MEMORY_OFFSET
```

Definition at line 175 of file project\_memory\_layout.h.

#### 8.243.1.29 IFX\_TEST\_AROT\_PARTITION\_MEMORY\_SIZE

```
#define IFX_TEST_AROT_PARTITION_MEMORY_SIZE CYMEM_CM33_0_S_TEST_AROT_PARTITION_MEMORY_SIZE
```

Definition at line 177 of file project\_memory\_layout.h.

#### 8.243.1.30 IFX\_TEST\_PROT\_PARTITION\_MEMORY\_ADDR

```
#define IFX_TEST_PROT_PARTITION_MEMORY_ADDR CYMEM_CM33_0_S_TEST_PROT_PARTITION_MEMORY_S_START
```

Definition at line 169 of file project\_memory\_layout.h.

#### 8.243.1.31 IFX\_TEST\_PROT\_PARTITION\_MEMORY\_LOCATION

```
#define IFX_TEST_PROT_PARTITION_MEMORY_LOCATION CYMEM_CM33_0_S_TEST_PROT_PARTITION_MEMORY_LO←
CATION
```

Definition at line 171 of file project\_memory\_layout.h.

#### 8.243.1.32 IFX\_TEST\_PROT\_PARTITION\_MEMORY\_OFFSET

```
#define IFX_TEST_PROT_PARTITION_MEMORY_OFFSET CYMEM_CM33_0_S_TEST_PROT_PARTITION_MEMORY_OFFSET
```

Definition at line 168 of file project\_memory\_layout.h.

#### 8.243.1.33 IFX\_TEST\_PROT\_PARTITION\_MEMORY\_SIZE

```
#define IFX_TEST_PROT_PARTITION_MEMORY_SIZE CYMEM_CM33_0_S_TEST_PROT_PARTITION_MEMORY_SIZE
```

Definition at line 170 of file project\_memory\_layout.h.

#### 8.243.1.34 IFX\_TFM\_BOOT\_SHARED\_DATA\_ADDR

```
#define IFX_TFM_BOOT_SHARED_DATA_ADDR CYMEM_CM33_0_S_m33_shared_S_START
```

Definition at line 157 of file project\_memory\_layout.h.

#### 8.243.1.35 IFX\_TFM\_BOOT\_SHARED\_DATA\_LOCATION

```
#define IFX_TFM_BOOT_SHARED_DATA_LOCATION CYMEM_CM33_0_S_m33_shared_LOCATION
```

Definition at line 159 of file project\_memory\_layout.h.

#### 8.243.1.36 IFX\_TFM\_BOOT\_SHARED\_DATA\_OFFSET

```
#define IFX_TFM_BOOT_SHARED_DATA_OFFSET CYMEM_CM33_0_S_m33_shared_OFFSET
```

Definition at line 156 of file project\_memory\_layout.h.

#### 8.243.1.37 IFX\_TFM\_BOOT\_SHARED\_DATA\_SIZE

```
#define IFX_TFM_BOOT_SHARED_DATA_SIZE CYMEM_CM33_0_S_m33_shared_SIZE
```

Definition at line 158 of file project\_memory\_layout.h.

#### 8.243.1.38 IFX\_TFM\_DATA\_ADDR

```
#define IFX_TFM_DATA_ADDR CYMEM_CM33_0_S_m33_data_S_START
```

Definition at line 138 of file project\_memory\_layout.h.

#### 8.243.1.39 IFX\_TFM\_DATA\_LOCATION

```
#define IFX_TFM_DATA_LOCATION CYMEM_CM33_0_S_m33_data_LOCATION
```

Definition at line 140 of file project\_memory\_layout.h.

#### 8.243.1.40 IFX\_TFM\_DATA\_OFFSET

```
#define IFX_TFM_DATA_OFFSET CYMEM_CM33_0_S_m33_data_OFFSET
```

Definition at line 137 of file project\_memory\_layout.h.

#### 8.243.1.41 IFX\_TFM\_DATA\_SIZE

```
#define IFX_TFM_DATA_SIZE CYMEM_CM33_0_S_m33_data_SIZE
```

Definition at line 139 of file project\_memory\_layout.h.

#### 8.243.1.42 IFX\_TFM\_ITS\_ADDR

```
#define IFX_TFM_ITS_ADDR CYMEM_CM33_0_S_TFM_ITS_S_START
```

Definition at line 89 of file project\_memory\_layout.h.

#### 8.243.1.43 IFX\_TFM\_ITS\_LOCATION

```
#define IFX_TFM_ITS_LOCATION CYMEM_CM33_0_S_TFM_ITS_LOCATION
```

Definition at line 91 of file project\_memory\_layout.h.

#### 8.243.1.44 IFX\_TFM\_ITS\_OFFSET

```
#define IFX_TFM_ITS_OFFSET CYMEM_CM33_0_S_TFM_ITS_OFFSET
```

Definition at line 88 of file project\_memory\_layout.h.

#### 8.243.1.45 IFX\_TFM\_ITS\_SIZE

```
#define IFX_TFM_ITS_SIZE CYMEM_CM33_0_S_TFM_ITS_SIZE
```

Definition at line 90 of file project\_memory\_layout.h.

#### 8.243.1.46 IFX\_TFM\_NV\_COUNTERS\_AREA\_ADDR

```
#define IFX_TFM_NV_COUNTERS_AREA_ADDR CYMEM_CM33_0_S_TFM_NV_COUNTERS_S_START
```

Definition at line 103 of file project\_memory\_layout.h.

#### 8.243.1.47 IFX\_TFM\_NV\_COUNTERS\_AREA\_LOCATION

```
#define IFX_TFM_NV_COUNTERS_AREA_LOCATION CYMEM_CM33_0_S_TFM_NV_COUNTERS_LOCATION
```

Definition at line 105 of file project\_memory\_layout.h.

#### 8.243.1.48 IFX\_TFM\_NV\_COUNTERS\_AREA\_OFFSET

```
#define IFX_TFM_NV_COUNTERS_AREA_OFFSET CYMEM_CM33_0_S_TFM_NV_COUNTERS_OFFSET
```

Definition at line 102 of file project\_memory\_layout.h.

#### 8.243.1.49 IFX\_TFM\_NV\_COUNTERS\_AREA\_SIZE

```
#define IFX_TFM_NV_COUNTERS_AREA_SIZE CYMEM_CM33_0_S_TFM_NV_COUNTERS_SIZE
```

Definition at line 104 of file project\_memory\_layout.h.



#### 8.243.1.50 IFX\_TFM\_PS\_ADDR

```
#define IFX_TFM_PS_ADDR CYMEM_CM33_0_S_TFM_PS_S_START
```

Definition at line 96 of file project\_memory\_layout.h.

#### 8.243.1.51 IFX\_TFM\_PS\_LOCATION

```
#define IFX_TFM_PS_LOCATION CYMEM_CM33_0_S_TFM_PS_LOCATION
```

Definition at line 98 of file project\_memory\_layout.h.

#### 8.243.1.52 IFX\_TFM\_PS\_OFFSET

```
#define IFX_TFM_PS_OFFSET CYMEM_CM33_0_S_TFM_PS_OFFSET
```

Definition at line 95 of file project\_memory\_layout.h.

#### 8.243.1.53 IFX\_TFM\_PS\_SIZE

```
#define IFX_TFM_PS_SIZE CYMEM_CM33_0_S_TFM_PS_SIZE
```

Definition at line 97 of file project\_memory\_layout.h.

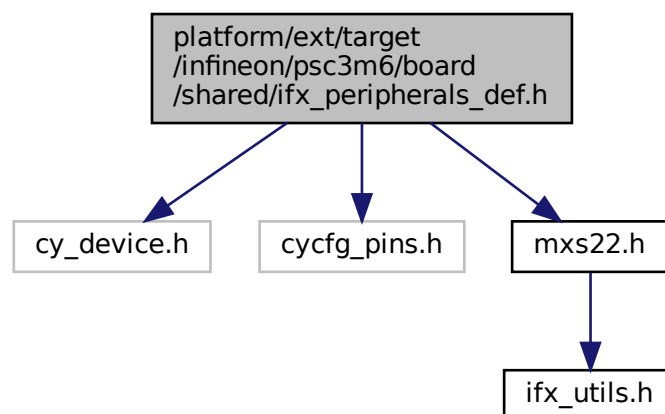
### 8.244 platform/ext/target/infineon/psc3m6/board/shared/ifx\_peripherals\_def.h File Reference

```
#include "cy_device.h"
```

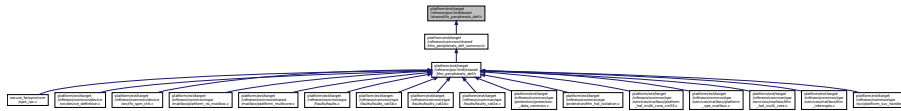
```
#include "cycfg_pins.h"
```

```
#include "mxs22.h"
```

Include dependency graph for ifx\_peripherals\_def.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define TFM_FPU_NS_TEST_IRQ` `ioss_interrupts_gpio_2_IRQn`
- `#define IFX_IRQ_TEST_NS_INTERRUPT` `CYBSP_USER_LED1_IRQ`
- `#define IFX_IRQ_TEST_NS_INTERRUPT_GPIO_PORT`
- `#define IFX_IRQ_TEST_NS_INTERRUPT_GPIO_PIN` `CYBSP_USER_LED1_PIN`
- `#define IFX_IRQ_TEST_FLIH_INTERRUPT` `CYBSP_DEBUG_UART_TX_IRQ`
- `#define IFX_IRQ_TEST_FLIH_INTERRUPT_GPIO_PORT` `CYBSP_DEBUG_UART_TX_PORT`
- `#define IFX_IRQ_TEST_FLIH_INTERRUPT_GPIO_PIN` `CYBSP_DEBUG_UART_TX_PIN`

## 8.244.1 Macro Definition Documentation

### 8.244.1.1 IFX\_IRQ\_TEST\_FLIH\_INTERRUPT

```
#define IFX_IRQ_TEST_FLIH_INTERRUPT CYBSP_DEBUG_UART_TX_IRQ
```

Definition at line 29 of file `ifx_peripherals_def.h`.

### 8.244.1.2 IFX\_IRQ\_TEST\_FLIH\_INTERRUPT\_GPIO\_PIN

```
#define IFX_IRQ_TEST_FLIH_INTERRUPT_GPIO_PIN CYBSP_DEBUG_UART_TX_PIN
```

Definition at line 31 of file `ifx_peripherals_def.h`.

### 8.244.1.3 IFX\_IRQ\_TEST\_FLIH\_INTERRUPT\_GPIO\_PORT

```
#define IFX_IRQ_TEST_FLIH_INTERRUPT_GPIO_PORT CYBSP_DEBUG_UART_TX_PORT
```

Definition at line 30 of file `ifx_peripherals_def.h`.

### 8.244.1.4 IFX\_IRQ\_TEST\_NS\_INTERRUPT

```
#define IFX_IRQ_TEST_NS_INTERRUPT CYBSP_USER_LED1_IRQ
```

Definition at line 24 of file `ifx_peripherals_def.h`.

### 8.244.1.5 IFX\_IRQ\_TEST\_NS\_INTERRUPT\_GPIO\_PIN

```
#define IFX_IRQ_TEST_NS_INTERRUPT_GPIO_PIN CYBSP_USER_LED1_PIN
```

Definition at line 27 of file `ifx_peripherals_def.h`.

### 8.244.1.6 IFX\_IRQ\_TEST\_NS\_INTERRUPT\_GPIO\_PORT

```
#define IFX_IRQ_TEST_NS_INTERRUPT_GPIO_PORT
```

**Value:**

\

```
IFX_NS_ADDRESS_ALIAS_T(GPIO_PRT_Type*,
CYBSP_USER_LED1_PORT)
```

Definition at line 25 of file `ifx_peripherals_def.h`.

## 8.244.1.7 TFM\_FPU\_NS\_TEST\_IRQ

```
#define TFM_FPU_NS_TEST_IRQ ioss_interrupts_gpio_2_IRQn
```

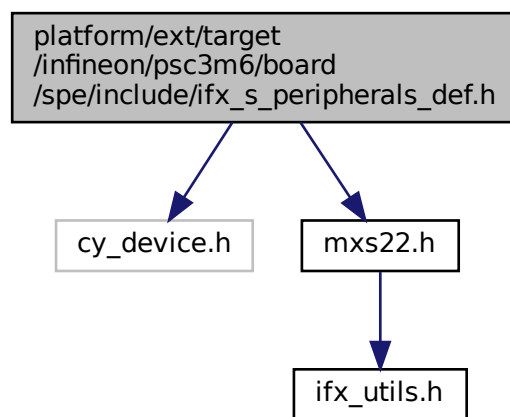
Definition at line 20 of file ifx\_peripherals\_def.h.

## 8.245 platform/ext/target/infineon/psc3m6/board/spe/include/ifx\_s\_peripherals\_def.h File Reference

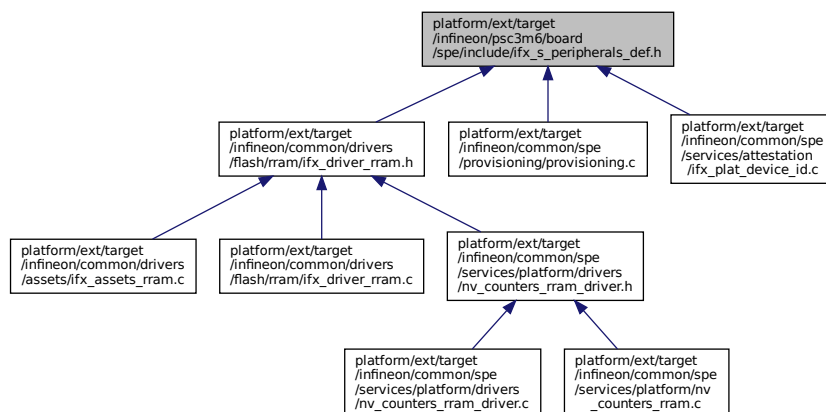
```
#include "cy_device.h"
```

```
#include "mxs22.h"
```

Include dependency graph for ifx\_s\_peripherals\_def.h:



This graph shows which files directly or indirectly include this file:



## Macros

- #define `TFM_FPU_S_TEST_IRQ` `ioss_interrupts_gpio_1_IRQn`
- #define `IFX_SFLASH_IFX_NS_ADDRESS_ALIAS_T` `(SFLASH_Type*, SFLASH)`

- `#define IFX_LCS_BASE IFX_NS_ADDRESS_ALIAS_T(EFUSE_Type*, EFUSE)`
- `#define IFX_TFM_FAULT_STRUCT FAULT_STRUCT0`
- `#define IFX_TFM_FAULT_IRQ cpuss_interrupts_fault_0_IRQn`
- `#define IFX_TFM_MSC_IRQ cpuss_interrupt_msc_IRQn`

## 8.245.1 Macro Definition Documentation

### 8.245.1.1 IFX\_LCS\_BASE

`#define IFX_LCS_BASE IFX_NS_ADDRESS_ALIAS_T(EFUSE_Type*, EFUSE)`  
 Definition at line 28 of file `ifx_s_peripherals_def.h`.

### 8.245.1.2 IFX\_SFLASH

`#define IFX_SFLASH IFX_NS_ADDRESS_ALIAS_T(SFLASH_Type*, SFLASH)`  
 Definition at line 22 of file `ifx_s_peripherals_def.h`.

### 8.245.1.3 IFX\_TFM\_FAULT\_IRQ

`#define IFX_TFM_FAULT_IRQ cpuss_interrupts_fault_0_IRQn`  
 Definition at line 32 of file `ifx_s_peripherals_def.h`.

### 8.245.1.4 IFX\_TFM\_FAULT\_STRUCT

`#define IFX_TFM_FAULT_STRUCT FAULT_STRUCT0`  
 Definition at line 31 of file `ifx_s_peripherals_def.h`.

### 8.245.1.5 IFX\_TFM\_MSC\_IRQ

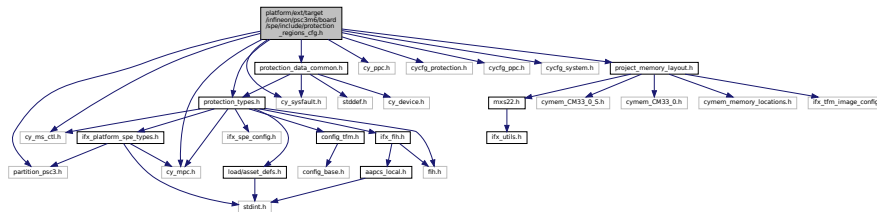
`#define IFX_TFM_MSC_IRQ cpuss_interrupt_msc_IRQn`  
 Definition at line 35 of file `ifx_s_peripherals_def.h`.

### 8.245.1.6 TFM\_FPU\_S\_TEST\_IRQ

`#define TFM_FPU_S_TEST_IRQ ioss_interrupts_gpio_1_IRQn`  
 Definition at line 19 of file `ifx_s_peripherals_def.h`.

## 8.246 platform/ext/target/infineon/psc3m6/board/spe/include/protection\_↵ \_regions\_cfg.h File Reference

```
#include "cy_mpc.h"
#include "cy_ms_ctl.h"
#include "cy_ppc.h"
#include "cy_sysfault.h"
#include "cycfg_protection.h"
#include "cycfg_ppc.h"
#include "cycfg_system.h"
#include "partition_psc3.h"
#include "project_memory_layout.h"
#include "protection_data_common.h"
```

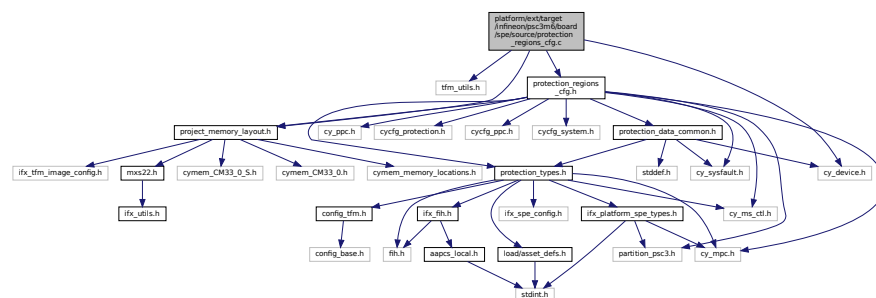
[illegible]

- #define CY\_SAU\_REGION\_CNT ((sizeof(SAU\_config)/sizeof(SAU\_config[0])) - 1u)

### 8.246.1.1 CY\_SAU\_REGION\_CNT

**8.247 platform/ext/target/infineon/p3c3m6/board/spe/source/protection\_↵  
regions\_cfg.c File Reference**

```
#include "tfm_utils.h"
#include "project_memory_layout.h"
#include "cy_device.h"
#include "protection_regions_cfg.h"
Include dependency graph for protection_regions_cfg.c:
```



## Variables

- const [ifx\\_ppcx\\_config\\_t](#) [ifx\\_ppcx\\_static\\_config](#) []
- const size\_t [ifx\\_ppcx\\_static\\_config\\_count](#) = ARRAY\_SIZE([ifx\\_ppcx\\_static\\_config](#))
- const cy\_en\_SysFault\_source\_t [ifx\\_tfm\\_fault\\_sources](#) []
- const size\_t [ifx\\_tfm\\_fault\\_sources\\_count](#) = ARRAY\_SIZE([ifx\\_tfm\\_fault\\_sources](#))

## 8.247.1 Variable Documentation

### 8.247.1.1 ifx\_ppcx\_static\_config

```
const ifx_ppcx_config_t ifx_ppcx_static_config[]
```

**Initial value:**

```
= {
 {
 .configs = cycfg_ppc_0_domains_config,
 .config_count = &cycfg_ppc_0_domains_count,
 .ppc_base = PPC0,
 },
}
```

Definition at line 60 of file protection\_regions\_cfg.c.

### 8.247.1.2 ifx\_ppcx\_static\_config\_count

```
const size_t ifx_ppcx_static_config_count = ARRAY_SIZE(ifx_ppcx_static_config)
```

Definition at line 69 of file protection\_regions\_cfg.c.

### 8.247.1.3 ifx\_tfm\_fault\_sources

```
const cy_en_SysFault_source_t ifx_tfm_fault_sources[]
```

**Initial value:**

```
= {
 PERI_PERI_MS0_PPC_VIO,
 PERI_PERI_MS1_PPC_VIO,
 PERI_PERI_PPC_PC_MASK_VIO,
 PERI_PERI_GP1_TIMEOUT_VIO,
 PERI_PERI_GP2_TIMEOUT_VIO,
 PERI_PERI_GP3_TIMEOUT_VIO,
 PERI_PERI_GP4_TIMEOUT_VIO,
 PERI_PERI_GP5_TIMEOUT_VIO,
 PERI_PERI_GP0_AHB_VIO,
 PERI_PERI_GP1_AHB_VIO,
 PERI_PERI_GP2_AHB_VIO,
 PERI_PERI_GP3_AHB_VIO,
 PERI_PERI_GP4_AHB_VIO,
 PERI_PERI_GP5_AHB_VIO,
 CPUSS_RAMC0_MPC_FAULT_MMIO,
 CPUSS_RAMC1_MPC_FAULT_MMIO,
 CPUSS_PROMC_MPC_FAULT_MMIO,
 CPUSS_FLASHC_MPC_FAULT,
}
```

Definition at line 72 of file protection\_regions\_cfg.c.

### 8.247.1.4 ifx\_tfm\_fault\_sources\_count

```
const size_t ifx_tfm_fault_sources_count = ARRAY_SIZE(ifx_tfm_fault_sources)
```

Definition at line 94 of file protection\_regions\_cfg.c.

## 8.248 platform/ext/target/infineon/p3m6/config/ifx\_platform\_spe\_↵ config.h File Reference

This file contains platform specific configuration used to build secure image.

## Macros

- `#define IFX_PC_TFM_SPM_ID 0x03UL /* TFM SPM */`
- `#define IFX_PC_TFM_PROT_ID 0x03UL /* TFM PSA RoT */`
- `#define IFX_PC_TFM_AROT_ID 0x03UL /* TFM Application RoT */`
- `#define IFX_PC_CM33_NSPE_ID 0x03UL /* CM33 NSPE Application */`
- `#define IFX_PC_TZ_NSPE_ID 0x03UL /* NSPE Application */`
- `#define IFX_PC_DEBUGGER_ID 0x07UL /* Debugger */`
- `#define IFX_PC_NONE (0x00) /* No PC, empty PC mask */`
- `#define IFX_PC_TFM_SPM (1UL << IFX_PC_TFM_SPM_ID)`
- `#define IFX_PC_TFM_PROT (1UL << IFX_PC_TFM_PROT_ID)`
- `#define IFX_PC_TFM_AROT (1UL << IFX_PC_TFM_AROT_ID)`
- `#define IFX_PC_CM33_NSPE (1UL << IFX_PC_CM33_NSPE_ID)`
- `#define IFX_PC_TZ_NSPE (1UL << IFX_PC_TZ_NSPE_ID)`
- `#define IFX_PC_DEBUGGER (1UL << IFX_PC_DEBUGGER_ID)`
- `#define IFX_NOT_ROT_CONFIGURATION (0x00)`
- `#define IFX_REGION_MAX_MPC_COUNT (1U)`
- `#define IFX_PC_DEFAULT IFX_PC_TFM_SPM`
- `#define IFX_MPC_CONFIGURED_BY_TFM 1`
- `#define IFX_PLATFORM_MPC_PRESENT 1`
- `#define IFX_MPC_DRIVER_HW_MPC_WITH_ROT 1`
- `#define IFX_MPC_DRIVER_HW_MPC_WITHOUT_ROT 1`
- `#define IFX_MPC_CM55_MPC 0`
- `#define IFX_MPC_DRIVER_SW_POLICY 0`
- `#define IFX_PLATFORM_PPC_PRESENT 1`
- `#define IFX_MSC_ACG_RESP_CONFIG 1`
- `#define IFX_MSC_ACG_RESP_CONFIG_V1 0`
- `#define IFX_MSC_TFM_CORE_BUS_MASTER_ID CY_MS_CTL_ID_CM33_0`
- `#define IFX_PSA_ROT_PRIVILEGED 1`
- `#define IFX_APP_ROT_PRIVILEGED 0`
- `#define IFX_NS_AGENT_TZ_PRIVILEGED 1`
- `#define IFX_APP_ROT_PPC_DYNAMIC_ISOLATION 1`
- `#define IFX_MULTIPLE_IAK_KEY_TYPES 1`
- `#define IFX_MAX_SPLIT_REGIONS_COUNT (2U)`

### 8.248.1 Detailed Description

This file contains platform specific configuration used to build secure image.

This file is part of Infineon platform configuration files. It's expected that this file provides platform dependent configuration used by Infineon common code.

### 8.248.2 Macro Definition Documentation

#### 8.248.2.1 IFX\_APP\_ROT\_PPC\_DYNAMIC\_ISOLATION

```
#define IFX_APP_ROT_PPC_DYNAMIC_ISOLATION 1
```

Definition at line 85 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.2 IFX\_APP\_ROT\_PRIVILEGED

```
#define IFX_APP_ROT_PRIVILEGED 0
```

Definition at line 79 of file ifx\_platform\_spe\_config.h.

### 8.248.2.3 IFX\_MAX\_SPLIT\_REGIONS\_COUNT

```
#define IFX_MAX_SPLIT_REGIONS_COUNT (2U)
```

Definition at line 92 of file ifx\_platform\_spe\_config.h.

### 8.248.2.4 IFX\_MPC\_CM55\_MPC

```
#define IFX_MPC_CM55_MPC 0
```

Definition at line 58 of file ifx\_platform\_spe\_config.h.

### 8.248.2.5 IFX\_MPC\_CONFIGURED\_BY\_TFM

```
#define IFX_MPC_CONFIGURED_BY_TFM 1
```

Definition at line 46 of file ifx\_platform\_spe\_config.h.

### 8.248.2.6 IFX\_MPC\_DRIVER\_HW\_MPC\_WITH\_ROT

```
#define IFX_MPC_DRIVER_HW_MPC_WITH_ROT 1
```

Definition at line 52 of file ifx\_platform\_spe\_config.h.

### 8.248.2.7 IFX\_MPC\_DRIVER\_HW\_MPC\_WITHOUT\_ROT

```
#define IFX_MPC_DRIVER_HW_MPC_WITHOUT_ROT 1
```

Definition at line 55 of file ifx\_platform\_spe\_config.h.

### 8.248.2.8 IFX\_MPC\_DRIVER\_SW\_POLICY

```
#define IFX_MPC_DRIVER_SW_POLICY 0
```

Definition at line 61 of file ifx\_platform\_spe\_config.h.

### 8.248.2.9 IFX\_MSC\_ACG\_RESP\_CONFIG

```
#define IFX_MSC_ACG_RESP_CONFIG 1
```

Definition at line 67 of file ifx\_platform\_spe\_config.h.

### 8.248.2.10 IFX\_MSC\_ACG\_RESP\_CONFIG\_V1

```
#define IFX_MSC_ACG_RESP_CONFIG_V1 0
```

Definition at line 70 of file ifx\_platform\_spe\_config.h.

### 8.248.2.11 IFX\_MSC\_TFM\_CORE\_BUS\_MASTER\_ID

```
#define IFX_MSC_TFM_CORE_BUS_MASTER_ID CY_MS_CTL_ID_CM33_0
```

Definition at line 73 of file ifx\_platform\_spe\_config.h.

### 8.248.2.12 IFX\_MULTIPLE\_IAK\_KEY\_TYPES

```
#define IFX_MULTIPLE_IAK_KEY_TYPES 1
```

Definition at line 88 of file ifx\_platform\_spe\_config.h.



#### 8.248.2.13 IFX\_NOT\_ROT\_CONFIGURATION

```
#define IFX_NOT_ROT_CONFIGURATION (0x00)
Definition at line 37 of file ifx_platform_spe_config.h.
```

#### 8.248.2.14 IFX\_NS\_AGENT\_TZ\_PRIVILEGED

```
#define IFX_NS_AGENT_TZ_PRIVILEGED 1
Definition at line 82 of file ifx_platform_spe_config.h.
```

#### 8.248.2.15 IFX\_PC\_CM33\_NSPE

```
#define IFX_PC_CM33_NSPE (1UL << IFX_PC_CM33_NSPE_ID)
Definition at line 33 of file ifx_platform_spe_config.h.
```

#### 8.248.2.16 IFX\_PC\_CM33\_NSPE\_ID

```
#define IFX_PC_CM33_NSPE_ID 0x03UL /* CM33 NSPE Application */
Definition at line 25 of file ifx_platform_spe_config.h.
```

#### 8.248.2.17 IFX\_PC\_DEBUGGER

```
#define IFX_PC_DEBUGGER (1UL << IFX_PC_DEBUGGER_ID)
Definition at line 35 of file ifx_platform_spe_config.h.
```

#### 8.248.2.18 IFX\_PC\_DEBUGGER\_ID

```
#define IFX_PC_DEBUGGER_ID 0x07UL /* Debugger */
Definition at line 27 of file ifx_platform_spe_config.h.
```

#### 8.248.2.19 IFX\_PC\_DEFAULT

```
#define IFX_PC_DEFAULT IFX_PC_TFM_SPM
Definition at line 43 of file ifx_platform_spe_config.h.
```

#### 8.248.2.20 IFX\_PC\_NONE

```
#define IFX_PC_NONE (0x00) /* No PC, empty PC mask */
Definition at line 29 of file ifx_platform_spe_config.h.
```

#### 8.248.2.21 IFX\_PC\_TFM\_AROT

```
#define IFX_PC_TFM_AROT (1UL << IFX_PC_TFM_AROT_ID)
Definition at line 32 of file ifx_platform_spe_config.h.
```

#### 8.248.2.22 IFX\_PC\_TFM\_AROT\_ID

```
#define IFX_PC_TFM_AROT_ID 0x03UL /* TFM Application RoT */
Definition at line 24 of file ifx_platform_spe_config.h.
```

#### 8.248.2.23 IFX\_PC\_TFM\_PROT

```
#define IFX_PC_TFM_PROT (1UL << IFX_PC_TFM_PROT_ID)
```

Definition at line 31 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.24 IFX\_PC\_TFM\_PROT\_ID

```
#define IFX_PC_TFM_PROT_ID 0x03UL /* TFM PSA RoT */
```

Definition at line 23 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.25 IFX\_PC\_TFM\_SPM

```
#define IFX_PC_TFM_SPM (1UL << IFX_PC_TFM_SPM_ID)
```

Definition at line 30 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.26 IFX\_PC\_TFM\_SPM\_ID

```
#define IFX_PC_TFM_SPM_ID 0x03UL /* TFM SPM */
```

Definition at line 22 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.27 IFX\_PC\_TZ\_NSPE

```
#define IFX_PC_TZ_NSPE (1UL << IFX_PC_TZ_NSPE_ID)
```

Definition at line 34 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.28 IFX\_PC\_TZ\_NSPE\_ID

```
#define IFX_PC_TZ_NSPE_ID 0x03UL /* NSPE Application */
```

Definition at line 26 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.29 IFX\_PLATFORM\_MPC\_PRESENT

```
#define IFX_PLATFORM_MPC_PRESENT 1
```

Definition at line 49 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.30 IFX\_PLATFORM\_PPC\_PRESENT

```
#define IFX_PLATFORM_PPC_PRESENT 1
```

Definition at line 64 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.31 IFX\_PSA\_ROT\_PRIVILEGED

```
#define IFX_PSA_ROT_PRIVILEGED 1
```

Definition at line 76 of file ifx\_platform\_spe\_config.h.

#### 8.248.2.32 IFX\_REGION\_MAX\_MPC\_COUNT

```
#define IFX_REGION_MAX_MPC_COUNT (1U)
```

Definition at line 39 of file ifx\_platform\_spe\_config.h.

## 8.249 platform/ext/target/infineon/psc3m6/config/ifx\_platform\_spe\_types.h File Reference

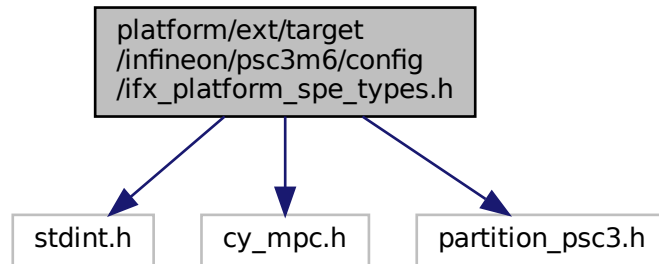
Platform-specific types and data declaration used to build the secure image.

```
#include <stdint.h>
```

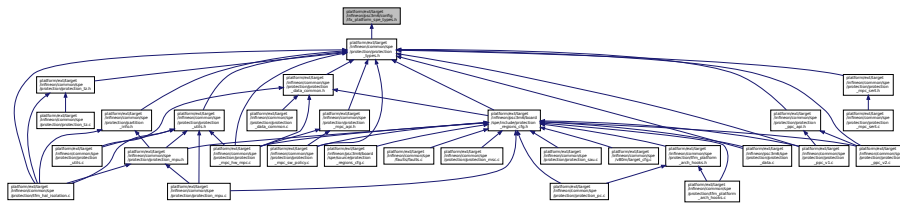
```
#include "cy_mpc.h"
```

```
#include "partition_psc3.h"
```

Include dependency graph for ifx\_platform\_spe\_types.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define IFX_PROTECTION_MPU_PERIPHERAL_REGION_START ((uint32_t)MMIO_NS_START)`
- `#define IFX_PROTECTION_MPU_PERIPHERAL_REGION_LIMIT ((uint32_t)(MMIO_S_START + MMIO_SIZE - 1))`
- `#define IFX_MPC_IS_EXTERNAL(base) false`

### 8.249.1 Detailed Description

Platform-specific types and data declaration used to build the secure image.

This file is part of Infineon platform configuration files. It's expected that this file provides platform dependent types and data declaration used by Infineon common code.

### 8.249.2 Macro Definition Documentation

#### 8.249.2.1 IFX\_MPC\_IS\_EXTERNAL

```
#define IFX_MPC_IS_EXTERNAL(
 base) false
```

Definition at line 31 of file ifx\_platform\_spe\_types.h.

#### 8.249.2.2 IFX\_PROTECTION\_MPU\_PERIPHERAL\_REGION\_LIMIT

```
#define IFX_PROTECTION_MPU_PERIPHERAL_REGION_LIMIT ((uint32_t) (MMIO_S_START + MMIO_SIZE - 1))
```

Definition at line 29 of file ifx\_platform\_spe\_types.h.

#### 8.249.2.3 IFX\_PROTECTION\_MPU\_PERIPHERAL\_REGION\_START

```
#define IFX_PROTECTION_MPU_PERIPHERAL_REGION_START ((uint32_t) MMIO_NS_START)
```

Definition at line 28 of file ifx\_platform\_spe\_types.h.

## 8.250 platform/ext/target/infineon/p3sc3m6/config\_tfm\_target.h File Reference

### Macros

- #define [IFX\\_MEMORY\\_CONFIGURATOR\\_MPC\\_CONFIG](#)
- #define [IFX\\_MEMORY\\_CONFIGURATOR\\_SAU\\_CONFIG](#)
- #define [ITS\\_BUF\\_SIZE](#) ITS\_MAX\_ASSET\_SIZE
- #define [CONFIG\\_TFM\\_NS\\_AGENT\\_TZ\\_STACK\\_SIZE](#) 0x600
- #define [SECURE\\_TEST\\_PARTITION\\_STACK\\_SIZE](#) 0x1200
- #define [ATTEST\\_TOKEN\\_PROFILE\\_PSA\\_2\\_0\\_0](#) 1
- #define [PLATFORM\\_SP\\_STACK\\_SIZE](#) 0x540
- #define [LOG\\_PRIV\\_BUFFER\\_SIZE](#) 64U
- #define [LOG\\_UNPRIV\\_BUFFER\\_SIZE](#) 64U

### 8.250.1 Macro Definition Documentation

#### 8.250.1.1 ATTEST\_TOKEN\_PROFILE\_PSA\_2\_0\_0

```
#define ATTEST_TOKEN_PROFILE_PSA_2_0_0 1
```

Definition at line 78 of file config\_tfm\_target.h.

#### 8.250.1.2 CONFIG\_TFM\_NS\_AGENT\_TZ\_STACK\_SIZE

```
#define CONFIG_TFM_NS_AGENT_TZ_STACK_SIZE 0x600
```

Definition at line 38 of file config\_tfm\_target.h.

#### 8.250.1.3 IFX\_MEMORY\_CONFIGURATOR\_MPC\_CONFIG

```
#define IFX_MEMORY_CONFIGURATOR_MPC_CONFIG
```

Definition at line 23 of file config\_tfm\_target.h.

#### 8.250.1.4 IFX\_MEMORY\_CONFIGURATOR\_SAU\_CONFIG

```
#define IFX_MEMORY_CONFIGURATOR_SAU_CONFIG
```

Definition at line 26 of file config\_tfm\_target.h.

### 8.250.1.5 ITS\_BUF\_SIZE

```
#define ITS_BUF_SIZE ITS_MAX_ASSET_SIZE
```

Definition at line 34 of file config\_tfm\_target.h.

### 8.250.1.6 LOG\_PRIV\_BUFFER\_SIZE

```
#define LOG_PRIV_BUFFER_SIZE 64U
```

Definition at line 88 of file config\_tfm\_target.h.

### 8.250.1.7 LOG\_UNPRIV\_BUFFER\_SIZE

```
#define LOG_UNPRIV_BUFFER_SIZE 64U
```

Definition at line 93 of file config\_tfm\_target.h.

### 8.250.1.8 PLATFORM\_SP\_STACK\_SIZE

```
#define PLATFORM_SP_STACK_SIZE 0x540
```

Definition at line 83 of file config\_tfm\_target.h.

### 8.250.1.9 SECURE\_TEST\_PARTITION\_STACK\_SIZE

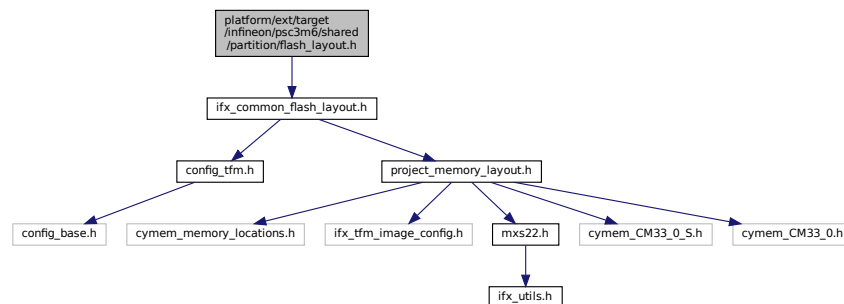
```
#define SECURE_TEST_PARTITION_STACK_SIZE 0x1200
```

Definition at line 42 of file config\_tfm\_target.h.

## 8.251 platform/ext/target/infineon/psc3m6/shared/partition/flash\_layout.h File Reference

```
#include "ifx_common_flash_layout.h"
```

Include dependency graph for flash\_layout.h:



This graph shows which files directly or indirectly include this file:

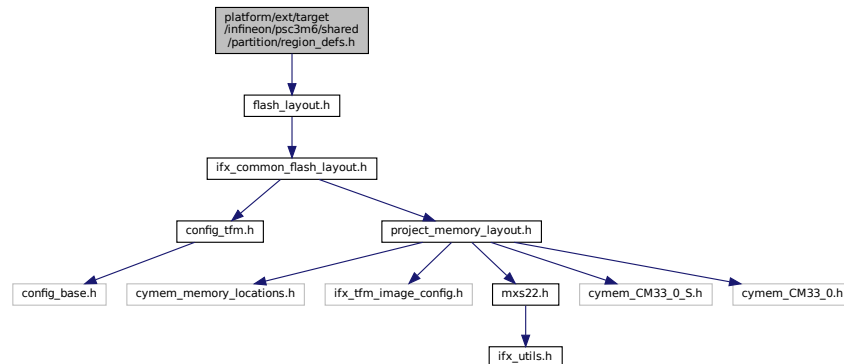


## 8.252 platform/ext/target/infineon/psc3m6/shared/partition/region\_defs.h

### File Reference

```
#include "flash_layout.h"
```

Include dependency graph for region\_defs.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define S_CODE_VECTOR_TABLE_SIZE (0x30C)`
- `#define S_MSP_STACK_SIZE (0x0000800)`
- `#define S_CODE_START (IFX_TFM_IMAGE_EXECUTE_ADDR + IFX_UNSIGNED_TO_VALUE(BL2_HEADER_SIZE))`
- `#define S_CODE_SIZE`
- `#define S_CODE_LIMIT (S_CODE_START + S_CODE_SIZE - 1)`
- `#define S_DATA_START (IFX_TFM_DATA_ADDR)`
- `#define S_DATA_SIZE (IFX_TFM_DATA_SIZE)`
- `#define S_DATA_LIMIT (S_DATA_START + S_DATA_SIZE - 1)`
- `#define IFX_CM33_NS_CODE_START`
- `#define IFX_CM33_NS_CODE_SIZE`
- `#define IFX_CM33_NS_CODE_LIMIT (IFX_CM33_NS_CODE_START + IFX_CM33_NS_CODE_SIZE - 1)`
- `#define IFX_CM33_NS_DATA_LIMIT (IFX_CM33_NS_DATA_ADDR + IFX_CM33_NS_DATA_SIZE - 1)`
- `#define BOOT_TFM_SHARED_DATA_BASE IFX_TFM_BOOT_SHARED_DATA_ADDR`
- `#define BOOT_TFM_SHARED_DATA_SIZE IFX_TFM_BOOT_SHARED_DATA_SIZE`
- `#define BOOT_TFM_SHARED_DATA_LIMIT`
- `#define SHARED_BOOT_MEASUREMENT_BASE BOOT_TFM_SHARED_DATA_BASE`
- `#define SHARED_BOOT_MEASUREMENT_SIZE BOOT_TFM_SHARED_DATA_SIZE`
- `#define SHARED_BOOT_MEASUREMENT_LIMIT BOOT_TFM_SHARED_DATA_LIMIT`

#### 8.252.1 Macro Definition Documentation

### 8.252.1.1 BOOT\_TFM\_SHARED\_DATA\_BASE

```
#define BOOT_TFM_SHARED_DATA_BASE IFX_TFM_BOOT_SHARED_DATA_ADDR
```

Definition at line 62 of file region\_defs.h.

### 8.252.1.2 BOOT\_TFM\_SHARED\_DATA\_LIMIT

```
#define BOOT_TFM_SHARED_DATA_LIMIT
```

**Value:**

```
(BOOT_TFM_SHARED_DATA_BASE + \
BOOT_TFM_SHARED_DATA_SIZE - 1)
```

Definition at line 64 of file region\_defs.h.

### 8.252.1.3 BOOT\_TFM\_SHARED\_DATA\_SIZE

```
#define BOOT_TFM_SHARED_DATA_SIZE IFX_TFM_BOOT_SHARED_DATA_SIZE
```

Definition at line 63 of file region\_defs.h.

### 8.252.1.4 IFX\_CM33\_NS\_CODE\_LIMIT

```
#define IFX_CM33_NS_CODE_LIMIT (IFX_CM33_NS_CODE_START + IFX_CM33_NS_CODE_SIZE - 1)
```

Definition at line 46 of file region\_defs.h.

### 8.252.1.5 IFX\_CM33\_NS\_CODE\_SIZE

```
#define IFX_CM33_NS_CODE_SIZE
```

**Value:**

```
(IFX_CM33_NS_IMAGE_EXECUTE_SIZE \
- IFX_UNSIGNED_TO_VALUE (BL2_HEADER_SIZE) \
- IFX_UNSIGNED_TO_VALUE (BL2_TRAILER_SIZE))
```

Definition at line 43 of file region\_defs.h.

### 8.252.1.6 IFX\_CM33\_NS\_CODE\_START

```
#define IFX_CM33_NS_CODE_START
```

**Value:**

```
(IFX_CM33_NS_IMAGE_EXECUTE_ADDR \
+ IFX_UNSIGNED_TO_VALUE (BL2_HEADER_SIZE))
```

Definition at line 41 of file region\_defs.h.

### 8.252.1.7 IFX\_CM33\_NS\_DATA\_LIMIT

```
#define IFX_CM33_NS_DATA_LIMIT (IFX_CM33_NS_DATA_ADDR + IFX_CM33_NS_DATA_SIZE - 1)
```

Definition at line 48 of file region\_defs.h.

### 8.252.1.8 S\_CODE\_LIMIT

```
#define S_CODE_LIMIT (S_CODE_START + S_CODE_SIZE - 1)
```

Definition at line 34 of file region\_defs.h.

### 8.252.1.9 S\_CODE\_SIZE

```
#define S_CODE_SIZE
```

**Value:**

```
IFX_UNSIGNED_TO_VALUE(BL2_HEADER_SIZE) - \
(IFX_TFM_IMAGE_EXECUTE_SIZE -
IFX_UNSIGNED_TO_VALUE(BL2_TRAILER_SIZE))
```

Definition at line 32 of file region\_defs.h.

#### 8.252.1.10 S\_CODE\_START

```
#define S_CODE_START (IFX_TFM_IMAGE_EXECUTE_ADDR + IFX_UNSIGNED_TO_VALUE(BL2_HEADER_SIZE))
```

Definition at line 31 of file region\_defs.h.

#### 8.252.1.11 S\_CODE\_VECTOR\_TABLE\_SIZE

```
#define S_CODE_VECTOR_TABLE_SIZE (0x30C)
```

Definition at line 19 of file region\_defs.h.

#### 8.252.1.12 S\_DATA\_LIMIT

```
#define S_DATA_LIMIT (S_DATA_START + S_DATA_SIZE - 1)
```

Definition at line 38 of file region\_defs.h.

#### 8.252.1.13 S\_DATA\_SIZE

```
#define S_DATA_SIZE (IFX_TFM_DATA_SIZE)
```

Definition at line 37 of file region\_defs.h.

#### 8.252.1.14 S\_DATA\_START

```
#define S_DATA_START (IFX_TFM_DATA_ADDR)
```

Definition at line 36 of file region\_defs.h.

#### 8.252.1.15 S\_MSP\_STACK\_SIZE

```
#define S_MSP_STACK_SIZE (0x0000800)
```

Definition at line 21 of file region\_defs.h.

#### 8.252.1.16 SHARED\_BOOT\_MEASUREMENT\_BASE

```
#define SHARED_BOOT_MEASUREMENT_BASE BOOT_TFM_SHARED_DATA_BASE
```

Definition at line 66 of file region\_defs.h.

#### 8.252.1.17 SHARED\_BOOT\_MEASUREMENT\_LIMIT

```
#define SHARED_BOOT_MEASUREMENT_LIMIT BOOT_TFM_SHARED_DATA_LIMIT
```

Definition at line 68 of file region\_defs.h.

#### 8.252.1.18 SHARED\_BOOT\_MEASUREMENT\_SIZE

```
#define SHARED_BOOT_MEASUREMENT_SIZE BOOT_TFM_SHARED_DATA_SIZE
```

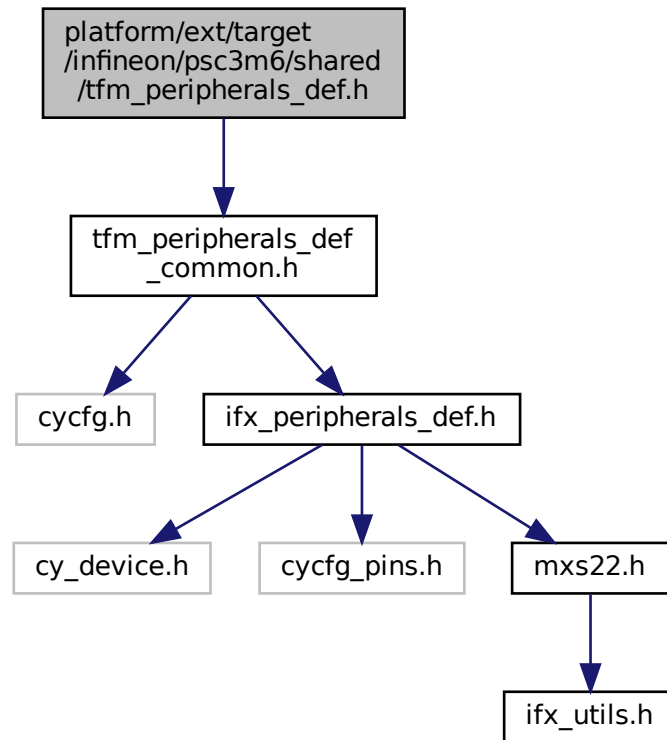
Definition at line 67 of file region\_defs.h.



## 8.253 platform/ext/target/infineon/psc3m6/shared/tfm\_peripherals\_def.h File Reference

```
#include "tfm_peripherals_def_common.h"
```

Include dependency graph for tfm\_peripherals\_def.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define IFX_TEST_PERIPHERAL_1` `PROT_GPIO_PRT0_PRT`
- `#define IFX_TEST_PERIPHERAL_1_BASE` `GPIO_PRT0`
- `#define IFX_TEST_PERIPHERAL_1_SIZE` `0x40U`
- `#define IFX_TEST_PERIPHERAL_2` `PROT_GPIO_PRT1_PRT`
- `#define IFX_TEST_PERIPHERAL_2_BASE` `GPIO_PRT1`
- `#define IFX_TEST_PERIPHERAL_2_SIZE` `0x40U`

### 8.253.1 Macro Definition Documentation

#### 8.253.1.1 IFX\_TEST\_PERIPHERAL\_1

```
#define IFX_TEST_PERIPHERAL_1 PROT_GPIO_PRT0_PRT
```

Definition at line 15 of file tfm\_peripherals\_def.h.

#### 8.253.1.2 IFX\_TEST\_PERIPHERAL\_1\_BASE

```
#define IFX_TEST_PERIPHERAL_1_BASE GPIO_PRT0
```

Definition at line 16 of file tfm\_peripherals\_def.h.

#### 8.253.1.3 IFX\_TEST\_PERIPHERAL\_1\_SIZE

```
#define IFX_TEST_PERIPHERAL_1_SIZE 0x40U
```

Definition at line 17 of file tfm\_peripherals\_def.h.

#### 8.253.1.4 IFX\_TEST\_PERIPHERAL\_2

```
#define IFX_TEST_PERIPHERAL_2 PROT_GPIO_PRT1_PRT
```

Definition at line 20 of file tfm\_peripherals\_def.h.

#### 8.253.1.5 IFX\_TEST\_PERIPHERAL\_2\_BASE

```
#define IFX_TEST_PERIPHERAL_2_BASE GPIO_PRT1
```

Definition at line 21 of file tfm\_peripherals\_def.h.

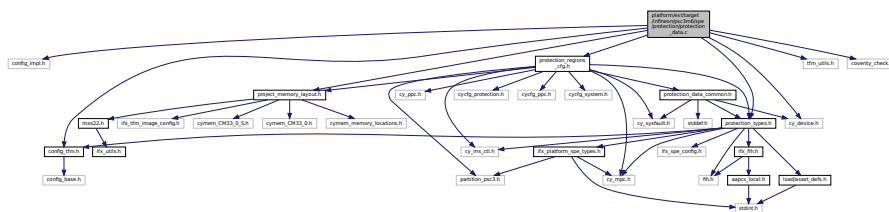
#### 8.253.1.6 IFX\_TEST\_PERIPHERAL\_2\_SIZE

```
#define IFX_TEST_PERIPHERAL_2_SIZE 0x40U
```

Definition at line 22 of file tfm\_peripherals\_def.h.

### 8.254 platform/ext/target/infineon/psc3m6/spe/protection/platform\_[↔](#) partition\_assets.h File Reference

```
#include <stddef.h>
#include "tfm_utils.h"
#include "load/asset_defs.h"
#include "region_defs.h"
```



- `p_asset` mem `start` = `mem_base` + `p_region->offset`
- `p_asset` mem `limit` = `p_asset->mem.start` + `p_region->size`
- `p_asset` `attr` = `IFX_ASSET_ATTR_COMBINE(ASSET_ATTR_NUMBERED_MMIO, ASSET_ATTR_READ_WRITE)`

## 8.255.1 Function Documentation

### 8.255.1.1 FIH\_RET()

```
FIH_RET (
 TFM_HAL_SUCCESS)
```

### 8.255.1.2 FIH\_RET\_TYPE()

```
FIH_RET_TYPE (
 enum tfm_hal_status_t) const
```

### 8.255.1.3 if()

```
else if (
 (void *) p_region-> base == (void*)FLASHC_MPC0)
```

Definition at line 83 of file `protection_data.c`.

### 8.255.1.4 ifx\_platform\_mpc\_is\_rot()

```
bool ifx_platform_mpc_is_rot (
 const MPC_Type * mpc)
```

Definition at line 100 of file `protection_data.c`.

Here is the call graph for this function:



## 8.255.2 Variable Documentation

### 8.255.2.1 attr

```
p_asset attr = IFX_ASSET_ATTR_COMBINE(ASSET_ATTR_NUMBERED_MMIO, ASSET_ATTR_READ_WRITE)
```

Definition at line 96 of file `protection_data.c`.

### 8.255.2.2 else

```
else
```

**Initial value:**

```
{
 FIH_RET(TFM_HAL_ERROR_INVALID_INPUT)
```

Definition at line 89 of file protection\_data.c.

### 8.255.2.3 ifx\_memory\_cm33\_config

```
const ifx_memory_config_t* const ifx_memory_cm33_config[]
```

**Initial value:**

```
= {
 &ifx_sram0_sbus_config,
 &ifx_sram1_sbus_config,
 &ifx_flash0_sbus_config,
 &ifx_sram0_cbus_config,
 &ifx_sram1_cbus_config,
 &ifx_flash0_cbus_config,
}
```

Definition at line 63 of file protection\_data.c.

### 8.255.2.4 ifx\_memory\_cm33\_config\_count

```
const size_t ifx_memory_cm33_config_count = ARRAY_SIZE(ifx_memory_cm33_config)
```

Definition at line 73 of file protection\_data.c.

### 8.255.2.5 limit

```
p_asset mem limit = p_asset->mem.start + p_region->size
```

Definition at line 95 of file protection\_data.c.

### 8.255.2.6 p\_asset

```
struct asset_desc_t* p_asset
```

Definition at line 77 of file protection\_data.c.

### 8.255.2.7 start

```
p_asset mem start = mem_base + p_region->offset
```

Definition at line 94 of file protection\_data.c.

### 8.255.2.8 valid

```
* valid
```

**Initial value:**

```
{
 uint32_t mem_base = 0UL
```

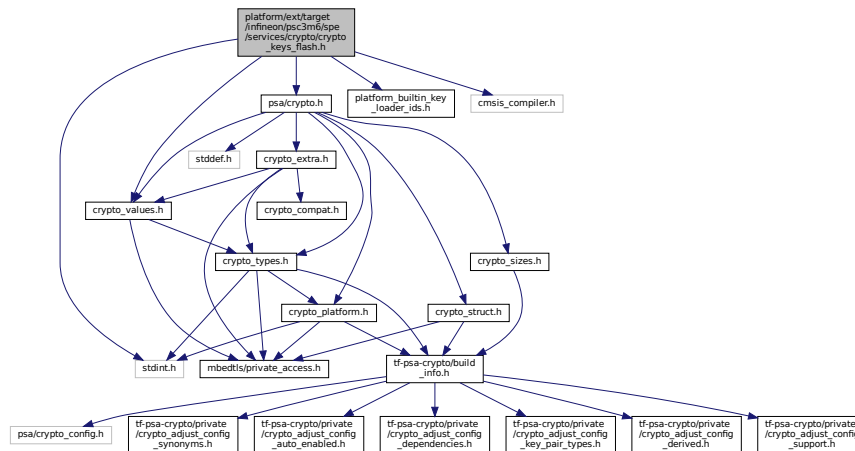
Definition at line 79 of file protection\_data.c.

## 8.256 platform/ext/target/infineon/psoc3m6/spe/services/crypto/crypto\_keys\_flash.h File Reference ↩

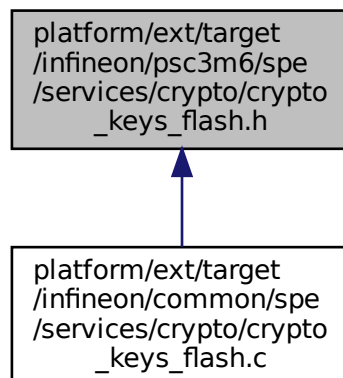
```
#include <stdint.h>
#include "psa/crypto.h"
#include "psa/crypto_values.h"
#include "platform_builtin_key_loader_ids.h"
```

```
#include "cmsis_compiler.h"
```

Include dependency graph for crypto\_keys\_flash.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IFX_IAK_CRYPTOKEYS_TYPE_SYMMETRIC 0xF300000CU /*AES256*/`
- `#define IFX_IAK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP256R1 0xC500003AU /*ECC256*/`
- `#define IFX_IAK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP384R1 0xA6000059U /*ECC384*/`
- `#define IFX_IAK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP521R1 0x9000006FU /*ECC521*/`
- `#define IFX_HUK_CRYPTOKEYS_MAX_LEN 32U`
- `#define IFX_IAK_CRYPTOKEYS_MAX_LEN 66U`
- `#define IFX_CRYPTOKEYS_PTR ((const ifx_plat_keys_t *)0x13407604U)`
- `#define IFX_CRYPTOKEYS_HUK_KEY_BITS 256`
- `#define IFX_CRYPTOKEYS_HUK_KEY_LEN PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE(256u)`
- `#define IFX_CRYPTOKEYS_HUK_ALGORITHM PSA_ALG_HKDF(PSA_ALG_SHA_256)`
- `#define IFX_CRYPTOKEYS_HUK_TYPE PSA_KEY_TYPE_DERIVE`
- `#define IFX_CRYPTOKEYS_IAK_IS_VALID(key_type)`

- `#define IFX_CRYPTO_KEYS_IK_SECP256R1_ALGORITHM PSA_ALG_ECDSA(PSA_ALG_SHA_256)`
- `#define IFX_CRYPTO_KEYS_IK_SECP256R1_TYPE PSA_KEY_TYPE_ECC_KEY_PAIR(PSA_ECC_FAMILY_SECP_R1)`
- `#define IFX_CRYPTO_KEYS_IK_SECP256R1_KEY_BITS 256`
- `#define IFX_CRYPTO_KEYS_IK_SECP256R1_KEY_LEN PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE(256u)`
- `#define IFX_CRYPTO_KEYS_IK_SECP384R1_ALGORITHM PSA_ALG_ECDSA(PSA_ALG_SHA_384)`
- `#define IFX_CRYPTO_KEYS_IK_SECP384R1_TYPE PSA_KEY_TYPE_ECC_KEY_PAIR(PSA_ECC_FAMILY_SECP_R1)`
- `#define IFX_CRYPTO_KEYS_IK_SECP384R1_KEY_BITS 384`
- `#define IFX_CRYPTO_KEYS_IK_SECP384R1_KEY_LEN PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE(384u)`
- `#define IFX_CRYPTO_KEYS_IK_SECP521R1_ALGORITHM PSA_ALG_ECDSA(PSA_ALG_SHA_512)`
- `#define IFX_CRYPTO_KEYS_IK_SECP521R1_TYPE PSA_KEY_TYPE_ECC_KEY_PAIR(PSA_ECC_FAMILY_SECP_R1)`
- `#define IFX_CRYPTO_KEYS_IK_SECP521R1_KEY_BITS 521`
- `#define IFX_CRYPTO_KEYS_IK_SECP521R1_KEY_LEN PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE(521u)`
- `#define IFX_CRYPTO_KEYS_IK_KEY_BITS_BY_TYPE(key_type)`
- `#define IFX_CRYPTO_KEYS_IK_KEY_LEN_BY_TYPE(key_type)`
- `#define IFX_CRYPTO_KEYS_IK_ALGORITHM_BY_TYPE(key_type)`
- `#define IFX_CRYPTO_KEYS_IK_TYPE_BY_TYPE(key_type)`

## Variables

- `typedef __PACKED_STRUCT`
- `uint32_t ik_key_type`
- `uint8_t ik [66U]`
- `ifx_plat_keys_t`

## 8.256.1 Macro Definition Documentation

### 8.256.1.1 IFX\_CRYPTO\_KEYS\_HUK\_ALGORITHM

```
#define IFX_CRYPTO_KEYS_HUK_ALGORITHM PSA_ALG_HKDF(PSA_ALG_SHA_256)
```

Definition at line 46 of file `crypto_keys_flash.h`.

### 8.256.1.2 IFX\_CRYPTO\_KEYS\_HUK\_KEY\_BITS

```
#define IFX_CRYPTO_KEYS_HUK_KEY_BITS 256
```

Definition at line 44 of file `crypto_keys_flash.h`.

### 8.256.1.3 IFX\_CRYPTO\_KEYS\_HUK\_KEY\_LEN

```
#define IFX_CRYPTO_KEYS_HUK_KEY_LEN PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE(256u)
```

Definition at line 45 of file `crypto_keys_flash.h`.

### 8.256.1.4 IFX\_CRYPTO\_KEYS\_HUK\_TYPE

```
#define IFX_CRYPTO_KEYS_HUK_TYPE PSA_KEY_TYPE_DERIVE
```

Definition at line 47 of file `crypto_keys_flash.h`.

**8.256.1.5 IFX\_CRYPT0\_KEYS\_IAK\_ALGORITHM\_BY\_TYPE**

```
#define IFX_CRYPT0_KEYS_IAK_ALGORITHM_BY_TYPE(
 key_type)
```

**Value:**

```
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP256R1) ?
 IFX_CRYPT0_KEYS_IAK_SECP256R1_ALGORITHM : \
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP384R1) ?
 IFX_CRYPT0_KEYS_IAK_SECP384R1_ALGORITHM : \
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP521R1) ?
 IFX_CRYPT0_KEYS_IAK_SECP521R1_ALGORITHM : 0)))
```

Definition at line 94 of file `crypto_keys_flash.h`.

**8.256.1.6 IFX\_CRYPT0\_KEYS\_IAK\_IS\_VALID**

```
#define IFX_CRYPT0_KEYS_IAK_IS_VALID(
 key_type)
```

**Value:**

```
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP256R1 || \
(key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP384R1 || \
(key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP521R1)
```

Definition at line 69 of file `crypto_keys_flash.h`.

**8.256.1.7 IFX\_CRYPT0\_KEYS\_IAK\_KEY\_BITS\_BY\_TYPE**

```
#define IFX_CRYPT0_KEYS_IAK_KEY_BITS_BY_TYPE(
 key_type)
```

**Value:**

```
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP256R1) ? IFX_CRYPT0_KEYS_IAK_SECP256R1_KEY_BITS
: \
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP384R1) ? IFX_CRYPT0_KEYS_IAK_SECP384R1_KEY_BITS
: \
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP521R1) ? IFX_CRYPT0_KEYS_IAK_SECP521R1_KEY_BITS
: 0)))
```

Definition at line 86 of file `crypto_keys_flash.h`.

**8.256.1.8 IFX\_CRYPT0\_KEYS\_IAK\_KEY\_LEN\_BY\_TYPE**

```
#define IFX_CRYPT0_KEYS_IAK_KEY_LEN_BY_TYPE(
 key_type)
```

**Value:**

```
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP256R1) ? IFX_CRYPT0_KEYS_IAK_SECP256R1_KEY_LEN
: \
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP384R1) ? IFX_CRYPT0_KEYS_IAK_SECP384R1_KEY_LEN
: \
((key_type) == IFX_IAK_CRYPT0_KEYS_TYPE_ASYMMETRIC_SECP521R1) ? IFX_CRYPT0_KEYS_IAK_SECP521R1_KEY_LEN
: 0)))
```

Definition at line 90 of file `crypto_keys_flash.h`.

**8.256.1.9 IFX\_CRYPT0\_KEYS\_IAK\_SECP256R1\_ALGORITHM**

```
#define IFX_CRYPT0_KEYS_IAK_SECP256R1_ALGORITHM PSA_ALG_ECDSA(PSA_ALG_SHA_256)
```

Definition at line 73 of file `crypto_keys_flash.h`.

**8.256.1.10 IFX\_CRYPT0\_KEYS\_IAK\_SECP256R1\_KEY\_BITS**

```
#define IFX_CRYPT0_KEYS_IAK_SECP256R1_KEY_BITS 256
```

Definition at line 75 of file `crypto_keys_flash.h`.



**8.256.1.11 IFX\_CRYPTOKEYS\_IK\_SECP256R1\_KEY\_LEN**

```
#define IFX_CRYPTOKEYS_IK_SECP256R1_KEY_LEN PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE (256u)
```

Definition at line 76 of file crypto\_keys\_flash.h.

**8.256.1.12 IFX\_CRYPTOKEYS\_IK\_SECP256R1\_TYPE**

```
#define IFX_CRYPTOKEYS_IK_SECP256R1_TYPE PSA_KEY_TYPE_ECC_KEY_PAIR (PSA_ECC_FAMILY_SECP_R1)
```

Definition at line 74 of file crypto\_keys\_flash.h.

**8.256.1.13 IFX\_CRYPTOKEYS\_IK\_SECP384R1\_ALGORITHM**

```
#define IFX_CRYPTOKEYS_IK_SECP384R1_ALGORITHM PSA_ALG_ECDSA (PSA_ALG_SHA_384)
```

Definition at line 77 of file crypto\_keys\_flash.h.

**8.256.1.14 IFX\_CRYPTOKEYS\_IK\_SECP384R1\_KEY\_BITS**

```
#define IFX_CRYPTOKEYS_IK_SECP384R1_KEY_BITS 384
```

Definition at line 79 of file crypto\_keys\_flash.h.

**8.256.1.15 IFX\_CRYPTOKEYS\_IK\_SECP384R1\_KEY\_LEN**

```
#define IFX_CRYPTOKEYS_IK_SECP384R1_KEY_LEN PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE (384u)
```

Definition at line 80 of file crypto\_keys\_flash.h.

**8.256.1.16 IFX\_CRYPTOKEYS\_IK\_SECP384R1\_TYPE**

```
#define IFX_CRYPTOKEYS_IK_SECP384R1_TYPE PSA_KEY_TYPE_ECC_KEY_PAIR (PSA_ECC_FAMILY_SECP_R1)
```

Definition at line 78 of file crypto\_keys\_flash.h.

**8.256.1.17 IFX\_CRYPTOKEYS\_IK\_SECP521R1\_ALGORITHM**

```
#define IFX_CRYPTOKEYS_IK_SECP521R1_ALGORITHM PSA_ALG_ECDSA (PSA_ALG_SHA_512)
```

Definition at line 81 of file crypto\_keys\_flash.h.

**8.256.1.18 IFX\_CRYPTOKEYS\_IK\_SECP521R1\_KEY\_BITS**

```
#define IFX_CRYPTOKEYS_IK_SECP521R1_KEY_BITS 521
```

Definition at line 83 of file crypto\_keys\_flash.h.

**8.256.1.19 IFX\_CRYPTOKEYS\_IK\_SECP521R1\_KEY\_LEN**

```
#define IFX_CRYPTOKEYS_IK_SECP521R1_KEY_LEN PSA_KEY_EXPORT_ECC_KEY_PAIR_MAX_SIZE (521u)
```

Definition at line 84 of file crypto\_keys\_flash.h.

**8.256.1.20 IFX\_CRYPTOKEYS\_IK\_SECP521R1\_TYPE**

```
#define IFX_CRYPTOKEYS_IK_SECP521R1_TYPE PSA_KEY_TYPE_ECC_KEY_PAIR (PSA_ECC_FAMILY_SECP_R1)
```

Definition at line 82 of file crypto\_keys\_flash.h.

**8.256.1.21 IFX\_CRYPTOKEYS\_IK\_TYPE\_BY\_TYPE**

```
#define IFX_CRYPTOKEYS_IK_TYPE_BY_TYPE(
 key_type)
```

**Value:**

```
((key_type) == IFX_IK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP256R1) ? IFX_CRYPTOKEYS_IK_SECP256R1_TYPE :
 \
((key_type) == IFX_IK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP384R1) ? IFX_CRYPTOKEYS_IK_SECP384R1_TYPE :
 \
((key_type) == IFX_IK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP521R1) ? IFX_CRYPTOKEYS_IK_SECP521R1_TYPE :
 0)))
```

Definition at line 98 of file `crypto_keys_flash.h`.

**8.256.1.22 IFX\_CRYPTOKEYS\_PTR**

```
#define IFX_CRYPTOKEYS_PTR ((const ifx_plat_keys_t *)0x13407604U)
```

Definition at line 42 of file `crypto_keys_flash.h`.

**8.256.1.23 IFX\_HUK\_CRYPTOKEYS\_MAX\_LEN**

```
#define IFX_HUK_CRYPTOKEYS_MAX_LEN 32U
```

Definition at line 24 of file `crypto_keys_flash.h`.

**8.256.1.24 IFX\_IK\_CRYPTOKEYS\_MAX\_LEN**

```
#define IFX_IK_CRYPTOKEYS_MAX_LEN 66U
```

Definition at line 25 of file `crypto_keys_flash.h`.

**8.256.1.25 IFX\_IK\_CRYPTOKEYS\_TYPE\_ASYMMETRIC\_SECP256R1**

```
#define IFX_IK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP256R1 0xC500003AU /*ECC256*/
```

Definition at line 19 of file `crypto_keys_flash.h`.

**8.256.1.26 IFX\_IK\_CRYPTOKEYS\_TYPE\_ASYMMETRIC\_SECP384R1**

```
#define IFX_IK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP384R1 0xA6000059U /*ECC384*/
```

Definition at line 20 of file `crypto_keys_flash.h`.

**8.256.1.27 IFX\_IK\_CRYPTOKEYS\_TYPE\_ASYMMETRIC\_SECP521R1**

```
#define IFX_IK_CRYPTOKEYS_TYPE_ASYMMETRIC_SECP521R1 0x9000006FU /*ECC521*/
```

Definition at line 21 of file `crypto_keys_flash.h`.

**8.256.1.28 IFX\_IK\_CRYPTOKEYS\_TYPE\_SYMMETRIC**

```
#define IFX_IK_CRYPTOKEYS_TYPE_SYMMETRIC 0xF300000CU /*AES256*/
```

Definition at line 18 of file `crypto_keys_flash.h`.

**8.256.2 Variable Documentation**

### 8.256.2.1 \_\_PACKED\_STRUCT

```
typedef __PACKED_STRUCT
```

**Initial value:**

```
{
 uint8_t huk[32U]
}
```

Definition at line 32 of file crypto\_keys\_flash.h.

### 8.256.2.2 iak

```
uint8_t iak[66U]
```

Definition at line 36 of file crypto\_keys\_flash.h.

### 8.256.2.3 iak\_key\_type

```
uint32_t iak_key_type
```

Definition at line 35 of file crypto\_keys\_flash.h.

### 8.256.2.4 ifx\_plat\_keys\_t

```
ifx_plat_keys_t
```

Definition at line 40 of file crypto\_keys\_flash.h.

## 8.257 platform/ext/target/infineon/ps3m6/spe/services/crypto/ix\_pdl\_cryptolite\_config.h File Reference

### Macros

- #define CY\_CRYPTOLITE\_CFG\_SHA\_C
- #define CY\_CRYPTOLITE\_CFG\_SHA2\_256\_ENABLED
- #define CY\_CRYPTOLITE\_CFG\_SHA2\_512\_ENABLED
- #define CY\_CRYPTOLITE\_CFG\_HMAC\_C
- #define CY\_CRYPTOLITE\_CFG\_HKDF\_C
- #define CY\_CRYPTOLITE\_CFG\_TRNG\_C
- #define CY\_CRYPTOLITE\_CFG\_AES\_C
- #define CY\_CRYPTOLITE\_CFG\_CBC\_MAC\_C
- #define CY\_CRYPTOLITE\_CFG\_CMAC\_C
- #define CY\_CRYPTOLITE\_CFG\_CIPHER\_MODE\_CBC
- #define CY\_CRYPTOLITE\_CFG\_CIPHER\_MODE\_CFB
- #define CY\_CRYPTOLITE\_CFG\_CIPHER\_MODE\_CTR
- #define CY\_CRYPTOLITE\_CFG\_CIPHER\_MODE\_CCM
- #define CY\_CRYPTOLITE\_CFG\_ECP\_C
- #define CY\_CRYPTOLITE\_CFG\_ECP\_DP\_SECP256R1\_ENABLED
- #define CY\_CRYPTOLITE\_CFG\_ECP\_DP\_SECP384R1\_ENABLED
- #define CY\_CRYPTOLITE\_CFG\_ECP\_DP\_SECP256K1\_ENABLED
- #define CY\_CRYPTOLITE\_CFG\_ECDSA\_C
- #define CY\_CRYPTOLITE\_CFG\_ECDSA\_SIGN\_C
- #define CY\_CRYPTOLITE\_CFG\_ECDSA\_VERIFY\_C
- #define IFX\_PSA\_CRYPTOLITE\_SHA
- #define IFX\_PSA\_CRYPTOLITE\_SHA\_224
- #define IFX\_PSA\_CRYPTOLITE\_SHA\_256
- #define IFX\_PSA\_CRYPTOLITE\_SHA\_384
- #define IFX\_PSA\_CRYPTOLITE\_SHA\_512
- #define IFX\_PSA\_CRYPTOLITE\_MAC

- `#define IFX_PSA_CRYPTOLITE_HMAC`
- `#define IFX_PSA_CRYPTOLITE_CMAC`
- `#define IFX_PSA_CRYPTOLITE_KEY_DERIVATION`
- `#define IFX_PSA_CRYPTOLITE_HKDF`
- `#define IFX_PSA_CRYPTOLITE_PBKDF2_HMAC`
- `#define IFX_PSA_CRYPTOLITE_TLS12_PRF`
- `#define IFX_PSA_CRYPTOLITE_TLS12_PSK_TO_MS`
- `#define IFX_PSA_CRYPTOLITE_CBC_NO_PADDING`
- `#define IFX_PSA_CRYPTOLITE_CFB`
- `#define IFX_PSA_CRYPTOLITE_CTR`
- `#define IFX_PSA_CRYPTOLITE_AEAD`
- `#define IFX_PSA_CRYPTOLITE_CCM`
- `#define IFX_PSA_CRYPTOLITE_ECC_SECP_R1_256`
- `#define IFX_PSA_CRYPTOLITE_ECDSA_SIGN`
- `#define IFX_PSA_CRYPTOLITE_ECDSA_VERIFY`
- `#define IFX_PSA_CRYPTOLITE_ECDSA_VERIFY_USE_PK`
- `#define IFX_PSA_CRYPTOLITE_ECDH`
- `#define IFX_PSA_CRYPTOLITE_KEY_GENERATION`
- `#define IFX_PSA_CRYPTOLITE_ECC_PUBLIC_KEY_EXPORT`
- `#define IFX_PSA_CRYPTOLITE_PUBLIC_KEY_EXPORT`
- `#define IFX_PSA_CRYPTOLITE_USE_STACK_MEM`

## 8.257.1 Macro Definition Documentation

### 8.257.1.1 CY\_CRYPTOLITE\_CFG\_AES\_C

```
#define CY_CRYPTOLITE_CFG_AES_C
```

Definition at line 29 of file ifx\_pdl\_cryptolite\_config.h.

### 8.257.1.2 CY\_CRYPTOLITE\_CFG\_CBC\_MAC\_C

```
#define CY_CRYPTOLITE_CFG_CBC_MAC_C
```

Definition at line 30 of file ifx\_pdl\_cryptolite\_config.h.

### 8.257.1.3 CY\_CRYPTOLITE\_CFG\_CIPHER\_MODE\_CBC

```
#define CY_CRYPTOLITE_CFG_CIPHER_MODE_CBC
```

Definition at line 34 of file ifx\_pdl\_cryptolite\_config.h.

### 8.257.1.4 CY\_CRYPTOLITE\_CFG\_CIPHER\_MODE\_CCM

```
#define CY_CRYPTOLITE_CFG_CIPHER_MODE_CCM
```

Definition at line 37 of file ifx\_pdl\_cryptolite\_config.h.

### 8.257.1.5 CY\_CRYPTOLITE\_CFG\_CIPHER\_MODE\_CFB

```
#define CY_CRYPTOLITE_CFG_CIPHER_MODE_CFB
```

Definition at line 35 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.6 CY\_CRYPTOLITE\_CFG\_CIPHER\_MODE\_CTR

```
#define CY_CRYPTOLITE_CFG_CIPHER_MODE_CTR
```

Definition at line 36 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.7 CY\_CRYPTOLITE\_CFG\_CMAC\_C

```
#define CY_CRYPTOLITE_CFG_CMAC_C
```

Definition at line 31 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.8 CY\_CRYPTOLITE\_CFG\_ECDSA\_C

```
#define CY_CRYPTOLITE_CFG_ECDSA_C
```

Definition at line 47 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.9 CY\_CRYPTOLITE\_CFG\_ECDSA\_SIGN\_C

```
#define CY_CRYPTOLITE_CFG_ECDSA_SIGN_C
```

Definition at line 48 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.10 CY\_CRYPTOLITE\_CFG\_ECDSA\_VERIFY\_C

```
#define CY_CRYPTOLITE_CFG_ECDSA_VERIFY_C
```

Definition at line 49 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.11 CY\_CRYPTOLITE\_CFG\_ECP\_C

```
#define CY_CRYPTOLITE_CFG_ECP_C
```

Definition at line 41 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.12 CY\_CRYPTOLITE\_CFG\_ECP\_DP\_SECP256K1\_ENABLED

```
#define CY_CRYPTOLITE_CFG_ECP_DP_SECP256K1_ENABLED
```

Definition at line 44 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.13 CY\_CRYPTOLITE\_CFG\_ECP\_DP\_SECP256R1\_ENABLED

```
#define CY_CRYPTOLITE_CFG_ECP_DP_SECP256R1_ENABLED
```

Definition at line 42 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.14 CY\_CRYPTOLITE\_CFG\_ECP\_DP\_SECP384R1\_ENABLED

```
#define CY_CRYPTOLITE_CFG_ECP_DP_SECP384R1_ENABLED
```

Definition at line 43 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.15 CY\_CRYPTOLITE\_CFG\_HKDF\_C

```
#define CY_CRYPTOLITE_CFG_HKDF_C
```

Definition at line 19 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.16 CY\_CRYPTOLITE\_CFG\_HMAC\_C**

```
#define CY_CRYPTOLITE_CFG_HMAC_C
```

Definition at line 18 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.17 CY\_CRYPTOLITE\_CFG\_SHA2\_256\_ENABLED**

```
#define CY_CRYPTOLITE_CFG_SHA2_256_ENABLED
```

Definition at line 14 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.18 CY\_CRYPTOLITE\_CFG\_SHA2\_512\_ENABLED**

```
#define CY_CRYPTOLITE_CFG_SHA2_512_ENABLED
```

Definition at line 15 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.19 CY\_CRYPTOLITE\_CFG\_SHA\_C**

```
#define CY_CRYPTOLITE_CFG_SHA_C
```

Definition at line 13 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.20 CY\_CRYPTOLITE\_CFG\_TRNG\_C**

```
#define CY_CRYPTOLITE_CFG_TRNG_C
```

Definition at line 22 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.21 IFX\_PSA\_CRYPTOLITE\_AEAD**

```
#define IFX_PSA_CRYPTOLITE_AEAD
```

Definition at line 80 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.22 IFX\_PSA\_CRYPTOLITE\_CBC\_NO\_PADDING**

```
#define IFX_PSA_CRYPTOLITE_CBC_NO_PADDING
```

Definition at line 75 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.23 IFX\_PSA\_CRYPTOLITE\_CCM**

```
#define IFX_PSA_CRYPTOLITE_CCM
```

Definition at line 81 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.24 IFX\_PSA\_CRYPTOLITE\_CFB**

```
#define IFX_PSA_CRYPTOLITE_CFB
```

Definition at line 76 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.25 IFX\_PSA\_CRYPTOLITE\_CMAC**

```
#define IFX_PSA_CRYPTOLITE_CMAC
```

Definition at line 62 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.26 IFX\_PSA\_CRYPTOLITE\_CTR

```
#define IFX_PSA_CRYPTOLITE_CTR
```

Definition at line 77 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.27 IFX\_PSA\_CRYPTOLITE\_ECC\_PUBLIC\_KEY\_EXPORT

```
#define IFX_PSA_CRYPTOLITE_ECC_PUBLIC_KEY_EXPORT
```

Definition at line 99 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.28 IFX\_PSA\_CRYPTOLITE\_ECC\_SECP\_R1\_256

```
#define IFX_PSA_CRYPTOLITE_ECC_SECP_R1_256
```

Definition at line 85 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.29 IFX\_PSA\_CRYPTOLITE\_ECDH

```
#define IFX_PSA_CRYPTOLITE_ECDH
```

Definition at line 95 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.30 IFX\_PSA\_CRYPTOLITE\_ECDSA\_SIGN

```
#define IFX_PSA_CRYPTOLITE_ECDSA_SIGN
```

Definition at line 90 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.31 IFX\_PSA\_CRYPTOLITE\_ECDSA\_VERIFY

```
#define IFX_PSA_CRYPTOLITE_ECDSA_VERIFY
```

Definition at line 91 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.32 IFX\_PSA\_CRYPTOLITE\_ECDSA\_VERIFY\_USE\_PK

```
#define IFX_PSA_CRYPTOLITE_ECDSA_VERIFY_USE_PK
```

Definition at line 92 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.33 IFX\_PSA\_CRYPTOLITE\_HKDF

```
#define IFX_PSA_CRYPTOLITE_HKDF
```

Definition at line 67 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.34 IFX\_PSA\_CRYPTOLITE\_HMAC

```
#define IFX_PSA_CRYPTOLITE_HMAC
```

Definition at line 60 of file ifx\_pdl\_cryptolite\_config.h.

#### 8.257.1.35 IFX\_PSA\_CRYPTOLITE\_KEY\_DERIVATION

```
#define IFX_PSA_CRYPTOLITE_KEY_DERIVATION
```

Definition at line 66 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.36 IFX\_PSA\_CRYPTOLITE\_KEY\_GENERATION**

```
#define IFX_PSA_CRYPTOLITE_KEY_GENERATION
```

Definition at line 98 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.37 IFX\_PSA\_CRYPTOLITE\_MAC**

```
#define IFX_PSA_CRYPTOLITE_MAC
```

Definition at line 59 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.38 IFX\_PSA\_CRYPTOLITE\_PBKDF2\_HMAC**

```
#define IFX_PSA_CRYPTOLITE_PBKDF2_HMAC
```

Definition at line 68 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.39 IFX\_PSA\_CRYPTOLITE\_PUBLIC\_KEY\_EXPORT**

```
#define IFX_PSA_CRYPTOLITE_PUBLIC_KEY_EXPORT
```

Definition at line 100 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.40 IFX\_PSA\_CRYPTOLITE\_SHA**

```
#define IFX_PSA_CRYPTOLITE_SHA
```

Definition at line 52 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.41 IFX\_PSA\_CRYPTOLITE\_SHA\_224**

```
#define IFX_PSA_CRYPTOLITE_SHA_224
```

Definition at line 53 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.42 IFX\_PSA\_CRYPTOLITE\_SHA\_256**

```
#define IFX_PSA_CRYPTOLITE_SHA_256
```

Definition at line 54 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.43 IFX\_PSA\_CRYPTOLITE\_SHA\_384**

```
#define IFX_PSA_CRYPTOLITE_SHA_384
```

Definition at line 55 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.44 IFX\_PSA\_CRYPTOLITE\_SHA\_512**

```
#define IFX_PSA_CRYPTOLITE_SHA_512
```

Definition at line 56 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.45 IFX\_PSA\_CRYPTOLITE\_TLS12\_PRF**

```
#define IFX_PSA_CRYPTOLITE_TLS12_PRF
```

Definition at line 69 of file ifx\_pdl\_cryptolite\_config.h.



**8.257.1.46 IFX\_PSA\_CRYPTOLITE\_TLS12\_PSK\_TO\_MS**

```
#define IFX_PSA_CRYPTOLITE_TLS12_PSK_TO_MS
```

Definition at line 70 of file ifx\_pdl\_cryptolite\_config.h.

**8.257.1.47 IFX\_PSA\_CRYPTOLITE\_USE\_STACK\_MEM**

```
#define IFX_PSA_CRYPTOLITE_USE_STACK_MEM
```

Definition at line 103 of file ifx\_pdl\_cryptolite\_config.h.

## 8.258 platform/ext/target/infineon/psc3m6/spe/services/crypto/ifx\_pdl\_cryptosuite\_config.h File Reference ↩

**Macros**

- `#define IFX_MXCRYPTOSUITE_USE_STATIC_MEM`

*Master AES enable.*

**8.258.1 Macro Definition Documentation****8.258.1.1 IFX\_MXCRYPTOSUITE\_USE\_STATIC\_MEM**

```
#define IFX_MXCRYPTOSUITE_USE_STATIC_MEM
```

Master AES enable.

Enable AES-ECB (Electronic Code book) mode Enable AES-CBC (Cipher Block Chaining) mode Enable AES-CFB (Cipher Feedback) mode Enable AES-CTR (Counter) mode

Enabled if any AES cipher mode is configured. This controls the availability of AES cipher operations in the transparent driver.

Enable CCM (Counter with CBC-MAC) AEAD mode

CCM provides authenticated encryption with associated data using AES. Requires IFX\_PSA\_CRYPTOSUITE\_AES to be enabled.

Enable CMAC (Cipher-based MAC) algorithm

CMAC provides message authentication using AES. Requires IFX\_PSA\_CRYPTOSUITE\_AES to be enabled. Enable cipher operation support (set if AES is enabled) Enable MAC operation support (set if CMAC is enabled) Enable AEAD operation support (set if CCM is enabled) Verify that at least one AES mode is selected when AES is enabled Verify that AES is enabled when CCM is requested Verify that AES is enabled when CMAC is requested

Definition at line 114 of file ifx\_pdl\_cryptosuite\_config.h.

## 8.259 platform/ext/target/infineon/psc3m6/spe/services/crypto/ifx\_pdl\_psa\_cryptolite\_config.h File Reference ↩

**Macros**

- `#define IFX_PSA_CRYPTOLITE_KEY_DERIVATION`
- `#define IFX_PSA_CRYPTOLITE_HMAC`
- `#define IFX_PSA_CRYPTOLITE_SHA_256`
- `#define IFX_PSA_CRYPTOLITE_SHA`
- `#define IFX_PSA_CRYPTOLITE_ECC_SECP_R1_256`
- `#define IFX_PSA_CRYPTOLITE_ECDH`
- `#define IFX_PSA_CRYPTOLITE_ECC_PUBLIC_KEY_EXPORT`
- `#define IFX_PSA_CRYPTOLITE_PUBLIC_KEY_EXPORT`
- `#define IFX_PSA_CRYPTOLITE_ECDSA_SIGN`
- `#define IFX_PSA_CRYPTOLITE_ECDSA_VERIFY`

- `#define IFX_PSA_CRYPTOLITE_ECDSA_VERIFY_USE_PK`
- `#define IFX_PSA_CRYPTOLITE_KEY_GENERATION`

## 8.259.1 Macro Definition Documentation

### 8.259.1.1 IFX\_PSA\_CRYPTOLITE\_ECC\_PUBLIC\_KEY\_EXPORT

```
#define IFX_PSA_CRYPTOLITE_ECC_PUBLIC_KEY_EXPORT
```

Definition at line 42 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.2 IFX\_PSA\_CRYPTOLITE\_ECC\_SECP\_R1\_256

```
#define IFX_PSA_CRYPTOLITE_ECC_SECP_R1_256
```

Definition at line 32 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.3 IFX\_PSA\_CRYPTOLITE\_ECDH

```
#define IFX_PSA_CRYPTOLITE_ECDH
```

Definition at line 38 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.4 IFX\_PSA\_CRYPTOLITE\_ECDSA\_SIGN

```
#define IFX_PSA_CRYPTOLITE_ECDSA_SIGN
```

Definition at line 45 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.5 IFX\_PSA\_CRYPTOLITE\_ECDSA\_VERIFY

```
#define IFX_PSA_CRYPTOLITE_ECDSA_VERIFY
```

Definition at line 46 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.6 IFX\_PSA\_CRYPTOLITE\_ECDSA\_VERIFY\_USE\_PK

```
#define IFX_PSA_CRYPTOLITE_ECDSA_VERIFY_USE_PK
```

Definition at line 47 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.7 IFX\_PSA\_CRYPTOLITE\_HMAC

```
#define IFX_PSA_CRYPTOLITE_HMAC
```

Definition at line 13 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.8 IFX\_PSA\_CRYPTOLITE\_KEY\_DERIVATION

```
#define IFX_PSA_CRYPTOLITE_KEY_DERIVATION
```

Definition at line 12 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.9 IFX\_PSA\_CRYPTOLITE\_KEY\_GENERATION

```
#define IFX_PSA_CRYPTOLITE_KEY_GENERATION
```

Definition at line 49 of file `ifx_pdl_psa_cryptolite_config.h`.

### 8.259.1.10 IFX\_PSA\_CRYPTOLITE\_PUBLIC\_KEY\_EXPORT

```
#define IFX_PSA_CRYPTOLITE_PUBLIC_KEY_EXPORT
```

Definition at line 43 of file ifx\_pdl\_psa\_cryptolite\_config.h.

### 8.259.1.11 IFX\_PSA\_CRYPTOLITE\_SHA

```
#define IFX_PSA_CRYPTOLITE_SHA
```

Definition at line 17 of file ifx\_pdl\_psa\_cryptolite\_config.h.

### 8.259.1.12 IFX\_PSA\_CRYPTOLITE\_SHA\_256

```
#define IFX_PSA_CRYPTOLITE_SHA_256
```

Definition at line 16 of file ifx\_pdl\_psa\_cryptolite\_config.h.

## 8.260 platform/ext/target/infineon/psc3m6/spe/services/crypto/mbedtls\_target\_config.h File Reference

### Macros

- #define [MBEDTLS\\_PSA\\_CRYPTO\\_BUILTIN\\_KEYS](#)

### 8.260.1 Macro Definition Documentation

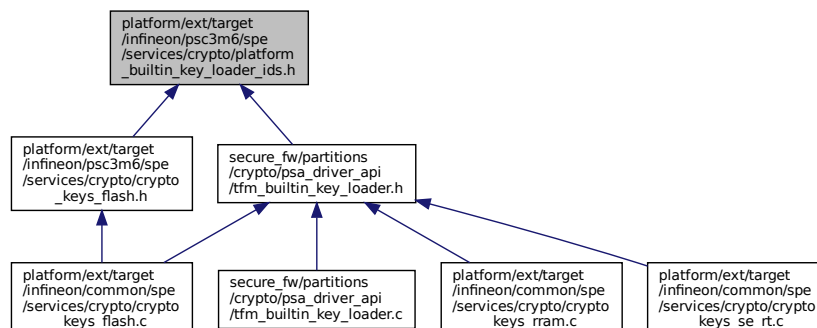
#### 8.260.1.1 MBEDTLS\_PSA\_CRYPTO\_BUILTIN\_KEYS

```
#define MBEDTLS_PSA_CRYPTO_BUILTIN_KEYS
```

Definition at line 37 of file mbedtls\_target\_config.h.

## 8.261 platform/ext/target/infineon/psc3m6/spe/services/crypto/platform\_builtin\_key\_loader\_ids.h File Reference

This graph shows which files directly or indirectly include this file:



### Macros

- #define [TFM\\_BUILTIN\\_MAX\\_KEY\\_LEN](#) 66 /\* the largest key size in bytes, that we can import \*/

## Enumerations

- enum `psa_drv_slot_number_t` { `TFM_BUILTIN_KEY_SLOT_HUK` = 0, `TFM_BUILTIN_KEY_SLOT_IAK`, `TFM_BUILTIN_KEY_SLOT_MAX` }

### 8.261.1 Macro Definition Documentation

#### 8.261.1.1 TFM\_BUILTIN\_MAX\_KEY\_LEN

`#define TFM_BUILTIN_MAX_KEY_LEN 66` /\* the largest key `size` in bytes, that we can import \*/  
 Definition at line 16 of file `platform_builtin_key_loader_ids.h`.

### 8.261.2 Enumeration Type Documentation

#### 8.261.2.1 psa\_drv\_slot\_number\_t

enum `psa_drv_slot_number_t`

Enumerator

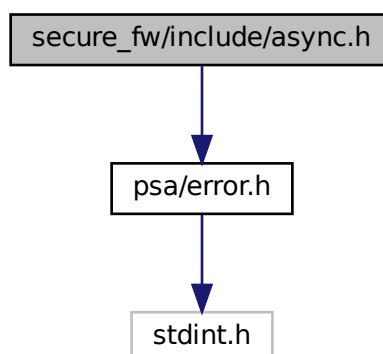
|                                       |  |
|---------------------------------------|--|
| <code>TFM_BUILTIN_KEY_SLOT_HUK</code> |  |
| <code>TFM_BUILTIN_KEY_SLOT_IAK</code> |  |
| <code>TFM_BUILTIN_KEY_SLOT_MAX</code> |  |

Definition at line 18 of file `platform_builtin_key_loader_ids.h`.

## 8.262 secure\_fw/include/async.h File Reference

`#include "psa/error.h"`

Include dependency graph for `async.h`:



```

graph BT
 A[secure_fw/partitions/ns_agent_mailbox/ns_agent_mailbox.c] --> D[secure_fw/include/async.h]
 B[secure_fw/partitions/ns_agent_mailbox/ns_agent_mailbox_rpc.c] --> D
 C[secure_fw/partitions/ns_agent_mailbox/tfm_spe_mailbox.c] --> D
 E[secure_fw/spm/core/backend_ipc.c] --> D
 F[secure_fw/spm/core/psa_api.c] --> D
 G[secure_fw/spm/core/spm_ipc.c] --> D

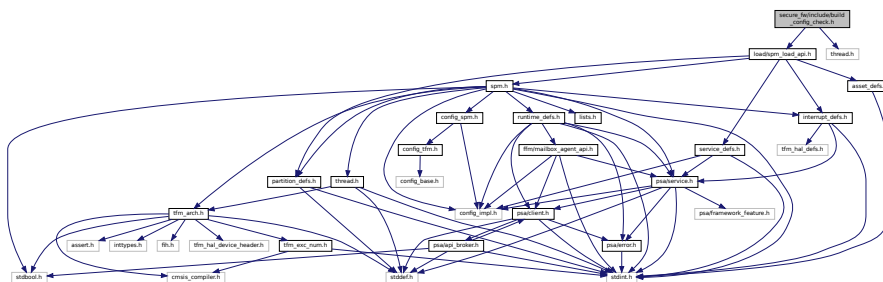
```

- #define **ASYNC\_MSG\_REPLY** (0x00000004u)

### 8.262.1.1 ASYNC\_MSG\_REPLY

The signal number for the Secure Partition message acknowledgment.  
Definition at line 17 of file `async.h`.

```
#include "load/spm_load_api.h"
#include "thread.h"
Include dependency graph for build_config_check.h:
```



```
graph BT; A[secure_fw/spm/core/main.c] --> B[secure_fw/include/build_config_check.h]
```

The diagram illustrates a dependency or relationship between two files. At the bottom, a white box contains the text `secure_fw/spm/core/main.c`. A blue arrow points upwards from this box to a gray box at the top, which contains the text `secure_fw/include/build_config_check.h`. This indicates that `main.c` depends on or includes `_config_check.h`.



```

graph TD
 Root[Introduction to the History of the World] --> T1[1. The World in the Middle Ages]
 Root --> T2[2. The World in the Renaissance]
 Root --> T3[3. The World in the 17th and 18th Centuries]
 Root --> T4[4. The World in the 19th Century]
 Root --> T5[5. The World in the 20th Century]
 Root --> T6[6. The World in the 21st Century]
 Root --> T7[7. The World in the 22nd Century]
 Root --> T8[8. The World in the 23rd Century]
 Root --> T9[9. The World in the 24th Century]
 Root --> T10[10. The World in the 25th Century]
 Root --> T11[11. The World in the 26th Century]
 Root --> T12[12. The World in the 27th Century]
 Root --> T13[13. The World in the 28th Century]
 Root --> T14[14. The World in the 29th Century]
 Root --> T15[15. The World in the 30th Century]
 T15 --> T15_1[The World in the 30th Century]
 T15 --> T15_2[The World in the 31st Century]
 T15 --> T15_3[The World in the 32nd Century]

```



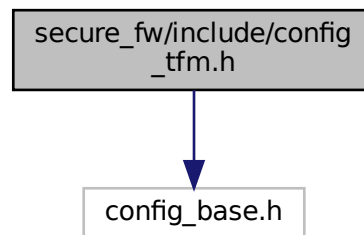
- ### 8.264.1 Macro Definition Documentation

```
#define SYNTAX_UNIFIED ".syntax unified \n"
Definition at line 32 of file compiler_ext defs.h.
```

## 8.265 secure\_fw/include/config\_tfm.h File Reference

Generated by Doxygen

Include dependency graph for config\_tfm.h:

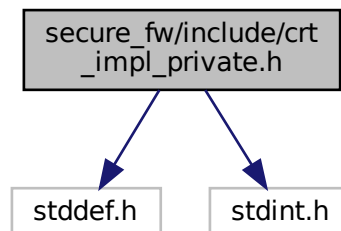


## 8.266 secure\_fw/include/crt\_impl\_private.h File Reference

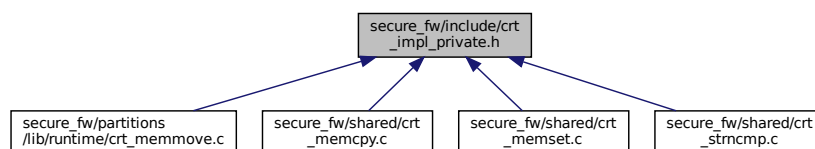
```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for crt\_impl\_private.h:



This graph shows which files directly or indirectly include this file:



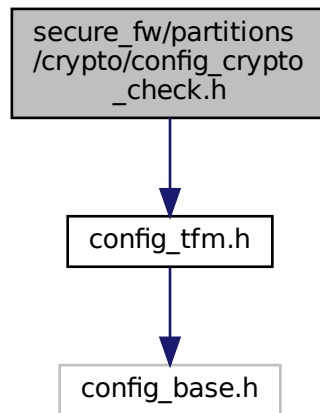
## Data Structures

- union [composite\\_addr\\_t](#)

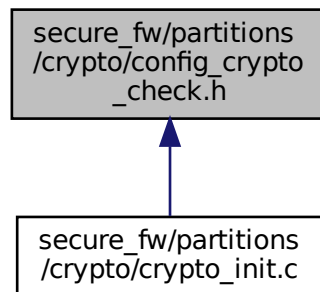




Include dependency graph for config\_crypto\_check.h:



This graph shows which files directly or indirectly include this file:

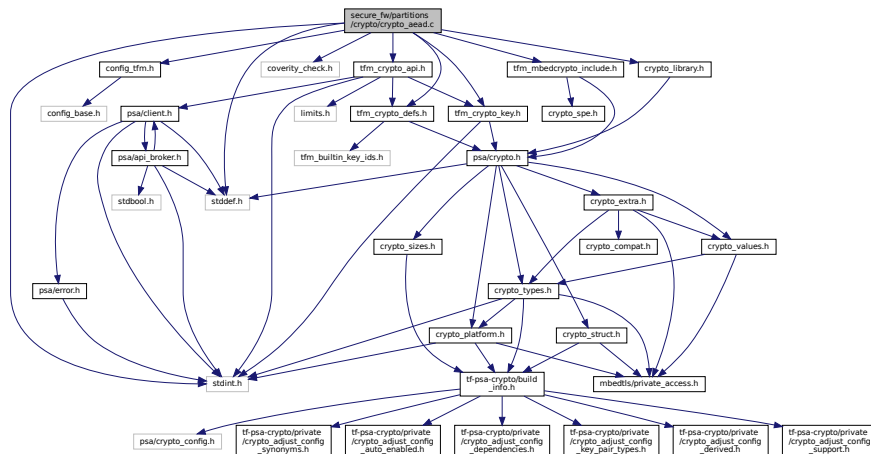


## 8.269 secure\_fw/partitions/crypto/config\_engine\_buf.h File Reference

## 8.270 secure\_fw/partitions/crypto/crypto\_aead.c File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "config_tfm.h"
#include "coverity_check.h"
#include "tfm_mbedcrypto_include.h"
#include "tfm_crypto_api.h"
#include "tfm_crypto_key.h"
#include "tfm_crypto_defs.h"
#include "crypto_library.h"
```

Include dependency graph for `crypto_aead.c`:



## Functions

- `psa_status_t tfm_crypto_aead_interface(psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`

*This function acts as interface for the AEAD module.*

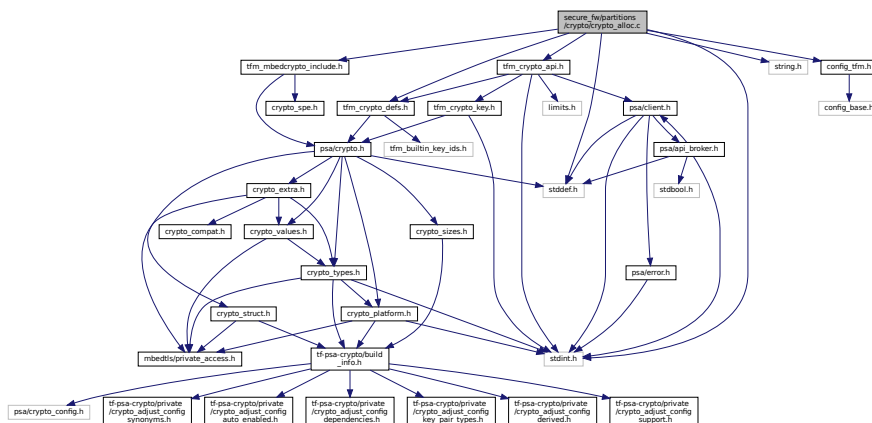
## 8.271 secure\_fw/partitions/crypto/crypto\_alloc.c File Reference

```

#include <stddef.h>
#include <stdint.h>
#include <string.h>
#include "config_tfm.h"
#include "tfm_mbedcrypto_include.h"
#include "tfm_crypto_api.h"
#include "tfm_crypto_defs.h"

```

Include dependency graph for `crypto_alloc.c`:



## Data Structures

- struct `tfm_crypto_operation_s`

*A type describing the context stored in Secure memory by the TF-M Crypto service to support multipart calls on secure side.*

## Macros

- `#define TFM_CRYPTO_INVALID_HANDLE (0x0u)`  
*This value is used to mark an handle for multipart operations as invalid.*

## Enumerations

- `enum { TFM_CRYPTO_NOT_IN_USE = 0, TFM_CRYPTO_IN_USE = 1 }`  
*Define miscellaneous literal constants that are used in the service.*

## Functions

- `psa_status_t tfm_crypto_init_alloc (void)`  
*Initialise the Alloc module.*
- `psa_status_t tfm_crypto_operation_alloc (enum tfm_crypto_operation_type type, uint32_t *handle, void **ctx)`  
*Allocate an operation context in the backend.*
- `psa_status_t tfm_crypto_operation_release (uint32_t *handle)`  
*Release an operation context in the backend.*
- `psa_status_t tfm_crypto_operation_lookup (enum tfm_crypto_operation_type type, uint32_t handle, void **ctx)`  
*Look up an operation context in the backend for the corresponding frontend operation.*

### 8.271.1 Macro Definition Documentation

#### 8.271.1.1 TFM\_CRYPTO\_INVALID\_HANDLE

```
#define TFM_CRYPTO_INVALID_HANDLE (0x0u)
```

This value is used to mark an handle for multipart operations as invalid.

Definition at line 29 of file crypto\_alloc.c.

### 8.271.2 Enumeration Type Documentation

#### 8.271.2.1 anonymous enum

anonymous enum

Define miscellaneous literal constants that are used in the service.

Enumerator

|                       |  |
|-----------------------|--|
| TFM_CRYPTO_NOT_IN_USE |  |
| TFM_CRYPTO_IN_USE     |  |

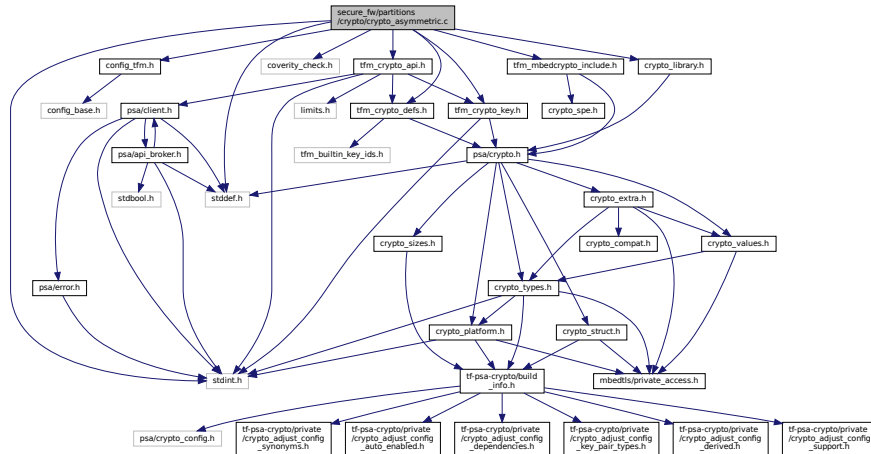
Definition at line 21 of file crypto\_alloc.c.

## 8.272 secure\_fw/partitions/crypto/crypto\_asymmetric.c File Reference

```
#include <stddef.h>
#include <stdint.h>
```

```
#include "config_tfm.h"
#include "coverity_check.h"
#include "tfm_mbedcrypto_include.h"
#include "tfm_crypto_api.h"
#include "tfm_crypto_key.h"
#include "tfm_crypto_defs.h"
#include "crypto_library.h"
```

Include dependency graph for crypto\_asymmetric.c:



## Functions

- [psa\\_status\\_t tfm\\_crypto\\_asymmetric\\_sign\\_interface](#) ([psa\\_invec](#) in\_vec[], [psa\\_outvec](#) out\_vec[], struct [tfm\\_crypto\\_key\\_id\\_s](#) \*encoded\_key)

*This function acts as interface for the Asymmetric signing module.*

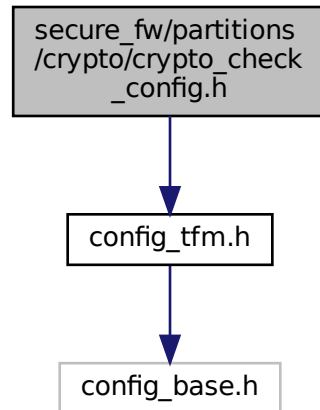
- [psa\\_status\\_t tfm\\_crypto\\_asymmetric\\_encrypt\\_interface](#) ([psa\\_invec](#) in\_vec[], [psa\\_outvec](#) out\_vec[], struct [tfm\\_crypto\\_key\\_id\\_s](#) \*encoded\_key)

*This function acts as interface for the Asymmetric encryption module.*

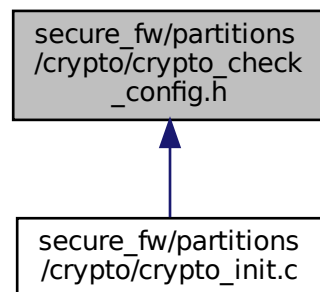
## 8.273 secure\_fw/partitions/crypto/crypto\_check\_config.h File Reference

```
#include "config_tfm.h"
```

Include dependency graph for crypto\_check\_config.h:



This graph shows which files directly or indirectly include this file:



## 8.274 secure\_fw/partitions/crypto/crypto\_cipher.c File Reference

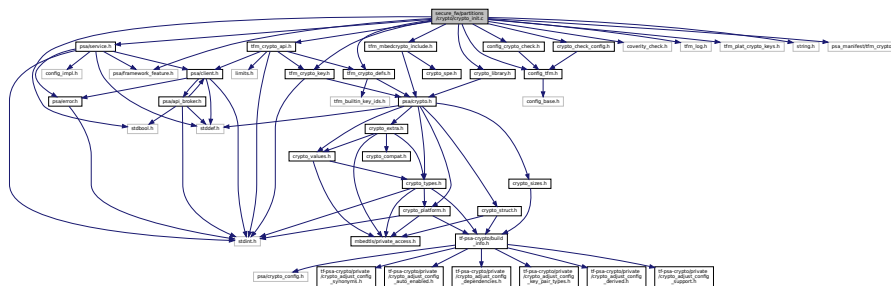
```
#include <stddef.h>
#include <stdint.h>
#include "config_tfm.h"
#include "tfm_mbedcrypto_include.h"
#include "tfm_crypto_api.h"
#include "tfm_crypto_key.h"
#include "tfm_crypto_defs.h"
#include "crypto_library.h"
```



## 8.276 secure\_fw/partitions/crypto/crypto\_init.c File Reference

```
#include <stdbool.h>
#include "config_tfm.h"
#include "config_crypto_check.h"
#include "coverity_check.h"
#include "tfm_mbedcrypto_include.h"
#include "tfm_crypto_api.h"
#include "tfm_crypto_key.h"
#include "tfm_crypto_defs.h"
#include "tfm_log.h"
#include "crypto_check_config.h"
#include "tfm_plat_crypto_keys.h"
#include "crypto_library.h"
#include <string.h>
#include "psa/framework_feature.h"
#include "psa/service.h"
#include "psa_manifest/tfm_crypto.h"
```

Include dependency graph for crypto\_init.c:



### Data Structures

- struct **tfm\_crypto\_scratch**  
*Internal scratch used for IOVec allocations.*

### Macros

- #define **ALIGN**(x, a) (((x) + ((a) - 1)) & ~((a) - 1))  
*Aligns a value x up to an alignment a.*
- #define **TFM\_CRYPTO\_IOVEC\_ALIGNMENT** (4u)  
*Maximum alignment required by any iovec parameters to the TF-M Crypto partition.*

### Functions

- **psa\_status\_t** **tfm\_crypto\_get\_caller\_id** (int32\_t \*id)  
*Returns the ID of the caller.*
- **psa\_status\_t** **tfm\_crypto\_init** (void)  
*Initialise the service.*
- **psa\_status\_t** **tfm\_crypto\_sfn** (const **psa\_msg\_t** \*msg)

#### 8.276.1 Macro Definition Documentation

### 8.276.1.1 ALIGN

```
#define ALIGN(
 x,
 a) (((x) + ((a) - 1)) & ~((a) - 1))
```

Aligns a value x up to an alignment a.

Definition at line 39 of file crypto\_init.c.

### 8.276.1.2 TFM\_CRYPTO\_IOVEC\_ALIGNMENT

```
#define TFM_CRYPTO_IOVEC_ALIGNMENT (4u)
```

Maximum alignment required by any iovec parameters to the TF-M Crypto partition.

Definition at line 45 of file crypto\_init.c.

## 8.276.2 Function Documentation

### 8.276.2.1 tfm\_crypto\_get\_caller\_id()

```
psa_status_t tfm_crypto_get_caller_id (
 int32_t * id)
```

Returns the ID of the caller.

#### Parameters

|     |    |                                      |
|-----|----|--------------------------------------|
| out | id | Pointer to hold the ID of the caller |
|-----|----|--------------------------------------|

#### Returns

Return values as described in [psa\\_status\\_t](#)

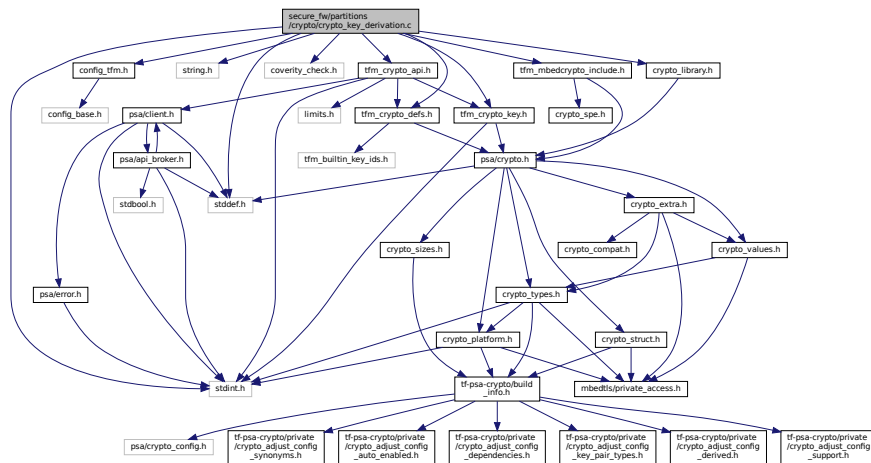
Definition at line 150 of file crypto\_init.c.

## 8.277 secure\_fw/partitions/crypto/crypto\_key\_derivation.c File Reference

```
#include <stddef.h>
#include <stdint.h>
#include <string.h>
#include "config_tfm.h"
#include "coverity_check.h"
#include "tfm_mbedcrypto_include.h"
#include "tfm_crypto_api.h"
#include "tfm_crypto_key.h"
#include "tfm_crypto_defs.h"
#include "crypto_library.h"
```



Include dependency graph for crypto\_key\_derivation.c:



## Functions

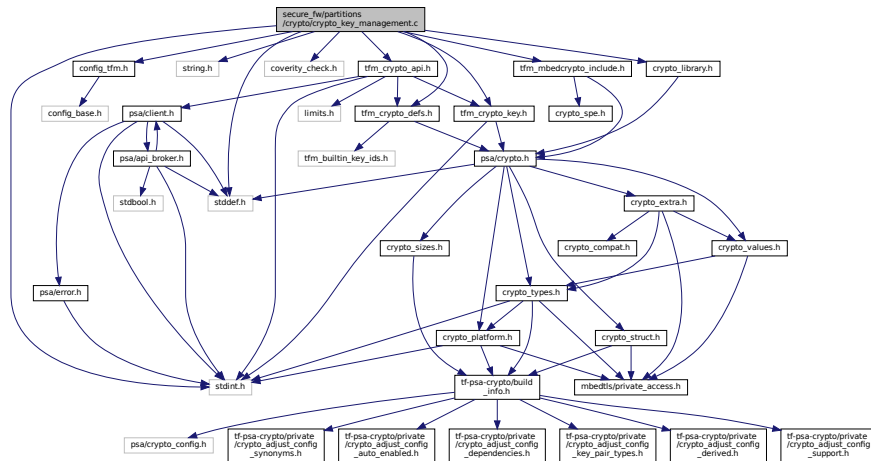
- `psa_status_t tfm_crypto_key_derivation_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`

*This function acts as interface for the Key derivation module.*

## 8.278 secure\_fw/partitions/crypto/crypto\_key\_management.c File Reference

```
#include <stddef.h>
#include <stdint.h>
#include <string.h>
#include "config_tfm.h"
#include "coverity_check.h"
#include "tfm_mbedcrypto_include.h"
#include "tfm_crypto_api.h"
#include "tfm_crypto_key.h"
#include "tfm_crypto_defs.h"
#include "crypto_library.h"
```

Include dependency graph for `crypto_key_management.c`:



## Functions

- `psa_status_t tfm_crypto_key_management_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`

*This function acts as interface for the Key management module.*

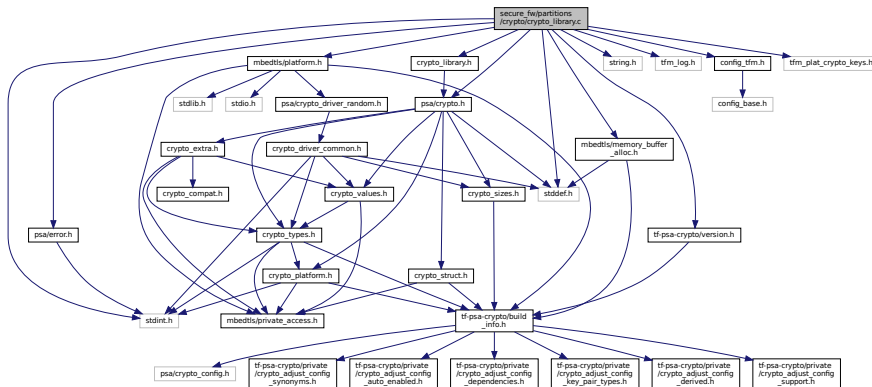
## 8.279 secure\_fw/partitions/crypto/crypto\_library.c File Reference

```

#include <stddef.h>
#include <stdint.h>
#include <string.h>
#include "tfm_log.h"
#include "config_tfm.h"
#include "psa/crypto.h"
#include "psa/error.h"
#include "crypto_library.h"
#include "tfm_plat_crypto_keys.h"
#include "mbedtls/memory_buffer_alloc.h"
#include "mbedtls/platform.h"
#include "tf-psa-crypto/version.h"

```

Include dependency graph for `crypto_library.c`:



## Functions

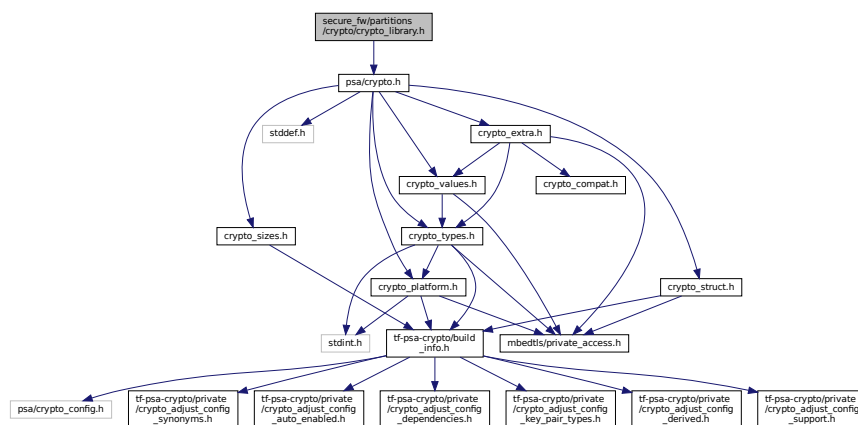
- `tfm_crypto_library_key_id_t tfm_crypto_library_key_id_init` (int32\_t owner, psa\_key\_id\_t key\_id)  
Function used to initialise an object of `tfm_crypto_library_key_id_t` to a (owner, key\_id) pair.
- `char * tfm_crypto_library_get_info` (void)  
This function is used to retrieve a string describing the library used in the backend to provide information to the crypto service and the user.
- `psa_status_t tfm_crypto_core_library_init` (void)  
This function is used to perform the necessary steps to initialise the underlying library that provides the implementation of the PSA Crypto core to the TF-M Crypto service.
- `void tfm_crypto_library_get_library_key_id_set_owner` (int32\_t owner, psa\_key\_attributes\_t \*attr)  
Allows to set the owner of a library key embedded into the key attributes structure.
- `psa_status_t mbedtls_psa_platform_get_built_in_key` (mbedtls\_svc\_key\_id\_t key\_id, psa\_key\_lifetime\_t \*lifetime, psa\_drv\_slot\_number\_t \*slot\_number)  
This function is required by TF-PSA-Crypto to enable support for platform builtin keys in the PSA Crypto core layer implemented by TF-PSA-Crypto. This function is not standardized by the API hence this layer directly provides the symbol required by the library.

## 8.280 secure\_fw/partitions/crypto/crypto\_library.h File Reference

This file contains some abstractions required to interface the TF-M Crypto service to an underlying cryptographic library that implements the PSA Crypto API. The TF-M Crypto service uses this library to provide a PSA Crypto core layer implementation and a software or hardware based implementation of crypto algorithms.

```
#include "psa/crypto.h"
```

Include dependency graph for `crypto_library.h`:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define TFM_CRYPTO_KEY_ATTR_OFFSET_CLIENT_SERVER` (sizeof(mbedtls\_key\_owner\_id\_t))  
Some integration might decide to enforce the same ABI on client and service interfaces to PSA Crypto defining the `CRYPTO_LIBRARY_ABI_COMPAT`. In this case the size of the structure describing the key attributes is the same both in client and server views. The semantics remain unchanged.

- `#define CRYPTO_LIBRARY_GET_KEY_ID(encoded_key_library) MBEDTLS_SVC_KEY_ID_GET_KEY_ID(encoded_key_library)`  
*This macro extracts the key ID from the library encoded key passed as parameter.*
- `#define CRYPTO_LIBRARY_GET_OWNER(encoded_key_library) MBEDTLS_SVC_KEY_ID_GET_OWNER_ID(encoded_key_library)`  
*This macro extracts the owner from the library encoded key passed as parameter.*

## Typedefs

- `typedef mbedtls_svc_key_id_t tfm_crypto_library_key_id_t`  
*The following typedef must be defined to the type associated to the key\_id in the underlying library.*

## Functions

- `tfm_crypto_library_key_id_t tfm_crypto_library_key_id_init (int32_t owner, psa_key_id_t key_id)`  
*Function used to initialise an object of `tfm_crypto_library_key_id_t` to a (owner, key\_id) pair.*
- `char * tfm_crypto_library_get_info (void)`  
*This function is used to retrieve a string describing the library used in the backend to provide information to the crypto service and the user.*
- `void tfm_crypto_library_get_library_key_id_set_owner (int32_t owner, psa_key_attributes_t *attr)`  
*Allows to set the owner of a library key embedded into the key attributes structure.*
- `psa_status_t tfm_crypto_core_library_init (void)`  
*This function is used to perform the necessary steps to initialise the underlying library that provides the implementation of the PSA Crypto core to the TF-M Crypto service.*

### 8.280.1 Detailed Description

This file contains some abstractions required to interface the TF-M Crypto service to an underlying cryptographic library that implements the PSA Crypto API. The TF-M Crypto service uses this library to provide a PSA Crypto core layer implementation and a software or hardware based implementation of crypto algorithms.

### 8.280.2 Macro Definition Documentation

#### 8.280.2.1 CRYPTO\_LIBRARY\_GET\_KEY\_ID

```
#define CRYPTO_LIBRARY_GET_KEY_ID(
 encoded_key_library) MBEDTLS_SVC_KEY_ID_GET_KEY_ID(encoded_key_library)
```

This macro extracts the key ID from the library encoded key passed as parameter.

Definition at line 44 of file `crypto_library.h`.

#### 8.280.2.2 CRYPTO\_LIBRARY\_GET\_OWNER

```
#define CRYPTO_LIBRARY_GET_OWNER(
 encoded_key_library) MBEDTLS_SVC_KEY_ID_GET_OWNER_ID(encoded_key_library)
```

This macro extracts the owner from the library encoded key passed as parameter.

Definition at line 50 of file `crypto_library.h`.

#### 8.280.2.3 TFM\_CRYPTO\_KEY\_ATTR\_OFFSET\_CLIENT\_SERVER

```
#define TFM_CRYPTO_KEY_ATTR_OFFSET_CLIENT_SERVER (sizeof(mbedtls_key_owner_id_t))
```

Some integration might decide to enforce the same ABI on client and service interfaces to PSA Crypto defining the `CRYPTO_LIBRARY_ABI_COMPAT`. In this case the size of the structure describing the key attributes is the same both in client and server views. The semantics remain unchanged.

Definition at line 37 of file `crypto_library.h`.





- `#define psa_aead_encrypt PSA_FUNCTION_NAME(psa_aead_encrypt)`
- `#define psa_aead_decrypt PSA_FUNCTION_NAME(psa_aead_decrypt)`
- `#define psa_aead_encrypt_setup PSA_FUNCTION_NAME(psa_aead_encrypt_setup)`
- `#define psa_aead_decrypt_setup PSA_FUNCTION_NAME(psa_aead_decrypt_setup)`
- `#define psa_aead_generate_nonce PSA_FUNCTION_NAME(psa_aead_generate_nonce)`
- `#define psa_aead_set_nonce PSA_FUNCTION_NAME(psa_aead_set_nonce)`
- `#define psa_aead_set_lengths PSA_FUNCTION_NAME(psa_aead_set_lengths)`
- `#define psa_aead_update_ad PSA_FUNCTION_NAME(psa_aead_update_ad)`
- `#define psa_aead_update PSA_FUNCTION_NAME(psa_aead_update)`
- `#define psa_aead_finish PSA_FUNCTION_NAME(psa_aead_finish)`
- `#define psa_aead_verify PSA_FUNCTION_NAME(psa_aead_verify)`
- `#define psa_aead_abort PSA_FUNCTION_NAME(psa_aead_abort)`
- `#define psa_import_key PSA_FUNCTION_NAME(psa_import_key)`
- `#define psa_destroy_key PSA_FUNCTION_NAME(psa_destroy_key)`
- `#define psa_get_key_attributes PSA_FUNCTION_NAME(psa_get_key_attributes)`
- `#define psa_reset_key_attributes PSA_FUNCTION_NAME(psa_reset_key_attributes)`
- `#define psa_export_key PSA_FUNCTION_NAME(psa_export_key)`
- `#define psa_export_public_key PSA_FUNCTION_NAME(psa_export_public_key)`
- `#define psa_purge_key PSA_FUNCTION_NAME(psa_purge_key)`
- `#define psa_copy_key PSA_FUNCTION_NAME(psa_copy_key)`
- `#define psa_cipher_operation_init PSA_FUNCTION_NAME(psa_cipher_operation_init)`
- `#define psa_cipher_generate_iv PSA_FUNCTION_NAME(psa_cipher_generate_iv)`
- `#define psa_cipher_set_iv PSA_FUNCTION_NAME(psa_cipher_set_iv)`
- `#define psa_cipher_encrypt_setup PSA_FUNCTION_NAME(psa_cipher_encrypt_setup)`
- `#define psa_cipher_decrypt_setup PSA_FUNCTION_NAME(psa_cipher_decrypt_setup)`
- `#define psa_cipher_update PSA_FUNCTION_NAME(psa_cipher_update)`
- `#define psa_cipher_encrypt PSA_FUNCTION_NAME(psa_cipher_encrypt)`
- `#define psa_cipher_decrypt PSA_FUNCTION_NAME(psa_cipher_decrypt)`
- `#define psa_cipher_finish PSA_FUNCTION_NAME(psa_cipher_finish)`
- `#define psa_cipher_abort PSA_FUNCTION_NAME(psa_cipher_abort)`
- `#define psa_hash_operation_init PSA_FUNCTION_NAME(psa_hash_operation_init)`
- `#define psa_hash_setup PSA_FUNCTION_NAME(psa_hash_setup)`
- `#define psa_hash_update PSA_FUNCTION_NAME(psa_hash_update)`
- `#define psa_hash_finish PSA_FUNCTION_NAME(psa_hash_finish)`
- `#define psa_hash_verify PSA_FUNCTION_NAME(psa_hash_verify)`
- `#define psa_hash_abort PSA_FUNCTION_NAME(psa_hash_abort)`
- `#define psa_hash_clone PSA_FUNCTION_NAME(psa_hash_clone)`
- `#define psa_hash_compute PSA_FUNCTION_NAME(psa_hash_compute)`
- `#define psa_hash_compare PSA_FUNCTION_NAME(psa_hash_compare)`
- `#define psa_mac_operation_init PSA_FUNCTION_NAME(psa_mac_operation_init)`
- `#define psa_mac_sign_setup PSA_FUNCTION_NAME(psa_mac_sign_setup)`
- `#define psa_mac_verify_setup PSA_FUNCTION_NAME(psa_mac_verify_setup)`
- `#define psa_mac_update PSA_FUNCTION_NAME(psa_mac_update)`
- `#define psa_mac_sign_finish PSA_FUNCTION_NAME(psa_mac_sign_finish)`
- `#define psa_mac_verify_finish PSA_FUNCTION_NAME(psa_mac_verify_finish)`
- `#define psa_mac_compute PSA_FUNCTION_NAME(psa_mac_compute)`
- `#define psa_mac_verify PSA_FUNCTION_NAME(psa_mac_verify)`
- `#define psa_mac_abort PSA_FUNCTION_NAME(psa_mac_abort)`
- `#define psa_sign_message PSA_FUNCTION_NAME(psa_sign_message)`
- `#define psa_verify_message PSA_FUNCTION_NAME(psa_verify_message)`
- `#define psa_sign_hash PSA_FUNCTION_NAME(psa_sign_hash)`
- `#define psa_verify_hash PSA_FUNCTION_NAME(psa_verify_hash)`
- `#define psa_asymmetric_encrypt PSA_FUNCTION_NAME(psa_asymmetric_encrypt)`
- `#define psa_asymmetric_decrypt PSA_FUNCTION_NAME(psa_asymmetric_decrypt)`
- `#define psa_generate_key PSA_FUNCTION_NAME(psa_generate_key)`
- `#define psa_key_derivation_verify_key PSA_FUNCTION_NAME(psa_key_derivation_verify_key)`
- `#define psa_key_derivation_verify_bytes PSA_FUNCTION_NAME(psa_key_derivation_verify_bytes)`

### 8.283.1 Detailed Description

When TF-PSA-Crypto is built with the MBEDTLS\_PSA\_CRYPTOSPM option enabled, this header is included by all .c files in TF-PSA-Crypto that use PSA Crypto function names. This avoids duplication of symbols between TF-M and TF-PSA-Crypto.

#### Note

This file should be included before including any PSA Crypto headers from TF-PSA-Crypto.

### 8.283.2 Macro Definition Documentation

#### 8.283.2.1 psa\_aead\_abort

```
#define psa_aead_abort PSA_FUNCTION_NAME(psa_aead_abort)
```

Definition at line 77 of file crypto\_spe.h.

#### 8.283.2.2 psa\_aead\_decrypt

```
#define psa_aead_decrypt PSA_FUNCTION_NAME(psa_aead_decrypt)
```

Definition at line 57 of file crypto\_spe.h.

#### 8.283.2.3 psa\_aead\_decrypt\_setup

```
#define psa_aead_decrypt_setup PSA_FUNCTION_NAME(psa_aead_decrypt_setup)
```

Definition at line 61 of file crypto\_spe.h.

#### 8.283.2.4 psa\_aead\_encrypt

```
#define psa_aead_encrypt PSA_FUNCTION_NAME(psa_aead_encrypt)
```

Definition at line 55 of file crypto\_spe.h.

#### 8.283.2.5 psa\_aead\_encrypt\_setup

```
#define psa_aead_encrypt_setup PSA_FUNCTION_NAME(psa_aead_encrypt_setup)
```

Definition at line 59 of file crypto\_spe.h.

#### 8.283.2.6 psa\_aead\_finish

```
#define psa_aead_finish PSA_FUNCTION_NAME(psa_aead_finish)
```

Definition at line 73 of file crypto\_spe.h.

#### 8.283.2.7 psa\_aead\_generate\_nonce

```
#define psa_aead_generate_nonce PSA_FUNCTION_NAME(psa_aead_generate_nonce)
```

Definition at line 63 of file crypto\_spe.h.

#### 8.283.2.8 psa\_aead\_set\_lengths

```
#define psa_aead_set_lengths PSA_FUNCTION_NAME(psa_aead_set_lengths)
```

Definition at line 67 of file crypto\_spe.h.



#### 8.283.2.9 psa\_aead\_set\_nonce

```
#define psa_aead_set_nonce PSA_FUNCTION_NAME (psa_aead_set_nonce)
```

Definition at line 65 of file crypto\_spe.h.

#### 8.283.2.10 psa\_aead\_update

```
#define psa_aead_update PSA_FUNCTION_NAME (psa_aead_update)
```

Definition at line 71 of file crypto\_spe.h.

#### 8.283.2.11 psa\_aead\_update\_ad

```
#define psa_aead_update_ad PSA_FUNCTION_NAME (psa_aead_update_ad)
```

Definition at line 69 of file crypto\_spe.h.

#### 8.283.2.12 psa\_aead\_verify

```
#define psa_aead_verify PSA_FUNCTION_NAME (psa_aead_verify)
```

Definition at line 75 of file crypto\_spe.h.

#### 8.283.2.13 psa\_asymmetric\_decrypt

```
#define psa_asymmetric_decrypt PSA_FUNCTION_NAME (psa_asymmetric_decrypt)
```

Definition at line 161 of file crypto\_spe.h.

#### 8.283.2.14 psa\_asymmetric\_encrypt

```
#define psa_asymmetric_encrypt PSA_FUNCTION_NAME (psa_asymmetric_encrypt)
```

Definition at line 159 of file crypto\_spe.h.

#### 8.283.2.15 psa\_can\_do\_cipher

```
#define psa_can_do_cipher PSA_FUNCTION_NAME (psa_can_do_cipher)
```

Definition at line 29 of file crypto\_spe.h.

#### 8.283.2.16 psa\_can\_do\_hash

```
#define psa_can_do_hash PSA_FUNCTION_NAME (psa_can_do_hash)
```

Definition at line 27 of file crypto\_spe.h.

#### 8.283.2.17 psa\_cipher\_abort

```
#define psa_cipher_abort PSA_FUNCTION_NAME (psa_cipher_abort)
```

Definition at line 113 of file crypto\_spe.h.

#### 8.283.2.18 psa\_cipher\_decrypt

```
#define psa_cipher_decrypt PSA_FUNCTION_NAME (psa_cipher_decrypt)
```

Definition at line 109 of file crypto\_spe.h.

**8.283.2.19 psa\_cipher\_decrypt\_setup**

```
#define psa_cipher_decrypt_setup PSA_FUNCTION_NAME(psa_cipher_decrypt_setup)
```

Definition at line 103 of file crypto\_spe.h.

**8.283.2.20 psa\_cipher\_encrypt**

```
#define psa_cipher_encrypt PSA_FUNCTION_NAME(psa_cipher_encrypt)
```

Definition at line 107 of file crypto\_spe.h.

**8.283.2.21 psa\_cipher\_encrypt\_setup**

```
#define psa_cipher_encrypt_setup PSA_FUNCTION_NAME(psa_cipher_encrypt_setup)
```

Definition at line 101 of file crypto\_spe.h.

**8.283.2.22 psa\_cipher\_finish**

```
#define psa_cipher_finish PSA_FUNCTION_NAME(psa_cipher_finish)
```

Definition at line 111 of file crypto\_spe.h.

**8.283.2.23 psa\_cipher\_generate\_iv**

```
#define psa_cipher_generate_iv PSA_FUNCTION_NAME(psa_cipher_generate_iv)
```

Definition at line 97 of file crypto\_spe.h.

**8.283.2.24 psa\_cipher\_operation\_init**

```
#define psa_cipher_operation_init PSA_FUNCTION_NAME(psa_cipher_operation_init)
```

Definition at line 95 of file crypto\_spe.h.

**8.283.2.25 psa\_cipher\_set\_iv**

```
#define psa_cipher_set_iv PSA_FUNCTION_NAME(psa_cipher_set_iv)
```

Definition at line 99 of file crypto\_spe.h.

**8.283.2.26 psa\_cipher\_update**

```
#define psa_cipher_update PSA_FUNCTION_NAME(psa_cipher_update)
```

Definition at line 105 of file crypto\_spe.h.

**8.283.2.27 psa\_copy\_key**

```
#define psa_copy_key PSA_FUNCTION_NAME(psa_copy_key)
```

Definition at line 93 of file crypto\_spe.h.

**8.283.2.28 psa\_crypto\_init**

```
#define psa_crypto_init PSA_FUNCTION_NAME(psa_crypto_init)
```

Definition at line 25 of file crypto\_spe.h.

**8.283.2.29 psa\_destroy\_key**

```
#define psa_destroy_key PSA_FUNCTION_NAME(psa_destroy_key)
```

Definition at line 81 of file crypto\_spe.h.

**8.283.2.30 psa\_export\_key**

```
#define psa_export_key PSA_FUNCTION_NAME(psa_export_key)
```

Definition at line 87 of file crypto\_spe.h.

**8.283.2.31 psa\_export\_public\_key**

```
#define psa_export_public_key PSA_FUNCTION_NAME(psa_export_public_key)
```

Definition at line 89 of file crypto\_spe.h.

**8.283.2.32 PSA\_FUNCTION\_NAME**

```
#define PSA_FUNCTION_NAME(
 x) tfpsacrypto__ ## x
```

Definition at line 23 of file crypto\_spe.h.

**8.283.2.33 psa\_generate\_key**

```
#define psa_generate_key PSA_FUNCTION_NAME(psa_generate_key)
```

Definition at line 163 of file crypto\_spe.h.

**8.283.2.34 psa\_generate\_random**

```
#define psa_generate_random PSA_FUNCTION_NAME(psa_generate_random)
```

Definition at line 53 of file crypto\_spe.h.

**8.283.2.35 psa\_get\_key\_attributes**

```
#define psa_get_key_attributes PSA_FUNCTION_NAME(psa_get_key_attributes)
```

Definition at line 83 of file crypto\_spe.h.

**8.283.2.36 psa\_hash\_abort**

```
#define psa_hash_abort PSA_FUNCTION_NAME(psa_hash_abort)
```

Definition at line 125 of file crypto\_spe.h.

**8.283.2.37 psa\_hash\_clone**

```
#define psa_hash_clone PSA_FUNCTION_NAME(psa_hash_clone)
```

Definition at line 127 of file crypto\_spe.h.

**8.283.2.38 psa\_hash\_compare**

```
#define psa_hash_compare PSA_FUNCTION_NAME(psa_hash_compare)
```

Definition at line 131 of file crypto\_spe.h.

**8.283.2.39 psa\_hash\_compute**

```
#define psa_hash_compute PSA_FUNCTION_NAME(psa_hash_compute)
```

Definition at line 129 of file crypto\_spe.h.

**8.283.2.40 psa\_hash\_finish**

```
#define psa_hash_finish PSA_FUNCTION_NAME(psa_hash_finish)
```

Definition at line 121 of file crypto\_spe.h.

**8.283.2.41 psa\_hash\_operation\_init**

```
#define psa_hash_operation_init PSA_FUNCTION_NAME(psa_hash_operation_init)
```

Definition at line 115 of file crypto\_spe.h.

**8.283.2.42 psa\_hash\_setup**

```
#define psa_hash_setup PSA_FUNCTION_NAME(psa_hash_setup)
```

Definition at line 117 of file crypto\_spe.h.

**8.283.2.43 psa\_hash\_update**

```
#define psa_hash_update PSA_FUNCTION_NAME(psa_hash_update)
```

Definition at line 119 of file crypto\_spe.h.

**8.283.2.44 psa\_hash\_verify**

```
#define psa_hash_verify PSA_FUNCTION_NAME(psa_hash_verify)
```

Definition at line 123 of file crypto\_spe.h.

**8.283.2.45 psa\_import\_key**

```
#define psa_import_key PSA_FUNCTION_NAME(psa_import_key)
```

Definition at line 79 of file crypto\_spe.h.

**8.283.2.46 psa\_key\_derivation\_abort**

```
#define psa_key_derivation_abort PSA_FUNCTION_NAME(psa_key_derivation_abort)
```

Definition at line 47 of file crypto\_spe.h.

**8.283.2.47 psa\_key\_derivation\_get\_capacity**

```
#define psa_key_derivation_get_capacity PSA_FUNCTION_NAME(psa_key_derivation_get_capacity)
```

Definition at line 31 of file crypto\_spe.h.

**8.283.2.48 psa\_key\_derivation\_input\_bytes**

```
#define psa_key_derivation_input_bytes PSA_FUNCTION_NAME(psa_key_derivation_input_bytes)
```

Definition at line 35 of file crypto\_spe.h.

**8.283.2.49 psa\_key\_derivation\_input\_integer**

```
#define psa_key_derivation_input_integer PSA_FUNCTION_NAME(psa_key_derivation_input_integer)
```

Definition at line 37 of file crypto\_spe.h.

**8.283.2.50 psa\_key\_derivation\_input\_key**

```
#define psa_key_derivation_input_key PSA_FUNCTION_NAME(psa_key_derivation_input_key)
```

Definition at line 41 of file crypto\_spe.h.

**8.283.2.51 psa\_key\_derivation\_key\_agreement**

```
#define psa_key_derivation_key_agreement PSA_FUNCTION_NAME(psa_key_derivation_key_agreement)
```

Definition at line 49 of file crypto\_spe.h.

**8.283.2.52 psa\_key\_derivation\_output\_bytes**

```
#define psa_key_derivation_output_bytes PSA_FUNCTION_NAME(psa_key_derivation_output_bytes)
```

Definition at line 39 of file crypto\_spe.h.

**8.283.2.53 psa\_key\_derivation\_output\_key**

```
#define psa_key_derivation_output_key PSA_FUNCTION_NAME(psa_key_derivation_output_key)
```

Definition at line 43 of file crypto\_spe.h.

**8.283.2.54 psa\_key\_derivation\_set\_capacity**

```
#define psa_key_derivation_set_capacity PSA_FUNCTION_NAME(psa_key_derivation_set_capacity)
```

Definition at line 33 of file crypto\_spe.h.

**8.283.2.55 psa\_key\_derivation\_setup**

```
#define psa_key_derivation_setup PSA_FUNCTION_NAME(psa_key_derivation_setup)
```

Definition at line 45 of file crypto\_spe.h.

**8.283.2.56 psa\_key\_derivation\_verify\_bytes**

```
#define psa_key_derivation_verify_bytes PSA_FUNCTION_NAME(psa_key_derivation_verify_bytes)
```

Definition at line 167 of file crypto\_spe.h.

**8.283.2.57 psa\_key\_derivation\_verify\_key**

```
#define psa_key_derivation_verify_key PSA_FUNCTION_NAME(psa_key_derivation_verify_key)
```

Definition at line 165 of file crypto\_spe.h.

**8.283.2.58 psa\_mac\_abort**

```
#define psa_mac_abort PSA_FUNCTION_NAME(psa_mac_abort)
```

Definition at line 149 of file crypto\_spe.h.

**8.283.2.59 psa\_mac\_compute**

```
#define psa_mac_compute PSA_FUNCTION_NAME(psa_mac_compute)
```

Definition at line 145 of file crypto\_spe.h.

**8.283.2.60 psa\_mac\_operation\_init**

```
#define psa_mac_operation_init PSA_FUNCTION_NAME(psa_mac_operation_init)
```

Definition at line 133 of file crypto\_spe.h.

**8.283.2.61 psa\_mac\_sign\_finish**

```
#define psa_mac_sign_finish PSA_FUNCTION_NAME(psa_mac_sign_finish)
```

Definition at line 141 of file crypto\_spe.h.

**8.283.2.62 psa\_mac\_sign\_setup**

```
#define psa_mac_sign_setup PSA_FUNCTION_NAME(psa_mac_sign_setup)
```

Definition at line 135 of file crypto\_spe.h.

**8.283.2.63 psa\_mac\_update**

```
#define psa_mac_update PSA_FUNCTION_NAME(psa_mac_update)
```

Definition at line 139 of file crypto\_spe.h.

**8.283.2.64 psa\_mac\_verify**

```
#define psa_mac_verify PSA_FUNCTION_NAME(psa_mac_verify)
```

Definition at line 147 of file crypto\_spe.h.

**8.283.2.65 psa\_mac\_verify\_finish**

```
#define psa_mac_verify_finish PSA_FUNCTION_NAME(psa_mac_verify_finish)
```

Definition at line 143 of file crypto\_spe.h.

**8.283.2.66 psa\_mac\_verify\_setup**

```
#define psa_mac_verify_setup PSA_FUNCTION_NAME(psa_mac_verify_setup)
```

Definition at line 137 of file crypto\_spe.h.

**8.283.2.67 psa\_purge\_key**

```
#define psa_purge_key PSA_FUNCTION_NAME(psa_purge_key)
```

Definition at line 91 of file crypto\_spe.h.

**8.283.2.68 psa\_raw\_key\_agreement**

```
#define psa_raw_key_agreement PSA_FUNCTION_NAME(psa_raw_key_agreement)
```

Definition at line 51 of file crypto\_spe.h.

#### 8.283.2.69 psa\_reset\_key\_attributes

```
#define psa_reset_key_attributes PSA_FUNCTION_NAME(psa_reset_key_attributes)
```

Definition at line 85 of file crypto\_spe.h.

#### 8.283.2.70 psa\_sign\_hash

```
#define psa_sign_hash PSA_FUNCTION_NAME(psa_sign_hash)
```

Definition at line 155 of file crypto\_spe.h.

#### 8.283.2.71 psa\_sign\_message

```
#define psa_sign_message PSA_FUNCTION_NAME(psa_sign_message)
```

Definition at line 151 of file crypto\_spe.h.

#### 8.283.2.72 psa\_verify\_hash

```
#define psa_verify_hash PSA_FUNCTION_NAME(psa_verify_hash)
```

Definition at line 157 of file crypto\_spe.h.

#### 8.283.2.73 psa\_verify\_message

```
#define psa_verify_message PSA_FUNCTION_NAME(psa_verify_message)
```

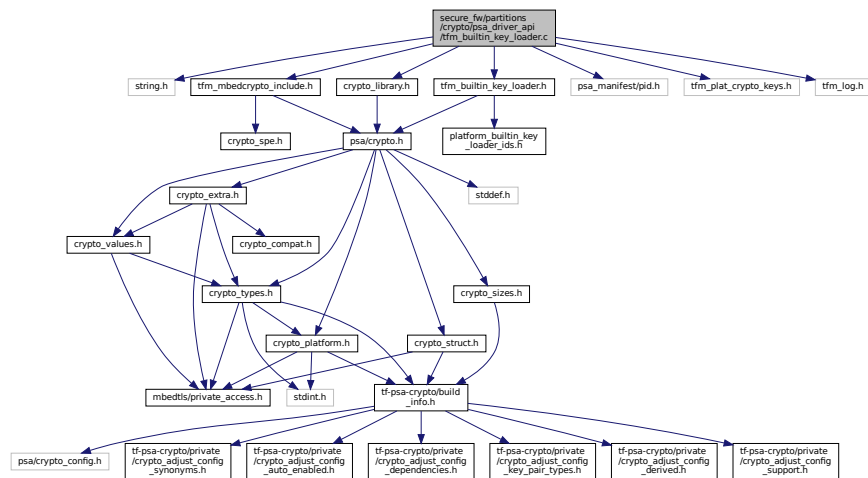
Definition at line 153 of file crypto\_spe.h.

## 8.284 secure\_fw/partitions/crypto/dir\_crypto.dox File Reference

### 8.285 secure\_fw/partitions/crypto/psa\_driver\_api/tfm\_builtin\_key\_loader.c File Reference

```
#include <string.h>
#include "tfm_builtin_key_loader.h"
#include "tfm_mbedcrypto_include.h"
#include "psa_manifest/pid.h"
#include "tfm_plat_crypto_keys.h"
#include "crypto_library.h"
#include "tfm_log.h"
```

Include dependency graph for `tfm_built_in_key_loader.c`:



## Data Structures

- struct [tfm\\_built\\_in\\_key\\_t](#)

A structure which describes a builtin key slot.

## Macros

- `#define TFM_BUILTIN_MAX_KEY_LEN (48)`
- `#define TFM_BUILTIN_MAX_KEYS (TFM_BUILTIN_KEY_SLOT_MAX)`
- `#define NUMBER_OF_ELEMENTS_OF(x) (sizeof(x)/sizeof(*(x)))`

## Functions

- `psa_status_t tfm_built_in_key_loader_init (void)`

This is the initialisation function for the `tfm_built_in_key_loader` driver, to be called from the PSA core initialisation subsystem. It discovers the keys available in the underlying hardware platform and loads them in memory visible to the PSA Crypto subsystem to be used to the normal APIs.

- `psa_status_t tfm_built_in_key_loader_get_key_buffer_size (tfm_crypto_library_key_id_t key_id, size_t *len)`

Returns the length of a key from the builtin driver.

- `psa_status_t tfm_built_in_key_loader_get_builtin_key (psa_drv_slot_number_t slot_number, psa_key_attributes_t *attributes, uint8_t *key_buffer, size_t key_buffer_size, size_t *key_buffer_length)`

Returns the attributes and key material of a key from the builtin driver to be used by the PSA Crypto core.

## 8.285.1 Macro Definition Documentation

### 8.285.1.1 NUMBER\_OF\_ELEMENTS\_OF

```
#define NUMBER_OF_ELEMENTS_OF(
 x) (sizeof(x)/sizeof(*(x)))
```

Definition at line 26 of file `tfm_built_in_key_loader.c`.



### 8.285.1.2 TFM\_BUILTIN\_MAX\_KEY\_LEN

```
#define TFM_BUILTIN_MAX_KEY_LEN (48)
```

Definition at line 19 of file tfm\_builtin\_key\_loader.c.

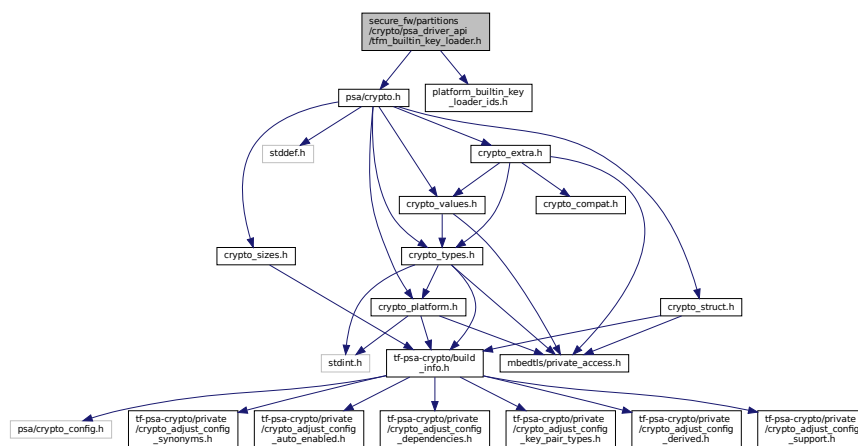
### 8.285.1.3 TFM\_BUILTIN\_MAX\_KEYS

```
#define TFM_BUILTIN_MAX_KEYS (TFM_BUILTIN_KEY_SLOT_MAX)
```

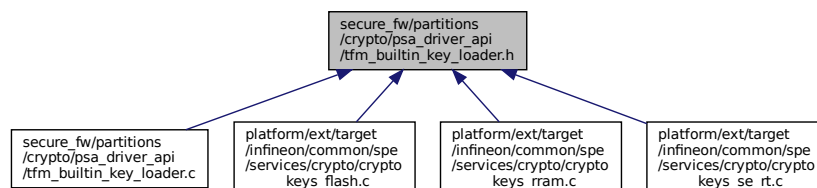
Definition at line 23 of file tfm\_builtin\_key\_loader.c.

## 8.286 secure\_fw/partitions/crypto/psa\_driver\_api/tfm\_builtin\_key\_loader.h File Reference

```
#include <psa/crypto.h>
#include "platform_builtin_key_loader_ids.h"
Include dependency graph for tfm_builtin_key_loader.h:
```



This graph shows which files directly or indirectly include this file:



## Macros

- #define [PSA\\_CRYPTO\\_DRIVER\\_TFM\\_BUILTIN\\_KEY\\_LOADER](#)  
Enables the `tfm_builtin_key_loader` driver in the PSA Crypto core subsystem.
- #define [TFM\\_BUILTIN\\_KEY\\_LOADER\\_KEY\\_LOCATION](#) (([psa\\_key\\_location\\_t](#))0x800001)  
The PSA driver location for TF-M builtin keys. Arbitrary within the ranges documented at [https://armmbed.github.io/mbed-crypto/html/api/keys/lifetimes.html#c.psa\\_key\\_location\\_t](https://armmbed.github.io/mbed-crypto/html/api/keys/lifetimes.html#c.psa_key_location_t).
- #define [TFM\\_BUILTIN\\_KEY\\_LOADER\\_LIFETIME](#)

*This macro defines the lifetime associated to TF-M builtin keys as persistent and as an ad-hoc location associated to the TFM\_BUILTIN\_KEY\_LOADER driver. To be handled by the tfm\_builtin\_key\_loader driver, the lifetime of the platform keys must be set equal to this particular lifetime value.*

## Functions

- [psa\\_status\\_t tfm\\_builtin\\_key\\_loader\\_init](#) (void)

*This is the initialisation function for the tfm\_builtin\_key\_loader driver, to be called from the PSA core initialisation subsystem. It discovers the keys available in the underlying hardware platform and loads them in memory visible to the PSA Crypto subsystem to be used to the normal APIs.*

- [psa\\_status\\_t tfm\\_builtin\\_key\\_loader\\_get\\_key\\_buffer\\_size](#) (mbedtls\_svc\_key\_id\_t key\_id, size\_t \*len)

*Returns the length of a key from the builtin driver.*

- [psa\\_status\\_t tfm\\_builtin\\_key\\_loader\\_get\\_builtin\\_key](#) (psa\_drv\_slot\_number\_t slot\_number, psa\_key\_attributes\_t \*attributes, uint8\_t \*key\_buffer, size\_t key\_buffer\_size, size\_t \*key\_buffer\_length)

*Returns the attributes and key material of a key from the builtin driver to be used by the PSA Crypto core.*

## 8.286.1 Macro Definition Documentation

### 8.286.1.1 PSA\_CRYPTO\_DRIVER\_TFM\_BUILTIN\_KEY\_LOADER

```
#define PSA_CRYPTO_DRIVER_TFM_BUILTIN_KEY_LOADER
```

Enables the tfm\_builtin\_key\_loader driver in the PSA Crypto core subsystem.

Definition at line 35 of file tfm\_builtin\_key\_loader.h.

### 8.286.1.2 TFM\_BUILTIN\_KEY\_LOADER\_KEY\_LOCATION

```
#define TFM_BUILTIN_KEY_LOADER_KEY_LOCATION ((psa_key_location_t)0x8000001)
```

The PSA driver location for TF-M builtin keys. Arbitrary within the ranges documented at [https://armmbed.github.io/mbed-crypto/html/api/keys/lifetimes.html#c.psa\\_key\\_location\\_t](https://armmbed.github.io/mbed-crypto/html/api/keys/lifetimes.html#c.psa_key_location_t).

Definition at line 43 of file tfm\_builtin\_key\_loader.h.

### 8.286.1.3 TFM\_BUILTIN\_KEY\_LOADER\_LIFETIME

```
#define TFM_BUILTIN_KEY_LOADER_LIFETIME
```

**Value:**

```
PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION(\
 PSA_KEY_LIFETIME_PERSISTENT, \
 TFM_BUILTIN_KEY_LOADER_KEY_LOCATION)
```

This macro defines the lifetime associated to TF-M builtin keys as persistent and as an ad-hoc location associated to the TFM\_BUILTIN\_KEY\_LOADER driver. To be handled by the tfm\_builtin\_key\_loader driver, the lifetime of the platform keys must be set equal to this particular lifetime value.

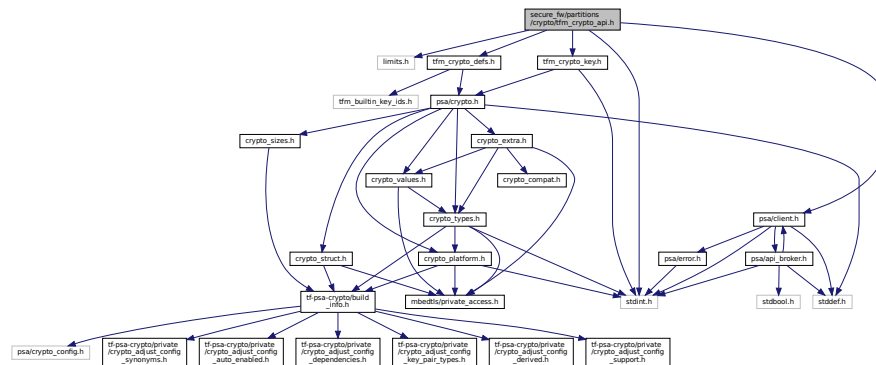
Definition at line 51 of file tfm\_builtin\_key\_loader.h.

## 8.287 secure\_fw/partitions/crypto/tfm\_crypto\_api.h File Reference

```
#include <limits.h>
#include <stdint.h>
#include "tfm_crypto_defs.h"
#include "tfm_crypto_key.h"
```

```
#include "psa/client.h"
```

Include dependency graph for tfm\_crypto\_api.h:



This graph shows which files directly or indirectly include this file:



## Enumerations

- enum [tfm\\_crypto\\_operation\\_type](#) {  
[TFM\\_CRYPTO\\_OPERATION\\_NONE](#) = 0, [TFM\\_CRYPTO\\_CIPHER\\_OPERATION](#) = 1, [TFM\\_CRYPTO\\_MAC\\_OPERATION](#) = 2, [TFM\\_CRYPTO\\_HASH\\_OPERATION](#) = 3,  
[TFM\\_CRYPTO\\_KEY\\_DERIVATION\\_OPERATION](#) = 4, [TFM\\_CRYPTO\\_AEAD\\_OPERATION](#) = 5,  
[TFM\\_CRYPTO\\_OPERATION\\_TYPE\\_MAX](#) = INT\_MAX }

List of possible operation types supported by the TFM based implementation. This type is needed by the operation allocation, lookup and release functions.

## Functions

- [psa\\_status\\_t tfm\\_crypto\\_init](#) (void)  
*Initialise the service.*
- [psa\\_status\\_t tfm\\_crypto\\_init\\_alloc](#) (void)  
*Initialise the Alloc module.*
- [psa\\_status\\_t tfm\\_crypto\\_get\\_caller\\_id](#) (int32\_t \*id)  
*Returns the ID of the caller.*
- [psa\\_status\\_t tfm\\_crypto\\_operation\\_alloc](#) (enum [tfm\\_crypto\\_operation\\_type](#) type, uint32\_t \*handle, void \*\*ctx)  
*Allocate an operation context in the backend.*
- [psa\\_status\\_t tfm\\_crypto\\_operation\\_release](#) (uint32\_t \*handle)  
*Release an operation context in the backend.*
- [psa\\_status\\_t tfm\\_crypto\\_operation\\_lookup](#) (enum [tfm\\_crypto\\_operation\\_type](#) type, uint32\_t handle, void \*\*ctx)  
*Look up an operation context in the backend for the corresponding frontend operation.*
- [psa\\_status\\_t tfm\\_crypto\\_key\\_management\\_interface](#) (psa\_invec in\_vec[], [psa\\_outvec](#) out\_vec[], struct [tfm\\_crypto\\_key\\_id\\_s](#) \*encoded\_key)  
*This function acts as interface for the Key management module.*
- [psa\\_status\\_t tfm\\_crypto\\_mac\\_interface](#) (psa\_invec in\_vec[], [psa\\_outvec](#) out\_vec[], struct [tfm\\_crypto\\_key\\_id\\_s](#) \*encoded\_key)

*This function acts as interface for the MAC module.*

- `psa_status_t tfm_crypto_cipher_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`

*This function acts as interface for the Cipher module.*

- `psa_status_t tfm_crypto_aead_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`

*This function acts as interface for the AEAD module.*

- `psa_status_t tfm_crypto_asymmetric_sign_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`

*This function acts as interface for the Asymmetric signing module.*

- `psa_status_t tfm_crypto_asymmetric_encrypt_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`

*This function acts as interface for the Asymmetric encryption module.*

- `psa_status_t tfm_crypto_key_derivation_interface (psa_invec in_vec[], psa_outvec out_vec[], struct tfm_crypto_key_id_s *encoded_key)`

*This function acts as interface for the Key derivation module.*

- `psa_status_t tfm_crypto_random_interface (psa_invec in_vec[], psa_outvec out_vec[])`

*This function acts as interface for the Random module.*

- `psa_status_t tfm_crypto_hash_interface (psa_invec in_vec[], psa_outvec out_vec[])`

*This function acts as interface for the Hash module.*

## 8.287.1 Enumeration Type Documentation

### 8.287.1.1 tfm\_crypto\_operation\_type

enum `tfm_crypto_operation_type`

List of possible operation types supported by the TFM based implementation. This type is needed by the operation allocation, lookup and release functions.

Enumerator

|                                     |  |
|-------------------------------------|--|
| TFM_CRYPTO_OPERATION_NONE           |  |
| TFM_CRYPTO_CIPHER_OPERATION         |  |
| TFM_CRYPTO_MAC_OPERATION            |  |
| TFM_CRYPTO_HASH_OPERATION           |  |
| TFM_CRYPTO_KEY_DERIVATION_OPERATION |  |
| TFM_CRYPTO_AEAD_OPERATION           |  |
| TFM_CRYPTO_OPERATION_TYPE_MAX       |  |

Definition at line 27 of file `tfm_crypto_api.h`.

## 8.287.2 Function Documentation

### 8.287.2.1 tfm\_crypto\_get\_caller\_id()

```
psa_status_t tfm_crypto_get_caller_id (
 int32_t * id)
```

Returns the ID of the caller.

Parameters

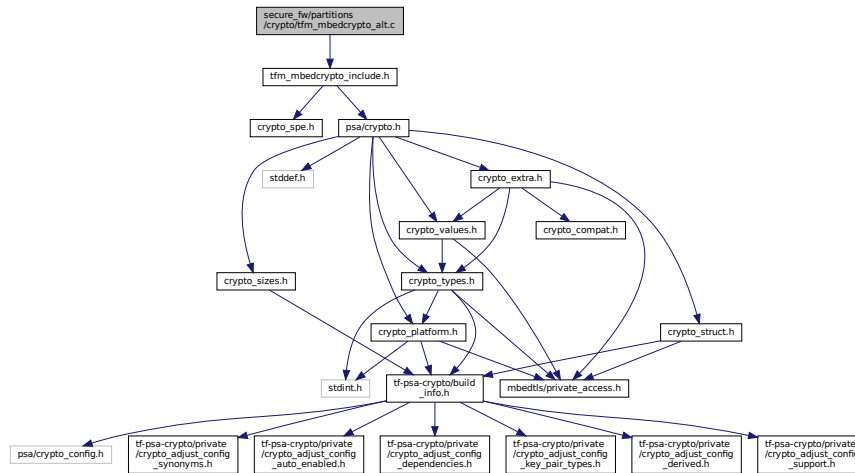
|     |    |                                      |
|-----|----|--------------------------------------|
| out | id | Pointer to hold the ID of the caller |
|-----|----|--------------------------------------|



## 8.289 secure\_fw/partitions/crypto/tfm\_mbedcrypto\_alt.c File Reference

```
#include "tfm_mbedcrypto_include.h"
```

Include dependency graph for tfm\_mbedcrypto\_alt.c:



### 8.289.1 Detailed Description

This file collects the alternative functions to replace the implementations in mbed-crypto if the corresponding mbed-crypto MBEDTLS\_\_FUNCTION\_NAME\_\_ALT is selected.

#### Note

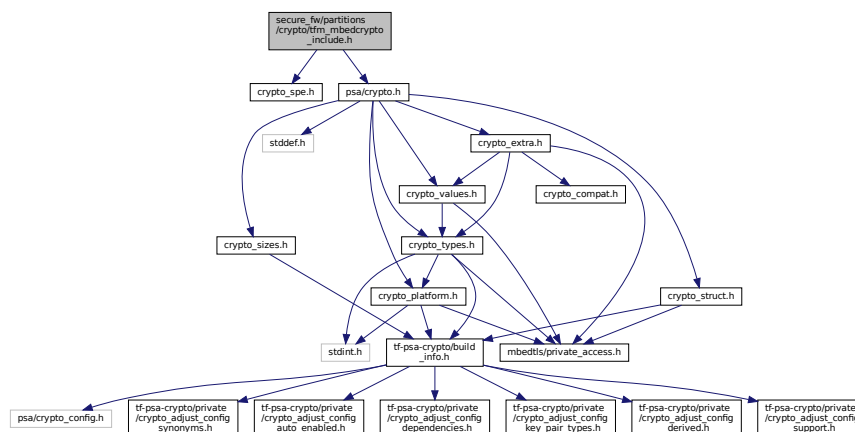
This applies only when the legacy driver API based on the \_ALT implementations is selected, and has no effect when the PSA driver interface is used. This is going to be deprecated in a future version of mbed TLS.

## 8.290 secure\_fw/partitions/crypto/tfm\_mbedcrypto\_include.h File Reference

```
#include "crypto_spe.h"
```

```
#include "psa/crypto.h"
```

Include dependency graph for tfm\_mbedcrypto\_include.h:



[illegible]

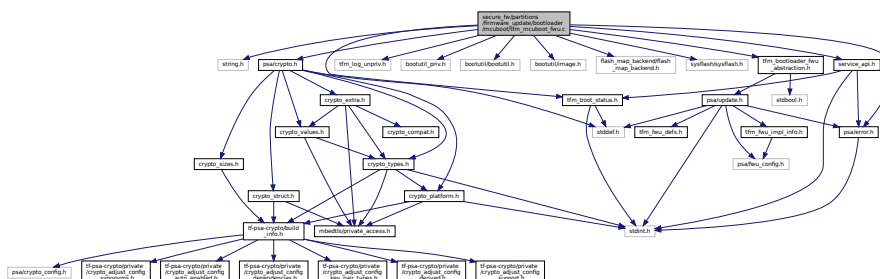
- `#define PSA_CRYPTO_SECURE 1`

```
#define PSA_CRYPTO_SECURE 1
```

Definition at line 12 of file `tfm_mbedcrypto_include.h`.

## 8.292 secure\_fw/partitions/firmware\_update/bootloader/mcuboot/tfm\_↵ mcuboot\_fw.c File Reference

Include dependency graph for tfm\_mcuboot\_fwu.c:



- struct fwu\_image\_info\_data\_s
- struct tfm fwu\_mcuboot\_ctx\_s

## Macros

- `#define MAX_IMAGE_INFO_LENGTH`

## Typedefs

- typedef struct `fwu_image_info_data_s` `fwu_image_info_data_t`
- typedef struct `tfm_fwu_mcuboot_ctx_s` `tfm_fwu_mcuboot_ctx_t`

## Functions

- `psa_status_t fwu_bootloader_init` (void)  
Initialize the staging area of the component.
- `psa_status_t fwu_bootloader_staging_area_init` (psa\_fwu\_component\_t component, const void \*manifest, size\_t manifest\_size)  
Load the image into the target component.
- `psa_status_t fwu_bootloader_load_image` (psa\_fwu\_component\_t component, size\_t image\_offset, const void \*block, size\_t block\_size)  
Starts the installation of an image.
- `psa_status_t fwu_bootloader_install_image` (const psa\_fwu\_component\_t \*candidates, uint8\_t number)  
Mark the TRIAL(running) image in component as confirmed.
- `psa_status_t fwu_bootloader_mark_image_accepted` (const psa\_fwu\_component\_t \*trials, uint8\_t number)  
Uninstall the staged image in the component.
- `psa_status_t fwu_bootloader_reject_staged_image` (psa\_fwu\_component\_t component)  
Reject the trial image in the component.
- `psa_status_t fwu_bootloader_reject_trial_image` (psa\_fwu\_component\_t component)  
Get the image information.
- `psa_status_t fwu_bootloader_abort` (psa\_fwu\_component\_t component)  
The component is in FAILED or UPDATED state. Clean the staging area of the component.
- `psa_status_t fwu_bootloader_get_image_info` (psa\_fwu\_component\_t component, bool query\_state, bool query\_impl\_info, psa\_fwu\_component\_info\_t \*info)
- `psa_status_t fwu_bootloader_clean_component` (psa\_fwu\_component\_t component)

## 8.292.1 Macro Definition Documentation

### 8.292.1.1 MAX\_IMAGE\_INFO\_LENGTH

```
#define MAX_IMAGE_INFO_LENGTH
```

**Value:**

```
(MCUBOOT_IMAGE_NUMBER * \
 (sizeof(struct image_version) + \
 SHARED_DATA_ENTRY_HEADER_SIZE))
```

Definition at line 24 of file `tfm_mcuboot_fw.c`.

## 8.292.2 Typedef Documentation

### 8.292.2.1 fwu\_image\_info\_data\_t

```
typedef struct fwu_image_info_data_s fwu_image_info_data_t
```

### 8.292.2.2 tfm\_fwu\_mcuboot\_ctx\_t

```
typedef struct tfm_fwu_mcuboot_ctx_s tfm_fwu_mcuboot_ctx_t
```



## 8.292.3 Function Documentation

### 8.292.3.1 fwu\_bootloader\_abort()

```
psa_status_t fwu_bootloader_abort (
 psa_fwu_component_t component)
```

Definition at line 488 of file tfm\_mcuboot\_fw.c.

### 8.292.3.2 fwu\_bootloader\_clean\_component()

```
psa_status_t fwu_bootloader_clean_component (
 psa_fwu_component_t component)
```

The component is in FAILED or UPDATED state. Clean the staging area of the component.

#### Parameters

|    |                  |                                         |
|----|------------------|-----------------------------------------|
| in | <i>component</i> | The identifier of the target component. |
|----|------------------|-----------------------------------------|

#### Returns

PSA\_SUCCESS On success PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter PSA\_ERROR↵  
\_STORAGE\_FAILURE

Definition at line 660 of file tfm\_mcuboot\_fw.c.

### 8.292.3.3 fwu\_bootloader\_get\_image\_info()

```
psa_status_t fwu_bootloader_get_image_info (
 psa_fwu_component_t component,
 bool query_state,
 bool query_impl_info,
 psa_fwu_component_info_t * info)
```

Get the image information.

Get the image information of the given component in staging area or the running area.

#### Parameters

|     |                        |                                                       |
|-----|------------------------|-------------------------------------------------------|
| in  | <i>component</i>       | The identifier of the target component in bootloader. |
| in  | <i>query_state</i>     | Query 'state' field.                                  |
| in  | <i>query_impl_info</i> | Query 'impl' field.                                   |
| out | <i>info</i>            | Buffer containing return the component information.   |

#### Returns

PSA\_SUCCESS On success PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter PSA\_ERROR↵  
\_GENERIC\_ERROR A fatal error occurred

Definition at line 587 of file tfm\_mcuboot\_fw.c.

### 8.292.3.4 fwu\_bootloader\_init()

```
psa_status_t fwu_bootloader_init (
 void)
```

Bootloader related initialization for firmware update, such as reading some necessary shared data from the memory if needed.

#### Returns

PSA\_SUCCESS On success PSA\_ERROR\_GENERIC\_ERROR On failure

Definition at line 88 of file tfm\_mcuboot\_fwu.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.292.3.5 fwu\_bootloader\_install\_image()

```

psa_status_t fwu_bootloader_install_image (
 const psa_fwu_component_t * candidates,
 uint8_t number)

```

Starts the installation of an image.

The components are in CANDIDATE state. Check the authenticity and integrity of the staged image in the components. If a reboot is required to complete the check, then mark this image as a candidate so that the next time bootloader runs it will take this image as a candidate one to boot-up. Return the error code PSA\_SUCCESS\_REBOOT.

#### Parameters

|    |                   |                                          |
|----|-------------------|------------------------------------------|
| in | <i>candidates</i> | A list of components in CANDIDATE state. |
| in | <i>number</i>     | Number of components in CANDIDATE state. |

#### Returns

PSA\_SUCCESS On success PSA\_SUCCESS\_REBOOT A system reboot is needed to finish installation TFM\_SUCCESS\_RESTART A restart of the updated component is required to complete the update PSA\_ERROR\_DEPENDENCY\_NEEDED Another image needs to be installed to finish installation PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter PSA\_ERROR\_INVALID\_SIGNATURE The signature is incorrect PSA\_ERROR\_GENERIC\_ERROR A fatal error occurred TFM\_ERROR\_ROLLBACK\_DETECTED The image is too old to be installed. If the image metadata contains a timestamp and it has expired, then this error is also returned.

Definition at line 202 of file tfm\_mcuboot\_fw.c.

### 8.292.3.6 fwu\_bootloader\_load\_image()

```
psa_status_t fwu_bootloader_load_image (
 psa_fwu_component_t component,
 size_t image_offset,
 const void * block,
 size_t block_size)
```

Load the image into the target component.

The component is in WRITING state. Write the image data into the target component.

#### Parameters

|    |                     |                                                                                        |
|----|---------------------|----------------------------------------------------------------------------------------|
| in | <i>component</i>    | The identifier of the target component in bootloader.                                  |
| in | <i>image_offset</i> | The offset of the image being passed into block, in bytes                              |
| in | <i>block</i>        | A buffer containing a block of image data. This might be a complete image or a subset. |
| in | <i>block_size</i>   | Size of block.                                                                         |

#### Returns

PSA\_SUCCESS On success PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter PSA\_ERROR\_INSUFFICIENT\_MEMORY There is no enough memory to process the update PSA\_ERROR\_INSUFFICIENT\_STORAGE There is no enough storage to process the update PSA\_ERROR\_GENERIC\_ERROR A fatal error occurred

Definition at line 139 of file tfm\_mcuboot\_fw.c.

### 8.292.3.7 fwu\_bootloader\_mark\_image\_accepted()

```
psa_status_t fwu_bootloader_mark_image_accepted (
 const psa_fwu_component_t * trials,
 uint8_t number)
```

Mark the TRIAL(running) image in component as confirmed.

Call this API to mark the running images as permanent/accepted to avoid revert when next time boot-up. Usually, this API is called after the running images have been verified as valid.

#### Parameters

|    |               |                                      |
|----|---------------|--------------------------------------|
| in | <i>trials</i> | A list of components in TRIAL state. |
| in | <i>number</i> | Number of components in TRIAL state. |

#### Returns

PSA\_SUCCESS On success PSA\_ERROR\_INSUFFICIENT\_MEMORY PSA\_ERROR\_INSUFFICIENT\_STORAGE PSA\_ERROR\_COMMUNICATION\_FAILURE PSA\_ERROR\_STORAGE\_FAILURE

Definition at line 390 of file tfm\_mcuboot\_fw.c.

### 8.292.3.8 fwu\_bootloader\_reject\_staged\_image()

```
psa_status_t fwu_bootloader_reject_staged_image (
 psa_fwu_component_t component)
```

Uninstall the staged image in the component.

The component is in STAGED state. Uninstall the staged image in the component so that this image will not be treated as a candidate next time boot-up.

## Parameters

|    |                  |                                         |
|----|------------------|-----------------------------------------|
| in | <i>component</i> | The identifier of the target component. |
|----|------------------|-----------------------------------------|

## Returns

PSA\_SUCCESS On success PSA\_ERROR\_INSUFFICIENT\_MEMORY PSA\_ERROR\_INSUFFICIENT\_STORAGE PSA\_ERROR\_COMMUNICATION\_FAILURE PSA\_ERROR\_STORAGE\_FAILURE

Definition at line 462 of file tfm\_mcuboot\_fwu.c.

**8.292.3.9 fwu\_bootloader\_reject\_trial\_image()**

```
psa_status_t fwu_bootloader_reject_trial_image (
 psa_fwu_component_t component)
```

Reject the trial image in the component.

The component is in TRIAL state. Mark the running image in the component as rejected.

## Parameters

|    |                  |                                         |
|----|------------------|-----------------------------------------|
| in | <i>component</i> | The identifier of the target component. |
|----|------------------|-----------------------------------------|

## Returns

PSA\_SUCCESS On success PSA\_ERROR\_INSUFFICIENT\_MEMORY PSA\_ERROR\_INSUFFICIENT\_STORAGE PSA\_ERROR\_COMMUNICATION\_FAILURE PSA\_ERROR\_STORAGE\_FAILURE

Definition at line 478 of file tfm\_mcuboot\_fwu.c.

**8.292.3.10 fwu\_bootloader\_staging\_area\_init()**

```
psa_status_t fwu_bootloader_staging_area_init (
 psa_fwu_component_t component,
 const void * manifest,
 size_t manifest_size)
```

Initialize the staging area of the component.

The component is in READY state. Prepare the staging area of the component for image download. For example, initialize the staging area, open the flash area, and so on. The image will be written into the staging area later.

## Parameters

|    |                      |                                                                                                                                                 |
|----|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>component</i>     | The identifier of the target component in bootloader.                                                                                           |
| in | <i>manifest</i>      | A pointer to a buffer containing a detached manifest for the update. If the manifest is bundled with the firmware image, manifest must be NULL. |
| in | <i>manifest_size</i> | Size of the manifest buffer in bytes.                                                                                                           |

## Returns

PSA\_SUCCESS On success PSA\_ERROR\_INVALID\_SIGNATURE A signature or integrity check on the manifest has failed. PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter PSA\_ERROR\_INSUFFICIENT\_MEMORY PSA\_ERROR\_INSUFFICIENT\_STORAGE PSA\_ERROR\_COMMUNICATION\_FAILURE PSA\_ERROR\_STORAGE\_FAILURE

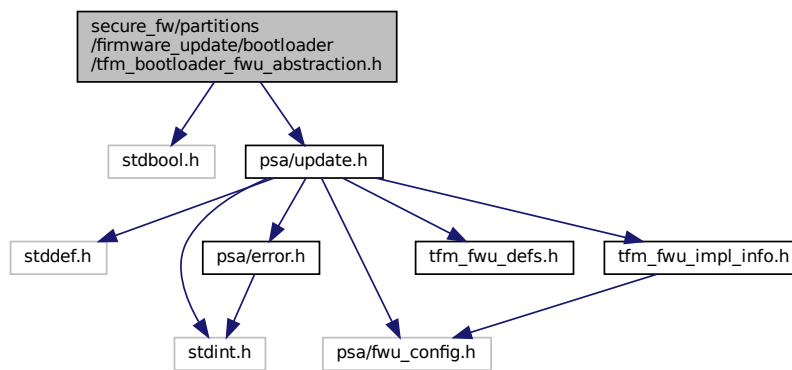
Definition at line 109 of file tfm\_mcuboot\_fwu.c.

## 8.293 secure\_fw/partitions/firmware\_update/bootloader/tfm\_bootloader\_fwu\_abstraction.h File Reference

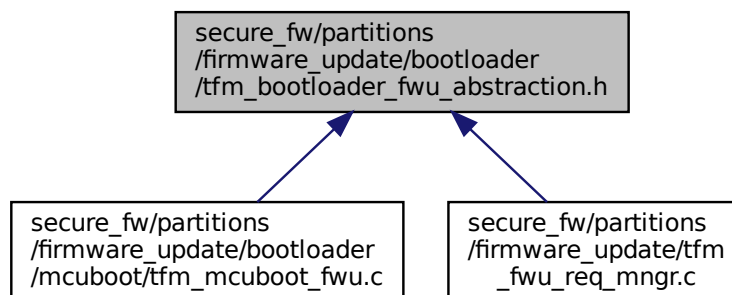
```
#include <stdbool.h>
```

```
#include "psa/update.h"
```

Include dependency graph for tfm\_bootloader\_fwu\_abstraction.h:



This graph shows which files directly or indirectly include this file:



## Functions

- `psa_status_t fwu_bootloader_init (void)`
- `psa_status_t fwu_bootloader_staging_area_init (psa_fwu_component_t component, const void *manifest, size_t manifest_size)`

*Initialize the staging area of the component.*

- `psa_status_t fwu_bootloader_load_image (psa_fwu_component_t component, size_t image_offset, const void *block, size_t block_size)`

*Load the image into the target component.*

- `psa_status_t fwu_bootloader_install_image (const psa_fwu_component_t *candidates, uint8_t number)`

*Starts the installation of an image.*

- `psa_status_t fwu_bootloader_mark_image_accepted (const psa_fwu_component_t *trials, uint8_t number)`

*Mark the TRIAL(running) image in component as confirmed.*

- `psa_status_t fwu_bootloader_reject_staged_image (psa_fwu_component_t component)`

*Uninstall the staged image in the component.*

- `psa_status_t fwu_bootloader_reject_trial_image (psa_fwu_component_t component)`

*Reject the trial image in the component.*

- `psa_status_t fwu_bootloader_clean_component (psa_fwu_component_t component)`

*The component is in FAILED or UPDATED state. Clean the staging area of the component.*

- `psa_status_t fwu_bootloader_get_image_info (psa_fwu_component_t component, bool query_state, bool query_impl_info, psa_fwu_component_info_t *info)`

*Get the image information.*

## 8.293.1 Function Documentation

### 8.293.1.1 fwu\_bootloader\_clean\_component()

```
psa_status_t fwu_bootloader_clean_component (
 psa_fwu_component_t component)
```

The component is in FAILED or UPDATED state. Clean the staging area of the component.

#### Parameters

|    |                  |                                         |
|----|------------------|-----------------------------------------|
| in | <i>component</i> | The identifier of the target component. |
|----|------------------|-----------------------------------------|

#### Returns

PSA\_SUCCESS On success  
PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter  
PSA\_ERROR\_STORAGE\_FAILURE

Definition at line 660 of file `tfm_mcuboot_fwu.c`.

### 8.293.1.2 fwu\_bootloader\_get\_image\_info()

```
psa_status_t fwu_bootloader_get_image_info (
 psa_fwu_component_t component,
 bool query_state,
 bool query_impl_info,
 psa_fwu_component_info_t * info)
```

Get the image information.

Get the image information of the given component in staging area or the running area.

#### Parameters

|     |                        |                                                       |
|-----|------------------------|-------------------------------------------------------|
| in  | <i>component</i>       | The identifier of the target component in bootloader. |
| in  | <i>query_state</i>     | Query 'state' field.                                  |
| in  | <i>query_impl_info</i> | Query 'impl' field.                                   |
| out | <i>info</i>            | Buffer containing return the component information.   |

**Returns**

PSA\_SUCCESS On success PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter PSA\_ERROR\_↵  
 \_GENERIC\_ERROR A fatal error occurred

Definition at line 587 of file tfm\_mcuboot\_fwu.c.

**8.293.1.3 fwu\_bootloader\_init()**

```
psa_status_t fwu_bootloader_init (
 void)
```

Bootloader related initialization for firmware update, such as reading some necessary shared data from the memory if needed.

**Returns**

PSA\_SUCCESS On success PSA\_ERROR\_GENERIC\_ERROR On failure

Definition at line 88 of file tfm\_mcuboot\_fwu.c.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.293.1.4 fwu\_bootloader\_install\_image()**

```
psa_status_t fwu_bootloader_install_image (
 const psa_fwu_component_t * candidates,
 uint8_t number)
```

Starts the installation of an image.

The components are in CANDIDATE state. Check the authenticity and integrity of the staged image in the components. If a reboot is required to complete the check, then mark this image as a candidate so that the next time bootloader runs it will take this image as a candidate one to boot-up. Return the error code PSA\_SUCCESS\_RE↵  
 BOOT.

**Parameters**

|    |                   |                                          |
|----|-------------------|------------------------------------------|
| in | <i>candidates</i> | A list of components in CANDIDATE state. |
| in | <i>number</i>     | Number of components in CANDIDATE state. |



**Returns**

PSA\_SUCCESS On success PSA\_SUCCESS\_REBOOT A system reboot is needed to finish installation TFM\_SUCCESS\_RESTART A restart of the updated component is required to complete the update PSA\_ERROR\_DEPENDENCY\_NEEDED Another image needs to be installed to finish installation PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter PSA\_ERROR\_INVALID\_SIGNATURE The signature is incorrect PSA\_ERROR\_GENERIC\_ERROR A fatal error occurred TFM\_ERROR\_ROLLBACK\_DETECTED The image is too old to be installed. If the image metadata contains a timestamp and it has expired, then this error is also returned.

Definition at line 202 of file tfm\_mcuboot\_fwu.c.

**8.293.1.5 fwu\_bootloader\_load\_image()**

```
psa_status_t fwu_bootloader_load_image (
 psa_fwu_component_t component,
 size_t image_offset,
 const void * block,
 size_t block_size)
```

Load the image into the target component.

The component is in WRITING state. Write the image data into the target component.

**Parameters**

|    |                     |                                                                                        |
|----|---------------------|----------------------------------------------------------------------------------------|
| in | <i>component</i>    | The identifier of the target component in bootloader.                                  |
| in | <i>image_offset</i> | The offset of the image being passed into block, in bytes                              |
| in | <i>block</i>        | A buffer containing a block of image data. This might be a complete image or a subset. |
| in | <i>block_size</i>   | Size of block.                                                                         |

**Returns**

PSA\_SUCCESS On success PSA\_ERROR\_INVALID\_ARGUMENT Invalid input parameter PSA\_ERROR\_INSUFFICIENT\_MEMORY There is not enough memory to process the update PSA\_ERROR\_INSUFFICIENT\_STORAGE There is not enough storage to process the update PSA\_ERROR\_GENERIC\_ERROR A fatal error occurred

Definition at line 139 of file tfm\_mcuboot\_fwu.c.

**8.293.1.6 fwu\_bootloader\_mark\_image\_accepted()**

```
psa_status_t fwu_bootloader_mark_image_accepted (
 const psa_fwu_component_t * trials,
 uint8_t number)
```

Mark the TRIAL(running) image in component as confirmed.

Call this API to mark the running images as permanent/accepted to avoid revert when next time boot-up. Usually, this API is called after the running images have been verified as valid.

**Parameters**

|    |               |                                      |
|----|---------------|--------------------------------------|
| in | <i>trials</i> | A list of components in TRIAL state. |
| in | <i>number</i> | Number of components in TRIAL state. |

**Returns**

PSA\_SUCCESS On success PSA\_ERROR\_INSUFFICIENT\_MEMORY PSA\_ERROR\_INSUFFICIENT\_STORAGE PSA\_ERROR\_COMMUNICATION\_FAILURE PSA\_ERROR\_STORAGE\_FAILURE

Definition at line 390 of file tfm\_mcuboot\_fwu.c.

**8.293.1.7 fwu\_bootloader\_reject\_staged\_image()**

```
psa_status_t fwu_bootloader_reject_staged_image (
 psa_fwu_component_t component)
```

Uninstall the staged image in the component.

The component is in STAGED state. Uninstall the staged image in the component so that this image will not be treated as a candidate next time boot-up.

**Parameters**

|    |                  |                                         |
|----|------------------|-----------------------------------------|
| in | <i>component</i> | The identifier of the target component. |
|----|------------------|-----------------------------------------|

**Returns**

PSA\_SUCCESS On success PSA\_ERROR\_INSUFFICIENT\_MEMORY PSA\_ERROR\_INSUFFICIENT\_STORAGE PSA\_ERROR\_COMMUNICATION\_FAILURE PSA\_ERROR\_STORAGE\_FAILURE

Definition at line 462 of file tfm\_mcuboot\_fwu.c.

**8.293.1.8 fwu\_bootloader\_reject\_trial\_image()**

```
psa_status_t fwu_bootloader_reject_trial_image (
 psa_fwu_component_t component)
```

Reject the trial image in the component.

The component is in TRIAL state. Mark the running image in the component as rejected.

**Parameters**

|    |                  |                                         |
|----|------------------|-----------------------------------------|
| in | <i>component</i> | The identifier of the target component. |
|----|------------------|-----------------------------------------|

**Returns**

PSA\_SUCCESS On success PSA\_ERROR\_INSUFFICIENT\_MEMORY PSA\_ERROR\_INSUFFICIENT\_STORAGE PSA\_ERROR\_COMMUNICATION\_FAILURE PSA\_ERROR\_STORAGE\_FAILURE

Definition at line 478 of file tfm\_mcuboot\_fwu.c.

**8.293.1.9 fwu\_bootloader\_staging\_area\_init()**

```
psa_status_t fwu_bootloader_staging_area_init (
 psa_fwu_component_t component,
 const void * manifest,
 size_t manifest_size)
```

Initialize the staging area of the component.

The component is in READY state. Prepare the staging area of the component for image download. For example, initialize the staging area, open the flash area, and so on. The image will be written into the staging area later.

**Parameters**

|    |                  |                                                       |
|----|------------------|-------------------------------------------------------|
| in | <i>component</i> | The identifier of the target component in bootloader. |
|----|------------------|-------------------------------------------------------|



## 8.294.1 Macro Definition Documentation

### 8.294.1.1 COMPONENTS\_ITER

```
#define COMPONENTS_ITER(
 x) for ((x) = 0; (x) < (FWU_COMPONENT_NUMBER); (x)++)
```

Definition at line 21 of file tfm\_fwu\_req\_mngr.c.

## 8.294.2 Typedef Documentation

### 8.294.2.1 tfm\_fwu\_ctx\_t

```
typedef struct tfm_fwu_ctx_s tfm_fwu_ctx_t
```

## 8.294.3 Function Documentation

### 8.294.3.1 tfm\_firmware\_update\_service\_sfn()

```
psa_status_t tfm_firmware_update_service_sfn (
 const psa_msg_t * msg)
```

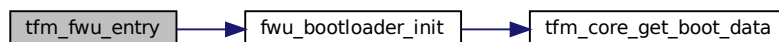
Definition at line 516 of file tfm\_fwu\_req\_mngr.c.

### 8.294.3.2 tfm\_fwu\_entry()

```
psa_status_t tfm_fwu_entry (
 void)
```

Definition at line 544 of file tfm\_fwu\_req\_mngr.c.

Here is the call graph for this function:

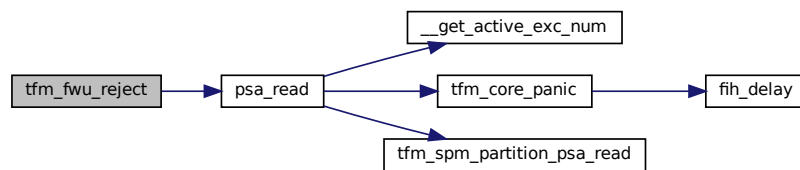


### 8.294.3.3 tfm\_fwu\_reject()

```
psa_status_t tfm_fwu_reject (
 const psa_msg_t * msg)
```

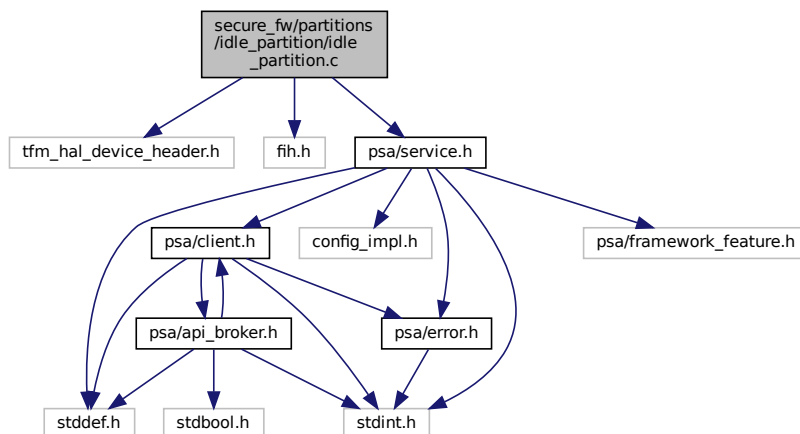
Definition at line 365 of file tfm\_fwu\_req\_mngr.c.

Here is the call graph for this function:



## 8.295 secure\_fw/partitions/idle\_partition/idle\_partition.c File Reference

```
#include "tfm_hal_device_header.h"
#include "fih.h"
#include "psa/service.h"
Include dependency graph for idle_partition.c:
```



## Functions

- [void tfm\\_idle\\_thread \(void\)](#)

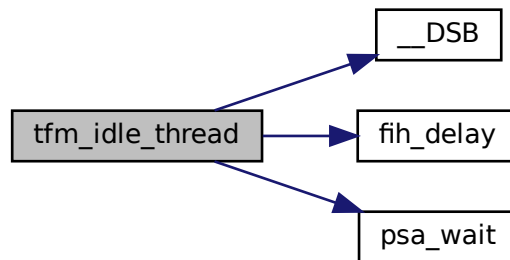
### 8.295.1 Function Documentation

#### 8.295.1.1 tfm\_idle\_thread()

```
void tfm_idle_thread (
 void)
```

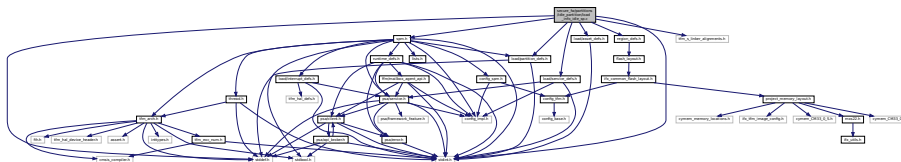
Definition at line 14 of file `idle_partition.c`.

Here is the call graph for this function:



## 8.296 secure\_fw/partitions/idle\_partition/load\_info\_idle\_sp.c File Reference

```
#include <stdint.h>
#include <stddef.h>
#include "spm.h"
#include "load/partition_defs.h"
#include "load/service_defs.h"
#include "load/asset_defs.h"
#include "region_defs.h"
#include "tfm_s_linker_alignments.h"
Include dependency graph for load_info_idle_sp.c:
```



## Data Structures

- struct [partition\\_tfm\\_sp\\_idle\\_load\\_info\\_t](#)

## Macros

- #define [IDLE\\_SP\\_STACK\\_SIZE](#) ROUND\_UP\_TO\_MULTIPLE(0x100, TFM\_LINKER\_IDLE\_PARTITION\_  
STACK\_ALIGNMENT)

## Functions

- void [tfm\\_idle\\_thread](#) (void)

## Variables

- uint8\_t [idle\\_sp\\_stack](#) [ROUND\_UP\_TO\_MULTIPLE(0x100, TFM\_LINKER\_IDLE\_PARTITION\_STACK\_ALIGN-  
MENT)]
- const struct [partition\\_tfm\\_sp\\_idle\\_load\\_info\\_t](#) [tfm\\_sp\\_idle\\_load](#)

## 8.296.1 Macro Definition Documentation

### 8.296.1.1 IDLE\_SP\_STACK\_SIZE

```
#define IDLE_SP_STACK_SIZE ROUND_UP_TO_MULTIPLE(0x100, TFM_LINKER_IDLE_PARTITION_STACK_ALIGN←
MENT)
```

Definition at line 25 of file load\_info\_idle\_sp.c.

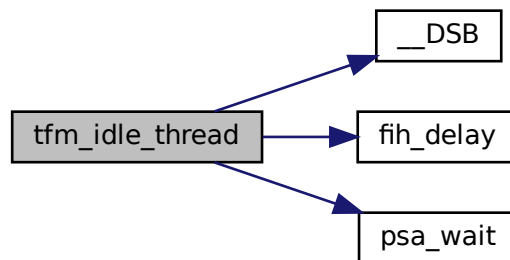
## 8.296.2 Function Documentation

### 8.296.2.1 tfm\_idle\_thread()

```
void tfm_idle_thread (
 void)
```

Definition at line 14 of file idle\_partition.c.

Here is the call graph for this function:



## 8.296.3 Variable Documentation

### 8.296.3.1 idle\_sp\_stack

```
uint8_t idle_sp_stack[ROUND_UP_TO_MULTIPLE(0x100, TFM_LINKER_IDLE_PARTITION_STACK_ALIGNMENT)]
```

Definition at line 42 of file load\_info\_idle\_sp.c.

### 8.296.3.2 tfm\_sp\_idle\_load

```
const struct partition_tfm_sp_idle_load_info_t tfm_sp_idle_load
```

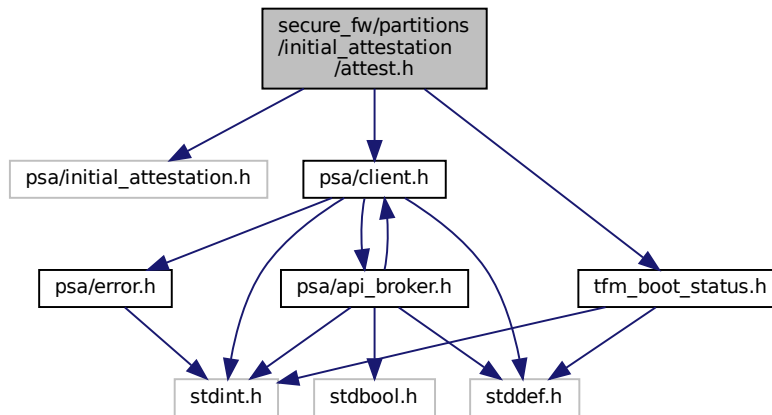
Definition at line 51 of file load\_info\_idle\_sp.c.

## 8.297 secure\_fw/partitions/initial\_attestation/attest.h File Reference

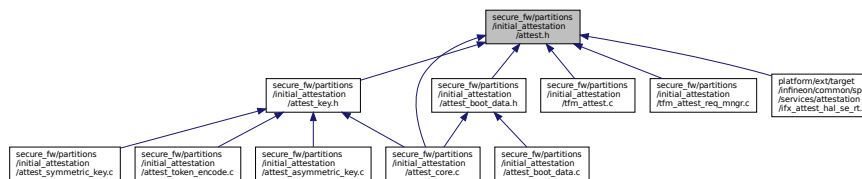
```
#include "psa/initial_attestation.h"
#include "psa/client.h"
```

```
#include "tfm_boot_status.h"
```

Include dependency graph for attest.h:



This graph shows which files directly or indirectly include this file:



## Enumerations

- enum [psa\\_attest\\_err\\_t](#) {  
[PSA\\_ATTEST\\_ERR\\_SUCCESS](#) = 0, [PSA\\_ATTEST\\_ERR\\_INIT\\_FAILED](#), [PSA\\_ATTEST\\_ERR\\_BUFFER\\_OVERFLOW](#),  
[PSA\\_ATTEST\\_ERR\\_CLAIM\\_UNAVAILABLE](#),  
[PSA\\_ATTEST\\_ERR\\_INVALID\\_INPUT](#), [PSA\\_ATTEST\\_ERR\\_GENERAL](#), [PSA\\_ATTEST\\_ERR\\_FORCE\\_INT\\_SIZE](#)  
= INT\_MAX }

*Initial attestation service error types.*

## Functions

- enum [psa\\_attest\\_err\\_t](#) [attest\\_get\\_boot\\_data](#) (uint8\_t major\_type, struct [tfm\\_boot\\_data](#) \*boot\_data, uint32\_t len)  
Copy the boot data (coming from boot loader) from shared memory area to service memory area.
- enum [psa\\_attest\\_err\\_t](#) [attest\\_get\\_caller\\_client\\_id](#) (int32\_t \*caller\_id)  
Get the ID of the caller thread.
- [psa\\_status\\_t](#) [attest\\_init](#) (void)  
Initialise the initial attestation service during the TF-M boot up process.
- [psa\\_status\\_t](#) [initial\\_attest\\_get\\_token](#) (const void \*challenge\_buf, size\_t challenge\_size, void \*token\_buf, size\_t token\_buf\_size, size\_t \*token\_size)  
Get initial attestation token.
- [psa\\_status\\_t](#) [initial\\_attest\\_get\\_token\\_size](#) (size\_t challenge\_size, size\_t \*token\_size)  
Get the size of the initial attestation token.



## 8.297.1 Enumeration Type Documentation

### 8.297.1.1 psa\_attest\_err\_t

enum `psa_attest_err_t`

Initial attestation service error types.

#### Enumerator

|                                  |                                                                                                                                                                                                                                      |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PSA_ATTEST_ERR_SUCCESS           | Action was performed successfully                                                                                                                                                                                                    |
| PSA_ATTEST_ERR_INIT_FAILED       | Boot status data is unavailable or malformed                                                                                                                                                                                         |
| PSA_ATTEST_ERR_BUFFER_OVERFLOW   | Buffer is too small to store required data                                                                                                                                                                                           |
| PSA_ATTEST_ERR_CLAIM_UNAVAILABLE | Some of the mandatory claims are unavailable                                                                                                                                                                                         |
| PSA_ATTEST_ERR_INVALID_INPUT     | Some parameter or combination of parameters are recognised as invalid: <ul style="list-style-type: none"> <li>challenge size is not allowed</li> <li>challenge object is unavailable</li> <li>token buffer is unavailable</li> </ul> |
| PSA_ATTEST_ERR_GENERAL           | Unexpected error happened during operation                                                                                                                                                                                           |
| PSA_ATTEST_ERR_FORCE_INT_SIZE    | Following entry is only to ensure the error code of integer size                                                                                                                                                                     |

Definition at line 25 of file `attest.h`.

## 8.297.2 Function Documentation

### 8.297.2.1 attest\_get\_boot\_data()

```
enum psa_attest_err_t attest_get_boot_data (
 uint8_t major_type,
 struct tfm_boot_data * boot_data,
 uint32_t len)
```

Copy the boot data (coming from boot loader) from shared memory area to service memory area.

#### Parameters

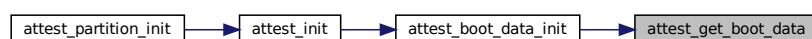
|     |                   |                                              |
|-----|-------------------|----------------------------------------------|
| in  | <i>major_type</i> | Major type of TLV entries to copy            |
| out | <i>boot_data</i>  | Pointer to the buffer to store the boot data |
| in  | <i>len</i>        | Size of the buffer to store the boot data    |

#### Returns

Returns error code as specified in `psa_attest_err_t`

Definition at line 26 of file `tfm_attest.c`.

Here is the caller graph for this function:



### 8.297.2.2 attest\_get\_caller\_client\_id()

```
enum psa_attest_err_t attest_get_caller_client_id (
 int32_t * caller_id)
```

Get the ID of the caller thread.

#### Parameters

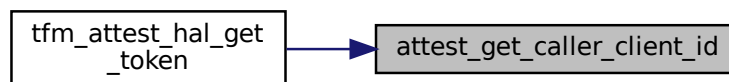
|     |                  |                                  |
|-----|------------------|----------------------------------|
| out | <i>caller_id</i> | Pointer where to store caller ID |
|-----|------------------|----------------------------------|

#### Returns

Returns error code as specified in [psa\\_attest\\_err\\_t](#)

Definition at line 17 of file `tfm_attest.c`.

Here is the caller graph for this function:



### 8.297.2.3 attest\_init()

```
psa_status_t attest_init (
 void)
```

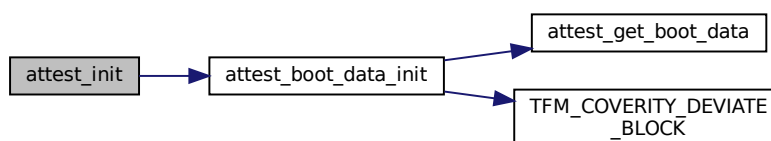
Initialise the initial attestation service during the TF-M boot up process.

#### Returns

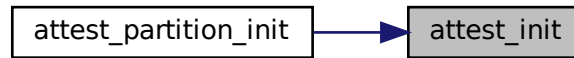
Returns `PSA_SUCCESS` if init has been completed, otherwise error as specified in [psa\\_status\\_t](#)

Definition at line 77 of file `attest_core.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.297.2.4 initial\_attest\_get\_token()

```

psa_status_t initial_attest_get_token (
 const void * challenge_buf,
 size_t challenge_size,
 void * token_buf,
 size_t token_buf_size,
 size_t * token_size)

```

Get initial attestation token.

##### Parameters

|     |                       |                                                               |
|-----|-----------------------|---------------------------------------------------------------|
| in  | <i>challenge_buf</i>  | Pointer to buffer where challenge input is stored.            |
| in  | <i>challenge_size</i> | Size of challenge object in bytes.                            |
| out | <i>token_buf</i>      | Pointer to the buffer where attestation token will be stored. |
| in  | <i>token_buf_size</i> | Size of allocated buffer for token, in bytes.                 |
| out | <i>token_size</i>     | Size of the token that has been returned, in bytes.           |

##### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 703 of file `attest_core.c`.

#### 8.297.2.5 initial\_attest\_get\_token\_size()

```

psa_status_t initial_attest_get_token_size (
 size_t challenge_size,
 size_t * token_size)

```

Get the size of the initial attestation token.

##### Parameters

|     |                       |                                                                              |
|-----|-----------------------|------------------------------------------------------------------------------|
| in  | <i>challenge_size</i> | Size of challenge object in bytes. This must be a supported challenge size.  |
| out | <i>token_size</i>     | Size of the token in bytes, which is created by initial attestation service. |



### 8.298.1.2 INSTANCE\_ID\_SIZE

```
#define INSTANCE_ID_SIZE (PSA_HASH_LENGTH(PSA_ALG_SHA_256) + 1)
```

Definition at line 39 of file attest\_asymmetric\_key.c.

### 8.298.1.3 MAX\_ENCODED\_COSE\_KEY\_SIZE

```
#define MAX_ENCODED_COSE_KEY_SIZE
```

**Value:**

```
1 + /* 1 byte to encode map */ \
2 + /* 2 bytes to encode key type */ \
2 + /* 2 bytes to encode curve */ \
2 * /* the X and Y coordinates + encoding */ \
(ECC_COORD_SIZE + 1 + 2)
```

Definition at line 29 of file attest\_asymmetric\_key.c.

## 8.298.2 Function Documentation

### 8.298.2.1 attest\_get\_instance\_id()

```
enum psa_attest_err_t attest_get_instance_id (
 struct q_useful_buf_c * id_buf)
```

Get the buffer of Instance ID data.

**Parameters**

|     |        |                                          |
|-----|--------|------------------------------------------|
| out | id_buf | Address and length of Instance ID buffer |
|-----|--------|------------------------------------------|

**Return values**

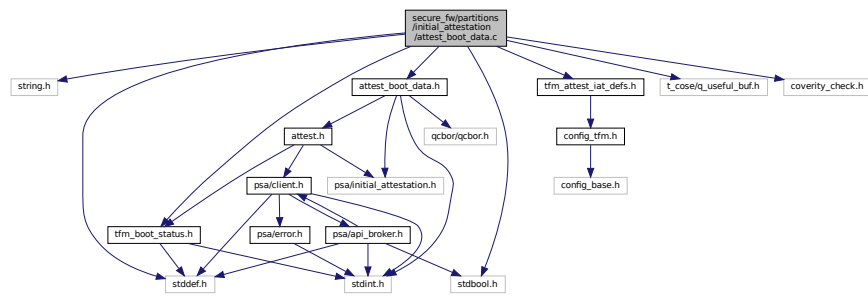
|                                  |                                        |
|----------------------------------|----------------------------------------|
| PSA_ATTEST_ERR_SUCCESS           | Instance ID was successfully returned. |
| PSA_ATTEST_ERR_CLAIM_UNAVAILABLE | Instance ID is unavailable             |
| PSA_ATTEST_ERR_GENERAL           | Instance ID could not be returned.     |

Definition at line 83 of file attest\_asymmetric\_key.c.

## 8.299 secure\_fw/partitions/initial\_attestation/attest\_boot\_data.c File Reference

```
#include <string.h>
#include <stddef.h>
#include <stdbool.h>
#include "attest_boot_data.h"
#include "tfm_boot_status.h"
#include "tfm_attest_iat_defs.h"
#include "t_cose/q_useful_buf.h"
#include "coverity_check.h"
```

Include dependency graph for `attest_boot_data.c`:



## Data Structures

- struct [attest\\_boot\\_data](#)

*Contains the received boot status information from bootloader.*

## Macros

- `#define` [ATTEST\\_BOOT\\_STATUS\\_MAX\\_SIZE](#) 512

## Functions

- enum [psa\\_attest\\_err\\_t](#) [attest\\_encode\\_sw\\_components\\_array](#) (QCBOREncodeContext \*encode\_ctx, const int32\_t \*map\_label, uint32\_t \*cnt)

*Function to encode all the software components.*

- enum [psa\\_attest\\_err\\_t](#) [attest\\_boot\\_data\\_init](#) (void)

*Gets the IAS TLV entries (boot data coming from boot loader) from shared memory area to service memory area.*

## 8.299.1 Macro Definition Documentation

### 8.299.1.1 ATTEST\_BOOT\_STATUS\_MAX\_SIZE

```
#define ATTEST_BOOT_STATUS_MAX_SIZE 512
```

Definition at line 24 of file `attest_boot_data.c`.

## 8.299.2 Function Documentation

### 8.299.2.1 attest\_boot\_data\_init()

```
enum psa_attest_err_t attest_boot_data_init (
 void)
```

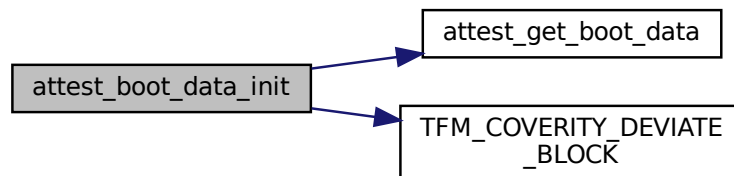
Gets the IAS TLV entries (boot data coming from boot loader) from shared memory area to service memory area.

**Returns**

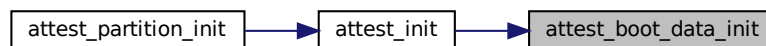
Returns error code as specified in [psa\\_attest\\_err\\_t](#)

Definition at line 347 of file `attest_boot_data.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.299.2.2 attest\_encode\_sw\_components\_array()**

```

enum psa_attest_err_t attest_encode_sw_components_array (
 QCBOREncodeContext * encode_ctx,
 const int32_t * map_label,
 uint32_t * cnt)

```

Function to encode all the software components.

The function creates a CBOR array in which 1 item is a SW component. The encoded list of these software components makes up the SW components claim. This encoded array can be stand-alone or part of a map, depending on the `map_label` parameter. The CBOR array is only created if at least 1 SW component has been found.

**Parameters**

|     |                   |                                                                                                                   |
|-----|-------------------|-------------------------------------------------------------------------------------------------------------------|
| out | <i>encode_ctx</i> | Pointer to the encoding context                                                                                   |
| in  | <i>map_label</i>  | Label/key of the map item to be added to the context. If NULL, the SW components are added as a stand-alone array |
| out | <i>cnt</i>        | Number of SW component in the encoded array                                                                       |

**Returns**

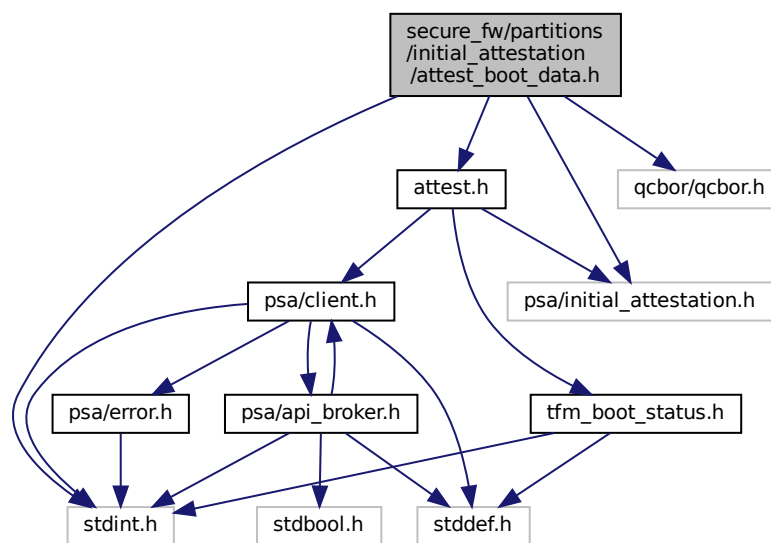
Returns error code as specified in [psa\\_attest\\_err\\_t](#)

Definition at line 214 of file attest\_boot\_data.c.

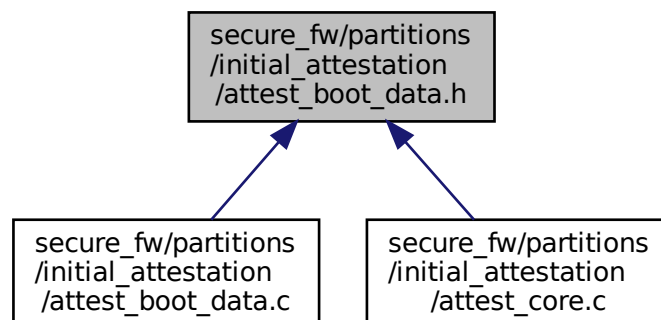
## 8.300 secure\_fw/partitions/initial\_attestation/attest\_boot\_data.h File Reference

```
#include <stdint.h>
#include "attest.h"
#include "psa/initial_attestation.h"
#include "qcbor/qcbor.h"
```

Include dependency graph for attest\_boot\_data.h:



This graph shows which files directly or indirectly include this file:





## Functions

- enum [psa\\_attest\\_err\\_t attest\\_encode\\_sw\\_components\\_array](#) (QCBOREncodeContext \*encode\_ctx, const int32\_t \*map\_label, uint32\_t \*cnt)  
*Function to encode all the software components.*
- enum [psa\\_attest\\_err\\_t attest\\_boot\\_data\\_init](#) (void)  
*Gets the IAS TLV entries (boot data coming from boot loader) from shared memory area to service memory area.*

### 8.300.1 Function Documentation

#### 8.300.1.1 attest\_boot\_data\_init()

```
enum psa_attest_err_t attest_boot_data_init (
 void)
```

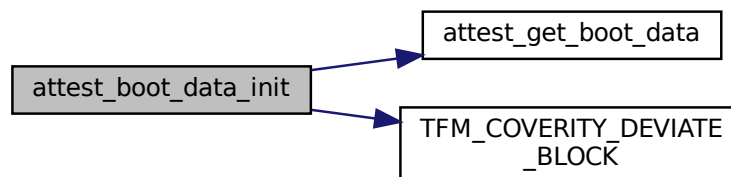
Gets the IAS TLV entries (boot data coming from boot loader) from shared memory area to service memory area.

##### Returns

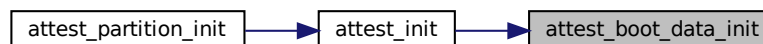
Returns error code as specified in [psa\\_attest\\_err\\_t](#)

Definition at line 347 of file `attest_boot_data.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.300.1.2 attest\_encode\_sw\_components\_array()

```
enum psa_attest_err_t attest_encode_sw_components_array (
 QCBOREncodeContext * encode_ctx,
 const int32_t * map_label,
 uint32_t * cnt)
```

Function to encode all the software components.

The function creates a CBOR array in which 1 item is a SW component. The encoded list of these software components makes up the SW components claim. This encoded array can be stand-alone or part of a map,

depending on the `map_label` parameter. The CBOR array is only created if at least 1 SW component has been found.

#### Parameters

|     |                   |                                                                                                                   |
|-----|-------------------|-------------------------------------------------------------------------------------------------------------------|
| out | <i>encode_ctx</i> | Pointer to the encoding context                                                                                   |
| in  | <i>map_label</i>  | Label/key of the map item to be added to the context. If NULL, the SW components are added as a stand-alone array |
| out | <i>cnt</i>        | Number of SW component in the encoded array                                                                       |

#### Returns

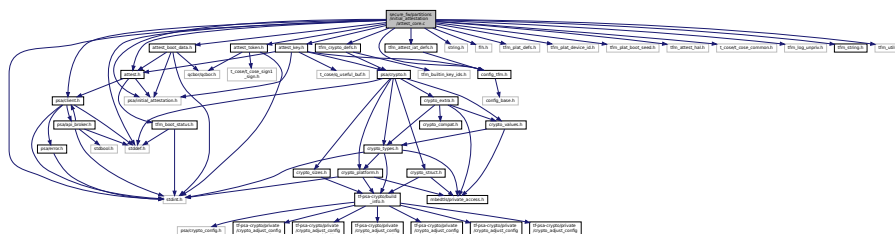
Returns error code as specified in [psa\\_attest\\_err\\_t](#)

Definition at line 214 of file `attest_boot_data.c`.

## 8.301 secure\_fw/partitions/initial\_attestation/attest\_core.c File Reference

```
#include <stdint.h>
#include <string.h>
#include <stddef.h>
#include "psa/client.h"
#include "psa/initial_attestation.h"
#include "attest.h"
#include "attest_boot_data.h"
#include "attest_key.h"
#include "attest_token.h"
#include "config_tfm.h"
#include "fih.h"
#include "tfm_plat_defs.h"
#include "tfm_plat_device_id.h"
#include "tfm_plat_boot_seed.h"
#include "tfm_attest_hal.h"
#include "tfm_attest_iat_defs.h"
#include "t_cose/t_cose_common.h"
#include "tfm_crypto_defs.h"
#include "tfm_log_unpriv.h"
#include "tfm_string.h"
#include "tfm_utils.h"
```

Include dependency graph for `attest_core.c`:



## Functions

- [psa\\_status\\_t attest\\_init \(void\)](#)

*Initialise the initial attestation service during the TF-M boot up process.*

- [psa\\_status\\_t initial\\_attest\\_get\\_token](#) (const [void](#) \*challenge\_buf, [size\\_t](#) challenge\_size, [void](#) \*token\_buf, [size\\_t](#) token\_buf\_size, [size\\_t](#) \*token\_size)  
*Get initial attestation token.*
- [psa\\_status\\_t initial\\_attest\\_get\\_token\\_size](#) ([size\\_t](#) challenge\_size, [size\\_t](#) \*token\_size)  
*Get the size of the initial attestation token.*

## 8.301.1 Function Documentation

### 8.301.1.1 attest\_init()

```
psa_status_t attest_init (
 void)
```

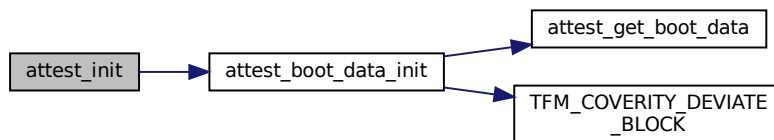
Initialise the initial attestation service during the TF-M boot up process.

#### Returns

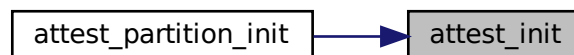
Returns PSA\_SUCCESS if init has been completed, otherwise error as specified in [psa\\_status\\_t](#)

Definition at line 77 of file attest\_core.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.301.1.2 initial\_attest\_get\_token()

```
psa_status_t initial_attest_get_token (
 const void * challenge_buf,
 size_t challenge_size,
 void * token_buf,
 size_t token_buf_size,
 size_t * token_size)
```

Get initial attestation token.

|     |                       |                                                               |
|-----|-----------------------|---------------------------------------------------------------|
| in  | <i>challenge_buf</i>  | Pointer to buffer where challenge input is stored.            |
| in  | <i>challenge_size</i> | Size of challenge object in bytes.                            |
| out | <i>token_buf</i>      | Pointer to the buffer where attestation token will be stored. |
| in  | <i>token_buf_size</i> | Size of allocated buffer for token, in bytes.                 |
| out | <i>token_size</i>     | Size of the token that has been returned, in bytes.           |

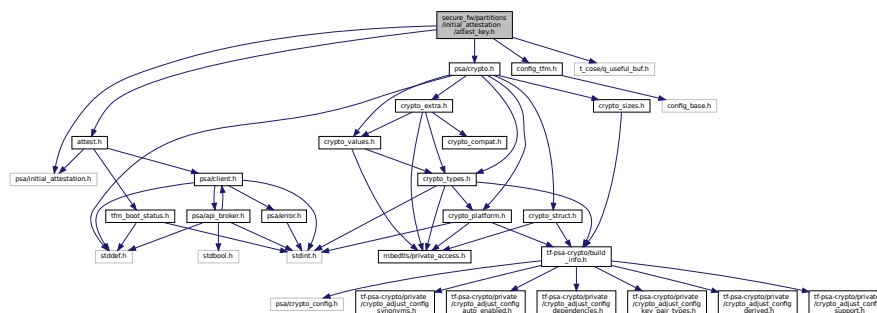
## Definition at line 703 of file attest\_core.c.

Get the size of the initial attestation token.

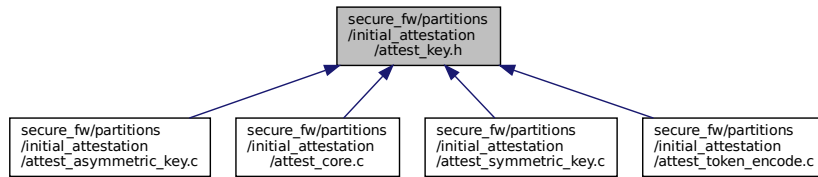
|     |                       |                                                                              |
|-----|-----------------------|------------------------------------------------------------------------------|
| in  | <i>challenge_size</i> | Size of challenge object in bytes. This must be a supported challenge size.  |
| out | <i>token_size</i>     | Size of the token in bytes, which is created by initial attestation service. |

Definition at line 779 of file attest\_core.c.

```
#include "attest.h"
#include "config_tfm.h"
#include "psa/initial_attestation.h"
#include "psa/crypto.h"
#include "t_cose/q_useful_buf.h"
Include dependency graph for attest_key.h:
```



This graph shows which files directly or indirectly include this file:



## Functions

- enum [psa\\_attest\\_err\\_t](#) [attest\\_get\\_instance\\_id](#) (struct q\_useful\_buf\_c \*id\_buf)

*Get the buffer of Instance ID data.*

### 8.302.1 Function Documentation

#### 8.302.1.1 attest\_get\_instance\_id()

```
enum psa_attest_err_t attest_get_instance_id (
 struct q_useful_buf_c * id_buf)
```

Get the buffer of Instance ID data.

##### Parameters

|     |               |                                          |
|-----|---------------|------------------------------------------|
| out | <i>id_buf</i> | Address and length of Instance ID buffer |
|-----|---------------|------------------------------------------|

##### Return values

|                                         |                                        |
|-----------------------------------------|----------------------------------------|
| <i>PSA_ATTEST_ERR_SUCCESS</i>           | Instance ID was successfully returned. |
| <i>PSA_ATTEST_ERR_CLAIM_UNAVAILABLE</i> | Instance ID is unavailable             |
| <i>PSA_ATTEST_ERR_GENERAL</i>           | Instance ID could not be returned.     |

Definition at line 83 of file `attest_asymmetric_key.c`.

## 8.303 secure\_fw/partitions/initial\_attestation/attest\_symmetric\_key.c File Reference

```
#include <stddef.h>
#include <stdint.h>
#include <string.h>
#include "attest_key.h"
#include "config_tfm.h"
#include "tfm_plat_defs.h"
#include "psa/crypto.h"
#include "psa/service.h"
#include "tfm_crypto_defs.h"
```



## Parameters

|     |        |                                          |
|-----|--------|------------------------------------------|
| out | id_buf | Address and length of Instance ID buffer |
|-----|--------|------------------------------------------|

## Return values

|                                         |                                        |
|-----------------------------------------|----------------------------------------|
| <i>PSA_ATTEST_ERR_SUCCESS</i>           | Instance ID was successfully returned. |
| <i>PSA_ATTEST_ERR_CLAIM_UNAVAILABLE</i> | Instance ID is unavailable             |
| <i>PSA_ATTEST_ERR_GENERAL</i>           | Instance ID could not be returned.     |

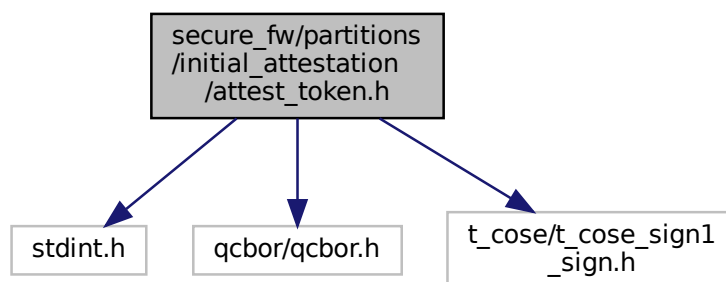
Definition at line 130 of file attest\_symmetric\_key.c.

## 8.304 secure\_fw/partitions/initial\_attestation/attest\_token.h File Reference

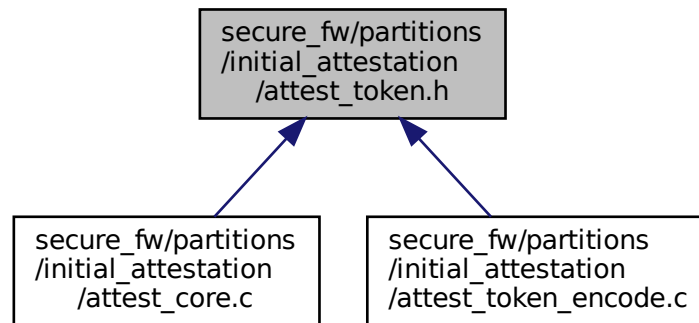
Attestation Token Creation Interface.

```
#include <stdint.h>
#include "qcbor/qcbor.h"
#include "t_cose/t_cose_sign1_sign.h"
```

Include dependency graph for attest\_token.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [attest\\_token\\_encode\\_ctx](#)

## Enumerations

- enum [attest\\_token\\_err\\_t](#) {  
[ATTEST\\_TOKEN\\_ERR\\_SUCCESS](#) = 0, [ATTEST\\_TOKEN\\_ERR\\_TOO\\_SMALL](#), [ATTEST\\_TOKEN\\_ERR\\_CBOR\\_FORMATTING](#),  
[ATTEST\\_TOKEN\\_ERR\\_GENERAL](#),  
[ATTEST\\_TOKEN\\_ERR\\_HASH\\_UNAVAILABLE](#), [ATTEST\\_TOKEN\\_ERR\\_CBOR\\_NOT\\_WELL\\_FORMED](#),  
[ATTEST\\_TOKEN\\_ERR\\_CBOR\\_STRUCTURE](#), [ATTEST\\_TOKEN\\_ERR\\_CBOR\\_TYPE](#),  
[ATTEST\\_TOKEN\\_ERR\\_INTEGER\\_VALUE](#), [ATTEST\\_TOKEN\\_ERR\\_COSE\\_FORMAT](#), [ATTEST\\_TOKEN\\_ERR\\_COSE\\_VALIDITY](#),  
[ATTEST\\_TOKEN\\_ERR\\_UNSUPPORTED\\_SIG\\_ALG](#),  
[ATTEST\\_TOKEN\\_ERR\\_INSUFFICIENT\\_MEMORY](#), [ATTEST\\_TOKEN\\_ERR\\_TAMPERING\\_DETECTED](#),  
[ATTEST\\_TOKEN\\_ERR\\_SIGNING\\_KEY](#), [ATTEST\\_TOKEN\\_ERR\\_VERIFICATION\\_KEY](#),  
[ATTEST\\_TOKEN\\_ERR\\_NO\\_VALID\\_TOKEN](#), [ATTEST\\_TOKEN\\_ERR\\_NOT\\_FOUND](#), [ATTEST\\_TOKEN\\_ERR\\_SW\\_COMPONENT](#)  
}

## Functions

- enum [attest\\_token\\_err\\_t](#) [attest\\_token\\_encode\\_start](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t key\_id, int32\_t cose\_alg\_id, const struct q\_useful\_buf \*out\_buf)  
Initialize a token creation context.
- QCBOREncodeContext \* [attest\\_token\\_encode\\_borrow\\_cbor\\_cntxt](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me)  
Get a copy of the CBOR encoding context.
- void [attest\\_token\\_encode\\_add\\_integer](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t label, int64\_t value)  
Add a 64-bit signed integer claim.
- void [attest\\_token\\_encode\\_add\\_bstr](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t label, const struct q\_useful\_buf\_c \*bstr)  
Add a binary string claim.
- void [attest\\_token\\_encode\\_add\\_tstr](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t label, const struct q\_useful\_buf\_c \*tstr)  
Add a text string claim.
- void [attest\\_token\\_encode\\_add\\_cbor](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t label, const struct q\_useful\_buf\_c \*encoded)  
Add some already-encoded CBOR to payload.



- enum [attest\\_token\\_err\\_t](#) [attest\\_token\\_encode\\_finish](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, struct q\_useful↵  
\_buf\_c \*completed\_token)

*Finish the token, complete the signing and get the result.*

### 8.304.1 Detailed Description

Attestation Token Creation Interface.

The context and functions here are the way to create an attestation token. The steps are roughly:

1. Create and initialize an [attest\\_token\\_encode\\_ctx](#) indicating the key, algorithm and such using [attest\\_token\\_encode\\_start\(\)](#).
2. Use various add methods to fill in the payload with claims. The encoding context can also be borrowed for more rich payloads.
3. Call [attest\\_token\\_encode\\_finish\(\)](#) to create the signature and finish formatting the COSE signed output.

### 8.304.2 Enumeration Type Documentation

#### 8.304.2.1 attest\_token\_err\_t

enum [attest\\_token\\_err\\_t](#)

Error codes returned from attestation token creation.

Enumerator

|                                       |                                                                                                                 |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| ATTEST_TOKEN_ERR_SUCCESS              | Success                                                                                                         |
| ATTEST_TOKEN_ERR_TOO_SMALL            | The buffer passed in to receive the output is too small.                                                        |
| ATTEST_TOKEN_ERR_CBOR_FORMATTING      | Something went wrong formatting the CBOR, most likely the payload has maps or arrays that are not closed.       |
| ATTEST_TOKEN_ERR_GENERAL              | A general, unspecific error when creating or decoding the token.                                                |
| ATTEST_TOKEN_ERR_HASH_UNAVAILABLE     | A hash function that is needed to make the token is not available.                                              |
| ATTEST_TOKEN_ERR_CBOR_NOT_WELL_FORMED | CBOR Syntax not well-formed – a CBOR syntax error.                                                              |
| ATTEST_TOKEN_ERR_CBOR_STRUCTURE       | Bad CBOR structure, for example not a map when was is required.                                                 |
| ATTEST_TOKEN_ERR_CBOR_TYPE            | Bad CBOR type, for example an not a text string, when a text string is required.                                |
| ATTEST_TOKEN_ERR_INTEGER_VALUE        | Integer too large, for example an <code>int32_t</code> is required, but value only fits in <code>int64_t</code> |
| ATTEST_TOKEN_ERR_COSE_FORMAT          | Something is wrong with the COSE message structure, missing headers or such.                                    |
| ATTEST_TOKEN_ERR_COSE_VALIDATION      | COSE signature or authentication tag is invalid, data is corrupted.                                             |
| ATTEST_TOKEN_ERR_UNSUPPORTED_SIG_ALG  | The signing algorithm is not supported.                                                                         |
| ATTEST_TOKEN_ERR_INSUFFICIENT_MEMORY  | Out of memory.                                                                                                  |
| ATTEST_TOKEN_ERR_TAMPERING_DETECTED   | Tampering detected in cryptographic function.                                                                   |
| ATTEST_TOKEN_ERR_SIGNING_KEY          | Signing key is not found or of wrong type.                                                                      |
| ATTEST_TOKEN_ERR_VERIFICATION_KEY     | Verification key is not found or of wrong type.                                                                 |
| ATTEST_TOKEN_ERR_NO_VALID_TOKEN       | No token was given or validated                                                                                 |
| ATTEST_TOKEN_ERR_NOT_FOUND            | Data item with label wasn't found.                                                                              |

## Enumerator

|                                        |                                                |
|----------------------------------------|------------------------------------------------|
| ATTEST_TOKEN_ERR_SW_COMPONENTS_MISSING | SW Components absence not correctly indicated. |
|----------------------------------------|------------------------------------------------|

Definition at line 49 of file attest\_token.h.

### 8.304.3 Function Documentation

#### 8.304.3.1 attest\_token\_encode\_add\_bstr()

```
void attest_token_encode_add_bstr (
 struct attest_token_encode_ctx * me,
 int32_t label,
 const struct q_useful_buf_c * bstr)
```

Add a binary string claim.

## Parameters

|    |              |                          |
|----|--------------|--------------------------|
| in | <i>me</i>    | Token creation context.  |
| in | <i>label</i> | Integer label for claim. |
| in | <i>bstr</i>  | The binary claim data.   |

Definition at line 338 of file attest\_token\_encode.c.

#### 8.304.3.2 attest\_token\_encode\_add\_cbor()

```
void attest_token_encode_add_cbor (
 struct attest_token_encode_ctx * me,
 int32_t label,
 const struct q_useful_buf_c * encoded)
```

Add some already-encoded CBOR to payload.

## Parameters

|    |                |                           |
|----|----------------|---------------------------|
| in | <i>me</i>      | Token creation context.   |
| in | <i>label</i>   | Integer label for claim.  |
| in | <i>encoded</i> | The already-encoded CBOR. |

Encoded CBOR must be a full map or full array or a non-aggregate type. It cannot be a partial map or array. It can be nested maps and arrays, but they must all be complete.

Definition at line 362 of file attest\_token\_encode.c.

#### 8.304.3.3 attest\_token\_encode\_add\_integer()

```
void attest_token_encode_add_integer (
 struct attest_token_encode_ctx * me,
 int32_t label,
 int64_t value)
```

Add a 64-bit signed integer claim.

## Parameters

|    |           |                         |
|----|-----------|-------------------------|
| in | <i>me</i> | Token creation context. |
|----|-----------|-------------------------|

## Parameters

|    |              |                          |
|----|--------------|--------------------------|
| in | <i>label</i> | Integer label for claim. |
| in | <i>value</i> | The integer claim data.  |

Definition at line 327 of file attest\_token\_encode.c.

**8.304.3.4 attest\_token\_encode\_add\_tstr()**

```
void attest_token_encode_add_tstr (
 struct attest_token_encode_ctx * me,
 int32_t label,
 const struct q_useful_buf_c * tstr)
```

Add a text string claim.

## Parameters

|    |              |                          |
|----|--------------|--------------------------|
| in | <i>me</i>    | Token creation context.  |
| in | <i>label</i> | Integer label for claim. |
| in | <i>tstr</i>  | The text claim data.     |

Definition at line 351 of file attest\_token\_encode.c.

**8.304.3.5 attest\_token\_encode\_borrow\_cbor\_cntxt()**

```
QCBOREncodeContext* attest_token_encode_borrow_cbor_cntxt (
 struct attest_token_encode_ctx * me)
```

Get a copy of the CBOR encoding context.

## Parameters

|    |           |                         |
|----|-----------|-------------------------|
| in | <i>me</i> | Token creation context. |
|----|-----------|-------------------------|

## Returns

The CBOR encoding context

Allows the caller to encode CBOR right into the output buffer using any of the `QCBOREncode_AddXXXX()` methods. Anything added here will be part of the payload that gets hashed. This can be used to make complex CBOR structures. All open arrays and maps must be close before calling any other `attest_token_encode` methods. `QCBOREncode_Finish()` should not be closed on this context.

Definition at line 318 of file attest\_token\_encode.c.

**8.304.3.6 attest\_token\_encode\_finish()**

```
enum attest_token_err_t attest_token_encode_finish (
 struct attest_token_encode_ctx * me,
 struct q_useful_buf_c * completed_token)
```

Finish the token, complete the signing and get the result.

## Parameters

|     |                        |                                        |
|-----|------------------------|----------------------------------------|
| in  | <i>me</i>              | Token Creation Context.                |
| out | <i>completed_token</i> | Pointer and length to completed token. |

**Returns**

one of the [attest\\_token\\_err\\_t](#) errors.

This completes the token after the payload has been added. When this is called the signing algorithm is run and the final formatting of the token is completed.

Definition at line 276 of file `attest_token_encode.c`.

**8.304.3.7 attest\_token\_encode\_start()**

```
enum attest_token_err_t attest_token_encode_start (
 struct attest_token_encode_ctx * me,
 int32_t key_select,
 int32_t cose_alg_id,
 const struct q_useful_buf * out_buf)
```

Initialize a token creation context.

**Parameters**

|     |                    |                                                                                                                                   |
|-----|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>me</i>          | The token creation context to be initialized.                                                                                     |
| in  | <i>key_select</i>  | Selects which attestation key to sign with.                                                                                       |
| in  | <i>cose_alg_id</i> | The algorithm to sign with. The IDs are defined in <a href="#">COSE (RFC 8152)</a> or in the <a href="#">IANA COSE Registry</a> . |
| out | <i>out_buf</i>     | The output buffer to write the encoded token into.                                                                                |

**Returns**

one of the [attest\\_token\\_err\\_t](#) errors.

The size of the buffer in `out_buf->len` determines the size of the token that can be created. It must be able to hold the final encoded and signed token. The data encoding overhead is just that of CBOR. The signing overhead depends on the signing key size. It is about 150 bytes for 256-bit ECDSA.

If `out_buf->ptr` is NULL and `out_buf->len` is large like `UINT32_MAX` no token will be created but the length of the token that would be created will be in `completed_token` as returned by [attest\\_token\\_encode\\_finish\(\)](#). None of the cryptographic functions run during this, but the sizes of what they would output is taken into account.

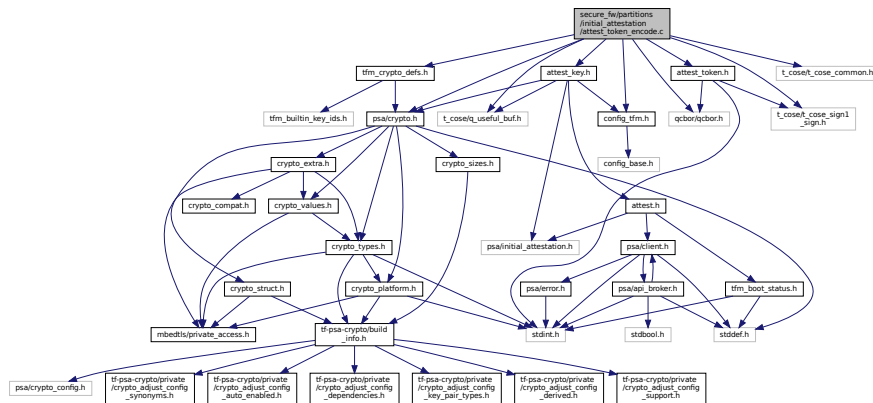
Definition at line 228 of file `attest_token_encode.c`.

## 8.305 secure\_fw/partitions/initial\_attestation/attest\_token\_encode.c File Reference

Attestation token creation implementation.

```
#include "attest_token.h"
#include "config_tfm.h"
#include "qcbor/qcbor.h"
#include "t_cose/t_cose_sign1_sign.h"
#include "t_cose/t_cose_common.h"
#include "t_cose/q_useful_buf.h"
#include "psa/crypto.h"
#include "attest_key.h"
#include "tfm_crypto_defs.h"
```

Include dependency graph for attest\_token\_encode.c:



## Functions

- enum [attest\\_token\\_err\\_t](#) [attest\\_token\\_encode\\_start](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t key\_↔select, int32\_t cose\_alg\_id, const struct q\_useful\_buf \*out\_buf)  
*Initialize a token creation context.*
- enum [attest\\_token\\_err\\_t](#) [attest\\_token\\_encode\\_finish](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, struct q\_useful\_↔\_buf\_c \*completed\_token)  
*Finish the token, complete the signing and get the result.*
- QCBOREncodeContext \* [attest\\_token\\_encode\\_borrow\\_cbor\\_cntxt](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me)  
*Get a copy of the CBOR encoding context.*
- void [attest\\_token\\_encode\\_add\\_integer](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t label, int64\_t value)  
*Add a 64-bit signed integer claim.*
- void [attest\\_token\\_encode\\_add\\_bstr](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t label, const struct q\_↔useful\_buf\_c \*bstr)  
*Add a binary string claim.*
- void [attest\\_token\\_encode\\_add\\_tstr](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t label, const struct q\_↔useful\_buf\_c \*tstr)  
*Add a text string claim.*
- void [attest\\_token\\_encode\\_add\\_cbor](#) (struct [attest\\_token\\_encode\\_ctx](#) \*me, int32\_t label, const struct q\_↔useful\_buf\_c \*encoded)  
*Add some already-encoded CBOR to payload.*

### 8.305.1 Detailed Description

Attestation token creation implementation.

### 8.305.2 Function Documentation

#### 8.305.2.1 attest\_token\_encode\_add\_bstr()

```
void attest_token_encode_add_bstr (
 struct attest_token_encode_ctx * me,
 int32_t label,
 const struct q_useful_buf_c * bstr)
```

Add a binary string claim.

## Parameters

|    |              |                          |
|----|--------------|--------------------------|
| in | <i>me</i>    | Token creation context.  |
| in | <i>label</i> | Integer label for claim. |
| in | <i>bstr</i>  | The binary claim data.   |

Definition at line 338 of file attest\_token\_encode.c.

**8.305.2.2 attest\_token\_encode\_add\_cbor()**

```
void attest_token_encode_add_cbor (
 struct attest_token_encode_ctx * me,
 int32_t label,
 const struct q_useful_buf_c * encoded)
```

Add some already-encoded CBOR to payload.

## Parameters

|    |                |                           |
|----|----------------|---------------------------|
| in | <i>me</i>      | Token creation context.   |
| in | <i>label</i>   | Integer label for claim.  |
| in | <i>encoded</i> | The already-encoded CBOR. |

Encoded CBOR must be a full map or full array or a non-aggregate type. It cannot be a partial map or array. It can be nested maps and arrays, but they must all be complete.

Definition at line 362 of file attest\_token\_encode.c.

**8.305.2.3 attest\_token\_encode\_add\_integer()**

```
void attest_token_encode_add_integer (
 struct attest_token_encode_ctx * me,
 int32_t label,
 int64_t value)
```

Add a 64-bit signed integer claim.

## Parameters

|    |              |                          |
|----|--------------|--------------------------|
| in | <i>me</i>    | Token creation context.  |
| in | <i>label</i> | Integer label for claim. |
| in | <i>value</i> | The integer claim data.  |

Definition at line 327 of file attest\_token\_encode.c.

**8.305.2.4 attest\_token\_encode\_add\_tstr()**

```
void attest_token_encode_add_tstr (
 struct attest_token_encode_ctx * me,
 int32_t label,
 const struct q_useful_buf_c * tstr)
```

Add a text string claim.

## Parameters

|    |              |                          |
|----|--------------|--------------------------|
| in | <i>me</i>    | Token creation context.  |
| in | <i>label</i> | Integer label for claim. |

## Parameters

|    |             |                      |
|----|-------------|----------------------|
| in | <i>tstr</i> | The text claim data. |
|----|-------------|----------------------|

Definition at line 351 of file attest\_token\_encode.c.

**8.305.2.5 attest\_token\_encode\_borrow\_cbor\_cntxt()**

```
QCBOREncodeContext* attest_token_encode_borrow_cbor_cntxt (
 struct attest_token_encode_ctx * me)
```

Get a copy of the CBOR encoding context.

## Parameters

|    |           |                         |
|----|-----------|-------------------------|
| in | <i>me</i> | Token creation context. |
|----|-----------|-------------------------|

## Returns

The CBOR encoding context

Allows the caller to encode CBOR right into the output buffer using any of the `QCBOREncode_AddXXXX()` methods. Anything added here will be part of the payload that gets hashed. This can be used to make complex CBOR structures. All open arrays and maps must be close before calling any other `attest_token_encode` methods. `QCBOREncode_Finish()` should not be closed on this context.

Definition at line 318 of file attest\_token\_encode.c.

**8.305.2.6 attest\_token\_encode\_finish()**

```
enum attest_token_err_t attest_token_encode_finish (
 struct attest_token_encode_ctx * me,
 struct q_useful_buf_c * completed_token)
```

Finish the token, complete the signing and get the result.

## Parameters

|     |                        |                                        |
|-----|------------------------|----------------------------------------|
| in  | <i>me</i>              | Token Creation Context.                |
| out | <i>completed_token</i> | Pointer and length to completed token. |

## Returns

one of the `attest_token_err_t` errors.

This completes the token after the payload has been added. When this is called the signing algorithm is run and the final formatting of the token is completed.

Definition at line 276 of file attest\_token\_encode.c.

**8.305.2.7 attest\_token\_encode\_start()**

```
enum attest_token_err_t attest_token_encode_start (
 struct attest_token_encode_ctx * me,
 int32_t key_select,
 int32_t cose_alg_id,
 const struct q_useful_buf * out_buf)
```

Initialize a token creation context.

## Parameters

|     |                     |                                                                                                                                   |
|-----|---------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>me</i>           | The token creation context to be initialized.                                                                                     |
| in  | <i>key_select</i>   | Selects which attestation key to sign with.                                                                                       |
| in  | <i>cose_alg↔_id</i> | The algorithm to sign with. The IDs are defined in <a href="#">COSE (RFC 8152)</a> or in the <a href="#">IANA COSE Registry</a> . |
| out | <i>out_buf</i>      | The output buffer to write the encoded token into.                                                                                |

## Returns

one of the [attest\\_token\\_err\\_t](#) errors.

The size of the buffer in `out_buf->len` determines the size of the token that can be created. It must be able to hold the final encoded and signed token. The data encoding overhead is just that of CBOR. The signing overhead depends on the signing key size. It is about 150 bytes for 256-bit ECDSA.

If `out_buf->ptr` is NULL and `out_buf->len` is large like `UINT32_MAX` no token will be created but the length of the token that would be created will be in `completed_token` as returned by [attest\\_token\\_encode\\_finish\(\)](#). None of the cryptographic functions run during this, but the sizes of what they would output is taken into account.

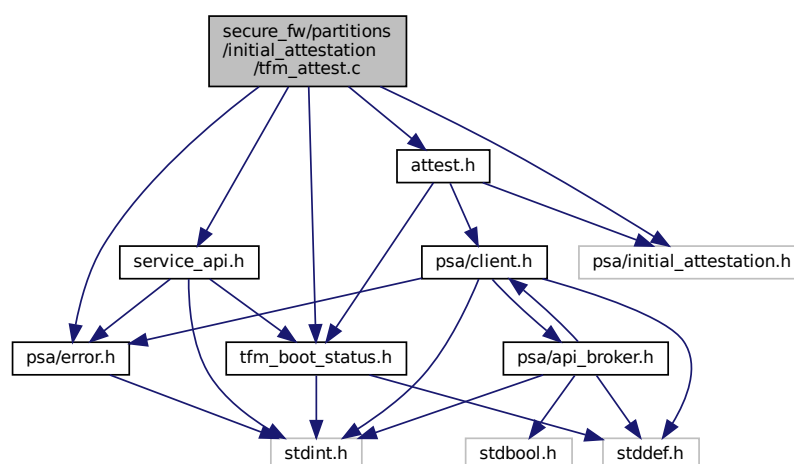
Definition at line 228 of file `attest_token_encode.c`.

## 8.306 secure\_fw/partitions/initial\_attestation/dir\_initial\_attestation.dox File Reference

## 8.307 secure\_fw/partitions/initial\_attestation/tfm\_attest.c File Reference

```
#include "service_api.h"
#include "attest.h"
#include "psa/error.h"
#include "psa/initial_attestation.h"
#include "tfm_boot_status.h"
```

Include dependency graph for `tfm_attest.c`:



## Functions

- enum [psa\\_attest\\_err\\_t](#) `attest_get_caller_client_id` (int32\_t \*caller\_id)



Get the ID of the caller thread.

- enum [psa\\_attest\\_err\\_t](#) [attest\\_get\\_boot\\_data](#) (uint8\_t major\_type, struct [tfm\\_boot\\_data](#) \*boot\_data, uint32\_t len)

Copy the boot data (coming from boot loader) from shared memory area to service memory area.

## Variables

- int32\_t [g\\_attest\\_caller\\_id](#)

## 8.307.1 Function Documentation

### 8.307.1.1 attest\_get\_boot\_data()

```
enum psa_attest_err_t attest_get_boot_data (
 uint8_t major_type,
 struct tfm_boot_data * boot_data,
 uint32_t len)
```

Copy the boot data (coming from boot loader) from shared memory area to service memory area.

#### Parameters

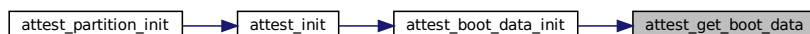
|     |                   |                                              |
|-----|-------------------|----------------------------------------------|
| in  | <i>major_type</i> | Major type of TLV entries to copy            |
| out | <i>boot_data</i>  | Pointer to the buffer to store the boot data |
| in  | <i>len</i>        | Size of the buffer to store the boot data    |

#### Returns

Returns error code as specified in [psa\\_attest\\_err\\_t](#)

Definition at line 26 of file `tfm_attest.c`.

Here is the caller graph for this function:



### 8.307.1.2 attest\_get\_caller\_client\_id()

```
enum psa_attest_err_t attest_get_caller_client_id (
 int32_t * caller_id)
```

Get the ID of the caller thread.

#### Parameters

|     |                  |                                  |
|-----|------------------|----------------------------------|
| out | <i>caller_id</i> | Pointer where to store caller ID |
|-----|------------------|----------------------------------|



## Typedefs

- typedef [psa\\_status\\_t](#)(\* [attest\\_func\\_t](#)) (const [psa\\_msg\\_t](#) \*msg)

## Functions

- [psa\\_status\\_t](#) [tfm\\_attestation\\_service\\_sfn](#) (const [psa\\_msg\\_t](#) \*msg)
- [psa\\_status\\_t](#) [attest\\_partition\\_init](#) (void)

## Variables

- int32\_t [g\\_attest\\_caller\\_id](#)

## 8.308.1 Macro Definition Documentation

### 8.308.1.1 ECC\_P256\_PUBLIC\_KEY\_SIZE

```
#define ECC_P256_PUBLIC_KEY_SIZE PSA_KEY_EXPORT_ECC_PUBLIC_KEY_MAX_SIZE (256)
```

Definition at line 25 of file [tfm\\_attest\\_req\\_mgr.c](#).

## 8.308.2 Typedef Documentation

### 8.308.2.1 attest\_func\_t

```
typedef psa_status_t(* attest_func_t) (const psa_msg_t *msg)
```

Definition at line 27 of file [tfm\\_attest\\_req\\_mgr.c](#).

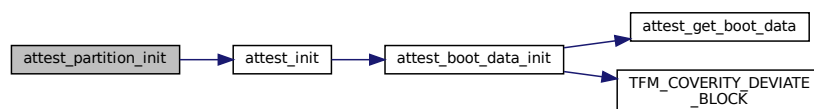
## 8.308.3 Function Documentation

### 8.308.3.1 attest\_partition\_init()

```
psa_status_t attest_partition_init (
 void)
```

Definition at line 157 of file [tfm\\_attest\\_req\\_mgr.c](#).

Here is the call graph for this function:



### 8.308.3.2 tfm\_attestation\_service\_sfn()

```
psa_status_t tfm_attestation_service_sfn (
 const psa_msg_t * msg)
```

Definition at line 145 of file [tfm\\_attest\\_req\\_mgr.c](#).

### 8.308.4 Variable Documentation

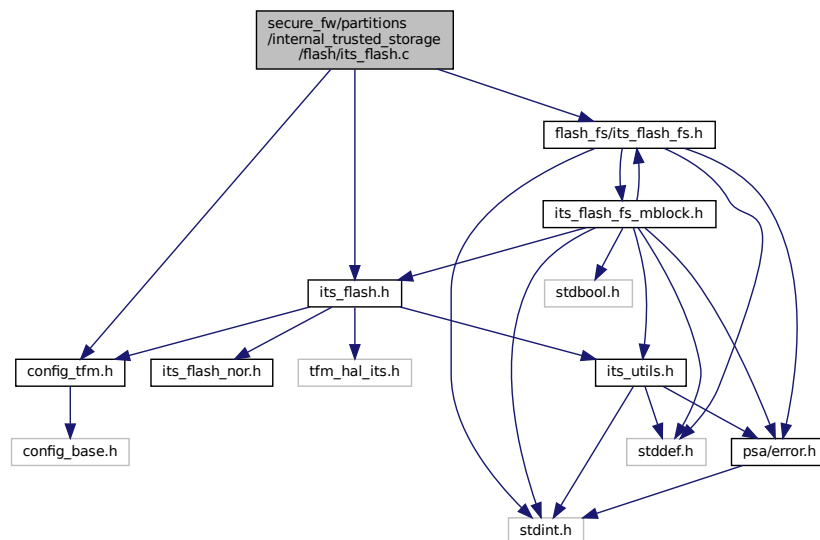
#### 8.308.4.1 g\_attest\_caller\_id

```
int32_t g_attest_caller_id
```

Definition at line 29 of file tfm\_attest\_req\_mgr.c.

### 8.309 secure\_fw/partitions/internal\_trusted\_storage/flash/its\_flash.c File Reference

```
#include "its_flash.h"
#include "config_tfm.h"
#include "flash_fs/its_flash_fs.h"
Include dependency graph for its_flash.c:
```



### 8.310 secure\_fw/partitions/internal\_trusted\_storage/flash/its\_flash.h File Reference

```
#include "config_tfm.h"
#include "its_utils.h"
#include "tfm_hal_its.h"
#include "its_flash_nor.h"
```

```

graph TD
 A["secure_fw/partitions
/internal_trusted_storage
/flash/its_flash.h"] --> B["config_tfm.h"]
 A --> C["its_utils.h"]
 A --> D["tfm_hal_its.h"]
 A --> E["its_flash_nor.h"]
 B --> F["config_base.h"]
 C --> G["stddef.h"]
 C --> H["psa/error.h"]
 H --> I["stdint.h"]

```

- #define ITS\_FLASH\_DEV TFM\_HAL\_ITS\_FLASH\_DRIVER
- #define ITS\_FLASH\_ALIGNMENT TFM\_HAL\_ITS\_PROGRAM\_UNIT
- #define ITS\_FLASH\_OPS its\_flash\_fs\_ops\_nor
- #define PS\_FLASH\_ALIGNMENT 1
- #define ITS\_FLASH\_MAX\_ALIGNMENT

```
#define ITS_FLASH_DEV TFM_HAL_ITS_FLASH_DRIVER
```

Definition at line 44 of file `its_flash.h`.

### 8.310.1.3 ITS\_FLASH\_MAX\_ALIGNMENT

```
#define ITS_FLASH_MAX_ALIGNMENT
```

**Value:**

```
ITS_UTILS_MAX(ITS_FLASH_ALIGNMENT, \
PS_FLASH_ALIGNMENT)
```

Provides a compile-time constant for the maximum program unit required by any flash device that can be accessed through this interface.

Definition at line 86 of file `its_flash.h`.

### 8.310.1.4 ITS\_FLASH\_OPS

```
#define ITS_FLASH_OPS its_flash_fs_ops_nor
```

Definition at line 46 of file `its_flash.h`.

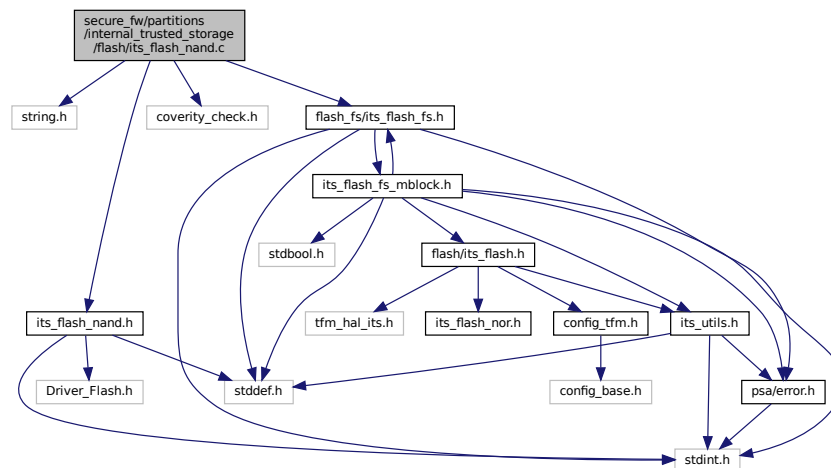
### 8.310.1.5 PS\_FLASH\_ALIGNMENT

```
#define PS_FLASH_ALIGNMENT 1
```

Definition at line 79 of file `its_flash.h`.

## 8.311 secure\_fw/partitions/internal\_trusted\_storage/flash/its\_flash\_nand.c File Reference

```
#include <string.h>
#include "its_flash_nand.h"
#include "coverity_check.h"
#include "flash_fs/its_flash_fs.h"
Include dependency graph for its_flash_nand.c:
```



## Variables

- const struct `its_flash_fs_ops_t` `its_flash_fs_ops_nand`

## 8.311.1 Variable Documentation

### 8.311.1.1 its\_flash\_fs\_ops\_nand

```
const struct its_flash_fs_ops_t its_flash_fs_ops_nand
```

**Initial value:**

```
= {
 .init = its_flash_nand_init,
 .read = its_flash_nand_read,
 .write = its_flash_nand_write,
 .flush = its_flash_nand_flush,
 .erase = its_flash_nand_erase,
}
```

Definition at line 252 of file its\_flash\_nand.c.

## 8.312 secure\_fw/partitions/internal\_trusted\_storage/flash/its\_flash\_nand.h File Reference

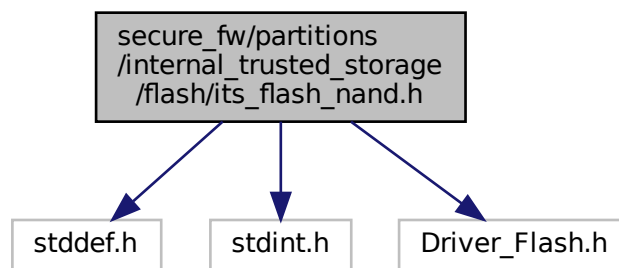
Implementations of the flash interface functions for a NAND flash device. See [its\\_flash\\_fs\\_ops\\_t](#) for full documentation of functions.

```
#include <stddef.h>
```

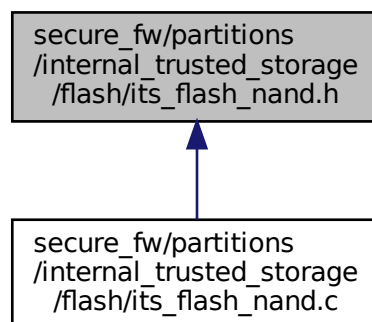
```
#include <stdint.h>
```

```
#include "Driver_Flash.h"
```

Include dependency graph for its\_flash\_nand.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [its\\_flash\\_nand\\_dev\\_t](#)

## Variables

- const struct [its\\_flash\\_fs\\_ops\\_t](#) [its\\_flash\\_fs\\_ops\\_nand](#)

### 8.312.1 Detailed Description

Implementations of the flash interface functions for a NAND flash device. See [its\\_flash\\_fs\\_ops\\_t](#) for full documentation of functions.

### 8.312.2 Variable Documentation

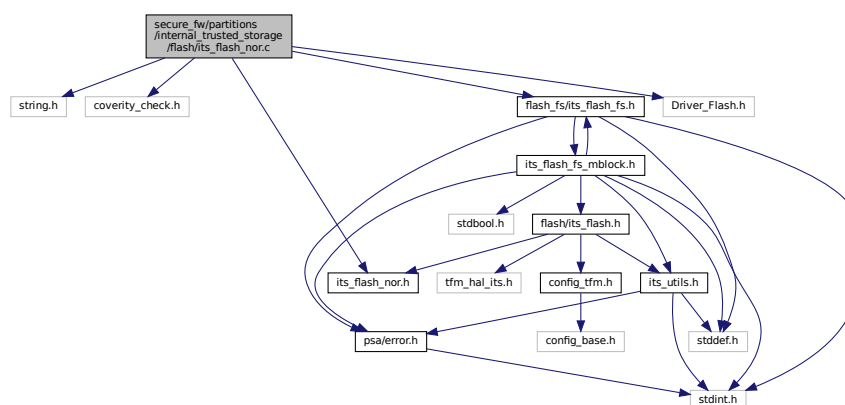
#### 8.312.2.1 its\_flash\_fs\_ops\_nand

const struct [its\\_flash\\_fs\\_ops\\_t](#) [its\\_flash\\_fs\\_ops\\_nand](#)  
Definition at line 252 of file [its\\_flash\\_nand.c](#).

## 8.313 secure\_fw/partitions/internal\_trusted\_storage/flash/its\_flash\_nor.c File Reference

```
#include <string.h>
#include "coverity_check.h"
#include "its_flash_nor.h"
#include "flash_fs/its_flash_fs.h"
#include "Driver_Flash.h"
```

Include dependency graph for [its\\_flash\\_nor.c](#):



## Variables

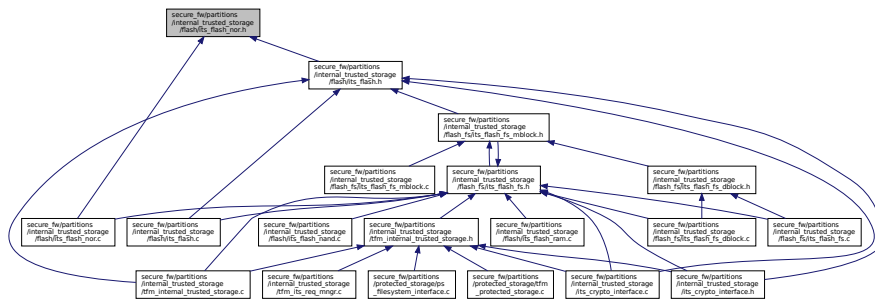
- const struct [its\\_flash\\_fs\\_ops\\_t](#) [its\\_flash\\_fs\\_ops\\_nor](#)

### 8.313.1 Variable Documentation



Definition at line 200 of file its\_flash\_nor.c.

This graph shows which files directly or indirectly include this file:



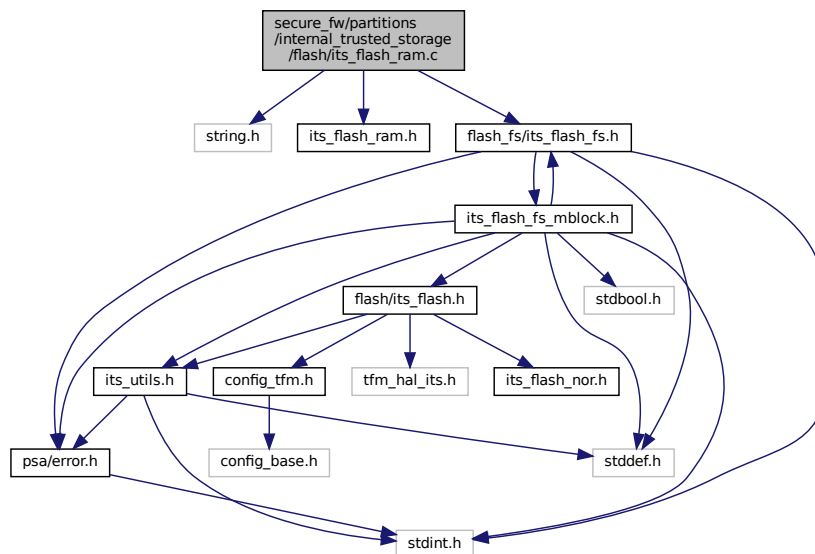
- `const struct its_flash_fs_ops t its_flash_fs_ops_nor`

Implementations of the flash interface functions for a NOR flash device. See [its\\_flash\\_fs\\_ops\\_t](#) for full documentation of functions.

Definition at line 200 of file its\_flash\_nor.c.

```
#include <string.h>
#include "its_flash_ram.h"
```

```
#include "flash_fs/its_flash_fs.h"
Include dependency graph for its_flash_ram.c:
```



## Variables

- const struct [its\\_flash\\_fs\\_ops\\_t](#) [its\\_flash\\_fs\\_ops\\_ram](#)

## 8.315.1 Variable Documentation

### 8.315.1.1 its\_flash\_fs\_ops\_ram

```
const struct its_flash_fs_ops_t its_flash_fs_ops_ram
```

Initial value:

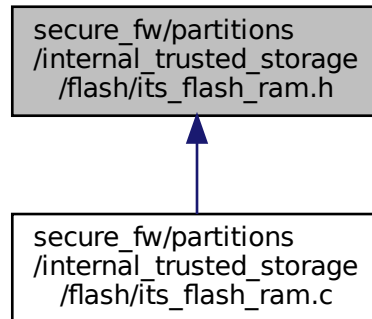
```
= {
 .init = its_flash_ram_init,
 .read = its_flash_ram_read,
 .write = its_flash_ram_write,
 .flush = its_flash_ram_flush,
 .erase = its_flash_ram_erase,
}
```

Definition at line 79 of file [its\\_flash\\_ram.c](#).

## 8.316 [secure\\_fw/partitions/internal\\_trusted\\_storage/flash/its\\_flash\\_ram.h](#) File Reference

Implementations of the flash interface functions for an emulated flash device using RAM. See [its\\_flash\\_fs\\_ops\\_t](#) for full documentation of functions.

This graph shows which files directly or indirectly include this file:



## Variables

- const struct [its\\_flash\\_fs\\_ops\\_t](#) `its_flash_fs_ops_ram`

### 8.316.1 Detailed Description

Implementations of the flash interface functions for an emulated flash device using RAM. See [its\\_flash\\_fs\\_ops\\_t](#) for full documentation of functions.

### 8.316.2 Variable Documentation

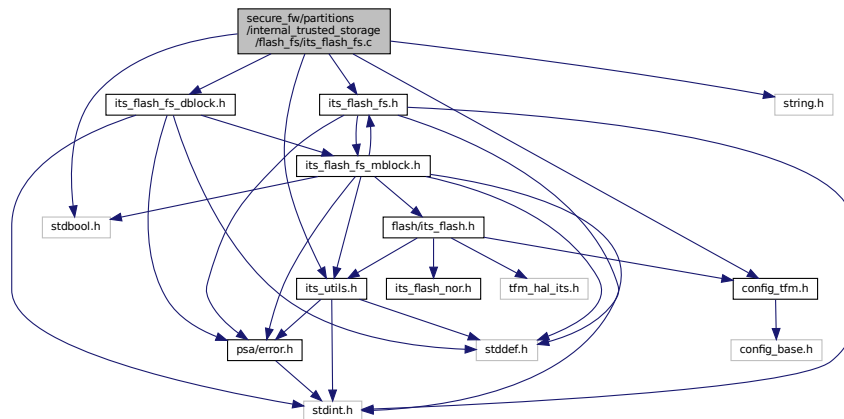
#### 8.316.2.1 `its_flash_fs_ops_ram`

const struct [its\\_flash\\_fs\\_ops\\_t](#) `its_flash_fs_ops_ram`  
Definition at line 79 of file `its_flash_ram.c`.

## 8.317 secure\_fw/partitions/internal\_trusted\_storage/flash\_fs/its\_flash\_↵ fs.c File Reference

```
#include <stdbool.h>
#include <string.h>
#include "config_tfm.h"
#include "its_flash_fs.h"
#include "its_flash_fs_dblock.h"
#include "its_utils.h"
```

Include dependency graph for `its_flash_fs.c`:



## Macros

- `#define ITS_FLASH_FS_INTERNAL_FLAGS_MASK (UINT32_MAX - ((1U << 24) - 1))`
- `#define ITS_FLASH_FS_FLAG_DELETE (1U << 24)`

## Functions

- `psa_status_t its_flash_fs_init_ctx` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const struct `its_flash_fs_config_t` \*fs\_cfg, const struct `its_flash_fs_ops_t` \*fs\_ops)  
*Initialises the filesystem context. Must be called successfully before any other filesystem API is called.*
- `psa_status_t its_flash_fs_prepare` (struct `its_flash_fs_ctx_t` \*fs\_ctx)  
*Prepares the filesystem to accept operations on the files.*
- `psa_status_t its_flash_fs_wipe_all` (struct `its_flash_fs_ctx_t` \*fs\_ctx)  
*Wipes all files from the filesystem.*
- `psa_status_t its_flash_fs_file_get_info` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const uint8\_t \*fid, struct `its_flash_fs_file_info_t` \*info)  
*Gets the file information referenced by the file ID.*
- `psa_status_t its_flash_fs_file_write` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const uint8\_t \*fid, struct `its_flash_fs_file_info_t` \*info, size\_t data\_size, size\_t offset, const uint8\_t \*data)  
*Writes data to a file.*
- `psa_status_t its_flash_fs_file_delete` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const uint8\_t \*fid)  
*Deletes file referenced by the file ID.*
- `psa_status_t its_flash_fs_file_read` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const uint8\_t \*fid, size\_t size, size\_t offset, uint8\_t \*data)  
*Reads data from an existing file.*

## 8.317.1 Macro Definition Documentation

### 8.317.1.1 ITS\_FLASH\_FS\_FLAG\_DELETE

```
#define ITS_FLASH_FS_FLAG_DELETE (1U << 24)
```

Definition at line 22 of file `its_flash_fs.c`.

### 8.317.1.2 ITS\_FLASH\_FS\_INTERNAL\_FLAGS\_MASK

```
#define ITS_FLASH_FS_INTERNAL_FLAGS_MASK (UINT32_MAX - ((1U << 24) - 1))
```

Definition at line 20 of file `its_flash_fs.c`.

## 8.317.2 Function Documentation

### 8.317.2.1 its\_flash\_fs\_file\_delete()

```
psa_status_t its_flash_fs_file_delete (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid)
```

Deletes file referenced by the file ID.

#### Parameters

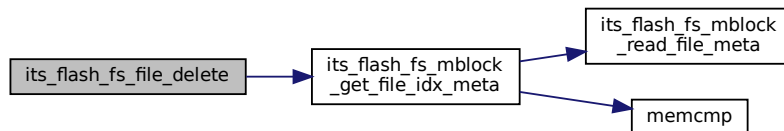
|         |               |                    |
|---------|---------------|--------------------|
| in, out | <i>fs_ctx</i> | Filesystem context |
| in      | <i>fid</i>    | File ID            |

#### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 555 of file `its_flash_fs.c`.

Here is the call graph for this function:



### 8.317.2.2 its\_flash\_fs\_file\_get\_info()

```
psa_status_t its_flash_fs_file_get_info (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 struct its_flash_fs_file_info_t * info)
```

Gets the file information referenced by the file ID.

#### Parameters

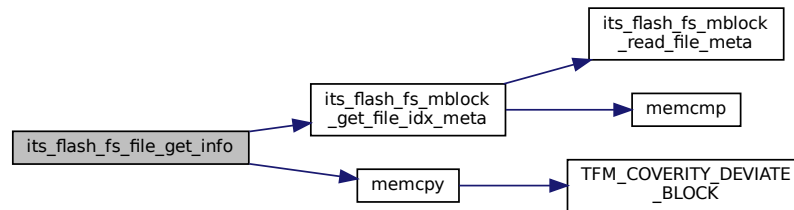
|         |               |                                                                                    |
|---------|---------------|------------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i> | Filesystem context                                                                 |
| in      | <i>fid</i>    | File ID                                                                            |
| out     | <i>info</i>   | Pointer to the file information structure <a href="#">its_flash_fs_file_info_t</a> |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 199 of file `its_flash_fs.c`.

Here is the call graph for this function:

**8.317.2.3 its\_flash\_fs\_file\_read()**

```

psa_status_t its_flash_fs_file_read (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 size_t size,
 size_t offset,
 uint8_t * data)

```

Reads data from an existing file.

**Parameters**

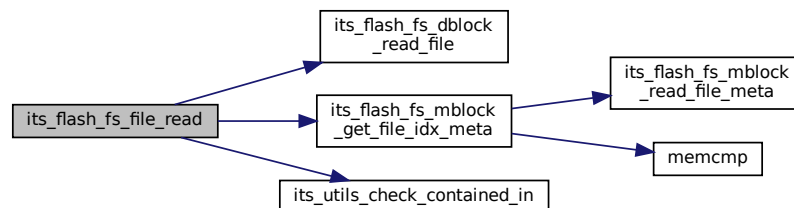
|         |               |                                     |
|---------|---------------|-------------------------------------|
| in, out | <i>fs_ctx</i> | Filesystem context                  |
| in      | <i>fid</i>    | File ID                             |
| in      | <i>size</i>   | Size to be read                     |
| in      | <i>offset</i> | Offset in the file                  |
| out     | <i>data</i>   | Pointer to buffer to store the data |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 570 of file `its_flash_fs.c`.

Here is the call graph for this function:



### 8.317.2.4 its\_flash\_fs\_file\_write()

```
psa_status_t its_flash_fs_file_write (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 struct its_flash_fs_file_info_t * finfo,
 size_t data_size,
 size_t offset,
 const uint8_t * data)
```

Writes data to a file.

#### Parameters

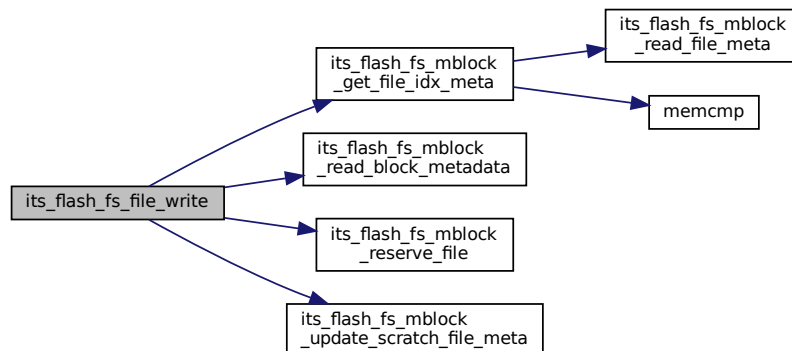
|         |                  |                                                                                   |
|---------|------------------|-----------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context                                                                |
| in      | <i>fid</i>       | File ID                                                                           |
| in      | <i>finfo</i>     | Pointer to <a href="#">its_flash_fs_file_info_t</a>                               |
| in      | <i>data_size</i> | Size of the incoming write data.                                                  |
| in      | <i>offset</i>    | Offset in the file to write. Must be less than or equal to the current file size. |
| in      | <i>data</i>      | Pointer to buffer containing data to be written                                   |

#### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 224 of file `its_flash_fs.c`.

Here is the call graph for this function:



### 8.317.2.5 its\_flash\_fs\_init\_ctx()

```
psa_status_t its_flash_fs_init_ctx (
 struct its_flash_fs_ctx_t * fs_ctx,
 const struct its_flash_fs_config_t * fs_cfg,
 const struct its_flash_fs_ops_t * fs_ops)
```

Initialises the filesystem context. Must be called successfully before any other filesystem API is called.

#### Parameters

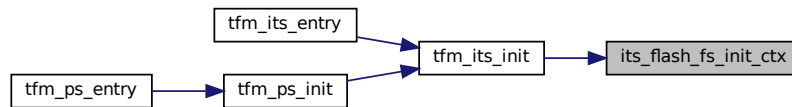
|         |               |                                                                           |
|---------|---------------|---------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i> | Filesystem context to initialise. Must have been allocated by the caller. |
| in      | <i>fs_cfg</i> | Filesystem configuration to associate with the context.                   |
| in      | <i>fs_ops</i> | Filesystem flash operations to associate with the context.                |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 142 of file `its_flash_fs.c`.

Here is the caller graph for this function:

**8.317.2.6 its\_flash\_fs\_prepare()**

```

psa_status_t its_flash_fs_prepare (
 struct its_flash_fs_ctx_t * fs_ctx)

```

Prepares the filesystem to accept operations on the files.

**Parameters**

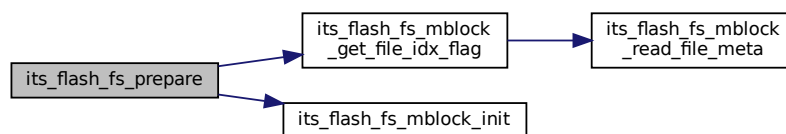
|                      |                     |                                                                                                        |
|----------------------|---------------------|--------------------------------------------------------------------------------------------------------|
| <code>in, out</code> | <code>fs_ctx</code> | Filesystem context to prepare. Must have been initialised by <a href="#">its_flash_fs_init_ctx()</a> . |
|----------------------|---------------------|--------------------------------------------------------------------------------------------------------|

**Returns**

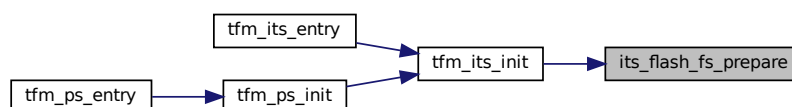
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 168 of file `its_flash_fs.c`.

Here is the call graph for this function:



Here is the caller graph for this function:





### 8.317.2.7 its\_flash\_fs\_wipe\_all()

```
psa_status_t its_flash_fs_wipe_all (
 struct its_flash_fs_ctx_t * fs_ctx)
```

Wipes all files from the filesystem.

#### Parameters

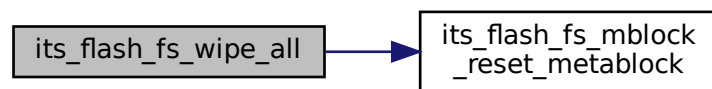
|         |        |                                                                        |
|---------|--------|------------------------------------------------------------------------|
| in, out | fs_ctx | Filesystem context to wipe. Must be prepared again before further use. |
|---------|--------|------------------------------------------------------------------------|

#### Returns

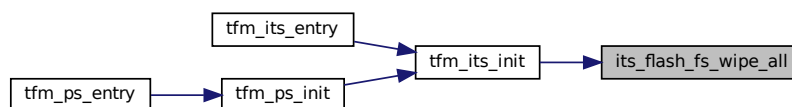
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 193 of file `its_flash_fs.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



## 8.318 secure\_fw/partitions/internal\_trusted\_storage/flash\_fs/its\_flash\_fs.h File Reference

Internal Trusted Storage service filesystem abstraction APIs. The purpose of this abstraction is to have the ability to plug-in other filesystems or filesystem proxies (supplciant).

```
#include <stddef.h>
#include <stdint.h>
#include "its_flash_fs_mblock.h"
#include "psa/error.h"
```



## Functions

- [psa\\_status\\_t its\\_flash\\_fs\\_init\\_ctx](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const struct [its\\_flash\\_fs\\_config\\_t](#) \*fs\_cfg, const struct [its\\_flash\\_fs\\_ops\\_t](#) \*fs\_ops)  
*Initialises the filesystem context. Must be called successfully before any other filesystem API is called.*
- [psa\\_status\\_t its\\_flash\\_fs\\_prepare](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)  
*Prepares the filesystem to accept operations on the files.*
- [psa\\_status\\_t its\\_flash\\_fs\\_wipe\\_all](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)  
*Wipes all files from the filesystem.*
- [psa\\_status\\_t its\\_flash\\_fs\\_file\\_get\\_info](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const uint8\_t \*fid, struct [its\\_flash\\_fs\\_file\\_info\\_t](#) \*info)  
*Gets the file information referenced by the file ID.*
- [psa\\_status\\_t its\\_flash\\_fs\\_file\\_write](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const uint8\_t \*fid, struct [its\\_flash\\_fs\\_file\\_info\\_t](#) \*info, size\_t data\_size, size\_t offset, const uint8\_t \*data)  
*Writes data to a file.*
- [psa\\_status\\_t its\\_flash\\_fs\\_file\\_read](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const uint8\_t \*fid, size\_t size, size\_t offset, uint8\_t \*data)  
*Reads data from an existing file.*
- [psa\\_status\\_t its\\_flash\\_fs\\_file\\_delete](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const uint8\_t \*fid)  
*Deletes file referenced by the file ID.*

### 8.318.1 Detailed Description

Internal Trusted Storage service filesystem abstraction APIs. The purpose of this abstraction is to have the ability to plug-in other filesystems or filesystem proxies (suppliment).

### 8.318.2 Macro Definition Documentation

#### 8.318.2.1 ITS\_BLOCK\_INVALID\_ID

```
#define ITS_BLOCK_INVALID_ID 0xFFFFFFFFU
```

Definition at line 44 of file [its\\_flash\\_fs.h](#).

#### 8.318.2.2 ITS\_FLASH\_FS\_FLAG\_CREATE

```
#define ITS_FLASH_FS_FLAG_CREATE (1UL << 16)
```

Definition at line 39 of file [its\\_flash\\_fs.h](#).

#### 8.318.2.3 ITS\_FLASH\_FS\_FLAG\_TRUNCATE

```
#define ITS_FLASH_FS_FLAG_TRUNCATE (1UL << 17)
```

Definition at line 41 of file [its\\_flash\\_fs.h](#).

#### 8.318.2.4 ITS\_FLASH\_FS\_USER\_FLAGS\_MASK

```
#define ITS_FLASH_FS_USER_FLAGS_MASK ((1UL << 16) - 1)
```

Definition at line 34 of file [its\\_flash\\_fs.h](#).

#### 8.318.2.5 ITS\_FLASH\_FS\_WRITE\_FLAGS\_MASK

```
#define ITS_FLASH_FS_WRITE_FLAGS_MASK ((1UL << 24) - (1UL << 16))
```

Definition at line 37 of file [its\\_flash\\_fs.h](#).

### 8.318.3 Typedef Documentation

#### 8.318.3.1 `its_flash_fs_ctx_t`

`typedef struct its\_flash\_fs\_ctx\_t its\_flash\_fs\_ctx\_t`

ITS flash filesystem context type, used to maintain state across FS operations.

The user should allocate a variable of this type, initialised to zero, before calling `its_flash_fs_prepare`, and then pass it to each subsequent FS operation. The contents are internal to the filesystem.

Definition at line 168 of file `its_flash_fs.h`.

### 8.318.4 Function Documentation

#### 8.318.4.1 `its_flash_fs_file_delete()`

```
psa_status_t its_flash_fs_file_delete (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid)
```

Deletes file referenced by the file ID.

##### Parameters

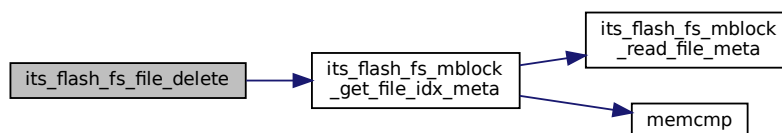
|         |               |                    |
|---------|---------------|--------------------|
| in, out | <i>fs_ctx</i> | Filesystem context |
| in      | <i>fid</i>    | File ID            |

##### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 555 of file `its_flash_fs.c`.

Here is the call graph for this function:



#### 8.318.4.2 `its_flash_fs_file_get_info()`

```
psa_status_t its_flash_fs_file_get_info (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 struct its_flash_fs_file_info_t * info)
```

Gets the file information referenced by the file ID.

##### Parameters

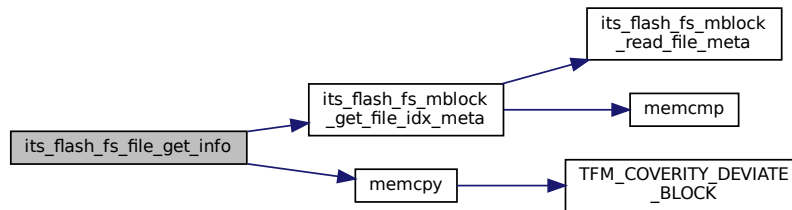
|         |               |                                                                                    |
|---------|---------------|------------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i> | Filesystem context                                                                 |
| in      | <i>fid</i>    | File ID                                                                            |
| out     | <i>info</i>   | Pointer to the file information structure <a href="#">its_flash_fs_file_info_t</a> |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 199 of file `its_flash_fs.c`.

Here is the call graph for this function:

**8.318.4.3 its\_flash\_fs\_file\_read()**

```

psa_status_t its_flash_fs_file_read (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 size_t size,
 size_t offset,
 uint8_t * data)

```

Reads data from an existing file.

**Parameters**

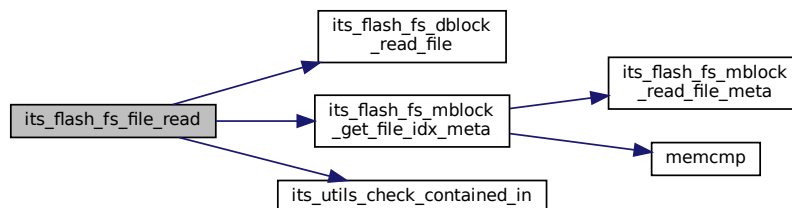
|         |               |                                     |
|---------|---------------|-------------------------------------|
| in, out | <i>fs_ctx</i> | Filesystem context                  |
| in      | <i>fid</i>    | File ID                             |
| in      | <i>size</i>   | Size to be read                     |
| in      | <i>offset</i> | Offset in the file                  |
| out     | <i>data</i>   | Pointer to buffer to store the data |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 570 of file `its_flash_fs.c`.

Here is the call graph for this function:



#### 8.318.4.4 `its_flash_fs_file_write()`

```
psa_status_t its_flash_fs_file_write (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 struct its_flash_fs_file_info_t * finfo,
 size_t data_size,
 size_t offset,
 const uint8_t * data)
```

Writes data to a file.

##### Parameters

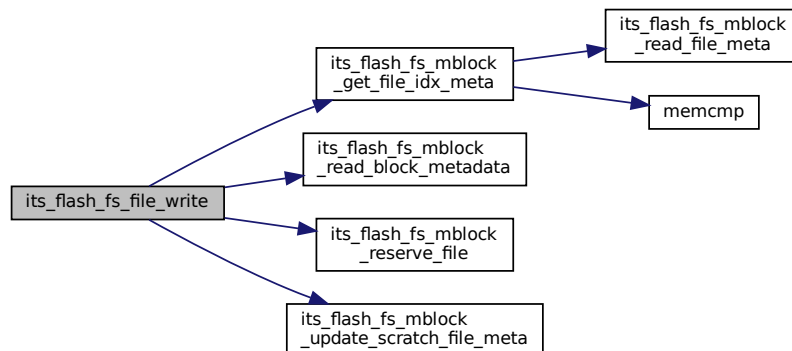
|         |                  |                                                                                   |
|---------|------------------|-----------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context                                                                |
| in      | <i>fid</i>       | File ID                                                                           |
| in      | <i>finfo</i>     | Pointer to <a href="#">its_flash_fs_file_info_t</a>                               |
| in      | <i>data_size</i> | Size of the incoming write data.                                                  |
| in      | <i>offset</i>    | Offset in the file to write. Must be less than or equal to the current file size. |
| in      | <i>data</i>      | Pointer to buffer containing data to be written                                   |

##### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 224 of file `its_flash_fs.c`.

Here is the call graph for this function:



#### 8.318.4.5 `its_flash_fs_init_ctx()`

```
psa_status_t its_flash_fs_init_ctx (
 struct its_flash_fs_ctx_t * fs_ctx,
 const struct its_flash_fs_config_t * fs_cfg,
 const struct its_flash_fs_ops_t * fs_ops)
```

Initialises the filesystem context. Must be called successfully before any other filesystem API is called.

##### Parameters

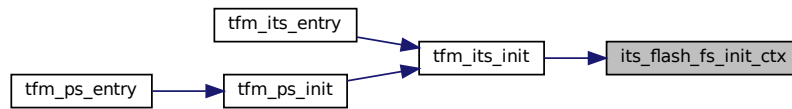
|         |               |                                                                           |
|---------|---------------|---------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i> | Filesystem context to initialise. Must have been allocated by the caller. |
| in      | <i>fs_cfg</i> | Filesystem configuration to associate with the context.                   |
| in      | <i>fs_ops</i> | Filesystem flash operations to associate with the context.                |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 142 of file `its_flash_fs.c`.

Here is the caller graph for this function:

**8.318.4.6 its\_flash\_fs\_prepare()**

```

psa_status_t its_flash_fs_prepare (
 struct its_flash_fs_ctx_t * fs_ctx)

```

Prepares the filesystem to accept operations on the files.

**Parameters**

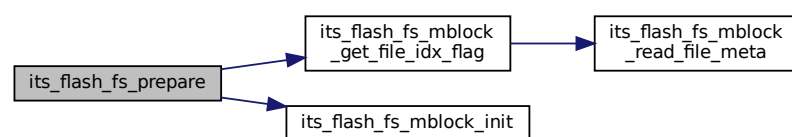
|                      |                     |                                                                                                        |
|----------------------|---------------------|--------------------------------------------------------------------------------------------------------|
| <code>in, out</code> | <code>fs_ctx</code> | Filesystem context to prepare. Must have been initialised by <a href="#">its_flash_fs_init_ctx()</a> . |
|----------------------|---------------------|--------------------------------------------------------------------------------------------------------|

**Returns**

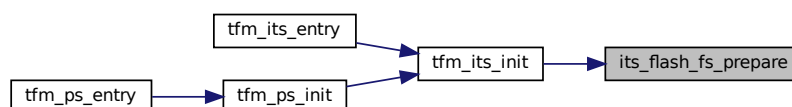
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 168 of file `its_flash_fs.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.318.4.7 its\_flash\_fs\_wipe\_all()

```
psa_status_t its_flash_fs_wipe_all (
 struct its_flash_fs_ctx_t * fs_ctx)
```

Wipes all files from the filesystem.

#### Parameters

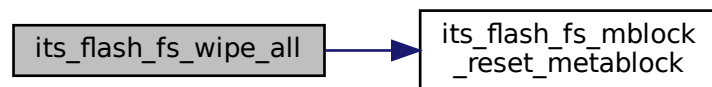
|         |        |                                                                        |
|---------|--------|------------------------------------------------------------------------|
| in, out | fs_ctx | Filesystem context to wipe. Must be prepared again before further use. |
|---------|--------|------------------------------------------------------------------------|

#### Returns

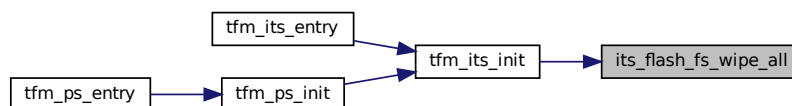
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 193 of file `its_flash_fs.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

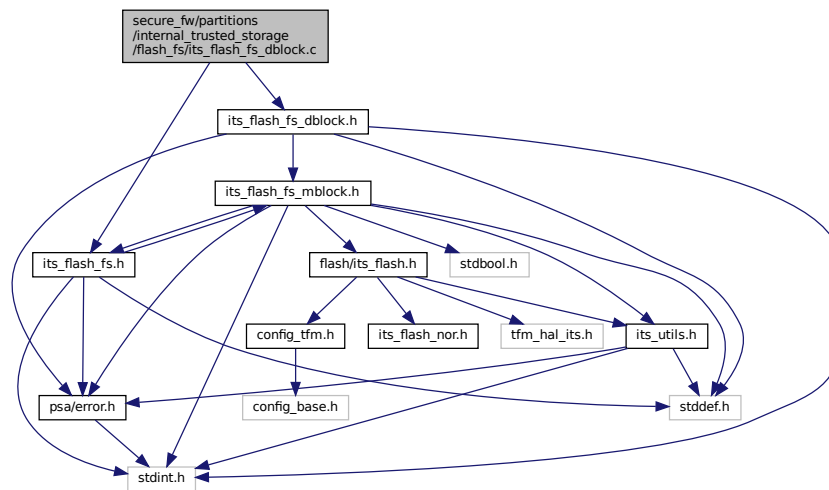


## 8.319 secure\_fw/partitions/internal\_trusted\_storage/flash\_fs/its\_flash\_fs\_dblock.c File Reference

```
#include "its_flash_fs_dblock.h"
#include "its_flash_fs.h"
```



Include dependency graph for its\_flash\_fs\_dblock.c:



## Functions

- `psa_status_t its_flash_fs_dblock_compact_block` (struct `its_flash_fs_ctx_t` \*fs\_ctx, uint32\_t lblock, size\_t free\_size, size\_t src\_offset, size\_t dst\_offset, size\_t size)  
*Compacts block data for the given logical block.*
- `psa_status_t its_flash_fs_dblock_read_file` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const struct `its_file_meta_t` \*file\_meta, size\_t offset, size\_t size, uint8\_t \*buf)  
*Reads the file content.*
- `psa_status_t its_flash_fs_dblock_write_file` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const struct `its_block_meta_t` \*block\_meta, const struct `its_file_meta_t` \*file\_meta, size\_t offset, size\_t size, const uint8\_t \*data)  
*Writes scratch data block content with requested data and the rest of the data from the given logical block.*

## 8.319.1 Function Documentation

### 8.319.1.1 its\_flash\_fs\_dblock\_compact\_block()

```

psa_status_t its_flash_fs_dblock_compact_block (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock,
 size_t free_size,
 size_t src_offset,
 size_t dst_offset,
 size_t size)

```

Compacts block data for the given logical block.

#### Parameters

|         |                   |                                                                                                   |
|---------|-------------------|---------------------------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context                                                                                |
| in      | <i>lblock</i>     | Logical data block to compact                                                                     |
| in      | <i>free_size</i>  | Available data size to compact                                                                    |
| in      | <i>src_offset</i> | Offset in the current data block which points to the data position to reallocate                  |
| in      | <i>dst_offset</i> | Offset in the scratch block which points to the data position to store the data to be reallocated |

## Parameters

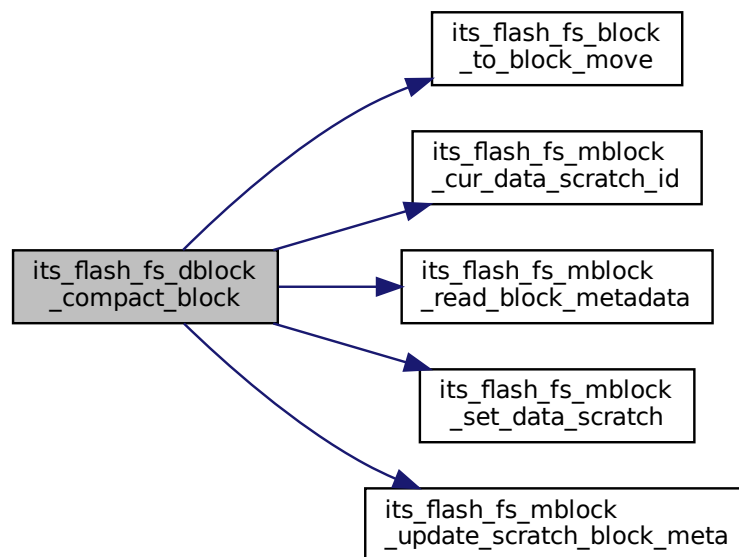
|    |      |                                   |
|----|------|-----------------------------------|
| in | size | Number of bytes to be reallocated |
|----|------|-----------------------------------|

## Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 34 of file `its_flash_fs_dblock.c`.

Here is the call graph for this function:

8.319.1.2 `its_flash_fs_dblock_read_file()`

```

psa_status_t its_flash_fs_dblock_read_file (
 struct its_flash_fs_ctx_t * fs_ctx,
 const struct its_file_meta_t * file_meta,
 size_t offset,
 size_t size,
 uint8_t * buf)

```

Reads the file content.

## Parameters

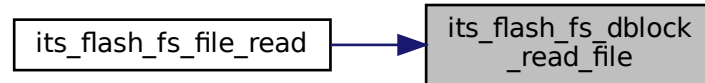
|         |                  |                                  |
|---------|------------------|----------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context               |
| in      | <i>file_meta</i> | File metadata                    |
| in      | <i>offset</i>    | Offset in the file               |
| in      | <i>size</i>      | Size to be read                  |
| out     | <i>buf</i>       | Buffer pointer to store the data |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 117 of file `its_flash_fs_dblock.c`.

Here is the caller graph for this function:

**8.319.1.3 its\_flash\_fs\_dblock\_write\_file()**

```

psa_status_t its_flash_fs_dblock_write_file (
 struct its_flash_fs_ctx_t * fs_ctx,
 const struct its_block_meta_t * block_meta,
 const struct its_file_meta_t * file_meta,
 size_t offset,
 size_t size,
 const uint8_t * data)

```

Writes scratch data block content with requested data and the rest of the data from the given logical block.

**Parameters**

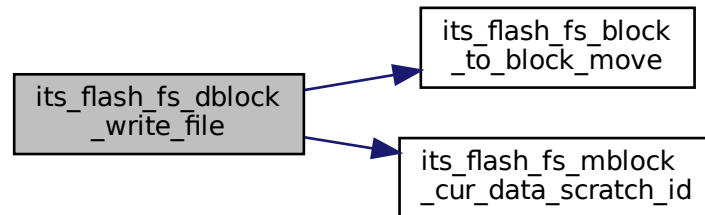
|         |                   |                                                                               |
|---------|-------------------|-------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context                                                            |
| in      | <i>block_meta</i> | Block metadata                                                                |
| in      | <i>file_meta</i>  | File metadata                                                                 |
| in      | <i>offset</i>     | Offset in the scratch data block where to start the copy of the incoming data |
| in      | <i>size</i>       | Size of the incoming data                                                     |
| in      | <i>data</i>       | Pointer to data buffer to copy in the scratch data block                      |

## Returns

Returns error code as specified in [psa\\_status\\_t](#)

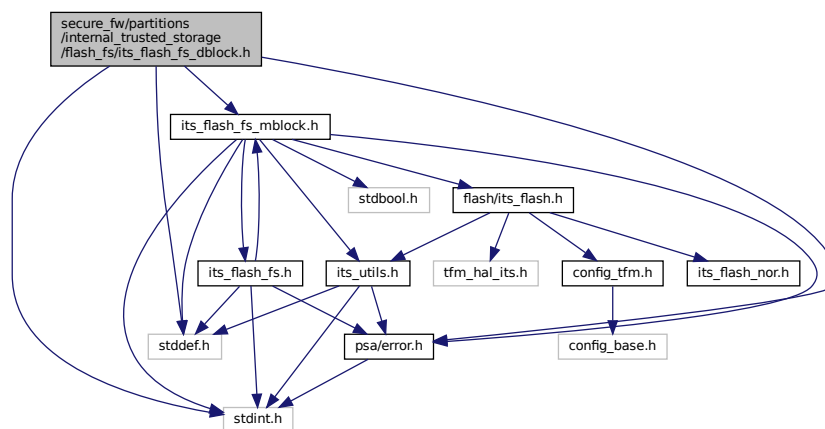
Definition at line 137 of file `its_flash_fs_dblock.c`.

Here is the call graph for this function:

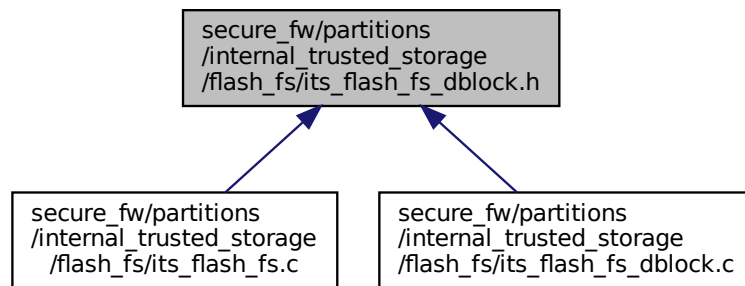


## 8.320 secure\_fw/partitions/internal\_trusted\_storage/flash\_fs/its\_flash\_fs\_dblock.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "psa/error.h"
#include "its_flash_fs_mblock.h"
Include dependency graph for its_flash_fs_dblock.h:
```



This graph shows which files directly or indirectly include this file:



## Functions

- `psa_status_t its_flash_fs_dblock_compact_block` (struct `its_flash_fs_ctx_t` \*fs\_ctx, uint32\_t lblock, size\_t free\_size, size\_t src\_offset, size\_t dst\_offset, size\_t size)  
*Compacts block data for the given logical block.*
- `psa_status_t its_flash_fs_dblock_read_file` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const struct `its_file_meta_t` \*file\_meta, size\_t offset, size\_t size, uint8\_t \*buf)  
*Reads the file content.*
- `psa_status_t its_flash_fs_dblock_write_file` (struct `its_flash_fs_ctx_t` \*fs\_ctx, const struct `its_block_meta_t` \*block\_meta, const struct `its_file_meta_t` \*file\_meta, size\_t offset, size\_t size, const uint8\_t \*data)  
*Writes scratch data block content with requested data and the rest of the data from the given logical block.*

## 8.320.1 Function Documentation

### 8.320.1.1 its\_flash\_fs\_dblock\_compact\_block()

```

psa_status_t its_flash_fs_dblock_compact_block (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock,
 size_t free_size,
 size_t src_offset,
 size_t dst_offset,
 size_t size)

```

Compacts block data for the given logical block.

#### Parameters

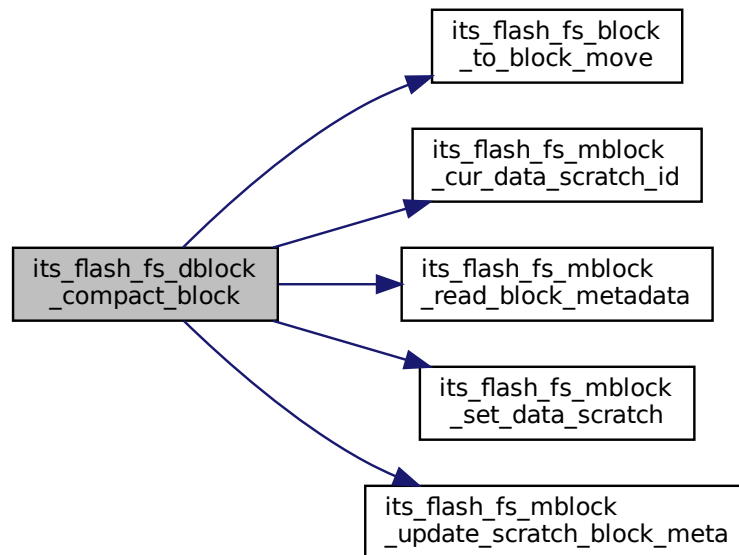
|         |                   |                                                                                                   |
|---------|-------------------|---------------------------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context                                                                                |
| in      | <i>lblock</i>     | Logical data block to compact                                                                     |
| in      | <i>free_size</i>  | Available data size to compact                                                                    |
| in      | <i>src_offset</i> | Offset in the current data block which points to the data position to reallocate                  |
| in      | <i>dst_offset</i> | Offset in the scratch block which points to the data position to store the data to be reallocated |
| in      | <i>size</i>       | Number of bytes to be reallocated                                                                 |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 34 of file `its_flash_fs_dblock.c`.

Here is the call graph for this function:

**8.320.1.2 its\_flash\_fs\_dblock\_read\_file()**

```

psa_status_t its_flash_fs_dblock_read_file (
 struct its_flash_fs_ctx_t * fs_ctx,
 const struct its_file_meta_t * file_meta,
 size_t offset,
 size_t size,
 uint8_t * buf)

```

Reads the file content.

**Parameters**

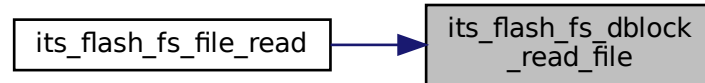
|         |                  |                                  |
|---------|------------------|----------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context               |
| in      | <i>file_meta</i> | File metadata                    |
| in      | <i>offset</i>    | Offset in the file               |
| in      | <i>size</i>      | Size to be read                  |
| out     | <i>buf</i>       | Buffer pointer to store the data |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 117 of file `its_flash_fs_dblock.c`.

Here is the caller graph for this function:

**8.320.1.3 its\_flash\_fs\_dblock\_write\_file()**

```

psa_status_t its_flash_fs_dblock_write_file (
 struct its_flash_fs_ctx_t * fs_ctx,
 const struct its_block_meta_t * block_meta,
 const struct its_file_meta_t * file_meta,
 size_t offset,
 size_t size,
 const uint8_t * data)

```

Writes scratch data block content with requested data and the rest of the data from the given logical block.

**Parameters**

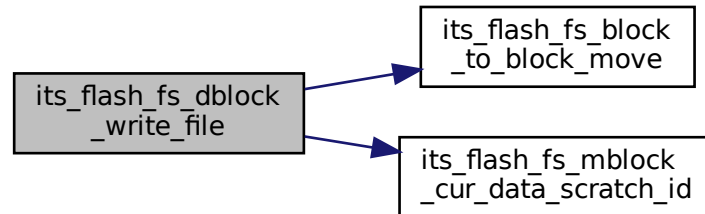
|         |                   |                                                                               |
|---------|-------------------|-------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context                                                            |
| in      | <i>block_meta</i> | Block metadata                                                                |
| in      | <i>file_meta</i>  | File metadata                                                                 |
| in      | <i>offset</i>     | Offset in the scratch data block where to start the copy of the incoming data |
| in      | <i>size</i>       | Size of the incoming data                                                     |
| in      | <i>data</i>       | Pointer to data buffer to copy in the scratch data block                      |

## Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 137 of file `its_flash_fs_dblock.c`.

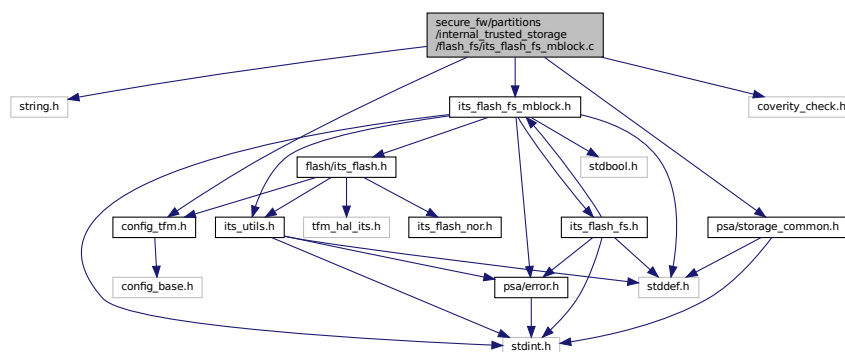
Here is the call graph for this function:



## 8.321 secure\_fw/partitions/internal\_trusted\_storage/flash\_fs/its\_flash\_fs\_mblock.c File Reference

```
#include <string.h>
#include "config_tfm.h"
#include "its_flash_fs_mblock.h"
#include "psa/storage_common.h"
#include "coverity_check.h"
```

Include dependency graph for `its_flash_fs_mblock.c`:



## Macros

- `#define ITS_MAX_BLOCK_DATA_COPY 256`
- `#define ITS_METADATA_BLOCK0 0`
- `#define ITS_METADATA_BLOCK1 1`
- `#define ITS_OTHER_META_BLOCK(metablock)`  
Macro to get the the swap metadata block.
- `#define ITS_BLOCK_META_HEADER_SIZE sizeof(struct its_metadata_block_header_t)`
- `#define ITS_BLOCK_METADATA_SIZE sizeof(struct its_block_meta_t)`
- `#define ITS_FILE_METADATA_SIZE sizeof(struct its_file_meta_t)`



## Functions

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_cp\\_file\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t idx\_start, uint32\_t idx\_end)  
*Copies the file metadata entries between two indexes from the active metadata block to the scratch metadata block.*
- [uint32\\_t its\\_flash\\_fs\\_mblock\\_cur\\_data\\_scratch\\_id](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t lblock)  
*Gets current scratch datablock physical ID.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_get\\_file\\_idx\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const uint8\_t \*fid, uint32\_t \*idx, struct [its\\_file\\_meta\\_t](#) \*file\_meta)  
*Gets file metadata entry index and file metadata.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_get\\_file\\_idx\\_flag](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t flags, uint32\_t \*idx)  
*Gets file metadata entry index of the first file with one of the provided flags set.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_init](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)  
*Initializes metadata block with the valid/active metablock.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_meta\\_update\\_finalize](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)  
*Finalizes an update operation. Last step when a create/write/delete is performed.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_migrate\\_lb0\\_data\\_to\\_scratch](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)  
*Writes the files data area of logical block 0 into the scratch block.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_read\\_file\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t idx, struct [its\\_file\\_meta\\_t](#) \*file\_meta)  
*Reads specified file metadata.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_read\\_block\\_metadata](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t lblock, struct [its\\_block\\_meta\\_t](#) \*block\_meta)  
*Reads specified logical block metadata.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_read\\_block\\_metadata\\_comp](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t lblock, struct [its\\_block\\_meta\\_t](#) \*block\_meta)  
*Reads specified logical block metadata based on the backward compatible FS.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_reserve\\_file](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const uint8\_t \*fid, bool use\_spare, size\_t size, uint32\_t flags, uint32\_t \*file\_meta\_idx, struct [its\\_file\\_meta\\_t](#) \*file\_meta, struct [its\\_block\\_meta\\_t](#) \*block\_meta)  
*Reserves space for a file.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_reset\\_metablock](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)  
*Resets metablock by cleaning and initializing the metablock.*
- [void its\\_flash\\_fs\\_mblock\\_set\\_data\\_scratch](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t phy\_id, uint32\_t lblock)  
*Sets current data scratch block.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_update\\_scratch\\_block\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t lblock, struct [its\\_block\\_meta\\_t](#) \*block\_meta)  
*Puts logical block's metadata in scratch metadata block.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_update\\_scratch\\_file\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t idx, const struct [its\\_file\\_meta\\_t](#) \*file\_meta)  
*Writes a file metadata entry into scratch metadata block.*
- [psa\\_status\\_t its\\_flash\\_fs\\_block\\_to\\_block\\_move](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t dst\_block, size\_t dst\_offset, uint32\_t src\_block, size\_t src\_offset, size\_t size)  
*Moves data from source block ID to destination block ID.*

## 8.321.1 Macro Definition Documentation

### 8.321.1.1 ITS\_BLOCK\_META\_HEADER\_SIZE

```
#define ITS_BLOCK_META_HEADER_SIZE sizeof(struct its_metadata_block_header_t)
```

Definition at line 35 of file [its\\_flash\\_fs\\_mblock.c](#).

### 8.321.1.2 ITS\_BLOCK\_METADATA\_SIZE

```
#define ITS_BLOCK_METADATA_SIZE sizeof(struct its_block_meta_t)
```

Definition at line 36 of file `its_flash_fs_mblock.c`.

### 8.321.1.3 ITS\_FILE\_METADATA\_SIZE

```
#define ITS_FILE_METADATA_SIZE sizeof(struct its_file_meta_t)
```

Definition at line 37 of file `its_flash_fs_mblock.c`.

### 8.321.1.4 ITS\_MAX\_BLOCK\_DATA\_COPY

```
#define ITS_MAX_BLOCK_DATA_COPY 256
```

Definition at line 16 of file `its_flash_fs_mblock.c`.

### 8.321.1.5 ITS\_METADATA\_BLOCK0

```
#define ITS_METADATA_BLOCK0 0
```

Definition at line 23 of file `its_flash_fs_mblock.c`.

### 8.321.1.6 ITS\_METADATA\_BLOCK1

```
#define ITS_METADATA_BLOCK1 1
```

Definition at line 24 of file `its_flash_fs_mblock.c`.

### 8.321.1.7 ITS\_OTHER\_META\_BLOCK

```
#define ITS_OTHER_META_BLOCK(
 metablock)
```

#### Value:

```
((metablock) == ITS_METADATA_BLOCK0) ? \
(ITS_METADATA_BLOCK1) : (ITS_METADATA_BLOCK0)
```

Macro to get the the swap metadata block.

Definition at line 31 of file `its_flash_fs_mblock.c`.

## 8.321.2 Function Documentation

### 8.321.2.1 its\_flash\_fs\_block\_to\_block\_move()

```
psa_status_t its_flash_fs_block_to_block_move (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t dst_block,
 size_t dst_offset,
 uint32_t src_block,
 size_t src_offset,
 size_t size)
```

Moves data from source block ID to destination block ID.

#### Parameters

|    |                         |                                                                    |
|----|-------------------------|--------------------------------------------------------------------|
| in | <code>fs_ctx</code>     | Filesystem context                                                 |
| in | <code>dst_block</code>  | Destination block ID                                               |
| in | <code>dst_offset</code> | Destination offset position from the init of the destination block |

## Parameters

|    |                   |                                                          |
|----|-------------------|----------------------------------------------------------|
| in | <i>src_block</i>  | Source block ID                                          |
| in | <i>src_offset</i> | Source offset position from the init of the source block |
| in | <i>size</i>       | Number of bytes to moves                                 |

## Note

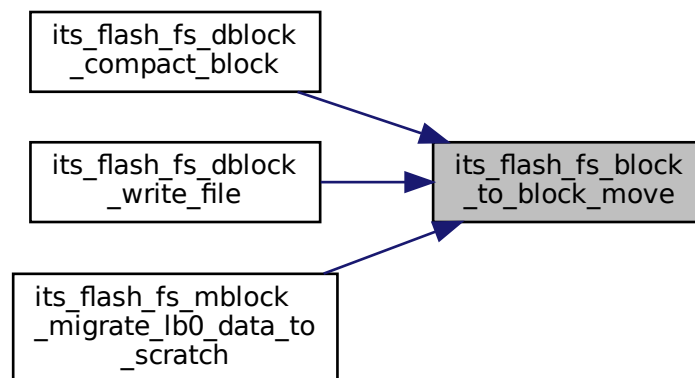
This function assumes all input values are valid. That is, the address range, based on blockid, offset and size, is a valid range in flash. It also assumes that the destination block is already erased and ready to be written.

## Returns

Returns PSA\_SUCCESS if the function is executed correctly. Otherwise, it returns PSA\_ERROR\_STORAGE\_FAILURE.

Definition at line 1383 of file its\_flash\_fs\_mblock.c.

Here is the caller graph for this function:



## 8.321.2.2 its\_flash\_fs\_mblock\_cp\_file\_meta()

```

psa_status_t its_flash_fs_mblock_cp_file_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t idx_start,
 uint32_t idx_end)

```

Copies the file metadata entries between two indexes from the active metadata block to the scratch metadata block.

## Parameters

|         |                  |                                                    |
|---------|------------------|----------------------------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context                                 |
| in      | <i>idx_start</i> | File metadata entry index to start copy, inclusive |
| in      | <i>idx_end</i>   | File metadata entry index to end copy, exclusive   |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 974 of file `its_flash_fs_mblock.c`.

**8.321.2.3 its\_flash\_fs\_mblock\_cur\_data\_scratch\_id()**

```
uint32_t its_flash_fs_mblock_cur_data_scratch_id (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock)
```

Gets current scratch datablock physical ID.

**Parameters**

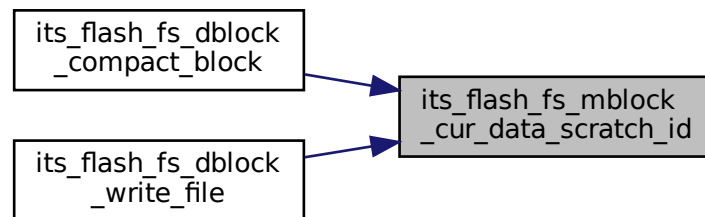
|         |               |                      |
|---------|---------------|----------------------|
| in, out | <i>fs_ctx</i> | Filesystem context   |
| in      | <i>lblock</i> | Logical block number |

**Returns**

current scratch data block

Definition at line 990 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:

**8.321.2.4 its\_flash\_fs\_mblock\_get\_file\_idx\_flag()**

```
psa_status_t its_flash_fs_mblock_get_file_idx_flag (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t flags,
 uint32_t * idx)
```

Gets file metadata entry index of the first file with one of the provided flags set.

**Parameters**

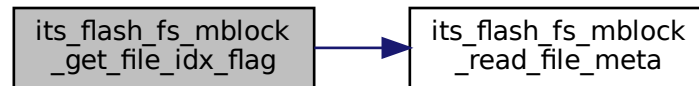
|         |               |                                               |
|---------|---------------|-----------------------------------------------|
| in, out | <i>fs_ctx</i> | Filesystem context                            |
| in      | <i>flags</i>  | Flags to search for                           |
| out     | <i>idx</i>    | Index of the file metadata in the file system |

### Returns

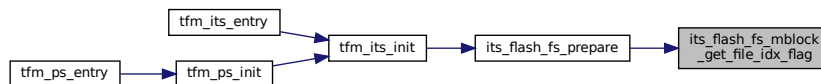
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1031 of file `its_flash_fs_mblock.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.321.2.5 its\_flash\_fs\_mblock\_get\_file\_idx\_meta()

```

psa_status_t its_flash_fs_mblock_get_file_idx_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 uint32_t * idx,
 struct its_file_meta_t * file_meta)

```

Gets file metadata entry index and file metadata.

### Note

A NULL [`file_meta`] indicates ignoring file meta.

### Parameters

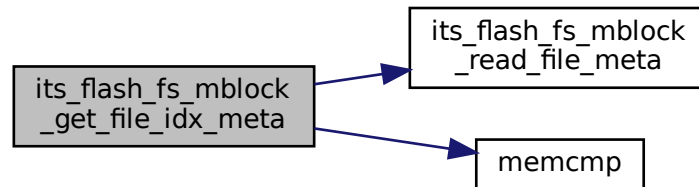
|         |                  |                                               |
|---------|------------------|-----------------------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context                            |
| in      | <i>fid</i>       | ID of the file                                |
| out     | <i>idx</i>       | Index of the file metadata in the file system |
| out     | <i>file_meta</i> | Pointer to file meta structure                |

**Returns**

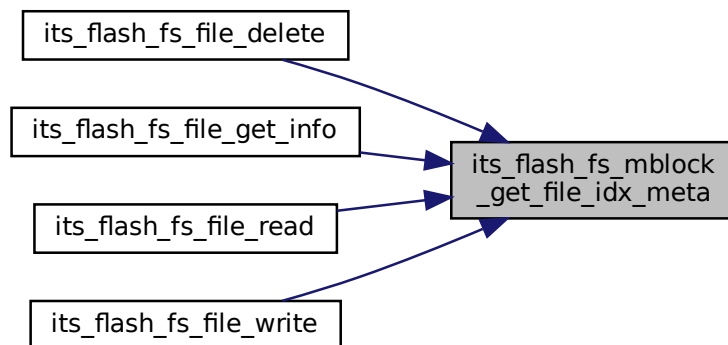
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1002 of file `its_flash_fs_mblock.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.321.2.6 its\_flash\_fs\_mblock\_init()**

```

psa_status_t its_flash_fs_mblock_init (
 struct its_flash_fs_ctx_t * fs_ctx)

```

Initializes metadata block with the valid/active metablock.

**Parameters**

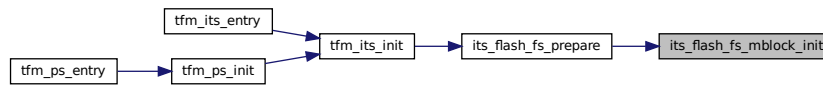
|                      |                     |                    |
|----------------------|---------------------|--------------------|
| <code>in, out</code> | <code>fs_ctx</code> | Filesystem context |
|----------------------|---------------------|--------------------|

**Returns**

Returns value as specified in [psa\\_status\\_t](#)

Definition at line 1056 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:



### 8.321.2.7 its\_flash\_fs\_mblock\_meta\_update\_finalize()

```
psa_status_t its_flash_fs_mblock_meta_update_finalize (
 struct its_flash_fs_ctx_t * fs_ctx)
```

Finalizes an update operation. Last step when a create/write/delete is performed.

#### Parameters

|                |               |                    |
|----------------|---------------|--------------------|
| <i>in, out</i> | <i>fs_ctx</i> | Filesystem context |
|----------------|---------------|--------------------|

#### Returns

Returns offset value in metadata block

Definition at line 1088 of file its\_flash\_fs\_mblock.c.

### 8.321.2.8 its\_flash\_fs\_mblock\_migrate\_lb0\_data\_to\_scratch()

```
psa_status_t its_flash_fs_mblock_migrate_lb0_data_to_scratch (
 struct its_flash_fs_ctx_t * fs_ctx)
```

Writes the files data area of logical block 0 into the scratch block.

#### Note

The files data in the logical block 0 is stored in same physical block where the metadata is stored. A change in the metadata requires a swap of physical blocks. So, the files data stored in the current metadata block needs to be copied in the scratch block, unless the data of the file processed is located in the logical block 0.

#### Parameters

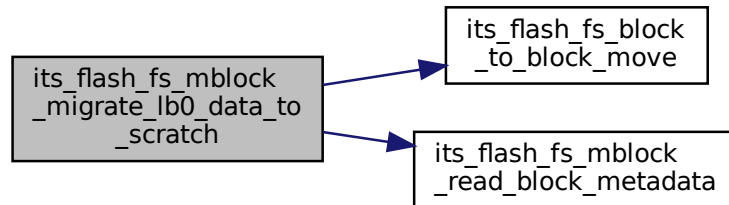
|                |               |                    |
|----------------|---------------|--------------------|
| <i>in, out</i> | <i>fs_ctx</i> | Filesystem context |
|----------------|---------------|--------------------|

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1112 of file `its_flash_fs_mblock.c`.

Here is the call graph for this function:

**8.321.2.9 its\_flash\_fs\_mblock\_read\_block\_metadata()**

```

psa_status_t its_flash_fs_mblock_read_block_metadata (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock,
 struct its_block_meta_t * block_meta)

```

Reads specified logical block metadata.

**Parameters**

|         |                   |                                 |
|---------|-------------------|---------------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context              |
| in      | <i>lblock</i>     | Logical block number            |
| out     | <i>block_meta</i> | Pointer to block meta structure |

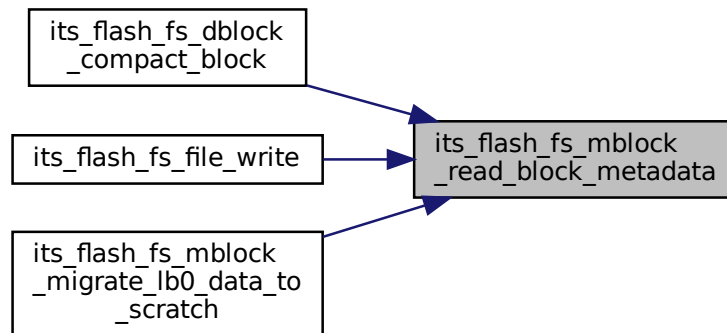
**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1157 of file `its_flash_fs_mblock.c`.



Here is the caller graph for this function:



#### 8.321.2.10 its\_flash\_fs\_mblock\_read\_block\_metadata\_comp()

```

psa_status_t its_flash_fs_mblock_read_block_metadata_comp (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock,
 struct its_block_meta_t * block_meta)

```

Reads specified logical block metadata based on the backward compatible FS.

##### Parameters

|         |                   |                                 |
|---------|-------------------|---------------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context              |
| in      | <i>lblock</i>     | Logical block number            |
| out     | <i>block_meta</i> | Pointer to block meta structure |

##### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1179 of file its\_flash\_fs\_mblock.c.

#### 8.321.2.11 its\_flash\_fs\_mblock\_read\_file\_meta()

```

psa_status_t its_flash_fs_mblock_read_file_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t idx,
 struct its_file_meta_t * file_meta)

```

Reads specified file metadata.

##### Parameters

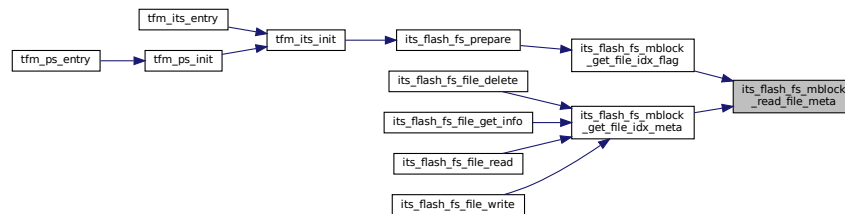
|         |                  |                                |
|---------|------------------|--------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context             |
| in      | <i>idx</i>       | File metadata entry index      |
| out     | <i>file_meta</i> | Pointer to file meta structure |

## Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1135 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:



### 8.321.2.12 its\_flash\_fs\_mblock\_reserve\_file()

```

psa_status_t its_flash_fs_mblock_reserve_file (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 bool use_spare,
 size_t size,
 uint32_t flags,
 uint32_t * file_meta_idx,
 struct its_file_meta_t * file_meta,
 struct its_block_meta_t * block_meta)

```

Reserves space for a file.

## Parameters

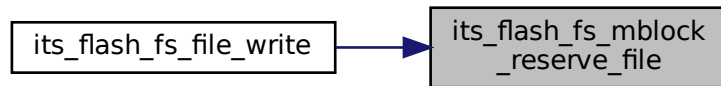
|         |                      |                                                                                         |
|---------|----------------------|-----------------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i>        | Filesystem context                                                                      |
| in      | <i>fid</i>           | File ID                                                                                 |
| in      | <i>use_spare</i>     | If true then the spare file will be used, otherwise at least one file will be left free |
| in      | <i>size</i>          | Size of the file for which space is reserve                                             |
| in      | <i>flags</i>         | Flags set when the file was created                                                     |
| out     | <i>file_meta_idx</i> | File metadata entry index                                                               |
| out     | <i>file_meta</i>     | File metadata entry                                                                     |
| out     | <i>block_meta</i>    | Block metadata entry                                                                    |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1203 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:

**8.321.2.13 its\_flash\_fs\_mblock\_reset\_metablock()**

```

psa_status_t its_flash_fs_mblock_reset_metablock (
 struct its_flash_fs_ctx_t * fs_ctx)

```

Resets metablock by cleaning and initializing the metadatablock.

**Parameters**

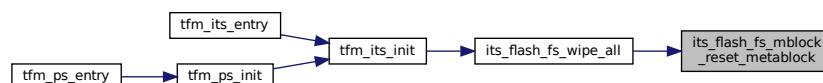
|                      |                     |                    |
|----------------------|---------------------|--------------------|
| <code>in, out</code> | <code>fs_ctx</code> | Filesystem context |
|----------------------|---------------------|--------------------|

**Returns**

Returns value as specified in [psa\\_status\\_t](#)

Definition at line 1227 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:

**8.321.2.14 its\_flash\_fs\_mblock\_set\_data\_scratch()**

```

void its_flash_fs_mblock_set_data_scratch (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t phy_id,
 uint32_t lblock)

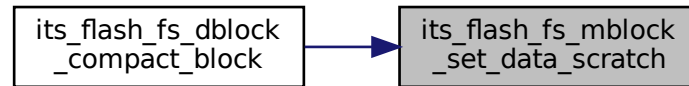
```

Sets current data scratch block.

**Parameters**

|                      |                     |                                   |
|----------------------|---------------------|-----------------------------------|
| <code>in, out</code> | <code>fs_ctx</code> | Filesystem context                |
| <code>in</code>      | <code>phy_id</code> | Physical ID of scratch data block |
| <code>in</code>      | <code>lblock</code> | Logical block number              |

Definition at line 1338 of file `its_flash_fs_mblock.c`.  
 Here is the caller graph for this function:



#### 8.321.2.15 `its_flash_fs_mblock_update_scratch_block_meta()`

```

psa_status_t its_flash_fs_mblock_update_scratch_block_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock,
 struct its_block_meta_t * block_meta)

```

Puts logical block's metadata in scratch metadata block.

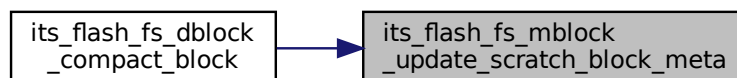
##### Parameters

|         |                   |                             |
|---------|-------------------|-----------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context          |
| in      | <i>lblock</i>     | Logical block number        |
| in      | <i>block_meta</i> | Pointer to block's metadata |

##### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1346 of file `its_flash_fs_mblock.c`.  
 Here is the caller graph for this function:



#### 8.321.2.16 `its_flash_fs_mblock_update_scratch_file_meta()`

```

psa_status_t its_flash_fs_mblock_update_scratch_file_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t idx,
 const struct its_file_meta_t * file_meta)

```

Writes a file metadata entry into scratch metadata block.

## Parameters

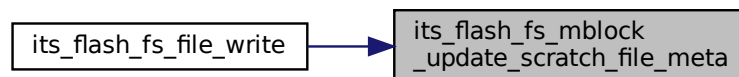
|         |                  |                                    |
|---------|------------------|------------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context                 |
| in      | <i>idx</i>       | File's index in the metadata table |
| in      | <i>file_meta</i> | Metadata pointer                   |

## Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1369 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:



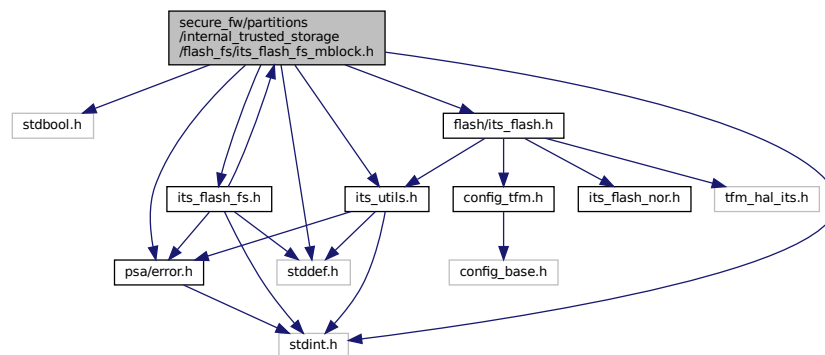
## 8.322 secure\_fw/partitions/internal\_trusted\_storage/flash\_fs/its\_flash\_fs\_mblock.h File Reference

```

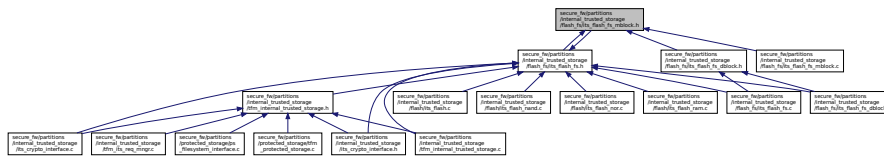
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "flash/its_flash.h"
#include "its_flash_fs.h"
#include "its_utils.h"
#include "psa/error.h"

```

Include dependency graph for `its_flash_fs_mblock.h`:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [its\\_metadata\\_block\\_header\\_t](#)  
*Structure to store the metadata block header.*
- struct [its\\_metadata\\_block\\_header\\_comp\\_t](#)  
*The structure of metadata block header in ITS\_BACKWARD\_SUPPORTED\_VERSION.*
- struct [its\\_block\\_meta\\_t](#)  
*Structure to store information about each physical flash memory block.*
- struct [its\\_file\\_meta\\_t](#)  
*Structure to store file metadata.*
- struct [its\\_flash\\_fs\\_ctx\\_t](#)  
*Structure to store the ITS flash file system context.*

## Macros

- `#define ITS_SUPPORTED_VERSION 0x02`  
*Defines the supported version.*
- `#define ITS_BACKWARD_SUPPORTED_VERSION 0x01`  
*Defines the backward supported version.*
- `#define ITS_METADATA_INVALID_INDEX 0xFFFF`  
*Defines the invalid index value when the metadata table is full.*
- `#define ITS_LOGICAL_DBLOCK0 0`  
*Defines logical data block 0 ID.*
- `#define _T1`
- `#define _T1_COMP`
- `#define _T2`
- `#define _T3`

## Functions

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_init](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)  
*Initializes metadata block with the valid/active metablock.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_cp\\_file\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t idx\_start, uint32\_t idx\_end)  
*Copies the file metadata entries between two indexes from the active metadata block to the scratch metadata block.*
- [uint32\\_t its\\_flash\\_fs\\_mblock\\_cur\\_data\\_scratch\\_id](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t lblock)  
*Gets current scratch datablock physical ID.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_get\\_file\\_idx\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const uint8\_t \*fid, uint32\_t \*idx, struct [its\\_file\\_meta\\_t](#) \*file\_meta)  
*Gets file metadata entry index and file metadata.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_get\\_file\\_idx\\_flag](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, uint32\_t flags, uint32\_t \*idx)  
*Gets file metadata entry index of the first file with one of the provided flags set.*
- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_meta\\_update\\_finalize](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)

*Finalizes an update operation. Last step when a create/write/delete is performed.*

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_migrate\\_lb0\\_data\\_to\\_scratch](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)

*Writes the files data area of logical block 0 into the scratch block.*

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_read\\_file\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, [uint32\\_t](#) idx, struct [its\\_file\\_meta\\_t](#) \*file\_meta)

*Reads specified file metadata.*

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_read\\_block\\_metadata](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, [uint32\\_t](#) lblock, struct [its\\_block\\_meta\\_t](#) \*block\_meta)

*Reads specified logical block metadata.*

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_read\\_block\\_metadata\\_comp](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, [uint32\\_t](#) lblock, struct [its\\_block\\_meta\\_t](#) \*block\_meta)

*Reads specified logical block metadata based on the backward compatible FS.*

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_reserve\\_file](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, const [uint8\\_t](#) \*fid, bool use\_spare, [size\\_t](#) size, [uint32\\_t](#) flags, [uint32\\_t](#) \*file\_meta\_idx, struct [its\\_file\\_meta\\_t](#) \*file\_meta, struct [its\\_block\\_meta\\_t](#) \*block\_meta)

*Reserves space for a file.*

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_reset\\_metablock](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx)

*Resets metablock by cleaning and initializing the metadata block.*

- [void its\\_flash\\_fs\\_mblock\\_set\\_data\\_scratch](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, [uint32\\_t](#) phy\_id, [uint32\\_t](#) lblock)

*Sets current data scratch block.*

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_update\\_scratch\\_block\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, [uint32\\_t](#) lblock, struct [its\\_block\\_meta\\_t](#) \*block\_meta)

*Puts logical block's metadata in scratch metadata block.*

- [psa\\_status\\_t its\\_flash\\_fs\\_mblock\\_update\\_scratch\\_file\\_meta](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, [uint32\\_t](#) idx, const struct [its\\_file\\_meta\\_t](#) \*file\_meta)

*Writes a file metadata entry into scratch metadata block.*

- [psa\\_status\\_t its\\_flash\\_fs\\_block\\_to\\_block\\_move](#) (struct [its\\_flash\\_fs\\_ctx\\_t](#) \*fs\_ctx, [uint32\\_t](#) dst\_block, [size\\_t](#) dst\_offset, [uint32\\_t](#) src\_block, [size\\_t](#) src\_offset, [size\\_t](#) size)

*Moves data from source block ID to destination block ID.*

## 8.322.1 Macro Definition Documentation

### 8.322.1.1 \_T1

```
#define _T1
```

**Value:**

```
uint32_t scratch_dblock; \
uint8_t fs_version; \
uint8_t metadata_xor; \
uint8_t active_swap_count;
```

Number of times the metadata blocks have \ been swapped \  
Definition at line 65 of file [its\\_flash\\_fs\\_mblock.h](#).

### 8.322.1.2 \_T1\_COMP

```
#define _T1_COMP
```

**Value:**

```
uint32_t scratch_dblock; \
uint8_t fs_version; \
uint8_t active_swap_count;
```

Number of times the metadata blocks have \ been swapped \  
Definition at line 97 of file [its\\_flash\\_fs\\_mblock.h](#).

### 8.322.1.3 \_T2

```
#define _T2
```

**Value:**

```
uint32_t phy_id; \
size_t data_start; \
size_t free_size;
```

Number of bytes free at end of block (set during \ block compaction for gap reuse) \

Definition at line 121 of file `its_flash_fs_mblock.h`.

### 8.322.1.4 \_T3

```
#define _T3
```

**Value:**

```
uint32_t lblock; /* Logical datablock where file is stored */ \
size_t data_idx; /* Offset in the logical data block */ \
size_t cur_size; /* Size in storage system for this fragment */ \
size_t max_size; /* Maximum size of this file */ \
uint32_t flags; /* Flags set when the file was created */ \
uint8_t id[ITS_FILE_ID_SIZE] /* ID of this file */
```

Definition at line 156 of file `its_flash_fs_mblock.h`.

### 8.322.1.5 ITS\_BACKWARD\_SUPPORTED\_VERSION

```
#define ITS_BACKWARD_SUPPORTED_VERSION 0x01
```

Defines the backward supported version.

Definition at line 36 of file `its_flash_fs_mblock.h`.

### 8.322.1.6 ITS\_LOGICAL\_DBLOCK0

```
#define ITS_LOGICAL_DBLOCK0 0
```

Defines logical data block 0 ID.

Definition at line 50 of file `its_flash_fs_mblock.h`.

### 8.322.1.7 ITS\_METADATA\_INVALID\_INDEX

```
#define ITS_METADATA_INVALID_INDEX 0xFFFF
```

Defines the invalid index value when the metadata table is full.

Definition at line 43 of file `its_flash_fs_mblock.h`.

### 8.322.1.8 ITS\_SUPPORTED\_VERSION

```
#define ITS_SUPPORTED_VERSION 0x02
```

Defines the supported version.

Definition at line 29 of file `its_flash_fs_mblock.h`.

## 8.322.2 Function Documentation

### 8.322.2.1 its\_flash\_fs\_block\_to\_block\_move()

```
psa_status_t its_flash_fs_block_to_block_move (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t dst_block,
 size_t dst_offset,
 uint32_t src_block,
```



```

size_t src_offset,
size_t size)

```

Moves data from source block ID to destination block ID.

#### Parameters

|    |                   |                                                                    |
|----|-------------------|--------------------------------------------------------------------|
| in | <i>fs_ctx</i>     | Filesystem context                                                 |
| in | <i>dst_block</i>  | Destination block ID                                               |
| in | <i>dst_offset</i> | Destination offset position from the init of the destination block |
| in | <i>src_block</i>  | Source block ID                                                    |
| in | <i>src_offset</i> | Source offset position from the init of the source block           |
| in | <i>size</i>       | Number of bytes to moves                                           |

#### Note

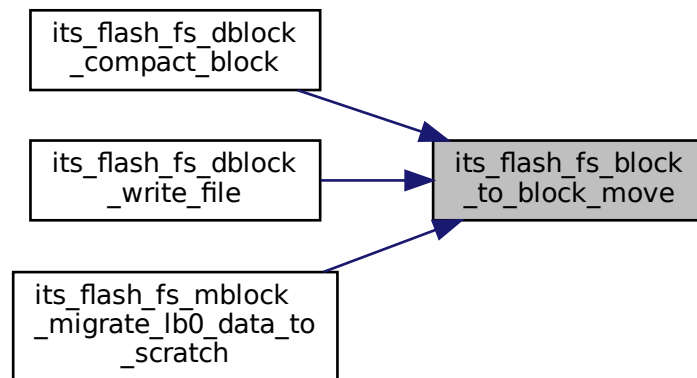
This function assumes all input values are valid. That is, the address range, based on blockid, offset and size, is a valid range in flash. It also assumes that the destination block is already erased and ready to be written.

#### Returns

Returns PSA\_SUCCESS if the function is executed correctly. Otherwise, it returns PSA\_ERROR\_STORAGE\_FAILURE.

Definition at line 1383 of file its\_flash\_fs\_mblock.c.

Here is the caller graph for this function:



#### 8.322.2.2 its\_flash\_fs\_mblock\_cp\_file\_meta()

```

psa_status_t its_flash_fs_mblock_cp_file_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t idx_start,
 uint32_t idx_end)

```

Copies the file metadata entries between two indexes from the active metadata block to the scratch metadata block.

**Parameters**

|         |                  |                                                    |
|---------|------------------|----------------------------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context                                 |
| in      | <i>idx_start</i> | File metadata entry index to start copy, inclusive |
| in      | <i>idx_end</i>   | File metadata entry index to end copy, exclusive   |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 974 of file `its_flash_fs_mblock.c`.

**8.322.2.3 its\_flash\_fs\_mblock\_cur\_data\_scratch\_id()**

```
uint32_t its_flash_fs_mblock_cur_data_scratch_id (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock)
```

Gets current scratch datablock physical ID.

**Parameters**

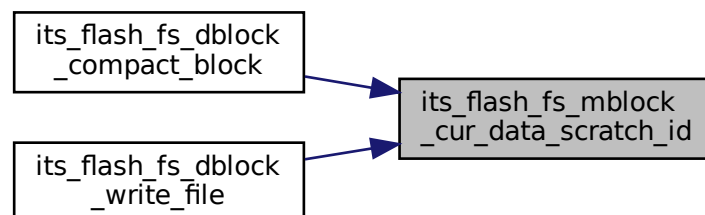
|         |               |                      |
|---------|---------------|----------------------|
| in, out | <i>fs_ctx</i> | Filesystem context   |
| in      | <i>lblock</i> | Logical block number |

**Returns**

current scratch data block

Definition at line 990 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:

**8.322.2.4 its\_flash\_fs\_mblock\_get\_file\_idx\_flag()**

```
psa_status_t its_flash_fs_mblock_get_file_idx_flag (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t flags,
 uint32_t * idx)
```

Gets file metadata entry index of the first file with one of the provided flags set.

## Parameters

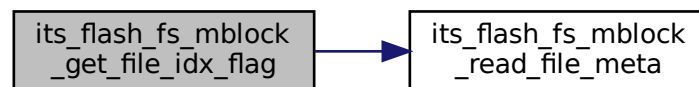
|         |               |                                               |
|---------|---------------|-----------------------------------------------|
| in, out | <i>fs_ctx</i> | Filesystem context                            |
| in      | <i>flags</i>  | Flags to search for                           |
| out     | <i>idx</i>    | Index of the file metadata in the file system |

## Returns

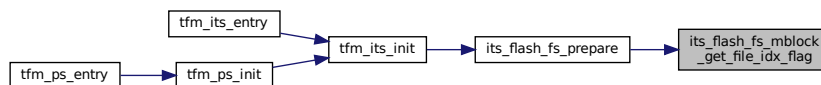
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1031 of file `its_flash_fs_mblock.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



## 8.322.2.5 its\_flash\_fs\_mblock\_get\_file\_idx\_meta()

```

psa_status_t its_flash_fs_mblock_get_file_idx_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 uint32_t * idx,
 struct its_file_meta_t * file_meta)

```

Gets file metadata entry index and file metadata.

## Note

A NULL [`file_meta`] indicates ignoring file meta.

## Parameters

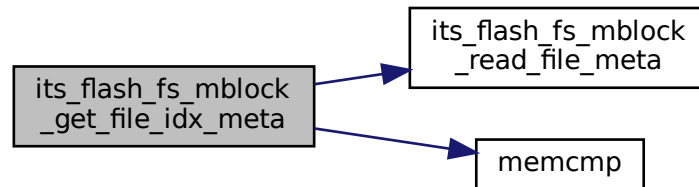
|         |                  |                                               |
|---------|------------------|-----------------------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context                            |
| in      | <i>fid</i>       | ID of the file                                |
| out     | <i>idx</i>       | Index of the file metadata in the file system |
| out     | <i>file_meta</i> | Pointer to file meta structure                |

**Returns**

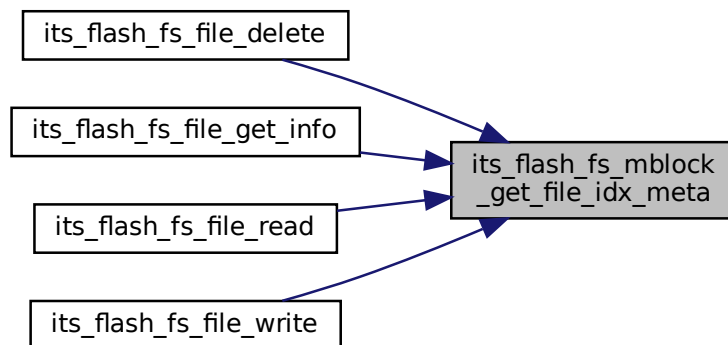
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1002 of file `its_flash_fs_mblock.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.322.2.6 its\_flash\_fs\_mblock\_init()**

```

psa_status_t its_flash_fs_mblock_init (
 struct its_flash_fs_ctx_t * fs_ctx)

```

Initializes metadata block with the valid/active metablock.

**Parameters**

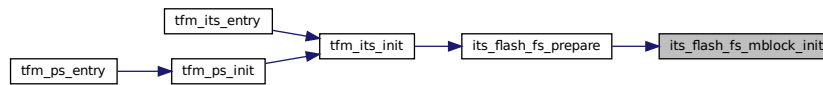
|                      |                     |                    |
|----------------------|---------------------|--------------------|
| <code>in, out</code> | <code>fs_ctx</code> | Filesystem context |
|----------------------|---------------------|--------------------|

**Returns**

Returns value as specified in [psa\\_status\\_t](#)

Definition at line 1056 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:



### 8.322.2.7 its\_flash\_fs\_mblock\_meta\_update\_finalize()

```
psa_status_t its_flash_fs_mblock_meta_update_finalize (
 struct its_flash_fs_ctx_t * fs_ctx)
```

Finalizes an update operation. Last step when a create/write/delete is performed.

#### Parameters

|                |               |                    |
|----------------|---------------|--------------------|
| <i>in, out</i> | <i>fs_ctx</i> | Filesystem context |
|----------------|---------------|--------------------|

#### Returns

Returns offset value in metadata block

Definition at line 1088 of file its\_flash\_fs\_mblock.c.

### 8.322.2.8 its\_flash\_fs\_mblock\_migrate\_lb0\_data\_to\_scratch()

```
psa_status_t its_flash_fs_mblock_migrate_lb0_data_to_scratch (
 struct its_flash_fs_ctx_t * fs_ctx)
```

Writes the files data area of logical block 0 into the scratch block.

#### Note

The files data in the logical block 0 is stored in same physical block where the metadata is stored. A change in the metadata requires a swap of physical blocks. So, the files data stored in the current metadata block needs to be copied in the scratch block, unless the data of the file processed is located in the logical block 0.

#### Parameters

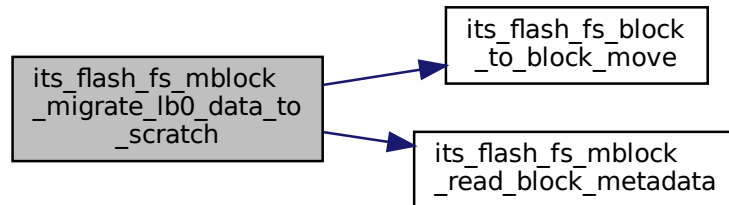
|                |               |                    |
|----------------|---------------|--------------------|
| <i>in, out</i> | <i>fs_ctx</i> | Filesystem context |
|----------------|---------------|--------------------|

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1112 of file `its_flash_fs_mblock.c`.

Here is the call graph for this function:

**8.322.2.9 its\_flash\_fs\_mblock\_read\_block\_metadata()**

```

psa_status_t its_flash_fs_mblock_read_block_metadata (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock,
 struct its_block_meta_t * block_meta)

```

Reads specified logical block metadata.

**Parameters**

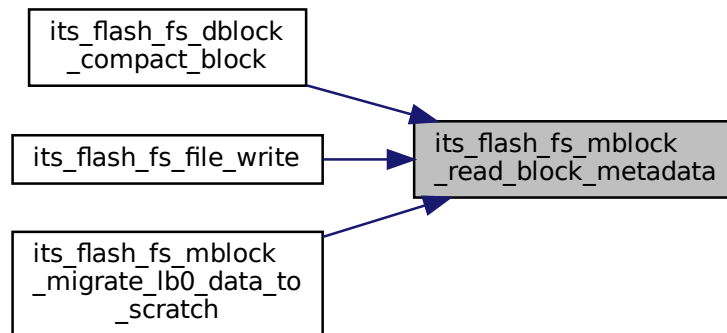
|         |                   |                                 |
|---------|-------------------|---------------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context              |
| in      | <i>lblock</i>     | Logical block number            |
| out     | <i>block_meta</i> | Pointer to block meta structure |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1157 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:



#### 8.322.2.10 its\_flash\_fs\_mblock\_read\_block\_metadata\_comp()

```

psa_status_t its_flash_fs_mblock_read_block_metadata_comp (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock,
 struct its_block_meta_t * block_meta)

```

Reads specified logical block metadata based on the backward compatible FS.

##### Parameters

|         |                   |                                 |
|---------|-------------------|---------------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context              |
| in      | <i>lblock</i>     | Logical block number            |
| out     | <i>block_meta</i> | Pointer to block meta structure |

##### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1179 of file `its_flash_fs_mblock.c`.

#### 8.322.2.11 its\_flash\_fs\_mblock\_read\_file\_meta()

```

psa_status_t its_flash_fs_mblock_read_file_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t idx,
 struct its_file_meta_t * file_meta)

```

Reads specified file metadata.

##### Parameters

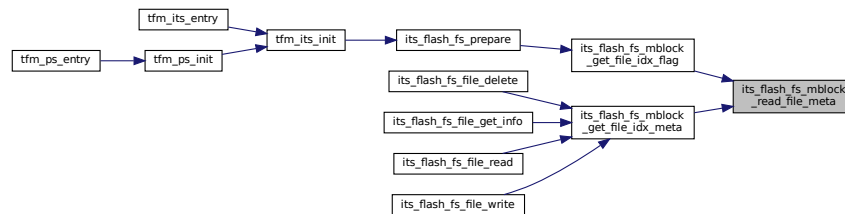
|         |                  |                                |
|---------|------------------|--------------------------------|
| in, out | <i>fs_ctx</i>    | Filesystem context             |
| in      | <i>idx</i>       | File metadata entry index      |
| out     | <i>file_meta</i> | Pointer to file meta structure |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1135 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:

**8.322.2.12 its\_flash\_fs\_mblock\_reserve\_file()**

```

psa_status_t its_flash_fs_mblock_reserve_file (
 struct its_flash_fs_ctx_t * fs_ctx,
 const uint8_t * fid,
 bool use_spare,
 size_t size,
 uint32_t flags,
 uint32_t * file_meta_idx,
 struct its_file_meta_t * file_meta,
 struct its_block_meta_t * block_meta)

```

Reserves space for a file.

**Parameters**

|         |                      |                                                                                         |
|---------|----------------------|-----------------------------------------------------------------------------------------|
| in, out | <i>fs_ctx</i>        | Filesystem context                                                                      |
| in      | <i>fid</i>           | File ID                                                                                 |
| in      | <i>use_spare</i>     | If true then the spare file will be used, otherwise at least one file will be left free |
| in      | <i>size</i>          | Size of the file for which space is reserve                                             |
| in      | <i>flags</i>         | Flags set when the file was created                                                     |
| out     | <i>file_meta_idx</i> | File metadata entry index                                                               |
| out     | <i>file_meta</i>     | File metadata entry                                                                     |
| out     | <i>block_meta</i>    | Block metadata entry                                                                    |

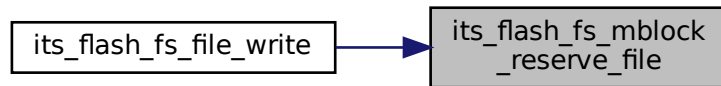


**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1203 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:

**8.322.2.13 its\_flash\_fs\_mblock\_reset\_metablock()**

```

psa_status_t its_flash_fs_mblock_reset_metablock (
 struct its_flash_fs_ctx_t * fs_ctx)

```

Resets metablock by cleaning and initializing the metadatablock.

**Parameters**

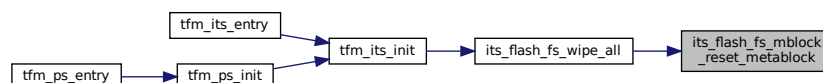
|                      |                     |                    |
|----------------------|---------------------|--------------------|
| <code>in, out</code> | <code>fs_ctx</code> | Filesystem context |
|----------------------|---------------------|--------------------|

**Returns**

Returns value as specified in [psa\\_status\\_t](#)

Definition at line 1227 of file `its_flash_fs_mblock.c`.

Here is the caller graph for this function:

**8.322.2.14 its\_flash\_fs\_mblock\_set\_data\_scratch()**

```

void its_flash_fs_mblock_set_data_scratch (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t phy_id,
 uint32_t lblock)

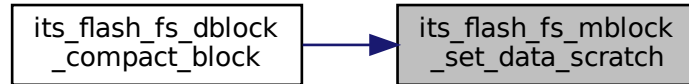
```

Sets current data scratch block.

**Parameters**

|                      |                     |                                   |
|----------------------|---------------------|-----------------------------------|
| <code>in, out</code> | <code>fs_ctx</code> | Filesystem context                |
| <code>in</code>      | <code>phy_id</code> | Physical ID of scratch data block |
| <code>in</code>      | <code>lblock</code> | Logical block number              |

Definition at line 1338 of file `its_flash_fs_mblock.c`.  
Here is the caller graph for this function:



#### 8.322.2.15 `its_flash_fs_mblock_update_scratch_block_meta()`

```

psa_status_t its_flash_fs_mblock_update_scratch_block_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t lblock,
 struct its_block_meta_t * block_meta)

```

Puts logical block's metadata in scratch metadata block.

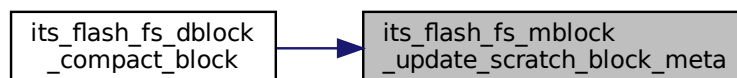
##### Parameters

|         |                   |                             |
|---------|-------------------|-----------------------------|
| in, out | <i>fs_ctx</i>     | Filesystem context          |
| in      | <i>lblock</i>     | Logical block number        |
| in      | <i>block_meta</i> | Pointer to block's metadata |

##### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1346 of file `its_flash_fs_mblock.c`.  
Here is the caller graph for this function:



#### 8.322.2.16 `its_flash_fs_mblock_update_scratch_file_meta()`

```

psa_status_t its_flash_fs_mblock_update_scratch_file_meta (
 struct its_flash_fs_ctx_t * fs_ctx,
 uint32_t idx,
 const struct its_file_meta_t * file_meta)

```

Writes a file metadata entry into scratch metadata block.



### 8.323.1 Function Documentation

#### 8.323.1.1 tfm\_its\_crypt\_file()

```
psa_status_t tfm_its_crypt_file (
 struct its_flash_fs_file_info_t * finfo,
 uint8_t * fid,
 const size_t fid_size,
 const uint8_t * input,
 const size_t input_size,
 uint8_t * output,
 const size_t output_size,
 const bool is_encrypt)
```

Perform encryption/decryption of the buffer using the tfm\_hal\_its APIs.

##### Parameters

|     |                    |                                                     |
|-----|--------------------|-----------------------------------------------------|
| in  | <i>finfo</i>       | Pointer to <a href="#">its_flash_fs_file_info_t</a> |
| in  | <i>fid</i>         | File identifier                                     |
| in  | <i>fid_size</i>    | File identifier size in bytes                       |
| in  | <i>input</i>       | Input buffer                                        |
| in  | <i>input_size</i>  | Input size in bytes                                 |
| out | <i>output</i>      | Output buffer                                       |
| in  | <i>output_size</i> | Output size in bytes                                |
| in  | <i>is_encrypt</i>  | Set the operation type (encryption/decryption)      |

##### Returns

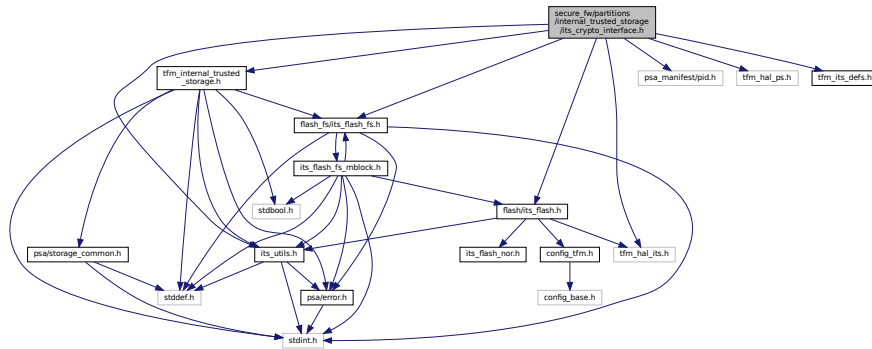
PSA\_SUCCESS on successful operation or a valid PSA error code

Definition at line 84 of file `its_crypto_interface.c`.

### 8.324 secure\_fw/partitions/internal\_trusted\_storage/its\_crypto\_↵ interface.h File Reference

```
#include "flash_fs/its_flash_fs.h"
#include "flash/its_flash.h"
#include "its_utils.h"
#include "psa_manifest/pid.h"
#include "tfm_hal_its.h"
#include "tfm_hal_ps.h"
#include "tfm_internal_trusted_storage.h"
#include "tfm_its_defs.h"
```

Include dependency graph for its\_crypto\_interface.h:



## Functions

- [psa\\_status\\_t tfm\\_its\\_crypt\\_file](#) (struct [its\\_flash\\_fs\\_file\\_info\\_t](#) \**finfo*, uint8\_t \**fid*, const size\_t *fid\_size*, const uint8\_t \**input*, const size\_t *input\_size*, uint8\_t \**output*, const size\_t *output\_size*, const bool *is\_encrypt*)  
Perform encryption/decryption of the buffer using the *tfm\_hal\_its* APIs.

## 8.324.1 Function Documentation

### 8.324.1.1 tfm\_its\_crypt\_file()

```
psa_status_t tfm_its_crypt_file (
 struct its_flash_fs_file_info_t * finfo,
 uint8_t * fid,
 const size_t fid_size,
 const uint8_t * input,
 const size_t input_size,
 uint8_t * output,
 const size_t output_size,
 const bool is_encrypt)
```

Perform encryption/decryption of the buffer using the *tfm\_hal\_its* APIs.

#### Parameters

|     |                    |                                                     |
|-----|--------------------|-----------------------------------------------------|
| in  | <i>finfo</i>       | Pointer to <a href="#">its_flash_fs_file_info_t</a> |
| in  | <i>fid</i>         | File identifier                                     |
| in  | <i>fid_size</i>    | File identifier size in bytes                       |
| in  | <i>input</i>       | Input buffer                                        |
| in  | <i>input_size</i>  | Input size in bytes                                 |
| out | <i>output</i>      | Output buffer                                       |
| in  | <i>output_size</i> | Output size in bytes                                |
| in  | <i>is_encrypt</i>  | Set the operation type (encryption/decryption)      |

#### Returns

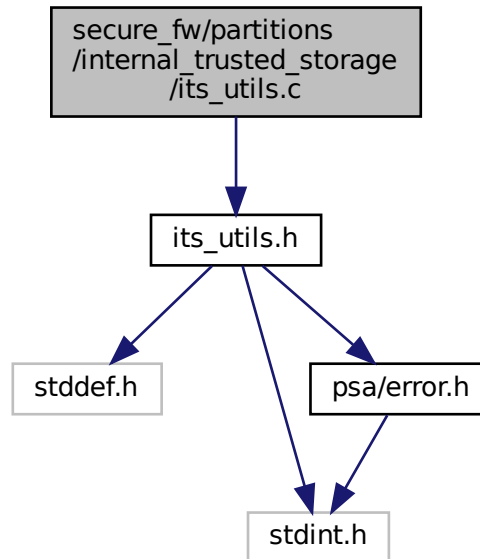
PSA\_SUCCESS on successful operation or a valid PSA error code

Definition at line 84 of file *its\_crypto\_interface.c*.

## 8.325 secure\_fw/partitions/internal\_trusted\_storage/its\_utils.c File Reference

```
#include "its_utils.h"
```

Include dependency graph for its\_utils.c:



### Functions

- [psa\\_status\\_t its\\_utils\\_check\\_contained\\_in](#) (size\_t superset\_size, size\_t subset\_offset, size\_t subset\_size)  
*Checks if a subset region is fully contained within a superset region.*
- [psa\\_status\\_t its\\_utils\\_validate\\_fid](#) (const uint8\_t \*fid)  
*Validates file ID.*

### 8.325.1 Function Documentation

#### 8.325.1.1 its\_utils\_check\_contained\_in()

```
psa_status_t its_utils_check_contained_in (
 size_t superset_size,
 size_t subset_offset,
 size_t subset_size)
```

Checks if a subset region is fully contained within a superset region.

#### Parameters

|    |                      |                                                                |
|----|----------------------|----------------------------------------------------------------|
| in | <i>superset_size</i> | Size of superset region                                        |
| in | <i>subset_offset</i> | Offset of start of subset region from start of superset region |
| in | <i>subset_size</i>   | Size of subset region                                          |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

**Return values**

|                                   |                                             |
|-----------------------------------|---------------------------------------------|
| <i>PSA_SUCCESS</i>                | The subset is contained within the superset |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | Otherwise                                   |

Definition at line 10 of file its\_utils.c.

Here is the caller graph for this function:

**8.325.1.2 its\_utils\_validate\_fid()**

```
psa_status_t its_utils_validate_fid (
 const uint8_t * fid)
```

Validates file ID.

**Parameters**

|    |            |         |
|----|------------|---------|
| in | <i>fid</i> | File ID |
|----|------------|---------|

**Returns**

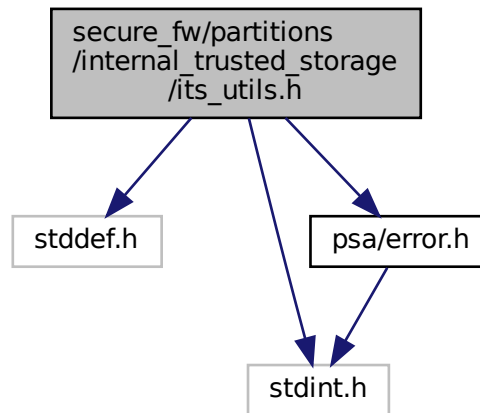
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 30 of file its\_utils.c.

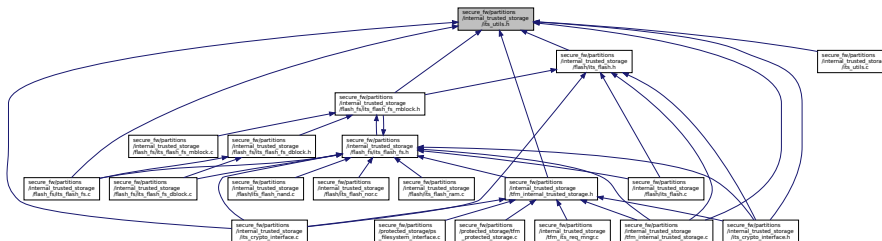
## 8.326 secure\_fw/partitions/internal\_trusted\_storage/its\_utils.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "psa/error.h"
```

Include dependency graph for its\_utils.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define ITS_FILE_ID_SIZE 12`
- `#define ITS_DATA_SIZE_FIELD_SIZE 4`
- `#define ITS_FLAG_SIZE 4`
- `#define ITS_DEFAULT_EMPTY_BUFF_VAL 0`
- `#define ITS_UTILS_BOUND_CHECK(err_msg, data_size, data_buf_size) typedef char err_msg[(data_size <= data_buf_size)*2 - 1]`  
*Macro to check, at compilation time, if data fits in data buffer.*
- `#define ITS_UTILS_MIN(x, y) (((x) < (y)) ? (x) : (y))`  
*Evaluates to the minimum of the two parameters.*
- `#define ITS_UTILS_MAX(x, y) (((x) > (y)) ? (x) : (y))`  
*Evaluates to the maximum of the two parameters.*
- `#define ITS_UTILS_ALIGN(x, a) (((x) + ((a) - 1)) & ~((a) - 1))`  
*Aligns a value up to the provided alignment.*
- `#define ITS_UTILS_IS_ALIGNED(x, a) (((x) & ((a) - 1)) == 0)`  
*Checks that a value is aligned to the provided alignment.*



## Functions

- [psa\\_status\\_t its\\_utils\\_check\\_contained\\_in](#) (size\_t superset\_size, size\_t subset\_offset, size\_t subset\_size)  
*Checks if a subset region is fully contained within a superset region.*
- [psa\\_status\\_t its\\_utils\\_validate\\_fid](#) (const uint8\_t \*fid)  
*Validates file ID.*

## 8.326.1 Macro Definition Documentation

### 8.326.1.1 ITS\_DATA\_SIZE\_FIELD\_SIZE

```
#define ITS_DATA_SIZE_FIELD_SIZE 4
```

Definition at line 21 of file its\_utils.h.

### 8.326.1.2 ITS\_DEFAULT\_EMPTY\_BUFF\_VAL

```
#define ITS_DEFAULT_EMPTY_BUFF_VAL 0
```

Definition at line 23 of file its\_utils.h.

### 8.326.1.3 ITS\_FILE\_ID\_SIZE

```
#define ITS_FILE_ID_SIZE 12
```

Definition at line 20 of file its\_utils.h.

### 8.326.1.4 ITS\_FLAG\_SIZE

```
#define ITS_FLAG_SIZE 4
```

Definition at line 22 of file its\_utils.h.

### 8.326.1.5 ITS\_UTILS\_ALIGN

```
#define ITS_UTILS_ALIGN(
 x,
 a) (((x) + ((a) - 1)) & ~((a) - 1))
```

Aligns a value up to the provided alignment.

#### Parameters

|    |          |                                    |
|----|----------|------------------------------------|
| in | <i>x</i> | Value to be aligned                |
| in | <i>a</i> | Alignment (must be a power of two) |

#### Returns

The least value not less than *x* that is aligned to *a*.

Definition at line 58 of file its\_utils.h.

### 8.326.1.6 ITS\_UTILS\_BOUND\_CHECK

```
#define ITS_UTILS_BOUND_CHECK(
 err_msg,
```

```
data_size,
data_buf_size) typedef char err_msg[(data_size <= data_buf_size)*2 - 1]
```

Macro to check, at compilation time, if data fits in data buffer.

#### Parameters

|    |                      |                                                                                   |
|----|----------------------|-----------------------------------------------------------------------------------|
| in | <i>err_msg</i>       | Error message which will be displayed in first instance if the error is triggered |
| in | <i>data_size</i>     | Data size to check if it fits                                                     |
| in | <i>data_buf_size</i> | Size of the data buffer                                                           |

#### Returns

Triggers a compilation error if *data\_size* is bigger than *data\_buf\_size*. The compilation error should be "... error: 'err\_msg' declared as an array with a negative size"

Definition at line 37 of file *its\_utils.h*.

### 8.326.1.7 ITS\_UTILS\_IS\_ALIGNED

```
#define ITS_UTILS_IS_ALIGNED(
 x,
 a) ((x) & ((a) - 1)) == 0)
```

Checks that a value is aligned to the provided alignment.

#### Parameters

|    |          |                                    |
|----|----------|------------------------------------|
| in | <i>x</i> | Value to check for alignment       |
| in | <i>a</i> | Alignment (must be a power of two) |

#### Returns

1 if *x* is aligned to *a*, 0 otherwise.

Definition at line 68 of file *its\_utils.h*.

### 8.326.1.8 ITS\_UTILS\_MAX

```
#define ITS_UTILS_MAX(
 x,
 y) ((x) > (y)) ? (x) : (y))
```

Evaluates to the maximum of the two parameters.

Definition at line 48 of file *its\_utils.h*.

### 8.326.1.9 ITS\_UTILS\_MIN

```
#define ITS_UTILS_MIN(
 x,
 y) ((x) < (y)) ? (x) : (y))
```

Evaluates to the minimum of the two parameters.

Definition at line 43 of file *its\_utils.h*.

## 8.326.2 Function Documentation

### 8.326.2.1 its\_utils\_check\_contained\_in()

```
psa_status_t its_utils_check_contained_in (
 size_t superset_size,
 size_t subset_offset,
 size_t subset_size)
```

Checks if a subset region is fully contained within a superset region.

#### Parameters

|    |                      |                                                                |
|----|----------------------|----------------------------------------------------------------|
| in | <i>superset_size</i> | Size of superset region                                        |
| in | <i>subset_offset</i> | Offset of start of subset region from start of superset region |
| in | <i>subset_size</i>   | Size of subset region                                          |

#### Returns

Returns error code as specified in [psa\\_status\\_t](#)

#### Return values

|                                   |                                             |
|-----------------------------------|---------------------------------------------|
| <i>PSA_SUCCESS</i>                | The subset is contained within the superset |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | Otherwise                                   |

Definition at line 10 of file its\_utils.c.

Here is the caller graph for this function:



### 8.326.2.2 its\_utils\_validate\_fid()

```
psa_status_t its_utils_validate_fid(
 const uint8_t * fid)
```

Validates file ID.

#### Parameters

|    |            |         |
|----|------------|---------|
| in | <i>fid</i> | File ID |
|----|------------|---------|



## 8.327.1 Function Documentation

### 8.327.1.1 tfm\_its\_get()

```
psa_status_t tfm_its_get (
 int32_t client_id,
 psa_storage_uid_t uid,
 size_t data_offset,
 size_t data_size,
 size_t * p_data_length)
```

Retrieve data associated with a provided UID.

Retrieves up to `data_size` bytes of the data associated with `uid`, starting at `data_offset` bytes from the beginning of the data. Upon successful completion, the data will be placed in the `p_data` buffer, which must be at least `data_size` bytes in size. The length of the data returned will be in `p_data_length`. If `data_size` is 0, the contents of `p_data_length` will be set to zero.

#### Parameters

|     |                      |                                                                                |
|-----|----------------------|--------------------------------------------------------------------------------|
| in  | <i>client_id</i>     | Identifier of the asset's owner (client)                                       |
| in  | <i>uid</i>           | The uid value                                                                  |
| in  | <i>data_offset</i>   | The starting offset of the data requested                                      |
| in  | <i>data_size</i>     | The amount of data requested                                                   |
| out | <i>p_data_length</i> | On success, this will contain size of the data placed in <code>p_data</code> . |

#### Returns

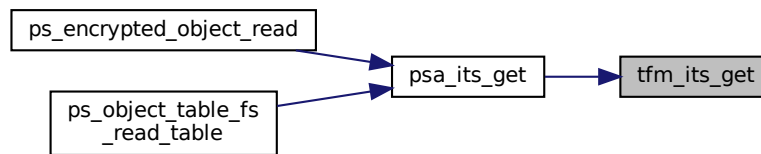
A status indicating the success/failure of the operation

#### Return values

|                                   |                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                                                                                                                                                                                             |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided <code>uid</code> value was not found in the storage                                                                                                                                                                                                                                                    |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                                                                                                                                                                                                                                                       |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided arguments ( <code>p_data</code> , <code>p_data_length</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access. In addition, this can also happen if <code>data_offset</code> is larger than the size of the data associated with <code>uid</code> . |

Definition at line 578 of file `tfm_internal_trusted_storage.c`.

Here is the caller graph for this function:



### 8.327.1.2 `tfm_its_get_info()`

```

psa_status_t tfm_its_get_info (
 int32_t client_id,
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Retrieve the metadata about the provided uid.

Retrieves the metadata stored for a given uid as a `psa_storage_info_t` structure.

#### Parameters

|     |                        |                                                                                                  |
|-----|------------------------|--------------------------------------------------------------------------------------------------|
| in  | <code>client_id</code> | Identifier of the asset's owner (client)                                                         |
| in  | <code>uid</code>       | The uid value                                                                                    |
| out | <code>p_info</code>    | A pointer to the <code>psa_storage_info_t</code> struct that will be populated with the metadata |

#### Returns

A status indicating the success/failure of the operation

#### Return values

|                                         |                                                                                                                                                                             |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>PSA_SUCCESS</code>                | The operation completed successfully                                                                                                                                        |
| <code>PSA_ERROR_DOES_NOT_EXIST</code>   | The operation failed because the provided uid value was not found in the storage                                                                                            |
| <code>PSA_ERROR_STORAGE_FAILURE</code>  | The operation failed because the physical storage has failed (Fatal error)                                                                                                  |
| <code>PSA_ERROR_INVALID_ARGUMENT</code> | The operation failed because one of the provided pointers( <code>p_info</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

Definition at line 642 of file `tfm_internal_trusted_storage.c`.

Here is the caller graph for this function:



### 8.327.1.3 tfm\_its\_init()

```

psa_status_t tfm_its_init (
 void)

```

Initializes the internal trusted storage system.

#### Returns

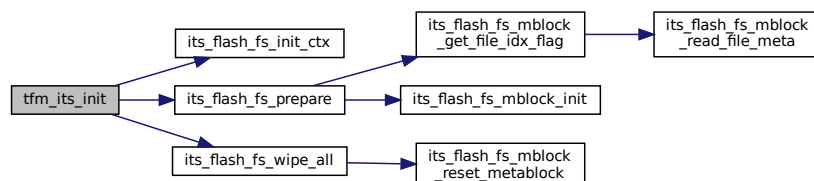
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

#### Return values

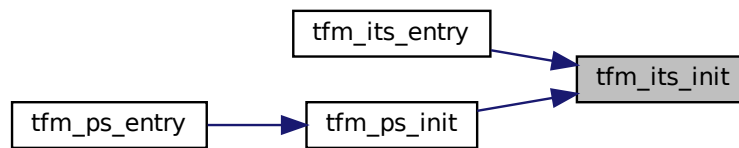
|                                  |                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>               | The operation completed successfully                                                    |
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the storage system initialization has failed (fatal error) |
| <i>PSA_ERROR_GENERIC_ERROR</i>   | The operation failed because of an unspecified internal failure                         |

Definition at line 285 of file `tfm_internal_trusted_storage.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.327.1.4 tfm\_its\_remove()

```

psa_status_t tfm_its_remove (
 int32_t client_id,
 psa_storage_uid_t uid)

```

Remove the provided uid and its associated data from the storage.  
Deletes the data from internal storage.

##### Parameters

|    |                  |                                          |
|----|------------------|------------------------------------------|
| in | <i>client_id</i> | Identifier of the asset's owner (client) |
| in | <i>uid</i>       | The uid value                            |

##### Returns

A status indicating the success/failure of the operation

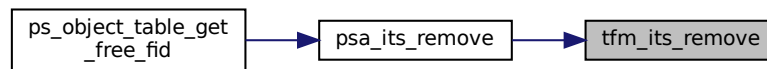
##### Return values

|                                   |                                                                                                                    |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                               |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                   |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code>      |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                         |

Definition at line 661 of file `tfm_internal_trusted_storage.c`.



Here is the caller graph for this function:



### 8.327.1.5 tfm\_its\_set()

```

psa_status_t tfm_its_set (
 int32_t client_id,
 psa_storage_uid_t uid,
 size_t data_length,
 psa_storage_create_flags_t create_flags)

```

Create a new, or modify an existing, uid/value pair.  
Stores data in the internal storage.

#### Parameters

|    |                     |                                                |
|----|---------------------|------------------------------------------------|
| in | <i>client_id</i>    | Identifier of the asset's owner (client)       |
| in | <i>uid</i>          | The identifier for the data                    |
| in | <i>data_length</i>  | The size in bytes of the data in <i>p_data</i> |
| in | <i>create_flags</i> | The flags that the data will be stored with    |

#### Returns

A status indicating the success/failure of the operation

#### Return values

|                                       |                                                                                                                                                                        |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                                                                                   |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because one or more of the flags provided in <i>create_flags</i> is not supported or is not valid                                                 |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because there was insufficient space on the storage medium                                                                                        |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (Fatal error)                                                                                             |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because one of the provided pointers ( <i>p_data</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

Definition at line 424 of file `tfm_internal_trusted_storage.c`.

Here is the caller graph for this function:



## 8.327.2 Variable Documentation

### 8.327.2.1 p\_psa\_dest\_data

```
uint8_t* p_psa_dest_data
```

Definition at line 15 of file ps\_filesystem\_interface.c.

### 8.327.2.2 p\_psa\_src\_data

```
uint8_t* p_psa_src_data
```

Definition at line 14 of file ps\_filesystem\_interface.c.

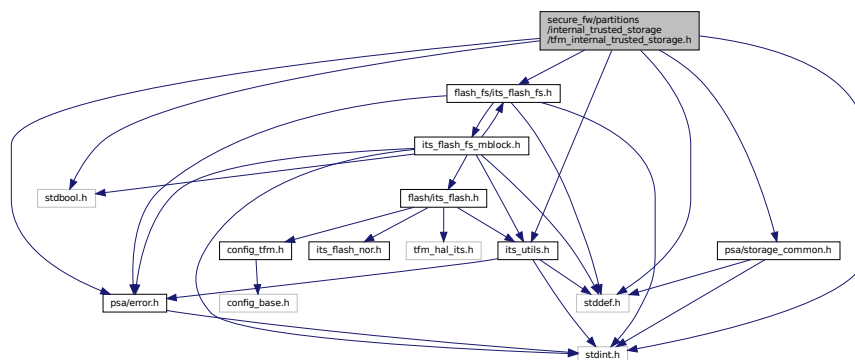
## 8.328 secure\_fw/partitions/internal\_trusted\_storage/tfm\_internal\_trusted\_storage.h File Reference

```

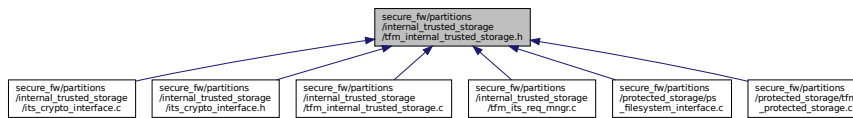
#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>
#include "psa/error.h"
#include "psa/storage_common.h"
#include "flash_fs/its_flash_fs.h"
#include "its_utils.h"

```

Include dependency graph for tfm\_internal\_trusted\_storage.h:



This graph shows which files directly or indirectly include this file:



## Functions

- [psa\\_status\\_t tfm\\_its\\_init \(void\)](#)  
*Initializes the internal trusted storage system.*
- [psa\\_status\\_t tfm\\_its\\_set \(int32\\_t client\\_id, psa\\_storage\\_uid\\_t uid, size\\_t data\\_length, psa\\_storage\\_create\\_flags\\_t create\\_flags\)](#)  
*Create a new, or modify an existing, uid/value pair.*
- [psa\\_status\\_t tfm\\_its\\_get \(int32\\_t client\\_id, psa\\_storage\\_uid\\_t uid, size\\_t data\\_offset, size\\_t data\\_size, size\\_t \\*p\\_data\\_length\)](#)  
*Retrieve data associated with a provided UID.*
- [psa\\_status\\_t tfm\\_its\\_get\\_info \(int32\\_t client\\_id, psa\\_storage\\_uid\\_t uid, struct psa\\_storage\\_info\\_t \\*p\\_info\)](#)  
*Retrieve the metadata about the provided uid.*
- [psa\\_status\\_t tfm\\_its\\_remove \(int32\\_t client\\_id, psa\\_storage\\_uid\\_t uid\)](#)  
*Remove the provided uid and its associated data from the storage.*

## 8.328.1 Function Documentation

### 8.328.1.1 tfm\_its\_get()

```

psa_status_t tfm_its_get (
 int32_t client_id,
 psa_storage_uid_t uid,
 size_t data_offset,
 size_t data_size,
 size_t * p_data_length)

```

Retrieve data associated with a provided UID.

Retrieves up to `data_size` bytes of the data associated with `uid`, starting at `data_offset` bytes from the beginning of the data. Upon successful completion, the data will be placed in the `p_data` buffer, which must be at least `data_size` bytes in size. The length of the data returned will be in `p_data_length`. If `data_size` is 0, the contents of `p_data_length` will be set to zero.

#### Parameters

|     |                      |                                                                                |
|-----|----------------------|--------------------------------------------------------------------------------|
| in  | <i>client_id</i>     | Identifier of the asset's owner (client)                                       |
| in  | <i>uid</i>           | The uid value                                                                  |
| in  | <i>data_offset</i>   | The starting offset of the data requested                                      |
| in  | <i>data_size</i>     | The amount of data requested                                                   |
| out | <i>p_data_length</i> | On success, this will contain size of the data placed in <code>p_data</code> . |

#### Returns

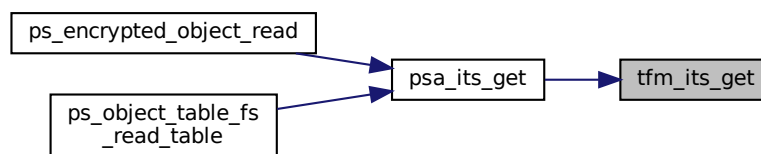
A status indicating the success/failure of the operation

## Return values

|                                   |                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                                                                                                                                                                                             |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided <code>uid</code> value was not found in the storage                                                                                                                                                                                                                                                    |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                                                                                                                                                                                                                                                       |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided arguments ( <code>p_data</code> , <code>p_data_length</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access. In addition, this can also happen if <code>data_offset</code> is larger than the size of the data associated with <code>uid</code> . |

Definition at line 578 of file `tfm_internal_trusted_storage.c`.

Here is the caller graph for this function:

8.328.1.2 `tfm_its_get_info()`

```

psa_status_t tfm_its_get_info (
 int32_t client_id,
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Retrieve the metadata about the provided `uid`.

Retrieves the metadata stored for a given `uid` as a `psa_storage_info_t` structure.

## Parameters

|     |                  |                                                                                                  |
|-----|------------------|--------------------------------------------------------------------------------------------------|
| in  | <i>client_id</i> | Identifier of the asset's owner (client)                                                         |
| in  | <i>uid</i>       | The <code>uid</code> value                                                                       |
| out | <i>p_info</i>    | A pointer to the <code>psa_storage_info_t</code> struct that will be populated with the metadata |

## Returns

A status indicating the success/failure of the operation

## Return values

|                                  |                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>               | The operation completed successfully                                                          |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>  | The operation failed because the provided <code>uid</code> value was not found in the storage |
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the physical storage has failed (Fatal error)                    |

## Return values

|                                   |                                                                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided pointers( <i>p_info</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Definition at line 642 of file `tfm_internal_trusted_storage.c`.

Here is the caller graph for this function:



## 8.328.1.3 tfm\_its\_init()

```

psa_status_t tfm_its_init (
 void)

```

Initializes the internal trusted storage system.

## Returns

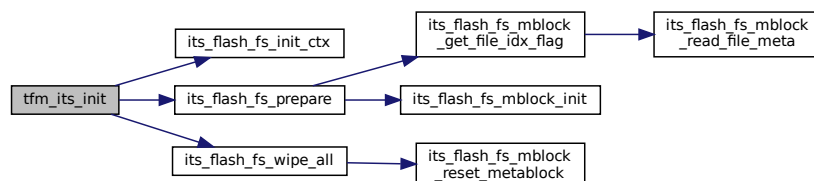
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

## Return values

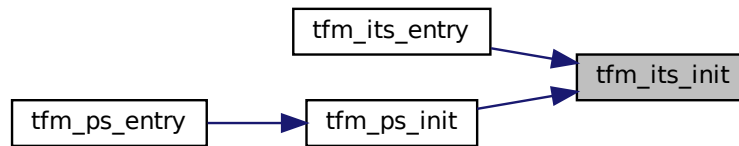
|                                  |                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>               | The operation completed successfully                                                    |
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the storage system initialization has failed (fatal error) |
| <i>PSA_ERROR_GENERIC_ERROR</i>   | The operation failed because of an unspecified internal failure                         |

Definition at line 285 of file `tfm_internal_trusted_storage.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.328.1.4 tfm\_its\_remove()

```

psa_status_t tfm_its_remove (
 int32_t client_id,
 psa_storage_uid_t uid)

```

Remove the provided uid and its associated data from the storage.  
Deletes the data from internal storage.

##### Parameters

|    |                  |                                          |
|----|------------------|------------------------------------------|
| in | <i>client_id</i> | Identifier of the asset's owner (client) |
| in | <i>uid</i>       | The uid value                            |

##### Returns

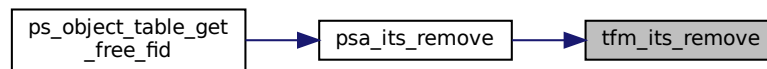
A status indicating the success/failure of the operation

##### Return values

|                                   |                                                                                                                    |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                               |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                                   |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code>      |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                         |

Definition at line 661 of file `tfm_internal_trusted_storage.c`.

Here is the caller graph for this function:



### 8.328.1.5 tfm\_its\_set()

```

psa_status_t tfm_its_set (
 int32_t client_id,
 psa_storage_uid_t uid,
 size_t data_length,
 psa_storage_create_flags_t create_flags)

```

Create a new, or modify an existing, uid/value pair.  
Stores data in the internal storage.

#### Parameters

|    |                     |                                                |
|----|---------------------|------------------------------------------------|
| in | <i>client_id</i>    | Identifier of the asset's owner (client)       |
| in | <i>uid</i>          | The identifier for the data                    |
| in | <i>data_length</i>  | The size in bytes of the data in <i>p_data</i> |
| in | <i>create_flags</i> | The flags that the data will be stored with    |

#### Returns

A status indicating the success/failure of the operation

#### Return values

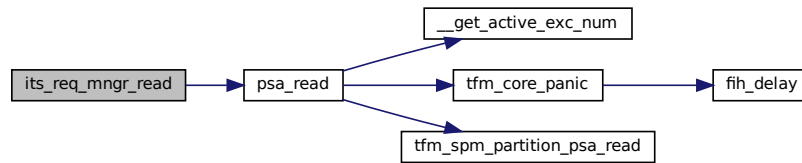
|                                       |                                                                                                                                                                        |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                                                                                   |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because one or more of the flags provided in <i>create_flags</i> is not supported or is not valid                                                 |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because there was insufficient space on the storage medium                                                                                        |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (Fatal error)                                                                                             |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because one of the provided pointers ( <i>p_data</i> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

Definition at line 424 of file `tfm_internal_trusted_storage.c`.





Here is the call graph for this function:

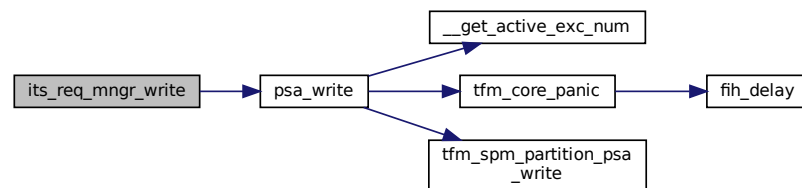


### 8.329.1.2 its\_req\_mgr\_write()

```
void its_req_mgr_write (
 const uint8_t * buf,
 size_t num_bytes)
```

Definition at line 183 of file tfm\_its\_req\_mgr.c.

Here is the call graph for this function:



### 8.329.1.3 tfm\_internal\_trusted\_storage\_service\_sfn()

```
psa_status_t tfm_internal_trusted_storage_service_sfn (
 const psa_msg_t * msg)
```

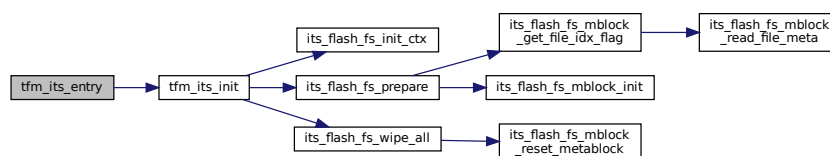
Definition at line 155 of file tfm\_its\_req\_mgr.c.

### 8.329.1.4 tfm\_its\_entry()

```
psa_status_t tfm_its_entry (
 void)
```

Definition at line 150 of file tfm\_its\_req\_mgr.c.

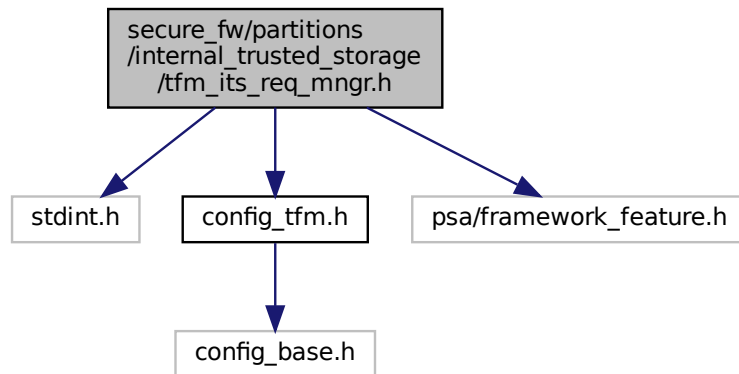
Here is the call graph for this function:



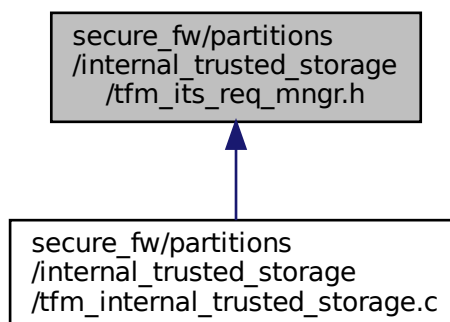
## 8.330 secure\_fw/partitions/internal\_trusted\_storage/tfm\_its\_req\_mgr.h

### File Reference

```
#include <stdint.h>
#include "config_tfm.h"
#include "psa/framework_feature.h"
Include dependency graph for tfm_its_req_mgr.h:
```



This graph shows which files directly or indirectly include this file:



### Functions

- `size_t its_req_mgr_read` (`uint8_t *buf`, `size_t num_bytes`)
- `void its_req_mgr_write` (`const uint8_t *buf`, `size_t num_bytes`)

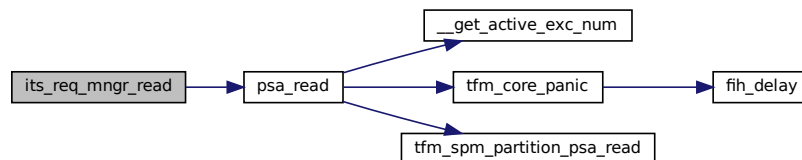
#### 8.330.1 Function Documentation

8.330.1.1 `its_req_mgr_read()`

```
size_t its_req_mgr_read (
 uint8_t * buf,
 size_t num_bytes)
```

Definition at line 178 of file `tfm_its_req_mgr.c`.

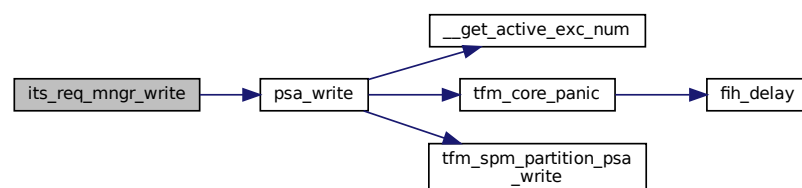
Here is the call graph for this function:

8.330.1.2 `its_req_mgr_write()`

```
void its_req_mgr_write (
 const uint8_t * buf,
 size_t num_bytes)
```

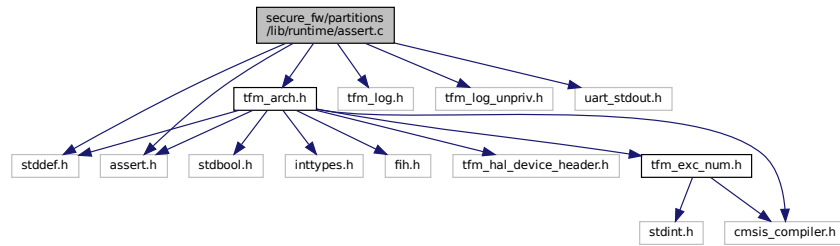
Definition at line 183 of file `tfm_its_req_mgr.c`.

Here is the call graph for this function:

8.331 `secure_fw/partitions/lib/runtime/assert.c` File Reference

```
#include <stddef.h>
#include <assert.h>
#include "tfm_log.h"
#include "tfm_log_unpriv.h"
#include "uart_stdout.h"
#include "tfm_arch.h"
```

Include dependency graph for `assert.c`:



## Functions

- `void __assert_func` (const char \*file, int line, const char \*func, const char \*reason)

### 8.331.1 Function Documentation

#### 8.331.1.1 \_\_assert\_func()

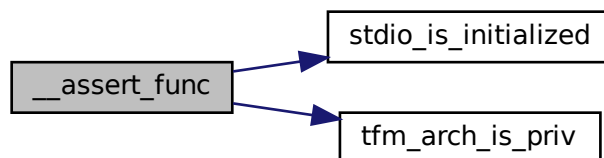
```

void __assert_func (
 const char * file,
 int line,
 const char * func,
 const char * reason)

```

Definition at line 16 of file `assert.c`.

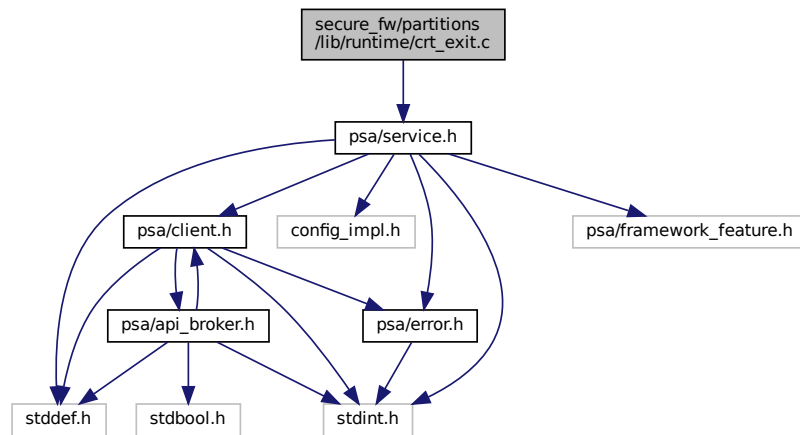
Here is the call graph for this function:



### 8.332 secure\_fw/partitions/lib/runtime/crt\_exit.c File Reference

```
#include "psa/service.h"
```

Include dependency graph for crt\_exit.c:



## Functions

- [void \\_exit](#) (int code)
- [void exit](#) (int code)

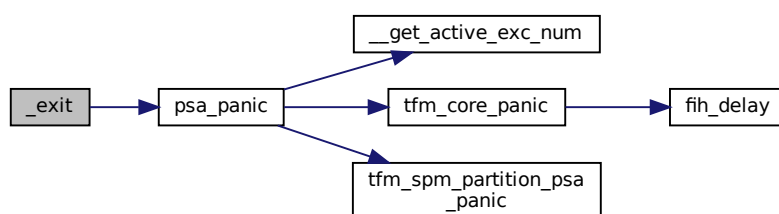
### 8.332.1 Function Documentation

#### 8.332.1.1 \_exit()

```
void _exit (
 int code)
```

Definition at line 19 of file `crt_exit.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

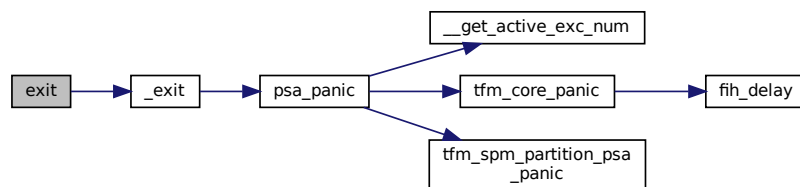


### 8.332.1.2 exit()

```
void exit (
 int code)
```

Definition at line 26 of file crt\_exit.c.

Here is the call graph for this function:

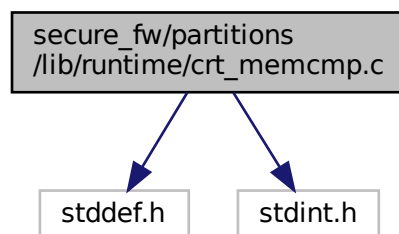


## 8.333 secure\_fw/partitions/lib/runtime/crt\_memcmp.c File Reference

```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for crt\_memcmp.c:



## Functions

- int [memcmp](#) (const void \*s1, const void \*s2, size\_t n)

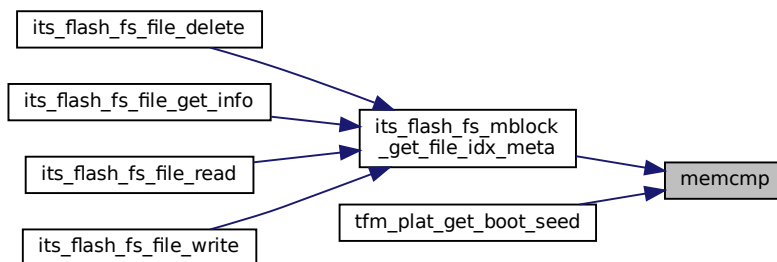
### 8.333.1 Function Documentation

#### 8.333.1.1 memcmp()

```
int memcmp (
 const void * s1,
 const void * s2,
 size_t n)
```

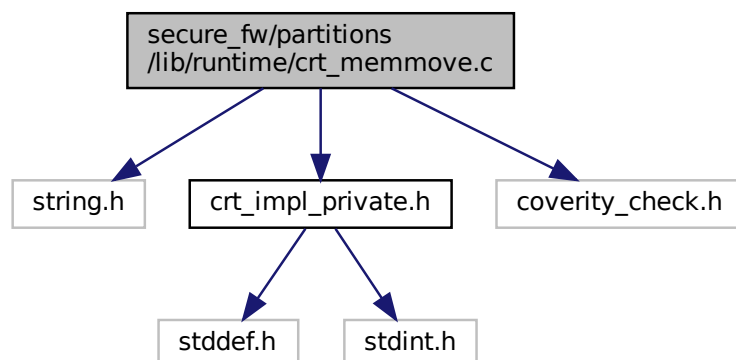
Definition at line 11 of file crt\_memcmp.c.

Here is the caller graph for this function:



### 8.334 secure\_fw/partitions/lib/runtime/crt\_memmove.c File Reference

```
#include <string.h>
#include "crt_impl_private.h"
#include "coverity_check.h"
Include dependency graph for crt_memmove.c:
```



### Macros

- #define `memcpy_f` `memcpy`

## Functions

- `void * memmove (void *dest, const void *src, size_t n)`

### 8.334.1 Macro Definition Documentation

#### 8.334.1.1 memcpy\_f

```
#define memcpy_f memcpy
```

Definition at line 43 of file crt\_memmove.c.

### 8.334.2 Function Documentation

#### 8.334.2.1 memmove()

```
void* memmove (
 void * dest,
 const void * src,
 size_t n)
```

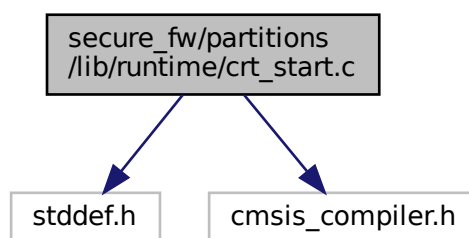
Definition at line 50 of file crt\_memmove.c.

## 8.335 secure\_fw/partitions/lib/runtime/crt\_start.c File Reference

```
#include <stddef.h>
```

```
#include "cmsis_compiler.h"
```

Include dependency graph for crt\_start.c:



## Functions

- `int main (int argc, char **argv)`
- `void _start (void)`

### 8.335.1 Function Documentation



### 8.335.1.1 `_start()`

```
void _start (
 void)
```

Definition at line 58 of file crt\_start.c.

Here is the call graph for this function:



### 8.335.1.2 `main()`

```
int main (
 int argc,
 char ** argv)
```

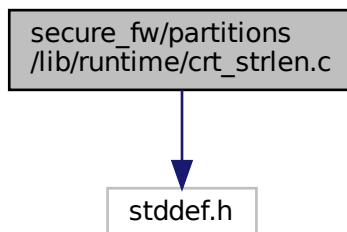
Here is the caller graph for this function:



## 8.336 secure\_fw/partitions/lib/runtime/crt\_strlen.c File Reference

```
#include <stddef.h>
```

Include dependency graph for crt\_strlen.c:



## Functions

- `size_t strlen` (`const char *s`)

### 8.336.1 Function Documentation

#### 8.336.1.1 `strlen()`

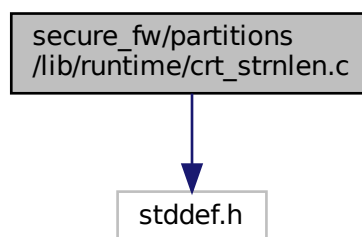
```
size_t strlen (
 const char * s)
```

Definition at line 9 of file `crt_strlen.c`.

## 8.337 `secure_fw/partitions/lib/runtime/crt_strnlen.c` File Reference

```
#include <stddef.h>
```

Include dependency graph for `crt_strnlen.c`:



## Functions

- `size_t strnlen` (`const char *s`, `size_t maxlen`)  
*Return the length of a given string `s`, up to a maximum of `maxlen` characters.*

### 8.337.1 Function Documentation

#### 8.337.1.1 `strnlen()`

```
size_t strnlen (
 const char * s,
 size_t maxlen)
```

Return the length of a given string `s`, up to a maximum of `maxlen` characters.

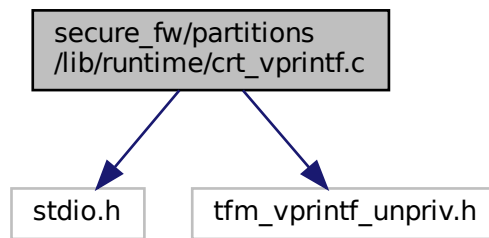
#### Parameters

|    |                     |                                              |
|----|---------------------|----------------------------------------------|
| in | <code>s</code>      | Points to the string to be examined.         |
| in | <code>maxlen</code> | The maximum number of characters to examine. |

Definition at line 10 of file `crt_strnlen.c`.

## 8.338 secure\_fw/partitions/lib/runtime/crt\_vprintf.c File Reference

```
#include <stdio.h>
#include "tfm_vprintf_unpriv.h"
Include dependency graph for crt_vprintf.c:
```



### Functions

- int [vprintf](#) (const char \*fmt, va\_list args)

### 8.338.1 Function Documentation

#### 8.338.1.1 vprintf()

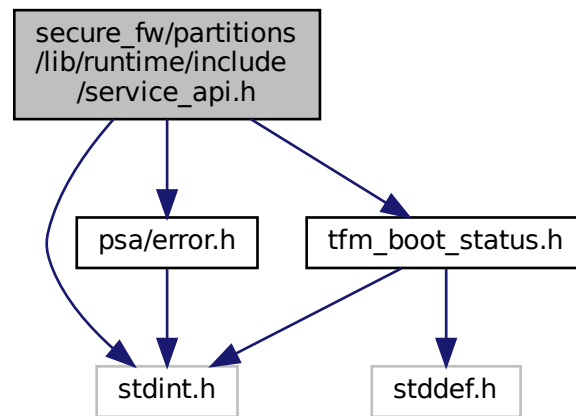
```
int vprintf (
 const char * fmt,
 va_list args)
```

Definition at line 11 of file `crt_vprintf.c`.

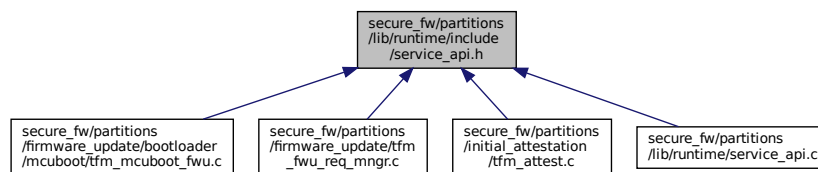
## 8.339 secure\_fw/partitions/lib/runtime/include/service\_api.h File Reference

```
#include <stdint.h>
#include "tfm_boot_status.h"
#include "psa/error.h"
```

Include dependency graph for `service_api.h`:



This graph shows which files directly or indirectly include this file:



## Functions

- [psa\\_status\\_t tfm\\_core\\_get\\_boot\\_data](#) (uint8\_t major\_type, struct [tfm\\_boot\\_data](#) \*boot\_data, uint32\_t len)

Retrieve secure partition related data from shared memory area, which stores shared data between bootloader and runtime firmware.

## 8.339.1 Function Documentation

### 8.339.1.1 tfm\_core\_get\_boot\_data()

```

psa_status_t tfm_core_get_boot_data (
 uint8_t major_type,
 struct tfm_boot_data * boot_data,
 uint32_t len)

```

Retrieve secure partition related data from shared memory area, which stores shared data between bootloader and runtime firmware.

#### Parameters

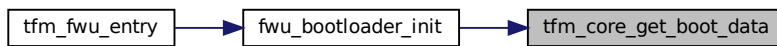
|    |                   |             |
|----|-------------------|-------------|
| in | <i>major_type</i> | Major type. |
|----|-------------------|-------------|

## Parameters

|     |                  |                              |
|-----|------------------|------------------------------|
| out | <i>boot_data</i> | Pointer to boot data.        |
| in  | <i>len</i>       | The length of the boot data. |

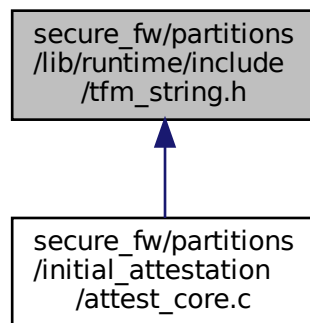
Definition at line 15 of file `service_api.c`.

Here is the caller graph for this function:



## 8.340 secure\_fw/partitions/lib/runtime/include/tfm\_string.h File Reference

This graph shows which files directly or indirectly include this file:



### Functions

- `size_t strlen (const char *s, size_t maxlen)`  
*Return the length of a given string `s`, up to a maximum of `maxlen` characters.*

#### 8.340.1 Function Documentation

##### 8.340.1.1 strlen()

```
size_t strlen (
 const char * s,
 size_t maxlen)
```

Return the length of a given string `s`, up to a maximum of `maxlen` characters.

## Parameters

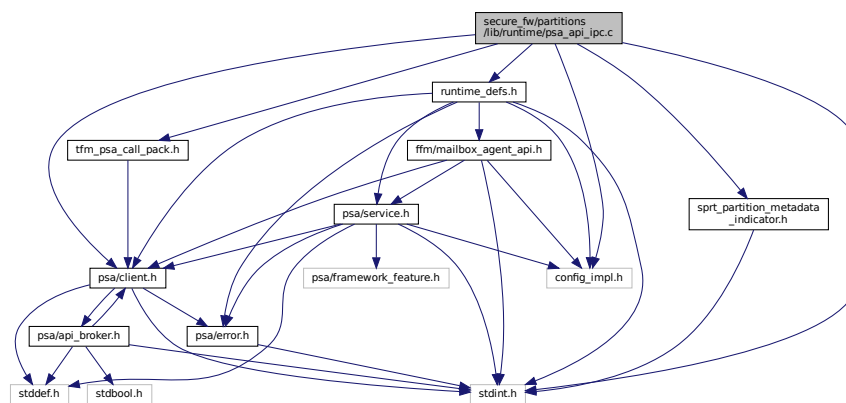
|    |               |                                              |
|----|---------------|----------------------------------------------|
| in | <i>s</i>      | Points to the string to be examined.         |
| in | <i>maxlen</i> | The maximum number of characters to examine. |

Definition at line 10 of file crt\_strnlen.c.

## 8.341 secure\_fw/partitions/lib/runtime/psa\_api\_ipc.c File Reference

```
#include <stdint.h>
#include "psa/client.h"
#include "config_impl.h"
#include "tfm_psa_call_pack.h"
#include "sprt_partition_metadata_indicator.h"
#include "runtime_defs.h"
```

Include dependency graph for `psa_api_ipc.c`:



## Functions

- `uint32_t` [psa\\_framework\\_version](#) (`void`)  
Retrieve the version of the PSA Framework API that is implemented.
- `uint32_t` [psa\\_version](#) (`uint32_t` sid)  
Retrieve the version of an RoT Service or indicate that it is not present on this system.
- `psa_status_t` [tfm\\_psa\\_call\\_pack](#) (`psa_handle_t` handle, `uint32_t` ctrl\_param, const `psa_invec` \*in\_vec, `psa_outvec` \*out\_vec)
- `psa_signal_t` [psa\\_wait](#) (`psa_signal_t` signal\_mask, `uint32_t` timeout)  
Return the Secure Partition interrupt signals that have been asserted from a subset of signals provided by the caller.
- `psa_status_t` [psa\\_get](#) (`psa_signal_t` signal, `psa_msg_t` \*msg)  
Retrieve the message which corresponds to a given RoT Service signal and remove the message from the RoT Service queue.
- `size_t` [psa\\_read](#) (`psa_handle_t` msg\_handle, `uint32_t` invec\_idx, `void` \*buffer, `size_t` num\_bytes)  
Read a message parameter or part of a message parameter from a client input vector.
- `size_t` [psa\\_skip](#) (`psa_handle_t` msg\_handle, `uint32_t` invec\_idx, `size_t` num\_bytes)  
Skip over part of a client input vector.
- `void` [psa\\_write](#) (`psa_handle_t` msg\_handle, `uint32_t` outvec\_idx, const `void` \*buffer, `size_t` num\_bytes)  
Write a message response to a client output vector.
- `void` [psa\\_reply](#) (`psa_handle_t` msg\_handle, `psa_status_t` status)  
Complete handling of a specific message and unblock the client.

- `void psa_panic (void)`  
Terminate execution within the calling Secure Partition and will not return.
- `uint32_t psa_rot_lifecycle_state (void)`

## 8.341.1 Function Documentation

### 8.341.1.1 `psa_framework_version()`

```
uint32_t psa_framework_version (
 void)
```

Retrieve the version of the PSA Framework API that is implemented.

#### Returns

version The version of the PSA Framework implementation that is providing the runtime services to the caller. The major and minor version are encoded as follows:

- version[15:8] – major version number.
- version[7:0] – minor version number.

Definition at line 20 of file `psa_api_ipc.c`.

### 8.341.1.2 `psa_get()`

```
psa_status_t psa_get (
 psa_signal_t signal,
 psa_msg_t * msg)
```

Retrieve the message which corresponds to a given RoT Service signal and remove the message from the RoT Service queue.

#### Parameters

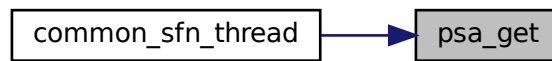
|     |               |                                                                     |
|-----|---------------|---------------------------------------------------------------------|
| in  | <i>signal</i> | The signal value for an asserted RoT Service.                       |
| out | <i>msg</i>    | Pointer to <code>psa_msg_t</code> object for receiving the message. |

#### Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>              | Success, *msg will contain the delivered message.                                                                                                                                                                                                                                                                                                        |
| <i>PSA_ERROR_DOES_NOT_EXIST</i> | Message could not be delivered.                                                                                                                                                                                                                                                                                                                          |
| <i>PROGRAMMER ERROR</i>         | The call is invalid because one or more of the following are true: <ul style="list-style-type: none"> <li>• signal has more than a single bit set.</li> <li>• signal does not correspond to an RoT Service.</li> <li>• The RoT Service signal is not currently asserted.</li> <li>• The msg pointer provided is not a valid memory reference.</li> </ul> |

Definition at line 44 of file `psa_api_ipc.c`.

Here is the caller graph for this function:



### 8.341.1.3 psa\_panic()

```
void psa_panic (
 void)
```

Terminate execution within the calling Secure Partition and will not return.

Definition at line 71 of file `psa_api_ipc.c`.

### 8.341.1.4 psa\_read()

```
size_t psa_read (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 void * buffer,
 size_t num_bytes)
```

Read a message parameter or part of a message parameter from a client input vector.

#### Parameters

|     |                   |                                                                                           |
|-----|-------------------|-------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                          |
| in  | <i>invec_idx</i>  | Index of the input vector to read from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| out | <i>buffer</i>     | Buffer in the Secure Partition to copy the requested data to.                             |
| in  | <i>num_bytes</i>  | Maximum number of bytes to be read from the client input vector.                          |

#### Return values

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>&gt;0</i>            | Number of bytes copied.                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>0</i>                | There was no remaining data in this input vector.                                                                                                                                                                                                                                                                                                                                                                |
| <i>PROGRAMMER ERROR</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• <i>msg_handle</i> is invalid.</li> <li>• <i>msg_handle</i> does not refer to a <a href="#">PSA_IPC_CALL</a> message.</li> <li>• <i>invec_idx</i> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> <li>• the memory reference for <i>buffer</i> is invalid or not writable.</li> </ul> |

Definition at line 49 of file `psa_api_ipc.c`.



### 8.341.1.5 psa\_reply()

```
void psa_reply (
 psa_handle_t msg_handle,
 psa_status_t status)
```

Complete handling of a specific message and unblock the client.

#### Parameters

|    |                   |                                                    |
|----|-------------------|----------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                   |
| in | <i>status</i>     | Message result value to be reported to the client. |

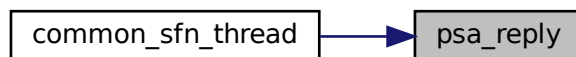
#### Note

The call is a "PROGRAMMER ERROR" if one or more of the following occur:

- *msg\_handle* is invalid.
- An invalid status code is specified for the type of message.

Definition at line 66 of file `psa_api_ipc.c`.

Here is the caller graph for this function:



### 8.341.1.6 psa\_rot\_lifecycle\_state()

```
uint32_t psa_rot_lifecycle_state (
 void)
```

Definition at line 79 of file `psa_api_ipc.c`.

### 8.341.1.7 psa\_skip()

```
size_t psa_skip (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 size_t num_bytes)
```

Skip over part of a client input vector.

#### Parameters

|    |                   |                                                                                       |
|----|-------------------|---------------------------------------------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                                                      |
| in | <i>invec_idx</i>  | Index of input vector to skip from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in | <i>num_bytes</i>  | Maximum number of bytes to skip in the client input vector.                           |

## Return values

|                         |                                                                                                                                                                                                                                                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $>0$                    | Number of bytes skipped.                                                                                                                                                                                                                                                                                                 |
| $0$                     | There was no remaining data in this input vector.                                                                                                                                                                                                                                                                        |
| <i>PROGRAMMER ERROR</i> | The call is invalid, one or more of the following are true: <ul style="list-style-type: none"> <li>• <code>msg_handle</code> is invalid.</li> <li>• <code>msg_handle</code> does not refer to a request message.</li> <li>• <code>invec_idx</code> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> </ul> |

Definition at line 55 of file `psa_api_ipc.c`.

### 8.341.1.8 `psa_version()`

```
uint32_t psa_version (
 uint32_t sid)
```

Retrieve the version of an RoT Service or indicate that it is not present on this system.

## Parameters

|                 |                  |                                 |
|-----------------|------------------|---------------------------------|
| <code>in</code> | <code>sid</code> | ID of the RoT Service to query. |
|-----------------|------------------|---------------------------------|

## Return values

|                         |                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------|
| <i>PSA_VERSION_NONE</i> | The RoT Service is not implemented, or the caller is not permitted to access the service. |
| $>$                     | $0$ The version of the implemented RoT Service.                                           |

Definition at line 25 of file `psa_api_ipc.c`.

### 8.341.1.9 `psa_wait()`

```
psa_signal_t psa_wait (
 psa_signal_t signal_mask,
 uint32_t timeout)
```

Return the Secure Partition interrupt signals that have been asserted from a subset of signals provided by the caller.

## Parameters

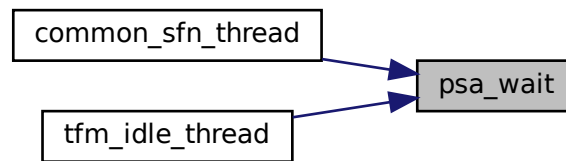
|                 |                          |                                                                                                  |
|-----------------|--------------------------|--------------------------------------------------------------------------------------------------|
| <code>in</code> | <code>signal_mask</code> | A set of signals to query. Signals that are not in this set will be ignored.                     |
| <code>in</code> | <code>timeout</code>     | Specify either blocking <a href="#">PSA_BLOCK</a> or polling <a href="#">PSA_POLL</a> operation. |

## Return values

|      |                                                                            |
|------|----------------------------------------------------------------------------|
| $>0$ | At least one signal is asserted.                                           |
| $0$  | No signals are asserted. This is only seen when a polling timeout is used. |

Definition at line 39 of file `psa_api_ipc.c`.

Here is the caller graph for this function:



#### 8.341.1.10 psa\_write()

```

void psa_write (
 psa_handle_t msg_handle,
 uint32_t outvec_idx,
 const void * buffer,
 size_t num_bytes)

```

Write a message response to a client output vector.

##### Parameters

|     |                   |                                                                                                  |
|-----|-------------------|--------------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                                 |
| out | <i>outvec_idx</i> | Index of output vector in message to write to. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in  | <i>buffer</i>     | Buffer with the data to write.                                                                   |
| in  | <i>num_bytes</i>  | Number of bytes to write to the client output vector.                                            |

##### Note

The call is a "PROGRAMMER ERROR" if one or more of the following occur:

- *msg\_handle* is invalid.
- *msg\_handle* does not refer to a request message.
- *outvec\_idx* is equal to or greater than [PSA\\_MAX\\_IOVEC](#).
- the memory reference for *buffer* is invalid.
- the call attempts to write data past the end of the client output vector.

Definition at line 60 of file `psa_api_ipc.c`.

#### 8.341.1.11 tfm\_psa\_call\_pack()

```

psa_status_t tfm_psa_call_pack (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * in_vec,
 psa_outvec * out_vec)

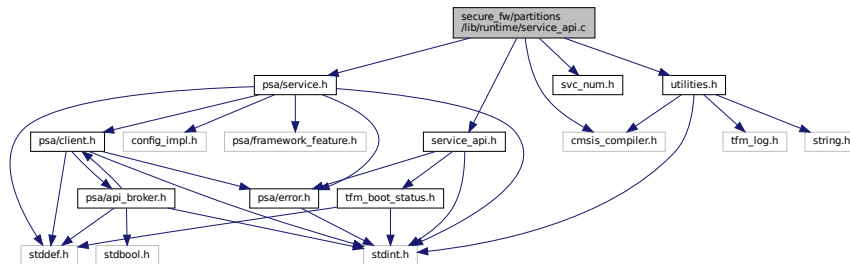
```

Definition at line 30 of file `psa_api_ipc.c`.

## 8.342 secure\_fw/partitions/lib/runtime/service\_api.c File Reference

```
#include "cmsis_compiler.h"
#include "service_api.h"
#include "psa/service.h"
#include "svc_num.h"
#include "utilities.h"
```

Include dependency graph for service\_api.c:



## Functions

- [psa\\_status\\_t tfm\\_core\\_get\\_boot\\_data](#) (uint8\_t major\_type, struct [tfm\\_boot\\_data](#) \*boot\_status, uint32\_t len)  
Retrieve secure partition related data from shared memory area, which stores shared data between bootloader and runtime firmware.
- [void tfm\\_flih\\_func\\_return](#) (psa\_flih\_result\_t result)

## 8.342.1 Function Documentation

### 8.342.1.1 tfm\_core\_get\_boot\_data()

```
psa_status_t tfm_core_get_boot_data (
 uint8_t major_type,
 struct tfm_boot_data * boot_data,
 uint32_t len)
```

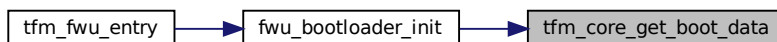
Retrieve secure partition related data from shared memory area, which stores shared data between bootloader and runtime firmware.

#### Parameters

|     |                   |                              |
|-----|-------------------|------------------------------|
| in  | <i>major_type</i> | Major type.                  |
| out | <i>boot_data</i>  | Pointer to boot data.        |
| in  | <i>len</i>        | The length of the boot data. |

Definition at line 15 of file service\_api.c.

Here is the caller graph for this function:

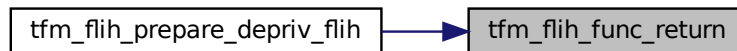


## 8.342.1.2 tfm\_flih\_func\_return()

```
void tfm_flih_func_return (
 psa_flih_result_t result)
```

Definition at line 28 of file service\_api.c.

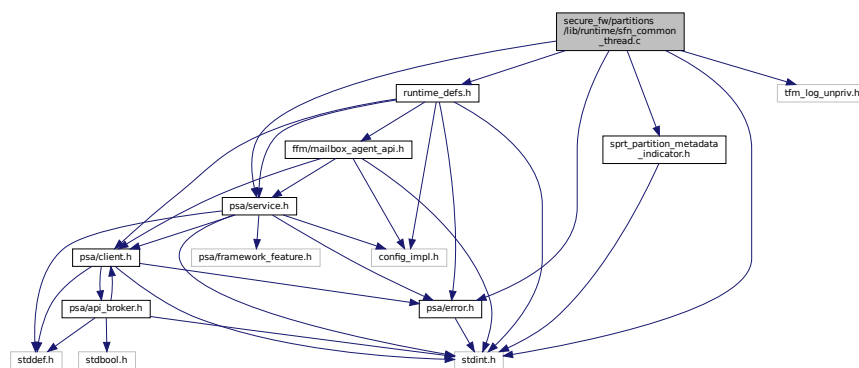
Here is the caller graph for this function:



## 8.343 secure\_fw/partitions/lib/runtime/sfn\_common\_thread.c File Reference

```
#include <stdint.h>
#include "runtime_defs.h"
#include "sprt_partition_metadata_indicator.h"
#include "tfm_log_unpriv.h"
#include "psa/error.h"
#include "psa/service.h"
```

Include dependency graph for sfn\_common\_thread.c:



## Functions

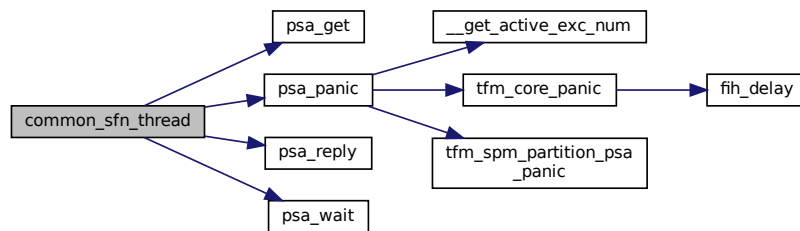
- `void common_sfn_thread (void *param)`

## 8.343.1 Function Documentation

## 8.343.1.1 common\_sfn\_thread()

```
void common_sfn_thread (
 void * param)
```

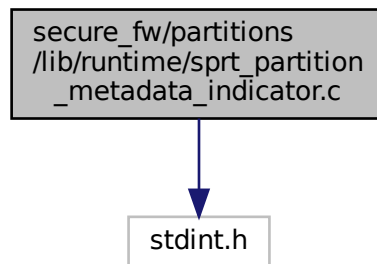
Definition at line 19 of file `sfn_common_thread.c`.  
 Here is the call graph for this function:



### 8.344 `secure_fw/partitions/lib/runtime/sprt_partition_metadata_↔ indicator.c` File Reference

```
#include <stdint.h>
```

Include dependency graph for `sprt_partition_metadata_indicator.c`:



#### Variables

- `const struct runtime_metadata_t * p_partition_metadata`

#### 8.344.1 Variable Documentation

##### 8.344.1.1 `p_partition_metadata`

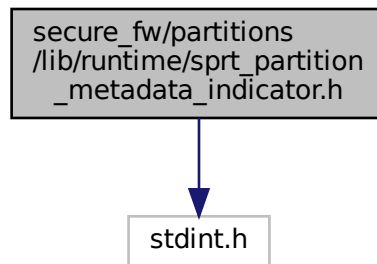
```
const struct runtime_metadata_t* p_partition_metadata
```

Definition at line 15 of file `sprt_partition_metadata_indicator.c`.

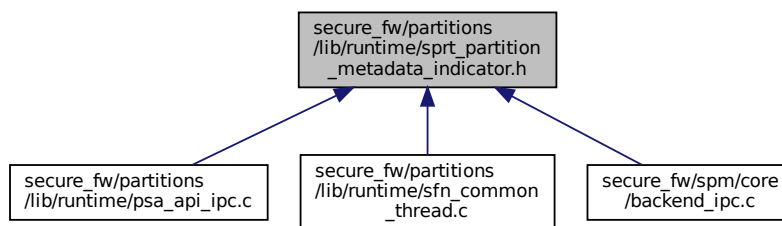
### 8.345 `secure_fw/partitions/lib/runtime/sprt_partition_metadata_↔ indicator.h` File Reference

```
#include <stdint.h>
```

Include dependency graph for sprt\_partition\_metadata\_indicator.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define PART_METADATA() p_partition_metadata`

## Variables

- `const struct runtime_metadata_t * p_partition_metadata`

## 8.345.1 Macro Definition Documentation

### 8.345.1.1 PART\_METADATA

```
#define PART_METADATA() p_partition_metadata
```

Definition at line 17 of file sprt\_partition\_metadata\_indicator.h.

## 8.345.2 Variable Documentation

### 8.345.2.1 p\_partition\_metadata

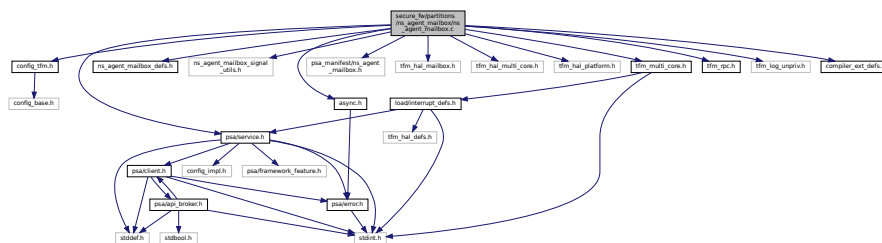
```
const struct runtime_metadata_t* p_partition_metadata
```

Definition at line 15 of file sprt\_partition\_metadata\_indicator.c.

## 8.346 secure\_fw/partitions/ns\_agent\_mailbox/ns\_agent\_mailbox.c File Reference

```
#include "async.h"
#include "config_tfm.h"
#include "ns_agent_mailbox_defs.h"
#include "ns_agent_mailbox_signal_utils.h"
#include "psa/service.h"
#include "psa_manifest/ns_agent_mailbox.h"
#include "tfm_hal_mailbox.h"
#include "tfm_hal_multi_core.h"
#include "tfm_hal_platform.h"
#include "tfm_multi_core.h"
#include "tfm_rpc.h"
#include "tfm_log_unpriv.h"
#include "compiler_ext_defs.h"
```

Include dependency graph for ns\_agent\_mailbox.c:



### Enumerations

- enum [mailbox\\_state](#) { [START](#), [NS\\_CORE\\_RUNNING](#), [LINK\\_ESTABLISHED](#) }

### Functions

- void [ns\\_agent\\_mailbox\\_entry](#) (void)

### 8.346.1 Enumeration Type Documentation

#### 8.346.1.1 mailbox\_state

```
enum mailbox_state
```

Enumerator

|                  |  |
|------------------|--|
| START            |  |
| NS_CORE_RUNNING  |  |
| LINK_ESTABLISHED |  |

Definition at line 26 of file ns\_agent\_mailbox.c.

### 8.346.2 Function Documentation



### 8.346.2.1 ns\_agent\_mailbox\_entry()

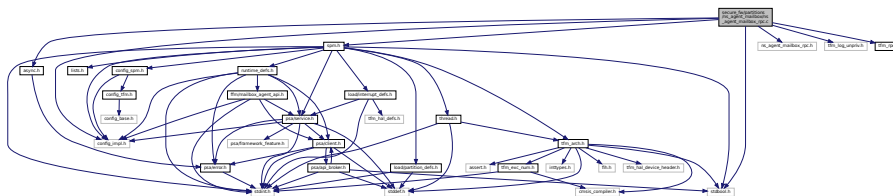
```
void ns_agent_mailbox_entry (
 void)
```

Definition at line 97 of file ns\_agent\_mailbox.c.

## 8.347 secure\_fw/partitions/ns\_agent\_mailbox/ns\_agent\_mailbox\_rpc.c File Reference

```
#include <stdbool.h>
#include "async.h"
#include "config_impl.h"
#include "ns_agent_mailbox_rpc.h"
#include "spm.h"
#include "tfm_log_unpriv.h"
#include "tfm_rpc.h"
```

Include dependency graph for ns\_agent\_mailbox\_rpc.c:



## Functions

- `int32_t tfm_rpc_register_ops` (const struct tfm\_rpc\_ops\_t \*ops\_ptr)
- `int32_t tfm_rpc_register_ops_multi_srcs` (const struct tfm\_rpc\_ops\_t \*ops\_ptr)
- `void tfm_rpc_unregister_ops` (void)
- `void tfm_rpc_client_call_handler` (psa\_signal\_t signal)
- `int32_t tfm_rpc_client_process_new_msg` (uint32\_t \*nr\_msg)

### 8.347.1 Function Documentation

#### 8.347.1.1 tfm\_rpc\_client\_call\_handler()

```
void tfm_rpc_client_call_handler (
 psa_signal_t signal)
```

Definition at line 107 of file ns\_agent\_mailbox\_rpc.c.

#### 8.347.1.2 tfm\_rpc\_client\_process\_new\_msg()

```
int32_t tfm_rpc_client_process_new_msg (
 uint32_t * nr_msg)
```

Definition at line 161 of file ns\_agent\_mailbox\_rpc.c.

#### 8.347.1.3 tfm\_rpc\_register\_ops()

```
int32_t tfm_rpc_register_ops (
 const struct tfm_rpc_ops_t * ops_ptr)
```

Definition at line 47 of file ns\_agent\_mailbox\_rpc.c.

### 8.347.1.4 tfm\_rpc\_register\_ops\_multi\_srcs()

```
int32_t tfm_rpc_register_ops_multi_srcs (
 const struct tfm_rpc_ops_t * ops_ptr)
```

Definition at line 73 of file ns\_agent\_mailbox\_rpc.c.

### 8.347.1.5 tfm\_rpc\_unregister\_ops()

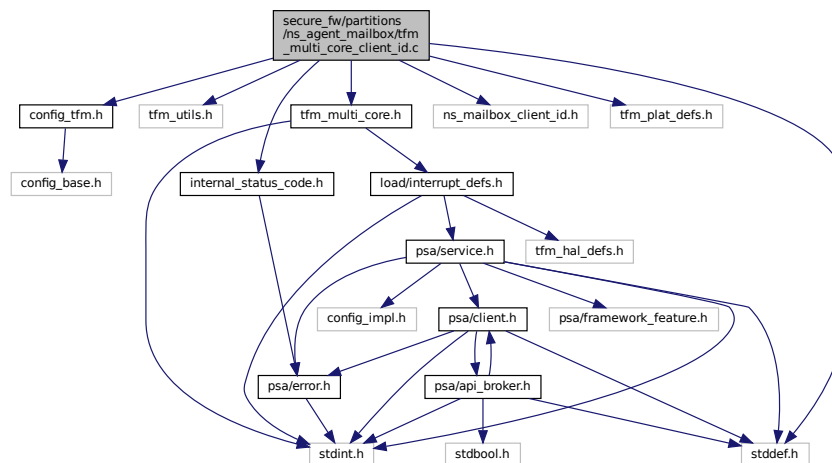
```
void tfm_rpc_unregister_ops (
 void)
```

Definition at line 99 of file ns\_agent\_mailbox\_rpc.c.

## 8.348 secure\_fw/partitions/ns\_agent\_mailbox/tfm\_multi\_core\_client\_id.c File Reference

```
#include <stddef.h>
#include "tfm_utils.h"
#include "config_tfm.h"
#include "internal_status_code.h"
#include "ns_mailbox_client_id.h"
#include "tfm_plat_defs.h"
#include "tfm_multi_core.h"
```

Include dependency graph for tfm\_multi\_core\_client\_id.c:



## Functions

- `int32_t tfm_multi_core_register_client_id_range(void *owner, uint32_t irq_source)`  
*Register a non-secure client ID range.*
- `int32_t tfm_multi_core_hal_client_id_translate(void *owner, int32_t client_id_in, int32_t *client_id_out)`  
*Translate a non-secure client ID range.*

## 8.348.1 Function Documentation

**8.348.1.1 tfm\_multi\_core\_hal\_client\_id\_translate()**

```
int32_t tfm_multi_core_hal_client_id_translate (
 void * owner,
 int32_t client_id_in,
 int32_t * client_id_out)
```

Translate a non-secure client ID range.

**Parameters**

|     |                      |                                                                                      |
|-----|----------------------|--------------------------------------------------------------------------------------|
| in  | <i>owner</i>         | Identifier of the non-secure client.                                                 |
| in  | <i>client_id_in</i>  | The input client ID.                                                                 |
| out | <i>client_id_out</i> | The translated client ID. Undefined if SPM_ERROR_GENERIC is returned by the function |

**Returns**

SPM\_SUCCESS if the translation is successful. SPM\_ERROR\_BAD\_PARAMETERS if a parameter is invalid.  
SPM\_ERROR\_GENERIC otherwise.

Definition at line 40 of file tfm\_multi\_core\_client\_id.c.

**8.348.1.2 tfm\_multi\_core\_register\_client\_id\_range()**

```
int32_t tfm_multi_core_register_client_id_range (
 void * owner,
 uint32_t irq_source)
```

Register a non-secure client ID range.

This function looks for a pre-defined client ID range in ns\_mailbox\_client\_id\_info[] according to irq\_source value. If matched, it links this non-secure client ID range to the owner.

**Note**

Each client ID range shall be registered only once.

**Parameters**

|    |                   |                                                                       |
|----|-------------------|-----------------------------------------------------------------------|
| in | <i>owner</i>      | Identifier of the non-secure client.                                  |
| in | <i>irq_source</i> | The mailbox interrupt source of the target non-secure client ID range |

**Returns**

SPM\_SUCCESS if the registration is successful. SPM\_ERROR\_BAD\_PARAMETERS if owner is null. SPM\_ERROR\_GENERIC otherwise.

Definition at line 16 of file tfm\_multi\_core\_client\_id.c.

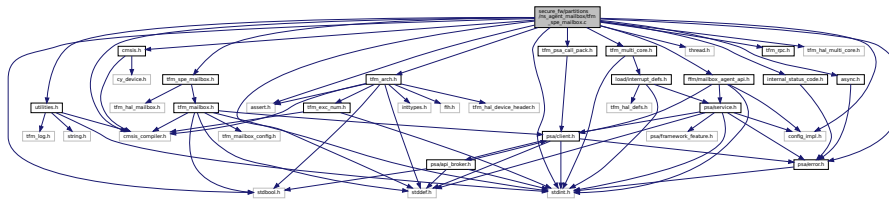
## 8.349 secure\_fw/partitions/ns\_agent\_mailbox/tfm\_multi\_core\_mbox.c File Reference

```
#include <assert.h>
#include <stddef.h>
#include "tfm_utils.h"
#include "internal_status_code.h"
#include "ns_mailbox_client_id.h"
#include "tfm_plat_defs.h"
#include "tfm_multi_core.h"
```



```
#include "async.h"
#include "config_impl.h"
#include "internal_status_code.h"
#include "psa/error.h"
#include "utilities.h"
#include "tfm_arch.h"
#include "thread.h"
#include "tfm_psa_call_pack.h"
#include "tfm_spe_mailbox.h"
#include "tfm_rpc.h"
#include "tfm_hal_multi_core.h"
#include "tfm_multi_core.h"
#include "ffm/mailbox_agent_api.h"
```

Include dependency graph for tfm\_spe\_mailbox.c:



## Data Structures

- struct [vectors](#)

## Macros

- #define [MAILBOX\\_CLEAN\\_CACHE](#)(addr, size) [\\_\\_DSB](#)()
- #define [MAILBOX\\_INVALIDATE\\_CACHE](#)(addr, size) do {} while (0)

## Functions

- [\\_\\_STATIC\\_INLINE void set\\_spe\\_queue\\_empty\\_status](#) (uint8\_t idx)
- [\\_\\_STATIC\\_INLINE void clear\\_spe\\_queue\\_empty\\_status](#) (uint8\_t idx)
- [\\_\\_STATIC\\_INLINE bool get\\_spe\\_queue\\_empty\\_status](#) (uint8\_t idx)
- [\\_\\_STATIC\\_INLINE mailbox\\_queue\\_status\\_t get\\_nspe\\_queue\\_pend\\_status](#) (const struct [mailbox\\_status\\_t](#) \*ns\_status)
- [\\_\\_STATIC\\_INLINE void set\\_nspe\\_queue\\_replied\\_status](#) (struct [mailbox\\_status\\_t](#) \*ns\_status, mailbox\_queue\_status\_t mask)
- [\\_\\_STATIC\\_INLINE void clear\\_nspe\\_queue\\_pend\\_status](#) (struct [mailbox\\_status\\_t](#) \*ns\_status, mailbox\_queue\_status\_t mask)
- [\\_\\_STATIC\\_INLINE int32\\_t get\\_spe\\_mailbox\\_msg\\_handle](#) (uint8\_t idx, mailbox\_msg\_handle\_t \*handle)
- [\\_\\_STATIC\\_INLINE int32\\_t get\\_spe\\_mailbox\\_msg\\_idx](#) (mailbox\_msg\_handle\_t handle, uint8\_t \*idx)
- [\\_\\_STATIC\\_INLINE struct mailbox\\_reply\\_t \\* get\\_nspe\\_reply\\_addr](#) (uint8\_t idx)
- [int32\\_t tfm\\_mailbox\\_handle\\_msg](#) (void)  
*Handle mailbox message(s) from NSPE.*
- [int32\\_t tfm\\_mailbox\\_reply\\_msg](#) (mailbox\_msg\_handle\_t handle, int32\_t reply)  
*Return PSA client call return result to NSPE.*
- [int32\\_t tfm\\_inter\\_core\\_comm\\_init](#) (void)  
*Initialization of the multi core communication.*
- [int32\\_t tfm\\_inter\\_core\\_comm\\_enable](#) (void)  
*Enable the multi core communication.*
- [int32\\_t tfm\\_inter\\_core\\_comm\\_disable](#) (void)

*Disable the multi core communication.*

- `int32_t tfm_inter_core_comm_deinit(void)`

*De-initialization of the multi core communication.*

## 8.350.1 Macro Definition Documentation

### 8.350.1.1 MAILBOX\_CLEAN\_CACHE

```
#define MAILBOX_CLEAN_CACHE(
 addr,
 size) __DSB()
```

Definition at line 34 of file `tfm_spe_mailbox.c`.

### 8.350.1.2 MAILBOX\_INVALIDATE\_CACHE

```
#define MAILBOX_INVALIDATE_CACHE(
 addr,
 size) do {} while (0)
```

Definition at line 35 of file `tfm_spe_mailbox.c`.

## 8.350.2 Function Documentation

### 8.350.2.1 clear\_nspe\_queue\_pend\_status()

```
__STATIC_INLINE void clear_nspe_queue_pend_status (
 struct mailbox_status_t * ns_status,
 mailbox_queue_status_t mask)
```

Definition at line 96 of file `tfm_spe_mailbox.c`.

### 8.350.2.2 clear\_spe\_queue\_empty\_status()

```
__STATIC_INLINE void clear_spe_queue_empty_status (
 uint8_t idx)
```

Definition at line 63 of file `tfm_spe_mailbox.c`.

### 8.350.2.3 get\_nspe\_queue\_pend\_status()

```
__STATIC_INLINE mailbox_queue_status_t get_nspe_queue_pend_status (
 const struct mailbox_status_t * ns_status)
```

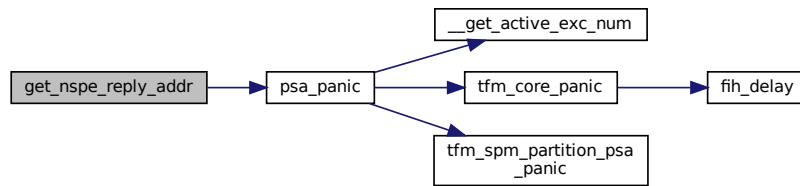
Definition at line 80 of file `tfm_spe_mailbox.c`.

### 8.350.2.4 get\_nspe\_reply\_addr()

```
__STATIC_INLINE struct mailbox_reply_t* get_nspe_reply_addr (
 uint8_t idx)
```

Definition at line 140 of file `tfm_spe_mailbox.c`.

Here is the call graph for this function:



#### 8.350.2.5 get\_spe\_mailbox\_msg\_handle()

```
__STATIC_INLINE int32_t get_spe_mailbox_msg_handle (
 uint8_t idx,
 mailbox_msg_handle_t * handle)
```

Definition at line 105 of file `tfm_spe_mailbox.c`.

#### 8.350.2.6 get\_spe\_mailbox\_msg\_idx()

```
__STATIC_INLINE int32_t get_spe_mailbox_msg_idx (
 mailbox_msg_handle_t handle,
 uint8_t * idx)
```

Definition at line 117 of file `tfm_spe_mailbox.c`.

#### 8.350.2.7 get\_spe\_queue\_empty\_status()

```
__STATIC_INLINE bool get_spe_queue_empty_status (
 uint8_t idx)
```

Definition at line 70 of file `tfm_spe_mailbox.c`.

#### 8.350.2.8 set\_nspe\_queue\_replied\_status()

```
__STATIC_INLINE void set_nspe_queue_replied_status (
 struct mailbox_status_t * ns_status,
 mailbox_queue_status_t mask)
```

Definition at line 87 of file `tfm_spe_mailbox.c`.

#### 8.350.2.9 set\_spe\_queue\_empty\_status()

```
__STATIC_INLINE void set_spe_queue_empty_status (
 uint8_t idx)
```

Definition at line 56 of file `tfm_spe_mailbox.c`.

#### 8.350.2.10 tfm\_inter\_core\_comm\_deinit()

```
int32_t tfm_inter_core_comm_deinit (
 void)
```

De-initialization of the multi core communication.

This may be called to return the multi-core communication from the initialised, but not enabled state to the pre-initialised state.

**Return values**

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>0</i>     | Operation succeeded.                             |
| <i>Other</i> | return code Operation failed with an error code. |

Definition at line 630 of file tfm\_spe\_mailbox.c.

**8.350.2.11 tfm\_inter\_core\_comm\_disable()**

```
int32_t tfm_inter_core_comm_disable (
 void)
```

Disable the multi core communication.

This may be called to return the multi core communication to the initialised, but not enabled, state.

**Return values**

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>0</i>     | Operation succeeded.                             |
| <i>Other</i> | return code Operation failed with an error code. |

Definition at line 625 of file tfm\_spe\_mailbox.c.

**8.350.2.12 tfm\_inter\_core\_comm\_enable()**

```
int32_t tfm_inter_core_comm_enable (
 void)
```

Enable the multi core communication.

This is called after tfm\_inter\_core\_init() and may be called again after tfm\_inter\_core\_disable().

**Return values**

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>0</i>     | Operation succeeded.                             |
| <i>Other</i> | return code Operation failed with an error code. |

Definition at line 620 of file tfm\_spe\_mailbox.c.

**8.350.2.13 tfm\_inter\_core\_comm\_init()**

```
int32_t tfm_inter_core_comm_init (
 void)
```

Initialization of the multi core communication.

This is called once only, after the NS core has booted.

**Return values**

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>0</i>     | Operation succeeded.                             |
| <i>Other</i> | return code Operation failed with an error code. |

Definition at line 615 of file tfm\_spe\_mailbox.c.



### 8.350.2.14 tfm\_mailbox\_handle\_msg()

```
int32_t tfm_mailbox_handle_msg (
 void)
```

Handle mailbox message(s) from NSPE.

#### Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | Successfully get PSA client call return result.  |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 350 of file tfm\_spe\_mailbox.c.

### 8.350.2.15 tfm\_mailbox\_reply\_msg()

```
int32_t tfm_mailbox_reply_msg (
 mailbox_msg_handle_t handle,
 int32_t reply)
```

Return PSA client call return result to NSPE.

#### Parameters

|    |               |                                                      |
|----|---------------|------------------------------------------------------|
| in | <i>handle</i> | The handle to the mailbox message                    |
| in | <i>reply</i>  | PSA client call return result to be written to NSPE. |

#### Return values

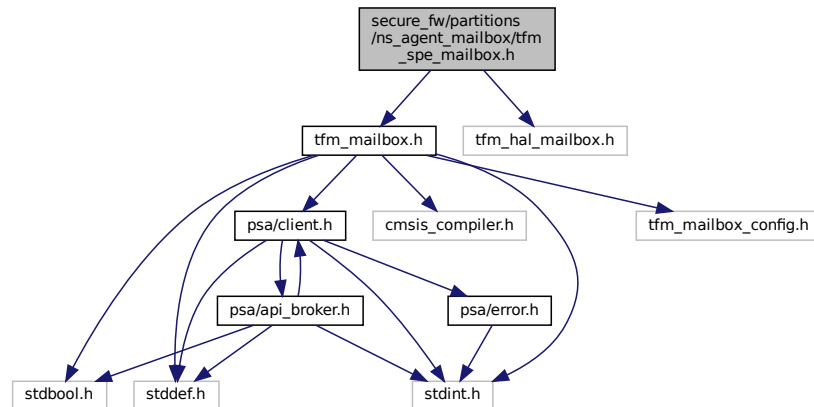
|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | Operation succeeded.                             |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 470 of file tfm\_spe\_mailbox.c.

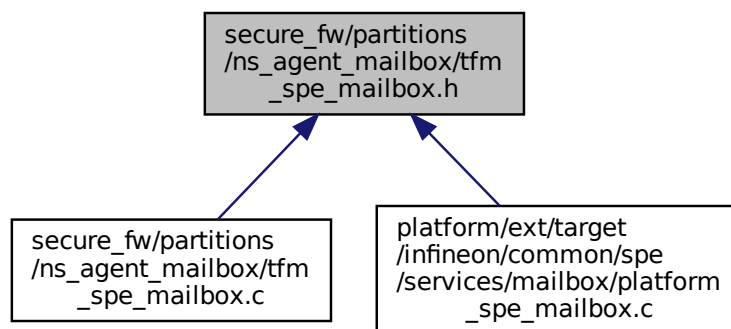
## 8.351 secure\_fw/partitions/ns\_agent\_mailbox/tfm\_spe\_mailbox.h File Reference

```
#include "tfm_mailbox.h"
#include "tfm_hal_mailbox.h"
```

Include dependency graph for `tfm_spe_mailbox.h`:



This graph shows which files directly or indirectly include this file:



## Functions

- `int32_t tfm_mailbox_handle_msg (void)`  
Handle mailbox message(s) from NSPE.
- `int32_t tfm_mailbox_reply_msg (mailbox_msg_handle_t handle, int32_t reply)`  
Return PSA client call return result to NSPE.

## 8.351.1 Function Documentation

### 8.351.1.1 tfm\_mailbox\_handle\_msg()

```
int32_t tfm_mailbox_handle_msg (
 void)
```

Handle mailbox message(s) from NSPE.

### Return values

|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | Successfully get PSA client call return result.  |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 350 of file tfm\_spe\_mailbox.c.

### 8.351.1.2 tfm\_mailbox\_reply\_msg()

```
int32_t tfm_mailbox_reply_msg (
 mailbox_msg_handle_t handle,
 int32_t reply)
```

Return PSA client call return result to NSPE.

## Parameters

|    |               |                                                      |
|----|---------------|------------------------------------------------------|
| in | <i>handle</i> | The handle to the mailbox message                    |
| in | <i>reply</i>  | PSA client call return result to be written to NSPE. |

### Return values

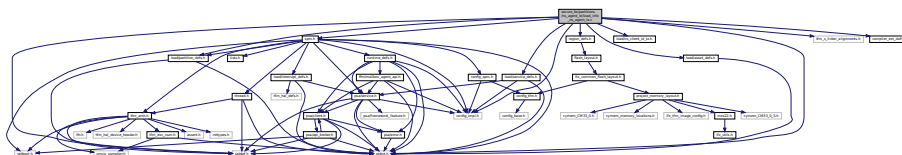
|                        |                                                  |
|------------------------|--------------------------------------------------|
| <i>MAILBOX_SUCCESS</i> | Operation succeeded.                             |
| <i>Other</i>           | return code Operation failed with an error code. |

Definition at line 470 of file tfm\_spe\_mailbox.c.

## 8.352 secure\_fw/partitions/ns\_agent\_tz/load\_info\_ns\_agent\_tz.c File Reference

```
#include <stdint.h>
#include <stddef.h>
#include "config_impl.h"
#include "spm.h"
#include "load/partition_defs.h"
#include "load/service_defs.h"
#include "load/asset_defs.h"
#include "load/ns_client_id_tz.h"
#include "region_defs.h"
#include "tfm_s_linker_alignments.h"
#include "compiler_ext_defs.h"
```

Include dependency graph for load\_info\_ns\_agent\_tz.c:



## Data Structures

- struct `partition_tfm_sp_ns_agent_tz_load_info_t`

## Macros

- `#define TFM_SP_NS_AGENT_NDEPS (0)`
- `#define TFM_SP_NS_AGENT_NSERVS (0)`

## Functions

- `void ns_agent_tz_main (uint32_t c_entry)`
- `uint8_t ns_agent_tz_stack[TFM_NS_AGENT_TZ_STACK_SIZE_ALIGNED] __aligned (TFM_LINKER_NS_AGENT_TZ_STACK_ALIGNMENT)`

## Variables

- `const struct partition_tfm_sp_ns_agent_tz_load_info_t tfm_sp_ns_agent_tz_load`

## 8.352.1 Macro Definition Documentation

### 8.352.1.1 TFM\_SP\_NS\_AGENT\_NDEPS

```
#define TFM_SP_NS_AGENT_NDEPS (0)
```

Definition at line 28 of file `load_info_ns_agent_tz.c`.

### 8.352.1.2 TFM\_SP\_NS\_AGENT\_NSERVS

```
#define TFM_SP_NS_AGENT_NSERVS (0)
```

Definition at line 29 of file `load_info_ns_agent_tz.c`.

## 8.352.2 Function Documentation

### 8.352.2.1 \_\_aligned()

```
uint8_t ns_agent_tz_stack [TFM_NS_AGENT_TZ_STACK_SIZE_ALIGNED] __aligned (
 TFM_LINKER_NS_AGENT_TZ_STACK_ALIGNMENT)
```

### 8.352.2.2 ns\_agent\_tz\_main()

```
void ns_agent_tz_main (
 uint32_t c_entry)
```

Definition at line 17 of file `ns_agent_tz.c`.

## 8.352.3 Variable Documentation

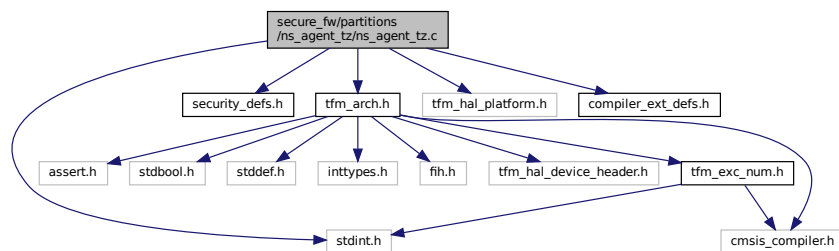
### 8.352.3.1 tfm\_sp\_ns\_agent\_tz\_load

```
const struct partition_tfm_sp_ns_agent_tz_load_info_t tfm_sp_ns_agent_tz_load
```

Definition at line 58 of file `load_info_ns_agent_tz.c`.

## 8.353 secure\_fw/partitions/ns\_agent\_tz/ns\_agent\_tz.c File Reference

```
#include <stdint.h>
#include "security_defs.h"
#include "tfm_arch.h"
#include "tfm_hal_platform.h"
#include "compiler_ext_defs.h"
Include dependency graph for ns_agent_tz.c:
```



### Functions

- `__naked void ns_agent_tz_main (uint32_t c_entry)`

#### 8.353.1 Function Documentation

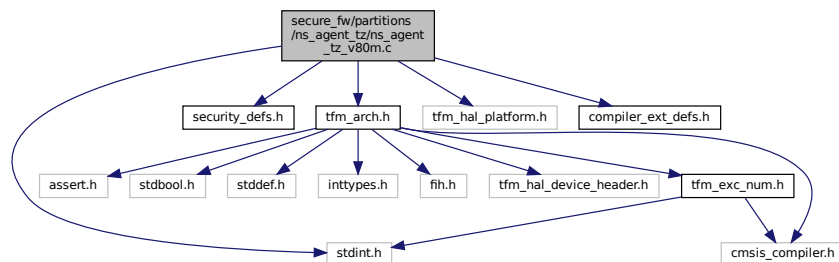
##### 8.353.1.1 ns\_agent\_tz\_main()

```
__naked void ns_agent_tz_main (
 uint32_t c_entry)
```

Definition at line 17 of file ns\_agent\_tz.c.

## 8.354 secure\_fw/partitions/ns\_agent\_tz/ns\_agent\_tz\_v80m.c File Reference

```
#include <stdint.h>
#include "security_defs.h"
#include "tfm_arch.h"
#include "tfm_hal_platform.h"
#include "compiler_ext_defs.h"
Include dependency graph for ns_agent_tz_v80m.c:
```



## Functions

- `__naked void ns_agent_tz_main (uint32_t c_entry)`

### 8.354.1 Function Documentation

#### 8.354.1.1 ns\_agent\_tz\_main()

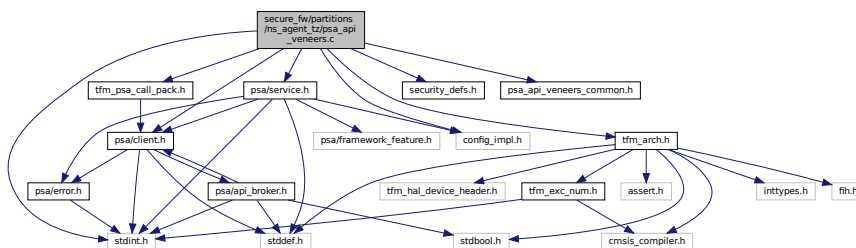
```
__naked void ns_agent_tz_main (
 uint32_t c_entry)
```

Definition at line 17 of file ns\_agent\_tz\_v80m.c.

## 8.355 secure\_fw/partitions/ns\_agent\_tz/psa\_api\_veneers.c File Reference

```
#include <stdint.h>
#include "config_impl.h"
#include "security_defs.h"
#include "tfm_arch.h"
#include "tfm_psa_call_pack.h"
#include "psa/client.h"
#include "psa/service.h"
#include "psa_api_veneers_common.h"
```

Include dependency graph for psa\_api\_veneers.c:



## Functions

- `__tz_c_veneer uint32_t tfm_psa_framework_version_veneer (void)`  
Retrieve the version of the PSA Framework API that is implemented.
- `__tz_c_veneer uint32_t tfm_psa_version_veneer (uint32_t sid)`  
Return version of secure function provided by secure binary.
- `__tz_c_veneer psa_status_t tfm_psa_call_veneer (psa_handle_t handle, uint32_t ctrl_param, const psa_invec *in_vec, psa_outvec *out_vec)`  
Call a secure function referenced by a connection handle.
- `__tz_c_veneer psa_handle_t tfm_psa_connect_veneer (uint32_t sid, uint32_t version)`  
Connect to secure function.
- `__tz_c_veneer void tfm_psa_close_veneer (psa_handle_t handle)`  
Close connection to secure function referenced by a connection handle.

### 8.355.1 Function Documentation

**8.355.1.1 tfm\_psa\_call\_veneer()**

```
__tz_c_veneer psa_status_t tfm_psa_call_veneer (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * in_vec,
 psa_outvec * out_vec)
```

Call a secure function referenced by a connection handle.

**Parameters**

|         |                   |                                                                             |
|---------|-------------------|-----------------------------------------------------------------------------|
| in      | <i>handle</i>     | Handle to connection.                                                       |
| in      | <i>ctrl_param</i> | Parameters combined in uint32_t, includes request type, in_num and out_num. |
| in      | <i>in_vec</i>     | Array of input <a href="#">psa_invec</a> structures.                        |
| in, out | <i>out_vec</i>    | Array of output <a href="#">psa_outvec</a> structures.                      |

**Returns**

Returns [psa\\_status\\_t](#) status code.

Definition at line 67 of file `psa_api_veneers.c`.

**8.355.1.2 tfm\_psa\_close\_veneer()**

```
__tz_c_veneer void tfm_psa_close_veneer (
 psa_handle_t handle)
```

Close connection to secure function referenced by a connection handle.

**Parameters**

|    |               |                      |
|----|---------------|----------------------|
| in | <i>handle</i> | Handle to connection |
|----|---------------|----------------------|

Definition at line 138 of file `psa_api_veneers.c`.

**8.355.1.3 tfm\_psa\_connect\_veneer()**

```
__tz_c_veneer psa_handle_t tfm_psa_connect_veneer (
 uint32_t sid,
 uint32_t version)
```

Connect to secure function.

**Parameters**

|    |                |                                    |
|----|----------------|------------------------------------|
| in | <i>sid</i>     | ID of secure service.              |
| in | <i>version</i> | Version of SF requested by client. |

**Returns**

Returns handle to connection.

Definition at line 129 of file `psa_api_veneers.c`.

**8.355.1.4 tfm\_psa\_framework\_version\_veneer()**

```
__tz_c_veneer uint32_t tfm_psa_framework_version_veneer (
 void)
```

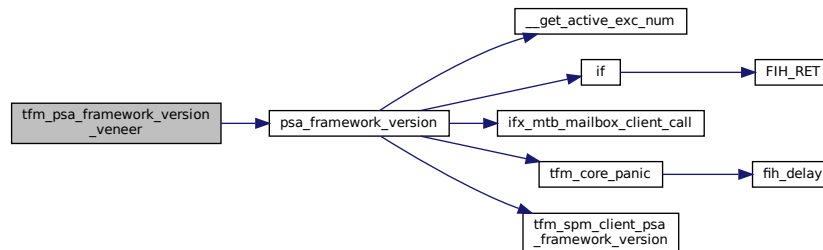
Retrieve the version of the PSA Framework API that is implemented.

#### Returns

The version of the PSA Framework.

Definition at line 29 of file `psa_api_veneers.c`.

Here is the call graph for this function:



#### 8.355.1.5 tfm\_psa\_version\_veneer()

```
__tz_c_veneer uint32_t tfm_psa_version_veneer (
 uint32_t sid)
```

Return version of secure function provided by secure binary.

#### Parameters

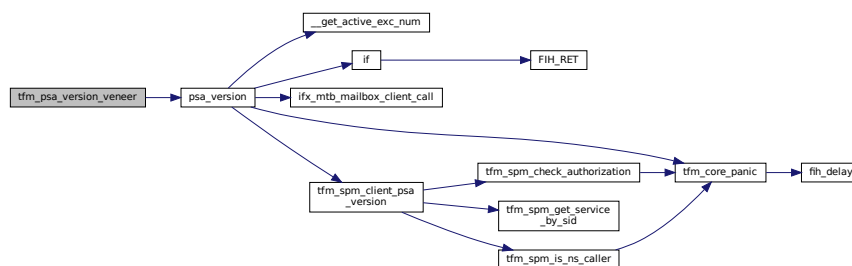
|    |            |                       |
|----|------------|-----------------------|
| in | <i>sid</i> | ID of secure service. |
|----|------------|-----------------------|

#### Returns

Version number of secure function.

Definition at line 48 of file `psa_api_veneers.c`.

Here is the call graph for this function:



## 8.356 secure\_fw/partitions/ns\_agent\_tz/psa\_api\_veneers\_common.c

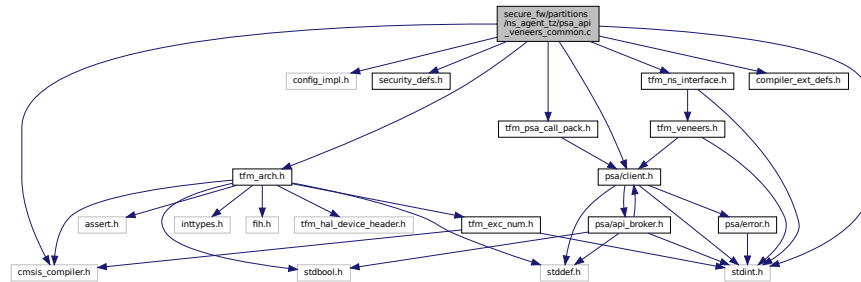
### File Reference

```
#include <stdint.h>
#include "config_impl.h"
```



```
#include "security_defs.h"
#include "tfm_arch.h"
#include "tfm_psa_call_pack.h"
#include "psa/client.h"
#include "cmsis_compiler.h"
#include "compiler_ext_defs.h"
#include "tfm_ns_interface.h"
```

Include dependency graph for psa\_api\_veneers\_common.c:



## Macros

- #define [S\\_STACK\\_INTEGRITY\\_SIGN](#) 0xFEFA125A

## Functions

- [\\_\\_tz\\_naked\\_veneer](#) int32\_t [tfm\\_ns\\_check\\_safe\\_entry\\_veneer](#) (void)  
NS-side to perform a safety check for reentrancy before proceeding with a PSA Call.
- [\\_\\_tz\\_c\\_veneer](#) int32\_t [tfm\\_ns\\_check\\_exit\\_veneer](#) (void)  
NS-side to let the test flag go.

## Variables

- volatile bool [ns\\_entry\\_flag\\_valid](#)

## 8.356.1 Macro Definition Documentation

### 8.356.1.1 S\_STACK\_INTEGRITY\_SIGN

```
#define S_STACK_INTEGRITY_SIGN 0xFEFA125A
```

Definition at line 58 of file `psa_api_veneers_common.c`.

## 8.356.2 Function Documentation

### 8.356.2.1 tfm\_ns\_check\_exit\_veneer()

```
__tz_c_veneer int32_t tfm_ns_check_exit_veneer (
 void)
```

NS-side to let the test flag go.

#### Note

Requires TFM\_TZ\_REENTRANCY\_CHECK option to be enabled

**Returns**

- 0 on success.
- 1 failure to clear the test flag.

Definition at line 118 of file psa\_api\_veneers\_common.c.

**8.356.2.2 tfm\_ns\_check\_safe\_entry\_veneer()**

```
__tz_naked_veneer int32_t tfm_ns_check_safe_entry_veneer (
 void)
```

NS-side to perform a safety check for reentrancy before proceeding with a PSA Call.

**Note**

Requires TFM\_TZ\_REENTRANCY\_CHECK option to be enabled

**Returns**

- 0 on success.
- 1 the test flag was already set.
- 2 invalid reenter attempt.

Definition at line 61 of file psa\_api\_veneers\_common.c.

**8.356.3 Variable Documentation****8.356.3.1 ns\_entry\_flag\_valid**

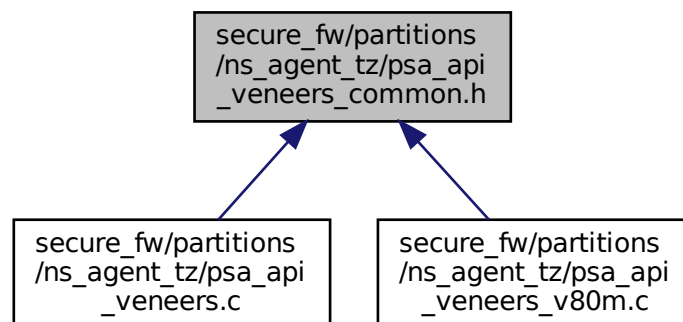
```
volatile bool ns_entry_flag_valid
```

Definition at line 52 of file psa\_api\_veneers\_common.c.

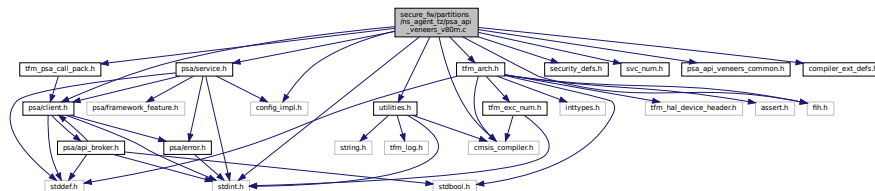
## 8.357 secure\_fw/partitions/ns\_agent\_tz/psa\_api\_veneers\_common.h

### File Reference

This graph shows which files directly or indirectly include this file:



```
#include <stdint.h>
#include "cmsis_compiler.h"
#include "config_impl.h"
#include "fih.h"
#include "security_defs.h"
#include "svc_num.h"
#include "tfm_psa_call_pack.h"
#include "utilities.h"
#include "psa/client.h"
#include "psa/service.h"
#include "tfm_arch.h"
#include "psa_api_veneers_common.h"
#include "compiler_ext_defs.h"
Include dependency graph for psa_api_veneers_v80m.c:
```



- `__tz_naked_veneer uint32_t tfm_psa_framework_version_veneer (void)`  
*Retrieve the version of the PSA Framework API that is implemented.*
- `__tz_naked_veneer uint32_t tfm_psa_version_veneer (uint32_t sid)`  
*Return version of secure function provided by secure binary.*
- `__tz_naked_veneer psa_status_t tfm_psa_call_veneer (psa_handle_t handle, uint32_t ctrl_param, const psa_invec *in_vec, psa_outvec *out_vec)`  
*Call a secure function referenced by a connection handle.*
- `__tz_naked_veneer psa_handle_t tfm_psa_connect_veneer (uint32_t sid, uint32_t version)`  
*Connect to secure function.*
- `__tz_naked_veneer void tfm_psa_close_veneer (psa_handle_t handle)`  
*Close connection to secure function referenced by a connection handle.*

- `const __used int32_t ret_err = (int32_t)PSA_ERROR_NOT_SUPPORTED`

### 8.358.1.1 tfm\_psa\_call\_veneer()

```
__tz_naked_veneer psa_status_t tfm_psa_call_veneer (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * in_vec,
 psa_outvec * out_vec)
```

Call a secure function referenced by a connection handle.

## Parameters

|         |                   |                                                                             |
|---------|-------------------|-----------------------------------------------------------------------------|
| in      | <i>handle</i>     | Handle to connection.                                                       |
| in      | <i>ctrl_param</i> | Parameters combined in uint32_t, includes request type, in_num and out_num. |
| in      | <i>in_vec</i>     | Array of input <a href="#">psa_invec</a> structures.                        |
| in, out | <i>out_vec</i>    | Array of output <a href="#">psa_outvec</a> structures.                      |

## Returns

Returns [psa\\_status\\_t](#) status code.

Definition at line 184 of file `psa_api_veneers_v80m.c`.

**8.358.1.2 tfm\_psa\_close\_veneer()**

```
__tz_naked_veneer void tfm_psa_close_veneer (
 psa_handle_t handle)
```

Close connection to secure function referenced by a connection handle.

## Parameters

|    |               |                      |
|----|---------------|----------------------|
| in | <i>handle</i> | Handle to connection |
|----|---------------|----------------------|

Definition at line 356 of file `psa_api_veneers_v80m.c`.

**8.358.1.3 tfm\_psa\_connect\_veneer()**

```
__tz_naked_veneer psa_handle_t tfm_psa_connect_veneer (
 uint32_t sid,
 uint32_t version)
```

Connect to secure function.

## Parameters

|    |                |                                    |
|----|----------------|------------------------------------|
| in | <i>sid</i>     | ID of secure service.              |
| in | <i>version</i> | Version of SF requested by client. |

## Returns

Returns handle to connection.

Definition at line 332 of file `psa_api_veneers_v80m.c`.

Here is the caller graph for this function:



### 8.358.1.4 tfm\_psa\_framework\_version\_veneer()

```
__tz_naked_veneer uint32_t tfm_psa_framework_version_veneer (
 void)
```

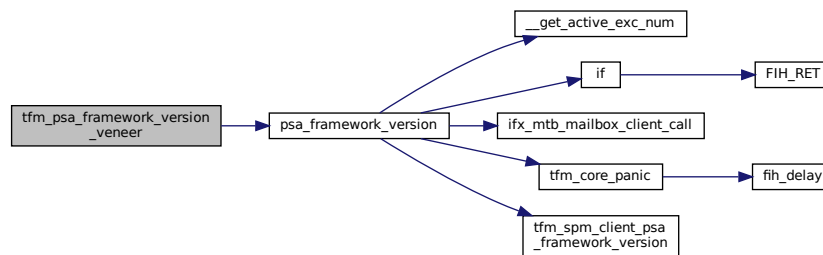
Retrieve the version of the PSA Framework API that is implemented.

#### Returns

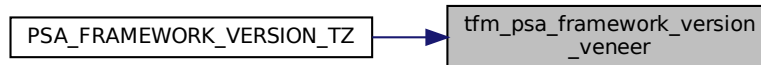
The version of the PSA Framework.

Definition at line 102 of file psa\_api\_veneers\_v80m.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.358.1.5 tfm\_psa\_version\_veneer()

```
__tz_naked_veneer uint32_t tfm_psa_version_veneer (
 uint32_t sid)
```

Return version of secure function provided by secure binary.

#### Parameters

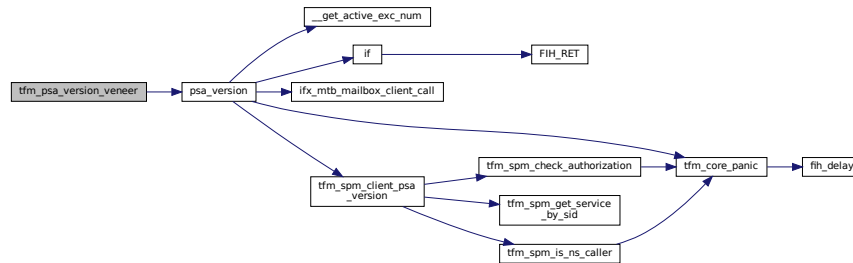
|    |            |                       |
|----|------------|-----------------------|
| in | <i>sid</i> | ID of secure service. |
|----|------------|-----------------------|

**Returns**

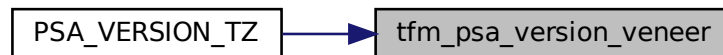
Version number of secure function.

Definition at line 143 of file psa\_api\_veneers\_v80m.c.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.358.2 Variable Documentation****8.358.2.1 ret\_err**

```
const __used int32_t ret_err = (int32_t)PSA_ERROR_NOT_SUPPORTED
```

Definition at line 326 of file psa\_api\_veneers\_v80m.c.

**8.359 secure\_fw/partitions/platform/dir\_platform.dox File Reference****8.360 secure\_fw/partitions/platform/platform\_sp.c File Reference**

```

#include "config_tfm.h"
#include "platform_sp.h"
#include "tfm_platform_system.h"
#include "load/partition_defs.h"
#include "psa_manifest/pid.h"
#include "tfm_plat_nv_counters.h"
#include "psa/client.h"
#include "psa/service.h"
#include "region_defs.h"
#include "psa_manifest/tfm_platform.h"
#include "coverity_check.h"

```





**8.360.3.1 platform\_sp\_init()**

```
psa_status_t platform_sp_init (
 void)
```

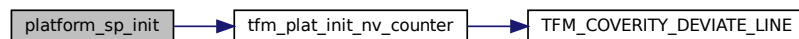
Initializes the secure partition.

**Returns**

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 236 of file platform\_sp.c.

Here is the call graph for this function:

**8.360.3.2 platform\_sp\_system\_reset()**

```
enum tfm_platform_err_t platform_sp_system_reset (
 void)
```

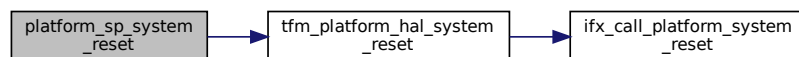
Resets the system.

**Returns**

Returns values as specified by the [tfm\\_platform\\_err\\_t](#)

Definition at line 32 of file platform\_sp.c.

Here is the call graph for this function:

**8.360.3.3 tfm\_platform\_service\_sfn()**

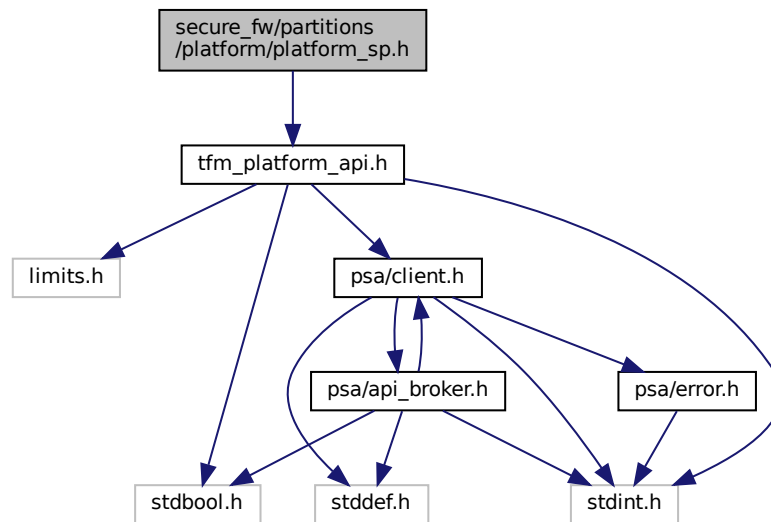
```
psa_status_t tfm_platform_service_sfn (
 const psa_msg_t * msg)
```

Definition at line 218 of file platform\_sp.c.

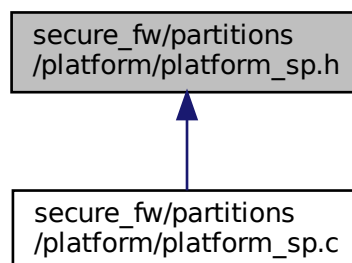
**8.361 secure\_fw/partitions/platform/platform\_sp.h File Reference**

```
#include "tfm_platform_api.h"
```

Include dependency graph for platform\_sp.h:



This graph shows which files directly or indirectly include this file:



## Functions

- `psa_status_t platform_sp_init (void)`  
*Initializes the secure partition.*
- `enum tfm_platform_err_t platform_sp_system_reset (void)`  
*Resets the system.*

### 8.361.1 Function Documentation

### 8.361.1.1 platform\_sp\_init()

```
psa_status_t platform_sp_init (
 void)
```

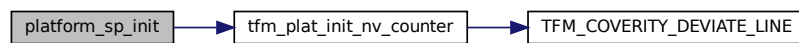
Initializes the secure partition.

#### Returns

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 236 of file platform\_sp.c.

Here is the call graph for this function:



### 8.361.1.2 platform\_sp\_system\_reset()

```
enum tfm_platform_err_t platform_sp_system_reset (
 void)
```

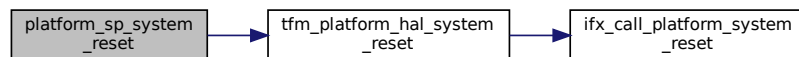
Resets the system.

#### Returns

Returns values as specified by the [tfm\\_platform\\_err\\_t](#)

Definition at line 32 of file platform\_sp.c.

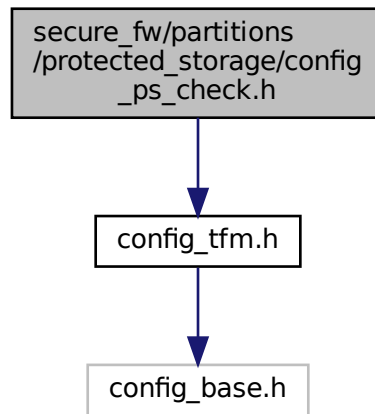
Here is the call graph for this function:



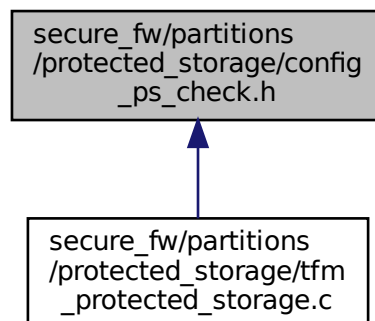
## 8.362 secure\_fw/partitions/protected\_storage/config\_ps\_check.h File Reference

```
#include "config_tfm.h"
```

Include dependency graph for config\_ps\_check.h:



This graph shows which files directly or indirectly include this file:



### 8.363 secure\_fw/partitions/protected\_storage/crypto/ps\_crypto\_interface.c File Reference

```

#include "ps_crypto_interface.h"
#include <stdbool.h>
#include <string.h>
#include "config_tfm.h"
#include "tfm_crypto_defs.h"
#include "psa/crypto.h"

```

```

graph TD
 Root["secure_fw/partitions/protected_storage/crypto
ps_crypto_interface.c"]
 Root --> Stoolool1["stoolool.h"]
 Root --> String["string.h"]
 Root --> ConfigTfm["config_tfm.h"]
 Root --> TfmCryptoDefs["tfm_crypto_defs.h"]

 TfmCryptoDefs --> ConfigBase["config_base.h"]
 TfmCryptoDefs --> PsaCryptoH["psa/crypto.h"]

 PsaCryptoH --> CryptoSizes["crypto_sizes.h"]
 PsaCryptoH --> CryptoExtra["crypto_extra.h"]
 PsaCryptoH --> CryptoValues["crypto_values.h"]
 PsaCryptoH --> CryptoCompat["crypto_compat.h"]
 PsaCryptoH --> CryptoTypes["crypto_types.h"]
 PsaCryptoH --> CryptoPlatform["crypto_platform.h"]
 PsaCryptoH --> CryptoStruct["crypto_struct.h"]
 PsaCryptoH --> MbedCryptoAccess["mbedcrypto_access.h"]

 PsaCryptoH --> PsaCryptoInterfaceH["psa_crypto_interface.h"]

 PsaCryptoInterfaceH --> TfmPlatPsaCryptoH["tfm_plat_psa_crypto.h"]
 PsaCryptoInterfaceH --> PsaProtectedStorageH["psa/protected_storage.h"]

 PsaProtectedStorageH --> PsaStorageCommonH["psa/storage_common.h"]
 PsaProtectedStorageH --> PsaMonH["psa/mon.h"]

 PsaMonH --> Stoolool2["stoolool.h"]
 PsaMonH --> PsaCryptoH

 PsaStorageCommonH --> Stoolool3["stoolool.h"]
 PsaStorageCommonH --> PsaCryptoH

 Stoolool3 --> PsaCryptoH
 Stoolool3 --> PsaCryptoPrivateAutoEnabledH["tf-psa-crypto-private_crypto_adjust_config_auto_enabled.h"]

 PsaCryptoH --> PsaCryptoConfigH["psa/crypto_config.h"]
 PsaCryptoH --> PsaCryptoPrivateAutoEnabledH
 PsaCryptoH --> PsaCryptoPrivateDependenciesH["tf-psa-crypto-private_crypto_dependencies.h"]
 PsaCryptoH --> PsaCryptoPrivateKeyPairTypesH["tf-psa-crypto-private_crypto_key_pair_types.h"]
 PsaCryptoH --> PsaCryptoPrivateDerivedH["tf-psa-crypto-private_crypto_adjust_config_derived.h"]
 PsaCryptoH --> PsaCryptoPrivateSupportH["tf-psa-crypto-private_crypto_support.h"]

 MbedCryptoAccess --> PsaCryptoH
 MbedCryptoAccess --> PsaCryptoPrivateSupportH

```

- #define PS\_CRYPT0\_AEAD\_ALG PSA\_ALG\_GCM
- #define PS\_KEY\_TYPE PSA\_KEY\_TYPE\_AES
- #define PS\_KEY\_USAGE (PSA\_KEY\_USAGE\_ENCRYPT | PSA\_KEY\_USAGE\_DECRYPT)
- #define PS\_CRYPT0\_ALG PSA\_ALG\_AEAD\_WITH\_SHORTENED\_TAG(PS\_CRYPT0\_AEAD\_ALG, PS←\_TAG\_LEN\_BYTES)
- #define LABEL\_LEN (sizeof(int32\_t) + sizeof(psa\_storage\_uid\_t))
- #define ps\_crypto\_setkey tfm\_platform\_ps\_set\_key

- typedef char PS\_ERROR\_NOT\_AEAD\_ALG[**((PSA\_ALG\_IS\_AEAD(PSA\_ALG\_AEAD\_WITH\_SHORTENED\_TAG(PSA\_ALG\_PS\_TAG\_LEN\_BYTES))) ? 1 :-1)**]

- `psa_status_t psa_crypto_init` (void)  
*Initializes the crypto engine.*
- `uint32_t psa_crypto_to_blocks` (size\_t in\_len)  
*Convert lengths to block count.*
- `void psa_crypto_set_iv` (const union `psa_crypto_t` \*crypto)  
*Provides current IV value to crypto layer.*
- `psa_status_t psa_crypto_get_iv` (union `psa_crypto_t` \*crypto)  
*Gets a new IV value into the crypto union.*
- `psa_status_t psa_crypto_encrypt_and_tag` (union `psa_crypto_t` \*crypto, const uint8\_t \*add, size\_t add\_len, const uint8\_t \*in, size\_t in\_len, uint8\_t \*out, size\_t out\_size, size\_t \*out\_len)  
*Encrypts and tags the given plaintext data.*
- `psa_status_t psa_crypto_auth_and_decrypt` (const union `psa_crypto_t` \*crypto, const uint8\_t \*add, size\_t add\_len, uint8\_t \*in, size\_t in\_len, uint8\_t \*out, size\_t out\_size, size\_t \*out\_len)  
*Decrypts and authenticates the given encrypted data.*
- `psa_status_t psa_crypto_generate_auth_tag` (union `psa_crypto_t` \*crypto, const uint8\_t \*add, uint32\_t add\_len)  
*Generates authentication tag for given data.*
- `psa_status_t psa_crypto_authenticate` (const union `psa_crypto_t` \*crypto, const uint8\_t \*add, uint32\_t add\_len)  
*Authenticate given data against the tag.*

## 8.363.1 Macro Definition Documentation

### 8.363.1.1 LABEL\_LEN

```
#define LABEL_LEN (sizeof(int32_t) + sizeof(psa_storage_uid_t))
```

Definition at line 34 of file `ps_crypto_interface.c`.

### 8.363.1.2 PS\_CRYPT0\_AEAD\_ALG

```
#define PS_CRYPT0_AEAD_ALG PSA_ALG_GCM
```

Definition at line 20 of file `ps_crypto_interface.c`.

### 8.363.1.3 PS\_CRYPT0\_ALG

```
#define PS_CRYPT0_ALG PSA_ALG_AEAD_WITH_SHORTENED_TAG(PS_CRYPT0_AEAD_ALG, PS_TAG_LEN_BYTES)
```

Definition at line 29 of file `ps_crypto_interface.c`.

### 8.363.1.4 ps\_crypto\_setkey

```
#define ps_crypto_setkey tfm_platform_ps_set_key
```

Definition at line 129 of file `ps_crypto_interface.c`.

### 8.363.1.5 PS\_KEY\_TYPE

```
#define PS_KEY_TYPE PSA_KEY_TYPE_AES
```

Definition at line 24 of file `ps_crypto_interface.c`.

### 8.363.1.6 PS\_KEY\_USAGE

```
#define PS_KEY_USAGE (PSA_KEY_USAGE_ENCRYPT | PSA_KEY_USAGE_DECRYPT)
```

Definition at line 26 of file `ps_crypto_interface.c`.

## 8.363.2 Typedef Documentation

### 8.363.2.1 PS\_ERROR\_NOT\_AEAD\_ALG

```
typedef char PS_ERROR_NOT_AEAD_ALG[(PSA_ALG_IS_AEAD(PSA_ALG_AEAD_WITH_SHORTENED_TAG(PSA_ALG_GCM, PS_TAG_LEN_BYTES))) ? 1 :-1]
```

Definition at line 46 of file `ps_crypto_interface.c`.

## 8.363.3 Function Documentation

### 8.363.3.1 ps\_crypto\_auth\_and\_decrypt()

```
psa_status_t ps_crypto_auth_and_decrypt (
 const union ps_crypto_t * crypto,
 const uint8_t * add,
 size_t add_len,
 uint8_t * in,
```

```

 size_t in_len,
 uint8_t * out,
 size_t out_size,
 size_t * out_len)

```

Decrypts and authenticates the given encrypted data.

#### Parameters

|     |                 |                                                 |
|-----|-----------------|-------------------------------------------------|
| in  | <i>crypto</i>   | Pointer to the crypto union                     |
| in  | <i>add</i>      | Pointer to the associated data                  |
| in  | <i>add_len</i>  | Length of the associated data                   |
| in  | <i>in</i>       | Pointer to the input data                       |
| in  | <i>in_len</i>   | Length of the input data                        |
| out | <i>out</i>      | Pointer to the output buffer for decrypted data |
| in  | <i>out_size</i> | Size of the output buffer                       |
| out | <i>out_len</i>  | On success, the length of the output data       |

#### Returns

Returns values as described in [psa\\_status\\_t](#)

Definition at line 259 of file ps\_crypto\_interface.c.

### 8.363.3.2 ps\_crypto\_authenticate()

```

psa_status_t ps_crypto_authenticate (
 const union ps_crypto_t * crypto,
 const uint8_t * add,
 uint32_t add_len)

```

Authenticate given data against the tag.

#### Parameters

|    |                |                                     |
|----|----------------|-------------------------------------|
| in | <i>crypto</i>  | Pointer to the crypto union         |
| in | <i>add</i>     | Pointer to the data to authenticate |
| in | <i>add_len</i> | Length of the data to authenticate  |

#### Returns

Returns values as described in [psa\\_status\\_t](#)

Definition at line 337 of file ps\_crypto\_interface.c.

### 8.363.3.3 ps\_crypto\_encrypt\_and\_tag()

```

psa_status_t ps_crypto_encrypt_and_tag (
 union ps_crypto_t * crypto,
 const uint8_t * add,
 size_t add_len,
 const uint8_t * in,
 size_t in_len,
 uint8_t * out,
 size_t out_size,
 size_t * out_len)

```

Encrypts and tags the given plaintext data.

**Parameters**

|         |                 |                                                 |
|---------|-----------------|-------------------------------------------------|
| in, out | <i>crypto</i>   | Pointer to the crypto union                     |
| in      | <i>add</i>      | Pointer to the associated data                  |
| in      | <i>add_len</i>  | Length of the associated data                   |
| in      | <i>in</i>       | Pointer to the input data                       |
| in      | <i>in_len</i>   | Length of the input data                        |
| out     | <i>out</i>      | Pointer to the output buffer for encrypted data |
| in      | <i>out_size</i> | Size of the output buffer                       |
| out     | <i>out_len</i>  | On success, the length of the output data       |

**Returns**

Returns values as described in [psa\\_status\\_t](#)

Definition at line 216 of file `ps_crypto_interface.c`.

**8.363.3.4 ps\_crypto\_generate\_auth\_tag()**

```
psa_status_t ps_crypto_generate_auth_tag (
 union ps_crypto_t * crypto,
 const uint8_t * add,
 uint32_t add_len)
```

Generates authentication tag for given data.

**Parameters**

|         |                |                                     |
|---------|----------------|-------------------------------------|
| in, out | <i>crypto</i>  | Pointer to the crypto union         |
| in      | <i>add</i>     | Pointer to the data to authenticate |
| in      | <i>add_len</i> | Length of the data to authenticate  |

**Returns**

Returns values as described in [psa\\_status\\_t](#)

Definition at line 302 of file `ps_crypto_interface.c`.

**8.363.3.5 ps\_crypto\_get\_iv()**

```
psa_status_t ps_crypto_get_iv (
 union ps_crypto_t * crypto)
```

Gets a new IV value into the crypto union.

**Parameters**

|     |               |                             |
|-----|---------------|-----------------------------|
| out | <i>crypto</i> | Pointer to the crypto union |
|-----|---------------|-----------------------------|



**Returns**

Returns values as described in [psa\\_status\\_t](#)

Definition at line 162 of file `ps_crypto_interface.c`.

Here is the call graph for this function:

**8.363.3.6 ps\_crypto\_init()**

```
psa_status_t ps_crypto_init (
 void)
```

Initializes the crypto engine.

**Returns**

Returns values as described in [psa\\_status\\_t](#)

Definition at line 132 of file `ps_crypto_interface.c`.

**8.363.3.7 ps\_crypto\_set\_iv()**

```
void ps_crypto_set_iv (
 const union ps_crypto_t * crypto)
```

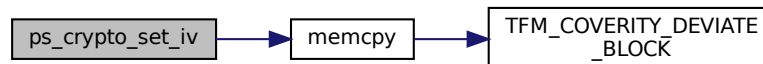
Provides current IV value to crypto layer.

**Parameters**

|    |               |                             |
|----|---------------|-----------------------------|
| in | <i>crypto</i> | Pointer to the crypto union |
|----|---------------|-----------------------------|

Definition at line 157 of file `ps_crypto_interface.c`.

Here is the call graph for this function:

**8.363.3.8 ps\_crypto\_to\_blocks()**

```
uint32_t ps_crypto_to_blocks (
 size_t in_len)
```

Convert lengths to block count.

## Parameters

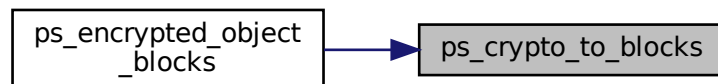
|                 |                     |                          |
|-----------------|---------------------|--------------------------|
| <code>in</code> | <code>in_len</code> | Length of the input data |
|-----------------|---------------------|--------------------------|

## Returns

Returns number of blocks encrypted/decrypted

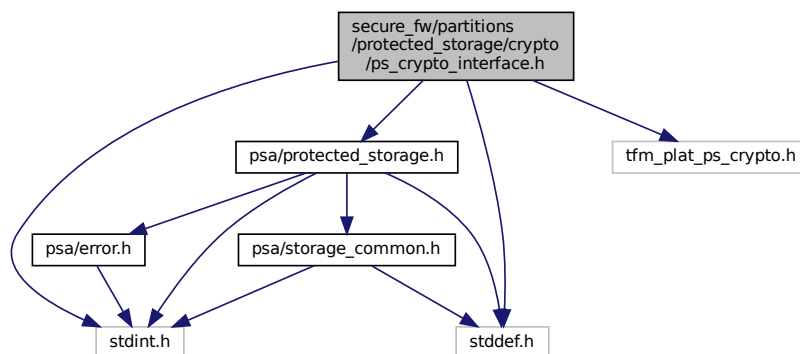
Definition at line 150 of file `ps_crypto_interface.c`.

Here is the caller graph for this function:

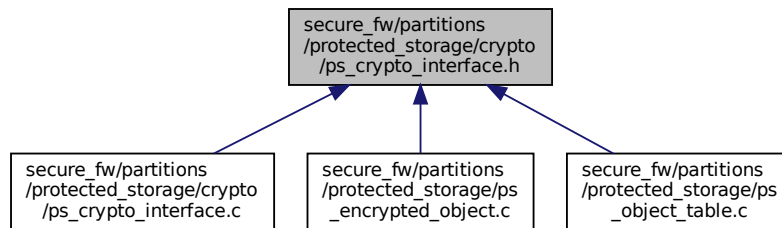


## 8.364 secure\_fw/partitions/protected\_storage/crypto/ps\_crypto\_interface.h File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "psa/protected_storage.h"
#include "tfm_plat_ps_crypto.h"
Include dependency graph for ps_crypto_interface.h:
```



This graph shows which files directly or indirectly include this file:



## Data Structures

- union [ps\\_crypto\\_t](#)

## Functions

- [psa\\_status\\_t ps\\_crypto\\_init](#) (void)  
*Initializes the crypto engine.*
- [uint32\\_t ps\\_crypto\\_to\\_blocks](#) (size\_t in\_len)  
*Convert lengths to block count.*
- [psa\\_status\\_t ps\\_crypto\\_encrypt\\_and\\_tag](#) (union [ps\\_crypto\\_t](#) \*crypto, const uint8\_t \*add, size\_t add\_len, const uint8\_t \*in, size\_t in\_len, uint8\_t \*out, size\_t out\_size, size\_t \*out\_len)  
*Encrypts and tags the given plaintext data.*
- [psa\\_status\\_t ps\\_crypto\\_auth\\_and\\_decrypt](#) (const union [ps\\_crypto\\_t](#) \*crypto, const uint8\_t \*add, size\_t add\_len, uint8\_t \*in, size\_t in\_len, uint8\_t \*out, size\_t out\_size, size\_t \*out\_len)  
*Decrypts and authenticates the given encrypted data.*
- [psa\\_status\\_t ps\\_crypto\\_generate\\_auth\\_tag](#) (union [ps\\_crypto\\_t](#) \*crypto, const uint8\_t \*add, uint32\_t add\_len)  
*Generates authentication tag for given data.*
- [psa\\_status\\_t ps\\_crypto\\_authenticate](#) (const union [ps\\_crypto\\_t](#) \*crypto, const uint8\_t \*add, uint32\_t add\_len)  
*Authenticate given data against the tag.*
- [void ps\\_crypto\\_set\\_iv](#) (const union [ps\\_crypto\\_t](#) \*crypto)  
*Provides current IV value to crypto layer.*
- [psa\\_status\\_t ps\\_crypto\\_get\\_iv](#) (union [ps\\_crypto\\_t](#) \*crypto)  
*Gets a new IV value into the crypto union.*

## 8.364.1 Function Documentation

### 8.364.1.1 ps\_crypto\_auth\_and\_decrypt()

```

psa_status_t ps_crypto_auth_and_decrypt (
 const union ps_crypto_t * crypto,
 const uint8_t * add,
 size_t add_len,
 uint8_t * in,
 size_t in_len,
 uint8_t * out,
 size_t out_size,
 size_t * out_len)

```

Decrypts and authenticates the given encrypted data.

## Parameters

|     |                 |                                                 |
|-----|-----------------|-------------------------------------------------|
| in  | <i>crypto</i>   | Pointer to the crypto union                     |
| in  | <i>add</i>      | Pointer to the associated data                  |
| in  | <i>add_len</i>  | Length of the associated data                   |
| in  | <i>in</i>       | Pointer to the input data                       |
| in  | <i>in_len</i>   | Length of the input data                        |
| out | <i>out</i>      | Pointer to the output buffer for decrypted data |
| in  | <i>out_size</i> | Size of the output buffer                       |
| out | <i>out_len</i>  | On success, the length of the output data       |

## Returns

Returns values as described in [psa\\_status\\_t](#)

Definition at line 259 of file ps\_crypto\_interface.c.

**8.364.1.2 ps\_crypto\_authenticate()**

```
psa_status_t ps_crypto_authenticate (
 const union ps_crypto_t * crypto,
 const uint8_t * add,
 uint32_t add_len)
```

Authenticate given data against the tag.

## Parameters

|    |                |                                     |
|----|----------------|-------------------------------------|
| in | <i>crypto</i>  | Pointer to the crypto union         |
| in | <i>add</i>     | Pointer to the data to authenticate |
| in | <i>add_len</i> | Length of the data to authenticate  |

## Returns

Returns values as described in [psa\\_status\\_t](#)

Definition at line 337 of file ps\_crypto\_interface.c.

**8.364.1.3 ps\_crypto\_encrypt\_and\_tag()**

```
psa_status_t ps_crypto_encrypt_and_tag (
 union ps_crypto_t * crypto,
 const uint8_t * add,
 size_t add_len,
 const uint8_t * in,
 size_t in_len,
 uint8_t * out,
 size_t out_size,
 size_t * out_len)
```

Encrypts and tags the given plaintext data.

## Parameters

|         |                |                                |
|---------|----------------|--------------------------------|
| in, out | <i>crypto</i>  | Pointer to the crypto union    |
| in      | <i>add</i>     | Pointer to the associated data |
| in      | <i>add_len</i> | Length of the associated data  |

**Parameters**

|     |                 |                                                 |
|-----|-----------------|-------------------------------------------------|
| in  | <i>in</i>       | Pointer to the input data                       |
| in  | <i>in_len</i>   | Length of the input data                        |
| out | <i>out</i>      | Pointer to the output buffer for encrypted data |
| in  | <i>out_size</i> | Size of the output buffer                       |
| out | <i>out_len</i>  | On success, the length of the output data       |

**Returns**

Returns values as described in [psa\\_status\\_t](#)

Definition at line 216 of file `ps_crypto_interface.c`.

**8.364.1.4 ps\_crypto\_generate\_auth\_tag()**

```
psa_status_t ps_crypto_generate_auth_tag (
 union ps_crypto_t * crypto,
 const uint8_t * add,
 uint32_t add_len)
```

Generates authentication tag for given data.

**Parameters**

|         |                |                                     |
|---------|----------------|-------------------------------------|
| in, out | <i>crypto</i>  | Pointer to the crypto union         |
| in      | <i>add</i>     | Pointer to the data to authenticate |
| in      | <i>add_len</i> | Length of the data to authenticate  |

**Returns**

Returns values as described in [psa\\_status\\_t](#)

Definition at line 302 of file `ps_crypto_interface.c`.

**8.364.1.5 ps\_crypto\_get\_iv()**

```
psa_status_t ps_crypto_get_iv (
 union ps_crypto_t * crypto)
```

Gets a new IV value into the crypto union.

**Parameters**

|     |               |                             |
|-----|---------------|-----------------------------|
| out | <i>crypto</i> | Pointer to the crypto union |
|-----|---------------|-----------------------------|

**Returns**

Returns values as described in [psa\\_status\\_t](#)

Definition at line 162 of file `ps_crypto_interface.c`.

Here is the call graph for this function:

**8.364.1.6 ps\_crypto\_init()**

```
psa_status_t ps_crypto_init (
 void)
```

Initializes the crypto engine.

**Returns**

Returns values as described in [psa\\_status\\_t](#)

Definition at line 132 of file `ps_crypto_interface.c`.

**8.364.1.7 ps\_crypto\_set\_iv()**

```
void ps_crypto_set_iv (
 const union ps_crypto_t * crypto)
```

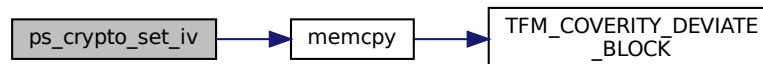
Provides current IV value to crypto layer.

**Parameters**

|    |               |                             |
|----|---------------|-----------------------------|
| in | <i>crypto</i> | Pointer to the crypto union |
|----|---------------|-----------------------------|

Definition at line 157 of file `ps_crypto_interface.c`.

Here is the call graph for this function:

**8.364.1.8 ps\_crypto\_to\_blocks()**

```
uint32_t ps_crypto_to_blocks (
 size_t in_len)
```

Convert lengths to block count.

## Parameters

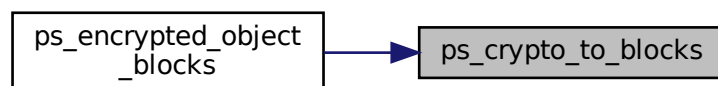
|                 |                     |                          |
|-----------------|---------------------|--------------------------|
| <code>in</code> | <code>in_len</code> | Length of the input data |
|-----------------|---------------------|--------------------------|

## Returns

Returns number of blocks encrypted/decrypted

Definition at line 150 of file `ps_crypto_interface.c`.

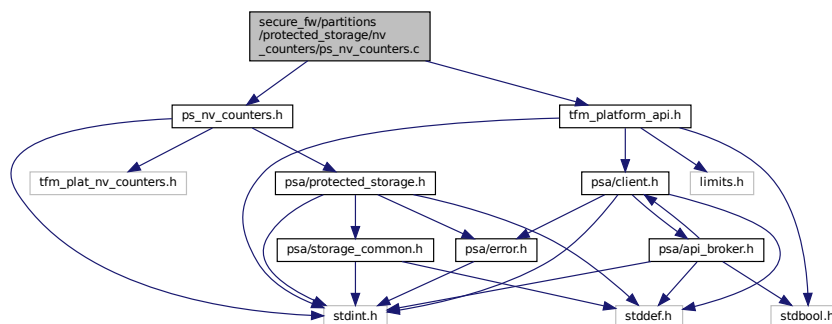
Here is the caller graph for this function:



## 8.365 secure\_fw/partitions/protected\_storage/dir\_protected\_storage.dox File Reference

## 8.366 secure\_fw/partitions/protected\_storage/nv\_counters/ps\_nv\_counters.c File Reference

```
#include "ps_nv_counters.h"
#include "tfm_platform_api.h"
Include dependency graph for ps_nv_counters.c:
```



## Functions

- [psa\\_status\\_t ps\\_read\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id, uint32\_t \*val)  
*Reads the given non-volatile (NV) counter.*
- [psa\\_status\\_t ps\\_increment\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id)  
*Increments the given non-volatile (NV) counter.*

### 8.366.1 Function Documentation



### 8.366.1.1 ps\_increment\_nv\_counter()

```
psa_status_t ps_increment_nv_counter (
 enum tfm_nv_counter_t counter_id)
```

Increments the given non-volatile (NV) counter.

#### Parameters

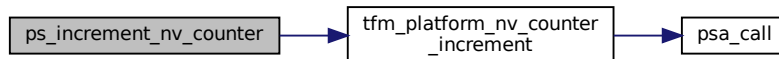
|    |                                |                |
|----|--------------------------------|----------------|
| in | <i>counter</i> ↔<br><i>_id</i> | NV counter ID. |
|----|--------------------------------|----------------|

#### Returns

If the counter is incremented correctly, it returns PSA\_SUCCESS. Otherwise, PSA\_ERROR\_GENERIC\_ERROR.

Definition at line 29 of file ps\_nv\_counters.c.

Here is the call graph for this function:



### 8.366.1.2 ps\_read\_nv\_counter()

```
psa_status_t ps_read_nv_counter (
 enum tfm_nv_counter_t counter_id,
 uint32_t * val)
```

Reads the given non-volatile (NV) counter.

#### Parameters

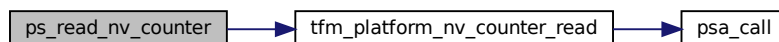
|     |                                |                                                |
|-----|--------------------------------|------------------------------------------------|
| in  | <i>counter</i> ↔<br><i>_id</i> | NV counter ID.                                 |
| out | <i>val</i>                     | Pointer to store the current NV counter value. |

#### Returns

PSA\_SUCCESS if the value is read correctly, otherwise PSA\_ERROR\_GENERIC\_ERROR

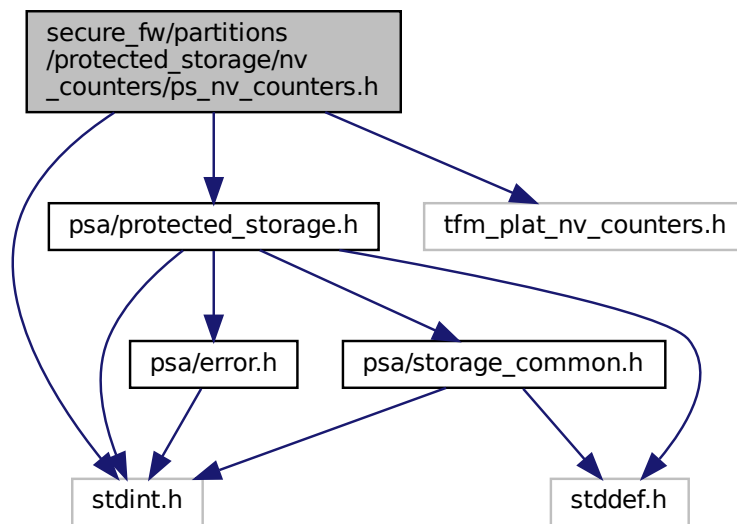
Definition at line 11 of file ps\_nv\_counters.c.

Here is the call graph for this function:

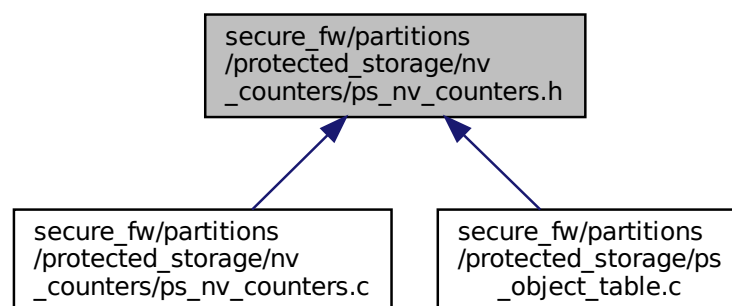


## 8.367 secure\_fw/partitions/protected\_storage/nv\_counters/ps\_nv\_counters.h File Reference

```
#include <stdint.h>
#include "psa/protected_storage.h"
#include "tfm_plat_nv_counters.h"
Include dependency graph for ps_nv_counters.h:
```



This graph shows which files directly or indirectly include this file:



### Macros

- `#define TFM_PS_NV_COUNTER_1 PLAT_NV_COUNTER_PS_0`
- `#define TFM_PS_NV_COUNTER_2 PLAT_NV_COUNTER_PS_1`
- `#define TFM_PS_NV_COUNTER_3 PLAT_NV_COUNTER_PS_2`
- `#define PS_NV_COUNTER_SIZE 4 /* In bytes */`

## Functions

- [psa\\_status\\_t ps\\_read\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id, uint32\_t \*val)  
*Reads the given non-volatile (NV) counter.*
- [psa\\_status\\_t ps\\_increment\\_nv\\_counter](#) (enum [tfm\\_nv\\_counter\\_t](#) counter\_id)  
*Increments the given non-volatile (NV) counter.*

## 8.367.1 Macro Definition Documentation

### 8.367.1.1 PS\_NV\_COUNTER\_SIZE

```
#define PS_NV_COUNTER_SIZE 4 /* In bytes */
```

Definition at line 27 of file [ps\\_nv\\_counters.h](#).

### 8.367.1.2 TFM\_PS\_NV\_COUNTER\_1

```
#define TFM_PS_NV_COUNTER_1 PLAT_NV_COUNTER_PS_0
```

Definition at line 23 of file [ps\\_nv\\_counters.h](#).

### 8.367.1.3 TFM\_PS\_NV\_COUNTER\_2

```
#define TFM_PS_NV_COUNTER_2 PLAT_NV_COUNTER_PS_1
```

Definition at line 24 of file [ps\\_nv\\_counters.h](#).

### 8.367.1.4 TFM\_PS\_NV\_COUNTER\_3

```
#define TFM_PS_NV_COUNTER_3 PLAT_NV_COUNTER_PS_2
```

Definition at line 25 of file [ps\\_nv\\_counters.h](#).

## 8.367.2 Function Documentation

### 8.367.2.1 ps\_increment\_nv\_counter()

```
psa_status_t ps_increment_nv_counter (
 enum tfm_nv_counter_t counter_id)
```

Increments the given non-volatile (NV) counter.

#### Parameters

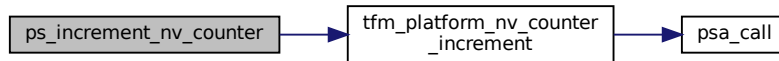
|    |                                  |                |
|----|----------------------------------|----------------|
| in | <a href="#">counter</a> ↔<br>_id | NV counter ID. |
|----|----------------------------------|----------------|

**Returns**

If the counter is incremented correctly, it returns `PSA_SUCCESS`. Otherwise, `PSA_ERROR_GENERIC_ERROR`.

Definition at line 29 of file `ps_nv_counters.c`.

Here is the call graph for this function:

**8.367.2.2 ps\_read\_nv\_counter()**

```

psa_status_t ps_read_nv_counter (
 enum tfm_nv_counter_t counter_id,
 uint32_t * val)

```

Reads the given non-volatile (NV) counter.

**Parameters**

|     |                   |                                                |
|-----|-------------------|------------------------------------------------|
| in  | <i>counter_id</i> | NV counter ID.                                 |
| out | <i>val</i>        | Pointer to store the current NV counter value. |

**Returns**

`PSA_SUCCESS` if the value is read correctly, otherwise `PSA_ERROR_GENERIC_ERROR`

Definition at line 11 of file `ps_nv_counters.c`.

Here is the call graph for this function:



## 8.368 secure\_fw/partitions/protected\_storage/ps\_encrypted\_object.c

### File Reference

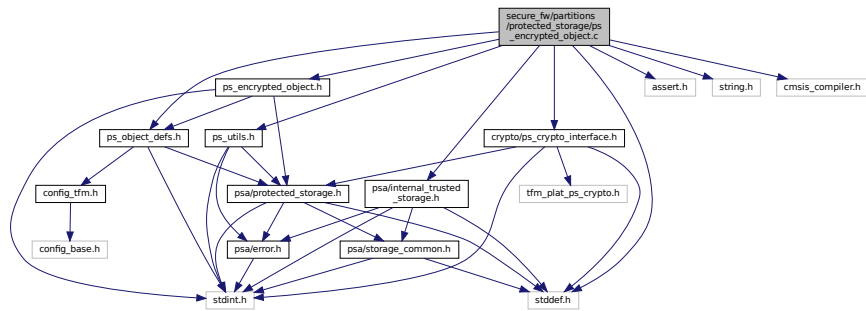
```

#include "ps_encrypted_object.h"
#include <assert.h>
#include <stddef.h>
#include <string.h>
#include "cmsis_compiler.h"
#include "crypto/ps_crypto_interface.h"
#include "psa/internal_trusted_storage.h"
#include "ps_object_defs.h"

```

```
#include "ps_utils.h"
```

Include dependency graph for ps\_encrypted\_object.c:



## Macros

- `#define STORED_HEADER_DATA_SIZE`
- `#define PS_ENCRYPT_SIZE(plaintext_size) ((plaintext_size) + offsetof(struct ps_object_t, data) - offsetof(struct ps_object_t, header.info))`
- `#define PS_OBJECT_START_POSITION 0`
- `#define PS_MAX_ENCRYPTED_OBJ_SIZE PS_ENCRYPT_SIZE(PS_MAX_OBJECT_DATA_SIZE)`
- `#define PS_CRYPTO_BUF_LEN (PS_MAX_ENCRYPTED_OBJ_SIZE + PS_TAG_LEN_BYTES)`

## Functions

- `psa_status_t ps_encrypted_object_read (uint32_t fid, struct ps_object_t *obj, uint32_t *p_blocks)`  
*Reads object referenced by the object File ID.*
- `uint32_t ps_encrypted_object_blocks (uint32_t size)`  
*Determines the number of encryption blocks that will be used to write an object of the specified size.*
- `psa_status_t ps_encrypted_object_write (uint32_t fid, struct ps_object_t *obj)`  
*Creates and writes a new encrypted object based on the given ps\_object\_t structure data.*

## Variables

- `__PACKED_STRUCT auth_data_t`

## 8.368.1 Macro Definition Documentation

### 8.368.1.1 PS\_CRYPTO\_BUF\_LEN

```
#define PS_CRYPTO_BUF_LEN (PS_MAX_ENCRYPTED_OBJ_SIZE + PS_TAG_LEN_BYTES)
```

Definition at line 37 of file ps\_encrypted\_object.c.

### 8.368.1.2 PS\_ENCRYPT\_SIZE

```
#define PS_ENCRYPT_SIZE(
 plaintext_size) ((plaintext_size) + offsetof(struct ps_object_t, data) - offsetof(struct
 ps_object_t, header.info))
```

Definition at line 25 of file ps\_encrypted\_object.c.

### 8.368.1.3 PS\_MAX\_ENCRYPTED\_OBJ\_SIZE

```
#define PS_MAX_ENCRYPTED_OBJ_SIZE PS_ENCRYPT_SIZE (PS_MAX_OBJECT_DATA_SIZE)
```

Definition at line 32 of file `ps_encrypted_object.c`.

### 8.368.1.4 PS\_OBJECT\_START\_POSITION

```
#define PS_OBJECT_START_POSITION 0
```

Definition at line 28 of file `ps_encrypted_object.c`.

### 8.368.1.5 STORED\_HEADER\_DATA\_SIZE

```
#define STORED_HEADER_DATA_SIZE
```

**Value:**

```
(offsetof(struct ps_object_t, header.info) \
 - offsetof(struct ps_object_t, header.crypto.ref.iv))
```

Definition at line 21 of file `ps_encrypted_object.c`.

## 8.368.2 Function Documentation

### 8.368.2.1 ps\_encrypted\_object\_blocks()

```
uint32_t ps_encrypted_object_blocks (
 uint32_t size)
```

Determines the number of encryption blocks that will be used to write an object of the specified size.

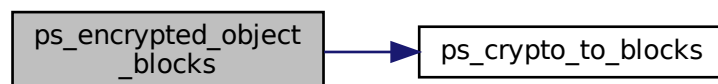
**Parameters**

|    |      |                                            |
|----|------|--------------------------------------------|
| in | size | Size in bytes of the object to be written. |
|----|------|--------------------------------------------|

**Returns**

Returns number of blocks

Definition at line 202 of file `ps_encrypted_object.c`.  
Here is the call graph for this function:



### 8.368.2.2 ps\_encrypted\_object\_read()

```
psa_status_t ps_encrypted_object_read (
 uint32_t fid,
 struct ps_object_t * obj,
 uint32_t * p_blocks)
```

Reads object referenced by the object File ID.

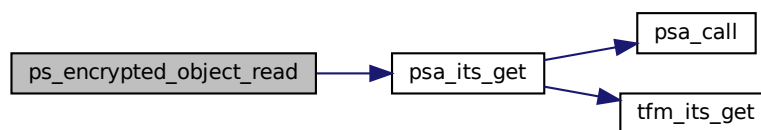
**Parameters**

|     |                 |                                                 |
|-----|-----------------|-------------------------------------------------|
| in  | <i>fid</i>      | File ID                                         |
| out | <i>obj</i>      | Pointer to the object structure to fill in      |
| out | <i>p_blocks</i> | Pointer to a counter of decryption blocks used. |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 149 of file `ps_encrypted_object.c`.  
Here is the call graph for this function:

**8.368.2.3 ps\_encrypted\_object\_write()**

```

psa_status_t ps_encrypted_object_write (
 uint32_t fid,
 struct ps_object_t * obj)

```

Creates and writes a new encrypted object based on the given [ps\\_object\\_t](#) structure data.

**Parameters**

|         |            |                                           |
|---------|------------|-------------------------------------------|
| in      | <i>fid</i> | File ID                                   |
| in, out | <i>obj</i> | Pointer to the object structure to write. |

Note: The function will use `obj` to store the encrypted data before write it into the flash to reduce the memory requirements and the number of internal copies. So, this object will contain the encrypted object stored in the flash.

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 209 of file `ps_encrypted_object.c`.

**8.368.3 Variable Documentation****8.368.3.1 auth\_data\_t**

`__PACKED_STRUCT` `auth_data_t`

**Initial value:**

```

{
 uint32_t fid

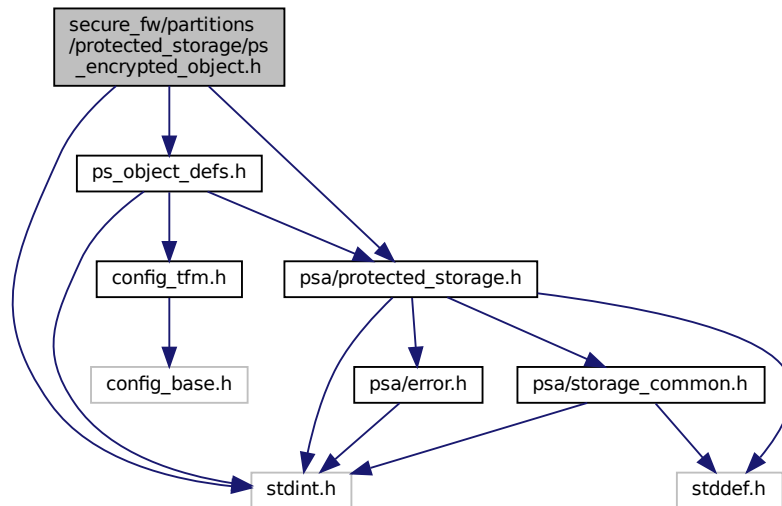
```

Definition at line 39 of file `ps_encrypted_object.c`.

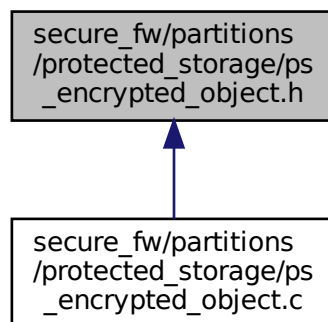


## 8.369 secure\_fw/partitions/protected\_storage/ps\_encrypted\_object.h File Reference

```
#include <stdint.h>
#include "ps_object_defs.h"
#include "psa/protected_storage.h"
Include dependency graph for ps_encrypted_object.h:
```



This graph shows which files directly or indirectly include this file:



## Functions

- [`psa\_status\_t ps\_encrypted\_object\_read`](#) (`uint32_t fid`, `struct ps_object_t *obj`, `uint32_t *p_blocks`)  
*Reads object referenced by the object File ID.*
- [`psa\_status\_t ps\_encrypted\_object\_write`](#) (`uint32_t fid`, `struct ps_object_t *obj`)  
*Creates and writes a new encrypted object based on the given [`ps\_object\_t`](#) structure data.*

- uint32\_t [ps\\_encrypted\\_object\\_blocks](#) (uint32\_t size)

*Determines the number of encryption blocks that will be used to write an object of the specified size.*

## 8.369.1 Function Documentation

### 8.369.1.1 ps\_encrypted\_object\_blocks()

```
uint32_t ps_encrypted_object_blocks (
 uint32_t size)
```

Determines the number of encryption blocks that will be used to write an object of the specified size.

#### Parameters

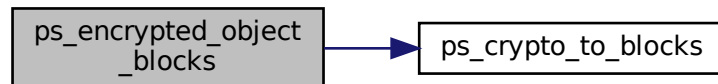
|    |      |                                            |
|----|------|--------------------------------------------|
| in | size | Size in bytes of the object to be written. |
|----|------|--------------------------------------------|

#### Returns

Returns number of blocks

Definition at line 202 of file ps\_encrypted\_object.c.

Here is the call graph for this function:



### 8.369.1.2 ps\_encrypted\_object\_read()

```
psa_status_t ps_encrypted_object_read (
 uint32_t fid,
 struct ps_object_t * obj,
 uint32_t * p_blocks)
```

Reads object referenced by the object File ID.

#### Parameters

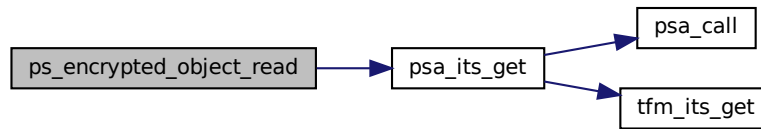
|     |          |                                                 |
|-----|----------|-------------------------------------------------|
| in  | fid      | File ID                                         |
| out | obj      | Pointer to the object structure to fill in      |
| out | p_blocks | Pointer to a counter of decryption blocks used. |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 149 of file `ps_encrypted_object.c`.

Here is the call graph for this function:

**8.369.1.3 ps\_encrypted\_object\_write()**

```

psa_status_t ps_encrypted_object_write (
 uint32_t fid,
 struct ps_object_t * obj)

```

Creates and writes a new encrypted object based on the given [ps\\_object\\_t](#) structure data.

**Parameters**

|         |            |                                           |
|---------|------------|-------------------------------------------|
| in      | <i>fid</i> | File ID                                   |
| in, out | <i>obj</i> | Pointer to the object structure to write. |

Note: The function will use `obj` to store the encrypted data before write it into the flash to reduce the memory requirements and the number of internal copies. So, this object will contain the encrypted object stored in the flash.

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 209 of file `ps_encrypted_object.c`.

## 8.370 secure\_fw/partitions/protected\_storage/ps\_filesystem\_interface.c File Reference

```

#include "psa/client.h"
#include "psa_manifest/sid.h"
#include "tfm_its_defs.h"
#include "psa_manifest/pid.h"
#include "tfm_internal_trusted_storage.h"

```



**Returns**

A status indicating the success/failure of the operation

**Return values**

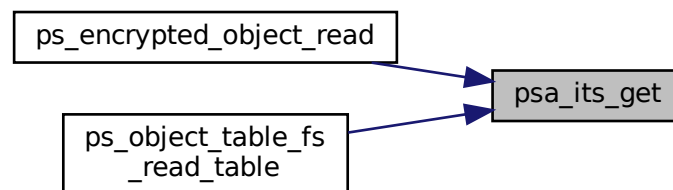
|                                   |                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                                                                                                                                                                                           |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided <code>uid</code> value was not found in the storage                                                                                                                                                                                                                                                  |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                                                                                                                                                                                                                                                     |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided arguments ( <code>p_data</code> , <code>p_data_length</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access. In addition, this can also happen if <code>data_offset</code> is larger than the size of the data associated with <code>uid</code> |

Definition at line 27 of file `ps_filesystem_interface.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.370.1.2 psa\_its\_get\_info()**

```

psa_status_t psa_its_get_info (
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Retrieve the metadata about the provided `uid`.

Retrieves the metadata stored for a given `uid` as a `psa_storage_info_t` structure.

**Parameters**

|     |               |                                                                                                     |
|-----|---------------|-----------------------------------------------------------------------------------------------------|
| in  | <i>uid</i>    | The <code>uid</code> value                                                                          |
| out | <i>p_info</i> | A pointer to the <a href="#">psa_storage_info_t</a> struct that will be populated with the metadata |

**Returns**

A status indicating the success/failure of the operation

**Return values**

|                                   |                                                                                                                                                                             |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                                                                                        |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided <code>uid</code> value was not found in the storage                                                                               |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (Fatal error)                                                                                                  |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one of the provided pointers( <code>p_info</code> ) is invalid, for example is <code>NULL</code> or references memory the caller cannot access |

Definition at line 42 of file `ps_filesystem_interface.c`.

Here is the call graph for this function:

**8.370.1.3 psa\_its\_remove()**

```

psa_status_t psa_its_remove (
 psa_storage_uid_t uid)

```

Remove the provided `uid` and its associated data from the storage.

Deletes the data from internal storage.

**Parameters**

|    |            |                            |
|----|------------|----------------------------|
| in | <i>uid</i> | The <code>uid</code> value |
|----|------------|----------------------------|

**Returns**

A status indicating the success/failure of the operation

**Return values**

|                                   |                                                                                                                    |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                               |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on) |

## Return values

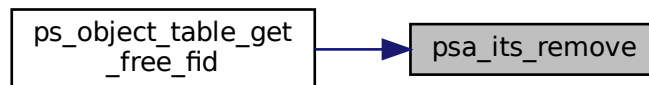
|                                  |                                                                                                         |
|----------------------------------|---------------------------------------------------------------------------------------------------------|
| <i>PSA_ERROR_DOES_NOT_EXIST</i>  | The operation failed because the provided uid value was not found in the storage                        |
| <i>PSA_ERROR_NOT_PERMITTED</i>   | The operation failed because the provided uid value was created with <i>PSA_STORAGE_FLAG_WRITE_ONCE</i> |
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the physical storage has failed (Fatal error)                              |

Definition at line 48 of file `ps_filesystem_interface.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.370.1.4 psa\_its\_set()

```

psa_status_t psa_its_set (
 psa_storage_uid_t uid,
 size_t data_length,
 void * p_data,
 psa_storage_create_flags_t create_flags)

```

Definition at line 17 of file `ps_filesystem_interface.c`.

Here is the call graph for this function:



## 8.370.2 Variable Documentation

### 8.370.2.1 p\_psa\_dest\_data

```
uint8_t* p_psa_dest_data
```

Definition at line 15 of file ps\_filesystem\_interface.c.

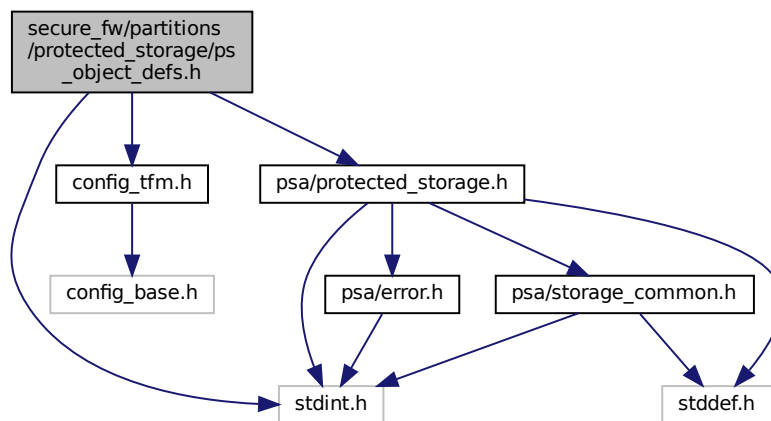
### 8.370.2.2 p\_psa\_src\_data

```
uint8_t* p_psa_src_data
```

Definition at line 14 of file ps\_filesystem\_interface.c.

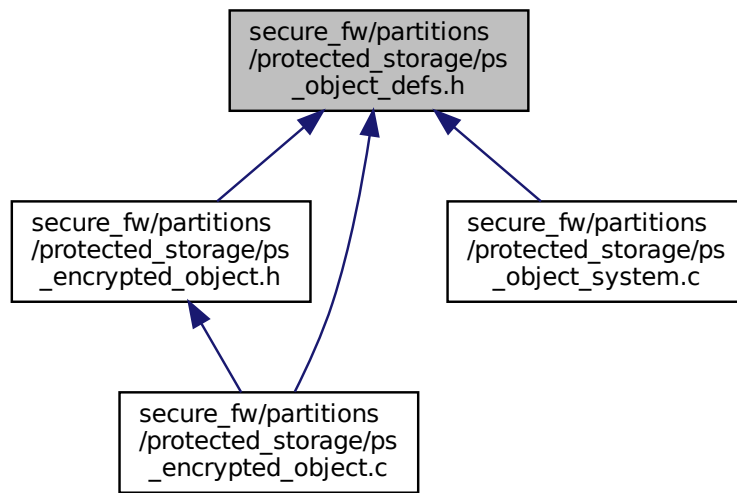
## 8.371 secure\_fw/partitions/protected\_storage/ps\_object\_defs.h File Reference

```
#include <stdint.h>
#include "config_tfm.h"
#include "psa/protected_storage.h"
Include dependency graph for ps_object_defs.h:
```





This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ps\\_object\\_info\\_t](#)  
*Object information.*
- struct [ps\\_obj\\_header\\_t](#)  
*Metadata attached as a header to object data before storage.*
- struct [ps\\_object\\_t](#)  
*The object to be written to the file system below. Made up of the object header and the object data.*

## Macros

- `#define PS_MAX_OBJECT_DATA_SIZE PS_MAX_ASSET_SIZE`
- `#define PS_OBJECT_BUF_SIZE PS_MAX_OBJECT_DATA_SIZE`
- `#define PS_OBJECT_HEADER_SIZE sizeof(struct ps_obj_header_t)`
- `#define PS_MAX_OBJECT_SIZE sizeof(struct ps_object_t)`
- `#define PS_MAX_NUM_OBJECTS (PS_NUM_ASSETS + 3)`  
*Specifies the maximum number of objects in the system, which is the number of defined assets, the object table and 2 temporary objects to store the temporary object table and temporary updated object.*

## 8.371.1 Macro Definition Documentation

### 8.371.1.1 PS\_MAX\_NUM\_OBJECTS

```
#define PS_MAX_NUM_OBJECTS (PS_NUM_ASSETS + 3)
```

Specifies the maximum number of objects in the system, which is the number of defined assets, the object table and 2 temporary objects to store the temporary object table and temporary updated object.

Definition at line 80 of file `ps_object_defs.h`.



## Functions

- `__STATIC_INLINE void ps_init_empty_object (psa_storage_uid_t uid, int32_t client_id, psa_storage_create_flags_t create_flags, uint32_t size, struct ps_object_t *obj)`  
*Initialize g\_ps\_object based on the input parameters and empty data.*
- `psa_status_t ps_system_prepare (void)`  
*Prepares the protected storage system for usage, populating internal structures. It identifies and validates the system metadata.*
- `psa_status_t ps_object_read (psa_storage_uid_t uid, int32_t client_id, uint32_t offset, uint32_t size, size_t *p_data_length)`  
*Gets the data of the object with the provided UID and client ID.*
- `psa_status_t ps_object_create (psa_storage_uid_t uid, int32_t client_id, psa_storage_create_flags_t create_flags, uint32_t size)`  
*Creates a new object with the provided UID and client ID.*
- `psa_status_t ps_object_write (psa_storage_uid_t uid, int32_t client_id, uint32_t offset, uint32_t size)`  
*Writes data into the object with the provided UID and client ID.*
- `psa_status_t ps_object_get_info (psa_storage_uid_t uid, int32_t client_id, struct psa_storage_info_t *info)`  
*Gets the asset information for the object with the provided UID and client ID.*
- `psa_status_t ps_object_delete (psa_storage_uid_t uid, int32_t client_id)`  
*Deletes the object with the provided UID and client ID.*
- `psa_status_t ps_system_wipe_all (void)`  
*Wipes the protected storage system and all object data.*

## 8.372.1 Macro Definition Documentation

### 8.372.1.1 PS\_OBJECT\_SIZE

```
#define PS_OBJECT_SIZE(
 max_size) (PS_OBJECT_HEADER_SIZE + (max_size))
```

Definition at line 30 of file ps\_object\_system.c.

### 8.372.1.2 PS\_OBJECT\_START\_POSITION

```
#define PS_OBJECT_START_POSITION 0
```

Definition at line 31 of file ps\_object\_system.c.

## 8.372.2 Enumeration Type Documentation

### 8.372.2.1 read\_type\_t

```
enum read_type_t
```

Enumerator

|                  |  |
|------------------|--|
| READ_HEADER_ONLY |  |
| READ_ALL_OBJECT  |  |

Definition at line 155 of file ps\_object\_system.c.

### 8.372.3 Function Documentation

#### 8.372.3.1 ps\_init\_empty\_object()

```
__STATIC_INLINE void ps_init_empty_object (
 psa_storage_uid_t uid,
 int32_t client_id,
 psa_storage_create_flags_t create_flags,
 uint32_t size,
 struct ps_object_t * obj)
```

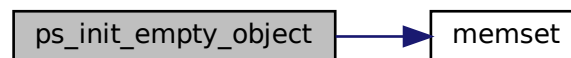
Initialize g\_ps\_object based on the input parameters and empty data.

##### Parameters

|     |                     |                                          |
|-----|---------------------|------------------------------------------|
| in  | <i>uid</i>          | Unique identifier for the data           |
| in  | <i>client_id</i>    | Identifier of the asset's owner (client) |
| in  | <i>create_flags</i> | Object create flags                      |
| in  | <i>size</i>         | Object size                              |
| out | <i>obj</i>          | Object to initialize                     |

Definition at line 49 of file ps\_object\_system.c.

Here is the call graph for this function:



#### 8.372.3.2 ps\_object\_create()

```
psa_status_t ps_object_create (
 psa_storage_uid_t uid,
 int32_t client_id,
 psa_storage_create_flags_t create_flags,
 uint32_t size)
```

Creates a new object with the provided UID and client ID.

##### Parameters

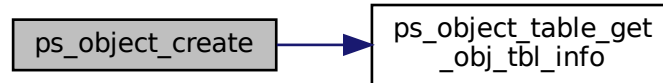
|    |                     |                                             |
|----|---------------------|---------------------------------------------|
| in | <i>uid</i>          | Unique identifier for the data              |
| in | <i>client_id</i>    | Identifier of the asset's owner (client)    |
| in | <i>create_flags</i> | Flags indicating the properties of the data |
| in | <i>size</i>         | Size of the contents of data in bytes       |

#### Returns

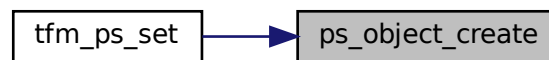
Returns error code specified in [psa\\_status\\_t](#)

Definition at line 378 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.372.3.3 ps\_object\_delete()

```
psa_status_t ps_object_delete (
 psa_storage_uid_t uid,
 int32_t client_id)
```

Deletes the object with the provided UID and client ID.

#### Parameters

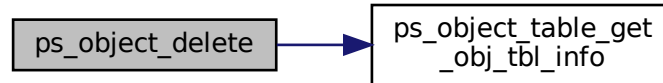
|    |                  |                                          |
|----|------------------|------------------------------------------|
| in | <i>uid</i>       | Unique identifier for the data           |
| in | <i>client_id</i> | Identifier of the asset's owner (client) |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 666 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.372.3.4 ps\_object\_get\_info()**

```

psa_status_t ps_object_get_info (
 psa_storage_uid_t uid,
 int32_t client_id,
 struct psa_storage_info_t * info)

```

Gets the asset information for the object with the provided UID and client ID.

**Parameters**

|     |                  |                                                                                                   |
|-----|------------------|---------------------------------------------------------------------------------------------------|
| in  | <i>uid</i>       | Unique identifier for the data                                                                    |
| in  | <i>client_id</i> | Identifier of the asset's owner (client)                                                          |
| out | <i>info</i>      | Pointer to the <a href="#">psa_storage_info_t</a> struct that will be populated with the metadata |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 608 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.372.3.5 ps\_object\_read()**

```

psa_status_t ps_object_read (
 psa_storage_uid_t uid,
 int32_t client_id,
 uint32_t offset,
 uint32_t size,
 size_t * p_data_length)

```

Gets the data of the object with the provided UID and client ID.

**Parameters**

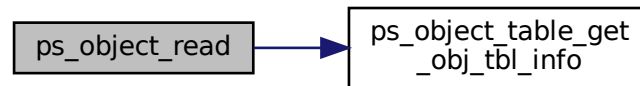
|     |                      |                                                                 |
|-----|----------------------|-----------------------------------------------------------------|
| in  | <i>uid</i>           | Unique identifier for the data                                  |
| in  | <i>client_id</i>     | Identifier of the asset's owner (client)                        |
| in  | <i>offset</i>        | Offset in the object at which to begin the read                 |
| in  | <i>size</i>          | Size of the contents of <code>data</code> in bytes              |
| out | <i>p_data_length</i> | On success, this will contain size of the data written to asset |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 305 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.372.3.6 ps\_object\_write()**

```

psa_status_t ps_object_write (
 psa_storage_uid_t uid,
 int32_t client_id,
 uint32_t offset,
 uint32_t size)

```

Writes data into the object with the provided UID and client ID.

**Parameters**

|    |                  |                                                    |
|----|------------------|----------------------------------------------------|
| in | <i>uid</i>       | Unique identifier for the data                     |
| in | <i>client_id</i> | Identifier of the asset's owner (client)           |
| in | <i>offset</i>    | Offset in the object at which to begin the write   |
| in | <i>size</i>      | Size of the contents of <code>data</code> in bytes |

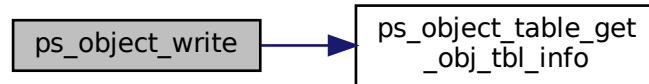


#### Returns

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 500 of file ps\_object\_system.c.

Here is the call graph for this function:



#### 8.372.3.7 ps\_system\_prepare()

```
psa_status_t ps_system_prepare (
 void)
```

Prepares the protected storage system for usage, populating internal structures. It identifies and validates the system metadata.

#### Returns

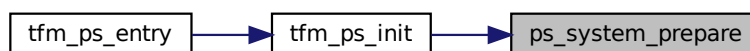
Returns error code specified in [psa\\_status\\_t](#)

Definition at line 275 of file ps\_object\_system.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.372.3.8 ps\_system\_wipe\_all()

```
psa_status_t ps_system_wipe_all (
 void)
```

Wipes the protected storage system and all object data.

**Returns**

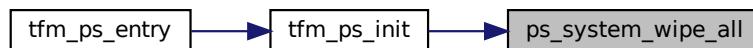
Returns error code specified in [psa\\_status\\_t](#)

Definition at line 750 of file `ps_object_system.c`.

Here is the call graph for this function:

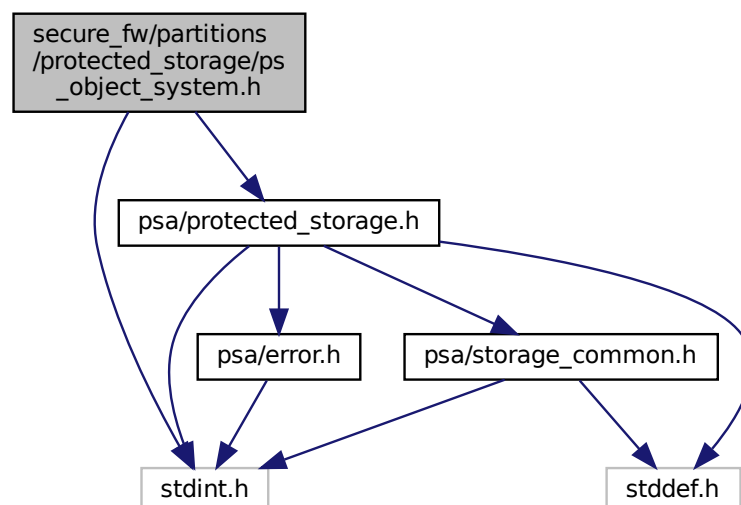


Here is the caller graph for this function:

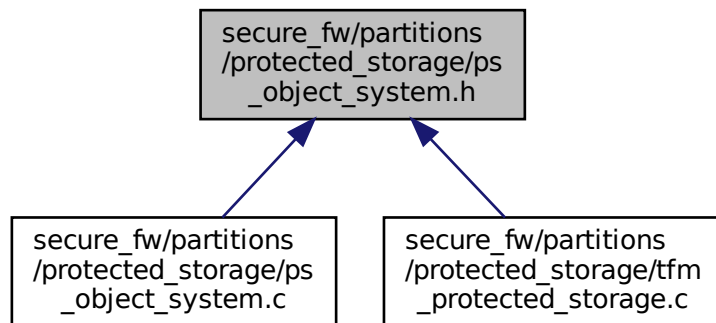


### 8.373 `secure_fw/partitions/protected_storage/ps_object_system.h` File Reference

```
#include <stdint.h>
#include "psa/protected_storage.h"
Include dependency graph for ps_object_system.h:
```



This graph shows which files directly or indirectly include this file:



## Functions

- [psa\\_status\\_t ps\\_system\\_prepare \(void\)](#)  
*Prepares the protected storage system for usage, populating internal structures. It identifies and validates the system metadata.*
- [psa\\_status\\_t ps\\_object\\_create \(psa\\_storage\\_uid\\_t uid, int32\\_t client\\_id, psa\\_storage\\_create\\_flags\\_t create\\_flags, uint32\\_t size\)](#)  
*Creates a new object with the provided UID and client ID.*
- [psa\\_status\\_t ps\\_object\\_read \(psa\\_storage\\_uid\\_t uid, int32\\_t client\\_id, uint32\\_t offset, uint32\\_t size, size\\_t \\*p\\_data\\_length\)](#)  
*Gets the data of the object with the provided UID and client ID.*
- [psa\\_status\\_t ps\\_object\\_write \(psa\\_storage\\_uid\\_t uid, int32\\_t client\\_id, uint32\\_t offset, uint32\\_t size\)](#)  
*Writes data into the object with the provided UID and client ID.*
- [psa\\_status\\_t ps\\_object\\_delete \(psa\\_storage\\_uid\\_t uid, int32\\_t client\\_id\)](#)  
*Deletes the object with the provided UID and client ID.*
- [psa\\_status\\_t ps\\_object\\_get\\_info \(psa\\_storage\\_uid\\_t uid, int32\\_t client\\_id, struct psa\\_storage\\_info\\_t \\*info\)](#)  
*Gets the asset information for the object with the provided UID and client ID.*
- [psa\\_status\\_t ps\\_system\\_wipe\\_all \(void\)](#)  
*Wipes the protected storage system and all object data.*

## 8.373.1 Function Documentation

### 8.373.1.1 ps\_object\_create()

```

psa_status_t ps_object_create (
 psa_storage_uid_t uid,
 int32_t client_id,
 psa_storage_create_flags_t create_flags,
 uint32_t size)

```

Creates a new object with the provided UID and client ID.

#### Parameters

|    |            |                                |
|----|------------|--------------------------------|
| in | <i>uid</i> | Unique identifier for the data |
|----|------------|--------------------------------|

**Parameters**

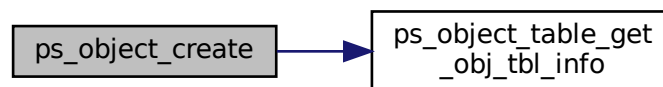
|    |                     |                                                    |
|----|---------------------|----------------------------------------------------|
| in | <i>client_id</i>    | Identifier of the asset's owner (client)           |
| in | <i>create_flags</i> | Flags indicating the properties of the data        |
| in | <i>size</i>         | Size of the contents of <code>data</code> in bytes |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 378 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.373.1.2 ps\_object\_delete()**

```

psa_status_t ps_object_delete (
 psa_storage_uid_t uid,
 int32_t client_id)

```

Deletes the object with the provided UID and client ID.

**Parameters**

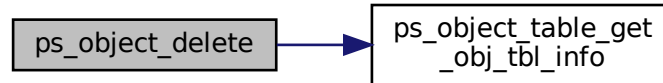
|    |                  |                                          |
|----|------------------|------------------------------------------|
| in | <i>uid</i>       | Unique identifier for the data           |
| in | <i>client_id</i> | Identifier of the asset's owner (client) |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 666 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.373.1.3 ps\_object\_get\_info()**

```

psa_status_t ps_object_get_info (
 psa_storage_uid_t uid,
 int32_t client_id,
 struct psa_storage_info_t * info)

```

Gets the asset information for the object with the provided UID and client ID.

**Parameters**

|     |                  |                                                                                                   |
|-----|------------------|---------------------------------------------------------------------------------------------------|
| in  | <i>uid</i>       | Unique identifier for the data                                                                    |
| in  | <i>client_id</i> | Identifier of the asset's owner (client)                                                          |
| out | <i>info</i>      | Pointer to the <a href="#">psa_storage_info_t</a> struct that will be populated with the metadata |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 608 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.373.1.4 ps\_object\_read()**

```

psa_status_t ps_object_read (
 psa_storage_uid_t uid,
 int32_t client_id,
 uint32_t offset,
 uint32_t size,
 size_t * p_data_length)

```

Gets the data of the object with the provided UID and client ID.

**Parameters**

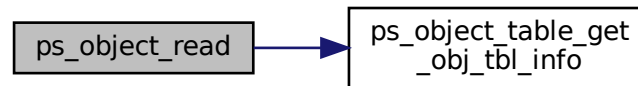
|     |                      |                                                                 |
|-----|----------------------|-----------------------------------------------------------------|
| in  | <i>uid</i>           | Unique identifier for the data                                  |
| in  | <i>client_id</i>     | Identifier of the asset's owner (client)                        |
| in  | <i>offset</i>        | Offset in the object at which to begin the read                 |
| in  | <i>size</i>          | Size of the contents of <code>data</code> in bytes              |
| out | <i>p_data_length</i> | On success, this will contain size of the data written to asset |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 305 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.373.1.5 ps\_object\_write()**

```

psa_status_t ps_object_write (
 psa_storage_uid_t uid,
 int32_t client_id,
 uint32_t offset,
 uint32_t size)

```

Writes data into the object with the provided UID and client ID.

**Parameters**

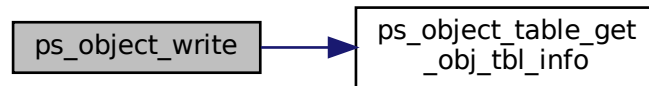
|    |                  |                                                    |
|----|------------------|----------------------------------------------------|
| in | <i>uid</i>       | Unique identifier for the data                     |
| in | <i>client_id</i> | Identifier of the asset's owner (client)           |
| in | <i>offset</i>    | Offset in the object at which to begin the write   |
| in | <i>size</i>      | Size of the contents of <code>data</code> in bytes |

**Returns**

Returns error code specified in [psa\\_status\\_t](#)

Definition at line 500 of file `ps_object_system.c`.

Here is the call graph for this function:

**8.373.1.6 ps\_system\_prepare()**

```
psa_status_t ps_system_prepare (
 void)
```

Prepares the protected storage system for usage, populating internal structures. It identifies and validates the system metadata.

**Returns**

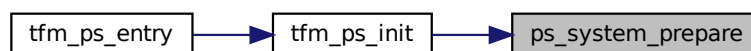
Returns error code specified in [psa\\_status\\_t](#)

Definition at line 275 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.373.1.7 ps\_system\_wipe\_all()**

```
psa_status_t ps_system_wipe_all (
 void)
```

Wipes the protected storage system and all object data.



**Returns**

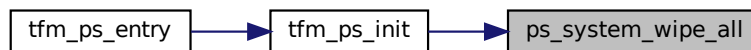
Returns error code specified in [psa\\_status\\_t](#)

Definition at line 750 of file `ps_object_system.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



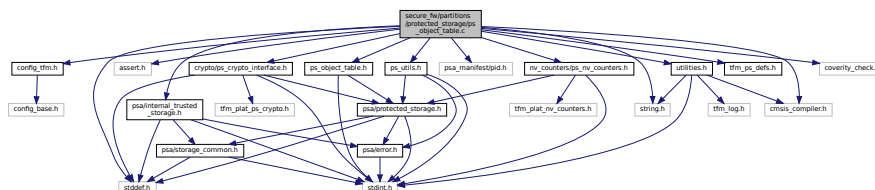
## 8.374 secure\_fw/partitions/protected\_storage/ps\_object\_table.c File Reference

```

#include "ps_object_table.h"
#include <assert.h>
#include <stddef.h>
#include <string.h>
#include "cmsis_compiler.h"
#include "config_tfm.h"
#include "crypto/ps_crypto_interface.h"
#include "psa_manifest/pid.h"
#include "nv_counters/ps_nv_counters.h"
#include "psa/internal_trusted_storage.h"
#include "ps_utils.h"
#include "tfm_ps_defs.h"
#include "utilities.h"
#include "coverity_check.h"

```

Include dependency graph for `ps_object_table.c`:



## Data Structures

- struct [ps\\_obj\\_table\\_entry\\_t](#)
- struct [ps\\_obj\\_table\\_t](#)  
*Object table structure.*
- struct [ps\\_obj\\_table\\_ctx\\_t](#)  
*Object table context structure.*
- struct [ps\\_obj\\_table\\_init\\_ctx\\_t](#)

## Macros

- #define [PS\\_FLASH\\_DEFAULT\\_VAL](#) 0xFFU
- #define [PS\\_OBJECT\\_SYSTEM\\_VERSION](#) 0x01  
*Current object system version.*
- #define [PS\\_OBJ\\_TABLE\\_ENTRIES](#) (PS\_NUM\_ASSETS + 1)
- #define [PS\\_OBJ\\_TABLE\\_IDX\\_0](#) 0
- #define [PS\\_OBJ\\_TABLE\\_IDX\\_1](#) 1
- #define [PS\\_NUM\\_OBJ\\_TABLES](#) 2
- #define [PS\\_TABLE\\_FS\\_ID](#)(idx) (idx + 1)  
*File ID to be used in order to store the object table in the file system.*
- #define [PS\\_OBJECT\\_FS\\_ID](#)(idx)  
*File ID to be used in order to store an object in the file system.*
- #define [PS\\_OBJECT\\_FS\\_ID\\_TO\\_IDX](#)(fid) ps\_object\_fs\_id\_to\_idx(fid)  
*Gets object index in the table based on the file ID.*
- #define [PS\\_OBJ\\_TABLE\\_SIZE](#) sizeof(struct [ps\\_obj\\_table\\_t](#))
- #define [PS\\_OBJECTS\\_TABLE\\_ENTRY\\_SIZE](#) sizeof(struct [ps\\_obj\\_table\\_entry\\_t](#))
- #define [PS\\_NON\\_AUTH\\_OBJ\\_TABLE\\_SIZE](#) sizeof(union [ps\\_crypto\\_t](#))
- #define [PS\\_OBJECT\\_TABLE\\_OBJECT\\_OFFSET](#) 0
- #define [PS\\_CRYPTO\\_ASSOCIATED\\_DATA](#)(crypto)
- #define [PS\\_CRYPTO\\_ASSOCIATED\\_DATA\\_LEN](#)
- #define [PS\\_INVALID\\_NVC\\_VALUE](#) 0

## Typedefs

- typedef char [OBJ\\_TABLE\\_NOT\\_FIT\\_IN\\_STATIC\\_OBJ\\_DATA\\_BUF](#)[(sizeof(struct [ps\\_obj\\_table\\_t](#))<=PS\_↵  
MAX\_ASSET\_SIZE) \*2 - 1]

## Enumerations

- enum [ps\\_obj\\_table\\_state](#) { [PS\\_OBJ\\_TABLE\\_VALID](#) = 0, [PS\\_OBJ\\_TABLE\\_INVALID](#), [PS\\_OBJ\\_TABLE\\_NVC\\_1\\_VALID](#), [PS\\_OBJ\\_TABLE\\_NVC\\_3\\_VALID](#) }

## Functions

- [\\_\\_STATIC\\_INLINE void ps\\_object\\_table\\_fs\\_read\\_table](#) (struct [ps\\_obj\\_table\\_init\\_ctx\\_t](#) \*init\_ctx)  
*Reads object table from persistent memory.*
- [\\_\\_STATIC\\_INLINE psa\\_status\\_t ps\\_object\\_table\\_fs\\_write\\_table](#) (struct [ps\\_obj\\_table\\_t](#) \*obj\_table)  
*Writes object table in persistent memory.*
- [\\_\\_STATIC\\_INLINE void ps\\_object\\_table\\_validate\\_version](#) (struct [ps\\_obj\\_table\\_init\\_ctx\\_t](#) \*init\_ctx)  
*Checks the validity of the table version.*
- [\\_\\_STATIC\\_INLINE psa\\_status\\_t ps\\_table\\_free\\_idx](#) (uint32\_t idx\_num, uint32\_t \*idx)  
*Gets free index in the table.*
- [psa\\_status\\_t ps\\_object\\_table\\_create](#) (void)  
*Creates object table.*

- [psa\\_status\\_t ps\\_object\\_table\\_init](#) (uint8\_t \*obj\_data)  
*Initializes object table.*
- [psa\\_status\\_t ps\\_object\\_table\\_obj\\_exist](#) (psa\_storage\_uid\_t uid, int32\_t client\_id)  
*Checks if there is an entry in the table for the provided UID and client ID pair.*
- [psa\\_status\\_t ps\\_object\\_table\\_get\\_free\\_fid](#) (uint32\_t fid\_num, uint32\_t \*p\_fid)  
*Gets a not in use file ID.*
- [psa\\_status\\_t ps\\_object\\_table\\_set\\_obj\\_tbl\\_info](#) (psa\_storage\_uid\_t uid, int32\_t client\_id, const struct ps\_obj\_table\_info\_t \*obj\_tbl\_info)  
*Sets object table information in the object table and stores it persistently, for the provided UID and client ID pair.*
- [psa\\_status\\_t ps\\_object\\_table\\_get\\_obj\\_tbl\\_info](#) (psa\_storage\_uid\_t uid, int32\_t client\_id, struct ps\_obj\_table\_info\_t \*obj\_tbl\_info)  
*Gets object table information from the object table for the provided UID and client ID pair.*
- [psa\\_status\\_t ps\\_object\\_table\\_delete\\_object](#) (psa\_storage\_uid\_t uid, int32\_t client\_id)  
*Deletes the table entry for the provided UID and client ID pair.*
- [psa\\_status\\_t ps\\_object\\_table\\_delete\\_old\\_table](#) (void)  
*Deletes old object table from the persistent area.*

## 8.374.1 Macro Definition Documentation

### 8.374.1.1 PS\_CRYPTO\_ASSOCIATED\_DATA

```
#define PS_CRYPTO_ASSOCIATED_DATA(
 crypto)
```

**Value:**

```
((uint8_t *)crypto + \
PS_NON_AUTH_OBJ_TABLE_SIZE)
```

Definition at line 169 of file ps\_object\_table.c.

### 8.374.1.2 PS\_CRYPTO\_ASSOCIATED\_DATA\_LEN

```
#define PS_CRYPTO_ASSOCIATED_DATA_LEN
```

**Value:**

```
(PS_OBJ_TABLE_SIZE - \
PS_NON_AUTH_OBJ_TABLE_SIZE)
```

Definition at line 186 of file ps\_object\_table.c.

### 8.374.1.3 PS\_FLASH\_DEFAULT\_VAL

```
#define PS_FLASH_DEFAULT_VAL 0xFFU
```

Definition at line 26 of file ps\_object\_table.c.

### 8.374.1.4 PS\_INVALID\_NVC\_VALUE

```
#define PS_INVALID_NVC_VALUE 0
```

Definition at line 214 of file ps\_object\_table.c.

### 8.374.1.5 PS\_NON\_AUTH\_OBJ\_TABLE\_SIZE

```
#define PS_NON_AUTH_OBJ_TABLE_SIZE sizeof(union ps_crypto_t)
```

Definition at line 163 of file ps\_object\_table.c.

**8.374.1.6 PS\_NUM\_OBJ\_TABLES**

```
#define PS_NUM_OBJ_TABLES 2
```

Definition at line 95 of file `ps_object_table.c`.

**8.374.1.7 PS\_OBJ\_TABLE\_ENTRIES**

```
#define PS_OBJ_TABLE_ENTRIES (PS_NUM_ASSETS + 1)
```

Definition at line 57 of file `ps_object_table.c`.

**8.374.1.8 PS\_OBJ\_TABLE\_IDX\_0**

```
#define PS_OBJ_TABLE_IDX_0 0
```

Definition at line 91 of file `ps_object_table.c`.

**8.374.1.9 PS\_OBJ\_TABLE\_IDX\_1**

```
#define PS_OBJ_TABLE_IDX_1 1
```

Definition at line 92 of file `ps_object_table.c`.

**8.374.1.10 PS\_OBJ\_TABLE\_SIZE**

```
#define PS_OBJ_TABLE_SIZE sizeof(struct ps_obj_table_t)
```

Definition at line 157 of file `ps_object_table.c`.

**8.374.1.11 PS\_OBJECT\_FS\_ID**

```
#define PS_OBJECT_FS_ID(
 idx)
```

**Value:**

```
((idx + 1) + \
 PS_TABLE_FS_ID(PS_OBJ_TABLE_IDX_1))
```

File ID to be used in order to store an object in the file system.

**Parameters**

|                 |                  |                                               |
|-----------------|------------------|-----------------------------------------------|
| <code>in</code> | <code>idx</code> | Object table index to convert into a file ID. |
|-----------------|------------------|-----------------------------------------------|

**Returns**

Returns file ID

Definition at line 120 of file `ps_object_table.c`.

**8.374.1.12 PS\_OBJECT\_FS\_ID\_TO\_IDX**

```
#define PS_OBJECT_FS_ID_TO_IDX(
 fid) ps_object_fs_id_to_idx(fid)
```

Gets object index in the table based on the file ID.

**Parameters**

|                 |                  |                                          |
|-----------------|------------------|------------------------------------------|
| <code>in</code> | <code>fid</code> | File ID of an object in the object table |
|-----------------|------------------|------------------------------------------|

**Returns**

Returns object table index

Definition at line 132 of file ps\_object\_table.c.

**8.374.1.13 PS\_OBJECT\_SYSTEM\_VERSION**

```
#define PS_OBJECT_SYSTEM_VERSION 0x01
```

Current object system version.

Definition at line 33 of file ps\_object\_table.c.

**8.374.1.14 PS\_OBJECT\_TABLE\_OBJECT\_OFFSET**

```
#define PS_OBJECT_TABLE_OBJECT_OFFSET 0
```

Definition at line 166 of file ps\_object\_table.c.

**8.374.1.15 PS\_OBJECTS\_TABLE\_ENTRY\_SIZE**

```
#define PS_OBJECTS_TABLE_ENTRY_SIZE sizeof(struct ps_obj_table_entry_t)
```

Definition at line 160 of file ps\_object\_table.c.

**8.374.1.16 PS\_TABLE\_FS\_ID**

```
#define PS_TABLE_FS_ID(
 idx) (idx + 1)
```

File ID to be used in order to store the object table in the file system.

**Parameters**

|           |            |                                        |
|-----------|------------|----------------------------------------|
| <i>in</i> | <i>idx</i> | Table index to convert into a file ID. |
|-----------|------------|----------------------------------------|

**Returns**

Returns file ID

Definition at line 108 of file ps\_object\_table.c.

**8.374.2 Typedef Documentation****8.374.2.1 OBJ\_TABLE\_NOT\_FIT\_IN\_STATIC\_OBJ\_DATA\_BUF**

```
typedef char OBJ_TABLE_NOT_FIT_IN_STATIC_OBJ_DATA_BUF[(sizeof(struct ps_obj_table_t) <= PS_MAX_ASSET_SIZE) *2 - 1]
```

Definition at line 204 of file ps\_object\_table.c.

**8.374.3 Enumeration Type Documentation****8.374.3.1 ps\_obj\_table\_state**

```
enum ps_obj_table_state
```

### Enumerator

|                          |                                      |
|--------------------------|--------------------------------------|
| PS_OBJ_TABLE_VALID       | Table content is valid               |
| PS_OBJ_TABLE_INVALID     | Table content is invalid             |
| PS_OBJ_TABLE_NVC_1_VALID | Table content valid with NVC 1 value |
| PS_OBJ_TABLE_NVC_3_VALID | Table content valid with NVC 3 value |

Definition at line 206 of file ps\_object\_table.c.

## 8.374.4 Function Documentation

### 8.374.4.1 ps\_object\_table\_create()

```
psa_status_t ps_object_table_create (
 void)
```

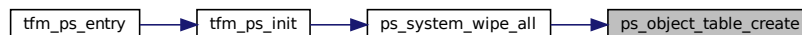
Creates object table.

#### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 856 of file ps\_object\_table.c.

Here is the caller graph for this function:



### 8.374.4.2 ps\_object\_table\_delete\_object()

```
psa_status_t ps_object_table_delete_object (
 psa_storage_uid_t uid,
 int32_t client_id)
```

Deletes the table entry for the provided UID and client ID pair.

#### Parameters

|    |                  |                                          |
|----|------------------|------------------------------------------|
| in | <i>uid</i>       | Identifier for the data.                 |
| in | <i>client_id</i> | Identifier of the asset's owner (client) |

#### Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1087 of file ps\_object\_table.c.

### 8.374.4.3 ps\_object\_table\_delete\_old\_table()

```
psa_status_t ps_object_table_delete_old_table (
 void)
```

Deletes old object table from the persistent area.

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1123 of file ps\_object\_table.c.

**8.374.4.4 ps\_object\_table\_fs\_read\_table()**

```
__STATIC_INLINE void ps_object_table_fs_read_table (
 struct ps_obj_table_init_ctx_t * init_ctx)
```

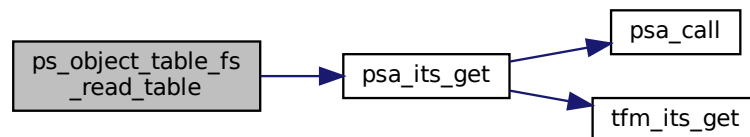
Reads object table from persistent memory.

**Parameters**

|     |                 |                                          |
|-----|-----------------|------------------------------------------|
| out | <i>init_ctx</i> | Pointer to the init object table context |
|-----|-----------------|------------------------------------------|

Definition at line 244 of file ps\_object\_table.c.

Here is the call graph for this function:

**8.374.4.5 ps\_object\_table\_fs\_write\_table()**

```
__STATIC_INLINE psa_status_t ps_object_table_fs_write_table (
 struct ps_obj_table_t * obj_table)
```

Writes object table in persistent memory.

**Parameters**

|         |                  |                                                        |
|---------|------------------|--------------------------------------------------------|
| in, out | <i>obj_table</i> | Pointer to the object table to generate authentication |
|---------|------------------|--------------------------------------------------------|

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 281 of file ps\_object\_table.c.

**8.374.4.6 ps\_object\_table\_get\_free\_fid()**

```
psa_status_t ps_object_table_get_free_fid (
 uint32_t fid_num,
 uint32_t * p_fid)
```

Gets a not in use file ID.

**Parameters**

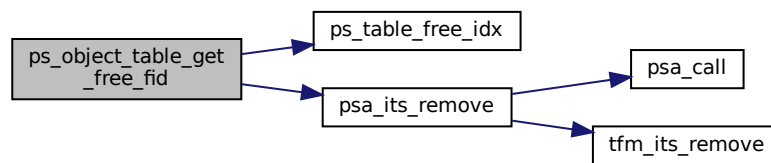
|     |                |                                                                                                                                                                               |
|-----|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>fid_num</i> | Amount of file IDs that the function will check are free before returning one. 0 is an invalid input and will error. Note that this function will only ever return 1 file ID. |
| out | <i>p_fid</i>   | Pointer to the location to store the file ID                                                                                                                                  |

**Returns**

Returns PSA\_SUCCESS if the fid is valid and fid\_num - 1 entries are still free in the table. Otherwise, it returns an error code as specified in [psa\\_status\\_t](#)

Definition at line 948 of file ps\_object\_table.c.

Here is the call graph for this function:

**8.374.4.7 ps\_object\_table\_get\_obj\_tbl\_info()**

```

psa_status_t ps_object_table_get_obj_tbl_info (
 psa_storage_uid_t uid,
 int32_t client_id,
 struct ps_obj_table_info_t * obj_tbl_info)

```

Gets object table information from the object table for the provided UID and client ID pair.

**Parameters**

|     |                     |                                                           |
|-----|---------------------|-----------------------------------------------------------|
| in  | <i>uid</i>          | Identifier for the data.                                  |
| in  | <i>client_id</i>    | Identifier of the asset's owner (client)                  |
| out | <i>obj_tbl_info</i> | Pointer to the location to store object table information |

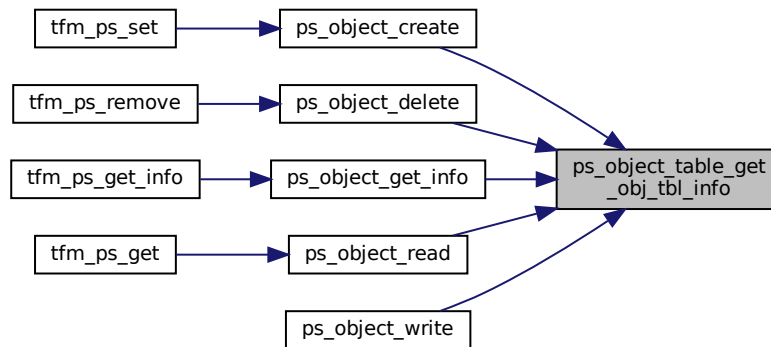


**Returns**

Returns PSA\_SUCCESS if the object exists. Otherwise, it returns PSA\_ERROR\_DOES\_NOT\_EXIST.

Definition at line 1059 of file ps\_object\_table.c.

Here is the caller graph for this function:

**8.374.4.8 ps\_object\_table\_init()**

```

psa_status_t ps_object_table_init (
 uint8_t * obj_data)

```

Initializes object table.

**Parameters**

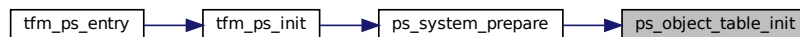
|                |                 |                                                                                                                 |
|----------------|-----------------|-----------------------------------------------------------------------------------------------------------------|
| <i>in, out</i> | <i>obj_data</i> | Pointer to the static object data allocated in order to reuse that memory to allocate a temporary object table. |
|----------------|-----------------|-----------------------------------------------------------------------------------------------------------------|

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 884 of file ps\_object\_table.c.

Here is the caller graph for this function:

**8.374.4.9 ps\_object\_table\_obj\_exist()**

```

psa_status_t ps_object_table_obj_exist (
 psa_storage_uid_t uid,
 int32_t client_id)

```

Checks if there is an entry in the table for the provided UID and client ID pair.

**Parameters**

|    |                  |                                          |
|----|------------------|------------------------------------------|
| in | <i>uid</i>       | Identifier for the data                  |
| in | <i>client_id</i> | Identifier of the asset's owner (client) |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

**Return values**

|                                 |                                          |
|---------------------------------|------------------------------------------|
| <i>PSA_SUCCESS</i>              | If there is a table entry for the object |
| <i>PSA_ERROR_DOES_NOT_EXIST</i> | If no table entry exists for the object  |

Definition at line 940 of file `ps_object_table.c`.

**8.374.4.10 ps\_object\_table\_set\_obj\_tbl\_info()**

```
psa_status_t ps_object_table_set_obj_tbl_info (
 psa_storage_uid_t uid,
 int32_t client_id,
 const struct ps_obj_table_info_t * obj_tbl_info)
```

Sets object table information in the object table and stores it persistently, for the provided UID and client ID pair.

**Parameters**

|    |                     |                                                                                               |
|----|---------------------|-----------------------------------------------------------------------------------------------|
| in | <i>uid</i>          | Identifier for the data.                                                                      |
| in | <i>client_id</i>    | Identifier of the asset's owner (client)                                                      |
| in | <i>obj_tbl_info</i> | Pointer to the location to store object table information <a href="#">ps_obj_table_info_t</a> |

**Note**

A call to this function results in writing the table to the file system.

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1000 of file `ps_object_table.c`.

**8.374.4.11 ps\_object\_table\_validate\_version()**

```
__STATIC_INLINE void ps_object_table_validate_version (
 struct ps_obj_table_init_ctx_t * init_ctx)
```

Checks the validity of the table version.

**Parameters**

|         |                 |                                          |
|---------|-----------------|------------------------------------------|
| in, out | <i>init_ctx</i> | Pointer to the init object table context |
|---------|-----------------|------------------------------------------|

Definition at line 644 of file `ps_object_table.c`.

**8.374.4.12 ps\_table\_free\_idx()**

```
__STATIC_INLINE psa_status_t ps_table_free_idx (
 uint32_t idx_num,
 uint32_t * idx)
```

Gets free index in the table.

**Parameters**

|     |                |                                                                                                                                                                           |
|-----|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>idx_num</i> | The number of indices required to be free before one can be allocated. Primarily used to prevent index exhaustion. Note that this function will only ever return 1 index. |
| out | <i>idx</i>     | Pointer to store the free index                                                                                                                                           |

**Note**

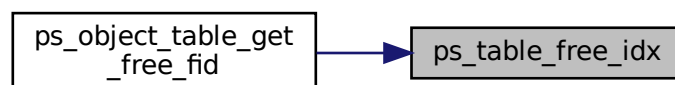
The table is dimensioned to fit PS\_NUM\_ASSETS + 1

**Returns**

Returns PSA\_SUCCESS and a table index if *idx\_num* free indices are available. Otherwise, it returns PSA\_ERROR\_INSUFFICIENT\_STORAGE.

Definition at line 817 of file ps\_object\_table.c.

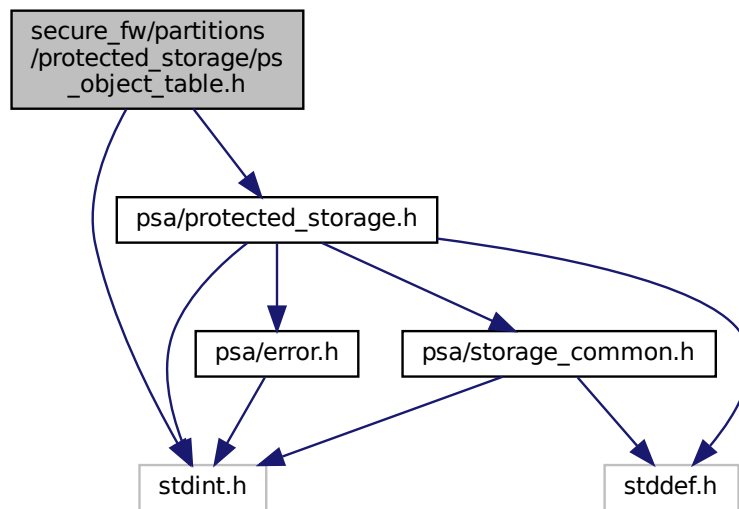
Here is the caller graph for this function:



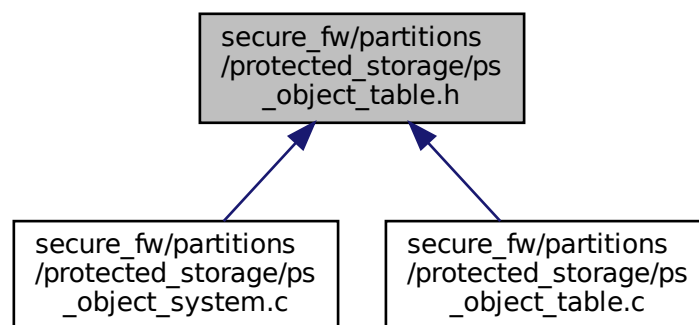
## 8.375 secure\_fw/partitions/protected\_storage/ps\_object\_table.h File Reference

```
#include <stdint.h>
#include "psa/protected_storage.h"
```

Include dependency graph for `ps_object_table.h`:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `ps_obj_table_info_t`  
*Object table information structure.*

## Functions

- `psa_status_t ps_object_table_create (void)`  
*Creates object table.*
- `psa_status_t ps_object_table_init (uint8_t *obj_data)`  
*Initializes object table.*

- [psa\\_status\\_t ps\\_object\\_table\\_obj\\_exist](#) ([psa\\_storage\\_uid\\_t](#) uid, [int32\\_t](#) client\_id)  
*Checks if there is an entry in the table for the provided UID and client ID pair.*
- [psa\\_status\\_t ps\\_object\\_table\\_get\\_free\\_fid](#) ([uint32\\_t](#) fid\_num, [uint32\\_t](#) \*p\_fid)  
*Gets a not in use file ID.*
- [psa\\_status\\_t ps\\_object\\_table\\_set\\_obj\\_tbl\\_info](#) ([psa\\_storage\\_uid\\_t](#) uid, [int32\\_t](#) client\_id, const struct [ps\\_obj\\_table\\_info\\_t](#) \*obj\_tbl\_info)  
*Sets object table information in the object table and stores it persistently, for the provided UID and client ID pair.*
- [psa\\_status\\_t ps\\_object\\_table\\_get\\_obj\\_tbl\\_info](#) ([psa\\_storage\\_uid\\_t](#) uid, [int32\\_t](#) client\_id, struct [ps\\_obj\\_table\\_info\\_t](#) \*obj\_tbl\_info)  
*Gets object table information from the object table for the provided UID and client ID pair.*
- [psa\\_status\\_t ps\\_object\\_table\\_delete\\_object](#) ([psa\\_storage\\_uid\\_t](#) uid, [int32\\_t](#) client\_id)  
*Deletes the table entry for the provided UID and client ID pair.*
- [psa\\_status\\_t ps\\_object\\_table\\_delete\\_old\\_table](#) (void)  
*Deletes old object table from the persistent area.*

## 8.375.1 Function Documentation

### 8.375.1.1 ps\_object\_table\_create()

```
psa_status_t ps_object_table_create (
 void)
```

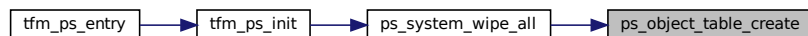
Creates object table.

Returns

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 856 of file [ps\\_object\\_table.c](#).

Here is the caller graph for this function:



### 8.375.1.2 ps\_object\_table\_delete\_object()

```
psa_status_t ps_object_table_delete_object (
 psa_storage_uid_t uid,
 int32_t client_id)
```

Deletes the table entry for the provided UID and client ID pair.

Parameters

|    |                  |                                          |
|----|------------------|------------------------------------------|
| in | <i>uid</i>       | Identifier for the data.                 |
| in | <i>client_id</i> | Identifier of the asset's owner (client) |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1087 of file `ps_object_table.c`.

**8.375.1.3 ps\_object\_table\_delete\_old\_table()**

```
psa_status_t ps_object_table_delete_old_table (
 void)
```

Deletes old object table from the persistent area.

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1123 of file `ps_object_table.c`.

**8.375.1.4 ps\_object\_table\_get\_free\_fid()**

```
psa_status_t ps_object_table_get_free_fid (
 uint32_t fid_num,
 uint32_t * p_fid)
```

Gets a not in use file ID.

**Parameters**

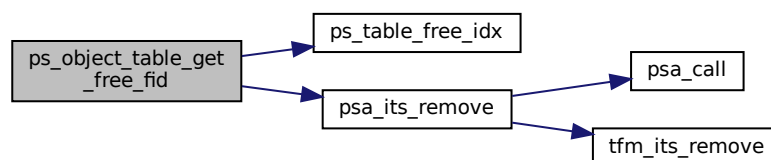
|     |                |                                                                                                                                                                               |
|-----|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>fid_num</i> | Amount of file IDs that the function will check are free before returning one. 0 is an invalid input and will error. Note that this function will only ever return 1 file ID. |
| out | <i>p_fid</i>   | Pointer to the location to store the file ID                                                                                                                                  |

**Returns**

Returns PSA\_SUCCESS if the fid is valid and fid\_num - 1 entries are still free in the table. Otherwise, it returns an error code as specified in [psa\\_status\\_t](#)

Definition at line 948 of file `ps_object_table.c`.

Here is the call graph for this function:

**8.375.1.5 ps\_object\_table\_get\_obj\_tbl\_info()**

```
psa_status_t ps_object_table_get_obj_tbl_info (
 psa_storage_uid_t uid,
 int32_t client_id,
 struct ps_obj_table_info_t * obj_tbl_info)
```

Gets object table information from the object table for the provided UID and client ID pair.

**Parameters**

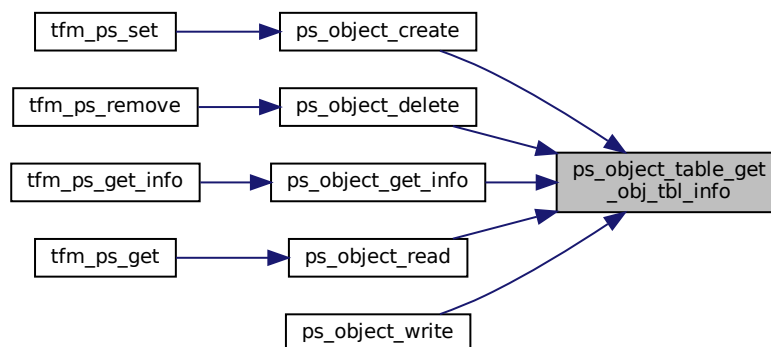
|     |                     |                                                           |
|-----|---------------------|-----------------------------------------------------------|
| in  | <i>uid</i>          | Identifier for the data.                                  |
| in  | <i>client_id</i>    | Identifier of the asset's owner (client)                  |
| out | <i>obj_tbl_info</i> | Pointer to the location to store object table information |

**Returns**

Returns PSA\_SUCCESS if the object exists. Otherwise, it returns PSA\_ERROR\_DOES\_NOT\_EXIST.

Definition at line 1059 of file ps\_object\_table.c.

Here is the caller graph for this function:

**8.375.1.6 ps\_object\_table\_init()**

```

psa_status_t ps_object_table_init (
 uint8_t * obj_data)

```

Initializes object table.

**Parameters**

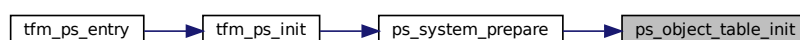
|         |                 |                                                                                                                  |
|---------|-----------------|------------------------------------------------------------------------------------------------------------------|
| in, out | <i>obj_data</i> | Pointer to the static object data allocated in other to reuse that memory to allocated a temporary object table. |
|---------|-----------------|------------------------------------------------------------------------------------------------------------------|

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 884 of file ps\_object\_table.c.

Here is the caller graph for this function:





**8.375.1.7 ps\_object\_table\_obj\_exist()**

```
psa_status_t ps_object_table_obj_exist (
 psa_storage_uid_t uid,
 int32_t client_id)
```

Checks if there is an entry in the table for the provided UID and client ID pair.

**Parameters**

|    |                  |                                          |
|----|------------------|------------------------------------------|
| in | <i>uid</i>       | Identifier for the data                  |
| in | <i>client_id</i> | Identifier of the asset's owner (client) |

**Returns**

Returns error code as specified in [psa\\_status\\_t](#)

**Return values**

|                                 |                                          |
|---------------------------------|------------------------------------------|
| <i>PSA_SUCCESS</i>              | If there is a table entry for the object |
| <i>PSA_ERROR_DOES_NOT_EXIST</i> | If no table entry exists for the object  |

Definition at line 940 of file ps\_object\_table.c.

**8.375.1.8 ps\_object\_table\_set\_obj\_tbl\_info()**

```
psa_status_t ps_object_table_set_obj_tbl_info (
 psa_storage_uid_t uid,
 int32_t client_id,
 const struct ps_obj_table_info_t * obj_tbl_info)
```

Sets object table information in the object table and stores it persistently, for the provided UID and client ID pair.

**Parameters**

|    |                     |                                                                                               |
|----|---------------------|-----------------------------------------------------------------------------------------------|
| in | <i>uid</i>          | Identifier for the data.                                                                      |
| in | <i>client_id</i>    | Identifier of the asset's owner (client)                                                      |
| in | <i>obj_tbl_info</i> | Pointer to the location to store object table information <a href="#">ps_obj_table_info_t</a> |

**Note**

A call to this function results in writing the table to the file system.

**Returns**

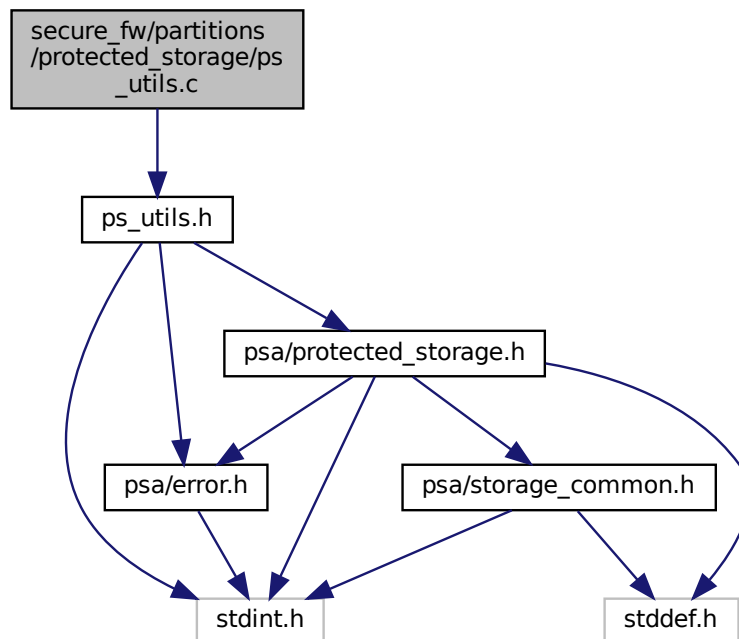
Returns error code as specified in [psa\\_status\\_t](#)

Definition at line 1000 of file ps\_object\_table.c.

**8.376 secure\_fw/partitions/protected\_storage/ps\_utils.c File Reference**

```
#include "ps_utils.h"
```

Include dependency graph for `ps_utils.c`:



## Functions

- [`psa\_status\_t ps\_utils\_check\_contained\_in`](#) (`uint32_t superset_size`, `uint32_t subset_offset`, `uint32_t subset_size`)  
Checks if a subset region is fully contained within a superset region.

## 8.376.1 Function Documentation

### 8.376.1.1 `ps_utils_check_contained_in()`

```

psa_status_t ps_utils_check_contained_in (
 uint32_t superset_size,
 uint32_t subset_offset,
 uint32_t subset_size)

```

Checks if a subset region is fully contained within a superset region.

#### Parameters

|    |                      |                                                                |
|----|----------------------|----------------------------------------------------------------|
| in | <i>superset_size</i> | Size of superset region                                        |
| in | <i>subset_offset</i> | Offset of start of subset region from start of superset region |
| in | <i>subset_size</i>   | Size of subset region                                          |

#### Returns

- Returns error code as specified in [`psa\_status\_t`](#)

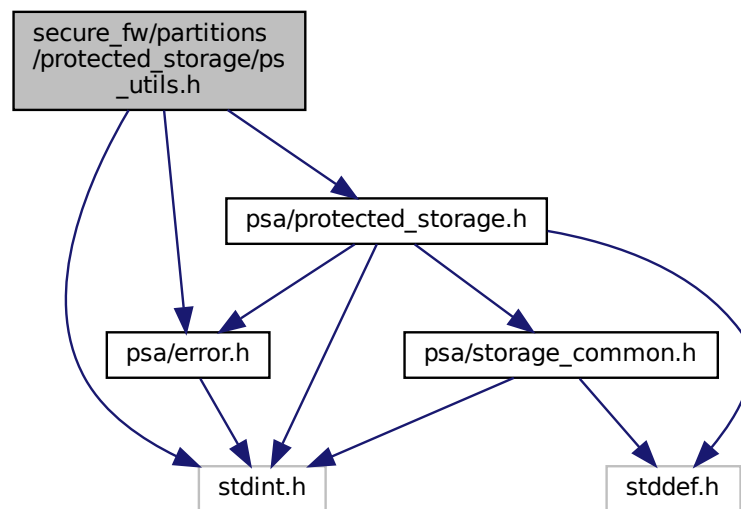
## Return values

|                                   |                                                                                                                                                                      |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The subset is contained within the superset                                                                                                                          |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The subset offset is greater than the size of the superset or when the subset offset is valid, but the subset offset + size is greater than the size of the superset |

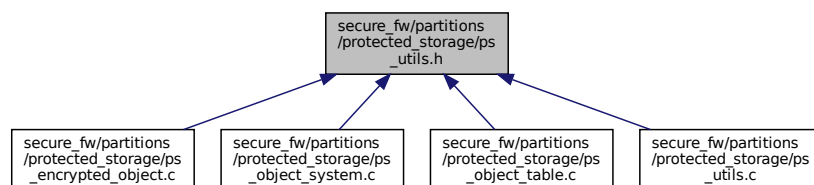
Definition at line 10 of file ps\_utils.c.

## 8.377 secure\_fw/partitions/protected\_storage/ps\_utils.h File Reference

```
#include <stdint.h>
#include "psa/error.h"
#include "psa/protected_storage.h"
Include dependency graph for ps_utils.h:
```



This graph shows which files directly or indirectly include this file:



## Macros

- `#define PS_INVALID_FID 0`
- `#define PS_DEFAULT_EMPTY_BUFF_VAL 0`

- `#define PS_UTILS_BOUND_CHECK(err_msg, data_size, data_buf_size) typedef char err_msg[(data_size <= data_buf_size)*2 - 1]`  
Macro to check, at compilation time, if data fits in data buffer.
- `#define PS_UTILS_MIN(x, y) (((x) < (y)) ? (x) : (y))`  
Evaluates to the minimum of the two parameters.

## Functions

- `psa_status_t ps_utils_check_contained_in (uint32_t superset_size, uint32_t subset_offset, uint32_t subset_size)`  
Checks if a subset region is fully contained within a superset region.

## 8.377.1 Macro Definition Documentation

### 8.377.1.1 PS\_DEFAULT\_EMPTY\_BUFF\_VAL

`#define PS_DEFAULT_EMPTY_BUFF_VAL 0`  
Definition at line 21 of file `ps_utils.h`.

### 8.377.1.2 PS\_INVALID\_FID

`#define PS_INVALID_FID 0`  
Definition at line 20 of file `ps_utils.h`.

### 8.377.1.3 PS\_UTILS\_BOUND\_CHECK

```
#define PS_UTILS_BOUND_CHECK(
 err_msg,
 data_size,
 data_buf_size) typedef char err_msg[(data_size <= data_buf_size)*2 - 1]
```

Macro to check, at compilation time, if data fits in data buffer.

#### Parameters

|    |                            |                                                                                   |
|----|----------------------------|-----------------------------------------------------------------------------------|
| in | <code>err_msg</code>       | Error message which will be displayed in first instance if the error is triggered |
| in | <code>data_size</code>     | Data size to check if it fits                                                     |
| in | <code>data_buf_size</code> | Size of the data buffer                                                           |

#### Returns

Triggers a compilation error if `data_size` is bigger than `data_buf_size`. The compilation error should be "... error: 'err\_msg' declared as an array with a negative size"

Definition at line 35 of file `ps_utils.h`.

### 8.377.1.4 PS\_UTILS\_MIN

```
#define PS_UTILS_MIN(
 x,
 y) (((x) < (y)) ? (x) : (y))
```

Evaluates to the minimum of the two parameters.

Definition at line 41 of file `ps_utils.h`.

## 8.377.2 Function Documentation

### 8.377.2.1 ps\_utils\_check\_contained\_in()

```
psa_status_t ps_utils_check_contained_in (
 uint32_t superset_size,
 uint32_t subset_offset,
 uint32_t subset_size)
```

Checks if a subset region is fully contained within a superset region.

#### Parameters

|    |                      |                                                                |
|----|----------------------|----------------------------------------------------------------|
| in | <i>superset_size</i> | Size of superset region                                        |
| in | <i>subset_offset</i> | Offset of start of subset region from start of superset region |
| in | <i>subset_size</i>   | Size of subset region                                          |

#### Returns

Returns error code as specified in [psa\\_status\\_t](#)

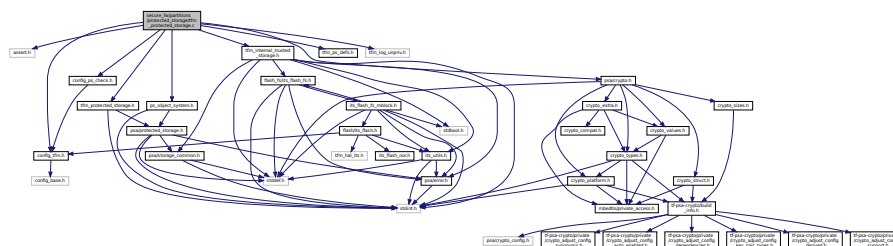
#### Return values

|                                   |                                                                                                                                                                      |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The subset is contained within the superset                                                                                                                          |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The subset offset is greater than the size of the superset or when the subset offset is valid, but the subset offset + size is greater than the size of the superset |

Definition at line 10 of file ps\_utils.c.

## 8.378 secure\_fw/partitions/protected\_storage/tfm\_protected\_storage.c File Reference

```
#include <assert.h>
#include "config_tfm.h"
#include "config_ps_check.h"
#include "tfm_protected_storage.h"
#include "ps_object_system.h"
#include "tfm_ps_defs.h"
#include "tfm_internal_trusted_storage.h"
#include "tfm_log_unpriv.h"
#include "psa/crypto.h"
Include dependency graph for tfm_protected_storage.c:
```



## Functions

- [psa\\_status\\_t tfm\\_ps\\_init](#) (void)  
*Initializes the protected storage system.*
- [psa\\_status\\_t tfm\\_ps\\_set](#) (int32\_t client\_id, [psa\\_storage\\_uid\\_t](#) uid, uint32\_t data\_length, [psa\\_storage\\_create\\_flags\\_t](#) create\_flags)  
*Creates a new or modifies an existing asset.*
- [psa\\_status\\_t tfm\\_ps\\_get](#) (int32\_t client\_id, [psa\\_storage\\_uid\\_t](#) uid, uint32\_t data\_offset, uint32\_t data\_size, size\_t \*p\_data\_length)  
*Gets the asset data for the provided uid.*
- [psa\\_status\\_t tfm\\_ps\\_get\\_info](#) (int32\_t client\_id, [psa\\_storage\\_uid\\_t](#) uid, struct [psa\\_storage\\_info\\_t](#) \*p\_info)  
*Gets the metadata for the provided uid.*
- [psa\\_status\\_t tfm\\_ps\\_remove](#) (int32\_t client\_id, [psa\\_storage\\_uid\\_t](#) uid)  
*Removes the provided uid and its associated data from storage.*
- uint32\_t [tfm\\_ps\\_get\\_support](#) (void)  
*Gets a bitmask with flags set for all of the optional features supported by the implementation.*

### 8.378.1 Function Documentation

#### 8.378.1.1 tfm\_ps\_get()

```
psa_status_t tfm_ps_get (
 int32_t client_id,
 psa_storage_uid_t uid,
 uint32_t data_offset,
 uint32_t data_size,
 size_t * p_data_length)
```

Gets the asset data for the provided uid.

##### Parameters

|     |                      |                                                                                               |
|-----|----------------------|-----------------------------------------------------------------------------------------------|
| in  | <i>client_id</i>     | Identifier of the asset's owner (client)                                                      |
| in  | <i>uid</i>           | Unique identifier for the data                                                                |
| in  | <i>data_offset</i>   | The offset within the data associated with the <code>uid</code> to start retrieving data      |
| in  | <i>data_size</i>     | The amount of data to read (and the minimum allocated size of the <code>p_data</code> buffer) |
| out | <i>p_data_length</i> | The pointer to the size of the data retrieved upon success.                                   |

##### Returns

A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

##### Return values

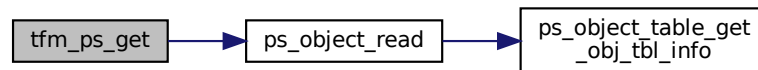
|                                   |                                                                                                   |
|-----------------------------------|---------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                              |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, etc.) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                  |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (fatal error)                        |
| <i>PSA_ERROR_GENERIC_ERROR</i>    | The operation failed because of an unspecified internal failure                                   |
| <i>PSA_ERROR_DATA_CORRUPT</i>     | The operation failed because the data associated with the UID was corrupt                         |

## Return values

|                                          |                                                                                     |
|------------------------------------------|-------------------------------------------------------------------------------------|
| <code>PSA_ERROR_INVALID_SIGNATURE</code> | The operation failed because the data associated with the UID failed authentication |
|------------------------------------------|-------------------------------------------------------------------------------------|

Definition at line 92 of file `tfm_protected_storage.c`.

Here is the call graph for this function:



## 8.378.1.2 tfm\_ps\_get\_info()

```

psa_status_t tfm_ps_get_info (
 int32_t client_id,
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Gets the metadata for the provided uid.

## Parameters

|     |                        |                                                                                                  |
|-----|------------------------|--------------------------------------------------------------------------------------------------|
| in  | <code>client_id</code> | Identifier of the asset's owner (client)                                                         |
| in  | <code>uid</code>       | Unique identifier for the data                                                                   |
| out | <code>p_info</code>    | A pointer to the <code>psa_storage_info_t</code> struct that will be populated with the metadata |

## Returns

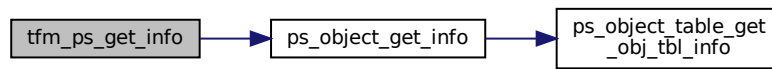
A status indicating the success/failure of the operation as specified in `psa_status_t`

## Return values

|                                          |                                                                                                   |
|------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>PSA_SUCCESS</code>                 | The operation completed successfully                                                              |
| <code>PSA_ERROR_INVALID_ARGUMENT</code>  | The operation failed because one or more of the given arguments were invalid (null pointer, etc.) |
| <code>PSA_ERROR_DOES_NOT_EXIST</code>    | The operation failed because the provided uid value was not found in the storage                  |
| <code>PSA_ERROR_STORAGE_FAILURE</code>   | The operation failed because the physical storage has failed (fatal error)                        |
| <code>PSA_ERROR_GENERIC_ERROR</code>     | The operation failed because of an unspecified internal failure                                   |
| <code>PSA_ERROR_DATA_CORRUPT</code>      | The operation failed because the data associated with the UID was corrupt                         |
| <code>PSA_ERROR_INVALID_SIGNATURE</code> | The operation failed because the data associated with the UID failed authentication               |

Definition at line 110 of file `tfm_protected_storage.c`.

Here is the call graph for this function:



### 8.378.1.3 tfm\_ps\_get\_support()

```
uint32_t tfm_ps_get_support (
 void)
```

Gets a bitmask with flags set for all of the optional features supported by the implementation.

#### Returns

Bitmask value which contains all the bits set for all the optional features supported by the implementation

Definition at line 147 of file `tfm_protected_storage.c`.

### 8.378.1.4 tfm\_ps\_init()

```
psa_status_t tfm_ps_init (
 void)
```

Initializes the protected storage system.

#### Returns

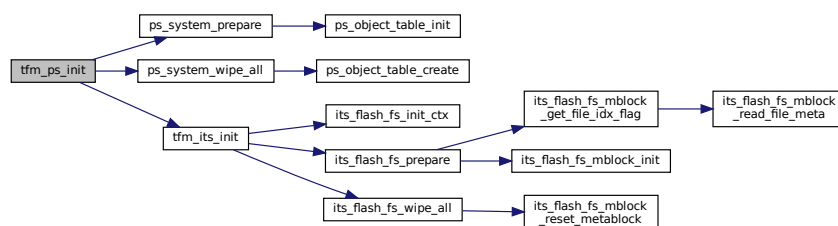
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

#### Return values

|                                  |                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>               | The operation completed successfully                                                    |
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the storage system initialization has failed (fatal error) |
| <i>PSA_ERROR_GENERIC_ERROR</i>   | The operation failed because of an unspecified internal failure                         |

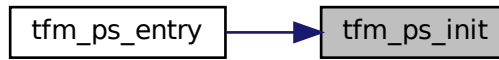
Definition at line 25 of file `tfm_protected_storage.c`.

Here is the call graph for this function:





Here is the caller graph for this function:



### 8.378.1.5 tfm\_ps\_remove()

```

psa_status_t tfm_ps_remove (
 int32_t client_id,
 psa_storage_uid_t uid)

```

Removes the provided uid and its associated data from storage.

#### Parameters

|    |                  |                                              |
|----|------------------|----------------------------------------------|
| in | <i>client_id</i> | Identifier of the asset's owner (client)     |
| in | <i>uid</i>       | Unique identifier for the data to be removed |

#### Returns

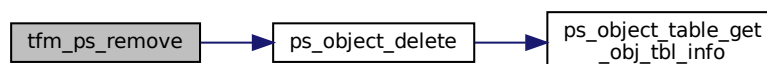
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

#### Return values

|                                   |                                                                                                               |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                          |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, etc.)             |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                              |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code> |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (fatal error)                                    |
| <i>PSA_ERROR_GENERIC_ERROR</i>    | The operation failed because of an unspecified internal failure                                               |

Definition at line 124 of file `tfm_protected_storage.c`.

Here is the call graph for this function:



### 8.378.1.6 tfm\_ps\_set()

```
psa_status_t tfm_ps_set (
 int32_t client_id,
 psa_storage_uid_t uid,
 uint32_t data_length,
 psa_storage_create_flags_t create_flags)
```

Creates a new or modifies an existing asset.

#### Parameters

|    |                     |                                                      |
|----|---------------------|------------------------------------------------------|
| in | <i>client_id</i>    | Identifier of the asset's owner (client)             |
| in | <i>uid</i>          | Unique identifier for the data                       |
| in | <i>data_length</i>  | The size in bytes of the data in <code>p_data</code> |
| in | <i>create_flags</i> | The flags indicating the properties of the data      |

#### Returns

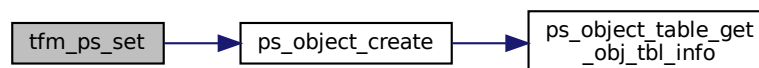
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

#### Return values

|                                       |                                                                                                                              |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                                         |
| <i>PSA_ERROR_NOT_PERMITTED</i>        | The operation failed because the provided uid value was already created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code>        |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because one or more of the given arguments were invalid (null pointer, etc.)                            |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because one or more of the flags provided in <code>create_flags</code> is not supported or is not valid |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because there was insufficient space on the storage medium                                              |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (fatal error)                                                   |
| <i>PSA_ERROR_GENERIC_ERROR</i>        | The operation failed because of an unspecified internal failure.                                                             |

Definition at line 71 of file `tfm_protected_storage.c`.

Here is the call graph for this function:



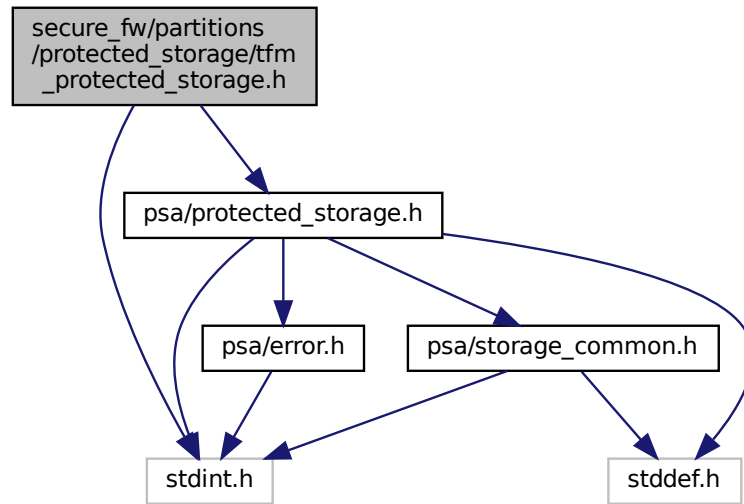
## 8.379 secure\_fw/partitions/protected\_storage/tfm\_protected\_storage.h

### File Reference

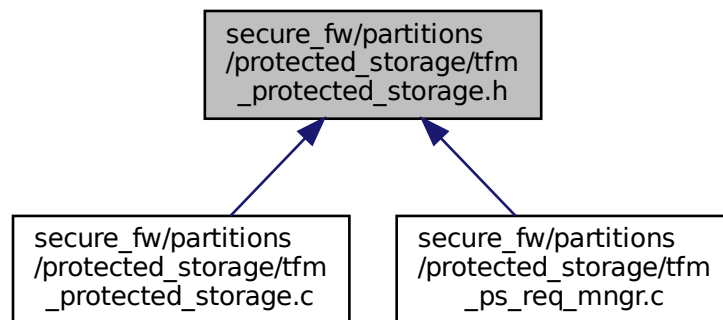
```
#include <stdint.h>
```

```
#include "psa/protected_storage.h"
```

Include dependency graph for tfm\_protected\_storage.h:



This graph shows which files directly or indirectly include this file:



## Functions

- [psa\\_status\\_t tfm\\_ps\\_init \(void\)](#)  
*Initializes the protected storage system.*
- [psa\\_status\\_t tfm\\_ps\\_set \(int32\\_t client\\_id, \[psa\\\_storage\\\_uid\\\_t\]\(#\) uid, uint32\\_t data\\_length, \[psa\\\_storage\\\_create\\\_flags\\\_t\]\(#\) create\\_flags\)](#)  
*Creates a new or modifies an existing asset.*
- [psa\\_status\\_t tfm\\_ps\\_get \(int32\\_t client\\_id, \[psa\\\_storage\\\_uid\\\_t\]\(#\) uid, uint32\\_t data\\_offset, uint32\\_t data\\_size, size\\_t \\*p\\_data\\_length\)](#)  
*Gets the asset data for the provided uid.*

- [psa\\_status\\_t tfm\\_ps\\_get\\_info](#) (int32\_t client\_id, [psa\\_storage\\_uid\\_t](#) uid, struct [psa\\_storage\\_info\\_t](#) \*p\_info)  
*Gets the metadata for the provided uid.*
- [psa\\_status\\_t tfm\\_ps\\_remove](#) (int32\_t client\_id, [psa\\_storage\\_uid\\_t](#) uid)  
*Removes the provided uid and its associated data from storage.*
- uint32\_t [tfm\\_ps\\_get\\_support](#) (void)  
*Gets a bitmask with flags set for all of the optional features supported by the implementation.*

## 8.379.1 Function Documentation

### 8.379.1.1 tfm\_ps\_get()

```
psa_status_t tfm_ps_get (
 int32_t client_id,
 psa_storage_uid_t uid,
 uint32_t data_offset,
 uint32_t data_size,
 size_t * p_data_length)
```

Gets the asset data for the provided uid.

#### Parameters

|     |                      |                                                                                               |
|-----|----------------------|-----------------------------------------------------------------------------------------------|
| in  | <i>client_id</i>     | Identifier of the asset's owner (client)                                                      |
| in  | <i>uid</i>           | Unique identifier for the data                                                                |
| in  | <i>data_offset</i>   | The offset within the data associated with the <code>uid</code> to start retrieving data      |
| in  | <i>data_size</i>     | The amount of data to read (and the minimum allocated size of the <code>p_data</code> buffer) |
| out | <i>p_data_length</i> | The pointer to the size of the data retrieved upon success.                                   |

#### Returns

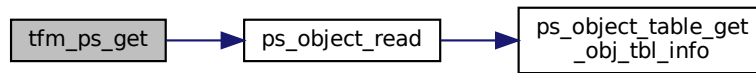
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

#### Return values

|                                    |                                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                 | The operation completed successfully                                                              |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>  | The operation failed because one or more of the given arguments were invalid (null pointer, etc.) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>    | The operation failed because the provided uid value was not found in the storage                  |
| <i>PSA_ERROR_STORAGE_FAILURE</i>   | The operation failed because the physical storage has failed (fatal error)                        |
| <i>PSA_ERROR_GENERIC_ERROR</i>     | The operation failed because of an unspecified internal failure                                   |
| <i>PSA_ERROR_DATA_CORRUPT</i>      | The operation failed because the data associated with the UID was corrupt                         |
| <i>PSA_ERROR_INVALID_SIGNATURE</i> | The operation failed because the data associated with the UID failed authentication               |

Definition at line 92 of file `tfm_protected_storage.c`.

Here is the call graph for this function:



### 8.379.1.2 tfm\_ps\_get\_info()

```

psa_status_t tfm_ps_get_info (
 int32_t client_id,
 psa_storage_uid_t uid,
 struct psa_storage_info_t * p_info)

```

Gets the metadata for the provided uid.

#### Parameters

|     |                  |                                                                                                  |
|-----|------------------|--------------------------------------------------------------------------------------------------|
| in  | <i>client_id</i> | Identifier of the asset's owner (client)                                                         |
| in  | <i>uid</i>       | Unique identifier for the data                                                                   |
| out | <i>p_info</i>    | A pointer to the <code>psa_storage_info_t</code> struct that will be populated with the metadata |

#### Returns

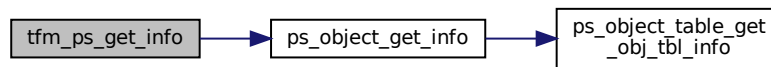
A status indicating the success/failure of the operation as specified in `psa_status_t`

#### Return values

|                                    |                                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                 | The operation completed successfully                                                              |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>  | The operation failed because one or more of the given arguments were invalid (null pointer, etc.) |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>    | The operation failed because the provided uid value was not found in the storage                  |
| <i>PSA_ERROR_STORAGE_FAILURE</i>   | The operation failed because the physical storage has failed (fatal error)                        |
| <i>PSA_ERROR_GENERIC_ERROR</i>     | The operation failed because of an unspecified internal failure                                   |
| <i>PSA_ERROR_DATA_CORRUPT</i>      | The operation failed because the data associated with the UID was corrupt                         |
| <i>PSA_ERROR_INVALID_SIGNATURE</i> | The operation failed because the data associated with the UID failed authentication               |

Definition at line 110 of file `tfm_protected_storage.c`.

Here is the call graph for this function:



### 8.379.1.3 tfm\_ps\_get\_support()

```
uint32_t tfm_ps_get_support (
 void)
```

Gets a bitmask with flags set for all of the optional features supported by the implementation.

#### Returns

Bitmask value which contains all the bits set for all the optional features supported by the implementation

Definition at line 147 of file `tfm_protected_storage.c`.

### 8.379.1.4 tfm\_ps\_init()

```
psa_status_t tfm_ps_init (
 void)
```

Initializes the protected storage system.

#### Returns

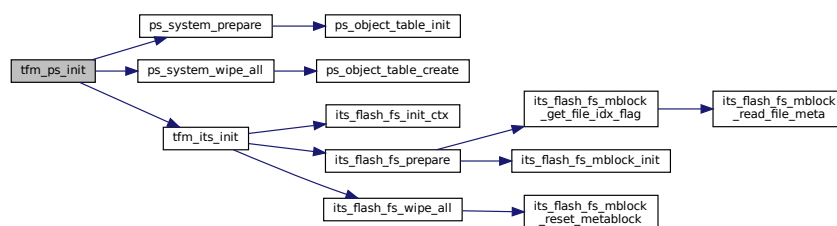
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

#### Return values

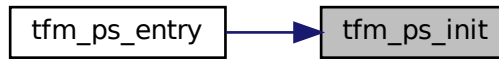
|                                  |                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>               | The operation completed successfully                                                    |
| <i>PSA_ERROR_STORAGE_FAILURE</i> | The operation failed because the storage system initialization has failed (fatal error) |
| <i>PSA_ERROR_GENERIC_ERROR</i>   | The operation failed because of an unspecified internal failure                         |

Definition at line 25 of file `tfm_protected_storage.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.379.1.5 tfm\_ps\_remove()

```

psa_status_t tfm_ps_remove (
 int32_t client_id,
 psa_storage_uid_t uid)

```

Removes the provided uid and its associated data from storage.

#### Parameters

|    |                  |                                              |
|----|------------------|----------------------------------------------|
| in | <i>client_id</i> | Identifier of the asset's owner (client)     |
| in | <i>uid</i>       | Unique identifier for the data to be removed |

#### Returns

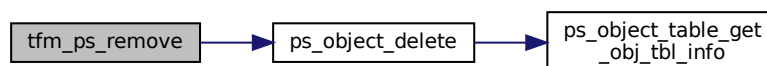
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

#### Return values

|                                   |                                                                                                               |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                | The operation completed successfully                                                                          |
| <i>PSA_ERROR_INVALID_ARGUMENT</i> | The operation failed because one or more of the given arguments were invalid (null pointer, etc.)             |
| <i>PSA_ERROR_DOES_NOT_EXIST</i>   | The operation failed because the provided uid value was not found in the storage                              |
| <i>PSA_ERROR_NOT_PERMITTED</i>    | The operation failed because the provided uid value was created with <code>PSA_STORAGE_FLAG_WRITE_ONCE</code> |
| <i>PSA_ERROR_STORAGE_FAILURE</i>  | The operation failed because the physical storage has failed (fatal error)                                    |
| <i>PSA_ERROR_GENERIC_ERROR</i>    | The operation failed because of an unspecified internal failure                                               |

Definition at line 124 of file `tfm_protected_storage.c`.

Here is the call graph for this function:



### 8.379.1.6 tfm\_ps\_set()

```
psa_status_t tfm_ps_set (
 int32_t client_id,
 psa_storage_uid_t uid,
 uint32_t data_length,
 psa_storage_create_flags_t create_flags)
```

Creates a new or modifies an existing asset.

#### Parameters

|    |                     |                                                 |
|----|---------------------|-------------------------------------------------|
| in | <i>client_id</i>    | Identifier of the asset's owner (client)        |
| in | <i>uid</i>          | Unique identifier for the data                  |
| in | <i>data_length</i>  | The size in bytes of the data in <i>p_data</i>  |
| in | <i>create_flags</i> | The flags indicating the properties of the data |

#### Returns

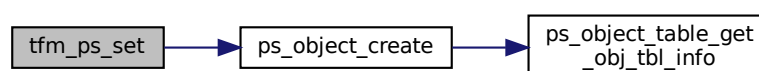
A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

#### Return values

|                                       |                                                                                                                        |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                    | The operation completed successfully                                                                                   |
| <i>PSA_ERROR_NOT_PERMITTED</i>        | The operation failed because the provided uid value was already created with <i>PSA_STORAGE_FLAG_WRITE_ONCE</i>        |
| <i>PSA_ERROR_INVALID_ARGUMENT</i>     | The operation failed because one or more of the given arguments were invalid (null pointer, etc.)                      |
| <i>PSA_ERROR_NOT_SUPPORTED</i>        | The operation failed because one or more of the flags provided in <i>create_flags</i> is not supported or is not valid |
| <i>PSA_ERROR_INSUFFICIENT_STORAGE</i> | The operation failed because there was insufficient space on the storage medium                                        |
| <i>PSA_ERROR_STORAGE_FAILURE</i>      | The operation failed because the physical storage has failed (fatal error)                                             |
| <i>PSA_ERROR_GENERIC_ERROR</i>        | The operation failed because of an unspecified internal failure.                                                       |

Definition at line 71 of file *tfm\_protected\_storage.c*.

Here is the call graph for this function:



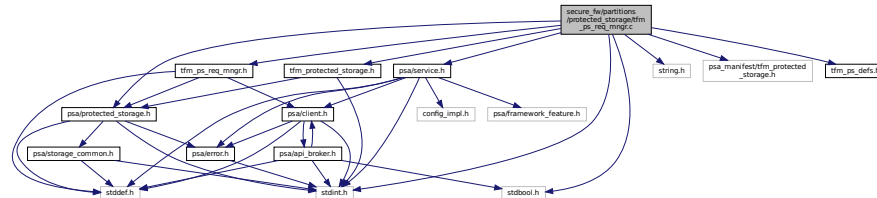
## 8.380 secure\_fw/partitions/protected\_storage/tfm\_ps\_req\_mgr.c File Reference

```
#include "tfm_ps_req_mgr.h"
#include <stdbool.h>
```



```
#include <stdint.h>
#include <string.h>
#include "psa/protected_storage.h"
#include "tfm_protected_storage.h"
#include "psa/service.h"
#include "psa_manifest/tfm_protected_storage.h"
#include "tfm_ps_defs.h"
```

Include dependency graph for tfm\_ps\_req\_mgr.c:



## Functions

- [psa\\_status\\_t tfm\\_protected\\_storage\\_service\\_sfn](#) (const [psa\\_msg\\_t](#) \*msg)
- [psa\\_status\\_t tfm\\_ps\\_entry](#) (void)
- [psa\\_status\\_t ps\\_req\\_mgr\\_read\\_asset\\_data](#) (uint8\_t \*out\_data, uint32\_t size)  
*Writes the asset data of a client iovec onto an output buffer.*
- [void ps\\_req\\_mgr\\_write\\_asset\\_data](#) (const uint8\_t \*in\_data, uint32\_t size)  
*Takes an input buffer containing asset data and writes its contents to the client iovec.*

### 8.380.1 Function Documentation

#### 8.380.1.1 ps\_req\_mgr\_read\_asset\_data()

```
psa_status_t ps_req_mgr_read_asset_data (
 uint8_t * out_data,
 uint32_t size)
```

Writes the asset data of a client iovec onto an output buffer.

#### Parameters

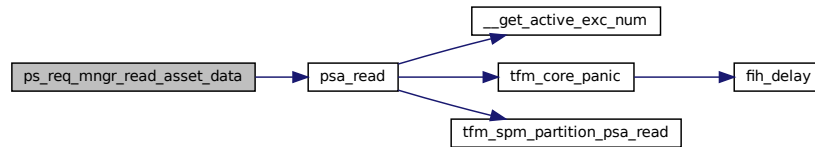
|     |                 |                                                |
|-----|-----------------|------------------------------------------------|
| out | <i>out_data</i> | Pointer to the buffer data will be written to. |
| in  | <i>size</i>     | The amount of data to write.                   |

**Returns**

A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

Definition at line 164 of file `tfm_ps_req_mgr.c`.

Here is the call graph for this function:

**8.380.1.2 ps\_req\_mgr\_write\_asset\_data()**

```
void ps_req_mgr_write_asset_data (
 const uint8_t * in_data,
 uint32_t size)
```

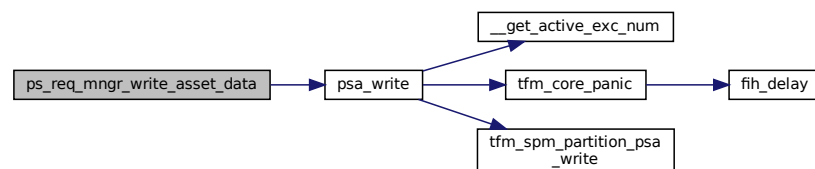
Takes an input buffer containing asset data and writes its contents to the client iovec.

**Parameters**

|    |                |                                            |
|----|----------------|--------------------------------------------|
| in | <i>in_data</i> | Pointer to the buffer data will read from. |
| in | <i>size</i>    | The amount of data to read.                |

Definition at line 175 of file `tfm_ps_req_mgr.c`.

Here is the call graph for this function:

**8.380.1.3 tfm\_protected\_storage\_service\_sfn()**

```
psa_status_t tfm_protected_storage_service_sfn (
 const psa_msg_t * msg)
```

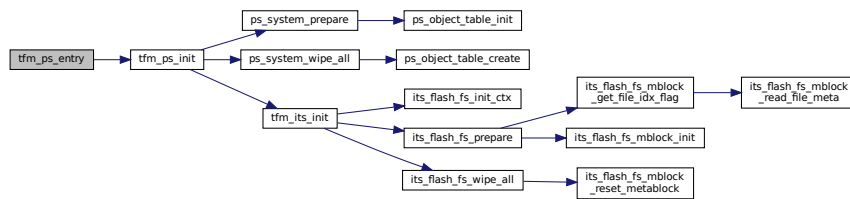
Definition at line 139 of file `tfm_ps_req_mgr.c`.

**8.380.1.4 tfm\_ps\_entry()**

```
psa_status_t tfm_ps_entry (
 void)
```

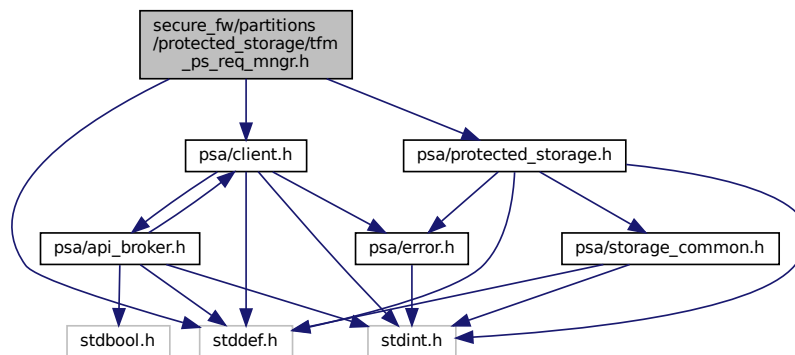
Definition at line 159 of file `tfm_ps_req_mgr.c`.

Here is the call graph for this function:

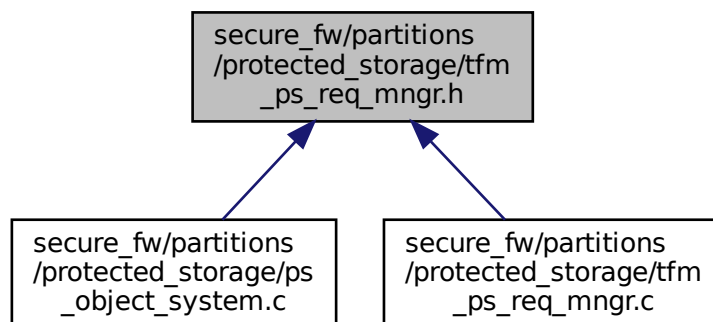


## 8.381 secure\_fw/partitions/protected\_storage/tfm\_ps\_req\_mgr.h File Reference

```
#include <stddef.h>
#include "psa/client.h"
#include "psa/protected_storage.h"
Include dependency graph for tfm_ps_req_mgr.h:
```



This graph shows which files directly or indirectly include this file:



## Functions

- [void ps\\_req\\_mgr\\_write\\_asset\\_data](#) (const uint8\_t \*in\_data, uint32\_t size)  
Takes an input buffer containing asset data and writes its contents to the client iovec.
- [psa\\_status\\_t ps\\_req\\_mgr\\_read\\_asset\\_data](#) (uint8\_t \*out\_data, uint32\_t size)  
Writes the asset data of a client iovec onto an output buffer.

### 8.381.1 Function Documentation

#### 8.381.1.1 ps\_req\_mgr\_read\_asset\_data()

```
psa_status_t ps_req_mgr_read_asset_data (
 uint8_t * out_data,
 uint32_t size)
```

Writes the asset data of a client iovec onto an output buffer.

##### Parameters

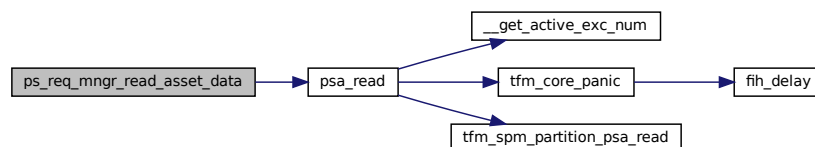
|     |                 |                                                |
|-----|-----------------|------------------------------------------------|
| out | <i>out_data</i> | Pointer to the buffer data will be written to. |
| in  | <i>size</i>     | The amount of data to write.                   |

##### Returns

A status indicating the success/failure of the operation as specified in [psa\\_status\\_t](#)

Definition at line 164 of file tfm\_ps\_req\_mgr.c.

Here is the call graph for this function:



#### 8.381.1.2 ps\_req\_mgr\_write\_asset\_data()

```
void ps_req_mgr_write_asset_data (
 const uint8_t * in_data,
 uint32_t size)
```

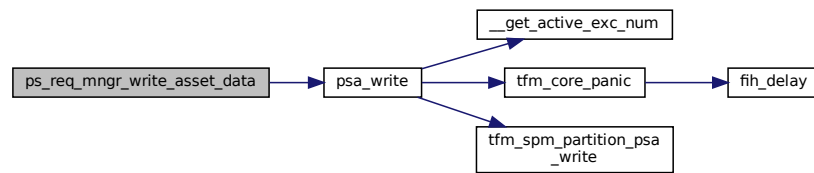
Takes an input buffer containing asset data and writes its contents to the client iovec.

##### Parameters

|    |                |                                            |
|----|----------------|--------------------------------------------|
| in | <i>in_data</i> | Pointer to the buffer data will read from. |
| in | <i>size</i>    | The amount of data to read.                |

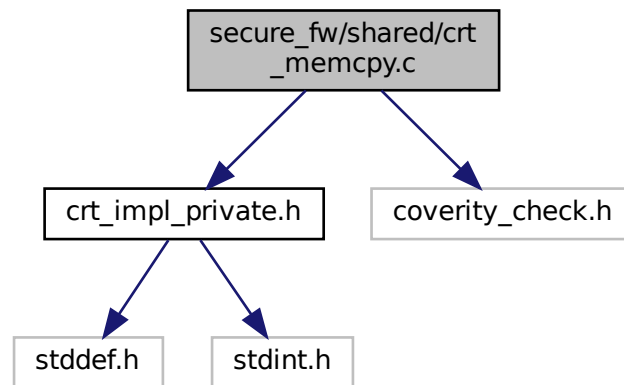
Definition at line 175 of file tfm\_ps\_req\_mgr.c.

Here is the call graph for this function:



## 8.382 secure\_fw/shared/crt\_memcpy.c File Reference

```
#include "crt_impl_private.h"
#include "coverity_check.h"
Include dependency graph for crt_memcpy.c:
```



## Functions

- `void * memcpy (void *dest, const void *src, size_t n)`

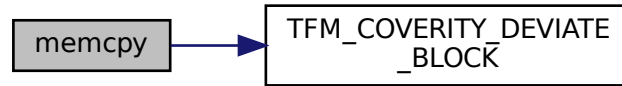
### 8.382.1 Function Documentation

#### 8.382.1.1 memcpy()

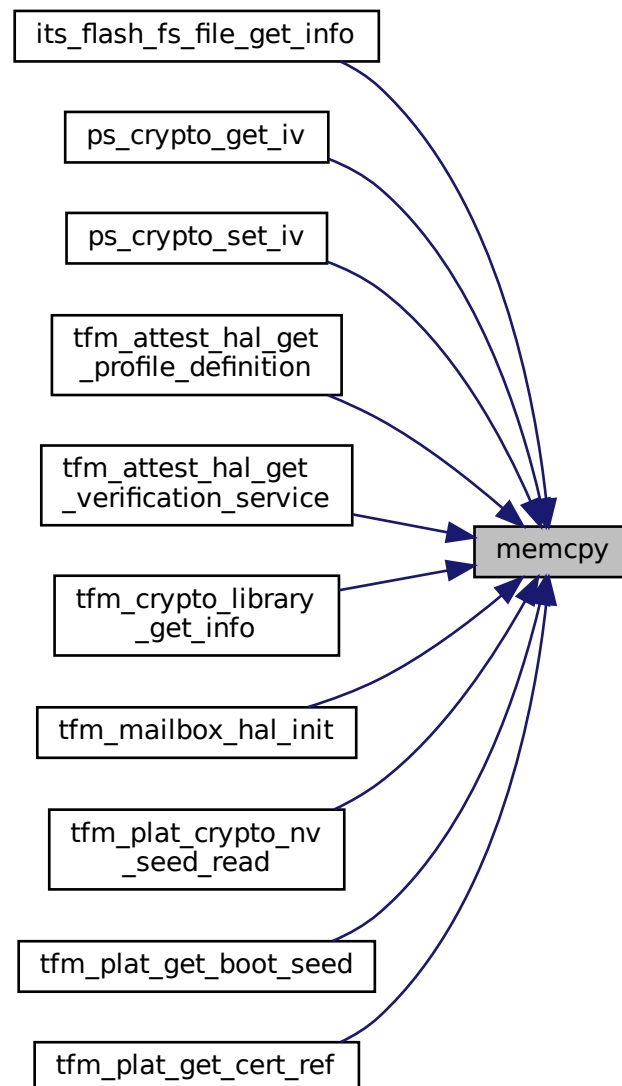
```
void* memcpy (
 void * dest,
 const void * src,
 size_t n)
```

Definition at line 11 of file `crt_memcpy.c`.

Here is the call graph for this function:



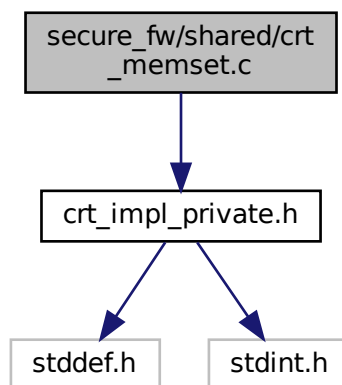
Here is the caller graph for this function:



## 8.383 secure\_fw/shared/crt\_memset.c File Reference

```
#include "crt_impl_private.h"
```

Include dependency graph for crt\_memset.c:



### Functions

- `void * memset (void *s, int c, size_t n)`

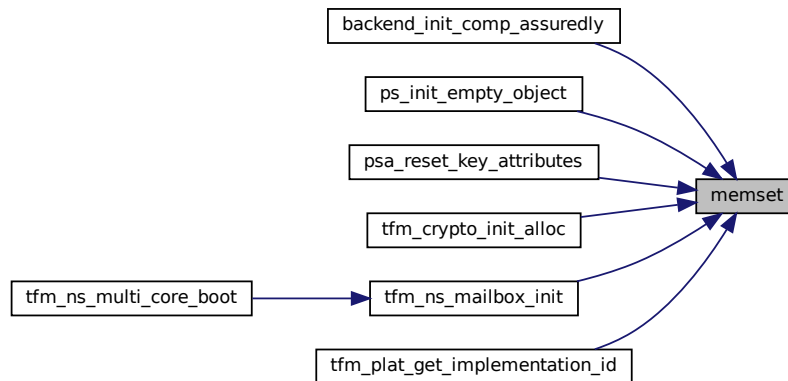
### 8.383.1 Function Documentation

#### 8.383.1.1 memset()

```
void* memset (
 void * s,
 int c,
 size_t n)
```

Definition at line 10 of file `crt_memset.c`.

Here is the caller graph for this function:



## 8.384 secure\_fw/shared/crt\_strcmp.c File Reference

### Functions

- int [strcmp](#) (const char \*s1, const char \*s2)

### 8.384.1 Function Documentation

#### 8.384.1.1 strcmp()

```
int strcmp (
 const char * s1,
 const char * s2)
```

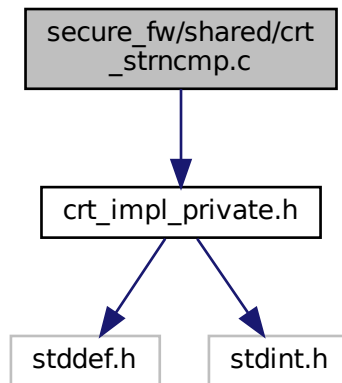
Definition at line 7 of file crt\_strcmp.c.

## 8.385 secure\_fw/shared/crt\_strncmp.c File Reference

```
#include "crt_impl_private.h"
```



Include dependency graph for crt\_strncmp.c:



## Functions

- int `strncmp` (const char \*s1, const char \*s2, size\_t n)

### 8.385.1 Function Documentation

#### 8.385.1.1 strncmp()

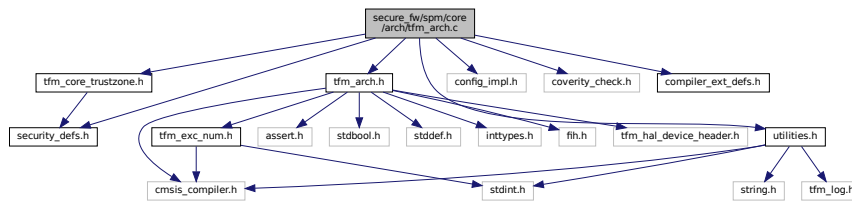
```
int strncmp (
 const char * s1,
 const char * s2,
 size_t n)
```

Definition at line 8 of file `crt_strncmp.c`.

### 8.386 secure\_fw/spm/core/arch/tfm\_arch.c File Reference

```
#include "security_defs.h"
#include "tfm_arch.h"
#include "tfm_core_trustzone.h"
#include "utilities.h"
#include "config_impl.h"
#include "coverity_check.h"
#include "compiler_ext_defs.h"
```

Include dependency graph for tfm\_arch.c:



## Functions

- `__naked void tfm_arch_free_msp_and_exc_ret` (uint32\_t msp\_base, uint32\_t exc\_return)
- `void tfm_arch_init_context` (struct context\_ctrl\_t \*p\_ctx\_ctrl, uintptr\_t pfn, void \*param, uintptr\_t pfnlr)
- `uint32_t tfm_arch_refresh_hardware_context` (const struct context\_ctrl\_t \*p\_ctx\_ctrl)

### 8.386.1 Function Documentation

#### 8.386.1.1 tfm\_arch\_free\_msp\_and\_exc\_ret()

```
__naked void tfm_arch_free_msp_and_exc_ret (
 uint32_t msp_base,
 uint32_t exc_return)
```

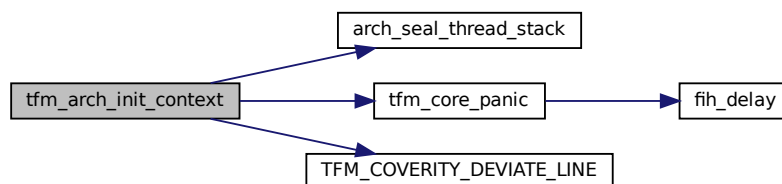
Definition at line 21 of file tfm\_arch.c.

#### 8.386.1.2 tfm\_arch\_init\_context()

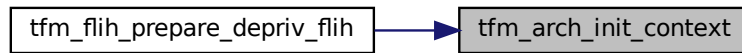
```
void tfm_arch_init_context (
 struct context_ctrl_t * p_ctx_ctrl,
 uintptr_t pfn,
 void * param,
 uintptr_t pfnlr)
```

Definition at line 137 of file tfm\_arch.c.

Here is the call graph for this function:



Here is the caller graph for this function:

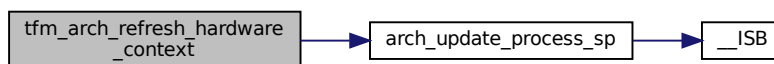


### 8.386.1.3 tfm\_arch\_refresh\_hardware\_context()

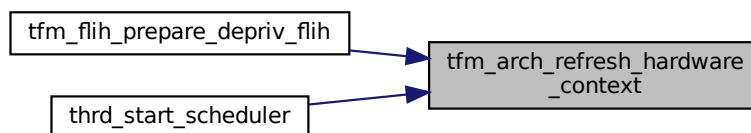
```
uint32_t tfm_arch_refresh_hardware_context (
 const struct context_ctrl_t * p_ctx_ctrl)
```

Definition at line 172 of file `tfm_arch.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



## 8.387 secure\_fw/spm/core/arch/tfm\_arch\_v6m\_v7m.c File Reference

```
#include <inttypes.h>
#include "security_defs.h"
#include "utilities.h"
#include "spm.h"
#include "tfm_arch.h"
#include "tfm_hal_device_header.h"
#include "tfm_svcalls.h"
#include "svc_num.h"
#include "compiler_ext_defs.h"
```



### 8.387.2.2 scheduler\_lock

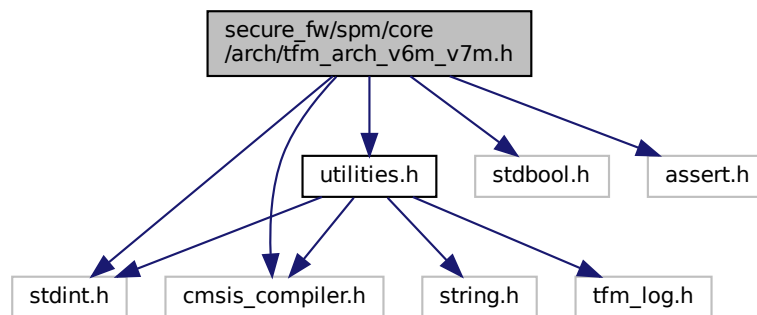
```
uint32_t scheduler_lock = SCHEDULER_UNLOCKED
```

Definition at line 27 of file tfm\_arch\_v6m\_v7m.c.

## 8.388 secure\_fw/spm/core/arch/tfm\_arch\_v6m\_v7m.h File Reference

```
#include <stdint.h>
#include <stdbool.h>
#include <assert.h>
#include "cmsis_compiler.h"
#include "utilities.h"
```

Include dependency graph for tfm\_arch\_v6m\_v7m.h:



## Macros

- #define [EXC\\_RETURN\\_SPSEL](#) (1UL << 2)
- #define [EXC\\_RETURN\\_MODE](#) (1UL << 3)
- #define [SCB\\_ICSR\\_ADDR](#) (0xE00ED04)
- #define [SCB\\_ICSR\\_PENDSVSET\\_BIT](#) (0x10000000)

## Functions

- `__STATIC_INLINE bool is\_return\_secure\_stack (uint32_t lr)`  
*Check whether Secure or Non-secure stack is used to restore stack frame on exception return.*
- `__STATIC_INLINE bool is\_default\_stacking\_rules\_apply (uint32_t lr)`  
*Check whether the default stacking rules apply, or whether the Additional state context, also known as callee registers, are already on the stack.*
- `__STATIC_INLINE uint32_t tfm\_arch\_get\_psplim (void)`  
*Get PSP Limit.*
- `__STATIC_INLINE void tfm\_arch\_set\_psplim (uint32_t psplim)`  
*Set PSP limit value.*
- `__STATIC_INLINE void tfm\_arch\_set\_msplim (uint32_t msplim)`  
*Set MSP limit value.*
- `__STATIC_INLINE uintptr_t arch\_seal\_thread\_stack (uintptr_t stk)`  
*Seal the thread stack.*
- `__STATIC_INLINE void tfm\_arch\_check\_msp\_sealing (void)`  
*Check MSP sealing.*

- `__STATIC_INLINE void arch_update_process_sp` (uint32\_t bottom, uint32\_t toplimit)
- `__STATIC_INLINE void tfm_arch_config_branch_protection` (void)  
*Empty implementation for branch protection. PACBTI applies for v8.1M Main Extension onwards.*

## Variables

- uint32\_t `psp_limit`

## 8.388.1 Macro Definition Documentation

### 8.388.1.1 EXC\_RETURN\_MODE

```
#define EXC_RETURN_MODE (1UL << 3)
```

Definition at line 33 of file `tfm_arch_v6m_v7m.h`.

### 8.388.1.2 EXC\_RETURN\_SPSEL

```
#define EXC_RETURN_SPSEL (1UL << 2)
```

Definition at line 31 of file `tfm_arch_v6m_v7m.h`.

### 8.388.1.3 SCB\_ICSR\_ADDR

```
#define SCB_ICSR_ADDR (0xE000ED04)
```

Definition at line 35 of file `tfm_arch_v6m_v7m.h`.

### 8.388.1.4 SCB\_ICSR\_PENDSVSET\_BIT

```
#define SCB_ICSR_PENDSVSET_BIT (0x10000000)
```

Definition at line 36 of file `tfm_arch_v6m_v7m.h`.

## 8.388.2 Function Documentation

### 8.388.2.1 arch\_seal\_thread\_stack()

```
__STATIC_INLINE uintptr_t arch_seal_thread_stack (
 uintptr_t stk)
```

Seal the thread stack.

#### Parameters

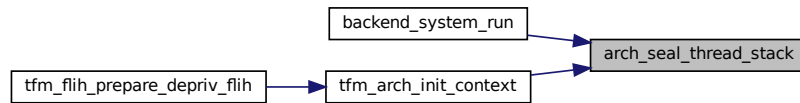
|                 |                  |                       |
|-----------------|------------------|-----------------------|
| <code>in</code> | <code>stk</code> | Thread stack address. |
|-----------------|------------------|-----------------------|

#### Return values

|                    |                               |
|--------------------|-------------------------------|
| <code>stack</code> | Updated thread stack address. |
|--------------------|-------------------------------|

Definition at line 143 of file `tfm_arch_v6m_v7m.h`.

Here is the caller graph for this function:

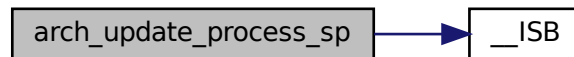


### 8.388.2.2 arch\_update\_process\_sp()

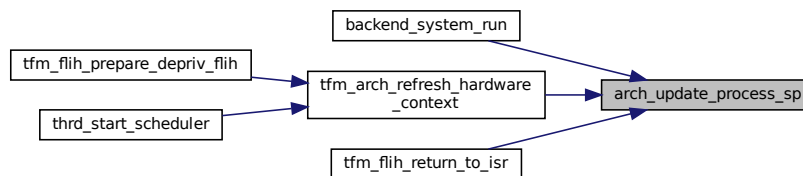
```
__STATIC_INLINE void arch_update_process_sp (
 uint32_t bottom,
 uint32_t toplimit)
```

Definition at line 161 of file tfm\_arch\_v6m\_v7m.h.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.388.2.3 is\_default\_stacking\_rules\_apply()

```
__STATIC_INLINE bool is_default_stacking_rules_apply (
 uint32_t lr)
```

Check whether the default stacking rules apply, or whether the Additional state context, also known as callee registers, are already on the stack.

#### Parameters

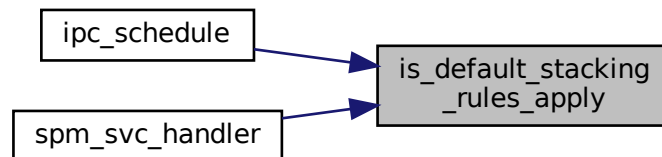
|    |    |                                              |
|----|----|----------------------------------------------|
| in | lr | LR register containing the EXC_RETURN value. |
|----|----|----------------------------------------------|

## Return values

|             |                                                             |
|-------------|-------------------------------------------------------------|
| <i>true</i> | Always use default stacking rules on v6m/v7m architectures. |
|-------------|-------------------------------------------------------------|

Definition at line 64 of file `tfm_arch_v6m_v7m.h`.

Here is the caller graph for this function:



#### 8.388.2.4 is\_return\_secure\_stack()

```
__STATIC_INLINE bool is_return_secure_stack (
 uint32_t lr)
```

Check whether Secure or Non-secure stack is used to restore stack frame on exception return.

## Parameters

|           |           |                                              |
|-----------|-----------|----------------------------------------------|
| <i>in</i> | <i>lr</i> | LR register containing the EXC_RETURN value. |
|-----------|-----------|----------------------------------------------|

## Return values

|             |                                                                      |
|-------------|----------------------------------------------------------------------|
| <i>true</i> | Always return to Secure stack on secure core in multi-core topology. |
|-------------|----------------------------------------------------------------------|

Definition at line 47 of file `tfm_arch_v6m_v7m.h`.

Here is the caller graph for this function:



#### 8.388.2.5 tfm\_arch\_check\_msp\_sealing()

```
__STATIC_INLINE void tfm_arch_check_msp_sealing (
 void)
```

Check MSP sealing.



Definition at line 152 of file tfm\_arch\_v6m\_v7m.h.

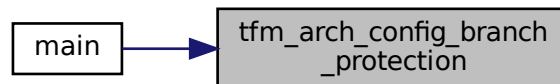
### 8.388.2.6 tfm\_arch\_config\_branch\_protection()

```
__STATIC_INLINE void tfm_arch_config_branch_protection (
 void)
```

Empty implementation for branch protection. PACBTI applies for v8.1M Main Extension onwards.

Definition at line 173 of file tfm\_arch\_v6m\_v7m.h.

Here is the caller graph for this function:



### 8.388.2.7 tfm\_arch\_get\_psplim()

```
__STATIC_INLINE uint32_t tfm_arch_get_psplim (
 void)
```

Get PSP Limit.

**Return values**

|            |                           |
|------------|---------------------------|
| <i>psp</i> | limit Limit of PSP stack. |
|------------|---------------------------|

Definition at line 107 of file tfm\_arch\_v6m\_v7m.h.

### 8.388.2.8 tfm\_arch\_set\_msplim()

```
__STATIC_INLINE void tfm_arch_set_msplim (
 uint32_t msplim)
```

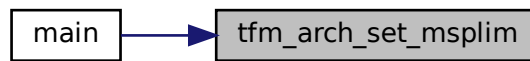
Set MSP limit value.

**Parameters**

|           |               |                                |
|-----------|---------------|--------------------------------|
| <i>in</i> | <i>msplim</i> | MSP limit value to be written. |
|-----------|---------------|--------------------------------|

Definition at line 127 of file tfm\_arch\_v6m\_v7m.h.

Here is the caller graph for this function:



### 8.388.2.9 tfm\_arch\_set\_psplim()

```
__STATIC_INLINE void tfm_arch_set_psplim (
 uint32_t psplim)
```

Set PSP limit value.

#### Parameters

|    |               |                                |
|----|---------------|--------------------------------|
| in | <i>psplim</i> | PSP limit value to be written. |
|----|---------------|--------------------------------|

Definition at line 117 of file `tfm_arch_v6m_v7m.h`.

## 8.388.3 Variable Documentation

### 8.388.3.1 psp\_limit

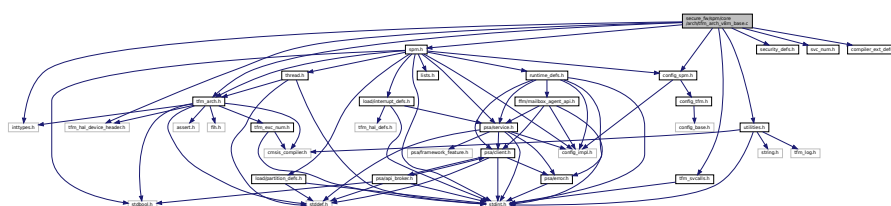
```
uint32_t psp_limit
```

Definition at line 24 of file `tfm_arch_v6m_v7m.c`.

## 8.389 secure\_fw/spm/core/arch/tfm\_arch\_v8m\_base.c File Reference

```
#include <inttypes.h>
#include "config_spm.h"
#include "security_defs.h"
#include "spm.h"
#include "svc_num.h"
#include "tfm_hal_device_header.h"
#include "tfm_arch.h"
#include "tfm_svcalls.h"
#include "utilities.h"
#include "compiler_ext_defs.h"
```

Include dependency graph for `tfm_arch_v8m_base.c`:



## Functions

- [void SVC\\_Handler \(void\)](#)
- [void tfm\\_arch\\_set\\_secure\\_exception\\_priorities \(void\)](#)
- [void tfm\\_arch\\_config\\_extensions \(void\)](#)

## Variables

- `uint32_t` [scheduler\\_lock](#) = [SCHEDULER\\_UNLOCKED](#)

### 8.389.1 Function Documentation

#### 8.389.1.1 SVC\_Handler()

```
void SVC_Handler (
 void)
```

Definition at line 172 of file `tfm_arch_v8m_base.c`.

#### 8.389.1.2 tfm\_arch\_config\_extensions()

```
void tfm_arch_config_extensions (
 void)
```

Definition at line 296 of file `tfm_arch_v8m_base.c`.

#### 8.389.1.3 tfm\_arch\_set\_secure\_exception\_priorities()

```
void tfm_arch_set_secure_exception_priorities (
 void)
```

Definition at line 252 of file `tfm_arch_v8m_base.c`.

### 8.389.2 Variable Documentation

#### 8.389.2.1 scheduler\_lock

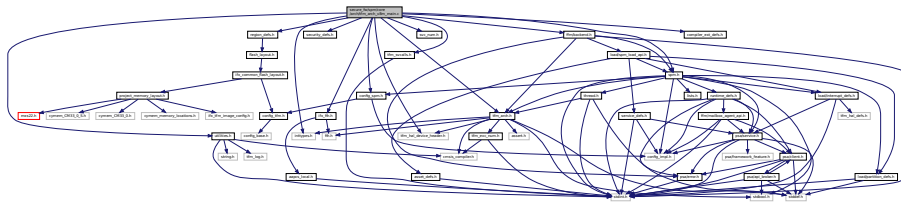
```
uint32_t scheduler_lock = SCHEDULER_UNLOCKED
```

Definition at line 27 of file `tfm_arch_v8m_base.c`.

## 8.390 secure\_fw/spm/core/arch/tfm\_arch\_v8m\_main.c File Reference

```
#include <inttypes.h>
#include "config_spm.h"
#include "ifx_fih.h"
#include "security_defs.h"
#include "region_defs.h"
#include "spm.h"
#include "svc_num.h"
#include "tfm_arch.h"
#include "tfm_hal_device_header.h"
#include "tfm_svcalls.h"
#include "utilities.h"
#include "ffm/backend.h"
```

```
#include "compiler_ext_defs.h"
Include dependency graph for tfm_arch_v8m_main.c:
```



## Functions

- [void SVC\\_Handler \(void\)](#)
- [void tfm\\_arch\\_set\\_secure\\_exception\\_priorities \(void\)](#)
- [void tfm\\_arch\\_config\\_extensions \(void\)](#)

## Variables

- [uint32\\_t scheduler\\_lock = SCHEDULER\\_UNLOCKED](#)

### 8.390.1 Function Documentation

#### 8.390.1.1 SVC\_Handler()

```
void SVC_Handler (
 void)
```

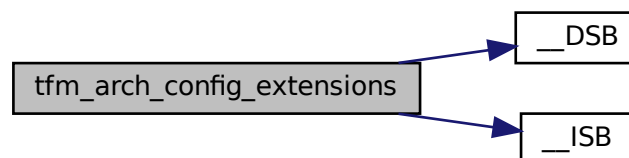
Definition at line 178 of file `tfm_arch_v8m_main.c`.

#### 8.390.1.2 tfm\_arch\_config\_extensions()

```
void tfm_arch_config_extensions (
 void)
```

Definition at line 332 of file `tfm_arch_v8m_main.c`.

Here is the call graph for this function:



### 8.390.1.3 tfm\_arch\_set\_secure\_exception\_priorities()

```
void tfm_arch_set_secure_exception_priorities (
 void)
```

Definition at line 269 of file tfm\_arch\_v8m\_main.c.

## 8.390.2 Variable Documentation

### 8.390.2.1 scheduler\_lock

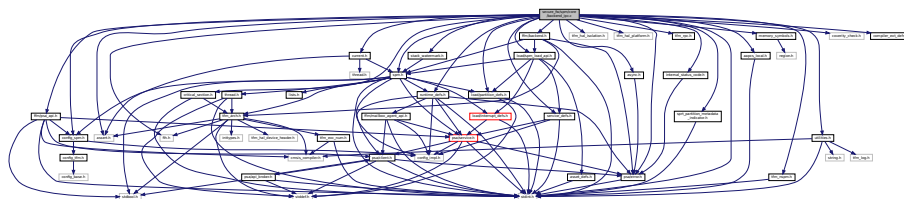
```
uint32_t scheduler_lock = SCHEDULER_UNLOCKED
```

Definition at line 34 of file tfm\_arch\_v8m\_main.c.

## 8.391 secure\_fw/spm/core/backend\_ipc.c File Reference

```
#include <stdint.h>
#include <assert.h>
#include "aapcs_local.h"
#include "async.h"
#include "config_spm.h"
#include "critical_section.h"
#include "current.h"
#include "ffm/psa_api.h"
#include "fih.h"
#include "runtime_defs.h"
#include "stack_watermark.h"
#include "spm.h"
#include "tfm_hal_isolation.h"
#include "tfm_hal_platform.h"
#include "tfm_nspm.h"
#include "tfm_rpc.h"
#include "ffm/backend.h"
#include "utilities.h"
#include "memory_symbols.h"
#include "load/partition_defs.h"
#include "load/service_defs.h"
#include "load/spm_load_api.h"
#include "psa/error.h"
#include "internal_status_code.h"
#include "sprt_partition_metadata_indicator.h"
#include "coverity_check.h"
#include "compiler_ext_defs.h"
```

Include dependency graph for backend\_ipc.c:



## Typedefs

- typedef `thrd_fn_t`(\* `comp_init_fn_t`) (struct `partition_t` \*, uint32\_t, uint32\_t \*)

## Functions

- [ARCH\\_CLAIM\\_CTXCTRL\\_INSTANCE](#) (spm\_thread\_context, spm\_thread\_stack, sizeof(spm\_thread\_stack))
- [psa\\_status\\_t backend\\_messaging](#) (struct [connection\\_t](#) \*p\_connection)
- [psa\\_status\\_t backend\\_replying](#) (struct [connection\\_t](#) \*handle, int32\_t status)
- [void common\\_sfn\\_thread](#) (void \*param)
- [void backend\\_init\\_comp\\_assuredly](#) (struct [partition\\_t](#) \*p\_pt, uint32\_t service\_setting)
- [uint32\\_t backend\\_system\\_run](#) (void)
- [psa\\_signal\\_t backend\\_wait\\_signals](#) (struct [partition\\_t](#) \*p\_pt, [psa\\_signal\\_t](#) signals)  
*Set the wait signal pattern in current partition.*
- [psa\\_status\\_t backend\\_assert\\_signal](#) (struct [partition\\_t](#) \*p\_pt, [psa\\_signal\\_t](#) signal)  
*Set the asserted signal pattern in current partition.*
- [uint64\\_t backend\\_abi\\_entering\\_spm](#) (void)
- [uint32\\_t backend\\_abi\\_leaving\\_spm](#) (uint32\_t result)
- [uint64\\_t ipc\\_schedule](#) (uint32\_t exc\_return)

## Variables

- struct [partition\\_head\\_t](#) [partition\\_listhead](#)
- struct [context\\_ctrl\\_t](#) \* [p\\_spm\\_thread\\_context](#) = &spm\_thread\_context
- struct [psa\\_api\\_tbl\\_t](#) [psa\\_api\\_thread\\_fn\\_call](#)
- struct [psa\\_api\\_tbl\\_t](#) [psa\\_api\\_svc](#)

## 8.391.1 Typedef Documentation

### 8.391.1.1 comp\_init\_fn\_t

typedef [thrd\\_fn\\_t](#)(\* comp\_init\_fn\_t) (struct [partition\\_t](#) \*, uint32\_t, uint32\_t \*)  
Definition at line 322 of file backend\_ipc.c.

## 8.391.2 Function Documentation

### 8.391.2.1 ARCH\_CLAIM\_CTXCTRL\_INSTANCE()

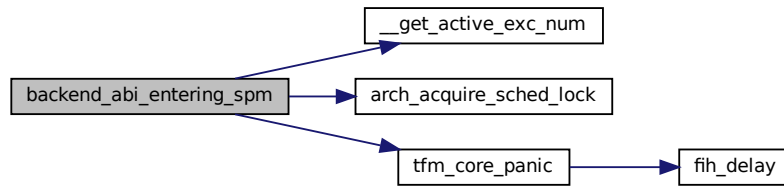
```
ARCH_CLAIM_CTXCTRL_INSTANCE (
 spm_thread_context ,
 spm_thread_stack ,
 sizeof(spm_thread_stack))
```

### 8.391.2.2 backend\_abi\_entering\_spm()

```
uint64_t backend_abi_entering_spm (
 void)
```

Definition at line 507 of file backend\_ipc.c.

Here is the call graph for this function:

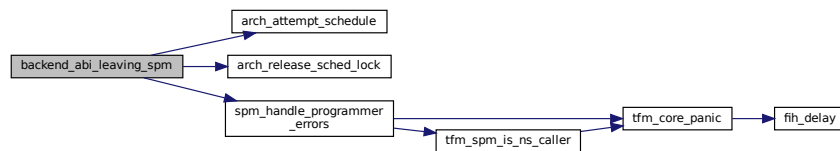


### 8.391.2.3 backend\_abi\_leaving\_spm()

```
uint32_t backend_abi_leaving_spm (
 uint32_t result)
```

Definition at line 538 of file backend\_ipc.c.

Here is the call graph for this function:



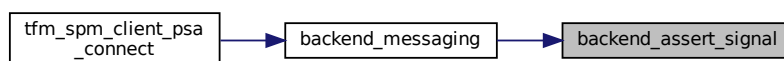
### 8.391.2.4 backend\_assert\_signal()

```
psa_status_t backend_assert_signal (
 struct partition_t * p_pt,
 psa_signal_t signal)
```

Set the asserted signal pattern in current partition.

Definition at line 481 of file backend\_ipc.c.

Here is the caller graph for this function:



### 8.391.2.5 backend\_init\_comp\_assuredly()

```
void backend_init_comp_assuredly (
 struct partition_t * p_pt,
 uint32_t service_setting)
```

Definition at line 329 of file backend\_ipc.c.

#### 8.391.2.6 backend\_messaging()

```
psa_status_t backend_messaging (
 struct connection_t * p_connection)
```

Definition at line 193 of file backend\_ipc.c.

Here is the caller graph for this function:



#### 8.391.2.7 backend\_replying()

```
psa_status_t backend_replying (
 struct connection_t * handle,
 int32_t status)
```

Definition at line 232 of file backend\_ipc.c.

#### 8.391.2.8 backend\_system\_run()

```
uint32_t backend_system_run (
 void)
```

Definition at line 361 of file backend\_ipc.c.

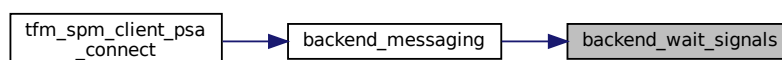
#### 8.391.2.9 backend\_wait\_signals()

```
psa_signal_t backend_wait_signals (
 struct partition_t * p_pt,
 psa_signal_t signals)
```

Set the wait signal pattern in current partition.

Definition at line 397 of file backend\_ipc.c.

Here is the caller graph for this function:



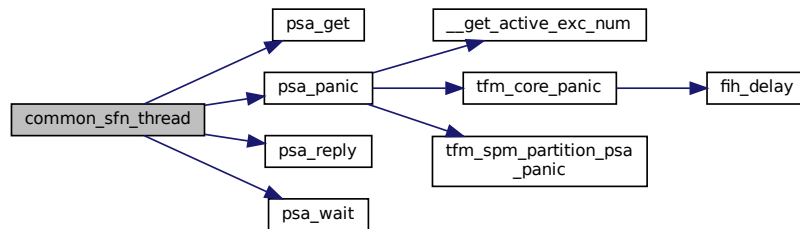


**8.391.2.10 common\_sfn\_thread()**

```
void common_sfn_thread (
 void * param)
```

Definition at line 19 of file sfn\_common\_thread.c.

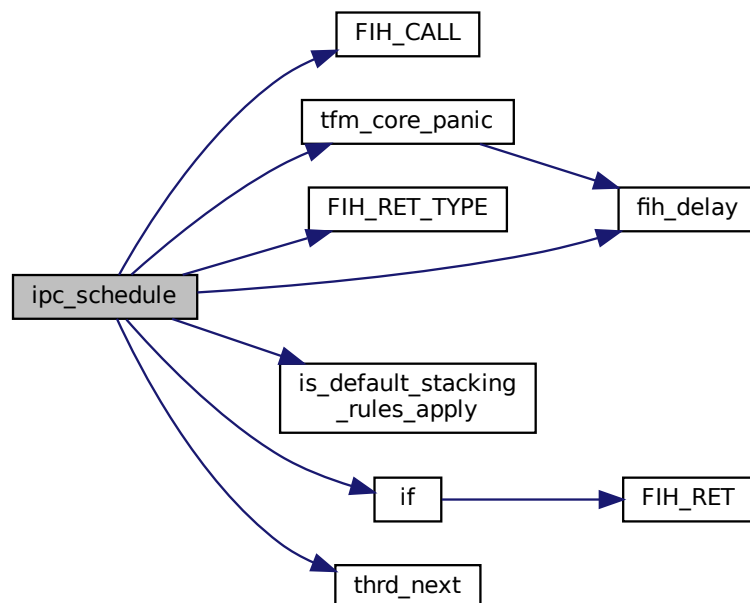
Here is the call graph for this function:

**8.391.2.11 ipc\_schedule()**

```
uint64_t ipc_schedule (
 uint32_t exc_return)
```

Definition at line 556 of file backend\_ipc.c.

Here is the call graph for this function:

**8.391.3 Variable Documentation**

### 8.391.3.1 p\_spm\_thread\_context

struct `context_ctrl_t`\* p\_spm\_thread\_context = &spm\_thread\_context  
Definition at line 57 of file backend\_ipc.c.

### 8.391.3.2 partition\_listhead

struct `partition_head_t` partition\_listhead  
Definition at line 45 of file backend\_ipc.c.

### 8.391.3.3 psa\_api\_svc

struct `psa_api_tbl_t` psa\_api\_svc  
Definition at line 225 of file psa\_interface\_svc.c.

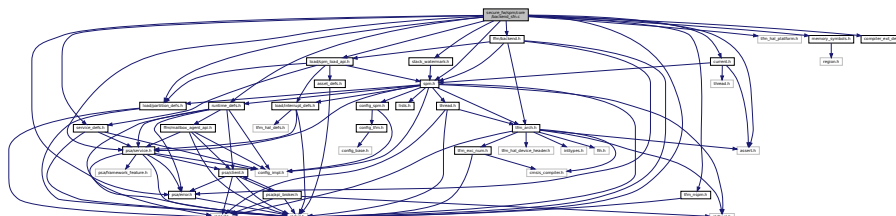
### 8.391.3.4 psa\_api\_thread\_fn\_call

struct `psa_api_tbl_t` psa\_api\_thread\_fn\_call  
Definition at line 242 of file psa\_interface\_thread\_fn\_call.c.

## 8.392 secure\_fw/spm/core/backend\_sfn.c File Reference

```
#include <stdint.h>
#include <assert.h>
#include "current.h"
#include "runtime_defs.h"
#include "tfm_hal_platform.h"
#include "tfm_nspm.h"
#include "ffm/backend.h"
#include "stack_watermark.h"
#include "load/partition_defs.h"
#include "load/service_defs.h"
#include "load/spm_load_api.h"
#include "psa/error.h"
#include "psa/service.h"
#include "spm.h"
#include "memory_symbols.h"
#include "compiler_ext_defs.h"
```

Include dependency graph for backend\_sfn.c:



## Macros

- `#define SFN_PARTITION_STATE_NOT_INITED 0`
- `#define SFN_PARTITION_STATE_INITED 1`

## Functions

- [psa\\_status\\_t backend\\_messaging](#) (struct [connection\\_t](#) \*p\_connection)
- [psa\\_status\\_t backend\\_replying](#) (struct [connection\\_t](#) \*handle, [int32\\_t](#) status)
- [void backend\\_init\\_comp\\_assuredly](#) (struct [partition\\_t](#) \*p\_pt, [uint32\\_t](#) service\_setting)
- [uint32\\_t backend\\_system\\_run](#) (void)
- [psa\\_signal\\_t backend\\_wait\\_signals](#) (struct [partition\\_t](#) \*p\_pt, [psa\\_signal\\_t](#) signals)  
*Set the wait signal pattern in current partition.*
- [psa\\_status\\_t backend\\_assert\\_signal](#) (struct [partition\\_t](#) \*p\_pt, [psa\\_signal\\_t](#) signal)  
*Set the asserted signal pattern in current partition.*

## Variables

- struct [partition\\_head\\_t](#) [partition\\_listhead](#)
- struct [partition\\_t](#) \* [p\\_current\\_partition](#)

## 8.392.1 Macro Definition Documentation

### 8.392.1.1 SFN\_PARTITION\_STATE\_INITED

```
#define SFN_PARTITION_STATE_INITED 1
```

Definition at line 30 of file backend\_sfn.c.

### 8.392.1.2 SFN\_PARTITION\_STATE\_NOT\_INITED

```
#define SFN_PARTITION_STATE_NOT_INITED 0
```

Definition at line 29 of file backend\_sfn.c.

## 8.392.2 Function Documentation

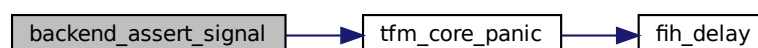
### 8.392.2.1 backend\_assert\_signal()

```
psa_status_t backend_assert_signal (
 struct partition_t * p_pt,
 psa_signal_t signal)
```

Set the asserted signal pattern in current partition.

Definition at line 201 of file backend\_sfn.c.

Here is the call graph for this function:

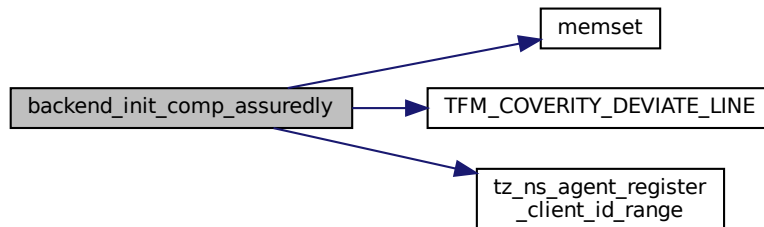


### 8.392.2.2 backend\_init\_comp\_assuredly()

```
void backend_init_comp_assuredly (
 struct partition_t * p_pt,
 uint32_t service_setting)
```

Definition at line 127 of file backend\_sfn.c.

Here is the call graph for this function:

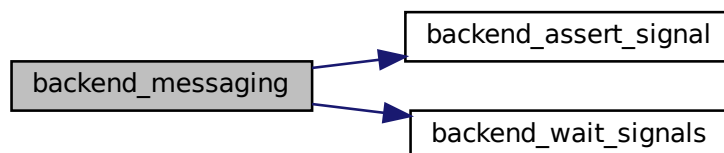


### 8.392.2.3 backend\_messaging()

```
psa_status_t backend_messaging (
 struct connection_t * p_connection)
```

Definition at line 42 of file backend\_sfn.c.

Here is the call graph for this function:



### 8.392.2.4 backend\_replying()

```
psa_status_t backend_replying (
 struct connection_t * handle,
 int32_t status)
```

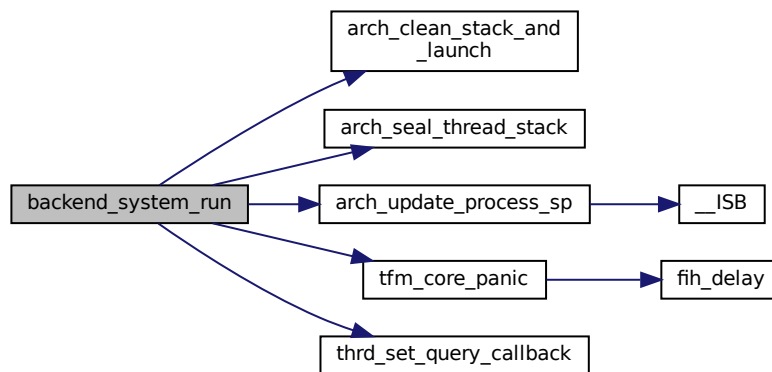
Definition at line 74 of file backend\_sfn.c.

### 8.392.2.5 backend\_system\_run()

```
uint32_t backend_system_run (
 void)
```

Definition at line 156 of file backend\_sfn.c.

Here is the call graph for this function:



#### 8.392.2.6 backend\_wait\_signals()

```

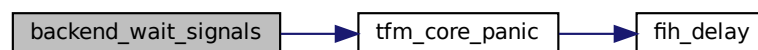
psa_signal_t backend_wait_signals (
 struct partition_t * p_pt,
 psa_signal_t signals)

```

Set the wait signal pattern in current partition.

Definition at line 192 of file `backend_sfn.c`.

Here is the call graph for this function:



### 8.392.3 Variable Documentation

#### 8.392.3.1 p\_current\_partition

```
struct partition_t* p_current_partition
```

Definition at line 36 of file `backend_sfn.c`.

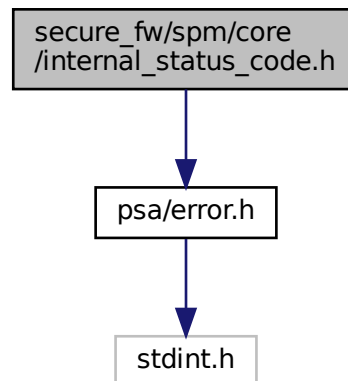
#### 8.392.3.2 partition\_listhead

```
struct partition_head_t partition_listhead
```

Definition at line 33 of file `backend_sfn.c`.

## 8.393 secure\_fw/spm/core/internal\_status\_code.h File Reference

```
#include "psa/error.h"
```



- #define SPM\_SUCCESS PSA\_SUCCESS
- #define SPM\_ERROR\_BAD\_PARAMETERS ((psa\_status\_t)-249)
- #define SPM\_ERROR\_SHORT\_BUFFER ((psa\_status\_t)-250)
- #define SPM\_ERROR\_VERSION ((psa\_status\_t)-251)
- #define SPM\_ERROR\_MEMORY\_CHECK ((psa\_status\_t)-252)
- #define SPM\_ERROR\_GENERIC ((psa\_status\_t)-253)
- #define STATUS\_NEED\_SCHEDULE ((psa\_status\_t)-254)

```
#define SPM_ERROR_BAD_PARAMETERS ((psa_status_t)-249)
```

Definition at line 15 of file `internal_status_code.h`.

```
#define SPM_ERROR_GENERIC ((psa_status_t)-253)
Definition at line 19 of file internal_status_code.h.
```

**8.393.1.3 SPM\_ERROR\_MEMORY\_CHECK**

```
#define SPM_ERROR_MEMORY_CHECK ((psa_status_t)-252)
```

Definition at line 18 of file internal\_status\_code.h.

**8.393.1.4 SPM\_ERROR\_SHORT\_BUFFER**

```
#define SPM_ERROR_SHORT_BUFFER ((psa_status_t)-250)
```

Definition at line 16 of file internal\_status\_code.h.

**8.393.1.5 SPM\_ERROR\_VERSION**

```
#define SPM_ERROR_VERSION ((psa_status_t)-251)
```

Definition at line 17 of file internal\_status\_code.h.

**8.393.1.6 SPM\_SUCCESS**

```
#define SPM_SUCCESS PSA_SUCCESS
```

Definition at line 12 of file internal\_status\_code.h.

**8.393.1.7 STATUS\_NEED\_SCHEDULE**

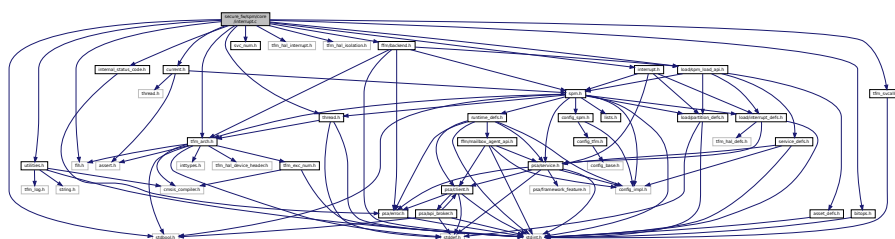
```
#define STATUS_NEED_SCHEDULE ((psa_status_t)-254)
```

Definition at line 21 of file internal\_status\_code.h.

**8.394 secure\_fw/spm/core/interrupt.c File Reference**

```
#include <stdbool.h>
#include "interrupt.h"
#include "bitops.h"
#include "current.h"
#include "fih.h"
#include "svc_num.h"
#include "tfm_arch.h"
#include "tfm_hal_interrupt.h"
#include "tfm_hal_isolation.h"
#include "tfm_svcalls.h"
#include "thread.h"
#include "utilities.h"
#include "load/spm_load_api.h"
#include "ffm/backend.h"
#include "internal_status_code.h"
```

Include dependency graph for interrupt.c:



## Functions

- `void tfm_flih_func_return (psa_flih_result_t result)`
- `uint32_t tfm_flih_prepare_depriv_flih (struct partition_t *p_owner_sp, uintptr_t flih_func)`
- `uint32_t tfm_flih_return_to_isr (psa_flih_result_t result, struct context_flih_ret_t *p_ctx_flih_ret)`
- `const struct irq_load_info_t * get_irq_info_for_signal (const struct partition_load_info_t *p_ldinf, psa_signal_t signal)`

*Return the IRQ load info context pointer associated with a signal.*

- `void spm_handle_interrupt (struct partition_t *p_pt, const struct irq_load_info_t *p_ildi)`

*Entry of Secure interrupt handler. Platforms can call this function to handle individual interrupts.*

### 8.394.1 Function Documentation

#### 8.394.1.1 get\_irq\_info\_for\_signal()

```
const struct irq_load_info_t* get_irq_info_for_signal (
 const struct partition_load_info_t * p_ldinf,
 psa_signal_t signal)
```

Return the IRQ load info context pointer associated with a signal.

##### Parameters

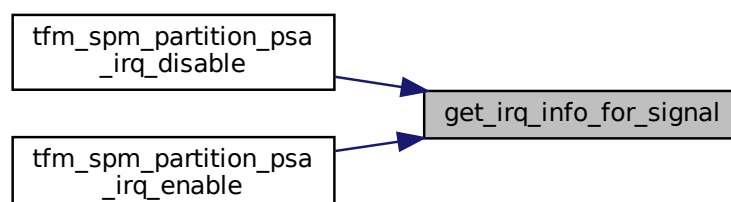
|    |                |                                                                 |
|----|----------------|-----------------------------------------------------------------|
| in | <i>p_ldinf</i> | The load info of the partition in which we look for the signal. |
| in | <i>signal</i>  | The signal to query for.                                        |

##### Return values

|             |                                                                                                                                                                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>NULL</i> | if one of more the following are true: <ul style="list-style-type: none"> <li>• the <code>psa_signal_t</code> signal indicates more than one signal</li> <li>• the <code>psa_signal_t</code> signal does not belong to the partition.</li> </ul> |
| <i>Any</i>  | other value The load info pointer associated with the signal                                                                                                                                                                                     |

Definition at line 191 of file interrupt.c.

Here is the caller graph for this function:





**8.394.1.2 spm\_handle\_interrupt()**

```
void spm_handle_interrupt (
 struct partition_t * p_pt,
 const struct irq_load_info_t * p_ildi)
```

Entry of Secure interrupt handler. Platforms can call this function to handle individual interrupts.

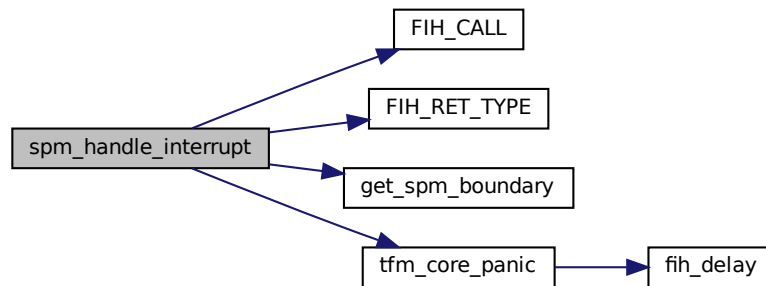
**Parameters**

|    |               |                                                                    |
|----|---------------|--------------------------------------------------------------------|
| in | <i>p_pt</i>   | The owner Partition of the interrupt to handle                     |
| in | <i>p_ildi</i> | The <code>irq_load_info_t</code> struct of the interrupt to handle |

Note: The input parameters are maintained by platforms and they must be init-ed in the interrupt init functions.

Definition at line 229 of file interrupt.c.

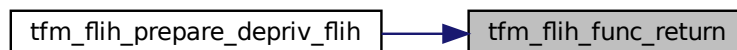
Here is the call graph for this function:

**8.394.1.3 tfm\_flih\_func\_return()**

```
void tfm_flih_func_return (
 psa_flih_result_t result)
```

Definition at line 28 of file service\_api.c.

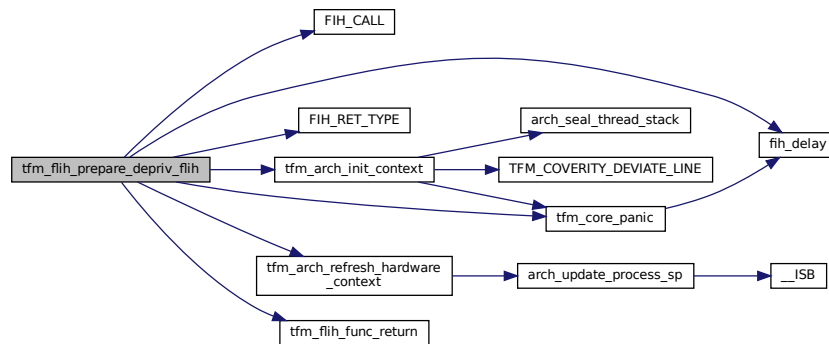
Here is the caller graph for this function:

**8.394.1.4 tfm\_flih\_prepare\_depriv\_flih()**

```
uint32_t tfm_flih_prepare_depriv_flih (
 struct partition_t * p_owner_sp,
 uintptr_t flih_func)
```

Definition at line 45 of file interrupt.c.

Here is the call graph for this function:



### 8.394.1.5 tfm\_flih\_return\_to\_isr()

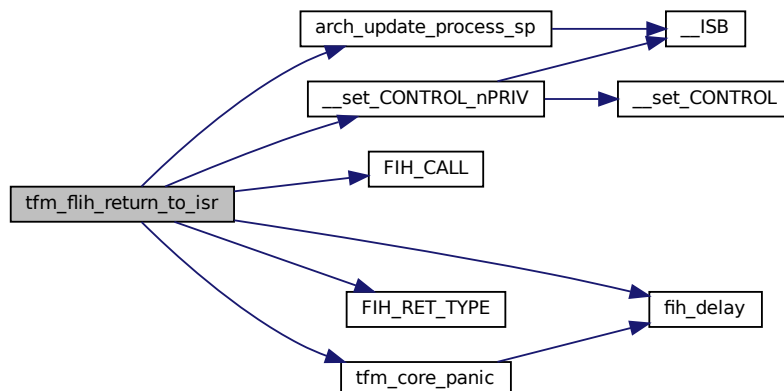
```

uint32_t tfm_flih_return_to_isr (
 psa_flih_result_t result,
 struct context_flih_ret_t * p_ctx_flih_ret)

```

Definition at line 121 of file interrupt.c.

Here is the call graph for this function:



## 8.395 secure\_fw/spm/core/interrupt.h File Reference

```

#include "spm.h"
#include "load/interrupt_defs.h"
#include "load/partition_defs.h"
#include "psa/service.h"

```

[illegible]

```
graph BT; A[secure_fw/spm/core/interrupt.h] <--> B[secure_fw/spm/core/interrupt.c]; A <--> C[secure_fw/spm/core/psa_api.c]; A <--> D[secure_fw/spm/core/psa_irq_api.c]; A <--> E[secure_fw/spm/core/tfm_svcalls.c]; A <--> F[platform/ext/target/infinion/common/spe/services/mailbox/tfm_interrupts.c];
```

- const struct [irq\\_load\\_info\\_t](#) \* [get\\_irq\\_info\\_for\\_signal](#) (const struct [partition\\_load\\_info\\_t](#) \*p\_ldinf, [psa\\_signal\\_t](#) signal)  
*Return the IRQ load info context pointer associated with a signal.*
- bool [tfm\\_get\\_isr\\_cookie](#) (void)  
*Clear and return the ISR cookie.*
- void [spm\\_handle\\_interrupt](#) (struct [partition\\_t](#) \*p\_pt, const struct [irq\\_load\\_info\\_t](#) \*p\_ildi)  
*Entry of Secure interrupt handler. Platforms can call this function to handle individual interrupts.*
- uint32\_t [tfm\\_flih\\_prepare\\_depriv\\_flih](#) (struct [partition\\_t](#) \*p\_owner\_sp, uintptr\_t flih\_func)
- uint32\_t [tfm\\_flih\\_return\\_to\\_isr](#) ([psa\\_flih\\_result\\_t](#) result, struct [context\\_flih\\_ret\\_t](#) \*p\_ctx\_flih\_ret)

### 8.395.1.1 `get_irq_info_for_signal()`

Return the IRQ load info context pointer associated with a signal.

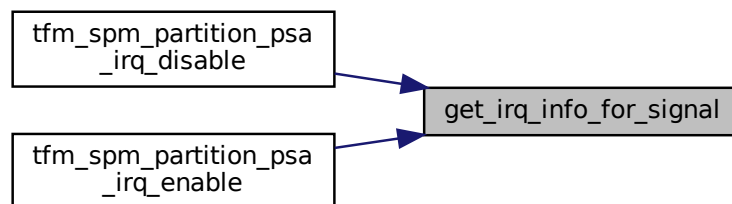
|    |                |                                                                 |
|----|----------------|-----------------------------------------------------------------|
| in | <i>p_ldinf</i> | The load info of the partition in which we look for the signal. |
| in | <i>signal</i>  | The signal to query for.                                        |

## Return values

|             |                                                                                                                                                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>NULL</i> | if one of more the following are true: <ul style="list-style-type: none"> <li>the <a href="#">psa_signal_t</a> signal indicates more than one signal</li> <li>the <a href="#">psa_signal_t</a> signal does not belong to the partition.</li> </ul> |
| <i>Any</i>  | other value The load info pointer associated with the signal                                                                                                                                                                                       |

Definition at line 191 of file interrupt.c.

Here is the caller graph for this function:



## 8.395.1.2 spm\_handle\_interrupt()

```

void spm_handle_interrupt (
 struct partition_t * p_pt,
 const struct irq_load_info_t * p_ildi)

```

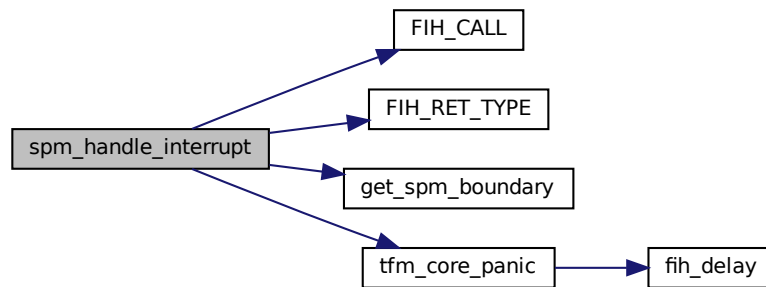
Entry of Secure interrupt handler. Platforms can call this function to handle individual interrupts.

## Parameters

|    |               |                                                                       |
|----|---------------|-----------------------------------------------------------------------|
| in | <i>p_pt</i>   | The owner Partition of the interrupt to handle                        |
| in | <i>p_ildi</i> | The <a href="#">irq_load_info_t</a> struct of the interrupt to handle |

Note: The input parameters are maintained by platforms and they must be init-ed in the interrupt init functions.  
 Definition at line 229 of file interrupt.c.

Here is the call graph for this function:



#### 8.395.1.3 tfm\_flih\_prepare\_depriv\_flih()

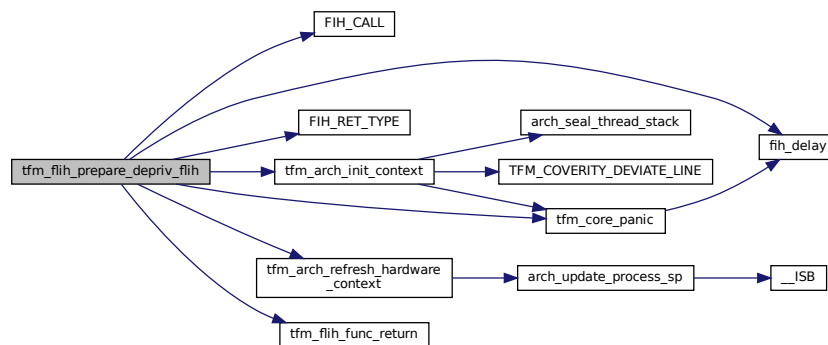
```

uint32_t tfm_flih_prepare_depriv_flih (
 struct partition_t * p_owner_sp,
 uintptr_t flih_func)

```

Definition at line 45 of file `interrupt.c`.

Here is the call graph for this function:



#### 8.395.1.4 tfm\_flih\_return\_to\_isr()

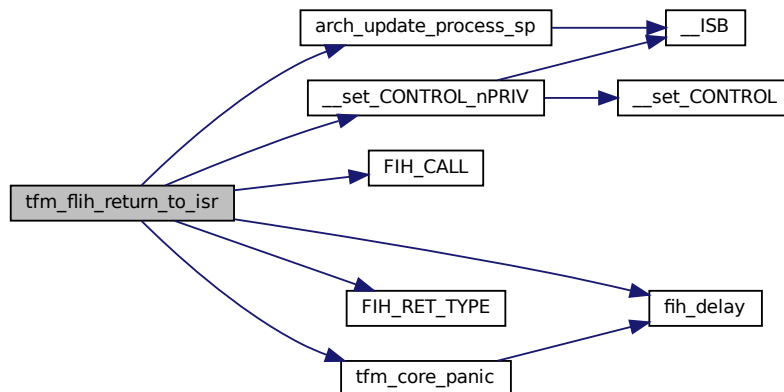
```

uint32_t tfm_flih_return_to_isr (
 psa_flih_result_t result,
 struct context_flih_ret_t * p_ctx_flih_ret)

```

Definition at line 121 of file `interrupt.c`.

Here is the call graph for this function:



#### 8.395.1.5 tfm\_get\_isr\_cookie()

```
bool tfm_get_isr_cookie (
 void)
```

Clear and return the ISR cookie.

In interrupt context, the backend asserts the partition's signals. If the partition requires scheduling, the spm interrupt handler sets a scheduling hint in the cookie. The cookie leaves a trace to the PSA API to run the scheduler for those partitions in polling mode.

##### Return values

|              |                                                                                                     |
|--------------|-----------------------------------------------------------------------------------------------------|
| <i>false</i> | There are no cookies left from ISR handling.                                                        |
| <i>true</i>  | At least one scheduling hint was set from ISR handling in the cookie. The cookie is always cleared. |

##### Returns

Returns a scheduling clue

## 8.396 secure\_fw/spm/core/mailbox\_agent\_api.c File Reference

```
#include "current.h"
#include "fih.h"
#include "internal_status_code.h"
#include "spm.h"
#include "tfm_hal_isolation.h"
#include "tfm_multi_core.h"
#include "ffm/mailbox_agent_api.h"
#include "ffm/backend.h"
#include "ffm/psa_api.h"
#include "psa/error.h"
```

[illegible]

- `psa_status_t tfm_spm_agent_psa_call (psa_handle_t handle, uint32_t control, const struct client\_params\_t *params, const void *client_data_stateless)`

```
psa_status_t tfm_spm_agent_psa_call (
 psa_handle_t handle,
 uint32_t control,
 const struct client_params_t * params,
 const void * client_data_stateless)
```

### 8.397 secure\_fw/spm/core/main.c File Reference

## Functions

- uintptr\_t [get\\_spm\\_boundary](#) (void)
- int [main](#) (void)

### 8.397.1 Function Documentation

#### 8.397.1.1 [get\\_spm\\_boundary](#)()

```
uintptr_t get_spm_boundary (
 void)
```

Definition at line 103 of file main.c.

Here is the caller graph for this function:

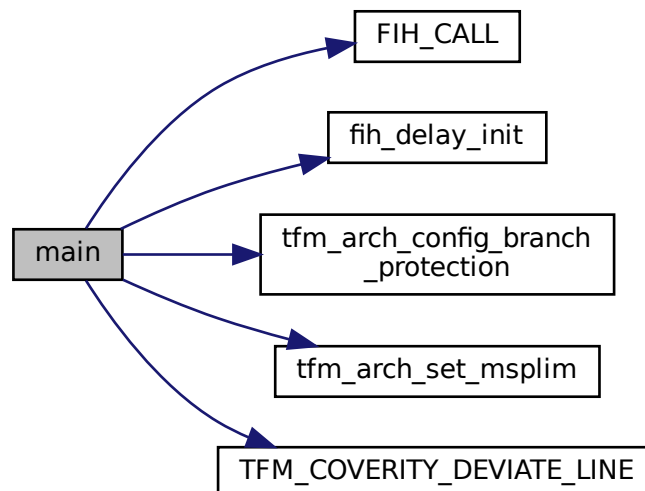


#### 8.397.1.2 [main](#)()

```
int main (
 void)
```

Definition at line 110 of file main.c.

Here is the call graph for this function:

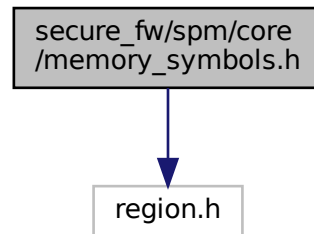




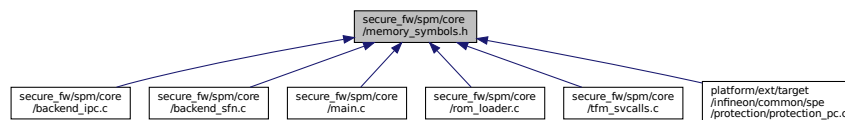
## 8.398 secure\_fw/spm/core/memory\_symbols.h File Reference

```
#include "region.h"
```

Include dependency graph for memory\_symbols.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define SPM_BOOT_STACK_TOP (uint32_t)&REGION_NAME(Image$$, ARM_LIB_STACK, $$ZI$$Base)`
- `#define SPM_BOOT_STACK_BOTTOM (uint32_t)&REGION_NAME(Image$$, ARM_LIB_STACK, $$ZI$$Limit)`
- `#define PART_INFOLIST_START (uintptr_t)&REGION_NAME(Image$$, TFM_SP_LOAD_LIST, $$RO$$Base)`
- `#define PART_INFOLIST_END (uintptr_t)&REGION_NAME(Image$$, TFM_SP_LOAD_LIST, $$RO$$Limit)`
- `#define PART_INFORAM_START (uintptr_t)&REGION_NAME(Image$$, ER_PART_RT_POOL, $$ZI$$Base)`
- `#define PART_INFORAM_END (uintptr_t)&REGION_NAME(Image$$, ER_PART_RT_POOL, $$ZI$$Limit)`
- `#define SERV_INFORAM_START (uintptr_t)&REGION_NAME(Image$$, ER_SERV_RT_POOL, $$ZI$$Base)`
- `#define SERV_INFORAM_END (uintptr_t)&REGION_NAME(Image$$, ER_SERV_RT_POOL, $$ZI$$Limit)`

### Functions

- `REGION_DECLARE (Image$$, ARM_LIB_STACK, $ $ZI$ $Base)`
- `REGION_DECLARE (Image$$, TFM_SP_LOAD_LIST, $ $RO$ $Base)`
- `REGION_DECLARE (Image$$, ER_PART_RT_POOL, $ $ZI$ $Base)`
- `REGION_DECLARE (Image$$, ER_SERV_RT_POOL, $ $ZI$ $Base)`

#### 8.398.1 Macro Definition Documentation

### 8.398.1.1 PART\_INFOLIST\_END

```
#define PART_INFOLIST_END (uintptr_t)®ION_NAME(Image$$, TFM_SP_LOAD_LIST, $$RO$$Limit)
```

Definition at line 44 of file memory\_symbols.h.

### 8.398.1.2 PART\_INFOLIST\_START

```
#define PART_INFOLIST_START (uintptr_t)®ION_NAME(Image$$, TFM_SP_LOAD_LIST, $$RO$$Base)
```

Definition at line 42 of file memory\_symbols.h.

### 8.398.1.3 PART\_INFORAM\_END

```
#define PART_INFORAM_END (uintptr_t)®ION_NAME(Image$$, ER_PART_RT_POOL, $$ZI$$Limit)
```

Definition at line 48 of file memory\_symbols.h.

### 8.398.1.4 PART\_INFORAM\_START

```
#define PART_INFORAM_START (uintptr_t)®ION_NAME(Image$$, ER_PART_RT_POOL, $$ZI$$Base)
```

Definition at line 46 of file memory\_symbols.h.

### 8.398.1.5 SERV\_INFORAM\_END

```
#define SERV_INFORAM_END (uintptr_t)®ION_NAME(Image$$, ER_SERV_RT_POOL, $$ZI$$Limit)
```

Definition at line 52 of file memory\_symbols.h.

### 8.398.1.6 SERV\_INFORAM\_START

```
#define SERV_INFORAM_START (uintptr_t)®ION_NAME(Image$$, ER_SERV_RT_POOL, $$ZI$$Base)
```

Definition at line 50 of file memory\_symbols.h.

### 8.398.1.7 SPM\_BOOT\_STACK\_BOTTOM

```
#define SPM_BOOT_STACK_BOTTOM (uint32_t)®ION_NAME(Image$$, ARM_LIB_STACK, $$ZI$$Limit)
```

Definition at line 27 of file memory\_symbols.h.

### 8.398.1.8 SPM\_BOOT\_STACK\_TOP

```
#define SPM_BOOT_STACK_TOP (uint32_t)®ION_NAME(Image$$, ARM_LIB_STACK, $$ZI$$Base)
```

Definition at line 25 of file memory\_symbols.h.

## 8.398.2 Function Documentation

### 8.398.2.1 REGION\_DECLARE() [1/4]

```
REGION_DECLARE (
 Image$$,
 ARM_LIB_STACK ,
 $ ZI)
```

**8.398.2.2 REGION\_DECLARE() [2/4]**

```
REGION_DECLARE (
 Image$$,
 ER_PART_RT_POOL ,
 $ ZI)
```

**8.398.2.3 REGION\_DECLARE() [3/4]**

```
REGION_DECLARE (
 Image$$,
 ER_SERV_RT_POOL ,
 $ ZI)
```

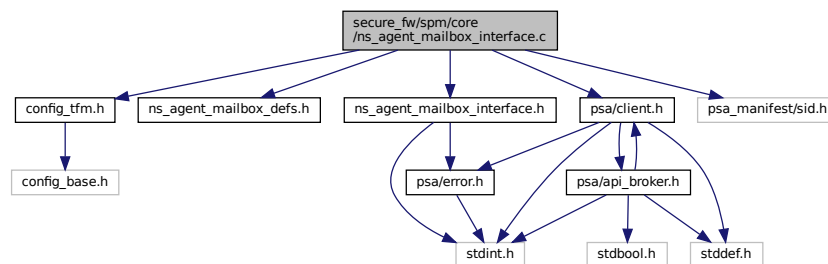
**8.398.2.4 REGION\_DECLARE() [4/4]**

```
REGION_DECLARE (
 Image$$,
 TFM_SP_LOAD_LIST ,
 $ RO)
```

**8.399 secure\_fw/spm/core/ns\_agent\_mailbox\_interface.c File Reference**

```
#include "config_tfm.h"
#include "ns_agent_mailbox_defs.h"
#include "ns_agent_mailbox_interface.h"
#include "psa/client.h"
#include "psa_manifest/sid.h"
```

Include dependency graph for ns\_agent\_mailbox\_interface.c:

**Functions**

- [psa\\_status\\_t ns\\_agent\\_boot\\_ns\\_core \(void\)](#)  
*Boot the non-secure core.*
- [psa\\_status\\_t ns\\_agent\\_mailbox\\_enable \(void\)](#)  
*Enable the mailbox.*
- [psa\\_status\\_t ns\\_agent\\_mailbox\\_disable \(void\)](#)  
*Disable the mailbox.*
- [psa\\_status\\_t ns\\_agent\\_ns\\_core\\_shutdown \(void\)](#)  
*Inform ns\_agent\_mailbox that the NS core has shut down.*

## 8.399.1 Function Documentation

### 8.399.1.1 ns\_agent\_boot\_ns\_core()

```
psa_status_t ns_agent_boot_ns_core (
 void)
```

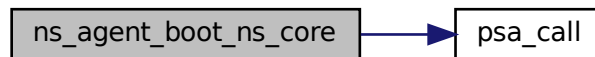
Boot the non-secure core.

#### Returns

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 16 of file ns\_agent\_mailbox\_interface.c.

Here is the call graph for this function:



### 8.399.1.2 ns\_agent\_mailbox\_disable()

```
psa_status_t ns_agent_mailbox_disable (
 void)
```

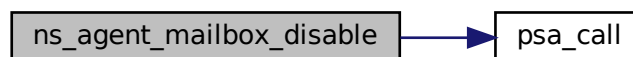
Disable the mailbox.

#### Returns

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 26 of file ns\_agent\_mailbox\_interface.c.

Here is the call graph for this function:



### 8.399.1.3 ns\_agent\_mailbox\_enable()

```
psa_status_t ns_agent_mailbox_enable (
 void)
```

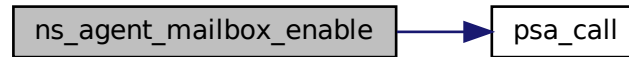
Enable the mailbox.

**Returns**

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 21 of file ns\_agent\_mailbox\_interface.c.

Here is the call graph for this function:

**8.399.1.4 ns\_agent\_ns\_core\_shutdown()**

```

psa_status_t ns_agent_ns_core_shutdown (
 void)

```

Inform ns\_agent\_mailbox that the NS core has shut down.

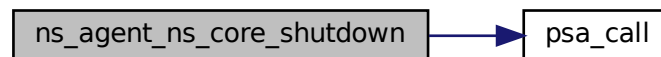
Before re-enabling the mailbox, [ns\\_agent\\_boot\\_ns\\_core\(\)](#) must be called.

**Returns**

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 31 of file ns\_agent\_mailbox\_interface.c.

Here is the call graph for this function:

**8.400 secure\_fw/spm/core/psa\_api.c File Reference**

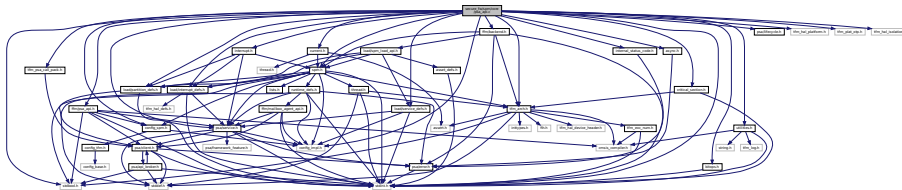
```

#include <stdbool.h>
#include <stdint.h>
#include "async.h"
#include "bitops.h"
#include "config_impl.h"
#include "config_spm.h"
#include "critical_section.h"
#include "current.h"
#include "internal_status_code.h"
#include "interrupt.h"
#include "psa/lifecycle.h"
#include "psa/service.h"
#include "spm.h"
#include "tfm_arch.h"

```

```
#include "load/partition_defs.h"
#include "load/service_defs.h"
#include "load/interrupt_defs.h"
#include "utilities.h"
#include "ffm/backend.h"
#include "ffm/psa_api.h"
#include "tfm_hal_platform.h"
#include "tfm_plat_otp.h"
#include "tfm_psa_call_pack.h"
#include "tfm_hal_isolation.h"
```

Include dependency graph for psa\_api.c:



## Functions

- [void spm\\_handle\\_programmer\\_errors](#) ([psa\\_status\\_t](#) status)  
*This function handles the specific programmer error cases.*
- [uint32\\_t tfm\\_spm\\_get\\_lifecycle\\_state](#) ([void](#))  
*This function get the current PSA RoT lifecycle state.*
- [psa\\_status\\_t tfm\\_spm\\_partition\\_psa\\_reply](#) ([psa\\_handle\\_t](#) msg\_handle, [psa\\_status\\_t](#) status)  
*Function body of [psa\\_reply](#).*
- [void tfm\\_spm\\_partition\\_psa\\_panic](#) ([void](#))  
*Function body of [psa\\_panic](#).*

## 8.400.1 Function Documentation

### 8.400.1.1 spm\_handle\_programmer\_errors()

```
void spm_handle_programmer_errors (
 psa_status_t status)
```

This function handles the specific programmer error cases.

#### Parameters

|    |        |                                   |
|----|--------|-----------------------------------|
| in | status | Standard error codes for the SPM. |
|----|--------|-----------------------------------|

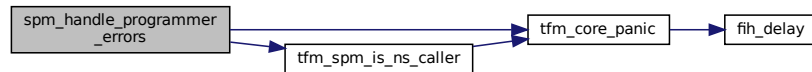
**Note**

This call will panic if the SP operation resulted in:

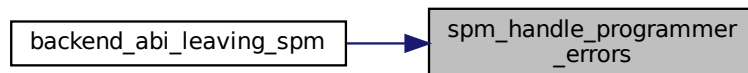
- PSA\_ERROR\_PROGRAMMER\_ERROR
- PSA\_ERROR\_CONNECTION\_REFUSED

Definition at line 35 of file psa\_api.c.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.400.1.2 tfm\_spm\_get\_lifecycle\_state()**

```
uint32_t tfm_spm_get_lifecycle_state (
 void)
```

This function get the current PSA RoT lifecycle state.

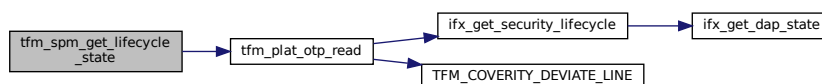
**Returns**

state The current security lifecycle state of the PSA RoT. The PSA state and implementation state are encoded as follows:

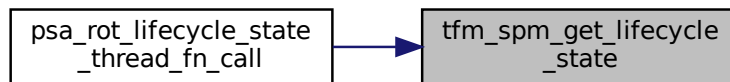
- state[15:8] – PSA lifecycle state
- state[7:0] – IMPLEMENTATION DEFINED state

Definition at line 45 of file psa\_api.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.400.1.3 tfm\_spm\_partition\_psa\_panic()

```
void tfm_spm_partition_psa_panic (
 void)
```

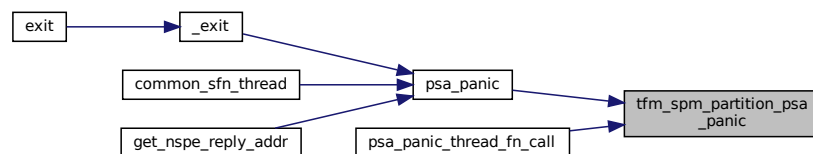
Function body of [psa\\_panic](#).

Return values

|                   |  |
|-------------------|--|
| Should not return |  |
|-------------------|--|

Definition at line 397 of file `psa_api.c`.

Here is the caller graph for this function:



#### 8.400.1.4 tfm\_spm\_partition\_psa\_reply()

```
psa_status_t tfm_spm_partition_psa_reply (
 psa_handle_t msg_handle,
 psa_status_t status)
```

Function body of [psa\\_reply](#).

Parameters

|    |                   |                                                    |
|----|-------------------|----------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                   |
| in | <i>status</i>     | Message result value to be reported to the client. |

Return values

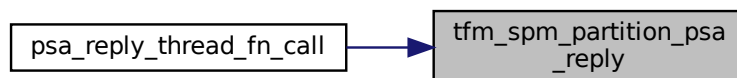
|                    |                                         |
|--------------------|-----------------------------------------|
| <i>Positive</i>    | integer Success, the connection handle. |
| <i>PSA_SUCCESS</i> | Success                                 |



## Return values

|                         |                                                                                                                                                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PROGRAMMER ERROR</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• msg_handle is invalid.</li> <li>• An invalid status code is specified for the type of message.</li> </ul> |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Definition at line 246 of file psa\_api.c.  
Here is the caller graph for this function:



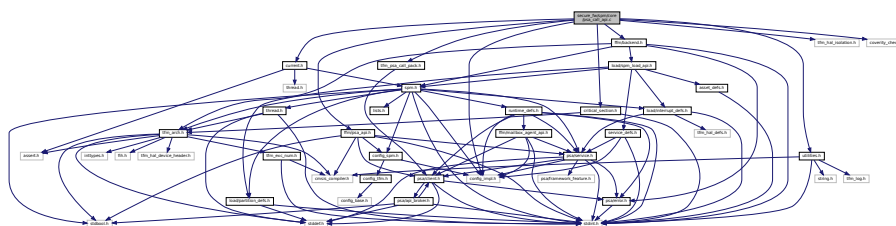
## 8.401 secure\_fw/spm/core/psa\_call\_api.c File Reference

```

#include "config_impl.h"
#include "critical_section.h"
#include "current.h"
#include "ffm/backend.h"
#include "ffm/psa_api.h"
#include "tfm_hal_isolation.h"
#include "tfm_psa_call_pack.h"
#include "utilities.h"
#include "coverity_check.h"

```

Include dependency graph for psa\_call\_api.c:



## Functions

- [psa\\_status\\_t spm\\_associate\\_call\\_params](#) (struct [connection\\_t](#) \*p\_connection, uint32\_t ctrl\_param, const [psa\\_invec](#) \*inptr, [psa\\_outvec](#) \*outptr)
- [psa\\_status\\_t tfm\\_spm\\_client\\_psa\\_call](#) ([psa\\_handle\\_t](#) handle, uint32\_t ctrl\_param, const [psa\\_invec](#) \*inptr, [psa\\_outvec](#) \*outptr)  
*handler for [psa\\_call](#).*

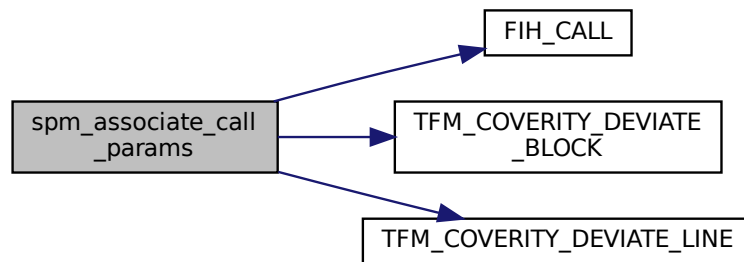
## 8.401.1 Function Documentation

### 8.401.1.1 spm\_associate\_call\_params()

```
psa_status_t spm_associate_call_params (
 struct connection_t * p_connection,
 uint32_t ctrl_param,
 const psa_invec * inptr,
 psa_outvec * outptr)
```

Definition at line 20 of file `psa_call_api.c`.

Here is the call graph for this function:



### 8.401.1.2 tfm\_spm\_client\_psa\_call()

```
psa_status_t tfm_spm_client_psa_call (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * inptr,
 psa_outvec * outptr)
```

handler for `psa_call`.

#### Parameters

|    |                   |                                                                                                                      |
|----|-------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>handle</i>     | Service handle to the established connection, <a href="#">psa_handle_t</a>                                           |
| in | <i>ctrl_param</i> | Parameters combined in <code>uint32_t</code> , includes request type, <code>in_num</code> and <code>out_num</code> . |
| in | <i>inptr</i>      | Array of input <a href="#">psa_invec</a> structures. <a href="#">psa_invec</a>                                       |
| in | <i>outptr</i>     | Array of output <a href="#">psa_outvec</a> structures. <a href="#">psa_outvec</a>                                    |

#### Return values

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>     | Success.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <i>Does not return</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• An invalid handle was passed.</li> <li>• The connection is already handling a request.</li> <li>• An invalid memory reference was provided.</li> <li>• <code>in_num + out_num &gt; PSA_MAX_IOVEC</code>.</li> <li>• The message is unrecognized by the RoT Service or incorrectly formatted.</li> </ul> |

## 8.402 secure\_fw/spm/core/psa\_connection\_api.c File Reference

[illegible]

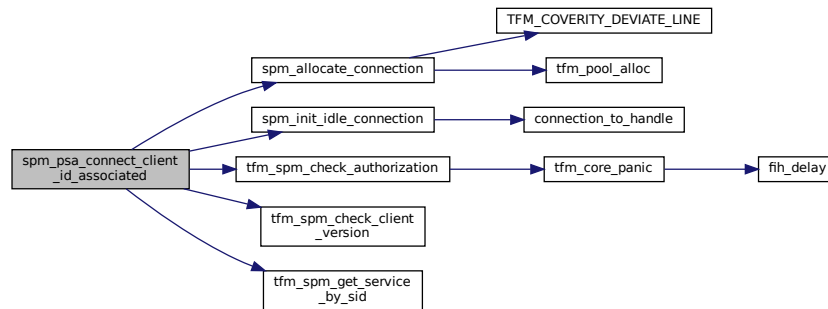
- `psa_status_t tfm_spm_client_psa_connect` (uint32\_t sid, uint32\_t version)
- `psa_status_t spm_psa_connect_client_id_associated` (struct `connection_t` \*\*p\_connection, uint32\_t sid, uint32\_t version, int32\_t client\_id)
- `psa_status_t tfm_spm_client_psa_close` (`psa_handle_t` handle)
- `psa_status_t spm_psa_close_client_id_associated` (`psa_handle_t` handle, int32\_t client\_id)
- `psa_status_t tfm_spm_partition_psa_set_rhandle` (`psa_handle_t` msg\_handle, void \*rhandle)

#### 8.402.1.1 spm\_psa\_close\_client\_id\_associated()

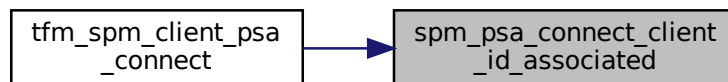
Definition at line 103 of file psa\_connection\_api.c.

Definition at line 37 of file psa\_connection\_api.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.402.1.3 tfm\_spm\_client\_psa\_close()

```
psa_status_t tfm_spm_client_psa_close (
 psa_handle_t handle)
```

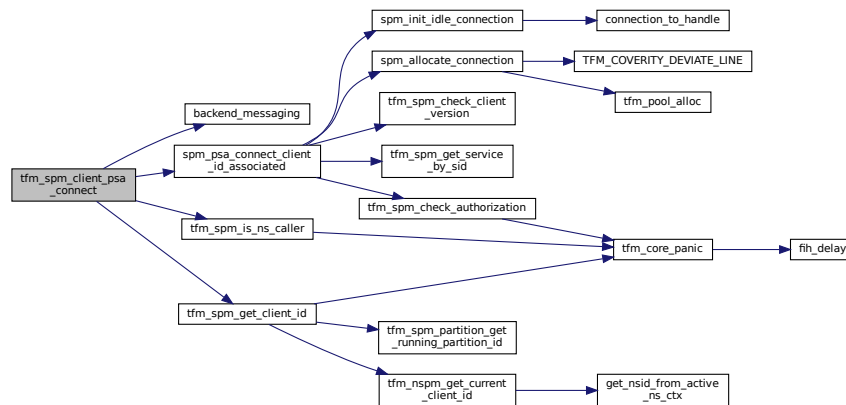
Definition at line 97 of file psa\_connection\_api.c.

### 8.402.1.4 tfm\_spm\_client\_psa\_connect()

```
psa_status_t tfm_spm_client_psa_connect (
 uint32_t sid,
 uint32_t version)
```

Definition at line 17 of file psa\_connection\_api.c.

Here is the call graph for this function:



#### 8.402.1.5 tfm\_spm\_partition\_psa\_set\_rhandle()

```

psa_status_t tfm_spm_partition_psa_set_rhandle (
 psa_handle_t msg_handle,
 void * rhandle)

```

Definition at line 136 of file psa\_connection\_api.c.

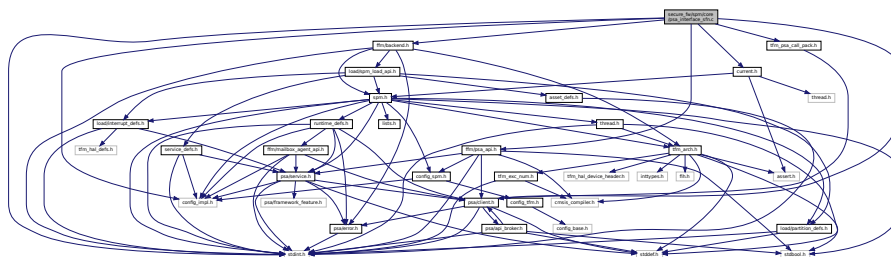
## 8.403 secure\_fw/spm/core/psa\_interface\_sfn.c File Reference

```

#include <stdint.h>
#include "config_impl.h"
#include "current.h"
#include "tfm_psa_call_pack.h"
#include "ffm/backend.h"
#include "ffm/psa_api.h"
#include "psa/client.h"

```

Include dependency graph for psa\_interface\_sfn.c:



## Functions

- uint32\_t [psa\\_framework\\_version](#) (void)  
Retrieve the version of the PSA Framework API that is implemented.
- uint32\_t [psa\\_version](#) (uint32\_t sid)  
Retrieve the version of an RoT Service or indicate that it is not present on this system.

- `psa_status_t tfm_psa_call_pack (psa_handle_t handle, uint32_t ctrl_param, const psa_invec *in_vec, psa_outvec *out_vec)`  
*Read a message parameter or part of a message parameter from a client input vector.*
- `size_t psa_read (psa_handle_t msg_handle, uint32_t invec_idx, void *buffer, size_t num_bytes)`  
*Skip over part of a client input vector.*
- `void psa_write (psa_handle_t msg_handle, uint32_t outvec_idx, const void *buffer, size_t num_bytes)`  
*Write a message response to a client output vector.*
- `void psa_panic (void)`  
*Terminate execution within the calling Secure Partition and will not return.*

## 8.403.1 Function Documentation

### 8.403.1.1 psa\_framework\_version()

```
uint32_t psa_framework_version (
 void)
```

Retrieve the version of the PSA Framework API that is implemented.

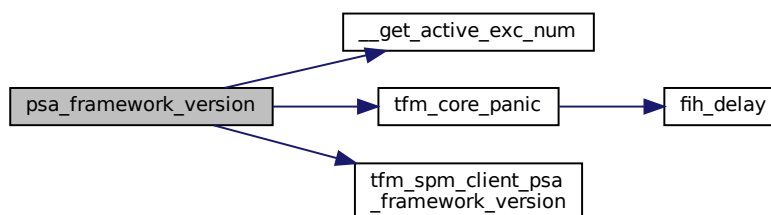
#### Returns

version The version of the PSA Framework implementation that is providing the runtime services to the caller. The major and minor version are encoded as follows:

- version[15:8] – major version number.
- version[7:0] – minor version number.

Definition at line 22 of file `psa_interface_sfn.c`.

Here is the call graph for this function:



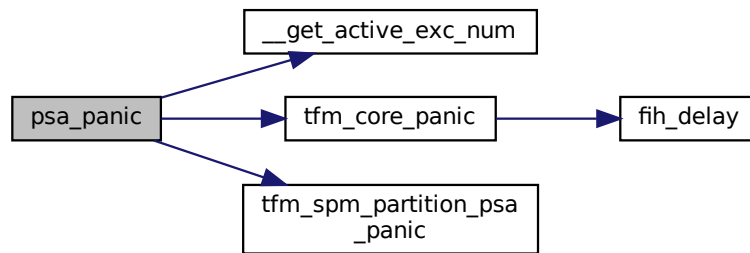
### 8.403.1.2 psa\_panic()

```
void psa_panic (
 void)
```

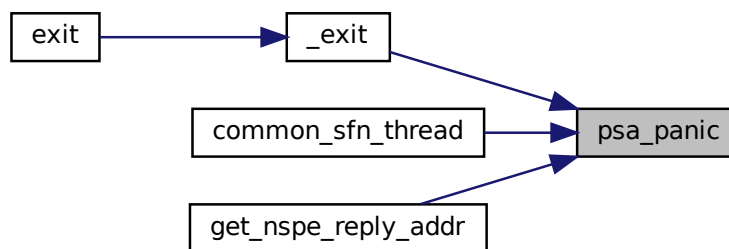
Terminate execution within the calling Secure Partition and will not return.

Definition at line 114 of file `psa_interface_sfn.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.403.1.3 psa\_read()

```

size_t psa_read (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 void * buffer,
 size_t num_bytes)

```

Read a message parameter or part of a message parameter from a client input vector.

#### Parameters

|     |                   |                                                                                           |
|-----|-------------------|-------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                          |
| in  | <i>invec_idx</i>  | Index of the input vector to read from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| out | <i>buffer</i>     | Buffer in the Secure Partition to copy the requested data to.                             |
| in  | <i>num_bytes</i>  | Maximum number of bytes to be read from the client input vector.                          |

#### Return values

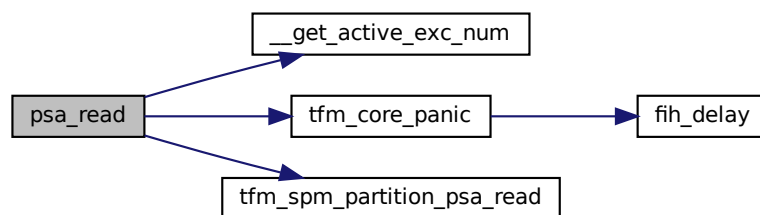
|              |                                                   |
|--------------|---------------------------------------------------|
| <i>&gt;0</i> | Number of bytes copied.                           |
| <i>0</i>     | There was no remaining data in this input vector. |

## Return values

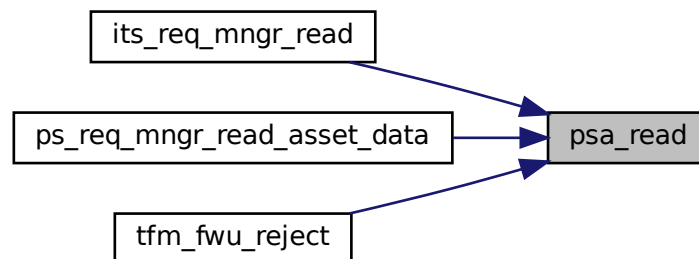
|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PROGRAMMER ERROR</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• <code>msg_handle</code> is invalid.</li> <li>• <code>msg_handle</code> does not refer to a <a href="#">PSA_IPC_CALL</a> message.</li> <li>• <code>invec_idx</code> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> <li>• the memory reference for <code>buffer</code> is invalid or not writable.</li> </ul> |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Definition at line 80 of file `psa_interface_sfn.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.403.1.4 psa\_skip()

```

size_t psa_skip (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 size_t num_bytes)

```

Skip over part of a client input vector.



## Parameters

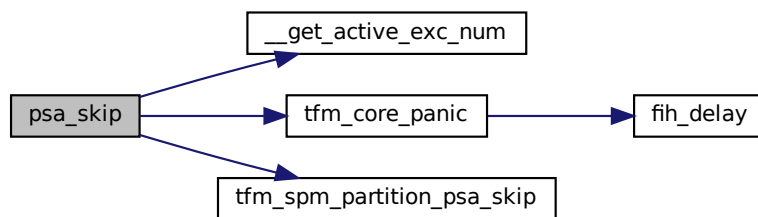
|    |                   |                                                                                       |
|----|-------------------|---------------------------------------------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                                                      |
| in | <i>invec_idx</i>  | Index of input vector to skip from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in | <i>num_bytes</i>  | Maximum number of bytes to skip in the client input vector.                           |

## Return values

|                         |                                                                                                                                                                                                                                                                                                        |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| >0                      | Number of bytes skipped.                                                                                                                                                                                                                                                                               |
| 0                       | There was no remaining data in this input vector.                                                                                                                                                                                                                                                      |
| <i>PROGRAMMER ERROR</i> | The call is invalid, one or more of the following are true: <ul style="list-style-type: none"> <li>• <i>msg_handle</i> is invalid.</li> <li>• <i>msg_handle</i> does not refer to a request message.</li> <li>• <i>invec_idx</i> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> </ul> |

Definition at line 92 of file `psa_interface_sfn.c`.

Here is the call graph for this function:

8.403.1.5 `psa_version()`

```
uint32_t psa_version (
 uint32_t sid)
```

Retrieve the version of an RoT Service or indicate that it is not present on this system.

## Parameters

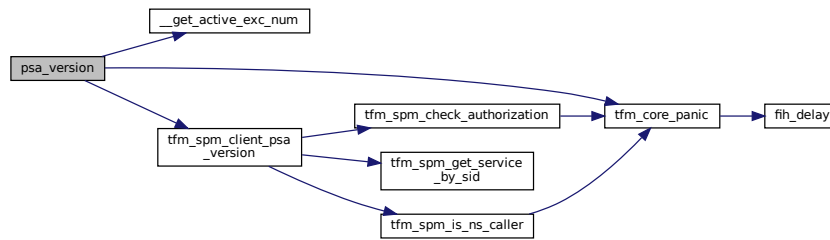
|    |            |                                 |
|----|------------|---------------------------------|
| in | <i>sid</i> | ID of the RoT Service to query. |
|----|------------|---------------------------------|

## Return values

|                         |                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------|
| <i>PSA_VERSION_NONE</i> | The RoT Service is not implemented, or the caller is not permitted to access the service. |
| >                       | 0 The version of the implemented RoT Service.                                             |

Definition at line 32 of file `psa_interface_sfn.c`.

Here is the call graph for this function:



### 8.403.1.6 psa\_write()

```

void psa_write (
 psa_handle_t msg_handle,
 uint32_t outvec_idx,
 const void * buffer,
 size_t num_bytes)

```

Write a message response to a client output vector.

#### Parameters

|     |                   |                                                                                                  |
|-----|-------------------|--------------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                                 |
| out | <i>outvec_idx</i> | Index of output vector in message to write to. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in  | <i>buffer</i>     | Buffer with the data to write.                                                                   |
| in  | <i>num_bytes</i>  | Number of bytes to write to the client output vector.                                            |

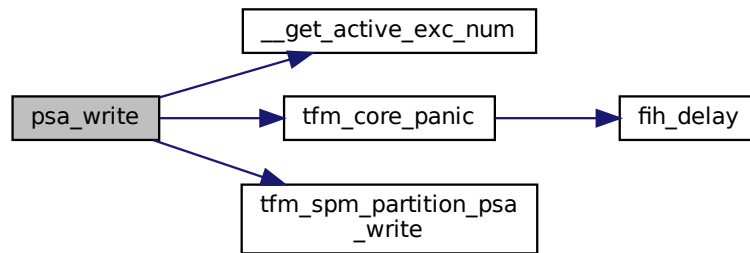
#### Note

The call is a "PROGRAMMER ERROR" if one or more of the following occur:

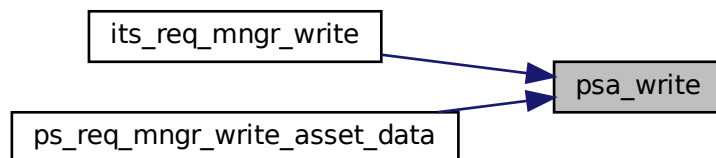
- *msg\_handle* is invalid.
- *msg\_handle* does not refer to a request message.
- *outvec\_idx* is equal to or greater than [PSA\\_MAX\\_IOVEC](#).
- the memory reference for *buffer* is invalid.
- the call attempts to write data past the end of the client output vector.

Definition at line 103 of file `psa_interface_sfn.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.403.1.7 tfm\_psa\_call\_pack()

```

psa_status_t tfm_psa_call_pack (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * in_vec,
 psa_outvec * out_vec)

```

Definition at line 53 of file `psa_interface_sfn.c`.

## 8.404 secure\_fw/spm/core/psa\_interface\_svc.c File Reference

```

#include <stdint.h>
#include "config_spm.h"
#include "runtime_defs.h"
#include "svc_num.h"
#include "tfm_psa_call_pack.h"
#include "utilities.h"
#include "psa/client.h"
#include "psa/lifecycle.h"
#include "psa/service.h"
#include "coverity_check.h"
#include "compiler_ext_defs.h"

```



#### 8.404.1.4 psa\_read\_svc()

```
__naked size_t psa_read_svc (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 void * buffer,
 size_t num_bytes)
```

Definition at line 52 of file psa\_interface\_svc.c.

#### 8.404.1.5 psa\_reply\_svc()

```
__naked void psa_reply_svc (
 psa_handle_t msg_handle,
 psa_status_t retval)
```

Definition at line 73 of file psa\_interface\_svc.c.

#### 8.404.1.6 psa\_rot\_lifecycle\_state\_svc()

```
__naked uint32_t psa_rot_lifecycle_state_svc (
 void)
```

Definition at line 107 of file psa\_interface\_svc.c.

#### 8.404.1.7 psa\_skip\_svc()

```
__naked size_t psa_skip_svc (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 size_t num_bytes)
```

Definition at line 59 of file psa\_interface\_svc.c.

#### 8.404.1.8 psa\_version\_svc()

```
__naked uint32_t psa_version_svc (
 uint32_t sid)
```

Definition at line 25 of file psa\_interface\_svc.c.

#### 8.404.1.9 psa\_wait\_svc()

```
__naked psa_signal_t psa_wait_svc (
 psa_signal_t signal_mask,
 uint32_t timeout)
```

Definition at line 40 of file psa\_interface\_svc.c.

#### 8.404.1.10 psa\_write\_svc()

```
__naked void psa_write_svc (
 psa_handle_t msg_handle,
 uint32_t outvec_idx,
 const void * buffer,
 size_t num_bytes)
```

Definition at line 66 of file psa\_interface\_svc.c.



- `__naked size_t psa_read_thread_fn_call (psa_handle_t msg_handle, uint32_t invec_idx, void *buffer, size_t num_bytes)`
- `__naked size_t psa_skip_thread_fn_call (psa_handle_t msg_handle, uint32_t invec_idx, size_t num_bytes)`
- `__naked void psa_write_thread_fn_call (psa_handle_t msg_handle, uint32_t outvec_idx, const void *buffer, size_t num_bytes)`
- `__naked void psa_reply_thread_fn_call (psa_handle_t msg_handle, psa_status_t status)`
- `__naked __NO_RETURN void psa_panic_thread_fn_call (void)`
- `__naked uint32_t psa_rot_lifecycle_state_thread_fn_call (void)`

## Variables

- `const struct psa_api_tbl_t psa_api_thread_fn_call`

## 8.405.1 Macro Definition Documentation

### 8.405.1.1 TFM\_THREAD\_FN\_CALL\_ENTRY

```
#define TFM_THREAD_FN_CALL_ENTRY(
 target_psa_api)
```

Value:

```
__asm volatile(
 SYNTAX_UNIFIED
 "push {r4, lr} \n" \
 "ldr r4, =M2S(target_psa_api) \n" \
 "mov r12, r4 \n" \
 "bl tfm_arch_thread_fn_call \n" \
 "pop {r4, pc} \n" \
)
```

Definition at line 28 of file `psa_interface_thread_fn_call.c`.

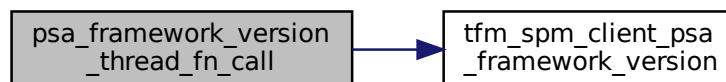
## 8.405.2 Function Documentation

### 8.405.2.1 psa\_framework\_version\_thread\_fn\_call()

```
__naked uint32_t psa_framework_version_thread_fn_call (
 void)
```

Definition at line 39 of file `psa_interface_thread_fn_call.c`.

Here is the call graph for this function:



### 8.405.2.2 psa\_get\_thread\_fn\_call()

```
__naked psa_status_t psa_get_thread_fn_call (
 psa_signal_t signal,
 psa_msg_t * msg)
```

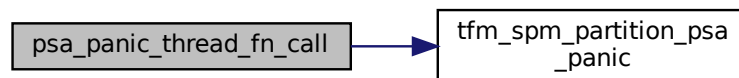
Definition at line 66 of file `psa_interface_thread_fn_call.c`.

#### 8.405.2.3 `psa_panic_thread_fn_call()`

```
__naked __NO_RETURN void psa_panic_thread_fn_call (
 void)
```

Definition at line 117 of file `psa_interface_thread_fn_call.c`.

Here is the call graph for this function:



#### 8.405.2.4 `psa_read_thread_fn_call()`

```
__naked size_t psa_read_thread_fn_call (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 void * buffer,
 size_t num_bytes)
```

Definition at line 72 of file `psa_interface_thread_fn_call.c`.

Here is the call graph for this function:



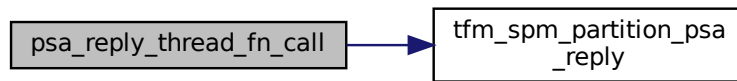
#### 8.405.2.5 `psa_reply_thread_fn_call()`

```
__naked void psa_reply_thread_fn_call (
 psa_handle_t msg_handle,
 psa_status_t status)
```

Definition at line 93 of file `psa_interface_thread_fn_call.c`.



Here is the call graph for this function:



#### 8.405.2.6 psa\_rot\_lifecycle\_state\_thread\_fn\_call()

```
__naked uint32_t psa_rot_lifecycle_state_thread_fn_call (
 void)
```

Definition at line 126 of file `psa_interface_thread_fn_call.c`.

Here is the call graph for this function:



#### 8.405.2.7 psa\_skip\_thread\_fn\_call()

```
__naked size_t psa_skip_thread_fn_call (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 size_t num_bytes)
```

Definition at line 79 of file `psa_interface_thread_fn_call.c`.

Here is the call graph for this function:

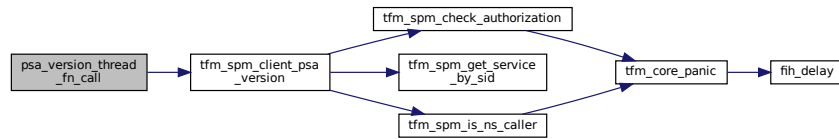


#### 8.405.2.8 psa\_version\_thread\_fn\_call()

```
__naked uint32_t psa_version_thread_fn_call (
 uint32_t sid)
```

Definition at line 45 of file `psa_interface_thread_fn_call.c`.

Here is the call graph for this function:



#### 8.405.2.9 psa\_wait\_thread\_fn\_call()

```

__naked psa_signal_t psa_wait_thread_fn_call (
 psa_signal_t signal_mask,
 uint32_t timeout)

```

Definition at line 60 of file `psa_interface_thread_fn_call.c`.

#### 8.405.2.10 psa\_write\_thread\_fn\_call()

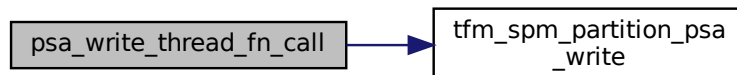
```

__naked void psa_write_thread_fn_call (
 psa_handle_t msg_handle,
 uint32_t outvec_idx,
 const void * buffer,
 size_t num_bytes)

```

Definition at line 86 of file `psa_interface_thread_fn_call.c`.

Here is the call graph for this function:



#### 8.405.2.11 tfm\_psa\_call\_pack\_thread\_fn\_call()

```

__naked psa_status_t tfm_psa_call_pack_thread_fn_call (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * in_vec,
 psa_outvec * out_vec)

```

Definition at line 51 of file `psa_interface_thread_fn_call.c`.

### 8.405.3 Variable Documentation

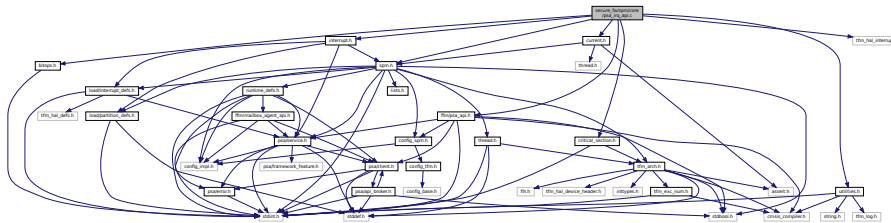
### 8.405.3.1 psa\_api\_thread\_fn\_call

const struct psa\_api\_tbl\_t psa\_api\_thread\_fn\_call  
 Definition at line 242 of file psa\_interface\_thread\_fn\_call.c.

## 8.406 secure\_fw/spm/core/psa\_irq\_api.c File Reference

```
#include "bitops.h"
#include "critical_section.h"
#include "current.h"
#include "ffm/psa_api.h"
#include "interrupt.h"
#include "spm.h"
#include "tfm_hal_interrupt.h"
#include "utilities.h"
```

Include dependency graph for psa\_irq\_api.c:



## Functions

- [psa\\_status\\_t tfm\\_spm\\_partition\\_psa\\_irq\\_enable \(psa\\_signal\\_t irq\\_signal\)](#)
- [psa\\_irq\\_status\\_t tfm\\_spm\\_partition\\_psa\\_irq\\_disable \(psa\\_signal\\_t irq\\_signal\)](#)

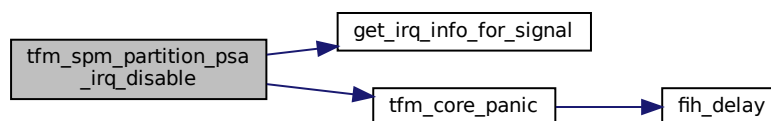
### 8.406.1 Function Documentation

#### 8.406.1.1 tfm\_spm\_partition\_psa\_irq\_disable()

```
psa_irq_status_t tfm_spm_partition_psa_irq_disable (
 psa_signal_t irq_signal)
```

Definition at line 34 of file `psa_irq_api.c`.

Here is the call graph for this function:

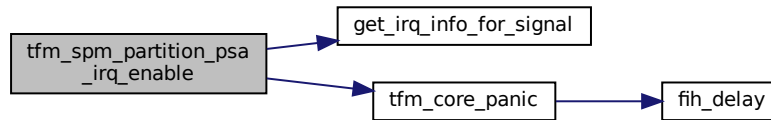


### 8.406.1.2 tfm\_spm\_partition\_psa\_irq\_enable()

```
psa_status_t tfm_spm_partition_psa_irq_enable (
 psa_signal_t irq_signal)
```

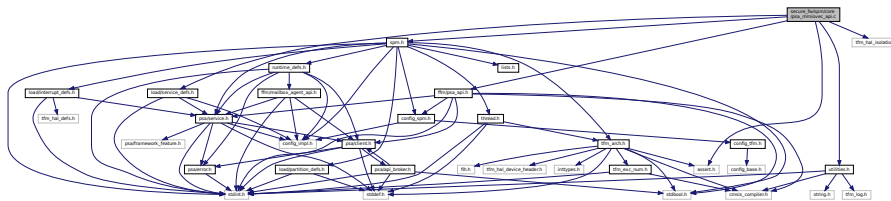
Definition at line 17 of file psa\_irq\_api.c.

Here is the call graph for this function:



## 8.407 secure\_fw/spm/core/psa\_mmiovec\_api.c File Reference

```
#include <assert.h>
#include "ffm/psa_api.h"
#include "spm.h"
#include "load/service_defs.h"
#include "utilities.h"
#include "tfm_hal_isolation.h"
Include dependency graph for psa_mmiovec_api.c:
```



## Functions

- `const void * tfm_spm_partition_psa_map_invec (psa_handle_t msg_handle, uint32_t invec_idx)`
- `void tfm_spm_partition_psa_unmap_invec (psa_handle_t msg_handle, uint32_t invec_idx)`
- `void * tfm_spm_partition_psa_map_outvec (psa_handle_t msg_handle, uint32_t outvec_idx)`
- `void tfm_spm_partition_psa_unmap_outvec (psa_handle_t msg_handle, uint32_t outvec_idx, size_t len)`

## 8.407.1 Function Documentation

### 8.407.1.1 tfm\_spm\_partition\_psa\_map\_invec()

```
const void* tfm_spm_partition_psa_map_invec (
 psa_handle_t msg_handle,
 uint32_t invec_idx)
```

Definition at line 16 of file `psa_mmiovec_api.c`.

**8.407.1.2 tfm\_spm\_partition\_psa\_map\_outvec()**

```
void* tfm_spm_partition_psa_map_outvec (
 psa_handle_t msg_handle,
 uint32_t outvec_idx)
```

Definition at line 147 of file `psa_mmiovec_api.c`.

**8.407.1.3 tfm\_spm\_partition\_psa\_unmap\_invec()**

```
void tfm_spm_partition_psa_unmap_invec (
 psa_handle_t msg_handle,
 uint32_t invec_idx)
```

Definition at line 93 of file `psa_mmiovec_api.c`.

**8.407.1.4 tfm\_spm\_partition\_psa\_unmap\_outvec()**

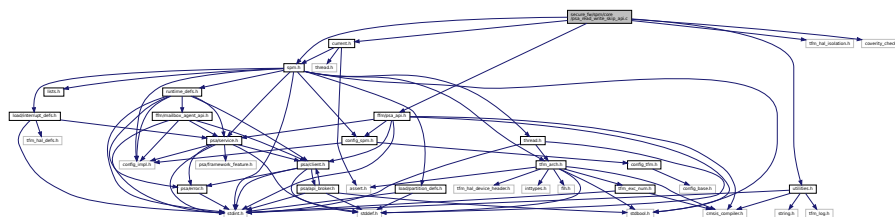
```
void tfm_spm_partition_psa_unmap_outvec (
 psa_handle_t msg_handle,
 uint32_t outvec_idx,
 size_t len)
```

Definition at line 222 of file `psa_mmiovec_api.c`.

**8.408 secure\_fw/spm/core/psa\_read\_write\_skip\_api.c File Reference**

```
#include "current.h"
#include "ffm/psa_api.h"
#include "spm.h"
#include "utilities.h"
#include "tfm_hal_isolation.h"
#include "coverity_check.h"
```

Include dependency graph for `psa_read_write_skip_api.c`:

**Functions**

- `size_t tfm_spm_partition_psa_read (psa_handle_t msg_handle, uint32_t invec_idx, void *buffer, size_t num_bytes)`  
Function body of `psa_read`.
- `size_t tfm_spm_partition_psa_skip (psa_handle_t msg_handle, uint32_t invec_idx, size_t num_bytes)`  
Function body of `psa_skip`.
- `psa_status_t tfm_spm_partition_psa_write (psa_handle_t msg_handle, uint32_t outvec_idx, const void *buffer, size_t num_bytes)`  
Function body of `psa_write`.

**8.408.1 Function Documentation**

### 8.408.1.1 tfm\_spm\_partition\_psa\_read()

```
size_t tfm_spm_partition_psa_read (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 void * buffer,
 size_t num_bytes)
```

Function body of [psa\\_read](#).

#### Parameters

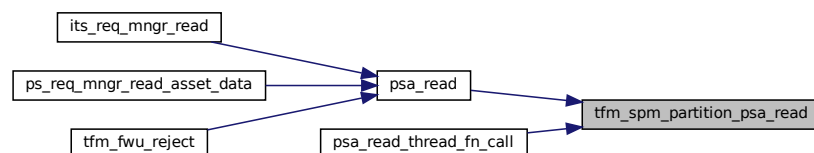
|     |                   |                                                                                           |
|-----|-------------------|-------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                          |
| in  | <i>invec_idx</i>  | Index of the input vector to read from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| out | <i>buffer</i>     | Buffer in the Secure Partition to copy the requested data to.                             |
| in  | <i>num_bytes</i>  | Maximum number of bytes to be read from the client input vector.                          |

#### Return values

|                         |                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $>0$                    | Number of bytes copied.                                                                                                                                                                                                                                                                                                                                                                                   |
| $0$                     | There was no remaining data in this input vector.                                                                                                                                                                                                                                                                                                                                                         |
| <i>PROGRAMMER ERROR</i> | The call is invalid, one or more of the following are true: <ul style="list-style-type: none"> <li>• <i>msg_handle</i> is invalid.</li> <li>• <i>msg_handle</i> does not refer to a <a href="#">PSA_IPC_CALL</a> message.</li> <li>• <i>invec_idx</i> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> <li>• the memory reference for <i>buffer</i> is invalid or not writable.</li> </ul> |

Definition at line 16 of file `psa_read_write_skip_api.c`.

Here is the caller graph for this function:



### 8.408.1.2 tfm\_spm\_partition\_psa\_skip()

```
size_t tfm_spm_partition_psa_skip (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 size_t num_bytes)
```

Function body of `psa_skip`.

#### Parameters

|    |                   |                                                                                       |
|----|-------------------|---------------------------------------------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                                                      |
| in | <i>invec_idx</i>  | Index of input vector to skip from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |

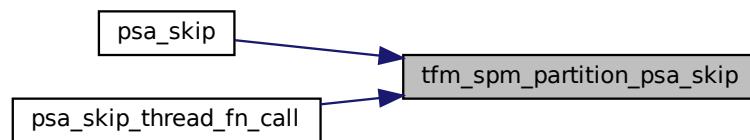
## Parameters

|    |                  |                                                             |
|----|------------------|-------------------------------------------------------------|
| in | <i>num_bytes</i> | Maximum number of bytes to skip in the client input vector. |
|----|------------------|-------------------------------------------------------------|

## Return values

|                         |                                                                                                                                                                                                                                                                                                        |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $> 0$                   | Number of bytes skipped.                                                                                                                                                                                                                                                                               |
| $0$                     | There was no remaining data in this input vector.                                                                                                                                                                                                                                                      |
| <i>PROGRAMMER ERROR</i> | The call is invalid, one or more of the following are true: <ul style="list-style-type: none"> <li>• <i>msg_handle</i> is invalid.</li> <li>• <i>msg_handle</i> does not refer to a request message.</li> <li>• <i>invec_idx</i> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> </ul> |

Definition at line 93 of file `psa_read_write_skip_api.c`.  
Here is the caller graph for this function:



## 8.408.1.3 tfm\_spm\_partition\_psa\_write()

```

psa_status_t tfm_spm_partition_psa_write (
 psa_handle_t msg_handle,
 uint32_t outvec_idx,
 const void * buffer,
 size_t num_bytes)

```

Function body of [psa\\_write](#).

## Parameters

|     |                   |                                                                                                  |
|-----|-------------------|--------------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                                 |
| out | <i>outvec_idx</i> | Index of output vector in message to write to. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in  | <i>buffer</i>     | Buffer with the data to write.                                                                   |
| in  | <i>num_bytes</i>  | Number of bytes to write to the client output vector.                                            |

## Return values

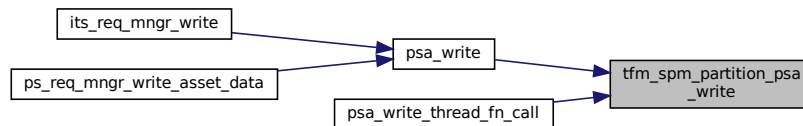
|                    |          |
|--------------------|----------|
| <i>PSA_SUCCESS</i> | Success. |
|--------------------|----------|

## Return values

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PROGRAMMER ERROR</b> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• <code>msg_handle</code> is invalid.</li> <li>• <code>msg_handle</code> does not refer to a request message.</li> <li>• <code>outvec_idx</code> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> <li>• The memory reference for buffer is invalid.</li> <li>• The call attempts to write data past the end of the client output vector.</li> </ul> |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Definition at line 159 of file `psa_read_write_skip_api.c`.

Here is the caller graph for this function:



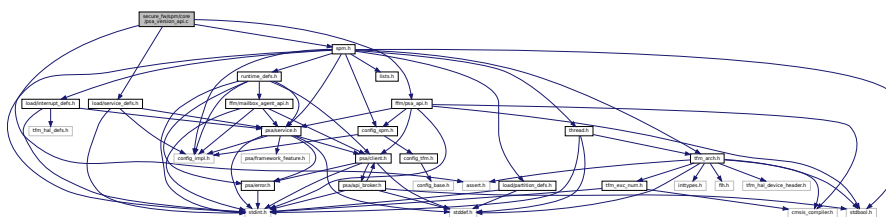
## 8.409 secure\_fw/spm/core/psa\_version\_api.c File Reference

```

#include <assert.h>
#include "ffm/psa_api.h"
#include "load/service_defs.h"
#include "spm.h"

```

Include dependency graph for `psa_version_api.c`:



## Functions

- `uint32_t tfm_spm_client_psa_framework_version (void)`  
*handler for [psa\\_framework\\_version](#).*
- `uint32_t tfm_spm_client_psa_version (uint32_t sid)`  
*handler for [psa\\_version](#).*

### 8.409.1 Function Documentation



**8.409.1.1 tfm\_spm\_client\_psa\_framework\_version()**

```
uint32_t tfm_spm_client_psa_framework_version (
 void)
```

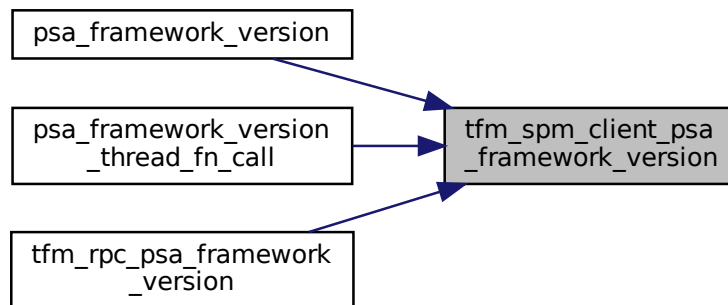
handler for [psa\\_framework\\_version](#).

**Returns**

version The version of the PSA Framework implementation that is providing the runtime services.

Definition at line 13 of file `psa_version_api.c`.

Here is the caller graph for this function:

**8.409.1.2 tfm\_spm\_client\_psa\_version()**

```
uint32_t tfm_spm_client_psa_version (
 uint32_t sid)
```

handler for [psa\\_version](#).

**Parameters**

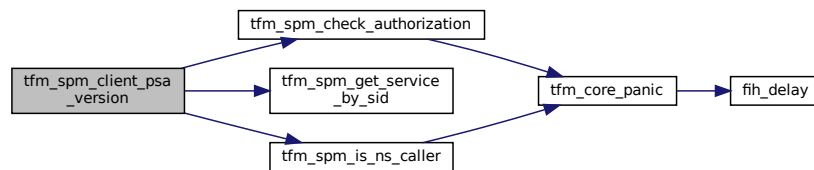
|    |            |                       |
|----|------------|-----------------------|
| in | <i>sid</i> | RoT Service identity. |
|----|------------|-----------------------|

**Return values**

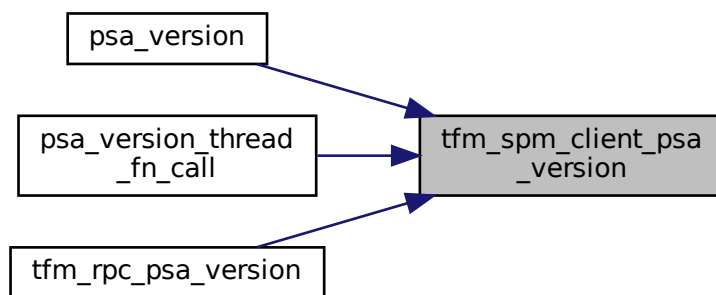
|                         |                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------|
| <i>PSA_VERSION_NONE</i> | The RoT Service is not implemented, or the caller is not permitted to access the service. |
| >                       | 0 The version of the implemented RoT Service.                                             |

Definition at line 18 of file `psa_version_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



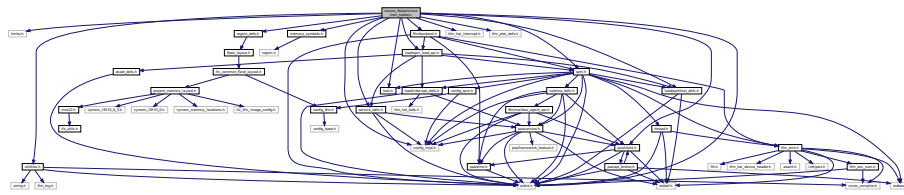
## 8.410 secure\_fw/spm/core/rom\_loader.c File Reference

```

#include <limits.h>
#include <stdint.h>
#include "config_impl.h"
#include "lists.h"
#include "memory_symbols.h"
#include "region_defs.h"
#include "spm.h"
#include "tfm_hal_interrupt.h"
#include "tfm_plat_defs.h"
#include "utilities.h"
#include "ffm/backend.h"
#include "load/partition_defs.h"
#include "load/spm_load_api.h"
#include "load/service_defs.h"
#include "psa/client.h"

```

Include dependency graph for rom\_loader.c:



## Functions

- struct [partition\\_t](#) \* [load\\_a\\_partition\\_assuredly](#) (struct [partition\\_head\\_t](#) \*head)
- uint32\_t [load\\_services\\_assuredly](#) (struct [partition\\_t](#) \*p\_partition, struct [service\\_head\\_t](#) \*services\_listhead, struct [service\\_t](#) \*\*stateless\_services\_ref\_tbl, size\_t ref\_tbl\_size)
- void [load\\_irqs\\_assuredly](#) (struct [partition\\_t](#) \*p\_partition)

### 8.410.1 Function Documentation

#### 8.410.1.1 load\_a\_partition\_assuredly()

```
struct partition_t* load_a_partition_assuredly (
 struct partition_head_t * head)
```

Definition at line 103 of file rom\_loader.c.

Here is the call graph for this function:

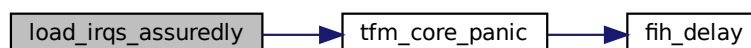


#### 8.410.1.2 load\_irqs\_assuredly()

```
void load_irqs_assuredly (
 struct partition_t * p_partition)
```

Definition at line 232 of file rom\_loader.c.

Here is the call graph for this function:





- #define `STATIC_HANDLE_VER_OFFSET` 8
- #define `STATIC_HANDLE_VER_MASK` (uint32\_t)((1UL << `STATIC_HANDLE_VER_BIT_WIDTH`) - 1)
- #define `GET_VERSION_FROM_STATIC_HANDLE`(handle) (uint32\_t)(((handle) >> `STATIC_HANDLE_VER_OFFSET`) & `STATIC_HANDLE_VER_MASK`)
- #define `STATIC_HANDLE_INDICATOR_OFFSET` 30
- #define `IS_STATIC_HANDLE`(handle) ((handle) & (1UL << `STATIC_HANDLE_INDICATOR_OFFSET`))
- #define `SPM_INVALID_PARTITION_IDX` (~0U)
- #define `GET_THRD_OWNER`(x) `TO_CONTAINER`(x, struct `partition_t`, thrd)
- #define `GET_CTX_OWNER`(x) `TO_CONTAINER`(x, struct `partition_t`, ctx\_ctrl)
- #define `TFM_CLIENT_ID_IS_NS`(client\_id) ((client\_id) < 0)
- #define `tfm_spm_is_rpc_msg`(x) (false)

## Enumerations

- enum `connection_status` {  
`TFM_HANDLE_STATUS_IDLE` = 0, `TFM_HANDLE_STATUS_ACTIVE` = 1, `TFM_HANDLE_STATUS_TO_FREE` = 2, `TFM_HANDLE_STATUS_MAX` = 3,  
`_TFM_HANDLE_STATUS_PAD` = `UINT32_MAX` }

## Functions

- `int32_t tfm_spm_partition_get_running_partition_id` (void)  
*Get the running partition ID.*
- `void spm_init_connection_space` (void)
- `struct connection_t * spm_allocate_connection` (void)
- `psa_status_t spm_validate_connection` (const struct `connection_t` \*p\_connection)
- `void spm_free_connection` (struct `connection_t` \*p\_connection)
- `const struct service_t * tfm_spm_get_service_by_sid` (uint32\_t sid)  
*Get the service context by service ID.*
- `psa_status_t spm_get_idle_connection` (struct `connection_t` \*\*p\_connection, `psa_handle_t` handle, int32\_t client\_id)  
*Convert the given user handle to an SPM recognised connection and verify that it is a valid idle connection that the caller is authorised to access.*
- `struct connection_t * spm_msg_handle_to_connection` (`psa_handle_t` msg\_handle)  
*Convert the given message handle to SPM recognised handle and verify it.*
- `void spm_init_idle_connection` (struct `connection_t` \*p\_connection, const struct `service_t` \*service, int32\_t client\_id)  
*Initialize connection, fill in with the input information and set to idle.*
- `psa_status_t spm_associate_call_params` (struct `connection_t` \*p\_connection, uint32\_t ctrl\_param, const `psa_invec` \*inptr, `psa_outvec` \*outptr)
- `int32_t tfm_spm_check_client_version` (const struct `service_t` \*service, uint32\_t version)  
*Check the client version according to version policy.*
- `int32_t tfm_spm_check_authorization` (uint32\_t sid, const struct `service_t` \*service, bool ns\_caller)  
*Check the client access authorization.*
- `bool tfm_spm_is_ns_caller` (void)  
*Get the ns\_caller info from runtime context.*
- `int32_t tfm_spm_get_client_id` (bool ns\_caller)  
*Get ID of current RoT Service client. This API ensures the caller gets a valid ID.*
- `uint64_t ipc_schedule` (uint32\_t exc\_return)
- `uint32_t tfm_spm_init` (void)  
*SPM initialization implementation.*
- `psa_handle_t connection_to_handle` (struct `connection_t` \*p\_connection)  
*Converts a handle instance into a corresponded user handle.*
- `struct connection_t * handle_to_connection` (`psa_handle_t` handle)  
*Converts a user handle into a corresponded handle instance.*

## 8.411.1 Macro Definition Documentation

### 8.411.1.1 CLIENT\_HANDLE\_VALUE\_MIN

```
#define CLIENT_HANDLE_VALUE_MIN 1
```

Definition at line 42 of file spm.h.

### 8.411.1.2 GET\_CTX\_OWNER

```
#define GET_CTX_OWNER(
 x) TO_CONTAINER(x, struct partition_t, ctx_ctrl)
```

Definition at line 70 of file spm.h.

### 8.411.1.3 GET\_INDEX\_FROM\_STATIC\_HANDLE

```
#define GET_INDEX_FROM_STATIC_HANDLE(
 handle) (uint32_t)((handle) & STATIC_HANDLE_IDX_MASK)
```

Definition at line 51 of file spm.h.

### 8.411.1.4 GET\_THRD\_OWNER

```
#define GET_THRD_OWNER(
 x) TO_CONTAINER(x, struct partition_t, thrd)
```

Definition at line 69 of file spm.h.

### 8.411.1.5 GET\_VERSION\_FROM\_STATIC\_HANDLE

```
#define GET_VERSION_FROM_STATIC_HANDLE(
 handle) (uint32_t)((handle) >> STATIC_HANDLE_VER_OFFSET) & STATIC_HANDLE_VER_MASK)
```

Definition at line 58 of file spm.h.

### 8.411.1.6 IS\_STATIC\_HANDLE

```
#define IS_STATIC_HANDLE(
 handle) ((handle) & (1UL << STATIC_HANDLE_INDICATOR_OFFSET))
```

Definition at line 63 of file spm.h.

### 8.411.1.7 PSA\_TIMEOUT\_MASK

```
#define PSA_TIMEOUT_MASK PSA_BLOCK
```

Definition at line 35 of file spm.h.

### 8.411.1.8 SPM\_INVALID\_PARTITION\_IDX

```
#define SPM_INVALID_PARTITION_IDX (~0U)
```

Definition at line 66 of file spm.h.

#### 8.411.1.9 STATIC\_HANDLE\_IDX\_BIT\_WIDTH

```
#define STATIC_HANDLE_IDX_BIT_WIDTH 5
```

Definition at line 48 of file spm.h.

#### 8.411.1.10 STATIC\_HANDLE\_IDX\_MASK

```
#define STATIC_HANDLE_IDX_MASK (uint32_t)((1UL << STATIC_HANDLE_IDX_BIT_WIDTH) - 1)
```

Definition at line 49 of file spm.h.

#### 8.411.1.11 STATIC\_HANDLE\_INDICATOR\_OFFSET

```
#define STATIC_HANDLE_INDICATOR_OFFSET 30
```

Definition at line 62 of file spm.h.

#### 8.411.1.12 STATIC\_HANDLE\_NUM\_LIMIT

```
#define STATIC_HANDLE_NUM_LIMIT 32
```

Definition at line 41 of file spm.h.

#### 8.411.1.13 STATIC\_HANDLE\_VER\_BIT\_WIDTH

```
#define STATIC_HANDLE_VER_BIT_WIDTH 8
```

Definition at line 54 of file spm.h.

#### 8.411.1.14 STATIC\_HANDLE\_VER\_MASK

```
#define STATIC_HANDLE_VER_MASK (uint32_t)((1UL << STATIC_HANDLE_VER_BIT_WIDTH) - 1)
```

Definition at line 56 of file spm.h.

#### 8.411.1.15 STATIC\_HANDLE\_VER\_OFFSET

```
#define STATIC_HANDLE_VER_OFFSET 8
```

Definition at line 55 of file spm.h.

#### 8.411.1.16 TFM\_CLIENT\_ID\_IS\_NS

```
#define TFM_CLIENT_ID_IS_NS(
 client_id) ((client_id) < 0)
```

Definition at line 73 of file spm.h.

#### 8.411.1.17 tfm\_spm\_is\_rpc\_msg

```
#define tfm_spm_is_rpc_msg(
 x) (false)
```

Definition at line 407 of file spm.h.

### 8.411.2 Enumeration Type Documentation

### 8.411.2.1 connection\_status

enum `connection_status`

Enumerator

|                           |  |
|---------------------------|--|
| TFM_HANDLE_STATUS_IDLE    |  |
| TFM_HANDLE_STATUS_ACTIVE  |  |
| TFM_HANDLE_STATUS_TO_FREE |  |
| TFM_HANDLE_STATUS_MAX     |  |
| _TFM_HANDLE_STATUS_PAD    |  |

Definition at line 25 of file `spm.h`.

## 8.411.3 Function Documentation

### 8.411.3.1 connection\_to\_handle()

```
psa_handle_t connection_to_handle (
 struct connection_t * p_connection)
```

Converts a handle instance into a corresponded user handle.

Definition at line 64 of file `spm_connection_pool.c`.

Here is the caller graph for this function:



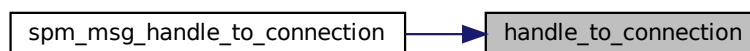
### 8.411.3.2 handle\_to\_connection()

```
struct connection_t* handle_to_connection (
 psa_handle_t handle)
```

Converts a user handle into a corresponded handle instance.

Definition at line 94 of file `spm_connection_pool.c`.

Here is the caller graph for this function:



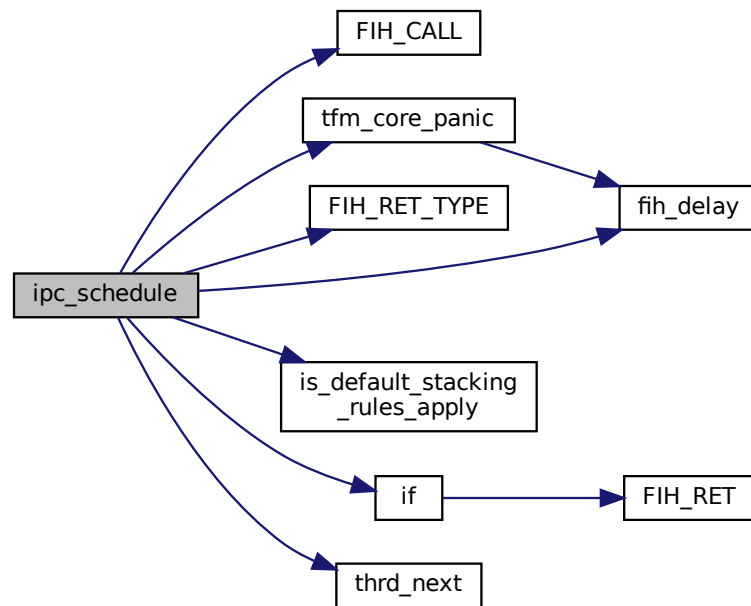
### 8.411.3.3 ipc\_schedule()

```
uint64_t ipc_schedule (
 uint32_t exc_return)
```

Definition at line 556 of file `backend_ipc.c`.



Here is the call graph for this function:

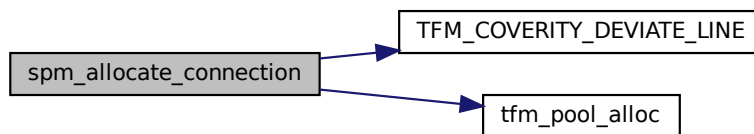


#### 8.411.3.4 `spm_allocate_connection()`

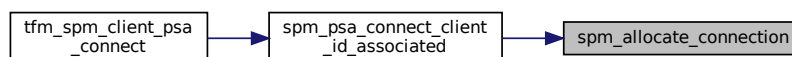
```
struct connection_t* spm_allocate_connection (
 void)
```

Definition at line 122 of file `spm_connection_pool.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

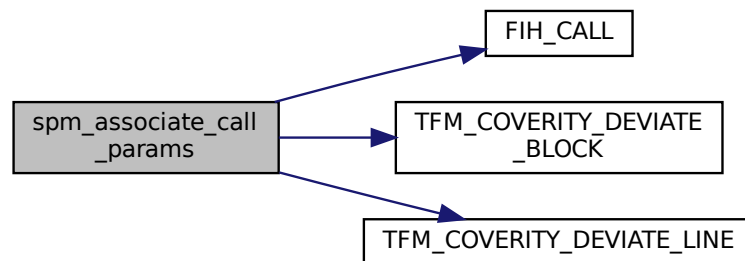


### 8.411.3.5 spm\_associate\_call\_params()

```
psa_status_t spm_associate_call_params (
 struct connection_t * p_connection,
 uint32_t ctrl_param,
 const psa_invec * inptr,
 psa_outvec * outptr)
```

Definition at line 20 of file psa\_call\_api.c.

Here is the call graph for this function:

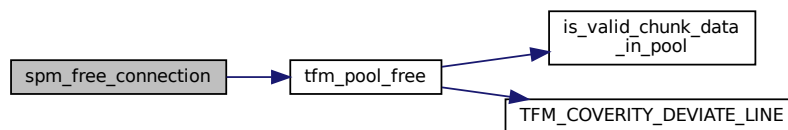


### 8.411.3.6 spm\_free\_connection()

```
void spm_free_connection (
 struct connection_t * p_connection)
```

Definition at line 139 of file spm\_connection\_pool.c.

Here is the call graph for this function:



### 8.411.3.7 spm\_get\_idle\_connection()

```
psa_status_t spm_get_idle_connection (
 struct connection_t ** p_connection,
 psa_handle_t handle,
 int32_t client_id)
```

Convert the given user handle to an SPM recognised connection and verify that it is a valid idle connection that the caller is authorised to access.

## Parameters

|     |                     |                                                                                                                             |
|-----|---------------------|-----------------------------------------------------------------------------------------------------------------------------|
| out | <i>p_connection</i> | The address of connection pointer to be converted from the given user handle.                                               |
| in  | <i>handle</i>       | Either a static handle or a handle to an established connection that was returned by a prior <code>psa_connect</code> call. |
| in  | <i>client_id</i>    | The client ID of the caller.                                                                                                |

## Return values

|                                     |                                                                                                                |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>                  | Success.                                                                                                       |
| <i>PSA_ERROR_CONNECTION_REFUSED</i> | The SPM or RoT Service has refused the connection.                                                             |
| <i>PSA_ERROR_CONNECTION_BUSY</i>    | The SPM or RoT Service cannot make the connection at the moment.                                               |
| <i>PSA_ERROR_PROGRAMMER_ERROR</i>   | The handle is invalid, the caller is not authorised to use it or the connection is already handling a request. |

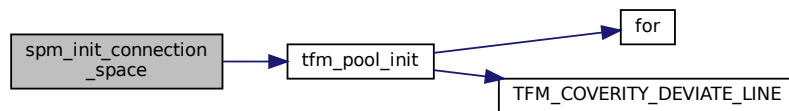
Definition at line 206 of file `spm_ipc.c`.

8.411.3.8 `spm_init_connection_space()`

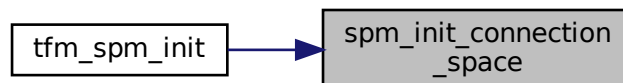
```
void spm_init_connection_space (
 void)
```

Definition at line 112 of file `spm_connection_pool.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

8.411.3.9 `spm_init_idle_connection()`

```
void spm_init_idle_connection (
 struct connection_t * p_connection,
 const struct service_t * service,
 int32_t client_id)
```

Initialize connection, fill in with the input information and set to idle.

#### Parameters

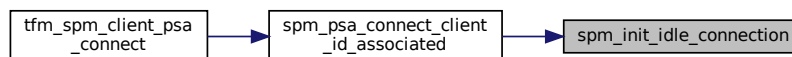
|    |                     |                                                                                         |
|----|---------------------|-----------------------------------------------------------------------------------------|
| in | <i>p_connection</i> | The 'p_connection' to initialize and fill information in.                               |
| in | <i>service</i>      | Target service context pointer, which can be obtained by partition management functions |
| in | <i>client_id</i>    | Partition ID of the sender of the message                                               |

Definition at line 329 of file spm\_ipc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.411.3.10 spm\_msg\_handle\_to\_connection()

```
struct connection_t* spm_msg_handle_to_connection (
 psa_handle_t msg_handle)
```

Convert the given message handle to SPM recognised handle and verify it.

#### Parameters

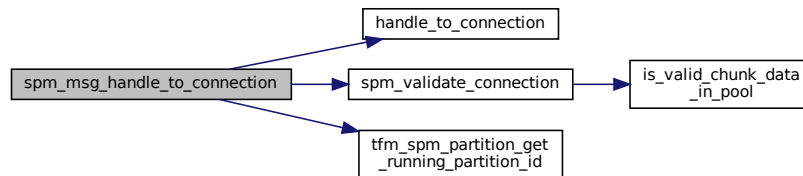
|    |                   |                                                                                 |
|----|-------------------|---------------------------------------------------------------------------------|
| in | <i>msg_handle</i> | Message handle which is a reference generated by the SPM to a specific message. |
|----|-------------------|---------------------------------------------------------------------------------|

## Returns

A SPM recognised handle or NULL. It is NULL when verification of the converted SPM handle fails.  
[connection\\_t](#) structures

Definition at line 302 of file `spm_ipc.c`.

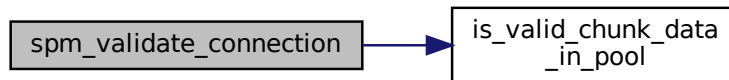
Here is the call graph for this function:

8.411.3.11 `spm_validate_connection()`

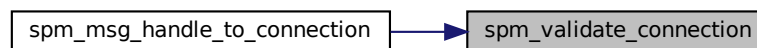
```
psa_status_t spm_validate_connection (
 const struct connection_t * p_connection)
```

Definition at line 128 of file `spm_connection_pool.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

8.411.3.12 `tfm_spm_check_authorization()`

```
int32_t tfm_spm_check_authorization (
 uint32_t sid,
 const struct service_t * service,
 bool ns_caller)
```

Check the client access authorization.

**Parameters**

|    |                  |                                                                                    |
|----|------------------|------------------------------------------------------------------------------------|
| in | <i>sid</i>       | Target RoT Service identity                                                        |
| in | <i>service</i>   | Target service context pointer, which can be get by partition management functions |
| in | <i>ns_caller</i> | Whether from NS caller                                                             |

**Return values**

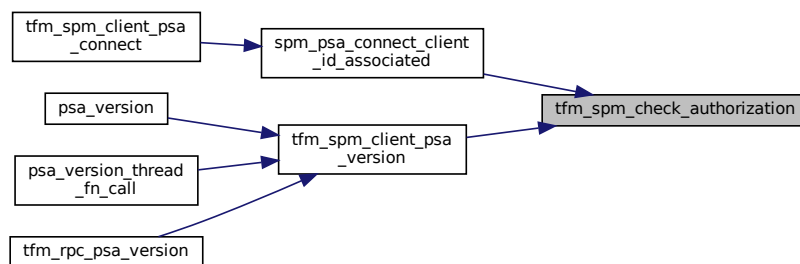
|                          |                            |
|--------------------------|----------------------------|
| <i>PSA_SUCCESS</i>       | Success                    |
| <i>SPM_ERROR_GENERIC</i> | Authorization check failed |

Definition at line 171 of file `spm_ipc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.411.3.13 tfm\_spm\_check\_client\_version()**

```

int32_t tfm_spm_check_client_version (
 const struct service_t * service,
 uint32_t version)

```

Check the client version according to version policy.

**Parameters**

|    |                |                                                                                    |
|----|----------------|------------------------------------------------------------------------------------|
| in | <i>service</i> | Target service context pointer, which can be get by partition management functions |
| in | <i>version</i> | Client support version                                                             |

**Return values**

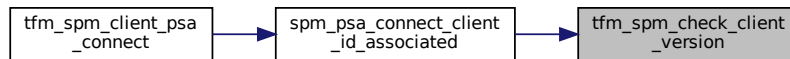
|                    |         |
|--------------------|---------|
| <i>PSA_SUCCESS</i> | Success |
|--------------------|---------|

## Return values

|                                       |                      |
|---------------------------------------|----------------------|
| <code>SPM_ERROR_BAD_PARAMETERS</code> | Bad parameters input |
| <code>SPM_ERROR_VERSION</code>        | Check failed         |

Definition at line 149 of file `spm_ipc.c`.

Here is the caller graph for this function:

8.411.3.14 `tfm_spm_get_client_id()`

```
int32_t tfm_spm_get_client_id (
 bool ns_caller)
```

Get ID of current RoT Service client. This API ensures the caller gets a valid ID.

## Parameters

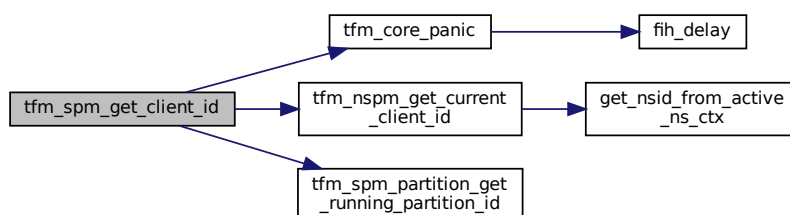
|    |                        |                                     |
|----|------------------------|-------------------------------------|
| in | <code>ns_caller</code> | If the client is Non-Secure or not. |
|----|------------------------|-------------------------------------|

## Return values

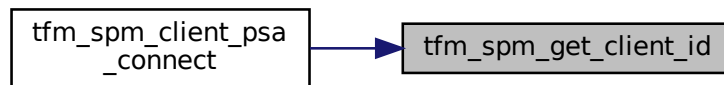
|     |           |
|-----|-----------|
| The | client ID |
|-----|-----------|

Definition at line 378 of file `spm_ipc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.411.3.15 tfm\_spm\_get\_service\_by\_sid()

```
const struct service_t* tfm_spm_get_service_by_sid (
 uint32_t sid)
```

Get the service context by service ID.

##### Parameters

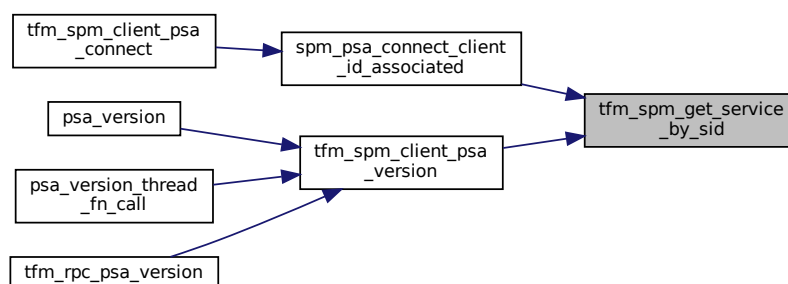
|    |            |                      |
|----|------------|----------------------|
| in | <i>sid</i> | RoT Service identity |
|----|------------|----------------------|

##### Return values

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| <i>NULL</i>     | Failed                                                               |
| <i>Not NULL</i> | Target service context pointer, <a href="#">service_t</a> structures |

Definition at line 111 of file `spm_ipc.c`.

Here is the caller graph for this function:



#### 8.411.3.16 tfm\_spm\_init()

```
uint32_t tfm_spm_init (
 void)
```

SPM initialization implementation.

This function must be called under handler mode.

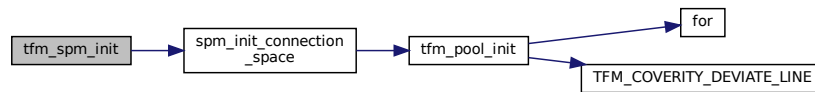


## Return values

|             |                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------|
| <i>This</i> | function returns an EXC_RETURN value. Other faults would panic the execution and never returned. |
|-------------|--------------------------------------------------------------------------------------------------|

Definition at line 396 of file spm\_ipc.c.

Here is the call graph for this function:



## 8.411.3.17 tfm\_spm\_is\_ns\_caller()

```
bool tfm_spm_is_ns_caller (
 void)
```

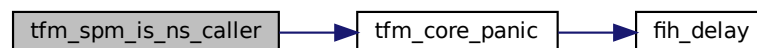
Get the ns\_caller info from runtime context.

## Return values

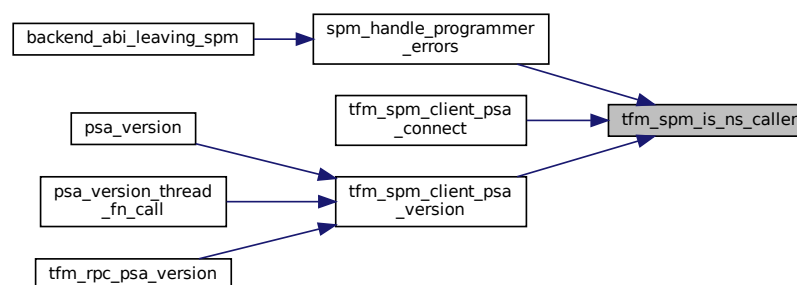
- |   |                                                                                           |
|---|-------------------------------------------------------------------------------------------|
| - | true: the PSA API caller is from non-secure<br>• false: the PSA API caller is from secure |
|---|-------------------------------------------------------------------------------------------|

Definition at line 367 of file spm\_ipc.c.

Here is the call graph for this function:



Here is the caller graph for this function:





- [psa\\_status\\_t spm\\_validate\\_connection](#) (const struct [connection\\_t](#) \*p\_connection)
- [void spm\\_free\\_connection](#) (struct [connection\\_t](#) \*p\_connection)

## 8.412.1 Macro Definition Documentation

### 8.412.1.1 CONVERSION\_FACTOR\_BITOFFSET

```
#define CONVERSION_FACTOR_BITOFFSET 3
```

Definition at line 28 of file `spm_connection_pool.c`.

### 8.412.1.2 CONVERSION\_FACTOR\_VALUE

```
#define CONVERSION_FACTOR_VALUE (1 << CONVERSION_FACTOR_BITOFFSET)
```

Definition at line 29 of file `spm_connection_pool.c`.

### 8.412.1.3 CONVERSION\_FACTOR\_VALUE\_MAX

```
#define CONVERSION_FACTOR_VALUE_MAX 0x20
```

Definition at line 31 of file `spm_connection_pool.c`.

## 8.412.2 Function Documentation

### 8.412.2.1 connection\_to\_handle()

```
psa_handle_t connection_to_handle (
 struct connection_t * p_connection)
```

Converts a handle instance into a corresponded user handle.

Definition at line 64 of file `spm_connection_pool.c`.

### 8.412.2.2 handle\_to\_connection()

```
struct connection_t* handle_to_connection (
 psa_handle_t handle)
```

Converts a user handle into a corresponded handle instance.

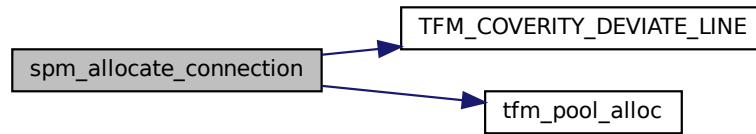
Definition at line 94 of file `spm_connection_pool.c`.

### 8.412.2.3 spm\_allocate\_connection()

```
struct connection_t* spm_allocate_connection (
 void)
```

Definition at line 122 of file `spm_connection_pool.c`.

Here is the call graph for this function:

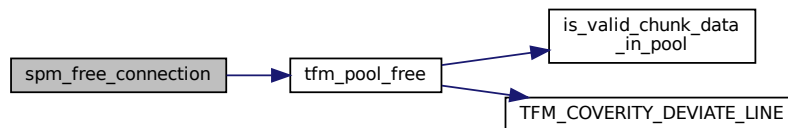


#### 8.412.2.4 `spm_free_connection()`

```
void spm_free_connection (
 struct connection_t * p_connection)
```

Definition at line 139 of file `spm_connection_pool.c`.

Here is the call graph for this function:

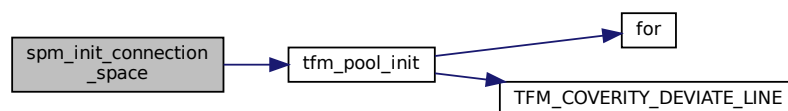


#### 8.412.2.5 `spm_init_connection_space()`

```
void spm_init_connection_space (
 void)
```

Definition at line 112 of file `spm_connection_pool.c`.

Here is the call graph for this function:



#### 8.412.2.6 `spm_validate_connection()`

```
psa_status_t spm_validate_connection (
 const struct connection_t * p_connection)
```

Definition at line 128 of file `spm_connection_pool.c`.

```
#include <inttypes.h>
```

Include dependency graph for `spm_ipc.c`:

## Functions

- const struct [service\\_t](#) \* [tfm\\_spm\\_get\\_service\\_by\\_sid](#) (uint32\_t sid)  
*Get the service context by service ID.*
- int32\_t [tfm\\_spm\\_check\\_client\\_version](#) (const struct [service\\_t](#) \*service, uint32\_t version)  
*Check the client version according to version policy.*
- int32\_t [tfm\\_spm\\_check\\_authorization](#) (uint32\_t sid, const struct [service\\_t](#) \*service, bool ns\_caller)  
*Check the client access authorization.*
- [psa\\_status\\_t](#) [spm\\_get\\_idle\\_connection](#) (struct [connection\\_t](#) \*\*p\_connection, [psa\\_handle\\_t](#) handle, int32\_t client\_id)  
*Convert the given user handle to an SPM recognised connection and verify that it is a valid idle connection that the caller is authorised to access.*
- struct [connection\\_t](#) \* [spm\\_msg\\_handle\\_to\\_connection](#) ([psa\\_handle\\_t](#) msg\_handle)  
*Convert the given message handle to SPM recognised handle and verify it.*
- void [spm\\_init\\_idle\\_connection](#) (struct [connection\\_t](#) \*p\_connection, const struct [service\\_t](#) \*service, int32\_t client\_id)  
*Initialize connection, fill in with the input information and set to idle.*
- int32\_t [tfm\\_spm\\_partition\\_get\\_running\\_partition\\_id](#) (void)  
*Get the running partition ID.*
- bool [tfm\\_spm\\_is\\_ns\\_caller](#) (void)  
*Get the ns\_caller info from runtime context.*
- int32\_t [tfm\\_spm\\_get\\_client\\_id](#) (bool ns\_caller)  
*Get ID of current RoT Service client. This API ensures the caller gets a valid ID.*
- uint32\_t [tfm\\_spm\\_init](#) (void)  
*SPM initialization implementation.*

## Variables

- struct [service\\_t](#) \* [stateless\\_services\\_ref\\_tbl](#) [32]

## 8.413.1 Function Documentation

### 8.413.1.1 spm\_get\_idle\_connection()

```
psa_status_t spm_get_idle_connection (
 struct connection_t ** p_connection,
 psa_handle_t handle,
 int32_t client_id)
```

Convert the given user handle to an SPM recognised connection and verify that it is a valid idle connection that the caller is authorised to access.

#### Parameters

|     |                     |                                                                                                                                |
|-----|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| out | <i>p_connection</i> | The address of connection pointer to be converted from the given user handle.                                                  |
| in  | <i>handle</i>       | Either a static handle or a handle to an established connection that was returned by a prior <a href="#">psa_connect</a> call. |
| in  | <i>client_id</i>    | The client ID of the caller.                                                                                                   |

#### Return values

|                                     |                                                    |
|-------------------------------------|----------------------------------------------------|
| <i>PSA_SUCCESS</i>                  | Success.                                           |
| <i>PSA_ERROR_CONNECTION_REFUSED</i> | The SPM or RoT Service has refused the connection. |

## Return values

|                                   |                                                                                                                |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------|
| <i>PSA_ERROR_CONNECTION_BUSY</i>  | The SPM or RoT Service cannot make the connection at the moment.                                               |
| <i>PSA_ERROR_PROGRAMMER_ERROR</i> | The handle is invalid, the caller is not authorised to use it or the connection is already handling a request. |

Definition at line 206 of file spm\_ipc.c.

## 8.413.1.2 spm\_init\_idle\_connection()

```
void spm_init_idle_connection (
 struct connection_t * p_connection,
 const struct service_t * service,
 int32_t client_id)
```

Initialize connection, fill in with the input information and set to idle.

## Parameters

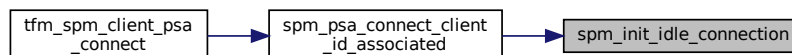
|    |                     |                                                                                         |
|----|---------------------|-----------------------------------------------------------------------------------------|
| in | <i>p_connection</i> | The 'p_connection' to initialize and fill information in.                               |
| in | <i>service</i>      | Target service context pointer, which can be obtained by partition management functions |
| in | <i>client_id</i>    | Partition ID of the sender of the message                                               |

Definition at line 329 of file spm\_ipc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



## 8.413.1.3 spm\_msg\_handle\_to\_connection()

```
struct connection_t* spm_msg_handle_to_connection (
 psa_handle_t msg_handle)
```

Convert the given message handle to SPM recognised handle and verify it.

## Parameters

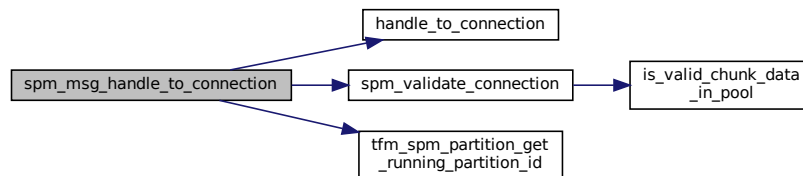
|    |                   |                                                                                 |
|----|-------------------|---------------------------------------------------------------------------------|
| in | <i>msg_handle</i> | Message handle which is a reference generated by the SPM to a specific message. |
|----|-------------------|---------------------------------------------------------------------------------|

## Returns

A SPM recognised handle or NULL. It is NULL when verification of the converted SPM handle fails. [connection\\_t](#) structures

Definition at line 302 of file `spm_ipc.c`.

Here is the call graph for this function:



#### 8.413.1.4 tfm\_spm\_check\_authorization()

```
int32_t tfm_spm_check_authorization (
 uint32_t sid,
 const struct service_t * service,
 bool ns_caller)
```

Check the client access authorization.

## Parameters

|    |                  |                                                                                    |
|----|------------------|------------------------------------------------------------------------------------|
| in | <i>sid</i>       | Target RoT Service identity                                                        |
| in | <i>service</i>   | Target service context pointer, which can be get by partition management functions |
| in | <i>ns_caller</i> | Whether from NS caller                                                             |

## Return values

|                          |                            |
|--------------------------|----------------------------|
| <i>PSA_SUCCESS</i>       | Success                    |
| <i>SPM_ERROR_GENERIC</i> | Authorization check failed |

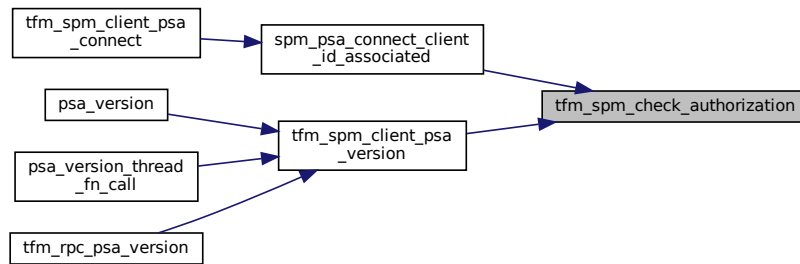
Definition at line 171 of file `spm_ipc.c`.

Here is the call graph for this function:





Here is the caller graph for this function:



#### 8.413.1.5 tfm\_spm\_check\_client\_version()

```
int32_t tfm_spm_check_client_version (
 const struct service_t * service,
 uint32_t version)
```

Check the client version according to version policy.

##### Parameters

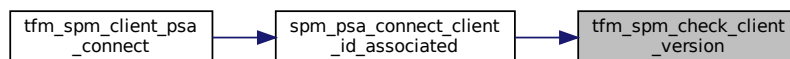
|    |                |                                                                                    |
|----|----------------|------------------------------------------------------------------------------------|
| in | <i>service</i> | Target service context pointer, which can be get by partition management functions |
| in | <i>version</i> | Client support version                                                             |

##### Return values

|                                 |                      |
|---------------------------------|----------------------|
| <i>PSA_SUCCESS</i>              | Success              |
| <i>SPM_ERROR_BAD_PARAMETERS</i> | Bad parameters input |
| <i>SPM_ERROR_VERSION</i>        | Check failed         |

Definition at line 149 of file `spm_ipc.c`.

Here is the caller graph for this function:



#### 8.413.1.6 tfm\_spm\_get\_client\_id()

```
int32_t tfm_spm_get_client_id (
 bool ns_caller)
```

Get ID of current RoT Service client. This API ensures the caller gets a valid ID.

## Parameters

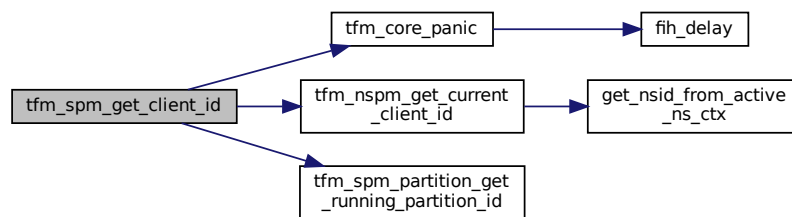
|    |                  |                                     |
|----|------------------|-------------------------------------|
| in | <i>ns_caller</i> | If the client is Non-Secure or not. |
|----|------------------|-------------------------------------|

## Return values

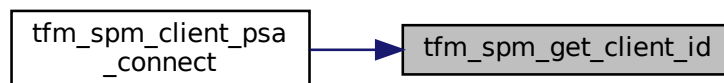
|            |           |
|------------|-----------|
| <i>The</i> | client ID |
|------------|-----------|

Definition at line 378 of file `spm_ipc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.413.1.7 tfm\_spm\_get\_service\_by\_sid()

```
const struct service_t* tfm_spm_get_service_by_sid (
 uint32_t sid)
```

Get the service context by service ID.

## Parameters

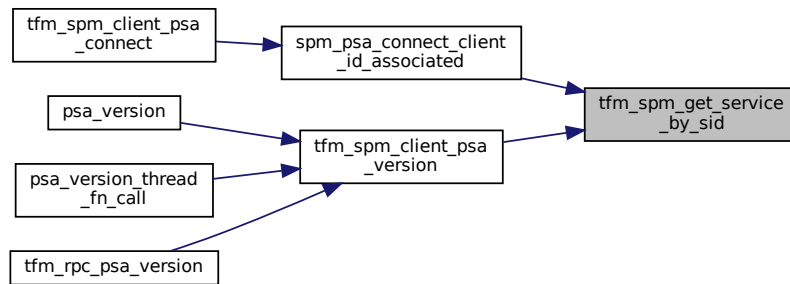
|    |            |                      |
|----|------------|----------------------|
| in | <i>sid</i> | RoT Service identity |
|----|------------|----------------------|

## Return values

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| <i>NULL</i>     | Failed                                                               |
| <i>Not NULL</i> | Target service context pointer, <a href="#">service_t</a> structures |

Definition at line 111 of file `spm_ipc.c`.

Here is the caller graph for this function:



#### 8.413.1.8 tfm\_spm\_init()

```
uint32_t tfm_spm_init (
 void)
```

SPM initialization implementation.

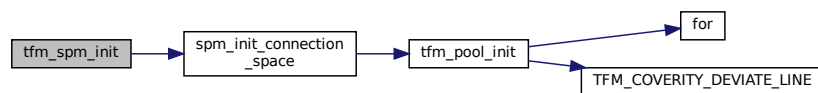
This function must be called under handler mode.

##### Return values

|             |                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------|
| <i>This</i> | function returns an EXC_RETURN value. Other faults would panic the execution and never returned. |
|-------------|--------------------------------------------------------------------------------------------------|

Definition at line 396 of file spm\_ipc.c.

Here is the call graph for this function:



#### 8.413.1.9 tfm\_spm\_is\_ns\_caller()

```
bool tfm_spm_is_ns_caller (
 void)
```

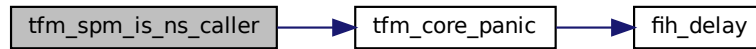
Get the ns\_caller info from runtime context.

##### Return values

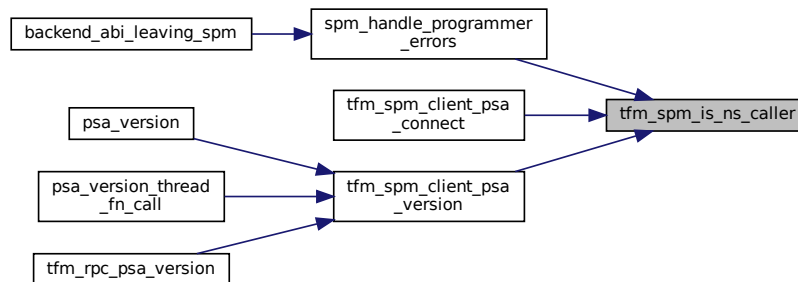
|   |                                                                                           |
|---|-------------------------------------------------------------------------------------------|
| - | true: the PSA API caller is from non-secure<br>• false: the PSA API caller is from secure |
|---|-------------------------------------------------------------------------------------------|

Definition at line 367 of file spm\_ipc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.413.1.10 tfm\_spm\_partition\_get\_running\_partition\_id()

```
int32_t tfm_spm_partition_get_running_partition_id (
 void)
```

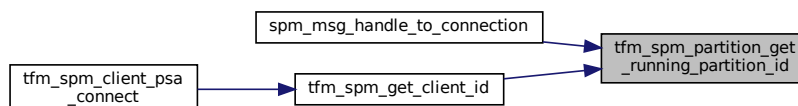
Get the running partition ID.

##### Returns

Returns the partition ID

Definition at line 355 of file `spm_ipc.c`.

Here is the caller graph for this function:



## 8.413.2 Variable Documentation

### 8.413.2.1 stateless\_services\_ref\_tbl

```
struct service_t* stateless_services_ref_tbl[32]
```

Definition at line 45 of file `spm_ipc.c`.



#### 8.414.2.1 connection\_to\_handle()

```
psa_handle_t connection_to_handle (
 struct connection_t * p_connection)
```

Converts a handle instance into a corresponded user handle.

Definition at line 52 of file `spm_local_connection.c`.

Here is the caller graph for this function:



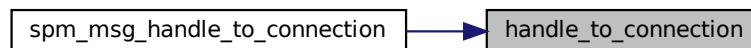
#### 8.414.2.2 handle\_to\_connection()

```
struct connection_t* handle_to_connection (
 psa_handle_t handle)
```

Converts a user handle into a corresponded handle instance.

Definition at line 59 of file `spm_local_connection.c`.

Here is the caller graph for this function:

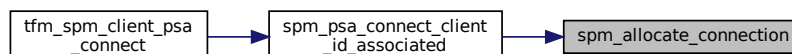


#### 8.414.2.3 spm\_allocate\_connection()

```
struct connection_t* spm_allocate_connection (
 void)
```

Definition at line 83 of file `spm_local_connection.c`.

Here is the caller graph for this function:



#### 8.414.2.4 spm\_free\_connection()

```
void spm_free_connection (
 struct connection_t * p_connection)
```

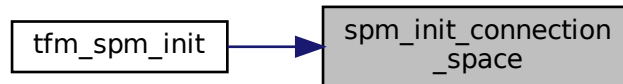
Definition at line 94 of file `spm_local_connection.c`.

### 8.414.2.5 spm\_init\_connection\_space()

```
void spm_init_connection_space (
 void)
```

Definition at line 74 of file `spm_local_connection.c`.

Here is the caller graph for this function:

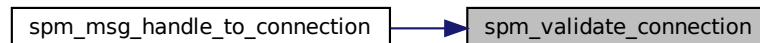


### 8.414.2.6 spm\_validate\_connection()

```
psa_status_t spm_validate_connection (
 const struct connection_t * p_connection)
```

Definition at line 88 of file `spm_local_connection.c`.

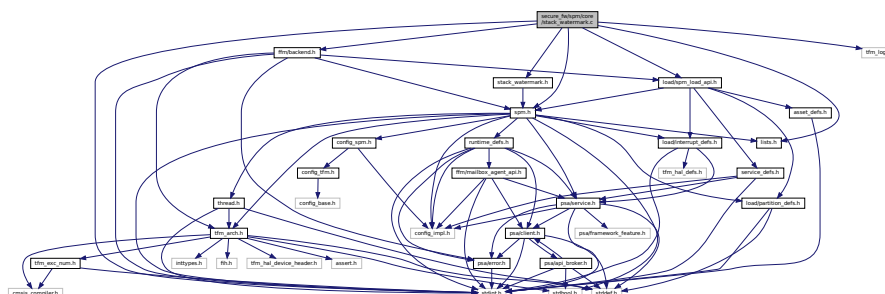
Here is the caller graph for this function:



## 8.415 secure\_fw/spm/core/stack\_watermark.c File Reference

```
#include <stdint.h>
#include "ffm/backend.h"
#include "stack_watermark.h"
#include "lists.h"
#include "load/spm_load_api.h"
#include "spm.h"
#include "tfm_log.h"
```

Include dependency graph for `stack_watermark.c`:



## Macros

- `#define SPMLOG(x) tfm_hal_output_spm_log((x), sizeof(x))`
- `#define SPMLOG_VAL(x, y) spm_log_msgval((x), sizeof(x), y)`
- `#define STACK_WATERMARK_VAL 0xdeadbeef`

## Functions

- `while (addr < SPM_THREAD_CONTEXT->sp_base)`
- `for (int i=0; i < p_pldi->stack_size/sizeof(uint32_t); i++)`
- `tfm_hal_output_spm_log ("Used stack sizes report\r\n", sizeof("Used stack sizes report\r\n"))`
- `tfm_hal_output_spm_log (" SPM\r\n", sizeof(" SPM\r\n"))`
- `spm_log_msgval (" Stack bytes: ", sizeof(" Stack bytes: "), CONFIG_TFM_SPM_THREAD_STACK_SIZE)`
- `spm_log_msgval (" Stack bytes used: ", sizeof(" Stack bytes used: "), used_spm_stack())`
- `UNI_LIST_FOREACH (p_pt, PARTITION_LIST_ADDR, next)`

## Variables

- `void`

## 8.415.1 Macro Definition Documentation

### 8.415.1.1 SPMLOG

```
#define SPMLOG(
 x) tfm_hal_output_spm_log((x), sizeof(x))
```

Definition at line 20 of file stack\_watermark.c.

### 8.415.1.2 SPMLOG\_VAL

```
#define SPMLOG_VAL(
 x,
 y) spm_log_msgval((x), sizeof(x), y)
```

Definition at line 21 of file stack\_watermark.c.

### 8.415.1.3 STACK\_WATERMARK\_VAL

```
#define STACK_WATERMARK_VAL 0xdeadbeef
```

Definition at line 23 of file stack\_watermark.c.

## 8.415.2 Function Documentation

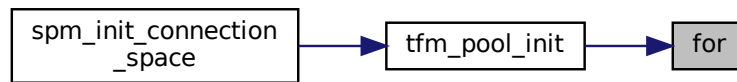
### 8.415.2.1 for()

```
for (
 int i = 0; i < p_pldi->stack_size / sizeof(uint32_t); i++)
```

Definition at line 39 of file stack\_watermark.c.



Here is the caller graph for this function:



#### 8.415.2.2 spm\_log\_msgval() [1/2]

```

spm_log_msgval (
 (" Stack bytes used: ") ,
 sizeof(" Stack bytes used: ") ,
 used_spm_stack())

```

#### 8.415.2.3 spm\_log\_msgval() [2/2]

```

spm_log_msgval (
 (" Stack bytes: ") ,
 sizeof(" Stack bytes: ") ,
 CONFIG_TFM_SPM_THREAD_STACK_SIZE)

```

#### 8.415.2.4 tfm\_hal\_output\_spm\_log() [1/2]

```

tfm_hal_output_spm_log (
 (" SPM\r\n") ,
 sizeof(" SPM\r\n"))

```

#### 8.415.2.5 tfm\_hal\_output\_spm\_log() [2/2]

```

tfm_hal_output_spm_log (
 ("Used stack sizes report\r\n") ,
 sizeof("Used stack sizes report\r\n"))

```

#### 8.415.2.6 UNI\_LIST\_FOREACH()

```

UNI_LIST_FOREACH (
 p_pt ,
 PARTITION_LIST_ADDR ,
 next)

```

Definition at line 91 of file stack\_watermark.c.

#### 8.415.2.7 while()

```

while (
 addr< SPM_THREAD_CONTEXT-> sp_base)

```

Definition at line 29 of file stack\_watermark.c.



### 8.416.1.2 watermark\_spm\_stack

```
#define watermark_spm_stack()
```

Definition at line 21 of file stack\_watermark.h.

### 8.416.1.3 watermark\_stack

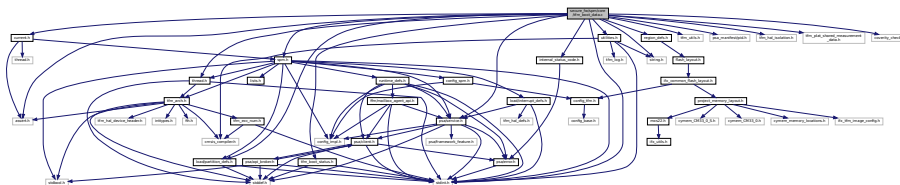
```
#define watermark_stack(
 p_pt)
```

Definition at line 22 of file stack\_watermark.h.

## 8.417 secure\_fw/spm/core/tfm\_boot\_data.c File Reference

```
#include <assert.h>
#include <stdint.h>
#include <string.h>
#include "current.h"
#include "tfm_utils.h"
#include "tfm_boot_status.h"
#include "region_defs.h"
#include "psa_manifest/pid.h"
#include "internal_status_code.h"
#include "utilities.h"
#include "psa/service.h"
#include "thread.h"
#include "spm.h"
#include "load/partition_defs.h"
#include "tfm_hal_isolation.h"
#include "tfm_plat_shared_measurement_data.h"
#include "coverity_check.h"
```

Include dependency graph for tfm\_boot\_data.c:



## Data Structures

- struct [boot\\_data\\_access\\_policy](#)

*Defines the access policy of secure partitions to data items in shared data area (between bootloader and runtime firmware).*

## Macros

- #define [BOOT\\_DATA\\_VALID](#) (1u)

*Indicates that shared data between bootloader and runtime firmware was passed the sanity check with success.*

- #define [BOOT\\_DATA\\_INVALID](#) (0u)

*Indicates that shared data between bootloader and runtime firmware was failed on sanity check.*

## Functions

- void [tfm\\_core\\_validate\\_boot\\_data](#) (void)

*Validate the content of shared memory area, which stores the shared data between bootloader and runtime firmware.*

- [void tfm\\_core\\_get\\_boot\\_data\\_handler](#) (uint32\_t args[])

*Retrieve secure partition related data from shared memory area, which stores shared data between bootloader and runtime firmware.*

## 8.417.1 Macro Definition Documentation

### 8.417.1.1 BOOT\_DATA\_INVALID

```
#define BOOT_DATA_INVALID (0u)
```

Indicates that shared data between bootloader and runtime firmware was failed on sanity check.

Definition at line 40 of file tfm\_boot\_data.c.

### 8.417.1.2 BOOT\_DATA\_VALID

```
#define BOOT_DATA_VALID (1u)
```

Indicates that shared data between bootloader and runtime firmware was passed the sanity check with success.

Definition at line 32 of file tfm\_boot\_data.c.

## 8.417.2 Function Documentation

### 8.417.2.1 tfm\_core\_get\_boot\_data\_handler()

```
void tfm_core_get_boot_data_handler (
 uint32_t args[])
```

Retrieve secure partition related data from shared memory area, which stores shared data between bootloader and runtime firmware.

#### Parameters

|    |      |                                                         |
|----|------|---------------------------------------------------------|
| in | args | Pointer to stack frame, which carries input parameters. |
|----|------|---------------------------------------------------------|

Definition at line 156 of file tfm\_boot\_data.c.

### 8.417.2.2 tfm\_core\_validate\_boot\_data()

```
void tfm_core_validate_boot_data (
 void)
```

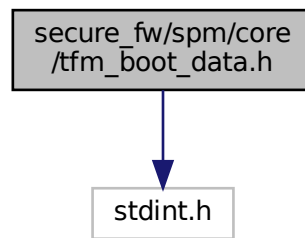
Validate the content of shared memory area, which stores the shared data between bootloader and runtime firmware.

Definition at line 122 of file tfm\_boot\_data.c.

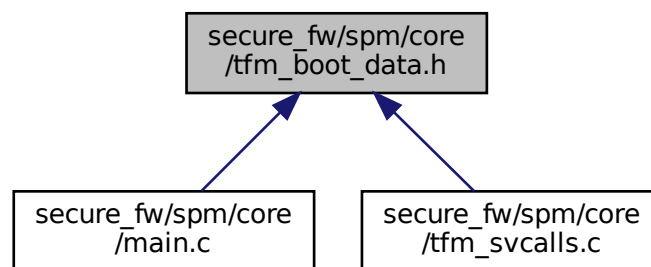
## 8.418 secure\_fw/spm/core/tfm\_boot\_data.h File Reference

```
#include <stdint.h>
```

Include dependency graph for tfm\_boot\_data.h:



This graph shows which files directly or indirectly include this file:



## Functions

- [void tfm\\_core\\_get\\_boot\\_data\\_handler](#) (uint32\_t args[])  
Retrieve secure partition related data from shared memory area, which stores shared data between bootloader and runtime firmware.
- [void tfm\\_core\\_validate\\_boot\\_data](#) (void)  
Validate the content of shared memory area, which stores the shared data between bootloader and runtime firmware.

## 8.418.1 Function Documentation

### 8.418.1.1 tfm\_core\_get\_boot\_data\_handler()

```
void tfm_core_get_boot_data_handler (
 uint32_t args[])
```

Retrieve secure partition related data from shared memory area, which stores shared data between bootloader and runtime firmware.

#### Parameters

|    |      |                                                         |
|----|------|---------------------------------------------------------|
| in | args | Pointer to stack frame, which carries input parameters. |
|----|------|---------------------------------------------------------|

Definition at line 156 of file tfm\_boot\_data.c.

#### 8.418.1.2 tfm\_core\_validate\_boot\_data()

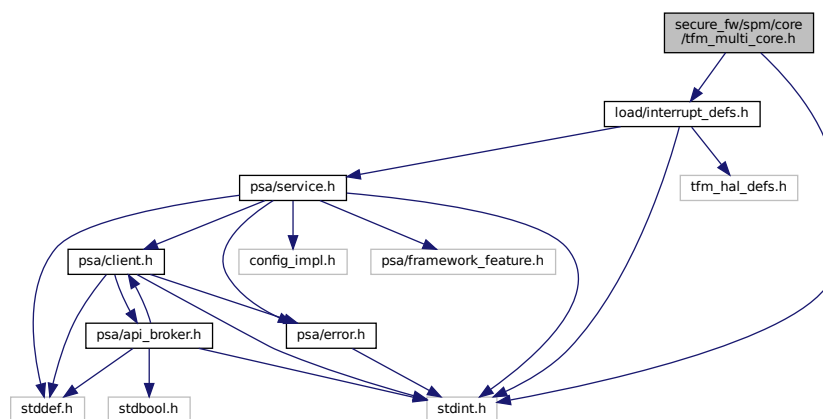
```
void tfm_core_validate_boot_data (
 void)
```

Validate the content of shared memory area, which stores the shared data between bootloader and runtime firmware.

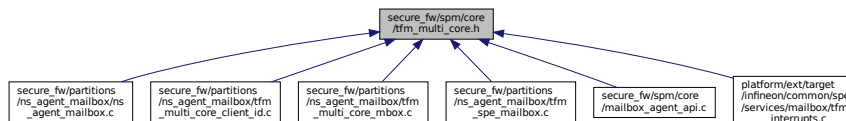
Definition at line 122 of file tfm\_boot\_data.c.

### 8.419 secure\_fw/spm/core/tfm\_multi\_core.h File Reference

```
#include <stdint.h>
#include "load/interrupt_defs.h"
Include dependency graph for tfm_multi_core.h:
```



This graph shows which files directly or indirectly include this file:



## Macros

- #define `CLIENT_ID_OWNER_MAGIC` (void \*)0xFFFFFFFF

## Functions

- `int32_t tfm_inter_core_comm_init` (void)  
*Initialization of the multi core communication.*
- `int32_t tfm_inter_core_comm_enable` (void)  
*Enable the multi core communication.*
- `int32_t tfm_inter_core_comm_disable` (void)

- Disable the multi core communication.*
- `int32_t tfm_inter_core_comm_deinit (void)`  
*De-initialization of the multi core communication.*
- `int32_t tfm_multi_core_register_client_id_range (void *owner, uint32_t irq_source)`  
*Register a non-secure client ID range.*
- `int32_t tfm_multi_core_hal_client_id_translate (void *owner, int32_t client_id_in, int32_t *client_id_out)`  
*Translate a non-secure client ID range.*
- `void tfm_multi_core_set_mbox_irq (const struct irq_load_info_t *p_ildi)`  
*Record a mailbox interrupt that will be completed later on.*
- `void tfm_multi_core_clear_mbox_irq (void)`  
*Clear all the mailbox interrupts that are to be processed.*

## 8.419.1 Macro Definition Documentation

### 8.419.1.1 CLIENT\_ID\_OWNER\_MAGIC

```
#define CLIENT_ID_OWNER_MAGIC (void *)0xFFFFFFFF
```

Definition at line 14 of file `tfm_multi_core.h`.

## 8.419.2 Function Documentation

### 8.419.2.1 tfm\_inter\_core\_comm\_deinit()

```
int32_t tfm_inter_core_comm_deinit (
 void)
```

De-initialization of the multi core communication.

This may be called to return the multi-core communication from the initialised, but not enabled state to the pre-initialised state.

#### Return values

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>0</i>     | Operation succeeded.                             |
| <i>Other</i> | return code Operation failed with an error code. |

Definition at line 630 of file `tfm_spe_mailbox.c`.

### 8.419.2.2 tfm\_inter\_core\_comm\_disable()

```
int32_t tfm_inter_core_comm_disable (
 void)
```

Disable the multi core communication.

This may be called to return the multi core communication to the initialised, but not enabled, state.

#### Return values

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>0</i>     | Operation succeeded.                             |
| <i>Other</i> | return code Operation failed with an error code. |

Definition at line 625 of file `tfm_spe_mailbox.c`.

**8.419.2.3 tfm\_inter\_core\_comm\_enable()**

```
int32_t tfm_inter_core_comm_enable (
 void)
```

Enable the multi core communication.

This is called after `tfm_inter_core_init()` and may be called again after `tfm_inter_core_disable()`.

**Return values**

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>0</i>     | Operation succeeded.                             |
| <i>Other</i> | return code Operation failed with an error code. |

Definition at line 620 of file `tfm_spe_mailbox.c`.

**8.419.2.4 tfm\_inter\_core\_comm\_init()**

```
int32_t tfm_inter_core_comm_init (
 void)
```

Initialization of the multi core communication.

This is called once only, after the NS core has booted.

**Return values**

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>0</i>     | Operation succeeded.                             |
| <i>Other</i> | return code Operation failed with an error code. |

Definition at line 615 of file `tfm_spe_mailbox.c`.

**8.419.2.5 tfm\_multi\_core\_clear\_mbox\_irq()**

```
void tfm_multi_core_clear_mbox_irq (
 void)
```

Clear all the mailbox interrupts that are to be processed.

Definition at line 21 of file `tfm_multi_core_mbox.c`.

**8.419.2.6 tfm\_multi\_core\_hal\_client\_id\_translate()**

```
int32_t tfm_multi_core_hal_client_id_translate (
 void * owner,
 int32_t client_id_in,
 int32_t * client_id_out)
```

Translate a non-secure client ID range.

**Parameters**

|     |                      |                                                                                                   |
|-----|----------------------|---------------------------------------------------------------------------------------------------|
| in  | <i>owner</i>         | Identifier of the non-secure client.                                                              |
| in  | <i>client_id_in</i>  | The input client ID.                                                                              |
| out | <i>client_id_out</i> | The translated client ID. Undefined if <code>SPM_ERROR_GENERIC</code> is returned by the function |

**Returns**

`SPM_SUCCESS` if the translation is successful. `SPM_ERROR_BAD_PARAMETERS` if a parameter is invalid. `SPM_ERROR_GENERIC` otherwise.

Definition at line 40 of file `tfm_multi_core_client_id.c`.



**8.419.2.7 tfm\_multi\_core\_register\_client\_id\_range()**

```
int32_t tfm_multi_core_register_client_id_range (
 void * owner,
 uint32_t irq_source)
```

Register a non-secure client ID range.

This function looks for a pre-defined client ID range in `ns_mailbox_client_id_info[]` according to `irq_source` value. If matched, it links this non-secure client ID range to the `owner`.

**Note**

Each client ID range shall be registered only once.

**Parameters**

|    |                   |                                                                       |
|----|-------------------|-----------------------------------------------------------------------|
| in | <i>owner</i>      | Identifier of the non-secure client.                                  |
| in | <i>irq_source</i> | The mailbox interrupt source of the target non-secure client ID range |

**Returns**

SPM\_SUCCESS if the registration is successful. SPM\_ERROR\_BAD\_PARAMETERS if `owner` is null. SP↵M\_ERROR\_GENERIC otherwise.

Definition at line 16 of file `tfm_multi_core_client_id.c`.

**8.419.2.8 tfm\_multi\_core\_set\_mbox\_irq()**

```
void tfm_multi_core_set_mbox_irq (
 const struct irq_load_info_t * p_ildi)
```

Record a mailbox interrupt that will be completed later on.

**Parameters**

|    |               |                                                                    |
|----|---------------|--------------------------------------------------------------------|
| in | <i>p_ildi</i> | The <code>irq_load_info_t</code> struct of the interrupt to record |
|----|---------------|--------------------------------------------------------------------|

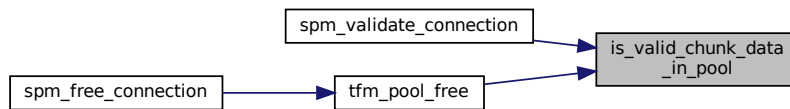
Definition at line 38 of file `tfm_multi_core_mbox.c`.

**8.420 secure\_fw/spm/core/tfm\_pools.c File Reference**

```
#include <inttypes.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include "thread.h"
#include "psa/client.h"
#include "psa/service.h"
#include "internal_status_code.h"
#include "cmsis_compiler.h"
#include "utilities.h"
#include "lists.h"
#include "tfm_pools.h"
#include "coverity_check.h"
```



Definition at line 106 of file tfm\_pools.c.  
Here is the caller graph for this function:



### 8.420.2.2 tfm\_pool\_alloc()

```
void* tfm_pool_alloc (
 struct tfm_pool_instance_t * pool)
```

Allocate a memory from pool.

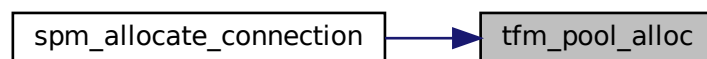
#### Parameters

|    |             |                                                           |
|----|-------------|-----------------------------------------------------------|
| in | <i>pool</i> | pool pointer declared by <a href="#">TFM_POOL_DECLARE</a> |
|----|-------------|-----------------------------------------------------------|

#### Return values

|               |                  |
|---------------|------------------|
| <i>buffer</i> | pointer Success. |
| <i>NULL</i>   | Failed.          |

Definition at line 62 of file tfm\_pools.c.  
Here is the caller graph for this function:



### 8.420.2.3 tfm\_pool\_free()

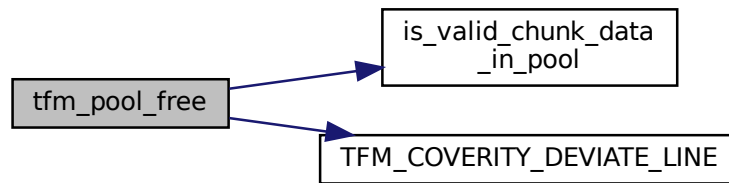
```
void tfm_pool_free (
 struct tfm_pool_instance_t * pool,
 void * ptr)
```

Free the allocated memory.

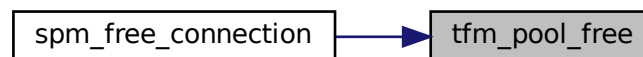
#### Parameters

|    |             |                                                           |
|----|-------------|-----------------------------------------------------------|
| in | <i>pool</i> | pool pointer declared by <a href="#">TFM_POOL_DECLARE</a> |
| in | <i>ptr</i>  | Buffer pointer want to free.                              |

Definition at line 86 of file tfm\_pools.c.  
Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.420.2.4 tfm\_pool\_init()

```

psa_status_t tfm_pool_init (
 struct tfm_pool_instance_t * pool,
 size_t poolsz,
 size_t chunksz,
 size_t num)

```

Register a memory pool.

##### Parameters

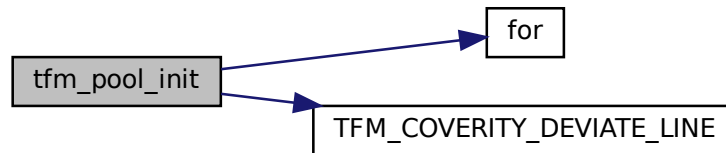
|    |                |                                                                     |
|----|----------------|---------------------------------------------------------------------|
| in | <i>pool</i>    | Pointer to memory pool declared by <a href="#">TFM_POOL_DECLARE</a> |
| in | <i>poolsz</i>  | Size of the pool buffer.                                            |
| in | <i>chunksz</i> | Size of chunks.                                                     |
| in | <i>num</i>     | Number of chunks.                                                   |

##### Return values

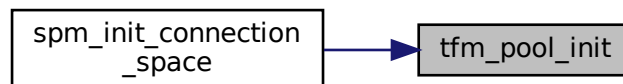
|                                 |                   |
|---------------------------------|-------------------|
| <i>PSA_SUCCESS</i>              | Success.          |
| <i>SPM_ERROR_BAD_PARAMETERS</i> | Parameters error. |

Definition at line 25 of file tfm\_pools.c.

Here is the call graph for this function:

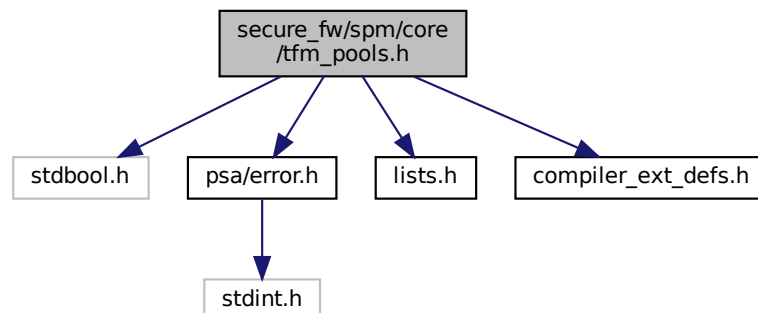


Here is the caller graph for this function:

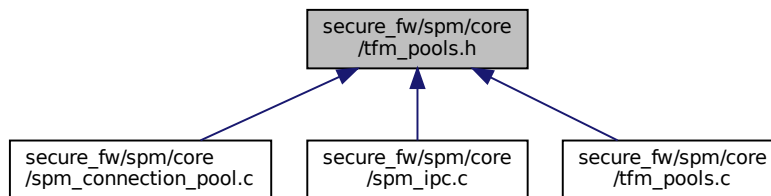


## 8.421 secure\_fw/spm/core/tfm\_pools.h File Reference

```
#include <stdbool.h>
#include "psa/error.h"
#include "lists.h"
#include "compiler_ext_defs.h"
Include dependency graph for tfm_pools.h:
```



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [tfm\\_pool\\_chunk\\_t](#)
- struct [tfm\\_pool\\_instance\\_t](#)

## Macros

- #define [TFM\\_POOL\\_DECLARE](#)(name, chunksz, num)
- #define [POOL\\_HEAD\\_SIZE](#)
- #define [POOL\\_BUFFER\\_SIZE](#)(name) sizeof(name##\_pool\_buf)

## Functions

- [psa\\_status\\_t tfm\\_pool\\_init](#) (struct [tfm\\_pool\\_instance\\_t](#) \*pool, size\_t poolsz, size\_t chunksz, size\_t num)  
*Register a memory pool.*
- [void \\* tfm\\_pool\\_alloc](#) (struct [tfm\\_pool\\_instance\\_t](#) \*pool)  
*Allocate a memory from pool.*
- [void tfm\\_pool\\_free](#) (struct [tfm\\_pool\\_instance\\_t](#) \*pool, void \*ptr)  
*Free the allocated memory.*
- [bool is\\_valid\\_chunk\\_data\\_in\\_pool](#) (struct [tfm\\_pool\\_instance\\_t](#) \*pool, uint8\_t \*data)  
*Checks whether a pointer points to a valid allocated chunk of data in the pool.*

## 8.421.1 Macro Definition Documentation

### 8.421.1.1 POOL\_BUFFER\_SIZE

```
#define POOL_BUFFER_SIZE(
 name) sizeof(name##_pool_buf)
```

Definition at line 53 of file [tfm\\_pools.h](#).

### 8.421.1.2 POOL\_HEAD\_SIZE

```
#define POOL_HEAD_SIZE
```

**Value:**

```
(sizeof(struct tfm_pool_instance_t) +
 sizeof(struct tfm_pool_chunk_t))
```

Definition at line 49 of file [tfm\\_pools.h](#).

### 8.421.1.3 TFM\_POOL\_DECLARE

```
#define TFM_POOL_DECLARE(
 name,
 chunksz,
 num)
```

#### Value:

```
static uint8_t name##_pool_buf[((chunksz) +
 sizeof(struct tfm_pool_chunk_t)) * (num)
 + sizeof(struct tfm_pool_instance_t)]
 __aligned(4);
static struct tfm_pool_instance_t *name =
 (struct tfm_pool_instance_t *)name##_pool_buf
```

Definition at line 40 of file tfm\_pools.h.

## 8.421.2 Function Documentation

### 8.421.2.1 is\_valid\_chunk\_data\_in\_pool()

```
bool is_valid_chunk_data_in_pool (
 struct tfm_pool_instance_t * pool,
 uint8_t * data)
```

Checks whether a pointer points to a valid allocated chunk of data in the pool.

#### Parameters

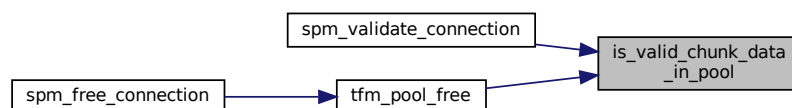
|    |             |                                                                       |
|----|-------------|-----------------------------------------------------------------------|
| in | <i>pool</i> | Pointer to memory pool declared by <a href="#">TFM_POOL_DECLARE</a> . |
| in | <i>data</i> | The pointer to check.                                                 |

#### Return values

|              |                                       |
|--------------|---------------------------------------|
| <i>true</i>  | Data is a chunk data in the pool.     |
| <i>false</i> | Data is not a chunk data in the pool. |

Definition at line 106 of file tfm\_pools.c.

Here is the caller graph for this function:



### 8.421.2.2 tfm\_pool\_alloc()

```
void* tfm_pool_alloc (
 struct tfm_pool_instance_t * pool)
```

Allocate a memory from pool.

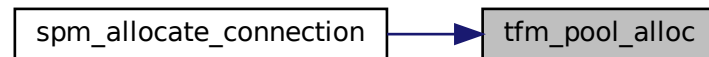
#### Parameters

|    |             |                                                           |
|----|-------------|-----------------------------------------------------------|
| in | <i>pool</i> | pool pointer declared by <a href="#">TFM_POOL_DECLARE</a> |
|----|-------------|-----------------------------------------------------------|

## Return values

|               |                  |
|---------------|------------------|
| <i>buffer</i> | pointer Success. |
| <i>NULL</i>   | Failed.          |

Definition at line 62 of file tfm\_pools.c.  
Here is the caller graph for this function:



## 8.421.2.3 tfm\_pool\_free()

```

void tfm_pool_free (
 struct tfm_pool_instance_t * pool,
 void * ptr)

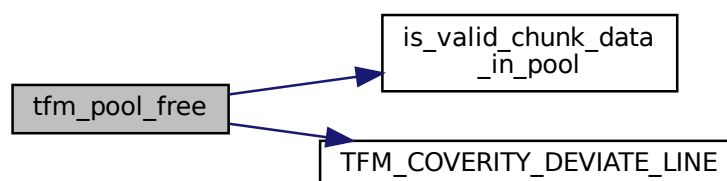
```

Free the allocated memory.

## Parameters

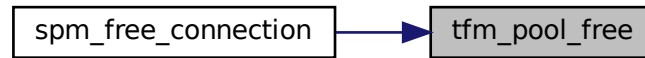
|    |             |                                                           |
|----|-------------|-----------------------------------------------------------|
| in | <i>pool</i> | pool pointer declared by <a href="#">TFM_POOL_DECLARE</a> |
| in | <i>ptr</i>  | Buffer pointer want to free.                              |

Definition at line 86 of file tfm\_pools.c.  
Here is the call graph for this function:





Here is the caller graph for this function:



#### 8.421.2.4 tfm\_pool\_init()

```

psa_status_t tfm_pool_init (
 struct tfm_pool_instance_t * pool,
 size_t poolsz,
 size_t chunksz,
 size_t num)

```

Register a memory pool.

##### Parameters

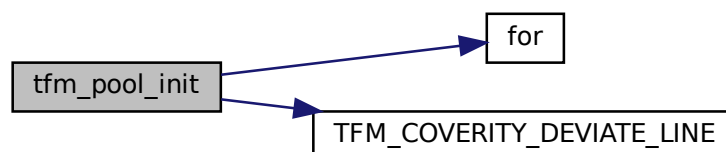
|    |                |                                                                     |
|----|----------------|---------------------------------------------------------------------|
| in | <i>pool</i>    | Pointer to memory pool declared by <a href="#">TFM_POOL_DECLARE</a> |
| in | <i>poolsz</i>  | Size of the pool buffer.                                            |
| in | <i>chunksz</i> | Size of chunks.                                                     |
| in | <i>num</i>     | Number of chunks.                                                   |

##### Return values

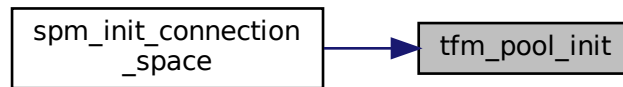
|                                 |                   |
|---------------------------------|-------------------|
| <i>PSA_SUCCESS</i>              | Success.          |
| <i>SPM_ERROR_BAD_PARAMETERS</i> | Parameters error. |

Definition at line 25 of file `tfm_pools.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



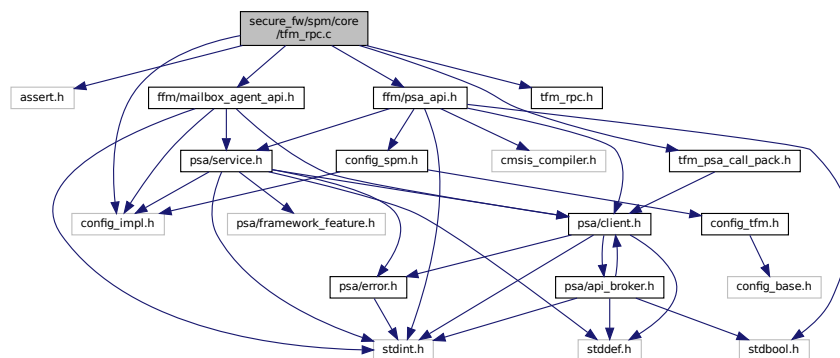
## 8.422 secure\_fw/spm/core/tfm\_rpc.c File Reference

```

#include <assert.h>
#include "config_impl.h"
#include "ffm/mailbox_agent_api.h"
#include "ffm/psa_api.h"
#include "tfm_rpc.h"
#include "tfm_psa_call_pack.h"

```

Include dependency graph for tfm\_rpc.c:



## Functions

- `uint32_t tfm_rpc_psa_framework_version (void)`
- `uint32_t tfm_rpc_psa_version (uint32_t sid)`
- `psa_status_t tfm_rpc_psa_call (psa_handle_t handle, uint32_t control, const struct client_params_t *params, const void *client_data_stateless)`

### 8.422.1 Function Documentation

#### 8.422.1.1 tfm\_rpc\_psa\_call()

```

psa_status_t tfm_rpc_psa_call (
 psa_handle_t handle,
 uint32_t control,
 const struct client_params_t * params,
 const void * client_data_stateless)

```

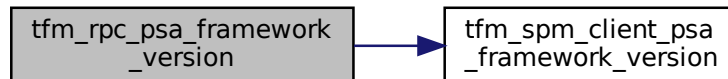
Definition at line 28 of file tfm\_rpc.c.

#### 8.422.1.2 tfm\_rpc\_psa\_framework\_version()

```
uint32_t tfm_rpc_psa_framework_version (
 void)
```

Definition at line 18 of file tfm\_rpc.c.

Here is the call graph for this function:

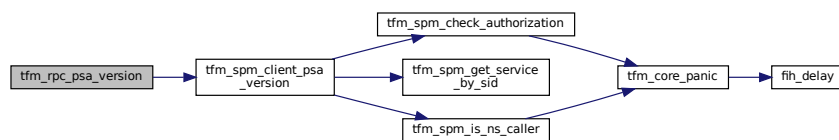


#### 8.422.1.3 tfm\_rpc\_psa\_version()

```
uint32_t tfm_rpc_psa_version (
 uint32_t sid)
```

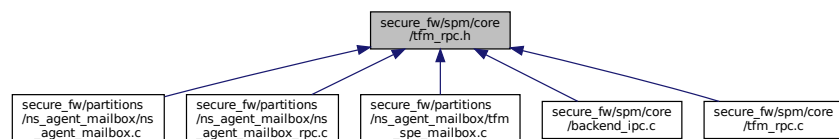
Definition at line 23 of file tfm\_rpc.c.

Here is the call graph for this function:



## 8.423 secure\_fw/spm/core/tfm\_rpc.h File Reference

This graph shows which files directly or indirectly include this file:



## 8.424 secure\_fw/spm/core/tfm\_svcalls.c File Reference

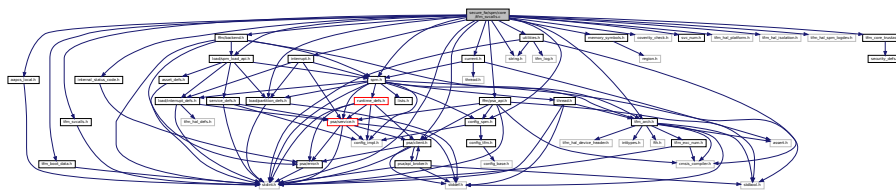
```
#include <string.h>
#include <stdint.h>
#include "aapcs_local.h"
```

```

#include "config_spm.h"
#include "current.h"
#include "interrupt.h"
#include "internal_status_code.h"
#include "memory_symbols.h"
#include "spm.h"
#include "coverity_check.h"
#include "svc_num.h"
#include "tfm_arch.h"
#include "tfm_svcalls.h"
#include "tfm_boot_data.h"
#include "tfm_hal_platform.h"
#include "tfm_hal_isolation.h"
#include "tfm_hal_spm_logdev.h"
#include "tfm_core_trustzone.h"
#include "utilities.h"
#include "ffm/backend.h"
#include "ffm/psa_api.h"
#include "load/spm_load_api.h"
#include "load/partition_defs.h"
#include "psa/client.h"

```

Include dependency graph for tfm\_svcalls.c:



## Macros

- `#define INVALID_PSP_VALUE 0xFFFFFFFFU`

## Functions

- `uint32_t spm_svc_handler (uint32_t *msp, uint32_t exc_return, uint32_t *psp)`

*The C source of SVCcall handlers.*

- `void tfm_svc_thread_mode_spm_return (psa_status_t result)`

*Return from PSA API function executed in Thread mode. Trigger svc handler to restore thread context before entering PSA API function.*

## 8.424.1 Macro Definition Documentation

### 8.424.1.1 INVALID\_PSP\_VALUE

```
#define INVALID_PSP_VALUE 0xFFFFFFFFU
```

Definition at line 38 of file `tfm_svcalls.c`.

## 8.424.2 Function Documentation

**8.424.2.1 spm\_svc\_handler()**

```
uint32_t spm_svc_handler (
 uint32_t * msp,
 uint32_t exc_return,
 uint32_t * psp)
```

The C source of SVCcall handlers.

**Parameters**

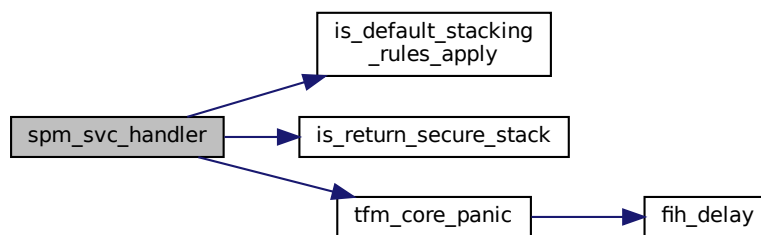
|    |                   |                              |
|----|-------------------|------------------------------|
| in | <i>msp</i>        | MSP at SVCcall entry.        |
| in | <i>exc_return</i> | EXC_RETURN value of the SVC. |
| in | <i>psp</i>        | PSP at SVCcall entry.        |

**Returns**

EXC\_RETURN value indicates where to return.

Definition at line 320 of file tfm\_svcalls.c.

Here is the call graph for this function:

**8.424.2.2 tfm\_svc\_thread\_mode\_spm\_return()**

```
void tfm_svc_thread_mode_spm_return (
 psa_status_t result)
```

Return from PSA API function executed in Thread mode. Trigger svc handler to restore thread context before entering PSA API function.

**Parameters**

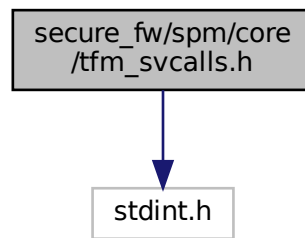
|    |               |                                |
|----|---------------|--------------------------------|
| in | <i>result</i> | PSA API function return value. |
|----|---------------|--------------------------------|

Definition at line 385 of file tfm\_svcalls.c.

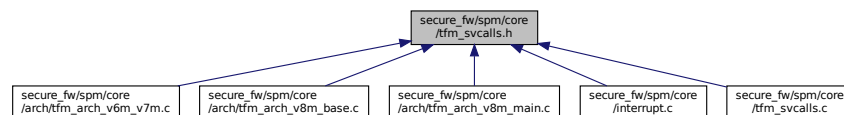
**8.425 secure\_fw/spm/core/tfm\_svcalls.h File Reference**

```
#include <stdint.h>
```

Include dependency graph for `tfm_svcalls.h`:



This graph shows which files directly or indirectly include this file:



## Functions

- `uint32_t spm_svc_handler` (`uint32_t *msp`, `uint32_t exc_return`, `uint32_t *psp`)

*The C source of SVCcall handlers.*

- `void tfm_svc_thread_mode_spm_return` (`psa_status_t result`)

*Return from PSA API function executed in Thread mode. Trigger svc handler to restore thread context before entering PSA API function.*

## 8.425.1 Function Documentation

### 8.425.1.1 `spm_svc_handler()`

```

uint32_t spm_svc_handler (
 uint32_t * msp,
 uint32_t exc_return,
 uint32_t * psp)

```

The C source of SVCcall handlers.

#### Parameters

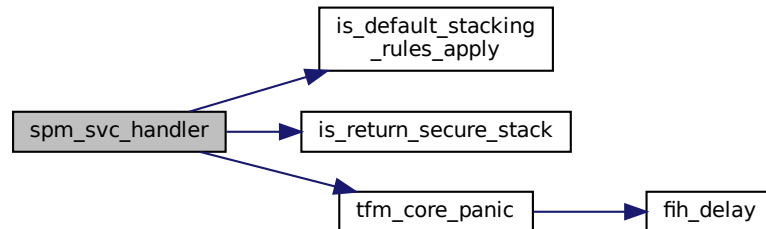
|    |                   |                              |
|----|-------------------|------------------------------|
| in | <i>msp</i>        | MSP at SVCcall entry.        |
| in | <i>exc_return</i> | EXC_RETURN value of the SVC. |
| in | <i>psp</i>        | PSP at SVCcall entry.        |

**Returns**

EXC\_RETURN value indicates where to return.

Definition at line 320 of file tfm\_svcalls.c.

Here is the call graph for this function:

**8.425.1.2 tfm\_svc\_thread\_mode\_spm\_return()**

```
void tfm_svc_thread_mode_spm_return (
 psa_status_t result)
```

Return from PSA API function executed in Thread mode. Trigger svc handler to restore thread context before entering PSA API function.

**Parameters**

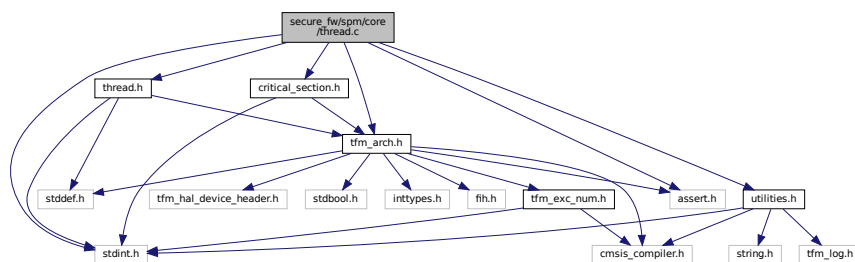
|    |        |                                |
|----|--------|--------------------------------|
| in | result | PSA API function return value. |
|----|--------|--------------------------------|

Definition at line 385 of file tfm\_svcalls.c.

**8.426 secure\_fw/spm/core/thread.c File Reference**

```
#include <stdint.h>
#include <assert.h>
#include "thread.h"
#include "tfm_arch.h"
#include "utilities.h"
#include "critical_section.h"
```

Include dependency graph for thread.c:



## Macros

- `#define LIST_HEAD p_thrd_head`
- `#define RNBL_HEAD p_rnbl_head`

## Functions

- `void thrd_set_query_callback (thrd_query_state_t fn)`
- `struct thread_t * thrd_next (void)`
- `void thrd_start (struct thread_t *p_thrd, thrd_fn_t fn, thrd_fn_t exit_fn, void *param)`
- `void thrd_set_state (struct thread_t *p_thrd, uint32_t new_state)`
- `uint32_t thrd_start_scheduler (struct thread_t **ppth)`

## Variables

- `struct thread_t * p_curr_thrd`

### 8.426.1 Macro Definition Documentation

#### 8.426.1.1 LIST\_HEAD

```
#define LIST_HEAD p_thrd_head
```

Definition at line 25 of file thread.c.

#### 8.426.1.2 RNBL\_HEAD

```
#define RNBL_HEAD p_rnbl_head
```

Definition at line 26 of file thread.c.

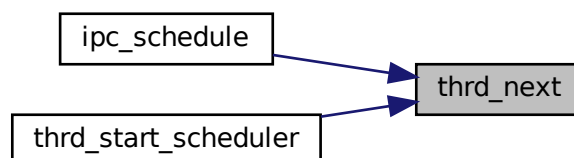
### 8.426.2 Function Documentation

#### 8.426.2.1 thrd\_next()

```
struct thread_t* thrd_next (
 void)
```

Definition at line 36 of file thread.c.

Here is the caller graph for this function:





**8.426.2.2 thrd\_set\_query\_callback()**

```
void thrd_set_query_callback (
 thrd_query_state_t fn)
```

Definition at line 31 of file thread.c.

Here is the caller graph for this function:

**8.426.2.3 thrd\_set\_state()**

```
void thrd_set_state (
 struct thread_t * p_thrd,
 uint32_t new_state)
```

Definition at line 102 of file thread.c.

**8.426.2.4 thrd\_start()**

```
void thrd_start (
 struct thread_t * p_thrd,
 thrd_fn_t fn,
 thrd_fn_t exit_fn,
 void * param)
```

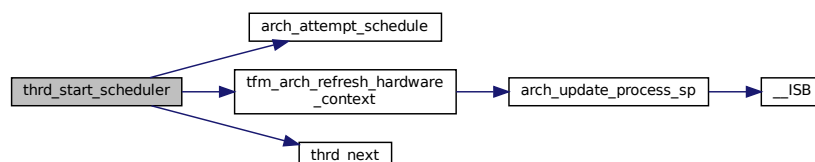
Definition at line 87 of file thread.c.

**8.426.2.5 thrd\_start\_scheduler()**

```
uint32_t thrd_start_scheduler (
 struct thread_t ** ppth)
```

Definition at line 120 of file thread.c.

Here is the call graph for this function:

**8.426.3 Variable Documentation**

### 8.426.3.1 p\_curr\_thrd

```
struct thread_t* p_curr_thrd
```

Definition at line 18 of file thread.c.

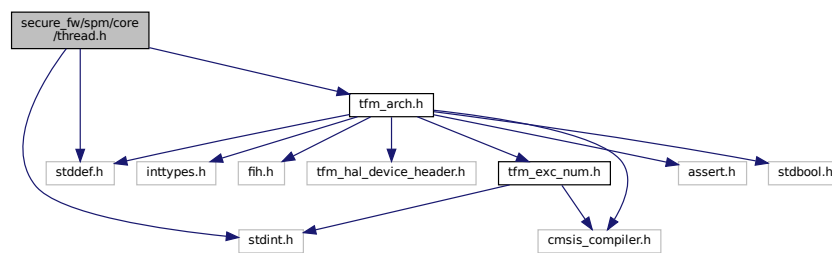
## 8.427 secure\_fw/spm/core/thread.h File Reference

```
#include <stddef.h>
```

```
#include <stdint.h>
```

```
#include "tfm_arch.h"
```

Include dependency graph for thread.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [thread\\_t](#)

## Macros

- #define [THRD\\_STATE\\_CREATING](#) 0
- #define [THRD\\_STATE\\_RUNNABLE](#) 1
- #define [THRD\\_STATE\\_BLOCK](#) 2
- #define [THRD\\_STATE\\_DETACH](#) 3
- #define [THRD\\_STATE\\_INVALID](#) 4
- #define [THRD\\_STATE\\_RET\\_VAL\\_AVAIL](#) 5
- #define [THRD\\_PRIOR\\_HIGHEST](#) 0x0
- #define [THRD\\_PRIOR\\_HIGH](#) 0xF
- #define [THRD\\_PRIOR\\_MEDIUM](#) 0x1F
- #define [THRD\\_PRIOR\\_LOW](#) 0x7F
- #define [THRD\\_PRIOR\\_LOWEST](#) 0xFF
- #define [THRD\\_SUCCESS](#) 0
- #define [THRD\\_ERR\\_GENERIC](#) 1
- #define [THRD\\_GENERAL\\_EXIT](#) (([thrd\\_fn\\_t](#))(0xFFFFFFFF))
- #define [CURRENT\\_THREAD](#) [p\\_curr\\_thrd](#)
- #define [THRD\\_INIT](#)(p\_thrd, p\_ctx\_ctrl, prio)
- #define [THRD\\_SET\\_PRIORITY](#)(p\_thrd, priority) p\_thrd->priority = (uint16\_t)(priority)
- #define [THRD\\_UPDATE\\_CUR\\_CTXCTRL](#)(x) [CURRENT\\_THREAD](#)->p\_context\_ctrl = (x)

## Typedefs

- typedef `void(* thrd_fn_t) (void *)`
- typedef `uint32_t(* thrd_query_state_t) (const struct thread_t *p_thrd, uint32_t *p_retval)`

## Functions

- `void thrd_set_query_callback (thrd_query_state_t fn)`
- `void thrd_set_state (struct thread_t *p_thrd, uint32_t new_state)`
- `void thrd_start (struct thread_t *p_thrd, thrd_fn_t fn, thrd_fn_t exit_fn, void *param)`
- `struct thread_t * thrd_next (void)`
- `uint32_t thrd_start_scheduler (struct thread_t **ppth)`

## Variables

- `struct thread_t * p_curr_thrd`

### 8.427.1 Macro Definition Documentation

#### 8.427.1.1 CURRENT\_THREAD

```
#define CURRENT_THREAD p_curr_thrd
```

Definition at line 60 of file thread.h.

#### 8.427.1.2 THRD\_ERR\_GENERIC

```
#define THRD_ERR_GENERIC 1
```

Definition at line 35 of file thread.h.

#### 8.427.1.3 THRD\_GENERAL\_EXIT

```
#define THRD_GENERAL_EXIT ((thrd_fn_t) (0xFFFFFFFF))
```

Definition at line 41 of file thread.h.

#### 8.427.1.4 THRD\_INIT

```
#define THRD_INIT(
 p_thrd,
 p_ctx_ctrl,
 prio)
```

Value:

```
do {
 (p_thrd)->priority = (uint16_t) (prio);
 (p_thrd)->state = THRD_STATE_CREATING;
 (p_thrd)->flags = 0;
 (p_thrd)->p_context_ctrl = p_ctx_ctrl;
} while (0)
```

Definition at line 70 of file thread.h.

#### 8.427.1.5 THRD\_PRIOR\_HIGH

```
#define THRD_PRIOR_HIGH 0xF
```

Definition at line 28 of file thread.h.

#### 8.427.1.6 THRD\_PRIOR\_HIGHEST

```
#define THRD_PRIOR_HIGHEST 0x0
```

Definition at line 27 of file thread.h.

#### 8.427.1.7 THRD\_PRIOR\_LOW

```
#define THRD_PRIOR_LOW 0x7F
```

Definition at line 30 of file thread.h.

#### 8.427.1.8 THRD\_PRIOR\_LOWEST

```
#define THRD_PRIOR_LOWEST 0xFF
```

Definition at line 31 of file thread.h.

#### 8.427.1.9 THRD\_PRIOR\_MEDIUM

```
#define THRD_PRIOR_MEDIUM 0x1F
```

Definition at line 29 of file thread.h.

#### 8.427.1.10 THRD\_SET\_PRIORITY

```
#define THRD_SET_PRIORITY(
 p_thrd,
 priority) p_thrd->priority = (uint16_t)(priority)
```

Definition at line 87 of file thread.h.

#### 8.427.1.11 THRD\_STATE\_BLOCK

```
#define THRD_STATE_BLOCK 2
```

Definition at line 21 of file thread.h.

#### 8.427.1.12 THRD\_STATE\_CREATING

```
#define THRD_STATE_CREATING 0
```

Definition at line 19 of file thread.h.

#### 8.427.1.13 THRD\_STATE\_DETACH

```
#define THRD_STATE_DETACH 3
```

Definition at line 22 of file thread.h.

#### 8.427.1.14 THRD\_STATE\_INVALID

```
#define THRD_STATE_INVALID 4
```

Definition at line 23 of file thread.h.

#### 8.427.1.15 THRD\_STATE\_RET\_VAL\_AVAIL

```
#define THRD_STATE_RET_VAL_AVAIL 5
```

Definition at line 24 of file thread.h.

### 8.427.1.16 THRD\_STATE\_RUNNABLE

```
#define THRD_STATE_RUNNABLE 1
```

Definition at line 20 of file thread.h.

### 8.427.1.17 THRD\_SUCCESS

```
#define THRD_SUCCESS 0
```

Definition at line 34 of file thread.h.

### 8.427.1.18 THRD\_UPDATE\_CUR\_CTXCTRL

```
#define THRD_UPDATE_CUR_CTXCTRL(
 x) CURRENT_THREAD->p_context_ctrl = (x)
```

Definition at line 96 of file thread.h.

## 8.427.2 Typedef Documentation

### 8.427.2.1 thrd\_fn\_t

```
typedef void(* thrd_fn_t) (void *)
```

Definition at line 38 of file thread.h.

### 8.427.2.2 thrd\_query\_state\_t

```
typedef uint32_t(* thrd_query_state_t) (const struct thread_t *p_thrd, uint32_t *p_retval)
```

Definition at line 53 of file thread.h.

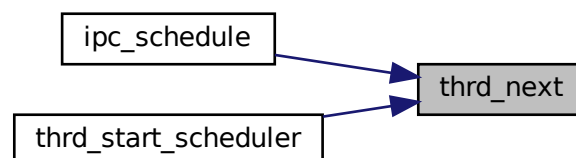
## 8.427.3 Function Documentation

### 8.427.3.1 thrd\_next()

```
struct thread_t* thrd_next (
 void)
```

Definition at line 36 of file thread.c.

Here is the caller graph for this function:



### 8.427.3.2 thrd\_set\_query\_callback()

```
void thrd_set_query_callback (
 thrd_query_state_t fn)
```

Definition at line 31 of file thread.c.

Here is the caller graph for this function:



### 8.427.3.3 thrd\_set\_state()

```
void thrd_set_state (
 struct thread_t * p_thrd,
 uint32_t new_state)
```

Definition at line 102 of file thread.c.

### 8.427.3.4 thrd\_start()

```
void thrd_start (
 struct thread_t * p_thrd,
 thrd_fn_t fn,
 thrd_fn_t exit_fn,
 void * param)
```

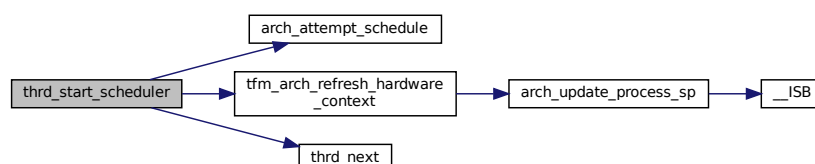
Definition at line 87 of file thread.c.

### 8.427.3.5 thrd\_start\_scheduler()

```
uint32_t thrd_start_scheduler (
 struct thread_t ** ppth)
```

Definition at line 120 of file thread.c.

Here is the call graph for this function:



## 8.427.4 Variable Documentation

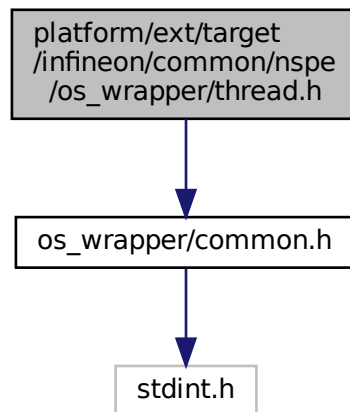
## 8.427.4.1 p\_curr\_thrd

struct `thread_t`\* p\_curr\_thrd  
 Definition at line 18 of file thread.c.

## 8.428 platform/ext/target/infineon/common/nspe/os\_wrapper/thread.h File Reference

```
#include "os_wrapper/common.h"
```

Include dependency graph for thread.h:



This graph shows which files directly or indirectly include this file:



### Typedefs

- typedef `void`(\* `os_wrapper_thread_func`) (`void` \*argument)

### Functions

- `void` \* `os_wrapper_thread_new` (`const char` \*name, `int32_t` stack\_size, `os_wrapper_thread_func` func, `void` \*arg, `uint32_t` priority)  
*Creates a new thread.*
- `void` \* `os_wrapper_thread_get_handle` (`void`)  
*Gets current thread handle.*
- `uint32_t` `os_wrapper_thread_get_priority` (`void` \*handle, `uint32_t` \*priority)  
*Gets thread priority.*
- `void` `os_wrapper_thread_exit` (`void`)  
*Exits the calling thread.*
- `uint32_t` `os_wrapper_thread_set_flag` (`void` \*handle, `uint32_t` flags)

*Set the event flags for synchronizing a thread specified by handle.*

- uint32\_t [os\\_wrapper\\_thread\\_set\\_flag\\_isr](#) (void \*handle, uint32\_t flags)

*Set the event flags in an interrupt handler for synchronizing a thread specified by handle.*

- uint32\_t [os\\_wrapper\\_thread\\_wait\\_flag](#) (uint32\_t flags, uint32\_t timeout)

*Wait for the event flags for synchronizing threads.*

## 8.428.1 Typedef Documentation

### 8.428.1.1 os\_wrapper\_thread\_func

```
typedef void(* os_wrapper_thread_func) (void *argument)
```

Definition at line 20 of file thread.h.

## 8.428.2 Function Documentation

### 8.428.2.1 os\_wrapper\_thread\_exit()

```
void os_wrapper_thread_exit (
 void)
```

Exits the calling thread.

### 8.428.2.2 os\_wrapper\_thread\_get\_handle()

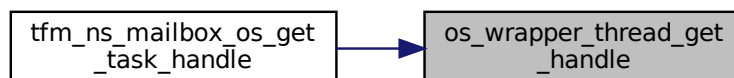
```
void* os_wrapper_thread_get_handle (
 void)
```

Gets current thread handle.

#### Returns

Returns the thread handle, or NULL in case of error

Here is the caller graph for this function:



### 8.428.2.3 os\_wrapper\_thread\_get\_priority()

```
uint32_t os_wrapper_thread_get_priority (
 void * handle,
 uint32_t * priority)
```

Gets thread priority.

#### Parameters

|     |                 |                            |
|-----|-----------------|----------------------------|
| in  | <i>handle</i>   | Thread handle              |
| out | <i>priority</i> | The priority of the thread |



**Returns**

Returns [OS\\_WRAPPER\\_SUCCESS](#) on success, or [OS\\_WRAPPER\\_ERROR](#) in case of error

**8.428.2.4 os\_wrapper\_thread\_new()**

```
void* os_wrapper_thread_new (
 const char * name,
 int32_t stack_size,
 os_wrapper_thread_func func,
 void * arg,
 uint32_t priority)
```

Creates a new thread.

**Parameters**

|    |                   |                                                                                                                                                                 |
|----|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>name</i>       | Name of the thread                                                                                                                                              |
| in | <i>stack_size</i> | Size of stack to be allocated for this thread. It can be <a href="#">OS_WRAPPER_DEFAULT_STACK_SIZE</a> to use the default value provided by the underlying RTOS |
| in | <i>func</i>       | Pointer to the function invoked by thread                                                                                                                       |
| in | <i>arg</i>        | Argument to pass to the function invoked by thread                                                                                                              |
| in | <i>priority</i>   | Initial thread priority                                                                                                                                         |

**Returns**

Returns the thread handle created, or NULL in case of error

**8.428.2.5 os\_wrapper\_thread\_set\_flag()**

```
uint32_t os_wrapper_thread_set_flag (
 void * handle,
 uint32_t flags)
```

Set the event flags for synchronizing a thread specified by handle.

**Note**

This function may not be allowed to be called from Interrupt Service Routines.

**Parameters**

|    |               |                              |
|----|---------------|------------------------------|
| in | <i>handle</i> | Thread handle to be notified |
| in | <i>flags</i>  | Event flags value            |

**Returns**

Returns [OS\\_WRAPPER\\_SUCCESS](#) on success, or [OS\\_WRAPPER\\_ERROR](#) in case of error

**8.428.2.6 os\_wrapper\_thread\_set\_flag\_isr()**

```
uint32_t os_wrapper_thread_set_flag_isr (
 void * handle,
 uint32_t flags)
```

Set the event flags in an interrupt handler for synchronizing a thread specified by handle.

## Parameters

|    |               |                              |
|----|---------------|------------------------------|
| in | <i>handle</i> | Thread handle to be notified |
| in | <i>flags</i>  | Event flags value            |

## Returns

Returns [OS\\_WRAPPER\\_SUCCESS](#) on success, or [OS\\_WRAPPER\\_ERROR](#) in case of error

Here is the caller graph for this function:



## 8.428.2.7 os\_wrapper\_thread\_wait\_flag()

```

uint32_t os_wrapper_thread_wait_flag (
 uint32_t flags,
 uint32_t timeout)

```

Wait for the event flags for synchronizing threads.

## Note

This function may not be allowed to be called from Interrupt Service Routines.

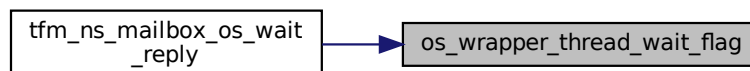
## Parameters

|    |                |                               |
|----|----------------|-------------------------------|
| in | <i>flags</i>   | Specify the flags to wait for |
| in | <i>timeout</i> | Timeout value                 |

## Returns

Returns [OS\\_WRAPPER\\_SUCCESS](#) on success, or [OS\\_WRAPPER\\_ERROR](#) in case of error

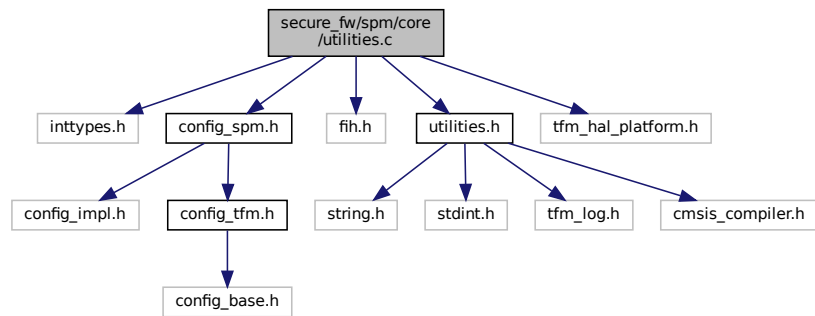
Here is the caller graph for this function:



## 8.429 secure\_fw/spm/core/utilities.c File Reference

```
#include <inttypes.h>
```

```
#include "config_spm.h"
#include "fih.h"
#include "utilities.h"
#include "tfm_hal_platform.h"
Include dependency graph for utilities.c:
```



## Functions

- [void tfm\\_core\\_panic \(void\)](#)

### 8.429.1 Function Documentation

#### 8.429.1.1 tfm\_core\_panic()

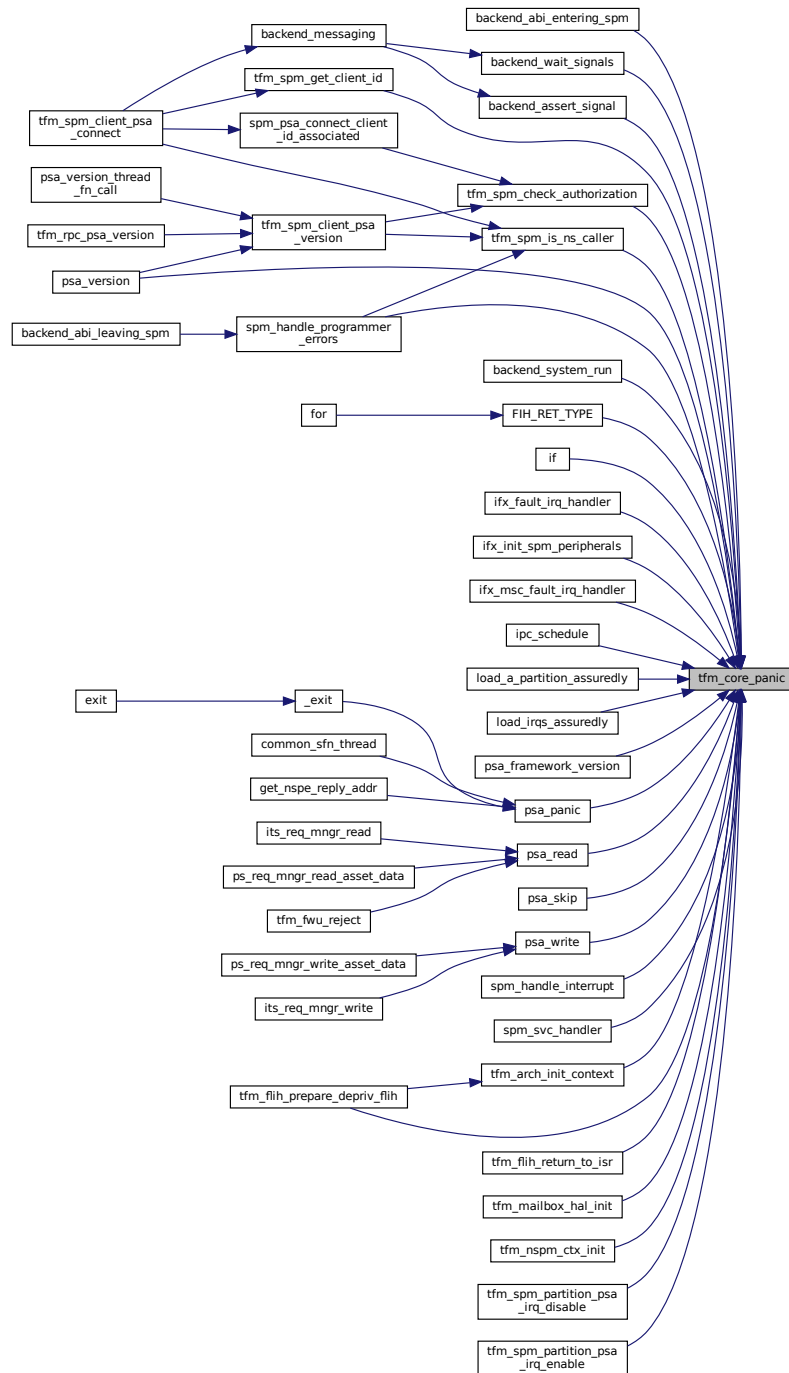
```
void tfm_core_panic (
 void)
```

Definition at line 20 of file `utilities.c`.

Here is the call graph for this function:



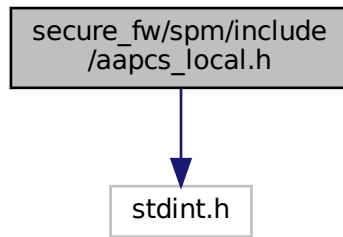
Here is the caller graph for this function:



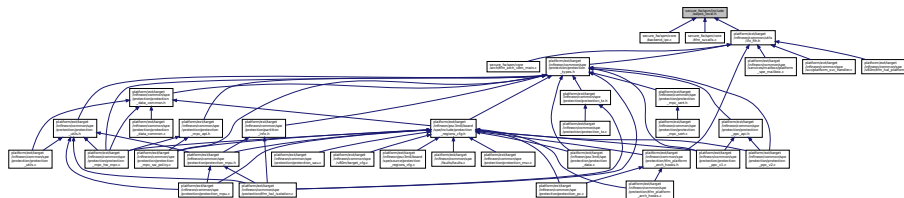
## 8.430 secure\_fw/spm/include/aapcs\_local.h File Reference

```
#include <stdint.h>
```

Include dependency graph for `aapcs_local.h`:



This graph shows which files directly or indirectly include this file:



## Data Structures

- union [u64\\_in\\_u32\\_regs\\_t](#)

## Macros

- `#define AAPCS_DUAL_U32_T union u64\_in\_u32\_regs\_t`
- `#define AAPCS_DUAL_U32_SET(v, a0, a1)`
- `#define AAPCS_DUAL_U32_SET_A0(v, a0) (v).u32_regs.r0 = (uint32_t)(a0)`
- `#define AAPCS_DUAL_U32_SET_A1(v, a1) (v).u32_regs.r1 = (uint32_t)(a1)`
- `#define AAPCS_DUAL_U32_AS_U64(v) (v).u64_val`

### 8.430.1 Macro Definition Documentation

#### 8.430.1.1 AAPCS\_DUAL\_U32\_AS\_U64

```
#define AAPCS_DUAL_U32_AS_U64(
 v) (v).u64_val
```

Definition at line 60 of file `aapcs_local.h`.

#### 8.430.1.2 AAPCS\_DUAL\_U32\_SET

```
#define AAPCS_DUAL_U32_SET(
 v,
 a0,
 a1)
```

Value:

```
do {
 (v).u32_regs.r0 = (uint32_t)(a0); \
 (v).u32_regs.r1 = (uint32_t)(a1); \
} while (0)
```

Definition at line 48 of file aapcs\_local.h.

#### 8.430.1.3 AAPCS\_DUAL\_U32\_SET\_A0

```
#define AAPCS_DUAL_U32_SET_A0(
 v,
 a0) (v).u32_regs.r0 = (uint32_t)(a0)
```

Definition at line 54 of file aapcs\_local.h.

#### 8.430.1.4 AAPCS\_DUAL\_U32\_SET\_A1

```
#define AAPCS_DUAL_U32_SET_A1(
 v,
 a1) (v).u32_regs.r1 = (uint32_t)(a1)
```

Definition at line 57 of file aapcs\_local.h.

#### 8.430.1.5 AAPCS\_DUAL\_U32\_T

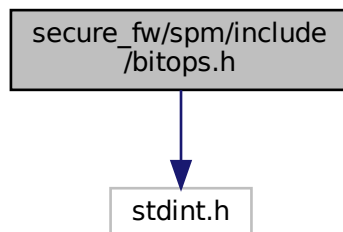
```
#define AAPCS_DUAL_U32_T union u64_in_u32_regs_t
```

Definition at line 46 of file aapcs\_local.h.

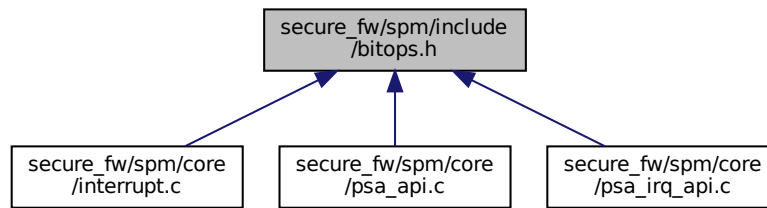
## 8.431 secure\_fw/spm/include/bitops.h File Reference

```
#include <stdint.h>
```

Include dependency graph for bitops.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define IS_ONLY_ONE_BIT_IN_UINT32(n) ((uint32_t)(n) && !((uint32_t)(n) & ((uint32_t)(n)-1)))`

## 8.431.1 Macro Definition Documentation

### 8.431.1.1 IS\_ONLY\_ONE\_BIT\_IN\_UINT32

```
#define IS_ONLY_ONE_BIT_IN_UINT32(
 n) ((uint32_t)(n) && !((uint32_t)(n) & ((uint32_t)(n)-1)))
```

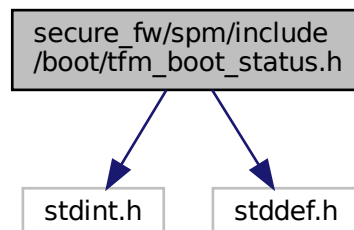
Definition at line 13 of file `bitops.h`.

## 8.432 secure\_fw/spm/include/boot/tfm\_boot\_status.h File Reference

```
#include <stdint.h>
```

```
#include <stddef.h>
```

Include dependency graph for `tfm_boot_status.h`:





[illegible]

- struct `shared_data_tlv_header`
- struct `shared_data_tlv_entry`
- struct `tfm_boot_data`

- #define TLV\_MAJOR\_CORE 0x0
- #define TLV\_MAJOR\_IAS 0x1
- #define TLV\_MAJOR\_FWU 0x2
- #define TLV\_MAJOR\_MBS 0x3
- #define TLV\_MAJOR\_INVALID 0xF
- #define SW\_GENERAL 0x00
- #define SW\_BL2 0x01
- #define SW\_PROT 0x02
- #define SW\_AROT 0x03
- #define SW\_SPE 0x04
- #define SW\_NSPE 0x05
- #define SW\_S\_NS 0x06
- #define SW\_MAX 0x07
- #define SW\_VERSION 0x00
- #define SW\_SIGNER\_ID 0x01
- #define SW\_TYPE 0x03
- #define SW\_MEASURE\_VALUE 0x08
- #define SW\_MEASURE\_TYPE 0x09
- #define SW\_MEASURE\_METADATA 0x0A
- #define SW\_MEASURE\_VALUE\_NON\_EXTENDABLE 0x0B
- #define SW\_BOOT\_RECORD 0x3F
- #define MAJOR\_MASK 0xF /\* 4 bit \*/
- #define MAJOR\_POS 12 /\* 12 bit \*/
- #define MINOR\_MASK 0xFFF /\* 12 bit \*/
- #define MINOR\_POS 0
- #define MASK\_LEFT\_SHIFT(data, mask, position) (((data) & (mask)) << (position))
- #define MASK\_RIGHT\_SHIFT(data, mask, position) ((uint16\_t)((data) & (mask)) >> (position))
- #define SET\_TLV\_TYPE(major, minor)
- #define GET\_MAJOR(tlv\_type) ((tlv\_type) >> MAJOR\_POS)
- #define GET\_MINOR(tlv\_type) ((tlv\_type) & MINOR\_MASK)
- #define MODULE\_POS 6 /\* 6 bit \*/
- #define MODULE\_MASK 0x3F /\* 6 bit \*/
- #define CLAIM\_POS 0
- #define CLAIM\_MASK 0x3F /\* 6 bit \*/
- #define MEASUREMENT\_CLAIM\_POS 3 /\* 3 bit \*/
- #define GET\_IAS\_MODULE(tlv\_type) MASK\_RIGHT\_SHIFT(tlv\_type, MINOR\_MASK, MODULE\_POS)

- `#define GET_IAS_CLAIM(tlv_type) MASK_RIGHT_SHIFT(tlv_type, CLAIM_MASK, CLAIM_POS)`
- `#define GET_IAS_MEASUREMENT_CLAIM(ias_claim) MASK_RIGHT_SHIFT(ias_claim, CLAIM_MASK, MEASUREMENT_CLAIM_POS)`
- `#define SET_IAS_MINOR(sw_module, claim)`
- `#define SLOT_ID_POS 6`
- `#define SLOT_ID_MASK 0x3F /* 6 bit */`
- `#define GET_MBS_SLOT(tlv_type) MASK_RIGHT_SHIFT(tlv_type, MINOR_MASK, SLOT_ID_POS)`
- `#define GET_MBS_CLAIM(tlv_type) MASK_RIGHT_SHIFT(tlv_type, CLAIM_MASK, CLAIM_POS)`
- `#define SET_MBS_MINOR(slot_index, claim)`
- `#define GET_FWU_MODULE(tlv_type) MASK_RIGHT_SHIFT(tlv_type, MINOR_MASK, MODULE_POS)`
- `#define GET_FWU_CLAIM(tlv_type) MASK_RIGHT_SHIFT(tlv_type, CLAIM_MASK, CLAIM_POS)`
- `#define SET_FWU_MINOR(sw_module, claim)`
- `#define SHARED_DATA_TLV_INFO_MAGIC 0x2016`
- `#define SHARED_DATA_HEADER_SIZE sizeof(struct shared_data_tlv_header)`
- `#define SHARED_DATA_ENTRY_HEADER_SIZE sizeof(struct shared_data_tlv_entry)`
- `#define SHARED_DATA_ENTRY_SIZE(size) (size + SHARED_DATA_ENTRY_HEADER_SIZE)`

## 8.432.1 Macro Definition Documentation

### 8.432.1.1 CLAIM\_MASK

```
#define CLAIM_MASK 0x3F /* 6 bit */
```

Definition at line 117 of file tfm\_boot\_status.h.

### 8.432.1.2 CLAIM\_POS

```
#define CLAIM_POS 0
```

Definition at line 116 of file tfm\_boot\_status.h.

### 8.432.1.3 GET\_FWU\_CLAIM

```
#define GET_FWU_CLAIM(
 tlv_type) MASK_RIGHT_SHIFT(tlv_type, CLAIM_MASK, CLAIM_POS)
```

Definition at line 147 of file tfm\_boot\_status.h.

### 8.432.1.4 GET\_FWU\_MODULE

```
#define GET_FWU_MODULE(
 tlv_type) MASK_RIGHT_SHIFT(tlv_type, MINOR_MASK, MODULE_POS)
```

Definition at line 145 of file tfm\_boot\_status.h.

### 8.432.1.5 GET\_IAS\_CLAIM

```
#define GET_IAS_CLAIM(
 tlv_type) MASK_RIGHT_SHIFT(tlv_type, CLAIM_MASK, CLAIM_POS)
```

Definition at line 124 of file tfm\_boot\_status.h.

### 8.432.1.6 GET\_IAS\_MEASUREMENT\_CLAIM

```
#define GET_IAS_MEASUREMENT_CLAIM(
 ias_claim) MASK_RIGHT_SHIFT(ias_claim, CLAIM_MASK, MEASUREMENT_CLAIM_POS)
```

Definition at line 126 of file tfm\_boot\_status.h.

#### 8.432.1.7 GET\_IAS\_MODULE

```
#define GET_IAS_MODULE(
 tlv_type) MASK_RIGHT_SHIFT(tlv_type, MINOR_MASK, MODULE_POS)
```

Definition at line 122 of file tfm\_boot\_status.h.

#### 8.432.1.8 GET\_MAJOR

```
#define GET_MAJOR(
 tlv_type) ((tlv_type) >> MAJOR_POS)
```

Definition at line 110 of file tfm\_boot\_status.h.

#### 8.432.1.9 GET\_MBS\_CLAIM

```
#define GET_MBS_CLAIM(
 tlv_type) MASK_RIGHT_SHIFT(tlv_type, CLAIM_MASK, CLAIM_POS)
```

Definition at line 138 of file tfm\_boot\_status.h.

#### 8.432.1.10 GET\_MBS\_SLOT

```
#define GET_MBS_SLOT(
 tlv_type) MASK_RIGHT_SHIFT(tlv_type, MINOR_MASK, SLOT_ID_POS)
```

Definition at line 136 of file tfm\_boot\_status.h.

#### 8.432.1.11 GET\_MINOR

```
#define GET_MINOR(
 tlv_type) ((tlv_type) & MINOR_MASK)
```

Definition at line 111 of file tfm\_boot\_status.h.

#### 8.432.1.12 MAJOR\_MASK

```
#define MAJOR_MASK 0xF /* 4 bit */
```

Definition at line 97 of file tfm\_boot\_status.h.

#### 8.432.1.13 MAJOR\_POS

```
#define MAJOR_POS 12 /* 12 bit */
```

Definition at line 98 of file tfm\_boot\_status.h.

#### 8.432.1.14 MASK\_LEFT\_SHIFT

```
#define MASK_LEFT_SHIFT(
 data,
 mask,
 position) (((data) & (mask)) << (position))
```

Definition at line 102 of file tfm\_boot\_status.h.

#### 8.432.1.15 MASK\_RIGHT\_SHIFT

```
#define MASK_RIGHT_SHIFT(
 data,
```

```

 mask,
 position) ((uint16_t)((data) & (mask)) >> (position))

```

Definition at line 104 of file tfm\_boot\_status.h.

#### 8.432.1.16 MEASUREMENT\_CLAIM\_POS

```
#define MEASUREMENT_CLAIM_POS 3 /* 3 bit */
```

Definition at line 120 of file tfm\_boot\_status.h.

#### 8.432.1.17 MINOR\_MASK

```
#define MINOR_MASK 0xFFF /* 12 bit */
```

Definition at line 99 of file tfm\_boot\_status.h.

#### 8.432.1.18 MINOR\_POS

```
#define MINOR_POS 0
```

Definition at line 100 of file tfm\_boot\_status.h.

#### 8.432.1.19 MODULE\_MASK

```
#define MODULE_MASK 0x3F /* 6 bit */
```

Definition at line 115 of file tfm\_boot\_status.h.

#### 8.432.1.20 MODULE\_POS

```
#define MODULE_POS 6 /* 6 bit */
```

Definition at line 114 of file tfm\_boot\_status.h.

#### 8.432.1.21 SET\_FWU\_MINOR

```
#define SET_FWU_MINOR(
 sw_module,
 claim)
```

**Value:**

```

 (MASK_LEFT_SHIFT(sw_module, MODULE_MASK, MODULE_POS) | \
 MASK_LEFT_SHIFT(claim, CLAIM_MASK, CLAIM_POS))

```

Definition at line 149 of file tfm\_boot\_status.h.

#### 8.432.1.22 SET\_IAS\_MINOR

```
#define SET_IAS_MINOR(
 sw_module,
 claim)
```

**Value:**

```

 (MASK_LEFT_SHIFT(sw_module, MODULE_MASK, MODULE_POS) | \
 MASK_LEFT_SHIFT(claim, CLAIM_MASK, CLAIM_POS))

```

Definition at line 128 of file tfm\_boot\_status.h.

#### 8.432.1.23 SET\_MBS\_MINOR

```
#define SET_MBS_MINOR(
 slot_index,
 claim)
```

**Value:**

```
(MASK_LEFT_SHIFT(slot_index, SLOT_ID_MASK, SLOT_ID_POS) | \
 MASK_LEFT_SHIFT(claim, CLAIM_MASK, CLAIM_POS))
```

Definition at line 140 of file tfm\_boot\_status.h.

#### 8.432.1.24 SET\_TLV\_TYPE

```
#define SET_TLV_TYPE(
 major,
 minor)
```

**Value:**

```
(MASK_LEFT_SHIFT(major, MAJOR_MASK, MAJOR_POS) | \
 MASK_LEFT_SHIFT(minor, MINOR_MASK, MINOR_POS))
```

Definition at line 107 of file tfm\_boot\_status.h.

#### 8.432.1.25 SHARED\_DATA\_ENTRY\_HEADER\_SIZE

```
#define SHARED_DATA_ENTRY_HEADER_SIZE sizeof(struct shared_data_tlv_entry)
```

Definition at line 194 of file tfm\_boot\_status.h.

#### 8.432.1.26 SHARED\_DATA\_ENTRY\_SIZE

```
#define SHARED_DATA_ENTRY_SIZE(
 size) (size + SHARED_DATA_ENTRY_HEADER_SIZE)
```

Definition at line 195 of file tfm\_boot\_status.h.

#### 8.432.1.27 SHARED\_DATA\_HEADER\_SIZE

```
#define SHARED_DATA_HEADER_SIZE sizeof(struct shared_data_tlv_header)
```

Definition at line 168 of file tfm\_boot\_status.h.

#### 8.432.1.28 SHARED\_DATA\_TLV\_INFO\_MAGIC

```
#define SHARED_DATA_TLV_INFO_MAGIC 0x2016
```

Definition at line 154 of file tfm\_boot\_status.h.

#### 8.432.1.29 SLOT\_ID\_MASK

```
#define SLOT_ID_MASK 0x3F /* 6 bit */
```

Definition at line 134 of file tfm\_boot\_status.h.

#### 8.432.1.30 SLOT\_ID\_POS

```
#define SLOT_ID_POS 6
```

Definition at line 133 of file tfm\_boot\_status.h.

**8.432.1.31 SW\_AROT**

```
#define SW_AROT 0x03
```

Definition at line 77 of file tfm\_boot\_status.h.

**8.432.1.32 SW\_BL2**

```
#define SW_BL2 0x01
```

Definition at line 75 of file tfm\_boot\_status.h.

**8.432.1.33 SW\_BOOT\_RECORD**

```
#define SW_BOOT_RECORD 0x3F
```

Definition at line 94 of file tfm\_boot\_status.h.

**8.432.1.34 SW\_GENERAL**

```
#define SW_GENERAL 0x00
```

The shared data between boot loader and runtime SW is TLV encoded. The shared data is stored in a well known location in secure memory and this is a contract between boot loader and runtime SW.

The structure of shared data must be the following:

- At the beginning there must be a header: struct [shared\\_data\\_tlv\\_header](#) This contains a magic number and a size field which covers the entire size of the shared data area including this header.
- After the header there come the entries which are composed from an entry header structure: struct [shared\\_data\\_tlv\\_entry](#) and the data. In the entry header is a type field (tlv\_type) which identify the consumer of the entry in the runtime SW and specify the subtype of that data item. There is a size field (tlv\_len) which covers the length of the data (not including the entry header structure). After this structure comes the actual data.
- Arbitrary number and size of data entry can be in the shared memory area.

This table gives of overview about the tlv\_type field in the entry header. The tlv\_type always composed from a major and minor number. Major number identifies the addressee in runtime SW, who should process the data entry. Minor number used to encode more info about the data entry. The actual definition of minor number could change per major number. In case of boot status data, which is going to be processed by initial attestation service the minor number is split further to two part: sw\_module and claim. The sw\_module identifies the SW component in the system which the data item belongs to and the claim part identifies the exact type of the data.

|             |  |               |              |          |              |               |           |
|-------------|--|---------------|--------------|----------|--------------|---------------|-----------|
| -----       |  | tlv_type (16) | -----        |          | tlv_major(4) | tlv_minor(12) | -----     |
| -----       |  | MAJOR_IAS     | sw_module(6) | claim(6) | -----        |               | MAJOR_MBS |
| slot ID (6) |  | claim(6)      | -----        |          | MAJOR_CORE   | TBD           | -----     |

Definition at line 74 of file tfm\_boot\_status.h.

**8.432.1.35 SW\_MAX**

```
#define SW_MAX 0x07
```

Definition at line 81 of file tfm\_boot\_status.h.

**8.432.1.36 SW\_MEASURE\_METADATA**

```
#define SW_MEASURE_METADATA 0x0A
```

Definition at line 92 of file tfm\_boot\_status.h.

**8.432.1.37 SW\_MEASURE\_TYPE**

```
#define SW_MEASURE_TYPE 0x09
```

Definition at line 91 of file tfm\_boot\_status.h.

**8.432.1.38 SW\_MEASURE\_VALUE**

```
#define SW_MEASURE_VALUE 0x08
```

Definition at line 90 of file tfm\_boot\_status.h.

**8.432.1.39 SW\_MEASURE\_VALUE\_NON\_EXTENDABLE**

```
#define SW_MEASURE_VALUE_NON_EXTENDABLE 0x0B
```

Definition at line 93 of file tfm\_boot\_status.h.

**8.432.1.40 SW\_NSPE**

```
#define SW_NSPE 0x05
```

Definition at line 79 of file tfm\_boot\_status.h.

**8.432.1.41 SW\_PROT**

```
#define SW_PROT 0x02
```

Definition at line 76 of file tfm\_boot\_status.h.

**8.432.1.42 SW\_S\_NS**

```
#define SW_S_NS 0x06
```

Definition at line 80 of file tfm\_boot\_status.h.

**8.432.1.43 SW\_SIGNER\_ID**

```
#define SW_SIGNER_ID 0x01
```

Definition at line 86 of file tfm\_boot\_status.h.

**8.432.1.44 SW\_SPE**

```
#define SW_SPE 0x04
```

Definition at line 78 of file tfm\_boot\_status.h.

**8.432.1.45 SW\_TYPE**

```
#define SW_TYPE 0x03
```

Definition at line 88 of file tfm\_boot\_status.h.

**8.432.1.46 SW\_VERSION**

```
#define SW_VERSION 0x00
```

Definition at line 85 of file tfm\_boot\_status.h.

#### 8.432.1.47 TLV\_MAJOR\_CORE

```
#define TLV_MAJOR_CORE 0x0
```

Definition at line 22 of file tfm\_boot\_status.h.

#### 8.432.1.48 TLV\_MAJOR\_FWU

```
#define TLV_MAJOR_FWU 0x2
```

Definition at line 24 of file tfm\_boot\_status.h.

#### 8.432.1.49 TLV\_MAJOR\_IAS

```
#define TLV_MAJOR_IAS 0x1
```

Definition at line 23 of file tfm\_boot\_status.h.

#### 8.432.1.50 TLV\_MAJOR\_INVALID

```
#define TLV_MAJOR_INVALID 0xF
```

Definition at line 26 of file tfm\_boot\_status.h.

#### 8.432.1.51 TLV\_MAJOR\_MBS

```
#define TLV_MAJOR_MBS 0x3
```

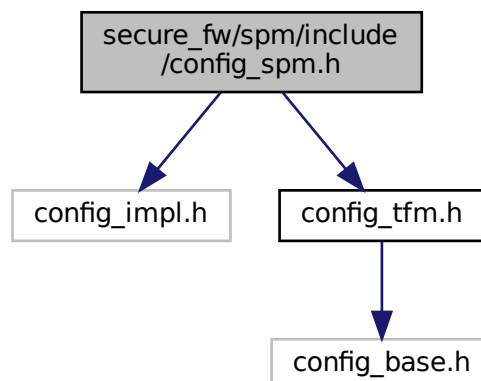
Definition at line 25 of file tfm\_boot\_status.h.

### 8.433 secure\_fw/spm/include/config\_spm.h File Reference

```
#include "config_impl.h"
```

```
#include "config_tfm.h"
```

Include dependency graph for config\_spm.h:





This graph shows which files directly or indirectly include this file:

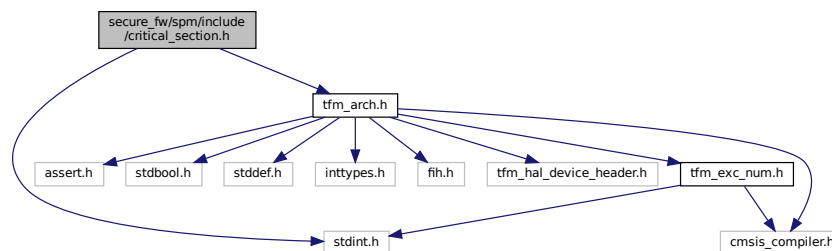


## 8.434 secure\_fw/spm/include/critical\_section.h File Reference

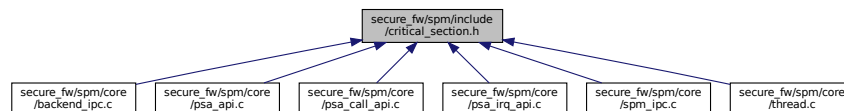
```
#include <stdint.h>
```

```
#include "tfm_arch.h"
```

Include dependency graph for critical\_section.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [critical\\_section\\_t](#)

## Macros

- #define [CRITICAL\\_SECTION\\_STATIC\\_INIT](#) { .state = 0, }
- #define [CRITICAL\\_SECTION\\_INIT](#)(cs) (cs).state = (0)
- #define [CRITICAL\\_SECTION\\_ENTER](#)(cs) (cs).state = [\\_\\_save\\_disable\\_irq\(\)](#)
- #define [CRITICAL\\_SECTION\\_LEAVE](#)(cs) [\\_\\_restore\\_irq](#)((cs).state)

### 8.434.1 Macro Definition Documentation

#### 8.434.1.1 CRITICAL\_SECTION\_ENTER

```
#define CRITICAL_SECTION_ENTER(
 cs) (cs).state = __save_disable_irq\(\)
```

Definition at line 20 of file critical\_section.h.





### 8.436.1.1 PARTITION\_LIST\_ADDR

```
#define PARTITION_LIST_ADDR (&partition_listhead)
```

Definition at line 60 of file backend.h.

### 8.436.1.2 SPM\_THREAD\_CONTEXT

```
#define SPM_THREAD_CONTEXT p_spm_thread_context
```

Definition at line 64 of file backend.h.

## 8.436.2 Function Documentation

### 8.436.2.1 backend\_assert\_signal()

```
psa_status_t backend_assert_signal (
 struct partition_t * p_pt,
 psa_signal_t signal)
```

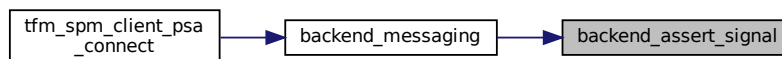
Set the asserted signal pattern in current partition.

Definition at line 481 of file backend\_ipc.c.

Here is the call graph for this function:



Here is the caller graph for this function:

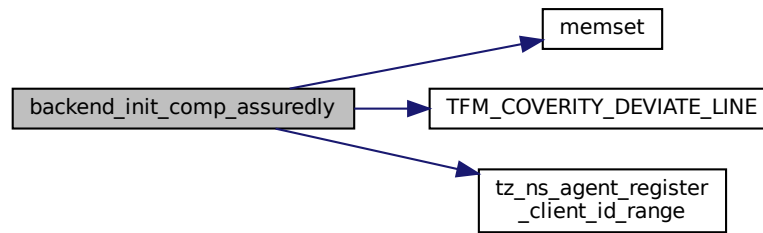


### 8.436.2.2 backend\_init\_comp\_assuredly()

```
void backend_init_comp_assuredly (
 struct partition_t * p_pt,
 uint32_t service_setting)
```

Definition at line 329 of file backend\_ipc.c.

Here is the call graph for this function:

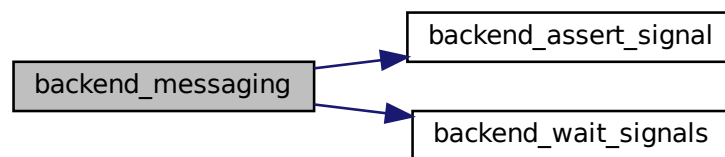


### 8.436.2.3 backend\_messaging()

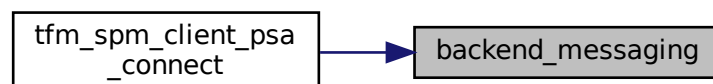
```
psa_status_t backend_messaging (
 struct connection_t * p_connection)
```

Definition at line 193 of file `backend_ipc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.436.2.4 backend\_replying()

```
psa_status_t backend_replying (
 struct connection_t * handle,
 int32_t status)
```

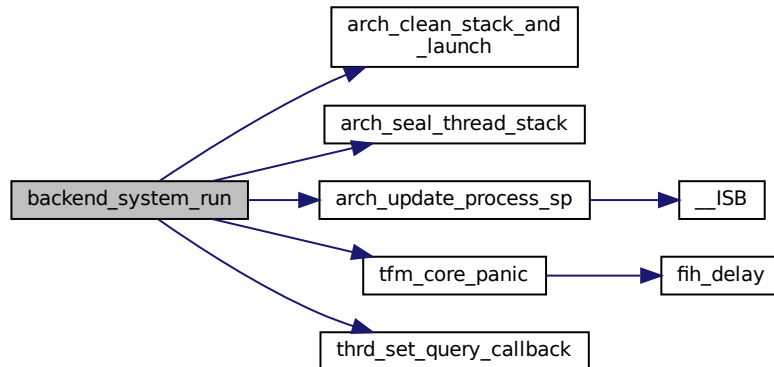
Definition at line 232 of file backend\_ipc.c.

#### 8.436.2.5 backend\_system\_run()

```
uint32_t backend_system_run (
 void)
```

Definition at line 361 of file backend\_ipc.c.

Here is the call graph for this function:



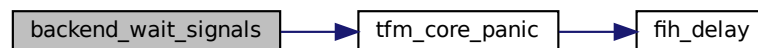
#### 8.436.2.6 backend\_wait\_signals()

```
psa_signal_t backend_wait_signals (
 struct partition_t * p_pt,
 psa_signal_t signals)
```

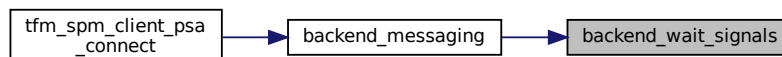
Set the wait signal pattern in current partition.

Definition at line 397 of file backend\_ipc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.436.3 Variable Documentation

#### 8.436.3.1 p\_spm\_thread\_context

struct [context\\_ctrl\\_t](#)\* p\_spm\_thread\_context  
Definition at line 57 of file backend\_ipc.c.

#### 8.436.3.2 partition\_listhead

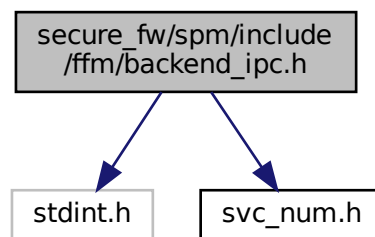
struct [partition\\_head\\_t](#) partition\_listhead  
Definition at line 45 of file backend\_ipc.c.

## 8.437 secure\_fw/spm/include/ffm/backend\_ipc.h File Reference

```
#include <stdint.h>
```

```
#include "svc_num.h"
```

Include dependency graph for backend\_ipc.h:



### Macros

- #define [BACKEND\\_SERVICE\\_SET](#)(set, p\_service) ((set) |= (p\_service)->signal)
- #define [BACKEND\\_SPM\\_INIT](#)()

### Functions

- uint64\_t [backend\\_abi\\_entering\\_spm](#) (void)
- uint32\_t [backend\\_abi\\_leaving\\_spm](#) (uint32\_t result)

### 8.437.1 Macro Definition Documentation

#### 8.437.1.1 BACKEND\_SERVICE\_SET

```
#define BACKEND_SERVICE_SET(
 set,
 p_service) ((set) |= (p_service)->signal)
```

Definition at line 15 of file backend\_ipc.h.

### 8.437.1.2 BACKEND\_SPM\_INIT

```
#define BACKEND_SPM_INIT()
```

**Value:**

```
__ASM volatile("SVC %0\n" \
 "BX LR\n" \
 : : "I" (TFM_SVC_SPM_INIT))
```

Definition at line 18 of file backend\_ipc.h.

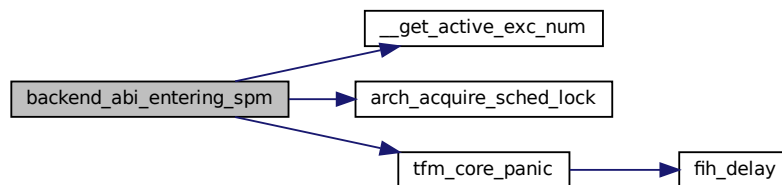
## 8.437.2 Function Documentation

### 8.437.2.1 backend\_abi\_entering\_spm()

```
uint64_t backend_abi_entering_spm (
 void)
```

Definition at line 507 of file backend\_ipc.c.

Here is the call graph for this function:

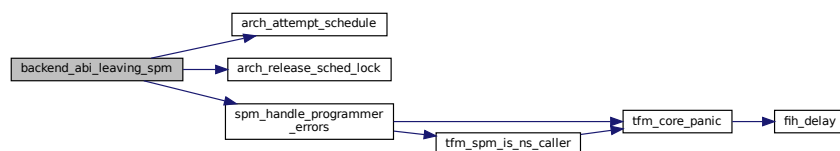


### 8.437.2.2 backend\_abi\_leaving\_spm()

```
uint32_t backend_abi_leaving_spm (
 uint32_t result)
```

Definition at line 538 of file backend\_ipc.c.

Here is the call graph for this function:



## 8.438 secure\_fw/spm/include/ffm/backend\_sfn.h File Reference

### Macros

- #define `BACKEND_SERVICE_SET`(set, p\_service)
- #define `BACKEND_SPM_INIT`() `tfm_spm_init`()



## 8.438.1 Macro Definition Documentation

### 8.438.1.1 BACKEND\_SERVICE\_SET

```
#define BACKEND_SERVICE_SET(
 set,
 p_service)
```

Definition at line 12 of file backend\_sfn.h.

### 8.438.1.2 BACKEND\_SPM\_INIT

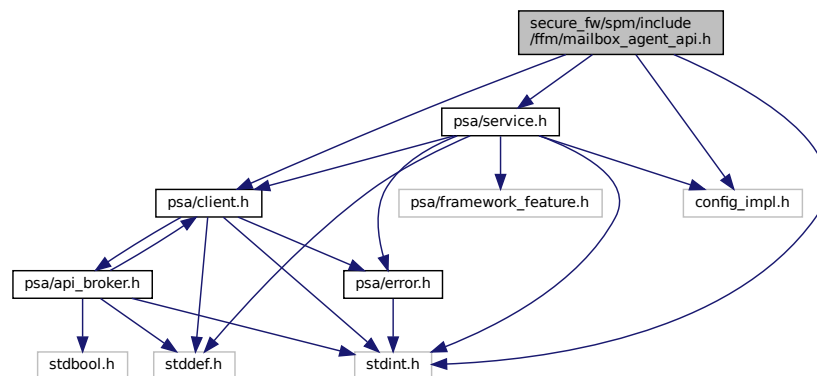
```
#define BACKEND_SPM_INIT() tfm_spm_init()
```

Definition at line 14 of file backend\_sfn.h.

## 8.439 secure\_fw/spm/include/ffm/mailbox\_agent\_api.h File Reference

```
#include <stdint.h>
#include "config_impl.h"
#include "psa/client.h"
#include "psa/service.h"
```

Include dependency graph for mailbox\_agent\_api.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [client\\_params\\_t](#)

## Macros

- #define [agent\\_psa\\_connect](#) NULL
- #define [agent\\_psa\\_close](#) NULL

## Functions

- `psa_status_t agent_psa_call` (`psa_handle_t` handle, `uint32_t` control, const struct `client_params_t` \*params, const `void` \*client\_data\_stateless)

Specific `psa_call()` variants for agents.

## 8.439.1 Macro Definition Documentation

### 8.439.1.1 agent\_psa\_close

```
#define agent_psa_close NULL
```

Definition at line 92 of file mailbox\_agent\_api.h.

### 8.439.1.2 agent\_psa\_connect

```
#define agent_psa_connect NULL
```

Definition at line 91 of file mailbox\_agent\_api.h.

## 8.439.2 Function Documentation

### 8.439.2.1 agent\_psa\_call()

```
psa_status_t agent_psa_call (
 psa_handle_t handle,
 uint32_t control,
 const struct client_params_t * params,
 const void * client_data_stateless)
```

Specific `psa_call()` variants for agents.

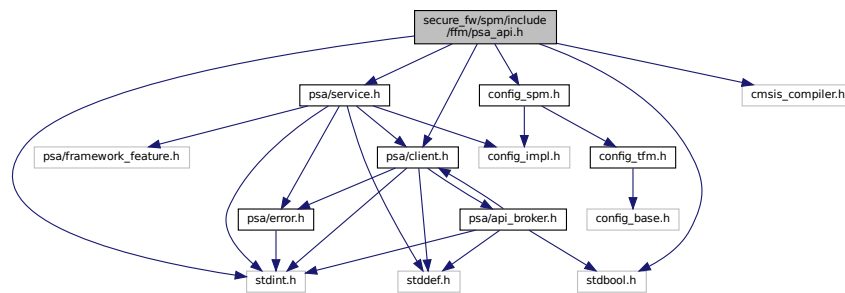
#### Parameters

|    |                              |                                                                                                                                                                                                             |
|----|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>handle</i>                | Handle to the service being accessed.                                                                                                                                                                       |
| in | <i>control</i>               | A composited <code>uint32_t</code> value for controlling purpose, containing call types, numbers of in/out vectors and attributes of vectors.                                                               |
| in | <i>params</i>                | Combines the <code>psa_invec</code> and <code>psa_outvec</code> params for the <code>psa_call()</code> to be made, as well as NS agent's client identifier, which is ignored for connection-based services. |
| in | <i>client_data_stateless</i> | Client data, treated as opaque by SPM.                                                                                                                                                                      |

#### Return values

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>     | Success.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <i>Does not return</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• An invalid handle was passed.</li> <li>• The connection is already handling a request.</li> <li>• An invalid memory reference was provided.</li> <li>• <code>in_num + out_num &gt; PSA_MAX_IOVEC</code>.</li> <li>• The message is unrecognized by the RoT Service or incorrectly formatted.</li> </ul> |

```
#include <stdint.h>
#include <stdbool.h>
#include "config_spm.h"
#include "cmsis_compiler.h"
#include "psa/client.h"
#include "psa/service.h"
Include dependency graph for psa_api.h:
```

[illegible]

- #define `tfm_spm_agent_psa_connect` NULL
- #define `tfm_spm_agent_psa_close` NULL
- #define `tfm_spm_agent_psa_call` NULL
- #define `tfm_spm_client_psa_connect` NULL
- #define `tfm_spm_client_psa_close` NULL
- #define `tfm_spm_partition_psa_notify` NULL
- #define `tfm_spm_partition_psa_clear` NULL
- #define `tfm_spm_partition_psa_set_rhandle` NULL
- #define `tfm_spm_partition_psa_irq_enable` NULL
- #define `tfm_spm_partition_psa_irq_disable` NULL
- #define `tfm_spm_partition_psa_reset_signal` NULL
- #define `tfm_spm_partition_psa_eoi` NULL

- `void spm_handle_programmer_errors (psa_status_t status)`  
*This function handles the specific programmer error cases.*
- `uint32_t tfm_spm_get_lifecycle_state (void)`  
*This function get the current PSA RoT lifecycle state.*
- `uint32_t tfm_spm_client_psa_framework_version (void)`  
*handler for psa\_framework\_version.*
- `uint32_t tfm_spm_client_psa_version (uint32_t sid)`  
*handler for psa version.*

- `psa_status_t tfm_spm_client_psa_call` (`psa_handle_t` handle, `uint32_t` ctrl\_param, const `psa_invec` \*inptr, `psa_outvec` \*outptr)  
*handler for `psa_call`.*
- `size_t tfm_spm_partition_psa_read` (`psa_handle_t` msg\_handle, `uint32_t` invec\_idx, `void` \*buffer, `size_t` num\_bytes)  
*Function body of `psa_read`.*
- `size_t tfm_spm_partition_psa_skip` (`psa_handle_t` msg\_handle, `uint32_t` invec\_idx, `size_t` num\_bytes)  
*Function body of `psa_skip`.*
- `psa_status_t tfm_spm_partition_psa_write` (`psa_handle_t` msg\_handle, `uint32_t` outvec\_idx, const `void` \*buffer, `size_t` num\_bytes)  
*Function body of `psa_write`.*
- `psa_status_t tfm_spm_partition_psa_reply` (`psa_handle_t` msg\_handle, `psa_status_t` status)  
*Function body of `psa_reply`.*
- `__NO_RETURN void tfm_spm_partition_psa_panic` (`void`)  
*Function body of `psa_panic`.*

## 8.440.1 Macro Definition Documentation

### 8.440.1.1 tfm\_spm\_agent\_psa\_call

```
#define tfm_spm_agent_psa_call NULL
```

Definition at line 156 of file `psa_api.h`.

### 8.440.1.2 tfm\_spm\_agent\_psa\_close

```
#define tfm_spm_agent_psa_close NULL
```

Definition at line 155 of file `psa_api.h`.

### 8.440.1.3 tfm\_spm\_agent\_psa\_connect

```
#define tfm_spm_agent_psa_connect NULL
```

Definition at line 154 of file `psa_api.h`.

### 8.440.1.4 tfm\_spm\_client\_psa\_close

```
#define tfm_spm_client_psa_close NULL
```

Definition at line 266 of file `psa_api.h`.

### 8.440.1.5 tfm\_spm\_client\_psa\_connect

```
#define tfm_spm_client_psa_connect NULL
```

Definition at line 265 of file `psa_api.h`.

### 8.440.1.6 tfm\_spm\_partition\_psa\_clear

```
#define tfm_spm_partition_psa_clear NULL
```

Definition at line 429 of file `psa_api.h`.

#### 8.440.1.7 tfm\_spm\_partition\_psa\_eoi

```
#define tfm_spm_partition_psa_eoi NULL
```

Definition at line 536 of file psa\_api.h.

#### 8.440.1.8 tfm\_spm\_partition\_psa\_irq\_disable

```
#define tfm_spm_partition_psa_irq_disable NULL
```

Definition at line 493 of file psa\_api.h.

#### 8.440.1.9 tfm\_spm\_partition\_psa\_irq\_enable

```
#define tfm_spm_partition_psa_irq_enable NULL
```

Definition at line 492 of file psa\_api.h.

#### 8.440.1.10 tfm\_spm\_partition\_psa\_notify

```
#define tfm_spm_partition_psa_notify NULL
```

Definition at line 428 of file psa\_api.h.

#### 8.440.1.11 tfm\_spm\_partition\_psa\_reset\_signal

```
#define tfm_spm_partition_psa_reset_signal NULL
```

Definition at line 516 of file psa\_api.h.

#### 8.440.1.12 tfm\_spm\_partition\_psa\_set\_rhandle

```
#define tfm_spm_partition_psa_set_rhandle NULL
```

Definition at line 455 of file psa\_api.h.

### 8.440.2 Function Documentation

#### 8.440.2.1 spm\_handle\_programmer\_errors()

```
void spm_handle_programmer_errors (
 psa_status_t status)
```

This function handles the specific programmer error cases.

##### Parameters

|    |               |                                   |
|----|---------------|-----------------------------------|
| in | <i>status</i> | Standard error codes for the SPM. |
|----|---------------|-----------------------------------|

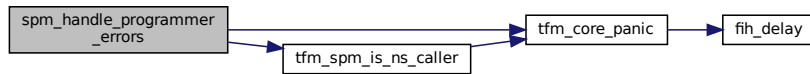
##### Note

This call will panic if the SP operation resulted in:

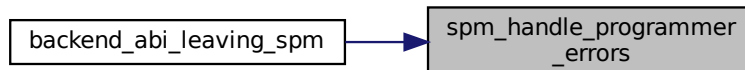
- PSA\_ERROR\_PROGRAMMER\_ERROR
- PSA\_ERROR\_CONNECTION\_REFUSED

Definition at line 35 of file psa\_api.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.440.2.2 tfm\_spm\_client\_psa\_call()

```

psa_status_t tfm_spm_client_psa_call (
 psa_handle_t handle,
 uint32_t ctrl_param,
 const psa_invec * inptr,
 psa_outvec * outptr)

```

handler for [psa\\_call](#).

##### Parameters

|    |                   |                                                                                                                      |
|----|-------------------|----------------------------------------------------------------------------------------------------------------------|
| in | <i>handle</i>     | Service handle to the established connection, <a href="#">psa_handle_t</a>                                           |
| in | <i>ctrl_param</i> | Parameters combined in <code>uint32_t</code> , includes request type, <code>in_num</code> and <code>out_num</code> . |
| in | <i>inptr</i>      | Array of input <a href="#">psa_invec</a> structures. <a href="#">psa_invec</a>                                       |
| in | <i>outptr</i>     | Array of output <a href="#">psa_outvec</a> structures. <a href="#">psa_outvec</a>                                    |

##### Return values

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>     | Success.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <i>Does not return</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• An invalid handle was passed.</li> <li>• The connection is already handling a request.</li> <li>• An invalid memory reference was provided.</li> <li>• <code>in_num + out_num &gt; PSA_MAX_IOVEC</code>.</li> <li>• The message is unrecognized by the RoT Service or incorrectly formatted.</li> </ul> |

Definition at line 167 of file `psa_call_api.c`.

**8.440.2.3 tfm\_spm\_client\_psa\_framework\_version()**

```
uint32_t tfm_spm_client_psa_framework_version (
 void)
```

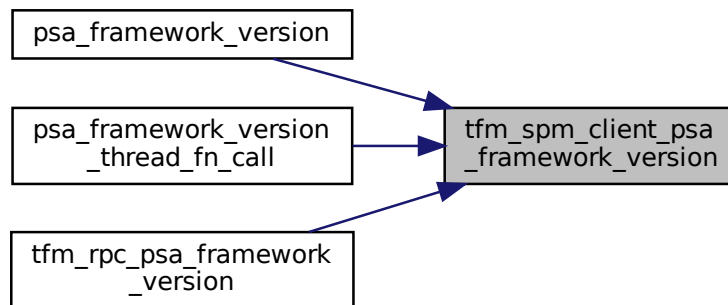
handler for [psa\\_framework\\_version](#).

**Returns**

version The version of the PSA Framework implementation that is providing the runtime services.

Definition at line 13 of file `psa_version_api.c`.

Here is the caller graph for this function:

**8.440.2.4 tfm\_spm\_client\_psa\_version()**

```
uint32_t tfm_spm_client_psa_version (
 uint32_t sid)
```

handler for [psa\\_version](#).

**Parameters**

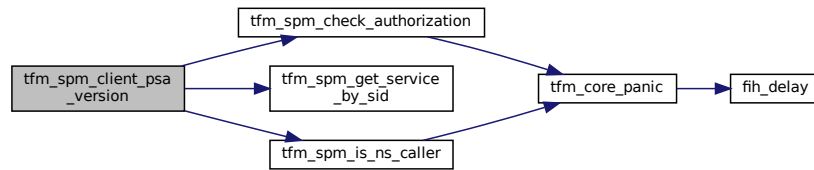
|    |            |                       |
|----|------------|-----------------------|
| in | <i>sid</i> | RoT Service identity. |
|----|------------|-----------------------|

**Return values**

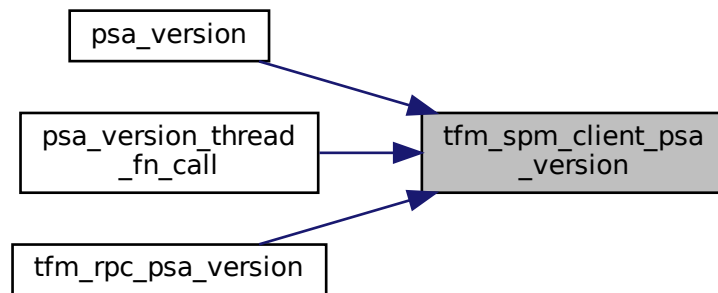
|                         |                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------|
| <i>PSA_VERSION_NONE</i> | The RoT Service is not implemented, or the caller is not permitted to access the service. |
| >                       | 0 The version of the implemented RoT Service.                                             |

Definition at line 18 of file `psa_version_api.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.440.2.5 tfm\_spm\_get\_lifecycle\_state()

```
uint32_t tfm_spm_get_lifecycle_state (
 void)
```

This function get the current PSA RoT lifecycle state.

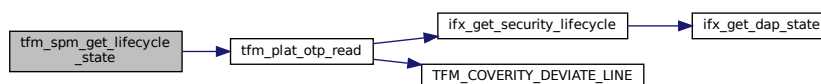
##### Returns

state The current security lifecycle state of the PSA RoT. The PSA state and implementation state are encoded as follows:

- state[15:8] – PSA lifecycle state
- state[7:0] – IMPLEMENTATION DEFINED state

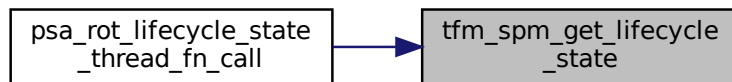
Definition at line 45 of file psa\_api.c.

Here is the call graph for this function:





Here is the caller graph for this function:



#### 8.440.2.6 tfm\_spm\_partition\_psa\_panic()

```
__NO_RETURN void tfm_spm_partition_psa_panic (
 void)
```

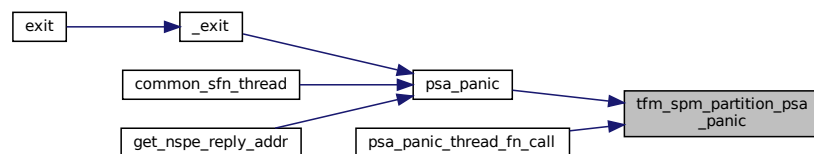
Function body of [psa\\_panic](#).

**Return values**

|                   |  |
|-------------------|--|
| Should not return |  |
|-------------------|--|

Definition at line 397 of file `psa_api.c`.

Here is the caller graph for this function:



#### 8.440.2.7 tfm\_spm\_partition\_psa\_read()

```
size_t tfm_spm_partition_psa_read (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 void * buffer,
 size_t num_bytes)
```

Function body of [psa\\_read](#).

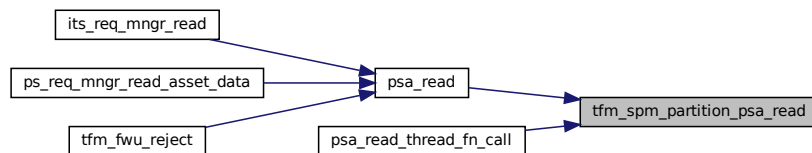
**Parameters**

|     |                   |                                                                                           |
|-----|-------------------|-------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                          |
| in  | <i>invec_idx</i>  | Index of the input vector to read from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| out | <i>buffer</i>     | Buffer in the Secure Partition to copy the requested data to.                             |
| in  | <i>num_bytes</i>  | Maximum number of bytes to be read from the client input vector.                          |

## Return values

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $>0$                    | Number of bytes copied.                                                                                                                                                                                                                                                                                                                                                                                              |
| $0$                     | There was no remaining data in this input vector.                                                                                                                                                                                                                                                                                                                                                                    |
| <i>PROGRAMMER ERROR</i> | The call is invalid, one or more of the following are true: <ul style="list-style-type: none"> <li>• <code>msg_handle</code> is invalid.</li> <li>• <code>msg_handle</code> does not refer to a <a href="#">PSA_IPC_CALL</a> message.</li> <li>• <code>invec_idx</code> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> <li>• the memory reference for buffer is invalid or not writable.</li> </ul> |

Definition at line 16 of file `psa_read_write_skip_api.c`.  
Here is the caller graph for this function:

8.440.2.8 `tfm_spm_partition_psa_reply()`

```

psa_status_t tfm_spm_partition_psa_reply (
 psa_handle_t msg_handle,
 psa_status_t status)

```

Function body of [psa\\_reply](#).

## Parameters

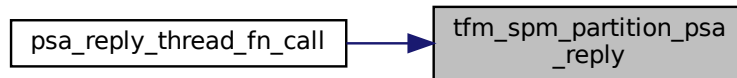
|    |                   |                                                    |
|----|-------------------|----------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                   |
| in | <i>status</i>     | Message result value to be reported to the client. |

## Return values

|                         |                                                                                                                                                                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Positive</i>         | integer Success, the connection handle.                                                                                                                                                                                     |
| <i>PSA_SUCCESS</i>      | Success                                                                                                                                                                                                                     |
| <i>PROGRAMMER ERROR</i> | The call is invalid, one or more of the following are true: <ul style="list-style-type: none"> <li>• <code>msg_handle</code> is invalid.</li> <li>• An invalid status code is specified for the type of message.</li> </ul> |

Definition at line 246 of file `psa_api.c`.

Here is the caller graph for this function:



### 8.440.2.9 tfm\_spm\_partition\_psa\_skip()

```

size_t tfm_spm_partition_psa_skip (
 psa_handle_t msg_handle,
 uint32_t invec_idx,
 size_t num_bytes)

```

Function body of `psa_skip`.

#### Parameters

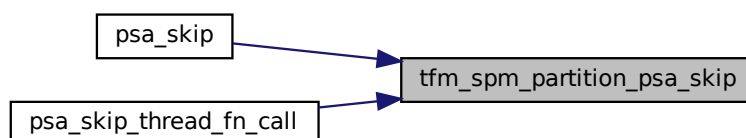
|    |                   |                                                                                       |
|----|-------------------|---------------------------------------------------------------------------------------|
| in | <i>msg_handle</i> | Handle for the client's message.                                                      |
| in | <i>invec_idx</i>  | Index of input vector to skip from. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in | <i>num_bytes</i>  | Maximum number of bytes to skip in the client input vector.                           |

#### Return values

|                         |                                                                                                                                                                                                                                                                                                                                 |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>&gt;0</i>            | Number of bytes skipped.                                                                                                                                                                                                                                                                                                        |
| <i>0</i>                | There was no remaining data in this input vector.                                                                                                                                                                                                                                                                               |
| <i>PROGRAMMER ERROR</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• <code>msg_handle</code> is invalid.</li> <li>• <code>msg_handle</code> does not refer to a request message.</li> <li>• <code>invec_idx</code> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> </ul> |

Definition at line 93 of file `psa_read_write_skip_api.c`.

Here is the caller graph for this function:



#### 8.440.2.10 tfm\_spm\_partition\_psa\_write()

```
psa_status_t tfm_spm_partition_psa_write (
 psa_handle_t msg_handle,
 uint32_t outvec_idx,
 const void * buffer,
 size_t num_bytes)
```

Function body of [psa\\_write](#).

##### Parameters

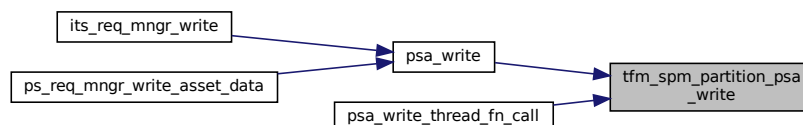
|     |                   |                                                                                                  |
|-----|-------------------|--------------------------------------------------------------------------------------------------|
| in  | <i>msg_handle</i> | Handle for the client's message.                                                                 |
| out | <i>outvec_idx</i> | Index of output vector in message to write to. Must be less than <a href="#">PSA_MAX_IOVEC</a> . |
| in  | <i>buffer</i>     | Buffer with the data to write.                                                                   |
| in  | <i>num_bytes</i>  | Number of bytes to write to the client output vector.                                            |

##### Return values

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PSA_SUCCESS</i>      | Success.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>PROGRAMMER ERROR</i> | <p>The call is invalid, one or more of the following are true:</p> <ul style="list-style-type: none"> <li>• <i>msg_handle</i> is invalid.</li> <li>• <i>msg_handle</i> does not refer to a request message.</li> <li>• <i>outvec_idx</i> is equal to or greater than <a href="#">PSA_MAX_IOVEC</a>.</li> <li>• The memory reference for <i>buffer</i> is invalid.</li> <li>• The call attempts to write data past the end of the client output vector.</li> </ul> |

Definition at line 159 of file `psa_read_write_skip_api.c`.

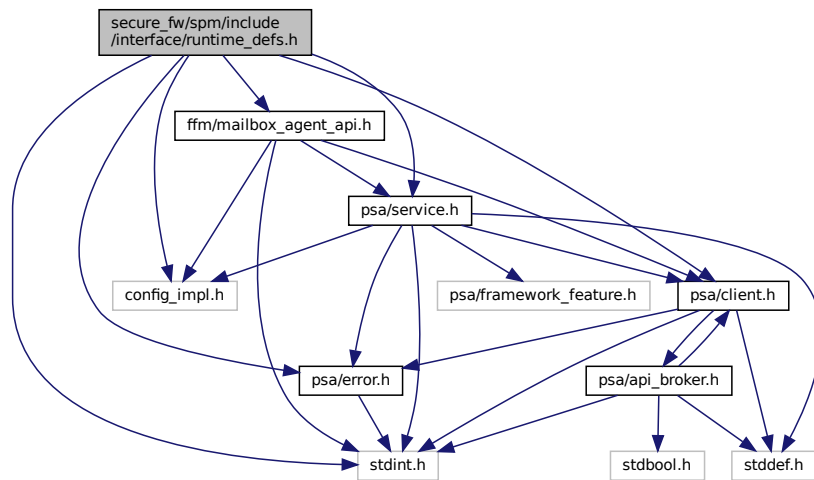
Here is the caller graph for this function:



### 8.441 secure\_fw/spm/include/interface/runtime\_defs.h File Reference

```
#include <stdint.h>
#include "config_impl.h"
#include "psa/client.h"
#include "psa/error.h"
#include "psa/service.h"
#include "ffm/mailbox_agent_api.h"
```

Include dependency graph for runtime\_defs.h:



This graph shows which files directly or indirectly include this file:



## Typedefs

- typedef [psa\\_status\\_t](#)(\* [service\\_fn\\_t](#)) ([psa\\_msg\\_t](#) \*msg)
- typedef [psa\\_status\\_t](#)(\* [sfn\\_init\\_fn\\_t](#)) (void \*param)

### 8.441.1 Typedef Documentation

#### 8.441.1.1 service\_fn\_t

```
typedef psa_status_t(* service_fn_t) (psa_msg_t *msg)
```

Definition at line 20 of file runtime\_defs.h.

#### 8.441.1.2 sfn\_init\_fn\_t

```
typedef psa_status_t(* sfn_init_fn_t) (void *param)
```

Definition at line 21 of file runtime\_defs.h.



- `#define TFM_SVC_PSA_MAP_OUTVEC TFM_SVC_NUM_PSA_API_THREAD(25)`
- `#define TFM_SVC_PSA_UNMAP_OUTVEC TFM_SVC_NUM_PSA_API_THREAD(26)`
- `#define TFM_SVC_IS_PLATFORM(svc_num) (!((svc_num) & TFM_SVC_NUM_PLATFORM_MSK))`
- `#define TFM_SVC_IS_HANDLER_MODE(svc_num) (!((svc_num) & TFM_SVC_NUM_HANDLER_MODE_MSK))`
- `#define TFM_SVC_IS_PSA_API(svc_num)`
- `#define TFM_SVC_IS_SPM(svc_num)`

## 8.442.1 Macro Definition Documentation

### 8.442.1.1 TFM\_SVC\_AGENT\_PSA\_CALL

`#define TFM_SVC_AGENT_PSA_CALL TFM_SVC_NUM_PSA_API_THREAD(20)`

Definition at line 103 of file `svc_num.h`.

### 8.442.1.2 TFM\_SVC\_AGENT\_PSA\_CLOSE

`#define TFM_SVC_AGENT_PSA_CLOSE TFM_SVC_NUM_PSA_API_THREAD(22)`

Definition at line 105 of file `svc_num.h`.

### 8.442.1.3 TFM\_SVC\_AGENT\_PSA\_CONNECT

`#define TFM_SVC_AGENT_PSA_CONNECT TFM_SVC_NUM_PSA_API_THREAD(21)`

Definition at line 104 of file `svc_num.h`.

### 8.442.1.4 TFM\_SVC\_FLIH\_FUNC\_RETURN

`#define TFM_SVC_FLIH_FUNC_RETURN TFM_SVC_NUM_SPM_THREAD(1)`

Definition at line 72 of file `svc_num.h`.

### 8.442.1.5 TFM\_SVC\_GET\_BOOT\_DATA

`#define TFM_SVC_GET_BOOT_DATA TFM_SVC_NUM_SPM_THREAD(3)`

Definition at line 74 of file `svc_num.h`.

### 8.442.1.6 TFM\_SVC\_IS\_HANDLER\_MODE

`#define TFM_SVC_IS_HANDLER_MODE(  
    svc_num ) (!((svc_num) & TFM_SVC_NUM_HANDLER_MODE_MSK))`

Definition at line 112 of file `svc_num.h`.

### 8.442.1.7 TFM\_SVC\_IS\_PLATFORM

`#define TFM_SVC_IS_PLATFORM(  
    svc_num ) (!((svc_num) & TFM_SVC_NUM_PLATFORM_MSK))`

Definition at line 111 of file `svc_num.h`.

**8.442.1.8 TFM\_SVC\_IS\_PSA\_API**

```
#define TFM_SVC_IS_PSA_API(
 svc_num)
```

**Value:**

```
((svc_num) & (TFM_SVC_NUM_PLATFORM_MSK | \
TFM_SVC_NUM_PSA_API_MSK)) \
 == TFM_SVC_NUM_PSA_API_MSK)
```

Definition at line 113 of file svc\_num.h.

**8.442.1.9 TFM\_SVC\_IS\_SPM**

```
#define TFM_SVC_IS_SPM(
 svc_num)
```

**Value:**

```
((svc_num) & (TFM_SVC_NUM_PLATFORM_MSK | \
TFM_SVC_NUM_PSA_API_MSK)) == 0)
```

Definition at line 116 of file svc\_num.h.

**8.442.1.10 TFM\_SVC\_NUM\_HANDLER\_MODE\_MSK**

```
#define TFM_SVC_NUM_HANDLER_MODE_MSK (1U << TFM_SVC_NUM_HANDLER_MODE_POS)
```

Definition at line 45 of file svc\_num.h.

**8.442.1.11 TFM\_SVC\_NUM\_HANDLER\_MODE\_POS**

```
#define TFM_SVC_NUM_HANDLER_MODE_POS 6U
```

Definition at line 44 of file svc\_num.h.

**8.442.1.12 TFM\_SVC\_NUM\_INDEX\_MSK**

```
#define TFM_SVC_NUM_INDEX_MSK (0x1FU)
```

Definition at line 50 of file svc\_num.h.

**8.442.1.13 TFM\_SVC\_NUM\_PLATFORM\_HANDLER**

```
#define TFM_SVC_NUM_PLATFORM_HANDLER(
 index)
```

**Value:**

```
(TFM_SVC_NUM_PLATFORM_MSK | \
TFM_SVC_NUM_HANDLER_MODE_MSK | \
((index) & TFM_SVC_NUM_INDEX_MSK))
```

Definition at line 66 of file svc\_num.h.

**8.442.1.14 TFM\_SVC\_NUM\_PLATFORM\_MSK**

```
#define TFM_SVC_NUM_PLATFORM_MSK (1U << TFM_SVC_NUM_PLATFORM_POS)
```

Definition at line 42 of file svc\_num.h.

**8.442.1.15 TFM\_SVC\_NUM\_PLATFORM\_POS**

```
#define TFM_SVC_NUM_PLATFORM_POS 7U
```

Definition at line 41 of file svc\_num.h.



**8.442.1.16 TFM\_SVC\_NUM\_PLATFORM\_THREAD**

```
#define TFM_SVC_NUM_PLATFORM_THREAD(
 index)
```

**Value:**

```
(TFM_SVC_NUM_PLATFORM_MSK | \
((index) & TFM_SVC_NUM_INDEX_MSK))
```

Definition at line 63 of file svc\_num.h.

**8.442.1.17 TFM\_SVC\_NUM\_PSA\_API\_MSK**

```
#define TFM_SVC_NUM_PSA_API_MSK (1U << TFM_SVC_NUM_PSA_API_POS)
```

Definition at line 48 of file svc\_num.h.

**8.442.1.18 TFM\_SVC\_NUM\_PSA\_API\_POS**

```
#define TFM_SVC_NUM_PSA_API_POS 5U
```

Definition at line 47 of file svc\_num.h.

**8.442.1.19 TFM\_SVC\_NUM\_PSA\_API\_THREAD**

```
#define TFM_SVC_NUM_PSA_API_THREAD(
 index)
```

**Value:**

```
(TFM_SVC_NUM_PSA_API_MSK | \
((index) & TFM_SVC_NUM_INDEX_MSK))
```

Definition at line 60 of file svc\_num.h.

**8.442.1.20 TFM\_SVC\_NUM\_SPM\_HANDLER**

```
#define TFM_SVC_NUM_SPM_HANDLER(
 index)
```

**Value:**

```
(TFM_SVC_NUM_HANDLER_MODE_MSK | \
((index) & TFM_SVC_NUM_INDEX_MSK))
```

Definition at line 57 of file svc\_num.h.

**8.442.1.21 TFM\_SVC\_NUM\_SPM\_THREAD**

```
#define TFM_SVC_NUM_SPM_THREAD(
 index) ((index) & TFM_SVC_NUM_INDEX_MSK)
```

Definition at line 55 of file svc\_num.h.

**8.442.1.22 TFM\_SVC\_OUTPUT\_UNPRIV\_STRING**

```
#define TFM_SVC_OUTPUT_UNPRIV_STRING TFM_SVC_NUM_SPM_THREAD(2)
```

Definition at line 73 of file svc\_num.h.

**8.442.1.23 TFM\_SVC\_PREPARE\_DEPRIV\_FLIH**

```
#define TFM_SVC_PREPARE_DEPRIV_FLIH TFM_SVC_NUM_SPM_HANDLER(0)
```

Definition at line 78 of file svc\_num.h.

#### 8.442.1.24 TFM\_SVC\_PSA\_CALL

```
#define TFM_SVC_PSA_CALL TFM_SVC_NUM_PSA_API_THREAD (2)
```

Definition at line 84 of file svc\_num.h.

#### 8.442.1.25 TFM\_SVC\_PSA\_CLEAR

```
#define TFM_SVC_PSA_CLEAR TFM_SVC_NUM_PSA_API_THREAD (13)
```

Definition at line 96 of file svc\_num.h.

#### 8.442.1.26 TFM\_SVC\_PSA\_CLOSE

```
#define TFM_SVC_PSA_CLOSE TFM_SVC_NUM_PSA_API_THREAD (4)
```

Definition at line 86 of file svc\_num.h.

#### 8.442.1.27 TFM\_SVC\_PSA\_CONNECT

```
#define TFM_SVC_PSA_CONNECT TFM_SVC_NUM_PSA_API_THREAD (3)
```

Definition at line 85 of file svc\_num.h.

#### 8.442.1.28 TFM\_SVC\_PSA\_EOI

```
#define TFM_SVC_PSA_EOI TFM_SVC_NUM_PSA_API_THREAD (14)
```

Definition at line 97 of file svc\_num.h.

#### 8.442.1.29 TFM\_SVC\_PSA\_FRAMEWORK\_VERSION

```
#define TFM_SVC_PSA_FRAMEWORK_VERSION TFM_SVC_NUM_PSA_API_THREAD (0)
```

Definition at line 82 of file svc\_num.h.

#### 8.442.1.30 TFM\_SVC\_PSA\_GET

```
#define TFM_SVC_PSA_GET TFM_SVC_NUM_PSA_API_THREAD (6)
```

Definition at line 89 of file svc\_num.h.

#### 8.442.1.31 TFM\_SVC\_PSA\_IRQ\_DISABLE

```
#define TFM_SVC_PSA_IRQ_DISABLE TFM_SVC_NUM_PSA_API_THREAD (18)
```

Definition at line 101 of file svc\_num.h.

#### 8.442.1.32 TFM\_SVC\_PSA\_IRQ\_ENABLE

```
#define TFM_SVC_PSA_IRQ_ENABLE TFM_SVC_NUM_PSA_API_THREAD (17)
```

Definition at line 100 of file svc\_num.h.

#### 8.442.1.33 TFM\_SVC\_PSA\_LIFECYCLE

```
#define TFM_SVC_PSA_LIFECYCLE TFM_SVC_NUM_PSA_API_THREAD (16)
```

Definition at line 99 of file svc\_num.h.

#### 8.442.1.34 TFM\_SVC\_PSA\_MAP\_INVEC

```
#define TFM_SVC_PSA_MAP_INVEC TFM_SVC_NUM_PSA_API_THREAD (23)
```

Definition at line 106 of file svc\_num.h.

#### 8.442.1.35 TFM\_SVC\_PSA\_MAP\_OUTVEC

```
#define TFM_SVC_PSA_MAP_OUTVEC TFM_SVC_NUM_PSA_API_THREAD (25)
```

Definition at line 108 of file svc\_num.h.

#### 8.442.1.36 TFM\_SVC\_PSA\_NOTIFY

```
#define TFM_SVC_PSA_NOTIFY TFM_SVC_NUM_PSA_API_THREAD (12)
```

Definition at line 95 of file svc\_num.h.

#### 8.442.1.37 TFM\_SVC\_PSA\_PANIC

```
#define TFM_SVC_PSA_PANIC TFM_SVC_NUM_PSA_API_THREAD (15)
```

Definition at line 98 of file svc\_num.h.

#### 8.442.1.38 TFM\_SVC\_PSA\_READ

```
#define TFM_SVC_PSA_READ TFM_SVC_NUM_PSA_API_THREAD (8)
```

Definition at line 91 of file svc\_num.h.

#### 8.442.1.39 TFM\_SVC\_PSA\_REPLY

```
#define TFM_SVC_PSA_REPLY TFM_SVC_NUM_PSA_API_THREAD (11)
```

Definition at line 94 of file svc\_num.h.

#### 8.442.1.40 TFM\_SVC\_PSA\_RESET\_SIGNAL

```
#define TFM_SVC_PSA_RESET_SIGNAL TFM_SVC_NUM_PSA_API_THREAD (19)
```

Definition at line 102 of file svc\_num.h.

#### 8.442.1.41 TFM\_SVC\_PSA\_SET\_RHANDLE

```
#define TFM_SVC_PSA_SET_RHANDLE TFM_SVC_NUM_PSA_API_THREAD (7)
```

Definition at line 90 of file svc\_num.h.

#### 8.442.1.42 TFM\_SVC\_PSA\_SKIP

```
#define TFM_SVC_PSA_SKIP TFM_SVC_NUM_PSA_API_THREAD (9)
```

Definition at line 92 of file svc\_num.h.

#### 8.442.1.43 TFM\_SVC\_PSA\_UNMAP\_INVEC

```
#define TFM_SVC_PSA_UNMAP_INVEC TFM_SVC_NUM_PSA_API_THREAD (24)
```

Definition at line 107 of file svc\_num.h.

**8.442.1.44 TFM\_SVC\_PSA\_UNMAP\_OUTVEC**

```
#define TFM_SVC_PSA_UNMAP_OUTVEC TFM_SVC_NUM_PSA_API_THREAD (26)
```

Definition at line 109 of file svc\_num.h.

**8.442.1.45 TFM\_SVC\_PSA\_VERSION**

```
#define TFM_SVC_PSA_VERSION TFM_SVC_NUM_PSA_API_THREAD (1)
```

Definition at line 83 of file svc\_num.h.

**8.442.1.46 TFM\_SVC\_PSA\_WAIT**

```
#define TFM_SVC_PSA_WAIT TFM_SVC_NUM_PSA_API_THREAD (5)
```

Definition at line 88 of file svc\_num.h.

**8.442.1.47 TFM\_SVC\_PSA\_WRITE**

```
#define TFM_SVC_PSA_WRITE TFM_SVC_NUM_PSA_API_THREAD (10)
```

Definition at line 93 of file svc\_num.h.

**8.442.1.48 TFM\_SVC\_SPM\_INIT**

```
#define TFM_SVC_SPM_INIT TFM_SVC_NUM_SPM_THREAD (0)
```

Definition at line 71 of file svc\_num.h.

**8.442.1.49 TFM\_SVC\_THREAD\_MODE\_SPM\_RETURN**

```
#define TFM_SVC_THREAD_MODE_SPM_RETURN TFM_SVC_NUM_SPM_THREAD (4)
```

Definition at line 75 of file svc\_num.h.

**8.443 secure\_fw/spm/include/lists.h File Reference**

This graph shows which files directly or indirectly include this file:

**Data Structures**

- struct [bi\\_list\\_node\\_t](#)

**Macros**

- #define [BI\\_LIST\\_INIT\\_NODE](#)(node)
- #define [BI\\_LIST\\_INSERT\\_AFTER](#)(curr, node)
- #define [BI\\_LIST\\_INSERT\\_BEFORE](#)(curr, node)
- #define [BI\\_LIST\\_REMOVE\\_NODE](#)(node)
- #define [BI\\_LIST\\_IS\\_EMPTY](#)(head) (((head) == NULL) || ((head)->bnext == (head)))
- #define [BI\\_LIST\\_NEXT\\_NODE](#)(node) (((node) != NULL) ? ((node)->bnext) : NULL)
- #define [BI\\_LIST\\_FOR\\_EACH](#)(node, head)

- `#define UNI_LIST_INIT_NODE(head, link)`
- `#define UNI_LIST_INSERT_AFTER(posi, node, link)`
- `#define UNI_LIST_IS_EMPTY(node, link) ((node) == NULL) || ((node)->link == NULL)`
- `#define UNI_LIST_NEXT_NODE(node, link) (((node) != NULL) ? ((node)->link) : NULL)`
- `#define UNI_LIST_REMOVE_NODE(prev, node, link)`
- `#define UNI_LIST_REMOVE_NODE_BY_PNODE(pnode, link)`
- `#define UNI_LIST_MOVE_AFTER(posi, prev, node, link)`
- `#define UNI_LIST_FOREACH(node, head, link)`
- `#define UNI_LIST_FOREACH_NODE_PREV(prev, node, head, link)`
- `#define UNI_LIST_FOREACH_NODE_PNODE(pnode, node, head, link)`

## 8.443.1 Macro Definition Documentation

### 8.443.1.1 BI\_LIST\_FOR\_EACH

```
#define BI_LIST_FOR_EACH(
 node,
 head)
```

**Value:**

```
for (node = ((head) != NULL) ? (head)->bnext : NULL); \
 ((head) != NULL) && ((node) != NULL) && (node != (head)); \
 node = (node)->bnext)
```

Definition at line 67 of file lists.h.

### 8.443.1.2 BI\_LIST\_INIT\_NODE

```
#define BI_LIST_INIT_NODE(
 node)
```

**Value:**

```
do {
 if ((node) != NULL) {
 (node)->bnext = (node);
 (node)->bprev = (node);
 }
} while (0)
```

Definition at line 18 of file lists.h.

### 8.443.1.3 BI\_LIST\_INSERT\_AFTER

```
#define BI_LIST_INSERT_AFTER(
 curr,
 node)
```

**Value:**

```
do {
 if ((curr) != NULL && (node) != NULL &&
 (curr)->bnext != NULL) {
 (node)->bnext = (curr)->bnext;
 (node)->bprev = (curr);
 (curr)->bnext->bprev = (node);
 (curr)->bnext = (node);
 }
} while (0)
```

Definition at line 27 of file lists.h.

### 8.443.1.4 BI\_LIST\_INSERT\_BEFORE

```
#define BI_LIST_INSERT_BEFORE(
 curr,
 node)
```

**Value:**

```

do {
 if ((curr) != NULL && (node) != NULL &&
 (curr)->bprev != NULL) {
 (curr)->bprev->bnext = (node);
 (node)->bprev = (curr)->bprev;
 (curr)->bprev = (node);
 (node)->bnext = (curr);
 }
} while (0)

```

Definition at line 38 of file lists.h.

**8.443.1.5 BI\_LIST\_IS\_EMPTY**

```

#define BI_LIST_IS_EMPTY(
 head) ((head) == NULL) || ((head)->bnext == (head))

```

Definition at line 59 of file lists.h.

**8.443.1.6 BI\_LIST\_NEXT\_NODE**

```

#define BI_LIST_NEXT_NODE(
 node) ((node) != NULL) ? ((node)->bnext) : NULL

```

Definition at line 63 of file lists.h.

**8.443.1.7 BI\_LIST\_REMOVE\_NODE**

```

#define BI_LIST_REMOVE_NODE(
 node)

```

**Value:**

```

do {
 if ((node) != NULL &&
 (node)->bprev != NULL &&
 (node)->bnext != NULL) {
 (node)->bprev->bnext = (node)->bnext;
 (node)->bnext->bprev = (node)->bprev;
 }
} while (0)

```

Definition at line 49 of file lists.h.

**8.443.1.8 UNI\_LIST\_FOREACH**

```

#define UNI_LIST_FOREACH(
 node,
 head,
 link)

```

**Value:**

```

for (node = ((head) != NULL) ? (head)->link : NULL);
 node != NULL;
 node = (node)->link)

```

Definition at line 123 of file lists.h.

**8.443.1.9 UNI\_LIST\_FOREACH\_NODE\_PNODE**

```

#define UNI_LIST_FOREACH_NODE_PNODE(
 pnode,
 node,
 head,
 link)

```

**Value:**

```

for (pnode = ((head) != NULL) ? &(head)->link : NULL),
 node = ((head) != NULL) ? (head)->link : NULL;
 node != NULL;

```

```
pnode = &(node)->link, node = (node)->link)
```

Definition at line 135 of file lists.h.

#### 8.443.1.10 UNI\_LIST\_FOREACH\_NODE\_PREV

```
#define UNI_LIST_FOREACH_NODE_PREV(
 prev,
 node,
 head,
 link)
```

**Value:**

```
for (prev = NULL, node = ((head) != NULL) ? (head)->link : NULL); \
 node != NULL;
 prev = node, node = (prev)->link)
```

Definition at line 129 of file lists.h.

#### 8.443.1.11 UNI\_LIST\_INIT\_NODE

```
#define UNI_LIST_INIT_NODE(
 head,
 link)
```

**Value:**

```
do { \
 if ((head) != NULL) { \
 (head)->link = NULL; \
 } \
} while (0)
```

Definition at line 74 of file lists.h.

#### 8.443.1.12 UNI\_LIST\_INSERT\_AFTER

```
#define UNI_LIST_INSERT_AFTER(
 posi,
 node,
 link)
```

**Value:**

```
do { \
 if ((posi) != NULL && (node) != NULL) { \
 (node)->link = (posi)->link; \
 (posi)->link = (node); \
 } \
} while (0)
```

Definition at line 81 of file lists.h.

#### 8.443.1.13 UNI\_LIST\_IS\_EMPTY

```
#define UNI_LIST_IS_EMPTY(
 node,
 link) ((node) == NULL) || ((node)->link == NULL)
```

Definition at line 89 of file lists.h.

#### 8.443.1.14 UNI\_LIST\_MOVE\_AFTER

```
#define UNI_LIST_MOVE_AFTER(
 posi,
 prev,
 node,
 link)
```

**Value:**

```

do { \
if ((posi) != NULL && (prev) != NULL &&
 (node) != NULL) {
 (prev)->link = (node)->link;
 (node)->link = (posi)->link;
 (posi)->link = node;
}
} while (0)

```

Definition at line 113 of file lists.h.

#### 8.443.1.15 UNI\_LIST\_NEXT\_NODE

```

#define UNI_LIST_NEXT_NODE(
 node,
 link) (((node) != NULL) ? ((node)->link) : NULL)

```

Definition at line 93 of file lists.h.

#### 8.443.1.16 UNI\_LIST\_REMOVE\_NODE

```

#define UNI_LIST_REMOVE_NODE(
 prev,
 node,
 link)

```

**Value:**

```

do { \
if ((prev) != NULL && (node) != NULL) {
 (prev)->link = (node)->link;
 (node)->link = NULL;
}
} while (0)

```

Definition at line 97 of file lists.h.

#### 8.443.1.17 UNI\_LIST\_REMOVE\_NODE\_BY\_PNODE

```

#define UNI_LIST_REMOVE_NODE_BY_PNODE(
 pnode,
 link)

```

**Value:**

```

do {
 if ((pnode) != NULL && *(pnode) != NULL) {
 (pnode) = ((pnode))->link;
 }
} while (0)

```

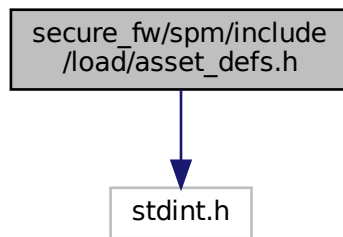
Definition at line 105 of file lists.h.

## 8.444 secure\_fw/spm/include/load/asset\_defs.h File Reference

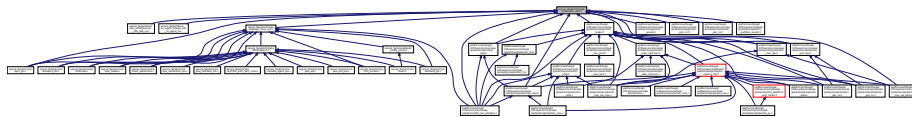
```
#include <stdint.h>
```



Include dependency graph for asset\_defs.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [asset\\_desc\\_t](#)  
*Asset descriptor.*

## Macros

- `#define ASSET_ATTR_MMIO (ASSET_ATTR_NAMED_MMIO | ASSET_ATTR_NUMBERED_MMIO)`

## Enumerations

- enum [assets\\_attribute\\_t](#) {  
`ASSET_ATTR_READ_ONLY = (1U << 0), ASSET_ATTR_READ_WRITE = (1U << 1), ASSET_ATTR_PPB = (1U << 2), ASSET_ATTR_NAMED_MMIO = (1U << 3),`  
`ASSET_ATTR_NUMBERED_MMIO = (1U << 4), ASSET_ATTR_PAD = UINT32_MAX }`  
*Memory, MMIO and PPB asset attributes.*

### 8.444.1 Macro Definition Documentation

#### 8.444.1.1 ASSET\_ATTR\_MMIO

```
#define ASSET_ATTR_MMIO (ASSET_ATTR_NAMED_MMIO | ASSET_ATTR_NUMBERED_MMIO)
```

Definition at line 25 of file `asset_defs.h`.

### 8.444.2 Enumeration Type Documentation

### 8.444.2.1 assets\_attribute\_t

enum `assets_attribute_t`

Memory, MMIO and PPB asset attributes.

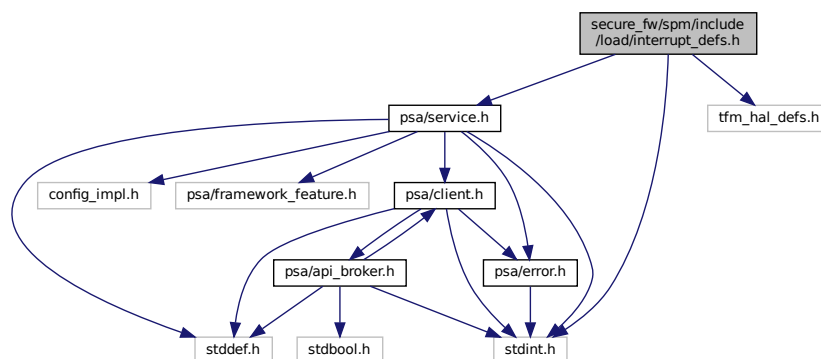
Enumerator

|                          |                         |
|--------------------------|-------------------------|
| ASSET_ATTR_READ_ONLY     | 1: Read-only MMIO       |
| ASSET_ATTR_READ_WRITE    | 1: Read-write MMIO      |
| ASSET_ATTR_PPB           | 1: PPB indicator        |
| ASSET_ATTR_NAMED_MMIO    | 1: Named MMIO object    |
| ASSET_ATTR_NUMBERED_MMIO | 1: Numbered MMIO object |
| ASSET_ATTR_PAD           |                         |

Definition at line 15 of file `asset_defs.h`.

## 8.445 secure\_fw/spm/include/load/interrupt\_defs.h File Reference

```
#include <stdint.h>
#include "tfm_hal_defs.h"
#include "psa/service.h"
Include dependency graph for interrupt_defs.h:
```



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `irq_load_info_t`
- struct `irq_t`

## Enumerations

- enum `mbox_irq_flags_t` { `MBOX_IRQ_FLAGS_NONE` = 0UL, `MBOX_IRQ_FLAGS_DEFAULT_SCHEDULE` = `MBOX_IRQ_FLAGS_NONE`, `MBOX_IRQ_FLAGS_DEFER_SCHEDULE` = (1UL << 0), `MBOX_IRQ_FLAGS_MAX` = `UINT32_MAX` }

## 8.445.1 Enumeration Type Documentation

### 8.445.1.1 mbox\_irq\_flags\_t

```
enum mbox_irq_flags_t
```

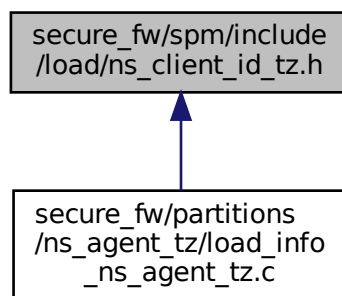
Enumerator

|                                 |  |
|---------------------------------|--|
| MBOX_IRQ_FLAGS_NONE             |  |
| MBOX_IRQ_FLAGS_DEFAULT_SCHEDULE |  |
| MBOX_IRQ_FLAGS_DEFER_SCHEDULE   |  |
| MBOX_IRQ_FLAGS_MAX              |  |

Definition at line 38 of file interrupt\_defs.h.

## 8.446 secure\_fw/spm/include/load/ns\_client\_id\_tz.h File Reference

This graph shows which files directly or indirectly include this file:



## Macros

- #define [TZ\\_NS\\_CLIENT\\_ID\\_BASE](#) (0xaaaa0000)
- #define [TZ\\_NS\\_CLIENT\\_ID\\_LIMIT](#) (0xaaaa7fff)

## 8.446.1 Macro Definition Documentation

### 8.446.1.1 TZ\_NS\_CLIENT\_ID\_BASE

```
#define TZ_NS_CLIENT_ID_BASE (0xaaaa0000)
```

Definition at line 11 of file ns\_client\_id\_tz.h.

### 8.446.1.2 TZ\_NS\_CLIENT\_ID\_LIMIT

```
#define TZ_NS_CLIENT_ID_LIMIT (0xaaaa7fff)
```

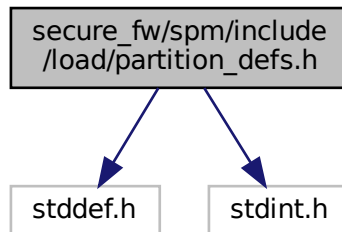
Definition at line 12 of file ns\_client\_id\_tz.h.

## 8.447 secure\_fw/spm/include/load/partition\_defs.h File Reference

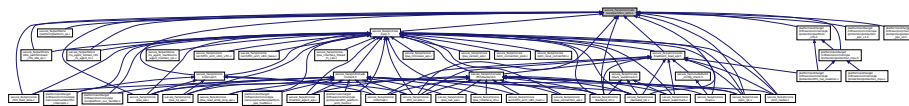
```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for partition\_defs.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [partition\\_load\\_info\\_t](#)

### Macros

- #define [TFM\\_SP\\_IDLE](#) (0x5555003f)
- #define [TFM\\_SP\\_TZ\\_AGENT](#) (0x555501c7)
- #define [INVALID\\_PARTITION\\_ID](#) (0U)
- #define [PARTITION\\_INFO\\_VERSION\\_MASK](#) (0x0000FFFF)
- #define [PARTITION\\_INFO\\_MAGIC\\_MASK](#) (0xFFFF0000)
- #define [PARTITION\\_INFO\\_MAGIC](#) (0x5F5F0000)
- #define [PARTITION\\_PRI\\_HIGHEST](#) (0x0)
- #define [PARTITION\\_PRI\\_HIGH](#) (0xF)
- #define [PARTITION\\_PRI\\_NORMAL](#) (0x1F)
- #define [PARTITION\\_PRI\\_LOW](#) (0x7F)
- #define [PARTITION\\_PRI\\_LOWEST](#) (0xFF)
- #define [PARTITION\\_PRI\\_MASK](#) (0xFF)
- #define [PARTITION\\_MODEL\\_PSA\\_ROT](#) (1UL << 8)
- #define [PARTITION\\_MODEL\\_IPC](#) (1UL << 9)
- #define [PARTITION\\_NS\\_AGENT\\_MB](#) (1UL << 10)
- #define [PARTITION\\_NS\\_AGENT\\_TZ](#) (1UL << 11)
- #define [TO\\_THREAD\\_PRIORITY](#)(x) (x)
- #define [ENTRY\\_TO\\_POSITION](#)(x) (uintptr\_t)(x)
- #define [POSITION\\_TO\\_ENTRY](#)(x, t) (t)(x)
- #define [PTR\\_TO\\_REFERENCE](#)(x) (uintptr\_t)(x)
- #define [REFERENCE\\_TO\\_PTR](#)(x, t) (t)(x)

- `#define IS_PSA_ROT(pldi)`
- `#define IS_IPC_MODEL(pldi)`
- `#define IS_NS_AGENT(pldi)`
- `#define IS_NS_AGENT_TZ(pldi) ((void)pldi, false)`
- `#define IS_NS_AGENT_MAILBOX(pldi) ((void)pldi, false)`
- `#define PARTITION_TYPE_TO_INDEX(type) (!!(type) & PARTITION_NS_AGENT_TZ))`
- `#define LOAD_ORDER_BY_PRIORITY(priority) (((uint16_t)(priority)) << 8)`

## 8.447.1 Macro Definition Documentation

### 8.447.1.1 ENTRY\_TO\_POSITION

```
#define ENTRY_TO_POSITION(
 x) (uintptr_t)(x)
```

Definition at line 58 of file `partition_defs.h`.

### 8.447.1.2 INVALID\_PARTITION\_ID

```
#define INVALID_PARTITION_ID (0U)
```

Definition at line 19 of file `partition_defs.h`.

### 8.447.1.3 IS\_IPC\_MODEL

```
#define IS_IPC_MODEL(
 pldi)
```

**Value:**

```
((!(pldi->flags \
& PARTITION_MODEL_IPC))
```

Definition at line 66 of file `partition_defs.h`.

### 8.447.1.4 IS\_NS\_AGENT

```
#define IS_NS_AGENT(
 pldi)
```

**Value:**

```
((!(pldi->flags \
& (PARTITION_NS_AGENT_MB | PARTITION_NS_AGENT_TZ)))
```

Definition at line 68 of file `partition_defs.h`.

### 8.447.1.5 IS\_NS\_AGENT\_MAILBOX

```
#define IS_NS_AGENT_MAILBOX(
 pldi) ((void)pldi, false)
```

Definition at line 78 of file `partition_defs.h`.

### 8.447.1.6 IS\_NS\_AGENT\_TZ

```
#define IS_NS_AGENT_TZ(
 pldi) ((void)pldi, false)
```

Definition at line 73 of file `partition_defs.h`.

#### 8.447.1.7 IS\_PSA\_ROT

```
#define IS_PSA_ROT(
 pldi)
```

**Value:**

```
(!!((pldi)->flags \
& PARTITION_MODEL_PSA_ROT))
```

Definition at line 64 of file partition\_defs.h.

#### 8.447.1.8 LOAD\_ORDER\_BY\_PRIORITY

```
#define LOAD_ORDER_BY_PRIORITY(
 priority) (((uint16_t)(priority)) << 8)
```

Definition at line 91 of file partition\_defs.h.

#### 8.447.1.9 PARTITION\_INFO\_MAGIC

```
#define PARTITION_INFO_MAGIC (0x5F5F0000)
```

Definition at line 24 of file partition\_defs.h.

#### 8.447.1.10 PARTITION\_INFO\_MAGIC\_MASK

```
#define PARTITION_INFO_MAGIC_MASK (0xFFFF0000)
```

Definition at line 23 of file partition\_defs.h.

#### 8.447.1.11 PARTITION\_INFO\_VERSION\_MASK

```
#define PARTITION_INFO_VERSION_MASK (0x0000FFFF)
```

Definition at line 22 of file partition\_defs.h.

#### 8.447.1.12 PARTITION\_MODEL\_IPC

```
#define PARTITION_MODEL_IPC (1UL << 9)
```

Definition at line 51 of file partition\_defs.h.

#### 8.447.1.13 PARTITION\_MODEL\_PSA\_ROT

```
#define PARTITION_MODEL_PSA_ROT (1UL << 8)
```

Definition at line 50 of file partition\_defs.h.

#### 8.447.1.14 PARTITION\_NS\_AGENT\_MB

```
#define PARTITION_NS_AGENT_MB (1UL << 10)
```

Definition at line 53 of file partition\_defs.h.

#### 8.447.1.15 PARTITION\_NS\_AGENT\_TZ

```
#define PARTITION_NS_AGENT_TZ (1UL << 11)
```

Definition at line 54 of file partition\_defs.h.

**8.447.1.16 PARTITION\_PRI\_HIGH**

```
#define PARTITION_PRI_HIGH (0xF)
```

Definition at line 44 of file partition\_defs.h.

**8.447.1.17 PARTITION\_PRI\_HIGHEST**

```
#define PARTITION_PRI_HIGHEST (0x0)
```

Definition at line 43 of file partition\_defs.h.

**8.447.1.18 PARTITION\_PRI\_LOW**

```
#define PARTITION_PRI_LOW (0x7F)
```

Definition at line 46 of file partition\_defs.h.

**8.447.1.19 PARTITION\_PRI\_LOWEST**

```
#define PARTITION_PRI_LOWEST (0xFF)
```

Definition at line 47 of file partition\_defs.h.

**8.447.1.20 PARTITION\_PRI\_MASK**

```
#define PARTITION_PRI_MASK (0xFF)
```

Definition at line 48 of file partition\_defs.h.

**8.447.1.21 PARTITION\_PRI\_NORMAL**

```
#define PARTITION_PRI_NORMAL (0x1F)
```

Definition at line 45 of file partition\_defs.h.

**8.447.1.22 PARTITION\_TYPE\_TO\_INDEX**

```
#define PARTITION_TYPE_TO_INDEX(
 type) (!((type) & PARTITION_NS_AGENT_TZ))
```

Definition at line 81 of file partition\_defs.h.

**8.447.1.23 POSITION\_TO\_ENTRY**

```
#define POSITION_TO_ENTRY(
 x,
 t) (t)(x)
```

Definition at line 59 of file partition\_defs.h.

**8.447.1.24 PTR\_TO\_REFERENCE**

```
#define PTR_TO_REFERENCE(
 x) (uintptr_t)(x)
```

Definition at line 61 of file partition\_defs.h.

**8.447.1.25 REFERENCE\_TO\_PTR**

```
#define REFERENCE_TO_PTR(
 x,
 t) (t) (x)
```

Definition at line 62 of file partition\_defs.h.

**8.447.1.26 TFM\_SP\_IDLE**

```
#define TFM_SP_IDLE (0x5555003f)
```

Definition at line 17 of file partition\_defs.h.

**8.447.1.27 TFM\_SP\_TZ\_AGENT**

```
#define TFM_SP_TZ_AGENT (0x555501c7)
```

Definition at line 18 of file partition\_defs.h.

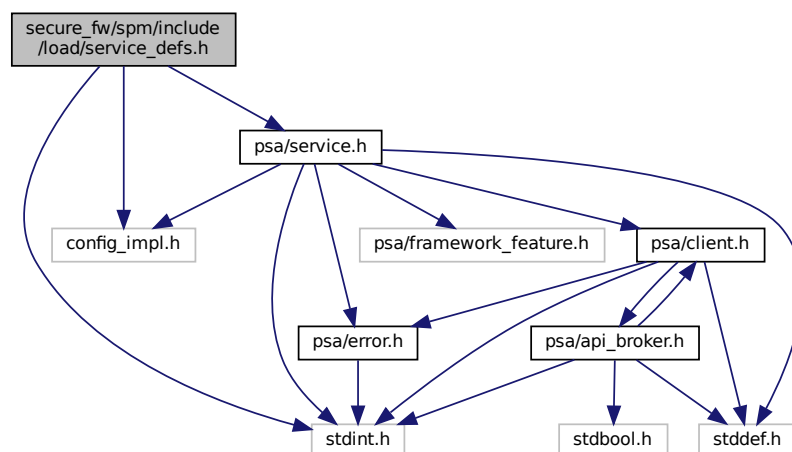
**8.447.1.28 TO\_THREAD\_PRIORITY**

```
#define TO_THREAD_PRIORITY(
 x) (x)
```

Definition at line 56 of file partition\_defs.h.

**8.448 secure\_fw/spm/include/load/service\_defs.h File Reference**

```
#include <stdint.h>
#include "config_impl.h"
#include "psa/service.h"
Include dependency graph for service_defs.h:
```





The diagram illustrates a complex network of dependencies between different security architecture components. At the top level, there are three main categories: 'Security Architecture Framework', 'Security Architecture Design', and 'Security Architecture Implementation'. These categories further branch out into more specific sub-components, eventually leading to a large number of individual modules or interfaces represented by boxes at the bottom. Arrows indicate the direction of the dependencies, showing how higher-level frameworks rely on more detailed design and implementation components.

- struct `service_load_info_t`

- #define [SERVICE\\_FLAG\\_STATELESS\\_HINDEX\\_MASK](#) (0xFF)
- #define [SERVICE\\_FLAG\\_NS\\_ACCESSIBLE](#) (1UL << 8)
- #define [SERVICE\\_FLAG\\_STATELESS](#) (1UL << 9)
- #define [SERVICE\\_FLAG\\_VERSION\\_POLICY\\_BIT](#) (1UL << 10)
- #define [SERVICE\\_VERSION\\_POLICY\\_RELAXED](#) (0UL << 10)
- #define [SERVICE\\_VERSION\\_POLICY\\_STRICT](#) (1UL << 10)
- #define [SERVICE\\_FLAG\\_MM\\_IOVEC](#) (1UL << 11)
- #define [SERVICE\\_GET\\_STATELESS\\_HINDEX](#)(flag) ((flag) & [SERVICE\\_FLAG\\_STATELESS\\_HINDEX\\_MASK](#))
- #define [SERVICE\\_IS\\_NS\\_ACCESSIBLE](#)(flag) ((flag) & [SERVICE\\_FLAG\\_NS\\_ACCESSIBLE](#))
- #define [SERVICE\\_IS\\_STATELESS](#)(flag) ((flag) & [SERVICE\\_FLAG\\_STATELESS](#))
- #define [SERVICE\\_GET\\_VERSION\\_POLICY](#)(flag) ((flag) & [SERVICE\\_FLAG\\_VERSION\\_POLICY\\_BIT](#))
- #define [SERVICE\\_ENABLED\\_MM\\_IOVEC](#)(flag) ((flag) & [SERVICE\\_FLAG\\_MM\\_IOVEC](#))
- #define [STRID\\_TO\\_STRING\\_PTR](#)(strid) (const char \*) (strid)
- #define [STRING\\_PTR\\_TO\\_STRID](#)(str) (uintptr\_t) (str)

#### 8.448.1.1 SERVICE\_ENABLED\_MM\_IOVEC

Definition at line 40 of file service defs.h.

### 8.448.1.3 SERVICE FLAG NS ACCESSIBLE

#### 8.448.1.4 SERVICE FLAG STATELESS

```
#define SERVICE_FLAG_STATELESS (1UL << 9)
Definition at line 25 of file service_defs.h.
```

#### 8.448.1.5 SERVICE\_FLAG\_STATELESS\_HINDEX\_MASK

```
#define SERVICE_FLAG_STATELESS_HINDEX_MASK (0xFF)
```

Definition at line 23 of file service\_defs.h.

#### 8.448.1.6 SERVICE\_FLAG\_VERSION\_POLICY\_BIT

```
#define SERVICE_FLAG_VERSION_POLICY_BIT (1UL << 10)
```

Definition at line 27 of file service\_defs.h.

#### 8.448.1.7 SERVICE\_GET\_STATELESS\_HINDEX

```
#define SERVICE_GET_STATELESS_HINDEX(
 flag) ((flag) & SERVICE_FLAG_STATELESS_HINDEX_MASK)
```

Definition at line 32 of file service\_defs.h.

#### 8.448.1.8 SERVICE\_GET\_VERSION\_POLICY

```
#define SERVICE_GET_VERSION_POLICY(
 flag) ((flag) & SERVICE_FLAG_VERSION_POLICY_BIT)
```

Definition at line 38 of file service\_defs.h.

#### 8.448.1.9 SERVICE\_IS\_NS\_ACCESSIBLE

```
#define SERVICE_IS_NS_ACCESSIBLE(
 flag) ((flag) & SERVICE_FLAG_NS_ACCESSIBLE)
```

Definition at line 34 of file service\_defs.h.

#### 8.448.1.10 SERVICE\_IS\_STATELESS

```
#define SERVICE_IS_STATELESS(
 flag) ((flag) & SERVICE_FLAG_STATELESS)
```

Definition at line 36 of file service\_defs.h.

#### 8.448.1.11 SERVICE\_VERSION\_POLICY\_RELAXED

```
#define SERVICE_VERSION_POLICY_RELAXED (0UL << 10)
```

Definition at line 28 of file service\_defs.h.

#### 8.448.1.12 SERVICE\_VERSION\_POLICY\_STRICT

```
#define SERVICE_VERSION_POLICY_STRICT (1UL << 10)
```

Definition at line 29 of file service\_defs.h.

#### 8.448.1.13 STRID\_TO\_STRING\_PTR

```
#define STRID_TO_STRING_PTR(
 strid) (const char *)(strid)
```

Definition at line 43 of file service\_defs.h.



## 8.449.1 Macro Definition Documentation

### 8.449.1.1 LOAD\_ALLOCED\_STACK\_ADDR

```
#define LOAD_ALLOCED_STACK_ADDR(
 pldinf) (*(uintptr_t *)((pldinf + 1)))
```

Definition at line 34 of file `spm_load_api.h`.

### 8.449.1.2 LOAD\_INFO\_ASSET

```
#define LOAD_INFO_ASSET(
 pldinf)
```

**Value:**

```
((const struct asset_desc_t *)((uintptr_t)LOAD_INFO_SERVICE(pldinf) + \
 (pldinf)->nservices * sizeof(struct service_load_info_t)))
```

Definition at line 41 of file `spm_load_api.h`.

### 8.449.1.3 LOAD\_INFO\_DEPS

```
#define LOAD_INFO_DEPS(
 pldinf) ((const uint32_t *)((uintptr_t)((pldinf + 1) + ((LOAD_INFO_EXT_LENGTH)
 * sizeof(uintptr_t))))
```

Definition at line 36 of file `spm_load_api.h`.

### 8.449.1.4 LOAD\_INFO\_EXT\_LENGTH

```
#define LOAD_INFO_EXT_LENGTH (2U)
```

Definition at line 24 of file `spm_load_api.h`.

### 8.449.1.5 LOAD\_INFO\_IRQ

```
#define LOAD_INFO_IRQ(
 pldinf)
```

**Value:**

```
((const struct irq_load_info_t *)((uintptr_t)LOAD_INFO_ASSET(pldinf) + \
 (pldinf)->nassets * sizeof(struct asset_desc_t)))
```

Definition at line 44 of file `spm_load_api.h`.

### 8.449.1.6 LOAD\_INFO\_SERVICE

```
#define LOAD_INFO_SERVICE(
 pldinf)
```

**Value:**

```
((const struct service_load_info_t *)((uintptr_t)LOAD_INFO_DEPS(pldinf) + \
 (pldinf)->ndeps * sizeof(uint32_t)))
```

Definition at line 38 of file `spm_load_api.h`.

### 8.449.1.7 LOAD\_INFSZ\_BYTES

```
#define LOAD_INFSZ_BYTES(
 pldinf)
```

**Value:**

```
(sizeof(*(pldinf)) + (LOAD_INFO_EXT_LENGTH * sizeof(uintptr_t)) + \
 (pldinf)->ndeps * sizeof(uint32_t)) + \
 (pldinf)->nservices * sizeof(struct service_load_info_t)) +
```

```
((pldinf)->nassets * sizeof(struct asset_desc_t)) +
((pldinf)->nirqs * sizeof(struct irq_load_info_t))
```

Definition at line 26 of file spm\_load\_api.h.

#### 8.449.1.8 NO\_MORE\_PARTITION

```
#define NO_MORE_PARTITION NULL
```

Definition at line 21 of file spm\_load\_api.h.

### 8.449.2 Function Documentation

#### 8.449.2.1 load\_a\_partition\_assuredly()

```
struct partition_t* load_a_partition_assuredly (
 struct partition_head_t * head)
```

Definition at line 103 of file rom\_loader.c.

Here is the call graph for this function:

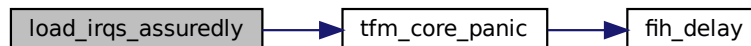


#### 8.449.2.2 load\_irqs\_assuredly()

```
void load_irqs_assuredly (
 struct partition_t * p_partition)
```

Definition at line 232 of file rom\_loader.c.

Here is the call graph for this function:



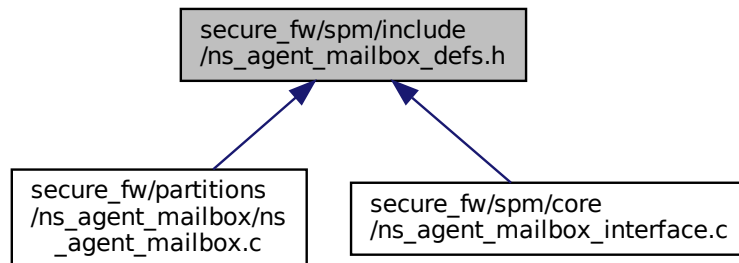
#### 8.449.2.3 load\_services\_assuredly()

```
uint32_t load_services_assuredly (
 struct partition_t * p_partition,
 struct service_head_t * services_listhead,
 struct service_t ** stateless_services_ref_tbl,
 size_t ref_tbl_size)
```

Definition at line 178 of file rom\_loader.c.

## 8.450 secure\_fw/spm/include/ns\_agent\_mailbox\_defs.h File Reference

This graph shows which files directly or indirectly include this file:



### Macros

- `#define BOOT_NS_CORE 1001`
- `#define DISABLE_MAILBOX 1002`
- `#define ENABLE_MAILBOX 1003`
- `#define NS_CORE_SHUTDOWN 1004`

### 8.450.1 Macro Definition Documentation

#### 8.450.1.1 BOOT\_NS\_CORE

```
#define BOOT_NS_CORE 1001
```

Definition at line 13 of file ns\_agent\_mailbox\_defs.h.

#### 8.450.1.2 DISABLE\_MAILBOX

```
#define DISABLE_MAILBOX 1002
```

Definition at line 14 of file ns\_agent\_mailbox\_defs.h.

#### 8.450.1.3 ENABLE\_MAILBOX

```
#define ENABLE_MAILBOX 1003
```

Definition at line 15 of file ns\_agent\_mailbox\_defs.h.

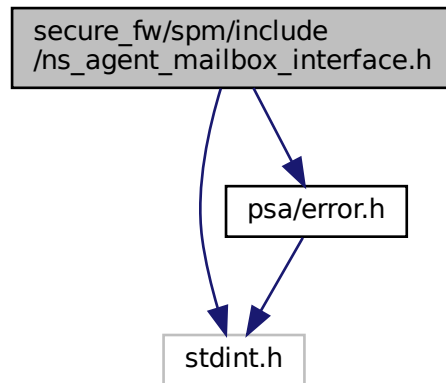
#### 8.450.1.4 NS\_CORE\_SHUTDOWN

```
#define NS_CORE_SHUTDOWN 1004
```

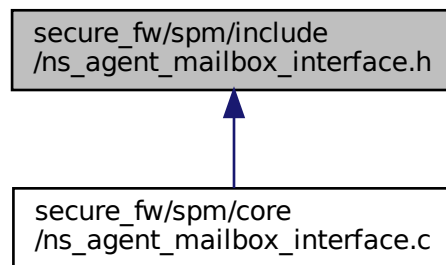
Definition at line 16 of file ns\_agent\_mailbox\_defs.h.

## 8.451 secure\_fw/spm/include/ns\_agent\_mailbox\_interface.h File Reference

```
#include <stdint.h>
#include "psa/error.h"
Include dependency graph for ns_agent_mailbox_interface.h:
```



This graph shows which files directly or indirectly include this file:



### Functions

- [psa\\_status\\_t ns\\_agent\\_boot\\_ns\\_core \(void\)](#)  
*Boot the non-secure core.*
- [psa\\_status\\_t ns\\_agent\\_mailbox\\_enable \(void\)](#)  
*Enable the mailbox.*
- [psa\\_status\\_t ns\\_agent\\_mailbox\\_disable \(void\)](#)  
*Disable the mailbox.*
- [psa\\_status\\_t ns\\_agent\\_ns\\_core\\_shutdown \(void\)](#)  
*Inform ns\_agent\_mailbox that the NS core has shut down.*

## 8.451.1 Function Documentation

### 8.451.1.1 ns\_agent\_boot\_ns\_core()

```
psa_status_t ns_agent_boot_ns_core (
 void)
```

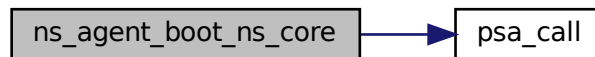
Boot the non-secure core.

#### Returns

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 16 of file ns\_agent\_mailbox\_interface.c.

Here is the call graph for this function:



### 8.451.1.2 ns\_agent\_mailbox\_disable()

```
psa_status_t ns_agent_mailbox_disable (
 void)
```

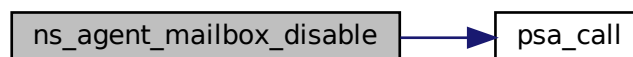
Disable the mailbox.

#### Returns

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 26 of file ns\_agent\_mailbox\_interface.c.

Here is the call graph for this function:



### 8.451.1.3 ns\_agent\_mailbox\_enable()

```
psa_status_t ns_agent_mailbox_enable (
 void)
```

Enable the mailbox.

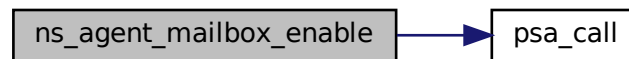


**Returns**

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 21 of file ns\_agent\_mailbox\_interface.c.

Here is the call graph for this function:

**8.451.1.4 ns\_agent\_ns\_core\_shutdown()**

```

psa_status_t ns_agent_ns_core_shutdown (
 void)

```

Inform ns\_agent\_mailbox that the NS core has shut down.

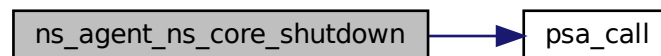
Before re-enabling the mailbox, [ns\\_agent\\_boot\\_ns\\_core\(\)](#) must be called.

**Returns**

Returns values as specified by the [psa\\_status\\_t](#)

Definition at line 31 of file ns\_agent\_mailbox\_interface.c.

Here is the call graph for this function:

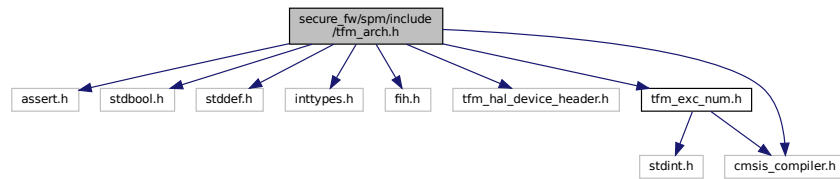
**8.452 secure\_fw/spm/include/tfm\_arch.h File Reference**

```

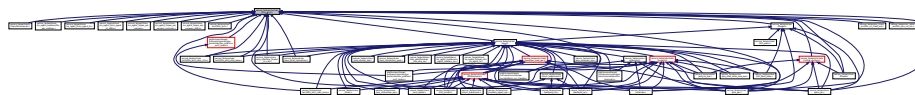
#include <assert.h>
#include <stdbool.h>
#include <stddef.h>
#include <inttypes.h>
#include "fih.h"
#include "tfm_hal_device_header.h"
#include "tfm_exc_num.h"
#include "cmsis_compiler.h"

```

Include dependency graph for tfm\_arch.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [tfm\\_state\\_context\\_t](#)
- struct [tfm\\_additional\\_context\\_t](#)
- struct [full\\_context\\_t](#)
- struct [context\\_ctrl\\_t](#)
- struct [context\\_flih\\_ret\\_t](#)

## Macros

- #define [SCHEDULER\\_ATTEMPTED](#) 2 /\* Schedule attempt when scheduler is locked. \*/
- #define [SCHEDULER\\_LOCKED](#) 1
- #define [SCHEDULER\\_UNLOCKED](#) 0
- #define [XPSR\\_T32](#) 0x01000000
- #define [PENDSV\\_PRIO\\_FOR\\_SCHED](#) ((1 << \_\_NVIC\_PRIO\_BITS) - 1)
- #define [TFM\\_FPU\\_CONTEXT\\_SIZE](#) 0
- #define [ARCH\\_CTXCTRL\\_INIT](#)(x, buf, sz)
- #define [ARCH\\_CTXCTRL\\_ALLOCATE\\_STACK](#)(x, size)
- #define [ARCH\\_CTXCTRL\\_ALLOCATED\\_PTR](#)(x) ((x)->sp)
- #define [ARCH\\_CTXCTRL\\_EXCRET\\_PATTERN](#)(x, param0, param1, param2, param3, pfn, pfnlr)
- #define [ARCH\\_STATE\\_CTX\\_SET\\_R0](#)(x, r0\_val) ((x)->r0 = (uint32\_t)(r0\_val))
- #define [ARCH\\_CLAIM\\_CTXCTRL\\_INSTANCE](#)(name, stack\_buf, stack\_size)
- #define [ARCH\\_FLUSH\\_FP\\_CONTEXT](#)()

## Functions

- [\\_\\_STATIC\\_INLINE uint32\\_t \\_\\_save\\_disable\\_irq](#) (void)
- [\\_\\_STATIC\\_INLINE void \\_\\_restore\\_irq](#) (uint32\_t status)
- [\\_\\_STATIC\\_INLINE void \\_\\_set\\_CONTROL\\_nPRIV](#) (uint32\_t nPRIV)
- [\\_\\_STATIC\\_INLINE bool tfm\\_arch\\_is\\_priv](#) (void)  
*Whether in privileged level.*
- [void tfm\\_arch\\_set\\_secure\\_exception\\_priorities](#) (void)
- [void tfm\\_arch\\_config\\_extensions](#) (void)
- [void tfm\\_arch\\_free\\_msp\\_and\\_exc\\_ret](#) (uint32\_t msp\_base, uint32\_t exc\_return)
- [void tfm\\_arch\\_set\\_context\\_ret\\_code](#) (const struct [context\\_ctrl\\_t](#) \*p\_ctx\_ctrl, uint32\_t ret\_code)
- [void tfm\\_arch\\_init\\_context](#) (struct [context\\_ctrl\\_t](#) \*p\_ctx\_ctrl, uintptr\_t pfn, void \*param, uintptr\_t pfnlr)
- [uint32\\_t tfm\\_arch\\_refresh\\_hardware\\_context](#) (const struct [context\\_ctrl\\_t](#) \*p\_ctx\_ctrl)

- `void arch_acquire_sched_lock (void)`
- `uint32_t arch_release_sched_lock (void)`
- `uint32_t arch_attempt_schedule (void)`
- `void tfm_arch_thread_fn_call (uint32_t a0, uint32_t a1, uint32_t a2, uint32_t a3)`
- `void arch_clean_stack_and_launch (void *param, uintptr_t spm_init_func, uintptr_t ns_agent_entry, uint32_t msp_base)`

## 8.452.1 Macro Definition Documentation

### 8.452.1.1 ARCH\_CLAIM\_CTXCTRL\_INSTANCE

```
#define ARCH_CLAIM_CTXCTRL_INSTANCE(
 name,
 stack_buf,
 stack_size)
```

**Value:**

```
struct context_ctrl_t name = {
 .sp = (uint32_t)&stack_buf[stack_size],
 .sp_base = (uint32_t)&stack_buf[stack_size],
 .sp_limit = (uint32_t)stack_buf,
 .exc_ret = 0,
}
```

Definition at line 203 of file `tfm_arch.h`.

### 8.452.1.2 ARCH\_CTXCTRL\_ALLOCATE\_STACK

```
#define ARCH_CTXCTRL_ALLOCATE_STACK(
 x,
 size)
```

**Value:**

```
do { \
 assert(size <= ((x)->sp - (x)->sp_limit)); \
 ((x)->sp -= ((size) + 7) & ~0x7); \
} while (0)
```

Definition at line 175 of file `tfm_arch.h`.

### 8.452.1.3 ARCH\_CTXCTRL\_ALLOCATED\_PTR

```
#define ARCH_CTXCTRL_ALLOCATED_PTR(
 x) ((x)->sp)
```

Definition at line 181 of file `tfm_arch.h`.

### 8.452.1.4 ARCH\_CTXCTRL\_EXCRET\_PATTERN

```
#define ARCH_CTXCTRL_EXCRET_PATTERN(
 x,
 param0,
 param1,
 param2,
 param3,
 pfn,
 pfnlr)
```

**Value:**

```
do { \
 (x)->r0 = (uint32_t)(param0);
 (x)->r1 = (uint32_t)(param1);
 (x)->r2 = (uint32_t)(param2);
 (x)->r3 = (uint32_t)(param3);
 (x)->ra = (uint32_t)(pfn);
```

```

 (x)->lr = (uint32_t)(pfnlr);
 (x)->xpsr = XPSR_T32;
 } while (0)

```

Definition at line 184 of file tfm\_arch.h.

#### 8.452.1.5 ARCH\_CTXCTRL\_INIT

```

#define ARCH_CTXCTRL_INIT(
 x,
 buf,
 sz)

```

Value:

```

 do {
 (x)->sp = ((uint32_t)(buf) + (uint32_t)(sz)) & ~0x7;
 (x)->sp_limit = ((uint32_t)(buf) + 7) & ~0x7;
 (x)->sp_base = (x)->sp;
 (x)->exc_ret = 0;
 } while (0)

```

Definition at line 167 of file tfm\_arch.h.

#### 8.452.1.6 ARCH\_FLUSH\_FP\_CONTEXT

```

#define ARCH_FLUSH_FP_CONTEXT()

```

Definition at line 262 of file tfm\_arch.h.

#### 8.452.1.7 ARCH\_STATE\_CTX\_SET\_R0

```

#define ARCH_STATE_CTX_SET_R0(
 x,
 r0_val) ((x)->r0 = (uint32_t)(r0_val))

```

Definition at line 195 of file tfm\_arch.h.

#### 8.452.1.8 PENDSV\_PRIO\_FOR\_SCHED

```

#define PENDSV_PRIO_FOR_SCHED ((1 << __NVIC_PRIO_BITS) - 1)

```

Definition at line 85 of file tfm\_arch.h.

#### 8.452.1.9 SCHEDULER\_ATTEMPTED

```

#define SCHEDULER_ATTEMPTED 2 /* Schedule attempt when scheduler is locked. */

```

Definition at line 31 of file tfm\_arch.h.

#### 8.452.1.10 SCHEDULER\_LOCKED

```

#define SCHEDULER_LOCKED 1

```

Definition at line 32 of file tfm\_arch.h.

#### 8.452.1.11 SCHEDULER\_UNLOCKED

```

#define SCHEDULER_UNLOCKED 0

```

Definition at line 33 of file tfm\_arch.h.

#### 8.452.1.12 TFM\_FPU\_CONTEXT\_SIZE

```
#define TFM_FPU_CONTEXT_SIZE 0
```

Definition at line 119 of file tfm\_arch.h.

#### 8.452.1.13 XPSR\_T32

```
#define XPSR_T32 0x01000000
```

Definition at line 35 of file tfm\_arch.h.

### 8.452.2 Function Documentation

#### 8.452.2.1 \_\_restore\_irq()

```
__STATIC_INLINE void __restore_irq (
 uint32_t status)
```

Definition at line 219 of file tfm\_arch.h.

#### 8.452.2.2 \_\_save\_disable\_irq()

```
__STATIC_INLINE uint32_t __save_disable_irq (
 void)
```

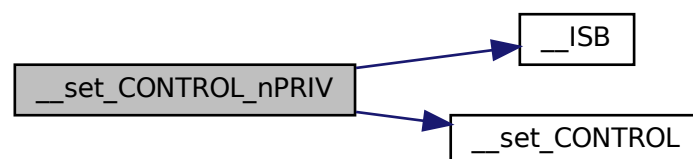
Definition at line 211 of file tfm\_arch.h.

#### 8.452.2.3 \_\_set\_CONTROL\_nPRIV()

```
__STATIC_INLINE void __set_CONTROL_nPRIV (
 uint32_t nPRIV)
```

Definition at line 225 of file tfm\_arch.h.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.452.2.4 arch\_acquire\_sched\_lock()

```
void arch_acquire_sched_lock (
 void)
```

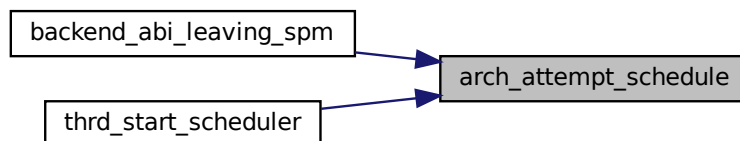
Here is the caller graph for this function:



#### 8.452.2.5 arch\_attempt\_schedule()

```
uint32_t arch_attempt_schedule (
 void)
```

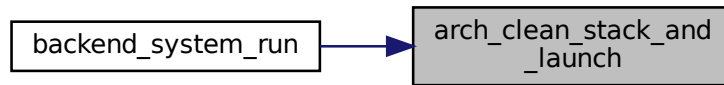
Here is the caller graph for this function:



#### 8.452.2.6 arch\_clean\_stack\_and\_launch()

```
void arch_clean_stack_and_launch (
 void * param,
 uintptr_t spm_init_func,
 uintptr_t ns_agent_entry,
 uint32_t msp_base)
```

Here is the caller graph for this function:



#### 8.452.2.7 arch\_release\_sched\_lock()

```
uint32_t arch_release_sched_lock (
 void)
```

Here is the caller graph for this function:



#### 8.452.2.8 tfm\_arch\_config\_extensions()

```
void tfm_arch_config_extensions (
 void)
```

Definition at line 247 of file `tfm_arch_v6m_v7m.c`.

Here is the call graph for this function:



#### 8.452.2.9 tfm\_arch\_free\_msp\_and\_exc\_ret()

```
void tfm_arch_free_msp_and_exc_ret (
 uint32_t msp_base,
 uint32_t exc_return)
```

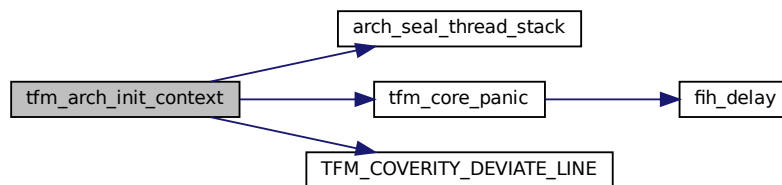
Definition at line 21 of file tfm\_arch.c.

#### 8.452.2.10 tfm\_arch\_init\_context()

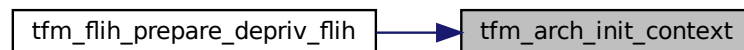
```
void tfm_arch_init_context (
 struct context_ctrl_t * p_ctx_ctrl,
 uintptr_t pfn,
 void * param,
 uintptr_t pfnlr)
```

Definition at line 137 of file tfm\_arch.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.452.2.11 tfm\_arch\_is\_priv()

```
__STATIC_INLINE bool tfm_arch_is_priv (
 void)
```

Whether in privileged level.

Return values

|              |                                                  |
|--------------|--------------------------------------------------|
| <i>true</i>  | If current execution runs in privileged level.   |
| <i>false</i> | If current execution runs in unprivileged level. |

Definition at line 241 of file tfm\_arch.h.



Here is the caller graph for this function:

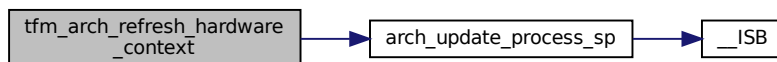


#### 8.452.2.12 tfm\_arch\_refresh\_hardware\_context()

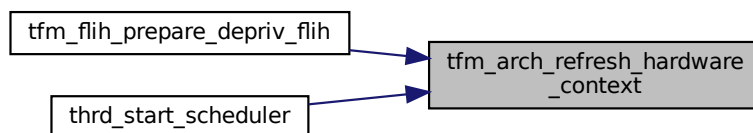
```
uint32_t tfm_arch_refresh_hardware_context (
 const struct context_ctrl_t * p_ctx_ctrl)
```

Definition at line 172 of file tfm\_arch.c.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 8.452.2.13 tfm\_arch\_set\_context\_ret\_code()

```
void tfm_arch_set_context_ret_code (
 const struct context_ctrl_t * p_ctx_ctrl,
 uint32_t ret_code)
```

#### 8.452.2.14 tfm\_arch\_set\_secure\_exception\_priorities()

```
void tfm_arch_set_secure_exception_priorities (
 void)
```

Definition at line 208 of file tfm\_arch\_v6m\_v7m.c.

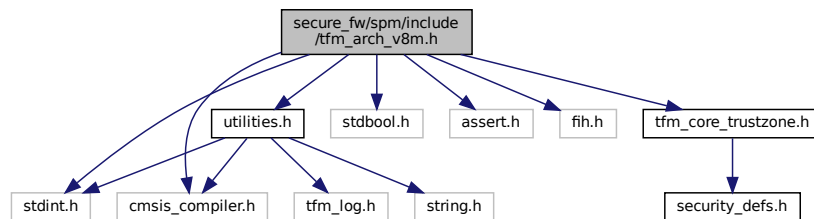
### 8.452.2.15 tfm\_arch\_thread\_fn\_call()

```
void tfm_arch_thread_fn_call (
 uint32_t a0,
 uint32_t a1,
 uint32_t a2,
 uint32_t a3)
```

## 8.453 secure\_fw/spm/include/tfm\_arch\_v8m.h File Reference

```
#include <stdint.h>
#include <stdbool.h>
#include <assert.h>
#include "cmsis_compiler.h"
#include "fih.h"
#include "tfm_core_trustzone.h"
#include "utilities.h"
```

Include dependency graph for tfm\_arch\_v8m.h:



## Macros

- #define [EXC\\_RETURN\\_RES1](#) (0x1FFFFFUL << 7)
- #define [EXC\\_RETURN\\_THREAD\\_PSP](#)
- #define [EXC\\_RETURN\\_THREAD\\_MSP](#)
- #define [EXC\\_RETURN\\_HANDLER](#)
- #define [SCB\\_ICSR\\_ADDR](#) (0xE000ED04)
- #define [SCB\\_ICSR\\_PENDSVSET\\_BIT](#) (0x10000000)
- #define [TFM\\_NS\\_EXC\\_DISABLE](#)() \_\_TZ\_set\_PRIMASK\_NS(1)
- #define [TFM\\_NS\\_EXC\\_ENABLE](#)() \_\_TZ\_set\_PRIMASK\_NS(0)

## Functions

- \_\_STATIC\_INLINE bool [is\\_return\\_secure\\_stack](#) (uint32\_t lr)  
*Check whether Secure or Non-secure stack is used to restore stack frame on exception return.*
- \_\_STATIC\_INLINE bool [is\\_default\\_stacking\\_rules\\_apply](#) (uint32\_t lr)  
*Check whether the default stacking rules apply, or whether the Additional state context, also known as callee registers, are already on the stack. DCRS bit is only present from V8M and above. If DCRS is 0 then Stack contains: r0, r1, r2, r3, r12, r14 (lr), the return address and xPSR.*
- \_\_STATIC\_INLINE bool [is\\_stack\\_alloc\\_fp\\_space](#) (uint32\_t lr)  
*Check whether the stack frame for this exception has space allocated for Floating Point(FP) state information.*
- \_\_STATIC\_INLINE uint32\_t [tfm\\_arch\\_get\\_psplim](#) (void)  
*Get value of PSPLIM register.*
- \_\_STATIC\_INLINE void [tfm\\_arch\\_set\\_psplim](#) (uint32\_t psplim)  
*Set PSPLIM register.*

- `__STATIC_INLINE void tfm_arch_set_msplim (uint32_t msplim)`  
*Set MSP limit value.*
- `__STATIC_INLINE uintptr_t arch_seal_thread_stack (uintptr_t stk)`  
*Seal the thread stack.*
- `__STATIC_INLINE void tfm_arch_check_msp_sealing (void)`  
*Check MSP sealing.*
- `__STATIC_INLINE void arch_update_process_sp (uint32_t bottom, uint32_t toplimit)`
- `__STATIC_INLINE void tfm_arch_config_branch_protection (void)`  
*Set branch protection PACBTI for v8.1M Main Extension.*

## Variables

- `uint64_t __STACK_SEAL`

## 8.453.1 Macro Definition Documentation

### 8.453.1.1 EXC\_RETURN\_HANDLER

```
#define EXC_RETURN_HANDLER
```

**Value:**

```
EXC_RETURN_PREFIX | EXC_RETURN_RES1 | \
EXC_RETURN_S | EXC_RETURN_DCRS | \
EXC_RETURN_FTYPE | EXC_RETURN_ES
```

Definition at line 36 of file `tfm_arch_v8m.h`.

### 8.453.1.2 EXC\_RETURN\_RES1

```
#define EXC_RETURN_RES1 (0x1FFFFFUL << 7)
```

Definition at line 21 of file `tfm_arch_v8m.h`.

### 8.453.1.3 EXC\_RETURN\_THREAD\_MSP

```
#define EXC_RETURN_THREAD_MSP
```

**Value:**

```
EXC_RETURN_PREFIX | EXC_RETURN_RES1 | \
EXC_RETURN_S | EXC_RETURN_DCRS | \
EXC_RETURN_FTYPE | EXC_RETURN_MODE | \
EXC_RETURN_ES
```

Definition at line 30 of file `tfm_arch_v8m.h`.

### 8.453.1.4 EXC\_RETURN\_THREAD\_PSP

```
#define EXC_RETURN_THREAD_PSP
```

**Value:**

```
EXC_RETURN_PREFIX | EXC_RETURN_RES1 | \
EXC_RETURN_S | EXC_RETURN_DCRS | \
EXC_RETURN_FTYPE | EXC_RETURN_MODE | \
EXC_RETURN_SPSEL | EXC_RETURN_ES
```

Definition at line 24 of file `tfm_arch_v8m.h`.

### 8.453.1.5 SCB\_ICSR\_ADDR

```
#define SCB_ICSR_ADDR (0xE000ED04)
```

Definition at line 41 of file `tfm_arch_v8m.h`.

### 8.453.1.6 SCB\_ICSR\_PENDSVSET\_BIT

```
#define SCB_ICSR_PENDSVSET_BIT (0x10000000)
```

Definition at line 42 of file `tfm_arch_v8m.h`.

### 8.453.1.7 TFM\_NS\_EXC\_DISABLE

```
#define TFM_NS_EXC_DISABLE() __TZ_set_PRIMASK_NS(1)
```

Definition at line 45 of file `tfm_arch_v8m.h`.

### 8.453.1.8 TFM\_NS\_EXC\_ENABLE

```
#define TFM_NS_EXC_ENABLE() __TZ_set_PRIMASK_NS(0)
```

Definition at line 47 of file `tfm_arch_v8m.h`.

## 8.453.2 Function Documentation

### 8.453.2.1 arch\_seal\_thread\_stack()

```
__STATIC_INLINE uintptr_t arch_seal_thread_stack (
 uintptr_t stk)
```

Seal the thread stack.

This function must be called only when the caller is using MSP.

#### Parameters

|           |            |                       |
|-----------|------------|-----------------------|
| <i>in</i> | <i>stk</i> | Thread stack address. |
|-----------|------------|-----------------------|

#### Return values

|              |                               |
|--------------|-------------------------------|
| <i>stack</i> | Updated thread stack address. |
|--------------|-------------------------------|

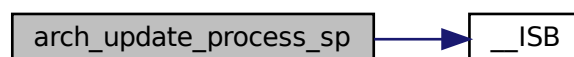
Definition at line 150 of file `tfm_arch_v8m.h`.

### 8.453.2.2 arch\_update\_process\_sp()

```
__STATIC_INLINE void arch_update_process_sp (
 uint32_t bottom,
 uint32_t toplimit)
```

Definition at line 181 of file `tfm_arch_v8m.h`.

Here is the call graph for this function:



**8.453.2.3 is\_default\_stacking\_rules\_apply()**

```
__STATIC_INLINE bool is_default_stacking_rules_apply (
 uint32_t lr)
```

Check whether the default stacking rules apply, or whether the Additional state context, also known as callee registers, are already on the stack. DCRS bit is only present from V8M and above. If DCRS is 0 then Stack contains: r0, r1, r2, r3, r12, r14 (lr), the return address and xPSR.

If DCRS is 1 then the stack contains the following too before the caller-saved registers: Integrity signature, res, r4, r5, r6, r7, r8, r9, r10, r11

**Parameters**

|    |    |                                              |
|----|----|----------------------------------------------|
| in | lr | LR register containing the EXC_RETURN value. |
|----|----|----------------------------------------------|

**Return values**

|       |                                                                             |
|-------|-----------------------------------------------------------------------------|
| true  | Default rules for stacking the Additional state context registers followed. |
| false | Stacking of the Additional state context registers skipped.                 |

Definition at line 86 of file tfm\_arch\_v8m.h.

**8.453.2.4 is\_return\_secure\_stack()**

```
__STATIC_INLINE bool is_return_secure_stack (
 uint32_t lr)
```

Check whether Secure or Non-secure stack is used to restore stack frame on exception return.

**Parameters**

|    |    |                                              |
|----|----|----------------------------------------------|
| in | lr | LR register containing the EXC_RETURN value. |
|----|----|----------------------------------------------|

**Return values**

|       |                                                                      |
|-------|----------------------------------------------------------------------|
| true  | Secure stack is used to restore stack frame on exception return.     |
| false | Non-secure stack is used to restore stack frame on exception return. |

Definition at line 62 of file tfm\_arch\_v8m.h.

**8.453.2.5 is\_stack\_alloc\_fp\_space()**

```
__STATIC_INLINE bool is_stack_alloc_fp_space (
 uint32_t lr)
```

Check whether the stack frame for this exception has space allocated for Floating Point(FP) state information.

**Parameters**

|    |    |                                              |
|----|----|----------------------------------------------|
| in | lr | LR register containing the EXC_RETURN value. |
|----|----|----------------------------------------------|

**Return values**

|       |                                                     |
|-------|-----------------------------------------------------|
| true  | The stack allocates space for FP information        |
| false | The stack doesn't allocate space for FP information |

Definition at line 100 of file tfm\_arch\_v8m.h.

#### 8.453.2.6 tfm\_arch\_check\_msp\_sealing()

```
__STATIC_INLINE void tfm_arch_check_msp_sealing (
 void)
```

Check MSP sealing.

Sealing must be done in the [Reset\\_Handler\(\)](#) on a 8 byte region (\_\_STACK\_SEAL) defined in the linker scripts. (It is a CMSIS recommendation)

Definition at line 171 of file tfm\_arch\_v8m.h.

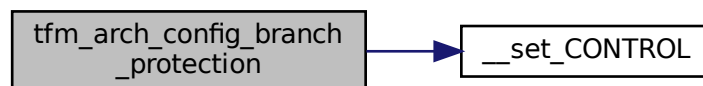
#### 8.453.2.7 tfm\_arch\_config\_branch\_protection()

```
__STATIC_INLINE void tfm_arch_config_branch_protection (
 void)
```

Set branch protection PACBTI for v8.1M Main Extension.

Definition at line 192 of file tfm\_arch\_v8m.h.

Here is the call graph for this function:



#### 8.453.2.8 tfm\_arch\_get\_psplim()

```
__STATIC_INLINE uint32_t tfm_arch_get_psplim (
 void)
```

Get value of PSPLIM register.

##### Return values

|               |                                    |
|---------------|------------------------------------|
| <i>psplim</i> | Register value in PSPLIM register. |
|---------------|------------------------------------|

Definition at line 110 of file tfm\_arch\_v8m.h.

#### 8.453.2.9 tfm\_arch\_set\_msplim()

```
__STATIC_INLINE void tfm_arch_set_msplim (
 uint32_t msplim)
```

Set MSP limit value.

##### Parameters

|    |               |                                |
|----|---------------|--------------------------------|
| in | <i>msplim</i> | MSP limit value to be written. |
|----|---------------|--------------------------------|

Definition at line 130 of file tfm\_arch\_v8m.h.

**8.453.2.10 tfm\_arch\_set\_psplim()**

```
__STATIC_INLINE void tfm_arch_set_psplim (
 uint32_t psplim)
```

Set PSPLIM register.

**Parameters**

|    |               |                                           |
|----|---------------|-------------------------------------------|
| in | <i>psplim</i> | Register value to be written into PSPLIM. |
|----|---------------|-------------------------------------------|

Definition at line 120 of file tfm\_arch\_v8m.h.

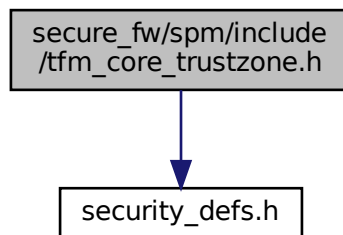
**8.453.3 Variable Documentation****8.453.3.1 \_\_STACK\_SEAL**

```
uint64_t __STACK_SEAL
```

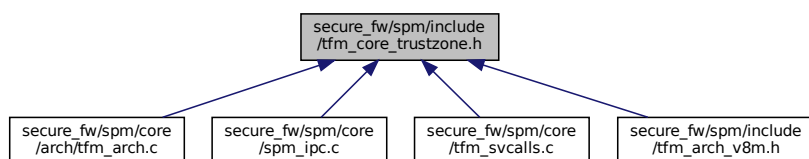
**8.454 secure\_fw/spm/include/tfm\_core\_trustzone.h File Reference**

```
#include "security_defs.h"
```

Include dependency graph for tfm\_core\_trustzone.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define TFM_STACK_SEALED_SIZE 8`
- `#define TFM_STACK_SEAL_VALUE STACK_SEAL_PATTERN`
- `#define TFM_STACK_SEAL_VALUE_64 (uint64_t)0xFE5EDA5FE5EDA5`
- `#define TFM_BASIC_FP_CONTEXT_WORDS 18`
- `#define TFM_ADDITIONAL_FP_CONTEXT_WORDS 16`
- `#define TFM_VENEER_LR_BIT0_MASK 1`

### 8.454.1 Macro Definition Documentation

#### 8.454.1.1 TFM\_ADDITIONAL\_FP\_CONTEXT\_WORDS

```
#define TFM_ADDITIONAL_FP_CONTEXT_WORDS 16
```

Definition at line 35 of file `tfm_core_trustzone.h`.

#### 8.454.1.2 TFM\_BASIC\_FP\_CONTEXT\_WORDS

```
#define TFM_BASIC_FP_CONTEXT_WORDS 18
```

Definition at line 29 of file `tfm_core_trustzone.h`.

#### 8.454.1.3 TFM\_STACK\_SEAL\_VALUE

```
#define TFM_STACK_SEAL_VALUE STACK_SEAL_PATTERN
```

Definition at line 22 of file `tfm_core_trustzone.h`.

#### 8.454.1.4 TFM\_STACK\_SEAL\_VALUE\_64

```
#define TFM_STACK_SEAL_VALUE_64 (uint64_t)0xFE5EDA5FE5EDA5
```

Definition at line 23 of file `tfm_core_trustzone.h`.

#### 8.454.1.5 TFM\_STACK\_SEALED\_SIZE

```
#define TFM_STACK_SEALED_SIZE 8
```

Definition at line 21 of file `tfm_core_trustzone.h`.

#### 8.454.1.6 TFM\_VENEER\_LR\_BIT0\_MASK

```
#define TFM_VENEER_LR_BIT0_MASK 1
```

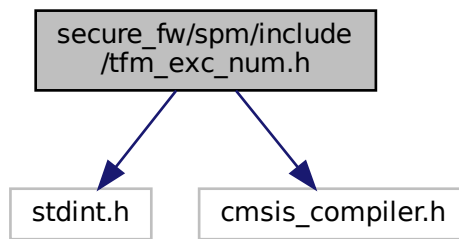
Definition at line 41 of file `tfm_core_trustzone.h`.

### 8.455 secure\_fw/spm/include/tfm\_exc\_num.h File Reference

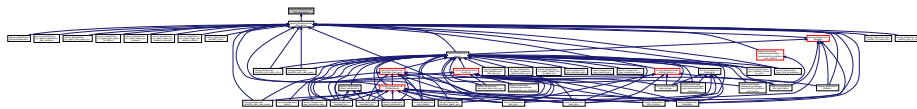
```
#include <stdint.h>
#include "cmsis_compiler.h"
```



Include dependency graph for tfm\_exc\_num.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define EXC_NUM_THREAD_MODE (0)`
- `#define EXC_NUM_SVCALL (11)`
- `#define EXC_NUM_PENDSV (14)`

## Functions

- `__STATIC_INLINE uint32_t __get_active_exc_num (void)`

## 8.455.1 Macro Definition Documentation

### 8.455.1.1 EXC\_NUM\_PENDSV

```
#define EXC_NUM_PENDSV (14)
```

Definition at line 22 of file tfm\_exc\_num.h.

### 8.455.1.2 EXC\_NUM\_SVCALL

```
#define EXC_NUM_SVCALL (11)
```

Definition at line 21 of file tfm\_exc\_num.h.

### 8.455.1.3 EXC\_NUM\_THREAD\_MODE

```
#define EXC_NUM_THREAD_MODE (0)
```

Definition at line 20 of file tfm\_exc\_num.h.

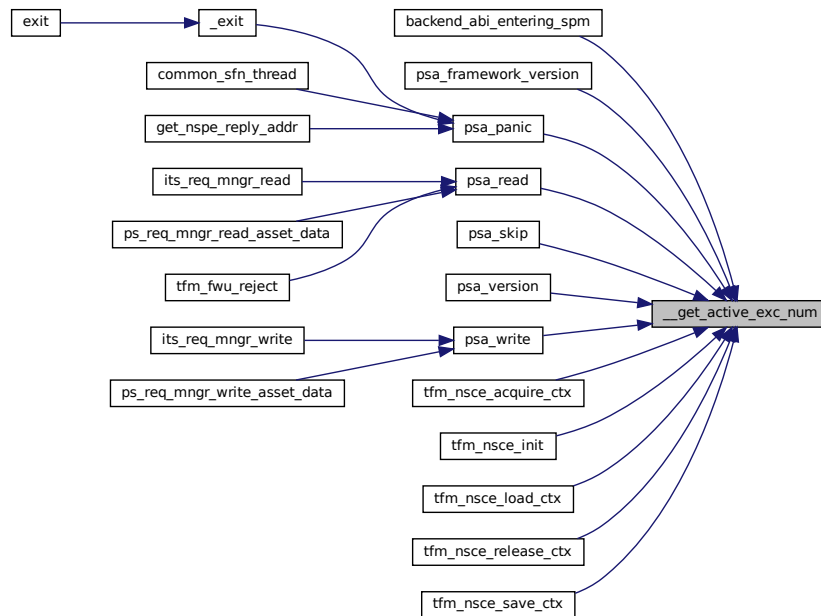
## 8.455.2 Function Documentation

### 8.455.2.1 `__get_active_exc_num()`

```
__STATIC_INLINE uint32_t __get_active_exc_num (
 void)
```

Definition at line 25 of file `tfm_exc_num.h`.

Here is the caller graph for this function:



## 8.456 `secure_fw/spm/include/tfm_hybrid_platform.h` File Reference

### Macros

- `#define TFM_HYBRID_PLAT_SCHED_OFF 0`
- `#define TFM_HYBRID_PLAT_SCHED_SPE 10`
- `#define TFM_HYBRID_PLAT_SCHED_NSPE 20`
- `#define TFM_HYBRID_PLAT_SCHED_BALANCED 30`

### 8.456.1 Macro Definition Documentation

#### 8.456.1.1 `TFM_HYBRID_PLAT_SCHED_BALANCED`

```
#define TFM_HYBRID_PLAT_SCHED_BALANCED 30
```

Definition at line 31 of file `tfm_hybrid_platform.h`.

#### 8.456.1.2 `TFM_HYBRID_PLAT_SCHED_NSPE`

```
#define TFM_HYBRID_PLAT_SCHED_NSPE 20
```

Definition at line 23 of file `tfm_hybrid_platform.h`.

### 8.456.1.3 TFM\_HYBRID\_PLAT\_SCHED\_OFF

```
#define TFM_HYBRID_PLAT_SCHED_OFF 0
```

Definition at line 10 of file tfm\_hybrid\_platform.h.

### 8.456.1.4 TFM\_HYBRID\_PLAT\_SCHED\_SPE

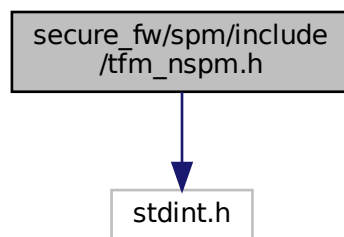
```
#define TFM_HYBRID_PLAT_SCHED_SPE 10
```

Definition at line 16 of file tfm\_hybrid\_platform.h.

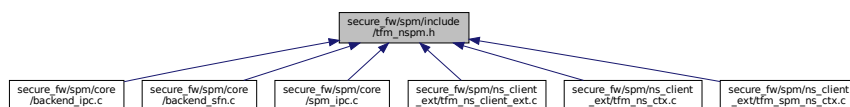
## 8.457 secure\_fw/spm/include/tfm\_nspm.h File Reference

```
#include <stdint.h>
```

Include dependency graph for tfm\_nspm.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define TFM_NS_CLIENT_INVALID_ID ((int32_t)0)`

## Functions

- `void tfm_nspm_ctx_init (void)`  
*initialise the NS context database*
- `int32_t tfm_nspm_get_current_client_id (void)`  
*Get the client ID of the current NS client.*
- `void tz_ns_agent_register_client_id_range (int32_t client_id_base, int32_t client_id_limit)`  
*Register a non-secure client ID range.*

## 8.457.1 Macro Definition Documentation

### 8.457.1.1 TFM\_NS\_CLIENT\_INVALID\_ID

```
#define TFM_NS_CLIENT_INVALID_ID ((int32_t)0)
```

Definition at line 15 of file tfm\_nspm.h.

## 8.457.2 Function Documentation

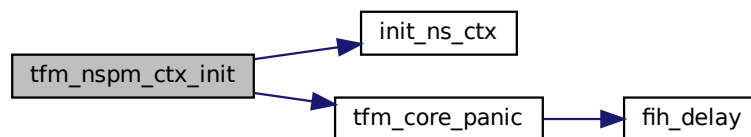
### 8.457.2.1 tfm\_nspm\_ctx\_init()

```
void tfm_nspm_ctx_init (
 void)
```

initialise the NS context database

Definition at line 90 of file tfm\_spm\_ns\_ctx.c.

Here is the call graph for this function:



### 8.457.2.2 tfm\_nspm\_get\_current\_client\_id()

```
int32_t tfm_nspm_get_current_client_id (
 void)
```

Get the client ID of the current NS client.

#### Returns

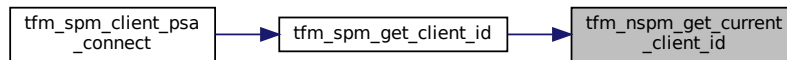
The client id of the current NS client. 0 (invalid client id) is returned in case of error.

Definition at line 75 of file tfm\_spm\_ns\_ctx.c.

Here is the call graph for this function:



Here is the caller graph for this function:



### 8.457.2.3 tz\_ns\_agent\_register\_client\_id\_range()

```
void tz_ns_agent_register_client_id_range (
 int32_t client_id_base,
 int32_t client_id_limit)
```

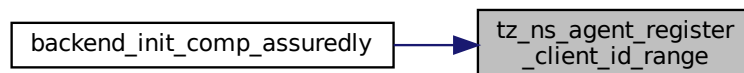
Register a non-secure client ID range.

#### Parameters

|    |                        |                                        |
|----|------------------------|----------------------------------------|
| in | <i>client_id_base</i>  | The minimum client ID for this client. |
| in | <i>client_id_limit</i> | The maximum client ID for this client. |

Definition at line 26 of file tfm\_spm\_ns\_ctx.c.

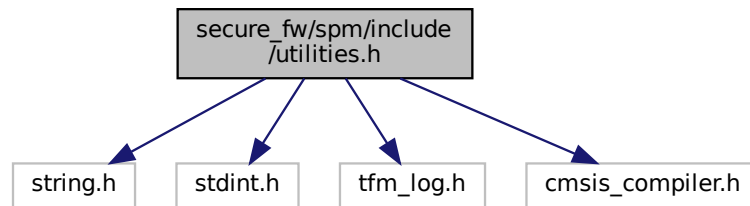
Here is the caller graph for this function:



## 8.458 secure\_fw/spm/include/utilities.h File Reference

```
#include <string.h>
#include <stdint.h>
#include "tfm_log.h"
#include "cmsis_compiler.h"
```

Include dependency graph for utilities.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define TO_CONTAINER(ptr, type, member) ((type *)((uintptr_t)(ptr) - offsetof(type, member)))`
- `#define ERROR_MSG(msg) ERROR_RAW(msg "\n")`
- `#define STRINGIFY_EXPAND(x) #x`
- `#define M2S(m) STRINGIFY_EXPAND(m)`
- `#define spm_memcpy memcpy`
- `#define spm_memset memset`

## Functions

- `__NO_RETURN void tfm_core_panic(void)`
- `uintptr_t get_spm_boundary(void)`

## 8.458.1 Macro Definition Documentation

### 8.458.1.1 ERROR\_MSG

```
#define ERROR_MSG(
 msg) ERROR_RAW(msg "\n")
```

Definition at line 30 of file utilities.h.

### 8.458.1.2 M2S

```
#define M2S(
 m) STRINGIFY_EXPAND(m)
```

Definition at line 34 of file utilities.h.

### 8.458.1.3 spm\_memcpy

```
#define spm_memcpy memcpy
```

Definition at line 38 of file utilities.h.

#### 8.458.1.4 spm\_memset

```
#define spm_memset memset
```

Definition at line 44 of file utilities.h.

#### 8.458.1.5 STRINGIFY\_EXPAND

```
#define STRINGIFY_EXPAND(
 x) #x
```

Definition at line 33 of file utilities.h.

#### 8.458.1.6 TO\_CONTAINER

```
#define TO_CONTAINER(
 ptr,
 type,
 member) ((type *) ((uintptr_t) (ptr) - offsetof(type, member)))
```

Definition at line 27 of file utilities.h.

### 8.458.2 Function Documentation

#### 8.458.2.1 get\_spm\_boundary()

```
uintptr_t get_spm_boundary (
 void)
```

Definition at line 103 of file main.c.

Here is the caller graph for this function:

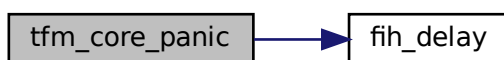


#### 8.458.2.2 tfm\_core\_panic()

```
__NO_RETURN void tfm_core_panic (
 void)
```

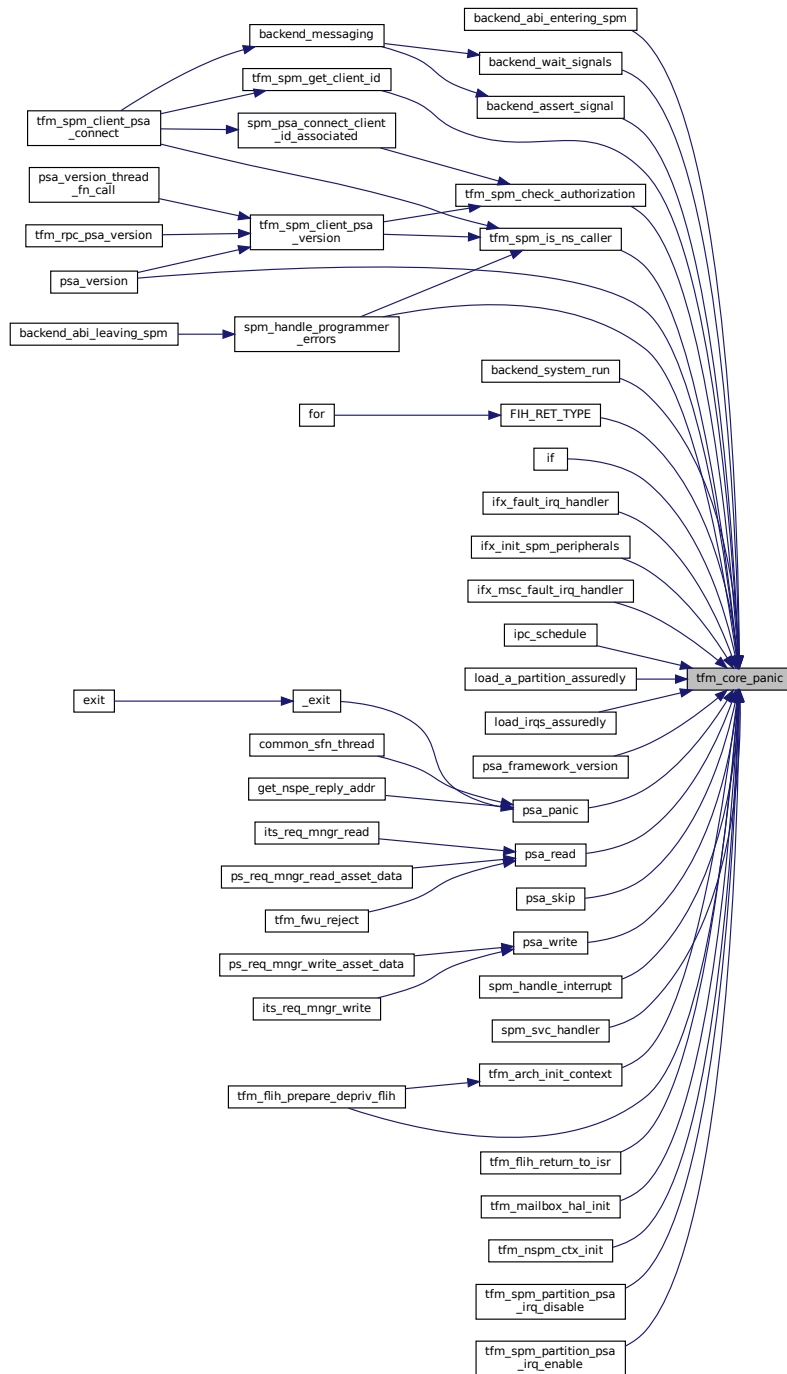
Definition at line 20 of file utilities.c.

Here is the call graph for this function:





Here is the caller graph for this function:



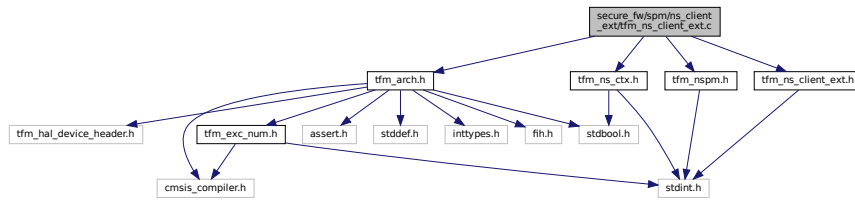
## 8.459 secure\_fw/spm/ns\_client\_ext/tfm\_ns\_client\_ext.c File Reference

```

#include "tfm_arch.h"
#include "tfm_nspm.h"
#include "tfm_ns_client_ext.h"
#include "tfm_ns_ctx.h"

```

Include dependency graph for tfm\_ns\_client\_ext.c:



## Macros

- `#define MAKE_NS_CLIENT_TOKEN(tid, gid, idx)`
- `#define IS_INVALID_TOKEN(token) ((token) & 0xff000000)`
- `#define NS_CLIENT_TOKEN_TO_CTX_IDX(token) (((token) >> 16) & 0xff)`
- `#define NS_CLIENT_TOKEN_TO_GID(token) (((token) >> 8) & 0xff)`
- `#define NS_CLIENT_TOKEN_TO_TID(token) ((token) & 0xff)`

## Functions

- `__tz_c_veneer uint32_t tfm_nsce_init (uint32_t ctx_requested)`  
*Initialize the non-secure client extension.*
- `__tz_c_veneer uint32_t tfm_nsce_acquire_ctx (uint8_t group_id, uint8_t thread_id)`  
*Acquire the context for a non-secure client.*
- `__tz_c_veneer uint32_t tfm_nsce_release_ctx (uint32_t token)`  
*Release the context for the non-secure client.*
- `__tz_c_veneer uint32_t tfm_nsce_load_ctx (uint32_t token, int32_t nsid)`  
*Load the context for the non-secure client.*
- `__tz_c_veneer uint32_t tfm_nsce_save_ctx (uint32_t token)`  
*Save the context for the non-secure client.*

## 8.459.1 Macro Definition Documentation

### 8.459.1.1 IS\_INVALID\_TOKEN

```
#define IS_INVALID_TOKEN(
 token) ((token) & 0xff000000)
```

Definition at line 24 of file tfm\_ns\_client\_ext.c.

### 8.459.1.2 MAKE\_NS\_CLIENT\_TOKEN

```
#define MAKE_NS_CLIENT_TOKEN(
 tid,
 gid,
 idx)
```

**Value:**

```
(uint32_t) (((uint32_t)tid & 0xff) \
| (((uint32_t)gid & 0xff) << 8) \
| (((uint32_t)idx & 0xff) << 16)) \
& 0x00ffffff)
```

Definition at line 19 of file tfm\_ns\_client\_ext.c.

### 8.459.1.3 NS\_CLIENT\_TOKEN\_TO\_CTX\_IDX

```
#define NS_CLIENT_TOKEN_TO_CTX_IDX(
 token) (((token) >> 16) & 0xff)
```

Definition at line 25 of file tfm\_ns\_client\_ext.c.

### 8.459.1.4 NS\_CLIENT\_TOKEN\_TO\_GID

```
#define NS_CLIENT_TOKEN_TO_GID(
 token) (((token) >> 8) & 0xff)
```

Definition at line 26 of file tfm\_ns\_client\_ext.c.

### 8.459.1.5 NS\_CLIENT\_TOKEN\_TO\_TID

```
#define NS_CLIENT_TOKEN_TO_TID(
 token) ((token) & 0xff)
```

Definition at line 27 of file tfm\_ns\_client\_ext.c.

## 8.459.2 Function Documentation

### 8.459.2.1 tfm\_nsce\_acquire\_ctx()

```
__tz_c_veneer uint32_t tfm_nsce_acquire_ctx (
 uint8_t group_id,
 uint8_t thread_id)
```

Acquire the context for a non-secure client.

This function should be called before a non-secure client calling the PSA API into TF-M. It is to request the allocation of the context for the upcoming service call from that non-secure client. The non-secure clients in one group share the same context. The thread ID is used to identify the different non-secure clients.

#### Parameters

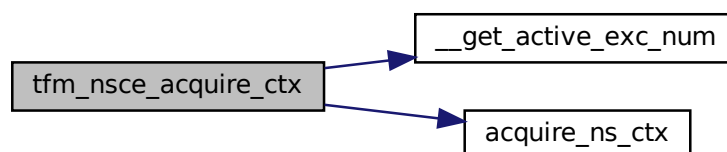
|    |                  |                                        |
|----|------------------|----------------------------------------|
| in | <i>group_id</i>  | The group ID of the non-secure client  |
| in | <i>thread_id</i> | The thread ID of the non-secure client |

#### Returns

Returns the token of the allocated context. 0xFFFFFFFF means the allocation failed and the token is invalid.

Definition at line 48 of file tfm\_ns\_client\_ext.c.

Here is the call graph for this function:



### 8.459.2.2 tfm\_nsce\_init()

```
__tz_c_veneer uint32_t tfm_nsce_init (
 uint32_t ctx_requested)
```

Initialize the non-secure client extension.

This function should be called before any other non-secure client APIs. It gives NSPE the opportunity to initialize the non-secure client extension in TF-M. Also, NSPE can get the number of allocated non-secure client context slots in the return value. That is useful if NSPE wants to decide the group (context) assignment at runtime.

#### Parameters

|    |                      |                                                                                                                        |
|----|----------------------|------------------------------------------------------------------------------------------------------------------------|
| in | <i>ctx_requested</i> | The number of non-secure context requested from the NS entity. If request maximum available context, then set it to 0. |
|----|----------------------|------------------------------------------------------------------------------------------------------------------------|

#### Returns

Returns the number of non-secure context allocated to the NS entity. The allocated context number  $\leq$  maximum supported context number. If the initialization is failed, then 0 is returned.

Definition at line 30 of file `tfm_ns_client_ext.c`.

Here is the call graph for this function:



### 8.459.2.3 tfm\_nsce\_load\_ctx()

```
__tz_c_veneer uint32_t tfm_nsce_load_ctx (
 uint32_t token,
 int32_t nsid)
```

Load the context for the non-secure client.

This function should be called when a non-secure client is going to be scheduled in at the non-secure side. The caller is usually the scheduler of the RTOS. The non-secure client ID is managed by the non-secure world and passed to TF-M as the input parameter of TF-M.

#### Parameters

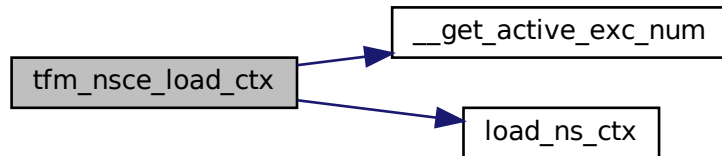
|    |              |                                                         |
|----|--------------|---------------------------------------------------------|
| in | <i>token</i> | The token returned by <code>tfm_nsce_acquire_ctx</code> |
| in | <i>nsid</i>  | The non-secure client ID for this client                |

**Returns**

Returns the error code.

Definition at line 96 of file tfm\_ns\_client\_ext.c.

Here is the call graph for this function:

**8.459.2.4 tfm\_nsce\_release\_ctx()**

```
__tz_c_veneer uint32_t tfm_nsce_release_ctx (
 uint32_t token)
```

Release the context for the non-secure client.

This function should be called when a non-secure client is going to be terminated or will not call TF-M secure services in the future. It is to release the context allocated for the calling non-secure client. If the calling non-secure client is the only thread in the group, then the context will be deallocated. Otherwise, the context will still be taken for the other threads in the group.

**Parameters**

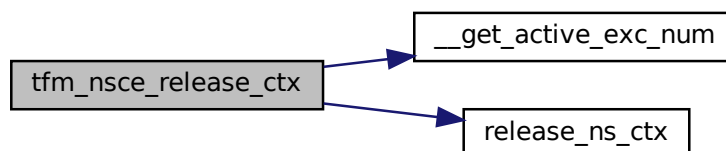
|    |              |                                                         |
|----|--------------|---------------------------------------------------------|
| in | <i>token</i> | The token returned by <code>tfm_nsce_acquire_ctx</code> |
|----|--------------|---------------------------------------------------------|

**Returns**

Returns the error code.

Definition at line 67 of file tfm\_ns\_client\_ext.c.

Here is the call graph for this function:



### 8.459.2.5 tfm\_nsce\_save\_ctx()

```
__tz_c_veiner uint32_t tfm_nsce_save_ctx (
 uint32_t token)
```

Save the context for the non-secure client.

This function should be called when a non-secure client is going to be scheduled out at the non-secure side. The caller is usually the scheduler of the RTOS.

#### Parameters

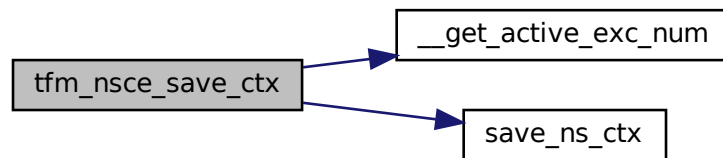
|    |              |                                                         |
|----|--------------|---------------------------------------------------------|
| in | <i>token</i> | The token returned by <code>tfm_nsce_acquire_ctx</code> |
|----|--------------|---------------------------------------------------------|

#### Returns

Returns the error code.

Definition at line 129 of file `tfm_ns_client_ext.c`.

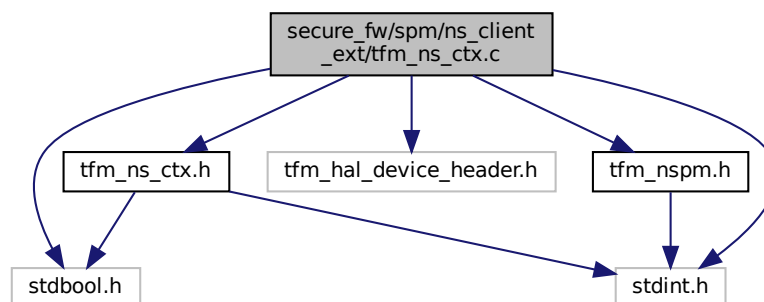
Here is the call graph for this function:



## 8.460 secure\_fw/spm/ns\_client\_ext/tfm\_ns\_ctx.c File Reference

```
#include <stdint.h>
#include <stdbool.h>
#include "tfm_hal_device_header.h"
#include "tfm_ns_ctx.h"
#include "tfm_nspm.h"
```

Include dependency graph for `tfm_ns_ctx.c`:



## Functions

- bool [init\\_ns\\_ctx](#) (void)
- bool [acquire\\_ns\\_ctx](#) (uint8\_t gid, uint8\_t \*idx)
- bool [release\\_ns\\_ctx](#) (uint8\_t gid, uint8\_t tid, uint8\_t idx)
- bool [load\\_ns\\_ctx](#) (uint8\_t gid, uint8\_t tid, int32\_t nsid, uint8\_t idx)
- bool [save\\_ns\\_ctx](#) (uint8\_t gid, uint8\_t tid, uint8\_t idx)
- int32\_t [get\\_nsid\\_from\\_active\\_ns\\_ctx](#) (void)

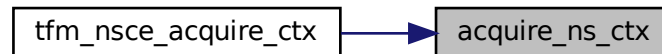
### 8.460.1 Function Documentation

#### 8.460.1.1 [acquire\\_ns\\_ctx\(\)](#)

```
bool acquire_ns_ctx (
 uint8_t gid,
 uint8_t * idx)
```

Definition at line 35 of file `tfm_ns_ctx.c`.

Here is the caller graph for this function:



#### 8.460.1.2 [get\\_nsid\\_from\\_active\\_ns\\_ctx\(\)](#)

```
int32_t get_nsid_from_active_ns_ctx (
 void)
```

Definition at line 183 of file `tfm_ns_ctx.c`.

Here is the caller graph for this function:

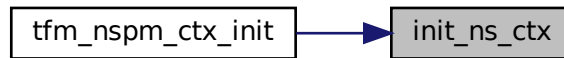


#### 8.460.1.3 [init\\_ns\\_ctx\(\)](#)

```
bool init_ns_ctx (
 void)
```

Definition at line 22 of file `tfm_ns_ctx.c`.

Here is the caller graph for this function:

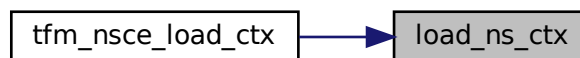


#### 8.460.1.4 load\_ns\_ctx()

```
bool load_ns_ctx (
 uint8_t gid,
 uint8_t tid,
 int32_t nsid,
 uint8_t idx)
```

Definition at line 138 of file `tfm_ns_ctx.c`.

Here is the caller graph for this function:

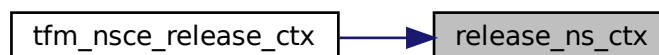


#### 8.460.1.5 release\_ns\_ctx()

```
bool release_ns_ctx (
 uint8_t gid,
 uint8_t tid,
 uint8_t idx)
```

Definition at line 90 of file `tfm_ns_ctx.c`.

Here is the caller graph for this function:



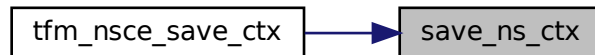


**8.460.1.6 save\_ns\_ctx()**

```
bool save_ns_ctx (
 uint8_t gid,
 uint8_t tid,
 uint8_t idx)
```

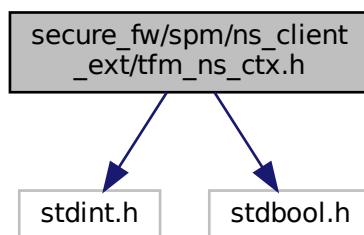
Definition at line 160 of file tfm\_ns\_ctx.c.

Here is the caller graph for this function:

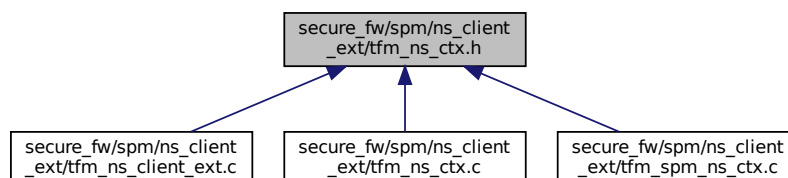
**8.461 secure\_fw/spm/ns\_client\_ext/tfm\_ns\_ctx.h File Reference**

```
#include <stdint.h>
#include <stdbool.h>
```

Include dependency graph for tfm\_ns\_ctx.h:



This graph shows which files directly or indirectly include this file:

**Data Structures**

- struct [tfm\\_ns\\_ctx\\_t](#)

## Macros

- `#define TFM_NS_CONTEXT_MAX 1`
- `#define TFM_NS_CONTEXT_MAX_TID 0xFF`

## Functions

- `bool init_ns_ctx (void)`
- `bool acquire_ns_ctx (uint8_t gid, uint8_t *idx)`
- `bool release_ns_ctx (uint8_t gid, uint8_t tid, uint8_t idx)`
- `bool load_ns_ctx (uint8_t gid, uint8_t tid, int32_t nsid, uint8_t idx)`
- `bool save_ns_ctx (uint8_t gid, uint8_t tid, uint8_t idx)`
- `int32_t get_nsid_from_active_ns_ctx (void)`

### 8.461.1 Macro Definition Documentation

#### 8.461.1.1 TFM\_NS\_CONTEXT\_MAX

```
#define TFM_NS_CONTEXT_MAX 1
```

Definition at line 14 of file `tfm_ns_ctx.h`.

#### 8.461.1.2 TFM\_NS\_CONTEXT\_MAX\_TID

```
#define TFM_NS_CONTEXT_MAX_TID 0xFF
```

Definition at line 16 of file `tfm_ns_ctx.h`.

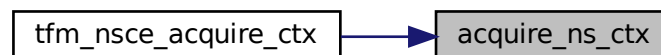
### 8.461.2 Function Documentation

#### 8.461.2.1 acquire\_ns\_ctx()

```
bool acquire_ns_ctx (
 uint8_t gid,
 uint8_t * idx)
```

Definition at line 35 of file `tfm_ns_ctx.c`.

Here is the caller graph for this function:

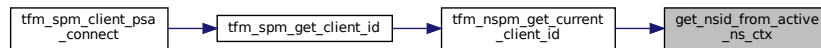


#### 8.461.2.2 get\_nsid\_from\_active\_ns\_ctx()

```
int32_t get_nsid_from_active_ns_ctx (
 void)
```

Definition at line 183 of file `tfm_ns_ctx.c`.

Here is the caller graph for this function:

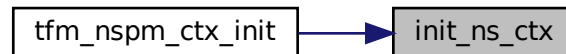


### 8.461.2.3 `init_ns_ctx()`

```
bool init_ns_ctx (
 void)
```

Definition at line 22 of file `tfm_ns_ctx.c`.

Here is the caller graph for this function:

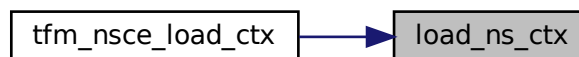


### 8.461.2.4 `load_ns_ctx()`

```
bool load_ns_ctx (
 uint8_t gid,
 uint8_t tid,
 int32_t nsid,
 uint8_t idx)
```

Definition at line 138 of file `tfm_ns_ctx.c`.

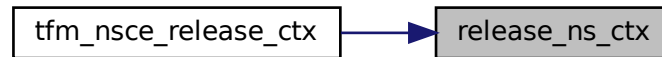
Here is the caller graph for this function:



### 8.461.2.5 `release_ns_ctx()`

```
bool release_ns_ctx (
 uint8_t gid,
 uint8_t tid,
 uint8_t idx)
```

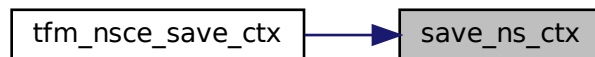
Definition at line 90 of file tfm\_ns\_ctx.c.  
Here is the caller graph for this function:



#### 8.461.2.6 save\_ns\_ctx()

```
bool save_ns_ctx (
 uint8_t gid,
 uint8_t tid,
 uint8_t idx)
```

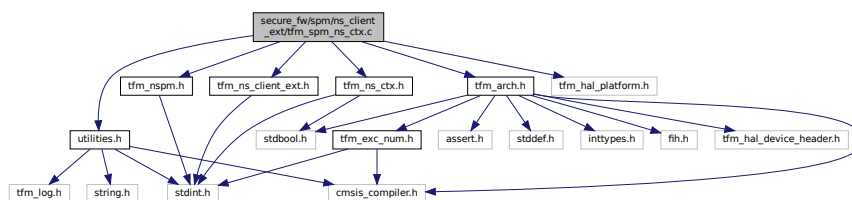
Definition at line 160 of file tfm\_ns\_ctx.c.  
Here is the caller graph for this function:



### 8.462 secure\_fw/spm/ns\_client\_ext/tfm\_spm\_ns\_ctx.c File Reference

```
#include "tfm_nspm.h"
#include "tfm_ns_ctx.h"
#include "tfm_ns_client_ext.h"
#include "utilities.h"
#include "tfm_arch.h"
#include "tfm_hal_platform.h"
```

Include dependency graph for `tfm_spm_ns_ctx.c`:



### Data Structures

- struct [client\\_id\\_region\\_t](#)

## Macros

- `#define DEFAULT_NS_CLIENT_ID ((int32_t)-1)`

## Functions

- `void tz_ns_agent_register_client_id_range (int32_t client_id_base, int32_t client_id_limit)`  
*Register a non-secure client ID range.*
- `int32_t tfm_nspm_get_current_client_id (void)`  
*Get the client ID of the current NS client.*
- `void tfm_nspm_ctx_init (void)`  
*initialise the NS context database*

### 8.462.1 Macro Definition Documentation

#### 8.462.1.1 DEFAULT\_NS\_CLIENT\_ID

```
#define DEFAULT_NS_CLIENT_ID ((int32_t)-1)
```

Definition at line 17 of file `tfm_spm_ns_ctx.c`.

### 8.462.2 Function Documentation

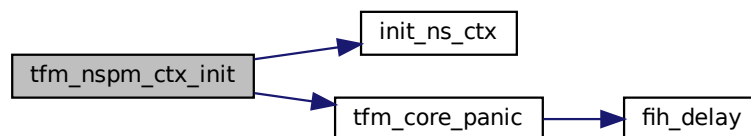
#### 8.462.2.1 tfm\_nspm\_ctx\_init()

```
void tfm_nspm_ctx_init (
 void)
```

initialise the NS context database

Definition at line 90 of file `tfm_spm_ns_ctx.c`.

Here is the call graph for this function:



#### 8.462.2.2 tfm\_nspm\_get\_current\_client\_id()

```
int32_t tfm_nspm_get_current_client_id (
 void)
```

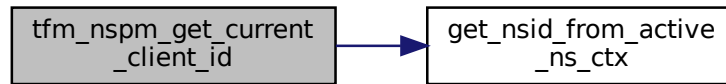
Get the client ID of the current NS client.

**Returns**

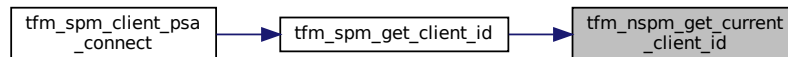
The client id of the current NS client. 0 (invalid client id) is returned in case of error.

Definition at line 75 of file `tfm_spm_ns_ctx.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

**8.462.2.3 tz\_ns\_agent\_register\_client\_id\_range()**

```

void tz_ns_agent_register_client_id_range (
 int32_t client_id_base,
 int32_t client_id_limit)

```

Register a non-secure client ID range.

**Parameters**

|    |                        |                                        |
|----|------------------------|----------------------------------------|
| in | <i>client_id_base</i>  | The minimum client ID for this client. |
| in | <i>client_id_limit</i> | The maximum client ID for this client. |

Definition at line 26 of file `tfm_spm_ns_ctx.c`.

Here is the caller graph for this function:

